IBM SPSS Statistics 27 Command Syntax Reference



Note

Before using this information and the product it supports, read the information in “Notices” on page 2223.

Product Information

This edition applies to version 27, release 0, modification 0 of IBM® SPSS® Statistics and to all subsequent releases and modifications until otherwise indicated in new editions.

© Copyright International Business Machines Corporation .
US Government Users Restricted Rights – Use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM Corp.

Contents

Introduction: A Guide to Command Syntax 1
- Add-On Modules 6
- Release History 10
- Extension Commands 31

Universals 33
- Commands 33
  - Running Commands 34
  - Subcommands 35
  - Keywords 35
  - Values in Command Specifications 35
  - String Values in Command Specifications 35
  - Delimiters 36
  - Command Order 36
- Files 39
  - Command File 39
  - Journal File 40
  - Data Files 40
- Variables 42
  - Variable Names 42
  - Keyword TO 43
  - Keyword ALL 44
  - Scratch Variables 44
  - System Variables 45
- Variable Types and Formats 46
  - Input and Output Formats 46
  - String Variable Formats 46
  - Numeric Variable Formats 48
  - Date and Time Formats 53
  - FORTRAN-like Input Format Specifications 58
- Transformation Expressions 58
  - Numeric expressions 58
  - Numeric functions 61
  - Arithmetic functions 61
  - Statistical functions 63
  - Random variable and distribution functions 64
  - Date and time functions 72
  - String expressions 78
  - String functions 78
  - String/numeric conversion functions 81
  - LAG function 82
  - VALUELABEL function 82
  - Logical expressions 82
  - Logical functions 85
  - Scoring expressions 86
  - Missing values 89

2SLS 93
- Overview 93
- Examples 94

EQUATION Subcommand 94
INSTRUMENTS Subcommand 94
ENDOGENOUS Subcommand 95
CONSTANT and NOCONSTANT Subcommands 95
SAVE Subcommand 95
PRINT Subcommand 95
APPLY Subcommand 95

ACF 97
Overview 97
Example 98
VARIABLES Subcommand 98
DIFF Subcommand 98
SDIFF Subcommand 99
PERIOD Subcommand 99
LN and NOLOG Subcommands 99
SEASONAL Subcommand 100
MXAUTO Subcommand 100
SERROR Subcommand 100
PACF Subcommand 101
APPLY Subcommand 101
References 102

ADD DOCUMENT 103
Overview 103

ADD FILES 105
Overview 105
Examples 106
FILE Subcommand 107
RENAME Subcommand 107
BY Subcommand 108
DROP and KEEP Subcommands 108
IN Subcommand 109
FIRST and LAST Subcommands 109
MAP Subcommand 110
Adding Cases from Different Data Sources 110

ADD VALUE LABELS 111
Overview 111
Examples 112
Value Labels for String Variables 112

ADP 113
Overview 114
Examples 115
FIELDS Subcommand 115
PREPDATETIME Subcommand 115
SCREENING Subcommand 117
ADJUSTLEVEL Subcommand 117
OUTLIERHANDLING Subcommand 118
REPLACEMISSING Subcommand 118
REORDERNOMINAL Subcommand 118
RESCALE Subcommand 118
TRANSFORM Subcommand 119
CRITERIA Subcommand 120
OUTFILE Subcommand 120

AGGREGATE **121**
Overview 121
Example 122
OUTFILE Subcommand 122
Creating a New Aggregated Data File versus Appending Aggregated Variables 123
BREAK Subcommand 124
DOCUMENT Subcommand 125
PRESORTED Subcommand 125
Aggregate Functions 125
MISSING Subcommand 127
Including Missing Values 127
Comparing Missing-Value Treatments 128

AIM **131**
Overview 131
Grouping Variable 131
CATEGORICAL Subcommand 132
CONTINUOUS Subcommand 132
CRITERIA Subcommand 132
MISSING Subcommand 132
PLOT Subcommand 133

ALSCAL **135**
Overview 135
Example 137
VARIABLES Subcommand 137
INPUT Subcommand 137
SHAPE Subcommand 138
LEVEL Subcommand 138
CONDITION Subcommand 139
FILE Subcommand 139
MODEL Subcommand 140
CRITERIA Subcommand 141
PRINT Subcommand 142
PLOT Subcommand 142
OUTFILE Subcommand 143
MATRIX Subcommand 144
Specification of Analyses 145
References 147

ALTER TYPE **149**
Overview 149
PRINT Subcommand 150

ANACOR **151**
Overview 151
Example 152
TABLE Subcommand 152
Casewise Data 152
Table Data 152
DIMENSION Subcommand 153
NORMALIZATION Subcommand 153
VARIANCES Subcommand 154
PRINT Subcommand 154
PLOT Subcommand 154
MATRIX Subcommand 155

Analyzing Aggregated Data 156

ANOVA 159
Overview 159
Examples 160
VARIABLES Subcommand 160
COVARIATES Subcommand 161
MAXORDERS Subcommand 161
METHOD Subcommand 161
Regression Approach 161
Classic Experimental Approach 162
Hierarchical Approach 162
Example 162
Summary of Analysis Methods 163
STATISTICS Subcommand 164
Cell Means 165
Regression Coefficients for the Covariates 165
Multiple Classification Analysis 165
MISSING Subcommand 165
References 165

APPLY DICTIONARY 167
Overview 167
FROM Subcommand 168
NEWVARS Subcommand 169
SOURCE and TARGET Subcommands 169
FILEINFO Subcommand 170
VARINFO Subcommand 171

AREG 173
Overview 173
VARIABLES Subcommand 174
METHOD Subcommand 175
CONSTANT and NOCONSTANT Subcommands 175
RHO Subcommand 175
MXITER Subcommand 175
APPLY Subcommand 176
References 177

ARIMA 179
Overview 179
VARIABLES Subcommand 181
MODEL Subcommand 181
Parameter-Order Subcommands 182
Initial Value Subcommands 183
Termination Criteria Subcommands 184
CINPCT Subcommand 184
APPLY Subcommand 184
FORECAST Subcommand 185
References 185

AUTORECODE 187
Overview 187
Example 188
VARIABLES Subcommand 188
INTO Subcommand 188
BLANK Subcommand 189

GROUP Subcommand 189
SAVE TEMPLATE Subcommand 190
Template File Format 191
APPLY TEMPLATE Subcommand 191
Interaction between APPLY TEMPLATE and SAVE TEMPLATE 192
PRINT Subcommand 192
DESCENDING Subcommand 192

BAYES ANOVA 193
Overview 193
CRITERIA Subcommand 194
DESIGN Subcommand 194
INFERENCE Subcommand 194
ESTBF Subcommand 194
PRIOR Subcommand 195
PLOT Subcommand 195

BAYES CORRELATION 197
Overview 197
Example 198
MISSING Subcommand 198
CRITERIA Subcommand 199
INFERENCE Subcommand 199
ESTBF Subcommand 199

BAYES INDEPENDENT 201
Overview 201
Example 202
MISSING Subcommand 202
CRITERIA Subcommand 202
INFERENCE Subcommand 203
ESTBF Subcommand 203
PRIOR Subcommand 203

BAYES LOGLINEAR 205
Overview 205
Example 206
CRITERIA Subcommand 206
INFERENCE Subcommand 207
TABLE Subcommand 207
ESTBF Subcommand 207
PRINT Subcommand 208

BAYES ONESAMPLE 209
BAYES ONESAMPLE: Binomial 209
Overview 209
Example 210
MISSING SCOPE Subcommand 210
CRITERIA Subcommand 211
INFERENCE Subcommand 211
DATA Subcommand 211
PRIOR Subcommand 212
BAYES ONESAMPLE: Normal 212
Overview 213
Example 214
MISSING SCOPE Subcommand 214
CRITERIA Subcommand 214

INFERENCE Subcommand 215
DATA Subcommand 215
PRIOR Subcommand 215
BAYES ONESAMPLE: Poisson 216
Overview 216
Example 217
MISSING SCOPE Subcommand 217
CRITERIA Subcommand 218
INFERENCE Subcommand 218
DATA Subcommand 218
PRIOR Subcommand 218

BAYES REGRESSION 221
Overview 221
Example 223
CRITERIA Subcommand 223
DESIGN Subcommand 223
INFERENCE Subcommand 223
ESTBF Subcommand 223
PRIOR Subcommand 224
PREDICT Subcommand 224
SAVE Subcommand 224
OUTFILE Subcommand 225
PLOT Subcommand 225
FTEST Subcommand 225

BAYES RELATED 227
Overview 227
Example 228
MISSING SCOPE Subcommand 228
CRITERIA Subcommand 229
INFERENCE Subcommand 229
PAIR Subcommand 229
PRIOR Subcommand 230

BAYES REPEATED 233
Overview 233
CRITERIA Subcommand 234
DESIGN Subcommand 234
INFERENCE Subcommand 234
ESTBF Subcommand 234
PLOT Subcommand 234

BEGIN DATA-END DATA 235
Overview 235
Examples 235

BEGIN EXPR-END EXPR 237
Overview 237
OUTFILE subcommand 238
Specifying expressions 238

BEGIN GPL-END GPL 243
Overview 243

BEGIN PROGRAM-END PROGRAM 245

Overview 245

BOOTSTRAP 247
Overview 247
Examples 248
SAMPLING Subcommand 249
VARIABLES Subcommand 249
CRITERIA Subcommand 249
MISSING Subcommand 250

BREAK 251
Overview 251
Examples 251

CACHE 253

CASEPLOT 255
Overview 255
Examples 257
VARIABLES Subcommand 257
DIFF Subcommand 257
SDIFF Subcommand 257
PERIOD Subcommand 257
LN and NOLOG Subcommands 258
ID Subcommand 258
FORMAT Subcommand 258
MARK Subcommand 260
SPLIT Subcommand 261
APPLY Subcommand 261

CASESTOVARS 263
Overview 263
Examples 265
ID subcommand 265
INDEX subcommand 266
VIND subcommand 266
COUNT subcommand 267
FIXED subcommand 267
AUTOFIX subcommand 267
RENAME subcommand 268
SEPARATOR subcommand 268
GROUPBY subcommand 268
DROP subcommand 269

CATPCA 271
Overview 272
Example 274
VARIABLES Subcommand 274
ANALYSIS Subcommand 275
Level Keyword 275
SPORD and SPNOM Keywords 276
DISCRETIZATION Subcommand 276
GROUPING Keyword 276
NCAT Keyword 276
MISSING Subcommand 276
PASSIVE Keyword 277

ACTIVE Keyword 277
SUPPLEMENTARY Subcommand 277
CONFIGURATION Subcommand 278
DIMENSION Subcommand 278
NORMALIZATION Subcommand 278
MAXITER Subcommand 279
CRITITER Subcommand 279
ROTATION Subcommand 279
RESAMPLE Subcommand 280
PRINT Subcommand 281
PLOT Subcommand 282
BIPLOT Keyword 284
SAVE Subcommand 284
OUTFILE Subcommand 286

CATREG 287
Overview 288
Examples 289
VARIABLES Subcommand 291
ANALYSIS Subcommand 291
LEVEL Keyword 292
SPORD and SPNOM Keywords 292
DISCRETIZATION Subcommand 292
GROUPING Keyword 293
DISTR Keyword 293
MISSING Subcommand 293
SUPPLEMENTARY Subcommand 293
INITIAL Subcommand 294
MAXITER Subcommand 295
CRITITER Subcommand 295
REGULARIZATION Subcommand 295
RESAMPLE Subcommand 296
PRINT Subcommand 296
PLOT Subcommand 297
SAVE Subcommand 298
OUTFILE Subcommand 298

CCF 299
Overview 299
Example 300
VARIABLES Subcommand 300
DIFF Subcommand 300
SDIFF Subcommand 301
PERIOD Subcommand 301
LN and NOLOG Subcommands 301
SEASONAL Subcommand 302
MXCROSS Subcommand 302
APPLY Subcommand 302
References 303

CD 305
Overview 305
Examples 306
Preserving and Restoring the Working Directory Setting 306

CLEAR TIME PROGRAM 307
Overview 307

Example 307

CLEAR TRANSFORMATIONS 309
Overview 309
Examples 309

CLUSTER 311
Overview 311
Example 312
Variable List 313
MEASURE Subcommand 313
Measures for Interval Data 313
Measures for Frequency Count Data 314
Measures for Binary Data 314
METHOD Subcommand 317
SAVE Subcommand 317
ID Subcommand 318
PRINT Subcommand 318
PLOT Subcommand 319
MISSING Subcommand 319
MATRIX Subcommand 319
Matrix Output 320
Matrix Input 320
Format of the Matrix Data File 321
Split Files 321
Missing Values 321
Example: Output to External File 321
Example: Output Replacing Active Dataset 321
Example: Input from Active Dataset 322
Example: Input from External File 322
Example: Input from Active Dataset 322

CODEBOOK 323
Overview 323
Examples 324
Variable List 325
VARINFO Subcommand 325
FILEINFO Subcommand 326
STATISTICS Subcommand 327
OPTIONS Subcommand 327

COMMENT 329
Overview 329
Examples 329

COMPARE DATASETS 331
Overview 331
COMPDATASET subcommand 332
VARIABLES subcommand 332
CASEID subcommand 332
SAVE subcommand 332
OUTPUT subcommand 334

COMPUTE 335
Overview 335
Syntax rules 335

Numeric variables 335
String variables 336
Operations 336
Numeric variables 336
String variables 336
Examples 337
Arithmetic operations 337
Arithmetic functions 337
Statistical functions 337
Missing-Value functions 338
String functions 338
Scoring functions 339

CONJOINT 341
Overview 341
Examples 342
PLAN Subcommand 343
DATA Subcommand 344
SEQUENCE, RANK, or SCORE Subcommand 344
SUBJECT Subcommand 345
FACTORS Subcommand 345
PRINT Subcommand 347
UTILITY Subcommand 347
PLOT Subcommand 348

CORRELATIONS 349
Overview 349
Example 350
VARIABLES Subcommand 350
PRINT Subcommand 351
STATISTICS Subcommand 351
MISSING Subcommand 351
MATRIX Subcommand 352
Format of the Matrix Data File 352
Split Files 352
Missing Values 352
Example 353
Example 353
Example 353
CI Subcommand 353

CORRESPONDENCE 355
Overview 355
Example 356
TABLE Subcommand 356
Casewise Data 357
Aggregated Data 357
Table Data 358
DIMENSION Subcommand 358
SUPPLEMENTARY Subcommand 359
EQUAL Subcommand 359
MEASURE Subcommand 359
STANDARDIZE Subcommand 360
NORMALIZATION Subcommand 360
PRINT Subcommand 361
PLOT Subcommand 361
OUTFILE Subcommand 362

COUNT 365
Overview 365
Examples 366

COXREG 367
Overview 368
VARIABLES Subcommand 369
STATUS Subcommand 369
STRATA Subcommand 370
CATEGORICAL Subcommand 370
CONTRAST Subcommand 370
METHOD Subcommand 371
MISSING Subcommand 373
PRINT Subcommand 373
CRITERIA Subcommand 373
PLOT Subcommand 374
PATTERN Subcommand 374
OUTFILE Subcommand 375
SAVE Subcommand 375
EXTERNAL Subcommand 376

CREATE 377
Overview 377
Examples 378
CSUM Function 379
DIFF Function 379
FFT Function 379
IFFT Function 380
LAG Function 380
LEAD Function 381
MA Function 381
PMA Function 382
RMED Function 382
SDIFF Function 383
T4253H Function 383
References 384

CROSSTABS 385
Overview 386
Examples 387
VARIABLES subcommand 387
TABLES subcommand 388
General mode 388
Integer mode 388
CELLS subcommand 389
STATISTICS subcommand 390
METHOD subcommand 391
MISSING subcommand 392
FORMAT subcommand 393
COUNT subcommand 393
BARCHART subcommand 393
WRITE subcommand 393
Reading a CROSSTABS Procedure Output file 394
HIDESMALLCOUNTS Subcommand 395
SHOWDIM Subcommand 395
References 396

CSCOXREG..... 397
Overview 398
Examples 399
Variable List Subcommand 399
VARIABLES Subcommand 400
PLAN Subcommand 401
JOINTPROB Subcommand 401
MODEL Subcommand 401
CUSTOM Subcommand 402
CRITERIA Subcommand 403
STATISTICS Subcommand 404
TEST Subcommand 404
TESTASSUMPTIONS Subcommand 405
DOMAIN Subcommand 405
MISSING Subcommand 405
SURVIVALMETHOD Subcommand 406
PRINT Subcommand 406
SAVE Subcommand 407
PLOT Subcommand 408
PATTERN Subcommand 408
OUTFILE Subcommand 409

CSDESCRIPTIVES..... 411
Overview 411
PLAN Subcommand 412
JOINTPROB Subcommand 412
SUMMARY Subcommand 412
MEAN Subcommand 412
SUM Subcommand 413
RATIO Subcommand 413
STATISTICS Subcommand 414
SUBPOP Subcommand 414
MISSING Subcommand 414

CSGLM..... 417
Overview 418
CSGLM Variable List 419
PLAN Subcommand 419
JOINTPROB Subcommand 419
MODEL Subcommand 419
INTERCEPT Subcommand 420
INCLUDE Keyword 420
SHOW Keyword 420
Example 420
CUSTOM Subcommand 420
EMMEANS Subcommand 422
CONTRAST Keyword 423
CRITERIA Subcommand 424
STATISTICS Subcommand 424
TEST Subcommand 424
TYPE Keyword 424
PADJUST keyword 424
DOMAIN Subcommand 425
MISSING Subcommand 425
PRINT Subcommand 425
SAVE Subcommand 426

OUTFILE Subcommand 426

CSLOGISTIC 427
Overview 428
CSLOGISTIC Variable List 429
PLAN Subcommand 429
JOINTPROB Subcommand 429
MODEL Subcommand 430
INTERCEPT Subcommand 430
INCLUDE Keyword 430
SHOW Keyword 431
Example 431
CUSTOM Subcommand 431
Example 432
Example 432
Example 433
ODDSRATIOS Subcommand 433
Example 434
Example 434
CRITERIA Subcommand 435
STATISTICS Subcommand 435
TEST Subcommand 436
TYPE Keyword 436
PADJUST Keyword 436
DOMAIN Subcommand 436
MISSING Subcommand 436
PRINT Subcommand 437
SAVE Subcommand 437
OUTFILE Subcommand 438

CSORDINAL 439
Overview 440
Variable List 441
PLAN Subcommand 441
JOINTPROB Subcommand 441
MODEL Subcommand 442
LINK Subcommand 442
CUSTOM Subcommand 443
ODDSRATIOS Subcommand 444
CRITERIA Subcommand 446
STATISTICS Subcommand 447
NONPARALLEL Subcommand 447
TEST Subcommand 447
DOMAIN Subcommand 448
MISSING Subcommand 448
PRINT Subcommand 449
SAVE Subcommand 449
OUTFILE Subcommand 450

CSPLAN 451
Overview 452
Basic Specification 453
Syntax Rules 454
Examples 454
CSPLAN Command 457
PLAN Subcommand 457
PLANVARS Subcommand 457

SRSESTIMATOR Subcommand 458
PRINT Subcommand 458
DESIGN Subcommand 458
STAGELABEL Keyword 458
STRATA Keyword 458
CLUSTER Keyword 459
METHOD Subcommand 459
ESTIMATION Keyword 460
SIZE Subcommand 460
RATE Subcommand 461
MINSIZE Keyword 462
MAXSIZE Keyword 462
MOS Subcommand 462
MIN Keyword 462
MAX Keyword 462
STAGEVARS Subcommand 462
STAGEVARS Variables 463
ESTIMATOR Subcommand 463
POPSIZE Subcommand 464
INCLPROB Subcommand 464

CSSELECT 467
Overview 467
Example 469
PLAN Subcommand 469
CRITERIA Subcommand 469
STAGES Keyword 469
SEED Keyword 469
CLASSMISSING Subcommand 470
DATA Subcommand 470
RENAMEVARS Keyword 470
PRESORTED Keyword 470
SAMPLEFILE Subcommand 470
OUTFILE Keyword 471
KEEP Keyword 471
DROP Keyword 471
JOINTPROB Subcommand 471
Structure of the Joint Probabilities File 471
SELECTRULE Subcommand 472
PRINT Subcommand 473

CSTABULATE 475
Overview 475
PLAN Subcommand 476
JOINTPROB Subcommand 476
TABLES Subcommand 476
CELLS Subcommand 477
STATISTICS Subcommand 477
TEST Subcommand 477
SUBPOP Subcommand 478
MISSING Subcommand 478

CTABLES 479
Overview 481
Syntax Conventions 482
Examples 483
TABLE Subcommand 484

Variable Types 484
Category Variables and Multiple Response Sets 484
Stacking and Nesting 485
Scale Variables 486
Specifying Summaries 486
Formats for Summaries 493
Missing Values in Summaries 493
SLABELS Subcommand 494
CLABELS Subcommand 494
CATEGORIES Subcommand 495
Explicit Category Specification 496
Implicit Category Specification 497
Totals 498
Empty Categories 499
CRITERIA Subcommand 499
TITLES Subcommand: Titles, Captions, and Corner Text 500
Significance Testing 501
Chi-Square Tests: SIGTEST Subcommand 501
Pairwise Comparisons of Proportions and Means: COMPARETEST Subcommand 502
FORMAT Subcommand 504
VLABELS Subcommand 504
SMISSING Subcommand 505
MRSETS Subcommand 505
WEIGHT Subcommand 505
PCOMPUTE Subcommand 506
PPROPERTIES Subcommand 507
HIDESMALLCOUNTS Subcommand 508

CURVEFIT 509
Overview 509
VARIABLES Subcommand 511
MODEL Subcommand 511
UPPERBOUND Subcommand 511
CONSTANT and NOCONSTANT Subcommands 512
CIN Subcommand 512
PLOT Subcommand 512
ID Subcommand 512
SAVE Subcommand 512
PRINT Subcommand 513
APPLY Subcommand 513
TEMPLATE Subcommand 514
References 514

DATA LIST 515
Overview 516
Examples 517
Operations 518
Fixed-Format Data 518
Freefield Data 518
FILE Subcommand 519
ENCODING Subcommand 519
FIXED, FREE, and LIST Keywords 519
TABLE and NOTABLE Subcommands 521
RECORDS Subcommand 521
SKIP Subcommand 522
END Subcommand 522
Variable Definition 523

Variable Names 524
Variable Location 524
Fixed-Format Data 524
Freefield Data 525
Variable Formats 525
Column-Style Format Specifications 526
FORTRAN-like Format Specifications 526
Numeric Formats 527
Implied Decimal Positions 527
String Formats 528

DATAFILE ATTRIBUTE 531
Overview 531

DATASET ACTIVATE 533
Overview 533

DATASET CLOSE 535
Overview 535

DATASET COPY 537
Overview 537

DATASET DECLARE 539
Overview 539

DATASET DISPLAY 541
Overview 541

DATASET NAME 543
Overview 543

DATE 547
Overview 547
Syntax Rules 548
Starting Value and Periodicity 549
BY Keyword 549
Example 1 549
Example 2 550
Example 3 550
Example 4 551
Example 5 551
Example 6 552
Example 7 552

DEFINE-!ENDDEFINE 555
Overview 556
Examples 558
Macro Arguments 559
Keyword Arguments 560
Positional Arguments 560
Assigning Tokens to Arguments 561
Defining Defaults 563
Controlling Expansion 563
Macro Directives 564
Macro Expansion in Comments 564

String Manipulation Functions 564
SET Subcommands for Use with Macro 565
Restoring SET Specifications 566
Conditional Processing 566
Unquoted String Constants in Conditional !IF Statements 567
Looping Constructs 567
Index Loop 567
List-Processing Loop 568
Direct Assignment of Macro Variables 569

DELETE VARIABLES 571
Overview 571

DESCRIPTIVES 573
Overview 573
VARIABLES Subcommand 574
Z Scores 574
SAVE Subcommand 574
STATISTICS Subcommand 575
SORT Subcommand 575
MISSING Subcommand 576

DETECTANOMALY 577
Overview 577
Examples 579
VARIABLES Subcommand 579
HANDLEMISSING Subcommand 580
CRITERIA Subcommand 580
SAVE Subcommand 581
OUTFILE Subcommand 582
PRINT Subcommand 582

DISCRIMINANT 583
Overview 583
GROUPS Subcommand 585
VARIABLES Subcommand 585
SELECT Subcommand 585
ANALYSIS Subcommand 586
Inclusion Levels 586
METHOD Subcommand 587
OUTFILE Subcommand 588
TOLERANCE Subcommand 588
PIN and POUT Subcommands 588
FIN and FOUT Subcommands 588
VIN Subcommand 589
MAXSTEPS Subcommand 589
FUNCTIONS Subcommand 589
PRIORS Subcommand 590
SAVE Subcommand 590
STATISTICS Subcommand 592
ROTATE Subcommand 593
HISTORY Subcommand 593
CLASSIFY Subcommand 593
PLOT Subcommand 594
MISSING Subcommand 594
MATRIX Subcommand 594
Matrix Output 595

Matrix Input 595
Format of the Matrix Data File 595
Split Files 595
STDDEV and CORR Records 596
Missing Values 596
Examples 596

DISPLAY 599
Overview 599
Examples 600
SORTED Keyword 600
VARIABLES Subcommand 600

DMCLUSTER 601

DMLOGISTIC 603

DMROC 605

DMTABLES 607

DMTREE 609

DO IF 611
Overview 611
Examples 612
Syntax Rules 612
Logical Expressions 613
Operations 614
Flow of Control 614
Missing Values and Logical Operators 615
ELSE Command 615
ELSE IF Command 616
Nested DO IF Structures 617
Complex File Structures 617

DO REPEAT-END REPEAT 619
Overview 619
Examples 620
PRINT Subcommand 621

DOCUMENT 623
Overview 623
Examples 623

DROP DOCUMENTS 625
Overview 625
Examples 625

ECHO 627
Overview 627

END CASE 629
Overview 629
Examples 630

END FILE **635**
Overview 635
Examples 635

ERASE **637**
Overview 637
Examples 637

EXAMINE **639**
Overview 639
Examples 640
VARIABLES Subcommand 641
COMPARE Subcommand 641
TOTAL and NOTOTAL Subcommands 641
ID Subcommand 642
PERCENTILES Subcommand 642
PLOT Subcommand 643
STATISTICS Subcommand 644
CINTERVAL Subcommand 644
MESTIMATORS Subcommand 644
MISSING Subcommand 645
References 646

EXECUTE **647**
Overview 647
Examples 647

EXPORT **649**
Overview 649
Examples 650
Methods of Transporting Portable Files 650
Magnetic Tape 650
Communications Programs 651
Character Translation 651
OUTFILE Subcommand 651
TYPE Subcommand 651
UNSELECTED Subcommand 652
DROP and KEEP Subcommands 652
RENAME Subcommand 652
MAP Subcommand 653
DIGITS Subcommand 653

EXSMOOTH **655**
Overview 655
VARIABLES Subcommand 656
MODEL Subcommand 657
PERIOD Subcommand 658
SEASFACT Subcommand 658
Smoothing Parameter Subcommands 658
Keyword GRID 659
INITIAL Subcommand 660
APPLY Subcommand 661
References 661

EXTENSION **663**

Overview 663
Examples 663
SPECIFICATION Subcommand 664

FACTOR 665
Overview 665
VARIABLES Subcommand 667
MISSING Subcommand 667
METHOD Subcommand 668
SELECT Subcommand 668
ANALYSIS Subcommand 668
FORMAT Subcommand 669
PRINT Subcommand 669
PLOT Subcommand 671
DIAGONAL Subcommand 671
CRITERIA Subcommand 671
EXTRACTION Subcommand 672
ROTATION Subcommand 673
SAVE Subcommand 674
MATRIX Subcommand 675
Matrix Output 675
Matrix Input 676
Format of the Matrix Data File 676
Split Files 676
Example: Factor Correlation Matrix Output to External File 676
Example: Factor Correlation Matrix Output Replacing Active Dataset 676
Example: Factor-Loading Matrix Output Replacing Active Dataset 677
Example: Matrix Input from active dataset 677
Example: Matrix Input from External File 677
Example: Matrix Input from active dataset 677
Example: Using Saved Coefficients to Score an External File 677
References 678

FILE HANDLE 679
Overview 679
Example 679
NAME Subcommand 680
MODE Subcommand 680
RECFORM Subcommand 680
LRECL Subcommand 680
ENCODING Subcommand 681

FILE LABEL 683
Overview 683

FILE TYPE-END FILE TYPE 685
Overview 685
Examples 687
Specification Order 688
Types of Files 688
Subcommands and Their Defaults for Each File Type 689
FILE Subcommand 689
ENCODING Subcommand 690
RECORD Subcommand 690
CASE Subcommand 690
WILD Subcommand 692
DUPLICATE Subcommand 692

MISSING Subcommand 693
ORDERED Subcommand 694

FILTER 697
Overview 697
Examples 698

FINISH 699
Overview 699

FIT 701
Overview 701
Example 702
ERRORS Subcommand 702
OBS Subcommand 702
DFE and DFH Subcommands 702
Output Considerations for SSE 703
References 703

FLEISS MULTIRATER KAPPA 705
Overview 705
MISSING Subcommand 706
CRITERIA Subcommand 706
PRINT Subcommand 707

FLIP 709
Overview 709
Example 710
VARIABLES Subcommand 710
NEWNAMES Subcommand 710

FORMATS 713
Overview 713
Syntax Rules 714
Examples 714

FREQUENCIES 715
Overview 715
VARIABLES subcommand 716
FORMAT subcommand 716
BARCHART subcommand 716
PIECHART subcommand 717
HISTOGRAM subcommand 718
GROUPED subcommand 718
PERCENTILES subcommand 719
NTILES subcommand 719
STATISTICS subcommand 720
MISSING subcommand 721
ORDER subcommand 721

GENLIN 723
Overview 725
Variable List 726
MODEL Subcommand 728
CRITERIA Subcommand 732
REPEATED Subcommand 737

EMMEANS Subcommand 741
MISSING Subcommand 744
PRINT Subcommand 744
SAVE Subcommand 746
OUTFILE Subcommand 748

GENLINMIXED 749
Overview 750
Examples 752
DATA_STRUCTURE Subcommand 755
FIELDS Subcommand 757
TARGET_OPTIONS Subcommand 757
FIXED Subcommand 760
RANDOM Subcommand 761
BUILD_OPTIONS Subcommand 763
EMMEANS Subcommand 764
EMMEANS_OPTIONS Subcommand 765
OUTFILE Subcommand 766
SAVE Subcommand 766

GENLOG 769
Overview 769
Variable List 770
Logit Model 770
Cell Covariates 771
CSTRUCTURE Subcommand 771
GRESID Subcommand 772
GLOR Subcommand 772
MODEL Subcommand 773
CRITERIA Subcommand 773
PRINT Subcommand 773
PLOT Subcommand 774
MISSING Subcommand 775
SAVE Subcommand 775
DESIGN Subcommand 775
References 777

GET 779
Overview 779
FILE Subcommand 780
DROP and KEEP Subcommands 780
RENAME Subcommand 781
MAP Subcommand 781

GET CAPTURE 783
Overview 783
CONNECT Subcommand 784
UNENCRYPTED Subcommands 784
SQL Subcommand 784
Data Conversion 784
Variable Names and Labels 784
Missing Values 784

GET DATA 785
Overview 786
TYPE Subcommand 786
FILE subcommand 787

Subcommands for TYPE=ODBC and TYPE=OLEDB 787
CONNECT subcommand 787
ENCRYPTED and UNENCRYPTED subcommands 787
SQL subcommand 787
ASSUMEDSTRWIDTH subcommand 788
DATATYPEMIN subcommand 788
LEADINGSPACES subcommand 788
Subcommands for TYPE=XLS, XLSX, and XLSM 789
SHEET subcommand 789
CELLRANGE subcommand 789
READNAMES subcommand 789
HIDDEN subcommand 789
TRAILINGSPACES subcommand 790
Subcommands for TYPE=TXT 790
ENCODING subcommand 790
ARRANGEMENT subcommand 790
FIRSTCASE subcommand 791
DELCASE subcommand 791
FIXCASE subcommand 791
IMPORTCASE subcommand 791
DELIMITERS subcommand 791
MULTIPLESPACES subcommand 791
QUALIFIER subcommand 792
VARIABLES subcommand for ARRANGEMENT = DELIMITED 792
VARIABLES subcommand for ARRANGEMENT = FIXED 792
Variable Format Specifications for TYPE = TXT 793
MAP subcommand 793

GET SAS 795
Overview 795
DATA Subcommand 796
ENCODING Subcommand 796
FORMATS Subcommand 797
Creating a Formats File with PROC FORMAT 798
SAS Data Conversion 798
Variable Names 798
Variable Labels 798
Value Labels 798
Missing Values 798
Variable Types 799

GET STATA 801
Overview 801
FILE Keyword 801
ENCODING Subcommand 802

GET TRANSLATE 805
Overview 805
Operations 806
Spreadsheets 806
Databases 807
Tab-Delimited ASCII Files 807
FILE Subcommand 808
TYPE Subcommand 808
FIELDNAMES Subcommand 809
RANGE Subcommand 809
DROP and KEEP Subcommands 809

MAP Subcommand 810

GETCOGNOS 811
Overview 811
MODE subcommand 812
CONNECTION subcommand 812
LOCATION subcommand 812
IMPORT subcommand 813
FILTER subcommand 813
PARAMETERS subcommand 813

GETTM1 815
Overview 815
CONNECTION subcommand 816
VIEW subcommand 817
RENAME subcommand 817

GGRAPH 819
Overview 820
GRAPHDATASET Subcommand 821
NAME Keyword 821
DATASET Keyword 821
VARIABLES Keyword 821
DATAFILTER Keyword 825
TRANSFORM Keyword 826
MISSING Keyword 826
REPORTMISSING Keyword 826
CASELIMIT Keyword 827
GRAPHSPEC Subcommand 827
SOURCE Keyword 827
EDITABLE Keyword 830
LABEL Keyword 830
DEFAULTTEMPLATE Keyword 830
TEMPLATE Keyword 830
VIZSTYLESHEET Keyword 831
VIZMAP Keyword 831
COLORCYCLE Subcommand 833
FRAME Subcommand 833
GRIDLINES Subcommand 833
FITLINES Subcommand 833
GPL Examples 834

GLM 837
Overview 838
General Linear Model (GLM) and MANOVA 839
Models 840
Custom Hypothesis Specifications 841
LMATRIX, MMATRIX, and KMATRIX Subcommands 841
CONTRAST Subcommand 842

GLM: Univariate 845
Overview 846
Example 847
GLM Variable List 847
RANDOM Subcommand 848
REGWGT Subcommand 848
METHOD Subcommand 849

INTERCEPT Subcommand 849
MISSING Subcommand 850
CRITERIA Subcommand 850
PRINT Subcommand 850
PLOT Subcommand 852
TEST Subcommand 853
LMATRIX Subcommand 853
KMATRIX Subcommand 854
CONTRAST Subcommand 855
POSTHOC Subcommand 856
EMMEANS Subcommand 859
ROBUST Subcommand 860
SAVE Subcommand 860
OUTFILE Subcommand 861
MBPDESIGN, BPDESIGN, and FDESIGN Subcommands 862
DESIGN Subcommand 862

GLM: Multivariate.... 865
Overview 865
GLM Variable List 866
PRINT Subcommand 866
MMATRIX Subcommand 867

GLM: Repeated Measures....869
Overview 869
Example 870
GLM Variable List 871
WSFACTOR Subcommand 871
Contrasts for WSFACTOR 872
WSDESIGN Subcommand 874
MEASURE Subcommand 874
EMMEANS Subcommand 875

GRAPH.... 877
Overview 879
Examples 880
TITLE, SUBTITLE, and FOOTNOTE Subcommands 880
BAR Subcommand 880
LINE Subcommand 881
PIE Subcommand 881
HILO Subcommand 881
ERRORBAR Subcommand 882
SCATTERPLOT Subcommand 882
HISTOGRAM Subcommand 883
PARETO Subcommand 883
PANEL Subcommand 883
COLVAR and ROWVAR Keywords 883
COLOP and ROWOP Keywords 884
INTERVAL Subcommand 885
CI Keyword 885
STDDEV Keyword 885
SE Keyword 885
TEMPLATE Subcommand 885
Elements and Attributes Independent of Chart Types or Data 885
Elements and Attributes Dependent on Chart Type 886
Elements and Attributes Dependent on Data 886
MISSING Subcommand 886

HILOGLINEAR 889
Overview 889
Example 891
Variable List 891
METHOD Subcommand 891
MAXORDER Subcommand 891
CRITERIA Subcommand 892
CWEIGHT Subcommand 892
PRINT Subcommand 894
PLOT Subcommand 894
MISSING Subcommand 895
DESIGN Subcommand 895
References 895

HOMALS 897
Overview 897
Example 898
VARIABLES Subcommand 898
ANALYSIS Subcommand 899
NOBSERVATIONS Subcommand 899
DIMENSION Subcommand 899
MAXITER Subcommand 900
CONVERGENCE Subcommand 900
PRINT Subcommand 900
PLOT Subcommand 900
SAVE Subcommand 902
MATRIX Subcommand 902

HOST 905
Overview 905
Syntax 905
Quoted Strings 906
TIMELIMIT Keyword 906
Using TIMELIMIT to Return Control 906
Working Directory 907
UNC Paths on Windows Operating Systems 907

IF 909
Overview 909
Examples 910
Operations 912
Numeric Variables 912
String Variables 912
Missing Values and Logical Operators 912

IGRAPH 915
Overview 917
General Syntax 918
X1, Y, and X2 Subcommands 918
CATORDER Subcommand 919
X1LENGTH, YLENGTH, and X2LENGTH Subcommands 919
NORMALIZE Subcommand 920
COLOR, STYLE, and SIZE Subcommands 920
STACK Subcommand 921
SUMMARYVAR Subcommand 921
PANEL Subcommand 921

POINTLABEL Subcommand 921
CASELABEL Subcommand 921
COORDINATE Subcommand 921
EFFECT Subcommand 922
TITLE, SUBTITLE, and CAPTION Subcommands 922
VIEWNAME Subcommand 922
CHARTLOOK Subcommand 922
REFLINE Subcommand 923
SPIKE Subcommand 923
FORMAT Subcommand 923
KEY Keyword 924
Element Syntax 924
SCATTER Subcommand 924
AREA Subcommand 924
BAR Subcommand 925
PIE Subcommand 926
BOX Subcommand 927
LINE Subcommand 928
ERRORBAR Subcommand 929
HISTOGRAM Subcommand 930
FITLINE Subcommand 930
Summary Functions 931

IMPORT 933
Overview 933
Examples 933
FILE Subcommand 934
TYPE Subcommand 934
DROP and KEEP Subcommands 934
RENAME Subcommand 935
MAP Subcommand 935

INCLUDE 937
Overview 937
ENCODING Keyword 938
Examples 938
FILE Subcommand 938

INFO 939

INPUT PROGRAM-END INPUT PROGRAM 941
Overview 941
Examples 941
Input Programs 942
Input State 943
More Examples 943

INSERT 945
OVERVIEW 945
FILE Keyword 946
SYNTAX Keyword 946
ERROR Keyword 946
CD Keyword 947
ENCODING Keyword 947
INSERT vs. INCLUDE 947

INSERT EXEC..... 949
SUCCESS subcommand 949
FAILURE subcommand 949
CLEAR subcommand 949
FILE Keyword 949
SYNTAX Keyword 950
ENCODING Keyword 950

INSERT HIDDEN..... 951
SOURCE keyword 951
SYNTAX Keyword 951
ERROR Keyword 951
ENCODING Keyword 952

KEYED DATA LIST 953
Overview 953
Examples 954
FILE Subcommand 956
KEY Subcommand 956
IN Subcommand 956
TABLE and NOTABLE Subcommands 957
ENCODING Subcommand 957

KM..... 959
Overview 959
Examples 960
Survival and Factor Variables 961
STATUS Subcommand 961
STRATA Subcommand 962
PLOT Subcommand 962
ID Subcommand 963
PRINT Subcommand 963
PERCENTILES Subcommand 963
TEST Subcommand 963
COMPARE Subcommand 964
TREND Subcommand 964
SAVE Subcommand 965

KNN..... 967
Overview 967
Examples 969
Variable Lists 969
EXCEPT Subcommand 970
CASELABELS Subcommand 970
FOCALCASES Subcommand 971
RESCALE Subcommand 971
PARTITION Subcommand 971
MODEL Subcommand 972
CRITERIA Subcommand 973
CROSSVALIDATION Subcommand 974
MISSING Subcommand 974
VIEWMODEL Subcommand 975
PRINT Subcommand 975
SAVE Subcommand 975
OUTFILE Subcommand 976

LEAVE 977
Overview 977
Examples 977

LINEAR 979
Overview 979
FIELDS Subcommand 980
BUILD_OPTIONS Subcommand 980
ENSEMBLES Subcommand 982
SAVE Subcommand 983
OUTFILE Subcommand 983

LIST 985
Overview 985
Examples 986
VARIABLES Subcommand 986
FORMAT Subcommand 986
CASES Subcommand 987

LOGISTIC REGRESSION 989
Overview 990
VARIABLES Subcommand 991
CATEGORICAL Subcommand 992
CONTRAST Subcommand 992
METHOD Subcommand 993
SELECT Subcommand 995
ORIGIN and NOORIGIN Subcommands 995
ID Subcommand 995
PRINT Subcommand 996
CRITERIA Subcommand 996
CLASSPLOT Subcommand 997
CASEWISE Subcommand 997
MISSING Subcommand 998
OUTFILE Subcommand 998
SAVE Subcommand 999
EXTERNAL Subcommand 999
References 999

LOGLINEAR 1001
Overview 1001
Variable List 1003
Logit Model 1003
Cell Covariates 1004
CWEIGHT Subcommand 1004
GRESID Subcommand 1005
CONTRAST Subcommand 1005
CRITERIA Subcommand 1007
PRINT Subcommand 1007
PLOT Subcommand 1008
MISSING Subcommand 1008
DESIGN Subcommand 1009

LOOP-END LOOP 1011
Overview 1011
Examples 1012

IF Keyword 1013
Indexing Clause 1013
BY Keyword 1016
Missing Values 1017
Creating Data 1017

MANOVA 1019
Overview 1020
MANOVA and General Linear Model (GLM) 1021

MANOVA: Univariate 1023
Overview 1023
Example 1025
MANOVA Variable List 1025
ERROR Subcommand 1026
CONTRAST Subcommand 1026
PARTITION Subcommand 1028
METHOD Subcommand 1028
PRINT and NOPRINT Subcommands 1029
CELLINFO Keyword 1030
PARAMETERS Keyword 1030
SIGNIF Keyword 1031
HOMOGENEITY Keyword 1031
DESIGN Keyword 1031
ERROR Keyword 1032
OMEANS Subcommand 1032
PMEANS Subcommand 1032
RESIDUALS Subcommand 1033
POWER Subcommand 1033
CINTERVAL Subcommand 1034
PLOT Subcommand 1035
MISSING Subcommand 1035
MATRIX Subcommand 1036
Format of the Matrix Data File 1036
Split Files and Variable Order 1036
Additional Statistics 1037
ANALYSIS Subcommand 1037
DESIGN Subcommand 1038
Partitioned Effects: Number in Parentheses 1039
Nested Effects: WITHIN Keyword 1039
Simple Effects: WITHIN and MWITHIN Keywords 1039
Pooled Effects: Plus Sign 1040
MUPLUS Keyword 1040
Effects of Continuous Variables 1040
Error Terms for Individual Effects 1041
CONSTANT Keyword 1042
References 1042

MANOVA: Multivariate 1043
Overview 1043
MANOVA Variable List 1044
TRANSFORM Subcommand 1044
Variable Lists 1045
CONTRAST, BASIS, and ORTHONORM Keywords 1045
Transformation Methods 1045
RENAME Subcommand 1047
PRINT and NOPRINT Subcommands 1047

ERROR Keyword 1048
SIGNIF Keyword 1048
TRANSFORM Keyword 1048
HOMOGENEITY Keyword 1049
PLOT Subcommand 1049
PCOMPS Subcommand 1049
DISCRIM Subcommand 1049
POWER Subcommand 1050
CINTERVAL Subcommand 1050
ANALYSIS Subcommand 1051
CONDITIONAL and UNCONDITIONAL Keywords 1051

MANOVA: Repeated Measures 1053
Overview 1053
Example 1054
MANOVA Variable List 1054
WSFACTORS Subcommand 1055
CONTRAST for WSFACTORS 1056
PARTITION for WSFACTORS 1057
WSDESIGN Subcommand 1057
MWITHIN Keyword for Simple Effects 1057
MEASURE Subcommand 1058
RENAME Subcommand 1058
PRINT Subcommand 1059
References 1059

MATCH FILES 1061
Overview 1061
FILE Subcommand 1063
Text Data Files 1064
BY Subcommand 1064
Duplicate Cases 1064
TABLE Subcommand 1065
RENAME Subcommand 1065
DROP and KEEP Subcommands 1066
IN Subcommand 1066
FIRST and LAST Subcommands 1067
MAP Subcommand 1068

MATRIX-END MATRIX 1069
Overview 1069
Terminology 1070
Matrix Variables 1070
String Variables in Matrix Programs 1071
Syntax of Matrix Language 1071
Comments in Matrix Programs 1071
Matrix Notation 1072
Matrix Notation Shorthand 1072
Construction of a Matrix from Other Matrices 1072
Matrix Operations 1072
Conformable Matrices 1072
Scalar Expansion 1073
Arithmetic Operators 1073
Relational Operators 1074
Logical Operators 1074
Precedence of Operators 1074
MATRIX and Other Commands 1075

Matrix Statements 1075
Exchanging Data with IBM SPSS Statistics Data Files 1075
MATRIX and END MATRIX Commands 1076
COMPUTE Statement 1076
String Values on COMPUTE Statements 1076
Arithmetic Operations and Comparisons 1077
Matrix Functions 1077
CALL Statement 1087
PRINT Statement 1088
Matrix Expression 1088
FORMAT Keyword 1088
TITLE Keyword 1088
SPACE Keyword 1089
RLABELS Keyword 1089
RNAMES Keyword 1089
CLABELS Keyword 1089
CNAMES Keyword 1089
Scaling Factor in Displays 1090
Matrix Control Structures 1090
DO IF Structures 1090
LOOP Structures 1091
Index Clause on the LOOP Statement 1092
IF Clause on the LOOP Statement 1092
IF Clause on the END LOOP Statement 1092
BREAK Statement 1092
READ Statement: Reading Character Data 1093
Variable Specification 1093
FILE Specification 1093
FIELD Specification 1093
SIZE Specification 1094
MODE Specification 1094
REREAD Specification 1094
FORMAT Specification 1095
WRITE Statement: Writing Character Data 1095
Matrix Expression Specification 1095
OUTFILE Specification 1095
FIELD Specification 1095
MODE Specification 1096
HOLD Specification 1096
FORMAT Specification 1096
GET Statement: Reading IBM SPSS Statistics Data Files 1097
Variable Specification 1097
FILE Specification 1097
VARIABLES Specification 1097
NAMES Specification 1098
MISSING Specification 1098
SYSMIS Specification 1098
SAVE Statement: Writing IBM SPSS Statistics Data Files 1099
Matrix Expression Specification 1099
OUTFILE Specification 1099
VARIABLES Specification 1099
NAMES Specification 1100
STRINGS Specification 1100
MGET Statement: Reading Matrix Data Files 1100
FILE Specification 1100
TYPE Specification 1101
Names of Matrix Variables from MGET 1101
MSAVE Statement: Writing Matrix Data Files 1102

Matrix Expression Specification 1102
TYPE Specification 1103
OUTFILE Specification 1103
VARIABLES Specification 1103
FACTOR Specification 1103
FNAMES Specification 1104
SPLIT Specification 1104
SNAMES Specification 1104
DISPLAY Statement 1105
RELEASE Statement 1105
Macros Using the Matrix Language 1105

MATRIX DATA 1107
Overview 1107
Examples 1109
Operations 1111
Format of the Raw Matrix Data File 1111
VARIABLES Subcommand 1111
Variable VARNAME_ 1112
Variable ROWTYPE_ 1112
FILE Subcommand 1113
FORMAT Subcommand 1113
Data-Entry Format 1113
Matrix Shape 1113
Diagonal Values 1114
SPLIT Subcommand 1114
FACTORS Subcommand 1115
CELLS Subcommand 1116
CONTENTS Subcommand 1117
Within-Cells Record Definition 1118
Optional Specification When ROWTYPE_ Is Explicit 1119
N Subcommand 1119

MCONVERT 1121
Overview 1121
Examples 1122
MATRIX Subcommand 1122
REPLACE and APPEND Subcommands 1122

MEANS 1125
Overview 1125
Examples 1126
TABLES Subcommand 1126
CELLS Subcommand 1126
STATISTICS Subcommand 1127
MISSING Subcommand 1128
References 1128

MISSING VALUES 1129
Overview 1129
Examples 1130
Specifying Ranges of Missing Values 1130

MIXED 1133
Overview 1134
Examples 1135
Case Frequency 1136

Covariance Structure List 1137
Variable List 1138
CRITERIA Subcommand 1138
EMMEANS Subcommand 1140
FIXED Subcommand 1141
METHOD Subcommand 1142
MISSING Subcommand 1142
PRINT Subcommand 1142
RANDOM Subcommand 1143
REGWGT Subcommand 1144
REPEATED Subcommand 1144
SAVE Subcommand 1146
TEST Subcommand 1146

MLP 1149
Overview 1150
Examples 1152
Variable Lists 1152
EXCEPT Subcommand 1153
RESCALE Subcommand 1153
PARTITION Subcommand 1155
ARCHITECTURE Subcommand 1156
CRITERIA Subcommand 1157
STOPPINGRULES Subcommand 1159
MISSING Subcommand 1161
PRINT Subcommand 1161
PLOT Subcommand 1162
SAVE Subcommand 1163
OUTFILE Subcommand 1164

MODEL CLOSE 1165
Overview 1165

MODEL HANDLE 1167
Overview 1167
NAME Subcommand 1169
FILE keyword 1169
OPTIONS subcommand 1170
MISSING keyword 1170
MAP subcommand 1171

MODEL LIST 1173
Overview 1173

MODEL NAME 1175
Overview 1175
Example 1175

MRSETS 1177
Overview 1177
Syntax Conventions 1178
MDGROUP Subcommand 1178
MCGROUP Subcommand 1179
DELETE Subcommand 1179
DISPLAY Subcommand 1179

MULT RESPONSE 1181
Overview 1181
GROUPS Subcommand 1182
VARIABLES Subcommand 1183
FREQUENCIES Subcommand 1184
TABLES Subcommand 1184
PAIRED Keyword 1185
CELLS Subcommand 1186
BASE Subcommand 1186
MISSING Subcommand 1187
FORMAT Subcommand 1187

MULTIPLE CORRESPONDENCE..... 1189
Overview 1189
Example 1190
Options 1191
VARIABLES Subcommand 1192
ANALYSIS Subcommand 1192
DISCRETIZATION Subcommand 1192
GROUPING Keyword 1193
NCAT Keyword 1193
MISSING Subcommand 1193
PASSIVE Keyword 1193
ACTIVE Keyword 1193
SUPPLEMENTARY Subcommand 1194
CONFIGURATION Subcommand 1194
DIMENSION Subcommand 1194
NORMALIZATION Subcommand 1195
MAXITER Subcommand 1195
CRITITER Subcommand 1195
PRINT Subcommand 1195
PLOT Subcommand 1197
SAVE Subcommand 1198
OUTFILE Subcommand 1199

MULTIPLE IMPUTATION..... 1201
Overview 1201
Examples 1203
Variable Lists 1204
IMPUTE Subcommand 1204
CONSTRAINTS Subcommand 1206
MISSINGSUMMARIES Subcommand 1207
IMPUTATIONSUMMARIES Subcommand 1208
ANALYSISWEIGHT Subcommand 1209
OUTFILE Subcommand 1209

MVA..... 1211
Overview 1212
Syntax Rules 1212
Symbols 1213
Missing Indicator Variables 1213
VARIABLES Subcommand 1213
CATEGORICAL Subcommand 1214
MAXCAT Subcommand 1214
ID Subcommand 1214
NOUNIVARIATE Subcommand 1214

TTEST Subcommand 1215
Display of Statistics 1215
CROSSTAB Subcommand 1216
MISMATCH Subcommand 1216
DPATTERN Subcommand 1217
MPATTERN Subcommand 1217
TPATTERN Subcommand 1218
LISTWISE Subcommand 1218
PAIRWISE Subcommand 1218
EM Subcommand 1219
REGRESSION Subcommand 1220

N OF CASES 1223
Overview 1223

NAIVEBAYES 1225
Overview 1225
Examples 1227
Variable Lists 1228
EXCEPT Subcommand 1228
FORCE Subcommand 1229
TRAININGSAMPLE Subcommand 1229
SUBSET Subcommand 1230
CRITERIA Subcommand 1230
MISSING Subcommand 1231
PRINT Subcommand 1231
SAVE Subcommand 1231
OUTFILE Subcommand 1232

NEW FILE 1233
Overview 1233

NLR 1235
Overview 1235
Operations 1237
Weighting Cases 1237
Missing Values 1237
Examples 1237
MODEL PROGRAM Command 1238
Caution: Initial Values 1238
DERIVATIVES Command 1238
CONSTRAINED FUNCTIONS Command 1239
CLEAR MODEL PROGRAMS Command 1240
CNLR and NLR Commands 1240
OUTFILE Subcommand 1240
FILE Subcommand 1240
PRED Subcommand 1241
SAVE Subcommand 1241
CRITERIA Subcommand 1242
Checking Derivatives for CNLR and NLR 1242
Iteration Criteria for CNLR 1243
Iteration Criteria for NLR 1244
BOUNDS Subcommand 1245
Simple Bounds and Linear Constraints 1245
Nonlinear Constraints 1245
LOSS Subcommand 1245
BOOTSTRAP Subcommand 1246

References 1247

NOMREG 1249
Overview 1250
Variable List 1250
CRITERIA Subcommand 1251
FULLFACTORIAL Subcommand 1252
INTERCEPT Subcommand 1252
MISSING Subcommand 1252
MODEL Subcommand 1252
STEPWISE Subcommand 1254
OUTFILE Subcommand 1255
PRINT Subcommand 1256
SAVE Subcommand 1256
SCALE Subcommand 1257
SUBPOP Subcommand 1257
TEST Subcommand 1258

NONPAR CORR 1259
Overview 1259
Examples 1260
VARIABLES Subcommand 1260
PRINT Subcommand 1261
SAMPLE Subcommand 1261
MISSING Subcommand 1262
MATRIX Subcommand 1262
Format of the Matrix Data File 1263
Split Files 1263
Missing Values 1263
Examples 1263
CI Subcommand 1263
References 1264

NPTESTS 1265
Overview 1266
MISSING Subcommand 1267
CRITERIA Subcommand 1267
ONESAMPLE Subcommand 1268
INDEPENDENT Subcommand 1271
RELATED Subcommand 1273

NPAR TESTS 1277
Overview 1278
BINOMIAL Subcommand 1279
CHISQUARE Subcommand 1280
COCHRAN Subcommand 1281
FRIEDMAN Subcommand 1281
J-T Subcommand 1282
K-S Subcommand (One-Sample) 1282
K-S Subcommand (Two-Sample) 1283
KS_SIM Subcommand (One-Sample) 1284
K-W Subcommand 1285
KENDALL Subcommand 1285
M-W Subcommand 1286
MCNEMAR Subcommand 1286
MEDIAN Subcommand 1287
MH Subcommand 1287

MOSES Subcommand 1288
RUNS Subcommand 1289
SIGN Subcommand 1289
W-W Subcommand 1290
WILCOXON Subcommand 1291
STATISTICS Subcommand 1291
MISSING Subcommand 1291
SAMPLE Subcommand 1292
METHOD Subcommand 1292
References 1292

NUMERIC 1293
Overview 1293
Examples 1293

OLAP CUBES 1295
Overview 1295
Options 1296
TITLE and FOOTNOTE Subcommands 1296
CELLS Subcommand 1296
CREATE Subcommand 1297
HIDESMALLCOUNTS Subcommand 1299

OMS 1301
Overview 1302
Basic Operation 1303
SELECT Subcommand 1303
IF Subcommand 1306
COMMANDS Keyword 1306
SUBTYPES Keyword 1306
LABELS Keyword 1307
INSTANCES Keyword 1307
Wildcards 1307
EXCEPTIF Subcommand 1308
DESTINATION Subcommand 1308
FORMAT Keyword 1308
NUMBERED Keyword 1309
IMAGES and IMAGEFORMAT Keywords 1309
CHARTSIZE and IMAGEROOT Keywords 1310
IMAGEMAP Keyword 1310
TREEFORMAT Keyword 1311
CHARTFORMAT Keyword 1311
MODELFORMAT Keyword 1311
TABLES Keyword 1312
OUTFILE Keyword 1312
XMLWORKSPACE Keyword 1312
OUTPUTSET Keyword 1312
FOLDER Keyword 1312
VIEWER Keyword 1313
COLUMNS Subcommand 1313
DIMNAMES Keyword 1313
SEQUENCE Keyword 1314
TAG Subcommand 1315
NOWARN Subcommand 1315
Routing Output to SAV Files 1316
Data File Created from One Table 1316
Data Files Created from Multiple Tables 1317

Data Files Not Created from Multiple Tables 1319
Controlling Column Elements to Control Variables in the Data File 1320
Variable Names 1322
OXML Table Structure 1323
Command and Subtype Identifiers 1325

OMSEND 1327
Overview 1327
TAG Keyword 1327
FILE Keyword 1327
LOG Keyword 1328

OMSINFO 1329
Overview 1329

OMSLOG 1331
Overview 1331
FILE Subcommand 1331
APPEND Subcommand 1332
FORMAT Subcommand 1332

ONEWAY 1333
Overview 1333
Analysis List 1334
POLYNOMIAL Subcommand 1335
CONTRAST Subcommand 1335
POSTHOC Subcommand 1336
RANGES Subcommand 1337
PLOT MEANS Subcommand 1338
STATISTICS Subcommand 1338
MISSING Subcommand 1338
MATRIX Subcommand 1338
Matrix Output 1339
Matrix Input 1339
Format of the Matrix Data File 1339
Split Files 1339
Missing Values 1339
Example 1340
Example 1340
Example 1340
Example 1340
TEMPLATE Subcommand 1340
CRITERIA Subcommand 1340
ES Subcommand 1341
References 1341

OPTIMAL BINNING 1343
Overview 1343
VARIABLES Subcommand 1344
CRITERIA Subcommand 1345
MISSING Subcommand 1346
OUTFILE Subcommand 1346
PRINT Subcommand 1346

ORTHOPLAN 1347
Overview 1347

Examples 1348
FACTORS Subcommand 1349
REPLACE Subcommand 1349
OUTFILE Subcommand 1349
MINIMUM Subcommand 1350
HOLDOUT Subcommand 1350
MIXHOLD Subcommand 1350

OUTPUT ACTIVATE 1351
Overview 1351

OUTPUT CLOSE 1353
Overview 1353

OUTPUT DISPLAY 1355
Overview 1355

OUTPUT EXPORT 1357
Overview 1359
Examples 1359
NAME Keyword 1360
CONTENTS Subcommand 1360
DOC Subcommand 1361
HTML Subcommand 1362
PDF Subcommand 1363
PPT Subcommand 1364
TEXT Subcommand 1365
XLS, XLSX, and XLSM subcommands 1367
BMP Subcommand 1368
EMF Subcommand 1369
EPS Subcommand 1369
JPG Subcommand 1370
PNG Subcommand 1370
TIF Subcommand 1371

OUTPUT MODIFY 1373
Overview 1374
Basic Operation 1375
NAME Keyword 1375
SELECT Subcommand 1376
IF Subcommand 1377
DELETEOBJECT Subcommand 1379
INDEXING Subcommand 1379
OBJECTPROPERTIES Subcommand 1380
TABLE Subcommand 1381
TABLECELLS Subcommand 1384
GRAPHS Subcommand 1388
TEXTS Subcommand 1389
REPORT Subcommand 1390

OUTPUT NAME 1391
Overview 1391

OUTPUT NEW 1393
Overview 1393

OUTPUT OPEN..... 1395
Overview 1395

OUTPUT SAVE..... 1399
Overview 1399
PASSPROTECT Subcommand 1401

OVERALS..... 1403
Overview 1403
Examples 1404
VARIABLES Subcommand 1405
ANALYSIS Subcommand 1405
SETS Subcommand 1405
NOBSERVATIONS Subcommand 1406
DIMENSION Subcommand 1406
INITIAL Subcommand 1406
MAXITER Subcommand 1406
CONVERGENCE Subcommand 1406
PRINT Subcommand 1407
PLOT Subcommand 1407
SAVE Subcommand 1408
MATRIX Subcommand 1409

PACF..... 1411
Overview 1411
Example 1412
VARIABLES Subcommand 1412
DIFF Subcommand 1412
SDIFF Subcommand 1412
PERIOD Subcommand 1413
LN and NOLOG Subcommands 1413
SEASONAL Subcommand 1413
MXAUTO Subcommand 1414
APPLY Subcommand 1414
References 1415

PARTIAL CORR..... 1417
Overview 1417
VARIABLES Subcommand 1418
SIGNIFICANCE Subcommand 1419
STATISTICS Subcommand 1419
FORMAT Subcommand 1419
MISSING Subcommand 1420
MATRIX Subcommand 1420
Matrix Output 1420
Matrix Input 1420
Format of the Matrix Data File 1421
Split Files 1421
Missing Values 1421
Examples 1421

PERMISSIONS 1423
Overview 1423
PERMISSIONS Subcommand 1423

PLANCARDS..... 1425
Overview 1425
Examples 1426
FACTORS Subcommand 1426
FORMAT Subcommand 1427
OUTFILE Subcommand 1427
TITLE Subcommand 1428
FOOTER Subcommand 1428

PLS..... 1431
Overview 1431
Examples 1432
Variable Lists 1433
ID Subcommand 1434
MODEL Subcommand 1434
OUTDATASET Subcommand 1434
CRITERIA Subcommand 1435

PLUM..... 1437
Overview 1437
Variable List 1438
Weight Variable 1438
CRITERIA Subcommand 1438
LINK Subcommand 1439
LOCATION Subcommand 1439
MISSING Subcommand 1440
PRINT Subcommand 1440
SAVE Subcommand 1440
SCALE Subcommand 1441
TEST Subcommand 1442

POINT..... 1443
Overview 1443
Examples 1444
FILE Subcommand 1444
ENCODING Subcommand 1445
KEY Subcommand 1445

POWER ONEWAY ANOVA..... 1447
Overview 1447
Example 1448
PARAMETERS Subcommand 1448
CONTRAST Subcommand 1449
PLOT Subcommand 1449

POWER MEANS INDEPENDENT..... 1451
Overview 1451
Example 1452
PARAMETERS Subcommand 1452
PLOT Subcommand 1453

POWER MEANS ONESAMPLE..... 1455
Overview 1455
Example 1456
PARAMETERS Subcommand 1456

PLOT Subcommand 1456

POWER MEANS RELATED..... 1459
Overview 1459
Example 1460
PARAMETERS Subcommand 1460
PLOT Subcommand 1461

POWER PARTIALCORR..... 1463
Overview 1463
Example 1464
PARAMETERS Subcommand 1464
PLOT Subcommand 1465

POWER PEARSON ONESAMPLE..... 1467
Overview 1467
Example 1468
PARAMETERS Subcommand 1468
PLOT Subcommand 1468

POWER PROPORTIONS INDEPENDENT..... 1471
Overview 1471
Example 1472
PARAMETERS Subcommand 1472
PLOT Subcommand 1473

POWER PROPORTIONS ONESAMPLE..... 1475
Overview 1475
Example 1476
PARAMETERS Subcommand 1476
PLOT Subcommand 1477

POWER PROPORTIONS RELATED..... 1479
Overview 1479
Example 1480
PARAMETERS Subcommand 1480
PLOT Subcommand 1481

POWER SPEARMAN ONESAMPLE..... 1483
Overview 1483
Example 1484
PARAMETERS Subcommand 1484
PLOT Subcommand 1485

POWER UNIVARIATE LINEAR..... 1487
Overview 1487
Example 1488
PARAMETERS Subcommand 1488
PLOT Subcommand 1489

PPLOT..... 1491
Overview 1491
Example 1492
VARIABLES Subcommand 1493
DISTRIBUTION Subcommand 1493
FRACTION Subcommand 1494

TIES Subcommand 1494
TYPE Subcommand 1494
PLOT Subcommand 1495
STANDARDIZE and NOSTANDARDIZE Subcommands 1496
DIFF Subcommand 1496
SDIFF Subcommand 1496
PERIOD Subcommand 1497
LN and NOLOG Subcommands 1497
APPLY Subcommand 1497
TEMPLATE Subcommand 1498
References 1498

PREDICT 1501
Overview 1501
Syntax Rules 1502
Date Specifications 1502
Case Specifications 1502
Valid Range 1502
Examples 1502

PREFSCAL 1505
Overview 1506
Examples 1507
VARIABLES Subcommand 1507
INPUT Subcommand 1507
PROXIMITIES Subcommand 1509
WEIGHTS Subcommand 1509
INITIAL Subcommand 1509
CONDITION Subcommand 1510
TRANSFORMATION Subcommand 1511
MODEL Subcommand 1512
RESTRICTIONS Subcommand 1512
PENALTY Subcommand 1513
CRITERIA Subcommand 1513
PRINT Subcommand 1514
PLOT Subcommand 1515
OPTIONS Subcommand 1516
OUTFILE Subcommand 1517

PRESERVE 1519
Overview 1519
Example 1519

PRINCALS 1521
Overview 1521
Example 1522
VARIABLES Subcommand 1522
ANALYSIS Subcommand 1523
NOBSERVATIONS Subcommand 1524
DIMENSION Subcommand 1524
MAXITER Subcommand 1524
CONVERGENCE Subcommand 1524
PRINT Subcommand 1524
PLOT Subcommand 1525
SAVE Subcommand 1526
MATRIX Subcommand 1527

PRINT 1529
Overview 1529
Examples 1530
Formats 1530
Strings 1531
RECORDS Subcommand 1532
OUTFILE Subcommand 1532
ENCODING Subcommand 1533
TABLE Subcommand 1533

PRINT EJECT 1535
Overview 1535
Examples 1536

PRINT FORMATS 1537
Overview 1537
Examples 1538

PRINT SPACE 1539
Overview 1539
Examples 1539

PROBIT 1541
Overview 1541
Variable Specification 1542
MODEL Subcommand 1543
LOG Subcommand 1544
CRITERIA Subcommand 1544
NATRES Subcommand 1544
PRINT Subcommand 1545
MISSING Subcommand 1546
References 1546

PROCEDURE OUTPUT 1547
Overview 1547
Examples 1547

PROPORTIONS 1549
Overview 1550
Examples 1551
ONESAMPLE subcommand 1551
PAIREDSAMPLES subcommand 1552
INDEPENDENTSAMPLES subcommand 1552
SUCCESS subcommand 1553
MISSING subcommand 1553
CRITERIA subcommand 1554
References 1554

PROXIMITIES 1555
Overview 1555
Example 1556
Variable Specification 1556
STANDARDIZE Subcommand 1556
VIEW Subcommand 1557
MEASURE Subcommand 1557

Measures for Interval Data 1558
Measures for Frequency-Count Data 1558
Measures for Binary Data 1559
Transforming Measures in Proximity Matrix 1562
PRINT Subcommand 1562
ID Subcommand 1562
MISSING Subcommand 1562
MATRIX Subcommand 1562
Matrix Output 1563
Matrix Input 1563
Format of the Matrix Data File 1564
Split Files 1564
Example: Matrix Output to IBM SPSS Statistics External File 1564
Example: Matrix Output to External File 1564
Example: Matrix Output to Working File 1564
Example: Matrix Input from External File 1565
Example: Matrix Input from Working File 1565
Example: Matrix Output to and Then Input from Working File 1565
Example: Q-factor Analysis 1565
References 1566

PROXSCAL 1567
Overview 1567
Variable List Subcommand 1568
TABLE Subcommand 1569
SHAPE Subcommand 1571
INITIAL Subcommand 1571
WEIGHTS Subcommand 1572
CONDITION Subcommand 1572
TRANSFORMATION Subcommand 1573
SPLINE Keyword 1573
PROXIMITIES Subcommand 1573
MODEL Subcommand 1574
RESTRICTIONS Subcommand 1574
VARIABLES Keyword 1575
SPLINE Keyword 1575
ACCELERATION Subcommand 1575
CRITERIA Subcommand 1576
PRINT Subcommand 1576
PLOT Subcommand 1577
OUTFILE Subcommand 1578
MATRIX Subcommand 1579

QUANTILE REGRESSION 1581
Overview 1581
CRITERIA Subcommand 1582
MODEL Subcommand 1583
SAVE Subcommand 1583
PREDICT_EFFECTS Subcommand 1584
PREDICTION_TABLES Subcommand 1584
PREDICTION_LINES Subcommand 1584
PRINT Subcommand 1584
PLOT Subcommand 1584
OUTFILE Subcommand 1584
EXTERNAL Subcommand 1585

QUICK CLUSTER 1587

Overview ... 1587
Variable List ... 1588
CRITERIA Subcommand ... 1588
METHOD Subcommand ... 1589
INITIAL Subcommand ... 1589
FILE Subcommand ... 1589
PRINT Subcommand ... 1590
OUTFILE Subcommand ... 1590
SAVE Subcommand ... 1591
MISSING Subcommand ... 1591

RANK ... 1593
Overview ... 1593
Example ... 1594
VARIABLES Subcommand ... 1594
Function Subcommands ... 1594
INTO Keyword ... 1595
TIES Subcommand ... 1596
FRACTION Subcommand ... 1596
PRINT Subcommand ... 1597
MISSING Subcommand ... 1597
References ... 1597

RATIO STATISTICS ... 1599
Overview ... 1599
Examples ... 1599
Case Frequency ... 1600
Variable List ... 1600
MISSING Subcommand ... 1600
OUTFILE Subcommand ... 1601
PRINT Subcommand ... 1602

RBF ... 1603
Overview ... 1603
Examples ... 1605
Variable Lists ... 1606
EXCEPT Subcommand ... 1606
RESCALE Subcommand ... 1607
PARTITION Subcommand ... 1608
ARCHITECTURE Subcommand ... 1609
CRITERIA Subcommand ... 1609
MISSING Subcommand ... 1610
PRINT Subcommand ... 1610
PLOT Subcommand ... 1611
SAVE Subcommand ... 1612
OUTFILE Subcommand ... 1613

READ MODEL ... 1615
Overview ... 1615
Example ... 1616
FILE Subcommand ... 1616
KEEP and DROP Subcommands ... 1616
TYPE Subcommand ... 1616
TSET Subcommand ... 1617

RECODE ... 1619
Overview ... 1619

Syntax Rules 1620
Numeric Variables 1620
String Variables 1620
Operations 1620
Numeric Variables 1620
String Variables 1621
Examples 1621
INTO Keyword 1621
Numeric Variables 1621
String Variables 1622
CONVERT Keyword 1622

RECORD TYPE 1625
Overview 1625
Examples 1626
OTHER Keyword 1627
SKIP Subcommand 1628
CASE Subcommand 1628
MISSING Subcommand 1629
DUPLICATE Subcommand 1629
SPREAD Subcommand 1630

REFORMAT 1631

REGRESSION 1633
Overview 1634
Examples 1637
VARIABLES Subcommand 1637
DEPENDENT Subcommand 1638
METHOD Subcommand 1638
STATISTICS Subcommand 1639
Global Statistics 1640
Equation Statistics 1640
Statistics for the Independent Variables 1640
CRITERIA Subcommand 1641
Tolerance and Minimum Tolerance Tests 1641
Criteria for Variable Selection 1641
Confidence Intervals 1642
ORIGIN and NOORIGIN Subcommands 1642
REGWGT Subcommand 1642
DESCRIPTIVES Subcommand 1643
SELECT Subcommand 1644
MATRIX Subcommand 1645
Format of the Matrix Data File 1645
Split Files 1645
Missing Values 1645
Example 1646
MISSING Subcommand 1646
RESIDUALS Subcommand 1646
CASEWISE Subcommand 1647
SCATTERPLOT Subcommand 1648
PARTIALPLOT Subcommand 1648
OUTFILE Subcommand 1649
SAVE Subcommand 1649
TEMPLATE Subcommand 1650
References 1650

RELIABILITY 1653
Overview 1653
VARIABLES Subcommand 1654
SCALE Subcommand 1654
MODEL Subcommand 1655
STATISTICS Subcommand 1655
ICC Subcommand 1656
SUMMARY Subcommand 1656
METHOD Subcommand 1656
MISSING Subcommand 1657
MATRIX Subcommand 1657
Matrix Output 1657
Matrix Input 1657
Format of the Matrix Data File 1657
Split Files 1658
Missing Values 1658
Example: Matrix Output to External File 1658
Example: Matrix Output to Active Dataset 1658
Example: Matrix Output to Active Dataset 1659
Example: Matrix Input from External File 1659
Example: Matrix Input from Working File 1659

RENAME VARIABLES 1661
Overview 1661
Examples 1661
Mixed Case Variable Names 1662

REPEATING DATA 1663
Overview 1663
Operations 1664
Cases Generated 1664
Records Read 1664
Reading Past End of Record 1665
Examples 1665
STARTS Subcommand 1667
OCCURS Subcommand 1668
DATA Subcommand 1668
FILE Subcommand 1669
ENCODING Subcommand 1669
LENGTH Subcommand 1669
CONTINUED Subcommand 1670
ID Subcommand 1672
TABLE and NOTABLE Subcommands 1672

REPORT 1673
Overview 1674
Examples 1675
Defaults 1676
Options 1676
FORMAT subcommand 1677
OUTFILE subcommand 1678
VARIABLES subcommand 1679
Column contents 1679
Column heading 1679
Column heading alignment 1680
Column format 1680

STRING subcommand 1681
BREAK subcommand 1682
Column contents 1682
Column heading 1682
Column heading alignment 1683
Column format 1683
Using Dates as break variables 1684
SUMMARY subcommand 1684
Aggregate functions 1685
Composite functions 1686
Summary titles 1687
Summary print formats 1688
Other summary keywords 1690
TITLE and FOOTNOTE subcommands 1690
MISSING subcommand 1691

REPOSITORY ATTRIBUTES 1693
Overview 1693
FILE Keyword 1694
DESCRIPTION Keyword 1694
KEYWORDS Keyword 1694
AUTHOR Keyword 1694
VERSIONLABEL Keyword 1695
EXPIRATION Keyword 1695
TOPICS Keyword 1695
SECURITY Subcommand 1696

REPOSITORY CONNECT 1697
Overview 1697
SERVER Subcommand 1698
LOGIN Subcommand 1698

REPOSITORY COPY 1699
Overview 1699
Examples 1700

REREAD 1703
Overview 1703
Examples 1704
FILE Subcommand 1705
COLUMN Subcommand 1705

RESPONSE RATE 1707
Overview 1707
Examples 1708
VARIABLES subcommand 1708
MINRATE subcommand 1709
MAXCOUNT subcommand 1709

RESTORE 1711
Overview 1711
Example 1711

RMV 1713
Overview 1713
LINT Function 1714

MEAN Function 1714
MEDIAN Function 1714
SMEAN Function 1715
TREND Function 1715

ROC ANALYSIS 1717
Overview 1717
varlist BY varname(varvalue) 1718
MISSING Subcommand 1719
CRITERIA Subcommand 1719
DESIGN Subcommand 1719
PLOT Subcommand 1720
PRINT Subcommand 1720

ROC 1723
Overview 1723
varlist BY varname(varvalue) 1724
MISSING Subcommand 1724
CRITERIA Subcommand 1724
PRINT Subcommand 1725
PLOT Subcommand 1725

SAMPLE 1727
Overview 1727
Examples 1728

SAVE 1729
Overview 1729
Examples 1730
OUTFILE Subcommand 1731
VERSION Subcommand 1731
Variable Names 1731
UNSELECTED Subcommand 1731
DROP and KEEP Subcommands 1731
RENAME Subcommand 1732
MAP Subcommand 1733
COMPRESSED, UNCOMPRESSED, and ZCOMPRESSED Subcommands 1733
NAMES Subcommand 1733
PERMISSIONS Subcommand 1733
PASSPROTECT Subcommand 1734

SAVE CODEPAGE 1735
Overview 1735
OUTFILE Subcommand 1736
ENCODING Subcommand 1736
UNSELECTED Subcommand 1736
DROP and KEEP Subcommands 1737
PASSPROTECT Subcommand 1737

SAVE DATA COLLECTION 1739
Overview 1739
OUTFILE subcommand 1740
METADATA subcommand 1740
UNSELECTED subcommand 1741
DROP and KEEP subcommands 1741
MAP subcommand 1741

SAVE MODEL..... 1743
Overview 1743
OUTFILE Subcommand 1744
KEEP and DROP Subcommands 1744
TYPE Subcommand 1744

SAVE TRANSLATE..... 1747
Overview 1749
Operations 1749
Spreadsheets 1749
dBASE 1750
Comma-Delimited (CSV) Text Files 1750
Tab-Delimited Text Files 1750
SAS Files 1751
Stata Files 1751
SPSS/PC+ System Files 1752
ODBC Database Sources 1752
TYPE Subcommand 1753
VERSION Subcommand 1754
ENCODING Subcommand 1755
OUTFILE Subcommand 1756
FIELDNAMES Subcommand 1756
CELLS Subcommand 1757
TEXTOPTIONS Subcommand 1757
EXCELOPTIONS subcommand 1758
EDITION Subcommand 1758
PLATFORM Subcommand 1758
VALFILE Subcommand 1759
ODBC Database Subcommands 1759
CONNECT Subcommand 1759
ENCRYPTED and UNENCRYPTED Subcommands 1759
TABLE Subcommand 1759
SQL Subcommand 1760
BULKLOADING Subcommand 1761
ODBCOPTIONS subcommand 1761
APPEND Subcommand 1761
REPLACE Subcommand 1762
UNSELECTED Subcommand 1762
DROP and KEEP Subcommands 1762
RENAME Subcommand 1762
MISSING Subcommand 1763
COMPRESSED and UNCOMPRESSED Subcommands 1763
MAP Subcommand 1763

SAVETM1..... 1765
Overview 1765
CONNECTION subcommand 1766
CUBE subcommand 1767
MAPPINGS subcommand 1767

SCRIPT..... 1769
Overview 1769
Running Basic Scripts That Contain Syntax Commands 1769

SEASON..... 1771
Overview 1771

VARIABLES Subcommand 1772
MODEL Subcommand 1772
MA Subcommand 1773
PERIOD Subcommand 1773
APPLY Subcommand 1773
References 1774

SELECT IF 1775
Overview 1775
Examples 1776

SELECTPRED 1779
Overview 1779
Examples 1781
Variable lists 1781
EXCEPT subcommand 1782
SCREENING subcommand 1782
CRITERIA subcommand 1782
MISSING Subcommand 1783
PRINT subcommand 1783
PLOT subcommand 1784

SET 1785
Overview 1788
Example 1789
BASETEXTDIRECTION Subcommand 1789
BLANKS Subcommand 1790
BLOCK Subcommand 1790
BOX Subcommand 1790
CACHE Subcommand 1790
CCA, CCB, CCC, CCD, and CCE Subcommands 1791
CELLSBREAK, ROWSBREAK, and TOLERANCE Subcommands 1792
CMPTRANS Subcommand 1792
CTEMPLATE and TLOOK Subcommands 1792
DECIMAL Subcommand 1793
DEFOLANG Subcommand 1793
DIGITGROUPING Subcommand 1793
EPOCH Subcommand 1794
ERRORS, MESSAGES, RESULTS, and PRINTBACK Subcommands 1794
EXTENSIONS Subcommand 1794
FOOTNOTE Subcommand 1794
FORMAT Subcommand 1795
FUZZBITS Subcommand 1795
HEADER Subcommand 1795
JOURNAL Subcommand 1795
LEADZERO Subcommand 1796
LENGTH and WIDTH Subcommands 1796
LOCALE Subcommand 1796
MCACHE Subcommand 1797
MDISPLAY Subcommand 1797
MEXPAND and MPRINT Subcommands 1797
MIOUTPUT Subcommand 1798
MITERATE and MNEST Subcommands 1798
MTINDEX, RNG, and SEED Subcommands 1798
MXCELLS and WORKSPACE Subcommands 1799
MXERRS Subcommand 1799
MXLOOPS Subcommand 1799

MXWARNS Subcommand 1799
OATTRS and XVERSION Subcommands 1800
ODISPLAY Subcommand 1800
OLANG Subcommand 1801
ONUMBERS, OVARS, TNUMBERS, and TVARS Subcommands 1801
REPDEFER Subcommand 1801
SCALEMIN Subcommand 1802
SIGLESS Subcommand 1802
SMALL Subcommand 1802
SORT Subcommand 1803
SUMMARY Subcommand 1803
TABLERENDER Subcommand 1804
TFIT Subcommand 1804
THREADS Subcommand 1804
UNDEFINED Subcommand 1805
UNICODE Subcommand 1805
ZCOMPRESSION Subcommand 1805

SHIFT VALUES 1807
Overview 1807

SHOW 1809
Overview 1810
Example 1810
Subcommands 1810

SIMPLAN 1815
Overview 1816
Examples 1818
MODEL subcommand 1819
TARGETOPTS subcommand 1820
SIMINPUT subcommand 1821
FIXEDINPUT Subcommand 1825
CORRELATIONS Subcommand 1826
CONTINGENCY Subcommand 1827
AUTOFIT Subcommand 1827
STOPCRITERIA subcommand 1828
MISSING Subcommand 1829
VALUELABELS Subcommand 1829
PLAN Subcommand 1830
SOURCE Keyword 1830

SIMPREP BEGIN-SIMPREP END 1831
OUTFILE subcommand 1831

SIMRUN 1833
Overview 1834
Example 1835
PLAN Subcommand 1835
CRITERIA Subcommand 1835
DISTRIBUTION Subcommand 1836
SCATTERPLOT Subcommand 1837
BOXPLOT Subcommand 1837
TORNADO Subcommand 1838
PRINT Subcommand 1838
VIZSTYLESHEET Keyword 1839
OUTFILE Subcommand 1839

SORT CASES....1841
Overview 1841
OUTFILE Subcommand 1842
PASSPROTECT Subcommand 1842
Examples 1843
SORT CASES with Other Procedures 1843

SORT VARIABLES.... 1845
Overview 1845

SPATIAL ASSOCIATION RULES.... 1847
Overview 1848
MAPSPEC subcommand 1849
AUTOBINNING subcommand 1849
AGGREGATION subcommand 1849
DATASET subcommand 1850
RULEGENERATION subcommand 1851
MODELTABLES subcommand 1852
MAPOUTPUT subcommand 1852
WORDCLOUD subcommand 1853
RULESTABLE subcommand 1853
SAVE subcommand 1854

SPATIAL MAPSPEC.... 1855
Overview 1855
MAPPROJECTION subcommand 1856
MAP subcommand 1856
DATA subcommand 1858
SPECFILE subcommand 1859

SPATIAL TEMPORAL PREDICTION.... 1861
Overview 1862
MAPSPEC subcommand 1862
AGGREGATION subcommand 1862
DATASET subcommand 1863
TIMEFIELDS subcommand 1864
MODELBUILDING subcommand 1865
MODELTABLES subcommand 1866
MAPOUTPUT subcommand 1866
SAVE subcommand 1867

SPCHART.... 1869
Overview 1870
Example 1871
TEMPLATE Subcommand 1872
TITLE, SUBTITLE, and FOOTNOTE Subcommands 1872
XR and XS Subcommands 1872
Data Organization 1874
Variable Specification 1875
(XBARONLY) Keyword 1875
I and IR Subcommands 1875
Data Organization 1876
Variable Specification 1876
P and NP Subcommands 1877
Data Organization 1878

Variable Specification 1878
C and U Subcommands 1879
Data Organization 1879
Variable Specification 1880
CPCHART Subcommand 1880
Data Organization 1881
Variable Specification 1882
STATISTICS Subcommand 1883
The Process Capability Indices 1883
The Process Performance Indices 1884
Process Data 1884
Measure(s) for Assessing Normality 1885
RULES Subcommand 1885
ID Subcommand 1885
CAPSIGMA Subcommand 1886
SPAN Subcommand 1886
CONFORM and NONCONFORM Subcommands 1887
SIGMAS Subcommand 1887
MINSAMPLE Subcommand 1887
LSL and USL Subcommand 1887
TARGET Subcommand 1888
MISSING Subcommand 1888
NORMAL Subcommand 1888
REFERENCE Subcommand 1888

SPECTRA 1891
Overview 1891
Example 1892
VARIABLES Subcommand 1892
CENTER Subcommand 1892
WINDOW Subcommand 1893
PLOT Subcommand 1893
BY Keyword 1894
CROSS Subcommand 1894
SAVE Subcommand 1895
APPLY Subcommand 1896
References 1896

SPLIT FILE 1897
Overview 1897
LAYERED and SEPARATE Subcommands 1898
Examples 1898

STAR JOIN 1899
Overview 1899
SELECT subcommand 1900
FROM subcommand 1900
JOIN subcommand 1901
OUTFILE subcommand 1902
PASSPROTECT subcommand 1902
Example: STAR JOIN with two lookup table files 1902
Example: STAR JOIN with two key-value pairs in the same file 1904
Example: STAR JOIN with two keys required to match cases 1904

STRING 1907
Overview 1907
Examples 1907

SUBTITLE 1909
Overview 1909
Examples 1909

SUMMARIZE 1911
Overview 1911
Example 1912
TABLES Subcommand 1913
TITLE and FOOTNOTE Subcommands 1913
CELLS Subcommand 1913
MISSING Subcommand 1914
FORMAT Subcommand 1914
STATISTICS Subcommand 1915

SURVIVAL 1917
Overview 1917
Examples 1918
TABLE Subcommand 1919
INTERVAL Subcommand 1919
STATUS Subcommand 1920
PLOTS Subcommand 1921
PRINT Subcommand 1922
COMPARE Subcommand 1922
CALCULATE Subcommand 1922
Using Aggregated Data 1923
MISSING Subcommand 1924
WRITE Subcommand 1924
Format 1925
Record Order 1926

SYSFILE INFO 1927
Overview 1927

TCM ANALYSIS 1929
Overview 1930
MODELSYSTEM Subcommand 1931
EXPRESSIONS Subcommand 1931
SCENARIOPERIOD subcommand 1931
SCENARIO subcommand 1932
SCENARIOGROUP subcommand 1934
TARGETLIST subcommand 1935
OPTIONS subcommand 1935

TCM APPLY 1937
Overview 1938
MODELSYSTEM Subcommand 1939
OPTIONS subcommand 1939
TARGETFILTER subcommand 1941
SERIESFILTER subcommand 1942
FILTEREDOUTPUT subcommand 1942
SYSTEMOUTPUT subcommand 1944
SAVE subcommand 1945
OUTFILE subcommand 1946
PASSPROTECT subcommand 1946

TCM MODEL..... 1949
Overview 1951
DATASETTINGS subcommand 1952
DATAFILTER subcommand 1958
BUILDOPTIONS subcommand 1958
TARGETFILTER subcommand 1960
SERIESFILTER subcommand 1961
FILTEREDOUTPUT subcommand 1961
SYSTEMOUTPUT subcommand 1963
SAVE subcommand 1964
OUTFILE subcommand 1965
PASSPROTECT subcommand 1965
FIELDSGROUP subcommand 1966
FIELDS subcommand 1966

TDISPLAY..... 1969
Overview 1969
TYPE Subcommand 1969

TEMPORARY..... 1971
Overview 1971
Examples 1972

TIME PROGRAM..... 1973
Overview 1973
Example 1973

TITLE..... 1975
Overview 1975
Examples 1975

TMS BEGIN..... 1977
Overview 1977
EXAMPLES 1980
DESTINATION Subcommand 1981

TMS END..... 1983
Overview 1983
PRINT Subcommand 1984

TMS IMPORT..... 1985
Overview 1985
Examples 1985
INFILE Subcommand 1986
SAVE Subcommand 1986
OUTFILE Subcommand 1986

TMS MERGE..... 1989
Overview 1989
TRANSFORMATIONS, MODEL, and DESTINATION Subcommands 1990
PRINT Subcommand 1990

TREE..... 1991
Overview 1992
Model Variables 1994

Measurement Level 1994
FORCE Keyword 1995
DEPCATEGORIES Subcommand 1995
TREE Subcommand 1996
PRINT Subcommand 1997
GAIN Subcommand 1998
PLOT Subcommand 1999
RULES Subcommand 2000
SAVE Subcommand 2001
METHOD Subcommand 2002
GROWTHLIMIT Subcommand 2003
VALIDATION Subcommand 2003
CHAID Subcommand 2004
CRT Subcommand 2006
QUEST Subcommand 2006
COSTS Subcommand 2007
Custom Costs 2007
PRIORS Subcommand 2007
SCORES Subcommand 2008
PROFITS Subcommand 2009
INFLUENCE Subcommand 2010
OUTFILE Subcommand 2010
MISSING Subcommand 2010
TARGETRESPONSE Subcommand 2011

TSAPPLY 2013
Overview 2014
Examples 2015
Goodness-of-Fit Measures 2016
MODELSUMMARY Subcommand 2016
MODELSTATISTICS Subcommand 2017
MODELDETAILS Subcommand 2018
SERIESPLOT Subcommand 2019
OUTPUTFILTER Subcommand 2019
SAVE Subcommand 2020
AUXILIARY Subcommand 2021
MISSING Subcommand 2022
MODEL Subcommand 2022

TSET 2025
Overview 2025
DEFAULT Subcommand 2025
ID Subcommand 2026
MISSING Subcommand 2026
MXNEWVARS Subcommand 2026
MXPREDICT Subcommand 2026
NEWVAR Subcommand 2026
PERIOD Subcommand 2026
PRINT Subcommand 2027

TSHOW 2029
Overview 2029
Example 2029

TSMODEL 2031
Overview 2032
Examples 2034

Goodness-of-Fit Measures 2035
MODELSUMMARY Subcommand 2036
MODELSTATISTICS Subcommand 2037
MODELDETAILS Subcommand 2037
SERIESPLOT Subcommand 2038
OUTPUTFILTER Subcommand 2038
SAVE Subcommand 2039
AUXILIARY Subcommand 2040
MISSING Subcommand 2041
MODEL Subcommand 2041
EXPERTMODELER Subcommand 2043
EXSMOOTH Subcommand 2044
ARIMA Subcommand 2045
TRANSFERFUNCTION Subcommand 2046
AUTOOUTLIER Subcommand 2048
OUTLIER Subcommand 2049

TSPLOT 2051
Overview 2051
Basic Specification 2052
Example 2053
VARIABLES Subcommand 2053
DIFF Subcommand 2053
SDIFF Subcommand 2053
PERIOD Subcommand 2053
LN and NOLOG Subcommands 2054
ID Subcommand 2054
FORMAT Subcommand 2054
MARK Subcommand 2057
SPLIT Subcommand 2058
APPLY Subcommand 2058

T-TEST 2061
Overview 2061
Examples 2062
VARIABLES Subcommand 2063
TESTVAL Subcommand 2063
GROUPS Subcommand 2063
PAIRS Subcommand 2063
CRITERIA Subcommand 2064
ES Subcommand 2064
MISSING Subcommand 2064

TWOSTEP CLUSTER 2065
Overview 2065
Variable List 2066
CATEGORICAL Subcommand 2066
CONTINUOUS Subcommand 2067
CRITERIA Subcommand 2067
DISTANCE Subcommand 2067
HANDLENOISE Subcommand 2067
INFILE Subcommand 2068
MEMALLOCATE Subcommand 2068
MISSING Subcommand 2068
NOSTANDARDIZE Subcommand 2068
NUMCLUSTERS Subcommand 2069
OUTFILE Subcommand 2069

PRINT Subcommand 2069
VIEWMODEL Subcommand 2070
SAVE Subcommand 2070

UNIANOVA 2071
Overview 2072
Example 2073
UNIANOVA Variable List 2074
RANDOM Subcommand 2074
REGWGT Subcommand 2074
METHOD Subcommand 2075
INTERCEPT Subcommand 2076
MISSING Subcommand 2076
CRITERIA Subcommand 2076
PRINT Subcommand 2076
PLOT Subcommand 2078
TEST Subcommand 2079
LMATRIX Subcommand 2079
KMATRIX Subcommand 2080
CONTRAST Subcommand 2081
POSTHOC Subcommand 2082
EMMEANS Subcommand 2085
ROBUST Subcommand 2086
SAVE Subcommand 2086
OUTFILE Subcommand 2087
MBPDESIGN, BPDESIGN, and FDESIGN Subcommands 2087
DESIGN Subcommand 2088

UPDATE 2091
Overview 2091
Examples 2093
FILE Subcommand 2093
Text Data Files 2093
BY Subcommand 2094
RENAME Subcommand 2094
DROP and KEEP Subcommands 2095
IN Subcommand 2095
MAP Subcommand 2095

USE 2097
Overview 2097
Syntax Rules 2097
DATE Specifications 2098
Case Specifications 2098
Keywords FIRST and LAST 2098
PERMANENT Subcommand 2098
Examples 2098

VALIDATEDATA 2099
Overview 2099
Examples 2101
Variable Lists 2102
VARCHECKS Subcommand 2103
IDCHECKS Subcommand 2103
CASECHECKS Subcommand 2103
RULESUMMARIES Subcommand 2104
CASEREPORT Subcommand 2104

SAVE Subcommand 2105
Defining Validation Rules 2105
Single-Variable Validation Rules 2105
Cross-Variable Validation Rules 2107

VALUE LABELS 2109
Overview 2109
Examples 2110

VARCOMP 2113
Overview 2113
Example 2114
Variable List 2114
RANDOM Subcommand 2114
METHOD Subcommand 2115
INTERCEPT Subcommand 2115
MISSING Subcommand 2115
REGWGT Subcommand 2115
CRITERIA Subcommand 2116
PRINT Subcommand 2116
OUTFILE Subcommand 2116
DESIGN Subcommand 2117

VARIABLE ALIGNMENT 2119
Overview 2119

VARIABLE ATTRIBUTE 2121
Overview 2121
Example 2122

VARIABLE LABELS 2125
Overview 2125
Examples 2125

VARIABLE LEVEL 2127
Overview 2127

VARIABLE ROLE 2129
Overview 2129

VARIABLE WIDTH 2131
Overview 2131

VARSTOCASES 2133
Overview 2133
Example 2134
MAKE Subcommand 2134
ID Subcommand 2135
INDEX Subcommand 2135
Simple Numeric Index 2135
Variable Name Index 2136
Multiple Numeric Indices 2136
NULL Subcommand 2137
COUNT Subcommand 2137
DROP and KEEP Subcommands 2137

VECTOR **2139**
Overview 2139
Examples 2140
VECTOR: Short Form 2141
VECTOR outside a Loop Structure 2142

VERIFY **2145**
Overview 2145
VARIABLES Subcommand 2145
Examples 2146

WEIGHT **2147**
Overview 2147
Examples 2148

WEIGHTED KAPPA **2149**
Overview (WEIGHTED KAPPA command) 2149
varlist1 BY varlist2 2150
CRITERIA Subcommand 2150
PRINT Subcommand 2150
MISSING Subcommand 2151

WLS **2153**
Overview 2153
Example 2154
VARIABLES Subcommand 2155
SOURCE Subcommand 2155
DELTA Subcommand 2155
WEIGHT Subcommand 2155
CONSTANT and NOCONSTANT Subcommands 2156
SAVE Subcommand 2156
PRINT Subcommand 2156
APPLY Subcommand 2156

WRITE **2159**
Overview 2159
Examples 2160
Formats 2160
Strings 2161
RECORDS Subcommand 2161
OUTFILE Subcommand 2162
ENCODING Subcommand 2162
TABLE Subcommand 2162

WRITE FORMATS **2165**
Overview 2165
Examples 2166

XGRAPH **2167**
Overview 2168
CHART Expression 2168
Functions 2168
Data Element Types 2170
Measurement Level 2170
Variable Placeholder 2170

Case Numbers 2170
Blending, Clustering, and Stacking 2171
Labels 2171
BIN Subcommand 2171
START Keyword 2172
SIZE Keyword 2172
DISPLAY Subcommand 2172
DOT Keyword 2172
DISTRIBUTION Subcommand 2173
TYPE Keyword 2173
COORDINATE Subcommand 2173
SPLIT Keyword 2173
ERRORBAR Subcommand 2173
CI Keyword 2173
STDDEV Keyword 2173
SE Keyword 2174
MISSING Subcommand 2174
USE Keyword 2174
REPORT Keyword 2174
PANEL Subcommand 2174
COLVAR and ROWVAR Keywords 2174
COLOP and ROWOP Keywords 2175
TEMPLATE Subcommand 2175
FILE Keyword 2176
TITLES Subcommand 2176
TITLE Keyword 2176
SUBTITLE Keyword 2176
FOOTNOTE Keyword 2176
3-D Bar Examples 2176
Population Pyramid Examples 2177
Dot Plot Examples 2179

XSAVE 2181
Overview 2181
Examples 2182
OUTFILE Subcommand 2183
DROP and KEEP Subcommands 2183
RENAME Subcommand 2184
MAP Subcommand 2184
COMPRESSED, UNCOMPRESSED, and ZCOMPRESSED Subcommands 2184
PERMISSIONS Subcommand 2185

Commands and Program States 2187
Program States 2187
Determining Command Order 2188
Unrestricted Utility Commands 2191
File Definition Commands 2191
Input Program Commands 2191
Transformation Commands 2191
Restricted Transformations 2193
Procedures 2193

Defining Complex Files 2195
Rectangular File 2195
Nested Files 2195
Nested Files with Missing Records 2196
Grouped Data 2197

Using DATA LIST ... 2197
Using FILE TYPE GROUPED ... 2198
Mixed Files ... 2200
Reading Each Record in a Mixed File ... 2200
Reading a Subset of Records in a Mixed File ... 2200
Repeating Data ... 2201
Fixed Number of Repeating Groups ... 2201
Varying Number of Repeating Groups ... 2202

Using the Macro Facility ... 2205
Example 1: Automating a File-Matching Task ... 2205
Example 2: Testing Correlation Coefficients ... 2209
Example 3: Generating Random Data ... 2211

Canonical Correlation and Ridge Regression Macros ... 2215
Canonical Correlation Macro ... 2215
Ridge Regression Macro ... 2215

File Specifications for IBM SPSS Collaboration and Deployment Services Repository Objects ... 2217
Versions ... 2218
Using File Handles for IBM SPSS Collaboration and Deployment Services Repository Locations ... 2218
Setting the Working Directory to a IBM SPSS Collaboration and Deployment Services Repository Location ... 2219

TABLES and IGRAPH Command Syntax Converter ... 2221

Notices ... 2223
Trademarks ... 2224

Index ... 2225

Introduction: A Guide to Command Syntax

The *Command Syntax Reference* is arranged alphabetically by command name to provide quick access to detailed information about each command in the syntax command language. This introduction groups commands into broad functional areas. Some commands are listed more than once because they perform multiple functions, and some older commands that have been deprecated in favor of newer and better alternatives (but are still supported) are not included here. Changes to the command syntax language (since version 12.0), including modifications to existing commands and addition of new commands, are provided in the section "Release History" on page 10.

Core System

The Core system contains the core functionality plus a number of charting procedures. There are also numerous add-on modules that contain specialized functionality.

Getting Data

You can read in a variety of data formats, including data files saved in IBM SPSS Statistics format, SAS datasets, database tables from many database sources, Excel and other spreadsheets, and text data files with both simple and complex structures.

Get. Reads IBM SPSS Statistics data files.

Import. Reads portable data files created with the Export command.

Add Files. Combines multiple data files by adding cases.

Match Files. Combines multiple data files by adding variables.

Update. Replaces values in a primary file with updated values.

Get Translate. Reads spreadsheet and dBASE files.

Get Data. Reads Excel files, text data files, and database tables.

Get Data. Reads Excel files, text data files, and database tables.

Get Capture. Reads database tables.

Get SAS. Reads SAS dataset and SAS transport files.

Get Stata. Reads Stata data files.

Data List. Reads text data files.

Begin Data-End Data. Used with Data List to read inline text data.

File Type. Defines mixed, nested, and grouped data structures.

Record Type. Used with File Type to read complex text data files.

Input Program. Generates case data and/or reads complex data files.

End Case. Used with Input Program to define cases.

End File. Used with Input Program to indicate end of file.

Repeating Data. Used with Input Program to read input cases whose records contain repeating groups of data.

Reread. Used with Input Program to reread a record.

Keyed Data List. Reads data from nonsequential files.

Point. Used with Keyed Data to establish the location at which sequential access begins (or resumes) in a keyed file.

Dataset Name. Provides the ability to have multiple data sources open at the same time.

Dataset Activate. Makes the named dataset the active dataset.

Saving and Exporting Data

You can save data in numerous formats, including IBM SPSS Statistics data file, Excel spreadsheet, database table, delimited text, and fixed-format text.

Save. Saves the active dataset in IBM SPSS Statistics format.

Xsave. Saves data in IBM SPSS Statistics format without requiring a separate data pass.

Export. Saves data in portable format.

Save Data Collection. Saves a data file in IBM SPSS Statistics format and a metadata file in Data Collection MDD format for use in Data Collection applications.

Write. Saves data as fixed-format text.

Save Translate. Saves data as tab-delimited text and comma-delimted (CSV) text.

Save Translate. Saves data in Excel and other spreadsheet formats and dBASE format.

Save Translate. Replaces or appends to existing database tables or creates new database tables.

Statistics Adapter

Repository Attributes. Sets attributes for an object in a

Repository Connect. Establishes a connection to a IBM SPSS Collaboration and Deployment Services Repository and logs in the user.

Repository Copy. Copies an arbitrary file from the local file system to a IBM SPSS Collaboration and Deployment Services Repository or copies a file from a IBM SPSS Collaboration and Deployment Services Repositoryto the local file system.

Data Definition

IBM SPSS Statistics data files can contain more than simply data values. The **dictionary** can contain a variety of metadata attributes, including measurement level, display format, descriptive variable and value labels, and special codes for missing values.

Apply Dictionary. Applies variable and file-based dictionary information from an external IBM SPSS Statistics data file.

Datafile Attribute. Creates user-defined attributes that can be saved with the data file.

Variable Attribute. Creates user-defined variable attributes that can be saved with variables in the data file.

Variable Labels. Assigns descriptive labels to variables.

Value Labels. Assigns descriptive labels to data values.

Add Value Labels. Assigns descriptive labels to data values.

Variable Level. Specifies the level of measurement (nominal, ordinal, or scale).

Missing Values. Specifies values to be treated as missing.

Rename. Changes variable names.

Formats. Changes variable print and write formats.

Print Formats. Changes variable print formats.

Write Formats. Changes variable write formats.

Variable Alignment. Specifies the alignment of data values in the Data Editor.

Variable Width. Specifies the column width for display of variables in the Data Editor.

Mrsets. Defines and saves multiple response set information.

Data Transformations

You can perform data transformations ranging from simple tasks, such as collapsing categories for analysis, to more advanced tasks, such as creating new variables based on complex equations and conditional statements.

Autorecode. Recodes the values of string and numeric variables to consecutive integers.

Compute. Creates new numeric variables or modifies the values of existing string or numeric variables.

Count. Counts occurrences of the same value across a list of variables.

Create. Produces new series as a function of existing series.

Date. Generates date identification variables.

Leave. Suppresses reinitialization and retains the current value of the specified variable or variables when the program reads the next case.

Numeric. Declares new numeric variables that can be referred to before they are assigned values.

Rank. Produces new variables containing ranks, normal scores, and Savage and related scores for numeric variables.

Recode. Changes, rearranges, or consolidates the values of an existing variable.

RMV. Replaces missing values with estimates computed by one of several methods.

Shift Values. Creates new variables that contain the values of existing variables from preceding or subsequent cases.

String. Declares new string variables.

Temporary. Signals the beginning of temporary transformations that are in effect only for the next procedure.

TMS Begin. Indicates the beginning of a block of transformations to be exported to a file in PMML format (with IBM SPSS Statistics extensions).

TMS End. Marks the end of a block of transformations to be exported as PMML.

TMS Import. Converts a PMML file containing ADP tranformations into command syntax.

TMS Merge. Merges a PMML file containing exported transformations with a PMML model file.

File Information

You can add descriptive information to a data file and display file and data attributes for the active dataset or any selected IBM SPSS Statistics data file.

Add Documents. Creates a block of text of any length in the active dataset.

Display. Displays information from the dictionary of the active dataset.

Compare Datasets. Compares the contents of the active dataset to another dataset in the current session or an external data file in IBM SPSS Statistics format.

Document. Creates a block of text of any length in the active dataset.

Drop Documents. Deletes all text added with Document or Add Documents.

Sysfile Info. Displays complete dictionary information for all variables in a IBM SPSS Statistics data file.

File Transformations

Data files are not always organized in the ideal form for your specific needs. You may want to combine data files, sort the data in a different order, select a subset of cases, or change the unit of analysis by grouping cases together. A wide range of file transformation capabilities is available.

Delete Variables. Deletes variables from the data file.

Sort Cases. Reorders the sequence of cases based on the values of one or more variables.

Weight. Case replication weights based on the value of a specified variable.

Filter. Excludes cases from analysis without deleting them from the file.

N of Cases. Deletes all but the first *n* cases in the data file.

Sample. Selects a random sample of cases from the data file, deleting unselected cases.

Select If. Selects cases based on logical conditions, deleting unselected cases.

Split File. Splits the data into separate analysis groups based on values of one or more split variables.

Use. Designates a range of observations for time series procedures.

Aggregate. Aggregates groups of cases or creates new variables containing aggregated values.

Casestovars. Restructures complex data that has multiple rows for a case.

Varstocases. Restructures complex data structures in which information about a variable is stored in more than one column.

Flip. Transposes rows (cases) and columns (variables).

Add Files. Combines multiple IBM SPSS Statistics data files or open datasets by adding cases.

Match Files. Combines multiple IBM SPSS Statistics data files or open datasets by adding variables.

Star Join. Combines multiple IBM SPSS Statistics data files or open datasets by adding variables.

Update. Replaces values in a primary file with updated values.

Programming Structures

As with other programming languages, the command syntax contains standard programming structures that can be used to do many things. These include the ability to perform actions only if some condition is true (if/then/else processing), repeat actions, create an array of elements, and use loop structures.

Break. Used with Loop and Do If-Else If to control looping that cannot be fully controlled with conditional clauses.

Do If-Else If. Conditionally executes one or more transformations based on logical expressions.

Do Repeat. Repeats the same transformations on a specified set of variables.

If. Conditionally executes a single transformation based on logical conditions.

Loop. Performs repeated transformations specified by the commands within the loop until they reach a specified cutoff.

Vector. Associates a vector name with a set of variables or defines a vector of new variables.

Programming Utilities

Define. Defines a program macro.

Echo. Displays a specified text string as text output.

Execute. Forces the data to be read and executes the transformations that precede it in the command sequence.

Host. Executes external commands at the operating system level.

Include. Includes commands from the specified file.

Insert. Includes commands from the specified file.

Script. Runs the specified script file.

General Utilities

Cache. Creates a copy of the data in temporary disk space for faster processing.

Clear Transformations. Discards all data transformation commands that have accumulated since the last procedure.

Erase. Deletes the specified file.

File Handle. Assigns a unique file handle to the specified file.

New File. Creates a blank, new active dataset.

Permissions. Changes the read/write permissions for the specified file.

Preserve. Stores current Set command specifications that can later be restored by the Restore command.

Print. Prints the values of the specified variables as text output.

Print Eject. Displays specified information at the top of a new page of the output.

Print Space. Displays blank lines in the output.

Restore. Restores Set specifications that were stored by Preserve.

Set. Customizes program default settings.

Show. Displays current settings, many of which are set by the Set command.

Subtitle. Inserts a subtitle on each page of output.

Title. Inserts a title on each page of output.

Matrix Operations

Matrix. Using matrix programs, you can write your own statistical routines in the compact language of matrix algebra.

Matrix Data. Reads raw matrix materials and converts them to a matrix data file that can be read by procedures that handle matrix materials.

Mconvert. Converts covariance matrix materials to correlation matrix materials or vice versa.

Output Management System

The Output Management System (OMS) provides the ability to automatically write selected categories of output to different output files in different formats, including IBM SPSS Statistics data file format, HTML, XML, and text.

OMS. Controls the routing and format of output. Output can be routed to external files in XML, HTML, text, and SAV (IBM SPSS Statistics data file) formats.

OMSEnd. Ends active OMS commands.

OMSInfo. Displays a table of all active OMS commands.

OMSLog. Creates a log of OMS activity.

Output Documents

These commands control Viewer windows and files.

Output Activate. Controls the routing of output to Viewer output documents.

Output Close. Closes the specified Viewer document.

Output Display. Displays a table of all open Viewer documents.

Output Export. Exports output to external files in various formats (e.g., Word, Excel, PDF, HTML, text).

Output Name. Assigns a name to the active Viewer document. The name is used to refer to the output document in subsequent Output commands.

Output New. Creates a new Viewer output document, which becomes the active output document.

Output Open. Opens a Viewer document, which becomes the active output document. You can use this command to append output to an existing output document.

Output Save. Saves the contents of an open output document to a file.

Charts

Caseplot. Casewise plots of sequence and time series variables.

GGraph. Bar charts, pie charts, line charts, scatterplots, custom charts.

Pplot. Probability plots of sequence and time series variables.

Spchart. Control charts, including X-Bar, r, s, individuals, moving range, and u.

Time Series

The Core system provides some basic time series functionality, including a number of time series chart types. Extensive time series analysis features are provided in SPSS Statistics Professional Edition or the Forecasting option. See the topic "Add-On Modules" on page 6 for more information.

ACF. Displays and plots the sample autocorrelation function of one or more time series.

CCF. Displays and plots the cross-correlation functions of two or more time series.

PACF. Displays and plots the sample partial autocorrelation function of one or more time series.

Tsplot. Plot of one or more time series or sequence variables.

Fit. Displays a variety of descriptive statistics computed from the residual series for evaluating the goodness of fit of models.

Predict. Specifies the observations that mark the beginning and end of the forecast period.

Tset. Sets global parameters to be used by procedures that analyze time series and sequence variables.

Tshow. Displays a list of all of the current specifications on the Tset, Use, Predict, and Date commands.

Verify. Produces a report on the status of the most current Date, Use, and Predict specifications.

Add-On Modules

Add-on modules are not included with the Core system. The commands available to you will depend on your software license.

SPSS Statistics Base Edition

ALSCAL. Multidimensional scaling (MDS) and multidimensional unfolding (MDU) using an alternating least-squares algorithm.

Cluster. Hierarchical clusters of items based on distance measures of dissimilarity or similarity. The items being clustered are usually cases, although variables can also be clustered.

Codebook. Reports the dictionary information -- such as variable names, variable labels, value labels, missing values -- and summary statistics for all or specified variables and multiple response sets in the active dataset.

Correlations. Pearson correlations with significance levels, univariate statistics, covariances, and cross-product deviations.

Crosstabs. Crosstabulations (contingency tables) and measures of association.

Curvefit. Fits selected curves to a line plot.

Descriptives. Univariate statistics, including the mean, standard deviation, and range.

Discriminant. Classifies cases into one of several mutually exclusive groups based on their values for a set of predictor variables.

Examine. Descriptive statistics, stem-and-leaf plots, histograms, boxplots, normal plots, robust estimates of location, and tests of normality.

Factor. Identifies underlying variables, or factors, that explain the pattern of correlations within a set of observed variables.

Frequencies. Tables of counts and percentages and univariate statistics, including the mean, median, and mode.

Graph. Bar charts, pie charts, line charts, histograms, scatterplots, etc.

KNN. Classifies and predicts cases based upon the values "nearest neighboring" cases.

Linear. Creates a predictive model for a continuous target.

List. Individual case listing.

Means. Group means and related univariate statistics for dependent variables within categories of one or more independent variables.

Mult Response. Frequency tables and crosstabulations for multiple-response data.

Nonparametric. Collection of one-sample, independent samples, and related samples nonparametric tests.

Nonpar Corr. Rank-order correlation coefficients: Spearman's rho and Kendall's tau-b, with significance levels.

Npar Tests. Collection of one-sample, independent samples, and related samples nonparametric tests.

OLAP Cubes. Summary statistics for scale variables within categories defined by one or more categorical grouping variables.

Oneway. One-way analysis of variance.

Partial Corr. Partial correlation coefficients between two variables, adjusting for the effects of one or more additional variables.

Plum. Analyzes the relationship between a polytomous ordinal dependent variable and a set of predictors.

Proximities. Measures of similarity, dissimilarity, or distance between pairs of cases or pairs of variables.

Quick Cluster. When the desired number of clusters is known, this procedure groups cases efficiently into clusters.

Ratio Statistics. Descriptive statistics for the ratio between two variables.

Regression. Multiple regression equations and associated statistics and plots.

Reliability. Estimates reliability statistics for the components of multiple-item additive scales.

Report. Individual case listing and group summary statistics.

ROC. Receiver operating characteristic (ROC) curve and an estimate of the area under the curve.

Simplan. Creates a simulation plan for use with the Simrun command.

Simprep Begin-Simprep End. Specifies a block of compute statements and variable definition statements that create a custom model for use with the Simplan command.

Simrun. Runs a simulation based on a simulation plan created by the Simplan command.

Summarize. Individual case listing and group summary statistics.

TTest. One sample, independent samples, and paired samples t tests.

Twostep Cluster. Groups observations into clusters based on a nearness criterion. The procedure uses a hierarchical agglomerative clustering procedure in which individual cases are successively combined to form clusters whose centers are far apart.

Unianova. Regression analysis and analysis of variance for one dependent variable by one or more factors and/or variables.

Xgraph. Creates 3-D bar charts, population pyramids, and dot plots.

Advanced Statistics

Coxreg. Cox proportional hazards regression for analysis of survival times.

Genlin. Generalized Linear Model. Genlin allows you to fit a broad spectrum of "generalized" models in which the distribution of the error term need not be normal and the relationship between the dependent variable and predictors need only be linear through a specified transformation.

Genlinmixed. Generalized linear mixed models extend the linear model so that the target is linearly related to the factors and covariates via a specified link function, the target can have a non-normal distribution, and the observations can be correlated. Generalized linear mixed models cover a wide variety of models, from simple linear regression to complex multilevel models for non-normal longitudinal data.

Genlog. A general procedure for model fitting, hypothesis testing, and parameter estimation for any model that has categorical variables as its major components.

GLM. General Linear Model. A general procedure for analysis of variance and covariance, as well as regression.

Hiloglinear. Fits hierarchical loglinear models to multidimensional contingency tables using an iterative proportional-fitting algorithm.

KM. Kaplan-Meier (product-limit) technique to describe and analyze the length of time to the occurrence of an event.

Mixed. The mixed linear model expands the general linear model used in the GLM procedure in that the data are permitted to exhibit correlation and non-constant variability.

Survival. Actuarial life tables, plots, and related statistics.

Varcomp. Estimates variance components for mixed models.

Regression

Logistic Regression. Regresses a dichotomous dependent variable on a set of independent variables.

Nomreg. Fits a multinomial logit model to a polytomous nominal dependent variable.

NLR, CNLR. Nonlinear regression is used to estimate parameter values and regression statistics for models that are not linear in their parameters.

WLS. Weighted Least Squares. Estimates regression models with different weights for different cases.

2SLS. Two-stage least-squares regression.

Custom Tables

Ctables. Produces tables in one, two, or three dimensions and provides a great deal of flexibility for organizing and displaying the contents.

Decision Trees

Tree. Tree-based classification models.

Categories

Catreg. Categorical regression with optimal scaling using alternating least squares.

CatPCA. Principal components analysis.

Overals. Nonlinear canonical correlation analysis on two or more sets of variables.

Correspondence . Displays the relationships between rows and columns of a two-way table graphically by a scatterplot matrix.

Multiple Correspondence. Quantifies nominal (categorical) data by assigning numerical values to the cases (objects) and categories, such that objects within the same category are close together and objects in different categories are far apart.

Proxscal. Multidimensional scaling of proximity data to find a least-squares representation of the objects in a low-dimensional space.

Complex Samples

CSPlan. Creates a complex sample design or analysis specification.

CSSelect. Selects complex, probability-based samples from a population.

CSDescriptives. Estimates means, sums, and ratios, and computes their standard errors, design effects, confidence intervals, and hypothesis tests.

CSTabulate. Frequency tables and crosstabulations, and associated standard errors, design effects, confidence intervals, and hypothesis tests.

CSGLM. Linear regression analysis, and analysis of variance and covariance.

CSLogistic. Logistic regression analysis on a binary or multinomial dependent variable using the generalized link function.

CSOrdinal. Fits a cumulative odds model to an ordinal dependent variable for data that have been collected according to a complex sampling design.

Neural Networks

MLP. Fits flexible predictive model for one or more target variables, which can be categorical or scale, based upon the values of factors and covariates.

RBF. Fits flexible predictive model for one or more target variables, which can be categorical or scale, based upon the values of factors and covariates. Generally trains faster than MLP at the slight cost of some model flexibility.

Forecasting

Season. Estimates multiplicative or additive seasonal factors.

Spectra. Periodogram and spectral density function estimates for one or more series.

Tsapply. Loads existing time series models from an external file and applies them to data.

Tsmodel. Estimates exponential smoothing, univariate Autoregressive Integrated Moving Average (ARIMA), and multivariate ARIMA (or transfer function models) models for time series, and produces forecasts.

Conjoint

Conjoint. Analyzes score or rank data from full-concept conjoint studies.

Orthoplan. Orthogonal main-effects plan for a full-concept conjoint analysis.

Plancards. Full-concept profiles, or cards, from a plan file for conjoint analysis.

Bootstrapping

Bootstrap. Bootstrapping is an alternative to parametric estimates when the assumptions of those methods are in doubt, or where parametric inference is impossible or requires very complicated formulas for the calculation of standard errors.

Missing Values

Multiple Imputation. Performs multiple imputations of missing values. Many other procedures can analyze a multiply-imputed dataset to produce pooled results which are more accurate than the singly-imputed datasets produced by MVA.

MVA. Missing Value Analysis. Describes missing value patterns and estimates (imputes) missing values.

Data Preparation

ADP. Automatically prepares data for modeling.

Detectanomaly. Searches for unusual cases based on deviations from the norms of their cluster groups.

Validatedata. Identifies suspicious and invalid cases, variables, and data values in the active dataset.

Optimal Binning. Discretizes scale "binning input" variables to produce categories that are "optimal" with respect to the relationship of each binning input variable with a specified categorical guide variable.

Release History

This section details changes to the command syntax language occurring after release 12.0. Information is organized alphabetically by command and changes for a given command are grouped by release. For commands introduced after 12.0, the introductory release is noted. Additions of new functions (used for instance with COMPUTE) and changes to existing functions are detailed under the heading *Functions*, located at the end of this section.

ADD FILES

Release 22.0

- PASSWORD keyword introduced on the FILE subcommand.

ADP

Release 18

- Command introduced.

AGGREGATE

Release 13.0

- MODE keyword introduced.
- OVERWRITE keyword introduced.

Release 17.0

- AGGREGATE runs without a break variable.

Release 22.0

- CLT, CGT, CIN, and COUT functions introduced.

ALTER TYPE

Release 16.0

- Command introduced.

APPLY DICTIONARY

Release 14.0

- ATTRIBUTES keyword introduced on FILEINFO and VARINFO subcommands.

Release 18

- ROLE keyword introduced on VARINFO subcommands.

Release 22.0

- PASSWORD keyword introduced on the FROM subcommand.

AUTORECODE

Release 13.0

- BLANK subcommand introduced.

- GROUP subcommand introduced.
- APPLY TEMPLATE and SAVE TEMPLATE subcommands introduced.

BEGIN EXPR - END EXPR

Release 21.0

- Command block introduced as SIMPREP BEGIN-SIMPREP END.

Release 23.0

- SIMPREP BEGIN-SIMPREP END deprecated. Command block renamed to BEGIN EXPR-END EXPR.

BEGIN GPL

Release 14.0

- Command introduced.

BEGIN PROGRAM

Release 14.0

- Command introduced.

BOOTSTRAP

Release 18

- Command introduced.

CASEPLOT

Release 14.0

- For plots with one variable, new option to specify a value with the REFERENCE keyword on the FORMAT subcommand.

CATPCA

Release 13.0

- NDIM keyword introduced on PLOT subcommand.
- The maximum label length on the PLOT subcommand is increased to 64 for variable names, 255 for variable labels, and 60 for value labels (previous value was 20).

Release 23.0

- RANDIMPU keyword introduced on MISSING subcommand.
- ROTATION subcommand introduced.
- RESAMPLE subcommand introduced.
- SORT and NOSORT keywords introduced for LOADING on the PRINT subcommand.
- VAF, OBELLAREA, LDELLAREA, CTELLAREA, NELLPNT, and keywords introduced on PLOT subcommand.
- OBELLAREA, LDELLAREA, and CTELLAREA keywords introduced on SAVE subcommand.
- ELLCOORD keyword introduced on OUTFILE subcommand.

CATREG

Release 13.0

- The maximum category label length on the PLOT subcommand is increased to 60 (previous value was 20).

Release 17.0

- MULTISTART and FIXSIGNS keywords added to INITIAL subcommand.
- REGULARIZATION subcommand added.

- RESAMPLE subcommand added.
- REGU keyword added to PRINT subcommand.
- REGU keyword added to PLOT subcommand.
- SUPPLEMENTARY categories not occuring in data used to create the model are now interpolated.

CD

Release 13.0

- Command introduced.

CODEBOOK

Release 17.0

- Command introduced.

Release 18

- ROLE keyword added to VARINFO subcommand.

COMPARE DATASETS

Release 21

- Command introduced.

Release 22.0

- PASSWORD keyword introduced on the COMPDATASET subcommand.
- MATCHPASS, MISMATCHPASS, and ENCRYPTEDPW keywords introduced on the SAVE subcommand.

CORRESPONDENCE

Release 13.0

- For the NDIM keyword on the PLOT subcommand, the default is changed to all dimensions.
- The maximum label length on the PLOT subcommand is increased to 60 (previous value was 20).

CROSSTABS

Release 19.0

- HIDESMALLCOUNTS subcommand introduced.
- SHOWDIM subcommand introduced.
- PROP and BPROP keywords introduced on the CELLS subcommand.

CSGLM

Release 13.0

- Command introduced.

CSLOGISTIC

Release 13.0

- Command introduced.

Release 17.0

- Added support for SET THREADS.

CSORDINAL

Release 15.0

- Command introduced.

Release 17.0

- Added support for SET THREADS.

CTABLES

Release 13.0

- HSUBTOTAL keyword introduced on the CATEGORIES subcommand.

Release 14.0

- INCLUDEMRSETS keyword introduced on the SIGTEST and COMPARETEST subcommands.
- CATEGORIES keyword introduced on the SIGTEST and COMPARETEST subcommands.
- MEANSVARIANCE keyword introduced on the COMPARETEST subcommand.

Release 18.0

- MERGE keyword introduced on the COMPARETEST subcommand.
- PCOMPUTE and PPROPERTIES subcommands introduced.

Release 19.0

- HIDESMALLCOUNTS subcommand introduced.

Release 24.0

- SHOWSIG keyword added to COMPARETEST subcommand.
- CRITERIA subcommand introduced.
- ADJUST=BH added to COMPARETEST subcommand for Benjamini-Hochberg correction.
- STYLE keyword added to COMPARETEST subcommand.
- Confidence intervals added for counts, percents, mean, median, and sum statistics.
- WEIGHT subcommand introduced.

CURVEFIT

Release 19.0

- TEMPLATE subcommand introduced.

DATA LIST

Release 16.0

- ENCODING subcommand added for Unicode support.

DATAFILE ATTRIBUTE

Release 14.0

- Command introduced.

DATASET ACTIVATE

Release 14.0

- Command introduced.

DATASET CLOSE

Release 14.0

- Command introduced.

DATASET COPY

Release 14.0

- Command introduced.

DATASET DECLARE

Release 14.0

- Command introduced.

DATASET DISPLAY

Release 14.0

- Command introduced.

DATASET NAME

Release 14.0

- Command introduced.

DEFINE-!ENDDEFINE

Release 14.0

- For syntax processed in interactive mode, modifications to the macro facility may affect macro calls occurring at the end of a command. See the topic “Overview ” on page 556 for more information.

DETECTANOMALY

Release 14.0

- Command introduced.

DISPLAY

Release 14.0

- ATTRIBUTES keyword introduced.

Release 15.0

- @ATTRIBUTES keyword introduced.

DO REPEAT-END REPEAT

Release 14.0

- ALL keyword introduced.

EXTENSION

Release 16.0

- Command introduced.

FILE HANDLE

Release 13.0

- The NAME subcommand is modified to accept a path and/or file.

Release 16.0

- ENCODING subcommand added for Unicode support.

FILE TYPE

Release 16.0

- ENCODING subcommand added for Unicode support.

GENLIN

Release 15.0

- Command introduced.

Release 16.0

- Added multinomial and tweedie distributions; added MLE estimation option for ancillary parameter of negative binomial distribution (MODEL subcommand, DISTRIBUTION keyword). Notes related to the addition of the new distributions added throughout.
- Added cumulative Cauchit, cumulative complementary log-log, cumulative logit, cumulative negative log-log, and cumulative probit link functions (MODEL subcommand, LINK keyword).
- Added likelihood-ratio chi-square statistics as an alternative to Wald statistics (CRITERIA subcommand, ANALYSISTYPE keyword).
- Added profile likelihood confidence intervals as an alternative to Wald confidence intervals (CRITERIA subcommand, CITYPE keyword).
- Added option to specify initial value for ancillary parameter of negative binomial distribution (CRITERIA subcommand, INITIAL keyword).
- Changed default display of the likelihood function for GEEs to show the full value instead of the kernel (CRITERIA subcommand, LIKELIHOOD keyword).

GENLINMIXED

Release 19

- Command introduced.

Release 20

- Ordinal targets can be analyzed using the Multinomial distribution and the complementary log-log, cauchit, logit, negative log-log, or probit link functions.

GET CAPTURE

Release 15.0

- UNENCRYPTED subcommand introduced.

GET DATA

Release 13.0

- ASSUMEDSTRWIDTH subcommand introduced for TYPE=ODBC.

Release 14.0

- ASSUMEDSTRWIDTH subcommand extended to TYPE=XLS.
- TYPE=OLEDB introduced.

Release 15.0

- ASSUMEDSTRWIDTH subcommand extended to TYPE=OLEDB.

Release 16.0

- TYPE=XLSX and TYPE=XLSM introduced.

Release 17.0

- ENCRYPTED subcommand introduced.

Release 21.0

- ENCODING subcommand introduced.

Release 23.0

- UTF16, UTF16BE, and UTF16LE keywords added to ENCODING subcommand.

Release 24.0

- AUTO keyword introduced.
- DATATYPEMIN subcommand introduced.
- HIDDEN subcommand introduced.

- LEADINGSPACES subcommand introduced.
- MAP subcommand introduced.
- MULTIPLESPACES subcommand introduced.
- TRAILINGSPACES subcommand introduced.

GET SAS

Release 19

- ENCODING subcommand introduced.

GET STATA

Release 14.0

- Command introduced.

Release 19

- ENCODING subcommand introduced.

GETCOGNOS

Release 21.0

- Command introduced.

Release 23.0

- CREDENTIAL keyword introduced on CONNECTION subcommand.
- Value STOREDCREDENTIAL added to MODE keyword on CONNECTION subcommand.

GETTM1

Release 22.0.0.1

- Command introduced.

Release 23.0

- MODE and CREDENTIAL keywords introduced on CONNECTION subcommand.

GGRAPH

Release 14.0

- Command introduced.

Release 15.0

- RENAME syntax qualifier deprecated.
- COUNTCI, MEDIANCI, MEANCI, MEANSD, and MEANSE functions introduced.

Release 17.0

- Added SOURCE=VIZTEMPLATE to support visualization templates.
- Added VIZSTYLESHEET keyword to support visualization stylesheets.

Release 19.0

- Added LOCATION=FILE to support visualization templates stored in an arbitrary location on the file system.

Release 20.0

- Added VIZMAP keyword to support map visualizations.

GLM

Release 17.0

- POSTHOC subcommand: T2, T3, GH, and C keywords are not valid when multiple factors in the model.

- PLOT subcommand: new WITH keyword allows you to fix covariate values for profile plots.

Release 25.0

- TYPE, ERRORBAR, MEANREFERENCE, and YAXIS keywords added to PLOT subcommand.
- ROBUST subcommand added.
- HCCOVB keyword added to OUTFILE subcommand. Any combination of COVB, CORB, and HCCOVB keywords can now be specified on OUTFILE.
- MBP, BP, F, and WHITE keywords added to PRINT subcommand.
- MBPDESIGN, BPDESIGN, and FDESIGN subcommands added.

GRAPH

Release 13.0

- PANEL subcommand introduced.
- INTERVAL subcommand introduced.

HOST

Release 13.0

- Command introduced.

INCLUDE

Release 16.0

- ENCODING keyword added for Unicode support.

Release 22.0

- PASSWORD keyword introduced on the FILE subcommand.

INSERT

Release 13.0

- Command introduced.

Release 16.0

- ENCODING keyword added for Unicode support.

Release 22.0

- PASSWORD keyword introduced.

KEYED DATA LIST

Release 16.0

- ENCODING subcommand added for Unicode support.

KNN

Release 17.0

- Command introduced.

LINEAR

Release 19

- Command introduced.

LOGISTIC REGRESSION

Release 13.0

- OUTFILE subcommand introduced.

Release 14.0

- Modification to the method of recoding string variables. See the topic “Overview ” on page 990 for more information.

MATCH FILES

Release 22.0

- PASSWORD keyword introduced on the FILE and TABLE subcommands.

MISSING VALUES

Release 16.0

- Limitation preventing assignment of missing values to strings with a defined width greater than eight bytes removed.

MLP

Release 16.0

- Command introduced.

MODEL CLOSE

Release 13.0

- Command introduced.

MODEL HANDLE

Release 13.0

- Command introduced.

MODEL LIST

Release 13.0

- Command introduced.

MRSETS

Release 14.0

- LABELSOURCE keyword introduced on MDGROUP subcommand.
- CATEGORYLABELS keyword introduced on MDGROUP subcommand.

MULTIPLE CORRESPONDENCE

Release 13.0

- Command introduced.

MULTIPLE IMPUTATION

Release 17.0

- Command introduced.

NAIVEBAYES

Release 14.0

- Command introduced.

NOMREG

Release 13.0

- ENTRYMETHOD keyword introduced on STEPWISE subcommand.
- REMOVALMETHOD keyword introduced on STEPWISE subcommand.
- IC keyword introduced on PRINT subcommand.

Release 15.0

- ASSOCIATION keyword introduced on PRINT subcommand.

Release 17.0

- Added support for SET THREADS and SET MCACHE.

NONPARAMETRIC

Release 18

- Command introduced.

NPAR TESTS

Release 17.0

- Increased limits on number of variables allowed in the analysis.

OLAP CUBES

Release 19.0

- HIDESMALLCOUNTS subcommand introduced.

OMS

Release 13.0

- TREES keyword introduced on SELECT subcommand.
- IMAGES, IMAGEROOT, CHARTSIZE, and IMAGEFORMAT keywords introduced on DESTINATION subcommand.

Release 14.0

- XMLWORKSPACE keyword introduced on DESTINATION subcommand.

Release 16.0

- IMAGEFORMAT=VML introduced for FORMAT=HTML on DESTINATION subcommand.
- IMAGEMAP keyword introduced for FORMAT=HTML on DESTINATION subcommand.
- FORMAT=SPV introduced for saving output in Viewer format.
- CHARTFORMAT keyword introduced.
- TREEFORMAT keyword introduced.
- TABLES keyword introduced.
- FORMAT=SVWSOXML is no longer supported.

Release 17.0

- MODELS keyword introduced on SELECT subcommand.
- FORMAT=DOC, XLS, PDF, and SPW introduced.
- MODELFORMAT keyword introduced.

Release 19.0

- IMAGEFORMAT=VML introduced for FORMAT=OXML on DESTINATION subcommand.
- For version 19.0.0.1 and higher, the IMAGEMAP keyword will no longer generate image map tooltips for major tick labels.

Release 21.0

- FORMAT=XLSX added to DESTINATION subcommand.

Release 22.0

- FORMAT=REPORTHTML and FORMAT=REPORTMHT added to DESTINATION subcommand.

- REPORTTITLE keyword added to DESTINATION subcommand.

Release 27.0

- FORMAT=REPORTHTML and FORMAT=REPORTMHT deprecated from the DESTINATION subcommand.
- REPORTTITLE keyword deprecated from the DESTINATION subcommand.

ONEWAY

Release 19.0

- TEMPLATE subcommand introduced.

OPTIMAL BINNING

Release 15.0

- Command introduced.

OUTPUT ACTIVATE

Release 15.0

- Command introduced.

OUTPUT CLOSE

Release 15.0

- Command introduced.

OUTPUT DISPLAY

Release 15.0

- Command introduced.

OUTPUT EXPORT

Release 17.0

- Command introduced.

Release 21.0

- Subcommands XLSX and XLSM added.
- STYLING keyword added to HTML subcommand.
- BREAKPOINTS keyword added to DOC subcommand.

Release 22.0.

- Subcommand REPORT added.
- INTERACTIVELAYERS keyword added to HTML subcommand

Release 27.0

- The REPORT subcommand was deprecated. The REPORT subcommand syntax has been mapped to the HTML subcommand.

OUTPUT NAME

Release 15.0

- Command introduced.

OUTPUT MODIFY

Release 22.0

- Command introduced.

OUTPUT NEW

Release 15.0

- Command introduced.

Release 16.0

- TYPE keyword is obsolete and is ignored.

OUTPUT OPEN

Release 15.0

- Command introduced.

Release 17.0

- LOCK keyword introduced.

Release 21.0

- PASSWORD keyword introduced.

OUTPUT SAVE

Release 15.0

- Command introduced.

Release 16.0

- TYPE keyword introduced.

Release 17.0

- LOCK keyword introduced.

Release 21.0

- PASSPROTECT subcommand introduced.

PER ATTRIBUTES

Release 16.0

- Command introduced as PER ATTRIBUTES.

Release 17.0

- VERSIONLABEL keyword extended to support multiple labels.

Release 18.0

- PER ATTRIBUTES deprecated. Command name changed to REPOSITORY ATTRIBUTES.

PER CONNECT

Release 15.0

- Command introduced as PER CONNECT.

Release 17.0

- DOMAIN keyword deprecated on the LOGIN subcommand.
- PROVIDER keyword introduced on the LOGIN subcommand.

Release 18.0

- PER CONNECT deprecated. Command name changed to REPOSITORY CONNECT.

PER COPY

Release 16.0

- Command introduced as PER COPY.

Release 18.0

- PER COPY deprecated. Command name changed to REPOSITORY COPY.

PLANCARDS

Release 14.0

- PAGINATE subcommand is obsolete and no longer supported.

PLS

Release 16.0

- Command introduced.

POINT

Release 16.0

- ENCODING subcommand added for Unicode support.

PPLOT

Release 19.0

- TEMPLATE subcommand introduced.

PREFSCAL

Release 14.0

- Command introduced.

PRINT

Release 16.0

- ENCODING subcommand added for Unicode support.

PRINT EJECT

Release 16.0

- ENCODING subcommand added for Unicode support.

PRINT SPACE

Release 16.0

- ENCODING subcommand added for Unicode support.

RBF

Release 16.0

- Command introduced.

REGRESSION

Release 13.0

- PARAMETER keyword introduced on OUTFILE subcommand.

Release 16.0

- Added support for SET THREADS and SET MCACHE.

Release 17.0

- Added option to specify confidence level on CI keyword of STATISTICS subcommand.

Release 19.0

- TEMPLATE subcommand introduced.

RELIABILITY

Release 17.0

- Increased limits on numbers of variables allowed on the VARIABLES and SCALE lists.

REPEATING DATA

Release 16.0

- ENCODING subcommand added for Unicode support.

REPOSITORY ATTRIBUTES

Release 16.0

- Command introduced as PER ATTRIBUTES.

Release 17.0

- VERSIONLABEL keyword extended to support multiple labels.

Release 18.0

- PER ATTRIBUTES deprecated. Command name changed to REPOSITORY ATTRIBUTES.

REPOSITORY CONNECT

Release 15.0

- Command introduced as PER CONNECT.

Release 17.0

- DOMAIN keyword deprecated on the LOGIN subcommand.
- PROVIDER keyword introduced on the LOGIN subcommand.

Release 18.0

- PER CONNECT deprecated. Command name changed to REPOSITORY CONNECT.

REPOSITORY COPY

Release 16.0

- Command introduced as PER COPY.

Release 18.0

- PER COPY deprecated. Command name changed to REPOSITORY COPY.

RESPONSE RATE

Release 18.0

- Command introduced.

ROC

Release 18.0

- MODELQUALITY keyword introduced.

SAVE

Release 21.0

- ZCOMPRESSED subcommand added.
- PASSPROTECT subcommand added.

SAVE CODEPAGE

Release 23.0

- Command introduced.

SAVE DATA COLLECTION

Release 15.0

- Command introduced as SAVE DIMENSIONS.

Release 18.0

- SAVE DIMENSIONS deprecated. Command name changed to SAVE DATA COLLECTION.

SAVE TRANSLATE

Release 14.0

- Value STATA added to list for TYPE subcommand.
- EDITION subcommand introduced for TYPE=STATA.
- SQL subcommand introduced.
- MISSING subcommand introduced.
- Field/column names specified on the RENAME subcommand can contain characters (for example, spaces, commas, slashes, plus signs) that are not allowed in IBM SPSS Statistics variable names.
- Continuation lines for connection strings on the CONNECT subcommand do not need to begin with a plus sign.

Release 15.0

- ENCRYPTED subcommand introduced.
- Value CSV added to list for TYPE subcommand.
- TEXTOPTIONS subcommand introduced for TYPE=CSV and TYPE=TAB.

Release 16.0

- VERSION=12 introduced for writing data in Excel 2007 XLSX format with TYPE=XLS.

Release 17.0

- UNENCRYPTED subcommand introduced.

Release 18.0

- VERSION=9 introduced for writing SAS 9+ files with TYPE=SAS.

Release 19

- ENCODING subcommand introduced.

Release 22.0

- BOM keyword added to ENCODING subcommand.

Release 23.0

- Support for versions 9-13 of Stata added to VERSION subcommand.
- BULKLOADING subcommand added.

Release 24.0

- VALUE keyword added to FIELDNAMES subcommand.
- ODBCOPTIONS subcommand added.
- EXCELOPTIONS subcommand added.

Release 25.0

- Support for version 14 of Stata added to VERSION subcommand. Stata 14 supports Unicode data.

SAVETM1

Release 22.0.0.1

- Command introduced.

SCRIPT

Release 16.0

- Scripts run from the SCRIPT command now run synchronously with the command syntax stream.

Release 17.0

- Ability to run Python scripts introduced.

SELECTPRED

Release 14.0

- Command introduced.

SET

Release 13.0

- RNG and MTINDEX subcommands introduced.
- Default for MXERRS subcommand increased to 100.
- SORT subcommand introduced.
- LOCALE subcommand introduced.

Release 14.0

- Default for WORKSPACE subcommand increased to 6144.

Release 15.0

- LABELS replaces VALUES as the default for the TNUMBERS subcommand.
- JOURNAL subcommand is obsolete and no longer supported.
- Value EXTERNAL added to list for SORT subcommand, replacing the value INTERNAL as the default. Value SS is deprecated.

Release 16.0

- MCACHE subcommand introduced.
- THREADS subcommand introduced.
- UNICODE subcommand introduced.

Release 16.0.1

- BOTHLARGE keyword introduced for the TFIT subcommand.

Release 17.0

- FUZZBITS subcommand introduced.
- MIOUTPUT subcommand introduced.

Release 18.0

- ROWSBREAK, CELLSBREAK, and TOLERANCE subcommands introduced for controlling display of large pivot tables.
- ZCOMPRESSION subcommand introduced.
- COMPRESSION subcommand is obsolete and ignored.
- REPDEFER subcommand introduced.

Release 19.0

- XVERSION subcommand introduced.
- OATTRS subcommand introduced.
- DIGITGROUPING subcommand introduced.
- TABLERENDER subcommand introduced.
- CMPTRANS subcommand introduced.

Release 20.0

- FAST keyword introduced for the TABLERENDER subcommand, replacing the LIGHT keyword, which is deprecated.
- Value BPortugu (Brazilian Portuguese) added to list for OLANG subcommand.

Release 21.0

- ODISPLAY subcommand introduced.

Release 22.0

- OSLOCALE keyword added to LOCALE subcommand.
- BASETEXTDIRECTION subcommand added.
- SUMMARY subcommand added.

Release 24.0

- LEADZERO subcommand added.

Release Subscription (November 2017 update)

- MDISPLAY subcommand added.

Release 27.0.1

- FOOTNOTE and SIGLESS subcommands added.

SHIFT VALUES

Release 17.0

- Command introduced.

SHOW

Release 13.0

- BLKSIZE and BUFNO subcommands are obsolete and no longer supported.
- HANDLES subcommand introduced.
- SORT subcommand introduced.

Release 15.0

- TMSRECORDING subcommand introduced.

Release 16.0

- UNICODE subcommand introduced.
- MCACHE subcommand introduced.
- THREADS subcommand introduced.

Release 17.0

- FUZZBITS subcommand introduced.

Release 18.0

- EXTPATHS subcommand introduced.
- ZCOMPRESSION subcommand introduced.
- COMPRESSION subcommand removed because it is obsolete.
- REPDEFER subcommand introduced.

Release 19.0

- TABLERENDER subcommand introduced.
- XVERSION subcommand introduced.

- OATTRS subcommand introduced.
- DIGITGROUPING subcommand introduced.
- CMPTRANS subcommand introduced.

Release 21.0

- ODISPLAY subcommand introduced.

Release 22.0

- PLUGINS subcommand introduced.

Release 24.0

- LEADZERO subcommand introduced.

Release Subscription (November 2017 update)

- MDISPLAY subcommand introduced.

SIMPLAN

Release 21.0

- Command introduced.

Release 22.0

- LOCK keyword introduced on FIXEDINPUT subcommand.
- CONTINGENCY subcommand added.
- CONTINGENCY keyword added to specifications for CATEGORICAL distribution on SIMINPUT subcommand.
- Added global SOURCE keyword and deprecated SOURCE keyword for DISTRIBUTION=EMPIRICAL.
- MISSING subcommand added.
- VALUELABELS subcommand added.

SIMPREP BEGIN-SIMPREP END

Release 21.0

- Command introduced.

Release 23.0

- Command block deprecated for release 23.0 and higher. Name of command block changed to BEGIN EXPR-END EXPR.

SIMRUN

Release 21.0

- Command introduced.

Release 22.0

- Added support for saving the simulated data to the active dataset by specifying an asterisk (*) on the FILE keyword of the OUTFILE subcommand.
- REFLINES keyword added to DISTRIBUTION subcommand.
- ASSOCIATIONS keyword added to PRINT subcommand.
- OPTIONS subcommand added.

SORT VARIABLES

Release 16.0.

- Command introduced.

Release 18.0.

- ROLE keyword introduced.

SPATIAL ASSOCIATION RULES

Release 23.0

- Command introduced.

SPATIAL MAPSPEC

Release 23.0

- Command introduced.

SPATIAL TEMPORAL PREDICTION

Release 23.0

- Command introduced.

SPCHART

Release 15.0

- (XBARONLY) keyword introduced on XR and XS subcommands.
- RULES subcommand introduced.
- ID subcommand introduced.

Release 19.0

- CPCHART subcommand introduced.
- NORMAL subcommand introduced.
- REFERENCE subcommand introduced.
- Following keywords introduced on STATISTICS subcommand: N, MEAN, STDDEV, CAPSIGMA, LSL, USL, TARGET, AZLOUT, AZUOUT, CZLOUT, CZUOUT, PZLOUT, PZUOUT.

STAR JOIN

Release 21.0

- Command introduced.

Release 22.0

- PASSWORD keyword introduced on the FROM and JOIN subcommands.

SYSFILE INFO

Release 22.0

- PASSWORD keyword introduced.

TCM ANALYSIS

Release 23.0

- Command introduced.

TCM APPLY

Release 23.0

- Command introduced.

TCM MODEL

Release 23.0

- Command introduced.

TMS BEGIN

Release 15.0

- Command introduced.

Release 16.0

- Added support for new string functions CHAR.CONCAT, CHAR.LENGTH, and CHAR.SUBSTR within TMS blocks.

Release 21.0

- Added support for comparison operators and logical operators.

TMS END

Release 15.0

- Command introduced.

TMS IMPORT

Release 18

- Command introduced.

TMS MERGE

Release 15.0

- Command introduced.

TREE

Release 13.0

- Command introduced.

Release 18.0

- TARGETRESPONSE subcommand introduced.

TSAPPLY

Release 14.0

- Command introduced.

TSMODEL

Release 14.0

- Command introduced.

TSPLOT

Release 14.0

- For plots with one variable, REFERENCE keyword modified to allow specification of a value.

UNIANOVA

Release 17.0

- POSTHOC subcommand: T2, T3, GH, and C keywords are not valid when multiple factors in the model.

Release 25.0

- TYPE, ERRORBAR, MEANREFERENCE, and YAXIS keywords added to PLOT subcommand.
- ROBUST subcommand added.
- HCCOVB keyword added to OUTFILE subcommand. Any combination of COVB, CORB, and HCCOVB keywords can now be specified on OUTFILE.
- MBP, BP, F, and WHITE keywords added to PRINT subcommand.
- MBPDESIGN, BPDESIGN, and FDESIGN subcommands added.

UPDATE

Release 22.0

- PASSWORD keyword introduced on the FILE subcommand.

VALIDATEDATA

Release 14.0

- Command introduced.

VALUE LABELS

Release 14.0

- The maximum length of a value label is extended to 120 bytes (previous limit was 60 bytes).

Release 16.0

- Limitation preventing assignment of missing values to strings with a defined width greater than eight bytes removed.

VARIABLE ATTRIBUTE

Release 14.0

- Command introduced.

VARIABLE ROLE

Release 18.0

- Command introduced.

WRITE

Release 16.0

- ENCODING subcommand added for Unicode support.

Release 22.0

- BOM keyword added.

XGRAPH

Release 13.0

- Command introduced.

XSAVE

Release 21.0

- ZCOMPRESSED subcommand added.

Functions

Release 13.0

- APPLYMODEL and STRAPPLYMODEL functions introduced.
- DATEDIFF and DATESUM functions introduced.

Release 14.0

- REPLACE function introduced.
- VALUELABEL function introduced.

Release 16.0

- CHAR.INDEX function introduced.
- CHAR.LENGTH function introduced.

- CHAR.LPAD function introduced.
- CHAR.MBLEN function introduced.
- CHAR.RINDEX function introduced.
- CHAR.RPAD function introduced.
- CHAR.SUBSTR function introduced.
- NORMALIZE function introduced.
- NTRIM function introduced.
- STRUNC function introduced.

Release 17.0

- MEDIAN function introduced.
- *mult* and *fuzzbits* arguments introduced for the RND and TRUNC functions.
- NEIGHBOR and DISTANCE functions added to APPLYMODEL and STRAPPLYMODEL.

Extension Commands

In addition to the commands available in the Core system and add-on modules, there are numerous extension commands available for use with IBM SPSS Statistics. Extension commands are IBM SPSS Statistics commands that are implemented in the Python®, R, or Java programming language. For example, IBM SPSS Statistics - Essentials for Python, which is installed by default with IBM SPSS Statistics, includes a set of Python extension commands that are installed with SPSS Statistics. And IBM SPSS Statistics - Essentials for R, which is on GitHub at https://github.com/IBMPredictiveAnalytics/R_Essentials_Statistics/releases, includes a set of extension commands that are implemented in the R programming language. Many more extension commands are hosted on the SPSS Statistics collection on GitHub and available from the Extension Hub, which is accessed from **Extensions** > **Extension Hub**.

By convention, extension commands that are authored by IBM Corp. have names that begin with SPSSINC or STATS. Complete syntax help for each of the extension commands is available by positioning the cursor within the command (in a syntax window) and pressing the F1 key. It is also available by running the command and including the /HELP subcommand. For example:

```
STATS TABLE CALC /HELP.
```

The command syntax help is not, however, integrated with the SPSS Statistics Help system and is not included in the *Command Syntax Reference*. Extension commands that are not authored by IBM Corp. might follow the convention of providing documentation with the HELP subcommand or the F1 key.

Note: The F1 mechanism for displaying help is not supported in distributed mode.

Extension commands require the IBM SPSS Statistics Integration Plug-in(s) for the language(s) in which the command is implemented; Python, R, or Java. For information, see How to Get Integration Plug-ins, available from Core System > Frequently Asked Questions in the Help system.

Note: The IBM SPSS Statistics - Integration Plug-in for Java™ is installed as part of IBM SPSS Statistics and does not require separate installation.

Information on writing your own extension commands is available from the following sources:

- The article *"Writing IBM SPSS Statistics Extension Commands"*, available from the IBM SPSS Statistics community at https://www.ibm.com/community/spss-statistics .
- The chapter on Extension Commands in *Programming and Data Management for IBM SPSS Statistics*, which is also available from the IBM SPSS Statistics community.

Universals

This part of the *Command Syntax Reference* discusses general topics pertinent to using command syntax. The topics are divided into five sections:

- Commands explains command syntax, including command specification, command order, and running commands in different modes. In this section, you will learn how to read syntax charts, which summarize command syntax in diagrams and provide an easy reference. Discussions of individual commands are found in an alphabetical reference in the next part of this manual.
- Files discusses different types of files used by the program. Terms frequently mentioned in this manual are defined. This section provides an overview of how files are handled.
- Variables and Variable Types and Formats contain important information about general rules and conventions regarding variables and variable definition.
- Transformations describes expressions that can be used in data transformation. Functions and operators are defined and illustrated. In this section, you will find a complete list of available functions and how to use them.

Commands

Commands are the instructions that you give the program to initiate an action. For the program to interpret your commands correctly, you must follow certain rules.

Syntax Diagrams

Each command described in this manual includes a syntax diagram that shows all of the subcommands, keywords, and specifications allowed for that command. By recognizing symbols and different type fonts, you can use the syntax diagram as a quick reference for any command.

- Lines of text in italics indicate limitation or operation mode of the command.
- Elements shown in upper case are keywords to identify commands, subcommands, functions, operators, and other specifications. In the sample syntax diagram below, T-TEST is the command and GROUPS is a subcommand.
- Elements in lower case describe specifications that you supply. For example, `varlist` indicates that you need to supply a list of variables.
- Elements in bold are defaults. There are two types of defaults. When the default is followed by **, as ANALYSIS** is in the sample syntax diagram below, the default (ANALYSIS) is in effect if the subcommand (MISSING) is not specified. If a default is not followed by **, it is in effect when the subcommand (or keyword) is specified by itself.
- Parentheses, apostrophes, and quotation marks are required where indicated.
- Unless otherwise noted, elements enclosed in square brackets (`[ ]`) are optional. For some commands, square brackets are part of the required syntax. The command description explains which specifications are required and which are optional.
- Braces (`{ }`) indicate a choice between elements. You can specify any one of the elements enclosed within the aligned braces.
- Ellipses indicate that you can repeat an element in the specification. The specification

  ```
  T-TEST PAIRS=varlist [WITH varlist [(PAIRED)]] [/varlist ...]
  ```

  means that you can specify multiple variable lists with optional WITH variables and the keyword PAIRED in parentheses.
- Most abbreviations are obvious; for example, `varname` stands for variable name and `varlist` stands for a variable list.
- The command terminator is not shown in the syntax diagram.

Command Specification

The following rules apply to all commands:

- Commands begin with a keyword that is the name of the command and often have additional specifications, such as subcommands and user specifications. Refer to the discussion of each command to see which subcommands and additional specifications are required.
- Commands and any command specifications can be entered in upper and lower case. Commands, subcommands, keywords, and variable names are translated to upper case before processing. All user specifications, including variable names, labels, and data values, preserve upper and lower case.
- Spaces can be added between specifications at any point where a single blank is allowed. In addition, lines can be broken at any point where a single blank is allowed. There are two exceptions: the END DATA command can have only one space between words, and string specifications on commands such as TITLE, SUBTITLE, VARIABLE LABELS, and VALUE LABELS can be broken across two lines only by specifying a plus sign (+) between string segments. See the topic "String Values in Command Specifications" on page 35 for more information.
- Many command names and keywords can be abbreviated to the first three or more characters that can be resolved without ambiguity. For example, COMPUTE can be abbreviated to COMP but not COM because the latter does not adequately distinguish it from COMMENT. Some commands, however, require that all specifications be spelled out completely. This restriction is noted in the syntax chart for those commands.

Running Commands

You can run commands in either batch (production) or interactive mode. In batch mode, commands are read and acted upon as a batch, so the system knows that a command is complete when it encounters a new command. In interactive mode, commands are processed immediately, and you must use a command terminator to indicate when a command is complete.

Interactive Mode

The following rules apply to command specifications in interactive mode:

- Each command must start on a new line. Commands can begin in any column of a command line and continue for as many lines as needed. The exception is the END DATA command, which must begin in the first column of the first line after the end of data.
- Each command should end with a period as a command terminator. It is best to omit the terminator on BEGIN DATA, however, so that inline data are treated as one continuous specification.
- The command terminator must be the last nonblank character in a command.
- In the absence of a period as the command terminator, a blank line is interpreted as a command terminator.

Note: For compatibility with other modes of command execution (including command files run with INSERT or INCLUDE commands in an interactive session), each line of command syntax should not exceed 256 characters.

Batch (Production) Mode

The following rules apply to command specifications in batch mode:

- All commands in the command file must begin in column 1. You can use plus (+) or minus (–) signs in the first column if you want to indent the command specification to make the command file more readable.
- If multiple lines are used for a command, column 1 of each continuation line must be blank.
- Command terminators are optional.
- A line cannot exceed 256 characters; any additional characters are truncated.

The following is a sample command file that will run in either interactive or batch mode:

```
GET FILE=/MYFILES/BANK.SAV'
  /KEEP ID TIME SEX JOBCAT SALBEG SALNOW
```

```
  /RENAME SALNOW = SAL90.

DO IF TIME LT 82.
+  COMPUTE RATE=0.05.
ELSE.
+  COMPUTE RATE=0.04.
END IF.

COMPUTE SALNOW=(1+RATE)*SAL90.

EXAMINE VARIABLES=SALNOW BY SEX.
```

Subcommands

Many commands include additional specifications called **subcommands**.

- Subcommands begin with a keyword that is the name of the subcommand. Most subcommands include additional specifications.
- Some subcommands are followed by an equals sign before additional specifications. The equals sign is usually optional but is required where ambiguity is possible in the specification. To avoid ambiguity, it is best to use the equals signs as shown in the syntax diagrams in this manual.
- Most subcommands can be named in any order. However, some commands require a specific subcommand order. The description of each command includes a section on subcommand order.
- Subcommands are separated from each other by a slash. To avoid ambiguity, it is best to use the slashes as shown in the syntax diagrams in this manual.

Keywords

Keywords identify commands, subcommands, functions, operators, and other specifications.

- Keywords identifying logical operators (AND, OR, and NOT); relational operators (EQ, GE, GT, LE, LT, and NE); and ALL, BY, TO, and WITH are reserved words and cannot be used as variable names.

Values in Command Specifications

The following rules apply to values specified in commands:

- A single lowercase character in the syntax diagram, such as *n*, *w*, or *d*, indicates a user-specified value.
- The value can be an integer or a real number within a restricted range, as required by the specific command or subcommand. For exact restrictions, read the individual command description.
- A number specified as an argument to a subcommand can be entered with or without leading zeros.

String Values in Command Specifications

- Each string specified in a command should be enclosed in single or double quotes.
- To specify a single quote or apostrophe within a quoted string, either enclose the entire string in double quotes or double the single quote/apostrophe. Both of the following specifications are valid:

```
'Client''s Satisfaction'
"Client's Satisfaction"
```

- To specify double quotes within a string, use single quotes to enclose the string:

```
'Categories Labeled "UNSTANDARD" in the Report'
```

- String specifications can be broken across command lines by specifying each string segment within quotes and using a plus (+) sign to join segments. For example,

```
'One, Two'
```

can be specified as

```
'One,'
+ ' Two'
```

The plus sign can be specified on either the first or the second line of the broken string. Any blanks separating the two segments must be enclosed within one or the other string segment.

- Multiple blank spaces within quoted strings are preserved and can be significant. For example, "This string" and "This string" are treated as different values.

Delimiters

Delimiters are used to separate data values, keywords, arguments, and specifications.

- A blank is usually used to separate one specification from another, except when another delimiter serves the same purpose or when a comma is required.
- Commas are required to separate arguments to functions. Otherwise, blanks are generally valid substitutes for commas.
- Arithmetic operators (+, –, *, and /) serve as delimiters in expressions.
- Blanks can be used before and after operators or equals signs to improve readability, but commas cannot.
- Special delimiters include parentheses, apostrophes, quotation marks, the slash, and the equals sign. Blanks before and after special delimiters are optional.
- The slash is used primarily to separate subcommands and lists of variables. Although slashes are sometimes optional, it is best to enter them as shown in the syntax diagrams.
- The equals sign is used between a keyword and its specifications, as in `STATISTICS=MEAN`, and to show equivalence, as in `COMPUTE target variable=expression`. Equals signs following keywords are frequently optional but are sometimes required. In general, you should follow the format of the syntax charts and examples and always include equals signs wherever they are shown.

Command Order

Command order is more often than not a matter of common sense and follows this logical sequence: variable definition, data transformation, and statistical analysis. For example, you cannot label, transform, analyze, or use a variable in any way before it exists. The following general rules apply:

- Commands that define variables for a session (`DATA LIST`, `GET`, `GET DATA`, `MATRIX DATA`, etc.) must precede commands that assign labels or missing values to those variables; they must also precede transformation and procedure commands that use those variables.
- Transformation commands (`IF`, `COUNT`, `COMPUTE`, etc.) that are used to create and modify variables must precede commands that assign labels or missing values to those variables, and they must also precede the procedures that use those variables.
- Generally, the logical outcome of command processing determines command order. For example, a procedure that creates new variables in the active dataset must precede a procedure that uses those new variables.

In addition to observing the rules above, it is often important to distinguish between commands that cause the data to be read and those that do not, and between those that are stored pending execution with the next command that reads the data and those that take effect immediately without requiring that the data be read.

- Commands that cause the data to be read, as well as execute pending transformations, include all statistical procedures (e.g., `CROSSTABS`, `FREQUENCIES`, `REGRESSION`); some commands that save/write the contents of the active dataset (e.g., `DATASET COPY`, `SAVE TRANSLATE`, `SAVE`); `AGGREGATE`; `AUTORECODE`; `EXECUTE`; `RANK`; and `SORT CASES`.
- Commands that are stored, pending execution with the next command that reads the data, include transformation commands that modify or create new data values (e.g., `COMPUTE`, `RECODE`), commands that define conditional actions (e.g., `DO IF`, `IF`, `SELECT IF`), `PRINT`, `WRITE`, and `XSAVE`. For a comprehensive list of these commands, see “Commands That Are Stored, Pending Execution” on page 39 .

- Commands that take effect immediately without reading the data or executing pending commands include transformations that alter dictionary information without affecting the data values (e.g., MISSING VALUES, VALUE LABELS) and commands that don't require an active dataset (e.g., DISPLAY, HOST, INSERT, OMS, SET). In addition to taking effect immediately, these commands are also processed unconditionally. For example, when included within a DO IF structure, these commands run regardless of whether or not the condition is ever met. For a comprehensive list of these commands, see “Commands That Take Effect Immediately” on page 37 .

Example

```
DO IF expense = 0.
- COMPUTE profit=-99.
- MISSING VALUES expense (0).
ELSE.
- COMPUTE profit=income-expense.
END IF.
LIST VARIABLES=expense profit.
```

- COMPUTE precedes MISSING VALUES and is processed first; however, execution is delayed until the data are read.
- MISSING VALUES takes effect as soon as it is encountered, *even if the condition is never met* (i.e., even if there are no cases where expense=0).
- LIST causes the data to be read; thus, both COMPUTE and LIST are executed during the same data pass.
- Because MISSING VALUES is already in effect by this time, the first condition in the DO IF structure will never be met, because an *expense* value of 0 is considered missing and so the condition evaluates to missing when *expense* is 0.

Commands That Take Effect Immediately

These commands take effect immediately. They do not read the active dataset and do not execute pending transformations.

Commands That Modify the Dictionary

“ADD DOCUMENT” on page 103
“ADD VALUE LABELS” on page 111
“APPLY DICTIONARY” on page 167
“DATAFILE ATTRIBUTE ” on page 531
“DELETE VARIABLES” on page 571
“DOCUMENT” on page 623
“DROP DOCUMENTS” on page 625
“EXTENSION” on page 663
“FILE LABEL” on page 683
“FORMATS” on page 713
“MISSING VALUES ” on page 1129
“MRSETS” on page 1177
“NUMERIC” on page 1293
“OUTPUT EXPORT” on page 1357
“PRINT FORMATS” on page 1537
“RENAME VARIABLES” on page 1661
“STRING ” on page 1907
“TMS IMPORT” on page 1985
“TMS MERGE” on page 1989
“VALUE LABELS ” on page 2109
“VARIABLE ALIGNMENT ” on page 2119
“VARIABLE ATTRIBUTE” on page 2121
“VARIABLE LABELS” on page 2125

“VARIABLE LEVEL ” on page 2127
“VARIABLE ROLE” on page 2129
“VARIABLE WIDTH ” on page 2131
“WEIGHT” on page 2147
“WRITE FORMATS” on page 2165

Other Commands That Take Effect Immediately

“CD” on page 305
“CLEAR TIME PROGRAM” on page 307
“CLEAR TRANSFORMATIONS ” on page 309
“CSPLAN” on page 451
“DATASET CLOSE” on page 535
“DATASET DECLARE” on page 539
“DATASET DISPLAY” on page 541
“DATASET NAME” on page 543
“DISPLAY” on page 599
“ECHO” on page 627
“ERASE” on page 637
“FILE HANDLE” on page 679
“FILTER” on page 697
“HOST ” on page 905
“INCLUDE” on page 937
“INSERT” on page 945
“MODEL CLOSE” on page 1165
“MODEL HANDLE” on page 1167
“MODEL LIST” on page 1173
“N OF CASES” on page 1223
“NEW FILE ” on page 1233
“OMS” on page 1301
“OMSEND” on page 1327
“OMSINFO” on page 1329
“OMSLOG” on page 1331
“OUTPUT ACTIVATE” on page 1351
“OUTPUT CLOSE” on page 1353
“OUTPUT DISPLAY” on page 1355
“OUTPUT NAME” on page 1391
“OUTPUT NEW” on page 1393
“OUTPUT OPEN” on page 1395
“OUTPUT SAVE” on page 1399
“PERMISSIONS ” on page 1423
“PRESERVE ” on page 1519
“READ MODEL” on page 1615
“RESTORE” on page 1711
“SAVE MODEL” on page 1743
“SCRIPT” on page 1769
“SET” on page 1785
“SHOW ” on page 1809
“SPLIT FILE” on page 1897
“SUBTITLE” on page 1909
“SYSFILE INFO” on page 1927

"TDISPLAY" on page 1969
"TITLE" on page 1975
"TSET" on page 2025
"TSHOW" on page 2029
"USE" on page 2097

Commands That Are Stored, Pending Execution

These commands are stored, pending execution with the next command that reads the data.

"BOOTSTRAP" on page 247
"BREAK" on page 251
"CACHE" on page 253
"COMPUTE" on page 335
"COUNT" on page 365
"DO IF" on page 611
"DO REPEAT-END REPEAT" on page 619
"IF " on page 909
"LEAVE " on page 977
"LOOP-END LOOP" on page 1011
"N OF CASES" on page 1223
"PRINT" on page 1529
"PRINT EJECT" on page 1535
"PRINT SPACE" on page 1539
"RECODE" on page 1619
"SAMPLE" on page 1727
"SELECT IF " on page 1775
"TEMPORARY" on page 1971
"TIME PROGRAM" on page 1973
"WRITE" on page 2159
"XSAVE" on page 2181

Files

IBM SPSS Statistics reads, creates, and writes different types of files. This section provides an overview of these types and discusses concepts and rules that apply to all files.

Command File

A command file is a text file that contains syntax commands. You can type commands in a syntax window in an interactive session, use the Paste button in dialog boxes to paste generated commands into a syntax window, and/or use any text editor to create a command file. You can also edit a journal file to produce a command file. See the topic "Journal File" on page 40 for more information. The following is an example of a simple command file that contains both commands and inline data:

```
DATA LIST /ID 1-3 Gender 4 (A) Age 5-6 Opinion1 TO Opinion5 7-11.
BEGIN DATA
001F2621221
002M5611122
003F3422212
329M2121212
END DATA.
LIST.
```

- Case does not matter for commands but is significant for inline data. If you specified *f* for female and *m* for male in column 4 of the data line, the value of *Gender* would be *f* or *m* instead of *F* or *M* as it is now.

- Commands can be in upper or lower case. Uppercase characters are used for all commands throughout this manual only to distinguish them from other text.

Journal File

IBM SPSS Statistics keeps a journal file to record all commands either run from a syntax window or generated from a dialog box during a session. You can retrieve this file with any text editor and review it to learn how the session went. You can also edit the file to build a new command file and use it in another run. An edited and tested journal file can be saved and used later for repeated tasks. The journal file also records any error or warning messages generated by commands. You can rerun these commands after making corrections and removing the messages.

The journal file is controlled by the File Locations tab of the Options dialog box, available from the Edit menu. You can turn journaling off and on, append or overwrite the journal file, and select the journal filename and location. By default, commands from subsequent sessions are appended to the journal.

The following example is a journal file for a short session with a warning message.

```
DATA LIST /ID 1-3 Gender 4 (A) Age 5-6 Opinion1 TO Opinion5 7-11.
BEGIN DATA
001F2621221
002M5611122
003F3422212
004F45112L2
>Warning # 1102
>An invalid numeric field has been found.  The result has been set to the
>system-missing value.
END DATA.
LIST.
```

Figure 1. Records from a journal file

- The warning message, marked by the > symbol, tells you that an invalid numeric field has been found. Checking the last data line, you will notice that column 10 is *L*, which is probably a typographic error. You can correct the typo (for example, by changing the *L* to 1), delete the warning message, and submit the file again.

Data Files

A wide variety of data file formats can be read and written, including raw data files created by a data entry device or a text editor, formatted data files produced by a data management program, data files generated by other software packages, and IBM SPSS Statistics data files.

Raw Data Files

Raw data files contain only data, either generated by a programming language or entered with a data entry device or a text editor. Raw data arranged in almost any format can be read, including raw matrix materials and nonprintable codes. User-entered data can be embedded within a command file as inline data (BEGIN DATA-END DATA) or saved as an external file. Nonprintable machine codes are usually stored in an external file.

Commands that read raw data files include:

- GET DATA
- DATA LIST
- MATRIX DATA

Complex and hierarchical raw data files can be read using commands such as:

- INPUT PROGRAM
- FILE TYPE
- REREAD
- REPEATING DATA

Data Files Created by Other Applications

You can read files from a variety of other software applications, including:

- Excel spreadsheets (GET DATA command).
- Database tables (GET DATA command).
- Data Collection data sources (GET DATA command).
- Delimited (including tab-delimited and CSV) and fixed-format text data files (DATA LIST, GET DATA).
- dBase and Lotus files (GET TRANSLATE command).
- SAS datasets (GET SAS command).
- Stata data files (GET STATA command).

IBM SPSS Statistics Data Files

IBM SPSS Statistics data files are files specifically formatted for use by IBM SPSS Statistics, containing both data and the metadata (dictionary) that define the data.

- To save the active dataset in IBM SPSS Statistics format, use SAVE or XSAVE. On most operating systems, the default extension of a saved IBM SPSS Statistics data file is *.sav*. IBM SPSS Statistics data files can also be matrix files created with the MATRIX=OUT subcommand on procedures that write matrices.
- To open IBM SPSS Statistics data files, use GET.

IBM SPSS Statistics Data File Structure

The basic structure of IBM SPSS Statistics data files is similar to a database table:

- Rows (records) are cases. Each row represents a case or an observation. For example, each individual respondent to a questionnaire is a case.
- Columns (fields) are variables. Each column represents a variable or characteristic that is being measured. For example, each item on a questionnaire is a variable.

IBM SPSS Statistics data files also contain metadata that describes and defines the data contained in the file. This descriptive information is called the **dictionary**. The information contained in the dictionary includes:

- Variable names and descriptive variable labels (VARIABLE LABELS command).
- Descriptive values labels (VALUE LABELS command).
- Missing values definitions (MISSING VALUES command).
- Print and write formats (FORMATS command).

Use DISPLAY DICTIONARY to display the dictionary for the active dataset. See the topic "DISPLAY" on page 599 for more information. You can also use SYSFILE INFO to display dictionary information for any IBM SPSS Statistics data file.

Long Variable Names

In some instances, data files with variable names longer than eight bytes require special consideration:

- If you save a data file in portable format (see EXPORT), variable names that exceed eight bytes are converted to unique eight-character names. For example, *mylongrootname1*, *mylongrootname2*, and *mylongrootname3* would be converted to *mylongro*, *mylong_2*, and *mylong_3*, respectively.
- When using data files with variable names longer than eight bytes in version 10.x or 11.x, unique, eight-byte versions of variable names are used; however, the original variable names are preserved for use in release 12.0 or later. In releases prior to 10.0, the original long variable names are lost if you save the data file.
- Matrix data files (commonly created with the MATRIX OUT subcommand, available in some procedures) in which the *VARNAME_* variable is longer than an eight-byte string cannot be read by releases prior to 12.0.

Variables

The columns in IBM SPSS Statistics data files are **variables**. Variables are similar to fields in a database table.

- Variable names can be defined with numerous commands, including DATA LIST, GET DATA, NUMERIC, STRING, VECTOR, COMPUTE, and RECODE. They can be changed with the RENAME VARIABLES command.
- Optional variable attributes can include descriptive variable labels (VARIABLE LABELS command), value labels (VALUE LABELS command), and missing value definitions (MISSING VALUES command).

The following sections provide information on variable naming rules, syntax for referring to inclusive lists of variables (keywords ALL and TO), scratch (temporary) variables, and system variables.

Variable Names

Variable names are stored in the dictionary of the data file. Observe the following rules when establishing variable names or referring to variables by their names on commands:

- Each variable name must be unique; duplication is not allowed.
- Variable names can be up to 64 bytes long, and the first character must be a letter or one of the characters @, #, or $. Subsequent characters can be any combination of letters, numbers, nonpunctuation characters, and a period (.). In code page mode, sixty-four bytes typically means 64 characters in single-byte languages (for example, English, French, German, Spanish, Italian, Hebrew, Russian, Greek, Arabic, and Thai) and 32 characters in double-byte languages (for example, Japanese, Chinese, and Korean). Many string characters that only take one byte in code page mode take two or more bytes in Unicode mode. For example, é is one byte in code page format but is two bytes in Unicode format; so *résumé* is six bytes in a code page file and eight bytes in Unicode mode.

  Note: Letters include any nonpunctuation characters used in writing ordinary words in the languages supported in the platform's character set.
- Variable names cannot contain spaces.
- A # character in the first position of a variable name defines a scratch variable. You can only create scratch variables with command syntax. You cannot specify a # as the first character of a variable in dialogs that create new variables.
- A $ sign in the first position indicates that the variable is a system variable. The $ sign is not allowed as the initial character of a user-defined variable.
- The period, the underscore, and the characters $, #, and @ can be used within variable names. For example, *A._$@#1* is a valid variable name.
- Variable names cannot begin or end with a period. Names that begin with a period are invalid; names the end with a period may be interpreted as a command terminator. You cannot create variables that begin or end with a period in dialogs that create new variables.
- Variable names ending in underscores should be avoided, since such names may conflict with names of variables automatically created by commands and procedures.
- Reserved keywords cannot be used as variable names. Reserved keywords are ALL, AND, BY, EQ, GE, GT, LE, LT, NE, NOT, OR, TO, and WITH.
- Variable names can be defined with any mixture of uppercase and lowercase characters, and case is preserved for display purposes.
- When long variable names need to wrap onto multiple lines in output, lines are broken at underscores, periods, and points where content changes from lower case to upper case.

Mixed Case Variable Names

Variable names can be defined with any mixture of upper- and lowercase characters, and case is preserved for display purposes.

- Variable names are stored and displayed exactly as specified on commands that read data or create new variables. For example, `compute NewVar = 1` creates a new variable that will be displayed as *NewVar* in the Data Editor and in output from any procedures that display variable names.
- Commands that refer to existing variable names are not case sensitive. For example, `FREQUENCIES VARIABLES = newvar`, `FREQUENCIES VARIABLES = NEWVAR`, and `FREQUENCIES VARIABLES = NewVar` are all functionally equivalent.
- In languages such as Japanese, where some characters exist in both narrow and wide forms, these characters are considered different and are displayed using the form in which they were entered.
- When long variable names need to wrap onto multiple lines in output, attempts are made to break lines at underscores, periods, and changes from lower to upper case.

You can use the `RENAME VARIABLES` command to change the case of any characters in a variable name.

Example

```
RENAME VARIABLES (newvariable = NewVariable).
```

- For the existing variable name specification, case is ignored. Any combination of upper and lower case will work.
- For the new variable name, case will be preserved as entered for display purposes.

For more information, see the `RENAME VARIABLES` command.

Long Variable Names

In some instances, data files with variable names longer than eight bytes require special consideration:

- If you save a data file in portable format (see EXPORT), variable names that exceed eight bytes are converted to unique eight-character names. For example, *mylongrootname1*, *mylongrootname2*, and *mylongrootname3* would be converted to *mylongro*, *mylong_2*, and *mylong_3*, respectively.
- When using data files with variable names longer than eight bytes in version 10.x or 11.x, unique, eight-byte versions of variable names are used; however, the original variable names are preserved for use in release 12.0 or later. In releases prior to 10.0, the original long variable names are lost if you save the data file.
- Matrix data files (commonly created with the `MATRIX OUT` subcommand, available in some procedures) in which the *VARNAME_* variable is longer than an eight-byte string cannot be read by releases prior to 12.0.

Keyword TO

You can establish names for a set of variables or refer to any number of consecutive variables by specifying the beginning and the ending variables joined by the keyword `TO`.

To establish names for a set of variables with the keyword `TO`, use a character prefix with a numeric suffix.

- The prefix can be any valid name. Both the beginning and ending variables must use the same prefix.
- The numeric suffix can be any integer, but the first number must be smaller than the second. For example, `ITEM1 TO ITEM5` establishes five variables named *ITEM1*, *ITEM2*, *ITEM3*, *ITEM4*, and *ITEM5*.
- Leading zeros used in numeric suffixes are included in the variable name. For example, `V001 TO V100` establishes 100 variables--*V001*, *V002*, *V003*, ..., *V100*. `V1 TO V100` establishes 100 variables--*V1*, *V2*, *V3*, ..., *V100*.

The keyword `TO` can also be used on procedures and other commands to refer to consecutive variables on the active dataset. For example, `AVAR TO VARB` refers to the variables *AVAR* and all subsequent variables up to and including *VARB*.

- In most cases, the `TO` specification uses the variable order on the active dataset. Use the `DISPLAY` command to see the order of variables on the active dataset.

- On some subcommands, the order in which variables are named on a previous subcommand, usually the VARIABLES subcommand, is used to determine which variables are consecutive and therefore are implied by the TO specification. This is noted in the description of individual commands.

Keyword ALL

The keyword ALL can be used in many commands to specify all of the variables in the active dataset. For example,

```
FREQUENCIES /VARIABLES = ALL.
```

or

```
OLAP CUBES income by ALL.
```

In the second example, a separate table will be created for every variable in the data file, including a table of *income* by *income*.

Scratch Variables

You can use **scratch variables** to facilitate operations in transformation blocks and input programs.

- To create a scratch variable, specify a variable name that begins with the # character—for example, *#ID*. Scratch variables can be either numeric or string.
- Scratch variables are initialized to 0 for numeric variables or blank for string variables.
- Scratch variables cannot be used in procedures and cannot be saved in a data file (but they can be written to an external text file with PRINT or WRITE).
- Scratch variables cannot be assigned missing values, variable labels, or value labels.
- Scratch variables can be created between procedures but are always discarded as the next procedure begins.
- Scratch variables are discarded once a TEMPORARY command is specified.
- The keyword TO cannot refer to scratch variables and permanent variables at the same time.
- Scratch variables cannot be specified on a WEIGHT command.
- Scratch variable cannot be specified on the LEAVE command.
- Scratch variables are not reinitialized when a new case is read. Their values are always carried across cases. (So using a scratch variable can be essentially equivalent to using the LEAVE command.)

Because scratch variables are discarded, they are often useful as loop index variables and as other variables that do not need to be retained at the end of a transformation block. See the topic “Indexing Clause ” on page 1013 for more information. Because scratch variables are not reinitialized for each case, they are also useful in loops that span cases in an input program. See the topic “Creating Data ” on page 1017 for more information.

Example

```
DATA LIST LIST (",") /Name (A15).
BEGIN DATA
Nick Lowe
Dave Edmunds
END DATA.
STRING LastName (A15).
COMPUTE #index=INDEX(Name, " ").
COMPUTE LastName=SUBSTR(Name, #index+1).
LIST.
```

```
Name            LastName

Nick Lowe       Lowe
Dave Edmunds    Edmunds
```

Figure 2. Listing of case values

- *#index* is a scratch variable that is set to the numeric position of the first occurrence of a blank space in *Name*.

- The scratch variable is then used in the second COMPUTE command to determine the starting position of *LastName* within *Name*.
- The default LIST command will list the values of all variables for all cases. It does not include *#index* because LIST is a procedure that reads the data, and all scratch variables are discarded at that point.

In this example, you could have obtained the same end result without the scratch variable, using:

```
COMPUTE LastName=SUBSTR(Name, INDEX(Name, " ")+1).
```

The use of a scratch variable here simply makes the code easier to read.

Example: Scratch variable initialization

```
DATA LIST FREE /Var1.
BEGIN DATA
2 2 2
END DATA.
COMPUTE Var2=Var1+Var2.
COMPUTE Var3=0.
COMPUTE Var3=Var1+Var3.
COMPUTE #ScratchVar=Var1+#ScratchVar.
COMPUTE Var4=#ScratchVar.
LIST.
```

```
    Var1     Var2     Var3     Var4

    2.00      .       2.00     2.00
    2.00      .       2.00     4.00
    2.00      .       2.00     6.00
```

Figure 3. Listing of case values

- The new variable *Var2* is reinitialized to system-missing for each case, therefore Var1+Var2 always results in system-missing.
- The new variable *Var3* is reset to 0 for each case (COMPUTE Var3=0), therefore Var1+Var3 is always equivalent to Var1+0.
- *#ScratchVar* is initialized to 0 for the first case and is not reinitialized for subsequent cases; so Var1+#ScratchVar is equivalent to Var1+0 for the first case, Var1+2 for the second case, and Var1+4 for the third case.
- *Var4* is set to the value of *#ScratchVar* in this example so that the value can be displayed in the case listing.

In this example, the commands:

```
COMPUTE #ScratchVar=Var1+#ScratchVar.
COMPUTE Var4=#ScratchVar.
```

are equivalent to:

```
COMPUTE Var4=Var1+Var4.
LEAVE Var4.
```

System Variables

System variables are special variables created during a working session to keep system-required information, such as the number of cases read by the system, the system-missing value, and the current date. System variables can be used in data transformations.

- The names of system variables begin with a dollar sign ($).
- You cannot modify a system variable or alter its print or write format. Except for these restrictions, you can use system variables anywhere that a normal variable is used in the transformation language.
- System variables are not available for procedures.

$CASENUM. *Current case sequence number.* For each case, *$CASENUM* is the number of cases read up to and including that case. The format is F8.0. The value of *$CASENUM* is not necessarily the row number in a Data Editor window (available in windowed environments), and the value changes if the file is sorted or new cases are inserted before the end of the file.

$SYSMIS. *System-missing value.* The system-missing value displays as a period (.) or whatever is used as the decimal point.

$JDATE. *Current date in number of days from October 14, 1582* (day 1 of the Gregorian calendar). The format is F6.0.

$DATE. *Current date in international date format with two-digit year.* The format is A9 in the form dd-mmm-yy.

$DATE11. *Current date in international date format with four-digit year.* The format is A11 in the form dd-mmm-yyyy.

$TIME. *Current date and time. $TIME* represents the number of seconds from midnight, October 14, 1582, to the date and time when the transformation command is executed. The format is F20. You can display this as a date in a number of different date formats. You can also use it in date and time functions.

$LENGTH. *The current page length.* The format is F11.0. For more information, see SET.

$WIDTH. *The current page width.* The format is F3.0. For more information, see SET.

Variable Types and Formats

There are two basic variable types:

- **String.** Also referred to alphanumeric. String values can contain any combination of letters, numbers, and other characters.
- **Numeric.** Numeric values are stored internally as double-precision floating-point numbers.

  Note: SPSS Statistics can at times process what would normally present as single decimal values as multiple decimal values (up to 15 decimals). For more information, refer to this IBM Support article.

Variable formats determine how raw data is read into storage and how values are displayed and written. For example, all dates and times are stored internally as numeric values, but you can use date and time format specifications to both read and display date and time values in standard date and time formats. The following sections provide details on how formats are specified and how those formats affect how data are read, displayed, and written.

Input and Output Formats

Values are read according to their **input** format and displayed according to their **output** format. The input and output formats differ in several ways.

- The input format is either specified or implied on the DATA LIST, GET DATA, or other data definition commands. It is in effect only when cases are built in an active dataset.
- Output formats are automatically generated from input formats, with output formats expanded to include punctuation characters, such as decimal indicators, grouping symbols, and dollar signs. For example, an input format of DOLLAR7.2 will generate an output format of DOLLAR10.2 to accommodate the dollar sign, grouping symbol (comma), and decimal indicator (period).
- The formats (specified or default) on NUMERIC, STRING, COMPUTE, or other commands that create new variables are output formats. You must specify adequate widths to accommodate all punctuation characters.
- The output format is in effect during the entire working session (unless explicitly changed) and is saved in the dictionary of IBM SPSS Statistics data files.
- Output formats for numeric variables can be changed with FORMATS, PRINT FORMATS, and WRITE FORMATS.

String Variable Formats

- The values of string variables can contain numbers, letters, and special characters and can be up to 32,767 bytes.

- System-missing values cannot be generated for string variables, since any character is a legal string value.
- When a transformation command that creates or modifies a string variable yields a missing or undefined result, a null string is assigned. The variable displays as blanks and is not treated as missing.
- String formats are used to read and write string variables. The input values can be alphanumeric characters (A format) or the hexadecimal representation of alphanumeric characters (AHEX format).
- For fixed-format raw data, the width can be explicitly specified on commands such as DATA LIST and GET DATA or implied if column-style specifications are used. For freefield data, the default width is 1; if the input string may be longer, w must be explicitly specified. Input strings shorter than the specified width are right-padded with blanks.
- The output format for a string variable is always A. The width is determined by the input format or the format assigned on the STRING command. Once defined, the width of a string variable can only be changed with the ALTER TYPE command.

A Format (Standard Characters)

The A format is used to read standard characters. Characters can include letters, numbers, punctuation marks, blanks, and most other characters on your keyboard. Numbers entered as values for string variables cannot be used in calculations unless you convert them to numeric format with the NUMBER function. See the topic "String/numeric conversion functions" on page 81 for more information.

Fixed data:

With fixed-format input data, any punctuation—including leading, trailing, and embedded blanks—within the column specifications is included in the string value. For example, a string value of

```
Mr. Ed
```

(with one embedded blank) is distinguished from a value of

```
Mr.  Ed
```

(with two embedded blanks). It is also distinguished from a string value of

```
MR. ED
```

(all upper case), and all three are treated as separate values. These can be important considerations for any procedures, transformations, or data selection commands involving string variables. Consider the following example:

```
DATA LIST FIXED /ALPHAVAR 1-10 (A).
BEGIN DATA
Mr. Ed
Mr. Ed
MR. ED
Mr.  Ed
 Mr. Ed
END DATA.
AUTORECODE ALPHAVAR /INTO NUMVAR.
LIST.
```

AUTORECODE recodes the values into consecutive integers. The following figure shows the recoded values.

```
ALPHAVAR   NUMVAR

Mr. Ed        4
Mr. Ed        4
MR. ED        2
Mr.  Ed       3
 Mr. Ed       1
```

Figure 4. Different string values illustrated

AHEX Format (Hexadecimal Characters)

The AHEX format is used to read the hexadecimal representation of standard characters. Each set of two hexadecimal characters represents one standard character.

- The w specification refers to columns of the hexadecimal representation and must be an even number. Leading, trailing, and embedded blanks are not allowed, and only valid hexadecimal characters can be used in input values.
- For some operating systems (e.g., IBM CMS), letters in hexadecimal values must be upper case.
- The default output format for variables read with the AHEX input format is the A format. The default width is half the specified input width. For example, an input format of AHEX14 generates an output format of A7.
- Used as an output format, the AHEX format displays the printable characters in the hexadecimal characters specific to your system. The following commands run on a UNIX system--where A=41 (decimal 65), a=61 (decimal 97), and so on--produce the output shown below:

```
DATA LIST FIXED
   /A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T,U,V,W,X,Y,Z 1-26 (A).
FORMATS ALL (AHEX2).
BEGIN DATA
ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz
END DATA.
LIST.
```

```
A  B  C  D  E  F  G  H  I  J  K  L  M  N  O  P  Q  R  S  T  U  V  W  X  Y  Z

41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A
61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A
```

Figure 5. Display of hexadecimal representation of the character set with AHEX format

Numeric Variable Formats

- By default, if no format is explicitly specified, commands that read raw data--such as DATA LIST and GET DATA--assume that variables are numeric with an F format type. The default width depends on whether the data are in fixed or freefield format. For a discussion of fixed data and freefield data, see DATA LIST .
- Numeric variables created by COMPUTE, COUNT, or other commands that create numeric variables are assigned a format type of F8.2 (or the default numeric format defined on SET FORMAT).
- If a data value exceeds its width specification, an attempt is made to display some value nevertheless. First, the decimals are rounded, then punctuation characters are taken out, then scientific notation is tried, and if there is still not enough space, an ellipsis (...) is displayed, indicating that a value is present but cannot be displayed in the assigned width.
- The output format does not affect the value stored in the file. A numeric value is always stored in double-precision.

  Note: SPSS Statistics can at times process what would normally present as single decimal values as multiple decimal values (up to 15 decimals). For more information, refer to this IBM Support article.
- For all numeric formats, the maximum width is 40.
- For numeric formats where decimals are allowed, the maximum number of decimals is 16.
- For default numeric (F) format and scientific notation (E) format, the decimal indicator of the input data from text data sources (read by commands such as DATA LIST and GET DATA) must match the IBM SPSS Statistics locale decimal indicator (period or comma). Use SET DECIMAL to set the decimal indicator. Use SHOW DECIMAL to display the current decimal indicator.

F, N, and E Formats

The following table lists the formats most commonly used to read in and write out numeric data. Format names are followed by total width (w) and an optional number of decimal positions (d). For example, a format of F5.2 represents a numeric value with a total width of 5, including two decimal positions and a decimal indicator.

Table 1. Common numeric formats						
Format type	**Sample format**	**Sample input**	**Fixed output format**	**Fixed output value**	**Freefield output format**	**Freefield output value**
Fw	F5	1234	F5.0	1234	F5.0	1234
Fw	F5	1.234	F5.0	1*	F5.0	1*
Fw.d	F5.2	1234	F6.2	1234.0	F6.2	1234.0
Fw.d	F5.2	1.234	F6.2	1.23*	F6.2	1.23*
Nw	N5	00123	N5	00123	N5	00123
Nw	N5	123	N5	.†	N5	00123
Ew.d	E8.0	1234E3	E10.3	1234E+06	E10.3	1234E+06‡
Ew.d	E8.0	1234	E10.3	1234E+03	E10.3	1234E+03

* Only the display is truncated. The value is stored in full precision.

† System-missing value.

‡ Scientific notation is accepted in input data with F, COMMA, DOLLAR, DOT, and PCT formats. The same rules apply as specified below.

For fixed data:

- With the N format, only unsigned integers are allowed as input values. Values not padded with leading zeros to the specified width or those containing decimal points are assigned the system-missing value. This input format is useful for reading and checking values that should be integers containing leading zeros.
- The E format reads all forms of scientific notation. If the sign is omitted, + is assumed. If the sign (+ or –) is specified before the exponent, the *E* or *D* can be omitted. A single space is permitted after the *E* or *D* and/or after the sign. If both the sign and the letter *E* or *D* are omitted, implied decimal places are assumed. For example, 1.234E3, 1.234+3, 1.234E+3, 1.234D3, 1.234D+3, 1.234E 3, and 1234 are all legitimate values. Only the last value can imply decimal places.
- E format input values can be up to 40 characters wide and include up to 15 decimal positions.
- The default output width (*w*) for the E format is either the specified input width or the number of specified decimal positions plus 7 (*d*+7), whichever is greater. The minimum width is 10 and the minimum decimal places are 3.
- The DATA LIST command can read fixed-format numeric data with **implied** decimal positions. See the topic "Implied Decimal Positions " on page 527 for more information.

For freefield data:

- F format *w* and *d* specifications do not affect how data are read. They only determine the output formats (expanded, if necessary). 1234 is always read as 1234 in freefield data, but a specified F5.2 format will be expanded to F6.2 and the value will be displayed as 1234.0 (the last decimal place is rounded because of lack of space).
- When the N format is used for freefield data, input values with embedded decimal indicators are assigned the system-missing value, but integer input values without leading zeroes are treated as valid. For example, with an input format of N5.0, a value of 123 is treated the same as a value of 00123, but a value of 12.34 is assigned the system-missing value.
- The E format for freefield data follows the same rules as for fixed data except that no blank space is permitted in the value. Thus, 1.234E3 and 1.234+3 are allowed, but the value 1.234 3 will cause mistakes when the data are read.
- The default output E format and the width and decimal place limitations are the same as with fixed data.

COMMA, DOT, DOLLAR, and PCT Formats

The numeric formats listed below read and write data with embedded punctuation characters and symbols, such as commas, dots, and dollar and percent signs. The input data may or may not contain such characters. The data values read in are stored as numbers but displayed using the appropriate formats.

- **DOLLAR.** Numeric values with a leading dollar sign, a comma used as the grouping separator, and a period used as the decimal indicator. For example, $1,234.56.
- **COMMA.** Numeric values with a comma used as the grouping separator and a period used as decimal indicator. For example, 1,234.56.
- **DOT.** Numeric values with a period used as the grouping separator and a comma used as the decimal indicator. For example, 1.234,56.
- **PCT.** Numeric values with a trailing percent sign. For example, 123.45%.

The input data values may or may not contain the punctuation characters allowed by the specified format, but the data values may not contain characters *not* allowed by the format. For example, with a DOLLAR input format, input values of 1234.56, 1,234.56, and $1,234.56 are all valid and stored internally as the same value--but with a COMMA input format, the input value with a leading dollar sign would be assigned the system-missing value.

Example

```
DATA LIST LIST (" ") /dollarVar (DOLLAR9.2) commaVar (COMMA9.2)
  dotVar (DOT9.2) pctVar (PCT9.2).
BEGIN DATA
1234 1234 1234 1234
$1,234.00 1,234.00 1.234,00 1234.00%
END DATA.
LIST.
```

```
    dollarVar     commaVar       dotVar      pctVar

    $1,234.00     1,234.00     1.234,00    1234.00%
    $1,234.00     1,234.00     1.234,00    1234.00%
```

Figure 6. Output illustrating DOLLAR, COMMA, DOT, and PCT formats

Other formats that use punctuation characters and symbols are date and time formats and custom currency formats. For more information on date and time formats, see “Date and Time Formats” on page 53 . Custom currency formats are output formats only, and are defined with the SET command.

Binary and Hexadecimal Formats

Data can be read and written in formats used by a number of programming languages such as PL/I, COBOL, FORTRAN, and Assembler. The data can be binary, hexadecimal, or zoned decimal. Formats described in this section can be used both as input formats and output formats, but with fixed data only.

The default output format for all formats described in this section is an equivalent F format, allowing the maximum number of columns for values with symbols and punctuation. To change the default, use FORMATS or WRITE FORMATS.

IBw.d (integer binary):

The IB format reads fields that contain fixed-point binary (integer) data. The data might be generated by COBOL using COMPUTATIONAL data items, by FORTRAN using INTEGER*2 or INTEGER*4, or by Assembler using fullword and halfword items. The general format is a signed binary number that is 16 or 32 bits in length.

The general syntax for the IB format is IBw.d, where w is the field width in bytes (omitted for column-style specifications) and d is the number of digits to the right of the decimal point. Since the width is expressed in bytes and the number of decimal positions is expressed in digits, *d* can be greater than *w*. For example, both of the following commands are valid:

```
DATA LIST FIXED /VAR1 (IB4.8).
```

```
DATA LIST FIXED /VAR1 1-4 (IB,8).
```

Widths of 2 and 4 represent standard 16-bit and 32-bit integers, respectively. Fields read with the IB format are treated as signed. For example, the one-byte binary value 11111111 would be read as –1.

PIBw.d (positive integer binary) :

The PIB format is essentially the same as IB except that negative numbers are not allowed. This restriction allows one additional bit of magnitude. The same one-byte value 11111111 would be read as 255.

PIBHEXw (hexadecimal of PIB):

The PIBHEX format reads hexadecimal numbers as unsigned integers and writes positive integers as hexadecimal numbers. The general syntax for the PIBHEX format is PIBHEXw, where w indicates the total number of hexadecimal characters. The w specification must be an even number with a maximum of 16.

For input data, each hexadecimal number must consist of the exact number of characters. No signs, decimal points, or leading and trailing blanks are allowed. For some operating systems (such as IBM CMS), hexadecimal characters must be upper case. The following example illustrates the kind of data that the PIBHEX format can read:

```
DATA LIST FIXED
 /VAR1 1-4 (PIBHEX) VAR2 6-9 (PIBHEX) VAR3 11-14 (PIBHEX).
BEGIN DATA
0001 0002 0003
0004 0005 0006
0007 0008 0009
000A 000B 000C
000D 000E 000F
00F0 0B2C FFFF
END DATA.
LIST.
```

The values for *VAR1*, *VAR2*, and *VAR3* are listed in the figure below. The PIBHEX format can also be used to write decimal values as hexadecimal numbers, which may be useful for programmers.

```
 VAR1   VAR2   VAR3

    1      2      3
    4      5      6
    7      8      9
   10     11     12
   13     14     15
  240   2860  65535
```

Figure 7. Output displaying values read in PIBHEX format

Zw.d (zoned decimal):

The Z format reads data values that contain zoned decimal data. Such numbers may be generated by COBOL systems using DISPLAY data items, by PL/I systems using PICTURE data items, or by Assembler using zoned decimal data items.

In zoned decimal format, one digit is represented by one byte, generally hexadecimal F1 representing 1, F2 representing 2, and so on. The last byte, however, combines the sign for the number with the last digit. In the last byte, hexadecimal A, F, or C assigns +, and B, D, or E assigns –. For example, hexadecimal D1 represents 1 for the last digit and assigns the minus sign (–) to the number.

The general syntax of the Z format is Zw.d, where w is the total number of bytes (which is the same as columns) and d is the number of decimals. For input data, values can appear anywhere within the column specifications. Both leading and trailing blanks are allowed. Decimals can be implied by the input format specification or explicitly coded in the data. Explicitly coded decimals override the input format specifications.

The following example illustrates how the Z format reads zoned decimals in their printed forms on IBM mainframe and PC systems. The printed form for the sign zone (A to I for +1 to +9, and so on) may vary from system to system.

```
DATA LIST FIXED /VAR1 1-5 (Z) VAR2 7-11 (Z,2) VAR3 13-17 (Z)
 VAR4 19-23 (Z,2) VAR5 25-29 (Z) VAR6 31-35 (Z,2).
BEGIN DATA
1234A 1234A 1234B 1234B 1234C 1234C
```

```
1234D 1234D 1234E 1234E 1234F 1234F
1234G 1234G 1234H 1234H 1234I 1234I
1234J 1234J 1234K 1234K 1234L 1234L
1234M 1234M 1234N 1234N 1234O 1234O
1234P 1234P 1234Q 1234Q 1234R 1234R
1234{ 1234{ 1234} 1234} 1.23M 1.23M
END DATA.
LIST.
```

The values for *VAR1* to *VAR6* are listed in the following figure.

```
VAR1    VAR2   VAR3    VAR4   VAR5    VAR6

 12341  123.41  12342  123.42  12343  123.43
 12344  123.44  12345  123.45  12346  123.46
 12347  123.47  12348  123.48  12349  123.49
-12341 -123.41 -12342 -123.42 -12343 -123.43
-12344 -123.44 -12345 -123.45 -12346 -123.46
-12347 -123.47 -12348 -123.48 -12349 -123.49
 12340  123.40 -12340 -123.40     -1   -1.23
```

Figure 8. Output displaying values read in Z format

The default output format for the Z format is the equivalent F format, as shown in the figure. The default output width is based on the input width specification plus one column for the sign and one column for the implied decimal point (if specified). For example, an input format of Z4.0 generates an output format of F5.0, and an input format of Z4.2 generates an output format of F6.2.

Pw.d (packed decimal):

The P format is used to read fields with packed decimal numbers. Such numbers are generated by COBOL using COMPUTATIONAL–3 data items and by Assembler using packed decimal data items. The general format of a packed decimal field is two four-bit digits in each byte of the field except the last. The last byte contains a single digit in its four leftmost bits and a four-bit sign in its rightmost bits. If the last four bits are 1111 (hexadecimal F), the value is positive; if they are 1101 (hexadecimal D), the value is negative. One byte under the P format can represent numbers from –9 to 9.

The general syntax of the P format is Pw.d, where w is the number of bytes (not digits) and d is the number of digits to the right of the implied decimal point. The number of digits in a field is (2*w–1).

PKw.d (unsigned packed decimal):

The PK format is essentially the same as P except that there is no sign. That is, even the rightmost byte contains two digits, and negative data cannot be represented. One byte under the PK format can represent numbers from 0 to 99. The number of digits in a field is 2*w.

RBw (real binary):

The RB format is used to read data values that contain internal format floating-point numbers. Such numbers are generated by COBOL using COMPUTATIONAL–1 or COMPUTATIONAL–2 data items, by PL/I using FLOATING DECIMAL data items, by FORTRAN using REAL or REAL*8 data items, or by Assembler using floating-point data items.

The general syntax of the RB format is RBw, where w is the total number of bytes. The width specification must be an even number between 2 and 8. Normally, a width specification of 8 is used to read double-precision values, and a width of 4 is used to read single-precision values.

Note: SPSS Statistics can at times process what would normally present as single decimal values as multiple decimal values (up to 15 decimals). For more information, refer to this IBM Support article.

RBHEXw (hexadecimal of RB):

The RBHEX format interprets a series of hexadecimal characters as a number that represents a floating-point number. This representation is system-specific. If the field width is less than twice the width of a floating-point number, the value is right-padded with binary zeros. For some operating systems (for example, IBM CMS), letters in hexadecimal values must be upper case.

The general syntax of the RBHEX format is RBHEXw, where w indicates the total number of columns. The width must be an even number. The values are real (floating-point) numbers. Leading and trailing blanks are not allowed. Any data values shorter than the specified input width must be padded with leading zeros.

Date and Time Formats

Date and time formats are both input and output formats. Like numeric formats, each input format generates a default output format, automatically expanded (if necessary) to accommodate display width. Internally, all date and time format values are stored as a number of seconds: date formats (e.g., DATE, ADATE, SDATE, DATETIME) are stored as the number of seconds since October 14, 1582; time formats (TIME, DTIME, and MTIME) are stored as a number of seconds that represents a time interval (e.g., 10:00:00 is stored internally as 36000, which is 60 seconds x 60 minutes x 10 hours).

- All date and time formats have a minimum input width, and some have a different minimum output. Wherever the input minimum width is less than the output minimum, the width is expanded automatically when displaying or printing values. However, when you specify output formats, you must allow enough space for displaying the date and time in the format you choose.
- Input data shorter than the specified width are correctly evaluated as long as all the necessary elements are present. For example, with the TIME format, `1:2`, `01 2`, and `01:02` are all correctly evaluated even though the minimum width is 5. However, if only one element (hours or minutes) is present, you must use a time function to aggregate or convert the data. See the topic "Date and time functions" on page 72 for more information.
- If a date or time value cannot be completely displayed in the specified width, values are truncated in the output. For example, an input time value of 1:20:59 (1 hour, 20 minutes, 59 seconds) displayed with a width of 5 will generate an output value of 01:20, not 01:21. The truncation of output does not affect the numeric value stored in the working file.

The following table shows all available date and time formats, where *w* indicates the total number of columns and *d* (if present) indicates the number of decimal places for fractional seconds. The example shows the output format with the minimum width and default decimal positions (if applicable). The format allowed in the input data is much less restrictive. See the topic "Input Data Specification" on page 54 for more information.

Table 2. Date and time formats

General form	Format type	Min w In	Min w Out	Max w	Max d	Example
dd-mmm-yy	DATEw	6	9	40		28-OCT-90
dd-mmm-yyyy	DATEw	8	11			28-OCT-1990
mm/dd/yy	ADATEw	6	8	40		10/28/90
mm/dd/yyyy	ADATEw	8	10			10/28/1990
dd.mm.yy	EDATEw	6	8	40		28.10.90
dd.mm.yyyy	EDATEw	8	10			28.10.1990
yyddd	JDATEw	5	5	40		90301
yyyyddd	JDATEw	7	7			1990301
yy/mm/dd	SDATEw	6	8	40		90/10/28
yyyy/mm/dd	SDATEw	8	10			1990/10/28
q Q yy	QYRw	4	6	40		4 Q 90
q Q yyyy	QYRw	6	8			4 Q 1990

Table 2. Date and time formats (continued)

General form	Format type	Min w In	Min w Out	Max w	Max d	Example
mmm yy	MOYRw	6	6	40		OCT 90
mmm yyyy	MOYRw	8	8			OCT 1990
ww WK yy	WKYRw	4	8	40		43 WK 90
ww WK yyyy	WKYRw	6	10			43 WK 1990
(name of the day)	WKDAYw	2	2	40		SU
(name of the month)	MONTHw	3	3	40		JAN
hh:mm	TIMEw	4	5	40		01:02
hh:mm:ss.s	TIMEw.d	8	10	40	16	01:02:34.75
hh:mm:ss	TIMEw	8	8	40		11:50:30
mm:ss	MTIMEw	4	5	40		02:34
mm:ss.s	MTIMEw.d	6	7	40	16	02:34.75
ddd hh:mm	DTIMEw	1	1	40		20 08:03
ddd hh:mm:ss	DTIMEw	12	23	40		113 04:20:18
dd-mmm-yyyy hh:mm	DATETIMEw	17	17	40		20-JUN-1990 08:03
dd-mmm-yyyy hh:mm:ss.s	DATETIMEw.d	20	20	40	5	20-JUN-1990 08:03:00
dd-mmm-yyyy hh:mm:ss	DATETIMEw	20	20	40		10-Dec-1982 10:19:59
yyyy-mm-dd hh:mm	YMDHMSw	12	16	40		1990-06-20 08:03
yyyy-mm-dd hh:mm:ss	YMDHMS	19	19	40		1987-02-04 21:13:49
yyyy-mm-dd hh:mm:ss.s	YMDHMSw.d	16	21	40	5	1990-06-20 08:03:00.0

* All date and time formats produce sortable data. SDATE, a date format used in a number of Asian countries, can be sorted in its character form and is used as a sortable format by many programmers.

Input Data Specification

The following general rules apply to date and time input formats:

- The century value for two-digit years is defined by the *SET EPOCH* value. By default, the century range begins 69 years prior to the current year and ends 30 years after the current year. Whether all four digits or only two digits are displayed in output depends on the width specification on the format.
- Dashes, periods, commas, slashes, or blanks can be used as delimiters in the input values. For example, with the DATE format, the following input forms are all acceptable:

  ```
  28-OCT-90 28/10/1990 28.OCT.90 28 October, 1990
  ```

  The displayed values, however, will be the same: *28-OCT-90* or *28-OCT-1990*, depending on whether the specified width allows 11 characters in output.

  For version 24 and higher, delimiters can be omitted in input values for DATE, ADATE, EDATE, and SDATE. For example, with the ADATE format, the form `10281990` is acceptable. When delimiters are omitted, single digit specifications for month and day are not supported and year specifications must be 2 or 4 digits. Also, when month names are used, they must be specified in the three letter format when delimiters are omitted, as in *28OCT1990*.
- The JDATE format does not allow internal delimiters and requires leading zeros for day values of less than 100 and two-digit-year values of less than 10. For example, for January 1, 1990, the following two specifications are acceptable:

  ```
  90001 1990001
  ```

  However, neither of the following is acceptable:

  ```
  90 1 90/1
  ```
- Months can be represented in digits, Roman numerals, or three-character abbreviations, and they can be fully spelled out. For example, all of the following specifications are acceptable for October:

  ```
  10 X OCT October
  ```
- The quarter in QYR format is expressed as 1, 2, 3, or 4. It is separated from the year by the letter *Q*. Blanks can be used as additional delimiters. For example, for the fourth quarter of 1990, all of the following specifications are acceptable:

  ```
  4Q90 4Q1990 4 Q 90 4 Q 1990
  ```

  On some operating systems, such as IBM CMS, *Q* must be upper case. The displayed output is *4 Q 90* or *4 Q 1990*, depending on whether the width specified allows all four digits of the year.

  For version 24 and higher, the form Qq can be used to specify the quarter. For example, for the fourth quarter of 1990, the forms `Q4 1990` and `Q41990` are acceptable.
- The week in the WKYR format is expressed as a number from 1 to 53. Week 1 begins on January 1, week 2 on January 8, and so on. The value may be different from the number of the calendar week. The week and year are separated by the string WK. Blanks can be used as additional delimiters. For example, for the 43rd week of 1990, all of the following specifications are acceptable:

  ```
  43WK90 43WK1990 43 WK 90 43 WK 1990
  ```

  On some operating systems, such as IBM CMS, WK must be upper case. The displayed output is *43 WK 90* or *43 WK 1990*, depending on whether the specified width allows enough space for all four digits of the year.

  For version 24 and higher, the week and year can be separated by a blank, or the delimiter can be omitted. For example, for the 43rd week of 1990, the forms `43 1990` and `431990` are acceptable.
- In time specifications, colons can be used as delimiters between hours, minutes, and seconds. For version 24 and higher, the delimiters can be omitted for TIME and MTIME (introduced in version 24). When delimiters are omitted, single digit specifications for hours, minutes, or seconds are not supported. When hours are included, both hours and minutes are required but seconds are optional. For MTIME format, which represents minutes and seconds, both minutes and seconds are required. A period is required to separate seconds from fractional seconds. Hours can be of unlimited magnitude, but the maximum value for minutes is 59 and for seconds 59.999. . . . For MTIME format, however, minutes can be of unlimited magnitude.

- Data values can contain a sign (+ or –) in TIME, DTIME, and MTIME formats to represent time intervals before or after a point in time.
- For YMDHMS format (introduced in version 24), the separator between the date and time parts can be a space, an uppercase T, or it can be omitted. If the separator is omitted, then the delimiters in the date and time parts must also be omitted.

Example: DATE, ADATE, and JDATE

```
DATA LIST FIXED
 /VAR1 1-17 (DATE) VAR2 21-37 (ADATE) VAR3 41-47 (JDATE).
BEGIN DATA
28-10-90           10/28/90           90301
28.OCT.1990        X 28 1990          1990301
28 October, 2001   Oct.  28, 2001     2001301
END DATA.
LIST.
```

- Internally, all date format variables are stored as the number of seconds from 0 hours, 0 minutes, and 0 seconds of Oct. 14, 1582.

The LIST output from these commands is shown in the following figure.

```
       VAR1            VAR2          VAR3

28-OCT-1990       10/28/1990        1990301
28-OCT-1990       10/28/1990        1990301
28-OCT-2001       10/28/2001        2001301
```

Figure 9. Output illustrating DATE, ADATE, and JDATE formats

Example: QYR, MOYR, and WKYR

```
DATA LIST FIXED /VAR1 1-10 (QYR) VAR2 12-25 (MOYR) VAR3 28-37 (WKYR).
BEGIN DATA
4Q90       10/90           43WK90
4 Q 90     Oct-1990        43 WK 1990
4 Q 2001   October, 2001   43 WK 2001
END DATA.
LIST.
```

- Internally, the value of a QYR variable is stored as midnight of the first day of the first month of the specified quarter, the value of a MOYR variable is stored as midnight of the first day of the specified month, and the value of a WKYR format variable is stored as midnight of the first day of the specified week. Thus, *4Q90* and *10/90* are both equivalent to October 1, 1990, and *43WK90* is equivalent to October 22, 1990.

The LIST output from these commands is shown in the following figure.

```
    VAR1       VAR2            VAR3

4 Q 1990   OCT 1990       43 WK 1990
4 Q 1990   OCT 1990       43 WK 1990
4 Q 2001   OCT 2001       43 WK 2001
```

Figure 10. Output illustrating QYR, MOYR, and WKYR formats

Example: TIME and MTIME

```
DATA LIST FIXED
 /VAR1 1-11 (TIME,2) VAR2 13-21 (TIME) VAR3 23-28 (TIME) VAR4 31-35 (MTIME).
BEGIN DATA
1:2:34.75   1:2:34.75 1:2:34  2:34
END DATA.
LIST.
```

- TIME reads and writes time of the day or a time interval. MTIME reads and writes a time interval that is specified in minutes and seconds.
- Internally, the TIME or MTIME values are stored as the number of seconds from midnight of the day or of the time interval.

The LIST output from these commands is shown in the following figure.

```
       VAR1      VAR2   VAR3    VAR4

 1:02:34.75   1:02:34   1:02   02:34
```

Figure 11. Output illustrating TIME and MTIME formats

Example: WKDAY and MONTH

```
DATA LIST FIXED
 /VAR1 1-9 (WKDAY) VAR2 10-18 (WKDAY)
  VAR3 20-29 (MONTH) VAR4 30-32 (MONTH) VAR5 35-37 (MONTH).
BEGIN DATA
Sunday   Sunday    January   1    Jan
Monday   Monday    February  2    Feb
Tues     Tues      March     3    Mar
Wed      Wed       April     4    Apr
Th       Th        Oct       10   Oct
Fr       Fr        Nov       11   Nov
Sa       Sa        Dec       12   Dec
END DATA.
FORMATS VAR2 VAR5 (F2).
LIST.
```

- WKDAY reads and writes the day of the week; MONTH reads and writes the month of the year.
- Values for WKDAY are entered as strings but stored as numbers. They can be used in arithmetic operations but not in string functions.
- Values for MONTH can be entered either as strings or as numbers but are stored as numbers. They can be used in arithmetic operations but not in string functions.
- To display the values as numbers, assign an F format to the variable, as was done for *VAR2* and *VAR5* in the above example.

The LIST output from these commands is shown in the following figure.

```
     VAR1 VAR2       VAR3 VAR4 VAR5

SUNDAY      1  JANUARY     JAN   1
MONDAY      2  FEBRUARY    FEB   2
TUESDAY     3  MARCH       MAR   3
WEDNESDAY   4  APRIL       APR   4
THURSDAY    5  OCTOBER     OCT  10
FRIDAY      6  NOVEMBER    NOV  11
SATURDAY    7  DECEMBER    DEC  12
```

Figure 12. Output illustrating WKDAY and MONTH formats

Example: DTIME, DATETIME, and YMDHMS

```
DATA LIST FIXED /VAR1 1-14 (DTIME) VAR2 18-42 (DATETIME) VAR3 46-67 (YMDHMS).
BEGIN DATA
20 8:3           20-6-90 8:3                 1990-06-20 8:3
20:8:03:46       20/JUN/1990 8:03:46         1990-06-20 8:03:46
20 08 03 46.75   20 June, 2001 08 03 46.75   2001-06-20T08:03:46.75
END DATA.
LIST.
```

- DTIME, DATETIME, and YMDHMS read and write time intervals.
- The decimal point is explicitly coded in the input data for fractional seconds.
- The DTIME format allows a – or + sign in the data value to indicate a time interval before or after a point in time.
- Internally, values for a DTIME variable are stored as the number of seconds of the time interval, while those for a DATETIME or YMDHMS variable are stored as the number of seconds from 0 hours, 0 minutes, and 0 seconds of Oct. 14, 1582.

The LIST output from these commands is shown in the following figure.

```
          VAR1                    VAR2                  VAR3

   20 08:03:00      20-JUN-1990 08:03:00    1990-06-20 08:03:00
   20 08:03:46      20-JUN-1990 08:03:46    1990-06-20 08:03:46
   20 08:03:46      20-JUN-2001 08:03:46    2001-06-20 08:03:46
```

Figure 13. Output illustrating DTIME, DATETIME, and YMDHMS formats

FORTRAN-like Input Format Specifications

You can use FORTRAN-like input format specifications to define formats for a set of variables, as in the following example:

```
DATA LIST FILE=HUBDATA RECORDS=3
     /MOHIRED, YRHIRED, DEPT1 TO DEPT4 (T12, 2F2.0, 4(1X,F1.0)).
```

- The specification T12 in parentheses tabs to the 12th column. The first variable (*MOHIRED*) will be read beginning from column 12.
- The specification 2F2.0 assigns the format F2.0 to two adjacent variables (*MOHIRED* and *YRHIRED*).
- The next four variables (*DEPT1* to *DEPT4*) are each assigned the format F1.0. The 4 in 4(1X,F1.0) distributes the same format to four consecutive variables. 1X skips one column before each variable. (The column-skipping specification placed within the parentheses is distributed to each variable.)

Transformation Expressions

Transformation expressions are used in commands such as COMPUTE, IF, DO IF, LOOP IF, and SELECT IF.

Release history

Release 13.0

- APPLYMODEL and STRAPPLYMODEL functions introduced.
- DATEDIFF and DATESUM functions introduced.

Release 14.0

- REPLACE function introduced.
- VALUELABEL function introduced.

Release 16.0

- CHAR.INDEX function introduced.
- CHAR.LENGTH function introduced.
- CHAR.LPAD function introduced.
- CHAR.MBLEN function introduced.
- CHAR.RINDEX function introduced.
- CHAR.RPAD function introduced.
- CHAR.SUBSTR function introduced.
- NORMALIZE function introduced.
- NTRIM function introduced.
- STRUNC function introduced.

Release 17.0

- MEDIAN function introduced.
- *mult* and *fuzzbits* arguments introduced for the RND and TRUNC functions.
- NEIGHBOR and DISTANCE functions added to APPLYMODEL and STRAPPLYMODEL.

Numeric expressions

Numeric expressions can be used with the COMPUTE and IF commands and as part of a logical expression for commands such as IF, DO IF, LOOP IF, and SELECT IF. Arithmetic expressions can also appear in the index portion of a LOOP command, on the REPEATING DATA command, and on the PRINT SPACES command.

New numeric variables created with transformation expressions have an unknown measurement level until after the next command that reads the data (such as a statistical or charting procedure or the EXECUTE command). For information on default measurement level assignment, see SET SCALEMIN.

Arithmetic operations

The following arithmetic operators are available:

+. *Addition*

–. *Subtraction*

*. *Multiplication*

/. *Division*

**. *Exponentiation*

- No two operators can appear consecutively.
- Arithmetic operators cannot be implied. For example, (VAR1)(VAR2) is not a legal specification; you must specify VAR1*VAR2.
- Arithmetic operators and parentheses serve as delimiters. To improve readability, blanks (not commas) can be inserted before and after an operator.
- To form complex expressions, you can use variables, constants, and functions with arithmetic operators.
- The order of execution is as follows: functions; exponentiation; multiplication, division, and unary –; and addition and subtraction.
- Operators at the same level are executed from left to right.
- To override the order of operation, use parentheses. Execution begins with the innermost set of parentheses and progresses out.

Numeric constants

- Constants used in numeric expressions or as arguments to functions can be integer or noninteger, depending on the application or function.
- You can specify as many digits in a constant as needed as long as you understand the precision restrictions of your computer.
- Numeric constants can be signed (+ or –) but cannot contain any other special characters, such as the comma or dollar sign.
- Numeric constants can be expressed with scientific notation. Thus, the exponent for a constant in scientific notation is limited to two digits. The range of values allowed for exponents in scientific notation is from –99 to +99.

Complex numeric arguments

- Except where explicitly restricted, complex expressions can be formed by nesting functions and arithmetic operators as arguments to functions.
- The order of execution for complex numeric arguments is as follows: functions; exponentiation; multiplication, division, and unary –; and addition and subtraction.
- To control the order of execution in complex numeric arguments, use parentheses.

Arithmetic operations with date and time variables

Most date and time variables are stored internally as the number of seconds from a particular date or as a time interval and therefore can be used in arithmetic operations. Many operations involving dates and time can be accomplished with the extensive collection of date and time functions.

- A *date* is a floating-point number representing the number of seconds from midnight, October 14, 1582. Dates, which represent a particular point in time, are stored as the number of seconds to that date. For example, October 28, 2007, is stored as 13,412,908,800.

- A date includes the time of day, which is the time interval past midnight. When time of day is not given, it is taken as 00:00 and the date is an even multiple of 86,400 (the number of seconds in a day).
- A *time interval* is a floating-point number representing the number of seconds in a time period, for example, an hour, minute, or day. For example, the value representing 5.5 days is 475,200; the value representing the time interval 14:08:17 is 50,897.
- QYR, MOYR, and WKYR variables are stored as midnight of the first day of the respective quarter, month, and week of the year. Therefore, *1 Q 90*, *1/90*, and *1 WK 90* are all equivalents of January 1, 1990, 0:0:00.
- WKDAY variables are stored as 1 to 7 and MONTH variables as 1 to 12.

You can perform virtually any arithmetic operation with both date format and time format variables. Of course, not all of these operations are particularly useful. You can calculate the number of days between two dates by subtracting one date from the other—but adding two dates does not produce a very meaningful result.

By default, any new numeric variables that you compute are displayed in F format. In the case of calculations involving time and date variables, this means that the default output is expressed as a number of seconds. Use the FORMATS (or PRINT FORMATS) command to specify an appropriate format for the computed variable.

Example

```
DATA LIST FREE /Date1 Date2 (2ADATE10).
BEGIN DATA
6/20/2006 10/28/2006
END DATA.
COMPUTE DateDiff1=(Date2-Date1)/60/60/24.
COMPUTE DateDiff2=DATEDIFF(Date2,Date1, "days").
COMPUTE FutureDate1=Date2+(10*60*60*24).
COMPUTE FutureDate2=DATESUM(Date2, 10, "days").
FORMATS FutureDate1 FutureDate2 (ADATE10).
```

- The first two COMPUTE commands both calculate the number of days between two dates. In the first one, Date2-Date1 yields the number of seconds between the two dates, which is then converted to the number of days by dividing by number of seconds in a minute, number of minutes in an hour, and number of hours in a day. In the second one, the DATEDIFF function is used to obtain the equivalent result, but instead of an arithmetic formula to produce a result expressed in days, it simply includes the argument "days".
- The second pair of COMPUTE commands both calculate a date 10 days from *Date2*. In the first one, 10 days needs to be converted to the number of seconds in ten days before it can be added to *Date2*. In the second one, the "days" argument in the DATESUM function handles that conversion.
- The FORMATS command is used to display the results of the second two COMPUTE commands as dates, since the default format is F, which would display the results as the number of seconds since October 14, 1582.

For more information on date and time functions, see “Date and time functions” on page 72 .

Conditional statements and case selection based on dates

To specify a date as a value in a conditional statement, use one of the data aggregation functions to express the date value. For example,

```
***this works***.
SELECT IF datevar >= date.mdy(3,1,2006).
***the following do not work***.
SELECT IF datevar >= 3/1/2006. /*this will select dates >= 0.0015.
SELECT IF datevar >= "3/1/2006" /*this will generate an error.
```

See the topic “Aggregation functions” on page 72 for more information.

Domain errors

Domain errors occur when numeric expressions are mathematically undefined or cannot be represented numerically on the computer for reasons other than missing data. Two common examples are division by 0 and the square root of a negative number. When there is a domain error, a warning is issued, and the

system-missing value is assigned to the expression. For example, the command `COMPUTE TESTVAR = TRUNC(SQRT(X/Y) * .5)` returns system-missing if *X/Y* is negative or if *Y* is 0.

The following are domain errors in numeric expressions:

**. *A negative number to a noninteger power.*

/. *A divisor of 0.*

MOD. *A divisor of 0.*

SQRT . *A negative argument.*

EXP. *An argument that produces a result too large to be represented on the computer.*

LG10. *A negative or 0 argument.*

LN. *A negative or 0 argument.*

ARSIN. *An argument whose absolute value exceeds 1.*

NORMAL. *A negative or 0 argument.*

PROBIT. *A negative or 0 argument, or an argument 1 or greater.*

Numeric functions

Numeric functions can be used in any numeric expression on `IF`, `SELECT IF`, `DO IF`, `ELSE IF`, `LOOP IF`, `END LOOP IF`, and `COMPUTE` commands. Numeric functions always return numbers (or the system-missing value whenever the result is indeterminate). The expression to be transformed by a function is called the *argument*. Most functions have a variable or a list of variables as arguments.

- In numeric functions with two or more arguments, each argument must be separated by a comma. Blanks alone cannot be used to separate variable names, expressions, or constants in transformation expressions.
- Arguments should be enclosed in parentheses, as in `TRUNC(INCOME)`, where the `TRUNC` function returns the integer portion of the variable *INCOME*.
- Multiple arguments should be separated by commas, as in `MEAN(Q1,Q2,Q3)`, where the `MEAN` function returns the mean of variables *Q1*, *Q2*, and *Q3*.

Example

```
COMPUTE Square_Root = SQRT(var4).
COMPUTE Remainder = MOD(var4, 3).
COMPUTE Average = MEAN.3(var1, var2, var3, var4).
COMPUTE Trunc_Mean = TRUNC(MEAN(var1 TO var4)).
```

- `SQRT(var4)` returns the square root of the value of *var4* for each case.
- `MOD(var4, 3)` returns the remainder (modulus) from dividing the value of *var4* by 3.
- `MEAN.3(var1, var2, var3, var4)` returns the mean of the four specified variables, provided that at least three of them have nonmissing values. The divisor for the calculation of the mean is the number of nonmissing values.
- `TRUNC(MEAN(var1 TO var4))` computes the mean of the values for the inclusive range of variables and then truncates the result. Since no minimum number of nonmissing values is specified for the function, a mean will be calculated (and truncated) as long as at least one of the variables has a nonmissing value for that case.

Arithmetic functions

- All arithmetic functions except `MOD`, `RND` and `TRUNC` have single arguments; `MOD` has two while `RND` and `TRUNC` have from one to three. Multiple arguments must be separated by a comma.
- Arguments can be numeric expressions, as in `RND(A**2/B)`.

ABS. ABS(numexpr). Numeric. Returns the absolute value of numexpr, which must be numeric.

RND. RND(numexpr[,mult,fuzzbits]). Numeric. With a single argument, returns the integer nearest to that argument. Numbers ending in .5 exactly are rounded away from 0. For example, RND(-4.5) rounds to -5. The optional second argument, mult, specifies that the result is an integer multiple of this value—for example, RND(-4.57,0.1) = -4.6. The value must be numeric but cannot be 0. The default is 1.

The optional third argument, fuzzbits, is the number of least-significant bits by which the internal representation of numexpr (expressed as a 64-bit floating point binary) may fall short of the threshold for rounding up (e.g., 0.5 when rounding to an integer) but still be rounded up. For example, the sum 9.62 - 5.82 - 9.21 + 6.91 has an internal representation of 1.499999999999998 (on an Intel processor). With fuzzbits set to 0 and mult set to 1, this expression will round to 1.0, although the exact sum is 1.50 which would round to 2.0. Allowing the rounding threshold to have a small fuzziness compensates for the minute differences between calculations with floating point numbers and exact results. In this case, adding a fuzziness of 4 bits is sufficient to produce the expected result of 2.0.

If the argument fuzzbits is omitted, the value specified by SET FUZZBITS is used. The installed setting of FUZZBITS is 6, which should be sufficient for most applications. Setting fuzzbits to 0 produces the same results as in release 10. Setting fuzzbits to 10 produces the same results as in releases 11 and 12.

To produce the same results as in release 13, use the following expression in place of the RND function:

```
TRUNC(numexpr,1,0) + ((.5+TRUNC(numexpr,1,0)-numexpr)<max(1e-13,min(.5,numexpr*1e-13)))
```

To produce the same results as in releases 14, 15, and 16 use:

```
RND(numexpr,1,12.5-ln(max(1e-50,abs(numexpr)))/ln(2))
```

TRUNC. TRUNC(numexpr[,mult,fuzzbits]). Numeric. Returns the value of numexpr truncated toward 0. The optional second argument, mult, specifies that the result is an integer multiple of this value—for example, TRUNC(4.579,0.1) = 4.5. The value must be numeric but cannot be 0. The default is 1.

The optional third argument, fuzzbits, is the number of least-significant bits by which the internal representation of numexpr (expressed as a 64-bit floating point binary) may fall short of the nearest rounding boundary and be rounded up before truncating. For example, the sum 9.62 - 5.82 - 9.21 + 6.91 has an internal representation of 1.499999999999998 (on an Intel processor). With fuzzbits set to 0 and mult set to 0.1, this expression will truncate to 1.4, although the exact sum is 1.50 which would truncate to 1.5. Adding a small fuzziness to the nearest rounding boundary (in this case, 1.5) compensates for the minute differences between calculations with floating point numbers and exact results. In this case, adding a fuzziness of 5 bits is sufficient to produce the expected result of 1.5.

If the argument fuzzbits is omitted, the value specified by SET FUZZBITS is used. The installed setting of FUZZBITS is 6, which should be sufficient for most applications. Setting fuzzbits to 0 produces the same results as in release 10. Setting fuzzbits to 10 produces the same results as in releases 11 and 12.

To produce the same results as in release 13 use:

```
TRUNC(numexpr,1,0)+(TRUNC(numexpr,1,0)+1-numexpr <= 1e-13)
```

To produce the same results as in releases 14, 15, and 16 use:

```
TRUNC(numexpr,1,12.5-ln(max(1e-50,abs(numexpr)))/ln(2))
```

MOD. MOD(numexpr,modulus). Numeric. Returns the remainder when numexpr is divided by modulus. Both arguments must be numeric, and modulus must not be 0.

SQRT. SQRT(numexpr). Numeric. Returns the positive square root of numexpr, which must be numeric and not negative.

EXP. EXP(numexpr). Numeric. Returns e raised to the power numexpr, where e is the base of the natural logarithms and numexpr is numeric. Large values of numexpr may produce results that exceed the capacity of the machine.

LG10. LG10(numexpr). Numeric. Returns the base-10 logarithm of numexpr, which must be numeric and greater than 0.

LN. LN(numexpr). Numeric. Returns the base-e logarithm of numexpr, which must be numeric and greater than 0.

LNGAMMA. LNGAMMA(numexpr). Numeric. Returns the logarithm of the complete Gamma function of numexpr, which must be numeric and greater than 0.

ARSIN. ARSIN(numexpr). Numeric. Returns the inverse sine (arcsine), in radians, of numexpr, which must evaluate to a numeric value between -1 and +1.

ARTAN. ARTAN(numexpr). Numeric. Returns the inverse tangent (arctangent), in radians, of numexpr, which must be numeric.

SIN. SIN(radians). Numeric. Returns the sine of radians, which must be a numeric value, measured in radians.

COS. COS(radians). Numeric. Returns the cosine of radians, which must be a numeric value, measured in radians.

Statistical functions

- Each argument to a statistical function (expression, variable name, or constant) must be separated by a comma.
- The *.n* suffix can be used with all statistical functions to specify the number of valid arguments. For example, `MEAN.2(A,B,C,D)` returns the mean of the valid values for variables *A*, *B*, *C*, and *D* only if at least two of the variables have valid values. The default for *n* is 2 for SD, VARIANCE, and CFVAR and 1 for other statistical functions. If the number specified exceeds the number of arguments in the function, the result is system-missing.
- The keyword TO can be used to refer to a set of variables in the argument list.

SUM. SUM(numexpr,numexpr[,..]). Numeric. Returns the sum of its arguments that have valid, nonmissing values. This function requires two or more arguments, which must be numeric. You can specify a minimum number of valid arguments for this function to be evaluated.

MEAN. MEAN(numexpr,numexpr[,..]). Numeric. Returns the arithmetic mean of its arguments that have valid, nonmissing values. This function requires two or more arguments, which must be numeric. You can specify a minimum number of valid arguments for this function to be evaluated.

MEDIAN. MEDIAN(numexpr,numexpr[,..]). Numeric. Returns the median (50th percentile) of its arguments that have valid, nonmissing values. This function requires two or more arguments, which must be numeric. You can specify a minimum number of valid arguments for this function to be evaluated.

SD. SD(numexpr,numexpr[,..]). Numeric. Returns the standard deviation of its arguments that have valid, nonmissing values. This function requires two or more arguments, which must be numeric. You can specify a minimum number of valid arguments for this function to be evaluated.

VARIANCE. VARIANCE(numexpr,numexpr[,..]). Numeric. Returns the variance of its arguments that have valid values. This function requires two or more arguments, which must be numeric. You can specify a minimum number of valid arguments for this function to be evaluated.

CFVAR. CFVAR(numexpr,numexpr[,...]). Numeric. Returns the coefficient of variation (the standard deviation divided by the mean) of its arguments that have valid values. This function requires two or more arguments, which must be numeric. You can specify a minimum number of valid arguments for this function to be evaluated.

MIN. MIN(value,value[,..]). Numeric or string. Returns the minimum value of its arguments that have valid, nonmissing values. This function requires two or more arguments. For numeric values, you can specify a minimum number of valid arguments for this function to be evaluated.

MAX. MAX(value,value[,..]). Numeric or string. Returns the maximum value of its arguments that have valid values. This function requires two or more arguments. For numeric values, you can specify a minimum number of valid arguments for this function to be evaluated.

Example

```
COMPUTE maxsum=MAX.2(SUM(var1 TO var3), SUM(var4 TO var6)).
```

- MAX.2 will return the maximum of the two sums provided that both sums are nonmissing.
- The .2 refers to the number of nonmissing arguments for the MAX function, which has only two arguments because each SUM function is considered a single argument.
- The new variable *maxsum* will be nonmissing if at least one variable specified for each SUM function is nonmissing.

Random variable and distribution functions

Random variable and distribution function keywords are all of the form `prefix.suffix`, where the prefix specifies the function to be applied to the distribution and the suffix specifies the distribution.

- Random variable and distribution functions take both constants and variables for arguments.
- A function argument, if required, must come first and is denoted by *x* (quantile, which must fall in the range of values for the distribution) for cumulative distribution and probability density functions and *p* (probability) for inverse distribution functions.
- All random variable and distribution functions must specify distribution parameters as noted in their definitions.
- All arguments are real numbers.
- Restrictions to distribution parameters apply to all functions for that distribution. Restrictions for the function parameter *x* apply to that particular distribution function. The program issues a warning and returns system-missing when it encounters an out-of-range value for an argument.

The following are possible prefixes:

CDF. *Cumulative distribution function.* A cumulative distribution function `CDF.d_spec(x,a,...)` returns a probability *p* that a variate with the specified distribution (d_spec) falls below *x* for continuous functions and at or below *x* for discrete functions.

IDF. *Inverse distribution function.* Inverse distribution functions are not available for discrete distributions. An inverse distribution function `IDF.d_spec(p,a,...)` returns a value *x* such that `CDF.d_spec(x,a,...)`= *p* with the specified distribution (d_spec).

PDF. *Probability density function.* A probability density function `PDF.d_spec(x,a,...)` returns the density of the specified distribution (d_spec) at *x* for continuous functions and the probability that a random variable with the specified distribution equals *x* for discrete functions.

RV. *Random number generation function.* A random number generation function `RV.d_spec(a,...)` generates an independent observation with the specified distribution (d_spec).

NCDF. *Noncentral cumulative distribution function.* A noncentral distribution function `NCDF.d_spec(x,a,b,...)` returns a probability *p* that a variate with the specified noncentral distribution falls below *x*. It is available only for beta, chi-square, *F*, and Student's *t*.

NPDF. *Noncentral probability density function.* A noncentral probability density function `NCDF.d_spec(x,a,...)` returns the density of the specified distribution (d_spec) at *x*. It is available only for beta, chi-square, *F*, and Student's *t*.

SIG. *Tail probability function.* A tail probability function `SIG.d_spec(x,a,...)` returns a probability *p* that a variate with the specified distribution (d_spec) is larger than *x.* The tail probability function is equal to 1 minus the cumulative distribution function.

The following are suffixes for continuous distributions:

BETA. *Beta distribution.* The beta distribution takes values in the range 0<*x*<1 and has two shape parameters, α and β. Both α and β must be positive, and they have the property that the mean of the distribution is α/(α+β).

Noncentral beta distribution. The noncentral beta distribution is a generalization of the beta distribution that takes values in the range 0<*x*<1 and has an extra noncentrality parameter, λ, which must be greater than or equal to 0.

BVNOR. *Bivariate normal distribution.* The bivariate normal distribution takes real values and has one correlation parameter, ρ, which must be between –1 and 1, inclusive.

CAUCHY. *Cauchy distribution.* The Cauchy distribution takes real values and has a location parameter, θ, and a scale parameter, ς; ς must be positive. The Cauchy distribution is symmetric about the location parameter, but has such slowly decaying tails that the distribution does not have a computable mean.

CHISQ. *Chi-square distribution.* The chi-square(v) distribution takes values in the range x>=0 and has one degrees of freedom parameter, v; it must be positive and has the property that the mean of the distribution is v.

Noncentral chi-square distribution. The noncentral chi-square distribution is a generalization of the chi-square distribution that takes values in the range x>=0 and has an extra noncentrality parameter, λ, which must be greater than or equal to 0.

EXP. *Exponential distribution.* The exponential distribution takes values in the range x>=0 and has one scale parameter, β, which must be greater than 0 and has the property that the mean of the distribution is 1/β.

F. F *distribution.* The F distribution takes values in the range x>=0 and has two degrees of freedom parameters, v1 and v2, which are the "numerator" and "denominator" degrees of freedom, respectively. Both v1 and v2 must be positive.

Noncentral F distribution. The noncentral F distribution is a generalization of the F distribution that takes values in the range x>=0 and has an extra noncentrality parameter, λ, which must be greater than or equal to 0.

GAMMA. *Gamma distribution.* The gamma distribution takes values in the range x>=0 and has one shape parameter, α, and one scale parameter, β. Both parameters must be positive and have the property that the mean of the distribution is α/β.

HALFNRM. *Half-normal distribution.* The half-normal distribution takes values in the range x>=μ and has one location parameter, μ, and one scale parameter, σ. Parameter σ must be positive.

IGAUSS. *Inverse Gaussian distribution.* The inverse Gaussian, or Wald, distribution takes values in the range x>0 and has two parameters, μ and λ, both of which must be positive. The distribution has mean μ.

LAPLACE. *Laplace or double exponential distribution.* The Laplace distribution takes real values and has one location parameter, μ, and one scale parameter, β. Parameter β must be positive. The distribution is symmetric about μ and has exponentially decaying tails.

LOGISTIC. *Logistic distribution.* The logistic distribution takes real values and has one location parameter, μ, and one scale parameter, ς. Parameter ς must be positive. The distribution is symmetric about μ and has longer tails than the normal distribution.

LNORMAL. *Lognormal distribution.* The lognormal distribution takes values in the range x>=0 and has two parameters, η and σ, both of which must be positive.

NORMAL. *Normal distribution.* The normal, or Gaussian, distribution takes real values and has one location parameter, μ, and one scale parameter, σ. Parameter σ must be positive. The distribution has mean μ and standard deviation σ.

Three functions in releases earlier than 6.0 are special cases of the normal distribution functions: `CDFNORM(arg)=CDF.NORMAL(x,0,1)`, where `arg` is *x*; `PROBIT(arg)=IDF.NORMAL(p,0,1)`, where `arg` is *p*; and `NORMAL(arg)=RV.NORMAL(0,`*σ*`)`, where `arg` is *σ*.

PARETO. *Pareto distribution.* The Pareto distribution takes values in the range xmin<x and has a threshold parameter, xmin, and a shape parameter, α. Both parameters must be positive.

SMOD. *Studentized maximum modulus distribution.* The Studentized maximum modulus distribution takes values in the range x>0 and has a number of comparisons parameter, k*, and degrees of freedom parameter, v, both of which must be greater than or equal to 1.

SRANGE. *Studentized range distribution.* The Studentized range distribution takes values in the range x>0 and has a number of samples parameter, k, and degrees of freedom parameter, v, both of which must be greater than or equal to 1.

T. *Student* t *distribution.* The Student t distribution takes real values and has one degrees of freedom parameter, v, which must be positive. The Student t distribution is symmetric about 0.

Noncentral t distribution. The noncentral t distribution is a generalization of the t distribution that takes real values and has an extra noncentrality parameter, λ, which must be greater than or equal to 0. When λ equals 0, this distribution reduces to the t distribution.

UNIFORM. *Uniform distribution.* The uniform distribution takes values in the range a<x<b and has a minimum value parameter, a, and a maximum value parameter, b.

The uniform random number function in releases earlier than 6.0 is a special case: `UNIFORM(arg)=RV.UNIFORM(0,b)`, where `arg` is parameter *b.* Among other uses, the uniform distribution commonly models the round-off error.

WEIBULL. *Weibull distribution.* The Weibull distribution takes values in the range x>=0 and has one scale parameter, β, and one shape parameter, α, both of which must be positive.

The following are suffixes for discrete distributions:

BERNOULLI. *Bernoulli distribution.* The Bernoulli distribution takes values 0 or 1 and has one success probability parameter, θ, which must be between 0 and 1, inclusive.

BINOM. *Binomial distribution.* The binomial distribution takes integer values 0<=x<=n, representing the number of successes in n trials, and has one number of trials parameter, n, and one success probability parameter, θ. Parameter n must be a positive integer and parameter θ must be between 0 and 1, inclusive.

GEOM. *Geometric distribution.* The geometric distribution takes integer values x>=1, representing the number of trials needed (including the last trial) before a success is observed, and has one success probability parameter, θ, which must be between 0 and 1, inclusive.

HYPER. *Hypergeometric distribution.* The hypergeometric distribution takes integer values in the range max(0, Np+n−N)<=x<=min(Np,n), and has three parameters, N, n, and Np, where N is the total number of objects in an urn model, n is the number of objects randomly drawn without replacement from the urn, Np is the number of objects with a given characteristic, and x is the number of objects with the given characteristic observed out of the withdrawn objects. All three parameters are positive integers, and both n and Np must be less than or equal to N.

NEGBIN. *Negative binomial distribution.* The negative binomial distribution takes integer values in the range x>=r, where x is the number of trials needed (including the last trial) before r successes are observed, and has one threshold parameter, r, and one success probability parameter, θ. Parameter r must be a positive integer and parameter θ must be greater than 0 and less than or equal to 1.

POISSON. *Poisson distribution.* The Poisson distribution takes integer values in the range x>=0 and has one rate or mean parameter, λ. Parameter λ must be positive.

Probability Density Functions

The following functions give the value of the density function with the specified distribution at the value *quant*, the first argument. Subsequent arguments are the parameters of the distribution. Note the period in each function name.

PDF.BERNOULLI. PDF.BERNOULLI(quant, prob). Numeric. Returns the probability that a value from the Bernoulli distribution, with the given probability parameter, will be equal to quant.

PDF.BETA. PDF.BETA(quant, shape1, shape2). Numeric. Returns the probability density of the beta distribution, with the given shape parameters, at quant.

PDF.BINOM. PDF.BINOM(quant, n, prob). Numeric. Returns the probability that the number of successes in n trials, with probability prob of success in each, will be equal to quant. When n is 1, this is the same as PDF.BERNOULLI.

PDF.BVNOR. PDF.BVNOR(quant1, quant2, corr). Numeric. Returns the probability density of the standard bivariate normal distribution, with the given correlation parameter, at quant1, quant2.

PDF.CAUCHY. PDF.CAUCHY(quant, loc, scale). Numeric. Returns the probability density of the Cauchy distribution, with the given location and scale parameters, at quant.

PDF.CHISQ. PDF.CHISQ(quant, df). Numeric. Returns the probability density of the chi-square distribution, with df degrees of freedom, at quant.

PDF.EXP. PDF.EXP(quant, shape). Numeric. Returns the probability density of the exponential distribution, with the given shape parameter, at quant.

PDF.F. PDF.F(quant, df1, df2). Numeric. Returns the probability density of the F distribution, with degrees of freedom df1 and df2, at quant.

PDF.GAMMA. PDF.GAMMA(quant, shape, scale). Numeric. Returns the probability density of the gamma distribution, with the given shape and scale parameters, at quant.

PDF.GEOM. PDF.GEOM(quant, prob). Numeric. Returns the probability that the number of trials to obtain a success, when the probability of success is given by prob, will be equal to quant.

PDF.HALFNRM. PDF.HALFNRM(quant, mean, stddev). Numeric. Returns the probability density of the half normal distribution, with specified mean and standard deviation, at quant.

PDF.HYPER. PDF.HYPER(quant, total, sample, hits). Numeric. Returns the probability that the number of objects with a specified characteristic, when sample objects are randomly selected from a universe of size total in which hits have the specified characteristic, will be equal to quant.

PDF.IGAUSS. PDF.IGAUSS(quant, loc, scale). Numeric. Returns the probability density of the inverse Gaussian distribution, with the given location and scale parameters, at quant.

PDF.LAPLACE. PDF.LAPLACE(quant, mean, scale). Numeric. Returns the probability density of the Laplace distribution, with the specified mean and scale parameters, at quant.

PDF.LOGISTIC. PDF.LOGISTIC(quant, mean, scale). Numeric. Returns the probability density of the logistic distribution, with the specified mean and scale parameters, at quant.

PDF.LNORMAL. PDF.LNORMAL(quant, a, b). Numeric. Returns the probability density of the log-normal distribution, with the specified parameters, at quant.

PDF.NEGBIN. PDF.NEGBIN(quant, thresh, prob). Numeric. Returns the probability that the number of trials to obtain a success, when the threshold parameter is thresh and the probability of success is given by prob, will be equal to quant.

PDF.NORMAL. PDF.NORMAL(quant, mean, stddev). Numeric. Returns the probability density of the normal distribution, with specified mean and standard deviation, at quant.

PDF.PARETO. PDF.PARETO(quant, threshold, shape). Numeric. Returns the probability density of the Pareto distribution, with the specified threshold and shape parameters, at quant.

PDF.POISSON. PDF.POISSON(quant, mean). Numeric. Returns the probability that a value from the Poisson distribution, with the specified mean or rate parameter, will be equal to quant.

PDF.T. PDF.T(quant, df). Numeric. Returns the probability density of Student's t distribution, with the specified degrees of freedom df, at quant.

PDF.UNIFORM. PDF.UNIFORM(quant, min, max). Numeric. Returns the probability density of the uniform distribution, with the specified minimum and maximum, at quant.

PDF.WEIBULL. PDF.WEIBULL(quant, a, b). Numeric. Returns the probability density of the Weibull distribution, with the specified parameters, at quant.

NPDF.BETA. NPDF.BETA(quant, shape1, shape2, nc). Numeric. Returns the probability density of the noncentral beta distribution, with the given shape and noncentrality parameters, at quant.

NPDF.CHISQ. NPDF.CHISQ(quant, df, nc). Numeric. Returns the probability density of the noncentral chi-square distribution, with df degrees of freedom and the specified noncentrality parameter, at quant.

NPDF.F. NPDF.F(quant, df1, df2, nc). Numeric. Returns the probability density of the noncentral F distribution, with degrees of freedom df1 and df2 and noncentrality nc, at quant.

NPDF.T. NPDF.T(quant, df, nc). Numeric. Returns the probability density of the noncentral Student's t distribution, with the specified degrees of freedom df and noncentrality nc, at quant.

Tail probability functions

The following functions give the probability that a random variable with the specified distribution will be greater than *quant*, the first argument. Subsequent arguments are the parameters of the distribution. Note the period in each function name.

SIG.CHISQ. SIG.CHISQ(quant, df). Numeric. Returns the cumulative probability that a value from the chi-square distribution, with df degrees of freedom, will be greater than quant

SIG.F. SIG.F(quant, df1, df2). Numeric. Returns the cumulative probability that a value from the F distribution, with degrees of freedom df1 and df2, will be greater than quant.

Cumulative distribution functions

The following functions give the probability that a random variable with the specified distribution will be less than *quant*, the first argument. Subsequent arguments are the parameters of the distribution. Note the period in each function name.

CDF.BERNOULLI. CDF.BERNOULLI(quant, prob). Numeric. Returns the cumulative probability that a value from the Bernoulli distribution, with the given probability parameter, will be less than or equal to quant.

CDF.BETA. CDF.BETA(quant, shape1, shape2). Numeric. Returns the cumulative probability that a value from the Beta distribution, with the given shape parameters, will be less than quant.

CDF.BINOM. CDF.BINOM(quant, n, prob). Numeric. Returns the cumulative probability that the number of successes in n trials, with probability prob of success in each, will be less than or equal to quant. When n is 1, this is the same as CDF.BERNOULLI.

CDF.BVNOR. CDF.BVNOR(quant1, quant2, corr). Numeric. Returns the cumulative probability that a value from the standard bivariate normal distribution, with the given correlation parameter, will be less than quant1 and quant2.

CDF.CAUCHY. CDF.CAUCHY(quant, loc, scale). Numeric. Returns the cumulative probability that a value from the Cauchy distribution, with the given location and scale parameters, will be less than quant.

CDF.CHISQ. CDF.CHISQ(quant, df). Numeric. Returns the cumulative probability that a value from the chi-square distribution, with df degrees of freedom, will be less than quant.

CDF.EXP. CDF.EXP(quant, scale). Numeric. Returns the cumulative probability that a value from the exponential distribution, with the given scale parameter, will be less than quant.

CDF.F. CDF.F(quant, df1, df2). Numeric. Returns the cumulative probability that a value from the F distribution, with degrees of freedom df1 and df2, will be less than quant.

CDF.GAMMA. CDF.GAMMA(quant, shape, scale). Numeric. Returns the cumulative probability that a value from the Gamma distribution, with the given shape and scale parameters, will be less than quant.

CDF.GEOM. CDF.GEOM(quant, prob). Numeric. Returns the cumulative probability that the number of trials to obtain a success, when the probability of success is given by prob, will be less than or equal to quant.

CDF.HALFNRM. CDF.HALFNRM(quant, mean, stddev). Numeric. Returns the cumulative probability that a value from the half normal distribution, with specified mean and standard deviation, will be less than quant.

CDF.HYPER. CDF.HYPER(quant, total, sample, hits). Numeric. Returns the cumulative probability that the number of objects with a specified characteristic, when sample objects are randomly selected from a universe of size total in which hits have the specified characteristic, will be less than or equal to quant.

CDF.IGAUSS. CDF.IGAUSS(quant, loc, scale). Numeric. Returns the cumulative probability that a value from the inverse Gaussian distribution, with the given location and scale parameters, will be less than quant.

CDF.LAPLACE. CDF.LAPLACE(quant, mean, scale). Numeric. Returns the cumulative probability that a value from the Laplace distribution, with the specified mean and scale parameters, will be less than quant.

CDF.LOGISTIC. CDF.LOGISTIC(quant, mean, scale). Numeric. Returns the cumulative probability that a value from the logistic distribution, with the specified mean and scale parameters, will be less than quant.

CDF.LNORMAL. CDF.LNORMAL(quant, a, b). Numeric. Returns the cumulative probability that a value from the log-normal distribution, with the specified parameters, will be less than quant.

CDF.NEGBIN. CDF.NEGBIN(quant, thresh, prob). Numeric. Returns the cumulative probability that the number of trials to obtain a success, when the threshold parameter is thresh and the probability of success is given by prob, will be less than or equal to quant.

CDFNORM. CDFNORM(zvalue). Numeric. Returns the probability that a random variable with mean 0 and standard deviation 1 would be less than zvalue, which must be numeric.

CDF.NORMAL. CDF.NORMAL(quant, mean, stddev). Numeric. Returns the cumulative probability that a value from the normal distribution, with specified mean and standard deviation, will be less than quant.

CDF.PARETO. CDF.PARETO(quant, threshold, shape). Numeric. Returns the cumulative probability that a value from the Pareto distribution, with the specified threshold and shape parameters, will be less than quant.

CDF.POISSON. CDF.POISSON(quant, mean). Numeric. Returns the cumulative probability that a value from the Poisson distribution, with the specified mean or rate parameter, will be less than or equal to quant.

CDF.SMOD. CDF.SMOD(quant, a, b). Numeric. Returns the cumulative probability that a value from the Studentized maximum modulus, with the specified parameters, will be less than quant.

CDF.SRANGE. CDF.SRANGE(quant, a, b). Numeric. Returns the cumulative probability that a value from the Studentized range statistic, with the specified parameters, will be less than quant.

CDF.T. CDF.T(quant, df). Numeric. Returns the cumulative probability that a value from Student's t distribution, with the specified degrees of freedom df, will be less than quant.

CDF.UNIFORM. CDF.UNIFORM(quant, min, max). Numeric. Returns the cumulative probability that a value from the uniform distribution, with the specified minimum and maximum, will be less than quant.

CDF.WEIBULL. CDF.WEIBULL(quant, a, b). Numeric. Returns the cumulative probability that a value from the Weibull distribution, with the specified parameters, will be less than quant.

NCDF.BETA. NCDF.BETA(quant, shape1, shape2, nc). Numeric. Returns the cumulative probability that a value from the noncentral Beta distribution, with the given shape and noncentrality parameters, will be less than quant.

NCDF.CHISQ. NCDF.CHISQ(quant, df, nc). Numeric. Returns the cumulative probability that a value from the noncentral chi-square distribution, with df degrees of freedom and the specified noncentrality parameter, will be less than quant.

NCDF.F. NCDF.F(quant, df1, df2, nc). Numeric. Returns the cumulative probability that a value from the noncentral F distribution, with degrees of freedom df1 and df2, and noncentrality nc, will be less than quant.

NCDF.T. NCDF.T(quant, df, nc). Numeric. Returns the cumulative probability that a value from the noncentral Student's t distribution, with the specified degrees of freedom df and noncentrality nc, will be less than quant.

Inverse distribution functions

The following functions give the value in a specified distribution having a cumulative probability equal to *prob*, the first argument. Subsequent arguments are the parameters of the distribution. Note the period in each function name.

IDF.BETA. IDF.BETA(prob, shape1, shape2). Numeric. Returns the value from the Beta distribution, with the given shape parameters, for which the cumulative probability is prob.

IDF.CAUCHY. IDF.CAUCHY(prob, loc, scale). Numeric. Returns the value from the Cauchy distribution, with the given location and scale parameters, for which the cumulative probability is prob.

IDF.CHISQ. IDF.CHISQ(prob, df). Numeric. Returns the value from the chi-square distribution, with the specified degrees of freedom df, for which the cumulative probability is prob. For example, the chi-square value that is significant at the 0.05 level with 3 degrees of freedom is IDF.CHISQ(0.95,3).

IDF.EXP. IDF.EXP(p, scale). Numeric. Returns the value of an exponentially decaying variable, with rate of decay scale, for which the cumulative probability is p.

IDF.F. IDF.F(prob, df1, df2). Numeric. Returns the value from the F distribution, with the specified degrees of freedom, for which the cumulative probability is prob. For example, the F value that is significant at the 0.05 level with 3 and 100 degrees of freedom is IDF.F(0.95,3,100).

IDF.GAMMA. IDF.GAMMA(prob, shape, scale). Numeric. Returns the value from the Gamma distribution, with the specified shape and scale parameters, for which the cumulative probability is prob.

IDF.HALFNRM. IDF.HALFNRM(prob, mean, stddev). Numeric. Returns the value from the half normal distribution, with the specified mean and standard deviation, for which the cumulative probability is prob.

IDF.IGAUSS. IDF.IGAUSS(prob, loc, scale). Numeric. Returns the value from the inverse Gaussian distribution, with the given location and scale parameters, for which the cumulative probability is prob.

IDF.LAPLACE. IDF.LAPLACE(prob, mean, scale). Numeric. Returns the value from the Laplace distribution, with the specified mean and scale parameters, for which the cumulative probability is prob.

IDF.LOGISTIC. IDF.LOGISTIC(prob, mean, scale). Numeric. Returns the value from the logistic distribution, with specified mean and scale parameters, for which the cumulative probability is prob.

IDF.LNORMAL. IDF.LNORMAL(prob, a, b). Numeric. Returns the value from the log-normal distribution, with specified parameters, for which the cumulative probability is prob.

IDF.NORMAL. IDF.NORMAL(prob, mean, stddev). Numeric. Returns the value from the normal distribution, with specified mean and standard deviation, for which the cumulative probability is prob.

IDF.PARETO. IDF.PARETO(prob, threshold, shape). Numeric. Returns the value from the Pareto distribution, with specified threshold and scale parameters, for which the cumulative probability is prob.

IDF.SMOD. IDF.SMOD(prob, a, b). Numeric. Returns the value from the Studentized maximum modulus, with the specified parameters, for which the cumulative probability is prob.

IDF.SRANGE. IDF.SRANGE(prob, a, b). Numeric. Returns the value from the Studentized range statistic, with the specified parameters, for which the cumulative probability is prob.

IDF.T. IDF.T(prob, df). Numeric. Returns the value from Student's t distribution, with specified degrees of freedom df, for which the cumulative probability is prob.

IDF.UNIFORM. IDF.UNIFORM(prob, min, max). Numeric. Returns the value from the uniform distribution between min and max for which the cumulative probability is prob.

IDF.WEIBULL. IDF.WEIBULL(prob, a, b). Numeric. Returns the value from the Weibull distribution, with specified parameters, for which the cumulative probability is prob.

PROBIT. PROBIT(prob). Numeric. Returns the value in a standard normal distribution having a cumulative probability equal to prob. The argument prob is a probability greater than 0 and less than 1.

Random variable functions

The following functions give a random variate from a specified distribution. The arguments are the parameters of the distribution. You can repeat the sequence of pseudorandom numbers by setting a seed in the Preferences dialog box before each sequence. Note the period in each function name.

NORMAL. NORMAL(stddev). Numeric. Returns a normally distributed pseudorandom number from a distribution with mean 0 and standard deviation stddev, which must be a positive number.

RV.BERNOULLI. RV.BERNOULLI(prob). Numeric. Returns a random value from a Bernoulli distribution with the specified probability parameter prob.

RV.BETA. RV.BETA(shape1, shape2). Numeric. Returns a random value from a Beta distribution with specified shape parameters.

RV.BINOM. RV.BINOM(n, prob). Numeric. Returns a random value from a binomial distribution with specified number of trials and probability parameter.

RV.CAUCHY. RV.CAUCHY(loc, scale). Numeric. Returns a random value from a Cauchy distribution with specified location and scale parameters.

RV.CHISQ. RV.CHISQ(df). Numeric. Returns a random value from a chi-square distribution with specified degrees of freedom df.

RV.EXP. RV.EXP(scale). Numeric. Returns a random value from an exponential distribution with specified scale parameter.

RV.F. RV.F(df1, df2). Numeric. Returns a random value from an F distribution with specified degrees of freedom, df1 and df2.

RV.GAMMA. RV.GAMMA(shape, scale). Numeric. Returns a random value from a Gamma distribution with specified shape and scale parameters.

RV.GEOM. RV.GEOM(prob). Numeric. Returns a random value from a geometric distribution with specified probability parameter.

RV.HALFNRM. RV.HALFNRM(mean, stddev). Numeric. Returns a random value from a half normal distribution with the specified mean and standard deviation.

RV.HYPER. RV.HYPER(total, sample, hits). Numeric. Returns a random value from a hypergeometric distribution with specified parameters.

RV.IGAUSS. RV.IGAUSS(loc, scale). Numeric. Returns a random value from an inverse Gaussian distribution with the specified location and scale parameters.

RV.LAPLACE. RV.LAPLACE(mean, scale). Numeric. Returns a random value from a Laplace distribution with specified mean and scale parameters.

RV.LOGISTIC. RV.LOGISTIC(mean, scale). Numeric. Returns a random value from a logistic distribution with specified mean and scale parameters.

RV.LNORMAL. RV.LNORMAL(a, b). Numeric. Returns a random value from a log-normal distribution with specified parameters.

RV.NEGBIN. RV.NEGBIN(threshold, prob). Numeric. Returns a random value from a negative binomial distribution with specified threshold and probability parameters.

RV.NORMAL. RV.NORMAL(mean, stddev). Numeric. Returns a random value from a normal distribution with specified mean and standard deviation.

RV.PARETO. RV.PARETO(threshold, shape). Numeric. Returns a random value from a Pareto distribution with specified threshold and shape parameters.

RV.POISSON. RV.POISSON(mean). Numeric. Returns a random value from a Poisson distribution with specified mean/rate parameter.

RV.T. RV.T(df). Numeric. Returns a random value from a Student's t distribution with specified degrees of freedom df.

RV.UNIFORM. RV.UNIFORM(min, max). Numeric. Returns a random value from a uniform distribution with specified minimum and maximum. See also the UNIFORM function.

WEIBULL. RV.WEIBULL(a, b). Numeric. Returns a random value from a Weibull distribution with specified parameters.

UNIFORM. UNIFORM(max). Numeric. Returns a uniformly distributed pseudorandom number between 0 and the argument max, which must be numeric (but can be negative).

Note: You can repeat the sequence of pseudorandom numbers by setting the same Random Number Seed (available in the Transform menu) before each sequence.

Date and time functions

Date and time functions provide aggregation, conversion, and extraction routines for dates and time intervals. Each function transforms an expression consisting of one or more arguments. Arguments can be complex expressions, variable names, or constants. Date and time expressions and variables are legitimate arguments.

Aggregation functions

Aggregation functions generate date and time intervals from values that were not read by date and time input formats.

- All aggregation functions begin with DATE or TIME, depending on whether a date or a time interval is requested. This is followed by a subfunction that corresponds to the type of values found in the data.
- The subfunctions are separated from the function by a period (.) and are followed by an argument list specified in parentheses.
- The arguments to the DATE and TIME functions must be separated by commas and must resolve to integer values.
- Functions that contain a day argument--for example, `DATE.DMY(d,m,y)`--check the validity of the argument. The value for day must be an integer between 0 and 31. If an invalid value is encountered, a warning is displayed and the value is set to system-missing. However, if the day value is invalid for a particular month—for example, 31 in September, April, June, and November or 29 through 31 for February in nonleap years—the resulting date is placed in the next month. For example `DATE.DMY(31, 9, 2006)` returns the date value for October 1, 2006. A day value of 0 returns the last day of the previous month.

DATE.DMY. DATE.DMY(day,month,year). Numeric. Returns a date value corresponding to the indicated day, month, and year. The arguments must resolve to integers, with day between 0 and 31, month between 1 and 13, and year a four-digit integer greater than 1582. To display the result as a date, assign a date format to the result variable.

DATE.MDY. DATE.MDY(month,day,year). Numeric. Returns a date value corresponding to the indicated month, day, and year. The arguments must resolve to integers, with day between 0 and 31, month between 1 and 13, and year a four-digit integer greater than 1582. To display the result as a date, assign a date format to the result variable.

DATE.MOYR. DATE.MOYR(month,year). Numeric. Returns a date value corresponding to the indicated month and year. The arguments must resolve to integers, with month between 1 and 13, and year a four-digit integer greater than 1582. To display the result as a date, assign a date format to the result variable.

DATE.QYR. DATE.QYR(quarter,year). Numeric. Returns a date value corresponding to the indicated quarter and year. The arguments must resolve to integers, with quarter between 1 and 4, and year a four-digit integer greater than 1582. To display the result as a date, assign a date format to the result variable.

DATE.WKYR. DATE.WKYR(weeknum,year). Numeric. Returns a date value corresponding to the indicated weeknum and year. The arguments must resolve to integers, with weeknum between 1 and 53, and year a four-digit integer greater than 1582. The date value returned represents the first day of the specified week for that year. The first week starts on January 1 of each year; so the date returned for any given week value will differ between years. To display the result as a date, assign a date format to the result variable.

DATE.YRDAY. DATE.YRDAY(year,daynum). Numeric. Returns a date value corresponding to the indicated year and daynum. The arguments must resolve to integers, with daynum between 1 and 366 and with year being a four-digit integer greater than 1582. To display the result as a date, assign a date format to the result variable.

TIME.DAYS. TIME.DAYS(days). Numeric. Returns a time interval corresponding to the indicated number of days. The argument must be numeric. To display the result as a time, assign a time format to the result variable.

TIME.HMS. TIME.HMS(hours[,minutes,seconds]). Numeric. Returns a time interval corresponding to the indicated number of hours, minutes, and seconds. The minutes and seconds arguments are optional.

Minutes and seconds must resolve to numbers less than 60 if any higher-order argument is non-zero. All arguments except the last non-zero argument must resolve to integers. For example TIME.HMS(25.5) and TIME.HMS(0,90,25.5) are valid, while TIME.HMS(25.5,30) and TIME.HMS(25,90) are invalid. All arguments must resolve to either all positive or all negative values. To display the result as a time, assign a time format to the result variable.

Example

```
DATA LIST FREE
  /Year Month Day Hour Minute Second Days.
BEGIN DATA
2006 10 28 23 54 30 1.5
END DATA.
COMPUTE Date1=DATE.DMY(Day, Month, Year).
COMPUTE Date2=DATE.MDY(Month, Day, Year).
COMPUTE MonthYear=DATE.MOYR(Month, Year).
COMPUTE Time=TIME.HMS(Hour, Minute, Second).
COMPUTE Duration=TIME.DAYS(Days).
LIST VARIABLES=Date1 to Duration.
FORMATS
  Date1 (DATE11) Date2 (ADATE10) MonthYear (MOYR8)
  Time (TIME8) Duration (Time8).
LIST VARIABLES=Date1 to Duration.

***LIST Results Before Applying Formats***
       Date1        Date2    MonthYear     Time Duration
 13381372800  13381372800  13379040000    86070   129600

***LIST Results After Applying Formats***
      Date1      Date2 MonthYear     Time Duration
28-OCT-2006 10/28/2006  OCT 2006 23:54:30 36:00:00
```

- Since dates and times are stored internally as a number of seconds, prior to applying the appropriate date or time formats, all the computed values are displayed as numbers that indicate the respective number of seconds.
- The internal values for *Date1* and *Date2* are exactly the same. The only difference between DATE.DMY and DATE.MDY is the order of the arguments.

Date and time conversion functions

The conversion functions convert time intervals from one unit of time to another. Time intervals are stored as the number of seconds in the interval; the conversion functions provide a means for calculating more appropriate units, for example, converting seconds to days.

Each conversion function consists of the CTIME function followed by a period (.), the target time unit, and an argument. The argument can consist of expressions, variable names, or constants. The argument must already be a time interval. See the topic “Aggregation functions” on page 72 for more information. Time conversions produce noninteger results with a default format of F8.2.

Since time and dates are stored internally as seconds, a function that converts to seconds is not necessary.

CTIME.DAYS. CTIME.DAYS(timevalue). Numeric. Returns the number of days, including fractional days, in timevalue, which is a number of seconds, a time expression, or a time format variable.

CTIME.HOURS. CTIME.HOURS(timevalue). Numeric. Returns the number of hours, including fractional hours, in timevalue, which is a number of seconds, a time expression, or a time format variable.

CTIME.MINUTES. CTIME.MINUTES(timevalue). Numeric. Returns the number of minutes, including fractional minutes, in timevalue, which is a number of seconds, a time expression, or a time format variable.

CTIME.SECONDS. CTIME.SECONDS(timevalue). Numeric. Returns the number of seconds, including fractional seconds, in timevalue, which is a number, a time expression, or a time format variable.

Example

```
DATA LIST FREE (",")
  /StartDate (ADATE12) EndDate (ADATE12)
  StartDateTime(DATETIME20) EndDateTime(DATETIME20)
  StartTime (TIME10) EndTime (TIME10).
BEGIN DATA
3/01/2003, 4/10/2003
01-MAR-2003 12:00, 02-MAR-2003 12:00
09:30, 10:15
END DATA.
COMPUTE days = CTIME.DAYS(EndDate-StartDate).
```

```
COMPUTE hours = CTIME.HOURS(EndDateTime-StartDateTime).
COMPUTE minutes = CTIME.MINUTES(EndTime-StartTime).
```

- CTIME.DAYS calculates the difference between *EndDate* and *StartDate* in days—in this example, 40 days.
- CTIME.HOURS calculates the difference between *EndDateTime* and *StartDateTime* in hours—in this example, 24 hours.
- CTIME.MINUTES calculates the difference between *EndTime* and *StartTime* in minutes—in this example, 45 minutes.

YRMODA function

YRMODA(arg list). *Convert year, month, and day to a day number.* The number returned is the number of days since October 14, 1582 (day 0 of the Gregorian calendar).

- Arguments for YRMODA can be variables, constants, or any other type of numeric expression but must yield integers.
- Year, month, and day must be specified in that order.
- The first argument can be any year between 0 and 99, or between 1582 to 47516.
- If the first argument yields a number between 00 and 99, 1900 through 1999 is assumed.
- The month can range from 1 through 13. Month 13 with day 0 yields the last day of the year. For example, YRMODA(1990,13,0) produces the day number for December 31, 1990. Month 13 with any other day yields the day of the first month of the coming year--for example, YRMODA(1990,13,1) produces the day number for January 1, 1991.
- The day can range from 0 through 31. Day 0 is the last day of the previous month regardless of whether it is 28, 29, 30, or 31. For example, YRMODA(1990,3,0) yields 148791.00, the day number for February 28, 1990.
- The function returns the system-missing value if any of the three arguments is missing or if the arguments do not form a valid date *after* October 14, 1582.
- Since YRMODA yields the number of days instead of seconds, you can not display it in date format unless you convert it to the number of seconds.

Extraction functions

The extraction functions extract subfields from dates or time intervals, targeting the day or a time from a date value. This permits you to classify events by day of the week, season, shift, and so forth.

Each extraction function begins with XDATE, followed by a period, the subfunction name (what you want to extract), and an argument.

XDATE.DATE. XDATE.DATE(datevalue). Numeric. Returns the date portion from a numeric value that represents a date. The argument can be a number, a date format variable, or an expression that resolves to a date. To display the result as a date, apply a date format to the variable.

XDATE.HOUR. XDATE.HOUR(datetime). Numeric. Returns the hour (an integer between 0 and 23) from a value that represents a time or a datetime. The argument can be a number, a time or datetime variable or an expression that resolves to a time or datetime value.

XDATE.JDAY. XDATE.JDAY(datevalue). Numeric. Returns the day of the year (an integer between 1 and 366) from a numeric value that represents a date. The argument can be a number, a date format variable, or an expression that resolves to a date.

XDATE.MDAY. XDATE.MDAY(datevalue). Numeric. Returns the day of the month (an integer between 1 and 31) from a numeric value that represents a date. The argument can be a number, a date format variable, or an expression that resolves to a date.

XDATE.MINUTE. XDATE.MINUTE(datetime). Numeric. Returns the minute (an integer between 0 and 59) from a value that represents a time or a datetime. The argument can be a number, a time or datetime variable, or an expression that resolves to a time or datetime value.

XDATE.MONTH. XDATE.MONTH(datevalue). Numeric. Returns the month (an integer between 1 and 12) from a numeric value that represents a date. The argument can be a number, a date format variable, or an expression that resolves to a date.

XDATE.QUARTER. XDATE.QUARTER(datevalue). Numeric. Returns the quarter of the year (an integer between 1 and 4) from a numeric value that represents a date. The argument can be a number, a date format variable, or an expression that resolves to a date.

XDATE.SECOND. XDATE.SECOND(datetime). Numeric. Returns the second (a number between 0 and 60) from a value that represents a time or a datetime. The argument can be a number, a time or datetime variable or an expression that resolves to a time or datetime value.

XDATE.TDAY. XDATE.TDAY(timevalue). Numeric. Returns the number of whole days (as an integer) from a numeric value that represents a time interval. The argument can be a number, a time format variable, or an expression that resolves to a time interval.

XDATE.TIME. XDATE.TIME(datetime). Numeric. Returns the time portion from a value that represents a time or a datetime. The argument can be a number, a time or datetime variable or an expression that resolves to a time or datetime value. To display the result as a time, apply a time format to the variable.

XDATE.WEEK. XDATE.WEEK(datevalue). Numeric. Returns the week number (an integer between 1 and 53) from a numeric value that represents a date. The argument can be a number, a date format variable, or an expression that resolves to a date.

XDATE.WKDAY. XDATE.WKDAY(datevalue). Numeric. Returns the day-of-week number (an integer between 1, Sunday, and 7, Saturday) from a numeric value that represents a date. The argument can be a number, a date format variable, or an expression that resolves to a date.

XDATE.YEAR. XDATE.YEAR(datevalue). Numeric. Returns the year (as a four-digit integer) from a numeric value that represents a date. The argument can be a number, a date format variable, or an expression that resolves to a date.

Example

```
DATA LIST FREE (",")
  /StartDateTime (datetime25).
BEGIN DATA
29-OCT-2003 11:23:02
1 January 1998 1:45:01
21/6/2000 2:55:13
END DATA.
COMPUTE dateonly=XDATE.DATE(StartDateTime).
FORMATS dateonly(ADATE10).
COMPUTE hour=XDATE.HOUR(StartDateTime).
COMPUTE DayofWeek=XDATE.WKDAY(StartDateTime).
COMPUTE WeekofYear=XDATE.WEEK(StartDateTime).
COMPUTE quarter=XDATE.QUARTER(StartDateTime).
```

- The date portion extracted with XDATE.DATE returns a date expressed in seconds; so, FORMATS is used to display the date in a readable date format.
- Day of the week is an integer between 1 (Sunday) and 7 (Saturday).
- Week of the year is an integer between 1 and 53 (January 1–7 = 1).

Date differences

The DATEDIFF function calculates the difference between two date values and returns an integer (with any fraction component truncated) in the specified date/time units. The general form of the expression is

```
DATEDIFF(datetime2, datetime1, “unit”).
```

where *datetime2* and *datetime1* are both date or time format variables (or numeric values that represent valid date/time values), and “unit” is one of the following string literal values, enclosed in quotes:

- Years
- Quarters
- Months
- Weeks

- Days
- Hours
- Minutes
- Seconds

Example

```
DATA LIST FREE /date1 date2 (2ADATE10).
BEGIN DATA
1/1/2004 2/1/2005
1/1/2004 2/15/2005
1/30/2004 1/29/2005
END DATA.
COMPUTE years=DATEDIFF(date2, date1, "years").
```

- The result will be the integer portion of the number of years between the two dates, with any fractional component truncated.
- One "year" is defined as the same month and day, one year before or after the second date argument.
- For the first two cases, the result is 1, since in both cases the number of years is greater than or equal to 1 and less than 2.
- For the third case, the result is 0, since the difference is one day short of a year based on the definition of year.

Example

```
DATA LIST FREE /date1 date2 (2ADATE10).
BEGIN DATA
1/1/2004 2/1/2004
1/1/2004 2/15/2004
1/30/2004 2/1/2004
END DATA.
COMPUTE months=DATEDIFF(date2, date1, "months").
```

- The result will be the integer portion of the number of months between the two dates, with any fractional component truncated.
- One "month" is defined as the same day of the month, one calendar month before or after the second date argument.
- For the first two cases, the result will be 1, since both February 1 and February 15, 2004, are greater than or equal to one month and less than two months after January 1, 2004.
- For the third case, the result will be 0. By definition, *any* date in February 2004 will be less than one month after January 30, 2004, resulting in a value of 0.

Date increments

The DATESUM function calculates a date or time value a specified number of units from a given date or time value. The general form of the function is:

```
DATESUM(datevar, value, "unit", "method").
```

- `datevar` is a date/time format variable (or a numeric value that represents a valid date/time value).
- `value` is a positive or negative number. For variable-length units (years, quarters, months), fractional values are truncated to integers.
- `"unit"` is one of the following string literal values enclosed in quotes: years, quarters, months, weeks, days, hours, minutes, seconds.
- `"method"` is an optional specification for variable-length units (years, quarters, months) enclosed in quotes. The method can be `"rollover"` or `"closest"`. The **rollover** method advances excess days into the next month. The **closest** method uses the closest legitimate date within the month. This is the default.

Example

```
DATA LIST FREE /datevar1 (ADATE10).
BEGIN DATA
2/28/2004
2/29/2004
```

```
END DATA.
COMPUTE rollover_year=DATESUM(datevar1, 1, "years", "rollover").
COMPUTE closest_year=DATESUM(datevar1, 1, "years", "closest").
COMPUTE fraction_year=DATESUM(datevar1, 1.5, "years").
FORMATS rollover_year closest_year fraction_year (ADATE10).
SUMMARIZE
 /TABLES=datevar1 rollover_year closest_year fraction_year
 /FORMAT=VALIDLIST NOCASENUM
 /CELLS=NONE.
```

	datevar1	rollover_year	closest_year	fraction_year
1	02/28/2004	02/28/2005	02/28/2005	08/28/2005
2	02/29/2004	03/01/2005	02/28/2005	08/29/2005

Figure 14. Results of rollover and closest year calculations

- The rollover and closest methods yield the same result when incrementing February 28, 2004, by one year: February 28, 2005.
- Using the rollover method, incrementing February 29, 2004, by one year returns a value of March 1, 2005. Since there is no February 29, 2005, the excess day is rolled over to the next month.
- Using the closest method, incrementing February 29, 2004, by one year returns a value of February 28, 2005, which is the closest day in the same month of the following year.
- The results for *fraction_year* are exactly the same as for *closest_year* because the closest method is used by default, and the value parameter of 1.5 is truncated to 1 for variable-length units.
- All three COMPUTE commands create new variables that display values in the default F format, which for a date value is a large integer. The FORMATS command specifies the ADATE format for the new variables.

Example

```
DATA LIST FREE /datevar1 (ADATE10).
BEGIN DATA
01/31/2003
01/31/2004
03/31/2004
05/31/2004
END DATA.
COMPUTE rollover_month=DATESUM(datevar1, 1, "months", "rollover").
COMPUTE closest_month=DATESUM(datevar1, 1, "months", "closest").
COMPUTE previous_month_rollover =
  DATESUM(datevar1, -1, "months", "rollover").
COMPUTE previous_month_closest =
  DATESUM(datevar1, -1, "months", "closest").
FORMATS rollover_month closest_month
  previous_month_rollover previous_month_closest (ADATE10).
SUMMARIZE
  /TABLES=datevar1 rollover_month closest_month
   previous_month_rollover previous_month_closest
  /FORMAT=VALIDLIST NOCASENUM
  /CELLS=NONE.
```

	datevar1	rollover_month	closest_month	previous_ month_rollover	previous_ month_closest
1	01/31/2003	03/03/2003	02/28/2003	12/31/2002	12/31/2002
2	01/31/2004	03/02/2004	02/29/2004	12/31/2003	12/31/2003
3	03/31/2004	05/01/2004	04/30/2004	03/02/2004	02/29/2004
4	05/31/2004	07/01/2004	06/30/2004	05/01/2004	04/30/2004

Figure 15. Results of month calculations

- Using the rollover method, incrementing by one month from January 31 yields a date in March, since February has a maximum of 29 days; and incrementing one month from March 31 and May 31 yields May 1 and July 1, respectively, since April and June each have only 30 days.
- Using the closest method, incrementing by one month from the last day of any month will always yield the closest valid date within the next month. For example, in a nonleap year, one month after January 31 is February 28, and one month after February 28 is March 28.
- Using the rollover method, decrementing by one month (by specifying a negative value parameter) from the last day of a month may sometimes yield unexpected results, since the excess days are rolled back to the original month. For example, one month prior to March 31 yields March 3 for nonleap years and March 2 for leap years.

- Using the closest method, decrementing by one month from the last day of the month will always yield the closest valid date within the previous month. For example, one month prior to April 30, is March 30 (not March 31), and one month prior to March 31 is February 28 in nonleap years and February 29 in leap years.

String expressions

Expressions involving string variables can be used on COMPUTE and IF commands and in logical expressions on commands such as IF, DO IF, LOOP IF, and SELECT IF.

- A string expression can be a constant enclosed in quotes (for example, 'IL'), a string function, or a string variable. See the topic "String functions" on page 78 for more information.
- An expression must return a string if the target variable is a string.
- The string returned by a string expression does not have to be the same length as the target variable; no warning messages are issued if the lengths are not the same. If the target variable produced by a COMPUTE command is shorter, the result is right-trimmed. If the target variable is longer, the result is right-padded.

String functions

- The target variable for each string function must be a string and *must have already been declared* (see STRING).
- Multiple arguments in a list must be separated by commas.
- When two strings are compared, the case in which they are entered is significant. The LOWER and UPCASE functions are useful for making comparisons of strings regardless of case.
- String functions that include a byte position or count argument or return a byte position or count may return different results in Unicode mode than in code page mode. For example, é is one byte in code page mode but is two bytes in Unicode mode; so *résumé* is six bytes in code page mode and eight bytes in Unicode mode.
- In Unicode mode, trailing blanks are always removed from the values of string variables in string functions unless explicitly preserved with the NTRIM function.
- In code page mode, trailing blanks are always preserved in the values of string variables unless explicitly removed with the RTRIM function.

For more information on Unicode mode, see "UNICODE Subcommand" on page 1805.

CHAR.INDEX. CHAR.INDEX(haystack, needle[, divisor]). Numeric. Returns a number indicating the character position of the first occurrence of needle in haystack. The optional third argument, divisor, is a number of characters used to divide needle into separate strings. Each substring is used for searching and the function returns the first occurrence of any of the substrings. For example, CHAR.INDEX(var1, 'abcd') will return the value of the starting position of the complete string "abcd" in the string variable var1; CHAR.INDEX(var1, 'abcd', 1) will return the value of the position of the first occurrence of any of the values in the string; and CHAR.INDEX(var1, 'abcd', 2) will return the value of the first occurrence of either "ab" or "cd". Divisor must be a positive integer and must divide evenly into the length of needle. Returns 0 if needle does not occur within haystack.

CHAR.LENGTH. CHAR.LENGTH(strexpr). Numeric. Returns the length of strexpr in characters, with any trailing blanks removed.

CHAR.LPAD. CHAR.LPAD(strexpr1,length[,strexpr2]). String. Left-pads strexpr1 to make its length the value specified by length using as many complete copies as will fit of strexpr2 as the padding string. The value of length represents the number of characters and must be a positive integer. If the optional argument strexpr2 is omitted, the value is padded with blank spaces.

CHAR.MBLEN. CHAR.MBLEN(strexpr,pos). Numeric. Returns the number of bytes in the character at character position pos of strexpr.

CHAR.RINDEX. CHAR.RINDEX(haystack,needle[,divisor]). Numeric. Returns an integer that indicates the starting character position of the last occurrence of the string needle in the string haystack. The optional

third argument, divisor, is the number of characters used to divide needle into separate strings. For example, CHAR.RINDEX(var1, 'abcd') will return the starting position of the last occurrence of the entire string "abcd" in the variable var1; CHAR.RINDEX(var1, 'abcd', 1) will return the value of the position of the last occurrence of any of the values in the string; and CHAR.RINDEX(var1, 'abcd', 2) will return the value of the starting position of the last occurrence of either "ab" or "cd". Divisor must be a positive integer and must divide evenly into the length of needle. If needle is not found, the value 0 is returned.

CHAR.RPAD. CHAR.RPAD(strexpr1,length[,strexpr2]). String. Right-pads strexpr1 with strexpr2 to extend it to the length given by length using as many complete copies as will fit of strexpr2 as the padding string. The value of length represents the number of characters and must be a positive integer. The optional third argument strexpr2 is a quoted string or an expression that resolves to a string. If strepxr2 is omitted, the value is padded with blanks.

CHAR.SUBSTR. CHAR.SUBSTR(strexpr,pos[,length]). String. Returns the substring beginning at character position pos of strexpr. The optional third argument represents the number of characters in the substring. If the optional argument length is omitted, returns the substring beginning at character position pos of strexpr and running to the end of strexpr. For example CHAR.SUBSTR('abcd', 2) returns 'bcd' and CHAR.SUBSTR('abcd', 2, 2) returns 'bc'. (Note: Use the SUBSTR function instead of CHAR.SUBSTR if you want to use the function on the left side of an equals sign to replace a substring.)

CONCAT. CONCAT(strexpr,strexpr[,..]). String. Returns a string that is the concatenation of all its arguments, which must evaluate to strings. This function requires two or more arguments. In code page mode, if strexpr is a string variable, use RTRIM if you only want the actual string value without the right-padding to the defined variable width. For example, CONCAT(RTRIM(stringvar1), RTRIM(stringvar2)).

LENGTH. LENGTH(strexpr). Numeric. Returns the length of strexpr in bytes, which must be a string expression. For string variables, in Unicode mode this is the number of bytes in each value, excluding trailing blanks, but in code page mode this is the defined variable length, including trailing blanks. To get the length (in bytes) without trailing blanks in code page mode, use LENGTH(RTRIM(strexpr)).

LOWER. LOWER(strexpr). String. Returns strexpr with uppercase letters changed to lowercase and other characters unchanged. The argument can be a string variable or a value. For example, LOWER(name1) returns charles if the value of name1 is Charles.

LTRIM. LTRIM(strexpr[,char]). String. Returns strexpr with any leading instances of char removed. If char is not specified, leading blanks are removed. Char must resolve to a single character.

MAX. MAX(value,value[,..]). Numeric or string. Returns the maximum value of its arguments that have valid values. This function requires two or more arguments. For numeric values, you can specify a minimum number of valid arguments for this function to be evaluated.

MIN. MIN(value,value[,..]). Numeric or string. Returns the minimum value of its arguments that have valid, nonmissing values. This function requires two or more arguments. For numeric values, you can specify a minimum number of valid arguments for this function to be evaluated.

MBLEN.BYTE. MBLEN.BYTE(strexpr,pos). Numeric. Returns the number of bytes in the character at byte position pos of strexpr.

NORMALIZE. NORMALIZE(strexp). String. Returns the normalized version of strexp. In Unicode mode, it returns Unicode NFC. In code page mode, it has no effect and returns strexp unmodified. The length of the result may be different from the length of the input.

NTRIM. NTRIM(varname). Returns the value of varname, without removing trailing blanks. The value of varname must be a variable name; it cannot be an expression.

REPLACE. REPLACE(a1, a2, a3[, a4]). String. In a1, instances of a2 are replaced with a3. The optional argument a4 specifies the number of occurrences to replace; if a4 is omitted, all occurrences are replaced. Arguments a1, a2, and a3 must resolve to string values (literal strings enclosed in quotes or string variables), and the optional argument a4 must resolve to a non-negative integer. For example, REPLACE("abcabc", "a", "x") returns a value of "xbcxbc" and REPLACE("abcabc", "a", "x", 1) returns a value of "xbcabc".

RTRIM. RTRIM(strexpr[,char]). String. Trims trailing instances of char within strexpr. The optional second argument char is a single quoted character or an expression that yields a single character. If char is omitted, trailing blanks are trimmed.

STRUNC. STRUNC(strexp, length). String. Returns strexp truncated to length (in bytes) and then trimmed of any trailing blanks. Truncation removes any fragment of a character that would be truncated.

UPCASE. UPCASE(strexpr). String. Returns strexpr with lowercase letters changed to uppercase and other characters unchanged.

Deprecated string functions

The following functions provide functionality similar to the newer CHAR functions, but they operate at the byte level rather than the character level. For example, the INDEX function returns the byte position of *needle* within *haystack*, whereas CHAR.INDEX returns the character position. These functions are supported primarily for compatibility with previous releases.

INDEX. INDEX(haystack,needle[,divisor]). Numeric. Returns a number that indicates the byte position of the first occurrence of needle in haystack. The optional third argument, divisor, is a number of bytes used to divide needle into separate strings. Each substring is used for searching and the function returns the first occurrence of any of the substrings. Divisor must be a positive integer and must divide evenly into the length of needle. Returns 0 if needle does not occur within haystack.

LPAD. LPAD(strexpr1,length[,strexpr2]). String. Left-pads strexpr1 to make its length the value specified by length using as many complete copies as will fit of strexpr2 as the padding string. The value of length represents the number of bytes and must be a positive integer. If the optional argument strexpr2 is omitted, the value is padded with blank spaces.

RINDEX. RINDEX(haystack,needle[,divisor]). Numeric. Returns an integer that indicates the starting byte position of the last occurrence of the string needle in the string haystack. The optional third argument, divisor, is the number of bytes used to divide needle into separate strings. Divisor must be a positive integer and must divide evenly into the length of needle. If needle is not found, the value 0 is returned.

RPAD. RPAD(strexpr1,length[,strexpr2]). String. Right-pads strexpr1 with strexpr2 to extend it to the length given by length using as many complete copies as will fit of strexpr2 as the padding string. The value of length represents the number of bytes and must be a positive integer. The optional third argument strexpr2 is a quoted string or an expression that resolves to a string. If strepxr2 is omitted, the value is padded with blanks.

SUBSTR. SUBSTR(strexpr,pos[,length]). String. Returns the substring beginning at byte position pos of strexpr. The optional third argument represents the number of bytes in the substring. If the optional argument length is omitted, returns the substring beginning at byte position pos of strexpr and running to the end of strexpr. When used on the left side of an equals sign, the substring is replaced by the string specified on the right side of the equals sign. The rest of the original string remains intact. For example, SUBSTR(ALPHA6,3,1)='*' changes the third character of all values for *ALPHA6* to *. If the replacement string is longer or shorter than the substring, the replacement is truncated or padded with blanks on the right to an equal length.

Example

```
STRING stringVar1 stringVar2 stringVar3 (A22).
COMPUTE stringVar1='  Does this'.
COMPUTE stringVar2='ting work?'.
COMPUTE stringVar3=
  CONCAT(RTRIM(LTRIM(stringVar1)), " ",
  REPLACE(stringVar2, "ting", "thing")).
```

- The CONCAT function concatenates the values of *stringVar1* and *stringVar2*, inserting a space as a literal string (" ") between them.
- The RTRIM function strips off trailing blanks from *stringVar1*. In code page mode, this is necessary to eliminate excessive space between the two concatenated string values because in code page mode all string variable values are automatically right-padded to the defined width of the string variables. In Unicode mode, this has no effect because trailing blanks are automatically removed from string variable values in Unicode mode.
- The LTRIM function removes the leading spaces from the beginning of the value of *stringVar1*.

- The REPLACE function replaces the misspelled "ting" with "thing" in *stringVar2*.

The final result is a string value of *"Does this thing work?"*

Example

This example extracts the numeric components from a string telephone number into three numeric variables.

```
DATA LIST FREE (",") /telephone (A16).
BEGIN DATA
111-222-3333
222 - 333 - 4444
333-444-5555
444 - 555-6666
555-666-0707
END DATA.
STRING #telstr(A16).
COMPUTE #telstr = telephone.
VECTOR tel(3,f4).
LOOP #i = 1 to 2.
- COMPUTE #dash = CHAR.INDEX(#telstr,"-").
- COMPUTE tel(#i) = NUMBER(CHAR.SUBSTR(#telstr,1,#dash-1),f10).
- COMPUTE #telstr = CHAR.SUBSTR(#telstr,#dash+1).
END LOOP.
COMPUTE tel(3) = NUMBER(#telstr,f10).
EXECUTE.
FORMATS tel1 tel2 (N3) tel3 (N4).
```

- A temporary (scratch) string variable, *#telstr*, is declared and set to the value of the original string telephone number.
- The VECTOR command creates three numeric variables--*tel1, tel2,* and *tel3*--and creates a vector containing those variables.
- The LOOP structure iterates twice to produce the values for *tel1* and *tel2*.
- `COMPUTE #dash = CHAR.INDEX(#telstr,"-")` creates another temporary variable, *#dash*, that contains the position of the first dash in the string value.
- On the first iteration, `COMPUTE tel(#i) = NUMBER(CHAR.SUBSTR(#telstr,1,#dash-1),f10)` extracts everything prior to the first dash, converts it to a number, and sets *tel1* to that value.
- `COMPUTE #telstr = CHAR.SUBSTR(#telstr,#dash+1)` then sets *#telstr* to the remaining portion of the string value after the first dash.
- On the second iteration, `COMPUTE #dash...` sets *#dash* to the position of the “first” dash in the modified value of *#telstr*. Since the area code and the original first dash have been removed from *#telstr*, this is the position of the dash between the exchange and the number.
- `COMPUTE tel(#)...` sets *tel2* to the numeric value of everything up to the “first” dash in the modified version of *#telstr*, which is everything after the first dash and before the second dash in the original string value.
- `COMPUTE #telstr...` then sets *#telstr* to the remaining segment of the string value--everything after the “first” dash in the modified value, which is everything after the second dash in the original value.
- After the two loop iterations are complete, `COMPUTE tel(3) = NUMBER(#telstr,f10)` sets *tel3* to the numeric value of the final segment of the original string value.

String/numeric conversion functions

NUMBER. NUMBER(strexpr,format). Numeric. Returns the value of the string expression strexpr as a number. The second argument, format, is the numeric format used to read strexpr. For example, NUMBER(stringDate,DATE11) converts strings containing dates of the general format dd-mmm-yyyy to a numeric number of seconds that represent that date. (To display the value as a date, use the FORMATS or PRINT FORMATS command.) If the string cannot be read using the format, this function returns system-missing.

STRING. STRING(numexpr,format). String. Returns the string that results when numexpr is converted to a string according to format. STRING(-1.5,F5.2) returns the string value '-1.50'. The second argument format must be a format for writing a numeric value.

Example

```
DATA LIST FREE /tel1 tel2 tel3.
BEGIN DATA
123 456 0708
END DATA.
STRING telephone (A12).
COMPUTE telephone=
  CONCAT(STRING(tel1,N3), "-", STRING(tel2, N3), "-", STRING(tel3, N4)).
```

- A new string variable, *telephone*, is declared to contain the computed string value.
- The three numeric variables are converted to strings and concatenated with dashes between the values.
- The numeric values are converted using N format to preserve any leading zeros.

LAG function

LAG. LAG(variable[, n]). Numeric or string. The value of variable in the previous case or n cases before. The optional second argument, n, must be a positive integer; the default is 1. For example, prev4=LAG(gnp,4) returns the value of gnp for the fourth case before the current one. The first four cases have system-missing values for prev4.

- The result is of the same type (numeric or string) as the variable specified as the first argument.
- The first *n* cases for string variables are set to blanks. For example, if PREV2=LAG (LNAME,2) is specified, blanks will be assigned to the first two cases for *PREV2*.
- When LAG is used with commands that select cases (for example, SELECT IF and SAMPLE), LAG counts cases *after* case selection, even if specified before these commands. See the topic "Command Order" on page 36 for more information.

Note: In a series of transformation commands without any intervening EXECUTE commands or other commands that read the data, lag functions are calculated after all other transformations, regardless of command order. For example,

```
COMPUTE lagvar=LAG(var1).
COMPUTE var1=var1*2.
```

and

```
COMPUTE lagvar=LAG(var1).
EXECUTE.
COMPUTE var1=var1*2.
```

yield very different results for the value of *lagvar*, since the former uses the transformed value of *var1* while the latter uses the original value.

VALUELABEL function

VALUELABEL. VALUELABEL(varname). String. Returns the value label for the value of variable or an empty string if there is no label for the value. The value of varname must be a variable name; it cannot be an expression.

Example

```
STRING labelvar (A120).
COMPUTE labelvar=VALUELABEL(var1).
DO REPEAT varlist=var2, var3, var4
         /newvars=labelvar2, labelvar3, labelvar4.
- STRING newvars(A120).
- COMPUTE newvars=VALUELABEL(varlist).
END REPEAT.
```

Logical expressions

Logical expressions can appear on the COMPUTE, IF, SELECT IF, DO IF, ELSE IF, LOOP IF, and END LOOP IF commands. A logical expression is evaluated as true or false, or as missing if it is indeterminate. A logical expression returns 1 if the expression is true, 0 if it is false, or system-missing if it is missing. Thus, logical expressions can be any expressions that yield this three-value logic.

- The simplest logical expression is a logical variable. A logical variable is any numeric variable that has the values 1, 0, or system-missing. Logical variables cannot be strings.
- Logical expressions can be simple logical variables or relations, or they can be complex logical tests involving variables, constants, functions, relational operators, logical operators, and parentheses to control the order of evaluation.
- On an IF command, a logical expression that is true causes the assignment expression to be executed. A logical expression that returns missing has the same effect as one that is false--that is, the assignment expression is not executed and the value of the target variable is not altered.
- On a DO IF command, a logical expression that is true causes the execution of the commands immediately following the DO IF, up to the next ELSE IF, ELSE, or END IF. If it is false, the next ELSE IF or ELSE command is evaluated. If the logical expression returns missing for each of these, the entire structure is skipped.
- On a SELECT IF command, a logical expression that is true causes the case to be selected. A logical expression that returns missing has the same effect as one that is false--that is, the case is not selected.
- On a LOOP IF command, a logical expression that is true causes looping to begin (or continue). A logical expression that returns missing has the same effect as one that is false--that is, the structure is skipped.
- On an END LOOP IF command, a logical expression that is false returns control to the LOOP command for that structure, and looping continues. If it is true, looping stops and the structure is terminated. A logical expression that returns a missing value has the same effect as one that is true--that is, the structure is terminated.

Example

```
DATA LIST FREE (",") /a.
BEGIN DATA
1, , 1 , ,
END DATA.
COMPUTE b=a.
* The following does NOT work since the second condition
  is never evaluated.
DO IF a=1.
COMPUTE a1=1.
ELSE IF MISSING(a).
COMPUTE a1=2.
END IF.
* On the other hand the following works.
DO IF MISSING(b).
COMPUTE b1=2.
ELSE IF b=1.
COMPUTE b1=1.
END IF.
```

- The first DO IF will never yield a value of 2 for *a1* because if *a* is missing, then DO IF a=1 evaluates as missing and control passes immediately to END IF. So *a1* will either be 1 or missing.
- In the second DO IF, however, we take care of the missing condition first; so if the value of *b* is missing, DO IF MISSING(b) evaluates as true and *b1* is set to 2; otherwise, *b1* is set to 1.

String variables in logical expressions

String variables, like numeric variables, can be tested in logical expressions.

- String variables must be declared before they can be used in a string expression.
- String variables cannot be compared to numeric variables.
- If strings of different lengths are compared, the shorter string is right-padded with blanks to equal the length of the longer string.
- The magnitude of strings can be compared using LT, GT, and so on, but the outcome depends on the sorting sequence of the computer. Use with caution.
- User-missing string values are treated the same as nonmissing string values when evaluating string variables in logical expressions. In other words, all string variable values are treated as valid, nonmissing values in logical expressions.

Relational operators

A relation is a logical expression that compares two values using a *relational operator*. In the command

```
IF (X EQ 0) Y=1
```

the variable *X* and 0 are expressions that yield the values to be compared by the EQ relational operator. The following are the relational operators:

EQ or =. Equal to

NE or ~= or ¬ = or <>. Not equal to

LT or <. Less than

LE or <=. Less than or equal to

GT or >. Greater than

GE or >=. Greater than or equal to

- The expressions in a relation can be variables, constants, or more complicated arithmetic expressions.
- Blanks (not commas) must be used to separate the relational operator from the expressions. To make the command more readable, use extra blanks or parentheses.
- For string values, “less than” and “greater than” results can vary by locale even for the same set of characters, since the national collating sequence is used. Language order, not ASCII order, determines where certain characters fall in the sequence.

NOT logical operator

The NOT logical operator reverses the true/false outcome of the expression that immediately follows.

- The NOT operator affects only the expression that immediately follows, unless a more complex logical expression is enclosed in parentheses.
- You can substitute ~ or ¬ for NOT as a logical operator.
- NOT can be used to check whether a numeric variable has the value 0, 1, or any other value. For example, all scratch variables are initialized to 0. Therefore, `NOT (#ID)` returns false or missing when *#ID* has been assigned a value other than 0.

AND and OR logical operators

Two or more relations can be logically joined using the logical operators AND and OR. Logical operators combine relations according to the following rules:

- The ampersand (&) symbol is a valid substitute for the logical operator AND. The vertical bar (|) is a valid substitute for the logical operator OR.
- Only one logical operator can be used to combine two relations. However, multiple relations can be combined into a complex logical expression.
- Regardless of the number of relations and logical operators used to build a logical expression, the result is either true, false, or indeterminate because of missing values.
- Operators or expressions cannot be implied. For example, `X EQ 1 OR 2` is illegal; you must specify `X EQ 1 OR X EQ 2`.
- The ANY and RANGE functions can be used to simplify complex expressions.

AND . *Both relations must be true for the complex expression to be true.*

OR . *If either relation is true, the complex expression is true.*

The following table lists the outcomes for AND and OR combinations.

Table 3. Logical outcomes			
Expression	**Outcome**	**Expression**	**Outcome**
true AND true	= true	true OR true	= true
true AND false	= false	true OR false	= true
false AND false	= false	false OR false	= false
true AND missing	= missing	true OR missing	= true*
missing AND missing	= missing	missing OR missing	= missing
false AND missing	= false*	false OR missing	= missing

* Expressions where the outcome can be evaluated with incomplete information. See the topic "Missing values in logical expressions" on page 91 for more information.

Example

```
DATA LIST FREE /var1 var2 var3.
BEGIN DATA
1 1 1
1 2 1
1 2 3
4 2 4
END DATA.
SELECT IF var1 = 4 OR ((var2 > var1) AND (var1 <> var3)).
```

- Any case that meets the first condition--`var1 = 4`--will be selected, which in this example is only the last case.
- Any case that meets the second condition will also be selected. In this example, only the third case meets this condition, which contains two criteria: *var2* is greater than *var1* and *var1* is not equal to *var3*.

Order of evaluation

- When arithmetic operators and functions are used in a logical expression, the order of operations is functions and arithmetic operations first, then relational operators, and then logical operators.
- When more than one logical operator is used, `NOT` is evaluated first, then `AND`, and then `OR`.
- To change the order of evaluation, use parentheses.

Logical functions

- Each argument to a logical function (expression, variable name, or constant) must be separated by a comma.
- The target variable for a logical function must be numeric.
- The functions `RANGE` and `ANY` can be useful shortcuts to more complicated specifications on the `IF`, `DO IF`, and other conditional commands. For example, for non-missing values, the command

  ```
  SELECT IF ANY(REGION,"NW","NE","SE").
  ```

  is equivalent to

  ```
  SELECT IF (REGION EQ "NW" OR REGION EQ "NE" OR REGION EQ "SE").
  ```

RANGE. RANGE(test,lo,hi[,lo,hi,..]). Logical. Returns 1 or true if test is within any of the inclusive range(s) defined by the pairs lo, hi. Arguments must be all numeric or all strings of the same length, and each of the lo, hi pairs must be ordered with lo <= hi. Note: For string values, results can vary by locale even for the same set of characters, since the national collating sequence is used. Language order, not ASCII order, determines where certain characters fall in the sequence.

ANY. ANY(test,value[,value,...]). Logical. Returns 1 or true if the value of test matches any of the subsequent values; returns 0 or false otherwise. This function requires two or more arguments. For example, ANY(var1, 1, 3, 5) returns 1 if the value of var1 is 1, 3, or 5 and 0 for other values. ANY can also be used to scan a list of variables or expressions for a value. For example, ANY(1, var1, var2, var3) returns 1 if any of the three specified variables has a value of 1 and 0 if all three variables have values other than 1.

See "Treatment of missing values in arguments " on page 89 for information on how missing values are handled by the ANY and RANGE functions.

Scoring expressions

Scoring expressions apply model XML from an external file to the active dataset and generate predicted values, predicted probabilities, and other values based on that model.

- Scoring expressions must be preceded by a `MODEL HANDLE` command that identifies the external XML model file and optionally does variable mapping.
- Scoring expressions require two arguments: the first identifies the model, and the second identifies the scoring function. An optional third argument allows users to obtain the probability (for each case) associated with a selected category, in the case of a categorical target variable. It is also used in nearest neighbor models to specify a particular neighbor.
- Prior to applying scoring functions to a set of data, a data validation analysis is performed. The analysis includes checking that data are of the correct type as well as checking that the data values are in the set of allowed values defined in the model. For example, for categorical variables, a value that is neither a valid category nor defined as user-missing would be treated as an invalid value. Values that are found to be invalid are treated as system-missing.

The following scoring expressions are available:

ApplyModel. ApplyModel(handle, "function", value). Numeric. Applies a particular scoring function to the input case data using the model specified by handle and where "function" is one of the following string literal values enclosed in quotes: predict, stddev, probability, confidence, nodeid, cumhazard, neighbor, distance. The model handle is the name associated with the external XML file, as defined on the MODEL HANDLE command. The optional third argument applies when the function is "probability", "neighbor", or "distance". For "probability", it specifies a category for which the probability is calculated. For "neighbor" and "distance", it specifies a particular neighbor (as an integer) for nearest neighbor models. ApplyModel returns system-missing if a value can not be computed.

StrApplyModel. StrApplyModel(handle, "function", value). String. Applies a particular scoring function to the input case data using the model specified by handle and where "function" is one of the following string literal values enclosed in quotes: predict, stddev, probability, confidence, nodeid, cumhazard, neighbor, distance. The model handle is the name associated with the external XML file, as defined on the MODEL HANDLE command. The optional third argument applies when the function is "probability", "neighbor", or "distance". For "probability", it specifies a category for which the probability is calculated. For "neighbor" and "distance", it specifies a particular neighbor (as an integer) for nearest neighbor models. StrApplyModel returns a blank string if a value cannot be computed.

- String values must be enclosed in quotation marks. For example, `ApplyModel(name1, 'probability', 'reject')`, where `name1` is the model's handle name and `'reject'` is a valid category for a target variable that is a string.
- Negative values must be enclosed in quotation marks. For example, `ApplyModel(name1, 'probability', '-1')`.

The following scoring functions are available:

Scoring function	Description
PREDICT	*Returns the predicted value of the target variable.*
STDDEV	*Standard deviation.*

Scoring function	Description
PROBABILITY	*Probability associated with a particular category of a target variable.* Applies only to categorical variables. In the absence of the optional third parameter, *category*, this is the probability that the predicted category is the correct one for the target variable. If a particular category is specified, then this is the probability that the specified category is the correct one for the target variable.
CONFIDENCE	*A probability measure associated with the predicted value of a categorical target variable.* Applies only to categorical variables.
NODEID	*The terminal node number.* Applies only to tree models.
CUMHAZARD	*Cumulative hazard value.* Applies only to Cox regression models.
NEIGHBOR	*The ID of the kth nearest neighbor.* Applies only to nearest neighbor models. In the absence of the optional third parameter, k, this is the ID of the nearest neighbor. The ID is the value of the case labels variable, if supplied, and otherwise the case number.
DISTANCE	*The distance to the kth nearest neighbor.* Applies only to nearest neighbor models. In the absence of the optional third parameter, k, this is the distance to the nearest neighbor. Depending on the model, either Euclidean or City Block distance will be used.

The following table lists the set of scoring functions available for each type of model that supports scoring. The function type denoted as PROBABILITY (category) refers to specification of a particular category (the optional third parameter) for the PROBABILITY function.

Table 4. Supported functions by model type

Model type	Supported functions
Tree (categorical target)	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE, NODEID
Tree (scale target)	PREDICT, NODEID, STDDEV
Boosted Tree (C5.0)	PREDICT, CONFIDENCE
Linear Regression	PREDICT, STDDEV
Automatic Linear Models	PREDICT
Binary Logistic Regression	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE
Conditional Logistic Regression	PREDICT
Multinomial Logistic Regression	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE
General Linear Model	PREDICT, STDDEV
Discriminant	PREDICT, PROBABILITY, PROBABILITY (category)
TwoStep Cluster	PREDICT
K-Means Cluster	PREDICT

Table 4. Supported functions by model type (continued)

Model type	Supported functions
Kohonen	PREDICT
Neural Net (categorical target)	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE
Neural Net (scale target)	PREDICT
Naive Bayes	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE
Anomaly Detection	PREDICT
Ruleset	PREDICT, CONFIDENCE
Generalized Linear Model (categorical target)	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE
Generalized Linear Model (scale target)	PREDICT, STDDEV
Generalized Linear Mixed Model (categorical target)	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE
Generalized Linear Mixed Model (scale target)	PREDICT
Ordinal Multinomial Regression	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE
Cox Regression	PREDICT, CUMHAZARD
Nearest Neighbor (scale target)	PREDICT, NEIGHBOR, NEIGHBOR(K), DISTANCE, DISTANCE(K)
Nearest Neighbor (categorical target)	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE,NEIGHBOR, NEIGHBOR(K),DISTANCE, DISTANCE(K)

- For the Binary Logistic Regression, Multinomial Logistic Regression, and Naive Bayes models, the value returned by the CONFIDENCE function is identical to that returned by the PROBABILITY function.
- For the K-Means model, the value returned by the CONFIDENCE function is the least distance.
- For tree and ruleset models, the confidence can be interpreted as an adjusted probability of the predicted category and is always less than the value given by PROBABILITY. For these models, the confidence value is more reliable than the value given by PROBABILITY.
- For neural network models, the confidence provides a measure of whether the predicted category is much more likely than the second-best predicted category.
- For Ordinal Multinomial Regression and Generalized Linear Model, the PROBABILITY function is supported when the target variable is binary.
- For nearest neighbor models without a target variable, the available functions are NEIGHBOR and DISTANCE.

Missing values

Functions and simple arithmetic expressions treat missing values in different ways. In the expression

```
(var1+var2+var3)/3
```

the result is missing if a case has a missing value for any of the three variables.

In the expression

```
MEAN(var1, var2, var3)
```

the result is missing only if the case has missing values for all three variables.

For statistical functions, you can specify the minimum number of arguments that must have nonmissing values. To do so, type a period and the minimum number after the function name, as in:

```
MEAN.2(var1, var2, var3)
```

The following sections contain more information on the treatment of missing values in functions and transformation expressions, including special missing value functions.

Treatment of missing values in arguments

If the logic of an expression is indeterminate because of missing values, the expression returns a missing value and the command is not executed. The following table summarizes how missing values are handled in arguments to various functions.

Table 5. Missing values in arguments

Function	**Returns system-missing if**
`MOD (x1,x2)`	x1 is missing, or x2 is missing and x1 is not 0.

`MAX.n (x1,x2,...xk)`
`MEAN.n (x1,x2,...xk)`
`MIN.n (x1,x2,...x1)`
`SUM.n (x1,x2,...xk)`
	Fewer than *n* arguments are valid; the default *n* is 1.

`CFVAR.n (x1,x2,...xk)`
`SD.n (x1,x2,...xk)`
`VARIANCE.n (x1,x2,...xk)`
	Fewer than *n* arguments are valid; the default *n* is 2.

`LPAD(x1,x2,x3)`
`LTRIM(x1,x2)`
`RTRIM(x1,x2)`
`RPAD(x1,x2,x3)`
	x1 or x2 is illegal or missing.

`SUBSTR(x1,x2,x3)`	x2 or x3 is illegal or missing.
`NUMBER(x,format)` `STRING(x,format)`	The conversion is invalid.
`INDEX(x1,x2,x3)` `RINDEX(x1,x2,x3)`	x3 is invalid or missing.

Table 5. Missing values in arguments (continued)	
Function	**Returns system-missing if**
`LAG (x,n)`	x is missing *n* cases previously (and always for the first *n* cases); the default *n* is 1.
`ANY (x,x1,x2,...xk)`	For numeric values, if x is missing or all the remaining arguments are missing, the result is system-missing. For string values, user-missing value are treated as valid values, and the result is never missing.
`RANGE (x,x1,x2,...xk1,xk2)`	For numeric values, the result is system-missing if: • x is missing, or • all the ranges defined by the remaining arguments are missing, or • any range has a starting value that is higher than the ending value. A numeric range is missing if either of the arguments that define the range is missing. This includes ranges for which one of the arguments is equal to the value of the first argument in the expression. For example: `RANGE(x, x1, x2)` is missing if any of the arguments is missing, even if x1 or x2 is equal to x. For string values, user-missing values are treated as valid values, and the result is only missing if any range has a starting value that is higher than the ending value.
`VALUE (x)`	x is system-missing.

`MISSING (x)`
`NMISS (x1,x2,...xk)`
`NVALID (x1,x2,...xk)`
`SYSMIS (x)`
	Never.

- Any function that is not listed in this table returns the system-missing value when the argument is missing.
- The system-missing value is a displayed as a period (.) for numeric variables.
- String variables do not have system-missing values. An invalid string expression nested within a complex transformation yields a null string, which is passed to the next level of operation and treated as missing. However, an invalid string expression that is not nested is displayed as a blank string and is *not* treated as missing.

Missing values in numeric expressions

Most numeric expressions receive the system-missing value when any one of the values in the expression is missing. Some arithmetic operations involving 0 can be evaluated even when the variables have missing values. These operations are:

- 0 * missing = 0
- 0 / missing = 0
- MOD(0, missing) = 0

The *.n* suffix can be used with the statistical functions SUM, MEAN, MIN, MAX, SD, VARIANCE, and CFVAR to specify the number of valid arguments that you consider acceptable. The default of *n* is 2 for SD, VARIANCE, and CFVAR, and 1 for other statistical functions. For example,

```
COMPUTE FACTOR = SUM.2(SCORE1 TO SCORE3).
```

computes the variable *FACTOR* only if a case has valid information for at least two scores. *FACTOR* is assigned the system-missing value if a case has valid values for fewer than two scores. If the number specified exceeds the number of arguments in the function, the result is system-missing.

Missing values in string expressions

- If the numeric argument (which can be an expression) for the functions LPAD and RPAD is illegal or missing, the result is a null string. If the padding or trimming is the only operation, the string is then padded to its entire length with blanks. If the operation is nested, the null string is passed to the next nested level.
- If a numeric argument to SUBSTR is illegal or missing, the result is a null string. If SUBSTR is the only operation, the string is blank. If the operation is nested, the null string is passed to the next nested level.
- If a numeric argument to INDEX or RINDEX is illegal or missing, the result is system-missing.

String user-missing values are treated as missing by statistical and charting procedures and missing values functions. They are treated as valid in other transformation expressions.

```
DATA LIST LIST /stringvar (a1) numvar(f5.2).
BEGIN DATA
"a" 1
"b" 2
"c" 99
END DATA.
MISSING VALUES stringvar ('c') numvar (99).
COMPUTE newnum1=numvar.
STRING newstring (a1).
COMPUTE newstring=stringvar.
DO IF numvar <> 1.
  COMPUTE num_eval=1.
END IF.
DO IF stringvar <> "a".
  COMPUTE string_eval=1.
END IF.
COMPUTE num_missing=missing(numvar).
COMPUTE string_missing=missing(stringvar).
LIST.

stringvar numvar  newnum1 newstring num_eval string_eval num_missing string_missing

a           1.00     1.00 a              .           .           .00          .00
b           2.00     2.00 b             1.00       1.00         .00          .00
c          99.00      .   c              .          1.00        1.00         1.00
```

- The value of "c" is declared user-missing for stringvar.
- All three values of stringvar are treated as valid `in COMPUTE newstring=stringvar`.
- `DO IF stringvar <> "a"` is evaluated as true for the value of "c" rather than missing. This returns a value of 1 for the variable string_eval rather than system-missing.
- The MISSING function recognizes "c" as missing. This returns a value of 1 for the variable string_missing.

Missing values in logical expressions

In a simple relation, the logic is indeterminate if the expression on either side of the relational operator is missing. When two or more relations are joined by logical operators AND and OR, a missing value is always returned if all of the relations in the expression are missing. However, if any one of the relations can be determined, IBM SPSS Statistics tries to return true or false according to the logical outcomes. See the topic “AND and OR logical operators” on page 84 for more information.

- When two relations are joined with the AND operator, the logical expression can never be true if one of the relations is indeterminate. The expression can, however, be false.
- When two relations are joined with the OR operator, the logical expression can never be false if one relation returns missing. The expression, however, can be true.

Missing value functions

- Each argument to a missing-value function (expression, variable name, or constant) must be separated by a comma.
- With the exception of the MISSING function, only numeric values can be used as arguments in missing-value functions.
- The keyword TO can be used to refer to a set of variables in the argument list for functions NMISS and NVALID.
- The functions MISSING and SYSMIS are logical functions and can be useful shortcuts to more complicated specifications on the IF, DO IF, and other conditional commands.

VALUE. VALUE(variable). Numeric. Returns the value of variable, ignoring user missing-value definitions for variable, which must be a numeric variable name or a vector reference to a variable name.

MISSING. MISSING(variable). Logical. Returns 1 or true if variable has a system- or user-missing value. The argument should be a variable name in the active dataset.

SYSMIS. SYSMIS(numvar). Logical. Returns 1 or true if the value of numvar is system-missing. The argument numvar must be the name of a numeric variable in the active dataset.

NMISS. NMISS(variable[,..]). Numeric. Returns a count of the arguments that have system- and user-missing values. This function requires one or more arguments, which should be variable names in the active dataset.

NVALID. NVALID(variable[,..]). Numeric. Returns a count of the arguments that have valid, nonmissing values. This function requires one or more arguments, which should be variable names in the active dataset.

2SLS

2SLS is available in SPSS Statistics Standard Edition or the Regression Option.

```
2SLS [EQUATION=]dependent variable WITH predictor variable
 [/[EQUATION=]dependent variable...]
  /INSTRUMENTS=varlist
 [/ENDOGENOUS=varlist]
 [/{CONSTANT**}
   {NOCONSTANT}
 [/PRINT=COV]
 [/SAVE = [PRED] [RESID]]
 [/APPLY[='model name']]
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
2SLS VAR01 WITH VAR02 VAR03
  /INSTRUMENTS VAR03 LAGVAR01.
```

Overview

2SLS performs two-stage least-squares regression to produce consistent estimates of parameters when one or more predictor variables might be correlated with the disturbance. This situation typically occurs when your model consists of a system of simultaneous equations wherein endogenous variables are specified as predictors in one or more of the equations. The two-stage least-squares technique uses instrumental variables to produce regressors that are not contemporaneously correlated with the disturbance. Parameters of a single equation or a set of simultaneous equations can be estimated.

Options

New Variables. You can change NEWVAR settings on the TSET command prior to 2SLS to evaluate the regression statistics without saving the values of predicted and residual variables, or you can save the new values to replace the values that were saved earlier, or you can save the new values without erasing values that were saved earlier (see the TSET command). You can also use the SAVE subcommand on 2SLS to override the NONE or the default CURRENT settings on NEWVAR.

Covariance Matrix. You can obtain the covariance matrix of the parameter estimates in addition to all of the other output by specifying PRINT=DETAILED on the TSET command prior to 2SLS. You can also use the PRINT subcommand to obtain the covariance matrix, regardless of the setting on PRINT.

Basic Specification

The basic specification is at least one EQUATION subcommand and one INSTRUMENTS subcommand.

- For each specified equation, 2SLS estimates and displays the regression analysis-of-variance table, regression standard error, mean of the residuals, parameter estimates, standard errors of the parameter estimates, standardized parameter estimates, *t* statistic significance tests and probability levels for the parameter estimates, tolerance of the variables, and correlation matrix of the parameter estimates.
- If the setting on NEWVAR is either ALL or the default CURRENT, two new variables containing the predicted and residual values are automatically created for each equation. The variables are labeled and added to the active dataset.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- The INSTRUMENTS subcommand must specify at least as many variables as are specified after WITH on the longest EQUATION subcommand.
- If a subcommand is specified more than once, the effect is cumulative (except for the APPLY subcommand, which executes only the last occurrence).

Operations

- 2SLS cannot produce forecasts beyond the length of any regressor series.
- 2SLS honors the WEIGHT command.
- 2SLS uses listwise deletion of missing data. Whenever a variable is missing a value for a particular observation, that observation will not be used in any of the computations.

Examples

```
TSET NEWVAR=NONE .
2SLS buyoff  WITH buycd buybk offer_type1 offer_type2
  /INSTRUMENTS offer_type1 offer_type2 lndisccd lndiscbk buycd_1 buybk_1
  /CONSTANT .
```

EQUATION Subcommand

EQUATION specifies the structural equations for the model and is required. The actual keyword EQUATION is optional.

- An equation specifies a single dependent variable, followed by keyword WITH and one or more predictor variables.
- You can specify more than one equation. Multiple equations are separated by slashes.

Example

```
2SLS EQUATION=Y1 WITH X1 X2
  /INSTRUMENTS=X1 LAGX2 X3.
```

- In this example, *Y1* is the dependent variable, and *X1* and *X2* are the predictors. The instruments that are used to predict the *X2* values are *X1*, *LAGX2*, and *X3*.

INSTRUMENTS Subcommand

INSTRUMENTS specifies the instrumental variables. These variables are used to compute predicted values for the endogenous variables in the first stage of 2SLS.

- At least one INSTRUMENTS subcommand must be specified.
- If more than one INSTRUMENTS subcommand is specified, the effect is cumulative. All variables that are named on INSTRUMENTS subcommands are used as instruments to predict all the endogenous variables.
- Any variable in the active dataset can be named as an instrument.
- Instrumental variables can be specified on the EQUATION subcommand, but this specification is not required.
- The INSTRUMENTS subcommand must name at least as many variables as are specified after WITH on the longest EQUATION subcommand.
- If all the predictor variables are listed as the only INSTRUMENTS, the results are the same as results from ordinary least-squares regression.

Example

```
2SLS DEMAND WITH PRICE, INCOME
  /PRICE WITH DEMAND, RAINFALL, LAGPRICE
  /INSTRUMENTS=INCOME, RAINFALL, LAGPRICE.
```

- The endogenous variables are PRICE and DEMAND.
- The instruments to be used to compute predicted values for the endogenous variables are INCOME, RAINFALL, and LAGPRICE.

ENDOGENOUS Subcommand

All variables that are not specified on the INSTRUMENTS subcommand are used as endogenous variables by 2SLS. The ENDOGENOUS subcommand simply allows you to document what these variables are.

- Computations are not affected by specifications on the ENDOGENOUS subcommand.

Example

```
2SLS Y1 WITH X1 X2 X3
  /INSTRUMENTS=X2 X4 LAGY1
  /ENDOGENOUS=Y1 X1 X3.
```

- In this example, the ENDOGENOUS subcommand is specified to document the endogenous variables.

CONSTANT and NOCONSTANT Subcommands

Specify CONSTANT or NOCONSTANT to indicate whether a constant term should be estimated in the regression equation. The specification of either subcommand overrides the CONSTANT setting on the TSET command for the current procedure.

- CONSTANT is the default and specifies that the constant term is used as an instrument.
- NOCONSTANT eliminates the constant term.

SAVE Subcommand

SAVE saves the values of predicted and residual variables that are generated during the current session to the end of the active dataset. The default names *FIT_n* and *ERR_n* will be generated, where *n* increments each time variables are saved for an equation. SAVE overrides the NONE or the default CURRENT setting on NEWVAR for the current procedure.

PRED . *Save the predicted value.* The new variable is named *FIT_n*, where *n* increments each time a predicted or residual variable is saved for an equation.

RESSID. *Save the residual value.* The new variable is named *ERR_n*, where *n* increments each time a predicted or residual variable is saved for an equation.

PRINT Subcommand

PRINT can be used to produce an additional covariance matrix for each equation. The only specification on this subcommand is keyword COV. The PRINT subcommand overrides the PRINT setting on the TSET command for the current procedure.

APPLY Subcommand

APPLY allows you to use a previously defined 2SLS model without having to repeat the specifications.

- The only specification on APPLY is the name of a previous model. If a model name is not specified, the model that was specified on the previous 2SLS command is used.
- To change the series that are used with the model, enter new series names before or after the APPLY subcommand.
- To change one or more model specifications, specify the subcommands of only those portions that you want to change after the APPLY subcommand.
- If no series are specified on the command, the series that were originally specified with the model that is being reapplied are used.

Example

```
2SLS Y1 WITH X1 X2 / X1 WITH Y1 X2
  /INSTRUMENTS=X2 X3.
2SLS APPLY
  /INSTRUMENTS=X2 X3 LAGX1.
```

- In this example, the first command requests 2SLS using *X2* and *X3* as instruments.
- The second command specifies the same equations but changes the instruments to *X2*, *X3*, and *LAGX1*.

ACF

```
ACF VARIABLES= series names
 [/DIFF={1}]
        {n  }
 [/SDIFF={1}]
         {n  }
 [/PERIOD=n]
 [/{NOLOG**}]
   {LN     }
 [/SEASONAL]
 [/MXAUTO={16**}]
          {n   }
 [/SERROR={IND**}]
          {MA   }
 [/PACF]
 [/APPLY [='model name']]
```

**Default if the subcommand is omitted and there is no corresponding specification on the TSET command.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
ACF TICKETS.
```

Overview

ACF displays and plots the sample autocorrelation function of one or more time series. You can also display and plot the autocorrelations of transformed series by requesting natural log and differencing transformations within the procedure.

Options

Modifying the Series. You can request a natural log transformation of the series using the LN subcommand and seasonal and nonseasonal differencing to any degree using the SDIFF and DIFF subcommands. With seasonal differencing, you can specify the periodicity on the PERIOD subcommand.

Statistical Output. With the MXAUTO subcommand, you can specify the number of lags for which you want autocorrelations to be displayed and plotted, overriding the maximum specified on TSET. You can also display and plot values at periodic lags only using the SEASONAL subcommand. In addition to autocorrelations, you can display and plot partial autocorrelations using the PACF subcommand.

Method of Calculating Standard Errors. You can specify one of two methods of calculating the standard errors for the autocorrelations on the SERROR subcommand.

Basic Specification

The basic specification is one or more series names.

- For each series specified, ACF automatically displays the autocorrelation value, standard error, Box-Ljung statistic, and probability for each lag.
- ACF plots the autocorrelations and marks the bounds of two standard errors on the plot. By default, ACF displays and plots autocorrelations for up to 16 lags or the number of lags specified on TSET.
- If a method has not been specified on TSET, the default method of calculating the standard error (IND) assumes that the process is white noise.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- VARIABLES can be specified only once.
- Other subcommands can be specified more than once, but only the last specification of each one is executed.

Operations

- Subcommand specifications apply to all series named on the ACF command.
- If the LN subcommand is specified, any differencing requested on that ACF command is done on the log-transformed series.
- Confidence limits are displayed in the plot, marking the bounds of two standard errors at each lag.

Limitations

- A maximum of one VARIABLES subcommand. There is no limit on the number of series named on the list.

Example

```
ACF VARIABLES = TICKETS
  /LN
  /DIFF=1
  /SDIFF=1
  /PER=12
  /MXAUTO=50.
```

- This example produces a plot of the autocorrelation function for the series *TICKETS* after a natural log transformation, differencing, and seasonal differencing have been applied. Along with the plot, the autocorrelation value, standard error, Box-Ljung statistic, and probability are displayed for each lag.
- LN transforms the data using the natural logarithm (base *e*) of the series.
- DIFF differences the series once.
- SDIFF and PERIOD apply one degree of seasonal differencing with a period of 12.
- MXAUTO specifies that the maximum number of lags for which output is to be produced is 50.

VARIABLES Subcommand

VARIABLES specifies the series names and is the only required subcommand.

DIFF Subcommand

DIFF specifies the degree of differencing used to convert a nonstationary series to a stationary one with a constant mean and variance before the autocorrelations are computed.

- You can specify 0 or any positive integer on DIFF.
- If DIFF is specified without a value, the default is 1.
- The number of values used in the calculations decreases by 1 for each degree–1 of differencing.

Example

```
ACF VARIABLES = SALES
  /DIFF=1.
```

- In this example, the series *SALES* will be differenced once before the autocorrelations are computed and plotted.

SDIFF Subcommand

If the series exhibits a seasonal or periodic pattern, you can use the SDIFF subcommand to seasonally difference the series before obtaining autocorrelations.

- The specification on SDIFF indicates the degree of seasonal differencing and can be 0 or any positive integer.
- If SDIFF is specified without a value, the degree of seasonal differencing defaults to 1.
- The number of seasons used in the calculations decreases by 1 for each degree of seasonal differencing.
- The length of the period used by SDIFF is specified on the PERIOD subcommand. If the PERIOD subcommand is not specified, the periodicity established on the TSET or DATE command is used (see the PERIOD subcommand).

PERIOD Subcommand

PERIOD indicates the length of the period to be used by the SDIFF or SEASONAL subcommands.

- The specification on PERIOD indicates how many observations are in one period or season and can be any positive integer greater than 1.
- The PERIOD subcommand is ignored if it is used without the SDIFF or SEASONAL subcommands.
- If PERIOD is not specified, the periodicity established on TSET PERIOD is in effect. If TSET PERIOD is not specified, the periodicity established on the DATE command is used. If periodicity was not established anywhere, the SDIFF and SEASONAL subcommands will not be executed.

Example

```
ACF VARIABLES = SALES
  /SDIFF=1M
  /PERIOD=12.
```

- This command applies one degree of seasonal differencing with a periodicity (season) of 12 to the series *SALES* before autocorrelations are computed.

LN and NOLOG Subcommands

LN transforms the data using the natural logarithm (base *e*) of the series and is used to remove varying amplitude over time. NOLOG indicates that the data should not be log transformed. NOLOG is the default.

- If you specify LN on an ACF command, any differencing requested on that command will be done on the log-transformed series.
- There are no additional specifications on LN or NOLOG.
- Only the last LN or NOLOG subcommand on an ACF command is executed.
- If a natural log transformation is requested when there are values in the series that are less than or equal to zero, the ACF will not be produced for that series because nonpositive values cannot be log transformed.
- NOLOG is generally used with an APPLY subcommand to turn off a previous LN specification.

Example

```
ACF VARIABLES = SALES
  /LN.
```

- This command transforms the series *SALES* using the natural log transformation and then computes and plots autocorrelations.

SEASONAL Subcommand

Use the SEASONAL subcommand to focus attention on the seasonal component by displaying and plotting autocorrelations at periodic lags only.

- There are no additional specifications on SEASONAL.
- If SEASONAL is specified, values are displayed and plotted at the periodic lags indicated on the PERIOD subcommand. If PERIOD is not specified, the periodicity established on the TSET or DATE command is used (see the PERIOD subcommand).
- If SEASONAL is not specified, autocorrelations for all lags up to the maximum are displayed and plotted.

Example

```
ACF VARIABLES = SALES
  /SEASONAL
  /PERIOD=12.
```

- In this example, autocorrelations are displayed only at every 12th lag.

MXAUTO Subcommand

MXAUTO specifies the maximum number of lags for a series.

- The specification on MXAUTO must be a positive integer.
- If MXAUTO is not specified, the default number of lags is the value set on TSET MXAUTO. If TSET MXAUTO is not specified, the default is 16.
- The value on MXAUTO overrides the value set on TSET MXAUTO.

Example

```
ACF VARIABLES = SALES
  /MXAUTO=14.
```

- This command sets the maximum number of autocorrelations to be displayed for the series *SALES* to 14.

SERROR Subcommand

SERROR specifies the method of calculating the standard errors for the autocorrelations.

- You must specify either the keyword IND or MA on SERROR.
- The method specified on SERROR overrides the method specified on the TSET ACFSE command.
- If SERROR is not specified, the method indicated on TSET ACFSE is used. If TSET ACFSE is not specified, the default is IND.

IND . *Independence model.* The method of calculating the standard errors assumes that the underlying process is white noise.

MA . *MA model*. The method of calculating the standard errors is based on Bartlett's approximation. With this method, appropriate where the true MA order of the process is k–1, standard errors grow at increased lags [1].

Example

```
ACF VARIABLES = SALES
  /SERROR=MA.
```

[1] Pankratz, A. 1983. *Forecasting with univariate Box-Jenkins models: Concepts and cases*. New York: John Wiley and Sons.

- In this example, the standard errors of the autocorrelations are computed using the MA method.

PACF Subcommand

Use the PACF subcommand to display and plot sample partial autocorrelations as well as autocorrelations for each series named on the ACF command.

- There are no additional specifications on PACF.
- PACF also displays the standard errors of the partial autocorrelations and indicates the bounds of two standard errors on the plot.
- With the exception of SERROR, all other subcommands specified on that ACF command apply to both the partial autocorrelations and the autocorrelations.

Example

```
ACF VARIABLES = SALES
  /DIFFERENCE=1
  /PACF.
```

- This command requests both autocorrelations and partial autocorrelations for the series *SALES* after it has been differenced once.

APPLY Subcommand

APPLY allows you to use a previously defined ACF model without having to repeat the specifications.

- The only specification on APPLY is the name of a previous model in quotation marks. If a model name is not specified, the model specified on the previous ACF command is used.
- To change one or more model specifications, specify the subcommands of only those portions you want to change after the APPLY subcommand.
- If no series are specified on the ACF command, the series that were originally specified with the model being reapplied are used.
- To change the series used with the model, enter new series names before or after the APPLY subcommand.

Example

```
ACF VARIABLES = TICKETS
  /LN
  /DIFF=1
  /SDIFF=1
  /PERIOD=12
  /MXAUTO=50.
ACF VARIABLES = ROUNDTRP
  /APPLY.
ACF APPLY
  /NOLOG.
ACF APPLY 'MOD_2'
  /PERIOD=6.
```

- The first command requests a maximum of 50 autocorrelations for the series *TICKETS* after a natural log transformation, differencing, and one degree of seasonal differencing with a periodicity of 12 have been applied. This model is assigned the default name *MOD_1*.
- The second command displays and plots the autocorrelation function for the series *ROUNDTRP* using the same model that was used for the series *TICKETS*. This model is assigned the name *MOD_2*.
- The third command requests another autocorrelation function of the series *ROUNDTRP* using the same model but without the natural log transformation. Note that when APPLY is the first specification after the ACF command, the slash (/) before it is not necessary. This model is assigned the name *MOD_3*.
- The fourth command reapplies *MOD_2*, autocorrelations for the series *ROUNDTRP* with the natural log and differencing specifications, but this time with a periodicity of 6. This model is assigned the name *MOD_4*. It differs from *MOD_2* only in the periodicity.

References

Box, G. E. P., and G. M. Jenkins. 1976. *Time series analysis: Forecasting and control*, Rev. ed. San Francisco: Holden-Day.

Pankratz, A. 1983. *Forecasting with univariate Box-Jenkins models: Concepts and cases*. New York: John Wiley and Sons.

ADD DOCUMENT

```
ADD DOCUMENT
  'text'
  'text'.
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic "Command Order" on page 36 for more information.

Example

```
ADD DOCUMENT
  "This data file is a 10% random sample from the"
  "primary data file. It's seed value is 13254689.".
```

Overview

ADD DOCUMENT saves a block of text of any length in the active dataset. The result is equivalent to the DOCUMENT command. The documentation can be displayed with the DISPLAY DOCUMENT command.

When GET retrieves a data file, or APPLY DICTIONARY is used to apply documents from another data file, or ADD FILES, MATCH FILES, or UPDATE is used to combine data files, all documents from each specified file are copied into the working file. DROP DOCUMENTS can be used to drop those documents from the working file.

Basic Specification

The basic specification is ADD DOCUMENT followed by one or more optional lines of quoted text. The text is stored in the file dictionary when the data file is saved in IBM SPSS Statistics format.

Syntax Rules

- Each line must be enclosed in single or double quotation marks, following the standard rules for quoted strings.
- Each line can be up to 80 bytes long (typically 80 characters in single-byte languages), including the command name but not including the quotation marks used to enclose the text. If any line exceeds 80 bytes, an error will result and the command will not be executed.
- The text can be entered on as many lines as needed.
- Multiple ADD DOCUMENT commands can be specified for the same data file.
- The text from each ADD DOCUMENT command is appended to the end of the list of documentation, followed by the date in parentheses.
- An ADD DOCUMENT command with no quoted text string appends a date in parentheses to the documentation.
- DISPLAY DOCUMENTS will display all documentation for the data file specified on the ADD DOCUMENT and/or DOCUMENT commands. Documentation is displayed exactly as entered; each line of the ADD DOCUMENT command is displayed as a separate line, and there is no line wrapping.
- DROP DOCUMENTS deletes all documentation created by both ADD DOCUMENT and DOCUMENT.

Example

If the command name and the quoted text string are specified on the same line, the command name counts toward the 80-byte line limit, so it's a good idea to put the command name on a separate line, as in:

```
ADD DOCUMENT
  "This is some text that describes this file.".
```

Example

To insert blank lines between blocks of text, enter a null string, as in:

```
ADD DOCUMENT
  "This is some text that describes this file."
  ""
  "This is some more text preceded by a blank line.".
```

ADD FILES

```
ADD FILES FILE={'savfile'|'dataset'} [PASSWORD='password']
              {*                   }

 [/RENAME=(old varnames=new varnames)...]

 [/IN=varname]

 /FILE=... [PASSWORD='password']... [/RENAME=...] [/IN=...]

 [/BY varlist]

 [/MAP]

 [/KEEP={ALL**  }] [/DROP=varlist]
        {varlist}

 [/FIRST=varname]  [/LAST=varname]
```

**Default if the subcommand is omitted.

Release History

Release 22.0

- PASSWORD keyword introduced on the FILE subcommand.

Example

```
ADD FILES FILE="/data/school1.sav" /FILE="/data/school2.sav".
```

Overview

ADD FILES combines cases from 2 up to 50 open data sets or external IBM SPSS Statistics data files by concatenating or interleaving cases. When cases are **concatenated**, all cases from one file are added to the end of all cases from another file. When cases are **interleaved**, cases in the resulting file are ordered according to the values of one or more key variables.

The files specified on ADD FILES can be external IBM SPSS Statistics data files and/or currently open datasets. The combined file becomes the new active dataset.

In general, ADD FILES is used to combine files containing the same variables but different cases. To combine files containing the same cases but different variables, use MATCH FILES. To update existing IBM SPSS Statistics data files, use UPDATE.

Options

Variable Selection. You can specify which variables from each input file are included in the new active dataset using the DROP and KEEP subcommands.

Variable Names. You can rename variables in each input file before combining the files using the RENAME subcommand. This permits you to combine variables that are the same but whose names differ in different input files or to separate variables that are different but have the same name.

Variable Flag. You can create a variable that indicates whether a case came from a particular input file using IN. When interleaving cases, you can use the FIRST or LAST subcommands to create a variable that flags the first or last case of a group of cases with the same value for the key variable.

Variable Map. You can request a map showing all variables in the new active dataset, their order, and the input files from which they came using the MAP subcommand.

Basic Specification

- The basic specification is two or more FILE subcommands, each of which specifies a file to be combined. If cases are to be interleaved, the BY subcommand specifying the key variables is also required.

- All variables from all input files are included in the new active dataset unless DROP or KEEP is specified.

Subcommand Order

- RENAME and IN must immediately follow the FILE subcommand to which they apply.
- BY, FIRST, and LAST must follow all FILE subcommands and their associated RENAME and IN subcommands.

Syntax Rules

- RENAME can be repeated after each FILE subcommand. RENAME applies only to variables in the file named on the FILE subcommand immediately preceding it.
- BY can be specified only once. However, multiple key variables can be specified on BY. When BY is used, all files must be sorted in ascending order by the key variables (see SORT CASES).
- FIRST and LAST can be used only when files are interleaved (when BY is used).
- MAP can be repeated as often as desired.

Operations

- ADD FILES reads all input files named on FILE and builds a new active dataset. ADD FILES is executed when the data are read by one of the procedure commands or the EXECUTE, SAVE, or SORT CASES commands.
  - If the current active dataset is included and is specified with an asterisk (FILE=*), the new merged dataset replaces the active dataset. If that dataset is a named dataset, the merged dataset retains that name. If the current active dataset is not included or is specified by name (for example, FILE=Dataset1), a new unnamed, merged dataset is created, and it becomes the active dataset. For information on naming datasets, see "DATASET NAME" on page 543.
- The resulting file contains complete dictionary information from the input files, including variable names, labels, print and write formats, and missing-value indicators. It also contains the documents from each input file. See DROP DOCUMENTS for information on deleting documents.
- For each variable, dictionary information is taken from the first file containing value labels, missing values, or a variable label for the common variable. If the first file has no such information, ADD FILES checks the second file, and so on, seeking dictionary information.
- Variables are copied in order from the first file specified, then from the second file specified, and so on. Variables that are not contained in all files receive the system-missing value for cases that do not have values for those variables.
- If the same variable name exists in more than one file but the format type (numeric or string) does not match, the command is not executed.
- If a numeric variable has the same name but different formats (for example, F8.0 and F8.2) in different input files, the format of the variable in the first-named file is used.
- If a string variable has the same name but different formats (for example, A24 and A16) in different input files, the command is not executed.
- If the active dataset is named as an input file, any N and SAMPLE commands that have been specified are applied to the active dataset before the files are combined.
- If only one of the files is weighted, the program turns weighting off when combining cases from the two files. To weight the cases, use the WEIGHT command again.

Limitations

- A maximum of 50 files can be combined on one ADD FILES command.
- The TEMPORARY command cannot be in effect if the active dataset is used as an input file.

Examples

```
ADD FILES FILE="/data/school1.sav"
        /FILE="/data/school2.sav".
```

- ADD FILES concatenates cases from the IBM SPSS Statistics data files *school1.sav* and *school2.sav*. All cases from *school1.sav* precede all cases from *school2.sav* in the resulting file.

```
SORT CASES BY LOCATN DEPT.
ADD FILES  FILE="/data/source.sav" /FILE=* /BY LOCATN DEPT
 /KEEP AVGHOUR AVGRAISE LOCATN DEPT SEX HOURLY RAISE /MAP.
SAVE OUTFILE="/data/prsnnl.sav".
```

- SORT CASES sorts cases in the active dataset in ascending order of their values for *LOCATN* and *DEPT*.
- ADD FILES combines two files: the external IBM SPSS Statistics data file *source.sav* and the sorted active dataset. The file *source.sav* must also be sorted by *LOCATN* and *DEPT*.
- BY indicates that the keys for interleaving cases are *LOCATN* and *DEPT*, the same variables used on SORT CASES.
- KEEP specifies the variables to be retained in the resulting file.
- MAP produces a list of variables in the resulting file and the two input files.
- SAVE saves the resulting file as a new IBM SPSS Statistics data file named *prsnnl.sav*.

FILE Subcommand

FILE identifies the files to be combined. A separate FILE subcommand must be used for each input file.

- An asterisk may be specified on FILE to indicate the active dataset.
- Dataset names instead of file names can be used to refer to currently open datasets.
- The order in which files are named determines the order of cases in the resulting file.

PASSWORD Keyword

The PASSWORD keyword specifies the password required to open an encrypted IBM SPSS Statistics data file. The specified value must be enclosed in quotation marks and can be provided as encrypted or as plain text. Encrypted passwords are created when pasting command syntax from the Save Data As dialog. The PASSWORD keyword is ignored if the file is not encrypted.

Example

```
GET DATA /TYPE=XLS  /FILE='/temp/excelfile1.xls'.
DATASET NAME exceldata1.
GET DATA /TYPE=XLS  /FILE='/temp/excelfile2.xls'.
ADD FILES FILE='exceldata1'
         /FILE=*
         /FILE='/temp/mydata.sav'.
```

RENAME Subcommand

RENAME renames variables in input files *before* they are processed by ADD FILES. RENAME follows the FILE subcommand that specifies the file containing the variables to be renamed.

- RENAME applies only to the FILE subcommand immediately preceding it. To rename variables from more than one input file, enter a RENAME subcommand after each FILE subcommand that specifies a file with variables to be renamed.
- Specifications for RENAME consist of a left parenthesis, a list of old variable names, an equals sign, a list of new variable names, and a right parenthesis. The two variable lists must name or imply the same number of variables. If only one variable is renamed, the parentheses are optional.
- More than one such specification can be entered on a single RENAME subcommand, each enclosed in parentheses.
- The TO keyword can be used to refer to consecutive variables in the file and to generate new variable names.
- RENAME takes effect immediately. KEEP and DROP subcommands entered prior to RENAME must use the old names, while those entered after RENAME must use the new names.

- All specifications within a single set of parentheses take effect simultaneously. For example, the specification RENAME (A,B = B,A) swaps the names of the two variables.
- Variables cannot be renamed to scratch variables.
- Input data files are not changed on disk; only the copy of the file being combined is affected.

Example

```
ADD FILES FILE="/data/clients.sav" /RENAME=(TEL_NO, ID_NO = PHONE, ID)
 /FILE="/data/primary.sav" /BY ID.
```

- ADD FILES adds new client cases from the file *clients.sav* to existing client cases in the file *primary.sav*.
- Two variables on *clients.sav* are renamed prior to the match. *TEL_NO* is renamed *PHONE* to match the name used for phone numbers in the primary file. *ID_NO* is renamed *ID* so that it will have the same name as the identification variable in the primary file and can be used on the BY subcommand.
- The BY subcommand orders the resulting file according to client ID number.

BY Subcommand

BY specifies one or more key variables that determine the order of cases in the resulting file. When BY is specified, cases from the input files are interleaved according to their values for the key variables.

- BY must follow the FILE subcommands and any associated RENAME and IN subcommands.
- The key variables specified on BY must be present and have the same names in all input files.
- Key variables can be string or numeric.
- All input files must be sorted in ascending order of the key variables. If necessary, use SORT CASES before ADD FILES.
- Cases in the resulting file are ordered by the values of the key variables. All cases from the first file with the first value for the key variable are first, followed by all cases from the second file with the same value, followed by all cases from the third file with the same value, and so forth. These cases are followed by all cases from the first file with the next value for the key variable, and so on.
- Cases with system-missing values are first in the resulting file. User-missing values are interleaved with other values.

DROP and KEEP Subcommands

DROP and KEEP are used to include only a subset of variables in the resulting file. DROP specifies a set of variables to exclude and KEEP specifies a set of variables to retain.

- DROP and KEEP do not affect the input files on disk.
- DROP and KEEP must follow all FILE and RENAME subcommands.
- DROP and KEEP must specify one or more variables. If RENAME is used to rename variables, specify the new names on DROP and KEEP.
- DROP and KEEP take effect immediately. If a variable specified on DROP or KEEP does not exist in the input files, was dropped by a previous DROP subcommand, or was not retained by a previous KEEP subcommand, the program displays an error message and does not execute the ADD FILES command.
- DROP cannot be used with variables created by the IN, FIRST, or LAST subcommands.
- KEEP can be used to change the order of variables in the resulting file. With KEEP, variables are kept in the order in which they are listed on the subcommand. If a variable is named more than once on KEEP, only the first mention of the variable is in effect; all subsequent references to that variable name are ignored.

- The keyword ALL can be specified on KEEP. ALL must be the last specification on KEEP, and it refers to all variables not previously named on that subcommand. It is useful when you want to arrange the first few variables in a specific order.

Example

```
ADD FILES FILE="/data/particle.sav" /RENAME=(PARTIC=pollute1)
 /FILE="/data/gas.sav" /RENAME=(OZONE TO SULFUR=pollut2 TO pollute4)
 /KEEP=pollute1 pollute2 pollute3 pollute4.
```

- The renamed variables are retained in the resulting file. KEEP is specified after all the FILE and RENAME subcommands, and it refers to the variables by their new names.

IN Subcommand

IN creates a new variable in the resulting file that indicates whether a case came from the input file named on the preceding FILE subcommand. IN applies only to the file specified on the immediately preceding FILE subcommand.

- IN has only one specification, the name of the flag variable.
- The variable created by IN has the value 1 for every case that came from the associated input file and the value 0 for every case that came from a different input file.
- Variables created by IN are automatically attached to the end of the resulting file and cannot be dropped. If FIRST or LAST are used, the variable created by IN precedes the variables created by FIRST or LAST.

Example

```
ADD FILES  FILE="/data/week10.sav" /FILE="/data/week11.sav"
  /IN=INWEEK11 /BY=EMPID.
```

- IN creates the variable *INWEEK11*, which has the value 1 for all cases in the resulting file that came from the input file *week11.sav* and the value 0 for those cases that were not in the file *week11.sav*.

Example

```
ADD FILES  FILE="/data/week10.sav" /FILE="/data/week11.sav"
  /IN=INWEEK11 /BY=EMPID.
IF  (NOT INWEEK11) SALARY1=0.
```

- The variable created by IN is used to screen partially missing cases for subsequent analyses.
- Since IN variables have either the value 1 or 0, they can be used as logical expressions, where 1 = true and 0 = false. The IF command sets the variable *SALARY1* equal to 0 for all cases that came from the file *INWEEK11*.

FIRST and LAST Subcommands

FIRST and LAST create logical variables that flag the first or last case of a group of cases with the same value on the BY variables. FIRST and LAST must follow all FILE subcommands and their associated RENAME and IN subcommands.

- FIRST and LAST have only one specification, the name of the flag variable.
- FIRST creates a variable with the value 1 for the first case of each group and the value 0 for all other cases.
- LAST creates a variable with the value 1 for the last case of each group and the value 0 for all other cases.
- Variables created by FIRST and LAST are automatically attached to the end of the resulting file and cannot be dropped.

Example

```
ADD FILES  FILE="/data/school1.sav" /FILE="/data/school2.sav"
 /BY=GRADE /FIRST=HISCORE.
```

- The variable *HISCORE* contains the value 1 for the first case in each grade in the resulting file and the value 0 for all other cases.

MAP Subcommand

MAP produces a list of the variables included in the new active dataset and the file or files from which they came. Variables are listed in the order in which they exist in the resulting file. MAP has no specifications and must follow after all FILE and RENAME subcommands.

- Multiple MAP subcommands can be used. Each MAP subcommand shows the current status of the active dataset and reflects only the subcommands that precede the MAP subcommand.
- To obtain a map of the active dataset in its final state, specify MAP last.
- If a variable is renamed, its original and new names are listed. Variables created by IN, FIRST, and LAST are not included in the map, since they are automatically attached to the end of the file and cannot be dropped.

Adding Cases from Different Data Sources

You can add cases from any data source that IBM SPSS Statistics can read by defining dataset names for each data source that you read (DATASET NAME command) and then using ADD FILES to add the cases from each file. The following example merges the contents of three text data files, but it could just as easily merge the contents of a text data file, and Excel spreadsheet, and a database table.

Example

```
DATA LIST FILE="/data/gasdata1.txt"
  /1 OZONE 10-12 CO 20-22 SULFUR 30-32.
DATASET NAME gasdata1.
DATA LIST FILE="/data/gasdata2.txt"
  /1 OZONE 10-12 CO 20-22 SULFUR 30-32.
DATASET NAME gasdata2.
DATA LIST FILE="/data/gasdata3.txt"
  /1 OZONE 10-12 CO 20-22 SULFUR 30-32.
DATASET NAME gasdata3.
ADD FILES FILE='gasdata1'
  /FILE='gasdata2'
  /FILE='gasdata3'.
SAVE OUTFILE='/data/combined_data.sav'.
```

ADD VALUE LABELS

```
ADD VALUE LABELS varlist value 'label' value 'label'...[/varlist...]
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
ADD VALUE LABELS JOBGRADE 'P' 'Parttime Employee'
                          'C' 'Customer Support'.
```

Overview

ADD VALUE LABELS adds or alters value labels without affecting other value labels already defined for that variable. In contrast, VALUE LABELS adds or alters value labels but deletes all existing value labels for that variable when it does so.

Basic Specification

The basic specification is a variable name and individual values with associated labels.

Syntax Rules

- Labels can be assigned to values of any previously defined variable. It is not necessary to enter value labels for all of a variable’s values.
- Each value label must be enclosed in single or double quotes.
- To specify a single quote or apostrophe within a quoted string, either enclose the entire string in double quotes or double the single quote/apostrophe.
- Value labels can contain any characters, including blanks.
- The same labels can be assigned to the same values of different variables by specifying a list of variable names. For string variables, the variables on the list must have the same defined width (for example, A8).
- Multiple sets of variable names and value labels can be specified on one ADD VALUE LABELS command as long as each set is separated from the previous one by a slash.
- To continue a label from one command line to the next, specify a plus sign (+) before the continuation of the label and enclose each segment of the label, including the blank between them, in single or double quotes.

Operations

- Unlike most transformations, ADD VALUE LABELS takes effect as soon as it is encountered in the command sequence. Thus, special attention should be paid to its position among commands.
- The added value labels are stored in the active dataset dictionary.
- ADD VALUE LABELS can be used for variables that have no previously assigned value labels.
- Adding labels to some values does not affect labels previously assigned to other values.

Limitations

- Value labels cannot exceed 120 bytes.

Examples

Adding Value Labels

```
ADD VALUE LABELS V1 TO V3 1 'Officials & Managers'
                          6 'Service Workers'
               /V4 'N' 'New Employee'.
```

- Labels are assigned to the values 1 and 6 of the variables between and including *V1* and *V3* in the active dataset.
- Following the required slash, a label for the value *N* for the variable *V4* is specified. *N* is a string value and must be enclosed in single or double quotes.
- If labels already exist for these values, they are changed in the dictionary. If labels do not exist for these values, new labels are added to the dictionary.
- Existing labels for other values for these variables are not affected.

Specifying a Label on Multiple Lines

```
ADD VALUE LABELS OFFICE88 1 "EMPLOYEE'S OFFICE ASSIGNMENT PRIOR"
   + " TO 1988".
```

- The label for the value 1 for *OFFICE88* is specified on two command lines. The plus sign concatenates the two string segments, and a blank is included at the beginning of the second string in order to maintain correct spacing in the label.

Value Labels for String Variables

- For string variables, the values and the labels must be enclosed in single or double quotes.
- If a specified value is longer than the defined width of the variable, the program displays a warning and truncates the value. The added label will be associated with the truncated value.
- If a specified value is shorter than the defined width of the variable, the program adds blanks to right-pad the value without warning. The added label will be associated with the padded value.
- If a single set of labels is to be assigned to a list of string variables, the variables must have the same defined width (for example, A8).

Example

```
ADD VALUE LABELS  STATE 'TEX' 'TEXAS' 'TEN' 'TENNESSEE'
                        'MIN' 'MINNESOTA'.
```

- ADD VALUE LABELS assigns labels to three values of the variable *STATE*. Each value and each label is specified in quotes.
- Assuming that the variable *STATE* is defined as three characters wide, the labels *TEXAS*, *TENNESSEE*, and *MINNESOTA* will be appropriately associated with the values *TEX*, *TEN*, and *MIN*. However, if *STATE* was defined as two characters wide, the program would truncate the specified values to two characters and would not be able to associate the labels correctly. Both *TEX* and *TEN* would be truncated to *TE* and would first be assigned the label *TEXAS*, which would then be changed to *TENNESSEE* by the second specification.

Example

```
ADD VALUE LABELS STATE REGION "U" "UNKNOWN".
```

- The label *UNKNOWN* is assigned to the value *U* for both *STATE* and *REGION*.
- *STATE* and *REGION* must have the same defined width. If they do not, a separate specification must be made for each, as in the following:

```
ADD VALUE LABELS STATE "U" "UNKNOWN" / REGION "U" "UNKNOWN".
```

ADP

ADP is available in Statistics Base Edition.

```
ADP
  /FIELDS [TARGET=targetField]
          INPUT=predictorFieldlist
          [ANALYSISWEIGHT=analysisweightField]
  [/PREPDATETIME
          [DATEDURATION={YES**(REFERENCE={CURRENT**    }
                                         {YMD(datespec)}
                              UNIT={AUTO**                      })}]
                                   {YEARS[(SUFFIX={'_years'  })] }
                                                  {suffixname}
                                   {MONTHS[(SUFFIX={'_months  })]}
                                                   {suffixname}
                                   {DAYS[(SUFFIX={'_days'   })]  }
                                                  {suffixname}
                        {NO                                      }
          [TIMEDURATION={YES**(REFERENCE={CURRENT**    }
                                         {HMS(timespec)}
                              UNIT={AUTO**                      })}]
                                   {HOURS[(SUFFIX={'_hours'  })]  }
                                                  {suffixname}
                                   {MINUTES[(SUFFIX={'_minutes'})]}
                                                   {suffixname}
                                   {SECONDS[(SUFFIX={'_seconds'})]}
                                                   {suffixname}
                        {NO                                       }
          [EXTRACTYEAR={YES[(SUFFIX={'_year'   })]}]
                                   {suffixname}
                       {NO**                      }
          [EXTRACTMONTH={YES[(SUFFIX={'_month'  })]}]
                                    {suffixname}
                        {NO**                      }
          [EXTRACTDAY={YES(SUFFIX={'_day'    })}]
                                  {suffixname}
                       {NO**                    }
          [EXTRACTHOUR={YES(SUFFIX={'_hour'   })}]
                                  {suffixname}
                       {NO**                    }
          [EXTRACTMINUTE={YES(SUFFIX={'_minute' })}]
                                   {suffixname}
                       {NO**                      }
          [EXTRACTSECOND={YES(SUFFIX={'_second' })}]
                                   {suffixname}
                        {NO**                      }
  [/SCREENING [PCTMISSING={YES**(MAXPCT={50**})}]
                                      {value}
                         {NO            }
             [UNIQUECAT={YES**(MAXCAT={100**   })}]
                                     {integer}
                         {NO                  }
             [SINGLECAT={YES(MAXPCT={95** })}]
                                     {value}
                         {NO**                }
  [/ADJUSTLEVEL [INPUT={YES**}] [TARGET={YES**}]]
                        {NO   }          {NO   }
           [MAXVALORDINAL={10**   }]
                          {integer}
           [MINVALCONTINUOUS={5**    }]
                            {integer}
  [/OUTLIERHANDLING [INPUT={YES**}] [TARGET={YES**}]]
                             {NO   }          {NO   }
                   [CUTOFF=SD({3**  })]
                              {value}
                   [REPLACEWITH={CUTOFFVALUE**}]
                                {MISSING      }
  [/REPLACEMISSING [INPUT={YES**[(EXCLUDE([CONTINUOUS] [NOMINAL] [ORDINAL]))]}]
                           {NO                                              }
                  [TARGET={YES[(EXCLUDE([CONTINUOUS] [NOMINAL] [ORDINAL]))]}]
                            {NO**                                           }
  [/REORDERNOMINAL [INPUT={YES }] [TARGET={YES }]
                          {NO**}          {NO**}
  [/RESCALE [INPUT={ZSCORE**([MEAN={0**  }] [SD={1**  }])}]
                                  {value}      {value}
                 {MINMAX([MIN={0**  }] [MAX={100**}])}
                                  {value}       {value}
                   {NONE                                 }
           [TARGET={BOXCOX**([MEAN={0**  }] [SD={1**  }])}]
                                  {value}      {value}
                    {NONE                                  }
  [/TRANSFORM [MERGESUPERVISED={YES**(PVALUE={0.05**})}]
                                        {value }
                               {NO                    }
             [MERGEUNSUPERVISED={YES{([ORDINAL] [NOMINAL] [MINPCT={10** })}]
                                                          {value}
                                 {NO**                                  }
             [BINNING={SUPERVISED**(PVALUE={0.05**})}]
                                       {value }
                      {NONE                          }
             [SELECTION={YES**(PVALUE={0.05**})}]
                        {NO                         }
             [CONSTRUCTION={YES }(ROOT={feature })]
                                      {rootname}
                          {NO**}
  [/CRITERIA [SUFFIX(TARGET={'_transformed'} INPUT={'_transformed'})]
                           {suffixname    }        {suffixname    }
  /OUTFILE PREPXML='filespec'
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 18

- Command introduced.

Example

```
ADP
  /FIELDS TARGET=targetVar
          INPUT=var1 var2 var3
  /OUTFILE PREPXML='file.xml'.
```

Overview

Automated Data Preparation helps to prepare data for analysis by automating tedious and repetitive data preparation tasks that would otherwise be done manually. The operations it performs improve analysis speed, predictive power, and robustness. A key capability of the component is feature space construction—the discovery of useful sets of predictors from the data through transformation and combination of existing fields. Feature *selection* offers the ability to narrow the attribute space by screening out irrelevant fields, but Automated Data Preparation pairs selection and construction capabilities in order to automatically remove irrelevant fields that slow down or confuse algorithms and create new fields that boost predictive power.

Note that supported operations are performed without knowing what algorithms will be run on the data in further analyses—it is not a generalized data cleaner, nor does it have an understanding of business rules. Basic cleaning and integrity checks can be done using the IBM SPSS Statistics Data Validation procedure.

Options

Date and Time Handling. The year, month, and day can be extracted from fields containing dates, and new fields containing the durations since a reference date computed. Likewise, the hour, minute, and second can be extracted from fields containing times, and new fields containing the time since a reference time computed.

Screening. Fields with too many missing values, and categorical fields with too many unique values, or too many values concentrated in a single value, can be screened and excluded from further analysis.

Rescaling. Continuous inputs can optionally be rescaled using a *z* score or min-max transformation. A continuous target can optionally be rescaled using a Box-Cox transformation.

Transformations. The procedure can suggest transformations used to merge similar categories of categorical inputs, bin values of continuous inputs, and construct and select new input fields from continuous inputs using principal components analysis.

Other Target and Input Handling. The procedure can apply rules for handling outliers, replace missing values, recode the categories of nominal fields, and adjust the measurement level of continuous and ordinal fields.

Output. The procedure creates an XML file containing suggested operations. This can be merged with a model XML file using the Merge Model XML dialog (Utilities>Merge Model XML) or transformed into command syntax using `TMS IMPORT`.

Basic Specification

The basic specification is the `ADP` command with a `FIELDS` subcommand specifying the inputs and optionally a target, and an `OUTFILE` subcommand specifying where the transformation rules should be saved.

Syntax Rules

- The VARIABLES and OUTFILE subcommands are required; all other subcommands are optional.
- Subcommands may be specified in any order.
- Only a single instance of each subcommand is allowed.
- An error occurs if a keyword is specified more than once within a subcommand.
- Parentheses, equals signs, and slashes shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not allowed.

Limitations

- SPLIT FILE is ignored by this command.

Examples

Basic Specification

```
ADP
  /FIELDS TARGET=targetField
          INPUT=field1 field2 field3
  /OUTFILE PREPXML='file.xml'.
```

- ADP processes the target and input fields using the default settings.
- OUTFILE saves an XML file containing suggested operations. This can be merged with a model XML file using the Merge Model XML dialog (Utilities>Merge Model XML) or transformed into command syntax using TMS IMPORT.

FIELDS Subcommand

The FIELDS subcommand is used to specify the target, inputs, and optional weights.

- The FIELDS subcommand and the INPUT keyword are required.

TARGET Keyword

Specify a single field that will be used as a target in further analyses. The target field is processed based upon its defined measurement level; nominal, ordinal, or continuous. Use the VARIABLE LEVEL command to change a target field's measurement level.

INPUT Keyword

Specify one or more fields that will be used as inputs in further analsyses. Input fields are processed based upon their defined measurement level; nominal, ordinal, or continuous. Use the VARIABLE LEVEL command to change an input field's measurement level.

ANALYSISWEIGHT Keyword

Specify a variable containing analysis (regression) weights. The procedure incorporates analysis weights where appropriate in operations used to prepare the data.

The analysis weight variable must be numeric. Cases with a negative or zero analysis weight are ignored.

PREPDATETIME Subcommand

The PREPDATETIME subcommand specifies handling of date and time fields.

- If PREPDATETIME is not specified, by default the procedure computes date and time durations since the current date and time.
- The original date and time fields will not be recommended as model inputs following automated data preparation.

DATEDURATION Keyword

The DATEDURATION keyword computes the number of years/months/days since a reference date for each variable containing dates.

REFERENCE = CURRENT | YMD('datespec') . *Reference date.* Specify CURRENT to use the current date as the reference date. Use YMDto specify a custom reference date with the year, month, and day, in that order, in parentheses using a valid date format in quotes. The default is CURRENT.

UNIT=AUTO | YEARS | MONTHS | DAYS. *Date units for computed durations.* Specify the units for the computed durations. AUTOdetermines the units based on the following rules. The default is AUTO.

- If the minimum number of elapsed days is less than 31, then the duration is returned in days.
- If the minimum number of elapsed days is less than 366 but greater than or equal to 31, then the duration is returned in months. The number of months between two dates is calculated based on average number of days in a month (30.4375): *months* = *days*/ 30.4375.
- If the minimum number of elapsed days greater than or equal to 366, then the duration is returned in years. The number of years between two dates is calculated based on average number of days in a year (365.25): *years* = *days* / 365.25.

Explicitly specifying YEARS, MONTHS, or DAYS returns the duration in years, months, or days, respectively. Optionally, in parentheses, specify SUFFIX= with a suffix in quotes. The default suffix depends upon the unit; YEARS, MONTHS, and DAYS have defaults *_years*, *_months*, and *_days*, respectively.

TIMEDURATION Keyword

The TIMEDURATION keyword computes the number of hours/minutes/seconds since a reference time for each variable containing times.

REFERENCE = CURRENT | HMS('timespec'). *Reference date.* Specify CURRENT to use the current time as the reference time or use HMSand the hour, minute, and second, in that order, in parentheses using a valid time format in quotes. The default is CURRENT.

UNIT = AUTO | HOURS | MINUTES | SECONDS. *Date units for computed durations.* Specify the units for the computed durations. AUTOdetermines the units based on the following rules. The default is AUTO.

- If the minimum number of elapsed seconds is less than 60, then the duration is returned in seconds.
- If the minimum number of elapsed seconds is larger than or equal to 60 but less than 3600, then the duration is returned in minutes.
- If the minimum number of elapsed seconds is larger than or equal to 3600, then the duration is returned in hours.

Explicitly specifying HOURS, MINUTES, or SECONDS returns the duration in hours, minutes, or seconds, respectively. Optionally, in parentheses, specify SUFFIX= with a suffix in quotes. The default suffix depends upon the unit; HOURS, MINUTES, and SECONDS have defaults *_hours*, *_minutes*, and *_seconds*, respectively.

EXTRACTYEAR Keyword

The EXTRACTYEAR keyword extracts the year element from a date variable.

Optionally specify the SUFFIX keyword in parentheses with a suffix in quotes. The default suffix is *_year*.

EXTRACTMONTH Keyword

The EXTRACTMONTH keyword extracts the month element from a date variable.

Optionally specify the SUFFIX keyword in parentheses with a suffix in quotes. The default suffix is *_month*.

EXTRACTDAY Keyword

The EXTRACTDAY keyword extracts the day element from a date variable.

Optionally specify the SUFFIX keyword in parentheses with a suffix in quotes. The default suffix is *_day*.

EXTRACTHOUR Keyword

The EXTRACTHOUR keyword extracts the hour element from a time variable.

Optionally specify the SUFFIX keyword in parentheses with a suffix in quotes. The default suffix is *_hour*.

EXTRACTMINUTE Keyword

The EXTRACTMINUTEkeyword extracts the minute element from a time variable.

Optionally specify the SUFFIX keyword in parentheses with a suffix in quotes. The default suffix is *_minute*.

EXTRACTSECOND Keyword

The EXTRACTSECOND keyword extracts the second element from a time variable.

Optionally specify the SUFFIX keyword in parentheses with a suffix in quotes. The default suffix is *_second*.

SCREENING Subcommand

The SCREENING subcommand specifies settings for excluding unsuitable fields.

PCTMISSING = YES(MAXPCT=value) | NO. *Screen out fields with too many missing values.* Fields with more than MAXPCT missing values are removed from further analysis. Specify a value greater than or equal to 0, which is equivalent to deselecting this option, and less than or equal to 100, though fields with all missing values are automatically excluded. The default is 50.

UNIQUECAT = YES(MAXCAT=integer) | NO. *Screen out nominal fields with too many unique categories.*Nominal fields with more than MAXCATcategories are removed from further analysis. Specify a positive integer. The default is 100.

SINGLECAT = YES(MAXPCT=value) | NO. *Screen out categorical fields that are nearly constant.* Ordinal and nominal fields with a category that contains more than MAXPCT of the records are removed from further analysis. Specify a value greater than or equal to 0, equivalent to deselecting this option, and less than or equal to 100, though constant fields are automatically excluded. The default is 95.

ADJUSTLEVEL Subcommand

The ADJUSTLEVEL subcommand recasts ordinal fields with too many categories as continuous and continuous fields with too few unique values as ordinal. By default, the measurement levels of ordinal fields with more than 10 categories and continuous fields with fewer than 5 unique values are adjusted.

INPUT=YES | NO. *Check inputs and adjust measurement level if necessary.* By default, inputs are checked.

TARGET = YES | NO. *Check target and adjust measurment level if necessary.* By default, the target is checked.

MAXVALORDINAL = integer. *Maximum number of categories allowed for ordinal fields.* Ordinal fields with more than MAXVALORDINALcategories are recast as continuous fields. Specify a positive integer. The default is 10. The value of MAXVALORDINALmust be greater than or equal to MINVALCONTINUOUS.

MINVALCONTINUOUS = integer. *Minimum number of unique values allowed for continuous fields.* Continuous fields with less than MINVALCONTINUOUS unique values are recast as ordinal fields. Specify a positive integer. The default is 5. The value of MINVALCONTINUOUS must be less than or equal to MAXVALORDINAL.

OUTLIERHANDLING Subcommand

The OUTLIERHANDLING subcommand checks fields for outliers and replaces the outlying values with less extreme values or missing values.

INPUT=YES | NO. *Check inputs for outliers.* By default, inputs are checked.

TARGET=YES | NO. *Check target for outliers.* By default, the target is checked.

CUTOFF=SD(value) . *Cutoff for determining outliers.* If a value is more than SD"robust" standard deviations from the mean value for a field, then it is considered an outlier. Specify a positive number. The default is 3 standard deviations.

REPLACEWITH = CUTOFFVALUE | MISSING. *Value to replace outliers with.* CUTOFFVALUE replaces outliers with the cutoff for determining outliers. MISSING replaces outliers with the system-missing value. These missing values can be further handled by the REPLACEMISSING subcommand. The default is CUTOFFVALUE.

REPLACEMISSING Subcommand

The REPLACEMISSING subcommand replaces missing values in continuous, ordinal, and nominal fields with the mean, median, or mode, respectively.

INPUT=YES | NO. *Replace missing values in input fields.* By default, missing values are replaced in inputs. Optionally specify the keyword EXCLUDE and a list in parentheses of the field measurement levels to determine which input fields should be checked; for example: INPUT=YES causes the procedure to replace missing values in all input fields, while INPUT=YES(EXCLUDE(CONTINUOUS NOMINAL)) causes the procedure to replace missing values in fields with the ordinal measurement level.

TARGET= NO | YES. *Replace missing values in the target.* By default, missing values are not replaced in the target. When replacing missing values in the target, optionally specify the keyword EXCLUDE as described for the INPUTkeyword above.

REORDERNOMINAL Subcommand

The REORDERNOMINAL subcommand recodes the values of nominal fields from least frequently occurring to most frequently occurring. The new field values start with 0 as the least frequent category. Note that the new field will be numeric even if the original field is a string. For example, if a nominal field's data values are "A", "A", "A", "B", "C", "C", then automated data preparation would recode "B" into 0, "C" into 1, and "A" into 2.

INPUT=NO | YES. *Reorder values of inputs.* By default, values of nominal inputs are not reordered. This specification is ignored if there are no nominal inputs.

TARGET=NO | YES. *Reorder values of the target.* By default, values of a nominal target are not reordered. This specification is ignored if the target is not nominal.

RESCALE Subcommand

The RESCALE subcommand is used to rescale continuous fields. Different methods are available for inputs and the target.

INPUT Keyword

The INPUT keyword specifies the method for rescaling continuous input fields.

- *Z* score rescaling is performed by default with a mean of 0 and standard deviation of 1.
- If there are no continuous inputs, INPUT is ignored.

ZSCORE(MEAN=value SD=value). *Z score transformation.* Using the observed mean and standard deviation as population parameter estimates, the fields are standardized and then the z scores are mapped to the corresponding values of a normal distribution with the specified MEAN and SD. Specify a number for MEAN and a positive number for SD. The defaults are 0 and 1, respectively, corresponding to standardized rescaling.

MINMAX(MIN=value MAX=value). *Min-Max transformation.* Using the observed minimum and maximum as population parameter estimates, the fields are mapped to the corresponding values of a uniform distribution with the specified MINand MAX. Specify numbers with MAX greater than MIN.

NONE. *Do not rescale inputs.*

TARGET Keyword

The TARGET keyword specifies the method for rescaling a continuous target.

- Box-Cox rescaling is performed by default with a target mean of 0 and target standard deviation of 1.
- If there is no target, or it is not continuous, TARGET is ignored.

BOXCOX(MEAN=value SD=value). *Box-Cox transformation.* This transforms a continuous target using the Box-Cox transformation into a field that has an approximately normal distribution with the specified MEAN and SD. Specify a number for MEANand a positive number for SD. The defaults are 0 and 1, respectively.

NONE. *Do not rescale target.*

TRANSFORM Subcommand

The TRANSFORM subcommand is used to merge similar categories of categorical inputs, bin values of continuous inputs, and construct and select new input fields from continuous inputs using principal components analysis.

MERGESUPERVISED Keyword

The MERGESUPERVISED keyword specifies how to merge similar categories of a nominal or ordinal input in the presence of a target.

- If there are no categorical inputs, MERGESUPERVISED is ignored.
- If there is no target specified on the FIELDS subcommand, MERGESUPERVISED is ignored.

YES(PVALUE=value). *Supervised merge.* Similar categories are identified based upon the relationship between the input and the target. Categories that are not significantly different; that is, having a *p*-value greater than the value of PVALUE, are merged. Specify a value greater than 0 and less than or equal to 1. The default is 0.05. YES is the default.

NO. *Do not merge categories.*

MERGEUNSUPERVISED Keyword

The MERGEUNSUPERVISED keyword specifies how to merge similar categories of a nominal or ordinal input when there is no target.

- If there are no categorical inputs, MERGEUNSUPERVISEDi s ignored.
- If there is a target specified on the FIELDS subcommand, MERGEUNSUPERVISED is ignored.

YES(ORDINAL|NOMINAL|MINPCT=value). *Unsupervised merge.* The equal frequency method is used to merge categories with less than MINPCTof the total number of records. Specify a value greater than or equal to 0 and less than or equal to 100. The default is 10 if MINPCTis not specified. If YES is specified without ORDINAL or NOMINAL, then no merging is performed.

NO. *Do not merge categories.* NO is the default.

BINNING Keyword

The BINNING keyword specifies how to discretize continuous inputs in the presence of a categorical target.

SUPERVISED(PVALUE=value). *Supervised binning.* Bins are created based upon the properties of "homogeneous subsets", which are identified by the Scheffe method using PVALUE as the alpha for the critical value for determining homogeneous subsets. SUPERVISED is the default. Specify a value greater than 0 and less than or equal to 1. The default is 0.05

If there is no target specified on the FIELDS subcommand, or the target is not categorical, or there are no continuous inputs, then SUPERVISED is ignored.

NONE. *Do not bin values of continuous inputs.*

SELECTION Keyword

The SELECTION keyword specifies how to perform feature selection for continuous inputs in the presence of a continuous target.

YES(PVALUE=value). *Perform feature selection.* A continuous input is removed from the analysis if the *p*-value for its correlation with the target is greater than PVALUE. YES is the default.

If there is no target specified on the FIELDSsubcommand, or the target is not continuous, or there are no continuous inputs, then YESis ignored.

NO. *Do not perform feature selection.*

CONSTRUCTION Keyword

The CONSTRUCTIONkeyword specifies how to perform feature construction for continuous inputs in the presence of a continuous target.

YES(ROOT=rootname). *Perform feature construction.* New predictors are constructed from groups of "similar" predictors using principal component analysis. Optionally specify the rootname for constructed predictors using ROOT in parentheses. Specify a rootname (no quotes). The default is *feature*

If there is no target specified on the FIELDS subcommand, or the target is not continuous, or there are no continuous inputs, then YES is ignored.

NO. *Do not perform feature construction.* NO is the default.

CRITERIA Subcommand

The CRITERIA subcommand is used to specify the suffixes applied to transformed target and inputs.

SUFFIX Keyword

The SUFFIX keyword specifies the suffixes applied to transformed target and inputs.

TARGET=suffixname. *Suffix for transformed target.* Specify a suffix in quotes. The default is *_transformed*. If there is no target specified on the FIELDS subcommand, TARGET is ignored.

INPUT=suffixname. *Suffix for transformed inputs.* Specify a suffix in quotes. The default is *_transformed*.

OUTFILE Subcommand

The OUTFILE subcommand saves an XML-format file containing the rules for preparing the data.

- The OUTFILEsubcommand is required.
- File names must be specified in full. ADP does not supply extensions.

PREPXML='filespec'. *Save rules for preparing data to an XML file.* The rules are saved in an XML format to the specified file. This file can be merged with model PMML using TMS MERGE or transformed into command syntax using TMS IMPORT

AGGREGATE

```
AGGREGATE [OUTFILE={'savfile'|'dataset'}]
                   {*              }
          [MODE={REPLACE     }] [OVERWRITE={NO }]
                {ADDVARIABLES}             {YES}

 [/MISSING=COLUMNWISE] [/DOCUMENT]

 [/PRESORTED] [/BREAK=[varlist[({A})]][varlist...]]
                              {D  }

 /aggvar['label'] aggvar['label']...=function(arguments)

 [/aggvar ...]
```

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 13.0

- MODE keyword introduced.
- OVERWRITE keyword introduced.

Release 17.0

- AGGREGATE runs without a break variable.

Release 22.0

- CLT, CGT, CIN, and COUT functions introduced.

Example

```
AGGREGATE
  /OUTFILE='/temp/temp.sav'
  /BREAK=gender
  /age_mean=MEAN(age).
```

Overview

AGGREGATE aggregates groups of cases in the active dataset into single cases and creates a new aggregated file or creates new variables in the active dataset that contain aggregated data. The values of one or more variables in the active dataset define the case groups. These variables are called **break variables**. A set of cases with identical values for each break variable is called a **break group**. If no break variables are specified, then the entire dataset is a single break group. Aggregate functions are applied to **source variables** in the active dataset to create new aggregated variables that have one value for each break group.

Options

Data. You can create new variables in the active dataset that contain aggregated data, replace the active dataset with aggregated results, or create a new data file that contains the aggregated results.

Documentary Text. You can copy documentary text from the original file into the aggregated file using the DOCUMENT subcommand. By default, documentary text is dropped.

Aggregated Variables. You can create aggregated variables using any of 19 aggregate functions. The functions SUM, MEAN, and SD can aggregate only numeric variables. All other functions can use both numeric and string variables.

Labels and Formats. You can specify variable labels for the aggregated variables. Variables created with the functions MAX, MIN, FIRST, and LAST assume the formats and value labels of their source variables. All other variables assume the default formats described under .

Basic Specification

The basic specification is at least one aggregate function and source variable. The aggregate function creates a new aggregated variable in the active dataset.

Subcommand Order

- If specified, OUTFILE must be specified first.
- If specified, DOCUMENT and PRESORTED must precede BREAK. No other subcommand can be specified between these two subcommands.
- MISSING, if specified, must immediately follow OUTFILE.
- The aggregate functions must be specified last.

Operations

- When replacing the active dataset or creating a new data file, the aggregated file contains the break variables plus the variables created by the aggregate functions.
- AGGREGATE excludes cases with missing values from all aggregate calculations except those involving the functions N, NU, NMISS, and NUMISS.
- Unless otherwise specified, AGGREGATE sorts cases in the aggregated file in ascending order of the values of the grouping variables.
- PRESORTED uses a faster, less memory-intensive algorithm that assumes the data are already sorted into the desired groups.
- AGGREGATE ignores split-file processing. To achieve the same effect, name the variable or variables used to split the file as break variables before any other break variables. AGGREGATE produces one file, but the aggregated cases will then be in the same order as the split files.

Example

```
AGGREGATE
  /OUTFILE='/temp/temp.sav'
  /BREAK=gender marital
  /age_mean=MEAN(age)
  /age_median=MEDIAN(age)
  /income_median=MEDIAN(income).
```

- AGGREGATE creates a new IBM SPSS Statistics data file, *temp.sav*, that contains two break variables (*gender* and *marital*) and all of the new aggregate variables.
- BREAK specifies *gender* and *marital* as the break variables. In the aggregated file, cases are sorted in ascending order of *gender* and in ascending order of *marital* within *gender.* The active dataset remains unsorted.
- Three aggregated variables are created: *age_mean* contains the mean age for each group defined by the two break variables; *age_median* contains the median age; and *income_median* contains the median income.

OUTFILE Subcommand

OUTFILE specifies the handling of the aggregated results. It must be the first subcommand on the AGGREGATE command.

- OUTFILE='file specification' saves the aggregated data to a new file, leaving the active dataset unaffected. The file contains the new aggregated variables and the break variables that define the aggregated cases.
- A defined dataset name can be used for the file specification, saving the aggregated data to a dataset in the current session. The dataset must be defined before being used in the AGGREGATE command. See the topic "DATASET DECLARE" on page 539 for more information.

- OUTFILE=* with no additional keywords on the OUTFILE subcommand will replace the active dataset with the aggregated results.
- OUTFILE=* MODE=ADDVARIABLES appends the new variables with the aggregated data to the active dataset (instead of replacing the active dataset with the aggregated data).
- OUTFILE=* MODE=ADDVARIABLES OVERWRITE=YES overwrites variables in the active dataset if those variable names are the same as the aggregate variable names specified on the AGGREGATE command.
- MODE and OVERWRITE can be used only with OUTFILE=*; they are invalid with OUTFILE='file specification'.
- Omission of the OUTFILE subcommand is equivalent to OUTFILE=* MODE=ADDVARIABLES.

Example

```
AGGREGATE
  /BREAK=region
  /sales_mean = MEAN(var1)
  /sales_median = MEDIAN(var1)
  /sales_sum = SUM(var1).
```

- The aggregated variables are appended to the end of each case in the active data file. No existing cases or variables are deleted.
- For each case, the new aggregated variable values represent the mean, median, and total (sum) sales values for its region.

Creating a New Aggregated Data File versus Appending Aggregated Variables

When you create a new aggregated data file with OUTFILE='file specification' or OUTFILE=* MODE=REPLACE, the new file contains:

- The break variables from the original data file and the new aggregate variables defined by the aggregate functions. Original variables other than the break variables are not retained.
- One case for each group defined by the break variables. If there is one break variable with two values, the new data file will contain only two cases.

When you append aggregate variables to the active dataset with OUTFILE=* MODE=ADDVARIABLES, the modified data file contains:

- All of the original variables plus all of the new variables defined by the aggregate functions, with the aggregate variables appended to the end of each case.
- The same number of cases as the original data file. The data file itself is not aggregated. Each case with the same value(s) of the break variable(s) receives the same values for the new aggregate variables. For example, if *gender* is the only break variable, all males would receive the same value for a new aggregate variable that represents the average age.

Example

```
DATA LIST FREE /age (F2) gender (F2).
BEGIN DATA
25 1
35 1
20 2
30 2
60 2
END DATA.
*create new file with aggregated results.
AGGREGATE
  /OUTFILE='/temp/temp.sav'
  /BREAK=gender
  /age_mean=MEAN(age)
  /groupSize=N.
*append aggregated variables to active dataset.
AGGREGATE
  /OUTFILE=* MODE=ADDVARIABLES
  /BREAK=gender
  /age_mean=MEAN(age)
  /groupSize=N.
```

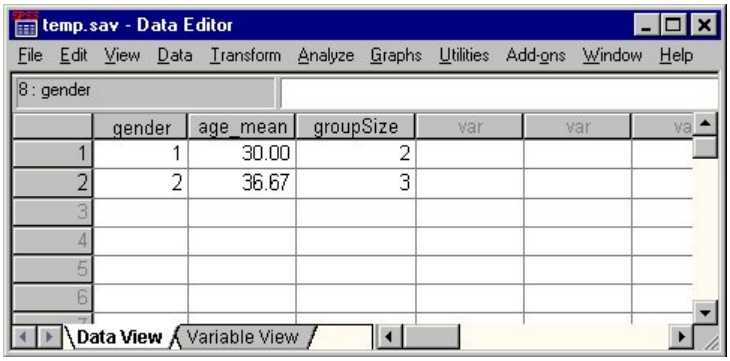


Figure 16. New aggregated data file

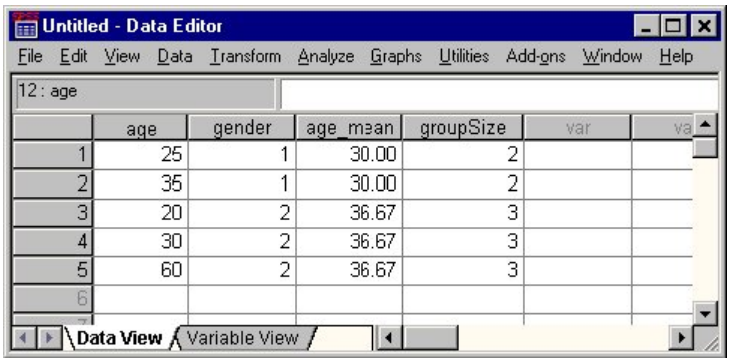


Figure 17. Aggregate variables appended to active dataset

BREAK Subcommand

BREAK lists the optional grouping variables, also called break variables. Each unique combination of values of the break variables defines one break group.

- The variables named on BREAK can be any combination of variables in the active dataset.
- Unless PRESORTED is specified, aggregated variables are appended to the active dataset (OUTFILE=* MODE=ADDVARIABLES), AGGREGATE sorts cases after aggregating. By default, cases are sorted in ascending order of the values of the break variables. AGGREGATE sorts first on the first break variable, then on the second break variable within the groups created by the first, and so on.
- Sort order can be controlled by specifying an A (for ascending) or D (for descending) in parentheses after any break variables.
- The designations A and D apply to all preceding undesignated variables.
- The subcommand PRESORTED overrides all sorting specifications, and no sorting is performed with OUTFILE=* MODE=ADDVARIABLES.

Example

```
AGGREGATE
  /BREAK=region
  /sales_mean = MEAN(var1)
  /sales_median = MEDIAN(var1)
  /sales_sum = SUM(var1).
```

For each case, the new aggregated variable values represent the mean, median, and total (sum) sales values for its region.

Example with no BREAK variable

```
AGGREGATE
  /sales_mean = MEAN(var1)
  /sales_median = MEDIAN(var1)
  /sales_sum = SUM(var1).
```

For each case, the new aggregated variable values represent the mean, median, and total (sum) sales values for the entire dataset.

DOCUMENT Subcommand

DOCUMENT copies documentation from the original file into the aggregated file.

- DOCUMENT must appear after OUTFILE but before BREAK.
- By default, documents from the original data file are not retained with the aggregated data file when creating a new aggregated data file with either OUTFILE='file specification' or OUTFILE=* MODE=REPLACE. The DOCUMENT subcommand retains the original data file documents.
- Appending variables with OUTFILE=* MODE=ADDVARIABLES has no effect on data file documents, and the DOCUMENT subcommand is ignored. If the data file previously had documents, they are retained.

PRESORTED Subcommand

If the data are already sorted into the desired groups, you can reduce run time and memory requirements by using the PRESORTED subcommand.

- If specified, PRESORTED must precede BREAK. The only specification is the keyword PRESORTED. PRESORTED has no additional specifications.
- When PRESORTED is specified, the program forms an aggregate case out of each group of *adjacent* cases with the same values for the break variables. Unless the cases are sorted by the break variables, the results will be quite different from what would be produced if PRESORTED were not specified.
- When PRESORTED is specified, if AGGREGATE is appending new variables to the active dataset rather than writing a new file or replacing the active dataset, the cases must be sorted in ascending order by the BREAK variables.

Example

```
AGGREGATE OUTFILE='/temp/temp.sav'
 /PRESORTED
 /BREAK=gender marital
 /mean_age=MEAN(age).
```

Aggregate Functions

An aggregated variable is created by applying an aggregate function to a variable in the active dataset. The variable in the active dataset is called the **source** variable, and the new aggregated variable is the **target** variable.

- The aggregate functions must be specified last on AGGREGATE.
- The simplest specification is a target variable list, followed by an equals sign, a function name, and a list of source variables.
- The number of target variables named must match the number of source variables.
- When several aggregate variables are defined at once, the first-named target variable is based on the first-named source variable, the second-named target is based on the second-named source, and so on.
- Only the functions MAX, MIN, FIRST, and LAST copy complete dictionary information from the source variable. For all other functions, new variables do not have labels and are assigned default dictionary print and write formats. The default format for a variable depends on the function used to create it (see the list of available functions below).
- You can provide a variable label for a new variable by specifying the label in single or double quotes immediately following the new variable name. Value labels cannot be assigned in AGGREGATE.

- To change formats or add value labels to an active dataset created by AGGREGATE, use the PRINT FORMATS, WRITE FORMATS, FORMATS, or VALUE LABELS command. If the aggregate file is written to disk, first retrieve the file using GET, specify the new labels and formats, and resave the file.

The following is a list of available functions:

SUM(varlist). *Sum across cases.*

MEAN(varlist). *Mean across cases.*

MEDIAN(varlist). *Median across cases*.

SD(varlist). *Standard deviation across cases.*

MAX(varlist). *Maximum value across cases.* Complete dictionary information is copied from the source variables to the target variables.

MIN(varlist). *Minimum value across cases.* Complete dictionary information is copied from the source variables to the target variables.

PGT(varlist,value). *Percentage of cases greater than the specified value.*

PLT(varlist,value). *Percentage of cases less than the specified value.*

PIN(varlist,value1,value2). *Percentage of cases between value1 and value2, inclusive.*

POUT(varlist,value1,value2). *Percentage of cases not between value1 and value2.* Cases where the source variable equals *value1* or *value2* are not counted.

FGT(varlist,value). *Fraction of cases greater than the specified value.*

FLT(varlist,value). *Fraction of cases less than the specified value*.

FIN(varlist,value1,value2). *Fraction of cases between value1 and value2, inclusive.*

FOUT(varlist,value1,value2). *Fraction of cases not between value1 and value2.* Cases where the source variable equals *value1* or *value2* are not counted.

CGT(varlist,value). *Count of cases greater than the specified value.*

CLT(varlist,value). *Count of cases less than the specified value*.

CIN(varlist,value1,value2). *Count of cases between value1 and value2, inclusive.*

COUT(varlist,value1,value2). *Count of cases not between value1 and value2.* Cases where the source variable equals *value1* or *value2* are not counted.

N(varlist). *Weighted number of cases in break group.*

NU(varlist). *Unweighted number of cases in break group.*

NMISS(varlist). *Weighted number of missing cases.*

NUMISS(varlist). *Unweighted number of missing cases.*

FIRST(varlist). *First nonmissing observed value in break group.* Complete dictionary information is copied from the source variables to the target variables.

LAST(varlist). *Last nonmissing observed value in break group.* Complete dictionary information is copied from the source variables to the target variables.

- The functions SUM, MEAN, and SD can be applied only to numeric source variables. All other functions can use short and long string variables as well as numeric ones.
- The N and NU functions do not require arguments. Without arguments, they return the number of weighted and unweighted valid cases in a break group. If you supply a variable list, they return the number of weighted and unweighted valid cases for the variables specified.
- For several functions, the argument includes values as well as a source variable designation. Either blanks or commas can be used to separate the components of an argument list.
- For percentage, fraction, and count within or outside a specified range, the first value specified should be less than or equal to the second. If not, they are automatically reversed. If the two values are equal,

PIN, FIN, and CIN calculate the percentage, fraction, or count equal to the argument. POUT, FOUT, and COUT calculate the percentage, fraction or count not equal to the argument.

- String values specified in an argument should be enclosed in quotes.

Using the MEAN Function

```
AGGREGATE OUTFILE='AGGEMP.SAV' /BREAK=LOCATN
 /AVGSAL 'Average Salary' AVGRAISE = MEAN(SALARY RAISE).
```

- AGGREGATE defines two aggregate variables, *AVGSAL* and *AVGRAISE*.
- *AVGSAL* is the mean of *SALARY* for each break group, and *AVGRAISE* is the mean of *RAISE*.
- The label *Average Salary* is assigned to *AVGSAL*.

Using the PLT Function

```
AGGREGATE OUTFILE=* /BREAK=DEPT
 /LOWVAC,LOWSICK = PLT (VACDAY SICKDAY,10).
```

- AGGREGATE creates two aggregated variables: *LOWVAC* and *LOWSICK*. *LOWVAC* is the percentage of cases with values less than 10 for *VACDAY*, and *LOWSICK* is the percentage of cases with values less than 10 for *SICKDAY*.

Using the FIN Function

```
AGGREGATE OUTFILE='GROUPS.SAV' /BREAK=OCCGROUP
 /COLLEGE = FIN(EDUC,13,16).
```

- AGGREGATE creates the variable *COLLEGE*, which is the fraction of cases with 13 to 16 years of education (variable *EDUC*).

Using the PIN Function

```
AGGREGATE OUTFILE=* /BREAK=CLASS
 /LOCAL = PIN(STATE,'IL','IO').
```

- AGGREGATE creates the variable *LOCAL*, which is the percentage of cases in each break group whose two-letter state code represents Illinois, Indiana, or Iowa. (The abbreviation for Indiana, IN, is between IL and IO in an alphabetical sort sequence.)

MISSING Subcommand

By default, AGGREGATE uses all nonmissing values of the source variable to calculate aggregated variables. An aggregated variable will have a missing value only if the source variable is missing for every case in the break group. You can alter the default missing-value treatment by using the MISSING subcommand. You can also specify the inclusion of user-missing values on any function.

- MISSING must immediately follow OUTFILE.
- COLUMNWISE is the only specification available for MISSING.
- If COLUMNWISE is specified, the value of an aggregated variable is missing for a break group if the source variable is missing for any case in the group.
- COLUMNWISE does not affect the calculation of the N, NU, NMISS, or NUMISS functions.
- COLUMNWISE does not apply to break variables. If a break variable has a missing value, cases in that group are processed and the break variable is saved in the file with the missing value. Use SELECT IF if you want to eliminate cases with missing values for the break variables.

Including Missing Values

You can force a function to include user-missing values in its calculations by specifying a period after the function name.

- AGGREGATE ignores periods used with the functions N, NU, NMISS, and NUMISS if these functions have no arguments.
- User-missing values are treated as valid when these four functions are followed by a period and have a variable as an argument. NMISS.(AGE) treats user-missing values as valid and thus gives the number of cases for which *AGE* has the system-missing value only.

The effect of specifying a period with N, NU, NMISS, and NUMISS is illustrated by the following:

```
N = N. = N(AGE) + NMISS(AGE) = N.(AGE) + NMISS.(AGE)
NU = NU. = NU(AGE) + NUMISS(AGE) = NU.(AGE) + NUMISS.(AGE)
```

- The function N (the same as N. with no argument) yields a value for each break group that equals the number of cases with valid values (N(AGE)) plus the number of cases with user- or system-missing values (NMISS(AGE)).
- This in turn equals the number of cases with either valid or user-missing values (N.(AGE)) plus the number with system-missing values (NMISS.(AGE)).
- The same identities hold for the NU, NMISS, and NUMISS functions.

Default Treatment of Missing Values

```
AGGREGATE OUTFILE='AGGEMP.SAV' /MISSING=COLUMNWISE /BREAK=LOCATN
 /AVGSAL = MEAN(SALARY).
```

- *AVGSAL* is missing for an aggregated case if *SALARY* is missing for any case in the break group.

Including User-Missing Values

```
AGGREGATE OUTFILE=* /BREAK=DEPT
 /LOVAC = PLT.(VACDAY,10).
```

- *LOVAC* is the percentage of cases within each break group with values less than 10 for *VACDAY*, even if some of those values are defined as user missing.

Aggregated Values that Retain Missing-Value Status

```
AGGREGATE OUTFILE='CLASS.SAV' /BREAK=GRADE
  /FIRSTAGE = FIRST.(AGE).
```

- The first value of *AGE* in each break group is assigned to the variable *FIRSTAGE*.
- If the first value of *AGE* in a break group is user missing, that value will be assigned to *FIRSTAGE*. However, the value will retain its missing-value status, since variables created with FIRST take dictionary information from their source variables.

Comparing Missing-Value Treatments

The table below demonstrates the effects of specifying the MISSING subcommand and a period after the function name. Each entry in the table is the number of cases used to compute the specified function for the variable *EDUC*, which has 10 nonmissing cases, 5 user-missing cases, and 2 system-missing cases for the group. Note that columnwise treatment produces the same results as the default for every function except the MEAN function.

Table 6. Default versus columnwise missing-value treatments

Function	Default	Columnwise
N	17	17
N.	17	17
N(EDUC)	10	10
N.(EDUC)	15	15

Table 6. Default versus columnwise missing-value treatments (continued)

Function	Default	Columnwise
MEAN(EDUC)	10	0
MEAN.(EDUC)	15	0
NMISS(EDUC)	7	7
NMISS.(EDUC)	2	2

AIM

AIM is available in Statistics Base Edition.

```
AIM grouping-var

  [/CATEGORICAL varlist]

  [/CONTINUOUS varlist]

  [/CRITERIA [ADJUST = {BONFERRONI**}] [CI = {95** }]
                       {NONE         }        {value}

             [HIDENOTSIG = {NO**}]] [SHOWREFLINE = {NO   }] ]
                           {YES }                  {YES**}

  [/MISSING {EXCLUDE**} ]
            {INCLUDE  }

  [/PLOT [CATEGORY] [CLUSTER [(TYPE = {BAR*})]] [ERRORBAR]
                                      {PIE }

         [IMPORTANCE [([X = {GROUP*  }] [Y = {TEST* }])]] ]
                            {VARIABLE}       {PVALUE}
```

* Default if the keyword is omitted.

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
AIM TSC_1
  /CATEGORICAL type
  /CONTINUOUS price engine_s horsepow wheelbas width length
              curb_wgt fuel_cap mpg
  /PLOT CLUSTER.
```

Overview

AIM provides graphical output to show the relative importance of categorical and scale variables to the formation of clusters of cases as indicated by the grouping variable.

Basic Specification

The basic specification is a grouping variable, a CATEGORICAL or CONTINUOUS subcommand, and a PLOT subcommand.

Subcommand Order

- The grouping variable must be specified first.
- Subcommands can be specified in any order.

Syntax Rules

- All subcommands should be specified only once. If a subcommand is repeated, only the last specification will be used.

Limitations

The WEIGHT variable, if specified, is ignored by this procedure.

Grouping Variable

- The grouping variable must be the first specification after the procedure name.
- The grouping variable can be of any type (numeric or string).

Example

```
AIM clu_id
  /CONTINUOUS age work salary.
```

- This is a typical example where *CLU_ID* is the cluster membership saved from a clustering procedure (say TwoStep Cluster) where *AGE*, *WORK*, and *SALARY* are the variables used to find the clusters.

CATEGORICAL Subcommand

Variables that are specified in this subcommand are treated as categorical variables, regardless of their defined measurement level.

- There is no restriction on the types of variables that can be specified on this subcommand.
- The grouping variable cannot be specified on this subcommand.

CONTINUOUS Subcommand

Variables that are specified in this subcommand are treated as scale variables, regardless of their defined measurement level.

- Variables specified on this subcommand must be numeric.
- The grouping variable cannot be specified on this subcommand.

CRITERIA Subcommand

The CRITERIA subcommand offers the following options in producing graphs.

ADJUST = BONFERRONI | NONE. *Adjust the confidence level for simultaneous confidence intervals or the tolerance level for simultaneous tests.* BONFERRONI uses Bonferroni adjustments. This is the default. NONE specifies that no adjustments should be applied.

CI = number. *Confidence Interval.* This option controls the confidence level. Specify a value greater than 0 and less than 100. The default value is 95.

HIDENOTSIG = NO | YES. *Hide groups or variables that are determined to be not significant.* YES specifies that all confidence intervals and all test results should be shown. This is the default. NO specifies that only the significant confidence intervals and test results should be shown.

SHOWREFLINE = NO | YES. *Display reference lines that are the critical values or the tolerance levels in tests.* YES specifies that the appropriate reference lines should be shown. This is the default. NO specifies that reference lines should not be shown.

MISSING Subcommand

The MISSING subcommand specifies the way to handle cases with user-missing values.

- A case is never used if it contains system-missing values in the grouping variable, categorical variable list, or the continuous variable list.
- If this subcommand is not specified, the default is EXCLUDE.

EXCLUDE. *Exclude both user-missing and system-missing values.* This is the default.

INCLUDE . *User-missing values are treated as valid.* Only system-missing values are not included in the analysis.

PLOT Subcommand

The PLOT subcommand specifies which graphs to produce.

CATEGORY. *Within Cluster Percentages.* This option displays a clustered bar chart for each categorical variable. The bars represent percentages of categories in each cluster. The cluster marginal count is used as the base for the percentages.

CLUSTER (TYPE=BAR | PIE). *Cluster frequency charts.* Displays a bar or pie chart, depending upon the option selected, representing the frequency of each level of the grouping variable.

ERRORBAR. *Error Bar.* This option displays an error bar by group ID for each continuous variable.

IMPORTANCE (X=GROUP | VARIABLE Y=TEST | PVALUE). *Attribute Importance.* This option displays a bar chart that shows the relative importance of the attributes/variables. The specified options further control the display. X = GROUP causes values of the grouping variable to be displayed on the *x* axis. A separate chart is produced for each variable. X = VARIABLE causes variable names to be displayed on the *x* axis. A separate chart is produced for each value of the grouping variable. Y = TEST causes test statistics to be displayed on the *y* axis. Student's *t* statistics are displayed for scale variables, and chi-square statistics are displayed for categorical variables. Y = PVALUE causes *p*-value-related measures to be displayed on the *y* axis. Specifically, $-\log_{10}$(pvalue) is shown so that in both cases larger values indicate "more significant" results.

Example: Importance Charts by Group

```
AIM clu_id
  /CONTINUOUS age work salary
  /CATEGORICAL minority
  /PLOT CATEGORY CLUSTER (TYPE = PIE) IMPORTANCE (X=GROUP Y=TEST).
```

- A frequency pie chart is requested.
- Student's *t* statistics are plotted against the group ID for each scale variable, and chi-square statistics are plotted against the group ID for each categorical variable.

Example: Importance Charts by Variable

```
AIM clu_id
  /CONTINUOUS age work salary
  /CATEGORICAL minority
  /CRITERIA HIDENOTSIG=YES CI=95 ADJUST=NONE
  /PLOT CATEGORY CLUSTER (TYPE = BAR)
            IMPORTANCE (X = VARIABLE, Y = PVALUE).
```

- A frequency bar chart is requested.
- –log10(pvalue) values are plotted against variables, both scale and categorical, for each level of the grouping variable.
- In addition, bars are not shown if their *p* values exceed 0.05.

ALSCAL

ALSCAL is available in the Statistics Base Edition.

```
ALSCAL  VARIABLES=varlist

 [/FILE='savfile'|'dataset']
             [CONFIG  [({INITIAL**})]]  [ROWCONF [({INITIAL**})]]
                       {FIXED    }                {FIXED    }

              [COLCONF [({INITIAL**})]]  [SUBJWGHT[({INITIAL**})]]
                        {FIXED    }                {FIXED    }


              [STIMWGHT[({INITIAL**})]]
                        {FIXED    }

 [/INPUT=ROWS ({ALL**})]
              { n   }

 [/SHAPE={SYMMETRIC**}]
         {ASYMMETRIC }
         {RECTANGULAR}

 [/LEVEL={ORDINAL**  [([UNTIE] [SIMILAR])]}]
         {INTERVAL[({1**})]                 }
         {        {n  }                     }
         {RATIO[({1**})]                    }
         {      {n  }                       }
         {NOMINAL                           }

 [/CONDITION={MATRIX**     }]
             {ROW          }
             {UNCONDITIONAL}

 [/{MODEL }={EUCLID**}]
   {METHOD} {INDSCAL }
            {ASCAL   }
            {AINDS   }
            {GEMSCAL }

 [/CRITERIA=[NEGATIVE] [CUTOFF({0**})] [CONVERGE({.001})]
                               { n }             { n  }

         [ITER({30})] [STRESSMIN({.005})] [NOULB]
               {n }               { n  }

         [DIMENS({2**      })] [DIRECTIONS(n)]
                 {min[,max]}

         [CONSTRAIN]  [TIESTORE(n)]]

 [/PRINT=[DATA] [HEADER]]    [/PLOT=[DEFAULT] [ALL]]

 [/OUTFILE='savfile'|'dataset']

 [/MATRIX=IN({'savfile'|'dataset'})]
            {*                  }
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
ALSCAL VARIABLES=ATLANTA TO TAMPA.
```

ALSCAL was originally designed and programmed by Forrest W. Young, Yoshio Takane, and Rostyslaw J. Lewyckyj of the Psychometric Laboratory, University of North Carolina.

Overview

ALSCAL uses an alternating least-squares algorithm to perform multidimensional scaling (MDS) and multidimensional unfolding (MDU). You can select one of the five models to obtain stimulus coordinates and/or weights in multidimensional space.

Options

Data Input. You can read inline data matrices, including all types of two- or three-way data, such as a single matrix or a matrix for each of several subjects, using the INPUT subcommand. You can read square (symmetrical or asymmetrical) or rectangular matrices of proximities with the SHAPE subcommand and

proximity matrices created by PROXIMITIES and CLUSTER with the MATRIX subcommand. You can also read a file of coordinates and/or weights to provide initial or fixed values for the scaling process with the FILE subcommand.

Methodological Assumptions. You can specify data as matrix-conditional, row-conditional, or unconditional on the CONDITION subcommand. You can treat data as nonmetric (nominal or ordinal) or as metric (interval or ratio) using the LEVEL subcommand. You can also use LEVEL to identify ordinal-level proximity data as measures of similarity or dissimilarity, and you can specify tied observations as untied (continuous) or leave them tied (discrete).

Model Selection. You can specify the most commonly used multidimensional scaling models by selecting the correct combination of ALSCAL subcommands, keywords, and criteria. In addition to the default Euclidean distance model, the MODEL subcommand offers the individual differences (weighted) Euclidean distance model (INDSCAL), the asymmetric Euclidean distance model (ASCAL), the asymmetric individual differences Euclidean distance model (AINDS), and the generalized Euclidean metric individual differences model (GEMSCAL).

Output. You can produce output that includes raw and scaled input data, missing-value patterns, normalized data with means, squared data with additive constants, each subject's scalar product and individual weight space, plots of linear or nonlinear fit, and plots of the data transformations using the PRINT and PLOT subcommands.

Basic Specification

The basic specification is VARIABLES followed by a variable list. By default, ALSCAL produces a two-dimensional nonmetric Euclidean multidimensional scaling solution. Input is assumed to be one or more square symmetric matrices with data elements that are dissimilarities at the ordinal level of measurement. Ties are not untied, and conditionality is by subject. Values less than 0 are treated as missing. The default output includes the improvement in Young's S-stress for successive iterations, two measures of fit for each input matrix (Kruskal's stress and the squared correlation, RSQ), and the derived configurations for each of the dimensions.

Subcommand Order

Subcommands can be named in any order.

Operations

- ALSCAL calculates the number of input matrices by dividing the total number of observations in the dataset by the number of rows in each matrix. All matrices must contain the same number of rows. This number is determined by the settings on SHAPE and INPUT (if used). For square matrix data, the number of rows in the matrix equals the number of variables. For rectangular matrix data, it equals the number of rows specified or implied. For additional information, see the INPUT and SHAPE subcommands below.
- ALSCAL ignores user-missing specifications in all variables in the configuration/weights file. See the topic "FILE Subcommand " on page 139 for more information. The system-missing value is converted to 0.
- With split-file data, ALSCAL reads initial or fixed configurations from the configuration/weights file for each split-file group. See the topic "FILE Subcommand " on page 139 for more information. If there is only one initial configuration in the file, ALSCAL rereads these initial or fixed values for successive split-file groups.
- By default, ALSCAL estimates upper and lower bounds on missing values in the active dataset in order to compute the initial configuration. To prevent this, specify CRITERIA=NOULB. Missing values are always ignored during the iterative process.

Limitations

- A maximum of 100 variables on the VARIABLES subcommand.
- A maximum of six dimensions can be scaled.
- ALSCAL does not recognize data weights created by the WEIGHT command.

- ALSCAL analyses can include no more than 32,767 values in each of the input matrices. Large analyses may require significant computing time.

Example

```
* Air distances among U.S. cities.
* Data are from Johnson and Wichern (1982), page 563.
DATA LIST
 /ATLANTA BOSTON CINCNATI COLUMBUS DALLAS INDNPLIS
   LITTROCK LOSANGEL MEMPHIS STLOUIS SPOKANE TAMPA 1-60.
BEGIN DATA
   0
1068    0
 461  867    0
 549  769  107    0
 805 1819  943 1050    0
 508  941  108  172  882    0
 505 1494  618  725  325  562    0
2197 3052 2186 2245 1403 2080 1701    0
 366 1355  502  586  464  436  137 1831    0
 558 1178  338  409  645  234  353 1848  294    0
2467 2747 2067 2131 1891 1959 1988 1227 2042 1820    0
 467 1379  928  985 1077  975  912 2480  779 1016 2821   0
END DATA.

ALSCAL VARIABLES=ATLANTA TO TAMPA
 /PLOT.
```

- By default, ALSCAL assumes a symmetric matrix of dissimilarities for ordinal-level variables. Only values below the diagonal are used. The upper triangle can be left blank. The 12 cities form the rows and columns of the matrix.
- The result is a classical MDS analysis that reproduces a map of the United States when the output is rotated to a north-south by east-west orientation.

VARIABLES Subcommand

VARIABLES identifies the columns in the proximity matrix or matrices that ALSCAL reads.

- VARIABLES is required and can name only numeric variables.
- Each matrix must have at least four rows and four columns.

INPUT Subcommand

ALSCAL reads data row by row, with each case in the active dataset representing a single row in the data matrix. (VARIABLES specifies the columns.) Use INPUT when reading rectangular data matrices to specify how many rows are in each matrix.

- The specification on INPUT is ROWS. If INPUT is not specified or is specified without ROWS, the default is ROWS(ALL). ALSCAL assumes that each case in the active dataset represents one row of a single input matrix and that the result is a square matrix.
- You can specify the number of rows (*n*) in each matrix in parentheses after the keyword ROWS. The number of matrices equals the number of observations divided by the number specified.
- The number specified on ROWS must be at least 4 and must divide evenly into the total number of rows in the data.
- With split-file data, *n* refers to the number of cases in each split-file group. All split-file groups must have the same number of rows.

Example

```
ALSCAL VARIABLES=V1 to V7 /INPUT=ROWS(8).
```

- INPUT indicates that there are eight rows per matrix, with each case in the active dataset representing one row.
- The total number of cases must be divisible by 8.

SHAPE Subcommand

Use SHAPE to specify the structure of the input data matrix or matrices.

- You can specify one of the three keywords listed below.
- Both SYMMETRIC and ASYMMETRIC refer to square matrix data.

SYMMETRIC. *Symmetric data matrix or matrices.* For a symmetric matrix, ALSCAL looks only at the values below the diagonal. Values on and above the diagonal can be omitted. This is the default.

ASYMMETRIC. *Asymmetric data matrix or matrices.* The corresponding values in the upper and lower triangles are not all equal. The diagonal is ignored.

RECTANGULAR. *Rectangular data matrix or matrices.* The rows and columns represent different sets of items.

Example

```
ALSCAL VAR=V1 TO V8 /SHAPE=RECTANGULAR.
```

- ALSCAL performs a classical MDU analysis, treating the rows and columns as separate sets of items.

LEVEL Subcommand

LEVEL identifies the level of measurement for the values in the data matrix or matrices. You can specify one of the keywords defined below.

ORDINAL. *Ordinal-level data.* This specification is the default. It treats the data as ordinal, using Kruskal's least-squares monotonic transformation [2]. The analysis is nonmetric. By default, the data are treated as discrete dissimilarities. Ties in the data remain tied throughout the analysis. To change the default, specify UNTIE and/or SIMILAR in parentheses. UNTIE treats the data as continuous and resolves ties in an optimal fashion; SIMILAR treats the data as similarities. UNTIE and SIMILAR cannot be used with the other levels of measurement.

INTERVAL(n). *Interval-level data.* This specification produces a metric analysis of the data using classical regression techniques. You can specify any integer from 1 to 4 in parentheses for the degree of polynomial transformation to be fit to the data. The default is 1.

RATIO(n). *Ratio-level data.* This specification produces a metric analysis. You can specify an integer from 1 to 4 in parentheses for the degree of polynomial transformation. The default is 1.

NOMINAL. *Nominal-level data.* This specification treats the data as nominal by using a least-squares categorical transformation [3]. This option produces a nonmetric analysis of nominal data. It is useful when there are few observed categories, when there are many observations in each category, and when the order of the categories is not known.

Example

```
ALSCAL VAR=ATLANTA TO TAMPA /LEVEL=INTERVAL(2).
```

- This example identifies the distances between U.S. cities as interval-level data. The 2 in parentheses indicates a polynomial transformation with linear and quadratic terms.

[2] Kruskal, J. B. 1964. Nonmetric multidimensional scaling: A numerical method. *Psychometrika,* 29, 115-129.

[3] Takane, Y., F. W. Young, and J. de Leeuw. 1977. Nonmetric individual differences multidimensional scaling: An alternating least squares method with optimal scaling features. *Psychometrika,* 42, 7-67.

CONDITION Subcommand

CONDITION specifies which numbers in a dataset are comparable.

MATRIX. *Only numbers within each matrix are comparable.* If each matrix represents a different subject, this specification makes comparisons conditional by subject. This is the default.

ROW. *Only numbers within the same row are comparable.* This specification is appropriate only for asymmetric or rectangular data. They cannot be used when ASCAL or AINDS is specified on MODEL.

UNCONDITIONAL. *All numbers are comparable.* Comparisons can be made among any values in the input matrix or matrices.

Example

```
ALSCAL VAR=V1 TO V8 /SHAPE=RECTANGULAR /CONDITION=ROW.
```

- ALSCAL performs a Euclidean MDU analysis conditional on comparisons within rows.

FILE Subcommand

ALSCAL can read proximity data from the active dataset or, with the MATRIX subcommand, from a matrix data file created by PROXIMITIES or CLUSTER. The FILE subcommand reads a file containing additional data--an initial or fixed configuration for the coordinates of the stimuli and/or weights for the matrices being scaled. This file can be created with the OUTFILE subcommand on ALSCAL or with an input program (created with the INPUT PROGRAM command).

- The minimum specification is the file that contains the configurations and/or weights.
- FILE can include additional specifications that define the structure of the configuration/weights file.
- The variables in the configuration/weights file that correspond to successive ALSCAL dimensions must have the names *DIM1*, *DIM2*, ..., *DIMr*, where *r* is the maximum number of ALSCAL dimensions. The file must also contain the short string variable *TYPE_* to identify the types of values in all rows.
- Values for the variable *TYPE_* can be CONFIG, ROWCONF, COLCONF, SUBJWGHT, and STIMWGHT, in that order. Each value can be truncated to the first three letters. Stimulus coordinate values are specified as CONFIG; row stimulus coordinates, as ROWCONF; column stimulus coordinates, as COLCONF; and subject and stimulus weights, as SUBJWGHT and STIMWGHT, respectively. ALSCAL accepts CONFIG and ROWCONF interchangeably.
- ALSCAL skips unneeded types as long as they appear in the file in their proper order. Generalized weights (GEM) and flattened subject weights (FLA) cannot be initialized or fixed and will always be skipped. (These weights can be generated by ALSCAL but cannot be used as input.)

The following list summarizes the optional specifications that can be used on FILE to define the structure of the configuration/weights file:

- Each specification can be further identified with the option INITIAL or FIXED in parentheses.
- INITIAL is the default. INITIAL indicates that the external configuration or weights are to be used as initial coordinates and are to be modified during each iteration.
- FIXED forces ALSCAL to use the externally defined structure without modification to calculate the best values for all unfixed portions of the structure.

CONFIG. *Read stimulus configuration.* The configuration/weights file contains initial stimulus coordinates. Input of this type is appropriate when SHAPE=SYMMETRIC or SHAPE= ASYMMETRIC, or when the number of variables in a matrix equals the number of variables on the ALSCAL command. The value of the *TYPE_* variable must be either CON or ROW for all stimulus coordinates for the configuration.

ROWCONF. *Read row stimulus configuration.* The configuration/weights file contains initial row stimulus coordinates. This specification is appropriate if SHAPE= RECTANGULAR and if the number of ROWCONF rows in the matrix equals the number of rows specified on the INPUT subcommand (or, if INPUT is

omitted, the number of cases in the active dataset). The value of *TYPE_* must be either ROW or CON for the set of coordinates for each row.

COLCONF. *Read column stimulus configuration.* The configuration/weights file contains initial column stimulus coordinates. This kind of file can be used only if SHAPE= RECTANGULAR and if the number of COLCONF rows in the matrix equals the number of variables on the ALSCAL command. The value of *TYPE_* must be COL for the set of coordinates for each column.

SUBJWGHT. *Read subject (matrix) weights.* The configuration/weights file contains subject weights. The number of observations in a subject-weights matrix must equal the number of matrices in the proximity file. Subject weights can be used only if the model is INDSCAL, AINDS, or GEMSCAL. The value of *TYPE_* for each set of weights must be SUB.

STIMWGHT. *Read stimulus weights.* The configuration/weights file contains stimulus weights. The number of observations in the configuration/weights file must equal the number of matrices in the proximity file. Stimulus weights can be used only if the model is AINDS or ASCAL. The value of *TYPE_* for each set of weights must be STI.

If the optional specifications for the configuration/weights file are not specified on FILE, ALSCAL sequentially reads the *TYPE_* values appropriate to the model and shape according to the defaults in the table below.

Example

```
ALSCAL VAR=V1 TO V8 /FILE=ONE CON(FIXED) STI(INITIAL).
```

- ALSCAL reads the configuration/weights file *ONE*.
- The stimulus coordinates are read as fixed values, and the stimulus weights are read as initial values.

Table 7. Default specifications for the FILE subcommand

Shape	Model	Default specifications
SYMMETRIC	EUCLID	CONFIG (or ROWCONF)
SYMMETRIC	INDSCAL	CONFIG (or ROWCONF), SUBJWGHT
SYMMETRIC	GEMSCAL	CONFIG (or ROWCONF), SUBJWGHT
ASYMMETRIC	EUCLID	CONFIG (or ROWCONF)
ASYMMETRIC	INDSCAL	CONFIG (or ROWCONF), SUBJWGHT
ASYMMETRIC	GEMSCAL	CONFIG (or ROWCONF), SUBJWGHT
ASYMMETRIC	ASCAL	CONFIG (or ROWCONF), STIMWGHT
ASYMMETRIC	AINDS	CONFIG (or ROWCONF), SUBJWGHT, STIMWGHT
RECTANGULAR	EUCLID	ROWCONF (or CONFIG), COLCONF
RECTANGULAR	INDSCAL	ROWCONF (or CONFIG, COLCONF, SUBJWGHT
RECTANGULAR	GEMSCAL	ROWCONF (or CONFIG, COLCONF, SUBJWGHT

MODEL Subcommand

MODEL (alias METHOD) defines the scaling model for the analysis. The only specification is MODEL (or METHOD) and any one of the five scaling and unfolding model types. EUCLID is the default.

EUCLID. *Euclidean distance model.* This model can be used with any type of proximity matrix and is the default.

INDSCAL. *Individual differences (weighted) Euclidean distance model.* ALSCAL scales the data using the weighted individual differences Euclidean distance model [4]. This type of analysis can be specified only if the analysis involves more than one data matrix and more than one dimension is specified on CRITERIA.

ASCAL. *Asymmetric Euclidean distance model.* This model [5] can be used only if SHAPE=ASYMMETRIC and more than one dimension is requested on CRITERIA.

AINDS. *Asymmetric individual differences Euclidean distance model.* This option combines Young's asymmetric Euclidean model [6] with the individual differences model [7]. This model can be used only when SHAPE=ASYMMETRIC, the analysis involves more than one data matrix, and more than one dimension is specified on CRITERIA.

GEMSCAL. *Generalized Euclidean metric individual differences model.* The number of directions for this model is set with the DIRECTIONS option on CRITERIA. The number of directions specified can be equal to but cannot exceed the group space dimensionality. By default, the number of directions equals the number of dimensions in the solution.

Example

```
ALSCAL VARIABLES = V1 TO V6
 /SHAPE = ASYMMETRIC
 /CONDITION = ROW
 /MODEL = GEMSCAL
 /CRITERIA = DIM(4) DIRECTIONS(4).
```

- In this example, the number of directions in the GEMSCAL model is set to 4.

CRITERIA Subcommand

Use CRITERIA to control features of the scaling model and to set convergence criteria for the solution. You can specify one or more of the following:

CONVERGE(n). *Stop iterations if the change in S-stress is less than* n. S-stress is a goodness-of-fit index. By default, *n*=0.001. To increase the precision of a solution, specify a smaller value, for example, 0.0001. To obtain a less precise solution (perhaps to reduce computing time), specify a larger value, for example, 0.05. Negative values are not allowed. If *n*=0, the algorithm will iterate 30 times unless a value is specified with the ITER option.

ITER(n). *Set the maximum number of iterations to* n. The default value is 30. A higher value will give a more precise solution but will take longer to compute.

STRESSMIN(n). *Set the minimum stress value to* n. By default, ALSCAL stops iterating when the value of S-stress is 0.005 or less. STRESSMIN can be assigned any value from 0 to 1.

NEGATIVE. *Allow negative weights in individual differences models.* By default, ALSCAL does not permit the weights to be negative. Weighted models include INDSCAL, ASCAL, AINDS, and GEMSCAL. The NEGATIVE option is ignored if the model is EUCLID.

CUTOFF(n). *Set the cutoff value for treating distances as missing to* n. By default, ALSCAL treats all negative similarities (or dissimilarities) as missing and 0 and positive similarities as nonmissing (*n*=0). Changing the CUTOFF value causes ALSCAL to treat similarities greater than or equal to that value as nonmissing. User- and system-missing values are considered missing regardless of the CUTOFF specification.

---

[4] Carroll, J. D., and J. J. Chang. 1970. Analysis of individual differences in multidimensional scaling via an n-way generalization of "Eckart-Young" decomposition. *Psychometrika,* 35, 238-319.

[5] Young, F. W. 1975. An asymmetric Euclidean model for multiprocess asymmetric data. In: *Proceedings of U.S.–Japan Seminar on Multidimensional Scaling*. San Diego: .

[6] Young, F. W. 1975. An asymmetric Euclidean model for multiprocess asymmetric data. In: *Proceedings of U.S.–Japan Seminar on Multidimensional Scaling*. San Diego: .

[7] Carroll, J. D., and J. J. Chang. 1970. Analysis of individual differences in multidimensional scaling via an n-way generalization of "Eckart-Young" decomposition. *Psychometrika,* 35, 238-319.

NOULB. *Do not estimate upper and lower bounds on missing values.* By default, ALSCAL estimates the upper and lower bounds on missing values in order to compute the initial configuration. This specification has no effect during the iterative process, when missing values are ignored.

DIMENS(min[,max]). *Set the minimum and maximum number of dimensions in the scaling solution.* By default, ALSCAL calculates a solution with two dimensions. To obtain solutions for more than two dimensions, specify the minimum and the maximum number of dimensions in parentheses after DIMENS. The minimum and maximum can be integers between 2 and 6. A single value represents both the minimum and the maximum. For example, DIMENS(3) is equivalent to DIMENS(3,3). The minimum number of dimensions can be set to 1 only if MODEL=EUCLID.

DIRECTIONS(n). *Set the number of principal directions in the generalized Euclidean model to* n. This option has no effect for models other than GEMSCAL. The number of principal directions can be any positive integer between 1 and the number of dimensions specified on the DIMENS option. By default, the number of directions equals the number of dimensions.

TIESTORE(n). *Set the amount of storage needed for ties to* n. This option estimates the amount of storage needed to deal with ties in ordinal data. By default, the amount of storage is set to 1000 or the number of cells in a matrix, whichever is smaller. Should this be insufficient, ALSCAL terminates and displays a message that more space is needed.

CONSTRAIN. *Constrain multidimensional unfolding solution.* This option can be used to keep the initial constraints throughout the analysis.

PRINT Subcommand

PRINT requests output not available by default. You can specify the following:

DATA. *Display input data.* The display includes both the initial data and the scaled data for each subject according to the structure specified on SHAPE.

HEADER. *Display a header page.* The header includes the model, output, algorithmic, and data options in effect for the analysis.

- Data options listed by PRINT=HEADER include the number of rows and columns, number of matrices, measurement level, shape of the data matrix, type of data (similarity or dissimilarity), whether ties are tied or untied, conditionality, and data cutoff value.
- Model options listed by PRINT=HEADER are the type of model specified (EUCLID, INDSCAL, ASCAL, AINDS, or GEMSCAL), minimum and maximum dimensionality, and whether or not negative weights are permitted.
- Output options listed by PRINT=HEADER indicate whether the output includes the header page and input data, whether ALSCAL plotted configurations and transformations, whether an output dataset was created, and whether initial stimulus coordinates, initial column stimulus coordinates, initial subject weights, and initial stimulus weights were computed.
- Algorithmic options listed by PRINT=HEADER include the maximum number of iterations permitted, the convergence criterion, the maximum S-stress value, whether or not missing data are estimated by upper and lower bounds, and the amount of storage allotted for ties in ordinal data.

Example

```
ALSCAL VAR=ATLANTA TO TAMPA /PRINT=DATA.
```

- In addition to scaled data, ALSCAL will display initial data.

PLOT Subcommand

PLOT controls the display of plots. The minimum specification is simply PLOT to produce the defaults.

DEFAULT. *Default plots.* Default plots include plots of stimulus coordinates, matrix weights (if the model is INDSCAL, AINDS, or GEMSCAL), and stimulus weights (if the model is AINDS or ASCAL). The default

also includes a scatterplot of the linear fit between the data and the model and, for certain types of data, scatterplots of the nonlinear fit and the data transformation.

ALL. *Transformation plots in addition to the default plots.* A separate plot is produced for each subject if CONDITION=MATRIX and a separate plot for each row if CONDITION=ROW. For interval and ratio data, PLOT=ALL has the same effect as PLOT=DEFAULT. This option can generate voluminous output, particularly when CONDITION=ROW.

Example

```
ALSCAL VAR=V1 TO V8 /INPUT=ROWS(8) /PLOT=ALL.
```

- This command produces all of the default plots. It also produces a separate plot for each subject's data transformation and a plot of *V1* through *V8* in a two-dimensional space for each subject.

OUTFILE Subcommand

OUTFILE saves coordinate and weight matrices to a data file in IBM SPSS Statistics format. The only specification is a name for the output file.

- The output data file has an alphanumeric (short string) variable named *TYPE_* that identifies the kind of values in each row, a numeric variable named *DIMENS* that specifies the number of dimensions, a numeric variable named *MATNUM* that indicates the subject (matrix) to which each set of coordinates corresponds, and variables named *DIM1*, *DIM2*, ..., *DIMn* that correspond to the *n* dimensions in the model.
- The values of any split-file variables are also included in the output file.
- The file created by OUTFILE can be used by subsequent ALSCAL commands as initial data.

The following are the types of configurations and weights that can be included in the output file:

CONFIG. *Stimulus configuration coordinates*.

ROWCONF. *Row stimulus configuration coordinates*.

COLCONF. *Column stimulus configuration coordinates*.

SUBJWGHT. *Subject (matrix) weights*.

FLATWGHT. *Flattened subject (matrix) weights*.

GEMWGHT. *Generalized weights*.

STIMWGHT. *Stimulus weights*.

Only the first three characters of each identifier are written to the variable *TYPE_* in the file. For example, CONFIG becomes CON. The structure of the file is determined by the SHAPE and MODEL subcommands, as shown in the following table.

Table 8. Types of configurations and/or weights in output files

Shape	Model	TYPE_
SYMMETRIC	EUCLID	CON
SYMMETRIC	INDSCAL	CON, SUB, FLA
SYMMETRIC	GEMSCAL	CON, SUB, FLA, GEM
ASYMMETRIC	EUCLID	CON
ASYMMETRIC	INDSCAL	CON, SUB, FLA
ASYMMETRIC	GEMSCAL	CON, SUB, FLA, GEM
ASYMMETRIC	ASCAL	CON, STI

Table 8. Types of configurations and/or weights in output files (continued)		
Shape	**Model**	**TYPE_**
ASYMMETRIC	AINDS	CON, SUB, FLA, STI
RECTANGULAR	EUCLID	ROW, COL
RECTANGULAR	INDSCAL	ROW, COL, SUB, FLA
RECTANGULAR	GEMSCAL	ROW, COL, SUB, FLA, GEM

Example

```
ALSCAL VAR=ATLANTA TO TAMPA /OUTFILE=ONE.
```

- OUTFILE creates the configuration/weights file *ONE* from the example of air distances between cities.

MATRIX Subcommand

MATRIX reads matrix data files. It can read a matrix written by either PROXIMITIES or CLUSTER.

- Generally, data read by ALSCAL are already in matrix form. If the matrix materials are in the active dataset, you do not need to use MATRIX to read them. Simply use the VARIABLES subcommand to indicate the variables (or columns) to be used. However, if the matrix materials are not in the active dataset, MATRIX must be used to specify the matrix data file that contains the matrix.
- The proximity matrices that ALSCAL reads have *ROWTYPE_* values of PROX. No additional statistics should be included with these matrix materials.
- ALSCAL ignores unrecognized *ROWTYPE_* values in the matrix file. In addition, it ignores variables present in the matrix file that are not specified on the VARIABLES subcommand in ALSCAL. The order of rows and columns in the matrix is unimportant.
- Since ALSCAL does not support case labeling, it ignores values for the *ID* variable (if present) in a CLUSTER or PROXIMITIES matrix.
- If split-file processing was in effect when the matrix was written, the same split file must be in effect when ALSCAL reads that matrix.
- The specification on MATRIX is the keyword IN and the matrix file in parentheses.
- MATRIX=IN cannot be used unless a active dataset has already been defined. To read an existing matrix data file at the beginning of a session, first use GET to retrieve the matrix file and then specify IN(*) on MATRIX.

IN (filename) . *Read a matrix data file.* If the matrix data file is the active dataset, specify an asterisk in parentheses (*). If the matrix data file is another file, specify the filename in parentheses. A matrix file read from an external file does not replace the active dataset.

Example

```
PROXIMITIES V1 TO V8 /ID=NAMEVAR /MATRIX=OUT(*).
ALSCAL VAR=CASE1 TO CASE10 /MATRIX=IN(*).
```

- PROXIMITIES uses *V1* through *V8* in the active dataset to generate a matrix file of Euclidean distances between each pair of cases based on the eight variables. The number of rows and columns in the resulting matrix equals the number of cases. MATRIX=OUT then replaces the active dataset with this new matrix data file.
- MATRIX=IN on ALSCAL reads the matrix data file, which is the new active dataset. In this instance, MATRIX is optional because the matrix materials are in the active dataset.
- If there were 10 cases in the original active dataset, ALSCAL performs a multidimensional scaling analysis in two dimensions on *CASE1* through *CASE10*.

Example

```
GET FILE PROXMTX.
ALSCAL VAR=CASE1 TO CASE10 /MATRIX=IN(*).
```

- GET retrieves the matrix data file *PROXMTX*.
- MATRIX=IN specifies an asterisk because the active dataset is the matrix. MATRIX is optional, however, since the matrix materials are in the active dataset.

Example

```
GET FILE PRSNNL.
FREQUENCIES VARIABLE=AGE.
ALSCAL VAR=CASE1 TO CASE10 /MATRIX=IN(PROXMTX).
```

- This example performs a frequencies analysis on the file *PRSNNL* and then uses a different file containing matrix data for ALSCAL. The file is an existing matrix data file.
- MATRIX=IN is required because the matrix data file, *PROXMTX*, is not the active dataset. *PROXMTX* does not replace *PRSNNL* as the active dataset.

Specification of Analyses

The following tables summarize the analyses that can be performed for the major types of proximity matrices that you can use with ALSCAL, list the specifications needed to produce these analyses for nonmetric models, and list the specifications for metric models. You can include additional specifications to control the precision of your analysis with CRITERIA.

Table 9. Models for types of matrix input

Matrix mode	Matrix form	Model class	Single matrix	Replications of single matrix	Two or more individual matrices
Object by object	Symmetric	Multidimensional scaling	CMDS Classical multidimensional scaling	RMDS Replicated multidimensional scaling	WMDS (INDSCAL) Weighted multidimensional scaling
Object by object	Asymmetric single process	Multidimensional scaling	CMDS (row conditional) Classical row conditional multidimensional scaling	RMDS (row conditional) Replicated row conditional multi dimensional scaling	WMDS (row conditional) Weighted row conditional multidimensional scaling
Object by object	Asymmetric multiple process	Internal asymmetric multidimensional scaling	CAMDS Classical asymmetric multidimensional scaling	RAMDS Replicated asymmetric multidimensional scaling	WAMDS Weighted asymmetric multidimensional scaling
Object by object	Asymmetric multiple process	External asymmetric multidimensional scaling	CAMDS (external) Classical external asymmetric multidimensional scaling	RAMDS (external) Replicated external asymmetric multidimensional scaling	WAMDS (external) Weighted external asymmetric multidimensional scaling

Table 9. Models for types of matrix input (continued)

Matrix mode	Matrix form	Model class	Single matrix	Replications of single matrix	Two or more individual matrices
Object by attribute	Rectangular	Internal unfolding	CMDU Classical internal multidimensional unfolding	RMDU Replicated internal multidimensional unfolding	WMDU Weighted internal multidimensional unfolding
Object by attribute	Rectangular	External unfolding	CMDU (external) Classical external multidimensional unfolding	RMDU (external) Replicated external multidimensional unfolding	WMDU (external) Weighted external multidimensional unfolding

Table 10. ALSCAL specifications for nonmetric models

Matrix mode	Matrix form	Model class	Single matrix	Replications of single matrix	Two or more individual matrices
Object by object	Symmetric	Multidimensional scaling	`ALSCAL VAR=varlist.`	`ALSCAL VAR= varlist.`	`ALSCAL VAR= varlist /MODEL=INDSCAL.`
Object by object	Asymmetric single process	Multidimensional scaling	`ALSCAL VAR= varlist /SHAPE=ASYMMETRIC /CONDITION=ROW.`	`ALSCAL VAR= varlist /SHAPE=ASYMMETRIC /CONDITION=ROW.`	`ALSCAL VAR= varlist /SHAPE=ASYMMETRIC /CONDITION=ROW /MODEL=INDSCAL.`
Object by object	Asymmetric multiple process	Internal asymmetric multidimensional scaling	`ALSCAL VAR= varlist /SHAPE=ASYMMETRIC /MODEL=ASCAL.`	`ALSCAL VAR= varlist /SHAPE=ASYMMETRIC /MODEL=ASCAL.`	`ALSCAL VAR= varlist /SHAPE=ASYMMETRIC /MODEL=AINDS.`
Object by object	Asymmetric multiple process	External asymmetric multidimensional scaling	`ALSCAL VAR= varlist /SHAPE=ASYMMETRIC /MODEL=ASCAL /FILE=file COLCONF(FIX).`	`ALSCAL VAR= varlist /SHAPE=ASYMMETRIC /MODEL=ASCAL /FILE=file COLCONF(FIX).`	`ALSCAL VAR=varlist /SHAPE=ASYMMETRIC /MODEL=AINDS /FILE=file COLCONF(FIX).`
Object by attribute	Rectangular	Internal unfolding	`ALSCAL VAR=varlist /SHAPE=REC /INP=ROWS /CONDITION=ROW.`	`ALSCAL VAR=varlist /SHAPE=REC /INP=ROWS /CONDITION(ROW).`	`ALSCAL VAR=varlist /SHAPE=REC /INP=ROWS /CONDITION=ROW /MODEL=INDSCAL.`
Object by attribute	Rectangular	External unfolding	`ALSCAL VAR=varlist /SHAPE=REC /INP=ROWS /CONDITION=ROW /FILE=file ROWCONF(FIX).`	`ALSCAL VAR= varlist /SHAPE=REC /INP=ROWS /CONDITION=ROW /FILE=file ROWCONF(FIX).`	`ALSCAL VAR=varlist /SHAPE=REC /INP=ROWS /CONDITION=ROW /FILE=file ROWCONF(FIX) /MODEL=INDSCAL.`

Table 11. ALSCAL specifications for metric models

Matrix mode	Matrix form	Model class	Single matrix	Replications of single matrix	Two or more individual matrices
Object by object	Symmetric	Multidimensional scaling	`ALSCAL VAR=varlist /LEVEL=INT.`	`ALSCAL VAR=varlist /LEVEL=INT.`	`ALSCAL VAR=varlist /LEVEL=INT /MODEL=INDSCAL.`
Object by object	Asymmetric single process	Multidimensional scaling	`ALSCAL VAR=varlist /SHAPE=ASYMMETRIC /CONDITION=ROW /LEVEL=INT.`	`ALSCAL VAR=varlist /SHAPE=ASYMMETRIC /CONDITION=ROW /LEVEL=INT.`	`ALSCAL VAR=varlist /SHAPE=ASYMMETRIC /CONDITION=ROW /LEVEL=INT /MODEL=INDSCAL.`
Object by object	Asymmetric multiple process	Internal asymmetric multidimensional scaling	`ALSCAL VAR=varlist /SHAPE=ASYMMETRIC /LEVEL=INT /MODEL=ASCAL.`	`ALSCAL VAR=varlist /SHAPE=ASYMMETRIC /LEVEL=INT /MODEL=ASCAL.`	`ALSCAL VAR=varlist /SHAPE=ASYMMETRIC /LEVEL=INT /MODEL=AINDS.`
Object by object	Asymmetric multiple process	External asymmetric multidimensional scaling	`ALSCAL VAR= varlist /SHAPE=ASYMMETRIC /LEVEL=INT /MODEL=ASCAL /FILE=file COLCONF(FIX).`	`ALSCAL VAR= varlist /SHAPE=ASYMMETRIC /LEVEL=INT /MODEL=ASCAL /FILE=file COLCONF(FIX).`	`ALSCAL VAR=varlist /SHAPE=ASYMMETRIC /LEVEL=INT /MODEL=AINDS /FILE=file COLCONF(FIX).`
Object by attribute	Rectangular	Internal unfolding	`ALSCAL VAR=varlist /SHAPE=REC /INP=ROWS /CONDITION=ROW /LEVEL=INT.`	`ALSCAL VAR=varlist /SHAPE=REC /INP=ROWS /CONDITION=ROW /LEVEL=INT.`	`ALSCAL VAR=varlist /SHAPE=REC /INP=ROWS /CONDITION=ROW /LEVEL=INT /MODEL=INDSCAL.`
Object by attribute	Rectangular	External unfolding	`ALSCAL VAR=varlist /SHAPE=REC /INP=ROWS /CONDITION=ROW /LEVEL=INT /FILE=file ROWCONF(FIX).`	`ALSCAL VAR=varlist /SHAPE=REC /INP=ROWS /CONDITION=ROW /LEVEL=INT /FILE=file ROWCONF(FIX).`	`ALSCAL VAR=varlist /SHAPE=REC /INP=ROWS /CONDITION=ROW /LEVEL=INT /FILE=file ROWCONF(FIX) /MODEL=INDSCAL.`

References

Carroll, J. D., and J. J. Chang. 1970. Analysis of individual differences in multidimensional scaling via an n-way generalization of “Eckart-Young” decomposition. *Psychometrika,* 35, 238-319.

Johnson, R., and D. W. Wichern. 1982. *Applied multivariate statistical analysis*. Englewood Cliffs, N.J.: Prentice-Hall.

Kruskal, J. B. 1964. Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis. *Psychometrika,* 29, 1-28.

Kruskal, J. B. 1964. Nonmetric multidimensional scaling: A numerical method. *Psychometrika,* 29, 115-129.

Takane, Y., F. W. Young, and J. de Leeuw. 1977. Nonmetric individual differences multidimensional scaling: An alternating least squares method with optimal scaling features. *Psychometrika,* 42, 7-67.

Young, F. W. 1975. An asymmetric Euclidean model for multiprocess asymmetric data. In: *Proceedings of U.S.–Japan Seminar on Multidimensional Scaling*. San Diego: .

ALTER TYPE

```
ALTER TYPE varlist([input format = ] {output format    }) [varlist...]
                                     {AMIN [+ [n[%]]   }
                                     {AHEXMIN [+ [n[%]]}
  [/PRINT {[ALTEREDTYPES**] [ALTEREDVALUES]}]
          {NONE                            }
```

** Default if subcommand omitted.

Release History

Release 16.0

- Command introduced.

Example

```
ALTER TYPE StringDate1 to StringDate4 (Date11).
ALTER TYPE ALL (A=AMIN).
```

Overview

ALTER TYPE can be used to change the fundamental type (string or numeric) or format of variables, including changing the defined width of string variables.

Options

- You can use the TO keyword to specify a list of variables or the ALL keyword to specify all variables in the active dataset.
- The optional input format specification restricts the type modification to only variables in the list that match the input format. If the input format doesn't include a width specification, all variables that match the basic format are included. An input format specification without a width specification includes all variables that match the basic format, regardless of defined width.
- AMIN or AHEXMIN can be used as the output format specification to change the defined width of a string variable to the minimum width necessary to display all observed values of that variable without truncation.
- AMIN + n or AHEXMIN + n sets the width of string variables to the minimum necessary width plus *n* bytes.
- AMIN + n% or AHEXMIN + n% sets the width of string variables to the minimum necessary width plus *n* percent of that width. The result is rounded to an integer.

Basic Specification

The basic specification is the name of a variable in the active dataset followed by an output format specification enclosed in parentheses, as in:

```
ALTER TYPE StringVar (A4).
```

Syntax Rules

- All variables specified or implied in the variable list(s) must exist in the active dataset.
- Each variable list must be followed by a format specification enclosed in parentheses.
- Format specifications must be valid IBM SPSS Statistics formats. For information on valid format specifications, see “Variable Types and Formats” on page 46.
- If specified, the optional input format must be followed by an equals sign and then the output format.

- If a variable is included in more than one variable list on the same ALTER TYPE command, only the format specification associated with the last instance of the variable name will be applied. (If you want to "chain" multiple modifications for the same variable, use multiple ALTER TYPE commands.)

Operations

- If there are no pending transformations and the command does not include any AMIN or AHEXMIN format specifications and does not include ALTEREDVALUES on the PRINT subcommand, the command takes effect immediately. It does not read the active dataset or execute pending transformations.
- If there are pending transformations or the command includes one or more AMIN or AHEXMIN format specifications or includes ALTEREDVALUES on the PRINT subcommand, the command reads the active dataset and causes execution of any pending transformations.
- Converting a numeric variable to string will result in truncated values if the numeric value cannot be represented in the specified string width.
- Converting a string variable to numeric will result in a system-missing value if the string contains characters that would be invalid for the specified numeric format.

Examples

```
DATA LIST FREE
  /Numvar1 (F2) Numvar2 (F1)
  StringVar1 (A20) StringVar2 (A30)
  StringDate1 (A11) StringDate2 (A10) StringDate3 (A10).
BEGIN DATA
1 23 a234 b2345 28-Oct-2007 10/28/2007 10/29/2008
END DATA.
ALTER TYPE Numvar1 (F5.2) Numvar2 (F3).
ALTER TYPE
   StringDate1 to StringDate3 (A11 = DATE11).
ALTER TYPE
   StringDate1 to StringDate3 (A10 = ADATE10).
ALTER TYPE ALL (A=AMIN).
```

- The first ALTER TYPE command changes the formats of *Numvar1* and *Numvar2* from F2 and F1 to F5.2 and F3.
- The next ALTER TYPE command converts all string variables between *StringDate1* and *StringDate3* (in file order) with a defined string width of 11 to the numeric date format DATE11 (dd-mmm-yyyy). The only variable that meets these criteria is *StringDate1*; so that is the only variable converted.
- The third ALTER TYPE command converts all string variables between *StringDate1* and *StringDate3* with a defined string width of 10 to the numeric date format ADATE11 (mm/dd/yyyy). In this example, this conversion is applied to *StringDate2* and *StringDate3*.
- The last ALTER TYPE command changes the defined width of all remaining string variables to the minimum width necessary for each variable to avoid truncation of any values. In this example, *StringVar1* changes from A20 to A4 and *StringVar2* changes from A30 to A5. This command reads the data and executes any pending transformation commands.

PRINT Subcommand

The optional PRINT subcommand controls the display of information about the variables modified by the ALTER TYPE command. The following options are available:

ALTEREDTYPES. *Display a list of variables for which the formats were changed and the old and new formats.* This is the default.

ALTEREDVALUES. *Display a report of values that were changed if the fundamental type (string or numeric) was changed or the defined string width was changed.* This report is limited to the first 25 values that were changed for each variable.

NONE. *Don't display any summary information.* This is an alternative to ALTEREDTYPES and/or ALTEREDVALUES and cannot be used in combination with them.

ANACOR

ANACOR is available in SPSS Statistics Professional Edition or the Categories option.

```
ANACOR  TABLE={row var (min, max) BY column var (min, max)}
              {ALL (# of rows, # of columns)              }

[/DIMENSION={2**  }]
            {value}

[/NORMALIZATION={CANONICAL**}]
                {PRINCIPAL  }
                {RPRINCIPAL }
                {CPRINCIPAL }
                {value      }

[/VARIANCES=[SINGULAR] [ROWS] [COLUMNS]]

[/PRINT=[TABLE**] [PROFILES] [SCORES**] [CONTRIBUTIONS**]
        [DEFAULT] [PERMUTATION] [NONE]]

[/PLOT=[NDIM=({1, 2**      })]
              {value, value}
              {ALL, MAX    }
       [ROWS**[(n)]][COLUMNS**[(n)]][DEFAULT[(n)]]
       [TRROWS] [TRCOLUMNS] [JOINT[(n)]] [NONE]]

[/MATRIX OUT=[SCORE({*                  })] [VARIANCE({*                   })]]
                    {'savfile'|'dataset'}             {'savfile'|'dataset'}
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
ANACOR TABLE=MENTAL(1,4) BY SES(1,6).
```

Overview

ANACOR performs correspondence analysis, which is an isotropic graphical representation of the relationships between the rows and columns of a two-way table.

Options

Number of Dimensions. You can specify how many dimensions ANACOR should compute.

Method of Normalization. You can specify one of five different methods for normalizing the row and column scores.

Computation of Variances and Correlations. You can request computation of variances and correlations for singular values, row scores, or column scores.

Data Input. You can analyze the usual individual casewise data or aggregated data from table cells.

Display Output. You can control which statistics are displayed and plotted. You can also control how many value-label characters are used on the plots.

Writing Matrices. You can write matrix data files containing row and column scores and variances for use in further analyses.

Basic Specification

- The basic specification is ANACOR and the TABLE subcommand. By default, ANACOR computes a two-dimensional solution, displays the TABLE, SCORES, and CONTRIBUTIONS statistics, and plots the row scores and column scores of the first two dimensions.

Subcommand Order

- Subcommands can appear in any order.

Operations

- If a subcommand is specified more than once, only the last occurrence is executed.

Limitations

- If the data within table cells contains negative values. ANACOR treats those values as 0.

Example

```
ANACOR TABLE=MENTAL(1,4) BY SES(1,6)
 /PRINT=SCORES CONTRIBUTIONS
 /PLOT=ROWS COLUMNS.
```

- Two variables, *MENTAL* and *SES*, are specified on the TABLE subcommand. *MENTAL* has values ranging from 1 to 4, and *SES* has values ranging from 1 to 6.
- The row and column scores and the contribution of each row and column to the inertia of each dimension are displayed.
- Two plots are produced. The first one plots the first two dimensions of row scores, and the second one plots the first two dimensions of column scores.

TABLE Subcommand

TABLE specifies the row and column variables, along with their value ranges for individual casewise data. For table data, TABLE specifies the keyword ALL and the number of rows and columns.

- The TABLE subcommand is required.

Casewise Data

- Each variable is followed by a value range in parentheses. The value range consists of the variable's minimum value, a comma, and the variable's maximum value.
- Values outside of the specified range are not included in the analysis.
- Values do not have to be sequential. Empty categories receive scores of 0 and do not affect the rest of the computations.

Example

```
DATA LIST FREE/VAR1 VAR2.
BEGIN DATA
3 1
6 1
3 1
4 2
4 2
6 3
6 3
6 3
3 2
4 2
6 3
END DATA.
ANACOR TABLE=VAR1(3,6) BY VAR2(1,3).
```

- DATA LIST defines two variables, *VAR1* and *VAR2*.
- *VAR1* has three levels, coded 3, 4, and 6, while *VAR2* also has three levels, coded 1, 2, and 3.
- Because a range of (3,6) is specified for *VAR1*, ANACOR defines four categories, coded 3, 4, 5, and 6. The empty category, 5, for which there is no data, receives zeros for all statistics but does not affect the analysis.

Table Data

- The cells of a table can be read and analyzed directly by using the keyword ALL after TABLE.

- The columns of the input table must be specified as variables on the DATA LIST command. Only columns are defined, not rows.
- ALL is followed by the number of rows in the table, a comma, and the number of columns in the table, all enclosed in parentheses.
- If you want to analyze only a subset of the table, the specified number of rows and columns can be smaller than the actual number of rows and columns.
- The variables (columns of the table) are treated as the column categories, and the cases (rows of the table) are treated as the row categories.
- Rows cannot be labeled when you specify TABLE=ALL. If labels in your output are important, use the WEIGHT command method to enter your data (see "Analyzing Aggregated Data " on page 156).

Example

```
DATA LIST /COL01 TO COL07 1-21.
BEGIN DATA
 50 19 26  8 18  6  2
 16 40 34 18 31  8  3
 12 35 65 66123 23 21
 11 20 58110223 64 32
 14 36114185714258189
  0  6 19 40179143 71
END DATA.
ANACOR TABLE=ALL(6,7).
```

- DATA LIST defines the seven columns of the table as the variables.
- The TABLE=ALL specification indicates that the data are the cells of a table. The (6,7) specification indicates that there are six rows and seven columns.

DIMENSION Subcommand

DIMENSION specifies the number of dimensions you want ANACOR to compute.

- If you do not specify the DIMENSION subcommand, ANACOR computes two dimensions.
- DIMENSION is followed by an integer indicating the number of dimensions.
- In general, you should choose as few dimensions as needed to explain most of the variation. The minimum number of dimensions that can be specified is 1. The maximum number of dimensions that can be specified is equal to the number of levels of the variable with the least number of levels, minus 1. For example, in a table where one variable has five levels and the other has four levels, the maximum number of dimensions that can be specified is (4 – 1), or 3. Empty categories (categories with no data, all zeros, or all missing data) are not counted toward the number of levels of a variable.
- If more than the maximum allowed number of dimensions is specified, ANACOR reduces the number of dimensions to the maximum.

NORMALIZATION Subcommand

The NORMALIZATION subcommand specifies one of five methods for normalizing the row and column scores. Only the scores and variances are affected; contributions and profiles are not changed.

The following keywords are available:

CANONICAL. *For each dimension, rows are the weighted average of columns divided by the matching singular value, and columns are the weighted average of rows divided by the matching singular value*. This is the default if the NORMALIZATION subcommand is not specified. DEFAULT is an alias for CANONICAL. Use this normalization method if you are primarily interested in differences or similarities between variables.

PRINCIPAL. *Distances between row points and column points are approximations of chi-square distances*. The distances represent the distance between the row or column and its corresponding average row or column profile. Use this normalization method if you want to examine both differences between

categories of the row variable and differences between categories of the column variable (but not differences between variables).

RPRINCIPAL. *Distances between row points are approximations of chi-square distances*. This method maximizes distances between row points. This is useful when you are primarily interested in differences or similarities between categories of the row variable.

CPRINCIPAL. *Distances between column points are approximations of chi-square distances*. This method maximizes distances between column points. This is useful when you are primarily interested in differences or similarities between categories of the column variable.

The fifth method has no keyword. Instead, any value in the range –2 to +2 is specified after NORMALIZATION. A value of 1 is equal to the RPRINCIPAL method, a value of 0 is equal to CANONICAL, and a value of –1 is equal to the CPRINCIPAL method. The inertia is spread over both row and column scores. This method is useful for interpreting joint plots.

VARIANCES Subcommand

Use VARIANCES to display variances and correlations for the singular values, the row scores, and/or the column scores. If VARIANCES is not specified, variances and correlations are not included in the output.

The following keywords are available:

SINGULAR. *Variances and correlations of the singular values*.

ROWS. *Variances and correlations of the row scores*.

COLUMNS. *Variances and correlations of the column scores*.

PRINT Subcommand

Use PRINT to control which correspondence statistics are displayed. If PRINT is not specified, displayed statistics include the numbers of rows and columns, all nontrivial singular values, proportions of inertia, and the cumulative proportion of inertia that is accounted for.

The following keywords are available:

TABLE. *A crosstabulation of the input variables showing row and column marginals*.

PROFILES. *The row and column profiles*. PRINT=PROFILES is analogous to the CELLS=ROW COLUMN subcommand in CROSSTABS.

SCORES. *The marginal proportions and scores of each row and column*.

CONTRIBUTIONS. *The contribution of each row and column to the inertia of each dimension, and the proportion of distance to the origin that is accounted for in each dimension*.

PERMUTATION. *The original table permuted according to the scores of the rows and columns for each dimension*.

NONE. *No output other than the singular values*.

DEFAULT. *TABLE, SCORES, and CONTRIBUTIONS*. These statistics are displayed if you omit the PRINT subcommand.

PLOT Subcommand

Use PLOT to produce plots of the row scores, column scores, and row and column scores, as well as to produce plots of transformations of the row scores and transformations of the column scores. If PLOT is not specified, plots are produced for the row scores in the first two dimensions and the column scores in the first two dimensions.

The following keywords are available:

TRROWS. *Plot of transformations of the row category values into row scores*.

TRCOLUMNS. *Plot of transformations of the column category values into column scores*.

ROWS. *Plot of row scores*.

COLUMNS. *Plot of column scores*.

JOINT. *A combined plot of the row and column scores*. This plot is not available when NORMALIZATION=PRINCIPAL.

NONE. *No plots*.

DEFAULT. *ROWS and COLUMNS*.

- The keywords ROWS, COLUMNS, JOINT, and DEFAULT can be followed by an integer value in parentheses to indicate how many characters of the value label are to be used on the plot. The value can range from 1 to 20; the default is 3. Spaces between words count as characters.
- TRROWS and TRCOLUMNS plots use the full value labels up to 20 characters.
- If a label is missing for any value, the actual values are used for all values of that variable.
- Value labels should be unique.
- The first letter of a label on a plot marks the place of the actual coordinate. Be careful that multiple-word labels are not interpreted as multiple points on a plot.

In addition to the plot keywords, the following keyword can be specified:

NDIM. *Dimension pairs to be plotted*. NDIM is followed by a pair of values in parentheses. If NDIM is not specified, plots are produced for dimension 1 by dimension 2.

- The first value indicates the dimension that is plotted against all higher dimensions. This value can be any integer from 1 to the number of dimensions minus 1.
- The second value indicates the highest dimension to be used in plotting the dimension pairs. This value can be any integer from 2 to the number of dimensions.
- Keyword ALL can be used instead of the first value to indicate that all dimensions are paired with higher dimensions.
- Keyword MAX can be used instead of the second value to indicate that plots should be produced up to, and including, the highest dimension fit by the procedure.

Example

```
ANACOR TABLE=MENTAL(1,4) BY SES(1,6)
 /PLOT NDIM(1,3) JOINT(5).
```

- The NDIM(1,3) specification indicates that plots should be produced for two dimension pairs—dimension 1 versus dimension 2 and dimension 1 versus dimension 3.
- JOINT requests combined plots of row and column scores. The (5) specification indicates that the first five characters of the value labels are to be used on the plots.

Example

```
ANACOR TABLE=MENTAL(1,4) BY SES(1,6)
 /PLOT NDIM(ALL,3) JOINT(5).
```

- This plot is the same as above except for the ALL specification following NDIM, which indicates that all possible pairs up to the second value should be plotted. Therefore, JOINT plots will be produced for dimension 1 versus dimension 2, dimension 2 versus dimension 3, and dimension 1 versus dimension 3.

MATRIX Subcommand

Use MATRIX to write row and column scores and variances to matrix data files.

MATRIX is followed by keyword OUT, an equals sign, and one or both of the following keywords:

SCORE ('file'|'dataset'). *Write row and column scores to a matrix data file*.

VARIANCE ('file'|'dataset'). *Write variances to a matrix data file*.

- You can specify the file with either an asterisk (*), to replace the active dataset , a quoted file specification or a previously declared dataset name (DATASET DECLARE command), enclosed in parentheses.
- If you specify both SCORE and VARIANCE on the same MATRIX subcommand, you must specify two different files.

The variables in the SCORE matrix data file and their values are:

ROWTYPE_. *String variable containing the value ROW for all rows and COLUMN for all columns*.

LEVEL. *String variable containing the values (or value labels, if present) of each original variable*.

VARNAME_. *String variable containing the original variable names*.

DIM1...DIMn. *Numeric variables containing the row and column scores for each dimension*. Each variable is labeled *DIMn*, where *n* represents the dimension number.

The variables in the VARIANCE matrix data file and their values are:

ROWTYPE_. *String variable containing the value COV for all cases in the file*.

SCORE. *String variable containing the values SINGULAR, ROW, and COLUMN*.

LEVEL. *String variable containing the system-missing value for SINGULAR and the sequential row or column number for ROW and COLUMN*.

VARNAME_. *String variable containing the dimension number*.

DIM1...DIMn. *Numeric variables containing the covariances for each dimension*. Each variable is labeled *DIM n*, where *n* represents the dimension number.

Analyzing Aggregated Data

To analyze aggregated data, such as data from a crosstabulation where cell counts are available but the original raw data are not, you can use the TABLE=ALL option or the WEIGHT command before ANACOR.

Example

To analyze a 3 x 3 table, such as the table that is shown below, you could use these commands:

```
DATA LIST FREE/ BIRTHORD ANXIETY COUNT.
BEGIN DATA
1 1 48
1 2 27
1 3 22
2 1 33
2 2 20
2 3 39
3 1 29
3 2 42
3 3 47
END DATA.
WEIGHT BY COUNT.
ANACOR TABLE=BIRTHORD (1,3) BY ANXIETY (1,3).
```

- The WEIGHT command weights each case by the value of *COUNT*, as if there are 48 subjects with *BIRTHORD*=1 and *ANXIETY*=1, 27 subjects with *BIRTHORD*=1 and *ANXIETY*=2, and so on.
- ANACOR can then be used to analyze the data.
- If any table cell value equals 0, the WEIGHT command issues a warning, but the ANACOR analysis is done correctly.
- The table cell values (the WEIGHT values) cannot be negative. WEIGHT changes system-missing values and negative values to 0.

- For large aggregated tables, you can use the TABLE=ALL option or the transformation language to enter the table “as is.”

Table 12. 3 by 3 table

Birth Order	**Anxiety High**	**Anxiety Med**	**Anxiety Low**
First	48	27	22
Second	33	20	39
Other	29	42	47

ANOVA

ANOVA is available in the Statistics Base Edition.

```
ANOVA VARIABLES= varlist BY varlist(min,max)...varlist(min,max)
 [WITH varlist] [/VARIABLES=...]

 [/COVARIATES={FIRST**}]
              {WITH   }
              {AFTER  }

 [/MAXORDERS={ALL** }]
             {n     }
             {NONE  }

 [/METHOD={UNIQUE**     }]
          {EXPERIMENTAL}
          {HIERARCHICAL}

 [/STATISTICS=[MCA] [†REG] [MEAN] [ALL] [NONE]]

 [/MISSING={EXCLUDE**}]
           {INCLUDE  }
```

**Default if the subcommand is omitted.

†REG (table of regression coefficients) is displayed only if the design is relevant.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
ANOVA VARIABLES=PRESTIGE BY REGION(1,9) SEX,RACE(1,2)
 /MAXORDERS=2
 /STATISTICS=MEAN.
```

Overview

ANOVA performs analysis of variance for factorial designs. The default is the full factorial model if there are five or fewer factors. Analysis of variance tests the hypothesis that the group means of the dependent variable are equal. The dependent variable is interval-level, and one or more categorical variables define the groups. These categorical variables are termed **factors**. ANOVA also allows you to include continuous explanatory variables, termed **covariates**. Other procedures that perform analysis of variance are ONEWAY, SUMMARIZE, and GLM. To perform a comparison of two means, use TTEST.

Options

Specifying Covariates. You can introduce covariates into the model using the WITH keyword on the VARIABLES subcommand.

Order of Entry of Covariates. By default, covariates are processed before main effects for factors. You can process covariates with or after main effects for factors using the COVARIATES subcommand.

Suppressing Interaction Effects. You can suppress the effects of various orders of interaction using the MAXORDERS subcommand.

Methods for Decomposing Sums of Squares. By default, the regression approach (keyword UNIQUE) is used. You can request the classic experimental or hierarchical approach using the METHOD subcommand.

Statistical Display. Using the STATISTICS subcommand, you can request means and counts for each dependent variable for groups defined by each factor and each combination of factors up to the fifth level. You also can request unstandardized regression coefficients for covariates and multiple classification analysis (MCA) results, which include the MCA table, the Factor Summary table, and the Model Goodness of Fit table. The MCA table shows **treatment effects** as deviations from the grand mean and includes a listing of unadjusted category effects for each factor, category effects adjusted for other factors, and category effects adjusted for all factors and covariates. The Factor Summary table displays eta and beta values. The Goodness of Fit table shows R and R^2 for each model.

Basic Specification

- The basic specification is a single VARIABLES subcommand with an analysis list. The minimum analysis list specifies a list of dependent variables, the keyword BY, a list of factor variables, and the minimum and maximum integer values of the factors in parentheses.
- By default, the model includes all interaction terms up to five-way interactions. The sums of squares are decomposed using the regression approach, in which all effects are assessed simultaneously, with each effect adjusted for all other effects in the model. A case that has a missing value for any variable in an analysis list is omitted from the analysis.

Subcommand Order

- The subcommands can be named in any order.

Operations

A separate analysis of variance is performed for each dependent variable in an analysis list, using the same factors and covariates.

Limitations

- A maximum of 5 analysis lists.
- A maximum of 5 dependent variables per analysis list.
- A maximum of 10 factor variables per analysis list.
- A maximum of 10 covariates per analysis list.
- A maximum of 5 interaction levels.
- A maximum of 25 value labels per variable displayed in the MCA table.
- The combined number of categories for all factors in an analysis list plus the number of covariates must be less than the sample size.

Examples

```
ANOVA VARIABLES=PRESTIGE BY REGION(1,9) SEX, RACE(1,2)
 /MAXORDERS=2
 /STATISTICS=MEAN.
```

- VARIABLES specifies a three-way analysis of variance—*PRESTIGE* by *REGION*, *SEX*, and *RACE*.
- The variables *SEX* and *RACE* each have two categories, with values 1 and 2 included in the analysis. *REGION* has nine categories, valued 1 through 9.
- MAXORDERS examines interaction effects up to and including the second order. All three-way interaction terms are pooled into the error sum of squares.
- STATISTICS requests a table of means of *PRESTIGE* within the combined categories of *REGION*, *SEX*, and *RACE*.

Example: Specifying Multiple Analyses

```
ANOVA VARIABLES=PRESTIGE BY REGION(1,9) SEX,RACE(1,2)
  /RINCOME BY SEX,RACE(1,2).
```

- ANOVA specifies a three-way analysis of variance of *PRESTIGE* by *REGION*, *SEX*, and *RACE*, and a two-way analysis of variance of *RINCOME* by *SEX* and *RACE*.

VARIABLES Subcommand

VARIABLES specifies the analysis list.

- More than one design can be specified on the same ANOVA command by separating the analysis lists with a slash.

- Variables named before the keyword BY are dependent variables. Value ranges are not specified for dependent variables.
- Variables named after BY are factor (independent) variables.
- Every factor variable must have a value range indicating its minimum and maximum values. The values must be separated by a space or a comma and enclosed in parentheses.
- Factor variables must have integer values. Non-integer values for factors are truncated.
- Cases with values outside the range specified for a factor are excluded from the analysis.
- If two or more factors have the same value range, you can specify the value range once following the last factor to which it applies. You can specify a single range that encompasses the ranges of all factors on the list. For example, if you have two factors, one with values 1 and 2 and the other with values 1 through 4, you can specify the range for both as 1,4. However, this may reduce performance and cause memory problems if the specified range is larger than some of the actual ranges.
- Variables named after the keyword WITH are covariates.
- Each analysis list can include only one BY and one WITH keyword.

COVARIATES Subcommand

COVARIATES specifies the order for assessing blocks of covariates and factor main effects.

- The order of entry is irrelevant when METHOD=UNIQUE.

FIRST. *Process covariates before factor main effects.* This is the default.

WITH. *Process covariates concurrently with factor main effects.*

AFTER. *Process covariates after factor main effects.*

MAXORDERS Subcommand

MAXORDERS suppresses the effects of various orders of interaction.

ALL. *Examine all interaction effects up to and including the fifth order.* This is the default.

n. *Examine all interaction effects up to and including the* n*th order.* For example, MAXORDERS=3 examines all interaction effects up to and including the third order. All higher-order interaction sums of squares are pooled into the error term.

NONE. *Delete all interaction terms from the model.* All interaction sums of squares are pooled into the error sum of squares. Only main and covariate effects appear in the ANOVA table.

METHOD Subcommand

METHOD controls the method for decomposing sums of squares.

UNIQUE. *Regression approach*. UNIQUE overrides any keywords on the COVARIATES subcommand. All effects are assessed simultaneously for their partial contribution. The MCA and MEAN specifications on the STATISTICS subcommand are not available with the regression approach. This is the default if METHOD is omitted.

EXPERIMENTAL. *Classic experimental approach.* Covariates, main effects, and ascending orders of interaction are assessed separately in that order.

HIERARCHICAL. *Hierarchical approach.*

Regression Approach

All effects are assessed simultaneously, with each effect adjusted for all other effects in the model. This is the default when the METHOD subcommand is omitted. Since MCA tables cannot be produced when the

regression approach is used, specifying MCA or ALL on STATISTICS with the default method triggers a warning.

Some restrictions apply to the use of the regression approach:

- The lowest specified categories of all the independent variables must have a marginal frequency of at least 1, since the lowest specified category is used as the reference category. If this rule is violated, no ANOVA table is produced and a message identifying the first offending variable is displayed.
- Given an *n*-way crosstabulation of the independent variables, there must be no empty cells defined by the lowest specified category of any of the independent variables. If this restriction is violated, one or more levels of interaction effects are suppressed and a warning message is issued. However, this constraint does not apply to categories defined for an independent variable but not occurring in the data. For example, given two independent variables, each with categories of 1, 2, and 4, the (1,1), (1,2), (1,4), (2,1), and (4,1) cells must not be empty. The (1,3) and (3,1) cells will be empty but the restriction on empty cells will not be violated. The (2,2), (2,4), (4,2), and (4,4) cells may be empty, although the degrees of freedom will be reduced accordingly.

To comply with these restrictions, specify precisely the lowest non-empty category of each independent variable. Specifying a value range of (0,9) for a variable that actually has values of 1 through 9 results in an error, and no ANOVA table is produced.

Classic Experimental Approach

Each type of effect is assessed separately in the following order (unless WITH or AFTER is specified on the COVARIATES subcommand):

- Effects of covariates
- Main effects of factors
- Two-way interaction effects
- Three-way interaction effects
- Four-way interaction effects
- Five-way interaction effects

The effects within each type are adjusted for all other effects of that type and also for the effects of all prior types. (See Table 13 on page 163.)

Hierarchical Approach

The hierarchical approach differs from the classic experimental approach only in the way it handles covariate and factor main effects. In the hierarchical approach, factor main effects and covariate effects are assessed hierarchically—factor main effects are adjusted only for the factor main effects already assessed, and covariate effects are adjusted only for the covariates already assessed. (See Table 13 on page 163.) The order in which factors are listed on the ANOVA command determines the order in which they are assessed.

Example

The following analysis list specifies three factor variables named *A*, *B*, and *C*:

```
ANOVA VARIABLES=Y BY A,B,C(0,3).
```

The following table summarizes the three methods for decomposing sums of squares for this example.

- With the default *regression* approach, each factor or interaction is assessed with all other factors and interactions held constant.
- With the *classic experimental* approach, each main effect is assessed with the two other main effects held constant, and two-way interactions are assessed with all main effects and other two-way interactions held constant. The three-way interaction is assessed with all main effects and two-way interactions held constant.

- With the *hierarchical* approach, the factor main effects A, B, and C are assessed with all prior main effects held constant. The order in which the factors and covariates are listed on the ANOVA command determines the order in which they are assessed in the hierarchical analysis. The interaction effects are assessed the same way as in the experimental approach.

Table 13. Terms adjusted for under each option

Effect	Regression (UNIQUE)	Experimental	Hierarchical
A	All others	B,C	None
B	All others	A,C	A
C	All others	A,B	A,B
AB	All others	A,B,C,AC,BC	A,B,C,AC,BC
AC	All others	A,B,C,AB,BC	A,B,C,AB,BC
BC	All others	A,B,C,AB,AC	A,B,C,AB,AC
ABC	All others	A,B,C,AB,AC,BC	A,B,C,AB,AC,BC

Summary of Analysis Methods

The following table describes the results obtained with various combinations of methods for controlling the entry of covariates and decomposing the sums of squares.

Table 14. Combinations of COVARIATES and METHOD subcommands

Method	Assessments between types of effects	Assessments within the same type of effect
METHOD=UNIQUE	Covariates, Factors, and Interactions simultaneously	Covariates: adjust for factors, interactions, and all other covariates Factors: adjust for covariates, interactions, and all other factors Interactions: adjust for covariates, factors, and all other interactions
METHOD=EXPERIMENTAL	Covariates then Factors then Interactions	Covariates: adjust for all other covariates Factors: adjust for covariates and all other factors Interactions: adjust for covariates, factors, and all other interactions of the same and lower orders
METHOD=HIERARCHICAL	Covariates then Factors then Interactions	Covariates: adjust for covariates that are preceding in the list Factors: adjust for covariates and factors preceding in the list Interactions: adjust for covariates, factors, and all other interactions of the same and lower orders

Table 14. Combinations of COVARIATES and METHOD subcommands (continued)

Method	Assessments between types of effects	Assessments within the same type of effect
COVARIATES=WITH and METHOD=EXPERIMENTAL	Factors and Covariates concurrently then Interactions	Covariates: adjust for factors and all other covariates Factors: adjust for covariates and all other factors Interactions: adjust for covariates, factors, and all other interactions of the same and lower orders
COVARIATES=WITH and METHOD=HIERARCHICAL	Factors and Covariates concurrently then Interactions	Factors: adjust only for preceding factors Covariates: adjust for factors and preceding covariates Interactions: adjust for covariates, factors, and all other interactions of the same and lower orders
COVARIATES=AFTER and METHOD=EXPERIMENTAL	Factors then Covariates then Interactions	Factors: adjust for all other factors Covariates: adjust for factors and all other covariates Interactions: adjust for covariates, factors, and all other interactions of the same and lower orders
COVARIATES=AFTER and METHOD=HIERARCHICAL	Factors then Covariates then Interactions	Factors: adjust only for preceding factors Covariates: adjust factors and preceding covariates Interactions: adjust for covariates, factors, and all other interactions of the same and lower orders

STATISTICS Subcommand

STATISTICS requests additional statistics. STATISTICS can be specified by itself or with one or more keywords.

- If you specify STATISTICS without keywords, ANOVA calculates MEAN and REG (each defined below).
- If you specify a keyword or keywords on the STATISTICS subcommand, ANOVA calculates only the additional statistics you request.

MEAN. *Means and counts table*. This statistic is not available when METHOD is omitted or when METHOD=UNIQUE. See "Cell Means" below.

REG. *Unstandardized regression coefficients*. Displays unstandardized regression coefficients for the covariates. See the topic "Regression Coefficients for the Covariates " on page 165 for more information.

MCA. *Multiple classification analysis*. The MCA, the Factor Summary, and the Goodness of Fit tables are not produced when METHOD is omitted or when METHOD=UNIQUE. See the topic "Multiple Classification Analysis " on page 165 for more information.

ALL. *Means and counts table, unstandardized regression coefficients, and multiple classification analysis.*

NONE. *No additional statistics.* ANOVA calculates only the statistics needed for analysis of variance. This is the default if the STATISTICS subcommand is omitted.

Cell Means

STATISTICS=MEAN displays the Cell Means table.

- This statistic is not available with METHOD=UNIQUE.
- The Cell Means table shows the means and counts of each dependent variable for each cell defined by the factors and combinations of factors. Dependent variables and factors appear in their order on the VARIABLES subcommand.
- If MAXORDERS is used to suppress higher-order interactions, cell means corresponding to suppressed interaction terms are not displayed.
- The means displayed are the observed means in each cell, and they are produced only for dependent variables, not for covariates.

Regression Coefficients for the Covariates

STATISTICS=REG requests the unstandardized regression coefficients for the covariates.

- The regression coefficients are computed at the point where the covariates are entered into the equation. Thus, their values depend on the type of design specified by the COVARIATES or METHOD subcommand.
- The coefficients are displayed in the ANOVA table.

Multiple Classification Analysis

STATISTICS=MCA displays the MCA, the Factor Summary, and the Model Goodness of Fit tables.

- The MCA table presents counts, predicted means, and deviations of predicted means from the grand mean for each level of each factor. The predicted and deviation means each appear in up to three forms: unadjusted, adjusted for other factors, and adjusted for other factors and covariates.
- The Factor Summary table displays the correlation ratio (eta) with the unadjusted deviations (the square of eta indicates the proportion of variance explained by all categories of the factor), a partial beta equivalent to the standardized partial regression coefficient that would be obtained by assigning the unadjusted deviations to each factor category and regressing the dependent variable on the resulting variables, and the parallel partial betas from a regression that includes covariates in addition to the factors.
- The Model Goodness of Fit table shows R and R^2 for each model.
- The tables cannot be produced if METHOD is omitted or if METHOD=UNIQUE. When produced, the MCA table does not display the values adjusted for factors if COVARIATES is omitted, if COVARIATES=FIRST, or if COVARIATES=WITH and METHOD=EXPERIMENTAL. A full MCA table is produced only if METHOD=HIERARCHICAL or if METHOD=EXPERIMENTAL and COVARIATES=AFTER.

MISSING Subcommand

By default, a case that has a missing value for any variable named in the analysis list is deleted for all analyses specified by that list. Use MISSING to include cases with user-missing data.

EXCLUDE. *Exclude cases with missing data.* This is the default.

INCLUDE. *Include cases with user-defined missing data.*

References

Andrews, F., J. Morgan, J. Sonquist, and L. Klein. 1973. *Multiple classification analysis*, 2nd ed. Ann Arbor: University of Michigan.

APPLY DICTIONARY

```
APPLY DICTIONARY FROM [{'savfile'|'dataset'}] [PASSWORD='password']
                      {*                     }

 [/SOURCE VARIABLES = varlist]

 [/TARGET VARIABLES = varlist]

 [/NEWVARS]

 [/FILEINFO [ATTRIBUTES = [{REPLACE}]]
                           {MERGE  }

            [DOCUMENTS = [{REPLACE}]]  ]
                         {MERGE  }

            [FILELABEL]
            [MRSETS = [{REPLACE}]]
                      {MERGE  }

            [VARSETS = [{REPLACE}]]
                       {MERGE  }

            [WEIGHT**]

            [ALL]

    [/VARINFO [ALIGNMENT**]  ]


           [ATTRIBUTES = [{REPLACE}]]
                         {MERGE  }
           [FORMATS**]

           [LEVEL**]

           [MISSING**]

           [ROLE**]

           [VALLABELS = [{REPLACE**}]]
                        {MERGE   }

           [VARLABEL**]

           [WIDTH**]

           [ALL]
```

**Default if the subcommand is not specified.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 14.0

- ATTRIBUTES keyword introduced on FILEINFO and VARINFO subcommands.

Release 18

- ROLE keyword introduced on VARINFO subcommands.

Release 22.0

- PASSWORD keyword introduced on the FROM subcommand.

Example

```
APPLY DICTIONARY FROM = 'lastmonth.sav'.
```

Overview

APPLY DICTIONARY can apply variable and file-based dictionary information from an external IBM SPSS Statistics data file or open dataset to the current active dataset. Variable-based dictionary information in the current active dataset can be applied to other variables in the current active dataset.

- The applied variable information includes variable and value labels, missing-value flags, alignments, variable print and write formats, measurement levels, and widths.
- The applied file information includes variable and multiple response sets, documents, file label, and weight.
- APPLY DICTIONARY can apply information selectively to variables and can apply selective file-based dictionary information.
- Individual variable attributes can be applied to individual and multiple variables of the same type (strings of the same character length or numeric).
- APPLY DICTIONARY can add new variables but cannot remove variables, change data, or change a variable's name or type.
- Undefined (empty) attributes in the source dataset do not overwrite defined attributes in the active dataset.

Basic Specification

The basic specification is the FROM subcommand and the name of an external IBM SPSS Statistics data file or open dataset. The file specification should be enclosed in quotation marks.

Subcommand Order

The subcommands can be specified in any order.

Syntax Rules

- The file containing the dictionary information to be applied (the **source file**) must be an external IBM SPSS Statistics data file or a currently open dataset.
- The file to which the dictionary information is applied (the **target file**) must be the active dataset. You cannot specify another file.
- If a subcommand is issued more than once, APPLY DICTIONARY will ignore all but the last instance of the subcommand.
- Equals signs displayed in the syntax chart and in the examples presented here are required elements; they are not optional.

Matching Variable Type

APPLY DICTIONARY considers two variables to have a matching variable type if:

- Both variables are numeric. This includes all numeric, currency, and date formats.
- Both variables are string (alphanumeric).

FROM Subcommand

FROM specifies an external IBM SPSS Statistics data file or an open dataset as the source file whose dictionary information is to be applied to the active dataset.

- FROM is required.
- Only one IBM SPSS Statistics data file or open dataset (including the active dataset) can be specified on FROM.
- The file specification should be enclosed in quotation marks.
- The active dataset can be specified in the FROM subcommand by using an asterisk (*) as the value. File-based dictionary information (FILEINFO subcommand) is ignored when the active dataset is used as the source file.

PASSWORD Keyword

The PASSWORD keyword specifies the password required to open an encrypted IBM SPSS Statistics data file. The specified value must be enclosed in quotation marks and can be provided as encrypted or as plain text. Encrypted passwords are created when pasting command syntax from the Save Data As dialog. The PASSWORD keyword is ignored if the file is not encrypted.

Example

```
APPLY DICTIONARY FROM "lastmonth.sav".
```

- This will apply variable information from *lastmonth.sav* to matching variables in the active dataset.
- The default variable information applied from the source file includes variable labels, value labels, missing values, level of measurement, alignment, column width (for Data Editor display), and print and write formats.
- If weighting is on in the source dataset and a matching weight variable exists in the active (target) dataset, weighting by that variable is turned on in the active dataset. No other file information (documents, file label, multiple response sets) from the source file is applied to the active dataset.

NEWVARS Subcommand

NEWVARS is required to create new variables in the active (target) dataset.

Example

```
APPLY DICTIONARY FROM “lastmonth.sav”
  /NEWVARS.
```

- For a new, blank active dataset, all variables with all of their variable definition attributes are copied from the source dataset, creating a new dataset with an identical set of variables (but no data values).
- For an active dataset that contains any variables, variable definition attributes from the source dataset are applied to the matching variables in the active (target) dataset. If the source dataset contains any variables that are not present in the active dataset (determined by variable name), these variables are created in the active dataset.

SOURCE and TARGET Subcommands

The SOURCE subcommand is used to specify variables in the source file from which to apply variable definition attributes. The TARGET subcommand is used to specify variables in the active dataset to which to apply variable definition attributes.

- All variables specified in the SOURCE subcommand must exist in the source file.
- If the TARGET subcommand is specified without the SOURCE subcommand, all variables specified must exist in the source file.
- If the NEWVARS subcommand is specified, variables that are specified in the SOURCE subcommand that exist in the source file but not in the target file will be created in the target file as new variables using the variable definition attributes (variable and value labels, missing values, etc.) from the source variable.
- For variables with matching name and type, variable definition attributes from the source variable are applied to the matching target variable.
- If both SOURCE and TARGET are specified, the SOURCE subcommand can specify only one variable. Variable definition attributes from that single variable in the SOURCE subcommand are applied to all variables of the matching type. When applying the attributes of one variable to many variables, all variables specified in the SOURCE and TARGET subcommands must be of the same type.
- For variables with matching names but different types, only variable labels are applied to the target variables.

Table 15. Variable mapping for SOURCE and TARGET subcommands		
SOURCE subcommand	**TARGET subcommand**	**Variable mapping**
none	none	Variable definition attributes from the source dataset are applied to matching variables in the active (target) dataset. New variables may be created if the NEWVARS subcommand is specified.
many	none	Variable definition attributes for the specified variables are copied from the source dataset to the matching variables in the active (target) dataset. All specified variables must exist in the source dataset. New variables may be created if the NEWVARS subcommand is specified.
none	many	Variable definition attributes for the specified variables are copied from the source dataset to the matching variables in the active (target) dataset. All specified variables must exist in the source dataset. New variables may be created if the NEWVARS subcommand is specified.
one	many	Variable definition attributes for the specified variable in the source dataset are applied to all specified variables in the active (target) dataset that have a matching type. New variables may be created if the NEWVARS subcommand is specified.
many	many	Invalid. Command not executed.

Example

```
APPLY DICTIONARY from *
  /SOURCE VARIABLES = var1
  /TARGET VARIABLES = var2 var3 var4
  /NEWVARS.
```

- Variable definition attributes for *var1* in the active dataset are copied to *var2*, *var3*, and *var4* in the same dataset if they have a matching type.
- Any variables specified in the TARGET subcommand that do not already exist are created, using the variable definition attributes of the variable specified in the SOURCE subcommand.

Example

```
APPLY DICTIONARY from “lastmonth.sav”
  /SOURCE VARIABLES = var1, var2, var3.
```

- Variable definition attributes from the specified variables in the source dataset are applied to the matching variables in the active dataset.
- For variables with matching names but different types, only variable labels from the source variable are copied to the target variable.
- In the absence of a NEWVARS subcommand, no new variables will be created.

FILEINFO Subcommand

FILEINFO applies global file definition attributes from the source dataset to the active (target) dataset.

- File definition attributes in the active dataset that are undefined in the source dataset are not affected.
- This subcommand is ignored if the source dataset is the active dataset.
- This subcommand is ignored if no keywords are specified.
- For keywords that contain an associated value, the equals sign between the keyword and the value is required—for example, DOCUMENTS = MERGE.

ATTRIBUTES. *Applies file attributes defined by the* DATAFILE ATTRIBUTE *command*. You can REPLACE or MERGE file attributes.

DOCUMENTS. *Applies documents (defined with the* DOCUMENTS *command) from the source dataset to the active (target) dataset.* You can REPLACE or MERGE documents. DOCUMENTS = REPLACE replaces any documents in the active dataset, deleting preexisting documents in the file. This is the default if DOCUMENTS is specified without a value. DOCUMENTS = MERGE merges documents from the source and active datasets. Unique documents in the source file that don't exist in the active dataset are added to the active dataset. All documents are then sorted by date.

FILELABEL. *Replaces the file label (defined with the* FILE LABEL *command)*.

MRSETS. *Applies multiple response set definitions from the source dataset to the active dataset.* Multiple response sets that contain no variables in the active dataset (including variables added by the same APPLY DICTIONARY command) are ignored. You can REPLACE or MERGE multiple response sets. MRSETS = REPLACE deletes any existing multiple response sets in the active dataset, replacing them with multiple response sets from the source dataset. MRSETS = MERGE adds multiple response sets from the source dataset to the collection of multiple response sets in the active dataset. If a set with the same name exists in both files, the existing set in the active dataset is unchanged.

VARSETS. *Applies variable set definitions from the source dataset to the active dataset.* Variable sets are used to control the list of variables that are displayed in dialog boxes. Variable sets are defined by selecting **Define Variable Sets** from the Utilities menu. Sets in the source data file that don't contain any variables in the active dataset are ignored unless those variables are created by the same APPLY DICTIONARY command. You can REPLACE or MERGE variable sets. VARSETS = REPLACE deletes any existing variable sets in the active dataset, replacing them with variable sets from the source dataset. VARSETS = MERGE adds variable sets from the source dataset to the collection of variable sets in the active dataset. If a set with the same name exists in both files, the existing set in the active dataset is unchanged.

WEIGHT. *Weights cases by the variable specified in the source file if there's a matching variable in the target file*. This is the default if the subcommand is omitted.

ALL. *Applies all file information from the source dataset to the active dataset*. Documents, multiple response sets, and variable sets are merged, not replaced. File definition attributes in the active dataset that are undefined in the source data file are not affected.

Example

```
APPLY DICTIONARY FROM "lastmonth.sav"
  /FILEINFO DOCUMENTS = REPLACE MRSETS = MERGE.
```

- Documents in the source dataset replace documents in the active dataset unless there are no defined documents in the source dataset.
- Multiple response sets from the source dataset are added to the collection of defined multiple response sets in the active dataset. Sets in the source dataset that contain variables that don't exist in the active dataset are ignored. If the same set name exists in both datasets, the set in the active dataset remains unchanged.

VARINFO Subcommand

VARINFO applies variable definition attributes from the source dataset to the matching variables in the active dataset. With the exception of VALLABELS, all keywords replace the variable definition attributes in the active dataset with the attributes from the matching variables in the source dataset.

ALIGNMENT. *Applies variable alignment for Data Editor display.* This setting affects alignment (left, right, center) only in the Data View display of the Data Editor.

ATTRIBUTES. *Applies variable attributes defined by the* VARIABLE ATTRIBUTE *command*. You can REPLACE or MERGE variable attributes.

FORMATS. *Applies variable print and write formats*. This is the same variable definition attribute that can be defined with the FORMATS command. This setting is primarily applicable only to numeric variables. For string variables, this affects only the formats if the source or target variable is AHEX format and the other is A format.

LEVEL. *Applies variable measurement level (nominal, ordinal, scale)*. This is the same variable definition attribute that can be defined with the VARIABLE LEVEL command.

MISSING. *Applies variable missing value definitions*. Any existing defined missing values in the matching variables in the active dataset are deleted. This is the same variable definition attribute that can be defined with the MISSING VALUES command. Missing values definitions are not applied to string variables if the source variable contains missing values of a longer width than the defined width of the target variable.

ROLE. *Applies role assignments*. See the topic “Overview ” on page 2129 for more information.

VALLABELS. *Applies value label definitions*. Value labels are not applied to string variables if the source variable contains defined value labels for values longer than the defined width of the target variable. You can REPLACE or MERGE value labels. VALLABELS = REPLACE replaces any defined value labels from variable in the active dataset with the value labels from the matching variable in the source dataset. VALLABELS = MERGE merges defined value labels for matching variables. If the same value has a defined value label in both the source and active datasets, the value label in the active dataset is unchanged.

WIDTH. *Display column width in the Data Editor.* This affects only column width in Data View in the Data Editor. It has no affect on the defined width of the variable.

Example

```
APPLY DICTIONARY from “lastmonth.sav”
  /VARINFO LEVEL MISSING VALLABELS = MERGE.
```

- The level of measurement and defined missing values from the source dataset are applied to the matching variables in the active (target) dataset. Any existing missing values definitions for those variables in the active dataset are deleted.
- Value labels for matching variables in the two datasets are merged. If the same value has a defined value label in both the source and active datasets, the value label in the active dataset is unchanged.

AREG

```
AREG [VARIABLES=] dependent series name WITH independent series names

 [/METHOD={PW**}]
          {CO  }
          {ML  }

 [/{†CONSTANT}]
   {NOCONSTANT}

 [/RHO={0**  }]
       {value}

 [/MXITER={10**}]
          {n   }

 [/APPLY [='model name'] [{SPECIFICATIONS}]]
                          {INITIAL       }
                          {FIT           }
```

**Default if the subcommand is omitted.

†CONSTANT is the default if the subcommand or keyword is omitted and there is no corresponding specification on the TSET command.

Method definitions:

PW. Prais-Winsten (GLS) estimation

CO. Cochrane-Orcutt estimation

ML. Exact maximum-likelihood estimation

Example

```
AREG VARY WITH VARX.
```

Overview

AREG estimates a regression model with AR(1) (first-order autoregressive) errors. (Models whose errors follow a general ARIMA process can be estimated using the ARIMA procedure.) AREG provides a choice among three estimation techniques.

For the Prais-Winsten and Cochrane-Orcutt estimation methods (keywords PW and CO), you can obtain the rho values and statistics at each iteration, and regression statistics for the ordinary least-square and final Prais-Winsten or Cochrane-Orcutt estimates. For the maximum-likelihood method (keyword ML), you can obtain the adjusted sum of squares and Marquardt constant at each iteration and, for the final parameter estimates, regression statistics, correlation and covariance matrices, Akaike's information criterion (AIC) [8], and Schwartz's Bayesian criterion (SBC) [9].

Options

Estimation Technique. You can select one of three available estimation techniques (Prais-Winsten, Cochrane-Orcutt, or exact maximum-likelihood) on the METHOD subcommand. You can request regression through the origin or inclusion of a constant in the model by specifying NOCONSTANT or CONSTANT to override the setting on the TSET command.

Rho Value. You can specify the value to be used as the initial rho value (estimate of the first autoregressive parameter) on the RHO subcommand.

Iterations. You can specify the maximum number of iterations the procedure is allowed to cycle through in calculating estimates on the MXITER subcommand.

[8] Akaike, H. 1974. A new look at the statistical model identification. *IEEE Transaction on Automatic Control,* AC–19, 716-723.

[9] Schwartz, G. 1978. Estimating the dimensions of a model. *Annals of Statistics,* 6, 461-464.

Statistical Output. To display estimates and statistics at each iteration in addition to the default output, specify TSET PRINT=DETAILED before AREG. To display only the final parameter estimates, use TSET PRINT=BRIEF (see TSET for more information).

New Variables. To evaluate the regression summary table without creating new variables, specify TSET NEWVAR=NONE prior to AREG. This can result in faster processing time. To add new variables without erasing the values of previous Forecasting-generated variables, specify TSET NEWVAR=ALL. This saves all new variables generated during the session to the active dataset and may require extra processing time.

Basic Specification

The basic specification is one dependent series name, the keyword WITH, and one or more independent series names.

- By default, procedure AREG estimates a regression model using the Prais-Winsten (GLS) technique. The number of iterations is determined by the convergence value set on TSET CNVERGE (default of 0.001), up to the default maximum number of 10 iterations. A 95% confidence interval is used unless it is changed by a TSET CIN command prior to the AREG procedure.
- Unless the default on TSET NEWVAR is changed prior to AREG, five variables are automatically created, labeled, and added to the active dataset: fitted values (*FIT#1*), residuals (*ERR#1*), lower confidence limits (*LCL#1*), upper confidence limits (*UCL#1*), and standard errors of prediction (*SEP#1*).

Subcommand Order

- VARIABLES must be specified first.
- The remaining subcommands can be specified in any order.

Syntax Rules

- VARIABLES can be specified only once.
- Other subcommands can be specified more than once, but only the last specification of each one is executed.

Operations

- AREG cannot forecast beyond the end of the regressor (independent) series (see PREDICT for more information).
- Method ML allows missing data anywhere in the series. Missing values at the beginning and end are skipped and the analysis proceeds with the first nonmissing case using Melard's algorithm. If imbedded missing values are found, they are noted and the Kalman filter is used for estimation.
- Methods PW and CO allow missing values at the beginning or end of the series but not within the series. Missing values at the beginning or end of the series are skipped. If imbedded missing values are found, a warning is issued suggesting the ML method be used instead and the analysis terminates. (See RMV for information on replacing missing values.)
- Series with missing cases may require extra processing time.

Limitations

- Maximum 1 VARIABLES subcommand.
- Maximum 1 dependent series in the series list. There is no limit on the number of independent series.

VARIABLES Subcommand

VARIABLES specifies the series list and is the only required subcommand. The actual keyword VARIABLES can be omitted.

- The dependent series is specified first, followed by the keyword WITH and one or more independent series.

METHOD Subcommand

METHOD specifies the estimation technique. Three different estimation techniques are available.

- If METHOD is not specified, the Prais-Winsten method is used.
- Only one method can be specified on the METHOD subcommand.

The available methods are:

PW. *Prais-Winsten method.* This generalized least-squares approach is the default [10].

CO. *Cochrane-Orcutt method.* [11]

ML. *Exact maximum-likelihood method.* This method can be used when one of the independent variables is the lagged dependent variable. It can also handle missing data anywhere in the series [12].

Example

```
AREG VARY WITH VARX
  /METHOD=CO.
```

In this example, the Cochrane-Orcutt method is used to estimate the regression model.

CONSTANT and NOCONSTANT Subcommands

CONSTANT and NOCONSTANT indicate whether a constant term should be estimated in the regression equation. The specification overrides the corresponding setting on the TSET command.

- CONSTANT indicates that a constant should be estimated. It is the default unless changed by TSET NOCONSTANT prior to the current procedure.
- NOCONSTANT eliminates the constant term from the model.

RHO Subcommand

RHO specifies the initial value of rho, an estimate of the first autoregressive parameter.

- If RHO is not specified, the initial rho value defaults to 0 (equivalent to ordinary least squares).
- The value specified on RHO can be any value greater than –1 and less than 1.
- Only one rho value can be specified per AREG command.

Example

```
AREG VAR01 WITH VAR02 VAR03
  /METHOD=CO
  /RHO=0.5.
```

- In this example, the Cochrane-Orcutt (CO) estimation method with an initial rho value of 0.5 is used.

MXITER Subcommand

MXITER specifies the maximum number of iterations of the estimation process.

- If MXITER is not specified, the maximum number of iterations defaults to 10.
- The specification on MXITER can be any positive integer.

[10] Johnston, J. 1984. *Econometric methods*. New York: McGraw-Hill.
[11] Johnston, J. 1984. *Econometric methods*. New York: McGraw-Hill.
[12] Kohn, R., and C. Ansley. 1986. Estimation, prediction, and interpolation for ARIMA models with missing data. *Journal of the American Statistical Association,* 81, 751-761.

- Iteration stops either when the convergence criterion is met or when the maximum is reached, whichever occurs first. The convergence criterion is set on the TSET CNVERGE command. The default is 0.001.

Example

```
AREG VARY WITH VARX
  /MXITER=5.
```

- In this example, AREG generates Prais-Winsten estimates and associated statistics with a maximum of 5 iterations.

APPLY Subcommand

APPLY allows you to use a previously defined AREG model without having to repeat the specifications.

- The specifications on APPLY can include the name of a previous model in quotes and one of three keywords. All of these specifications are optional.
- If a model name is not specified, the model specified on the previous AREG command is used.
- To change one or more specifications of the model, specify the subcommands of only those portions you want to change after the APPLY subcommand.
- If no series are specified on the AREG command, the series that were originally specified with the model being reapplied are used.
- To change the series used with the model, enter new series names before or after the APPLY subcommand. If a series name is specified before APPLY, the slash before the subcommand is required.
- APPLY with the keyword FIT sets MXITER to 0. If you apply a model that used FIT and want to obtain estimates, you will need to respecify MXITER.

The keywords available for APPLY with AREG are:

SPECIFICATIONS. *Use only the specifications from the original model.* AREG should create the initial values. This is the default.

INITIAL. *Use the original model's final estimates as initial values for estimation.*

FIT. *No estimation.* Estimates from the original model should be applied directly.

Example

```
AREG VARY WITH VARX
  /METHOD=CO
  /RHO=0.25
  /MXITER=15.
AREG VARY WITH VARX
  /METHOD=ML.
AREG VARY WITH VAR01
  /APPLY.
AREG VARY WITH VAR01
  /APPLY='MOD_1'
  /MXITER=10.
AREG VARY WITH VAR02
  /APPLY FIT.
```

- The first command estimates a regression model for *VARY* and *VARX* using the Cochrane-Orcutt method, an initial rho value of 0.25, and a maximum of 15 iterations. This model is assigned the name *MOD_1*.
- The second command estimates a regression model for *VARY* and *VARX* using the ML method. This model is assigned the name *MOD_2*.
- The third command displays the regression statistics for the series *VARY* and *VAR01* using the same method, ML, as in the second command. This model is assigned the name *MOD_3*.
- The fourth command applies the same method and rho value as in the first command but changes the maximum number of iterations to 10. This new model is named *MOD_4*.

- The last command applies the last model, *MOD_4*, using the series *VARY* and *VAR02*. The FIT specification means the final estimates of *MOD_4* should be applied directly to the new series with no new estimation.

References

Akaike, H. 1974. A new look at the statistical model identification. *IEEE Transaction on Automatic Control,* AC–19, 716-723.

Harvey, A. C. 1981. *The econometric analysis of time series*. Oxford: Philip Allan.

Johnston, J. 1984. *Econometric methods*. New York: McGraw-Hill.

Kohn, R., and C. Ansley. 1986. Estimation, prediction, and interpolation for ARIMA models with missing data. *Journal of the American Statistical Association,* 81, 751-761.

Schwartz, G. 1978. Estimating the dimensions of a model. *Annals of Statistics,* 6, 461-464.

ARIMA

```
ARIMA [VARIABLES=] dependent series name [WITH independent series names]

 [/MODEL =[(p,d,q)[(sp,sd,sq)[period]]]

          [{CONSTANT† }] [{NOLOG†     }]]
           {NOCONSTANT}   {LG10 or LOG}
                          {LN          }

 [/P={value       }]  [/D=value]  [/Q={value       }]
     {(value list)}                  {(value list)}

 [/SP={value       }]  [/SD=value]  [/SQ={value       }]
      {(value list)}                     {(value list)}

 [/AR=value list] [/MA=value list]

 [/SAR=value list] [/SMA=value list]

 [/REG=value list] [/CON=value]

 [/MXITER={10** }] [/MXLAMB={1.0E9**}]
          {value}           {value  }

 [/SSQPCT={0.001**}] [/PAREPS={0.001†}]
          {value  }          {value }

 [/CINPCT={95†  }]
          {value}

 [/APPLY [='model name'] [{SPECIFICATIONS}]]
                          {INITIAL       }
                          {FIT           }

 [/FORECAST=[{EXACT   }]]
            {CLS     }
            {AUTOINIT}
```

**Default if the subcommand is omitted.

†Default if the subcommand or keyword is omitted and there is no corresponding specification on the TSET command.

Example

```
ARIMA SALES
  /MODEL=(0,1,1)(0,1,1).
```

Overview

ARIMA estimates nonseasonal and seasonal univariate ARIMA models with or without fixed regressor variables. The procedure uses a subroutine library written by Craig Ansley that produces maximum-likelihood estimates and can process time series with missing observations.

Options

Model Specification. The traditional ARIMA (p,d,q)(sp,sd,sq) model incorporates nonseasonal and seasonal parameters multiplicatively and can be specified on the MODEL subcommand. You can also specify ARIMA models and constrained ARIMA models by using the separate parameter-order subcommands P, D, Q, SP, SD, and SQ.

Parameter Specification. If you specify the model in the traditional (p,d,q) (sp,sd,sq) format on the MODEL subcommand, you can additionally specify the period length, whether a constant should be included in the model (using the keyword CONSTANT or NOCONSTANT), and whether the series should first be log transformed (using the keyword NOLOG, LG10, or LN). You can fit single or nonsequential parameters by using the separate parameter-order subcommands to specify the exact lags. You can also specify initial values for any of the parameters using the AR, MA, SAR, SMA, REG, and CON subcommands.

Iterations. You can specify termination criteria using the MXITER, MXLAMB, SSQPCT, and PAREPS subcommands.

Confidence Intervals. You can control the size of the confidence interval using the CINPCT subcommand.

Statistical Output. To display only the final parameter statistics, specify TSET PRINT=BRIEF before ARIMA. To include parameter estimates at each iteration in addition to the default output, specify TSET PRINT=DETAILED.

New Variables. To evaluate model statistics without creating new variables, specify TSET NEWVAR=NONE prior to ARIMA. This could result in faster processing time. To add new variables without erasing the values of Forecasting-generated variables, specify TSET NEWVAR=ALL. This saves all new variables generated during the current session to the active dataset and may require extra processing time.

Forecasting. When used with the PREDICT command, an ARIMA model with no regressor variables can produce forecasts and confidence limits beyond the end of the series (see PREDICT for more information).

Basic Specification

The basic specification is the dependent series name. To estimate an ARIMA model, the MODEL subcommand and/or separate parameter-order subcommands (or the APPLY subcommand) must also be specified. Otherwise, only the constant will be estimated.

- ARIMA estimates the parameter values of a model using the parameter specifications on the MODEL subcommand and/or the separate parameter-order subcommands P, D, Q, SP, SD, and SQ.
- A 95% confidence interval is used unless it is changed by a TSET CIN command prior to the ARIMA procedure.
- Unless the default on TSET NEWVAR is changed prior to ARIMA, five variables are automatically created, labeled, and added to the active dataset: fitted values (*FIT#1*), residuals (*ERR#1*), lower confidence limits (*LCL#1*), upper confidence limits (*UCL#1*), and standard errors of prediction (*SEP#1*).
- By default, ARIMA will iterate up to a maximum of 10 unless one of three termination criteria is met: the change in all parameters is less than the TSET CNVERGE value (the default value is 0.001); the sum-of-squares percentage change is less than 0.001%; or the Marquardt constant exceeds 10^9 (1.0E9).
- At each iteration, the Marquardt constant and adjusted sum of squares are displayed. For the final estimates, the displayed results include the parameter estimates, standard errors, *t* ratios, estimate of residual variance, standard error of the estimate, log likelihood, Akaike's information criterion (AIC) [13], Schwartz's Bayesian criterion (SBC) [14], and covariance and correlation matrices.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- VARIABLES can be specified only once.
- Other subcommands can be specified more than once, but only the last specification of each one is executed.
- The CONSTANT, NOCONSTANT, NOLOG, LN, and LOG specifications are optional keywords on the MODEL subcommand and are not independent subcommands.

Operations

- If differencing is specified in models with regressors, both the dependent series and the regressors are differenced. To difference only the dependent series, use the DIFF or SDIFF function on CREATE to create a new series (see CREATE for more information).
- When ARIMA is used with the PREDICT command to forecast values beyond the end of the series, the original series and residual variable are assigned the system-missing value after the last case in the original series.
- The USE and PREDICT ranges cannot be exactly the same; at least one case from the USE period must precede the PREDICT period. (See USE and PREDICT for more information.)

[13] Akaike, H. 1974. A new look at the statistical model identification. *IEEE Transaction on Automatic Control,* AC–19, 716-723.

[14] Schwartz, G. 1978. Estimating the dimensions of a model. *Annals of Statistics,* 6, 461-464.

- If a LOG or LN transformation is specified, the residual (error) series is reported in the logged metric; it is not transformed back to the original metric. This is so the proper diagnostic checks can be done on the residuals. However, the predicted (forecast) values *are* transformed back to the original metric. Thus, the observed value minus the predicted value will not equal the residual value. A new residual variable in the original metric can be computed by subtracting the predicted value from the observed value.
- Specifications on the P, D, Q, SP, SD, and SQ subcommands override specifications on the MODEL subcommand.
- For ARIMA models with a fixed regressor, the number of forecasts and confidence intervals produced cannot exceed the number of observations for the regressor (independent) variable. Regressor series cannot be extended.
- Models of series with imbedded missing observations can take longer to estimate.

Limitations

- Maximum 1 VARIABLES subcommand.
- Maximum 1 dependent series. There is no limit on the number of independent series.
- Maximum 1 model specification.

VARIABLES Subcommand

VARIABLES specifies the dependent series and regressors, if any, and is the only required subcommand. The actual keyword VARIABLES can be omitted.

- The dependent series is specified first, followed by the keyword WITH and the regressors (independent series).

MODEL Subcommand

MODEL specifies the ARIMA model, period length, whether a constant term should be included in the model, and whether the series should be log transformed.

- The model parameters are listed using the traditional ARIMA (p,d,q) (sp,sd,sq) syntax.
- Nonseasonal parameters are specified with the appropriate *p, d,* and *q* values separated by commas and enclosed in parentheses.
- The value *p* is a positive integer indicating the order of nonseasonal autoregressive parameters, *d* is a positive integer indicating the degree of nonseasonal differencing, and *q* is a positive integer indicating the nonseasonal moving-average order.
- Seasonal parameters are specified after the nonseasonal parameters with the appropriate *sp*, *sd*, and *sq* values. They are also separated by commas and enclosed in parentheses.
- The value *sp* is a positive integer indicating the order of seasonal autoregressive parameters, *sd* is a positive integer indicating the degree of seasonal differencing, and *sq* is a positive integer indicating the seasonal moving-average order.
- After the seasonal model parameters, a positive integer can be specified to indicate the length of a seasonal period.
- If the period length is not specified, the periodicity established on TSET PERIOD is in effect. If TSET PERIOD is not specified, the periodicity established on the DATE command is used. If periodicity was not established anywhere and a seasonal model is specified, the ARIMA procedure is not executed.

The following optional keywords can be specified on MODEL:

CONSTANT. *Include a constant in the model.* This is the default unless the default setting on the TSET command is changed prior to the ARIMA procedure.

NOCONSTANT . *Do not include a constant.*

NOLOG. *Do not log transform the series.* This is the default.

LG10. *Log transform the series before estimation using the base 10 logarithm.* The keyword LOG is an alias for LG10.

LN. *Log transform the series before estimation using the natural logarithm (base* e).

- Keywords can be specified anywhere on the MODEL subcommand.
- CONSTANT and NOCONSTANT are mutually exclusive. If both are specified, only the last one is executed.
- LG10 (LOG), LN, and NOLOG are mutually exclusive. If more than one is specified, only the last one is executed.
- CONSTANT and NOLOG are generally used as part of an APPLY subcommand to turn off previous NOCONSTANT, LG10, or LN specifications

Example

```
ARIMA SALES WITH INTERVEN
  /MODEL=(1,1,1)(1,1,1) 12 NOCONSTANT LN.
```

- This example specifies a model with a first-order nonseasonal autoregressive parameter, one degree of nonseasonal differencing, a first-order nonseasonal moving average, a first-order seasonal autoregressive parameter, one degree of seasonal differencing, and a first-order seasonal moving average.
- The 12 indicates that the length of the period for *SALES* is 12.
- The keywords NOCONSTANT and LN indicate that a constant is not included in the model and that the series is log transformed using the natural logarithm before estimation.

Parameter-Order Subcommands

P, D, Q, SP, SD, and SQ can be used as additions or alternatives to the MODEL subcommand to specify particular lags in the model and degrees of differencing for fitting single or nonsequential parameters. These subcommands are also useful for specifying a constrained model. The subcommands represent the following parameters:

P. *Autoregressive order.*

D. *Order of differencing.*

Q. *Moving-average order.*

SP. *Seasonal autoregressive order.*

SD. *Order of seasonal differencing.*

SQ. *Seasonal moving-average order.*

- The specification on P, Q, SP, or SQ indicates which lags are to be fit and can be a single positive integer or a list of values in parentheses.
- A single value *n* denotes lags 1 through *n*.
- A single value *in parentheses,* for example *(n)*, indicates that only lag *n* should be fit.
- A list of values in parentheses *(i, j, k)* denotes lags *i*, *j*, and *k* only.
- You can specify as many values in parentheses as you want.
- D and SD indicate the degrees of differencing and can be specified only as single values, not value lists.
- Specifications on P, D, Q, SP, SD, and SQ override specifications for the corresponding parameters on the MODEL subcommand.

Example

```
ARIMA SALES
  /P=2
  /D=1.
ARIMA INCOME
  /MODEL=LOG NOCONSTANT
  /P=(2).
ARIMA VAR01
```

```
  /MODEL=(1,1,4)(1,1,4)
  /Q=(2,4)
  /SQ=(2,4).
ARIMA VAR02
  /MODEL=(1,1,0)(1,1,0)
  /Q=(2,4)
  /SQ=(2,4).
```

- The first command fits a model with autoregressive parameters at lags 1 and 2 (P=2) and one degree of differencing (D=1) for the series *SALES*. This command is equivalent to:

```
ARIMA SALES
  /MODEL=(2,1,0).
```

- In the second command, the series *INCOME* is log transformed and no constant term is estimated. There is one autoregressive parameter at lag 2, as indicated by P=(2).
- The third command specifies a model with one autoregressive parameter, one degree of differencing, moving-average parameters at lags 2 and 4, one seasonal autoregressive parameter, one degree of seasonal differencing, and seasonal moving-average parameters at lags 2 and 4. The 4's in the MODEL subcommand for moving average and seasonal moving average are ignored because of the Q and SQ subcommands.
- The last command specifies the same model as the previous command. Even though the MODEL command specifies no nonseasonal or seasonal moving-average parameters, these parameters are estimated at lags 2 and 4 because of the Q and SQ specifications.

Initial Value Subcommands

AR, MA, SAR, SMA, REG, and CON specify initial values for parameters. These subcommands refer to the following parameters:

AR. *Autoregressive parameter values.*

MA. *Moving-average parameter values.*

SAR. *Seasonal autoregressive parameter values.*

SMA. *Seasonal moving-average parameter values.*

REG. *Fixed regressor parameter values.*

CON. *Constant value.*

- Each subcommand specifies a value or value list indicating the initial values to be used in estimating the parameters.
- CON can be specified only as a single value, not a value list.
- Values are matched to parameters in sequential order. That is, the first value is used as the initial value for the first parameter of that type, the second value is used as the initial value for the second parameter of that type, and so on.
- Specify only the subcommands for which you can supply a complete list of initial values (one for every lag to be fit for that parameter type).
- If you specify an inappropriate initial value for AR, MA, SAR, or SMA, ARIMA will reset the value and issue a message.
- If MXITER=0, these subcommands specify final parameter values to use for forecasting.

Example

```
ARIMA VARY
  /MODEL (1,0,2)
  /AR=0.5
  /MA=0.8, -0.3.
ARIMA VARY
  /MODEL (1,0,2)
  /AR=0.5.
```

- The first command specifies initial estimation values for the autoregressive term and for the two moving-average terms.

- The second command specifies the initial estimation value for the autoregressive term only. The moving-average initial values are estimated by ARIMA.

Termination Criteria Subcommands

ARIMA will continue to iterate until one of four termination criteria is met. The values of these criteria can be changed using any of the following subcommands followed by the new value:

MXITER. *Maximum number of iterations.* The value specified can be any integer equal to or greater than 0. If MXITER equals 0, initial parameter values become final estimates to be used in forecasting. The default value is 10.

PAREPS. *Parameter change tolerance.* The value specified can be any real number greater than 0. A change in all of the parameters by less than this amount causes termination. The default is the value set on TSET CNVERGE. If TSET CNVERGE is not specified, the default is 0.001. A value specified on PAREPS overrides the value set on TSET CNVERGE.

SSQPCT. *Sum of squares percentage.* The value specified can be a real number greater than 0 and less than or equal to 100. A relative change in the adjusted sum of squares by less than this amount causes termination. The default value is 0.001%.

MXLAMB. *Maximum lambda.* The value specified can be any integer. If the Marquardt constant exceeds this value, estimation is terminated. The default value is 1,000,000,000 (10^9).

CINPCT Subcommand

CINPCT controls the size of the confidence interval.

- The specification on CINPCT can be any real number greater than 0 and less than 100.
- The default is the value specified on TSET CIN. If TSET CIN is not specified, the default is 95.
- CINPCT overrides the value set on the TSET CIN command.

APPLY Subcommand

APPLY allows you to use a previously defined ARIMA model without having to repeat the specifications.

- The specifications on APPLY can include the name of a previous model in quotes and one of three keywords. All of these specifications are optional.
- If a model name is not specified, the model specified on the previous ARIMA command is used.
- To change one or more of the specifications of the model, specify the subcommands of only those portions you want to change after the subcommand APPLY.
- If no series are specified on the ARIMA command, the series that were originally specified with the model being reapplied are used.
- To change the series used with the model, enter new series names before or after the APPLY subcommand. If a series name is specified before APPLY, the slash before the subcommand is required.
- APPLY with the keyword FIT sets MXITER to 0. If you apply a model that used FIT and want to obtain estimates, you will need to respecify MXITER.

The keywords available for APPLY with ARIMA are:

SPECIFICATIONS . *Use only the specifications from the original model.* ARIMA should create the initial values. This is the default.

INITIAL. *Use the original model's final estimates as initial values for estimation.*

FIT. *No estimation.* Estimates from the original model should be applied directly.

Example

```
ARIMA VAR1
  /MODEL=(0,1,1)(0,1,1) 12 LOG NOCONSTANT.
ARIMA APPLY
  /MODEL=CONSTANT.
ARIMA VAR2
  /APPLY INITIAL.
ARIMA VAR2
  /APPLY FIT.
```

- The first command specifies a model with one degree of differencing, one moving-average term, one degree of seasonal differencing, and one seasonal moving-average term. The length of the period is 12. A base 10 log of the series is taken before estimation and no constant is estimated. This model is assigned the name *MOD_1*.
- The second command applies the same model to the same series, but this time estimates a constant term. Everything else stays the same. This model is assigned the name *MOD_2*.
- The third command uses the same model as the previous command (*MOD_2*) but applies it to series *VAR2*. Keyword INITIAL specifies that the final estimates of *MOD_2* are to be used as the initial values for estimation.
- The last command uses the same model but this time specifies no estimation. Instead, the values from the previous model are applied directly.

FORECAST Subcommand

The FORECAST subcommand specifies the forecasting method to use. Available methods are:

EXACT. *Unconditional least squares*. The forecasts are unconditional least squares forecasts. They are also called finite memory forecasts. This is the default.

CLS. *Conditional least squares using model constraint for initialization.* The forecasts are computed by assuming that the unobserved past errors are zero and the unobserved past values of the response series are equal to the mean.

AUTOINIT. *Conditional least squares using the beginning series values for initialization.* The beginning series values are used to initialize the recursive conditional least squares forecasting algorithm.

References

Akaike, H. 1974. A new look at the statistical model identification. *IEEE Transaction on Automatic Control,* AC–19, 716-723.

Box, G. E. P., and G. C. Tiao. 1975. Intervention analysis with applications to economic and environmental problems. *Journal of the American Statistical Association,* 70:3, 70-79.

Cryer, J. D. 1986. *Time series analysis*. Boston, Mass.: Duxbury Press.

Harvey, A. C. 1981. *The econometric analysis of time series*. Oxford: Philip Allan.

Harvey, A. C. 1981. *Time series models*. Oxford: Phillip Allan.

Kohn, R., and C. Ansley. 1985. Efficient estimation and prediction in time series regression models. *Biometrika,* 72:3, 694-697.

Kohn, R., and C. Ansley. 1986. Estimation, prediction, and interpolation for ARIMA models with missing data. *Journal of the American Statistical Association,* 81, 751-761.

McCleary, R., and R. A. Hay. 1980. *Applied time series analysis for the social sciences*. Beverly Hills, Calif.: Sage Publications.

Melard, G. 1984. A fast algorithm for the exact likelihood of autoregressive-moving average models. *Applied Statistics,* 33:1, 104-119.

Schwartz, G. 1978. Estimating the dimensions of a model. *Annals of Statistics,* 6, 461-464.

AUTORECODE

```
AUTORECODE VARIABLES=varlist
 /INTO new varlist
 [/BLANK={VALID**}
         {MISSING}
 [/GROUP]
 [/APPLY TEMPLATE='filespec']
 [/SAVE TEMPLATE='filespec']
 [/DESCENDING]
 [/PRINT]
```

**Default if the subcommand omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 13.0

- BLANK subcommand introduced.
- GROUP subcommand introduced.
- APPLY TEMPLATE and SAVE TEMPLATE subcommands introduced.

Example

```
AUTORECODE VARIABLES=Company /INTO Rcompany.
```

Overview

AUTORECODE recodes the values of string and numeric variables to consecutive integers and puts the recoded values into a new variable called a **target variable**. The value labels or values of the original variable are used as value labels for the target variable. AUTORECODE is useful for creating numeric independent (grouping) variables from string variables for procedures such as ONEWAY and DISCRIMINANT. AUTORECODE can also recode the values of factor variables to consecutive integers, which may be required by some procedures and which reduces the amount of workspace needed by some statistical procedures.

Basic Specification

The basic specification is VARIABLES and INTO. VARIABLES specifies the variables to be recoded. INTO provides names for the target variables that store the new values. VARIABLES and INTO must name or imply the same number of variables.

Subcommand Order

- VARIABLES must be specified first.
- INTO must immediately follow VARIABLES.
- All other subcommands can be specified in any order.

Syntax Rules

- A variable cannot be recoded into itself. More generally, target variable names cannot duplicate any variable names already in the working file.
- If the GROUP or APPLY TEMPLATE subcommand is specified, all variables on the VARIABLES subcommand must be the same type (numeric or string).

- If APPLY TEMPLATE is specified, all variables on the VARIABLES subcommand must be the same type (numeric or string) as the type defined in the template.
- File specifications on the APPLY TEMPLATE and SAVE TEMPLATE subcommands follow the normal conventions for file specifications. Enclosing file specifications in quotation marks is recommended.

Operations

- The values of each variable to be recoded are sorted and then assigned numeric values. By default, the values are assigned in ascending order: 1 is assigned to the lowest nonmissing value of the original variable; 2, to the second-lowest nonmissing value; and so on, for each value of the original variable.
- Values of the original variables are unchanged.
- Missing values are recoded into values higher than any nonmissing values, with their order preserved. For example, if the original variable has 10 nonmissing values, the first missing value is recoded as 11 and retains its user-missing status. System-missing values remain system-missing. (See the GROUP, APPLY TEMPLATE, and SAVE TEMPLATE subcommands for additional rules for user-missing values.)
- AUTORECODE does not sort the cases in the working file. As a result, the consecutive numbers assigned to the target variables may not be in order in the file.
- Target variables are assigned the same variable labels as the original source variables. To change the variable labels, use the VARIABLE LABELS command after AUTORECODE.
- Value labels are automatically generated for each value of the target variables. If the original value had a label, that label is used for the corresponding new value. If the original value did not have a label, the old value itself is used as the value label for the new value. The defined print format of the old value is used to create the new value label.
- AUTORECODE ignores SPLIT FILE specifications. However, any SELECT IF specifications are in effect for AUTORECODE.

Example

```
DATA LIST / COMPANY 1-21 (A) SALES 24-28.
BEGIN DATA
CATFOOD JOY            10000
OLD FASHIONED CATFOOD  11200
. . .
PRIME CATFOOD          10900
CHOICE CATFOOD         14600
END DATA.

AUTORECODE VARIABLES=COMPANY /INTO=RCOMPANY /PRINT.

TABLES TABLE = SALES BY RCOMPANY
  /TTITLE='CATFOOD SALES BY COMPANY'.
```

- AUTORECODE recodes *COMPANY* into a numeric variable *RCOMPANY*. Values of *RCOMPANY* are consecutive integers beginning with 1 and ending with the number of different values entered for *COMPANY*. The values of *COMPANY* are used as value labels for *RCOMPANY*'s numeric values. The *PRINT* subcommand displays a table of the original and recoded values.

VARIABLES Subcommand

VARIABLES specifies the variables to be recoded. VARIABLES is required and must be specified first. The actual keyword VARIABLES is optional.

- Values from the specified variables are recoded and stored in the target variables listed on INTO. Values of the original variables are unchanged.

INTO Subcommand

INTO provides names for the target variables that store the new values. INTO is required and must immediately follow VARIABLES.

- The number of target variables named or implied on INTO must equal the number of source variables listed on VARIABLES.

Example

```
AUTORECODE VARIABLES=V1 V2 V3 /INTO=NEWV1 TO NEWV3 /PRINT.
```

- AUTORECODE stores the recoded values of *V1*, *V2*, and *V3* into target variables named *NEWV1*, *NEWV2*, and *NEWV3*.

BLANK Subcommand

The BLANK subcommand specifies how to autorecode blank string values.

- BLANK is followed by an equals sign (=) and the keyword VALID or MISSING.
- The BLANK subcommand applies only to string variables (both short and long strings). System-missing numeric values remain system-missing in the new, autorecoded variable(s).
- The BLANK subcommand has no effect if there are no string variables specified on the VARIABLES subcommand.

VALID . *Blank string values are treated as valid, nonmissing values and are autorecoded into nonmissing values.* This is the default.

MISSING . *Blank string values are autorecoded into a user-missing value higher than the highest nonmissing value*.

Example

```
DATA LIST /stringVar (A1).
BEGIN DATA
a
b

c
d
END DATA.
AUTORECODE
  VARIABLES=stringVar  /INTO NumericVar
  /BLANK=MISSING.
```

- The values *a*, *b*, *c*, and *d* are autorecoded into the numeric values 1 through 4.
- The blank value is autorecoded to 5, and 5 is defined as user-missing.

GROUP Subcommand

The subcommand GROUP allows you to specify that a single autorecoding scheme should be generated for all the specified variables, yielding consistent coding for all of the variables.

- The GROUP subcommand has no additional keywords or specifications. By default, variables are not grouped for autorecoding.
- All variables must be the same type (numeric or string).
- All observed values for all specified variables are used to create a sorted order of values to recode into sequential integers.
- String variables can be of any length and can be of unequal length.
- User-missing values for the target variables are based on the first variable in the original variable list with defined user-missing values. All other values from other original variables, except for system-missing, are treated as valid.
- If only one variable is specified on the VARIABLES subcommand, the GROUP subcommand is ignored.
- If GROUP and APPLY TEMPLATE are used on the same AUTORECODE command, value mappings from the template are applied first. All remaining values are recoded into values higher than the last value in the template, with user-missing values (based on the first variable in the list with defined user-missing

values) recoded into values higher than the last valid value. See the APPLY TEMPLATE subcommand for more information.

Example

```
DATA LIST FREE /var1 (a1) var2 (a1).
BEGIN DATA
a d
b e
c f
END DATA.
MISSING VALUES var1 ("c") var2 ("f").
AUTORECODE VARIABLES=var1 var2
  /INTO newvar1 newvar2
  /GROUP.
```

- A single autorecoding scheme is created and applied to both new variables.
- The user-missing value "c" from *var1* is autorecoded into a user-missing value.
- The user-missing value "f" from *var2* is autorecoded into a valid value.

Table 16. Original and recoded values

Original value	Autorecoded value
a	1
b	2
c	6 (user-missing)
d	3
e	4
f	5

SAVE TEMPLATE Subcommand

The SAVE TEMPLATE subcommand allows you to save the autorecode scheme used by the current AUTORECODE command to an external template file, which you can then use when autorecoding other variables using the APPLY TEMPLATE subcommand.

- SAVE TEMPLATE is followed by an equals sign (=) and a quoted file specification. The default file extension for autorecode templates is *.sat*.
- The template contains information that maps the original nonmissing values to the recoded values.
- Only information for nonmissing values is saved in the template. User-missing value information is not retained.
- If more than one variable is specified on the VARIABLES subcommand, the first variable specified is used for the template, unless GROUP or APPLY TEMPLATE is also specified, in which case a common autorecoding scheme for all variables is saved in the template.

Example

```
DATA LIST FREE /var1 (a1) var2 (a1).
BEGIN DATA
a d
b e
c f
END DATA.
MISSING VALUES var1 ("c") var2 ("f").
AUTORECODE VARIABLES=var1 var2
  /INTO newvar1 newvar2
  /SAVE TEMPLATE='/temp/var1_template.sat'.
```

- The saved template contains an autorecode scheme that maps the string values of "a" and "b" from *var1* to the numeric values 1 and 2, respectively.
- The template contains no information for the value of "c" for *var1* because it is defined as user-missing.

- The template contains no information for values associated with *var2* because the GROUP subcommand was not specified.

Template File Format

An autorecode template file is actually a data file in IBM SPSS Statistics format that contains two variables: *Source_* contains the original, unrecoded valid values, and *Target_* contains the corresponding recoded values. Together these two variables provide a mapping of original and recoded values.

You can therefore, theoretically, build your own custom template files, or simply include the two mapping variables in an existing data file--but this type of use has not been tested.

APPLY TEMPLATE Subcommand

The APPLY TEMPLATE subcommand allows you to apply a previously saved autorecode template to the variables in the current AUTORECODE command, appending any additional values found in the variables to the end of the scheme, preserving the relationship between the original and autorecode values stored in the saved scheme.

- APPLY TEMPLATE is followed by an equals sign (=) and a quoted file specification.
- All variables on the VARIABLES subcommand must be the same type (numeric or string), and that type must match the type defined in the template.
- Templates do not contain any information on user-missing values. User-missing values for the target variables are based on the first variable in the original variable list with defined user-missing values. All other values from other original variables, except for system-missing, are treated as valid.
- Value mappings from the template are applied first. All remaining values are recoded into values higher than the last value in the template, with user-missing values (based on the first variable in the list with defined user-missing values) recoded into values higher than the last valid value.
- If multiple variables are specified on the VARIABLES subcommand, APPLY TEMPLATE generates a grouped recoding scheme, with or without an explicit GROUP subcommand.

Example

```
DATA LIST FREE /var1 (a1).
BEGIN DATA
a b d
END DATA.
AUTORECODE VARIABLES=var1
  /INTO newvar1
  /SAVE TEMPLATE='/temp/var1_template.sat'.
DATA LIST FREE /var2 (a1).
BEGIN DATA
a b c
END DATA.
AUTORECODE VARIABLES=var2
  /INTO newvar2
  /APPLY TEMPLATE='/temp/var1_template.sat'.
```

- The template file *var1_template.sat* maps the string values a, b, and d to the numeric values 1, 2, and 3, respectively.
- When the template is applied to the variable *var2* with the string values a, b, and c, the autorecoded values for *newvar2* are 1, 2, and 4, respectively. The string value "c" is autorecoded to 4 because the template maps 3 to the string value "d".
- The data dictionary contains defined value labels for all four values--the three from the template and the one new value read from the file.

Table 17. Defined value labels for newvar2

Value	Label
1	a
2	b

Table 17. Defined value labels for newvar2 (continued)	
Value	**Label**
3	d
4	c

Interaction between APPLY TEMPLATE and SAVE TEMPLATE

- If APPLY TEMPLATE and SAVE TEMPLATE are both used in the same AUTORECODE command, APPLY TEMPLATE is always processed first, regardless of subcommand order, and the autorecode scheme saved by SAVE TEMPLATE is the union of the original template plus any appended value definitions.
- APPLY TEMPLATE and SAVE TEMPLATE can specify the same file, resulting in the template being updated to include any newly appended value definitions.

Example

```
AUTORECODE VARIABLES=products
 /INTO productCodes
 /APPLY TEMPLATE='/mydir/product_codes.sat'
 /SAVE TEMPLATE='/mydir/product_codes.sat.
```

- The autorecode scheme in the template file is applied for autorecoding *products* into *productCodes*.
- Any data values for *products* not defined in the template are autorecoded into values higher than the highest value in the original template.
- Any user-missing values for *products* are autorecoded into values higher than the highest nonmissing autorecoded value.
- The template saved is the autorecode scheme used to autorecode *product*--the original autorecode scheme plus any additional values in *product* that were appended to the scheme.

PRINT Subcommand

PRINT displays a correspondence table of the original values of the source variables and the new values of the target variables. The new value labels are also displayed.

- The only specification is the keyword PRINT. There are no additional specifications.

DESCENDING Subcommand

By default, values for the source variable are recoded in ascending order (from lowest to highest). DESCENDING assigns the values to new variables in descending order (from highest to lowest). The largest value is assigned 1, the second-largest, 2, and so on.

- The only specification is the keyword DESCENDING. There are no additional specifications.

BAYES ANOVA

BAYES ANOVA is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
BAYES ANOVA dependent varname BY factor
     /CRITERIA
          CILEVEL = {95**} {value}
          TOL = {0.000001**} {value}
          MAXITER = {500**} {integer}
     /DESIGN
          REGWGT = {varname}††
     /INFERENCE
          ANALYSIS = {POSTERIOR**} {BAYESFACTOR} {BOTH}
     /ESTBF
          COMPUTATION = {JZS**} {ZELLNER(value)} {HYPER(3**{value})} {ROUDER(1** {value})}
     /PRIOR
          TYPE = {REFERENCE**} {CONJUGATE}
          SHAPEPARAM = {value}
          SCALEPARAM = {value}
          MEANVECTOR = {values}
          VMATRIX = {IDENTITY**} {lower_triangular_matrix}
     /PLOT
          MEANS = {group_list}
          ERRORVAR = {FALSE**} {TRUE}
```

** Default if the subcommand or keyword is omitted.

†† Only one regression weight variable is allowed.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 25.0

- Command introduced

Overview

BAYES ANOVA invokes the Bayesian One-Way ANOVA procedure. The One-Way ANOVA procedure produces a one-way analysis of variance for a quantitative dependent variable by a single factor (independent) variable. Analysis of variance is used to test the hypothesis that several means are equal. IBM SPSS Statistics supports Bayes-factors, conjugate priors and noninformative priors.

Options

Bayesian analysis method
You can specify POSTERIOR, BAYESFACTOR, or BOTH.

Credible interval percentage
You can specify the significance level for computing credible intervals.

Numerical method
You can specify the numerical method that is used to estimate the integral.

Prior distribution settings
You can specify REFERENCE and CONJUGATE prior distribution settings.

Bayes factor settings
You can specify the approach that is used to estimate the Bayes factor for the Bayesian One-way ANOVA models.

Output plots
You can control the plots that are output.

Basic specification

- `BAYES ANOVA dependent varname BY factor`: Invokes the Bayesian one-way ANOVA procedure.

Subcommand order

- The subcommands can be named in any order.

Syntax rules

- When a REGWGT value is specified, only one regression weight variable is allowed.
- The MEANS group list is a subset of the categories of the factor variable, so the format should be consistent with the factor's data type and actual values.

Limitations

- Maximum set $\in$ `[0, 50]` of plots in the output.
- The value $\in$ `(-1,1)` applies only to the FRACTION option.

CRITERIA Subcommand

CRITERIA controls the statistical criteria used in building the Bayesian ANOVA models. The default criteria are noted in the description of each keyword below.

CILEVEL = value. *Specify the significance level for computing credible intervals.* Must be single double value $\in$ `(0, 100)` with 95 as the default setting.

TOL = value. *Specify the tolerance value for the numerical methods.* `0.000001` is the default setting. Only in effect when `METHOD = AGL`. Must be one value $\in (0, 10^{-3}]$.

MAXITER = integer. *Specify the maximum number of iterations the AGL method can go through in its computations.* The integer `2000` is the default setting. You must specify a positive integer. Valid when `METHOD = AGL`.

DESIGN Subcommand

DESIGN specifies the model design for the analysis.

REGWGT = varname. *An optional parameter to specify the regression weights.* Only one variable is allowed.

INFERENCE Subcommand

INFERENCE controls which Bayesian analysis method is used to make statistical inference.

Variables = varlist.*A required parameter that specifies the variables users would like to use in the procedure.* Each variable in the list is analyzed.

Analysis. *Specifies the method that is used to make statistical inference.* Only one value is allowed. POSTERIOR distribution characterization is the default setting. When `ANALYSIS = BAYESFACTOR`, only the Bayes-factor estimation procedure is invoked. When `ANALYSIS = BOTH`, both procedures are used.

ESTBF Subcommand

ESTBF specifies the approach that is used to estimate Bayes factors.

COMPUTATION = value. *Specifies the approach to estimate Bayes factors.* Four different keywords are offered. Siow's approach is the default setting.

- JZS: Invokes the Zellner-Siow's approach where no additional parameters are to specified. This is the default setting.

- ZELLNER: Invokes the Zellner's approach where users have to specify a single fixed value > 0. There is no default setting.
- HYPER: Invokes the hyper-g approach where users have to specify a single fixed value > 2. The default setting is 3.
- ROUDER: Invokes the Rouder's approach where users have to specify a single fixed value > 0. The default setting is 1.

PRIOR Subcommand

PRIOR specifies the prior distribution settings for the regression parameters and the variance of the errors.

TYPE. *Specify the prior distribution for the regression parameters and the variance of the errors.*

SHAPEPARAM = value. *Specify the shape parameter a_0 for Inverse-Gamma distribution.*

SCALEPARAM = value. *Specify the scale parameter b_0 for Inverse-Gamma distribution.*

MEANVECTOR = values. *Specify the mean vector β_0 for the group means.*

VMATRIX. *Specifies V_0 the values in the lower triangle in the variance-covariance matrix for the multivariate normal prior.*

PLOT Subcommand

PLOT controls the plots to be output.

MEANS. *Specifies the subgroups to be plotted.* Plots the likelihood, prior, and posterior for the means of the specified groups. The group list is a subset of the categories of the factor variable, so the format should be consistent with the factor's data type and actual values.

ERRORVAR. *Specifies whether or not the variance of errors is plotted.* FALSE is the default setting.

BAYES CORRELATION

BAYES CORRELATION is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
BAYES CORRELATION
     /MISSING SCOPE = {PAIRWISE**} {LISTWISE}
     /CRITERIA
          CILEVEL = {95**} {value}
          SEED = {RANDOM**} {2000000} {integer}
          TOL = {0.0001**} {value}
          MAXITER = {2000**} {integer}
          MCSAMPLES = {10000**} {integer}
          POSTSAMPLES = {10000**} {integer}++
     /INFERENCE
          VARIABLES = {varlist}
          ANALYSIS = {POSTERIOR**} {BAYESFACTOR} {BOTH}
          CVALUE = {0**} {value}@@
          MAXPLOTS = {10**}{integer}##
     /ESTBF
          TYPE = {JZS**} {FRACTION(0.5** {value})}^^
          NULLVALUE = {0**} {value}$$
```

** Default if the subcommand or keyword is omitted.

++ Draw samples for posterior distribution.

@@ Parameter for the prior. Any real number is allowed.

Maximum set $\in$ `[0,50]` of plots in the output.

^^ Value $\in$ `(0,1)`

$$ Value $\in$ `(-1,1)`. Only applies to the FRACTION option

Release History

Release 25.0

- Command introduced

Overview

`BAYES CORRELATION` invokes the Bayesian pairwise correlation inference procedure. Pearson correlation coefficient measures the linear relation between two scale variables jointly following a bivariate normal distribution. The conventional statistical inference about the correlation coefficient has been broadly discussed, and its practice has long been offered in IBM SPSS Statistics. The design of the Bayesian inference about Pearson correlation coefficient allows users to draw Bayesian inference by estimating Bayes factors and characterizing posterior distributions.

Options

Bayesian analysis method
You can specify `POSTERIOR`, `BAYESFACTOR`, or `BOTH`.

Maximum number of plots
You can specify the maximum number of plots to see in the output. The plots are generated in order from the first variable versus the remaining variables, then the second variable versus the remaining variables, and so on.

Credible interval percentage
You can specify the significance level for computing credible intervals.

Missing values
You can specify the method in which to control missing values (exclude cases pairwise or listwise).

Numerical method
You can specify the numerical method that is used to estimate the integral.

Prior distribution settings
You can specify REFERENCE and CONJUGATE prior distribution settings.

Bayes factor settings
You can specify the approach that is used to estimate the Bayes factor for the Bayesian Pairwise Correlation models.

Basic specification

- POSTSAMPLES draws samples for posterior distribution.
- Any real number is allowed for CVALUE.

Subcommand order

- The subcommands can be named in any order.

Syntax rules

- When more than two **VARIABLES** are specified, all possible pairs are analyzed.
- For **MAXPLOTS**, the plots are generated in the order of the first variable versus all the rest after it, then the second versus all the rest after it, and so on.
- **NULLVALUE** take affect when **TYPE = FRACTION**.

Limitations

- **NULLVALUE** only applies to the FRACTION option.

Example

```
BAYES CORRELATION
     /MISSING SCOPE = PAIRWISE
     /CRITERIA CILEVEL = 95
     /INFERENCE VARIABLES = salary salbegin jobtime ANALYSIS = BAYESFACTOR
     /ESTBF TYPE = JZS .
```

- **MISSING** specifies how missing values are handled. The example specifies that missing values **SCOPE** is controlled **PAIRWISE**.
- **CRITERIA** specifies that the significance level for computing credible intervals (**CILEVEL**) is set to the default value of 95.
- **INFERENCE** specifies the distribution and variables for the analysis. The example uses the **VARIABLES salary** and **salbegin**, and uses estimate Bayes factor as the **ANALYSIS** method.
- **ESTBF** specifies the method used to estimate the Bayes factor. The example uses the default **JZS** setting, which computes the JZS Bayes factor.

MISSING Subcommand

MISSING specifies the method in which to control missing values. Missing values can be controlled **PAIRWISE** or **LISTWISE**.

Example

```
BAYES CORRELATION
     /MISSING SCOPE = PAIRWISE
     /CRITERIA CILEVEL = 95
     /INFERENCE VARIABLES = salary salbegin jobtime ANALYSIS = BAYESFACTOR
     /ESTBF TYPE = JZS .
```

- The example specifies that missing values are controlled Pairwise.

CRITERIA Subcommand

CRITERIA controls the statistical criteria used in building the Bayesian Pearson Correlation models. The default criteria are noted in the description of each keyword below.

CILEVEL = value. *Specify the significance level for computing credible intervals.* Must be a single double value $\in (0,100)$. The default level is 95%

SEED = integer. *Specify a random seed set for the MCA method, with a random seed assigned by default.* Only in effect when **METHOD = MCA**. You can set a seed by specifying a positive integer $\in [1,2147483647]$.

TOL = value. *Specify the tolerance value for the numerical methods.* 0.000001 is the default setting. Only in effect when METHOD = AGL. Must be one value $\in (0, 10^{-3}]$.

MAXITER = integer. *Specify the maximum number of iterations the AGL method can go through in its computations.* The integer 2000 is the default setting. You must specify a positive integer. Valid when METHOD = AGL.

MCSAMPLES = integer. *Specify the number of points that are sampled for the Monte Carlo approximation.* Only in effect when **METHOD = MCA**. 10,000 is the default setting. The value must be an integer $\in [10^3,10^6]$.

POSTSAMPLES = integer. *Specify the number of samples that are used to draw the desired posterior distribution.* 10,000 is the default setting. Only a single positive integer $\in [10^3,10^6]$ is allowed.

INFERENCE Subcommand

INFERENCE specifies the distribution and variables for the analysis.

VARIABLES = varlist. *Specify the variables to use for pairwise correlation inference.* At least two source variables must be specified. When more than two variables are specified, the analysis is run on all of the specified variables' pairwise combinations.

ANALYSIS. *Specify the Bayesian analysis method that is used to make statistical inference.* Only one value is allowed. Posterior distribution characterization is the default setting. When **ANALYSIS = BAYESFACTOR**, only the Bayes-factor estimation procedure is invoked. When **ANALYSIS = BOTH**, both of the procedures are used.

CVALUE = value. *Specify the value c for the prior* $p(\rho) \propto (1-\rho^2)^c$. 0 is the default setting, which indicates the uniform prior. Any single real number is allowed.

MAXPLOTS = integer. *Specify the maximum set of the plots to see in the output.* A set of plots can contain 3 plots on the same pane. The plots are generated in order from the first variable versus the remaining variables, then the second variable versus the remaining variables, and so on. The defined integer value must be between 0 and 50. By default, 10 sets of plots are output to accommodate five variables. This option is not available when the Estimate Bayes Factor is selected.

ESTBF Subcommand

ESTBF controls the method that is used to estimate the Bayes factor.

TYPE = value. *Specify the method used to estimate the Bayes factor.* **JZS** computes the JZS Bayes factor (the default setting). **FRACTION** computes the fractional Bayes factor, and requires a value $\in (0,1)$ (0.5 is the default setting).

NULLVALUE = value. *Specify the null value of the correlation coefficient.* Must be a single value $\in (0,1)$. The setting takes effect when **TYPE = FRACTION**.

BAYES INDEPENDENT

BAYES INDEPENDENT is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
BAYES INDEPENDENT
     /MISSING SCOPE = {ANALYSIS**} {LISTWISE}
     /CRITERIA
          CILEVEL = {95**} {value}
          TOL = {0.000001**} {value}
          MAXITER = {2000**} {integer}
     /INFERENCE
          VARIABLES = {varlist}
          GROUP = {varname}
          SELECT = {LEVEL(value1/string1 value2/string2)} {MIDPOINT} {CUTPOINT(value)}
          ANALYSIS = {POSTERIOR**} {BAYESFACTOR} {BOTH}
          DATAVAR = {value1 value2}++
     /ESTBF
          COMPUTATION = {ROUDER**} {GONEN(0**{value1} value2)} {HYPER(-0.75**{value})}
     /PRIOR
          EQUALDATAVAR = {FALSE**} {TRUE}
          VARDIST = {DIFFUSE**} {JEFFREYS} {INVCHISQ(value1 value2)}
          MEANDIST = {DIFFUSE**} {NORMAL(mean1 mean2 variance1 variance2)}@@
```

** Default if the subcommand or keyword is omitted.

++ Either null or two variance values.

@@ The order is X and Y.

Release History

Release 25.0

- Command introduced

Overview

BAYES INDEPENDENT invokes the independent-sample inference procedure. The procedure provides options for using a group variable to define two unrelated groups, and make Bayesian inference on the difference of the two group means. You can estimate the Bayes factors by using different approaches, and also characterize the desired posterior distribution either assuming the variances are known or unknown.

Options

Missing values
You can specify the way in which missing values are handled. You can exclude records with missing values on a test-by-test basis, or exclude records with missing values listwise.

Analysis options
You can specify the significance level for computing credible intervals, the tolerance value for numerical methods, and the maximum number of iterations the AGL method can go through in its computations.

Bayesian analysis method
You can specify POSTERIOR, BAYESFACTOR, or BOTH. You can also specify the condition to defined two groups. The groups are defined by two numerical values or two strings.

Bayes factor settings
You can specify the approach that is used to estimate the Bayes factor.

Prior distribution settings
You can specify the prior distribution for the variance parameter.

Subcommand order

- The subcommands can be named in any order.

Syntax rules

- **DATAVAR** must either be null or two variance values.

Example

```
BAYES INDEPENDENT
     /MISSING SCOPE = ANALYSIS
     /CRITERIA CILEVEL = 95
     /INFERENCE DISTRIBUTION = NORMAL VARIABLES = salary ANALYSIS = POSTERIOR
          GROUP = gender SELECT = LEVEL(f m)
     /PRIOR EQUALDATAVAR = FALSE VARDIST = DIFFUSE MEANDIST = DIFFUSE .
```

- **MISSING** specifies how missing values are handled. The example specifies **ANALYSIS** as the missing values **SCOPE**. The example excludes records with missing values on a test-by-test basis. Records with missing values for a field, that is used for a specific test, are omitted from the test.
- **CRITERIA** specifies that the significance level for computing credible intervals (**CILEVEL**) is set to the default value of 95.
- **INFERENCE** specifies the distribution and variables for the analysis. The example uses the **salary** variable, Characterize Posterior Distribution (**POSTERIOR**) as the **ANALYSIS** method, and **gender** as the grouping variable (**GROUP**).
- **PRIOR** specifies the prior distribution. The example uses diffuse priors for both the mean (**MEANDIST**) and variance (**VARDIST**) parameters.

MISSING Subcommand

MISSING specifies the method in which to control missing values. Missing values can be controlled test-by-test (**ANALYSIS**) or **LISTWISE**. The **ANALYSIS** option, the default setting, excludes records with missing values on a test-by-test basis. Records with missing values for a field that is used for a specific test are omitted from the test. The **LISTWISE** option excludes records with missing values listwise. Records with missing values for any field that is named on any subcommand are excluded from all analysis.

Example

```
BAYES INDEPENDENT
     /MISSING SCOPE = ANALYSIS
     /CRITERIA CILEVEL = 95
     /INFERENCE DISTRIBUTION = NORMAL VARIABLES = salary ANALYSIS = POSTERIOR
          GROUP = gender SELECT = LEVEL(f m)
     /PRIOR EQUALDATAVAR = FALSE VARDIST = DIFFUSE MEANDIST = DIFFUSE .
```

- The example excludes records with missing values on a test-by-test basis. Records that include missing values, for a field that is used for a specific test, are omitted from the test.

CRITERIA Subcommand

CRITERIA controls the analysis criteria used in building the Bayesian Independent Samples models. The default criteria are noted in the description of each keyword below.

CILEVEL = value. *Specify the significance level for computing credible intervals.* Must be a single double value $\in (0,100)$. The default level is 95%

TOL = value. *Specify the tolerance value for the numerical methods.* 0.000001 is the default setting. Only in effect when METHOD = AGL. Must be one value $\in (0, 10^{-3}]$.

MAXITER = integer. *Specify the maximum number of iterations the AGL method can go through in its computations.* The integer 2000 is the default setting. You must specify a positive integer. Valid when METHOD = AGL.

INFERENCE Subcommand

INFERENCE specifies the variables for the analysis.

VARIABLES = varlist. *Specify the variables to use for the procedure.* The parameter is required. Each variable in the list is analyzed.

GROUP = varname. *Specify a group variable to define two groups for the unpaired t-test.* This is a required parameter. A single variable name is allowed.

SELECT. *Specify the condition to defined two groups.* This is a required parameter. You can defined the groups by specifying two numerical values or two strings. Note that the values or strings must be those in the variable. Otherwise, an error message is generated to indicate that at least one of the groups is empty. When **varname** is continuous, you can specify either a midpoint or a cut point value to separate the group into < cut point value and ≥ cut point value, or < midpoint value and ≥ midpoint value.

ANALYSIS. *Specify the Bayesian analysis method that is used to make statistical inference.* Only one value is allowed. Posterior distribution characterization is the default setting (**ANALYSIS = POSTERIOR**). When **ANALYSIS = BAYESFACTOR**, only the Bayes-factor estimation procedure is invoked. When **ANALYSIS = BOTH**, both of the procedures are used.

DATAVAR = value1 value2. *Specify the two group variances (if known).* This is an optional parameter. Both specified values must be > 0.

ESTBF Subcommand

ESTBF specifies the approach that is used to estimate Bayes factors.

COMPUTATION = value. *Specifies the approach to estimate Bayes factors.* Three different keywords are offered. Rouder's approach is the default setting.

- ROUDER: Invokes the Rouder's approach where users have to specify a single fixed value > 0. The default setting is 1.
- GONEN: Invokes Gönen's approach where users are required to specify `value1` and `value2`, the latter of which must be > 0.
- HYPER: Invokes the hyper-prior approach. The value must be $\in$`(-1,-0.5]` with `-0.75` as the default setting.

PRIOR Subcommand

PRIOR specifies the prior distribution for two equal variances and for two group means.

EQUALDATAVAR. *Control whether or not the two group variances are assumed to be equal.* By default, it is assumed that group variances are not equal. This parameter is ignored when **DATA** is specified.

VARDIST. *Specify the prior distribution for the two equal variances.* **DIFFUSE** is the default setting. The parameter is valid only if **DATAVAR** is not specified. It is a requirement that `value1, value2 > 0`. This parameter is ignored when **DATA** is specified.

MEANDIST. *Specify the prior distribution for the two group means.* **DIFFUSE** is the default setting. When **NORMAL** is specified, users are required to input four values for μ_x, μ_y, σ^2_x, and σ^2_y. Note that **NORMAL** is supported only when **DATAVAR** is specified. Otherwise, only **DIFFUSE** is allowed. When **DATAVAR** is specified:

- **EQUALDATAVAR**, if specified, has no affect and is ignored.
- **VARDIST**, if specified, has no affect and is ignored.

When **DATAVAR** is not specified:

- **VARDIST** must be specified.

- If **EQUALDATAVAR = TRUE**:
  - If **VARDIST = JEFFREYS** or **INVCHISQ**, and **MEANDIST = NORMAL**, **NORMAL** is ignored, and **MEANDIST = DIFFUSE** is used.
  - If **VARDIST = DIFFUSE**, **EQUALDATAVAR = FALSE** and **MEANDIST = DIFFUSE** are used.
- If **EQUALDATAVAR = FALSE**:
  - Both **DIFFUSE** are used for **VARDIST** and **MEANDIST**. All other specifications are ignored.

BAYES LOGLINEAR

BAYES LOGLINEAR is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
BAYES LOGLINEAR
     /CRITERIA
          CILEVEL = {95**} {value}
          SEED = {RANDOM**} {2000000} {integer}
          TOL = {0.000001**} {value}
          MAXITER = {2000**} {integer}
          POSTSAMPLES = {10000**} {integer}
          ORDER = {ASCENDING**} {DESCENDING}
     /INFERENCE
          ANALYSIS = {POSTERIOR**} {BAYESFACTOR} {BOTH}
     /TABLE
          rowFactorName BY colFactorName @@
     /ESTBF
          [MODEL = {MULTINOMIAL**} {POISSON} {NONPARAMETRIC}]
          [ FIX MARGIN = {TOTAL**} {COLUMN} {ROW}]
          PRIOR_TYPE = {CONJUGATE**} {MIXTURE_DIRICHLET} {INTRINSIC}
          SHAPEPARAMS = {1**} {values}
          SCALEPARAM = {DATA**} {value}##
          LAMBDAS = {1**} {values}$$
     [/PRINT
          [ SUPPRESS_TABLE = {FALSE**} {TRUE}]
          [ CELLS = [OBSERVED**] [ROW] [COLUMN] [EXPECTED] [TOTAL] [ALL]]
          [ STATISTICS = [CHISQ**] [LRT]]]%%
```

** Default if the subcommand or keyword is omitted.

@@ Only two variables are allowed (row by column).

Only for Poisson models with conjugate prior.

$$ Only for nonparametric models.

%% Only for Fisher and Yates 2 by 2 tables.

Release History

Release 25.0

- Command introduced

Overview

BAYES LOGLINEAR invokes the log linear regression procedure. The procedure tests the independence of two factors requires two categorical variables for the construction of a contingency table, and makes Bayesian inference on the row-column association. You can estimate the Bayes factors by assuming different models, and characterize the desired posterior distribution by simulating the simultaneous credible interval for the interaction terms.

Options

Analysis options
You can specify the significance level for computing credible intervals, the tolerance value for numerical methods, the maximum number of iterations the AGL method can go through in its computations, the number of samples used to draw the posterior distribution, and the category display order.

Bayesian analysis method
You can specify POSTERIOR, BAYESFACTOR, or BOTH.

Contingency table
You can construct a contingency table to test the independence of two factors.

Bayes factor settings
You can specify the approach that is used to estimate the Bayes factor.

Output table display
You can specify how the contents are displayed in the output tables.

Subcommand order

- The subcommands can be named in any order.

Syntax rules

- Only two variables are allowed for **rowFactorName BY colFactorName**.

Limitations

- **SCALEPARAM** only applies to Poisson models with conjugate priors.
- **LAMBDAS** applies only to nonparametric models.
- Only two variables are allowed for **rowFactorName BY colFactorName**.

Example

```
BAYES LOGLINEAR
     /CRITERIA ORDER = ASCENDING
     /INFERENCE ANALYSIS = BAYESFACTOR
     /TABLE gender BY minority
     /ESTBF MODEL = MULTINOMIAL FIXMARGIN = TOTAL PRIORTYPE = CONJUGATE
     /PRINT CELLS = OBSERVED EXPECTED STATISTICS = CHISQ .
```

- **CRITERIA** specifies that categories are displayed in ascending order.
- **INFERENCE** specifies the distribution for the analysis. The example uses Estimate Bayes Factor (**BAYESFACTOR**) as the **ANALYSIS** method.
- **TABLE** constructs a contingency table to test the independence of two factors. The example constructs a table using **gender** as the row value and **minority** as the column value.
- **ESTBF** specifies the model that is assumed for the observed data, a parameter that specifies if there are any fixed marginal totals for the contingency table, and the type of the prior distribution that is used when estimating the Bayes factor. The example specifies **MULTINOMIAL** as the model, **TOTAL** (fixed grand total) as the fixed marginal total, and **CONJUGATE** as the prior distribution.
- **PRINT** specifies how the contents display in the output tables. The example uses the **CELLS** parameter to specify that both **OBSERVED** and **EXPECTED** cells counts will display in the output tables. The examples also uses the **STATISTICS** parameter to specify that the Pearson Chi-Square statistic (**CHISQ**) is used for testing of independence.

CRITERIA Subcommand

CRITERIA controls the statistical criteria used in building the Bayesian Log Linear models. The default criteria are noted in the description of each keyword below.

CILEVEL = value. *Specify the significance level for computing credible intervals.* Must be a single double value $\in (0,100)$. The default level is 95%

SEED = integer. *Specify a random seed set for the MCA method, with a random seed assigned by default.* Only in effect when **METHOD = MCA**. You can set a seed by specifying a positive integer $\in [1,2147483647]$.

TOL = value. *Specify the tolerance value for the numerical methods.* 0.000001 is the default setting. Only in effect when METHOD = AGL. Must be one value $\in (0, 10^{-3}]$.

MAXITER = integer. *Specify the maximum number of iterations the AGL method can go through in its computations.* The integer 2000 is the default setting. You must specify a positive integer. Valid when METHOD = AGL.

POSTSAMPLES = integer. *Specify the number of samples that are used to draw the desired posterior distribution.* 10,000 is the default setting. Only a single positive integer $\in [10^3, 10^6]$ is allowed.

ORDER. *Determine whether or not the categories are displayed from lowest to highest value.* **TRUE** is the default setting. The parameter has no affects when **SUPPRESSTABLE = {TRUE}**.

INFERENCE Subcommand

INFERENCE controls which Bayesian analysis method is used to make statistical inference.

ANALYSIS. *Specify the Bayesian analysis method that is used to make statistical inference.* Only one value is allowed. Posterior distribution characterization is the default setting (**ANALYSIS = POSTERIOR**). When **ANALYSIS = BAYESFACTOR**, only the Bayes-factor estimation procedure is invoked. When **ANALYSIS = BOTH**, both of the procedures are used.

TABLE Subcommand

TABLE constructs a contingency table to test the independence of two factors.

rowFactorName BY colFactorName. *Specify the design of the contingency table.* The parameter is required. Only two distinct variables are allowed. The first variable denotes the row, and the second denotes the column.

ESTBF Subcommand

ESTBF specifies the model that is assumed for the observed data, fixed marginal totals for the contingency table, prior distribution type when estimating the Bayes factor, shape parameters a_{rs} for Gamma distribution, the scale parameter b for Gamma distribution, and the parameters for the Dirichlet priors

[MODEL]. *Specify the model that is assumed for the observed data.* Valid values include **MULTINOMIAL**, **POISSON**, and **NONPARAMETRIC**. Multinomial distribution (**MULTINOMIAL**) is the default setting.

[FIX_MARGINS]. *Specify if there are any fixed marginal totals for the contingency table.* Valid values include **TOTAL**, **COLUMN**, and **ROW**. This parameter is required when **MODEL** is not POISSON. A fixed grand total (**TOTAL**) is the default setting when **FIXMARGINS** is specified with no keywords. When **MODEL = POISSON**, this parameter has no affect. When **MODEL = NONPARAMETRIC**, this parameter can be either **COLUMN** or **ROW** (**TOTAL** cannot be specified).

PRIOR_TYPE. *Specify the type of the prior distribution when estimating the Bayes factor.* The parameter is required. Valid values include **CONJUGATE**, **MIXTURE_DIRICHLET**, and **INTRINSIC**. A natural conjugate prior (**CONJUGATE**) is the default setting. The parameter has no affect when **MODEL = POISSON**.

SHAPEPARAMS = values. *Specify the shape parameters a_{rs} for Gamma distribution.* The parameter is required when **PRIOR = CONJUGATE**. When a single value is specified, all a_{rs}'s are assumed to be equal to this value. a_{rs}= 1 is the default setting. When more than one value is specified, the values are separated by blank spaces. Different rows must be separated by semicolon characters. The number of the numerical values that are specified in each row and each column must match the contingency table dimensions. All specified values must be > 0.

SCALEPARAM = value. *Specify the scale parameter b for Gamma distribution.* Only a single value > 0 is allowed. **DATA** is the default setting, which is equal to $R \times S \times \min(a_{rs}) / Y$, where R and S are the number of rows and columns for the contingency table, a_{rs} are the values specified for **SHAPEPARAMS**, and Y is the total cell count.

LAMBDAS = values. *Specify the parameters for the Dirichlet priors.* The parameter is required when **MODEL = NONPARAMETRIC**. If a single value is specified, all λ_s are assumed to be equal to this value. λ_s = 1 is the default setting. If more than one value is specified, the values are separated by blank spaces. All specified values must be > 0. The number of the specified numerical values must match the row and column dimensions that are not fixed for the contingency table.

PRINT Subcommand

PRINT controls how content displays in the output tables.

[SUPPRESS_TABLE]. *Determine whether or not content is output to the contingency table.* The parameter is optional. Valid values include **TRUE** or **FALSE** (the default setting).

[CELLS]. *Specify the contents to display in the contingency table.* The parameter is optional and has no affect when **SUPPRESSTABLE = TRUE**. The following values can be specified:

- **OBSERVED**: Display observed cell counts.
- **ROW**: Display row percentages.
- **COLUMN**: Display column percentages.
- **EXPECTED**: Display expected cell counts.
- **TOTAL**: Display total percentages.
- **ALL**: Display all content.

[STATISTICS]. *Specify the statistics for the testing of independence.* The parameter is optional. Valid values include **CHISQ** (the default setting when the parameter is specified with no keywords) and **LRT**.

- **CHISQ**: Compute the Pearson Chi-Square statistic, degrees of freedom, and 2-sided asymptotic significance. For a 2 by 2 contingency table, **CHISQ** also computes the Yates continuity corrected statistics,degrees of freedom, and associated 2-sided asymptotic significance. For a 2 by 2 contingency table with at least one expected cell count < 5, it also computes the 2-sided and 1-sided exact significance of the Fisher's exact test.
- **LRT**: Compute the likelihood ratio test statistic, degrees of freedom, and associated 2-sided asymptotic significance.

BAYES ONESAMPLE

The BAYES ONESAMPLE procedures are available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

BAYES ONESAMPLE: Binomial

BAYES ONESAMPLE: Binomial is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
BAYES ONESAMPLE
     /MISSING SCOPE = {ANALYSIS**} {LISTWISE}
     /CRITERIA
          CILEVEL = {95**} {value}
     /INFERENCE
          DISTRIBUTION = {NORMAL} {BINOMIAL} {POISSON}
          VARIABLES = {varlist}
          ANALYSIS = {POSTERIOR**} {BAYESFACTOR} {BOTH}
```

Binomial numerical variable /DATA example:

```
     /DATA
          VARIABLES = {sublist}††
          SUCCESS = {LAST**} {FIRST} {MIDPOINT} {CUTPOINT(value)} {LEVEL(values)}
          NULLPRIORPARAM = {2** 2**} {value1 value2}
          ALTERPRIORPARAM = {value1 value2}
```

Binomial string variable /DATA example:

```
     /DATA
          VARIABLES = {sublist}††
          SUCCESS = {LAST**} {FIRST} {LEVEL(values)}
          NULLPRIORPARAM = {2** 2**} {value1 value2}
          ALTERPRIORPARAM = {value1 value2}
```

```
/PRIOR $$
          VARDIST = {DIFFUSE**} {INVCHISQ(value1 value2)} {INVGAMMA(value1 value2)}
          PRECDIST = {GAMMA(value1 value2)} {CHISQ(value)}++
          TYPE = {CONJUGATE(value1 value2)}
```

** Default if the subcommand or keyword is omitted.

†† Must be a subset of VARIABLES.

$$ Required for POSTERIOR and BOTH.

++ PRECDIST and VARDIST are mutually exclusive.

Release History

Release 25.0

- Command introduced

Overview

BAYES ONESAMPLE: Binomial invokes the one-sample inference procedure. The procedure provides options for executing Bayesian one-sample inference on Binomial distribution. The parameter of interest is π, which denotes the probability of success in a fixed number of trials that may lead to either success or failure. Note that each trial is independent of each other, and the probability π remains the same in each trial. A binomial random variable can be seen as the sum of a fixed number of independent Bernoulli trials.

Although it is not necessary, a prior from the Beta distribution family is normally chosen when estimating a binomial parameter. The Beta family is conjugate for the binomial family, and as such leads to the posterior distribution with a closed form still in the Beta distribution family.

Options

Missing values
You can specify the method in which to control missing values (exclude cases pairwise or listwise).

Credible interval percentage
You can specify the significance level for computing credible intervals.

Bayesian analysis method
You can specify POSTERIOR, BAYESFACTOR, or BOTH.

Recode data
You can specify the recode data and set parameters for the estimation of Bayes factors.

Prior distribution settings
You can specify the prior distribution for the variance parameter.

Subcommand order

- The subcommands can be named in any order.

Syntax rules

- PRECDIST and VARDIST are mutually exclusive.
- The **/DATA** sublist must be a subset of **VARIABLES**.

Limitations

- PRECDIST and VARDIST are mutually exclusive.

Example

```
BAYES ONESAMPLE
     /MISSING SCOPE = ANALYSIS
     /CRITERIA CILEVEL = 95
     /INFERENCE DISTRIBUTION = BINOMIAL VARIABLES = gender ANALYSIS = BAYESFACTOR
     /DATA VARIABLES = gender SUCCESS = LEVEL(f) ALTERPRIORPARAM = 5 5 .
```

- **MISSING** specifies how missing values are handled. The example specifies **ANALYSIS** as the missing values **SCOPE**. The example excludes records with missing values on a test-by-test basis. Records with missing values for a field, that is used for a specific test, are omitted from the test.
- **CRITERIA** specifies that the significance level for computing credible intervals (**CILEVEL**) is set to the default value of 95.
- **INFERENCE** specifies the distribution and variables for the analysis. The example uses a binomial distribution, the **gender** variable, and Estimate Bayes Factor (**BAYESFACTOR**) as the **ANALYSIS** method.
- **DATA** specifies data variance and set parameters for the estimation of Bayes factors. The example uses the **gender** variable.

MISSING SCOPE Subcommand

MISSING specifies the method in which to control missing values. Missing values can be controlled test-by-test (**ANALYSIS**) or **LISTWISE**. The **ANALYSIS** option, the default setting, excludes records with missing values on a test-by-test basis. Records with missing values for a field that is used for a specific test are omitted from the test. The **LISTWISE** option excludes records with missing values listwise. Records with missing values for any field that is named on any subcommand are excluded from all analysis.

Example

```
BAYES ONESAMPLE
     /MISSING SCOPE = ANALYSIS
     /CRITERIA CILEVEL = 95
     /INFERENCE DISTRIBUTION = BINOMIAL VARIABLES = gender ANALYSIS = BAYESFACTOR
     /DATA VARIABLES = gender SUCCESS = LEVEL(f) ALTERPRIORPARAM = 5 5 .
```

- The example excludes records with missing values on a test-by-test basis. Records that include missing values, for a field that is used for a specific test, are omitted from the test.

CRITERIA Subcommand

CRITERIA controls the statistical criteria used in building the Bayesian Binomial Proportion models.

CILEVEL = value. *Specify the significance level for computing credible intervals.* Must be a single double value ∈(0,100). The default level is 95%

INFERENCE Subcommand

INFERENCE specifies the distribution and variables for the analysis.

DISTRIBUTION. *Specify the distribution about which to draw inference.* The parameter is required. Valid values include **NORMAL**, **BINOMIAL**, and **POSSION**. The specified value determines the subsequent syntax.

VARIABLES = varlist. *Specify the variables to use for the procedure.* The parameter is required. Each variable in the list is analyzed.

ANALYSIS. *Specify the Bayesian analysis method that is used to make statistical inference.* Only one value is allowed. Posterior distribution characterization is the default setting (**ANALYSIS = POSTERIOR**). When **ANALYSIS = BAYESFACTOR**, only the Bayes-factor estimation procedure is invoked. When **ANALYSIS = BOTH**, both of the procedures are used.

DATA Subcommand

DATA specifies recode data and set parameters for the estimation of Bayes factors. At least one **/DATA** subcommand is required to run the analysis. Multiple **DATA** subcommands are allowed.

Numerical variables

VARIABLES = sublist. *Specify the variables to use for the procedure.* The parameter is required. The available variables are a subset of the original variable list. Each variable in the list is analyzed.

SUCCESS. *Specify how"success", the data value(s) tested against the test value, is defined for numerical variables.* Valid when **DISTRIBUTION = BINOMIAL**.

- **LAST** is the default setting. The parameter performs the binomial test using the last value found in the category after sorted in an ascending order to define "success".
- **FIRST** uses the first value found in the category after sorted in an ascending order.
- **MIDPOINT** uses the values ≥ the midpoint as cases. A midpoint value is the average of the minimum and maximum sample data.
- **CUTPOINT(value)** uses the values ≥ a specified cutoff values as cases. The value must be a single numeric.
- **LEVEL(values)** treats the user-specified values (there can be more than 1) as cases.

NULLPRIORPARAM = value1 value 2. *Specify the shape parameter a_0 and the scale parameter b_0 under the null hypothesis of Binomial inference.* Valid when **DISTRIBUTION= BINOMIAL**. For Binomial-Beta relationship, the support range is **value1, value2 > 0** or **value1 = value2 = 0** to accommodate the Beta and the Haldane's priors.

ALTERPRIORPARAM = value1 value 2. *Specify a_1 and b_1 under the alternative hypothesis of Binomial inference if Bayes factor is to be estimated.* The support range is the same as those in **NULLPRIORPARAM**. The parameter is valid when **DISTRIBUTION = BINOMIAL**. For Binomial-Beta relationship, the support range is **value1, value2 > 0** or **value1 = value2 = 0** to accommodate the Beta and the Haldane's priors.

String variables

VARIABLES = sublist. *Specify the variables to use for the procedure.* The parameter is required. The available variables are a subset of the original variable list. Each variable in the list is analyzed.

SUCCESS. *Specify how "success", the data value(s) tested against the test value, is defined for string variables.* Valid when **DISTRIBUTION = BINOMIAL**.

- **LAST** is the default setting. The parameter performs the binomial test using the last string found in the category after sorted in an ascending order to define "success".
- **FIRST** uses the first string found in the category after sorted in an ascending order.
- **LEVEL(values)** treats the user-specified strings (there can be more than 1) as cases.

NULLPRIORPARAM = value1 value 2. *Specify the shape parameter a_0 and the scale parameter b_0 under the null hypothesis of Binomial inference.* Valid when **DISTRIBUTION= BINOMIAL**. For Binomial-Beta relationship, the support range is **value1, value2 > 0** or **value1 = value2 = 0** to accommodate the Beta and the Haldane's priors.

ALTERPRIORPARAM = value1 value 2. *Specify a_1 and b_1 under the alternative hypothesis of Binomial inference if Bayes factor is to be estimated.* The support range is the same as those in **NULLPRIORPARAM**. The parameter is valid when **DISTRIBUTION = BINOMIAL**. For Binomial-Beta relationship, the support range is **value1, value2 > 0** or **value1 = value2 = 0** to accommodate the Beta and the Haldane's priors.

PRIOR Subcommand

PRIOR specifies the prior distribution.

VARDIST. *Specify the prior distribution for the variance parameter.* Valid when neither the data variance nor **PRECDIST** is specified. The diffuse prior is the default setting. When the data variance is specified for some variables, **VARDIST** is ignored for those variables.

- **DIFFUSE**: Specify the diffuse prior $\propto 1$.
- **INVCHISQ(value1 value2)**: Specify the distribution and parameters for inverse -$\chi^2(v_0,\sigma^2_0)$, where $v_0 > 0$ is the degree of freedom, and $\sigma^2_0 > 0$ is the scale parameter.
- **INVGAMMA(value1 value2)**: Specify the distribution and parameters for inverse-Gamma(α_0,β_0),where $\alpha_0 > 0$ is the shape parameter, and $\beta_0 > 0$ is the scale parameter.

PREDIST. *Specify the prior distribution for the precision parameter.* The parameter is valid when neither the data variance nor **VARDIST** are specified. The subcommand is an mutually exclusive to **VARDIST**.

- **GAMMA(value1 value2)**: Specify the distribution and parameters for Gamma(α_0,β_0), where $\alpha_0 > 0$ is the shape parameter, and β_0 is the scale parameter.
- **CHISQ(value)**: Specify the distribution and parameters for $\chi^2(v_0)$, where $v_0 > 0$ is the degree of freedom.

TYPE. *Specify the conjugate prior for Binomial parameters for characterizing posterior distribution.* For Binomial, the conjugate prior follows a Beta distribution. For Binomial-Beta relationship, the support range is **value1**, **value2 > 0** or **value1 = value2 = 0** to accommodate the Beta and the Haldane's priors. The parameter is valid only when **DISTRIBUTION = BINOMIAL**.

BAYES ONESAMPLE: Normal

BAYES ONESAMPLE: Normal is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
BAYES ONESAMPLE
     /MISSING SCOPE = {ANALYSIS**} {LISTWISE}
     /CRITERIA
          CILEVEL = {95**} {value}
          METHOD = {AGL**} {MCA}
          SEED = {RANDOM**} {2000000} {integer}
          TOL = {0.000001**} {value}
          MAXITER = {2000**} {integer}
          MCSAMPLES = {10000**} {integer}
     /INFERENCE
          DISTRIBUTION = {NORMAL} {BINOMIAL} {POISSON}
          VARIABLES = {varlist}
          ANALYSIS = {POSTERIOR**} {BAYESFACTOR} {BOTH}
     /DATA
          VARIABLES = {sublist}††
          [VARIANCE = {value}]^^
```

```
          NULLVALUE = {0**} {value}
          ALTERGVALUE = {1**} {value}@@
     /PRIOR $$
          VARDIST = {DIFFUSE**} {INVCHISQ(value1 value2)} {INVGAMMA(value1 value2)}
                    {JEFFREYS2} {JEFFREYS4              }
          PRECDIST = {GAMMA(value1 value2)} {CHISQ(value)}++
          MEANDIST = {DIFFUSE**} {NORMAL(value1 value2)}
          KAPPA = {1**} {value}
```

** Default if the subcommand or keyword is omitted.

†† Must be a subset of VARIABLES.

^^ Optional for Normal with known variance.

@@ If VARIANCE is given, this parameter must be specified.

$$ Required for POSTERIOR and BOTH.

++ PRECDIST and VARDIST are mutually exclusive.

Release History

Release 25.0

- Command introduced

Overview

BAYES ONESAMPLE: Normal invokes the one-sample inference procedure. The procedure provides options for making Bayesian inference on one-sample and two-sample paired t-test by characterizing posterior distributions. When you have normal data, you can use a normal prior to obtain a normal posterior.

Options

Missing values
You can specify the method in which to control missing values (exclude cases pairwise or listwise).

Analysis options
You can specify the significance level for computing credible intervals, the numerical method to estimate the integral, a random seed set for the MCA method, the tolerance value for numerical methods, the maximum number of iterations the AGL method can go through in its computations, and the number of samples used to draw the posterior distribution.

Bayesian analysis method
You can specify POSTERIOR, BAYESFACTOR, or BOTH.

Data variance
You can specify the data variance and set parameters for the estimation of Bayes factors.

Prior distribution settings
You can specify the prior distribution for the variance parameter.

Subcommand order

- The subcommands can be named in any order.

Syntax rules

- PRECDIST and VARDIST are mutually exclusive.
- The **/DATA** sublist must be a subset of **VARIABLES**.

Limitations

- PRECDIST and VARDIST are mutually exclusive.

Example

```
BAYES ONESAMPLE
     /MISSING SCOPE = ANALYSIS
     /CRITERIA CILEVEL = 95
     /INFERENCE DISTRIBUTION = NORMAL VARIABLES = logSalary logSalbegin ANALYSIS = POSTERIOR
     /PRIOR VARDIST = INVCHISQ(2 1) MEANDIST = NORMAL(10) KAPPA = 2 .
```

- **MISSING** specifies how missing values are handled. The example specifies **ANALYSIS** as the missing values **SCOPE**. The example excludes records with missing values on a test-by-test basis. Records with missing values for a field, that is used for a specific test, are omitted from the test.
- **CRITERIA** specifies that the significance level for computing credible intervals (**CILEVEL**) is set to the default value of 95.
- **INFERENCE** specifies the distribution and variables for the analysis. The example uses a normal distribution, the **logSalary** and **logSalbegin** variables, and Characterize Posterior Distribution (**POSTERIOR**) as the **ANALYSIS** method.
- **PRIOR** specifies the prior distribution. The example uses the **INVCHISQ** prior for the variance parameter (**VARDIST**), the **NORMAL** prior for the mean parameter (**MEANDIST**), and a **KAPPA** value of 2.

MISSING SCOPE Subcommand

MISSING specifies the method in which to control missing values. Missing values can be controlled test-by-test (**ANALYSIS**) or **LISTWISE**. The **ANALYSIS** option, the default setting, excludes records with missing values on a test-by-test basis. Records with missing values for a field that is used for a specific test are omitted from the test. The **LISTWISE** option excludes records with missing values listwise. Records with missing values for any field that is named on any subcommand are excluded from all analysis.

Example

```
BAYES ONESAMPLE
     /MISSING SCOPE = ANALYSIS
     /CRITERIA CILEVEL = 95
     /INFERENCE DISTRIBUTION = NORMAL VARIABLES = logSalary logSalbegin ANALYSIS = POSTERIOR
     /PRIOR VARDIST = INVCHISQ(2 1) MEANDIST = NORMAL(10) KAPPA = 2 .
```

- The example excludes records with missing values on a test-by-test basis. Records that include missing values, for a field that is used for a specific test, are omitted from the test.

CRITERIA Subcommand

CRITERIA controls the statistical criteria used in building the Bayesian One Sample/Pair Sample models. The way in which these criteria are used depends on the method specified on the METHOD subcommand. The default criteria are noted in the description of each keyword below.

CILEVEL = value. *Specify the significance level for computing credible intervals.* Must be a single double value $\in$ (0,100). The default level is 95%

METHOD. *Specify the numerical method to estimate the integral.* Valid values include **AGL** and **MCA**. **AGL** calls the adaptive Gauss-Lobatto quadrature approach (the default setting). **MCA** calls the Monte Carlo approximation approach.

SEED = integer. *Specify a random seed set for the MCA method, with a random seed assigned by default.* Only in effect when **METHOD = MCA**. You can set a seed by specifying a positive integer $\in$ [1,2147483647].

TOL = value. *Specify the tolerance value for the numerical methods.* 0.000001 is the default setting. Only in effect when METHOD = AGL. Must be one value $\in (0, 10^{-3}]$.

MAXITER = integer. *Specify the maximum number of iterations the AGL method can go through in its computations.* The integer 2000 is the default setting. You must specify a positive integer. Valid when METHOD = AGL.

MCSAMPLES = integer. *Specify the number of points that are sampled for the Monte Carlo approximation.* Only in effect when **METHOD = MCA**. 10,000 is the default setting. The value must be an integer $\in [10^3, 10^6]$.

INFERENCE Subcommand

INFERENCE specifies the distribution and variables for the analysis.

DISTRIBUTION. *Specify the distribution about which to draw inference.* The parameter is required. Valid values include **NORMAL**, **BINOMIAL**, and **POSSION**. The specified value determines the subsequent syntax.

VARIABLES = varlist. *Specify the variables to use for the procedure.* The parameter is required. Each variable in the list is analyzed.

ANALYSIS. *Specify the Bayesian analysis method that is used to make statistical inference.* Only one value is allowed. Posterior distribution characterization is the default setting (**ANALYSIS = POSTERIOR**). When **ANALYSIS = BAYESFACTOR**, only the Bayes-factor estimation procedure is invoked. When **ANALYSIS = BOTH**, both of the procedures are used.

DATA Subcommand

DATA specifies data variance and set parameters for the estimation of Bayes factors. Multiple **DATA** subcommands are allowed.

VARIABLES = sublist. *Specify the variables to use for the procedure.* The parameter is required. The available variables are a subset of the original variable list. Each variable in the list is analyzed.

[VARIANCE = value]. *Specify the value of variance, if known, for observed data.* The parameter is optional. Only one value > 0 is allowed and the parameter is valid only when **DISTRIBUTION = NORMAL**.

NULLVALUE = value. *Specify the null value in the Bayes factor estimation.* The parameter is required. Only one value is allowed (the default value is 0) and the parameter is valid only when **DISTRIBUTION = NORMAL**.

ALTERGVALUE = value. *Specify the value to define $\psi^2 = g\sigma^2{}_x$ in the Bayes factor estimation.* Only one value > 0 is allowed and the parameter is valid only when **DISTRIBUTION = NORMAL**. When **VARIANCE** is specified, the parameter must be specified with 1 as the default setting. When **VARIANCE** is not specified, you can either specify a fixed *g* or omit the value to integrate it out.

PRIOR Subcommand

PRIOR specifies the prior distribution.

VARDIST. *Specify the prior distribution for the variance parameter.* Valid when neither the data variance nor **PRECDIST** is specified. The diffuse prior is the default setting. When the data variance is specified for some variables, **VARDIST** is ignored for those variables.

- **DIFFUSE**: Specify the diffuse prior $\propto 1$.
- **INVCHISQ(value1 value2)**: Specify the distribution and parameters for inverse -$\chi^2(\nu_0,\sigma^2{}_0)$, where $\nu_0 > 0$ is the degree of freedom, and $\sigma^2{}_0 > 0$ is the scale parameter.
- **INVGAMMA(value1 value2)**: Specify the distribution and parameters for inverse-Gamma(α_0,β_0),where $\alpha_0 > 0$ is the shape parameter, and $\beta_0 > 0$ is the scale parameter.
- **JEFFREYS2**: Specify the noninformative prior $\propto 1/\sigma^2{}_0$.
- **JEFFREYS4**: Specify the noninformative prior $\propto 1/\sigma^4{}_0$.

PRECDIST. *Specify the prior distribution for the precision parameter.* The parameter is valid when neither the data variance nor **VARDIST** are specified. The subcommand is an mutually exclusive to **VARDIST**.

- **GAMMA(value1 value2)**: Specify the distribution and parameters for Gamma(α_0,β_0), where $\alpha_0 > 0$ is the shape parameter, and β_0 is the scale parameter.
- **CHISQ(value)**: Specify the distribution and parameters for $\chi^2(\nu_0)$, where $\nu_0 > 0$ is the degree of freedom.

MEANDIST. *Specify the prior distribution for the mean parameter conditional on the variance or the precision parameter.* **DIFFUSE** is the default setting.

- **DIFFUSE**: Specify the diffuse prior $\propto 1$.
- **NORMAL(value1 value2)**: Specify the distribution and parameters for Normal(μ_0,$\kappa_0^{-1}\sigma^2_0$) on variance or Normal(μ_0,κ_0/σ^2_0) on precision, where $\mu_0 \in (-\infty,\infty)$ and $\sigma^2 > 0$.

If **VARIANCE** is specified for a **/DATA** subcommand:

- **VARDIST**, if specified, has no affect on **VARIABLES** specified for the **/DATA** subcommand.
- When **MEANDIST = NORMAL(value1 value2)**, both **value1** and **value2** are required.

When **VARIANCE** is not specified for a **/DATA** subcommand:

- **VARDIST** or **PRECDIST** must be specified for normal **VARIABLES**.
- When **VARDIST = INVCHISQ** or **INVGAMMA**, or **PRECDIST = GAMMA** or **CHISQ**
  - When **MEANDIST = NORMAL(value1 value2)**, only **value1** takes effect (**value2** is ignored).
  - When **MEANDIST = DIFFUSE**, **VARDIST** and **PRECDIST** are ignored, and **VARDIST = DIFFUSE** is used.
- When **VARDIST = DIFFUSE**, **JEFFREYS2**, or **JEFFREYS4**:
  - When **MEANDIST = NORMAL(value1 value2)**, **NORMAL** is ignored, and **MEANDIST =DIFFUSE** is used.

KAPPA = value. *Specify the value of κ_0 in Normal(μ_0,$\kappa^{-1}{}_0\sigma^2{}_0$) or Normal($\mu_0$,$\kappa_0/\sigma^2{}_0$).* Must be a single value > 0 (1 is the default value).

BAYES ONESAMPLE: Poisson

BAYES ONESAMPLE: Poisson is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
BAYES ONESAMPLE
     /MISSING SCOPE = {ANALYSIS**} {LISTWISE}
     /CRITERIA
          CILEVEL = {95**} {value}
     /INFERENCE
          DISTRIBUTION = {NORMAL} {BINOMIAL} {POISSON}
          VARIABLES = {varlist}
          ANALYSIS = {POSTERIOR**} {BAYESFACTOR} {BOTH}
     /DATA
          VARIABLES = {sublist}††
          NULLPRIORPARAM = {2** 2**} {value1 value2}
          ALTERPRIORPARAM = {value1 value2}
     /PRIOR $$
          VARDIST = {DIFFUSE**} {INVCHISQ(value1 value2)} {INVGAMMA(value1 value2)}
          PRECDIST = {GAMMA(value1 value2)} {CHISQ(value)}++
          TYPE = {CONJUGATE(value1 value2)}
```

** Default if the subcommand or keyword is omitted.

†† Must be a subset of VARIABLES.

$$ Required for POSTERIOR and BOTH.

++ PRECDIST and VARDIST are mutually exclusive.

Release History

Release 25.0

- Command introduced

Overview

BAYES ONESAMPLE: Poisson invokes the one-sample inference procedure. The procedure provides options for executing Bayesian one-sample inference on Binomial distribution. Poisson distribution, a useful model for rare events, assumes that within small time intervals, the probability of an event to occur

is proportional to the length of waiting time. A conjugate prior within the Gamma distribution family is used when drawing Bayesian statistical inference on Poisson distribution.

Options

Missing values
You can specify the method in which to control missing values (exclude cases pairwise or listwise).

Credible interval percentage
You can specify the significance level for computing credible intervals.

Bayesian analysis method
You can specify POSTERIOR, BAYESFACTOR, or BOTH.

Set parameters
You can set parameters for the estimation of Bayes factors.

Prior distribution settings
You can specify the prior distribution for the variance parameter.

Subcommand order

- The subcommands can be named in any order.

Syntax rules

- PRECDIST and VARDIST are mutually exclusive.
- The **/DATA** sublist must be a subset of **VARIABLES**.

Limitations

- PRECDIST and VARDIST are mutually exclusive.

Example

```
BAYES ONESAMPLE
     /MISSING SCOPE = ANALYSIS
     /CRITERIA CILEVEL = 95
     /INFERENCE DISTRIBUTION = POISSON VARIABLES = prevexp ANALYSIS = BOTH
     /DATA VARIABLES = prevexp NULLPRIORPARAM = 1 1 ALTERPRIORPARAM = 0.5 1
     /PRIOR TYPE = CONJUGATE(0.5 0) .
```

- **MISSING** specifies how missing values are handled. The example specifies **ANALYSIS** as the missing values **SCOPE**. The example excludes records with missing values on a test-by-test basis. Records with missing values for a field, that is used for a specific test, are omitted from the test.
- **CRITERIA** specifies that the significance level for computing credible intervals (**CILEVEL**) is set to the default value of 95.
- **INFERENCE** specifies the distribution and variables for the analysis. The example uses a Poisson distribution, the **prevexp** variable, and both Characterize Posterior Distribution and Estimate Bayes Factor (**BOTH**) as the **ANALYSIS** method.
- **DATA** specifies shape parameters and alternate hypothesis values for the estimation of Bayes factors. The example uses the **prevexp** variable.
- **PRIOR** specifies the prior distribution. The example uses conjugate prior.

MISSING SCOPE Subcommand

MISSING specifies the method in which to control missing values. Missing values can be controlled test-by-test (**ANALYSIS**) or **LISTWISE**. The **ANALYSIS** option, the default setting, excludes records with missing values on a test-by-test basis. Records with missing values for a field that is used for a specific test are omitted from the test. The **LISTWISE** option excludes records with missing values listwise. Records with missing values for any field that is named on any subcommand are excluded from all analysis.

Example

```
BAYES ONESAMPLE
     /MISSING SCOPE = ANALYSIS
     /CRITERIA CILEVEL = 95
     /INFERENCE DISTRIBUTION = POISSON VARIABLES = prevexp ANALYSIS = BOTH
     /DATA VARIABLES = prevexp NULLPRIORPARAM = 1 1 ALTERPRIORPARAM = 0.5 1
     /PRIOR TYPE = CONJUGATE(0.5 0) .
```

- The example excludes records with missing values on a test-by-test basis. Records that include missing values, for a field that is used for a specific test, are omitted from the test.

CRITERIA Subcommand

CRITERIA controls the statistical criteria used in building the Bayesian Poisson Distribution models.

CILEVEL = value. *Specify the significance level for computing credible intervals.* Must be a single double value $\in(0,100)$. The default level is 95%

INFERENCE Subcommand

INFERENCE specifies the distribution and variables for the analysis.

DISTRIBUTION. *Specify the distribution about which to draw inference.* The parameter is required. Valid values include **NORMAL**, **BINOMIAL**, and **POSSION**. The specified value determines the subsequent syntax.

VARIABLES = varlist. *Specify the variables to use for the procedure.* The parameter is required. Each variable in the list is analyzed.

ANALYSIS. *Specify the Bayesian analysis method that is used to make statistical inference.* Only one value is allowed. Posterior distribution characterization is the default setting (**ANALYSIS = POSTERIOR**). When **ANALYSIS = BAYESFACTOR**, only the Bayes-factor estimation procedure is invoked. When **ANALYSIS = BOTH**, both of the procedures are used.

DATA Subcommand

DATA specifies the set parameters for the estimation of Bayes factors. At least one **/DATA** subcommand is required to run the analysis. Multiple **DATA** subcommands are allowed.

VARIABLES = sublist. *Specify the variables to use for the procedure.* The parameter is required. The available variables are a subset of the original variable list. Each variable in the list is analyzed.

NULLPRIORPARAM = value1 value 2. *Specify the shape parameter a_0 and the scale parameter b_0 under the null hypothesis of Poisson inference.* Valid when **DISTRIBUTION= POISSION**. The support range is **value1,value2 > 0**.

ALTERPRIORPARAM = value1 value 2. *Specify a_1 and b_1 under the alternative hypothesis of Poisson inference if Bayes factor is to be estimated.* The support range is the same as those in **NULLPRIORPARAM**. The parameter is valid when **DISTRIBUTION = POISSON**. The support range is **value1,value2 > 0**.

PRIOR Subcommand

PRIOR specifies the prior distribution.

VARDIST. *Specify the prior distribution for the variance parameter.* Valid when neither the data variance nor **PRECDIST** is specified. The diffuse prior is the default setting. When the data variance is specified for some variables, **VARDIST** is ignored for those variables.

- **DIFFUSE**: Specify the diffuse prior $\propto 1$.
- **INVCHISQ(value1 value2)**: Specify the distribution and parameters for inverse -$\chi^2(v_0,\sigma^2_0)$, where $v_0 > 0$ is the degree of freedom, and $\sigma^2_0 > 0$ is the scale parameter.
- **INVGAMMA(value1 value2)**: Specify the distribution and parameters for inverse-Gamma(α_0,β_0),where $\alpha_0 > 0$ is the shape parameter, and $\beta_0 > 0$ is the scale parameter.

PREDIST. *Specify the prior distribution for the precision parameter.* The parameter is valid when neither the data variance nor **VARDIST** are specified. The subcommand is an mutually exclusive to **VARDIST**.

- **GAMMA(value1 value2)**: Specify the distribution and parameters for Gamma(α_0,β_0), where $\alpha_0 > 0$ is the shape parameter, and β_0 is the scale parameter.
- **CHISQ(value)**: Specify the distribution and parameters for $\chi^2(\nu_0)$, where $\nu_0 > 0$ is the degree of freedom.

TYPE. *Specify the conjugate prior for Poisson parameters for characterizing posterior distribution.* For Poisson, the conjugate prior follows a Gamma distribution. For Poisson-Gamma relationship, the support range is **value1 > 0** and **value2 ≥ 0** to accommodate the Uniform and the Jeffreys priors. The parameter is valid only when **DISTRIBUTION = POISSON**.

BAYES REGRESSION

BAYES REGRESSION is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
BAYES REGRESSION dependent varname [BY factor list] [WITH covariate list]
     /CRITERIA
          CILEVEL = {95**} {value}
          TOL = {0.000001**} {value}
          MAXITER = {2000**} {integer}
     /DESIGN
          REGWGT = {variable}@@
          FACTORS = {factors}##
          COVARIATES = {covariates}$$
     /INFERENCE
          ANALYSIS = {POSTERIOR**} {BAYESFACTOR} {BOTH}
     /ESTBF
          COMPUTATION = {JZS**} {ZELLNER(value)} {HYPER(3** {value})} {ROUDER(1** {value})}
          COMPARE = {NULL**} {FULL}
     /PRIOR
          TYPE = {REFERENCE**} {CONJUGATE}
          SHAPEPARAM = {value}
          SCALEPARAM = {value}
          MEANVECTOR = {values}
          VMATRIX = {IDENTITY**} {lower_triangular_matrix}%%
     [/PREDICT
          REGRESSORS = {values}]
     [/SAVE
          [ MEANPRED[(varname)]] [VARIANCE[(varname)]] [MODE[(varname)]] [CIMEANPREDL[(varname)]]
          [ CIMEANPREDU[(varname)]]]
     [/OUTFILE
          PARAMETER = 'filename']
     [/PLOT
          COVARIATES = {covariate_sublist}
          FACTORS = {factor_sublist}
          MAX_CATEGORY = {2**} {integer}^^
          INTERCEPT = {FALSE**} {TRUE}
          ERRORVAR = {FALSE**} {TRUE}
          BAYESPRED = {FALSE**} {TRUE}]&&
     [/FTEST ⊕
          [ LABEL = 'labelName']
          VARIABLES = {varlist}⊖
          INTERCEPT = {FALSE**} {TRUE}⊗
          [ TESTVALUES = {values}]] ⊜
```

** Default if the subcommand or keyword is omitted.

@@ Only one regression weight variable is allowed.

Must be a subset of the factor list.

$$ Must be a subset of the covariate list.

%% Separate rows with semicolons.

^^ Controls the number of factor levels to be plotted.

&& Can be TRUE when values are specified for REGRESSORS.

⊕ Multiple /FTEST subcommands are allowed.

⊖ Must be a subset of COVARIATES and FACTORS.

⊗ Must be TRUE when VARIABLES is empty.

⊜ Assumes all values are equal to 0 when not specified.

Release History

Release 25.0

- Command introduced

Overview

`BAYES REGRESSION dependent_varname [BY factor_list] [WITH covariate_list]` invokes the linear regression procedure, and defines a full model. A single dependent variable is required. **BY** and **WITH** are followed by factor(s) and covariate(s), respectively. At least one factor or covariate is required.

Options

Analysis options
You can specify the significance level for computing credible intervals, the tolerance value for numerical methods, and the maximum number of iterations the AGL method can go through in its computations.

Model design
You can specify the model design for the analysis, which includes regression weights, categorical variables, scale variables

Bayesian analysis method
You can specify POSTERIOR, BAYESFACTOR, or BOTH.

Bayes factor settings
You can specify the approach that is used to estimate the Bayes factor.

Prior distribution settings
You can specify REFERENCE and CONJUGATE prior distribution settings, as well as define shape parameters, scale parameters, mean vector, and lower triangle values in the variance-covariance matrix for the multivariate normal prior.

Regressors
You can specify an observed vector with the values for the regressors.

Statistics to be scored
You can specify the statistics to be scored for the Bayesian prediction distribution.

PMML file generation
You can specify a target file to generate the PMML file.

Output plots
You can control the plots that are output including:

- Covariates to be plotted
- Factors to be plotted
- Maximum factor levels to be plotted
- Plot intercept term
- Plot variance of errors
- Plot the predictive distribution

F-tests
You can invoke (partial) F-tests.

Basic specification

- `BAYES REGRESSION dependent_varname [BY factor_list] [WITH covariate_list]` invokes the linear regression procedure, and defines a full model.

Subcommand order

- The subcommands can be named in any order.

Syntax rules

- Only one regression weight variable is allowed.
- The specified **FACTORS** must be a subset of **factor_list**.
- The specified **COVARIATES** must be a subset of **covariate_list**.
- **BAYESPRED** can be **TRUE** if values are specified for **REGRESSORS**.
- **VARIABLES** must be a subset of **COVARIATES** and **FACTORS**.
- **INTERCEPT** must be **TRUE** when **VARIABLES** is empty.

Example

```
BAYES REGRESSION salary WITH educ prevexp
     /CRITERIA CILEVEL = 95
     /DESIGN COVARIATES = educ prevexp
     /INFERENCE ANALYSIS = BAYESFACTOR
     /PRIOR TYPE = REFERENCE .
```

- **CRITERIA** specifies that the significance level for computing credible intervals (**CILEVEL**) is set to the default value of 95.
- **DESIGN** specifies the model design for the analysis. The example uses the **educ** and **prevexp** as the scale variables.
- **INFERENCE** specifies the distribution for the analysis. The example uses Estimate Bayes Factor (**BAYESFACTOR**) as the **ANALYSIS** method.
- **PRIOR** specifies the prior distribution.

CRITERIA Subcommand

CRITERIA controls the statistical criteria used in building the Bayesian Linear Regression models. The default criteria are noted in the description of each keyword below.

CILEVEL = value. *Specify the significance level for computing credible intervals.* Must be a single double value $\in (0,100)$. The default level is 95%

TOL = value. *Specify the tolerance value for the numerical methods.* 0.000001 is the default setting. Only in effect when METHOD = AGL. Must be one value $\in (0, 10^{-3}]$.

MAXITER = integer. *Specify the maximum number of iterations the AGL method can go through in its computations.* The integer 2000 is the default setting. You must specify a positive integer. Valid when METHOD = AGL.

DESIGN Subcommand

DESIGN specifies the model design for the analysis.

REGWGT = variable. *Specify the regression weights.* The parameter is optional. Only one variable is allowed.

FACTORS = factors. *Specify categorical variables for the model.* Must be a subset of the factors in **BAYES REGRESSION**.

COVARIATES = covariates. *Specify scale variables for the model.* Must be a subset of the factors in **BAYES REGRESSION**.

INFERENCE Subcommand

INFERENCE controls which Bayesian analysis method is used to make statistical inference.

ANALYSIS. *Specify the Bayesian analysis method that is used to make statistical inference.* Only one value is allowed. Posterior distribution characterization is the default setting (**ANALYSIS = POSTERIOR**). When **ANALYSIS = BAYESFACTOR**, only the Bayes-factor estimation procedure is invoked. When **ANALYSIS = BOTH**, both of the procedures are used.

ESTBF Subcommand

ESTBF specifies the approach that is used to estimate Bayes factors.

COMPUTATION = value. *Specifies the approach to estimate Bayes factors.* Four different keywords are offered. Siow's approach is the default setting.

- JZS: Invokes the Zellner-Siow's approach where no additional parameters are to specified. This is the default setting.
- ZELLNER: Invokes the Zellner's approach where users have to specify a single fixed value > 0. There is no default setting.
- HYPER: Invokes the hyper-g approach where users have to specify a single fixed value > 2. The default setting is 3.
- ROUDER: Invokes the Rouder's approach where users have to specify a single fixed value > 0. The default setting is 1.

COMPARE. *Determine whether the estimated Bayes factors are based on the null model or the full model.* A null model is the default setting.

PRIOR Subcommand

PRIOR specifies the prior distribution settings for the regression parameters and the variance of the errors.

TYPE. *Specifies the prior distribution for the regression parameters and the variance of the errors.* Reference priors are the default setting.

SHAPEPARAM = value. *Specifies the shape parameter a_0 for Inverse-Gamma distribution.* Must be a single value > 0. The parameter is required when **TYPE = CONJUGATE**.

SCALEPARAM = value. *Specifies the scale parameter b_0 for Inverse-Gamma distribution.* Must be a single value > 0. The parameter is required when **TYPE = CONJUGATE**.

MEANVECTOR = values. *Specifies the mean vector ϑ_0 for the regression parameters.* The number of the values must meet the number of regression parameters, including the intercept term. The values are separated by blank spaces. The parameter is required when **TYPE = CONJUGATE**.

VMATRIX. *Specifies V_0 the values in the lower triangle in the variance-covariance matrix for the multivariate normal prior.* Note that V_0 must be semi-positive definite. The values are separated by blank spaces. Different rows must be separated by semicolon characters. The last value of each row must be positive. The next row should have one more value than its previous row. The identity matrix is the default setting.

PREDICT Subcommand

PREDICT specifies the regressors to generate predictive distributions.

REGRESSORS = values. *Specify an observed vector with the values for the regressors.* Each regressor is assigned one value or string (allow one case can be predicted). For factors, both values and strings are allowed.

SAVE Subcommand

SAVE specifies the statistics to be scored for the Bayesian prediction distribution. Score the statistics derived from the Bayesian predictions, which include to following:

- **{MEANPRED(varname)}**: Mean of the posterior predictive distribution.
- **{VARIANCE(varname)}**: Variance of the posterior predictive distribution.
- **{MODE(varname)}**: Mode of the posterior predictive distribution.
- **{CIMEANPREDL(varname)}**: Lower bound of the credible interval of the posterior predictive distribution.

- **{CIMEANPREDU(varname)}**: Upper bound of the credible interval of the posterior predictive distribution.

Corresponding variable names can also be assigned.

OUTFILE Subcommand

OUTFILE specifies the target file to generate the PMML file.

PARAMETER = 'filename'. *Specify whether or not the variance-covariance matrix of the parameters will be scored in the output PMML file.*

PLOT Subcommand

PLOT controls the plots to be output.

COVARIATES = {covariate_sublist}. *Specify the covariate(s) to be plotted.* Must be a sublist of the covariates specified in **/DESIGN**.

FACTORS = {factor_sublist}. *Specify the factor(s) to be plotted.* Must be a sublist of the factors specified in **/DESIGN**.

MAX_CATEGORY = integer. *Specify the maximum factor levels to be plotted.* Applies to all factors. By default, the first 2 levels are plotted for each factor. Only a single, positive integer is valid.

INTERCEPT. *Specify whether the intercept term is plotted.* **FALSE** is the default setting.

ERRORVAR. *Specify whether the variance of errors is plotted.* **FALSE** is the default setting.

BAYESPRED. *Specify whether the predictive distribution is plotted.* **FALSE** is the default setting. The value can be **TRUE** only when valid values are specified for **REGRESSORS**.

FTEST Subcommand

FTEST invokes (partial) F-tests. Multiple **/FTEST** subcommand are allowed.

[LABEL = 'labelName']. *Specify the optional label name.* The value can be a string with a maximum length of 255 bytes. Only one label per **/FTEST** subcommand is allowed.

VARIABLES = varlist. *Specify the covariates and factors to be tested.* Must be a subset of **COVARIATES** and **FACTORS**. When no variables are specified, **INTERCEPT** must be **TRUE**.

INTERCEPT. *Specify whether or not the intercept term is included in the test.* **FALSE** is the default setting.

[TESTVALUES = values]. *Specify the values to be tested.* The number of values must match the number of parameters in the original model. If included, the first value must be specified for the intercept term. Assume all values are equal to 0 if not specified. The order of the test values must match the order of the variables specified by **VARIABLES**.

BAYES RELATED

BAYES RELATED is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
BAYES RELATED
     /MISSING SCOPE = {ANALYSIS**} {LISTWISE}
     /CRITERIA
          CILEVEL = {95**} {value}
          METHOD = {AGL**} {MCA}
          SEED = {RANDOM**} {2000000} {integer}
          TOL = {0.000001**} {value}
          MAXITER = {2000**} {integer}
          MCSAMPLES = {10000**} {integer}
     /INFERENCE
          ANALYSIS = {POSTERIOR**} {BAYESFACTOR} {BOTH}
```

A /PAIR example for known variance. Requires at least one PAIR subcommand.

```
     /PAIR
          VARIABLES = {varname1 pairname1}
          [VARIANCE = {value}]^^
          NULLVALUE = {0**} {value}
          ALTERGVALUE = {1**} {value}@@
```

A /PAIR example for unknown variance. Allows multiple PAIR subcommands.

```
     /PAIR
          VARIABLES = {varname2 pairname2}
          NULLVALUE = {0**} {value}
          [ ALTERGVALUE = {1**} {value}]##
```

```
     /PRIOR
          VARDIST = {DIFFUSE**} {INVCHISQ(value1 value2)} {INVGAMMA(value1 value2)} {JEFFREYS2}
          {JEFFREYS4}
          PRECDIST = {GAMMA(value1 value2)} {(CHISQ(value)}%%
          MEANDIST = {DIFFUSE**} {NORMAL(value1 value2)}
          KAPPA = {1**} {value}
```

** Default if the subcommand or keyword is omitted.

++ Must be two variables.

^^ This is an optional parameter to specify the variance value (if known).

@@ If VARIANCE is given, this parameter must be specified.

Optional if VARIANCE is not specified.

%% PRECDIST and VARDIST are mutually exclusive.

Release History

Release 25.0

- Command introduced

Overview

BAYES RELATED invokes the related-sample inference procedure. The procedure provides Bayesian one-sample inference options for paired samples. You can specify the variable names in pairs, and run the Bayesian analysis on the mean difference.

Options

Missing values

You can specify the way in which missing values are handled. You can exclude records with missing values on a test-by-test basis, or exclude records with missing values listwise.

Analysis options
You can specify the significance level for computing credible intervals, the tolerance value for numerical methods, and the maximum number of iterations the AGL method can go through in its computations.

Bayesian analysis method
You can specify POSTERIOR, BAYESFACTOR, or BOTH.

Variable pairs
You can specify pairs of variables when the variance is known and unknown. At least one variable pair is required. Multiple **/PAIR** sub-commands are allowed.

Prior distribution settings
You can specify the prior distribution for the variance parameter.

Subcommand order

- The subcommands can be named in any order.

Syntax rules

- **/PAIR** requires at least one **PAIR** subcommand.
- **ALTERGVALUE** is required when **VARIANCE** is specified.
- PRECDIST and VARDIST are mutually exclusive.

Limitations

- PRECDIST and VARDIST are mutually exclusive.

Example

```
BAYES RELATED
     /MISSING SCOPE = ANALYSIS
     /CRITERIA CILEVEL = 95
     /INFERENCE DISTRIBUTION = NORMAL ANALYSIS = BAYESFACTOR
     /PAIR VARIABLES = salary salbegin NULLVALUE = 15000 .
```

- **MISSING** specifies how missing values are handled. The example specifies **ANALYSIS** as the missing values **SCOPE**. The example excludes records with missing values on a test-by-test basis. Records with missing values for a field, that is used for a specific test, are omitted from the test.
- **CRITERIA** specifies that the significance level for computing credible intervals (**CILEVEL**) is set to the default value of 95.
- **INFERENCE** specifies the distribution and variables for the analysis. The example uses the **NORMAL** distribution, and uses estimate Bayes factor (**BAYESFACTOR**) as the **ANALYSIS** method.
- **PAIR** specifies the pairs of variables. The example use the **salary** and **salbegin** variable pair.

MISSING SCOPE Subcommand

MISSING specifies the method in which to control missing values. Missing values can be controlled test-by-test (**ANALYSIS**) or **LISTWISE**. The **ANALYSIS** option, the default setting, excludes records with missing values on a test-by-test basis. Records with missing values for a field that is used for a specific test are omitted from the test. The **LISTWISE** option excludes records with missing values listwise. Records with missing values for any field that is named on any subcommand are excluded from all analysis.

Example

```
BAYES RELATED
     /MISSING SCOPE = ANALYSIS
     /CRITERIA CILEVEL = 95
```

```
/INFERENCE DISTRIBUTION = NORMAL ANALYSIS = BAYESFACTOR
/PAIR VARIABLES = salary salbegin NULLVALUE = 15000 .
```

- The example excludes records with missing values on a test-by-test basis. Records that include missing values, for a field that is used for a specific test, are omitted from the test.

CRITERIA Subcommand

CRITERIA controls the statistical criteria used in building the Bayesian Related Sample Inference models. The way in which these criteria are used depends on the method specified on the METHOD subcommand. The default criteria are noted in the description of each keyword below.

CILEVEL = value. *Specify the significance level for computing credible intervals.* Must be a single double value $\in (0,100)$. The default level is 95%.

METHOD. *Specify the numerical method to estimate the integral.* Valid values include **AGL** and **MCA**. **AGL** calls the adaptive Gauss-Lobatto quadrature approach (the default setting). **MCA** calls the Monte Carlo approximation approach.

SEED = integer. *Specify a random seed set for the MCA method, with a random seed assigned by default.* Only in effect when **METHOD = MCA**. You can set a seed by specifying a positive integer $\in [1,2147483647]$.

TOL = value. *Specify the tolerance value for the numerical methods.* 0.000001 is the default setting. Only in effect when METHOD = AGL. Must be one value $\in (0, 10^{-3}]$.

MAXITER = integer. *Specify the maximum number of iterations the AGL method can go through in its computations.* The integer 2000 is the default setting. You must specify a positive integer. Valid when METHOD = AGL.

MCSAMPLES = integer. *Specify the number of points that are sampled for the Monte Carlo approximation.* Only in effect when **METHOD = MCA**. 10,000 is the default setting. The value must be an integer $\in [10^3,10^6]$.

INFERENCE Subcommand

INFERENCE controls the Bayesian analysis method that is used to make statistical inference.

ANALYSIS. *Specify the Bayesian analysis method that is used to make statistical inference.* Only one value is allowed. Posterior distribution characterization is the default setting (**ANALYSIS = POSTERIOR**). When **ANALYSIS = BAYESFACTOR**, only the Bayes-factor estimation procedure is invoked. When **ANALYSIS = BOTH**, both of the procedures are used.

PAIR Subcommand

PAIR specifies the pairs of variables. At least one **/PAIR** is required. Multiple **/PAIR** subcommands are allowed.

Known variance

VARIABLES = varname2 pairname 2. *Specify the names of two related variables under testing.* The parameter is required. You must specify two variable names per pair.

[VARIANCE = value]. *Specify the variance of (varname1–pairname1), if known.* Only a single, positive value is allowed. The parameter is optional. When **VARIANCE** is specified for a **/PAIR** subcommand:

- **VARDIST**, if specified, has no effect on **VARIABLES** that are specified for the **/PAIR** subcommand.
- When **MEANDIST = NORMAL(value1 value2)**, both **value1** and **value2** are required.

When **VARIANCE** is not specified for a **/PAIR** subcommand:

- **VARDIST** or **PRECDIST** must be specified.
- When **VARDIST = INVCHISQ** or **INVGAMMA**, or **PRECDIST = GAMMA** or **CHISQ**:
  - When **MEANDIST = NORMAL(value1 value2)**, only **value1** takes effect, and **value2** is ignored.
  - When **MEANDIST = DIFFUSE**, **VARDIST** or **PRECDIST** is ignored, and **VARDIST = DIFFUSE** is used.
- When **VARDIST = DIFFUSE**, **JEFFREYS2**, or **JEFFREYS4**:
  - When **MEANDIST = NORMAL(value1 value2)**, **NORMAL** is ignored, and **MEANDIST = DIFFUSE** is used.

NULLVALUE = value. *Specify the null value in the Bayes factor estimation.* The parameter is required. Only one value is allowed (the default value is 0) and the parameter is valid only when **DISTRIBUTION = NORMAL**.

ALTERGVALUE = value. *Specify the value to define $\psi^2 = g\sigma^2_x$ in the Bayes factor estimation.* Only one value > 0 is allowed and the parameter is valid only when **DISTRIBUTION = NORMAL**. When **VARIANCE** is specified, the parameter must be specified with 1 as the default setting. When **VARIANCE** is not specified, you can either specify a fixed *g* or omit the value to integrate it out.

Unknown variance

VARIABLES = varname2 pairname 2. *Specify the names of two related variables under testing.* The parameter is required. You must specify two variable names per pair.

NULLVALUE = value. *Specify the null value in the Bayes factor estimation.* The parameter is required. Only one value is allowed (the default value is 0) and the parameter is valid only when **DISTRIBUTION = NORMAL**.

[ALTERGVALUE = value]. *Specify the value to define $\psi^2 = g\sigma^2_x$ in the Bayes factor estimation.* Only one value > 0 is allowed and the parameter is valid only when **DISTRIBUTION = NORMAL**. When **VARIANCE** is specified, the parameter must be specified with 1 as the default setting. When **VARIANCE** is not specified, you can either specify a fixed *g* or omit the value to integrate it out.

PRIOR Subcommand

PRIOR specifies the prior distribution.

VARDIST. *Specify the prior distribution for the variance parameter.* Valid when neither the data variance nor **PRECDIST** is specified. The diffuse prior is the default setting. When the data variance is specified for some variables, **VARDIST** is ignored for those variables.

- **DIFFUSE**: Specify the diffuse prior $\propto 1$.
- **INVCHISQ(value1 value2)**: Specify the distribution and parameters for inverse -$\chi^2(\nu_0,\sigma^2_0)$, where $\nu_0 > 0$ is the degree of freedom, and $\sigma^2_0 > 0$ is the scale parameter.
- **INVGAMMA(value1 value2)**: Specify the distribution and parameters for inverse-Gamma(α_0,β_0),where $\alpha_0 > 0$ is the shape parameter, and $\beta_0 > 0$ is the scale parameter.
- **JEFFREYS2**: Specify the noninformative prior $\propto 1/\sigma^2_0$.
- **JEFFREYS4**: Specify the noninformative prior $\propto 1/\sigma^4_0$.

PRECDIST. *Specify the prior distribution for the precision parameter.* The parameter is valid when neither the data variance nor **VARDIST** are specified. The subcommand is an mutually exclusive to **VARDIST**.

- **GAMMA(value1 value2)**: Specify the distribution and parameters for Gamma(α_0,β_0), where $\alpha_0 > 0$ is the shape parameter, and β_0 is the scale parameter.
- **CHISQ(value)**: Specify the distribution and parameters for $\chi^2(\nu_0)$, where $\nu_0 > 0$ is the degree of freedom.

MEANDIST. *Specify the prior distribution for the mean parameter conditional on the variance or the precision parameter.* **DIFFUSE** is the default setting.

- **DIFFUSE**: Specify the diffuse prior $\propto 1$.
- **NORMAL(value1 value2)**: Specify the distribution and parameters for Normal(μ_0,$\kappa_0^{-1}\sigma^2_0$) on variance or Normal(μ_0,κ_0/σ^2_0) on precision, where $\mu_0 \in (-\infty,\infty)$ and $\sigma^2 > 0$.

If **VARIANCE** is specified for a **/DATA** subcommand:

- **VARDIST**, if specified, has no affect on **VARIABLES** specified for the **/DATA** subcommand.
- When **MEANDIST = NORMAL(value1 value2)**, both **value1** and **value2** are required.

When **VARIANCE** is not specified for a **/DATA** subcommand:

- **VARDIST** or **PRECDIST** must be specified for normal **VARIABLES**.
- When **VARDIST = INVCHISQ** or **INVGAMMA**, or **PRECDIST = GAMMA** or **CHISQ**
  - When **MEANDIST = NORMAL(value1 value2)**, only **value1** takes effect (**value2** is ignored).
  - When **MEANDIST = DIFFUSE**, **VARDIST** and **PRECDIST** are ignored, and **VARDIST = DIFFUSE** is used.
- When **VARDIST = DIFFUSE**, **JEFFREYS2**, or **JEFFREYS4**:
  - When **MEANDIST = NORMAL(value1 value2)**, **NORMAL** is ignored, and **MEANDIST =DIFFUSE** is used.

KAPPA = value. *Specify the value of κ_0 in Normal(μ_0,$\kappa^{-1}_0\sigma^2_0$) or Normal(μ_0,κ_0/σ^2_0).* Must be a single value > 0 (1 is the default value).

BAYES REPEATED

BAYES REPEATED is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
BAYES REPEATED {variable_list}
     [/CRITERIA
          [ CILEVEL = {95**} {value}]
          [ SEED = {RANDOM**} {2000000} {integer}]
          [ MCSAMPLES = {30000**} {integer}]
     /INFERENCE
          ANALYSIS = {POSTERIOR**} {BAYESFACTOR} {BOTH}
     /DESIGN
          [ REGWGT = {variable}]
     [/ESTBF
          COMPUTATION = {BIC**} {ROUDER}]
     [/PLOT
          MEANS = {group_list}]
```

** Default if the subcommand or keyword is omitted.

Release History

Release 26.0

- Command introduced

Overview

BAYES REGRESSION {variable_list} invokes the Bayesian One-way Repeated Measures ANOVA procedure, and specifies the variables for repeated measures. Different variables are separated by a blank space. The variable_list value must be ≥ 2.

Options

Analysis options
You can specify the significance level for computing credible intervals and the tolerance value for numerical methods.

Model design
You can specify the model design for the analysis, which includes repeated measure variables and regression weights.

Bayesian analysis method
You can specify POSTERIOR, BAYESFACTOR, or BOTH.

Bayes factor settings
You can specify the approach that is used to estimate the Bayes factor.

Output plots
You can control the plots that are output.

Basic specification

- BAYES REGRESSION {variable_list} invokes the Bayesian One-way Repeated Measures ANOVA procedure, and specifies the variables for repeated measures.

Subcommand order

- The subcommands can be named in any order.

Syntax rules

- Only one regression weight variable is allowed.
- The variable_list value must be ≥ 2.

- For the /PLOT subcommand, group_list must be either a subset of, or the same as, variable_list.

CRITERIA Subcommand

CRITERIA controls the statistical criteria used in building the Bayesian One-way Repeated Measures ANOVA models. The default criteria are noted in the description of each keyword below.

CILEVEL = value. *Specify the significance level for computing credible intervals.* An optional parameter to specify the significance level for estimating credible intervals. Must be a single double value $\in (0,100)$. The default level is 95%

SEED = integer. *Specify a random seed set for Rouder's method.* An optional parameter to specify a random seed for Rouder's method. It is only in effect when /ESTBF COMPUTATION = ROUDER. By default, the procedure assigns a random seed. You can set a number to overwrite the random seed. The number must be a positive integer $\in [1,2147483647]$. The default setting is 2,000,000.

MCSAMPLES = integer. *Specify the number of points that are sampled for the Monte Carlo approximation.* This optional parameter is only in effect when /ESTBF COMPUTATION = ROUDER. 30,000 is the default setting. The value must be an integer $\in [10^3,10^6]$.

DESIGN Subcommand

DESIGN specifies the model design for the analysis.

REGWGT = variable. *Specify the regression weights.* The parameter is optional. Only a single, numerical variable is allowed.

INFERENCE Subcommand

INFERENCE controls which Bayesian analysis method is used to make statistical inference.

ANALYSIS. *Specify the Bayesian analysis method that is used to make statistical inference.* Only one value is allowed. Posterior distribution characterization is the default setting (**ANALYSIS = POSTERIOR**). When **ANALYSIS = BAYESFACTOR**, only the Bayes-factor estimation procedure is invoked. When **ANALYSIS = BOTH**, both of the procedures are used.

ESTBF Subcommand

ESTBF specifies the approach that is used to estimate Bayes factors.

COMPUTATION = value. *Specifies the approach to estimate Bayes factors.* Two different keywords are offered. BIC is the default setting.

- BIC: Invokes the BIC approximation method. This is the default setting.
- ROUDER: Invokes Rouder's method for mixed designs.

PLOT Subcommand

PLOT controls the plots to be output.

MEANS = {group_list}. *Specify the group means to be plotted.* This optional parameter allows you to specify the variable names to be plotted. Note that group_list must be either a subset of, or the same as variable_list.

BEGIN DATA-END DATA

```
BEGIN DATA
data records
END DATA
```

Example

```
BEGIN DATA
1  3424  274 ABU DHABI 2
2 39932   86 AMSTERDAM 4
3  8889  232 ATHENS
4  3424  294 BOGOTA    3
END DATA.
```

Overview

BEGIN DATA and END DATA are used when data are entered within the command sequence (inline data). BEGIN DATA and END DATA are also used for inline matrix data. BEGIN DATA signals the beginning of data lines and END DATA signals the end of data lines.

Basic Specification

The basic specification is BEGIN DATA, the data lines, and END DATA. BEGIN DATA must be specified by itself on the line that immediately precedes the first data line. END DATA is specified by itself on the line that immediately follows the last data line.

Syntax Rules

- BEGIN DATA, the data, and END DATA must precede the first procedure.
- The command terminator after BEGIN DATA is optional. It is best to leave it out so that the program will treat inline data as one continuous specification.
- END DATA must always begin in column 1. It must be spelled out in full and can have only one space between the words END and DATA. Procedures and additional transformations can follow the END DATA command.
- Data lines must *not* have a command terminator. For inline data formats, see DATA LIST.
- Inline data records are limited to a maximum of 80 columns. (On some systems, the maximum may be fewer than 80 columns.) If data records exceed 80 columns, they must be stored in an external file that is specified on the FILE subcommand of the DATA LIST (or similar) command.

Operations

- When the program encounters BEGIN DATA, it begins to read and process data on the next input line. All preceding transformation commands are processed as the working file is built.
- The program continues to evaluate input lines as data until it encounters END DATA, at which point it begins evaluating input lines as commands.
- No other commands are recognized between BEGIN DATA and END DATA.
- The INCLUDE command can specify a file that contains BEGIN DATA, data lines, and END DATA . The data in such a file are treated as inline data. Thus, the FILE subcommand should be omitted from the DATA LIST (or similar) command.
- When running the program from prompts, the prompt DATA> appears immediately after BEGIN DATA is specified. After END DATA is specified, the command line prompt returns.

Examples

```
DATA LIST /XVAR 1 YVAR ZVAR 3-12 CVAR 14-22(A) JVAR 24.
BEGIN DATA
```

```
1  3424  274 ABU DHABI 2
2 39932   86 AMSTERDAM 4
3  8889  232 ATHENS
4  3424  294 BOGOTA    3
5 11323  332 LONDON    3
6   323  232 MANILA    1
7  3234  899 CHICAGO   4
8 78998 2344 VIENNA    3
9  8870  983 ZURICH    5
END DATA.
MEANS XVAR BY JVAR.
```

- DATA LIST defines the names and column locations of the variables. The FILE subcommand is omitted because the data are inline.
- There are nine cases in the inline data. Each line of data completes a case.
- END DATA signals the end of data lines. It begins in column 1 and has only a single space between END and DATA.

BEGIN EXPR-END EXPR

BEGIN EXPR-END EXPR is available in the Statistics Base Edition.

```
BEGIN EXPR
/OUTFILE PREPXML='filespec'

variable definition statements
COMPUTE statements

END EXPR
```

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 21.0

- Command block introduced as SIMPREP BEGIN-SIMPREP END.

Release 23.0

- SIMPREP BEGIN-SIMPREP END deprecated. Command block renamed to BEGIN EXPR-END EXPR.

Example for SIMPLAN

```
BEGIN EXPR
/OUTFILE PREPXML='/models/mymodel.xml'.
NUMERIC price volume fixed unit_cost_materials unit_cost_labor.
COMPUTE revenue = price*volume.
COMPUTE expenses = fixed + volume*(unit_cost_materials + unit_cost_labor).
COMPUTE profit = revenue - expenses.
END EXPR.
```

Example for TCM ANALYSIS

```
BEGIN EXPR
/OUTFILE PREPXML='/scenarios/myscenarios.xml'.
COMPUTE advertising = 1.2*advertising.
END EXPR.
```

Overview

BEGIN EXPR indicates the beginning of a block of statements that define a set of expressions for one or more variables. Expressions are specified with COMPUTE statements. The END EXPR command terminates the block and writes an XML file that contains the specifications for the expressions. The XML file is used as input to one of the following commands that then consumes the expressions:

- The SIMPLAN command creates a simulation plan for a custom model that is defined by the expressions.
- The TCM ANALYSIS command uses the expressions to generate scenario values.

Basic Specification

The only specification for BEGIN EXPR is the command name followed by the OUTFILE subcommand with the PREPXML keyword specifying the file where the results are written. The only specification for END EXPR is the command name.

Syntax Rules

- The OUTFILE subcommand is required.
- Equal signs (=) shown in the syntax chart are required.
- Subcommand names and keywords must be spelled in full.

- With the IBM SPSS Statistics Batch Facility (available only with IBM SPSS Statistics Server), use the -i switch when submitting command files that contain BEGIN EXPR-END EXPR blocks.

Limitations

- COMPUTE statements within BEGIN EXPR-END EXPR blocks support a limited set of functions for building expressions. See the topic "Specifying expressions " on page 238 for more information.
- BEGIN EXPR-END EXPR blocks can be contained in command syntax files run via the INSERT command, with the default SYNTAX=INTERACTIVE setting.
- BEGIN EXPR-END EXPR blocks cannot be contained within command syntax files run via the INCLUDE command.
- Custom simulation models created with BEGIN EXPR-END EXPR do not support systems of simultaneous equations or equations that are non-linear in the target variable. They also do not support equations with string targets.

Operations

- COMPUTE statements that are used in BEGIN EXPR-END EXPR blocks do not act on the active dataset.

Related information

"Specifying expressions " on page 238

OUTFILE subcommand

The OUTFILE subcommand of BEGIN EXPR saves an XML-format file that specifies the expressions.

PREPXML
Specifies the XML-format file. Enclose file specifications in quotation marks and specify full file names. BEGIN EXPR does not supply file extensions. If the file specification refers to an existing file, then the file is overwritten.

Note: The optional combination of an asterisk (*) and a backslash (\) preceding the XML file name specifies that the file is a temporary file--for example, PREPXML='*\myexpressions.xml'.

Specifying expressions

Expressions for temporal causal model scenarios

You can create expressions for computing scenario values for use with the TCM ANALYSIS command. The structure of a BEGIN EXPR-END EXPR block for defining scenario expressions is as follows:

```
BEGIN EXPR
/OUTFILE PREPXML='filespec'.
COMPUTE statements
END EXPR.
```

- You can include multiple expressions, each for a different scenario, in a single BEGIN EXPR-END EXPR block. Each expression can be defined by a single COMPUTE statement or by a set of coupled COMPUTE statements. Coupled statements are evaluated in the order in which they are specified, as is the case for any sequence of COMPUTE statements.
- Each variable in an expression must either exist in the active dataset and be an input or target in the model system, or be defined by a prior COMPUTE statement in the BEGIN EXPR-END EXPR block.
- You cannot reassign a variable in a COMPUTE statement. For example, you cannot specify COMPUTE advertising=1.1*advertising.

Example

This example specifies expressions for two scenarios that are based on the same root field *advertising*.

```
BEGIN EXPR
/OUTFILE PREPXML='/scenarios/myscenarios.xml'.
COMPUTE advert_10_pct = 1.1*advertising.
COMPUTE advert_20_pct = 1.2*advertising.
END EXPR.
```

- The first COMPUTE statement defines a scenario whose values are 10 percent larger than the values of the root field. The second COMPUTE statement defines a scenario whose values are 20 percent larger than the values of the root field.
- The target variable of each COMPUTE statement identifies the expression and is used in the TCM ANALYSIS command to reference the expression.

Expressions for custom simulation models

You can create expressions that define custom simulation models for use with the SIMPLAN command. A custom simulation model consists of a set of equations that specify the relationship between a set of targets and a set of inputs. The relationship between each target and its associated inputs is specified with a COMPUTE statement. In addition, variable definition commands must be provided for all input fields that do not exist in the active dataset. The structure of a BEGIN EXPR-END EXPR block for defining custom simulation models is as follows:

```
BEGIN EXPR
/OUTFILE PREPXML='filespec'.
NUMERIC or STRING statements
VARIABLE LEVEL statements
VALUE LABELS statements
COMPUTE statements
END EXPR.
```

- You must include a NUMERIC or STRING statement to define each input that is not in the active dataset. Inputs that are in the active dataset, however, must not be included on NUMERIC or STRING statements. Targets (which can only be numeric) are defined by COMPUTE statements and do not need to be defined with NUMERIC statements.
- By default, the measurement level for all targets and for all inputs not in the active dataset is continuous. Use VARIABLE LEVEL statements to specify the measurement level for targets and such inputs that are ordinal or nominal. For targets, the measurement level determines the set of output charts and tables that are generated. For inputs that will be simulated, the measurement level determines the default set of distributions used when fitting inputs to historical data.
- Use VALUE LABELS statements to define any value labels for targets and for inputs that are not in the active dataset. Value labels are used in output charts and tables.
- For inputs that are in the active dataset, measurement levels and value labels are taken from the active dataset. You can override the settings from the active dataset by specifying VARIABLE LEVEL and VALUE LABELS statements for those inputs, within the BEGIN EXPR-END EXPR block.
- Use a separate COMPUTE statement for each equation in your model. The equations may be coupled but are evaluated in the order in which they are specified, as is the case for any sequence of COMPUTE statements.

Examples

This example creates a custom model based on an equation that relates the target *revenue* to the inputs *price* and *volume*, where *volume* is a field in the active dataset but *price* is not.

```
BEGIN EXPR
/OUTFILE PREPXML='/models/mymodel.xml'.
NUMERIC price.
COMPUTE revenue = price*volume.
END EXPR.
```

This example creates a custom model based on a set of three equations that specify *profit* as a function of both *revenue* and *expenses*. None of the inputs are fields in the active dataset.

```
BEGIN EXPR
/OUTFILE PREPXML='/models/mymodel.xml'.
NUMERIC price volume fixed unit_cost_materials unit_cost_labor.
COMPUTE revenue = price*volume.
```

```
COMPUTE expenses = fixed + volume*(unit_cost_materials + unit_cost_labor).
COMPUTE profit = revenue - expenses.
END EXPR.
```

- The NUMERIC statement defines the five inputs that are used in the model since none of the inputs are fields in the active dataset.
- Although *revenue* and *expenses* are inputs to *profit*, they are defined by COMPUTE statements, so they do not need to be defined by NUMERIC statements.
- The COMPUTE statement for *profit* depends on *revenue* and *expenses* so the COMPUTE statements for *revenue* and *expenses* precede the one for *profit*.

Supported functions and operators

COMPUTE statements within BEGIN EXPR-END EXPR blocks support the following set of functions and operators for building expressions.

Table 18. Arithmetic operators and functions

Symbol or keyword	Definition
+	Addition
-	Subtraction
*	Multiplication
/	Division
**	Exponentiation
ABS	Absolute value
EXP	Exponential function
LG10	Base 10 logarithm
LN	Natural logarithm
MAX	Maximum of list
MIN	Minimum of list
MOD	Modulo
RND	Round
SQRT	Square root
TRUNC	Truncation
&	Logical AND
|	Logical OR
~	Logical NOT
=	Equal to
~=	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
( )	Grouping

Alternative forms of relational operators, such as AND instead of &, are supported. For a complete list, see the section on “Logical expressions” on page 82.

Related information

“Overview ” on page 237
“SIMPLAN” on page 1815

BEGIN GPL-END GPL

```
BEGIN GPL
  gpl specification
END GPL
```

Release History

Release 14.0

- Command introduced.

Example

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat COUNT()
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: count=col(source(s), name("COUNT"))
  GUIDE: axis(dim(1), label("Employment Category"))
  GUIDE: axis(dim(2), label("Count"))
  ELEMENT: interval(position(jobcat*count))
END GPL.
```

If you are looking for more details about GPL, see the *GPL Reference Guide* on the manuals CD.

Overview

BEGIN GPL and END GPL are used when Graphics Production Language (GPL) code is entered within the command sequence (inline graph specification). BEGIN GPL and END GPL must follow a GGRAPH command, without any blank lines between BEGIN GPL and the command terminator line for GGRAPH. Only comments are allowed between BEGIN GPL and the command terminator line for GGRAPH. BEGIN GPL must be at the start of the line on which it appears, with no preceding spaces. BEGIN GPL signals the beginning of GPL code, and END GPL signals the end of GPL code.

For more information about GGRAPH, see “GGRAPH” on page 819. See the *GPL Reference Guide* on the manuals CD for more details about GPL. The examples in the GPL documentation may look different compared to the syntax pasted from the Chart Builder. The main difference is when aggregation occurs. See “Working with the GPL ” on page 829 for information about the differences. See “GPL Examples ” on page 834 for examples with GPL that is similar to the pasted syntax.

Syntax Rules

- Within a GPL block, only GPL statements are allowed.
- Strings in GPL are enclosed in quotation marks. You cannot use single quotes (apostrophes).
- With the IBM SPSS Statistics Batch Facility (available only with IBM SPSS Statistics Server), use the -i switch when submitting command files that contain GPL blocks.

Scope and Limitations

- GPL blocks cannot be nested within GPL blocks.
- GPL blocks cannot be contained within DEFINE-!ENDDEFINE macro definitions.
- GPL blocks can be contained in command syntax files run via the INSERT command, with the default SYNTAX=INTERACTIVE setting.
- GPL blocks cannot be contained within command syntax files run via the INCLUDE command.

BEGIN PROGRAM-END PROGRAM

BEGIN PROGRAM-END PROGRAM is available in the IBM SPSS Statistics Programmability Extension. It is not available in Statistical Services for SQL Server 2005.

```
BEGIN PROGRAM [{PYTHON3**}].
              {R       }
programming language-specific statements
END PROGRAM.
```

Release History

Release 14.0

- Command introduced.

Release 24.0

- PYTHON3 keyword introduced.

Overview

BEGIN PROGRAM-END PROGRAM provides the ability to integrate the capabilities of external programming languages with IBM SPSS Statistics. One of the major benefits of these program blocks is the ability to add **jobwise** flow control to the command stream. Outside of program blocks, IBM SPSS Statistics can execute **casewise** conditional actions, based on criteria that evaluate each case, but jobwise flow control, such as running different procedures for different variables based on data type or level of measurement or determining which procedure to run next based on the results of the last procedure is much more difficult. Program blocks make jobwise flow control much easier to accomplish. With program blocks, you can control the commands that are run based on many criteria, including:

- Dictionary information (e.g., data type, measurement level, variable names)
- Data conditions
- Output values
- Error codes (that indicate if a command ran successfully or not)

You can also read data from the active dataset to perform additional computations, update the active dataset with results, create new datasets, and create custom pivot table output.

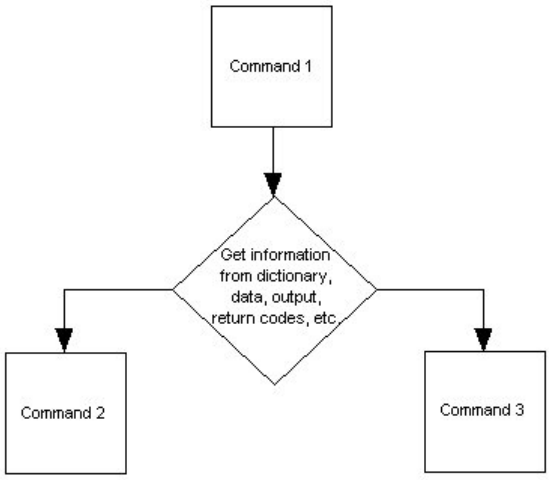


Figure 18. Jobwise Flow Control

Operations

- BEGIN PROGRAM signals the beginning of a set of code instructions controlled by an external programming language.
- After BEGIN PROGRAM is executed, other commands do not execute until END PROGRAM is encountered.

Syntax rules

- Within a program block, only statements recognized by the specified programming language are allowed.
- Command syntax generated within a program block must follow **interactive** syntax rules. See the topic for more information.
- Within a program block, each line should not exceed 251 bytes (although syntax generated by those lines can be longer).
- With the IBM SPSS Statistics Batch Facility (available only with IBM SPSS Statistics Server), use the -i switch when submitting command files that contain program blocks. All command syntax (not just the program blocks) in the file must adhere to interactive syntax rules.
- The keyword PYTHON3 indicates that the Python 3 processor is used to process the Python statements.

Within a program block, the programming language is in control, and the syntax rules for that programming language apply. Command syntax generated from within program blocks must always follow interactive syntax rules. For most practical purposes this means command strings you build in a programming block must contain a period (.) at the end of each command.

Scope and limitations

- Programmatic variables created in a program block cannot be used outside of program blocks.
- Program blocks cannot be contained within DEFINE-!ENDDEFINE macro definitions.
- Program blocks can be contained in command syntax files run via the INSERT command, with the default SYNTAX=INTERACTIVE setting.
- Program blocks cannot be contained within command syntax files run via the INCLUDE command.

Using external programming languages

Use of the IBM SPSS Statistics Programmability Extension requires an Integration Plug-in for an external language. Integration Plug-ins supported for use with BEGIN PROGRAM-END PROGRAM blocks are available for the Python and R programming languages. For information, see How to Get Integration Plug-ins, available from **Core System** > **Frequently Asked Questions** in the Help system. Documentation for the plug-ins is available from the topics Integration Plug-in for Python and Integration Plug-in for R in the Help system.

Resources for use with Integration Plug-ins are available on the IBM SPSS Statistics community at https://www.ibm.com/community/spss-statistics . Many of the resources are packaged as extension bundles that you can download from the Extension Hub. It is available from the menus by choosing **Extensions > Extension Hub**.

BOOTSTRAP

B00TSTRAP is available in the Base edition.

```
BOOTSTRAP
  [/SAMPLING METHOD={SIMPLE**                    }]
                    {STRATIFIED(STRATA=varlist)  }
                    {RESIDUAL({RESIDUALS=varlist})}
                              {PREDICTED=varlist})
                    {WILD({RESIDUALS=varlist})   }
                          {PREDICTED=varlist})
  [/VARIABLES [TARGET=varlist] [INPUT=varlist]]
  [/CRITERIA [CILEVEL={95** } [CITYPE={PERCENTILE**}]]
                      {value}         {BCA         }
             [NSAMPLES={1000**}]
                       {int   }
  [/MISSING [USERMISSING={EXCLUDE**}]]
                         {INCLUDE  }.
```

** Default if the subcommand or keyword is omitted.

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

Release History

Release 18

- Command introduced.

Example

```
BOOTSTRAP.
```

Overview

Bootstrapping is a method for deriving robust estimates of standard errors and confidence intervals for estimates such as the mean, median, proportion, odds ratio, correlation coefficient or regression coefficient. It may also be used for constructing hypothesis tests. Bootstrapping is most useful as an alternative to parametric estimates when the assumptions of those methods are in doubt (as in the case of regression models with heteroscedastic residuals fit to small samples), or where parametric inference is impossible or requires very complicated formulas for the calculation of standard errors (as in the case of computing confidence intervals for the median, quartiles, and other percentiles).

The B00TSTRAP command signals the beginning of temporary bootstrap samples that are in effect only for the next procedure. See the "Procedures that Support Bootstrapping" topic in the Statistics help system for a list of procedures that support bootstrapping.

Options

Resampling method. Simple, stratified, and residuals bootstrap resampling are supported. You can also specify the number of bootstrap samples to take.

Pooling method. Choose between percentile and BCa methods for computing confidence intervals. You can also specify the confidence level.

Basic Specification

The basic specification is the B00TSTRAP command.

By default, B00TSTRAP draws 1000 samples using simple bootstrap resampling. When the procedure following B00TSTRAP is run, the pooling algorithm produces 95% confidence intervals using the percentile method. Since no variables have been specified, no records are excluded from resampling.

Syntax Rules

- All subcommands are optional.

- Subcommands may be specified in any order.
- Only a single instance of each subcommand is allowed.
- An error occurs if a keyword is specified more than once within a subcommand.
- Parentheses, equals signs, and slashes shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not allowed.
- Any split variable defined on the SPLIT FILE command may not be used on the BOOTSTRAP command.

Limitations

- BOOTSTRAP does not work with multiply imputed datasets. If there is an *Imputation_* variable in the dataset, running BOOTSTRAP will cause an error.
- BOOTSTRAP should not be used in conjunction with the N OF CASES command.

Examples

Simple Resampling; Maintaining a Consistent Case Basis

```
BOOTSTRAP.
DESCRIPTIVES VARIABLES=var1 var2 var3
  /MISSING=VARIABLE.
```

- The BOOTSTRAP command requests 1000 bootstrap samples.
- No variables are specified on the BOOTSTRAP command, so no records are deleted from the resampling. This allows the DESCRIPTIVES procedure to use variablewise deletion of missing values on the full set of records; however, the case basis will be inconsistent across bootstrap resamples, and inferences made from the results would be questionable.

```
BOOTSTRAP
  /VARIABLES ANALYSIS(INPUT=var1 var2 var3).
DESCRIPTIVES VARIABLES=var1 var2 var3
  /STATISTICS MEAN STDDEV MIN MAX
  /MISSING=VARIABLE.
```

- This is the same as the previous analysis, but variables *var1*, *var2*, and *var3* are used to determine the case basis for resampling. Records with missing values on any of these variables are deleted from the analysis.
- The DESCRIPTIVES procedure following BOOTSTRAP is run on the bootstrap samples.
- The STATISTICS subcommand produces the mean, standard deviation, minimum, and maximum for variables *var1*, *var2*, and *var3* on the original data. Additionally, pooled statistics are produced for the mean and standard deviation.
- Even though the MISSING subcommand specifies variablewise deletion of missing values, the listwise deletion performed by BOOTSTRAP is what determines the case basis. In effect, the MISSING specification on DESCRIPTIVES is irrelevant here.

Stratified Resampling

```
BOOTSTRAP
  /VARIABLES SAMPLING(STRATA=strataVar)
             ANALYSIS(INPUTS=var1).
DESCRIPTIVES var1.
```

- The BOOTSTRAP command requests 1000 bootstrap samples stratified by *strataVar*.
- Variables *var1* and *strataVar* are used to determine the case basis for resampling. Records with missing values on these variables are deleted from the analysis.
- The DESCRIPTIVES procedure following BOOTSTRAP is run on the bootstrap samples, and produces the mean, standard deviation, minimum, and maximum for the variable *var1* on the original data. Additionally, pooled statistics are produced for the mean and standard deviation.

SAMPLING Subcommand

The SAMPLING subcommand is used to specify the sampling method and any associated variables.

- If SAMPLING is not specified, the procedure performs simple bootstrap resampling..

SIMPLE. *Simple resampling.* This performs case resampling with replacement from the original dataset. This is the default.

STRATIFIED (STRATA = varlist). *Stratified resampling.* Specify one or more variables that define strata within the dataset. This performs case resampling with replacement from the original dataset, *within* the strata defined by the cross-classification of strata variables, preserving the size of each stratum. Stratified bootstrap sampling can be useful when units within strata are relatively homogeneous while units across strata are very different.

RESIDUAL (RESIDUALS=varlist | PREDICTED=varlist). *Residual resampling.* Specify one or more variables containing residuals from fitting a model to the data. The model that produced the residuals should ideally be the same model that follows BOOTSTRAP. A residual sample is drawn by replacing each target variable value with that case's predicted value plus a residual sampled from the entire original set of residuals.

Specify PREDICTED as an alternative to RESIDUALS when the model residuals are not immediately available but the predicted values are. Specify one or more variables containing predicted values from fitting a model to the data.

If RESIDUAL is specified, the TARGET keyword is required and the variables specified on RESIDUAL should be the residuals (or predicted values) for, and match the order of, the variables specified on TARGET.

WILD (RESIDUALS=varlist | PREDICTED=varlist). *Wild bootstrap resampling.* Specify one or more variables containing residuals from fitting a model to the data. The model that produced the residuals should ideally be the same model that follows BOOTSTRAP. A wild sample is drawn by replacing each target variable value with that case's predicted value plus either the case's residual or the negative of the case's residual.

Specify PREDICTED as an alternative to RESIDUALS when the model residuals are not immediately available but the predicted values are. Specify one or more variables containing predicted values from fitting a model to the data.

If WILD is specified, the TARGET keyword is required and the variables specified on WILD should be the residuals (or predicted values) for, and match the order of, the variables specified on TARGET.

VARIABLES Subcommand

The VARIABLES subcommand is used to specify the target and inputs.

- If VARIABLES is not specified, the procedure performs bootstrap resampling on all the records in the dataset. TARGET is required when performing residual resampling, but these specifications are otherwise *technically* optional. However, these variables are used to determine the case basis for bootstrap resampling, so it is important to specify these variables when there are missing values in the data.

TARGET=varlist. *Target variables.* Specify one or more variables that will be used as targets (responses, dependent variables) in the procedure following BOOTSTRAP.

INPUT=varlist. *Input variables.* Specify one or more variables that will be used as inputs (factors, covariates) in the procedure following BOOTSTRAP.

CRITERIA Subcommand

The CRITERIA subcommand controls pooling options and the number of bootstrap samples to take.

CILEVEL = number. *Confidence interval level.* Specify a number greater than or equal to 0, and less than 100. The default value is 95. Note that bootstrapping can only support intervals up to confidence level 100*(1–2/(NSAMPLES+1)).

CITYPE = PERCENTILE | BCA. *Confidence interval type.* Specify PERCENTILE for percentile intervals or BCA for BCa intervals. The default value is PERCENTILE.

NSAMPLES = integer . *Number of bootstrap samples.* Specify a positive integer. The default value is 1000.

MISSING Subcommand

The MISSING subcommand is used to control whether user-missing values for categorical variables are treated as valid values. By default, user-missing values for categorical variables are treated as invalid. The setting used here should be the same as that used on the procedure following the BOOTSTRAP command.

- Cases with invalid values are deleted listwise.
- The MISSING subcommand defines categorical variables as variables with measurement level set at Ordinal or Nominal in the data dictionary. Use the VARIABLE LEVEL command to change a variable's measurement level.
- User-missing values for continuous variables are always treated as invalid.
- System-missing values for any variables are always treated as invalid.

USERMISSING=EXCLUDE. *User-missing values for categorical variables are treated as invalid.* This is the default.

USERMISSING=INCLUDE. *User-missing values for categorical variables are treated as valid values.*

BREAK

```
BREAK
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

Overview

BREAK controls looping that cannot be fully controlled with IF clauses. Generally, BREAK is used within a DO IF–END IF structure. The expression on the DO IF command specifies the condition in which BREAK is executed.

Basic Specification

- The only specification is the keyword BREAK. There are no additional specifications.
- BREAK must be specified within a loop structure. Otherwise, an error results.

Operations

- A BREAK command inside a loop structure but not inside a DO IF–END IF structure terminates the first iteration of the loop for all cases, since no conditions for BREAK are specified.
- A BREAK command within an inner loop terminates only iterations in that structure, not in any outer loop structures.

Examples

```
VECTOR         #X(10).
LOOP           #I = 1 TO #NREC.
+  DATA LIST      NOTABLE/ #X1 TO #X10 1-20.
+  LOOP           #J = 1 TO 10.
+     DO IF          SYSMIS(#X(#J)).
+        BREAK.
+     END IF.
+     COMPUTE        X = #X(#J).
+     END CASE.
+  END LOOP.
END LOOP.
```

- The inner loop terminates when there is a system-missing value for any of the variables *#X1* to *#X10*.
- The outer loop continues until all records are read.

CACHE

```
CACHE.
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

Although the virtual active file can vastly reduce the amount of temporary disk space required, the absence of a temporary copy of the “active” file means that the original data source has to be reread for each procedure. For data tables read from a database source, this means that the SQL query that reads the information from the database must be reexecuted for any command or procedure that needs to read the data. Since virtually all statistical analysis procedures and charting procedures need to read the data, the SQL query is reexecuted for each procedure that you run, which can result in a significant increase in processing time if you run a large number of procedures.

If you have sufficient disk space on the computer performing the analysis (either your local computer or a remote server), you can eliminate multiple SQL queries and improve processing time by creating a data cache of the active file with the CACHE command. The CACHE command copies all of the data to a temporary disk file the next time the data are passed to run a procedure. If you want the cache written immediately, use the EXECUTE command after the CACHE command.

- The only specification is the command name CACHE.
- A cache file will not be written during a procedure that uses temporary variables.
- A cache file will not be written if the data are already in a temporary disk file and that file has not been modified since it was written.

Example

```
CACHE.
TEMPORARY.
RECODE alcohol(0 thru .04 = 'sober') (.04 thru .08 = 'tipsy')
  (else = 'drunk') into state.
FREQUENCIES var=state.
GRAPH...
```

No cache file will be written during the FREQUENCIES procedure. It will be written during the GRAPH procedure.

CASEPLOT

```
CASEPLOT VARIABLES=varlist

 [/DIFF={1}]
        {n}

 [/SDIFF={1}]
         {n}

 [/PERIOD=n]

 [/{NOLOG**}]
   {LN     }

 [/ID=varname]

 [/MARK={varname           }]
        {date specification}

 [/SPLIT {UNIFORM**}]
         {SCALE    }

 [/APPLY [='model name']]
```

For plots with one variable:

```
 [/FORMAT=[{NOFILL**}]  [{NOREFERENCE**     }]]
           {LEFT    }    {REFERENCE[(value)]}
```

For plots with multiple variables:

```
 [/FORMAT={NOJOIN**}]
          {JOIN    }
          {HILO    }
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 14.0

- For plots with one variable, new option to specify a value with the REFERENCE keyword on the FORMAT subcommand.

Example

```
CASEPLOT VARIABLES = TICKETS
  /LN
  /DIFF
  /SDIFF
  /PERIOD=12
  /FORMAT=REFERENCE
  /MARK=Y 55 M 6.
```

Overview

CASEPLOT produces a plot of one or more time series or sequence variables. You can request natural log and differencing transformations to produce plots of transformed variables. Several plot formats are available.

Options

Modifying the Variables. You can request a natural log transformation of the variable using the LN subcommand and seasonal and nonseasonal differencing to any degree using the SDIFF and DIFF subcommands. With seasonal differencing, you can also specify the periodicity on the PERIOD subcommand.

Plot Format. With the FORMAT subcommand, you can fill in the area on one side of the plotted values on plots with one variable. You can also plot a reference line indicating the variable mean. For plots with two

or more variables, you can specify whether you want to join the values for each case with a horizontal line. With the ID subcommand, you can label the vertical axis with the values of a specified variable. You can mark the onset of an intervention variable on the plot with the MARK subcommand.

Split-File Processing. You can control how to plot data that have been divided into subgroups by a SPLIT FILE command using the SPLIT subcommand.

Basic Specification

The basic specification is one or more variable names.

- If the DATE command has been specified, the vertical axis is labeled with the *DATE_* variable at periodic intervals. Otherwise, sequence numbers are used. The horizontal axis is labeled with the value scale determined by the plotted variables.

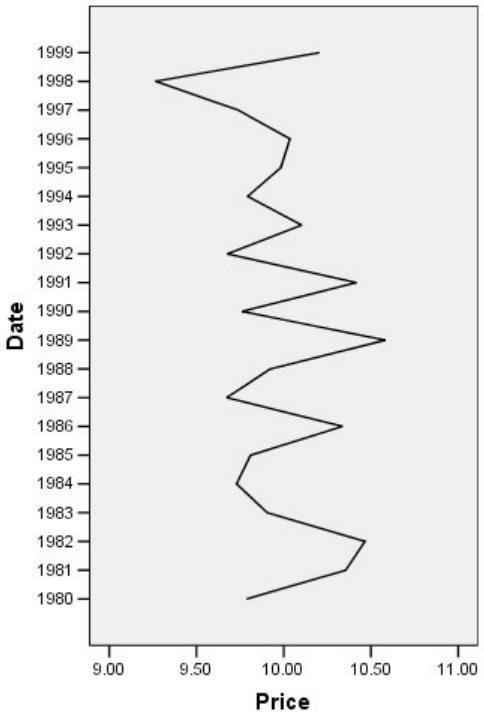


Figure 19. CASEPLOT with DATE variable

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- VARIABLES can be specified only once.
- Other subcommands can be specified more than once, but only the last specification of each one is executed.

Operations

- Subcommand specifications apply to all variables named on the CASEPLOT command.
- If the LN subcommand is specified, any differencing requested on that CASEPLOT command is done on the log-transformed variables.
- Split-file information is displayed as part of the subtitle, and transformation information is displayed as part of the footnote.

Limitations

- A maximum of one VARIABLES subcommand. There is no limit on the number of variables named on the list.

Examples

```
CASEPLOT VARIABLES = TICKETS
  /LN
  /DIFF
  /SDIFF
  /PERIOD=12
  /FORMAT=REFERENCE
  /MARK=Y 55 M 6.
```

- This example produces a plot of *TICKETS* after a natural log transformation, differencing, and seasonal differencing have been applied.
- LN transforms the data using the natural logarithm (base *e*) of the variable.
- DIFF differences the variable once.
- SDIFF and PERIOD apply one degree of seasonal differencing with a periodicity of 12.
- FORMAT=REFERENCE adds a reference line at the variable mean.
- MARK provides a marker on the plot at June, 1955. The marker is displayed as a horizontal reference line.

VARIABLES Subcommand

VARIABLES specifies the names of the variables to be plotted and is the only required subcommand.

DIFF Subcommand

DIFF specifies the degree of differencing used to convert a nonstationary variable to a stationary one with a constant mean and variance before plotting.

- You can specify any positive integer on DIFF.
- If DIFF is specified without a value, the default is 1.
- The number of values displayed decreases by 1 for each degree of differencing.

Example

```
CASEPLOT VARIABLES = TICKETS
  /DIFF=2.
```

- In this example, TICKETS is differenced twice before plotting.

SDIFF Subcommand

If the variable exhibits a seasonal or periodic pattern, you can use the SDIFF subcommand to seasonally difference a variable before plotting.

- The specification on SDIFF indicates the degree of seasonal differencing and can be any positive integer.
- If SDIFF is specified without a value, the degree of seasonal differencing defaults to 1.
- The number of seasons displayed decreases by 1 for each degree of seasonal differencing.
- The length of the period used by SDIFF is specified on the PERIOD subcommand. If the PERIOD subcommand is not specified, the periodicity established on the TSET or DATE command is used (see the PERIOD subcommand below).

PERIOD Subcommand

PERIOD indicates the length of the period to be used by the SDIFF subcommand.

- The specification on PERIOD indicates how many observations are in one period or season and can be any positive integer.
- PERIOD is ignored if it is used without the SDIFF subcommand.
- If PERIOD is not specified, the periodicity established on TSET PERIOD is in effect. If TSET PERIOD is not specified either, the periodicity established on the DATE command is used. If periodicity is not established anywhere, the SDIFF subcommand will not be executed.

Example

```
CASEPLOT VARIABLES = TICKETS
  /SDIFF=1
  /PERIOD=12.
```

- This command applies one degree of seasonal differencing with 12 observations per season to *TICKETS* before plotting.

LN and NOLOG Subcommands

LN transforms the data using the natural logarithm (base *e*) of the variable and is used to remove varying amplitude over time. NOLOG indicates that the data should not be log transformed. NOLOG is the default.

- If you specify LN on CASEPLOT, any differencing requested on that command will be done on the log-transformed variable.
- There are no additional specifications on LN or NOLOG.
- Only the last LN or NOLOG subcommand on a CASEPLOT command is executed.
- If a natural log transformation is requested, any value less than or equal to zero is set to system-missing.
- NOLOG is generally used with an APPLY subcommand to turn off a previous LN specification.

Example

```
CASEPLOT VARIABLES = TICKETS
  /LN.
```

- In this example, *TICKETS* is transformed using the natural logarithm before plotting.

ID Subcommand

ID names a variable whose values will be used as the left-axis labels.

- The only specification on ID is a variable name. If you have a variable named *ID* in your active dataset, the equals sign after the subcommand is required.
- ID overrides the specification on TSET ID.
- If ID or TSET ID is not specified, the left vertical axis is labeled with the *DATE_* variable created by the DATE command. If the *DATE_* variable has not been created, the observation or sequence number is used as the label.

Example

```
CASEPLOT VARIABLES = VARA
  /ID=VARB.
```

- In this example, the values of the variable *VARB* will be used to label the left axis of the plot of *VARA*.

FORMAT Subcommand

FORMAT controls the plot format.

- The specification on FORMAT is one of the keywords listed below.
- The keywords NOFILL, LEFT, NOREFERENCE, and REFERENCE apply to plots with one variable. NOFILL and LEFT are alternatives and indicate how the plot is filled. NOREFERENCE and REFERENCE are alternatives and specify whether a reference line is displayed. One keyword *from each set* can be specified. NOFILL and NOREFERENCE are the defaults.
- The keywords JOIN, NOJOIN, and HILO apply to plots with multiple variables and are alternatives. NOJOIN is the default. Only one keyword can be specified on a FORMAT subcommand for plots with two variables.

The following formats are available for plots of one variable:

NOFILL. *Plot only the values for the variable with no fill.* NOFILL produces a plot with no fill to the left or right of the plotted values. This is the default format when one variable is specified.

LEFT. *Plot the values for the variable and fill in the area to the left.* If the plotted variable has missing or negative values, the keyword LEFT is ignored and the default NOFILL is used instead.

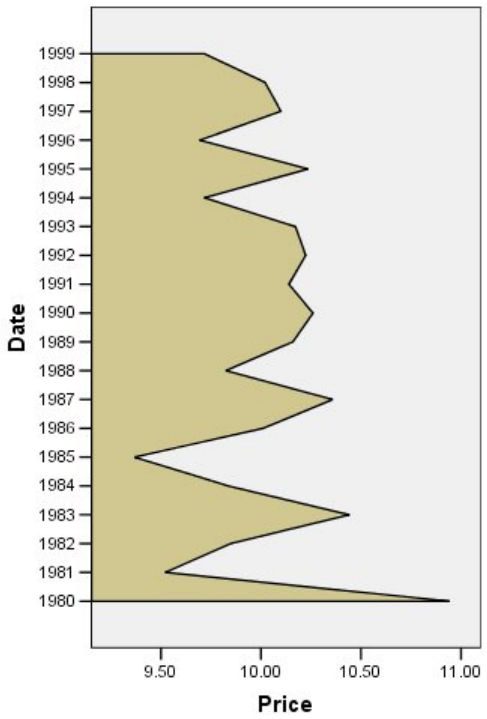


Figure 20. FORMAT=LEFT

NOREFERENCE. *Do not plot a reference line.* This is the default when one variable is specified.

REFERENCE(value) . *Plot a reference line at the specified value or at the variable mean if no value is specified.* A fill chart is displayed as an area chart with a reference line and a non-fill chart is displayed as a line chart with a reference line.

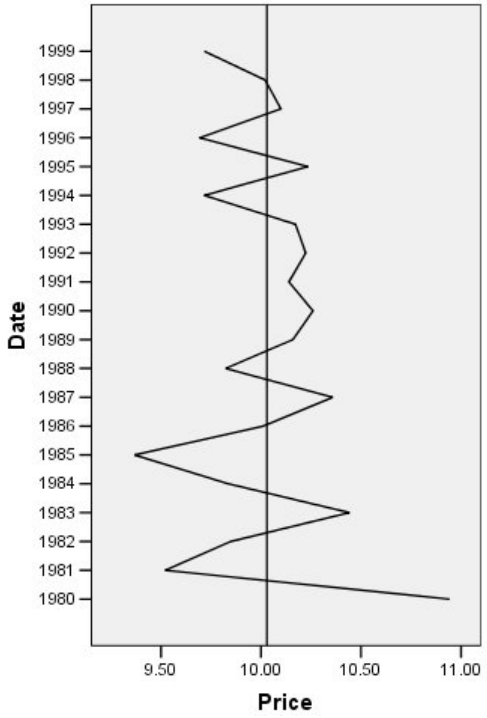


Figure 21. FORMAT=REFERENCE

The following formats are available for plots of multiple variables:

NOJOIN. *Plot the values of each variable named.* Different colors or line patterns are used for multiple variables. Multiple occurrences of the same value for a single observation are plotted using a dollar sign ($). This is the default format for plots of multiple variables.

JOIN. *Plot the values of each variable and join the values for each case.* Values are plotted as described for NOJOIN, and the values for each case are joined together by a line.

HILO. *Plot the highest and lowest values across variables for each case and join the two values together.* The high and low values are plotted as a pair of vertical bars and are joined with a dashed line. HILO is ignored if more than three variables are specified, and the default NOJOIN is used instead.

MARK Subcommand

Use MARK to indicate the onset of an intervention variable.

- The onset date is indicated by a horizontal reference line.
- The specification on MARK can be either a variable name or an onset date if the *DATE_* variable exists.
- If a variable is named, the reference line indicates where the values of that variable change.
- A date specification follows the same format as the DATE command—that is, a keyword followed by a value. For example, the specification for June, 1955, is Y 1955 M 6 (or Y 55 M 6 if only the last two digits of the year are used on DATE).

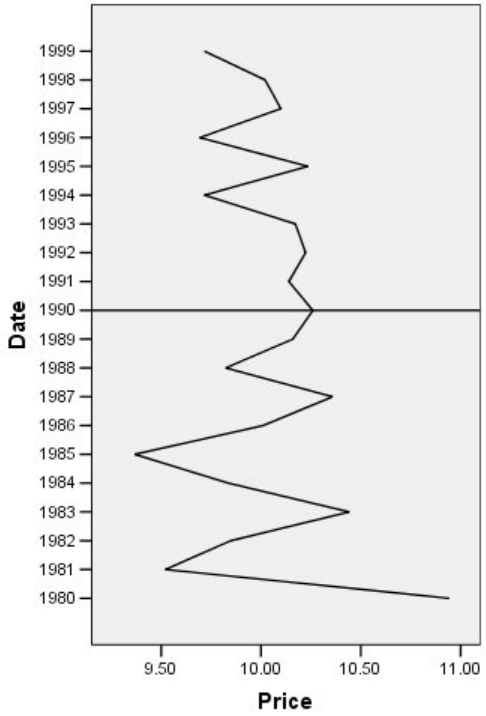


Figure 22. MARK Y=1990

SPLIT Subcommand

SPLIT specifies how to plot data that have been divided into subgroups by a SPLIT FILE command. The specification on SPLIT is either SCALE or UNIFORM.

- If FORMAT=REFERENCE is specified when SPLIT=SCALE, the reference line is placed at the mean of the subgroup. If FORMAT=REFERENCE is specified when SPLIT=UNIFORM, the reference line is placed at the overall mean.

UNIFORM. *Uniform scale.* The horizontal axis is scaled according to the values of the entire dataset. This is the default if SPLIT is not specified.

SCALE. *Individual scale.* The horizontal axis is scaled according to the values of each individual subgroup.

Example

```
SPLIT FILE BY REGION.
CASEPLOT VARIABLES = TICKETS / SPLIT=SCALE.
```

- This example produces one plot for each *REGION* subgroup.
- The horizontal axis for each plot is scaled according to the values of *TICKETS* for each particular region.

APPLY Subcommand

APPLY allows you to produce a caseplot using previously defined specifications without having to repeat the CASEPLOT subcommands.

- The only specification on APPLY is the name of a previous model in quotes. If a model name is not specified, the specifications from the previous CASEPLOT command are used.
- If no variables are specified, the variables that were specified for the original plot are used.
- To change one or more plot specifications, specify the subcommands of only those portions you want to change after the APPLY subcommand.
- To plot different variables, enter new variable names before or after the APPLY subcommand.

Example

```
CASEPLOT VARIABLES = TICKETS
  /LN
  /DIFF=1
  /SDIFF=1
  /PER=12.
CASEPLOT VARIABLES = ROUNDTRP
  /APPLY.
CASEPLOT APPLY
  /NOLOG.
```

- The first command produces a plot of *TICKETS* after a natural log transformation, differencing, and seasonal differencing.
- The second command plots *ROUNDTRP* using the same transformations specified for *TICKETS*.
- The third command produces a plot of *ROUNDTRP* but this time without any natural log transformation. The variable is still differenced once and seasonally differenced with a periodicity of 12.

CASESTOVARS

```
CASESTOVARS
[/ID = varlist]
[/FIXED = varlist]
[/AUTOFIX = {YES**}]
            {NO   }
[/VIND [ROOT = rootname]]
[/COUNT = new variable ["label"]]
[/RENAME varname=rootname varname=rootname ...]
[/SEPARATOR = {"."     }]
              {"string"}]
[/INDEX = varlist]
[/GROUPBY = {VARIABLE**}]
            {INDEX     }]
[/DROP = varlist]
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
CASESTOVARS /ID idvar /INDEX var1.
```

Overview

A **variable** contains information that you want to analyze, such as a measurement or a test score. A **case** is an observation, such as an individual or an institution.

In a **simple data file**, each variable is a single **column** in your data, and each case is a single **row** in your data. So, if you were recording the score on a test for all students in a class, the scores would appear in only one column and there would be only one row for each student.

Complex data files store data in more than one column or row. For example, in a complex data file, information about a case could be stored in more than one row. So, if you were recording monthly test scores for all students in a class, there would be multiple rows for each student—one for each month.

CASESTOVARS restructures complex data that has multiple rows for a case. You can use it to restructure data in which repeated measurements of a single case were recorded in multiple rows (row groups) into a new data file in which each case appears as separate variables (variable groups) in a single row. It replaces the active dataset.

Options

Automatic classification of fixed variables. The values of **fixed variables** do not vary within a row group. You can use the AUTOFIX subcommand to let the procedure determine which variables are fixed and which variables are to become variable groups in the new data file.

Naming new variables. You can use the RENAME, SEPARATOR, and INDEX subcommands to control the names for the new variables.

Ordering new variables. You can use the GROUPBY subcommand to specify how to order the new variables in the new data file.

Creating indicator variables. You can use the VIND subcommand to create indicator variables. An **indicator variable** indicates the presence or absence of a value for a case. An indicator variable has the value of 1 if the case has a value; otherwise, it is 0.

Creating a count variable. You can use the COUNT subcommand to create a count variable that contains the number of rows in the original data that were used to create a row in the new data file.

Variable selection. You can use the DROP subcommand to specify which variables from the original data file are dropped from the new data file.

Basic specification

The basic specification is simply the command keyword.

- If split-file processing is in effect, the basic specification creates a row in the new data file for each combination of values of the SPLIT FILE variables. If split-file processing is not in effect, the basic specification results in a new data file with one row.
- Because the basic specification can create quite a few new columns in the new data file, the use of an ID subcommand to identify groups of cases is recommended.

Subcommand order

Subcommands can be specified in any order.

Syntax rules

Each subcommand can be specified only once.

Operations

- **Original row order.** CASESTOVARS assumes that the original data are sorted by SPLIT and ID variables.
- **Identifying row groups in the original file.** A **row group** consists of rows in the original data that share the same values of variables listed on the ID subcommand. Row groups are consolidated into a single row in the new data file. Each time a new combination of ID values is encountered, a new row is created.
- **Split-file processing and row groups.** If split-file processing is in effect, the split variables are automatically used to identify row groups (they are treated as though they appeared first on the ID subcommand). Split-file processing remains in effect in the new data file unless a variable that is used to split the file is named on the DROP subcommand.
- **New variable groups.** A **variable group** is a group of related columns in the new data file that is created from a variable in the original data. Each variable group contains a variable for each index value or combination of index values encountered.
- **Candidate variables.** A variable in the original data is a candidate to become a variable group in the new data file if it is not used on the SPLIT command or the ID, FIXED, or DROP subcommands and its values vary within the row group. Variables named on the SPLIT, ID, and FIXED subcommands are assumed to not vary within the row group and are simply copied into the new data file.
- **New variable names.** The names of the variables in a new group are constructed by the procedure. For numeric variables, you can override the default naming convention using the RENAME and SEPARATOR subcommands. If there is a single index variable and it is a string, the string values are used as the new variable names. For string values that do not form valid variable names, names of the general form *Vn* are used, where *n* is a sequential integer.
- **New variable formats.** With the exception of names and labels, the dictionary information for all of the new variables in a group (for example, value labels and format) is taken from the variable in the original data.
- **New variable order.** New variables are created in the order specified by the GROUPBY subcommand.
- **Weighted files.** The WEIGHT command does not affect the results of CASESTOVARS. If the original data are weighted, the new data file will be weighted unless the variable that is used as the weight is dropped from the new data file.
- **Selected cases.** The FILTER and USE commands do not affect the results of CASESTOVARS. It processes all cases.

Limitations

The TEMPORARY command cannot be in effect when CASESTOVARS is executed.

Examples

The following is the LIST output for a data file in which repeated measurements for the same case are stored on separate rows in a single variable.

```
insure     caseid    month      bps      bpd

BCBS            1        1      160      100
BCBS            2        1      120       70
BCBS            2        2      130       86
Prucare         1        1      160       94
Prucare         1        2      200      105
Prucare         1        3      180      105
Pruecare        2        1      135       90
```

The commands:

```
SPLIT FILE BY insure.
CASESTOVARS
    /ID=caseid
    /INDEX=month.
```

create a new variable group for *bps* and a new group for *bpd*. The LIST output for the new active dataset is as follows:

- The row groups in the original data are identified by *insure* and *caseid*.
- There are four row groups—one for each combination of the values in *insure* and *caseid*.
- The command creates four rows in the new data file, one for each row group.
- The candidate variables from the original file are *bps* and *bpd*. They vary within the row group, so they will become variable groups in the new data file.
- The command creates two new variable groups—one for *bps* and one for *bpd*.
- Each variable group contains three new variables—one for each unique value of the index variable *month*.

ID subcommand

The ID subcommand specifies variables that identify the rows from the original data that should be grouped together in the new data file.

- If the ID subcommand is omitted, only SPLIT FILE variables (if any) will be used to group rows in the original data and to identify rows in the new data file.
- CASESTOVARS expects the data to be sorted by SPLIT FILE variables and then by ID variables. If split-file processing is in effect, the original data should be sorted on the split variables in the order given on the SPLIT FILE command and then on the ID variables in the order in which they appear in the ID subcommand.
- A variable may appear on both the SPLIT FILE command and the ID subcommand.
- Variables listed on the SPLIT FILE command and on the ID subcommand are copied into the new data file with their original values and dictionary information unless they are dropped with the DROP subcommand.
- Variables listed on the ID subcommand may not appear on the FIXED or INDEX subcommands.
- Rows in the original data for which any ID variable has the system-missing value or is blank are not included in the new data file, and a warning message is displayed.
- ID variables are not candidates to become a variable group in the new data file.

INDEX subcommand

In the original data, a variable appears in a single column. In the new data file, that variable will appear in multiple new columns. The INDEX subcommand names the variables in the original data that should be used to create the new columns. INDEX variables are also used to name the new columns.

Optionally, with the GROUPBY subcommand, INDEX variables can be used to determine the order of the new columns, and, with the VIND subcommand, INDEX variables can be used to create indicator variables.

- String variables can be used as index variables. They cannot contain blank values for rows in the original data that qualify for inclusion in the new data file.
- Numeric variables can be used as index variables. They must contain only non-negative integer values and cannot have system-missing or blank values.
- Within each row group in the original file, each row must have a different combination of values of the index variables.
- If the INDEX subcommand is not used, the index starts with 1 within each row group and increments each time a new value is encountered in the original variable.
- Variables listed on the INDEX subcommand may not appear on the ID, FIXED, or DROP subcommands.
- Index variables are not are not candidates to become a variable group in the new data file.

VIND subcommand

The VIND subcommand creates indicator variables in the new data file. An indicator variable indicates the presence or absence of a value for a case. An indicator variable has the value of 1 if the case has a value; otherwise, it is 0.

- One new indicator variable is created for each unique value of the variables specified on the INDEX subcommand.
- If the INDEX subcommand is not used, an indicator variable is created each time a new value is encountered within a row group.
- An optional rootname can be specified after the ROOT keyword on the subcommand. The default rootname is *ind*.
- The format for the new indicator variables is F1.0.

Example

If the original variables are:

```
insure     caseid     month     bps     bpd
```

and the data are as shown in the first example, the commands:

```
SPLIT FILE BY insure.
CASESTOVARS
    /ID=caseid
    /INDEX=month
    /VIND
    /DROP=caseid bpd.
```

create a new file with the following data:

- The command created three new indicator variables—one for each unique value of the index variable *month*.

COUNT subcommand

CASESTOVARS consolidates row groups in the original data into a single row in the new data file. The COUNT subcommand creates a new variable that contains the number of rows in the original data that were used to generate the row in the new data file.

- One new variable is named on the COUNT subcommand. It must have a unique name.
- The label for the new variable is optional and, if specified, must be delimited by single or double quotes.
- The format of the new count variable is F4.0.

Example

If the original data are as shown in the first example, the commands:

```
SPLIT FILE BY insure.
CASESTOVARS
    /ID=caseid
    /COUNT=countvar
    /DROP=insure month bpd.
```

create a new file with the following data:

- The command created a count variable, *countvar*, which contains the number of rows in the original data that were used to generate the current row.

FIXED subcommand

The FIXED subcommand names the variables that should be copied from the original data to the new data file.

- CASESTOVARS assumes that variables named on the FIXED subcommand do not vary within row groups in the original data. If they vary, a warning message is generated and the command is executed.
- Fixed variables appear as a single column in the new data file. Their values are simply copied to the new file.
- The AUTOFIX subcommand can automatically determine which variables in the original data are fixed. By default, the AUTOFIX subcommand overrides the FIXED subcommand.

AUTOFIX subcommand

The AUTOFIX subcommand evaluates candidate variables and classifies them as either fixed or as the source of a variable group.

- A **candidate variable** is a variable in the original data that does not appear on the SPLIT command or on the ID, INDEX, and DROP subcommands.
- An original variable that does not vary within any row group is classified as a **fixed variable** and is copied into a single variable in the new data file.
- An original variable that has only a single valid value plus the system-missing value within a row group is classified as a fixed variable and is copied into a single variable in the new data file.
- An original variable that does vary within the row group is classified as the **source of a variable group**. It becomes a variable group in the new data file.
- Use AUTOFIX=NO to overrule the default behavior and expand all variables not marked as ID or fixed or record into a variable group.

YES. *Evaluate and automatically classify all candidate variables.* The procedure automatically evaluates and classifies all candidate variables. This is the default. If there is a FIXED subcommand, the procedure displays a warning message for each misclassified variable and automatically corrects the error. Otherwise, no warning messages are displayed. This option overrides the FIXED subcommand.

NO. *Evaluate all candidate variables and issue warnings.* The procedure evaluates all candidate variables and determines if they are fixed. If a variable is listed on the FIXED subcommand but it is not actually fixed (that is, it varies within the row group), a warning message is displayed and the command is not executed. If a variable is not listed on the FIXED subcommand but it is actually fixed (that is, it does not vary within the row group), a warning message is displayed and the command is executed. The variable is classified as the source of a variable group and becomes a variable group in the new data file.

RENAME subcommand

CASESTOVARS creates variable groups with new variables. The first part of the new variable name is either derived from the name of the original variable or is the rootname specified on the RENAME subcommand.

- The specification is the original variable name followed by a rootname.
- The named variable cannot be a SPLIT FILE variable and cannot appear on the ID, FIXED, INDEX, or DROP subcommands.
- A variable can be renamed only once.
- Only one RENAME subcommand can be used, but it can contain multiple specifications.
- If there is a single index variable and it is a string, RENAME is ignored. The string values are used as the new variable names. For string values that do not form valid variable names, names of the general form *Vn* are used, where *n* is a sequential integer.

SEPARATOR subcommand

CASESTOVARS creates variable groups that contain new variables. There are two parts to the name of a new variable—a rootname and an index. The parts are separated by a string. The separator string is specified on the SEPARATOR subcommand.

- If a separator is not specified, the default is a period.
- A separator can contain multiple characters.
- The separator must be delimited by single or double quotes.
- You can suppress the separator by specifying /SEPARATOR="".
- If there is a single index variable and it is a string, SEPARATOR is ignored. The string values are used as the new variable names. For string values that do not form valid variable names, names of the general form *Vn* are used, where *n* is a sequential integer.

GROUPBY subcommand

The GROUPBY subcommand controls the order of the new variables in the new data file.

VARIABLE. *Group new variables by original variable.* The procedure groups all variables created from an original variable together. This is the default.

INDEX. *Group new variables by index variable.* The procedure groups variables according to the index variables.

Example

If the original variables are:

```
insure     caseid     month     bps     bpd
```

and the data are as shown in the first example, the commands:

```
SPLIT FILE BY insure.
CASESTOVARS
    /ID=caseid
```

```
    /INDEX=month
    /GROUPBY=VARIABLE.
```

create a new data file with the following variable order:

- Variables are grouped by variable group—*bps* and *bpd*.

Example

Using the same original data, the commands:

```
SPLIT FILE BY insure.
CASESTOVARS
    /ID=insure caseid
    /INDEX=month
    /GROUPBY=INDEX.
```

create a new data file with the following variable order:

- Variables are grouped by values of the index variable *month*—*1*, *2*, and *3*.

DROP subcommand

The DROP subcommand specifies the subset of variables to exclude from the new data file.

- You can drop variables that appear on the ID list.
- Variables listed on the DROP subcommand may not appear on the FIXED or INDEX subcommand.
- Dropped variables are not candidates to become a variable group in the new data file.
- You cannot drop all variables. The new data file is required to have at least one variable.

CATPCA

CATPCA is available in SPSS Statistics Professional Edition or the Categories option.

```
CATPCA VARIABLES = varlist

 /ANALYSIS = varlist
      [[(WEIGHT={1**}] [LEVEL={SPORD**}] [DEGREE={2}] [INKNOT={2}]]
                {n  }                    {n}          {n}
                                 {SPNOM  }  [DEGREE={2}] [INKNOT={2}]
                                                  {n}          {n}
                                 {ORDI   }
                                 {NOMI   }
                                 {MNOM   }
                                 {NUME   }

 [/DISCRETIZATION = [varlist[([{GROUPING   }] [{NCAT={7}}] [DISTR={NORMAL }])]]]
                                                  {n}            {UNIFORM}
                                  {EQINTV={n} }
                                  {RANKING    }
                                  {MULTIPLYING}

 [/MISSING = [varlist [([{PASSIVE**}] [{MODEIMPU}])]]]
                                      {RANDIMPU }
                                      {EXTRACAT }
                           {ACTIVE   }   {MODEIMPU }
                                      {RANDIMPU }
                                      {EXTRACAT }
                                      {LISTWISE }

 [/SUPPLEMENTARY = [OBJECT(varlist)] [VARIABLE(varlist)]]

 [/CONFIGURATION = [{INITIAL}] (file)]
                   {FIXED   }

 [/DIMENSION = {2**}]
               {n  }

 [/NORMALIZATION = {VPRINCIPAL**}]
                   {OPRINCIPAL  }
                   {SYMMETRICAL }
                   {INDEPENDENT }
                   {n           }

 [/MAXITER = {100**}]
             {n    }

 [/CRITITER = {.00001**}]
              {value   }

 [/ROTATION = [{NOROTATE**}] [{KAISER**}]]
               {VARIMAX   }   {NOKAISER}
               {EQUAMAX   }
               {QUARTIMAX }
               {PROMAX    } [({4})]
                            {k }
               {OBLIMIN   } [({0})]
                            {k }

 [/RESAMPLE = [{NONE**   }]]
               {BOOTSTRAP} [({1000}] [{95}] [{BALANCED }][{PROCRU}])]
                             {n    }   {m  }   {UNBALANCED}  {REFLEC }

 [/PRINT = [DESCRIP**[(varlist)]]]
           [LOADING** [{NOSORT}]]
                      {SORT   }
           [CORR**]
           [VAF]
           [OCORR]
           [QUANT[(varlist)]]
           [HISTORY]
           [OBJECT[([(varname)]varlist)]]
           [NONE]

 [/PLOT = [OBJECT**[(varlist)][(n)]]
          [LOADING**[(varlist [(CENTR[(varlist)])])][(n)]]
          [CATEGORY (varlist)[(n)]]
          [JOINTCAT[({varlist})][(n)]]
          [TRANS[(varlist[({1})])[(n)]]
                        {n }
          [BIPLOT[({LOADING}[(varlist)])[(varlist)]] [(n)]]
                  {CENTR  }
          [TRIPLOT[(varlist[(varlist)])][(n)]]
          [RESID(varlist[({1})])[(n)]]
                         {n }
          [PROJCENTR(varname, varlist)[(n)]] [NONE]]
          [NDIM(value,value)]
          [VAF]
          [OBELLAREA   [({>}{STDEV} {2})]]
                         {GT}{AREA  } {2}
                         {< }
                         {LT}
          [LDELLAREA   [({>} {AREA} {0})]]
                         {GT} {STDEV} {2}
                         {< }
                         {LT}
          [CTELLAREA   [({>} {AREA} {2})]]
                         {GT} {STDEV} {2}
                         {< }
                         {LT}
```

```
          [NELLPNT({40})
                   {n  }

  [/SAVE = [TRDATA[({TRA    }[(n)])]] [OBJECT[({OBSCO  }[(n)])]]
                   {rootname}                 {rootname}
           [APPROX[({APP    })]] [ELLAREAOBJ]
                   {rootname}
           [LDELLAREA]
           [OBELLAREA]
           [CTELLAREA]

  [/OUTFILE = [TRDATA('savfile'|'dataset')]]
              [DISCRDATA('savfile'|'dataset')]
              [OBJECT('savfile'|'dataset')]
              [APPROX('savfile'|'dataset')]
              [ELLCOORD ('savfile'|'dataset)]
```

** Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 13.0

- NDIM keyword introduced on PLOT subcommand.
- The maximum label length on the PLOT subcommand is increased to 64 for variable names, 255 for variable labels, and 60 for value labels (previous value was 20).

Release 23.0

- RANDIMPU keyword introduced on MISSING subcommand.
- ROTATION subcommand introduced.
- RESAMPLE subcommand introduced.
- SORT and NOSORT keywords introduced for LOADING on the PRINT subcommand.
- VAF, OBELLAREA, LDELLAREA, CTELLAREA, NELLPNT, and keywords introduced on PLOT subcommand.
- OBELLAREA, LDELLAREA, and CTELLAREA keywords introduced on SAVE subcommand.
- ELLCOORD keyword introduced on OUTFILE subcommand.

Overview

CATPCA performs principal components analysis on a set of variables. The variables can be given mixed optimal scaling levels, and the relationships among observed variables are not assumed to be linear.

In CATPCA, dimensions correspond to components (that is, an analysis with two dimensions results in two components), and object scores correspond to component scores.

Options

Optimal Scaling Level. You can specify the optimal scaling level at which you want to analyze each variable (levels include spline ordinal, spline nominal, ordinal, nominal, multiple nominal, or numerical).

Discretization. You can use the DISCRETIZATION subcommand to discretize fractional-value variables or to recode categorical variables.

Missing Data. You can use the MISSING subcommand to specify the treatment of missing data on a per-variable basis.

Rotation. You can use the ROTATION subcommand to choose a rotation method: Varimax, Equamax, Quartimax, Promax, or Oblimin.

Bootstrapping. You can use the RESAMPLE subcommand to produce bootstrap estimates and confidence intervals.

Supplementary Objects and Variables. You can specify objects and variables that you want to treat as supplementary to the analysis and then fit them into the solution.

Read Configuration. CATPCA can read a configuration from a file through the CONFIGURATION subcommand. This information can be used as the starting point for your analysis or as a fixed solution in which to fit variables.

Number of Dimensions. You can specify how many dimensions (components) CATPCA should compute.

Normalization. You can specify one of five different options for normalizing the objects and variables.

Algorithm Tuning. You can use the MAXITER and CRITITER subcommands to control the values of algorithm-tuning parameters.

Optional Output. You can request optional output through the PRINT subcommand.

Optional Plots. You can request a plot of object points, transformation plots per variable, and plots of category points per variable or a joint plot of category points for specified variables. Other plot options include residuals plots, a biplot, a triplot, component loadings plot, and a plot of projected centroids.

Writing Discretized Data, Transformed Data, Object (Component) Scores, and Approximations. You can write the discretized data, transformed data, object scores, and approximations to external files for use in further analyses.

Saving Transformed Data, Object (Component) Scores, and Approximations. You can save the transformed variables, object scores, and approximations to the working data file.

Basic specification

The basic specification is the CATPCA command with the VARIABLES and ANALYSIS subcommands.

Syntax rules

- The VARIABLES and ANALYSIS subcommands must always appear.
- All subcommands can be specified in any order.
- Variables that are specified in the ANALYSIS subcommand must be found in the VARIABLES subcommand.
- Variables that are specified in the SUPPLEMENTARY subcommand must be found in the ANALYSIS subcommand.
- You cannot specify both ROTATION and RESAMPLE on the same command.

Operations

- If a subcommand is repeated, it causes a syntax error, and the procedure terminates.

Limitations

- CATPCA operates on category indicator variables. The category indicators should be positive integers. You can use the DISCRETIZATION subcommand to convert fractional-value variables and string variables into positive integers.
- In addition to system-missing values and user-defined missing values, category indicator values that are less than 1 are treated by CATPCA as missing. If one of the values of a categorical variable has been coded 0 or a negative value and you want to treat it as a valid category, use the COMPUTE command to add a constant to the values of that variable such that the lowest value will be 1. You can also use the RANKING option of the DISCRETIZATION subcommand for this purpose, except for variables that you want to treat as numeric, because the characteristic of equal intervals in the data will not be maintained.
- There must be at least three valid cases.
- Split-file has no implications for CATPCA.

Example

```
CATPCA VARIABLES = TEST1 TEST2 TEST3 TO TEST6 TEST7 TEST8
  /ANALYSIS = TEST1 TO TEST2(WEIGHT=2 LEVEL=ORDI)
              TEST3 TO TEST5(LEVEL=SPORD INKNOT=3)
              TEST6 TEST7(LEVEL=SPORD DEGREE=3)
              TEST8(LEVEL=NUME)
  /DISCRETIZATION = TEST1(GROUPING NCAT=5 DISTR=UNIFORM)
              TEST6(GROUPING) TEST8(MULTIPLYING)
  /MISSING = TEST5(ACTIVE) TEST6(ACTIVE EXTRACAT) TEST8(LISTWISE)
  /SUPPLEMENTARY = OBJECT(1 3) VARIABLE(TEST1)
  /CONFIGURATION = ('iniconf.sav')
  /DIMENSION = 2
  /NORMALIZATION = VPRINCIPAL
  /MAXITER = 150
  /CRITITER = .000001
  /PRINT =  DESCRIP LOADING CORR QUANT(TEST1 TO TEST3) OBJECT
  /PLOT = TRANS(TEST2 TO TEST5) OBJECT(TEST2 TEST3)
  /SAVE = TRDATA OBJECT
  /OUTFILE = TRDATA('/data/trans.sav') OBJECT('/data/obs.sav').
```

- VARIABLES defines variables. The keyword TO refers to the order of the variables in the working data file.
- The ANALYSIS subcommand defines variables that are used in the analysis. *TEST1* and *TEST2* have a weight of 2. For the other variables, WEIGHT is not specified; thus, they have the default weight value of 1. The optimal scaling level for *TEST1* and *TEST2* is ordinal. The optimal scaling level for *TEST3* to *TEST7* is spline ordinal. The optimal scaling level for *TEST8* is numerical. The keyword TO refers to the order of the variables in the VARIABLES subcommand. The splines for *TEST3* to *TEST5* have degree 2 (default because unspecified) and 3 interior knots. The splines for *TEST6* and *TEST7* have degree 3 and 2 interior knots (default because unspecified).
- DISCRETIZATION specifies that *TEST6* and *TEST8*, which are fractional-value variables, are discretized: *TEST6* by recoding into 7 categories with a normal distribution (default because unspecified) and *TEST8* by "multiplying." *TEST1*, which is a categorical variable, is recoded into 5 categories with a close-to-uniform distribution.
- MISSING specifies that objects with missing values on *TEST5* and *TEST6* are included in the analysis; missing values on *TEST5* are replaced with the mode (default if not specified), and missing values on *TEST6* are treated as an extra category. Objects with a missing value on *TEST8* are excluded from the analysis. For all other variables, the default is in effect; that is, missing values (not objects) are excluded from the analysis.
- CONFIGURATION specifies *iniconf.sav* as the file containing the coordinates of a configuration that is to be used as the initial configuration (default because unspecified).
- DIMENSION specifies 2 as the number of dimensions; that is, 2 components are computed. This setting is the default, so this subcommand could be omitted here.
- The NORMALIZATION subcommand specifies optimization of the association between variables. This setting is the default, so this subcommand could be omitted here.
- MAXITER specifies 150 as the maximum number of iterations (instead of the default value of 100).
- CRITITER sets the convergence criterion to a value that is smaller than the default value.
- PRINT specifies descriptives, component loadings and correlations (all default), quantifications for *TEST1* to *TEST3*, and the object (component) scores.
- PLOT requests transformation plots for the variables *TEST2* to *TEST5*, an object points plot labeled with the categories of *TEST2*, and an object points plot labeled with the categories of *TEST3*.
- The SAVE subcommand adds the transformed variables and the component scores to the working data file.
- The OUTFILE subcommand writes the transformed data to a data file called *trans.sav* and writes the component scores to a data file called *obs.sav*, both in the directory */data*.

VARIABLES Subcommand

VARIABLES specifies the variables that may be analyzed in the current CATPCA procedure.

- The VARIABLES subcommand is required.
- At least two variables must be specified, except when the CONFIGURATION subcommand is used with the FIXED keyword.
- The keyword TO on the VARIABLES subcommand refers to the order of variables in the working data file. This behavior of TO is different from the behavior in the variable list in the ANALYSIS subcommand.

ANALYSIS Subcommand

ANALYSIS specifies the variables to be used in the computations, the optimal scaling level, and the variable weight for each variable or variable list. ANALYSIS also specifies supplementary variables and their optimal scaling level. No weight can be specified for supplementary variables.

- At least two variables must be specified, except when the CONFIGURATION subcommand is used with the FIXED keyword.
- All variables on ANALYSIS must be specified on the VARIABLES subcommand.
- The ANALYSIS subcommand is required.
- The keyword TO in the variable list honors the order of variables in the VARIABLES subcommand.
- Optimal scaling levels and variable weights are indicated by the keywords LEVEL and WEIGHT in parentheses following the variable or variable list.

WEIGHT. *Specifies the variable weight with a positive integer.* The default value is 1. If WEIGHT is specified for supplementary variables, it is ignored, and a syntax warning is issued.

LEVEL. *Specifies the optimal scaling level.*

Level Keyword

The following keywords are used to indicate the optimal scaling level:

SPORD. *Spline ordinal (monotonic).* This setting is the default. The order of the categories of the observed variable is preserved in the optimally scaled variable. Category points will lie on a straight line (vector) through the origin. The resulting transformation is a smooth monotonic piecewise polynomial of the chosen degree. The pieces are specified by the user-specified number and procedure-determined placement of the interior knots.

SPNOM. *Spline nominal (nonmonotonic).* The only information in the observed variable that is preserved in the optimally scaled variable is the grouping of objects in categories. The order of the categories of the observed variable is not preserved. Category points will lie on a straight line (vector) through the origin. The resulting transformation is a smooth, possibly nonmonotonic, piecewise polynomial of the chosen degree. The pieces are specified by the user-specified number and procedure-determined placement of the interior knots.

MNOM. *Multiple nominal.* The only information in the observed variable that is preserved in the optimally scaled variable is the grouping of objects in categories. The order of the categories of the observed variable is not preserved. Category points will be in the centroid of the objects in the particular categories. Multiple indicates that different sets of quantifications are obtained for each dimension.

ORDI. *Ordinal.* The order of the categories on the observed variable is preserved in the optimally scaled variable. Category points will lie on a straight line (vector) through the origin. The resulting transformation fits better than SPORD transformation but is less smooth.

NOMI. *Nominal.* The only information in the observed variable that is preserved in the optimally scaled variable is the grouping of objects in categories. The order of the categories of the observed variable is not preserved. Category points will lie on a straight line (vector) through the origin. The resulting transformation fits better than SPNOM transformation but is less smooth.

NUME. *Numerical.* Categories are treated as equally spaced (interval level). The order of the categories and the equal distances between category numbers of the observed variables are preserved in the optimally scaled variable. Category points will lie on a straight line (vector) through the origin. When all

variables are scaled at the numerical level, the CATPCA analysis is analogous to standard principal components analysis.

SPORD and SPNOM Keywords

The following keywords are used with SPORD and SPNOM:

DEGREE. *The degree of the polynomial.* It can be any positive integer. The default degree is 2.

INKNOT. *The number of interior knots.* The minimum is 0, and the maximum is the number of categories of the variable minus 2. If the specified value is too large, the procedure adjusts the number of interior knots to the maximum. The default number of interior knots is 2.

DISCRETIZATION Subcommand

DISCRETIZATION specifies fractional-value variables that you want to discretize. Also, you can use DISCRETIZATION for ranking or for two ways of recoding categorical variables.

- A string variable's values are always converted into positive integers, according to the internal numeric representations. DISCRETIZATION for string variables applies to these integers.
- When the DISCRETIZATION subcommand is omitted or used without a variable list, fractional-value variables are converted into positive integers by grouping them into seven categories with a distribution of close to "normal."
- When no specification is given for variables in a variable list following DISCRETIZATION, these variables are grouped into seven categories with a distribution of close to "normal."
- In CATPCA, values that are less than 1 are considered to be missing (see MISSING subcommand). However, when discretizing a variable, values that are less than 1 are considered to be valid and are thus included in the discretization process.

GROUPING. *Recode into the specified number of categories or recode intervals of equal size into categories.*

RANKING. *Rank cases.* Rank 1 is assigned to the case with the smallest value on the variable.

MULTIPLYING. *Multiply the standardized values of a fractional-value variable by 10, round, and add a value such that the lowest value is 1.*

GROUPING Keyword

GROUPING has the following keywords:

NCAT. *Number of categories.* When NCAT is not specified, the number of categories is set to 7.

EQINTV. *Recode intervals of equal size.* The size of the intervals must be specified (no default). The resulting number of categories depends on the interval size.

NCAT Keyword

NCAT has the keyword DISTR, which has the following keywords:

NORMAL. *Normal distribution.* This setting is the default when DISTR is not specified.

UNIFORM. *Uniform distribution.*

MISSING Subcommand

In CATPCA, we consider a system-missing value, user-defined missing values, and values that are less than 1 as missing values. The MISSING subcommand allows you to indicate how to handle missing values for each variable.

PASSIVE. *Exclude missing values on a variable from analysis.* This setting is the default when MISSING is not specified. Passive treatment of missing values means that in optimizing the quantification of a variable, only objects with nonmissing values on the variable are involved and that only the nonmissing values of variables contribute to the solution. Thus, when PASSIVE is specified, missing values do not affect the analysis. Further, if all variables are given passive treatment of missing values, objects with missing values on every variable are treated as supplementary.

ACTIVE. *Impute missing values.* You can choose to use mode imputation. You can also consider objects with missing values on a variable as belonging to the same category and impute missing values with an extra category indicator.

LISTWISE. *Exclude cases with missing values on a variable.* The cases that are used in the analysis are cases without missing values on the specified variables. Also, any variable that is not included in the subcommand receives this specification.

- The ALL keyword may be used to indicate all variables. If ALL is used, it must be the only variable specification.
- A mode or extracat imputation is done before listwise deletion.

PASSIVE Keyword

If correlations are requested on the PRINT subcommand, and passive treatment of missing values is specified for a variable, the missing values must be imputed. For the correlations of the quantified variables, you can specify the imputation with one of the following keywords:

MODEIMPU. *Impute missing values on a variable with the mode of the quantified variable.* MODEIMPU is the default.

EXTRACAT. *Impute missing values on a variable with the quantification of an extra category.* This treatment implies that objects with a missing value are considered to belong to the same (extra) category.

RANDIMPU. *Impute each missing value on a variable with the quantified value of a different random category number based on the marginal frequencies of the categories of the variable.*

Note that with passive treatment of missing values, imputation applies only to correlations and is done afterward. Thus, the imputation has no effect on the quantification or the solution.

ACTIVE Keyword

The ACTIVE keyword has the following keywords:

MODEIMPU. *Impute missing values on a variable with the most frequent category (mode).* When there are multiple modes, the smallest category indicator is used. MODEIMPU is the default.

EXTRACAT. *Impute missing values on a variable with an extra category indicator.* This implies that objects with a missing value are considered to belong to the same (extra) category.

RANDIMPU. *Impute each missing value on a variable with a different random category number based on the marginal frequencies of the categories*.

Note that with active treatment of missing values, imputation is done before the analysis starts and thus will affect the quantification and the solution.

SUPPLEMENTARY Subcommand

The SUPPLEMENTARY subcommand specifies the objects and/or variables that you want to treat as supplementary. Supplementary variables must be found in the ANALYSIS subcommand. You cannot weight supplementary objects and variables (specified weights are ignored). For supplementary variables, all options on the MISSING subcommand can be specified except LISTWISE.

OBJECT. *Objects that you want to treat as supplementary are indicated with an object number list in parentheses following OBJECT.* The keyword TO is allowed. The OBJECT specification is not allowed when CONFIGURATION = FIXED.

VARIABLE. *Variables that you want to treat as supplementary are indicated with a variable list in parentheses following VARIABLE.* The keyword TO is allowed and honors the order of variables in the VARIABLES subcommand. The VARIABLE specification is ignored when CONFIGURATION = FIXED, because in that case all variables in the ANALYSIS subcommand are automatically treated as supplementary variables.

CONFIGURATION Subcommand

The CONFIGURATION subcommand allows you to read data from a file containing the coordinates of a configuration. The first variable in this file should contain the coordinates for the first dimension, the second variable should contain the coordinates for the second dimension, and so forth.

INITIAL(file). *Use the configuration in the external file as the starting point of the analysis.*

FIXED(file). *Fit variables in the fixed configuration that is found in the external file.* The variables to fit in should be specified on the ANALYSIS subcommand but will be treated as supplementary. The SUPPLEMENTARY subcommand and variable weights are ignored.

DIMENSION Subcommand

DIMENSION specifies the number of dimensions (components) that you want CATPCA to compute.

- The default number of dimensions is 2.
- DIMENSION is followed by an integer indicating the number of dimensions.
- If there are no variables specified as MNOM (multiple nominal), the maximum number of dimensions that you can specify is the smaller of the number of observations minus 1 and the total number of variables.
- If some or all of the variables are specified as MNOM (multiple nominal), the maximum number of dimensions is the smaller of a) the number of observations minus 1 and b) the total number of valid MNOM variable levels (categories) plus the number of SPORD, SPNOM, ORDI, NOMI, and NUME variables minus the number of MNOM variables (if the MNOM variables do not have missing values to be treated as passive). If there are MNOM variables with missing values to be treated as passive, the maximum number of dimensions is the smaller of a) the number of observations minus 1 and b) the total number of valid MNOM variable levels (categories) plus the number of SPORD, SPNOM, ORDI, NOMI, and NUME variables, minus the larger of c) 1 and d) the number of MNOM variables without missing values to be treated as passive.
- If the specified value is too large, CATPCA adjusts the number of dimensions to the maximum.
- The minimum number of dimensions is 1.

NORMALIZATION Subcommand

The NORMALIZATION subcommand specifies one of five options for normalizing the object scores and the variables. Only one normalization method can be used in a given analysis.

VPRINCIPAL. *This option optimizes the association between variables.* With VPRINCIPAL, the coordinates of the variables in the object space are the component loadings (correlations with object scores) for SPORD, SPNOM, ORDI, NOMI, and NUME variables, and the centroids for MNOM variables. This setting is the default if the NORMALIZATION subcommand is not specified. This setting is useful when you are primarily interested in the correlations between the variables.

OPRINCIPAL. *This option optimizes distances between objects.* This setting is useful when you are primarily interested in differences or similarities between the objects.

SYMMETRICAL. *Use this normalization option if you are primarily interested in the relation between objects and variables.*

INDEPENDENT. *Use this normalization option if you want to examine distances between objects and correlations between variables separately.*

The fifth method allows the user to specify any real value in the closed interval [–1, 1]. A value of 1 is equal to the OPRINCIPAL method, a value of 0 is equal to the SYMMETRICAL method, and a value of –1 is equal to the VPRINCIPAL method. By specifying a value that is greater than –1 and less than 1, the user can spread the eigenvalue over both objects and variables. This method is useful for making a tailor-made biplot or triplot. If the user specifies a value outside of this interval, the procedure issues a syntax error message and terminates.

MAXITER Subcommand

MAXITER specifies the maximum number of iterations that the procedure can go through in its computations. If not all variables are specified as NUME and/or MNOM, the output starts from iteration 0, which is the last iteration of the initial phase, in which all variables except MNOM variables are treated as NUME.

- If MAXITER is not specified, the maximum number of iterations is 100.
- The specification on MAXITER is a positive integer indicating the maximum number of iterations. There is no uniquely predetermined (that is, hard-coded) maximum for the value that can be used.

CRITITER Subcommand

CRITITER specifies a convergence criterion value. CATPCA stops iterating if the difference in fit between the last two iterations is less than the CRITITER value.

- If CRITITER is not specified, the convergence value is 0.00001.
- The specification on CRITITER is any positive value.

ROTATION Subcommand

The ROTATION subcommand specifies the method for rotation to a simple component structure.

- When a rotation method is specified, both the unrotated loadings results and rotated loadings are displayed (if LOADING is specified on the PRINT or PLOT subcommand).
- If VARIMAX, QUARTIMAX, or EQUAMAX is specified, the component transformation matrix is also displayed. If PROMAX or OBLIMIN is specified, the pattern and structure maxtrices are displayed, as well as the components correlation matrix.
- Besides the loadings, rotation also affects component scores and category scores, for which only the rotated results are displayed.
- The same command cannot contain both ROTATION and RESAMPLE subcommands.

The following alternatives are available:

NOROTATE. *No rotation*. This is the default setting.

VARIMAX. *Varimax rotation*. An orthogonal rotation method that minimizes the number of variables that have high loadings on each component. It simplifies the interpretation of the components.

QUARTIMAX. *Quartimax rotation*. A rotation method that minimizes the number of components needed to explain each variable. It simplifies the interpretation of the observed variables.

EQUAMAX. *Equamax rotation*. A rotation method that is a combination of the Varimax method, which simplifies the components, and the Quartimax method, which simplifies the variables. The number of variables that load highly on a component and the number of components needed to explain a variable are minimized .

PROMAX(kappa). *Promax Rotation*. An oblique (non-orthogonal) rotation, which allows components to be correlated. It can be calculated more quickly than a direct Oblimin rotation, so it is useful for large datasets. The amount of correlation (obliqueness) that is allowed is controlled by the kappa parameter. The value must be greater than or equal to 1 and less 10,000. The default value is 4.

OBLIMIN(delta). *Direct Oblimin rotation*. A method for oblique (non-orthogonal) rotation. When delta equals 0, components are most oblique. As delta becomes more negative, the components become less oblique. Positive values permit additional component correlation. The value must be less than or equal to 0.8. The default value is 0.

KAISER. *Kaiser normalization*. In the rotation process the loadings are divided by the square root of their communalities, to prevent relatively large loadings dominating the rotation. This is the default setting.

NOKAISER. *Turn off Kaiser normalization*.

RESAMPLE Subcommand

The RESAMPLE subcommand specifies the resampling method used for estimation of stability.

- If plots of loadings, categories, or component scores are requested, additional plots are given, jointly displaying the points for the data sample and the bootstrap estimates. Transformation plots include confidence regions. A plot for the eigenvalues is also displayed.
- If a two-dimensional solution is specified, confidence ellipse plots for the eigenvalues, the component loadings, the category points, and the object points are displayed.
- The display of ellipses in the loadings, categories, and component scores plots can be controlled by specifying the keywords LDELLAREA, CTELLAREA, OBELLAREA, and NELLPNT on the PLOT subcommand.
- The same command cannot contain both ROTATION and RESAMPLE subcommands.

The following alternatives are available:

NONE. *Do not perform resampling*. This is the default setting.

BOOTSTRAP. *Perform resampling*.

BOOTSTRAP parameters

The BOOTSTRAP keyword can be followed by a list of optional parameters, enclosed in parentheses. The general form is: (number of samples, confidence interval, BALANCED | UNBALANCED, PROCRU | REFLEC)

- The first parameter is the number of bootstrap samples. The value must be a positive integer. The default value is 1000.
- The second parameter is the confidence interval, expressed as a percentage. The value must be a positive number less than 100. The default value is 95.
- If only one of the two numeric parameters is specified, it is used as the number of bootstrap samples.
- BALANCED specifies a balanced bootstrap, and UNBALANCED specifies an unbalanced bootstrap. The default setting is BALANCED.
- PROCRU specifies the Procrustes rotation method, and REFLEC specifies the reflection rotation method. The default setting is PROCRU.

Example

```
/RESAMPLE=BOOTSTRAP(5000,REFLEC)
```

- Since only one numeric parameter is specified, it is used as the number of bootstrap samples.
- In the absence of BALANCED or UNBALANCED, the bootstrap sample is balanced.
- The reflection rotation method is used.

PRINT Subcommand

The Model Summary (Cronbach's alpha and Variance Accounted For) and the HISTORY statistics (the variance accounted for, the loss, and the increase in variance accounted for) for the initial solution (if applicable) and last iteration are always displayed. That is, they cannot be controlled by the PRINT subcommand. The PRINT subcommand controls the display of additional optional output. The output of the procedure is based on the transformed variables. However, the keyword OCORR can be used to request the correlations of the original variables, as well.

The default keywords are DESCRIP, LOADING, and CORR. However, when some keywords are specified, the default is nullified and only what was specified comes into effect. If a keyword is duplicated or if a contradicting keyword is encountered, the last specified keyword silently becomes effective (in case of contradicting use of NONE, only the keywords following NONE are effective). An example is as follows:

```
/PRINT <=> /PRINT = DESCRIP LOADING CORR

/PRINT = VAF VAF <=> /PRINT = VAF

/PRINT = VAF NONE CORR <=> /PRINT = CORR
```

If a keyword that can be followed by a variable list is duplicated, a syntax error occurs, and the procedure will terminate.

The following keywords can be specified:

DESCRIP(varlist). *Descriptive statistics (frequencies, missing values, and mode).* The variables in the varlist must be specified on the VARIABLES subcommand but need not appear on the ANALYSIS subcommand. If DESCRIP is not followed by a varlist, descriptives tables are displayed for all variables in the varlist on the ANALYSIS subcommand.

VAF. *Variance accounted for (centroid coordinates, vector coordinates, and total) per variable and per dimension.*

LOADING. *Component loadings for variables with optimal scaling level that result in vector quantification* (that is, SPORD, SPNOM, ORDI, NOMI, and NUME). The LOADING keyword can be followed by SORT or NOSORT in parentheses. If you specify SORT, the loadings are sorted by size. The default setting is NOSORT.

QUANT(varlist). *Category quantifications and category coordinates for each dimension.* Any variable in the ANALYSIS subcommand may be specified in parentheses after QUANT. (For MNOM variables, the coordinates are the quantifications.) If QUANT is not followed by a variable list, quantification tables are displayed for all variables in the varlist on the ANALYSIS subcommand.

HISTORY. *History of iterations.* For each iteration (including 0, if applicable), the variance accounted for, the loss (variance not accounted for), and the increase in variance accounted for are shown.

CORR. *Correlations of the transformed variables and the eigenvalues of this correlation matrix.* If the analysis includes variables with optimal scaling level MNOM, *ndim* (the number of dimensions in the analysis) correlation matrices are computed; in the *i*th matrix, the quantifications of dimension *i*, *i* = 1, ... *ndim*, of MNOM variables are used to compute the correlations. For variables with missing values specified to be treated as PASSIVE on the MISSING subcommand, the missing values are imputed according to the specification on the PASSIVE keyword (if no specification is made, mode imputation is used).

OCORR. *Correlations of the original variables and the eigenvalues of this correlation matrix.* For variables with missing values specified to be treated as PASSIVE on the MISSING subcommand, the missing values are imputed with the variable mode.

OBJECT((varname)varlist). *Object scores (component scores).* Following the keyword, a varlist can be given in parentheses to display variables (category indicators), along with object scores. If you want to use a variable to label the objects, this variable must occur in parentheses as the first variable in the varlist. If no labeling variable is specified, the objects are labeled with case numbers. The variables to display, along with the object scores and the variable to label the objects, must be specified on the

VARIABLES subcommand but need not appear on the ANALYSIS subcommand. If no variable list is given, only the object scores are displayed.

NONE. *No optional output is displayed.* The only output that is shown is the model summary and the HISTORY statistics for the initial iteration (if applicable) and last iteration.

The keyword TO in a variable list can only be used with variables that are in the ANALYSIS subcommand, and TO applies only to the order of the variables in the ANALYSIS subcommand. For variables that are in the VARIABLES subcommand but not in the ANALYSIS subcommand, the keyword TO cannot be used. For example, if /VARIABLES = v1 TO v5 and /ANALYSIS = v2 v1 v4, then /PLOT OBJECT(v1 TO v4) will give two object plots (one plot labeled with *v1* and one plot labeled with *v4*).

PLOT Subcommand

The PLOT subcommand controls the display of plots. The default keywords are OBJECT and LOADING. That is, the two keywords are in effect when the PLOT subcommand is omitted or when the PLOT subcommand is given without any keyword. If a keyword is duplicated (for example, /PLOT = RESID RESID), only the last keyword is effective. If the keyword NONE is used with other keywords (for example, /PLOT = RESID NONE LOADING), only the keywords following NONE are effective. When keywords contradict, the later keyword overwrites the earlier keywords.

- All the variables to be plotted must be specified on the ANALYSIS subcommand.
- If the variable list following the keywords CATEGORIES, TRANS, RESID, and PROJCENTR is empty, it will cause a syntax error, and the procedure will terminate.
- The variables in the variable list for labeling the object point following OBJECT, BIPLOT, and TRIPLOT must be specified on the VARIABLES subcommand but need not appear on the ANALYSIS subcommand. This flexibility means that variables that are not included in the analysis can still be used to label plots.
- The keyword TO in a variable list can only be used with variables that are in the ANALYSIS subcommand, and TO applies only to the order of the variables in the ANALYSIS subcommand. For variables that are in the VARIABLES subcommand but not in the ANALYSIS subcommand, the keyword TO cannot be used. For example, if /VARIABLES = v1 TO v5 and /ANALYSIS = v2 v1 v4, then /PLOT OBJECT(v1 TO v4) will give two object plots, one plot labeled with *v1* and one plot labeled with *v4*.
- For multidimensional plots, all of the dimensions in the solution are produced in a matrix scatterplot if the number of dimensions in the solution is greater than 2 and the NDIM plot keyword is not specified; if the number of dimensions in the solution is 2, a scatterplot is produced.

The following keywords can be specified:

OBJECT(varlist)(n). *Plots of the object points.* Following the keyword, a list of variables in parentheses can be given to indicate that plots of object points labeled with the categories of the variables should be produced (one plot for each variable). The variables to label the objects must be specified on the VARIABLES subcommand but need not appear on the ANALYSIS subcommand. If the variable list is omitted, a plot that is labeled with case numbers is produced.

CATEGORY(varlist)(n). *Plots of the category points.* Both the centroid coordinates and the vector coordinates are plotted. A list of variables must be given in parentheses following the keyword. For variables with optimal scaling level MNOM, categories are in the centroids of the objects in the particular categories. For all other optimal scaling levels, categories are on a vector through the origin.

LOADING(varlist (CENTR(varlist)))(l). *Plot of the component loadings optionally with centroids.* By default, all variables with an optimal scaling level that results in vector quantification (that is, SPORD, SPNOM, ORDI, NOMI, and NUME) are included in this plot. LOADING can be followed by a varlist to select the loadings to include in the plot. When "LOADING(" or the varlist following "LOADING(" is followed by the keyword CENTR in parentheses, centroids are included in the plot for all variables with optimal scaling level MNOM. CENTR can be followed by a varlist in parentheses to select MNOM variables whose centroids are to be included in the plot. When all variables have the MNOM scaling level, this plot cannot be produced.

TRANS(varlist(n))(n). *Transformation plots per variable (optimal category quantifications against category indicators).* Following the keyword, a list of variables in parentheses must be given. MNOM variables in the varlist can be followed by a number of dimensions in parentheses to indicate that you want to display *p* transformation plots, one plot for each of the first *p* dimensions. If the number of dimensions is not specified, a plot for the first dimension is produced.

RESID(varlist(n))(n). *Plot of residuals per variable (approximation against optimal category quantifications).* Following the keyword, a list of variables in parentheses must be given. MNOM variables in the varlist can be followed by a number of dimensions in parentheses to indicate that you want to display *p* residual plots, one plot for each of the first *p* dimensions. If the number of dimensions is not specified, a plot for the first dimension is produced.

BIPLOT(keyword(varlist)) (varlist)(n). *Plot of objects and variables.* The coordinates for the variables can be chosen to be component loading or centroids, using the LOADING or CENTR keyword in parentheses following BIPLOT. When no keyword is given, component loadings are plotted. When NORMALIZATION = INDEPENDENT, this plot is incorrect and therefore not available. Following LOADING or CENTR, a list of variables in parentheses can be given to indicate the variables to be included in the plot. If the variable list is omitted, a plot including all variables is produced. Following BIPLOT, a list of variables in parentheses can be given to indicate that plots with objects that are labeled with the categories of the variables should be produced (one plot for each variable). The variables to label the objects must be specified on the VARIABLES subcommand but need not appear on the ANALYSIS subcommand. If the variable list is omitted, a plot with objects labeled with case numbers is produced.

TRIPLOT(varlist(varlist))(n). *A plot of object points, component loadings for variables with an optimal scaling level that results in vector quantification (that is,* SPORD, SPNOM, ORDI, NOMI, *and* NUME *), and centroids for variables with optimal scaling level* MNOM. Following the keyword, a list of variables in parentheses can be given to indicate the variables to include in the plot. If the variable list is omitted, all variables are included. The varlist can contain a second varlist in parentheses to indicate that triplots with objects labeled with the categories of the variables in this variable list should be produced (one plot for each variable). The variables to label the objects must be specified on the VARIABLES subcommand but need not appear on the ANALYSIS subcommand. If this second variable list is omitted, a plot with objects labeled with case numbers is produced. When NORMALIZATION = INDEPENDENT, this plot is incorrect and therefore not available.

JOINTCAT(varlist)(n). *Joint plot of the category points for the variables in the varlist.* If no varlist is given, the category points for all variables are displayed.

PROJCENTR(varname, varlist)(n). *Plot of the centroids of a variable projected on each of the variables in the varlist.* You cannot project centroids of a variable on variables with MNOM optimal scaling level; thus, a variable that has MNOM optimal scaling level can be specified as the variable to be projected but not in the list of variables to be projected on. When this plot is requested, a table with the coordinates of the projected centroids is also displayed.

VAF. *Barcharts of variable variance accounted for*. There is one barchart for each dimension and one for the total variance accounted for over all dimensions..

LDELLAREA(threshold). *Confidence ellipses for loading plots*. If RESAMPLE=BOOTSTRAP and DIMENSION=2, confidence ellipses are plotted. You can control the display of loadings with their confidence ellipses in the plot by specifying a threshold ellipse area in parentheses of the general form: (GT | LT STDEV | AREA value). STDEV represents the mean area plus the number of standard deviations specified for the value. You can use the greater than (>) and less than signs (<) instead of GT and LT. The default setting is (> AREA 0). This displays all loadings with confidence ellipses.

OBELLAREA(threshold). *Confidence ellipses for object plots*. If RESAMPLE=BOOTSTRAP and DIMENSION=2, confidence ellipses are plotted. You can control the display of objects with their confidence ellipses in the plot by specifying a threshold ellipse area in parentheses of the general form: (GT | LT STDEV | AREA value). STDEV represents the mean area plus the number of standard deviations specified for the value. You can use the greater than (>) and less than signs (<) instead of GT and LT. The default setting is (> STDEV 2). This displays all objects with confidence ellipses.

CTELLAREA(threshold). *Confidence ellipses for category plots*. If RESAMPLE=BOOTSTRAP and DIMENSION=2, confidence ellipses are plotted. You can control the display of loadings with their

confidence ellipses in the plot by specifying a threshold ellipse area in parentheses of the general form: (GT | LT STDEV | AREA value). STDEV represents the mean area plus the number of standard deviations specified for the value. You can use the greater than (>) and less than signs (<) instead of GT and LT. The default setting is (> AREA 2). This displays all categories with confidence ellipses.

NELLPNT(integer). *Number of ellipse contour points*. If RESAMPLE=BOOTSTRAP and DIMENSION=2, confidence ellipses are plotted as a path between a number of points on the ellipse contours. The number of these points influences how smooth the ellipses look. The default number of ellipse contour points is 40.

NONE. *No plots.*

- For all keywords that allow a variable list, the user can specify an optional parameter *l* in parentheses after the variable list in order to control the global upper boundary of variable name/label and value label lengths in the plot. Note that this boundary is applied uniformly to all variables in the list. The label length parameter *l* can take any non-negative integer that is less than or equal to the applicable maximum length (64 for variable names, 255 for variable labels, and 60 for value labels). If *l* = 0, names/values instead of variable/value labels are displayed to indicate variables/categories. If *l* is not specified, CATPCA assumes that each variable name/label and value label is displayed at its full length. If *l* is an integer that is larger than the applicable maximum, we reset it to the applicable maximum but do not issue a warning. If a positive value of *l* is given but some or all variables/category values do not have labels, then, for those variables/values, the names/values themselves are used as the labels.

In addition to the plot keywords, the following keyword can be specified:

NDIM(value,value). *Dimension pairs to be plotted.* NDIM is followed by a pair of values in parentheses. If NDIM is not specified or is specified without parameter values, a matrix scatterplot including all dimensions is produced.

- The first value (an integer that can range from 1 to the number of dimensions in the solution minus 1) indicates the dimension that is plotted against higher dimensions.
- The second value (an integer that can range from 2 to the number of dimensions in the solution) indicates the highest dimension to be used in plotting the dimension pairs.
- The NDIM specification applies to all requested multidimensional plots.

BIPLOT Keyword

BIPLOT takes the following keywords:

LOADING(varlist). *Object points and component loadings.*

CENTR(varlist). *Object points and centroids.*

SAVE Subcommand

The SAVE subcommand is used to add the transformed variables (category indicators that are replaced with optimal quantifications), the object scores, and the approximation to the working data file. Excluded cases are represented by a dot (the system-missing symbol) on every saved variable.

TRDATA. *Transformed variables.* Missing values that are specified to be treated as passive are represented by a dot.

OBJECT. *Object (component) scores.*

APPROX. *Approximation for variables that do not have optimal scaling level MNOM.* For variables with MNOM scaling level, the approximations in dimension *s* are the object scores in dimension *s*.

LDELLAREA. *Confidence ellipse areas for the loadings*. These values are saved only if RESAMPLE=BOOTSTRAP and DIMENSIONS=2.

CTELLAREA. *Confidence ellipse areas for the categories*. These values are saved only if RESAMPLE=BOOTSTRAP and DIMENSIONS=2.

OBELLAREA. *Confidence ellipse areas for the object (component) scores*. These values are saved only if RESAMPLE=BOOTSTRAP and DIMENSIONS=2.

- Following TRDATA, a rootname and the number of dimensions to be saved for variables that are specified as MNOM can be specified in parentheses.
- For variables that are not specified as MNOM, CATPCA adds two numbers separated by the symbol _. For variables that are specified as MNOM, CATPCA adds three numbers. The first number uniquely identifies the source variable names, and the last number uniquely identifies the CATPCA procedures with the successfully executed SAVE subcommands. For variables that are specified as MNOM, the middle number corresponds to the dimension number (see the next bullet for more details). Only one rootname can be specified, and it can contain up to five characters for variables that are not specified as MNOM and three characters for variables that are specified as MNOM. If more than one rootname is specified, the first rootname is used. If a rootname contains more than five characters (MNOM variables), the first five characters are used at most. If a rootname contains more than three characters (MNOM variables), the first three characters are used at most.
- If a rootname is not specified for TRDATA, rootname *TRA* is used to automatically generate unique variable names. The formulas are *ROOTNAMEk_n* and *ROOTNAMEk_m_n*. In this formula, *k* increments from 1 to identify the source variable names by using the source variables' position numbers in the ANALYSIS subcommand, *m* increments from 1 to identify the dimension number, and *n* increments from 1 to identify the CATPCA procedures with the successfully executed SAVE subcommands for a given data file in a continuous session. For example, with three variables specified on ANALYSIS, LEVEL = MNOM for the second variable, and with two dimensions to save, the first set of default names—if they do not exist in the data file—would be *TRA1_1*, *TRA2_1_1*, *TRA2_2_1*, and *TRA3_1*. The next set of default names—if they do not exist in the data file—would be *TRA1_2*, *TRA2_1_2*, *TRA2_2_2*, and *TRA3_2*. However, if, for example, *TRA1_2* already exists in the data file, the default names should be attempted as *TRA1_3*, *TRA2_1_3*, *TRA2_2_3*, and *TRA3_3*. That is, the last number increments to the next available integer.
- Following OBJECT, a rootname and the number of dimensions can be specified in parentheses, to which CATPCA adds two numbers separated by the symbol _. The first number corresponds to the dimension number. The second number uniquely identifies the CATPCA procedures with the successfully executed SAVE subcommands (see the next bullet for more details). Only one rootname can be specified, and it can contain up to five characters. If more than one rootname is specified, the first rootname is used; if a rootname contains more than five characters, the first five characters are used at most.
- If a rootname is not specified for OBJECT, rootname *OBSCO* is used to automatically generate unique variable names. The formula is *ROOTNAMEm_n*. In this formula, *m* increments from 1 to identify the dimension number, and *n* increments from 1 to identify the CATPCA procedures with the successfully executed SAVE subcommands for a given data file in a continuous session. For example, if two dimensions are specified following OBJECT, the first set of default names—if they do not exist in the data file—would be *OBSCO1_1* and *OBSCO2_1*. The next set of default names—if they do not exist in the data file—would be *OBSCO1_2* and *OBSCO2_2*. However, if, for example, *OBSCO2_2* already exists in the data file, the default names should be attempted as *OBSCO1_3* and *OBSCO2_3*. That is, the second number increments to the next available integer.
- Following APPROX, a rootname can be specified in parentheses, to which CATPCA adds two numbers separated by the symbol _. The first number uniquely identifies the source variable names, and the last number uniquely identifies the CATPCA procedures with the successfully executed SAVE subcommands (see the next bullet for more details). Only one rootname can be specified, and it can contain up to five characters. If more than one rootname is specified, the first rootname is used; if a rootname contains more than five characters, the first five characters are used at most.
- If a rootname is not specified for APPROX, rootname *APP* is used to automatically generate unique variable names. The formula is *ROOTNAMEk_n*. In this formula, *k* increments from 1 to identify the source variable names by using the source variables' position numbers in the ANALYSIS subcommand. Additionally, *n* increments from 1 to identify the CATPCA procedures with the successfully executed SAVE subcommands for a given data file in a continuous session. For example, with three variables specified on ANALYSIS and LEVEL = MNOM for the second variable, the first set of default names—if they do not exist in the data file—would be *APP1_1*, *APP2_1*, and *APP3_1*. The next set of default names—if they do not exist in the data file—would be *APP1_2*, *APP2_2*, and *APP3_2*. However, if, for

example, *APP1_2* already exists in the data file, the default names should be attempted as *APP1_3*, *APP2_3*, and *APP3_3*. That is, the last number increments to the next available integer.

- Variable labels are created automatically. (They are shown in the Notes table and can also be displayed in the Data Editor window.)
- If the number of dimensions is not specified, the SAVE subcommand saves all dimensions.

OUTFILE Subcommand

The OUTFILE subcommand is used to write the discretized data, transformed data (category indicators replaced with optimal quantifications), the object scores, and the approximation to a data file or previously declared data set. Excluded cases are represented by a dot (the system-missing symbol) on every saved variable.

DISCRDATA('savfile'|'dataset'). *Discretized data.*

TRDATA('savfile'|'dataset'). *Transformed variables.* This setting is the default if the OUTFILE subcommand is specified with a filename and without a keyword. Missing values that are specified to be treated as passive are represented by a dot.

OBJECT('savfile'|'dataset'). *Object (component) scores.*

APPROX('savfile'|'dataset'). *Approximation for variables that do not have optimal scaling level MNOM*.

ELLCOORD('savfile'|'dataset'). *Coordinates of ellipse plots*. The coordinates file is saved only if RESAMPLE=BOOTSTRAP and DIMENSIONS=2.

- Filenames should be enclosed in quotes and are stored in the working directory unless a path is included as part of the file specification. Data sets are available during the current session but are not available in subsequent sessions unless you explicitly save them as data files. The names should be different for each of the keywords.

In principle, the active data set should not be replaced by this subcommand, and the asterisk (*) file specification is not supported. This strategy also prevents OUTFILE interference with the SAVE subcommand.

CATREG

CATREG is available in SPSS Statistics Professional Edition or the Categories option.

```
CATREG VARIABLES = varlist

/ANALYSIS =
        depvar [([LEVEL={SPORD**}] [DEGREE={2}] [INKNOT={2}])]
                                          {n  }          {n  }
                        {SPNOM  }  [DEGREE={2}] [INKNOT={2}]
                                           {n  }          {n  }
                        {ORDI   }
                        {NOMI   }
                        {NUME   }
   WITH indvarlist [([LEVEL={SPORD**}] [DEGREE={2}] [INKNOT={2}])]
                                          {n  }          {n  }
                           {SPNOM  }  [DEGREE={2}] [INKNOT={2}]
                                           {n  }          {n  }
                           {ORDI   }
                           {NOMI   }
                           {NUME   }

[/DISCRETIZATION = [varlist [([{GROUPING  }] [{NCAT*={7}}] [DISTR={NORMAL }])]]]
                                              {n }          {UNIFORM}
                             {EQINTV=n   }
                             {RANKING    }
                             {MULTIPLYING}

[/MISSING = [{varlist}({LISTWISE**})]]
             {ALL**  } {MODEIMPU  }
                                {EXTRACAT  }

[/SUPPLEMENTARY = OBJECT(objlist)]

[/INITIAL = [{NUMERICAL**}]]
             {RANDOM     }
             {MULTISTART } ({50 }) ('savfile'|'dataset')
                           {n   }
                           {ALL }
             {FIXSIGNS   } (n) ('filename')

[/MAXITER = [{100**}]]
             {n    }

[/CRITITER = [{.00001**}]]
              {value   }

[/REGULARIZATION = [{NONE**}]]
                    {RIDGE }   [{(    0,   1.0,  0.02)}] ('filename')
                                {(value, value, value)  }
                    {LASSO }   [{(    0,   1.0,  0.02)}] ('filename')
                                {(value, value, value)  }
                    {ENET  }   [{(    0,   1.0,   0.1)(    0,   1.0,   .02)}] ('filename')
                                {(value, value, value)(value, value, value)}

[/RESAMPLE = [{NONE**   }]]
              {CROSSVAL }[({10})]
                         {n }
              {BOOTSTRAP}[({50})]
                         {n }

[/PRINT = [R**] [COEFF**] [DESCRIP**[(varlist)]] [HISTORY] [ANOVA**]
          [CORR] [OCORR] [QUANT[(varlist)]] [REGU] [NONE]]

[/PLOT = [TRANS(varlist)[(h)]] [RESID(varlist)[(h)]] [REGU({valuelist})]]
                                                     {ALL      }

[/SAVE = [TRDATA[({TRA   })]] [PRED[({PRE   })]] [RES[({RES      })]]]
                 {rootname}           {rootname}          {rootname}

[/OUTFILE = [TRDATA('savfile'|'dataset')] [DISCRDATA('savfile'|'dataset')]] .
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 13.0

- The maximum category label length on the PLOT subcommand is increased to 60 (previous value was 20).

Release 17.0

- MULTISTART and FIXSIGNS keywords added to INITIAL subcommand.
- REGULARIZATION subcommand added.

- RESAMPLE subcommand added.
- REGU keyword added to PRINT subcommand.
- REGU keyword added to PLOT subcommand.
- SUPPLEMENTARY categories not occuring in data used to create the model are now interpolated.

Overview

CATREG (categorical regression with optimal scaling using alternating least squares) quantifies categorical variables using optimal scaling, resulting in an optimal linear regression equation for the transformed variables. The variables can be given mixed optimal scaling levels, and no distributional assumptions about the variables are made.

Options

Transformation Type. You can specify the transformation type (spline ordinal, spline nominal, ordinal, nominal, or numerical) at which you want to analyze each variable.

Discretization. You can use the DISCRETIZATION subcommand to discretize fractional-value variables or to recode categorical variables.

Initial Configuration. You can specify the kind of initial configuration through the INITIAL subcommand. Also, multiple systematic starts or fixed signs for the regression coefficients can be specified through this subcommand.

Tuning the Algorithm. You can control the values of algorithm-tuning parameters with the MAXITER and CRITITER subcommands.

Regularized regression. You can specifiy one of three methods for regularized regression: Ridge regression, the Lasso, or the Elastic Net.

Resampling. You can specify cross validation or the .632 bootstrap for estimation of prediction error.

Missing Data. You can specify the treatment of missing data with the MISSING subcommand.

Optional Output. You can request optional output through the PRINT subcommand.

Transformation Plot per Variable. You can request a plot per variable of its quantification against the category numbers.

Residual Plot per Variable. You can request an overlay plot per variable of the residuals and the weighted quantification against the category numbers.

Ridge, Lasso, or Elastic Net plot. You can request a plot of the regularized coefficients paths. For the Elastic Net, the plots for all values of the Ridge penalty can be requested, or plots for specified values of the Ridge penalty.

Writing External Data. You can write the transformed data (category numbers replaced with optimal quantifications) to an outfile for use in further analyses. You can also write the discretized data to an outfile.

Saving Variables. You can save the transformed variables, the predicted values, and/or the residuals in the working data file.

Basic Specification

The basic specification is the command CATREG with the VARIABLES and ANALYSIS subcommands.

Syntax Rules

- The VARIABLES and ANALYSIS subcommands must always appear, and the VARIABLES subcommand must be the first subcommand specified. The other subcommands, if specified, can be in any order.
- Variables specified in the ANALYSIS subcommand must be found in the VARIABLES subcommand.
- In the ANALYSIS subcommand, exactly one variable must be specified as a dependent variable and at least one variable must be specified as an independent variable after the keyword WITH.

- The word WITH is reserved as a keyword in the CATREG procedure. Thus, it may not be a variable name in CATREG. Also, the word TO is a reserved word.

Operations

- If a subcommand is specified more than once, the last one is executed but with a syntax warning. Note this is true also for the VARIABLES and ANALYSIS subcommands.

Limitations

- If more than one dependent variable is specified in the ANALYSIS subcommand, CATREG is not executed.
- CATREG operates on category indicator variables. The category indicators should be positive integers. You can use the DISCRETIZATION subcommand to convert fractional-value variables and string variables into positive integers. If DISCRETIZATION is not specified, fractional-value variables are automatically converted into positive integers by grouping them into seven categories with a close to normal distribution and string variables are automatically converted into positive integers by ranking.
- In addition to system missing values and user defined missing values, CATREG treats category indicator values less than 1 as missing. If one of the values of a categorical variable has been coded 0 or some negative value and you want to treat it as a valid category, use the COMPUTE command to add a constant to the values of that variable such that the lowest value will be 1. You can also use the RANKING option of the DISCRETIZATION subcommand for this purpose, except for variables you want to treat as numerical, since the characteristic of equal intervals in the data will not be maintained.
- There must be at least three valid cases.
- The number of valid cases must be greater than the number of independent variables plus 1.
- The maximum number of independent variables is 200.
- Split-File has no implications for CATREG.

Examples

```
CATREG VARIABLES = TEST1 TEST3 TEST2 TEST4 TEST5 TEST6
   TEST7 TO TEST9 STATUS01 STATUS02
  /ANALYSIS TEST4 (LEVEL=NUME)
   WITH TEST1 TO TEST2 (LEVEL=SPORD DEGREE=1 INKNOT=3)
   TEST5 TEST7 (LEVEL=SPNOM) TEST8 (LEVEL=ORDI)
   STATUS01 STATUS02 (LEVEL=NOMI)
  /DISCRETIZATION = TEST1(GROUPING NCAT=5 DISTR=UNIFORM)
                    TEST5(GROUPING) TEST7(MULTIPLYING)
  /INITIAL = RANDOM
  /MAXITER = 100
  /CRITITER = .000001
 /RESAMPLE BOOTSTRAP (100)
  /MISSING = MODEIMPU
  /PRINT = R COEFF DESCRIP ANOVA QUANT(TEST1 TO TEST2 STATUS01 STATUS02)
  /PLOT = TRANS (TEST2 TO TEST7 TEST4)
  /SAVE
  /OUTFILE = '/data/qdata.sav'.
```

- VARIABLES defines variables. The keyword TO refers to the order of the variables in the working data file.
- The ANALYSIS subcommand defines variables used in the analysis. It is specified that *TEST4* is the dependent variable, with optimal scaling level numerical and that the variables *TEST1*, *TEST2*, *TEST3*, *TEST5*, *TEST7*, *TEST8*, *STATUS01*, and *STATUS02* are the independent variables to be used in the analysis. (The keyword TO refers to the order of the variables in the VARIABLES subcommand.) The optimal scaling level for *TEST1*, *TEST2*, and *TEST3* is spline ordinal; for *TEST5* and *TEST7*, spline nominal; for *TEST8*, ordinal; and for *STATUS01* and *STATUS02*, nominal. The splines for *TEST1* and *TEST2* have degree 1 and three interior knots, and the splines for *TEST5* and *TEST7* have degree 2 and two interior knots (default because unspecified).
- DISCRETIZATION specifies that *TEST5* and *TEST7*, which are fractional-value variables, are discretized: *TEST5* by recoding into seven categories with a normal distribution (default because unspecified) and *TEST7* by "multiplying." *TEST1*, which is a categorical variable, is recoded into five categories with a close-to-uniform distribution.

- Because there are nominal variables, a random initial solution is requested by the INITIAL subcommand.
- MAXITER specifies the maximum number of iterations to be 100. This is the default, so this subcommand could be omitted here.
- CRITITER sets the convergence criterion to a value smaller than the default value.
- To include cases with missing values, the MISSING subcommand specifies that for each variable, missing values are replaced with the most frequent category (the mode).
- RESAMPLE specifies the .632 bootstrap for estimation of the prediction error using 100 bootstrap samples (in stead of the default of 50).
- PRINT specifies the correlations, the coefficients, the descriptive statistics for all variables, the ANOVA table, the category quantifications for variables *TEST1*, *TEST2*, *TEST3*, *STATUS01*, and *STATUS02*, and the transformed data list of all cases.
- PLOT is used to request quantification plots for the variables *TEST2*, *TEST5*, *TEST7*, and *TEST4*.
- The SAVE subcommand adds the transformed variables to the working data file. The names of these new variables are *TRANS1_1*, ..., *TRANS9_1*.
- The OUTFILE subcommand writes the transformed data to a data file called *qdata.sav* in the directory */data*.

Example: Multiple Systematic Starts

```
CATREG
 ...
 /INITIAL MULTISTART(ALL)('c:\data\startsigns.sav')
 /PRINT = R COEFF DESCRIP ANOVA QUANT(TEST1 TO TEST2 STATUS01 STATUS02)
 /PLOT = TRANS (TEST2 TO TEST7 TEST4)
 /SAVE TRDATA PRED RES
 /OUTFILE = TRDATA('c:\data\qdata.sav') DISCRDATA('c:\data\discr.sav')
```

- Because the ordinal and spline ordinal scaling levels are specified for some variables, there is chance of obtaining a suboptimal solution when applying the numerical or random initial solution. To ascertain obtaining the optimal solution, all multiple systematic starts are used. Using all systematic starts is feasible here because the number of variables with (spline) ordinal scaling is only 3; then the number of all starts is 2 to the power of 3 is 8. With a larger number of variables with (spline) ordinal scaling level, a reduced number of starts is recommended, which can be requested by specifying /INITIAL MULTISTART(value).
- The specifications at the PRINT, PLOT, SAVE, and OUTFILE subcommands will be applied to the optimal solution.

Example: Fixing Initial Signs for Regression Coefficients

```
CATREG
 ...
 /INITIAL FIXSIGNS (63) ('c:\data\startsigns.sav')
```

- The INITIAL subcommand specifies using a specific set of fixed signs for the regression coefficients. The signs are in the file *startsigns.sav* in the directory *c:\data*. This file was created by a previous run of CATREG with keyword MULTISTART at the INITIAL subcommand (see previous example). The signs of start number 63 are specified to be used.

Example: Elastic Net Regularization

```
CATREG
 ...
 /REGULARIZATION ENET (.5 2.5 .25) (.01 3.8 .05)('c:\data\regu_enet.sav')
 /PRINT = REGU R COEFF DESCRIP ANOVA QUANT(TEST1 TO TEST2)
 /PLOT = REGU (.75 1.5) TRANS (TEST2 TO TEST7 TEST4)
 /SAVE TRDATA PRED RES
 /OUTFILE = TRDATA('c:\data\qdata.sav') DISCRDATA('c:\data\discr.sav').
```

- REGULARIZATION specifies application of Elastic Net regularization, with start value of the Lasso penalty 0.01, stop value 3.8, and increment 0.05, resulting in 76 regularized models, with Lasso penalty values 0.01,0 .06, ..., 3.76. To each of these 76 Lasso models 10 Ridge penalties are applied (0.5, 0.75, ..., 2.5), resulting in 76 × 10 = 760 Elastic Net models.

- PRINT specifies displaying a table with the penalty values, R-squared, and the regression coefficients for each regularized model. The contents of this table is written to a data file called *regu_enet.sav* in the directory *c:\data*.
- The PLOT subcommand requests two Elastic Net plots: a Lasso plot with a fixed Ridge penalty of 0.75 and a Lasso plot with a fixed Ridge penalty of 1.50. Any other keywords than REGU at the PLOT subcommand are ignored.
- Other specifications then REGU at the PRINT and PLOT subcommands, the SAVE subcommand and the TRDATA keyword at the OUTFILE subcommand are ignored.

Example: Elastic Net Regularization with Crossvalidation Resampling

```
CATREG
 ...
 /REGULARIZATION ENET (.5 2.5 .25)(.01 3.8 .05)('c:\data\regu_enet.sav')
 /RESAMPLE CROSSVAL (5)
 /PRINT = REGU R COEFF DESCRIP ANOVA
 /PLOT = REGU (.75 1.5) TRANS (TEST2 TO TEST7 TEST4)
 /SAVE TRDATA PRED RES
 /OUTFILE = TRDATA('c:\data\qdata.sav') DISCRDATA('c:\data\discr.sav').
```

- REGULARIZATION is the same as in the previous example.
- The RESAMPLE subcommand specifies 5-fold cross-validation to estimate the prediction error for each of the 760 Elastic Net models.
- PRINT specifies displaying a table with the penalty values, R-squared, the regression coefficients, and the estimated prediction error for each regularized model. The contents of this table is written to a data file called *regu_enet.sav* in the directory *c:\data*.
- The specification at the PLOT subcommand result in the same plots as in the previous example.
- The other specifications at the PRINT and PLOT subcommands, and the SAVE and OUTFILE specifications will be applied to the model with lowest prediction error.

Example: Obtaining a Specific Elastic Net Model

```
CATREG
 ...
 /REGULARIZATION ENET (1.25 1.25 0)(.46 .46 0)
 /PRINT = R COEFF DESCRIP ANOVA QUANT(TEST1 TO TEST2 STATUS01 STATUS02)
 /PLOT = TRANS (TEST2 TO TEST7 TEST4)
 /SAVE TRDATA PRED RES
 /OUTFILE = TRDATA('c:\data\qdata.sav') DISCRDATA('c:\data\discr.sav').
```

- REGULARIZATION is specified here (stop value equal to start value, increment zero) to obtain output for a specific Elastic Net model: the model with penalty values 1.25 (Ridge) and .46 (Lasso).

VARIABLES Subcommand

VARIABLES specifies the variables that may be analyzed in the current CATREG procedure.

- The VARIABLES subcommand is required and precedes all other subcommands.
- The keyword TO on the VARIABLES subcommand refers to the order of variables in the working data file. (Note that this behavior of TO is different from that in the indvarlist on the ANALYSIS subcommand.)

ANALYSIS Subcommand

ANALYSIS specifies the dependent variable and the independent variables following the keyword WITH.

- All the variables on ANALYSIS must be specified on the VARIABLES subcommand.
- The ANALYSIS subcommand is required and follows the VARIABLES subcommand.
- The first variable list contains exactly one variable as the dependent variable, while the second variable list following WITH contains at least one variable as an independent variable. Each variable may have at most one keyword in parentheses indicating the transformation type of the variable.

- The keyword TO in the independent variable list honors the order of variables on the VARIABLES subcommand.
- Optimal scaling levels are indicated by the keyword LEVEL in parentheses following the variable or variable list.

LEVEL. *Specifies the optimal scaling level.*

LEVEL Keyword

The following keywords are used to indicate the optimal scaling level:

SPORD. *Spline ordinal (monotonic).* This is the default for a variable listed without any optimal scaling level, for example, one without LEVEL in the parentheses after it or with LEVEL without a specification. Categories are treated as ordered. The order of the categories of the observed variable is preserved in the optimally scaled variable. Categories will be on a straight line through the origin. The resulting transformation is a smooth nondecreasing piecewise polynomial of the chosen degree. The pieces are specified by the number and the placement of the interior knots.

SPNOM. *Spline nominal (non-monotonic).* Categories are treated as unordered. Objects in the same category obtain the same quantification. Categories will be on a straight line through the origin. The resulting transformation is a smooth piecewise polynomial of the chosen degree. The pieces are specified by the number and the placement of the interior knots.

ORDI. *Ordinal.* Categories are treated as ordered. The order of the categories of the observed variable is preserved in the optimally scaled variable. Categories will be on a straight line through the origin. The resulting transformation fits better than SPORD transformation, but is less smooth.

NOMI. *Nominal.* Categories are treated as unordered. Objects in the same category obtain the same quantification. Categories will be on a straight line through the origin. The resulting transformation fits better than SPNOM transformation, but is less smooth.

NUME. *Numerical.* Categories are treated as equally spaced (interval level). The order of the categories and the differences between category numbers of the observed variables are preserved in the optimally scaled variable. Categories will be on a straight line through the origin. When all variables are scaled at the numerical level, the CATREG analysis is analogous to standard multiple regression analysis.

SPORD and SPNOM Keywords

The following keywords are used with SPORD and SPNOM :

DEGREE. *The degree of the polynomial.* If DEGREE is not specified the degree is assumed to be 2.

INKNOT. *The number of the interior knots.* If INKNOT is not specified the number of interior knots is assumed to be 2.

DISCRETIZATION Subcommand

DISCRETIZATION specifies fractional-value variables that you want to discretize. Also, you can use DISCRETIZATION for ranking or for two ways of recoding categorical variables.

- A string variable's values are always converted into positive integers by assigning category indicators according to the ascending alphanumeric order. DISCRETIZATION for string variables applies to these integers.
- When the DISCRETIZATION subcommand is omitted, or when the DISCRETIZATION subcommand is used without a varlist, fractional-value variables are converted into positive integers by grouping them into seven categories (or into the number of distinct values of the variable if this number is less than 7) with a close to normal distribution.
- When no specification is given for variables in a varlist following DISCRETIZATION, these variables are grouped into seven categories with a close-to-normal distribution.

- In CATREG, a system-missing value, user-defined missing values, and values less than 1 are considered to be missing values (see next section). However, in discretizing a variable, values less than 1 are considered to be valid values, and are thus included in the discretization process. System-missing values and user-defined missing values are excluded.

GROUPING. *Recode into the specified number of categories.*

RANKING. *Rank cases.* Rank 1 is assigned to the case with the smallest value on the variable.

MULTIPLYING. *Multiplying the standardized values (z-scores) of a fractional-value variable by 10, rounding, and adding a value such that the lowest value is 1.*

GROUPING Keyword

NCAT. *Recode into ncat categories.* When NCAT is not specified, the number of categories is set to 7 (or the number of distinct values of the variable if this number is less than 7). The valid range is from 2 to 36. You may either specify a number of categories or use the keyword DISTR.

EQINTV. *Recode intervals of equal size into categories.* The interval size must be specified (there is no default value). The resulting number of categories depends on the interval size.

DISTR Keyword

DISTR has the following keywords:

NORMAL. *Normal distribution.* This is the default when DISTR is not specified.

UNIFORM. *Uniform distribution.*

MISSING Subcommand

In CATREG, we consider a system missing value, user defined missing values, and values less than 1 as missing values. However, in discretizing a variable (see previous section), values less than 1 are considered as valid values. The MISSING subcommand allows you to indicate how to handle missing values for each variable.

LISTWISE. *Exclude cases with missing values on the specified variable(s).* The cases used in the analysis are cases without missing values on the variable(s) specified. This is the default applied to all variables, when the MISSING subcommand is omitted or is specified without variable names or keywords. Also, any variable that is not included in the subcommand gets this specification.

MODEIMPU. *Impute missing value with mode.* All cases are included and the imputations are treated as valid observations for a given variable. When there are multiple modes, the smallest mode is used.

EXTRACAT. *Impute missing values on a variable with an extra category indicator.* This implies that objects with a missing value are considered to belong to the same (extra) category. This category is treated as nominal, regardless of the optimal scaling level of the variable.

- The ALL keyword may be used to indicate all variables. If it is used, it must be the only variable specification.
- A mode or extra-category imputation is done before listwise deletion.

SUPPLEMENTARY Subcommand

The SUPPLEMENTARY subcommand specifies the objects that you want to treat as supplementary. You cannot weight supplementary objects (specified weights are ignored). This subcommand can be used to specify test cases.

OBJECT. *Supplementary objects.* Objects that you want to treat as supplementary are indicated with an object number list in parentheses following OBJECT. The keyword TO is allowed—for example, `OBJECT(1 TO 1 3 5 TO 9)`.

- Supplementary object are exluced from the analysis. The quantifications resulting from the analysis for the active objects are applied to the categories of supplementary objects, and predicted and residual values for supplementary objects are provided.
- If a supplementary object has a category that does not occur in the active data, the following strategies are applied:

  If the variable on which the non-occuring category occurs has a numeric or spline scaling level, and the non-occuring category lies within the range of categories in the active data, then interpolation is applied.

  If the variable has numeric scaling level and the non-occuring category lies outside the range of categories in the active data, then extrapolation is applied.

  Otherwise the case is excluded.
- Excluded cases are represented by a dot (the sysmis symbol) on every saved variable.

INITIAL Subcommand

INITIAL specifies the method used to compute the initial value/configuration.

- The specification on INITIAL is keyword NUMERICAL, RANDOM, MULTISTART or FIXSIGNS. If INITIAL is not specified, NUMERICAL is the default.

NUMERICAL. *Treat all variables as numerical.* This is usually best to use when there are only numerical and/or ordinal variables.

RANDOM. *Provide a random initial value.* This should be used only when there is at least one nominal variable.

MULTISTART(integer|ALL)('savfile'|'dataset'). *Multiple Systematic Starts.* Multiple final solutions are computed and the best solution is selected. For each solution the same initial values are used, but with different signs for the regression coefficients of variables with ordinal or spline ordinal scaling level. This option is only applicable when there is at least one variable with ordinal or spline ordinal scaling level. With these scaling levels, the CATREG algorithm can result in a suboptimal solution. The optimal solution is always found when multiple systematic starts (using all possible sign patterns for the regression coefficients) are applied. You can specify ALL in parentheses following the keyword to ascertain obtaining the optimal solution. However, the number of all possible sign patterns for the regression coefficients is 2 to the power of *q*, where *q* is the number of variables with ordinal or spline ordinal scaling level. So, the number of all possible sign patterns rapidly increases with increasing *q*.

When *q* is large, a reduced number of multiple systematic starts can be requested by specifying a value in parentheses following the keyword. This option selects a reduced number of sign patterns by applying a hierarchical strategy combined with a percentage of loss criterion. The value to specify is the threshold for the percentage of loss of variance that a variable suffers due to ordinal restriction. Specify a non-negative value leass than or equal to 100. A variable for which this percentage is below the specified threshold, is not allowed to have a negative sign. So, specifying a threshold value excludes sign patterns in which variables with a loss of variance percentage below the specified threshold have a negative sign. Thus, the higher the threshold, the more sign patterns will be excluded. With this option, obtaining the optimal solution is not garantueed, but the chance of obtaining a suboptimal solution is diminished. Also, if with the reduced number of starts the optimal solution is not found, the chance that the suboptimal solution is much different from the optimal solution is diminished. Note that there is a trade-off between the chance of obtaining a suboptimal solution and the number of starts: a higher threshold results in more reduction of the number of starts, but a higher chance of obtaining a suboptimal solution.

When this keyword is used, a dataset name or filename in parentheses must be specified. The signs of the regression coefficients for each start will be written to this file.

To give an impression of computing time: when q is 15, and all variables have seven categories (in CATREG, CPU time depends upon the number of categories, not upon the number of cases), the number of all starts is 32768, which requires 4 minutes on a 2.2 Ghz computer. When q is 20, the number of all

starts is 1048576, requiring 4.5 hours, and when q is 21, the number of all starts is 2097152, requiring 11.5 hours.

FIXSIGNS(integer startnumber)('savfile'|'dataset'). Use fixed signs for the regression coefficients. The signs (indicated by 1 and –1) need to be in (a row of) the specified dataset or file. The integer-valued startnumber to specify is the case number of the row in this file that contains the signs that are to be used. If in a previous run MULTISTART was specified, a file containing the signs for each start was created and can be used here.

MAXITER Subcommand

MAXITER specifies the maximum number of iterations CATREG can go through in its computations. Note that the output starts from the iteration number 0, which is the initial value before any iteration, when INITIAL = NUMERICAL is in effect.

- If MAXITER is not specified, CATREG will iterate up to 100 times.
- The specification on MAXITER is a positive integer indicating the maximum number of iterations. There is no uniquely predetermined (hard coded) maximum for the value that can be used.

CRITITER Subcommand

CRITITER specifies a convergence criterion value. CATREG stops iterating if the difference in fit between the last two iterations is less than the CRITITER value.

- If CRITITER is not specified, the convergence value is 0.00001.
- The specification on CRITITER is any value less than or equal to 0.1 and greater than or equal to 0.000001. (Values less than the lower bound might seriously affect performance. Therefore, they are not supported.)

REGULARIZATION Subcommand

REGULARIZATION specifies the method for regularized regression.

The specification on REGULARIZATION is keyword NONE, RIDGE, LASSO or ENET. If REGULARIZATION is not specified, NONE is the default. Also, a dataset name or filename must be specified. The statistics and coefficients and, if applicable, the estimated prediction error, for all regularized models will be written to this file.

NONE. *No regularization.*

RIDGE(start value, stop value, increment)('savfile'|'dataset'). *Ridge Regression.* A value list in parentheses following the keyword should be given. The first value specifies the start value of the penalty parameter, the second value the stop value, and the third value specifies the increment.

LASSO(start value, stop value, increment)('savfile'|'dataset'). *LASSO (Least Absolute Shrinkage and Selection Operator).* A value list in parentheses following the keyword should be given. The first value specifies the start value of the penalty parameter, the second value the stop value, and the third value specifies the increment.

ENET(start, stop , incr)(start, stop , incr)('savfile'|'dataset'). *Elastic Net.* Two value lists in parentheses following the keyword should be given. The first list specifies the start, stop, and increment values for the Ridge penalty, the second list specifies the start, stop, and increment values for the Lasso penalty.

- If a REGULARIZATION method is specified without specification of a resample method at the RESAMPLE subcommand or specification of test cases at the SUPPLEMENTARY subcommand, any other keywords than REGU at the PRINT and PLOT subcommands are ignored. Also, the SAVE subcommand, and the TRDATA keyword at the OUTFILE subcommand are ignored.

- If a a resample method is specified at the RESAMPLE subcommand, or if test cases are specified at the SUPPLEMENTARY subcommand, specified PRINT, PLOT, SAVE, and OUTFILE output will be given for the model with lowest prediction error or with lowest Test MSE.
- Output of an analysis with a specific value of the penalty parameter(s) is obtained by setting the start value(s) to specific penalty value(s), the stop value(s) equal to the start value(s) and the increment value(s) to 0.

RESAMPLE Subcommand

RESAMPLE specifies the resampling method used for estimation of the prediction error.

The specification on RESAMPLE is keyword NONE, CROSSVAL or BOOTSTRAP. If RESAMPLE is not specified, NONE is the default.

NONE. *No resampling.*

CROSSVAL(integer). *Cross-validation.* The keyword can be followed by a positive integer in parentheses specifying the number of folds. If this value is not specified, 10-fold cross-validation is used.

BOOTSTRAP(integer). *.632 Bootstap.* The keyword can be followed by a positive integer in parentheses specifying the number of bootstrap samples. If this value is not specified, 50 bootstrap samples are used.

PRINT Subcommand

The PRINT subcommand controls the display of output. The output of the CATREG procedure is always based on the transformed variables. However, the correlations of the original predictor variables can be requested as well by the keyword OCORR. The default keywords are R, COEFF, DESCRIP, and ANOVA. That is, the four keywords are in effect when the PRINT subcommand is omitted or when the PRINT subcommand is given without any keyword. If a keyword is duplicated or it encounters a contradicting keyword, such as /PRINT = R R NONE, then the last one silently becomes effective.

- The REGU keyword is only applicable if a REGULARIZATION method is specified.

R. *Multiple R.* Includes R^2, adjusted R^2, and adjusted R^2 taking the optimal scaling into account.

COEFF. *Standardized regression coefficients (beta).* This option gives three tables: a Coefficients table that includes betas, standard error of the betas, *t* values, and significance; a Coefficients-Optimal Scaling table, with the standard error of the betas taking the optimal scaling degrees of freedom into account; and a table with the zero-order, part, and partial correlation, Pratt's relative importance measure for the transformed predictors, and the tolerance before and after transformation. If the tolerance for a transformed predictor is lower than the default tolerance value in the Regression procedure (0.0001) but higher than 10E–12, this is reported in an annotation. If the tolerance is lower than 10E–12, then the COEFF computation for this variable is not done and this is reported in an annotation. Note that the regression model includes the intercept coefficient but that its estimate does not exist because the coefficients are standardized.

DESCRIP(varlist). *Descriptive statistics (frequencies, missing values, and mode).* The variables in the varlist must be specified on the VARIABLES subcommand but need not appear on the ANALYSIS subcommand. If DESCRIP is not followed by a varlist, Descriptives tables are displayed for all of the variables in the variable list on the ANALYSIS subcommand.

HISTORY. *History of iterations.* For each iteration, including the starting values for the algorithm, the multiple *R* and the regression error (square root of (1–multiple R^2)) are shown. The increase in multiple *R* is listed from the first iteration.

ANOVA. *Analysis-of-variance tables.* This option includes regression and residual sums of squares, mean squares, and *F*. This options gives two ANOVA tables: one with degrees of freedom for the regression equal to the number of predictor variables and one with degrees of freedom for the regression taking the optimal scaling into account.

CORR. *Correlations of the transformed predictors.*

OCORR. *Correlations of the original predictors.*

QUANT(varlist). *Category quantifications.* Any variable in the ANALYSIS subcommand may be specified in parentheses after QUANT. If QUANT is not followed by a varlist, Quantification tables are displayed for all variables in the variable list on the ANALYSIS subcommand.

REGU. *Penalty values, R-squared, and the regression coefficients for each regularized model, and, if a RESAMPLE method is specified or if supplementary objects (tests cases) are specified, the prediction error or Test MSE.*

NONE. *No* PRINT *output is shown.* This is to suppress the default PRINT output.

- The keyword TO in a variable list can be used only with variables that are in the ANALYSIS subcommand, and TO applies only to the order of the variables in the ANALYSIS subcommand. For variables that are in the VARIABLES subcommand but not in the ANALYSIS subcommand, the keyword TO cannot be used. For example, if /VARIABLES = v1 TO v5 and /ANALYSIS is v2 v1 v4, then /PRINT QUANT(v1 TO v4) will give two quantification plots, one for *v1* and one for *v4*. (/PRINT QUANT(v1 TO v4 v2 v3 v5) will give quantification tables for *v1*, *v2*, *v3*, *v4*, and *v5*.)

PLOT Subcommand

The PLOT subcommand controls the display of plots.

- The REGU keyword is only applicable if a REGULARIZATION method is specified.
- In this subcommand, if no plot keyword is given, then no plot is created. Further, if the variable list following the plot keyword is empty, then no plot is created, either.
- All of the variables to be plotted must be specified in the ANALYSIS subcommand. Further, for the residual plots, the variables must be independent variables.

TRANS(varlist)(l). *Transformation plots (optimal category quantifications against category indicators).* A list of variables must come from the ANALYSIS variable list and must be given in parentheses following the keyword. Further, the user can specify an optional parameter l in parentheses after the variable list in order to control the global upper boundary of category label lengths in the plot. Note that this boundary is applied uniformly to all transformation plots.

RESID(varlist)(l). *Residual plots (residuals when the dependent variable is predicted from all predictor variables in the analysis except the predictor variable in varlist, against category indicators, and the optimal category quantifications multiplied with beta against category indicators).* A list of variables must come from the ANALYSIS variable list's independent variables and must be given in parentheses following the keyword. Further, the user can specify an optional parameter l in parentheses after the variable list in order to control the global upper boundary of category label lengths in the plot. Note that this boundary is applied uniformly to all residual plots.

REGU(valuelist). *Ridge, Lasso, or Elastic Net plot(s), depending on the regularization method specified at the REGULARIZATION subcommand.* A value or valuelist of Ridge penalties must be given in parentheses following the keyword if the regularization method is Elastic Net. The Elastic Net method results in multiple plots: a Lasso plot for each value of the Ridge penalty. To obtain all Elastic Net plots, the keyword ALL in stead of a valuelist can be used.

- The category label length parameter (l) can take any non-negative integer less than or equal to 60. If l = 0, values instead of value labels are displayed to indicate the categories on the *x* axis in the plot. If l is not specified, CATREG assumes that each value label at its full length is displayed as a plot's category label. If l is an integer larger than 60, then we reset it to 60 but do not issue a warning.
- If a positive value of l is given but if some or all of the values do not have value labels, then for those values, the values themselves are used as the category labels.
- The keyword TO in a variable list can be used only with variables that are in the ANALYSIS subcommand, and TO applies only to the order of the variables in the ANALYSIS subcommand. For variables that are in the VARIABLES subcommand but not in the ANALYSIS subcommand, the keyword TO cannot be used. For example, if /VARIABLES = v1 TO v5 and /ANALYSIS is v2 v1 v4, then /

PLOT TRANS(v1 TO v4) will give two transformation plots, one for *v1* and for *v4*. (/PLOT TRANS(v1 TO v4 v2 v3 v5) will give transformation plots for *v1*, *v2*, *v3*, *v4*, and *v5*.)

SAVE Subcommand

The SAVE subcommand is used to add the transformed variables (category indicators replaced with optimal quantifications), the predicted values, and the residuals to the working data file.

Excluded cases are represented by a dot (the sysmis symbol) on every saved variable.

TRDATA. *Transformed variables.*

PRED. *Predicted values.*

RES. *Residuals.*

- A variable rootname can be specified with each of the keywords. Only one rootname can be specified with each keyword, and it can contain up to five characters (if more than one rootname is specified with a keyword, the first rootname is used; if a rootname contains more than five characters, the first five characters are used at most). If a rootname is not specified, the default rootnames (TRA, PRE, and RES) are used.
- CATREG adds two numbers separated by an underscore (_) to the rootname. The formula is *ROOTNAMEk_n*, where *k* increments from 1 to identify the source variable names by using the source variables' position numbers in the ANALYSIS subcommand (that is, the dependent variable has the position number 1, and the independent variables have the position numbers 2, 3, ..., etc., as they are listed), and *n* increments from 1 to identify the CATREG procedures with the successfully executed SAVE subcommands for a given data file in a continuous session. For example, with two predictor variables specified on ANALYSIS, the first set of default names for the transformed data, if they do not exist in the data file, would be *TRA1_1* for the dependent variable, and *TRA2_1*, *TRA3_1* for the predictor variables. The next set of default names, if they do not exist in the data file, would be *TRA1_2*, *TRA2_2*, *TRA3_2*. However, if, for example, *TRA1_2* already exists in the data file, then the default names should be attempted as *TRA1_3*, *TRA2_3*, *TRA3_3*—that is, the last number increments to the next available integer.
- Variable labels are created automatically. (They are shown in the Procedure Information Table (the Notes table) and can also be displayed in the Data Editor window.)

OUTFILE Subcommand

The OUTFILE subcommand is used to write the discretized data and/or the transformed data (category indicators replaced with optimal quantifications) to a data file or previously declared data set name. Excluded cases are represented by a dot (the sysmis symbol) on every saved variable.

DISCRDATA('savfile'|'dataset') . *Discretized data.*

TRDATA('savfile'|'dataset'). *Transformed variables.*

- Filenames should be enclosed in quotes and are stored in the working directory unless a path is included as part of the file specification. Data sets are available during the current session but are not available in subsequent sessions unless you explicitly save them as data files.
- An active data set, in principle, should not be replaced by this subcommand, and the asterisk (*) file specification is not supported. This strategy also prevents the OUTFILE interference with the SAVE subcommand.

CCF

```
CCF VARIABLES= series names [WITH series names]

 [/DIFF={1}]
        {n}

 [/SDIFF={1}]
         {n}

 [/PERIOD=n]

 [/{NOLOG**}]
   {LN     }

 [/SEASONAL]

 [/MXCROSS={7**}]
           {n  }

 [/APPLY['model name']]
```

**Default if the subcommand is omitted and there is no corresponding specification on the TSET command.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
CCF VARIABLES = VARX VARY.
```

Overview

CCF displays and plots the cross-correlation functions of two or more time series. You can also display and plot the cross-correlations of transformed series by requesting natural log and differencing transformations within the procedure.

Options

Modifying the Series. You can request a natural log transformation of the series using the LN subcommand and seasonal and nonseasonal differencing to any degree using the SDIFF and DIFF subcommands. With seasonal differencing, you can also specify the periodicity on the PERIOD subcommand.

Statistical Display. You can control which series are paired by using the keyword WITH. You can specify the range of lags for which you want values displayed and plotted with the MXCROSS subcommand, overriding the maximum specified on TSET. You can also display and plot values at periodic lags only using the SEASONAL subcommand.

Basic Specification

The basic specification is two or more series names. By default, CCF automatically displays the cross-correlation coefficient and standard error for the negative lags (second series leading), the positive lags (first series leading), and the 0 lag for all possible pair combinations in the series list. It also plots the cross-correlations and marks the bounds of two standard errors on the plot. By default, CCF displays and plots values up to 7 lags (lags −7 to +7), or the range specified on TSET.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- The VARIABLES subcommand can be specified only once.
- Other subcommands can be specified more than once, but only the last specification of each one is executed.

Operations

- Subcommand specifications apply to all series named on the CCF command.
- If the LN subcommand is specified, any differencing requested on that CCF command is done on the log-transformed series.
- Confidence limits are displayed in the plot, marking the bounds of two standard errors at each lag.

Limitations

- A maximum of 1 VARIABLES subcommand. There is no limit on the number of series named on the list.

Example

```
CCF VARIABLES = VARX VARY
  /LN
  /DIFF=1
  /SDIFF=1
  /PERIOD=12
  /MXCROSS=25.
```

- This example produces a plot of the cross-correlation function for *VARX* and *VARY* after a natural log transformation, differencing, and seasonal differencing have been applied to both series. Along with the plot, the cross-correlation coefficients and standard errors are displayed for each lag.
- LN transforms the data using the natural logarithm (base *e*) of each series.
- DIFF differences each series once.
- SDIFF and PERIOD apply one degree of seasonal differencing with a periodicity of 12.
- MXCROSS specifies 25 for the maximum range of positive and negative lags for which output is to be produced (lags –25 to +25).

VARIABLES Subcommand

VARIABLES specifies the series to be plotted and is the only required subcommand.

- The minimum VARIABLES specification is a pair of series names.
- If you do not use the keyword WITH, each series is paired with every other series in the list.
- If you specify the keyword WITH, every series named before WITH is paired with every series named after WITH.

Example

```
CCF VARIABLES=VARA VARB WITH VARC VARD.
```

- This example displays and plots the cross-correlation functions for the following pairs of series: *VARA* with *VARC*, *VARA* with *VARD*, *VARB* with *VARC*, and *VARB* with *VARD*.
- *VARA* is not paired with *VARB*, and *VARC* is not paired with *VARD*.

DIFF Subcommand

DIFF specifies the degree of differencing used to convert a nonstationary series to a stationary one with a constant mean and variance before obtaining cross-correlations.

- You can specify 0 or any positive integer on DIFF.
- If DIFF is specified without a value, the default is 1.
- The number of values used in the calculations decreases by 1 for each degree of differencing.

Example

```
CCF VARIABLES = VARX VARY
  /DIFF=1.
```

- This command differences series *VARX* and *VARY* before calculating and plotting the cross-correlation function.

SDIFF Subcommand

If the series exhibits seasonal or periodic patterns, you can use SDIFF to seasonally difference the series before obtaining cross-correlations.

- The specification on SDIFF indicates the degree of seasonal differencing and can be 0 or any positive integer.
- If SDIFF is specified without a value, the degree of seasonal differencing defaults to 1.
- The number of seasons used in the calculations decreases by 1 for each degree of seasonal differencing.
- The length of the period used by SDIFF is specified on the PERIOD subcommand. If the PERIOD subcommand is not specified, the periodicity established on the TSET or DATE command is used (see the PERIOD subcommand).

Example

```
CCF VARIABLES = VAR01 WITH VAR02 VAR03
  /SDIFF=1.
```

- In this example, one degree of seasonal differencing using the periodicity established on the TSET or DATE command is applied to the three series.
- Two cross-correlation functions are then plotted, one for the pair *VAR01* and *VAR02*, and one for the pair *VAR01* and *VAR03*.

PERIOD Subcommand

PERIOD indicates the length of the period to be used by the SDIFF or SEASONAL subcommands.

- The specification on PERIOD indicates how many observations are in one period or season and can be any positive integer greater than 1.
- PERIOD is ignored if it is used without the SDIFF or SEASONAL subcommands.
- If PERIOD is not specified, the periodicity established on TSET PERIOD is in effect. If TSET PERIOD is not specified, the periodicity established on the DATE command is used. If periodicity was not established anywhere, the SDIFF and SEASONAL subcommands will not be executed.

Example

```
CCF VARIABLES = VARX WITH VARY
  /SDIFF=1
  /PERIOD=6.
```

- This command applies one degree of seasonal differencing with a periodicity of 6 to both series and computes and plots the cross-correlation function.

LN and NOLOG Subcommands

LN transforms the data using the natural logarithm (base *e*) of each series and is used to remove varying amplitude over time. NOLOG indicates that the data should not be log transformed. NOLOG is the default.

- There are no additional specifications on LN or NOLOG.

- Only the last LN or NOLOG subcommand on a CCF command is executed.
- LN and NOLOG apply to all series named on the CCF command.
- If a natural log transformation is requested and any values in either series in a pair are less than or equal to 0, the CCF for that pair will not be produced because nonpositive values cannot be log transformed.
- NOLOG is generally used with an APPLY subcommand to turn off a previous LN specification.

Example

```
CCF VARIABLES = VAR01 VAR02
  /LN.
```

- This command transforms the series *VAR01* and *VAR02* using the natural log before computing cross-correlations.

SEASONAL Subcommand

Use SEASONAL to focus attention on the seasonal component by displaying and plotting cross-correlations at periodic lags only.

- There are no additional specifications on SEASONAL.
- If SEASONAL is specified, values are displayed and plotted at the periodic lags indicated on the PERIOD subcommand. If no PERIOD subcommand is specified, the periodicity first defaults to the TSET PERIOD specification and then to the DATE command periodicity. If periodicity is not established anywhere, SEASONAL is ignored (see the PERIOD subcommand).
- If SEASONAL is not used, cross-correlations for all lags up to the maximum are displayed and plotted.

Example

```
CCF VARIABLES = VAR01 VAR02 VAR03
  /SEASONAL.
```

- This command plots and displays cross-correlations at periodic lags.
- By default, the periodicity established on TSET PERIOD (or the DATE command) is used. If no periodicity is established, cross-correlations for all lags are displayed and plotted.

MXCROSS Subcommand

MXCROSS specifies the maximum range of lags for a series.

- The specification on MXCROSS must be a positive integer.
- If MXCROSS is not specified, the default range is the value set on TSET MXCROSS. If TSET MXCROSS is not specified, the default is 7 (lags -7 to +7).
- The value specified on the MXCROSS subcommand overrides the value set on TSET MXCROSS.

Example

```
CCF VARIABLES = VARX VARY
  /MXCROSS=5.
```

- The maximum number of cross-correlations can range from lag –5 to lag +5.

APPLY Subcommand

APPLY allows you to use a previously defined CCF model without having to repeat the specifications.

- The only specification on APPLY is the name of a previous model enclosed in single or double quotes. If a model name is not specified, the model specified on the previous CCF command is used.

- To change one or more model specifications, specify the subcommands of only those portions you want to change after the APPLY subcommand.
- If no series are specified on the command, the series that were originally specified with the model being applied are used.
- To change the series used with the model, enter new series names before or after the APPLY subcommand.

Example

```
CCF VARIABLES = VARX VARY
  /LN
  /DIFF=1
  /MXCROSS=25.
CCF VARIABLES = VARX VARY
  /LN
  /DIFF=1
  /SDIFF=1
  /PERIOD=12
  /MXCROSS=25.
CCF VARIABLES = VARX VAR01
  /APPLY.
CCF VARIABLES = VARX VAR01
  /APPLY='MOD_1'.
```

- The first command displays and plots the cross-correlation function for *VARX* and *VARY* after each series is log transformed and differenced. The maximum range is set to 25 lags. This model is assigned the name *MOD_1* as soon as the command is executed.
- The second command displays and plots the cross-correlation function for *VARX* and *VARY* after each series is log transformed, differenced, and seasonally differenced with a periodicity of 12. The maximum range is again set to 25 lags. This model is assigned the name *MOD_2*.
- The third command requests the cross-correlation function for the series *VARX* and *VAR01* using the same model and the same range of lags as used for *MOD_2*.
- The fourth command applies *MOD_1* (from the first command) to the series *VARX* and *VAR01*.

References

Box, G. E. P., and G. M. Jenkins. 1976. *Time series analysis: Forecasting and control*, Rev. ed. San Francisco: Holden-Day.

CD

```
CD 'directory specification'.
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 13.0

- Command introduced.

Example

```
CD '/main/sales/consumer_division/2004/data'.
GET FILE='julydata.sav'.
INSERT FILE='../commands/monthly_report.sps'.
```

Overview

CD changes the working directory location, making it possible to use relative paths for subsequent file specifications in command syntax, including data files specified on commands such as GET and SAVE, command syntax files specified on commands such as INSERT and INCLUDE, and output files specified on commands such as OMS and WRITE.

Basic Specification

The only specification is the command name followed by a quoted directory specification.

- The directory specification can contain a drive specification.
- The directory specification can be a previously defined file handle (see the FILE HANDLE command for more information).
- The directory specification can include paths defined in operating system environment variables.

Operations

The change in the working directory remains in effect until some other condition occurs that changes the working directory during the session, such as explicitly changing the working directory on another CD command or an INSERT command with a CD keyword that specifies a different directory.

- If the directory path is a relative path, it is relative to the current working directory.
- If the directory specification contains a filename, the filename portion is ignored.
- If the last (most-nested) subdirectory in the directory specification does not exist, then it is assumed to be a filename and is ignored.
- If any directory specification prior to the last directory (or file) is invalid, the command will fail, and an error message is issued.

Limitations

The CD command has no effect on the relative directory location for SET TLOOK file specifications. File specifications for the TLOOK subcommand of the SET command should include complete path information.

Examples

Working with Absolute Paths

```
CD '/sales/data/july.sav'.
CD '/sales/data/july'.
CD '/sales/data/july'.
```

If */sales/data* is a valid directory:

- The first CD command will ignore the filename *july.sav* and set the working directory to */sales/data*.
- If the subdirectory *july* exists, the second CD command will change the working directory to */sales/data/july*; otherwise, it will change the working directory to */sales/data*.
- The third CD command will fail if the *data* subdirectory doesn't exist.

Working with Relative Paths

```
CD '/sales'.
CD 'data'.
CD 'july'.
```

If */sales* is a valid directory:

- The first CD command will change the working directory to */sales*.
- The relative path in the second CD command will change the working directory to */sales/data*.
- The relative path in the third CD command will change the working directory to */sales/data/july*.

Preserving and Restoring the Working Directory Setting

The original working directory can be preserved with the PRESERVE command and later restored with the RESTORE command.

Example

```
CD '/sales/data'.
PRESERVE.
CD '/commands/examples'.
RESTORE.
```

- PRESERVE retains the working directory location set on the preceding CD command.
- The second CD command changes the working directory.
- RESTORE resets the working directory back to */sales/data*.

CLEAR TIME PROGRAM

```
CLEAR TIME PROGRAM.
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Overview

CLEAR TIME PROGRAM deletes all time-dependent covariates created in the previous TIME PROGRAM command. It is primarily used in interactive mode to remove temporary variables associated with the time program so that you can redefine time-dependent covariates. It is not necessary to use this command if you have run a procedure that executes the TIME PROGRAM transformations, since all temporary variables created by TIME PROGRAM are automatically deleted.

Basic Specification

The only specification is the command itself. CLEAR TIME PROGRAM has no additional specifications.

Example

```
TIME PROGRAM.
COMPUTE Z=AGE + T_.

CLEAR TIME PROGRAM.

TIME PROGRAM.
COMPUTE Z=AGE + T_ - 18.

COXREG SURVIVAL WITH Z
  /STATUS SURVSTA EVENT(1).
```

- The first TIME PROGRAM command defines the time-dependent covariate *Z* as the current age.
- The CLEAR TIME PROGRAM command deletes the time-dependent covariate *Z*.
- The second TIME PROGRAM command redefines the time-dependent covariate *Z* as the number of years since turning 18.. *Z* is then specified as a covariate in COXREG.

CLEAR TRANSFORMATIONS

```
CLEAR TRANSFORMATIONS
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Overview

CLEAR TRANSFORMATIONS discards previous data transformation commands.

Basic Specification

The only specification is the command itself. CLEAR TRANSFORMATIONS has no additional specifications.

Operations

- CLEAR TRANSFORMATIONS discards all data transformation commands that have accumulated since the last procedure.
- CLEAR TRANSFORMATIONS has no effect if a command file is submitted to your operating system for execution. It generates a warning when a command file is present.
- Be sure to delete CLEAR TRANSFORMATIONS and any unwanted transformation commands from the journal file if you plan to submit the file to the operating system for batch mode execution. Otherwise, the unwanted transformations will cause problems.

Examples

```
GET FILE="/data/query.sav".
FREQUENCIES=ITEM1 ITEM2 ITEM3.
RECODE ITEM1, ITEM2, ITEM3 (0=1) (1=0) (2=-1).
COMPUTE INDEXQ=(ITEM1 + ITEM2 + ITEM3)/3.
VARIABLE LABELS INDEXQ 'SUMMARY INDEX OF QUESTIONS'.
CLEAR TRANSFORMATIONS.
DISPLAY DICTIONARY.
```

- The GET and FREQUENCIES commands are executed.
- The RECODE, COMPUTE, and VARIABLE LABELS commands are transformations. They do not affect the data until the next procedure is executed.
- The CLEAR TRANSFORMATIONS command discards the RECODE, COMPUTE, and VARIABLE LABELS commands.
- The DISPLAY command displays the working file dictionary. Data values and labels are exactly as they were when the FREQUENCIES command was executed. The variable *INDEXQ* does not exist because CLEAR TRANSFORMATIONS discarded the COMPUTE command.

CLUSTER

CLUSTER is available in the Statistics Base Edition.

```
CLUSTER varlist [/MISSING=[EXCLUDE**] [INCLUDE]]

[/MEASURE=[{SEUCLID**         }]
          {EUCLID             }
          {COSINE             }
          {CORRELATION        }
          {BLOCK              }
          {CHEBYCHEV          }
          {POWER(p,r)         }
          {MINKOWSKI(p)       }
          {CHISQ              }
          {PH2                }
          {RR[(p[,np])]       }
          {SM[(p[,np])]       }
          {JACCARD[(p[,np])]  }
          {DICE[(p[,np])]     }
          {SS1[(p[,np])]      }
          {RT[(p[,np])]       }
          {SS2[(p[,np])]      }
          {K1[(p[,np])]       }
          {SS3[(p[,np])]      }
          {K2[(p[,np])]       }
          {SS4[(p[,np])]      }
          {HAMANN[(p[,np])]   }
          {OCHIAI[(p[,np])]   }
          {SS5[(p[,np])]      }
          {PHI[(p[,np])]      }
          {LAMBDA[(p[,np])]   }
          {D[(p[,np])]        }
          {Y[(p[,np])]        }
          {Q[(p[,np])]        }
          {BEUCLID[(p[,np])]  }
          {SIZE[(p[,np])]     }
          {PATTERN[(p[,np])]  }
          {BSEUCLID[(p[,np])]}
          {BSHAPE[(p[,np])]   }
          {DISPER[(p[,np])]   }
          {VARIANCE[(p[,np])]}
          {BLWMN[(p[,np])]    }


[/METHOD={BAVERAGE**}[(rootname)] [...]]
         {WAVERAGE  }
         {SINGLE    }
         {COMPLETE  }
         {CENTROID  }
         {MEDIAN    }
         {WARD      }
         {DEFAULT** }

 [/SAVE=CLUSTER({level  })]  [/ID=varname]
               {min,max}

 [/PRINT=[CLUSTER({level  })] [DISTANCE] [SCHEDULE**] [NONE]]
                 {min,max}

 [/PLOT=[VICICLE**[(min[,max[,inc]])]] [DENDROGRAM] [NONE]]
        [HICICLE[(min[,max[,inc]])]]

 [/MATRIX=[IN({'savfile'|'dataset'})] [OUT({'savfile'|'dataset'})]]
             {*                  }        {*                  }
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
CLUSTER V1 TO V4
 /PLOT=DENDROGRAM
 /PRINT=CLUSTER (2,4).
```

Overview

CLUSTER produces hierarchical clusters of items based on distance measures of dissimilarity or similarity. The items being clustered are usually cases from the active dataset, and the distance measures are computed from their values for one or more variables. You can also cluster variables if you read in a matrix measuring distances between variables. Cluster analysis is discussed in Anderberg (1973).

Options

Cluster Measures and Methods. You can specify one of 37 similarity or distance measures on the MEASURE subcommand and any of the seven methods on the METHOD subcommand.

New Variables. You can save cluster membership for specified solutions as new variables in the active dataset using the SAVE subcommand.

Display and Plots. You can display cluster membership, the distance or similarity matrix used to cluster variables or cases, and the agglomeration schedule for the cluster solution with the PRINT subcommand. You can request either a horizontal or vertical icicle plot or a dendrogram of the cluster solution and control the cluster levels displayed in the icicle plot with the PLOT subcommand. You can also specify a variable to be used as a case identifier in the display on the ID subcommand.

Matrix Input and Output. You can write out the distance matrix and use it in subsequent CLUSTER, PROXIMITIES, or ALSCAL analyses or read in matrices produced by other CLUSTER or PROXIMITIES procedures using the MATRIX subcommand.

Basic Specification

The basic specification is a variable list. CLUSTER assumes that the items being clustered are cases and uses the squared Euclidean distances between cases on the variables in the analysis as the measure of distance.

Subcommand Order

- The variable list must be specified first.
- The remaining subcommands can be specified in any order.

Syntax Rules

- The variable list and subcommands can each be specified once.
- More than one clustering method can be specified on the METHOD subcommand.

Operations

The CLUSTER procedure involves four steps:

- First, CLUSTER obtains distance measures of similarities between or distances separating initial clusters (individual cases or individual variables if the input is a matrix measuring distances between variables).
- Second, it combines the two nearest clusters to form a new cluster.
- Third, it recomputes similarities or distances of existing clusters to the new cluster.
- It then returns to the second step until all items are combined in one cluster.

This process yields a hierarchy of cluster solutions, ranging from one overall cluster to as many clusters as there are items being clustered. Clusters at a higher level can contain several lower-level clusters. Within each level, the clusters are disjoint (each item belongs to only one cluster).

- CLUSTER identifies clusters in solutions by sequential integers (1, 2, 3, and so on).

Limitations

- CLUSTER stores cases and a lower-triangular matrix of proximities in memory. Storage requirements increase rapidly with the number of cases. You should be able to cluster 100 cases using a small number of variables in an 80K workspace.
- CLUSTER does not honor weights.

Example

```
CLUSTER V1 TO V4
  /PLOT=DENDROGRAM
  /PRINT=CLUSTER (2 4).
```

- This example clusters cases based on their values for all variables between and including *V1* and *V4* in the active dataset.
- The analysis uses the default measure of distance (squared Euclidean) and the default clustering method (average linkage between groups).
- PLOT requests a dendrogram.
- PRINT displays a table of the cluster membership of each case for the two-, three-, and four-cluster solutions.

Used with the PROXIMITIES command to create distances

```
PROXIMITIES  price engine_s horsepow wheelbas width length
  curb_wgt fuel_cap mpg
  /MATRIX OUT  ('C:/TEMP/spssclus.tmp')
  /VIEW= CASE
  /MEASURE= SEUCLID
  /PRINT  NONE
  /ID= model
  /STANDARDIZE= VARIABLE Z .
CLUSTER
  /MATRIX IN  ('C:/TEMP/spssclus.tmp')
  /METHOD SINGLE
  /ID= model
  /PRINT SCHEDULE
  /PLOT DENDROGRAM.
ERASE FILE= 'C:/TEMP/spssclus.tmp'.
```

Variable List

The variable list identifies the variables used to compute similarities or distances between cases.

- The variable list is required except when matrix input is used. It must be specified before the optional subcommands.
- If matrix input is used, the variable list can be omitted. The names for the items in the matrix are used to compute similarities or distances.
- You can specify a variable list to override the names for the items in the matrix. This allows you to read in a subset of cases for analysis. Specifying a variable that does not exist in the matrix results in an error.

MEASURE Subcommand

MEASURE specifies the distance or similarity measure used to cluster cases.

- If the MEASURE subcommand is omitted or included without specifications, squared Euclidean distances are used.
- Only one measure can be specified.

Measures for Interval Data

For interval data, use any one of the following keywords on MEASURE:

SEUCLID. *Squared Euclidean distance.* The distance between two items, *x* and *y*, is the sum of the squared differences between the values for the items. SEUCLID is the measure commonly used with centroid, median, and Ward's methods of clustering. SEUCLID is the default and can also be requested with keyword DEFAULT.

EUCLID. *Euclidean distance.* This is the default specification for MEASURE. The distance between two items, *x* and *y*, is the square root of the sum of the squared differences between the values for the items.

CORRELATION. *Correlation between vectors of values.* This is a pattern similarity measure.

COSINE. *Cosine of vectors of values.* This is a pattern similarity measure.

CHEBYCHEV. *Chebychev distance metric.* The distance between two items is the maximum absolute difference between the values for the items.

BLOCK. *City-block or Manhattan distance.* The distance between two items is the sum of the absolute differences between the values for the items.

MINKOWSKI(p). *Distance in an absolute Minkowski power metric.* The distance between two items is the *p*th root of the sum of the absolute differences to the *p*th power between the values for the items. Appropriate selection of the integer parameter *p* yields Euclidean and many other distance metrics.

POWER(p,r). *Distance in an absolute power metric.* The distance between two items is the *r*th root of the sum of the absolute differences to the *p*th power between the values for the items. Appropriate selection of the integer parameters *p* and *r* yields Euclidean, squared Euclidean, Minkowski, city-block, and many other distance metrics.

Measures for Frequency Count Data

For frequency count data, use any one of the following keywords on MEASURE:

CHISQ. *Based on the chi-square test of equality for two sets of frequencies.* The magnitude of this dissimilarity measure depends on the total frequencies of the two cases or variables whose dissimilarity is computed. Expected values are from the model of independence of cases or variables *x* and *y*.

PH2. *Phi-square between sets of frequencies.* This is the CHISQ measure normalized by the square root of the combined frequency. Therefore, its value does not depend on the total frequencies of the two cases or variables whose dissimilarity is computed.

Measures for Binary Data

Different binary measures emphasize different aspects of the relationship between sets of binary values. However, all the measures are specified in the same way. Each measure has two optional integer-valued parameters, *p* (present) and *np* (not present).

- If both parameters are specified, CLUSTER uses the value of the first as an indicator that a characteristic is present and the value of the second as an indicator that a characteristic is absent. CLUSTER skips all other values.
- If only the first parameter is specified, CLUSTER uses that value to indicate presence and all other values to indicate absence.
- If no parameters are specified, CLUSTER assumes that 1 indicates presence and 0 indicates absence.

Using the indicators for presence and absence within each item (case or variable), CLUSTER constructs a 2 x 2 contingency table for each pair of items in turn. It uses this table to compute a proximity measure for the pair.

Table 19. 2 x 2 contingency table

	Item 2 characteristics Present	**Item 2 characteristics Absent**
Item 1 characteristics Present	a	b
Item 1 characteristics Absent	c	d

CLUSTER computes all binary measures from the values of *a*, *b*, *c*, and *d*. These values are tallied across variables (when the items are cases) or across cases (when the items are variables). For example, if the variables *V*, *W*, *X*, *Y*, *Z* have values 0, 1, 1, 0, 1 for case 1 and values 0, 1, 1, 0, 0 for case 2 (where 1 indicates presence and 0 indicates absence), the contingency table is as follows:

Table 20. 2 x 2 contingency table

	Case 2 characteristics Present	**Case 2 characteristics Absent**
Case 1 characteristics Present	2	1
Case 1 characteristics Absent	0	2

The contingency table indicates that both cases are present for two variables (*W* and *X*), both cases are absent for two variables (*V* and *Y*), and case 1 is present and case 2 is absent for one variable (*Z*). There are no variables for which case 1 is absent and case 2 is present.

The available binary measures include matching coefficients, conditional probabilities, predictability measures, and others.

Matching Coefficients. The table below shows a classification scheme for matching coefficients. In this scheme, *matches* are joint presences (value *a* in the contingency table) or joint absences (value *d*). *Nonmatches* are equal in number to value *b* plus value *c*. Matches and non-matches may or may not be weighted equally. The three coefficients JACCARD, DICE, and SS2 are related monotonically, as are SM, SS1, and RT. All coefficients in the table are similarity measures, and all except two (K1 and SS3) range from 0 to 1. K1 and SS3 have a minimum value of 0 and no upper limit.

Table 21. Binary matching coefficients, all matches included in denominator

	Joint absences excluded from numerator	**Joint absences included in numerator**
Equal weight for matches and non-matches	RR	SM
Double weight for matches		SSL
Double weight for non-matches		RT

Table 22. Binary matching coefficients, joint absences excluded from denominator

	Joint absences excluded from numerator	**Joint absences included in numerator**
Equal weight for matches and non-matches	JACCARD	
Double weight for matches	DICE	
Double weight for non-matches	SS2	

Table 23. Binary matching coefficients, all matches excluded from denominator

	Joint absences excluded from numerator	**Joint absences included in numerator**
Equal weight for matches and non-matches	K1	SS3

RR[(p[,np])]. Russell and Rao similarity measure. This is the binary dot product.

SM[(p[,np])]. Simple matching similarity measure. This is the ratio of the number of matches to the total number of characteristics.

JACCARD[(p[,np])]. Jaccard similarity measure. This is also known as the *similarity ratio.*

DICE[(p[,np])]. Dice (or Czekanowski or Sorenson) similarity measure.

SS1[(p[,np])]. Sokal and Sneath similarity measure 1.

RT[(p[,np])]. Rogers and Tanimoto similarity measure.

SS2[(p[,np])]. Sokal and Sneath similarity measure 2.

K1[(p[,np])]. Kulczynski similarity measure 1. This measure has a minimum value of 0 and no upper limit. It is undefined when there are no non-matches (*b*=0 and *c*=0).

SS3[(p[,np])]. Sokal and Sneath similarity measure 3. This measure has a minimum value of 0 and no upper limit. It is undefined when there are no non-matches (*b*=0 and *c*=0).

Conditional Probabilities. The following binary measures yield values that can be interpreted in terms of conditional probability. All three are similarity measures.

K2[(p[,np])]. Kulczynski similarity measure 2. This yields the average conditional probability that a characteristic is present in one item given that the characteristic is present in the other item. The measure is an average over both items acting as predictors. It has a range of 0 to 1.

SS4[(p[,np])]. Sokal and Sneath similarity measure 4. This yields the conditional probability that a characteristic of one item is in the same state (presence or absence) as the characteristic of the other item. The measure is an average over both items acting as predictors. It has a range of 0 to 1.

HAMANN[(p[,np])]. Hamann similarity measure. This measure gives the probability that a characteristic has the same state in both items (present in both or absent from both) minus the probability that a characteristic has different states in the two items (present in one and absent from the other). HAMANN has a range of –1 to +1 and is monotonically related to SM, SS1, and RT.

Predictability Measures. The following four binary measures assess the association between items as the predictability of one given the other. All four measures yield similarities.

LAMBDA[(p[,np])]. Goodman and Kruskal's lambda (similarity). This coefficient assesses the predictability of the state of a characteristic on one item (present or absent) given the state on the other item. Specifically, LAMBDA measures the proportional reduction in error using one item to predict the other when the directions of prediction are of equal importance. LAMBDA has a range of 0 to 1.

D[(p[,np])]. Anderberg's *D* (similarity). This coefficient assesses the predictability of the state of a characteristic on one item (present or absent) given the state on the other. D measures the actual reduction in the error probability when one item is used to predict the other. The range of D is 0 to 1.

Y[(p[,np])]. Yule's *Y* coefficient of colligation (similarity). This is a function of the cross-ratio for a 2 x 2 table. It has range of -1 to +1.

Q[(p[,np])]. Yule's *Q* (similarity). This is the 2 x 2 version of Goodman and Kruskal's ordinal measure *gamma.* Like Yule's *Y*, Q is a function of the cross-ratio for a 2 x 2 table and has a range of -1 to +1.

Other Binary Measures. The remaining binary measures available in CLUSTER are either binary equivalents of association measures for continuous variables or measures of special properties of the relationship between items.

OCHIAI[(p[,np])]. Ochiai similarity measure. This is the binary form of the cosine. It has a range of 0 to 1.

SS5[(p[,np])]. Sokal and Sneath similarity measure 5. The range is 0 to 1.

PHI[(p[,np])]. Fourfold point correlation (similarity). This is the binary form of the Pearson product-moment correlation coefficient.

BEUCLID[(p[,np])]. Binary Euclidean distance. This is a distance measure. Its minimum value is 0, and it has no upper limit.

BSEUCLID[(p[,np])]. Binary squared Euclidean distance. This is a distance measure. Its minimum value is 0, and it has no upper limit.

SIZE[(p[,np])]. Size difference. This is a dissimilarity measure with a minimum value of 0 and no upper limit.

PATTERN[(p[,np])]. Pattern difference. This is a dissimilarity measure. The range is 0 to 1.

BSHAPE[(p[,np])]. Binary shape difference. This dissimilarity measure has no upper or lower limit.

DISPER[(p[,np])]. Dispersion similarity measure. The range is –1 to +1.

VARIANCE[(p[,np])]. Variance dissimilarity measure. This measure has a minimum value of 0 and no upper limit.

BLWMN[(p[,np])]. Binary Lance-and-Williams nonmetric dissimilarity measure. This measure is also known as the Bray-Curtis nonmetric coefficient. The range is 0 to 1.

METHOD Subcommand

METHOD specifies one or more clustering methods.

- If the METHOD subcommand is omitted or included without specifications, the method of average linkage between groups is used.
- Only one METHOD subcommand can be used, but more than one method can be specified on it.
- When the number of items is large, CENTROID and MEDIAN require significantly more CPU time than other methods.

BAVERAGE. *Average linkage between groups (UPGMA).* BAVERAGE is the default and can also be requested with keyword DEFAULT.

WAVERAGE. *Average linkage within groups.*

SINGLE. *Single linkage or nearest neighbor.*

COMPLETE. *Complete linkage or furthest neighbor.*

CENTROID. *Centroid clustering (UPGMC).* Squared Euclidean distances are commonly used with this method.

MEDIAN. *Median clustering (WPGMC).* Squared Euclidean distances are commonly used with this method.

WARD. *Ward's method.* Squared Euclidean distances are commonly used with this method.

Example

```
CLUSTER V1 V2 V3
 /METHOD=SINGLE COMPLETE WARDS.
```

- This example clusters cases based on their values for the variables *V1*, *V2*, and *V3* and uses three clustering methods: single linkage, complete linkage, and Ward's method.

SAVE Subcommand

SAVE allows you to save cluster membership at specified solution levels as new variables in the active dataset.

- The specification on SAVE is the CLUSTER keyword, followed by either a single number indicating the level (number of clusters) of the cluster solution or a range separated by a comma indicating the minimum and maximum numbers of clusters when membership of more than one solution is to be saved. The number or range must be enclosed in parentheses and applies to all methods specified on METHOD.
- You can specify a rootname in parentheses after each method specification on the METHOD subcommand. CLUSTER forms new variable names by appending the number of the cluster solution to the rootname.
- If no rootname is specified, CLUSTER forms variable names using the formula *CLUn_m*, where *m* increments to create a unique rootname for the set of variables saved for one method and *n* is the number of the cluster solution.
- The names and descriptive labels of the new variables are displayed in the procedure information notes.
- You cannot use the SAVE subcommand if you are replacing the active dataset with matrix materials (See the topic "Matrix Output " on page 320 for more information.)

Example

```
CLUSTER A B C
 /METHOD=BAVERAGE SINGLE (SINMEM) WARD
 /SAVE=CLUSTERS(3,5).
```

- This command creates nine new variables: *CLU5_1*, *CLU4_1*, and *CLU3_1* for BAVERAGE, *SINMEM5*, *SINMEM4*, and *SINMEM3* for SINGLE, and *CLU5_2*, *CLU4_2*, and *CLU3_2* for WARD. The variables contain the cluster membership for each case at the five-, four-, and three-cluster solutions using the three clustering methods. Ward's method is the third specification on METHOD but uses the second set of default names, since it is the second method specified without a rootname.
- The order of the new variables in the active dataset is the same as listed above, since the solutions are obtained in the order from 5 to 3.
- New variables are listed in the procedure information notes.

ID Subcommand

ID names a string variable to be used as the case identifier in cluster membership tables, icicle plots, and dendrograms. If the ID subcommand is omitted, cases are identified by case numbers alone.

- When used with the MATRIX IN subcommand, the variable specified on the ID subcommand identifies the labeling variable in the matrix file.

PRINT Subcommand

PRINT controls the display of cluster output (except plots, which are controlled by the PLOT subcommand).

- If the PRINT subcommand is omitted or included without specifications, an agglomeration schedule is displayed. If any keywords are specified on PRINT, the agglomeration schedule is displayed only if explicitly requested.
- CLUSTER automatically displays summary information (the method and measure used, the number of cases) for each method named on the METHOD subcommand. This summary is displayed regardless of specifications on PRINT.

You can specify any or all of the following on the PRINT subcommand:

SCHEDULE. *Agglomeration schedule.* The agglomeration schedule shows the order and distances at which items and clusters combine to form new clusters. It also shows the cluster level at which an item joins a cluster. SCHEDULE is the default and can also be requested with the keyword DEFAULT.

CLUSTER(min,max). *Cluster membership.* For each item, the display includes the value of the case identifier (or the variable name if matrix input is used), the case sequence number, and a value (1, 2, 3, and so on) identifying the cluster to which that case belongs in a given cluster solution. Specify either a single integer value in parentheses indicating the level of a single solution or a minimum value and a maximum value indicating a range of solutions for which display is desired. If the number of clusters specified exceeds the number produced, the largest number of clusters is used (the number of items minus 1). If CLUSTER is specified more than once, the last specification is used.

DISTANCE. *Proximities matrix.* The proximities matrix table displays the distances or similarities between items computed by CLUSTER or obtained from an input matrix. DISTANCE produces a large volume of output and uses significant CPU time when the number of cases is large.

NONE. *None of the above.* NONE overrides any other keywords specified on PRINT.

Example

```
CLUSTER V1 V2 V3 /PRINT=CLUSTER(3,5).
```

- This example displays cluster membership for each case for the three-, four-, and five-cluster solutions.

PLOT Subcommand

PLOT controls the plots produced for each method specified on the METHOD subcommand. For icicle plots, PLOT allows you to control the cluster solution at which the plot begins and ends and the increment for displaying intermediate cluster solutions.

- If the PLOT subcommand is omitted or included without specifications, a vertical icicle plot is produced.
- If any keywords are specified on PLOT, only those plots requested are produced.
- The icicle plots are generated as pivot tables and the dendrogram is generated as text output.
- If there is not enough memory for a dendrogram or an icicle plot, the plot is skipped and a warning is issued.
- The size of an icicle plot can be controlled by specifying range values or an increment for VICICLE or HICICLE. Smaller plots require significantly less workspace and time.

VICICLE(min,max,inc). *Vertical icicle plot.* This is the default. The range specifications are optional. If used, they must be integer and must be enclosed in parentheses. The specification *min* is the cluster solution at which to start the display (the default is 1), and the specification *max* is the cluster solution at which to end the display (the default is the number of cases minus 1). If *max* is greater than the number of cases minus 1, the default is used. The increment to use between cluster solutions is *inc* (the default is 1). If *max* is specified, *min* must be specified, and if *inc* is specified, both *min* and *max* must be specified. If VICICLE is specified more than once, only the last range specification is used.

HICICLE(min,max,inc). *Horizontal icicle plot.* The range specifications are the same as for VICICLE. If both VICICLE and HICICLE are specified, the last range specified is used for both. If a range is not specified on the last instance of VICICLE or HICICLE, the defaults are used even if a range is specified earlier.

DENDROGRAM. *Tree diagram.* The dendrogram is scaled by the joining distances of the clusters.

NONE. *No plots.*

Example

```
CLUSTER V1 V2 V3 /PLOT=VICICLE(1,20).
```

- This example produces a vertical icicle plot for the 1-cluster through the 20-cluster solution.

Example

```
CLUSTER V1 V2 V3 /PLOT=VICICLE(1,151,5).
```

- This example produces a vertical icicle plot for every fifth cluster solution starting with 1 and ending with 151 (1 cluster, 6 clusters, 11 clusters, and so on).

MISSING Subcommand

MISSING controls the treatment of cases with missing values. A case that has a missing value for any variable on the variable list is omitted from the analysis. By default, user-missing values are excluded from the analysis.

EXCLUDE. *Exclude cases with user-missing values.* This is the default.

INCLUDE. *Include cases with user-missing values.* Only cases with system-missing values are excluded.

MATRIX Subcommand

MATRIX reads and writes IBM SPSS Statistics data files.

- Either IN or OUT and a matrix file in parentheses are required. When both IN and OUT are used on the same CLUSTER procedure, they can be specified on separate MATRIX subcommands or on the same subcommand.
- The input or output matrix information is displayed in the procedure information notes.

OUT ('savfile'|'dataset'). *Write a matrix data file.* Specify either a quoted file specification, a previously declared dataset (DATASET DECLARE), or an asterisk in parentheses (*). If you specify an asterisk (*), the matrix data file replaces the active dataset.

IN ('savfile'|'dataset'). *Read a matrix data file.* Specify either a quoted file specification, a previously declared dataset (DATASET DECLARE), or an asterisk in parentheses (*). The asterisk specifies the active dataset. A matrix file read from an external file does not replace the active dataset.

When a matrix is produced using the MATRIX OUT subcommand, it corresponds to a unique dataset. All subsequent analyses performed on this matrix would match the corresponding analysis on the original data. However, if the data file is altered in any way, this would no longer be true.

For example, if the original file is edited or rearranged, it would in general no longer correspond to the initially produced matrix. You need to make sure that the data match the matrix whenever inferring the results from the matrix analysis. Specifically, when saving the cluster membership into an active dataset in the CLUSTER procedure, the proximity matrix in the MATRIX IN statement must match the current active dataset.

Matrix Output

- CLUSTER writes proximity-type matrices with *ROWTYPE_* values of PROX. CLUSTER neither reads nor writes additional statistics with its matrix materials. See the topic "Format of the Matrix Data File " on page 321 for more information.
- The matrices produced by CLUSTER can be used by subsequent CLUSTER procedures or by the PROXIMITIES and ALSCAL procedures.
- Any documents contained in the active dataset are not transferred to the matrix file.

Matrix Input

- CLUSTER can read matrices written by a previous CLUSTER command or by PROXIMITIES, or created by MATRIX DATA. When the input matrix contains distances between variables, CLUSTER clusters all or a subset of the variables.
- Values for split-file variables should precede values for *ROWTYPE_*. *CASENO_* and the labeling variable (if present) should come after *ROWTYPE_* and before *VARNAME_*.
- If *CASENO_* is of type string rather than numeric, it will be considered unavailable and a warning is issued.
- If *CASENO_* appears on a variable list, a syntax error results.
- CLUSTER ignores unrecognized *ROWTYPE_* values.
- When you are reading a matrix created with MATRIX DATA, you should supply a value label for PROX of either *SIMILARITY* or *DISSIMILARITY* so that the matrix is correctly identified. If you do not supply a label, CLUSTER assumes *DISSIMILARITY*. (See "Format of the Matrix Data File" below.)
- The program reads variable names, variable and value labels, and print and write formats from the dictionary of the matrix data file.
- MATRIX=IN cannot be specified unless an active dataset has already been defined. To read an existing matrix data file at the beginning of a session, use GET to retrieve the matrix file and then specify IN(*) on MATRIX.
- The variable list on CLUSTER can be omitted when a matrix data file is used as input. By default, all cases or variables in the matrix data file are used in the analysis. Specify a variable list when you want to read in a subset of items for analysis.

Format of the Matrix Data File

- The matrix data file can include three special variables created by the program: *ROWTYPE_*, *ID*, and *VARNAME_*.
- The variable *ROWTYPE_* is a string variable with the value PROX (for proximity measure). PROX is assigned value labels containing the distance measure used to create the matrix and either *SIMILARITY* or *DISSIMILARITY* as an identifier. The variable *VARNAME_* is a short string variable whose values are the names of the new variables. The variable *CASENO_* is a numeric variable with values equal to the original case numbers.
- *ID* is included only when an identifying variable is not specified on the ID subcommand. *ID* is a short string and takes the value CASE *m*, where *m* is the actual number of each case. Note that *m* may not be consecutive if cases have been selected.
- If an identifying variable is specified on the ID subcommand, it takes the place of *ID* between *ROWTYPE_* and *VARNAME_*. Up to 20 characters can be displayed for the identifying variable.
- *VARNAME_* is a string variable that takes the values VAR1, VAR2, ..., VAR*n* to correspond to the names of the distance variables in the matrix (*VAR1, VAR2, ...*, *VARn*, where *n* is the number of cases in the largest split file). The numeric suffix for the variable names is consecutive and may not be the same as the actual case number.
- The remaining variables in the matrix file are the distance variables used to form the matrix. The distance variables are assigned variable labels in the form of CASE *m* to identify the actual number of each case.

Split Files

- When split-file processing is in effect, the first variables in the matrix data file are the split variables, followed by *ROWTYPE_*, the case-identifier variable or *ID*, *VARNAME_*, and the distance variables.
- A full set of matrix materials is written for each split-file group defined by the split variables.
- A split variable cannot have the same name as any other variable written to the matrix data file.
- If split-file processing is in effect when a matrix is written, the same split file must be in effect when that matrix is read by any procedure.

Missing Values

Missing-value treatment affects the values written to a matrix data file. When reading a matrix data file, be sure to specify a missing-value treatment on CLUSTER that is compatible with the treatment that was in effect when the matrix materials were generated.

Example: Output to External File

```
DATA LIST FILE=ALMANAC1 RECORDS=3
  /1 CITY 6-18(A) POP80 53-60
  /2 CHURCHES 10-13 PARKS 14-17 PHONES 18-25 TVS 26-32
      RADIOST 33-35 TVST 36-38 TAXRATE 52-57(2).
N OF CASES 8.

CLUSTER CHURCHES TO TAXRATE
  /ID=CITY
  /MEASURE=EUCLID
  /MATRIX=OUT(CLUSMTX).
```

- CLUSTER reads raw data from file *ALMANAC1* and writes one set of matrix materials to file *CLUSMTX.*
- The active dataset is still the *ALMANAC1* file defined on DATA LIST. Subsequent commands are executed on *ALMANAC1*.

Example: Output Replacing Active Dataset

```
DATA LIST FILE=ALMANAC1 RECORDS=3
  /1 CITY 6-18(A) POP80 53-60
  /2 CHURCHES 10-13 PARKS 14-17 PHONES 18-25 TVS 26-32
      RADIOST 33-35 TVST 36-38 TAXRATE 52-57(2).
N OF CASES 8.

CLUSTER CHURCHES TO TAXRATE
  /ID=CITY
```

```
  /MEASURE=EUCLID
  /MATRIX=OUT(*).
LIST.
```

- CLUSTER writes the same matrix as in the previous example. However, the matrix data file replaces the active dataset. The LIST command is executed on the matrix file, not on *ALMANAC1*.

Example: Input from Active Dataset

```
GET FILE=CLUSMTX.
CLUSTER
  /ID=CITY
  /MATRIX=IN(*).
```

- This example starts a new session and reads an existing matrix data file. GET retrieves the matrix data file *CLUSMTX*.
- MATRIX=IN specifies an asterisk because the matrix data file is the active dataset. If MATRIX=IN(CLUSMTX) is specified, the program issues an error message.
- If the GET command is omitted, the program issues an error message.

Example: Input from External File

```
GET FILE=PRSNNL.
FREQUENCIES VARIABLE=AGE.

CLUSTER
  /ID=CITY
  /MATRIX=IN(CLUSMTX).
```

- This example performs a frequencies analysis on the file *PRSNNL* and then uses a different file for CLUSTER. The file is an existing matrix data file.
- The variable list is omitted on the CLUSTER command. By default, all cases in the matrix file are used in the analysis.
- MATRIX=IN specifies the matrix data file *CLUSMTX*.
- *CLUSMTX* does not replace *PRSNNL* as the active dataset.

Example: Input from Active Dataset

```
GET FILE='data/crime.sav.
PROXIMITIES MURDER TO MOTOR
  /VIEW=VARIABLE
  /MEASURE=PH2
  /MATRIX=OUT(*).
CLUSTER
  /MATRIX=IN(*).
```

- PROXIMITIES uses the data from *crime.sav*, which is now the active dataset. The VIEW subcommand specifies computation of proximity values between variables. The MATRIX subcommand writes the matrix to the active dataset.
- MATRIX=IN(*) on the CLUSTER command reads the matrix materials from the active dataset. Since the matrix contains distances between variables, CLUSTER clusters variables based on distance measures in the input. The variable list is omitted on the CLUSTER command, so all variables are used in the analysis. The slash preceding the MATRIX subcommand is required because there is an implied variable list. Without the slash, CLUSTER would attempt to interpret MATRIX as a variable name rather than a subcommand name.

CODEBOOK

CODEBOOK is available in Statistics Base Edition.

Note: Square brackets used in the CODEBOOK syntax chart are required parts of the syntax and are not used to indicate optional elements. Equals signs (=) used in the syntax chart are required elements. All subcommands are optional.

```
CODEBOOK variable [level] variable [level] variable [level]...

  /VARINFO
    POSITION LABEL TYPE FORMAT MEASURE ROLE ATTRIBUTES VALUELABELS MISSING
    RESERVEDATTRIBUTES

  /FILEINFO NAME LOCATION CASECOUNT LABEL ATTRIBUTES DOCUMENTS WEIGHT
    RESERVEDATTRIBUTES

  /STATISTICS NONE COUNT PERCENT MEAN STDDEV QUARTILES

  /OPTIONS MAXCATS=200*
           VARORDER={FILE**         }
                    {ALPHA          }
                    {VARLIST†       }
                    {MEASURE        }
                    {ATTRIBUTE(name)}
               SORT={ASCENDING** }
                    {DESCENDING }
```

*Default if subcommand or keyword omitted.

**Default if subcommand omitted and there is no variable list.

†Default if subcommand omitted and the command includes a variable list.

- If the VARINFO subcommand is omitted, all variable information except RESERVEDATTRIBUTES is included.
- If the STATISTICS subcommand is omitted, all statistics are included.

Release History

Release 17.0

- Command introduced.

Release 18

- ROLE keyword added to VARINFO subcommand.

Example

```
CODEBOOK Age Income $MultCars.
```

Overview

CODEBOOK reports the dictionary information -- such as variable names, variable labels, value labels, missing values -- and summary statistics for all or specified variables and multiple response sets in the active dataset. For nominal and ordinal variables and multiple response sets, summary statistics include counts and percents. For scale variables, summary statistics include mean, standard deviation, and quartiles.

Options

Optionally, you can:

- Specify the variables and/or multiple response sets to include in the report.
- Choose the types of dictionary information to display.
- Suppress the display of summary statistics for any nominal and ordinal variables and multiple response set with more than a specified number of unique values.

- Override the defined measurement level for a variable, thereby changing the available summary statistics for the variable.
- Include a table of file information, such as file name and location and number of cases.
- Sort the variables in the report by variable name or label or other dictionary attributes such as type, format, or measurement level.

Basic Specification

The basic specification is the command name CODEBOOK with no additional specifications.

Subcommand Order

The command name is followed by the optional variable list, followed by the optional subcommands in any order.

Syntax Rules

- Variables and multiple response sets listed on the optional variable list must already exist in the active dataset.
- Each variable can only be specified or implied once.
- Multiple response set names must include the leading dollar sign ($).
- The keyword TO can be used to specify consecutive variables in file order. It cannot be used to specify a list of multiple response sets.
- Each subcommand can only be specified once.
- Equals signs and square brackets shown in examples are required syntax elements.

Operations

- By default, CODEBOOK reads the active dataset and causes the execution of any pending transformations.
- If /STATISTICS NONE is specified, CODEBOOK does not read the active dataset or execute pending transformations.
- SPLIT FILE status is ignored. This includes split-file groups created by the MULTIPLE IMPUTATION command (available in SPSS Statistics Professional Edition or the Missing Values option).
- FILTER status is honored for computing summary statistics.

Examples

CODEBOOK with No Additional Specifications

```
CODEBOOK.
```

The default output includes:

- Variable information for all variables in the dataset, except for reserved system attributes.
- Counts and percents for all categories of nominal and ordinal variables, labeled categories of scale variables, and multiple response sets.
- Mean, standard deviation, and quartiles for scale variables.

Specifying Specific Variables, Variable Information, and Statistics

```
CODEBOOK Var1 Var3 [N] $Multvar
  /VARINFO LABEL MEASURE VALUELABELS MISSING
  /STATISTICS COUNT MEAN
  /OPTIONS MAXCATS=10.
```

- The results will only include information for the two specified variables and the one multiple response set.
- Var3 [N] indicates that *Var3* should be treated as nominal for summary statistics. This has no effect on the defined measurement level for the variable or the measurement level displayed in the results.

- Dictionary information will be limited to variable label, measurement level, value labels, and missing values.
- Only counts will be included for nominal/ordinal variables, multiple response sets and labeled categories of scale variables.
- Only the mean will be included for scale variables.
- For nominal/ordinal variables, multiple response sets, and labeled values of scale variables, MAXCATS=10 will suppress the display of value labels and counts if there are more than 10 unique, valid values.

Variable List

The optional variable list specification allows you to limit the results to specified variables and/or multiple response sets and override the defined measurement level for specified numeric variables.

- Each variable or multiple response set can be specified or implied only once.
- Multiple response set names must include the leading dollar sign ($).
- Keyword TO can be used to specify consecutive variables in file order. It cannot be used to specify a list of multiple response sets.
- Keyword ALL can be used to specify all variables (does not include multiple response sets). Note: ALL cannot be used in combination with a list of one or more specific variable names since a variable can be specified only once.
- Keyword $ALL can be used to specify all multiple response sets (does not include variables). Note: $ALL cannot be used in combination with a list of one or more specific multiple response sets, since a set can be specified only once.

Overriding Defined Measurement Level

Available summary statistics are determined by the measurement level of the variable. You can override the defined measurement level by including a measurement level specification in square brackets after the variable name: [N] for nominal, [O] for ordinal, and [S] for scale. This does not change the defined measurement level for the variable, and if results include measurement level, the defined measurement level is displayed in the results.

- For string variables and multiple response sets, measurement level can only be nominal or ordinal.
- Measurement level specification cannot be used with keywords ALL or $ALL.

Example

```
CODEBOOK Var1 Var3 [N] Var5 TO Var8 [N] $Multvar.
```

- *Var3* and *Var8* will be treated as nominal for summary statistics, so the available summary statistics for those two variables are counts and percents.
- The defined measurement level will be used to determine available summary statistics for all other variables, including all the variables preceding *Var8* in the set of variables defined by Var5 TO Var8.

VARINFO Subcommand

The optional VARINFO subcommand allows you to control the variable information included in the results.

- By default, all available variable information, with the exception of reserved system attributes, is included.
- If you include the VARINFO subcommand, it should be followed by one or more of the available keywords that indicate the dictionary information to include.

The available options are:

POSITION. *File position.* An integer that represents the position of the variable in file order. This is not available for multiple response sets.

LABEL. *Defined variable label.* See the topic "VARIABLE LABELS" on page 2125 for more information.

TYPE. *Fundamental data type.* This is either *Numeric*, *String*, or *Multiple Response Set*.

FORMAT. *Print format.* The display format for the variable, such as *A4*, *F8.2*, or *DATE11*. See the topic "Variable Types and Formats" on page 46 for more information. This is not available for multiple response sets.

MEASURE. *Measurement level.* The possible values are *Nominal*, *Ordinal*, *Scale*, and *Unknown*. The value displayed is the measurement level stored in the dictionary and is not affected by any temporary measurement level override specified by the CODEBOOK command. See the topic "VARIABLE LEVEL " on page 2127 for more information. This is not available for multiple response sets. (Note: The measurement level for numeric variables may be "unknown" prior to the first data pass when the measurement level has not been explicitly set, such as data read from an external source or newly created variables. The measurement level for string variables is always known.)

ROLE. *Role when using predefined roles in dialogs.* Some dialogs support predefined roles that can be used to pre-select variables for analysis. See the topic "Overview " on page 2129 for more information.

ATTRIBUTES. *User-defined custom variable attributes.* Output includes both the names and values for any custom variable attributes associated with each variable. See the topic "VARIABLE ATTRIBUTE" on page 2121 for more information. This is not available for multiple response sets.

VALUELABELS. *Defined value labels.* If the STATISTICS subcommand includes COUNT or PERCENT, defined value labels are included in the output even if this keyword is omitted from the VARINFO subcommand. For information on defining value labels, see "VALUE LABELS " on page 2109.

MISSING. *User-defined missing values.* If the STATISTICS subcommand includes COUNT or PERCENT, defined value labels are included in the output even if this keyword is omitted from the VARINFO subcommand. For information on defining missing values, see "MISSING VALUES " on page 1129. This is not available for multiple response sets.

RESERVEDATTRIBUTES. *Reserved system variable attributes.* You can display system attributes, but you should not alter them. System attribute names start with a dollar sign ($) . Non-display attributes, with names that begin with either "@" or "$@", are not included. Output includes both the names and values for any system attributes associated with each variable. This is not available for multiple response sets.

Example

```
CODEBOOK
  /VARINFO LABEL MEASURE VALUELABELS MISSING.
```

FILEINFO Subcommand

The optional FILEINFO subcommand allows you to control the file information included in the results.

- By default, no file information is included.
- If you include the FILEINFO subcommand, it should be followed by one or more of the available keywords that indicate the file information to include.

The available options are:

NAME. *Name of the IBM SPSS Statistics data file.* If the dataset has never been saved in IBM SPSS Statistics format, then there is no data file name.

LOCATION. *Directory (folder) location of the IBM SPSS Statistics data file.* If the dataset has never been saved in IBM SPSS Statistics format, then there is no location.

CASECOUNT. *Number of cases in the active dataset.* This is the total number of cases, including any cases that may be excluded from summary statistics due to filter conditions.

LABEL. *File label.* See the topic "FILE LABEL" on page 683 for more information.

ATTRIBUTES. *User-defined custom data file attributes.* See the topic "DATAFILE ATTRIBUTE " on page 531 for more information.

DOCUMENTS. *Data file document text.* Document text created with the DOCUMENT or ADD DOCUMENT commands. See the topic "ADD DOCUMENT" on page 103 for more information.

WEIGHT. *Weight status* If weighting is on, the name of the weight variable is displayed.

RESERVEDATTRIBUTES. *Reserved system data file attributes.* You can display system attributes, but you should not alter them. System attribute names start with a dollar sign ($) . Non-display attributes, with names that begin with either "@" or "$@", are not included. Output includes both the names and values for any system data file attributes.

Example

```
CODEBOOK
  /FILEINFO NAME LOCATION CASECOUNT.
```

- The file information table will only include the name and location of the IBM SPSS Statistics data file and the total number of cases in the file.
- The output will also include a table of variable information and summary statistics for each variable and multiple response set, because all variable information is included by default.

STATISTICS Subcommand

The optional STATISTICS subcommand allows you to control the summary statistics that are included in the output, or suppress the display of summary statistics entirely.

- By default, all summary statistics are included.
- If you include the STATISTICS subcommand, it should be followed by one or more of the available keywords.

The available options are:

COUNT. *Number of cases in each category.* This applies to nominal and ordinal variables, multiple response sets, and labeled values of scale variables.

PERCENT. *Percent of cases in each category.* This applies to nominal and ordinal variables, multiple response sets, and labeled values of scale variables. The denominator is the total number of cases, including cases with missing values for the variable. If filtering is in effect, it is the total number of unfiltered cases. For multiple response sets, percentages can sum to more than 100%.

MEAN. *Mean.* This applies to scale variables only.

STDDEV. *Standard deviation.* This applies to scale variables only.

QUARTILES. *25th, 50th (median), and 75th percentiles.* This applies to scale variables only.

NONE. *Do not include any summary statistics.* If specified, this can be the only keyword included on the STATISTICS subcommand.

Example

```
CODEBOOK
    /STATISTICS COUNT MEAN.
```

OPTIONS Subcommand

The OPTIONS subcommand allows you to suppress the display of value labels, counts, and percents for variables with more than a specified number of values or value labels and control the order in which variables are displayed in the output.

- If the CODEBOOK command does not include a variable list, the default display order is ascending file order.
- If the CODEBOOK command includes a variable list, the default display order is the order in which the variables are listed on the command.

MAXCATS Keyword

MAXCATS=N. Suppress counts and percents for variables with more than the specified number of valid values. The default is 200. This applies to nominal and ordinal variables, multiple response sets, and labeled values of scale variables. For scale variables, the number of categories is the number of labeled values.

VARORDER Keyword

The optional VARORDER keyword is followed by an equals sign (=) and one of the following alternatives:

FILE. File order.

ALPHA. Alphabetic order by variable name.

VARLIST. Order in which variables and multiple response sets are listed on the command. If there is no variable list, this setting is ignored, and the default file order is used.

MEASURE. Sort by measurement level. This creates four sorting groups: nominal, ordinal, scale, and unknown. (Note: The measurement level for numeric variables may be "unknown" prior to the first data pass when the measurement level has not been explicitly set, such as data read from an external source or newly created variables. The measurement level for string variables is always known.)

ATTRIBUTE (name). Alphabetic order by user-defined custom attribute name and value. In ascending order, variables that don't have the attribute sort to the top, followed by variables that have the attribute but no defined value for the attribute, followed by variables with defined values for the attribute in alphabetic order of the values. See the topic “VARIABLE ATTRIBUTE” on page 2121 for more information.

SORT Keyword

The optional SORT keyword is followed by an equals sign (=) and one of the following alternatives:

ASCENDING. Ascending order.

DESCENDING. Descending order.

Example

```
CODEBOOK
    /OPTIONS MAXCATS=50 VARORDER=ALPHA SORT=DESCENDING.
```

COMMENT

```
{COMMENT} text
{   *   }
```

Overview

COMMENT inserts explanatory text within the command sequence. Comments are included among the commands printed back in the output; they do not become part of the information saved in a data file. To include commentary in the dictionary of a data file, use the DOCUMENT command.

Syntax Rules

- The first line of a comment can begin with the keyword COMMENT or with an asterisk (*). Comment text can extend for multiple lines and can contain any characters.
- Use /* and */ to set off a comment within a command. The comment can be placed wherever a blank is valid (except within strings) and should be preceded by a blank. Comments within a command cannot be continued onto the next line.
- The closing */ is optional when the comment is at the end of the line. The command can continue onto the next line just as if the inserted comment was a blank.
- Comments cannot be inserted within data lines.
- A comment on a separate line by itself within a command will cause an error. The comment line will be interpreted as a blank line, which is interpreted as a command terminator.

Examples

Comment As a Separate Command

```
* Create a new variable as a combination of two old variables;
  the new variable is a scratch variable used later in the
  session;  it will not be saved with the data file.

COMPUTE #XYVAR=0.
IF (XVAR EQ 1 AND YVAR EQ 1) #XYVAR=1.
```

The three-line comment will be included in the display file but will not be part of the data file if the active dataset is saved.

Comments within Commands

```
IF (RACE EQ 1 AND GENDER EQ 1) GENDERRACE = 1.  /*White males.
```

The comment is entered on a command line. The closing */ is not needed because the comment is at the end of the line.

Comment on Separate Line within a Command

```
FREQUENCIES VARIABLES=Var1 to Var5
  /*this will cause an error*/
  /FORMAT=NOTABLE
  /BARCHART.
```

A comment on a separate line within a command will cause an error. The comment is interpreted as a blank line, and a blank line is interpreted as a command terminator. So /FORMAT=NOTABLE will be intrepreted as the start of a different command, resulting in an error.

COMPARE DATASETS

```
COMPARE DATASETS
  /COMPDATASET {'savfile' | dataset} [PASSWORD='password']

  /VARIABLES {varlist | ALL}

  [/CASEID varlist]

  [/SAVE]
    [FLAGMISMATCHES={YES**} [VARNAME={CasesCompare**}]]
                    {NO   }          {varname       }
    [MATCHDATASET={NO**} [MATCHNAME={dataset name                        }]]
                  {YES }            {'savfile' [MATCHPASS={'password'}]}
                                                {NONE**    }
    [MISMATCHDATASET={NO**} [MISMATCHNAME={dataset name                           }]]
                     {YES } {'savfile' [MISMATCHPASS={'password'}]}
                                                    {NONE**    }
    [ENCRYPTEDPW={NO**}]
                 {YES }

  [/OUTPUT]
    [VARPROPERTIES={NONE**}]
                   {ALL   }
                   {MEASURE LABEL VALUELABELS COLUMNS
                    MISSING ALIGN ROLE ATTRIBUTES WIDTH}
    [CASETABLE={YES**} [TABLELIMIT={100**}]]
               {NO   }             {value}
                                   {NONE}

  **Default if subcommand or keyword omitted
```

- The COMPDATASET and VARIABLES subcommand are required. All other subcommands are optional.
- All subcommands must be spelled out in full. Abbreviation is not allowed.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 21

- Command introduced.

Release 22.0

- PASSWORD keyword introduced on the COMPDATASET subcommand.
- MATCHPASS, MISMATCHPASS, and ENCRYPTEDPW keywords introduced on the SAVE subcommand.

Example

```
COMPARE DATASETS
  /COMPDATASET '/myfiles/datafile.sav'
  /VARIABLES ALL
  /CASEID Customer_Number
  /SAVE FLAGMISMATCHES=YES VARNAME=FlagVar
  /OUTPUT VARPROPERTIES=MEASURE MISSING.
```

Overview

COMPARE DATASETS compares the active dataset to another dataset in the current session or an external file in IBM SPSS Statistics format. Features include:

- Comparison of data values for the specified variables
- Comparison of selected variable attributes, such as measurement level, user-missing values, and value labels
- Summary output tables that describe the file differences
- Creation of new datasets that contain only matched cases or only mismatched cases

Syntax

- The COMPDATASET and VARIABLES subcommand are required. All other subcommands are optional.

- All subcommands must be spelled out in full. Abbreviation is not allowed.

Operations

- Split file and filter status are ignored.
- This command only compares IBM SPSS Statistics data files or datasets already open in the current session.

COMPDATASET subcommand

The required COMPDATASET subcommand specifies the open dataset or external IBM SPSS Statistics data file that will be compared to the active dataset. The subcommand name is followed by the name of a dataset in the current session or an external IBM SPSS Statistics data file. External file specifications must be enclosed in quotes.

PASSWORD Keyword

The PASSWORD keyword specifies the password required to open an encrypted IBM SPSS Statistics data file. The specified value must be enclosed in quotation marks and can be provided as encrypted or as plain text. Encrypted passwords are created when pasting command syntax from the Save Data As dialog. The PASSWORD keyword is ignored if the file is not encrypted.

Example

```
/COMPDATASET '/myfiles/datafile.sav'
```

VARIABLES subcommand

The required VARIABLES subcommand specifies the variables to be compared. The subcommand name is followed by a list of variables or the keyword ALL.

Example

```
/VARIABLES name address1 address2 address3 ssn
```

CASEID subcommand

The optional CASEID subcommand specifies one or more variables that identify each case. The subcommand name is followed by a list of variables.

- If you specify multiple variables, each unique combination of values identifies a case.
- Both files must be sorted in ascending order of the case ID variables.
- If you do not include the CASEID subcommand, cases are compared in file order. That is, the first case (row) in the active dataset is compared to the first case in the other dataset, and so on.

Example

```
/CASEID Account_Number
```

SAVE subcommand

You can use the optional SAVE subcommand to create a new variable in the active dataset that identifies mismatches and create new datasets that contain only cases that match in both files or only cases that have differences.

FLAGMISMATCHES=YES|NO. Creates a new variable in the active dataset that indicates if the corresponding case in the other dataset contains any values that differ from the values for that case in the active dataset. The default is YES.

- The value of the new variable is 1 if there are differences and 0 if all the values are the same. If there cases in the active dataset that are not present in the other dataset, the value is -1.
- The default name of the new variable is *CasesCompare*. Use the optional VARNAME keyword to specify a different name. The name must conform to variable naming rules. See the topic "Variable Names" on page 42 for more information.

MATCHDATASET=NO|YES. Creates a new dataset or external data file that contains only cases from the active dataset that have exact matches in the other dataset. The default is NO.

- Use the MATCHNAME keyword to specify a dataset name or an external file. External file specifications must be enclosed in quotes. If it is a dataset in the current session, it must be an existing or previously declared dataset.
- If a dataset or external file with the specified name already exists, it will be overwritten.
- Use the MATCHPASS keyword if you are creating an external data file and you want to save it as an encrypted file. The specified value is the password that is required to open the file and it must be enclosed in quotation marks. Passwords are limited to 10 characters and are case-sensitive. The keyword NONE is the default and it specifies that the file is not encrypted.

MISMATCHDATASET=NO|YES. Creates a new dataset or external data file that contains only cases from the active dataset that do not have exact matches in the other dataset. The default is NO.

- Use the MISMATCHNAME keyword to specify a dataset name or an external file. External file specifications must be enclosed in quotes. If it is a dataset in the current session, it must be an existing or previously declared dataset.
- If a dataset or external file with the specified name already exists, it will be overwritten.
- Use the MISMATCHPASS keyword if you are creating an external data file and you want to save it as an encrypted file. The specified value is the password that is required to open the file and it must be enclosed in quotation marks. Passwords are limited to 10 characters and are case-sensitive. The keyword NONE is the default and it specifies that the file is not encrypted.

ENCRYPTEDPW Keyword

The ENCRYPTEDPW keyword specifies whether the password is encrypted and applies to both the MATCHPASS and MISMATCHPASS keywords.

NO. The password is not encrypted. It is treated as plain text. This setting is the default.

YES. The password is encrypted. Use ENCRYPTEDPW=YES only when the password is known to be encrypted. For reference, passwords are always encrypted in syntax that is pasted from the Save Data As dialog.

Note:

- Passwords cannot be recovered if they are lost. If the password is lost, then an encrypted file cannot be opened.
- Encrypted files cannot be opened in versions of IBM SPSS Statistics prior to version 21.

Creating strong passwords

- Use eight or more characters.
- Include numbers, symbols and even punctuation in your password.
- Avoid sequences of numbers or characters, such as "123" and "abc", and avoid repetition, such as "111aaa".
- Do not create passwords that use personal information such as birthdays or nicknames.
- Periodically change the password.

For information on declaring a new dataset before specifying it on the COMPARE DATASETS command, see “DATASET DECLARE” on page 539.

Example

```
/SAVE
  FLAGMISMATCHES=YES VARNAME=Mismatch
  MATCHDATASET=YES MATCHNAME=Matches
  MISMATCHDATASET=YES MISMATCHNAME='/myfiles/'mismatches.sav'.
```

OUTPUT subcommand

You can use the optional OUTPUT command to produce a table that compares dictionary information between the two files and control the display of the case-by-case comparison table.

VARPROPERTIES=NONE | ALL | MEASURE LABEL VALUELABELS COLUMNS MISSING ALIGN ROLE ATTRIBUTES WIDTH. Produces a table that compares the specified data dictionary properties for each variable in the two datasets. The default is NONE.

- **MEASURE.** Measurement level. See the topic “VARIABLE LEVEL ” on page 2127 for more information.
- **LABEL.** Descriptive variable label. See the topic “VARIABLE LABELS” on page 2125 for more information.
- **VALUELABELS.** Descriptive value labels. See the topic “VALUE LABELS ” on page 2109 for more information.
- **COLUMNS.** Column width in Data view of the Data Editor. See the topic “VARIABLE WIDTH ” on page 2131 for more information.
- **MISSING.** Defined user-missing values. See the topic “MISSING VALUES ” on page 1129 for more information.
- **ALIGN.** Alignment in Data view of the Data Editor. See the topic “VARIABLE ALIGNMENT ” on page 2119 for more information.
- **ROLE.** Variable role. See the topic “VARIABLE ROLE” on page 2129 for more information.
- **ATTRIBUTES.** User-defined custom variable attributes. See the topic “VARIABLE ATTRIBUTE” on page 2121 for more information.
- **WIDTH.** For numeric variables, the maximum number of characters displayed (digits plus formatting characters, such as currency symbols, grouping symbols, and decimal indicator). For string variables, the maximum number of bytes allowed.

CASETABLE=YES|NO. Produces a case-by-case comparison table that contains mismatch details. For each case and each variable, the table displays the values that are different in the two files. The default is YES. Use the optional TABLELIMIT keyword to limit the table to the first *n* cases with mismatches. The default is 100. TABLELIMIT=NONE will display all mismatches.

Example

```
/OUTPUT
  VARPROPERTIES=MEASURE WIDTH
  CASETABLE=YES TABLELIMIT=500
```

COMPUTE

```
COMPUTE target variable=expression
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

Example

```
COMPUTE newvar1=var1+var2.
COMPUTE newvar2=RND(MEAN(var1 to var4).
COMPUTE logicalVar=(var1>5).
STRING newString (A10).
COMPUTE newString=CONCAT((RTRIM(stringVar1), stringVar2).
```

Functions and operators available for COMPUTE are described in .

Overview

COMPUTE creates new numeric variables or modifies the values of existing string or numeric variables. The variable named on the left of the equals sign is the **target variable**. The variables, constants, and functions on the right side of the equals sign form an **assignment expression**. For a complete discussion of functions, see .

Numeric Transformations

Numeric variables can be created or modified with COMPUTE. The assignment expression for numeric transformations can include combinations of constants, variables, numeric operators, and functions.

String transformations

String variables can be modified but cannot be created with COMPUTE. However, a new string variable can be declared and assigned a width with the STRING command and then assigned values by COMPUTE. The assignment expression can include string constants, string variables, and any of the string functions. All other functions are available for numeric transformations only.

Basic specification

The basic specification is a target variable, an equals sign (required), and an assignment expression.

Syntax rules

- The target variable must be named first, and the equals sign is required. Only one target variable is allowed per COMPUTE command.
- If the target variable is numeric, the expression must yield a numeric value; if the target variable is a string, the expression must yield a string value.
- Each function must specify at least one argument enclosed in parentheses. If a function has two or more arguments, the arguments must be separated by commas. For a complete discussion of functions and their arguments, see .
- You can use the TO keyword to refer to a set of variables where the argument is a list of variables.

Numeric variables

- Parentheses are used to indicate the order of execution and to set off the arguments to a function.
- Numeric functions use simple or complex expressions as arguments. Expressions must be enclosed in parentheses.

String variables

- String values and constants must be enclosed in single or double quotes.
- When strings of different lengths are compared using the ANY or RANGE functions, the shorter string is right-padded with blanks so that its length equals that of the longer string.

Operations

- If the target variable already exists, its values are replaced.
- If the target variable does not exist and the assignment expression is numeric, the program creates a new variable.
- If the target variable does not exist and the assignment expression is a string, the program displays an error message and does not execute the command. Use the "STRING " on page 1907 command to declare new string variables before using them as target variables.

Numeric variables

- New numeric variables created with COMPUTE are assigned a dictionary format of F8.2 and are initialized to the system-missing value for each case (unless the LEAVE command is used). Existing numeric variables transformed with COMPUTE retain their original dictionary formats. The format of a numeric variable can be changed with the FORMATS command.
- All expressions are evaluated in the following order: first functions, then exponentiation, and then arithmetic operations. The order of operations can be changed with parentheses.
- COMPUTE returns the system-missing value when it doesn't have enough information to evaluate a function properly. Arithmetic functions that take only one argument cannot be evaluated if that argument is missing. The date and time functions cannot be evaluated if any argument is missing. Statistical functions are evaluated if a sufficient number of arguments is valid. For example, in the command

  ```
  COMPUTE FACTOR = SCORE1 + SCORE2 + SCORE3.
  ```

  FACTOR is assigned the system-missing value for a case if any of the three score values is missing. It is assigned a valid value only when all score values are valid. In the command

  ```
  COMPUTE FACTOR = SUM(SCORE1 TO SCORE3).
  ```

  FACTOR is assigned a valid value if at least one score value is valid. It is system-missing only when all three score values are missing. See "Missing values in numeric expressions" on page 90 for information on how to control the minimum number of non-missing arguments required to return a non-missing result.

String variables

- String variables can be modified but not created on COMPUTE. However, a new string variable can be created and assigned a width with the STRING command and then assigned new values with COMPUTE.
- Existing string variables transformed with COMPUTE retain their original dictionary formats. String variables declared on STRING and transformed with COMPUTE retain the formats assigned to them on STRING.
- The format of string variables cannot be changed with FORMATS. Instead, use STRING to create a new variable with the desired width and then use COMPUTE to set the values of the new string equal to the values of the original.
- The string returned by a string expression does not have to be the same width as the target variable. If the target variable is shorter, the result is right-trimmed. If the target variable is longer, the result is right-padded. The program displays no warning messages when trimming or padding.
- To control the width of strings, use the functions that are available for padding (LPAD, RPAD), trimming (LTRIM, RTRIM), and selecting a portion of strings (SUBSTR).

- To determine whether a character in a string is single-byte or double-byte, use the MBLEN.BYTE function. Specify the string and, optionally, its beginning byte position. If the position is not specified, it defaults to 1.

See the topic "String functions" on page 78 for more information.

Examples

A number of examples are provided to illustrate the use of COMPUTE. For a complete list of available functions and detailed function descriptions, see "Transformation Expressions" on page 58.

Arithmetic operations

```
COMPUTE V1=25-V2.
COMPUTE V3=(V2/V4)*100.

DO IF Tenure GT 5.
COMPUTE Raise=Salary*.12.
ELSE IF Tenure GT 1.
COMPUTE Raise=Salary*.1.
ELSE.
COMPUTE Raise=0.
END IF.
```

- *V1* is 25 minus *V2* for all cases. *V3* is *V2* expressed as a percentage of *V4*.
- *Raise* is 12% of *Salary* if *Tenure* is greater than 5. For remaining cases, *Raise* is 10% of *Salary* if *Tenure* is greater than 1. For all other cases, *Raise* is 0.

Arithmetic functions

```
COMPUTE WtChange=ABS(Weight1-Weight2).
COMPUTE NewVar=RND((V1/V2)*100).
COMPUTE Income=TRUNC(Income).
COMPUTE MinSqrt=SQRT(MIN(V1,V2,V3,V4)).

COMPUTE Test = TRUNC(SQRT(X/Y)) * .5.
COMPUTE Parens = TRUNC(SQRT(X/Y) * .5).
```

- *WtChange* is the absolute value of *Weight1* minus *Weight2*.
- *NewVar* is the percentage *V1* is of *V2*, rounded to an integer.
- *Income* is truncated to an integer.
- *MinSqrt* is the square root of the minimum value of the four variables *V1* to *V4*. MIN determines the minimum value of the four variables, and SQRT computes the square root.
- The last two examples above illustrate the use of parentheses to control the order of execution. For a case with value 2 for *X* and *Y*, *Test* equals 0.5, since 2 divided by 2 (*X*/*Y*) is 1, the square root of 1 is 1, truncating 1 returns 1, and 1 times 0.5 is 0.5. However, *Parens* equals 0 for the same case, since SQRT(*X*/*Y*) is 1, 1 times 0.5 is 0.5, and truncating 0.5 returns 0.

Statistical functions

```
COMPUTE NewSalary = SUM(Salary,Raise).
COMPUTE MinValue = MIN(V1,V2,V3,V4).
COMPUTE MeanValue = MEAN(V1,V2,V3,V4).
COMPUTE NewMean = MEAN.3(V1,V2,V3,V4).
```

- *NewSalary* is the sum of *Salary* plus *Raise*.
- *MinValue* is the minimum of the values for *V1* to *V4*.
- *MeanValue* is the mean of the values for *V1* to *V4*. Since the mean can be computed for one, two, three, or four values, *MeanValue* is assigned a valid value as long as any one of the four variables has a valid value for that case.
- In the last example above, the .3 suffix specifies the minimum number of valid arguments required. *NewMean* is the mean of variables *V1* to *V4 only* if at least three of these variables have valid values. Otherwise, *NewMean* is system-missing for that case.

Missing-Value functions

```
MISSING VALUE V1 V2 V3 (0).
COMPUTE AllValid=V1 + V2 + V3.
COMPUTE UM=VALUE(V1) + VALUE(V2) + VALUE(V3).
COMPUTE SM=SYSMIS(V1) + SYSMIS(V2) + SYSMIS(V3).
COMPUTE M=MISSING(V1) + MISSING(V2) + MISSING(V3).
```

- The MISSING VALUE command declares the value 0 as missing for *V1*, *V2*, and *V3*.
- *AllValid* is the sum of three variables only for cases with valid values for all three variables. *AllValid* is assigned the system-missing value for a case if any variable in the assignment expression has a system- or user-missing value.
- The VALUE function overrides user-missing value status. Thus, *UM* is the sum of *V1*, *V2*, and *V3* for each case, including cases with the value 0 (the user-missing value) for any of the three variables. Cases with the system-missing value for *V1*, *V2*, and *V3* are system-missing.
- The SYSMIS function on the third COMPUTE returns the value 1 if the variable is system-missing. Thus, *SM* ranges from 0 to 3 for each case, depending on whether the variables *V1*, *V2*, and *V3* are system-missing for that case.
- The MISSING function on the fourth COMPUTE returns the value 1 if the variable named is system- or user-missing. Thus, *M* ranges from 0 to 3 for each case, depending on whether the variables *V1*, *V2*, and *V3* are user- or system-missing for that case.
- Alternatively, you could use the COUNT command to create the variables *SM* and *M*.

```
* Test for listwise deletion of missing values.

DATA LIST /V1 TO V6 1-6.
BEGIN DATA
213 56
123457
123457
9234 6
END DATA.
MISSING VALUES V1 TO V6(6,9).

COMPUTE NotValid=NMISS(V1 TO V6).
FREQUENCIES VAR=NotValid.
```

- COMPUTE determines the number of missing values for each case. For each case without missing values, the value of *NotValid* is 0. For each case with one missing value, the value of *NotValid* is 1, and so on. Both system- and user-missing values are counted.
- FREQUENCIES generates a frequency table for *NotValid*. The table gives a count of how many cases have all valid values, how many cases have one missing value, how many cases have two missing values, and so on, for variables *V1* to *V6*. This table can be used to determine how many cases would be dropped in an analysis that uses listwise deletion of missing values. For other ways to check listwise deletion, see the examples for the ELSE command (in the DO IF command) and those for the IF command.

See the topic "Missing value functions" on page 92 for more information.

String functions

```
DATA LIST FREE / FullName (A20).
BEGIN DATA
"Fred Smith"
END DATA.
STRING FirstName LastName LastFirstName (A20).
COMPUTE #spaceLoc=INDEX(FullName, " ").
COMPUTE FirstName=SUBSTR(FullName, 1, (#spaceLoc-1)).
COMPUTE LastName=SUBSTR(FullName, (#spaceLoc+1)).
COMPUTE LastFirstName=CONCAT(RTRIM(LastName), ", ", FirstName).
COMPUTE LastFirstName=REPLACE(LastFirstName, "Fred", "Ted").
```

- The INDEX function returns a number that represents the location of the first blank space in the value of the string variable *FullName*.
- The first SUBSTR function sets *FirstName* to the portion of *FullName* prior to the first space in the value. So, in this example, the value of *FirstName* is "Fred".
- The second SUBSTR function sets *LastName* to the portion of *FullName* after the first blank space in the value. So, in this example, the value of *LastName* is "Smith".

- The CONCAT function combines the values of *LastName* and *FirstName*, with a comma and a space between the two values. So, in this example, the value of *LastFirstName* is "Smith, Fred". Since all string values are right-padded with blank spaces to the defined width of the string variable, the RTRIM function is needed to remove all the extra blank spaces from *LastName*.
- The REPLACE function changes any instances of the string "Fred" in *LastFirstName* to "Ted". So, in this example, the value of *LastFirstName* is changed to "Smith, Ted".

See the topic "String functions" on page 78 for more information.

Scoring functions

```
STRING SPECIES(A20).
COMPUTE SCOREPROB=ApplyModel(CREDITMOD1,'PROBABILIT').
COMPUTE SPECIES=StrApplyModel(QUESTMOD1,'PREDICT').
```

- *SCOREPROB* is the probability that the value predicted from the model specified by CREDITMOD1 is correct.
- *SPECIES* is the predicted result from the model specified by QUESTMOD1 as applied to the active dataset. The prediction is returned as a string value.

CONJOINT

CONJOINT is available in SPSS Statistics Premium Edition or the Conjoint option.

```
CONJOINT  [PLAN={*                  }]
                {'savfile'|'dataset'}

 [/DATA={*                   }]
        {'savfile'|'dataset'}

  /{SEQUENCE}=varlist
   {RANK    }
   {SCORE   }

 [/SUBJECT=variable]

 [/FACTORS=varlist['labels'] ([{DISCRETE[{MORE}]}]
                              {          {LESS} }
                              {LINEAR[{MORE}]   }
                              {       {LESS}    }
                              {IDEAL            }
                              {ANTIIDEAL        }
           [values['labels']])]

           varlist...


 [/PRINT={ALL**      } [SUMMARYONLY]]
         {ANALYSIS   }
         {SIMULATION }
         {NONE       }

 [/UTILITY=file]

 [/PLOT={[SUMMARY] [SUBJECT] [ALL]}]
        {[NONE**]                  }
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example:

```
CONJOINT PLAN='/DATA/CARPLAN.SAV'
 /FACTORS=SPEED (LINEAR MORE) WARRANTY (DISCRETE MORE)
  PRICE (LINEAR LESS) SEATS
 /SUBJECT=SUBJ /RANK=RANK1 TO RANK15 /UTILITY='UTIL.SAV'.
```

Overview

CONJOINT analyzes score or rank data from full-concept conjoint studies. A plan file that is generated by ORTHOPLAN or entered by the user describes the set of full concepts that are scored or ranked in terms of preference. A variety of continuous and discrete models is available to estimate utilities for each individual subject and for the group. Simulation estimates for concepts that are not rated can also be computed.

Options

Data Input. You can analyze data recorded as rankings of an ordered set of profiles (or cards) as the profile numbers arranged in rank order, or as preference scores of an ordered set of profiles.

Model Specification. You can specify how each factor is expected to be related to the scores or ranks.

Display Output. The output can include the analysis of the experimental data, results of simulation data, or both.

Writing an External File. An data file containing utility estimates and associated statistics for each subject can be written for use in further analyses or graphs.

Basic Specification

- The basic specification is CONJOINT, a PLAN or DATA subcommand, and a SEQUENCE, RANK, or SCORE subcommand to describe the type of data.

- CONJOINT requires two files: a plan file and a data file. If only the PLAN subcommand or the DATA subcommand—but not both—is specified, CONJOINT will read the file that is specified on the PLAN or DATA subcommand and use the active dataset as the other file.
- By default, estimates are computed by using the DISCRETE model for all variables in the plan file (except those named *STATUS_* and *CARD_*). Output includes Kendall's tau and Pearson's product-moment correlation coefficients measuring the relationship between predicted scores and actual scores. Significance levels for one-tailed tests are displayed.

Subcommand Order

- Subcommands can appear in any order.

Syntax Rules

- Multiple FACTORS subcommands are all executed. For all other subcommands, only the last occurrence is executed.

Operations

- Both the plan and data files can be external IBM SPSS Statistics data files. In this case, CONJOINT can be used before an active dataset is defined.
- The variable *STATUS_* in the plan file must equal 0 for experimental profiles, 1 for holdout profiles, and 2 for simulation profiles. Holdout profiles are judged by the subjects but are not used when CONJOINT estimates utilities. Instead, these profiles are used as a check on the validity of the estimated utilities. Simulation profiles are factor-level combinations that are not rated by the subjects but are estimated by CONJOINT based on the ratings of the experimental profiles. If there is no *STATUS_* variable, all profiles in the plan file are assumed to be experimental profiles.
- All variables in the plan file except *STATUS_* and *CARD_* are used by CONJOINT as factors.
- In addition to the estimates for each individual subject, average estimates for each split-file group that is identified in the data file are computed. The plan file cannot have a split-file structure.
- Factors are tested for orthogonality by CONJOINT. If all of the factors are not orthogonal, a matrix of Cramér's *V* statistics is displayed to describe the non-orthogonality.
- When SEQUENCE or RANK data are used, CONJOINT internally reverses the ranking scale so that the computed coefficients are positive.
- The plan file cannot be sorted or modified in any way after the data are collected, because the sequence of profiles in the plan file must match the sequence of values in the data file in a one-to-one correspondence. (CONJOINT uses the order of profiles as they appear in the plan file, not the value of *CARD_*, to determine profile order.) If RANK or SCORE is the data-recording method, the first response from the first subject in the data file is the rank or score of the first profile in the plan file. If SEQUENCE is the data-recording method, the first response from the first subject in the data file is the profile number (determined by the order of profiles in the plan file) of the most preferred profile.

Limitations

- Factors must be numeric.
- The plan file cannot contain missing values or case weights. In the active dataset, profiles with missing values on the SUBJECT variable are grouped together and averaged at the end. If any preference data (the ranks, scores, or profile numbers) are missing, that subject is skipped.
- Factors must have at least two levels. The maximum number of levels for each factor is 99. Note that ORTHOPLAN will only produce plans with factors with 9 or fewer levels for each factor.

Examples

```
CONJOINT PLAN='/DATA/CARPLAN.SAV'
 /FACTORS=SPEED (LINEAR MORE) WARRANTY (DISCRETE MORE)
  PRICE (LINEAR LESS) SEATS
 /SUBJECT=SUBJ /RANK=RANK1 TO RANK15 /UTILITY='UTIL.SAV'.
```

- The PLAN subcommand specifies the IBM SPSS Statistics data file *CARPLAN.SAV* as the plan file containing the full-concept profiles. Because there is no DATA subcommand, the active dataset is assumed to contain the subjects' rankings of these profiles.
- The FACTORS subcommand specifies the ways in which the factors are expected to be related to the rankings. For example, speed is expected to be linearly related to the rankings, so that cars with higher speeds will receive lower (more-preferred) rankings.
- The SUBJECT subcommand specifies the variable *SUBJ* in the active dataset as an identification variable. All consecutive cases with the same value on this variable are combined to estimate utilities.
- The RANK subcommand specifies that each data point is a ranking of a specific profile and identifies the variables in the active dataset that contain these rankings.
- UTILITY writes out an external data file named *UTIL.SAV* containing the utility estimates and associated statistics for each subject.

PLAN Subcommand

PLAN identifies the file containing the full-concept profiles.

- PLAN is followed by quoted file specification for an external IBM SPSS Statistics data file or currently open dataset containing the plan. An asterisk instead of a file specification indicates the active dataset.
- If the PLAN subcommand is omitted, the active dataset is assumed by default. However, you must specify at least one IBM SPSS Statistics data file or dataset on a PLAN or DATA subcommand. The active dataset cannot be specified as both the plan file and data file.
- The plan file is a specially prepared file that is generated by ORTHOPLAN or entered by the user. The plan file can contain the variables *CARD_* and *STATUS_*, and it must contain the factors of the conjoint study. The value of *CARD_* is a profile identification number. The value of *STATUS_* is 0, 1, or 2, depending on whether the profile is an experimental profile (0), a holdout profile (1), or a simulation profile (2).
- The sequence of the profiles in the plan file must match the sequence of values in the data file.
- Any simulation profiles (*STATUS_*=2) must follow experimental and holdout profiles in the plan file.
- All variables in the plan file except *CARD_* and *STATUS_* are used as factors by CONJOINT.

Example

```
DATA LIST FREE /CARD_ WARRANTY SEATS PRICE SPEED STATUS_.
BEGIN DATA
1 1 4 14000 130 2
2 1 4 14000 100 2
3 3 4 14000 130 2
4 3 4 14000 100 2
END DATA.
ADD FILES FILE='/DATA/CARPLAN.SAV'/FILE=*.
CONJOINT PLAN=* /DATA='/DATA/CARDATA.SAV'
  /FACTORS=PRICE (ANTIIDEAL) SPEED (LINEAR) WARRANTY (DISCRETE MORE)
  /SUBJECT=SUBJ /RANK=RANK1 TO RANK15 /PRINT=SIMULATION.
```

- DATA LIST defines six variables—a *CARD_* identification variable, four factors, and a *STATUS_* variable.
- The data between BEGIN DATA and END DATA are four simulation profiles. Each profile contains a *CARD_* identification number and the specific combination of factor levels of interest.
- The variable *STATUS_* is equal to 2 for all cases (profiles). CONJOINT interprets profiles with *STATUS_* equal to 2 as simulation profiles.
- The ADD FILES command joins an old plan file, *CARPLAN.SAV*, with the active dataset. Note that the active dataset is indicated last on the ADD FILES command so that the simulation profiles are appended to the end of *CARPLAN.SAV*.
- The PLAN subcommand on CONJOINT defines the new active dataset as the plan file. The DATA subcommand specifies a data file from a previous CONJOINT analysis.

DATA Subcommand

DATA identifies the file containing the subjects' preference scores or rankings.

- DATA is followed by a quoted file specification for an external IBM SPSS Statistics data file or a currently open dataset containing the data. An asterisk instead of a file specification indicates the active dataset.
- If the DATA subcommand is omitted, the active dataset is assumed by default. However, you must specify at least one IBM SPSS Statistics data file on a DATA or PLAN subcommand. The active dataset cannot be specified as both the plan file and data file.
- One variable in the data file can be a subject identification variable. All other variables are the subject responses and are equal in number to the number of experimental and holdout profiles in the plan file.
- The subject responses can be in the form of ranks assigned to an ordered sequence of profiles, scores assigned to an ordered sequence of profiles, or profile numbers in preference order from most liked to least liked.
- Tied ranks or scores are allowed. If tied ranks are present, CONJOINT issues a warning and then proceeds with the analysis. Data recorded in SEQUENCE format, however, cannot have ties, because each profile number must be unique.

Example

```
DATA LIST FREE /SUBJ RANK1 TO RANK15.
BEGIN DATA
01  3  7  6  1  2  4  9 12 15 13 14  5  8 10 11
02  7  3  4  9  6 15 10 13  5 11  1  8  4  2 12
03 12 13  5  1 14  8 11  2  7  6  3  4 15  9 10
04  3  6  7  4  2  1  9 12 15 11 14  5  8 10 13
05  9  3  4  7  6 10 15 13  5 12  1  8  4  2 11
50 12 13  8  1 14  5 11  6  7  2  3  4 15 10  9
END DATA.
SAVE OUTFILE='/DATA/RANKINGS.SAV'.
DATA LIST FREE /CARD_ WARRANTY SEATS PRICE SPEED.
BEGIN DATA
 1 1 4 14000 130
 2 1 4 14000 100
 3 3 4 14000 130
 4 3 4 14000 100
 5 5 2 10000 130
 6 1 4 10000 070
 7 3 4 10000 070
 8 5 2 10000 100
 9 1 4 07000 130
10 1 4 07000 100
11 5 2 07000 070
12 5 4 07000 070
13 1 4 07000 070
14 5 2 10000 070
15 5 2 14000 130
END DATA.
CONJOINT PLAN=* /DATA='/DATA/RANKINGS.SAV'
  /FACTORS=PRICE (ANTIIDEAL) SPEED (LINEAR)
   WARRANTY (DISCRETE MORE)
  /SUBJECT=SUBJ /RANK=RANK1 TO RANK15.
```

- The first set of DATA LIST and BEGIN–END DATA commands creates a data file containing the rankings. This file is saved in the external file *RANKINGS.SAV*.
- The second set of DATA LIST and BEGIN–END DATA commands defines the plan file as the active dataset.
- The CONJOINT command uses the active dataset as the plan file and uses *RANKINGS.SAV* as the data file.

SEQUENCE, RANK, or SCORE Subcommand

The SEQUENCE, RANK, or SCORE subcommand is specified to indicate the way in which the preference data were recorded.

SEQUENCE. *Each data point in the data file is a profile number, starting with the most-preferred profile and ending with the least-preferred profile*. This is how the data are recorded if the subject is asked to order the deck of profiles from most preferred to least preferred. The researcher records which profile number was first, which profile number was second, and so on.

RANK. *Each data point is a ranking, starting with the ranking of profile 1, then the ranking of profile 2, and so on*. This is how the data are recorded if the subject is asked to assign a rank to each profile, ranging from 1 to *n*, where *n* is the number of profiles. A lower rank implies greater preference.

SCORE. *Each data point is a preference score assigned to the profiles, starting with the score of profile 1, then the score of profile 2, and so on*. These types of data might be generated, for example, by asking subjects to use a Likert scale to assign a score to each profile or by asking subjects to assign a number from 1 to 100 to show how much they like the profile. A higher score implies greater preference.

- You must specify one, and only one, of these three subcommands.
- After each subcommand, the names of the variables containing the preference data (the profile numbers, ranks, or scores) are listed. There must be as many variable names listed as there are experimental and holdout profiles in the plan file.

Example

```
CONJOINT PLAN=* /DATA='DATA.SAV'
  /FACTORS=PRICE (ANTIIDEAL) SPEED (LINEAR) WARRANTY (DISCRETE MORE)
  /SUBJECT=SUBJ
  /RANK=RANK1 TO RANK15.
```

- The RANK subcommand indicates that the data are rankings of an ordered sequence of profiles. The first data point after *SUBJ* is variable *RANK1*, which is the ranking that is given by subject 1 to profile 1.
- There are 15 profiles in the plan file, so there must be 15 variables listed on the RANK subcommand.
- The example uses the TO keyword to refer to the 15 rank variables.

SUBJECT Subcommand

SUBJECT specifies an identification variable. All consecutive cases having the same value on this variable are combined to estimate the utilities.

- If SUBJECT is not specified, all data are assumed to come from one subject, and only a group summary is displayed.
- SUBJECT is followed by the name of a variable in the active dataset.
- If the same SUBJECT value appears later in the data file, it is treated as a different subject.

FACTORS Subcommand

FACTORS specifies the way in which each factor is expected to be related to the rankings or scores.

- If FACTORS is not specified, the DISCRETE model is assumed for all factors.
- All variables in the plan file except *CARD_* and *STATUS_* are used as factors, even if they are not specified on FACTORS.
- FACTORS is followed by a variable list and a model specification in parentheses that describes the expected relationship between scores or ranks and factor levels for that variable list.
- The model specification consists of a model name and, for the DISCRETE and LINEAR models, an optional MORE or LESS keyword to indicate the direction of the expected relationship. Values and value labels can also be specified.
- MORE and LESS keywords will *not* affect estimates of utilities. They are used simply to identify subjects whose estimates do not match the expected direction.

The four available models are as follows:

DISCRETE. *No assumption*. The factor levels are categorical, and no assumption is made about the relationship between the factor and the scores or ranks. This setting is the default. Specify keyword MORE after DISCRETE to indicate that higher levels of a factor are expected to be more preferred. Specify keyword LESS after DISCRETE to indicate that lower levels of a factor are expected to be more preferred.

LINEAR. *Linear relationship*. The scores or ranks are expected to be linearly related to the factor. Specify keyword MORE after LINEAR to indicate that higher levels of a factor are expected to be more preferred. Specify keyword LESS after LINEAR to indicate that lower levels of a factor are expected to be more preferred.

IDEAL. *Quadratic relationship, decreasing preference*. A quadratic relationship is expected between the scores or ranks and the factor. It is assumed that there is an ideal level for the factor, and distance from this ideal point, in either direction, is associated with decreasing preference. Factors that are described with this model should have at least three levels.

ANTIIDEAL. *Quadratic relationship, increasing preference*. A quadratic relationship is expected between the scores or ranks and the factor. It is assumed that there is a worst level for the factor, and distance from this point, in either direction, is associated with increasing preference. Factors that are described with this model should have at least three levels.

- The DISCRETE model is assumed for those variables that are not listed on the FACTORS subcommand.
- When a MORE or LESS keyword is used with DISCRETE or LINEAR, a reversal is noted when the expected direction does not occur.
- Both IDEAL and ANTIIDEAL create a quadratic function for the factor. The only difference is whether preference increases or decreases with distance from the point. The estimated utilities are the same for these two models. A reversal is noted when the expected model (IDEAL or ANTIIDEAL) does not occur.
- The optional value and value label lists allow you to recode data and/or replace value labels. The new values, in the order in which they appear on the value list, replace existing values, starting with the smallest existing value. If a new value is not specified for an existing value, the value remains unchanged.
- New value labels are specified in apostrophes or quotation marks. New values without new labels retain existing labels; new value labels without new values are assigned to values in the order in which they appear, starting with the smallest existing value.
- For each factor that is recoded, a table is displayed, showing the original and recoded values and the value labels.
- If the factor levels are coded in discrete categories (for example, 1, 2, 3), these values are the values used by CONJOINT in computations, even if the value labels contain the actual values (for example, 80, 100, 130). Value labels are never used in computations. You can recode the values as described above to change the coded values to the real values. Recoding does not affect DISCRETE factors but does change the coefficients of LINEAR, IDEAL, and ANTIIDEAL factors.
- In the output, variables are described in the following order:

1. All DISCRETE variables in the order in which they appear on the FACTORS subcommand.
2. All LINEAR variables in the order in which they appear on the FACTORS subcommand.
3. All IDEAL and ANTIIDEAL factors in the order in which they appear on the FACTORS subcommand.

Example

```
CONJOINT DATA='DATA.SAV'
 /FACTORS=PRICE (LINEAR LESS) SPEED (IDEAL 70 100 130)
  WARRANTY (DISCRETE MORE)
 /RANK=RANK1 TO RANK15.
```

- The FACTORS subcommand specifies the expected relationships. A linear relationship is expected between price and rankings, so that the higher the price, the lower the preference (higher ranks). A quadratic relationship is expected between speed levels and rankings, and longer warranties are expected to be associated with greater preference (lower ranks).
- The *SPEED* factor has a new value list. If the existing values were 1, 2, and 3, 70 replaces 1, 100 replaces 2, and 130 replaces 3.
- Any variable in the plan file (except *CARD_* and *STATUS_*) that is not listed on the FACTORS subcommand uses the DISCRETE model.

PRINT Subcommand

PRINT controls whether your output includes the analysis of the experimental data, the results of the simulation data, both, or none.

The following keywords are available:

ANALYSIS. *Only the results of the experimental data analysis are included*.

SIMULATION. *Only the results of the simulation data analysis are included*. The results of three simulation models—maximum utility, Bradley-Terry-Luce (BTL), and logit—are displayed.

SUMMARYONLY. *Only the summaries in the output are included, not the individual subjects*. Thus, if you have a large number of subjects, you can see the summary results without having to generate output for each subject.

ALL . *The results of both the experimental data and simulation data analyses are included*. ALL is the default.

NONE. *No results are written to the display file*. This keyword is useful if you are interested only in writing the utility file (see "UTILITY Subcommand" below).

UTILITY Subcommand

UTILITY writes a utility file to the specified IBM SPSS Statistics file.

- If UTILITY is not specified, no utility file is written.
- UTILITY is followed by the name of the file to be written.
- The file is specified in the usual manner for your operating system.
- The utility file contains one case for each subject. If SUBJECT is not specified, the utility file contains a single case with statistics for the group as a whole.

The variables that are written to the utility file are in the following order:

- Any SPLIT FILE variables in the active dataset.
- Any SUBJECT variable.
- The constant for the regression equation for the subject. The regression equation constant is named CONSTANT.
- For DISCRETE factors, all of the utilities that are estimated for the subject. The names of the utilities that are estimated with DISCRETE factors are formed by appending a digit after the factor name. The first utility gets a 1, the second utility gets a 2, and so on.
- For LINEAR factors, a single coefficient. The name of the coefficient for LINEAR factors is formed by appending *_L* to the factor name. (To calculate the predicted score, multiply the factor value by the coefficient.)
- For IDEAL or ANTIIDEAL factors, two coefficients. The name of the two coefficients for IDEAL or ANTIIDEAL factors are formed by appending *_L* and *_Q*, respectively, to the factor name. (To use these coefficients in calculating the predicted score, multiply the factor value by the first coefficient and add that to the product of the second coefficient and the square of the factor value.)
- The estimated ranks or scores for all profiles in the plan file. The names of the estimated ranks or scores are of the form *SCORE n* for experimental and holdout profiles, or *SIMUL n* for simulation profiles, where *n* is the position in the plan file. The name is *SCORE* for experimental and holdout profiles even if the data are ranks.

If the variable names that are created are too long, letters are truncated from the end of the original variable name before new suffixes are appended.

PLOT Subcommand

The PLOT subcommand produces plots in addition to the output that is usually produced by CONJOINT.

The following keywords are available for this subcommand:

SUMMARY. *Produces a bar chart of the importance values for all variables, plus a utility bar chart for each variable*. This setting is the default if the PLOT subcommand is specified with no keywords.

SUBJECT. *Plots a clustered bar chart of the importance values for each factor, clustered by subjects, and one clustered bar chart for each factor, showing the utilities for each factor level, clustered by subjects*. If no SUBJECT subcommand was specified naming the variables, no plots are produced and a warning is displayed.

ALL. *Plots both summary and subject charts*.

NONE. *Does not produce any charts*. This setting is the default if the subcommand is omitted.

CORRELATIONS

CORRELATIONS is available in Statistics Base Edition.

```
CORRELATIONS VARIABLES= varlist [WITH varlist] [/varlist...]

 [/MISSING={PAIRWISE**}  [{INCLUDE}]]
           {LISTWISE  }   {EXCLUDE}

 [/PRINT={TWOTAIL**} {SIG**} {FULL**  }]
         {ONETAIL  } {NOSIG} {LOWER   }
                             {LNODIAG }
                             {NOMATRIX}

 [/MATRIX=OUT({*                 })]
              {'savfile'|'dataset'}

 [/STATISTICS=[DESCRIPTIVES] [XPROD] [ALL]]

 [/CI
    [METHOD ={FISHER**} {EXACT}]
    [BIAS ={FALSE**} {TRUE}]
    [CILEVEL ={95**} {value}]
 ]
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 27.0.1

- Added support for the **NOMATRIX** keyword in the **PRINT** subcommand.
- Added support for the **CI** subcommand.

Release 27.0

- Added support for the **FULL**, **LOWER** and **LNODIAG** keywords in the **PRINT** subcommand.

Release 16.0

- Added support for SET THREADS and SET MCACHE.

Example

```
CORRELATIONS VARIABLES=FOOD RENT PUBTRANS TEACHER COOK ENGINEER
  /MISSING=INCLUDE.
```

Overview

CORRELATIONS (alias PEARSON CORR) produces Pearson product-moment correlations with significance levels and, optionally, univariate statistics, covariances, and cross-product deviations. Other procedures that produce correlation matrices are PARTIAL CORR, REGRESSION, DISCRIMINANT, and FACTOR.

Options

Types of Matrices. A simple variable list on the VARIABLES subcommand produces a square matrix. You can also request a rectangular matrix of correlations between specific pairs of variables or between variable lists using the keyword WITH on VARIABLES.

Significance Levels. By default, CORRELATIONS displays the number of cases and significance levels for each coefficient. Significance levels are based on a two-tailed test. You can request a one-tailed test, and you can display the significance level for each coefficient as an annotation using the PRINT subcommand.

Additional Statistics. You can obtain the mean, standard deviation, and number of nonmissing cases for each variable, and the cross-product deviations and covariance for each pair of variables using the STATISTICS subcommand.

Matrix Output. You can write matrix materials to a data file using the MATRIX subcommand. The matrix materials include the mean, standard deviation, number of cases used to compute each coefficient, and Pearson correlation coefficient for each variable. The matrix data file can be read by several other procedures.

Basic Specification

- The basic specification is the VARIABLES subcommand, which specifies the variables to be analyzed.
- By default, CORRELATIONS produces a matrix of correlation coefficients. The number of cases and the significance level are displayed for each coefficient. The significance level is based on a two-tailed test.

Subcommand Order

- The VARIABLES subcommand must be first.
- The remaining subcommands can be specified in any order.

Operations

- The correlation of a variable with itself is displayed as 1.0000.
- A correlation that cannot be computed is displayed as a period (.).
- CORRELATIONS does not execute if string variables are specified on the variable list.
- This procedure uses the multithreaded options specified by SET THREADS and SET MCACHE.

Limitations

- A maximum of 40 variable lists.
- A maximum of 500 variables total per command.
- A maximum of 250 syntax elements. Each individual occurrence of a variable name, keyword, or special delimiter counts as 1 toward this total. Variables implied by the TO keyword do not count toward this total.

Example

```
CORRELATIONS
  /VARIABLES=sales mpg
  /PRINT=TWOTAIL NOSIG
  /MISSING=PAIRWISE .
```

VARIABLES Subcommand

VARIABLES specifies the variable list.

- A simple variable list produces a square matrix of correlations of each variable with every other variable.
- Variable lists joined by the keyword WITH produce a rectangular correlation matrix. Variables before WITH define the rows of the matrix and variables after WITH define the columns.
- The keyword ALL can be used on the variable list to refer to all user-defined variables.
- You can specify multiple VARIABLES subcommands on a single CORRELATIONS command. The slash between the subcommands is required; the keyword VARIABLES is not.
- If only **varlist1**is specified, the confidence limits are estimated for each different pair of the variables contained within **varlist1**. If varlist1 WITH varlist2 is specified, the confidence limits are estimated for each different pair of the variables with one contained within **varlist1** and the other **varlist2**. The confidence limits of the pair containing a variable and itself are not estimated.

Example

```
CORRELATIONS VARIABLES=FOOD RENT PUBTRANS TEACHER COOK ENGINEER
  /VARIABLES=FOOD RENT WITH COOK TEACHER MANAGER ENGINEER
  /MISSING=INCLUDE.
```

- The first VARIABLES subcommand requests a square matrix of correlation coefficients among the variables *FOOD*, *RENT*, *PUBTRANS*, *TEACHER*, *COOK*, and *ENGINEER*.
- The second VARIABLES subcommand requests a rectangular correlation matrix in which *FOOD* and *RENT* are the row variables and *COOK*, *TEACHER*, *MANAGER*, and *ENGINEER* are the column variables.

PRINT Subcommand

PRINT
Controls whether the significance level is based on a one- or two-tailed test and whether the number of cases and the significance level for each correlation coefficient are displayed.

TWOTAIL
Two-tailed test of significance. This test is appropriate when the direction of the relationship cannot be determined in advance, as is often the case in exploratory data analysis. This is the default.

ONETAIL
One-tailed test of significance. This test is appropriate when the direction of the relationship between a pair of variables can be specified in advance of the analysis.

SIG
Do not flag significant values. SIG is the default.

NOSIG
Flag significant values. Values significant at the 0.05 level are flagged with a single asterisk; those that are significant at the 0.01 level are flagged with two asterisks.

FULL
Show the full table. The full correlation matrix table is presented in the output.

LOWER
Show only the table's lower triangle. Only the correlation matrix table's lower triangle is presented in the output. The keyword allows table output to adhere to APA style guidelines.

LNODIAG
Show only the table's lower triangle and hide diagonal values. Only the correlation matrix table's lower triangle is presented in the output. Diagonal values are also hidden. The keyword allows table output to adhere to APA style guidelines.

NOMATRIX
Suppresses the Correlations table. Honored only when **/CI** is specified; otherwise, **FULL** is assumed.

STATISTICS Subcommand

The correlation coefficients are automatically displayed in the Correlations table for an analysis specified by a VARIABLES list. STATISTICS requests additional statistics.

DESCRIPTIVES . *Display mean, standard deviation, and number of nonmissing cases for each variable on the Variables list in the Descriptive Statistics table.* This table precedes all Correlations tables. Variables specified on more than one VARIABLES list are displayed only once. Missing values are handled on a variable-by-variable basis regardless of the missing-value option in effect for the correlations.

XPROD . *Display cross-product deviations and covariance for each pair of variables in the Correlations table(s).*

ALL . *All additional statistics.* This produces the same statistics as DESCRIPTIVES and XPROD together.

MISSING Subcommand

MISSING controls the treatment of missing values.

- The PAIRWISE and LISTWISE keywords are alternatives; however, each can be specified with INCLUDE or EXCLUDE.

- The default is PAIRWISE and EXCLUDE.

PAIRWISE . *Exclude missing values pairwise.* Cases that have missing values for one or both of a pair of variables for a specific correlation coefficient are excluded from the computation of that coefficient. Since each coefficient is based on all cases that have valid values for that particular pair of variables, this can result in a set of coefficients based on a varying number of cases. The valid number of cases is displayed in the Correlations table. This is the default.

LISTWISE . *Exclude missing values listwise.* Cases that have missing values for any variable named on any VARIABLES list are excluded from the computation of all coefficients across lists. The valid number of cases is the same for all analyses and is displayed in a single annotation.

INCLUDE . *Include user-missing values.* User-missing values are included in the analysis.

EXCLUDE . *Exclude all missing values.* Both user- and system-missing values are excluded from the analysis.

MATRIX Subcommand

MATRIX writes matrix materials to a data file or previously declared dataset (DATASET DECLARE command). The matrix materials include the mean and standard deviation for each variable, the number of cases used to compute each coefficient, and the Pearson correlation coefficients. Several procedures can read matrix materials produced by CORRELATIONS, including PARTIAL CORR, REGRESSION, FACTOR, and CLUSTER.

- CORRELATIONS cannot write rectangular matrices (those specified with the keyword WITH) to a file.
- If you specify more than one variable list on CORRELATIONS, only the last list that does not use the keyword WITH is written to the matrix data file.
- The keyword OUT specifies the file to which the matrix is written. Specify an asterisk to replace the active dataset or a quoted file specification or dataset name, enclosed in parentheses.
- Documents from the original file will not be included in the matrix file and will not be present if the matrix file becomes the working data file.

Format of the Matrix Data File

- The matrix data file has two special variables created by the program: *ROWTYPE_* and *VARNAME_*. The variable *ROWTYPE_* is a short string variable with values *MEAN*, *STDDEV*, *N*, and *CORR* (for Pearson correlation coefficient). The next variable, *VARNAME_*, is a short string variable whose values are the names of the variables used to form the correlation matrix. When *ROWTYPE_* is *CORR*, *VARNAME_* gives the variable associated with that row of the correlation matrix.
- The remaining variables in the file are the variables used to form the correlation matrix.

Split Files

- When split-file processing is in effect, the first variables in the matrix file will be split variables, followed by *ROWTYPE_*, *VARNAME_*, and the variables used to form the correlation matrix.
- A full set of matrix materials is written for each subgroup defined by the split variables.
- A split variable cannot have the same name as any other variable written to the matrix data file.
- If split-file processing is in effect when a matrix is written, the same split-file specifications must be in effect when that matrix is read by another procedure.

Missing Values

- With pairwise treatment of missing values (the default), a matrix of the number of cases used to compute each coefficient is included with the matrix materials.
- With listwise treatment, a single number indicating the number of cases used to calculate all coefficients is included.

Example

```
GET FILE=CITY /KEEP FOOD RENT PUBTRANS TEACHER COOK ENGINEER.
CORRELATIONS VARIABLES=FOOD TO ENGINEER
 /MATRIX OUT(CORRMAT).
```

- CORRELATIONS reads data from the file *CITY* and writes one set of matrix materials to the file *CORRMAT*. The working file is still *CITY*. Subsequent commands are executed on *CITY*.

Example

```
GET FILE=CITY /KEEP FOOD RENT PUBTRANS TEACHER COOK ENGINEER.
CORRELATIONS VARIABLES=FOOD TO ENGINEER
 /MATRIX OUT(*).
LIST.
DISPLAY DICTIONARY.
```

- CORRELATIONS writes the same matrix as in the example above. However, the matrix data file replaces the working file. The LIST and DISPLAY commands are executed on the matrix file, not on the *CITY* file.

Example

```
CORRELATIONS VARIABLES=FOOD RENT COOK TEACHER MANAGER ENGINEER
 /FOOD TO TEACHER /PUBTRANS WITH MECHANIC
 /MATRIX OUT(*).
```

- Only the matrix for *FOOD* TO *TEACHER* is written to the matrix data file because it is the last variable list that does not use the keyword WITH.

CI Subcommand

The optional CI subcommand controls the estimation of the confidence intervals. The subcommand is not specified by default, which suppresses the estimation. When specified, the confidence limits are estimated and displayed in the output.

METHOD
The optional keyword determines the method to estimate the confidence interval. **FISHER** is the default setting, which estimates the confidence limits based on the Fisher's transformation. **EXACT** uses the exact method.

BIAS
The optional keyword controls whether the bias adjustment is applied. **FALSE** is the default setting, which does not take the bias term into consideration. **TRUE** applies the bias adjustment to the estimation of the confidence limits. The keyword takes effect when METHOD = FISHER, and is ignored otherwise.

CILEVEL
The optional keyword determines the level of the confidence intervals or limits. The value must be a single value $\in (0,100)$ with 95 as the default setting.

CORRESPONDENCE

CORRESPONDENCE is available in SPSS Statistics Professional Edition or the Categories option.

```
CORRESPONDENCE

/TABLE = {rowvar (min, max) BY colvar (min, max)}
         {ALL (# of rows, # of columns)         }

[/SUPPLEMENTARY = [{rowvar (valuelist)}] [{colvar (valuelist)}]]
                   {ROW (valuelist)   }   {COLUMN (valuelist)}

[/EQUAL = [{rowvar (valuelist)}] [{colvar (valuelist)}]]
           {ROW (valuelist)   }   {COLUMN (valuelist)}

[/MEASURE = {CHISQ**}]
            {EUCLID }

[/STANDARDIZE = {RMEAN   }]
                {CMEAN   }
                {RCMEAN**}
                {RSUM    }
                {CSUM    }

[/DIMENSION = {2**  }]
              {value}

[/NORMALIZATION = {SYMMETRICAL**}]
                  {PRINCIPAL    }
                  {RPRINCIPAL   }
                  {CPRINCIPAL   }
                  {value        }

[/PRINT = [TABLE**] [RPROF] [CPROF] [RPOINTS**] [CPOINTS**]
          [RCONF] [CCONF] [PERMUTATION[(n)]] [DEFAULT] [NONE]]

[/PLOT = [NDIM({value,value})]
               {value,MAX  }
         [RPOINTS[(n)]] [CPOINTS[(n)] [TRROWS[(n)]]
         [TRCOLUMNS[(n)]] [BIPLOT**[(n)]] [NONE]]

[/OUTFILE = [SCORE('savfile'|'dataset')] [VARIANCE('savfile'|'dataset')]
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 13.0

- For the NDIM keyword on the PLOT subcommand, the default is changed to all dimensions.
- The maximum label length on the PLOT subcommand is increased to 60 (previous value was 20).

Overview

CORRESPONDENCE displays the relationships between rows and columns of a two-way table graphically by a biplot. It computes the row and column scores and statistics and produces plots based on the scores. Also, confidence statistics are computed.

Options

Number of Dimensions. You can specify how many dimensions CORRESPONDENCE should compute.

Supplementary Points. You can specify supplementary rows and columns.

Equality Restrictions. You can restrict rows and columns to have equal scores.

Measure. You can specify the distance measure to be the chi-square of Euclidean.

Standardization. You can specify one of five different standardization methods.

Method of Normalization. You can specify one of five different methods for normalizing the row and column scores.

Confidence Statistics. You can request computation of confidence statistics (standard deviations and correlations) for row and column scores. For singular values, confidence statistics are always computed.

Data Input. You can analyze individual casewise data, aggregated data, or table data.

Display Output. You can control which statistics are displayed and plotted.

Writing Matrices. You can write the row and column scores and the confidence statistics (variances and covariances) for the singular values to external files.

Basic Specification

- The basic specification is CORRESPONDENCE and the TABLE subcommand. By default, CORRESPONDENCE computes a two-dimensional solution and displays the correspondence table, the summary table, an overview of the row and column scores, and a biplot of the row and column points.

Subcommand Order

- The TABLE subcommand must appear first.
- All other subcommands can appear in any order.

Syntax Rules

- Only one keyword can be specified on the MEASURE subcommand.
- Only one keyword can be specified on the STANDARDIZE subcommand.
- Only one keyword can be specified on the NORMALIZATION subcommand.
- Only one parameter can be specified on the DIMENSION subcommand.

Operations

- If a subcommand is specified more than once, only the last occurrence is executed.

Limitations

- The table input data and the aggregated input data cannot contain negative values. CORRESPONDENCE will treat such values as 0.
- Rows and columns that are specified as supplementary cannot be equalized.
- The maximum number of supplementary points for a variable is 200.
- The maximum number of equalities for a variable is 200.

Example

```
CORRESPONDENCE TABLE=MENTAL(1,4) BY SES(1,6)
/PRINT=RPOINTS CPOINTS
/PLOT=RPOINTS CPOINTS.
```

- Two variables, *MENTAL* and *SES*, are specified on the TABLE subcommand. *MENTAL* has values ranging from 1 to 4, and *SES* has values ranging from 1 to 6.
- The summary table and overview tables of the row and column scores are displayed.
- The row points plot and the column points plot are produced.

TABLE Subcommand

TABLE specifies the row and column variables along with their integer value ranges. The two variables are separated by the keyword BY.

- The TABLE subcommand is required.

Casewise Data

- Each variable is followed by an integer value range in parentheses. The value range consists of the variable's minimum value and its maximum value.
- Values outside of the specified range are not included in the analysis.
- Values do not have to be sequential. Empty categories yield a zero in the input table and do not affect the statistics for other categories.

Example

```
DATA LIST FREE/VAR1 VAR2.
BEGIN DATA
3 1
6 1
3 1
4 2
4 2
6 3
6 3
6 3
3 2
4 2
6 3
END DATA.
CORRESPONDENCE TABLE=VAR1(3,6) BY VAR2(1,3).
```

- DATA LIST defines two variables, *VAR1* and *VAR2*.
- *VAR1* has three levels, coded 3, 4, and 6. *VAR2* also has three levels, coded 1, 2, and 3.
- Since a range of (3,6) is specified for *VAR1*, CORRESPONDENCE defines four categories, coded 3, 4, 5, and 6. The empty category, 5, for which there is no data, receives system-missing values for all statistics and does not affect the analysis.

Aggregated Data

To analyze aggregated data, such as data from a crosstabulation where cell counts are available but the original raw data are not, you can use the WEIGHT command before CORRESPONDENCE.

Example

To analyze a 3×3 table, such as the one shown below, you could use these commands:

```
DATA LIST FREE/ BIRTHORD ANXIETY COUNT.
BEGIN DATA
1 1 48
1 2 27
1 3 22
2 1 33
2 2 20
2 3 39
3 1 29
3 2 42
3 3 47
END DATA.
WEIGHT BY COUNT.
CORRESPONDENCE TABLE=BIRTHORD (1,3) BY ANXIETY (1,3).
```

- The WEIGHT command weights each case by the value of *COUNT*, as if there are 48 subjects with *BIRTHORD*=1 and *ANXIETY*=1, 27 subjects with *BIRTHORD*=1 and *ANXIETY*=2, and so on.
- CORRESPONDENCE can then be used to analyze the data.
- If any of the table cell values (the values of the WEIGHT variable) equals 0, the WEIGHT command issues a warning, but the CORRESPONDENCE analysis is done correctly.
- The table cell values (the values of the WEIGHT variable) cannot be negative.

Table 24. 3 x 3 table

Birth Order	Anxiety High	Anxiety Med	Anxiety Low
First	48	27	22
Second	33	20	39
Other	29	42	47

Table Data

- The cells of a table can be read and analyzed directly by using the keyword ALL after TABLE.
- The columns of the input table must be specified as variables on the DATA LIST command. Only columns are defined, not rows.
- ALL is followed by the number of rows in the table, a comma, and the number of columns in the table, all in parentheses.
- The row variable is named *ROW,* and the column variable is named *COLUMN*.
- The number of rows and columns specified can be smaller than the actual number of rows and columns if you want to analyze only a subset of the table.
- The variables (columns of the table) are treated as the column categories, and the cases (rows of the table) are treated as the row categories.
- Row categories can be assigned values (category codes) when you specify TABLE=ALL by the optional variable *ROWCAT_*. This variable must be defined as a numeric variable with unique values corresponding to the row categories. If *ROWCAT_* is not present, the row index (case) numbers are used as row category values.

Example

```
DATA LIST /ROWCAT_ 1 COL1 3-4 COL2 6-7 COL3 9-10.
BEGIN DATA
1 50 19 26
2 16 40 34
3 12 35 65
4 11 20 58
END DATA.
VALUE LABELS ROWCAT_ 1 'ROW1' 2 'ROW2' 3 'ROW3' 4 'ROW4'.
CORRESPONDENCE TABLE=ALL(4,3).
```

- DATA LIST defines the row category naming variable *ROWCAT_* and the three columns of the table as the variables.
- The TABLE=ALL specification indicates that the data are the cells of a table. The (4,3) specification indicates that there are four rows and three columns.
- The column variable is named *COLUMN* with categories labeled *COL1*, *COL2*, and *COL3*.
- The row variable is named *ROW* with categories labeled *ROW1*, *ROW2*, *ROW3*, and *ROW4*.

DIMENSION Subcommand

DIMENSION specifies the number of dimensions you want CORRESPONDENCE to compute.

- If you do not specify the DIMENSION subcommand, CORRESPONDENCE computes two dimensions.
- DIMENSION is followed by a positive integer indicating the number of dimensions. If this parameter is omitted, a value of 2 is assumed.
- In general, you should choose as few dimensions as needed to explain most of the variation. The minimum number of dimensions that can be specified is 1. The maximum number of dimensions that can be specified equals the minimum of the number of active rows and the number of active columns minus 1. An active row or column is a nonsupplementary row or column that is used in the analysis. For example, in a table where the number of rows is 5 (2 of which are supplementary) and the number of columns is 4, the number of active rows (3) is smaller than the number of active columns (4). Thus, the maximum number of dimensions that can be specified is (5–2)–1, or 2. Rows and columns that are restricted to have equal scores count as 1 toward the number of active rows or columns. For example, in a table with five rows and four columns, where two columns are restricted to have equal scores, the number of active rows is 5 and the number of active columns is (4–1), or 3. The maximum number of dimensions that can be specified is (3–1), or 2. Empty rows and columns (rows or columns with no data, all zeros, or all missing data) are not counted toward the number of rows and columns.
- If more than the maximum allowed number of dimensions is specified, CORRESPONDENCE reduces the number of dimensions to the maximum.

SUPPLEMENTARY Subcommand

The SUPPLEMENTARY subcommand specifies the rows and/or columns that you want to treat as supplementary (also called passive or illustrative).

- For casewise data, the specification on SUPPLEMENTARY is the row and/or column variable name, followed by a value list in parentheses. The values must be in the value range specified on the TABLE subcommand for the row or column variable.
- For table data, the specification on SUPPLEMENTARY is *ROW* and/or *COLUMN*, followed by a value list in parentheses. The values represent the row or column indices of the table input data.
- The maximum number of supplementary rows or columns is the number of rows or columns minus 2. Rows and columns that are restricted to have equal scores count as 1 toward the number of rows or columns.
- Supplementary rows and columns cannot be equalized.

Example

```
CORRESPONDENCE TABLE=MENTAL(1,8) BY SES(1,6)
/SUPPLEMENTARY MENTAL(3) SES(2,6).
```

- SUPPLEMENTARY specifies the third level of *MENTAL* and the second and sixth levels of *SES* to be supplementary.

Example

```
CORRESPONDENCE TABLE=ALL(8,6)
/SUPPLEMENTARY ROW(3) COLUMN(2,6).
```

- SUPPLEMENTARY specifies the third level of the row variable and the second and sixth levels of the column variable to be supplementary.

EQUAL Subcommand

The EQUAL subcommand specifies the rows and/or columns that you want to restrict to have equal scores.

- For casewise data, the specification on EQUAL is the row and/or column variable name, followed by a list of at least two values in parentheses. The values must be in the value range specified on the TABLE subcommand for the row or column variable.
- For table data, the specification on EQUAL is *ROW* and/or *COLUMN*, followed by a value list in parentheses. The values represent the row or column indices of the table input data.
- Rows or columns that are restricted to have equal scores cannot be supplementary.
- The maximum number of equal rows or columns is the number of active rows or columns minus 1.

Example

```
CORRESPONDENCE TABLE=MENTAL(1,8) BY SES(1,6)
/EQUAL MENTAL(1,2) (6,7) SES(1,2,3).
```

- EQUAL specifies the first and second level of *MENTAL*, the sixth and seventh level of *MENTAL*, and the first, second, and third levels of *SES* to have equal scores.

MEASURE Subcommand

The MEASURE subcommand specifies the measure of distance between the row and column profiles.

- Only one keyword can be used.

The following keywords are available:

CHISQ. *Chi-square distance*. This is the weighted distance, where the weight is the mass of the rows or columns. This is the default specification for MEASURE and is the necessary specification for standard correspondence analysis.

EUCLID. *Euclidean distance.* The distance is the square root of the sum of squared differences between the values for two rows or columns.

STANDARDIZE Subcommand

When MEASURE=EUCLID, the STANDARDIZE subcommand specifies the method of standardization.

- Only one keyword can be used.
- If MEASURE is CHISQ, only RCMEAN standardization can be used, resulting in standard correspondence analysis.

The following keywords are available:

RMEAN. *The row means are removed.*

CMEAN. *The column means are removed.*

RCMEAN. *Both the row and column means are removed.* This is the default specification.

RSUM. *First the row totals are equalized and then the row means are removed.*

CSUM. *First the column totals are equalized and then the column means are removed.*

NORMALIZATION Subcommand

The NORMALIZATION subcommand specifies one of five methods for normalizing the row and column scores. Only the scores and confidence statistics are affected; contributions and profiles are not changed.

The following keywords are available:

SYMMETRICAL. *For each dimension, rows are the weighted average of columns divided by the matching singular value, and columns are the weighted average of rows divided by the matching singular value*. This is the default if the NORMALIZATION subcommand is not specified. Use this normalization method if you are primarily interested in differences or similarities between rows and columns.

PRINCIPAL. *Distances between row points and distances between column points are approximations of chi-square distances or of Euclidean distances (depending on MEASURE)*. The distances represent the distance between the row or column and its corresponding average row or column profile. Use this normalization method if you want to examine both differences between categories of the row variable and differences between categories of the column variable (but not differences between variables).

RPRINCIPAL. *Distances between row points are approximations of chi-square distances or of Euclidean distances (depending on MEASURE)*. This method maximizes distances between row points, resulting in row points that are weighted averages of the column points. This is useful when you are primarily interested in differences or similarities between categories of the row variable.

CPRINCIPAL. *Distances between column points are approximations of chi-square distances or of Euclidean distances (depending on MEASURE)*. This method maximizes distances between column points, resulting in column points that are weighted averages of the row points. This is useful when you are primarily interested in differences or similarities between categories of the column variable.

The fifth method allows the user to specify any value in the range –1 to +1, inclusive. A value of 1 is equal to the RPRINCIPAL method, a value of 0 is equal to the SYMMETRICAL method, and a value of –1 is equal to the CPRINCIPAL method. By specifying a value between –1 and 1, the user can spread the inertia over both row and column scores to varying degrees. This method is useful for making tailor-made biplots.

PRINT Subcommand

Use PRINT to control which of several correspondence statistics are displayed. The summary table (singular values, inertia, proportion of inertia accounted for, cumulative proportion of inertia accounted for, and confidence statistics for the maximum number of dimensions) is always produced. If PRINT is not specified, the input table, the summary table, the overview of row points table, and the overview of column points table are displayed.

The following keywords are available:

TABLE. *A crosstabulation of the input variables showing row and column marginals*.

RPROFILES. *The row profiles*. PRINT=RPROFILES is analogous to the CELLS=ROW subcommand in CROSSTABS.

CPROFILES. *The column profiles*. PRINT=CPROFILES is analogous to the CELLS= COLUMN subcommand in CROSSTABS.

RPOINTS. *Overview of row points (mass, scores, inertia, contribution of the points to the inertia of the dimension, and the contribution of the dimensions to the inertia of the points).*

CPOINTS. *Overview of column points (mass, scores, inertia, contribution of the points to the inertia of the dimension, and the contribution of the dimensions to the inertia of the points).*

RCONF. *Confidence statistics (standard deviations and correlations) for the active row points.*

CCONF. *Confidence statistics (standard deviations and correlations) for the active column points.*

PERMUTATION(n). *The original table permuted according to the scores of the rows and columns.* PERMUTATION can be followed by a number in parentheses indicating the maximum number of dimensions for which you want permuted tables. The default number of dimensions is 1.

NONE. *No output other than the SUMMARY table*.

DEFAULT. *TABLE, RPOINTS, CPOINTS, and the SUMMARY tables*. These statistics are displayed if you omit the PRINT subcommand.

PLOT Subcommand

Use PLOT to produce a biplot of row and column points, plus plots of the row points, column points, transformations of the categories of the row variable, and transformations of the categories of the column variable. If PLOT is not specified or is specified without keywords, a biplot is produced.

The following keywords are available:

TRROWS(n). *Transformation plots for the rows (row category scores against row category indicator values).*

TRCOLUMNS(n). *Transformation plots for the columns (column category scores against column category indicator values).*

RPOINTS(n). *Plot of the row points*.

CPOINTS(n). *Plot of the column points*.

BIPLOT(n). *Biplot of the row and column points*. This is the default plot. This plot is not available when NORMALIZATION=PRINCIPAL.

NONE. *No plots*.

- For all of the keywords except NONE the user can specify an optional parameter *l* in parentheses in order to control the global upper boundary of value label lengths in the plot. The label length parameter *l* can take any nonnegative integer less than or equal to the applicable maximum length of 60. If *l* is not specified, CORRESPONDENCE assumes that each value label at its full length is displayed. If *l* is an integer larger than the applicable maximum, then we reset it to the applicable maximum, but do not

issue a warning. If a positive value of *l* is given but if some or all of the category values do not have labels, then for those values the values themselves are used as the labels.

In addition to the plot keywords, the following can be specified:

NDIM(value,value) . *Dimension pairs to be plotted.* NDIM is followed by a pair of values in parentheses. If NDIM is not specified or if NDIM is specified without parameter values, a matrix scatterplot including all dimensions is produced.

- The first value must be any integer from 1 to the number of dimensions in the solution minus 1.
- The second value must be an integer from 2 to the number of dimensions in the solution. The second value must exceed the first. Alternatively, the keyword MAX can be used instead of a value to indicate the highest dimension of the solution.
- For TRROWS and TRCOLUMNS, the first and second values indicate the range of dimensions for which the plots are created.
- For RPOINTS, CPOINTS, and BIPLOT, the first and second values indicate plotting pairs of dimensions. The first value indicates the dimension that is plotted against higher dimensions. The second value indicates the highest dimension to be used in plotting the dimension pairs.

Example

```
CORRESPONDENCE TABLE=MENTAL(1,4) BY SES(1,6)
/PLOT NDIM(1,3) BIPLOT(5).
```

- BIPLOT and NDIM(1,3) requests that a scatterplot for dimensions 1 and 2, and a scatterplot for dimensions 1 and 3 should be produced.
- The 5 following BIPLOT indicates that only the first five characters of each label are to be shown in the biplot matrix.

Example

```
CORRESPONDENCE TABLE=MENTAL(1,4) BY SES(1,6)
/DIMENSION = 3
/PLOT NDIM(1,MAX) TRROWS.
```

- Three transformation plots for the row categories are produced, one for each dimension from 1 to the highest dimension of the analysis (in this case, 3). The label parameter is not specified, and so the category labels in the plot are shown up their full lengths.

OUTFILE Subcommand

Use OUTFILE to write row and column scores and/or confidence statistics (variances and covariances) for the singular values and row and column scores to an an external IBM SPSS Statistics data file or previously declared dataset.

OUTFILE must be followed by one or both of the following keywords:

SCORE ('file'|'dataset'). *Write row and column scores*.

VARIANCE ('file'|'dataset'). *Write variances and covariances*.

- Filenames should be enclosed in quotes and are stored in the working directory unless a path is included as part of the file specification. Datasets are available during the current session but are not available in subsequent sessions unless you explicitly save them as data files. The names should be different for the each of the keywords.
- For VARIANCE, supplementary and equality constrained rows and columns are not produced in the external file.

The variables in the SCORE matrix data file and their values are:

ROWTYPE_. *String variable containing the value ROW for all of the rows and COLUMN for all of the columns*.

LEVEL_. *String variable containing the values (or value labels, if present) of each original variable*.

VARNAME_. *String variable containing the original variable names*.

DIM1...DIMn. *Numerical variables containing the row and column scores for each dimension*. Each variable is named *DIM n*, where *n* represents the dimension number.

The variables in the VARIANCE matrix data file and their values are:

ROWTYPE_. *String variable containing the value COV for all of the cases in the file*.

VARNAME_. *String variable containing the value SINGULAR, the row variable's name, and the column variable's name*.

LEVEL_. *String variable containing the row variable's values (or labels), the column variable's values (or labels), and a blank value for VARNAME_ = SINGULAR*.

DIMNMBR_. *String variable containing the dimension number*.

DIM1...DIMn. *Numerical variables containing the variances and covariances for each dimension*. Each variable is named *DIM n*, where *n* represents the dimension number.

COUNT

```
COUNT varname=varlist(value list) [/varname=...]
```

Keywords for numeric value lists:

`LOWEST, LO, HIGHEST, HI, THRU, MISSING, SYSMIS`

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic "Command Order" on page 36 for more information.

Example

```
COUNT TARGET=V1 V2 V3 (2).
```

Overview

COUNT creates a numeric variable that, for each case, counts the occurrences of the same value (or list of values) across a list of variables. The new variable is called the *target* variable. The variables and values that are counted are the *criterion* variables and values. Criterion variables can be either numeric or string.

Basic Specification

The basic specification is the target variable, an equals sign, the criterion variable(s), and the criterion value(s) enclosed in parentheses.

Syntax Rules

- Use a slash to separate the specifications for each target variable.
- The criterion variables specified for a single target variable must be either all numeric or all string.
- Each value on a list of criterion values must be separated by a comma or space. String values must be enclosed in quotes.
- The keywords THRU, LOWEST (LO), HIGHEST (HI), SYSMIS, and MISSING can be used only with numeric criterion variables.
- A variable can be specified on more than one criterion variable list.
- You can use the keyword TO to specify consecutive criterion variables that have the same criterion value or values.
- You can specify multiple variable lists for a single target variable to count different values for different variables.

Operations

- Target variables are always numeric and are initialized to 0 for each case. They are assigned a dictionary format of F8.2.
- If the target variable already exists, its previous values are replaced.
- COUNT ignores the missing-value status of user-missing values. It counts a value even if that value has been previously declared as missing.
- The target variable is never system-missing. To define user-missing values for target variables, use the RECODE or MISSING VALUES command.
- SYSMIS counts system-missing values for numeric variables.
- MISSING counts both user- and system-missing values for numeric variables.

Examples

Counting Occurrences of a Single Value

```
COUNT TARGET=V1 V2 V3 (2).
```

- The value of *TARGET* for each case will be either 0, 1, 2, or 3, depending on the number of times the value 2 occurs across the three variables for each case.
- *TARGET* is a numeric variable with an F8.2 format.

Counting Occurrences of a Range of Values and System-Missing Values

```
COUNT QLOW=Q1 TO Q10 (LO THRU 0)
  /QSYSMIS=Q1 TO Q10 (SYSMIS).
```

- Assuming that there are 10 variables between and including *Q1* and *Q10* in the active dataset, *QLOW* ranges from 0 to 10, depending on the number of times a case has a negative or 0 value across the variables *Q1* to *Q10*.
- *QSYSMIS* ranges from 0 to 10, depending on how many system-missing values are encountered for *Q1* to *Q10* for each case. User-missing values are not counted.
- Both *QLOW* and *QSYSMIS* are numeric variables and have F8.2 formats.

Counting Occurrences of String Values

```
COUNT SVAR=V1 V2 ('male  ') V3 V4 V5 ('female').
```

- *SVAR* ranges from 0 to 5, depending on the number of times a case has a value of male for *V1* and *V2* and a value of female for *V3*, *V4*, and *V5*.
- *SVAR* is a numeric variable with an F8.2 format.

COXREG

COXREG is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
COXREG VARIABLES = survival varname [WITH varlist]
    / STATUS = varname [EVENT] (vallist) [LOST (vallist)]
   [/STRATA  = varname]
   [/CATEGORICAL = varname]
   [/CONTRAST (varname) = {DEVIATION (refcat)}]
                          {SIMPLE (refcat)   }
                          {DIFFERENCE        }
                          {HELMERT           }
                          {REPEATED          }
                          {POLYNOMIAL(metric)}
                          {SPECIAL (matrix)  }
                          {INDICATOR (refcat)}

   [/METHOD = {ENTER**      }   [{varlist}]]
              {BSTEP [{COND}]}    {ALL    }
                      {LR  }
                      {WALD}
              {FSTEP [{COND}]}
                      {LR  }
                      {WALD}

   [/MISSING = {EXCLUDE**}]
               {INCLUDE  }

   [/PRINT = [{DEFAULT**}]  [CI ({95})]]
             {SUMMARY   }        {n }
             {BASELINE  }
             {CORR      }
             {ALL       }

   [/CRITERIA = [{BCON}({1E-4**})]   [LCON({1E-5**})]
                 {PCON} { n    }            { n    }
                [ITERATE({20**})]
                         { n  }
                [PIN({0.05**})]      [POUT({0.1**})]]
                     { n    }             { n   }

   [/PLOT = [NONE**] [SURVIVAL] [HAZARD] [LML] [OMS]]
   [/PATTERN = [varname(value)...] [BY varname]]
   [/OUTFILE = [COEFF('savfile' | 'dataset')] [TABLE('savfile' | 'dataset')]
               [PARAMETER('file')]]
   [/SAVE = tempvar [(newvarname)],tempvar ...]
   [/EXTERNAL]
```

**Default if subcommand or keyword is omitted.

Temporary variables created by COXREG are:

- SURVIVAL
- SE
- HAZARD
- RESID
- LML
- DFBETA
- PRESID
- XBETA

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
TIME PROGRAM.
COMPUTE Z=AGE + T_.

COXREG SURVIVAL WITH Z
  /STATUS SURVSTA EVENT(1).
```

Overview

COXREG applies Cox proportional hazards regression to analysis of survival times—that is, the length of time before the occurrence of an event. COXREG supports continuous and categorical independent variables (covariates), which can be time dependent. Unlike SURVIVAL and KM, which compare only distinct subgroups of cases, COXREG provides an easy way of considering differences in subgroups as well as analyzing effects of a set of covariates.

Options

Processing of Independent Variables. You can specify which of the independent variables are categorical with the CATEGORICAL subcommand and control treatment of these variables with the CONTRAST subcommand. You can select one of seven methods for entering independent variables into the model using the METHOD subcommand. You can also indicate interaction terms using the keyword BY between variable names on either the VARIABLES subcommand or the METHOD subcommand.

Specifying Termination and Model-Building Criteria. You can specify the criteria for termination of iteration and control variable entry and removal with the CRITERIA subcommand.

Adding New Variables to Active Dataset. You can use the SAVE subcommand to save the cumulative survival, standard error, cumulative hazard, log-minus-log-of-survival function, residuals, XBeta, and, wherever available, partial residuals and DfBeta.

Output. You can print optional output using the PRINT subcommand, suppress or request plots with the PLOT subcommand, and, with the OUTFILE subcommand, write data files containing coefficients from the final model or a survival table. When only time-constant covariates are used, you can use the PATTERN subcommand to specify a pattern of covariate values in addition to the covariate means to use for the plots and the survival table.

Basic Specification

- The minimum specification on COXREG is a dependent variable with the STATUS subcommand.
- To analyze the influence of time-constant covariates on the survival times, the minimum specification requires either the WITH keyword followed by at least one covariate (independent variable) on the VARIABLES subcommand or a METHOD subcommand with at least one independent variable.
- To analyze the influence of time-dependent covariates on the survival times, the TIME PROGRAM command and transformation language are required to define the functions for the time-dependent covariate(s).

Subcommand Order

- The VARIABLES subcommand must be specified first; the subcommand keyword is optional.
- Remaining subcommands can be named in any order.

Syntax Rules

- Only one dependent variable can be specified for each COXREG command.
- Any number of covariates (independent variables) can be specified. The dependent variable cannot appear on the covariate list.
- The covariate list is required if any of the METHOD subcommands are used without a variable list or if the METHOD subcommand is not used.
- Only one status variable can be specified on the STATUS subcommand. If multiple STATUS subcommands are specified, only the last specification is in effect.
- You can use the BY keyword to specify interaction between covariates.

Operations

- TIME PROGRAM computes the values for time-dependent covariates. See the topic "TIME PROGRAM" on page 1973 for more information.

- COXREG replaces covariates specified on CATEGORICAL with sets of contrast variables. In stepwise analyses, the set of contrast variables associated with one categorical variable is entered or removed from the model as a block.
- Covariates are screened to detect and eliminate redundancies.
- COXREG deletes all cases that have negative values for the dependent variable.

Limitations

- Only one dependent variable is allowed.
- Maximum 100 covariates in a single interaction term.
- Maximum 35 levels for a BY variable on PATTERN.

VARIABLES Subcommand

VARIABLES identifies the dependent variable and the covariates to be included in the analysis.

- The minimum specification is the dependent variable.
- Cases whose dependent variable values are negative are excluded from the analysis.
- You must specify the keyword WITH and a list of all covariates if no METHOD subcommand is specified or if a METHOD subcommand is specified without naming the variables to be used.
- If the covariate list is not specified on VARIABLES but one or more METHOD subcommands are used, the covariate list is assumed to be the union of the sets of variables listed on all of the METHOD subcommands.
- You can specify an interaction of two or more covariates using the keyword BY. For example, A B BY C D specifies the three terms *A*, *B*C*, and *D*.
- The keyword TO can be used to specify a list of covariates. The implied variable order is the same as in the active dataset.

STATUS Subcommand

To determine whether the event has occurred for a particular observation, COXREG checks the value of a status variable. STATUS lists the status variable and the code for the occurrence of the event.

- Only one status variable can be specified. If multiple STATUS subcommands are specified, COXREG uses the last specification and displays a warning.
- The keyword EVENT is optional, but the value list in parentheses must be specified.
- The value list must be enclosed in parentheses. All cases with non-negative times that do not have a code within the range specified after EVENT are classified as **censored cases**—that is, cases for which the event has not yet occurred.
- The value list can be one value, a list of values separated by blanks or commas, a range of values using the keyword THRU, or a combination.
- If missing values occur within the specified ranges, they are ignored if MISSING=EXCLUDE (the default) is specified, but they are treated as valid values for the range if MISSING=INCLUDE is specified.
- The status variable can be either numeric or string. If a string variable is specified, the EVENT values must be enclosed in apostrophes and the keyword THRU cannot be used.

Example

```
COXREG VARIABLES = SURVIVAL WITH GROUP
  /STATUS SURVSTA (3 THRU 5, 8 THRU 10).
```

- STATUS specifies that *SURVSTA* is the status variable.
- A value between either 3 and 5 or 8 and 10, inclusive, means that the terminal event occurred.
- Values outside the specified ranges indicate censored cases.

STRATA Subcommand

STRATA identifies a stratification variable. A different baseline survival function is computed for each stratum.

- The only specification is the subcommand keyword with one, and only one, variable name.
- If you have more than one stratification variable, create a new variable that corresponds to the combination of categories of the individual variables before invoking the COXREG command.
- There is no limit to the number of levels for the strata variable.

Example

```
COXREG VARIABLES = SURVIVAL WITH GROUP
 /STATUS SURVSTA (1)
 /STRATA=LOCATION.
```

- STRATA specifies *LOCATION* as the strata variable.
- Different baseline survival functions are computed for each value of *LOCATION*.

CATEGORICAL Subcommand

CATEGORICAL identifies covariates that are nominal or ordinal. Variables that are declared to be categorical are automatically transformed to a set of contrast variables (see "CONTRAST Subcommand " on page 370). If a variable coded as 0–1 is declared as categorical, by default, its coding scheme will be changed to deviation contrasts.

- Covariates not specified on CATEGORICAL are assumed to be at least interval, except for strings.
- Variables specified on CATEGORICAL but not on VARIABLES or any METHOD subcommand are ignored.
- Variables specified on CATEGORICAL are replaced by sets of contrast variables. If the categorical variable has *n* distinct values, *n*–1 contrast variables will be generated. The set of contrast variables associated with one categorical variable are entered or removed from the model together.
- If any one of the variables in an interaction term is specified on CATEGORICAL, the interaction term is replaced by contrast variables.
- All string variables are categorical. Only the first eight bytes of each value of a string variable are used in distinguishing among values. Thus, if two values of a string variable are identical for the first eight characters, the values are treated as though they were the same.

CONTRAST Subcommand

CONTRAST specifies the type of contrast used for categorical covariates. The interpretation of the regression coefficients for categorical covariates depends on the contrasts used. The default is DEVIATION. For illustration of contrast types, see the appendix.

- The categorical covariate is specified in parentheses following CONTRAST.
- If the categorical variable has *n* values, there will be *n*–1 rows in the contrast matrix. Each contrast matrix is treated as a set of independent variables in the analysis.
- Only one variable can be specified per CONTRAST subcommand, but multiple CONTRAST subcommands can be specified.
- You can specify one of the contrast keywords in parentheses following the variable specification to request a specific contrast type.

The following contrast types are available:

DEVIATION(refcat). *Deviations from the overall effect.* This is the default. The effect for each category of the independent variable except one is compared to the overall effect. *Refcat* is the category for which parameter estimates are not displayed (they must be calculated from the others). By default, *refcat* is the

last category. To omit a category other than the last, specify the sequence number of the omitted category (which is not necessarily the same as its value) in parentheses following the keyword DEVIATION.

SIMPLE(refcat). *Each category of the independent variable except the last is compared to the last category.* To use a category other than the last as the omitted reference category, specify its sequence number (which is not necessarily the same as its value) in parentheses following the keyword SIMPLE.

DIFFERENCE. *Difference or reverse Helmert contrasts.* The effects for each category of the covariate except the first are compared to the mean effect of the previous categories.

HELMERT. *Helmert contrasts.* The effects for each category of the independent variable except the last are compared to the mean effects of subsequent categories.

POLYNOMIAL(metric). *Polynomial contrasts.* The first degree of freedom contains the linear effect across the categories of the independent variable, the second contains the quadratic effect, and so on. By default, the categories are assumed to be equally spaced; unequal spacing can be specified by entering a metric consisting of one integer for each category of the independent variable in parentheses after the keyword POLYNOMIAL. For example, CONTRAST (STIMULUS) = POLYNOMIAL(1,2,4) indicates that the three levels of *STIMULUS* are actually in the proportion 1:2:4. The default metric is always (1,2,...,*k*), where *k* categories are involved. Only the relative differences between the terms of the metric matter: (1,2,4) is the same metric as (2,3,5) or (20,30,50) because, in each instance, the difference between the second and third numbers is twice the difference between the first and second.

REPEATED. *Comparison of adjacent categories.* Each category of the independent variable except the last is compared to the next category.

SPECIAL(matrix). *A user-defined contrast.* After this keyword, a matrix is entered in parentheses with *k* –1 rows and *k* columns, where *k* is the number of categories of the independent variable. The rows of the contrast matrix contain the special contrasts indicating the desired comparisons between categories. If the special contrasts are linear combinations of each other, COXREG reports the linear dependency and stops processing. If *k* rows are entered, the first row is discarded and only the last *k*–1 rows are used as the contrast matrix in the analysis.

INDICATOR(refcat). *Indicator variables.* Contrasts indicate the presence or absence of category membership. By default, refcat is the last category (represented in the contrast matrix as a row of zeros). To omit a category other than the last, specify the sequence number of the category (which is not necessarily the same as its value) in parentheses after the keyword INDICATOR.

Example

```
COXREG VARIABLES = SURVIVAL WITH GROUP
 /STATUS SURVSTA (1)
 /STRATA=LOCATION
 /CATEGORICAL = GROUP
 /CONTRAST(GROUP)=SPECIAL(2 -1 -1
                          0  1 -1).
```

- The specification of *GROUP* on CATEGORICAL replaces the variable with a set of contrast variables.
- *GROUP* identifies whether a case is in one of the three treatment groups.
- A SPECIAL type contrast is requested. A three-column, two-row contrast matrix is entered in parentheses.

METHOD Subcommand

METHOD specifies the order of processing and the manner in which the covariates enter the model. If no METHOD subcommand is specified, the default method is ENTER.

- The subcommand keyword METHOD can be omitted.
- You can list all covariates to be used for the method on a variable list. If no variable list is specified, the default is ALL; all covariates named after WITH on the VARIABLES subcommand are used for the method.
- The keyword BY can be used between two variable names to specify an interaction term.

- Variables specified on CATEGORICAL are replaced by sets of contrast variables. The contrast variables associated with a categorical variable are entered or removed from the model together.
- Three keywords are available to specify how the model is to be built:

  ENTER. *Forced entry.* All variables are entered in a single step. This is the default if the METHOD subcommand is omitted.

  FSTEP. *Forward stepwise.* The covariates specified on FSTEP are tested for entry into the model one by one based on the significance level of the score statistic. The variable with the smallest significance less than PIN is entered into the model. After each entry, variables that are already in the model are tested for possible removal based on the significance of the Wald statistic, likelihood ratio, or conditional criterion. The variable with the largest probability greater than the specified POUT value is removed and the model is reestimated. Variables in the model are then again evaluated for removal. Once no more variables satisfy the removal criteria, covariates not in the model are evaluated for entry. Model building stops when no more variables meet entry or removal criteria, or when the current model is the same as a previous one.

  BSTEP. *Backward stepwise.* As a first step, the covariates specified on BSTEP are entered into the model together and are tested for removal one by one. Stepwise removal and entry then follow the same process as described for FSTEP until no more variables meet entry and removal criteria, or when the current model is the same as a previous one.
- Multiple METHOD subcommands are allowed and are processed in the order in which they are specified. Each method starts with the results from the previous method. If BSTEP is used, all eligible variables are entered at the first step. All variables are then eligible for entry and removal unless they have been excluded from the METHOD variable list.
- The statistic used in the test for removal can be specified by an additional keyword in parentheses following FSTEP or BSTEP. If FSTEP or BSTEP is specified by itself, the default is COND.

COND. *Conditional statistic.* This is the default if FSTEP or BSTEP is specified by itself

WALD. *Wald statistic.* The removal of a covariate from the model is based on the significance of the Wald statistic.

LR. *Likelihood ratio.* The removal of a covariate from the model is based on the significance of the change in the log-likelihood. If LR is specified, the model must be reestimated without each of the variables in the model. This can substantially increase computational time. However, the likelihood-ratio statistic is better than the Wald statistic for deciding which variables are to be removed.

Example

```
COXREG VARIABLES = SURVIVAL WITH GROUP SMOKE DRINK
 /STATUS SURVSTA (1)
 /CATEGORICAL = GROUP SMOKE DRINK
 /METHOD ENTER GROUP
 /METHOD BSTEP (LR) SMOKE DRINK SMOKE BY DRINK.
```

- *GROUP*, *SMOKE*, and *DRINK* are specified as covariates and as categorical variables.
- The first METHOD subcommand enters GROUP into the model.
- Variables in the model at the termination of the first METHOD subcommand are included in the model at the beginning of the second METHOD subcommand.
- The second METHOD subcommand adds *SMOKE*, *DRINK*, and the interaction of *SMOKE* with *DRINK* to the previous model.
- Backward stepwise regression analysis is then done using the likelihood-ratio statistic as the removal criterion. The variable *GROUP* is not eligible for removal because it was not specified on the BSTEP subcommand.
- The procedure continues until the removal of a variable will result in a decrease in the log-likelihood with a probability smaller than POUT.

MISSING Subcommand

MISSING controls missing value treatments. If MISSING is omitted, the default is EXCLUDE.

- Cases with negative values on the dependent variable are automatically treated as missing and are excluded.
- To be included in the model, a case must have nonmissing values for the dependent, status, strata, and all independent variables specified on the COXREG command.

EXCLUDE. *Exclude user-missing values.* User-missing values are treated as missing. This is the default if MISSING is omitted.

INCLUDE. *Include user-missing values.* User-missing values are included in the analysis.

PRINT Subcommand

By default, COXREG prints a full regression report for each step. You can use the PRINT subcommand to request specific output. If PRINT is not specified, the default is DEFAULT.

DEFAULT. *Full regression output including overall model statistics and statistics for variables in the equation and variables not in the equation.* This is the default when PRINT is omitted.

SUMMARY. *Summary information.* The output includes –2 log-likelihood for the initial model, one line of summary for each step, and the final model printed with full detail.

CORR. *Correlation/covariance matrix of parameter estimates for the variables in the model.*

BASELINE. *Baseline table.* For each stratum, a table is displayed showing the baseline cumulative hazard, as well as survival, standard error, and cumulative hazard evaluated at the covariate means for each observed time point in that stratum.

CI (value). *Confidence intervals for* e^{β}. Specify the confidence level in parentheses. The requested intervals are displayed whenever a variables-in-equation table is printed. The default is 95%.

ALL. *All available output.*

Estimation histories showing the last 10 iterations are printed if the solution fails to converge.

Example

```
COXREG VARIABLES = SURVIVAL WITH GROUP
 /STATUS = SURVSTA (1)
 /STRATA = LOCATION
 /CATEGORICAL = GROUP
 /METHOD = ENTER
 /PRINT ALL.
```

PRINT requests summary information, a correlation matrix for parameter estimates, a baseline survival table for each stratum, and confidence intervals for e^{β} with each variables-in-equation table, in addition to the default output.

CRITERIA Subcommand

CRITERIA controls the statistical criteria used in building the Cox Regression models. The way in which these criteria are used depends on the method specified on the METHOD subcommand. The default criteria are noted in the description of each keyword below. Iterations will stop if any of the criteria for BCON, LCON, or ITERATE are satisfied.

BCON(value). *Change in parameter estimates for terminating iteration.* Alias PCON. Iteration terminates when the parameters change by less than the specified value. BCON defaults to 1E–4. To eliminate this criterion, specify a value of 0.

ITERATE(value). *Maximum number of iterations.* If a solution fails to converge after the maximum number of iterations has been reached, COXREG displays an iteration history showing the last 10 iterations and terminates the procedure. The default for ITERATE is 20.

LCON(value). *Percentage change in the log-likelihood ratio for terminating iteration.* If the log-likelihood decreases by less than the specified value, iteration terminates. LCON defaults to 1E–5. To eliminate this criterion, specify a value of 0.

PIN(value). *Probability of score statistic for variable entry.* A variable whose significance level is greater than PIN cannot enter the model. The default for PIN is 0.05.

POUT(value). *Probability of Wald, LR, or conditional LR statistic to remove a variable.* A variable whose significance is less than POUT cannot be removed. The default for POUT is 0.1.

Example

```
COXREG VARIABLES = SURVIVAL WITH GROUP AGE BP TMRSZ
 /STATUS = SURVSTA (1)
 /STRATA = LOCATION
 /CATEGORICAL = GROUP
 /METHOD BSTEP
 /CRITERIA BCON(0) ITERATE(10) PIN(0.01) POUT(0.05).
```

- A backward stepwise Cox Regression analysis is performed.
- CRITERIA alters four of the default statistical criteria that control the building of a model.
- Zero specified on BCON indicates that change in parameter estimates is not a criterion for termination. BCON can be set to 0 if only LCON and ITER are to be used.
- ITERATE specifies that the maximum number of iterations is 10. LCON is not changed and the default remains in effect. If either ITERATE or LCON is met, iterations will terminate.
- POUT requires that the probability of the statistic used to test whether a variable should remain in the model be smaller than 0.05. This is more stringent than the default value of 0.1.
- PIN requires that the probability of the score statistic used to test whether a variable should be included be smaller than 0.01. This makes it more difficult for variables to be included in the model than does the default PIN, which has a value of 0.05.

PLOT Subcommand

You can request specific plots to be produced with the PLOT subcommand. Each requested plot is produced once for each pattern specified on the PATTERN subcommand. If PLOT is not specified, the default is NONE (no plots are printed). Requested plots are displayed at the end of the final model.

- The set of plots requested is displayed for the functions at the mean of the covariates and at each combination of covariate values specified on PATTERN.
- If time-dependent covariates are included in the model, no plots are produced.
- Lines on a plot are connected as step functions.

NONE. *Do not display plots.*

SURVIVAL. *Plot the cumulative survival distribution.*

HAZARD. *Plot the cumulative hazard function.*

LML. *Plot the log-minus-log-of-survival function.*

OMS. *Plot the one-minus-survival function.*

PATTERN Subcommand

PATTERN specifies the pattern of covariate values to be used for the requested plots and coefficient tables.

- A value must be specified for each variable specified on PATTERN.

- Continuous variables that are included in the model but not named on PATTERN are evaluated at their means.
- Categorical variables that are included in the model but not named on PATTERN are evaluated at the means of the set of contrasts generated to replace them.
- You can request separate lines for each category of a variable that is in the model. Specify the name of the categorical variable after the keyword BY. The BY variable must be a categorical covariate. You cannot specify a value for the BY covariate.
- Multiple PATTERN subcommands can be specified. COXREG produces a set of requested plots for each specified pattern.
- PATTERN cannot be used when time-dependent covariates are included in the model.

OUTFILE Subcommand

OUTFILE writes data to an external IBM SPSS Statistics data file or a previously declared dataset (DATASET DECLARE command). COXREG writes two types of data files. You can specify the file type to be created with one of the two keywords, followed by a quoted file specification in parentheses. It also saves model information in XML format.

COEFF('savfile' | 'dataset'). *Write a data file containing the coefficients from the final model.*

TABLE('savfile' | 'dataset'). *Write the survival table to a data file.* The file contains cumulative survival, standard error, and cumulative hazard statistics for each uncensored time within each stratum evaluated at the baseline and at the mean of the covariates. Additional covariate patterns can be requested on PATTERN.

PARAMETER('file'). *Write parameter estimates only to an XML file.* You can use this model file to apply the model information to other data files for scoring purposes. See the topic "Scoring expressions" on page 86 for more information.

SAVE Subcommand

SAVE saves the temporary variables created by COXREG. The temporary variables include:

SURVIVAL. *Survival function evaluated at the current case.*

SE. *Standard error of the survival function.*

HAZARD. *Cumulative hazard function evaluated at the current case.* Alias RESID.

LML. *Log-minus-log-of-survival function.*

DFBETA. *Change in the coefficient if the current case is removed.* There is one *DFBETA* for each covariate in the final model. If there are time-dependent covariates, only *DFBETA* can be requested. Requests for any other temporary variable are ignored.

PRESID. *Partial residuals.* There is one residual variable for each covariate in the final model. If a covariate is not in the final model, the corresponding new variable has the system-missing value.

XBETA. *Linear combination of mean corrected covariates times regression coefficients from the final model.*

- To specify variable names for the new variables, assign the new names in parentheses following each temporary variable name.
- Assigned variable names must be unique in the active dataset. Scratch or system variable names cannot be used (that is, the variable names cannot begin with # or $).
- If new variable names are not specified, COXREG generates default names. The default name is composed of the first three characters of the name of the temporary variable (two for *SE*), followed by an underscore and a number to make it unique.
- A temporary variable can be saved only once on the same SAVE subcommand.

Example

```
COXREG VARIABLES = SURVIVAL WITH GROUP
 /STATUS = SURVSTA (1)
 /STRATA = LOCATION
 /CATEGORICAL = GROUP
 /METHOD = ENTER
 /SAVE SURVIVAL HAZARD.
```

COXREG saves cumulative survival and hazard in two new variables, *SUR_1* and *HAZ_1*, provided that neither of the two names exists in the active dataset. If one does, the numeric suffixes will be incremented to make a distinction.

EXTERNAL Subcommand

EXTERNAL specifies that the data for each split-file group should be held in an external scratch file during processing. This helps conserve working space when running analyses with large datasets.

- The EXTERNAL subcommand takes no other keyword and is specified by itself.
- If time-dependent covariates exist, external data storage is unavailable, and EXTERNAL is ignored.

CREATE

```
CREATE new series={CSUM (series)                        }
                  {DIFF (series, order)                 }
                  {FFT (series)                         }
                  {IFFT (series)                        }
                  {LAG (series, order [,order ])        }
                  {LEAD (series, order [,order ])       }
                  {MA (series, span [,minimum span])    }
                  {PMA (series, span)                   }
                  {RMED (series, span [,minimum span]) }
                  {SDIFF (series, order [,periodicity])}
                  {T4253H (series)                      }

[/new series=function (series {,span  {,minimum span}})]
                              {,order {,order       }}
                                      {,periodicity }
```

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Function keywords:

CSUM. Cumulative sum

DIFF. Difference

FFT. Fast Fourier transform

IFFT. Inverse fast Fourier transform

LAG. Lag

LEAD. Lead

MA. Centered moving averages

PMA. Prior moving averages

RMED. Running medians

SDIFF. Seasonal difference

T4253H. Smoothing

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
CREATE NEWVAR1 NEWVAR2 = CSUM(TICKETS RNDTRP).
```

Overview

CREATE produces new series as a function of existing series. You can also use CREATE to replace the values of existing series.

CREATE displays a list of the new series, the case numbers of the first and last nonmissing cases, the number of valid cases, and the functions used to create the variables.

Basic specification

The basic specification is a new series name, an equals sign, a function, and the existing series, along with any additional specifications needed.

Syntax rules

- The existing series together with any additional specifications (order, span, or periodicity) must be enclosed in parentheses.
- The equals sign is required.
- Series names and additional specifications must be separated by commas or spaces.
- You can specify only one function per equation.
- You can create more than one new series per equation by specifying more than one new series name on the left side of the equation and either multiple existing series names or multiple orders on the right.
- The number of new series named on the left side of the equation must equal the number of series created on the right. Note that the FFT function creates two new series for each existing series, and IFFT creates one series from two existing series.
- You can specify more than one equation on a CREATE command. Equations are separated by slashes.
- A newly created series can be specified in subsequent equations on the same CREATE command.

Operations

- Each new series created is added to the active dataset.
- If the new series named already exist, their values are replaced.
- If the new series named do not already exist, they are created.
- Series are created in the order in which they are specified on the CREATE command.
- If multiple series are created by a single equation, the first new series named is assigned the values of the first series created, the second series named is assigned the values of the second series created, and so on.
- CREATE automatically generates a variable label for each new series describing the function and series used to create it.
- The format of the new series is based on the function specified and the format of the existing series.
- CREATE honors the TSET MISSING setting that is currently in effect.
- If split file processing is on, the scope is limited to each split group. A new value cannot be created from a case in a preceding or subsequent split group.
- CREATE does not honor the USE command.
- When an even-length span is specified for the functions MA and RMED, the centering algorithm uses an average of two spans of the specified length. The first span ranges from span/2 cases before the current observation to the span length. The second span ranges from (span/2)–1 cases before the current observation to the span length.

Limitations

- A maximum of 1 function per equation.
- There is no limit on the number of series created by an equation.
- There is no limit on the number of equations.

Examples

```
CREATE NEWVAR1 = DIFF(OLDVAR,1).
```

- In this example, the series *NEWVAR1* is created by taking the first-order difference of *OLDVAR*.

CSUM Function

CSUM produces new series based on the cumulative sums of the existing series. Cumulative sums are the inverse of first-order differencing.

- The only specification on CSUM is the name or names of the existing series in parentheses.
- Cases with missing values in the existing series are not used to compute values for the new series. The values of these cases are system-missing in the new series.

Example

```
CREATE NEWVAR1 NEWVAR2 = CSUM(TICKETS RNDTRP).
```

- This example produces a new series called *NEWVAR1*, which is the cumulative sum of the series *TICKETS*, and a new series called *NEWVAR2*, which is the cumulative sum of the series *RNDTRP*.

DIFF Function

DIFF produces new series based on nonseasonal differences of existing series.

- The specification on DIFF is the name or names of the existing series and the degree of differencing, in parentheses.
- The degree of differencing must be specified; there is no default.
- Since one observation is lost for each order of differencing, system-missing values will appear at the beginning of the new series.
- You can specify only one degree of differencing per DIFF function.
- If either of the pair of values involved in a difference computation is missing, the result is set to system-missing in the new series.

Example

```
CREATE ADIF2 = DIFF(VARA,2) /
       YDIF1 ZDIF1 = DIFF(VARY VARZ,1).
```

- The series *ADIF2* is created by differencing *VARA* twice.
- The series *YDIF1* is created by differencing *VARY* once.
- The series *ZDIF1* is created by differencing *VARZ* once.

FFT Function

FFT produces new series based on fast Fourier transformations of existing series [15].

- The only specification on FFT is the name or names of the existing series in parentheses.
- FFT creates two series, the cosine and sine parts (also called real and imaginary parts), for each existing series named. Thus, you must specify two new series names on the left side of the equation for each existing series specified on the right side.
- The first new series named becomes the real series, and the second new series named becomes the imaginary series.
- The existing series cannot have embedded missing values.
- The existing series must be of even length. If an odd-length series is specified, FFT pads it with a 0 to make it even. Alternatively, you can make the series even by adding or dropping an observation.

[15] Brigham, E. O. 1974. *The fast Fourier transform*. Englewood Cliffs, N.J.: Prentice-Hall.

- The new series will be only half as long as the existing series. The remaining cases are assigned the system-missing value.

Example

```
CREATE A B = FFT(C).
```

- Two series, *A* (real) and *B* (imaginary), are created by applying a fast Fourier transformation to series *C*.

IFFT Function

IFFT produces new series based on the inverse Fourier transformation of existing series.

- The only specification on IFFT is the name or names of the existing series in parentheses.
- IFFT needs two existing series to compute each new series. Thus, you must specify two existing series names on the right side of the equation for each new series specified on the left.
- The first existing series specified is the real series and the second series is the imaginary series.
- The existing series cannot have embedded missing values.
- The new series will be twice as long as the existing series. Thus, the last half of each existing series must be system-missing to allow enough room to create the new series.

Example

```
CREATE C = IFFT(A B).
```

- This command creates one new series, *C*, from the series *A* (real) and *B* (imaginary).

LAG Function

LAG creates new series by copying the values of the existing series and moving them forward the specified number of observations. This number is called the **lag order**. The table below shows a first-order lag for a hypothetical dataset.

- The specification on LAG is the name or names of the existing series and one or two lag orders, in parentheses.
- At least one lag order must be specified; there is no default.
- Two lag orders indicate a range. For example, 2,6 indicates lag orders two through six. A new series is created for each lag order in the range.
- The number of new series specified must equal the number of existing series specified times the number of lag orders in the range.
- The first *n* cases at the beginning of the new series, where *n* is the lag order, are assigned the system-missing value.
- Missing values in the existing series are lagged and are assigned the system-missing value in the new series.
- A first-order lagged series can also be created using COMPUTE. COMPUTE does not cause a data pass (see COMPUTE).

Table 25. First-order lag and lead of series X

X	Lag	Lead
198	.	220
220	198	305
305	220	470
470	305	.

Example

```
CREATE LAGVAR2 TO LAGVAR5 = LAG(VARA,2,5).
```

- Four new variables are created based on lags on *VARA*. *LAGVAR2* is *VARA* lagged two steps, *LAGVAR3* is *VARA* lagged three steps, *LAGVAR4* is *VARA* lagged four steps, and *LAGVAR5* is *VARA* lagged five steps.

LEAD Function

LEAD creates new series by copying the values of the existing series and moving them back the specified number of observations. This number is called the lead order.

- The specification on LEAD is the name or names of the existing series and one or two lead orders, in parentheses.
- At least one lead order must be specified; there is no default.
- Two lead orders indicate a range. For example, 1,5 indicates lead orders one through five. A new series is created for each lead order in the range.
- The number of new series must equal the number of existing series specified times the number of lead orders in the range.
- The last *n* cases at the end of the new series, where *n* equals the lead order, are assigned the system-missing value.
- Missing values in the existing series are moved back and are assigned the system-missing value in the new series.

Example

```
CREATE LEAD1 TO LEAD4 = LEAD(VARA,1,4).
```

- Four new series are created based on leads of *VARA*. *LEAD1* is *VARA* led one step, *LEAD2* is *VARA* led two steps, *LEAD3* is *VARA* led three steps, and *LEAD4* is *VARA* led four steps.

MA Function

MA produces new series based on the centered moving averages of existing series.

- The specification on MA is the name or names of the existing series and the span to be used in averaging, in parentheses.
- A span must be specified; there is no default.
- If the specified span is odd, the MA is naturally associated with the middle term. If the specified span is even, the MA is centered by averaging each pair of uncentered means [16].
- After the initial span, a second span can be specified to indicate the minimum number of values to use in averaging when the number specified for the initial span is unavailable. This makes it possible to produce nonmissing values at or near the ends of the new series.
- The second span must be greater than or equal to 1 and less than or equal to the first span.
- The second span should be even (or 1) if the first span is even; it should be odd if the first span is odd. Otherwise, the next higher span value will be used.
- If no second span is specified, the minimum span is simply the value of the first span.
- If the number of values specified for the span or the minimum span is not available, the case in the new series is set to system-missing. Thus, unless a minimum span of 1 is specified, the endpoints of the new series will contain system-missing values.

[16] Velleman, P. F., and D. C. Hoaglin. 1981. *Applications, basics, and computing of exploratory data analysis*. Boston, Mass.: Duxbury Press.

- When MA encounters an embedded missing value in the existing series, it creates two subsets, one containing cases before the missing value and one containing cases after the missing value. Each subset is treated as a separate series for computational purposes.
- The endpoints of these subset series will have missing values according to the rules described above for the endpoints of the entire series. Thus, if the minimum span is 1, the endpoints of the subsets will be nonmissing; the only cases that will be missing in the new series are cases that were missing in the original series.

Example

```
CREATE TICKMA = MA(TICKETS,4,2).
```

- This example creates the series *TICKMA* based on centered moving average values of the series *TICKETS*.
- A span of 4 is used for computing averages. At the endpoints, where four values are not available, the average is based on the specified minimum of two values.

PMA Function

PMA creates new series based on the prior moving averages of existing series. The prior moving average for each case in the original series is computed by averaging the values of a span of cases preceding it.

- The specification on PMA is the name or names of the existing series and the span to be used, in parentheses.
- Only one span can be specified and it is required. There is no default span.
- If the number of values specified for the span is not available, the case is set to system-missing. Thus, the number of cases with system-missing values at the beginning of the new series equals the number specified for the span.
- When PMA encounters an imbedded missing value in the existing series, it creates two subsets, one containing cases before the missing value and one containing cases after the missing value. Each subset is treated as a separate series for computational purposes. The first *n* cases in the second subset will be system-missing, where *n* is the span.

Example

```
CREATE PRIORA = PMA(VARA,3).
```

- This command creates the series *PRIORA* by computing prior moving averages for the series *VARA*. Since the span is 3, the first three cases in the series *PRIORA* are system-missing. The fourth case equals the average of cases 1, 2, and 3 of *VARA*, the fifth case equals the average of cases 2, 3, and 4 of *VARA*, and so on.

RMED Function

RMED produces new series based on the centered running medians of existing series.

- The specification on RMED is the name or names of the existing series and the span to be used in finding the median, in parentheses.
- A span must be specified; there is no default.
- If the specified span is odd, RMED is naturally the middle term. If the specified span is even, the RMED is centered by averaging each pair of uncentered medians [17].

[17] Velleman, P. F., and D. C. Hoaglin. 1981. *Applications, basics, and computing of exploratory data analysis*. Boston, Mass.: Duxbury Press.

- After the initial span, a second span can be specified to indicate the minimum number of values to use in finding the median when the number specified for the initial span is unavailable. This makes it possible to produce nonmissing values at or near the ends of the new series.
- The second span must be greater than or equal to 1 and less than or equal to the first span.
- The second span should be even (or 1) if the first span is even; it should be odd if the first span is odd. Otherwise, the next higher span value will be used.
- If no second span is specified, the minimum span is simply the value of the first span.
- If the number of values specified for the span or the minimum span is not available, the case in the new series is set to system-missing. Thus, unless a minimum span of 1 is specified, the endpoints of the new series will contain system-missing values.
- When RMED encounters an imbedded missing value in the existing series, it creates two subsets, one containing cases before the missing value and one containing cases after the missing value. Each subset is treated as a separate series for computational purposes.
- The endpoints of these subset series will have missing values according to the rules described above for the endpoints of the entire series. Thus, if the minimum span is 1, the endpoints of the subsets will be nonmissing; the only cases that will be missing in the new series are cases that were missing in the original series.

Example

```
CREATE TICKRMED = RMED(TICKETS,4,2).
```

- This example creates the series *TICKRMED* using centered running median values of the series *TICKETS*.
- A span of 4 is used for computing medians. At the endpoints, where four values are not available, the median is based on the specified minimum of two values.

SDIFF Function

SDIFF produces new series based on seasonal differences of existing series.

- The specification on SDIFF is the name or names of the existing series, the degree of differencing, and, optionally, the periodicity, all in parentheses.
- The degree of differencing must be specified; there is no default.
- Since the number of seasons used in the calculations decreases by 1 for each order of differencing, system-missing values will appear at the beginning of the new series.
- You can specify only one degree of differencing per SDIFF function.
- If no periodicity is specified, the periodicity established on TSET PERIOD is in effect. If TSET PERIOD has not been specified, the periodicity established on the DATE command is used. If periodicity was not established anywhere, the SDIFF function cannot be executed.
- If either of the pair of values involved in a seasonal difference computation is missing, the result is set to system-missing in the new series.

Example

```
CREATE SDVAR = SDIFF(VARA,1,12).
```

- The series *SDVAR* is created by applying one seasonal difference with a periodicity of 12 to the series *VARA*.

T4253H Function

T4253H produces new series by applying a compound data smoother to the original series. The smoother starts with a running median of 4, which is centered by a running median of 2. It then resmooths these

values by applying a running median of 5, a running median of 3, and hanning (running weighted averages). Residuals are computed by subtracting the smoothed series from the original series. This whole process is then repeated on the computed residuals. Finally, the smoothed residuals are added to the smoothed values obtained the first time through the process [18].

- The only specification on T4253H is the name or names of the existing series in parentheses.
- The existing series cannot contain imbedded missing values.
- Endpoints are smoothed through extrapolation and are not system-missing.

Example

```
CREATE SMOOTHA = T4253H(VARA).
```

- The series *SMOOTHA* is a smoothed version of the series *VARA*.

References

Box, G. E. P., and G. M. Jenkins. 1976. *Time series analysis: Forecasting and control*, Rev. ed. San Francisco: Holden-Day.

Brigham, E. O. 1974. *The fast Fourier transform*. Englewood Cliffs, N.J.: Prentice-Hall.

Cryer, J. D. 1986. *Time series analysis*. Boston, Mass.: Duxbury Press.

Makridakis, S. G., S. C. Wheelwright, and R. J. Hyndman. 1997. *Forecasting: Methods and applications*, 3rd ed. ed. New York: John Wiley and Sons.

Monro, D. M. 1975. Algorithm AS 83: Complex discrete fast Fourier transform. *Applied Statistics,* 24, 153-160.

Monro, D. M., and J. L. Branch. 1977. Algorithm AS 117: The Chirp discrete Fourier transform of general length. *Applied Statistics,* 26, 351-361.

Velleman, P. F., and D. C. Hoaglin. 1981. *Applications, basics, and computing of exploratory data analysis*. Boston, Mass.: Duxbury Press.

[18] Velleman, P. F., and D. C. Hoaglin. 1981. *Applications, basics, and computing of exploratory data analysis*. Boston, Mass.: Duxbury Press.

CROSSTABS

CROSSTABS is available in Statistics Base Edition.

General mode:

```
CROSSTABS [TABLES=]varlist BY varlist [BY...] [/varlist...]

 [/MISSING={TABLE**}]
           {INCLUDE}

 [/WRITE[={NONE**}]]
          {CELLS }

 [/HIDESMALLCOUNTS [COUNT = {5      }] ]
                            {integer}

 [/SHOWDIM = integer]

 [/CELLS = [PROP] [BPROP]
```

Integer mode :

```
CROSSTABS VARIABLES=varlist(min,max) [varlist...]

 /TABLES=varlist BY varlist [BY...] [/varlist...]

 [/MISSING={TABLE**}]
           {INCLUDE}
           {REPORT }

 [/WRITE[={NONE**}]]
          {CELLS }
          {ALL   }
```

Both modes:

```
 [/FORMAT= {AVALUE**}  {TABLES**}]
           {DVALUE  }  {NOTABLES}

 [/COUNT = [{CELL}] [{ROUND }]
            {CASE  }   {TRUNCATE}
            {ASIS  }

 [/CELLS=[{COUNT**}]  [ROW    ]  [EXPECTED]  [SRESID ]]
          {NONE   }   [COLUMN]  [RESID   ]  [ASRESID]
                      [TOTAL ]              [ALL     ]

 [/STATISTICS=[CHISQ]  [LAMBDA]  [BTAU ] [GAMMA  ]  [ETA    ]]
              [PHI  ]  [UC    ]  [CTAU ] [D      ]  [CORR   ]
              [CC   ]  [RISK  ]  [KAPPA] [MCNEMAR]  [CMH(1*)]
              [ALL  ]  [NONE  ]

 [/METHOD={MC [CIN({99.0 })] [SAMPLES({10000})]}]††
                  {value}           {value}
          {EXACT [TIMER({5    })]             }
                        {value}

 [/BARCHART]
```

**Default if the subcommand is omitted.

†† The METHOD subcommand is available only if SPSS Statistics Premium Edition or the Exact Tests option is installed (available only on Windows operating systems).

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 19.0

- HIDESMALLCOUNTS subcommand introduced.
- SHOWDIM subcommand introduced.
- PROP and BPROP keywords introduced on the CELLS subcommand.

Example

```
CROSSTABS TABLES=FEAR BY SEX
 /CELLS=ROW COLUMN EXPECTED RESIDUALS
 /STATISTICS=CHISQ.
```

Overview

CROSSTABS produces contingency tables showing the joint distribution of two or more variables that have a limited number of distinct values. The frequency distribution of one variable is subdivided according to the values of one or more variables. The unique combination of values for two or more variables defines a cell.

CROSSTABS can operate in two different modes: *general* and *integer*. Integer mode builds some tables more efficiently but requires more specifications than general mode. Some subcommand specifications and statistics are available only in integer mode.

Options

Methods for building tables. To build tables in general mode, use the TABLES subcommand. Integer mode requires the TABLES and VARIABLES subcommands and minimum and maximum values for the variables.

Cell contents. By default, CROSSTABS displays only the number of cases in each cell. You can request row, column, and total percentages, and also expected values and residuals, by using the CELLS subcommand.

Statistics. In addition to the tables, you can obtain measures of association and tests of hypotheses for each subtable using the STATISTICS subcommand.

Formatting options. With the FORMAT subcommand, you can control the display order for categories in rows and columns of subtables and suppress crosstabulation. With the SHOWDIM subcommand you can display a subset of the variables as table layers in the crosstabulation table.

Writing and reproducing tables. You can write cell frequencies to a file and reproduce the original tables with the WRITE subcommand.

Basic specification

In general mode, the basic specification is TABLES with a table list. The actual keyword TABLES can be omitted. In integer mode, the minimum specification is the VARIABLES subcommand, specifying the variables to be used and their value ranges, and the TABLES subcommand with a table list.

- The minimum table list specifies a list of row variables, the keyword BY, and a list of column variables.
- In integer mode, all variables must be numeric with integer values. In general mode, variables can be numeric (integer or non-integer) or string.
- The default table shows cell counts.

Subcommand order

- In general mode, the table list must be first if the keyword TABLES is omitted. If the keyword TABLES is explicitly used, subcommands can be specified in any order.
- In integer mode, VARIABLES must precede TABLES. The keyword TABLES must be explicitly specified.

Operations

- Integer mode builds tables more quickly but requires more workspace if a table has many empty cells.
- In integer mode, the PROP and BPROP keywords on the CELLS command are ignored. If no other cell contents are requested, no table will be produced.
- Statistics are calculated separately for each two-way table or two-way subtable. Missing values are reported for the table as a whole.
- In general mode, the keyword TO on the TABLES subcommand refers to the order of variables in the active dataset. ALL refers to all variables in the active dataset. In integer mode, TO and ALL refer to the position and subset of variables specified on the VARIABLES subcommand.

Limitations

The following limitations apply to CROSSTABS in *general mode*:

- A maximum of 200 variables named or implied on the TABLES subcommand
- A maximum of 1000 non-empty rows or columns for each table
- A maximum of 20 table lists per CROSSTABS command
- A maximum of 10 dimensions (9 BY keywords) per table
- A maximum of 400 value labels displayed on any single table

The following limitations apply to CROSSTABS in *integer mode*:

- A maximum of 100 variables named or implied on the VARIABLES subcommand
- A maximum of 100 variables named or implied on the TABLES subcommand
- A maximum of 1000 non-empty rows or columns for each table
- A maximum of 20 table lists per CROSSTABS command
- A maximum of 8 dimensions (7 BY keywords) per table
- A maximum of 20 rows or columns of missing values when REPORT is specified on MISSING
- The minimum value that can be specified is –99,999
- The maximum value that can be specified is 999,999

Examples

Nominal by nominal relationships

```
CROSSTABS
  /TABLES=store  BY service
  /FORMAT= AVALUE TABLES
  /STATISTIC=CHISQ CC PHI LAMBDA UC
  /CELLS= COUNT .
```

Ordinal by ordinal relationships

```
CROSSTABS
  /TABLES=regular  BY overall
  /FORMAT= AVALUE TABLES
  /STATISTIC=D BTAU CTAU GAMMA
  /CELLS= COUNT .
```

VARIABLES subcommand

The VARIABLES subcommand is required for integer mode. VARIABLES specifies a list of variables to be used in the crosstabulations and the lowest and highest values for each variable. Values are specified in parentheses and must be integers. Non-integer values are truncated.

- Variables can be specified in any order. However, the order in which they are named on VARIABLES determines their implied order on TABLES (see the TABLES subcommand below).
- A range must be specified for each variable. If several variables can have the same range, it can be specified once after the last variable to which it applies.
- CROSSTABS uses the specified ranges to allocate tables. One cell is allocated for each possible combination of values of the row and column variables before the data are read. Thus, if the specified ranges are larger than the actual ranges, workspace will be wasted.
- Cases with values outside the specified range are considered missing and are not used in the computation of the table. This allows you to select a subset of values within CROSSTABS.
- If the table is sparse because the variables do not have values throughout the specified range, consider using general mode or recoding the variables.

Example

```
CROSSTABS VARIABLES=FEAR SEX RACE (1,2) MOBILE16 (1,3)
   /TABLES=FEAR BY SEX MOBILE16 BY RACE.
```

- VARIABLES defines values 1 and 2 for *FEAR*, *SEX*, and *RACE* and values 1, 2, and 3 for *MOBILE16*.

TABLES subcommand

TABLES specifies the table lists and is required in both integer mode and general mode. The following rules apply to both modes:

- You can specify multiple TABLES subcommands on a single CROSSTABS command. The slash between the subcommands is required; the keyword TABLES is required only in integer mode.
- Variables named before the first BY on a table list are row variables, and variables named after the first BY on a table list are column variables.
- When the table list specifies two dimensions (one BY keyword), the first variable before BY is crosstabulated with each variable after BY, then the second variable before BY with each variable after BY, and so on.
- Each subsequent use of the keyword BY on a table list adds a new dimension to the tables requested. Variables named after the second (or subsequent) BY are control variables.
- When the table list specifies more than two dimensions, a two-way subtable is produced for each combination of values of control variables. The value of the last specified control variable changes the most slowly in determining the order in which tables are displayed.
- You can name more than one variable in each dimension.

General mode

- The actual keyword TABLES can be omitted in general mode.
- In general mode, both numeric and string variables can be specified.
- The keywords ALL and TO can be specified in any dimension. In general mode, TO refers to the order of variables in the active dataset and ALL refers to all variables defined in the active dataset.

Example

```
CROSSTABS  TABLES=FEAR BY SEX BY RACE.
```

- This example crosstabulates *FEAR* by *SEX* controlling for *RACE*. In each subtable, *FEAR* is the row variable and *SEX* is the column variable.
- A subtable is produced for each value of the control variable *RACE*.

Example

```
CROSSTABS  TABLES=CONFINAN TO CONARMY BY SEX TO REGION.
```

- This command produces crosstabulations of all variables in the active dataset between and including *CONFINAN* and *CONARMY* by all variables between and including *SEX* and *REGION*.

Integer mode

- In integer mode, variables specified on TABLES must first be named on VARIABLES.
- The keywords TO and ALL can be specified in any dimension. In integer mode, TO and ALL refer to the position and subset of variables specified on the VARIABLES subcommand, not to the variables in the active dataset.

Example

```
CROSSTABS  VARIABLES=FEAR (1,2) MOBILE16 (1,3)
  /TABLES=FEAR BY MOBILE16.
```

- VARIABLES names two variables, *FEAR* and *MOBILE16*. Values 1 and 2 for *FEAR* are used in the tables, and values 1, 2, and 3 are used for the variable *MOBILE16*.
- TABLES specifies a Crosstabulation table with two rows (values 1 and 2 for *FEAR*) and three columns (values 1, 2, and 3 for *MOBILE16*). *FEAR* and *MOBILE16* can be named on TABLES because they were named on the previous VARIABLES subcommand.

Example

```
CROSSTABS  VARIABLES=FEAR SEX RACE DEGREE (1,2)
  /TABLES=FEAR BY SEX BY RACE BY DEGREE.
```

- This command produces four subtables. The first subtable crosstabulates *FEAR* by *SEX*, controlling for the first value of *RACE* and the first value of *DEGREE*; the second subtable controls for the second value of *RACE* and the first value of *DEGREE*; the third subtable controls for the first value of *RACE* and the second value of *DEGREE*; and the fourth subtable controls for the second value of *RACE* and the second value of *DEGREE*.

CELLS subcommand

By default, CROSSTABS displays only the number of cases in each cell of the Crosstabulation table. Use CELLS to display row, column, or total percentages, expected counts, or residuals. These are calculated separately for each Crosstabulation table or subtable.

- CELLS specified without keywords displays cell counts plus row, column, and total percentages for each cell.
- If CELLS is specified with keywords, CROSSTABS displays only the requested cell information.
- Scientific notation is used for cell contents when necessary.
- BPROP overrides PROP if both are specified.
- If BPROP or PROP is specified without specifying COUNT or COLUMN, then the observed cell counts are included in the Crosstabulation table, with APA-style subscripts indicating the results of the column proportions tests.
- In integer mode, the PROP and BPROP keywords on the CELLS command are ignored. If no other cell contents are requested, no table will be produced.

COUNT. *Observed cell counts.* This is the default if CELLS is omitted.

ROW. *Row percentages.* The number of cases in each cell in a row is expressed as a percentage of all cases in that row.

COLUMN. *Column percentages.* The number of cases in each cell in a column is expressed as a percentage of all cases in that column.

TOTAL. *Two-way table total percentages.* The number of cases in each cell of a subtable is expressed as a percentage of all cases in that subtable.

EXPECTED. *Expected counts.* Expected counts are the number of cases expected in each cell if the two variables in the subtable are statistically independent.

RESID. *Residuals.* Residuals are the difference between the observed and expected cell counts.

SRESID. *Standardized residuals* [19].

ASRESID. *Adjusted standardized residuals* (Haberman, 1978).

[19] Haberman, S. J. 1978. *Analysis of qualitative data*. London: Academic Press.

ALL. *All cell information.* This includes cell counts; row, column, and total percentages; expected counts; residuals; standardized residuals; adjusted standardized residuals; and pairwise comparison of column proportions using the Bonferroni correction.

NONE. *No cell information.* Use NONE when you want to write tables to a procedure output file without displaying them. See the topic "WRITE subcommand " on page 393 for more information. This is the same as specifying NOTABLES on FORMAT.

PROP. *Pairwise comparison of column proportions.* Indicates which pairs of columns (for a given row) are significantly different. Significant differences (at the 0.05 level) are indicated with APA-style formatting using subscript letters. PROP is only available in general mode.

BPROP. *Pairwise comparison of column proportions using the Bonferroni correction.* Indicates which pairs of columns (for a given row) are significantly different, making use of the Bonferroni correction. Significant differences (at the 0.05 level) are indicated with APA-style formatting using subscript letters. BPROP is only available in general mode.

Note: The **BPROP** keyword takes precedence when both the **PROP** and **BPROP** keywords are specified.

Example: pairwise comparison of column proportions

```
CROSSTABS
  /TABLES= news BY inccat
  /FORMAT=AVALUE TABLES
  /CELLS=COLUMN BPROP
  /COUNT ROUND CELL.
```

Newspaper subscription * Income category in thousands Crosstabulation

% within Income category in thousands

		Income category in thousands				Total
		Under $25	$25 - $49	$50 - $74	$75+	
Newspaper subscription	Yes	35.9%$_a$	36.7%$_a$	43.4%$_b$	57.3%$_c$	43.3%
	No	64.1%$_a$	63.3%$_a$	56.6%$_b$	42.7%$_c$	56.8%
Total		100.0%	100.0%	100.0%	100.0%	100.0%

The column proportions test assigns a subscript letter to the categories of the column variable. For each pair of columns, the column proportions are compared using a z test. If a pair of values is significantly different, the values have different subscript letters assigned to them.

The table in this example is a crosstabulation of survey respondents who have a newspaper subscription by the income category of the respondent, with column percentages shown as the summary statistic. The percentages in the *Under $25* and *$25 - $49* categories both have the subscript *a* so the percentages in those columns are not significantly different. However, the subscripts in the *$50 - $74* and *$75+* categories differ from each other as well as from the subscript for the *Under $25* and *$25 - $49* categories. This means that the percentages in the *$50 - $74* and *$75+* categories are significantly different from each other as well as from the percentages in the *Under $25* and *$25 - $49* categories.

STATISTICS subcommand

STATISTICS requests measures of association and related statistics. By default, CROSSTABS does not display any additional statistics.

- STATISTICS without keywords displays the chi-square test.
- If STATISTICS is specified with keywords, CROSSTABS calculates only the requested statistics.
- In integer mode, values that are not included in the specified range are *not* used in the calculation of the statistics, even if these values exist in the data.
- If user-missing values are included with MISSING, cases with user-missing values are included in the calculation of statistics as well as in the tables.

CHISQ. *Display the Chi-Square Test table*. Chi-square statistics include Pearson chi-square, likelihood-ratio chi-square, and Mantel-Haenszel chi-square (linear-by-linear association). Mantel-Haenszel is valid

only if both variables are numeric. Fisher's exact test and Yates' corrected chi-square are computed for all 2 × 2 tables. This is the default if STATISTICS is specified with no keywords.

PHI. *Display phi and Cramér's* V *in the Symmetric Measures table*.

CC. *Display contingency coefficient in the Symmetric Measures table*.

LAMBDA. *Display lambda (symmetric and asymmetric) and Goodman and Kruskal's tau in the Directional Measures table*.

UC. *Display uncertainty coefficient (symmetric and asymmetric) in the Directional Measures table*.

BTAU. *Display Kendall's tau-*b *in the Symmetric Measures table*.

CTAU. *Display Kendall's tau-*c *in the Symmetric Measures table*.

GAMMA. *Display gamma in the Symmetric Measures table or Zero-Order and Partial Gammas table*. The Zero-Order and Partial Gammas table is produced only for tables with more than two variable dimensions.

D. *Display Somers'* d *(symmetric and asymmetric) in the Directional Measures table*.

ETA. *Display eta in the Directional Measures table*. Available for numeric data only.

CORR. *Display Pearson's* r *and Spearman's correlation coefficient in the Symmetric Measures table*. This is available for numeric data only.

KAPPA. *Display kappa coefficient* [20] *in the Symmetric Measures table*. Kappa is based on a square table in which row and column values represent the same scale. Any cell that has observed values for one variable but not the other is assigned a count of 0. Kappa is not computed if the data storage type (string or numeric) is not the same for the two variables. For string variable, both variables must have the same defined length.

RISK. *Display relative risk* [21] *in the Risk Estimate table*. Relative risk can be calculated only for 2 x 2 tables.

MCNEMAR. *Display a test of symmetry for square tables.* The McNemar test is displayed for 2 x 2 tables, and the McNemar-Bowker test, for larger tables.

CMH(1*). *Conditional independence and homogeneity tests.* Cochran's and the Mantel-Haenszel statistics are computed for the test for conditional independence. The Breslow-Day and Tarone's statistics are computed for the test for homogeneity. For each test, the chi-squared statistic with its degrees of freedom and asymptotic *p* value are computed. *Mantel-Haenszel relative risk (common odds ratio) estimate*. The Mantel-Haenszel relative risk (common odds ratio) estimate, the natural log of the estimate, the standard error of the natural log of the estimate, the asymptotic *p* value, and the asymptotic confidence intervals for common odds ratio and for the natural log of the common odds ratio are computed. The user can specify the null hypothesis for the common odds ratio in parentheses after the keyword. The passive default is 1. (The parameter value must be positive.)

ALL. *All statistics available.*

NONE. *No summary statistics.* This is the default if STATISTICS is omitted.

METHOD subcommand

METHOD displays additional results for each statistic requested. If no METHOD subcommand is specified, the standard asymptotic results are displayed. If fractional weights have been specified, results for all methods will be calculated on the weight rounded to the nearest integer. This subcommand is available only if you have the Exact Tests add-on option installed, which is only available on Windows operating systems.

[20] Kraemer, H. C. 1982. Kappa Coefficient. In: *Encyclopedia of Statistical Sciences*, S. Kotz, and N. L. Johnson, eds. New York: John Wiley and Sons.

[21] Bishop, Y. M., S. E. Feinberg, and P. W. Holland. 1975. *Discrete multivariate analysis: Theory and practice*. Cambridge, Mass.: MIT Press.

MC. Displays an unbiased point estimate and confidence interval based on the Monte Carlo sampling method, for all statistics. Asymptotic results are also displayed. When exact results can be calculated, they will be provided instead of the Monte Carlo results.

CIN(n). Controls the confidence level for the Monte Carlo estimate. CIN is available only when /METHOD=MC is specified. CIN has a default value of 99.0. You can specify a confidence interval between 0.01 and 99.9, inclusive.

SAMPLES. Specifies the number of tables sampled from the reference set when calculating the Monte Carlo estimate of the exact *p* value. Larger sample sizes lead to narrower confidence limits but also take longer to calculate. You can specify any integer between 1 and 1,000,000,000 as the sample size. SAMPLES has a default value of 10,000.

EXACT. Computes the exact significance level for all statistics in addition to the asymptotic results. EXACT and MC are mutually exclusive alternatives (you cannot specify both on the same command). Calculating the exact *p* value can be memory-intensive. If you have specified /METHOD=EXACT and find that you have insufficient memory to calculate results, you should first close any other applications that are currently running in order to make more memory available. You can also enlarge the size of your swap file (see your Windows documentation for more information). If you still cannot obtain exact results, specify /METHOD=MC to obtain the Monte Carlo estimate of the exact *p* value. An optional TIMER keyword is available if you choose /METHOD=EXACT.

TIMER(n). Specifies the maximum number of minutes allowed to run the exact analysis for each statistic. If the time limit is reached, the test is terminated, no exact results are provided, and the program begins to calculate the next test in the analysis. TIMER is available only when /METHOD=EXACT is specified. You can specify any integer value for TIMER. Specifying a value of 0 for TIMER turns the timer off completely. TIMER has a default value of 5 minutes. If a test exceeds a time limit of 30 minutes, it is recommended that you use the Monte Carlo, rather than the exact, method.

Example

```
CROSSTABS TABLES=FEAR BY SEX
  /CELLS=ROW COLUMN EXPECTED RESIDUALS
  /STATISTICS=CHISQ
  /METHOD=MC SAMPLES(10000) CIN(95).
```

- This example requests chi-square statistics.
- An unbiased point estimate and confidence interval based on the Monte Carlo sampling method are displayed with the asymptotic results.

MISSING subcommand

By default, CROSSTABS deletes cases with missing values on a table-by-table basis. Cases with missing values for any variable specified for a table are not used in the table or in the calculation of statistics. Use MISSING to specify alternative missing-value treatments.

- The only specification is a single keyword.
- The number of missing cases is always displayed in the Case Processing Summary table.
- If the missing values are not included in the range specified on VARIABLES, they are excluded from the table regardless of the keyword you specify on MISSING.

TABLE. *Delete cases with missing values on a table-by-table basis.* When multiple table lists are specified, missing values are handled separately for each list. This is the default.

INCLUDE. *Include user-missing values.*

REPORT. *Report missing values in the tables.* This option includes missing values in tables but not in the calculation of percentages or statistics. The missing status is indicated on the categorical label. REPORT is available only in integer mode.

FORMAT subcommand

By default, CROSSTABS displays tables and subtables. The values for the row and column variables are displayed in order from lowest to highest. Use FORMAT to modify the default table display.

AVALUE. *Display row and column variables from lowest to highest value.* This is the default.

DVALUE. *Display row variables from highest to lowest. This setting has no effect on column variables.*

TABLES. *Display tables.* This is the default.

NOTABLES. *Suppress Crosstabulation tables.* NOTABLES is useful when you want to write tables to a file without displaying them or when you want only the Statistics table. This is the same as specifying NONE on CELLS.

COUNT subcommand

The COUNT subcommand controls how case weights are handled.

ASIS. *The case weights are used as is.* However, when Exact Statistics are requested, the accumulated weights in the cells are either truncated or rounded before computing the Exact test statistics.

CASE. *The case weights are either rounded or truncated before use.*

CELL. *The case weights are used as is but the accumulated weights in the cells are either truncated or rounded before computing any statistics.*

ROUND. *Performs Rounding operation.*

TRUNCATE. *Performs Truncation operation.*

BARCHART subcommand

BARCHART produces a clustered bar chart where bars represent categories defined by the first variable in a crosstabulation while clusters represent categories defined by the second variable in a crosstabulation. Any controlling variables in a crosstabulation are collapsed over before the clustered bar chart is created.

- BARCHART takes no further specification.
- If integer mode is in effect and MISSING=REPORT, BARCHART displays valid and user-missing values. Otherwise only valid values are used.

WRITE subcommand

Use the WRITE subcommand to write cell frequencies to a file for subsequent use by the current program or another program. CROSSTABS can also use these cell frequencies as input to reproduce tables and compute statistics. When WRITE is specified, an Output File Summary table is displayed before all other tables. See the OMS command for alternative and more flexible methods for writing out results to external files in various formats.

- The only specification is a single keyword.
- The name of the file must be specified on the PROCEDURE OUTPUT command preceding CROSSTABS.
- If you include missing values with INCLUDE or REPORT on MISSING, no values are considered missing and all non-empty cells, including those with missing values, are written, even if CELLS is specified.
- If you exclude missing values on a table-by-table basis (the default), no records are written for combinations of values that include a missing value.
- If multiple tables are specified, the tables are written in the same order as they are displayed.

- WRITE is not supported for long string variables (defined width greater than 8 bytes) and will result in an error if any of the variables are long strings.
- WRITE is not supported for tables with more than eight dimensions (seven BY variables) and will result in an error if the table has more than eight dimensions.

NONE. *Do not write cell counts to a file.* This is the default.

CELLS. *Write cell counts for non-empty and nonmissing cells to a file.* Combinations of values that include a missing value are not written to the file.

ALL. *Write cell counts for all cells to a file.* A record for each combination of values defined by VARIABLES and TABLES is written to the file. ALL is available only in integer mode.

The file contains one record for each cell. Each record contains the following:

Columns 1–4. *Split-file group number, numbered consecutively from 1.* Note that this is not the value of the variable or variables used to define the splits.

Columns 5–8. *Table number.* Tables are defined by the TABLES subcommand.

Columns 9–16. *Cell frequency.* The number of times this combination of variable values occurred in the data, or, if case weights are used, the sum of case weights for cases having this combination of values.

Columns 17–24. *The value of the row variable* (the one named before the first BY).

Columns 25–32. *The value of the column variable* (the one named after the first BY).

Columns 33–40. *The value of the first control variable* (the one named after the second BY).

Columns 41–48. *The value of the second control variable* (the one named after the third BY).

Columns 49–56. *The value of the third control variable* (the one named after the fourth BY).

Columns 57–64. *The value of the fourth control variable* (the one named after the fifth BY).

Columns 65–72. *The value of the fifth control variable* (the one named after the sixth BY).

Columns 73–80. *The value of the sixth control variable* (the one named after the seventh BY).

- The split-file group number, table number, and frequency are written as integers.
- In integer mode, the values of variables are also written as integers. In general mode, the values are written according to the print format specified for each variable. Alphanumeric values are written at the left end of any field in which they occur.
- Within each table, records are written from one column of the table at a time, and the value of the last control variable changes the most slowly.

Example

```
PROCEDURE OUTPUT  OUTFILE='/data/celldata.txt'.
CROSSTABS VARIABLES=FEAR SEX (1,2)
 /TABLES=FEAR BY SEX
 /WRITE=ALL.
```

- CROSSTABS writes a record for each cell in the table *FEAR* by *SEX* to the file *celldata.txt*.

Example

```
PROCEDURE OUTPUT  OUTFILE='/data/xtabdata.txt'.
CROSSTABS  TABLES=V1 TO V3 BY V4 BY V10 TO V15
  /WRITE=CELLS.
```

- CROSSTABS writes a set of records for each table to file *xtabdata.txt*.
- Records for the table *V1* by *V4* by *V10* are written first, followed by records for *V1* by *V4* by *V11*, and so on. The records for *V3* by *V4* by *V15* are written last.

Reading a CROSSTABS Procedure Output file

You can use the file created by WRITE in a subsequent session to reproduce a table and compute statistics for it. Each record in the file contains all of the information used to build the original table. The

cell frequency information can be used as a weight variable on the WEIGHT command to replicate the original cases.

Example

```
DATA LIST FILE='/celldata.txt'
 /WGHT 9-16 FEAR 17-24 SEX 25-32.
VARIABLE LABELS FEAR 'AFRAID TO WALK AT NIGHT IN NEIGHBORHOODS'.
VALUE LABELS  FEAR 1 'YES' 2 'NO'/ SEX 1 'MALE' 2 'FEMALE'.
WEIGHT BY WGHT.
CROSSTABS TABLES=FEAR BY SEX
 /STATISTICS=ALL.
```

- DATA LIST reads the cell frequencies and row and column values from the *celldata.txt* file. The cell frequency is read as a weighting factor (variable *WGHT*). The values for the rows are read as *FEAR*, and the values for the columns are read as *SEX*, the two original variables.
- The WEIGHT command recreates the sample size by weighting each of the four cases (cells) by the cell frequency.

If you do not have the original data or the CROSSTABS procedure output file, you can reproduce a crosstabulation and compute statistics simply by entering the values from the table:

```
DATA LIST  /FEAR 1 SEX 3 WGHT 5-7.
VARIABLE LABELS  FEAR 'AFRAID TO WALK AT NIGHT IN NEIGHBORHOOD'.
VALUE LABELS  FEAR 1 'YES' 2 'NO'/ SEX 1 'MALE' 2 'FEMALE'.
 WEIGHT  BY WGHT.
BEGIN DATA
1 1  55
2 1 172
1 2 180
2 2  89
END DATA.
CROSSTABS  TABLES=FEAR BY SEX
/STATISTICS=ALL.
```

HIDESMALLCOUNTS Subcommand

HIDESMALLCOUNTS allows you to hide counts displayed in tables for count values that are less than a specified integer. Hidden values are displayed as **<N**, where **N** is the specified integer.

COUNT Keyword

The COUNT keyword specifies the cutoff, below which count values are hidden. Values equal to the cutoff are not hidden. The value of COUNT must be an integer and greater than or equal to 2, although the value 0 is permitted and specifies that there is no cutoff. The default is 5.

Notes

- Percentages associated with hidden counts are also hidden.
- HIDESMALLCOUNTS is only available in general mode.

SHOWDIM Subcommand

SHOWDIM specifies the number of dimensions to be displayed as rows and columns in the crosstabulation table, with the remainder of the dimensions displayed as table layers.

- The specification must be a positive integer. For a specified value of *n*, the dimensions corresponding to the variables before the *n*th BY keyword, as well as cell statistics (such as Count and Expected Count), will be displayed as rows or columns in the crosstabulation table. Variables after the *n*th BY keyword will be displayed as layers in the crosstabulation table.
- For SHOWDIM = 1, values of the variables before the first BY keyword are shown as row headings and cell statistics are shown as column headings, in the crosstabulation table. All other dimensions appear in layers of the crosstabulation table.
- When SHOWDIM is omitted all dimensions will be displayed as rows or columns.
- An integer value greater than or equal to the command dimensionality (the number of BY keywords plus one) is equivalent to omitting the SHOWDIM subcommand.

Note: SHOWDIM is only available in general mode.

References

Bishop, Y. M., S. E. Feinberg, and P. W. Holland. 1975. *Discrete multivariate analysis: Theory and practice*. Cambridge, Mass.: MIT Press.

Haberman, S. J. 1978. *Analysis of qualitative data*. London: Academic Press.

Kraemer, H. C. 1982. Kappa Coefficient. In: *Encyclopedia of Statistical Sciences*, S. Kotz, and N. L. Johnson, eds. New York: John Wiley and Sons.

CSCOXREG

CSCOXREG is available in SPSS Statistics Premium Edition or the Complex Samples option.

Note: Square brackets used in the CSCOXREG syntax chart are required parts of the syntax and are not used to indicate optional elements. Equals signs (=) used in the syntax chart are required elements. All subcommands are optional.

```
CSCOXREG starttime endtime BY factor list WITH covariate list

/VARIABLES STATUS = varname(valuelist)
          ID = varname BASELINESTRATA = varname

/PLAN FILE = 'file'

/JOINTPROB FILE = 'savfile' | 'dataset'

/MODEL effect list
/CUSTOM LABEL = 'label'
      LMATRIX = {effect list, effect list ...; ...      }
                {effect list, effect list ...           }
                {ALL list; ALL ...                      }
                {ALL list                               }

      KMATRIX = {number; number; ...}
                {number             }

/CUSTOM ...

/CRITERIA CILEVEL = {95**  } DF = number
                    {number}

          LCONVERGE = [{0**   } {RELATIVE**}]
                      {number} {ABSOLUTE  }

          MXITER = {100**  } MXSTEP = {5**    }
                   {integer}          {integer}

          PCONVERGE = [{1E-6**} {RELATIVE**}]
                      {number} {ABSOLUTE  }
          SINGULAR = {1E-12**}
                     {number }
          TIES = {EFRON**}
                 {BRESLOW}

/STATISTICS PARAMETER EXP SE TTEST CINTERVAL DEFF DEFFSQRT

/TEST TYPE = {F**         } PADJUST = {LSD**        }
             {ADJF        }           {BONFERRONI   }
             {CHISQUARE   }           {SEQBONFERRONI}
             {ADJCHISQUARE}           {SIDAK        }
                                      {SEQSIDAK     }

/TESTASSUMPTIONS PROPHAZARD = {KM**                        }
                              {IDENTITY                    }
                              {LOG                         }
                              {RANK                        }
                 PARAMETER COVB

/DOMAIN VARIABLE = varname (value)

/MISSING CLASSMISSING = {EXCLUDE**}
                        {INCLUDE  }

/SURVIVALMETHOD BASELINE = {BRESLOW     } CI = {ORIGINAL}
                           {EFRON       }      {LOG**   }
                           {PRODUCTLIMIT}      {LML     }

/PRINT SAMPLEINFO** EVENTINFO** RISKINFO
       HISTORY({1**    }) GEF LMATRIX COVB CORB BASELINE NONE
              {integer}

/SAVE SCHOENFELD(rootname:{25**   }) MARTINGALE(varname) DEVIANCE(varname)
                          {integer}
      COXSNELL(varname) SCORE(rootname:{25**   }) DFBETA(rootname:{25**   })
                                       {integer}                 {integer}
      AGGMARTINGALE(varname)  AGGDEVIANCE(varname) AGGCOXSNELL(varname)
      AGGSCORE(rootname:{25**   }) AGGDFBETA(rootname:{25**   })
                        {integer}                  {integer}
      SURVIVAL(varname) LCL_SURVIVAL(varname) UCL_SURVIVAL(varname)
      CUMHAZARD(varname) LCL_CUMHAZARD(varname) UCL_CUMHAZARD(varname)
      XBETA(varname)

/PLOT SURVIVAL CUMHAZARD LML OMS CI = {YES }
                                      {NO**}

/PATTERN {varname(value)...                          }
         {endtime(valuelist) varname({valuelist}) ...}
                                     {value    }
         BY factor

/PATTERN ...

/OUTFILE {COVB = 'savfile' | 'dataset'    } {MODEL = 'file'    }
         {CORB = 'savfile' | 'dataset'    }
         {SURVIVAL = 'savfile' | 'dataset'}
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 16.0

- Command introduced.

Release 17.0

- Added support for SET THREADS.

Example

```
CSCOXREG endtime_var BY a b c WITH x y z
  /VARIABLES STATUS=status_var(1)
  /PLAN FILE='/survey/myfile.csplan'.
```

Overview

For samples drawn by complex sampling methods, CSCOXREG applies Cox proportional hazards regression to analysis of survival times—that is, the length of time before the occurrence of an event. CSCOXREG supports scale and categorical predictors, which can be time dependent. CSCOXREG provides an easy way of considering differences in subgroups as well as analyzing effects of a set of predictors. The procedure estimates variances by taking into account the sample design used to select the sample, including equal probability and probability proportional to size (PPS) methods and with replacement (WR) and without replacement (WOR) sampling procedures. Optionally, CSCOXREG performs analyses for a subpopulation.

Basic Specification

- The basic specification is a variable list identifying the time variables (at least one but can be up to two), the factors (if any), and the covariates (if any); a VARIABLES subcommand specifying the event status variable; and a PLAN subcommand with the name of a complex sample analysis plan file, which may be generated by the CSPLAN procedure.
- The default model includes main effects for any factors and any covariates.
- The basic specification displays summary information about the sample and all analysis variables, model summary statistics, and Wald F tests for all model effects. Additional subcommands must be used for other output.
- Minimum syntax is a time variable, a status variable, and the PLAN subcommand. This specification fits a baseline-only model.

Syntax Rules

- The *endtime* variable, STATUS in VARIABLES subcommand, and PLAN subcommand are required. All other variables and subcommands are optional.
- Multiple CUSTOM and PATTERN subcommands may be specified; each is treated independently. All other subcommands may be specified only once.
- Empty subcommands are not allowed; all subcommands must be specified with options.
- Each keyword may be specified only once within a subcommand.
- Subcommand names and keywords must be spelled in full.
- Equals signs (=) and slashes shown in the syntax chart are required.
- Bold square brackets shown in the syntax chart are required parts of the syntax and are not used to indicate optional elements.
- Subcommands may be specified in any order.

- The factors, ID variable, baseline strata variable, and the subpopulation can be numeric or string variables, but covariates must be numeric.
- Across the time variables, factor, and covariate variable lists, a variable may be specified only once.
- The status variable, ID variable, baseline strata variable, and subpopulation variables may not be specified on the variable list
- Minimum syntax is a time variable, a status variable, and the PLAN subcommand. This specification fits a baseline-only model.
- Only factors and covariates can be defined by the TIME PROGRAM, all other variables cannot be defined there. See the topic "TIME PROGRAM" on page 1973 for more information.

Operations

- TIME PROGRAM computes the values for time-dependent predictors (see TIME PROGRAM syntax help).
- CSCOXREG performs Cox proportional hazards regression analysis for sampling designs supported by the CSPLAN and CSSELECT procedures.
- The input dataset must contain the variables to be analyzed and variables related to the sampling design.
- The complex sample analysis plan file provides an analysis plan based on the sampling design.
- By default, CSCOXREG uses a model that includes main effects for any factors and any covariates.
- Other effects, including interaction and nested effects, may be specified using the MODEL subcommand.
- The default output for the specified model is summary information about the sample and all analysis variables, model summary statistics, and Wald F tests for all model effects.
- This procedure uses the multithreaded options specified by SET THREADS.

Limitations

WEIGHT and SPLIT FILE settings are ignored with a warning by the CSCOXREG procedure.

Examples

```
CSCOXREG t BY a b c WITH x
  /VARIABLES STATUS = dead(1)
  /PLAN FILE='c:\survey\myfile.csplan'.
```

- *t* is the time variable; *a*, *b*, and *c* are factors; *x* is a covariate.
- The status variable is *dead* with a value of 1 representing the terminal event.
- The complex sampling plan is given in the file *c:\survey\myfile.csplan*.
- CSCOXREG will fit the default model including the main effects for factors *a*, *b*, and *c* and the covariate *x*.

Variable List Subcommand

The variable list specifies the time variable(s), the factors, and the covariates in the model.

- The time variables *starttime* (if specified) and *endtime* must be listed first. These variables represent the endpoints of a time interval (*starttime*, *endtime*) during which the case is at risk.
- When *starttime* is not specified, it is implied that *starttime* = 0 for all cases if an ID variable is not specified. If an ID variable is specified, it is assumed that *starttime* = 0 for the first observation for that subject and *starttime* for following observations equals *endtime* of the previous observation. See the example below.
- The time variables must be numeric and non-negative.
- If the time variables are of Date or Time type, the internal numeric representation will be used and a warning will be given. For example, November 8, 1957, is 1.2E+10 (the number of seconds from

midnight, October 14, 1582). See “Date and time functions” on page 72 for detailed internal numeric representation of different date and time formats.

- The names of the factors and covariates, if any, follow the time variables. Specify any factors following the keyword BY. Specify any covariates following the keyword WITH.
- Factors can be numeric or string variables, but covariates must be numeric.
- Each variable may be specified only once on the variable list.
- The status variable, ID variable, baseline strata variable, and subpopulation variables may not be specified on the variable list.

Example

```
CSCOXREG tstart tend BY a b c WITH x
  /VARIABLES STATUS = dead(1) ID = SSN
  /PLAN FILE='c:\survey\myfile.csplan'
  /MODEL a b c a*b a*c b*c x.
```

- Two time variables, *tstart* and *tend*, are specified.
- ID specifies *SSN* as the subject ID variable. All cases sharing the same value of *SSN* belong to the same subject.
- This example fits a model that includes the main effects for factors *a*, *b*, and *c*; all two-way interactions among the factors; and the covariate *x*.

```
CSCOXREG tend BY a b c WITH x
  /VARIABLES STATUS = dead(1) ID = SSN
  /PLAN FILE='c:\survey\myfile.csplan'
  /MODEL a b c a*b a*c b*c x.
```

- This is the same as the above example except that only one time variable, *tend*, is specified.

VARIABLES Subcommand

VARIABLES specifies the status variable, ID variable, and baseline strata variable.

STATUS = varname. *Event status variable.* To determine whether the event has occurred at *endtime* for a particular observation, CSCOXREG checks the value of a status variable. STATUS lists the status variable and the values that indicate the occurrence of the event.

- The value list must be enclosed in parentheses. All cases with non-negative times that do not have a value within the range specified are classified as censored cases—that is, cases for which the event has not yet occurred at *endtime*.
- The value list can be one value, a list of values separated by blanks or commas, a range of values using the keyword THRU, or a combination.
- The status variable can be either numeric or string. If a string variable is specified, the event values must be enclosed in apostrophes and the keyword THRU cannot be used.
- *ID variable.* Cases with the same ID value are repeated observations from the same subject. If ID is not specified, each case represents one subject.

BASELINESTRATA = varname. *Baseline stratification variable.* A separate baseline hazard and survival function is computed for each value of this variable, while a single set of model coefficients is estimated across strata.

Example

```
CSCOXREG SURVIVAL by GROUP
/VARIABLES STATUS=SURVSTA(3 THRU 5, 8 THRU 10) BASELINESTRATA=LOCATION
/PLAN FILE='c:\survey\myfile.csplan'.
```

- STATUS specifies that *SURVSTA* is the status variable.
- A value between either 3 and 5 or 8 and 10, inclusive, means that the terminal event occurred.
- Values outside the specified ranges indicate censored cases.
- BASELINESTRATA specifies *LOCATION* as the strata variable.

- Different baseline survival functions are computed for each value of *LOCATION*.

PLAN Subcommand

The PLAN subcommand specifies the name of an XML file containing analysis design specifications. This file is written by the CSPLAN procedure.

- The PLAN subcommand is required.

FILE. *Specifies the name of an external file*.

JOINTPROB Subcommand

The JOINTPROB subcommand is used to specify the file or dataset containing the first stage joint inclusion probabilities for UNEQUAL_WOR estimation. The CSSELECT procedure writes this file in the same location and with the same name (but a different extension) as the plan file. When UNEQUAL_WOR estimation is specified, the procedure will use the default location and name of the file unless the JOINTPROB subcommand is used to override them.

FILE. *Specifies the name of the file or dataset containing the joint inclusion probabilities*.

MODEL Subcommand

The MODEL subcommand is used to specify the effects to be included in the model.

- Specify a list of terms to be included in the model, separated by spaces or commas.
- If the MODEL subcommand is not specified, CSCOXREG uses a model that includes main effects for any factors, and any covariates, in the order specified on the variable list.
- To include a term for the main effect of a factor, enter the name of the factor.
- To include a term for an interaction among factors, use the keyword BY or the asterisk (*) to join the factors involved in the interaction. For example, A*B means a two-way interaction effect of A and B, where A and B are factors. A*A is not allowed because factors inside an interaction effect must be distinct.
- To include a term for nesting one factor within another, use a pair of parentheses. For example, A(B) means that A is nested within B. A(A) is not allowed because factors inside a nested effect must be distinct.
- Multiple nesting is allowed. For example, A(B(C)) means that B is nested within C, and A is nested within B(C). When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, A(B)(C) is not valid.
- Nesting within an interaction effect is valid. For example, A(B*C) means that A is nested within B*C.
- Interactions among nested effects are allowed. The correct syntax is the interaction followed by the common nested effect inside the parentheses. For example, interaction between A and B within levels of C should be specified as A*B(C) instead of A(C)*B(C).
- To include a covariate term in the design, enter the name of the covariate.
- Covariates can be connected, but not nested, through the * operator or using the keyword BY to form another covariate effect. Interactions among covariates such as X1*X1 and X1*X2 are valid, but X1(X2) is not.
- Factor and covariate effects can be connected in various ways except that no effects can be nested within a covariate effect. Suppose A and B are factors and X1 and X2 are covariates, examples of valid combinations of factor and covariate effects are A*X1, A*B*X1, X1(A), X1(A*B), X1*A(B), X1*X2(A*B), and A*B*X1*X2.

CUSTOM Subcommand

The CUSTOM subcommand defines custom hypothesis tests by specifying the **L** matrix (contrast coefficients matrix) and the **K** matrix (contrast results matrix) in the general form of the linear hypothesis **LB** = **K**. The vector **B** is the parameter vector in the linear model.

- Multiple CUSTOM subcommands are allowed. Each subcommand is treated independently.
- An optional label may be specified by using the LABEL keyword. The label is a string with a maximum length of 255 characters. Only one label can be specified.
- Either the LMATRIX or KMATRIX keyword, or both, must be specified.

  LMATRIX. *Contrast coefficients matrix.* This matrix specifies coefficients of contrasts, which can be used for studying the effects in the model. An **L** matrix can be specified by using the LMATRIX keyword.

  KMATRIX. *Contrast results matrix.* This matrix specifies the results of the linear hypothesis. A **K** matrix can be specified by using the KMATRIX keyword.
- The number of rows in the **L** and **K** matrices must be equal.
- A custom hypothesis test can be formed by specifying an **L** or **K** matrix, or both. If only one matrix is specified, the unspecified matrix uses the defaults described below.
- If KMATRIX is specified but LMATRIX is not specified, the **L** matrix is assumed to be the row vector corresponding to the intercept in the estimable function, provided that INCLUDE = YES or ONLY is specified on the INTERCEPT subcommand. In this case, the **K** matrix can be only a scalar matrix.
- The default **K** matrix is a zero matrix—that is, **LB** = **0** is assumed.
- There are three general formats that can be used on the LMATRIX keyword: (1) Specify a coefficient value for the intercept, followed optionally by an effect name and a list of real numbers. (2) Specify an effect name and a list of real numbers. (3) Specify keyword ALL and a list of real numbers. In all three formats, there can be multiple effect names (or instances of the keyword ALL) and number lists.
- Only valid effects in the default model or on the MODEL subcommand can be specified on the LMATRIX keyword.
- The length of the list of real numbers on the LMATRIX keyword must be equal to the number of parameters (including the redundant parameters) corresponding to the specified effect. For example, if the effect A*B takes up six columns in the design matrix, the list after A*B must contain exactly six numbers.
- When ALL is specified, the length of the list that follows ALL must be equal to the total number of parameters (including the redundant parameters) in the model.
- Effects that are in the model but not specified on the LMATRIX keyword are assumed to have entries of 0 in the corresponding columns of the **L** matrix.
- When an **L** matrix is being defined, a number can be specified as a fraction with a positive denominator. For example, 1/3 and –1/3 are valid, but 1/–3 is invalid.
- A semicolon (;) indicates the end of a row in the **L** matrix.
- The format for the KMATRIX keyword is one or more real numbers. If more than one number is specified, then separate adjacent numbers using a semicolon (;). Each semicolon indicates the end of a row in the **K** matrix. Each number is the hypothesized value for a contrast, which is defined by a row in the **L** matrix.
- For the KMATRIX keyword to be valid, either the LMATRIX keyword, or INCLUDE = YES on the INTERCEPT subcommand, must be specified.

Example

Suppose that factors A and B each have three levels.

```
CSCOXREG t BY a b
  /VARIABLES STATUS=death(1)
  /PLAN FILE='c:\survey\myfile.csplan'
  /MODEL a b a*b
  /CUSTOM  LABEL = "Effect A"
     LMATRIX = a 1 0 -1
```

```
        a*b  1/3  1/3  1/3
             0    0    0
            -1/3 -1/3 -1/3;
        a 0 1 -1
        a*b  0    0    0
             1/3  1/3  1/3
            -1/3 -1/3 -1/3.
```

- The preceding syntax specifies a test of effect A.
- Because there are three levels in effect A, two independent contrasts can be formed at most; thus, there are two rows in the **L** matrix separated by a semicolon (;).
- There are three levels each in effects A and B; thus, the interaction effect A*B takes nine columns in the design matrix.
- The first row in the **L** matrix tests the difference between levels 1 and 3 of effect A; the second row tests the difference between levels 2 and 3 of effect A.
- The KMATRIX keyword is not specified, so the null hypothesis value for both tests is 0.

Example

Suppose that factor A has three levels.

```
CSCOXREG t BY a
  /VARIABLES STATUS=death(1)
  /PLAN FILE='c:\survey\myfile.csplan'
  /MODEL a
  /CUSTOM  LABEL = 'Effect A'
   LMATRIX = a 1 0 -1;
             a 0 1 -1
   KMATRIX = 1; 1.
```

- The syntax specifies a model with a main effect for factor A and a custom hypothesis test of effect A.
- The equivalent LMATRIX keyword using the ALL option follows.

```
LMATRIX = ALL 1  0 -1;
          ALL 0  1 -1
```

- The KMATRIX keyword is specified and the hypothesis that the difference between levels 1 and 3 and levels 2 and 3 of effect A are both equal to 1 is tested.

CRITERIA Subcommand

The CRITERIA subcommand controls the iterative algorithm used for estimation, specifies the numerical tolerance for checking for singularity, and specifies the ties breaking method used in estimating regression parameters.

CILEVEL = number. *Confidence interval level for coefficient estimates, exponentiated coefficient estimates, survival function estimates, and cumulative hazard function estimates.* Specify a value greater than or equal to 0 and less than 100. The default value is 95.

DF = number. *Sampling design degrees of freedom to use in computing p values for all test statistics.* Specify a positive number. The default value is the difference between the number of primary sampling units and the number of strata in the first stage of sampling.

LCONVERGE = [number RELATIVE|ABSOLUTE]. *Log-likelihood function convergence criterion.* Convergence is assumed if the relative or absolute change in the log-likelihood function is less than the given value. This criterion is not used if the value is 0.

Specify square brackets containing a non-negative number followed optionally by keyword RELATIVE or ABSOLUTE, which indicates the type of change. The default value is 0; the default type is RELATIVE.

MXITER = integer. *Maximum number of iterations.* Specify a non-negative integer. The default value is 100.

MXSTEP = integer. *Maximum step-halving allowed.* Specify a positive integer. The default value is 5.

PCONVERGE = [number RELATIVE|ABSOLUTE] . *Parameter estimates convergence criterion.* Convergence is assumed if the relative or absolute change in the parameter estimates is less than the given value. This criterion is not used if the value is 0.

Specify square brackets containing a non-negative number followed optionally by keyword RELATIVE or ABSOLUTE, which indicates the type of change. The default value is 10^{-6}; the default type is RELATIVE.

SINGULAR = number. *Tolerance value used to test for singularity.* Specify a positive value. The default value is 10^{-12}.

TIES = EFRON|BRESLOW. *Tie breaking method in estimating parameters.* The default Efron method is specified by the keyword EFRON; the default Breslow method is specified by the keyword BRESLOW.

STATISTICS Subcommand

The STATISTICS subcommand requests various statistics associated with the parameter estimates.

There are no default keywords on the STATISTICS subcommand. If this subcommand is not specified, then none of the statistics listed below are displayed.

PARAMETER. *Parameter estimates.*

EXP . *The exponentiated parameter estimates.*

SE . *Standard error for each parameter estimate.*

TTEST. *t test for each parameter estimate.*

CINTERVAL. *Confidence interval for each parameter estimate and/or exponentiated parameter estimate.*

DEFF. *Design effect for each parameter estimate.*

DEFFSQRT. *Square root of design effect for each parameter estimate.*

TEST Subcommand

The TEST subcommand specifies the type of test statistic and the method of adjusting the significance level to be used for hypothesis tests requested on the MODEL, CUSTOM, and PRINT subcommands.

TYPE Keyword

The TYPE keyword indicates the type of test statistic.

F. *Wald F test.* This is the default test statistic if the TYPE keyword is not specified.

ADJF. *Adjusted Wald F test.*

CHISQUARE. *Wald chi-square test.*

ADJCHISQUARE. *Adjusted Wald chi-square test.*

PADJUST Keyword

The PADJUST keyword indicates the method of adjusting the significance level.

LSD. *Least significant difference.* This method does not control the overall probability of rejecting the hypotheses that some linear contrasts are different from the null hypothesis value(s). This is the default.

BONFERRONI. *Bonferroni.* This method adjusts the observed significance level for the fact that multiple contrasts are being tested.

SEQBONFERRONI. *Sequential Bonferroni.* This is a sequentially step-down rejective Bonferroni procedure that is much less conservative in terms of rejecting individual hypotheses but maintains the same overall significance level.

SIDAK. *Sidak.* This method provides tighter bounds than the Bonferroni approach.

SEQSIDAK. *Sequential Sidak.* This is a sequentially step-down rejective Sidak procedure that is much less conservative in terms of rejecting individual hypotheses but maintains the same overall significance level.

TESTASSUMPTIONS Subcommand

The TESTASSUMPTIONS subcommand produces tests of the proportional hazards and covariate form model assumptions. You can request various statistics associated with the alternative models.

PROPHAZARD Keyword

The PROPHAZARD keyword produces a test for proportional hazards assumption. The time function used in testing for proportional hazards is specified in parentheses. Specify one of the following options.

KM. *Kaplan-Meier estimation of survival function.* This is the default.

IDENTITY. *Identity function of time.*

LOG. *Log function of time.*

RANK. *Rank of death time.*

PARAMETER Keyword

The PARAMETER keyword displays the parameter estimates of the alternative model. The alternative model is estimated using the same convergence criteria as the original model. Both parameters and their standard errors are estimated.

COVB Keyword

The COVB keyword displays the covariance matrix for the alternative model parameters

DOMAIN Subcommand

The DOMAIN subcommand specifies the subpopulation for which the analysis is to be performed.

- The keyword VARIABLE, followed by an equals sign, a variable, and a value in parentheses are required. Put the value inside a pair of quotes if the value is formatted (such as date or currency) or if the factor is of string type.
- The subpopulation is defined by all cases having the given value on the specified variable.
- Analyses are performed only for the specified subpopulation.
- For example, `DOMAIN VARIABLE = myvar (1)` defines the subpopulation by all cases for which variable *MYVAR* has value 1.
- The specified variable may be numeric or string and must exist at the time the CSCOXREG procedure is invoked.
- Stratification or cluster variables may be specified, but no other plan file variables are allowed on the DOMAIN subcommand.
- Analysis variables may not be specified on the DOMAIN subcommand.

MISSING Subcommand

The MISSING subcommand specifies how missing values are handled.

- In general, cases must have valid data for all design variables as well as for the dependent variable and any covariates. Cases with invalid data for any of these variables are excluded from the analysis.
- There is one important exception to the preceding rule. This exception applies when an inclusion probability or population size variable is defined in an analysis plan file. Within a stratum at a given stage, if the inclusion probability or population size values are unequal across cases or missing for a case, then the first valid value found within that stratum is used as the value for the stratum. If strata are not defined, then the first valid value found in the sample is used. If the inclusion probability or population size values are missing for all cases within a stratum (or within the sample if strata are not defined) at a given stage, then an error message is issued.

- The CLASSMISSING keyword specifies whether user-missing values are treated as valid. This specification is applied to categorical design variables (that is, strata, cluster, and subpopulation variables), the dependent variable, and any factors.

EXCLUDE. *Exclude user-missing values among the strata, cluster, subpopulation, dependent variable, and factor variables.* This is the default.

INCLUDE. *Include user-missing values among the strata, cluster, subpopulation, dependent variable, and factor variables.* Treat user-missing values for these variables as valid data.

SURVIVALMETHOD Subcommand

The SURVIVALMETHOD subcommand controls the methods for estimating baseline functions and the confidence interval of the survival function.

BASELINE Keyword

The BASELINE keyword controls the method for estimating baseline functions. Specify one of the following options.

EFRON. *Efron method.* Default if EFRON is chosen in TIES.

BRESLOW. *Breslow method.* Default if BRESLOW is chosen in TIES.

PRODUCTLIMIT . *Product limit method.*

CI Keyword

The CI keyword controls the method for estimating the confidence interval of the survival function. Specify one of the following options.

ORIGINAL. *Based on original scale.* Calculate the confidence interval for the survival function directly.

LOG. *Based on log scale.* Calculate the confidence interval for ln(survival) first, then back transform to get the confidence interval for the survival function.

LML. *Based on log-log scale.* Calculate the confidence interval for ln(–ln(survival)) first, then back transform to get the confidence interval for the survival function.

PRINT Subcommand

The PRINT subcommand is used to display optional output.

- If the PRINT subcommand is not specified, then the default output includes sample information, variable and factor information, and model summary statistics.
- If the PRINT subcommand is specified, then CSCOXREG displays output only for those keywords that are specified.

SAMPLEINFO. *Sample information table.* Displays summary information about the sample, including the unweighted count, the event and censoring counts, and the population size. This is default output if the PRINT subcommand is not specified.

EVENTINFO. *Event and censoring information.* Displays event and censoring information for each baseline stratum. This is the default output if the PRINT subcommand is not specified.

RISKINFO. *Risk and event information.* Displays number of events and number at risk for each event time in each baseline stratum.

HISTORY(n). *Iteration history.* Displays coefficient estimates and statistics at every *n*th iteration beginning with the *0*th iteration (the initial estimates). The default is to print every iteration (*n* = 1). The last iteration is always printed if HISTORY is specified, regardless of the value of *n*.

GEF. *General estimable function table.*

LMATRIX. *Set of contrast coefficients (L) matrices.* These are the Type III contrast matrices used in testing model effects.

COVB. *Covariance matrix for model parameters.*

CORB. *Correlation matrix for model parameters.*

BASELINE. *Baseline functions.* Displays the baseline survival function, baseline cumulative hazards function and their standard errors. If time-dependent covariates defined by TIME PROGRAM are included in the model, no baseline functions are produced.

NONE. *No PRINT output.* None of the PRINT subcommand default output is displayed. However, if NONE is specified with one or more other keywords, then the other keywords override NONE.

SAVE Subcommand

The SAVE subcommand writes optional model variables to the working data file.

- Specify one or more temporary variables, each followed by an optional new name in parentheses.
- The optional names must be valid variable names.
- If new names are not specified, CSCOXREG uses the default names.
- The optional variable name must be unique. If the default name is used and it conflicts with existing variable names, then a suffix is added to the default name to make it unique.
- If a subpopulation is defined on the DOMAIN subcommand, then SAVE applies only to cases within the subpopulation.
- Aggregated residuals are residuals aggregated over records with the same ID value. If ID is not specified, aggregated residuals are not available and a warning is issued if they are requested. The aggregated residual for a subject is written in the last case (or first case if it is easier) of that subject.
- If time-dependent covariates defined by TIME PROGRAM are included in the model, the following options are not available: MARTINGALE, DEVIANCE, COXSNELL, AGGMARTINGALE, AGGDEVIANCE, AGGCOXSNELL, SURVIVAL, LCL_SURVIVAL, UCL_SURVIVAL, CUMHAZARD, LCL_CUMHAZARD, and UCL_CUMHAZARD. A warning is issued if they are requested.
- In situations when rootname is needed, the rootname can be followed by a colon and a positive integer giving the maximum number of variables with the same rootname to be saved. The first *n* variables are saved. The default *n* is 25. To specify *n* without a rootname, enter a colon before the number.

SCHOENFELD(rootname:n). *Schoenfeld residual.* A separate variable is saved for each nonredundant parameter and calculated only for noncensored observations. The default variable name is *Resid_Schoenfeld*.

MARTINGALE(varname). *Martingale residual.* The default variable name is *Resid_Martingale*.

DEVIANCE(varname). *Deviance residual.* The default variable name is *Resid_Deviance*.

COXSNELL(varname). *Cox_Snell residual.* The default variable name is *Resid_CoxSnell*.

SCORE(rootname:n). *Score residual.* A separate variable is saved for each nonredundant parameter. The default variable name is *Resid_Score*.

DFBETA(rootname:n). *DFBETA.* A separate variable is saved for each nonredundant parameter. The default variable name is *Resid_DFBETA*.

AGGMARTINGALE(varname). *Aggregated Martingale residual.* The default variable name is *AggResid_Martingale*.

AGGDEVIANCE(varname). *Aggregated Deviance residual.* The default variable name is *AggResid_Deviance*.

AGGCOXSNELL(varname) . *Aggregated CoxSnell residual.* The default variable name is *AggResid_CoxSnell*.

AGGSCORE(rootname:n). *Aggregated Score residual.* A separate variable is saved for each nonredundant parameter. The default variable name is *AggResid_Score*.

AGGDFBETA(rootname:n). *Aggregated DFBETA.* A separate variable is saved for each nonredundant parameter. The default variable name is *AggResid_DFBETA*.

XBETA(varname). *Linear combination of reference value corrected predictors times regression coefficients.* The default variable name is *XBETA*.

SURVIVAL(varname). *Survival function.* For one-time input data, it is the survival function at the observed time and predictor pattern for each record. For two-time input data, it is the survival function at *endtime* assuming that the predictor is fixed. The default variable name is *Survival*.

LCL_SURVIVAL(varname). *Lower confidence level of survival function.* The default variable name is *LCL_Survival*.

UCL_SURVIVAL(varname). *Upper confidence level of survival function.* The default variable name is *UCL_Survival*.

CUMHAZARD(varname). *Cumulative hazards function.* The default variable name is *CumHazard*.

LCL_CUMHAZARD(varname). *Lower confidence level of cumulative hazards function.* The default variable name is *LCL_CumHazard*.

UCL_CUMHAZARD(varname). *Upper confidence level of cumulative hazards function.* The default variable name is *SECumHazard*.

PLOT Subcommand

You can request specific plots to be produced with the PLOT subcommand. Each requested plot is produced once for each pattern specified on the PATTERN subcommand.

- The set of plots requested is displayed for the functions at the mean of the covariates and at each combination of covariate values specified on PATTERN.
- Lines on a plot are connected as step functions.

SURVIVAL. *Plot the survival function.*

HAZARD. *Plot the cumulative hazard function.*

LML. *Plot the log-minus-log-of-survival function.*

OMS. *Plot the one-minus-survival function.*

CI = NO | YES. *Plot confidence intervals along with the specified functions.* NO is the default.

PATTERN Subcommand

PATTERN specifies the pattern of predictor values to be used for requested plots on the PLOT subcommand and the exported survival file on the OUTFILE subcommand. PATTERN cannot be used when time-dependent predictors calculated by TIME PROGRAM are included in the model.

- A value must be specified for each variable specified on PATTERN.
- Covariates that are included in the model but not named on PATTERN are evaluated at their means.
- Factors that are included in the model but not named on PATTERN are evaluated at the reference category.
- You can request separate lines for each category of a factor that is in the model. Specify the name of the categorical variable after the keyword BY. The BY variable must be a categorical variable. You cannot specify a value for the BY variable.
- Multiple PATTERN subcommands can be specified. CSCOXREG produces a set of requested plots for each specified pattern.

- Piecewise constant predictor paths are also allowed. The path is specified by endtime(valuelist) varname(valuelist) varname(value).... If varname(valuelist) is used, the length of valuelist must be the same as that for endtime. The varname(value) means that the value of the variable is constant over time.

Example

```
CSCOXREG t by A with x1 x2
/VARIABLES STATUS=dead(1)
/PLAN FILE='c:\survey\myfile.csplan'
/PATTERN x1(0.1) x2(3) A(3)
/PLOT SURVIVAL.
```

- Predictor pattern x1 = 0.1, x2 = 3, A = 3 is specified by PATTERN.
- The survival function is plotted for the specified pattern.

Example: Piecewise constant predictor path

```
CSCOXREG t1 t2 by A with x1 x2
/VARIABLES STATUS=dead(1)
/PLAN FILE='c:\survey\myfile.csplan'.
/PATTERN t2(10 20 30 50) x1(1 1.2 1.7 1.9) x2(3) BY A
/OUTFILE SURVIVAL='surv.sav'.
```

- Two time variables are specified on the CSCOXREG variable list.
- PATTERN defines the following predictor paths for x1 and x2.

Table 26. Predictor paths for x1 and x2

starttime	endtime	x1	x2
0	10	1.0	3
10	20	1.2	3
20	30	1.7	3
30	50	1.9	3

- PATTERN, through BY A, also specifies that each category of factor A is considered separately. Combining different categories of A with the paths for x1 and x2, the total number of paths considered here actually equals the number of categories of A.
- The survival table for the specified predictor paths are calculated and written to the file *surv.sav*.

OUTFILE Subcommand

The OUTFILE subcommand saves an external IBM SPSS Statistics data file or previously declared dataset containing the parameter covariance or correlation matrix with parameter estimates, standard errors, significance values, and sampling design degrees of freedom. It also saves the parameter estimates and the parameter covariance matrix in XML format.

- At least one keyword and a filename are required.
- The COVB and CORB keywords are mutually exclusive, as are the MODEL and PARAMETER keywords.
- The filename must be specified in full. CSCOXREG does not supply an extension.

COVB = 'savfile'|'dataset'. *Writes the parameter covariance matrix and other statistics to an external IBM SPSS Statistics data file or previously declared dataset.*

CORB = 'savfile'|'dataset'. *Writes the parameter correlation matrix and other statistics to an external IBM SPSS Statistics data file or previously declared dataset.*

MODEL = 'file'. *Writes all information needed to predict the survival function, including the parameter estimates and baseline survival function, to a PMML file.*

SURVIVAL= 'savfile'|'dataset'. *Writes survival table to an external IBM SPSS Statistics data file or previously declared dataset.* The file contains the survival function, standard error of the survival function,

upper and lower bounds of the confidence interval of the survival function, and the cumulative hazards function for each failure or event time evaluated at the baseline and at the covariate patterns specified on PATTERN. If time-dependent covariates are included in the model, no file is written.

CSDESCRIPTIVES

CSDESCRIPTIVES is available in SPSS Statistics Premium Edition or the Complex Samples option.

```
CSDESCRIPTIVES

/PLAN FILE = file

[/JOINTPROB FILE = file]

[/SUMMARY VARIABLES = varlist]

[/MEAN [TTEST = {value    }]
               {valuelist}

[/SUM [TTEST = {value    }]
              {valuelist}

[/RATIO NUMERATOR = varlist DENOMINATOR = varlist

       [TTEST = {value    }]]
                {valuelist}

[/RATIO...]


[/STATISTICS [COUNT] [POPSIZE] [SE] [CV] [DEFF] [DEFFSQRT]
             [CIN [({95** })]]]
                    {value}

[/SUBPOP TABLE = varname [BY varname [BY ...]] [DISPLAY = {LAYERED }]]
                                                          {SEPARATE}

[/MISSING [SCOPE = {ANALYSIS}] [CLASSMISSING = {EXCLUDE}]]
                   {LISTWISE}                  {INCLUDE}
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
CSDESCRIPTIVES
  /PLAN FILE = '/survey/myfile.xml'
  /SUMMARY VARIABLES = y1 y2
  /MEAN.
```

Overview

CSDESCRIPTIVES estimates means, sums, and ratios, and computes their standard errors, design effects, confidence intervals, and hypothesis tests, for samples that are drawn by complex sampling methods. The procedure estimates variances by taking into account the sample design that is used to select the sample, including equal probability and probability proportional to size (PPS) methods, and with replacement (WR) and without replacement (WOR) sampling procedures. Optionally, CSDESCRIPTIVES performs analyses for subpopulations.

Basic Specification

- The basic specification is a PLAN subcommand and the name of a complex sample analysis plan file (which may be generated by the CSPLAN procedure) and a MEAN, SUM, or RATIO subcommand. If a MEAN or SUM subcommand is specified, a SUMMARY subcommand must also be present.
- The basic specification displays the overall population size estimate. Additional subcommands must be used for other results.

Operations

- CSDESCRIPTIVES computes estimates for sampling designs that are supported by the CSPLAN and CSSELECT procedures.
- The input dataset must contain the variables to be analyzed and variables that are related to the sampling design.
- The complex sample analysis plan file provides an analysis plan based on the sampling design.

- The default output for each requested mean, sum, or ratio is the estimate and its standard error.
- WEIGHT and SPLIT FILE settings are ignored by the CSDESCRIPTIVES procedure.

Syntax Rules

- The PLAN subcommand is required. In addition, the SUMMARY subcommand and the MEAN or SUM subcommand must be specified, or the RATIO subcommand must be specified. All other subcommands are optional.
- Multiple instances of the RATIO subcommand are allowed—each instance is treated independently. All other subcommands may be specified only once.
- Subcommands can be specified in any order.
- All subcommand names and keywords must be spelled in full.
- Equals signs (=) that are shown in the syntax chart are required.
- The MEAN and SUM subcommands can be specified without further keywords, but no other subcommands may be empty.

PLAN Subcommand

PLAN specifies the name of an XML file containing analysis design specifications. This file is written by the CSPLAN procedure.

- The PLAN subcommand is required.

FILE. *Specifies the name of an external file.*

JOINTPROB Subcommand

JOINTPROB is used to specify the file or dataset containing the first-stage joint inclusion probabilities for UNEQUAL_WOR estimation. The CSSELECT procedure writes this file in the same location and with the same name (but different extension) as the plan file. When UNEQUAL_WOR estimation is specified, the CSDESCRIPTIVES procedure will use the default location and name of the file unless the JOINTPROB subcommand is used to override them.

FILE. *Specifies the name of the file or dataset containing the joint inclusion probabilities*.

SUMMARY Subcommand

SUMMARY specifies the analysis variables that are used by the MEAN and SUM subcommands.

- A variable list is required only if means or sums are to be estimated. If only ratios are to be estimated (that is, if the RATIO subcommand is specified but the MEAN and SUM subcommands are not specified), the SUMMARY subcommand is ignored.
- All specified variables must be numeric.
- All specified variables must be unique.
- Plan file and subpopulation variables may not be specified on the SUMMARY subcommand.

VARIABLES. *Specifies the variables used by the MEAN and SUM subcommands*.

MEAN Subcommand

MEAN is used to request that means be estimated for variables that are specified on the SUMMARY subcommand.

The TTEST keyword requests *t* tests of the population means(s) and gives the null hypothesis value(s). If subpopulations are defined on the SUBPOP subcommand, null hypothesis values are used in the test(s) for each subpopulation, as well as for the entire population.

value. The null hypothesis is that the population mean equals the specified value for all *t* tests.

valuelist. This list gives the null hypothesis value of the population mean for each variable on the SUMMARY subcommand. The number and order of values must correspond to the variables on the SUMMARY subcommand.

- Commas or spaces must be used to separate the values.

SUM Subcommand

SUM is used to request that sums be estimated for variables specified on the SUMMARY subcommand.

The TTEST keyword requests *t* tests of the population sum(s) and gives the null hypothesis value(s). If subpopulations are defined on the SUBPOP subcommand, then null hypothesis values are used in the test(s) for each subpopulation as well as for the entire population.

value. The null hypothesis is that the population sum equals the specified value for all *t* tests.

valuelist. This list gives the null hypothesis value of the population sum for each variable on the SUMMARY subcommand. The number and order of values must correspond to the variables on the SUMMARY subcommand.

- Commas or spaces must be used to separate the values.

RATIO Subcommand

RATIO specifies ratios of variables to be estimated.

- Ratios are defined by crossing variables on the NUMERATOR keyword with variables on the DENOMINATOR keyword, with DENOMINATOR variables looping fastest, irrespective of the order of the keywords. For example, `/RATIO NUMERATOR = N1 N2 DENOMINATOR = D1 D2` yields the following ordered list of ratios: N1/D1, N1/D2, N2/D1, N2/D2.
- Multiple RATIO subcommands are allowed. Each subcommand is treated independently.
- Variables that are specified on the RATIO subcommand do not need to be specified on the SUMMARY subcommand.
- All specified variables must be numeric.
- Within each variable list, all specified variables must be unique.
- Plan file and subpopulation variables may not be specified on the RATIO subcommand.

The TTEST keyword requests *t* tests of the population ratio(s) and gives the null hypothesis value(s). If subpopulations are defined on the SUBPOP subcommand, then null hypothesis values are used in the test(s) for each subpopulation as well as for the entire population.

value. The null hypothesis is that the population ratio equals the specified value for all *t* tests.

valuelist. This list gives the null hypothesis value of the population ratio for each ratio specified on the RATIO subcommand. The number and order of values must correspond to the ratios defined on the RATIO subcommand.

- Commas or spaces must be used to separate the values.

STATISTICS Subcommand

STATISTICS requests various statistics that are associated with the mean, sum, or ratio estimates. If the STATISTICS subcommand is not specified, the standard error is computed for any displayed estimates. If the STATISTICS subcommand is specified, only statistics that are requested are computed.

COUNT. *The number of valid observations in the dataset for each mean, sum, or ratio estimate*.

POPSIZE. *The population size for each mean, sum, or ratio estimate*.

SE. *The standard error for each mean, sum, or ratio estimate*. This output is default output if the STATISTICS subcommand is not specified.

CV. *Coefficient of variation*.

DEFF. *Design effect*.

DEFFSQRT. *Square root of the design effect*.

CIN [(value)]. *Confidence interval*. If the CIN keyword is specified alone, the default 95% confidence interval is computed. Optionally, CIN may be followed by a value in parentheses, where 0 ≤ value < 100.

SUBPOP Subcommand

SUBPOP specifies subpopulations for which analyses are to be performed.

- The set of subpopulations is defined by specifying a single categorical variable or specifying two or more categorical variables, separated by the BY keyword, whose values are crossed.
- For example, /SUBPOP TABLE = A defines subpopulations based on the levels of variable *A*.
- For example, /SUBPOP TABLE = A BY B defines subpopulations based on crossing the levels of variables *A* and *B*.
- A maximum of 17 variables may be specified.
- Numeric or string variables may be specified.
- All specified variables must be unique.
- Stratification or cluster variables may be specified, but no other plan file variables are allowed on the SUBPOP subcommand.
- Analysis variables may not be specified on the SUBPOP subcommand.
- The BY keyword is used to separate variables.

The DISPLAY keyword specifies the layout of results for subpopulations.

LAYERED. *Results for all subpopulations are displayed in the same table*. This is the default.

SEPARATE. *Results for different subpopulations are displayed in different tables.*

MISSING Subcommand

MISSING specifies how missing values are handled.

- All design variables must have valid data. Cases with invalid data for any design variable are deleted from the analysis.

The SCOPE keyword specifies which cases are used in the analyses. This specification is applied to analysis variables but not design variables.

ANALYSIS. *Each statistic is based on all valid data for the analysis variable(s) used in computing the statistic*. Ratios are computed by using all cases with valid data for both of the specified variables. Statistics for different variables may be based on different sample sizes. This setting is the default.

LISTWISE. *Only cases with valid data for all analysis variables are used in computing any statistics*. Statistics for different variables are always based on the same sample size.

The CLASSMISSING keyword specifies whether user-missing values are treated as valid. This specification is applied only to categorical design variables (strata, cluster, and subpopulation variables).

EXCLUDE. *Exclude user-missing values among the strata, cluster, and subpopulation variables*. This setting is the default.

INCLUDE. *Include user-missing values among the strata, cluster, and subpopulation variables*. Treat user-missing values for these variables as valid data.

CSGLM

CSGLM is available in SPSS Statistics Premium Edition or the Complex Samples option.

Note: Square brackets that are used in the CSGLM syntax chart are required parts of the syntax and are not used to indicate optional elements. Equals signs (=) that are used in the syntax chart are required elements. All subcommands, save the PLAN subcommand, are optional.

```
CSGLM dependent var BY factor list WITH covariate list

  /PLAN FILE = file

  /JOINTPROB FILE = file

  /MODEL effect list

  /INTERCEPT INCLUDE = {YES**} SHOW = {YES**}
                       {NO   }        {NO   }
                       {ONLY }

  /CUSTOM LABEL = "label"

           LMATRIX = {number effect list effect list ...; ...}
                     {number effect list effect list ...     }
                     {effect list effect list ...; ...       }
                     {effect list effect list ...            }
                     {ALL list; ALL ...                      }
                     {ALL list                               }

           KMATRIX = {number; number; ...}
                     {number            }

  /CUSTOM ...

  /EMMEANS TABLES = {factor          }
                    {factor*factor...}

           OTHER = [varname (value) varname (value) ...]

           COMPARE = factor

           CONTRAST = {SIMPLE** (value)        }
                      {DEVIATION (value)       }
                      {DIFFERENCE              }
                      {HELMERT                 }
                      {REPEATED                }
                      {POLYNOMIAL (number list)}

  /EMMEANS ...

  /CRITERIA CILEVEL = {95** } DF = n SINGULAR = {1E-12**}
                      {value}                   {value  }

  /STATISTICS PARAMETER SE TTEST CINTERVAL DEFF DEFFSQRT

  /TEST TYPE = {F**         } PADJUST = {LSD**        }
               {ADJF        }           {BONFERRONI   }
               {CHISQUARE   }           {SEQBONFERRONI}
               {ADJCHISQUARE}           {SIDAK        }
                                        {SEQSIDAK     }

  /DOMAIN VARIABLE = varname (value)

  /MISSING CLASSMISSING = {EXCLUDE**}
                          {INCLUDE  }

  /PRINT SAMPLEINFO** VARIABLEINFO** SUMMARY**

         GEF LMATRIX COVB CORB NONE

  /SAVE PRED(varname) RESID(varname)

  /OUTFILE {COVB='savfile'|'dataset'} {MODEL = 'file'    }
           {CORB='savfile'|'dataset'} {PARAMETER = 'file'}
```

**Default if the keyword or subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 13.0

- Command introduced.

Example

```
CSGLM y BY a b c WITH x
  /PLAN FILE='/survey/myfile.csplan'.
```

Overview

CSGLM performs linear regression analysis, as well as analysis of variance and covariance, for samples that are drawn by complex sampling methods. The procedure estimates variances by taking into account the sample design that is used to select the sample, including equal probability and probability proportional to size (PPS) methods, and with replacement (WR) and without replacement (WOR) sampling procedures. Optionally, CSGLM performs analyses for a subpopulation.

Basic Specification

- The basic specification is a variable list (identifying the dependent variable, the factors, if any, and the covariates, if any) and a PLAN subcommand with the name of a complex sample analysis plan file, which may be generated by the CSPLAN procedure.
- The default model includes the intercept term, main effects for any factors, and any covariates.
- The basic specification displays summary information about the sample design, *R*-square and root mean square error for the model, regression coefficient estimates and *t* tests, and Wald *F* tests for all model effects. Additional subcommands must be used for other results.

Operations

- CSGLM computes linear model estimates for sampling designs that are supported by the CSPLAN and CSSELECT procedures.
- The input dataset must contain the variables to be analyzed and variables that are related to the sampling design.
- The complex sample analysis plan file provides an analysis plan based on the sampling design.
- By default, CSGLM uses a model that includes the intercept term, main effects for any factors, and any covariates.
- Other effects, including interaction and nested effects, may be specified by using the MODEL subcommand.
- The default output for the specified model is summary information about the sample design, *R*-square and root mean square error, regression coefficient estimates and *t* tests, and Wald *F* tests for all effects.
- WEIGHT and SPLIT FILE settings are ignored by the CSGLM procedure.

Syntax Rules

- The dependent variable and PLAN subcommand are required. All other variables and subcommands are optional.
- Multiple CUSTOM and EMMEANS subcommands may be specified; each subcommand is treated independently. All other subcommands may be specified only once.
- The EMMEANS subcommand may be specified without options. All other subcommands must be specified with options.
- Each keyword may be specified only once within a subcommand.
- Subcommand names and keywords must be spelled in full.
- Equals signs (=) that are shown in the syntax chart are required.
- Subcommands may be specified in any order.
- The dependent variable and covariates must be numeric, but factors and the subpopulation variable can be numeric or string variables.
- Across the dependent, factor, and covariate variable lists, a variable may be specified only once.
- Plan file and subpopulation variables may not be specified on the variable list.

- Minimum syntax is a dependent variable and the PLAN subcommand. This specification fits an intercept-only model.

Limitations

- WEIGHT and SPLIT FILE settings are ignored with a warning by the CSGLM procedure.

CSGLM Variable List

The variable list specifies the dependent variable, the factors, and the covariates in the model.

- The dependent variable must be the first specification on CSGLM.
- The names of the factors and covariates, if any, follow the dependent variable. Specify any factors following the keyword BY. Specify any covariates following the keyword WITH.
- The dependent variable and covariates must be numeric, but factors can be numeric or string variables.
- Each variable may be specified only once on the variable list.
- Plan file and subpopulation variables may not be specified on the variable list.

PLAN Subcommand

The PLAN subcommand specifies the name of an XML file containing analysis design specifications. This file is written by the CSPLAN procedure.

- The PLAN subcommand is required.

FILE. *Specifies the name of an external file*.

JOINTPROB Subcommand

The JOINTPROB subcommand is used to specify the file or dataset containing the first stage joint inclusion probabilities for UNEQUAL_WOR estimation. The CSSELECT procedure writes this file in the same location and with the same name (but different extension) as the plan file. When UNEQUAL_WOR estimation is specified, the CSGLM procedure will use the default location and name of the file unless the JOINTPROB subcommand is used to override them.

FILE. *Specifies the name of the file or dataset containing the joint inclusion probabilities*.

MODEL Subcommand

The MODEL subcommand is used to specify the effects to be included in the model. Use the INTERCEPT subcommand to control whether the intercept is included.

- The MODEL subcommand defines the cells in a design. In particular, cells are defined by all of the possible combinations of levels of the factors in the design. The number of cells equals the product of the number of levels of all the factors. A design is balanced if each cell contains the same number of cases. CSGLM can analyze balanced and unbalanced designs.
- The format is a list of effects to be included in the model, separated by spaces or commas.
- If the MODEL subcommand is not specified, CSGLM uses a model that includes the intercept term (unless it is excluded on the INTERCEPT subcommand), main effects for any factors, and any covariates.
- To include a term for the main effect of a factor, enter the name of the factor.
- To include a term for an interaction between factors, use the keyword BY or the asterisk (*) to join the factors that are involved in the interaction. For example, A*B means a two-way interaction effect of A

and B, where A and B are factors. A*A is not allowed because factors inside an interaction effect must be distinct.

- To include a term for nesting one effect within another effect, use a pair of parentheses. For example, A(B) means that A is nested within B. When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, A(B)(C) is not valid.
- Multiple nesting is allowed. For example, A(B(C)) means that B is nested within C, and A is nested within B(C).
- Interactions between nested effects are not valid. For example, neither A(C)*B(C) nor A(C)*B(D) is valid.
- To include a covariate term in the design, enter the name of the covariate.
- Covariates can be connected, but not nested, through the * operator to form another covariate effect. Interactions among covariates such as X1*X1 and X1*X2 are valid, but X1(X2) is not.
- Factor and covariate effects can be connected only by the * operator. Suppose A and B are factors, and X1 and X2 are covariates. Examples of valid factor-by-covariate interaction effects are A*X1, A*B*X1, X1*A(B), A*X1*X1, and B*X1*X2.

INTERCEPT Subcommand

The INTERCEPT subcommand controls whether an intercept term is included in the model. This subcommand can also be used to display or suppress the intercept term in output tables.

INCLUDE Keyword

The INCLUDE keyword specifies whether the intercept is included in the model, or the keyword requests the intercept-only model.

YES. *The intercept is included in the model.* This setting is the default.

NO. *The intercept is not included in the model.* If no factors or covariates are defined, specifying INCLUDE = NO is invalid syntax.

ONLY. *The intercept-only model is fit.* If the MODEL subcommand is specified, specifying INCLUDE = ONLY is invalid syntax.

SHOW Keyword

The SHOW keyword specifies whether the intercept is displayed or suppressed in output tables.

YES. *The intercept is displayed in output tables.* This setting is the default.

NO. *The intercept is not displayed in output tables.* If INCLUDE = NO or ONLY is specified, SHOW = NO is ignored.

Example

```
CSGLM y BY a b c
  /PLAN FILE='/survey/myfile.csplan'
  /INTERCEPT INCLUDE = ONLY.
```

- The preceding syntax defines the model space using factors A, B, and C but fits the intercept-only model.

CUSTOM Subcommand

The CUSTOM subcommand defines custom hypothesis tests by specifying the **L** matrix (contrast coefficients matrix) and the **K** matrix (contrast results matrix) in the general form of the linear hypothesis **LB** = **K**. The vector **B** is the parameter vector in the linear model.

- Multiple CUSTOM subcommands are allowed. Each subcommand is treated independently.

- An optional label may be specified by using the LABEL keyword. The label is a string with a maximum length of 255 characters. Only one label can be specified.
- Either the LMATRIX or KMATRIX keyword, or both, must be specified.

  LMATRIX. *Contrast coefficients matrix.* This matrix specifies coefficients of contrasts, which can be used for studying the effects in the model. An **L** matrix can be specified by using the LMATRIX keyword.

  KMATRIX. *Contrast results matrix.* This matrix specifies the results of the linear hypothesis. A **K** matrix can be specified by using the KMATRIX keyword.
- The number of rows in the **L** and **K** matrices must be equal.
- A custom hypothesis test can be formed by specifying an **L** or **K** matrix, or both. If only one matrix is specified, the unspecified matrix uses the defaults described below.
- If KMATRIX is specified but LMATRIX is not specified, the **L** matrix is assumed to be the row vector corresponding to the intercept in the estimable function, provided that INCLUDE = YES or ONLY is specified on the INTERCEPT subcommand. In this case, the **K** matrix can be only a scalar matrix.
- The default **K** matrix is a zero matrix; that is, **LB** = **0** is assumed.
- There are three general formats that can be used on the LMATRIX keyword: (1) Specify a coefficient value for the intercept, followed optionally by an effect name and a list of real numbers. (2) Specify an effect name and a list of real numbers. (3) Specify keyword ALL and a list of real numbers. In all three formats, there can be multiple effect names (or instances of the keyword ALL) and number lists.
- Only valid effects in the default model or on the MODEL subcommand can be specified on the LMATRIX keyword.
- The length of the list of real numbers on the LMATRIX keyword must be equal to the number of parameters (including the redundant parameters) corresponding to the specified effect. For example, if the effect A*B takes up six columns in the design matrix, the list after A*B must contain exactly six numbers.
- When ALL is specified, the length of the list that follows ALL must be equal to the total number of parameters (including the redundant parameters) in the model.
- Effects that are in the model but not specified on the LMATRIX keyword are assumed to have entries of 0 in the corresponding columns of the **L** matrix.
- When an **L** matrix is being defined, a number can be specified as a fraction with a positive denominator. For example, 1/3 and –1/3 are valid, but 1/–3 is invalid.
- A semicolon (;) indicates the end of a row in the **L** matrix.
- The format for the KMATRIX keyword is one or more real numbers. If more than one number is specified, then separate adjacent numbers using a semicolon (;). Each semicolon indicates the end of a row in the **K** matrix. Each number is the hypothesized value for a contrast, which is defined by a row in the **L** matrix.
- For the KMATRIX keyword to be valid, either the LMATRIX keyword, or INCLUDE = YES on the INTERCEPT subcommand, must be specified.

Example

Suppose that factors A and B each have three levels.

```
CSGLM y BY a b
  /PLAN FILE='/survey/myfile.csplan'
  /MODEL a b a*b
  /CUSTOM  LABEL = “Effect A”
     LMATRIX = a 1 0 -1
         a*b  1/3  1/3  1/3
              0    0    0
             -1/3 -1/3 -1/3;
         a 0 1 -1
        a*b  0    0    0
              1/3  1/3  1/3
             -1/3 -1/3 -1/3.
```

- The preceding syntax specifies a test of effect A.
- Because there are three levels in effect A, two independent contrasts can be formed at most; thus, there are two rows in the **L** matrix, separated by a semicolon (;).

- There are three levels each in effects A and B; thus, the interaction effect A*B takes nine columns in the design matrix.
- The first row in the **L** matrix tests the difference between levels 1 and 3 of effect A; the second row tests the difference between levels 2 and 3 of effect A.
- The KMATRIX keyword is not specified, so the null hypothesis value for both tests is 0.

Example

Suppose that factors A and B each have three levels.

```
CSGLM y BY a b
  /PLAN FILE='/survey/myfile.csplan'
  /CUSTOM LABEL = “Effect A”
     LMATRIX = a 1 0 -1; a 1 -1 0
  /CUSTOM LABEL = “Effect B”
     LMATRIX = b 1 0 -1; b 1 -1 0
     KMATRIX = 0; 0.
```

- The preceding syntax specifies tests of effects A and B.
- The MODEL subcommand is not specified, so the default model—which includes the intercept and main effects for A and B—is used.
- There are two CUSTOM subcommands; each subcommand specifies two rows in the **L** matrix.
- The first CUSTOM subcommand does not specify the KMATRIX keyword. By default, this subcommand tests whether the effect of factor A is 0.
- The second CUSTOM subcommand specifies the KMATRIX keyword. This subcommand tests whether the effect of factor B is 0.

EMMEANS Subcommand

The EMMEANS subcommand displays estimated marginal means of the dependent variable in the cells for the specified factors. Note that these means are predicted, not observed, means.

- Multiple EMMEANS subcommands are allowed. Each subcommand is treated independently.
- The EMMEANS subcommand may be specified with no additional keywords. The output for an empty EMMEANS subcommand is the overall estimated marginal mean of the dependent variable, collapsing over any factors, and with any covariates held at their overall means.

TABLES = option. Valid options are factors appearing on the factor list and crossed factors that are constructed of factors on the factor list. Crossed factors can be specified by using an asterisk (*) or the keyword BY. All factors in a crossed factor specification must be unique. If a factor or a crossing of factors is specified on the TABLES keyword, CSGLM collapses over any other factors before computing the estimated marginal means for the dependent variable. If the TABLES keyword is not specified, the overall estimated marginal mean of the dependent variable, collapsing over any factors, is computed.

OTHER = [option]. Specifies the covariate values to use when computing the estimated marginal means. If the OTHER keyword is used, it must be followed by an equals sign and one or more elements enclosed in square brackets. Valid elements are covariates appearing on the CSGLM covariate list, each of which must be followed by a numeric value or the keyword MEAN in parentheses.

- If a numeric value is used, the estimated marginal mean will be computed by holding the specified covariate at the supplied value. If the keyword MEAN is used, the estimated marginal mean will be computed by holding the covariate at its overall mean. If a covariate is not specified on the OTHER option, its overall mean will be used in estimated marginal mean calculations.
- Any covariate may occur only once on the OTHER keyword.

CONTRAST = type. Specifies the type of contrast that is desired among the levels of the factor that is given on the COMPARE keyword. This keyword creates an **L** matrix such that the columns corresponding to the factor match the contrast that is given. The other columns are adjusted so that the **L** matrix is estimable. The CONTRAST keyword is ignored if the COMPARE keyword is not specified.

COMPARE = factor. Compares levels of a factor specified on the TABLES keyword and displays results for each individual comparison as well as for the overall set of comparisons. If only one factor is specified on TABLES, COMPARE can be specified by itself; otherwise, the factor specification is required. In the latter case, levels of the specified factor are compared for each level of the other factors that are specified on TABLES. The type of comparison that is performed is determined by the CONTRAST keyword. The TABLES keyword must be specified for the COMPARE keyword to be valid.

CONTRAST Keyword

The contrast types that may be specified on the CONTRAST keyword are described below.

The CSGLM procedure sorts levels of the factor in ascending order and defines the highest level as the last level. (If the factor is a string variable, the value of the highest level is locale-dependent.)

SIMPLE (value). *Each level of the factor (except the highest level) is compared to the highest level.* SIMPLE is the default contrast type if the COMPARE keyword is specified.

- The SIMPLE keyword may be followed optionally by parentheses containing a value. Put the value inside a pair of quotation marks if the value is formatted (such as date or currency) or if the factor is of string type. If a value is specified, the factor level with that value is used as the omitted reference category. If the specified value does not exist in the data, a warning is issued and the highest level is used.
- For example: `CSGLM y BY a … /EMMEANS TABLES=a COMPARE=a CONTRAST=SIMPLE(1).` The specified contrast compares all levels of factor A (except level 1) to level 1. Simple contrasts are not orthogonal.

DEVIATION (value). *Each level of the factor (except the highest level) is compared to the grand mean.*

The DEVIATION keyword may be followed optionally by parentheses containing a value. Put the value inside a pair of quotation marks if the value is formatted (such as date or currency) or if the factor is of string type. If a value is specified, the factor level with that value is used as the omitted reference category. If the specified value does not exist in the data, a warning is issued and the highest level is used. For example: `CSGLM y BY a … /EMMEANS TABLES=a COMPARE=a CONTRAST=DEVIATION(1).` The specified contrast omits level 1 of A. Deviation contrasts are not orthogonal.

DIFFERENCE. *Each level of the factor (except the lowest level) is compared to the mean of previous levels.* In a balanced design, difference contrasts are orthogonal.

HELMERT. *Each level of the factor (except the highest level) is compared to the mean of subsequent levels.* In a balanced design, Helmert contrasts are orthogonal.

REPEATED. *Each level of the factor (except the highest level) is compared to the previous level.* Repeated contrasts are not orthogonal.

POLYNOMIAL (number list). *Polynomial contrasts.* The first degree of freedom contains the linear effect across the levels of the factor, the second contains the quadratic effect, and so on. By default, the levels are assumed to be equally spaced; the default metric is (1 2 . . . k), where k levels are involved.

- The POLYNOMIAL keyword may be followed optionally by parentheses containing a number list. Numbers in the list must be separated by spaces or commas. Unequal spacing may be specified by entering a metric consisting of one integer for each level of the factor. Only the relative differences between the terms of the metric matter. Thus, for example, (1 2 4) is the same metric as (2 3 5) or (20 30 50) because, in each instance, the difference between the second and third numbers is twice the difference between the first and second numbers. All numbers in the metric must be unique; thus, (1 1 2) is not valid.
- For example: `CSGLM y BY a … /EMMEANS TABLES=a COMPARE=a CONTRAST=POLYNOMIAL(1 2 4).` Suppose that factor A has three levels. The specified contrast indicates that the three levels of A are actually in the proportion 1:2:4. In a balanced design, polynomial contrasts are orthogonal.

Orthogonal contrasts are particularly useful. In a balanced design, contrasts are orthogonal if the sum of the coefficients in each contrast row is 0 and if, for any pair of contrast rows, the products of corresponding coefficients sum to 0.

CRITERIA Subcommand

The CRITERIA subcommand controls statistical criteria and specifies numerical tolerance for checking singularity.

CILEVEL = value. *Confidence interval level for coefficient estimates and estimated marginal means.* Specify a value that is greater than or equal to 0 and less than 100. The default value is 95.

DF = value. *Sampling design degrees of freedom to use in computing* p *values for all test statistics.* Specify a positive number. The default value is the difference between the number of primary sampling units and the number of strata in the first stage of sampling.

SINGULAR = value . *Tolerance value used to test for singularity.* Specify a positive value. The default value is 10^{-12}.

STATISTICS Subcommand

The STATISTICS subcommand requests various statistics associated with the coefficient estimates.

- There are no default keywords on the STATISTICS subcommand. If this subcommand is not specified, no statistics that are listed below are displayed.

PARAMETER. *Coefficient estimates.*

SE. *Standard error for each coefficient estimate.*

TTEST. t *test for each coefficient estimate.*

CINTERVAL. *Confidence interval for each coefficient estimate.*

DEFF. *Design effect for each coefficient estimate.*

DEFFSQRT. *Square root of the design effect for each coefficient estimate.*

TEST Subcommand

The TEST subcommand specifies the type of test statistic and the method of adjusting the significance level to be used for hypothesis tests that are requested on the MODEL, CUSTOM, and EMMEANS subcommands.

TYPE Keyword

The TYPE keyword indicates the type of test statistic.

F. *Wald* F *test.* This is the default test statistic if the TYPE keyword is not specified.

ADJF. *Adjusted Wald* F *test.*

CHISQUARE. *Wald chi-square test.*

ADJCHISQUARE. *Adjusted Wald chi-square test.*

PADJUST keyword

The PADJUST keyword indicates the method of adjusting the significance level.

LSD. *Least significant difference.* This method does not control the overall probability of rejecting the hypotheses that some linear contrasts are different from the null hypothesis value(s). This setting is the default.

BONFERRONI. *Bonferroni.* This method adjusts the observed significance level for the fact that multiple contrasts are being tested.

SEQBONFERRONI. *Sequential Bonferroni.* This procedure is a sequentially step-down rejective Bonferroni procedure that is much less conservative in terms of rejecting individual hypotheses but maintains the same overall significance level.

SIDAK. *Sidak.* This method provides tighter bounds than the Bonferroni approach.

SEQSIDAK. *Sequential Sidak.* This procedure is a sequentially rejective step-down rejective Sidak procedure that is much less conservative in terms of rejecting individual hypotheses but maintains the same overall significance level.

DOMAIN Subcommand

The DOMAIN subcommand specifies the subpopulation for which the analysis is to be performed.

- Keyword VARIABLE, followed by an equals sign, a variable, and a value in parentheses, are required. Put the value inside a pair of quotation marks if the value is formatted (such as date or currency) or if the variable is of string type.
- The subpopulation is defined by all cases having the given value on the specified variable.
- Analyses are performed only for the specified subpopulation.
- For example, `DOMAIN VARIABLE = myvar (1)` defines the subpopulation by all cases for which variable *MYVAR* has value 1.
- The specified variable may be numeric or string and must exist at the time that the CSGLM procedure is invoked.
- Stratification or cluster variables may be specified, but no other plan file variables are allowed on the DOMAIN subcommand.
- Analysis variables may not be specified on the DOMAIN subcommand.

MISSING Subcommand

The MISSING subcommand specifies how missing values are handled.

- All design variables, as well as the dependent variable and any covariates, must have valid data. Cases with invalid data for any of these variables are deleted from the analysis.
- The CLASSMISSING keyword specifies whether user-missing values are treated as valid. This specification is applied to categorical design variables (i.e., strata, cluster, and subpopulation variables) and any factors.

EXCLUDE. *Exclude user-missing values among the strata, cluster, subpopulation, and factor variables.* This setting is the default.

INCLUDE. *Include user-missing values among the strata, cluster, subpopulation, and factor variables.* Treat user-missing values for these variables as valid data.

PRINT Subcommand

The PRINT subcommand is used to display optional output.

- If the PRINT subcommand is not specified, the default output includes sample information, variable and factor information, and model summary statistics.
- If the PRINT subcommand is specified, CSGLM displays output only for those keywords that are specified.

SAMPLEINFO. *Sample information table.* Displays summary information about the sample, including the unweighted count and the population size. This output is default output if the PRINT subcommand is not specified.

VARIABLEINFO. *Variable information.* Displays summary information about the dependent variable, covariates, and factors. This output is default output if the PRINT subcommand is not specified.

SUMMARY. *Model summary statistics.* Displays R^2 and root mean squared error statistics. This output is default output if the PRINT subcommand is not specified.

GEF. *General estimable function(s) table.*

LMATRIX. *Set of contrast coefficients (L) matrices.*

COVB. *Covariance matrix for regression coefficients.*

CORB. *Correlation matrix for regression coefficients.*

NONE. *No PRINT subcommand output.* None of the PRINT subcommand output is displayed. However, if NONE is specified with one or more other keywords, the other keywords override NONE.

SAVE Subcommand

The SAVE subcommand adds predicted or residual values to the active dataset.

- Specify one or more temporary variables, each variable followed by an optional new name in parentheses.
- The optional names must be unique, valid variable names.
- If new names are not specified, CSGLM uses the default names. If the default names conflict with existing variable names, a suffix is added to the default name to make it unique.

PRED. *Saves predicted values.* The default variable name is Predicted.

RESID. *Saves residuals.* The default variable name is Residual.

OUTFILE Subcommand

The OUTFILE subcommand saves an external IBM SPSS Statistics data file or previously declared dataset containing the parameter covariance or correlation matrix with parameter estimates, standard errors, significance values, and sampling design degrees of freedom. It also saves the parameter estimates and the parameter covariance matrix in XML format.

- At least one keyword and a filename are required. Specify the keyword followed by a quoted file specification.
- The COVB and CORB keywords are mutually exclusive, as are the MODEL and PARAMETER keywords.
- The filename must be specified in full. CSGLM does not supply an extension.
- For COVB and CORB, you can specify a previously declared dataset name (DATASET DECLARE command) instead of a file specification.

COVB = 'savfile'|'dataset'. *Writes the parameter covariance matrix and other statistics to an external IBM SPSS Statistics datafile or previously declared dataset.*

CORB = 'savfile'|'dataset'. *Writes the parameter correlation matrix and other statistics to an external IBM SPSS Statistics data file or previously declared dataset.*

MODEL = 'file'. *Writes the parameter estimates and the parameter covariance matrix to an XML file.*

PARAMETER = 'file'. *Writes the parameter estimates to an XML file.*

CSLOGISTIC

CSLOGISTIC is available in SPSS Statistics Premium Edition or the Complex Samples option.

Note: Square brackets that are used in the CSLOGISTIC syntax chart are required parts of the syntax and are not used to indicate optional elements. Equals signs (=) that are used in the syntax chart are required elements. All subcommands, save the PLAN subcommand, are optional.

```
CSLOGISTIC dependent var ({LOW   }) BY factor list WITH covariate list
                         {HIGH**}
                         {value }

  /PLAN FILE = file

  /JOINTPROB FILE = file

  /MODEL effect list

  /INTERCEPT INCLUDE = {YES**} SHOW = {YES**}
                       {NO   }        {NO   }
                       {ONLY }

  /CUSTOM LABEL = "label"

     LMATRIX = {number effect list effect list ...; ...}
               {number effect list effect list ...     }
               {effect list effect list ...; ...       }
               {effect list effect list ...            }
               {ALL list; ALL ...                      }
               {ALL list                               }

     KMATRIX = {number; number; ...}
               {number             }

  /CUSTOM ...

  /ODDSRATIOS {FACTOR = [varname ({LOW   }) varname ...] }
                                  {HIGH**}
                                  {value }

              {COVARIATE = [varname ({1**        }) varname ...]}
                                     {number list}

               CONTROL = [varname (value) varname (value) ...]

  /ODDSRATIOS ...

  /CRITERIA CHKSEP = {20**} CILEVEL = {95** } DF = n
                     {n   }           {value}

            LCONVERGE = [{0**   } {RELATIVE**}] MXITER = {100**}
                    {value} {ABSOLUTE  }           {n    }

            MXSTEP = {5**} PCONVERGE = [{1E-6**} {RELATIVE**}]
                     {n  }             {value } {ABSOLUTE }

            SINGULAR = {1E-12**}
                       {value  }

  /STATISTICS PARAMETER EXP SE TTEST CINTERVAL DEFF DEFFSQRT

  /TEST TYPE = {F**         } PADJUST = {LSD**        }
               {ADJF        }           {BONFERRONI   }
               {CHISQUARE   }           {SEQBONFERRONI}
               {ADJCHISQUARE}           {SIDAK        }
                                        {SEQSIDAK     }


  /DOMAIN VARIABLE = varname (value)

  /MISSING CLASSMISSING = {EXCLUDE**}
                          {INCLUDE  }

  /PRINT SAMPLEINFO** VARIABLEINFO** SUMMARY**

         HISTORY({1**}) GEF LMATRIX COVB CORB CLASSTABLE NONE
                 {n  }

  /SAVE PREDPROB(rootname:{25**}) PREDVAL(varname)
                          {n   }

  /OUTFILE {COVB = 'savfile'|'dataset'} {MODEL = 'file'    }
           {CORB = 'savfile'|'dataset'} {PARAMETER = 'file'}
```

**Default if the keyword or subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 13.0

- Command introduced.

Release 17.0

- Added support for SET THREADS.

Example

```
CSLOGISTIC y BY a b c WITH x
  /PLAN FILE='/survey/myfile.csplan'.
```

Overview

CSLOGISTIC performs logistic regression analysis on a binary or multinomial dependent variable using the generalized link function for samples that are drawn by complex sampling methods. The procedure estimates variances by taking into account the sample design that is used to select the sample, including equal probability and probability proportional to size (PPS) methods, and with replacement (WR) and without replacement (WOR) sampling procedures. Optionally, CSLOGISTIC performs analyses for a subpopulation.

Basic Specification

- The basic specification is a variable list (identifying the dependent variable, the factors, if any, and the covariates, if any) and a PLAN subcommand with the name of a complex sample analysis plan file, which may be generated by the CSPLAN procedure.
- The default model includes the intercept term, main effects for any factors, and any covariates.
- The basic specification displays summary information about the sample and all analysis variables, model summary statistics, and Wald *F* tests for all model effects. Additional subcommands must be used for other output.

Operations

- CSLOGISTIC performs logistic regression analysis for sampling designs that are supported by the CSPLAN and CSSELECT procedures.
- The input dataset must contain the variables to be analyzed and variables that are related to the sampling design.
- The complex sample analysis plan file provides an analysis plan based on the sampling design.
- By default, CSLOGISTIC uses a model that includes the intercept term, main effects for any factors, and any covariates.
- Other effects, including interaction and nested effects, may be specified by using the MODEL subcommand.
- The default output for the specified model is summary information about the sample and all analysis variables, model summary statistics, and Wald *F* tests for all model effects.
- WEIGHT and SPLIT FILE settings are ignored by the CSLOGISTIC procedure.
- This procedure uses the multithreaded options specified by SET THREADS.

Syntax Rules

- The dependent variable and PLAN subcommand are required. All other variables and subcommands are optional.
- Multiple CUSTOM and ODDSRATIOS subcommands may be specified; each subcommand is treated independently. All other subcommands may be specified only once.
- Empty subcommands are not allowed; all subcommands must be specified with options.
- Each keyword may be specified only once within a subcommand.
- Subcommand names and keywords must be spelled in full.
- Equals signs (=) that are shown in the syntax chart are required.

- Subcommands may be specified in any order.
- The dependent variable, factors, and the subpopulation variable can be numeric or string variables, but covariates must be numeric.
- Across the dependent, factor, and covariate variable lists, a variable may be specified only once.
- Plan file and subpopulation variables may not be specified on the variable list.
- Minimum syntax is a dependent variable and the PLAN subcommand. This specification fits an intercept-only model.

Limitations

- WEIGHT and SPLIT FILE settings are ignored with a warning by the CSLOGISTIC procedure.

CSLOGISTIC Variable List

The variable list specifies the dependent variable and reference category, the factors, and the covariates in the model.

- The dependent variable must be the first specification on CLOGISTIC.
- The dependent variable can be numeric or string.
- The CSLOGISTIC procedure sorts levels of the dependent variable in ascending order and defines the highest level as the last level. (If the dependent variable is a string variable, the value of the highest level is locale-dependent.) By default, the highest response category is used as the base (or reference) category.
- A custom reference category may be specified in parentheses immediately following the dependent variable.

  LOW. *The lowest category is the reference category.*

  HIGH. *The highest category is the reference category.* This setting is the default.

  value. *User-specified reference category.* The category that corresponds to the specified value is the reference category. Put the value inside a pair of quotation marks if the value is formatted (such as date or time) or if the dependent variable is of string type. Note, however, that this does not work for custom currency formats.
- If a value is specified as the reference category of the dependent variable, but the value does not exist in the data, a warning is issued and the default HIGH is used.
- The names of the factors and covariates, if any, follow the dependent variable. Specify any factors following the keyword BY. Specify any covariates following the keyword WITH.
- Factors can be numeric or string variables, but covariates must be numeric.
- Each variable may be specified only once on the variable list.
- Plan file and subpopulation variables may not be specified on the variable list.

PLAN Subcommand

The PLAN subcommand specifies the name of an XML file containing analysis design specifications. This file is written by the CSPLAN procedure.

- The PLAN subcommand is required.

FILE. *Specifies the name of an external file*.

JOINTPROB Subcommand

The JOINTPROB subcommand is used to specify the file or dataset containing the first stage joint inclusion probabilities for UNEQUAL_WOR estimation. The CSSELECT procedure writes this file in the same

location and with the same name (but different extension) as the plan file. When UNEQUAL_WOR estimation is specified, the CSLOGISTIC procedure will use the default location and name of the file unless the JOINTPROB subcommand is used to override them.

FILE. *Specifies the name of the file or dataset containing the joint inclusion probabilities.*

MODEL Subcommand

The MODEL subcommand is used to specify the effects to be included in the model. Use the INTERCEPT subcommand to control whether the intercept is included.

- The MODEL subcommand defines the cells in a design. In particular, cells are defined by all of the possible combinations of levels of the factors in the design. The number of cells equals the product of the number of levels of all the factors. A design is balanced if each cell contains the same number of cases. CSLOGISTIC can analyze balanced and unbalanced designs.
- The format is a list of effects to be included in the model, separated by spaces or commas.
- If the MODEL subcommand is not specified, CSLOGISTIC uses a model that includes the intercept term (unless it is excluded on the INTERCEPT subcommand), main effects for any factors, and any covariates.
- To include a term for the main effect of a factor, enter the name of the factor.
- To include a term for an interaction between factors, use the keyword BY or the asterisk (*) to join the factors that are involved in the interaction. For example, A*B means a two-way interaction effect of A and B, where A and B are factors. A*A is not allowed because factors that are inside an interaction effect must be distinct.
- To include a term for nesting one effect within another effect, use a pair of parentheses. For example, A(B) means that A is nested within B. When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, A(B)(C) is not valid.
- Multiple nesting is allowed. For example, A(B(C)) means that B is nested within C, and A is nested within B(C).
- Interactions between nested effects are not valid. For example, neither A(C)*B(C) nor A(C)*B(D) is valid.
- To include a covariate term in the design, enter the name of the covariate.
- Covariates can be connected, but not nested, through the * operator to form another covariate effect. Interactions among covariates such as X1*X1 and X1*X2 are valid, but X1(X2) is not.
- Factor and covariate effects can be connected only by the * operator. Suppose A and B are factors, and X1 and X2 are covariates. Examples of valid factor-by-covariate interaction effects are A*X1, A*B*X1, X1*A(B), A*X1*X1, and B*X1*X2.

INTERCEPT Subcommand

The INTERCEPT subcommand controls whether an intercept term is included in the model. This subcommand can also be used to display or suppress the intercept term in output tables.

INCLUDE Keyword

The INCLUDE keyword specifies whether the intercept is included in the model, or the keyword requests the intercept-only model.

YES. *The intercept is included in the model.* This setting is the default.

NO. *The intercept is not included in the model.* If no factors or covariates are defined, specifying INCLUDE = NO is invalid syntax.

ONLY. *The intercept-only model is fit.* If the MODEL subcommand is specified, specifying INCLUDE = ONLY is invalid syntax.

SHOW Keyword

The SHOW keyword specifies whether the intercept is displayed or suppressed in output tables.

YES. *The intercept is displayed in output tables.* This setting is the default.

NO. *The intercept is not displayed in output tables.* If INCLUDE = NO or ONLY is specified, SHOW = NO is ignored.

Example

```
CSLOGISTIC y BY a b c
  /PLAN FILE='/survey/myfile.csplan'
  /INTERCEPT INCLUDE = ONLY.
```

- The preceding syntax defines the model space using factors A, B, and C but fits the intercept-only model.

CUSTOM Subcommand

The CUSTOM subcommand defines custom hypothesis tests by specifying the **L** matrix (contrast coefficients matrix) and the **K** matrix (contrast results matrix) in the general form of the linear hypothesis **LB** = **K**. The vector **B** is the parameter vector in the linear model.

For a binary dependent variable, CSLOGISTIC models a single logit. In this case, there is one set of parameters associated with the logit.

For a multinomial dependent variable with K levels, CSLOGISTIC models K−1 logits. In this case, there are K−1 sets of parameters, each associated with a different logit. The CUSTOM subcommand allows you to specify an **L** matrix in which the same or different contrast coefficients are used across logits.

- Multiple CUSTOM subcommands are allowed. Each subcommand is treated independently.
- An optional label may be specified by using the LABEL keyword. The label is a string with a maximum length of 255 characters. Only one label can be specified.
- Either the LMATRIX or KMATRIX keyword, or both, must be specified.

  LMATRIX. *Contrast coefficients matrix.* This matrix specifies coefficients of contrasts, which can be used for studying the effects in the model. An **L** matrix can be specified by using the LMATRIX keyword.

  KMATRIX. *Contrast results matrix.* This matrix specifies the results of the linear hypothesis. A **K** matrix can be specified by using the KMATRIX keyword.
- The number of rows in the **L** and **K** matrices must be equal.
- A custom hypothesis test can be formed by specifying an **L** or **K** matrix, or both. If only one matrix is specified, the unspecified matrix uses the defaults described below.
- If KMATRIX is specified but LMATRIX is not specified, the **L** matrix is assumed to be the row vector corresponding to the intercept in the estimable function, provided that INCLUDE = YES or ONLY is specified on the INTERCEPT subcommand.
- The default **K** matrix is a zero matrix; that is, **LB = 0** is assumed.
- There are three general formats that can be used on the LMATRIX keyword: (1) Specify a coefficient value for the intercept, followed optionally by an effect name and a list of real numbers. (2) Specify an effect name and a list of real numbers. (3) Specify keyword ALL and a list of real numbers. In all three formats, there can be multiple effect names (or instances of the keyword ALL) and number lists.
- Only valid effects in the default model or on the MODEL subcommand can be specified on the LMATRIX keyword.
- The length of the list of real numbers on the LMATRIX keyword must be equal to the number of parameters (including the redundant parameters) corresponding to the specified effect. For example, if the effect A*B takes up six columns in the design matrix, the list after A*B must contain exactly six numbers.

- When ALL is specified, the length of the list that follows ALL must be equal to the total number of parameters (including the redundant parameters) in the model. For a binary dependent variable, the contrast coefficients for the one set of parameters must be listed following the ALL keyword. For a multinomial dependent variable with K levels, the contrast coefficients for the K–1 sets of parameters must be listed in order following the ALL keyword. That is, first list all parameters (including the redundant parameters) for the first logit, then list all parameters for the second logit, and so forth.
- In general, for a multinomial dependent variable with K levels, each contrast and its associated hypothesized value are generated separately for each of the K–1 logits; that is, any given contrast is generated K–1 times. However, if the LMATRIX ALL keyword is used to define a contrast, then that contrast and its associated hypothesized value are generated once, simultaneously covering all logits.
- Effects that are in the model but not specified on the LMATRIX keyword are assumed to have entries of 0 in the corresponding columns of the **L** matrix.
- When an **L** matrix is defined, a number can be specified as a fraction with a positive denominator. For example, 1/3 and –1/3 are valid, but 1/–3 is invalid.
- A semicolon (;) indicates the end of a row in the **L** matrix.
- The format for the KMATRIX keyword is one or more real numbers. If more than one number is specified, then separate adjacent numbers using a semicolon (;). Each semicolon indicates the end of a row in the **K** matrix. Each number is the hypothesized value for a contrast, which is defined by a row in the **L** matrix.
- For the KMATRIX keyword to be valid, either the LMATRIX keyword, or INCLUDE = YES on the INTERCEPT subcommand, must be specified.

Example

Suppose that dependent variable *Y* is binary, and factors A and B each have three levels.

```
CSLOGISTIC y BY a b
  /PLAN FILE='/survey/myfile.csplan'
  /MODEL a b a*b
  /CUSTOM  LABEL = 'Effect A'
     LMATRIX = a 1 0 -1
       a*b  1/3  1/3  1/3
            0    0    0
           -1/3 -1/3 -1/3;
       a 0 1 -1
       a*b  0    0    0
            1/3  1/3  1/3
           -1/3 -1/3 -1/3.
```

- The preceding syntax specifies a test of effect A.
- Because there are three levels in effect A, two independent contrasts can be formed at most; thus, there are two rows in the **L** matrix, separated by a semicolon (;).
- There are three levels each in effects A and B; thus, the interaction effect A*B takes nine columns in the design matrix.
- The first row in the **L** matrix tests the difference between levels 1 and 3 of effect A; the second row tests the difference between levels 2 and 3 of effect A.
- The KMATRIX keyword is not specified, so the null hypothesis value for both tests is 0.

Example

Suppose that dependent variable *Z* and factor A each have three levels.

```
CSLOGISTIC z BY a
  /PLAN FILE='/survey/myfile.csplan'
  /MODEL a
  /CUSTOM LABEL = 'Effect A'
     LMATRIX = a 1 0 -1;
               a 0 1 -1
```

- The dependent variable *Z* has three categories, so there will be two logits.
- The syntax specifies a model with an intercept and a main effect for factor A and a custom hypothesis test of effect A.

- Because the ALL option is not used on the LMATRIX keyword, the same set of contrast coefficients for the parameters will be used across both logits. That is, the resulting **L** matrix is block diagonal with the same 2-by-4 matrix of coefficients in each block. The equivalent LMATRIX keyword using the ALL option is as follows:

```
    LMATRIX = ALL 0 1  0 -1 0 0  0  0;
              ALL 0 0  1 -1 0 0  0  0;
              ALL 0 0  0  0 0 1  0 -1;
              ALL 0 0  0  0 0 0  1 -1
```

Example

Suppose that dependent variable *Z* has three categories, and factors A and B each have three levels.

```
CSLOGISTIC z BY a b
  /PLAN FILE='/survey/myfile.csplan'
  /CUSTOM LABEL = 'Effect A for All Logits'
     LMATRIX = a 1 0 -1; a 0 1 -1
  /CUSTOM LABEL = 'Effect A for 1st Logit, Effect B for 2nd Logit'
     LMATRIX = ALL 0 1 0 -1 0 0 0 0 0 0 0 1 0 -1;
       ALL 0 0 1 -1 0 0 0 0 0 0 0 1 -1
     KMATRIX = 0; 0.
```

- The dependent variable *Z* has three categories, so there will be two logits.
- The MODEL subcommand is not specified; thus the default model—which includes the intercept and main effects for A and B—is used.
- The first CUSTOM subcommand tests whether the effect of factor A is 0 across both logits.
- The second CUSTOM subcommand specifies different contrast coefficients for each logit. In particular, the **L** matrix tests the effect of factor A for the first logit and factor B for the second logit. The KMATRIX keyword explicitly states that each linear combination that is formed from the contrast coefficients and the parameter estimates is tested against the value 0.

ODDSRATIOS Subcommand

The ODDSRATIOS subcommand estimates odds ratios for the specified factor(s) or covariate(s). Note that these odds ratios are model-based and are not directly computed by using the observed data.

A separate set of odds ratios is computed for each category of the dependent variable (except the reference category).

If the FACTOR keyword is specified, the odds ratios compare the odds at each category j with the odds at category J, where J is the reference category defined in parentheses following the variable name of the factor. All other factors and covariates are fixed as defined on the CONTROL keyword.

If the COVARIATE keyword is specified, the odds ratios compare the odds at value x with the odds at value x + Δx, where Δx is the change in x defined in parentheses following the variable name of the covariate. To define the value x, specify the covariate and the value on the CONTROL keyword. All other factors and covariates are fixed as defined on the CONTROL keyword.

If a specified factor or covariate interacts with other predictors in the model, the odds ratios depend not only on the change in the specified variable but also on the values of the variables with which it interacts. If a specified covariate interacts with itself in the model (for example, X*X), the odds ratios depend on both the change in the covariate and the value of the covariate. The values of interacting factors and covariates can be customized by using the CONTROL keyword.

The CSLOGISTIC procedure sorts levels of each factor in ascending order and defines the highest level as the last level. (If the factor is a string variable, the value of the highest level is locale-dependent.)

- Multiple ODDSRATIOS subcommands are allowed. Each subcommand is treated independently.
- Either the FACTOR keyword and one or more factors, or the COVARIATE keyword and one or more covariates, but not both, are required. All other keywords are optional.
- The FACTOR, COVARIATE, and CONTROL keywords must be followed by an equals sign and one or more elements enclosed in square brackets.

- If a variable is specified on the FACTOR or COVARIATE keyword and is also specified on the CONTROL keyword, the CONTROL specification for that variable is ignored when the variable's odds ratios are computed. Thus, FACTOR = [A B] CONTROL = [A(1) B(2)] estimates odds ratios for factor A holding factor B at level 2 and for factor B holding factor A at level 1.

FACTOR = [option]. Valid options are one or more factors appearing on the factor list. Optionally, each factor may be followed by parentheses containing the level to use as the reference category when computing odds ratios. Keyword LOW or HIGH, or a value, may be specified. Put the value inside a pair of quotes if the value is formatted (such as date or currency) or if the factor is of string type. By default, the highest category is used as the reference category.

- If a value is specified but the value does not exist in the data, a warning is issued and the default HIGH is used.
- Any factor may occur only once on the FACTOR keyword.

COVARIATE = [option]. Valid options are one or more covariates appearing on the covariate list. Optionally, each covariate may be followed by parentheses containing one or more nonzero numbers giving unit(s) of change to use for covariates when computing odds ratios. Odds ratios are estimated for each distinct value. The default value is 1. Any covariate may occur only once on the COVARIATE keyword.

CONTROL= [option]. Specifies the factor and/or covariate values to use when computing odds ratios. Factors must appear on the factor list, and covariates must appear on the covariate list, of the CSLOGISTIC command.

- Factors must be followed by the keyword LOW or HIGH, or a value, in parentheses. Put the value inside a pair of quotation marks if the value is formatted (such as date or currency) or if the factor is of string type. If keyword LOW or HIGH is used, each odds ratio is computed by holding the factor at its lowest or highest level, respectively. If a value is used, each odds ratio is computed by holding the specified factor at the supplied value. If a factor is not specified on the CONTROL option, its highest category is used in odds ratio calculations.
- If a factor value is specified but the value does not exist in the data, a warning is issued and the default HIGH is used.
- Covariates must be followed by the keyword MEAN or a number in parentheses. If the keyword MEAN is used, each odds ratio is computed by holding the covariate at its overall mean. If a number is used, each odds ratio is computed by holding the specified covariate at the supplied value. If a covariate is not specified on the CONTROL option, its overall mean is used in odds ratio calculations.
- Any factor or covariate may occur only once on the CONTROL keyword.

Example

Suppose that dependent variable *Y* is binary; factor A has two levels; and factor B has three levels coded 1, 2, and 3.

```
CSLOGISTIC y BY a b WITH x
  /PLAN FILE='/survey/myfile.csplan'
  /MODEL a b a*b x
  /ODDSRATIOS FACTOR=[a] CONTROL=[b(1)]
  /ODDSRATIOS FACTOR=[a] CONTROL=[b(2)]
  /ODDSRATIOS FACTOR=[a] CONTROL=[b(3)].
```

- The default reference category (the highest category) is used for the dependent variable.
- The model includes the intercept, main effects for factors A and B, the A*B interaction effect, and the covariate X.
- Odds ratios are requested for factor A. Assuming the A*B interaction effect is significant, the odds ratio for factor A will differ across levels of factor B. The specified syntax requests three odds ratios for factor A; each odds ratio is computed at a different level of factor B.

Example

```
CSLOGISTIC y BY a b c WITH x
  /PLAN FILE='/survey/myfile.csplan'
```

```
/MODEL a b c x
/ODDSRATIOS COVARIATE=[x(1 3 5)].
```

- The preceding syntax will compute three odds ratios for covariate X.
- The parenthesized list following variable X provides the unit of change values to use when computing odds ratios. Odds ratios will be computed for X increasing by 1, 3, and 5 units.

CRITERIA Subcommand

The CRITERIA subcommand offers controls on the iterative algorithm that is used for estimation, and the subcommand specifies numerical tolerance for checking singularity.

CHKSEP = value . *Starting iteration for checking complete separation.* Specify a non-negative integer. This criterion is not used if the value is 0. The default value is 20.

CILEVEL = value. *Confidence interval level for coefficient estimates, exponentiated coefficient estimates, and odds ratio estimates.* Specify a value that is greater than or equal to 0 and less than 100. The default value is 95.

DF = value. *Sampling design degrees of freedom to use in computing* p *values for all test statistics.* Specify a positive number. The default value is the difference between the number of primary sampling units and the number of strata in the first stage of sampling.

LCONVERGE = [option] . *Log-likelihood function convergence criterion.* Convergence is assumed if the absolute or relative change in the log-likelihood function is less than the given value. This criterion is not used if the value is 0. Specify square brackets containing a non-negative number followed optionally by keyword ABSOLUTE or RELATIVE, which indicates the type of change. The default value is 0, and the default type is RELATIVE.

MXITER = value . *Maximum number of iterations.* Specify a non-negative integer. The default value is 100.

MXSTEP = value . *Maximum step-halving allowed.* Specify a positive integer. The default value is 5.

PCONVERGE = [option] . *Parameter estimates convergence criterion.* Convergence is assumed if the absolute or relative change in the parameter estimates is less than the given value. This criterion is not used if the value is 0. Specify square brackets containing a non-negative number followed optionally by keyword ABSOLUTE or RELATIVE, which indicates the type of change. The default value is 10^{-6}, and the default type is RELATIVE.

SINGULAR = value . *Tolerance value used to test for singularity.* Specify a positive value. The default value is 10^{-12}.

STATISTICS Subcommand

The STATISTICS subcommand requests various statistics that are associated with the coefficient estimates.

- There are no default keywords on the STATISTICS subcommand. If this subcommand is not specified, no statistics that are listed below are displayed.

PARAMETER. *Coefficient estimates.*

EXP. *The exponentiated coefficient estimates.*

SE. *Standard error for each coefficient estimate.*

TTEST. t *test for each coefficient estimate.*

CINTERVAL. *Confidence interval for each coefficient estimate and/or exponentiated coefficient estimate.*

DEFFSQRT. *Square root of the design effect for each coefficient estimate.*

DEFF. *Design effect for each coefficient estimate.*

TEST Subcommand

The TEST subcommand specifies the type of test statistic and the method of adjusting the significance level to be used for hypothesis tests that are requested on the MODEL and CUSTOM subcommands.

TYPE Keyword

The TYPE keyword indicates the type of test statistic.

F. *Wald* F *test.* This is the default test statistic if the TYPE keyword is not specified.

ADJF. *Adjusted Wald* F *test.*

CHISQUARE. *Wald chi-square test.*

ADJCHISQUARE. *Adjusted Wald chi-square test.*

PADJUST Keyword

The PADJUST keyword indicates the method of adjusting the significance level.

LSD. *Least significant difference.* This method does not control the overall probability of rejecting the hypotheses that some linear contrasts are different from the null hypothesis value(s). This setting is the default.

BONFERRONI. *Bonferroni.* This method adjusts the observed significance level for the fact that multiple contrasts are being tested.

SEQBONFERRONI. *Sequential Bonferroni.* This procedure is a sequentially step-down rejective Bonferroni procedure that is much less conservative in terms of rejecting individual hypotheses but maintains the same overall significance level.

SIDAK. *Sidak.* This method provides tighter bounds than the Bonferroni approach.

SEQSIDAK. *Sequential Sidak.* This procedure is a sequentially rejective step-down rejective Sidak procedure that is much less conservative in terms of rejecting individual hypotheses but maintains the same overall significance level.

DOMAIN Subcommand

The DOMAIN subcommand specifies the subpopulation for which the analysis is to be performed.

- Keyword VARIABLE, followed by an equals sign, a variable, and a value in parentheses, are required. Put the value inside a pair of quotation marks if the value is formatted (such as date or currency) or if the factor is of string type.
- The subpopulation is defined by all cases having the given value on the specified variable.
- Analyses are performed only for the specified subpopulation.
- For example, `DOMAIN VARIABLE = myvar (1)` defines the subpopulation by all cases for which variable *MYVAR* has value 1.
- The specified variable may be numeric or string and must exist at the time that the CSLOGISTIC procedure is invoked.
- Stratification or cluster variables may be specified, but no other plan file variables are allowed on the DOMAIN subcommand.
- Analysis variables may not be specified on the DOMAIN subcommand.

MISSING Subcommand

The MISSING subcommand specifies how missing values are handled.

- All design variables, as well as the dependent variable and any covariates, must have valid data. Cases with invalid data for any of these variables are deleted from the analysis.
- The CLASSMISSING keyword specifies whether user-missing values are treated as valid. This specification is applied to categorical design variables (i.e., strata, cluster, and subpopulation variables), the dependent variable, and any factors.

EXCLUDE. *Exclude user-missing values among the strata, cluster, subpopulation, and factor variables.* This setting is the default.

INCLUDE. *Include user-missing values among the strata, cluster, subpopulation, and factor variables.* Treat user-missing values for these variables as valid data.

PRINT Subcommand

The PRINT subcommand is used to display optional output.

- If the PRINT subcommand is not specified, the default output includes sample information, variable and factor information, and model summary statistics.
- If the PRINT subcommand is specified, CSLOGISTIC displays output only for those keywords that are specified.

SAMPLEINFO. *Sample information table.* Displays summary information about the sample, including the unweighted count and the population size. This output is default output if the PRINT subcommand is not specified.

VARIABLEINFO. *Variable information.* Displays summary information about the dependent variable, covariates, and factors. This output is default output if the PRINT subcommand is not specified.

SUMMARY. *Model summary statistics.* Displays pseudo-R^2 statistics. This output is default output if the PRINT subcommand is not specified.

HISTORY(n). *Iteration history.* Displays coefficient estimates and statistics at every *n*th iteration beginning with the zeroth iteration (the initial estimates). The default is to print every iteration (n = 1). The last iteration is always printed if HISTORY is specified, regardless of the value of *n*.

GEF. *General estimable function(s) table.*

LMATRIX. *Set of contrast coefficients (L) matrices.*

COVB. *Covariance matrix for regression coefficients.*

CORB. *Correlation matrix for regression coefficients.*

CLASSTABLE. *Classification table.* Displays frequencies of observed versus predicted response categories.

NONE. *No PRINT subcommand output.* None of the PRINT subcommand output is displayed. However, if NONE is specified with one or more other keywords, the other keywords override NONE.

SAVE Subcommand

The SAVE subcommand writes optional model variables to the active dataset.

- Specify one or more temporary variables, each variable followed by an optional new name in parentheses.
- The optional names must be unique, valid variable names.
- If new names are not specified, CSLOGISTIC generates a name using the temporary variable name with a suffix.

PREDPROB. *Predicted probability.* The user-specified or default name is treated as the rootname, and a suffix is added to get new unique variables names. The rootname can be followed by a colon and a positive integer giving the number of predicted probabilities to save. The predicted probabilities of the

first *n* response categories are saved. One predicted probability variable can be saved for each category of the dependent variable. The default rootname is PredictedProbability. The default *n* of predicted probabilities to save is 25. To specify *n* without a rootname, enter a colon before the number.

PREDVAL. *Predicted value.* The class or value that is predicted by the model. The optional variable name must be unique. If the default name is used and it conflicts with existing variable names, a suffix is added to the default name to make it unique. The default variable name is PredictedValue.

OUTFILE Subcommand

The OUTFILE subcommand saves an external IBM SPSS Statistics data file or previously declared dataset containing the parameter covariance or correlation matrix with parameter estimates, standard errors, significance values, and sampling design degrees of freedom. It also saves the parameter estimates and the parameter covariance matrix in XML format.

- At least one keyword and a file specification are required. The file specification should be enclosed in quotes.
- The COVB and CORB keywords are mutually exclusive, as are the MODEL and PARAMETER keywords.
- The filename must be specified in full. CSLOGISTIC does not supply an extension.
- For COVB and CORB, you can specify a previously declared dataset name (DATASET DECLARE command) instead of a file specification.

COVB = 'savfile'|'dataset'. *Writes the parameter covariance matrix and other statistics to an external IBM SPSS Statistics data file or previously declared dataset.*

CORB = 'savfile'|'dataset'. *Writes the parameter correlation matrix and other statistics to an external IBM SPSS Statistics data file or previously declared dataset.*

MODEL = 'file'. *Writes the parameter estimates and the parameter covariance matrix to an XML file.*

PARAMETER = 'file'. *Writes the parameter estimates to an XML file.*

CSORDINAL

CSORDINAL is available in SPSS Statistics Premium Edition or the Complex Samples option.

Note: Square brackets that are used in the CSORDINAL syntax chart are required parts of the syntax and are not used to indicate optional elements. Equals signs (=) that are used in the syntax chart are required elements. Except for the PLAN subcommand, all subcommands are optional.

```
CSORDINAL dependent varname ({ASCENDING**}) BY factor list
                            {DESCENDING }
          WITH covariate list

/PLAN FILE = 'file'

/JOINTPROB FILE = 'savfile' | 'dataset'

/MODEL effect-list

/LINK  FUNCTION = {CAUCHIT}]
                  {CLOGLOG}
                  {LOGIT**}
                  {NLOGLOG}
                  {PROBIT }

/CUSTOM LABEL = "label"

      LMATRIX = {list, effect list, effect list ...; ...}
                {list, effect list, effect list ...     }
                {effect list, effect list ...; ...      }
                {effect list, effect list ...           }
                {ALL list; ALL ...                      }
                {ALL list                               }

      KMATRIX = {number; number; ...}
                {number             }

/CUSTOM ...

/ODDSRATIOS  {FACTOR = [varname ({LOW   }) varname ...]       }
                               {HIGH**}
                               {value }

             {COVARIATE = [varname ({1**        }) varname ...]}
                                    {number list}

             CONTROL = [varname (value) varname (value) ...]

/ODDSRATIOS ...

/CRITERIA CHKSEP = {20**}] CILEVEL = {95** }] [DF = value]
                   {n   }            {value}

          LCONVERGE = [{0**  } {RELATIVE**}] METHOD = {FISHER(n)}
                       {value} {ABSOLUTE  }           {NEWTON** }

           MXITER = {100**} MXSTEP = {5**}
                    {n    }          {n  }

           PCONVERGE = [{1E-6**} {RELATIVE**}] SINGULAR = {1E-12**}
                        {value } {ABSOLUTE  }             {value  }

/STATISTICS PARAMETER EXP SE TTEST CINTERVAL DEFF DEFFSQRT

/NONPARALLEL TEST PARAMETER COVB

/TEST TYPE = {F**         } PADJUST = {LSD**        }
             {ADJF        }           {BONFERRONI   }
             {CHISQUARE   }           {SEQBONFERRONI}
             {ADJCHISQUARE}           {SIDAK        }
                                      {SEQSIDAK     }

/DOMAIN VARIABLE = varname (value)

/MISSING CLASSMISSING = {EXCLUDE**}
                        {INCLUDE  }

/PRINT SAMPLEINFO** VARIABLEINFO** SUMMARY**
       HISTORY({1**}) GEF LMATRIX COVB CORB
               {n  }
       CLASSTABLE NONE

/SAVE CUMPROB(rootname:{25**}) PREDPROB(rootname:{25**})
                       {n   }                   {n   }
      PREDVAL(varname) PREDVALPROB(varname) OBSVALPROB(varname)

/OUTFILE {COVB = 'savfile' | 'dataset'} {MODEL = 'file'    }
         {CORB = 'savfile' | 'dataset'} {PARAMETER = 'file'}
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 15.0

- Command introduced.

Release 17.0

- Added support for SET THREADS.

Example

```
CSORDINAL y BY a b c WITH x
  /PLAN FILE='/survey/myfile.csplan'.
```

Overview

CSORDINAL performs regression analysis on a binary or ordinal polytomous dependent variable using the selected cumulative link function for samples drawn by complex sampling methods. The procedure estimates variances by taking into account the sample design used to select the sample, including equal probability and probability proportional to size (PPS) methods and with replacement (WR) and without replacement (WOR) sampling procedures. Optionally, CSORDINAL performs analyses for a subpopulation.

Basic Specification

- The basic specification is a variable list identifying the dependent variable, the factors (if any), and the covariates (if any) and a PLAN subcommand with the name of a complex sample analysis plan file, which may be generated by the CSPLAN procedure.
- The default model includes threshold parameters, main effects for any factors, and any covariates.
- The basic specification displays summary information about the sample and all analysis variables, model summary statistics, and Wald *F* tests for all model effects. Additional subcommands must be used for other output.

Syntax Rules

- The dependent variable and PLAN subcommand are required. All other variables and subcommands are optional.
- Multiple CUSTOM and ODDSRATIOS subcommands may be specified; each is treated independently. All other subcommands may be specified only once.
- Empty subcommands are not allowed; all subcommands must be specified with options.
- Each keyword may be specified only once within a subcommand.
- Subcommand names and keywords must be spelled in full.
- Equals signs (=) shown in the syntax chart are required.
- Square brackets shown in the syntax chart are required parts of the syntax and are not used to indicate optional elements. (See the ODDSRATIOS and CRITERIA subcommands.)
- Subcommands may be specified in any order.
- The dependent variable, factors, and the subpopulation variable can be numeric or string variables, but covariates must be numeric.
- Across the dependent, factor, and covariate variable lists, a variable may be specified only once.
- Plan file and subpopulation variables may not be specified on the variable list.
- Minimum syntax is a dependent variable and the PLAN subcommand. This specification fits a thresholds-only model.

Operations

- CSORDINAL performs ordinal regression analysis for sampling designs supported by the CSPLAN and CSSELECT procedures.

- The input data set must contain the variables to be analyzed and variables related to the sampling design.
- The complex sample analysis plan file provides an analysis plan based on the sampling design.
- By default, CSORDINAL uses a model that includes thresholds, main effects for any factors, and any covariates.
- Other effects, including interaction and nested effects, may be specified using the MODEL subcommand.
- The default output for the specified model is summary information about the sample and all analysis variables, model summary statistics, and Wald *F* tests for all model effects.
- WEIGHT and SPLIT FILE settings are ignored by the CSORDINAL procedure.
- This procedure uses the multithreaded options specified by SET THREADS.

Limitations

- WEIGHT and SPLIT FILE settings are ignored with a warning by the CSORDINAL procedure.

Variable List

The variable list specifies the dependent variable with the categories order, the factors, and the covariates in the model.

- The dependent variable must be the first specification on CSORDINAL.
- The dependent variable can be numeric or string.
- The CSORDINAL procedure sorts levels of the dependent variable in ascending or descending order. (If the dependent variable is a string variable, then the order is locale-dependent.)
- Sorting order for the values of the dependent variable may be specified in parentheses immediately following the dependent variable.

  ASCENDING. *Sort dependent variable values in ascending order.* This is the default setting.

  DESCENDING. *Sort dependent variable values in descending order.*
- The names of the factors and covariates, if any, follow the dependent variable. Specify any factors following the keyword BY. Specify any covariates following the keyword WITH.
- Factors can be numeric or string variables, but covariates must be numeric.
- Each variable may be specified only once on the variable list.
- Plan file and subpopulation variables may not be specified on the variable list.

PLAN Subcommand

The PLAN subcommand specifies the name of an XML file containing analysis design specifications. This file is written by the CSPLAN procedure.

- The PLAN subcommand is required.

'FILE'. *Specifies the name of an external file.*

JOINTPROB Subcommand

The JOINTPROB subcommand is used to specify the file containing the first stage joint inclusion probabilities for UNEQUAL_WOR estimation. The CSSELECT procedure writes this file in the same location and with the same name (but different extension) as the plan file. When UNEQUAL_WOR estimation is specified, the CSORDINAL procedure will use the default location and name of the file unless the JOINTPROB subcommand is used to override them.

'FILE' | 'dataset'. *The name of the joint inclusion probabilities file.* It can be an external file or an open dataset.

MODEL Subcommand

The MODEL subcommand is used to specify the effects to be included in the model. Threshold parameters are included automatically. Their number is one less then the number of categories of the dependent variable found in the data.

- Specify a list of terms to be included in the model, separated by spaces or commas.
- If the MODEL subcommand is not specified, CSORDINAL uses a model that includes threshold parameters, main effects for any factors, and any covariates in the order specified on the variable list.
- To include a term for the main effect of a factor, enter the name of the factor.
- To include a term for an interaction among factors, use the keyword BY or the asterisk (*) to join the factors involved in the interaction. For example, A*B means a two-way interaction effect of A and B, where A and B are factors. A*A is not allowed because factors inside an interaction effect must be distinct.
- To include a term for nesting one factor within another, use a pair of parentheses. For example, A(B) means that A is nested within B. A(A) is not allowed because factors inside a nested effect must be distinct.
- Multiple nesting is allowed. For example, A(B(C)) means that B is nested within C, and A is nested within B(C). When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, A(B)(C) is not valid.
- Nesting within an interaction effect is valid. For example, A(B*C) means that A is nested within B*C.
- Interactions among nested effects are allowed. The correct syntax is the interaction followed by the common nested effect inside the parentheses. For example, interaction between A and B within levels of C should be specified as A*B(C) instead of A(C)*B(C).
- To include a covariate term in the design, enter the name of the covariate.
- Covariates can be connected, but not nested, through the * operator or using the keyword BY to form another covariate effect. Interactions among covariates such as X1*X1 and X1*X2 are valid, but X1(X2) is not.
- Factor and covariate effects can be connected in various ways except that no effects can be nested within a covariate effect. Suppose A and B are factors and X1 and X2 are covariates. Examples of valid combinations of factor and covariate effects are A*X1, A*B*X1, X1(A), X1(A*B), X1*A(B), X1*X2(A*B), and A*B*X1*X2.

LINK Subcommand

The LINK subcommand offers the choice of a cumulative link function to specify the model.

- The keyword FUNCTION, followed by an equals sign, and a link function keyword are required.
- If the subcommand is not specified, LOGIT is the default cumulative link function.
- Only a single cumulative link function can be specified.

CAUCHIT. *Cauchit function.* $f(x)=\tan(\pi(x-0.5))$

CLOGLOG . *Complementary log-log function.* $f(x)=\log(-\log(1-x)$

LOGIT. *Logit function.* $f(x)=\log(x / (1-x))$. This is the default link function.

NLOGLOG. *Negative log-log function.* $f(x)=-\log(-\log(x))$

PROBIT. *Probit function.* $f(x)=\Phi^{-1}(x)$, where Φ^{-1} is the inverse standard normal cumulative distribution function.

CUSTOM Subcommand

The CUSTOM subcommand defines custom hypothesis tests by specifying the **L** matrix (contrast coefficients matrix) and the **K** matrix (contrast results matrix) in the general form of the linear hypothesis **LB** = **K**. The vector **B** is the parameter vector in the cumulative link model.

For a binary dependent variable, CSORDINAL models a single threshold parameter and a set of regression parameters. For a polytomous ordinal dependent variable with *K* levels, CSORDINAL models a threshold parameter for each category except the last and a single set of regression parameters for all response categories. The CUSTOM subcommand allows you to specify an **L** matrix with contrast coefficients for all thresholds and regression parameters.

- Multiple CUSTOM subcommands are allowed. Each is treated independently.
- An optional label may be specified using the LABEL keyword. The label is a string with a maximum length of 255 characters. Only one label can be specified.
- The **L** matrix is the contrast coefficients matrix. This matrix specifies coefficients of contrasts, which can be used for studying the effects in the model. An **L** matrix must always be specified using the LMATRIX keyword.
- The **K** matrix is the contrast results matrix. This matrix specifies the results of the linear hypothesis. A **K** matrix can be specified using the KMATRIX keyword.
- The number of rows in the **L** and **K** matrices must be equal.
- The default **K** matrix is a zero matrix; that is, **LB** = **0** is assumed.
- There are three general formats that can be used on the LMATRIX keyword: (1) Specify coefficient values for thresholds, followed optionally by an effect name and a list of real numbers. (2) Specify an effect name and a list of real numbers. (3) Specify the keyword ALL and a list of real numbers. In all three formats, there can be multiple effect names (or instances of the keyword ALL) and number lists.
- When specifying threshold coeffients in the first or the third general format, a complete list of *K*–1 coefficient values must be given in the increasing threshold order.
- Only valid effects in the default model or on the MODEL subcommand can be specified on the LMATRIX keyword.
- The length of the list of real numbers on the LMATRIX keyword must be equal to the number of parameters (including the redundant ones) corresponding to the specified effect. For example, if the effect A*B takes up six columns in the design matrix, then the list after A*B must contain exactly six numbers.
- When ALL is specified, the length of the list that follows ALL must be equal to the total number of parameters (including the redundant ones) in the model. For a binary dependent variable, the contrast coefficients for the single threshold and all regression parameters must be listed following the ALL keyword. For a polytomous dependent variable with *K* levels, the contrast coefficients for the *K*–1 thresholds and all regression parameters must be listed in order following the ALL keyword.
- Effects that are in the model but not specified on the LMATRIX keyword are assumed to have entries of 0 in the corresponding columns of the **L** matrix.
- When defining an **L** matrix, a number can be specified as a fraction with a positive denominator—for example, 1/3 and –1/3 are valid, but 1/–3 is invalid.
- A semicolon (;) indicates the end of a row in the **L** matrix.
- The format for the KMATRIX keyword is one or more real numbers. If more than one number is specified, then separate adjacent numbers using a semicolon (;). Each semicolon indicates the end of a row in the **K** matrix. Each number is the hypothesized value for a contrast, which is defined by a row in the **L** matrix.
- If rows of the **L** matrix are not independent, a submatrix of **L** with independent rows is used for testing. Tested rows are indicated when the **K** matrix is not a zero matrix.

Example

Suppose that factors A and B each have three levels.

```
CSORDINAL y BY a b
  /PLAN FILE='/survey/myfile.csplan'
  /MODEL a b a*b
  /CUSTOM  LABEL = 'Effect A'
   LMATRIX = a 1 0 -1
             a*b  1/3  1/3  1/3
                    0    0    0
                -1/3 -1/3 -1/3;
             a 0 1 -1
             a*b    0    0    0
                  1/3  1/3  1/3
                -1/3 -1/3 -1/3.
```

- The preceding syntax specifies a test of effect A.
- Because there are three levels in effect A, at most two independent contrasts can be formed; thus, there are two rows in the **L** matrix, separated by a semicolon (;).
- There are three levels each in effects A and B; thus, the interaction effect A*B takes nine columns in the design matrix.
- The first row in the **L** matrix tests the difference between levels 1 and 3 of effect A; the second row tests the difference between levels 2 and 3 of effect A.
- The KMATRIX keyword is not specified, so the null hypothesis value for both tests is 0.

Example

Suppose that dependent variable Z and factor A each have three levels.

```
CSORDINAL z BY a
  /PLAN FILE='/survey/myfile.csplan'
  /MODEL a
  /CUSTOM  LABEL = 'Effect A'
   LMATRIX = a 1 0 -1;
             a 0 1 -1
   KMATRIX = 1; 1.
```

- The dependent variable Z has three categories, so there will be two thresholds.
- The syntax specifies a model with thresholds and a main effect for factor A, and a custom hypothesis test of effect A.
- Because the ALL option is not used on the LMATRIX keyword, threshold coefficients are set to zero. The equivalent LMATRIX keyword using the ALL option follows.

```
    LMATRIX = ALL 0 0  1  0 -1;
              ALL 0 0  0  1 -1
```

- The KMATRIX keyword is specified and the hypothesis that the difference between levels 1 and 3 and levels 2 and 3 of effect A are both equal to 1 is tested.

ODDSRATIOS Subcommand

The ODDSRATIOS subcommand estimates cumulative odds ratios for the specified factor(s) or covariate(s). The subcommand is available only for LOGIT link. For other link functions, the subcommand is ignored and a warning is issued. Note that these cumulative odds ratios are model-based and are not directly computed using the observed data.

A single cumulative odds ratio is computed for all categories of the dependent variable except the last; the proportional odds model postulates that they are all equal.

If the FACTOR keyword is specified, the cumulative odds ratios compare the cumulative odds at each factor category *j* with the cumulative odds at category *J*, where *J* is the reference category defined in parentheses following the variable name of the factor. All other factors and covariates are fixed as defined on the CONTROL keyword.

If the COVARIATE keyword is specified, the cumulative odds ratios compare the cumulative odds at value *x* with the cumulative odds at value $x + \Delta x$, where Δx is the change in *x* defined in parentheses following the variable name of the covariate. To define the value *x*, specify the covariate and the value on the

CONTROL keyword. The value of all other factors and covariates are fixed as defined on the CONTROL keyword also.

If a specified factor or covariate interacts with other predictors in the model, then the cumulative odds ratios depend not only on the change in the specified variable but also on the values of the variables with which it interacts. If a specified covariate interacts with itself in the model (for example, X*X), then the cumulative odds ratios depend on both the change in the covariate and the value of the covariate. The values of interacting factors and covariates can be customized using the CONTROL keyword.

The CSORDINAL procedure sorts levels of each factor in ascending order and defines the highest level as the last level. (If the factor is a string variable, then the value of the highest level is locale-dependent.)

- Multiple ODDSRATIOS subcommands are allowed. Each is treated independently.
- Either the FACTOR keyword and one or more factors, or the COVARIATE keyword and one or more covariates, but not both, are required. All other keywords are optional.
- The FACTOR, COVARIATE, and CONTROL keywords must be followed by an equals sign and one or more elements enclosed in square brackets.
- If a variable is specified on the FACTOR keyword and is also specified on the CONTROL keyword, then the CONTROL specification for that variable is ignored when the variable's odds ratios are computed. Thus, `FACTOR = [A B] CONTROL = [A(1) B(2)]` estimates odds ratios for factor A holding factor B at level 2, and for factor B holding factor A at level 1.

FACTOR = [option]. Valid options are one or more factors appearing on the factor list. Optionally, each factor may be followed by parentheses containing the level to use as the reference category when computing cumulative odds ratios. The keyword LOW or HIGH, or a value, may be specified. Put the value inside a pair of quotes if the value is formatted (such as date or currency) or if the factor is of string type. By default, the highest category is used as the reference category.

- If a value is specified but the value does not exist in the data, then a warning is issued and the default HIGH is used.
- Any factor may occur only once on the FACTOR keyword.

COVARIATE = [option]. Valid options are one or more covariates appearing on the covariate list. Optionally, each covariate may be followed by parentheses containing one or more nonzero numbers giving unit(s) of change to use for covariates when computing cumulative odds ratios. Cumulative odds ratios are estimated for each distinct value. The default value is 1. Any covariate may occur only once on the COVARIATE keyword.

CONTROL = [option]. Specifies the factor and/or covariate values to use when computing cumulative odds ratios. Factors must appear on the factor list, and covariates on the covariate list, of the CSORDINAL command.

- Factors must be followed by the keyword LOW or HIGH, or a value, in parentheses. Put the value inside a pair of quotes if the value is formatted (such as date or currency) or if the factor is of string type. If keyword LOW or HIGH is used, then each cumulative odds ratio is computed by holding the factor at its lowest or highest level, respectively. If a value is used, then each cumulative odds ratio is computed by holding the specified factor at the supplied value. If a factor is not specified on the CONTROL option, then its highest category is used in cumulative odds ratio calculations.
- If a factor value is specified but the value does not exist in the data, then a warning is issued and the default HIGH is used.
- Covariates must be followed by keyword MEAN or a number in parentheses. If keyword MEAN is used, then each cumulative odds ratio is computed by holding the covariate at its overall mean. If a number is used, then each cumulative odds ratio is computed by holding the specified covariate at the supplied value. If a covariate is not specified on the CONTROL option, then its overall mean is used in cumulative odds ratio calculations.
- Any factor or covariate may occur only once on the CONTROL keyword.

Example

Suppose that dependent variable Y has three levels; factor A has two levels; and factor B has three levels coded 1, 2, and 3.

```
CSORDINAL y BY a b WITH x
  /PLAN FILE='/survey/myfile.csplan'
  /MODEL a b a*b x
  /ODDSRATIOS FACTOR=[a] CONTROL=[b(1)]
  /ODDSRATIOS FACTOR=[a] CONTROL=[b(2)]
  /ODDSRATIOS FACTOR=[a] CONTROL=[b(3)].
```

- The default LOGIT cumulative link function is used and the cumulative odds ratios are computed. They are equal across all response levels by the model definition.
- The model includes two thresholds, main effects for factors A and B, the A*B interaction effect, and the covariate X.
- Cumulative odds ratios are requested for factor A. Assuming the A*B interaction effect is significant, the cumulative odds ratio for factor A will differ across levels of factor B. The specified syntax requests three cumulative odds ratios for factor A; each is computed at a different level of factor B.

Example

```
CSORDINAL z BY a b c WITH x y
  /PLAN FILE='/survey/myfile.csplan'
  /MODEL a b c x*y
  /ODDSRATIOS COVARIATE=[x(1 3 5)] CONTROL=[y(1)].
```

- The preceding syntax will compute three cumulative odds ratios for covariate X.
- The parenthesized list following variable X provides the unit of change values to use when computing cumulative odds ratios. Cumulative odds ratios will be computed for X increasing by 1, 3, and 5 units and holding covariate Y equal to 1.

CRITERIA Subcommand

The CRITERIA subcommand offers controls on the iterative algorithm used for estimation, and specifies numerical tolerance for checking singularity.

CHKSEP = integer. *Starting iteration for checking complete and quasi-complete separation.* Specify a non-negative integer. This criterion is not used if the value is 0. The default value is 20.

CILEVEL = value. *Confidence interval level for coefficient estimates, exponentiated coefficient estimates, and cumulative odds ratio estimates.* Specify a value greater than or equal to 0, and less than 100. The default value is 95.

DF = value. *Sampling design degrees of freedom to use in computing p values for all test statistics.* Specify a positive number. The default value is the difference between the number of primary sampling units and the number of strata in the first stage of sampling.

LCONVERGE = [number (RELATIVE | ABSOLUTE)]. *Log-likelihood function convergence criterion.* Convergence is assumed if the relative or absolute change in the log-likelihood function is less than the given value. This criterion is not used if the value is 0. Specify square brackets containing a non-negative number followed optionally by the keyword RELATIVE or ABSOLUTE, which indicates the type of change. The default value is 0; the default type is RELATIVE.

METHOD = FISHER(number) | NEWTON. *Model parameters estimation method.* The Fisher scoring method is specified by the keyword FISHER, the Newton-Raphson method, by the keyword NEWTON, and a hybrid method is available by specifying FISHER(n). In the hybrid method, *n* is the maximal number of Fisher scoring iterations before switching to the Newton-Raphson method. If convergence is achieved during the Fisher scoring phase of the hybrid method, iterations continue with the Newton-Raphson method.

MXITER = integer. *Maximum number of iterations.* Specify a non-negative integer. The default value is 100.

MXSTEP = integer. *Maximum step-halving allowed.* Specify a positive integer. The default value is 5.

PCONVERGE = [number (RELATIVE | ABSOLUTE)]. *Parameter estimates convergence criterion.* Convergence is assumed if the relative or absolute change in the parameter estimates is less than the given value. This criterion is not used if the value is 0. Specify square brackets containing a non-negative number followed optionally by the keyword RELATIVE or ABSOLUTE, which indicates the type of change. The default value is 10^{-6}; the default type is RELATIVE.

SINGULAR = value. *Tolerance value used to test for singularity.* Specify a positive value. The default value is 10^{-12}.

STATISTICS Subcommand

The STATISTICS subcommand requests various statistics associated with the parameter estimates.

- There are no default keywords on the STATISTICS subcommand. If this subcommand is not specified, then none of the statistics listed below are displayed

PARAMETER. *Parameter estimates.*

EXP. *The exponentiated parameter estimates.* It is available only for the LOGIT link.

SE. *Standard error for each parameter estimate.*

TTEST. *t test for each parameter estimate.*

CINTERVAL. *Confidence interval for each parameter estimate and/or exponentiated parameter estimate.*

DEFF. *Design effect for each parameter estimate.*

DEFFSQRT. *Square root of design effect for each parameter estimate.*

NONPARALLEL Subcommand

The NONPARALLEL subcommand requests various statistics associated with a general cumulative link model with non-parallel lines where a separate regression line is fitted for each response category except for the last.

TEST. *Test of parallel lines assumption.* Test whether regression parameters are equal for all cumulative responses. The general model with non-parallel lines is estimated and the Wald test of equal parameters is applied.

PARAMETER. *Parameters of the general model with non-parallel lines.* The general model is estimated using the same convergence criteria as for the original model. Both parameters and their standard errors are estimated.

COVB. *Covariance matrix for the general model parameters.*

TEST Subcommand

The TEST subcommand specifies the type of test statistic and the method of adjusting the significance level to be used for hypothesis tests requested on the MODEL, CUSTOM, and PRINT subcommands.

TYPE Keyword

The TYPE keyword indicates the type of test statistic.

F. *Wald F test.* This is the default test statistic if the TYPE keyword is not specified.

ADJF. *Adjusted Wald F test.*

CHISQUARE. *Wald chi-square test.*

ADJCHISQUARE. *Adjusted Wald chi-square test.*

PADJUST Keyword

The PADJUST keyword indicates the method of adjusting the significance level.

LSD. *Least significant difference.* This method does not control the overall probability of rejecting the hypotheses that some linear contrasts are different from the null hypothesis value(s). This is the default.

BONFERRONI. *Bonferroni.* This method adjusts the observed significance level for the fact that multiple contrasts are being tested.

SEQBONFERRONI. *Sequential Bonferroni.* This is a sequentially step-down rejective Bonferroni procedure that is much less conservative in terms of rejecting individual hypotheses but maintains the same overall significance level.

SIDAK. *Sidak.* This method provides tighter bounds than the Bonferroni approach.

SEQSIDAK. *Sequential Sidak.* This is a sequentially step-down rejective Sidak procedure that is much less conservative in terms of rejecting individual hypotheses but maintains the same overall significance level.

DOMAIN Subcommand

The DOMAIN subcommand specifies the subpopulation for which the analysis is to be performed.

- The keyword VARIABLE, followed by an equals sign, a variable, and a value in parentheses, are required. Put the value inside a pair of quotes if the value is formatted (such as date or currency) or if the factor is of string type.
- The subpopulation is defined by all cases having the given value on the specified variable.
- Analyses are performed only for the specified subpopulation.
- For example, `DOMAIN VARIABLE = myvar (1)` defines the subpopulation by all cases for which variable *MYVAR* has value 1.
- The specified variable may be numeric or string and must exist at the time the CSORDINAL procedure is invoked.
- Stratification or cluster variables may be specified, but no other plan file variables are allowed on the DOMAIN subcommand.
- Analysis variables may not be specified on the DOMAIN subcommand.

MISSING Subcommand

The MISSING subcommand specifies how missing values are handled.

- In general, cases must have valid data for all design variables as well as for the dependent variable and any covariates. Cases with invalid data for any of these variables are excluded from the analysis.
- There is one important exception to the preceding rule. This exception applies when an inclusion probability or population size variable is defined in an analysis plan file. Within a stratum at a given stage, if the inclusion probability or population size values are unequal across cases or missing for a case, then the first valid value found within that stratum is used as the value for the stratum. If strata are not defined, then the first valid value found in the sample is used. If the inclusion probability or population size values are missing for all cases within a stratum (or within the sample if strata are not defined) at a given stage, then an error message is issued.
- The CLASSMISSING keyword specifies whether user-missing values are treated as valid. This specification is applied to categorical design variables that is, strata, cluster, and subpopulation variables), the dependent variable, and any factors.

EXCLUDE. *Exclude user-missing values among the strata, cluster, subpopulation, the dependent variable, and factor variables.* This is the default.

INCLUDE . *Include user-missing values among the strata, cluster, subpopulation, the dependent variable, and factor variables.* Treat user-missing values for these variables as valid data.

PRINT Subcommand

The PRINT subcommand is used to display optional output.

- If the PRINT subcommand is not specified, then the default output includes sample information, variable and factor information, and model summary statistics.
- If the PRINT subcommand is specified, then CSORDINAL displays output only for those keywords that are specified.

SAMPLEINFO. *Sample information table.* Displays summary information about the sample, including the unweighted count and the population size. This is default output if the PRINT subcommand is not specified.

VARIABLEINFO. *Variable information.* Displays summary information about the dependent variable, covariates, and factors. This is default output if the PRINT subcommand is not specified.

SUMMARY. *Model summary statistics.* Displays pseudo-R^2 statistics. This is default output if the PRINT subcommand is not specified.

HISTORY(n). *Iteration history.* Displays coefficient estimates and statistics at every n^{th} iteration beginning with the 0th iteration (the initial estimates). The default is to print every iteration ($n = 1$). The last iteration is always printed if HISTORY is specified, regardless of the value of *n*.

GEF. *General estimable function table.*

LMATRIX. *Set of contrast coefficients (L) matrices.* These are the Type III contrast matrices used in testing model effects.

COVB. *Covariance matrix for model parameters.*

CORB. *Correlation matrix for model parameters.*

CLASSTABLE. *Classification table.* Displays frequencies of observed versus predicted response categories.

NONE. *No PRINT subcommand output.* None of the PRINT subcommand output is displayed. However, if NONE is specified with one or more other keywords, then the other keywords override NONE.

SAVE Subcommand

The SAVE subcommand writes optional model variables to the active dataset.

- Specify one or more temporary variables, each followed by an optional new name in parentheses.
- The optional names must be valid variable names.
- If new names are not specified, CSORDINAL uses the default names.
- If a subpopulation is defined on the DOMAIN subcommand, then SAVE applies only to cases within the subpopulation.

The following rules describe the functionality of the SAVE subcommand in relation to the predictor values for each case.

- If all factors and covariates in the model have valid values for the case, then the procedure computes the predicted values. (The MISSING subcommand setting is taken into account when defining valid/invalid values for a factor.)
- An additional restriction for factors is that only those values of the factor actually used in building the model are considered valid. For example, suppose factor A takes values 1, 2, and 3 when the procedure builds the model. Also suppose there is a case with a value of 4 on factor A, and valid values on all other factors and covariates. For this case, no predicted values are saved because there is no model coefficient corresponding to factor A = 4.

Computation of predicted values for a given case does not depend on the value of the dependent variable; it could be missing.

CUMPROB (rootname:n). *Cumulative probability.* The user-specified or default name is treated as the root name, and a suffix is added to get new unique variable names. The root name can be followed by a colon and a positive integer giving the number of predicted cumulative probabilities to save. The predicted cumulative probabilities of the first *n* response categories are saved. One cumulative predicted probability variable can be saved for each category of the dependent variable. The default root name is *CumulativeProbability*. The default *n* is 25. To specify *n* without a root name, enter a colon before the number.

PREDPROB (rootname:n). *Predicted probability.* The user-specified or default name is treated as the root name, and a suffix is added to get new unique variable names. The root name can be followed by a colon and a positive integer giving the number of predicted probabilities to save. The predicted probabilities of the first *n* response categories are saved. One predicted probability variable can be saved for each category of the dependent variable. The default root name is *PredictedProbability*. The default *n* is 25. To specify *n* without a root name, enter a colon before the number.

PREDVAL (varname). *Predicted value.* The class or value predicted by the model. The optional variable name must be unique. If the default name is used and it conflicts with existing variable names, then a suffix is added to the default name to make it unique. The default variable name is *PredictedValue*.

PREDVALPROB (varname). *Predicted value probability.* The probability of value predicted by the model. This probability is the maximum probability predicted by the model for a given case. The optional variable name must be unique. If the default name is used and it conflicts with existing variable names, then a suffix is added to the default name to make it unique. The default variable name is *PredictedValueProbability*.

OBSVALPROB (varname). *Observed value probability.* The probability predicted for the observed response value. The optional variable name must be unique. If the default name is used and it conflicts with existing variable names, then a suffix is added to the default name to make it unique. The default variable name is *ObservedValueProbability*.

OUTFILE Subcommand

The OUTFILE subcommand saves an external IBM SPSS Statistics data file or previously declared dataset containing the parameter covariance or correlation matrix with parameter estimates, standard errors, significance values, and sampling design degrees of freedom. It also saves the parameter estimates and the parameter covariance matrix in XML format.

- At least one keyword and a filename are required.
- The COVB and CORB keywords are mutually exclusive, as are the MODEL and PARAMETER keywords.
- The filename must be specified in full. CSORDINAL does not supply an extension.

COVB = 'savfile' | 'dataset'. *Writes the parameter covariance matrix and other statistics to an external IBM SPSS Statistics data file or previously declared dataset.*

CORB = 'savfile' | 'dataset'. *Writes the parameter correlation matrix and other statistics to an external IBM SPSS Statistics data file or previously declared dataset.*

MODEL = 'file'. *Writes the parameter estimates and the parameter covariance matrix to an XML file.*

PARAMETER = 'file'. *Writes the parameter estimates to an XML file.*

CSPLAN

CSPLAN is available in SPSS Statistics Premium Edition or the Complex Samples option.

```
CSPLAN SAMPLE

/PLAN FILE=file

[/PLANVARS  [SAMPLEWEIGHT=varname]]
           [PREVIOUSWEIGHT=varname]

[/PRINT [PLAN**] [MATRIX]]
```

Design Block: Stage 1

```
/DESIGN [STAGELABEL='label']
        [STRATA=varname [varname [...] ] ]
        [CLUSTER=varname [varname [...] ] ]

/METHOD TYPE={SIMPLE_WOR       }    [ESTIMATION={DEFAULT**}]
             {SIMPLE_WR        }                {WR       }
             {SIMPLE_SYSTEMATIC}
             {SIMPLE_CHROMY    }
             {PPS_WOR          }
             {PPS_WR           }
             {PPS_SYSTEMATIC   }
             {PPS_BREWER       }
             {PPS_MURTHY       }
             {PPS_SAMPFORD     }
             {PPS_CHROMY       }

[/MOS     {VARIABLE=varname} [MIN=value]  [MAX=value] ]
          {SOURCE=FROMDATA }

[/SIZE {VALUE=sizevalue                                                    }]
       {VARIABLE=varname                                                   }
       {MATRIX=varname [varname [...] ]; catlist value [;catlist value [;...]]}

[/RATE {VALUE=ratevalue                                                    }]
       {VARIABLE=varname                                                   }
       {MATRIX=varname [varname [...] ]; catlist value [;catlist value [;...]]}
       [MINSIZE=value]
       [MAXSIZE=value]

[/STAGEVARS  [INCLPROB[(varname)]]]
             [CUMWEIGHT[varname)]]
             [INDEX[(varname)]]
             [POPSIZE[(varname)]]
             [SAMPSIZE[(varname)]]
             [RATE[(varname)]]
             [WEIGHT[(varname)]]
```

Design Block: Stages 2 and 3

```
/DESIGN [STAGELABEL='label']
        [STRATA=varname [varname [...] ] ]
        [CLUSTER=varname [varname [...] ] ]

/METHOD TYPE={SIMPLE_WOR       }
             {SIMPLE_WR        }
             {SIMPLE_SYSTEMATIC}
             {SIMPLE_CHROMY    }

[/SIZE {VALUE=sizevalue                                                    }]
       {VARIABLE=varname                                                   }
       {MATRIX=varname [varname [...] ]; catlist value [;catlist value [;...]]}

[/RATE {VALUE=ratevalue                                                    }]
       {VARIABLE=varname                                                   }
       {MATRIX=varname [varname [...] ]; catlist value [;catlist value [;...]]}
       [MINSIZE=value]
       [MAXSIZE=value]

[/STAGEVARS  [INCLPROB[(varname)]]]
             [CUMWEIGHT[varname)]]
             [INDEX[(varname)]]
             [POPSIZE[(varname)]]
             [SAMPSIZE[(varname)]]
             [RATE[(varname)]]
             [WEIGHT[(varname)]]
```

Create an Analysis Design

```
CSPLAN ANALYSIS

/PLAN FILE=file

/PLANVARS ANALYSISWEIGHT=varname

[/SRSESTIMATOR TYPE={WOR**}]
```

```
                  {WR   }

[/PRINT [PLAN**] [MATRIX]]
```

Design Block: Stage 1

```
/DESIGN [STAGELABEL='label']
        [STRATA=varname [varname [...] ] ]
        [CLUSTER=varname [varname [...] ] ]

/ESTIMATOR TYPE= {EQUAL_WOR  }
                 {UNEQUAL_WOR}
                 {WR         }

[/POPSIZE {VALUE=sizevalue                                                  }]
          {VARIABLE=varname                                                 }
          {MATRIX=varname [varname [...] ]; catlist value [;catlist value [;...]]}

[/INCLPROB {VALUE=probvalue                                                 }]
           {VARIABLE=varname                                                }
           {MATRIX=varname [varname [...]]; catlist value [;catlist value[;...]]}
```

Design Block: Stages 2 and 3

```
/DESIGN [STAGELABEL='label']
        [STRATA=varname [varname [...] ] ]
        [CLUSTER=varname [varname [...] ] ]

/ESTIMATOR TYPE= {EQUAL_WOR}
                 {WR       }

[/POPSIZE {VALUE=sizevalue                                                  }]
          {VARIABLE=varname                                                 }
          {MATRIX=varname [varname [...]]; catlist value [;catlist value [;...]]}

[/INCLPROB {VALUE=probvalue                                                 }]
           {VARIABLE=varname                                                }
           {MATRIX=varname [varname [...]]; catlist value [;catlist value[;...]]}
```

Display an Existing Plan

```
CSPLAN VIEW

/PLAN FILE=file

[/PRINT [PLAN**] [MATRIX]]
```

** Default if the subcommand is omitted.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
CSPLAN  SAMPLE
  /PLAN FILE= '/survey/myfile.csplan'
  /DESIGN STRATA=region CLUSTER=school
  /METHOD TYPE=PPS_WOR
  /MOS VARIABLE=mysizevar
  /SIZE VALUE=100.

CSPLAN  ANALYSIS
  /PLAN FILE= '/survey/myfile.csaplan'
  /PLANVARS ANALYSISWEIGHT=sampleweight
  /DESIGN CLUSTER=district
  /ESTIMATOR TYPE=UNEQUAL_WOR
  /DESIGN CLUSTER=school
  /ESTIMATOR TYPE=EQUAL_WOR
  /INCLPROB VARIABLE=sprob.

CSPLAN  VIEW
  /PLAN FILE= '/survey/myfile.csplan'.
```

Overview

CSPLAN creates a complex sample design or analysis specification that is used by companion procedures in SPSS Statistics Premium Edition or the Complex Samples option. CSSELECT uses specifications from a plan file when selecting cases from the active file. Analysis procedures in SPSS Statistics Premium Edition or the Complex Samples option, such as CSDESCRIPTIVES, require a plan file in order to produce summary statistics for a complex sample. You can also use CSPLAN to view sample or analysis specifications within an existing plan file.

The CSPLAN design specification is used only by procedures in Sampling and Testing.

Options

Design Specification. CSPLAN writes a sample or analysis design to a file. A sample design can be used to extract sampling units from the active file. An analysis design is used to analyze a complex sample. When a sample design is created, the procedure automatically saves an appropriate analysis design to the plan file. Thus, a plan file created for designing a sample can be used for both sample selection and analysis.

Both sample and analysis designs can specify **stratification**, or independent sampling within nonoverlapping groups, as well as **cluster sampling**, in which groups of sampling units are selected. A single or multistage design can be specified with a maximum of three stages.

CSPLAN does not actually execute the plan (that is, it does not extract the sample or analyze data). To sample cases, use a sample design created by CSPLAN as input to CSSELECT. To analyze sample data, use an analysis design created by CSPLAN as input to Complex Samples procedures, such as CSDESCRIPTIVES.

Sample Design. A variety of equal- and unequal-probability methods are available for sample selection, including simple and systematic random sampling. CSPLAN offers several methods for sampling with probability proportionate to size (**PPS**), including Brewer's method, Murthy's method, and Sampford's method. Units can be drawn with replacement (**WR**) or without replacement (**WOR**) from the population. At each stage of the design, you can control the number or percentage of units to be drawn. You can also choose output variables, such as stagewise sampling weights, that are created when the sample design is executed.

Analysis Design. The following estimation methods are available: with replacement, equal probability without replacement, and unequal probability without replacement. Unequal probability estimation without replacement can be requested in the first stage only. You can specify variables to be used as input to the estimation process, such as overall sample weights and inclusion probabilities.

Operations

- If a sample design is created, the procedure automatically writes a suitable analysis design to the plan file. The default analysis design specifies stratification variables and cluster variables for each stage, as well as an estimation method appropriate for the chosen extraction method.
- CSPLAN writes design specifications in XML format.
- By default, CSPLAN displays output that summarizes the sample or analysis design.

Subcommand Order

- The first DESIGN subcommand must precede all other subcommands except PLAN, PLANVARS, and PRINT.
- PLAN, PLANVARS, and PRINT subcommands can be used in any order.

Limitations

- A maximum of three design blocks can be specified.
- CSPLAN ignores SPLIT FILE and WEIGHT commands with a warning.

Basic Specification

You can specify a sample or analysis design to be created or a plan file to be displayed.

Creating a Sample Plan

- The SAMPLE keyword must be specified on the CSPLAN command.
- A PLAN subcommand is required that specifies a file that will contain the design specification.
- A DESIGN subcommand is required.
- A METHOD subcommand must specify an extraction method.

- Sample size or rate must be specified unless the PPS_MURTHY or PPS_BREWER extraction method is chosen.

Creating an Analysis Plan

- The ANALYSIS keyword must be specified on the CSPLAN command.
- A PLAN subcommand is required that specifies a file that will contain the analysis specification.
- A PLANVARS subcommand is required that specifies a sample weight variable.
- A DESIGN subcommand is required.
- An ESTIMATOR subcommand must specify an estimator.
- The POPSIZE or INCLPROB subcommand must be specified if the EQUAL_WOR estimator is selected.

Displaying an Existing Plan

- The VIEW keyword must be specified on the CSPLAN command.
- A PLAN subcommand is required that specifies a file whose specifications are to be displayed.

Syntax Rules

General

- PLAN, PLANVARS, and PRINT are global. Only a single instance of each global subcommand is allowed.
- Within a subcommand, an error occurs if a keyword or attribute is specified more than once.
- Equals signs shown in the syntax chart are required.
- Subcommand names and keywords (for example, PPS_WR) must be spelled in full.
- In general, empty subcommands (that is, those that have no specifications) generate an error. DESIGN is the only subcommand that can be empty.
- Any variable names that are specified must be valid IBM SPSS Statistics variable names.

Creating a Plan

- Stages are specified in design blocks. The DESIGN subcommand signals the start of a block. The first block corresponds to stage 1, the second to stage 2, and the third to stage 3. One DESIGN subcommand must be specified per stage.
- The following subcommands are local and apply to the immediately preceding DESIGN subcommand: METHOD, MOS, SIZE, RATE, STAGEVARS, ESTIMATOR, POPSIZE, and INCLPROB. An error occurs if any of these subcommands appears more than once within a block.
- Available METHOD and ESTIMATOR options depend on the stage.
- The following subcommands are honored only if a sample design is requested: METHOD, MOS, SIZE, RATE, and STAGEVARS. An error occurs if any of these subcommands is specified for an analysis design.
- MOS can be specified in stage 1 only.
- The following subcommands can be used only if an analysis design is requested: ESTIMATOR, POPSIZE, and INCLPROB. An error occurs if any of these subcommands is specified for a sample design.
- In general, each variable specified in the design can assume only one role. For example, a weight variable cannot be used as a stratification or cluster variable.

Displaying a Plan

- If CSPLAN VIEW is used, only the PLAN and PRINT subcommands can be specified.

Examples

Simple Sample Design

```
CSPLAN SAMPLE
  /PLAN FILE='/survey/myfile.csplan'
  /DESIGN
```

```
  /METHOD TYPE=SIMPLE_WOR
  /SIZE VALUE=100.
```

- A single-stage sample design is created that is saved in *myfile.csplan*.
- One hundred cases will be selected from the active file when the sample design is executed by the CSSELECT procedure.
- The extraction method is simple random sampling without replacement.
- The plan file also includes a default analysis design that uses the EQUAL_WOR estimator (the default when units are extracted using the SIMPLE_WOR method).

Stratified Sample Design

```
CSPLAN SAMPLE
  /PLAN FILE='/survey/myfile.csplan'
  /DESIGN STRATA=region
  /METHOD TYPE=SIMPLE_WOR
  /RATE MATRIX=REGION; 'East' 0.1 ; 'West' 0.2;
                       'North' 0.1; 'South' 0.3.
```

- A stratified sample design is specified with disproportionate sampling rates for the strata. Sample elements will be drawn independently within each region.
- The extraction method is simple random sampling without replacement.
- CSPLAN generates a default analysis design using *region* as a stratification variable and the EQUAL_WOR estimator.

Stratified Cluster Sample Design

```
CSPLAN SAMPLE
  /PLAN FILE='/survey/myfile.csplan'
  /DESIGN STRATA=region CLUSTER=school
  /METHOD TYPE=PPS_WOR
  /SIZE VALUE=10
  /MOS VARIABLE=mysizevar.
```

- A stratified cluster sample design is specified.
- Ten schools will be selected within each region with probability proportionate to size.
- Size values for the strata are read from *mysizevar*.
- CSPLAN generates a default analysis design using *region* as a stratification variable and *school* as a cluster variable.
- The UNEQUAL_WOR estimator will be used for analysis. UNEQUAL_WOR is the default when units are sampled with probability proportionate to size.

Multistage Cluster Sample Design

```
CSPLAN SAMPLE
  /PLAN FILE='/survey/myfile.csplan'
  /DESIGN STAGELABEL='school districts' CLUSTER=district
  /METHOD TYPE=PPS_WOR
  /RATE VALUE=.2
  /MOS VARIABLE=districtsize
  /DESIGN STAGELABEL='schools' CLUSTER=school
  /METHOD TYPE=SIMPLE_WOR
  /RATE VALUE=0.3.
```

- A multistage cluster sample design is specified.
- Twenty percent of school districts will be drawn with probability proportionate to size.
- Within each selected school district, 30% of schools will be drawn without replacement.
- CSPLAN generates a default analysis design. Since the PPS_WOR sampling method is specified in stage 1, the UNEQUAL_WOR estimator will be used for analysis for that stage. The EQUAL_WOR method will be used to analyze stage 2.

Simple Analysis Design

```
CSPLAN ANALYSIS
  /PLAN FILE='/survey/myfile.csaplan'
  /PLANVARS ANALYSISWEIGHT=sampleweight
  /DESIGN
  /ESTIMATOR TYPE=EQUAL_WOR
  /POPSIZE VALUE=5000.
```

- An analysis design is specified.
- The variable *sampleweight* is specified as the variable containing sample weights for analysis.
- The EQUAL_WOR estimator will be used for analysis.
- POPSIZE specifies that the sample was drawn from a population of 5,000.

Simple Analysis Design

```
CSPLAN ANALYSIS
  /PLAN FILE='/survey/myfile.csaplan'
  /PLANVARS ANALYSISWEIGHT=sampleweight
  /DESIGN
  /ESTIMATOR TYPE=EQUAL_WOR
  /INCLPROB VALUE=0.10.
```

- An analysis design is specified.
- The variable *sampleweight* is specified as the variable containing sample weights for analysis.
- The EQUAL_WOR estimator will be used for analysis.
- INCLPROB specifies that 10% of population units were selected for inclusion in the sample.

Stratified Analysis Design

```
CSPLAN ANALYSIS
  /PLAN FILE='/survey/myfile.csaplan'
  /PLANVARS ANALYSISWEIGHT=sampleweight
  /DESIGN STRATA=region
  /ESTIMATOR TYPE=EQUAL_WOR
  /INCLPROB MATRIX=REGION; 'East' 0.1; 'West' 0.2;
                           'North' 0.1; 'South' 0.3.
```

- The analysis design specifies that the sample is stratified by *region*.
- Inclusion probabilities are specified for each stratum.
- The variable *sampleweight* is specified as the variable containing sample weights for analysis.

Stratified Clustering Analysis Design

```
CSPLAN ANALYSIS
  /PLAN FILE='/survey/myfile.csaplan'
  /PLANVARS ANALYSISWEIGHT=sampleweight
  /DESIGN STRATA=district CLUSTER=school
  /ESTIMATOR TYPE=UNEQUAL_WOR.
```

- The analysis design specifies that units were sampled using stratified clustering.
- The variable *sampleweight* is specified as the variable containing sample weights for analysis.
- *District* is defined as a stratification variable and *school* is defined as a cluster variable.
- The UNEQUAL_WOR estimator will be used for analysis.

Multistage Analysis Design

```
CSPLAN ANALYSIS
  /PLAN FILE='/survey/myfile.csaplan'
  /PLANVARS ANALYSISWEIGHT=sampleweight
  /DESIGN CLUSTER=district
  /ESTIMATOR TYPE=UNEQUAL_WOR
  /DESIGN CLUSTER=school
  /ESTIMATOR TYPE=EQUAL_WOR
  /INCLPROB VARIABLE=sprob.
```

- The analysis design specifies that cases were sampled using multistage clustering. Schools were sampled within districts.
- The UNEQUAL_WOR estimator will be used in stage 1.
- The EQUAL_WOR estimator will be used in stage 2.
- The variable *sprob* contains inclusion probabilities, which are required for analysis of the second stage.
- The variable *sampleweight* is specified as the variable containing sample weights for analysis.

Display Plan

```
CSPLAN  VIEW
  /PLAN FILE='/survey/myfile.csplan'.
```

- The syntax displays the specifications in the plan file *myfile.csplan*.

CSPLAN Command

CSPLAN creates a complex sample design or analysis specification.

SAMPLE. *Creates a sample design.*

ANALYSIS. *Creates an analysis design.*

VIEW. *Displays a sample or analysis design.*

PLAN Subcommand

The PLAN subcommand specifies the name of a design file to be written or displayed by CSPLAN. The file contains sample and/or analysis design specifications.

FILE. *Sampling design file.* Specify the filename in full. If you are creating a plan and the file already exists, it is overwritten without warning.

PLANVARS Subcommand

PLANVARS is used to name planwise variables to be created when a sample is extracted or used as input to the selection or estimation process.

ANALYSISWEIGHT. *Final sample weights for each unit to be used by Complex Samples analysis procedures in the estimation process.* ANALYSISWEIGHT is required if an analysis design is specified. It is ignored with a warning if a sample design is specified.

SAMPLEWEIGHT. *Overall sample weights that will be generated when the sample design is executed using CSSELECT.* A final sampling weight is created automatically when the sample plan is executed.

- SAMPLEWEIGHT is honored only if a sampling design is specified. It is ignored with a warning if an analysis design is specified.
- Sample weights are positive for selected units. They take into account all stages of the design as well as previous sampling weights if specified.
- If SAMPLEWEIGHT is not specified, a default name (*SampleWeight_Final_*) is used for the sample weight variable.

PREVIOUSWEIGHT. *Weights to be used in computing final sampling weights in a multistage design.*

PREVIOUSWEIGHT is honored only if a sampling design is specified. It is ignored with a warning if an analysis design is specified.

Typically, the previous weight variable is produced in an earlier stage of a stage-by-stage sample selection process. CSSELECT multiplies previous weights with those for the current stage to obtain final sampling weights.

For example, suppose that you want to sample individuals within cities but only city data are available at the outset of the study. For the first stage of extraction, a design plan is created that specifies that 10 cities are to be sampled from the active file. The PLANVARS subcommand specifies that sampling weights are to be saved under the name *CityWeights*.

```
CSPLAN SAMPLE
  /PLAN FILE='/survey/city.csplan'
  /PLANVARS SAMPLEWEIGHT=CityWeights
  /DESIGN CLUSTER=city
  /METHOD TYPE=PPS_WOR
  /MOS VARIABLE=SizeVar
  /SIZE VALUE=10.
```

This plan would be executed using CSSELECT on an active file in which each case is a city.

For the next stage of extraction, a design plan is created that specifies that 50 individuals are to be sampled within cities. The design uses the PREVIOUSWEIGHT keyword to specify that sample weights generated in the first stage are to be used when computing final sampling weights for selected individuals. Final weights are saved to the variable *FinalWeights*.

```
CSPLAN SAMPLE
  /PLAN FILE='/survey/individuals.csplan'
  /PLANVARS PREVIOUSWEIGHT=CityWeights
            SAMPLEWEIGHT=FinalWeights
  /DESIGN STRATA=city
  /METHOD TYPE=SIMPLE_WOR
  /SIZE VALUE=50.
```

The plan for stage 2 would be executed using CSSELECT on an active file in which cases represent individuals and both *city* and *CityWeights* are recorded for each individual. Note that *city* is identified as a stratification variable in this stage, so individuals are sampled within cities.

SRSESTIMATOR Subcommand

The SRSESTIMATOR subcommand specifies the variance estimator used under the simple random sampling assumption. This estimate is needed, for example, in computation of design effects in Complex Samples analysis procedures.

WOR. *SRS variance estimator includes the finite population correction.* This estimator is the default.

WR. *SRS variance estimator does not include the finite population correction.* This estimator is recommended when the analysis weights have been scaled so that they do not add up to the population size.

PRINT Subcommand

PLAN. *Displays a summary of plan specifications.* The output reflects your specifications at each stage of the design. The plan is shown by default. The PRINT subcommand is used to control output from CSPLAN.

MATRIX. *Displays a table of MATRIX specifications.* MATRIX is ignored if you do not use the MATRIX form of the SIZE, RATE, POPSIZE, or INCLPROB subcommand. By default, the table is not shown.

DESIGN Subcommand

The DESIGN subcommand signals a stage of the design. It also can be used to define stratification variables, cluster variables, or a descriptive label for a particular stage.

STAGELABEL Keyword

STAGELABEL allows a descriptive label to be entered for the stage that appears in Complex Samples procedure output.

'Label'. *Descriptive stage label.* The label must be specified within quotes. If a label is not provided, a default label is generated that indicates the stage number.

STRATA Keyword

STRATA is used to identify stratification variables whose values represent nonoverlapping subgroups. Stratification is typically done to decrease sampling variation and/or to ensure adequate representation of small groups in a sample.

If STRATA is used, CSSELECT draws samples independently within each stratum. For example, if *region* is a stratification variable, separate samples are drawn for each region (for example, East, West, North, and South). If multiple STRATA variables are specified, sampling is performed within each combination of strata.

varlist. *Stratification variables.*

CLUSTER Keyword

CLUSTER is used to sample groups of sampling units, such as states, counties, or school districts. Cluster sampling is often performed to reduce travel and/or interview costs in social surveys. For example, if census tracts are sampled within a particular city and each interviewer works within a particular tract, he or she would be able to conduct interviews within a small area, thus minimizing time and travel expenses.

- If CLUSTER is used, CSSELECT samples from values of the cluster variable as opposed to sampling elements (cases).
- If two or more cluster variables are specified, samples are drawn from among all combinations of values of the variables.
- CLUSTER is required for nonfinal stages of a sample or analysis plan.
- CLUSTER is required if any of the following sampling methods is specified: PPS_WOR, PPS_BREWER, PPS_MURTHY, or PPS_SAMPFORD.
- CLUSTER is required if the UNEQUAL_WOR estimator is specified.

varlist. *Cluster variables.*

METHOD Subcommand

The METHOD subcommand specifies the sample extraction method. A variety of equal- and unequal-probability methods are available.

The following table lists extraction methods and their availability at each stage of the design. For details on each method, see the CSSELECT algorithms document.

- PPS methods are available only in stage 1. WR methods are available only in the final stage. Other methods are available in any stage.
- If a PPS method is chosen, a measure of size (MOS) must be specified.
- If the PPS_WOR, PPS_BREWER, PPS_SAMPFORD, or PPS_MURTHY method is selected, first-stage joint inclusion probabilities are written to an external file when the sample plan is executed. Joint probabilities are needed for UNEQUAL_WOR estimation by Complex Samples analysis procedures.
- By default, CSPLAN chooses an appropriate estimation method for the selected sampling method. If ESTIMATION=WR, Complex Samples analysis procedures use the WR (with replacement) estimator regardless of the sampling method.

Type	Description	Default estimator
SIMPLE_WOR	Selects units with equal probability. Units are extracted without replacement.	EQUAL_WOR
SIMPLE_WR	Selects units with equal probability. Units are extracted with replacement.	WR
SIMPLE_SYSTEMATIC	Selects units at a fixed interval throughout the sampling frame or stratum. A random starting point is chosen within the first interval.	WR
SIMPLE_CHROMY	Selects units sequentially with equal probability. Units are extracted without replacement.	WR
PPS_WOR	Selects units with probability proportional to size. Units are extracted without replacement.	UNEQUAL_WOR

Type	Description	Default estimator
PPS_WR	Selects units with probability proportional to size. Units are extracted with replacement.	WR
PPS_SYSTEMATIC	Selects units by systematic random sampling with probability proportional to size. Units are extracted without replacement.	WR
PPS_CHROMY	Selects units sequentially with probability proportional to size without replacement.	WR
PPS_BREWER	Selects two units from each stratum with probability proportional to size. Units are extracted without replacement.	UNEQUAL_WOR
PPS_MURTHY	Selects two units from each stratum with probability proportional to size. Units are extracted without replacement.	UNEQUAL_WOR
PPS_SAMPFORD	An extension of the Brewer's method that selects more than two units from each stratum with probability proportional to size. Units are extracted without replacement.	UNEQUAL_WOR

ESTIMATION Keyword

By default, the estimation method used when sample data are analyzed is implied by the specified extraction method. If ESTIMATION=WR is specified, the with-replacement estimator is used when summary statistics are produced using Complex Samples analysis procedures.

- The WR keyword has no effect if the specified METHOD implies WR estimation.
- If ESTIMATION=WR is specified, the joint probabilities file is not created when the sample plan is executed.
- ESTIMATION=WR is available only in the first stage.

SIZE Subcommand

The SIZE subcommand specifies the number of sampling units to draw at the current stage.

- You can specify a single value, a variable name, or a matrix of counts for design strata.
- Size values must be positive integers.
- The SIZE subcommand is ignored with a warning if the PPS_MURTHY or PPS_BREWER method is specified.
- The SIZE or RATE subcommand must be specified for each stage. An error occurs if both are specified.

VALUE. *Apply a single value to all strata.* For example, VALUE=10 selects 10 units per stratum.

MATRIX. *Specify disproportionate sample sizes for different strata.* Specify one or more variables after the MATRIX keyword. Then provide one size specification per stratum. A size specification includes a set of category values and a size value. Category values should be listed in the same order as variables to which they apply. Semicolons are used to separate the size specifications.

For example, the following syntax selects 10 units from the North stratum and 20 from the South stratum:

```
/SIZE MATRIX=region;  'North' 10; 'South' 20
```

If there is more than one variable, specify one size per combination of strata. For example, the following syntax specifies rate values for combinations of Region and Sex strata:

```
/SIZE MATRIX=region sex; 'North' 'Male' 10; 'North' 'Female'15;
                         'South' 'Male' 24; 'South' 'Female' 30
```

- The variable list must contain all or a subset of stratification variables from the same and previous stages and cluster variables from the previous stages. An error occurs if the list contains variables that are not defined as strata or cluster variables.
- Each size specification must contain one category value per variable. If multiple size specifications are provided for the same strata or combination of strata, only the last one is honored.
- String and date category values must be quoted.
- A semicolon must appear after the variable list and after each size specification. The semicolon is not allowed after the last size specification.

VARIABLE. *Specify the name of a single variable that contains the sample sizes.*

RATE Subcommand

The RATE subcommand specifies the percentage of units to draw at the current stage—that is, the sampling fraction.

- Specify a single value, a variable name, or a matrix of rates for design strata. In all cases, the value 1 is treated as 100%.
- Rate values must be positive.
- RATE is ignored with a warning if the PPS_MURTHY or PPS_BREWER method is specified.
- Either SIZE or RATE must be specified for each stage. An error occurs if both are specified.

VALUE. *Apply a single value to all strata.* For example, VALUE=.10 selects 10% of units per stratum.

MATRIX. *Specify disproportionate rates for different strata.* Specify one or more variables after the MATRIX keyword. Then provide one rate specification per stratum. A rate specification includes a set of category values and a rate value. Category values should be listed in the same order as variables to which they apply. Semicolons are used to separate the rate specifications.

For example, the following syntax selects 10% of units from the North stratum and 20% from the South stratum:

```
/RATE MATRIX=region;  'North' .1; 'South' .2
```

If there is more than one variable, specify one rate per combination of strata. For example, the following syntax specifies rate values for combinations of Region and Sex strata:

```
/RATE MATRIX=region sex; 'North' 'Male' .1; 'North' 'Female' .15;
                         'South' 'Male' .24; 'South' 'Female' .3
```

- The variable list must contain all or a subset of stratification variables from the same and previous stages and cluster variables from the previous stages. An error occurs if the list contains variables that are not defined as strata or cluster variables.
- Each rate specification must contain one category value per variable. If multiple rate specifications are provided for the same strata or combination of strata, only the last one is honored.
- String and date category values must be quoted. A semicolon must appear after the variable list and after each rate specification.
- The semicolon is not allowed after the last rate specification.

VARIABLE. *Specify the name of a single variable that contains the sample rates.*

MINSIZE Keyword

MINSIZE specifies the minimum number of units to draw when RATE is specified. MINSIZE is useful when the sampling rate for a particular stratum turns out to be very small due to rounding.

value. *The value must be a positive integer.* An error occurs if the value exceeds MAXSIZE.

MAXSIZE Keyword

MAXSIZE specifies the maximum number of units to draw when RATE is specified. MAXSIZE is useful when the sampling rate for a particular stratum turns out to be larger than desired due to rounding.

value. *The value must be a positive integer.* An error occurs if the value is less than MINSIZE.

MOS Subcommand

The MOS subcommand specifies the measure of size for population units in a PPS design. Specify a variable that contains the sizes or request that sizes be determined when CSSELECT scans the sample frame.

VARIABLE. *Specify a variable containing the sizes.*

SOURCE=FROMDATA. *The CSSELECT procedure counts the number of cases that belong to each cluster to determine the MOS.* SOURCE=FROMDATA can be used only if a CLUSTER variable is defined. Otherwise, an error is generated.

- The MOS subcommand is required for PPS designs. Otherwise, it is ignored with a warning.

MIN Keyword

MIN specifies a minimum MOS for population units that overrides the value specified in the MOS variable or obtained by scanning the data.

value. *The value must be positive.* MIN must be less than or equal to MAX.

- MIN is optional for PPS methods. It is ignored for other methods.

MAX Keyword

MAX specifies a maximum MOS for population units that overrides the value specified in the MOS variable or obtained by scanning the data.

value. *The value must be positive.* MAX must be greater than or equal to MIN.

- MAX is optional for PPS methods. It is ignored for other methods.

STAGEVARS Subcommand

The STAGEVARS subcommand is used to obtain stagewise sample information variables when a sample design is executed. Certain variables are created automatically and cannot be suppressed. The names of both automatic and optional stagewise variables can be user-specified.

- Stagewise inclusion probabilities and cumulative sampling weights are always created.
- A stagewise duplication index is created only when sampling is done with replacement. A warning occurs if index variables are requested when sampling is done without replacement.
- If a keyword is specified without a variable name, a default name is used. The default name indicates the stage to which the variable applies.

Example

```
/STAGEVARS POPSIZE
          INCLPROB(SelectionProb)
```

- The syntax requests that the population size for the stage be saved using a default name.
- Inclusion probabilities for the stage will be saved using the name *SelectionProb*. (Note that inclusion probabilities are always saved when the sample design is executed. The syntax shown here requests that they be saved using a nondefault name.)

STAGEVARS Variables

The following table shows available STAGEVARS variables. See the CSSELECT algorithms document for a detailed explanation of each quantity.

If the default variable name is used, a numeric suffix that corresponds to the stage number is added to the root shown below. All names end in an underscore—for example, *InclusionProbability_1_*.

Keyword	Default root name	Description	Generated automatically when sample executed?
INCLPROB	*InclusionProbability_*	Stagewise inclusion (selection) probabilities. The proportion of units drawn from the population at a particular stage.	Yes
CUMWEIGHT	*SampleWeightCumulative_*	Cumulative sampling weight for a given stage. Takes into account prior stages.	Yes
INDEX	*Index_*	Duplication index for units selected in a given stage. The index uniquely identifies units selected more than once when sampling is done with replacement.	Yes, when sampling is done with replacement.
POPSIZE	*PopulationSize_*	Population size for a given stage.	No
SAMPSIZE	*SampleSize_*	Number of units drawn at a given stage.	No
RATE	*SamplingRate_*	Stagewise sampling rate.	No
WEIGHT	*SampleWeight_*	Sampling weight for a given stage. The inverse of the stagewise inclusion probability. Stage weights are positive for each unit selected in a particular stage.	No

ESTIMATOR Subcommand

The ESTIMATOR subcommand is used to choose an estimation method for the current stage. There is no default estimator.

Available estimators depend on the stage:

- EQUAL_WOR can be specified in any stage of the design.

- UNEQUAL_WOR can be specified in the first stage only. An error occurs if it is used in stage 2 or 3.
- WR can be specified in any stage. However, the stage in which it is specified is treated as the last stage. Any subsequent stages are ignored when the data are analyzed.

EQUAL_WOR. *Equal selection probabilities without replacement.* POPSIZE or INCLPROB must be specified.

UNEQUAL_WOR. *Unequal selection probabilities without replacement.* If POPSIZE or INCLPROB is specified, it is ignored and a warning is issued.

WR. *Selection with replacement.* If POPSIZE or INCLPROB is specified, it is ignored and a warning is issued.

POPSIZE Subcommand

The POPSIZE subcommand specifies the population size for each sample element. Specify a single value, a variable name, or a matrix of counts for design strata.

- The POPSIZE and INCLPROB subcommands are mutually exclusive. An error occurs if both are specified for a particular stage.
- Population size values must be positive integers.

VALUE. *Apply a single value to all strata.* For example, VALUE=1000 indicates that each stratum has a population size of 1,000.

MATRIX. *Specify disproportionate population sizes for different strata.* Specify one or more variables after the MATRIX keyword. Then provide one size specification per stratum. A size specification includes a set of category values and a population size value. Category values should be listed in the same order as variables to which they apply. Semicolons are used to separate the size specifications.

For example, the following syntax specifies that units in the North stratum were sampled from a population of 1,000. The population size for the South stratum is specified as 2,000:

```
/SIZE MATRIX=region;  'North' 1000; 'South' 2000
```

If there is more than one variable, specify one size per combination of strata. For example, the following syntax specifies rate values for combinations of Region and Sex strata:

```
/SIZE MATRIX=region sex; 'North' 'Male' 1000; 'North' 'Female' 1500;
                         'South' 'Male' 2400; 'South' 'Female' 3000
```

- The variable list must contain all or a subset of stratification variables from the same and previous stages and cluster variables from the previous stages. An error occurs if the list contains variables that are not defined as strata or cluster variables.
- Each size specification must contain one category value per variable. If multiple size specifications are provided for the same strata or combination of strata, only the last one is honored.
- String and date category values must be quoted.
- A semicolon must appear after the variable list and after each size specification. The semicolon is not allowed after the last size specification.

VARIABLE. *Specify the name of a single variable that contains the population sizes.*

INCLPROB Subcommand

The INCLPROB subcommand specifies the proportion of units drawn from the population at a given stage. Specify a single value, a variable name, or a matrix of inclusion probabilities for design strata.

- The POPSIZE and INCLPROB subcommands are mutually exclusive. An error occurs if both are specified for a particular stage.

- Proportions must be a positive value less than or equal to 1.

VALUE. *Apply a single value to all strata.* For example, VALUE=0.10 indicates that 10% of elements in each stratum were selected.

MATRIX. *Specify unequal proportions for different strata.* Specify one or more variables after the MATRIX keyword. Then provide one proportion per stratum. A proportion specification includes a set of category values and a proportion value. Category values should be listed in the same order as variables to which they apply. Semicolons are used to separate the proportion specifications.

For example, the following syntax indicates that 10% of units were selected from the North stratum and 20% were selected from the South stratum:

```
/INCLPROB MATRIX=region;  'North' 0.1; 'South' 0.2
```

If there is more than one variable, specify one proportion per combination of strata. For example, the following syntax specifies proportions for combinations of Region and Sex strata:

```
/INCLPROB MATRIX=region sex; 'North' 'Male' 0.1; 'North' 'Female' 0.15;
                             'South' 'Male' 0.24; 'South' 'Female' 0.3
```

- The variable list must contain all or a subset of stratification variables from the same and previous stages and cluster variables from the previous stages. An error occurs if the list contains variables that are not defined as strata or cluster variables.
- Each proportion specification must contain one category value per variable. If multiple proportions are provided for the same strata or combination of strata, only the last one is honored.
- String and date category values must be quoted.
- A semicolon must appear after the variable list and after each proportion specification. The semicolon is not allowed after the last proportion specification.

VARIABLE. *Specify the name of a single variable that contains inclusion probabilities.*

CSSELECT

CSSELECT is available in SPSS Statistics Premium Edition or the Complex Samples option.

```
CSSELECT

/PLAN FILE='file'

[/CRITERIA [STAGES=n [n [n]]] [SEED={RANDOM**}]]
                                    {value   }

[/CLASSMISSING {EXCLUDE**}]
               {INCLUDE  }

[/DATA [RENAMEVARS] [PRESORTED]]

[/SAMPLEFILE OUTFILE='savfile'|'dataset' [KEEP=varlist] [DROP=varlist]]

[/JOINTPROB OUTFILE='savfile'|'dataset']

[/SELECTRULE OUTFILE='file']

[/PRINT [SELECTION**] [CPS]]
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 16.0

- Added support for SET THREADS.

Example

```
CSSELECT
  /PLAN FILE='/survey/myfile.csplan'.
```

Overview

CSSELECT selects complex, probability-based samples from a population. CSSELECT selects units according to a sample design created using the CSPLAN procedure.

Options

Scope of Execution. By default, CSSELECT executes all stages defined in the sampling plan. Optionally, you can execute specific stages of the design. This capability is useful if a full sampling frame is not available at the outset of the sampling process, in which case new stages can be sampled as they become available. For example, CSSELECT might first be used to sample cities, then to sample blocks, and finally to sample individuals. Each time a different stage of the sampling plan would be executed.

Seed. By default, a random seed value is used by the CSSELECT random number generator. You can specify a seed to ensure that the same sample will be drawn when CSSELECT is invoked repeatedly using the same sample plan and population frame. The CSSELECT seed value is independent of the global seed specified via the SET command.

Missing Values. A case is excluded from the sample frame if it has a system-missing value for any input variable in the plan file. You can control whether user-missing values of stratification and cluster variables are treated as invalid. User-missing values of measure variables are always treated as invalid.

Input Data. If the sampling frame is sorted in advance, you can specify that the data are presorted, which may improve performance when stratification and/or clustering is requested for a large sampling frame.

Sample Data. CSSELECT writes data to the active dataset (the default) or an external file. Regardless of the data destination, CSSELECT generates final sampling weights, stagewise inclusion probabilities, stagewise cumulative sampling weights, as well as variables requested in the sampling plan.

External files or datasets produced by CSSELECT include selected cases only. By default, all variables in the active dataset are copied to the external file or dataset. Optionally, you can specify that only certain variables are to be copied.

Joint Probabilities. First-stage joint inclusion probabilities are automatically saved to an external file when the plan file specifies a PPS without-replacement sampling method. Joint probabilities are used by Complex Samples analysis procedures, such as CSDESCRIPTIVES and CSTABULATE. You can control the name and location of the joint probabilities file.

Output. By default, CSSELECT displays the distribution of selected cases by stratum. Optionally, you can display a case-processing summary.

Basic Specification

- The basic specification is a PLAN subcommand that specifies a sample design file.
- By default, CSPLAN writes output data to the active dataset including final sample weights, stagewise cumulative weights, and stagewise inclusion probabilities. See the CSPLAN design for a description of available output variables.

Operations

- CSSELECT selects sampling units according to specifications given in a sample plan. Typically, the plan is created using the CSPLAN procedure.
- In general, elements are selected. If cluster sampling is performed, groups of elements are selected.
- CSSELECT assumes that the active dataset represents the sampling frame. If a multistage sample design is executed, the active dataset should contain data for all stages. For example, if you want to sample individuals within cities and city blocks, then each case should be an individual, and city and block variables should be coded for each individual. When CSSELECT is used to execute particular stages of the sample design, the active dataset should represent the subframe for those stages only.
- A case is excluded from the sample frame if it has a system-missing value for any input variable in the plan.
- You can control whether user-missing values of stratification and cluster variables are treated as valid. By default, they are treated as invalid.
- User-missing values of measure variables are always treated as invalid.
- The CSSELECT procedure has its own seed specification that is independent of the global SET command.
- First-stage joint inclusion probabilities are automatically saved to an external file when the plan file specifies a PPS without-replacement sampling method. By default, the joint probabilities file is given the same name as the plan file (with a different extension) and is written to the same location.
- Output data must be written to an external data file if with-replacement sampling is specified in the plan file.
- This procedure uses the multithreaded options specified by SET THREADS.

Syntax Rules

- The PLAN subcommand is required. All other subcommands are optional.
- Only a single instance of each subcommand is allowed.
- An error occurs if an attribute or keyword is specified more than once within a subcommand.
- An error occurs if the same output file is specified for more than one subcommand.
- Equals signs shown in the syntax chart are required.
- Subcommand names and keywords must be spelled in full.
- Empty subcommands are not allowed.

Limitations

- WEIGHT and SPLIT FILE settings are ignored with a warning by the CSSELECT procedure.

Example

```
CSSELECT
  /PLAN FILE='/survey/myfile.csplan'
  /CRITERIA SEED=99999
  /SAMPLEFILE OUTFILE='/survey/sample.sav'.
```

- CSSELECT reads the plan file *myfile.csplan*.
- CSSELECT draws cases according to the sampling design specified in the plan file.
- Sampled cases and weights are written to an external file. By default, output data include final sample weights, stagewise inclusion probabilities, stagewise cumulative weights, and any other variables requested in the sample plan.
- The seed value for the random number generator is 99999.

PLAN Subcommand

PLAN identifies the plan file whose specifications are to be used for selecting sampling units. FILE specifies the name of the file. An error occurs if the file does not exist.

CRITERIA Subcommand

CRITERIA is used to control the scope of execution and specify a seed value.

STAGES Keyword

STAGES specifies the scope of execution.

- By default, all stages defined in the sampling plan are executed. STAGES is used to limit execution to specific stages of the design.
- Specify one or more stages. The list can include up to three integer values—for example, STAGES=1 2 3. If two or more values are provided, they must be consecutive. An error occurs if a stage is specified that does not correspond to a stage in the plan file.
- If the sample plan specifies a previous weight variable, it is used in the first stage of the plan.
- When executing latter stages of a multistage sampling design in which the earlier stages have already been sampled, CSSELECT requires the cumulative sampling weights of the last stage sampled, in order to compute the correct final sampling weights for the whole design. For example, if you have executed the first two stages of a three-stage design and saved the second-stage cumulative weights to *SampleWeightCumulative_2_*, when you sample the third stage of the design, the active dataset must contain *SampleWeightCumulative_2_* to compute the final sampling weights.

SEED Keyword

SEED specifies the random number seed used by the CSSELECT procedure.

- By default, a random seed value is selected. To replicate a particular sample, the same seed, sample plan, and sample frame should be specified when the procedure is executed.
- The CSSELECT seed value is independent of the global seed specified via the SET command.

RANDOM. *A seed value is selected at random*. This is the default.

value . *Specifies a custom seed value*. The seed value must be a positive integer.

CLASSMISSING Subcommand

CLASSMISSING is used to control whether user-missing values of classification (stratification and clustering) variables are treated as valid values. By default, they are treated as invalid.

EXCLUDE. *User-missing values of stratification and cluster variables are treated as invalid.* This is the default.

INCLUDE. *User-missing values of stratification and cluster variables are treated as valid values.*

CSSELECT always treats user-missing values of measure variables (previous weight, MOS, size, and rate) as invalid.

DATA Subcommand

DATA specifies general options concerning input and output files.

RENAMEVARS Keyword

The RENAMEVARS keyword handles name conflicts between existing variables and variables to be created by the CSSELECT procedure.

- If the RENAMEVARS keyword is not specified, conflicting variable names generate an error. This is the default.
- If output data are directed to the active dataset, RENAMEVARS specifies that an existing variable should be renamed with a warning if its name conflicts with that of a variable created by the CSSELECT procedure.
- If output data are directed to an external file or dataset, RENAMEVARS specifies that a variable to be copied from the active dataset should be renamed, with a warning if its name conflicts with that of a variable created by the CSSELECT procedure. See the SAMPLEFILE subcommand for details about copying variables from the active dataset.

PRESORTED Keyword

By default, CSSELECT assumes that the active dataset is unsorted. The PRESORTED keyword specifies that the data are sorted in advance, which may improve performance when stratification and/or clustering is requested for a large sample frame.

If PRESORTED is used, the data should be sorted first by all stratification variables then by cluster variables consecutively in each stage. The data can be sorted in ascending or descending order. For example, given a sample plan created using the following CSPLAN syntax, the sample frame should be sorted by region, ses, district, type, and school, in that order.

Example

```
CSPLAN
  /PLAN OUTFILE='/survey/myfile.csplan'
  /DESIGN STRATA=region ses CLUSTER=district type
  /SAMPLE RATE=.2 MOS=districtsize METHOD=PPS_WOR
  /DESIGN CLUSTER=school
  /SAMPLE RATE=.3 METHOD=SIMPLE_WOR.
```

An error occurs if PRESORTED is specified and the data are not sorted in proper order.

SAMPLEFILE Subcommand

SAMPLEFILE is used to write sampled units to an external file or dataset. Datasets are available during the current session but are not available in subsequent sessions unless you explicitly save them as data files.

- The external file or dataset contains sampled cases only. By default, all variables in the active dataset are copied to the external file or dataset.
- If SAMPLEFILE is specified, data are not written to the active dataset.
- SAMPLEFILE must be used if with-replacement sampling is specified in the plan file. Otherwise, an error is generated.
- KEEP and DROP can be used simultaneously; the effect is cumulative. An error occurs if you specify a variable already named on a previous DROP or one not named on a previous KEEP.

OUTFILE Keyword

The OUTFILE keyword specifies the name of the external file or the name of a dataset. An external file, a file handle, or a dataset name must be specified. If the file or dataset exists, it is overwritten without warning.

KEEP Keyword

The KEEP keyword lists variables to be copied from the active dataset to the file or dataset specified on the OUTFILE keyword. KEEP has no bearing on the active dataset.

- At least one variable must be specified.
- Variables not listed are not copied.
- An error occurs if a specified variable does not exist in the active dataset.
- Variables are copied in the order in which they are listed.

DROP Keyword

The DROP keyword excludes variables from the file or dataset specified on the OUTFILE keyword. DROP has no bearing on the active dataset.

- At least one variable must be specified.
- Variables not listed are copied.
- The ALL keyword can be used to drop all variables.
- An error occurs if a specified variable does not exist in the active dataset.

JOINTPROB Subcommand

First-stage joint inclusion probabilities are automatically saved to an external IBM SPSS Statistics data file when the plan file specifies a PPS without-replacement sampling method. By default, the joint probabilities file is given the same name as the plan file (with a different extension), and it is written to the same location. JOINTPROB is used to override the default name and location of the file.

- OUTFILE specifies the name of the file. In general, if the file exists, it is overwritten without warning.
- The joint probabilities file is generated only when the plan file specifies PPS_WOR, PPS_BREWER, PPS_SAMPFORD, or PPS_MURTHY as the sampling method. A warning is generated if JOINTPROB is used when any other sampling method is requested in the plan file.

Structure of the Joint Probabilities File

Complex Samples analysis procedures will expect the following variables in the joint probability file in the order listed below. If there are other variables beyond the joint probability variables, they will be silently ignored.

1. **Stratification variables.** These are the stratification variables used in the first stage of sampling. If there is no stratification in first stage, no stratification variables are included in the file.

2. **Cluster variables.** These are variables used to identify each primary sampling unit (PSU) within a stratum. At least one cluster variable is always included, since it is required for all selection methods that generate the joint probabilities as well as for the estimation method using them.
3. **System PSU id.** This variable labels PSU's within a stratum. The variable name used is *Unit_No_*.
4. **Joint probability variables.** These variables store the joint inclusion probabilities for each pair of units. The default names of these variables will have the form *Joint_Prob_n_*; for example, the joint inclusion probabilities of the 2nd and 3rd units will be the values located at case 2 of *Joint_Prob_3_* or case 3 of *Joint_Prob_2_*. Since the analysis procedures extract joint probabilities by location, it is safe to rename these variables at your convenience. Within each stratum, these joint inclusion probabilities will form a square symmetric matrix. Since the joint inclusion probabilities only vary for the off diagonal entries, the diagonal elements correspond to the first stage inclusion probabilities. The maximum number of joint inclusion probability variables will be equal to the maximum sample size across all strata.

Example

	county	town	Unit_No_	Joint_Prob_1_	Joint_Prob_2_	Joint_Prob_3_	Joint_Prob_4_	Joint_Prob_5_
1	1	10	1	.31	.10	.11	.12	.
2	1	11	2	.10	.39	.15	.16	.
3	1	9	3	.11	.15	.44	.21	.
4	1	12	4	.12	.16	.21	.48	.
5	2	12	1	.22	.04	.07	.08	.
6	2	6	2	.04	.23	.07	.08	.
7	2	7	3	.07	.07	.41	.19	.
8	2	2	4	.08	.08	.19	.45	.
9	3	5	1	.58	.31	.32	.	.
10	3	3	2	.31	.61	.36	.	.
11	3	4	3	.32	.36	.63	.	.
12	4	14	1	.26	.06	.06	.07	.09
13	4	8	2	.06	.29	.07	.08	.10
14	4	4	3	.06	.07	.29	.08	.10
15	4	2	4	.07	.08	.08	.33	.12
16	4	13	5	.09	.10	.10	.12	.43
17	5	3	1	.74	.25	.27	.	.
18	5	6	2	.25	.41	.13	.	.
19	5	4	3	.27	.13	.43	.	.

Data View | Variable View

Figure 23. Joint probabilities file

The file *poll_jointprob.sav* contains first-stage joint probabilities for selected townships within counties. *County* is a first-stage stratification variable, and *Township* is a cluster variable. Combinations of these variables identify all first-stage PSUs uniquely. *Unit_No_* labels PSUs within each stratum and is used to match up with *Joint_Prob_1_*, *Joint_Prob_2_*, *Joint_Prob_3_*, *Joint_Prob_4_*, and *Joint_Prob_5_*. The first two strata each have 4 PSUs; therefore, the joint inclusion probability matrices are 4×4 for these strata, and the *Joint_Prob_5_* column is left empty for these rows. Similarly, strata 3 and 5 have 3×3 joint inclusion probability matrices, and stratum 4 has a 5×5 joint inclusion probability matrix.

The need for a joint probabilities file is seen by perusing the values of the joint inclusion probability matrices. When the sampling method is not a PPS WOR method, the selection of a PSU is independent of the selection of another PSU, and their joint inclusion probability is simply the product of their inclusion probabilities. In contrast, the joint inclusion probability for Townships 9 and 10 of County 1 is approximately 0.11 (see the first case of *Joint_Prob_3_* or the third case of *Joint_Prob_1_*), or less than the product of their individual inclusion probabilities (the product of the first case of *Joint_Prob_1_* and the third case of *Joint_Prob_3_* is 0.31×0.44=0.1364).

SELECTRULE Subcommand

SELECTRULE generates a text file containing a rule that describes characteristics of selected units.

- The selection rule is not generated by default.
- OUTFILE specifies the name of the file. If the file exists, it is overwritten without warning.
- The selection rule is written in generic notation, for example—`(a EQ 1) AND (b EQ 2)'`. You can transform the selection rule into SQL code or command syntax that can be used to extract a subframe for the next stage of a multistage extraction.

PRINT Subcommand

PRINT controls output display.

SELECTION. *Summarizes the distribution of selected cases across strata*. The information is reported per design stage. The table is shown by default.

CPS. *Displays a case processing summary.*

CSTABULATE

CSTABULATE is available in SPSS Statistics Premium Edition or the Complex Samples option.

```
CSTABULATE
/PLAN FILE = file
[/JOINTPROB FILE = file]
/TABLES VARIABLES = varlist [BY varname]
[/CELLS [POPSIZE] [ROWPCT] [COLPCT] [TABLEPCT]]
[/STATISTICS [SE] [CV] [DEFF] [DEFFSQRT] [CIN [({95**  })]] [COUNT]
                                            {value}
             --- options for one-way frequency tables ---
             [CUMULATIVE]
             --- options for two-way crosstabulations ---
             [EXPECTED] [RESID] [ASRESID]]
[/TEST       --- options for one-way frequency tables ---
             [HOMOGENEITY]
             --- options for two-way crosstabulations ---
             [INDEPENDENCE]
             --- options for two-by-two crosstabulations ---
             [ODDSRATIO] [RELRISK] [RISKDIFF]]
[/SUBPOP TABLE = varname [BY varname [BY ...]] [DISPLAY = {LAYERED  }]]
                                                          {SEPARATE }
[/MISSING [SCOPE = {TABLE   }] [CLASSMISSING = {EXCLUDE }]]
                   {LISTWISE}                  {INCLUDE }
```

** Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
CSTABULATE
  /PLAN FILE = '/survey/myfile.xml'
  /TABLES VARIABLES = a.
```

Overview

CSTABULATE displays one-way frequency tables or two-way crosstabulations, and associated standard errors, design effects, confidence intervals, and hypothesis tests, for samples drawn by complex sampling methods. The procedure estimates variances by taking into account the sample design used to select the sample, including equal probability and probability proportional to size (PPS) methods, and with-replacement (WR) and without-replacement (WOR) sampling procedures. Optionally, CSTABULATE creates tables for subpopulations.

Basic Specification

- The basic specification is a PLAN subcommand and the name of a complex sample analysis specification file, which may be generated by CSPLAN, and a TABLES subcommand with at least one variable specified.
- This specification displays a population size estimate and its standard error for each cell in the defined table, as well as for all marginals.

Operations

- CSTABULATE computes table statistics for sampling designs supported by CSPLAN and CSSELECT.

- The input dataset must contain the variables to be analyzed and variables related to the sampling design.
- The complex sample analysis specification file provides an analysis plan based on the sampling design.
- For each cell and marginal in the defined table, the default output is the population size estimate and its standard error.
- WEIGHT and SPLIT FILE settings are ignored by CSTABULATE.

Syntax Rules

- The PLAN and TABLES subcommands are required. All other subcommands are optional.
- Each subcommand may be specified only once.
- Subcommands can be specified in any order.
- All subcommand names and keywords must be spelled in full.
- Equals signs (=) shown in the syntax chart are required.
- Empty subcommands are not allowed.

PLAN Subcommand

The PLAN subcommand specifies the name of an XML file containing analysis design specifications. This file is written by CSPLAN.

- The PLAN subcommand is required.

FILE. *Specifies the name of an external file.*

JOINTPROB Subcommand

The JOINTPROB subcommand is used to specify the file or dataset containing the first stage joint inclusion probabilities for the UNEQUAL_WOR estimation. CSSELECT writes this file in the same location and with the same name (but different extension) as the plan file. When the UNEQUAL_WOR estimation is specified, CSTABULATE will use the default location and name of the file unless the JOINTPROB subcommand is used to override them.

FILE. *Specifies the name of the file or dataset containing the joint inclusion probabilities*.

TABLES Subcommand

The TABLES subcommand specifies the tabulation variables.

- If a single variable list is specified, then a one-way frequency table is displayed for each variable in the list.
- If the variable list is followed by the BY keyword and a variable, then two-way crosstabulations are displayed for each pair of variables. Pairs of variables are defined by crossing the variable list to the left of the BY keyword with the variable to the right. Each variable on the left defines the row dimension in a two-way crosstabulation, and the variable to the right defines the column dimension. For example, TABLES VARIABLES = A B BY C displays two tables: *A* by *C* and *B* by *C*.
- Numeric or string variables may be specified.
- Plan file and subpopulation variables may not be specified on the TABLES subcommand.
- Within the variable list, all specified variables must be unique. Also, if a variable is specified after the BY keyword, then it must be different from all variables preceding the BY keyword.

VARIABLES. *Specifies the tabulation variables.*

CELLS Subcommand

The CELLS subcommand requests various summary value estimates associated with the table cells.

If the CELLS subcommand is not specified, then CSTABULATE displays the population size estimate for each cell in the defined table(s), as well as for all marginals. However, if the CELLS subcommand is specified, then only those summary values that are requested are displayed.

POPSIZE. *The population size estimate for each cell and marginal in a table.* This is the default output if the CELLS subcommand is not specified.

ROWPCT. *Row percentages.* The population size estimate in each cell in a row is expressed as a percentage of the population size estimate for that row. Available for two-way crosstabulations. For one-way frequency tables, specifying this keyword gives the same output as the TABLEPCT keyword.

COLPCT. *Column percentages.* The population size estimate in each cell in a column is expressed as a percentage of the population size estimate for that column. Available for two-way crosstabulations. For one-way frequency tables, specifying this keyword gives the same output as the TABLEPCT keyword.

TABLEPCT. *Table percentages.* The population size estimate in each cell of a table is expressed as a percentage of the population size estimate for that table.

STATISTICS Subcommand

The STATISTICS subcommand requests various statistics associated with the summary value estimates in the table cells.

If the STATISTICS subcommand is not specified, then CSTABULATE displays the standard error for each summary value estimate in the defined table(s) cells. However, if the STATISTICS subcommand is specified, then only those statistics that are requested are displayed.

SE. *The standard error for each summary value estimate.* This is the default output if the STATISTICS subcommand is not specified.

CV. *Coefficient of variation.*

DEFF. *Design effects.*

DEFFSQRT. *Square root of the design effects.*

CIN [(value)]. *Confidence interval.* If the CIN keyword is specified alone, then the default 95% confidence interval is computed. Optionally, CIN may be followed by a value in parentheses, where 0 ≤ value < 100.

COUNT. *Unweighted counts.* The number of valid observations in the dataset for each summary value estimate.

CUMULATIVE. *Cumulative summary value estimates.* Available for one-way frequency tables only.

EXPECTED. *Expected summary value estimates.* The summary value estimate in each cell if the two variables in a crosstabulation are statistically independent. Available for two-way crosstabulations only and displayed only if the TABLEPCT keyword is specified on the CELLS subcommand.

RESID. *Residuals.* The difference between the observed and expected summary value estimates in each cell. Available for two-way crosstabulations only and displayed only if the TABLEPCT keyword is specified on the CELLS subcommand.

ASRESID. *Adjusted Pearson residuals.* Available for two-way crosstabulations only and displayed only if the TABLEPCT keyword is specified on the CELLS subcommand.

TEST Subcommand

The TEST subcommand requests statistics or tests for summarizing the entire table.

Furthermore, if subpopulations are defined on the SUBPOP subcommand using only first-stage stratification variables (or a subset of them), then tests are performed for each subpopulation also.

HOMOGENEITY. *Test of homogeneous proportions.* Available for one-way frequency tables only.

INDEPENDENCE. *Test of independence.* Available for two-way crosstabulations only.

ODDSRATIO. *Odds ratio.* Available for two-by-two crosstabulations only.

RELRISK. *Relative risk.* Available for two-by-two crosstabulations only.

RISKDIFF. *Risk difference.* Available for two-by-two crosstabulations only.

SUBPOP Subcommand

The SUBPOP subcommand specifies subpopulations for which analyses are to be performed.

- The set of subpopulations is defined by specifying a single categorical variable, or two or more categorical variables, separated by the BY keyword, whose values are crossed.
- For example, `/SUBPOP TABLE = A` defines subpopulations based on the levels of variable *A*.
- For example, `/SUBPOP TABLE = A BY B` defines subpopulations based on crossing the levels of variables *A* and *B*.
- A maximum of 16 variables may be specified.
- Numeric or string variables may be specified.
- All specified variables must be unique.
- Stratification or cluster variables may be specified, but no other plan file variables are allowed on the SUBPOP subcommand.
- Tabulation variables may not be specified on the SUBPOP subcommand.
- The BY keyword is used to separate variables.

The DISPLAY keyword specifies the layout of results for subpopulations.

LAYERED. *Results for all subpopulations are displayed in the same table.* This is the default.

SEPARATE. *Results for different subpopulations are displayed in different tables.*

MISSING Subcommand

The MISSING subcommand specifies how missing values are handled.

- All design variables must have valid data. Cases with invalid data for any design variable are deleted from the analysis.

The SCOPE keyword specifies which cases are used in the analyses. This specification is applied to tabulation variables but not design variables.

TABLE. *Each table is based on all valid data for the tabulation variable(s) used in creating the table.* Tables for different variables may be based on different sample sizes. This is the default.

LISTWISE. *Only cases with valid data for all tabulation variables are used in creating the tables.* Tables for different variables are always based on the same sample size.

The CLASSMISSING keyword specifies whether user-missing values are treated as valid. This specification is applied to tabulation variables and categorical design variables (that is, strata, cluster, and subpopulation variables).

EXCLUDE. *Exclude user-missing values.* This is the default.

INCLUDE. *Include user-missing values.* Treat user-missing values as valid data.

CTABLES

CTABLES is available in SPSS Statistics Standard Edition or the Custom Tables Option.

Note: Square brackets that are used in the CTABLES syntax chart are required parts of the syntax and are not used to indicate optional elements. All subcommands except /TABLE are optional.

```
CTABLES

 /FORMAT MINCOLWIDTH={DEFAULT}  MAXCOLWIDTH={DEFAULT}
                     {value    }              {value    }

         UNITS={POINTS}   EMPTY= {ZERO  }   MISSING= {'.'    }
               {INCHES  }          {BLANK  }            {'chars'}
               {CM      }          {'chars'}

 /VLABELS VARIABLES= varlist

         DISPLAY= {DEFAULT}
                  {NAME     }
                  {LABEL    }
                  {BOTH     }
                  {NONE     }

 /MRSETS COUNTDUPLICATES= {NO}
                          {YES }

 /SMISSING {VARIABLE}
           {LISTWISE  }

 /PCOMPUTE &catname= EXPR(expression)

 /PPROPERTIES &catname &catname...
              LABEL= "label"
              FORMAT= summary format ... summary format
              HIDESOURCECATS= {NO**}
                              {YES }

 /WEIGHT VARIABLE=varname

 /HIDESMALLCOUNTS COUNT = {5      }
                          {integer}

 /TABLE  rows BY columns BY layers

 /SLABELS POSITION= {COLUMN}  VISIBLE= {YES}
                    {ROW    }          {NO   }
                    {LAYER }

 /CLABELS {AUTO                  }
          {ROWLABELS= {OPPOSITE} }
                     {LAYER   }
          {COLLABELS= {OPPOSITE} }
                     {LAYER   }

 /CRITERIA CILEVEL={95** }
                   {value}

 /CATEGORIES  VARIABLES= varlist

    { [value, value, value...]                              }
    { ORDER= {A} KEY= {VALUE       } MISSING= {EXCLUDE} }
             {D  }      {LABEL        }          {INCLUDE  }
                        {summary(varname)}

     TOTAL= {NO}    LABEL= "label" POSITION= {AFTER } EMPTY= {INCLUDE}
            {YES }                           {BEFORE}        {EXCLUDE  }

  Explicit value lists can include SUBTOTAL='label', HSUBTOTAL='label', MISSING, OTHERNM.


 /TITLES  CAPTION= ['text' 'text'...]
          CORNER=  ['text' 'text'...]
          TITLE=   ['text' 'text'...]
          Text can contain the symbols )DATE  )TIME  )TABLE

 /SIGTEST TYPE= CHISQUARE  ALPHA= {0.05              }
                                  {significance level}

          INCLUDEMRSETS={YES  }
                        {NO   }

          CATEGORIES={ALLVISIBLE  }
                     {SUBTOTALS   }

 /COMPARETEST TYPE= {PROP}  ALPHA= {0.05        }
                    {MEAN}         {value, value}

          ADJUST= {BONFERRONI  }
                  {BH          }
                  {NONE        }

          INCLUDEMRSETS={YES**}  MEANSVARIANCE={ALLCATS   }
                        {NO   }                {TESTEDCATS}

          CATEGORIES={ALLVISIBLE**}
                     {SUBTOTALS   }
```

```
           MERGE={NO**}
                 {YES}

           STYLE={APA** }
                 {SIMPLE}

           SHOWSIG={NO**}
                   {YES }
```

** Default if subcommand or keyword omitted.

Row, column, and layer elements each have the general form

```
varname {[C]} [summary 'label' format...] {+}  varname ...
        {[S]}                             {>}
```

When nesting (>) and concatenation (+) are combined, as in a + b > c, nesting occurs before concatenation; parentheses can be used to change precedence, as in (a + b) > c.

Summary functions available for all variables

COUNT ROWPCT.COUNT COLPCT.COUNT TABLEPCT.COUNT SUBTABLEPCT.COUNT LAYERPCT.COUNT LAYERROWPCT.COUNT LAYERCOLPCT.COUNT ROWPCT.VALIDN COLPCT.VALIDN TABLEPCT.VALIDN SUBTABLEPCT.VALIDN LAYERPCT.VALIDN LAYERROWPCT.VALIDN LAYERCOLPCT.VALIDN ROWPCT.TOTALN COLPCT.TOTALN TABLEPCT.TOTALN SUBTABLEPCT.TOTALN LAYERPCT.TOTALN LAYERROWPCT.TOTALN LAYERCOLPCT.TOTALN

Summary functions available for scale variables and for totals and subtotals of numeric variables

MAXIMUM MEAN MEDIAN MINIMUM MISSING MODE PTILE RANGE SEMEAN STDDEV SUM TOTALN VALIDN VARIANCE ROWPCT.SUM COLPCT.SUM TABLEPCT.SUM SUBTABLEPCT.SUM LAYERPCT.SUM LAYERROWPCT.SUM LAYERCOLPCT.SUM

Summary functions available for multiple response variables and their totals

RESPONSES ROWPCT.RESPONSES COLPCT.RESPONSES TABLEPCT.RESPONSES SUBTABLEPCT.RESPONSES LAYERPCT.RESPONSES LAYERROWPCT.RESPONSES LAYERCOLPCT.RESPONSES ROWPCT.RESPONSES.COUNT COLPCT.RESPONSES.COUNT TABLEPCT.RESPONSES.COUNT SUBTABLEPCT.RESPONSES.COUNT LAYERPCT.RESPONSES.COUNT LAYERROWPCT.RESPONSES.COUNT LAYERCOLPCT.RESPONSES.COUNT ROWPCT.COUNT.RESPONSES COLPCT.COUNT.RESPONSES TABLEPCT.COUNT.RESPONSES SUBTABLEPCT.COUNT.RESPONSES LAYERPCT.COUNT.RESPONSES LAYERROWPCT. COUNT.RESPONSES LAYERCOLPCT.COUNT.RESPONSES

Summary functions for effective base weight

If you specify an effective base weight variable on the WEIGHT subcommand, the following summary statistics are available:

- ECOUNT. The adjusted count used in effective base weight calculations.
- EVALIDN. The adjusted valid N used in effective base weight calculations.
- ETOTALN. The adjusted total N used in effective base weight calculations.

Unweighted summaries

For unweighted summaries, prefix U to a function name, as in UCOUNT.

Confidence intervals and standard error

Upper and lower confidence limits are available for counts, percents, mean. median, percentiles, and sum.

- The keyword for the lower confidence limit is the summary statistic keyword followed by the suffix .LCL. For example, COUNT.LCL.

- The keyword for the upper confidence limit is the summary statistic keyword followed by the suffix .UCL. For example, COUNT.UCL.

Standard error is available for counts, percents, mean, and sum.

- The keyword for standard error is the summary statistic keyword followed by the suffix .SE. For example, COUNT.SE.

Confidence intervals and standard errors are not available for multiple response sets.

Formats for summaries

COMMAw.d DOLLARw.d Fw.d NEGPARENw.d NEQUALw.d PARENw.d PCTw.d PCTPARENw.d DOTw.d CCA...CCEw.d Nw.d Ew.d and all DATE formats

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 13.0

- HSUBTOTAL keyword introduced on the CATEGORIES subcommand.

Release 14.0

- INCLUDEMRSETS keyword introduced on the SIGTEST and COMPARETEST subcommands.
- CATEGORIES keyword introduced on the SIGTEST and COMPARETEST subcommands.
- MEANSVARIANCE keyword introduced on the COMPARETEST subcommand.

Release 18.0

- MERGE keyword introduced on the COMPARETEST subcommand.
- PCOMPUTE and PPROPERTIES subcommands introduced.

Release 19.0

- HIDESMALLCOUNTS subcommand introduced.

Release 24.0

- SHOWSIG keyword added to COMPARETEST subcommand.
- CRITERIA subcommand introduced.
- ADJUST=BH added to COMPARETEST subcommand for Benjamini-Hochberg correction.
- STYLE keyword added to COMPARETEST subcommand.
- Confidence intervals added for counts, percents, mean, median, and sum statistics.
- WEIGHT subcommand introduced.

Examples

```
CTABLES /TABLE POLVIEWS [COLPCT] BY AGECAT.


CTABLES /TABLE $MLTNEWS [COUNT COLPCT] BY SEX
  /SLABELS VISIBLE=NO
  /CATEGORIES VARIABLES=SEX TOTAL=YES.


CTABLES /TABLE (CONFINAN + CONBUS + CONBUS + CONEDUC
  + CONPRESS + CONMEDIC)[COUNT ROWPCT]
  /CLABELS ROWLABELS=OPPOSITE.
```

Overview

The Custom Tables procedure produces tables in one, two, or three dimensions and provides a great deal of flexibility for organizing and displaying the contents.

- In each dimension (row, column, and layer), you can stack multiple variables to concatenate tables and nest variables to create subtables. See the TABLE subcommand.
- You can let Custom Tables determine summary statistics according to the measurement level in the dictionary, or you can assign one or more summaries to specific variables and override the measurement level without altering the dictionary. See the TABLE subcommand.
- You can create multiple response sets with the MRSETS command and use them like ordinary categorical variables in a table expression. You can control the percentage base by choosing an appropriate summary function, and you can control with the MRSETS subcommand whether duplicate responses from a single respondent are counted.
- You can assign totals to categorical variables at different nesting levels to create subtable and table totals, and you can assign subtotals across subsets of the values of a variable. See the CATEGORIES subcommand.
- You can determine, on a per-variable basis, which categories to display in the table, including whether to display missing values and empty categories for which variable labels exist. You can also sort categories by name, label, or the value of a summary function. See the CATEGORIES subcommand.
- You can specify whether to show or hide summary and category labels and where to position the labels. For variable labels, you can specify whether to show labels, names, both, or neither. See the SLABELS, CLABELS, and VLABELS subcommands.
- You can request chi-square tests and pairwise comparisons of column proportions and means. See the SIGTEST and COMPARETEST subcommands.
- You can assign custom titles and captions (see the TITLES subcommand) and control what is displayed for empty cells and those for which a summary function cannot be computed. See the FORMAT subcommand.
- CTABLES ignores SPLIT FILE requests if layered splits (compare groups in the graphical user interface) are requested. You can compare groups by using the split variables at the highest nesting level for row variables. See the TABLE subcommand for nesting variables.

Note: Since measurement level can affect the results, if any variables (fields) have an unknown measurement level, an initial data pass will be performed to determine default measurement level for any variables with an unknown measurement level. For information on the criteria used to determine default measurement level, see SET SCALEMIN.

Syntax Conventions

- The basic specification is a TABLE subcommand with at least one variable in one dimension. Multiple TABLE subcommands can be included in one CTABLES command.
- The global subcommands FORMAT, VLABELS, MRSETS, and SMISSING must precede the first TABLE subcommand and can be named in any order.
- The local subcommands SLABELS, CLABELS, CATEGORIES, TITLES, SIGTEST, and COMPARETEST follow the TABLE subcommand in any order and refer to the immediately preceding table expression.
- The HIDESMALLCOUNTS subcommand applies to all TABLE subcommands and must precede the first TABLE subcommand.
- In general, if subcommands are repeated, their specifications are merged. The last value of each specified attribute is honored.
- Equals signs that are shown in the syntax charts are required.
- Square brackets that are shown in the syntax charts are required.
- All keywords except summary function names, attribute values, and explicit category list keywords can be truncated to as few as three characters. Function names must be spelled in full.
- The slash before all subcommands, including the first subcommand, is required.

Examples

Example: Column Percentages

```
CTABLES /TABLE POLVIEWS [COLPCT] BY AGECAT.
```

		Age category					
		Less than 25	25 to 34	35 to 44	45 to 54	55 to 64	65 or older
		Column N %	Column N %	Column N %	Column N %	Column N %	Column N %
Think of self as liberal or conservative	Extremely liberal	4.5%	2.5%	2.1%	2.4%	1.3%	2.2%
	Liberal	18.8%	15.7%	14.6%	11.3%	10.5%	9.4%
	Slightly liberal	13.5%	14.2%	13.2%	15.4%	10.5%	10.5%
	Moderate	36.8%	37.1%	32.7%	37.2%	39.3%	38.8%
	Slightly conservative	14.3%	14.9%	19.3%	15.0%	18.4%	13.4%
	Conservative	11.7%	13.0%	14.6%	15.4%	16.4%	21.2%
	Extremely conservative	.4%	2.7%	3.5%	3.3%	3.6%	4.5%

Figure 24. Table with column percentages

- *POLVIEWS* defines the rows, and *AGECAT* defines the columns. Column percentages are requested, overriding the default COUNT function.

Example: Using a Multiple Response Set

```
CTABLES  /TABLE $MLTNEWS [COUNT COLPCT] BY SEX
  /SLABELS VISIBLE=NO
  /CATEGORIES VARIABLES=SEX TOTAL=YES.
```

		Gender					
		Male		Female		Total	
News sources	Get news from newspapers	375	41.9%	430	36.3%	805	38.7%
	Get news from news magazines	121	13.5%	173	14.6%	294	14.1%
	Get news from television	451	50.3%	626	52.8%	1077	51.8%
	Get news from radio	233	26.0%	318	26.8%	551	26.5%
	Get news from internet	359	40.1%	508	42.9%	867	41.7%

Figure 25. Multiple response set table

- *$MLTNEWS* is a multiple response set.
- The COLPCT function uses the number of respondents as the percentage base, so each cell shows the percentage of males or females who gave each response, and the sum of percentage for each column is greater than 100.
- Summary labels are hidden.
- The CATEGORIES subcommand creates a total for both sexes.

Example: Concatenation

```
CTABLES /TABLE (CONFINAN + CONBUS + CONBUS + CONEDUC
  + CONPRESS + CONMEDIC)[COUNT ROWPCT]
  /CLABELS ROWLABELS=OPPOSITE.
```

	A great deal		Only some		Hardly any	
	Count	Row N %	Count	Row N %	Count	Row N %
Confidence in banks & financial institutions	490	26.3%	1068	57.3%	306	16.4%
Confidence in major companies	500	27.5%	1078	59.2%	243	13.3%
Confidence in major companies	500	27.5%	1078	59.2%	243	13.3%
Confidence in education	511	27.2%	1055	56.1%	315	16.7%
Confidence in press	176	9.5%	878	47.2%	808	43.4%
Confidence in medicine	844	45.0%	864	46.1%	167	8.9%

Figure 26. Table with category labels in columns

- The six confidence variables all have the same categories with the same value labels for each category.
- The CLABELS subcommand moves the category labels to the columns.

TABLE Subcommand

The TABLE subcommand specifies the structure of the table, including the variables and summary functions that define each dimension. The TABLE subcommand has the general form

```
/TABLE  rows BY columns BY layers
```

The minimum specification for a row, column, or layer is a variable name. You can specify one or more dimensions.

Variable Types

The variables that are used in a table expression can be category variables, scale variables, or multiple response sets. Multiple response sets are defined by the MRSETS command and always begin with a $. Custom Tables uses the measurement level in the dictionary for the active data file to identify category and scale variables. You can override the default variable type for numeric variables by placing [C] or [S] after the variable name. Thus, to treat the category variable *HAPPY* as a scale variable and obtain a mean, you would specify

```
/TABLE HAPPY [S].
```

Category Variables and Multiple Response Sets

Category variables define one cell per value. See the CATEGORIES subcommand for ways of controlling how categories are displayed. Multiple response sets also define one cell per value.

Example

```
CTABLES /TABLE HAPPY.
```

		Count
General happiness	Very happy	891
	Pretty happy	1575
	Not too happy	340

Figure 27. Table with counts in rows

- The counts for *HAPPY* are in the rows.

Example

```
CTABLES /TABLE BY HAPPY.
```

General happiness		
Very happy	Pretty happy	Not too happy
Count	Count	Count
891	1575	340

Figure 28. Table with counts in columns

- The counts for *HAPPY* are in the columns.

Example

```
CTABLES /TABLE BY BY HAPPY
```

General happiness Very happy

Count
891

Figure 29. Table with counts in layers

- The counts for *HAPPY* are in layers.

Stacking and Nesting

Stacking (or concatenating) variables creates multiple logical tables within a single table structure.

Example

```
CTABLES /TABLE HAPPY + HAPMAR BY CHILDCAT.
```

		Number of children (grouped categories)			
		None	1-2	3-4	5 or more
		Count	Count	Count	Count
General happiness	Very happy	197	412	221	59
	Pretty happy	499	662	314	97
	Not too happy	98	136	79	27
Happiness of marriage	Very happy	111	462	232	49
	Pretty happy	51	238	133	22
	Not too happy	5	18	10	4

Figure 30. Table with multiple, stacked variables

- The output contains two tables: one table for general happiness by number of children and one table for happiness in marriage by number of children. Except for missing values, all of the cases in the data appear in both tables.

Nesting variables creates hierarchical tables.

Example

```
CTABLES /TABLE SEX > HAPMAR BY CHILDCAT.
```

				Number of children (grouped categories)			
				None	1-2	3-4	5 or more
				Count	Count	Count	Count
Gender	Male	Happiness of marriage	Very happy	48	216	102	30
			Pretty happy	25	110	58	11
			Not too happy	3	7	4	1
	Female	Happiness of marriage	Very happy	63	246	130	19
			Pretty happy	26	128	75	11
			Not too happy	2	11	6	3

Figure 31. Table with multiple, nested variables

- The output contains one table with a subtable for each value of *SEX*. The same subtables would result from the table expression `HAPMAR BY CHILDCAT BY SEX`, but the subtables would appear in separate layers.

Stacking and nesting can be combined. When they are combined, by default, nesting takes precedence over stacking. You can use parentheses to alter the order of operations.

Example

```
CTABLES /TABLE (HAPPY + HAPMAR) > SEX.
```

				Count
General happiness	Very happy	Gender	Male	373
			Female	518
	Pretty happy	Gender	Male	712
			Female	863
	Not too happy	Gender	Male	133
			Female	207
Happiness of marriage	Very happy	Gender	Male	396
			Female	459
	Pretty happy	Gender	Male	205
			Female	240
	Not too happy	Gender	Male	15
			Female	22

Figure 32. Nested and stacked table

- The output contains two tables. Without the parentheses, the first table, for general happiness, would not have separate rows for male and female.

Scale Variables

Scale variables, such as *age in years* or *population of towns*, do not define multiple cells within a table. The table expression /TABLE AGE creates a table with one cell containing the mean of *AGE* across all cases in the data. You can use nesting and/or dimensions to display summary statistics for scale variables within categories. The nature of scale variables prevents their being arranged hierarchically. Therefore:

- A scale variable cannot be nested under another scale variable.
- Scale variables can be used in only one dimension.

Example

```
CTABLES /TABLE AGE > HAPPY BY SEX.
```

			Gender	
			Male	Female
			Mean	Mean
Age of respondent	General happiness	Very happy	47	47
		Pretty happy	44	45
		Not too happy	43	47

Figure 33. Table of means for scale variable

Specifying Summaries

You can specify one or more summary functions for variables in any one dimension. For category variables, summaries can be specified only for the variables at the lowest nesting level. Thus, in the table expression

```
/TABLE SEX > (HAPPY + HAPMAR) BY AGECAT
```

you can assign summaries to *HAPPY* and *HAPMAR* or to *AGECAT*, but not to both and not to *SEX*.

If a scale variable appears in a dimension, that dimension becomes the statistics dimension, and all statistics must be specified for that dimension. A scale variable need not be at the lowest level of nesting. Thus, the following is a valid specification:

```
CTABLES /TABLE AGE [MINIMUM, MAXIMUM, MEAN] > SEX > HAPPY.
```

A multiple response variable also need not be at the lowest level of nesting. The following specification is a valid specification:

```
CTABLES /TABLE $MLTCARS [COUNT, RESPONSES] > SEX.
```

However, if two multiple response variables are nested, as in $MULTCARS > $MULTNEWS, summaries can be requested only for the variable at the innermost nesting level (in this case, *$MULTNEWS*).

The general form for a summary specification is

```
[summary 'label' format, ..., summary 'label' format]
```

- The specification follows the variable name in the table expression. You can apply a summary specification to multiple variables by enclosing the variables in parentheses. The following specifications are equivalent:

```
/TABLE SEX [COUNT] + HAPPY [COUNT, COLPCT]
/TABLE (SEX + HAPPY [COLPCT])[COUNT]
```

- The brackets are required even if only one summary is specified.
- Commas are optional.
- Label and format are both optional; defaults are used if label and format are not specified.
- You can include)CILEVEL in the quoted label to display the confidence level for confidence intervals.
- If totals or subtotals are defined for a variable (on the CATEGORIES subcommand), by default, the same functions that are specified for the variable are used for the totals. You can use the keyword TOTALS within the summary specification to specify different summary functions for the totals and subtotals. The specification then has the form `[summary 'label' format ... TOTALS [summary 'label' format...]]`. You must still specify TOTAL=YES on the CATEGORIES subcommand to see the totals.
- Summaries that are available for category variables are also available for scale variables and multiple response sets. Functions that are specific to scale variables and to multiple response sets are also available.
- If case weighting is in effect, summaries are calculated taking into account the current WEIGHT value. To obtain unweighted summaries, prefix a U to the function name, as in UCOUNT. Unweighted functions are not available where weighting would not apply, as in the MINIMUM and MAXIMUM functions.

Example

```
CTABLES /TABLE SEX > HAPMAR [COLPCT] BY CHILDCAT.
```

				Number of children (grouped categories)			
				None	1-2	3-4	5 or more
				Column N %	Column N %	Column N %	Column N %
Gender	Male	Happiness of marriage	Very happy	63.2%	64.9%	62.2%	71.4%
			Pretty happy	32.9%	33.0%	35.4%	26.2%
			Not too happy	3.9%	2.1%	2.4%	2.4%
	Female	Happiness of marriage	Very happy	69.2%	63.9%	61.6%	57.6%
			Pretty happy	28.6%	33.2%	35.5%	33.3%
			Not too happy	2.2%	2.9%	2.8%	9.1%

Figure 34. Table of nested column percentages

Example

```
CTABLES /TABLE AGECAT > TVHOURS [MEAN F5.2,
   STDDEV 'Standard Deviation' F5.2, PTILE 90 '90th Percentile'].
```

			Mean	Standard Deviation	90th Percentile
Age category	Less than 25	Hours per day watching TV	2.85	2.03	5
	25 to 34	Hours per day watching TV	2.78	2.37	5
	35 to 44	Hours per day watching TV	2.56	2.11	5
	45 to 54	Hours per day watching TV	2.58	1.97	5
	55 to 64	Hours per day watching TV	3.02	2.22	6
	65 or older	Hours per day watching TV	3.58	2.50	6

Figure 35. Table with custom percentiles and custom column labels

- Each summary function for the row variable appears by default in a column.
- Labels for standard deviation and the 90th percentile override the defaults.
- Because *TVHOURS* is recorded in whole hours and has an integer print format, the default general print formats for mean and standard deviation would also be integer, so overrides are specified.

Table 27. Summary functions: all variables

Function	Description	Default Label*	Default Format
COUNT	Number of cases in each category. This is the default for categorical and multiple response variables.	Count	Count
ECOUNT	The adjusted count used in effective base weight calculations if an effective base weight variable is specified on the WEIGHT subcommand. Otherwise, the value is the same as COUNT.	Adjusted Count	Count
ROWPCT.COUNT	Row percentage based on cell counts. Computed within subtable.	Row %	Percent
COLPCT.COUNT	Column percentage based on cell counts. Computed within subtable.	Column %	Percent
TABLEPCT.COUNT	Table percentage based on cell counts.	Table %	Percent
SUBTABLEPCT.COUNT	Subtable percentage based on cell counts.	Subtable %	Percent
LAYERPCT.COUNT	Layer percentage based on cell counts. Same as table percentage if no layers are defined.	Layer %	Percent
LAYERROWPCT.COUNT	Row percentage based on cell counts. Percentages sum to 100% across the entire row (that is, across subtables).	Layer Row %	Percent
LAYERCOLPCT.COUNT	Column percentage based on cell counts. Percentages sum to 100% across the entire column (that is, across subtables).	Layer Column %	Percent
ROWPCT.VALIDN	Row percentage based on valid count.	Row Valid N %	Percent
COLPCT.VALIDN	Column percentage based on valid count.	Column Valid N %	Percent
TABLEPCT.VALIDN	Table percentage based on valid count.	Table Valid N %	Percent
SUBTABLEPCT.VALIDN	Subtable percentage based on valid count.	Subtable Valid N %	Percent
LAYERPCT.VALIDN	Layer percentage based on valid count.	Layer Valid N %	Percent

Table 27. Summary functions: all variables (continued)

Function	Description	Default Label*	Default Format
LAYERROWPCT.VALIDN	Row percentage based on valid count. Percentages sum to 100% across the entire row.	Layer Row Valid N %	Percent
LAYERCOLPCT.VALIDN	Column percentage based on valid count. Percentages sum to 100% across the entire column.	Layer Column Valid N %	Percent
ROWPCT.TOTALN	Row percentage based on total count, including user-missing and system-missing values.	Row Total N %	Percent
COLPCT.TOTALN	Column percentage based on total count, including user-missing and system-missing values.	Column Total N %	Percent
TABLEPCT.TOTALN	Table percentage based on total count, including user-missing and system-missing values.	Table Total N %	Percent
SUBTABLEPCT.TOTALN	Subtable percentage based on total count, including user-missing and system-missing values.	Subtable Total N %	Percent
LAYERPCT.TOTALN	Layer percentage based on total count, including user-missing and system-missing values.	Layer Total N %	Percent
LAYERROWPCT.TOTALN	Row percentage based on total count, including user-missing and system-missing values. Percentages sum to 100% across the entire row.	Layer Row Total N %	Percent
LAYERCOLPCT.TOTALN	Column percentage based on total count, including user-missing and system-missing values. Percentages sum to 100% across the entire column.	Layer Column Total N %	Percent

* This is the default on a U.S.-English system.

The .COUNT suffix can be omitted from percentages that are based on cell counts. Thus, ROWPCT is equivalent to ROWPCT.COUNT.

Table 28. Summary functions: scale variables, totals, and subtotals

Function	Description	Default Label	Default Format
MAXIMUM	Largest value.	Maximum	General
MEAN	Arithmetic mean. The default for scale variables.	Mean	General
MEDIAN	50th percentile.	Median	General
MINIMUM	Smallest value.	Minimum	General

Table 28. Summary functions: scale variables, totals, and subtotals (continued)

Function	Description	Default Label	Default Format
MISSING	Count of missing values (both user-missing and system-missing).	Missing	General
MODE	Most frequent value. If there is a tie, the smallest value is shown.	Mode	General
PTILE	Percentile. Takes a numeric value between 0 and 100 as a required parameter. PTILE is computed the same way as APTILE in the TABLES command. Note that in the TABLES command, the default percentile method was HPTILE.	Percentile ####.##	General
RANGE	Difference between maximum and minimum values.	Range	General
SEMEAN	Standard error of the mean.	Std Error of Mean	General
STDDEV	Standard deviation.	Std Deviation	General
SUM	Sum of values.	Sum	General
TOTALN	Count of nonmissing, user-missing, and system-missing values. The count excludes valid values hidden via the CATEGORIES subcommand.	Total N	Count
ETOTALN	Adjusted total N used in effective base weight calculations if an effective base weight variable is specified on the WEIGHT subcommand. Otherwise, the value is the same as TOTALN. Not available for multiple response sets.	Adjusted Total N	Count
VALIDN	Count of nonmissing values.	Valid N	Count
EVALIDN	Adjusted valid N used in effective base weight calculations if an effective base weight variable is specified on the WEIGHT subcommand. Otherwise, the value is the same as VALIDN. Not available for multiple response sets.	Adjusted Valid N	Count
VARIANCE	Variance.	Variance	General
ROWPCT.SUM	Row percentage based on sums.	Row Sum %	Percent
COLPCT.SUM	Column percentage based on sums.	Column Sum %	Percent
TABLEPCT.SUM	Table percentage based on sums.	Table Sum %	Percent
SUBTABLEPCT.SUM	Subtable percentage based on sums.	Subtable Sum %	Percent
LAYERPCT.SUM	Layer percentage based on sums.	Layer Sum %	Percent
LAYERROWPCT.SUM	Row percentage based on sums. Percentages sum to 100% across the entire row.	Layer Row Sum %	Percent

Table 28. Summary functions: scale variables, totals, and subtotals (continued)

Function	Description	Default Label	Default Format
LAYERCOLPCT.SUM	Column percentage based on sums. Percentages sum to 100% across the entire column.	Layer Column Sum %	Percent

Table 29. Summary functions: multiple response sets

Function	Description	Default Label	Default Format
RESPONSES	Count of responses.	Responses	Count
ROWPCT.RESPONSES	Row percentage based on responses. Total number of responses is the denominator.	Row Responses %	Percent
COLPCT.RESPONSES	Column percentage based on responses. Total number of responses is the denominator.	Column Responses %	Percent
TABLEPCT.RESPONSES	Table percentage based on responses. Total number of responses is the denominator.	Table Responses %	Percent
SUBTABLEPCT.RESPONSES	Subtable percentage based on responses. Total number of responses is the denominator.	Subtable Responses %	Percent
LAYERPCT.RESPONSES	Layer percentage based on responses. Total number of responses is the denominator.	Layer Responses %	Percent
LAYERROWPCT.RESPONSES	Row percentage based on responses. Total number of responses is the denominator. Percentages sum to 100% across the entire row (that is, across subtables).	Layer Row Responses %	Percent
LAYERCOLPCT.RESPONSES	Column percentage based on responses. Total number of responses is the denominator. Percentages sum to 100% across the entire column (that is, across subtables).	Layer Column Responses %	Percent
ROWPCT.RESPONSES.COUNT	Row percentage: Responses are the numerator, and total count is the denominator.	Row Responses % (Base: Count)	Percent
COLPCT.RESPONSES.COUNT	Column percentage: Responses are the numerator, and total count is the denominator.	Column Responses % (Base: Count)	Percent
TABLEPCT.RESPONSES.COUNT	Table percentage: Responses are the numerator, and total count is the denominator.	Table Responses % (Base: Count)	Percent

Table 29. Summary functions: multiple response sets (continued)

Function	Description	Default Label	Default Format
SUBTABLEPCT.RESPONSES.COUNT	Subtable percentage: Responses are the numerator, and total count is the denominator.	Subtable Responses % (Base: Count)	Percent
LAYERPCT.RESPONSES.COUNT	Layer percentage: Responses are the numerator, and total count is the denominator.	Layer Responses % (Base: Count)	Percent
LAYERROWPCT.RESPONSES.COUNT	Row percentage: Responses are the numerator, and total count is the denominator. Percentages sum to 100% across the entire row (that is, across subtables).	Layer Row Responses % (Base: Count)	Percent
LAYERCOLPCT.RESPONSES.COUNT	Column percentage: Responses are the numerator, and total count is the denominator. Percentages sum to 100% across the entire column (that is, across subtables).	Layer Column Responses % (Base: Count)	Percent
ROWPCT.COUNT.RESPONSES	Row percentage: Count is the numerator, and total responses are the denominator.	Row Count % (Base: Responses)	Percent
COLPCT.COUNT.RESPONSES	Column percentage: Count is the numerator, and total responses are the denominator.	Column Count % (Base: Responses)	Percent
TABLEPCT.COUNT.RESPONSES	Table percentage: Count is the numerator, and total responses are the denominator.	Table Count % (Base: Responses)	Percent
SUBTABLEPCT.COUNT. RESPONSES	Subtable percentage: Count is the numerator, and total responses are the denominator.	Subtable Count % (Base: Responses)	Percent
LAYERPCT.COUNT. RESPONSES	Layer percentage: Count is the numerator, and total responses are the denominator.	Layer Count % (Base: Responses)	Percent
LAYERROWPCT.COUNT.RESPONSES	Row percentage: Count is the numerator, and total responses are the denominator. Percentages sum to 100% across the entire row (that is, across subtables).	Layer Row Count % (Base: Responses)	Percent
LAYERCOLPCT.COUNT.RESPONSES	Row percentage: Count is the numerator, and total responses are the denominator. Percentages sum to 100% across the entire column (that is, across subtables).	Layer Column Count % (Base: Responses)	Percent

Formats for Summaries

A default format is assigned to each summary function:

Count. The value is expressed in F (standard numeric) format with 0 decimal places. If you have fractional weights and want a count that reflects those weights, use F format with appropriate decimal places.

Percent. The value is expressed with one decimal place and a percent symbol.

General. The value is expressed in the variable's print format.

These default formats are internal to CTABLES and cannot be used in table expressions. To override the default formats, use any of the print formats that are available in the Core system except Z, PBHEX, and HEX, or use the additional formats that are described in the following table.

Table 30. Additional formats for summaries

Format	Description	Example
NEGPARENw.d	Parentheses appear around negative numbers.	–1234.567 formatted as NEGPAREN9.2 yields (1234.57).
NEQUALw.d	"N=" precedes the number.	1234.567 formatted as NEQUAL9.2 yields N=1234.57.
PARENw.d	The number is parenthesized.	1234.567 formatted as PAREN8.2 yields (1234.57).
PCTPARENw.d	A percent symbol follows the parenthesized value.	1234.567 formatted as PCTPAREN10.2 yields (1234.57%).

Missing Values in Summaries

The following table presents the rules for including cases in a table for VALIDN, COUNT, and TOTALN functions when values are included or excluded explicitly through an explicit category list or implicitly through inclusion or exclusion of user-missing values.

Table 31. Inclusion/exclusion of values in summaries

Variable and Value Type	VALIDN	COUNT	TOTALN
Categorical Variable: shown valid value	Include	Include	Include
Multiple Dichotomy Set: at least one "true" value	Include	Include	Include
Multiple Category Set: at least one shown valid value	Include	Include	Include
Scale Variable: valid value	Include	Include	Include
Categorical Variable: included user-missing value	Exclude	Include	Include
Multiple Category Set: all values are included user-missing	Exclude	Include	Include
Scale Variable: user-missing or system-missing	Exclude	Include	Include
Categorical Variable: excluded user-missing or system-missing value	Exclude	Exclude	Include
Multiple Dichotomy Set: all values are "false"	Exclude	Exclude	Include
Multiple Category Set: all values are excluded user-missing, system-missing, or excluded valid, but at least one value is not excluded valid	Exclude	Exclude	Include
Categorical Variable: excluded valid value	Exclude	Exclude	Exclude
Multiple Dichotomy Set: all values are excluded valid values	Exclude	Exclude	Exclude

SLABELS Subcommand

The SLABELS subcommand controls the position of summary statistics in the table and controls whether summary labels are shown.

```
/SLABELS POSITION= {COLUMN}  VISIBLE= {YES}
                   {ROW   }          {NO }
                   {LAYER }
```

By default, summaries appear in the columns and labels are visible.

Example: Summary Label Positioning

```
CTABLES /TABLE NEWS [COUNT COLPCT].
```

		Count	Column N %
How often does respondent read newspaper	Every day	805	43.0%
	Few times a week	420	22.5%
	Once a week	294	15.7%
	Less than once a week	202	10.8%
	Never	149	8.0%

Figure 36. Default format with summaries in columns and labels displayed

```
CTABLES /TABLE NEWS [COUNT COLPCT]
  /SLABELS POSITION=ROW VISIBLE=NO.
```

How often does respondent read newspaper	Every day	805 43.0%
	Few times a week	420 22.5%
	Once a week	294 15.7%
	Less than once a week	202 10.8%
	Never	149 8.0%

Figure 37. Table with summaries in rows and labels not displayed

CLABELS Subcommand

The CLABELS subcommand controls the location of category labels.

```
/CLABELS {AUTO                    }
         {ROWLABELS= {OPPOSITE} }
                     {LAYER    }
         {COLLABELS= {OPPOSITE} }
                     {LAYER    }
```

By default, category labels are nested under the variables to which they belong. Category labels for row and column variables can be moved to the opposite dimension or to the layers. If labels exist in both dimensions, only one dimension, row labels or column labels, can be moved; they cannot be swapped.

Example

```
CTABLES
  /TABLE (CONFINAN + CONEDUC + CONBUS + CONMEDIC + CONPRESS + CONTV )
```

		Count
Confidence in banks & financial institutions	A great deal	490
	Only some	1068
	Hardly any	306
Confidence in education	A great deal	511
	Only some	1055
	Hardly any	315
Confidence in major companies	A great deal	500
	Only some	1078
	Hardly any	243
Confidence in medicine	A great deal	844
	Only some	864
	Hardly any	167
Confidence in press	A great deal	176
	Only some	878
	Hardly any	808
Confidence in television	A great deal	196
	Only some	936
	Hardly any	744

Figure 38. Multiple variables stacked in rows with default category position

- Six variables are stacked in the rows, and their category labels are stacked under them.

```
CTABLES
  /TABLE (CONFINAN + CONEDUC + CONBUS + CONMEDIC + CONPRESS + CONTV )
  /SLABELS VISIBLE=NO   /CLABELS ROWLABELS=OPPOSITE
```

	A great	Only some	Hardly any
Confidence in banks & financial institutions	490	1068	306
Confidence in education	511	1055	315
Confidence in major companies	500	1078	243
Confidence in medicine	844	864	167
Confidence in press	176	878	808
Confidence in television	196	936	744

Figure 39. Multiple variables stacked in rows with categories in columns

- The category labels are moved to the columns. Where variables are stacked, as in this example, the value labels for all of the variables must be exactly the same to allow for this format. Additionally, all must have the same category specifications, and data-dependent sorting is not allowed.

CATEGORIES Subcommand

The CATEGORIES subcommand controls the order of categories in the rows and columns of the table, controls the showing and hiding of ordinary and user-missing values, and controls the computation of totals and subtotals.

```
/CATEGORIES  VARIABLES= varlist

    { [value, value, value...]                                  }
    { ORDER= {A} KEY= {VALUE           } MISSING= {EXCLUDE} }
             {D}      {LABEL           }          {INCLUDE}
                      {summary(varname)}

     TOTAL= {NO  }  LABEL= "label" POSITION= {AFTER } EMPTY= {INCLUDE}
            {YES }                           {BEFORE}        {EXCLUDE}
```

The minimum specification is a variable list and one of the following specifications: a category specification, TOTAL specification, or EMPTY specification. The variable list can be a list of variables or the keyword ALL, which refers to all category variables in the table expression. ALL cannot be used with the explicit category list.

Explicit Category Specification

The explicit category specification is a bracketed list of data values or value ranges in the order in which they are to be displayed in the table. Values not included in the list are excluded from the table. This form allows for subtotals and showing or hiding of specific values (both ordinary and user-missing).

- The list can include both ordinary and user-missing values but not the system-missing value (.).
- Values are optionally separated by commas.
- String and date values must be quoted. Date values must be consistent with the variable's print format.
- The LO, THRU, and HI keywords can be used in the value list to refer to a range of categories. LO and HI can be used only as part of a range specification.
- The MISSING keyword can be used to refer to all user-missing values.
- The OTHERNM keyword can be used to refer to all nonmissing values that are not explicitly named in the list. The keyword can be placed anywhere within the list. The values to which it refers appear in ascending order.
- If a value is repeated in the list, the last instance is honored. Thus, for a variable *RATING* with integer values 1 through 5, the following specifications are equal:

```
/CATEGORIES VARIABLES = RATING [1,2,4,5,3]
/CATEGORIES VARIABLES = RATING [1 THRU 5,3]
/CATEGORIES VARIABLES = RATING [OTHERNM,3]
```

- For a multiple dichotomy set, you can order the variables in the set by using the names of the variables in the set. The variable names are not enclosed in quotation marks.
- The SUBTOTAL keyword is used within a category list to request subtotals for a variable. The position of a subtotal within the list determines where it will appear in the table and the categories to which it applies. By default, a subtotal applies to all values that precede it up to the next subtotal. If POSITION=BEFORE is specified (See the topic “Totals ” on page 498 for more information.), subtotals apply to the categories that follow them in the list. Hierarchical and overlapping subtotals are not supported. You can specify a label for a subtotal by placing the label in quotation marks immediately following the SUBTOTAL keyword and an equals sign, as illustrated in the following example:

Example

```
CTABLES  /TABLE AGECAT
   /CATEGORIES VARIABLES=AGECAT [1, 2, 3, SUBTOTAL='Subtotal < 45',
   4, 5, 6, SUBTOTAL='Subtotal 45+'].
```

		Count
Age category	Less than 25	242
	25 to 34	627
	35 to 44	679
	Subtotal < 45	1548
	45 to 54	481
	55 to 64	320
	65 or older	479
	Subtotal 45+	1280

Figure 40. Table with categories and subtotals

- The HSUBTOTAL keyword functions just like the SUBTOTAL keyword, except that only the subtotal is displayed in the table; the categories that define the subtotal are not included in the table. So you can use HSUBTOTAL to collapse categories in a table without recoding the original variables.

Example

```
CTABLES  /TABLE AGECAT
   /CATEGORIES VARIABLES=AGECAT [1, 2, 3, HSUBTOTAL='Under 45',
  4, 5, 6, HSUBTOTAL='45 or older']..
```

		Count
Age category	Under 45	1548
	45 or older	1280

Figure 41. Table with subtotals only

Implicit Category Specification

The implicit list allows you to sort the categories and to show or hide user-missing values without having to enumerate the values. The implicit list also provides for data-dependent sorting. If you do not supply an explicit value list, you can use the following keywords:

ORDER. *The sorting order.* You can select A (the default) for ascending order, or D for descending order.

KEY. *The sort key.* You can specify VALUE (the default) to sort by the values or LABEL to sort by the value labels. When values are sorted by label, any unlabeled values appear after the labeled values in the table. You can also specify a summary function for data-dependent sorting.

MISSING. *Whether user-missing values are included*. You can specify EXCLUDE (the default) or INCLUDE. System-missing values are never included.

Data-Dependent Sorting. The following conventions and limitations apply to sorting by using a summary function as the key:

- The sort function must be a summary function that is supported in CTABLES.
- The sort function must be used in the table. The exception to this rule is COUNT. You can sort by COUNT even if counts do not appear in the table.
- Data-dependent sorting is not available if category labels are repositioned by using the CLABELS subcommand.
- Summary functions that are available only for scale variables require that you give the variable name in parentheses, as in MEAN(age). For percentiles, the variable name must be followed by a comma and an integer value between 0 and 100, as in PTILE(age, 75). Other functions, such as COUNT, do not require a variable name, but you can supply a variable name to restrict the sort.
- When a variable name is given, and multiple logical tables are created through stacking, the entire table is sorted based on the first logical table that includes the categorical variable that is being sorted and the variable that is specified in the key.
- When a table contains more than one dimension, the sort is based on the distribution of the key within the categories of the sorted variable, without regard to the contents of the other dimensions. Thus, given the table

  ```
  CTABLES /TABLE A BY B + C /CAT VAR=A ORDER=A KEY=COUNT(A),
  ```

  the rows are sorted according to the counts for the categories of *A*, without regard to the values of *B* and *C*. If there are no missing values in the other dimension, the result is the same as sorting on the totals for that dimension (in this case, *B* or *C*). If the other dimension has an unbalanced pattern of missing values, the sorting may give unexpected results; however, the result is unaffected by differences in the pattern for *B* and *C*.
- If the sort variable is crossed with stacked category variables, the first table in the stack determines the sort order.
- To ensure that the categories are sorted the same way in each layer of the pivot table, layer variables are ignored for the purpose of sorting.

Example

```
CTABLES
  /TABLE CAR1 BY AGECAT
  /CATEGORIES VARIABLES=AGECAT TOTAL=YES
  /CATEGORIES VARIABLES=CAR1 ORDER=D KEY=COUNT.
```

		Age category						
		Less than 25	25 to 34	35 to 44	45 to 54	55 to 64	65 or older	Total
		Count	Count	Count	Count	Count	Count	Count
Car maker, most recent car	American	99	267	293	214	140	215	1228
	Japanese	73	136	140	107	66	104	626
	German	18	91	69	63	36	61	338
	Korean	23	77	88	45	35	50	318
	Swedish	18	32	46	20	24	25	165
	Other	11	24	43	32	19	24	153

Figure 42. Table with custom sort order

- The first CATEGORIES subcommand requests a total across all age categories.
- The second CATEGORIES subcommand requests a sort of the categories of *CAR1* in descending order (using COUNT as the key). The categories of *CAR1* are sorted according to the total counts.

Example

```
CTABLES
  /TABLE AGE [MEAN F5.1] > CAR1 BY SEX
  /CATEGORIES VARIABLES=SEX TOTAL=YES
  /CATEGORIES VARIABLES=CAR1 KEY=MEAN(AGE).
```

			Gender		
			Male	Female	Total
			Mean	Mean	Mean
Age of respondent	Car maker, most recent car	Swedish	42.6	45.6	44.3
		Japanese	43.5	45.5	44.7
		Korean	43.4	46.2	45.0
		American	45.3	46.5	45.9
		German	44.3	47.6	46.2
		Other	48.6	46.4	47.3

Figure 43. Table with categories sorted by means

- The first CATEGORIES subcommand requests a total across the values of *SEX*.
- The second CATEGORIES subcommand requests that the categories of *CAR1* be sorted according to the mean of *AGE*. The categories are sorted according to the total means for both sexes, and that would be the case if the totals were not shown in the table.

Totals

A total can be specified for any category variable regardless of its level of nesting within a dimension. Totals can be requested in more than one dimension. The following options are available:

TOTAL. *Whether to display a total for a variable.* You can specify TOTAL=NO (the default) or TOTAL=YES.

LABEL. *The label for the total.* The specification is a quoted string.

POSITION. *Whether a total comes after or before the categories of the variable being totaled.* You can specify AFTER (the default) or BEFORE. POSITION also determines whether subtotals that are specified in an explicit list of categories apply to the categories that precede them (AFTER) or follow them (BEFORE).

Scale variables cannot be totaled directly. To obtain a total or subtotals for a scale variable, request the total or subtotals for the category variable within whose categories the summaries for the scale variable appear.

Example

```
CTABLES /TABLE AGECAT
  /CATEGORIES VARIABLES=AGECAT TOTAL=YES LABEL='Total Respondents'.
```

		Count
Age category	Less than 25	242
	25 to 34	627
	35 to 44	679
	45 to 54	481
	55 to 64	320
	65 or older	479
	Total Respondents	2828

Figure 44. Table with total count for a categorical variable

Example

```
CTABLES /TABLE AGE [MEAN 'Average' F5.1] > SEX
  /CATEGORIES VARIABLES=SEX TOTAL=YES LABEL='Combined'.
```

			Average
Age of respondent	Gender	Male	44.6
		Female	46.3
		Combined	45.6

Figure 45. Table with combined summary of a scale variable

- The summary function for *AGE* appears in cells that are determined by the values of *SEX*. The total is requested for *SEX* to obtain the average age across both sexes.

Empty Categories

Empty categories are those categories for which no cases appear in the data. For an explicit category list, this includes all explicitly named values and all labeled values that are implied by THRU, OTHERNM, or MISSING. For an implicit category list, this includes all values for which value labels exist.

EMPTY. *Whether to show categories whose count is zero*. You can specify EMPTY=INCLUDE (the default) or EMPTY=EXCLUDE.

CRITERIA Subcommand

The CRITERIA subcommand specifies the confidence level for confidence intervals.

The general format of the subcommand is /CRITERIA CILEVEL=value.

- CRITERIA is a local subcommand and must come after a TABLE subcommand.
- The specified confidence level applies to the preceding TABLE subcommand.
- CILELEVEL represents the confidence level, expressed as a percentage. The value must be greater than 0 and less than 100. The default value is 95.

Example

```
CTABLES
  /TABLE EdCat [C]
   [COLPCT.COUNT, COLPCT.COUNT.LCL,COLPCT.COUNT.UCL]
  /CRITERIA CILEVEL=95
  /TABLE EdCat [C]
   [COLPCT.COUNT, COLPCT.COUNT.LCL,COLPCT.COUNT.UCL]
  /CRITERIA CILEVEL=99.
```

- The 95% confidence level is used to compute the upper and lower confidence levels in the first TABLE subcommand.
- The 99% confidence level is used to compute the upper and lower confidence levels in the second TABLE subcommand.

TITLES Subcommand: Titles, Captions, and Corner Text

The TITLES subcommand specifies table annotations. If the subcommand is used, a title, caption, or corner text must be specified. No caption, title, or corner text is displayed by default.

```
/TITLES  CAPTION= ['text' 'text'...]
         CORNER=  ['text' 'text'...]
         TITLE=   ['text' 'text'...]
```

CAPTION. *Caption lines.* The caption appears below the table. Multiple lines can be specified. Each line must be quoted.

CORNER. *Corner text.* Corner text appears in the corner cell of the table, above row titles and next to column titles. Multiple lines can be specified. Each line must be quoted. Pivot tables show all corner text that fits in the corner cell. The specified text is ignored if the table has no corner cell. The system default TableLook uses the corner area for display of row dimension labels. To display CTABLES corner text, the Row Dimension Labels setting in Table Properties should be set to Nested. This choice can be preset in the default TableLook.

TITLE. *Title text.* The title appears above the table. Multiple lines can be specified. Each line must be quoted.

The following symbols can be used within any caption, corner text, or title line. Each symbol must be specified by using an opening right parenthesis and all uppercase letters.

)DATE . *Current date.* Displays a locale-appropriate date stamp that includes the year, month, and day.

)TIME. *Current time.* Displays a locale-appropriate time stamp.

)TABLE. *Table description.* Inserts a description of the table, which consists of the table expression stripped of measurement levels, statistics specifications, and /TABLE. If variable labels are available, they are used instead of variable names in the table expression.

Example

```
CTABLES /VLABELS VARIABLES=SEX HAPMAR DISPLAY=NONE
  /TABLE SEX > HAPMAR BY CHILDCAT [COLPCT]
  /SLABELS VISIBLE=NO
  /TITLE TITLE = 'Marital Happiness for Men and Women '+
               'by Number of Children'
         CAPTION= 'Report created at )TIME on )DATE' ')TABLE'.
```

Marital Happiness for Men and Women by Number of Children

		Number of children (grouped categories)			
		None	1-2	3-4	5 or more
Male	Very happy	63.2%	64.9%	62.2%	71.4%
	Pretty happy	32.9%	33.0%	35.4%	26.2%
	Not too happy	3.9%	2.1%	2.4%	2.4%
Female	Very happy	69.2%	63.9%	61.6%	57.6%
	Pretty happy	28.6%	33.2%	35.5%	33.3%
	Not too happy	2.2%	2.9%	2.8%	9.1%

Report created at 8:48:27 AM on 11/22/2004

Gender > Happiness of marriage BY Number of children (grouped categories)

Figure 46. Table with custom title and without variable labels

- The VLABELS subcommand suppresses the display of variable labels for *SEX* and *HAPMAR*.
- The SLABELS subcommand suppresses the default label for the summary function.
- The TITLE specification on the TITLE subcommand uses the standard IBM SPSS Statistics convention to break a single string across input lines.
- The CAPTION specification uses the)DATE,)TIME, and)TABLE keywords to print the date, time, and a description of the table structure.

Significance Testing

Custom Tables can perform the chi-square test of independence and pairwise comparisons of column proportions for tables that contain at least one category variable in both the rows and the columns. Custom Tables can perform pairwise comparisons of column means for tables that contain at least one summary variable in the rows and one category variable in the columns.

Chi-Square Tests: SIGTEST Subcommand

```
/SIGTEST TYPE= CHISQUARE  ALPHA= {0.05             }
                                 {significance level}

         INCLUDEMRSETS={YES**}
                       {NO   }

         CATEGORIES={ALLVISIBLE**}
                    {SUBTOTALS   }
```

The SIGTEST subcommand has the following specifications:

TYPE. *Type of significance test*. The specification is required. The only current choice is CHISQUARE.

ALPHA. *Significance level for the test*. The specification must be greater than 0 and less than 1. The default is 0.05.

INCLUDEMRSETS. *Include multiple response variables in tests*. If there are no multiple response sets, this keyword is ignored. If INCLUDEMRSETS=YES and COUNTDUPLICATES=YES on the MRSETS subcommand, multiple response sets are suppressed with a warning.

CATEGORIES. *Replacing categories with subtotals for testing*. If SUBTOTALS is specified, each subtotal replaces its categories for significance testing. If ALLVISIBLE is specified, only subtotals that are specified by using the HSUBTOTAL keyword replace their categories for testing.

Significance tests are not computed for post-computed categories specified on the PCOMPUTE subcommand.

Example

```
CTABLES /TABLE AGECAT BY MARITAL
  /CATEGORIES VARIABLES=AGECAT MARITAL TOTAL=YES
  /SIGTEST TYPE=CHISQUARE.
```

		Marital status					
		Married	Widowed	Divorced	Separated	Never married	Total
		Count	Count	Count	Count	Count	Count
Age category	Less than 25	37	1	5	5	194	242
	25 to 34	271	13	63	16	263	626
	35 to 44	379	11	129	44	116	679
	45 to 54	275	18	123	13	52	481
	55 to 64	186	31	76	7	20	320
	65 or older	197	209	48	8	17	479
	Total	1345	283	444	93	662	2827

Figure 47. Crosstabulation of two categorical variables

Pearson Chi-Square Tests

		Marital status
Age category	Chi-square	1473.381
	df	20
	Sig.	.000*

Results are based on nonempty rows and columns in each innermost subtable.

*. The Chi-square statistic is significant at the 0.05 level.

Figure 48. Chi-square test for crosstabulation

Pairwise Comparisons of Proportions and Means: COMPARETEST Subcommand

```
/COMPARETEST TYPE= {PROP}  ALPHA= {value, value}
                  {MEAN}

         ADJUST= {BONFERRONI}
                 {BH        }
                 {NONE      }

         INCLUDEMRSETS={YES**}  MEANSVARIANCE={ALLCATS   }
                       {NO   }                {TESTEDCATS}

         CATEGORIES={ALLVISIBLE**}
                    {SUBTOTALS   }

         MERGE={NO }
               {YES}

         STYLE={APA   }
               {SIMPLE}

         SHOWSIG={NO }
                 {YES}
```

TYPE
The type of pairwise comparison.

PROP
Comparison of column proportions. The table must have a categorical row variable and a categorical column variable. The table must include counts or percentages.

MEAN
Comparison of column means. The test variable must be a scale (continuous) variable, and the table must include the MEAN summary statistic.

ALPHA
The significance level for the test.

- The value must be greater than 0 and less than 1. The default value is 0.05.
- You can specify two significance levels, separated by spaces or commas.
- If you specify two significance levels, capital letters are used to identify significance values less than or equal to the smaller level. Lower case letters are used to identify significance values less than or equal to the larger level.
- If MERGE=YES and STYLE=APA, the larger significance level value is ignored.

ADJUST
The method for adjusting p values for multiple comparisons.

BONFERRONI
Bonferroni correction. This method adjusts for the family-wise error rate (FWER).

BH
Benjamini-Hochberg false discovery rate (FDR). This method is less conservative than the Bonferroni correction.

NONE
No adjustment for multiple comparisons.

INCLUDEMRSETS=NO | YES
Include multiple response variables in tests. If there are no multiple response sets, this keyword is ignored.

MEANSVARIANCE
Computation of variance for means test. This keyword is ignored unless TYPE=MEAN.

ALLCATS
For ordinary categorical variables, variance is estimated from all categories.

TESTEDCATS
For ordinary categorical variables, variance is estimated from the categories that are compared. The variance for the means test is always based on the categories that are compared for multiple response tests.

CATEGORIES
Use categories or subtotals for significance tests.

ALLVISIBLE
Significance tests are performed on regular categories. If regular categories are hidden, tests are performed on subtotals.

SUBTOTALS
For any categories included in a defined subtotal, tests are performed on the subtotals.

MERGE=NO | YES
Merge significance indicators into the main table. If NO is specified, significant differences are indicated in a separate table that follows the main table. If YES is specified, significance differences indicators are merged with the main table.

STYLE
The format for the significance indicators in the main table. This keyword is ignored if MERGE=NO.

APA
APA-style subscripts are used to indicate significant differences. This setting is the default.

SIMPLE
Each column category in the table is identified with an alphabetic key. For each significant pair, the key of the category with the smaller column mean or proportion appears in the category with the larger column mean or proportion.

SHOWSIG=NO |YES
Display the significance values. The default value is NO. If YES is specified, significance values are displayed in parentheses. Significance values are only displayed when the test results are displayed in a separate table. If SHOWSIG=YES and MERGE=YES, the MERGE specification is ignored and the test results are displayed in a separate table.

ORIGIN
This keyword is deprecated and has no effect.

Significance tests are not computed for post-computed categories specified on the PCOMPUTE subcommand.

Example

```
CTABLES /TABLE AGECAT BY MARITAL
  /CATEGORIES VARIABLES=AGECAT MARITAL TOTAL=YES
  /COMPARETEST TYPE=PROP ALPHA=.01.
```

Comparisons of Column Proportions[a]

		Marital status				
		Married	Widowed	Divorced	Separated	Never married
		(A)	(B)	(C)	(D)	(E)
Age category	Less than 25				B	A B C D
	25 to 34	B		B	B	A B C D
	35 to 44	B E		B E	A B C E	B
	45 to 54	B E		B E		
	55 to 64	E	E	E		
	65 or older	E	A C D E	E		

Results are based on two-sided tests with significance level .01. For each significant pair, the key of the category with the smaller column proportion appears under the category with the larger column proportion.

a. Tests are adjusted for all pairwise comparisons within a row of each innermost subtable using the Bonferroni correction.

Figure 49. Significance test for column differences

- The table of counts is identical to that shown in the example for chi-square above.
- The comparison output shows a number of predictable pairs for marital status among different age groups that are significant at the 0.01 level that is specified with ALPHA in the command.

Example

```
CTABLES /TABLE AGE > SEX BY MARITAL
  /CATEGORIES VARIABLES=SEX TOTAL=YES
  /COMPARETEST TYPE=MEAN.
```

			Marital status				
			Married	Widowed	Divorced	Separated	Never married
			Mean	Mean	Mean	Mean	Mean
Age of respondent	Gender	Male	49	66	48	44	32
		Female	45	70	48	41	32
		Total	47	70	48	42	32

Figure 50. Results with column difference significance in separate table

Comparisons of Column Means[a]

			Marital status				
			Married	Widowed	Divorced	Separated	Never married
			(A)	(B)	(C)	(D)	(E)
Age of respondent	Gender	Male	E	A C D E	E	E	
		Female	E	A C D E	D E	E	

Results are based on two-sided tests assuming equal variances with significance level 0.05. For each significant pair, the key of the smaller category appears under the category with larger mean.

a. Tests are adjusted for all pairwise comparisons within a row of each innermost subtable using the Bonferroni correction.

Figure 51. Separate table of column difference significance

FORMAT Subcommand

```
/FORMAT MINCOLWIDTH={DEFAULT}  MAXCOLWIDTH={DEFAULT}
                    {value  }              {value  }

        UNITS={POINTS}   EMPTY= {ZERO   }   MISSING= {'.'    }
              {INCHES}          {BLANK  }           {'chars'}
              {CM    }          {'chars'}
```

The FORMAT subcommand controls the appearance of the table. At least one of the following attributes must be specified: MINCOLWIDTH, MAXCOLWIDTH, UNITS, EMPTY, or MISSING.

MINCOLWIDTH. *The minimum width of columns in the table.* This setting includes the main tables as well as any tables of significance tests. DEFAULT honors the column labels setting in the current TableLook. The value must be less than or equal to the setting for MAXCOLWIDTH.

MAXCOLWIDTH. *The maximum width of columns in the table.* This setting includes the main tables as well as any tables of significance tests. DEFAULT honors the column labels setting in the current TableLook. The value must be greater than or equal to the setting for MINCOLWIDTH.

UNITS. *The measurement system for column width values.* The default is POINTS. You can also specify INCHES or CM (centimeters). UNITS is ignored unless MINCOLWIDTH or MAXCOLWIDTH is specified.

EMPTY. *Fill characters used when a count or percentage is zero.* ZERO (the default) displays a 0 using the format for the cell statistic. BLANK leaves the statistic blank. You can also specify a quoted character string. If the string is too wide for the cell, the text is truncated. If FORMAT EMPTY=BLANK, there will be no visible difference between cells that have a count of 0 and cells for which no statistics are defined.

MISSING. *Fill characters used when a cell statistic cannot be computed.* This specification applies to non-empty cells for which a statistic, such as standard deviation, cannot be computed. The default is a period (.). You can specify a quoted string. If the string is too wide for the cell, the text is truncated.

VLABELS Subcommand

```
/VLABELS VARIABLES=varlist
```

```
          DISPLAY={DEFAULT}
                  {NAME   }
                  {LABEL  }
                  {BOTH   }
                  {NONE   }
```

By default, the display of variable labels is controlled by the TVARS specification on the SET command in the Core system. The VLABELS subcommand allows you to show a name, label, or both for each table variable. The minimum specification is a variable list and a DISPLAY specification. To give different specifications for different variables, use multiple VLABELS subcommands.

VARIABLES. *The variables to which the subcommand applies.* You can use ALL or *VARNAME* TO *VARNAME,* which refers to the order of variables in the current active data file. If a specified variable does not appear in a table, VLABELS is ignored for that variable.

DISPLAY. *Whether the variable's name, label, both, or neither is shown in the table.* DEFAULT honors the SET TVARS setting. NAME shows the variable name only. LABEL shows the variable label only. BOTH shows the variable name and label. NONE hides the name and label.

SMISSING Subcommand

```
/SMISSING {VARIABLE}
          {LISTWISE}
```

If more than one scale variable is included in a table, you can control whether cases that are missing on one variable are included in summaries for which they have valid values.

VARIABLE. *Exclude cases variable by variable.* A case is included in summaries for each scale variable for which the case has a valid value regardless of whether the case has missing values for other scale variables in the table.

LISTWISE. *Exclude cases that are missing on any scale variable in the table.* This process ensures that summaries for all scale variables in the table are based on the same set of cases.

Listwise deletion applies on a per-table basis. Thus, given the specification

```
/TABLE (AGE [MEAN,COUNT]>SEX) + (AGE+CHILDS)[MEAN,COUNT] > HAPPY
```

all cases with valid values for *AGE* will be used in the AGE > SEX table, regardless of whether they have missing values for *CHILDS* (assuming that they also have valid values for *SEX*).

MRSETS Subcommand

```
/MRSETS COUNTDUPLICATES= {NO }
                         {YES}
```

For multiple response sets that combine multiple category variables, a respondent can select the same response for more than one of the variables. Typically, only one response is desired. For example, *$MAGS* can combine *MAG1* to *MAG5* to record which magazines a respondent reads regularly. If a respondent indicated the same magazine for *MAG1* and *MAG2*, you would not want to count that magazine twice. However, if *$CARS* combines *CAR1* to *CAR5* to indicate which cars a respondent owns now, and a respondent owns two cars of the same make, you might want to count both responses. The MRSETS subcommand allows you to specify whether duplicates are counted. By default, duplicates are not counted.

The MRSETS specification applies only to RESPONSES and percentages based on RESPONSES. MRSETS does not affect counts, which always ignore duplicates.

WEIGHT Subcommand

```
/WEIGHT=varname
```

If you have a variable that represents adjustment weights rather than frequency weights, you can use that variable as an effective base weight variable. The concept of effective base or effective sample size weighting is based in methods for the analysis of data from complex samples. An effective base weight allows for approximate handling of statistical inference in analyses involving ad hoc adjustments to data from simple random sampling designs via the use of adjustment weights.

- The effective base weight affects weighted summary statistics values and column means and column proportions significance tests.
- If weighting is turned on for the dataset (WEIGHT command), the dataset weight variable is ignored and results are weighted by the effective base weight variable.
- The effective base weight variable must be numeric.
- Cases with negative weight values, a weight value of 0, or missing weight values are excluded from all results.
- The WEIGHT subcommand is a global subcommand. The subcommand applies to all tables specified on the CTABLES command.
- The WEIGHT subcommand must appear before the first TABLE subcommand.

Weights and Rounding

Data can be weighted with the WEIGHT command or with the WEIGHT subcommand in the CTABLES command.

- If weighting with the WEIGHT command is used, non-integer weights are rounded at the cell or category level for significance tests, confidence intervals, and standard errors.
- If weighting with the WEIGHT subcommand of the CTABLES command is used, non-integer weights are not rounded.
- If both are specified, CTABLES uses the variable specified on the WEIGHT subcommand, and non-integer weights are not rounded.

PCOMPUTE Subcommand

In addition to displaying the aggregated results of summary statistics, a table can display one or more categories computed from those aggregated results and/or constant values. These are known as computed categories or postcomputes and are specified by the PCOMPUTE subcommand.

```
/PCOMPUTE &catname= EXPR(expression)
```

The left side of the equation specifies the computed category name, which must be a valid name preceded by an ampersand (&). (For more information about valid names, see the topic "Variable Names" on page 42 .) If two or more PCOMPUTE subcommands specify the same category name, the last one takes precedence. Computed categories *cannot* be referenced in a table expression or in the expression for another computed category.

The right side of the equation defines the expression used to calculate the postcomputed category. The expression is calculated for each summary statistic and can consist of:

- **References to category values**. These are enclosed in brackets and must be quoted if they are strings or dates (for example, [1] or ['Male']). All categories in an expression must be of the same type (numeric, string, or date). Both valid and user-missing values are allowed.
- **Arithmetic operators**. The supported operators are - (subtraction), + (addition), * (multiplication), / (division), and ** (exponentiation).
- **Parentheses**. These are used to control the order of evaluation.
- **Numeric constants**. Constants can be used by themselves (for example, &x=EXPR(50) or as part of a larger expression (for example, &x=EXPR(([1] + [2]) * 0.8)).

- **Keywords**. The following keywords are supported in the expression: LO, THRU, HIGH, MISSING, and OTHERNM. If a range is specified, it must appear in square brackets (for example, &x=EXPR([LO THRU HIGH]) or &x=EXPR([1 THRU 5])). The values in the range are totaled.
- **References to totals and subtotals**. If only one SUBTOTAL or HSUBTOTAL is used in a category list, the PCOMPUTE expression may refer to it by the SUBTOTAL keyword only or by keyword and position. (Both SUBTOTAL and HSUBTOTAL are referenced by the SUBTOTAL keyword in the computed category's expression.) If multiple subtotals are used, the expression must refer to them by position. The position is specified within square brackets following the keyword and must be an integer greater than or equal to 1 (for example, SUBTOTAL[2]). Totals are always referenced by keyword only.

All categories referenced in the expression must be also be explicitly specified in the CATEGORIES subcommand. As with other items in the table, the computed category's position in the CATEGORIES subcommand determines its position in the table.

By default, the values of computed categories use the same formatting for summary statistics as the other categories in the variable. ThePPROPERTIES subcommand allows you to specify different formatting. This subcommand also allows you to specify a label for the computed category and hide the categories used in the computed category's expression. See the topic “PPROPERTIES Subcommand ” on page 507 for more information.

Significance tests available on the SIGTEST and COMPARETEST subcommands are not computed for post-computed results.

Examples

```
CTABLES
  /PCOMPUTE &x=EXPR([2]-[1])
  /PPROPERTIES &x LABEL = "Female - Male"
  /TABLE SEX
  /CATEGORIES VARIABLES=SEX [1, 2, &x].
```

```
CTABLES
  /PCOMPUTE &x=EXPR([2]/[1])
  /PPROPERTIES &x LABEL = "Female / Male" FORMAT = COUNT F8.3
  /TABLE SEX
  /CATEGORIES VARIABLES=SEX [1, 2, &x].
```

```
CTABLES
  /PCOMPUTE &x = EXPR(SUBTOTAL[1] + SUBTOTAL[2])
  /PPROPERTIES &x LABEL="0-12"
  /TABLE educ[C]
  /CATEGORIES VARIABLES=educ [0 THRU 8, HSUBTOTAL="0-8", 9 THRU 12, HSUBTOTAL="9-12", &x,
   OTHERNM, HSUBTOTAL=">12"] TOTAL=YES.
```

PPROPERTIES Subcommand

```
 /PPROPERTIES &catname &catname...
              LABEL= "label"
              FORMAT= summary format ... summary format
              HIDESOURCECATS= {NO**}
                              {YES }
```

The PPROPERTIES subcommand defines properties for one or more computed categories. (For more information about creating computed categories, see the topic “PCOMPUTE Subcommand ” on page 506 .) One PPROPERTIES subcommand can be used to define properties for multiple computed categories. If multiple PPROPERTIES subcommands are specified for a single computed category, the results are cumulative.

The PPROPERTIES subcommand has the following keywords:

LABEL. *Label for the computed category*. The quoted text defines the label that appears in the table. If no label is specified, the computed category's expression is used for the label. Like other labels, the maximum length is 120 bytes. The label can also include the symbol) LABEL[n] to display the label for the *n*th category specified in the CATEGORIES subcommand.

FORMAT. *One or more pairs of summary statistics and display formats for the computed category*. Each summary statistic is followed by a display format for that statistic. For a list of summary statistics, see the

topic “Specifying Summaries ” on page 486 . For a list of formats, see the topic “Formats for Summaries ” on page 493 .

HIDESOURCECATS. *Hide categories, totals, or subtotals used in the computed category's expression.* Use YES to hide the categories, totals, or subtotals used to evaluate the computed category. This setting ignores the EMPTY keyword. If HIDESOURCECAT=YES, categories are hidden, even if EMPTY is set to INCLUDE. You cannot use YES if you request significance tests.

HIDESMALLCOUNTS Subcommand

HIDESMALLCOUNTS allows you to hide counts displayed in tables for count values that are less than a specified integer. Hidden values are displayed as **<N**, where **N** is the specified integer.

COUNT Keyword

The COUNT keyword specifies the cutoff, below which count values are hidden. Values equal to the cutoff are not hidden. The value of COUNT must be an integer and greater than or equal to 2. The default is 5. If weighting is on, the weighted value determines the cutoff.

CURVEFIT

CURVEFIT is available in Statistics Base Edition

```
CURVEFIT VARIABLES= varname [WITH varname]
 [/MODEL= [LINEAR**] [LOGARITHMIC] [INVERSE]
          [QUADRATIC] [CUBIC] [COMPOUND]
          [POWER] [S] [GROWTH] [EXPONENTIAL]
          [LGSTIC] [ALL]]
 [/CIN={95** }]
       {value}
 [/UPPERBOUND={NO**}]
              {n   }
 [/{CONSTANT† }
   {NOCONSTANT}
 [/PLOT={FIT**}]
        {NONE }
 [/ID = varname]
 [/PRINT=ANOVA]
 [/SAVE=[PRED] [RESID] [CIN]]
 [/APPLY ['model name'] [{SPECIFICATIONS}]]
                         {FIT           }
 [/TEMPLATE='filename']
```

**Default if the subcommand is omitted.

†Default if the subcommand is omitted and there is no corresponding specification on the TSET command.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 19.0

- TEMPLATE subcommand introduced.

Example

```
CURVEFIT VARIABLES = VARY
  /MODEL=CUBIC.
```

Overview

CURVEFIT fits selected curves to a line plot, allowing you to examine the relationship between one or more dependent variables and one independent variable. CURVEFIT also fits curves to time series and produces forecasts, forecast errors, lower confidence limits, and upper confidence limits. You can choose curves from a variety of regression models.

Options

Model Specification. There are 11 regression models available on the MODEL subcommand. You can fit any or all of these to the data. The keyword ALL is available to fit all 11 models. You can control whether the regression equation includes a constant term using the CONSTANT or NOCONSTANT subcommand.

Upperbound Value. You can specify the upperbound value for the logistic model using the UPPERBOUND subcommand.

Output. You can produce an analysis-of-variance summary table using the PRINT subcommand. You can suppress the display of the curve-fitting plot using the PLOT subcommand.

New Variables. To evaluate the regression statistics without saving predicted and residual variables, specify TSET NEWVAR=NONE prior to CURVEFIT. To save the new variables and replace the variables saved earlier, use TSET NEWVAR=CURRENT (the default). To save the new variables without erasing variables saved earlier, use TSET NEWVAR=ALL or the SAVE subcommand on CURVEFIT.

Forecasting. When used with the PREDICT command, CURVEFIT can produce forecasts and confidence limits beyond the end of the series. See the topic "PREDICT " on page 1501 for more information.

Templates. You can specify a template, using the TEMPLATE subcommand, to override the default chart attribute settings on your system.

Basic Specification

The basic specification is one or more dependent variables. If the variables are not time series, you must also specify the keyword WITH and an independent variable.

- By default, the LINEAR model is fit.
- A 95% confidence interval is used unless it is changed by a TSET CIN command prior to the procedure.
- CURVEFIT produces a plot of the curve, a regression summary table displaying the type of curve used, the R^2 coefficient, degrees of freedom, overall *F* test and significance level, and the regression coefficients.
- For each variable and model combination, CURVEFIT creates four variables: fit/forecast values, residuals, lower confidence limits, and upper confidence limits. These variables are automatically labeled and added to the active dataset unless TSET NEWVAR=NONE is specified prior to CURVEFIT. See the topic "SAVE Subcommand " on page 512 for more information.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- VARIABLES can be specified only once.
- Other subcommands can be specified more than once, but only the last specification of each one is executed.

Operations

- When CURVEFIT is used with the PREDICT command to forecast values beyond the end of a time series, the original and residual series are assigned the system-missing value after the last case in the original series.
- If a model requiring a log transformation (COMPOUND, POWER, S, GROWTH, EXPONENTIAL, or LGSTIC) is requested and there are values in the dependent variable(s) less than or equal to 0, the model cannot be fit because nonpositive values cannot be log-transformed.
- CURVEFIT uses listwise deletion of missing values. Whenever one dependent variable is missing a value for a particular case or observation, that case or observation will not be included in any computations.
- For the models QUADRATIC and CUBIC, a message is issued if the tolerance criterion is not met. (See TSET for information on changing the tolerance criterion.)
- Since CURVEFIT automatically generates four variables for each dependent variable and model combination, the ALL specification after MODEL should be used cautiously to avoid creating and adding to the active dataset many more variables than are necessary.
- The residual variable is always reported in the original metric. To compute the logged residual (which should be used for diagnostic checks) for the models COMPOUND, POWER, S, GROWTH, and EXPONENTIAL, specify

  ```
  COMPUTE NEWVAR = LN(VAR) - LN(FIT#n).
  ```

  where *NEWVAR* is the logged residual, *VAR* is the name of the dependent variable or observed series, and *FIT#n* is the name of the fitted variable generated by CURVEFIT.

  For the LGSTIC (logistic) model, the logged residual can be obtained by

```
COMPUTE NEWERR = LN(VAR) - LN(1/FIT#n).
```

or, if upperbound value *u* is specified on the UPPERBOUND subcommand, by

```
COMPUTE NEWVAR = LN(1/VAR - 1/u) - LN(1/FIT#n).
```

- CURVEFIT obeys the WEIGHT command when there is an independent variable. The WEIGHT specification is ignored if no independent variable is specified.

Limitations

- A maximum of 1 VARIABLES subcommand. There is no limit on the number of dependent variables or series named on the subcommand.
- A maximum of 1 independent variable can be specified after the keyword WITH.

VARIABLES Subcommand

VARIABLES specifies the variables and is the only required subcommand.

- If the dependent variables specified are not time series, you must also specify the keyword WITH and an independent variable.

MODEL Subcommand

MODEL specifies the model or models to be fit to the data. The default model is LINEAR.

- You can fit any or all of the 11 available models.
- Model name keywords can be abbreviated to the first three characters.
- You can use the keyword ALL to fit all models.
- When the LGSTIC model is specified, the upperbound value is included in the output.

The following model keywords are available:

LINEAR, LGSTIC, POWER, COMPOUND, CUBIC, QUADRATIC, INVERSE, LOGARITHMIC, S, GROWTH, EXPONENTIAL

Example

```
CURVEFIT VARIABLES = VARX.
```

- This command fits a curve to *VARX* using the linear regression model (the default).

Example

```
CURVEFIT VARIABLES = VARY
  /MODEL=GROWTH EXPONENTIAL.
```

- This command fits two curves to *VARY*, one using the growth model and the other using the exponential model.

UPPERBOUND Subcommand

UPPERBOUND is used with the logistic model (keyword LGSTIC) to specify an upper boundary value to be used in the regression equation.

- The specification on UPPERBOUND must be a positive number and must be greater than the largest data value in any of the specified dependent variables.
- The default UPPERBOUND value is infinity, so that 1/*u* = 0 and is dropped from the equation.
- You can specify UPPERBOUND NO to reset the value to infinity when applying a previous model.
- If you specify UPPERBOUND without LGSTIC, it is ignored.

- Note that UPPERBOUND is a subcommand and cannot be used within a MODEL subcommand. For example, the following specification is *not* valid:

```
/MODEL=CUBIC LGSTIC   /UPPER=99 LINEAR
```

The correct specification is:

```
/MODEL=CUBIC LGSTIC LINEAR
/UPPER=99
```

CONSTANT and NOCONSTANT Subcommands

CONSTANT and NOCONSTANT indicate whether a constant term should be estimated in the regression equation. The specification overrides the corresponding setting on the TSET command.

- CONSTANT indicates that a constant should be estimated. It is the default unless changed by TSET NOCONSTANT prior to the current procedure.
- NOCONSTANT eliminates the constant term from the model.

Example

```
CURVEFIT VARIABLES = Y1
  /MODEL=COMPOUND
  /NOCONSTANT.
```

- In this example, a compound curve is fit to *Y1* with no constant term in the model.

CIN Subcommand

CIN controls the size of the confidence interval for confidence interval variables saved with the SAVE subcommand. It has no effect on the model results displayed in output tables.

- The specification on CIN must be greater than 0 and less than 100.
- The default confidence interval is 95.
- The CIN subcommand overrides the TSET CIN setting.

PLOT Subcommand

PLOT specifies whether the curve-fitting plot is displayed. If PLOT is not specified, the default is FIT. The curve-fitting plot is displayed. PLOT=FIT is generally used with an APPLY subcommand to turn off a PLOT=NONE specification in the applied model.

FIT. *Display the curve-fitting plot.*

NONE. *Do not display the plot.*

ID Subcommand

ID specifies an identification variable. When in point selection mode, you can click on an individual chart point to display the value of the ID variable for the selected case.

SAVE Subcommand

SAVE saves the values of predicted, residual, and/or confidence interval variables generated during the current session in the active dataset.

- SAVE saves the specified variables with default names: *FIT_n* for predicted values, *ERR_n* for residuals, *LCL_n* for the lower confidence limit, and *UCL_n* for the upper confidence limit, where *n* increments each time any variable is saved for a model.
- SAVE overrides the CURRENT or NONE setting on TSET NEWVARS (see TSET).

PRED. *Predicted variable.*

RESID. *Residual variable.*

CIN. *Confidence interval.* The default is a 95% confidence interval. Use the CIN subcommand to specify a different confidence interval.

PRINT Subcommand

PRINT is used to produce an additional analysis-of-variance table for each model and variable.

- The only specification on PRINT is the keyword ANOVA.

APPLY Subcommand

APPLY allows you to use a previously defined CURVEFIT model without having to repeat the specifications.

- The specifications on APPLY can include the name of a previous model in quotes and one of two keywords. All of these specifications are optional.
- If a model name is not specified, the model specified on the previous CURVEFIT command is used.
- To change one or more of the specifications of the model, specify the subcommands of only those portions you want to change after the subcommand APPLY.
- If no variables or series are specified on the CURVEFIT command, the dependent variables that were originally specified with the model being reapplied are used.
- To change the dependent variables used with the model, enter new variable names before or after the APPLY subcommand.

The keywords available for APPLY on CURVEFIT are:

SPECIFICATIONS. *Use only the specifications from the original model.* This is the default.

FIT. *Use the coefficients estimated for the original model in the equation.*

Example

```
CURVEFIT VARIABLES = X1
  /MODEL=QUADRATIC.
CURVEFIT VARIABLES = Z1
  /APPLY.
```

- The first command fits a quadratic curve to *X1*.
- The second command fits the same type of curve to *Z1*.

Example

```
CURVEFIT VARIABLES = X1 Y1 Z1
  /MODEL=QUADRATIC.
CURVEFIT APPLY
  /MODEL=CUBIC.
```

- The first command fits quadratic curves to *X1*, *Y1*, and *Z1*.
- The second command fits curves to the same three series using the cubic model.

TEMPLATE Subcommand

TEMPLATE uses an existing file as a template and applies it to any charts generated by the current CURVEFIT command. The template overrides the default settings that are used to create any charts, and the specifications on the current CURVEFIT command override the template. Templates are created in the Chart Editor by saving an existing chart as a template.

Example

```
CURVEFIT VARIABLES = vary
  /MODEL=CUBIC
  /TEMPLATE='/templates/mytemplate.sgt'.
```

References

Abraham, B., and J. Ledolter. 1983. *Statistical methods of forecasting*. New York: John Wiley and Sons.

Draper, N. R., and H. Smith. 1981. *Applied regression analysis*, 2nd ed. New York: John Wiley and Sons.

Montgomery, D. C., and E. A. Peck. 1982. *Introduction to linear regression analysis*. New York: John Wiley and Sons.

DATA LIST

```
DATA LIST [FILE='file'] [ENCODING='encoding specification']

[{FIXED  }]
  {FREE   }  [{("delimiter", "delimiter",..., TAB)}]
  {LIST   }

[RECORDS={1}] [SKIP={n}] [{TABLE   }]
         {n}             {NOTABLE}

 /{1    } varname {col location [(format)]} [varname ...]
  {rec #}         {(FORTRAN-like format)  }

 [/{2    } ...] [/ ...]
   {rec #}
```

Table 32. Numeric and string input formats

Type	Column-style format	FORTRAN-like format
Numeric (default)	d or F,d	Fw.d
Restricted numeric	N,d	Nw.d
Scientific notation	E,d	Ew.d
Numeric with commas	COMMA,d	COMMAw.d
Numeric with dots	DOT,d	DOTw.d
Numeric with commas and dollar sign	DOLLAR,d	DOLLARw.d
Numeric with percent sign	PCT,d	PCTw.d
Zoned decimal	Z,d	Zw.d
String	A	Aw

Table 33. Format elements to skip columns

Type	Column-style format	FORTRAN-like format
Tab to column *n*		Tn
Skip *n* columns		nX

Table 34. Date and time input formats

Type	Data input	Format	FORTRAN-like format
International date	dd-mmm-yyyy	DATE	DATEw
American date	mm/dd/yyyy	ADATE	ADATEw
European date	dd/mm/yy	EDATE	EDATEw
Julian date	yyddd	JDATE	JDATEw
Sorted date	yy/mm/dd	SDATE	SDATEw
Quarter and year	qQyyyy	QYR	QYRw
Month and year	mm/yyyy	MOYR	MOYRw
Week and year	wkWKyyyy	WKYR	WKYRw
Date and time	dd-mmm-yyyy hh:mm:ss.ss	DATETIME	DATETIMEw.d
Time	hh:mm:ss.ss	TIME	TIMEw.d

Table 34. Date and time input formats (continued)

Type	Data input	Format	FORTRAN-like format
Days and time	ddd hh:mm:ss.ss	DTIME	DTIMEw.d
Day of the week	string	WKDAY	WKDAYw
Month	string	MONTH	MONTHw

Note: For default numeric (F) format and scientific notation (E) format, the decimal indicator of the input data must match the IBM SPSS Statistics locale decimal indicator (period or comma). Use SHOW DECIMAL to display the current decimal indicator and SET DECIMAL to set the decimal indicator. (Comma and Dollar formats only recognize a period as the decimal indicator, and Dot format only recognizes the comma as the decimal indicator.)

Release History

Release 16.0

- ENCODING subcommand added for Unicode support.

Example

```
DATA LIST /ID 1-3 SEX 5 (A) AGE 7-8 OPINION1 TO OPINION5 10-14.
```

Overview

DATA LIST defines a text data file by assigning names and formats to each variable in the file. Text data can be inline (entered with your commands between BEGIN DATA and END DATA) or stored in an external file. They can be in fixed format (values for the same variable are always entered in the same location on the same record for each case) or in freefield format (values for consecutive variables are not in particular columns but are entered one after the other, separated by a specified delimiter).

Important: Inline data are limited to 1024 bytes per line; any data past that limit will be truncated. Consider moving the inline data to an external file and using DATA LIST FILE=<path/file> to avoid any possible data loss.

For information on defining matrix materials, see MATRIX DATA. For information on defining complex data files that cannot be defined with DATA LIST, see FILE TYPE and REPEATING DATA. For information on reading IBM SPSS Statistics data files and portable files, see GET and IMPORT.

The program can also read data files created by other software applications. Commands that read these files include GET CAPTURE and GET TRANSLATE.

Options

Data Source. You can use inline data or data from an external file.

Data Formats. You can define numeric (with or without decimal places) and string variables using an array of input formats (percent, dollar, date and time, and so forth). You can also specify column binary and unaligned positive integer binary formats (available only if used with the MODE=MULTIPUNCH setting on the FILE HANDLE command).

Data Organization. You can define data that are in fixed format (values in the same location on the same record for each case), in freefield format with multiple cases per record, or in freefield format with one case on each record using the FIXED, FREE, and LIST keywords.

Multiple Records. For fixed-format data, you can indicate the number of records per case on the RECORDS subcommand. You can specify which records to read in the variable definition portion of DATA LIST.

Summary Table. For fixed-format data, you can display a table that summarizes the variable definitions using the TABLE subcommand. You can suppress this table using NOTABLE.

Value Delimiter. For freefield-format data (keywords FREE and LIST), you can specify the character(s) that separate data values, or you can use the keyword TAB to specify the tab character as the delimiter. Any delimiter other than the TAB keyword must be enclosed in quotation marks, and the specification must be enclosed in parentheses, as in DATA LIST FREE(",").

End-of-File Processing. You can specify a logical variable that indicates the end of the data using the END subcommand. This logical variable can be used to invoke special processing after all the cases from the data file have been read.

Basic Specification

- The basic specification is the FIXED, LIST, or FREE keyword followed by a slash that signals the beginning of variable definition.
- FIXED is the default.
- If the data are in an external file, the FILE subcommand must be used.
- If the data are inline, the FILE subcommand is omitted and the data are specified between the BEGIN DATA and END DATA commands.
- Variable definition for fixed-format data includes a variable name, a column location, and a format (unless the default numeric format is used). The column location is not specified if FORTRAN-like formats are used, since these formats include the variable width.
- Variable definition for freefield data includes a variable name and, optionally, a delimiter specification and a FORTRAN-like format specification. If format specifications include a width and number of decimal positions (for example, F8.2), the width and decimal specifications are not used to read the data but are assigned as print and write formats for the variables.

Subcommand Order

Subcommands can be named in any order. However, all subcommands must precede the first slash, which signals the beginning of variable definition.

Syntax Rules

Subcommands on DATA LIST are separated by spaces or commas, not by slashes.

Examples

```
* Column-style format specifications.

DATA LIST /ID 1-3 SEX 5 (A) AGE 7-8 OPINION1 TO OPINION5 10-14.
BEGIN DATA
001 m 28 12212
002 f 29 21212
003 f 45 32145
 ...
128 m 17 11194
END DATA.
```

- The data are inline between the BEGIN DATA and END DATA commands, so the FILE subcommand is not specified. The data are in fixed format. The keyword FIXED is not specified because it is the default.
- Variable definition begins after the slash. Variable *ID* is in columns 1 through 3. Because no format is specified, numeric format is assumed. Variable *ID* is therefore a numeric variable that is three digits wide.
- Variable *SEX* is a short string variable in column 5. Variable *SEX* is one byte wide.
- *AGE* is a two-column numeric variable in columns 7 and 8.
- Variables *OPINION1*, *OPINION2*, *OPINION3*, *OPINION4*, and *OPINION5* are named using the TO keyword. Each is a one-column numeric variable, with *OPINION1* located in column 10 and *OPINION5* located in column 14.
- The BEGIN DATA and END DATA commands enclose the inline data. Note that the values of *SEX* are lowercase letters and must be specified as such on subsequent commands.

Operations

- DATA LIST creates a new active dataset.
- Variable names are stored in the active dataset dictionary.
- Formats are stored in the active dataset dictionary and are used to display and write the values. To change output formats of numeric variables defined on DATA LIST, use the FORMATS command.
- For default numeric (F) format and scientific notation (E) format, the decimal indicator of the input data must match the IBM SPSS Statistics locale decimal indicator (period or comma). Use SHOW DECIMAL to display the current decimal indicator and SET DECIMAL to set the decimal indicator. (Comma and Dollar formats only recognize a period as the decimal indicator, and Dot format only recognizes the comma as the decimal indicator.)

Fixed-Format Data

- The order of the variables in the active dataset dictionary is the order in which they are defined on DATA LIST, not their sequence in the input data file. This order is important if you later use the TO keyword to refer to variables on subsequent commands.
- In numeric format, blanks to the left or right of a number are ignored; embedded blanks are invalid. When the program encounters a field that contains one or more blanks interspersed among the numbers, it issues a warning message and assigns the system-missing value to that case.
- Alphabetical and special characters, except the decimal point and leading plus and minus signs, are not valid in numeric variables and are set to system-missing if encountered in the data.
- See "String Formats " on page 528 for information on column width specifications for string variables in UTF-8 data files.
- The system-missing value is assigned to a completely blank field for numeric variables. The value assigned to blanks can be changed using the BLANKS specification on the SET command.
- The program ignores data contained in columns and records that are not specified in the variable definition.

Freefield Data

FREE can read freefield data with multiple cases recorded on one record or with one case recorded on more than one record. LIST can read freefield data with one case on each record.

- Line endings are read as delimiters between values.
- If you use FORTRAN-like format specifications (for example, DOLLAR12.2), width and decimal specifications are not used to read the data but are assigned as print and write formats for the variable.

For freefield data *without* explicitly specified value delimiters:

- Commas and blanks are interpreted as delimiters between values.
- Extra blanks are ignored.
- Multiple commas with or without blank space between them can be used to specify missing data.
- If a valid value contains commas or blank spaces, enclose the values in quotes.

For data with explicitly specified value delimiters (for example, DATA LIST FREE (",")):

- Multiple delimiters without any intervening space can be used to specify missing data.
- The specified delimiters cannot occur within a data value, even if you enclose the value in quotes.

Note: Freefield format with specified value delimiters is typically used to read data in text format written by a computer program, not for data manually entered in a text editor.

FILE Subcommand

FILE specifies the raw data file. FILE is required when data are stored in an external data file. FILE must not be used when the data are stored in a file that is included with the INCLUDE command or when the data are inline (see INCLUDE and BEGIN DATA–END DATA).

- FILE must be separated from other DATA LIST subcommands by at least one blank or comma.
- FILE must precede the first slash, which signals the beginning of variable definition.

ENCODING Subcommand

ENCODING specifies the encoding format of the file. The keyword is followed by an equals sign and a quoted encoding specification.

- In Unicode mode, the default is UTF8. For more information, see SET command, UNICODE subcommand.
- In code page mode, the default is the current locale setting. For more information, see SET command, LOCALE subcommand.
- The quoted encoding value can be: Locale (the current locale setting), UTF8, UTF16, UTF16BE (big endian), UTF16LE (little endian), a numeric Windows code page value (for example, '1252'), or an IANA code page value (for example, 'iso8859-1' or cp1252).
- In Unicode mode, the defined width of string variables is tripled for code page and UTF-16 text data files. Use ALTER TYPE to automatically adjust the defined width of string variables.
- See "String Formats " on page 528 for information on column width specifications for string variables in UTF-8 data files.
- If there is no FILE subcommand, the ENCODING subcommand is ignored.

FIXED, FREE, and LIST Keywords

FIXED, FREE, or LIST indicates the format of the data. Only one of these keywords can be used on each DATA LIST. The default is FIXED.

FIXED. *Fixed-format data.* Each variable is recorded in the same column location on the same record for each case in the data. FIXED is the default.

FREE. *Freefield data.* The variables are recorded in the same order for each case but not necessarily in the same column locations. More than one case can be entered on the same record. By default, values are separated by blanks or commas. You can also specify different value delimiters.

LIST. *Freefield data with one case on each record.* The variables are recorded in freefield format as described for the keyword FREE except that the variables for each case must be recorded on one record.

- FIXED, FREE, or LIST must be separated from other DATA LIST subcommands by at least one blank or comma.
- FIXED, FREE, or LIST must precede the first slash, which signals the beginning of data definition.
- For fixed-format data, you can use column-style or FORTRAN-like formats, or a combination of both. For freefield data, you can use only FORTRAN-like formats.
- For fixed-format data, the program reads values according to the column locations specified or implied by the FORTRAN-like format. Values in the data do *not* have to be in the same order as the variables named on DATA LIST and do *not* have to be separated by a space or column.
- See "String Formats " on page 528 for information on column width specifications for string variables in UTF-8 format data files.

- For freefield data, the program reads values sequentially in the order in which the variables are named on DATA LIST. Values in the data *must* be in the order in which the variables are named on DATA LIST and *must* be separated by at least one valid delimiter.
- For freefield data, multiple blank spaces can be used to indicate missing information only if a blank space is explicitly specified as the delimiter. In general, it is better to use multiple nonblank delimiters (for example, two commas with no intervening space) to specify missing data.
- In freefield format, a value cannot be split across records.

Example

```
* Data in fixed format.

DATA LIST FILE="/data/hubdata.txt" FIXED RECORDS=3
  /1 YRHIRED 14-15 DEPT 19 SEX 20.
```

- FIXED indicates explicitly that the *hubdata.txt* file is in fixed format. Because FIXED is the default, the keyword FIXED could have been omitted.
- Variable definition begins after the slash. Column locations are specified after each variable. Since formats are not specified, the default numeric format is used. Variable widths are determined by the column specifications: *YRHIRED* is two digits wide, and *DEPT* and *SEX* are each one digit wide.

Example

```
* Data in freefield format.

DATA LIST FREE / POSTPOS NWINS.
BEGIN DATA
2, 19, 7, 5, 10, 25, 5, 17, 8, 11, 3,, 6, 8, 1, 29
END DATA.
```

- Data are inline, so FILE is omitted. The keyword FREE is used because data are in freefield format with multiple cases on a single record. Two variables, *POSTPOS* and *NWINS*, are defined. Since formats are not specified, both variables receive the default F8.2 format.
- All of the data are recorded on one record. The first two values build the first case in the active dataset. For the first case, *POSTPOS* has value 2 and *NWINS* has value 19. For the second case, *POSTPOS* has value 7 and *NWINS* has value 5, and so on. The active dataset will contain eight cases.
- The two commas without intervening space after the data value 3 indicate a missing data value.

Example

```
* Data in list format.

DATA LIST LIST (",")/ POSTPOS NWINS.
BEGIN DATA
2,19
7,5
10,25
5,17
8,11
3,
6,8
1,29
END DATA.
```

- This example defines the same data as the previous example, but LIST is used because each case is recorded on a separate record. FREE could also be used. However, LIST is less prone to errors in data entry. If you leave out a value in the data with FREE format, all values after the missing value are assigned to the wrong variable. Since LIST format reads a case from each record, a missing value will affect only one case.
- A comma is specified as the delimiter between values.
- Since line endings are interpreted as delimiters between values, the second comma after the value 3 (in the sixth line of data) is not necessary to indicate that the value of *NWINS* is missing for that case.

TABLE and NOTABLE Subcommands

TABLE displays a table summarizing the variable definitions supplied on DATA LIST. NOTABLE suppresses the summary table. TABLE is the default.

- TABLE and NOTABLE can be used only for fixed-format data.
- TABLE and NOTABLE must be separated from other DATA LIST subcommands by at least one blank or comma.
- TABLE and NOTABLE must precede the first slash, which signals the beginning of variable definition.

RECORDS Subcommand

RECORDS indicates the number of records per case for fixed-format data. In the variable definition portion of DATA LIST, each record is preceded by a slash. By default, DATA LIST reads one record per case.

- The only specification on RECORDS is a single integer indicating the *total* number of records for each case (even if the DATA LIST command does not define all the records).
- RECORDS can be used only for fixed-format data and must be separated from other DATA LIST subcommands by at least one blank or comma. RECORDS must precede the first slash, which signals the beginning of variable definition.
- Each slash in the variable definition portion of DATA LIST indicates the beginning of a new record. The first slash indicates the first (or only) record. The second and any subsequent slashes tell the program to go to a new record.
- To skip a record, specify a slash without any variables for that record.
- The number of slashes in the variable definition cannot exceed the value of the integer specified on RECORDS.
- The sequence number of the record being defined can be specified after each slash. DATA LIST reads the number to determine which record to read. If the sequence number is used, you *do not* have to use a slash for any skipped records. However, the records to be read must be in their sequential order.
- The slashes for the second and subsequent records can be specified within the variable list, or they can be specified on a format list following the variable list (see the example below).
- All variables to be read from one record should be defined before you proceed to the next record.
- Since RECORDS can be used only with fixed format, it is not necessary to define all the variables on a given record or to follow their order in the input data file.

Example

```
DATA LIST FILE="/data/hubdata.txt" RECORDS=3
 /2 YRHIRED 14-15 DEPT 19 SEX 20.
```

- DATA LIST defines fixed-format data. RECORDS can be used only for fixed-format data.
- RECORDS indicates that there are three records per case in the data. Only one record per case is defined in the data definition.
- The sequence number (2) before the first variable definition indicates that the variables being defined are on the second record. Because the sequence number is provided, a slash is not required for the first record, which is skipped.
- The variables *YRHIRED*, *DEPT*, and *SEX* are defined and will be included in the active dataset. Any other variables on the second record or on the other records are not defined and are not included in the active dataset.

Example

```
DATA LIST FILE="/data/hubdata.txt" RECORDS=3
  / /YRHIRED 14-15 DEPT 19 SEX 20.
```

- This command is equivalent to the one in the previous example. Because the record sequence number is omitted, a slash is required to skip the first record.

Example

```
DATA LIST FILE="/data/hubdata.txt" RECORDS=3
  /YRHIRED (T14,F2.0) /  /NAME (T25,A24).
```

- RECORDS indicates there are three records for each case in the data.
- *YRHIRED* is the only variable defined on the first record. The FORTRAN-like format specification T14 means tab over 14 columns. Thus, *YRHIRED* begins in column 14 and has format F2.0.
- The second record is skipped. Because the record sequence numbers are not specified, a slash must be used to skip the second record.
- *NAME* is the only variable defined for the third record. *NAME* begins in column 25 and is a string variable with a width of 24 bytes (format A24).

Example

```
DATA LIST FILE="/data/hubdata.txt" RECORDS=3
  /YRHIRED NAME (T14,F2.0 /  / T25,A24).
```

- This command is equivalent to the one in the previous example. *YRHIRED* is located on the first record, and *NAME* is located on the third record.
- The slashes that indicate the second and third records are specified within the format specifications. The format specifications follow the complete variable list.

SKIP Subcommand

SKIP skips the first *n* records of the data file.

Example

```
DATA LIST LIST SKIP=2 /numvar.
BEGIN DATA
Some text describing the file
followed by some more text
1
2
3
END DATA.
```

END Subcommand

END provides control of end-of-file processing by specifying a variable that is set to a value of 0 until the end of the data file is encountered, at which point the variable is set to 1. The values of all variables named on DATA LIST are left unchanged. The logical variable created with END can then be used on DO IF and LOOP commands to invoke special processing after all of the cases from a particular input file have been built.

- DATA LIST and the entire set of commands used to define the cases must be enclosed within an INPUT PROGRAM–END INPUT PROGRAM structure. The END FILE command must also be used to signal the end of case generation.
- END can be used only with fixed-format data. An error is generated if the END subcommand is used with FREE or LIST.

Example

```
INPUT PROGRAM.
NUMERIC        TINCOME (DOLLAR8.0).                /* Total income
LEAVE          TINCOME.
DO IF          $CASENUM EQ 1.
+  PRINT       EJECT.
+  PRINT       / 'Name        Income'.
END IF
DATA LIST     FILE=INCOME END=#EOF NOTABLE / NAME 1-10(A)
                                             INCOME 16-20(F).
```

```
DO IF          #EOF.
+  PRINT       / 'TOTAL     ', TINCOME.
+  END FILE.
ELSE.
+  PRINT       / NAME, INCOME (A10,COMMA8).
+  COMPUTE     TINCOME = TINCOME+INCOME.  /* Accumulate total income
END IF.
END INPUT PROGRAM.

EXECUTE.
```

- The data definition commands are enclosed within an INPUT PROGRAM—END INPUT PROGRAM structure.
- NUMERIC indicates that a new numeric variable, *TINCOME*, will be created.
- LEAVE tells the program to leave variable *TINCOME* at its value for the previous case as each new case is read, so that it can be used to accumulate totals across cases.
- The first DO IF structure, enclosing the PRINT EJECT and PRINT commands, tells the program to display the headings *Name* and *Income* at the top of the display (when *$CASENUM* equals 1).
- DATA LIST defines variables *NAME* and *INCOME*, and it specifies the scratch variable *#EOF* on the END subcommand.
- The second DO IF prints the values for *NAME* and *INCOME* and accumulates the variable *INCOME* into *TINCOME* by passing control to ELSE as long as *#EOF* is not equal to 1. At the end of the file, *#EOF* equals 1, and the expression on DO IF is true. The label *TOTAL* and the value for *TINCOME* are displayed, and control is passed to END FILE.

Example

```
* Concatenate three raw data files.

INPUT PROGRAM.
NUMERIC #EOF1 TO #EOF3.  /*These will be used as the END variables.

DO IF #EOF1 & #EOF2 & #EOF3.
+   END FILE.
ELSE IF #EOF1 & #EOF2.
+    DATA LIST  FILE=THREE END=#EOF3 NOTABLE / NAME 1-20(A)
              AGE 25-26 SEX 29(A).
+    DO IF NOT #EOF3.
+       END CASE.
+    END IF.
ELSE IF #EOF1.
+    DATA LIST  FILE=TWO END=#EOF2 NOTABLE / NAME 1-20(A)
              AGE 21-22 SEX 24(A).
+    DO IF NOT #EOF2.
+       END CASE.
+    END IF.
ELSE.
+    DATA LIST  FILE=ONE END=#EOF1 NOTABLE /1 NAME 1-20(A)
              AGE 21-22 SEX 24 (A).
+    DO IF NOT #EOF1.
+       END CASE.
+    END IF.
END IF.
END INPUT PROGRAM.

REPORT FORMAT AUTOMATIC LIST /VARS=NAME AGE SEX.
```

- The input program contains a DO IF—ELSE IF—END IF structure.
- Scratch variables are used on each END subcommand so the value will not be reinitialized to the system-missing value after each case is built.
- Three data files are read, two of which contain data in the same format. The third requires a slightly different format for the data items. All three DATA LIST commands are placed within the DO IF structure.
- END CASE builds cases from each record of the three files. END FILE is used to trigger end-of-file processing once all data records have been read.

Variable Definition

The variable definition portion of DATA LIST assigns names and formats to the variables in the data. Depending on the format of the file, you may also need to specify record and column location. The following sections describe variable names, location, and formats.

Variable Names

- Variable names must conform to variable-naming rules. System variables (beginning with a $) cannot be defined on DATA LIST. See the topic "Variable Names" on page 42 for more information.
- The keyword TO can be used to generate names for consecutive variables in the data. Leading zeros in the number are preserved in the name. *X1* TO *X100* and *X001* TO *X100* both generate 100 variable names, but the first 99 names are not the same in the two lists. *X01* TO *X9* is not a valid specification.
- The order in which variables are named on DATA LIST determines their order in the active dataset.

Example

```
DATA LIST FREE / ID SALARY #V1 TO #V4.
```

- The FREE keyword indicates that the data are in freefield format. Six variables are defined: *ID*, *SALARY*, *#V1*, *#V2*, *#V3*, and *#V4*. *#V1* to *#V4* are scratch variables that are not stored in the active dataset. Their values can be used in transformations but not in procedure commands.

Variable Location

For fixed-format data, variable locations are specified either explicitly using column locations or implicitly using FORTRAN-like formats. For freefield data, variable locations are not specified. Values are read sequentially in the order in which variables are named on the variable list.

Fixed-Format Data

- If column-style formats are used, you must specify the column location of each variable after the variable name. If the variable is one column wide, specify the column number. Otherwise, specify the first column number followed by a dash (–) and the last column number.
- If several adjacent variables on the same record have the same width and format type, you can use one column specification after the last variable name. Specify the beginning column location of the first variable, a dash, and the ending column location of the last variable. The program divides the total number of columns specified equally among the variables. If the number of columns does not divide equally, an error message is issued.
- The same column locations can be used to define multiple variables.
- For FORTRAN-like formats, column locations are implied by the width specified on the formats. See the topic "Variable Formats " on page 525 for more information. To skip columns, use the Tn or nX format specifications.
- With fixed format, column-style and FORTRAN-like specifications can be mixed on the same DATA LIST command.
- Record location is indicated by a slash or a slash and record number before the names of the variables on that record. See the topic "RECORDS Subcommand " on page 521 for more information.
- The program ignores data in columns and on records that are not specified on DATA LIST.
- In the data, values do not have to be separated by a space or comma.
- See "String Formats " on page 528 for information on column width specifications for string variables in UTF-8 format data files.

Example

```
DATA LIST  FILE="/data/hubdata.txt" RECORDS=3
  /1 YRHIRED 14-15 DEPT 19 SEX 20
  /2 SALARY 21-25.
```

- The data are in fixed format (the default) and are read from the file *HUBDATA*.
- Three variables, *YRHIRED*, *DEPT*, and *SEX*, are defined on the first record of the *HUBDATA* file. One variable, *SALARY*, is read from columns 21 through 25 on the second record. The total number of

records per case is specified as 3 even though no variables are defined on the third record. The third record is simply skipped in data definition.

Example

```
DATA LIST  FILE="/data/hubdata.txt" RECORDS=3
  /1 DEPT 19 SEX 20 YRHIRED 14-15 MOHIRED 12-13 HIRED 12-15
  /2 SALARY 21-25.
```

- The first two defined variables are *DEPT* and *SEX*, located in columns 19 and 20 on record 1. The next three variables, *YRHIRED*, *MOHIRED*, and *HIRED*, are also located on the first record.
- *YRHIRED* is read from columns 14 and 15, *MOHIRED* from columns 12 and 13, and *HIRED* from columns 12 through 15. The variable *HIRED* is a four-column variable with the first two columns representing the month when an employee was hired (the same as *MOHIRED*) and the last two columns representing the year of employment (the same as *YRHIRED*).
- The order of the variables in the dictionary is the order in which they are defined on DATA LIST, not their sequence in the *HUBDATA* file.

Example

```
DATA LIST  FILE="/data/hubdata.txt" RECORDS=3
  /1 DEPT 19 SEX 20 MOHIRED YRHIRED 12-15
  /2 SALARY 21-25.
```

- A single column specification follows *MOHIRED* and *YRHIRED*. DATA LIST divides the total number of columns specified equally between the two variables. Thus, each variable has a width of two columns.

Example

```
* Mixing column-style and FORTRAN-like format specifications.

DATA LIST FILE=PRSNL / LNAME M_INIT STREET (A20,A1,1X,A10)
    AGE 35-36.
```

- FORTRAN-like format specifications are used for string variables *LNAME*, *M_INIT*, and *STREET*. These variables must be adjacent in the data file. *LNAME* is 20 bytes wide and is located in columns 1–20. *M_INIT* is one byte wide and is located in column 21. The 1X specification defines a blank column between *M_INIT* and *STREET*. *STREET* is 10 bytes wide and is located in columns 23–32.
- A column-style format is used for the variable *AGE*. *AGE* begins in column 35, ends in column 36, and by default has numeric format.

Freefield Data

- In freefield data, column location is irrelevant since values are not in fixed column positions. Instead, values are simply separated from each other by blanks, commas, or a specified delimiter. Any number of consecutive blanks are interpreted as one delimiter unless a blank space is explicitly specified as the value delimiter. A value cannot be split across records.
- If there are not enough values to complete the last case, a warning is issued and the incomplete case is dropped.
- The specified delimiter can only be used within data values if the value is enclosed in quotes.
- To include a single quote (apostrophe) in a string value, enclose the value in double quotes. To include double quotes in a string value, enclose the value in single quotes. See the topic “String Values in Command Specifications” on page 35 for more information.
- See “String Formats ” on page 528 for information on width specifications for string variables in UTF-8 data files.

Variable Formats

Two types of format specifications are available: column-style and FORTRAN-like. With each type, you can specify both numeric and string formats. The difference between the two types is that FORTRAN-like formats include the width of the variable and column-style formats do not.

- Column-style formats are available only for fixed-format data.
- Column-style and FORTRAN-like formats can be mixed on the same DATA LIST to define fixed-format data.
- A value that cannot be read according to the format type specified is assigned the system-missing value and a warning message is issued.

The following sections discuss the rules for specifying column-style and FORTRAN-like formats, followed by additional considerations for numeric and string formats.

Column-Style Format Specifications

The following rules apply to column-style formats:

- Data must be in a fixed format.
- Column locations must be specified after variable names. The width of a variable is determined by the number of specified columns. See the topic "Fixed-Format Data " on page 524 for more information.
- Following the column location, specify the format type in parentheses. The format type applies only to the variable or the list of variables associated with the column location specification immediately before it. If no format type is specified, numeric (F) format is used.
- To include decimal positions in the format, specify the format type followed by a comma and the number of decimal positions. For example, (DOLLAR) specifies only whole dollar amounts, and (DOLLAR,2) specifies DOLLAR format with two decimal positions.
- Since column positions are explicitly specified, the variables can be named in any order.

FORTRAN-like Format Specifications

The following rules apply to FORTRAN-like formats:

- Data can be in either fixed or freefield format.
- Column locations cannot be specified. The width of a variable is determined by the width portion (w) of the format specification. The width must specify the number of bytes in the widest value.
- One format specification applies to only one variable. The format is specified in parentheses after the variable to which it applies. Alternatively, a variable list can be followed by an equal number of format specifications contained in one set of parentheses. When a number of consecutive variables have the same format, the number can be used as a multiplying factor preceding the format. For example, (3F5.2) assigns the format F5.2 to three consecutive variables.
- For fixed data, the number of formats specified (either explicitly or implied by the multiplication factor) must be the same as the number of variables. Otherwise, the program issues an error message. If no formats are specified, all variables have the default format F8.2.
- For freefield data, variables with no specified formats take the default F8.2 format. However, an asterisk (*) must be used to indicate where the default format stops. Otherwise, the program tries to apply the next specified format to every variable before it and issues an error message if the number of formats specified is less than the number of variables.
- For freefield data, width and decimal specifications are not used to read the data but are assigned as print and write formats for the variable.
- For fixed data, Tn can be used before a format to indicate that the variable begins at the *n*th column, and nX can be used to skip *n* columns before reading the variable. When Tn is specified, variables named do not have to follow the order of the variables in the data.
- For freefield data, variables are located according to the sequence in which they are named on DATA LIST. The order of variables on DATA LIST must correspond to the order of variables in the data.
- To include decimal positions in the format for fixed-format data, specify the total width followed by a decimal point and the number of decimal positions. For example, (DOLLAR5) specifies a five-column DOLLAR format without decimal positions, and (DOLLAR5.2) specifies a five-column DOLLAR format, two columns of which are decimal positions.

Numeric Formats

- Format specifications on DATA LIST are input formats. Based on the width specification and format type, the program generates output (print and write) formats for each variable. The program automatically expands the output format to accommodate punctuation characters such as decimal points, commas, dollar signs, or date and time delimiters. (The program does not automatically expand the output formats you assign on the FORMATS, PRINT FORMATS, and WRITE FORMATS commands. For information on assigning output formats, refer to these commands.)
- Scientific notation is accepted in input data with F, COMMA, DOLLAR, DOT, and PCT formats. The same rules apply to these formats as to E format. The values 1.234E3, 1.234+3, and 1.234E 3 are all legitimate. The last value (with a blank space) will cause freefield data to be misread and therefore should be avoided when LIST or FREE is specified.

Implied Decimal Positions

- For fixed-format data, decimal positions can be coded in the data or implied by the format. If decimal positions are implied but are not entered in the data, the program interprets the rightmost digits in each value as the decimal digits. A coded decimal point in a value overrides the number of implied decimal places. For example, (DOLLAR,2) specifies two decimal positions. The value 123 is interpreted as 1.23; however, the value 12.3 is interpreted as 12.3 because the coded decimal position overrides the number of implied decimal positions.
- For freefield data, decimal positions cannot be implied but must be coded in the data. If decimal positions are specified in the format but a data value does not include a decimal point, the program fills the decimal places with zeros. For example, with F3.1 format (three columns with one decimal place), the value 22 is displayed as 22.0. If a value in the data has more decimal digits than are specified in the format, the additional decimals are truncated in displayed output (but not in calculations). For example, with F3.1 format, the value 2.22 is displayed as 2.2 even though in calculations it remains 2.22.

The table below compares how values are interpreted for fixed and freefield formats. Values in the table are for a four-column numeric variable.

Table 35. Interpretation of values in fixed and freefield format

Values	Fixed, default	Fixed, two defined decimal places	Freefield, default	Freefield, two defined decimal places
2001	2001	20.01	2001.00	2001.00
201	201	2.01	201.00	201.00
–201	–201	–2.01	–201.00	–201.00
2	2	.02	2.00	2.00
20	20	.20	20.00	20.00
2.2	2.2	2.2	2.20	2.20
.201	.201	.201	.201	.201
2 01	Undefined	Undefined	Two values	Two values

Example

```
DATA LIST
  /MODEL 1 RATE 2-6(PCT,2) COST 7-11(DOLLAR) READY 12-21(ADATE).
BEGIN DATA
1935  7878811-07-1988
2 16754654606-08-1989
3 17684783612-09-1989
END DATA.
```

- Data are inline and in fixed format (the default).

- Each variable is followed by its column location. After the column location, a column-style format is specified in parentheses.
- *MODEL* begins in column 1, is one column wide, and receives the default numeric F format.
- *RATE* begins in column 2 and ends in column 6. The PCT format is specified with two decimal places. A comma is used to separate the format type from the number of decimal places. Decimal points are not coded in the data. Thus, the program reads the rightmost digits of each value as decimal digits. The value 935 for the first case in the data is interpreted as 9.35. Note that it does not matter where numbers are entered within the column width.
- *COST* begins in column 7 and ends in column 11. DOLLAR format is specified.
- *READY* begins in column 12 and ends in column 21. ADATE format is specified.

Example

```
DATA LIST FILE="/data/data1.txt"
  /MODEL (F1) RATE (PCT5.2) COST (DOLLAR5) READY (ADATE10).
```

- In this example, the FILE subcommand is used because the data are in an external file.
- The variable definition is the same as in the preceding example except that FORTRAN-like format specifications are used rather than column-style. Column locations are not specified. Instead, the format specifications include a width for each format type.
- The width (w) portion of each format must specify the total number of bytes in the widest value. DOLLAR5 format for *COST* accepts the five-digit value 78788, which displays as $78,788. Thus, the specified input format DOLLAR5 generates an output format DOLLAR7. The program automatically expands the width of the output format to accommodate the dollar sign and comma in displayed output.

String Formats

String (alphanumeric) variables can contain any numbers, letters, or characters, including special characters and embedded blanks. Numbers entered as values for string variables cannot be used in calculations unless you convert them to numeric format (see RECODE). On DATA LIST, a string variable is defined with an A format if data are in standard character form or an AHEX format if data are in hexadecimal form.

- For fixed-format data, the width of a string variable is either implied by the column location specification or specified by the *w* on the FORTRAN-like format. For freefield data, the width must be specified on the FORTRAN-like format.
- AHEX format is available only for fixed-format data. Since each set of two hexadecimal characters represents one standard character, the width specification must be an even number. The output format for a variable in AHEX format is A format with half the specified width.
- If a string in the data is longer than its specified width, the string is truncated and a warning message is displayed. If the string in the data is shorter, it is right-padded with blanks and no warning message is displayed.
- For fixed-format data, all characters within the specified or implied columns, including leading, trailing, and embedded blanks and punctuation marks, are read as the value of the string.
- For freefield data without a specified delimiter, string values in the data must be enclosed in quotes if the string contains a blank or a comma. Otherwise, the blank or comma is treated as a delimiter between values. See the topic "String Values in Command Specifications" on page 35 for more information.

Example

```
DATA LIST FILE="/data/wins.txt" FREE /POSTPOS NWINS * POSNAME (A24).
```

- *POSNAME* is specified as a 24-byte string. The asterisk preceding *POSNAME* indicates that *POSTPOS* and *NWINS* are read with the default format. If the asterisk was not specified, the program would apply the A24 format to *POSNAME* and then issue an error message indicating that there are more variables than specified formats.

Example

```
DATA LIST FILE="/data/wins.txt" FREE /POSTPOS * NWINS (A5) POSWINS.
```

- Both *POSTPOS* and *POSWINS* receive the default numeric format F8.2.
- *NWINS* receives the specified format of A5.

String Width in Code Page, UTF-8, and UTF-16 Files

"Column" and width specifications for numbers are the same for all encodings. Each digit represents a single column. For example, a column specification of VAR1 1-4 (F) or a width specification of VAR1 (F4) defines a width wide enough for a 4-digit number.

For string variables, however, column and width specifications can be different for different encodings.

- For code page and UTF-8 files, column and width specifications for alphabetic characters represent bytes.
- In code page files, Latin (Roman) characters are each one byte. This includes characters with diacritical marks, such as accents. For example, resume and résumé both have a width of 6 bytes (or columns) in a code page file. Asian characters, however, require two bytes for each character.
- In UTF-8 files, Latin characters without diacritical marks are each one byte, but characters with diacritical marks require two bytes. For example, resume requires a width of 6 bytes while résumé requires a width of 8 bytes.
- Characters from Greek, Cyrillic, Coptic, Armenian, Hebrew, Arabic, Syriac and Tana alphabets each require two bytes in UTF-8 files.
- Characters from Asian alphabets require 3 or more bytes for each character in UTF-8 files.
- For UTF-16 files, column and width specifications represent characters. Therefore, each character requires one column for all alphabets.

For more information on Unicode encoding, go to http://www.unicode.org.

DATAFILE ATTRIBUTE

```
DATAFILE ATTRIBUTE
         ATTRIBUTE=name('value') name('value')...
                   arrayname[1]('value') arrayname[2]('value')...
         DELETE=name name...
                arrayname[n] arrayname...
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic "Command Order" on page 36 for more information.

Release History

Release 14.0

- Command introduced.

Example

```
DATAFILE ATTRIBUTE
  ATTRIBUTE=OriginalVersion ('1').
```

Overview

DATAFILE ATTRIBUTE provides the ability for you to define your own data file attributes and assign attribute values to the active dataset.

- User-defined data file attributes are saved with the data file in the data dictionary.
- The DATAFILE ATTRIBUTE command takes effect immediately, updating the data dictionary without requiring a data pass.
- You can display a list of data file and variable attributes with DISPLAY ATTRIBUTES. See the topic "DISPLAY" on page 599 for more information.

Basic Specification

The basic specification is:

- ATTRIBUTE keyword followed by an equals sign (=) and one or more attribute names that follow variable naming rules, with each attribute name followed by a quoted attribute value, enclosed in parentheses.

or

- DELETE keyword followed by an equals sign (=) and a list of defined attribute names or attribute arrays.

Syntax Rules

- The keywords ATTRIBUTE and DELETE must each be followed by an equals sign (=).
- Each ATTRIBUTE keyword must be followed by a name that follows variable naming rules and a single, quoted attribute value, enclosed in parentheses. See the topic "Variable Names" on page 42 for more information.
- Attribute names that begin with @ are not displayed by DISPLAY DICTIONARY or DISPLAY ATTRIBUTES. They can only be displayed with DISPLAY @ATTRIBUTES.
- Attribute names that begin with a dollar sign ($) are reserved for internal use.
- All attribute values must be quoted (single or double quotes), even if the values are numbers.
- Attribute values can be up to 32,767 bytes in length.

Example

```
DATAFILE ATTRIBUTE
  ATTRIBUTE=OriginalVersion ('1')
```

```
            CreationDate('10/28/2004')
            RevisionDate('10/29/2004').
```

Attribute Arrays

If you append an integer enclosed in square brackets to the end of an attribute name, the attribute is interpreted as an array of attributes. For example:

```
DATAFILE ATTRIBUTE
  ATTRIBUTE=FileAttribute[99]('not quite 100').
```

will create 99 attributes--*FileAttribute[01]* through *FileAttribute[99]*--and will assign the value "not quite 100" to the last one.

- Array subscripts (the value enclosed in square brackets) must be integers greater than 0. (Array subscript numbering starts with 1, not 0.)
- If the root name of an attribute array is the same as an existing attribute name, the attribute array replaces the existing attribute. If no value is assigned to the first element in the array (subscript [1]), the original attribute value is used for that element value.

With the DELETE keyword, the following rules apply to attribute arrays:

- If you specify DELETE followed by an array root name and no value in square brackets, all attributes in the array are deleted.
- If you specify DELETE with an array name followed by an integer value in square brackets, the specified array element is deleted and the integer values for all subsequent attributes in the array (in numeric order) are changed to reflect the new order of array elements.

Example

```
DATAFILE ATTRIBUTE
  ATTRIBUTE=RevisionDate('10/29/2004').
 DATAFILE ATTRIBUTE
   ATTRIBUTE=RevisionDate[2] ('10/21/2005').
DATAFILE ATTRIBUTE
  DELETE=RevisionDate[1].
DATAFILE ATTRIBUTE
  DELETE=RevisionDate.
```

- The first DATAFILE ATTRIBUTE command creates the attribute *RevisionDate* with a value of 10/29/2004.
- The second DATAFILE ATTRIBUTE command creates an array attribute named *RevisionDate*, which replaces the original attribute of the same name. Two array elements are created: *RevisionDate[1]* retains the original value of *RevisionDate*, and *RevisionDate[2]* has a value of 10/21/2005.
- The third DATAFILE ATTRIBUTE command deletes *RevisionDate[1]*, and the array element formerly known as *RevisionDate[2]* becomes the new *RevisionDate[1]* (with a value of 10/21/2005).
- The last DATAFILE ATTRIBUTE command deletes all attributes in the *RevisionDate* array, since it specifies the array root name without an integer value in brackets.

DATASET ACTIVATE

```
DATASET ACTIVATE name
        [WINDOW={ASIS }]
                {FRONT}
```

Release History

Release 14.0

- Command introduced.

Example

```
GET FILE='/data/mydata.sav'.
DATASET NAME file1.
COMPUTE AvgIncome=income/famsize.
GET DATA /TYPE=XLS
  /FILE='/data/exceldata.xls'.
COMPUTE TotIncome=SUM(income1, income2, income3).
DATASET NAME file2.
DATASET ACTIVATE file1.
```

Overview

The DATASET commands (DATASET NAME, DATASET ACTIVATE, DATASET DECLARE, DATASET COPY, DATASET CLOSE) provide the ability to have multiple data sources open at the same time and control which open data source is active at any point in the session. Using defined dataset names, you can then:

- Merge data (for example, MATCH FILES, ADD FILES, UPDATE) from multiple different source types (for example, text data, database, spreadsheet) without saving each one as an external IBM SPSS Statistics data file first.
- Create new datasets that are subsets of open data sources (for example, males in one subset, females in another, people under a certain age in another, or original data in one set and transformed/computed values in another subset).
- Copy and paste variables, cases, and/or variable properties between two or more open data sources in the Data Editor.

The DATASET ACTIVATE command makes the named dataset the active dataset in the session.

- If the previous active dataset does not have a defined dataset name, it is no longer available in the session.
- If the previous active dataset has a defined dataset name, it remains available for subsequent use in its current state.
- If the named dataset does not exist, an error occurs, and the command is not executed.
- DATASET ACTIVATE cannot be used within transformation structures such as DO IF, DO REPEAT, or LOOP.

Basic Specification

The basic specification for DATASET ACTIVATE is the command name followed by a name of a previously defined dataset. See the topic “DATASET NAME” on page 543 for more information.

WINDOW keyword

The WINDOW keyword controls the state of the Data Editor window associated with the dataset.

ASIS. The Data Editor window containing the dataset is not affected. This is the default.

FRONT. The Data Editor window containing the dataset is brought to the front and the dataset becomes the active dataset for dialog boxes.

Operations

- Commands operate on the active dataset. The **active** dataset is the data source most recently opened (for example, by commands such as GET DATA, GET SAS, GET STATA, GET TRANSLATE) or most recently activated by a DATASET ACTIVATE command.

  Note: The active dataset can also be changed by clicking anywhere in the Data Editor window of an open data source or selecting a dataset from the list of available datasets in a syntax window toolbar.
- Variables from one dataset are not available when another dataset is the active dataset.
- Transformations to the active dataset--before or after defining a dataset name--are preserved with the named dataset during the session, and any pending transformations to the active dataset are automatically executed whenever a different data source becomes the active dataset.
- Dataset names can be used in most commands that can contain references to IBM SPSS Statistics data files.
- For commands that can create a new dataset or overwrite an existing dataset, you cannot use the dataset name of the active dataset to overwrite the active dataset. For example, if the active dataset is *mydata*, a command with the subcommand /OUTFILE=mydata will result in an error. To overwrite a named active dataset, use an asterisk instead of the dataset name, as in: /OUTFILE=*.
- Wherever a dataset name, file handle (defined by the FILE HANDLE command), or filename can be used to refer to IBM SPSS Statistics data files, defined dataset names take precedence over file handles, which take precedence over filenames. For example, if *file1* exists as both a dataset name and a file handle, FILE=file1 in the MATCH FILES command will be interpreted as referring to the dataset named *file1*, not the file handle.

Example

```
GET FILE='/data/mydata.sav'.
DATASET NAME file1.
COMPUTE AvgIncome=income/famsize.
GET DATA /TYPE=XLS
  /FILE='/data/exceldata.xls'.
COMPUTE TotIncome=SUM(income1, income2, income3).
DATASET NAME file2.
DATASET ACTIVATE file1.
```

- Reading a new data source automatically changes the active dataset; so the GET DATA command changes the active dataset to the data read from the Excel worksheet.
- Since the previous active dataset has a defined dataset name associated with it, it is preserved in its current state for subsequent use in the session. The "current state" includes the new variable *AvgIncome* generated by the COMPUTE command, since pending transformations are automatically executed before the Excel worksheet become the active dataset.
- When the dataset *file1* is activated again, any pending transformations associated with dataset *file2* are automatically executed; so the new variable *TotIncome* is preserved with the dataset.

DATASET CLOSE

```
DATASET CLOSE {name}
              {*   }
              {ALL }
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 14.0

- Command introduced.

Example

```
DATASET CLOSE file1.
```

Overview

The DATASET commands (DATASET NAME, DATASET ACTIVATE, DATASET DECLARE, DATASET COPY, DATASET CLOSE) provide the ability to have multiple data sources open at the same time and control which open data source is active at any point in the session. Using defined dataset names, you can then:

- Merge data (for example, MATCH FILES, ADD FILES, UPDATE) from multiple different source types (for example, text data, database, spreadsheet) without saving each one as an external IBM SPSS Statistics data file first.
- Create new datasets that are subsets of open data sources (for example, males in one subset, females in another, people under a certain age in another, or original data in one set and transformed/computed values in another subset).
- Copy and paste variables, cases, and/or variable properties between two or more open data sources in the Data Editor.

The DATASET CLOSE command closes the named dataset.

- If the dataset name specified is not the active dataset, that dataset is closed and no longer available in the session.
- If the dataset name specified is the active dataset or if an asterisk (*) is specified and the active dataset has a name, the association with that name is broken. The active dataset remains active but has no name.
- If ALL is specified, all associations with datasets are broken. All the datasets except the active dataset and their data windows are closed and no longer available in the session. This includes datasets that have been declared (DATASET DECLARE) but have not been used yet. The active dataset remains active but has no name.

Basic Specification

The only specification for DATASET CLOSE is the command name followed by a dataset name, an asterisk (*), or the keyword ALL.

DATASET COPY

```
DATASET COPY name
        [WINDOW={MINIMIZED}]
                {HIDDEN   }
                {FRONT    }
```

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 14.0

- Command introduced.

Example

```
DATASET NAME original.
DATASET COPY males.
DATASET ACTIVATE males.
SELECT IF gender=0.
DATASET ACTIVATE original.
DATASET COPY females.
DATASET ACTIVATE females.
SELECT IF gender=1.
```

Overview

The DATASET commands (DATASET NAME, DATASET ACTIVATE, DATASET DECLARE, DATASET COPY, DATASET CLOSE) provide the ability to have multiple data sources open at the same time and control which open data source is active at any point in the session. Using defined dataset names, you can then:

- Merge data (for example, MATCH FILES, ADD FILES, UPDATE) from multiple different source types (for example, text data, database, spreadsheet) without saving each one as an external IBM SPSS Statistics data file first.
- Create new datasets that are subsets of open data sources (for example, males in one subset, females in another, people under a certain age in another, or original data in one set and transformed/computed values in another subset).
- Copy and paste variables, cases, and/or variable properties between two or more open data sources in the Data Editor.

The DATASET COPY command creates a new dataset that captures the current state of the active dataset. This is particularly useful for creating multiple subsets of data from the same original data source.

- If the active dataset has a defined dataset name, its name remains associated with subsequent changes.
- If this command occurs when there are transformations pending, those transformations are executed, as if EXECUTE had been run prior to making the copy; so the transformations appear in both the original and the copy. The command is illegal where EXECUTE would be illegal. If no transformations are pending, the data are not passed.
- If the specified dataset name is already associated with a dataset, a warning is issued, the old dataset is destroyed, and the specified name becomes associated with the current state of the active dataset.
- If the specified name is associated with the active dataset, it becomes associated with the current state and the active dataset becomes unnamed.

Basic Specification

The basic specification for DATASET COPY is the command name followed by a new dataset name that conforms to variable naming rules. See the topic “Variable Names” on page 42 for more information.

WINDOW Keyword

The WINDOW keyword controls the state of the Data Editor window associated with the dataset.

MINIMIZED. The Data Editor window associated with the new dataset is opened in a minimized state. This is the default.

HIDDEN. The Data Editor window associated with the new dataset is not displayed.

FRONT . The Data Editor window containing the dataset is brought to the front and the dataset becomes the active dataset for dialog boxes.

Operations

- Commands operate on the active dataset. The **active** dataset is the data source most recently opened (for example, by commands such as GET DATA, GET SAS, GET STATA, GET TRANSLATE) or most recently activated by a DATASET ACTIVATE command.

  Note: The active dataset can also be changed by clicking anywhere in the Data Editor window of an open data source or selecting a dataset from the list of available datasets in a syntax window toolbar.
- Variables from one dataset are not available when another dataset is the active dataset.
- Transformations to the active dataset--before or after defining a dataset name--are preserved with the named dataset during the session, and any pending transformations to the active dataset are automatically executed whenever a different data source becomes the active dataset.
- Dataset names can be used in most commands that can contain references to IBM SPSS Statistics data files.
- For commands that can create a new dataset or overwrite an existing dataset, you cannot use the dataset name of the active dataset to overwrite the active dataset. For example, if the active dataset is *mydata*, a command with the subcommand /OUTFILE=mydata will result in an error. To overwrite a named active dataset, use an asterisk instead of the dataset name, as in: /OUTFILE=*.
- Wherever a dataset name, file handle (defined by the FILE HANDLE command), or filename can be used to refer to IBM SPSS Statistics data files, defined dataset names take precedence over file handles, which take precedence over filenames. For example, if *file1* exists as both a dataset name and a file handle, FILE=file1 in the MATCH FILES command will be interpreted as referring to the dataset named *file1*, not the file handle.

Limitations

Because each window requires a minimum amount of memory, there is a limit to the number of windows, IBM SPSS Statistics or otherwise, that can be concurrently open on a given system. The particular number depends on the specifications of your system and may be independent of total memory due to OS constraints.

Example

```
DATASET NAME original.
DATASET COPY males.
DATASET ACTIVATE males.
SELECT IF gender=0.
DATASET ACTIVATE original.
DATASET COPY females.
DATASET ACTIVATE females.
SELECT IF gender=1.
```

- The first DATASET COPY command creates a new dataset, *males*, that represents the state of the active dataset at the time it was copied.
- The *males* dataset is activated and a subset of males is created.
- The original dataset is activated, restoring the cases deleted from the *males* subset.
- The second DATASET COPY command creates a second copy of the original dataset with the name *females*, which is then activated and a subset of females is created.
- Three different versions of the initial data file are now available in the session: the original version, a version containing only data for males, and a version containing only data for females.

DATASET DECLARE

```
DATASET DECLARE name
        [WINDOW={MINIMIZED}]
                {HIDDEN   }
                {FRONT    }
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 14.0

- Command introduced.

Example

```
DATASET DECLARE corrmatrix.
REGRESSION
  /DEPENDENT=var1
  /METHOD=ENTER var2 to var10
  /OUTFILE=CORB(corrmatrix).
DATASET ACTIVATE corrmatrix.
```

Overview

The DATASET commands (DATASET NAME, DATASET ACTIVATE, DATASET DECLARE, DATASET COPY, DATASET CLOSE) provide the ability to have multiple data sources open at the same time and control which open data source is active at any point in the session. Using defined dataset names, you can then:

- Merge data (for example, MATCH FILES, ADD FILES, UPDATE) from multiple different source types (for example, text data, database, spreadsheet) without saving each one as an external IBM SPSS Statistics data file first.
- Create new datasets that are subsets of open data sources (for example, males in one subset, females in another, people under a certain age in another, or original data in one set and transformed/computed values in another subset).
- Copy and paste variables, cases, and/or variable properties between two or more open data sources in the Data Editor.

The DATASET DECLARE command creates a new dataset name that is not associated with any open dataset. It can become associated with a dataset if it is used in a command that writes IBM SPSS Statistics data files. This is particularly useful if you need to create temporary IBM SPSS Statistics data files as an intermediate step in a program.

Basic Specification

The basic specification for DATASET DECLARE is the command name followed by a new dataset name that conforms to variable naming rules. See the topic “Variable Names” on page 42 for more information.

WINDOW Keyword

The WINDOW keyword controls the state of the Data Editor window associated with the dataset.

MINIMIZED. The Data Editor window associated with the new dataset is opened in a minimized state. This is the default.

HIDDEN. The Data Editor window associated with the new dataset is not displayed.

FRONT. The Data Editor window containing the dataset is brought to the front and the dataset becomes the active dataset for dialog boxes.

Example

```
DATASET DECLARE corrmatrix.
REGRESSION
  /DEPENDENT=var1
  /METHOD=ENTER= var2 to var10
  /OUTFILE=CORB(corrmatrix).
```

- The DATASET DECLARE command creates a new dataset name, *corrmatrix*, that is initially not assigned to any data source.
- The REGRESSION command writes a correlation matrix to a data file.
- Instead of specifying an external data file, the OUTFILE subcommand specifies the dataset name *corrmatrix*, which is now available for subsequent use in the session. If not explicitly saved (for example, with the SAVE command), this dataset will be automatically deleted at the end of the session.

DATASET DISPLAY

```
DATASET DISPLAY
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 14.0

- Command introduced.

Release 22.0

- Output expanded to include creation timestamp and associated file.

Example

```
DATASET DISPLAY.
```

Overview

The DATASET commands (DATASET NAME, DATASET ACTIVATE, DATASET DECLARE, DATASET COPY, DATASET CLOSE) provide the ability to have multiple data sources open at the same time and control which open data source is active at any point in the session. Using defined dataset names, you can then:

- Merge data (for example, MATCH FILES, ADD FILES, UPDATE) from multiple different source types (for example, text data, database, spreadsheet) without saving each one as an external IBM SPSS Statistics data file first.
- Create new datasets that are subsets of open data sources (for example, males in one subset, females in another, people under a certain age in another, or original data in one set and transformed/computed values in another subset).
- Copy and paste variables, cases, and/or variable properties between two or more open data sources in the Data Editor.

The DATASET DISPLAY command displays a list of currently available datasets. The only specification is the command name DATASET DISPLAY.

DATASET NAME

```
DATASET NAME name
        [WINDOW={ASIS }]
                {FRONT}
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 14.0

- Command introduced.

Example

```
GET FILE='/data/mydata.sav'.
DATASET NAME file1.
SORT CASES BY ID.
GET FILE '/data/moredata.sav'
SORT CASES BY ID.
DATASET NAME file2.
GET DATA  /TYPE=XLS
  /FILE='/data/exceldata.xls'.
SORT CASES BY ID.
MATCH FILES FILE=*
  /FILE=file1
  /FILE=file2
  /BY ID.
SAVE OUTFILE='/data/mergedata.sav'.
```

Overview

The DATASET commands (DATASET NAME, DATASET ACTIVATE, DATASET DECLARE, DATASET COPY, DATASET CLOSE) provide the ability to have multiple data sources open at the same time and control which open data source is active at any point in the session. Using defined dataset names, you can then:

- Merge data (for example, MATCH FILES, ADD FILES, UPDATE) from multiple different source types (for example, text data, database, spreadsheet) without saving each one as an external IBM SPSS Statistics data file first.
- Create new datasets that are subsets of open data sources (for example, males in one subset, females in another, people under a certain age in another, or original data in one set and transformed/computed values in another subset).
- Copy and paste variables, cases, and/or variable properties between two or more open data sources in the Data Editor.

The DATASET NAME command:

- Assigns a unique name to the active dataset, which can be used in subsequent file access commands and subsequent DATASET commands.
- Makes the current data file available even after other data sources have been opened/activated.

The following general rules apply:

- If the active dataset already has a defined dataset name, the existing association is broken, and the new name is associated with the active file.
- If the name is already associated with another dataset, that association is broken, and the new association is created. The dataset previously associated with that name is closed and is no longer available.

Basic Specification

The basic specification for DATASET NAME is the command name followed by a name that conforms to variable naming rules. See the topic “Variable Names” on page 42 for more information.

WINDOW Keyword

The WINDOW keyword controls the state of the Data Editor window associated with the dataset.

ASIS. The Data Editor window containing the dataset is not affected. This is the default.

FRONT. The Data Editor window containing the dataset is brought to the front and the dataset becomes the active dataset for dialog boxes.

Operations

- Commands operate on the active dataset. The **active** dataset is the data source most recently opened (for example, by commands such as GET DATA, GET SAS, GET STATA, GET TRANSLATE) or most recently activated by a DATASET ACTIVATE command.

  Note: The active dataset can also be changed by clicking anywhere in the Data Editor window of an open data source or selecting a dataset from the list of available datasets in a syntax window toolbar.
- Variables from one dataset are not available when another dataset is the active dataset.
- Transformations to the active dataset--before or after defining a dataset name--are preserved with the named dataset during the session, and any pending transformations to the active dataset are automatically executed whenever a different data source becomes the active dataset.
- Dataset names can be used in most commands that can contain references to IBM SPSS Statistics data files.
- For commands that can create a new dataset or overwrite an existing dataset, you cannot use the dataset name of the active dataset to overwrite the active dataset. For example, if the active dataset is *mydata*, a command with the subcommand /OUTFILE=mydata will result in an error. To overwrite a named active dataset, use an asterisk instead of the dataset name, as in: /OUTFILE=*.
- Wherever a dataset name, file handle (defined by the FILE HANDLE command), or filename can be used to refer to IBM SPSS Statistics data files, defined dataset names take precedence over file handles, which take precedence over filenames. For example, if *file1* exists as both a dataset name and a file handle, FILE=file1 in the MATCH FILES command will be interpreted as referring to the dataset named *file1*, not the file handle.

Example

```
GET FILE='/examples/data/mydata.sav'.
SORT CASES BY ID.
DATASET NAME mydata.
GET DATA /TYPE=XLS
  /FILE='/examples/data/excelfile.xls'.
SORT CASES BY ID.
DATASET NAME excelfile.
GET DATA /TYPE=ODBC /CONNECT=
 'DSN=MS Access Database;DBQ=/examples/data/dm_demo.mdb;'+
 'DriverId=25;FIL=MS Access;MaxBufferSize=2048;PageTimeout=5;'
 /SQL='SELECT * FROM main'.
SORT CASES BY ID.
MATCH FILES
 /FILE='mydata'
 /FILE='excelfile'
 /FILE=*
 /BY ID.
```

- A data file in IBM SPSS Statistics format and assigned the dataset name *mydata*. Since it has been assigned a dataset name, it remains available for subsequent use even after other data sources have been opened.
- An Excel file is then read and assigned the dataset name *excelfile*. Like the IBM SPSS Statistics data file, since it has been assigned a dataset name, it remains available after other data sources have been opened.
- Then a table from a database is read. Since it is the most recently opened or activated dataset, it is the active dataset.
- The three datasets are then merged together with MATCH FILES command, using the dataset names on the FILE subcommands instead of file names.
- An asterisk (*) is used to specify the active dataset, which is the database table in this example.
- The files are merged together based on the value of the key variable *ID*, specified on the BY subcommand.

- Since all the files being merged need to be sorted in the same order of the key variable(s), SORT CASES is performed on each dataset.

DATE

```
DATE  keyword [starting value [periodicity]]
      [keyword [starting value [periodicity]]]
      [BY increment]
```

Table 36. Keywords for long time periods

Keyword	Abbreviation	Default starting value	Default periodicity
YEAR	Y	1	none
QUARTER	Q	1	4
MONTH	M	1	12

Table 37. Keywords for short time periods

Keyword	Abbreviation	Default starting value	Default periodicity
WEEK	W	1	none
DAY	D	1	7
HOUR	H	0	24
MINUTE	MI	0	60
SECOND	S	0	60

Table 38. Keywords for any time periods

Keyword	Abbreviation	Default starting value	Default periodicity
CYCLE	C	1	none
OBS	O	none	none

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
DATE Y 1960 M.
```

Overview

DATE generates date identification variables. You can use these variables to label plots and other output, establish periodicity, and distinguish between historical, validation, and forecasting periods.

Options

You can specify the starting value and periodicity. You can also specify an increment for the lowest-order keyword specified.

Basic Specification

The basic specification on DATE is a single keyword.

- For each keyword specified, DATE creates a numeric variable whose name is the keyword with an underscore as a suffix. Values for this variable are assigned to observations sequentially, beginning with

the specified starting value. DATE also creates a string variable named *DATE_,* which combines the information from the numeric date variables and is used for labeling.

- If no starting value is specified, either the default is used or the value is inferred from the starting value of another DATE keyword.
- All variables created by DATE are automatically assigned variable labels that describe periodicity and associated formats. DATE produces a list of the names of the variables it creates and their variable labels.

Subcommand Order

- Keywords can be specified in any order.

Operations

- DATE creates a numeric variable for every keyword specified, plus a string variable *DATE_*, which combines information from all the specified keywords.
- DATE automatically creates variable labels for each keyword specified indicating the variable name and its periodicity. For the *DATE_* variable, the label indicates the variable name and format.
- If the highest-order DATE variable specified has a periodicity, the *CYCLE_* variable will automatically be created. *CYCLE_* cannot have a periodicity. See the topic "Example 3 " on page 550 for more information.
- Default periodicities are not used for the highest-order keyword specified. The exception is QUARTER, which will always have a default periodicity.
- The periodicity of the lowest-order variable is the default periodicity used by the procedures when periodicity is not defined either within the procedure or by the TSET command.
- The keyword name with an underscore is always used as the new variable name, even if keyword abbreviations are used in the specifications.
- Each time the DATE command is used, any DATE variables already in the active dataset are deleted.
- The DATE command invalidates any previous USE and PREDICT commands specified. The USE and PREDICT periods must be respecified after DATE.

Limitations

- There is no limit on the number of keywords on the DATE command. However, keywords that describe long time periods (YEAR, QUARTER, MONTH) cannot be used on the same command with keywords that describe short time periods (WEEK, DAY, HOUR, MINUTE, SECOND).
- User-defined variable names must not conflict with DATE variable names.

Syntax Rules

- You can specify more than one keyword per command.
- If a keyword is specified more than once, only the last one is executed.
- Keywords that describe long time periods (YEAR, QUARTER, MONTH) cannot be used on the same command with keywords that describe short time periods (WEEK, DAY, HOUR, MINUTE, SECOND).
- Keywords CYCLE and OBS can be used with any other keyword.
- The lowest-order keyword specified should correspond to the level at which observations occur. For example, if observations are daily, the lowest-order keyword should be DAY.
- Keywords (except MINUTE) can be abbreviated down to the first character. MINUTE must have at least two characters (MI) to distinguish it from keyword MONTH.
- Keywords and additional specifications are separated by commas or spaces.

Starting Value and Periodicity

- A starting value and periodicity can be entered for any keyword except CYCLE. CYCLE can have only a starting value.
- Starting value and periodicity *must* be specified for keyword OBS.
- The starting value is specified first, followed by the periodicity, if any.
- You cannot specify a periodicity without first specifying a starting value.
- Starting values for HOUR, MINUTE, and SECOND can range from 0 to the periodicity minus 1 (for example, 0 to 59). For all other keywords, the range is 1 to the periodicity.
- If both MONTH and QUARTER are specified, DATE can infer the starting value of one from the other. See the topic "Example 5 " on page 551 for more information.
- Specifying conflicting starting values for MONTH and QUARTER, such as `Q 1 M 4`, results in an error.
- For keyword YEAR, the starting value can be specified as the last two digits (93) instead of the whole year (1993) when the series and any forecasting are all within the same century. The same format (2 digits or 4 digits) must be used in all other commands that use year values.
- If you specify keywords that describe short time periods and skip over a level of measurement (for example, if you specify HOUR and SECOND but not MINUTE), you must specify the starting value and periodicity of the keyword after the skipped keywords. Otherwise, inappropriate periodicities will be generated. See the topic "Example 7 " on page 552 for more information.

BY Keyword

- Keyword BY and a positive integer can be specified after the lowest-order keyword on the command to indicate an increment value. This value indicates how much to increment values of the lowest-order date variable as they are assigned to observations. See the topic "Example 4 " on page 551 for more information.
- The increment value must divide evenly into the periodicity of the lowest-order DATE variable specified.

Example 1

```
DATE Y 1960 M.
```

- This command generates variables *DATE_*, *YEAR_*, and *MONTH_*.
- *YEAR_* has a starting value of 1960. *MONTH_* starts at the default value of 1.
- By default, *YEAR_* has no periodicity, and *MONTH_* has a periodicity of 12.

DATE reports the following:

```
  Name        Label

  YEAR_       YEAR, not periodic
  MONTH_      MONTH, period 12
  DATE_       DATE.  FORMAT:  "MMM YYYY"
```

The following is a partial listing of the new variables:

```
  YEAR_ MONTH_ DATE_

   1960    1   JAN 1960
   1960    2   FEB 1960
   1960    3   MAR 1960
   1960    4   APR 1960
    ...
   1960   10   OCT 1960
   1960   11   NOV 1960
   1960   12   DEC 1960
   1961    1   JAN 1961
   1961    2   FEB 1961
    ...
   1999    4   APR 1999
   1999    5   MAY 1999
   1999    6   JUN 1999
```

Example 2

```
DATE WEEK DAY 1 5 HOUR 1 8.
```

- This command creates four variables (*DATE_*, *WEEK_*, *DAY_*, and *HOUR_*) in a file where observations occur hourly in a 5-day, 40-hour week.
- For *WEEK,* the default starting value is 1 and the default periodicity is none.
- For *DAY_,* the starting value has to be specified, even though it is the same as the default, because a periodicity is specified. The periodicity of 5 means that observations are measured in a 5-day week.
- For *HOUR_,* a starting value of 1 is specified. The periodicity of 8 means that observations occur in an 8-hour day.

DATE reports the following:

```
  Name        Label

  WEEK_       WEEK, not periodic
  DAY_        DAY, period 5
  HOUR_       HOUR, period 24
  DATE_       DATE.  FORMAT:  "WWW D HH"
```

The following is a partial listing of the new variables:

```
  WEEK_ DAY_ HOUR_ DATE_

      1   1     1    1 1  1
      1   1     2    1 1  2
      1   1     3    1 1  3
      1   1     4    1 1  4
      1   1     5    1 1  5
      ...
      1   1    22    1 1 22
      1   1    23    1 1 23
      1   2     0    1 2  0
      1   2     1    1 2  1
      1   2     2    1 2  2
      ...
      4   5    16    4 5 16
      4   5    17    4 5 17
      4   5    18    4 5 18
```

Example 3

```
DATE DAY 1 5 HOUR 3 8.
```

- This command creates four variables (*DATE_*, *CYCLE_*, *DAY_*, and *HOUR_*) in a file where observations occur hourly.
- For *HOUR_,* the starting value is 3 and the periodicity is 8.
- For *DAY_,* the starting value is 1 and the periodicity is 5. Since *DAY_* is the highest-order variable and it has a periodicity assigned, variable *CYCLE_* is automatically created.

DATE reports the following:

```
  Name        Label

  CYCLE_      CYCLE, not periodic
  DAY_        DAY, period 5
  HOUR_       HOUR, period 8
  DATE_       DATE.  FORMAT:  "CCCC D H"
```

The following is a partial listing of the new variables:

```
  CYCLE_ DAY_ HOUR_ DATE_

       1   1    3      1 1 3
       1   1    4      1 1 4
       1   1    5      1 1 5
       1   1    6      1 1 6
       1   1    7      1 1 7
       1   2    0      1 2 0
       1   2    1      1 2 1
      ...
      12   4    6     12 4 6
      12   4    7     12 4 7
```

```
  12   5    0     12 5 0
  12   5    1     12 5 1
  12   5    2     12 5 2
  12   5    3     12 5 3
  12   5    4     12 5 4
```

Example 4

```
DATE DAY HOUR 1 24 BY 2.
```

- This command creates three variables (*DATE_*, *DAY_*, and *HOUR_*) in a file where observations occur every two hours in a 24-hour day.
- *DAY_* uses the default starting value of 1. It has no periodicity, since none is specified, and it is the highest-order keyword on the command.
- *HOUR_* starts with a value of 1 and has a periodicity of 24.
- Keyword BY specifies an increment of 2 to use in assigning hour values.

DATE reports the following:

```
Name        Label

DAY_        DAY, not periodic
HOUR_       HOUR, period 24 by 2
DATE_       DATE.  FORMAT:  "DDDD HH"
```

The following is a partial listing of the new variables:

```
DAY_ HOUR_ DATE_

  1    1     1  1
  1    3     1  3
  1    5     1  5
 ...
 39   17    39 17
 39   19    39 19
 39   21    39 21
 39   23    39 23
 40    1    40  1
 40    3    40  3
 40    5    40  5
 40    7    40  7
 40    9    40  9
 40   11    40 11
```

Example 5

```
DATE Y 1950 Q 2 M.
```

- This example creates four variables (*DATE_*, *YEAR_*, *QUARTER_*, and *MONTH_*) in a file where observations are quarterly, starting with April 1950.
- The starting value for *MONTH_* is inferred from *QUARTER_*.
- This specification is equivalent to `DATE Y 1950 Q M 4`. Here, the starting value for *QUARTER_* (2) would be inferred from MONTH.

DATE reports the following:

```
Name        Label

YEAR_       YEAR, not periodic
QUARTER_    QUARTER, period 4
MONTH_      MONTH, period 12
DATE_       DATE.  FORMAT:  "MMM YYYY"
```

The following is a partial listing of the new variables:

```
YEAR_ QUARTER_ MONTH_ DATE_

 1950     2       4   APR 1950
 1950     2       5   MAY 1950
 1950     2       6   JUN 1950
 1950     3       7   JUL 1950
 1950     3       8   AUG 1950
  ...
```

```
   1988     4      11   NOV 1988
   1988     4      12   DEC 1988
   1989     1       1   JAN 1989
   1989     1       2   FEB 1989
   1989     1       3   MAR 1989
   1989     2       4   APR 1989
   1989     2       5   MAY 1989
   1989     2       6   JUN 1989
   1989     3       7   JUL 1989
   1989     3       8   AUG 1989
   1989     3       9   SEP 1989
```

Example 6

```
DATE OBS 9 17.
```

- This command creates variables *DATE_*, *CYCLE_*, and *OBS_* and assigns values to observations sequentially, starting with value 9. The periodicity is 17.

DATE reports the following:

```
  Name        Label

  CYCLE_      CYCLE, not periodic
  OBS_        OBS, period 17
  DATE_       DATE.  FORMAT:  "CCCC OO"
```

The following is a partial listing of the new variables:

```
 CYCLE_ OBS_ DATE_

      1   9     1  9
      1  10     1 10
      1  11     1 11
      1  12     1 12
      1  13     1 13
      1  14     1 14
      1  15     1 15
      1  16     1 16
      1  17     1 17
      2   1     2  1
      2   2     2  2
     ...
     28  15    28 15
     28  16    28 16
     28  17    28 17
     29   1    29  1
     29   2    29  2
     29   3    29  3
     29   4    29  4
     29   5    29  5
     29   6    29  6
```

Example 7

```
DATE W H 1 168
```

- This example creates three variables (*DATE_*, *WEEK_*, and *HOUR_*) in a file where observations occur hourly.
- Since the DAY keyword is not specified, a periodicity must be specified for HOUR. The value 168 indicates that there are 168 hours in a week.
- The starting value of HOUR is specified as 1.

DATE reports the following:

```
  Name        Label

  WEEK_       WEEK, not periodic
  HOUR_       HOUR, period 168
  DATE_       DATE.  FORMAT:  "WWWW HHH"
```

The following is a partial listing of the new variables:

```
   WEEK_ HOUR_ DATE_

      1    1     1   1
      1    2     1   2
      1    3     1   3
```

```
      1    4     1   4
      1    5     1   5
      1    6     1   6
    ...
      1  161     1 161
      1  162     1 162
      1  163     1 163
      1  164     1 164
      1  165     1 165
      1  166     1 166
      1  167     1 167
      2    0     2   0
      2    1     2   1
      2    2     2   2
      2    3     2   3
      2    4     2   4
      2    5     2   5
    ...
      3  131     3 131
      3  132     3 132
      3  133     3 133
      3  134     3 134
      3  135     3 135
      3  136     3 136
      3  137     3 137
      3  138     3 138
```

DEFINE-!ENDDEFINE

```
DEFINE macro name

 ([{argument name=} [!DEFAULT (string)] [!NOEXPAND] {!TOKENS  (n)             }]
   {!POSITIONAL=  }                                 {!CHAREND ('char')        }
                                                    {!ENCLOSE ('char', 'char')}
                                                    {!CMDEND                  }

 [/{argument name=} ...])
   {!POSITIONAL=  }

macro body

!ENDDEFINE
```

SET command controls :

```
PRESERVE
RESTORE
```

Assignment :

```
!LET var=expression
```

Conditional processing :

```
!IF (expression) !THEN statements
    [!ELSE statements]
!IFEND
```

Looping constructs :

```
!DO !varname=start !TO finish [!BY step]
   statements  [!BREAK]
!DOEND

!DO !varname !IN (list)
   statements  [!BREAK]
!DOEND
```

Macro directives :

```
!OFFEXPAND
!ONEXPAND
```

String manipulation functions :

```
!LENGTH (string)
!CONCAT (string1,string2)
!SUBSTR (string,from,[length])
!INDEX (string1,string2)
!HEAD (string)
!TAIL (string)
!QUOTE (string)
!UNQUOTE (string)
!UPCASE (string)
!BLANKS (n)
!NULL
!EVAL (string)
```

Release History

Release 14.0

- For syntax processed in interactive mode, modifications to the macro facility may affect macro calls occurring at the end of a command. See the topic “Overview ” on page 556 for more information.

Example

```
DEFINE sesvars ()
  age sex educ religion
!ENDDEFINE.
```

Overview

DEFINE–!ENDDEFINE defines a program macro, which can then be used within a command sequence. A macro can be useful in several different contexts. For example, it can be used to:

- Issue a series of the same or similar commands repeatedly, using looping constructs rather than redundant specifications.
- Specify a set of variables.
- Produce output from several program procedures with a single command.
- Create complex input programs, procedure specifications, or whole sessions that can then be executed.

A macro is defined by specifying any part of a valid command and giving it a macro name. This name is then specified in a macro call within a command sequence. When the program encounters the macro name, it expands the macro.

In the examples of macro definition throughout this reference, the macro name, body, and arguments are shown in lowercase for readability. Macro keywords, which are always preceded by an exclamation point (!), are shown in uppercase.

Options

Macro Arguments. You can declare and use arguments in the macro definition and then assign specific values to these arguments in the macro call. You can define defaults for the arguments and indicate whether an argument should be expanded when the macro is called. See the topic "Macro Arguments " on page 559 for more information.

Macro Directives. You can turn macro expansion on and off. See the topic "Macro Directives " on page 564 for more information.

String Manipulation Functions. You can process one or more character strings and produce either a new character string or a character representation of a numeric result. See the topic "String Manipulation Functions " on page 564 for more information.

Conditional Processing. You can build conditional and looping constructs. See the topic "Conditional Processing " on page 566 for more information.

Macro Variables. You can directly assign values to macro variables See the topic "Direct Assignment of Macro Variables " on page 569 for more information.

Basic Specification

All macros must start with DEFINE and end with !ENDDEFINE. These commands identify the beginning and end of a macro definition and are used to separate the macro definition from the rest of the command sequence.

- Immediately after DEFINE, specify the **macro name**. All macros must have a name. The name is used in the macro call to refer to the macro. Macro names can begin with an exclamation point (!), but other than this, follow the usual naming conventions. Starting a name with an ! ensures that it will not conflict with the other text or variables in the session.
- Immediately after the macro name, specify an optional **argument** definition in parentheses. This specification indicates the arguments that will be read when the macro is called. If you do not want to include arguments, specify just the parentheses; *the parentheses are required, whether or not they enclose an argument.*
- Next specify the body of the macro. The **macro body** can include commands, parts of commands, or macro statements (macro directives, string manipulation statements, and looping and conditional processing statements).
- At the end of the macro body, specify !ENDDEFINE.

To invoke the macro, issue a **macro call** in the command sequence. To call a macro, specify the macro name and any necessary arguments. If there are no arguments, only the macro name is required.

Operations

- When the program reads the macro definition, it translates into uppercase all text (except arguments) not enclosed in quotation marks. Arguments are read in upper- and lowercase.
- The macro facility does not build and execute commands; rather, it expands strings in a process called **macro expansion**. A macro call initiates macro expansion. After the strings are expanded, the commands (or parts of commands) that contain the expanded strings are executed as part of the command sequence.
- Any elements on the macro call that are not used in the macro expansion are read and combined with the expanded strings.
- The expanded strings and the remaining elements from the macro call, if any, must conform to the syntax rules for the program. If not, the program generates either a warning or an error message, depending on the nature of the syntax problem.

Syntax Rules

Just like other commands, expanded macros must adhere to the rules of the processing mode under which they are run. While it is desirable to create macro syntax that will run in both interactive and batch modes, this may sometimes add a layer of complexity that you may want to avoid. So we recommend that you write macro syntax that adheres to interactive syntax rules and structure your jobs to execute macro syntax under interactive syntax rules.

- DEFINE and !ENDDEFINE must be spelled in full. They cannot be abbreviated.
- The macro !ENDDEFINE statement should end with a period. A period as the last character on a line is interpreted as a command terminator in interactive mode.
- Other macro statements (for example, !IF, !LOOP, !LET) should not end with a period.
- Text within the body of the macro that represent commands that will be generated when the macro is expanded should include the period at the end of each command, and each command should start on a new line.

The macro body cannot contain the characters \ or ^ unless they are contained within a quoted string, as in "a valid backslash \ in a quoted string".

Example

```
DEFINE !macro1(type = !DEFAULT(1) !TOKENS(1)
                              /varlist=!CMDEND)
!IF (!type = 1)!THEN
frequencies variables=!varlist.
!ELSE
descriptives variables=!varlist.
!IFEND
!ENDDEFINE.
```

- The macro statements DEFINE, !IF, !ELSE, and !IFEND do not end with a period.
- !ENDDEFINE ends with a period.
- The FREQUENCIES and DESCRIPTIVES commands generated by the macro each start on a new line and end with a period.

To structure your command syntax jobs so that interactive processing rules are always used instead of batch processing rules:

- Use INSERT instead of INCLUDE to combine command files containing macros with other command files. See the topic “INSERT” on page 945 for more information.
- In Production Facility jobs, select **Interactive** for the Syntax Input Format. See the topic for more information.
- In the Batch Facility (available only with IBM SPSS Statistics Server), use the -i switch to use interactive processing rules.

Compatibility

Improvements to the macro facility may cause errors in jobs that previously ran without errors. Specifically, for syntax that is processed with interactive rules, if a macro call occurs at the end of a

command, and there is no command terminator (either a period or a blank line), the next command after the macro expansion will be interpreted as a continuation line instead of a new command, as in:

```
DEFINE !macro1()
var1 var2 var3
!ENDDEFINE.
FREQUENCIES VARIABLES = !macro1
DESCRIPTIVES VARIABLES = !macro1.
```

In interactive mode, the DESCRIPTIVES command will be interpreted as a continuation of the FREQUENCIES command, and neither command will run.

Limitations

- The BEGIN DATA–END DATA commands are not allowed within a macro.
- BEGIN PROGRAM-END PROGRAM commands are not supported within a macro.
- The DEFINE command is not allowed within a macro.

Examples

Example

```
* Macro without arguments: Specify a group of variables.

DEFINE sesvars ()
   age sex educ religion
!ENDDEFINE.

FREQUENCIES VARIABLES=sesvars.
```

- The macro name is sesvars. Because the parentheses are empty, sesvars has no arguments. The macro body defines four variables: *age*, *sex*, *educ*, and *religion*.
- The macro call is specified on FREQUENCIES. When the call is executed, sesvars is expanded into the variables *age*, *sex*, *educ*, and *religion*.
- After the macro expansion, FREQUENCIES is executed.

Example

```
* Macro without arguments: Repeat a sequence of commands.

DATA LIST FILE = MAC4D /GROUP 1   REACTIME 3-5 ACCURACY 7-9.
VALUE LABELS GROUP    1'normal'
                      2'learning disabled'.
* Macro definition.
DEFINE check ()
split file by group.
frequencies variables = reactime accuracy
  /histogram.
descriptives reactime accuracy.
list.
split file off.
regression variables = group reactime accuracy
  /dependent = accuracy
  /enter
  /scatterplot (reactime, accuracy).
!ENDDEFINE.

check.                        /* First call of defined macro check

COMPUTE REACTIME = SQRT (REACTIME).
COMPUTE ACCURACY = SQRT (ACCURACY).

check.                        /* Second call of defined macro check

COMPUTE REACTIME = lg10 (REACTIME * REACTIME).
COMPUTE ACCURACY = lg10 (ACCURACY * ACCURACY).

check.                        /* Third call of defined macro check
```

- The name of the macro is check. The empty parentheses indicate that there are no arguments to the macro.
- The macro definition (between DEFINE and !ENDDEFINE) contains the command sequence to be repeated: SPLIT FILE, FREQUENCIES, DESCRIPTIVES, LIST, SPLIT FILE, and REGRESSION.
- The macro is called three times. Every time check is encountered, it is replaced with the command sequence SPLIT FILE, FREQUENCIES, DESCRIPTIVES, LIST, SPLIT FILE OFF, and REGRESSION.

The command sequence using the macro facility is identical to the command sequence in which the specified commands are explicitly stated three separate times.

Example

```
* Macro with an argument.

DEFINE myfreq (vars = !CHAREND('/'))
frequencies variables = !vars
  /format = notable
  /statistics = default skewness kurtosis.
!ENDDEFINE.

myfreq vars = age sex educ religion /.
```

- The macro definition defines `vars` as the macro argument. In the macro call, four variables are specified as the argument to the macro `myfreq`. When the program expands the `myfreq` macro, it substitutes the argument, `age`, `sex`, `educ`, and `religion`, for `!vars` and executes the resulting commands.

Macro Arguments

The macro definition can include macro arguments, which can be assigned specific values in the macro call. There are two types of arguments: keyword and positional. Keyword arguments are assigned names in the macro definition; in the macro call, they are identified by name. Positional arguments are defined after the keyword `!POSITIONAL` in the macro definition; in the macro call, they are identified by their relative position within the macro definition.

- There is no limit to the number of arguments that can be specified in a macro.
- All arguments are specified in parentheses and must be separated by slashes.
- If both keyword and positional arguments are defined in the same definition, the positional arguments must be defined, used in the macro body, and invoked in the macro call before the keyword arguments.

Example

```
* A keyword argument.

DEFINE macname (arg1 = !TOKENS(1))
frequencies variables = !arg1.
!ENDDEFINE.

macname arg1 = V1.
```

- The macro definition defines `macname` as the macro name and `arg1` as the argument. The argument `arg1` has one token and can be assigned any value in the macro call.
- The macro call expands the `macname` macro. The argument is identified by its name, `arg1`, and is assigned the value V1. V1 is substituted wherever `!arg1` appears in the macro body. The macro body in this example is the FREQUENCIES command.

Example

```
* A positional argument.

DEFINE macname (!POSITIONAL !TOKENS(1)
               /!POSITIONAL !TOKENS(2))
frequencies variables = !1 !2.
!ENDDEFINE.

macname V1 V2 V3.
```

- The macro definition defines `macname` as the macro name with two positional arguments. The first argument has one token and the second argument has two tokens. The tokens can be assigned any values in the macro call.
- The macro call expands the `macname` macro. The arguments are identified by their positions. V1 is substituted for `!1` wherever `!1` appears in the macro body. V2 and V3 are substituted for `!2` wherever `!2` appears in the macro body. The macro body in this example is the FREQUENCIES command.

Keyword Arguments

Keyword arguments are called with user-defined keywords that can be specified in any order. In the macro body, the argument name is preceded by an exclamation point. On the macro call, the argument is specified without the exclamation point.

- Keyword argument definitions contain the argument name, an equals sign, and the !TOKENS, !ENCLOSE, !CHAREND, or !CMDEND keyword. See the topic "Assigning Tokens to Arguments " on page 561 for more information.
- Argument names are limited to 63 bytes and cannot match the character portion of a macro keyword, such as DEFINE, TOKENS, CHAREND, and so forth.
- The keyword !POSITIONAL cannot be used in keyword argument definitions.
- Keyword arguments do not have to be called in the order they were defined.

Example

```
DATA LIST FILE=MAC / V1 1-2 V2 4-5 V3 7-8.

* Macro definition.
DEFINE macdef2 (arg1 = !TOKENS(1)
               /arg2 = !TOKENS(1)
               /arg3 = !TOKENS(1))
frequencies  variables = !arg1 !arg2 !arg3.
!ENDDEFINE.

* Macro call.
macdef2 arg1=V1  arg2=V2  arg3=V3.
macdef2 arg3=V3  arg1=V1  arg2=V2.
```

- Three arguments are defined: arg1, arg2, and arg3, each with one token. In the first macro call, arg1 is assigned the value V1, arg2 is assigned the value V2, and arg3 is assigned the value V3. V1, V2, and V3 are then used as the variables in the FREQUENCIES command.
- The second macro call yields the same results as the first one. With keyword arguments, you do not need to call the arguments in the order in which they were defined.

Positional Arguments

Positional arguments must be defined in the order in which they will be specified on the macro call. In the macro body, the first positional argument is referred to by !1, the second positional argument defined is referred to by !2, and so on. Similarly, the value of the first argument in the macro call is assigned to !1, the value of the second argument is assigned to !2, and so on.

- Positional arguments can be collectively referred to in the macro body by specifying !*. The !* specification concatenates arguments, separating individual arguments with a blank.

Example

```
DATA LIST FILE='/data/mac.txt' / V1 1-2 V2 4-5 V3 7-8.

* Macro definition.
DEFINE macdef (!POS !TOKENS(1)
              /!POS !TOKENS(1)
              /!POS !TOKENS(1))
frequencies variables = !1 !2 !3.
!ENDDEFINE.

* Macro call.
macdef  V1    V2   V3.
macdef  V3    V1   V2.
```

- Three positional arguments with one token each are defined. The first positional argument is referred to by !1 on the FREQUENCIES command, the second by !2, and the third by !3.
- When the first call expands the macro, the first positional argument (!1) is assigned the value V1, the second positional argument (!2) is assigned the value V2, and the third positional argument (!3) is assigned the value V3.
- In the second call, the first positional argument is assigned the value V3, the second positional argument is assigned the value V1, and the third positional argument is assigned the value V2.

Example

```
DEFINE macdef (!POS !TOKENS(3))
frequencies variables = !1.
!ENDDEFINE.

macdef  V1    V2   V3.
```

- This example is the same as the previous one, except that it assigns three tokens to one argument instead of assigning one token to each of three arguments. The result is the same.

Example

```
DEFINE macdef (!POS !TOKENS(1)
               /!POS !TOKENS(1)
               /!POS !TOKENS(1)
frequencies variables = !*.
!ENDDEFINE.

macdef  V1    V2   V3.
```

- This is a third alternative for achieving the macro expansion shown in the previous two examples. It specifies three arguments but then joins them all together on one FREQUENCIES command using the symbol !*.

Assigning Tokens to Arguments

A **token** is a character or group of characters that has a predefined function in a specified context. The argument definition must include a keyword that indicates which tokens following the macro name are associated with each argument.

- Any program keyword, variable name, or delimiter (a slash, comma, and so on) is a valid token.
- The arguments for a given macro can use a combination of the token keywords.

!TOKENS (n). *Assign the next* n *tokens to the argument*. The value *n* can be any positive integer and must be enclosed in parentheses. !TOKENS allows you to specify exactly how many tokens are desired.

!CHAREND ('char'). *Assign all tokens up to the specified character to the argument.* The character must be a one-character string specified in apostrophes and enclosed in parentheses. !CHAREND specifies the character that ends the argument assignment. This is useful when the number of assigned tokens is arbitrary or not known in advance.

!ENCLOSE ('char','char'). *Assign all tokens between the indicated characters to the argument.* The starting and ending characters can be any one-character strings, and they do not need to be the same. The characters are each enclosed in apostrophes and separated by a comma. The entire specification is enclosed in parentheses. !ENCLOSE allows you to group multiple tokens within a specified pair of symbols. This is useful when the number of tokens to be assigned to an argument is indeterminate, or when the use of an ending character is not sufficient.

!CMDEND. *Assign to the argument all of the remaining text on the macro call, up to the start of the next command.* !CMDEND is useful for changing the defaults on an existing command. Since !CMDEND reads up to the next command, only the last argument on the argument list can be specified with !CMDEND. If !CMDEND is not the final argument, the arguments following !CMDEND are read as text.

Example

```
* Keyword !TOKENS.

DEFINE macname (!POSITIONAL !TOKENS (3)
frequencies variables = !1.
!ENDDEFINE.

macname ABC DEFG HI.
```

- The three tokens following macname (ABC, DEFG, and HI) are assigned to the positional argument !1, and FREQUENCIES is then executed.

Example

```
* Keyword !TOKENS.

* Macro definition.
```

```
DEFINE earnrep (varrep = !TOKENS (1))
sort cases by !varrep.
report variables = earnings
  /break = !varrep
  /summary = mean.
!ENDDEFINE.

* Call the macro three times.
earnrep varrep= SALESMAN.  /*First macro call
earnrep varrep = REGION.   /*Second macro call
earnrep varrep = MONTH.    /*Third macro call
```

- This macro runs a REPORT command three times, each time with a different break variable.
- The macro name is `earnrep`, and there is one keyword argument, `varrep`, which has one token.
- In the first macro call, the token SALESMAN is substituted for `!varrep` when the macro is expanded. REGION and MONTH are substituted for `!varrep` when the macro is expanded in the second and third calls.

Example

```
* Keyword !CHAREND'.

DEFINE macname (!POSITIONAL !CHAREND ('/')
               /!POSITIONAL !TOKENS(2))
frequencies variables = !1.
correlations variables= !2.
!ENDDEFINE.

macname A B C D / E F.
```

- When the macro is called, all tokens up to the slash (A, B, C, and D) are assigned to the positional argument `!1`. E and F are assigned to the positional argument `!2`.

Example

```
* Keyword !CHAREND.

DEFINE macname (!POSITIONAL !CHAREND ('/'))
frequencies variables = !1.
!ENDDEFINE.

macname A B C D / E F.
```

- Although E and F are not part of the positional argument and are not used in the macro expansion, the program still reads them as text and interprets them in relation to where the macro definition ends. In this example, macro definition ends after the expanded variable list (D). E and F are names of variables. Thus, E and F are added to the variable list and FREQUENCIES is executed with six variables: *A*, *B*, *C*, *D*, *E*, and *F*.

Example

```
* Keyword !ENCLOSE.

DEFINE macname (!POSITIONAL !ENCLOSE('(',')'))
frequencies variables = !1
  /statistics = default skewness.
!ENDDEFINE.

macname (A B C) D E.
```

- When the macro is called, the three tokens enclosed in parentheses--A, B, and C--are assigned to the positional argument `!1` in the macro body.
- After macro expansion is complete, the program reads the remaining characters on the macro call as text. In this instance, the macro definition ends with keyword SKEWNESS on the STATISTICS subcommand. Adding variable names to the STATISTICS subcommand is not valid syntax. The program generates a warning message but is still able to execute the frequencies command. Frequency tables and the specified statistics are generated for the variables *A*, *B*, and *C*.

Example

```
* Keyword !CMDEND'.

DEFINE macname (!POSITIONAL !TOKENS(2)
               /!POSITIONAL !CMDEND)
frequencies variables = !1.
correlations variables= !2.
!ENDDEFINE.

macname A B C D E.
```

- When the macro is called, the first two tokens following `macname` (A and B) are assigned to the positional argument `!1`. C, D, and E are assigned to the positional argument `!2`. Thus, the variables used for `FREQUENCIES` are *A* and *B*, and the variables used for `CORRELATION` are *C*, *D*, and *E*.

Example

```
* Incorrect order for !CMDEND.

DEFINE macname  (!POSITIONAL !CMDEND
                 /!POSITIONAL !tokens(2))
frequencies variables = !1.
correlations variables= !2.
!ENDDEFINE.

macname  A B C D E.
```

- When the macro is called, all five tokens, A, B, C, D, and E, are assigned to the first positional argument. No variables are included on the variable list for `CORRELATIONS`, causing the program to generate an error message. The previous example declares the arguments in the correct order.

Example

```
* Using !CMDEND.
SUBTITLE 'CHANGING DEFAULTS ON A COMMAND'.

DEFINE myfreq (!POSITIONAL !CMDEND)
frequencies !1
  /statistics=default skewness  /* Modify default statistics.
!ENDDEFINE.

myfreq VARIABLES = A B /HIST.
```

- The macro `myfreq` contains options for the `FREQUENCIES` command. When the macro is called, `myfreq` is expanded to perform a `FREQUENCIES` analysis on the variables *A* and *B*. The analysis produces default statistics and the skewness statistic, plus a histogram, as requested on the macro call.

Example

```
* Keyword arguments: Using a combination of token keywords.

DATA LIST FREE / A B C D E.
DEFINE macdef3 (arg1 = !TOKENS(1)
               /arg2 = !ENCLOSE ('(',')')
               /arg3 = !CHAREND('%'))
frequencies variables = !arg1  !arg2 !arg3.
!ENDDEFINE.
macdef arg1 = A  arg2=(B C)  arg3=D E %.
```

- Because `arg1` is defined with the `!TOKENS` keyword, the value for `arg1` is simply specified as A. The value for `arg2` is specified in parentheses, as indicated by `!ENCLOSE`. The value for `arg3` is followed by a percentage sign, as indicated by `!CHAREND`.

Defining Defaults

The optional `!DEFAULT` keyword in the macro definition establishes default settings for arguments.

!DEFAULT. *Default argument.* After `!DEFAULT`, specify the value you want to use as a default for that argument. A default can be specified for each argument.

Example

```
DEFINE macdef (arg1 = !DEFAULT (V1) !TOKENS(1)
              /arg2 = !TOKENS(1)
              /arg3 = !TOKENS(1))
frequencies variables = !arg1 !arg2 !arg3.
!ENDDEFINE.

macdef arg2=V2  arg3=V3.
```

- V1 is defined as the default value for argument `arg1`. Since `arg1` is not specified on the macro call, it is set to V1.
- If `!DEFAULT (V1)` were not specified, the value of `arg1` would be set to a null string.

Controlling Expansion

`!NOEXPAND` indicates that an argument should not be expanded when the macro is called.

!NOEXPAND. *Do not expand the specified argument.* !NOEXPAND applies to a single argument and is useful only when a macro calls another macro (embedded macros).

Macro Directives

!ONEXPAND and !OFFEXPAND determine whether macro expansion is on or off. !ONEXPAND activates macro expansion and !OFFEXPAND stops macro expansion. All symbols between !OFFEXPAND and !ONEXPAND in the macro definition will not be expanded when the macro is called.

!ONEXPAND. *Turn macro expansion on.*

!OFFEXPAND . *Turn macro expansion off.* !OFFEXPAND is effective only when SETMEXPAND is ON (the default).

Macro Expansion in Comments

When macro expansion is on, a macro is expanded when its name is specified in a comment line beginning with *. To use a macro name in a comment, specify the comment within slashes and asterisks (/*...*/) to avoid unwanted macro expansion. (See COMMENT.)

String Manipulation Functions

String manipulation functions process one or more character strings and produce either a new character string or a character representation of a numeric result.

- The result of any string manipulation function is treated as a character string.
- The arguments to string manipulation functions can be strings, variables, or even other macros. A macro argument or another function can be used in place of a string.
- The strings within string manipulation functions must be either single tokens, such as ABC, or delimited by apostrophes or quotation marks, as in 'A B C'.

Table 39. Expressions and results

Expression	Result
!UPCASE(abc)	ABC
!UPCASE('abc')	ABC
!UPCASE(a b c)	error
!UPCASE('a b c')	A B C
!UPCASE(a/b/c)	error
!UPCASE('a/b/c')	A/B/C
!UPCASE(!CONCAT(a,b,c))	ABC
!UPCASE(!CONCAT('a','b','c'))	ABC
!UPCASE(!CONCAT(a, b, c))	ABC
!UPCASE(!CONCAT('a ','b ','c '))	A B C
!UPCASE(!CONCAT('a,b,c'))	A,B,C
!QUOTE(abc)	'ABC'
!QUOTE('abc')	abc
!QUOTE('Bill"s')	'Bill"s'
!QUOTE("Bill's")	"Bill's"

Table 39. Expressions and results (continued)

Expression	Result
!QUOTE(Bill's)	error
!QUOTE(!UNQUOTE('Bill"s'))	'Bill"s'

!LENGTH (str). *Return the length of the specified string.* The result is a character representation of the string length. `!LENGTH(abcdef)` returns 6. If the string is specified with apostrophes around it, each apostrophe adds 1 to the length. `!LENGTH ('abcdef')` returns 8. If an argument is used in place of a string and it is set to null, this function will return 0.

!CONCAT (str1,str2 . . .). *Return a string that is the concatenation of the strings.* For example, `!CONCAT (abc,def)` returns abcdef.

!SUBSTR (str,from,[length]). *Return a substring of the specified string.* The substring starts at the *from* position and continues for the specified *length*. If the length is not specified, the substring ends at the end of the input string. For example, `!SUBSTR (abcdef, 3, 2)` returns cd.

!INDEX (haystack,needle). *Return the position of the first occurrence of the needle in the haystack*. If the needle is not found in the haystack, the function returns 0. `!INDEX (abcdef,def)` returns 4.

!HEAD (str). *Return the first token within a string.* The input string is not changed. `!HEAD ('a b c')` returns a.

!TAIL (str). *Return all tokens except the head token.* The input string is not changed. `!TAIL('a b c')` returns b c.

!QUOTE (str). *Put apostrophes around the argument.* `!QUOTE` replicates any embedded apostrophe. `!QUOTE(abc)` returns 'abc'. If `!1` equals Bill's, `!QUOTE(!1)` returns 'Bill''s'.

!UNQUOTE (str). *Remove quotation marks and apostrophes from the enclosed string.* If `!1` equals 'abc', `!UNQUOTE(!1)` is abc. Internal paired quotation marks are unpaired; if `!1` equals 'Bill"s', `!UNQUOTE(!1)` is Bill's. The specification `!UNQUOTE(!QUOTE(Bill))` returns Bill.

!UPCASE (str). *Convert all lowercase characters in the argument to uppercase.* `!UPCASE('abc def')` returns ABC DEF.

!BLANKS (n). *Generate a string containing the specified number of blanks.* The *n* specification must be a positive integer. `!BLANKS(5)` returns a string of five blank spaces. Unless the blanks are quoted, they cannot be processed, since the macro facility compresses blanks.

!NULL. *Generate a string of length 0.* This can help determine whether an argument was ever assigned a value, as in `!IF (!1 !EQ !NULL) !THEN. . .`

!EVAL (str). *Scan the argument for macro calls.* During macro definition, an argument to a function or an operand in an expression is not scanned for possible macro calls unless the `!EVAL` function is used. It returns a string that is the expansion of its argument. For example, if mac1 is a macro, then `!EVAL(mac1)` returns the expansion of mac1. If mac1 is not a macro, `!EVAL(mac1)` returns mac1.

SET Subcommands for Use with Macro

Four subcommands on the SET command were designed for use with the macro facility.

MPRINT. *Display a list of commands after macro expansion.* The specification on `MPRINT` is YES or NO (alias ON or OFF). By default, the output does not include a list of commands after macro expansion (`MPRINT NO`). The `MPRINT` subcommand on SET is independent of the `PRINTBACK` command.

MEXPAND. *Macro expansion.* The specification on `MEXPAND` is YES or NO (alias ON or OFF). By default, `MEXPAND` is on. `SET MEXPAND OFF` prevents macro expansion. Specifying `SET MEXPAND ON` reestablishes macro expansion.

MNEST. *Maximum nesting level for macros.* The default number of levels that can be nested is 50. The maximum number of levels depends on storage capacity.

MITERATE. *Maximum loop iterations permitted in macro expansions.* The default number of iterations is 1000.

Restoring SET Specifications

The PRESERVE and RESTORE commands bring more flexibility and control over SET. PRESERVE and RESTORE are available generally within the program but are especially useful with macros.

- The settings of all SET subcommands—those set explicitly and those set by default (except MEXPAND)—are saved with PRESERVE. PRESERVE has no further specifications.
- With RESTORE, all SET subcommands are changed to what they were when the PRESERVE command was executed. RESTORE has no further specifications.
- PRESERVE...RESTORE sequences can be nested up to five levels.

PRESERVE. *Store the SET specifications that are in effect at this point in the session.*

RESTORE. *Restore the SET specifications to what they were when PRESERVE was specified.*

Example

```
* Two nested levels of preserve and restore'.

DEFINE macdef ()
preserve.
set format F5.3.
descriptives v1 v2.
+ preserve.
set format F3.0 blanks=999.
descriptives v3 v4.
+ restore.
descriptives v5 v6.
restore.
!ENDDEFINE.
```

- The first PRESERVE command saves all of the current SET conditions. If none have been specified, the default settings are saved.
- Next, the format is set to F5.3 and descriptive statistics for *V1* and *V2* are obtained.
- The second PRESERVE command saves the F5.3 format setting and all other settings in effect.
- The second SET command changes the format to F3.0 and sets BLANKS to 999 (the default is SYSMIS). Descriptive statistics are then obtained for *V3* and *V4*.
- The first RESTORE command restores the format to F5.3 and BLANKS to the default, the setting in effect at the second PRESERVE. Descriptive statistics are then obtained for *V5* and *V6*.
- The last RESTORE restores the settings in effect when the first PRESERVE was specified.

Conditional Processing

The !IF construct specifies conditions for processing. The syntax is as follows:

```
!IF (expression) !THEN statements
                 [!ELSE statements]
!IFEND
```

- !IF, !THEN, and !IFEND are all required. !ELSE is optional.
- If the result of the expression is true, the statements following !THEN are executed. If the result of the expression is false and !ELSE is specified, the statements following !ELSE are executed. Otherwise, the program continues.
- Valid operators for the expressions include !EQ, !NE, !GT, !LT, !GE, !LE, !OR, !NOT, and !AND, or =, ~= (¬=), >, <, >=, <=, |, ~ (¬), and &.
- When a macro is expanded, conditional processing constructs are interpreted after arguments are substituted and functions are executed.

- !IF statements can be nested whenever necessary. Parentheses can be used to specify the order of evaluation. The default order is the same as for transformations: !NOT has precedence over !AND, which has precedence over !OR.

Example

```
DEFINE mymacro(type = !DEFAULT(1) !TOKENS(1))
!IF (!type = 1)!then
frequencies variables=varone.
!ELSE
descriptives variables=vartwo.
!IFEND
!ENDDEFINE.
```

Unquoted String Constants in Conditional !IF Statements

Prior to version 12.0, under certain circumstances unquoted string constants in conditional !IF statements were not case sensitive. Starting with version 12.0, unquoted string constants are case sensitive. For backward compatibility, always use quoted string constants.

Example

```
DEFINE noquote(type = !DEFAULT(a) !TOKENS(1))
!IF (!type = A)!THEN
frequencies variables=varone.
!ELSE
descriptives variables=vartwo.
!IFEND
!ENDDEFINE.
DEFINE yesquote(type = !DEFAULT('a') !TOKENS(1)).
!IF (!type = 'A')!THEN
FREQUENCIES varone.
!ELSE
DESCRIPTIVES vartwo.
!IFEND.
!ENDDEFINE.
```

- In the first macro, !IF(!type = A) is evaluated as false if the value of the unquoted string constant is lowercase 'a'—and is therefore evaluated as false in this example.
- Prior to version 12.0, !IF (!type = A) was evaluated as true if the value of the unquoted string constant was lowercase 'a' or uppercase 'A'—and was therefore evaluated as true in this example.
- In the second macro, !IF (!type = 'A') is always evaluated as false if the value of the string constant is lowercase 'a.'

Looping Constructs

Looping constructs accomplish repetitive tasks. Loops can be nested to whatever depth is required, but loops cannot be crossed. The macro facility has two looping constructs: the index loop (DO loop) and the list-processing loop (DO IN loop).

- When a macro is expanded, looping constructs are interpreted after arguments are substituted and functions are executed.

Index Loop

The syntax of an index loop is as follows:

```
!DO !var = start !TO finish [ !BY step ]
    statements
!BREAK
!DOEND
```

- The indexing variable is *!var* and must begin with an exclamation point.
- The start, finish, and step values must be numbers or expressions that evaluate to numbers.
- The loop begins at the start value and continues until it reaches the finish value (unless a !BREAK statement is encountered). The step value is optional and can be used to specify a subset of iterations. If start is set to 1, finish to 10, and step to 3, the loop will be executed four times with the index variable assigned values 1, 4, 7, and 10.

- The statements can be any valid commands or macro keywords. !DOEND specifies the end of the loop.
- !BREAK is an optional specification. It can be used in conjunction with conditional processing to exit the loop.

Example

```
DEFINE macdef (arg1 = !TOKENS(1)
               /arg2 = !TOKENS(1))
!DO !i = !arg1 !TO !arg2.
frequencies variables = !CONCAT(var,!i).
!DOEND
!ENDDEFINE.
macdef arg1 = 1 arg2 = 3.
```

- The variable *!i* is initially assigned the value 1 (arg1) and is incremented until it equals 3 (arg2), at which point the loop ends.
- The first loop concatenates var and the value for *!I*, which is 1 in the first loop. The second loop concatenates var and 2, and the third concatenates var and 3. The result is that FREQUENCIES is executed three times, with variables *VAR1*, *VAR2*, and *VAR3*, respectively.

List-Processing Loop

The syntax of a list-processing loop is as follows:

```
!DO !var !IN (list)
   statements
!BREAK
!DOEND
```

- The !DO and !DOEND statements begin and end the loop. !BREAK is used to exit the loop.
- The !IN function requires one argument, which must be a list of items. The number of items on the list determines the number of iterations. At each iteration, the index variable *!var* is set to each item on the list.
- The list can be any expression, although it is usually a string. Only one list can be specified in each list-processing loop.

Example

```
DEFINE macdef (!POS !CHAREND('/'))
!DO !i !IN (!1)
frequencies variables = !i.
!DOEND
!ENDDEFINE.
macdef VAR1 VAR2 VAR3  /.
```

- The macro call assigns three variables, *VAR1*, *VAR2*, and *VAR3*, to the positional argument !1. Thus, the loop completes three iterations.
- In the first iteration, *!i* is set to value *VAR1*. In the second and third iterations, *!I* is set to *VAR2* and *VAR3*, respectively. Thus, FREQUENCIES is executed three times, respectively with *VAR1*, *VAR2*, and *VAR3*.

Example

```
DEFINE macdef (!POS !CHAREND('/'))
!DO !i !IN (!1)
sort cases by !i.
report var = earnings
  /break = !i
  /summary = mean.
!DOEND
!ENDDEFINE.

macdef SALESMAN REGION MONTH /.
```

- The positional argument !1 is assigned the three variables *SALESMAN*, *REGION*, and *MONTH*. The loop is executed three times and the index variable *!i* is set to each of the variables in succession. The macro creates three reports.

Direct Assignment of Macro Variables

The macro command !LET assigns values to macro variables. The syntax is as follows:

```
!LET !var = expression
```

- The expression must be either a single token or enclosed in parentheses.
- The macro variable *!var* cannot be a macro keyword, and it cannot be the name of one of the arguments within the macro definition. Thus, !LET cannot be used to change the value of an argument.
- The macro variable *!var* can be a new variable or one previously assigned by a !DO command or another !LET command.

Example

```
!LET !a = 1
!LET !b = !CONCAT(ABC,!SUBSTR(!1,3,1),DEF)
!LET !c = (!2 ~= !NULL)
```

- The first !LET sets *!a* equal to 1.
- The second !LET sets *!b* equal to ABC followed by 1 character taken from the third position of !1 followed by DEF.
- The last !LET sets *!c* equal to 0 (false) if !2 is a null string or to 1 (true) if !2 is not a null string.

DELETE VARIABLES

```
DELETE VARIABLES  varlist.
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
DElETE VARIABLES varX varY thisVar TO thatVar.
```

Overview

DELETE VARIABLES deletes the specified variables from the active dataset.

Basic Specification

- The basic specification is one or more variable names.

Syntax Rules

- The variables must exist in the active dataset.
- The keyword TO can be used to specify consecutive variable in the active dataset.
- This command cannot be executed when there are pending transformations. For example, DELETE VARIABLES cannot be immediately preceded by transformation commands such as COMPUTE or RECODE.
- DELETE VARIABLES cannot be used with TEMPORARY.
- You cannot use this command to delete all variables in the active dataset. If the variable list includes all variables in the active dataset, an error results and the command is not executed. Use NEW FILE to delete all variables.

DESCRIPTIVES

DESCRIPTIVES is available in Statistics Base Edition.

```
DESCRIPTIVES VARIABLES= varname[(zname)] [varname...]

 [/MISSING={VARIABLE**}  [INCLUDE]]
           {LISTWISE  }

 [/SAVE]

 [/STATISTICS=[DEFAULT**]  [MEAN**]  [MIN**]  [SKEWNESS]]
              [STDDEV** ]  [SEMEAN]  [MAX**]  [KURTOSIS]
              [VARIANCE ]  [SUM   ]  [RANGE]  [ALL]

 [/SORT=[{MEAN    }] [{(A)}]]
         {SEMEAN  }  {(D)}
         {STDDEV  }
         {VARIANCE}
         {KURTOSIS}
         {SKEWNESS}
         {RANGE   }
         {MIN     }
         {MAX     }
         {SUM     }
         {NAME    }
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
DESCRIPTIVES VARIABLES=FOOD RENT, APPL TO COOK
```

Overview

DESCRIPTIVES computes univariate statistics--including the mean, standard deviation, minimum, and maximum--for numeric variables. Because it does not sort values into a frequency table, DESCRIPTIVES is an efficient means of computing descriptive statistics for continuous variables. Other procedures that display descriptive statistics include FREQUENCIES, MEANS, and EXAMINE.

Options

Z Scores. You can create new variables that contain *z* scores (standardized deviation scores from the mean) and add them to the active dataset by specifying *z*-score names on the VARIABLES subcommand or by using the SAVE subcommand.

Statistical Display. Optional statistics available with the STATISTICS subcommand include the standard error of the mean, variance, kurtosis, skewness, range, and sum. DESCRIPTIVES does not compute the median or mode (see FREQUENCIES or EXAMINE).

Display Order. You can list variables in ascending or descending alphabetical order or by the numerical value of any of the available statistics using the SORT subcommand.

Basic Specification

The basic specification is the VARIABLES subcommand with a list of variables. All cases with valid values for a variable are included in the calculation of statistics for that variable. Statistics include the mean, standard deviation, minimum, maximum, and number of cases with valid values.

Subcommand Order

- Subcommands can be used in any order.

Operations

- If a string variable is specified on the variable list, no statistics are displayed for that variable.

- If there is insufficient memory available to calculate statistics for all variables requested, DESCRIPTIVES truncates the variable list.

VARIABLES Subcommand

VARIABLES names the variables for which you want to compute statistics.

- The keyword ALL can be used to refer to all user-defined variables in the active dataset.
- Only one variable list can be specified.

Z Scores

The *z*-score transformation standardizes variables to the same scale, producing new variables with a mean of 0 and a standard deviation of 1. These variables are added to the active dataset.

- To obtain *z* scores for all specified variables, use the SAVE subcommand.
- To obtain *z* scores for a subset of variables, name the new variable in parentheses following the source variable on the VARIABLES subcommand and do not use the SAVE subcommand.
- Specify new names individually; a list in parentheses is not recognized.
- The new variable name can be any acceptable variable name that is not already part of the active dataset. For information on variable naming rules, see “Variable Names” on p. 36.

Example

```
DESCRIPTIVES VARIABLES=NTCSAL NTCPUR (PURCHZ) NTCPRI (PRICEZ).
```

- DESCRIPTIVES creates *z*-score variables named *PURCHZ* and *PRICEZ* for *NTCPUR* and *NTCPRI*, respectively. No *z*-score variable is created for *NTCSAL*.

SAVE Subcommand

SAVE creates a *z*-score variable for each variable specified on the VARIABLES subcommand. The new variables are added to the active dataset.

- When DESCRIPTIVES creates new *z*-score variables, it displays the source variable names, the new variable names, and their labels in the Notes table.
- DESCRIPTIVES automatically supplies variable names for the new variables. The new variable name is created by prefixing the letter *Z* to the source variable name. For example, *ZNTCPRI* is the *z*-score variable for *NTCPRI*.
- If the default naming convention duplicates variable names in the active dataset, DESCRIPTIVES uses an alternative naming convention: first *ZSC001* through *ZSC099*, then *STDZ01* through *STDZ09*, then *ZZZZ01* through *ZZZZ09*, and then *ZQZQ01* through *ZQZQ09*.
- Variable labels are created by prefixing *ZSCORE* to the source variable label. If the alternative naming convention is used, DESCRIPTIVES prefixes *ZSCORE(varname)* to the label. If the source variable does not have a label, DESCRIPTIVES uses *ZSCORE(varname)* for the label.
- If you specify new names on the VARIABLES subcommand *and* use the SAVE subcommand, DESCRIPTIVES creates one new variable for each variable on the VARIABLES subcommand, using default names for variables not assigned names on VARIABLES.
- If at any time you want to change any of the variable names, whether those DESCRIPTIVES created or those you previously assigned, you can do so with the RENAME VARIABLES command.

Example

```
DESCRIPTIVES VARIABLES=ALL
  /SAVE.
```

- SAVE creates a *z*-score variable for all variables in the active dataset. All *z*-score variables receive the default name.

Example

```
DESCRIPTIVES VARIABLES=NTCSAL NTCPUR (PURCHZ) NTCPRI (PRICEZ)
  /SAVE.
```

- DESCRIPTIVES creates three *z*-score variables named *ZNTCSAL* (the default name), *PURCHZ*, and *PRICEZ*.

STATISTICS Subcommand

By default, DESCRIPTIVES displays the mean, standard deviation, minimum, and maximum. Use the STATISTICS subcommand to request other statistics.

- When you use STATISTICS, DESCRIPTIVES displays *only* those statistics you request.
- The keyword ALL obtains all statistics.
- You can specify the keyword DEFAULT to obtain the default statistics without having to name MEAN, STDDEV, MIN, and MAX.
- The median and mode, which are available in FREQUENCIES and EXAMINE, are not available in DESCRIPTIVES. These statistics require that values be sorted, and DESCRIPTIVES does not sort values (the SORT subcommand does not sort values, it simply lists variables in the order you request).
- If you request a statistic that is not available, DESCRIPTIVES issues an error message and the command is not executed.

MEAN. *Mean*.

SEMEAN. *Standard error of the mean*.

STDDEV. *Standard deviation*.

VARIANCE. *Variance*.

KURTOSIS. *Kurtosis and standard error of kurtosis.*

SKEWNESS. *Skewness and standard error of skewness.*

RANGE. *Range*.

MIN. *Minimum observed value.*

MAX. *Maximum observed value.*

SUM. *Sum*.

DEFAULT. *Mean, standard deviation, minimum, and maximum*. These are the default statistics.

ALL. *All statistics available in DESCRIPTIVES.*

SORT Subcommand

By default, DESCRIPTIVES lists variables in the order in which they are specified on VARIABLES. Use SORT to list variables in ascending or descending alphabetical order of variable name or in ascending or descending order of numeric value of any of the statistics.

- If you specify SORT without any keywords, variables are sorted in ascending order of the mean.
- SORT can sort variables by the value of any of the statistics available with DESCRIPTIVES, but only those statistics specified on STATISTICS (or the default statistics) are displayed.

Only one of the following keywords can be specified on SORT:

MEAN. *Sort by mean.* This is the default when SORT is specified without a keyword.

SEMEAN. *Sort by standard error of the mean.*

STDDEV. *Sort by standard deviation.*

VARIANCE. *Sort by variance.*

KURTOSIS. *Sort by kurtosis.*

SKEWNESS. *Sort by skewness.*

RANGE. *Sort by range.*

MIN. *Sort by minimum observed value.*

MAX. *Sort by maximum observed value.*

SUM. *Sort by sum.*

NAME. *Sort by variable name.*

Sort order can be specified in parentheses following the specified keyword:

A. *Sort in ascending order.* This is the default when SORT is specified without keywords.

D. *Sort in descending order.*

Example

```
DESCRIPTIVES VARIABLES=A B C
  /STATISTICS=DEFAULT RANGE
  /SORT=RANGE (D).
```

- DESCRIPTIVES sorts variables *A*, *B*, and *C* in descending order of range and displays the mean, standard deviation, minimum and maximum values, range, and the number of valid cases.

MISSING Subcommand

MISSING controls missing values.

- By default, DESCRIPTIVES deletes cases with missing values on a variable-by-variable basis. A case with a missing value for a variable will not be included in the summary statistics for that variable, but the case *will* be included for variables where it is not missing.
- The VARIABLE and LISTWISE keywords are alternatives; however, each can be specified with INCLUDE.
- When either the keyword VARIABLE or the default missing-value treatment is used, DESCRIPTIVES reports the number of valid cases for each variable. It always displays the number of cases that would be available if listwise deletion of missing values had been selected.

VARIABLE. *Exclude cases with missing values on a variable-by-variable basis.* This is the default.

LISTWISE. *Exclude cases with missing values listwise.* Cases with missing values for any variable named are excluded from the computation of statistics for all variables.

INCLUDE. *Include user-missing values.*

DETECTANOMALY

DETECTANOMALY is available in Statistics Base Edition.

```
DETECTANOMALY

[/VARIABLES [CATEGORICAL=varlist]
            [SCALE=varlist]
            [ID=variable]
            [EXCEPT=varlist]]

[/HANDLEMISSING [APPLY={NO**}]
                       {YES }
                [CREATEMISPROPVAR={NO**}]]
                                  {YES }

[/CRITERIA [MINNUMPEERS={1**    }]
                        {integer}
           [MAXNUMPEERS={15**   }]
                        {integer}
           [MLWEIGHT={6**   }]
                     {number}
           [NUMREASONS={1**    }]
                       {integer}
           [PCTANOMALOUSCASES={5**   }]
                              {number}
           [NUMANOMALOUSCASES={integer}]
           [ANOMALYCUTPOINT={2**   }
                            {number}
                            {NONE  }]]

[/SAVE [ANOMALY[(varname)]]
       [PEERID[(varname)]]
       [PEERSIZE[(varname)]]
       [PEERPCTSIZE[(varname)]]
       [REASONVAR[(rootname)]]
       [REASONMEASURE[(rootname)]]
       [REASONVALUE[(rootname)]]
       [REASONNORM[(rootname)]]]

[/OUTFILE [MODEL=filespec]]

[/PRINT [CPS]  [ANOMALYLIST**]  [NORMS]
        [ANOMALYSUMMARY]  [REASONSUMMARY]
        [NONE]]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 14.0

- Command introduced.

Example

```
DETECTANOMALY.
```

Overview

The anomaly detection procedure searches for unusual cases based on deviations from the norms of their cluster groups. The procedure is designed to quickly detect unusual cases for data-auditing purposes in the exploratory data analysis step, prior to any inferential data analysis. This algorithm is designed for generic anomaly detection; that is, the definition of an anomalous case is not specific to any particular application, such as detection of unusual payment patterns in the healthcare industry or detection of money laundering in the finance industry, in which the definition of an anomaly can be well-defined.

Options

Methods. The DETECTANOMALY procedure clusters cases into peer groups based on the similarities of a set of input variables. An anomaly index is assigned to each case to reflect the unusualness of a case with respect to its peer group. All cases are sorted by the values of the anomaly index, and the top portion of the cases is identified as the set of anomalies. For each variable, an impact measure is assigned to each

case that reflects the contribution of the variable to the deviation of the case from its peer group. For each case, the variables are sorted by the values of the variable impact measure, and the top portion of variables is identified as the set of reasons why the case is anomalous.

Data File. The DETECTANOMALY procedure assumes that the input data is a flat file in which each row represents a distinct case and each column represents a distinct variable. Moreover, it is assumed that all input variables are non-constant and that no case has missing values for all of the input variables.

Missing Values. The DETECTANOMALY procedure allows missing values. By default, missing values of continuous variables are substituted by their corresponding grand means, and missing categories of categorical variables are grouped and treated as a valid category. Moreover, an additional variable called the *Missing Proportion Variable*, which represents the proportion of missing variables in each case, is created. The processed variables are used to detect the anomalies in the data. You can turn off either of the options. If the first is turned off, cases with missing values are excluded from the analysis. In this situation, the second option is turned off automatically.

ID Variable. A variable that is the unique identifier of the cases in the data can optionally be specified in the ID keyword. If this keyword is not specified, the case sequence number of the active dataset is assumed to be the ID.

Weights. The DETECTANOMALY procedure ignores specification on the WEIGHT command.

Output. The DETECTANOMALY procedure displays an anomaly list in pivot table output, or offers an option for suppressing it. The procedure can also save the anomaly information to the active dataset as additional variables. Anomaly information can be grouped into three sets of variables: anomaly, peer, and reason. The anomaly set consists of the anomaly index of each case. The peer set consists of the peer group ID of each case, the size, and the percentage size of the peer group. The reason set consists of a number of reasons. Each reason consists of information such as the variable impact, the variable name for this reason, the value of the variable, and the corresponding norm value of the peer group.

Basic Specification

The basic specification is the DETECTANOMALY command. By default, all variables in the active dataset are used in the procedure, with the dictionary setting of each variable in the dataset determining its measurement level.

Syntax Rules

- All subcommands are optional.
- Only a single instance of each subcommand is allowed.
- An error occurs if an attribute or keyword is specified more than once within a subcommand.
- Parentheses, slashes, and equals signs shown in the syntax chart are required.
- Subcommand names and keywords must be spelled in full.
- Empty subcommands are not honored.

Operations

The DETECTANOMALY procedure begins by applying the missing value handling option and the create missing proportion variable option to the data.

Then the procedure groups cases into their peer groups based on the similarities of the processed variables. An anomaly index is assigned to each case to measure the overall deviation of the case from its peer group. All cases are sorted by the values of the anomaly index, and the top portion of the cases is identified as the anomaly list.

For each anomalous case, the variables are sorted by their corresponding variable impact values. The top variables, their values, and the corresponding norm values are presented as the reasons why a case is identified as an anomaly.

By default, the anomaly list is presented in a pivot table. Optionally, the anomaly information can be added to the active dataset as additional variable. The anomaly detection model may be written to an XML model file.

Limitations

WEIGHT and SPLIT FILE settings are ignored with a warning by the DETECTANOMALY procedure.

Examples

```
DETECTANOMALY
  /VARIABLES CATEGORICAL=A B C SCALE=D E
  /SAVE ANOMALY REASON.
```

- DETECTANOMALY treats variables *A*, *B*, and *C* as categorical variables and *D* and *E* as continuous variables.
- All of the processed variables are then used in the analysis to generate the anomaly list. Since no ID option is specified, the case number is used as the case identity variable. The size of the list is the number of cases with an anomaly index of at least 2.0 and is not more than 5% of the size of the working file. The anomaly list consists of the ID variable, the anomaly index, the peer group ID and size, and the reason. By default, there is one reason for each anomaly. Each reason consists of information such as the variable impact measure, the variable name for this reason, the value of the variable, and the value of the corresponding peer group. The anomaly list is presented in the pivot table output.
- Since the keywords ANOMALY and REASON are specified in the SAVE subcommand, the additional variables for the anomaly index and the anomaly reason are added to the active dataset.

Specifying a Case ID Variable and Excepted Variables

```
DETECTANOMALY
  /VARIABLES ID=CaseID EXCEPT=GeoID DemoID AddressID.
```

- DETECTANOMALY uses all variables in the active dataset in the analysis, except the variables *GeoID*, *DemoID*, and *AddressID*, which are excluded. Moreover, it treats the variable *CaseID* as the ID variable in the procedure. The processed variables are used in the analysis to generate the anomaly list.

VARIABLES Subcommand

The VARIABLES subcommand specifies the variables to be used in the procedure. If the CATEGORICAL option or the SCALE option are specified, then variables that are listed in these options are used in the analysis. If neither the CATEGORICAL option nor the SCALE option is specified, then all variables in the active dataset are used, except the variable specified in the ID option and those in the EXCEPT option, if any. In the latter situation, the dictionary setting of each variable determines its measurement level. The procedure treats ordinal and nominal variables equivalently as categorical.

CATEGORICAL=varlist. *List of categorical variables.* If this option is specified, at least one variable must be listed. Variables in the list can be numeric or string. They are treated as categorical variables and are used in the analysis. If duplicate variables are specified in the list, the duplicates are ignored. After the EXCEPT option (if specified) is applied, if there are variables also specified in either the continuous list or as an ID variable, an error is issued. TO and ALL keywords may be used.

SCALE=varlist . *List of continuous variables.* If this option is specified, at least one variable must be listed. Variables in the list must be numeric and are used in the analysis. If duplicate variables are specified in the list, the duplicates are ignored. After the EXCEPT option (if specified) is applied, if there are variables also specified in either the categorical list or as an ID variable, an error is issued. TO and ALL keywords may be used.

ID=variable. *Case ID variable.* If this option is specified, one variable must be listed. The variable can be numeric or string. It is used as a unique identifier for the cases in the data file and is not used in the analysis. If this option is not specified, the case number of the active dataset is used as the identifier variable. If the identifier variable is specified in either the categorical list or continuous list, an error is issued.

EXCEPT=varlist. *List of variables that are excluded from the analysis.* If this option is specified, at least one variable must be listed. Variables in the list are not used in the analysis, even if they are specified in

the continuous or categorical lists. This option ignores duplicate variables and variables that are not specified on the continuous or categorical list. Specifying the ALL keyword causes an error. The TO keyword may be used. This option can be useful if the categorical list or continuous list contains a large number of variables but there are a few variables that should be excluded.

HANDLEMISSING Subcommand

The HANDLEMISSING subcommand specifies the methods of handling missing values in this procedure.

APPLY=optionvalue. *Apply missing value handling.* Valid option values are YES or NO. If YES, the missing values of continuous variables are substituted by their corresponding grand means, and missing categories of categorical variables are grouped and treated as a valid category. The processed variables are used in the analysis. If NO, cases with missing values are excluded from the analysis. The default value is NO.

CREATEMISPROPVAR=optionvalue. *Create an additional Missing Proportion Variable and use it in the analysis.* Valid option values are YES or NO. If YES, an additional variable called the *Missing Proportion Variable* that represents the proportion of missing variables in each case is created, and this variable is used in the analysis. If NO, the *Missing Proportion Variable* is not created. The default value is NO.

CRITERIA Subcommand

The CRITERIA subcommand specifies settings for the DETECTANOMALY procedure.

MINNUMPEERS=integer. *Minimum number of peer groups.* The procedure will search for the best number of peer groups between the specified value and the value in the MAXNUMPEERS keyword. The specified value must be a positive integer less than or equal to the value in the MAXNUMPEERS keyword. When the specified value is equal to the value in the MAXNUMPEERS keyword, the procedure assumes a fixed number of peer groups. The default value is 1. *Note*: Depending on the amount of variation in your data, there may be situations in which the number of peer groups that the data can support is less than the number specified in the MINNUMPEERS option. In such a situation, the procedure may produce a smaller number of peer groups.

MAXNUMPEERS=integer. *Maxim number of peer groups.* The procedure will search for the best number of peer groups between the value in the MINNUMPEERS keyword and the specified value. The specified value must be a positive integer greater than or equal to the value in the MINNUMPEERS keyword. When the specified value is equal to the value in the MINNUMPEERS keyword, the procedure assumes a fixed number of peer groups. The default value is 15.

MLWEIGHT=number. *An adjustment weight on the measurement level.* This parameter is used to balance the influences between continuous and categorical variables during the calculation of the indices. A large value increases the influence of a continuous variable. Specify a positive number. The default value is 6.

NUMREASONS=integer. *Number of reasons in the anomaly list.* A reason consists of information such as the variable impact measure, the variable name for this reason, the value of the variable, and the value of the corresponding peer group. Specify a non-negative integer less than or equal to the number of processed variables used in the analysis. The specified option value will be adjusted downward to the maximum number of variables used in the analysis if it is set larger than the number of variables. The default value is 1.

PCTANOMALOUSCASES=number. *Percentage of cases considered as anomalies and included in the anomaly list.* Specify a non-negative number less than or equal to 100. The default value is 5.

NUMANOMALOUSCASES=integer. *Number of cases considered as anomalies and included in the anomaly list.* Specify a non-negative integer less than or equal to the total number of cases in the active dataset and used in the analysis. If this option is specified, an option value must be listed. The specified option value will be adjusted downward to the maximum available if it is set larger than the number of cases used in the analysis. This option, if specified, overrides the PCTANOMALOUSCASES option.

ANOMALYCUTPOINT=number. *Cut point of the anomaly index to determine whether a case is considered as an anomaly.* Specify a non-negative number. A case is considered anomalous if its anomaly index value is larger than or equal to the specified cut point. This option can be used together with the PCTANOMALOUSCASES and NUMANOMALOUSCASES options. For example, if NUMANOMALOUSCASES=50 and ANOMALYCUTPOINT=2 are specified, the anomaly list will consist of at most 50 cases each with an anomaly index value larger than or equal to 2. The default value is 2. If NONE is specified, the option is suppressed and no cut point is set.

SAVE Subcommand

The SAVE subcommand specifies the additional variables to save to the active dataset.

- One or more keywords should be specified, each followed by an optional variable name or rootname in parentheses.
- The variable name or the rootname, if specified, must be a valid variable name.
- If no variable name or rootname is specified, a default varname or rootname is used. If the default varname or rootname is used and it conflicts with that of an existing variable, a suffix is added to make the name unique.
- The values of the additional variables are assigned to all cases included in the analysis, even if the cases are not in the anomaly list.
- This subcommand is not affected by the specifications on the PCTANOMALOUSCASES, NUMANOMALOUSCASES, or ANOMALYCUTPOINT keywords in the CRITERIA subcommand.

ANOMALY(varname). *The anomaly index.* If an optional varname is not specified, the default varname is *AnomalyIndex*, which is the anomaly index. If an optional varname is specified, the specified varname is used to replace the default varname. For example, if ANOMALY(MyAnomaly) is specified, the variable name will be *MyAnomaly*.

PEERID(varname). *Peer group ID.* If an optional varname is not specified, the default varname *PeerId* is used. If an optional varname is specified, the specified name is used.

PEERSIZE(varname) . *Peer group size.* If an optional varname is not specified, the default variable name *PeerSize* is used. If an optional varname is specified, the specified name is used.

PEERPCTSIZE(varname) . *Peer group size in percentage.* If an optional varname is not specified, the default varname *PeerPctSize* is used. If an optional varname is specified, the specified name is used.

REASONVAR(rootname). *The variable associated with a reason.* The number of REASONVAR variables created depends on the number of reasons specified on the CRITERIA subcommand NUMREASONS option. If an optional rootname is not specified, the default rootname *ReasonVar* is used to automatically generate one or more varnames, *ReasonVar_k*, where *k* is the *k*th reason. If an optional rootname is specified, the specified name is used. If NUMREASONS=0 is specified, this option is ignored and a warning is issued.

REASONMEASURE(rootname). *The variable impact measure associated with a reason.* The number of REASONMEASURE variables created depends on the number of reasons specified on the CRITERIA subcommand NUMREASONS option. If an optional rootname is not specified, the default rootname *ReasonMeasure* is used to automatically generate one or more varnames, *ReasonMeasure_k*, where *k* is the *k*th reason. If an optional rootname is specified, the specified name is used. If NUMREASONS=0 is specified, this option is ignored and a warning is issued.

REASONVALUE(rootname). *The variable value associated with a reason.* The number of REASONVALUE variables created depends on the number of reasons specified on the CRITERIA subcommand NUMREASONS option. If an optional rootname is not specified, the default rootname *ReasonValue* is used to automatically generate one or more varnames, *ReasonValue_k*, where *k* is the *k*th reason. If an optional rootname is specified, the specified name is used. If NUMREASONS=0 is specified, this option is ignored and a warning is issued.

REASONNORM(rootname). *The norm value associated with a reason.* The number of REASONNORM variables created depends on the number of reasons specified on the CRITERIA subcommand

NUMREASONS option. If an optional rootname is not specified, the default rootname *ReasonNorm* is used to automatically generate one or more varnames, *ReasonNorm_k*, where *k* is the *k*th reason. If an optional rootname is specified, the specified name is used. If NUMREASONS=0 is specified, this option is ignored and a warning is issued.

OUTFILE Subcommand

The OUTFILE subcommand directs the DETECTANOMALY procedure to write its model to the specified filename as XML.

MODEL=filespec. *File specification to which the model is written.*

PRINT Subcommand

The PRINT subcommand controls the display of the output results.

- If the PRINT subcommand is not specified, the default output is the anomaly list. If the PRINT subcommand is specified, DETECTANOMALY displays output only for the keywords that are specified.

CPS. *Display a case processing summary.* The case processing summary displays the counts and count percentages for all cases in the active dataset, the cases included and excluded in the analysis, and the cases in each peer.

ANOMALYLIST. *Display the anomaly index list, the anomaly peer ID list, and the anomaly reason list.* The anomaly index list displays the case number and its corresponding anomaly index value. The anomaly peer ID list displays the case number, its corresponding peer group ID, peer size, and size in percent. The anomaly reason list displays the case number, the reason variable, the variable impact value, the value of the variable, and the norm of the variable for each reason. All tables are sorted by anomaly index in descending order. Moreover, the IDs of the cases are displayed if the case identifier variable is specified in the ID option of the VARIABLES subcommand. This is the default output.

NORMS. *Display the continuous variable norms table if any continuous variable is used in the analysis, and display the categorical variable norms table if any categorical variable is used in the analysis.* In the continuous variable norms table, the mean and standard deviation of each continuous variable for each peer group is displayed. In the categorical variable norms table, the mode (most popular category), its frequency, and frequency percent of each categorical variable for each peer group is displayed. The mean of a continuous variable and the mode of a categorical variable are used as the norm values in the analysis.

ANOMALYSUMMARY. *Display the anomaly index summary.* The anomaly index summary displays descriptive statistics for the anomaly index of the cases identified as the most unusual.

REASONSUMMARY. *Display the reason summary table for each reason.* For each reason, the table displays the frequency and frequency percent of each variable's occurrence as a reason. It also reports the descriptive statistics of the impact of each variable. If NUMREASONS=0 is specified on the CRITERIA subcommand, this option is ignored and a warning is issued.

NONE. *Suppress all displayed output except the notes table and any warnings.* If NONE is specified with one or more other keywords, the other keywords override NONE.

DISCRIMINANT

DISCRIMINANT is available in Statistics Base Edition.

```
DISCRIMINANT GROUPS=varname(min,max) /VARIABLES=varlist

 [/SELECT=varname(value)]

 [/ANALYSIS=varlist[(level)] [varlist...]]

 [/OUTFILE MODEL('file')]

 [/METHOD={DIRECT**}] [/TOLERANCE={0.001}]
          {WILKS   }              {   n   }
          {MAHAL   }
          {MAXMINF }
          {MINRESID}
          {RAO     }

 [/MAXSTEPS={n}]

 [/FIN={3.84**}] [/FOUT={2.71**}] [/PIN={n}]
       { n    }         { n    }

 [/POUT={n}] [/VIN={0**}]
                   { n }

 [/FUNCTIONS={g-1,100.0,1.0**}] [/PRIORS={EQUAL**   }]
             {n1 , n2  , n3  }           {SIZE      }
                                         {value list}
 [/SAVE=[CLASS[=varname]] [PROBS[=rootname]]

        [SCORES[=rootname]]]

 [/ANALYSIS=...]

 [/MISSING={EXCLUDE**}]
           {INCLUDE  }

 [/MATRIX=[OUT({*                  })] [IN({*                  })]]
               {'savfile'|'dataset'}       {'savfile'|'dataset'}

 [/HISTORY={STEP**} ]
           {NONE  }

 [/ROTATE={NONE**   }]
          {COEFF    }
          {STRUCTURE}

 [/CLASSIFY={NONMISSING  }  {POOLED  }  [MEANSUB]]
            {UNSELECTED  }  {SEPARATE}
            {UNCLASSIFIED}

 [/STATISTICS=[MEAN]  [COV  ]  [FPAIR]  [RAW ] [STDDEV]
              [GCOV]  [UNIVF]  [COEFF]  [CORR] [TCOV  ]
              [BOXM]  [TABLE]  [CROSSVALID]
              [ALL]]

 [/PLOT=[MAP]  [SEPARATE]  [COMBINED]  [CASES[(n)]]  [ALL]]
```

**Default if subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 17.0

- Added support for SET THREADS and SET MCACHE.

Example

```
DISCRIMINANT GROUPS=OUTCOME (1,4)
  /VARIABLES=V1 TO V7.
```

Overview

DISCRIMINANT performs linear discriminant analysis for two or more groups. The goal of discriminant analysis is to classify cases into one of several mutually exclusive groups based on their values for a set of predictor variables. In the analysis phase, a classification rule is developed using cases for which group membership is known. In the classification phase, the rule is used to classify cases for which group

membership is not known. The grouping variable must be categorical, and the independent (predictor) variables must be interval or dichotomous, since they will be used in a regression-type equation.

Options

Variable Selection Method. In addition to the direct-entry method, you can specify any of several stepwise methods for entering variables into the discriminant analysis using the METHOD subcommand. You can set the values for the statistical criteria used to enter variables into the equation using the TOLERANCE, FIN, PIN, FOUT, POUT, and VIN subcommands, and you can specify inclusion levels on the ANALYSIS subcommand. You can also specify the maximum number of steps in a stepwise analysis using the MAXSTEPS subcommand.

Case Selection. You can select a subset of cases for the analysis phase using the SELECT subcommand.

Prior Probabilities. You can specify prior probabilities for membership in a group using the PRIORS subcommand. Prior probabilities are used in classifying cases.

New Variables. You can add new variables to the active dataset containing the predicted group membership, the probability of membership in each group, and discriminant function scores using the SAVE subcommand.

Classification Options. With the CLASSIFY subcommand, you can classify only those cases that were not selected for inclusion in the discriminant analysis, or only those cases whose value for the grouping variable was missing or fell outside the range analyzed. In addition, you can classify cases based on the separate-group covariance matrices of the functions instead of the pooled within-groups covariance matrix.

Statistical Display. You can request any of a variety of statistics on the STATISTICS subcommand. You can rotate the pattern or structure matrices using the ROTATE subcommand. You can compare actual with predicted group membership using a classification results table requested with the STATISTICS subcommand or compare any of several types of plots or histograms using the PLOT subcommand.

Basic Specification

The basic specification requires two subcommands:

- GROUPS specifies the variable used to group cases.
- VARIABLES specifies the predictor variables.

By default, DISCRIMINANT enters all variables simultaneously into the discriminant equation (the DIRECT method), provided that they are not so highly correlated that multicollinearity problems arise. Default output includes analysis case processing summary, valid numbers of cases in group statistics, variables failing tolerance test, a summary of canonical discriminant functions, standardized canonical discriminant function coefficients, a structure matrix showing pooled within-groups correlations between the discriminant functions and the predictor variables, and functions at group centroids.

Subcommand Order

- The GROUPS, VARIABLES, and SELECT subcommands must precede all other subcommands and may be entered in any order.
- The analysis block follows, which may include ANALYSIS, METHOD, TOLERANCE, MAXSTEPS, FIN, FOUT, PIN, POUT, VIN, FUNCTIONS, PRIORS, SAVE, and OUTFILE. Each analysis block performs a single analysis. To do multiple analyses, specify multiple analysis blocks.
- The keyword ANALYSIS is optional for the first analysis block. Each new analysis block must begin with an ANALYSIS subcommand. Remaining subcommands in the block may be used in any order and apply only to the analysis defined within the same block.
- No analysis block subcommands can be specified after any of the global subcommands, which apply to all analysis blocks. The global subcommands are MISSING, MATRIX, HISTORY, ROTATE, CLASSIFY, STATISTICS, and PLOT. If an analysis block subcommand appears after a global subcommand, the program displays a warning and ignores it.

Syntax Rules

- Only one GROUPS, one SELECT, and one VARIABLES subcommand can be specified per DISCRIMINANT command.

Operations

- DISCRIMINANT first estimates one or more discriminant functions that best distinguish among the groups.
- Using these functions, DISCRIMINANT then classifies cases into groups (if classification output is requested).
- If more than one analysis block is specified, the above steps are repeated for each block.
- This procedure uses the multithreaded options specified by SET THREADS and SET MCACHE.

Limitations

- Pairwise deletion of missing data is not available.

GROUPS Subcommand

GROUPS specifies the name of the grouping variable, which defines the categories or groups, and a range of categories.

- GROUPS is required and can be specified only once.
- The specification consists of a variable name followed by a range of values in parentheses.
- Only one grouping variable may be specified; its values must be integers. To use a string variable as the grouping variable, first use AUTORECODE to convert the string values to integers and then specify the recoded variable as the grouping variable.
- Empty groups are ignored and do not affect calculations. For example, if there are no cases in group 2, the value range (1, 5) will define only four groups.
- Cases with values outside the value range or missing are ignored during the analysis phase but are classified during the classification phase.

VARIABLES Subcommand

VARIABLES identifies the predictor variables, which are used to classify cases into the groups defined on the GROUPS subcommand. The list of variables follows the usual conventions for variable lists.

- VARIABLES is required and can be specified only once. Use the ANALYSIS subcommand to obtain multiple analyses.
- Only numeric variables can be used.
- Variables should be suitable for use in a regression-type equation, either measured at the interval level or dichotomous.

SELECT Subcommand

SELECT limits cases used in the analysis phase to those with a specified value for any one variable.

- Only one SELECT subcommand is allowed. It can follow the GROUPS and VARIABLES subcommands but must precede all other subcommands.
- The specification is a variable name and a single integer value in parentheses. Multiple variables or values are not permitted.
- The selection variable does not have to be specified on the VARIABLES subcommand.
- Only cases with the specified value for the selection variable are used in the analysis phase.
- All cases, whether selected or not, are classified by default. Use CLASSIFY=UNSELECTED to classify only the unselected cases.

- When SELECT is used, classification statistics are reported separately for selected and unselected cases, unless CLASSIFY=UNSELECTED is used to restrict classification.

Example

```
DISCRIMINANT GROUPS=APPROVAL(1,5)
  /VARS=Q1 TO Q10
  /SELECT=COMPLETE(1)
  /CLASSIFY=UNSELECTED.
```

- Using only cases with the value 1 for the variable *COMPLETE*, DISCRIMINANT estimates a function of *Q1* to *Q10* that discriminates between the categories 1 to 5 of the grouping variable *APPROVAL*.
- Because CLASSIFY=UNSELECTED is specified, the discriminant function will be used to classify only the unselected cases (cases for which *COMPLETE* does not equal 1).

ANALYSIS Subcommand

ANALYSIS is used to request several different discriminant analyses using the same grouping variable, or to control the order in which variables are entered into a stepwise analysis.

- ANALYSIS is optional for the first analysis block. By default, all variables specified on the VARIABLES subcommand are included in the analysis.
- The variables named on ANALYSIS must first be specified on the VARIABLES subcommand.
- The keyword ALL includes all variables on the VARIABLES subcommand.
- If the keyword TO is used to specify a list of variables on an ANALYSIS subcommand, it refers to the order of variables on the VARIABLES subcommand, which is not necessarily the order of variables in the active dataset.

Example

```
DISCRIMINANT GROUPS=SUCCESS(0,1)
  /VARIABLES=V10 TO V15, AGE, V5
  /ANALYSIS=V15 TO V5
  /ANALYSIS=ALL.
```

- The first analysis will use the variables *V15*, *AGE*, and *V5* to discriminate between cases where *SUCCESS* equals 0 and *SUCCESS* equals 1.
- The second analysis will use all variables named on the VARIABLES subcommand.

Inclusion Levels

When you specify a stepwise method on the METHOD subcommand (any method other than the default direct-entry method), you can control the order in which variables are considered for entry or removal by specifying inclusion levels on the ANALYSIS subcommand. By default, all variables in the analysis are entered according to the criterion requested on the METHOD subcommand.

- An **inclusion level** is an integer between 0 and 99, specified in parentheses after a variable or list of variables on the ANALYSIS subcommand.
- The default inclusion level is 1.
- Variables with higher inclusion levels are considered for entry before variables with lower inclusion levels.
- Variables with even inclusion levels are entered as a group.
- Variables with odd inclusion levels are entered individually, according to the stepwise method specified on the METHOD subcommand.
- Only variables with an inclusion level of 1 are considered for removal. To make a variable with a higher inclusion level eligible for removal, name it twice on the ANALYSIS subcommand, first specifying the desired inclusion level and then an inclusion level of 1.
- Variables with an inclusion level of 0 are never entered. However, the statistical criterion for entry is computed and displayed.

- Variables that fail the tolerance criterion are not entered regardless of their inclusion level.

The following are some common methods of entering variables and the inclusion levels that could be used to achieve them. These examples assume that one of the stepwise methods is specified on the METHOD subcommand (otherwise, inclusion levels have no effect).

Direct. ANALYSIS=ALL(2) forces all variables into the equation. (This is the default and can be requested with METHOD=DIRECT or simply by omitting the METHOD subcommand.)

Stepwise. ANALYSIS=ALL(1) yields a stepwise solution in which variables are entered and removed in stepwise fashion. (This is the default when anything other than DIRECT is specified on the METHOD subcommand.)

Forward. ANALYSIS=ALL(3) enters variables into the equation stepwise but does not remove variables.

Backward. ANALYSIS=ALL(2) ALL(1) forces all variables into the equation and then allows them to be removed stepwise if they satisfy the criterion for removal.

Inclusion Levels Used With a Stepwise Method

```
DISCRIMINANT GROUPS=SUCCESS(0,1)
  /VARIABLES=A, B, C, D, E
  /ANALYSIS=A TO C (2) D, E (1)
  /METHOD=WILKS.
```

- *A*, *B*, and *C* are entered into the analysis first, assuming that they pass the tolerance criterion. Since their inclusion level is even, they are entered together.
- *D* and *E* are then entered stepwise. The one that minimizes the overall value of Wilks' lambda is entered first.
- After entering *D* and *E*, the program checks whether the partial *F* for either one justifies removal from the equation (see the FOUT and POUT subcommands).

Inclusion Levels Without a Stepwise Method

```
DISCRIMINANT GROUPS=SUCCESS(0,1)
  /VARIABLES=A, B, C, D, E
  /ANALYSIS=A TO C (2) D, E (1).
```

- Since no stepwise method is specified, inclusion levels have no effect and all variables are entered into the model at once.

METHOD Subcommand

METHOD is used to select a method for entering variables into an analysis.

- A variable will never be entered into the analysis if it does not pass the tolerance criterion specified on the TOLERANCE subcommand (or the default).
- A METHOD subcommand applies only to the *preceding* ANALYSIS subcommand, or to an analysis using all predictor variables if no ANALYSIS subcommand has been specified before it.
- If more than one METHOD subcommand is specified within one analysis block, the last is used.

Any one of the following methods can be specified on the METHOD subcommand:

DIRECT. *All variables passing the tolerance criteria are entered simultaneously.* This is the default method.

WILKS. *At each step, the variable that minimizes the overall Wilks' lambda is entered.*

MAHAL. *At each step, the variable that maximizes the Mahalanobis distance between the two closest groups is entered.*

MAXMINF. *At each step, the variable that maximizes the smallest* F *ratio between pairs of groups is entered.*

MINRESID. *At each step, the variable that minimizes the sum of the unexplained variation for all pairs of groups is entered.*

RAO. *At each step, the variable that produces the largest increase in Rao's V is entered.*

OUTFILE Subcommand

Exports model information to the specified file in XML format. You can use this model file to apply the model information to other data files for scoring purposes. See the topic "Scoring expressions" on page 86 for more information.

- The minimum specification is the keyword MODEL and a file name enclosed in parentheses.
- The OUTFILE subcommand cannot be used if split file processing is on (SPLIT FILE command).

TOLERANCE Subcommand

TOLERANCE specifies the minimum tolerance a variable can have and still be entered into the analysis. The tolerance of a variable that is a candidate for inclusion in the analysis is the proportion of its within-groups variance not accounted for by other variables in the analysis. A variable with very low tolerance is nearly a linear function of the other variables; its inclusion in the analysis would make the calculations unstable.

- The default tolerance is 0.001.
- You can specify any decimal value between 0 and 1 as the minimum tolerance.

PIN and POUT Subcommands

PIN specifies the minimum probability of *F* that a variable can have to enter the analysis and POUT specifies the maximum probability of *F* that a variable can have and not be removed from the model.

- PIN and POUT take precedence over FIN and FOUT. That is, if all are specified, PIN and POUT values are used.
- If PIN and POUT are omitted, FIN and FOUT are used by default.
- You can set PIN and POUT to any decimal value between 0 and 1. However, POUT should be greater than PIN if PIN is also specified.
- PIN and POUT apply only to the stepwise methods and are ignored if the METHOD subcommand is omitted or if DIRECT is specified on METHOD.

FIN and FOUT Subcommands

FIN specifies the minimum partial *F* value that a variable must have to enter the analysis. As additional variables are entered into the analysis, the partial *F* for variables already in the equation changes. FOUT specifies the smallest partial *F* that a variable can have and not be removed from the model.

- PIN and POUT take precedence over FIN and FOUT. That is, if all are specified, PIN and POUT values are used.
- If PIN and POUT are omitted, FIN and FOUT are used by default. If FOUT is specified but FIN is omitted, the default value for FIN is 3.84. If FIN is specified, the default value for FOUT is 2.71.
- You can set FIN and FOUT to any non-negative number. However, FOUT should be less than FIN if FIN is also specified.
- FIN and FOUT apply only to the stepwise methods and are ignored if the METHOD subcommand is omitted or if DIRECT is specified on METHOD.

VIN Subcommand

VIN specifies the minimum Rao's *V* that a variable must have to enter the analysis. When you use METHOD=RAO, variables satisfying one of the other criteria for entering the equation may actually cause a decrease in Rao's *V* for the equation. The default VIN prevents this but does not prevent the addition of variables that provide no additional separation between groups.

- You can specify any value for VIN. The default is 0.
- VIN should be used only when you have specified METHOD=RAO. Otherwise, it is ignored.

MAXSTEPS Subcommand

MAXSTEPS is used to decrease the maximum number of steps allowed. By default, the maximum number of steps allowed in a stepwise analysis is the number of variables with inclusion levels greater than 1 plus twice the number of variables with inclusion levels equal to 1. This is the maximum number of steps possible without producing a loop in which a variable is repeatedly cycled in and out.

- MAXSTEPS applies only to the stepwise methods (all except DIRECT).
- MAXSTEPS applies only to the preceding METHOD subcommand.
- The format is MAX=*n,* where *n* is the maximum number of steps desired.
- If multiple MAXSTEPS subcommands are specified, the last is used.

FUNCTIONS Subcommand

By default, DISCRIMINANT computes all possible functions. This is either the number of groups minus 1 or the number of predictor variables, whichever is less. Use FUNCTIONS to set more restrictive criteria for the extraction of functions.

FUNCTIONS has three parameters:

n. *Maximum number of functions*. The default is the number of groups minus 1 or the number of predictor variables, whichever is less.

n. *Cumulative percentage of the sum of the eigenvalues*. The default is 100.

n. *Significance level of function*. The default is 1.0.

- The parameters must always be specified in sequential order (n_1, n_2, n_3). To specify n_2, you must explicitly specify the default for n_1. Similarly, to specify n_3, you must specify the defaults for n_1 and n_2.
- If more than one restriction is specified, the program stops extracting functions when any one of the restrictions is met.
- When multiple FUNCTIONS subcommands are specified, the program uses the last; however, if n_2 or n_3 are omitted on the last FUNCTIONS subcommand, the corresponding specifications on the previous FUNCTIONS subcommands will remain in effect.

Example

```
DISCRIMINANT  GROUPS=CLASS(1,5)
  /VARIABLES = SCORE1 TO SCORE20
  /FUNCTIONS=4,100,.80.
```

- The first two parameters on the FUNCTIONS subcommand are defaults: the default for n_1 is 4 (the number of groups minus 1), and the default for n_2 is 100.
- The third parameter tells DISCRIMINANT to use fewer than four discriminant functions if the significance level of a function is greater than 0.80.

PRIORS Subcommand

By default, DISCRIMINANT assumes equal prior probabilities for groups when classifying cases. You can provide different prior probabilities with the PRIORS subcommand.

- Prior probabilities are used only during classification.
- If you provide unequal prior probabilities, DISCRIMINANT adjusts the classification coefficients to reflect this.
- If adjacent groups have the same prior probability, you can use the notation *n*c* on the value list to indicate that *n* adjacent groups have the same prior probability *c.*
- You can specify a prior probability of 0. No cases are classified into such a group.
- If the sum of the prior probabilities is not 1, the program rescales the probabilities to sum to 1 and issues a warning.

EQUAL. *Equal prior probabilities*. This is the default.

SIZE. *Proportion of the cases analyzed that fall into each group.* If 50% of the cases included in the analysis fall into the first group, 25% in the second, and 25% in the third, the prior probabilities are 0.5, 0.25, and 0.25, respectively. Group size is determined after cases with missing values for the predictor variables are deleted.

Value list. *User-specified prior probabilities*. The list of probabilities must sum to 1.0. The number of prior probabilities named or implied must equal the number of groups.

Example

```
DISCRIMINANT  GROUPS=TYPE(1,5)
  /VARIABLES=A TO H
  /PRIORS = 4*.15,.4.
```

- The PRIORS subcommand establishes prior probabilities of 0.15 for the first four groups and 0.4 for the fifth group.

SAVE Subcommand

SAVE allows you to save casewise information as new variables in the active dataset.

- SAVE applies only to the current analysis block. To save casewise results from more than one analysis, specify a SAVE subcommand in each analysis block.
- You can specify a variable name for CLASS and rootnames for SCORES and PROBS to obtain descriptive names for the new variables.
- If you do not specify a variable name for CLASS, the program forms variable names using the formula *DSC_m*, where *m* increments to distinguish group membership variables saved on different SAVE subcommands for different analysis blocks.
- If you do not specify a rootname for SCORES or PROBS, the program forms new variable names using the formula *DSCn_m*, where *m* increments to create unique rootnames and *n* increments to create unique variable names. For example, the first set of default names assigned to discriminant scores or probabilities are *DSC1_1*, *DSC2_1*, *DSC3_1*, and so on. The next set of default names assigned will be *DSC1_2*, *DSC2_2*, *DSC3_2*, and so on, regardless of whether discriminant scores or probabilities are being saved or whether they are saved by the same SAVE subcommand.
- The keywords CLASS, SCORES, and PROBS can be used in any order, but the new variables are always added to the end of the active dataset in the following order: first the predicted group, then the discriminant scores, and finally probabilities of group membership.
- Appropriate variable labels are automatically generated. The labels describe whether the variables contain predictor group membership, discriminant scores, or probabilities, and for which analysis they are generated.

- The CLASS variable will use the value labels (if any) from the grouping variable specified for the analysis.
- When SAVE is specified with any keyword, DISCRIMINANT displays a classification processing summary table and a prior probabilities for groups table.
- You cannot use the SAVE subcommand if you are replacing the active dataset with matrix materials (see) .

CLASS [(varname)]. *Predicted group membership.*

SCORES [(rootname)]. *Discriminant scores.* One score is saved for each discriminant function derived. If a rootname is specified, DISCRIMINANT will append a sequential number to the name to form new variable names for the discriminant scores.

PROBS [(rootname)]. *For each case, the probabilities of membership in each group.* As many variables are added to each case as there are groups. If a rootname is specified, DISCRIMINANT will append a sequential number to the name to form new variable names.

Example

```
DISCRIMINANT GROUPS=WORLD(1,3)
 /VARIABLES=FOOD TO FSALES
 /SAVE CLASS=PRDCLASS SCORES=SCORE PROBS=PRB
 /ANALYSIS=FOOD SERVICE COOK MANAGER FSALES
 /SAVE CLASS SCORES PROBS.
```

- Two analyses are specified. The first uses all variables named on the VARIABLES subcommand and the second narrows down to five variables. For each analysis, a SAVE subcommand is specified.
- For each analysis, DISCRIMINANT displays a classification processing summary table and a prior probabilities for groups table.
- On the first SAVE subcommand, a variable name and two rootnames are provided. With three groups, the following variables are added to each case:

Table 40. Saved variables based on rootnames

Name	Variable label	Description
PRDCLASS	Predicted group for analysis 1	Predicted group membership
SCORE1	Function 1 for analysis 1	Discriminant score for function 1
SCORE2	Function 2 for analysis 1	Discriminant score for function 2
PRB1	Probability 1 for analysis 1	Probability of being in group 1
PRB2	Probability 2 for analysis 1	Probability of being in group 2
PRB3	Probability 3 for analysis 1	Probability of being in group 3

- Since no variable name or rootnames are provided on the second SAVE subcommand, DISCRIMINANT uses default names. Note that *m* serves only to distinguish variables saved as a set and does not correspond to the sequential number of an analysis. To find out what information a new variable holds, read the variable label, as shown in the following table:

Table 41. Default saved variable names

Name	Variable label	Description
DSC_1	Predicted group for analysis 2	Predicted group membership
DSC1_1	Function 1 for analysis 2	Discriminant score for function 1
DSC2_1	Function 2 for analysis 2	Discriminant score for function 2
DSC1_2	Probability 1 for analysis 2	Probability of being in group 1
DSC2_2	Probability 2 for analysis 2	Probability of being in group 2
DSC3_2	Probability 3 for analysis 2	Probability of being in group 3

STATISTICS Subcommand

By default, DISCRIMINANT produces the following statistics for each analysis: analysis case processing summary, valid numbers of cases in group statistics, variables failing tolerance test, a summary of canonical discriminant functions, standardized canonical discriminant function coefficients, a structure matrix showing pooled within-groups correlations between the discriminant functions and the predictor variables, and functions at group centroids.

- *Group statistics.* Only valid number of cases is reported.
- *Summary of canonical discriminant functions.* Displayed in two tables: an eigenvalues table with percentage of variance, cumulative percentage of variance, and canonical correlations and a Wilks' lambda table with Wilks' lambda, chi-square, degrees of freedom, and significance.
- *Stepwise statistics.* Wilks' lambda, equivalent *F*, degrees of freedom, significance of *F* and number of variables are reported for each step. Tolerance, *F*-to-remove, and the value of the statistic used for variable selection are reported for each variable in the equation. Tolerance, minimum tolerance, *F*-to-enter, and the value of the statistic used for variable selection are reported for each variable not in the equation. (These statistics can be suppressed with HISTORY=NONE.)
- *Final statistics.* Standardized canonical discriminant function coefficients, the structure matrix of discriminant functions and all variables named in the analysis (whether they were entered into the equation or not), and functions evaluated at group means are reported following the last step.

In addition, you can request optional statistics on the STATISTICS subcommand. STATISTICS can be specified by itself or with one or more keywords.

- STATISTICS without keywords displays MEAN, STDDEV, and UNIVF. If you include a keyword or keywords on STATISTICS, only the statistics you request are displayed.

MEAN. *Means*. Total and group means for all variables named on the ANALYSIS subcommand are displayed.

STDDEV. *Standard deviations*. Total and group standard deviations for all variables named on the ANALYSIS subcommand are displayed.

UNIVF. *Univariate* F *ratios*. The analysis-of-variance *F* statistic for equality of group means for each predictor variable is displayed. This is a one-way analysis-of-variance test for equality of group means on a single discriminating variable.

COV. *Pooled within-groups covariance matrix*.

CORR. *Pooled within-groups correlation matrix*.

FPAIR. *Matrix of pairwise* F *ratios*. The *F* ratio for each pair of groups is displayed. This *F* is the significance test for the Mahalanobis distance between groups. This statistic is available only with stepwise methods.

BOXM. *Box's* M *test*. This is a test for equality of group covariance matrices.

GCOV. *Group covariance matrices*.

TCOV. *Total covariance matrix*.

RAW. *Unstandardized canonical discriminant functions*.

COEFF. *Classification function coefficients*. Although DISCRIMINANT does not directly use these coefficients to classify cases, you can use them to classify other samples (see the CLASSIFY subcommand).

TABLE. *Classification results*. If both selected and unselected cases are classified, the results are reported separately. To obtain cross-validated results for selected cases, specify CROSSVALID.

CROSSVALID. *Cross-validated classification results.* The cross-validation is done by treating *n*–1 out of *n* observations as the training dataset to determine the discrimination rule and using the rule to classify the one observation left out. The results are displayed only for selected cases.

ALL. *All optional statistics.*

ROTATE Subcommand

The coefficient and correlation matrices can be rotated to facilitate interpretation of results. To control varimax rotation, use the ROTATE subcommand.

- Neither COEFF nor STRUCTURE affects the classification of cases.

COEFF. *Rotate pattern matrix.* DISCRIMINANT displays a varimax transformation matrix, a rotated standardized canonical discriminant function coefficients table, and a correlations between variables and rotated functions table.

STRUCTURE. *Rotate structure matrix*. DISCRIMINANT displays a varimax transformation matrix, a rotated structure matrix, and a rotated standardized canonical discriminant function coefficients table.

NONE. *Do not rotate.* This is the default.

HISTORY Subcommand

HISTORY controls the display of stepwise and summary output.

- By default, HISTORY displays both the step-by-step output and the summary table (keyword STEP, alias END).

STEP. *Display step-by-step and summary output*. Alias END. This is the default. See *Stepwise statistics* in .

NONE. *Suppress the step-by-step and summary table*. Alias NOSTEP, NOEND.

CLASSIFY Subcommand

CLASSIFY determines how cases are handled during classification.

- By default, all cases with nonmissing values for all predictors are classified, and the pooled within-groups covariance matrix is used to classify cases.
- The default keywords for CLASSIFY are NONMISSING and POOLED.

NONMISSING. *Classify all cases that do not have missing values on any predictor variables.* Two sets of classification results are produced, one for selected cases (those specified on the SELECT subcommand) and one for unselected cases. This is the default.

UNSELECTED. *Classify only unselected cases*. The classification phase is suppressed for cases selected via the SELECT subcommand. If all cases are selected (when the SELECT subcommand is omitted), the classification phase is suppressed for all cases and no classification results are produced.

UNCLASSIFIED. *Classify only unclassified cases*. The classification phase is suppressed for cases that fall within the range specified on the GROUPS subcommand.

POOLED. *Use the pooled within-groups covariance matrix to classify cases.* This is the default.

SEPARATE. *Use separate-groups covariance matrices of the discriminant functions for classification.* DISCRIMINANT displays the group covariances of canonical discriminant functions and Box's test of equality of covariance matrices of canonical discriminant functions. Since classification is based on the discriminant functions and not the original variables, this option is not necessarily equivalent to quadratic discrimination.

MEANSUB. *Substitute means for missing predictor values during classification*. During classification, means are substituted for missing values and cases with missing values are classified. Cases with missing values are not used during analysis.

PLOT Subcommand

PLOT requests additional output to help you examine the effectiveness of the discriminant analysis.

- If PLOT is specified without keywords, the default is COMBINED and CASES.
- If any keywords are requested on PLOT, only the requested plots are displayed.
- If PLOT is specified with any keyword except MAP, DISCRIMINANT displays a classification processing summary table and a prior probabilities for groups table.

COMBINED. *All-groups plot*. For each case, the first two function values are plotted.

CASES(n). *Casewise statistics.* For each case, classification information, squared Mahalanobis distance to centroid for the highest and second highest groups, and discriminant scores of all functions are plotted. Validated statistics are displayed for selected cases if CROSSVALID is specified on STATISTICS. If *n* is specified, DISCRIMINANT displays the first *n* cases only.

MAP. *Territorial map*. A plot of group centroids and boundaries used for classifying groups.

SEPARATE. *Separate-groups plots*. These are the same types of plots produced by the keyword COMBINED, except that a separate plot is produced for each group. If only one function is used, a histogram is displayed.

ALL. *All available plots.*

MISSING Subcommand

MISSING controls the treatment of cases with missing values in the analysis phase. By default, cases with missing values for any variable named on the VARIABLES subcommand are not used in the analysis phase but are used in classification.

- The keyword INCLUDE includes cases with user-missing values in the analysis phase.
- Cases with missing or out-of-range values for the grouping variable are always excluded.

EXCLUDE. *Exclude all cases with missing values*. Cases with user or system-missing values are excluded from the analysis. This is the default.

INCLUDE. *Include cases with user-missing values*. User-missing values are treated as valid values. Only the system-missing value is treated as missing.

MATRIX Subcommand

MATRIX reads and writes IBM SPSS Statistics matrix data files.

- Either IN or OUT and the matrix file in parentheses are required. When both IN and OUT are used in the same DISCRIMINANT procedure, they can be specified on separate MATRIX subcommands or on the same subcommand.

OUT ('savfile'|'dataset'). *Write a matrix data file.* Specify either a quoted file specification, a previously declared dataset name (DATASET DECLARE command) or an asterisk (*), enclosed in parentheses. If you specify an asterisk (*), the matrix data file replaces the active dataset .

IN ('savfile'|'dataset'). *Read a matrix data file.* Specify either a quoted file specification, a previously declared dataset name (DATASET DECLARE command) or an asterisk (*), enclosed in parentheses. An asterisk indicates the active dataset. A matrix file read from an a file or dataset does not replace the active dataset.

Matrix Output

- In addition to Pearson correlation coefficients, the matrix materials written by DISCRIMINANT include weighted and unweighted numbers of cases, means, and standard deviations. (See for a description of the file.) These materials can be used in subsequent DISCRIMINANT procedures.
- Any documents contained in the active dataset are not transferred to the matrix file.
- If BOXM or GCOV is specified on the STATISTICS subcommand or SEPARATE is specified on the CLASSIFY subcommand when a matrix file is written, the *STDDEV* and *CORR* records in the matrix materials represent within-cell data, and separate covariance matrices are written to the file. When the matrix file is used as input for a subsequent DISCRIMINANT procedure, at least one of these specifications must be used on that DISCRIMINANT command.

Matrix Input

- DISCRIMINANT can read correlation matrices written by a previous DISCRIMINANT command or by other procedures. Matrix materials read by DISCRIMINANT must contain records with *ROWTYPE_* values *MEAN*, *N* or *COUNT* (or both), *STDDEV*, and *CORR*.
- If the data do not include records with *ROWTYPE_* value *COUNT* (unweighted number of cases), DISCRIMINANT uses information from records with *ROWTYPE_* value *N* (weighted number of cases). Conversely, if the data do not have *N* values, DISCRIMINANT uses the *COUNT* values. These records can appear in any order in the matrix input file with the following exceptions: the order of split-file groups cannot be violated and all *CORR* vectors must appear consecutively within each split-file group.
- If you want to use a covariance-type matrix as input to DISCRIMINANT, you must first use the MCONVERT command to change the covariance matrix to a correlation matrix.
- DISCRIMINANT can use a matrix from a previous dataset to classify data in the active dataset. The program checks to make sure that the grouping variable (specified on GROUPS) and the predictor variables (specified on VARIABLES) are the same in the active dataset as in the matrix file. If they are not, the program displays an error message and the classification will not be executed.
- MATRIX=IN cannot be used unless a active dataset has already been defined. To read an existing matrix data file at the beginning of a session, first use GET to retrieve the matrix file and then specify IN(*) on MATRIX.

Format of the Matrix Data File

- The matrix data file has two special variables created by the program: *ROWTYPE_* and *VARNAME_*. Variable *ROWTYPE_* is a short string variable having values *N*, *COUNT*, *MEAN*, *STDDEV*, and *CORR* (for Pearson correlation coefficient). The variable *VARNAME_* is a short string variable whose values are the names of the variables used to form the correlation matrix.
- When *ROWTYPE_* is *CORR*, *VARNAME_* gives the variable associated with that row of the correlation matrix.
- Between *ROWTYPE_* and *VARNAME_* is the grouping variable, which is specified on the GROUPS subcommand of DISCRIMINANT.
- The remaining variables are the variables used to form the correlation matrix.

Split Files

- When split-file processing is in effect, the first variables in the matrix data file will be split variables, followed by *ROWTYPE_*, the grouping variable, *VARNAME_*, and then the variables used to form the correlation matrix.
- A full set of matrix materials is written for each subgroup defined by the split variables.
- A split variable cannot have the same variable name as any other variable written to the matrix data file.
- If split-file processing is in effect when a matrix is written, the same split file must be in effect when that matrix is read by another procedure.

STDDEV and CORR Records

Records written with *ROWTYPE_* values *STDDEV* and *CORR* are influenced by specifications on the STATISTICS and CLASSIFY subcommands.

- If BOXM or GCOV is specified on STATISTICS or SEPARATE is specified on CLASSIFY, the *STDDEV* and *CORR* records represent within-cell data and receive values for the grouping variable.
- If none of the above specifications is in effect, the *STDDEV* and *CORR* records represent pooled values. The *STDDEV* vector contains the square root of the mean square error for each variable, and *STDDEV* and *CORR* records receive the system-missing value for the grouping variable.

Missing Values

Missing-value treatment affects the values written to a matrix data file. When reading a matrix data file, be sure to specify a missing-value treatment on DISCRIMINANT that is compatible with the treatment that was in effect when the matrix materials were generated.

Examples

Writing Output to a Matrix Data File

```
GET FILE='/DATA/UNIONBK.SAV' /KEEP WORLD FOOD SERVICE BUS MECHANIC
                CONSTRUC COOK MANAGER FSALES APPL RENT.
DISCRIMINANT  GROUPS=WORLD(1,3)
 /VARIABLES=FOOD SERVICE BUS MECHANIC CONSTRUC COOK MANAGER FSALES
 /METHOD=WILKS
 /PRIORS=SIZE
 /MATRIX=OUT('/DATA/DISCMTX.SAV').
```

- DISCRIMINANT reads data from the data file *UNIONBK.SAV* and writes one set of matrix materials to the file *DISCMTX.SAV*.
- The active dataset is still *UNIONBK.SAV*. Subsequent commands are executed on this file.

Using Matrix Output to Classify Data in a Different File

```
GET FILE=UB2 /KEEP WORLD FOOD SERVICE BUS MECHANIC
             CONSTRUC COOK MANAGER FSALES APPL RENT.
DISCRIMINANT  GROUPS=WORLD(1,3)
 /VARIABLES=FOOD SERVICE BUS MECHANIC CONSTRUC COOK MANAGER FSALES
 /METHOD=WILKS
 /PRIORS=SIZE
 /MATRIX=IN(DISCMTX).
```

- The matrix data file created in the previous example is used to classify data from the file *UB2*.

Replacing the Active Dataset with Matrix Data Output

```
GET FILE=UNIONBK /KEEP WORLD FOOD SERVICE BUS MECHANIC
                 CONSTRUC COOK MANAGER FSALES APPL RENT.
DISCRIMINANT  GROUPS=WORLD(1,3)
 /VARIABLES=FOOD SERVICE BUS MECHANIC CONSTRUC COOK MANAGER FSALES
 /METHOD=WILKS
 /PRIORS=SIZE
 /MATRIX=OUT(*).
LIST.
```

- DISCRIMINANT writes the same matrix as in the first example. However, the matrix data file replaces the active dataset.
- The LIST command is executed on the matrix file, not on the *UNIONBK* file.

Using the Active Dataset as Matrix Input

```
GET FILE=DISCMTX.
DISCRIMINANT  GROUPS=WORLD(1,3)
 /VARIABLES=FOOD SERVICE BUS MECHANIC CONSTRUC COOK MANAGER FSALES
 /METHOD=RAO
 /MATRIX=IN(*).
```

- This example assumes that you are starting a new session and want to read an existing matrix data file. GET retrieves the matrix data file *DISCMTX*.
- MATRIX=IN specifies an asterisk because the matrix data file is the active dataset. If MATRIX=IN(DISCMTX) is specified, the program issues an error message.

- If the GET command is omitted, the program issues an error message.

Using Matrix Output as Matrix Input in the Active Dataset

```
GET FILE=UNIONBK /KEEP WORLD FOOD SERVICE BUS MECHANIC
                  CONSTRUC COOK MANAGER FSALES APPL RENT.
DISCRIMINANT  GROUPS=WORLD(1,3)
 /VARIABLES=FOOD SERVICE BUS MECHANIC CONSTRUC COOK MANAGER FSALES
 /CLASSIFY=SEPARATE
 /MATRIX=OUT(*).
DISCRIMINANT  GROUPS=WORLD(1,3)
 /VARIABLES=FOOD SERVICE BUS MECHANIC CONSTRUC COOK MANAGER FSALES
 /STATISTICS=BOXM
 /MATRIX=IN(*).
```

- The first DISCRIMINANT command creates a matrix with CLASSIFY=SEPARATE in effect. To read this matrix, the second DISCRIMINANT command must specify either BOXM or GCOV on STATISTICS or SEPARATE on CLASSIFY. STATISTICS=BOXM is used.

DISPLAY

```
DISPLAY [SORTED] [{NAMES**    }] [/VARIABLES=varlist]
                 {INDEX       }
                 {VARIABLES   }
                 {LABELS      }
                 {DICTIONARY  }
                 {ATTRIBUTES  }
                 {@ATTRIBUTES}

        {[SCRATCH]  }
        {[VECTOR]   }
        {[MACROS]   }
        {[DOCUMENTS]}
```

**Default if the subcommand is omitted.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic "Command Order" on page 36 for more information.

Release History

Release 14.0

- ATTRIBUTES keyword introduced.

Release 15.0

- @ATTRIBUTES keyword introduced.

Example

```
DISPLAY SORTED DICTIONARY /VARIABLES=DEPT SALARY SEX TO JOBCAT.
```

Overview

DISPLAY exhibits information from the dictionary of the active dataset. The information can be sorted, and it can be limited to selected variables.

Basic Specification

The basic specification is simply the command keyword, which displays an unsorted list of the variables in the active dataset.

Syntax Rules

DISPLAY can be specified by itself or with one of the keywords defined below. NAMES is the default. To specify two or more keywords, use multiple DISPLAY commands.

NAMES. *Variable names.* A list of the variables in the active dataset is displayed.

DOCUMENTS. *Documentary text.* Documentary text is provided on the DOCUMENT and ADD DOCUMENT commands. No error message is issued if there is no documentary information in the active dataset.

DICTIONARY. *Complete dictionary information for variables.* Information includes variable names, labels, role, sequential position of each variable in the file, print and write formats, missing values, and value labels.

ATTRIBUTES. *Variable and data file attributes, except attributes with names that begin with "@" or "$@".* Custom attributes defined by the VARIABLE ATTRIBUTE and DATAFILE ATTRIBUTE commands.

@ATTRIBUTES. *All variable and data file attributes, including those with names that begin with "@" or "$@".*

INDEX. *Variable names and positions.*

VARIABLES. *Variable names, positions, print and write formats, and missing values.*

LABELS. *Variable names, positions, and variable labels.*

SCRATCH. *Scratch variable names.*

VECTOR. *Vector names.*

MACROS. *Currently defined macros.* The macro names are always sorted.

Operations

- DISPLAY directs information to the output.
- If SORTED is not specified, information is displayed according to the order of variables in the active dataset.
- DISPLAY is executed as soon as it is encountered in the command sequence, as long as a dictionary has been defined.

Examples

```
GET FILE="/data/hub.sav".
DISPLAY DOCUMENTS.
DISPLAY DICTIONARY.
```

- Each DISPLAY command specifies only one keyword. The first requests documentary text and the second requests complete dictionary information for the *hub.sav* file.

SORTED Keyword

SORTED alphabetizes the display by variable name. SORTED can precede the keywords NAMES, DICTIONARY, INDEX, VARIABLES, LABELS, SCRATCH, or VECTOR.

Example

```
DISPLAY SORTED DICTIONARY.
```

- This command displays complete dictionary information for variables in the active dataset, sorted alphabetically by variable name.

VARIABLES Subcommand

VARIABLES (alias NAMES) limits the displayed information to a set of specified variables. VARIABLES must be the last specification on DISPLAY and can follow any specification that requests information about variables (all except VECTOR, SCRATCH, DOCUMENTS, and MACROS).

- The only specification is a slash followed by a list of variables. The slash is optional.
- If the keyword SORTED is not specified, information is displayed in the order in which variables are stored in the active dataset, regardless of the order in which variables are named on VARIABLES.

Example

```
DISPLAY SORTED DICTIONARY
 /VARIABLES=DEPT, SALARY, SEX TO JOBCAT.
```

- DISPLAY exhibits dictionary information only for the variables named and implied by the keyword TO on the VARIABLES subcommand, sorted alphabetically by variable name.

DMCLUSTER

DMCLUSTER is available in SPSS Statistics Premium Edition or the Direct Marketing option.

This is an alias for TWOSTEP CLUSTER. See the topic "TWOSTEP CLUSTER" on page 2065 for more information.

DMLOGISTIC

DMLOGISTIC is available in SPSS Statistics Premium Edition or the Direct Marketing option.

This is an alias for LOGISTIC REGRESSION. See the topic "LOGISTIC REGRESSION" on page 989 for more information.

DMROC

DMROC is available in SPSS Statistics Premium Edition or the Direct Marketing option.

This is an alias for ROC. See the topic "ROC" on page 1723 for more information.

DMTABLES

DMTABLES is available in SPSS Statistics Premium Edition or the Direct Marketing option.

This is an alias for CTABLES. See the topic “CTABLES” on page 479 for more information.

DMTREE

DMTREE is available in SPSS Statistics Premium Edition or the Direct Marketing option.

This is an alias for TREE. See the topic "TREE" on page 1991 for more information.

DO IF

```
DO IF [(]logical expression[)]
transformation commands
[ELSE IF [(]logical expression[)]]
transformation commands
[ELSE IF [(]logical expression[)]]
  .
  .
  .
[ELSE]
transformation commands
END IF
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

The following relational operators can be used in logical expressions:

Table 42. Relational operators

Symbol	Definition
EQ or =	Equal to
NE or ~= or ¬ = or <>	Not equal to
LT or <	Less than
LE or <=	Less than or equal to
GT or >	Greater than
GE or >=	Greater than or equal to

The following logical operators can be used in logical expressions:

Table 43. Logical operators

Symbol	Definition
AND or &	Both relations must be true
OR or |	Either relation can be true
NOT	Reverses the outcome of an expression

Example

```
DO IF (YearHired GT 87).
COMPUTE            Bonus = 0.
ELSE IF (Dept87 EQ 3).
COMPUTE            Bonus = .1*Salary87.
ELSE IF (Dept87 EQ 1).
COMPUTE            Bonus = .12*Salary87.
ELSE IF (Dept87 EQ 4).
COMPUTE            Bonus = .08*Salary87.
ELSE IF (Dept87 EQ 2).
COMPUTE            Bonus = .14*Salary87.
END IF.
```

Overview

The DO IF–END IF structure conditionally executes one or more transformations on subsets of cases based on one or more logical expressions. The ELSE command can be used within the structure to

execute one or more transformations when the logical expression on DO IF is not true. The ELSE IF command within the structure provides further control.

The DO IF–END IF structure is best used for conditionally executing multiple transformation commands, such as COMPUTE, RECODE, and COUNT. DO IF–END IF transforms data for subsets of cases defined by logical expressions. To perform repeated transformations on the same case, use LOOP–END LOOP.

A DO IF–END IF structure can be used within an input program to define complex files that cannot be handled by standard file definition facilities. See the topic "Complex File Structures " on page 617 for more information.

See END FILE for information on using DO IF–END IF to instruct the program to stop reading data before it encounters the end of the file or to signal the end of the file when creating data.

Basic Specification

The basic specification is DO IF followed by a logical expression, a transformation command, and the END IF command, which has no specifications.

Examples

Simple, One-Condition Example

```
DO IF (YearHired LT 87).
RECODE Ethnicity(1=5)(2=4)(4=2)(5=1).
END IF.
```

- The RECODE command recodes *Ethnicity* for those individuals hired before 1987 (*YearHired* is less than 87). The *Ethnicity* variable is not recoded for individuals hired in 1987 or later.
- The RECODE command is skipped for any case with a missing value for *YearHired*.

Conditional Execution Based on a Logical Expression

```
DATA LIST      FREE / X(F1).
NUMERIC        #QINIT.
DO IF          NOT #QINIT.
+  PRINT EJECT.
+  COMPUTE        #QINIT = 1.
END IF.
PRINT          / X.

BEGIN DATA
1 2 3 4 5
END DATA.
EXECUTE.
```

- This example shows how to execute a command only once.
- The NUMERIC command creates scratch variable *#QINIT*, which is initialized to 0.
- The NOT logical operator on DO IF reverses the outcome of a logical expression. In this example, the logical expression is a numeric variable that takes only 0 (false) or 1 (true) as its values. The PRINT EJECT command is executed only once, when the value of scratch variable *#QINIT* equals 0. After the COMPUTE command sets *#QINIT* to 1, the DO IF structure is skipped for all subsequent cases. A scratch variable is used because it is initialized to 0 and is not reinitialized after each case.

Syntax Rules

- The ELSE IF command is optional and can be repeated as many times as needed.
- The ELSE command is optional. It can be used only once and must follow any ELSE IF commands.
- The END IF command must follow any ELSE IF and ELSE commands.
- A logical expression must be specified on the DO IF and ELSE IF commands. Logical expressions are not used on the ELSE and END IF commands.

- String values used in expressions must be specified in quotation marks and must include any leading or trailing blanks. Lowercase letters are distinguished from uppercase letters.
- To create a new string variable within a DO IF—END IF structure, you must first declare the variable on the STRING command.
- DO IF—END IF structures can be nested to any level permitted by available memory. They can be nested within LOOP—END LOOP structures, and loop structures can be nested within DO IF structures.

Example

```
DATA LIST FREE /var1.
BEGIN DATA
1 2 3 4 5
END DATA.
DO IF (var1 > 2) & (var1 < 5).
- COMPUTE var2=1.
ELSE IF (var1=2).
- COMPUTE var2=2.
ELSE.
- COMPUTE var2=3.
END IF.
```

Table 44. DO IF results

var1	var2
1	3
2	2
3	1
4	1
5	3

Example

```
INPUT PROGRAM.
+ STRING odd (A3).
+ LOOP numvar=1 TO 5.
+  DO IF MOD(numvar, 2)=0.
+   COMPUTE odd='No'.
+  ELSE.
+   COMPUTE odd='Yes'.
+  END IF.
+  END CASE.
+ END LOOP.
+ END FILE.
END INPUT PROGRAM.
```

Table 45. DO IF results for input program

numvar	odd
1	Yes
2	No
3	Yes
4	No
5	Yes

Logical Expressions

- Logical expressions can be simple logical variables or relations, or they can be complex logical tests involving variables, constants, functions, relational operators, and logical operators. Logical expressions can use any of the numeric or string functions allowed in COMPUTE transformations (see COMPUTE).
- Parentheses can be used to enclose the logical expression itself and to specify the order of operations within a logical expression. Extra blanks or parentheses can be used to make the expression easier to read.

- Blanks (*not* commas) are used to separate relational operators from expressions.
- A relation can include variables, constants, or more complicated arithmetic expressions. Relations cannot be abbreviated. For example, the first relation below is valid; the second is not:

Valid: `(A EQ 2 OR A EQ 5)` *Not valid:* `(A EQ 2 OR 5)`

- A relation cannot compare a string variable to a numeric value or variable, or vice versa. A relation cannot compare the result of a logical function (SYSMIS, MISSING, ANY, or RANGE) to a number.

Operations

- DO IF marks the beginning of the control structure and END IF marks the end. Control for a case is passed out of the structure as soon as a logical condition is met on a DO IF, ELSE IF, or ELSE command.
- A logical expression is evaluated as true, false, or missing. A transformation specified for a logical expression is executed only if the expression is true.
- Logical expressions are evaluated in the following order: functions, exponentiation, arithmetic operations, relations, and finally, logical operators. (For strings, the order is functions, relations, and then logical operators.) When more than one logical operator is used, NOT is evaluated first, followed by AND, and then OR. You can change the order of operations using parentheses.
- Numeric variables created within a DO IF structure are initially set to the system-missing value. By default, they are assigned an F8.2 format.
- New string variables created within a DO IF structure are initially set to a blank value and are assigned the format specified on the STRING command that creates them.
- If the transformed value of a string variable exceeds the variable's defined format, the value is truncated. If the value is shorter than the format, the value is right-padded with blanks.
- If WEIGHT is specified within a DO IF structure, it takes effect unconditionally.
- Commands that take effect immediately without reading the data or executing pending commands (e.g., VALUE LABELS, MISSING VALUES, VARIABLE LABEL), commands that don't require an active dataset (e.g., DISPLAY, HOST, INSERT, OMS, SET) and the GET command are processed unconditionally and immediately each time they are encountered in a DO IF structure. The specified condition is ignored for these commands.
- Commands that require a data pass, such as statistical or charting procedures, cannot be included in a DO IF structure and will cause the command to fail.
- The DO IF–END IF structure (like LOOP–END LOOP) can include commands such as DATA LIST, END CASE, END FILE, and REREAD, which define complex file structures.

Flow of Control

- If the logical expression on DO IF is true, the commands immediately following DO IF are executed up to the next ELSE IF, ELSE, or END IF command. Control then passes to the first statement following END IF.
- If the expression on DO IF is false, control passes to the following ELSE IF command. Multiple ELSE IF commands are evaluated in the order in which they are specified until the logical expression on one of them is true. Commands following that ELSE IF command are executed up to the ELSE or END IF command, and control passes to the first statement following END IF.
- If none of the expressions are true on the DO IF or any of the ELSE IF commands, the commands following ELSE are executed and control passes out of the structure. If there is no ELSE command, a case goes through the entire structure with no change.
- Missing values returned by the logical expression on DO IF or on any ELSE IF cause control to pass to the END IF command at that point.

Missing Values and Logical Operators

When two or more relations are joined by logical operators AND and OR, the program always returns missing if all of the relations in the expression are missing. However, if any one of the relations can be determined, the program tries to return true or false according to the logical outcomes shown in the following table. The asterisk indicates situations where the program can evaluate the outcome with incomplete information.

Table 46. Logical outcomes

Expression	Outcome	Expression	Outcome
true AND true	= true	true OR true	= true
true AND false	= false	true OR false	= true
false AND false	= false	false OR false	= false
true AND missing	= missing	true OR missing	= true*
missing AND missing	= missing	missing OR missing	= missing
false AND missing	= false*	false OR missing	= missing

ELSE Command

ELSE executes one or more transformations when none of the logical expressions on DO IF or any ELSE IF commands is true.

- Only one ELSE command is allowed within a DO IF–END IF structure.
- ELSE must follow all ELSE IF commands (if any) in the structure.
- If the logical expression on DO IF or any ELSE IF command is true, the program ignores the commands following ELSE.

Example

```
DO IF (X EQ 0).
COMPUTE Y=1.
ELSE.
COMPUTE Y=2.
END IF.
```

- *Y* is set to 1 for all cases with value 0 for *X*, and *Y* is 2 for all cases with any other valid value for *X*.
- The value of *Y* is not changed by this structure if *X* is missing.

Example

```
DO IF (YearHired GT 87).
COMPUTE            Bonus = 0.
ELSE.
IF (Dept87 EQ 1) Bonus = .12*Salary87.
IF (Dept87 EQ 2) Bonus = .14*Salary87.
IF (Dept87 EQ 3) Bonus = .1*Salary87.
IF (Dept87 EQ 4) Bonus = .08*Salary87.
END IF.
```

- If an individual was hired after 1987 (*YearHired* is greater than 87), *Bonus* is set to 0 and control passes out of the structure. Otherwise, control passes to the IF commands following ELSE.
- Each IF command evaluates every case. The value of *Bonus* is transformed only when the case meets the criteria specified on IF. Compare this structure with the second example for the ELSE IF command, which performs the same task more efficiently.

Example

```
* Test for listwise deletion of missing values.

DATA LIST / V1 TO V6 1-6.
BEGIN DATA
123456
```

```
    56
1 3456
123456
123456
END DATA.

DO IF NMISS(V1 TO V6)=0.
+  COMPUTE SELECT='V'.
ELSE
+  COMPUTE SELECT='M'.
END IF.

FREQUENCIES VAR=SELECT.
```

- If there are no missing values for any of the variables *V1* to *V6*, COMPUTE sets the value of *SELECT* equal to V (for valid). Otherwise, COMPUTE sets the value of *SELECT* equal to M (for missing).
- FREQUENCIES generates a frequency table for *SELECT*. The table gives a count of how many cases have missing values for one or more variables, and how many cases have valid values for all variables. Commands in this example can be used to determine how many cases are dropped from an analysis that uses listwise deletion of missing values.

ELSE IF Command

ELSE IF executes one or more transformations when the logical expression on DO IF is not true.

- Multiple ELSE IF commands are allowed within the DO IF–END IF structure.
- If the logical expression on DO IF is true, the program executes the commands immediately following DO IF up to the first ELSE IF. Then control passes to the command following the END IF command.
- If the result of the logical expression on DO IF is false, control passes to ELSE IF.

Example

```
STRING Stock(A9).
DO IF (ITEM EQ 0).
COMPUTE Stock='New'.
ELSE IF (ITEM LE 9).
COMPUTE Stock='Old'.
ELSE.
COMPUTE Stock='Cancelled'.
END IF.
```

- STRING declares string variable *Stock* and assigns it a width of nine characters.
- The first COMPUTE is executed for cases with value 0 for *ITEM*, and then control passes out of the structure. Such cases are not reevaluated by ELSE IF, even though 0 is less than 9.
- When the logical expression on DO IF is false, control passes to the ELSE IF command, where the second COMPUTE is executed only for cases with *ITEM* less than or equal to 9. Then control passes out of the structure.
- If the logical expressions on both the DO IF and ELSE IF commands are false, control passes to ELSE, where the third COMPUTE is executed.
- The DO IF–END IF structure sets *Stock* equal to New when *ITEM* equals 0, to Old when *ITEM* is less than or equal to 9 but not equal to 0 (including negative numbers if they are valid), and to Cancelled for all valid values of *ITEM* greater than 9. The value of *Stock* remains blank if *ITEM* is missing.

Example

```
DO IF (YearHired GT 87).
COMPUTE            Bonus = 0.
ELSE IF (Dept87 EQ 3).
COMPUTE            Bonus = .1*Salary87.
ELSE IF (Dept87 EQ 1).
COMPUTE            Bonus = .12*Salary87.
ELSE IF (Dept87 EQ 4).
COMPUTE            Bonus = .08*Salary87.
ELSE IF (Dept87 EQ 2).
COMPUTE            Bonus = .14*Salary87.
END IF.
```

- For cases hired after 1987, *Bonus* is set to 0 and control passes out of the structure. For a case that was hired before 1987 with value 3 for *Dept87*, *Bonus* equals 10% of salary. Control then passes out of the structure. The other three ELSE IF commands are not evaluated for that case. This differs from the

second example for the ELSE command, where the IF command is evaluated for every case. The DO IF–ELSE IF structure shown here is more efficient.

- If Department 3 is the largest, Department 1 the next largest, and so forth, control passes out of the structure quickly for many cases. For a large number of cases or a command file that will be executed frequently, these efficiency considerations can be important.

Nested DO IF Structures

To perform transformations involving logical tests on two variables, you can use nested DO IF–END IF structures.

- There must be an END IF command for every DO IF command in the structure.

Example

```
DO IF (Ethnicity EQ 5).         /*Do whites
+  DO IF (Gender EQ 2).       /*White female
+  COMPUTE Gender_Ethnicity=3.
+  ELSE.                      /*White male
+  COMPUTE Gender_Ethnicity=1.
+  END IF.                    /*Whites done
ELSE IF (Gender EQ 2).        /*Nonwhite female
COMPUTE Gender_Ethnicity=4.
ELSE.                         /*Nonwhite male
COMPUTE Gender_Ethnicity=2.
END IF.                       /*Nonwhites done
```

- This structure creates variable *Gender_Ethnicity*, which indicates both the sex and minority status of an individual.
- An optional plus sign, minus sign, or period in the first column allows you to indent commands so you can easily see the nested structures.

Complex File Structures

Some complex file structures may require you to embed more than one DATA LIST command inside a DO IF–END IF structure. For example, consider a data file that has been collected from various sources. The information from each source is basically the same, but it is in different places on the records:

```
111295100FORD      CHAPMAN AUTO SALES
121199005VW    MIDWEST VOLKSWAGEN SALES
11 395025FORD      BETTER USED CARS
11       CHEVY 195005        HUFFMAN SALES & SERVICE
11       VW    595020        MIDWEST VOLKSWAGEN SALES
11       CHEVY 295015        SAM'S AUTO REPAIR
12       CHEVY 210 20        LONGFELLOW CHEVROLET
 9555032 VW                  HYDE PARK IMPORTS
```

In the above file, an automobile part number always appears in columns 1 and 2, and the automobile manufacturer always appears in columns 10 through 14. The location of other information, such as price and quantity, depends on both the part number and the type of automobile. The DO IF–END IF structure in the following example reads records for part type 11.

Example

```
INPUT PROGRAM.
DATA LIST FILE="/data/carparts.txt"
  /PARTNO 1-2 KIND 10-14 (A).

DO IF (PARTNO EQ 11 AND KIND EQ 'FORD').
+ REREAD.
+ DATA LIST /PRICE 3-6 (2) QUANTITY 7-9 BUYER 20-43 (A).
+ END CASE.

ELSE IF (PARTNO EQ 11 AND (KIND EQ 'CHEVY' OR KIND EQ 'VW')).
+ REREAD.
+ DATA LIST /PRICE 15-18 (2) QUANTITY 19-21 BUYER 30-53 (A).
+ END CASE.
END IF.
END INPUT PROGRAM.

PRINT FORMATS PRICE (DOLLAR6.2).
PRINT /PARTNO TO BUYER.
WEIGHT BY QUANTITY.
DESCRIPTIVES PRICE.
```

- The first DATA LIST extracts the part number and the type of automobile.
- Depending on the information from the first DATA LIST, the records are reread, pulling the price, quantity, and buyer from different places.
- The two END CASE commands limit the working file to only those cases with a part number of 11 and automobile type of Ford, Chevrolet, or Volkswagen. Without the END CASE commands, cases would be created in the working file for other part numbers and automobile types with missing values for price, quantity, and buyer.
- The results of the PRINT command are shown below.

```
11 FORD $12.95 100 CHAPMAN AUTO SALES
11 FORD  $3.95 25  BETTER USED CARS
11 CHEVY $1.95 5   HUFFMAN SALES & SERVICE
11 VW    $5.95 20  MIDWEST VOLKSWAGEN SALES
11 CHEVY $2.95 15  SAM'S AUTO REPAIR
```

Figure 52. Printed information for part 11

DO REPEAT-END REPEAT

```
DO REPEAT stand-in var={varlist  | ALL  } [/stand-in var=...]
                       {value list}

transformation commands

END REPEAT [PRINT]
```

Release History

Release 14.0

- ALL keyword introduced.

Example

```
DO REPEAT var=var1 to var5
         /value=1 to 5.
COMPUTE var=value.
END REPEAT.
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic "Command Order" on page 36 for more information.

Overview

The DO REPEAT—END REPEAT structure repeats the same transformations on a specified set of variables, reducing the number of commands you must enter to accomplish a task. This utility does not reduce the number of commands the program executes, just the number of commands you enter. To display the expanded set of commands the program generates, specify PRINT on END REPEAT.

DO REPEAT uses a **stand-in variable** to represent a **replacement list** of variables or values. The stand-in variable is specified as a placeholder on one or more transformation commands within the structure. When the program repeats the transformation commands, the stand-in variable is replaced, in turn, by each variable or value specified on the replacement list.

The following commands can be used within a DO REPEAT—END REPEAT structure:

- Data transformations: COMPUTE, RECODE, IF, COUNT, and SELECT IF
- Data declarations: VECTOR, STRING, NUMERIC, and LEAVE
- Data definition: DATA LIST, MISSING VALUES (but not VARIABLE LABELS or VALUE LABELS)
- Loop structure commands: LOOP, END LOOP, and BREAK
- Do-if structure commands: DO IF, ELSE IF, ELSE, and END IF
- Print and write commands: PRINT, PRINT EJECT, PRINT SPACE, and WRITE
- Format commands: PRINT FORMATS, WRITE FORMATS, and FORMATS

Basic Specification

The basic specification is DO REPEAT, a stand-in variable followed by a required equals sign and a replacement list of variables or values, and at least one transformation command. The structure must end with the END REPEAT command. On the transformation commands, a single stand-in variable represents every variable or value specified on the replacement list.

Syntax Rules

- Multiple stand-in variables can be specified on a DO REPEAT command. Each stand-in variable must have its own equals sign and associated variable or value list and must be separated from other stand-in variables by a slash. All lists must name or generate the same number of items.
- Stand-in variables can be assigned any valid variable names: permanent, temporary, scratch, system, and so forth. A stand-in variable does not exist outside the DO REPEAT—END REPEAT structure and has

no effect on variables with the same name that exist outside the structure. However, two stand-in variables cannot have the same name within the same DO REPEAT structure.

- A replacement variable list can include either new or existing variables. You cannot mix new and existing variables in the same replacement list.
- Keyword TO can be used to name consecutive existing variables and to create a set of new variables, and keyword ALL can be used to specify all variables.
- New string variables must be declared on the STRING command either before DO REPEAT or within the DO REPEAT structure.
- All replacement variable and value lists must have the same number of items.
- A replacement value list can be a list of strings or numeric values, or it can be of the form n_1 TO n_2, where n_1 is less than n_2 and both are integers. (Note that the keyword is TO, not THRU.)

Operations

- DO REPEAT marks the beginning of the control structure and END REPEAT marks the end. Once control passes out of the structure, all stand-in variables defined within the structure cease to exist.
- The program repeats the commands between DO REPEAT and END REPEAT once for each variable or value on the replacement list.
- Numeric variables created within the structure are initially set to the system-missing value. By default, they are assigned an F8.2 format.
- New string variables declared within the structure are initially set to a blank value and are assigned the format specified on the STRING command that creates them.
- If DO REPEAT is used to create new variables, the order in which they are created depends on how the transformation commands are specified. Variables created by specifying the TO keyword (for example, *V1* TO *V5*) are not necessarily consecutive in the active dataset. See the topic "PRINT Subcommand " on page 621 for more information.
- Multiple replacement lists are stepped through in parallel, not in a nested fashion, and all replacement lists must name or generate the same number of items.

Examples

Creating Multiple New Variables with the Same Value

```
DO REPEAT R=REGION1 TO REGION5.
COMPUTE R=0.
END REPEAT.
```

- DO REPEAT defines the stand-in variable *R*, which represents five new numeric variables: *REGION1, REGION2*, *REGION3, REGION4*, and *REGION5*.
- The five variables are initialized to 0 by a single COMPUTE specification that is repeated for each variable on the replacement list. Thus, the program generates five COMPUTE commands from the one specified.
- Stand-in variable *R* ceases to exist once control passes out of the DO REPEAT structure.

Multiple Replacement Lists

```
DO REPEAT existVar=firstVar TO var5
         /newVar=new1 TO new5
         /value=1 TO 5.
COMPUTE newVar=existVar*value.
END REPEAT PRINT.

****generated COMPUTE commands****
  57 +COMPUTE        new1=firstVar*1
  58 +COMPUTE        new2=secondVar*2
  59 +COMPUTE        new3=var3*3
  60 +COMPUTE        new4=fourthVar*4
  61 +COMPUTE        new5=var5*5.
```

- existVar=firstVar to var5 includes all existing variables from *firstVar* to *var5*, in file order.
- newVar=new1 TO new5 specifies five new variables: *var1*, *var2*, *var3*, *var4*, and *var5*.

- `value=1 to 5` specifies a list of five consecutive values: 1, 2, 3, 4, 5.
- All three replacement lists contain five items, and five COMPUTE commands are generated.

Generating Data with DO REPEAT, LOOP, and INPUT PROGRAM

```
* This example shows a typical application of INPUT PROGRAM, LOOP,
   and DO REPEAT. A data file containing random numbers is generated.

INPUT PROGRAM.
+  LOOP #I = 1 TO 1000.
+     DO REPEAT RESPONSE = R1 TO R400.
+          COMPUTE RESPONSE = UNIFORM(1) > 0.5.
+     END REPEAT.
+     COMPUTE AVG = MEAN(R1 TO R400).
+     END CASE.
+   END LOOP.
+   END FILE.
END INPUT PROGRAM.

FREQUENCIES VARIABLE=AVG
 /FORMAT=CONDENSE
 /HISTOGRAM
 /STATISTICS=MEAN MEDIAN MODE STDDEV MIN MAX.
```

- The INPUT PROGRAM–END INPUT PROGRAM structure encloses an input program that builds cases from transformation commands.
- The indexing variable (*#I*) on LOOP–END LOOP indicates that the loop should be executed 1000 times.
- The DO REPEAT–END REPEAT structure generates 400 variables, each with a 50% chance of being 0 and a 50% chance of being 1. This is accomplished by specifying a logical expression on COMPUTE that compares the values returned by UNIFORM(1) to the value 0.5. (UNIFORM(1) generates random numbers between 0 and 1.) Logical expressions are evaluated as false (0), true (1), or missing. Thus, each random number returned by UNIFORM that is 0.5 or less is evaluated as false and assigned the value 0, and each random number returned by UNIFORM that is greater than 0.5 is evaluated as true and assigned the value 1.
- The second COMPUTE creates variable *AVG*, which is the mean of *R1* to *R400* for each case.
- END CASE builds a case with the variables created within each loop. Thus, the loop structure creates 1000 cases, each with 401 variables (*R1* to *R400*, and *AVG)*.
- END FILE signals the end of the data file generated by the input program. If END FILE were not specified in this example, the input program would go into an infinite loop. No dataset would be built, and the program would display an error message for every procedure that follows the input program.
- FREQUENCIES produces a condensed frequency table, histogram, and statistics for *AVG*. The histogram for *AVG* shows a normal distribution.

PRINT Subcommand

The PRINT subcommand on END REPEAT displays the commands generated by the DO REPEAT–END REPEAT structure. PRINT can be used to verify the order in which commands are executed.

Example

```
DO REPEAT Q=Q1 TO Q5/ R=R1 TO R5.
COMPUTE Q=0.
COMPUTE R=1.
END REPEAT PRINT.
```

- The DO REPEAT–END REPEAT structure initializes one set of variables to 0 and another set to 1.
- The output from the PRINT subcommand is shown below. The generated commands are preceded by plus signs.
- The COMPUTE commands are generated in such a way that variables are created in alternating order: *Q1*, *R1*, *Q2*, *R2*, and so forth. If you plan to use the TO keyword to refer to *Q1* to *Q5* later, you should use two separate DO REPEAT utilities; otherwise, *Q1* to *Q5* will include four of the five *R* variables. Alternatively, use the NUMERIC command to predetermine the order in which variables are added to the active dataset, or specify the replacement value lists as shown in the next example.

```
  2      0          DO REPEAT Q=Q1 TO Q5/ R=R1 TO R5
  3      0          COMPUTE Q=0
  4      0          COMPUTE R=1
  5      0          END REPEAT PRINT

  6      0         +COMPUTE Q1=0
  7      0         +COMPUTE R1=1
  8      0         +COMPUTE Q2=0
  9      0         +COMPUTE R2=1
 10      0         +COMPUTE Q3=0
 11      0         +COMPUTE R3=1
 12      0         +COMPUTE Q4=0
 13      0         +COMPUTE R4=1
 14      0         +COMPUTE Q5=0
 15      0         +COMPUTE R5=1
```

Figure 53. Output from the PRINT subcommand

Example

```
DO REPEAT Q=Q1 TO Q5,R1 TO R5/ N=0,0,0,0,0,1,1,1,1,1.
COMPUTE Q=N.
END REPEAT PRINT.
```

- In this example, a series of constants is specified as a stand-in value list for *N*. All the *Q* variables are initialized first, and then all the *R* variables, as shown below.

```
  2      0               DO REPEAT Q=Q1 TO Q5,R1 TO R5/ N=0,0,0,0,0,1,1,1,1,1
  3      0               COMPUTE Q=N
  4      0               END REPEAT PRINT

  5      0              +COMPUTE Q1=0
  6      0              +COMPUTE Q2=0
  7      0              +COMPUTE Q3=0
  8      0              +COMPUTE Q4=0
  9      0              +COMPUTE Q5=0
 10      0              +COMPUTE R1=1
 11      0              +COMPUTE R2=1
 12      0              +COMPUTE R3=1
 13      0              +COMPUTE R4=1
 14      0              +COMPUTE R5=1
```

Figure 54. Output from the PRINT subcommand

Example

```
DO REPEAT R=REGION1 TO REGION5/ X=1 TO 5.
COMPUTE R=REGION EQ X.
END REPEAT PRINT.
```

- In this example, stand-in variable *R* represents the variable list *REGION1* to *REGION5*. Stand-in variable *X* represents the value list 1 to 5.
- The DO REPEAT–END REPEAT structure creates dummy variables *REGION1* to *REGION5* that equal 0 or 1 for each of 5 regions, depending on whether variable *REGION* equals the current value of stand-in variable *X*.
- PRINT on END REPEAT causes the program to display the commands generated by the structure, as shown below.

```
  2      0      DO REPEAT R=REGION1 TO REGION5/ X=1 TO 5
  3      0      COMPUTE R=REGION EQ X
  4      0      END REPEAT PRINT

  5      0     +COMPUTE REGION1=REGION EQ 1
  6      0     +COMPUTE REGION2=REGION EQ 2
  7      0     +COMPUTE REGION3=REGION EQ 3
  8      0     +COMPUTE REGION4=REGION EQ 4
  9      0     +COMPUTE REGION5=REGION EQ 5
```

Figure 55. Commands generated by DO REPEAT

DOCUMENT

```
DOCUMENT text
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
DOCUMENT    This file contains a subset of variables from the
            General Social Survey data.  For each case it records
            only the age, sex, education level, marital status,
            number of children, and type of medical insurance
            coverage.
```

Overview

DOCUMENT saves a block of text of any length in data files saved in IBM SPSS Statistics format. The documentation can be displayed with the DISPLAY command. (See also ADD DOCUMENT.)

When GET retrieves a data file, or when ADD FILES, MATCH FILES, or UPDATE is used to combine data files, all documents from each specified file are copied into the active dataset. DROP DOCUMENTS can be used to drop those documents from the active dataset. Whether or not DROP DOCUMENTS is used, new documents can be added to the active dataset with the DOCUMENT command.

Basic Specification

The basic specification is DOCUMENT followed by any length of text. The text is stored in the file dictionary when the data file is saved in IBM SPSS Statistics format.

Syntax Rules

- The text can be entered on as many lines as needed.
- Blank lines can be used to separate paragraphs.
- A period at the end of a line terminates the command, so you should not place a period at the end of any line but the last.
- Multiple DOCUMENT commands can be used within the command sequence. However, the DISPLAY command cannot be used to exhibit the text from a particular DOCUMENT command. DISPLAY shows all existing documentation.

Operations

- The documentation and the date it was entered are saved in the data file’s dictionary. New documentation is saved along with any documentation already in the active dataset.
- If a DROP DOCUMENTS command *follows* a DOCUMENT command anywhere in the command sequence, the documentation added by that DOCUMENT command is dropped from the active dataset along with all other documentation.

Examples

Adding Descriptive Text to a Data File

```
GET FILE="/data/gensoc.sav" /KEEP=AGE SEX EDUC MARITAL CHILDRN MED_INS.
FILE LABEL  General Social Survey subset.

DOCUMENT    This file contains a subset of variables from the
            General Social Survey data.  For each case it records
            only the age, sex, education level, marital status,
            number of children, and type of medical insurance
            coverage.

SAVE OUTFILE="/data/subsoc.sav".
```

- GET keeps only a subset of variables from the file *gensoc.sav*. All documentation from the file *GENSOC* is copied into the active dataset.
- FILE LABEL creates a label for the new active dataset.
- DOCUMENT specifies the new document text. Both existing documents from the file *GENSOC* and the new document text are saved in the file *subsoc.sav*.

Replacing Existing DOCUMENT Text

```
GET FILE="/data/gensoc.sav" /KEEP=AGE SEX EDUC MARITAL CHILDRN MED_INS.

DROP DOCUMENTS.

FILE LABEL  General Social Survey subset.

DOCUMENT    This file contains a subset of variables from the
            General Social Survey data.  For each case it records
            only the age, sex, education level, marital status,
            number of children, and type of medical insurance
            coverage.

SAVE OUTFILE="/data/subsoc.sav".
```

- DROP DOCUMENTS drops the documentation from the file *gensoc.sav* as data are copied into the active dataset. Only the new documentation specified on DOCUMENT is saved in the file *subsoc.sav*.

DROP DOCUMENTS

```
DROP DOCUMENTS
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Overview

When GET retrieves IBM SPSS Statistics data files, or when ADD FILES, MATCH FILES, or UPDATE are used to combine IBM SPSS Statistics data files, all documents from each specified file are copied into the active dataset. DROP DOCUMENTS is used to drop these or any documents added with the DOCUMENT command from the active dataset. Whether or not DROP DOCUMENTS is used, new documents can be added to the active dataset with the DOCUMENT command.

Basic Specification

The only specification is DROP DOCUMENTS. There are no additional specifications.

Operations

- Documents are dropped from the active dataset only. The original data file is unchanged, unless it is resaved.
- DROP DOCUMENTS drops all documentation, including documentation added by any DOCUMENT commands specified prior to the DROP DOCUMENTS command.

Examples

```
GET FILE="/data/gensoc.sav"
  /KEEP=AGE SEX EDUC MARITAL CHILDRN MED_INS.

DROP DOCUMENTS.

FILE LABEL  General Social Survey Subset.
DOCUMENT    This file contains a subset of variables from the
            General Social Survey data.  For each case it records
            only the age, sex, education level, marital status,
            number of children, and type of medical insurance
            coverage.

SAVE OUTFILE="/data/subsoc.sav".
```

- DROP DOCUMENTS drops the documentation text from data file. Only the new documentation added with the DOCUMENT command is saved in file *subsoc.sav*.
- The original file *gensoc.sav* is unchanged.

ECHO

```
ECHO "text".
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
ECHO "Hey! Look at this!".
```

Overview

ECHO displays the quoted text string as text output.

Basic Specification

The basic specification is the command name ECHO followed by a quoted text string.

Syntax Rules

The text string must be enclosed in single or double quotes, following the standard rules for quoted strings.

- The text string can be continued on multiple lines by enclosing each line in quotes and using the plus sign (+) to combine the strings; the string will be displayed on a single line in output.

END CASE

```
END CASE
```

Example

```
* Restructure a data file to make each data item into a single case.

INPUT PROGRAM.
DATA LIST /#X1 TO #X3 (3(F1,1X)).

VECTOR V=#X1 TO #X3.

LOOP #I=1 TO 3.
- COMPUTE X=V(#I).
- END CASE.
END LOOP.
END INPUT PROGRAM.
```

Overview

END CASE is used in an INPUT PROGRAM–END INPUT PROGRAM structure to signal that a case is complete. Control then passes to the commands immediately following the input program. After these commands are executed for the newly created case, the program returns to the input program and continues building cases by processing the commands immediately after the last END CASE command that was executed. For more information about the flow control in an input program, see INPUT PROGRAM–END INPUT PROGRAM.

END CASE is especially useful for restructuring files, either building a single case from several cases or building several cases from a single case. It can also be used to generate data without any data input (see DO REPEAT for an example).

Basic Specification

The basic specification is simply END CASE. There are no additional specifications.

Syntax Rules

- END CASE is available only within an input program and is generally specified within a loop.
- Multiple END CASE commands can be used within an input program. Each builds a case from the transformation and data definition commands executed since the last END CASE command.
- If no END CASE is explicitly specified, an END CASE command is implied immediately before END INPUT PROGRAM and the input program loops until an end-of-file is encountered or specified (see END FILE).

Operations

- When an END CASE command is encountered, the program suspends execution of the rest of the commands before the END INPUT PROGRAM command and passes control to the commands after the input program. After these commands are executed for the new case, control returns to the input program. The program continues building cases by processing the commands immediately after the most recent END CASE command. Use a loop to build cases from the same set of transformation and data definition commands.
- When multiple END CASE commands are specified, the program follows the flow of the input program and builds a case whenever it encounters an END CASE command, using the set of commands executed since the last END CASE.
- Unless LEAVE is specified, all variables are reinitialized each time the input program is resumed.
- When transformations such as COMPUTE, definitions such as VARIABLE LABELS, and utilities such as PRINT are specified between the last END CASE command and END INPUT PROGRAM, they are executed while a case is being initialized, not when it is complete. This may produce undesirable results.

Examples

Restructuring a data file to make each data item a single case

```
INPUT PROGRAM.
DATA LIST /#X1 TO #X3 (3(F1,1X)).

VECTOR V=#X1 TO #X3.

LOOP #I=1 TO 3.
- COMPUTE X=V(#I).
- END CASE.
END LOOP.
END INPUT PROGRAM.

BEGIN DATA
2 1 1
3 5 1
END DATA.
FORMAT X(F1.0).
PRINT / X.
EXECUTE.
```

- The input program encloses the commands that build cases from the input file. An input program is required because END CASE is used to create multiple cases from single input records.
- DATA LIST defines three variables. In the format specification, the number 3 is a repetition factor that repeats the format in parentheses three times, once for each variable. The specified format is F1 and the 1X specification skips one column.
- VECTOR creates the vector *V* with the original scratch variables as its three elements. The indexing expression on the LOOP command increments the variable *#I* three times to control the number of iterations per input case and to provide the index for the vector *V*.
- COMPUTE sets *X* equal to each of the scratch variables. END CASE tells the program to build a case. Thus, the first loop (for the first case) sets *X* equal to the first element of vector *V*. Since *V(1)* references *#X1*, and *#X1* is 2, the value of *X* is 2. Variable *X* is then formatted and printed before control returns to the command END LOOP. The loop continues, since indexing is not complete. Thus, the program then sets *X* to *#X2*, which is 1, builds the second case, and passes it to the FORMAT and PRINT commands. After the third iteration, which sets *X* equal to 1, the program formats and prints the case and terminates the loop. Since the end of the file has not been encountered, END INPUT PROGRAM passes control to the first command in the input program, DATA LIST, to read the next input case. After the second loop, however, the program encounters END DATA and completes building the active dataset.
- The six new cases are shown below.

```
2
1
1
3
5
1
```

Figure 56. Outcome for multiple cases read from a single case

Restructuring a data file to create a separate case for each book order

```
INPUT PROGRAM.
DATA LIST  /ORDER 1-4 #X1 TO #X22 (1X,11(F3.0,F2.0,1X)).

LEAVE ORDER.
VECTOR BOOKS=#X1 TO #X22.

LOOP #I=1 TO 21 BY 2 IF NOT SYSMIS(BOOKS(#I)).
- COMPUTE ISBN=BOOKS(#I).
- COMPUTE QUANTITY=BOOKS(#I+1).
- END CASE.
END LOOP.
END INPUT PROGRAM.
BEGIN DATA
1045 182 2 155 1 134 1 153 5
1046 155 3 153 5 163 1
1047 161 5 182 2 163 4 186 6
1048 186 2
1049 155 2 163 2 153 2 074 1 161 1
END DATA.

SORT CASES ISBN.
DO IF $CASENUM EQ 1.
- PRINT EJECT /'Order ISBN Quantity'.
- PRINT SPACE.
END IF.
```

```
FORMATS ISBN (F3)/ QUANTITY (F2).
PRINT /' ' ORDER ' ' ISBN '  ' QUANTITY.

EXECUTE.
```

- Data are extracted from a file whose records store values for an invoice number and a series of book codes and quantities ordered. For example, invoice 1045 is for four different titles and a total of nine books: two copies of book 182, one copy each of 155 and 134, and five copies of book 153. The task is to break each individual book order into a record, preserving the order number on each new case.
- The input program encloses the commands that build cases from the input file. They are required because the END CASE command is used to create multiple cases from single input records.
- DATA LIST specifies *ORDER* as a permanent variable and defines 22 scratch variables to hold the book numbers and quantities (this is the maximum number of numbers and quantities that will fit in 72 columns). In the format specification, the first element skips one space after the value for the variable *ORDER*. The number 11 repeats the formats that follow it 11 times: once for each book number and quantity pair. The specified format is F3.0 for book numbers and F2.0 for quantities. The 1X specification skips one column after each quantity value.
- LEAVE preserves the value of the variable *ORDER* across the new cases to be generated.
- VECTOR sets up the vector *BOOKS* with the 22 scratch variables as its elements. The first element is *#X1*, the second is *#X2*, and so on.
- If the element for the vector *BOOKS* is not system-missing, LOOP initiates the loop structure that moves through the vector *BOOKS*, picking off the book numbers and quantities. The indexing clause initiates the indexing variable *#I* at 1, to be increased by 2 to a maximum of 21.
- The first COMPUTE command sets the variable *ISBN* equal to the element in the vector *BOOKS* indexed by *#I*, which is the current book number. The second COMPUTE sets the variable *QUANTITY* equal to the next element in the vector *BOOKS*, *#I* +1, which is the quantity associated with the book number in *BOOKS(#I)*.
- END CASE tells the program to write out a case with the current values of the three variables: *ORDER*, *ISBN*, and *QUANTITY*.
- END LOOP terminates the loop structure and control is returned to the LOOP command, where *#I* is increased by 2 and looping continues until the entire input case is read or until *#I* exceeds the maximum value of 21.
- SORT CASES sorts the new cases by book number.
- The DO IF structure encloses a PRINT EJECT command and a PRINT SPACE command to set up titles for the output.
- FORMATS establishes dictionary formats for the new variables *ISBN* and *QUANTITY*. PRINT displays the new cases.
- EXECUTE runs the commands. The output is shown below.

```
Order ISBN Quantity
 1049   74    1
 1045  134    1
 1045  153    5
 1046  153    5
 1049  153    2
 1045  155    1
 1046  155    3
 1049  155    2
 1047  161    5
 1049  161    1
 1046  163    1
 1047  163    4
 1049  163    2
 1045  182    2
 1047  182    2
 1047  186    6
 1048  186    2
```

Figure 57. PRINT output showing new cases

Create variable that approximates a log-normal distribution

```
SET FORMAT=F8.0.
```

```
INPUT PROGRAM.
LOOP I=1 TO 1000.
+ COMPUTE SCORE=EXP(NORMAL(1)).
+ END CASE.
END LOOP.
END FILE.
END INPUT PROGRAM.

FREQUENCIES VARIABLES=SCORE /FORMAT=NOTABLE /HISTOGRAM
 /PERCENTILES=1 10 20 30 40 50 60 70 80 90 99
 /STATISTICS=ALL.
```

- The input program creates 1,000 cases with a single variable *SCORE*. Values for *SCORE* approximate a log-normal distribution.

Restructure a data file to create a separate case for each individual

```
INPUT PROGRAM.
DATA LIST  /#RECS 1 HEAD1 HEAD2 3-4(A).    /*Read header info
LEAVE  HEAD1 HEAD2.

LOOP  #I=1 TO #RECS.
DATA LIST  /INDIV 1-2(1).                  /*Read individual info
PRINT  /#RECS HEAD1 HEAD2 INDIV.
END CASE.                                  /*Create combined case
END LOOP.
END INPUT PROGRAM.
BEGIN DATA
1 AC
91
2 CC
35
43
0 XX
1 BA
34
3 BB
42
96
37
END DATA.
LIST.
```

- Data are in a file with header records that indicate the type of record and the number of individual records that follow. The number of records following each header record varies. For example, the 1 in the first column of the first header record (AC) says that only one individual record (91) follows. The 2 in the first column of the second header record (CC) says that two individual records (35 and 43) follow. The next header record has no individual records, indicated by the 0 in column 1, and so on.
- The first DATA LIST reads the expected number of individual records for each header record into temporary variable *#RECS*. *#RECS* is then used as the terminal value in the indexing variable to read the correct number of individual records using the second DATA LIST.
- The variables *HEAD1* and *HEAD2* contain the information in columns 3 and 4, respectively, in the header records. The LEAVE command retains *HEAD1* and *HEAD2* so that this information can be spread to the individual records.
- The variable *INDIV* is the information from the individual record. *INDIV* is combined with *#RECS*, *HEAD1*, and *HEAD2* to create the new case. Notice in the output from the PRINT command below that no case is created for the header record with 0 for *#RECS*.
- END CASE passes each case out of the input program to the LIST command. Without END CASE, the PRINT command would still display the cases because it is inside the loop. However, only one (the last) case per header record would pass out of the input program. The outcome for LIST will be quite different.

```
1 A C 9.1
2 C C 3.5
2 C C 4.3
1 B A 3.4
3 B B 4.2
3 B B 9.6
3 B B 3.7
```

Figure 58. PRINT output

```
HEAD1 HEAD2 INDIV

A     C      9.1
C     C      3.5
C     C      4.3
B     A      3.4
B     B      4.2
B     B      9.6
B     B      3.7
```

Figure 59. LIST output when END CASE is specified

```
HEAD1 HEAD2 INDIV

A     C      9.1
C     C      4.3
X     X       .
B     A      3.4
B     B      3.7
```

Figure 60. LIST output when END CASE is not specified

END FILE

```
END FILE
```

Example

```
INPUT PROGRAM.
DATA LIST FILE=PRICES /YEAR 1-4 QUARTER 6 PRICE 8-12(2).
DO IF (YEAR GE 1881).  /*Stop reading before 1881
END FILE.
END IF.
END INPUT PROGRAM.
```

Overview

END FILE is used in an INPUT PROGRAM–END INPUT PROGRAM structure to tell the program to stop reading data before it actually encounters the end of the file. END FILE can be used with END CASE to concatenate raw data files by causing the program to delay end-of-file processing until it has read multiple data files. END FILE can also be used with LOOP and END CASE to generate data without any data input.

Basic Specification

The basic specification is simply END FILE. There are no additional specifications. The end of file is defined according to the conditions specified for END FILE in the input program.

Syntax Rules

- END FILE is available only within an INPUT PROGRAM structure.
- Only one END FILE command can be executed per input program. However, multiple END FILE commands can be specified within a conditional structure in the input program.

Operations

- When END FILE is encountered, the program stops reading data and puts an end of file in the active dataset it was building. The case that causes the execution of END FILE is not read. To include this case, use the END CASE command before END FILE (see the examples below).
- END FILE has the same effect as the end of the input data file. It terminates the input program (see INPUT PROGRAM–END INPUT PROGRAM).

Examples

Stop reading a file based on a data value

```
*Select cases.

INPUT PROGRAM.
DATA LIST FILE=PRICES /YEAR 1-4 QUARTER 6 PRICE 8-12(2).

DO IF (YEAR GE 1881).  /*Stop reading before 1881
END FILE.
END IF.

END INPUT PROGRAM.

LIST.
```

- This example assumes that data records are entered chronologically by year. The DO IF–END IF structure specifies an end of file when the first case with a value of 1881 or later for *YEAR* is reached.
- LIST executes the input program and lists cases in the active dataset. The case that causes the end of the file is not included in the active dataset.

- As an alternative to an input program with END FILE, you can use N OF CASES to select cases if you know the exact number of cases. Another alternative is to use SELECT IF to select cases before 1881, but then the program would unnecessarily read the entire input file.

END FILE with END CASE

```
*Select cases but retain the case that causes end-of-file processing.

INPUT PROGRAM.
DATA LIST FILE=PRICES /YEAR 1-4 QUARTER 6 PRICE 8-12(2).

DO IF (YEAR GE 1881).   /*Stop reading before 1881 (or at end of file)
END CASE.              /*Create case 1881
END FILE.

ELSE.
END CASE.              /*Create all other cases
END IF.
END INPUT PROGRAM.

LIST.
```

- The first END CASE command forces the program to retain the case that causes end-of-file processing.
- The second END CASE indicates the end of case for all other cases and passes them out of the input program one at a time. It is required because the first END CASE command causes the program to abandon default end-of-case processing (see END CASE).

ERASE

```
ERASE FILE='file'
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
ERASE FILE='PRSNL.DAT'.
```

Overview

ERASE removes a file from a disk.

Basic Specification

The basic specification is the keyword FILE followed by a file specification enclosed in quotes. The specified file is erased from the disk. The file specification may vary from operating system to operating system.

Syntax Rules

- The keyword FILE is required, but the equals sign is optional.
- ERASE allows one file specification only and does not accept wildcard characters. To erase more than one file, specify multiple ERASE commands.
- The file to be erased must be specified in full. ERASE does not recognize any default file extension.

Operations

ERASE deletes the specified file regardless of its type. No message is displayed unless the command cannot be executed. Use ERASE with caution.

Examples

```
ERASE FILE '/data/textdata.txt'.
```

EXAMINE

EXAMINE provides stem-and-leaf plots, histograms, boxplots, normal plots, robust estimates of location, tests of normality, and other descriptive statistics. Separate analyses can be obtained for subgroups of cases.

```
EXAMINE VARIABLES=varlist [[BY varlist] [varname BY varname]]

 [/COMPARE={GROUPS** }]
           {VARIABLES}

 [/{TOTAL**}]
   {NOTOTAL}

 [/ID={case number**}]
      {varname      }

 [/PERCENTILES [{NONE                    }]]
               {({5,10,25,50,75,90,95})}=[{HAVERAGE  }]
                 {value list         }    {WAVERAGE  }
                                          {ROUND     }
                                          {AEMPIRICAL}
                                          {EMPIRICAL }

 [/PLOT=[STEMLEAF**] [BOXPLOT**] [NPPLOT] [SPREADLEVEL(n)] [HISTOGRAM]]

        [{ALL   }]
         {NONE}

 [/STATISTICS=[DESCRIPTIVES**] [EXTREME({5})]]
                                        {n}
              [{ALL }]
               {NONE}

 [/CINTERVAL {95**}]
             {n   }

 [/MESTIMATOR=[{NONE**}]]
               {ALL   }

              [HUBER({1.339})] [ANDREW({1.34}]
                     {c    }           {c    }

              [HAMPEL({1.7,3.4,8.5})]
                      {a  ,b  ,c  }

              [TUKEY({4.685})]
                     {c    }

[/MISSING=[{LISTWISE**}] [{EXCLUDE**}] [{NOREPORT**}]]
          {PAIRWISE  }   {INCLUDE  }   {REPORT    }
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Examples

```
EXAMINE VARIABLES=ENGSIZE,COST.

EXAMINE VARIABLES=MIPERGAL BY MODEL, MODEL BY CYLINDERS.
```

Overview

EXAMINE provides stem-and-leaf plots, histograms, boxplots, normal plots, robust estimates of location, tests of normality, and other descriptive statistics. Separate analyses can be obtained for subgroups of cases.

Options

Cells. You can subdivide cases into cells based on their values for grouping (factor) variables using the BY keyword on the VARIABLES subcommand.

Output. You can control the display of output using the COMPARE subcommand. You can specify the computational method and break points for percentiles with the PERCENTILES subcommand, and you can assign a variable to be used for labeling outliers on the ID subcommand.

Plots. You can request stem-and-leaf plots, histograms, vertical boxplots, spread-versus-level plots with Levene tests for homogeneity of variance, and normal and detrended probability plots with tests for normality. These plots are available through the PLOT subcommand.

Statistics. You can request univariate statistical output with the STATISTICS subcommand and maximum-likelihood estimators with the MESTIMATORS subcommand.

Basic Specification

- The basic specification is VARIABLES and at least one dependent variable.
- The default output includes a Descriptives table displaying univariate statistics (mean, median, standard deviation, standard error, variance, kurtosis, kurtosis standard error, skewness, skewness standard error, sum, interquartile range (IQR), range, minimum, maximum, and 5% trimmed mean), a vertical boxplot, and a stem-and-leaf plot. Outliers are labeled on the boxplot with the system variable *$CASENUM*.

Subcommand Order

Subcommands can be named in any order.

Limitations

- When string variables are used as factors, only the first eight characters are used to form cells. String variables cannot be specified as dependent variables.
- When more than eight crossed factors (for example, `A, B, ...` in the specification `Y by A by B by ...`) are specified, the command is not executed.

Examples

Univariate statistics

```
EXAMINE VARIABLES=ENGSIZE,COST.
```

- *ENGSIZE* and *COST* are the dependent variables.
- EXAMINE produces univariate statistics for *ENGSIZE* and *COST* in the Descriptives table and a vertical boxplot and a stem-and-leaf plot for each variable.

Descriptive statistics across groups

```
EXAMINE  VARIABLES=toxin BY yield
  /PLOT BOXPLOT STEMLEAF
  /COMPARE GROUP
  /STATISTICS DESCRIPTIVES
  /CINTERVAL 95
  /MISSING LISTWISE
  /NOTOTAL.
```

- The procedure examines values of the variable *toxin* by values of the variable *yield*.
- All other options are set to their default values.

Exploring distributions

```
EXAMINE  VARIABLES=temp BY batch /ID= labrunid
  /PLOT BOXPLOT STEMLEAF NPPLOT
  /COMPARE GROUP
  /MESTIMATORS  HUBER(1.339) ANDREW(1.34) HAMPEL(1.7,3.4,8.5) TUKEY(4.685)
  /STATISTICS DESCRIPTIVES EXTREME
  /CINTERVAL 95
  /MISSING LISTWISE
  /NOTOTAL.
```

- The procedure examines values of *temp* by values of *batch*.
- The ID subcommand specifies that cases in the output should be labeled by values of *labrunid*.
- The PLOT subcommand requests boxplots, stem-and-leaf plots, and normality plots.
- The MESTIMATORS subcommand requests Huber's, Andrews', Hampel's, and Tukey's robust estimators of location with default weights.

- The STATISTICS subcommand requests a descriptive statistics table and an extreme values table with the 5 largest and 5 smallest values.
- All other options are set to their default values.

VARIABLES Subcommand

VARIABLES specifies the dependent variables and the cells. The dependent variables are specified first, followed by the keyword BY and the variables that define the cells. Repeated models on the same EXAMINE are discarded.

- To create cells defined by the combination of values of two or more factors, specify the factor names separated by the keyword BY.

Caution. Large amounts of output can be produced if many cells are specified. If there are many factors or if the factors have many values, EXAMINE will produce a large number of separate analyses.

Example

```
EXAMINE VARIABLES=SALARY,YRSEDUC BY RACE,SEX,DEPT,RACE BY SEX.
```

- *SALARY* and *YRSEDUC* are dependent variables.
- Cells are formed first for the values of *SALARY* and *YRSEDUC* individually, and then each by values for *RACE*, *SEX, DEPT,* and the combination of *RACE* and *SEX*.
- By default, EXAMINE produces Descriptives tables, stem-and-leaf plots, and boxplots.

COMPARE Subcommand

COMPARE controls how boxplots are displayed. This subcommand is most useful if there is more than one dependent variable and at least one factor in the design.

GROUPS. *For each dependent variable, boxplots for all cells are displayed together.* With this display, comparisons across cells for a single dependent variable are easily made. This is the default.

VARIABLES. *For each cell, boxplots for all dependent variables are displayed together.* With this display, comparisons of several dependent variables are easily made. This is useful in situations where the dependent variables are repeated measures of the same variable (see the following example) or have similar scales, or when the dependent variable has very different values for different cells, and plotting all cells on the same scale would cause information to be lost.

Example

```
EXAMINE VARIABLES=GPA1 GPA2 GPA3 GPA4 BY MAJOR   /COMPARE=VARIABLES.
```

- The four GPA variables are summarized for each value of *MAJOR*.
- COMPARE=VARIABLES groups the boxplots for the four GPA variables together for each value of *MAJOR*.

Example

```
EXAMINE VARIABLES=GPA1 GPA2 GPA3 GPA4 BY MAJOR /COMPARE=GROUPS.
```

- COMPARE=GROUPS groups the boxplots for *GPA1* for all majors together, followed by boxplots for *GPA2* for all majors, and so on.

TOTAL and NOTOTAL Subcommands

TOTAL and NOTOTAL control the amount of output produced by EXAMINE when factor variables are specified.

- TOTAL is the default. By default, or when TOTAL is specified, EXAMINE produces statistics and plots for each dependent variable overall and for each cell specified by the factor variables.
- NOTOTAL suppresses overall statistics and plots.
- TOTAL and NOTOTAL are alternatives.
- NOTOTAL is ignored when the VARIABLES subcommand does not specify factor variables.

ID Subcommand

ID assigns a variable from the active dataset to identify the cases in the output. By default the case number is used for labeling outliers and extreme cases in boxplots.

- The identification variable can be either string or numeric. If it is numeric, value labels are used to label cases. If no value labels exist, the values are used.
- Only one identification variable can be specified.

Example

```
EXAMINE VARIABLES=SALARY BY RACE BY SEX /ID=LASTNAME.
```

- ID displays the value of *LASTNAME* for outliers and extreme cases in the boxplots.

PERCENTILES Subcommand

PERCENTILES displays the Percentiles table. If PERCENTILES is omitted, no percentiles are produced. If PERCENTILES is specified without keywords, HAVERAGE is used with default break points of 5, 10, 25, 50, 75, 90, and 95.

- Values for break points are specified in parentheses following the subcommand. EXAMINE displays up to six decimal places for user-specified percentile values.
- The method keywords follow the specifications for break points.

For each of the following methods of percentile calculation, *w* is the sum of the weights for all nonmissing cases, *p* is the specified percentile divided by 100, and X_i is the value of the *i*th case (cases are assumed to be ranked in ascending order). For details on the specific formulas used, see the algorithms documentation included on the installation CD.

HAVERAGE. Weighted average at $X_{(w+1)p}$. The percentile value is the weighted average of X_i and X_{i+1}, where *i* is the integer part of $(w + 1)p$. This is the default if PERCENTILES is specified without a keyword.

WAVERAGE. Weighted average at *Xwp*. The percentile value is the weighted average of X_i and $X_{(i+1)}$, where *i* is the integer portion of *wp*.

ROUND. *Observation closest to* wp. The percentile value is X_i or X_{i+1}, depending upon whether *i* or *i + 1* is "closer" to *wp*.

EMPIRICAL. *Empirical distribution function.* The percentile value is X_i, where *i* is equal to *wp* rounded up to the next integer.

AEMPIRICAL. *Empirical distribution with averaging.* This is equivalent to EMPIRICAL, except when *i=wp,* in which case the percentile value is the average of X_i and X_{i+1}.

NONE. *Suppress percentile output.* This is the default if PERCENTILES is omitted.

Example

```
EXAMINE VARIABLE=SALARY /PERCENTILES(10,50,90)=EMPIRICAL.
```

- PERCENTILES produces the 10th, 50th, and 90th percentiles for the dependent variable *SALARY* using the EMPIRICAL distribution function.

PLOT Subcommand

PLOT controls plot output. The default is a vertical boxplot and a stem-and-leaf plot for each dependent variable for each cell in the model.

- Spread-versus-level plots can be produced only if there is at least one factor variable on the VARIABLES subcommand. If you request a spread-versus-level plot and there are no factor variables, the program issues a warning and no spread-versus-level plot is produced.
- If you specify the PLOT subcommand, only those plots explicitly requested are produced.

BOXPLOT. *Vertical boxplot.* The boundaries of the box are Tukey's hinges. The median is identified by a line inside the box. The length of the box is the interquartile range (IQR) computed from Tukey's hinges. Values more than three IQR's from the end of a box are labeled as extreme, denoted with an asterisk (*). Values more than 1.5 IQR's but less than 3 IQR's from the end of the box are labeled as outliers (o).

STEMLEAF. *Stem-and-leaf plot.* In a stem-and-leaf plot, each observed value is divided into two components—leading digits (stem) and trailing digits (leaf).

HISTOGRAM. *Histogram.*

SPREADLEVEL(n). *Spread-versus-level plot with the Test of Homogeneity of Variance table.* If the keyword appears alone, the natural logs of the interquartile ranges are plotted against the natural logs of the medians for all cells. If a power for transforming the data (*n*) is given, the IQR and median of the transformed data are plotted. If 0 is specified for *n*, a natural log transformation of the data is done. The slope of the regression line and Levene tests for homogeneity of variance are also displayed. The Levene tests are based on the original data if no transformation is specified and on the transformed data if a transformation is requested.

NPPLOT. *Normal and detrended Q-Q plots with the Tests of Normality table presenting Shapiro-Wilk's statistic and a Kolmogorov-Smirnov statistic with a Lilliefors significance level for testing normality.* If non-integer weights are specified, the Shapiro-Wilk's statistic is calculated when the weighted sample size lies between 3 and 50. For no weights or integer weights, the statistic is calculated when the weighted sample size lies between 3 and 5,000.

ALL. *All available plots.*

NONE. *No plots.*

Example

```
EXAMINE VARIABLES=CYCLE BY TREATMNT /PLOT=NPPLOT.
```

- PLOT produces normal and detrended Q-Q plots for each value of *TREATMNT* and a Tests of Normality table.

Example

```
EXAMINE VARIABLES=CYCLE BY TREATMNT /PLOT=SPREADLEVEL(.5).
```

- PLOT produces a spread-versus-level plot of the medians and interquartile ranges of the square root of *CYCLE*. Each point on the plot represents one of the *TREATMNT* groups.
- A Test of Homogeneity of Variance table displays Levene statistics.

Example

```
EXAMINE VARIABLES=CYCLE BY TREATMNT /PLOT=SPREADLEVEL(0).
```

- PLOT generates a spread-versus-level plot of the medians and interquartile ranges of the natural logs of *CYCLE* for each *TREATMENT* group.
- A Test of Homogeneity of Variance table displays Levene statistics.

Example

```
EXAMINE VARIABLES=CYCLE BY TREATMNT /PLOT=SPREADLEVEL.
```

- PLOT generates a spread-versus-level plot of the natural logs of the medians and interquartile ranges of *CYCLE* for each *TREATMNT* group.
- A Test of Homogeneity of Variance table displays Levene statistics.

STATISTICS Subcommand

STATISTICS requests univariate statistics and determines how many extreme values are displayed. DESCRIPTIVES is the default. If you specify keywords on STATISTICS, only the requested statistics are displayed.

DESCRIPTIVES. *Display the Descriptives table showing univariate statistics (the mean, median, 5% trimmed mean, standard error, variance, standard deviation, minimum, maximum, range, interquartile range, skewness, skewness standard error, kurtosis, and kurtosis standard error)*. This is the default.

EXTREME(n). *Display the Extreme Values table presenting cases with the* n *largest and* n *smallest values.* When there is a **BY** specification for **VARIABLES**, the *n* applies to each category of the **BY** variable(s) and shows two tables. The first table lists the extremes without the **BY**, and the second uses the **BY**. When multiple **BY** variables are present, each variable is represented in its own table (rather than one combined table). When *n* is omitted, the five largest and five smallest values are displayed. Extreme cases are labeled with their values for the identification variable if the ID subcommand is used or with their values for the system variable *$CASENUM* if ID is not specified.

ALL. *Display the Descriptives and Extreme Values tables.*

NONE. *Display neither the Descriptives nor the Extreme Values tables.*

Example

```
EXAMINE VARIABLE=FAILTIME /ID=BRAND
  /STATISTICS=EXTREME(10) /PLOT=NONE.
```

- STATISTICS identifies the cases with the 10 lowest and 10 highest values for *FAILTIME*. These cases are labeled with the first characters of their values for the variable *BRAND*. The Descriptives table is not displayed.

CINTERVAL Subcommand

CINTERVAL controls the confidence level when the default DESCRIPTIVES statistics is displayed. CINTERVAL has a default value of 95.

- You can specify a CINTERVAL value (*n)* between 50 and 99.99 inclusive. If the value you specify is out of range, the program issues a warning and uses the default 95% intervals.
- If you specify a keyword on STATISTICS subcommand that turns off the default DESCRIPTIVES, the CINTERVAL subcommand is ignored.
- The confidence interval appears in the output with the label *n% CI for Mean*, followed by the confidence interval in parentheses. For example, `95% CI for Mean (.0001,.00013)`

The *n* in the label shows up to six decimal places. That is, input `/CINTERVAL 95` displays as *95% CI* while input `/CINTERVAL 95.975` displays as *95.975% CI*.

MESTIMATORS Subcommand

M-estimators are robust maximum-likelihood estimators of location. Four M-estimators are available for display in the M-Estimators table. They differ in the weights they apply to the cases. MESTIMATORS with

no keywords produces Huber's M-estimator with *c*=1.339; Andrews' wave with *c*=1.34π; Hampel's M-estimator with *a*=1.7, *b*=3.4, and *c*=8.5; and Tukey's biweight with *c*=4.685.

HUBER(c). *Huber's M-estimator.* The value of weighting constant *c* can be specified in parentheses following the keyword. The default is *c*=1.339.

ANDREW(c). *Andrews' wave estimator.* The value of weighting constant *c* can be specified in parentheses following the keyword. Constants are multiplied by π. The default is 1.34π.

HAMPEL(a,b,c). *Hampel's M-estimator.* The values of weighting constants *a*, *b*, and *c* can be specified in order in parentheses following the keyword. The default values are *a*=1.7, *b*=3.4, and *c*=8.5.

TUKEY(c). *Tukey's biweight estimator.* The value of weighting constant *c* can be specified in parentheses following the keyword. The default is *c*=4.685.

ALL. *All four above M-estimators.* This is the default when MESTIMATORS is specified with no keyword. The default values for weighting constants are used.

NONE. *No M-estimators.* This is the default if MESTIMATORS is omitted.

Example

```
EXAMINE VARIABLE=CASTTEST /MESTIMATORS.
```

- MESTIMATORS generates all four M-estimators computed with the default constants.

Example

```
EXAMINE VARIABLE=CASTTEST /MESTIMATORS=HAMPELS(2,4,8).
```

- MESTIMATOR produces Hampel's M-estimator with weighting constants *a*=2, *b*=4, and *c*=8.

MISSING Subcommand

MISSING controls the processing of missing values in the analysis. The default is LISTWISE, EXCLUDE, and NOREPORT.

- LISTWISE and PAIRWISE are alternatives and apply to all variables. They are modified for dependent variables by INCLUDE/EXCLUDE and for factor variables by REPORT/NOREPORT.
- INCLUDE and EXCLUDE are alternatives; they apply only to dependent variables.
- REPORT and NOREPORT are alternatives; they determine if missing values for factor variables are treated as valid categories.

LISTWISE . *Delete cases with missing values listwise.* A case with missing values for any dependent variable or any factor in the model specification is excluded from statistics and plots unless modified by INCLUDE or REPORT. This is the default.

PAIRWISE. *Delete cases with missing values pairwise.* A case is deleted from the analysis only if it has a missing value for the dependent variable or factor being analyzed.

EXCLUDE. *Exclude user-missing values.* User-missing values and system-missing values for dependent variables are excluded. This is the default.

INCLUDE. *Include user-missing values.* Only system-missing values for dependent variables are excluded from the analysis.

NOREPORT. *Exclude user- and system-missing values for factor variables.* This is the default.

REPORT . *Include user- and system-missing values for factor variables.* User- and system-missing values for factors are treated as valid categories and are labeled as missing.

Example

```
EXAMINE VARIABLES=RAINFALL MEANTEMP BY REGION.
```

- MISSING is not specified and the default is used. Any case with a user- or system-missing value for *RAINFALL*, *MEANTEMP*, or *REGION* is excluded from the analysis and display.

Example

```
EXAMINE VARIABLES=RAINFALL MEANTEMP BY REGION
  /MISSING=PAIRWISE.
```

- Only cases with missing values for *RAINFALL* are excluded from the analysis of *RAINFALL*, and only cases with missing values for *MEANTEMP* are excluded from the analysis of *MEANTEMP*. Missing values for *REGION* are not used.

Example

```
EXAMINE VARIABLES=RAINFALL MEANTEMP BY REGION
  /MISSING=REPORT.
```

- Missing values for *REGION* are considered valid categories and are labeled as missing.

References

Hoaglin, D. C., F. Mosteller, and J. W. Tukey. 1983. *Understanding robust and exploratory data analysis*. New York: John Wiley and Sons.

Hoaglin, D. C., F. Mosteller, and J. W. Tukey. 1985. *Exploring data tables, trends, and shapes*. New York: John Wiley and Sons.

Tukey, J. W. 1977. *Exploratory data analysis*. Reading, MA: Addison-Wesley.

Velleman, P. F., and D. C. Hoaglin. 1981. *Applications, basics, and computing of exploratory data analysis*. Boston, Mass.: Duxbury Press.

EXECUTE

```
EXECUTE.
```

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Overview

EXECUTE forces the data to be read and executes the transformations that precede it in the command sequence.

Basic Specification

The basic specification is simply the command keyword. EXECUTE has no additional specifications.

Operations

- EXECUTE causes the data to be read but has no other influence on the session.
- EXECUTE is designed for use with transformation commands and facilities such as ADD FILES, MATCH FILES, UPDATE, PRINT, and WRITE, which do not read data and are not executed unless followed by a data-reading procedure.

Examples

```
DATA LIST  FILE=RAWDATA / 1 LNAME 1-13 (A) FNAME 15-24 (A)
  MMAIDENL 40-55.
VAR LABELS  MMAIDENL 'MOTHER''S MAIDEN NAME'.
DO IF (MMAIDENL EQ 'Smith').
WRITE OUTFILE=SMITHS/LNAME FNAME.
END IF.
EXECUTE.
```

- This example writes the last and first names of all people whose mother’s maiden name was Smith to the data file *SMITHS*.
- DO IF-END IF and WRITE do not read data and are executed only when data are read for a procedure. Because there is no procedure in this session, EXECUTE is used to read the data and execute all of the preceding transformation commands. Otherwise, the commands would not be executed.

EXPORT

```
EXPORT OUTFILE='file'
 [/TYPE={COMM**}]
        {TAPE  }
 [/UNSELECTED=[{RETAIN}]
              {DELETE}
 [/KEEP={ALL**  }] [/DROP=varlist]
        {varlist}
 [/RENAME=(old varnames=new varnames)...]
 [/MAP]
 [/DIGITS=n]
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
EXPORT OUTFILE="/data/newdata.por"
  /RENAME=(V1 TO V3=ID, SEX, AGE) /MAP.
```

Overview

The EXPORT command is deprecated and will not be supported in future releases. The EXPORT command is not supported in Unicode mode.

EXPORT produces a portable data file. A portable data file is a data file created used to transport data between different types of computers and operating systems, or other software using the same portable file format. A portable file contains all of the data and dictionary information stored in the active dataset from which it was created.

EXPORT is similar to the SAVE command. It can occur in the same position in the command sequence as the SAVE command and saves the active dataset. The file includes the results of all permanent transformations and any temporary transformations made just prior to the EXPORT command. The active dataset is unchanged after the EXPORT command.

- In most cases, saving data in portable format is no longer necessary, since IBM SPSS Statistics data files should be platform/operating system independent.
- To export data in external data formats (e.g., Excel, SAS, Stata, CSV, tab-delimited), use "SAVE TRANSLATE" on page 1747.

Options

Format. You can control the format of the portable file using the TYPE subcommand.

Variables. You can save a subset of variables from the active dataset and rename the variables using the DROP, KEEP, and RENAME subcommands. You can also produce a record of all variables and their names on the exported file with the MAP subcommand.

Precision. You can specify the number of decimal digits of precision for the values of all numeric variables on the DIGITS subcommand.

Basic Specification

The basic specification is the OUTFILE subcommand with a file specification. All variables from the active dataset are written to the portable file, with variable names, variable and value labels, missing-value flags, and print and write formats.

Subcommand Order

Subcommands can be named in any order.

Operations

- Portable files are written with 80-character record lengths.
- Portable files may contain some unprintable characters.
- The active dataset is still available for transformations and procedures after the portable file is created.
- The system variables *$CASENUM* and *$DATE* are assigned when the file is read by IMPORT.
- If the WEIGHT command is used before EXPORT, the weighting variable is included in the portable file.
- Variable names that exceed eight bytes are converted to unique eight-byte names—for example, *mylongrootname1*, *mylongrootname2*, and *mylongrootname3* would be converted to *mylongro*, *mylong_2*, and *mylong_3*, respectively.

Limitations

The EXPORT command is not supported in Unicode mode. For more information, see SET command, UNICODE subcommand.

Examples

```
EXPORT OUTFILE="/newdata.por"
  /RENAME=(V1 TO V3=ID,SEX,AGE) /MAP.
```

- The portable file is written to *newdata.por*.
- The variables *V1*, *V2*, and *V3* are renamed *ID*, *SEX*, and *AGE* in the portable file. Their names remain *V1*, *V2*, and *V3* in the active dataset. None of the other variables written to the portable file are renamed.
- MAP requests a display of the variables in the portable file.

Methods of Transporting Portable Files

Portable files can be transported on magnetic tape or by a communications program.

Magnetic Tape

Before transporting files on a magnetic tape, make sure the receiving computer can read the tape being sent. The following tape specifications must be known before you write the portable file on the tape:

- **Number of tracks.** Either 7 or 9.
- **Tape density.** 200, 556, 800, 1600, or 6250 bits per inch (BPI).
- **Parity.** Even or odd. This must be known only when writing a 7-track tape.
- **Tape labeling.** Labeled or unlabeled. Check whether the site can use tape labels. Also make sure that the site has the ability to read multivolume tape files if the file being written uses more than one tape.
- **Blocksize.** The maximum blocksize the receiving computer can accept.

A tape written with the following characteristics can be read by most computers: 9 track, 1600 BPI, unlabeled, and a blocksize of 3200 characters. However, there is no guarantee that a tape written with these characteristics can be read successfully. The best policy is to know the requirements of the receiving computer ahead of time.

The following advice may help ensure successful file transfers by magnetic tape:

- Unless you are certain that the receiving computer can read labels, prepare an unlabeled tape.
- Make sure the record length of 80 is not changed.
- Do not use a separate character translation program, especially ASCII/EBCDIC translations. EXPORT/IMPORT takes care of this for you.

- Make sure the same blocking factor is used when writing and reading the tape. A blocksize of 3200 is frequently a good choice.
- If possible, write the portable file directly to tape to avoid possible interference from copy programs. Read the file directly from the tape for the same reason.
- Use the INFO LOCAL command to find out about using the program on your particular computer and operating system. INFO LOCAL generally includes additional information about reading and writing portable files.

Communications Programs

Transmission of a portable file by a communications program may not be possible if the program misinterprets any characters in the file as control characters (for example, as a line feed, carriage return, or end of transmission). This can be prevented by specifying TYPE=COMM on EXPORT. This specification replaces each control character with the character 0. The affected control characters are in positions 0–60 of the IMPORT/EXPORT character set.

The line length that the communications program uses must be set to 80 to match the 80-character record length of portable files. A transmitted file must be checked for blank lines or special characters inserted by the communications program. These must be edited out prior to reading the file with the IMPORT command.

Character Translation

Portable files are character files, not binary files, and they have 80-character records so they can be transmitted over data links. A receiving computer may not use the same character set as the computer where the portable file was written. When it imports a portable file, the program translates characters in the file to the character set used by the receiving computer. Depending on the character set in use, some characters in labels and in string data may be lost in the translation. For example, if a file is transported from a computer using a seven-bit ASCII character set to a computer using a six-bit ASCII character set, some characters in the file may have no matching characters in six-bit ASCII. For a character that has no match, the program generates an appropriate nonprintable character (the null character in most cases).

For a table of the character-set translations available with IMPORT and EXPORT, refer to Appendix B. A blank in a column of the table means that there is no matching character for that character set and an appropriate nonprintable character will be generated when you import a file.

OUTFILE Subcommand

OUTFILE specifies the portable file. OUTFILE is the only required subcommand on EXPORT.

TYPE Subcommand

TYPE indicates whether the portable file should be formatted for magnetic tape or for a communications program. You can specify either COMM or TAPE. See the topic “Methods of Transporting Portable Files ” on page 650 for more information.

COMM. *Transport portable files by a communications program.* When COMM is specified on TYPE, the program removes all control characters and replaces them with the character 0. This is the default.

TAPE. *Transport portable files on magnetic tape.*

Example

```
EXPORT TYPE=TAPE /OUTFILE=HUBOUT.
```

- File *HUBOUT* is saved as a tape-formatted portable file.

UNSELECTED Subcommand

UNSELECTED determines whether cases excluded on a previous FILTER or USE command are to be retained or deleted in the data file. The default is RETAIN. The UNSELECTED subcommand has no effect when the active dataset does not contain unselected cases.

RETAIN. *Retain the unselected cases*. All cases in the active dataset are saved. This is the default when UNSELECTED is specified by itself.

DELETE. *Delete the unselected cases*. Only cases that meet the FILTER or USE criteria are saved in the data file.

DROP and KEEP Subcommands

DROP and KEEP save a subset of variables in the portable file.

- DROP excludes a variable or list of variables from the portable file. All variables not named are included in the portable file.
- KEEP includes a variable or list of variables in the portable file. All variables not named are excluded.
- Variables can be specified on DROP and KEEP in any order. With the DROP subcommand, the order of variables in the portable file is the same as their order in the active dataset. With the KEEP subcommand, the order of variables in the portable file is the order in which they are named on KEEP. Thus, KEEP can be used to reorder variables in the portable file.
- Both DROP and KEEP can be used on the same EXPORT command; the effect is cumulative. If you specify a variable already named on a previous DROP or one not named on a previous KEEP, the variable is considered nonexistent and the program displays an error message. The command is aborted and no portable file is saved.

Example

```
EXPORT OUTFILE=NEWSUM /DROP=DEPT TO DIVISION.
```

- The portable file is written to file *NEWSUM*. Variables between and including *DEPT* and *DIVISION* in the active dataset are excluded from the portable file.
- All other variables are saved in the portable file.

RENAME Subcommand

RENAME renames variables being written to the portable file. The renamed variables retain their original variable and value labels, missing-value flags, and print formats. The names of the variables are not changed in the active dataset.

- To rename a variable, specify the name of the variable in the active dataset, an equals sign, and the new name.
- A variable list can be specified on both sides of the equals sign. The number of variables on both sides must be the same, and the entire specification must be enclosed in parentheses.
- The keyword TO can be used for both variable lists.
- If you specify a renamed variable on a subsequent DROP or KEEP subcommand, the new variable name must be used.

Example

```
EXPORT OUTFILE=NEWSUM /DROP=DEPT TO DIVISION
  /RENAME=(NAME,WAGE=LNAME,SALARY).
```

- RENAME renames *NAME* and *WAGE* to *LNAME* and *SALARY*.

- *LNAME* and *SALARY* retain the variable and value labels, missing-value flags, and print formats assigned to *NAME* and *WAGE*.

MAP Subcommand

MAP displays any changes that have been specified by the RENAME, DROP, or KEEP subcommands.

- MAP can be specified as often as desired.
- Each MAP subcommand maps the results of subcommands that precede it; results of subcommands that follow it are not mapped. When MAP is specified last, it also produces a description of the portable file.

Example

```
EXPORT OUTFILE=NEWSUM /DROP=DEPT TO DIVISION /MAP
  /RENAME NAME=LNAME WAGE=SALARY /MAP.
```

- The first MAP subcommand produces a listing of the variables in the file after DROP has dropped the specified variables.
- RENAME renames *NAME* and *WAGE*.
- The second MAP subcommand shows the variables in the file after renaming. Since this is the last subcommand, the listing will show the variables as they are written in the portable file.

DIGITS Subcommand

DIGITS specifies the degree of precision for all noninteger numeric values written to the portable file.

- DIGITS has the general form DIGITS=*n*, where *n* is the number of digits of precision.
- DIGITS applies to all numbers for which rounding is required.
- Different degrees of precision *cannot* be specified for different variables. Thus, DIGITS should be set according to the requirements of the variable that needs the most precision.
- Default precision methods used by EXPORT work perfectly for integers that are not too large and for fractions whose denominators are products of 2, 3, and 5 (all decimals, quarters, eighths, sixteenths, thirds, thirtieths, sixtieths, and so forth.) For other fractions and for integers too large to be represented exactly in the active dataset (usually more than 9 digits, often 15 or more), the representation used in the active dataset contains some error already, so no exact way of sending these numbers is possible. The program sends enough digits to get very close. The number of digits sent in these cases depends on the originating computer: on mainframe IBM versions of the program, it is the equivalent of 13 decimal digits (integer and fractional parts combined). If many numbers on a file require this level of precision, the file can grow quite large. If you do not need the full default precision, you can save some space in the portable file by using the DIGITS subcommand.

Example

```
EXPORT OUTFILE=NEWSUM /DROP=DEPT TO DIVISION /MAP /DIGITS=4.
```

- DIGITS guarantees the accuracy of values to four significant digits. For example, 12.34567890876 will be rounded to 12.35.

EXSMOOTH

```
EXSMOOTH  [VARIABLES=] series names

 [/MODEL={NN** or SINGLE }]
         {NA             }
         {NM             }

         {LN or HOLT     }
         {LA             }
         {LM or WINTERS  }

         {EN             }
         {EA             }
         {EM             }

         {DN             }
         {DA             }
         {DM             }

 [/PERIOD=n]

 [/SEASFACT={(value list)}]
            {varname     }

 [/ALPHA={0.1**                          }]
         {value                          }
         {GRID ({0,1,0.1              })}
                {start, end, increment}

 [/GAMMA={0.1**                          }]
         {value                          }
         {GRID ({0,1,0.2              })}
                {start, end, increment}

 [/DELTA={0.1**                          }]
         {value                          }
         {GRID ({0,1,0.2              })}
                {start, end, increment}

 [/PHI={0.1**                          }]
       {value                          }
       {GRID ({0.1,0.9,0.2          })}
              {start, end, increment}

 [/INITIAL={CALCULATE**                }]
           {(start value, trend value)}

 [/APPLY['model name']]
```

**Default if the subcommand is omitted.

Example

```
EXSMOOTH VAR2
  /MODEL=LN.
```

Overview

EXSMOOTH produces fit/forecast values and residuals for one or more time series. A variety of models differing in trend (none, linear, or exponential) and seasonality (none, additive, or multiplicative) are available [22].

Options

Model Specification. You can specify a model with any combination of trend and seasonality components using the MODEL subcommand. For seasonal models, you can specify the periodicity using the PERIOD subcommand.

Parameter Specification. You can specify values for the smoothing parameters using the ALPHA, GAMMA, DELTA, and PHI subcommands. You can also specify initial values using the subcommand INITIAL and seasonal factor estimates using the subcommand SEASFACT.

Statistical Output. To get a list of all the SSE's and parameters instead of just the 10 smallest, specify TSET PRINT=DETAILED prior to EXSMOOTH.

[22] Gardner, E. S. 1985. Exponential smoothing: The state of the art. *Journal of Forecasting,* 4, 1-28.

New Variables. Because of the number of parameter and value combinations available, EXSMOOTH can create many new variables (up to the maximum specified on the TSET MXNEWVARS command). To evaluate the sum of squared errors without creating and saving new variables to the active dataset, use TSET NEWVAR=NONE prior to EXSMOOTH. To add new variables without erasing the values of previous Forecasting-generated variables, specify TSET NEWVAR=ALL. This saves all new variables generated during the current session to the active dataset.

Forecasting. When used with the PREDICT command, EXSMOOTH can produce forecasts beyond the end of the series (see PREDICT for more information).

Basic Specification

The basic specification is one or more series names.

- If a model is not specified, the NN (no trend and nonseasonal) model is used. The default value for each of the smoothing parameters is 0.1.
- Unless the default on the TSET NEWVAR is changed prior to the EXSMOOTH procedure, for each combination of smoothing parameters and series specified, EXSMOOTH creates two variables: *FIT#n* to contain the predicted values and *ERR#n* to contain residuals. These variables are automatically labeled and added to the active dataset.
- The output displays the initial values used in the analysis [23], the error degrees of freedom (DFE), and an ascending list of the smallest sum of squared errors (SSE) next to the associated set of smoothing parameters, up to a maximum of 10. For seasonal series, initial seasonal factor estimates are also displayed.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- VARIABLES can be specified only once.
- Other subcommands can be specified more than once, but only the last specification of each one is executed.
- The value list for subcommand SEASFACT and the grid values for the smoothing parameters must be enclosed within parentheses.

Operations

- If a smoothing parameter is specified for an inappropriate model, it is ignored. See the topic "Smoothing Parameter Subcommands " on page 658 for more information.
- EXSMOOTH cannot process series with missing observations. (You can use the RMV command to replace missing values, and USE to ignore missing observations at the beginning or end of a series. See RMV and USE for more information.)
- When EXSMOOTH is used with PREDICT, error series are assigned the system-missing value in the entire PREDICT range. The original series is system-missing beyond the last original case if the series is extended. (See PREDICT for more information.)

Limitations

- Maximum 1 VARIABLES subcommand. There is no limit on the number of series named on the list.
- Maximum 1 model keyword on the MODEL subcommand.

VARIABLES Subcommand

VARIABLES specifies the series names and is the only required subcommand. The actual keyword VARIABLES can be omitted.

[23] Ledolter, J., and B. Abraham. 1984. Some comments on the initialization of exponential smoothing. *Journal of Forecasting,* 3, 79-84.

- For seasonal models, the series must contain at least four full seasons of data.

MODEL Subcommand

MODEL specifies the type of model to be used.

- The only specification on MODEL is a model keyword.
- Only one model keyword can be specified. If more than one is specified, only the first is used.

The following models are available.

No trend models:

NN. *No trend and no seasonality.* This is the default model. The keyword SINGLE is an alias for NN.

NA. *No trend and an additive seasonal component.*

NM. *No trend and a multiplicative seasonal component.*

Linear trend models:

LN. *Linear trend component and no seasonality.* The keyword HOLT is an alias for LN.

LA. *Linear trend component and an additive seasonal component.*

LM. *Linear trend component and a multiplicative seasonal component.* The keyword WINTERS is an alias for LM.

Exponential trend models:

EN. *Exponential trend component and no seasonality.*

EA. *Exponential trend component and an additive seasonal component.*

EM. *Exponential trend component and a multiplicative seasonal component.*

Damped trend models:

DN. *Damped trend component and no seasonality.*

DA. *Damped trend component and an additive seasonal component.*

DM. *Damped trend component and a multiplicative seasonal component.*

Table 47. Models for different types of trends and seasons

Trend	None	Additive	Multiplicative
None	NN	NA	NM
Linear	LN	LA	LM
Exponential	EN	EA	EM
Damped	DN	DA	DM

Example

```
EXSMOOTH VAR1.
```

- This example uses the default model NN for series *VAR1*.

Example

```
EXSMOOTH VAR2
  /MODEL=LN.
```

- This example uses model LN (linear trend with no seasonality) for series *VAR2*.

PERIOD Subcommand

PERIOD indicates the periodicity of the seasonal component for seasonal models.

- The specification on PERIOD indicates how many observations are in one period or season and can be any positive integer.
- PERIOD is ignored if it is specified with a nonseasonal model.
- If PERIOD is not specified, the periodicity established on TSET PERIOD is in effect. If TSET PERIOD is not specified, the periodicity established on the DATE command is used. If periodicity was not established anywhere and a seasonal model is specified, EXSMOOTH will terminate.

Example

```
EXSMOOTH VAR1
  /MODEL=LA
  /PERIOD=12.
```

- This example specifies a periodicity of 12 for the seasonal *VAR1* series.

SEASFACT Subcommand

SEASFACT specifies initial seasonal factor estimates for seasonal models.

- The specification on SEASFACT is either a value list enclosed in parentheses or a variable name.
- If a value list is specified, the number of values in the list must equal the periodicity. For example, if the periodicity is 12, then 12 initial values must be specified.
- For multiplicative models, the sum of the values in the list should equal the periodicity. For additive models, the sum of the values should equal 0.
- A variable specification on SEASFACT indicates the name of a variable in the active dataset containing the seasonal factor estimates (see SEASON).
- If the model is seasonal and SEASFACT is not specified, EXSMOOTH calculates the initial seasonal factors.
- The seasonal factor estimates of a SEASFACT subcommand are not used when the model is respecified using the APPLY subcommand.

Example

```
EXSMOOTH VAR2
  /MODEL=LA
  /PERIOD=8
  /SEASFACT=(-25.30 -3 -14.70 17 4 3 13 6).
```

- This command uses the list of values specified on the SEASFACT subcommand as the initial seasonal factor estimates.
- Eight values are specified, since the periodicity is 8.
- The eight values sum to 0, since this is an additive seasonal model.

Example

```
EXSMOOTH VAR3
  /MODEL=LA
  /SEASFACT=SAF#1.
```

- This command uses the initial seasonal factors contained in variable *SAF#1*, which was saved in the active dataset by a previous SEASON command.

Smoothing Parameter Subcommands

ALPHA, GAMMA, DELTA, and PHI specify the values that are used for the smoothing parameters.

- The specification on each subcommand is either a value within the valid range, or the keyword GRID followed by optional range values.
- If GAMMA, DELTA, or PHI are not specified but are required for the model, the default values are used.
- ALPHA is applied to all models. If it is not specified, the default value is used.

ALPHA. *General smoothing parameter.* This parameter is applied to all models. Alpha can be any value between and including 0 and 1. (For EM models, alpha must be greater than 0 and less than or equal to 1.) The default value is 0.1.

GAMMA. *Trend smoothing parameter.* Gamma is used only with models that have a trend component, excluding damped seasonal (DA, DM) models. It is ignored if it is specified with a damped seasonal or no-trend model. Gamma can be any value between and including 0 and 1. The default value is 0.1.

DELTA. *Seasonal smoothing parameter.* Delta is used only with models that have a seasonal component. It is ignored if it is specified with any of the nonseasonal models. Delta can be any value between and including 0 and 1. The default value is 0.1.

PHI. *Trend modification parameter.* Phi is used only with models that have a damped trend component. It is ignored if it is specified with models that do not have a damped trend. Phi can be any value greater than 0 and less than 1. The default value is 0.1.

The following table summarizes the parameters that are used with each EXSMOOTH model. An *X* indicates that the parameter is used for the model.

Table 48. Parameters that can be specified with EXSMOOTH models

Model	ALPHA	DELTA	GAMMA	PHI
NN	x			
NA	x	x		
NM	x	x		
LN	x		x	
LA	x	x	x	
LM	x	x	x	
EN	x		x	
EA	x	x	x	
EM	x	x	x	
DN	x		x	x
DA	x	x		x
DM	x	x		x

Keyword GRID

The keyword GRID specifies a range of values to use for the associated smoothing parameter. When GRID is specified, new variables are saved only for the optimal set of parameters on the grid.

- The first value on GRID specifies the start value, the second value is the end value, and the last value is the increment.
- The start, end, and increment values on GRID are separated by commas or spaces and enclosed in parentheses.
- If you specify any grid values, you must specify all three.
- If no values are specified on GRID, the default values are used.

- Grid start and end values for alpha, gamma, and delta can range from 0 to 1. The defaults are 0 for the start value and 1 for the end value.
- Grid start and end values for phi can range from 0 to 1, exclusive. The defaults are 0.1 for the start value and 0.9 for the end value.
- Grid increment values must be within the range specified by start and end values. The default is 0.1 for alpha, and 0.2 for gamma, delta, and phi.

Example

```
EXSMOOTH VAR1
  /MODEL=LA
  /PERIOD=12
  /GAMMA=0.20
  /DELTA=0.20.
```

- This example uses a model with a linear trend and additive seasonality.
- The parameters and values are alpha = 0.10, gamma = 0.20, and delta = 0.20. Alpha is not specified but is always used by default.
- This command generates one *FIT* variable and one *ERR* variable to contain the forecasts and residuals generated by this one set of parameters.

Example

```
EXSMOOTH VAR2
  /MODEL=EA
  /ALPHA=GRID
  /DELTA=GRID(0.2,0.6,0.2).
```

- This example specifies a model with an exponential trend component and an additive seasonal component.
- The default start, end, and increment values (0, 1, and 0.1) are used for the grid search of alpha. Thus, the values used for alpha are 0, 0.1, 0.2, 0.3, ..., 0.9, and 1.
- The grid specification for delta indicates a start value of 0.2, an end value of 0.6, and an increment of 0.2. Thus, the values used for delta are 0.2, 0.4, and 0.6.
- Since this is an exponential trend model, the parameter gamma will be supplied by EXSMOOTH with the default value of 0.1, even though it is not specified on the command.
- Two variables (*FIT* and *ERR*) will be generated for the parameters resulting in the best-fitting model.

INITIAL Subcommand

INITIAL specifies the initial start and trend values used in the models.

- The specification on INITIAL is the start and trend values enclosed in parentheses. You must specify both values.
- The values specified on INITIAL are saved as part of the model and can be reapplied with the APPLY subcommand.
- If INITIAL is not specified, the initial start and trend values are calculated by EXSMOOTH. These calculated initial values are *not* saved as part of the model.
- To turn off the values specified on INITIAL when the model is used on an APPLY subcommand, specify INITIAL=CALCULATE. New initial values will then be calculated by EXSMOOTH.

Example

```
EXSMOOTH VAR2
  /MODEL=LA
  /PERIOD=4
  /SEASFACT=(23 -14.4 7 -15.6)
  /ALPHA=0.20
  /GAMMA=0.20
  /DELTA=0.30
  /INITIAL=(112,17).
```

- In this example, an initial start value of 112 and trend value of 17 is specified for series *VAR2*.

APPLY Subcommand

APPLY allows you to use a previously defined EXSMOOTH model without having to repeat the specifications.

- The only specification on APPLY is the name of a previous model in quotes. If a model name is not specified, the model specified on the previous EXSMOOTH command is used.
- To change one or more model specifications, specify the subcommands of only those portions you want to change after the APPLY subcommand.
- If no series are specified on the command, the series that were originally specified with the model being reapplied are used.
- To change the series used with the model, enter new series names before or after the APPLY subcommand. If a series name is specified before APPLY, the slash before the subcommand is required.
- Initial values from the previous model's INITIAL subcommand are applied unless you specify INITIAL = CALCULATE or a new set of initial values. Initial values from the original model are not applied if they were calculated by EXSMOOTH.
- Seasonal factor estimates from the original model's SEASFACT subcommand are not applied. To use seasonal factor estimates, you must respecify SEASFACT.

Example

```
EXSMOOTH VAR1
  /MODEL=NA
  /PERIOD=12
  /ALPHA=0.2
  /DELTA=0.2.
EXSMOOTH APPLY
  /DELTA=0.3.
EXSMOOTH VAR2
  /APPLY.
```

- The first command uses a model with no trend but additive seasonality for series *VAR1*. The length of the season (PERIOD) is 12. A general smoothing parameter (ALPHA) and a seasonal smoothing parameter (DELTA) are used, both with values set equal to 0.2.
- The second command applies the same model to the same series but changes the delta value to 0.3. Everything else stays the same.
- The last command applies the model and parameter values used in the second EXSMOOTH command to series *VAR2*.

Example

```
EXSMOOTH VAR3
  /MOD=NA
  /ALPHA=0.20
  /DELTA=0.4
  /INITIAL=(114,20).
EXSMOOTH VAR4
  /APPLY
  /INITIAL=CALCULATE.
```

- The first command uses a model with no trend and additive seasonality model with alpha set to 0.2 and delta set to 0.4. Initial start and trend values of 114 and 20 are specified.
- The second command applies the previous model and parameter values to a new variable, *VAR4*, but without the initial starting values. The initial starting values will be calculated by EXSMOOTH.

References

Abraham, B., and J. Ledolter. 1983. *Statistical methods of forecasting*. New York: John Wiley and Sons.

Gardner, E. S. 1985. Exponential smoothing: The state of the art. *Journal of Forecasting,* 4, 1-28.

Ledolter, J., and B. Abraham. 1984. Some comments on the initialization of exponential smoothing. *Journal of Forecasting,* 3, 79-84.

Makridakis, S., S. C. Wheelwright, and V. E. McGee. 1983. *Forecasting: Methods and applications*. New York: John Wiley and Sons.

EXTENSION

```
EXTENSION ACTION={ADD**  }
                 {REMOVE}

  /SPECIFICATION COMMAND=file.
```

** Default if the subcommand or keyword is omitted.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 16.0

- Command introduced.

Example

```
EXTENSION
  /SPECIFICATION COMMAND="c:\mypls.xml"
```

Overview

EXTENSION allows you to add user-created "extension" commands to the command table, enabling the use of "native" command syntax to drive functions written in an external programming language.

Basic Specification

The minimum specification is the SPECIFICATION subcommand with a COMMAND file specified.

ACTION Keyword

The ACTION keyword specifies the action to be taken on the command table.

ADD. *Add command name to command table.* This is the default. Note that ADD will replace a built-in command of the same name as well as an existing extension command of the same name, regardless of the languages in which the extension commands are written. It will not replace an abbreviated form of a built-in command unless the name matches the abbreviation. For example, an extension command named FREQUENCIES will replace the built-in FREQUENCIES command but the abbreviation FREQ will still run the built-in command.

REMOVE. *Remove command name from command table.* Any built-in command replaced by the specified extension command will be restored when the extension command is removed.

For information on creating extension commands, see the article *"Writing IBM SPSS Statistics Extension Commands"*, available from the IBM SPSS Predictive Analytics community at https://www.ibm.com/community/spss-statistics . You may also want to consult the chapter on Extension Commands in *Programming and Data Management for IBM SPSS Statistics* , which is also available from the IBM SPSS Predictive Analytics community.

Examples

Adding a New Command

```
EXTENSION
  /SPECIFICATION COMMAND="<installdir>/extensions/plscommand.xml".
```

The SPECIFICATION subcommand specifies that the syntax diagram for the extension command is defined by *plscommand.xml* in the *\extensions* subdirectory of the main installation directory (replace *<installdir>* with the path to the installation directory on your system). The default action is for the

command name provided in the XML file to be added to the command table or replace an existing command name.

Removing a Command

```
EXTENSION ACTION=REMOVE
  /SPECIFICATION COMMAND="<installdir>/extensions/plscommand.xml".
```

The ACTION keyword specifies that the command name provided in *plscommand.xml* should be removed from the command table.

SPECIFICATION Subcommand

The SPECIFICATION subcommand allows you to specify the location of the XML file that defines the syntax diagram for the extension command.

Note: At startup, the system processes all XML files in the Locations for extension commands setting displayed in the output of SHOW EXTPATHS. EXTENSION is most useful when you have XML files that are not in one of those locations, or want to add a new extension command to an already-running session.

COMMAND Keyword

The COMMAND keyword specifies the location of the XML file that defines the syntax diagram for the extension command. This file provides the command name and is used by the universal parser to pass the correct arguments to the extension command.

FACTOR

FACTOR is available in the Statistics Base Edition.

```
FACTOR VARIABLES=varlist† [/MISSING=[{LISTWISE**}] [INCLUDE]]
                                    {PAIRWISE  }
                                    {MEANSUB   }
                                    {DEFAULT** }

 [/MATRIX=[IN({COR='savfile'|'dataset'})]  [OUT({COR='savfile'|'dataset'})]]
             {COR=*                   }         {COR=*                   }
             {COV='savfile'|'dataset'}         {COV='savfile'|'dataset'}
             {COV=*                   }         {COV=*                   }
             {FAC='savfile'|'dataset'}         {FAC='savfile'|'dataset'}
             {FAC=*                   }         {FAC=*                   }
                       {FSC='savfile'|'dataset'}
                       {FSC=*                   }

 [/METHOD = {CORRELATION**}]
            {COVARIANCE   }

 [/SELECT=varname(value)]

 [/ANALYSIS=varlist...]

 [/PRINT=[DEFAULT**] [INITIAL**] [EXTRACTION**] [ROTATION**]
         [UNIVARIATE] [CORRELATION] [COVARIANCE] [DET] [INV]
         [REPR] [AIC] [KMO] [FSCORE] [SIG] [ALL]]

 [/PLOT=[EIGEN] [ROTATION [(n1,n2)]]]

 [/DIAGONAL={value list}]
            {DEFAULT** }

 [/FORMAT=[SORT] [BLANK(n)] [DEFAULT**]]

 [/CRITERIA=[FACTORS(n)] [MINEIGEN({1.0**})] [ITERATE({25**})]
                                   {n    }            {n   }

            [RCONVERGE({0.0001**})]  [{KAISER**}]
                       {n       }     {NOKAISER}

            [ECONVERGE({0.001**})] [DEFAULT**]]
                       {n      }

 [/EXTRACTION={PC**     } [/ROTATION={VARIMAX**   }]
              {PA1**    }            {EQUAMAX    }
              {PAF      }            {QUARTIMAX  }
              {ALPHA    }            {OBLIMIN({0})}
              {IMAGE    }                     {n}
              {ULS      }            {PROMAX({4}  }
              {GLS      }                    {n}
              {ML       }            {NOROTATE   }
              {DEFAULT**}            {DEFAULT**  }

 [/SAVE=[{REG    } ({ALL}[rootname])]]
         {BART   }  {n  }
         {AR     }
         {DEFAULT}
```

† Omit VARIABLES with matrix input.

**Default if subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 16.0

- Added support for SET THREADS and SET MCACHE.

Example

```
FACTOR VARIABLES=V1 TO V12.
```

Overview

FACTOR performs factor analysis based either on correlations or covariances and using one of the seven extraction methods. FACTOR also accepts matrix input in the form of correlation matrices, covariance matrices, or factor-loading matrices and can write the matrix materials to a matrix data file.

Options

Analysis Phase Options. You can choose to analyze a correlation or covariance matrix using the METHOD subcommand. You can select a subset of cases for the analysis phase using the SELECT subcommand. You can tailor the statistical display for an analysis using the PRINT subcommand. You can sort the output in the factor pattern and structure matrices with the FORMAT subcommand. You can also request scree plots and plots of the variables in factor space on the PLOT subcommand.

Extraction Phase Options. With the EXTRACTION subcommand, you can specify one of six extraction methods in addition to the default principal components extraction: principal axis factoring, alpha factoring, image factoring, unweighted least squares, generalized least squares, and maximum likelihood. You can supply initial diagonal values for principal axis factoring on the DIAGONAL subcommand. On the CRITERIA subcommand, you can alter the default statistical criteria used in the extraction.

Rotation Phase Options. You can control the criteria for factor rotation with the CRITERIA subcommand. On the ROTATION subcommand, you can choose among four rotation methods (equamax, quartimax, promax, and oblimin) in addition to the default varimax rotation, or you can specify no rotation.

Factor Scores. You can save factor scores as new variables in the active dataset using any of the three methods available on the SAVE subcommand.

Matrix Input and Output. With the MATRIX subcommand, you can write a correlation matrix, a covariance matrix, or a factor-loading matrix. You can also read matrix materials written either by a previous FACTOR procedure or by a procedure that writes correlation or covariance matrices.

Basic Specification

The basic specification is the VARIABLES subcommand with a variable list. FACTOR performs principal components analysis with a varimax rotation on all variables in the analysis using default criteria.

- When matrix materials are used as input, do not specify VARIABLES. Use the ANALYSIS subcommand to specify a subset of the variables in the matrix.
- METHOD and SELECT can be specified anywhere. VARIABLES must be specified before any other subcommands, unless an input matrix is specified. MISSING must be specified before ANALYSIS.
- The ANALYSIS, EXTRACTION, ROTATION, and SAVE subcommands must be specified in the order they are listed here. If you specify these subcommands out of order, you may get unexpected results. For example, if you specify EXTRACTION before ANALYSIS and SAVE before ROTATION, EXTRACTION and SAVE are ignored. If no EXTRACTION and SAVE subcommands are specified in proper order, the default will be used (that is, PC for EXTRACTION and no SAVE).
- The FORMAT subcommand can be specified anywhere after the VARIABLES subcommand.
- If an ANALYSIS subcommand is present, the statistical display options on PRINT, PLOT, or DIAGONAL must be specified after it. PRINT, PLOT, and DIAGONAL subcommands specified before the ANALYSIS subcommand are ignored. If no such commands are specified after the ANALYSIS subcommand, the default is used.
- The CRITERIA subcommand can be specified anywhere, but applies only to the subcommands that follow. If no CRITERIA subcommand is specified before EXTRACTION or ROTATION, the default criteria for the respective subcommand are used.

Example

```
FACTOR VAR=V1 TO V12
  /ANALYSIS=V1 TO V8
  /CRITERIA=FACTORS(3)
  /EXTRACTION=PAF
  /ROTATION=QUARTIMAX.
```

- The default CORRELATION method is used. FACTOR performs a factor analysis of the correlation matrix based on the first eight variables in the active dataset (*V1* to *V8*).
- The procedure extracts three factors using the principal axis method and quartimax rotation.
- LISTWISE (the default for MISSING) is in effect. Cases with missing values for any one of the variables from *V1* to *V12* are omitted from the analysis. As a result, if you ask for the factor analysis using VAR=V1

TO V8 and ANALYSIS=ALL, the results may be different even though the variables used in the analysis are the same.

Syntax Rules

- Each FACTOR procedure performs only one analysis with one extraction and one rotation. Use multiple FACTOR commands to perform multiple analyses.
- VARIABLES or MATRIX=IN can be specified only once. Any other subcommands can be specified multiple times but only the last in proper order takes effect.

Operations

- VARIABLES calculates a correlation and a covariance matrix. If SELECT is specified, only the selected cases are used.
- The correlation or covariance matrix (either calculated from the data or read in) is the basis for the factor analysis.
- Factor scores are calculated for all cases (selected and unselected).
- This procedure uses the multithreaded options specified by SET THREADS and SET MCACHE.

VARIABLES Subcommand

VARIABLES names all the variables to be used in the FACTOR procedure.

- VARIABLES is required except when matrix input is used. When FACTOR reads a matrix data file, the VARIABLES subcommand cannot be used.
- The specification on VARIABLES is a list of numeric variables.
- Keyword ALL on VARIABLES refers to all variables in the active dataset.
- Only one VARIABLES subcommand can be specified, and it must be specified first.

MISSING Subcommand

MISSING controls the treatment of cases with missing values.

- If MISSING is omitted or included without specifications, listwise deletion is in effect.
- MISSING must precede the ANALYSIS subcommand.
- The LISTWISE, PAIRWISE, and MEANSUB keywords are alternatives, but any one of them can be used with INCLUDE.

LISTWISE. *Delete cases with missing values listwise*. Only cases with nonmissing values for all variables named on the VARIABLES subcommand are used. Cases are deleted even if they have missing values only for variables listed on VARIABLES and have valid values for all variables listed on ANALYSIS. Alias DEFAULT.

PAIRWISE. *Delete cases with missing values pairwise*. All cases with nonmissing values for each pair of variables correlated are used to compute that correlation, regardless of whether the cases have missing values for any other variable.

MEANSUB. *Replace missing values with the variable mean*. All cases are used after the substitution is made. If INCLUDE is also specified, user-missing values are included in the computation of the means, and means are substituted only for the system-missing value. If SELECT is in effect, only the values of selected cases are used in calculating the means used to replace missing values for selected cases in analysis and for all cases in computing factor scores.

INCLUDE. *Include user-missing values*. Cases with user-missing values are treated as valid.

METHOD Subcommand

METHOD specifies whether the factor analysis is performed on a correlation matrix or a covariance matrix.

- Only one METHOD subcommand is allowed. If more than one is specified, the last is in effect.

CORRELATION. *Perform a correlation matrix analysis.* This is the default.

COVARIANCE. *Perform a covariance matrix analysis.* Valid only with principal components, principal axis factoring, or image factoring methods of extraction. The program issues an error if this keyword is specified when the input is a factor-loading matrix or a correlation matrix that does not contain standard deviations (STDDEV or SD).

SELECT Subcommand

SELECT limits cases used in the analysis phase to those with a specified value for any one variable.

- Only one SELECT subcommand is allowed. If more than one is specified, the last is in effect.
- The specification is a variable name and a valid value in parentheses. A string value must be specified within quotes. Multiple variables or values are not permitted.
- The selection variable does not have to be specified on the VARIABLES subcommand.
- Only cases with the specified value for the selection variable are used in computing the correlation or covariance matrix. You can compute and save factor scores for the unselected cases as well as the selected cases.
- SELECT is not valid if MATRIX = IN is specified.

Example

```
FACTOR VARIABLES = V1 TO V10
  /SELECT=COMPLETE(1)
  /SAVE (4).
```

- FACTOR analyzes all ten variables named on VARIABLES, using only cases with a value of 1 for the variable *COMPLETE*.
- By default, FACTOR uses the CORRELATION method and performs the principal components analysis of the selected cases. Those with eigenvalues greater than 1 are rotated using varimax rotation.
- Four factor scores, for both selected and unselected cases, are computed using the default regression method and four new variables are saved in the active dataset.

ANALYSIS Subcommand

The ANALYSIS subcommand specifies a subset of the variables named on VARIABLES for use in an analysis.

- The specification on ANALYSIS is a list of variables, all of which must have been named on the VARIABLES subcommand. For matrix input, ANALYSIS can specify a subset of the variables in a correlation or covariance matrix.
- Only one ANALYSIS subcommand is allowed. When multiple ANALYSIS subcommands are specified, the last is in effect.
- If no ANALYSIS is specified, all variables named on the VARIABLES subcommand (or included in the matrix input file) are used.
- Keyword TO in a variable list on ANALYSIS refers to the order in which variables are named on the VARIABLES subcommand, not to their order in the active dataset.
- Keyword ALL refers to all variables named on the VARIABLES subcommand.

Example

```
FACTOR VARIABLES=V1 V2 V3 V4 V5 V6
   /ANALYSIS=V4 TO V6.
```

- This example requests a factor analysis of *V4*, *V5*, and *V6*. Keyword TO on ANALYSIS refers to the order of variables on VARIABLES, not the order in the active dataset.
- Cases with missing values for all variables specified on VARIABLES are omitted from the analysis. (The default setting for MISSING.)
- By default, the CORRELATION method is used and a principal components analysis with a varimax rotation is performed.

FORMAT Subcommand

FORMAT modifies the format of factor pattern and structure matrices.

- FORMAT can be specified anywhere after VARIABLES and MISSING. If more than one FORMAT is specified, the last is in effect.
- If FORMAT is omitted or included without specifications, variables appear in the order in which they are named on ANALYSIS and all matrix entries are displayed.

SORT. *Order the factor loadings in descending order.* Variables are displayed in descending order of the factor 1 loadings until a loading for factor 2 exceeds the loading for factor 1. The remaining variables are then displayed in descending order of the factor 2 loadings until a loading for factor 3 exceeds the loading for factor 2, and so on. The result shows blocks of variables that are similar.

BLANK(n). *Suppress display of coefficients lower than* n *in absolute value*. The corresponding cells in the table will be blank.

DEFAULT. *Turn off keywords SORT and BLANK.*

Example

```
FACTOR VARIABLES=V1 TO V12
  /MISSING=MEANSUB
  /FORMAT=SORT BLANK(.3)
  /EXTRACTION=ULS
  /ROTATION=NOROTATE.
```

- This example specifies an analysis of all variables between and including *V1* and *V12* in the active dataset.
- The default CORRELATION method is used.
- The MISSING subcommand substitutes variable means for missing values.
- The FORMAT subcommand orders variables in factor pattern matrices by descending value of loadings. Factor loadings with an absolute value less than 0.3 are omitted.
- Factors are extracted using unweighted least squares and are not rotated.

PRINT Subcommand

PRINT controls the statistical display in the output.

- Keywords INITIAL, EXTRACTION, and ROTATION are the defaults if PRINT is omitted or specified without keywords.
- If any keywords are specified, only the output specifically requested is produced.
- The requested statistics are displayed only for variables specified on the last ANALYSIS subcommand.
- If more than one PRINT subcommand is specified, the last is in effect.

- If any ANALYSIS subcommand is explicitly specified, all PRINT subcommands specified before the last ANALYSIS subcommand are ignored. If no PRINT subcommand is specified after the last ANALYSIS subcommand, the default takes effect.

INITIAL. *Initial communalities for each variable, eigenvalues of the unreduced correlation matrix, and percentage of variance for each factor.*

EXTRACTION. *Factor pattern matrix, revised communalities, the eigenvalue of each factor retained, and the percentage of variance each eigenvalue represents.*

ROTATION. *Rotated factor pattern matrix, factor transformation matrix, factor correlation matrix, and the post-rotation sums of squared loadings*.

UNIVARIATE. *Valid number of cases, means, and standard deviations*. (Not available with matrix input.) If MISSING=MEANSUB or PAIRWISE, the output also includes the number of missing cases.

CORRELATION. *Correlation matrix.* Ignored if the input is a factor-loading matrix.

COVARIANCE. *Covariance matrix.* Ignored if the input is a factor-loading matrix or a correlation matrix that does not contain standard deviations (STDDEV or SD).

SIG. *Matrix of significance levels of correlations*.

DET. *Determinant of the correlation or covariance matrix, depending on the specification on* METHOD.

INV. *Inverse of the correlation or covariance matrix, depending on the specification on* METHOD.

AIC. *Anti-image covariance and correlation matrices* [24]. The measure of sampling adequacy for the individual variable is displayed on the diagonal of the anti-image correlation matrix.

KMO. *Kaiser-Meyer-Olkin measure of sampling adequacy and Bartlett's test of sphericity*. Always based on the correlation matrix. Not computed for an input matrix when it does not contain *N* values.

REPR. *Reproduced correlations and residuals or reproduced covariance and residuals, depending on the specification on* METHOD.

FSCORE. *Factor score coefficient matrix*. Factor score coefficients are calculated using the method requested on the SAVE subcommand. The default is the regression method.

ALL. *All available statistics*.

DEFAULT. INITIAL, EXTRACTION, *and* ROTATION.

Example

```
FACTOR VARS=V1 TO V12
  /SELECT=COMPLETE ('yes')
  /MISS=MEANSUB
  /PRINT=DEF AIC KMO REPR
  /EXTRACT=ULS
  /ROTATE=VARIMAX.
```

- This example specifies a factor analysis that includes all variables between and including *V1* and *V12* in the active dataset.
- Only cases with the value "yes" on *COMPLETE* are used.
- Variable means are substituted for missing values. Only values for the selected cases are used in computing the mean. This mean is used to substitute missing values in analyzing the selected cases and in computing factor scores for all cases.
- The output includes the anti-image correlation and covariance matrices, the Kaiser-Meyer-Olkin measure of sampling adequacy, the reproduced correlation and residual matrix, as well as the default statistics.
- Factors are extracted using unweighted least squares.
- The factor pattern matrix is rotated using the varimax rotation.

[24] Kaiser, H. F. 1970. A second-generation Little Jiffy. *Psychometrika,* 35, 401-415.

PLOT Subcommand

Use PLOT to request scree plots or plots of variables in rotated factor space.

- If PLOT is omitted, no plots are produced. If PLOT is used without specifications, it is ignored.
- If more than one PLOT subcommand is specified, only the last one is in effect.
- If any ANALYSIS subcommand is explicitly specified, all PLOT subcommands specified before the last ANALYSIS subcommand are ignored. If no PLOT subcommand is specified after the last ANALYSIS subcommand, no plot is produced.

EIGEN. *Scree plot* [25]. The eigenvalues from each extraction are plotted in descending order.

ROTATION. *Plots of variables in factor space.* When used without any additional specifications, ROTATION can produce only high-resolution graphics. If three or more factors are extracted, a 3-D plot is produced with the factor space defined by the first three factors. You can request two-dimensional plots by specifying pairs of factor numbers in parentheses; for example, PLOT ROTATION(1,2)(1,3)(2,3) requests three plots, each defined by two factors. The ROTATION subcommand must be explicitly specified when you enter the keyword ROTATION on the PLOT subcommand.

DIAGONAL Subcommand

DIAGONAL specifies values for the diagonal in conjunction with principal axis factoring.

- If DIAGONAL is omitted or included without specifications, FACTOR uses the default method for specifying the diagonal.
- DIAGONAL is ignored with extraction methods other than PAF. The values are automatically adjusted by corresponding variances if METHOD=COVARIANCE.
- If more than one DIAGONAL subcommand is specified, only the last one is in effect.
- If any ANALYSIS subcommand is explicitly specified, DIAGONAL subcommands specified before the last ANALYSIS subcommand are ignored. If no DIAGONAL is specified after the last ANALYSIS subcommand, the default is used.
- Default communality estimates for PAF are squared multiple correlations. If these cannot be computed, the maximum absolute correlation between the variable and any other variable in the analysis is used.

valuelist. *Diagonal values.* The number of values supplied must equal the number of variables in the analysis block. Use the notation n* before a value to indicate that the value is repeated *n* times.

DEFAULT. *Initial communality estimates.*

Example

```
FACTOR VARIABLES=V1 TO V12
  /DIAGONAL=.56 .55 .74 2*.56 .70 3*.65 .76 .64 .63
  /EXTRACTION=PAF
  /ROTATION=VARIMAX.
```

- The factor analysis includes all variables between and including *V1* and *V12* in the active dataset.
- DIAGONAL specifies 12 values to use as initial estimates of communalities in principal axis factoring.
- The factor pattern matrix is rotated using varimax rotation.

CRITERIA Subcommand

CRITERIA controls extraction and rotation criteria.

- CRITERIA can be specified anywhere after VARIABLES and MISSING.

[25] Cattell, R. B. 1966. The scree test for the number of factors. *Journal of Multivariate Behavioral Research,* 1, 245-276.

- Only explicitly specified criteria are changed. Unspecified criteria keep their defaults.
- Multiple CRITERIA subcommands are allowed. Changes made by a previous CRITERIA subcommand are overwritten by a later CRITERIA subcommand.
- Any CRITERIA subcommands specified after the last EXTRACTION subcommand have no effect on extraction.
- Any CRITERIA subcommands specified after the last ROTATION subcommand have no effect on rotation.

The following keywords on CRITERIA apply to extractions:

FACTORS(n). *Number of factors extracted*. The default is the number of eigenvalues greater than MINEIGEN. When specified, FACTORS overrides MINEIGEN.

MINEIGEN(n). *Minimum eigenvalue used to control the number of factors extracted.* If METHOD=CORRELATION, the default is 1. If METHOD=COVARIANCE, the default is computed as (Total Variance/Number of Variables)**n*, where Total Variance is the total weighted variance principal components or principal axis factoring extraction and the total image variance for image factoring extraction.

ECONVERGE(n). *Convergence criterion for extraction*. The default is 0.001.

The following keywords on CRITERIA apply to rotations:

RCONVERGE(n). *Convergence criterion for rotation*. The default is 0.0001.

KAISER. *Kaiser normalization in the rotation phase.* This is the default. The alternative is NOKAISER.

NOKAISER. *No Kaiser normalization*.

The following keywords on CRITERIA apply to both extractions and rotations:

ITERATE(n). *Maximum number of iterations for solutions in the extraction or rotation phases.* The default is 25.

DEFAULT. *Reestablish default values for all criteria.*

Example

```
FACTOR VARIABLES=V1 TO V12
  /CRITERIA=FACTORS(6)
  /EXTRACTION=PC
  /ROTATION=NOROTATE
  /PLOT=ROTATION.
```

- This example analyzes all variables between and including *V1* and *V12* in the active dataset.
- Six factors are extracted using the default principal components method, and the factor pattern matrix is not rotated.
- PLOT sends all extracted factors to the graphics editor and shows a 3-D plot of the first three factors.

EXTRACTION Subcommand

EXTRACTION specifies the factor extraction technique.

- Only one EXTRACTION subcommand is allowed. If multiple EXTRACTION subcommands are specified, only the last is performed.
- If any ANALYSIS subcommand is explicitly specified, all EXTRACTION subcommands before the last ANALYSIS subcommand are ignored. If no EXTRACTION subcommand is specified after the last ANALYSIS subcommand, the default extraction is performed.
- If EXTRACTION is not specified or is included without specifications, principal components extraction is used.
- If you specify criteria for EXTRACTION, the CRITERIA subcommand must precede the EXTRACTION subcommand.

- When you specify EXTRACTION, you should always explicitly specify the ROTATION subcommand. If ROTATION is not specified, the factors are not rotated.

PC. *Principal components analysis* [26]. This is the default. PC can also be requested with keyword PA1 or DEFAULT.

PAF. *Principal axis factoring.* PAF can also be requested with keyword PA2.

ALPHA. *Alpha factoring* [27]. Invalid if METHOD=COVARIANCE.

IMAGE. *Image factoring* [28].

ULS. *Unweighted least squares* [29]. Invalid if METHOD=COVARIANCE.

GLS. *Generalized least squares.* Invalid if METHOD=COVARIANCE.

ML. *Maximum likelihood* [30]. Invalid if METHOD=COVARIANCE.

Example

```
FACTOR VARIABLES=V1 TO V12
  /ANALYSIS=V1 TO V6
  /EXTRACTION=ULS
  /ROTATE=NOROTATE.
```

- This example analyzes variables *V1* through *V6* with an unweighted least-squares extraction. No rotation is performed.

ROTATION Subcommand

ROTATION specifies the factor rotation method. It can also be used to suppress the rotation phase entirely.

- Only one ROTATION subcommand is allowed. If multiple ROTATION subcommands are specified, only the last is performed.
- If any ANALYSIS subcommand is explicitly specified, all ROTATION subcommands before the last ANALYSIS subcommand are ignored. If any EXTRACTION subcommand is explicitly specified, all ROTATION subcommands before the last EXTRACTION subcommand are ignored.
- If ROTATION is omitted together with EXTRACTION, varimax rotation is used.
- If ROTATION is omitted but EXTRACTION is not, factors are not rotated.
- Keyword NOROTATE on the ROTATION subcommand produces a plot of variables in unrotated factor space if the PLOT subcommand is also included for the analysis.

VARIMAX. *Varimax rotation*. This is the default if ROTATION is entered without specifications or if EXTRACTION and ROTATION are both omitted. Varimax rotation can also be requested with keyword DEFAULT.

EQUAMAX. *Equamax rotation*.

QUARTIMAX. *Quartimax rotation*.

OBLIMIN(n). *Direct oblimin rotation*. This is a nonorthogonal rotation; thus, a factor correlation matrix will also be displayed. You can specify a delta ($n \leq 0.8$) in parentheses. The value must be less than or equal to 0.8. The default is 0.

---

[26] Harman, H. H. 1976. *Modern Factor Analysis*, 3rd ed. Chicago: University of Chicago Press.

[27] Kaiser, H. F., and J. Caffry. 1965. Alpha factor analysis. *Psychometrika,* 30, 1-14.

[28] Kaiser, H. F. 1963. Image analysis. In: *Problems in Measuring Change*, C. W. Harris, eds. Madison: University of Wisconsin Press.

[29] Jöreskog, K. G. 1977. Factor analysis by least-square and maximum-likelihood method. In: *Statistical Methods for Digital Computers, volume 3*, K. Enslein, A. Ralston, and R. S. Wilf, eds. New York: John Wiley and Sons.

[30] Jöreskog, K. G., and D. N. Lawley. 1968. New methods in maximum likelihood factor analysis. *British Journal of Mathematical and Statistical Psychology,* 21, 85-96.

PROMAX(n). *Promax rotation*. This is a nonorthogonal rotation; thus, a factor correlation matrix will also be displayed. For this method, you can specify a real-number value greater than 1. The default is 4.

NOROTATE. *No rotation*.

Example

```
FACTOR VARIABLES=V1 TO V12
  /EXTRACTION=ULS
  /ROTATION
  /ROTATION=OBLIMIN.
```

- The first ROTATION subcommand specifies the default varimax rotation.
- The second ROTATION subcommand specifies an oblimin rotation based on the same extraction of factors.

SAVE Subcommand

SAVE allows you to save factor scores from any rotated or unrotated extraction as new variables in the active dataset. You can use any of the three methods for computing the factor scores.

- Only one SAVE subcommand is executed. If you specify multiple SAVE subcommands, only the last is executed.
- SAVE must follow the last ROTATION subcommand.
- If no ROTATION subcommand is specified after the last EXTRACTION subcommand, SAVE must follow the last EXTRACTION subcommand and no rotation is used.
- If neither ROTATION nor EXTRACTION is specified, SAVE must follow the last ANALYSIS subcommand and the default extraction and rotation are used to compute the factor scores.
- SAVE subcommands before any explicitly specified ANALYSIS, EXTRACTION, or ROTATION subcommands are ignored.
- You cannot use the SAVE subcommand if you are replacing the active dataset with matrix materials. (See the topic “Matrix Output ” on page 675 for more information.)
- The new variables are added to the end of the active dataset.

Keywords to specify the method of computing factor scores are:

REG. *Regression method.* This is the default.

BART. *Bartlett method.*

AR. *Anderson-Rubin method.*

DEFAULT. *The same as REG.*

- After one of the above keywords, specify in parentheses the number of scores to save and a rootname to use in naming the variables.
- You can specify either an integer or the keyword ALL. The maximum number of scores you can specify is the number of factors in the solution.
- FACTOR forms variable names by appending sequential numbers to the rootname you specify. The rootname must begin with a letter and conform to the rules for variable names. For information on variable naming rules, see “Variable Names” on page 42 .
- If you do not specify a rootname, FACTOR forms unique variable names using the formula *FACn_m,* where *m* increments to create a new rootname and *n* increments to create a unique variable name. For example, *FAC1_1, FAC2_1, FAC3_1*, and so on will be generated for the first set of saved scores and *FAC1_2, FAC2_2, FAC3_2*, and so on for the second set.
- FACTOR automatically generates variable labels for the new variables. Each label contains information about the method of computing the factor score, its sequential number, and the sequential number of the analysis.

Example

```
FACTOR VARIABLES=V1 TO V12
  /CRITERIA FACTORS(4)
  /ROTATION
  /SAVE REG (4,PCOMP).
```

- Since there is no EXTRACTION subcommand before the ROTATION subcommand, the default principal components extraction is performed.
- The CRITERIA subcommand specifies that four principal components should be extracted.
- The ROTATION subcommand requests the default varimax rotation for the principal components.
- The SAVE subcommand calculates scores using the regression method. Four scores will be added to the file: *PCOMP1*, *PCOMP2*, *PCOMP3*, and *PCOMP4*.

MATRIX Subcommand

MATRIX reads and writes matrix data files.

- MATRIX must always be specified first.
- Only one IN and one OUT keyword can be specified on the MATRIX subcommand. If either IN or OUT is specified more than once, the FACTOR procedure is not executed.
- The matrix type must be indicated on IN or OUT. The types are COR for a correlation matrix, COV for a covariance matrix, and FAC for a factor-loading matrix. Indicate the matrix type within parentheses immediately before you identify the matrix file.
- If you use both IN and OUT on MATRIX, you can specify them in either order. You cannot write a covariance matrix if the input matrix is a factor-loading matrix or a correlation matrix that does not contain standard deviations (STDDEV or SD).
- If you read in a covariance matrix and write out a factor-loading matrix, the output factor loadings are rescaled.

OUT (matrix type= 'savfile'|'dataset'). *Write a matrix data file.* Specify the matrix type (COR, COV, FAC, or FSC) and the matrix file in parentheses. For the matrix data file, specify a filename to store the matrix materials on disk, a previously declared dataset available in the current session, or an asterisk to replace the active dataset. If you specify an asterisk or a dataset name, the matrix data file is not stored on disk unless you use SAVE or XSAVE.

IN (matrix type= 'savfile'|'dataset'). *Read a matrix data file.* Specify the matrix type (COR, COV, or FAC) and the matrix file in parentheses. For the matrix data file, specify an asterisk if the matrix data file is the active dataset. If the matrix file is another file, specify the filename or dataset name in parentheses. A matrix file read from an external file or another dataset in the current session does not replace the active dataset.

Matrix Output

FACTOR can write matrix materials in the form of a correlation matrix, a covariance matrix, a factor-loading matrix, or a factor score coefficients matrix.

- The correlation and covariance matrix materials include counts, means, and standard deviations in addition to correlations or covariances.
- The factor-loading matrix materials contain only factor values and no additional statistics.
- The factor score coefficients materials include means and standard deviations, in addition to factor score coefficients.
- See “Format of the Matrix Data File ” on page 676 for a description of the file.
- FACTOR generates one matrix per split file.
- Any documents contained in the active dataset are not transferred to the matrix file.

Matrix Input

- FACTOR can read matrix materials written either by a previous FACTOR procedure or by a procedure that writes correlation or covariance matrices. See the topic "Universals" on page 33 for more information.
- MATRIX=IN cannot be used unless a active dataset has already been defined. To read an existing matrix data file at the beginning of a session, first use GET to retrieve the matrix file and then specify IN(COR=*), IN(COV=*) or IN(FAC=*) on MATRIX.
- The VARIABLES subcommand cannot be used with matrix input.
- For correlation and covariance matrix input, the ANALYSIS subcommand can specify a subset of the variables in the matrix. You cannot specify a subset of variables for factor-loading matrix input. By default, the ANALYSIS subcommand uses all variables in the matrix.

Format of the Matrix Data File

- For correlation or covariance matrices, the matrix data file has two special variables created by the program: *ROWTYPE_* and *VARNAME_*. Variable *ROWTYPE_* is a short string variable with the value CORR (for Pearson correlation coefficient) or COV (for covariance) for each matrix row. Variable *VARNAME_* is a short string variable whose values are the names of the variables used to form the correlation matrix.
- For factor-loading matrices, the program generates two special variables named *ROWTYPE_* and *FACTOR_*. The value for *ROWTYPE_* is always FACTOR. The values for *FACTOR_* are the ordinal numbers of the factors.
- For factor score coefficient matrices, the matrix data file has two special variables created: *ROWTYPE_* and *VARNAME_*. If split-file processing is in effect, the split variables appear first in the matrix output file, followed by *ROWTYPE_*, *VARNAME_*, and the variables in the analysis. *ROWTYPE_* is a short string with three possible values: MEAN, STDDEV, and FSCOEF. There is always one occurrence of the value MEAN. If /METHOD = CORRELATION then there is one occurrence of the value STDDEV. Otherwise, this value does not appear. There are as many occurrences of FSCOEF as the number of extracted factors. *VARNAME_* is a short string who values are FAC*n* where *n* is the sequence of the saved factor when *ROWTYPE_* equals FSCOEF. Otherwise the value is empty.
- The remaining variables are the variables used to form the matrix.

Split Files

- FACTOR can read or write split-file matrices.
- When split-file processing is in effect, the first variables in the matrix data file are the split variables, followed by *ROWTYPE_*, *VARNAME_* (or *FACTOR_*), and then the variables used to form the matrix.
- A full set of matrix materials is written for each split-file group defined by the split variables.
- A split variable cannot have the same variable name as any other variable written to the matrix data file.
- If split-file processing is in effect when a matrix is written, the same split file must be in effect when that matrix is read by any other procedure.

Example: Factor Correlation Matrix Output to External File

```
GET FILE='/data/GSS80.sav'
  /KEEP ABDEFECT TO ABSINGLE.
FACTOR VARIABLES=ABDEFECT TO ABSINGLE
  /MATRIX OUT(COR='/data/cormtx.sav').
```

- FACTOR retrieves the *GSS80.sav* file and writes a factor correlation matrix to the file *cormtx.sav*.
- The active dataset is still *GSS80.sav*. Subsequent commands will be executed on this file.

Example: Factor Correlation Matrix Output Replacing Active Dataset

```
GET FILE='/data/GSS80.sav'
  /KEEP ABDEFECT TO ABSINGLE.
FACTOR VARIABLES=ABDEFECT TO ABSINGLE
```

```
  /MATRIX OUT(COR=*).
LIST.
```

- FACTOR writes the same matrix as in the previous example.
- The active dataset is replaced with the correlation matrix. The LIST command is executed on the matrix file, not on *GSS80*.

Example: Factor-Loading Matrix Output Replacing Active Dataset

```
GET FILE='/dataGSS80.sav'
  /KEEP ABDEFECT TO ABSINGLE.
FACTOR VARIABLES=ABDEFECT TO ABSINGLE
  /MATRIX OUT(FAC=*).
```

- FACTOR generates a factor-loading matrix that replaces the active dataset.

Example: Matrix Input from active dataset

```
GET FILE='/data/country.sav'
  /KEEP SAVINGS POP15 POP75 INCOME GROWTH.
REGRESSION MATRIX OUT(*)
  /VARS=SAVINGS TO GROWTH
  /MISS=PAIRWISE
  /DEP=SAVINGS /ENTER.
FACTOR MATRIX IN(COR=*) /MISSING=PAIRWISE.
```

- The GET command retrieves the *country.sav* file and selects the variables needed for the analysis.
- The REGRESSION command computes correlations among five variables with pairwise deletion. MATRIX=OUT writes a matrix data file, which replaces the active dataset.
- MATRIX IN(COR=*) on FACTOR reads the matrix materials REGRESSION has written to the active dataset. An asterisk is specified because the matrix materials are in the active dataset. FACTOR uses pairwise deletion, since this is what was in effect when the matrix was built.

Example: Matrix Input from External File

```
GET FILE='/data/country.sav'
  /KEEP SAVINGS POP15 POP75 INCOME GROWTH.
REGRESSION
  /VARS=SAVINGS TO GROWTH
  /MISS=PAIRWISE
  /DEP=SAVINGS /ENTER.
FACTOR MATRIX IN(COR=CORMTX).
```

- This example performs a regression analysis on file *country.sav* and then uses a different file for FACTOR. The file is an existing matrix data file.
- MATRIX=IN specifies the matrix data file *CORMTX*.
- *CORMTX* does not replace *country.sav* as the active dataset.

Example: Matrix Input from active dataset

```
GET FILE='/data/cormtx.sav'.
FACTOR MATRIX IN(COR=*).
```

- This example starts a new session and reads an existing matrix data file. GET retrieves the matrix data file *cormtx.sav*.
- MATRIX = IN specifies an asterisk because the matrix data file is the active dataset. If MATRIX = IN(cormtx.sav) is specified, the program issues an error message.
- If the GET command is omitted, the program issues an error message.

Example: Using Saved Coefficients to Score an External File

```
MATRIX.
GET A /FILE="fsc.sav".
GET B /FILE="ext_data_standardized.sav" /VAR=varlist.
COMPUTE SCORES=A*TRANSPOS(B).
SAVE SCORES /OUTFILE="scored.sav".
END MATRIX.
```

- This example scores an external file using the factor score coefficients from a previous analysis, say, upon *original.sav*.
- Factor score coefficients are read from *fsc.sav* into *A*.
- The data are read from *ext_data_standardized.sav* into *B*. The variable values in the external file should be standardized. Note that if the values in *ext_data_standardized.sav* are standardized based upon the means and standard deviations of the unstandardized values in *ext_data.sav*, and not upon the means and standard deviations of values in *original.sav*, then this analysis will be different from one in which *original.sav* and *ext_data.sav* are merged and FACTOR is run using the SELECT subcommand with cases from *original.sav* selected for use in computing the correlation or covariance matrix.

  If there are missing values, add `/MISSING=OMIT` or `/MISSING=0` to the second GET statement to remove cases with missing values or impute the mean (0, since the variables are standardized).
- The scores are saved to *scored.sav*.

References

Cattell, R. B. 1966. The scree test for the number of factors. *Journal of Multivariate Behavioral Research,* 1, 245-276.

Harman, H. H. 1976. *Modern Factor Analysis*, 3rd ed. Chicago: University of Chicago Press.

Jöreskog, K. G. 1977. Factor analysis by least-square and maximum-likelihood method. In: *Statistical Methods for Digital Computers, volume 3*, K. Enslein, A. Ralston, and R. S. Wilf, eds. New York: John Wiley and Sons.

Jöreskog, K. G., and D. N. Lawley. 1968. New methods in maximum likelihood factor analysis. *British Journal of Mathematical and Statistical Psychology,* 21, 85-96.

Kaiser, H. F. 1963. Image analysis. In: *Problems in Measuring Change*, C. W. Harris, eds. Madison: University of Wisconsin Press.

Kaiser, H. F. 1970. A second-generation Little Jiffy. *Psychometrika,* 35, 401-415.

Kaiser, H. F., and J. Caffry. 1965. Alpha factor analysis. *Psychometrika,* 30, 1-14.

FILE HANDLE

```
FILE HANDLE handle /NAME='path and/or file specifications'
                  [/MODE={CHARACTER }] [/RECFORM={FIXED   } [/LRECL=n]
                         {BINARY    }            {VARIABLE}
                         {MULTIPUNCH}            {SPANNED }
                         {IMAGE     }
                         {360       }
                  [/ENCODING = 'encoding specification']
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 13.0

- The NAME subcommand is modified to accept a path and/or file.

Release 16.0

- ENCODING subcommand added for Unicode support.

Example

```
FILE HANDLE thisMonthFile /NAME='/sales/data/july.sav'.
FILE HANDLE dataDirectory /NAME='/sales/data'.
GET FILE 'thisMonthFile'.
GET FILE 'dataDirectory/july.sav'.
```

Overview

FILE HANDLE assigns a unique *file handle* to a path and/or file and supplies operating system specifications for the file. A defined file handle can be specified on any subsequent FILE, OUTFILE, MATRIX, or WRITE subcommands of various procedures.

The minimum specification is the command name followed by the string that will be used as the handle, followed by the NAME subcommand that specifies either a file or directory, as in:

```
FILE HANDLE thisMonthFile /NAME='/sales/data/july.sav'.
```

Syntax Rules

- The handle, specified immediately after the command name, must conform to variable naming rules. See the topic “Variable Names” on page 42 for more information.
- FILE HANDLE is required for reading data files with record lengths greater than 8,192. See the topic “LRECL Subcommand ” on page 680 for more information.
- FILE HANDLE is required for reading IBM VSAM datasets, EBCDIC data files, binary data files, and character data files that are not delimited by ASCII line feeds.
- If you specify 360 on the MODE subcommand, you must specify RECFORM.
- If you specify IMAGE on the MODE subcommand, you must specify LRECL.

Operations

A file handle is used only during the current session. The handle is never saved as part of a data file. The normal quoting conventions for file specifications apply, with or without file handles.

Example

```
FILE HANDLE thisMonthFile /NAME='/sales/data/july.sav'.
FILE HANDLE dataDirectory /NAME='/sales/data'.
```

```
GET FILE 'thisMonthFile'.
GET FILE 'dataDirectory/july.sav'.
```

- The first FILE HANDLE command defines a file handle that refers to a specific file.
- The second FILE HANDLE command only specifies a directory path.
- The two subsequent GET FILE commands are functionally equivalent. Note that both file specifications are enclosed in quotes (a good general practice).

NAME Subcommand

NAME specifies the path and/or file you want to refer to by the file handle. The file specifications must conform to the file naming convention for the type of computer and operating system on which the program is run. See the documentation for your system for specific information about the file naming convention.

If NAME specifies a relative path or does not include a path, the path is set to the current working directory at the time the FILE HANDLE command is executed.

MODE Subcommand

MODE specifies the type of file you want to refer to by the file handle.

CHARACTER. *Character file whose logical records are delimited by ASCII line feeds.*

BINARY. *Unformatted binary file generated by Microsoft FORTRAN.*

MULTIPUNCH. *Column binary file.*

IMAGE. *Binary file consisting of fixed-length records.*

360. *EBCDIC data file.*

Example

```
FILE HANDLE ELE48 /NAME='/data/ELE48.DAT' /MODE=MULTIPUNCH.
DATA LIST FILE=ELE48.
```

- FILE HANDLE defines *ELE48* as the handle for the file.
- The MODE subcommand indicates that the file contains multipunch data.
- The file specification on NAME conforms to VMS convention: the file *ELE48.DAT* is located in the directory *data*.
- The FILE subcommand on DATA LIST refers to the handle defined on the FILE HANDLE command.

RECFORM Subcommand

RECFORM specifies the record format and is necessary when you specify 360 on MODE. RECFORM has no effect with other specifications on MODE.

FIXED. *Fixed-length record.* All records have the same length. Alias F. When FIXED is specified, the record length must be specified on the LRECL subcommand.

VARIABLE. *Variable-length record.* No logical record is larger than one physical block. Alias V.

SPANNED. *Spanned record.* Records may be larger than fixed-length physical blocks. Alias VS.

LRECL Subcommand

LRECL specifies the length of each record in the file. When you specify IMAGE under UNIX, OS/2, or Microsoft Windows, or 360 for IBM360 EBCDIC data files, you must specify LRECL. You can specify a

record length greater than the default (8,192) for an image file, a character file, or a binary file. The maximum record length is 2,147,483,647. Do not use LRECL with MULTIPUNCH.

Example

```
FILE HANDLE TRGT1 /NAME='/data/RGT.DAT'
                  /MODE=IMAGE LRECL=16.
DATA LIST FILE=TRGT1.
```

- IMAGE is specified on the MODE subcommand. Subcommand LRECL must be specified.
- The file handle is used on the DATA LIST command.

ENCODING Subcommand

ENCODING specifies the encoding format of the file. The keyword is followed by an equals sign and a quoted encoding specification.

- In Unicode mode, the default is UTF8. For more information, see SET command, UNICODE subcommand.
- In code page mode, the default is the current locale setting. For more information, see SET command, LOCALE subcommand.
- The quoted encoding value can be: Locale (the current locale setting), UTF8, UTF16, UTF16BE (big endian), UTF16LE (little endian), a numeric Windows code page value (for example, '1252'), or an IANA code page value (for example, 'iso8859-1' or cp1252).
- In Unicode mode, the defined width of string variables is tripled for code page and UTF-16 text data files. Use ALTER TYPE to automatically adjust the defined width of string variables.

FILE LABEL

```
FILE LABEL label text
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
FILE LABEL Original survey responses prior to recoding.
```

Overview

FILE LABEL provides a descriptive label for a data file.

Basic Specification

The basic specification is the command name followed by the label text.

Syntax Rules

- The label text cannot exceed 64 bytes.
- Labels do not need to be enclosed in quotes.
- If the label is enclosed in quotes—or starts with a quotation mark (single or double)—standard rules for quoted strings apply. See the topic “String Values in Command Specifications” on page 35 for more information.

Operations

- If the data file is saved in IBM SPSS Statistics format, the label is saved in the dictionary.
- The file label is displayed in the Notes tables generated by procedures.
- A data file can only contain one file label. Subsequent FILE LABEL commands replace the label text.

Example

```
FILE LABEL Respondent's original data.
FILE LABEL "Respondent's original data".
FILE LABEL 'Respondent's original data.'
```

- The first two commands are functionally equivalent. The enclosing double-quotes in the second command are not included as part of the label text, and the apostrophe (single quote) is preserved.
- In the last command, everything after the apostrophe in *Respondent's* will be omitted from the label because the apostrophe will be interpreted as the closing single quote to match the opening single quote.

FILE TYPE-END FILE TYPE

For mixed file types:

```
FILE TYPE MIXED [FILE='file'] [ENCODING='encoding specification']
  RECORD=[varname] column location [(format)]

  [WILD={NOWARN**}]
        {WARN   }
```

For grouped file types:

```
FILE TYPE GROUPED [FILE='file'] [ENCODING='encoding specification']
  RECORD=[varname] column location [(format)]

 CASE=[varname] column location [(format)]

 [WILD={WARN   **}] [DUPLICATE={WARN   **}]
       {NOWARN   }             {NOWARN   }

 [MISSING={WARN   **}] [ORDERED={YES**}]
          {NOWARN   }           {NO   }
```

For nested file types:

```
FILE TYPE NESTED [FILE='file'] [ENCODING='encoding specification']
  RECORD=[varname] column location [(format)]

 [CASE=[varname] column location [(format)]]

 [WILD={NOWARN**}] [DUPLICATE={NOWARN**}]
       {WARN   }              {WARN   }
                                {CASE   }

 [MISSING={NOWARN**}]
          {WARN   }

END FILE TYPE
```

** Default if subcommand or keyword omitted.

Release History

Release 16.0

- ENCODING subcommand added for Unicode support.

Example

```
FILE TYPE  MIXED RECORD=RECID 1-2.
RECORD TYPE 23.
DATA LIST   /SEX 5 AGE 6-7 DOSAGE 8-10 RESULT 12.
END FILE TYPE.

BEGIN DATA
21  145010 1
22  257200 2
25  235  250  2
35  167          300    3
24  125150 1
23  272075 1
21  149050 2
25  134  035  3
30  138          300    3
32  229          500    3
END DATA.
```

Overview

The FILE TYPE-END FILE TYPE structure defines data for any one of the three types of complex raw data files: **mixed files**, which contain several types of records that define different types of cases; **hierarchical** or **nested files**, which contain several types of records with a defined relationship among the record types; or **grouped files**, which contain several records for each case with some records missing or duplicated. A fourth type of complex file, files with **repeating groups** of information, can be defined with the REPEATING DATA command.

FILE TYPE must be followed by at least one RECORD TYPE command and one DATA LIST command. Each pair of RECORD TYPE and DATA LIST commands defines one type of record in the data. END FILE TYPE signals the end of file definition.

Within the FILE TYPE structure, the lowest-level record in a nested file can be read with a REPEATING DATA command rather than a DATA LIST command. In addition, any record in a mixed file can be read with REPEATING DATA.

Basic Specification

The basic specification on FILE TYPE is one of the three file type keywords (MIXED, GROUPED, or NESTED) and the RECORD subcommand. RECORD names the record identification variable and specifies its column location. If keyword GROUPED is specified, the CASE subcommand is also required. CASE names the case identification variable and specifies its column location.

The FILE TYPE-END FILE TYPE structure must enclose at least one RECORD TYPE and one DATA LIST command. END FILE TYPE is required to signal the end of file definition.

- RECORD TYPE specifies the values of the record type identifier (see RECORD TYPE).
- DATA LIST defines variables for the record type specified on the preceding RECORD TYPE command (see DATA LIST).
- Separate pairs of RECORD TYPE and DATA LIST commands must be used to define each different record type.

The resulting active dataset is always a rectangular file, regardless of the structure of the original data file.

Syntax Rules

- For mixed files, if the record types have different variables or if they have the same variables recorded in different locations, separate RECORD TYPE and DATA LIST commands are required for each record type.
- For mixed files, the same variable name can be used on different DATA LIST commands, since each record type defines a separate case.
- For mixed files, if the same variable is defined for more than one record type, the format type and length of the variable should be the same on all DATA LIST commands. The program refers to the *first* DATA LIST command that defines a variable for the print and write formats to include in the dictionary of the active dataset.
- For grouped and nested files, the variable names on each DATA LIST must be unique, since a case is built by combining all record types together into a single record.
- For nested files, the order of the RECORD TYPE commands defines the hierarchical structure of the file. The first RECORD TYPE defines the highest-level record type, the next RECORD TYPE defines the next highest-level record, and so forth. The last RECORD TYPE command defines a case in the active dataset. By default, variables from higher-level records are spread to the lowest-level record.
- For nested files, the SPREAD subcommand on RECORD TYPE can be used to spread the values in a record type only to the *first* case built from each record of that type. All other cases associated with that record are assigned the system-missing value for the variables defined on that type. See RECORD TYPE for more information.
- String values specified on the RECORD TYPE command must be enclosed in quotes.

Operations

- For mixed file types, the program skips all records that are not specified on one of the RECORD TYPE commands.
- If different variables are defined for different record types in mixed files, the variables are assigned the system-missing value for those record types on which they are not defined.
- For nested files, the first record in the file should be the type specified on the first RECORD TYPE command—the highest level of the hierarchy. If the first record in the file is not the highest-level type, the program skips all records until it encounters a record of the highest-level type. If MISSING or

DUPLICATE has been specified, these records may produce warning messages but will not be used to build a case in the active dataset.

- When defining complex files, you are effectively building an input program and can use only commands that are allowed in the input state.

Examples

Reading multiple record types from a mixed file

```
FILE TYPE  MIXED FILE='/data/treatmnt.txt' RECORD=RECID 1-2.
+ RECORD TYPE 21,22,23,24.
+ DATA LIST   /SEX 5 AGE 6-7 DOSAGE 8-10 RESULT 12.
+ RECORD TYPE 25.
+ DATA LIST   /SEX 5 AGE 6-7 DOSAGE 10-12 RESULT 15.
END FILE TYPE.
```

- Variable *DOSAGE* is read from columns 8–10 for record types 21, 22, 23, and 24 and from columns 10–12 for record type 25. *RESULT* is read from column 12 for record types 21, 22, 23, and 24, and from column 15 for record type 25.
- The active dataset contains values for all variables defined on the DATA LIST commands for record types 21 through 25. All other record types are skipped.

Reading only one record type from a mixed file

```
FILE TYPE  MIXED RECORD=RECID 1-2.
RECORD TYPE 23.
DATA LIST   /SEX 5 AGE 6-7 DOSAGE 8-10 RESULT 12.
END FILE TYPE.

BEGIN DATA
21  145010 1
22  257200 2
25  235  250  2
35  167          300    3
24  125150 1
23  272075 1
21  149050 2
25  134  035  3
30  138          300    3
32  229          500    3
END DATA.
```

- FILE TYPE begins the file definition and END FILE TYPE indicates the end of file definition. FILE TYPE specifies a mixed file type. Since the data are included between BEGIN DATA-END DATA, the FILE subcommand is omitted. The record identification variable *RECID* is located in columns 1 and 2.
- RECORD TYPE indicates that records with value 23 for variable *RECID* will be copied into the active dataset. All other records are skipped. the program does not issue a warning when it skips records in mixed files.
- DATA LIST defines variables on records with the value 23 for variable *RECID*.

A grouped file of student test scores

```
FILE TYPE GROUPED RECORD=#TEST 6 CASE=STUDENT 1-4.
RECORD TYPE 1.
DATA LIST  /ENGLISH 8-9 (A).
RECORD TYPE 2.
DATA LIST /READING 8-10.
RECORD TYPE 3.
DATA LIST /MATH 8-10.
END FILE TYPE.

BEGIN DATA
0001 1 B+
0001 2  74
0001 3  83
0002 1 A
0002 2 100
0002 3  71
0003 1 B-
0003 2  88
0003 3  81
0004 1 C
0004 2  94
0004 3  91
END DATA.
```

- FILE TYPE identifies the file as a grouped file. As required for grouped files, all records for a single case are together in the data. The record identification variable *#TEST* is located in column 6. A scratch

variable is specified so it won't be saved in the active dataset. The case identification variable *STUDENT* is located in columns 1–4.

- Because there are three record types, there are three RECORD TYPE commands. For each RECORD TYPE, there is a DATA LIST to define variables on that record type.
- END FILE TYPE signals the end of file definition.
- The program builds four cases—one for each student. Each case includes the case identification variable plus the variables defined for each record type (the test scores). The values for *#TEST* are not saved in the active dataset. Thus, each case in the active dataset has four variables: *STUDENT*, *ENGLISH*, *READING*, and *MATH*.

A nested file of accident records

```
FILE TYPE NESTED RECORD=6 CASE=ACCID 1-4.
RECORD TYPE 1.
DATA LIST /ACC_ID 9-11 WEATHER 12-13 STATE 15-16 (A) DATE 18-24 (A).
RECORD TYPE 2.
DATA LIST /STYLE 11 MAKE 13 OLD 14 LICENSE 15-16(A) INSURNCE 18-21 (A).
RECORD TYPE 3.
DATA LIST /PSNGR_NO 11 AGE 13-14 SEX 16 (A) INJURY 18 SEAT 20-21 (A)
          COST 23-24.
END FILE TYPE.

BEGIN DATA
0001 1  322 1 IL 3/13/88   /* Type 1:  accident record
0001 2    1 44MI 134M      /* Type 2:    vehicle record
0001 3    1 34 M 1 FR  3   /* Type 3:      person record
0001 2    2 16IL 322F      /*            vehicle record
0001 3    1 22 F 1 FR 11   /*              person record
0001 3    2 35 M 1 FR  5   /*              person record
0001 3    3 59 M 1 BK  7   /*              person record
0001 2    3 21IN 146M      /*            vehicle record
0001 3    1 46 M 0 FR  0   /*              person record
END DATA.
```

- FILE TYPE specifies a nested file type. The record identifier, located in column 6, is not assigned a variable name, so the default scratch variable name *####RECD* is used. The case identification variable *ACCID* is located in columns 1–4.
- Because there are three record types, there are three RECORD TYPE commands. For each RECORD TYPE, there is a DATA LIST command to define variables on that record type. The order of the RECORD TYPE commands defines the hierarchical structure of the file.
- END FILE TYPE signals the end of file definition.
- The program builds a case for each lowest-level (type 3) record, representing each person in the file. There can be only one type 1 record for each type 2 record, and one type 2 record for each type 3 record. Each vehicle can be in only one accident, and each person can be in only one vehicle. The variables from the type 1 and type 2 records are spread to their corresponding type 3 records.

Specification Order

- FILE TYPE must be the first command in the FILE TYPE-END FILE TYPE structure. FILE TYPE subcommands can be named in any order.
- Each RECORD TYPE command must precede its corresponding DATA LIST command.
- END FILE TYPE must be the last command in the structure.

Types of Files

The first specification on FILE TYPE is a file type keyword, which defines the structure of the data file. There are three file type keywords: MIXED, GROUPED, and NESTED. Only one of the three types can be specified on FILE TYPE.

MIXED. *Mixed file type.* MIXED specifies a file in which each record type named on a RECORD TYPE command defines a case. You do not need to define all types of records in the file. In fact, FILE TYPE MIXED is useful for reading only one type of record because the program can decide whether to execute the DATA LIST for a record by simply reading the variable that identifies the record type.

GROUPED. *Grouped file type.* GROUPED defines a file in which cases are defined by grouping together record types with the same identification number. Each case usually has one record of each type. All records for a single case must be together in the file. By default, the program assumes that the records are in the same sequence within each case.

NESTED. *Nested file type.* NESTED defines a file in which the record types are related to each other hierarchically. The record types are grouped together by a case identification number that identifies the highest level—the first record type—of the hierarchy. Usually, the last record type specified—the lowest level of the hierarchy—defines a case. For example, in a file containing household records and records for each person living in the household, each person record defines a case. Information from higher record types may be *spread* to each case. For example, the value for a variable on the household record, such as *CITY*, can be spread to the records for each person in the household.

Subcommands and Their Defaults for Each File Type

The specifications on the FILE TYPE differ for each type of file. The following table shows whether each subcommand is required or optional and, where applicable, what the default specification is for each file type. N/A indicates that the subcommand is not applicable to that type of file.

Table 49. Summary of FILE TYPE subcommands for different file types

Subcommand	Mixed	Grouped	Nested
FILE	Conditional	Conditional	Conditional
RECORD	Required	Required	Required
CASE	Not Applicable	Required	Optional
WILD	NOWARN	WARN	NOWARN
DUPLICATE	N/A	WARN	NOWARN
MISSING	N/A	WARN	NOWARN
ORDERED	N/A	YES	N/A

- FILE is required unless data are inline (included between BEGIN DATA-END DATA).
- RECORD is always required.
- CASE is required for grouped files.
- The subcommands CASE, DUPLICATE, and MISSING can also be specified on the associated RECORD TYPE commands for grouped files. However, DUPLICATE=CASE is invalid.
- For nested files, CASE and MISSING can be specified on the associated RECORD TYPE commands.
- If the subcommands CASE, DUPLICATE, or MISSING are specified on a RECORD TYPE command, the specification on the FILE TYPE command (or the default) is overridden only for the record types listed on that RECORD TYPE command. The FILE TYPE specification or default applies to all other record types.

FILE Subcommand

FILE specifies a text file containing the data. FILE is not used when the data are inline.

Example

```
FILE TYPE  MIXED FILE='/data/treatmnt.txt' RECORD=RECID 1-2.
```

- Data are in the file *treatmnt.txt*. The file type is mixed. The record identification variable *RECID* is located in columns 1 and 2 of each record.

ENCODING Subcommand

ENCODING specifies the encoding format of the file. The keyword is followed by an equals sign and a quoted encoding specification.

- In Unicode mode, the default is UTF8. For more information, see SET command, UNICODE subcommand.
- In code page mode, the default is the current locale setting. For more information, see SET command, LOCALE subcommand.
- The quoted encoding value can be: `Locale` (the current locale setting), UTF8, UTF16, UTF16BE (big endian), UTF16LE (little endian), a numeric Windows code page value (for example, '1252'), or an IANA code page value (for example, 'iso8859-1' or cp1252).
- In Unicode mode, the defined width of string variables is tripled for code page and UTF-16 text data files. Use ALTER TYPE to automatically adjust the defined width of string variables.
- If there is no FILE subcommand, the ENCODING subcommand is ignored.

RECORD Subcommand

RECORD specifies the name and column location of the record identification variable.

- The column location of the record identifier is required. The variable name is optional.
- If you do not want to save the record type variable, you can assign a scratch variable name by using the # character as the first character of the name. If a variable name is not specified on RECORD, the record identifier is defined as the scratch variable *####RECD*.
- The value of the identifier for each record type must be unique and must be in the same location on all records. However, records do not have to be sorted according to type.
- A column-style format can be specified for the record identifier. For example, the following two specifications are valid:

```
RECORD=V1 1-2(N)
RECORD=V1 1-2(F,1)
```

FORTRAN-like formats cannot be used because the column location must be specified explicitly.

- Specify A in parentheses after the column location to define the record type variable as a string variable.

Example

```
FILE TYPE  MIXED FILE='/data/treatmnt.txt' RECORD=RECID 1-2.
```

- The record identifier is variable *RECID*, located in columns 1 and 2 of the hospital treatment data file.

CASE Subcommand

CASE specifies a name and column location for the case identification variable. CASE is required for grouped files and optional for nested files. It cannot be used with mixed files.

- For grouped files, each unique value for the case identification variable defines a case in the active dataset.
- For nested files, the case identification variable identifies the highest-level record of the hierarchy. The program issues a warning message for each record with a case identification number not equal to the case identification number on the last highest-level record. However, the record with the invalid case number is used to build the case.
- The column location of the case identifier is required. The variable name is optional.

- If you do not want to save the case identification variable, you can assign a scratch variable name by using the # character as the first character of the name. If a variable name is not specified on CASE, the case identifier is defined as the scratch variable *####CASE*.
- A column-style format can be specified for the case identifier. For example, the following two specifications are valid:

```
CASE=V1 1-2(N)
CASE=V1 1-2(F,1)
```

FORTRAN-like formats cannot be used because the column location must be specified explicitly.

- Specify A in parentheses after the column location to define the case identification variable as a string variable.
- If the case identification number is not in the same columns on all record types, use the CASE subcommand on the RECORD TYPE commands as well as on the FILE TYPE command (see RECORD TYPE).

Example

```
* A grouped file of student test scores.

FILE TYPE GROUPED RECORD=#TEST 6 CASE=STUDENT 1-4.
RECORD TYPE 1.
DATA LIST  /ENGLISH 8-9 (A).
RECORD TYPE 2.
DATA LIST /READING 8-10.
RECORD TYPE 3.
DATA LIST /MATH 8-10.
END FILE TYPE.

BEGIN DATA
0001 1 B+
0001 2  74
0001 3  83
0002 1 A
0002 2 100
0002 3  71
0003 1 B-
0003 2  88
0003 3  81
0004 1 C
0004 2  94
0004 3  91
END DATA.
```

- CASE is required for grouped files. CASE specifies variable *STUDENT*, located in columns 1–4, as the case identification variable.
- The data contain four different values for *STUDENT*. The active dataset therefore has four cases, one for each value of *STUDENT*. In a grouped file, each unique value for the case identification variable defines a case in the active dataset.
- Each case includes the case identification variable plus the variables defined for each record type. The values for *#TEST* are not saved in the active dataset. Thus, each case in the active dataset has four variables: *STUDENT, ENGLISH, READING*, and *MATH*.

Example

```
* A nested file of accident records.

FILE TYPE NESTED RECORD=6 CASE=ACCID 1-4.
RECORD TYPE 1.
DATA LIST   /ACC_ID 9-11 WEATHER 12-13 STATE 15-16 (A) DATE 18-24 (A).
RECORD TYPE 2.
DATA LIST /STYLE 11 MAKE 13 OLD 14 LICENSE 15-16 (A) INSURNCE 18-21 (A).
RECORD TYPE 3.
DATA LIST /PSNGR_NO 11 AGE 13-14 SEX 16 (A) INJURY 18 SEAT 20-21 (A)
          COST 23-24.
END FILE TYPE.

BEGIN DATA
0001 1  322 1 IL 3/13/88   /* Type 1:  accident record
0001 2    1 44MI 134M      /* Type 2:    vehicle record
0001 3    1 34 M 1 FR  3   /* Type 3:       person record
0001 2    2 16IL 322F      /*            vehicle record
0001 3    1 22 F 1 FR 11   /*               person record
0001 3    2 35 M 1 FR  5   /*               person record
0001 3    3 59 M 1 BK  7   /*               person record
0001 2    3 21IN 146M      /*            vehicle record
0001 3    1 46 M 0 FR  0   /*               person record
END DATA.
```

- CASE specifies variable *ACCID*, located in columns 1–4, as the case identification variable. *ACCID* identifies the highest level of the hierarchy: the level for the accident records.
- As each case is built, the value of the variable *ACCID* is checked against the value of *ACCID* on the last highest-level record (record type 1). If the values do not match, a warning message is issued. However, the record is used to build the case.
- The data in this example contain only one value for *ACCID*, which is spread across all cases. In a nested file, the lowest-level record type determines the number of cases in the active dataset. In this example, the active dataset has five cases because there are five person records.

Example

```
* Specifying case on the RECORD TYPE command.

FILE TYPE GROUPED FILE=HUBDATA RECORD=#RECID 80 CASE=ID 1-5.
RECORD TYPE 1.
DATA LIST   /MOHIRED YRHIRED 12-15 DEPT79 TO DEPT82 SEX 16-20.
RECORD TYPE 2.
DATA LIST   /SALARY79 TO SALARY82 6-25 HOURLY81 HOURLY82 40-53 (2)
            PROMO81 72  AGE 54-55 RAISE82 66-70.
RECORD TYPE 3  CASE=75-79.
DATA LIST   /JOBCAT 6 NAME 25-48 (A).
END FILE TYPE.
```

- The CASE subcommand on FILE TYPE indicates that the case identification number is located in columns 1–5. However, for type 3 records, the case identification number is located in columns 75–79. The CASE subcommand is therefore specified on the third RECORD TYPE command to override the case setting for type 3 records.
- The format of the case identification variable must be the same on all records. If the case identification variable is defined as a string on the FILE TYPE command, it cannot be defined as a numeric variable on the RECORD TYPE command, and vice versa.

WILD Subcommand

WILD determines whether the program issues a warning when it encounters undefined record types in the data file. Regardless of whether the warning is issued, undefined records are not included in the active dataset.

- The only specification on WILD is keyword WARN or NOWARN.
- WARN cannot be specified if keyword OTHER is specified on the last RECORD TYPE command to indicate all other record types (see RECORD TYPE).

WARN. *Issue warning messages.* The program displays a warning message and the first 80 characters of the record for each record type that is not mentioned on a RECORD TYPE command. This is the default for grouped file types.

NOWARN. *Suppress warning messages.* The program simply skips all record types not mentioned on a RECORD TYPE command and does not display warning messages. This is the default for mixed and nested file types.

Example

```
FILE TYPE  MIXED FILE='/data/treatmnt.txt' RECORD=RECID 1-2 WILD=WARN.
```

- WARN is specified on the WILD subcommand. The program displays a warning message and the first 80 characters of the record for each record type that is not mentioned on a RECORD TYPE command.

DUPLICATE Subcommand

DUPLICATE determines how the program responds when it encounters more than one record of each type for a single case. DUPLICATE is optional for grouped and nested files. DUPLICATE cannot be used with mixed files.

- The only specification on DUPLICATE is keyword WARN, NOWARN, or CASE.

WARN. *Issue warning messages.* The program displays a warning message and the first 80 characters of the last record of the duplicate set of record types. Only the *last* record from a set of duplicates is included in the active dataset. This is the default for grouped files.

NOWARN. *Suppress warning messages.* The program does not display warning messages when it encounters duplicate record types. Only the *last* record from a set of duplicates is included in the active dataset. This is the default for nested files.

CASE. *Build a case in the active dataset for each duplicate record.* The program builds one case in the active dataset for each duplicate record, spreading information from any higher-level records and assigning system-missing values to the variables defined on lower-level records. This option is available only for nested files.

Example

```
* A nested file of accident records.
* Issue a warning for duplicate record types.

FILE TYPE NESTED RECORD=6 CASE=ACCID 1-4 DUPLICATE=WARN.
RECORD TYPE 1.
DATA LIST   /ACC_ID 9-11 WEATHER 12-13 STATE 15-16 (A) DATE 18-24 (A).
RECORD TYPE 2.
DATA LIST /STYLE 11 MAKE 13 OLD 14 LICENSE 15-16 (A) INSURNCE 18-21 (A).
RECORD TYPE 3.
DATA LIST /PSNGR_NO 11 AGE 13-14 SEX 16 (A) INJURY 18 SEAT 20-21 (A)
          COST 23-24.
END FILE TYPE.

BEGIN DATA
0001 1  322 1 IL 3/13/88   /*          accident record
0001 2    1 44MI 134M      /*            vehicle record
0001 3    1 34 M 1 FR  3   /*               person record
0001 2    1 31IL 134M      /*  duplicate vehicle record
0001 2    2 16IL 322F      /*            vehicle record
0001 3    1 22 F 1 FR 11   /*               person record
0001 3    2 35 M 1 FR  5   /*               person record
0001 3    3 59 M 1 BK  7   /*               person record
0001 2    3 21IN 146M      /*            vehicle record
0001 3    1 46 M 0 FR  0   /*               person record
END DATA.
```

- In the data, there are two vehicle (type 2) records above the second set of person (type 3) records. This implies that an empty (for example, parked) vehicle was involved, or that each of the three persons was in two vehicles, which is impossible.
- DUPLICATE specifies keyword WARN. The program displays a warning message and the first 80 characters of the second of the duplicate set of type 2 records. The first duplicate record is skipped, and only the second is included in the active dataset. This assumes that no empty vehicles were involved in the accident.
- If the duplicate record represents an empty vehicle, it can be included in the active dataset by specifying keyword CASE on DUPLICATE. The program builds one case in the active dataset for the first duplicate record, spreading information to that case from the previous type 1 record and assigning system-missing values to the variables defined for type 3 records. The second record from the duplicate set is used to build the three cases for the associated type 3 records.

MISSING Subcommand

MISSING determines whether the program issues a warning when it encounters a missing record type for a case. Regardless of whether the program issues the warning, it builds the case in the active dataset with system-missing values for the variables defined on the missing record. MISSING is optional for grouped and nested files.

- MISSING cannot be used with mixed files and is optional for grouped and nested files.
- For grouped and nested files, the program verifies that each defined case includes one record of each type.
- The only specification is keyword WARN or NOWARN.

WARN. *Issue a warning message when a record type is missing for a case.* This is the default for grouped files.

NOWARN. *Suppress the warning message when a record type is missing for a case.* This is the default for nested files.

Example

```
* A grouped file with missing records.

FILE TYPE GROUPED RECORD=#TEST 6 CASE=STUDENT 1-4 MISSING=NOWARN.
RECORD TYPE 1.
DATA LIST  /ENGLISH 8-9 (A).
RECORD TYPE 2.
DATA LIST /READING 8-10.
RECORD TYPE 3.
DATA LIST /MATH 8-10.
END FILE TYPE.

BEGIN DATA
0001 1 B+
0001 2  74
0002 1 A
0002 2 100
0002 3  71
0003 3  81
0004 1 C
0004 2  94
0004 3  91
END DATA.
```

- The data contain records for three tests administered to four students. However, not all students took all tests. The first student took only the English and reading tests. The third student took only the math test.
- One case in the active dataset is built for each of the four students. If a student did not take a test, the system-missing value is assigned in the active dataset to the variable for the missing test. Thus, the first student has the system-missing value for the math test, and the third student has missing values for the English and reading tests.
- Keyword NOWARN is specified on MISSING. Therefore, no warning messages are issued for the missing records.

Example

```
* A nested file with missing records.

FILE TYPE NESTED RECORD=6 CASE=ACCID 1-4 MISSING=WARN.
RECORD TYPE 1.
DATA LIST   /ACC_ID 9-11 WEATHER 12-13 STATE 15-16 (A) DATE 18-24 (A).
RECORD TYPE 2.
DATA LIST /STYLE 11 MAKE 13 OLD 14 LICENSE 15-16 (A) INSURNCE 18-21 (A).
RECORD TYPE 3.
DATA LIST /PSNGR_NO 11 AGE 13-14 SEX 16 (A) INJURY 18 SEAT 20-21 (A)
          COST 23-24.
END FILE TYPE.

BEGIN DATA
0001 1  322 1 IL 3/13/88   /*          accident record
0001 3    1 34 M 1 FR  3   /*              person record
0001 2    2 16IL 322F      /*            vehicle record
0001 3    1 22 F 1 FR 11   /*               person record
0001 3    2 35 M 1 FR  5   /*               person record
0001 3    3 59 M 1 BK  7   /*               person record
0001 2    3 21IN 146M      /*            vehicle record
0001 3    1 46 M 0 FR  0   /*               person record
END DATA.
```

- The data contain records for one accident. The first record is a type 1 (accident) record, and the second record is a type 3 (person) record. However, there is no type 2 record, and therefore no vehicle associated with the first person. The person may have been a pedestrian, but it is also possible that the vehicle record is missing.
- One case is built for each person record. The first case has missing values for the variables specified on the vehicle record.
- Keyword WARN is specified on MISSING. A warning message is issued for the missing record.

ORDERED Subcommand

ORDERED indicates whether the records are in the same order as they are defined on the RECORD TYPE commands. Regardless of the order of the records in the data file and the specification on ORDERED, the program builds cases in the active dataset with records in the order defined on the RECORD TYPE commands.

- ORDERED can be used only for grouped files.
- The only specification is keyword YES or NO.
- If YES is in effect but the records are not in the order defined on the RECORD TYPE commands, the program issues a warning for each record that is out of order. The program still uses these records to build cases.

YES. *Records for each case are in the same order as they are defined on the RECORD TYPE commands.* This is the default.

NO. *Records are not in the same order within each case.*

Example

```
* A grouped file with records out of order.

FILE TYPE GROUPED RECORD=#TEST 6 CASE=STUDENT 1-4  MISSING=NOWARN
  ORDERED=NO.
RECORD TYPE 1.
DATA LIST  /ENGLISH 8-9 (A).
RECORD TYPE 2.
DATA LIST /READING 8-10.
RECORD TYPE 3.
DATA LIST /MATH 8-10.
END FILE TYPE.

BEGIN DATA
0001 2  74
0001 1 B+
0002 3  71
0002 2 100
0002 1 A
0003 2  81
0004 2  94
0004 1 C
0004 3  91
END DATA.
```

- The first RECORD TYPE command specifies record type 1, the second specifies record type 2, and the third specifies record type 3. However, records for each case are not always ordered type 1, type 2, and type 3.
- NO is specified on ORDERED. The program builds cases without issuing a warning that they are out of order in the data.
- Regardless of whether YES or NO is in effect for ORDERED, the program builds cases in the active dataset in the same order specified on the RECORD TYPE commands.

FILTER

```
FILTER  {BY var}
        {OFF   }
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
FILTER BY SEX.
FREQUENCIES BONUS.
```

Overview

FILTER is used to exclude cases from program procedures without deleting them from the active dataset. When FILTER is in effect, cases with a zero or missing value for the specified variable are not used in program procedures. Those cases are not actually deleted and are available again if the filter is turned off. To see the current filter status, use the SHOW command.

Basic Specification

The basic specification is keyword BY followed by a variable name. Cases that have a zero or missing value for the filter variable are excluded from subsequent procedures.

Syntax Rules

- Only one numeric variable can be specified. The variable can be one of the original variables in the data file or a variable computed with transformation commands.
- Keyword OFF turns off the filter. All cases in the active dataset become available to subsequent procedures.
- If FILTER is specified without a keyword, FILTER OFF is assumed but the program displays a warning message.
- FILTER can be specified anywhere in the command sequence. Unlike SELECT IF, FILTER has the same effect within an input program as it does outside an input program. Attention must be paid to the placement of any transformation command used to compute values for the filter variable (see INPUT PROGRAM).

Operations

- FILTER performs case selection without changing the active dataset. Cases that have a zero or missing value are excluded from subsequent procedures but are not deleted from the file.
- Both system-missing and user-missing values are treated as missing. The FILTER command does not offer options for changing selection criteria. To set up different criteria for exclusion, create a numeric variable and conditionally compute its values before specifying it on FILTER.
- If FILTER is specified after TEMPORARY, FILTER affects the next procedure only. After that procedure, the filter status reverts to whatever it was before the TEMPORARY command.
- The filter status does not change until another FILTER command is specified, a USE command is specified, or the active dataset is replaced.
- FILTER and USE are mutually exclusive. USE automatically turns off any previous FILTER command, and FILTER automatically turns off any previous USE command.
- If the specified filter variable is renamed, it is still in effect. The SHOW command will display the new name of the filter variable. However, the filter is turned off if the filter variable is recoded into a string variable or is deleted from the file.

- If the active dataset is replaced after a MATCH FILES, ADD FILES, or UPDATE command and the active dataset is one of the input files, the filter remains in effect if the new active dataset has a numeric variable with the name of the filter variable. If the active dataset does not have a numeric variable with that name (for example, if the filter variable was dropped or renamed), the filter is turned off.
- If the active dataset is replaced by an entirely new data file (for example, by a DATA LIST, GET, or IMPORT command), the filter is turned off.
- The FILTER command changes the filter status and takes effect when a procedure is executed or an EXECUTE command is encountered.

Examples

Filter by a variable with values of 0 and 1

```
FILTER BY SEX.
FREQUENCIES BONUS.
```

- This example assumes that *SEX* is a numeric variable, with male and female coded as 0 and 1, respectively. The FILTER command excludes males and cases with missing values for *SEX* from the subsequent procedures. The FREQUENCIES command generates a frequency table of *BONUS* for females only.

Recoding the filter variable to change the filter criterion

```
RECODE SEX (1=0)(0=1).
FILTER BY SEX.
FREQUENCIES BONUS.
```

- This example assumes the same coding scheme for *SEX* as the previous example. Before FILTER is specified, variable *SEX* is recoded. The FILTER command then excludes females and cases with missing values for *SEX*. The FREQUENCIES command generates a frequency table of *BONUS* for males only.

FINISH

```
FINISH
```

Overview

FINISH ends a prompted IBM SPSS Statistics Server session. It has no effect elsewhere.

FIT

```
FIT [[ERRORS=] residual series names]

    [/OBS=observed series names]

    [/{DFE=error degrees of freedom     }]
      {DFH=hypothesis degrees of freedom}
```

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
FIT.
```

Overview

FIT displays a variety of descriptive statistics computed from the residual series as an aid in evaluating the goodness of fit of one or more models.

Options

Statistical Output. You can produce statistics for a particular residual series by specifying the names of the series after FIT. You can also obtain percent error statistics for specified residual series by specifying observed series on the OBS subcommand.

Degrees of Freedom. You can specify the degrees of freedom for the residual series using the DFE or DFH subcommands.

Basic Specification

The basic specification is simply the command keyword FIT. All other specifications are optional.

- By default, FIT calculates the mean error, mean percent error, mean absolute error, mean absolute percent error, sum of squared errors, mean square error, root mean square error, and the Durbin-Watson statistic for the last *ERR_n* (residual) series generated and the corresponding observed series in the active dataset.
- If neither residual nor observed series are specified, percent error statistics for the default residual and observed series are included.

Syntax Rules

- If OBS is specified, the ERRORS subcommand naming the residual series is required.

Operations

- Observed series and degrees of freedom are matched with residual series according to the order in which they are specified.
- If residual series are explicitly specified but observed series are not, percent error statistics are not included in the output. If neither residual nor observed series are specified, percent error statistics for the default residual and observed series are included.
- If subcommand DFH is specified, FIT calculates the DFE (error degrees of freedom) by subtracting the DFH (hypothesis degrees of freedom) from the number of valid cases in the series.
- If a PREDICT period (validation period) starts before the end of the observed series, statistics are reported separately for the USE period (historical period) and the PREDICT period.

Limitations

- There is no limit on the number of residual series specified. However, the number of observed series must equal the number of residual series.

Example

```
FIT ERR_4 ERR_5 ERR_6.
```

- This command requests goodness-of-fit statistics for the residual series *ERR_4*, *ERR_5*, and *ERR_6*, which were generated by previous procedures. Percent error statistics are not included in the output, since only residual series are named.

ERRORS Subcommand

ERRORS specifies the residual (error) series.

- The actual keyword ERRORS can be omitted. VARIABLES is an alias for ERRORS.
- The minimum specification on ERRORS is a residual series name.
- The ERRORS subcommand is required if the OBS subcommand is specified.

OBS Subcommand

OBS specifies the observed series to use for calculating the mean percentage error and mean absolute percentage error.

- OBS can be used only when the residual series are explicitly specified.
- The number and order of observed series must be the same as that of the residual series.
- If more than one residual series was calculated from a single observed series, the observed series is specified once for each residual series that is based on it.

Example

```
FIT ERRORS=ERR#1 ERR#2
  /OBS=VAR1 VAR1.
```

- This command requests FIT statistics for two residual series, *ERR#1* and *ERR#2*, which were computed from the same observed series, *VAR1*.

DFE and DFH Subcommands

DFE and DFH specify the degrees of freedom for each residual series. With DFE, error degrees of freedom are entered directly. DFH specifies hypothesis degrees of freedom so FIT can compute the DFE.

- Only one DFE or DFH subcommand should be specified. If both are specified, only the last one is in effect.
- The specification on DFE or DFH is a list of numeric values. The order of these values should correspond to the order of the residual series list.
- The error degrees of freedom specified on DFE are used to compute the mean square error (MSE) and root mean square (RMS).
- The value specified for DFH should equal the number of parameters in the model (including the constant if it is present). Differencing is not considered in calculating DFH, since any observations lost due to differencing are system-missing.
- If neither DFE or DFH are specified, FIT sets DFE equal to the number of observations.

Example

```
FIT ERR#1 ERR#2
  /OBS=VAR1 VAR2
  /DFE=47 46.
```

- In this example, the error degrees of freedom for the first residual series, *ERR#1*, is 47. The error degrees of freedom for the second residual series, *ERR#2*, is 46.

Output Considerations for SSE

The sum of squared errors (SSE) reported by FIT may not be the same as the SSE reported by the estimation procedure. The SSE from the procedure is an estimate of sigma squared for that model. The SSE from FIT is simply the sum of the squared residuals.

References

Makridakis, S., S. C. Wheelwright, and V. E. McGee. 1983. *Forecasting: Methods and applications*. New York: John Wiley and Sons.

McLaughlin, R. L. 1984. *Forecasting techniques for decision making*. Rockville, Md.: Control Data Management Institute.

FLEISS MULTIRATER KAPPA

```
FLEISS MULTIRATER KAPPA {variable_list}
     [ /MISSING
          [ CLASSMISSING ={EXCLUDE**} {INCLUDE}] ]
     [ /CRITERIA
          [ ASYMPTOTICCILEVEL ={95**} {value}]
          [ IGNORECASE ={FALSE**} {TRUE}] ]
     [ /PRINT
          [ CATEGORY ]
          [ CASE({UPPER**} {LOWER}) ] ]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 26

- Command introduced

Overview

FLEISS MULTIRATER KAPPA assesses the interrater agreement to determine the reliability among the various raters. A higher agreement provides more confidence in the ratings reflecting the true circumstance, generalized the unweighted kappa statistic to measure the agreement among any constant number of raters while assuming:

- There is no connection between raters.
- The number of raters is a constant.
- Each subject is rated by the same group containing only a single rater.
- No weights can be assigned to the various disagreements.

FLEISS MULTIRATER KAPPA {variable_list} is a required command that invokes the procedure to estimate the Fleiss' multiple rater kappa statistics. The command names all the variables to be used in the FLEISS MULTIRATER KAPPA procedure.

- At least two item variables must be specified to run any reliability statistic.
- At least two ratings variables must be specified.
- The variables selected as items can also be selected as ratings.

Options

Fleiss' multiple-rater kappa
Enables or disables the Fleiss' multiple-rater kappa statistics.

Display agreement on individual categories
Specifies whether or not to output the agreement on individual categories. By default, the output suppresses the estimation on any individual categories. When enabled, multiple tables display in the output.

Ignore string cases
Controls whether or not the string variables are case sensitive. **/CRITERIA IGNORECASE=FALSE** is the default setting, which indicates that the strings are case sensitive. When **/CRITERIA IGNORECASE=TRUE** is specified, the strings are not case sensitive.

String category labels are displayed in uppercase
Controls whether the category labels in the output tables are displayed in uppercase or lowercase. The keyword takes effect only when **/CRITERIA IGNORECASE=TRUE**. **/PRINT [CASE(UPPER)]** is

the default setting, which displays the string category labels in uppercase. When **/PRINT [CASE(LOWER)]** is specified, the string category labels are all displayed in lowercase.

Asymptotic significance level (%)
Specifies the significance level for the asymptotic confidence intervals. The value must be a single double value in (0,100) with /CRITERIA ASYMPTOTIC_CILEVEL=95 as the default setting.

Exclude both user-missing and system missing values
Controls the exclusion of user-missing and system-missing values. By default **/MISSING CLASSMISSING=EXCLUDE** is the default setting, which excludes user-missing and system-missing values.

User-missing values are treated as valid
/MISSING CLASSMISSING=INCLUDE treats user-missing and system-missing values as valid data. The specification is applied to factors and categorical design variables.

Basic specification

FLEISS MULTIRATER KAPPA variable_list: Invokes the Fleiss' Multiple-Rater Kappa procedure.

Subcommand order

The subcommands can be named in any order.

Syntax rules

- At least two item variables must be selected to run any reliability statistic.
- When at least two ratings variables are selected, the **FLEISS MULTIRATER KAPPA** syntax is pasted.
- There is no connection between raters.
- The number of raters is a constant.
- Each subject is rated by the same group containing only a single rater.
- The variables selected as items can also be selected as ratings.

Limitation

- No weights can be assigned to the various disagreements.

MISSING Subcommand

MISSING specifies how missing values are handled. The optional **CLASSMISSING** keyword specifies whether user-missing values are treated as valid. The specification applies to factors and categorical design variables.

CLASSMISSING=EXCLUDE
Exclude user-missing values. User-missing values are excluded. This is the default.

CLASSMISSING=INCLUDE
Include user-missing values. User-missing values are included.

Example

```
FLEISS MULTIRATER KAPPA
     /VARIABLES=accid pop
     /MISSING CLASSMISSING=EXCLUDE.
```

CRITERIA Subcommand

CRITERIA controls the asymptotic confidence intervals and case sensitivity for string variables.

ASYMPTOTIC_CILEVEL
An optional keyword that specifies the significance level for the asymptotic confidence intervals. Must be a single double value in (**0,100**) with /CRITERIA ASYMPTOTIC_CILEVEL = 95 as the default.

IGNORE_CASE
An optional keyword that controls whether or not the string variables are case sensitive. **/CRITERIA IGNORECASE=FALSE** is the default setting, which indicates that the strings are case sensitive. When **/CRITERIA IGNORECASE=TRUE** is specified, the strings are not case sensitive.

Example

```
FLEISS MULTIRATER KAPPA
     /VARIABLES=accid pop
     /CRITERIA IGNORE_CASE=FALSE ASYMPTOTIC_CILEVEL=95.
```

PRINT Subcommand

PRINT controls the statistical display in the output.

CATEGORY
An optional keyword that specifies whether or not to output the agreement on individual categories. By default, the output suppresses the estimation on any individual categories. The setting takes effect only when a least two rating variables of the same type (string or numeric) are specified.

When the keyword is specified,both tables are displayed in the output.

CASE
An optional keyword that controls whether the category labels in the output tables are displayed in uppercase or lowercase. The keyword takes effect only when **/CRITERIA IGNORECASE=TRUE**. **/PRINT [CASE(UPPER)]** is the default setting, which displays the string category labels in uppercase. When **/PRINT [CASE(LOWER)]** is specified, the string category labels are all displayed in lowercase.

Example

```
FLEISS MULTIRATER KAPPA
     /VARIABLES=accid pop
     /PRINT [CASE(UPPER)].
```

FLIP

```
FLIP [VARIABLES= {ALL     }]
                 {varlist}

     [/NEWNAMES=variable]
```

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
FLIP.
```

Overview

The program requires a file structure in which the variables are the columns and observations (cases) are the rows. If a file is organized such that variables are in rows and observations are in columns, you need to use FLIP to reorganize it. FLIP transposes the rows and columns of the data in the active dataset so that, for example, row 1, column 2 becomes row 2, column 1, and so forth.

Options

Variable Subsets. You can transpose specific variables (columns) from the original file using the VARIABLES subcommand.

Variable Names. You can use the values of one of the variables from the original file as the variable names in the new file, using the NEWNAMES subcommand.

Basic Specification

The basic specification is the command keyword FLIP, which transposes all rows and columns.

- By default, FLIP assigns variable names *VAR001* to *VARn* to the variables in the new file. It also creates the new variable *CASE_LBL*, whose values are the variable names that existed before transposition.

Subcommand Order

VARIABLES must precede NEWNAMES.

Operations

- FLIP replaces the active dataset with the transposed file and displays a list of variable names in the transposed file.
- FLIP discards any previous VARIABLE LABELS, VALUE LABELS, and WEIGHT settings. Values defined as user-missing in the original file are translated to system-missing in the transposed file.
- FLIP obeys any SELECT IF, N, and SAMPLE commands in effect.
- FLIP does not obey the TEMPORARY command. Any transformations become permanent when followed by FLIP.
- String variables in the original file are assigned system-missing values after transposition.
- Numeric variables are assigned a default format of F8.2 after transposition (with the exceptions of *CASE_LBL* and the variable specified on NEWNAMES).
- The variable *CASE_LBL* is created and added to the active dataset each time FLIP is executed.
- If *CASE_LBL* already exists as the result of a previous FLIP, its current values are used as the names of variables in the new file (if NEWNAMES is not specified).

Example

The following is the LIST output for a data file arranged in a typical spreadsheet format, with variables in rows and observations in columns:

```
A             B         C         D

Income     22.00     31.00     43.00
Price      34.00     29.00     50.00
Year     1970.00   1971.00   1972.00
```

The command

```
FLIP.
```

transposes all variables in the file. The LIST output for the transposed file is as follows:

```
CASE_LBL    VAR001    VAR002    VAR003

A             .         .         .
B           22.00     34.00   1970.00
C           31.00     29.00   1971.00
D           43.00     50.00   1972.00
```

- The values for the new variable *CASE_LBL* are the variable names from the original file.
- Case A has system-missing values, since variable *A* had the string values *Income*, *Price*, and *Year*.
- The names of the variables in the new file are *CASE_LBL, VAR001, VAR002,* and *VAR003*.

VARIABLES Subcommand

VARIABLES names one or more variables (columns) to be transposed. The specified variables become observations (rows) in the new active dataset.

- The VARIABLES subcommand is optional. If it is not used, all variables are transposed.
- If the VARIABLES subcommand is specified, variables that are not named are discarded.

Example

Using the untransposed file from the previous example, the command

```
FLIP VARIABLES=A TO C.
```

transposes only variables *A* through *C*. Variable *D* is not transposed and is discarded from the active dataset. The LIST output for the transposed file is as follows:

```
CASE_LBL    VAR001    VAR002    VAR003

A             .         .         .
B           22.00     34.00   1970.00
C           31.00     29.00   1971.00
```

NEWNAMES Subcommand

NEWNAMES specifies a variable whose values are used as the new variable names.

- The NEWNAMES subcommand is optional. If it is not used, the new variable names are either *VAR001* to *VARn*, or the values of *CASE_LBL* if it exists.
- Only one variable can be specified on NEWNAMES.
- The variable specified on NEWNAMES does not become an observation (case) in the new active dataset, regardless of whether it is specified on the VARIABLES subcommand.
- If the variable specified is numeric, its values become a character string beginning with the prefix*K_*.

- Characters not allowed in variables names, such as blank spaces, are replaced with underscore (_) characters.
- If the variable's values are not unique, unique variable names are created by appending a sequential suffix of the general form _A, _B, _C,..._AA, _AB, _AC,..._AAA, _AAB, _AAC,...etc.

Example

Using the untransposed file from the first example, the command

```
FLIP NEWNAMES=A.
```

uses the values for variable *A* as variable names in the new file. The LIST output for the transposed file is as follows:

```
CASE_LBL    INCOME     PRICE      YEAR

B            22.00     34.00   1970.00
C            31.00     29.00   1971.00
D            43.00     50.00   1972.00
```

- Variable *A* does not become an observation in the new file. The string values for *A* are converted to upper case.

The following command transposes this file back to a form resembling its original structure:

```
FLIP.
```

The LIST output for the transposed file is as follows:

```
CASE_LBL       B         C         D

INCOME     22.00     31.00     43.00
PRICE      34.00     29.00     50.00
YEAR     1970.00   1971.00   1972.00
```

- Since the NEWNAMES subcommand is not used, the values of *CASE_LBL* from the previous FLIP (*B, C,* and *D*) are used as variable names in the new file.
- The values of *CASE_LBL* are now *INCOME*, *PRICE*, and *YEAR*.

FORMATS

```
FORMATS varlist(format) [varlist...]
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
FORMATS SALARY (DOLLAR8) / HOURLY (DOLLAR7.2) / RAISE BONUS (PCT2).
```

Overview

FORMATS changes variable print and write formats. In this program, print and write formats are *output* formats. **Print formats**, also called display formats, control the form in which values are displayed by a procedure or by the PRINT command; **write formats** control the form in which values are written by the WRITE command.

FORMATS changes both print and write formats. To change only print formats, use PRINT FORMATS. To change only write formats, use WRITE FORMATS. For information on assigning input formats during data definition, see DATA LIST.

For detailed information on available formats and specifications, see “Variable Types and Formats” on page 46.

Basic Specification

The basic specification is a variable list followed by a format specification in parentheses. All variables on the list receive the new format.

Operations

- Unlike most transformations, FORMATS takes effect as soon as it is encountered in the command sequence. Special attention should be paid to its position among commands. See the topic “Command Order” on page 36 for more information.
- Variables not specified on FORMATS retain their current print and write formats in the active dataset. To see the current formats, use the DISPLAY command.
- The new formats are changed only in the active dataset and are in effect for the duration of the current session or until changed again with a FORMATS, PRINT FORMATS, or WRITE FORMATS command. Formats in the original data file (if one exists) are not changed unless the file is resaved with the SAVE or XSAVE command.
- New numeric variables created with transformation commands are assigned default print and write formats of F8.2 (or the format specified on the FORMAT subcommand of SET). The FORMATS command can be used to change the new variable’s print and write formats.
- For string variables, you can only use FORMATS to switch between A and AHEX formats, and the AHEX length must be exactly twice the A length. FORMATS cannot be used to change the length of string variables. To change the defined length of a string variable, use the ALTER TYPE command.
- If a numeric data value exceeds its width specification, the program attempts to display some value nevertheless. The program first rounds decimal values, then removes punctuation characters, then tries scientific notation, and finally, if there is still not enough space, produces asterisks indicating that a value is present but cannot be displayed in the assigned width.

Syntax Rules

- You can specify more than one variable or variable list, followed by a format in parentheses. Only one format can be specified after each variable list. For clarity, each set of specifications can be separated by a slash.
- You can use keyword TO to refer to consecutive variables in the active dataset.
- The specified width of a format must include enough positions to accommodate any punctuation characters such as decimal points, commas, dollar signs, or date and time delimiters. (This differs from assigning an *input* format on DATA LIST, where the program automatically expands the input format to accommodate punctuation characters in output.)
- Custom currency formats (CCw, CCw.d) must first be defined on the SET command before they can be used on FORMATS.
- For string variables, you can only use FORMATS to switch between A and AHEX formats.FORMATS cannot be used to change the length of string variables. To change the length of a string variable, use ALTER TYPE.

Examples

Changing Formats for Multiple Variables

```
FORMATS SALARY (DOLLAR8) /HOURLY (DOLLAR7.2)
        /RAISE BONUS (PCT2).
```

- The print and write formats for *SALARY* are changed to DOLLAR format with eight positions, including the dollar sign and comma when appropriate. The value 11550 is displayed as $11,550. An eight-digit number would require a DOLLAR11 format: eight characters for the digits, two characters for commas, and one character for the dollar sign.
- The print and write formats for *HOURLY* are changed to DOLLAR format with seven positions, including the dollar sign, decimal point, and two decimal places. The value 115 is displayed as $115.00. If DOLLAR6.2 had been specified, the value 115 would be displayed as $115.0. The program would truncate the last 0 because a width of 6 is not enough to display the full value.
- The print and write formats for both *RAISE* and *BONUS* are changed to PCT with two positions: one position for the percentage and one position for the percent sign. The value 9 is displayed as 9%. Because the width allows for only two positions, the value 10 is displayed as 10, since the percent sign is truncated.

Changing Default Variable Formats

```
COMPUTE V3=V1 + V2.
FORMATS V3 (F3.1).
```

- COMPUTE creates the new numeric variable *V3*. By default, *V3* is assigned an F8.2 format (or the default format specified on SET).
- FORMATS changes both the print and write formats for *V3* to F3.1.

Working With Custom Currency Formats

```
SET CCA='-/-.Dfl ..-'.
FORMATS COST (CCA14.2).
```

- SET defines a European currency format for the custom currency format type CCA.
- FORMATS assigns format CCA to variable *COST*. With the format defined for CCA on SET, the value 37419 is displayed as Dfl 37.419,00. See the SET command for more information on custom currency formats.

FREQUENCIES

FREQUENCIES is available in Statistics Base Edition.

```
FREQUENCIES VARIABLES=varlist [varlist...]

 [/FORMAT= [{NOTABLE }] [{AVALUE**}]
           {LIMIT(n)}   {DVALUE  }
                        {AFREQ   }
                        {DFREQ   }

 [/MISSING=INCLUDE]

 [/BARCHART=[MINIMUM(n)] [MAXIMUM(n)] [{FREQ(n)   }]]
                                      {PERCENT(n)}

 [/PIECHART=[MINIMUM(n)] [MAXIMUM(n)] [{FREQ   }] [{MISSING  }]]
                                      {PERCENT}   {NOMISSING}

 [/HISTOGRAM=[MINIMUM(n)] [MAXIMUM(n)] [{FREQ(n)   }] [{NONORMAL}] ]
                                                       {NORMAL  }

 [/GROUPED=varlist [{(width)        }]]
                   {(boundary list)}

 [/NTILES=n]

 [/PERCENTILES=value list]

 [/STATISTICS=[DEFAULT] [MEAN] [STDDEV] [MINIMUM] [MAXIMUM]
              [SEMEAN] [VARIANCE] [SKEWNESS] [SESKEW] [RANGE]
              [MODE] [KURTOSIS] [SEKURT] [MEDIAN] [SUM] [ALL]
                [NONE]]

 [/ORDER=[{ANALYSIS}] [{VARIABLE}]
```

** Default if subcommand is omitted or specified without keyword.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
FREQUENCIES VARIABLES = RACE.
```

Overview

FREQUENCIES produces tables that show frequency counts and percentages of the values of individual variables. You can also use FREQUENCIES to obtain summary statistics tables for both categorical and continuous variables. .

Options

Display Format. You can suppress tables and alter the order of values within tables using the FORMAT subcommand.

Statistical Display. Percentiles and ntiles are available for numeric variables with the PERCENTILES and NTILES subcommands. The following statistics are available with the STATISTICS subcommand: mean, median, mode, standard deviation, variance, skewness, kurtosis, and sum.

Plots. Histograms can be specified for numeric variables on the HISTOGRAM subcommand. Bar charts can be specified for numeric or string variables on the BARCHART subcommand.

Input Data. On the GROUPED subcommand, you can indicate whether the input data are grouped (or collapsed) so that a better estimate can be made of percentiles.

Basic Specification

The basic specification is the VARIABLES subcommand and the name of at least one variable. By default, FREQUENCIES produces a Frequency table.

Subcommand Order

Subcommands can be named in any order.

Syntax Rules

- You can specify multiple NTILES subcommands.
- BARCHART and HISTOGRAM are mutually exclusive.
- You can specify numeric variables (with or without decimal values) or string variables.
- Keyword ALL can be used on VARIABLES to refer to all user-defined variables in the active dataset.

Operations

- Variables are tabulated in the order they are mentioned on the VARIABLES subcommand.
- If a requested ntile or percentile cannot be calculated, a period (.) is displayed.
- FREQUENCIES dynamically builds the table, setting up one cell for each unique value encountered in the data.

Limitations

- Maximum 1,000 variables total per FREQUENCIES command.

VARIABLES subcommand

VARIABLES names the variables to be tabulated and is the only required subcommand.

FORMAT subcommand

FORMAT controls various features of the output, including order of categories and suppression of tables.

- The minimum specification is a single keyword.
- By default, FREQUENCIES displays the Frequency table and sort categories in ascending order of values for numeric variables and in alphabetical order for string variables.

Table order

AVALUE. *Sort categories in ascending order of values (numeric variables) or in alphabetical order (string variables).* This is the default.

DVALUE. *Sort categories in descending order of values (numeric variables) or in reverse alphabetical order (string variables).* This is ignored when HISTOGRAM, NTILES, or PERCENTILES is requested.

AFREQ. *Sort categories in ascending order of frequency.* This is ignored when HISTOGRAM, NTILES, or PERCENTILES is requested.

DFREQ. *Sort categories in descending order of frequency.* This is ignored when HISTOGRAM, NTILES, or PERCENTILES is requested.

Table suppression

LIMIT(n). *Suppress frequency tables with more than* n *categories.* The number of missing and valid cases and requested statistics are displayed for suppressed tables.

NOTABLE. *Suppress all frequency tables.* The number of missing and valid cases are displayed for suppressed tables. NOTABLE overrides LIMIT.

BARCHART subcommand

BARCHART produces a bar chart for each variable named on the VARIABLES subcommand. By default, the horizontal axis for each bar chart is scaled in frequencies, and the interval width is determined by the largest frequency count for the variable being plotted. Bar charts are labeled with value labels or with the value itself if no label is defined.

Note: Charts are not produced in the output when bootstrapping is enabled for the **FREQUENCIES** command.

- The minimum specification is the BARCHART keyword, which generates default bar charts.
- BARCHART cannot be used with HISTOGRAM.

MIN(n). *Lower bound below which values are not plotted.*

MAX(n). *Upper bound above which values are not plotted.*

FREQ(n). *Vertical axis scaled in frequencies, where optional* n *is the maximum.* If *n* is not specified or if it is too small, FREQUENCIES chooses 5, 10, 20, 50, 100, 200, 500, 1000, 2000, and so forth, depending on the largest category. This is the default.

PERCENT(n) . *Vertical axis scaled in percentages, where optional* n *is the maximum.* If *n* is not specified or if it is too small, FREQUENCIES chooses 5, 10, 25, 50, or 100, depending on the frequency count for the largest category.

Producing a Basic Bar Chart

```
FREQUENCIES VARIABLES = RACE /BARCHART.
```

- FREQUENCIES produces a frequency table and the default bar chart for variable *RACE*.

Producing a Custom Bar Chart

```
FREQUENCIES VARIABLES = V1 V2 /BAR=MAX(10).
```

- FREQUENCIES produces a frequency table and bar chart with values through 10 for each of variables *V1* and *V2*.

PIECHART subcommand

PIECHART produces a pie chart for each variable named on the VARIABLES subcommand. By default, one slice corresponds to each category defined by the variable with one slice representing all missing values. Pie charts are labeled with value labels or with the value if no label is defined.

Note: Charts are not produced in the output when bootstrapping is enabled for the **FREQUENCIES** command.

- The minimum specification is the PIECHART keyword, which generates default pie charts.
- PIECHART can be requested together with either BARCHART or HISTOGRAM.
- FREQ and PERCENT are mutually exclusive. If both are specified, only the first specification is in effect.
- MISSING and NOMISSING are mutually exclusive. If both are specified, only the first specification is in effect.

MIN(n). *Lower bound below which values are not plotted.*

MAX(n). *Upper bound above which values are not plotted.*

FREQ. *The pie charts are based on frequencies.* Frequencies are displayed when you request values in the Chart Editor. This is the default.

PERCENT. *The pie charts are based on percentage.* Percentage is displayed when you request values in the Chart Editor.

MISSING. *User-missing and system-missing values are treated as one category.* This is the default. Specify INCLUDE on the MISSING subcommand to display system-missing and user-missing values as separate slices.

NOMISSING. *Missing values are excluded from the chart.* If you specify INCLUDE on the MISSING subcommand, each user-missing value is represented by one slice.

Producing a Basic Pie Chart

```
FREQUENCIES VARIABLES = RACE /PIECHART.
```

- FREQUENCIES produces a frequency table and the default pie chart for variable *RACE*.

Producing a Custom Pie Chart

```
FREQUENCIES VARIABLES = V1 V2 /PIE=MAX(10).
```

- For each variable *V1* and *V2*, FREQUENCIES produces a frequency table and a pie chart with values through 10.

HISTOGRAM subcommand

HISTOGRAM displays a plot for each numeric variable named on the VARIABLES subcommand. By default, the horizontal axis of each histogram is scaled in frequencies and the interval width is determined by the largest frequency count of the variable being plotted.

Note: Charts are not produced in the output when bootstrapping is enabled for the **FREQUENCIES** command.

- The minimum specification is the HISTOGRAM keyword, which generates default histograms.
- HISTOGRAM cannot be used with BARCHART.

MIN(n). *Lower bound below which values are not plotted.*

MAX(n). *Upper bound above which values are not plotted.*

FREQ(n). *Vertical axis scaled in frequencies, where optional* n *is the scale.* If *n* is not specified or if it is too small, FREQUENCIES chooses 5, 10, 20, 50, 100, 200, 500, 1000, 2000, and so forth, depending on the largest category. This is the default.

NORMAL. *Superimpose a normal curve.* The curve is based on all valid values for the variable, including values excluded by MIN and MAX.

NONORMAL. *Suppress the normal curve.* This is the default.

Example

```
FREQUENCIES VARIABLES = V1 /HIST=NORMAL.
```

- FREQUENCIES requests a histogram with a superimposed normal curve.

GROUPED subcommand

When the values of a variable represent grouped or collapsed data, it is possible to estimate percentiles for the original, ungrouped data from the grouped data. The GROUPED subcommand specifies which variables have been grouped. It affects only the output from the PERCENTILES and NTILES subcommands and the MEDIAN statistic from the STATISTICS subcommand.

- Multiple GROUPED subcommands can be used on a single FREQUENCIES command. Multiple variable lists, separated by slashes, can appear on a single GROUPED subcommand.
- The variables named on GROUPED must have been named on the VARIABLES subcommand.
- The value or value list in the parentheses is optional. When it is omitted, the program treats the values of the variables listed on GROUPED as midpoints. If the values are not midpoints, they must first be recoded with the RECODE command.
- A single value in parentheses specifies the width of each grouped interval. The data values must be group midpoints, but there can be empty categories. For example, if you have data values of 10, 20, and 30 and specify an interval width of 5, the categories are 10 ± 2.5, 20 ± 2.5, and 30 ± 2.5. The categories 15 ± 2.5 and 25 ± 2.5 are empty.

- A value list in the parentheses specifies interval boundaries. The data values do not have to represent midpoints, but the lowest boundary must be lower than any value in the data. If any data values exceed the highest boundary specified (the last value within the parentheses), they will be assigned to an open-ended interval. In this case, some percentiles cannot be calculated.

Basic Example

```
RECODE AGE (1=15) (2=25) (3=35) (4=45) (5=55)
          (6=65) (7=75) (8=85) (9=95)
  /INCOME  (1=5)  (2=15) (3=25) (4=35) (5=45)
          (6=55) (7=65) (8=75) (9=100).

FREQUENCIES VARIABLES=AGE, SEX, RACE, INCOME
  /GROUPED=AGE, INCOME
  /PERCENTILES=5,25,50,75,95.
```

- The *AGE* and *INCOME* categories of 1, 2, 3, and so forth are recoded to category midpoints. Note that data can be recoded to category midpoints on any scale; here *AGE* is recoded in years, but *INCOME* is recoded in thousands of dollars.
- The GROUPED subcommand on FREQUENCIES allows more accurate estimates of the requested percentiles.

Specifying the Width of Each Grouped Interval

```
FREQUENCIES VARIABLES=TEMP
  /GROUPED=TEMP (0.5)
  /NTILES=10.
```

- The values of *TEMP* (temperature) in this example were recorded using an inexpensive thermometer whose readings are precise only to the nearest half degree.
- The observed values of 97.5, 98, 98.5, 99, and so on, are treated as group midpoints, smoothing out the discrete distribution. This yields more accurate estimates of the deciles.

Specifying Interval Boundaries

```
FREQUENCIES VARIABLES=AGE
  /GROUPED=AGE (17.5, 22.5, 27.5, 32.5, 37.5, 42.5, 47.5
                52.5, 57.5, 62.5, 67.5, 72.5, 77.5, 82.5)
  /PERCENTILES=5, 10, 25, 50, 75, 90, 95.
```

- The values of *AGE* in this example have been estimated to the nearest five years. The first category is 17.5 to 22.5, the second is 22.5 to 27.5, and so forth. The artificial clustering of age estimates at multiples of five years is smoothed out by treating *AGE* as grouped data.
- It is not necessary to recode the ages to category midpoints, since the interval boundaries are explicitly given.

PERCENTILES subcommand

PERCENTILES displays the value below which the specified percentage of cases falls. The desired percentiles must be explicitly requested. There are no defaults.

Example

```
FREQUENCIES VARIABLES = V1 /PERCENTILES=10 25 33.3 66.7 75.
```

- FREQUENCIES requests the values for percentiles 10, 25, 33.3, 66.7, and 75 for *V1*.

NTILES subcommand

NTILES calculates the percentages that divide the distribution into the specified number of categories and displays the values below which the requested percentages of cases fall. There are no default ntiles.

- Multiple NTILES subcommands are allowed. Each NTILES subcommand generates separate percentiles. Any duplicate percentiles generated by different NTILES subcommands are consolidated in the output.

Basic Example

```
FREQUENCIES VARIABLES=V1 /NTILES=4.
```

- FREQUENCIES requests quartiles (percentiles 25, 50, and 75) for *V1*.

Working With Multiple NTILES Subcommands

```
FREQUENCIES VARIABLES=V1 /NTILES=4 /NTILES=10.
```

- The first NTILES subcommand requests percentiles 25, 50, and 75.
- The second NTILES subcommand requests percentiles 10 through 90 in increments of 10.
- The 50th percentile is produced by both specifications but is displayed only once in the output.

STATISTICS subcommand

STATISTICS controls the display of statistics. By default, cases with missing values are excluded from the calculation of statistics.

- The minimum specification is the keyword STATISTICS, which generates the mean, standard deviation, minimum, and maximum (these statistics are also produced by keyword DEFAULT).
- This subcommand is ignored for string variables.

MEAN. *Mean.*

SEMEAN. *Standard error of the mean*.

MEDIAN. *Median.* Ignored when AFREQ or DFREQ are specified on the FORMAT subcommand.

MODE. *Mode.* If there is more than one mode, only the first mode is displayed.

STDDEV. *Standard deviation*.

VARIANCE. *Variance.*

SKEWNESS. *Skewness.*

SESKEW. *Standard error of the skewness statistic*.

KURTOSIS. *Kurtosis.*

SEKURT. *Standard error of the kurtosis statistic*.

RANGE. *Range.*

MINIMUM. *Minimum.*

MAXIMUM. *Maximum.*

SUM. *Sum.*

DEFAULT. *Mean, standard deviation, minimum, and maximum.*

ALL. *All available statistics*.

NONE. *No statistics.*

Specifying a particular statistic

```
FREQUENCIES VARIABLES = AGE /STATS=MODE.
```

- STATISTICS requests the mode of *AGE*.

Including the Default Statistics

```
FREQUENCIES VARIABLES = AGE /STATS=DEF MODE.
```

- STATISTICS requests the default statistics (mean, standard deviation, minimum, and maximum) plus the mode of *AGE*.

MISSING subcommand

By default, both user-missing and system-missing values are labeled as missing in the table but are not included in the valid and cumulative percentages, in the calculation of descriptive statistics, or in charts and histograms.

INCLUDE. *Include cases with user-missing values.* Cases with user-missing values are included in statistics and plots.

ORDER subcommand

You can organize your output by variable or by analysis. Frequencies output that is organized by analysis has a single statistics table for all variables. Output organized by variable has a statistics table and a frequency table for each variable.

ANALYSIS. *Organize output by analysis.* Displays a single statistics table for all variables. This is the default.

VARIABLE. *Organize output by variable.* Displays a statistics table and a frequency table for each variable.

GENLIN

GENLIN is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

Note: Equals signs (=) used in the syntax chart are required elements. All subcommands are optional.

```
GENLIN
    {dependent-var  [(REFERENCE = {LAST**})] [(ORDER = {ASCENDING**})]}
                                  {FIRST }             {DESCENDING }
                                  {value }             {DATA       }

    {events-var OF {trials-var}                                      }
                   {n          }

    [BY factor-list [(ORDER = {ASCENDING**})]]
                              {DESCENDING }
                              {DATA       }

    [WITH covariate-list]

[/MODEL  [effect-list]    [INTERCEPT = {YES**}]
                                       {NO   }

    [OFFSET = {variable}]    [SCALEWEIGHT = variable]
              {value   }

    [DISTRIBUTION = {BINOMIAL        }]
                    {GAMMA           }
                    {IGAUSS          }
                    {MULTINOMIAL     }
                    {NEGBIN ({1**  })}
                             {value}
                             {MLE  }
                    {NORMAL          }
                    {POISSON         }
                    {TWEEDIE (value) }

    [LINK = {CLOGLOG         }]]
            {IDENTITY        }
            {LOG             }
            {LOGC            }
            {LOGIT           }
            {NEGBIN          }
            {NLOGLOG         }
            {ODDSPOWER(value)}
            {POWER(value)    }
            {PROBIT          }
            {CUMCAUCHIT      }
            {CUMCLOGLOG      }
            {CUMLOGIT        }
            {CUMNLOGLOG      }
            {CUMPROBIT       }

[/CRITERIA  [ANALYSISTYPE = {3**} [({WALD**})]   [CHECKSEP = {20**}]
                            {1  }   {LR    }                 {n   }
                            {ALL}

    [CILEVEL = {95** }]    [CITYPE = {WALD**             }]
               {value}                {PROFILE [({1E-4**})]}
                                               {value }

    [COVB = {MODEL**}]     [HCONVERGE = {0**  } [({ABSOLUTE**})]]
            {ROBUST }                   {value}   {RELATIVE  }

    [INITIAL = {number-list [NEGBIN({1**  })]}]
                            {value}
               {'savfile' | 'dataset'        }

    [LCONVERGE = {0**  } [({ABSOLUTE**})]]    [LIKELIHOOD = {FULL**}]
                 {value}   {RELATIVE  }                     {KERNEL}

    [MAXITERATIONS = {100**}]    [MAXSTEPHALVING = {5**}]
                     {n    }                      {n  }

    [METHOD = {FISHER ({1**})}]
                      {n  }
              {NEWTON        }

    [PCONVERGE = {1E-6**} [({ABSOLUTE**})]]
                 {value }   {RELATIVE  }

    [SCALE = {MLE**   }]
             {DEVIANCE}
             {PEARSON }
             {1**     }
             {value   }

    [SINGULAR = {1E-12**}]]
                {value  }

[/REPEATED SUBJECT = variable [* variable...]

    [WITHINSUBJECT = variable [* variable...]]

    [SORT = {YES**}]
            {NO   }

    [CORRTYPE = {INDEPENDENT**      }]
                {AR(1)              }
```

```
                {EXCHANGEABLE       }
                {FIXED (number-list)}
                {MDEPENDENT (n)     }
                {UNSTRUCTURED       }

    [ADJUSTCORR = {YES**}]
                  {NO   }

    [COVB = {ROBUST**}]
            {MODEL   }

    [HCONVERGE = {0**  }]
                 {value}

    [MAXITERATIONS = {100**}]
                     {n    }

    [PCONVERGE = {1E-6**} [({ABSOLUTE**})]]
                 {value }   {RELATIVE  }

    [UPDATECORR = {1**}]]
                  {n  }

[/EMMEANS [TABLES = factor [* factor ...]]

    [CONTROL = variable (value) [variable (value) ...]]

    [SCALE = {ORIGINAL** }]
             {TRANSFORMED}

    [COMPARE = factor [* factor ...]]

    [CONTRAST = {PAIRWISE**                }]
                {DEVIATION                 }
                {DIFFERENCE                }
                {HELMERT                   }
                {POLYNOMIAL ({1,2,3,...**})}
                            {number-list}
                {REPEATED                  }
                {SIMPLE   (value)          }

    [PADJUST = {LSD**        }]
               {BONFERRONI   }
               {SEQBONFERRONI}
               {SIDAK        }
               {SEQSIDAK     }

[/EMMEANS...]

[/MISSING CLASSMISSING = {EXCLUDE**}]
                         {INCLUDE  }

[/PRINT  [CORB] [COVB] [CPS**] [FIT**] [GEF] [HISTORY[({1**})]]
                                             {n  }
        [LAGRANGE] [LMATRIX] [MODELINFO**] [SOLUTION**[(EXPONENTIATED)]]
        [SUMMARY**] [DESCRIPTIVES**] [WORKINGCORR] [NONE]]

[/SAVE [XBPRED[({varname            })]] [XBSTDERROR[({varname            })]]
               {rootname[:{25**   }]}                {rootname[:{25**   }]}
                          {integer}                             {integer}
       [MEANPRED[({varname          })]]
               {rootname[:{25**   }]}
                          {integer}
       [CIMEANPREDL[({varname            })]] CIMEANPREDU[({varname            })]]
                    {rootname[:{25**   }]}                {rootname[:{25**   }]}
                               {integer}                             {integer}
       [PREDVAL[(varname)]]  [LEVERAGE[(varname)]]  [RESID[(varname)]]
       [PEARSONRESID[(varname)]] [DEVIANCERESID[(varname)]]
       [STDPEARSONRESID[(varname)]]  [STDDEVIANCERESID[(varname)]]
       [LIKELIHOODRESID[(varname)]]  [COOK[(varname)]]

/OUTFILE  {CORB = 'savfile' | 'dataset'}  {MODEL = 'file'    }
          {COVB = 'savfile' | 'dataset'}  {PARAMETER = 'file'}
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 15.0

- Command introduced.

Release 16.0

- Added multinomial and tweedie distributions; added MLE estimation option for ancillary parameter of negative binomial distribution (MODEL subcommand, DISTRIBUTION keyword). Notes related to the addition of the new distributions added throughout.
- Added cumulative Cauchit, cumulative complementary log-log, cumulative logit, cumulative negative log-log, and cumulative probit link functions (MODEL subcommand, LINK keyword).
- Added likelihood-ratio chi-square statistics as an alternative to Wald statistics (CRITERIA subcommand, ANALYSISTYPE keyword).

- Added profile likelihood confidence intervals as an alternative to Wald confidence intervals (CRITERIA subcommand, CITYPE keyword).
- Added option to specify initial value for ancillary parameter of negative binomial distribution (CRITERIA subcommand, INITIAL keyword).
- Changed default display of the likelihood function for GEEs to show the full value instead of the kernel (CRITERIA subcommand, LIKELIHOOD keyword).

Example

```
GENLIN mydepvar BY a b c WITH x y z
  /MODEL a b c x y z.
```

Overview

The GENLIN procedure fits the generalized linear model and generalized estimating equations.

The generalized linear model includes one dependent variable and usually one or more independent effects. Subjects are assumed to be independent. The generalized linear model covers not only widely used statistical models such as linear regression for normally distributed responses, logistic models for binary data, and loglinear models for count data, but also many other statistical models via its very general model formulation. However, the independence assumption prohibits the model from being applied to correlated data.

Generalized estimating equations extend the generalized linear model to correlated longitudinal data and clustered data. More particularly, generalized estimating equations model correlations within subjects. Data across subjects are still assumed independent.

Options

Independence Assumption. The GENLIN procedure fits either the generalized linear model assuming independence across subjects, or generalized estimating equations assuming correlated measurements within subjects but independence across subjects.

Events/Trials Specification for Binomial Distribution. The typical dependent variable specification will be a single variable, but for the binomial distribution the dependent variable may be specified using a number-of-events variable and a number-of-trials variable. Alternatively, if the number of trials is the same across all subjects, then trials may be specified using a fixed number instead of a variable.

Probability Distribution of Dependent Variable. The probability distribution of the dependent variable may be specified as normal, binomial, gamma, inverse Gaussian, multinomial, negative binomial, Poisson, or Tweedie.

Link Function. GENLIN offers the following link functions: Identity, complementary log-log, log, log-complement, logit, negative binomial, negative log-log, odds power, power, and probit. For the multinomial distribution, the following link functions are available: cumulative Cauchit, cumulative complementary log-log, cumulative logit, cumulative negative log-log, and cumulative probit.

Correlation Structure for Generalized Estimating Equations. When measurements within subjects are assumed correlated, the correlation structure may be specified as independent, AR(1), exchangeable, fixed, *m*-dependent, or unstructured.

Estimated Marginal Means. Estimated marginal means may be computed for one or more crossed factors and may be based on either the response or the linear predictor.

Basic Specification

- The basic specification is a MODEL subcommand with one or more model effects and a variable list identifying the dependent variable, the factors (if any), and the covariates (if any).
- If the MODEL subcommand is not specified, or is specified with no model effects, then the default model is the intercept-only model using the normal distribution and identity link.
- If the REPEATED subcommand is not specified, then subjects are assumed to be independent.

- If the REPEATED subcommand is specified, then generalized estimating equations, which model correlations within subjects, are fit. By default, generalized estimating equations use the independent correlation structure.
- The basic specification displays default output, including a case processing summary table, variable information, model information, goodness of fit statistics, model summary statistics, and parameter estimates and related statistics.

Syntax Rules

- The dependent variable, or an events/trials specification is required. All other variables and subcommands are optional.
- It is invalid to specify a dependent variable and an events/trials specification in the same GENLIN command.
- Multiple EMMEANS subcommands may be specified; each is treated independently. All other subcommands may be specified only once.
- The EMMEANS subcommand may be specified without options. All other subcommands must be specified with options.
- Each keyword may be specified only once within a subcommand.
- The command name, all subcommand names, and all keywords must be spelled in full.
- Subcommands may be specified in any order.
- Within subcommands, keyword settings may be specified in any order.
- The following variables, if specified, must be numeric: events and trials variables, covariates, OFFSET variable, and SCALEWEIGHT variable. The following, if specified, may be numeric or string variables: the dependent variable, factors, SUBJECT variables, and WITHINSUBJECT variables.
- All variables must be unique within and across the following variables or variable lists: the dependent variable, events variable, trials variable, factor list, covariate list, OFFSET variable, and SCALEWEIGHT variable.
- The dependent variable, events variable, trials variable, and covariates may not be specified as SUBJECT or WITHINSUBJECT variables.
- SUBJECT variables may not be specified as WITHINSUBJECT variables.
- The minimum syntax is a dependent variable. This specification fits an intercept-only model.

Case Frequency

- If an WEIGHT variable is specified, then its values are used as frequency weights by the GENLIN procedure.
- Weight values are rounded to the nearest whole numbers before use. For example, 0.5 is rounded to 1, and 2.4 is rounded to 2.
- The WEIGHT variable may not be specified on any subcommand in the GENLIN procedure.
- Cases with missing weights or weights less than 0.5 are not used in the analyses.

Limitations

- The TO keyword is not supported on variable lists in this procedure.

Variable List

The GENLIN command variable list specifies the dependent variable using either a single variable or events and trials variables. Alternatively, the number of trials may be specified as a fixed number. The variable list also specifies any factors and covariates.

If an events/trials specification is used for the dependent variable, then the GENLIN procedure automatically computes the ratio of the events variable over the trials variable or number. Technically, the procedure treats the events variable as the dependent variable in the sense that predicted values and residuals are based on the events variable rather than the events/trials ratio.

- The first specification on GENLIN must be a single dependent variable name or an events/trials specification.
- If the dependent variable is specified as a single variable, then it may be scale, an integer-valued count variable, binary, or ordinal.
- If the dependent variable is binary, then it may be numeric or string and there may be only two distinct valid data values.
- If the dependent variable is categorical, then it may be numeric or string and must have at least two distinct valid data values.
- If the dependent variable is not binary or categorical, then it must be numeric.

  REFERENCE Keyword

  The REFERENCE keyword specifies the dependent variable value to use as the reference category for parameter estimation. No model parameters are assigned to the reference category.

  LAST. *The last dependent variable value is the reference category.* The last dependent variable value is defined based on the ascending order of the data values. This is the default. If REFERENCE = LAST, then the procedure models the first value as the response, treating the last value as the reference category.

  FIRST. *The first dependent variable value is the reference category.* The first dependent variable value is defined based on the ascending order of the data values. If REFERENCE = FIRST, then the procedure models the last value as the response, treating the first value as the reference category.

  value. *The specified dependent variable value is the reference category.* Put the value inside a pair of quotes if it is formatted (such as date or time) or if the dependent variable is of string type; note, however, that this does not work for custom currency formats. If REFERENCE = value, then the procedure models the unspecified value as the response, treating the specified value as the reference category.

  – The REFERENCE specification is honored only if the dependent variable is binary and the binomial distribution is used (that is, DISTRIBUTION = BINOMIAL is specified on the MODEL subcommand). Otherwise, this specification is silently ignored.
  – If the dependent variable is a string variable, then the value at the highest or lowest level is locale-dependent.
  – If a value is specified as the reference category of the dependent variable, then the value must exist in the data.

  ORDER Keyword

  The ORDER keyword following the dependent variable is honored only if the dependent variable is categorical and the multinomial distribution is used (/MODEL DISTRIBUTION = MULTINOMIAL). Otherwise, this specification is silently ignored. ORDER determines the sort order of the dependent variable's values. Cumulative link functions are applied based on this order.

  ASCENDING. *Dependent variable values are sorted in ascending order, from the lowest value to the highest value.* This is the default.

  DATA. *Dependent variable values are not sorted.* The first value encountered in the data defines the first category, the last value encountered defines the last category. This option may not be specified if splits are defined on the SPLIT FILE command.

  DESCENDING. *Dependent variable values are sorted in descending order, from the highest value to the lowest value.*
- If the dependent variable is a string variable, then ascending and descending order are locale-dependent.
- If an events/trials specification is used, then the events variable must be specified first, followed by the 0F keyword, and then the trials variable or number.

- If an events/trials specification is used, then DISTRIBUTION = BINOMIAL must be specified on the MODEL subcommand. In this case, the procedure automatically computes the ratio of the events variable over the trials variable or number.
- The events and trials variables must be numeric.
- The events variable is usually the number of successes for each case. Data values must be nonnegative integers. Cases with invalid values are not used in the analysis.
- If a trials variable is specified, data values must be positive integers, and each value must be greater than or equal to the corresponding events value for a case. Cases with invalid values are not used in the analysis. If a number is specified, then it must be a positive integer, and it must be greater than or equal to the events value for each case. Cases with invalid values are not used in the analysis.
- The events and trials options are invalid if a dependent variable name is specified.
- The names of the factors and covariates, if any, follow the dependent variable or events/trials specification. Names of factors are specified following the keyword BY. Names of covariates are specified following the keyword WITH.

  ORDER Keyword

  The ORDER specification following a list of factor variable names determines the sort order of factor values. This order is relevant for determining a factor's last level, which may be associated with a redundant parameter in the estimation algorithm.

  ASCENDING. *Factor variable values are sorted in ascending order, from the lowest value to the highest value.* This is the default order.

  DATA. *Factor variable values are not sorted.* The first value encountered in the data defines the first category; the last value encountered defines the last category. This option may not be specified if splits are defined on the SPLIT FILE command.

  DESCENDING. *Factor variable values are sorted in descending order, from the highest value to the lowest value.*
- Covariates must be numeric, but factors can be numeric or string variables.
- Each variable may be specified only once on the variable list.
- The OFFSET and SCALEWEIGHT variables may not be specified on the GENLIN command variable list.
- The SUBJECT and WITHINSUBJECT variables may not be specified as dependent, events, or trials variables on the GENLIN command variable list.
- Cases with missing values on the dependent variable, the events or trials variable, or any covariate are not used in the analysis.

MODEL Subcommand

The MODEL subcommand is used to specify model effects, an offset or scale weight variable if either exists, the probability distribution, and the link function.

- The effect list includes all effects to be included in the model except for the intercept, which is specified using the INTERCEPT keyword. Effects must be separated by spaces or commas.
- If the multinomial distribution is used (DISTRIBUTION = MULTINOMIAL), then the intercept is inapplicable. The multinomial model always includes threshold parameters. The number of threshold parameters is one less than the number of dependent variable categories.
- To include a term for the main effect of a factor, enter the variable name of the factor.
- To include a term for an interaction between factors, use the keyword BY or an asterisk (*) to join the factors involved in the interaction. For example, A*B means a two-way interaction effect of A and B, where A and B are factors. A*A is not allowed because factors in an interaction effect must be distinct.
- To include a term for nesting one effect within another, use a pair of parentheses. For example, A(B) means that A is nested within B.

- Multiple nesting is allowed. For example, A(B(C)) means that B is nested within C, and A is nested within B(C). When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, A(B)(C) is not valid.
- Interactions between nested effects are not valid. For example, neither A(C)*B(C) nor A(C)*B(D) is valid.
- To include a covariate term in the design, enter the variable name of the covariate.
- Covariates can be connected, but not nested, through the * operator to form another covariate effect. Interactions among covariates such as X1*X1 and X1*X2 are valid, but X1(X2) is not.
- Factor and covariate effects can be connected only by the * operator. Suppose A and B are factors, and X1 and X2 are covariates. Examples of valid factor-by-covariate interaction effects are A*X1, A*B*X1, X1*A(B), A*X1*X1, and B*X1*X2.
- If the MODEL subcommand is not specified, or if it is specified with no model effects, then the GENLIN procedure fits the intercept-only model (unless the intercept is excluded on the INTERCEPT keyword). If the multinomial distribution is being used, then the GENLIN procedure fits the thresholds-only model.

  INTERCEPT Keyword

  The INTERCEPT keyword controls whether an intercept term is included in the model.

  – If the multinomial distribution is in effect (DISTRIBUTION = MULTINOMIAL), then the INTERCEPT keyword is silently ignored.

  YES. *The intercept is included in the model.* This is the default.

  NO. *The intercept is not included in the model.* If no model effects are defined and INTERCEPT = NO is specified, then a null model is fit.

  OFFSET Keyword

  The OFFSET keyword specifies an offset variable or fixes the offset at a number.

  – The offset variable, if specified, must be numeric.
  – The offset variable may not be a dependent variable, events or trials variable, factor, covariate, SCALEWEIGHT, SUBJECT, or WITHINSUBJECT variable.
  – Cases with missing values on the OFFSET variable are not used in the analysis.
  – Specifying a number when INTERCEPT = YES is equivalent to adding a constant to the intercept.
  – Specifying a number when INTERCEPT = NO is equivalent to fixing the intercept at the specified number.

  SCALEWEIGHT Keyword

  The SCALEWEIGHT keyword specifies a variable that contains omega weight values for the scale parameter.

  – The scale weight variable must be numeric.
  – The scale weight variable may not be a dependent variable, events or trials variable, factor, covariate, OFFSET, SUBJECT, or WITHINSUBJECT variable.
  – Cases with scale weight values that are less than or equal to 0, or missing, are not used in the analysis.

  DISTRIBUTION Keyword

  The DISTRIBUTION keyword specifies the probability distribution of the dependent variable.

  – The default probability distribution depends on the specification format of the dependent variable. If an events/trials specification is used, then the default distribution is BINOMIAL. If a single variable specification is used, then the default distribution is NORMAL.
  – Caution must be exercised when the dependent variable has events/trials format, and the LINK but not the DISTRIBUTION keyword is used. In this condition, depending on the LINK specification, the default DISTRIBUTION = BINOMIAL may yield an improper combination of DISTRIBUTION and LINK settings.

– Also, caution must be exercised when the dependent variable has single variable format, and the LINK but not the DISTRIBUTION keyword is used. In this condition, if the dependent variable is a string then an error will result because a string variable cannot have a normal probability distribution. Moreover, depending on the LINK specification, the default DISTRIBUTION = NORMAL may yield an improper combination of DISTRIBUTION and LINK settings.
– The discussion of the LINK keyword below gives details about proper and improper combinations of DISTRIBUTION and LINK settings.

BINOMIAL. *Binomial probability distribution.* If the dependent variable is specified as a single variable, then it may be numeric or string and there may be only two distinct valid data values.

If the events and trials options are specified on the GENLIN command, then the procedure automatically computes the ratio of the events variable over the trials variable or number. The events variable—and the trials variable if specified—must be numeric. Data values for the events variable must be integers greater than or equal to zero. Data values for the trials variable must be integers greater than zero. For each case, the trials value must be greater than or equal to the events value. If an events value is noninteger, less than zero, or missing, then the corresponding case is not used in the analysis. If a trials value is noninteger, less than or equal to zero, less than the events value, or missing, then the corresponding case is not used in the analysis.

If the trials option specifies a number, then it must be a positive integer, and it must be greater than or equal to the events value for each case. Cases with invalid values are not used in the analysis.

This is the default probability distribution if the dependent variable is specified using events/trials format.

GAMMA. *Gamma probability distribution.* The dependent variable must be numeric, with data values greater than zero. If a data value is less than or equal to zero, or missing, then the corresponding case is not used in the analysis.

IGAUSS. *Inverse Gaussian probability distribution.* The dependent variable must be numeric, with data values greater than zero. If a data value is less than or equal to zero, or missing, then the corresponding case is not used in the analysis.

MULTINOMIAL. *Multinomial probability distribution.* The dependent variable must be specified as a single variable, it may be numeric or string, and it must have at least two distinct valid data values. The dependent variable is assumed to be ordinal with values having an intrinsic ordering.

NEGBIN(number | MLE). *Negative binomial probability distribution.* The dependent variable must be numeric, with data values that are integers greater than or equal to zero. If a data value is noninteger, less than zero, or missing, then the corresponding case is not used in the analysis.

The option specifies the negative binomial distribution's ancillary parameter. Specify a number greater than or equal to zero to fix the parameter at the number. Specify MLE to use the maximum likelihood estimate of the parameter. The default value is 1.

If the REPEATED subcommand is specified, then the ancillary parameter is treated as follows. (1) If the ancillary parameter is specified as a number, then it is fixed at that number for the initial generalized linear model and the generalized estimating equations. (2) If the ancillary parameter is estimated using maximum likelihood in the initial generalized linear model, then the estimate is passed to the generalized estimating equations, where it is treated as a fixed number. (3) If NEGBIN(MLE) is specified but the initial generalized linear model is bypassed and initial values are directly input to the generalized estimating equations (see the discussion of Initial Values and Generalized Estimating Equations in "REPEATED Subcommand " on page 737), then the initial value of the ancillary parameter is passed to the generalized estimating equations, where it is treated as a fixed number.

NORMAL. *Normal probability distribution.* The dependent variable must be numeric. This is the default probability distribution if the dependent variable is specified using single-variable format.

POISSON. *Poisson probability distribution.* The dependent variable must be numeric, with data values that are integers greater than or equal to zero. If a data value is noninteger, less than zero, or missing, then the corresponding case is not used in the analysis.

TWEEDIE(number). *Tweedie probability distribution.* The dependent variable must be numeric, with data values greater than or equal to zero. If a data value is less than zero or missing, then the corresponding case is not used in the analysis.The required number specification is the fixed value of the Tweedie distribution's parameter. Specify a number greater than one and less than two. There is no default value.

LINK Keyword

The LINK keyword specifies the link function. The following link functions are available.

– If the multinomial distribution is in effect (DISTRIBUTION = MULTINOMIAL), then only the the cumulative link functions are available. Keywords corresponding to these functions have prefix CUM. If a non-cumulative link function is specified for the multinomial distribution, then an error is issued.

IDENTITY. *Identity link function.* $f(x)=x$

CLOGLOG. *Complementary log-log link function.* $f(x)=\ln(-\ln(1-x))$

LOG. *Log link function.* $f(x)=\ln(x)$

LOGC . *Log complement link function.* $f(x)=\ln(1-x)$

LOGIT . *Logit link function.* $f(x)=\ln(x / (1-x))$

NEGBIN. *Negative binomial link function.* $f(x)=\ln(x / (x+k^{-1}))$

NLOGLOG. *Negative log-log link function.* $f(x)=-\ln(-\ln(x))$

ODDSPOWER(number). *Odds power link function.* $f(x)=[(x/(1-x))^{\alpha}-1]/\alpha$, if $\alpha\neq 0$. $f(x)=\ln(x)$, if $\alpha=0$. α is the required number specification and must be a real number. There is no default value.

POWER(number). *Power link function.* $f(x)=x^{\alpha}$, if $\alpha\neq 0$. $f(x)=\ln(x)$, if $\alpha=0$. α is the required number specification and must be a real number. If $|\alpha| < 2.2e\text{-}16$, α is treated as 0. There is no default value.

PROBIT. *Probit link function.* $f(x)=\Phi^{-1}(x)$, where Φ^{-1} is the inverse standard normal cumulative distribution function.

CUMCAUCHIT. *Cumulative Cauchit link function.* $f(x) = \tan(\pi (x - 0.5))$. May be specified only if DISTRIBUTION = MULTINOMIAL.

CUMCLOGLOG. *Cumulative complementary log-log link function.* $f(x)=\ln(-\ln(1-x))$. May be specified only if DISTRIBUTION = MULTINOMIAL.

CUMLOGIT. *Cumulative logit link function.* $f(x)=\ln(x / (1-x))$. May be specified only if DISTRIBUTION = MULTINOMIAL.

CUMNLOGLOG. *Cumulative negative log-log link function.* $f(x)=-\ln(-\ln(x))$. May be specified only if DISTRIBUTION = MULTINOMIAL.

CUMPROBIT. *Cumulative probit link function.* $f(x)=\Phi^{-1}(x)$, where Φ^{-1} is the inverse standard normal cumulative distribution function. May be specified only if DISTRIBUTION = MULTINOMIAL.

- If neither the DISTRIBUTION nor the LINK keyword is specified, then the default link function is IDENTITY.
- If DISTRIBUTION is specified but LINK is not, then the default setting for LINK depends on the DISTRIBUTION setting as shown in the following table.

Table 50. Default link function for each distribution

DISTRIBUTION Setting	**Default LINK Setting**
NORMAL	IDENTITY
BINOMIAL	LOGIT
GAMMA	POWER(–1)
IGAUSS	POWER(–2)

Table 50. Default link function for each distribution (continued)	
DISTRIBUTION Setting	**Default LINK Setting**
MULTINOMIAL	CUMLOGIT
NEGBIN	LOG
POISSON	LOG
TWEEDIE	POWER(1–p), where p is the Tweedie distrubution's parameter

- The GENLIN procedure will fit a model if a permissible combination of LINK and DISTRIBUTION specifications is given. The table below indicates the permissible LINK and DISTRIBUTION combinations. Specifying an improper combination will yield an error message.
- Note that the default setting for DISTRIBUTION is NORMAL irrespective of the LINK specification, and that not all LINK specifications are valid for DISTRIBUTION = NORMAL. Thus, if LINK is specified but DISTRIBUTION is not, then the default DISTRIBUTION = NORMAL may yield an improper combination of DISTRIBUTION and LINK settings.

Table 51. Valid combinations of distribution and link function

Link	NORMAL	BINOMIAL	GAMMA	IGAUSS	NEGBIN	POISSON	TWEEDIE
IDENTITY	X	X	X	X	X	X	X
CLOGLOG		X					
LOG	X	X	X	X	X	X	X
LOGC		X					
LOGIT		X					
NEGBIN					X		
NLOGLOG		X					
ODDSPOWER		X					
PROBIT		X					
POWER	X	X	X	X	X	X	X

Note: The NEGBIN link function is not available if DISTRIBUTION = NEGBIN(0) is specified.

CRITERIA Subcommand

The CRITERIA subcommand controls statistical criteria for the generalized linear model and specifies numerical tolerance for checking singularity.

Note that if the REPEATED subcommand is used, then the GENLIN procedure fits generalized estimating equations, which comprise a generalized linear model and a working correlation matrix that models within-subject correlations. In this case, the GENLIN procedure first fits a generalized linear model assuming independence and uses the final parameter estimates as the initial values for the linear model part of the generalized estimating equations. (See the topic “REPEATED Subcommand ” on page 737 for more information.) The description of each CRITERIA subcommand keyword below is followed by a statement indicating how the keyword is affected by specification of the REPEATED subcommand.

ANALYSISTYPE = 3 | 1 | ALL (WALD | LR). *Type of analysis for each model effect.* Specify 1 for a type I analysis, 3 for type III analysis, or ALL for both. Each of these specifications computes chi-square statistics for each model effect. The default value is 3(WALD).

- Optionally, 1, 3, or ALL may be followed by WALD or LR in parentheses to specify the type of chi-square statistics to compute. WALD computes Wald statistics, LR computes likelihood-ratio statistics.
- If likelihood-ratio statistics are computed, then the log-likelihood convergence criterion is used in all reduced models if type I analysis is in effect , or in all constrained models if type III analysis is in effect, irrespective of the convergence criteria used for parameter estimation in the full model. That is, for reduced or constrained models, any HCONVERGE and PCONVERGE specifications are not used, but all LCONVERGE specifications are used. (See the discussions of the HCONVERGE, PCONVERGE, and LCONVERGE keywords below.) If the log-likelihood convergence criterion is not in effect for the full model, then the reduced or constrained models use the log-likelihood convergence criterion with tolerance level 1E-4 and absolute change. The maximum number of iterations (MAXITERATIONS), maximum number of step-halvings (MAXSTEPHALVING), and starting iteration for checking complete and quasi-complete separation (CHECKSEP) are the same for reduced or constrained models as for the full model.
- If the REPEATED subcommand is specified, then the option on the ANALYSISTYPE keyword is used for the generalized estimating equations. In this case, the WALD option computes Wald statistics, but the LR option computes generalized score statistics instead of likelihood-ratio statistics. For generalized score statistics, the convergence criteria for reduced or constrained models are the same as for the full model; that is, HCONVERGE or PCONVERGE as specified on the REPEATED subcommand.

CHECKSEP = integer. *Starting iteration for checking complete and quasi-complete separation.* Specify an integer greater than or equal to zero. This criterion is not used if the value is 0. The default value is 20. This criterion is used only for the binomial or multinomial probability distributions (that is, if DISTRIBUTION = BINOMIAL or MULTINOMIAL is specified on the MODEL subcommand). For all other probability distributions, it is silently ignored.

- If the CHECKSEP value is greater than 0 and the binomial or multinomial probability distribution is being used, then separation is always checked following the final iteration.
- If the REPEATED subcommand is specified, then the CHECKSEP keyword is applicable only to the initial generalized linear model.

CILEVEL = number. *Confidence interval level for coefficient estimates and estimated marginal means.* Specify a number greater than or equal to 0, and less than 100. The default value is 95.

- If the REPEATED subcommand is specified, then the CILEVEL keyword is applicable to any parameter that is fit in the process of computing the generalized estimating equations.

CITYPE = WALD | PROFILE(number). *Confidence interval type.* Specify WALD for Wald confidence intervals, or PROFILE for profile likelilhood confidence intervals. The default value is WALD.

- PROFILE may be followed optionally by parentheses containing the tolerance level used by the two convergence criteria. The default value is 1E-4.
- If the REPEATED subcommand is specified, then the CITYPE keyword is applicable only to the initial generalized linear model. For the linear model part of the generalized estimating equations, Wald confidence intervals are always used.

COVB = MODEL | ROBUST. *Parameter estimate covariance matrix.* Specify MODEL to use the model-based estimator of the parameter estimate covariance matrix, or ROBUST to use the robust estimator. The default value is MODEL.

- If the REPEATED subcommand is specified, then the CRITERIA subcommand COVB keyword is silently ignored. The REPEATED subcommand COVB keyword is applicable to the linear model part of the generalized estimating equations.

HCONVERGE = number (ABSOLUTE | RELATIVE). *Hessian convergence criterion.* Specify a number greater than or equal to 0, and the ABSOLUTE or RELATIVE keyword in parentheses to define the type of convergence. The number and keyword may be separated by a space character or a comma. If HCONVERGE = 0, the absolute Hessian convergence criterion will be checked with value 1E-4 after any specified convergence criteria have been satisfied. If it is not met, a warning is displayed. The default value is 0 (ABSOLUTE).

- At least one of the CRITERIA subcommand keywords HCONVERGE, LCONVERGE, PCONVERGE must specify a nonzero number.
- For a model with a normal distribution and identity link function, an iterative process is not used for parameter estimation. Thus, if DISTRIBUTION = NORMAL and LINK = IDENTITY on the MODEL subcommand, then the HCONVERGE keyword is silently ignored.
- If the REPEATED subcommand is specified, then the CRITERIA subcommand HCONVERGE keyword is applicable only to the initial generalized linear model. The REPEATED subcommand HCONVERGE keyword is applicable to the linear model part of the generalized estimating equations.

INITIAL = number-list | 'savfile' | 'dataset'. *Initial values for parameter estimates.* Specify a list of numbers, an external IBM SPSS Statistics data file, or a currently open dataset. If a list of numbers is specified, then each number must be separated by a space character or a comma. If an external data file is specified, then the full path and filename must be given in quotes.

- If the INITIAL keyword is specified, then initial values must be supplied for all parameters (including redundant parameters) in the generalized linear model. The ordering of the initial values should correspond to the ordering of the model parameters used by the GENLIN procedure. One way to determine how parameters are ordered for a given model is to run the GENLIN procedure for the model – without the INITIAL keyword – and examine the PRINT subcommand SOLUTION output.
- If INITIAL is not specified, then the GENLIN procedure automatically determines the initial values.
- If DISTRIBUTION = NORMAL and LINK = IDENTITY on the MODEL subcommand, then the INITIAL keyword is ignored with a warning.
- If the REPEATED subcommand is specified, then the CRITERIA subcommand INITIAL keyword is applicable only to the initial generalized linear model. See the REPEATED subcommand below for a detailed discussion of initial values and generalized estimating equations.

Initial Values Specified using a List of Numbers

For all distributions except multinomial, if MODEL INTERCEPT = YES, then the initial values must begin with the initial value for the intercept parameter. If MODEL INTERCEPT = NO, then the initial values must begin with the initial value for the first regression parameter.

If SCALE = MLE, then the initial values must continue with the initial value for the scale parameter. If SCALE = DEVIANCE, PEARSON, or a fixed number, then a value may be given for the scale parameter but it is optional and always silently ignored.

Finally, if DISTRIBUTION = NEGBIN(MLE), then the initial values may end with an initial value for the negative binomial distribution's ancillary parameter. The initial value for this parameter must be specified as NEGBIN(number), where number is a number greater than or equal to zero. The default value is 1. If DISTRIBUTION = NEGBIN(MLE) is not in effect, then NEGBIN(number) is silently ignored.

For the multinomial distribution, the ordering of initial values is: threshold parameters, regression parameters.

Any additional unused numbers at the end of the list; that is, any numbers beyond those that are mapped to parameters, are silently ignored.

If the SPLIT FILE command is in effect, then the exact same list is applied to all splits. That is, each split must have the same set of parameters, and the same list is applied to each split. If the list contains too few or too many numbers for any split, then an error message is displayed.

Initial Values Specified using a Dataset or External IBM SPSS Statistics Data File

If a currently open dataset or external IBM SPSS Statistics data file is specified, then the file structure must be the same as that used in the OUTFILE subcommand CORB and COVB files. This structure allows the final values from one run of the GENLIN procedure to be saved in a CORB or COVB file and input as initial values in a subsequent run of the procedure.

In the dataset, the ordering of variables from left to right must be: *RowType_*, *VarName_*, *P1*, *P2*, ... The variables *RowType_* and *VarName_* are string variables. *P1*, *P2*, ... are numeric variables corresponding to

an ordered list of the parameters. (Variable names *P1*, *P2*, ... are not required; the procedure will accept any valid variable names for the parameters. The mapping of variables to parameters is based on variable position, not variable name.) Any variables beyond the last parameter are ignored.

Initial values are supplied on a record with value 'EST' for variable *RowType_*; the actual initial values are given under variables *P1*, *P2*, ... The GENLIN procedure ignores all records for which *RowType_* has a value other than 'EST', as well as any records beyond the first occurrence of *RowType_* equal to 'EST'.

The required order of the intercept (if any) or threshold parameters, and regression parameters, is the same as for the list of numbers specification. However, when initial values are entered via a dataset or external data file, these parameters must always be followed by the scale parameter and then, if DISTRIBUTION = NEGBIN, by the negative binomial parameter.

If SPLIT FILE is in effect, then the variables must begin with the split-file variable or variables in the order specified on the SPLIT FILE command, followed by *RowType_*, *VarName_*, *P1*, *P2*, ... as above. Splits must occur in the specified dataset in the same order as in the original dataset.

Examples

```
GENLIN depvar BY a WITH x
  /MODEL a x
  /CRITERIA INITIAL = 1 1.5 2.5 0 3.
```

The next example outputs the final estimates from one run of the GENLIN procedure and inputs these estimates as the initial values in the second run.

```
GENLIN depvar BY a WITH x
  /MODEL a x
  /OUTFILE COVB = '/work/estimates.sav'.
GENLIN depvar BY a WITH x
  /MODEL a x
  /CRITERIA INITIAL = '/work/estimates.sav'.
```

LCONVERGE = number (ABSOLUTE | RELATIVE). *Log-likelihood convergence criterion.* Specify a number greater than or equal to 0, and the ABSOLUTE or RELATIVE keyword in parentheses to define the type of convergence. The number and keyword may be separated by a space character or a comma. The log-likelihood convergence criterion is not used if the number is 0. The default value is 0 (ABSOLUTE).

- At least one of the CRITERIA subcommand keywords HCONVERGE, LCONVERGE, PCONVERGE must specify a nonzero number.
- If DISTRIBUTION = NORMAL and LINK = IDENTITY on the MODEL subcommand, then the LCONVERGE keyword is silently ignored.
- If the REPEATED subcommand is specified, then the LCONVERGE keyword is applicable only to the initial generalized linear model.

LIKELIHOOD = FULL | KERNEL. *Form of the log-likelihood or log-quasi-likelihood function.* Specify FULL for the full function, or KERNEL for the kernel of the function. The default value is FULL.

- For generalized linear models, the LIKELIHOOD keyword specifies the form of the log likelihood function. If the REPEATED subcommand is specified, then it specifies the form of the log quasi-likelihood function.

MAXITERATIONS = integer. *Maximum number of iterations.* Specify an integer greater than or equal to 0. The default value is 100.If DISTRIBUTION = NORMAL and LINK = IDENTITY on the MODEL subcommand, then the MAXITERATIONS keyword is silently ignored.

- If the REPEATED subcommand is specified, then the CRITERIA subcommand MAXITERATIONS keyword is applicable only to the initial generalized linear model. The REPEATED subcommand MAXITERATIONS keyword is applicable to the linear model part of the generalized estimating equations.

MAXSTEPHALVING = integer. *Maximum number of steps in step-halving method.* Specify an integer greater than 0. The default value is 5.If DISTRIBUTION = NORMAL and LINK = IDENTITY on the MODEL subcommand, then the MAXSTEPHALVING keyword is silently ignored.

- If the REPEATED subcommand is specified, then the MAXSTEPHALVING keyword is applicable only to the initial generalized linear model.

METHOD = FISHER | NEWTON | FISHER(integer). *Model parameters estimation method.* Specify FISHER to use the Fisher scoring method, NEWTON to use the Newton-Raphson method, or FISHER(integer) to use a hybrid method.

- In the hybrid method option, *integer* is an integer greater than 0 and specifies the maximum number of Fisher scoring iterations before switching to the Newton-Raphson method. If convergence is achieved during the Fisher scoring phase of the hybrid method, then additional Newton-Raphson steps are performed until convergence is achieved for Newton-Raphson too.
- The default algorithm for the generalized linear model uses Fisher scoring in the first iteration and Newton-Raphson thereafter; the default value for the METHOD keyword is FISHER(1).
- If DISTRIBUTION = NORMAL and LINK = IDENTITY on the MODEL subcommand, then the METHOD keyword is silently ignored.
- If the REPEATED subcommand is specified, then the METHOD keyword is applicable only to the initial generalized linear model.

PCONVERGE = number (ABSOLUTE | RELATIVE). *Parameter convergence criterion.* Specify a number greater than or equal to 0, and the ABSOLUTE or RELATIVE keyword in parentheses to define the type of convergence. The number and keyword may be separated by a space character or a comma. The parameter convergence criterion is not used if the number is 0. The default value is 1E-6 (ABSOLUTE).

- At least one of the CRITERIA subcommand keywords HCONVERGE, LCONVERGE, PCONVERGE must specify a nonzero number.
- If DISTRIBUTION = NORMAL and LINK = IDENTITY on the MODEL subcommand, then the PCONVERGE keyword is silently ignored.
- If the REPEATED subcommand is specified, then the CRITERIA subcommand PCONVERGE keyword is applicable only to the initial generalized linear model. The REPEATED subcommand PCONVERGE keyword is applicable to the linear model part of the generalized estimating equations.

SCALE = MLE | DEVIANCE | PEARSON | number. *Method of fitting the scale parameter.* Specify MLE to compute a maximum likelihood estimate, DEVIANCE to compute the scale parameter using the deviance, PEARSON to compute it using the Pearson chi-square, or a number greater than 0 to fix the scale parameter.

If the MODEL subcommand specifies DISTRIBUTION = NORMAL, IGAUSS, GAMMA, or TWEEDIE then any of the SCALE options may be used. For these distributions, the default value is MLE.

If the MODEL subcommand specifies DISTRIBUTION = NEGBIN, POISSON, BINOMIAL, or MULTINOMIAL, then DEVIANCE, PEARSON, or a fixed number may be used. For these distributions, the default value is the fixed number 1.

If the REPEATED subcommand is specified, then the SCALE keyword is directly applicable only to the initial generalized linear model. For the linear model part of the generalized estimating equations, the scale parameter is treated as follows:

- If SCALE = MLE, then the scale parameter estimate from the initial generalized linear model is passed to the generalized estimating equations, where it is updated by the Pearson chi-square divided by its degrees of freedom.
- If SCALE = DEVIANCE or PEARSON, then the scale parameter estimate from the initial generalized linear model is passed to the generalized estimating equations, where it is treated as a fixed number.
- If SCALE is specified with a fixed number, then the scale parameter is also held fixed at the same number in the generalized estimating equations.

SINGULAR = number. *Tolerance value used to test for singularity.* Specify a number greater than 0. The default value is 1E-12.

- If the REPEATED subcommand is specified, then the SINGULAR keyword is applicable to any linear model that is fit in the process of computing the generalized estimating equations.

REPEATED Subcommand

The REPEATED subcommand specifies the correlation structure used by generalized estimating equations to model correlations within subjects and controls statistical criteria in the nonlikelihood-based iterative fitting algorithm. If the REPEATED subcommand is not specified, then the GENLIN procedure fits a generalized linear model assuming independence.

Initial Values and Generalized Estimating Equations

Generalized estimating equations require initial values for the parameter estimates in the linear model. Initial values are not needed for the working correlation matrix because matrix elements are based on the parameter estimates.

The GENLIN procedure automatically supplies initial values to the generalized estimating equations algorithm. The default initial values are the final parameter estimates from the ordinary generalized linear model, assuming independence, that is fit based on the MODEL and CRITERIA subcommand specifications.

Recall that if the REPEATED subcommand is specified, then the CRITERIA subcommand SCALE keyword is directly applicable only to the initial generalized linear model. For the linear model part of the generalized estimating equations, the scale parameter is treated as follows.

- If SCALE = MLE, then the scale parameter estimate from the initial generalized linear model is passed to the generalized estimating equations, where it is updated by the Pearson chi-square divided by its degrees of freedom. Pearson chi-square is used because generalized estimating equations do not have the concept of likelihood, and hence the scale estimate cannot be updated by methods related to maximum likelihood estimation.
- If SCALE = DEVIANCE or PEARSON, then the scale parameter estimate from the initial generalized linear model is passed to the generalized estimating equations, where it is treated as a fixed number.
- If SCALE is specified with a fixed number, then the scale parameter is also held fixed in the generalized estimating equations.

It is possible to bypass fitting the generalized linear model and directly input initial values to the generalized estimating equations algorithm. To do this, specify the linear model as usual on the MODEL subcommand. Then, on the CRITERIA subcommand, specify initial values for the linear model on the INITIAL keyword and set MAXITERATIONS = 0.

For example, suppose factor A has three levels. The INITIAL keyword supplies initial value 1 for the intercept, 1.5 for the first level of factor A, 2.5 for the second level, 0 for the last level, and 3 for the covariate X. Because MAXITERATIONS = 0, no iterations are performed for the generalized linear model and the specified initial values are passed directly to the generalized estimating equations algorithm.

```
GENLIN depvar BY a WITH x
  /MODEL a x
    DISTRIBUTION = BINOMIAL
    LINK = LOGIT
    INITIAL = 1 1.5 2.5 0 3
    MAXITERATIONS = 0
  /REPEATED
    SUBJECT=idvar.
```

It is also possible to use a maximum likelihood estimate of the scale parameter as the initial value and to fix the scale parameter at this initial value for the generalized estimating equations. That is, we can override the default updating by the Pearson chi-square divided by its degrees of freedom. To do this, first fit a generalized linear model, estimating the scale parameter via maximum likelihood, and save the final parameter estimates in an external file (using the OUTFILE subcommand CORB or COVB option). Next, open this external file and copy the scale parameter estimate in full precision. Finally, fit the generalized estimating equations, using the final parameter estimates from the generalized linear model as the initial values, with MAXITERATIONS = 0 on the CRITERIA subcommand and SCALE fixed at the scale parameter estimate on the CRITERIA subcommand. The following example syntax assumes that the maximum likelihood estimate of the scale parameter is 0.1234567890123456.

```
GENLIN depvar BY a WITH x
  /MODEL a x
     DISTRIBUTION = NORMAL
```

```
      LINK = LOG
   /CRITERIA
      SCALE = MLE
   /OUTFILE
      COVB = '/work/estimates.sav'.

GENLIN depvar BY a WITH x
   /MODEL a x
      DISTRIBUTION = NORMAL
      LINK = LOG
   /CRITERIA
      INITIAL = '/work/estimates.sav'
      MAXITERATIONS = 0
      SCALE = 0.1234567890123456
   /REPEATED
      SUBJECT=idvar.
```

When the negative binomial distribution is used (`/MODEL DISTRIBUTION = NEGBIN`), the distribution's ancillary parameter is treated as follows:

1. If the ancillary parameter is specified as a number, then it is fixed at that number for the initial generalized linear model and the generalized estimating equations.
2. If the ancillary parameter is estimated using maximum likelihood in the initial generalized linear model, then the estimate is passed to the generalized estimating equations, where it is treated as a fixed number.
3. If `NEGBIN(MLE)` is specified but the initial generalized linear model is bypassed and initial values are directly input to the generalized estimating equations, then the initial value of the ancillary parameter is passed to the generalized estimating equations, where it is treated as a fixed number.

SUBJECT Keyword

The `SUBJECT` keyword identifies subjects in the active dataset. Complete independence is assumed *across* subjects, but responses *within* subjects are assumed to be correlated.

- Specify a single variable or a list of variables connected by asterisks (*) or the keyword BY.
- Variables may be numeric or string variables.
- The number of subjects equals the number of distinct combinations of values of the variables.
- If the active dataset is sorted by the subject variables, then all records with equal values on the subject variables are contiguous and define the measurements for one subject.
- In contrast, if the active dataset is not sorted, then the `GENLIN` procedure reads the data record by record. Each block of equal values on the subject variables defines a new subject. Please be aware that this approach may produce invalid results if all records for a subject are not contiguous in the active dataset.
- By default, the `GENLIN` procedure automatically sorts the active dataset by subject and any within-subject variables before performing analyses. See the `SORT` keyword below for more information.
- All specified variables must be unique.
- The dependent, events, trials, and `WITHINSUBJECT` variables may not be specified as `SUBJECT` variables.
- The `SUBJECT` keyword is required if the `REPEATED` subcommand is used.
- Cases with missing values for any of the subject variables are not used in the analysis.

WITHINSUBJECT Keyword

The `WITHINSUBJECT` keyword gives the within-subject or time effect. This effect defines the ordering of measurements within subjects. If some measurements do not appear in the data for some subjects, then the existing measurements are ordered and the omitted measurements are treated as missing values. If `WITHINSUBJECT` is not used, then measurements may be improperly ordered and missing values assumed for the last measurements within subjects.

- Specify a single variable or a list of variables connected by asterisks (*) or the keyword BY.
- Variables may be numeric or string variables.
- The `WITHINSUBJECT` keyword is honored only if the default `SORT = YES` is in effect. The number of measurements within a subject equals the number of distinct combinations of values of the `WITHINSUBJECT` variables.

- The WITHINSUBJECT keyword is ignored and a warning is issued if SORT = NO is in effect. In this case, the GENLIN procedure reads the records for a subject in the order given in the active dataset.
- By default, the GENLIN procedure automatically sorts the active dataset by subject and any within-subject variables before performing analyses. See the SORT keyword below for more information.
- All specified variables must be unique.
- The dependent, events, trials, and SUBJECT variables may not be specified as WITHINSUBJECT variables.
- The WITHINSUBJECT keyword is not required if the data are properly ordered within each subject.
- Cases with missing values for any of the within-subject variables are not used in the analysis.

SORT Keyword

The SORT keyword indicates whether to sort cases in the working dataset by the subject effect and the within-subject effect.

YES. *Sort cases by subject and any within-subject variables.* The GENLIN procedure sorts the active dataset before performing analyses. The subject and any within-subject variables are sorted based on the ascending sort order of their data values. If any of the variables are strings, then their sort order is locale-dependent. This is the default. This sort is temporary—it is in effect only for the duration of the GENLIN procedure.

NO. *Do not sort cases by subject and any within-subject variables.* If SORT = NO is specified, then the GENLIN procedure does not sort the active dataset before performing analyses.

CORRTYPE Keyword

The CORRTYPE keyword specifies the working correlation matrix structure.

INDEPENDENT. Independent working correlation matrix. This is the default working correlation matrix structure.

AR(1). AR(1) working correlation matrix.

EXCHANGEABLE. Exchangeable working correlation matrix.

FIXED(list). Fixed working correlation matrix. Specify a list of numbers, with each number separated by a space character or a comma.

The list of numbers must define a valid working correlation matrix. The number of rows and the number of columns must equal the dimension of the working correlation matrix. This dimension depends on the subject effect, the within-subject effect, whether the active dataset is sorted, and the data. The simplest way to determine the working correlation matrix dimension is to run the GENLIN procedure first for the model using the default working correlation matrix structure (instead of the FIXED structure) and examine the PRINT MODELINFO output for the working correlation matrix dimension. Then, rerun the procedure with the FIXED specification.

Specify only the lower triangular portion of the matrix. Matrix elements must be specified row-by-row. All elements must be between 0 and 1 inclusive.

For example, if there are three measurements per subject, then the following specification defines a 3 * 3 working correlation matrix.

```
CORRTYPE = FIXED(0.84 0.65 0.75)
```

```
1.00 0.84 0.65
0.84 1.00 0.75
0.65 0.75 1.00
```

There is no default value for the fixed working correlation matrix.

MDEPENDENT(integer). m-dependent working correlation matrix. Specify the value of *m* in parentheses as an integer greater than or equal to 0. The specified m should be less than the number of row or column levels in the working correlation matrix. If the specified *m* is greater than the dimension of the working correlation matrix, then *m* is set equal to the number of row or column levels minus 1. For example, if the

dimension of the working correlation matrix is 4, then *m* should be 3 or less. In this case, if you specify *m* > 3, then *m* will be set equal to 3. There is no default value.

UNSTRUCTURED. Unstructured working correlation matrix.

ADJUSTCORR Keyword

The ADJUSTCORR keyword indicates whether to adjust the working correlation matrix estimator by the number of nonredundant parameters.

YES. Adjust the working correlation matrix estimator. This is the default.

NO. Compute the working correlation matrix estimator without the adjustment.

COVB Keyword

The COVB keyword specifies whether to use the robust or the model-based estimator of the parameter estimate covariance matrix for generalized estimating equations.

ROBUST. Robust estimator of the parameter estimate covariance matrix. This is the default.

MODEL. Model-based estimator of the parameter estimate covariance matrix.

HCONVERGE Keyword

The HCONVERGE keyword specifies the Hessian convergence criterion for the generalized estimating equations algorithm. For generalized estimating equations, the Hessian convergence criterion is always absolute.

- Specify a number greater than or equal to 0. If HCONVERGE = 0, the Hessian convergence criterion will be checked value 1E-4 after any specified convergence criteria have been satisfied. If it is not met, a warning is displayed. The default value is 0.
- At least one of the REPEATED subcommand keywords HCONVERGE, PCONVERGE must specify a nonzero number.

MAXITERATIONS Keyword

The MAXITERATIONS keyword specifies the maximum number of iterations for the generalized estimating equations algorithm.

- Specify an integer greater than or equal to 0. The default value is 100.

PCONVERGE Keyword

The PCONVERGE keyword specifies the parameter convergence criterion for the generalized estimating equations algorithm.

- Specify a number greater than or equal to 0, and the ABSOLUTE or RELATIVE keyword in parentheses to define the type of convergence. The number and keyword may be separated by a space character or a comma. The parameter convergence criterion is not used if the number is 0. The default value is 1E-6 (ABSOLUTE).
- At least one of the REPEATED subcommand keywords HCONVERGE, PCONVERGE must specify a nonzero number.

UPDATECORR Keyword

The UPDATECORR keyword specifies the number of iterations between updates of the working correlation matrix. Elements in the working correlation matrix are based on the parameter estimates, which are updated in each iteration of the algorithm. The UPDATECORR keyword specifies the iteration interval at which to update working correlation matrix elements. Specifying a value greater than 1 may reduce processing time.

- Specify an integer greater than 0.
- The working correlation matrix is not updated at all if the value is 0. In this case, the initial working correlation matrix is used throughout the estimation process.
- The default value is 1. By default, the working correlation matrix is updated after every iteration, beginning with the first.

- The UPDATECORR value must be less than or equal to the REPEATED MAXITERATIONS value.

EMMEANS Subcommand

The EMMEANS subcommand displays estimated marginal means of the dependent variable for all level combinations of a set of factors. Note that these are predicted, not observed, means. Estimated marginal means can be computed based on the original scale of the dependent variable or the based on the link function transformation.

- Multiple EMMEANS subcommands are allowed. Each is treated independently.
- The EMMEANS subcommand may be specified with no additional keywords. The output for an empty EMMEANS subcommand is the overall estimated marginal mean of the response, collapsing over any factors and holding any covariates at their overall means.
- Estimated marginal means are not available if the multinomial distribution is used. If DISTRIBUTION = MULTINOMIAL on the MODEL subcommand and the EMMEANS subcommand is specified, then EMMEANS is ignored and a warning is issued.

TABLES Keyword

The TABLES keyword specifies the cells for which estimated marginal means are displayed.

- Valid options are factors appearing on the GENLIN command factor list, and crossed factors constructed of factors on the factor list. Crossed factors can be specified using an asterisk (*) or the keyword BY. All factors in a crossed factor specification must be unique.
- If the TABLES keyword is specified, then the GENLIN procedure collapses over any factors on the GENLIN command factor list but not on the TABLES keyword before computing the estimated marginal means for the dependent variable.
- If the TABLES keyword is not specified, then the overall estimated marginal mean of the dependent variable, collapsing over any factors, is computed.

CONTROL Keyword

The CONTROL keyword specifies the covariate values to use when computing the estimated marginal means.

- Specify one or more covariates appearing on the GENLIN command covariate list, each of which must be followed by a numeric value or the keyword MEAN in parentheses.
- If a numeric value is given for a covariate, then the estimated marginal means will be computed by holding the covariate at the supplied value. If the keyword MEAN is used, then the estimated marginal means will be computed by holding the covariate at its overall mean. If a covariate is not specified on the CONTROL option, then its overall mean will be used in estimated marginal means calculations.
- Any covariate may occur only once on the CONTROL keyword.

The SCALE keyword specifies whether to compute estimated marginal means based on the original scale of the dependent variable or based on the link function transformation.

ORIGINAL. *Estimated marginal means are based on the original scale of the dependent variable.* Estimated marginal means are computed for the response. This is the default.Note that when the dependent variable is specified using the events/trials option, ORIGINAL gives the estimated marginal means for the events/trials proportion rather than for the number of events.

TRANSFORMED. *Estimated marginal means are based on the link function transformation.* Estimated marginal means are computed for the linear predictor.

Example

The following syntax specifies a logistic regression model with binary dependent variable Y and categorical predictor A. Estimated marginal means are requested for each level of A. Because SCALE = ORIGINAL is used, the estimated marginal means are based on the original response. Thus, the estimated marginal means are real numbers between 0 and 1. If SCALE = TRANSFORMED had been used

instead, then the estimated marginal means would be based on the logit-transformed response and would be real numbers between negative and positive infinity.

```
GENLIN y BY a
  /MODEL a
    DISTRIBUTION=BINOMIAL
    LINK=LOGIT
  /EMMEANS TABLES=a SCALE=ORIGINAL.
```

COMPARE Keyword

The COMPARE keyword specifies a factor or a set of crossed factors, the levels or level combinations of which are compared using the contrast type specified on the CONTRAST keyword.

- Valid options are factors appearing on the TABLES keyword. Crossed factors can be specified using an asterisk (*) or the keyword BY. All factors in a crossed factor specification must be unique.
- The COMPARE keyword is valid only if the TABLES keyword is also specified.
- If a single factor is specified, then levels of the factor are compared for each level combination of any other factors on the TABLES keyword.
- If a set of crossed factors is specified, then level combinations of the crossed factors are compared for each level combination of any other factors on the TABLES keyword. Crossed factors may be specified only if PAIRWISE is specified on the CONTRAST keyword.
- By default, the GENLIN procedure sorts levels of the factors in ascending order and defines the highest level as the last level. (If the factor is a string variable, then the value of the highest level is locale-dependent.) However, the sort order can be modified using the ORDER keyword following the factor list on the GENLIN command.
- Only one COMPARE keyword is allowed on a given EMMEANS subcommand.

CONTRAST Keyword

The CONTRAST keyword specifies the type of contrast to use for the levels of the factor, or level combinations of the crossed factors, on the COMPARE keyword. The CONTRAST keyword creates an **L** matrix (that is, a coefficient matrix) such that the columns corresponding to the factor(s) match the contrast given. The other columns are adjusted so that the **L** matrix is estimable.

- The CONTRAST keyword is valid only if the COMPARE keyword is also specified.
- If a single factor is specified on the COMPARE keyword, then any contrast type may be specified on the CONTRAST keyword.
- If a set of crossed factors is specified on the COMPARE keyword, then only the PAIRWISE keyword may be specified on the CONTRAST keyword.
- Only one CONTRAST keyword is allowed on a given EMMEANS subcommand.
- If the COMPARE keyword is specified without CONTRAST, then pairwise comparisons are performed for the factor(s) on COMPARE.
- DIFFERENCE, HELMERT, REPEATED, and SIMPLE contrasts are defined with respect to a first or last level. The first or last level is determined by the ORDER specification following the factors on the GENLIN command line. By default, ORDER = ASCENDING and the last level corresponds to the last level.

The following contrast types are available.

PAIRWISE. *Pairwise comparisons are computed for all level combinations of the specified or implied factors.* This is the default contrast type.

For example,

```
GENLIN y BY a b c
  …
  /EMMEANS TABLES=a*b*c COMPARE a*b CONTRAST=PAIRWISE.
```

The specified contrast performs pairwise comparisons of all level combinations of factors A and B, for each level of factor C.

Pairwise contrasts are not orthogonal.

DEVIATION (value). *Each level of the factor is compared to the grand mean.* Deviation contrasts are not orthogonal.

DIFFERENCE. *Each level of the factor except the first is compared to the mean of previous levels.* In a balanced design, difference contrasts are orthogonal.

HELMERT. *Each level of the factor except the last is compared to the mean of subsequent levels.* In a balanced design, Helmert contrasts are orthogonal.

POLYNOMIAL (number list). *Polynomial contrasts.* The first degree of freedom contains the linear effect across the levels of the factor, the second contains the quadratic effect, and so on. By default, the levels are assumed to be equally spaced; the default metric is (1 2 . . . *k*), where *k* levels are involved.The POLYNOMIAL keyword may be followed optionally by parentheses containing a number list. Numbers in the list must be separated by spaces or commas. Unequal spacing may be specified by entering a metric consisting of one number for each level of the factor. Only the relative differences between the terms of the metric matter. Thus, for example, (1 2 4) is the same metric as (2 3 5) or (20 30 50) because, in each instance, the difference between the second and third numbers is twice the difference between the first and second. All numbers in the metric must be unique; thus, (1 1 2) is not valid.

A user-specified metric must supply at least as many numbers as there are levels of the compared factor. If too few numbers are specified, then a warning is issued and hypothesis tests are not performed. If too many numbers are specified, then a warning is issued but hypothesis tests are still performed. In the latter case, the contrast is created based on the specified numbers beginning with the first and using as many numbers as there are levels of the compared factor. In any event, we recommend printing the L matrix (/PRINT LMATRIX) to confirm that the proper contrast is being constructed.

For example,

```
GENLIN y BY a
  …
  /EMMEANS TABLES=a CONTRAST=POLYNOMIAL(1 2 4).
```

Suppose that factor A has three levels. The specified contrast indicates that the three levels of A are actually in the proportion 1:2:4.

Alternatively, suppose that factor A has two levels. In this case, the specified contrast indicates that the two levels of A are in the proportion 1:2.

In a balanced design, polynomial contrasts are orthogonal.

REPEATED. *Each level of the factor except the last is compared to the next level.* Repeated contrasts are not orthogonal.

SIMPLE (value). *Each level of the factor except the last is compared to the last level.* The SIMPLE keyword may be followed optionally by parentheses containing a value. Put the value inside a pair of quotes if it is formatted (such as date or currency) or if the factor is of string type. If a value is specified, then the factor level with that value is used as the omitted reference category. If the specified value does not exist in the data, then a warning is issued and the last level is used.

For example,

```
GENLIN y BY a
  …
  /EMMEANS TABLES=a CONTRAST=SIMPLE(1).
```

The specified contrast compares all levels of factor A (except level 1) to level 1.

Simple contrasts are not orthogonal.

PADJUST Keyword

The PADJUST keyword indicates the method of adjusting the significance level.

LSD. *Least significant difference.* This method does not control the overall probability of rejecting the hypotheses that some linear contrasts are different from the null hypothesis value(s). This is the default.

BONFERRONI. *Bonferroni.* This method adjusts the observed significance level for the fact that multiple contrasts are being tested.

SEQBONFERRONI. *Sequential Bonferroni.* This is a sequentially step-down rejective Bonferroni procedure that is much less conservative in terms of rejecting individual hypotheses but maintains the same overall significance level.

SIDAK. *Sidak.* This method provides tighter bounds than the Bonferroni approach.

SEQSIDAK. *Sequential Sidak.* This is a sequentially step-down rejective Sidak procedure that is much less conservative in terms of rejecting individual hypotheses but maintains the same overall significance level.

MISSING Subcommand

The MISSING subcommand specifies how missing values are handled.

- Cases with system missing values on any variable used by the GENLIN procedure are excluded from the analysis.
- Cases must have valid data for the dependent variable or the events and trials variables, any covariates, the OFFSET variable if it exists, the SCALEWEIGHT variable if it exists, and any SUBJECT and WITHINSUBJECT variables. Cases with missing values for any of these variables are not used in the analysis.
- The CLASSMISSING keyword specifies whether user-missing values of any factors are treated as valid.

EXCLUDE. *Exclude user-missing values among any factor or subpopulation variables.* Treat user-missing values for these variables as invalid data. This is the default.

INCLUDE. *Include user-missing values among any factor or subpopulation variables.* Treat user-missing values for these variables as valid data.

PRINT Subcommand

The PRINT subcommand is used to display optional output.

- If the PRINT subcommand is not specified, then the default output indicated below is displayed.
- If the PRINT subcommand is specified, then the GENLIN procedure displays output only for those keywords that are specified.

CORB. Correlation matrix for parameter estimates.

COVB. Covariance matrix for parameter estimates.

CPS. Case processing summary. For generalized estimating equations, this keyword also displays the Correlated Data Summary table. This is the default output if the PRINT subcommand is not specified.

DESCRIPTIVES. Descriptive statistics. Displays descriptive statistics and summary information about the dependent variable, covariates, factors. This is the default output if the PRINT subcommand is not specified.

FIT. Goodness of fit. For generalized linear models, displays deviance and scaled deviance, Pearson chi-square and scaled Pearson chi-square, log likelihood, Akaike's information criterion (AIC), finite sample corrected AIC (AICC), Bayesian information criterion (BIC), and consistent AIC (CAIC).

Note that when the scale parameter is fit using the deviance (/CRITERIA SCALE = DEVIANCE) or Pearson chi-square (/CRITERIA SCALE = PEARSON), the algorithm begins by assuming the scale parameter equals 1. Following estimation of the regression coefficients, the estimated scale parameter is calculated. Finally, estimated standard errors, Wald confidence intervals, and significance tests are adjusted based on the estimated scale parameter. However, in order to ensure fair comparison in the information criteria and the model fit omnibus test (see the SUMMARY keyword below), the log likelihood

is not revised by the estimated scale parameter. Instead, when the scale parameter is fit using the deviance or Pearson chi-square, the log likelihood is computed with the scale parameter set equal to 1.

For generalized estimating equations, displays two extensions of AIC for model selection: Quasi-likelihood under the independence model criterion (QIC) for choosing the best correlation structure, and corrected quasi-likelihood under the independence model criterion (QICC) for choosing the best subset of predictors.

The quasi-likelihood functions are computed with the scale parameter set equal to a fixed value if a fixed value is specified on the /CRITERIA SCALE keyword. Otherwise, if /CRITERIA SCALE = MLE, DEVIANCE, or PEARSON, then the quasi-likelihood functions are computed with the scale parameter set equal to 1.

Goodness of fit statistics are not available for generalized estimating equations when the multinomial distribution is used. Thus, if the REPEATED subcommand and /MODEL DISTRIBUTION = MULTINOMIAL are specified, then the FIT keyword is silently ignored.

This is the default output if the PRINT subcommand is not specified.

GEF. General estimable function(s).

HISTORY (integer). Iteration history. For generalized linear models, displays the iteration history for the parameter estimates and log-likelihood, and prints the last evaluation of the gradient vector and the Hessian matrix. Also displays the iteration history for the profile likelihood confidence intervals (if requested via CRITERIA CITYPE = PROFILE) and for type I or III analyses (if requested via PRINT SUMMARY).

For generalized estimating equations, displays the iteration history for the parameter estimates, and prints the last evaluation of the generalized gradient and the Hessian matrix. Also displays the iteration history for type III analyses (if requested via PRINT SUMMARY).

The HISTORY keyword may be followed optionally by an integer n in parentheses, where the integer is greater than zero. The iteration history table displays parameter estimates for every n iterations beginning with the 0th iteration (the initial estimates). The default is to print every iteration (n = 1). If HISTORY is specified, then the last iteration is always displayed regardless of the value of n.

LAGRANGE. Lagrange multiplier test. For the normal, gamma, inverse Gaussian, and Tweedie distributions, displays Lagrange multiplier test statistics for assessing the validity of a scale parameter that is computed using the deviance or Pearson chi-square, or set at a fixed number. For the negative binomial distribution, tests the fixed ancillary parameter.

The LAGRANGE keyword is honored if MODEL DISTRIBUTION = NORMAL, GAMMA, IGAUSS, or TWEEDIE and CRITERIA SCALE = DEVIANCE, PEARSON, or number; or if MODEL DISTRIBUTION = NEGBIN(number) is specified. Otherwise the keyword is ignored and a warning is issued.

If the REPEATED subcommand is specified, then the LAGRANGE keyword is silently ignored.

LMATRIX. Set of contrast coefficient (L) matrices. Displays contrast coefficients for the default effects and for the estimated marginal means if requested.

MODELINFO. Model information. Displays the dataset name, dependent variable or events and trials variables, offset variable, scale weight variable, probability distribution, and link function.

For generalized estimating equations, also displays the subject variables, within-subject variables, and working correlation matrix structure.

This is the default output if the PRINT subcommand is not specified.

SOLUTION. Parameter estimates and corresponding statistics. This is the default output if the PRINT subcommand is not specified.

The SOLUTION keyword may be followed optionally by the keyword EXPONENTIATED in parentheses to display exponentiated parameter estimates in addition to the raw parameter estimates.

SUMMARY. Model summary statistics. Displays model fit tests, including likelihood ratio statistics for the model fit omnibus test, and statistics for the type I or III contrasts for each effect (depending on the

CRITERIA ANALYSISTYPE specification). This is default output if the PRINT subcommand is not specified.

If the REPEATED subcommand is specified, then only the statistics for each effect are displayed.

WORKINGCORR. Working correlation matrix. This keyword is honored only if the REPEATED is in effect. Otherwise it is silently ignored.

NONE. No PRINT subcommand output. None of the PRINT subcommand output is displayed. If NONE is specified, then no other keywords are allowed on the PRINT subcommand.

SAVE Subcommand

The SAVE subcommand adds predicted, residual, leverage, or Cook's distance values to the working dataset.

- Specify one or more temporary variables, each followed by an optional new name in parentheses.
- The optional names must be unique, valid variable names.
- If new names are not specified, then GENLIN uses the default names. If the default names conflict with existing variable names, then a suffix is added to the default names to make them unique.

The following rules describe the functionality of the SAVE subcommand when the response variable—either the dependent variable or the events or trials variable—has an invalid value for a case.

- If all factors and covariates in the model have valid values for the case, then the procedure computes predicted values but not the residuals. (The MISSING subcommand setting is taken into account when defining valid/invalid values for a factor.)
- An additional restriction for factors is that only those values of the factor actually used in building the model are considered valid. For example, suppose factor A takes values 1, 2, and 3 when the procedure builds the model. Also suppose there is a case with an invalid dependent variable value, a value of 4 on factor A, and valid values on all other factors and covariates. For this case, no predicted value is saved because there is no model coefficient corresponding to factor A = 4.

XBPRED (varname | rootname:n). *Predicted value(s) of the linear predictor.* For all distributions except the multinomial, XBPRED creates one variable and the default variable name is *XBPredicted*. Specify a variable name in parentheses to override the default.

For the multinomial distribution, one variable is created for each dependent variable category except the last (see the dependent variable ORDER keyword in the section "Variable List " on page 726). XBPRED saves the predicted values of the linear predictor for the first 25 categories, up to but not including the last, by default. The default root name is *XBPredicted*, and the default variable names are *XBPredicted_1*, *XBPredicted_2*, and so on, corresponding to the order of the dependent variable categories. Specify a root name in parentheses to override the default. Specify a colon and a positive integer giving the number of categories to override the default 25. To specify a number without a root name, simply enter a colon before the number.

XBSTDERROR (varname | rootname:n). *Estimated standard error(s) of the predicted value of the linear predictor.* For all distributions except the multinomial, XBSTDERROR creates one variable and the default variable name is *XBStandardError*. Specify a variable name in parentheses to override the default.

For the multinomial distribution, one variable is created for each dependent variable category except the last (see the dependent variable ORDER keyword in the section "Variable List " on page 726). XBSTDERROR saves the estimated standard errors for the first 25 categories, up to but not including the last, by default. The default root name is *XBStandardError*, and the default variable names are *XBStandardError_1*, *XBStandardError_2*, and so on, corresponding to the order of the dependent variable categories. Specify a root name in parentheses to override the default. Specify a colon and a positive integer giving the number of categories to override the default 25. To specify a number without a root name, simply enter a colon before the number.

MEANPRED (varname | rootname:n). *Predicted value(s) of the mean of the response.* For all distributions except the multinomial, MEANPRED creates one variable and the default variable name is *MeanPredicted*. Specify a variable name in parentheses to override the default.

If the binomial distribution is used and the dependent variable is in single variable format, then MEANPRED computes a predicted probability. Suppose the dependent variable has data values 0 and 1. If the default reference category is in effect, that is, REFERENCE = LAST on the GENLIN command line, then 1 is the reference category and MEANPRED computes the predicted probability that the dependent variable equals 0. To compute the predicted probability that the dependent variable equals 1 instead, specify REFERENCE = FIRST on the GENLIN command line.

If the binomial distribution is used and the dependent variable is in events/trials format, then MEANPRED computes the predicted number of events.

For the multinomial distribution, one variable is created for each dependent variable category except the last (see the dependent variable ORDER keyword in the section "Variable List " on page 726). MEANPRED saves the cumulative predicted probability for the first 25 categories, up to but not including the last, by default. The default root name is *CumMeanPredicted*, and the default variable names are *CumMeanPredicted_1*, *CumMeanPredicted_2*, and so on, corresponding to the order of the dependent variable categories. Specify a root name in parentheses to override the default. Specify a colon and a positive integer giving the number of categories to override the default 25. To specify a number without a root name, simply enter a colon before the number.

CIMEANPREDL (varname | rootname:n). *Lower bound(s) of the confidence interval for the mean of the response.* For all distributions except the multinomial, CIMEANPREDL creates one variable and the default variable name is *CIMeanPredictedLower*. Specify a variable name in parentheses to override the default.

For the multinomial distribution, one variable is created for each dependent variable category except the last (see the dependent variable ORDER keyword in the section "Variable List " on page 726). CIMEANPREDL saves the lower bound of the cumulative predicted probability for the first 25 categories, up to but not including the last, by default. The default root name is *CICumMeanPredictedLower*, and the default variable names are *CICumMeanPredictedLower_1*, *CICumMeanPredictedLower_2*, and so on, corresponding to the order of the dependent variable categories. Specify a root name in parentheses to override the default. Specify a colon and a positive integer giving the number of categories to override the default 25. To specify a number without a root name, simply enter a colon before the number.

CIMEANPREDU (varname | rootname:n). *Upper bound(s) of the confidence interval for the mean of the response.* For all distributions except the multinomial, CIMEANPREDU creates one variable and the default variable name is *CIMeanPredictedUpper*. Specify a variable name in parentheses to override the default.

For the multinomial distribution, one variable is created for each dependent variable category except the last (see the dependent variable ORDER keyword in the section "Variable List " on page 726). CIMEANPREDU saves the upper bound of the cumulative predicted probability for the first 25 categories, up to but not including the last, by default. The default root name is *CICumMeanPredictedUpper*, and the default variable names are *CICumMeanPredictedUpper_1*, *CICumMeanPredictedUpper_2*, and so on, corresponding to the order of the dependent variable categories. Specify a root name in parentheses to override the default. Specify a colon and a positive integer giving the number of categories to override the default 25. To specify a number without a root name, simply enter a colon before the number.

PREDVAL (varname). *Predicted category value for binomial or multinomial distribution.* The class or value predicted by the model if the binomial or multinomial distribution is in effect. This keyword is honored only if the binomial distribution is used, that is, if DISTRIBUTION = BINOMIAL is specified or implied on the MODEL subcommand and the dependent variable is in single variable format, or the multinomial distribution is used (DISTRIBUTION = MULTINOMIAL). Otherwise, the PREDVAL keyword is ignored with a warning. The default variable name is *PredictedValue*.

LEVERAGE (varname). *Leverage value.* The default variable name is *Leverage*. Leverage values are not available for the multinomial distribution or generalized estimating equations.

RESID (varname). *Raw residual.* The default variable name is *Residual*. Raw residuals are not available for the multinomial distribution.

PEARSONRESID (varname). *Pearson residual.* The default variable name is *PearsonResidual*. Pearson residuals are not available for the multinomial distribution.

DEVIANCERESID (varname). *Deviance residual.* The default variable name is *DevianceResidual*. Deviance residuals are not available for the multinomial distribution or generalized estimating equations.

STDPEARSONRESID (varname). *Standardized Pearson residual.* The default variable name is *StdPearsonResidual*. Standardized Pearson residuals are not available for the multinomial distribution or generalized estimating equations.

STDDEVIANCERESID (varname). *Standardized deviance residual.* The default variable name is *StdDevianceResidual*. Standardized deviance residuals are not available for the multinomial distribution or generalized estimating equations.

LIKELIHOODRESID (varname). *Likelihood residual.* The default variable name is *LikelihoodResidual*. Likelihood residuals are not available for the multinomial distribution or generalized estimating equations.

COOK (varname). *Cook's distance.* The default variable name is *CooksDistance*. Cook's distances are not available for the multinomial distribution or generalized estimating equations.

OUTFILE Subcommand

The OUTFILE subcommand saves an external IBM SPSS Statistics data file or previously declared dataset containing the parameter correlation or covariance matrix with parameter estimates, standard errors, significance values, and degrees of freedom. It also saves the parameter estimates and the parameter covariance matrix in XML format.

- At least one keyword and a filename are required.
- The COVB and CORB keywords are mutually exclusive, as are the MODEL and PARAMETER keywords.
- The filename must be specified in full. GENLIN does not supply an extension.

COVB = 'savfile' | 'dataset'. *Writes the parameter covariance matrix and other statistics to an external IBM SPSS Statistics data file or previously declared dataset.*

CORB = 'savfile' | 'dataset'. *Writes the parameter correlation matrix and other statistics to an external IBM SPSS Statistics data file or previously declared dataset..*

MODEL = 'file'. *Writes the parameter estimates and the parameter covariance matrix to an XML file.*

PARAMETER = 'file'. *Writes the parameter estimates to an XML file.*

GENLINMIXED

GENLINMIXED is available in the Advanced Models option.

```
GENLINMIXED
  [/DATA_STRUCTURE]
     SUBJECTS = fieldName[*fieldName[*...]]
     [REPEATED_MEASURES = fieldName[*fieldName[*...]]]
     [KRONECKER_MEASURES = fieldName[*fieldName[*...]]]
     [GROUPING = fieldName[*fieldName[*...]]]
     [COVARIANCE_TYPE = DIAGONAL** | AR1 | ARH1 |ARMA11 | COMPOUND_SYMMETRY | CSH | IDENTITY |
                        SP_POWER | SP_EXPONENTIAL | SP_GAUSSIAN | SP_LINEAR | SP_LINEARLOG |
                        SP_SPHERICAL | TOEPLITZ | UN_AR1 | UN_CS | UN_UN | UNSTRUCTURED |
                        VARIANCE_COMPONENTS]
     [SPCOORDS = fieldName...]
  /FIELDS
    TARGET = fieldName
    [TRIALS = {NONE**          }]
              {VALUE(number)   }
              {FIELD(fieldName)}
    [ANALYSIS_WEIGHT = fieldName]
    [OFFSET = {NONE**          }]
              {VALUE(number)   }
              {FIELD(fieldName)}
  [/TARGET_OPTIONS]
    [REFERENCE = value]
    [DISTRIBUTION = NORMAL | BINOMIAL | MULTINOMIAL | GAMMA |
                    INVERSE_GAUSSIAN | NEGATIVE_BINOMIAL | POISSON]
    [LINK = IDENTITY | CAUCHIT | CLOGLOG | LOG | LOGC | LOGIT | NLOGLOG | POWER | PROBIT ]
    [LINK_PARAMETER = number]
  [/FIXED]
    [EFFECTS = effect [effect ...]]
    [USE_INTERCEPT = TRUE** | FALSE]
  [/RANDOM]
    [EFFECTS = effect [effect ...]]
    [USE_INTERCEPT = TRUE | FALSE**]
    [SUBJECTS = fieldName[*fieldName[*...]]]
    [GROUPING = fieldName[*fieldName[*...]]]
    [COVARIANCE_TYPE = DIAGONAL | AR1 | ARH1 | ARMA11 | COMPOUND_SYMMETRY | CSH |
                       IDENTITY | TOEPLITZ | UNSTRUCTURED | VARIANCE_COMPONENTS**]
    [SOLUTION = TRUE | FALSE**]
  [/RANDOM ...]
  [/BUILD_OPTIONS]
    [TARGET_CATEGORY_ORDER = {ASCENDING**}]
                             {DESCENDING }
                             {DATA       }
    [INPUTS_CATEGORY_ORDER = {ASCENDING**}]
                             {DESCENDING }
                             {DATA       }
    [MAX_ITERATIONS = {100** }]
                      {number}
    [CONFIDENCE_LEVEL = {95**  }]
                        {number}
    [DF_METHOD = {RESIDUAL**   }]
                 {SATTERTHWAITE}
                 {KENWARD_ROGER}
    [COVB = {MODEL**}]
            {ROBUST }
    [PCONVERGE = {1E-6**}[({ABSOLUTE**})] ]
                 {value }  {RELATIVE  }
    [LCONVERGE = {0**  }[({ABSOLUTE**})] ]
                 {value}  {RELATIVE  }
    [HCONVERGE = {0**   }[({ABSOLUTE**})] ]
                 {value }  {RELATIVE  }
    [SCORING = {0**}]
               {n  }
    [SINGULAR = {1E-12**}]
                {value  }
  [/EMMEANS
    [TABLES = factor[*factor...]]
    [COMPARE = factor]
    [CONTRAST = {NONE**   }]
                {PAIRWISE }
                {DEVIATION}
                {SIMPLE   }
  [/EMMEANS...]
  [/EMMEANS_OPTIONS]
    [CONTROL = fieldName (number) [fieldName (number) ...]]
    [SCALE = {ORIGINAL** }]
             {TRANSFORMED}
    [PADJUST = {LSD**        }]
               {SEQBONFERRONI}
               {SEQSIDAK     }
  [/OUTFILE MODEL = 'file']
  [/SAVE [PREDICTED_VALUES(fieldName)]]
    [PREDICTED_PROBABILITY(rootname)]
    [MAX_CATEGORIES({25**   })]
                    {integer}
    [CONFIDENCE(fieldName)]
    [CONFIDENCE_METHOD({PREDICTEDPROBABILITY**})]
                       {PROBABILITYIMPROVEMENT}
    [CONFIDENCE_INTERVALS(rootname)]
    [PEARSON_RESIDUALS(fieldName)].
```

** Default if the subcommand or keyword is omitted.

The KRONECKER_MEASURES keyword should be only when COVARIANCE_TYPE is defined as UN_AR1, UN_CS, or UN_UN.

When both KRONECKER_MEASURES and REPEATED_MEASURES keywords are in effect, they may or may not have common fields, but their values cannot be exactly the same (even when the values are not in the same order).

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 19

- Command introduced.

Release 20

- Ordinal targets can be analyzed using the Multinomial distribution and the complementary log-log, cauchit, logit, negative log-log, or probit link functions.

Release 22

- PCONVERGE, LCONVERGE, HCONVERGE, SCORING, and SINGULAR keywords introduced on the BUILD_OPTIONS subcommand.

Release 25

- SP_POWER, SP_EXPONENTIAL, SP_GAUSSIAN, SP_LINEAR, SP_LINEARLOG, and SP_SPHERICAL options added to the COVARIANCE_TYPE keyword on the DATA_STRUCTURE subcommand.
- SPCOORDS keyword introduced on the DATA_STRUCTURE subcommand.
- SOLUTION keyword introduced on the RANDOM subcommand.

Subscription (April 2018 update)

- ARH1 and CSH options added to the COVARIANCE_TYPE keyword on the RANDOM subcommand.
- ARH1 and CSH options added to the COVARIANCE_TYPE keyword on the DATA_STRUCTURE subcommand.
- KENWARD_ROGER option added to the DF_METHOD keyword on the BUILD_OPTIONS subcommand.
- KRONECKER_MEASURES keyword added to the DATA_STRUCTURE subcommand. The keyword should be used only when COVARIANCE_TYPE is one of three following Kronecker types.
- UN_AR1, UN_CS and UN_UN options added to the COVARIANCE_TYPE keyword on the DATA_STRUCTURE subcommand.

Example

```
GENLINMIXED
  /FIELDS TARGET=y1.
```

Overview

The GENLINMIXED procedure fits generalized linear mixed models. Generalized linear mixed models extend the linear model so that:

- The target is linearly related to the factors and covariates via a specified link function
- The target can have a non-normal distribution
- The observations can be correlated.

Generalized linear mixed models cover a wide variety of models, from simple linear regression to complex multilevel models for non-normal longitudinal data.

Options

Probability Distribution of Target. The probability distribution of the target can be specified as normal, binomial, gamma, inverse Gaussian, multinomial, negative binomial, or Poisson.

Events/Trials Target Specification. The typical target specification will be a single field, but for the binomial distribution the target can be specified using a number-of-events field and a number-of-trials field. Alternatively, if the number of trials is the same across all subjects, then trials can be specified using a fixed number instead of a field.

Link Function. The target is linearly related to the model effects via one of the following link functions: Identity, complementary log-log, log, log-complement, logit, negative log-log, power, and probit.

Repeated Measurements. You can specify the subject structure for repeated measurements and how the errors of the repeated measurements are correlated. Complete independence is assumed across subject blocks.

Knonecker Measurements. You can specify the subject structure for Knonecker measurements and how the errors of the measurements are correlated. Knonecker should only be used when COVARIANCE_TYPE is defined as UN_AR1, UN_CS, or UN_UN.

Random Effects. Random-effects are effects whose values in the data file can be considered a random sample from a larger population of values. They are useful for explaining excess variability in the target.

Estimated Marginal Means. Estimated marginal means can be computed for one or more crossed factors and can be based on either the response or the linear predictor.

Output. The procedure produces a model viewer object containing tables and charts for assessing the quality of the model; also you can save predicted values to the active dataset and the model to a file in PMML format.

Basic Specification

The basic specification is the GENLINMIXED command and the FIELDS subcommand with the TARGET keyword. This fits an intercept-only model that can be used as a basis for comparing more complicated models.

Note: Since measurement level can affect the results, if any variables (fields) have an unknown measurement level, an initial data pass will be performed to determine default measurement level for any variables with an unknown measurement level. For information on the criteria used to determine default measurement level, see SET SCALEMIN.

Syntax Rules

- The FIELDS subcommand is required. All other subcommands are optional.
- Subcommands can be specified in any order.
- There can be multiple RANDOM and EMMEANS subcommands; each RANDOM subcommand defines a separate random effects block, while each EMMEANS subcommand defines a separate set of estimated means. Only a single instance of any other subcommand is allowed.
- An error occurs if a keyword is specified more than once within a subcommand.
- Parentheses, equals signs, and slashes shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not allowed.
- Any split field defined on the SPLIT FILE command cannot be used on this command.
- Any field specifed on the WEIGHT command cannot be used on this command.

Limitations

- The SPLIT FILE setting is ignored with a warning by the procedure.

Examples

The following examples correspond to the predefined distribution and link function combinations on the Target settings of the dialog.

Linear model

```
GENLINMIXED
  /FIELDS TARGET=y
  /TARGET_OPTIONS DISTRIBUTION=NORMAL LINK=IDENTITY
  /FIXED EFFECTS=x1 x2 x3.
```

- The FIELDS subcommand specifies *y* as the target.
- The TARGET_OPTIONS subcommand that the target has a normal distribution and is linearly related to the model effects.
- The FIXED subcommand specifies a main effects model with fields *x1*, *x2*, and *x3*. If they are continuous, they will be treated as covariates, if categorical, they will be treated as factors.

Gamma regression

```
GENLINMIXED
  /FIELDS TARGET=claim_amount ANALYSISWEIGHT=number_of_claims
  /TARGET_OPTIONS DISTRIBUTION=GAMMA LINK=POWER(-1)
  /FIXED EFFECTS=holder_age vehicle_group vehicle_age
  /BUILD_OPTIONS INPUTS_CATEGORY_ORDER=DESCENDING.
```

- The procedure fits a model for the target *claim_amount*, using *holder_age*, *vehicle_group*, and *vehicle_age* as main effects. In order to account for the varying number of claims used to compute the average claim amounts, you specify *number_of_claims* as the analysis weight.
- The TARGET_OPTIONS specification assumes that *claim_amount* has a gamma distribution. A power link function with −1 as the exponent relates the distribution of *claim_amount* to a linear combination of the predictors, including an intercept term.
- The BUILD_OPTIONS subcommand specifies that the category order for factors is descending values of factor levels; thus, the first category of each categorical field is used as the reference category.

(Poisson) Loglinear model

```
GENLINMIXED
  /FIELDS TARGET=damage_incidents OFFSET=log_months_service
  /TARGET_OPTIONS DISTRIBUTION=POISSON LINK=LOG
  /FIXED EFFECTS=type construction operation
  /BUILD_OPTIONS INPUTS_CATEGORY_ORDER=DESCENDING.
```

- The procedure fits a model for the target *damage_incidents*, using *type*, *construction*, and *operation* as main effects.
- The TARGET_OPTIONS specification assumes that *damage_incidents* has a Poisson distribution. A log link function relates the distribution of *damage_incidents* to a linear combination of the predictors, including an intercept term, and an offset equal to the values of *log_months_service*.
- The BUILD_OPTIONS subcommand specifies that the category order for factors is descending values of factor levels; thus, the first category of each categorical field is used as the reference category.

Negative binomial regression

```
GENLINMIXED
  /FIELDS TARGET=damage_incidents OFFSET=log_months_service
  /TARGET_OPTIONS DISTRIBUTION=NEGATIVE_BINOMIAL LINK=LOG
  /FIXED EFFECTS=type construction operation.
```

- The negative binomial distribution is an alternative to the Poisson when the observations are overdispersed; that is, since the mean and variance of the Poisson distribution are the same, when the data show greater variability, the negative binomial distribution can provide a better fit.

(Nominal) Multinomial logistic regression

```
GENLINMIXED
  /FIELDS TARGET=bfast
```

```
  /TARGET_OPTIONS DISTRIBUTION=MULTINOMIAL LINK=LOGIT
  /FIXED EFFECTS=agecat gender active.
```

- The procedure fits a model for *bfast* using *agecat*, *gender*, and *active* as main effects.
- The TARGET_OPTIONS specification assumes that *bfast* has a (nominal) multinomial distribution. A logit link function relates the distribution of *bfast* to a linear combination of the predictors, including an intercept term.

(Ordinal) Multinomial logistic regression

```
GENLINMIXED
  /FIELDS TARGET=chist
  /TARGET_OPTIONS DISTRIBUTION=MULTINOMIAL LINK=CLOGLOG
  /FIXED EFFECTS=numcred othnstal housng age duration.
```

- The procedure fits a model for *chist* using *numcred*, *othnstal*, *housing*, *age*, and *duration* as main effects. Because *numcred*, *othnstal*, and *housing* have categorical measurement level, they are treated as factors; *age*, and *duration* have continuous (scale) measurement level and are treated as covariates.
- The TARGET_OPTIONS specification assumes that *chist* has an (ordinal) multinomial distribution. A (cumulative) complementary log–log link function relates the distribution of *chist* to a linear combination of the predictors, including threshold terms for the categories of *chist* (except the last category).

Binary logistic regression

```
GENLINMIXED
  /FIELDS TARGET=default
  /TARGET_OPTIONS DISTRIBUTION=BINOMIAL LINK=LOGIT
  /FIXED EFFECTS=age ed employ address income debtinc creddebt othdebt.
```

- The procedure fits a model for *default* using *age*, *ed*, *employ*, *address*, *income*, *debtinc*, *creddebt*, and *othdebt* as main effects.
- The TARGET_OPTIONS specification assumes that *default* has a binomial distribution. A logit link function relates the distribution of *default* to a linear combination of the predictors, including an intercept term.

Binary probit model

```
GENLINMIXED
  /FIELDS TARGET=response TRIALS=nsubj
  /TARGET_OPTIONS DISTRIBUTION=BINOMIAL LINK=PROBIT
  /FIXED EFFECTS=site value.
```

- The procedure fits a model for the number of responses *response* within trials *nsubj* using site and value as main effects.
- The TARGET_OPTIONS specification assumes that *response* within *nsubj* has a binomial distribution. A probit link function relates the distribution of the target to a linear combination of the predictors, including an intercept term.

Interval censored survival

```
GENLINMIXED
  /FIELDS TARGET=result2
  /TARGET_OPTIONS DISTRIBUTION=BINOMIAL LINK=CLOGLOG
  /FIXED EFFECTS=duration treatment period age USE_INTERCEPT=FALSE
  /BUILD_OPTIONS TARGET_CATEGORY_ORDER=DESCENDING INPUTS_CATEGORY_ORDER=DESCENDING.
```

- The procedure fits a model for the target *result2*, using *duration*, *treatment*, *period*, and *age* as main effects.
- The BUILD_OPTIONS subcommand specifies that the category order for the target and all factors is descending values of factor levels; thus, the first category of each categorical field is used as the reference category.
- The TARGET_OPTIONS specification assumes that *result2* has a binomial distribution. A complementary log-log link function relates the probability of *result2* to a linear combination of the predictors, excluding an intercept term.

Linear mixed model

```
GENLINMIXED
  /DATA_STRUCTURE SUBJECTS=school*classroom*student_id
  /FIELDS TARGET=posttest TRIALS=NONE OFFSET=NONE
  /TARGET_OPTIONS DISTRIBUTION=NORMAL LINK=IDENTITY
  /FIXED  EFFECTS=school_setting school_type teaching_method n_student
    gender lunch pretest
    USE_INTERCEPT=TRUE
  /RANDOM USE_INTERCEPT=TRUE SUBJECTS=school
    COVARIANCE_TYPE=VARIANCE_COMPONENTS
  /RANDOM USE_INTERCEPT=TRUE SUBJECTS=school*classroom
    COVARIANCE_TYPE=VARIANCE_COMPONENTS
  /BUILD_OPTIONS TARGET_CATEGORY_ORDER=ASCENDING
    INPUTS_CATEGORY_ORDER=ASCENDING MAX_ITERATIONS=100
    CONFIDENCE_LEVEL=95 DF_METHOD=RESIDUAL COVB=MODEL
  /EMMEANS_OPTIONS SCALE=ORIGINAL PADJUST=LSD.
```

- The **DATA_STRUCTURE** subcommand specifies that subjects are defined by *school*, *classroom*, and *student_id*.
- The **FIELDS** and **TARGET_OPTIONS** subcommands specify that the procedure creates a model for *posttest* using a normal distribution to fit the test score and an identity link to relate the target to a linear combination of the predictors.
- The **FIXED** subcommand speficies a model with *school_setting*, *school_type*, *teaching_method*, *n_student*, *gender*, *lunch*, and *pretest* as main effects.
- The first **RANDOM** subcommand specifies an intercept-only random effect block with *school* as the subject field. This should account for correlation between classrooms within the same school.
- The second **RANDOM** subcommand specifies an intercept-only random effect block with *school*classroom* as the subject field. This should account for correlation between students within the same classroom.
- All other options are set to their default values.

For discussion of output from this syntax and to place the example in context, see .

Poisson loglinear mixed model

```
GENLINMIXED
  /DATA_STRUCTURE SUBJECTS=patient_id
  /FIELDS TARGET=convulsions TRIALS=NONE OFFSET=NONE
  /TARGET_OPTIONS DISTRIBUTION=POISSON LINK=LOG
  /FIXED  EFFECTS=after_t treatment*after_t USE_INTERCEPT=TRUE
  /RANDOM EFFECTS=after_t USE_INTERCEPT=TRUE
    SUBJECTS=patient_id COVARIANCE_TYPE=UNSTRUCTURED
  /BUILD_OPTIONS TARGET_CATEGORY_ORDER=ASCENDING
    INPUTS_CATEGORY_ORDER=ASCENDING MAX_ITERATIONS=100
    CONFIDENCE_LEVEL=95 DF_METHOD=RESIDUAL COVB=MODEL
  /EMMEANS_OPTIONS SCALE=ORIGINAL PADJUST=LSD.
```

- The **DATA_STRUCTURE** subcommand specifies that subjects are defined by *patient_id*.
- The **FIELDS** and **TARGET_OPTIONS** subcommands specify that the procedure creates a model for *convulsions* using a Poisson distribution to fit the number of convulsions and a log link to relate the target to a linear combination of the predictors.
- The **FIXED** subcommand specifies a model with *after_t* and *treatment*after_t* as effects.
- The **RANDOM** subcommand specifies *after_t* and an intercept as effects in a random effect block with *patient_id* as the subject field. This should account for correlation between repeated observations of the same patient.
- All other options are set to their default values.

For discussion of output from this syntax and to place the example in context, see .

Multinomial logistic mixed model

```
GENLINMIXED
  /DATA_STRUCTURE SUBJECTS=customer_id
  /FIELDS TARGET=service_usage TRIALS=NONE OFFSET=NONE
  /TARGET_OPTIONS DISTRIBUTION=MULTINOMIAL LINK=LOGIT
  /FIXED  EFFECTS=edcat inccat reside service_type
    USE_INTERCEPT=TRUE
  /RANDOM USE_INTERCEPT=TRUE SUBJECTS=customer_id
    COVARIANCE_TYPE=VARIANCE_COMPONENTS
  /BUILD_OPTIONS TARGET_CATEGORY_ORDER=ASCENDING
    INPUTS_CATEGORY_ORDER=ASCENDING MAX_ITERATIONS=100
    CONFIDENCE_LEVEL=95 DF_METHOD=RESIDUAL COVB=MODEL
  /EMMEANS_OPTIONS SCALE=ORIGINAL PADJUST=LSD.
```

- The **DATA_STRUCTURE** subcommand specifies that subjects are defined by *customer_id*.
- The **FIELDS** and **TARGET_OPTIONS** subcommands specify that the procedure creates a model for *service_usage* using a multinomial distribution and a logit link to relate the target to a linear combination of the predictors.
- The **FIXED** subcommand specifies a model with *edcat*, *inccat*, *reside*, and *service_type* as main effects.
- The **RANDOM** subcommand specifies an intercept-only random effect block with *customer_id* as the subject field. This should account for correlation between answers to the service usage questions across service types (tv, phone, internet) within a given survey responder's answers.
- All other options are set to their default values.

For discussion of output from this syntax and to place the example in context, see .

DATA_STRUCTURE Subcommand

The DATA_STRUCTURE subcommand specifies the subject structure for repeated measurements and how the errors of the repeated measurements are correlated. If the DATA_STRUCTURE subcommand is not specified, then the model assumes that all error terms are independent.

- Records with missing values for any field on the DATA_STRUCTURE subcommand are not used in the analysis.

SUBJECTS Keyword

The SUBJECTS keyword identifies subjects in the active dataset. Complete independence is assumed *across* subjects, but responses *within* subjects are assumed to be correlated.

- Specify a single categorical field or a list of categorical fields connected by asterisks (*) or the keyword BY.
- The number of subjects equals the number of distinct combinations of values of the fields.
- Any field specified on the SUBJECTS keyword cannot be used as a REPEATED_MEASURES field or on the FIELDS subcommand.
- The SUBJECTS keyword is required if the DATA_STRUCTURE subcommand is used.

REPEATED_MEASURES Keyword

The REPEATED_MEASURES keyword gives the repeated (or within-subject) effect. This effect defines the ordering of repeated measurements within subjects. If some measurements do not appear in the data for some subjects, then the existing measurements are ordered and the omitted measurements are treated as missing values. If REPEATED_MEASURES is not specified, then no repeated measures model is built.

- Specify a single field or a list of fields connected by asterisks (*) or the keyword BY.
- Each distinct combination of the values of the fields defines a separate repeated measure.
- Any field specified on the REPEATED_MEASURES keyword cannot be used as a SUBJECTS keyword or on the FIELDS subcommand.
- Each distinct repeated measures value must occur only once within a subject. Generally speaking, you should not use a WEIGHT field if REPEATED MEASURES is specified.

KNONECKER_MEASURES Keyword

The Kronecker product of two matrices can handle doubly repeated-measure data in which there are two repeated effects; one of them indicating the multivariate observation of target, and the other the time or the order of the data measured. The KNONECKER_MEASURES keyword should be only when COVARIANCE_TYPE is defined as UN_AR1, UN_CS, or UN_UN. When both KRONECKER_MEASURES and REPEATED_MEASURES keywords are in effect, they may or may not have common fields, but their values cannot be exactly the same (even when the values are not in the same order).

GROUPING Keyword

The GROUPING keyword allows you to define independent sets of covariance parameters. All subjects have the same covariance type; subjects within the same covariance grouping will have the same values for the parameters.

- Specify a single categorical field or a list of categorical fields connected by asterisks (*) or the keyword BY.
- Each distinct combination of the values of the fields defines a separate covariance grouping.
- A field specified on the GROUPING keyword can also be used as a SUBJECTS or REPEATED_MEASURES field, the TARGET on the FIELDS subcommand, or on the EFFECTS keyword or the FIXED or RANDOM subcommand.
- The GROUPING keyword is optional.
- If the GROUPING keyword is used, there must also be a REPEATED_MEASURES specification.

COVARIANCE_TYPE Keyword

The COVARIANCE_TYPE keyword gives the covariance structure of the residual covariance matrix.

AR1. First-order autoregressive.

ARH1. Heterogenous first-order autoregressive.

ARMA11. Autoregressive moving average (1,1).

COMPOUND_SYMMETRY. This structure has constant variance and constant covariance.

CSH. Heterogenous compound symmetry. This structure has non-constant variance and constant correlation.

DIAGONAL. This is a diagonal structure with heterogenous variance. This is the default.

IDENTITY. This is a scaled identity matrix.

SP_POWER. Adapts the first-order autoregressive structure to a time-decaying correlation of unequal-spaced and repeated measurements.

SP_EXPONENTIAL. Assumes that time-decaying correlations decrease exponentially with increasing spatial distances between repeated measurements.

SP_GAUSSIAN. Assumes that time-decaying correlations decrease more rapidly, with increasing spatial distances between repeated measurements, than for the exponential structure.

SP_LINEAR. Assumes that time-decaying correlations decrease linearly with increasing spatial distances between repeated measurements.

SP_LINEARLOG. Assumes that time-decaying correlations decrease linearly with increasing logarithmic spatial distances between repeated measurements.

SP_SPHERICAL. Allows cubic terms of both the correlation function and the spatial distances between repeated measurements.

TOEPLITZ.

UN_AR1. Specifies the Kronecker product of one unstructured matrix and the other first-order auto-regression covariance matrix. The first unstructured matrix models the multivariate observation, and the second first-order auto-regression covariance structure models the data covariance across time or another factor.

UN_CS. Specifies the Kronecker product of one unstructured matrix and the other compound-symmetry covariance matrix with constant variance and covariance. The first unstructured matrix models the multivariate observation, and the second compound symmetry covariance structure models the data covariance across time or another factor.

UN_UN. Specifies the Kronecker product of two unstructured matrices, with the first one modeling the multivariate observation, and second one modeling the data covariance across time or another factor.

UNSTRUCTURED. This is a completely general covariance matrix.

VARIANCE_COMPONENTS. For repeated measures, the variance components structure is equivalent to the diagonal (DIAGONAL) structure.

SPCOORDS Keyword

The SPCOORDS keyword specifies the variables that define the spatial covariance coordinates. One or more numeric variables must be specified. SPCOORDS is required when SP_POWER, SP_EXPONENTIAL, SP_GAUSSIAN, SP_LINEAR, SP_LINEARLOG, or SP_SPHERICAL is specified for the COVARIANCE_TYPE keyword. It is ignored otherwise.

FIELDS Subcommand

The FIELDS subcommand specifies the target, optional offset, and optional analysis (regression) weight.

- This subcommand, and the TARGET keyword, are required.
- A field can only be specified once on the FIELDS subcommand.
- A categorical TARGET can be specified as a GROUPING field on the DATA_STRUCTURE or RANDOM subcommand, but not in any other capacity. Any other field specified on the FIELDS subcommand cannot be used on any other subcommand.

TARGET. The target field is what the model tries to predict. Specify a single field.

TRIALS = NONE | VALUE(number) | FIELD(field).** If the model response can be expressed as the number of events occurring within a number of trials, then the TARGET keyword specifies the number of events and TRIALS specifies the number of trials. Use VALUE with the number of trials in parentheses if the number of trials is fixed for all subjects and FIELD with a field name in parentheses if the number of trials varies across subjects and there is a field containing the number of trials.

The procedure automatically computes the ratio of the events field over the trials field or number. Technically, the procedure treats the events field as the target in the sense that predicted values and residuals are based on the events field rather than the events/trials ratio.

ANALYSIS_WEIGHT. The model uses analysis (regression) weights to account for differences in variability across levels of the target field. Specify a single continuous field.

OFFSET = NONE | VALUE(value) | FIELD(field).** The offset is a constant added to the intercept. Specify a continuous field or number.

- Records with missing values on the OFFSET field are not used in the analysis.
- Specifying a number when /FIXED USE_INTERCEPT = YES is equivalent to adding a constant to the intercept.
- Specifying a number when /FIXED USE_INTERCEPT = NO is equivalent to fixing the intercept at the specified number.

TARGET_OPTIONS Subcommand

The TARGET_OPTIONS subcommand specifies the reference category, distribution, and link function for the target.

- This subcommand is optional. If the TARGET_OPTIONS subcommand is not specified, then GENLINMIXED assigns a default distribution and link function based on the measurement level of the target. A continuous target has NORMAL distribution and IDENTITY link. A categorical target has MULTINOMIAL distribution and LOGIT link.

REFERENCE. For categorical targets, this is the target value to use as the reference category for parameter estimation. No model parameters are assigned to the reference category. Specify a single value. Put the value inside a pair of quotes if it is formatted (such as date or time) or if the target is of string storage type. The REFERENCE value must be found in the dataset. If REFERENCE is not specified, the last category in ascending order is used as the reference.

- If the target has ordinal measurement level, the REFERENCE keyword is ignored.

DISTRIBUTION Keyword

The DISTRIBUTION keyword specifies the probability distribution of the target.

- The default probability distribution depends on the measurement level of the target. If the target is continuous, the default distribution is NORMAL. If the target is categorical, then the default is MULTINOMIAL. If an events/trials specification is used, then the default distribution is BINOMIAL.
- Caution must be exercised when the LINK but not the DISTRIBUTION keyword is used. The discussion of the LINK keyword below gives details about proper and improper combinations of DISTRIBUTION and LINK settings.

BINOMIAL. *Binomial probability distribution.* If the target is specified as a single field, then it can be numeric or string and it can have only two distinct valid data values.

If the events and trials options are specified, then the procedure automatically computes the ratio of the events field over the trials field or number. The events field—and the trials field if specified—must be numeric. Data values for the events field must be integers greater than or equal to zero. Data values for the trials field must be integers greater than zero. For each record, the trials value must be greater than or equal to the events value. If an events value is noninteger, less than zero, or missing, then the corresponding record is not used in the analysis. If a trials value is noninteger, less than or equal to zero, less than the events value, or missing, then the corresponding record is not used in the analysis.

If the trials option specifies a number, then it must be a positive integer, and it must be greater than or equal to the events value for each record. Records with invalid values are not used in the analysis.

This is the default probability distribution if the target is specified using events/trials format.

GAMMA. *Gamma probability distribution.* The target must be numeric, with data values greater than zero. If a data value is less than or equal to zero, or missing, then the corresponding record is not used in the analysis.

INVERSE_GAUSSIAN. *Inverse Gaussian probability distribution.* The target must be numeric, with data values greater than zero. If a data value is less than or equal to zero, or missing, then the corresponding record is not used in the analysis.

MULTINOMIAL. *Multinomial probability distribution.* The target can be numeric or string, and it must have at least two distinct, valid data values. This is the default probability distribution if the target is categorical. Note that the form of the model will depend on the measurement level of the target.

- A **nominal** target will result in a nominal multinomial model in which a separate set of model parameters are estimated for each category of the target (except the reference category). The parameter estimates for a given predictor show the relationship between that predictor and the likelihood of each category of the target, relative to the reference category.
- An **ordinal** target will result in an ordinal multinomial model in which the traditional intercept term is replaced with a set of **threshold** parameters that relate to the cumulative probability of the target categories.

NEGATIVE_BINOMIAL. *Negative binomial probability distribution.* The target must be numeric, with data values that are integers greater than or equal to zero. If a data value is noninteger, less than zero, or missing, then the corresponding record is not used in the analysis.

NORMAL. *Normal probability distribution.* The target must be numeric. This is the default probability distribution if the target is continuous.

POISSON. *Poisson probability distribution.* The target must be numeric, with data values that are integers greater than or equal to zero. If a data value is noninteger, less than zero, or missing, then the corresponding record is not used in the analysis.

LINK Keyword

The LINK keyword specifies the link function. The following link functions are available.

IDENTITY. *Identity link function. f*(*x*)=*x.* This link function is valid with any distribution except the multinomial.

CAUCHIT. *Cauchit link function. f*(*x*) = tan(π (*x* – 0.5)). This is only valid with ordinal multinomial models.

CLOGLOG. *Complementary log-log link function. f*(*x*)=ln(–ln(1–*x*)). This valid with binomial and ordinal multinomial models.

LOG. *Log link function. f*(*x*)=ln(*x*). This link function is valid with any distribution except the multinomial.

LOGC. *Log complement link function. f*(*x*)=ln(1–*x*). This is only valid with the binomial distribution.

LOGIT. *Logit link function. f*(*x*)=ln(*x* / (1–*x*)). This is valid with the binomial and multinomial distributions.

NLOGLOG. *Negative log-log link function. f*(*x*)=–ln(–ln(*x*)). This valid with binomial and ordinal multinomial models.

POWER. *Power link function. f*(*x*)=x^{α}, if α≠0. *f*(*x*)=ln(*x*), if α=0. α is a required number specified on the LINKPARAMETER keyword and must be a real number. If |α| < 2.2e-16, α is treated as 0. This link function is valid with any distribution except the multinomial.

PROBIT. *Probit link function. f*(*x*)=$\Phi^{-1}(x)$, where Φ^{-1} is the inverse standard normal cumulative distribution function. This valid with binomial and ordinal multinomial models.

- The default setting for LINK depends on the DISTRIBUTION setting, whether DISTRIBUTION is set explicitly or is the default, as shown in the following table.

Table 52. Default link function for each distribution

DISTRIBUTION Setting	Default LINK Setting
NORMAL	IDENTITY
BINOMIAL	LOGIT
GAMMA	POWER(–1)
INVERSE_GAUSSIAN	POWER(–2)
MULTINOMIAL	LOGIT
NEGATIVE_BINOMIAL	LOG
POISSON	LOG

- The procedure will fit a model if a permissible combination of LINK and DISTRIBUTION specifications is given. The table below indicates the permissible LINK and DISTRIBUTION combinations. Specifying an improper combination will yield an error message.
- Note that the default setting for DISTRIBUTION is irrespective of the LINK specification, and that not all LINK specifications are valid for the default DISTRIBUTION. Thus, if LINK = CLOGLOG is specified for a continuous target but DISTRIBUTION is not, then the default DISTRIBUTION = NORMAL yields an improper combination of DISTRIBUTION and LINK settings.

Table 53. Valid combinations of distribution and link function

Link	NORMAL	BI-NOMIAL	GAMMA	INVERSE_GAUSSIAN	Nominal MULTI-NOMIAL	Ordinal MULTI-NOMIAL	NEGATIVE_BINOMIAL	POISSON
IDENTITY	X	X	X	X			X	X
CAUCHIT						X		
CLOGLOG		X				X		

Table 53. Valid combinations of distribution and link function (continued)								
Link	**NORMAL**	**BI-NOMIAL**	**GAMMA**	**INVERSE_GAUSSIAN**	**Nominal MULTI-NOMIAL**	**Ordinal MULTI-NOMIAL**	**NEGATIVE_BINOMIAL**	**POISSON**
LOG	X	X	X	X			X	X
LOGC		X						
LOGIT		X			X	X		
NLOGLOG		X				X		
PROBIT		X				X		
POWER	X	X	X	X			X	X

LINKPARAMETER Keyword

The LINKPARAMETER keyword specifies the value of α for the POWER link.

- Specify a number. There is no default.

FIXED Subcommand

The FIXED subcommand is used to specify fixed model effects, including the intercept.

- This subcommand is optional. If not specified, then the procedure builds an intercept-only model.

EFFECTS. The effect list includes all effects to be included in the fixed model except for the intercept, which is specified using the USE_INTERCEPT keyword.

- To include a term for the main effect of a factor (categorical predictor) or covariate (continuous predictor), enter its field name. Whether a field is treated as a factor or covariate depends upon its measurement level. Fields with categorical (flag, nominal, or ordinal) measurement level are treated as factors while fields with continuous (scale) measurement level are treated as covariates.
- To include a term for an interaction between factors, use the keyword BY or an asterisk (*) to join the factors involved in the interaction. For example, A*B means a two-way interaction effect of A and B, where A and B are factors. A*A is not allowed because factors in an interaction effect must be distinct.
- To include a term for nesting one effect within another, use a pair of parentheses. For example, A(B) means that A is nested within B.
- Multiple nesting is allowed. For example, A(B(C)) means that B is nested within C, and A is nested within B(C). When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, A(B)(C) is not valid.
- Interactions between nested effects are not valid. For example, neither A(C)*B(C) nor A(C)*B(D) is valid.
- Covariates can be connected, but not nested, through the * operator to form another covariate effect. Interactions among covariates such as X1*X1 and X1*X2 are valid, but X1(X2) is not.
- Factor and covariate effects can be connected only by the * operator. Suppose A and B are factors, and X1 and X2 are covariates. Examples of valid factor-by-covariate interaction effects are A*X1, A*B*X1, X1*A(B), A*X1*X1, and B*X1*X2.

USE_INTERCEPT = TRUE | FALSE.** This keyword controls whether an intercept term is included in the model. Specifying USE_INTERCEPT=FALSE when there are no effects in the model fits a "null" model. The default is TRUE.

RANDOM Subcommand

The RANDOM subcommand specifies the random effects in the mixed model.

- Depending on the covariance type specified, random effects specified in one RANDOM subcommand may be correlated.
- One block of the covariance G matrix will be constructed for each RANDOM subcommand. The dimension of the random effect covariance G matrix is equal to the sum of the levels of all random effects across all RANDOM subcommands.
- When the variance components (VC) structure is specified, a scaled identity (ID) structure will be assigned to each of the effects specified. This is the default covariance type for the RANDOM subcommand.
- Use a separate RANDOM subcommand when a different covariance structure is assumed for a list of random effects. If the same effect is listed on more than one RANDOM subcommand, it must be associated with a different SUBJECT combination.
- No random effects are included in the mixed model unless a RANDOM subcommand is specified correctly.

EFFECTS. The effect list includes all effects to be included in the random effects model block except for the intercept, which is specified using the USE_INTERCEPT keyword.

- To include a term for the main effect of a factor (categorical predictor) or covariate (continuous predictor), enter its field name. Whether a field is treated as a factor or covariate depends upon its measurement level.
- To include a term for an interaction between factors, use the keyword BY or an asterisk (*) to join the factors involved in the interaction. For example, A*B means a two-way interaction effect of A and B, where A and B are factors. A*A is not allowed because factors in an interaction effect must be distinct.
- To include a term for nesting one effect within another, use a pair of parentheses. For example, A(B) means that A is nested within B.
- Multiple nesting is allowed. For example, A(B(C)) means that B is nested within C, and A is nested within B(C). When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, A(B)(C) is not valid.
- Interactions between nested effects are not valid. For example, neither A(C)*B(C) nor A(C)*B(D) is valid.
- Covariates can be connected, but not nested, through the * operator to form another covariate effect. Interactions among covariates such as X1*X1 and X1*X2 are valid, but X1(X2) is not.
- Factor and covariate effects can be connected only by the * operator. Suppose A and B are factors, and X1 and X2 are covariates. Examples of valid factor-by-covariate interaction effects are A*X1, A*B*X1, X1*A(B), A*X1*X1, and B*X1*X2.

USE_INTERCEPT. This keyword controls whether an intercept term is included in the model. It is invalid to specify USE_INTERCEPT=FALSE if there are no effects specified on the EFFECTS keyword. The default is FALSE.

Examples

```
GENLINMIXED
  /FIXED EFFECTS=a b
  /RANDOM SUBJECTS=id.

GENLINMIXED
  /FIXED EFFECTS=a b
  /RANDOM USE_INTERCEPT=TRUE SUBJECTS=id.

GENLINMIXED
  /FIXED EFFECTS=a b
  /RANDOM EFFECTS=c d SUBJECTS=id.
```

- The first command fails because the RANDOM subcommand has no effects or intercept. The next two commands succeed.

SUBJECTS Keyword

The SUBJECTS keyword identifies subjects in the active dataset. Complete independence is assumed *across* subjects, but responses *within* subjects are assumed to be correlated.

- Specify a single field or a list of fields connected by asterisks (*) or the keyword BY.
- The number of subjects equals the number of distinct combinations of values of the fields.
- The fields in the RANDOM SUBJECTS list must be a subset of the fields on the DATA_STRUCTURE SUBJECTS list.
- The SUBJECTS keyword is optional.

GROUPING Keyword

The GROUPING keyword allows you to define independent sets of covariance parameters. All subjects have the same covariance type; subjects within the same covariance grouping will have the same values for the parameters.

- Specify a single categorical field or a list of categorical fields connected by asterisks (*) or the keyword BY.
- Each distinct combination of the values of the fields defines a separate covariance grouping.
- A field specified on the GROUPING keyword can also be used as a SUBJECTS or REPEATED_MEASURES field, the TARGET on the FIELDS subcommand, or on the EFFECTS keyword or the FIXED or RANDOM subcommand.
- The GROUPING keyword is optional.

COVARIANCE_TYPE Keyword

The COVARIANCE_TYPE keyword gives the covariance structure of the random effect covariance matrix.

- Random effects are considered independent of each other, and a separate covariance matrix is computed for each effect.

AR1. First-order autoregressive.

ARH1. Heterogenous first-order autoregressive.

ARMA11. Autoregressive moving average (1,1).

COMPOUND_SYMMETRY. This structure has constant variance and constant covariance.

CSH. Heterogenous compound symmetry. This structure has non-constant variance and constant correlation.

DIAGONAL. This is a diagonal structure with heterogenous variance.

IDENTITY. This is a scaled identity matrix.

UNSTRUCTURED. This is a completely general covariance matrix.

VARIANCE_COMPONENTS. *Variance components.* This is the default covariance structure for random effects. The variance components structure for random effects is a scaled identity (ID) structure assigned to each of the effects specified on the subcommand.

SOLUTION Keyword

The SOLUTION keyword specifies whether the Empirical Best Linear Unbiased Predictions table displays for the set of random effects specified on the current RANDOM subcommand. The Empirical Best Linear Unbiased Predictions table displays the predictions of the parameters for a particular random effect. The default is FALSE, which specifies that the random-effects parameter estimates are not displayed. The SOLUTION keyword has the following syntax in the context of RANDOM:

```
[/RANDOM]
    [SOLUTION = TRUE | FALSE**]
```

BUILD_OPTIONS Subcommand

The BUILD_OPTIONS subcommand specifies the criteria used to build the model.

TARGET_CATEGORY_ORDER. This determines the order of the target's categories for purposes of determining the "last" category, which is used as the reference category if the REFERENCE keyword is not used. This keyword is ignored if the target is not categorical.

- **ASCENDING.** *Target categories are sorted in ascending order, from the lowest value to the highest value.* The highest value is treated as the "last" category. This is the default.
- **DESCENDING.** *Target categories are sorted in descending order, from the highest value to the lowest value.* The lowest value is treated as the "last" category.
- **DATA.** *Target values are not sorted.* The first value encountered in the data defines the first category, the last value encountered defines the last category. This option is technically not valid if splits are defined on the SPLIT FILE command, but the procedure will run while it does not honor splits.

INPUTS_CATEGORY_ORDER. This determines the order of the categories for factors (categorical inputs) for purposes of determining the "last" category, which is used as the reference category. This keyword is ignored if there are no factors.

- **ASCENDING.** *Factor values are sorted in ascending order, from the lowest value to the highest value.* The highest value is treated as the "last" category. This is the default.
- **DESCENDING.** *Factor values are sorted in descending order, from the highest value to the lowest value.* The lowest value is treated as the "last" category.
- **DATA.** *Factor values are not sorted.* The first value encountered in the data defines the first category, the last value encountered defines the last category. This option is technically not valid if splits are defined on the SPLIT FILE command, but the procedure will run while it does not honor splits.

MAX_ITERATIONS = number. The model building algorithm stops after a certain number of iterations. By default, this is 100. Alternatively, specify a non-negative integer maximum number of iterations. The algorithm uses a doubly iterative process that consists of an inner loop and an outer loop. The value that is specified for the maximum number of iterations applies to both loops.

CONFIDENCE_LEVEL. This is the level of confidence used to compute interval estimates of the model coefficients. Specify a value greater than 0 and less than 100. The default is 95.

DF_METHOD. This specifies how degrees of freedom are computed for significance tests.

- **RESIDUAL.** The residual method has a fixed degrees of freedom for all tests. It is useful if your sample size is sufficiently large, or the data are balanced, or the model uses a simpler covariance type; for example, scaled identity or diagonal. This is the default.
- **SATTERTHWAITE.** The Satterthwaite method has a field degrees of freedom across tests. It is useful if your sample size is small, or the data are unbalanced, or the model uses a complicated covariance type; for example, unstructured.
- **KENWARD_ROGER.** The Kenward-Roger method offers a more precise small-sample estimator for the variance-covariance of the fixed effects parameters and the approximate denominator degrees of freedom in t-tests and F-tests. The method introduces a scale factor for the F-statistic and estimates it and the denominator degrees of freedom by using a Taylor series expansion for the estimated random structure within the data. The value of the estimated denominator degrees of freedom is the Kenward-Roger degrees of freedom in GLMM. The corresponding derived F- and t-statistic are used in EMMEANS and Customs Tests for statistical inference.

  Note: The Kenward-Roger method is used in the model based covariance (instead of robust covariance). When both the Kenward-Roger method and robust covariance is selected, the Kenward-Roger method is applied to model based covariance, and the following warning is presented: “Since Kenward-Roger method is selected, the robust covariance method is changed to model-based covariance method”.

COVB. This is the method for computing the parameter estimates covariance matrix.

- **MODEL.** The model-based estimate assumes that the model assumptions are correct. This is the default.
- **ROBUST.** The robust estimate is useful if you are concerned that the model assumptions are violated.

The model building algorithm uses a doubly iterative process that consists of an inner loop and an outer loop. The following settings apply to the inner loop.

PCONVERGE = number (ABSOLUTE | RELATIVE). *Parameter estimates convergence criterion.* Convergence is assumed if the maximum ABSOLUTE or maximum RELATIVE change in the parameter estimates is less than the specified value. The criterion is not used if the specified value is 0. Specify a non-negative value and the ABSOLUTE or RELATIVE keyword in parentheses to define the type of convergence. The default value is 1E-6 (ABSOLUTE).

Note: The default setting of PCONVERGE=0.000001(ABSOLUTE) might produce results that differ from the results that are obtained in versions before version 22. To reproduce results from pre-22 versions, use PCONVERGE=0.000001(RELATIVE).

LCONVERGE = number (ABSOLUTE | RELATIVE). *Log-likelihood function convergence criterion.* Convergence is assumed if the ABSOLUTE or RELATIVE change in the log-likelihood function is less than the specified value. The criterion is not used if the specified value is 0. This criterion is not used by default. Specify a non-negative value and the ABSOLUTE or RELATIVE keyword in parentheses to define the type of convergence.

HCONVERGE = number (ABSOLUTE | RELATIVE). *Hessian convergence criterion.* Convergence is assumed if $g'_k H_k^{-1} g_k$ is less than a multiplier of the specified value. The multiplier is 1 for ABSOLUTE type and is the absolute value of the current log-likelihood function for RELATIVE type. The criterion is not used if the specified value is 0. This criterion is not used by default. Specify a non-negative value and the ABSOLUTE or RELATIVE keyword in parentheses to define the type of convergence.

SCORING = integer. *Apply scoring algorithm.* Specify a non-negative integer. The default is 0, which specifies the Newton-Raphson method. Values greater than 0 specify to use the Fisher scoring algorithm up to iteration number *n*, where *n* is the specified integer, and Newton-Raphson thereafter.

SINGULAR = number. *Value used as tolerance in checking singularity.* Specify a positive value. The default value is 1E-12.

EMMEANS Subcommand

The EMMEANS subcommand displays estimated marginal means of the target for all level combinations of a set of factors. Note that these are predicted, not observed, means. Estimated marginal means can be computed based on the original scale of the target or the based on the link function transformation.

- Multiple EMMEANS subcommands are allowed. Each is treated independently.
- The EMMEANS subcommand can be specified with no additional keywords. The output for an empty EMMEANS subcommand is the overall estimated marginal mean of the response, collapsing over any factors.
- Estimated marginal means are not available if the multinomial distribution is used. If DISTRIBUTION = MULTINOMIAL on the MODEL subcommand and the EMMEANS subcommand is specified, then EMMEANS is ignored and a warning is issued.

The TABLES keyword specifies the cells for which estimated marginal means are displayed.

- Valid options are all-factor effects appearing on the EFFECTS keyword of the FIXED subcommand, including main effects factors such as a b, all-factor interactions such as a*b, and all-factor nested effects such as a(b), if *a* and *b* are factors.
- If the TABLES keyword is specified, then the procedure collapses over any other factors specified on the command but not on the TABLES keyword before computing the estimated marginal means for the target.
- If the TABLES keyword is not specified, then the overall estimated marginal mean of the target, collapsing over any factors, is computed.

Examples

```
GENLINMIXED
  /FIXED EFFECTS=a b
  /EMMEANS TABLES=a*b.

GENLINMIXED
  /FIXED EFFECTS=a*b
  /EMMEANS TABLES=a
  /EMMEANS TABLES=b.

GENLINMIXED
  /FIXED EFFECTS=a b a*b
  /EMMEANS TABLES=a
  /EMMEANS TABLES=b
  /EMMEANS TABLES=a*b.
```

- The first two commands fail because the effects listed on the TABLES keywords are not listed on the EFFECTS keyword. The third command succeeds.

COMPARE Keyword

The COMPARE keyword specifies a factor, the levels of which are compared using the contrast type specified on the CONTRAST keyword.

- Valid options are factors appearing on the TABLES keyword.
- The COMPARE keyword is valid only if the TABLES and CONTRAST keyword is also specified.
- By default, the procedure sorts levels of the factors in ascending order and defines the highest level as the last level. (If the factor is a string variable, then the value of the highest level is locale-dependent.) However, the sort order can be modified using the INPUTS_SORT_ORDER keyword on the BUILD_OPTIONS subcommand.

CONTRAST Keyword

The CONTRAST keyword specifies the type of contrast to use for the levels of the factor on the COMPARE keyword. The CONTRAST keyword creates an **L** matrix (that is, a coefficient matrix) such that the columns corresponding to the factor match the contrast given. The other columns are adjusted so that the **L** matrix is estimable.

- The CONTRAST keyword is valid only if the COMPARE keyword is also specified.
- If the COMPARE keyword is specified without CONTRAST, then pairwise comparisons are performed for the factor on COMPARE.
- SIMPLE contrasts are defined with respect to the last level as determined by the INPUTS_CATEGORY_ORDER specification on the BUILD_OPTIONS subcommand.

The following contrast types are available.

NONE. *No comparisons.* This is the default.

PAIRWISE. *Pairwise comparisons are computed for all levels of the specified or factor.* Pairwise contrasts are not orthogonal.

DEVIATION. *Each level of the factor is compared to the grand mean.* Deviation contrasts are not orthogonal.

SIMPLE. *Each level of the factor except the last is compared to the last level.* Simple contrasts are not orthogonal.

EMMEANS_OPTIONS Subcommand

The EMMEANS_OPTIONS subcommand contains specifications that are used across all EMMEANS subcommands.

CONTROL Keyword

The CONTROL keyword specifies the covariate values to use when computing the estimated marginal means.

- Specify one or more covariates appearing on the FIXED subcommand EFFECTS list, each of which must be followed by a numeric value.
- If a covariate is not specified on the CONTROL option, then its overall mean will be used in estimated marginal means calculations.
- Any covariate can occur only once on the CONTROL keyword.

SCALE Keyword

The SCALE keyword specifies whether to compute estimated marginal means based on the original scale of the target or based on the link function transformation.

ORIGINAL. *Estimated marginal means are based on the original scale of the target.* Estimated marginal means are computed for the target. This is the default. Note that when the target is specified using the events/trials option, ORIGINAL gives the estimated marginal means for the events/trials proportion rather than for the number of events.

TRANSFORMED. *Estimated marginal means are based on the link function transformation.* Estimated marginal means are computed for the linear predictor.

PADJUST Keyword

The PADJUST keyword indicates the method of adjusting the significance level.

LSD. *Least significant difference.* This method does not control the overall probability of rejecting the hypotheses that some linear contrasts are different from the null hypothesis value(s). This is the default.

SEQBONFERRONI. *Sequential Bonferroni.* This is a sequentially step-down rejective Bonferroni procedure that is much less conservative in terms of rejecting individual hypotheses but maintains the same overall significance level.

SEQSIDAK. *Sequential Sidak.* This is a sequentially step-down rejective Sidak procedure that is much less conservative in terms of rejecting individual hypotheses but maintains the same overall significance level.

OUTFILE Subcommand

The OUTFILE subcommand saves a file containing the scoring model.

MODEL = 'file'. Writes the fixed effects portion of the model to an external *.zip* file. You can use this model file to apply the model information to other data files for scoring purposes. See the topic "Scoring expressions" on page 86 for more information. Specify a unique, valid filename. If the file specification refers to an existing file, then the file is overwritten.

SAVE Subcommand

The SAVE subcommand writes optional temporary fields to the active dataset.

PREDICTED_VALUES(varname). *Predicted value of the target.* The default field name is *PredictedValue*.

PREDICTED_PROBABILITY(rootname). *Predicted probabilities of the target.* If the target is categorical, this keyword saves the predicted probabilities of the first *n* categories. The values are cumulative probabilities for ordinal targets. The value of *n* is specified by the MAX_CATEGORIES keyword.

- Specify a unique, valid variable name. The default root name is *PredictedProbability*.
- If the dependent variable is continuous, this keyword is ignored with a warning.

MAX_CATEGORIES. *The maximum number of categories for which predicted probabilities and confidence intervals (for a categorical target) should be saved.*

- Specify a positive integer. The default is 25.
- This keyword is ignored if predicted probabilities or confidence intervals (for a categorical target) are not saved.

CONFIDENCE(varname). *Confidence in predicted value for categorical target.* The computed confidence can be based on the probability of the predicted value (the highest predicted probability) or the difference between the highest predicted probability and the second highest predicted probability. The default field name is *Confidence*.

CONFIDENCE_METHOD. *Method for computing confidence.* The computed confidence can be based on the probability of the predicted value (the highest predicted probability) or the difference between the highest predicted probability and the second highest predicted probability.

CONFIDENCE_INTERVALS(rootname). *Upper and lower confidence intervals.* For all distributions except the multinomial, this creates two variables and the default root name is CI, with _Lower and _Upper as the suffixes. Specify a root name in parentheses to override the default.

For the multinomial distribution and a nominal target, one field is created for each dependent variable category. `CONFIDENCE_INTERVALS` saves the lower and upper bounds of the predicted probability for the first 25 categories by default. The default root name is *CI*, and the default field names are *CI_Lower_1*, *CI_Upper_1*, *CI_Lower_2*, *CI_Upper_2*, and so on, corresponding to the order of the target categories. Specify a root name in parentheses to override the default. The value of *n* is specified by the `MAX_CATEGORIES` keyword.

For the multinomial distribution and an ordinal target, one field is created for each dependent variable category except the last (see the `TARGET_SORT_ORDER` keyword). `CONFIDENCE_INTERVALS` saves the lower and upper bounds of the cumulative predicted probability for the first 25 categories, up to but not including the last, by default. The default root name is *CI*, and the default field names are *CI_Lower_1*, *CI_Upper_1*, *CI_Lower_2*, *CI_Upper_2*, and so on, corresponding to the order of the target categories. Specify a root name in parentheses to override the default. The value of *n* is specified by the `MAX_CATEGORIES` keyword.

PEARSON_RESIDUALS(varname). *Pearson residuals.* The default field name is *PearsonResidual*.

GENLOG

GENLOG is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
GENLOG varlist[BY] varlist [WITH covariate varlist]

 [/CSTRUCTURE=varname]

 [/GRESID=varlist]

 [/GLOR=varlist]

 [/MODEL={POISSON**  }]
         {MULTINOMIAL}

 [/CRITERIA=[CONVERGE({0.001**})][ITERATE({20**})][DELTA({0.5**})]
                      {n      }           {n   }         {n    }

            [CIN({95**})] [EPS({1E-8**})]
                 {n   }       {n     }

            [DEFAULT]

 [/PRINT=[FREQ**][RESID**][ADJRESID**][DEV**]
         [ZRESID][ITERATION][COV][DESIGN][ESTIM][CORR]
         [ALL] [NONE]
         [DEFAULT]]

 [/PLOT={DEFAULT**                 }]
        {RESID([ADJRESID][DEV])    }
        {NORMPROB([ADJRESID][DEV]) }
        {NONE                      }

 [/SAVE=tempvar (newvar)[tempvar (newvar)...]]

 [/MISSING=[{EXCLUDE**}]]
           {INCLUDE  }

 [/DESIGN=effect[(n)] effect[(n)]... effect {BY} effect...]
                                           {*  }
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
GENLOG DPREF RACE CAMP.
```

Overview

GENLOG is a general procedure for model fitting, hypothesis testing, and parameter estimation for any model that has categorical variables as its major components. As such, GENLOG subsumes a variety of related techniques, including general models of multiway contingency tables, logit models, logistic regression on categorical variables, and quasi-independence models.

GENLOG, following the regression approach, uses dummy coding to construct a design matrix for estimation and produces maximum likelihood estimates of parameters by means of the Newton-Raphson algorithm. Since the regression approach uses the original parameter spaces, the parameter estimates correspond to the original levels of the categories and are therefore easier to interpret.

HILOGLINEAR, which uses an iterative proportional-fitting algorithm, is more efficient for hierarchical models and useful in model building, but it cannot produce parameter estimates for unsaturated models, does not permit specification of contrasts for parameters, and does not display a correlation matrix of the parameter estimates.

The General Loglinear Analysis and Logit Loglinear Analysis dialog boxes are both associated with the GENLOG command. In previous releases, these dialog boxes were associated with the LOGLINEAR command. The LOGLINEAR command is now available only as a syntax command. The differences are described in the discussion of the LOGLINEAR command.

Options

Cell Weights. You can specify cell weights (such as structural zero indicators) for the model with the CSTRUCTURE subcommand.

Linear Combinations. You can compute linear combinations of observed cell frequencies, expected cell frequencies, and adjusted residuals using the GRESID subcommand.

Generalized Log-Odds Ratios. You can specify contrast variables on the GLOR subcommand and test whether the generalized log-odds ratio equals 0.

Model Assumption. You can specify POISSON or MULTINOMIAL on the MODEL subcommand to request the Poisson loglinear model or the product multinomial loglinear model.

Tuning the Algorithm. You can control the values of algorithm-tuning parameters with the CRITERIA subcommand.

Output Display. You can control the output display with the PRINT subcommand.

Optional Plots. You can request plots of adjusted or deviance residuals against observed and expected counts, or normal plots and detrended normal plots of adjusted or deviance residuals using the PLOT subcommand.

Basic Specification

The basic specification is one or more factor variables that define the tabulation. By default, GENLOG assumes a Poisson distribution and estimates the saturated model. Default output includes the factors or effects, their levels, and any labels; observed and expected frequencies and percentages for each factor and code; and residuals, adjusted residuals, and deviance residuals.

Limitations

- Maximum 10 factor variables (dependent *and* independent).
- Maximum 200 covariates.

Subcommand Order

- The variable specification must come first.
- Subcommands can be specified in any order.
- When multiple subcommands are specified, only the last specification takes effect.

Variable List

The variable list specifies the variables to be included in the model. GENLOG analyzes two classes of variables—categorical and continuous. Categorical variables are used to define the cells of the table. Continuous variables are used as cell covariates.

- The list of categorical variables must be specified first. Categorical variables must be numeric.
- Continuous variables can be specified only after the WITH keyword following the list of categorical variables.
- To specify a logit model, use the keyword BY (see “Logit Model ” on page 770). A variable list without the keyword BY generates a general loglinear model.
- A variable can be specified only once in the variable list—as a dependent variable immediately following GENLOG, as an independent variable following the keyword BY, or as a covariate following the keyword WITH.
- No range needs to be specified for categorical variables.

Logit Model

The logit model examines the relationships between dependent and independent factor variables.

- To separate the independent variables from the dependent variables in a logit model, use the keyword BY. The categorical variables preceding BY are the dependent variables; the categorical variables following BY are the independent variables.
- Up to 10 variables can be specified, including both dependent and independent variables.
- For the logit model, you must specify MULTINOMIAL on the MODEL subcommand.
- GENLOG displays an analysis of dispersion and two measures of association—entropy and concentration. These measures are discussed elsewhere [31] and can be used to quantify the magnitude of association among the variables. Both are proportional-reduction-in-error measures. The entropy statistic is analogous to Theil's entropy measure, while the concentration statistic is analogous to Goodman and Kruskal's tau-*b.* Both statistics measure the strength of association between the dependent variable and the independent variable set.

Example

```
GENLOG  GSLEVEL BY EDUC SEX
  /MODEL=MULTINOMIAL
  /DESIGN=GSLEVEL, GSLEVEL BY EDUC, GSLEVEL BY SEX.
```

- The keyword BY on the variable list specifies a logit model in which *GSLEVEL* is the dependent variable and *EDUC* and *SEX* are the independent variables.
- A logit model is multinomial.
- DESIGN specifies a model that can test for the absence of the joint effect of *SEX* and *EDUC* on *GSLEVEL*.

Cell Covariates

- Continuous variables can be used as covariates. When used, the covariates must be specified after the WITH keyword following the list of categorical variables.
- A variable cannot be named as both a categorical variable and a cell covariate.
- To enter cell covariates into a model, the covariates must be specified on the DESIGN subcommand.
- Cell covariates are not applied on a case-by-case basis. The weighted covariate mean for a cell is applied to that cell.

Example

```
GENLOG DPREF RACE CAMP WITH X
  /DESIGN=DPREF RACE CAMP X.
```

- The variable *X* is a continuous variable specified as a cell covariate. Cell covariates must be specified after the keyword WITH following the variable list. No range is defined for cell covariates.
- To include the cell covariate in the model, the variable *X* is specified on DESIGN.

CSTRUCTURE Subcommand

CSTRUCTURE specifies the variable that contains values for computing cell weights, such as structural zero indicators. By default, cell weights are equal to 1.

- The specification must be a numeric variable.
- Variables specified as dependent or independent variables in the variable list cannot be specified on CSTRUCTURE.
- Cell weights are not applied on a case-by-case basis. The weighted mean for a cell is applied to that cell.
- CSTRUCTURE can be used to impose structural, or *a priori*, zeros on the model. This feature is useful in specifying a quasi-symmetry model and in excluding cells from entering into estimation.

[31] Haberman, S. J. 1982. Analysis of dispersion of multinomial responses. *Journal of the American Statistical Association,* 77, 568-580.

- If multiple CSTRUCTURE subcommands are specified, the last specification takes effect.

Example

```
COMPUTE  CWT=(HUSED NE WIFED).
GENLOG HUSED WIFED WITH DISTANCE
  /CSTRUCTURE=CWT
  /DESIGN=HUSED WIFED DISTANCE.
```

- The Boolean expression assigns *CWT* the value of 1 when *HUSED* is not equal to *WIFED*, and the value of 0 otherwise.
- CSTRUCTURE imposes structural zeros on the diagonal of the symmetric crosstabulation.

GRESID Subcommand

GRESID (Generalized Residual) calculates linear combinations of observed and expected cell frequencies as well as simple, standardized, and adjusted residuals.

- The variables specified must be numeric, and they must contain coefficients of the desired linear combinations.
- Variables specified as dependent or independent variables in the variable list cannot be specified on GRESID.
- The generalized residual coefficient is not applied on a case-by-case basis. The weighted coefficient mean of the value for all cases in a cell is applied to that cell.
- Each variable specified on the GRESID subcommand contains a single linear combination.
- If multiple GRESID subcommands are specified, the last specification takes effect.

Example

```
COMPUTE GR_1=(MONTH LE 6).
COMPUTE GR_2=(MONTH GE 7).
GENLOG  MONTH WITH Z
 /GRESID=GR_1 GR_2
 /DESIGN=Z.
```

- The first variable, *GR_1*, combines the first six months into a single effect; the second variable, *GR_2*, combines the rest of the months.
- For each effect, GENLOG displays the observed and expected counts as well as the simple, standardized, and adjusted residuals.

GLOR Subcommand

GLOR (Generalized Log-Odds Ratio) specifies the population contrast variable(s). For each variable specified, GENLOG tests the null hypothesis that the generalized log-odds ratio equals 0 and displays the Wald statistic and the confidence interval. You can specify the level of the confidence interval using the CIN significance-level keyword on CRITERIA. By default, the confidence level is 95%.

- The variable sum is 0 for the loglinear model and for each combined level of independent variables for the logit model.
- Variables specified as dependent or independent variables in the variable list cannot be specified on GLOR.
- The coefficient is not applied on a case-by-case basis. The weighted mean for a cell is applied to that cell.
- If multiple GLOR subcommands are specified, the last specification takes effect.

Example

```
GENLOG A B
 /GLOR=COEFF
 /DESIGN=A B.
```

- The variable *COEFF* contains the coefficients of two dichotomous factors *A* and *B*.
- If the weighted cell mean for *COEFF* is 1 when *A* equals *B* and –1 otherwise, this example tests whether the log-odds ratio equals 0, or in this case, whether variables *A* and *B* are independent.

MODEL Subcommand

MODEL specifies the assumed distribution of your data.

- You can specify only one keyword on MODEL. The default is POISSON.
- If more than one MODEL subcommand is specified, the last specification takes effect.

POISSON. *The Poisson distribution*. This is the default.

MULTINOMIAL. *The multinomial distribution.* For the logit model, you must specify MULTINOMIAL.

CRITERIA Subcommand

CRITERIA specifies the values used in tuning the parameters for the Newton-Raphson algorithm.

- If multiple CRITERIA subcommands are specified, the last specification takes effect.

CONVERGE(n). *Convergence criterion.* Specify a positive value for the convergence criterion. The default is 0.001.

ITERATE(n). *Maximum number of iterations.* Specify an integer. The default number is 20.

DELTA(n). *Cell delta value.* Specify a non-negative value to add to each cell frequency for the first iteration. (For the saturated model, the delta value is added for all iterations.) The default is 0.5. The delta value is used to solve mathematical problems created by 0 observations; if all of your observations are greater than 0, we recommend that you set DELTA to 0.

CIN(n). *Level of confidence interval.* Specify the percentage interval used in the test of generalized log-odds ratios and parameter estimates. The value must be between 50 and 99.99, inclusive. The default is 95.

EPS(n). *Epsilon value used for redundancy checking in design matrix.* Specify a positive value. The default is 0.00000001.

DEFAULT. *Default values are used.* DEFAULT can be used to reset all criteria to default values.

Example

```
GENLOG  DPREF BY RACE ORIGIN CAMP
 /MODEL=MULTINOMIAL
 /CRITERIA=ITERATION(50) CONVERGE(.0001).
```

- ITERATION increases the maximum number of iterations to 50.
- CONVERGE lowers the convergence criterion to 0.0001.

PRINT Subcommand

PRINT controls the display of statistics.

- By default, GENLOG displays the frequency table and simple, adjusted, and deviance residuals.
- When PRINT is specified with one or more keywords, only the statistics requested by these keywords are displayed.
- When multiple PRINT subcommands are specified, the last specification takes effect.

The following keywords can be used on PRINT:

FREQ. *Observed and expected cell frequencies and percentages.* This is displayed by default.

RESID. *Simple residuals.* This is displayed by default.

ZRESID. *Standardized residuals.*

ADJRESID. *Adjusted residuals.* This is displayed by default.

DEV. *Deviance residuals.* This is displayed by default.

DESIGN. *The design matrix of the model.* The design matrix corresponding to the specified model is displayed.

ESTIM. *The parameter estimates of the model.* The parameter estimates refer to the original categories.

CORR. *The correlation matrix of the parameter estimates.*

COV. *The covariance matrix of the parameter estimates.*

ALL. *All available output.*

DEFAULT. *FREQ, RESID, ADJRESID, and DEV.* This keyword can be used to reset PRINT to its default setting.

NONE. *The design and model information with goodness-of-fit statistics only.* This option overrides all other specifications on the PRINT subcommand.

Example

```
GENLOG A B
 /PRINT=ALL
 /DESIGN=A B.
```

- The DESIGN subcommand specifies a main-effects model, which tests the hypothesis of no interaction. The PRINT subcommand displays all available output for this model.

PLOT Subcommand

PLOT specifies which plots you want to display. Plots of adjusted residuals against observed and expected counts, and normal and detrended normal plots of the adjusted residuals are displayed if PLOT is not specified or is specified without a keyword. When multiple PLOT subcommands are specified, only the last specification is executed.

DEFAULT. *RESID (ADJRESID) and NORMPROB (ADJRESID).* This is the default if PLOT is not specified or is specified with no keyword.

RESID (type). *Plots of residuals against observed and expected counts.* You can specify the type of residuals to plot. ADJRESID plots adjusted residuals; DEV plots deviance residuals. ADJRESID is the default if you do not specify a type.

NORMPROB (type). *Normal and detrended normal plots of the residuals.* You can specify the type of residuals to plot. ADJRESID plots adjusted residuals; DEV plots deviance residuals. ADJRESID is the default if you do not specify a type.

NONE. *No plots.*

Example

```
GENLOG  RESPONSE BY SEASON
  /MODEL=MULTINOMIAL
  /PLOT=RESID(ADJRESID,DEV)
  /DESIGN=RESPONSE SEASON(1) BY RESPONSE.
```

- This example requests plots of adjusted and deviance residuals against observed and expected counts.
- Note that if you specify /PLOT=RESID(ADJRESID) RESID(DEV), only the deviance residuals are plotted. The first keyword specification, RESID(ADJRESID), is ignored.

MISSING Subcommand

MISSING controls missing values. By default, GENLOG excludes all cases with system- or user-missing values for any variable. You can specify INCLUDE to include user-missing values.

EXCLUDE. *Delete cases with user-missing values.* This is the default if the subcommand is omitted. You can also specify the keyword DEFAULT.

INCLUDE. *Include cases with user-missing values.* Only cases with system-missing values are deleted.

Example

```
MISSING VALUES A(0).
GENLOG A B
 /MISSING=INCLUDE
 /DESIGN=B.
```

- Even though 0 was specified as missing, it is treated as a nonmissing category of *A* in this analysis.

SAVE Subcommand

SAVE saves specified temporary variables into the active dataset. You can assign a new name to each temporary variable saved.

- The temporary variables you can save include *RESID* (raw residual), *ZRESID* (standardized residual), *ADJRESID* (adjusted residual), *DEV* (deviance residual), and *PRED* (predicted cell frequency). An explanatory label is assigned to each saved variable.
- A temporary variable can be saved only once on a SAVE subcommand.
- To assign a name to a saved temporary variable, specify the new name in parentheses following that temporary variable. The new name must conform to IBM SPSS Statistics naming conventions and must be unique in the active dataset. The names cannot begin with # or $.
- If you do not specify a variable name in parentheses, GENLOG assigns default names to the saved temporary variables. A default name starts with the first three characters of the name of the saved temporary variable, followed by an underscore and a unique number. For example, *RESID* will be saved as *RES_n*, where *n* is a number incremented each time a default name is assigned to a saved *RESID*.
- The saved variables are pertinent to cells in the contingency table, not to individual observations. In the Data Editor, all cases that define one cell receive the same value. To make sense of these values, you need to aggregate the data to obtain cell counts.

Example

```
GENLOG A B
 /SAVE PRED (PREDA_B)
 /DESIGN = A, B.
```

- SAVE saves the predicted values for two independent variables *A* and *B*.
- The saved variable is renamed *PREDA_B* and added to the active dataset.

DESIGN Subcommand

DESIGN specifies the model to be fit. If DESIGN is omitted or used with no specifications, the saturated model is produced. The saturated model fits all main effects and all interaction effects.

- Only one design can be specified on the subcommand.
- To obtain main-effects models, name all of the variables listed on the variables specification.
- To obtain interactions, use the keyword BY or an asterisk (*) to specify each interaction, for example, A BY B or C*D. To obtain the single-degree-of-freedom partition of a specified factor, specify the partition in parentheses following the factor (see the example below).

- To include cell covariates in the model, first identify them on the variable list by naming them after the keyword WITH, and then specify the variable names on DESIGN.
- Effects that involve only independent variables result in redundancy. GENLOG removes these effects from the model.
- If your variable list includes a cell covariate (identified by the keyword WITH), you cannot imply the saturated model by omitting DESIGN or specifying it alone. You need to request the model explicitly by specifying all main effects and interactions on DESIGN.

Example

```
COMPUTE X=MONTH.
GENLOG MONTH WITH X
  /DESIGN X.
```

- This example tests the linear effect of the dependent variable.
- The variable specification identifies *MONTH* as a categorical variable. The keyword WITH identifies *X* as a covariate.
- DESIGN tests the linear effect of *MONTH*.

Example

```
GENLOG A B
  /DESIGN=A.


GENLOG A B
  /DESIGN=A,B.
```

- Both designs specify main-effects models.
- The first design tests the homogeneity of category probabilities for *B*; it fits the marginal frequencies on *A* but assumes that membership in any of the categories of *B* is equiprobable.
- The second design tests the independence of *A* and *B*. It fits the marginals on both *A* and *B*.

Example

```
GENLOG A  B  C
  /DESIGN=A,B,C, A BY B.
```

- This design consists of the *A* main effect, the *B* main effect, the *C* main effect, and the interaction of *A* and *B*.

Example

```
GENLOG A BY B
 /MODEL=MULTINOMIAL
 /DESIGN=A,A BY B(1).
```

- This example specifies single-degree-of-freedom partitions.
- The value 1 following *B* to the first category of *B*.

Example

```
GENLOG HUSED WIFED WITH DISTANCE
  /DESIGN=HUSED WIFED DISTANCE.
```

- The continuous variable *DISTANCE* is identified as a cell covariate by the keyword WITH. The cell covariate is then included in the model by naming it on DESIGN.

Example

```
COMPUTE  X=1.
GENLOG  MONTH WITH X
  /DESIGN=X.
```

- This example specifies an equiprobability model.
- The design tests whether the frequencies in the table are equal by using a constant of 1 as a cell covariate.

References

Haberman, S. J. 1982. Analysis of dispersion of multinomial responses. *Journal of the American Statistical Association,* 77, 568-580.

GET

```
GET FILE='file' [LOCK={YES**}] [PASSWORD='password']
                      {NO   }

 [/KEEP={ALL**  }] [/DROP=varlist]
        {varlist}

 [/RENAME=(old varnames=new varnames)...]

 [/MAP]
```

**Default if the subcommand is omitted.

Release History

Release 17.0

- LOCK keyword introduced on the FILE subcommand.

Release 21.0

- PASSWORD keyword introduced on the FILE subcommand.

Example

```
GET FILE='/data/empl.sav'.
```

Overview

GET reads IBM SPSS Statistics data files. It also reads SPSS/PC+ data files, but you should not read SPSS/PC+ data files in Unicode mode (see Operations below).

GET is used only for reading IBM SPSS Statistics data files. See DATA LIST for information on reading and defining data in a text data file. See MATRIX DATA for information on defining matrix materials in a text data file. For information on defining complex data files that cannot be defined with DATA LIST alone, see FILE TYPE and REPEATING DATA.

The program can also read data files created for other software applications. See IMPORT for information on reading *portable files* created with EXPORT. See the relevant commands, such as GET DATA, GET STATA, and GET SAS, for information on reading files created by other software programs.

Options

Variable Subsets and Order. You can read a subset of variables and reorder the variables that are copied into the active dataset using the DROP and KEEP subcommands.

Variable Names. You can rename variables as they are copied into the active dataset with the RENAME subcommand. Long variable names are supported when explicitly specified on the /VARIABLES subcommand. Variable names are truncated to 8 bytes when they are referenced through a vector in the /NAMES subcommand.

Variable Map. To confirm the names and order of variables in the active dataset, use the MAP subcommand.

Basic Specification

- The basic specification is the FILE subcommand, which specifies the IBM SPSS Statistics data file to be read.

Subcommand Order

- FILE must be specified first.
- The remaining subcommands can be specified in any order.

Syntax Rules

- FILE is required and can be specified only once.
- KEEP, DROP, RENAME, and MAP can be used as many times as needed.
- GET cannot be used inside a DO IF–END IF or LOOP–END LOOP structure.

Operations

- If KEEP is not specified, variables in the active dataset are in the same order as the original data file.
- A file saved with weighting in effect maintains weighting the next time the file is accessed. For a discussion of turning off weights, see WEIGHT.
- In Unicode mode, for code page data files and data files created in releases prior to 16.0, the defined width of string variables in tripled. You can use ALTER TYPE to automatically adjust the width of all string variables.See SET command, UNICODE subcommand for more information.
- In Unicode mode, SPSS/PC+ data files may not be read correctly.
- SPSS/PC+ data files can be read only on Windows operating systems.

FILE Subcommand

FILE specifies the IBM SPSS Statistics data file to be read. FILE is required and can be specified only once. It must be the first specification on GET.

LOCK Keyword

The LOCK keyword only applies when opening a file from a IBM SPSS Collaboration and Deployment Services Repository. It specifies whether the file is locked (in the repository) when opened. Locking a file prevents other users from modifying it while you are working with it. Other users can open locked files but they cannot save them back to the repository. The lock is released when the file is closed or when the current session ends.

- By default, files are opened as locked.
- The LOCK keyword is ignored if the specified file does not reside in a IBM SPSS Collaboration and Deployment Services Repository.

See the topic "File Specifications for IBM SPSS Collaboration and Deployment Services Repository Objects" on page 2217 for more information.

PASSWORD Keyword

The PASSWORD keyword specifies the password required to open an encrypted IBM SPSS Statistics data file. The specified value must be enclosed in quotation marks and can be provided as encrypted or as plain text. Encrypted passwords are created when pasting command syntax from the Save Data As dialog. The PASSWORD keyword is ignored if the file is not encrypted.

DROP and KEEP Subcommands

DROP and KEEP are used to copy a subset of variables into the active dataset. DROP specifies variables that should not be copied into the active dataset. KEEP specifies variables that should be copied. Variables not specified on KEEP are dropped.

- Variables can be specified in any order. The order of variables on KEEP determines the order of variables in the active dataset. The order of variables on DROP does not affect the order of variables in the active dataset.
- The keyword ALL on KEEP refers to all remaining variables not previously specified on KEEP. ALL must be the last specification on KEEP.
- If a variable is specified twice on the same subcommand, only the first mention is recognized.
- Multiple DROP and KEEP subcommands are allowed. However, specifying a variable named on a previous DROP or not named on a previous KEEP results in an error, and the GET command is not executed.

- The keyword TO can be used to specify a group of consecutive variables in the data file.

Example

```
GET FILE='/data/hubtemp.sav'   /DROP=DEPT79 TO DEPT84 SALARY79.
```

- All variables between and including *DEPT79* and *DEPT84*, as well as *SALARY79*, are excluded from the active dataset. All other variables are included in the active dataset.
- Variables in the active dataset are in the same order as the variables in the original file.

Example

```
GET FILE='/data/prsnl.sav' /DROP=GRADE STORE
              /KEEP=LNAME NAME TENURE JTENURE ALL.
```

- The variables *GRADE* and *STORE* are dropped when the file *prsnl.sav* is copied into the active dataset.
- KEEP specifies that *LNAME*, *NAME*, *TENURE*, and *JTENURE* are the first four variables in the active dataset, followed by all remaining variables (except those dropped by the previous DROP subcommand). These remaining variables are copied into the active dataset in the same sequence in which they appear in the *prsnl.sav* file.

RENAME Subcommand

RENAME changes the names of variables as they are copied into the active dataset.

- The specification on RENAME is a list of old variable names followed by an equals sign and a list of new variable names. The same number of variables must be specified on both lists. The keyword TO can be used on the first list to refer to consecutive variables in the data file and on the second list to generate new variable names. The entire specification must be enclosed in parentheses.
- Alternatively, you can specify each old variable name individually, followed by an equals sign and the new variable name. Multiple sets of variable specifications are allowed. The parentheses around each set of specifications are optional.
- Old variable names do not need to be specified according to their order in the data file.
- Name changes take place in one operation. Therefore, variable names can be exchanged between two variables.
- Variables cannot be renamed to scratch variables.
- Multiple RENAME subcommands are allowed.
- On a subsequent DROP or KEEP subcommand, variables are referred to by their new names.

Example

```
GET FILE='/data/empl88.sav'
  /RENAME  AGE=AGE88 JOBCAT=JOBCAT88.
```

- RENAME specifies two name changes for the active dataset. *AGE* is renamed to *AGE88* and *JOBCAT* is renamed to *JOBCAT88*.

Example

```
GET FILE='/data/empl88.sav'
  /RENAME (AGE JOBCAT=AGE88 JOBCAT88).
```

- The name changes are identical to those in the previous example. *AGE* is renamed to *AGE88* and *JOBCAT* is renamed to *JOBCAT88*. The parentheses are required with this method.

MAP Subcommand

MAP displays a list of the variables in the active dataset and their corresponding names in the original data file.

- The only specification is the keyword MAP. There are no additional specifications.
- Multiple MAP subcommands are allowed. Each MAP subcommand maps the results of subcommands that precede it; results of subcommands that follow it are not mapped.

Example

```
GET FILE='/data/empl88.sav'
 /RENAME=(AGE=AGE88) (JOBCAT=JOBCAT88)
 /KEEP=LNAME NAME JOBCAT88 ALL /MAP.
```

- MAP is specified to confirm the new names for the variables *AGE* and *JOBCAT* and the order of variables in the active dataset (*LNAME*, *NAME*, and *JOBCAT88*, followed by all remaining variables in the data file).

GET CAPTURE

GET CAPTURE is supported for compatibility purposes. GET DATA is the preferred command for reading databases. See the topic “GET DATA” on page 785 for more information.

```
GET CAPTURE {ODBC    }*

 [/CONNECT='connection string']
 [/UNENCRYPTED]
 [/LOGIN=login] [/PASSWORD=password]
 [/SERVER=host] [/DATABASE=database name]†

 /SQL 'select statement'
      ['continuation of select statement']
```

* You can import data from any database for which you have an ODBC driver installed.

† Optional subcommands are database-specific. For more information, see “Overview ” on page 783 below.

Release History

Release 15.0

- UNENCRYPTED subcommand introduced.

Example

```
GET CAPTURE ODBC
  /CONNECT='DSN=sales.mdb;DBQ=/data/saledata.mdb;DriverId=281;FIL=MS'+
  ' Access;MaxBufferSize=2048;PageTimeout=5;'
  /SQL = 'SELECT  T0.ID AS ID`,  T0.JOBCAT AS JOBCAT,  '
  ''T0`.`REGION` AS `REGION`,  `T0`.`DIVISION` AS `DIVISION`,`T0`.`TRAVEL`'
  ' AS `TRAVEL`,  `T0`.`SALES` AS `SALES`,  `T0`.`VOLUME96` AS `VOLUME96`,  '
  ''T1`.`REGION` AS `REGION1`,  `T1`.`AVGINC` AS `AVGINC`,`T1`.`AVGAGE` AS'
  ' `AVGAGE`,  `T1`.`POPULAT` AS `POPULAT` FROM { oj `Regions` `T1` LEFT '
  'OUTER JOIN `EmployeeSales` `T0` ON `T1`.`REGION` = `T0`.`REGION` } '.
```

Overview

GET CAPTURE retrieves data from a database and converts them to a format that can be used by program procedures. GET CAPTURE retrieves data and data information and builds an active dataset for the current session.

Note: Although GET CAPTURE is still supported, equivalent functionality and additional features are provided in the newer GET DATA command.

Basic Specification

The basic specification is one of the subcommands specifying the database type followed by the SQL subcommand and any select statement in quotation marks or apostrophes. Each line of the select statement should be enclosed in quotation marks or apostrophes, and no quoted string should exceed 255 characters.

Subcommand Order

The subcommand specifying the type of database must be the first specification. The SQL subcommand must be the last.

Syntax Rules

- Only one subcommand specifying the database type can be used.
- The CONNECT subcommand must be specified if you use the Microsoft ODBC (Open Database Connectivity) driver.

Operations

- GET CAPTURE retrieves the data specified on SQL.

- The variables are in the same order in which they are specified on the SQL subcommand.
- The data definition information captured from the database is stored in the active dataset dictionary.

Limitations

- A maximum of 3,800 characters (approximately) can be specified on the SQL subcommand. This translates to 76 lines of 50 characters. Characters beyond the limit are ignored.

CONNECT Subcommand

CONNECT is required to access any database that has an installed Microsoft ODBC driver.

- You cannot specify the connection string directly in the syntax window, but you can paste it with the rest of the command from the Results dialog box, which is the last of the series of dialog boxes opened with the Database Wizard.

UNENCRYPTED Subcommands

UNENCRYPTED allows unencrypted passwords to be used in the CONNECT subcommand. By default, passwords are assumed to be encrypted.

SQL Subcommand

SQL specifies any SQL select statement accepted by the database that you access. With ODBC, you can now select columns from more than one related table in an ODBC data source using either the inner join or the outer join.

Data Conversion

GET CAPTURE converts variable names, labels, missing values, and data types, wherever necessary, to a format that conforms to IBM SPSS Statistics conventions.

Variable Names and Labels

Database columns are read as variables.

- A column name is converted to a variable name if it conforms to variable naming conventions and is different from all other names created for the active dataset. If not, GET CAPTURE gives the column a name formed from the first few letters of the column and its column number. If this is not possible, the letters *COL* followed by the column number are used. For example, the seventh column specified in the select statement could be *COL7*.
- GET CAPTURE labels each variable with its full column name specified in the original database.
- You can display a table of variable names with their original database column names using the DISPLAY LABELS command.

Missing Values

Null values in the database are transformed into the system-missing value in numeric variables or into blanks in string variables.

GET DATA

```
GET DATA
  /TYPE = {ODBC }
          {OLEDB}
          {XLS  }
          {XLSX }
          {XLSM }
          {TXT  }


  /FILE = 'filename'


Subcommands for TYPE = ODBC and OLEDB
  /CONNECT='connection string'
  [{/ENCRYPTED  }]
   {/UNENCRYPTED}
  /SQL 'select statement'
      ['select statement continued']


Subcommands for TYPE=ODBC, TYPE=OLEDB, XLS, XLSX, and XLSM
 [/ASSUMEDSTRWIDTH={255**}]
                   {n    }


Subcommands for TYPE = XLS, XLSX, and XLSM*
  [/SHEET = {INDEX**} {sheet number}]
            {NAME   } {'sheet name'}
  [/CELLRANGE = {RANGE }  {'start point:end point' }]
                {FULL**}
  [/READNAMES = {on**  }]
                {off   }
  [/DATATYPEMIN PERCENTAGE=value]
  [/HIDDEN IGNORE=(NO**}]
                  {YES }
  [/LEADINGSPACES IGNORE={NO**}]
                         {YES }
  [/TRAILINGSPACES IGNORE={NO**}]
                          {YES }

Subcommands for TYPE = TXT
  [/ENCODING = {'UTF8'   }]
               {'UTF16'  }
               {'UTF16BE'}
               {'UTF16LE'}
               {'LOCALE' }
  [/ARRANGEMENT = {FIXED      }]
                  {DELIMITED**}
  [/FIRSTCASE = {n}]
  [/DELCASE = {LINE**     }]1
              {VARIABLES n}
  [/FIXCASE = n]
  [/DELIMITERS = {"delimiters"}]
  [/QUALIFIER = "qualifier"]
  [/IMPORTCASE {ALL**}
               {FIRST n}
  [/DATATYPEMIN PERCENTAGE=value]
  [/LEADINGSPACES IGNORE={NO**}]
                         {YES }
  [/MULTIPLESPACES IGNORE={NO**}]
                          {YES }

VARIABLES subcommand for ARRANGEMENT = DELIMITED
  /VARIABLES = varname  {AUTO  }
                        {format}

VARIABLES subcommand for ARRANGEMENT = FIXED
  /VARIABLES varname {startcol - endcol} {AUTO | format}
     {/rec#} varname {startcol - endcol} {AUTO | format}

  [/MAP]
```

*For Excel 4.0 or earlier files, use GET TRANSLATE.

**Default if the subcommand is omitted.

Release history

Release 13.0

- ASSUMEDSTRWIDTH subcommand introduced for TYPE=ODBC.

Release 14.0

- ASSUMEDSTRWIDTH subcommand extended to TYPE=XLS.
- TYPE=OLEDB introduced.

Release 15.0

- ASSUMEDSTRWIDTH subcommand extended to TYPE=OLEDB.

Release 16.0

- TYPE=XLSX and TYPE=XLSM introduced.

Release 17.0

- ENCRYPTED subcommand introduced.

Release 21.0

- ENCODING subcommand introduced.

Release 23.0

- UTF16, UTF16BE, and UTF16LE keywords added to ENCODING subcommand.

Release 24.0

- AUTO keyword introduced.
- DATATYPEMIN subcommand introduced.
- HIDDEN subcommand introduced.
- LEADINGSPACES subcommand introduced.
- MAP subcommand introduced.
- MULTIPLESPACES subcommand introduced.
- TRAILINGSPACES subcommand introduced.

Example

```
GET DATA
   /TYPE=XLS
   /FILE='/PlanningDocs/files10.xls'
   /SHEET=name 'First Quarter'
   /CELLRANGE=full
   /READNAMES=on.
```

Overview

GET DATA reads data from ODBC and OLE DB data sources (databases), Excel files (release 5 or later), and text data files. It contains functionality and syntax similar to GET CAPTURE, GET TRANSLATE, and DATA LIST.

- GET DATA /TYPE=ODBC is almost identical to GET CAPTURE ODBC in both syntax and functionality.
- GET DATA /TYPE=XLS reads Excel 95 through Excel 2003 files; GET DATA /TYPE=XLSX and GET DATA /TYPE=XLSM read Excel 2007 or later files. GET TRANSLATE reads Excel 4 or earlier, Lotus, and dBASE files.
- GET DATA /TYPE=TXT is similar to DATA LIST but does not create a temporary copy of the data file, significantly reducing temporary file space requirements for large data files.

Limitations

TYPE Subcommand

The TYPE subcommand is required and must be the first subcommand specified.

ODBC. *Data sources accessed with ODBC drivers.*

XLS. *Excel 95 through Excel 2003 files.* For earlier versions of Excel files, Lotus 1-2-3 files, and dBASE files, see the GET TRANSLATE command.

XLSX and **XLSM**. *Excel 2007 files.* Macros in XLSM files are ignored. XLSB (binary) format files are not supported.

TXT. *Simple (ASCII) text data files.*

FILE subcommand

The FILE subcommand is required for TYPE=XLS, TYPE=XLSX, TYPE=XLSM, and TYPE=TXT and must immediately follow the TYPE subcommand. It specifies the file to read. File specifications should be enclosed in quotes.

Subcommands for TYPE=ODBC and TYPE=OLEDB

The CONNECT and SQL subcommands are both required, and SQL must be the last subcommand.

Example

```
GET DATA /TYPE=ODBC
  /CONNECT=
 'DSN=MS Access Database;DBQ=/examples/data/dm_demo.mdb;'+
 'DriverId=25;FIL=MS Access;MaxBufferSize=2048;PageTimeout=5;'
 /SQL = 'SELECT * FROM CombinedTable'.
```

Note: If you are running the Windows 64-bit version of IBM SPSS Statistics, you cannot read Excel, Access, or dBASE database sources with GET DATA /TYPE=ODBC. The 32-bit ODBC drivers for these products are not compatible. You can, however, read Excel files with GET DATA /TYPE=XLS.

CONNECT subcommand

The CONNECT subcommand identifies the database source. The recommended method for generating a valid CONNECT specification is to initially use the Database Wizard and paste the resulting syntax to a syntax window in the last step of the wizard. See the topic for more information.

- The entire connect string must be enclosed in quotation marks.
- For long connect strings, you can use multiple quoted strings on separate lines, using a plus sign (+) to combine the quoted strings.

Connection Pooling

If you access the same database source multiple times in the same job, you can improve performance with connection pooling. At the end of the quoted CONNECT string, add Pooling=true.

ENCRYPTED and UNENCRYPTED subcommands

UNENCRYPTED allows unencrypted passwords to be used in the CONNECT subcommand. The default is ENCRYPTED.

SQL subcommand

SQL specifies any SQL select statement accepted by the database that you access.

- You can select columns from more than one related table in a data source using either the inner join or the outer join.
- Each line of SQL must be enclosed in quotation marks and cannot exceed 255 characters.
- When the command is processed, all of the lines of the SQL statement are merged together in a very literal fashion; so each line should either begin or end with a blank space where spaces should occur between specifications.
- For TYPE=OLEDB (available only on Windows operating systems), table joins are not supported; you can specify fields only from a single table.

Example

```
GET DATA /TYPE=ODBC /CONNECT=
  'DSN=Microsoft Access;DBQ=/data/demo.mdb;DriverId=25;'+
```

```
  'FIL=MS Access;MaxBufferSize=2048;PageTimeout=5;'
 /SQL =
  'SELECT  SurveyResponses.ID,  SurveyResponses.Internet,'
  ' [Value Labels].[Internet Label]'
  ' FROM SurveyResponses LEFT OUTER JOIN [Value Labels]'
  ' ON SurveyResponses.Internet'
  ' = [Value Labels].[Internet Value]'.
```

If the SQL contains WHERE clauses with expressions for case selection, dates and times in expressions need to be specified in a special manner (including the curly braces shown in the examples):

- Date literals should be specified using the general form `{d 'yyyy-mm-dd'}`.
- Time literals should be specified using the general form `{t 'hh:mm:ss'}`.
- Date/time literals (timestamps) should be specified using the general form `{ts 'yyyy-mm-dd hh:mm:ss'}`.
- The entire date and/or time value must be enclosed in single quotes. Years must be expressed in four-digit form, and dates and times must contain two digits for each portion of the value. For example January 1, 2005, 1:05 AM would be expressed as:

  `{ts '2005-01-01 01:05:00'}`

For functions used in expressions, a list of standard functions is available at http://msdn.microsoft.com/en-us/library/ms711813(VS.85).aspx.

ASSUMEDSTRWIDTH subcommand

For TYPE=ODBC, TYPE=OLEDB, and TYPE=XLS, this controls the width of variable-width string values. By default, the width is 255 bytes, and only the first 255 bytes will be read. The width can be up to 32,767 bytes. Although you probably don't want to truncate string values, you also don't want to specify an unnecessarily large value, since this will be used as the display width for those string values.

DATATYPEMIN subcommand

For TYPE=XLS, XLSX, XLSM, and TXT, the DATATYPEMIN subcommand specifies the percentage of non-blank values that determine the data format. The data type for each variable is determined by the percentage of values that conform to the same format.

The format of the subcommand is `DATATYPEMIN PERCENTAGE=value`.

- The value must be greater than 50 and less than or equal to 100.
- The denominator used to determine the percentage is the number of non-blank values for each variable.
- If no consistent format is used by the specified percentage of values, the variable is assigned the string data type.
- For variables that are assigned a numeric format, values that do not conform to that format are assigned the system-missing value. Numeric format includes all formats that are not string formats, including dates and times.
- For TYPE=TXT, this subcommand affects only variables with the AUTO format specification. The default value is 95.
- For TYPE=XLS, XLSX, and XLSM, if DATATYPEMIN is not specified, the data format is determined based on the same criteria that are used in releases before 24.0.

LEADINGSPACES subcommand

For TYPE=XLS, XLSX, XLSM, and TXT, LEADINGSPACES controls the treatment of blank spaces at the start of string values.

- For TYPE=TXT, this subcommand applies only to ARRANGEMENT=DELIMITED.

- For TYPE=TXT, if a blank space is specified as a delimiter on the DELIMITERS subcommand, this subcommand only affects string values enclosed in specified QUALIFIER values.

IGNORE=NO
Blank spaces at the start of string values are included in the value. This is the default setting.

IGNORE=YES
Blank spaces at the start of string values are removed.

Subcommands for TYPE=XLS, XLSX, and XLSM

For Excel 95 or later files, you can specify a spreadsheet within the workbook, a range of cells to read, and the contents of the first row of the spreadsheet (variable names or data). For files from earlier versions of Excel, use .

Example

```
GET DATA /TYPE=XLS
   /FILE='/data/sales.xls'
   /SHEET=name 'June Sales'
   /CELLRANGE=range 'A1:C3'
   /READNAMES=on.
```

SHEET subcommand

The SHEET subcommand indicates the worksheet in the Excel file that will be read. Only one sheet can be specified. If no sheet is specified, the first sheet will be read.

INDEX n. *Read the specified sheet number.* The number represents the sequential order of the sheet within the workbook.

NAME 'name'. *Read the specified sheet name*. If the name contains spaces, it must be enclosed in quotes.

CELLRANGE subcommand

The CELLRANGE subcommand specifies a range of cells to read within the specified worksheet. By default, the entire worksheet is read.

FULL. *Read the entire worksheet*. This is the default.

RANGE 'start:end'. *Read the specified range of cells.* Specify the beginning column letter and row number, a colon, and the ending column letter and row number, as in A1:K14. The cell range must be enclosed in quotes.

READNAMES subcommand

ON. *Read the first row of the sheet or specified range as variable names.* This is the default. Values that contain invalid characters or do not meet other criteria for variable names are converted to valid variable names. See the topic “Variable Names” on page 42 for more information.

OFF. *Read the first row of the sheet or specified range as data.* Default variable names are assigned, and all rows are read as data.

HIDDEN subcommand

For TYPE=XLSX and TYPE=XLSM, the HIDDEN subcommand controls the treatment of rows and columns that are hidden in the Excel file.

IGNORE=NO
Hidden rows and columns are included. This setting is the default.

IGNORE=YES
Hidden rows and columns are excluded.

This subcommand is not valid for TYPE=XLS.

TRAILINGSPACES subcommand

The TRAILINGSPACES subcommand controls the treatment of trailing spaces for string values. This setting affects the determination of the width of string variables.

IGNORE=NO
Trailing spaces in string values are included in the determination of the width of string variables. This setting is the default.

IGNORE=YES
Trailing spaces are ignored in the determination of the width of string variables.

Subcommands for TYPE=TXT

The VARIABLES subcommand is required and must be the last GET DATA subcommand.

Example

```
GET DATA  /TYPE = TXT
  /ENCODING='Locale'
  /FILE = '/data/textdata.dat'
  /DELCASE = LINE
  /DELIMITERS = "\t ,"
  /ARRANGEMENT = DELIMITED
  /FIRSTCASE = 2
  /VARIABLES = id F3.0 gender A1 bdate DATE10 educ F2.0
   jobcat F1.0 salary DOLLAR8 salbegin DOLLAR8
   jobtime F4.2 prevexp F4.2 minority F3.0.
```

ENCODING subcommand

You can use the optional ENCODING subcommand to specify the character encoding for the file. The subcommand name is followed by an equals sign (=) and one of the following values enclosed in quotes:

UTF8
The file is read in UTF-8 Unicode encoding. This is the default in Unicode mode (see SET command, Unicode subcommand) .

UTF16
The file is read in UTF-16 encoding. The endianness of the encoding is determined by the endianness of the operating system.

UTF16BE
The file is read as UTF-16, big endian.

UTF16LE
The file is read as UTF-16, little endian.

LOCALE
The file is read in the current locale code page encoding. This is the default in code page mode (see SET command, LOCALE subcommand).

The ENCODING subcommand is ignored if the file contains a UTF-8 or UTF-16 byte order mark, and the file is read according to the byte order mark. If the byte order mark indicates any other Unicode encoding, the file is not read and an error message is issued.

ARRANGEMENT subcommand

The ARRANGEMENT subcommand specifies the data format.

DELIMITED. *Spaces, commas, tabs, or other characters are used to separate variables.* The variables are recorded in the same order for each case but not necessarily in the same column locations. This is the default.

FIXED. *Each variable is recorded in the same column location for every case.*

FIRSTCASE subcommand

FIRSTCASE specifies the first line (row) to read for the first case of data. This allows you to bypass information in the first *n* lines of the file that either don't contain data or contain data that you don't want to read. This subcommand applies to both fixed and delimited file formats.

The only specification for this subcommand is an integer greater than zero that indicates the number of lines to skip. The default is 1.

DELCASE subcommand

The DELCASE subcommand applies to delimited data (ARRANGEMENT=DELIMITED) only.

LINE. *Each case is contained on a single line (row).* This is the default.

VARIABLES n. *Each case contains n variables.* Multiple cases can be contained on the same line, and data for one case can span more than one line. A case is defined by the number of variables.

FIXCASE subcommand

The FIXCASE subcommand applies to fixed data (ARRANGEMENT=FIXED) only. It specifies the number of lines (records) to read for each case.

The only specification for this subcommand is an integer greater than zero that indicates the number of lines (records) per case. The default is 1.

IMPORTCASE subcommand

The IMPORTCASE subcommand allows you to specify the number of cases to read.

ALL. *Read all cases in the file.* This is the default.

FIRST n. *Read the first n cases.* The value must be a positive integer.

DELIMITERS subcommand

The DELIMITERS subcommand applies to delimited data (ARRANGEMENT=DELIMITED) only. It specifies the characters to read as delimiters between data values.

- Each delimiter can be only a single character, except for the specification of a tab or a backslash as a delimiter (see below).
- The list of delimiters must be enclosed in quotes.
- There should be no spaces or other delimiters between delimiter specifications, except for a space that indicates a space as a delimiter.
- To specify a tab as a delimiter use "\t". This must be the first delimiter specified.
- To specify a backslash as a delimiter, use two backslashes ("\\"). This must be the first delimiter specified unless you also specify a tab as a delimiter, in which case the backslash specification should come second—immediately after the tab specification.

Missing data with delimited data. Multiple delimiters without any intervening data indicate missing data. This includes multiple consecutive spaces.

Example

```
DELIMITERS "\t\\ ,;"
```

In this example, tabs, backslashes, spaces, commas, and semicolons will be read as delimiters between data values.

MULTIPLESPACES subcommand

MULTIPLESPACES controls the treatment of multiple, consecutive blank spaces.

- This subcommand applies only to ARRANGEMENT=DELIMITED.
- This subcommand has no effect unless a blank space is specified as a delimiter on the DELIMITERS subcommand.

IGNORE=NO
Multiple consecutive blank spaces are treated as separate delimiters. Each blank space is treated as a delimiter between values. For numeric variables, the values are set to system-missing. For string variables, the value is empty. This setting is the default.

IGNORE=YES
Multiple consecutive blank spaces are treated as a single delimiter.

QUALIFIER subcommand

The QUALIFIERS subcommand applies to delimited data (ARRANGEMENT=DELIMITED) only. It specifies the character used to enclose values that contain delimiter characters. For example, if a comma is the delimiter, values that contain commas will be read incorrectly unless there is a text qualifier enclosing the value, preventing the commas in the value from being interpreted as delimiters between values. CSV-format data files exported from Excel use a double quote (") as a text qualifier.

- The text qualifier appears at both the beginning and end of the value, enclosing the entire value.
- The qualifier value must be enclosed in single or double quotes. If the qualifier is a single quote, the value should be enclosed in double quotes. If the qualifier value is a double quote, the value should be enclosed in single quotes.

Example

```
/QUALIFIER = '"'
```

VARIABLES subcommand for ARRANGEMENT = DELIMITED

For delimited files, the VARIABLES subcommand specifies the variable names and variable formats.

- Variable names must conform to variable naming rules. See the topic “Variable Names” on page 42 for more information.
- Each variable name must be followed by a format specification. See the topic “Variable Format Specifications for TYPE = TXT ” on page 793 for more information.

VARIABLES subcommand for ARRANGEMENT = FIXED

For fixed-format files, the VARIABLES subcommand specifies variable names, start and end column locations, and variable formats.

- Variable names must conform to variable naming rules. See the topic “Variable Names” on page 42 for more information.
- Each variable name must be followed by column specifications. Start and end columns must be separated by a dash, as in 0-10.
- Column specifications must include both the start and end column positions, even if the width is only one column, as in 32-32.
- Each column specification must be followed by a format specification.
- Column numbering starts with 0, not 1 (in contrast to DATA LIST).

Multiple records. If each case spans more than one record (as specified with the FIXCASE subcommand), delimit variable specifications for each record with a slash (/) followed by the record number, as in:

```
VARIABLES = /1 var1 0-10 F var2 11-20 DATE
/2 var3 0-5 A var4 6-10 F
/3 var5 0-20 A var6 21-30 DOLLAR
```

Variable Format Specifications for TYPE = TXT

For both fixed and delimited files, available formats include (but are not limited to):

AUTO. Automatic. The format is automatically determined based on an evaluation of all the data values. The DATATYPEMIN subcommand specifies the percentage of values that must conform to the same data type. If no consistent format is used by the specified percentage of values, the variable is assigned the string data type.

Fn.d. Numeric. Specification of the total number of characters (n) and decimals (d) is optional.

An. String (alphanumeric). Specification of the maximum string length (n) is optional.

DATEn. n Dates of the general format dd-mmm-yyyy. Specification of the maximum length (n) is optional but must be eight or greater if specified.

ADATEn. Dates of the general format mm/dd/yyyy. Specification of the maximum length (n) is optional but must be eight or greater if specified.

DOLLARn.d. Currency with or without a leading dollar sign ($). Input values can include a leading dollar sign, but it is not required. Specification of the total number of characters (n) and decimals (d) is optional.

X. Skip this variable. The specified variable and any values for that variable are not included. Although the variable is not included, a valid variable name is required.

For a complete list of variable formats, see “Variable Types and Formats” on page 46.

Example

Note: For default numeric (F) format and scientific notation (E) format, the decimal indicator of the input data must match the IBM SPSS Statistics locale decimal indicator (period or comma). Use SHOW DECIMAL to display the current decimal indicator and SET DECIMAL to set the decimal indicator. (Comma and Dollar formats recognize only the period as the decimal indicator, and Dot format recognizes only the comma as the decimal indicator.)

MAP subcommand

The MAP subcommand displays a summary of how the data were read. The summary includes:

- Variable names
- Formats
- For numeric variables, a notification if any values are set to system-missing.
- For string variables, a notification if any values are truncated.

GET SAS

```
GET SAS DATA='file' [DSET(dataset)]

 [/ENCODING={"Locale"**}]
            {"System"  }
            {"UTF-8"   }
            {"value"   }

 [/FORMATS=file]
```

**Default if subcommand omitted.

Release History

Release 19

- ENCODING subcommand introduced.

Example

```
GET SAS DATA='/data/elect.sd7'.
```

Overview

GET SAS builds an active dataset from a SAS dataset or a SAS transport file. A SAS transport file is a sequential file written in SAS transport format and can be created by the SAS export engine available in SAS Release 6.06 or higher or by the EXPORT option on the COPY or XCOPY procedure in earlier versions. GET SAS reads SAS version 6-9 files.

Options

Retrieving User-Defined Value Labels. For native SAS datasets, you can specify a file on the FORMATS subcommand to retrieve user-defined value labels associated with the data being read. This file must be created by the SAS PROC FORMAT statement and can be used only for native SAS datasets. For SAS transport files, the FORMATS subcommand is ignored.

Specifying the Dataset. You can name a dataset contained in a specified SAS file, using DSET on the DATA subcommand. GET SAS reads the specified dataset from the SAS file.

Basic Specification

The basic specification is the DATA subcommand followed by the name of the SAS file to read. By default, the first SAS dataset is copied into the active dataset and any necessary data conversions are made.

Syntax Rules

- The subcommand DATA and the SAS filename are required and must be specified first.
- The subcommand FORMATS is optional. This subcommand is ignored for SAS transport files.
- The subcommand ENCODING is optional.
- GET SAS does not allow KEEP, DROP, RENAME, and MAP subcommands. To use a subset of the variables, rename them, or display the file content, you can specify the appropriate commands after the active dataset is created.

Operations

- GET SAS reads data from the specified or default dataset contained in the SAS file named on the DATA subcommand.
- Value labels retrieved from a SAS user-defined format are used for variables associated with that format, becoming part of the IBM SPSS Statistics dictionary.
- All variables from the SAS dataset are included in the active dataset, and they are in the same order as in the SAS dataset.

DATA Subcommand

DATA specifies the file that contains the SAS dataset to be read.

- DATA is required and must be the first specification on GET SAS.
- The file specification varies from operating system to operating system. File specifications should be enclosed in quotes.
- The optional DSET keyword on DATA determines which dataset within the specified SAS file is to be read. The default is the first dataset.

DSET (dataset) . *Dataset to be read.* Specify the name of the dataset in parentheses. If the specified dataset does not exist in the SAS file, GET SAS displays a message informing you that the dataset was not found.

Example

```
GET SAS DATA='/data/elect.sd7' DSET(Y1948).
```

- The SAS file *elect.sd7* is opened and the dataset named *Y1948* is used to build the active dataset for the session.

ENCODING Subcommand

The ENCODING subcommand specifies the character encoding of the SAS data file.

- The encoding must be correctly identified or the file cannot be read.
- If there is a formats files, the encoding of the formats file must be the same as the encoding of the data file. See the topic “FORMATS Subcommand ” on page 797 for more information.
- The subcommand is followed by an optional equals sign and a quoted encoding value.
- The quoted value can be any of the values in the Encoding column in the Character Encoding table or "UTF-8" for UTF-8 Unicode encoding.
- The default encoding is "Locale", which is the encoding of the current IBM SPSS Statistics locale. See the topic “LOCALE Subcommand ” on page 1796 for more information.

Example

```
GET SAS DATA='/data/elect.sas7bdat'
  /ENCODING='Windows-1252'.
```

Table 54. Character Encoding

Character Set	**Encoding**
IBM SPSS Statistics Locale	Locale
Operating System Locale	System
Western	ISO-8859-1
Western	ISO-8859-15
Western	IBM850
Western	Windows-1252
Celtic	ISO-8859-14
Greek	ISO-8859-7
Greek	Windows-1253
Nordic	ISO-8859-10

Table 54. Character Encoding (continued)	
Character Set	**Encoding**
Baltic	Windows-1257
Central European	IBM852
Central European	ISO-8859-2
Cyrillic	IBM855
Cyrillic	ISO-8859-5
Cyrillic	Windows-1251
Cyrillic/Russian	CP-866
Chinese Simplified	GBK
Chinese Simplified	ISO-2022-CN
Chinese Traditional	Big5
Chinese Traditional	EUC-TW
Japanese	EUC-JP
Japanese	ISO-2022-JP
Japanese	Shift-JIS
Korean	EUC-KR
Thai	Windows-874
Turkish	IBM857
Turkish	ISO-8859-9
Arabic	Windows-1256
Arabic	IBM864
Hebrew	ISO-8859-8
Hebrew	Windows-1255
Hebrew	IBM862

FORMATS Subcommand

FORMATS specifies the file containing user-defined value labels to be applied to the retrieved data.

- File specifications should be enclosed in quotation marks.
- If FORMATS is omitted, no value labels are available.
- Value labels are applied only to numeric integer values. They are not applied to non-integer numeric values or string variables.
- The file specified on the FORMATS subcommand must be created with the SAS PROC FORMAT statement.
- For SAS transport files, the FORMATS subcommand is ignored.

Example

```
GET SAS /DATA='/data/elect.sas7bdat' DSET(Y1948)
 /FORMATS='data/formats.sas7bcat'.
```

- Value labels read from the SAS catalog file *formats.sas7bcat* are converted to conform to IBM SPSS Statistics conventions.

Creating a Formats File with PROC FORMAT

The following is an example of how to create a SAS format catalog file:

```
libname mylib 'c:\mydir\' ;

proc format library = mylib ;
value YesNo
  0='No'
  1='Yes' ;
value HighLow
   1 = 'Low'
   2 = 'Medium'
   3 = 'High' ;

options  fmtsearch=(mylib);

proc datasets library = mylib ;
modify mydata;
   format    var1 var2 var3 YesNo.;
   format    var4 var5 var6 HighLow.;

quit;
```

- `libname` defines a "library," which is a directory path.
- `proc format` defines two formats: *YesNo* and *HighLow*. Each format defines a set of value labels associated with data values.
- `proc datasets` identifies the data file--*mydata*--and the variables to which each of the defined formats should be applied. So the *YesNo* format is applied to variables *var1*, *var2*, and *var3*, and the *HighLow* format is applied to the variables *var4*, *var5*, and *var6*.
- This creates the SAS catalog file *c:\mydir\formats.sas7bcat*.

SAS Data Conversion

Although SAS and IBM SPSS Statistics data files have similar attributes, they are not identical. The following conversions are made to force SAS datasets to comply with IBM SPSS Statistics conventions.

Variable Names

SAS variable names that do not conform to IBM SPSS Statistics variable name rules are converted to valid variable names.

Variable Labels

SAS variable labels specified on the LABEL statement in the DATA step are used as variable labels.

Value Labels

SAS value formats that assign value labels are read from the dataset specified on the FORMATS subcommand. The SAS value labels are then converted to IBM SPSS Statistics value labels in the following manner:

- Labels assigned to single values are retained.
- Labels assigned to a range of values are ignored.
- Labels assigned to the SAS keywords LOW, HIGH, and OTHER are ignored.
- Labels assigned to string variables and non-integer numeric values are ignored.

Missing Values

Since SAS has no user-defined missing values, all SAS missing codes are converted to IBM SPSS Statistics system-missing values.

Variable Types

- Both SAS and IBM SPSS Statistics allow two basic types of variables: numeric and character string. During conversion, SAS numeric variables become IBM SPSS Statistics numeric variables, and SAS string variables become IBM SPSS Statistics string variables of the same length.
- Date, time, and date/time SAS variables are converted to equivalent IBM SPSS Statistics date, time, and date/time variables. All other numeric formats are converted to the default numeric format.

GET STATA

```
GET STATA FILE='file'

  [/ENCODING={"Locale"**}]
             {"System"  }
             {"value"   }
```

**Default if subcommand omitted.

Release History

Release 14.0

- Command introduced.

Release 19

- ENCODING subcommand introduced.

Example

```
GET STATA FILE='/data/empl.dta'.
```

Overview

GET STATA reads Stata-format data files created by Stata versions 4–14.

Note: Stata 14 supports Unicode data.

Basic Specification

- The basic specification is the FILE keyword, which specifies the Stata data file to be read.

Operations

- **Variable names.** Stata variable names are converted to IBM SPSS Statistics variable names in case-sensitive form. Stata variable names that are identical except for case are converted to valid variable names by appending an underscore and a sequential letter (*_A*, *_B*, *_C*, ..., *_Z*, *_AA*, *_AB*, ..., and so forth).
- **Variable labels.** Stata variable labels are converted to IBM SPSS Statistics variable labels.
- **Value labels.** Stata value labels are converted to IBM SPSS Statistics value labels, except for Stata value labels assigned to "extended" missing values. Value labels longer than 120 bytes are truncated.
- **String variables**. Stata strl variables are converted to string variables. Values longer than 32K bytes are truncated. Stata strl values that contain blobs (binary large objects) are converted to blank strings.
- **Missing values.** Stata "extended" missing values are converted to system-missing values.
- **Date conversion.** Stata date format values are converted to IBM SPSS Statistics DATE format (d-m-y) values. Stata "time-series" date format values (weeks, months, quarters, and so on) are converted to simple numeric (F) format, preserving the original, internal integer value, which is the number of weeks, months, quarters, and so on, since the start of 1960.

FILE Keyword

FILE specifies the Stata data file to be read. FILE is the only specification; it is required and can be specified only once. The keyword name is followed by an equals sign and a quoted file specification (or quoted file handle) that specifies the Stata data file to read.

ENCODING Subcommand

The ENCODING subcommand specifies the character encoding of the Stata data file.

- The encoding must be correctly identified or the file cannot be read.
- The subcommand is followed by an optional equals sign and a quoted encoding value.
- The quoted value can be any of the values in the Encoding column in the Character Encoding table.
- The default encoding is "Locale", which is the encoding of the current IBM SPSS Statistics locale. See the topic “LOCALE Subcommand ” on page 1796 for more information.

Example

```
GET STATA FILE='/data/empl.dta'
  /ENCODING='Windows-1252'.
```

Table 55. Character Encoding

Character Set	Encoding
IBM SPSS Statistics Locale	Locale
Operating System Locale	System
Western	ISO-8859-1
Western	ISO-8859-15
Western	IBM850
Western	Windows-1252
Celtic	ISO-8859-14
Greek	ISO-8859-7
Greek	Windows-1253
Nordic	ISO-8859-10
Baltic	Windows-1257
Central European	IBM852
Central European	ISO-8859-2
Cyrillic	IBM855
Cyrillic	ISO-8859-5
Cyrillic	Windows-1251
Cyrillic/Russian	CP-866
Chinese Simplified	GBK
Chinese Simplified	ISO-2022-CN
Chinese Traditional	Big5
Chinese Traditional	EUC-TW
Japanese	EUC-JP
Japanese	ISO-2022-JP
Japanese	Shift-JIS
Korean	EUC-KR

Table 55. Character Encoding (continued)	
Character Set	**Encoding**
Thai	Windows-874
Turkish	IBM857
Turkish	ISO-8859-9
Arabic	Windows-1256
Arabic	IBM864
Hebrew	ISO-8859-8
Hebrew	Windows-1255
Hebrew	IBM862

GET TRANSLATE

```
GET TRANSLATE FILE=file

 [/TYPE={WK }]
        {WK1}
        {WKS}
        {SYM}
        {SLK}
        {XLS}
        {DBF}
        {TAB}

  [/FIELDNAMES]*

  [/RANGE={range name }]*
          {start..stop}
          {start:stop }

  [/KEEP={ALL**  }] [/DROP=varlist]
         {varlist}

[/MAP]
```

*Available only for spreadsheet and tab-delimited ASCII files.

**Default if the subcommand is omitted.

Table 56. File types

Keyword	Type of file
WK	Any Lotus 1-2-3 or Symphony file
WK1	1-2-3 Release 2.0
WKS	1-2-3 Release 1A
WR1	Symphony Release 2.0
WRK	Symphony Release 1.0
SLK	Microsoft Excel and Multiplan in SYLK (symbolic link) format
XLS	Microsoft Excel (for Excel 5 or later, use GET DATA)
DBF	All dBASE files
TAB	Tab-delimited ASCII file

Example

```
GET TRANSLATE FILE='PROJECT.WKS'
 /FIELDNAMES
 /RANGE=D3..J279.
```

Overview

GET TRANSLATE reads dBASE files, Excel files prior to release 5, Lotus 1-2-3, Symphony, and Multiplan files. To read more recent version of Excel or text data files, use GET DATA.

Options

Variable Subsets. You can use the DROP and KEEP subcommands to specify variables to omit or retain in the resulting active dataset.

Variable Names. You can rename variables as they are translated using the RENAME subcommand.

Variable Map. To confirm the names and order of the variables in the active dataset, use the MAP subcommand. MAP displays the variables in the active dataset and their corresponding names in the other application.

Spreadsheet Files. You can use the RANGE subcommand to translate a subset of cells from a spreadsheet file. You can use the FIELDNAMES subcommand to translate field names in the spreadsheet file to variable names.

Basic Specification

- The basic specification is FILE with a file specification enclosed in apostrophes.
- If the file's extension is not the default for the type of file you are reading, TYPE must also be specified.

Subcommand Order

Subcommands can be named in any order.

Limitations

The maximum number of variables that can be translated into the active dataset depends on the maximum number of variables that the other software application can handle:

Table 57. Maximum number of variables for each file type

Application	Maximum variables
1-2-3	256
Symphony	256
Multiplan	255
Excel	256
dBASE IV	255
dBASE III	128
dBASE II	32

Operations

GET TRANSLATE replaces an existing active dataset.

Spreadsheets

A spreadsheet file suitable for this program should be arranged so that each row represents a case and each column, a variable.

- By default, the new active dataset contains all rows and up to 256 columns from Lotus 1-2-3, Symphony, or Excel, or up to 255 columns from Multiplan.
- By default, GET TRANSLATE uses the column letters as variable names in the active dataset.
- The first row of a spreadsheet or specified range may contain field labels immediately followed by rows of data. These names can be transferred as IBM SPSS Statistics variable names. See the topic "FIELDNAMES Subcommand " on page 809 for more information.
- The current value of a formula is translated to the active dataset.
- Blank, ERR, and NA values in 1-2-3 and Symphony and error values such as #N/A in Excel are translated as system-missing values in the active dataset.
- Hidden columns and cells in 1-2-3 Release 2 and Symphony files are translated and copied into the active dataset.
- Column width and format type are transferred to the dictionary of the active dataset.
- The format type is assigned from values in the first data row. By default, the first data row is row 1. If RANGE is specified, the first data row is the first row in the range. If FIELDNAMES is specified, the first data row follows immediately after the single row containing field names.

- If a cell in the first data row is empty, the variable is assigned the global default format from the spreadsheet.

The formats from 1-2-3, Symphony, Excel, and Multiplan are translated as follows:

- If a string is encountered in a column with numeric format, it is converted to the system-missing value in the active dataset.
- If a numeric value is encountered in a column with string format, it is converted to a blank in the active dataset.
- Blank lines are translated as cases containing the system-missing value for numeric variables and blanks for string variables.
- 1-2-3 and Symphony date and time indicators (shown at the bottom of the screen) are not transferred from *WKS*, *WK1*, or *SYM* files.

Databases

Database tables are logically very similar to IBM SPSS Statistics data files.

- By default, all fields and records from dBASE II, dBASE III, or dBASE IV files are included in the active dataset.
- Field names are automatically translated into variable names. If the FIELDNAMES subcommand is used with database files, it is ignored.
- Field names are converted to valid IBM SPSS Statistics variable names.
- Colons used in dBASE II field names are translated to underscores.
- Records in dBASE II, dBASE III, or dBASE IV that have been marked for deletion but that have not actually been purged are included in the active dataset. To differentiate these cases, GET TRANSLATE creates a new string variable, *D_R*, which contains an asterisk for cases marked for deletion. Other cases contain a blank for *D_R*.
- Character, floating, and numeric fields are transferred directly to variables. Logical fields are converted into string variables. Memo fields are ignored.

dBASE formats are converted as follows:

Table 58. Conversion of dBASE formats

dBASE	IBM SPSS Statistics
Character	String
Logical	String
Date	Date
Numeric	Number
Floating	Number
Memo	Ignored

Tab-Delimited ASCII Files

Tab-delimited ASCII files are simple spreadsheets produced by a text editor, with the columns delimited by tabs and rows, by carriage returns. The first row is usually occupied by column headings.

- By default all columns of all rows are treated as data. Default variable names *VAR1*, *VAR2*, and so on are assigned to each column. The data type (numeric or string) for each variable is determined by the first data value in the column.
- If FIELDNAMES is specified, the program reads in the first row as variable names and determines the data type by the values read in from the second row.

- Any value that contains non-numeric characters is considered a string value. Dollar and date formats are not recognized and are treated as strings. When string values are encountered for a numeric variable, they are converted to the system-missing value.
- For numeric variables, the assigned format is F8.2 or the format of the first data value in the column, whichever is wider. Values that exceed the defined width are rounded for display, but the entire value is stored internally.
- For string variables, the assigned format is A8 or the format of the first data value in the column, whichever is wider. Values that exceed the defined width are truncated.
- ASCII data files delimited by space (instead of tabs) or in fixed format should be read by DATA LIST.

FILE Subcommand

FILE names the file to read. The only specification is the name of the file.

- File specifications should be enclosed in quotation marks or apostrophes.

Example

```
GET TRANSLATE FILE='PROJECT.WKS'.
```

- GET TRANSLATE creates an active dataset from the 1-2-3 Release 1.0 spreadsheet with the name *PROJECT.WKS*.
- The active dataset contains all rows and columns and uses the column letters as variable names.
- The format for each variable is determined by the format of the value in the first row of each column.

TYPE Subcommand

TYPE indicates the format of the file.

- TYPE can be omitted if the file extension named on FILE is the default for the type of file that you are reading.
- The TYPE subcommand takes precedence over the file extension.
- You can create a Lotus format file in Multiplan and translate it to an active dataset by specifying WKS on TYPE.

WK . *Any Lotus 1-2-3 or Symphony file.*

WK1. *1-2-3 Release 2.0.*

WKS. *1-2-3 Release 1A.*

SYM. *Symphony Release 2.0 or Symphony Release 1.0.*

SLK. *Microsoft Excel and Multiplan saved in SYLK (symbolic link) format.*

XLS. *Microsoft Excel.* For Excel 5 or later, use GET DATA.

DBF. *All dBASE files.*

TAB. *Tab-delimited ASCII data file.*

Example

```
GET TRANSLATE FILE='PROJECT.OCT' /TYPE=SLK.
```

- GET TRANSLATE creates an active dataset from the Multiplan file *PROJECT.OCT*.

FIELDNAMES Subcommand

FIELDNAMES translates spreadsheet field names into variable names.

- FIELDNAMES can be used with spreadsheet and tab-delimited ASCII files only. FIELDNAMES is ignored when used with database files.
- Each cell in the first row of the spreadsheet file (or the specified range) must contain a field name. If a column does not contain a name, the column is dropped.
- Field names are converted to valid IBM SPSS Statistics variable names.
- If two or more columns in the spreadsheet have the same field name, digits are appended to all field names after the first, making them unique.
- Illegal characters in field names are changed to underscores in this program.
- If the spreadsheet file uses reserved words (ALL, AND, BY, EQ, GE, GT, LE, LT, NE, NOT, OR, TO, or WITH) as field names, GET TRANSLATE appends a dollar sign ($) to the variable name. For example, columns named *GE*, *GT*, *EQ*, and *BY* will be renamed *GE$*, *GT$*, *EQ$*, and *BY$* in the active dataset.

Example

```
GET TRANSLATE FILE='MONTHLY.SYM' /FIELDNAMES.
```

- GET TRANSLATE creates a active dataset from a Symphony 1.0 spreadsheet. The first row in the spreadsheet contains field names that are used as variable names in the active dataset.

RANGE Subcommand

RANGE translates a specified set of cells from a spreadsheet file.

- RANGE cannot be used for translating database files.
- For 1-2-3 or Symphony, specify the beginning of the range with a column letter and row number followed by two periods and the end of the range with a column letter and row number, as in A1..K14.
- For Multiplan spreadsheets, specify the beginning and ending cells of the range separated by a colon, as in R1C1:R14C11.
- For Excel files, specify the beginning column letter and row number, a colon, and the ending column letter and row number, as in A1:K14.
- You can also specify the range using range names supplied in Symphony, 1-2-3, or Multiplan.
- If you specify FIELDNAMES with RANGE, the first row of the range must contain field names.

Example

```
GET TRANSLATE FILE='PROJECT.WKS' /FIELDNAMES /RANGE=D3..J279.
```

- GET TRANSLATE creates an active dataset from the 1-2-3 Release 1A file *PROJECT.WKS*.
- The field names in the first row of the range (row 3) are used as variable names.
- Data from cells D4 through J279 are transferred to the active dataset.

DROP and KEEP Subcommands

DROP and KEEP are used to copy a subset of variables into the active dataset. DROP specifies the variables not to copy into the active dataset. KEEP specifies the variables to copy. Variables not specified on KEEP are dropped.

- DROP and KEEP cannot precede the FILE or TYPE subcommands.
- DROP and KEEP specifications use variable names. By default, this program uses the column letters from spreadsheets and the field names from databases as variable names.

- If FIELDNAMES is specified when translating from a spreadsheet, the DROP and KEEP subcommands must refer to the field names, not the default column letters.
- Variables can be specified in any order. Neither DROP nor KEEP affects the order of variables in the resulting file. Variables are kept in their original order.
- If a variable is referred to twice on the same subcommand, only the first mention of the variable is recognized.
- Multiple DROP and KEEP subcommands are allowed; the effect is cumulative. Specifying a variable named on a previous DROP or not named on a previous KEEP results in an error and the command is not executed.
- If you specify both RANGE and KEEP, the resulting file contains only variables that are both within the range and specified on KEEP.
- If you specify both RANGE and DROP, the resulting file contains only variables within the range and excludes those mentioned on DROP, even if they are within the range.

Example

```
GET TRANSLATE FILE='ADDRESS.DBF' /DROP=PHONENO, ENTRY.
```

- GET TRANSLATE creates an active dataset from the dBASE file *ADDRESS.DBF*, omitting the fields named *PHONENO* and *ENTRY*.

Example

```
GET TRANSLATE FILE='PROJECT.OCT' /TYPE=WK1 /FIELDNAMES
 /KEEP=NETINC, REP, QUANTITY, REGION, MONTH, DAY, YEAR.
```

- GET TRANSLATE creates a active dataset from the 1-2-3 Release 2.0 file called *PROJECT.OCT*.
- The subcommand FIELDNAMES indicates that the first row of the spreadsheet contains field names, which will be translated into variable names in the active dataset.
- The subcommand KEEP translates columns with the field names *NETINC*, *REP*, *QUANTITY*, *REGION*, *MONTH*, *DAY*, and *YEAR* to the active dataset.

MAP Subcommand

MAP displays a list of the variables in the active dataset and their corresponding names in the other application.

- The only specification is the keyword MAP. There are no additional specifications.
- Multiple MAP subcommands are allowed. Each MAP subcommand maps the results of subcommands that precede it; results of subcommands that follow it are not mapped.

Example

```
GET TRANSLATE FILE='ADDRESS.DBF' /DROP=PHONENO, ENTRY /MAP.
```

- MAP is specified to confirm that the variables *PHONENO* and *ENTRY* have been dropped.

GETCOGNOS

Note: Square brackets used in the GETCOGNOS syntax chart are required parts of the syntax and are not used to indicate optional elements. Equals signs (=) used in the syntax chart are required elements.

```
GETCOGNOS
  /MODE TYPE={DATA**}
            {REPORT}

  /CONNECTION SERVERURL='string'
   MODE={CREDENTIALS     }
        {ANONYMOUS       }
        {STOREDCREDENTIAL}
   NAMESPACE='string'
   USERNAME='string'
   PASSWORD='string' ENCRYPTED={YES**}
                               {NO   }
   CREDENTIAL='name'

  /LOCATION PACKAGE='string'

  /IMPORT
    {FIELDS =
     '[namespace].[query subject].[query item]' (newname),
     '[namespace].[query subject].[query item]' (newname),
     '[namespace].[query subject].[query item]' (newname)...}
    {REPORT = 'report name'}
    AGGREGATE={NO**}
              {YES }

  /FILTER VALUE='[namespace].[filter name]',
    '[namespace].[filter name]'...

  /PARAMETERS 'parameter'='value''parameter'='value'...
```

**Default if keyword omitted.

- The CONNECTION, LOCATION, and IMPORT subcommands are required.
- The MODE, FILTER and PARAMETERS subcommands are optional.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 21.0

- Command introduced.

Release 23.0

- CREDENTIAL keyword introduced on CONNECTION subcommand.
- Value STOREDCREDENTIAL added to MODE keyword on CONNECTION subcommand.

Example

```
GETCOGNOS
  /MODE TYPE=DATA
  /CONNECTION SERVERURL='http://myserver:9300/p2pd/servlet/dispatch'
   MODE=ANONYMOUS
  /LOCATION PACKAGE='/My Folders/Sales'
  /IMPORT FIELDS='[great_outdoor_sales].[my package].[ID]',
   '[great_outdoor_sales].[my package].[Geographic region]'  (Region),
   '[great_outdoor_sales].[my package].[sales]'.
```

Overview

GETCOGNOS reads IBM Cognos® Business Intelligence version 11 and higher data packages and list reports. You must have access to a IBM Cognos Business Intelligence server to use this feature.

MODE subcommand

The MODE subcommand is optional. It is followed by the TYPE keyword, an equals sign, and one of the following alternatives:

- **DATA.** IBM Cognos Business Intelligence data package. This is the default.
- **REPORT.** IBM Cognos Business Intelligence list report. The only type of report that can be read is a list report.

Example

```
/MODE TYPE=DATA
```

CONNECTION subcommand

The required CONNECTION subcommand specifies the IBM Cognos Business Intelligence server and any credential required to access data on the IBM Cognos Business Intelligence server.

SERVERURL. Specifies the URL for the server. The keyword is followed by an equals sign (=) and a URL enclosed in quotes.

MODE. Specifies if you are using explicitly specified credentials, stored credentials, or no credentials to access the IBM Cognos Business Intelligence server. The keyword is followed by an equals sign (=) and either the CREDENTIALS, STOREDCREDENTIAL, or ANONYMOUS keyword.

CREDENTIALS

For CREDENTIALS, you must also specify the following:

- **NAMESPACE.** The IBM Cognos Business Intelligence security authentication provider used to log on to the server. The authentication provider is used to define and maintain users, groups, and roles, and to control the authentication process. The keyword is followed by an equals sign (=) and a value enclosed in quotes.
- **USERNAME.** The IBM Cognos Business Intelligence user name with which to log on to the server. The keyword is followed by an equals sign (=) and a value enclosed in quotes.
- **PASSWORD.** The password associated with the specified user name. The keyword is followed by an equals sign (=) and a value enclosed in quotes. By default, the password is encrypted. If you don't want the password encrypted, specify ENCRYPT=NO.

STOREDCREDENTIAL

Specifies to use the login information from a stored credential. To use a stored credential, you must be connected to the IBM SPSS Collaboration and Deployment Services Repository that contains the credential. For STOREDCREDENTIAL, you must specify the CREDENTIAL keyword, followed by an equals sign (=) and the name of the credential in quotation marks.

Example

```
/CONNECTION
  SERVERURL='http://9.116.202.300:9300/p2pd/servlet/dispatch'
  MODE=ANONYMOUS
```

LOCATION subcommand

The required LOCATION subcommand specifies the IBM Cognos Business Intelligence package location and name. The subcommand name is followed by the keyword PACKAGE, an equals sign (=), and the package location and name enclosed in quotes.

Example

```
/LOCATION PACKAGE='/Public Folders/Samples/Models/GO Sales (query)'
```

IMPORT subcommand

The required IMPORT command specifies the data to import. The subcommand is followed by one of the following alternatives:

FIELDS. Specifies a list of fields to include.

- This is required if MODE=DATA.
- The keyword is followed by an equals sign (=) and a list of fields.
- Each field specification must be enclosed in quotes. The general form of a field specification is '[namespace].[query subject].[query item]'.
- Valid variable names are automatically generated based on the query item name. Optionally, you can specify a variable name in parentheses after the field specification. Names must be unique and must conform to variable naming rules. See the topic "Variable Names" on page 42 for more information.
- Optionally, you can also specify AGGREGATE=YES to import aggregated data instead of raw data. If no aggregation is defined in the IBM Cognos Business Intelligence package, this setting is ignored.

REPORT. Specifies the name of a list report to read.

- This is required if MODE=REPORT.
- The keyword is followed by and equals sign (=) and a report name enclosed in quotes.
- The only type of report that can be read is a list report.

Example

```
/IMPORT FIELDS=
 '[Sales (query)].[Product brand].[Product number]',
 '[Sales (query)].[Product brand].[Introduction date]',
 '[Sales (query)].[Product brand].[Product color code]' (Color),
 '[Sales target (query)].[Product brand].[Product size code]'(Size)
```

FILTER subcommand

The optional FILTER subcommand specifies one or more filters to apply.

- The FILTER subcommand is only valid with MODE=DATA. It is ignored with MODE=REPORT.
- The subcommand is followed by the keyword VALUE, an equals sign (=), and one or more filter specifications.
- Each filter specification must be enclosed in quotes. The general form is '[namespace].[filter name]'.
- The filters must be defined in the IBM Cognos Business Intelligence package.

Example

```
/FILTER VALUE='[go_data_warehouse].[2004]'.
```

PARAMETERS subcommand

The optional PARAMETERS subcommand can be used to specify parameter values for any defined parameters in a data package or report. The general form of the specification for each parameter is 'parameter name'='value'. Both the name and the value must be enclosed in quotes.

GETTM1

```
GETTM1

  /CONNECTION PMSYSURL='string'
   SERVER='string'
  [MODE={SETCREDENTIALS**}]
        {STOREDCREDENTIAL}
   NAMESPACE='string'
   USERNAME='string'
   PASSWORD='string' ENCRYPTED={YES**}
                               {NO   }
   CREDENTIAL='name'

  /VIEW  NAME='string'
         CUBE='string'

  [/RENAME  FIELDS='dimension or measure name' (varname) ... ]
```

**Default if keyword omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 22.0.0.1

- Command introduced.

Release 23.0

- MODE and CREDENTIAL keywords introduced on CONNECTION subcommand.

Example

```
GETTM1
  /CONNECTION PMSYSURL='http://mypmsystem:9510/pmhub/pm'
   SERVER='myserver'
   USERNAME='tm1user'
   PASSWORD='#o/B!a.],-./.-#('  ENCRYPTED=YES

  /VIEW NAME='myview' CUBE='mycube'.
```

Overview

GETTM1 reads the data from an IBM Cognos TM1® cube view. The multidimensional OLAP cube data is flattened when read into IBM SPSS Statistics. You must have access to a TM1 Performance Management system and an associated TM1 server to use this feature.

Important: To enable the exchange of data between SPSS Statistics and TM1, you must copy the following three processes from SPSS Statistics to the TM1 server: ExportToSPSS.pro, ImportFromSPSS.pro, and SPSSCreateNewMeasures.pro. To add these processes to the TM1 server, you must copy them to the data directory of the TM1 server and restart the TM1 server. These files are available from the common/scripts/TM1 directory under the SPSS Statistics installation directory.

Syntax Rules

- The CONNECTION and VIEW subcommands are required.

Limitations

- The TM1 view from which you import must include one or more elements from a measure dimension.
- The data to be imported from TM1 must be in UTF-8 format.

Operations

- Empty cells and cells with a value of zero in TM1 are converted to the system-missing value.
- Cells with string values that cannot be converted to a numeric value are converted to the system-missing value.
- When the data are imported, a separate SPSS Statistics variable is created for each regular dimension and for each element in the measure dimension.
- All of the data in the specified TM1 view are imported. It is therefore best to limit the view to the data that are required for the analysis. Any necessary filtering of the data is best done in TM1, for example with the TM1 Subset Editor.

Related information

"Overview " on page 1765

CONNECTION subcommand

The required CONNECTION subcommand specifies the IBM Cognos TM1 Performance Management system and associated TM1 server and any credentials that are required to access data on the TM1 server.

PMSYSURL
Specifies the URL for the Performance Management system where the TM1 server you want to connect to is installed. The Performance Management system is defined as a single URL for all TM1 servers. From this URL, all TM1 servers that are installed and running on your environment can be discovered and accessed. The keyword is followed by an equals sign (=) and a URL enclosed in quotation marks.

SERVER
Specifies the TM1 server. The keyword is followed by an equals sign (=) and the name of the server enclosed in quotation marks.

MODE=SETCREDENTIALS
Specifies that credentials are explicitly specified. This setting is the default. For SETCREDENTIALS, you must specify values for USERNAME and PASSWORD.

USERNAME
The TM1 user name with which to log on to the server. The keyword is followed by an equals sign (=) and a value that is enclosed in quotation marks.

PASSWORD
The password that is associated with the specified user name. The keyword is followed by an equals sign (=) and a value that is enclosed in quotation marks. By default, the password is assumed to be encrypted. If you do not want the password to be treated as encrypted, then specify ENCRYPTED=NO.

NAMESPACE
The IBM Cognos Business Intelligence security authentication provider that is used to log on to the server. This setting is only required when the server uses authentication mode 5 (IBM Cognos security). The keyword is followed by an equals sign (=) and a value that is enclosed in quotation marks. The keyword is ignored if the authentication mode is anything other than mode 5.

MODE=STOREDCREDENTIAL
Specifies to use the login information from a stored credential. To use a stored credential, you must be connected to the IBM SPSS Collaboration and Deployment Services Repository that contains the credential. For STOREDCREDENTIAL, you must specify the CREDENTIAL keyword, followed by an equals sign (=) and the name of the credential in quotation marks.

VIEW subcommand

The required VIEW subcommand specifies the IBM Cognos TM1 cube view from which the data are imported.

- The required CUBE keyword specifies the name of the cube that contains the view, and the required NAME keyword specifies the name of the view.
- Each keyword is followed by an equals sign (=) and the value of the keyword specified in quotation marks.

RENAME subcommand

By default, valid IBM SPSS Statistics variable names are automatically generated from the dimension names and measure names from the IBM Cognos TM1 cube view. Use the RENAME subcommand to define SPSS Statistics variable names for a set of TM1 dimension names or names of elements in the measure dimension.

- Specify each TM1 dimension name or measure element name in quotation marks, followed by the associated SPSS Statistics variable name in parentheses.
- Variable names must be unique and must conform to variable naming rules. See the topic for more information.

Example

```
/RENAME FIELDS='Business Unit' (Unit) 'Gross Sales' (GrossSales)
```

In this example, the dimension named *Business Unit* is renamed to *Unit* in the active dataset. The element named *Gross Sales* in the measure dimension is renamed to *GrossSales* in the active dataset.

GGRAPH

Note: Square brackets used in the GGRAPH syntax chart are required parts of the syntax and are not used to indicate optional elements. Any equals signs (=) displayed in the syntax chart are required. The GRAPHSPEC subcommand is required.

```
GGRAPH
  /GRAPHDATASET NAME="name"
                DATASET=datasetname
                VARIABLES=variablespec
                DATAFILTER=varname(VALUES={valuelist} [UNLABELED = {INCLUDE**}]) ...
                                          {ALL      }              {EXCLUDE  }
                TRANSFORM={NO**                                          }
                          {VARSTOCASES(SUMMARY="varname" INDEX="varname")}
                MISSING={LISTWISE**    } REPORTMISSING={NO**}
                        {VARIABLEWISE  }               {YES }
                CASELIMIT={1000000**}
                          {value    }
  /GRAPHSPEC  SOURCE={INLINE                                                        }
                     {GPLFILE("filespec")                                           }
                     {VIZMLFILE("filespec")                                         }
                     {VIZTEMPLATE(NAME={"template name"}[LOCATION={LOCAL**                }]}
                                                                  {"repository folderspec"}
                                       {"filespec"     }          {FILE                    }
                                  MAPPING(templatemapping))
              EDITABLE={YES**}
                       {NO   }
              LABEL="string"
              DEFAULTTEMPLATE={YES**} TEMPLATE=["filespec" ...]
                              {NO   }
              VIZSTYLESHEET="stylesheet name"[LOCATION={LOCAL**                }]
                                                      {"repository folderspec"}
              VIZMAP={"map name"}[LOCATION={LOCAL**                }
                                           {"repository folderspec"}
                     {"filespec"}          {FILE                   }
                                  TYPE={DATA**   } MAPKEY="attribute label"
                                       {REFERENCE}
                                  SHOWALLFEATURES={YES**}
                                                  {NO   }]
  /COLORCYCLE COLORn (R, G, B) COLORn (R, G, B)...
  /FRAME OUTER={YES}  INNER={YES}
               {NO }        {NO }
  /GRIDLINES XAXIS={YES} YAXIS={YES}
                  {NO }       {NO }
  /FITLINES TOTAL={NO**} SUBGROUPS={NO**}
                  {YES }           {YES }
```

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 14.0

- Command introduced.

Release 15.0

- RENAME syntax qualifier deprecated.
- COUNTCI, MEDIANCI, MEANCI, MEANSD, and MEANSE functions introduced.

Release 17.0

- Added SOURCE=VIZTEMPLATE to support visualization templates.
- Added VIZSTYLESHEET keyword to support visualization stylesheets.

Release 19.0

- Added LOCATION=FILE to support visualization templates stored in an arbitrary location on the file system.

Release 20.0

- Added VIZMAP keyword to support map visualizations.

Release 25.0

- Added FITLINES subcommand.
- Added GRIDLINES subcommand.
- Added FRAME subcommand.
- Added COLORCYCLE subcommand.
- DATAFILTER keyword added to GRAPHDATASET subcommand.

Examples

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat COUNT()
  /GRAPHSPEC SOURCE=GPLFILE("simplebarchart.gpl").
```

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat COUNT()
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: count=col(source(s), name("COUNT"))
  GUIDE: axis(dim(1), label("Employment Category"))
  GUIDE: axis(dim(2), label("Count"))
  ELEMENT: interval(position(jobcat*count))
END GPL.
```

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat
  /GRAPHSPEC SOURCE=VIZTEMPLATE(NAME="Bar"
                                MAPPING("Categories"="jobcat" "Summary"="Count"))
   DEFAULTTEMPLATE=NO.
```

Overview

GGRAPH generates a graph by computing statistics from variables in a data source and constructing the graph according to the graph specification, which may be written in the Graphics Productions Language (GPL) or ViZml.

Basic Specification

The basic specification is the GRAPHSPEC subcommand.

Syntax Rules

- Subcommands and keywords can appear in any order.
- Subcommand names and keywords must be spelled out in full.
- The GRAPHDATASET and GRAPHSPEC subcommands are repeatable.
- Parentheses, equals signs, and slashes shown in the syntax chart are required.
- Strings in the GPL are enclosed in quotation marks. You cannot use single quotes (apostrophes).

Scope and Limitations

- With the IBM SPSS Statistics Batch Facility (available only with IBM SPSS Statistics Server), use the -i switch when submitting command files that contain BEGIN GPL-END GPL blocks.
- With the Production Facility, set the Syntax Input Mode to Interactive for jobs that contain GPL blocks.
- GPL blocks cannot be nested within GPL blocks.

- Inline GPL blocks cannot be contained within DEFINE-!ENDDEFINE macro definitions. If you want to use GPL within macro definitions, do not use inline GPL blocks. Put the GPL in a file and reference it with SOURCE=GPLFILE.
- GPL blocks can be contained in command syntax files run via the INSERT command, with the default SYNTAX=INTERACTIVE setting.
- GPL blocks cannot be contained within command syntax files run via the INCLUDE command.

GRAPHDATASET Subcommand

GRAPHDATASET creates graph datasets based on open IBM SPSS Statistics data files. The subcommand is repeatable, allowing you to create multiple graph datasets that can be referenced in a graph specification. Furthermore, multiple graph specifications (the ViZml or GPL code that defines a graph) can reference the same graph dataset.

Graph datasets contain the data that accompany a graph. The actual variables and statistics in the graph dataset are specified by the VARIABLES keyword.

Example

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat COUNT()
  /GRAPHSPEC SOURCE=GPLFILE("simplebarchart.gpl").
```

NAME Keyword

The NAME keyword specifies the name that identifies the graph dataset when it is referenced in a graph specification. There is no default name, so you must specify one. You can choose any name that honors variable naming rules. (For more information about naming rules, see “Variable Names” on page 42.) When the same graph dataset name is used in multiple GRAPHDATASET subcommands, the name in the last GRAPHDATASET subcommand is honored.

DATASET Keyword

The DATASET keyword specifies the dataset name of an open data file to use for the graph dataset. If the keyword is omitted, GGRAPH uses the active dataset. You can also use an asterisk (*) to refer to the active dataset.

The following are honored only for the active dataset (which cannot be named except with an asterisk):

- FILTER
- USE
- SPLIT FILE
- Weight filtering (exclusion of cases with non-positive weights)
- Temporary transformations
- Pending transformations

Example

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" DATASET=DataSet2 VARIABLES=jobcat COUNT()
  /GRAPHSPEC SOURCE=GPLFILE("simplebarchart.gpl").
```

VARIABLES Keyword

The VARIABLES keyword identifies the variables, statistics, and utility function results that are included in the graph dataset. These are collectively identified as a **variable specification**. The minimum variable specification is a variable. An aggregation or summary function is required when the variable specification includes a multiple-response set. The order of the variables and functions in the variable specification does not matter. Multiple aggregation or summary functions are allowed so that you can graph more than

one statistic. You can also use the ALL and TO keywords to include multiple variables without explicitly listing them. For information about the ALL keyword, see “Keyword ALL” on page 44. For information about the TO keyword, see “Keyword TO” on page 43.

When the variable specification includes an aggregation function and does *not* include the CASEVALUE function, the graph dataset is aggregated. Any stand-alone variables in the variable specification act as categorical break variables for the aggregation (including scale variables that are not parameters of a summary function). The function is evaluated for each unique value in each break variable. When the variable specification includes only variables or includes the CASEVALUE function, the graph dataset is unaggregated. The built-in variable $CASENUM is included in the unaggregated dataset. $CASENUM cannot be specified or renamed in the variable specification, but you can refer to it in the graph specification.

An unaggregated graph dataset includes a case for every case in the IBM SPSS Statistics dataset. An aggregated dataset includes a case for every combination of unique break variable values. For example, assume that there are two categorical variables that act as break variables. If there are three categories in one variable and two in the other, there are six cases in the aggregated graph dataset, as long as there are values for each category.

Note: If the dataset is aggregated, be sure to include all of the break variables in the graph specification (the ViZml or GPL). For example, if the variable specification includes two categorical variables and a summary function of a scale variable, the graph specification should use one of the categorical variables as the *x*-axis variable and one as a grouping or panel variable. Otherwise, the resulting graph will not be correct because it does not contain all of the information used for the aggregation.

Example

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat[NAME="empcat" LEVEL=NOMINAL] COUNT()
  /GRAPHSPEC SOURCE=GPLFILE("simplebarchart.gpl").
```

- The NAME qualifier renames a variable. See the topic “Variable and Function Names ” on page 822 for more information.
- The LEVEL qualifier specifies a temporary measurement level for a variable. See the topic “Measurement Level ” on page 823 for more information.

Variable and Function Names

The variable name that you use in the variable specification is the same as the name defined in the data dictionary. This also the default name for referencing the variable in the graph specification. To use a different name in the graph specification, rename the variable by appending the qualifier `[NAME="name"]` to the name in the variable specification. You might do this to avoid name conflicts across datasets, to shorten the name, or to reuse the same graph specification even if the datasets have different variable names. For example:

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat[NAME="catvar"] COUNT()
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: catvar=col(source(s), name("catvar"), unit.category())
  DATA: count=col(source(s), name("COUNT"))
  GUIDE: axis(dim(1), label("Employment Category"))
  GUIDE: axis(dim(2), label("Count"))
  ELEMENT: interval(position(catvar*count))
END GPL.
```

The default name for a summary function is the function name in uppercase letters followed by the parameters separated by underscores. For example, if the function is `MEAN(salary)`, the default name for referencing this function in the graph specification is `MEAN_salary`. For `GPTILE(salary,90)`, the default name is `GPTILE_salary_90`. You can also change the default function name using the qualifier `[NAME="name"]`, just as you do with variables. For example:

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset"
                VARIABLES=jobcat MEDIAN(salary) MEAN(salary)[NAME="meansal"]
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
```

```
  DATA: medsal=col(source(s), name("MEDIAN_salary"))
  DATA: meansal=col(source(s), name("meansal"))
  GUIDE: axis(dim(1), label("Employment Category"))
  GUIDE: axis(dim(2), label("Salary"))
  ELEMENT: line(position(jobcat*medsal), color("Median"))
  ELEMENT: line(position(jobcat*meansal), color("Mean"))
END GPL.
```

Error interval functions produce three values (a summary value, an upper bound, and a lower bound), so there are three default names for these functions. The default name for the summary value follows the same rule as the default name for a summary function: the function name in uppercase letters followed by the parameters separated by underscores. The other two values are this name with _HIGH appended to the name for the upper bound and _LOW appended to the name for the lower bound. For example, if the function is MEANCI(salary, 95), the default names for referencing the results of this function in the graph specification are MEANCI_salary_95, MEANCI_salary_95_HIGH, and MEANCI_salary_95_LOW. You can change the names of the values using the qualifiers [NAME="name" HIGH="name" LOW="name"]. For example:

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset"
                VARIABLES=jobcat COUNTCI(95)[NAME="stat" HIGH="high" LOW="low"]
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: stat=col(source(s), name("stat"))
  DATA: high=col(source(s), name("high"))
  DATA: low=col(source(s), name("low"))
  GUIDE: axis(dim(1), label("Employment Category"))
  GUIDE: axis(dim(2), label("Count with 95% CI"))
  ELEMENT: point(position(jobcat*stat))
  ELEMENT: interval(position(region.spread.range(jobcat*(low+high))), shape(shape.ibeam))
END GPL.
```

Measurement Level

You can change a variable's measurement level temporarily by appending the qualifier [LEVEL=measurement level] to the name in the variable specification. (The variable's measurement level in the dictionary is unaffected.) Valid values for the measurement level are SCALE, NOMINAL, and ORDINAL. Currently, the measurement level qualifier is used to influence the behavior of the REPORTMISSING keyword. If the measurement level is set to SCALE, missing values are not reported for that variable, even if the value of the REPORTMISSING keyword is YES. If you are using the NAME qualifier for the same variable, both qualifiers are enclosed in the same pair of square brackets. For example:

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat[NAME="empcat" LEVEL=NOMINAL] COUNT()
  /GRAPHSPEC SOURCE=GPLFILE("simplebarchart.gpl").
```

Functions

Utility functions:

CASEVALUE(var). Yields the value of the specified variable for each case. CASEVALUE always produces one value for each case and always results in GGRAPH creating an unaggregated graph dataset. Use this function when you are creating graphs of individual cases and want to use the values of the specified variable as the axis tick labels for each case. This function cannot be used with multiple response sets or aggregation functions.

Aggregation functions:

Three groups of aggregation functions are available: count functions, summary functions, and error interval functions.

Count functions:

Note: Percent and cumulative statistic functions are not available in the variable specification. Use the summary percent and cumulative statistic functions that are available in the Graphics Production Language (GPL) itself.

COUNT(). Frequency of cases in each category.

RESPONSES(). Number of responses for a multiple dichotomy set.

RESPONSES (DUP / NODUP). Number of responses for a multiple category set. The argument (DUP or NODUP) specifies whether the function counts duplicates. The argument is optional, and the default is not to count duplicates. This function cannot be used with a multiple dichotomy set.

- Count functions yield the count of valid cases within categories determined by the other variables in the variable specification (including other scale variables that are not parameters of a summary function).
- Count functions do not use variables as parameters.

Summary functions:

MINIMUM(var). Minimum value of the variable.

MAXIMUM(var). Maximum value of the variable.

VALIDN(var). Number of cases for which the variable has a nonmissing value.

SUM(var). Sum of the values of the variable.

MEAN(var). Mean of the variable.

STDDEV(var). Standard deviation of the variable.

VARIANCE(var). Variance of the variable.

MEDIAN(var). Median of the variable.

GMEDIAN(var). Group median of the variable.

MODE(var). Mode of the variable.

PTILE(var,x). Xth percentile value of the variable. X must be greater than 0 and less than 100.

GPTILE(var,x). Xth percentile value of the variable, where the percentile is calculated as if the values were uniformly distributed over the whole interval. X must be greater than 0 and less than 100.

PLT(var,x). Percentage of cases for which the value of the variable is less than x.

PGT(var,x). Percentage of cases for which the value of the variable is greater than x.

NLT(var,x). Number of cases for which the value of the variable is less than x.

NGT(var,x). Number of cases for which the value of the variable is greater than x.

PIN(var,x1,x2). Percentage of cases for which the value of the variable is greater than or equal to x1 and less than or equal to x2. x1 cannot exceed x2.

NIN(var,x1,x2). Number of cases for which the value of the variable is greater than or equal to x1 and less than or equal to x2. x1 cannot exceed x2.

NLE(var,x). Number of cases for which the value of the variable is less than or equal to x.

PLE(var,x). Percentage of cases for which the value of the variable is less than or equal to x.

NEQ(var,x). Number of cases for which the value of the variable is equal to x.

PEQ(var,x). Percentage of cases for which the value of the variable is equal to x.

NGE(var,x). Number of cases for which the value of the variable is greater than or equal to x.

PGE(var,x). Percentage of cases for which the value of the variable is greater than or equal to x.

- Summary functions yield a single value.
- Summary functions operate on summary variables (variables that record continuous values, such as age or expenses). To use a summary function, specify the name of one or more variables as the first parameter of the function and then specify other required parameters as shown. The variable used as a parameter cannot contain string data.

Error interval functions:

COUNTCI(alpha). Confidence intervals for the count with a confidence level of alpha. alpha must be greater than or equal to 50 and less than 100.

MEDIANCI(var,alpha). Confidence intervals for median of the variable with a confidence level of alpha. alpha must be greater than or equal to 50 and less than 100.

MEANCI(var,alpha). Confidence intervals for mean of the variable with a confidence level of alpha. alpha must be greater than or equal to 50 and less than 100.

MEANSD (var,multiplier). Standard deviations for mean of the variable with a multiplier. multiplier must be an integer greater than 0.

MEANSE (var,multiplier). Standard errors for mean of the variable with a multiplier. multiplier must be an integer greater than 0.

- Error functions yield three values: a summary value, a lower bound value, and an upper bound value.
- Error functions may or may not operate on summary variables (variables that record continuous values, such as age or expenses). To use a summary function that operates on a variable, specify the name of the variable as the first parameter of the function and then specify other required parameters as shown. The variable used as a parameter cannot contain string data.

DATAFILTER Keyword

The DATAFILTER keyword filters values of categorical variables in the data that are used to generate the chart. Any categorical (nominal or ordinal) numeric or string variable from the active dataset can be specified on DATAFILTER. The filter applies only to the data that are used to generate the chart and does not affect the active dataset.

- You can specify multiple variables on the DATAFILTER keyword but a particular variable can be specified only once.
- Variables that are specified on the DATAFILTER keyword do not support ALL and TO.
- String values do not need to be quoted, unless the string contains apostrophes or quotation marks. In addition, the string value ALL must be quoted if it is used as the value of a variable. If the string contains apostrophes or single quotation marks, then enclose the string in double quotation marks. If the string contains double quotation marks, then enclose the string in single quotation marks.
- For variables that are not specified on DATAFILTER, all values are included.
- DATAFILTER honors measurement level settings that are specified on the VARIABLES keyword.

VALUES

The VALUES keyword, which is required, specifies a blank-separated list of values (from the associated variable) to include or the keyword ALL.

- If ALL is specified and UNLABELED=INCLUDE, only values without value labels are included.
- If ALL is specified and UNLABELED=EXCLUDE, only values with value labels are included.

UNLABELED

The UNLABELED keyword specifies the handling of values that do not have value labels. The value INCLUDE, which is the default, specifies that any unlabeled values of the variable that are not specified on the VALUES keyword are included. The value EXCLUDE specifies that any unlabeled values of the variable that are not specified on the VALUES keyword are excluded.

Example

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat MEAN(salary)[name="MEAN_salary"]
    MISSING=LISTWISE REPORTMISSING=NO DATAFILTER=jobcat(VALUES=1 2)
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: MEAN_salary=col(source(s), name("MEAN_salary"))
  GUIDE: axis(dim(1), label("Employment Category"))
  GUIDE: axis(dim(2), label("Mean Current Salary"))
  GUIDE: text.title(label("Simple Bar Mean of Current Salary by Employment Category"))
  SCALE: cat(dim(1), include("1", "2"))
  SCALE: linear(dim(2), include(0))
  ELEMENT: interval(position(jobcat*MEAN_salary), shape.interior(shape.square))
END GPL.
```

TRANSFORM Keyword

The TRANSFORM keyword applies a transformation to the graph dataset.

NO. *Do not transform the graph dataset.*

VARSTOCASES (SUMMARY="varname" INDEX="varname"). *Transform the summary function results to cases in the graph dataset.* Use this when you are creating graphs of separate variables. The results of each summary function becomes a case in the graph dataset, and the data elements drawn for each case act like categories in a categorical graph. Each case is identified by an index variable whose value is a unique sequential number. The result of the summary function is stored in the summary variable. The upper and lower bound of error interval functions are also stored in two other variables. By default, the names of the variables are #INDEX for the index variable, #SUMMARY for the summary variable, #HIGH for the upper bound variable, and #LOW for the lower bound variable. You can change these names by using the SUMMARY, INDEX, HIGH, and LOW qualifiers. Furthermore, break variables in the variable specification are treated as fixed variables and are not transposed. Note that this transformation is similar to the VARSTOCASES command (see "VARSTOCASES" on page 2133).

Examples

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=MEAN(salbegin) MEAN(salary)
   TRANSFORM=VARSTOCASES(SUMMARY="meansal" INDEX="variables")
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: variables=col(source(s), name("variables"), unit.category())
  DATA: meansal=col(source(s), name("meansal"))
  GUIDE: axis(dim(2), label("Mean"))
  ELEMENT: interval(position(variables*meansal))
END GPL.
```

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=MEANCI(salbegin, 95) MEANCI(salary, 95)
   TRANSFORM=VARSTOCASES(SUMMARY="meansal" INDEX="variables" LOW="low" HIGH="high")
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: variables=col(source(s), name("variables"), unit.category())
  DATA: meansal=col(source(s), name("meansal"))
  DATA: low=col(source(s), name("low"))
  DATA: high=col(source(s), name("high"))
  GUIDE: axis(dim(2), label("Mean with 95% CI"))
  ELEMENT: point(position(variables*meansal))
  ELEMENT: interval(position(region.spread.range(variables*(low+high))),
                    shape(shape.ibeam))
END GPL.
```

MISSING Keyword

The MISSING keyword specifies how missing values are handled when the variable specification includes an aggregation function. When the variable specification includes only variables or includes the CASEVALUE function, this keyword does not affect the treatment of missing values. The graph dataset is unaggregated, so cases with system- and user-missing values are always included in the graph dataset.

LISTWISE. *Exclude the whole case if any one of the variables in the variable specification has a missing value.* This is the default.

VARIABLEWISE. *Exclude a case from the aggregation function if the value is missing for a particular variable being analyzed.* This means that a case is excluded if that case has a missing value for a variable that is a summary function parameter.

REPORTMISSING Keyword

The REPORTMISSING keyword specifies whether to create a category for each unique user-missing value.

NO. *Do not create a category for each unique user-missing value.* User-missing values are treated like system-missing values. This is the default.

YES. *Create a category for each unique user-missing value.* User-missing values are treated as valid categories, are included as break variables for aggregation functions, and are drawn in the graph. Note that this does not affect variables identified as SCALE by the LEVEL qualifier in the VARIABLES keyword.

CASELIMIT Keyword

The CASELIMIT keyword specifies a limit to the number of cases that are included in the graph dataset. The limit does not apply to the number of cases use for analysis in any functions specified by the VARIABLES keyword. It only limits the number of cases in the graph dataset, which may or may not affect the number cases drawn in the resulting chart. You may want to limit the number of cases for very large datasets that are not summarized by a function. A scatterplot is an example. Limiting cases may improve performance.

value. *Limit the number of cases in the graph dataset to the specified value.* The default value is 1000000.

GRAPHSPEC Subcommand

GRAPHSPEC defines a graph specification. A graph specification identifies the source used to create the graph, in addition to other features like templates. GRAPHSPEC is repeatable, allowing you to define multiple graph specifications to create multiple graphs with one GGRAPH command.

Example

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat COUNT()
  /GRAPHSPEC SOURCE=GPLFILE("simplebarchart.gpl").
```

SOURCE Keyword

The SOURCE keyword specifies the source of the graph specification.

INLINE. *An inline graph specification follows the GGRAPH command.* Currently, the BEGIN GPL/END GPL block is used for the inline graph specification. This block must follow the GGRAPH command, and there must be as many blocks as there are GRAPHSPEC subcommands with SOURCE=INLINE. See the topic “BEGIN GPL-END GPL” on page 243 for more information. See “Overview ” on page 243 for limitations.

GPLFILE ("filespec"). *Use the specified GPL file as the graph specification.* See the *GPL Reference Guide* on the manuals CD for more details about GPL. The examples in the GPL documentation may look different compared to the syntax pasted from the Chart Builder. The main difference is in when aggregation occurs. See “Working with the GPL ” on page 829 below for information about the differences. See “GPL Examples ” on page 834 for examples with GPL that is similar to the pasted syntax.

VIZMLFILE ("filespec"). *Use the specified ViZml file as the graph specification.* You can save ViZml from the Chart Editor. See the topic for more information.

VIZTEMPLATE(NAME="template name"[LOCATION=LOCAL].

MAPPING(templatemapping)). Use the specified visualization template and mapping as the graph specification.

VIZTEMPLATE(NAME="template name"[LOCATION="repository folderspec"] .

MAPPING(templatemapping)). Use the specified visualization template and mapping as the graph specification.

VIZTEMPLATE(NAME="filespec"[LOCATION=FILE].

MAPPING(templatemapping)). Use the specified visualization template and mapping as the graph specification.

The LOCATION qualifier specifies the location in which to find the named template. You specify whether the template is a local imported template, is obtained from a specific IBM SPSS Collaboration and Deployment Services Repository, or is stored in a arbitrary location on the file system. Note that a visualization template is not the same as a template for use with the TEMPLATE keyword.

The template mapping is a series of key-value pairs separated by spaces. An example is `MAPPING("Categories"="jobcat" "Summary"="Count")`.

- The key is a slot defined by the template, and the value is a variable or string assigned to the slot.
- If a variable is renamed in the VARIABLES keyword using the `[NAME="name"]` qualifier, the value refers to the rename, not the original name.
- In the example, the keys are `Categories` and `Summary`, and the respective values are `jobcat` and `Count`.
- Individual keys and values are enclosed in quotes.
- Each value can also be followed by the qualifier `[DATASET="dataset name"]` to indicate a variable's dataset.
- At this time, only one dataset can be used with each template.
- To determine valid slot names, open the Graphboard Template Chooser, add variables to slots of interest, and paste the syntax. You can access the Template Chooser by selecting **Graphboard Template Chooser** from the Graphs menu. Visualization templates on the file system are not available in the Graphboard Template Chooser. You will have to import them first, or you will have to open them in a text editor to determine the slot names.

New templates can be created with the separate IBM SPSS Visualization Designer product. The GGRAPH command can use these templates if they are stored in the IBM SPSS Collaboration and Deployment Services Repository or if they are imported using the Graphboard Template Chooser.

Note: When using a visualization template, it is recommended that you include DEFAULTTEMPLATE=NO. This setting will prevent the default template from overriding the styles in the visualization template.

- If you received a template and want to reference it in syntax, you must first manually import it with the Graphboard Template Chooser to make it a local template.
- For built-in or imported templates, set the location to LOCAL.
- For templates stored in a IBM SPSS Collaboration and Deployment Services Repository, use a IBM SPSS Collaboration and Deployment Services Repository file specification. For more information about IBM SPSS Collaboration and Deployment Services Repository file specifications, see “File Specifications for IBM SPSS Collaboration and Deployment Services Repository Objects” on page 2217 .
- For templates stored on the file system, set the location to FILE and specify the full path to the visualization template file with NAME.

Examples

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat COUNT()
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: count=col(source(s), name("COUNT"))
  GUIDE: axis(dim(1), label("Employment Category"))
  GUIDE: axis(dim(2), label("Count"))
  ELEMENT: interval(position(jobcat*count))
END GPL.
```

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat COUNT()
  /GRAPHSPEC SOURCE=GPLFILE("simplebarchart.gpl").
```

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat COUNT()
  /GRAPHSPEC SOURCE=VIZMLFILE("simplebarchart.xml").
```

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat
  /GRAPHSPEC SOURCE=VIZTEMPLATE(NAME="Bar"
                MAPPING("Categories"="jobcat" "Summary"="Count"))
   DEFAULTTEMPLATE=NO.
```

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=salbegin salary gender[NAME="sex"]
  /GRAPHSPEC SOURCE=VIZTEMPLATE(NAME="Binned Scatter"[LOCATION="SPSSCR:///viz/templates"]
```

```
                                MAPPING("X"="salbegin"[DATASET="graphdataset"]
                                        "Y"="salary"[DATASET="graphdataset"]
                                        "Panel across"="sex"[DATASET="graphdataset"]))
   DEFAULTTEMPLATE=NO.
```

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=salbegin salary gender[NAME="sex"]
  /GRAPHSPEC SOURCE=VIZTEMPLATE(NAME="/vizfiles/MyBinnedScatter.viztemplate"[LOCATION=FILE]
                                MAPPING("X"="salbegin"[DATASET="graphdataset"]
                                        "Y"="salary"[DATASET="graphdataset"]
                                        "Panel across"="sex"[DATASET="graphdataset"]))
   DEFAULTTEMPLATE=NO.
```

Working with the GPL

The Chart Builder allows you to paste GGRAPH syntax. This syntax contains inline GPL

You may want to edit the GPL to create a chart or add a feature that isn't available from the Chart Builder. You can use the GPL documentation to help you. However, the GPL documentation always uses unaggregated data and includes GPL statistics in the examples to aggregate the data. The pasted syntax, on the other hand, may use data aggregated by a GGRAPH summary function. Also, the pasted syntax includes defaults that you may have to change when you edit the syntax. Therefore, it may be confusing how you can use the pasted syntax to create the examples. Following are some tips.

- Variables must be specified in two places: in the VARIABLES keyword in the GGRAPH command and in the DATA statements in the GPL. So, if you add a variable, make sure a reference to it appears in both places.
- Pasted syntax often uses the VARIABLES keyword to specify summary statistics. Like other variables, the summary function name is specified in the GPL DATA statement. You do not need to use GGRAPH summary functions. Instead, you can use the equivalent GPL statistic for aggregation. However, for very large data sets, you may find that pre-aggregating the data with GGRAPH is faster than using the aggregation in the GPL itself. Try both approaches and stick with the one that feels comfortable to you. In the examples that follow, you can compare the different approaches.
- Make sure that you understand how the functions are being used in the GPL. You may need to modify one or more of them when you add a variable to pasted syntax. For example, if you change the dimension on which a categorical variable appears, you may need to change references to the dimension in the GUIDE and SCALE statements. If you are unsure about whether you need a particular function, try removing it and see if you get the results you expect.

Here's an example from the GPL documentation:

```
DATA: jobcat = col(source(s), name("jobcat"), unit.category())
DATA: gender = col(source(s), name("gender"), unit.category())
DATA: salary = col(source(s), name("salary"))
SCALE: linear(dim(2), include(0))
GUIDE: axis(dim(3), label("Gender"))
GUIDE: axis(dim(2), label("Mean Salary"))
GUIDE: axis(dim(1), label("Job Category"))
ELEMENT: interval(position(summary.mean(jobcat*salary*gender)))
```

Figure 61. Example from GPL documentation

The simplest way to use the example is to use unaggregated data and VARIABLES=ALL like this:

```
GGRAPH
  /GRAPHDATASET NAME="Employeedata" VARIABLES=ALL
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
SOURCE: s=usersource(id("Employeedata"))
DATA: jobcat = col(source(s), name("jobcat"), unit.category())
DATA: gender = col(source(s), name("gender"), unit.category())
DATA: salary = col(source(s), name("salary"))
SCALE: linear(dim(2), include(0))
GUIDE: axis(dim(3), label("Gender"))
GUIDE: axis(dim(2), label("Mean Salary"))
GUIDE: axis(dim(1), label("Job Category"))
ELEMENT: interval(position(summary.mean(jobcat*salary*gender)))
END GPL
```

Figure 62. Modified example with unaggregated data

Note that specifying VARIABLES=ALL includes all the data in the graph. You can improve performance by using only those variables that you need. In this example, VARIABLES=jobcat gender salary would have been sufficient.

You can also use aggregated data like the following, which is more similar to the pasted syntax:

```
GGRAPH
  /GRAPHDATASET NAME="Employeedata" VARIABLES=jobcat gender MEAN(salary)
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("Employeedata"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: gender=col(source(s), name("gender"), unit.category())
  DATA: MEAN_salary=col(source(s), name("MEAN_salary"))
  SCALE: linear(dim(2), include(0))
  GUIDE: axis(dim(3), label("Gender"))
  GUIDE: axis(dim(2), label("Mean Salary"))
  GUIDE: axis(dim(1), label("Job Category"))
  ELEMENT: interval(position(jobcat*MEAN_salary*gender))
END GPL.
```

Figure 63. Modified example with aggregated data

EDITABLE Keyword

The EDITABLE keyword specifies that the resulting graph can be edited. If you are creating a complicated graph with the graph specification, it may be useful to prevent editing because not all of the graph's features may be supported in the editor.

YES. *The graph can be edited.* This is the default.

NO. *The graph cannot be edited.*

LABEL Keyword

The LABEL keyword specifies the output label. This label appears in the Output Viewer. It is also used in Output XML (OXML) as a `chartTitle` element, which is not the same as the title in the graph itself.

string. *Use the specified string as the label.*

DEFAULTTEMPLATE Keyword

The DEFAULTTEMPLATE keyword specifies whether GGRAPH applies the default styles to the graph. Most default styles are defined in the Options dialog box, which you can access by choosing **Options** from the Edit menu. Then click the **Charts** tab. Some SET commands also define default aesthetics. Finally, other default styles are set to improve the presentation of graphs. These are controlled by the *chart_style.sgt* template file located in the installation directory.

YES. *Apply default styles to the graph.* This is the default.

NO. *Do not apply default styles to the graph.* This option is useful when you are using a custom ViZml or GPL file that defines styles that you do not want to be overridden by the default styles.

TEMPLATE Keyword

The TEMPLATE keyword identifies an existing template file or files and applies them to the graph requested by the current GGRAPH command. The template overrides the default settings that are used to create any graph, and the specifications on the current GGRAPH command override the template. Templates do *not* override visualization stylesheets specified with the VIZSTYLESHEET keyword.

Templates are created in the Chart Editor by saving an existing chart as a template. Note that these templates are not the same as visualization templates. Visualization templates are created in a separate IBM SPSS Visualization Designer product and are referenced by the SOURCE keyword.

The TEMPLATE keyword is followed by an equals sign (=) and square brackets ([ ]) that contain one or more file specifications. Each file specification is enclosed in quotation marks. The square brackets are optional if there is only one file, but the file must be enclosed in quotation marks. Note that the order in which the template files are specified is the order in which GGRAPH applies the templates. Therefore, template files that appear after other template files can override the templates that were applied earlier.

filespec. *Apply the specified template file or files to the graph being created.*

Example

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat COUNT()
  /GRAPHSPEC SOURCE=GPLFILE("simplebarchart.gpl")
             TEMPLATE=["mytemplate.sgt"
                       "/myothertemplate.sgt"].
```

VIZSTYLESHEET Keyword

The VIZSTYLESHEET keyword identifies an existing visualization stylesheet and applies it to the graph requested by the current GGRAPH command. Styles stored in the original visualization template, GPL specification, ViZml specification, or user-specified templates are preserved. Because the default template may set styles that cannot be replaced by the visualization stylesheet, it is recommended that you use DEFAULTTEMPLATE=NO in combination with the VIZSTYLESHEET keyword.

The keyword is followed by an equals sign (=) and the name of the stylesheet in quotation marks. By default, GGRAPH looks for a built-in or imported template that is stored on the local machine. This default behavior is equivalent to appending the qualifier `[LOCATION=LOCAL]` to the quoted stylesheet name. To use a stylesheet in a IBM SPSS Collaboration and Deployment Services Repository, append the qualifier `[LOCATION="repository folderspec"]` to the quoted stylesheet name. For more information about IBM SPSS Collaboration and Deployment Services Repository file specifications, see “File Specifications for IBM SPSS Collaboration and Deployment Services Repository Objects” on page 2217 . The IBM SPSS Collaboration and Deployment Services Repository file specification is enclosed in quotation marks.

New stylesheets can be created with the separate IBM SPSS Visualization Designer product. The GGRAPH command can reference these stylesheets if they are stored in the IBM SPSS Collaboration and Deployment Services Repository or if they are imported using the Graphboard Template Chooser. You can access the Template Chooser by selecting **Graphboard Template Chooser** from the Graphs menu.

"stylesheet name". *Apply the specified stylesheet to the graph being created.*

Example

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat
  /GRAPHSPEC SOURCE=VIZTEMPLATE(NAME="Binned Scatter"[LOCATION=LOCAL]
                                MAPPING("X"="salbegin" "Y"="salary"
                                        "Panel across"="gender"))
   VIZSTYLESHEET="Corporate"[LOCATION="SPSSCR:///viz/stylesheets"]
   DEFAULTTEMPLATE=NO.
```

VIZMAP Keyword

The VIZMAP keyword identifies an existing map file for use with a map visualization template. The keyword has an effect only if SOURCE=VIZTEMPLATE and the named template identifies a map template. The keyword is followed by an equals sign (=) and the name of the map file in quotation marks.

There are four qualifiers that affect how the map visualization is rendered. These qualifiers are grouped together in square brackets after the quoted map file name:

LOCATION. By default, GGRAPH looks for a built-in or imported map file that is stored on the local machine. This default behavior is equivalent to appending the qualifier `[LOCATION=LOCAL]` to the quoted map file name. To use a map file in a IBM SPSS Collaboration and Deployment Services Repository, append the qualifier `[LOCATION="repository folderspec"]` to the quoted map file name. For more information about IBM SPSS Collaboration and Deployment Services Repository file specifications, see “File Specifications for IBM SPSS Collaboration and Deployment Services Repository Objects” on page 2217 . The IBM SPSS Collaboration and Deployment Services Repository file specification is enclosed in quotation marks.

For viztemplate files or map files that are stored on the file system, set `[LOCATION=FILE]` and specify the full path to the visualization template file with VIZMAP or NAME. Using a map stored on the file system is necessary when the IBM SPSS Statistics client is not available (for example, when using IBM SPSS Statistics Batch Facility or running the Production Facility disconnected).

TYPE. By default, GGRAPH assumes that the map file is the main data map file. This default behavior is equivalent to appending the qualifier [TYPE=DATA] to the quoted map file name. The main data map file is keyed to the data to create the map visualization. Overlay map visualization templates also require a reference map file that is drawn behind the main data map. To specify a reference map file, use the qualifier [TYPE=REFERENCE].

MAPKEY. When you are specifying a main data map file ([TYPE=DATA]), you need to identify the map attribute that is used as the key for linking to the data. Enclose the map attribute name in quotation marks.

SHOWALLFEATURES. By default, GGRAPH will render all the features in the map, even if a feature has no matching key in the data. This default behavior is equivalent to appending the qualifier [SHOWALLFEATURES=YES] to the quoted map file name. If you want the map visualization to show only those features for which there is data, use the qualifier [SHOWALLFEATURES=NO].

Map files are SMZ files. IBM SPSS Statistics is pre-installed with several SMZ files. SMZ files can also be created by converting ESRI shapefiles with the Map Conversion Utility. The GGRAPH command can reference these map files if they are stored in the IBM SPSS Collaboration and Deployment Services Repository or if they are imported using the Graphboard Template Chooser. You can access the Template Chooser by selecting **Graphboard Template Chooser** from the Graphs menu. For more information using the Map Conversion Utility, see .

"map name". *Apply the specified map file to the map visualization being created.*

Examples

For templates that are stored on the file system, set **[LOCATION=FILE]** and specify the full path to the visualization template file with **NAME**.

```
GGRAPH
     /GRAPHDATASET NAME="graphdataset" VARIABLES=salbegin salary gender[NAME="sex"]
     /GRAPHSPEC SOURCE=VIZTEMPLATE(NAME="/vizfiles/MyBinnedScatter.viztemplate"[LOCATION=FILE]
          MAPPING("X"="salbegin"[DATASET="graphdataset"]
          "Y"="salary"[DATASET="graphdataset"]
          "Panel across"="sex"[DATASET="graphdataset"]))
     DEFAULTTEMPLATE=NO.
```

For maps that are stored on the file system, set [LOCATION=FILE] and specify the full path to the visualization template file with **VIZMAP**.

MacOS example:

```
GET
FILE='/Applications/IBMSPSSStatistics/Subscription/Samples/English/Employee data.sav'.
GGRAPH
     /GRAPHDATASET NAME="graphdataset" VARIABLES=Continent[LEVEL=nominal] MISSING=LISTWISE REPORTMISSING=NO
     /GRAPHSPEC SOURCE=VIZTEMPLATE(NAME="Choropleth of Counts"[LOCATION=LOCAL]
MAPPING( "Data_Key"="Continent"[DATASET="graphdataset"])) VIZSTYLESHEET="Traditional"[LOCATION=LOCAL]
          LABEL='CHOROPLETH OF COUNTS: Continent'
          DEFAULTTEMPLATE=NO
          VIZMAP="/Users/<user_name>/Downloads/Continents.smz"[LOCATION=FILE TYPE=DATA mapkey="CONTINENT" SHOWALLFEATURES=YES].
```

Windows example:

```
GET
FILE='C:\Program Files\IBM\SPSS\Statistics\26\Samples\English\Employee data.sav'.
GGRAPH
     /GRAPHDATASET NAME="graphdataset" VARIABLES=Continent[LEVEL=nominal] MISSING=LISTWISE REPORTMISSING=NO
     /GRAPHSPEC SOURCE=VIZTEMPLATE(NAME="Choropleth of Counts"[LOCATION=LOCAL]
MAPPING( "Data_Key"="Continent"[DATASET="graphdataset"])) VIZSTYLESHEET="Traditional"[LOCATION=LOCAL]
          LABEL='CHOROPLETH OF COUNTS: Continent'
          DEFAULTTEMPLATE=NO
          VIZMAP="C:\samples\Continents.smz"[LOCATION=FILE TYPE=DATA mapkey="CONTINENT" SHOWALLFEATURES=YES].
```

COLORCYCLE Subcommand

The COLORCYCLE subcommand specifies custom color settings for charts with group elements identified by color. For example, a clustered bar chart.

```
/COLORCYCLE
  COLORn (R, G, B), COLORn (R, G, B)...
```

- n is an integer that represents the order in the color cycle.
- The three values in parentheses are RGB color values.
- Default or specified template colors are overridden only for the specified categories.
- If pattern cycling is in effect (**Edit** > **Options** > **Charts**) and the TEMPLATE keyword of the GRAPHSPEC subcommand does not specify a template that includes color cycles, this keyword has no effect.

FRAME Subcommand

The FRAME subcommand specifies custom inner and outer frame attributes.

```
/FRAME
  OUTER=YES | NO
  INNER=YES | NO
```

The default frame settings are the frame settings defined in the default template or the template specified on the TEMPLATE keyword of the GRAPHSPEC subcommand.

GRIDLINES Subcommand

The GRIDLINES subcommand specifies custom grid line settings.

```
/GRIDLINES
  XAXIS=YES | NO
  YAXIS=YES | NO
```

The default grid line settings are the grid line settings defined in the default template or the template specified on the TEMPLATE keyword of the GRAPHSPEC subcommand.

FITLINES Subcommand

The FITLINES subcommand specifies linear fit line options for simple scatterplots (two variables), simple scatterplots with a grouping variable, and scatterplot matrixes.

```
/FITLINES
  TOTAL=NO | YES
  SUBGROUPS= NO | YES
```

- The default setting for both TOTAL and SUBGROUPS is NO.
- If the chart does not have a grouping variable, the SUBGROUPS keyword is ignored.
- If the chart is not a simple scatterplot, a simple scatterplot with a grouping variable, or a scatterplot matrix, this subcommand is ignored.

GPL Examples

Following are some graph examples using GPL. Pictures are not included to encourage you to run the examples. Except when noted, all examples use *Employee data.sav*, which is located in the product installation directory. See the *GPL Reference Guide* on the manuals CD for more examples.

Simple Bar Chart

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat MEAN(salary)
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: meansal=col(source(s), name("MEAN_salary"))
  GUIDE: axis(dim(1), label("Employment Category"))
  GUIDE: axis(dim(2), label("Mean Current Salary"))
  ELEMENT: interval(position(jobcat*meansal))
END GPL.
```

Simple Bar Chart Using a Multiple-Response Set

Note: This example uses *1991 U.S. General Social Survey.sav*, which is located in the product installation directory.

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=$prob RESPONSES()[NAME="RESPONSES"]
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: prob=col(source(s), name("$prob"), unit.category())
  DATA: responses=col(source(s), name("RESPONSES"))
  GUIDE: axis(dim(1), label("Most Important Problems in Last 12 Months"))
  GUIDE: axis(dim(2), label("Responses"))
  ELEMENT: interval(position(prob*responses))
END GPL.
```

Stacked Bar Chart

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat gender COUNT()
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: gender=col(source(s), name("gender"), unit.category())
  DATA: count=col(source(s), name("COUNT"))
  GUIDE: axis(dim(1), label("Employment Category"))
  GUIDE: axis(dim(2), label("Count"))
  ELEMENT: interval.stack(position(jobcat*count), color(gender))
END GPL.
```

Clustered Bar Chart

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat gender MEAN(salary)
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: gender=col(source(s), name("gender"), unit.category())
  DATA: meansal=col(source(s), name("MEAN_salary"))
  COORD: rect(dim(1,2), cluster(3,0))
  GUIDE: axis(dim(1), label("Employment Category"))
  GUIDE: axis(dim(2), label("Mean Current Salary"))
  ELEMENT: interval(position(gender*meansal*jobcat), color(gender))
END GPL.
```

Paneled Bar Chart

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat gender MEAN(salary)
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: gender=col(source(s), name("gender"), unit.category())
  DATA: meansal=col(source(s), name("MEAN_salary"))
  GUIDE: axis(dim(1), label("Gender"))
  GUIDE: axis(dim(2), label("Mean Current Salary"))
  GUIDE: axis(dim(3), label("Employment Category"))
  ELEMENT: interval(position(gender*meansal*jobcat))
END GPL.
```

3-D Bar Chart

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat gender MEAN(salary)
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: gender=col(source(s), name("gender"), unit.category())
  DATA: meansal=col(source(s), name("MEAN_salary"))
  GUIDE: axis(dim(1), label("Gender"))
  GUIDE: axis(dim(2), label("Employment Category"))
  GUIDE: axis(dim(3), label("Mean Current Salary"))
  COORD: rect(dim(1,2,3))
  ELEMENT: interval(position(gender*jobcat*meansal))
END GPL.
```

Simple Scatterplot

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=salbegin salary
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: salbegin=col(source(s), name("salbegin"))
  DATA: salary=col(source(s), name("salary"))
  GUIDE: axis(dim(1), label("Beginning Salary"))
  GUIDE: axis(dim(2), label("Current Salary"))
  ELEMENT: point(position(salbegin*salary))
END GPL.
```

Simple Scatterplot with Fit Line

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=salbegin salary
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: salbegin=col(source(s), name("salbegin"))
  DATA: salary=col(source(s), name("salary"))
  GUIDE: axis(dim(1), label("Beginning Salary"))
  GUIDE: axis(dim(2), label("Current Salary"))
  ELEMENT: point(position(salbegin*salary))
  ELEMENT: line(position(smooth.linear(salbegin*salary)))
END GPL.
```

Pie Chart

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat COUNT()
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE:s=userSource(id("graphdataset"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: count=col(source(s), name("COUNT"))
  GUIDE: axis(dim(1), null())
  COORD: polar.theta()
  SCALE: linear(dim(1), dataMinimum(), dataMaximum())
  ELEMENT: interval.stack(position(summary.percent(count)),color(jobcat))
END GPL.
```

Area Chart

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat MEAN(salary)
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: meansal=col(source(s), name("MEAN_salary"))
  GUIDE: axis(dim(1), label("Employment Category"))
  GUIDE: axis(dim(2), label("Mean Current Salary"))
  ELEMENT: area(position(jobcat*meansal))
END GPL.
```

Grouped Line Chart

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=jobcat gender MEAN(salary)
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: gender=col(source(s), name("gender"), unit.category())
  DATA: meansal=col(source(s), name("MEAN_salary"))
  GUIDE: axis(dim(1), label("Employment Category"))
  GUIDE: axis(dim(2), label("Mean Current Salary"))
```

```
  ELEMENT: line(position(jobcat*meansal), color(gender), missing.wings())
END GPL.
```

Bar Chart of Separate Variables

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=MEAN(salary) MEAN(salbegin)
   TRANSFORM=VARSTOCASES(SUMMARY="meansal" INDEX="variables")
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: variables=col(source(s), name("variables"), unit.category())
  DATA: meansal=col(source(s), name("meansal"))
  GUIDE: axis(dim(2), label("Mean"))
  ELEMENT: interval(position(variables*meansal))
END GPL.
```

Bar Chart Clustered by Separate Variables

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=MEAN(salary) MEAN(salbegin) jobcat
   TRANSFORM=VARSTOCASES(SUMMARY="meansal" INDEX="variables")
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: variables=col(source(s), name("variables"), unit.category())
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: meansal=col(source(s), name("meansal"))
  COORD: rect(dim(1,2), cluster(3,0))
  GUIDE: axis(dim(2), label("Mean"))
  GUIDE: legend(aesthetic(aesthetic.color), label("Variables"))
  ELEMENT: interval(position(variables*meansal*jobcat), color(variables))
END GPL.
```

Bar Chart of Separate Variables Clustered by Categorical Variable

```
GGRAPH
  /GRAPHDATASET NAME="graphdataset" VARIABLES=MEAN(salary) MEAN(salbegin) jobcat
   TRANSFORM=VARSTOCASES(SUMMARY="meansal" INDEX="variables")
  /GRAPHSPEC SOURCE=INLINE.
BEGIN GPL
  SOURCE: s=userSource(id("graphdataset"))
  DATA: variables=col(source(s), name("variables"), unit.category())
  DATA: jobcat=col(source(s), name("jobcat"), unit.category())
  DATA: meansal=col(source(s), name("meansal"))
  COORD: rect(dim(1,2), cluster(3,0))
  GUIDE: axis(dim(2), label("Mean"))
  GUIDE: legend(aesthetic(aesthetic.color), label("Employment Category"))
  ELEMENT: interval(position(jobcat*meansal*variables), color(jobcat))
END GPL.
```

GLM

GLM is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
GLM dependent varlist [BY factor list [WITH covariate list]]

[/WSFACTOR=name levels [{DEVIATION [(refcat)]       }] name...
                        {SIMPLE [(refcat)]          }
                        {DIFFERENCE                 }
                        {HELMERT                    }
                        {REPEATED                   }
                        {POLYNOMIAL [({1,2,3...})]**}
                        {           {metric  }      }
                        {SPECIAL (matrix)           }

[/MEASURE=newname newname...]

[/WSDESIGN=effect effect...]†

[/RANDOM=factor factor...]

[/REGWGT=varname]

[/METHOD=SSTYPE({1  })]
                {2  }
                {3**}
                {4  }

[/INTERCEPT=[INCLUDE**] [EXCLUDE]]

[/MISSING=[INCLUDE] [EXCLUDE**]]

[/CRITERIA=[EPS({1E-8**})][ALPHA({0.05**})]
                {a     }          {a     }

[/PRINT  = [DESCRIPTIVE] [HOMOGENEITY] [PARAMETER][ETASQ] [RSSCP]
           [GEF] [LOF] [OPOWER] [TEST [([SSCP] [LMATRIX] [MMATRIX])]
           [MBP] [BP] [F] [WHITE] ]

[/PLOT=[SPREADLEVEL] [RESIDUALS]
       [PROFILE (factor factor*factor factor*factor*factor ...)]
                [WITH(covariate={value} [...])]
                              {MEAN }
                [TYPE={LINE**}]
                      {BAR   }
                [MEANREFERENCE={NO**}]
                               {YES }
                [ERRORBAR={NO**      }]
                          {CI        }
                          {SE [(2**)]}
                [YAXIS={AUTO**}]
                       {0     }

[/TEST=effect VS {linear combination [DF(df)]}]
                 {value DF (df)              }

[/LMATRIX={["label"] effect list effect list ...;...}]
          {["label"] effect list effect list ...    }
          {["label"] ALL list; ALL...               }
          {["label"] ALL list                       }

[/CONTRAST (factor name)={DEVIATION[(refcat)]** ‡   }]
                         {SIMPLE [(refcat)]         }
                         {DIFFERENCE                }
                         {HELMERT                   }
                         {REPEATED                  }
                         {POLYNOMIAL  [({1,2,3...})]}
                         {             {metric  }   }
                         {SPECIAL (matrix)          }

[/MMATRIX= {["label"] depvar value depvar value ...;["label"]...}]
           {["label"] depvar value depvar value ...             }
           {["label"] ALL list; ["label"] ...                   }
           {["label"] ALL list                                  }

[/KMATRIX= {list of numbers    }]
           {list of numbers;...}

[/POSTHOC = effect [effect...]
          ([SNK] [TUKEY] [BTUKEY][DUNCAN]
           [SCHEFFE] [DUNNETT(refcat)] [DUNNETTL(refcat)]
           [DUNNETTR(refcat)] [BONFERRONI] [LSD] [SIDAK]
           [GT2] [GABRIEL] [FREGW] [QREGW]  [T2] [T3] [GH][C]
           [WALLER ({100** })]]
                   {kratio}
           [VS effect]

[/EMMEANS=TABLES({OVERALL              })  [WITH(covariate={value} [...])]]
                 {factor               }                  {MEAN }
                 {factor*factor...     }
                 {wsfactor             }
                 {wsfactor*wsfactor ... }
                 {factor*...wsfactor*...}

          [COMPARE ADJ(LSD) (BONFERRONI) (SIDAK)]

[/ROBUST={HC0  })]
         {HC1  }
         {HC2  }
         {HC3**}
```

```
         {HC4  }

[/SAVE=[tempvar [(list of names)]] [tempvar [(list of names)]]...]
       [DESIGN]

[/OUTFILE=[COVB ('savfile'|'dataset')]
          [CORB ('savfile'|'dataset')]
          [HCCOVB ('savfile'|'dataset')]
          [EFFECT('savfile'|'dataset')] [DESIGN('savfile'|'dataset')]

[/MBPDESIGN=[effect effect...]]

[/BPDESIGN=[effect effect...]]

[/FDESIGN=[effect effect...]]

[/DESIGN={[INTERCEPT...]    }]
         {[effect effect...]}
```

† WSDESIGN uses the same specification as DESIGN, with only within-subjects factors.

‡ DEVIATION is the default for between-subjects factors, while POLYNOMIAL is the default for within-subjects factors.

** Default if the subcommand or keyword is omitted.

Temporary variables (tempvar) are:

```
PRED, WPRED, RESID, WRESID, DRESID, ZRESID, SRESID, SEPRED, COOK, LEVER
```

Release 17.0

- POSTHOC subcommand: T2, T3, GH, and C keywords are not valid when multiple factors in the model.
- PLOT subcommand: new WITH keyword allows you to fix covariate values for profile plots.

Release 25.0

- TYPE, ERRORBAR, MEANREFERENCE, and YAXIS keywords added to PLOT subcommand.
- ROBUST subcommand added.
- HCCOVB keyword added to OUTFILE subcommand. Any combination of COVB, CORB, and HCCOVB keywords can now be specified on OUTFILE.
- MBP, BP, F, and WHITE keywords added to PRINT subcommand.
- MBPDESIGN, BPDESIGN, and FDESIGN subcommands added.

Overview

GLM (general linear model) is a general procedure for analysis of variance and covariance, as well as regression. GLM is the most versatile of the analysis-of-variance procedures and can be used for both univariate and multivariate designs. GLM allows you to:

- Include interaction and nested effects in your design model. Multiple nesting is allowed; for example, *A* within *B* within *C* is specified as *A*(*B*(*C*)).
- Include covariates in your design model. GLM also allows covariate-by-covariate and covariate-by-factor interactions, such as *X* by *X* (or *X***X*), *X* by *A* (or *X***A*), and *X* by *A* within *B* (or *X***A*(*B*)). Thus, polynomial regression or a test of the homogeneity of regressions can be performed.
- Select appropriate sums-of-squares hypothesis tests for effects in balanced design models, unbalanced all-cells-filled design models, and some-cells-empty design models. The estimable functions that correspond to the hypothesis test for each effect in the model can also be displayed.
- Display the general form of estimable functions.
- Display expected mean squares, automatically detecting and using the appropriate error term for testing each effect in mixed-effects and random-effects models.
- Select commonly used contrasts or specify custom contrasts to perform hypothesis tests.
- Customize hypothesis testing, based on the null hypothesis **LBM** = **K**, where **B** is the parameter vector or matrix.

- Display a variety of post hoc tests for multiple comparisons.
- Display estimates of population marginal cell means for both between-subjects factors and within-subjects factors, adjusted for covariates.
- Perform multivariate analysis of variance and covariance.
- Estimate parameters by using the method of weighted least squares and a generalized inverse technique.
- Graphically compare the levels in a model by displaying plots of estimated marginal cell means for each level of a factor, with separate lines for each level of another factor in the model.
- Display a variety of estimates and measures that are useful for diagnostic checking. All of these estimates and measures can be saved in a data file for use by another procedure.
- Perform repeated measures analysis of variance.
- Display homogeneity tests for testing underlying assumptions in multivariate and univariate analyses.

General Linear Model (GLM) and MANOVA

MANOVA, the other generalized procedure for analysis of variance and covariance, is available only in syntax. The major distinction between GLM and MANOVA in terms of statistical design and functionality is that GLM uses a non-full-rank, or overparameterized, indicator variable approach to parameterization of linear models instead of the full-rank reparameterization approach that is used in MANOVA. GLM employs a generalized inverse approach and employs aliasing of redundant parameters to 0. These processes employed by GLM allow greater flexibility in handling a variety of data situations, particularly situations involving empty cells. GLM offers the following features that are unavailable in MANOVA:

- Identification of the general forms of estimable functions.
- Identification of forms of estimable functions that are specific to four types of sums of squares (Types I–IV).
- Tests that use the four types of sums of squares, including Type IV, specifically designed for situations involving empty cells.
- Flexible specification of general comparisons among parameters, using the syntax subcommands LMATRIX, MMATRIX, and KMATRIX; sets of contrasts can be specified that involve any number of orthogonal or nonorthogonal linear combinations.
- Nonorthogonal contrasts for within-subjects factors (using the syntax subcommand WSFACTORS).
- Tests against nonzero null hypotheses, using the syntax subcommand KMATRIX.
- Feature where estimated marginal means (EMMEANS) and standard errors (adjusted for other factors and covariates) are available for all between-subjects and within-subjects factor combinations in the original variable metrics.
- Uncorrected pairwise comparisons among estimated marginal means for any main effect in the model, for both between- and within-subjects factors.
- Feature where post hoc or multiple comparison tests for unadjusted one-way factor means are available for between-subjects factors in ANOVA designs; twenty different types of comparisons are offered.
- Weighted least squares (WLS) estimation, including saving of weighted predicted values and residuals.
- Automatic handling of random effects in random-effects models and mixed models, including generation of expected mean squares and automatic assignment of proper error terms.
- Specification of several types of nested models via dialog boxes with proper use of the interaction operator (*), due to the nonreparameterized approach.
- Univariate homogeneity-of-variance assumption, tested by using the Levene test.
- Between-subjects factors that do not require specification of levels.
- Profile (interaction) plots of estimated marginal means for visual exploration of interactions involving combinations of between-subjects and/or within-subjects factors.

- Saving of casewise temporary variables for model diagnosis: unstandardized (raw) and weighted unstandarized predicted values; unstandardized, weighted unstandardized, standardized, Studentized, and deleted residuals; standard error of prediction, Cook's distance, leverage.
- Saving of a datafile in IBM SPSS Statistics format with parameter estimates and their degrees of freedom and significance level.

To simplify the presentation, GLM reference material is divided into three sections: *univariate* designs with one dependent variable, *multivariate* designs with several interrelated dependent variables, and *repeated measures* designs, in which the dependent variables represent the same types of measurements, taken at more than one time.

The full syntax diagram for GLM is presented here. The following GLM sections include partial syntax diagrams, showing the subcommands and specifications that are discussed in that section. Individually, those diagrams are incomplete. Subcommands that are listed for univariate designs are available for any analysis, and subcommands that are listed for multivariate designs can be used in any multivariate analysis, including repeated measures.

Models

The following examples are models that can be specified by using GLM:

Model 1: Univariate or Multivariate Simple and Multiple Regression

```
GLM Y WITH X1 X2.

GLM Y1 Y2 WITH X1 X2 X3.
```

Model 2: Fixed-effects ANOVA and MANOVA

```
GLM Y1 Y2 BY B.
```

Model 3: ANCOVA and Multivariate ANCOVA (MANCOVA)

```
GLM Y1 Y2 BY B WITH X1 X2 X3.
```

Model 4: Random-effects ANOVA and ANCOVA

```
GLM Y1 BY C WITH X1 X2
 /RANDOM = C.
```

Model 5: Mixed-model ANOVA and ANCOVA

```
GLM Y1 BY B, C WITH X1 X2
 /RANDOM = C.
```

Model 6: Repeated Measures Analysis Using a Split-plot Design

(Univariate mixed models approach with subject as a random effect)

If *drug* is a between-subjects factor and *time* is a within-subjects factor,

```
GLM Y BY DRUG SUBJECT TIME
 /RANDOM = SUBJECT
 /DESIGN = DRUG SUBJECT*DRUG TIME DRUG*TIME.
```

Model 7: Repeated Measures Using the WSFACTOR Subcommand

Use this model only when there is no random between-subjects effect in the model. For example, if *Y1*, *Y2*, *Y3*, and *Y4* are the dependent variables, measured at times 1 to 4,

```
GLM Y1 Y2 Y3 Y4 BY DRUG
 /WSFACTOR = TIME 4
 /DESIGN.
```

Model 8: Repeated Measures Doubly Multivariate Model

Repeated measures fixed-effects MANOVA is also called a doubly multivariate model. Varying or time-dependent covariates are not available. This model can be used only when there is no random between-subjects effect in the model.

```
GLM X11 X12 X13 X21 X22 X23
          Y11 Y12 Y13 Y21 Y22 Y23 BY C D
/MEASURE = X Y
/WSFACTOR = A 2 B 3
/WSDESIGN = A B A*B
/DESIGN = C D.
```

Model 9: Means Model for ANOVA and MANOVA

This model takes only fixed-effect factors (no random effects and covariates) and always assumes the highest order of the interactions among the factors. For example, *B*, *D*, and *E* are fixed factors, and *Y1* and *Y2* are two dependent variables. You can specify a means model by suppressing the intercept effect and specifying the highest order of interaction on the DESIGN subcommand.

```
GLM Y1 Y2 BY B, D, E
/INTERCEPT = EXCLUDE
/DESIGN = B*D*E.
```

Custom Hypothesis Specifications

GLM provides a flexible way to customize hypothesis testing based on the general linear hypothesis **LBM** = **K**, where **B** is the parameter vector or matrix. You can specify a customized linear hypothesis by using one or more of the subcommands LMATRIX, MMATRIX, KMATRIX, and CONTRAST.

LMATRIX, MMATRIX, and KMATRIX Subcommands

- The **L** matrix is called the **contrast coefficients matrix**. This matrix specifies coefficients of contrasts, which can be used for studying the between-subjects effects in the model. One way to define the **L** matrix is by specifying the CONTRAST subcommand, on which you select a type of contrast. Another way is to specify your own **L** matrix directly by using the LMATRIX subcommand. See the topic "LMATRIX Subcommand " on page 853 for more information.
- The **M** matrix is called the **transformation coefficients matrix**. This matrix provides a transformation for the dependent variables. This transformation can be used to construct contrasts among the dependent variables in the model. The **M** matrix can be specified on the MMATRIX subcommand. See the topic "MMATRIX Subcommand " on page 867 for more information.
- The **K** matrix is called the **contrast results matrix**. This matrix specifies the results matrix in the general linear hypothesis. To define your own **K** matrix, use the KMATRIX subcommand. See the topic "KMATRIX Subcommand " on page 854 for more information.

For univariate and multivariate models, you can specify one, two, or all three of the **L**, **M**, and **K** matrices. If only one or two types are specified, the unspecified matrices use the defaults that are shown in the following table (read across the rows).

Table 59. Default matrices for univariate and multivariate models if one matrix is specified

L matrix	**M matrix**	**K matrix**
If LMATRIX is used to specify the **L** matrix	Default = identity matrix*	Default = zero matrix
Default = intercept matrix†	If MMATRIX is used to specify the **M** matrix	Default = zero matrix
Default = intercept matrix†	Default = identity matrix*	If KMATRIX is used to specify the **K** matrix

* The dimension of the identity matrix is the same as the number of dependent variables that are being studied.

† The intercept matrix is the matrix that corresponds to the estimable function for the intercept term in the model, provided that the intercept term is included in the model. If the intercept term is not included in the model, the **L** matrix is not defined, and this custom hypothesis test cannot be performed.

Example

```
GLM Y1 Y2 BY A B
    /LMATRIX = A 1 -1
    /DESIGN A B.
```

Assume that factor *A* has two levels.

- Because there are two dependent variables, this model is a multivariate model with two main factor effects, *A* and *B*.
- A custom hypothesis test is requested by the LMATRIX subcommand.
- Because no MMATRIX or KMATRIX is specified, the **M** matrix is the default two-dimensional identity matrix, and the **K** matrix is a zero-row vector (0, 0).

For a repeated measures model, you can specify one, two, or all three of the **L**, **M**, and **K** matrices. If only one or two types are specified, the unspecified matrices use the defaults that are shown in the following table (read across the rows).

Table 60. Default matrices for repeated measures models if only one matrix is specified

L matrix	**M matrix**	**K matrix**
If LMATRIX is used to specify the **L** matrix	Default = average matrix*	Default = zero matrix
Default = intercept matrix†	If MMATRIX is used to specify the **M** matrix	Default = zero matrix
Default = intercept matrix†	Default = average matrix*	If KMATRIX is used to specify the **K** matrix

* The average matrix is the transformation matrix that corresponds to the transformation for the between-subjects test. The dimension is the number of measures.

† The intercept matrix is the matrix that corresponds to the estimable function for the intercept term in the model, provided that the intercept term is included in the model. If the intercept term is not included in the model, the **L** matrix is not defined, and this custom hypothesis test cannot be performed.

Example

```
GLM Y1 Y2 BY A B
 /WSFACTOR TIME (2)
 /MMATRIX Y1 1 Y2 1; Y1 1 Y2 -1
 /DESIGN A B.
```

- Because WSFACTOR is specified, this model is a repeated measures model with two between-subjects factors *A* and *B*, and a within-subjects factor, *TIME*.
- A custom hypothesis is requested by the MMATRIX subcommand. The **M** matrix is a 2 x 2 matrix:

```
1  1
1 -1
```

- Because the **L** matrix and **K** matrix are not specified, their defaults are used. The default for the **L** matrix is the matrix that corresponds to the estimable function for the intercept term in the between-subjects model, and the default for the **K** matrix is a zero-row vector (0, 0).

CONTRAST Subcommand

When the CONTRAST subcommand is used, an **L** matrix, which is used in custom hypothesis testing, is generated according to the chosen contrast. The **K** matrix is always taken to be the zero matrix. If the model is univariate or multivariate, the **M** matrix is always the identity matrix, and its dimension is equal

to the number of dependent variables. For a repeated measures model, the **M** matrix is always the average matrix that corresponds to the average transformation for the dependent variable.

See the topic “CONTRAST Subcommand ” on page 855 for more information.

GLM: Univariate

GLM is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
GLM dependent var [BY factor list [WITH covariate list]]

[/RANDOM=factor factor...]

[/REGWGT=varname]

[/METHOD=SSTYPE({1  })]
               {2  }
               {3**}
               {4  }

[/INTERCEPT=[INCLUDE**] [EXCLUDE]]

[/MISSING=[INCLUDE] [EXCLUDE**]]

[/CRITERIA=[EPS({1E-8**})][ALPHA({0.05**})]
               {a     }         {a     }

[/PRINT = [DESCRIPTIVE] [HOMOGENEITY] [PARAMETER][ETASQ]
          [GEF] [LOF] [OPOWER] [TEST(LMATRIX)]
          [MBP] [BP] [F] [WHITE] ]

[/PLOT=[SPREADLEVEL] [RESIDUALS]
       [PROFILE (factor factor*factor factor*factor*factor ...)]
               [WITH(covariate={value} [...])]
                             {MEAN }
               [TYPE={LINE**}]
                     {BAR   }
               [MEANREFERENCE={NO**}]
                              {YES }
               [ERRORBAR={NO**      }]
                         {CI        }
                         {SE [(2**)]}
               [YAXIS={AUTO**}]
                      {0     }

[/TEST=effect VS {linear combination [DF(df)]}]
                 {value DF (df)              }

[/LMATRIX={["label"] effect list effect list ...;...}]
          {["label"] effect list effect list ...    }
          {["label"] ALL list; ALL...               }
          {["label"] ALL list                       }

[/KMATRIX= {number    }]
           {number;...}

[/CONTRAST (factor name)={DEVIATION[(refcat)]**     }]
                         {SIMPLE [(refcat)]         }
                         {DIFFERENCE                }
                         {HELMERT                   }
                         {REPEATED                  }
                         {POLYNOMIAL  [({1,2,3...})]}
                                      {metric  }
                         {SPECIAL (matrix)          }

[/POSTHOC =effect [effect...]
          ([SNK] [TUKEY] [BTUKEY][DUNCAN]
           [SCHEFFE] [DUNNETT(refcat)] [DUNNETTL(refcat)]
           [DUNNETTR(refcat)] [BONFERRONI] [LSD] [SIDAK]
           [GT2] [GABRIEL] [FREGW] [QREGW]  [T2] [T3] [GH] [C]
           [WALLER ({100** })])]
                   {kratio}
           [VS effect]

[/EMMEANS=TABLES({OVERALL         }) [WITH(covariate={value} [...])]]
                 {factor          }                  {MEAN }
                 {factor*factor...}

          [COMPARE ADJ(LSD) (BONFERRONI) (SIDAK)]

[/ROBUST={HC0  })]
         {HC1  }
         {HC2  }
         {HC3**}
         {HC4  }

[/SAVE=[tempvar [(name)]] [tempvar [(name)]]...]


[/OUTFILE=[COVB ('savfile'|'dataset')]
          [CORB ('savfile'|'dataset')]
          [HCCOVB ('savfile'|'dataset')]
          [EFFECT('savfile'|'dataset')] [DESIGN('savfile'|'dataset')]

[/MBPDESIGN=[effect effect...]]

[/BPDESIGN=[effect effect...]]

[/FDESIGN=[effect effect...]]

[/DESIGN={[INTERCEPT...]    }]
         {[effect effect...]}
```

** Default if the subcommand or keyword is omitted.

Temporary variables (tempvar) are:

```
PRED, WPRED, RESID, WRESID, DRESID, ZRESID, SRESID, SEPRED, COOK, LEVER
```

Example

```
GLM YIELD BY SEED FERT
  /DESIGN.
```

Overview

This section describes the use of GLM for univariate analyses. However, most of the subcommands that are described here can be used in any type of analysis with GLM. For additional subcommands that are used in multivariate analysis, see “GLM: Multivariate” on page 865. For additional subcommands that are used in repeated measures analysis, see “GLM: Repeated Measures” on page 869. For basic specification, syntax rules, and limitations of the GLM procedures, see “GLM” on page 837.

Options

Design Specification. You can use the DESIGN subcommand to specify which terms to include in the design. This allows you to estimate a model other than the default full factorial model, incorporate factor-by-covariate interactions or covariate-by-covariate interactions, and indicate nesting of effects.

Contrast Types. You can specify contrasts other than the default deviation contrasts on the CONTRAST subcommand.

Optional Output. You can choose from a variety of optional output on the PRINT subcommand. Output that is appropriate to univariate designs includes descriptive statistics for each cell, parameter estimates, Levene tests for equality of variance across cells, tests for heteroskedasticity, partial eta-squared for each effect and each parameter estimate, the general estimable function(s) matrix, and a contrast coefficients table (**L'** matrix). The OUTFILE subcommand allows you to write out the covariance or correlation matrix, the design matrix, or the statistics from the between-subjects ANOVA table into a separate data file.

Using the EMMEANS subcommand, you can request tables of estimated marginal means of the dependent variable and their standard deviations. The SAVE subcommand allows you to save predicted values and residuals in weighted or unweighted and standardized or unstandardized forms. You can use the POSTHOC subcommand to specify different means comparison tests for comparing all possible pairs of cell means. In addition, you can specify your own hypothesis tests by specifying an **L** matrix and a **K** matrix to test the univariate hypothesis **LB** = **K**.

You can display robust or heteroskedasticity-consistent (HC) standard errors by using the ROBUST subcommand, and you can write the robust covariance matrix estimates to a new file or dataset. You can also display a second set of output for custom hypothesis tests, with results based on the specified robust covariance matrix estimator.

Basic Specification

- The basic specification is a variable list identifying the dependent variable, the factors (if any), and the covariates (if any).
- By default, GLM uses a model that includes the intercept term, the covariate (if any), and the full factorial model, which includes all main effects and all possible interactions among factors. The intercept term is excluded if it is excluded in the model by specifying the keyword EXCLUDE on the INTERCEPT subcommand. Sums of squares are calculated and hypothesis tests are performed by using type-specific estimable functions. Parameters are estimated by using the normal equation and a generalized inverse of the SSCP matrix.

Subcommand Order

- The variable list must be specified first.
- Subcommands can be used in any order.

Syntax Rules

- For many analyses, the GLM variable list and the DESIGN subcommand are the only specifications that are needed.
- If you do not enter a DESIGN subcommand, GLM uses a full factorial model, with main effects of covariates, if any.
- At least one dependent variable must be specified, and at least one of the following specifications must occur: INTERCEPT, a between-subjects factor, or a covariate. The design contains the intercept by default.
- If more than one DESIGN subcommand is specified, only the last subcommand is in effect.
- Dependent variables and covariates must be numeric, but factors can be numeric or string variables.
- If more than one MISSING subcommand is specified, only the last subcommand is in effect.
- If more than one ROBUST subcommand is specified, only the last one is in effect.
- The following words are reserved as keywords or internal commands in the GLM procedure:

```
INTERCEPT, BY, WITH, ALL, OVERALL, WITHIN
```

Variable names that duplicate these words should be changed before you run GLM.

Limitations

- Any number of factors can be specified, but if the number of between-subjects factors plus the number of split variables exceeds 18, the Descriptive Statistics table is not printed even when you request it.
- Memory requirements depend primarily on the number of cells in the design. For the default full factorial model, this equals the product of the number of levels or categories in each factor.

Example

```
GLM YIELD BY SEED FERT WITH RAINFALL
  /PRINT=DESCRIPTIVE PARAMETER
  /DESIGN.
```

- *YIELD* is the dependent variable; *SEED* and *FERT* are factors; *RAINFALL* is a covariate.
- The PRINT subcommand requests the descriptive statistics for the dependent variable for each cell and the parameter estimates, in addition to the default tables Between-Subjects Factors and Univariate Tests.
- The DESIGN subcommand requests the default design (a full factorial model with a covariate). This subcommand could have been omitted or could have been specified in full as

```
/DESIGN = INTERCEPT RAINFALL, SEED, FERT, SEED BY FERT.
```

GLM Variable List

The variable list specifies the dependent variable, the factors, and the covariates in the model.

- The dependent variable must be the first specification on GLM.
- The names of the factors follow the dependent variable. Use the keyword BY to separate the factors from the dependent variable.
- Enter the covariates, if any, following the factors. Use the keyword WITH to separate covariates from factors (if any) and the dependent variable.

Example

```
GLM DEPENDNT BY FACTOR1 FACTOR2, FACTOR3.
```

- In this example, three factors are specified.

- A default full factorial model is used for the analysis.

Example

```
GLM Y BY A WITH X
  /DESIGN.
```

- In this example, the DESIGN subcommand requests the default design, which includes the intercept term, the covariate *X*, and the factor *A*.

RANDOM Subcommand

RANDOM allows you to specify which effects in your design are random. When the RANDOM subcommand is used, a table of expected mean squares for all effects in the design is displayed, and an appropriate error term for testing each effect is calculated and used automatically.

- Random always implies a univariate mixed-model analysis.
- If you specify an effect on RANDOM, higher-order effects containing the specified effect (excluding any effects containing covariates) are automatically treated as random effects.
- The keyword INTERCEPT and effects containing covariates are not allowed on this subcommand.
- The RANDOM subcommand cannot be used if there is any within-subjects factor in the model (that is, RANDOM cannot be specified if WSFACTOR is specified).
- When the RANDOM subcommand is used, the appropriate error terms for the hypothesis testing of all effects in the model are automatically computed and used.
- More than one RANDOM subcommand is allowed. The specifications are accumulated.

Example

```
GLM DEP BY A B
  /RANDOM = B
  /DESIGN = A,B, A*B.
```

- In the example, effects *B* and *A*B* are considered as random effects. If only effect *B* is specified in the RANDOM subcommand, *A*B* is automatically considered as a random effect.
- The hypothesis testing for each effect in the design (*A*, *B*, and *A*B*) will be carried out by using the appropriate error term, which is calculated automatically.

REGWGT Subcommand

The only specification on REGWGT is the name of the variable containing the weights to be used in estimating a weighted least-squares model.

- Specify a numeric weight variable name following the REGWGT subcommand. Only observations with positive values in the weight variable will be used in the analysis.
- If more than one REGWGT subcommand is specified, only the last subcommand is in effect.

Example

```
GLM OUTCOME BY TREATMNT
  /REGWGT WT.
```

- The procedure performs a weighted least-squares analysis. The variable *WT* is used as the weight variable.

METHOD Subcommand

METHOD controls the computational aspects of the GLM analysis. You can specify one of four different methods for partitioning the sums of squares. If more than one METHOD subcommand is specified, only the last subcommand is in effect.

SSTYPE(1). *Type I sum-of-squares method.* The Type I sum-of-squares method is also known as the hierarchical decomposition of the sum-of-squares method. Each term is adjusted only for the terms that precede it on the DESIGN subcommand. Under a balanced design, it is an orthogonal decomposition, and the sums of squares in the model add up to the total sum of squares.

SSTYPE(2). *Type II sum-of-squares method.* This method calculates the sum of squares of an effect in the model, adjusted for all other "appropriate" effects. An appropriate effect is an effect that corresponds to all effects that do not contain the effect that is being examined. For any two effects *F1* and *F2* in the model, *F1* is **contained** in *F2* under the following three conditions: Both effects *F1* and *F2* have the same covariate (if any), *F2* consists of more factors than *F1*, or all factors in *F1* also appear in *F2*. The intercept effect is treated as contained in all the pure factor effects. However, the intercept effect is not contained in any effect involving a covariate. No effect is contained in the intercept effect. Thus, for any one effect *F* of interest, all other effects in the model can be classified as being in one of the following two groups: the effects that do not contain *F* or the effects that contain *F*. If the model is a main-effects design (that is, only main effects are in the model), the Type II sum-of-squares method is equivalent to the regression approach sums of squares, meaning that each main effect is adjusted for every other term in the model.

SSTYPE(3). *Type III sum-of-squares method.* This setting is the default. This method calculates the sum of squares of an effect *F* in the design as the sum of squares adjusted for any other effects that do not contain it, and *orthogonal* to any effects (if any) that contain it. The Type III sums of squares have one major advantage—they are invariant with respect to the cell frequencies as long as the general form of estimability remains constant. Hence, this type of sums of squares is often used for an unbalanced model with no missing cells. In a factorial design with no missing cells, this method is equivalent to the Yates' weighted squares of means technique, and it also coincides with the overparameterized ∑-restricted model.

SSTYPE(4). *Type IV sum-of-squares method.* This method is designed for a situation in which there are missing cells. For any effect *F* in the design, if *F* is not contained in any other effect, then Type IV = Type III = Type II. When *F* is contained in other effects, Type IV equitably distributes the contrasts being made among the parameters in *F* to all higher-level effects.

Example

```
GLM DEP BY A B C
  /METHOD=SSTYPE(3)
  /DESIGN=A, B, C.
```

- The design is a main-effects model.
- The METHOD subcommand requests that the model be fitted with Type III sums of squares.

INTERCEPT Subcommand

INTERCEPT controls whether an intercept term is included in the model. If more than one INTERCEPT subcommand is specified, only the last subcommand is in effect.

INCLUDE. *Include the intercept term.* The intercept (constant) term is included in the model. This setting is the default.

EXCLUDE. *Exclude the intercept term.* The intercept term is excluded from the model. Specification of the keyword INTERCEPT on the DESIGN subcommand overrides INTERCEPT = EXCLUDE.

MISSING Subcommand

By default, cases with missing values for any of the variables on the GLM variable list are excluded from the analysis. The MISSING subcommand allows you to include cases with user-missing values.

- If MISSING is not specified, the default is EXCLUDE.
- Pairwise deletion of missing data is not available in GLM.
- Keywords INCLUDE and EXCLUDE are mutually exclusive.
- If more than one MISSING subcommand is specified, only the last subcommand is in effect.

EXCLUDE. *Exclude both user-missing and system-missing values.* This setting is the default when MISSING is not specified.

INCLUDE. *Treat user-missing values as valid.* System-missing values cannot be included in the analysis.

CRITERIA Subcommand

CRITERIA controls the statistical criteria used to build the models.

- More than one CRITERIA subcommand is allowed. The specifications are accumulated. Conflicts across CRITERIA subcommands are resolved by using the conflicting specification that was given on the last CRITERIA subcommand.
- The keyword must be followed by a positive number in parentheses.

EPS(n). *The tolerance level in redundancy detection.* This value is used for redundancy checking in the design matrix. The default value is 1E-8.

ALPHA(n). *The alpha level.* This keyword has two functions. First, the keyword gives the alpha level at which the power is calculated for the *F* test. After the noncentrality parameter for the alternative hypothesis is estimated from the data, the power is the probability that the test statistic is greater than the critical value under the alternative hypothesis. (The observed power is displayed by default for GLM.) The second function of alpha is to specify the level of the confidence interval. If the specified alpha level is *n*, the value (1–*n*)×100 indicates the level of confidence for all individual and simultaneous confidence intervals that are generated for the specified model. The value of *n* must be between 0 and 1, exclusive. The default value of alpha is 0.05, which means that the default power calculation is at the 0.05 level, and the default level of the confidence intervals is 95%, because (1–0.05)×100=95.

PRINT Subcommand

PRINT controls the display of optional output.

- Some PRINT output applies to the entire GLM procedure and is displayed only once.
- Additional output can be obtained on the EMMEANS, PLOT, and SAVE subcommands.
- Some optional output may greatly increase the processing time. Request only the output that you want to see.
- If no PRINT command is specified, default output for a univariate analysis includes a factor information table and a Univariate Tests table (ANOVA) for all effects in the model.
- If more than one PRINT subcommand is specified, only the last subcommand is in effect.

The following keywords are available for GLM univariate analyses. For information about PRINT specifications that are appropriate for other GLM models, see *GLM: Multivariate* and *GLM: Repeated Measures*.

DESCRIPTIVES. *Basic information about each cell in the design.* This process determines observed means, standard deviations, and counts for the dependent variable in all cells. The cells are constructed from the highest-order crossing of the between-subjects factors. For a multivariate model, statistics are

given for each dependent variable. If the number of between-subjects factors plus the number of split variables exceeds 18, the Descriptive Statistics table is not printed.

HOMOGENEITY. *Tests of homogeneity of variance.* Levene tests for equality of variances for the dependent variable across all level combinations of the between-subjects factors. If there are no between-subjects factors, this keyword is not valid. For a multivariate model, tests are displayed for each dependent variable.

PARAMETER. *Parameter estimates.* Parameter estimates, standard errors, *t* tests, and confidence intervals.

ETASQ. *Partial eta-squared* (η^2). This value is an overestimate of the actual effect size in an *F* test.

GEF. *General estimable function table.* This table shows the general form of the estimable functions.

LOF. *Instruction to perform a lack-of-fit test (which requires at least one cell to have multiple observations).* If the test is rejected, it implies that the current model cannot adequately account for the relationship between the response variable and the predictors. Either a variable is omitted or extra terms are needed in the model.

OPOWER. *Observed power for each test.* The observed power gives the probability that the *F* test would detect a population difference between groups that is equal to the difference that is implied by the sample difference.

TEST(LMATRIX). *Set of contrast coefficients (L) matrices.* The transpose of the **L** matrix (**L'**) is displayed. This set always includes one matrix displaying the estimable function for each between-subjects effect that appears or is implied in the DESIGN subcommand. Also, any **L** matrices generated by the LMATRIX or CONTRAST subcommands are displayed. TEST(ESTIMABLE) can be used in place of TEST(LMATRIX).

MBP. *Modified Breusch-Pagan test for heteroskedasticity.* Tests the null hypothesis that the variance of the errors does not depend on the values of the independent variables. You can use the MBPDESIGN subcommand to specify the model on which the test is based. If the MBP keyword is specified without MBPDESIGN, then the model consists of a constant term, a term that is linear in the predicted values, a term that is quadratic in the predicted values, and an error term. For a multivariate model, tests are displayed for each dependent variable.

BP. *Breusch-Pagan test for heteroskedasticity.* Tests the null hypothesis that the variance of the errors does not depend on the values of the independent variables. You can use the BPDESIGN subcommand to specify the model on which the test is based. If the BP keyword is specified without BPDESIGN, then the model consists of a constant term, a term that is linear in the predicted values, a term that is quadratic in the predicted values, and an error term. For a multivariate model, tests are displayed for each dependent variable.

F. *F test for heteroskedasticity.* Tests the null hypothesis that the variance of the errors does not depend on the values of the independent variables. You can use the FDESIGN subcommand to specify the model on which the test is based. If the F keyword is specified without FDESIGN, then the model consists of a constant term, a term that is linear in the predicted values, a term that is quadratic in the predicted values, and an error term. For a multivariate model, tests are displayed for each dependent variable.

WHITE. *White's test for heteroskedasticity.* Tests the null hypothesis that the variance of the errors does not depend on the values of the independent variables. For a multivariate model, tests are displayed for each dependent variable.

Example

```
GLM DEP BY A B WITH COV
  /PRINT=DESCRIPTIVE, TEST(LMATRIX), PARAMETER
  /DESIGN.
```

- Because the design in the DESIGN subcommand is not specified, the default design is used. In this case, the design includes the intercept term, the covariate *COV*, and the full factorial terms of *A* and *B*, which are *A*, *B*, and *A*B*.
- For each combination of levels of *A* and *B*, the descriptive statistics of *DEP* are displayed.

- The set of L matrices that generates the sums of squares for testing each effect in the design is displayed.
- The parameter estimates, their standard errors, *t* tests, confidence intervals, and the observed power for each test are displayed.

PLOT Subcommand

PLOT provides a variety of plots that are useful in checking the assumptions that are needed in the analysis. The PLOT subcommand can be specified more than once. All of the plots that are requested on each PLOT subcommand are produced.

Use the following keywords on the PLOT subcommand to request plots:

SPREADLEVEL
Spread-versus-level plots. Plots of observed cell means versus standard deviations, and versus variances.

RESIDUALS
Observed by predicted by standardized residuals plot. A plot is produced for each dependent variable. In a univariate analysis, a plot is produced for the single dependent variable.

PROFILE
Plot of dependent variable means for one-way, two-way, or three-way crossed factors. The PROFILE keyword must be followed by parentheses containing a list of one or more factor combinations. All factors specified (either individual or crossed) must be made up of only valid factors on the factor list. Factor combinations on the PROFILE keyword may use an asterisk (*) or the keyword BY to specify crossed factors. A factor cannot occur in a single factor combination more than once.

- The order of factors in a factor combination is important, and there is no restriction on the order of factors. If a single factor is specified after the PROFILE keyword, a line plot of estimated means at each level of the factor is produced. If a two-way crossed factor combination is specified, the output includes a multiple-line plot of estimated means at each level of the first specified factor, with a separate line drawn for each level of the second specified factor. If a three-way crossed factor combination is specified, the output includes multiple-line plots of estimated means at each level of the first specified factor, with separate lines for each level of the second factor, and separate plots for each level of the third factor.
- The PROFILE keyword may be followed by the WITH keyword and a parenthesized list of covariates and values. Plots are produced with the values of the covariates fixed at the specified values. The keyword MEAN causes the covariate to be fixed at its observed mean value.

TYPE=LINE | BAR
Type of plot for profile plots. The default type is LINE.

ERRORBAR=NO | CI | SE (value)
Include error bars in the profile plot. The default setting is NO.

CI
Error bars display the confidence interval. The confidence interval is determined by the ALPHA value on the CRITERIA subcommand. The confidence interval is 1-alpha. For example, if the ALPHA value is 0.05, the confidence interval is 95%.

SE (value)
Error bars display standard errors. The default value is 2. The value must be a positive number.

Example

```
GLM DEP BY A B
  /PLOT = SPREADLEVEL PROFILE(A A*B A*B*C) WITH(X=1.0)
  /DESIGN.
```

Assume that each of the factors *A*, *B*, and *C* has three levels.

- Spread-versus-level plots are produced, showing observed cell means versus standard deviations and observed cell means versus variances.
- Five profile plots are produced. For factor *A*, a line plot of estimated means at each level of *A* is produced (one plot). For the two-way crossed factor combination *A*B*, a multiple-line plot of estimated means at each level of *A*is produced (one plot), with a separate line for each level of *B*. For the three-way crossed factor combination *A*B*C*, a multiple-line plot of estimated means at each level of *A* is produced for each of the three levels of *C* (three plots), with a separate line for each level of *B*. Plots are produced with the value of the covariate *X* fixed at 1.0.

TEST Subcommand

The TEST subcommand allows you to test a hypothesis term against a specified error term.

- TEST is valid only for univariate analyses. Multiple TEST subcommands are allowed, with each subcommand being executed independently.
- You must specify both the hypothesis term and the error term. There is no default.
- The hypothesis term is specified before the keyword VS and must be a valid effect that is specified or implied on the DESIGN subcommand.
- The error term is specified after the keyword VS. You can specify either a linear combination or a value. The linear combination of effects takes the general form: `coefficient*effect +/– coefficient*effect ...`
- All effects in the linear combination must be specified or implied on the DESIGN subcommand. Effects that are specified or implied on DESIGN but not listed after VS are assumed to have a coefficient of 0.
- Duplicate effects are allowed. GLM adds coefficients associated with the same effect before performing the test. For example, the linear combination 5*A–0.9*B–A is combined to 4*A–0.9B.
- A coefficient can be specified as a fraction with a positive denominator (for example, 1/3 or –1/3 are valid, but 1/–3 is invalid).
- If you specify a value for the error term, you must specify the degrees of freedom after the keyword DF. The degrees of freedom must be a positive real number. DF and the degrees of freedom are optional for a linear combination.

Example

```
GLM DEP BY A B
  /TEST = A VS B + A*B
  /DESIGN = A, B, A*B.
```

- *A* is tested against the pooled effect of *B + A*B*.

LMATRIX Subcommand

The LMATRIX subcommand allows you to customize your hypotheses tests by specifying the **L** matrix (contrast coefficients matrix) in the general form of the linear hypothesis **LB** = **K**, where **K** = **0** if it is not specified on the KMATRIX subcommand. The vector **B** is the parameter vector in the linear model.

- The basic format for the LMATRIX subcommand is an optional label in quotation marks, one or more effect names or the keyword ALL, and one or more lists of real numbers.
- The optional label is a string with a maximum length of 255 bytes. Only one label can be specified.
- Only valid effects that appear or are implied on the DESIGN subcommand can be specified on the LMATRIX subcommand.
- The length of the list of real numbers must be equal to the number of parameters (including the redundant parameters) corresponding to that effect. For example, if the effect *A*B* uses six columns in the design matrix, the list after *A*B* must contain exactly six numbers.

- A number can be specified as a fraction with a positive denominator (for example, 1/3 or –1/3 are valid, but 1/–3 is invalid).
- A semicolon (;) indicates the end of a row in the **L** matrix.
- When ALL is specified, the length of the list that follows ALL is equal to the total number of parameters (including the redundant parameters) in the model.
- Effects that appear or are implied on the DESIGN subcommand must be explicitly specified here.
- If LMATRIX is specified along with the ROBUST subcommand, then contrast estimates and tests are calculated with the robust covariance matrix, in addition to the tests that use the standard (or model-based) covariance matrix.
- Multiple LMATRIX subcommands are allowed. Each subcommand is treated independently.

Example

```
GLM DEP BY A B
  /LMATRIX = "B1 vs B2 at A1"
              B 1 -1 0 A*B 1 -1 0 0 0 0 0 0 0
  /LMATRIX = "Effect A"
              A 1 0 -1
              A*B 1/3  1/3  1/3
                   0    0    0
                 -1/3 -1/3 -1/3;
              A 0 1 -1
              A*B  0    0    0
                  1/3  1/3  1/3
                 -1/3 -1/3 -1/3
  /LMATRIX = "B1 vs B2 at A2"
             ALL  0
                  0  0  0
                  1 -1  0
                  0  0  0  1 -1  0 0  0  0
  /DESIGN = A, B, A*B.
```

Assume that factors *A* and *B* each have three levels. There are three LMATRIX subcommands; each subcommand is treated independently.

- **B1 Versus B2 at A1.** In the first LMATRIX subcommand, the difference is tested between levels 1 and 2 of effect *B* when effect *A* is fixed at level 1. Because there are three levels each in effects *A* and *B*, the interaction effect *A*B* should use nine columns in the design matrix.
- **Effect A.** In the second LMATRIX subcommand, effect *A* is tested. Because there are three levels in effect *A*, no more than two independent contrasts can be formed; thus, there are two rows in the **L** matrix, which are separated by a semicolon (;). The first row tests the difference between levels 1 and 3 of effect *A*, while the second row tests the difference between levels 2 and 3 of effect *A*.
- **B1 Versus B2 at A2.** In the last LMATRIX subcommand, the keyword ALL is used. The first 0 corresponds to the intercept effect; the next three instances of 0 correspond to effect *A*.

KMATRIX Subcommand

The KMATRIX subcommand allows you to customize your hypothesis tests by specifying the **K** matrix (contrast results matrix) in the general form of the linear hypothesis **LB** = **K**. The vector **B** is the parameter vector in the linear model.

- The default **K** matrix is a zero matrix; that is, **LB** = **0** is assumed.
- For the KMATRIX subcommand to be valid, at least one of the following subcommands must be specified: the LMATRIX subcommand or the INTERCEPT = INCLUDE subcommand.
- If KMATRIX is specified but LMATRIX is not specified, the LMATRIX is assumed to take the row vector corresponding to the intercept in the estimable function, provided that the subcommand INTERCEPT = INCLUDE is specified. In this case, the **K** matrix can be only a scalar matrix.
- If KMATRIX and LMATRIX are specified, the number of rows in the requested **K** and **L** matrices must be equal. If there are multiple LMATRIX subcommands, all requested **L** matrices must have the same number of rows, and **K** must have the same number of rows as these **L** matrices.
- A semicolon (;) can be used to indicate the end of a row in the **K** matrix.
- If more than one KMATRIX subcommand is specified, only the last subcommand is in effect.

Example

```
GLM DEP BY A B
  /LMATRIX = “Effect A”
             A 1  0 -1; A 1 -1  0
  /LMATRIX = “Effect B”
             B 1  0 -1; B 1 -1  0
  /KMATRIX = 0; 0
  /DESIGN = A B.
```

In this example, assume that factors *A* and *B* each have three levels.

- There are two LMATRIX subcommands; both subcommands have two rows.
- The first LMATRIX subcommand tests whether the effect of *A* is 0, while the second LMATRIX subcommand tests whether the effect of *B* is 0.
- The KMATRIX subcommand specifies that the **K** matrix also has two rows, each row with value 0.

CONTRAST Subcommand

CONTRAST specifies the type of contrast that is desired among the levels of a factor. For a factor with *k* levels or values, the contrast type determines the meaning of its *k*–1 degrees of freedom.

- Specify the factor name in parentheses following the subcommand CONTRAST.
- You can specify only one factor per CONTRAST subcommand, but you can enter multiple CONTRAST subcommands.
- After closing the parentheses, enter an equals sign followed by one of the contrast keywords.
- This subcommand creates an **L** matrix where the columns corresponding to the factor match the contrast that is given. The other columns are adjusted so that the **L** matrix is estimable.

The following contrast types are available:

DEVIATION. *Deviations from the grand mean.* This setting is the default for between-subjects factors. Each level of the factor except one is compared to the grand mean. One category (by default, the last category) must be omitted so that the effects will be independent of one another. To omit a category other than the last category, specify the number of the omitted category (which is not necessarily the same as its value) in parentheses after the keyword DEVIATION. An example is as follows:

```
GLM Y BY B
  /CONTRAST(B)=DEVIATION(1).
```

Suppose factor *B* has three levels, with values 2, 4, and 6. The specified contrast omits the first category, in which *B* has the value 2. Deviation contrasts are not orthogonal.

POLYNOMIAL. *Polynomial contrasts.* This setting is the default for within-subjects factors. The first degree of freedom contains the linear effect across the levels of the factor, the second degree of freedom contains the quadratic effect, and so on. In a balanced design, polynomial contrasts are orthogonal. By default, the levels are assumed to be equally spaced; you can specify unequal spacing by entering a metric consisting of one integer for each level of the factor in parentheses after the keyword POLYNOMIAL. (All metrics that are specified cannot be equal; thus, (1, 1, . . . 1) is not valid.) An example is as follows:

```
GLM RESPONSE BY STIMULUS
  /CONTRAST(STIMULUS) = POLYNOMIAL(1,2,4)
```

Suppose that factor *STIMULUS* has three levels. The specified contrast indicates that the three levels of *STIMULUS* are actually in the proportion 1:2:4. The default metric is always (1, 2, . . . *k*), where *k* levels are involved. Only the relative differences between the terms of the metric matter. (1, 2, 4) is the same metric as (2, 3, 5) or (20, 30, 50) because, in each instance, the difference between the second and third numbers is twice the difference between the first and second.

DIFFERENCE. *Difference or reverse Helmert contrasts.* Each level of the factor (except the first level) is compared to the mean of the previous levels. In a balanced design, difference contrasts are orthogonal.

HELMERT. *Helmert contrasts.* Each level of the factor (except the last level) is compared to the mean of subsequent levels. In a balanced design, Helmert contrasts are orthogonal.

SIMPLE. *Contrast where each level of the factor (except the last level) is compared to the last level.* To use a category other than the last category as the omitted reference category, specify the category's number (which is not necessarily the same as its value) in parentheses following the keyword SIMPLE. An example is as follows:

```
GLM Y BY B
  /CONTRAST(B)=SIMPLE(1).
```

Suppose that factor *B* has three levels with values 2, 4, and 6. The specified contrast compares the other levels to the first level of *B*, in which *B* has the value 2. Simple contrasts are not orthogonal.

REPEATED. *Comparison of adjacent levels.* Each level of the factor (except the last level) is compared to the next level. Repeated contrasts are not orthogonal.

SPECIAL. *A user-defined contrast.* Values that are specified after this keyword are stored in a matrix in column major order. For example, if factor *A* has three levels, then `CONTRAST(A)= SPECIAL(1 1 1 1 -1 0 0 1 -1)` produces the following contrast matrix:

```
1   1   0
1  -1   1
1   0  -1
```

Note that unlike the LMATRIX subcommand, the CONTRAST subcommand does not accept fractions.

Orthogonal contrasts are particularly useful. In a balanced design, contrasts are orthogonal if the sum of the coefficients in each contrast row is 0 and if, for any pair of contrast rows, the products of corresponding coefficients sum to 0. DIFFERENCE, HELMERT, and POLYNOMIAL contrasts always meet these criteria in balanced designs.

Example

```
GLM DEP BY FAC
  /CONTRAST(FAC)=DIFFERENCE
  /DESIGN.
```

- Suppose that the factor *FAC* has five categories and, therefore, has four degrees of freedom.
- CONTRAST requests DIFFERENCE contrasts, which compare each level (except the first level) with the mean of the previous levels.

POSTHOC Subcommand

POSTHOC allows you to produce multiple comparisons between means of a factor. These comparisons are usually not planned at the beginning of the study but are suggested by the data during the course of study.

- Post hoc tests are computed for the dependent variable. The alpha value that is used in the tests can be specified by using the keyword ALPHA on the CRITERIA subcommand. The default alpha value is 0.05. The confidence level for any confidence interval that is constructed is (1–α)×100. The default confidence level is 95. For a multivariate model, tests are computed for all specified dependent variables.
- Only between-subjects factors that appear in the factor list are valid in this subcommand. Individual factors can be specified.
- You can specify one or more effects to be tested. Only fixed main effects that appear or are implied on the DESIGN subcommand are valid test effects.
- Optionally, you can specify an effect defining the error term following the keyword VS after the test specification. The error effect can be any single effect in the design that is not the intercept or a main effect that is named on a POSTHOC subcommand.

- A variety of multiple comparison tests are available. Some tests are designed for detecting homogeneity subsets among the groups of means, some tests are designed for pairwise comparisons among all means, and some tests can be used for both purposes.
- For tests that are used for detecting homogeneity subsets of means, non-empty group means are sorted in ascending order. Means that are not significantly different are included together to form a homogeneity subset. The significance for each homogeneity subset of means is displayed. In a case where the numbers of valid cases are not equal in all groups, for most post hoc tests, the harmonic mean of the group sizes is used as the sample size in the calculation. For QREGW or FREGW, individual sample sizes are used.
- For tests that are used for pairwise comparisons, the display includes the difference between each pair of compared means, the confidence interval for the difference, and the significance. The sample sizes of the two groups that are being compared are used in the calculation.
- Output for tests that are specified on the POSTHOC subcommand is available according to their statistical purposes. The following table illustrates the statistical purpose of the post hoc tests:

Table 61. Post hoc methods

Keyword	Homogeneity Subsets Detection	Pairwise Comparison and Confidence Interval
LSD		Yes
SIDAK		Yes
BONFERRONI		Yes
GH		Yes
T2		Yes
T3		Yes
C		Yes
DUNNETT		Yes*
DUNNETTL		Yes*
DUNNETTR		Yes*
SNK	Yes	
BTUKEY	Yes	
DUNCAN	Yes	
QREGW	Yes	
FREGW	Yes	
WALLER	Yes†	
TUKEY	Yes	Yes
SCHEFFE	Yes	Yes
GT2	Yes	Yes
GABRIEL	Yes	Yes

* Only CIs for differences between test group means and control group means are given.

† No significance for Waller test is given.

- Tests that are designed for homogeneity subset detection display the detected homogeneity subsets and their corresponding significances.

- Tests that are designed for both homogeneity subset detection and pairwise comparisons display both kinds of output.
- For the DUNNETT, DUNNETTL, and DUNNETTR keywords, only individual factors can be specified.
- The default reference category for DUNNETT, DUNNETTL, and DUNNETTR is the last category. An integer that is greater than 0, specified within parentheses, can be used to specify a different reference category. For example, POSTHOC = A (DUNNETT(2)) requests a DUNNETT test for factor *A*, using the second level of *A* as the reference category.
- The keywords DUNCAN, DUNNETT, DUNNETTL, and DUNNETTR must be spelled out in full; using the first three characters alone is not sufficient.
- If the REGWGT subcommand is specified, weighted means are used in performing post hoc tests.
- Multiple POSTHOC subcommands are allowed. Each specification is executed independently so that you can test different effects against different error terms.

SNK. *Student-Newman-Keuls procedure based on the Studentized range test.*

TUKEY. *Tukey's honestly significant difference.* This test uses the Studentized range statistic to make all pairwise comparisons between groups.

BTUKEY. *Tukey's* b. This procedure is a multiple comparison procedure based on the average of Studentized range tests.

DUNCAN. *Duncan's multiple comparison procedure based on the Studentized range test*.

SCHEFFE. *Scheffé's multiple comparison* t *test.*

DUNNETT(refcat). *Dunnett's two-tailed* t *test*. Each level of the factor is compared to a reference category. A reference category can be specified in parentheses. The default reference category is the last category. This keyword must be spelled out in full.

DUNNETTL(refcat). *Dunnett's one-tailed* t *test*. This test indicates whether the mean at any level (except the reference category) of the factor is *smaller* than the mean of the reference category. A reference category can be specified in parentheses. The default reference category is the last category. This keyword must be spelled out in full.

DUNNETTR(refcat). *Dunnett's one-tailed* t *test*. This test indicates whether the mean at any level (except the reference category) of the factor is *larger* than the mean of the reference category. A reference category can be specified in parentheses. The default reference category is the last category. This keyword must be spelled out in full.

BONFERRONI. *Bonferroni* t *test*. This test is based on Student's *t* statistic and adjusts the observed significance level based on the fact that multiple comparisons are made.

LSD. *Least significant difference* t *test*. This test is equivalent to multiple *t* tests between all pairs of groups. This test does not control the overall probability of rejecting the hypotheses that some pairs of means are different, while in fact they are equal.

SIDAK. *Sidak* t *test*. This test provides tighter bounds than the Bonferroni test.

GT2. *Hochberg's* GT2. This test is a pairwise comparisons test based on the Studentized maximum modulus test. Unless the cell sizes are extremely unbalanced, this test is fairly robust even for unequal variances.

GABRIEL. *Gabriel's pairwise comparisons test based on the Studentized maximum modulus test.*

FREGW. *Ryan-Einot-Gabriel-Welsch's multiple stepdown procedure based on an* F *test.*

QREGW. *Ryan-Einot-Gabriel-Welsch's multiple stepdown procedure based on the Studentized range test.*

T2. *Tamhane's T2.* Tamhane's pairwise comparisons test based on a *t* test. This test can be applied in situations where the variances are unequal. This test is invalid when there are multiple factors in the model, and the keyword is ignored with a warning in such cases.

T3. *Dunnett's T3.* Pairwise comparisons test based on the Studentized maximum modulus. This test is appropriate when the variances are unequal. This test is invalid when there are multiple factors in the model, and the keyword is ignored with a warning in such cases.

GH. *Games and Howell's pairwise comparisons test based on the Studentized range test.* This test can be applied in situations where the variances are unequal. This test is invalid when there are multiple factors in the model, and the keyword is ignored with a warning in such cases.

C. *Dunnett's C.* Pairwise comparisons based on the weighted average of Studentized ranges. This test can be applied in situations where the variances are unequal. This test is invalid when there are multiple factors in the model, and the keyword is ignored with a warning in such cases.

WALLER(kratio). *Waller-Duncan* t *test.* This test uses a Bayesian approach. The test is restricted to cases with equal sample sizes. For cases with unequal sample sizes, the harmonic mean of the sample size is used. The kratio is the Type 1/Type 2 error seriousness ratio. The default value is 100. You can specify an integer that is greater than 1, enclosed within parentheses.

EMMEANS Subcommand

EMMEANS displays estimated marginal means of the dependent variable in the cells (with covariates held at their overall mean value) and their standard errors of the means for the specified factors. These means are predicted, not observed, means. The estimated marginal means are calculated by using a modified definition by Searle, Speed, and Milliken (1980).

- TABLES, followed by an option in parentheses, is required. COMPARE is optional; if specified, COMPARE must follow TABLES.
- Multiple EMMEANS subcommands are allowed. Each subcommand is treated independently.
- If identical EMMEANS subcommands are specified, only the last identical subcommand is in effect. EMMEANS subcommands that are redundant but not identical (for example, crossed factor combinations such as *A*B* and *B*A*) are all processed.

TABLES(option). *Table specification.* Valid options are the keyword OVERALL, factors appearing on the factor list, and crossed factors that are constructed of factors on the factor list. Crossed factors can be specified by using an asterisk (*) or the keyword BY. All factors in a crossed factor specification must be unique.

- If OVERALL is specified, the estimated marginal means of the dependent variable are displayed, collapsing over between-subjects factors.
- If a between-subjects factor, or a crossing of between-subjects factors, is specified on the TABLES keyword, GLM collapses over any other between-subjects factors before computing the estimated marginal means for the dependent variable. For a multivariate model, GLM collapses over any other between-subjects or within-subjects factors.
- The TABLES keyword may be followed by the WITH keyword and a parenthesized list of covariates and values. Tables are produced with the values of the covariates fixed at the specified values. The keyword MEAN causes the covariate to be fixed at its observed mean value.

COMPARE(factor) ADJ(method). *Main-effects or simple-main-effects omnibus tests and pairwise comparisons of the dependent variable.* This option gives the mean difference, standard error, significance, and confidence interval for each pair of levels for the effect that is specified in the TABLES command, as well as an omnibus test for that effect. If only one factor is specified on TABLES, COMPARE can be specified by itself; otherwise, the factor specification is required. In this case, levels of the specified factor are compared with each other for each level of the other factors in the interaction.

- The optional ADJ keyword allows you to apply an adjustment to the confidence intervals and significance values to account for multiple comparisons. Available methods are LSD (no adjustment), BONFERRONI, or SIDAK.
- If OVERALL is specified on TABLES, COMPARE is invalid.

Example

```
GLM DEP BY A B
  /EMMEANS = TABLES(A*B) WITH(X=1.0) COMPARE(A)
  /DESIGN.
```

- The output of this analysis includes a pairwise comparisons table for the dependent variable *DEP*.
- Assume that *A* has three levels and *B* has two levels. The first level of *A* is compared with the second and third levels, the second level is compared with the first and third levels, and the third level is compared with the first and second levels, with the value of the covariate *X* fixed at 1.0. The pairwise comparison is repeated for the two levels of *B*.

ROBUST Subcommand

ROBUST displays a table of parameter estimates, along with robust or heteroskedasticity-consistent (HC) standard errors; and *t* statistics, significance values, and confidence intervals that use the robust standard errors.

- ROBUST enables specification of the HCCOVB keyword on the OUTFILE subcommand, saving the robust covariance matrix estimates to a new file or dataset.
- ROBUST displays a second set of output for any custom hypothesis tests that are specified on LMATRIX subcommands, with results based on the specified robust covariance matrix estimator.

Five different methods are available for the robust covariance matrix estimation. You can specify only one method. If more than one ROBUST subcommand is specified, then the last subcommand is in effect.

HC0
Based on the original asymptotic or large sample robust, empirical, or "sandwich" estimator of the covariance matrix of the parameter estimates. The middle part of the sandwich contains squared OLS (ordinary least squares) or squared weighted WLS (weighted least squares) residuals.

HC1
A finite-sample modification of HC0, multiplying it by N/(N-p), where N is the sample size and p is the number of non-redundant parameters in the model.

HC2
A modification of HC0 that involves dividing the squared residual by 1-h, where h is the leverage for the case.

HC3
A modification of HC0 that approximates a jackknife estimator. Squared residuals are divided by the square of 1-h. This method is the default if ROBUST is specified without specifying a method.

HC4
A modification of HC0 that divides the squared residuals by 1-h to a power that varies according to h, N, and p, with an upper limit of 4.

SAVE Subcommand

Use SAVE to add one or more residual or fit values to the active dataset.

- Specify one or more temporary variables, each variable followed by an optional new name in parentheses. For a multivariate model, you can optionally specify a new name for the temporary variable related to each dependent variable.
- WPRED and WRESID can be saved only if REGWGT has been specified.
- Specifying a temporary variable on this subcommand results in a variable being added to the active data file for each dependent variable.
- You can specify variable names for the temporary variables. These names must be unique, valid variable names. For a multivariate model, there should be as many variable names specified as there are dependent variables, and names should be listed in the order of the dependent variables as

specified on the GLM command. If you do not specify enough variable names, default variable names are used for any remaining variables.

- If new names are not specified, GLM generates a rootname by using a shortened form of the temporary variable name with a suffix. For a multivariate model, the suffix _*n* is added to the temporary variable name, where *n* is the ordinal number of the dependent variable as specified on the GLM command.
- If more than one SAVE subcommand is specified, only the last subcommand is in effect.

PRED. *Unstandardized predicted values.*

WPRED. *Weighted unstandardized predicted values.* This setting is available only if REGWGT has been specified.

RESID. *Unstandardized residuals.*

WRESID. *Weighted unstandardized residuals.* This setting is available only if REGWGT has been specified.

DRESID. *Deleted residuals.*

ZRESID. *Standardized residuals.*

SRESID. *Studentized residuals.*

SEPRED. *Standard errors of predicted value.*

COOK. *Cook's distances.*

LEVER. *Uncentered leverage values.*

OUTFILE Subcommand

The OUTFILE subcommand writes data to a data file or previously declared dataset that can be used in other procedures.

- You must specify a keyword on OUTFILE. There is no default.
- You must specify a quoted file specification or previously declared dataset name (DATASET DECLARE command) in parentheses after a keyword. The asterisk (*) is not allowed.
- If you specify more than one keyword, a different filename is required for each keyword.
- If more than one OUTFILE subcommand is specified, only the last subcommand is in effect.
- For COVB or CORB, the output will contain, in addition to the covariance or correlation matrix, four rows for each dependent variable: a row of parameter estimates, a row of standard errors of the parameter estimates, a row of residual degrees of freedom, and a row of significance values for the *t* statistics corresponding to the parameter estimates. All statistics are displayed separately by split. For HCCOVB, the covariance matrix is calculated with a robust estimator and contains only diagonal terms (the robust estimator is supported only for univariate models). In addition, the row of standard errors displays the robust standard errors, and the significance values reflect the robust errors.

COVB ('savfile'|'dataset'). *Writes the parameter covariance matrix.*

CORB ('savfile'|'dataset'). *Writes the parameter correlation matrix.*

HCCOVB ('savfile'|'dataset'). *Writes the parameter covariance matrix, calculated with a robust estimator.* HCCOVB is valid only if the ROBUST subcommand is specified.

EFFECT ('savfile'|'dataset'). *Writes the statistics from the between-subjects ANOVA table.* This specification is invalid for repeated measures analyses.

DESIGN ('savfile'|'dataset'). *Writes the design matrix.* The number of rows equals the number of cases, and the number of columns equals the number of parameters. The variable names are *DES_1*, *DES_2*, *..., DES_p*, where *p* is the number of the parameters.

MBPDESIGN, BPDESIGN, and FDESIGN Subcommands

MBPDESIGN, BPDESIGN, and FDESIGN specify the auxiliary regression model that is used to test for heteroskedasticity. MBPDESIGN specifies the model for the Modified Breusch-Pagan test. BPDESIGN specifies the model for the Breusch-Pagan test. FDESIGN specifies the model for the F test.

Note: MBPDESIGN, BPDESIGN, and FDESIGN are necessary only when you don't want to use the default model for the associated test. The default model consists of a constant term, a term that is linear in the predicted values, a term that is quadratic in the predicted values, and an error term. The default model is requested with the MBP, BP, or F keyword on the PRINT subcommand.

- Specify a list of terms to be included in the model, and separate the terms by spaces or commas. The model that is used to test for heteroskedasticity always includes an intercept term, so specifying INTERCEPT has no effect.
- If MBPDESIGN, BPDESIGN, or FDESIGN is specified by itself (without any explicit terms), then the model that is specified by the DESIGN subcommand is used. An intercept term is included if the specified model does not contain one.
- To include a term for the main effect of a factor, enter the name of the factor.
- To include a term for an interaction between factors, use the keyword BY or the asterisk (*) to join the factors that are involved in the interaction. For example, *A*B* means a two-way interaction effect of *A* and *B*, where *A* and *B* are factors. *A*A* is not allowed because factors inside an interaction effect must be distinct.
- To include a term for nesting one effect within another effect, use the keyword WITHIN or use a pair of parentheses. For example, *A(B)* means that *A* is nested within *B*. The expression *A(B)* is equivalent to the expression *A WITHIN B*. When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, *A(B)(C)* is not valid.
- Multiple nesting is allowed. For example, *A(B(C))* means that *B* is nested within *C* and *A* is nested within *B(C)*.
- Interactions between nested effects are not valid. For example, neither *A(C)*B(C)* nor *A(C)*B(D)* is valid.
- To include a covariate term in the design, enter the name of the covariate.
- Covariates can be connected—but not nested—through the * operator to form another covariate effect. Therefore, interactions among covariates such as *X1*X1* and *X1*X2* are valid but not *X1(X2)*.
- Factor and covariate effects can be connected only by the * operator. Suppose *A* and *B* are factors and *X1* and *X2* are covariates. Examples of valid factor-by-covariate interaction effects are *A*X1*, *A*B*X1*, *X1*A(B)*, *A*X1*X1*, and *B*X1*X2*.
- If more than one MBPDESIGN, BPDESIGN, or FDESIGN subcommand is specified, only the last subcommand is in effect.

DESIGN Subcommand

DESIGN specifies the effects included in a specific model. The cells in a design are defined by all of the possible combinations of levels of the factors in that design. The number of cells equals the product of the number of levels of all the factors. A design is *balanced* if each cell contains the same number of cases. GLM can analyze both balanced and unbalanced designs.

- Specify a list of terms to be included in the model, and separate the terms by spaces or commas.
- The default design, if the DESIGN subcommand is omitted or is specified by itself, is a design consisting of the following terms in order: the intercept term (if INTERCEPT=INCLUDE is specified), the covariates that are given in the covariate list, and the full factorial model defined by all factors on the factor list and excluding the intercept.
- To include a term for the main effect of a factor, enter the name of the factor on the DESIGN subcommand.

- To include the intercept term in the design, use the keyword INTERCEPT on the DESIGN subcommand. If INTERCEPT is specified on the DESIGN subcommand, the subcommand INTERCEPT=EXCLUDE is overridden.
- To include a term for an interaction between factors, use the keyword BY or the asterisk (*) to join the factors that are involved in the interaction. For example, *A*B* means a two-way interaction effect of *A* and *B*, where *A* and *B* are factors. *A*A* is not allowed because factors inside an interaction effect must be distinct.
- To include a term for nesting one effect within another effect, use the keyword WITHIN or use a pair of parentheses on the DESIGN subcommand. For example, *A(B)* means that *A* is nested within *B*. The expression *A(B)* is equivalent to the expression *A WITHIN B*. When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, *A(B)(C)* is not valid.
- Multiple nesting is allowed. For example, *A(B(C))* means that *B* is nested within *C* and *A* is nested within *B(C)*.
- Interactions between nested effects are not valid. For example, neither *A(C)*B(C)* nor *A(C)*B(D)* is valid.
- To include a covariate term in the design, enter the name of the covariate on the DESIGN subcommand.
- Covariates can be connected—but not nested—through the * operator to form another covariate effect. Therefore, interactions among covariates such as *X1*X1* and *X1*X2* are valid but not *X1(X2)*. Using covariate effects such as *X1*X1*, *X1*X1*X1*, *X1*X2*, and *X1*X1*X2*X2* makes fitting a polynomial regression model easy in GLM.
- Factor and covariate effects can be connected only by the * operator. Suppose *A* and *B* are factors and *X1* and *X2* are covariates. Examples of valid factor-by-covariate interaction effects are *A*X1*, *A*B*X1*, *X1*A(B)*, *A*X1*X1*, and *B*X1*X2*.
- If more than one DESIGN subcommand is specified, only the last subcommand is in effect.

Example

```
GLM Y BY A B C WITH X
  /DESIGN A B(A) X*A.
```

- In this example, the design consists of a main effect *A*, a nested effect *B* within *A*, and an interaction effect of a covariate *X* with a factor *A*.

GLM: Multivariate

GLM is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
GLM dependent varlist [BY factor list [WITH covariate list]]

[/REGWGT=varname]

[/METHOD=SSTYPE({1  })]
                {2  }
                {3**}
                {4  }

[/INTERCEPT=[INCLUDE**] [EXCLUDE]]

[/MISSING=[INCLUDE] [EXCLUDE**]]

[/CRITERIA=[EPS({1E-8**})] [ALPHA({0.05**})]
                {a     }           {a     }

[/PRINT  = [DESCRIPTIVE] [HOMOGENEITY] [PARAMETER][ETASQ] [RSSCP]
           [GEF] [LOF] [OPOWER] [TEST [([SSCP] [LMATRIX] [MMATRIX])]]

[/PLOT=[SPREADLEVEL] [RESIDUALS]
       [PROFILE (factor factor*factor factor*factor*factor ...)]
                [WITH(covariate={value} [...])]
                               {MEAN }
                [TYPE={LINE**}]
                      {BAR   }
                [MEANREFERENCE={NO**}]
                               {YES }
                [ERRORBAR={NO**      }]
                          {CI        }
                          {SE [(2**)]}
                [YAXIS={AUTO**}]
                       {0     }

[/LMATRIX={["label"] effect list effect list ...;...}]
          {["label"] effect list effect list ...    }
          {["label"] ALL list; ALL...               }
          {["label"] ALL list                       }

[/MMATRIX= {["label"] depvar value depvar value ...;["label"]...}]
           {["label"] depvar value depvar value ...             }
           {["label"] ALL list; ["label"] ...                   }
           {["label"] ALL list                                  }

[/KMATRIX= {list of numbers    }]
           {list of numbers;...}

[/SAVE=[tempvar [(list of names)]] [tempvar [(list of names)]]...]
       [DESIGN]

[/OUTFILE=[{COVB('savfile'|'dataset')}]
          {CORB('savfile'|'dataset')}
          [EFFECT('savfile'|'dataset')] [DESIGN('savfile'|'dataset')]

[/DESIGN={[INTERCEPT...]    }]
         {[effect effect...]}
```

** Default if the subcommand or keyword is omitted.

Temporary variables (tempvar) are:

```
PRED, WPRED, RESID, WRESID, DRESID, ZRESID, SRESID, SEPRED, COOK, LEVER
```

Example

```
GLM SCORE1 TO SCORE4 BY METHOD(1,3).
```

Overview

This section discusses the subcommands that are used in multivariate general linear models and covariance designs with several interrelated dependent variables. The discussion focuses on subcommands and keywords that do not apply—or apply in different manners—to univariate analyses. The discussion does not contain information about all subcommands that you will need to specify the design. For subcommands that are not covered here, see “GLM: Univariate” on page 845.

Options

Optional Output. In addition to the output that is described in *GLM: Univariate*, you can have both multivariate and univariate *F* tests. Using the PRINT subcommand, you can request the hypothesis and error sums-of-squares and cross-product matrices for each effect in the design, the transformation coefficient table (M matrix), Box's *M* test for equality of covariance matrices, and Bartlett's test of sphericity.

Basic Specification

- The basic specification is a variable list identifying the dependent variables, with the factors (if any) named after BY and the covariates (if any) named after WITH.
- By default, GLM uses a model that includes the intercept term, the covariates (if any), and the full factorial model, which includes all main effects and all possible interactions among factors. The intercept term is excluded if it is excluded in the model by specifying EXCLUDE on the INTERCEPT subcommand. GLM produces multivariate and univariate *F* tests for each effect in the model. GLM also calculates the power for each test, based on the default alpha value.

Subcommand Order

- The variable list must be specified first.
- Subcommands can be used in any order.

Syntax Rules

- The syntax rules that apply to univariate analysis also apply to multivariate analysis.
- If you enter one of the multivariate specifications in a univariate analysis, GLM ignores it.

Limitations

- Any number of factors can be specified, but if the number of between-subjects factors plus the number of split variables exceeds 18, the Descriptive Statistics table is not printed even when you request it.
- Memory requirements depend primarily on the number of cells in the design. For the default full factorial model, this equals the product of the number of levels or categories in each factor.

GLM Variable List

- Multivariate GLM calculates statistical tests that are valid for analyses of dependent variables that are correlated with one another. The dependent variables must be specified first.
- The factor and covariate lists follow the same rules as in univariate analyses.
- If the dependent variables are uncorrelated, the univariate significance tests have greater statistical power.

PRINT Subcommand

By default, if no PRINT subcommand is specified, multivariate GLM produces multivariate tests (MANOVA) and univariate tests (ANOVA) for all effects in the model. All PRINT specifications that are described in "PRINT Subcommand " on page 850 are available in multivariate analyses. The following additional output can be requested:

TEST(SSCP). *Sums-of-squares and cross-product matrices.* Hypothesis (HSSCP) and error (ESSCP) sums-of-squares and cross-product matrices for each effect in the design are displayed. Each between-subjects effect has a different HSSCP matrix, but there is a single ESSCP matrix for all between-subjects effects. For a repeated measures design, each within-subjects effect has an HSSCP matrix and an ESSCP matrix. If there are no within-subjects effects, the ESSCP matrix for the between-subjects effects is the same as the RSSCP matrix.

TEST(MMATRIX). *Set of transformation coefficients (M) matrices.* Any M matrices that are generated by the MMATRIX subcommand are displayed. If no M matrix is specified on the MMATRIX subcommand, this specification is skipped, unless you are using a repeated measures design. In a repeated measures

design, this set always includes the M matrix that is determined by the WSFACTOR subcommand. The specification TEST(TRANSFORM) is equivalent to TEST(MMATRIX).

HOMOGENEITY. *Tests of homogeneity of variance.* In addition to Levene's test for equality of variances for each dependent variable, the display includes Box's *M* test of homogeneity of the covariance matrices of the dependent variables across all level combinations of the between-subjects factors.

RSSCP. *Sums-of-squares and cross-products of residuals.* Three matrices are displayed:

- *Residual SSCP matrix.* This matrix is a square matrix of sums of squares and cross- products of residuals. The dimension of this matrix is the same as the number of dependent variables in the model.
- *Residual covariance matrix.* This matrix is the residual SSCP matrix divided by the degrees of freedom of the residual.
- *Residual correlation matrix*. This matrix is the standardized form of the residual covariance matrix.

Example

```
GLM Y1 Y2 Y3 BY A B
 /PRINT = HOMOGENEITY RSSCP
 /DESIGN.
```

- Since there are three dependent variables, this model is a multivariate model.
- The keyword RSSCP produces three matrices of sums of squares and cross-products of residuals. The output also contains the result of Bartlett's test of the sphericity of the residual covariance matrix.
- In addition to the Levene test for each dependent variable, the keyword HOMOGENEITY produces the result of Box's *M* test of homogeneity in the multivariate model.

MMATRIX Subcommand

The MMATRIX subcommand allows you to customize your hypothesis tests by specifying the M matrix (transformation coefficients matrix) in the general form of the linear hypothesis LBM = K, where K = 0 if it is not specified on the KMATRIX subcommand. The vector B is the parameter vector in the linear model.

- Specify an optional label in quotation marks. Then either list dependent variable names, each name followed by a real number, or specify the keyword ALL followed by a list of real numbers. Only variable names that appear on the dependent variable list can be specified on the MMATRIX subcommand.
- You can specify one label for each column in the M matrix.
- If you specify ALL, the length of the list that follows ALL should be equal to the number of dependent variables.
- There is no limit on the length of the label.
- For the MMATRIX subcommand to be valid, at least one of the following specifications must be made: the LMATRIX subcommand or INTERCEPT=INCLUDE. (Either of these specifications defines an L matrix.)
- If both LMATRIX and MMATRIX are specified, the L matrix is defined by the LMATRIX subcommand.
- If MMATRIX or KMATRIX is specified but LMATRIX is not specified, the **L** matrix is defined by the estimable function for the intercept effect, provided that the intercept effect is included in the model.
- If LMATRIX is specified but MMATRIX is not specified, the **M** matrix is assumed to be an *r* x *r* identity matrix, where *r* is the number of dependent variables.
- A semicolon (;) indicates the end of a column in the **M** matrix.
- Dependent variables that do not appear on a list of dependent variable names and real numbers are assigned a value of 0.
- Dependent variables that do not appear in the MMATRIX subcommand will have a row of zeros in the **M** matrix.
- A number can be specified as a fraction with a positive denominator (for example, 1/3 or –1/3 is valid, but 1/–3 is invalid).

- The number of columns must be greater than 0. You can specify as many columns as you need.
- If more than one MMATRIX subcommand is specified, only the last subcommand is in effect.

Example

```
GLM Y1 Y2 Y3 BY A B
 /MMATRIX = "Y1–Y2" Y1 1 Y2 –1; "Y1–Y3" Y1 1 Y3 –1
            "Y2–Y3" Y2 1 Y3 –1
 /DESIGN.
```

- In the above example, *Y1*, *Y2*, and *Y3* are the dependent variables.
- The MMATRIX subcommand requests all pairwise comparisons among the dependent variables.
- Because LMATRIX was not specified, the L matrix is defined by the estimable function for the intercept effect.

GLM: Repeated Measures

GLM is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
GLM dependent varlist [BY factor list [WITH covariate list]]

/WSFACTOR=name levels [{DEVIATION [(refcat)]       }] name...
                       {SIMPLE [(refcat)]          }
                       {DIFFERENCE                 }
                       {HELMERT                    }
                       {REPEATED                   }
                       {POLYNOMIAL [({1,2,3...})]**}
                       {           {metric  }      }
                       {SPECIAL (matrix)           }

[/MEASURE=newname newname...]

[/WSDESIGN=effect effect...]

[/REGWGT=varname]

[/METHOD=SSTYPE({1  })]
                {2  }
                {3**}
                {4  }

[/INTERCEPT=[INCLUDE**] [EXCLUDE]]

[/MISSING=[INCLUDE] [EXCLUDE**]]

[/PRINT  = [DESCRIPTIVE] [HOMOGENEITY] [PARAMETER][ETASQ] [RSSCP]
           [GEF] [LOF] [OPOWER] [TEST [([SSCP] [LMATRIX] [MMATRIX])]]

[/SAVE=[tempvar [(list of names)]] [tempvar [(list of names)]]...]
       [DESIGN]

[/EMMEANS=TABLES({OVERALL             })] [COMPARE ADJ(LSD)(BONFERRONI)(SIDAK)]
                 {factor                }
                 {factor*factor...      }
                 {wsfactor              }
                 {wsfactor*wsfactor...  }
                 {factor*...wsfactor*...}
                 {factor*factor...      }

[/DESIGN={[INTERCEPT...]    }]*
         {[effect effect...]}
```

* The DESIGN subcommand has the same syntax as is described in *GLM: Univariate*.

** Default if the subcommand or keyword is omitted.

Example

```
GLM Y1 TO Y4 BY GROUP
  /WSFACTOR=YEAR 4.
```

Overview

This section discusses the subcommands that are used in repeated measures designs, in which the dependent variables represent measurements of the same variable (or variables) taken repeatedly. This section does not contain information on all of the subcommands that you will need to specify the design. For some subcommands or keywords not covered here, such as DESIGN, see “GLM: Univariate” on page 845. For information on optional output and the multivariate significance tests available, see “GLM: Multivariate” on page 865.

- In a simple repeated measures analysis, all dependent variables represent different measurements of the same variable for different values (or levels) of a within-subjects factor. Between-subjects factors and covariates can also be included in the model, just as in analyses not involving repeated measures.
- A **within-subjects factor** is simply a factor that distinguishes measurements made on the same subject or case, rather than distinguishing different subjects or cases.
- GLM permits more complex analyses, in which the dependent variables represent levels of two or more within-subjects factors.

- GLM also permits analyses in which the dependent variables represent measurements of several variables for the different levels of the within-subjects factors. These are known as **doubly multivariate designs**.
- A repeated measures analysis includes a within-subjects design describing the model to be tested with the within-subjects factors, as well as the usual between-subjects design describing the effects to be tested with between-subjects factors. The default for the within-subjects factors design is a full factorial model which includes the main within-subjects factor effects and all their interaction effects.
- If a custom hypothesis test is required (defined by the CONTRAST, LMATRIX, or KMATRIX subcommands), the default transformation matrix (**M** matrix) is taken to be the average transformation matrix, which can be displayed by using the keyword TEST(MMATRIX) on the PRINT subcommand. The default contrast result matrix (**K** matrix) is the zero matrix.
- If the contrast coefficient matrix (**L** matrix) is not specified, but a custom hypothesis test is required by the MMATRIX or the KMATRIX subcommand, the contrast coefficient matrix (**L** matrix) is taken to be the **L** matrix which corresponds to the estimable function for the intercept in the between-subjects model. This matrix can be displayed by using the keyword TEST(LMATRIX) on the PRINT subcommand.

Basic Specification

- The basic specification is a variable list followed by the WSFACTOR subcommand.
- Whenever WSFACTOR is specified, GLM performs special repeated measures processing. The multivariate and univariate tests are provided. In addition, for any within-subjects effect involving more than one transformed variable, the Mauchly test of sphericity is displayed to test the assumption that the covariance matrix of the transformed variables is constant on the diagonal and zero off the diagonal. The Greenhouse-Geisser epsilon and the Huynh-Feldt epsilon are also displayed for use in correcting the significance tests in the event that the assumption of sphericity is violated.

Subcommand Order

- The list of dependent variables, factors, and covariates must be first.

Syntax Rules

- The WSFACTOR (within-subjects factors), WSDESIGN (within-subjects design), and MEASURE subcommands are used only in repeated measures analysis.
- WSFACTOR is required for any repeated measures analysis.
- If WSDESIGN is not specified, a full factorial within-subjects design consisting of all main effects and all interactions among within-subjects factors is used by default.
- The MEASURE subcommand is used for doubly multivariate designs, in which the dependent variables represent repeated measurements of more than one variable.

Limitations

- Any number of factors can be specified, but if the number of between-subjects factors plus the number of split variables exceeds 18, the Descriptive Statistics table is not printed even when you request it.
- Maximum of 18 within-subjects factors.
- Memory requirements depend primarily on the number of cells in the design. For the default full factorial model, this equals the product of the number of levels or categories in each factor.

Example

Repeated Measures ANOVA

```
GLM Y1 TO Y4 BY GROUP
  /WSFACTOR=YEAR 4 POLYNOMIAL
  /WSDESIGN=YEAR
  /PRINT=PARAMETER
  /DESIGN=GROUP.
```

- WSFACTOR specifies a repeated measures analysis in which the four dependent variables represent a single variable measured at four levels of the within-subjects factor. The within-subjects factor is called *YEAR* for the duration of the GLM procedure.
- POLYNOMIAL requests polynomial contrasts for the levels of *YEAR*. Because the four variables, *Y1*, *Y2*, *Y3*, and *Y4*, in the active dataset represent the four levels of *YEAR*, the effect is to perform an orthonormal polynomial transformation of these variables.
- PRINT requests that the parameter estimates be displayed.
- WSDESIGN specifies a within-subjects design that includes only the effect of the *YEAR* within-subjects factor. Because *YEAR* is the only within-subjects factor specified, this is the default design, and WSDESIGN could have been omitted.
- DESIGN specifies a between-subjects design that includes only the effect of the *GROUP* between-subjects factor. This subcommand could have been omitted.

GLM Variable List

The list of dependent variables, factors, and covariates must be specified first.

- WSFACTOR determines how the dependent variables on the GLM variable list will be interpreted.
- The number of dependent variables on the GLM variable list must be a multiple of the number of cells in the within-subjects design. If there are six cells in the within-subjects design, each group of six dependent variables represents a single within-subjects variable that has been measured in each of the six cells.
- Normally, the number of dependent variables should equal the number of cells in the within-subjects design multiplied by the number of variables named on the MEASURE subcommand (if one is used). If you have more groups of dependent variables than are accounted for by the MEASURE subcommand, GLM will choose variable names to label the output, which may be difficult to interpret.
- Covariates are specified after keyword WITH. You can specify constant covariates. Constant covariates represent variables whose values remain the same at each within-subjects level.

Example

```
GLM MATH1 TO MATH4 BY METHOD WITH SES
  /WSFACTOR=SEMESTER 4.
```

- The four dependent variables represent a score measured four times (corresponding to the four levels of *SEMESTER*).
- *SES* is a constant covariate. Its value does not change over the time covered by the four levels of *SEMESTER*.
- Default contrast (POLYNOMIAL) is used.

WSFACTOR Subcommand

WSFACTOR names the within-subjects factors, specifies the number of levels for each, and specifies the contrast for each.

- Presence of the WSFACTOR subcommand implies that the repeated measures model is being used.
- Mauchly's test of sphericity is automatically performed when WSFACTOR is specified.
- Names and number levels for the within-subjects factors are specified on the WSFACTOR subcommand. Factor names must not duplicate any of the dependent variables, factors, or covariates named on the GLM variable list. A type of contrast can also be specified for each within-subjects factor in order to perform comparisons among its levels. This contrast amounts to a transformation on the dependent variables.
- If there are more than one within-subjects factors, they must be named in the order corresponding to the order of the dependent variables on the GLM variable list. GLM varies the levels of the last-named

within-subjects factor most rapidly when assigning dependent variables to within-subjects cells (see the example below).

- The number of cells in the within-subjects design is the product of the number of levels for all within-subjects factors.
- Levels of the factors must be represented in the data by the dependent variables named on the GLM variable list.
- The number of levels of each factor must be at least two. Enter an integer equal to or greater than 2 after each factor to indicate how many levels the factor has. Optionally, you can enclose the number of levels in parentheses.
- Enter only the *number of levels* for within-subjects factors, not a range of values.
- If more than one WSFACTOR subcommand is specified, only the last one is in effect.

Contrasts for WSFACTOR

The levels of a within-subjects factor are represented by different dependent variables. Therefore, contrasts between levels of such a factor compare these dependent variables. Specifying the type of contrast amounts to specifying a transformation to be performed on the dependent variables.

- In testing the within-subjects effects, an orthonormal transformation is automatically performed on the dependent variables in a repeated measures analysis.
- The contrast for each within-subjects factor is entered after the number of levels. If no contrast keyword is specified, POLYNOMIAL(1,2,3...) is the default. This contrast is used in comparing the levels of the within-subjects factors. Intrinsically orthogonal contrast types are recommended for within-subjects factors if you wish to examine each degree-of-freedom test, provided compound symmetry is assumed within each within-subjects factor. Other orthogonal contrast types are DIFFERENCE and HELMERT.
- If there are more than one within-subjects factors, the transformation matrix (**M** matrix) is computed as the Kronecker product of the matrices generated by the contrasts specified.
- The transformation matrix (**M** matrix) generated by the specified contrasts can be displayed by using the keyword TEST(MMATRIX) on the subcommand PRINT.
- The contrast types available for within-subjects factors are the same as those on the CONTRAST subcommand for between-subjects factors, described in “CONTRAST Subcommand ” on page 855.

The following contrast types are available:

DEVIATION. *Deviations from the grand mean.* This is the default for between-subjects factors. Each level of the factor except one is compared to the grand mean. One category (by default the last) must be omitted so that the effects will be independent of one another. To omit a category other than the last, specify the number of the omitted category in parentheses after the keyword DEVIATION. For example

```
GLM Y1 Y2 Y3 BY GROUP
  /WSFACTOR = Y 3 DEVIATION (1)
```

Deviation contrasts are not orthogonal.

POLYNOMIAL. *Polynomial contrasts.* This is the default for within-subjects factors. The first degree of freedom contains the linear effect across the levels of the factor, the second contains the quadratic effect, and so on. In a balanced design, polynomial contrasts are orthogonal. By default, the levels are assumed to be equally spaced; you can specify unequal spacing by entering a metric consisting of one integer for each level of the factor in parentheses after the keyword POLYNOMIAL. (All metrics specified cannot be equal; thus (1,1,...,1) is not valid.) For example:

```
/WSFACTOR=D 3 POLYNOMIAL(1,2,4).
```

Suppose that factor D has three levels. The specified contrast indicates that the three levels of D are actually in the proportion 1:2:4. The default metric is always (1,2,...,*k*), where *k* levels are involved. Only the relative differences between the terms of the metric matter (1,2,4) is the same metric as (2,3,5) or

(20,30,50) because, in each instance, the difference between the second and third numbers is twice the difference between the first and second.

DIFFERENCE. *Difference or reverse Helmert contrasts.* Each level of the factor except the first is compared to the mean of the previous levels. In a balanced design, difference contrasts are orthogonal.

HELMERT. *Helmert contrasts.* Each level of the factor except the last is compared to the mean of subsequent levels. In a balanced design, Helmert contrasts are orthogonal.

SIMPLE. *Each level of the factor except the last is compared to the last level.* To use a category other than the last as the omitted reference category, specify its number in parentheses following keyword SIMPLE. For example:

```
/WSFACTOR=B 3 SIMPLE (1).
```

Simple contrasts are not orthogonal.

REPEATED. *Comparison of adjacent levels.* Each level of the factor except the last is compared to the next level. Repeated contrasts are not orthogonal.

SPECIAL. *A user-defined contrast.* Values specified after this keyword are stored in a matrix in column major order. For example, if factor A has three levels, then WSFACTOR(A)= SPECIAL(1 1 1 1 -1 0 0 1 -1) produces the following contrast matrix:

```
1   1   0
1  -1   1
1   0  -1
```

Example

```
GLM X1Y1 X1Y2 X2Y1 X2Y2 X3Y1 X3Y2 BY TREATMNT GROUP
  /WSFACTOR=X 3 Y 2
  /DESIGN.
```

- The GLM variable list names six dependent variables and two between-subjects factors, *TREATMNT* and *GROUP*.
- WSFACTOR identifies two within-subjects factors whose levels distinguish the six dependent variables. *X* has three levels, and *Y* has two. Thus, there are 3 × 2 = 6 cells in the within-subjects design, corresponding to the six dependent variables.
- Variable *X1Y1* corresponds to levels 1,1 of the two within-subjects factors; variable *X1Y2* corresponds to levels 1,2; *X2Y1* to levels 2,1; and so on up to *X3Y2*, which corresponds to levels 3,2. The first within-subjects factor named, *X*, varies most slowly, and the last within-subjects factor named, *Y*, varies most rapidly on the list of dependent variables.
- Because there is no WSDESIGN subcommand, the within-subjects design will include all main effects and interactions: *X*, *Y*, and *X* by *Y*.
- Likewise, the between-subjects design includes all main effects and interactions (*TREATMNT*, *GROUP*, and *TREATMNT* by *GROUP*) plus the intercept.
- In addition, a repeated measures analysis always includes interactions between the within-subjects factors and the between-subjects factors. There are three such interactions for each of the three within-subjects effects.

Example

```
GLM SCORE1 SCORE2 SCORE3 BY GROUP
  /WSFACTOR=ROUND 3 DIFFERENCE
  /CONTRAST(GROUP)=DEVIATION
  /PRINT=PARAMETER TEST(LMATRIX).
```

- This analysis has one between-subjects factor, *GROUP*, and one within-subjects factor, *ROUND*, with three levels that are represented by the three dependent variables.
- The WSFACTOR subcommand also specifies difference contrasts for *ROUND*, the within-subjects factor.
- There is no WSDESIGN subcommand, so a default full factorial within-subjects design is assumed. This could also have been specified as WSDESIGN=ROUND, or simply WSDESIGN.

- The CONTRAST subcommand specifies deviation contrasts for *GROUP*, the between-subjects factor. This subcommand could have been omitted because deviation contrasts are the default.
- PRINT requests the display of the parameter estimates for the model and the **L** matrix.
- There is no DESIGN subcommand, so a default full factorial between-subjects design is assumed. This could also have been specified as DESIGN=GROUP, or simply DESIGN.

WSDESIGN Subcommand

WSDESIGN specifies the design for within-subjects factors. Its specifications are like those of the DESIGN subcommand, but it uses the within-subjects factors rather than the between-subjects factors.

- The default WSDESIGN is a full factorial design, which includes all main effects and all interactions for within-subjects factors. The default is in effect whenever a design is processed without a preceding WSDESIGN or when the preceding WSDESIGN subcommand has no specifications.
- A WSDESIGN specification cannot include between-subjects factors or terms based on them, nor does it accept interval-level variables.
- The keyword INTERCEPT is not allowed on WSDESIGN.
- Nested effects are not allowed. Therefore, the symbols ( ) are not allowed here.
- If more than one WSDESIGN subcommand is specified, only the last one is in effect.

Example

```
GLM JANLO,JANHI,FEBLO,FEBHI,MARLO,MARHI BY SEX
  /WSFACTOR MONTH 3 STIMULUS 2
  /WSDESIGN MONTH, STIMULUS
  /DESIGN SEX.
```

- There are six dependent variables, corresponding to three months and two different levels of stimulus.
- The dependent variables are named on the GLM variable list in an order such that the level of stimulus varies more rapidly than the month. Thus, *STIMULUS* is named last on the WSFACTOR subcommand.
- The WSDESIGN subcommand specifies only the main effects for within-subjects factors. There is no *MONTH*-by-*STIMULUS* interaction term.

MEASURE Subcommand

In a doubly multivariate analysis, the dependent variables represent multiple variables measured under the different levels of the within-subjects factors. Use MEASURE to assign names to the variables that you have measured for the different levels of within-subjects factors.

- Specify a list of one or more variable names to be used in labeling the averaged results. If no within-subjects factor has more than two levels, MEASURE has no effect. You can use up to 255 bytes for each name.
- The number of dependent variables in the dependent variables list should equal the product of the number of cells in the within-subjects design and the number of names on MEASURE.
- If you do not enter a MEASURE subcommand and there are more dependent variables than cells in the within-subjects design, GLM assigns names (normally *MEASURE_1*, *MEASURE_2*, and so on) to the different measures.
- All of the dependent variables corresponding to each measure should be listed together and ordered so that the within-subjects factor named last on the WSFACTORS subcommand varies most rapidly.

Example

```
GLM TEMP11 TEMP12 TEMP21 TEMP22 TEMP31 TEMP32,
WEIGHT11 WEIGHT12 WEIGHT21 WEIGHT22 WEIGHT31 WEIGHT32 BY GROUP
  /WSFACTOR=DAY 3 AMPM 2
  /MEASURE=TEMP WEIGHT
  /WSDESIGN=DAY, AMPM, DAY BY AMPM
  /DESIGN.
```

- There are 12 dependent variables: six temperatures and six weights, corresponding to morning and afternoon measurements on three days.
- WSFACTOR identifies the two factors (*DAY* and *AMPM*) that distinguish the temperature and weight measurements for each subject. These factors define six within-subjects cells.
- MEASURE indicates that the first group of six dependent variables correspond to *TEMP* and the second group of six dependent variables correspond to *WEIGHT*.
- These labels, *TEMP* and *WEIGHT*, are used on the output as the measure labels.
- WSDESIGN requests a full factorial within-subjects model. Because this is the default, WSDESIGN could have been omitted.

EMMEANS Subcommand

EMMEANS displays estimated marginal means of the dependent variables in the cells, adjusted for the effects of covariates at their overall means, for the specified factors. Note that these are predicted, not observed, means. The standard errors are also displayed. See the topic "EMMEANS Subcommand " on page 859 for more information.

- For the TABLES and COMPARE keywords, valid options include the within-subjects factors specified in the WSFACTOR subcommand, crossings among them, and crossings among factors specified in the factor list and factors specified on the WSFACTOR subcommand.
- All factors in a crossed-factors specification must be unique.
- If a between- or within-subjects factor, or a crossing of between- or within-subjects factors, is specified on the TABLES keyword, then GLM will collapse over any other between- or within-subjects factors before computing the estimated marginal means for the dependent variables.

GRAPH

GRAPH is available in the Statistics Base Edition.

```
GRAPH

  [/TITLE='line 1' ['line 2']]
  [/SUBTITLE='line 1']
  [/FOOTNOTE='line 1' ['line 2']]

  {/BAR [{(SIMPLE)       }]=function/variable specification†       }
        {(GROUPED)       }
        {(STACKED)       }
        {(RANGE)         }

  {/LINE [{(SIMPLE)    }]=function/variable specification†         }
         {(MULTIPLE)   }
         {(DROP)       }
         {(AREA)       }
         {(DIFFERENCE)}

  {/PIE                                                            }

  {/PARETO[{(CUM)  }][{(SIMPLE) }]=function/variable specification†}
          {(NOCUM)}  {(STACKED)}

  {/HILO[{(SIMPLE) }]=function/variable specification††            }
        {(GROUPED)}

  {/HISTOGRAM [(NORMAL)]=var                                       }

  {/SCATTERPLOT[{(BIVARIATE)}]=variable specification†††           }
               {(OVERLAY)   }
               {(MATRIX)    }
               {(XYZ)       }

  {/ERRORBAR[{(CI[{95}])     }]={var [var var ...][BY var]}        }
                {n }            {var BY var BY var        }
           {(STERRIR[{12}])}
                    {n }
           {(STDDEV[{2}])  }
                    {n}

  [/PANEL COLVAR=varlist COLOP={CROSS**} ROWVAR=varlist ROWOP={CROSS**}]
                               {NEST   }                     {NEST   }

  [/INTERVAL {CI     {(95)}}]
             {STDDEV {(2) }}
             {SE     {(n) }}

  [/TEMPLATE=file]

  [/MISSING=[{LISTWISE**}][{NOREPORT**}][{EXCLUDE**}]]
            {VARIABLE  }] {REPORT    }  {INCLUDE  }
```

** Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 13.0

- PANEL subcommand introduced.
- INTERVAL subcommand introduced.

Value function:

The VALUE function yields the value of the specified variable for each case. It always produces one bar, point, or slice for each case. The VALUE(X) specification implies the value of *X* by *n*, where *n* is the number of each case. You can specify multiple variables, as in:

```
GRAPH /BAR = VALUE(SALARY BONUS BENEFIT).
```

This command draws a bar chart with the values of *SALARY*, *BONUS*, and *BENEFIT* for each employee (case). A BY variable can be used to supply case labels, but it does not affect the layout of the chart, even if values of the BY variable are the same for multiple cases.

Aggregation functions:

Two groups of aggregation functions are available: count functions and summary functions.

Count functions:

COUNT. Frequency of cases in each category.

PCT. Frequency of cases in each category expressed as a percentage of the whole.

CUPCT. Cumulative percentage sorted by category value.

CUFREQ. Cumulative frequency sorted by category value.

- Count functions yield the count or percentage of valid cases within categories determined by one or more BY variables, as in:

```
GRAPH /BAR (SIMPLE) = PCT BY REGION.
```

- Count functions do not have any arguments.
- You can omit the keyword COUNT and the subsequent keyword BY and specify just a variable, as in

```
GRAPH /BAR = DEPT.
```

This command is interpreted as

```
GRAPH /BAR = COUNT BY DEPT.
```

Summary functions:

MINIMUM. Minimum value of the variable.

MAXIMUM. Maximum value of the variable.

N. Number of cases for which the variable has a nonmissing value.

SUM. Sum of the values of the variable.

CUSUM. Sum of the summary variable accumulated across values of the category variable.

MEAN. Mean.

STDDEV. Standard deviation.

VARIANCE. Variance.

MEDIAN. Median.

GMEDIAN. Group median.

MODE. Mode.

PTILE(x). Xth percentile value of the variable. X must be greater than 0 and less than 100.

PLT(x). Percentage of cases for which the value of the variable is less than x.

PGT(x). Percentage of cases for which the value of the variable is greater than x.

NLT(x). Number of cases for which the value of the variable is less than x.

NGT(x). Number of cases for which the value of the variable is greater than x.

PIN(x1,x2) . Percentage of cases for which the value of the variable is greater than or equal to x1 and less than or equal to x2. x1 cannot exceed x2.

NIN(x1,x2) . Number of cases for which the value of the variable is greater than or equal to x1 and less than or equal to x2. x1 cannot exceed x2.

- Summary functions are usually used with summary variables (variables that record continuous values, such as age or expenses). To use a summary function, specify the name of one or more variables in parentheses after the name of the function, as in:

```
GRAPH /BAR = SUM(SALARY) BY DEPT.
```

- You can specify multiple summary functions for more chart types. For example, the same function can be applied to a list of variables, as in:

```
GRAPH /BAR = SUM(SALARY BONUS BENEFIT) BY DEPT.
```

This syntax is equivalent to:

```
GRAPH /BAR = SUM(SALARY) SUM(BONUS) SUM(BENEFIT) BY DEPT.
```

Different functions can be applied to the same variable, as in:

```
GRAPH /BAR = MEAN(SALARY) MEDIAN(SALARY) BY DEPT.
```

Different functions and variables can be combined, as in:

```
GRAPH /BAR = MIN(SALARY81) MAX(SALARY81)
                MIN(SALARY82) MAX(SALARY82) BY JOBCAT.
```

The effect of multiple summary functions on the structure of the charts is illustrated under the discussion of specific chart types.

Overview

GRAPH creates bar charts, pie charts, line charts, error bar charts, high-low-close charts, scatterplots, and Pareto charts.

Options

Titles and Footnotes. You can specify a title, subtitle, and footnote for the chart using the TITLE, SUBTITLE, and FOOTNOTE subcommands.

Chart Type. You can request a specific type of chart using the BAR, LINE, PIE, ERRORBAR, HILO, HISTOGRAM, SCATTERPLOT, or PARETO subcommand.

Chart Content. You can specify an aggregated categorical chart using various aggregation functions or a nonaggregated categorical chart using the VALUE function.

Templates. You can specify a template, using the TEMPLATE subcommand, to override the default chart attribute settings on your system.

Basic Specification

The basic specification is a chart type subcommand. By default, the generated chart will have no title, subtitle, or footnote.

Subcommand Order

Subcommands can be specified in any order.

Syntax Rules

- Only one chart type subcommand can be specified.
- The function/variable specification is required for all subtypes of bar, line, error bar, hilo, and Pareto charts; the variable specification is required for histograms and all subtypes of scatterplots.
- The function/variable or variable specifications should match the subtype keywords. If there is a discrepancy, GRAPH produces the default chart for the function/variable or variable specification regardless of the specified keyword.

Operations

- GRAPH computes aggregated functions to obtain the values needed for the requested chart and calculates an optimal scale for charting.

- The chart title, subtitle, and footnote are assigned as they are specified on the TITLE, SUBTITLE, and FOOTNOTE subcommands. If you do not use these subcommands, the chart title, subtitle, and footnote are null. The split-file information is displayed as a subtitle if split-file is in effect.
- GRAPH creates labels that provide information about the source of the values being plotted. Labeling conventions vary for different subtypes. Where variable or value labels are defined in the active dataset, GRAPH uses the labels; otherwise, variable names or values are used.

Limitations

Categorical charts cannot display fewer than 2 or more than 3,000 categories.

Examples

```
GRAPH /BAR=SUM (MURDER) BY CITY.
```

- This command generates a simple (default) bar chart showing the number of murders in each city.
- The category axis (*x* axis) labels are defined by the value labels (or values if no value labels exist) of the variable *CITY.*
- The default span (2) and sigma value (3) are used.
- Since no BY variable is specified, the *x* axis is labeled by sequence numbers.

TITLE, SUBTITLE, and FOOTNOTE Subcommands

TITLE, SUBTITLE, and FOOTNOTE specify lines of text placed at the top or bottom of the chart.

- One or two lines of text can be specified for TITLE or FOOTNOTE, and one line of text can be specified for SUBTITLE.
- Each line of text must be enclosed in quotes. The maximum length of any line is 72 characters.
- The default font sizes and types are used for the title, subtitle, and footnote.
- By default, the title, subtitle, and footnote are left-aligned with the *y* axis.
- If you do not specify TITLE, the default title, subtitle, and footnote are null, which leaves more space for the chart. If split-file processing is in effect, the split-file information is provided as a default subtitle.

Example

```
GRAPH TITLE = 'Murder in Major U.S. Cities'
 /SUBTITLE='per 100,000 people'
 /FOOTNOTE='The above data was reported on August 26, 1987'
 /BAR=SUM(MURDER) BY CITY.
```

BAR Subcommand

BAR creates one of five types of bar charts using the keywords SIMPLE, COMPOSITIONAL, GROUPED, STACKED, or RANGE.

- Only one keyword can be specified, and it must be specified in parentheses.
- When no keyword is specified, the default is either SIMPLE or GROUPED, depending on the type of function/variable specification.

SIMPLE. *Simple bar chart.* This is the default if no keyword is specified on the BAR subcommand and the variables define a simple bar chart. A simple bar chart can be defined by a single summary or count function and a single BY variable or by multiple summary functions and no BY variable.

GROUPED. *Clustered bar chart.* A clustered bar chart is defined by a single function and two BY variables or by multiple functions and a single BY variable. This is the default if no keyword is specified on the BAR subcommand and the variables define a clustered bar chart.

STACKED. *Stacked bar chart.* A stacked bar chart displays a series of bars, each divided into segments stacked one on top of the other. The height of each segment represents the value of the category. Like a clustered bar chart, it is defined by a single function and two BY variables or by multiple functions and a single BY variable.

RANGE. *Range bar chart.* A range bar chart displays a series of floating bars. The height of each bar represents the range of the category and its position in the chart indicates the minimum and maximum values. A range bar chart can be defined by a single function and two BY variables or by multiple functions and a single BY variable. If a variable list is used as the argument for a function, the list must be of an even number. If a second BY variable is used to define the range, the variable must be dichotomous.

LINE Subcommand

LINE creates one of five types of line charts using the keywords SIMPLE, MULTIPLE, DROP, AREA, or DIFFERENCE.

- Only one keyword can be specified, and it must be specified in parentheses.
- When no keyword is specified, the default is either SIMPLE or MULTIPLE, depending on the type of function/variable specification.

SIMPLE. *Simple line chart.* A simple line chart is defined by a single function and a single BY variable or by multiple functions and no BY keyword. This is the default if no keyword is specified on LINE and the data define a simple line.

MULTIPLE. *Multiple line chart.* A multiple line chart is defined by a single function and two BY variables or by multiple functions and a single BY variable. This is the default if no keyword is specified on LINE and the data define a multiple line.

DROP. *Drop-line chart.* A drop-line chart shows the difference between two or more fluctuating variables. It is defined by a single function and two BY variables or by multiple functions and a single BY variable.

AREA. *Area line chart.* An area line chart fills the area beneath each line with a color or pattern. When multiple lines are specified, the second line is the sum of the first and second variables, the third line is the sum of the first, second, and third variables, and so on. The specification is the same as that for a simple or multiple line chart.

DIFFERENCE. *Difference area chart.* A difference area chart fills the area between a pair of lines. It highlights the difference between two variables or two groups. A difference area chart is defined by a single function and two BY variables or by two summary functions and a single BY variable. If a second BY variable is used to define the two groups, the variable must be dichotomous.

PIE Subcommand

PIE creates pie charts. A pie chart can be defined by a single function and a single BY variable or by multiple summary functions and no BY variable. A pie chart divides a circle into slices. The size of each slice indicates the value of the category relative to the whole. Cumulative functions (CUPCT, CUFREQ, and CUSUM) are inappropriate for pie charts but are not prohibited. When specified, all cases except those in the last category are counted more than once in the resulting pie.

HILO Subcommand

HILO creates one of two types of high-low-close charts using the keywords SIMPLE or GROUPED. High-low-close charts show the range and the closing (or average) value of a series.

- Only one keyword can be specified.

- When a keyword is specified, it must be specified in parentheses.
- When no keyword is specified, the default is either SIMPLE or GROUPED, depending on the type of function/variable specification.

SIMPLE. *Simple high-low-close chart.* A simple high-low-close chart can be defined by a single summary or count function and two BY variables, by three summary functions and one BY variable, or by three values with one or no BY variable. When a second BY variable is used to define a high-low-close chart, the variable must be dichotomous or trichotomous. If dichotomous, the first value defines low and the second value defines high; if trichotomous, the first value defines high, the second defines low, and the third defines close.

GROUPED. *Grouped high-low-close chart.* A grouped high-low-close chart is defined by a single function and two BY variables or by multiple functions and a single BY variable. When a variable list is used for a single function, the list must contain two or three variables. If it contains two variables, the first defines the high value and the second defines the low value. If it contains three variables, the first defines the high value, the second defines the low value, and the third defines the close value. Likewise, if multiple functions are specified, they must be in groups of either two or three. The first function defines the high value, the second defines the low value, and the third, if specified, defines the close value.

ERRORBAR Subcommand

ERRORBAR creates either a simple or a clustered error bar chart, depending on the variable specification on the subcommand. A simple error bar chart is defined by one numeric variable with or without a BY variable or a variable list. A clustered error bar chart is defined by one numeric variable with two BY variables or a variable list with a BY variable.

Error bar charts can display confidence intervals, standard deviations, or standard errors of the mean. To specify the statistics to be displayed, one of the following keywords is required:

CI value. *Display confidence intervals for mean.* You can specify a confidence level between 50 and 99.9. The default is 95.

STERROR n. *Display standard errors of mean.* You can specify any positive number for *n*. The default is 2.

STDDEV n. *Display standard deviations.* You can specify any positive number for *n*. The default is 2.

SCATTERPLOT Subcommand

SCATTERPLOT produces two- or three-dimensional scatterplots. Multiple two-dimensional plots can be plotted within the same frame or as a scatterplot matrix. Only variables can be specified; aggregated functions cannot be plotted. When SCATTERPLOT is specified without keywords, the default is BIVARIATE.

BIVARIATE. *One two-dimensional scatterplot*. A basic scatterplot is defined by two variables separated by the keyword WITH. This is the default when SCATTERPLOT is specified without keywords.

OVERLAY. *Multiple plots drawn within the same frame.* Specify a variable list on both sides of WITH. By default, one scatterplot is drawn for each combination of variables on the left of WITH with variables on the right. You can specify PAIR in parentheses to indicate that the first variable on the left is paired with the first variable on the right, the second variable on the left with the second variable on the right, and so on. All plots are drawn within the same frame and are differentiated by color or pattern. The axes are scaled to accommodate the minimum and maximum values across all variables.

MATRIx. *Scatterplot matrix.* Specify at least two variables. One scatterplot is drawn for each combination of the specified variables above the diagonal and a second below the diagonal in a square matrix.

XYZ. *One three-dimensional plot.* Specify three variables, each separated from the next with the keyword WITH.

- If you specify a control variable using BY, GRAPH produces a control scatterplot where values of the BY variable are indicated by different colors or patterns. A control variable cannot be specified for overlay plots.
- You can display the value label of an identification variable at the plotting position for each case by adding BY *var* (NAME) or BY *var* (IDENTIFY) to the end of any valid scatterplot specification. When the chart is created, NAME turns the labels on, while IDENTIFY turns the labels off. You can use the Point Selection tool to turn individual labels off or on in the scatterplot.

HISTOGRAM Subcommand

HISTOGRAM creates a histogram.

- Only one variable can be specified on this subcommand.
- GRAPH divides the values of the variable into several evenly spaced intervals and produces a bar chart showing the number of times the values for the variable fall within each interval.
- You can request a normal distribution line by specifying the keyword NORMAL in parentheses.

PARETO Subcommand

PARETO creates one of two types of Pareto charts. A Pareto chart is used in quality control to identify the few problems that create the majority of nonconformities. Only SUM, VALUE, and COUNT can be used with the PARETO subcommand.

Before plotting, PARETO sorts the plotted values in descending order by category. The right axis is always labeled by the cumulative percentage from 0 to 100. By default, a cumulative line is displayed. You can eliminate the cumulative line or explicitly request it by specifying one of the following keywords:

CUM. *Display the cumulative line.* This is the default.

NOCUM. *Do not display the cumulative line.*

You can request a simple or a stacked Pareto chart by specifying one of the following keywords and define it with appropriate function/variable specifications:

SIMPLE. *Simple Pareto chart.* Each bar represents one type of nonconformity. A simple Pareto chart can be defined by a single variable, a single VALUE function, a single SUM function with a BY variable, or a SUM function with a variable list as an argument with no BY variable.

STACKED. *Stacked Pareto chart.* Each bar represents one or more types of nonconformity within the category. A stacked Pareto chart can be defined by a single SUM function with two BY variables, a single variable with a BY variable, a VALUE function with a variable list as an argument, or a SUM function with a variable list as an argument and a BY variable.

PANEL Subcommand

The PANEL subcommand specifies the variables and method used for paneling. Each keyword in the subcommand is followed by an equals sign (=) and the value for that keyword.

COLVAR and ROWVAR Keywords

The COLVAR and ROWVAR keywords identify the column and row variables, respectively. Each category in a column variable appears as a vertical column in the resulting chart. Each category in a row variable appears as a horizontal row in the resulting chart.

- If multiple variables are specified for a keyword, the COLOP and ROWOP keywords can be used to change the way in which variable categories are rendered in the chart.
- The ROWVAR keyword is not available for population pyramids.

varlist. *The list of variables used for paneling.*

Examples

```
GRAPH
  /BAR(SIMPLE)=COUNT BY educ
  /PANEL COLVAR=gender COLOP=CROSS
```

- There are two columns in the resulting paneled chart, one for each gender.
- Because there is only one paneling variable, there are only as many panels as there are variable values. Therefore, there are two panels.

```
GRAPH
  /BAR(SIMPLE)=COUNT BY educ
  /PANEL COLVAR=minority ROWVAR=jobcat.
```

- There are two columns in the resulting paneled chart (for the *gender* variable values) and three rows (for the *jobcat* variable values).

COLOP and ROWOP Keywords

The COLOP and ROWOP keywords specify the paneling method for the column and row variables, respectively. These keywords have no effect on the chart if there is only one variable in the rows and/or columns. They also have no effect if the data are not nested.

CROSS. *Cross variables in the rows or columns.* When the variables are crossed, a panel is created for every combination of categories in the variables. For example, if the categories in one variable are A and B and the categories in another variable are 1 and 2, the resulting chart will display a panel for the combinations of A and 1, A and 2, B and 1, and B and 2. A panel can be empty if the categories in that panel do not cross (for example, if there are no cases in the B category *and* the 1 category). This is the default.

NEST. *Nest variables in the rows or columns.* When the variables are nested, a panel is created for each category that is nested in the parent category. For example, if the data contain variables for states and cities, a panel is created for each city and the relevant state. However, panels are not created for cities that are not in certain states, as would happen with CROSS. When nesting, make sure that the variables specified for ROWVAR or COLVAR are in the correct order. Parent variables precede child variables.

Example

Assume you have the following data:

Table 62. Nested data

State	City	Temperature
NJ	Springfield	70
MA	Springfield	60
IL	Springfield	50
NJ	Trenton	70
MA	Boston	60

You can create a paneled chart from these data with the following syntax:

```
GRAPH
  /HISTOGRAM=temperature
  /PANEL COLVAR=state city COLOP=CROSS.
```

The command crosses every variable value to create the panels. Because not every state contains every city, the resulting paneled chart will contain blank panels. For example, there will be a blank panel for

Springfield and New Jersey. In this dataset, the *city* variable is really nested in the *state* variable. To nest the variables in the panels and eliminate any blank panels, use the following syntax:

```
GRAPH
  /HISTOGRAM=temperature
  /PANEL COLVAR=state city COLOP=NEST.
```

INTERVAL Subcommand

The INTERVAL subcommand adds errors bars to the chart. This is different from the ERRORBAR subcommand. The ERRORBAR subcommand adds error bar data elements. INTERVAL adds errors bars to other data elements (for example, areas, bars, and lines).

Error bars indicate the variability of the summary statistic being displayed. The length of the error bar on either side of the summary statistic represents a confidence interval or a specified number of standard errors or standard deviations. GRAPH supports error bars for simple or clustered categorical charts displaying means, medians, counts, and percentages.

The keywords are not followed by an equals sign (=). They are followed by a value in parentheses.

Example

```
GRAPH
  /BAR(SIMPLE)=COUNT BY jobcat
  /INTERVAL CI(95).
```

CI Keyword

(value). *The percentage of the confidence interval to use as the length of the error bars.*

STDDEV Keyword

(value). *A multiplier indicating the number of standard deviations to use as the length of the error bars.*

SE Keyword

(value). *A multiplier indicating the number of standard errors to use as the length of the error bars.*

TEMPLATE Subcommand

TEMPLATE uses an existing chart as a template and applies it to the chart requested by the current GRAPH command.

- The specification on TEMPLATE is a chart file saved during a previous session.
- The general rule of application is that the template overrides the default setting, but the specifications on the current GRAPH command override the template. Nonapplicable elements and attributes are ignored.
- Three types of elements and attributes can be applied from a chart template: those dependent on data, those dependent on the chart type, and those dependent on neither.

Elements and Attributes Independent of Chart Types or Data

Elements and attributes common to all chart types are always applied unless overridden by the specifications on the current GRAPH command.

- The title, subtitle, and footnote, including text, color, font type and size, and line alignment are always applied. To give your chart a new title, subtitle, or footnote, specify the text on the TITLE, SUBTITLE, or FOOTNOTE subcommand. You cannot change other attributes.

- The outer frame of the chart, including line style, color, and fill pattern, is always applied. The inner frame is applied except for those charts that do not have an inner frame. The template overrides the system default.
- Label formats are applied wherever applicable. The template overrides the system default. Label text, however, is not applied. GRAPH automatically provides axis labels according to the function/variable specification.
- Legends and the legend title attributes, including color, font type and size, and alignment, are applied provided the current chart requires legends. The legend title text, however, is not applied. GRAPH provides the legend title according to the function/variable specification.

Elements and Attributes Dependent on Chart Type

Elements and attributes dependent on the chart type are those that exist only in a specific chart type. They include bars (in bar charts), lines and areas (in line charts), markers (in scatterplots), boxes (in boxplots), and pie sectors (in pie charts). These elements and their attributes are usually applied only when the template chart and the requested chart are of the same type. Some elements or their attributes may override the default settings across chart type.

- Color and pattern are always applied except for pie charts. The template overrides the system default.
- Scale axis lines are applied across chart types.
- Interval axis lines are applied from interval axis to interval axis. Interval axis bins are never applied.
- If the template is a 3-D bar chart and you request a chart with one category axis, attributes of the first axis are applied from the template. If you request a 3-D bar chart and the template is not a 3-D chart, no category axis attributes are applied.

Elements and Attributes Dependent on Data

Data-dependent elements and attributes are applied only when the template and the requested chart are of the same type and the template has at least as many series assigned to the same types of chart elements as the requested chart.

- Category attributes and elements, including fill, border, color, pattern, line style, weight of pie sectors, pie sector explosion, reference lines, projection lines, and annotations, are applied only when category values in the requested chart match those in the template.
- The attributes of data-related elements with on/off states are always applied. For example, the line style, weight, and color of a quadratic fit in a simple bivariate scatterplot are applied if the requested chart is also a simple bivariate scatterplot. The specification on the GRAPH command, for example, HISTOGRAM(NORMAL), overrides the applied on/off status; in this case, a normal curve is displayed regardless of whether the template displays a normal curve.
- In bar, line, and area charts, the assignment of series to bars, lines, and areas is not applied.

MISSING Subcommand

MISSING controls the treatment of missing values in the chart drawn by GRAPH.

- The default is LISTWISE.
- The MISSING subcommand has no effect on variables used with the VALUE function to create nonaggregated charts. User-missing and system-missing values create empty cells.
- LISTWISE and VARIABLE are alternatives and apply to variables used in summary functions for a chart or to variables being plotted in a scatterplot.
- REPORT and NOREPORT are alternatives and apply only to category variables. They control whether categories and series with missing values are created. NOREPORT is the default.
- INCLUDE and EXCLUDE are alternatives and apply to both summary and category variables. EXCLUDE is the default.

- When a case has a missing value for the name variable but contains valid values for the dependent variable in a scatterplot, the case is always included. User-missing values are displayed as point labels; system-missing values are not displayed.
- For an aggregated categorical chart, if every aggregated series is empty in a category, the empty category is excluded.
- A nonaggregated categorical chart created with the VALUE function can contain completely empty categories. There are always as many categories as rows of data. However, at least one nonempty cell must be present; otherwise the chart is not created.

LISTWISE. *Listwise deletion of cases with missing values.* A case with a missing value for any dependent variable is excluded from computations and graphs.

VARIABLE. *Variable-wise deletion.* A case is deleted from the analysis only if it has a missing value for the dependent variable being analyzed.

NOREPORT. *Suppress missing-value categories.* This is the default.

REPORT. *Report and graph missing-value categories.*

EXCLUDE. *Exclude user-missing values.* Both user- and system-missing values for dependent variables are excluded from computations and graphs. This is the default.

INCLUDE. *Include user-missing values.* Only system-missing values for dependent variables are excluded from computations and graphs.

HILOGLINEAR

HILOGLINEAR is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
HILOGLINEAR {varlist} (min,max) [varlist ...]
            {ALL    }

 [/METHOD [= BACKWARD]]

 [/MAXORDER = k]

 [/CRITERIA = [CONVERGE({0.25**})] [ITERATE({20**})] [P({0.05**})]
                        {n     }          {n   }       {prob  }
              [DELTA({0.5**})] [MAXSTEPS({10**})]
                     {d    }             {n   }
              [DEFAULT] ]

 [/CWEIGHT = {varname }]
             {(matrix)}

 [/PRINT = {[FREQ**] [RESID**] [ESTIM**][ASSOCIATION**]}]
           {DEFAULT**                                  }
           {ALL                                        }
           {NONE                                       }

 [/PLOT = [{NONE**            } ]
           {DEFAULT           }
           {[RESID] [NORMPROB]}
           {ALL               }

 [/MISSING = [{EXCLUDE**}]]
              {INCLUDE  }

 [/DESIGN = effectname effectname*effectname ...]
```

** Default if subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
HILOGLINEAR V1(1,2) V2(1,2)
  /DESIGN=V1*V2.
```

Overview

HILOGLINEAR fits hierarchical loglinear models to multidimensional contingency tables using an iterative proportional-fitting algorithm. HILOGLINEAR also estimates parameters for saturated models. These techniques are described elsewhere in [32], [33], and [34]. HILOGLINEAR is much more efficient for these models than the LOGLINEAR procedure because HILOGLINEAR uses an iterative proportional-fitting algorithm rather than the Newton-Raphson method used in LOGLINEAR.

Options

Design Specification. You can request automatic model selection using backward elimination with the METHOD subcommand. You can also specify any hierarchical design and request multiple designs using the DESIGN subcommand.

Design Control. You can control the criteria used in the iterative proportional-fitting and model-selection routines with the CRITERIA subcommand. You can also limit the order of effects in the model with the MAXORDER subcommand and specify structural zeros for cells in the tables you analyze with the CWEIGHT subcommand.

[32] Everitt, B. S. 1977. *The Analysis of Contingency Tables*. London: Chapman & Hall.
[33] Bishop, Y. M., S. E. Feinberg, and P. W. Holland. 1975. *Discrete multivariate analysis: Theory and practice*. Cambridge, Mass.: MIT Press.
[34] Goodman, L. A. 1978. *Analyzing qualitative/categorical data*. New York: University Press of America.

Display and Plots. You can select the display for each design with the PRINT subcommand. For saturated models, you can request tests for different orders of effects as well. With the PLOT subcommand, you can request residuals plots or normal probability plots of residuals.

Basic Specification

- The basic specification is a variable list with at least two variables followed by their minimum and maximum values.
- HILOGLINEAR estimates a saturated model for all variables in the analysis.
- By default, HILOGLINEAR displays parameter estimates, measures of partial association, goodness of fit, and frequencies for the saturated model.

Subcommand Order

- The variable list must be specified first.
- Subcommands affecting a given DESIGN must appear before the DESIGN subcommand. Otherwise, subcommands can appear in any order.
- MISSING can be placed anywhere after the variable list.

Syntax Rules

- DESIGN is optional. If DESIGN is omitted or the last specification is not a DESIGN subcommand, a default saturated model is estimated.
- You can specify multiple PRINT, PLOT, CRITERIA, MAXORDER, and CWEIGHT subcommands. The last of each type specified is in effect for subsequent designs.
- PRINT, PLOT, CRITERIA, MAXORDER, and CWEIGHT specifications remain in effect until they are overridden by new specifications on these subcommands.
- You can specify multiple METHOD subcommands, but each one affects only the next design.
- MISSING can be specified only once.

Operations

- HILOGLINEAR builds a contingency table using all variables on the variable list. The table contains a cell for each possible combination of values within the range specified for each variable.
- HILOGLINEAR assumes that there is a category for every integer value in the range of each variable. Empty categories waste space and can cause computational problems. If there are empty categories, use the RECODE command to create consecutive integer values for categories.
- Cases with values outside the range specified for a variable are excluded.
- If the last subcommand is not a DESIGN subcommand, HILOGLINEAR displays a warning and generates the default model. This is the saturated model unless MAXORDER is specified. This model is in addition to any that are explicitly requested.
- If the model is not saturated (for example, when MAXORDER is less than the number of factors), only the goodness of fit and the observed and expected frequencies are given.
- The display uses the WIDTH subcommand defined on the SET command. If the defined width is less than 132, some portions of the display may be deleted.

Limitations

The HILOGLINEAR procedure cannot estimate all possible frequency models, and it produces limited output for unsaturated models.

- It can estimate only hierarchical loglinear models.
- It treats all table variables as nominal. (You can use LOGLINEAR to fit nonhierarchical models to tables involving variables that are ordinal.)
- It can produce parameter estimates for saturated models only (those with all possible main-effect and interaction terms).
- It can estimate partial associations for saturated models only.

- It can handle tables with no more than 10 factors.

Example

```
HILOGLINEAR V1(1,2) V2(1,2) V3(1,3) V4(1,3)
  /DESIGN=V1*V2*V3, V4.
```

- HILOGLINEAR builds a 2 × 2 × 3 × 3 contingency table for analysis.
- DESIGN specifies the generating class for a hierarchical model. This model consists of main effects for all four variables, two-way interactions among *V1*, *V2*, and *V3*, and the three-way interaction term *V1* by *V2* by *V3*.

Variable List

The required variable list specifies the variables in the analysis. The variable list must precede all other subcommands.

- Variables must be numeric and have integer values. If a variable has a fractional value, the fractional portion is truncated.
- Keyword ALL can be used to refer to all user-defined variables in the active dataset.
- A range must be specified for each variable, with the minimum and maximum values separated by a comma and enclosed in parentheses.
- If the same range applies to several variables, the range can be specified once after the last variable to which it applies.
- If ALL is specified, all variables must have the same range.

METHOD Subcommand

By default, HILOGLINEAR tests the model specified on the DESIGN subcommand (or the default model) and does not perform any model selection. All variables are entered and none are removed. Use METHOD to specify automatic model selection using backward elimination for the next design specified.

- You can specify METHOD alone or with the keyword BACKWARD for an explicit specification.
- When the backward-elimination method is requested, a step-by-step output is displayed regardless of the specification on the PRINT subcommand.
- METHOD affects only the next design.

BACKWARD. *Backward elimination.* Perform backward elimination of terms in the model. All terms are entered. Those that do not meet the P criterion specified on the CRITERIA subcommand (or the default P) are removed one at a time.

MAXORDER Subcommand

MAXORDER controls the maximum order of terms in the model estimated for subsequent designs. If MAXORDER is specified, HILOGLINEAR tests a model only with terms of that order or less.

- MAXORDER specifies the highest-order term that will be considered for the next design. MAXORDER can thus be used to abbreviate computations for the BACKWARD method.
- If the integer on MAXORDER is less than the number of factors, parameter estimates and measures of partial association are not available. Only the goodness of fit and the observed and expected frequencies are displayed.
- You can use MAXORDER with backward elimination to find the best model with terms of a certain order or less. This is computationally much more efficient than eliminating terms from the saturated model.

Example

```
HILOGLINEAR V1 V2 V3(1,2)
  /MAXORDER=2
  /DESIGN=V1 V2 V3
  /DESIGN=V1*V2*V3.
```

- HILOGLINEAR builds a 2 × 2 × 2 contingency table for *V1*, *V2*, and *V3*.
- MAXORDER has no effect on the first DESIGN subcommand because the design requested considers only main effects.
- MAXORDER restricts the terms in the model specified on the second DESIGN subcommand to two-way interactions and main effects.

CRITERIA Subcommand

Use the CRITERIA subcommand to change the values of constants in the iterative proportional-fitting and model-selection routines for subsequent designs.

- The default criteria are in effect if the CRITERIA subcommand is omitted (see below).
- You cannot specify the CRITERIA subcommand without any keywords.
- Specify each CRITERIA keyword followed by a criterion value in parentheses. Only those criteria specifically altered are changed.
- You can specify more than one keyword on CRITERIA, and they can be in any order.

DEFAULT. *Reset parameters to their default values.* If you have specified criteria other than the defaults for a design, use this keyword to restore the defaults for subsequent designs.

CONVERGE(n). *Convergence criterion*. The default is 10^{-3} times the largest cell size, or 0.25, whichever is larger.

ITERATE(n). *Maximum number of iterations.* The default is 20.

P(n). *Probability for change in chi-square if term is removed.* Specify a value between (but not including) 0 and 1 for the significance level. The default is 0.05. P is in effect only when you request BACKWARD on the METHOD subcommand.

MAXSTEPS(n). *Maximum number of steps for model selection.* Specify an integer between 1 and 99, inclusive. The default is 10.

DELTA(d). *Cell delta value.* The value of delta is added to each cell frequency for the first iteration when estimating saturated models; it is ignored for unsaturated models. The default value is 0.5. You can specify any decimal value between 0 and 1 for *d*. HILOGLINEAR does not display parameter estimates or the covariance matrix of parameter estimates if any zero cells (either structural or sampling) exist in the expected table after delta is added.

CWEIGHT Subcommand

CWEIGHT specifies cell weights for a model. CWEIGHT is typically used to specify structural zeros in the table. You can also use CWEIGHT to adjust tables to fit new margins.

- You can specify the name of a variable whose values are cell weights, or provide a matrix of cell weights enclosed in parentheses.
- If you use a variable to specify cell weights, you are allowed only one CWEIGHT subcommand.
- If you specify a matrix, you must provide a weight for every cell in the contingency table, where the number of cells equals the product of the number of values of all variables.
- Cell weights are indexed by the values of the variables in the order in which they are specified on the variable list. The index values of the rightmost variable change the most quickly.
- You can use the notation *n*cw* to indicate that cell weight *cw* is repeated *n* times in the matrix.

Example

```
HILOGLINEAR V1(1,2) V2(1,2) V3(1,3)
  /CWEIGHT=CELLWGT
  /DESIGN=V1*V2, V2*V3, V1*V3.
```

- This example uses the variable *CELLWGT* to assign cell weights for the table. Only one CWEIGHT subcommand is allowed.

Example

```
HILOGLINEAR V4(1,3) V5(1,3)
  /CWEIGHT=(0 1 1  1 0 1  1 1 0)
  /DESIGN=V4, V5.
```

- The HILOGLINEAR command sets the diagonal cells in the model to structural zeros. This type of model is known as a **quasi-independence model**.
- Because both *V4* and *V5* have three values, weights must be specified for nine cells.
- The first cell weight is applied to the cell in which *V4* is 1 and *V5* is 1; the second weight is applied to the cell in which *V4* is 1 and *V5* is 2; and so on.

Example

```
HILOGLINEAR V4(1,3) V5(1,3)
  /CWEIGHT=(0 3*1 0 3*1 0)
  /DESIGN=V4,V5.
```

- This example is the same as the previous example except that the *n*cw* notation is used.

Example

```
* An Incomplete Rectangular Table

DATA LIST FREE / LOCULAR RADIAL FREQ.
WEIGHT BY FREQ.
BEGIN DATA
1 1 462
1 2 130
1 3 2
1 4 1
2 1 103
2 2 35
2 3 1
2 4 0
3 5 614
3 6 138
3 7 21
3 8 14
3 9 1
4 5 443
4 6 95
4 7 22
4 8 8
4 9 5
END DATA.
HILOGLINEAR LOCULAR (1,4) RADIAL (1,9)
  /CWEIGHT=(4*1 5*0   4*1 5*0   4*0 5*1   4*0 5*1)
  /DESIGN LOCULAR RADIAL.
```

- This example uses aggregated table data as input.
- The DATA LIST command defines three variables. The values of *LOCULAR* and *RADIAL* index the levels of those variables, so that each case defines a cell in the table. The values of *FREQ* are the cell frequencies.
- The WEIGHT command weights each case by the value of the variable *FREQ*. Because each case represents a cell in this example, the WEIGHT command assigns the frequencies for each cell.
- The BEGIN DATA and END DATA commands enclose the inline data.
- The HILOGLINEAR variable list specifies two variables. *LOCULAR* has values 1, 2, 3, and 4. *RADIAL* has integer values 1 through 9.
- The CWEIGHT subcommand identifies a block rectangular pattern of cells that are logically empty. There is one weight specified for each cell of the 36-cell table.
- In this example, the matrix form needs to be used in CWEIGHT because the structural zeros do not appear in the actual data. (For example, there is no case corresponding to *LOCULAR* = 1, *RADIAL* = 5.)

- The DESIGN subcommand specifies main effects only for *LOCULAR* and *RADIAL*. Lack of fit for this model indicates an interaction of the two variables.
- Because there is no PRINT or PLOT subcommand, HILOGLINEAR produces the default output for an unsaturated model.

PRINT Subcommand

PRINT controls the display produced for the subsequent designs.

- If PRINT is omitted or included with no specifications, the default display is produced.
- If any keywords are specified on PRINT, only output specifically requested is displayed.
- HILOGLINEAR displays Pearson and likelihood-ratio chi-square goodness-of-fit tests for models. For saturated models, it also provides tests that the *k*-way effects and the *k*-way and higher-order effects are 0.
- Both adjusted and unadjusted degrees of freedom are displayed for tables with sampling or structural zeros. *K*-way and higher-order tests use the unadjusted degrees of freedom.
- The unadjusted degrees of freedom are not adjusted for zero cells, and they estimate the upper bound of the true degrees of freedom. These are the same degrees of freedom you would get if all cells were filled.
- The adjusted degrees of freedom are calculated from the number of non-zero-fitted cells minus the number of parameters that would be estimated if all cells were filled (that is, unadjusted degrees of freedom minus the number of zero-fitted cells). This estimate of degrees of freedom may be too low if some parameters do not exist because of zeros.

DEFAULT. *Default displays*. This option includes FREQ and RESID output for nonsaturated models, and FREQ, RESID, ESTIM, and ASSOCIATION output for saturated models. For saturated models, the observed and expected frequencies are equal, and the residuals are zeros.

FREQ. *Observed and expected cell frequencies.*

RESID. *Raw and standardized residuals.*

ESTIM. *Parameter estimates for a saturated model.*

ASSOCIATION. *Partial associations*. You can request partial associations of effects only when you specify a saturated model. This option is computationally expensive for tables with many factors.

ALL. *All available output.*

NONE. *Design information and goodness-of-fit statistics only.* Use of this option overrides all other specifications on PRINT.

PLOT Subcommand

Use PLOT to request residuals plots.

- If PLOT is included without specifications, standardized residuals and normal probability plots are produced.
- No plots are displayed for saturated models.
- If PLOT is omitted, no plots are produced.

RESID. *Standardized residuals by observed and expected counts.*

NORMPLOT. *Normal probability plots of adjusted residuals.*

NONE. *No plots.* Specify NONE to suppress plots requested on a previous PLOT subcommand. This is the default if PLOT is omitted.

DEFAULT. *Default plots.* Includes RESID and NORMPLOT. This is the default when PLOT is specified without keywords.

ALL. *All available plots.*

MISSING Subcommand

By default, a case with either system-missing or user-missing values for any variable named on the HILOGLINEAR variable list is omitted from the analysis. Use MISSING to change the treatment of cases with user-missing values.

- MISSING can be named only once and can be placed anywhere following the variable list.
- MISSING cannot be used without specifications.
- A case with a system-missing value for any variable named on the variable list is always excluded from the analysis.

EXCLUDE. *Delete cases with missing values.* This is the default if the subcommand is omitted. You can also specify keyword DEFAULT.

INCLUDE. *Include user-missing values as valid.* Only cases with system-missing values are deleted.

DESIGN Subcommand

By default, HILOGLINEAR uses a saturated model that includes all variables on the variable list. The model contains all main effects and interactions for those variables. Use DESIGN to specify a different generating class for the model.

- If DESIGN is omitted or included without specifications, the default model is estimated. When DESIGN is omitted, a warning message is issued.
- To specify a design, list the highest-order terms, using variable names and asterisks (*) to indicate interaction effects.
- In a hierarchical model, higher-order interaction effects imply lower-order interaction and main effects. V1*V2*V3 implies the three-way interaction *V1* by *V2* by *V3*, two-way interactions *V1* by *V2*, *V1* by *V3*, and *V2* by *V3*, and main effects for *V1*, *V2*, and *V3*. The highest-order effects to be estimated are the generating class.
- Any PRINT, PLOT, CRITERIA, METHOD, and MAXORDER subcommands that apply to a DESIGN subcommand must appear before it.
- All variables named on DESIGN must be named or implied on the variable list.
- You can specify more than one DESIGN subcommand. One model is estimated for each DESIGN subcommand.
- If the last subcommand on HILOGLINEAR is not DESIGN, the default model will be estimated in addition to models explicitly requested. A warning message is issued for a missing DESIGN subcommand.

References

Bishop, Y. M., S. E. Feinberg, and P. W. Holland. 1975. *Discrete multivariate analysis: Theory and practice*. Cambridge, Mass.: MIT Press.

Everitt, B. S. 1977. *The Analysis of Contingency Tables*. London: Chapman & Hall.

Goodman, L. A. 1978. *Analyzing qualitative/categorical data*. New York: University Press of America.

HOMALS

HOMALS is available in SPSS Statistics Professional Edition or the Categories option.

```
HOMALS  VARIABLES=varlist(max)

[/ANALYSIS=varlist]

[/NOBSERVATIONS=value]

[/DIMENSION={2**  }]
            {value}

[/MAXITER={100**}]
          {value}

[/CONVERGENCE={.00001**}]
              {value   }

[/PRINT=[DEFAULT**] [FREQ**] [EIGEN**] [DISCRIM**]
        [QUANT**] [OBJECT] [HISTORY] [ALL] [NONE]]

[/PLOT=[NDIM=({1, 2        }**)]
              {value, value}
              {ALL, MAX    }
       [QUANT**[(varlist)][(n)]] [OBJECT**[(varlist)][(n)]]
       [DEFAULT**[(n)]] [DISCRIM[(n)]] [ALL[(n)]] [NONE]]

[/SAVE=[rootname] [(value)]]

[/MATRIX=OUT({*   })]
            {'savfile'|'dataset'}
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
HOMALS  VARIABLES=ACOLA(2) BCOLA(2) CCOLA(2) DCOLA(2).
```

Overview

HOMALS (*hom*ogeneity analysis by means of *a*lternating *l*east *s*quares) estimates category quantifications, object scores, and other associated statistics that separate categories (levels) of nominal variables as much as possible and divide cases into homogeneous subgroups.

Options

Data and variable selection. You can use a subset of the variables in the analysis and restrict the analysis to the first *n* observations.

Number of dimensions. You can specify the number of dimensions HOMALS should compute.

Iterations and convergence. You can specify the maximum number of iterations and the value of a convergence criterion.

Display output. The output can include all available statistics; just the default frequencies, eigenvalues, discrimination measures and category quantifications; or just the specific statistics you request. You can also control which statistics are plotted and specify the number of characters used in plot labels.

Saving scores. You can save object scores in the working data file.

Writing matrices. You can write a matrix data file containing category quantifications for use in further analyses.

Basic Specification

- The basic specification is HOMALS and the VARIABLES subcommand. By default, HOMALS analyzes all of the variables listed for all cases and computes two solutions. Frequencies, eigenvalues, discrimination

measures, and category quantifications are displayed, and category quantifications and object scores are plotted.

Subcommand Order

- Subcommands can appear in any order.

Syntax Rules

- If ANALYSIS is specified more than once, HOMALS is not executed. For all other subcommands, if a subcommand is specified more than once, only the last occurrence is executed.

Operations

- HOMALS treats every value in the range of 1 to the maximum value specified on VARIABLES as a valid category. If the data are not sequential, the empty categories (categories with no valid data) are assigned zeros for all statistics. You may want to use RECODE or AUTORECODE before HOMALS to get rid of these empty categories and avoid the unnecessary output (see RECODE and AUTORECODE for more information).

Limitations

- String variables are not allowed; use AUTORECODE to recode string variables into numeric variables.
- The data (category values) must be positive integers. Zeros and negative values are treated as system-missing, which means that they are excluded from the analysis. Fractional values are truncated after the decimal and are included in the analysis. If one of the levels of a variable has been coded 0 or a negative value and you want to treat it as a valid category, use the AUTORECODE or RECODE command to recode the values of that variable.
- HOMALS ignores user-missing value specifications. Positive user-missing values less than the maximum value specified on the VARIABLES subcommand are treated as valid category values and are included in the analysis. If you do not want the category included, use COMPUTE or RECODE to change the value to something outside of the valid range. Values outside of the range (less than 1 or greater than the maximum value) are treated as system-missing and are excluded from the analysis.

Example

```
HOMALS  VARIABLES=ACOLA(2) BCOLA(2) CCOLA(2) DCOLA(2)
  /PRINT=FREQ EIGEN QUANT OBJECT.
```

- The four variables are analyzed using all available observations. Each variable has two categories, 1 and 2.
- The PRINT subcommand lists the frequencies, eigenvalues, category quantifications, and object scores.
- By default, plots of the category quantifications and the object scores are produced.

VARIABLES Subcommand

VARIABLES specifies the variables that will be used in the analysis.

- The VARIABLES subcommand is required. The actual word VARIABLES can be omitted.
- After each variable or variable list, specify in parentheses the maximum number of categories (levels) of the variables.
- The number specified in parentheses indicates the number of categories *and* the maximum category value. For example, *VAR1(3)* indicates that *VAR1* has three categories coded 1, 2, and 3. However, if a variable is not coded with consecutive integers, the number of categories used in the analysis will differ from the number of observed categories. For example, if a three-category variable is coded {2, 4, 6}, the maximum category value is 6. The analysis treats the variable as having six categories, three of which (categories 1, 3, and 5) are not observed and receive quantifications of 0.

- To avoid unnecessary output, use the AUTORECODE or RECODE command before HOMALS to recode a variable that does not have sequential values (see AUTORECODE and RECODE for more information).

Example

```
DATA LIST FREE/V1 V2 V3.
BEGIN DATA
3 1 1
6 1 1
3 1 3
3 2 2
3 2 2
6 2 2
6 1 3
6 2 2
3 2 2
6 2 1
END DATA.
AUTORECODE V1 /INTO NEWVAR1.
HOMALS VARIABLES=NEWVAR1 V2(2) V3(3).
```

- DATA LIST defines three variables, *V1*, *V2*, and *V3*.
- *V1* has two levels, coded 3 and 6, *V2* has two levels, coded 1 and 2, and *V3* has three levels, coded 1, 2, and 3.
- The AUTORECODE command creates *NEWVAR1* containing recoded values of *V1*. Values of 3 are recoded to 1; values of 6 are recoded to 2.
- The maximum category value for both *NEWVAR1* and *V2* is 2. A maximum value of 3 is specified for *V3*.

ANALYSIS Subcommand

ANALYSIS limits the analysis to a specific subset of the variables named on the VARIABLES subcommand.

- If ANALYSIS is not specified, all variables listed on the VARIABLES subcommand are used.
- ANALYSIS is followed by a variable list. The variables on the list must be specified on the VARIABLES subcommand.
- Variables listed on the VARIABLES subcommand but not on the ANALYSIS subcommand can still be used to label object scores on the PLOT subcommand.

Example

```
HOMALS VARIABLES=ACOLA(2) BCOLA(2) CCOLA(2) DCOLA(2)
  /ANALYSIS=ACOLA BCOLA
  /PRINT=OBJECT QUANT
  /PLOT=OBJECT(CCOLA).
```

- The VARIABLES subcommand specifies four variables.
- The ANALYSIS subcommand limits analysis to the first two variables. The PRINT subcommand lists the object scores and category quantifications from this analysis.
- The plot of the object scores is labeled with variable *CCOLA*, even though this variable is not included in the computations.

NOBSERVATIONS Subcommand

NOBSERVATIONS specifies how many cases are used in the analysis.

- If NOBSERVATIONS is not specified, all available observations in the working data file are used.
- NOBSERVATIONS is followed by an integer indicating that the first *n* cases are to be used.

DIMENSION Subcommand

DIMENSION specifies the number of dimensions you want HOMALS to compute.

- If you do not specify the DIMENSION subcommand, HOMALS computes two dimensions.
- The specification on DIMENSION is a positive integer indicating the number of dimensions.
- The minimum number of dimensions is 1.

MAXITER Subcommand

MAXITER specifies the maximum number of iterations HOMALS can go through in its computations.

- If MAXITER is not specified, HOMALS will iterate up to 100 times.
- The specification on MAXITER is a positive integer indicating the maximum number of iterations.

CONVERGENCE Subcommand

CONVERGENCE specifies a convergence criterion value. HOMALS stops iterating if the difference in total fit between the last two iterations is less than the CONVERGENCE value.

- If CONVERGENCE is not specified, the default value is 0.00001.
- The specification on CONVERGENCE is a positive value.

PRINT Subcommand

PRINT controls which statistics are included in your display output. The default display includes the frequencies, eigenvalues, discrimination measures, and category quantifications.

The following keywords are available:

FREQ. *Marginal frequencies for the variables in the analysis*.

HISTORY. *History of the iterations*.

EIGEN. *Eigenvalues*.

DISCRIM. *Discrimination measures for the variables in the analysis*.

OBJECT. *Object scores*.

QUANT. *Category quantifications for the variables in the analysis*.

DEFAULT. *FREQ, EIGEN, DISCRIM, and QUANT*. These statistics are also displayed when you omit the PRINT subcommand.

ALL. *All available statistics*.

NONE. *No statistics*.

PLOT Subcommand

PLOT can be used to produce plots of category quantifications, object scores, and discrimination measures.

- If PLOT is not specified, plots of the object scores and of the quantifications are produced.
- No plots are produced for a one-dimensional solution.

The following keywords can be specified on PLOT:

DISCRIM. *Plots of the discrimination measures*.

OBJECT. *Plots of the object scores*.

QUANT. *Plots of the category quantifications*.

DEFAULT. *QUANT and OBJECT*.

ALL. *All available plots*.

NONE. *No plots*.

- Keywords OBJECT and QUANT can each be followed by a variable list in parentheses to indicate that plots should be labeled with those variables. For QUANT, the labeling variables must be specified on both the VARIABLES and ANALYSIS subcommands. For OBJECT, the variables must be specified on the VARIABLES subcommand but need not appear on the ANALYSIS subcommand. This means that variables not used in the computations can be used to label OBJECT plots. If the variable list is omitted, the default object and quantification plots are produced.
- Object score plots labeled with variables that appear on the ANALYSIS subcommand use category labels corresponding to all categories within the defined range. Objects in a category that is outside the defined range are labeled with the label corresponding to the category immediately following the defined maximum category value.
- Object score plots labeled with variables not included on the ANALYSIS subcommand use all category labels, regardless of whether or not the category value is inside the defined range.
- All keywords except NONE can be followed by an integer value in parentheses to indicate how many characters of the variable or value label are to be used on the plot. (If you specify a variable list after OBJECT or QUANT, specify the value in parentheses after the list.) The value can range from 1 to 20; the default is to use 12 characters. Spaces between words count as characters.
- DISCRIM plots use variable labels; all other plots use value labels.
- If a variable label is not supplied, the variable name is used for that variable. If a value label is not supplied, the actual value is used.
- Variable and value labels should be unique.
- When points overlap, the points involved are described in a summary following the plot.

Example

```
HOMALS VARIABLES COLA1 (4) COLA2 (4) COLA3 (4) COLA4 (2)
/ANALYSIS COLA1 COLA2 COLA3 COLA4
/PLOT OBJECT(COLA4).
```

- Four variables are included in the analysis.
- OBJECT requests a plot of the object scores labeled with the values of *COLA4*. Any object whose *COLA4* value is not 1 or 2, is labeled 3 (or the value label for category 3, if supplied).

Example

```
HOMALS VARIABLES COLA1 (4) COLA2 (4) COLA3 (4) COLA4 (2)
/ANALYSIS COLA1 COLA2 COLA3
/PLOT OBJECT(COLA4).
```

- Three variables are included in the analysis.
- OBJECT requests a plot of the object scores labeled with the values of *COLA4,* a variable not included in the analysis. Objects are labeled using all values of *COLA4.*

In addition to the plot keywords, the following can be specified:

NDIM. *Dimension pairs to be plotted*. NDIM is followed by a pair of values in parentheses. If NDIM is not specified, plots are produced for dimension 1 versus dimension 2.

- The first value indicates the dimension that is plotted against all higher dimensions. This value can be any integer from 1 to the number of dimensions minus 1.
- The second value indicates the highest dimension to be used in plotting the dimension pairs. This value can be any integer from 2 to the number of dimensions.
- Keyword ALL can be used instead of the first value to indicate that all dimensions are paired with higher dimensions.

- Keyword MAX can be used instead of the second value to indicate that plots should be produced up to and including the highest dimension fit by the procedure.

Example

```
HOMALS COLA1 COLA2 COLA3 COLA4 (4)
 /PLOT NDIM(1,3) QUANT(5).
```

- The NDIM(1,3) specification indicates that plots should be produced for two dimension pairs—dimension 1 versus dimension 2 and dimension 1 versus dimension 3.
- QUANT requests plots of the category quantifications. The (5) specification indicates that the first five characters of the value labels are to be used on the plots.

Example

```
HOMALS COLA1 COLA2 COLA3 COLA4 (4)
 /PLOT NDIM(ALL,3) QUANT(5).
```

- This plot is the same as above except for the ALL specification following NDIM. This indicates that all possible pairs up to the second value should be plotted, so QUANT plots will be produced for dimension 1 versus dimension 2, dimension 2 versus dimension 3, and dimension 1 versus dimension 3.

SAVE Subcommand

SAVE lets you add variables containing the object scores computed by HOMALS to the working data file.

- If SAVE is not specified, object scores are not added to the working data file.
- A variable rootname can be specified on the SAVE subcommand to which HOMALS adds the number of the dimension. Only one rootname can be specified and it can contain up to six characters.
- If a rootname is not specified, unique variable names are automatically generated. The variable names are *HOM n_m*, where *n* is a dimension number and *m* is a set number. If three dimensions are saved, the first set of names is *HOM1_1*, *HOM2_1*, and *HOM3_1*. If another HOMALS is then run, the variable names for the second set are *HOM1_2*, *HOM2_2*, *HOM3_2*, and so on.
- Following the rootname, the number of dimensions for which you want to save object scores can be specified in parentheses. The number cannot exceed the value on the DIMENSION subcommand.
- If the number of dimensions is not specified, the SAVE subcommand saves object scores for all dimensions.
- If you replace the working data file by specifying an asterisk (*) on a MATRIX subcommand, the SAVE subcommand is not executed.

Example

```
HOMALS CAR1 CAR2 CAR3 CAR4(5)
 /DIMENSION=3
 /SAVE=DIM(2).
```

- Four variables, each with five categories, are analyzed.
- The DIMENSION subcommand specifies that results for three dimensions will be computed.
- SAVE adds the object scores from the first two dimensions to the working data file. The names of these new variables will be *DIM00001* and *DIM00002*, respectively.

MATRIX Subcommand

The MATRIX subcommand is used to write category quantifications to a matrix data file or a previously declared dataset name (DATASET DECLARE command).

- The specification on MATRIX is keyword OUT and a quoted file specification of dataset name, enclosed in parentheses.

- You can specify an asterisk (*) replace the active dataset.
- The matrix data file has one case for each value of each original variable.

The variables of the matrix data file and their values are:

ROWTYPE_. *String variable containing value QUANT for all cases*.

LEVEL. *String variable LEVEL containing the values (or value labels if present) of each original variable*.

VARNAME_. *String variable containing the original variable names*.

DIM1...DIMn. *Numeric variable containing the category quantifications for each dimension*. Each variable is labeled *DIMn*, where *n* represents the dimension number.

HOST

Note: Square brackets used in the HOST syntax chart are required parts of the syntax and are not used to indicate optional elements. Equals signs (=) used in the syntax chart are required elements.

```
HOST COMMAND=['command' 'command'...'command']
  TIMELIMIT=n.
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic "Command Order" on page 36 for more information.

Release History

Release 13.0

- Command introduced.

Example

```
HOST COMMAND=['dir c:\myfiles\*.sav'].
```

Overview

The HOST command executes external commands at the operating system level. For a Windows operating system, for example, this is equivalent to running commands from a command prompt in a command window.

- No output is displayed in a command window. Output is either displayed in the Viewer or redirected as specified in the operating system command.
- Standard output is either displayed in a text object in the Viewer window or redirected as specified in the operating system command.
- Standard errors are displayed as text objects in the Viewer.
- Commands that return a prompt for user input result in an EOF condition without waiting for any user input (unless input has been redirected to read from a file).
- A command that generates an error condition terminates the HOST command, and no subsequent commands specified on the HOST command are executed.
- The HOST command runs synchronously. Commands that launch applications result in the suspension of further IBM SPSS Statistics processing until the application finishes execution, unless you also specify a time limit (see keyword TIMELIMIT) . For example, in Windows operating systems, if a file extension is associated with an application, simply specifying a file a name an extension on the command line will launch the associated application, and no further commands will be executed until the application is closed.
- The HOST command starts in the current working directory. By default, the initial working directory is the installation directory.
- In distributed analysis mode (available with IBM SPSS Statistics Server), file paths in command specifications are relative to the remote server.

Syntax

The minimum specification is the command name HOST, followed by the keyword COMMAND, an equals sign (=), and one or more operating system level commands, each enclosed in quotes, with the entire set of commands enclosed in square brackets.

Example

```
HOST COMMAND=['dir c:\myfiles\*.sav'
              'dir c:\myfiles\*.sps > c:\myfiles\command_files.txt'
              'copy c:\myfiles\file1.txt > c:\myfiles\file2.txt'
              'dur c:\myfiles\*.xml > c:\myfiles\xmlfiles.txt'
              'c:\myfiles\myjobs\report.bat'].
```

- The directory listing for all *.sav* files is displayed in a text output object in the Viewer window.
- The directory listing for *.sps* files is redirected to a text file; so no output is displayed in the Viewer window.
- If *file2.txt* does not already exist, the `copy` command will copy the contents of *file1.txt* to a new file called *file2.txt*. If *file2.txt* exists, the `copy` command will not be executed since this would result in a user prompt asking for the user to confirm overwriting the file.
- The invalid `dur` command generates an error, which is displayed in the Viewer, and no output for that command is redirected to specified text file.
- The error condition caused by the invalid `dur` command terminates the HOST command, and *report.bat* is not run.

Quoted Strings

If the command at the operating system level uses quoted strings, the standard rules for quoted strings within quoted strings apply. In general, use double-quotes to enclose a string that includes a string enclosed in single quotes, and vice-versa. See the topic "String Values in Command Specifications" on page 35 for more information.

TIMELIMIT Keyword

The optional `TIMELIMIT` keyword sets a time limit in seconds for execution of the bracketed list of commands. Fractional time values are rounded to the nearest integer.

Example

```
HOST COMMAND=['c:\myfiles\report.bat'] TIMELIMIT=10.
```

Using TIMELIMIT to Return Control

Since the HOST command runs synchronously, commands that launch applications result in the suspension of further IBM SPSS Statistics processing until the application finishes execution. That means that any commands that follow the HOST command will not be executed until any applications launched by the command are closed.

Example

```
OMS /DESTINATION FORMAT=HTML
  OUTFILE='c:\temp\temp.htm'.
FREQUENCIES VARIABLES=ALL.
OMSEND.
HOST COMMAND=['c:\temp\temp.htm'].
DESCRIPTIVES VARIABLES=ALL.
```

- On Windows operating systems, if the *.htm* extension is associated with an application (typically Internet Explorer), the HOST command in this example will launch the associated application.
- In the absence of a `TIMELIMIT` specification, the subsequent `DESCRIPTIVES` command will not be executed until the application launched by the HOST command is closed.

To make sure control is automatically returned to IBM SPSS Statistics and subsequent commands are executed, include a `TIMELIMIT` value, as in:

```
OMS /DESTINATION FORMAT=HTML
  OUTFILE='c:\temp\temp.htm'.
FREQUENCIES VARIABLES=ALL.
OMSEND.
```

```
HOST COMMAND=['c:\temp\temp.htm'] TIMELIMIT=5.
DESCRIPTIVES VARIABLES=ALL.
```

Working Directory

The HOST command starts in the current working directory. By default, the initial working directory is the installation directory. So, for example, HOST COMMAND=['dir'] executed at the start of a session would typically return a directory listing of the installation directory. The working directory can be changed, however, by the CD command and the CD keyword of the INSERT command.

Example

```
*start of session.
HOST COMMAND=['dir']. /*lists contents of install directory.
CD 'c:\temp'.
HOST COMMAND=['dir']. /*lists contents of c:\temp directory.
```

UNC Paths on Windows Operating Systems

To start in the IBM SPSS Statistics working directory, the HOST command actually issues an OS-level CD command that specifies the IBM SPSS Statistics working directory. On Windows operating systems, if you use UNC path specifications of the general form:

```
\\servername\sharename\path
```

on IBM SPSS Statistics commands such as CD or INSERT to set the working directory location, the HOST command will fail because UNC paths are not valid on the Windows CD command.

Example

```
INSERT FILE='\\hqserver\public\report.sps' CD=YES.
HOST ['dir'].
```

- The INSERT command uses a UNC path specification, and CD=YES makes that directory the working directory.
- The subsequent HOST command will generate an OS-level error message that says the current directory path is invalid because UNC paths are not supported.

IF

```
IF [(]logical expression[)] target variable=expression
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

The following relational operators can be used in logical expressions:

Table 63. Relational operators

Symbol	Definition
EQ or =	Equal to
NE or ~= or ¬ = or <>	Not equal to
LT or <	Less than
LE or <=	Less than or equal to
GT or >	Greater than
GE or >=	Greater than or equal to

The following logical operators can be used in logical expressions:

Table 64. Logical operators

Symbol	Definition
AND or &	Both relations must be true
OR or |	Either relation can be true
NOT	Reverses the outcome of an expression

Example

```
IF (AGE > 20 AND SEX = 1) GROUP=2.
```

Overview

IF conditionally executes a single transformation command based upon logical conditions found in the data. The transformation can create a new variable or modify the values of an existing variable for each case in the active dataset. You can create or modify the values of both numeric and string variables. If you create a new string variable, you must first declare it on the STRING command.

IF has three components: a *logical expression* that sets up the logical criteria, a *target variable* (the one to be modified or created), and an *assignment expression.* The target variable’s values are modified according to the assignment expression.

IF is most efficient when used to execute a single, conditional, COMPUTE-like transformation. If you need multiple IF statements to define the condition, it is usually more efficient to use the RECODE command or a DO IF–END IF structure.

Basic Specification

The basic specification is a logical expression followed by a target variable, a required equals sign, and the assignment expression. The assignment is executed only if the logical expression is true.

Syntax Rules

- Logical expressions can be simple logical variables or relations, or complex logical tests involving variables, constants, functions, relational operators, and logical operators. Both the logical expression and the assignment expression can use any of the numeric or string functions allowed in COMPUTE transformations.
- Parentheses can be used to enclose the logical expression. Parentheses can also be used within the logical expression to specify the order of operations. Extra blanks or parentheses can be used to make the expression easier to read.
- A relation can compare variables, constants, or more complicated arithmetic expressions. Relations cannot be abbreviated. For example, `(A EQ 2 OR A EQ 5)` is valid, while `(A EQ 2 OR 5)` is not. Blanks (not commas) must be used to separate relational operators from the expressions being compared.
- A relation cannot compare a string variable to a numeric value or variable, or vice versa. A relation cannot compare the result of the logical functions SYSMIS, MISSING, ANY, or RANGE to a number.
- String values used in expressions must be specified in quotes and must include any leading or trailing blanks. Lowercase letters are considered distinct from uppercase letters.
- String variables that are used as target variables must already exist. To declare a new string variable, first create the variable with the STRING command and then specify the new variable as the target variable on IF.

Examples

IF with Numeric Values

```
IF (AGE > 20 AND SEX = 1) GROUP=2.
```

- The numeric variable *GROUP* is set to 2 for cases where *AGE* is greater than 20 *and SEX* is equal to 1.
- When the expression is false or missing, the value of *GROUP* remains unchanged. If *GROUP* has not been previously defined, it contains the system-missing value.

IF with String Values

```
IF (SEX EQ 'F') EEO=QUOTA+GAIN.
```

- The logical expression tests the string variable *SEX* for the value *F*.
- When the expression is true (when *SEX* equals *F*), the value of the numeric variable *EEO* is assigned the value of *QUOTA* plus *GAIN*. Both *QUOTA* and *GAIN* must be previously defined numeric variables.
- When the expression is false or missing (for example, if *SEX* equals *F*), the value of *EEO* remains unchanged. If *EEO* has not been previously defined, it contains the system-missing value.

Conditional Expressions with Arithmetic Operations

```
COMPUTE V3=0.
IF ((V1-V2) LE 7) V3=V1**2.
```

- COMPUTE assigns *V3* the value 0.
- The logical expression tests whether *V1* minus *V2* is less than or equal to 7. If it is, the value of *V3* is assigned the value of *V1* squared. Otherwise, the value of *V3* remains at 0.

Conditional Expressions with Arithmetic Operations and Functions

```
IF (ABS(A-C) LT 100) INT=100.
```

- IF tests whether the absolute value of the variable *A* minus the variable *C* is less than 100. If it is, *INT* is assigned the value 100. Otherwise, the value is unchanged. If *INT* has not been previously defined, it is system-missing.

Testing for Missing Values

```
* Test for listwise deletion of missing values.

DATA LIST  /V1 TO V6 1-6.
STRING SELECT(A1).
COMPUTE SELECT='V'.
VECTOR V=V1 TO V6.

LOOP #I=1 TO 6.
IF MISSING(V(#I)) SELECT='M'.
END LOOP.

BEGIN DATA
123456
     56
1 3456
123456
123456
END DATA.

FREQUENCIES VAR=SELECT.
```

- STRING creates the string variable *SELECT* with an A1 format and COMPUTE sets the value of *SELECT* to *V*.
- VECTOR defines the vector *V* as the original variables *V1* to *V6*. Variables on a single vector must be all numeric or all string variables. In this example, because the vector *V* is used as an argument on the MISSING function of IF, the variables must be numeric (MISSING is not available for string variables).
- The loop structure executes six times: once for each VECTOR element. If a value is missing for any element, SELECT is set equal to *M*. In effect, if any case has a missing value for any of the variables *V1* to *V6*, SELECT is set to *M*.
- FREQUENCIES generates a frequency table for SELECT. The table gives a count of how many cases have missing values for at least one variable and how many cases have valid values for all variables. This table can be used to determine how many cases would be dropped from an analysis that uses listwise deletion of missing values.

Example

```
IF YRHIRED LT 1980 RATE=0.02.
IF DEPT='SALES' DIVISION='TRANSFERRED'.
```

- The logical expression on the first IF command tests whether *YRHIRED* is less than 1980 (hired before 1980). If so, the variable *RATE* is set to 0.02.
- The logical expression on the second IF command tests whether *DEPT* equals *SALES*. When the condition is true, the value for the string variable *DIVISION* is changed to *TRANSFERRED* but is truncated if the format for *DIVISION* is not at least 11 characters wide. For any other value of *DEPT*, the value of *DIVISION* remains unchanged.
- Although there are two IF statements, each defines a separate and independent condition. The IF command is used rather than the DO IF—END IF structure in order to test both conditions on every case. If DO IF—END IF is used, control passes out of the structure as soon as the first logical condition is met.

Example

```
IF (STATE EQ 'IL' AND CITY EQ 13) COST=1.07 * COST.
```

- The logical expression tests whether *STATE* equals *IL* and *CITY* equals 13.
- If the logical expression is true, the numeric variable *COST* is increased by 7%.
- For any other value of *STATE* or *CITY*, the value of *COST* remains unchanged.

Example

```
STRING GROUP (A18).
IF (HIRED GE 1988) GROUP='Hired after merger'.
```

- STRING declares the string variable *GROUP* and assigns it a width of 18 characters.
- When *HIRED* is greater than or equal to 1988, *GROUP* is assigned the value *Hired after merger*. When *HIRED* is less than 1988, *GROUP* remains blank.

Example

```
IF (RECV GT DUE OR (REVNUES GE EXPNS AND BALNCE GT 0))STATUS='SOLVENT'.
```

- First, the program tests whether *REVNUES* is greater than or equal to *EXPNS* and whether *BALNCE* is greater than 0.
- Second, the program evaluates if *RECV* is greater than *DUE*.
- If either of these expressions is true, *STATUS* is assigned the value *SOLVENT*.
- If both expressions are false, *STATUS* remains unchanged.
- *STATUS* is an existing string variable in the active dataset. Otherwise, it would have to be declared on a preceding STRING command.

Operations

- Each IF command evaluates every case in the data. Compare IF with DO IF, which passes control for a case out of the DO IF–END IF structure as soon as a logical condition is met.
- The logical expression is evaluated as true, false, or missing. The assignment is executed only if the logical expression is true. If the logical expression is false or missing, the assignment is not made. Existing target variables remain unchanged; new numeric variables retain their initial (system-missing) values.
- In general, a logical expression is evaluated as missing if any one of the variables used in the logical expression is system- or user-missing. However, when relations are joined by the logical operators AND or OR, the expression can sometimes be evaluated as true or false even when variables have missing values. See the topic “Missing Values and Logical Operators ” on page 912 for more information.

Numeric Variables

- Numeric variables created with IF are initially set to the system-missing value. By default, they are assigned an F8.2 format.
- Logical expressions are evaluated in the following order: functions, followed by exponentiation, arithmetic operations, relations, and logical operators. When more than one logical operator is used, NOT is evaluated first, followed by AND, and then OR. You can change the order of operations using parentheses.
- Assignment expressions are evaluated in the following order: functions, then exponentiation, and then arithmetic operators.

String Variables

- New string variables declared on IF are initially set to a blank value and are assigned the format specified on the STRING command that creates them.
- Logical expressions are evaluated in the following order: string functions, then relations, and then logical operators. When more than one logical operator is used, NOT is evaluated first, followed by AND, and then OR. You can change the order of operations using parentheses.
- If the transformed value of a string variable exceeds the variable’s defined width, the transformed value is truncated. If the transformed value is shorter than the defined width, the string is right-padded with blanks.

Missing Values and Logical Operators

When two or more relations are joined by logical operators AND or OR, the program always returns a missing value if all of the relations in the expression are missing. However, if any one of the relations can be determined, the program interprets the expression as true or false according to the logical outcomes below. The asterisk flags expressions where the program can evaluate the outcome with incomplete information.

Table 65. Logical outcomes

Expression	Outcome	Expression	Outcome
true AND true	= true	true OR true	= true
true AND false	= false	true OR false	= true
false AND false	= false	false OR false	= false
true AND missing	= missing	true OR missing	= true*
missing AND missing	= missing	missing OR missing	= missing
false AND missing	= false*	false OR missing	= missing

IGRAPH

IGRAPH is available in the Statistics Base Edition.

Note: IGRAPH syntax is deprecated. It is highly recommended that you convert existing IGRAPH syntax with the syntax converter. Search for "syntax converter" in the help.

```
IGRAPH

  [/Y=[VAR(varname1)]
      [TYPE={SCALE ([MIN=value] [MAX=value])}]
            {CATEGORICAL                    }
      [TITLE='string']]

  [/X1=[VAR(varname2)]]
       [TYPE={SCALE([MIN=value] [MAX=value])}]
             {CATEGORICAL                   }
       [TITLE='string']]

  [/X2=[VAR(varname3)]]
       [TYPE={SCALE([MIN=value] [MAX=value])}]
             {CATEGORICAL                   }
       [TITLE='string']]

  [/YLENGTH=value]

  [/X1LENGTH=value]

  [/CATORDER VAR(varname)
    ({COUNT     }   [{ASCENDING }] [{SHOWEMPTY])]
     {OCCURRENCE}    {DESCENDING}   {OMITEMPTY}
     {LABEL}
     {VALUE}

  [/COLOR=varname
      [TYPE={SCALE([MIN=value] [MAX=value])}]
            {CATEGORICAL                   }
      [LEGEND={ON|OFF}]
      [TITLE='string']]
      [{CLUSTER}]]
       {STACK  }

  [/REFLINE varname value [LABEL={ON|OFF}]
                          [SPIKE = {ON|OFF}]]
                          [COLOR={ON|OFF}]
                          [STYLE={ON|OFF}]

  [/STYLE=varname
      [LEGEND={ON|OFF}]
      [TITLE='string']]
      [{CLUSTER}]
       {STACK  }

  [/NORMALIZE]

  [/SIZE=varname
      [TYPE={SCALE([MIN=value] [MAX=value])}]
            {CATEGORICAL                   }
      [LEGEND={ON|OFF}]
      [TITLE='string']]

  [/STACK=varname]

  [/SUMMARYVAR=varname]

  [/PANEL varlist]

  [/POINTLABEL=varname]

  [/CASELABEL=varname]

  [/COORDINATE={HORIZONTAL}]
               {VERTICAL  }
               {THREE     }

  [/EFFECT={NONE }]
           {THREE}

  [/TITLE='string']

  [/SUBTITLE='string']


  [/CAPTION='string']


  [/VIEWNAME='line 1']


  [/CHARTLOOK='filename']


  [/SCATTER
      [COINCIDENT={NONE  }]
                  {JITTER}]


  [/BAR [(summary function)]
```

```
    [LABEL {INSIDE }[VAL][N]]
           {OUTSIDE}
    [BASELINE=(value)]]


[/PIE [(summary function)]
     [START value]
     [{CW|CCW}]
     [SLICE={INSIDE } [LABEL] [PCT] [VAL] [N]]
            {OUTSIDE}
     [STACK=LABEL]


[/BOX [OUTLIERS={ON|OFF}] [EXTREME={ON|OFF}]
      [MEDIAN={ON|OFF}]
      [WHISKER={T   }]
               {LINE}
      [CAPWIDTH (pct)]]


[/LINE [(summary function)]
    STYLE={DOTLINE}
          {LINE   }
          {DOT    }
          {NONE   }
    [DROPLINE={ON|OFF}]
    [LABEL=[VAL] [N] [PCT]]
    [LINELABEL=[CAT] [N] [PCT]]
    [INTERPOLATE={STRAIGHT}]
                 {LSTEP    }
                 {CSTEP    }
                 {RSTEP    }
                 {LJUMP    }
                 {RJUMP    }
                 {CJUMP    }
                 {SPLINE   }
    [BREAK={MISSING}]]
           {NONE   }

[/AREA [(summary function)]]
   [POINTLABEL = [VAL] [N] [PCT]]
   [AREALABEL = [CAT] [N] [PCT]]
   [BASELINE=(value)]
   [INTERPOLATE={STRAIGHT}]
                {LSTEP   }
                {CSTEP   }
                {RSTEP   }
   [BREAK={MISSING}]
          {NONE   }

[/ERRORBAR [{CI(pctvalue)}]
           {SD(sdval)   }
           {SE(seval)   }
    [LABEL [VAL][N]]
    [CAPWIDTH (pct)]
    [CAPSTYLE={NONE}]
              {T   }
    [SYMBOL={ON|OFF}]
    [BASELINE=value]]


[/HISTOGRAM [CUM]
    [SHAPE={HISTOGRAM}]
    [X1INTERVAL={AUTO   }]
                {NUM=n  }
                {WIDTH=n}
    [X2INTERVAL={AUTO   }]
                {NUM=n  }
                {WIDTH=n}
    [X1START=n]
    [X2START=n]
    [CURVE={OFF|ON}]


[/FITLINE [METHOD={NONE             }]
                  {REGRESSION LINEAR}
                  {ORIGIN LINEAR    }
                  {MEAN             }
                  {LLR [(NORMAL|EPANECHNIKOV|UNIFORM)]
                       [BANDWIDTH={FAST|CONSTRAINED}]
                       [X1MULTIPLIER=multiplier]
                       [X2MULTIPLIER=multiplier]}
    [INTERVAL[(cval)]=[MEAN] [INDIVIDUAL]]
    [LINE=[TOTAL] [MEFFECT]]]

[/SPIKE  {X1      }]
         {X2      }
         {Y       }
         {CORNER  }
         {ORIGIN  }
         {FLOOR   }
         {CENTROID}

 [/FORMAT [ SPIKE [COLOR={ON|OFF}] [STYLE={ON|OFF}]]
```

Release History

Release 18.0

- IGRAPH syntax is deprecated. It will be removed in an upcoming release. It is highly recommended that you convert existing IGRAPH syntax with the syntax converter. See the topic for more information.
- LABEL=N keyword for the BOX subcommand is ignored.

Release 16.0

- X2LENGTH subcommand is ignored.
- ChartLook *.clo* files are no longer supported by the CHARTLOOK subcommand. Use chart templates (*.sgt* files) instead. See the topic for more information.
- COINCIDENT keyword for the SCATTER subcommand can no longer specify a jittering amount.
- SHAPE keyword for the BAR subcommand is ignored. The shape of the bars is always a rectangle.
- BARBASE keyword for the BAR subcommand is ignored.
- CLUSTER keyword for the PIE subcommand is now an alias for STACK.
- TEXTIN and NUMIN are ignored by the SLICE keyword for the PIE subcommand.
- Label position values (URIGHT, LRIGHT, ULEFT, and LLEFT) are ignored by STACK keyword for the PIE subcommand. The position is always an optimal one.
- BOXBASE keyword for the BOX subcommand is ignored.
- FANCY value is ignored by the WHISKER keyword for the BOX subcommand.
- LAGRANGE3 and LAGRANGE5 values are now aliases for SPLINE for the INTERPOLATE keyword for the LINE subcommand.
- DIRECTION keyword is ignored by the ERRORBAR subcommand. Error bars always extend both above and below the mean values.
- FANCY value is ignored by the CAPSTYLE keyword for the ERRORBAR subcommand.
- TOTAL and MEFFECT values are ignored by the CENTROID keyword for the SPIKE subcommand. Spikes are always drawn to subgroup means.
- When using the FITLINE subcommand, the formula of the fit line is no longer displayed in the graph.
- The CLUSTER subcommand is now an alias for STACK.

Example

```
IGRAPH
 /VIEWNAME='Scatterplot'
 /X1=VAR(trial1) TYPE=SCALE
 /Y=VAR(trial3) TYPE=SCALE
 /X2=VAR(trial2) TYPE=SCALE
 /COORDINATE=THREE
 /X1LENGTH=3.0
 /YLENGTH=3.0
 /SCATTER COINCIDENT=NONE
 /FITLINE METHOD=REGRESSION LINEAR INTERVAL(90.0)=MEAN  LINE=TOTAL.
```

Overview

The interactive Chart Editor is designed to emulate the experience of drawing a statistical chart with a pencil and paper. The Chart Editor is a highly interactive, direct manipulation environment that automates the data manipulation and drawing tasks required to draw a chart by hand, such as determining data ranges for axes; drawing ticks and labels; aggregating and summarizing data; drawing data representations such as bars, boxes, or clouds; and incorporating data dimensions as legends when the supply of dependent axes is exhausted.

The IGRAPH command creates a chart in an interactive environment. The interactive Chart Editor allows you to make extensive and fundamental changes to this chart instead of creating a new chart. The Chart Editor allows you to replace data, add new data, change dimensionality, create separate chart panels for different groups, or change the way data are represented in a chart (that is, change a bar chart into a boxplot). The Chart Editor is not a “typed” chart system. You can use chart elements in any combination, and you are not limited by “types” that the application recognizes.

To create a chart, you assign data dimensions to the domain (independent) and range (dependent) axes to create a “data region.” You also add data representations such as bars or clouds to the data region. Data representations automatically position themselves according to the data dimensions assigned to the data region.

There is no required order for assigning data dimensions or adding data representations; you can add the data dimensions first or add the data representations first. When defining the data region, you can define the range axis first or the domain axis first.

Options

Titles and Captions. You can specify a title, subtitle, and caption for the chart.

Chart Type. You can request a specific type of chart using the BAR, PIE, BOX, LINE, ERRORBAR, HISTOGRAM, and SCATTERPLOT subcommands.

Chart Content. You can combine elements in a single chart. For example, you can add error bars to a bar chart.

Chart Legends. You can specify either scale legends or categorical legends. Moreover, you can specify whether a color or style is used to distinguish the legend variables.

Chart Appearance. You can specify a template, using the CHARTLOOK subcommand, to override the default chart attribute settings.

Basic Specification

The minimum syntax to create a graph is simply the IGRAPH command, without any variable assignment. This will create an empty graph. To create an element in a chart, a dependent variable must be assigned and a chart element specified.

Subcommand Order

- Subcommands can be used in any order.

Syntax Rules

- EFFECT=THREE and COORDINATE=THREE cannot be specified together. If they are, the EFFECT keyword will be ignored.

Operations

- The chart title, subtitle, and caption are assigned as they are specified on the TITLE, SUBTITLE, and CAPTION subcommands. In the absence of any of these subcommands, the missing title, subtitle, or caption are null.

General Syntax

Following are the most general-purpose subcommands. Even so, not all plots will use all subcommands. For example, if the only element in a chart is a bar, the SIZE subcommand will not be shown in the graph.

Each general subcommand may be specified only once. If one of these subcommands appears more than once, the last one is used.

X1, Y, and X2 Subcommands

X1 and Y, and X2 assign variables to the *X1*, *Y*, and *X2* dimensions of the chart.

- The variable must be enclosed in parentheses after the VAR keyword.
- Each of these subcommands can include the TITLE keyword, specifying a string with which to title the corresponding axis.
- Each variable must be either a scale variable, a categorical variable, or a built-in data dimension. If a type is not specified, a default type is used from the variable's definition.

SCALE. A scale dimension is interpreted as a measurement on some continuous scale for each case. Optionally, the minimum (MIN) and maximum (MAX) scale values can be specified. In the absence of MIN and MAX, the entire data range is used.

CATEGORICAL. A categorical dimension partitions cases into exclusive groups (each case is a member of exactly one group). The categories are represented by evenly spaced ticks.

A built-in dimension is a user interface object used to create a chart of counts or percentages and to make a casewise chart of elements that usually aggregate data like bars or lines. The built-in dimensions are count ($COUNT), percentage ($PCT), and case ($CASE).

- To create a chart that displays counts or percentages, one of the built-in data dimensions is assigned to the range (*Y*) axis. The VAR keyword is not used for built-in dimensions.
- Built-in count and percentage data dimensions cannot be assigned to a domain axis (*X1* or *X2*) or to a legend subcommand.
- The count and percentage data dimensions are all scales and cannot be changed into categorizations.

CATORDER Subcommand

The CATORDER subcommand defines the order in which categories are displayed in a chart and controls the display of empty categories, based on the characteristics of a variable specified in parenthesis after the subcommand name.

- You can display categories in ascending or descending order based on category values, category value labels, counts, or values of a summary variable.
- You can either show or hide empty categories (categories with no cases).

Keywords for the CATORDER subcommand include:

ASCENDING. *Display categories in ascending order of the specified order keyword.*

DESCENDING. *Display categories in descending order of the specified order keyword.*

SHOWEMPTY. *Include empty categories in the chart.*

OMITEMPTY. *Do not include empty categories in the chart.*

ASCENDING and DESCENDING are mutually exclusive. SHOWEMPTY and OMITEMPTY are mutually exclusive.

Order keywords include:

COUNT. *Sort categories based on the number of observations in each category.*

OCCURRENCE. *Sort categories based on the first occurrence of each unique value in the data file.*

LABEL. *Sort categories based on defined value labels for each category.* For categories without defined value labels, the category value is used.

VALUE. *Sort categories based on the values of the categories or the values of a specified summary function for the specified variable.* See the topic “Summary Functions ” on page 931 for more information.

Order keywords are mutually exclusive. You can specify only one order keyword on each CATORDER subcommand.

X1LENGTH, YLENGTH, and X2LENGTH Subcommands

X1LENGTH and YLENGTH define the length in inches of the chart size in the direction of the corresponding axis. X2LENGTH is no longer supported and is ignored.

Example

```
IGRAPH
 /VIEWNAME='Scatterplot'
 /Y=VAR(sales96) TYPE=SCALE
 /X1=VAR(sales95) TYPE=SCALE
 /X2=VAR(region) TYPE=CATEGORICAL
 /X1LENGTH=2.39
 /YLENGTH=2.42
 /SCATTER.
```

- Y assigns *sales96* to the dependent axis, defining it to be continuous.
- X1 assigns *sales95* to the *X1* axis, defining it to be a scale variable (continuous).
- X1LENGTH and YLENGTH define the width and height of the chart in inches.

NORMALIZE Subcommand

The NORMALIZE subcommand creates 100% stacking for counts and converts statistics to percents. It has no additional specifications. This subcommand is valid only with the SUM, SUMAV, and SUMSQ summary functions or the $count and $pct built-in dimensions.

COLOR, STYLE, and SIZE Subcommands

COLOR, STYLE, and SIZE specify variables used to create a legend. Each value of these variables corresponds to a unique property of the chart. The effect of these variables depends on the type of chart.

- Most charts use color in a similar fashion; casewise elements draw each case representation using the color value for the case, and summary elements draw each group representation in the color that represents a summarized value in the color data dimension.
- For dot-line charts, dot charts, and scatterplots, symbol shape is used for style variables and symbol size is used for size variables.
- For line charts and lines in a scatterplot, dash patterns encode style variables and line thickness encodes size variables.
- For bar charts, pie charts, boxplots, histograms, and error bars, fill pattern encodes style variables. Typically, these charts are not sensitive to size variables.

CATEGORICAL legend variables split the elements in the chart into categories. A categorical legend shows the reader which color, style, or size is associated with which category of the variable. The colors, styles, or sizes are assigned according to the discrete categories of the variable.

SCALE legend variables apply color or size to the elements by the value or a summary value of the legend variable, creating a continuum across the values. COLOR and SIZE can create either scale legends or categorical legends. STYLE can create categorical legends only.

Scale variables have the following keywords:

MIN. *Defines the minimum value of the scale.*

MAX. *Defines the maximum value of the scale.*

- The keywords MIN and MAX and their assigned values must be enclosed in parentheses.

In addition, the following keywords are available for COLOR, STYLE, and SIZE:

LEGEND. *Determines if the legend is displayed or not.* The legend explains how to decode color, size, or style in a chart.

TITLE. *Specifies a string used to title the legend.*

The following keywords are available for COLOR and STYLE:

CLUSTER. *Creates clustered charts based on color or size variables.*

STACK. *Creates stacked charts based on color or size variables.*

CLUSTER and STACK are mutually exclusive. Only one can be specified. Also, CLUSTER should not be used for *both* COLOR and STYLE.

Example

```
IGRAPH
 /VIEWNAME='Scatterplot'
 /Y=VAR(sales96) TYPE=SCALE
 /X1=VAR(sales95) TYPE=SCALE
 /X2=VAR(region) TYPE=CATEGORICAL
 /COLOR=VAR(tenure) TYPE=SCALE
 /STYLE=VAR(vol94)
 /SCATTER.
```

- The chart contains a three-dimensional scatterplot.
- COLOR defines a scale legend corresponding to the variable *TENURE*. Points appear in a continuum of colors, with the point color reflecting the value of *TENURE*.

- STYLE defines a categorical legend. Points appear with different shapes, with the point shape reflecting the value of *VOL94*.

STACK Subcommand

STACK defines the variable used to create stacked pie charts. The variable specified must be categorical. The stack will contain as many pies as there are categories in the stack variable.

SUMMARYVAR Subcommand

SUMMARYVAR specifies the variable or function for summarizing a pie element. It can only have the built-in variables $COUNT or $PCT or a user-defined variable name.

Specifying a user-defined variable on SUMMARYVAR requires specifying a summary function on the PIE subcommand. Valid summary functions include SUM, SUMAV, SUMSQ, NLT(x), NLE(x), NEQ(x), NGT(x), and NGE(x). The slices of the pie represent categories defined by the values of the summary function applied to SUMMARYVAR.

PANEL Subcommand

PANEL specifies a categorical variable or variables for which separate charts will be created.

- Specifying a single panel variable results in a separate chart for each level of the panel variable.
- Specifying multiple panel variables results in a separate chart for each combination of levels of the panel variables.

POINTLABEL Subcommand

POINTLABEL specifies a variable used to label points in a boxplot or scatterplot.

- If a label variable is specified without ALL or NONE, no labels are turned on (NONE).
- The keyword NONE turns all labels off.

CASELABEL Subcommand

CASELABEL specifies a variable used to label cases in a chart of individual cases. For example, if you were creating a bar chart whose *x* axis specification was $case, CASELABEL would specify the content of the tick labels that appear on the *x* axis.

COORDINATE Subcommand

COORDINATE specifies the orientation of the chart.

HORIZONTAL. *The* Y *variable appears along the horizontal axis and the* X1 *variable appears along the vertical axis.*

VERTICAL. *The* Y *variable appears along the vertical axis and the* X1 *variable appears along the horizontal axis.*

THREE. *Create a three-dimensional chart.* Three-dimensional charts have a default orientation that cannot be altered.

Example

```
IGRAPH
 /VIEWNAME='Scatterplot'
 /Y=VAR(sales96) TYPE=SCALE
 /X1=VAR(region) TYPE=CATEGORICAL
 /COORDINATE=HORIZONTAL
 /BAR (mean).
```

- The COORDINATE subcommand defines the bar chart as horizontal with *region* on the vertical dimension and means of *sales96* on the horizontal dimension.

EFFECT Subcommand

EFFECT displays a two-dimensional chart with additional depth along a third dimension. Two-dimensional objects other than points are displayed as three-dimensional solids.

- EFFECT is unavailable for three-dimensional charts.

TITLE, SUBTITLE, and CAPTION Subcommands

TITLE, SUBTITLE, and CAPTION specify lines of text placed at the top or bottom of a chart.

- Multiple lines of text can be entered using the carriage control character (\n).
- Each title, subtitle, or caption must be enclosed in apostrophes or quotation marks.
- The maximum length of a title, subtitle, or caption is 255 characters.
- The font, point size, color, alignment, and orientation of the title, subtitle, and caption text is determined by the ChartLook.

VIEWNAME Subcommand

VIEWNAME assigns a name to the chart, which will appear in the outline pane of the Viewer. The name can have a maximum of 255 characters.

CHARTLOOK Subcommand

CHARTLOOK identifies a template file containing specifications concerning the initial visual properties of a chart, such as fill, color, font, style, and symbol. By specifying a template, you can control cosmetic properties that are not explicitly available as syntax keywords.

Valid template files have an *.sgt* extension (old ChartLook *.clo* files are no longer supported). Files designated on CHARTLOOK must either be included with the software or created in the Chart Editor by saving a chart as a template. For more information about templates, see .

You can specify multiple templates by listing them in square brackets and separating each file name with a space (for example, CHARTLOOK=['template1.sgt' 'template2.sgt']). Templates are applied in the order in which they appear. If any of the settings in multiple templates conflict, the settings in the last template override the conflicting settings in previous templates.

A template contains values for the following properties:

- Color sequence for categorical color legends
- Color range for scale color legends
- Line style sequence for categorical style legends
- Symbol style sequence for categorical style legends
- Categorical legend fill styles
- Categorical symbol size sequence for categorical size legends
- Symbol size sequence for scale size sequences
- Categorical line weight sequence for categorical size legends
- Font, size, alignment, bold, and italic properties for text objects
- Fill and border for filled objects
- Style, weight, and color for line objects
- Font, shape, size, and color for symbol objects
- Style, weight, and color for visual connectors
- Axis properties: axis line style, color, and weight; major tick shape, location, color, and size

Example

```
IGRAPH
 /VIEWNAME='Slide 1'
```

```
  /X1=VAR(sales95) TYPE=SCALE
  /Y=VAR(sales96) TYPE=SCALE
  /X2=VAR(region) TYPE=CATEGORICAL
  /COORDINATE=THREE
  /POINTLABEL=VAR(division) NONE
  /TITLE='Scatterplot Comparing Regions'
  /SUBTITLE='Predicting 1996 Sales\nfrom 1995 Sales'
  /CHARTLOOK='Classic.sgt'
  /SCATTER.
```

- VIEWNAME assigns the name *Slide 1* to the chart. The outline pane of the Viewer uses this name for the chart.
- Points in the chart are labeled with the values of *division*. Initially, all labels are off. Labels for individual points can be turned on interactively after creating the chart.
- TITLE and SUBTITLE define text to appear of the plot. The subtitle contains a carriage return between *Sales* and *from*.
- The appearance of the chart is defined in the Classic template.

REFLINE Subcommand

The REFLINE subcommand inserts a reference line for the specified variable at the specified value. Optional keywords are:

LABEL={ON|OFF}. *Display a label for the reference line.* For variables with defined value labels, the value label for the specified value is displayed. If there is no defined value label for the specified value, the specified value is displayed.

SPIKE={ON|OFF}. *Display spikes from the reference line to individual data points.*

Example

```
IGRAPH
  /X1 = VAR(gender) TYPE = CATEGORICAL
  /Y = VAR(salary) TYPE = SCALE
  /BAR(MEAN)
  /REFLINE salary 30000 LABEL=ON.
```

SPIKE Subcommand

The SPIKE subcommand inserts spikes from individual data points to the specified location. Keywords for location include:

X1. *Display spikes to the* X1 *axis.*

X2. *Display spikes to the* X2 *axis.*

Y. *Display spikes to the* Y *axis.*

CORNER. *Display spikes to the corner defined by the lowest displayed values of the* X1, X2, *and* Y *axes.*

ORIGIN. *Display spikes to the origin. The origin is the point defined by the 0 values for the* X1, X2, *and* Y *axes.*

FLOOR. *Display spikes to the "floor" defined by the* X1 *and* X2 *axes.*

CENTROID. *Display spikes to the point defined by the subgroup mean values of the* X1, X2, *and* Y *variables.* CENTROID=TOTAL is no longer supported. Spikes are always drawn to subgroup means defined by color and/or style variables.

Example:

```
IGRAPH
  /X1 = VAR(salbegin) TYPE = SCALE
  /Y = VAR(salary) TYPE = SCALE
  /COLOR = VAR(gender) TYPE = CATEGORICAL
  /SPIKE CENTROID.
```

FORMAT Subcommand

For charts with color or style variables, the FORMAT subcommand controls the color and style attributes of spikes. The keywords are:

SPIKE. *Applies color and style specifications to spikes.* This keyword is required.

COLOR{ON|OFF}. *Controls use of color in spikes as defined by color variable.* The default is ON.

STYLE {ON|OFF}. *Controls use of line style in spikes as defined by style variable.* The default is ON.

Example

```
IGRAPH
  /X1 = VAR(salbegin) TYPE = SCALE
  /Y = VAR(salary) TYPE = SCALE
  /COLOR = VAR(gender) TYPE = CATEGORICAL
  /SPIKE CENTROID
  /FORMAT SPIKE COLOR=OFF.
```

KEY Keyword

All interactive chart types except histograms include a key element that identifies the summary measures displayed in the chart (for example, counts, means, and medians). The KEY keyword controls the display of the key in the chart. The default is ON, which displays the key. The OFF specification hides the key. The KEY specification is part of the subcommand that defines the chart type.

Example

```
IGRAPH
  /X1 = VAR(jobcat) TYPE = CATEGORICAL
  /Y = $count
  /BAR KEY=OFF.
```

Element Syntax

The following subcommands add elements to a chart. The same subcommand can be specified more than once. Each subcommand adds another element to the chart.

SCATTER Subcommand

SCATTER produces two- or three-dimensional scatterplots. Scatterplots can use either categorical or scale dimensions to create color or size legends. Categorical dimensions are required to create style legends.

The keyword COINCIDENT controls the placement of markers that have identical values on all axes. COINCIDENT can have one of the following two values:

NONE. *Places coincident markers on top of one another.* This is the default value.

JITTER. *Adds a small amount of random noise to all scale axis dimensions.* Specifying an amount is no longer supported and is ignored.

Example

```
IGRAPH
 /Y=VAR(sales96) TYPE=SCALE
 /X1=VAR(sales95) TYPE=SCALE
 /COORDINATE=VERTICAL
 /SCATTER COINCIDENT=JITTER.
```

- COORDINATE defines the chart as two-dimensional with *sales96* on the vertical dimension.
- SCATTER creates a scatterplot of *sales96* and *sales95*.
- The scale axes have random noise added by the JITTER keyword allowing separation of coincident points.

AREA Subcommand

AREA creates area charts. These charts summarize categories of one or more variables. The following keywords are available:

summary function. *Defines a function used to summarize the variable defined on the Y subcommand.* If the *Y* axis assignment is $COUNT or $PCT, the AREA subcommand cannot have a summary function. If the Y subcommand specifies TYPE=CATEGORICAL, then AREA can only specify MODE as the summary function.

POINTLABEL. *Labels points with the actual values corresponding to the dependent axis (* VAL *), the percentage of cases (* PCT *), and the number of cases included in each data point (* N *).* The default is no labels.

AREALABEL. *Labels area with category labels (* CAT *), the percentage of cases (* PCT *), and the number of cases included in each line (* N *).* The default is no labels.

BREAK. *Indicates whether the lines break at missing values (* MISSING *) or not (* NONE *).*

BASELINE. *The baseline value determines the location from which the areas will hang (vertical) or extend (horizontal).* The default value is 0.

The INTERPOLATE keyword determines how the lines connecting the points are drawn. Options include:

STRAIGHT. *Straight lines.*

LSTEP. *A horizontal line extends from each data point.* A vertical riser connects the line to the next data point.

CSTEP. *Each data point is centered on a horizontal line that extends half of the distance between consecutive points.* Vertical risers connect the line to the next horizontal line.

RSTEP. *A horizontal line terminates at each data point.* A vertical riser extends from each data point, connecting to the next horizontal line.

BAR Subcommand

BAR creates a bar element in a chart, corresponding to the *X1*, *X2*, and *Y* axis assignments. Bars can be clustered by assigning variables to COLOR or STYLE. Horizontal or vertical orientation is specified by the COORDINATE subcommand.

summary function. *Defines a function used to summarize the variable defined on the* Y *subcommand.* If the *Y* axis assignment is $COUNT or $PCT, the BAR subcommand cannot have a summary function. If the Y subcommand specifies TYPE=CATEGORICAL, then BAR can specify only MODE as the summary function.

LABEL. *Bars can be labeled with the actual values corresponding to the dependent axis* (VAL) *or with the number of cases included in each bar* (N). The default is no labels. The placement of the labels is inside the bars (INSIDE) or outside the bars (OUTSIDE).

SHAPE. This keyword is no longer supported and is ignored. Bars are always drawn as rectangles.

BARBASE. This keyword is no longer supported and is ignored.

BASELINE. *The baseline value determines the location from which the bars will hang (vertical) or extend (horizontal).* The default value is 0.

Example

```
IGRAPH
 /X1=VAR(volume96) TYPE=CATEGORICAL
 /Y=$count
 /COORDINATE=VERTICAL
 /EFFECT=THREE
 /BAR LABEL INSIDE N.
```

- X1 assigns the categorical variable *volume96* to the *X1* axis.
- Y assigns the built-in dimension $count to the range axis.
- VERTICAL defines the counts to appear along the vertical dimension.
- BAR adds a bar element to the chart.
- LABEL labels the bars in the chart with the number of cases included in the bars. These labels appear inside the bars.

Example

```
IGRAPH
 /X1=VAR(volume94) TYPE=CATEGORICAL
 /Y=VAR(sales96) TYPE=SCALE
 /COORDINATE=HORIZONTAL
 /EFFECT=NONE
 /BAR (MEAN) LABEL OUTSIDE VAL BASELINE=370.00.
```

- X1 assigns the categorical variable *volume94* to the *X1* axis.
- Y assigns the scale variable *sales96* to the range axis.
- HORIZONTAL defines *sales96* to appear along the horizontal dimension.
- EFFECT defines the chart as two-dimensional.
- BAR adds a bar element to the chart.
- MEAN defines the summary function to apply to *sales96*. Each bar represents the mean *sales96* value for the corresponding category of *volume94*.
- LABEL labels the bars in the chart with the mean *sales96* value. These labels appear outside the bars.
- BASELINE indicates that bars should extend from 370. Any bar with a mean value above 370 extends to the right. Any bar with a mean value below 370 extends to the left.

PIE Subcommand

A simple pie chart summarizes categories defined by a single variable or by a group of related variables. A clustered pie chart contains a cluster of simple pies, all of which are stacked into categories by the same variable. The pies are of different sizes and appear to be stacked on top of one another. The cluster contains as many pies as there are categories in the cluster variable. For both simple and clustered pie charts, the size of each slice represents the count, the percentage, or a summary function of a variable.

The following keywords are available:

summary function. *Defines a function used to summarize the variable defined on the SUMMARYVAR subcommand*. If the SUMMARYVAR assignment is $COUNT or $PCT, the PIE subcommand cannot have a summary function. Otherwise, SUM, SUMAV, SUMSQ, NGT(x), NLE(x), NEQ(x), NGE(x), NGT(x), and NIN(x1,x2) are available. See the topic “Summary Functions ” on page 931 for more information.

START num. *Indicates the starting position of the smallest slice of the pie chart.* Any integer can be specified for num. The value is converted to a number between 0 and 360, which represents the degree of rotation of the smallest slice.

CW | CCW. *Sets the positive rotation of the pie to either clockwise (* CW *) or counterclockwise (* CCW *)*. The default rotation is clockwise.

SLICE. *Sets the labeling characteristics for the slices of the pie.* The pie slices can be labeled with the category labels (LABEL), the category percentages (PCT), the number of cases (N), and the category values (VAL). Label position is either all labels inside the pie (INSIDE) or all labels outside the pie (OUTSIDE). TEXTIN and NUMIN are no longer supported and are ignored.

STACK. *Sets the labeling characteristics for the pies from stacks.* This keyword is always set to LABEL so the pies are labeled with the category labels. (The PCT, N, and VAL values are no longer supported and are ignored.) Options for specifying the label position are no longer supported and are ignored. An optimal label position is always used.

Example

```
IGRAPH
 /SUMMARYVAR=$count
 /COLOR=VAR(volume96) TYPE=CATEGORICAL
 /EFFECT=THREE
 /PIE START 180 CW SLICE=INSIDE LABEL PCT N.
```

- The pie slices represent the number of cases (SUMMARYVAR=$count) in each category of *volume96* (specified on the COLOR subcommand).
- EFFECT yields a pie chart with an additional third dimension.

- PIE creates a pie chart.
- The first slice begins at 180 degrees and the rotation of the pie is clockwise.
- SLICE labels the slices with category labels, the percentage in each category, and the number of cases in each category. INSIDE places the category and numeric labels inside the pie slices .

Example

```
IGRAPH
 /SUMMARYVAR=VAR(sales96)
 /COLOR=VAR(volume95) TYPE=CATEGORICAL
 /X1=VAR(region) TYPE=CATEGORICAL
 /Y=VAR(division) TYPE=CATEGORICAL
 /COORDINATE=VERTICAL
 /PIE (SUM) START 0 CW SLICE=INSIDE VAL.
```

- The pie slices represent the sums of *sales96* values for each category of *volume95* (specified on the COLOR subcommand).
- X1 and Y define two axes representing *region* and *division*. A pie chart is created for each combination of these variables.
- The first slice in each pie begins at 0 degrees and the rotation of the pie is clockwise.
- SUM indicates the summary function applied to the summary variable, *sales96*. The pie slices represent the sum of the *sales96* values.
- SLICE labels the slices with the value of the summary function. INSIDE places the labels inside the pie slices.

BOX Subcommand

BOX creates a boxplot, sometimes called a box-and-whiskers plot, showing the median, quartiles, and outlier and extreme values for a scale variable. The interquartile range (IQR) is the difference between the 75th and 25th percentiles and corresponds to the length of the box.

The following keywords are available:

OUTLIERS. *Indicates whether outliers should be displayed.* Outliers are values between 1.5 IQR's and 3 IQR's from the end of a box. By default, the boxplot displays outliers (ON).

EXTREME. *Indicates whether extreme values should be displayed.* Values more than 3 IQR's from the end of a box are defined as extreme. By default, the boxplot displays extreme values (ON).

MEDIAN. *Indicates whether a line representing the median should be included in the box.* By default, the boxplot displays the median line (ON).

BOXBASE. This keyword is no longer supported and is ignored.

WHISKER. *Controls the appearance of the whiskers.* Whiskers can be straight lines (LINE) or end in a *T*-shape (T). FANCY is no longer supported and is ignored.

CAPWIDTH(pct). *Controls the width of the whisker cap relative to the corresponding box.* Pct equals the percentage of the box width. The default value for pct is 45.

Example

```
IGRAPH
 /X1=VAR(region) TYPE=CATEGORICAL
 /Y=VAR(sales96) TYPE=SCALE
 /COORDINATE=HORIZONTAL
 /BOX OUTLIERS=ON EXTREME=ON MEDIAN=ON WHISKER=LINE.
```

- X1 assigns the variable *region* to the *X1* axis.
- Y assigns the variable *sales96* to the range axis.
- COORDINATE positions the range axis along the horizontal dimension.
- BOX creates a boxplot. The outliers and extreme vales are shown. In addition, a line representing the median is added to the box.
- WHISKER yields whiskers ending in a straight lines.

Example

```
IGRAPH
 /X1=VAR(region) TYPE=CATEGORICAL
 /Y=VAR(sales96) TYPE=SCALE
 /X2=VAR(division) TYPE=CATEGORICAL
 /COORDINATE=THREE
 /BOX OUTLIERS=OFF EXTREME=ON MEDIAN=OFF WHISKER=T.
```

- X2 adds a third dimension, corresponding to *division*, to the boxplot in the previous example.
- COORDINATE indicates that the chart displays the third dimension.
- BOX creates a boxplot without outliers or a median line. Extreme values are shown.

LINE Subcommand

LINE creates line charts, dot charts, and ribbon charts. These charts summarize categories of one or more variables. Line charts tend to emphasize flow or movement instead of individual values. They are commonly used to display data over time and therefore can be used to give a good sense of trends. A ribbon chart is similar to a line chart, with the lines displayed as ribbons in a third dimension. Ribbon charts can either have two dimensions displayed with a 3-D effect, or they can have three dimensions.

The following keywords are available:

summary function. *Defines a function used to summarize the variable defined on the Y subcommand.* If the *Y* axis assignment is $COUNT or $PCT, the LINE subcommand cannot have a summary function. If the Y subcommand specifies TYPE=CATEGORICAL, then LINE can specify only MODE as the summary function.

STYLE. *Chart can include dots and lines (* DOTLINE *), lines only (* LINE *), or dots only (* DOT *).* The keyword NONE creates an empty chart.

DROPLINE. *Indicates whether drop lines through points having the same value of a variable are included in the chart (* ON *) or not (* OFF *).* To include drop lines, specify a categorical variable on the STYLE, COLOR, or SIZE subcommands.

LABEL. *Labels points with the actual values corresponding to the dependent axis (* VAL *), the percentage of cases (* PCT *), and the number of cases included in each data point (* N *).* The default is no labels.

LINELABEL. *Labels lines with category labels (* CAT *), the percentage of cases (* PCT *), and the number of cases included in each line (* N *).* The default is no labels.

BREAK. *Indicates whether the lines break at missing values (* MISSING *) or not (* NONE *).*

The INTERPOLATE keyword determines how the lines connecting the points are drawn. Options include:

STRAIGHT. *Straight lines.*

LSTEP. *A horizontal line extends from each data point.* A vertical riser connects the line to the next data point.

CSTEP. *Each data point is centered on a horizontal line that extends half of the distance between consecutive points.* Vertical risers connect the line to the next horizontal line.

RSTEP. *A horizontal line terminates at each data point.* A vertical riser extends from each data point, connecting to the next horizontal line.

LJUMP. *A horizontal line extends from each data point.* No vertical risers connect the lines to the points.

RJUMP. *A horizontal line terminates at each data point.* No vertical risers connect the points to the next horizontal line.

CJUMP. *A horizontal line is centered at each data point, extending half of the distance between consecutive points.* No vertical risers connect the lines.

SPLINE. *Connects data points with a cubic spline.*

LAGRANGE3. This is no longer supported and is now an alias for SPLINE.

LAGRANGE5. This is no longer supported and is now an alias for SPLINE.

Example

```
IGRAPH
 /X1=VAR(volume95) TYPE=CATEGORICAL
 /Y=VAR(sales96) TYPE=SCALE
 /COLOR=VAR(volume94) TYPE=CATEGORICAL
 /COORDINATE=VERTICAL
 /LINE (MEAN) STYLE=LINE DROPLINE=ON LABEL VAL
 INTERPOLATE=STRAIGHT BREAK=MISSING.
```

- LINE creates a line chart. The lines represent the mean value of *sales96* for each category of *volume95*.
- The chart contains a line for each category of *volume94*, with droplines connecting the lines at each category of *volume95*.
- LABEL labels the lines with the mean *sales96* value for each category of *volume95*.
- INTERPOLATE specifies that straight lines connect the mean *sales96* values across the *volume95* categories.
- BREAK indicates that the lines will break at any missing values.

ERRORBAR Subcommand

Error bars help you to visualize distributions and dispersion by indicating the variability of the measure being displayed. The mean of a scale variable is plotted for a set of categories, and the length of an error bar on either side of the mean value indicates a confidence interval or a specified number of standard errors or standard deviations. Error bars can extend in one direction or in both directions from the mean. Error bars are sometimes displayed in the same chart with other chart elements, such as bars.

One of the following three keywords indicating the statistic and percentage/multiplier applied to the error bars must be specified:

CI(Pct). *Error bars represent confidence intervals.* Pct indicates the level of confidence and varies from 0 to 100.

SD(sdval). *Error bars represent standard deviations.* Sdval indicates how many standard deviations above and below the mean the error bars extend. Sdval must between 0 and 6.

SE(seval). *Error bars represent standard errors.* Seval indicates how many standard errors above and below the mean the error bars extend. Seval must between 0 and 6.

In addition, the following keywords can be specified:

LABEL. *Labels error bars with means (* VAL *) and the number of cases (* N *).*

DIRECTION. This keyword is no longer supported and is ignored. Error bars always extend both above and below the mean values.

CAPSTYLE. *For error bars, the style can be* T*-shaped (* T *) or no cap (* NONE *).* The default style is *T*-shaped. FANCY is no longer supported and is ignored.

SYMBOL. *Displays the mean marker (* ON *).* For no symbol, specify OFF.

BASELINE val. *Defines the value (* val *) above which the error bars extend above the bars and below which the error bars extend below the bars.*

CAPWIDTH(pct). *Controls the width of the cap relative to the distance between categories.* Pct equals the percent of the distance. The default value for pct is 45.

Example

```
IGRAPH
 /X1=VAR(volume94) TYPE=CATEGORICAL
 /Y=VAR(sales96) TYPE=SCALE
 /BAR (MEAN) LABEL INSIDE VAL SHAPE=RECTANGLE BASELINE=0.00
 /ERRORBAR SE(2.0) CAPWIDTH (45) CAPSTYLE=NONE.
```

- BAR creates a bar chart with rectangular bars. The bars represent the mean *sales96* values for the *volume94* categories.
- ERRORBAR adds error bars to the bar chart. The error bars extend two standard errors above and below the mean.

HISTOGRAM Subcommand

HISTOGRAM creates a histogram element in a chart, corresponding to the *X1*, *X2*, and *Y* axis assignments. Horizontal or vertical orientation is specified by the COORDINATE subcommand. A histogram groups the values of a variable into evenly spaced groups (intervals or bins) and plots a count of the number of cases in each group. The count can be expressed as a percentage. Percentages are useful for comparing datasets of different sizes. The count or percentage can also be accumulated across the groups.

- $COUNT or $PCT must be specified on the Y subcommand.

The following keywords are available:

SHAPE. *Defines the shape of the histogram*. Currently, the only value for SHAPE is HISTOGRAM.

CUM. *Specifies a cumulative histogram.* Counts or percentages are aggregated across the values of the domain variables.

X1INTERVAL. *Intervals on the* X1 *axis can be set automatically, or you can specify the number of intervals (1 to 250) along the axis* (NUM) *or the width of an interval* (WIDTH).

X2INTERVAL. *Intervals on the* X2 *axis can be set automatically, or you can specify the number of intervals (1 to 250) along the axis* (NUM) *or the width of an interval* (WIDTH).

CURVE. *Superimposes a normal curve on a 2-D histogram.* The normal curve has the same mean and variance as the data.

X1START. *The starting point along the* X1 *axis.* Indicates the percentage of an interval width above the minimum value along the *X1* axis at which to begin the histogram. The value can range from 0 to 99.

X2START. *The starting point along the* X2 *axis.* Indicates the percentage of an interval width above the minimum value along the *X2* axis at which to begin the histogram. The value can range from 0 to 99.

Example

```
IGRAPH
 /X1=VAR(sales96) TYPE=SCALE
 /Y=$count
 /Histogram SHAPE=HISTOGRAM CURVE=ON X1INTERVAL WIDTH=100.
```

- Histogram creates a histogram of *sales96*. The *sales96* intervals are 100 units wide.
- CURVE superimposes a normal curve on the histogram.

FITLINE Subcommand

FITLINE adds a line or surface to a scatterplot to help you discern the relationship shown in the plot. The following general methods are available:

NONE. *No line is fit.*

REGRESSION. *Fits a straight line (or surface) using ordinary least squares.* Must be followed by the keyword LINEAR.

ORIGIN. *Fits a straight line (or surface) through the origin.* Must be followed by the keyword LINEAR.

MEAN. *For a 2-D chart, fits a line at the mean of the dependent (Y) variable.* For a 3-D chart, the *Y* mean is shown as a plane.

LLR. *Fits a local linear regression curve or surface.* A normal (NORMAL) kernel is the default. With EPANECHNIKOV, the curve is not as smooth as with a normal kernel and is smoother than with a uniform (UNIFORM) kernel.

The keyword LINE indicates the number of fit lines. TOTAL fits the line to all of the cases. MEFFECT fits a separate line to the data for each value of a legend variable.

The REGRESSION, ORIGIN, and MEAN methods offer the option of including prediction intervals with the following keyword:

INTERVAL[(cval)]. *The intervals are based on the mean* (MEAN) *or on the individual cases* (INDIVIDUAL). Cval indicates the size of the interval and ranges from 50 to 100.

The local linear regression (LLR) smoother offers the following controls for the smoothing process:

BANDWIDTH. *Constrains the bandwidth to be constant across subgroups or panels* (CONSTRAINED). The default is unconstrained (FAST).

X1MULTIPLIER. *Specifies the bandwidth multiplier for the* X1 *axis.* The bandwidth multiplier changes the amount of data that is included in each calculation of a small part of the smoother. The multiplier can be adjusted to emphasize specific features of the plot that are of interest. Any positive multiplier (including fractions) is allowed. The larger the multiplier, the smoother the curve. The range between 0 and 10 should suffice in most applications.

X2MULTIPLIER. *Specifies the bandwidth multiplier for the* X2 *axis.* The bandwidth multiplier changes the amount of data that is included in each calculation of a small part of the smoother. The multiplier can be adjusted to emphasize specific features of the plot that are of interest. Any positive multiplier (including fractions) is allowed. The larger the multiplier, the smoother the curve. The range between 0 and 10 should suffice in most applications.

Example

```
IGRAPH
 /X1=VAR(sales95) TYPE=SCALE
 /Y=VAR(sales96) TYPE=SCALE
 /COLOR=VAR(region) TYPE=CATEGORICAL
 /SCATTER
 /FITLINE METHOD=LLR EPANECHNIKOV BANDWIDTH=CONSTRAINED
 X1MULTIPLIER=2.00 LINE=MEFFECT.
```

- SCATTER creates a scatterplot of *sales95* and *sales96*.
- FITLINE adds a local linear regression smoother to the scatterplot. The Epanechnikov smoother is used with an *X1* multiplier of 2. A separate line is fit for each category of region, and the bandwidth is constrained to be equal across region categories.

Summary Functions

Summary functions apply to scale variables selected for a dependent axis or a slice summary. Percentages are based on the specified percent base. For a slice summary, only summary functions appropriate for the type of chart are available.

The following summary functions are available:

First Values (FIRST). The value found in the first case for each category in the data file at the time the summary was defined.

Kurtosis (KURTOSIS). A measure of the extent to which observations cluster around a central point. For a normal distribution, the value of the kurtosis statistic is 0. Positive kurtosis indicates that the observations cluster more and have longer tails than those in the normal distribution, and negative kurtosis indicates the observations cluster less and have shorter tails.

Last Values (LAST). The value found in the last case for each category in the data file at the time the summary was defined.

Maximum Values (MAXIMUM). The largest value for each category.

Minimum Values (MINIMUM). The smallest value within the category.

Means (MEAN). The arithmetic average for each category.

Medians (MEDIAN). The values below which half of the cases fall in each category.

Modes (MODE). The most frequently occurring value within each category.

Number of Cases Above (NGT(x)). The number of cases having values above the specified value.

Number of Cases Between (NIN(x1,x2)). The number of cases between two specified values.

Number of Cases Equal to (NEQ(x)). The number of cases equal to the specified value.

Number of Cases Greater Than or Equal to (NGE(x)). The number of cases having values above or equal to the specified value.

Number of Cases Less Than (NLT(x)). The number of cases below the specified value.

Number of Cases Less Than or Equal to (NLE(x)). The number of cases below or equal to the specified value.

Percentage of Cases Above (PGT(x)). The percentage of cases having values above the specified value.

Percentage of Cases Between (PIN(x1,x2)). The percentage of cases between two specified values.

Percentage of Cases Equal to (PEQ(x)). The percentage of cases equal to the specified value.

Percentage of Cases Greater Than or Equal to (PGE(x)). The percentage of cases having values above or equal to the specified value.

Percentage of Cases Less Than (PLT(x)). The percentage of cases having values below the specified value.

Percentage of Cases Less Than or Equal to (PLE(x)). The percentage of cases having values below or equal to the specified value.

Percentiles (PTILE(x)). The data value below which the specified percentage of values fall within each category.

Skewness (SKEW). A measure of the asymmetry of a distribution. The normal distribution is symmetric and has a skewness value of 0. A distribution with a significant positive skewness has a long right tail. A distribution with a significant negative skewness has a long left tail.

Standard Deviations (STDDEV). A measure of dispersion around the mean, expressed in the same units of measurement as the observations, equal to the square root of the variance. In a normal distribution, 68% of cases fall within one SD of the mean and 95% of cases fall within two SD's.

Standard Errors of Kurtosis (SEKURT). The ratio of kurtosis to its standard error can be used as a test of normality (that is, you can reject normality if the ratio is less than –2 or greater than +2). A large positive value for kurtosis indicates that the tails of the distribution are longer than those of a normal distribution; a negative value for kurtosis indicates shorter tails (becoming like those of a box-shaped uniform distribution).

Standard Errors of the Mean (SEMEAN). A measure of how much the value of the mean may vary from sample to sample taken from the same distribution. It can be used to roughly compare the observed mean to a hypothesized value (that is, you can conclude the two values are different if the ratio of the difference to the standard error is less than –2 or greater than +2).

Standard Errors of Skewness (SESKEW). The ratio of skewness to its standard error can be used as a test of normality (that is, you can reject normality if the ratio is less than –2 or greater than +2). A large positive value for skewness indicates a long right tail; an extreme negative value, a long left tail.

Sums (SUM). The sums of the values within each category.

Sums of Absolute Values (SUMAV). The sums of the absolute values within each category.

Sums of Squares (SUMSQ). The sums of the squares of the values within each category.

Variances (VARIANCE). A measure of how much observations vary from the mean, expressed in squared units.

IMPORT

```
IMPORT FILE='file'
 [/TYPE={COMM}]
        {TAPE}
 [/KEEP={ALL**  }] [/DROP=varlist]
        {varlist}
 [/RENAME=(old varnames=new varnames)...]
 [/MAP]
```

**Default if the subcommand is omitted.

Example

```
IMPORT FILE='/data/newdata.por'.
```

Overview

IMPORT reads portable data files created with the EXPORT command. A portable data file is a data file created by the program and used to transport data between different types of computers and operating systems (such as between IBM CMS and Digital VAX/VMS) or between IBM SPSS Statistics and other software using the same portable file format. A portable file contains all of the data and dictionary information stored in the active dataset from which it was created.

The program can also read data files created by other software programs. See GET DATA and GET TRANSLATE for information on reading files created by spreadsheet and database programs such as dBASE, Lotus, and Excel.

Options

Format. You can specify the format of the portable file (magnetic tape or communications program) on the TYPE subcommand.

Variables. You can read a subset of variables from the active dataset with the DROP and KEEP subcommands. You can rename variables using RENAME. You can also produce a record of all variables and their names in the active dataset with the MAP subcommand.

Basic Specification

The basic specification is the FILE subcommand with a file specification. All variables from the portable file are copied into the active dataset with their original names, variable and value labels, missing-value flags, and print and write formats.

Subcommand Order

- FILE and TYPE must precede all other subcommands.
- No specific order is required between FILE and TYPE or among other subcommands.

Operations

- The portable data file and dictionary become the active dataset and dictionary.
- A file saved with weighting in effect (using the WEIGHT command) automatically uses the case weights when the file is read.

Examples

```
IMPORT FILE="/data/newdata.por"
  /RENAME=(V1 TO V3=ID,SEX,AGE) /MAP.
```

- The active dataset is generated from the portable file *newdata.por*.
- Variables *V1, V2,* and *V3* are renamed *ID, SEX,* and *AGE* in the active dataset. Their names remain *V1, V2,* and *V3* in the portable file. None of the other variables copied into the active dataset are renamed.
- MAP requests a display of the variables in the active dataset.

FILE Subcommand

FILE specifies the portable file. FILE is the only required subcommand on IMPORT.

TYPE Subcommand

TYPE indicates whether the portable file is formatted for magnetic tape or for a communications program. TYPE can specify either COMM or TAPE. For more information on magnetic tapes and communications programs, see EXPORT.

COMM. *Communications-formatted file.* This is the default.

TAPE. *Tape-formatted file.*

Example

```
IMPORT TYPE=TAPE /FILE='hubout.por'.
```

- The file *hubout.por* is read as a tape-formatted portable file.

DROP and KEEP Subcommands

DROP and KEEP are used to read a subset of variables from the portable file.

- DROP excludes a variable or list of variables from the active dataset. All variables not named are included in the file.
- KEEP includes a variable or list of variables in the active dataset. All variables not specified on KEEP are excluded.
- DROP and KEEP cannot precede the FILE or TYPE subcommands.
- Variables can be specified in any order. The order of variables on KEEP determines the order of variables in the active dataset. The order on DROP does not affect the order of variables in the active dataset.
- If a variable is referred to twice on the same subcommand, only the first mention is recognized.
- Multiple DROP and KEEP subcommands are allowed; the effect is cumulative. Specifying a variable named on a previous DROP or not named on a previous KEEP results in an error and the command is not executed.
- The keyword TO can be used to specify a group of consecutive variables in the portable file.
- The portable file is not affected by DROP or KEEP.

Example

```
IMPORT FILE='/data/newsum.por'
  /DROP=DEPT TO DIVISION.
```

- The active dataset is generated from the portable file *newsum.por*. Variables between and including *DEPT* and *DIVISION* in the portable file are excluded from the active dataset.
- All other variables are copied into the active dataset.

RENAME Subcommand

RENAME renames variables being read from the portable file. The renamed variables retain the variable and value labels, missing-value flags, and print formats contained in the portable file.

- To rename a variable, specify the name of the variable in the portable file, a required equals sign, and the new name.
- A variable list can be specified on both sides of the equals sign. The number of variables on both sides must be the same, and the entire specification must be enclosed in parentheses.
- The keyword TO can be used for both variable lists.
- Any DROP or KEEP subcommand after RENAME must use the new variable names.

Example

```
IMPORT FILE='/data/newsum.por'
  /DROP=DEPT TO DIVISION
  /RENAME=(NAME,WAGE=LNAME,SALARY).
```

- RENAME renames *NAME* and *WAGE* to *LNAME* and *SALARY*.
- *LNAME* and *SALARY* retain the variable and value labels, missing-value flags, and print formats assigned to *NAME* and *WAGE*.

MAP Subcommand

MAP displays a list of variables in the active dataset, showing all changes that have been specified on the RENAME, DROP, or KEEP subcommands.

- MAP can be specified as often as desired.
- MAP confirms only the changes specified on the subcommands that precede the MAP request.
- Results of subcommands that follow MAP are not mapped. When MAP is specified last, it also produces a description of the file.

Example

```
IMPORT FILE='/data/newsum.por'
  /DROP=DEPT TO DIVISION /MAP
  /RENAME NAME=LNAME WAGE=SALARY /MAP.
```

- The first MAP subcommand produces a listing of the variables in the file after DROP has dropped the specified variables.
- RENAME renames *NAME* and *WAGE*.
- The second MAP subcommand shows the variables in the file after renaming.

INCLUDE

```
INCLUDE FILE='file' [PASSWORD='password']
       [ENCODING = 'encoding specification']
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 16.0

- ENCODING keyword added for Unicode support.

Release 22.0

- PASSWORD keyword introduced on the FILE subcommand.

Example

```
INCLUDE FILE='/data/gsslabs.sps'.
```

Overview

INCLUDE includes a file of commands in a session. INCLUDE is especially useful for including a long series of data definition statements or transformations. Another use for INCLUDE is to set up a library of commonly used commands and include them in the command sequence as they are needed.

Note: The newer INSERT provides equivalent functionality, plus additional features not available with INCLUDE. See the topic “INSERT” on page 945 for more information.

INCLUDE allows you to run multiple commands together during a session and can save time. Complex or repetitive commands can be stored in a command file and included in the session, while simpler commands or commands unique to the current analysis can be entered during the session, before and after the included file.

Basic Specification

The only specification is the FILE subcommand, which specifies the file to include. When INCLUDE is executed, the commands in the specified file are processed.

Syntax Rules

- Commands in an included file must begin in column 1, and continuation lines for each command must be indented at least one column.
- The maximum line length for a command syntax file run via the INCLUDE command is 256 characters. Any characters beyond this limit are truncated.
- As many INCLUDE commands as needed can be used in a session.
- INCLUDE commands can be nested so that one set of included commands includes another set of commands. This nesting can go to five levels. However, a file cannot be included that is still open from a previous step.

Operations

- If an included file contains a FINISH command, the session ends and no further commands are processed.
- If a journal file is created for the session, INCLUDE is copied to the journal file. Commands from the included file are also copied to the journal file but are treated like printed messages. Thus, INCLUDE can be executed from the journal file if the journal file is later used as a command file. Commands from the included file are executed only once.

ENCODING Keyword

ENCODING specifies the encoding format of the file. The keyword is followed by an equals sign and a quoted encoding specification.

- In Unicode mode, the default is UTF8. For more information, see SET command, UNICODE subcommand.
- In code page mode, the default is the current locale setting. For more information, see SET command, LOCALE subcommand.
- The quoted encoding value can be: Locale (the current locale setting), UTF8, UTF16, UTF16BE (big endian), UTF16LE (little endian), a numeric Windows code page value (for example, '1252'), or an IANA code page value (for example, 'iso8859-1' or cp1252).

Examples

```
INCLUDE FILE='/data/gsslabs.sps'.
```

- INCLUDE includes the file *gsslabs.sps* in the prompted session. When INCLUDE is executed, the commands in *gsslabs.sps* are processed.
- Assume that the include file *gsslabs.sps* contains the following:

```
DATA LIST FILE='/data/data52.txt'
  /RELIGION 5 OCCUPAT 7 SES 12 ETHNIC 15
   PARTY 19 VOTE48 33 VOTE52 41.
```

The active dataset will be defined and ready for analysis after INCLUDE is executed.

FILE Subcommand

FILE identifies the file that contains the commands and is required.

PASSWORD Keyword

The PASSWORD keyword specifies the password required to open an encrypted IBM SPSS Statistics syntax file. The specified value must be enclosed in quotation marks and can be provided as encrypted or as plain text. For reference, passwords are always encrypted in pasted syntax. The PASSWORD keyword is ignored if the file is not encrypted.

INFO

This command is obsolete and no longer supported.

INPUT PROGRAM-END INPUT PROGRAM

```
INPUT PROGRAM

commands to create or define cases

END INPUT PROGRAM
```

Example

```
INPUT PROGRAM.
DATA LIST FILE=PRICES /YEAR 1-4 QUARTER 6 PRICE 8-12(2).

DO IF (YEAR GE 1881).  /*Stop reading before 1881
END FILE.
END IF.
END INPUT PROGRAM.
```

Overview

The INPUT PROGRAM and END INPUT PROGRAM commands enclose data definition and transformation commands that build cases from input records. The input program often encloses one or more DO IF-END IF or LOOP-END LOOP structures, and it must include at least one file definition command, such as DATA LIST. One of the following utility commands is also usually used:

END CASE. *Build cases from the commands within the input program and pass the cases to the commands immediately following the input program*.

END FILE. *Terminate processing of a data file before the actual end of the file or define the end of the file when the input program is used to read raw data*.

REREAD. *Reread the current record using a different* DATA LIST.

REPEATING DATA. *Read repeating groups of data from the same input record*.

For more information on the commands used in an input program, see the discussion of each command.

Input programs create a dictionary and data for an active dataset from raw data files; they cannot be used to read IBM SPSS Statistics data files. They can be used to process direct-access and keyed data files. For details, see KEYED DATA LIST.

Basic Specification

The basic specification is INPUT PROGRAM, the commands used to create cases and define the active dataset, and END INPUT PROGRAM.

- INPUT PROGRAM and END INPUT PROGRAM each must be specified on a separate line and have no additional specifications.
- To define an active dataset, the input program must include at least one DATA LIST or END FILE command.

Operations

- The INPUT PROGRAM-END INPUT PROGRAM structure defines an active dataset and is not executed until the program encounters a procedure or the EXECUTE command.
- INPUT PROGRAM clears the current active dataset.

Examples

Select Cases with an Input Program

```
INPUT PROGRAM.
DATA LIST FILE=PRICES /YEAR 1-4 QUARTER 6 PRICE 8-12(2).
```

```
DO IF (YEAR GE 1881).  /*Stop reading when reaching 1881
END FILE.
END IF.
END INPUT PROGRAM.

LIST.
```

- The input program is defined between the INPUT PROGRAM and END INPUT PROGRAM commands.
- This example assumes that data records are entered chronologically by year. The DO IF-END IF structure specifies an end of file when the first case with a value of 1881 or later for *YEAR* is reached.
- LIST executes the input program and lists cases in the active dataset. The case that causes the end of the file is not included in the active dataset generated by the input program.
- As an alternative to this input program, you can use N OF CASES to select cases if you know the exact number of cases. Another alternative is to use SELECT IF to select cases before 1881, but then the program would unnecessarily read the entire input file.

Skip the First n Records in a File

```
INPUT PROGRAM.
NUMERIC        #INIT.
DO IF          NOT (#INIT).
+  LOOP           #I = 1 TO 5.
+     DATA LIST      NOTABLE/.   /* No data - just skip record
+  END LOOP.
+  COMPUTE        #INIT = 1.
END IF.
DATA LIST      NOTABLE/ X 1.
END INPUT PROGRAM.

BEGIN DATA
A                          /* The first 5 records are skipped
B
C
D
E
1
2
3
4
5
END DATA.
LIST.
```

- NUMERIC declares the scratch variable *#INIT*, which is initialized to system-missing.
- The DO IF structure is executed as long as *#INIT* does not equal 1.
- LOOP is executed five times. Within the loop, DATA LIST is specified without variable names, causing the program to read records in the data file without copying them into the active dataset. LOOP is executed five times, so the program reads five records in this manner. END LOOP terminates this loop.
- COMPUTE creates the scratch variable *#INIT* and sets it equal to 1. The DO IF structure is therefore not executed again.
- END IF terminates the DO IF structure.
- The second DATA LIST specifies numeric variable *X*, which is located in column 1 of each record. Because the program has already read five records, the first value for *X* that is copied into the active dataset is read from record 6.

Input Programs

The program builds the active dataset dictionary when it encounters commands that create and define variables. At the same time, the program builds an *input program* that constructs cases and an optional *transformation program* that modifies cases prior to analysis or display. By the time the program encounters a procedure command that tells it to read the data, the active dataset dictionary is ready, and the programs that construct and modify the cases in the active dataset are built.

The internal input program is usually built from either a single DATA LIST command or from any of the commands that read or combine IBM SPSS Statistics data files (for example, GET, ADD FILES, MATCH FILES, UPDATE, and so on). The input program can also be built from the FILE TYPE-END FILE TYPE structure used to define nested, mixed, or grouped files. The third type of input program is specified with the INPUT PROGRAM-END INPUT PROGRAM commands.

With INPUT PROGRAM-END INPUT PROGRAM, you can create your own input program to perform many different operations on raw data. You can use transformation commands to build cases. You can read nonrectangular files, concatenate raw data files, and build cases selectively. You can also create an active dataset without reading any data at all.

Input State

There are four program states in the program: the *initial state,* in which there is no active dataset dictionary; the *input state,* in which cases are created from the input file; the *transformation state,* in which cases are transformed; and the *procedure state,* in which procedures are executed. When you specify INPUT PROGRAM-END INPUT PROGRAM, you must pay attention to which commands are allowed within the input state, which commands can appear only within the input state, and which are not allowed within the input state.

More Examples

For additional examples of input programs, refer to DATA LIST, DO IF, DO REPEAT, END CASE, END FILE, LOOP, NUMERIC, POINT, REPEATING DATA, REREAD, and VECTOR.

INSERT

Note: Equals signs (=) used in the syntax chart are required elements.

```
INSERT  FILE='file specification' [PASSWORD='password']
       [SYNTAX = {INTERACTIVE*}]
                 {BATCH       }
       [ERROR = {CONTINUE*}]
                {STOP     }
       [CD = {NO*}]
             {YES}
       [ENCODING = 'encoding specification']
```

*Default if keyword omitted.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic "Command Order" on page 36 for more information.

Release History

Release 13.0

- Command introduced.

Release 16.0

- ENCODING keyword added for Unicode support.

Release 22.0

- PASSWORD keyword introduced.

Example

```
INSERT FILE='/examples/commands/file1.sps'
  SYNTAX=BATCH ERROR=STOP CD=YES ENCODING='UTF8'.
```

OVERVIEW

INSERT includes a file of commands in a session. INSERT is especially useful for including a long series of data definition statements or transformations. Another use for INSERT is to set up a library of commonly used commands and include them in the command sequence as they are needed.

INSERT allows you to run multiple commands together during a session and can save time. Complex or repetitive commands can be stored in a command file and included in the session, while simpler commands or commands unique to the current analysis can be entered during the session, before and after the included file.

INSERT provides the same basic functionality as INCLUDE, plus the ability to:

- Insert files that use either batch or interactive syntax rules.
- Control treatment of error conditions in inserted files.
- Change the working directory to the directory containing an inserted file.

Limitations

The maximum line length for a command syntax file run via the INSERT command is 256 characters. Any characters beyond this limit are truncated.

FILE Keyword

The minimum specification is the FILE keyword, followed by an equals sign and a quoted file specification (or quoted file handle) that specifies the file to insert. When the INSERT command is run, the commands in the specified file are processed.

PASSWORD Keyword

The PASSWORD keyword specifies the password required to open an encrypted IBM SPSS Statistics syntax file. The specified value must be enclosed in quotation marks. The PASSWORD keyword is ignored if the file is not encrypted.

To create an encrypted syntax file with a password:

1. From the menus in the syntax window, select **File** > **Save As**.
2. For the file type, select **Encrypted Syntax (*.spsx)**.

Example

```
INSERT FILE='/examples/commands/file1.sps'.
INSERT FILE='/examples/commands/file2.spsx PASSWORD='swordfish'.
```

SYNTAX Keyword

The optional SYNTAX keyword specifies the syntax rules that apply to the inserted file. The keyword is followed by an equals sign (=) and one of the following alternatives:

INTERACTIVE
Each command must end with a period. Periods can appear anywhere within the command, and commands can continue on multiple lines, but a period as the last non-blank character on a line is interpreted as the end of the command. Continuation lines and new commands can start anywhere on a new line. These are the "interactive" rules in effect when you select and run commands in a syntax window. This is the default if the SYNTAX keyword is omitted.

BATCH
Each command must start at the beginning of a new line (no blank spaces before the start of the command), and continuation lines must be indented at least one space. If you want to indent new commands, you can use a plus sign, dash, or period as the first character at the start of the line and then indent the actual command. The period at the end of the command is optional. This setting is compatible with the syntax rules for command files included with the INCLUDE command.

Command syntax created with the Paste button in dialogs will work in either interactive or batch modes.

ERROR Keyword

The optional ERROR keyword controls the handling of error conditions in inserted files. The keyword is followed by an equals sign (=) and one of the following alternatives:

CONTINUE
Errors in inserted files do not automatically stop command processing. The inserted commands are treated as part of the normal command stream, and command processing continues in the normal fashion. This is the default if the ERROR keyword is omitted.

STOP
Command processing stops when the first error in an inserted file is encountered. This is compatible with the behavior of command files included with the INCLUDE command.

CD Keyword

The optional CD keyword can specify the directory containing the inserted file as the working directory, making it possible to use relative paths for file specifications within the inserted file. The keyword is followed by an equals sign (=) and one of the following alternatives:

NO. *The working directory is not changed.* This is the default if the CD keyword is omitted.

YES. *The working directory is changed to the directory containing the inserted file.* Subsequent relative paths in command file specifications are interpreted as being relative to the location of the inserted file.

The change in the working directory remains in effect until some other condition occurs that changes the working directory during the session, such as explicitly changing the working directory on another INSERT command with a CD keyword or a CD command that specifies a different directory (see) .

The CD keyword has no effect on the relative directory location for SET command file specifications, including JOURNAL , CTEMPLATE, and TLOOK. File specifications on the SET command should include complete path information.

The original working directory can be preserved with the PRESERVE command and later restored with the RESTORE command, as in:

```
PRESERVE.
INSERT FILE='/commands/examples/file1.sps'
  CD=YES.
INSERT FILE='file2.sps'.
RESTORE.
```

- PRESERVE retains the original working directory location.
- The first INSERT command changes the working directory.
- The second INSERT command will look for `file2.sps` in `/commands/examples`.
- RESTORE resets the working directory to whatever it was prior to the first INSERT command.

For more information, see the PRESERVE and RESTORE commands.

ENCODING Keyword

ENCODING specifies the encoding format of the file. The keyword is followed by an equals sign and a quoted encoding specification.

- In Unicode mode, the default is UTF8. For more information, see SET command, UNICODE subcommand.
- In code page mode, the default is the current locale setting. For more information, see SET command, LOCALE subcommand.
- The quoted encoding value can be: `Locale` (the current locale setting), UTF8, UTF16, UTF16BE (big endian), UTF16LE (little endian), a numeric Windows code page value (for example, '1252'), or an IANA code page value (for example, 'iso8859-1' or cp1252).

INSERT vs. INCLUDE

INSERT is a newer, more powerful and flexible alternative to INCLUDE. Files included with INCLUDE must always adhere to batch syntax rules, and command processing stops when the first error in an included file is encountered. You can effectively duplicate the INCLUDE behavior with SYNTAX=BATCH and ERROR=STOP on the INSERT command.

INSERT EXEC

Note: Equals signs (=) used in the syntax chart are required elements.

```
INSERT EXEC
     [/SUCCESS FILE='file specification']
          [PASSWORD='password']
          [SYNTAX={INTERACTIVE*}]
                  {BATCH}
          [ENCODING='encoding specification']
     [/FAILURE FILE='file specification']
          [PASSWORD='password']
          [SYNTAX={INTERACTIVE*}]
                  {BATCH}
          [ENCODING='encoding specification']
     [/CLEAR]
```

*Default if keyword omitted.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 25.0 Fix Pack 2

SUCCESS subcommand

The **/SUCCESS** subcommand executes syntax when there are no fatal errors after the latest Fatal Error Count Reset. The **FILE** keyword is required when **/SUCCESS** is specified (other keywords are optional).

The subcommand does nothing in instances where there is at least one fatal error after the latest Fatal Error Count Reset.

FAILURE subcommand

The **/FAILURE** subcommand executes syntax when there is at least one fatal error after the latest Fatal Error Count Reset. The **FILE** keyword is required when **/FAILURE** is specified (other keywords are optional).

The subcommand does nothing in instances where there are no fatal errors after the latest Fatal Error Count Reset.

CLEAR subcommand

The **/CLEAR** subcommand resets the system's fatal error count to zero. When the **/CLEAR** subcommand is specified, neither **/SUCCESS** nor **/FAILURE** can be specified.

Example

```
INSERT EXEC
     /CLEAR
```

FILE Keyword

The minimum specification is the FILE keyword, followed by an equals sign and a quoted file specification (or quoted file handle) that specifies the file to insert. When the INSERT command is run, the commands in the specified file are processed.

PASSWORD Keyword

The PASSWORD keyword specifies the password required to open an encrypted IBM SPSS Statistics syntax file. The specified value must be enclosed in quotation marks. The PASSWORD keyword is ignored if the file is not encrypted.

To create an encrypted syntax file with a password:

1. From the menus in the syntax window, select **File** > **Save As**.
2. For the file type, select **Encrypted Syntax (*.spsx)**.

Example

```
INSERT EXEC
     /SUCCESS FILE='/examples/commands/file2.spsx PASSWORD='swordfish'.
```

SYNTAX Keyword

The optional SYNTAX keyword specifies the syntax rules that apply to the inserted file. The keyword is followed by an equals sign (=) and one of the following alternatives:

INTERACTIVE
Each command must end with a period. Periods can appear anywhere within the command, and commands can continue on multiple lines, but a period as the last non-blank character on a line is interpreted as the end of the command. Continuation lines and new commands can start anywhere on a new line. These are the "interactive" rules in effect when you select and run commands in a syntax window. This is the default if the SYNTAX keyword is omitted.

BATCH
Each command must start at the beginning of a new line (no blank spaces before the start of the command), and continuation lines must be indented at least one space. If you want to indent new commands, you can use a plus sign, dash, or period as the first character at the start of the line and then indent the actual command. The period at the end of the command is optional. This setting is compatible with the syntax rules for command files included with the INCLUDE command.

Command syntax created with the Paste button in dialogs will work in either interactive or batch modes.

ENCODING Keyword

ENCODING specifies the encoding format of the file. The keyword is followed by an equals sign and a quoted encoding specification.

- In Unicode mode, the default is UTF8. For more information, see SET command, UNICODE subcommand.
- In code page mode, the default is the current locale setting. For more information, see SET command, LOCALE subcommand.
- The quoted encoding value can be: Locale (the current locale setting), UTF8, UTF16, UTF16BE (big endian), UTF16LE (little endian), a numeric Windows code page value (for example, '1252'), or an IANA code page value (for example, 'iso8859-1' or cp1252).

INSERT HIDDEN

Note: Equals signs (=) used in the syntax chart are required elements.

```
INSERT HIDDEN
     SOURCE='source specification'
     [SYNTAX = {INTERACTIVE*}]
               {BATCH}
     [ERROR = {CONTINUE*}]
              {STOP}
     [ENCODING = 'encoding specification']
```

*Default if keyword omitted.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 25.0 Fix Pack 2

SOURCE keyword

The required **SOURCE** keyword specifies a physical syntax file from an external source whose syntax usage is not written to the command log or journal. The keyword is followed by an equals sign and a quoted file specification (or quoted file handle) that specifies the syntax file to insert. When the INSERT HIDDEN command is run, the commands in the specified syntax file are processed.

Example

```
INSERT HIDDEN SOURCE='C:\Procedures\Insert\Syntax\hidden_syntax.sps'.
```

SYNTAX Keyword

The optional SYNTAX keyword specifies the syntax rules that apply to the inserted file. The keyword is followed by an equals sign (=) and one of the following alternatives:

INTERACTIVE
Each command must end with a period. Periods can appear anywhere within the command, and commands can continue on multiple lines, but a period as the last non-blank character on a line is interpreted as the end of the command. Continuation lines and new commands can start anywhere on a new line. These are the "interactive" rules in effect when you select and run commands in a syntax window. This is the default if the SYNTAX keyword is omitted.

BATCH
Each command must start at the beginning of a new line (no blank spaces before the start of the command), and continuation lines must be indented at least one space. If you want to indent new commands, you can use a plus sign, dash, or period as the first character at the start of the line and then indent the actual command. The period at the end of the command is optional. This setting is compatible with the syntax rules for command files included with the INCLUDE command.

Command syntax created with the Paste button in dialogs will work in either interactive or batch modes.

ERROR Keyword

The optional ERROR keyword controls the handling of error conditions in inserted files. The keyword is followed by an equals sign (=) and one of the following alternatives:

CONTINUE
Errors in inserted files do not automatically stop command processing. The inserted commands are treated as part of the normal command stream, and command processing continues in the normal fashion. This is the default if the ERROR keyword is omitted.

STOP
Command processing stops when the first error in an inserted file is encountered. This is compatible with the behavior of command files included with the INCLUDE command.

ENCODING Keyword

ENCODING specifies the encoding format of the file. The keyword is followed by an equals sign and a quoted encoding specification.

- In Unicode mode, the default is UTF8. For more information, see SET command, UNICODE subcommand.
- In code page mode, the default is the current locale setting. For more information, see SET command, LOCALE subcommand.
- The quoted encoding value can be: Locale (the current locale setting), UTF8, UTF16, UTF16BE (big endian), UTF16LE (little endian), a numeric Windows code page value (for example, '1252'), or an IANA code page value (for example, 'iso8859-1' or cp1252).

KEYED DATA LIST

```
KEYED DATA LIST KEY=varname IN=varname

 FILE='file' [{TABLE  }] [ENCODING='encoding specification']
             {NOTABLE}

 /varname {col location [(format)]} [varname ..]
          {(FORTRAN-like format)  }
```

** Default if subcommand or keyword omitted.

Release History

Release 16.0

- ENCODING subcommand added for Unicode support.

Example

```
FILE HANDLE EMPL/ file specifications.
KEYED DATA LIST FILE=EMPL KEY=#NXTCASE IN=#FOUND
          /YRHIRED 1-2 SEX 3 JOBCLASS 4.
```

Overview

KEYED DATA LIST reads raw data from two types of nonsequential files: direct-access files, which provide direct access by a record number, and keyed files, which provide access by a record key. An example of a direct-access file is a file of 50 records, each corresponding to one of the United States. If you know the relationship between the states and the record numbers, you can retrieve the data for any specific state. An example of a keyed file is a file containing social security numbers and other information about a firm's employees. The social security number can be used to identify the records in the file.

Direct-Access Files

There are various types of direct-access files. This program's concept of a direct-access file, however, is very specific. The file must be one from which individual records can be selected according to their number. The records in a 100-record direct-access file, for example, are numbered from 1 to 100.

Although the concept of record number applies to almost any file, not all files can be treated by this program as direct-access files. In fact, some operating systems provide no direct-access capabilities at all, and others permit only a narrowly defined subset of all files to be treated as direct access.

Very few files turn out to be good candidates for direct-access organization. In the case of an inventory file, for example, the usual large gaps in the part numbering sequence would result in large amounts of wasted file space. Gaps are not a problem, however, if they are predictable. For example, if you recognize that telephone area codes have first digits of 2 through 9, second digits of 0 or 1, and third digits of 0 through 9, you can transform an area code into a record number by using the following COMPUTE statement:

```
COMPUTE RECNUM = 20*(DIGIT1-2) + 10*DIGIT2 + DIGIT3 + 1.
```

where *DIGIT1*, *DIGIT2*, and *DIGIT3* are variables corresponding to the respective digits in the area code, and *RECNUM* is the resulting record number. The record numbers would range from 1, for the nonexistent area code 200, through 160, for area code 919. The file would then have a manageable number of unused records.

Keyed Files

Of the many kinds of keyed files, the ones to which the program can provide access are generally known as *indexed sequential files.* A file of this kind is basically a sequential file in which an index is maintained so that the file can be processed either sequentially or selectively. In effect, there is an underlying data file that is accessed through a file of index entries. The file of index entries may, for example, contain the

fact that data record 797 is associated with social security number 476-77-1359. Depending on the implementation, the underlying data may or may not be maintained in sequential order.

The key for each record in the file generally comprises one or more pieces of information found within the record. An example of a complex key is a customer's last name and house number, plus the consonants in the street name, plus the zip code, plus a unique digit in case there are duplicates. Regardless of the information contained in the key, the program treats it as a character string.

On some systems, more than one key is associated with each record. That is, the records in a file can be identified according to different types of information. Although the primary key for a file normally must be unique, sometimes the secondary keys need not be. For example, the records in an employee file might be identified by social security number and job classification.

Options

Data Source. You can specify the name of the keyed file on the FILE subcommand. By default, the last file that was specified on an input command, such as DATA LIST or REPEATING DATA, is read.

Summary Table. You can display a table that summarizes the variable definitions.

Basic Specification

- The basic specification requires FILE, KEY, and IN, each of which specifies one variable, followed by a slash and variable definitions.
- FILE specifies the direct-access or keyed file. The file must have a file handle already defined.
- KEY specifies the variable whose value will be used to read a record. For direct-access files, the variable must be numeric; for keyed files, it must be string.
- IN creates a logical variable that flags whether a record was successfully read.
- Variable definitions follow all subcommands; the slash preceding them is required. Variable definitions are similar to those specified on DATA LIST.

Subcommand Order

- Subcommands can be named in any order.
- Variable definitions must follow all specified subcommands.

Syntax Rules

- Specifications for the variable definitions are the same as those described for DATA LIST. The only difference is that only one record can be defined per case.
- The FILE HANDLE command must be used if the FILE subcommand is specified on KEYED DATA LIST.
- KEYED DATA LIST can be specified in an input program, or it can be used as a transformation language to change an existing active dataset. This differs from all other input commands, such as GET and DATA LIST, which create new active datasets.

Operations

- Variable names are stored in the active dataset dictionary.
- Formats are stored in the active dataset dictionary and are used to display and write the values. To change output formats of numeric variables, use the FORMATS command.

Examples

Specifying a Key Variable

```
FILE HANDLE EMPL/ file specifications.
KEYED DATA LIST FILE=EMPL KEY=#NXTCASE IN=#FOUND
          /YRHIRED 1-2 SEX 3 JOBCLASS 4.
```

- FILE HANDLE defines the handle for the data file to be read by KEYED DATA LIST. The handle is specified on the FILE subcommand of KEYED DATA LIST.

- KEY on KEYED DATA LIST specifies the variable to be used as the access key. For a direct-access file, the value of the variable must be between 1 and the number of records in the file. For a keyed file, the value must be a string.
- IN creates the logical scratch variable *#FOUND*, whose value will be 1 if the record is successfully read, or 0 if the record is not found.
- The variable definitions are the same as those used for DATA LIST.

Reading a Direct-Access File

```
* Reading a direct-access file: sampling 1 out of every 25 records.

FILE HANDLE    EMPL/ file specifications.
INPUT PROGRAM.
COMPUTE #INTRVL = TRUNC(UNIF(48))+1. /* Mean interval = 25
COMPUTE #NXTCASE = #NXTCASE+#INTRVL. /* Next record number
COMPUTE #EOF = #NXTCASE > 1000.      /* End of file check
DO IF   #EOF.
+  END FILE.
ELSE.
+  KEYED DATA LIST  FILE=EMPL, KEY=#NXTCASE, IN=#FOUND, NOTABLE
                  /YRHIRED 1-2 SEX 3 JOBCLASS 4.
+  DO IF       #FOUND.
+     END CASE.                    /* Return a case
+  ELSE.
+     PRINT / 'Oops. #NXTCASE=' #NXTCASE.
+  END IF.
END IF.
END INPUT PROGRAM.
EXECUTE.
```

- FILE HANDLE defines the handle for the data file to be read by the KEYED DATA LIST command. The record numbers for this example are generated by the transformation language; they are not based on data taken from another file.
- The INPUT PROGRAM and END INPUT PROGRAM commands begin and end the block of commands that build cases from the input file. Since the session generates cases, an input program is required.
- The first two COMPUTE statements determine the number of the next record to be selected. This is done in two steps. First, the integer portion is taken from the sum of 1 and a uniform pseudo-random number between 1 and 49. The result is a mean interval of 25. Second, the variable *#NXTCASE* is added to this number to generate the next record number. This record number, *#NXTCASE*, will be used for the key variable on the KEYED DATA LIST command. The third COMPUTE creates a logical scratch variable, *#EOF*, that has a value of 0 if the record number is less than or equal to 1000, or 1 if the value of the record number is greater than 1000.
- The DO IF—END IF structure controls the building of cases. If the record number is greater than 1000, *#EOF* equals 1, and the END FILE command tells the program to stop reading data and end the file.
- If the record number is less than or equal to 1000, the record is read via KEYED DATA LIST using the value of *#NXTCASE*. A case is generated if the record exists (*#FOUND* equals 1). If not, the program displays the record number and continues to the next case. The sample will have about 40 records.
- EXECUTE causes the transformations to be executed.
- This example illustrates the difference between DATA LIST, which always reads the next record in a file, and KEYED DATA LIST, which reads only specified records. The record numbers must be generated by another command or be contained in the active dataset.

Reading a Keyed File

```
* Reading a keyed file: reading selected records.

GET FILE=STUDENTS/KEEP=AGE,SEX,COURSE.
FILE HANDLE COURSES/ file specifications.
STRING #KEY(A4).
COMPUTE #KEY = STRING(COURSE,N4). /* Create a string key
KEYED DATA LIST FILE=COURSES KEY=#KEY IN=#FOUND NOTABLE
     /PERIOD 13 CREDITS 16.
SELECT IF #FOUND.
LIST.
```

- GET reads the *STUDENTS* file, which contains information on students, including a course identification for each student. The course identification will be used as the key for selecting one record from a file of courses.
- The FILE HANDLE command defines a file handle for the file of courses.

- The STRING and COMPUTE commands transform the course identification from numeric to string for use as a key. For keyed files, the key variable must be a string.
- KEYED DATA LIST uses the value of the newly created string variable *#KEY* as the key to search the course file. If a record that matches the value of *#KEY* is found, *#FOUND* is set to 1; otherwise, it is set to 0. Note that KEYED DATA LIST appears outside an input program in this example.
- If the course file contains the requested record, *#FOUND* equals 1. The variables *PERIOD* and *CREDITS* are added to the case and the case is selected via the SELECT IF command; otherwise, the case is dropped.
- LIST lists the values of the selected cases.
- This example shows how existing cases can be updated on the basis of information read from a keyed file.
- This task could also be accomplished by reading the entire course file with DATA LIST and combining it with the student file via the MATCH FILES command. The technique you should use depends on the percentage of the records in the course file that need to be accessed. If fewer than 10% of the course file records are read, KEYED DATA LIST is probably more efficient. As the percentage of the records that are read increases, reading the entire course file and using MATCH makes more sense.

FILE Subcommand

FILE specifies the handle for the direct-access or keyed data file. The file handle must have been defined on a previous FILE HANDLE command (or, in the case of the IBM OS environment, on a DD statement in the JCL).

KEY Subcommand

KEY specifies the variable whose value will be used as the key. This variable must already exist as the result of a prior DATA LIST, KEYED DATA LIST, GET, or transformation command.

- KEY is required. Its only specification is a single variable. The variable can be a permanent variable or a scratch variable.
- For direct-access files, the key variable must be numeric, and its value must be between 1 and the number of records in the file.
- For keyed files, the key variable must be string. If the keys are numbers, such as social security numbers, the STRING function can be used to convert the numbers to strings. For example, the following might be required to get the value of a numeric key into exactly the same format as used on the keyed file:

```
COMPUTE #KEY=STRING(123,IB4).
```

IN Subcommand

IN creates a numeric variable whose value indicates whether or not the specified record is found.

- IN is required. Its only specification is a single numeric variable. The variable can be a permanent variable or a scratch variable.
- The value of the variable is 1 if the record is successfully read or 0 if the record is not found. The IN variable can be used to select all cases that have been updated by KEYED DATA LIST.

Example

```
FILE HANDLE EMPL/ file specifications.
KEYED DATA LIST FILE=EMPL KEY=#NXTCASE IN=#FOUND
          /YRHIRED 1-2 SEX 3 JOBCLASS 4.
```

- IN creates the logical scratch variable *#FOUND*. The values of *#FOUND* will be 1 if the record indicated by the key value in *#NXTCASE* is found or 0 if the record does not exist.

TABLE and NOTABLE Subcommands

TABLE and NOTABLE determine whether the program displays a table that summarizes the variable definitions. TABLE, the default, displays the table. NOTABLE suppresses the table.

- TABLE and NOTABLE are optional and mutually exclusive.
- The only specification for TABLE or NOTABLE is the subcommand keyword. Neither subcommand has additional specifications.

ENCODING Subcommand

ENCODING specifies the encoding format of the file. The keyword is followed by an equals sign and a quoted encoding specification.

- In Unicode mode, the default is UTF8. For more information, see SET command, UNICODE subcommand.
- In code page mode, the default is the current locale setting. For more information, see SET command, LOCALE subcommand.
- The quoted encoding value can be: Locale (the current locale setting), UTF8, UTF16, UTF16BE (big endian), UTF16LE (little endian), a numeric Windows code page value (for example, '1252'), or an IANA code page value (for example, 'iso8859-1' or cp1252).
- In Unicode mode, the defined width of string variables is tripled for code page and UTF-16 text data files. Use ALTER TYPE to automatically adjust the defined width of string variables.

KM

KM is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
KM varname [BY factor varname]

   /STATUS = varname [EVENT](vallist) [LOST(vallist)]

   [/STRATA = varname]

   [/PLOT = [SURVIVAL**][LOGSURV][HAZARD][OMS]]

   [/ID  = varname]

   [/PRINT = [TABLE**][MEAN**][NONE]]

   [/PERCENTILES = [(]{25, 50, 75 }[)]]
                     {value list }

   [/TEST = [LOGRANK**][BRESLOW][TARONE]]

   [/COMPARE = [{OVERALL**}][{POOLED**}]]
               {PAIRWISE }  {STRATA  }

   [/TREND = [(METRIC)]]

   [/SAVE = tempvar[(newvar)],...]
```

**Default if the subcommand or keyword is omitted.

Temporary variables created by Kaplan-Meier are:

SURVIVAL

HAZARD

SE

CUMEVENT

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
KM LENGTH BY SEXRACE
 /STATUS=EMPLOY  EVENT (1) LOST (2)
 /STRATA=LOCATION.
```

Overview

KM (alias K-M) uses the Kaplan-Meier (product-limit) technique to describe and analyze the length of time to the occurrence of an event, often known as **survival time**. KM is similar to SURVIVAL in that it produces nonparametric estimates of the survival functions. However, instead of dividing the period of time under examination into arbitrary intervals, KM evaluates the survival function at the observed event times. For analysis of survival times with covariates, including time-dependent covariates, see the COXREG command.

Options

KM Tables. You can include one factor variable on the KM command. A KM table is produced for each level of the factor variable. You can also suppress the KM tables in the output with the PRINT subcommand.

Survival Status. You can specify the code(s) indicating that an event has occurred as well as code(s) for cases lost to follow-up using the STATUS subcommand.

Plots. You can plot the survival functions on a linear or log scale or plot the hazard function for each combination of factor and stratum with the PLOT subcommand.

Test Statistics. When a factor variable is specified, you can specify one or more tests of equality of survival distributions for the different levels of the factor using the TEST subcommand. You can also specify a trend metric for the requested tests with the TREND subcommand.

Display ID and Percentiles. You can specify an ID variable on the ID subcommand to identify each case. You can also request the display of percentiles in the output with the PERCENTILES subcommand.

Comparisons. When a factor variable is specified, you can use the COMPARE subcommand to compare the different levels of the factor, either pairwise or across all levels, and either pooled across all strata or within a stratum.

Add New Variables to Active Dataset. You can save new variables appended to the end of the active dataset with the SAVE subcommand.

Basic Specification

- The basic specification requires a survival variable and the STATUS subcommand naming a variable that indicates whether the event occurred.
- The basic specification prints one survival table followed by the mean and median survival time with standard errors and 95% confidence intervals.

Subcommand Order

- The survival variable and the factor variable (if there is one) must be specified first.
- Remaining subcommands can be specified in any order.

Syntax Rules

- Only one survival variable can be specified. To analyze multiple survival variables, use multiple KM commands.
- Only one factor variable can be specified following the BY keyword. If you have multiple factors, use the transformation language to create a single factor variable before invoking KM.
- Only one status variable can be listed on the STATUS subcommand. You must specify the value(s) indicating that the event occurred.
- Only one variable can be specified on the STRATA subcommand. If you have more than one stratum, use the transformation language to create a single variable to specify on the STRATA subcommand.

Operations

- KM deletes all cases that have negative values for the survival variable.
- KM estimates the survival function and associated statistics for each combination of factor and stratum.
- Three statistics can be computed to test the equality of survival functions across factor levels within a stratum or across all factor levels while controlling for strata. The statistics are the log rank (Mantel-Cox), generalized Wilcoxon (Breslow), and Tarone-Ware tests.
- When the PLOTS subcommand is specified, KM produces one plot of survival functions for each stratum, with all factor levels represented by different symbols or colors.

Limitations

- A maximum of 500 factor levels (symbols) can appear in a plot.

Examples

```
KM LENGTH BY SEXRACE
 /STATUS=EMPLOY EVENT (1) LOST (2)
 /STRATA=LOCATION.
```

- Survival analysis is used to examine the length of unemployment. The survival variable *LENGTH* contains the number of months a subject is unemployed. The factor variable *SEXRACE* combines sex and race factors.

- A value of 1 on the variable *EMPLOY* indicates the occurrence of the event (employment). All other observed cases are censored. A value of 2 on *EMPLOY* indicates cases lost to follow-up. Cases with other values for *EMPLOY* are known to have remained unemployed during the course of the study. KM separates the two types of censored cases in the KM table if LOST is specified.
- For each combination of *SEXRACE* and *LOCATION*, one KM table is produced, followed by the mean and median survival times with standard errors and confidence intervals.

Survival and Factor Variables

You must identify the survival and factor variables for the analysis.

- The minimum specification is one, and only one, survival variable.
- Only one factor variable can be specified using the BY keyword. If you have more than one factor, create a new variable combining all factors. There is no limit to the factor levels.

Example

```
DO IF SEX = 1.
+ COMPUTE SEXRACE = RACE.
ELSE.
+ COMPUTE SEXRACE = RACE + SEX.
END IF.
KM LENGTH BY SEXRACE
 /STATUS=EMPLOY EVENT (1) LOST (2).
```

- The two control variables, *SEX* and *RACE*, each with two values, 1 and 2, are combined into one factor variable, *SEXRACE*, with four values, 1 to 4.
- KM specifies *LENGTH* as the survival variable and *SEXRACE* as the factor variable.
- One KM table is produced for each factor level.

STATUS Subcommand

To determine whether the terminal event has occurred for a particular observation, KM checks the value of a status variable. STATUS lists the status variable and the code(s) for the occurrence of the event. The code(s) for cases lost to follow-up can also be specified.

- Only one status variable can be specified. If multiple STATUS subcommands are specified, KM uses the last specification and displays a warning.
- The keyword EVENT is optional, but the value list in parentheses must be specified. Use EVENT for clarity's sake, especially when LOST is specified.
- The value list must be enclosed in parentheses. All cases with non-negative times that do not have a code within the range specified after EVENT are classified as **censored cases**—that is, cases for which the event has not yet occurred.
- The keyword LOST and the following value list are optional. LOST cannot be omitted if the value list for lost cases is specified.
- When LOST is specified, all cases with non-negative times that have a code within the specified value range are classified as lost to follow-up. Cases lost to follow-up are treated as censored in the analysis, and the statistics do not change, but the two types of censored cases are listed separately in the KM table.
- The value lists on EVENT or LOST can be one value, a list of values separated by blanks or commas, a range of values using the keyword THRU, or a combination.
- The status variable can be either numeric or string. If a string variable is specified, the EVENT or LOST values must be enclosed in apostrophes, and the keyword THRU cannot be used.

Example

```
KM LENGTH BY SEXRACE
 /STATUS=EMPLOY  EVENT (1) LOST (3,5 THRU 8).
```

- STATUS specifies that *EMPLOY* is the status variable.
- A value of 1 for *EMPLOY* means that the event (employment) occurred for the case.
- Values of 3 and 5 through 8 for *EMPLOY* mean that contact was lost with the case. The different values code different causes for the loss of contact.
- The summary table in the output includes columns for number lost and percentage lost, as well as for number censored and percentage censored.

STRATA Subcommand

STRATA identifies a **stratification variable**—that is, a variable whose values are used to form subgroups (strata) within the categories of the factor variable. Analysis is done within each level of the strata variable for each factor level, and estimates are pooled over strata for an overall comparison of factor levels.

- The minimum specification is the subcommand keyword with one, and only one, variable name.
- If you have more than one strata variable, create a new variable to combine the levels on separate variables before invoking the KM command.
- There is no limit to the number of levels for the strata variable.

Example

```
KM LENGTH BY SEXRACE
 /STATUS=EMPLOY EVENT (1) LOST (3,5 THRU 8)
 /STRATA=LOCATION.
```

- STRATA specifies *LOCATION* as the stratification variable. Analysis of the length of unemployment is done for each location within each sex and race subgroup.

PLOT Subcommand

PLOT plots the cumulative survival distribution on a linear or logarithmic scale or plots the cumulative hazard function. A separate plot with all factor levels is produced for each stratum. Each factor level is represented by a different symbol or color. Censored cases are indicated by markers.

- When PLOT is omitted, no plots are produced. The default is NONE.
- When PLOT is specified without a keyword, the default is SURVIVAL. A plot of survival functions for each stratum is produced.
- To request specific plots, specify, following the PLOT subcommand, any combination of the keywords defined below.
- Multiple keywords can be used on the PLOT subcommand, each requesting a different plot. The effect is cumulative.

SURVIVAL. *Plot the cumulative survival distribution on a linear scale.* SURVIVAL is the default when PLOT is specified without a keyword.

LOGSURV. *Plot the cumulative survival distribution on a logarithmic scale.*

HAZARD. *Plot the cumulative hazard function.*

OMS. *Plot the one-minus-survival function.*

Example

```
KM LENGTH BY SEXRACE
 /STATUS=EMPLOY EVENT (1) LOST (3,5 THRU 8)
 /STRATA=LOCATION
 /PLOT = SURVIVAL HAZARD.
```

- PLOT produces one plot of the cumulative survival distribution on a linear scale and one plot of the cumulative hazard rate for each value of *LOCATION*.

ID Subcommand

ID specifies a variable used for labeling cases. If the ID variable is a string, KM uses the string values as case identifiers in the KM table. If the ID variable is numeric, KM uses value labels or numeric values if value labels are not defined.

- ID is the first column of the KM table displayed for each combination of factor and stratum.
- If a string value or a value label exceeds 20 bytes in width, KM truncates the case identifier and displays a warning.

PRINT Subcommand

By default, KM prints survival tables and the mean and median survival time with standard errors and confidence intervals if PRINT is omitted. If PRINT is specified, only the specified keyword is in effect. Use PRINT to suppress tables or the mean statistics.

TABLE. *Print the KM tables.* If PRINT is not specified, TABLE, together with MEAN, is the default. Specify TABLE on PRINT to suppress the mean statistics.

MEAN. *Print the mean statistics.* KM prints the mean and median survival time with standard errors and confidence intervals. If PRINT is not specified, MEAN, together with TABLE, is the default. Specify MEAN on PRINT to suppress the KM tables.

NONE. *Suppress both the KM tables and the mean statistics.* Only plots and comparisons are printed.

Example

```
KM LENGTH BY SEXRACE
 /STATUS=EMPLOY EVENT (1) LOST (3,5 THRU 8)
 /STRATA=LOCATION
 /PLOT=SURVIVAL HAZARD
 /PRINT=NONE.
```

- PRINT=NONE suppresses both the KM tables and the mean statistics.

PERCENTILES Subcommand

PERCENTILES displays percentiles for each combination of factor and stratum. Percentiles are not displayed without the PERCENTILES subcommand. If the subcommand is specified without a value list, the default is 25, 50, and 75 for quartile display. You can specify any values between 0 and 100.

TEST Subcommand

TEST specifies the test statistic to use for testing the equality of survival distributions for the different levels of the factor.

- TEST is valid only when a factor variable is specified. If no factor variable is specified, KM issues a warning and TEST is not executed.
- If TEST is specified without a keyword, the default is LOGRANK. If a keyword is specified on TEST, only the specified test is performed.
- Each of the test statistics has a chi-square distribution with one degree of freedom.

LOGRANK. *Perform the log rank (Mantel-Cox) test.*

BRESLOW. *Perform the Breslow (generalized Wilcoxon) test.*

TARONE. *Perform the Tarone-Ware test.*

COMPARE Subcommand

COMPARE compares the survival distributions for the different levels of the factor. Each of the keywords specifies a different method of comparison.

- COMPARE is valid only when a factor variable is specified. If no factor variable is specified, KM issues a warning and COMPARE is not executed.
- COMPARE uses whatever tests are specified on the TEST subcommand. If no TEST subcommand is specified, the log rank test is used.
- If COMPARE is not specified, the default is OVERALL and POOLED. All factor levels are compared across strata in a single test. The test statistics are displayed after the summary table at the end of output.
- Multiple COMPARE subcommands can be specified to request different comparisons.

  OVERALL. *Compare all factor levels in a single test.* OVERALL, together with POOLED, is the default when COMPARE is not specified.

  PAIRWISE. *Compare each pair of factor levels.* KM compares all distinct pairs of factor levels.

  POOLED. *Pool the test statistics across all strata.* The test statistics are displayed after the summary table for all strata. POOLED, together with OVERALL, is the default when COMPARE is not specified.

  STRATA. *Compare the factor levels for each stratum.* The test statistics are displayed for each stratum separately.
- If a factor variable has different levels across strata, you cannot request a pooled comparison. If you specify POOLED on COMPARE, KM displays a warning and ignores the request.

Example

```
KM LENGTH BY SEXRACE
 /STATUS=EMPLOY EVENT (1) LOST (3,5 THRU 8)
 /STRATA=LOCATION
 /TEST = BRESLOW
 /COMPARE = PAIRWISE.
```

- TEST specifies the Breslow test.
- COMPARE uses the Breslow test statistic to compare all distinct pairs of *SEXRACE* values and pools the test results over all strata defined by *LOCATION*.
- Test statistics are displayed at the end of output for all strata.

TREND Subcommand

TREND specifies that there is a trend across factor levels. This information is used when computing the tests for equality of survival functions specified on the TEST subcommand.

- The minimum specification is the subcommand keyword by itself.
- If TREND is specified but COMPARE is not, KM performs the default log rank test with the trend metric for an OVERALL POOLED comparison.
- If the metric specified on TREND is longer than required by the factor levels, KM displays a warning and ignores extra values.

Example

```
KM LENGTH BY SEXRACE
 /STATUS=EMPLOY EVENT (1) LOST (3,5 THRU 8)
 /STRATA=LOCATION
 /TREND.
```

- TREND is specified by itself. KM uses the default metric. Since *SEXRACE* has four levels, the default is (–3, –1, 1, 3).
- Even though no TEST or COMPARE subcommand is specified, KM performs the default log rank test with the trend metric and does a default OVERALL POOLED comparison.

SAVE Subcommand

SAVE saves the temporary variables created by KM. The following temporary variables can be saved:

SURVIVAL. *Survival function evaluated at current case.*

SE. *Standard error of the survival function.*

HAZARD. *Cumulative hazard function evaluated at current case.*

CUMEVENT. *Cumulative number of events.*

- To specify variable names for the new variables, assign the new names in parentheses following each temporary variable name.
- Assigned variable names must be unique in the active dataset. Scratch or system variable names cannot be used (that is, variable names cannot begin with # or $).
- If new variable names are not specified, KM generates default names. The default name is composed of the first three characters of the name of the temporary variable (two for *SE*), followed by an underscore and a number to make it unique.
- A temporary variable can be saved only once on the same SAVE subcommand.

Example

```
KM LENGTH BY SEXRACE
 /STATUS=EMPLOY EVENT (1) LOST (3,5 THRU 8)
 /STRATA=LOCATION
 /SAVE SURVIVAL HAZARD.
```

- KM saves cumulative survival and cumulative hazard rates in two new variables, *SUR_1* and *HAZ_1*, provided that neither name exists in the active dataset. If one does, the numeric suffixes will be incremented to make a distinction.

KNN

KNN is available in the Statistics Base Edition.

```
KNN [dependent variable [(MLEVEL = {S})]]
                                   {O}
                                   {N}
    [BY factor-list] [WITH covariate-list]

[/EXCEPT VARIABLES = varlist]

[/CASELABELS VARIABLE = varname]

[/FOCALCASES VARIABLE = varname]

[/RESCALE [COVARIATE = {ADJNORMALIZED**}]]
                       {NONE           }

[/PARTITION {TRAINING = {70**   } HOLDOUT = {30**   }}]
                        {integer}           {integer}
            {VARIABLE = varname                      }

[/MODEL [METRIC = {EUCLID** }]
                  {CITYBLOCK}

        [NEIGHBORS = {FIXED**} [(K={3**    })                   ]]
                               {integer}
                    {AUTO   } [(KMIN={3      }, KMAX={5      })]
                                     {integer}       {integer}

        [FEATURES = {ALL**}                                     ]
                    {AUTO } [(FORCE = variable [variable ...])]

[/CRITERIA  [NUMFEATURES = {AUTO**                       }]
                           {FIXED(integer)               }
                           {ERRORRATIO(MINCHANGE={0.01 })}
                                                 {value}

        [PREDICTED = {MEAN**}]
                     {MEDIAN}

        [WEIGHTFEATURES = {NO**}]
                          {YES }

[/CROSSVALIDATION {FOLDS = {10**   } }]
                           {integer}
                  {VARIABLE = varname}

[/MISSING USERMISSING = {EXCLUDE**}]
                        {INCLUDE  }

[/VIEWMODEL [DISPLAY = {YES**}]]
                       {NO   }

[/PRINT [CPS**] [NONE]]

[/SAVE [PREDVAL[(varname)]]     ]
       [PREDPROB[(rootname)]]
       [PARTITION[(varname)]]
       [FOLD[(varname)]]
       [MAXCAT({25**   })]
               {integer}

[/OUTFILE [MODEL ='filename']                 ]
          [FOCALCASES = 'savfile' | 'dataset'].
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 17.0

- Command introduced.

Example

```
KNN DepVar BY A B C WITH X Y Z.
```

Overview

Nearest Neighbor Analysis is a method for classifying cases based on their similarity to other cases. In machine learning, it was developed as a way to recognize patterns of data without requiring an exact

match to any stored patterns, or cases. Similar cases are near each other and dissimilar cases are distant from each other. Thus, the distance between two cases is a measure of their dissimilarity.

Cases that are near each other are said to be “neighbors.” When a new case (holdout) is presented, its distance from each of the cases in the model is computed. The classifications of the most similar cases – the nearest neighbors – are tallied and the new case is placed into the category that contains the greatest number of nearest neighbors.

You can specify the number of nearest neighbors to examine; this value is called *k*. The pictures show how a new case would be classified using two different values of *k*. When *k* = 5, the new case is placed in category *1* because a majority of the nearest neighbors belong to category *1*. However, when *k* = 9, the new case is placed in category *0* because a majority of the nearest neighbors belong to category *0*.

Nearest neighbor analysis can also be used to compute values for a continuous target. In this situation, the average or median target value of the nearest neighbors is used to obtain the predicted value for the new case.

Options

Prediction or classification. The dependent variable may be scale, categorical, or a combination. If a dependent variable has scale measurement level, then the model predicts continuous values that approximate the “true” value of some continuous function of the input data. If a dependent variable is categorical, then the model is used to classify cases into the “best” category based on the input predictors.

Rescaling. KNN optionally rescales covariates; that is, predictors with scale measurement level, before training the model. Adjusted normalization is the rescaling method.

Training and holdout partitions. KNN optionally divides the data set into training and holdout partitions. The model is trained using the training partition. The holdout partition is completely excluded from the training process and is used for independent assessment of the final model.

Missing Values. The KNN procedure has an option for treating user-missing values of categorical variables as valid. User-missing values of scale variables are always treated as invalid. The procedure uses listwise deletion; that is, cases with invalid values for any variable are excluded from the model.

Output. KNN displays a case processing summary as pivot table output, and an interactive model view of other output. Tables in the model view include *k* nearest neighbors and distances for focal cases, classification of categorical response variables, and an error summary. Graphical output in the model view includes an automatic selection error log, feature importance chart, feature space chart, peers chart, and quadrant map. The procedure also optionally saves predicted values in the active dataset, PMML to an external file, and distances to focal cases to a new dataset or external file.

Basic Specification

The basic specification is the KNN command followed by zero or one dependent variable, the BY keyword and one or more factors, and the WITH keyword and one or more covariates.

By default, the KNN procedure normalizes covariates and selects a training sample before training the model. The model uses Euclidean distance to select the three nearest neighbors. User-missing values are excluded and default output is displayed.

If there are zero response variables, then the procedure finds the k nearest neighbors only – no classification or prediction is done.

Note: Since measurement level can affect the results, if any variables (fields) have an unknown measurement level, an initial data pass will be performed to determine default measurement level for any variables with an unknown measurement level. For information on the criteria used to determine default measurement level, see SET SCALEMIN.

Syntax Rules

- All subcommands are optional.
- Subcommands may be specified in any order.
- Only a single instance of each subcommand is allowed.

- An error occurs if a keyword is specified more than once within a subcommand.
- Parentheses, equals signs, and slashes shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not allowed.
- Any split variable defined on the SPLIT FILE command may not be used as a dependent variable, factor, covariate, or partition variable.

Limitations

Frequency weights specified on the WEIGHT command are ignored with a warning by the KNN procedure.

Categorical Variables

Although the KNN procedure accepts categorical variables as predictors or dependent variables, the user should be cautious when using a categorical variable with a very large number of categories.

The KNN procedure temporarily recodes categorical predictors using one-of-*c* coding for the duration of the procedure. If there are *c* categories of a variable, then the variable is stored as *c* vectors, with the first category denoted (1,0,...,0), the next category (0,1,0,...,0), ..., and the final category (0,0,...,0,1).

This coding scheme increases the dimensionality of the feature space. In particular, the total number of dimensions is the number of scale predictors plus the number of categories across all categorical predictors. As a result, this coding scheme can lead to slower training. If your nearest neighbors training is proceeding very slowly, you might try reducing the number of categories in your categorical predictors by combining similar categories or dropping cases that have extremely rare categories before running the KNN procedure.

All one-of-*c* coding is based on the training data, even if a holdout sample is defined (see "PARTITION Subcommand " on page 971). Thus, if the holdout sample contains cases with predictor categories that are not present in the training data, then those cases are not scored. If the holdout sample contains cases with dependent variable categories that are not present in the training data, then those cases are scored.

Replicating Results

The KNN procedure uses random number generation during random assignment of partitions and cross-validation folds. To reproduce the same randomized results in the future, use the SET command to set the initialization value for the random number generator before each run of the KNN procedure, or use variables to define partitions and cross-validation folds.

Examples

Basic specification with default settings

```
KNN DepVar BY A B C WITH X Y Z.
```

- The KNN procedure treats *DepVar* as the dependent variable. Predictors *A*, *B*, and *C* are factors, and *X*, *Y*, and *Z* are covariates.
- By default, covariates are normalized before training. Also, the active dataset is partitioned into training and holdout data samples, with 70% going to the training data and 30% to the holdout data sample.
- The model uses Euclidean distance to select the three nearest neighbors.
- User-missing values are excluded and default output is displayed.

Variable Lists

The command line variable lists specify the dependent variable, any categorical predictors (also known as factors), and any scale predictors (also known as covariates).

Dependent Variable

- An optional dependent variable must be the first specification on the KNN command.
- The dependent variable may be followed by the measurement level specification, which contains, in parentheses, the MLEVEL keyword followed by an equals sign and then S for scale, O for ordinal, or N for nominal. KNN treats ordinal and nominal dependent variables equivalently as categorical.
- If a measurement level is specified, then it temporarily overrides a dependent variable's setting in the data dictionary.
- If no measurement level is specified, then KNN defaults to the dictionary setting.
- If a measurement level is not specified and no setting is recorded in the data dictionary, then a numeric variable is treated as scale and a string variable is treated as categorical.
- Dependent variables can be numeric or string.
- A string variable may be defined as ordinal or nominal only.

Predictor Variables

- The names of the factors, if any, must be preceded by the keyword BY.
- If keyword BY is specified with no factors, then a warning is issued and BY is ignored.
- The names of the covariates, if any, must be preceded by the keyword WITH.
- If keyword WITH is specified with no covariates, then a warning is issued and WITH is ignored.
- A dependent variable may not be specified within a factor or covariate list. If a dependent variable is specified within one of these lists, then an error is issued.
- All variables specified within a factor or covariate list must be unique. If duplicate variables are specified within a list, then the duplicates are ignored.
- If duplicate variables are specified across the factor and covariate lists, then an error is issued.
- Factor variables can be numeric or string.
- Covariates must be numeric.
- At least one predictor must be specified.

EXCEPT Subcommand

The EXCEPT subcommand lists any variables that the KNN procedure should exclude from the factor or covariate lists on the command line. This subcommand is useful if the factor or covariate lists contain a large number of variables – specified using the TO or ALL keyword, for example – but there are a few variables (for example, Case ID) that should be excluded.

The EXCEPT subcommand is introduced strictly for the purpose of simplifying syntax. Missing values on factors or covariates specified on EXCEPT do not affect whether a case is included in the analysis. For example, the following two KNN commands are equivalent. In both commands, listwise deletion is based on the dependent variable and factors *A*, *B*, and *C*.

```
KNN DepVar BY A B C.

KNN DepVar BY A B C D
   /EXCEPT VARIABLES=D.
```

- The EXCEPT subcommand ignores duplicate variables, and variables that are not specified on the command line's factor or covariate lists.
- There is no default variable list on the EXCEPT subcommand.

CASELABELS Subcommand

The CASELABELS subcommand allows you to specify a variable containing values used to label cases in the output.

- Cases are labeled using these values in the feature space chart, peers chart, and quadrant map.

- There is no default variable list on the CASELABELS subcommand.

FOCALCASES Subcommand

The FOCALCASES subcommand allows you to mark cases of particular interest. For example, a researcher wants to determine whether the test scores from one school district – the focal case – are comparable to those from similar school districts. He uses nearest neighbor analysis to find the school districts that are most similar with respect to a given set of features. Then he compares the test scores from the focal school district to those from the nearest neighbors.

Focal cases could also be used in clinical studies to select control cases that are similar to clinical cases. Focal cases are displayed in the *k* nearest neighbors and distances table, feature space chart, peers chart, and quadrant map. Information on focal cases is saved to the files specified on the OUTFILE subcommand.

- Cases with a positive value on the specified variable are treated as focal cases. It is invalid to specify a variable with no positive values.
- There is no default variable list on the FOCALCASES subcommand.

RESCALE Subcommand

The RESCALE subcommand is used to rescale covariates.

All rescaling is performed based on the training data, even if a holdout sample is defined (see "PARTITION Subcommand " on page 971). That is, the minimum value or maximum value of a covariate are computed using only the training data. It is important that these covariates have similar distributions across the training and holdout samples. If the data are partitioned by specifying ratios on the PARTITION subcommand, then the KNN procedure attempts to ensure this similarity by random assignment. However, if you use the PARTITION subcommand VARIABLE keyword to assign cases to the training and holdout samples, then we recommend that you confirm the distributions are similar across samples before running the KNN procedure.

COVARIATE Keyword

The COVARIATE keyword specifies the rescaling method to use for covariates specified following WITH on the command line. If no covariates are specified on the command line, then the COVARIATE keyword is ignored.

ADJNORMALIZED. *Adjusted version of subtract the minimum and divide by the range, [2*(x–min)/(max –min)]–1.*

NONE. *No rescaling of covariates.*

PARTITION Subcommand

The PARTITION subcommand specifies the method of partitioning the active dataset into training and holdout samples. The training sample comprises the data records used to train the model. The holdout sample is an independent set of data records used to assess the final model.

- The partition can be defined by specifying the ratio of cases randomly assigned to each sample (training and holdout), or by a variable that assigns each case to the training or holdout sample.
- If the PARTITION subcommand is not specified, then the default partition randomly assigns 70% of the cases to the training sample and 30% to the holdout sample. If you want to specify a different random assignment, then you must specify new values for the TRAINING and HOLDOUT keywords. The value specified on each keyword gives the relative number of cases in the active dataset to assign to each sample. For example, /PARTITION TRAINING = 50 HOLDOUT = 50 is equivalent to /PARTITION TRAINING = 5 HOLDOUT = 5; both subcommands randomly assign 50% of the cases to the training sample and 50% to the holdout sample.

- If you want to be able to reproduce results based on the TRAINING and HOLDOUT keywords later, use the SET command to set the initialization value for the random number generator before running the KNN procedure.
- See the discussion of the relationship between rescaling and partitioning in the RESCALE subcommand section.
- All partitioning is performed after listwise deletion of any cases with invalid data for any variable used by the procedure. See the MISSING subcommand for details about valid and invalid data.
- It is invalid to specify both TRAINING and VARIABLE.

TRAINING Keyword

The TRAINING keyword specifies the relative number of cases in the active dataset to randomly assign to the training sample.

The value must be an integer greater than 0. The default (if the PARTITION subcommand is not specified) is 70.

HOLDOUT Keyword

The HOLDOUT keyword specifies the relative number of cases in the active dataset to randomly assign to the holdout sample.

The value must be an integer greater than or equal to 0. The default (if the PARTITION subcommand is not specified) is 30.

VARIABLE Keyword

The VARIABLE keyword specifies a variable that assigns each case in the active dataset to the training or holdout sample. Cases with a positive value on the variable are assigned to the training sample and cases with a non-positive value to the holdout sample. Cases with a system-missing value are excluded from the analysis. (Any user-missing values for the partition variable are always treated as valid.)

The variable may not be a dependent variable or any variable specified on the command line factor or covariate lists. The variable must be numeric.

MODEL Subcommand

The MODEL subcommand is used to specify the nearest neighbor “model”. By default, the procedure builds a model based on 3 nearest neighbors, using all features specified on the variables list and the Euclidean distance measure of “nearness”.

METRIC Keyword

The METRIC keyword allows you to specify the distance metric used to measure the similarity of cases.

EUCLID. *Euclidean distance.* This is the default specification for METRIC. The distance between two cases, x and y, is the square root of the sum, over all dimensions, of the squared differences between the values for the cases.

CITYBLOCK. *City-block or Manhattan distance.* The distance between two cases is the sum, over all dimensions, of the absolute differences between the values for the cases.

NEIGHBORS Keyword

The NEIGHBORS keyword indicates whether to use automatic selection of the number of nearest neighbors.

If no dependent variable is specified, then any specification other than NEIGHBORS=FIXED is ignored with a warning.

FIXED. *Use a fixed number of neighbors.* This is the default. The FIXED keyword may be followed by parentheses containing the K option, which specifies the number of neighbors. K must be a positive integer. The default value is 3. If NEIGHBORS=FIXED is specified, then any CROSSVALIDATION subcommand specifications are ignored with a warning.

AUTO. *Use automatic selection to determine the "best" number of neighbors.* The AUTO keyword may be follwed by parentheses containing the KMIN and KMAX options, which specify the minimum and maximum number of neighbors, respectively, that automatic number of neighbors selection will consider in determining the "best" number of neighbors. It is invalid to specify only one option; you must specify both or neither. The options may be specified in any order and must be separated by a comma or space character. Both numbers must be integers greater than 0, with KMIN less than KMAX. The defaults are KMIN=3, KMAX=5. If NEIGHBORS=AUTO and FEATURES=ALL are specified, then *V*-fold cross-validation is used to select the "best" number of neighbors. The CROSSVALIDATION subcommand specifies the settings for *V*-fold cross-validation.

FEATURES Keyword

The FEATURES keyword indicates whether to use automatic selection of features (predictors).

If no dependent variable is specified, then any specification other than FEATURES=ALL is ignored with a warning.

ALL. *Use all predictors on the command line variable lists.* This is the default.

AUTO . *Use forward selection to determine the "best" feature set.* The AUTO keyword may be followed by parentheses containing the FORCE option, which specifies the starting set of predictors that must be included in the model. There is no default variable list on the FORCE option.

- If FEATURES=AUTO is specified, then any CROSSVALIDATION subcommand specifications are ignored with a warning.
- It is invalid for the variable list on the FORCE option to include all possible predictors; that is, there must be at least one predictor available for feature selection if FEATURES=AUTO is specified.

Combined Neighbors and Features Selection

When NEIGHBORS=AUTO and FEATURES=AUTO, the following method is used for combined neighbors and features selection:

1. For each *k*, use the forward selection method for feature selection.
2. Select the *k*, and accompanying feature set, with the lowest error rate or the lowest sum-of-squares error.

CRITERIA Subcommand

The CRITERIA subcommand specifies computational and resource settings for the KNN procedure.

NUMFEATURES Keyword

The NUMFEATURES keyword specifies how automatic feature selection should select the number of features.

This keyword is ignored if /MODEL FEATURES=AUTO is not in effect.

AUTO. *Select a fixed number of features, where the number of features is computed from a formula.* The number of features the procedure selects is equal to min(20,*P*) - J_{Forced}, where *P* is the total number of features and J_{Forced} is the number of forced features. This is the default.

FIXED (integer). *Select a fixed number of features, where the number of features is specified in advance.* Specify a positive integer. It must be less than or equal to the number of unique predictors available for feature selection. This will be the number of predictors specified on the KNN command, minus any specified on the EXCEPT subcommand, minus any forced into the model.

ERRORRATIO (MINCHANGE=value). *Select features until the absolute change in the error ratio compared to the previous step is less than the criterion value.* Specify a number greater than 0. The default value is 0.01.

PREDICTED Keyword

The PREDICTED keyword specifies the function used to compute the predicted value of scale response variables.

This keyword is ignored if no dependent variable is specified.

MEAN. *Compute predicted values based upon the mean value of the nearest neighbors.* This is the default.

MEDIAN. *Compute predicted values based upon the median value of the nearest neighbors.*

WEIGHTFEATURES Keyword

The WEIGHTFEATURES keyword specifies whether to weight features by their normalized importance when computing distances.

NO. *Do not weight features by normalized importance.* This is the default.

YES. *Weight features by normalized importance.* Feature importance for a predictor is calculated by the ratio of the error rate or sum-of-squares error of the model with the predictor removed from the model to the error rate or sum-of-squares error for the full model. Normalized importance is calculated by reweighting the feature importance values so that they sum to 1. The specification WEIGHTFEATURES=YES is ignored with a warning if no dependent variable is specified.

CROSSVALIDATION Subcommand

The CROSSVALIDATION subcommand specifies settings for performing *V*-fold cross-validation to determine the "best" number of neighbors.

- *V*-fold cross validation divides the data into *V* folds. Then, for a fixed *k*, it applies nearest neighbor analysis to make predictions on the *v*th fold (using the other *V*–1 folds as the training sample) and evaluates the error. This process is successively applied to all possible choices of *v*. At the end of *V* folds, the computed errors are averaged. The above steps are repeated for various values of *k*. The value achieving the lowest average error is selected as the optimal value for *k*.
- If multiple values of *k* are tied on the lowest average error, then the smallest *k* among those that are tied is selected.
- Cross-validation is not used when /MODEL NEIGHBORS=FIXED or when /MODEL FEATURES=AUTO.
- It is invalid to specify both the FOLDS and VARIABLE keywords on the CROSSVALIDATION subcommand.

FOLDS Keyword

The FOLDS keyword specifies the number of folds that should be used for cross-validation. The procedure randomly assigns cases to folds, numbered from 1 to the number of folds.

- Specify an integer greater than 1. The default is 10.
- For a given training set, the upper limit to the number of folds is the number of cases. If the value of FOLDS is greater than the number of cases in the training partition (and for any split, if SPLIT FILE is in effect), then the number of folds is set to the number of cases in the training partition (for that split).

VARIABLE Keyword

The VARIABLE keyword specifies a variable that assigns each case in the active dataset to a fold from 1 to *V*.

The variable may not be a dependent variable or any variable specified on the command line factor or covariate lists. The variable must be numeric and take values from 1 to *V*. If any values in this range are missing on any splits (if SPLIT FILE is in effect) this will cause an error.

MISSING Subcommand

The MISSING subcommand is used to control whether user-missing values for categorical variables are treated as valid values. By default, user-missing values for categorical variables are treated as invalid.

- The MISSING subcommand defines categorical variables as dependent variables with MLEVEL specification O or N, dependent variables with measurement level set at Ordinal or Nominal in the data dictionary and no MLEVEL specification, and factors. Scale variables are dependent variables with MLEVEL specification S, dependent variables with Scale measurement level and no MLEVEL specification, and covariates.
- User-missing values for scale variables are always treated as invalid.
- System-missing values for any variables are always treated as invalid.

USERMISSING=EXCLUDE. *User-missing values for categorical variables are treated as invalid.* This is the default.

USERMISSING=INCLUDE. *User-missing values for categorical variables are treated as valid values.*

VIEWMODEL Subcommand

The VIEWMODEL subcommand controls options for display of model-related output, including tables and charts. Tables in the model view include k nearest neighbors and distances for focal cases, classification of categorical response variables, and an error summary. Graphical output in the model view includes an automatic selection error log, feature importance chart, feature space chart, peers chart, and quadrant map. See the topic for more information.

DISPLAY Keyword

The DISPLAY keyword specifies whether to display the model view in the output.

YES. *Display the model view.* This is the default.

NO. *Do not display the model view.*

PRINT Subcommand

The PRINT subcommand indicates the tabular output to display outside the Model Viewer. If PRINT is not specified, then the default tables are displayed. If PRINT is specified, then only the requested PRINT output is displayed.

CPS Keyword

The CPS keyword displays the case processing summary table, which summarizes the number of cases included and excluded in the analysis, in total and by training and holdout samples. This table is shown by default.

NONE Keyword

The NONE keyword suppresses all PRINT output. This keyword may not be specified with any other PRINT keywords.

SAVE Subcommand

The SAVE subcommand writes optional temporary variables to the active dataset.

PREDVAL Keyword

This saves the predicted value for a scale dependent variable and the predicted category for a categorical dependent variable.

Specify a unique, valid variable name. The default variable name is *KNN_PredictedValue*.

If there is no dependent variabe, this keyword is ignored with a warning.

PREDPROB Keyword

If the dependent variable is categorical, then this keyword saves the predicted probabilities of the first *n* categories. The value of *n* is specified by the MAXCAT keyword.

- Specify a unique, valid variable name. The default root name is *KNN_Probability*.
- If there is no dependent variable, this keyword is ignored with a warning.
- If the dependent variable is scale, this keyword is ignored with a warning.

PARTITION Keyword

This saves the value of the partition (training or holdout) to which the case was assigned.

- Specify a valid variable name. The default variable name is *KNN_Partition*.
- This keyword is ignored with a warning if you specify a variable on the PARTITION subcommand.

FOLD Keyword

This saves the value of the fold to which the case was assigned.

- Specify a valid variable name. The default variable name name is *KNN_Fold*.
- This keyword is ignored if /MODEL NEIGHBORS=FIXED or /MODEL FEATURES=AUTO.
- This keyword is ignored with a warning if you specify a variable on the CROSSVALIDATION subcommand.

MAXCAT Keyword

The MAXCAT keyword specifies the number of categories for which predicted probabilities should be saved.

- Specify a positive integer in parentheses. The default is 25.
- This keyword is ignored if predicted probabilities are not saved.

OUTFILE Subcommand

The OUTFILE subcommand saves an XML-format file containing the nearest neighbor model. It also saves IBM SPSS Statistics-format data files containing distances from focal cases.

- File names must be specified in full. KNN does not supply extensions.
- The MODEL keyword is not honored if split-file processing is in effect (see the SPLIT FILE command). If this keyword is specified when split-file processing is on, then a warning is displayed.

MODEL = 'file'. *Writes the nearest neighbor mdoel to an XML file.* Specify a valid file name. If the 'file' specification refers to an existing file, then the file is overwritten. The model is written in PMML with extensions.

FOCALCASES = 'savfile' | 'dataset'. *Writes the distances from focal cases to IBM SPSS Statistics-format data files.* Specify a valid file or dataset name. If the specification refers to an existing file or dataset, then the file or dataset is overwritten.

For each focal case, a separate variable is created for each of the focal case's *k* nearest neighbors (from the training set) and the corresponding k nearest distances. The variable names are *KNN_FocalCase_CaseNumber, KNN_NearestNeighbor_CaseNumber_1* through *KNN_NearestNeighbor_CaseNumber_k* and *KNN_NearestNeighbor_Distance_1* through *KNN_NearestNeighbor_CaseNumber_k*.

LEAVE

```
LEAVE varlist
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

Example

```
COMPUTE TSALARY=TSALARY+SALARY.
LEAVE TSALARY.
FORMAT TSALARY (DOLLAR8)/ SALARY (DOLLAR7).
EXECUTE.
```

Overview

Normally, the program reinitializes variables each time it prepares to read a new case. LEAVE suppresses reinitialization and retains the current value of the specified variable or variables when the program reads the next case. It also sets the initial value received by a numeric variable to 0 instead of system-missing. LEAVE is frequently used with COMPUTE to create a variable to store an accumulating sum. LEAVE is also used to spread a variable’s values across multiple cases when VECTOR is used within an input program to restructure a data file.

LEAVE cannot be used with scratch variables. See the topic “Scratch Variables” on page 44 for more information.

Basic Specification

The basic specification is the variable(s) whose values are not to be reinitialized as each new case is read.

Syntax Rules

- Variables named on LEAVE must be new variables that do not already exist in the active dataset prior to the transformation block that defines them, but they must be defined in the transformation block prior to the LEAVE command that specifies them. See the topic “Examples ” on page 977 for more information.
- Variables named on LEAVE cannot be scratch variables (but scratch variables can be used to obtain functionality equivalent to LEAVE). See the topic “Scratch Variables” on page 44 for more information.
- Multiple variables can be named. The keyword TO can be used to refer to a list of consecutive variables.
- String and numeric variables can be specified on the same LEAVE command.

Operations

- Numeric variables named on LEAVE are initialized to 0 for the first case, and string variables are initialized to blanks. These variables are not reinitialized when new cases are read.

Examples

Correct vs. Invalid Specifications for LEAVE

```
DATA LIST LIST /Var1 Var2 Var3.
BEGIN DATA
1 2 3
4 5 6
7 8 9
END DATA.

*this is the correct form.
COMPUTE TotalVar1=TotalVar1+Var1.
LEAVE TotalVar1.

*this will change the value of Var2 but LEAVE will fail,
 generating an error because Var2 already exists.
COMPUTE Var2=Var2+Var2.
```

```
LEAVE Var2.

*this will fail, generating an error because the LEAVE command
 occurs before the command that defines the variable named on LEAVE.
LEAVE TotalVar3.
COMPUTE TotalVar3=TotalVar3+Var3.

LIST.
```

Running Total

```
COMPUTE TSALARY=TSALARY+SALARY.
LEAVE TSALARY.
FORMAT TSALARY (DOLLAR8)/ SALARY (DOLLAR7).
```

- These commands keep a running total of salaries across all cases. *SALARY* is the variable containing the employee's salary, and *TSALARY* is the new variable containing the cumulative salaries for all previous cases.
- For the first case, *TSALARY* is initialized to 0, and *TSALARY* equals *SALARY*. For the rest of the cases, *TSALARY* stores the cumulative totals for *SALARY*.
- LEAVE follows COMPUTE because *TSALARY* must first be defined before it can be specified on LEAVE.
- If LEAVE were not specified for this computation, *TSALARY* would be initialized to system-missing for all cases. *TSALARY* would remain system-missing because its value would be missing for every computation.

Separate Sums for Each Category of a Grouping Variable

```
SORT CASES DEPT.
IF DEPT NE LAG(DEPT,1) TSALARY=0.   /*Initialize for new dept
COMPUTE TSALARY=TSALARY+SALARY.     /*Sum salaries
LEAVE TSALARY.                    /*Prevent  initialization  each case
FORMAT TSALARY (DOLLAR8)/ SALARY (DOLLAR7).
```

- These commands accumulate a sum across cases for each department.
- SORT first sorts cases by the values of variable *DEPT*.
- IF specifies that if the value of *DEPT* for the current case is not equal to the value of *DEPT* for the previous case, *TSALARY* equals 0. Thus, *TSALARY* is reset to 0 each time the value of *DEPT* changes. (For the first case in the file, the logical expression on IF is missing. However, the desired effect is obtained because LEAVE initializes *TSALARY* to 0 for the first case, independent of the IF statement.)
- LEAVE prevents *TSALARY* from being initialized for cases within the same department.

LINEAR

```
LINEAR
  /FIELDS
    TARGET = field
    INPUTS = fieldList
    [ANALYSIS_WEIGHT = field]

  [/BUILD_OPTIONS              ]
    [OBJECTIVE = {STANDARD**}]
                 {BOOSTING  }
                 {BAGGING   }
                 {LARGE     }
    [USE_AUTO_DATA_PREPARATION = {TRUE**}]
                                 {FALSE }
    [CONFIDENCE_LEVEL = {95**  }]
                        {number}
    [MODEL_SELECTION = {FORWARDSTEPWISE**}]
                       {BESTSUBSETS      }
                       {NONE             }
    [CRITERIA_FORWARD_STEPWISE = {AICC**          }]
                                 {F               }
                                 {ADJUSTEDRSQUARED}
                                 {ASE             }
    [PROBABILITY_ENTRY = {0.05**}]
                         {number}
    [PROBABILITY_REMOVAL = {0.1** }]
                           {number}
    [MAX_EFFECTS = {number}]
    [MAX_STEPS = {number}]
    [CRITERIA_BEST_SUBSETS = {AICC**          }]
                             {ADJUSTEDRSQUARED}
                             {ASE             }
    [REPLICATE_RESULTS = {TRUE**}]
                         {FALSE }
    [SEED = {54752075**}]
            {number    }

  [/ENSEMBLES]
    [COMBINING_RULE_CONTINUOUS = {MEAN**}]
                                 {MEDIAN}
    [COMPONENT_MODELS_N = {10**   }]
                          {integer}

  [/OUTFILE MODEL = 'file']

  [/SAVE PREDICTED_VALUES(varname)].
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 19

- Command introduced.

Example

```
LINEAR
  /FIELDS
    TARGET=y1
    INPUTS=x1 x2.
```

Overview

Linear models predict a continuous target based on linear relationships between the target and one or more predictors.

Linear models are relatively simple and give an easily interpreted mathematical formula for scoring. The properties of these models are well understood and are typically very fast to train.

Options

Ensemble models. In addition to creating a "standard" linear model, you can use ensembles to improve model accuracy (boosting), improve model stability (bagging), or create a model for very large datasets.

Model selection. All fields can be entered into the model together, or forward stepwise or best subsets model selection can be used to choose the best fields.

Output. The procedure produces a model viewer object containing tables and charts for assessing the quality of the model; also you can save predicted values to the active dataset and the model to a file in PMML format.

Basic Specification

The basic specification is the `LINEAR` command and `FIELDS` subcommand with `TARGET` and `INPUTS` keywords. There must be a single continuous target and at least one input.

Note: Since measurement level can affect the results, if any variables (fields) have an unknown measurement level, an initial data pass will be performed to determine default measurement level for any variables with an unknown measurement level. For information on the criteria used to determine default measurement level, see SET SCALEMIN.

Syntax Rules

- All subcommands are optional.
- Subcommands may be specified in any order.
- Only a single instance of each subcommand is allowed.
- An error occurs if a keyword is specified more than once within a subcommand.
- Parentheses, equals signs, and slashes shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not allowed.
- Any split field defined on the `SPLIT FILE` command cannot be used on this command.
- Any field specifed on the `WEIGHT` command cannot be used on this command.

Limitations

- If a categorical field has more than 1000 categories, the procedure does not run and no model is built.

FIELDS Subcommand

The `FIELDS` subcommand specifies the target, predictors, and optional analysis (regression) weight.

TARGET. The target field is what the model tries to predict. Specify a single continuous field.

INPUTS. The model uses values of the input fields to predict the target. Specify one or more fields. Continuous inputs are treated as covariates in the model, while categorical inputs are treated as factors.

- The universal keyword `TO` may be specified on the `INPUTS` list.

ANALYSIS_WEIGHT. The model uses analysis (regression) weights to account for differences in variability across levels of the target field. Specify a single continuous field.

- This subcommand, and the `TARGET` and `INPUTS` keywords, are required.

BUILD_OPTIONS Subcommand

The `BUILD_OPTIONS` subcommand specifies the criteria used to build the model.

OBJECTIVE. The objective determines which of the following model types is built.

- **STANDARD.** *Create a standard model.* The method builds a single model to predict the target using the predictors. Generally speaking, standard models are easier to interpret and can be faster to score than boosted, bagged, or large dataset ensembles.

- **BOOSTING.** *Enhance model accuracy (boosting).* The method builds an ensemble model using boosting, which generates a sequence of models to obtain more accurate predictions. Ensembles can take longer to build and to score than a standard model.
- **BAGGING.** *Enhance model stability (bagging).* The method builds an ensemble model using bagging (bootstrap aggregating), which generates multiple models to obtain more reliable predictions. Ensembles can take longer to build and to score than a standard model.
- **LARGE.** *Create a model for very large datasets.* The method builds an ensemble model by splitting the dataset into separate data blocks. Choose this option if your dataset is too large to build any of the models above, or for incremental model building. This option can take less time to build, but can take longer to score than a standard model.

USE_AUTO_DATA_PREPARATION = TRUE | FALSE. This option allows the procedure to transform the target and predictors in order to maximize the predictive power of the model. The original versions of transformed fields are excluded from the model. By default, the following automatic data preparation actions are performed. Note that this option is ignored and no automatic data preparation is performed if `OBJECTIVE=LARGE`.

- **Date and Time handling.** Each date predictor is transformed into new a continuous predictor containing the elapsed time since a reference date (1970-01-01). Each time predictor is transformed into a new continuous predictor containing the time elapsed since a reference time (00:00:00).
- **Adjust measurement level.** Continuous predictors with less than 5 distinct values are recast as ordinal predictors. Ordinal predictors with greater than 10 distinct values are recast as continuous predictors.
- **Outlier handling.** Values of continuous predictors that lie beyond a cutoff value (3 standard deviations from the mean) are set to the cutoff value.
- **Missing value handling.** Missing values of nominal predictors are replaced with the mode. Missing values of ordinal predictors are replaced with the median. Missing values of continuous predictors are replaced with the mean.
- **Supervised merging.** This makes a more parsimonious model by reducing the number of fields to be processed in association with the target. Similar categories are identified based upon the relationship between the input and the target. Categories that are not significantly different (that is, having a p-value greater than 0.1) are merged. If all categories are merged into one, the original and derived versions of the field are excluded from the model because they have no value as a predictor.

CONFIDENCE_LEVEL. This is the level of confidence used to compute interval estimates of the model coefficients in the Coefficients view. Specify a value greater than 0 and less than 100. The default is 95.

MODEL_SELECTION. Model selection methods determines how predictors are entered into the model.

- **FORWARDSTEPWISE.** This starts with no effects in the model and adds and removes effects one step at a time until no more can be added or removed according to the stepwise criteria. This is the default.
- **BESTSUBSETS.** This checks "all possible" models, or at least a larger subset of the possible models than forward stepwise, to choose the best according to the best subsets criterion. The model with the greatest value of the criterion is chosen as the best model. Note that Best subsets selection is more computationally intensive than forward stepwise selection. When best subsets is performed in conjunction with boosting, bagging, or very large datasets, it can take considerably longer to build than a standard model built using forward stepwise selection.
- **NONE.** Enters all available predictors into the model.

CRITERIA_FORWARD_STEPWISE. This is the statistic used to determine whether an effect should be added to or removed from the model when forward stepwise selection is used. If `MODEL_SELECTION = FORWARDSTEPWISE` is not specified, this keyword is ignored.

- **AICC.** Information Criterion (AICC) is based on the likelihood of the data given the model, and is adjusted to penalize overly complex models.
- **F.** The *F* statistic criterion is based on a statistical test of the improvement in model error.
- **ADJUSTEDRSQUARED.** Adjusted R-squared is based on the fit of the data, and is adjusted to penalize overly complex models.

- **ASE.** The average squared error (ASE) is an overfit prevention criterion based on the fit of the overfit prevention set. The overfit prevention set is a random subsample of approximately 30% of the original dataset that is not used to train the model.

If any criterion other than the *F* statistic is chosen, then at each step the effect that corresponds to the optimal change in the criterion is added to the model (the greatest increase for Adjusted R-squared, decrease for AICC and ASE). Any effects in the model that correspond to a decrease (increase for AICC and ASE) in the criterion are removed.

If the *F* statistic is chosen as the criterion, then at each step the effect that has the smallest p-value less than the specified PROBABILITY_ENTRY threshold is added to the model. Any effects in the model with a p-value greater than the specified PROBABILITY_REMOVAL threshold are removed.

PROBABILITY_ENTRY. When forward stepwise selection is used with F Statistics as the criterion, this is the threshold for entering effects into the model. Specify a number greater than 0 and less than 1. The default is 0.05.

- The value of PROBABILITY_ENTRY must be less than the value of PROBABILITY_REMOVAL.

PROBABILITY_REMOVAL When forward stepwise selection is used with F Statistics as the criterion, this is the threshold for removing effects from the model. Specify a number greater than 0 and less than 1. The default is 0.10.

MAX_EFFECTS = number. Customize maximum number of effects in the final model when forward selection is used. By default, all available effects can be entered into the model. Alternatively, if the stepwise algorithm ends a step with the specified maximum number of effects, the algorithm stops with the current set of effects.

MAX_STEPS = number. Customize the maximum number of steps when forward selection is used. The stepwise algorithm stops after a certain number of steps. By default, this is 3 times the number of available effects. Alternatively, specify a positive integer maximum number of steps.

CRITERIA_BEST_SUBSETS. This is the statistic used to choose the "best" model when best subsets selection is used. If MODEL_SELECTION = FORWARDSTEPWISE is not specified, this keyword is ignored.

- **AICC.** Information Criterion (AICC) is based on the likelihood of the data given the model, and is adjusted to penalize overly complex models.
- **ADJUSTEDRSQUARED.** Adjusted R-squared is based on the fit of the data, and is adjusted to penalize overly complex models.
- **ASE.** Overfit Prevention Criterion (ASE) is based on the fit of the overfit prevention set. The overfit prevention set is a random subsample of approximately 30% of the original dataset that is not used to train the model.

REPLICATE_RESULTS = TRUE|FALSE.** Setting a random seed allows you to replicate analyses. The random number generator is used to choose which records are in the overfit prevention set.

SEED = 54752075 | number.** When REPLICATE_RESULTS=TRUE, this is the value of the random seed. Specify an integer. The default is 54752075.

ENSEMBLES Subcommand

The ENSEMBLES subcommand determines the behavior of ensembling that occurs when boosting, bagging, or very large datasets are requested on the OBJECTIVE keyword. Options that do not apply to the selected objective are ignored.

COMBINING_RULE_CONTINUOUS = MEAN | MEDIAN.** Ensemble predicted values for continuous targets can be combined using the mean or median of the predicted values from the base models.

Note that when the objective is to enhance model accuracy (BAGGING), the combining rule selections are ignored. Boosting always uses a weighted majority vote to score categorical targets and a weighted median to score continuous targets.

COMPONENT_MODELS_N = 10 | integer.** Specify the number of base models to build when the objective is to enhance model accuracy or stability; for bagging, this is the number of bootstrap samples. It should be a positive integer.

SAVE Subcommand

The SAVE subcommand writes optional temporary variables to the active dataset.

PREDICTED_VALUES(varname). *Predicted value of the target.* The default variable name is *PredictedValue*.

OUTFILE Subcommand

The OUTFILE subcommand saves a file containing the scoring model.

MODEL = 'file'. Writes the model to an external *.zip* file. You can use this model file to apply the model information to other data files for scoring purposes. See the topic "Scoring expressions" on page 86 for more information. Specify a unique, valid filename. If the file specification refers to an existing file, then the file is overwritten.

LIST

```
LIST [VARIABLES={ALL**  }] [/FORMAT=[{WRAP**}] [{UNNUMBERED**}]]
                {varlist}           {SINGLE}   {NUMBERED    }

 [/CASES=[FROM {1**}] [TO {eof**}] [BY {1**}]]
              {n  }      {n    }      {n  }
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
LIST VARIABLES=V1 V2.
```

Overview

LIST displays case values for variables in the active dataset. The output is similar to the output produced by the PRINT command. However, LIST is a procedure and reads data, whereas PRINT is a transformation and requires a procedure (or the EXECUTE command) to execute it.

Options

Selecting and Ordering Variables. You can specify a list of variables to be listed using the VARIABLES subcommand.

Format. You can limit each case listing to a single line, and you can display the case number for each listed case with the FORMAT subcommand.

Selecting Cases. You can limit the listing to a particular sequence of cases using the CASES subcommand.

Basic Specification

- The basic specification is simply LIST, which displays the values for all variables in the active dataset.
- By default, cases wrap to multiple lines if all the values do not fit within the page width (the page width is determined by the SET WIDTH command). Case numbers are not displayed for the listed cases.

Subcommand Order

All subcommands are optional and can be named in any order.

Operations

- If VARIABLES is not specified, variables are listed in the order in which they appear in the active dataset.
- LIST does not display values for scratch or system variables.
- LIST uses print formats contained in the dictionary of the active dataset. Alternative formats cannot be specified on LIST. See FORMATS or PRINT FORMATS for information on changing print formats.
- LIST output uses the width specified on SET.
- If a numeric value is longer than its defined width, the program first attempts to list the value by removing punctuation characters, then uses scientific notation, and finally prints asterisks.
- If a long string variable cannot be listed within the output width, it is truncated.
- Values of the variables listed for a case are always separated by at least one blank.
- System-missing values are displayed as a period for numeric variables and a blank for string variables.

- If cases fit on one line, the column width for each variable is determined by the length of the variable name or the format, whichever is greater. If the variable names do not fit on one line, they are printed vertically.
- If cases do not fit on one line within the output width specified on SET, they are wrapped. LIST displays a table illustrating the location of the variables in the output and prints the name of the first variable in each line at the beginning of the line.
- Each execution of LIST begins at the top of a new page. If SPLIT FILE is in effect, each split also begins at the top of a new page.

Examples

LIST with No Subcommands

```
LIST.
```

- LIST by itself requests a display of the values for all variables in the active dataset.

Controlling Listed Cases with CASES Subcommand

```
LIST VARIABLES=V1 V2 /CASES=FROM 10 TO 100 BY 2.
```

- LIST produces a list of every second case for variables *V1* and *V2*, starting with case 10 and stopping at case 100.

VARIABLES Subcommand

VARIABLES specifies the variables to be listed.

- The variables must already exist, and they cannot be scratch or system variables.
- If VARIABLES is used, only the specified variables are listed.
- Variables are listed in the order in which they are named on VARIABLES.
- If a variable is named more than once, it is listed more than once.
- The keyword ALL (the default) can be used to request all variables. ALL can also be used with a variable list (see example below).

ALL. *List all user-defined variables.* Variables are listed in the order in which they appear in the active dataset. This is the default if VARIABLES is omitted.

Example

```
LIST VARIABLES=V15 V31 ALL.
```

- VARIABLES is used to list values for *V15* and *V31* before all other variables. The keyword ALL then lists all variables, including *V15* and *V31*, in the order in which they appear in the active dataset. Values for *V15* and *V31* are therefore listed twice.

FORMAT Subcommand

FORMAT controls whether cases wrap if they cannot fit on a single line and whether the case number is displayed for each listed case. The default display uses more than one line per case (if necessary) and does not number cases.

- The minimum specification is a single keyword.
- WRAP and SINGLE are alternatives, as are NUMBERED and UNNUMBERED. Only one of each pair can be specified.

- If SPLIT FILE is in effect for NUMBERED, case numbering restarts at each split. To get sequential numbering regardless of splits, create a variable and set it equal to the system variable *$CASENUM* and then name this variable as the first variable on the VARIABLES subcommand. An appropriate format should be specified for the new variable before it is used on LIST.

WRAP. *Wrap cases if they do not fit on a single line.* Page width is determined by the SET WIDTH command. This is the default.

SINGLE. *Limit each case to one line.* Only variables that fit on a single line are displayed.

UNNUMBERED. *Do not include the sequence number of each case.* This is the default.

NUMBERED. *Include the sequence number of each case.* The sequence number is displayed to the left of the listed values.

CASES Subcommand

CASES limits the number of cases listed. By default, all cases in the active dataset are listed.

- Any or all of the keywords below can be used. Defaults that are not changed remain in effect.
- If LIST is preceded by a SAMPLE or SELECT IF command, case selections specified by CASES are taken from those cases that were selected by SAMPLE or SELECT IF.
- If SPLIT FILE is in effect, case selections specified by CASES are restarted for each split.

FROM n. *Number of the first case to be listed.* The default is 1.

TO n. *Number of the last case to be listed.* The default is the end of the active dataset. CASES 100 is interpreted as CASES TO 100.

BY n. *Increment used to choose cases for listing.* The default is 1.

Example

```
LIST CASES BY 3 /FORMAT=NUMBERED.
```

- Every third case is listed for all variables in the active dataset. The listing begins with the first case and includes every third case up to the end of the file.
- FORMAT displays the case number of each listed case.

Example

```
LIST CASES FROM 10 TO 20.
```

- Cases from case 10 through case 20 are listed for all variables in the active dataset.

LOGISTIC REGRESSION

LOGISTIC REGRESSION is available in SPSS Statistics Standard Edition or the Regression Option.

```
LOGISTIC REGRESSION VARIABLES = dependent var
       [WITH independent varlist [BY var [BY var] ... ]]

[/CATEGORICAL = var1, var2, ... ]

[/CONTRAST (categorical var) = [{INDICATOR [(refcat)]    }]]
                                {DEVIATION [(refcat)]    }
                                {SIMPLE [(refcat)]       }
                                {DIFFERENCE              }
                                {HELMERT                 }
                                {REPEATED                }
                                {POLYNOMIAL[({1,2,3...})]}
                                           {metric  }
                                {SPECIAL (matrix)        }

 [/METHOD = {ENTER**       }   [{ALL    }]]
            {BSTEP [{COND}]}    {varlist}
                    {LR  }
                    {WALD}
            {FSTEP [{COND}]}
                    {LR  }
                    {WALD}

 [/SELECT = {ALL**                 }]
            {varname relation value}

 [/{NOORIGIN**}]
   {ORIGIN    }

 [/ID = [variable]]

 [/PRINT = [DEFAULT**] [SUMMARY] [CORR] [ALL] [ITER [({1})]] [GOODFIT]]
                                                      {n}
           [CI(level)]

 [/CRITERIA = [BCON ({0.001**})] [ITERATE({20**})] [LCON({0**   })]
                     {value  }            {n   }         {value }

              [PIN({0.05**})] [POUT({0.10**})] [EPS({.00000001**})]]
                   {value }         {value }        {value      }

              [CUT[{0.5**  }]]
                   [value  }

 [/CLASSPLOT]

 [/MISSING = {EXCLUDE **}]
             {INCLUDE   }

 [/CASEWISE = [tempvarlist]  [OUTLIER({2    })]]
                                      {value}

 [/SAVE = tempvar[(newname)] tempvar[(newname)]...]


 [/OUTFILE = [{MODEL    }(filename)]]
              {PARAMETER}

 [/EXTERNAL]
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 13.0

- OUTFILE subcommand introduced.

Release 14.0

- Modification to the method of recoding string variables. See the topic “Overview ” on page 990 for more information.

Example

```
LOGISTIC REGRESSION VARIABLES = PROMOTED WITH AGE, JOBTIME, JOBRATE.
```

Overview

LOGISTIC REGRESSION regresses a dichotomous dependent variable on a set of independent variables. Categorical independent variables are replaced by sets of contrast variables, each set entering and leaving the model in a single step.

Options

Processing of Independent Variables. You can specify which independent variables are categorical in nature on the CATEGORICAL subcommand. You can control treatment of categorical independent variables by the CONTRAST subcommand. Seven methods are available for entering independent variables into the model. You can specify any one of them on the METHOD subcommand. You can also use the keyword BY between variable names to enter interaction terms.

Selecting Cases. You can use the SELECT subcommand to define subsets of cases to be used in estimating a model.

Regression through the Origin. You can use the ORIGIN subcommand to exclude a constant term from a model.

Specifying Termination and Model-Building Criteria. You can further control computations when building the model by specifying criteria on the CRITERIA subcommand.

Adding New Variables to the Active Dataset. You can save the residuals, predicted values, and diagnostics that are generated by LOGISTIC REGRESSION in the active dataset.

Output. You can use the PRINT subcommand to print optional output, use the CASEWISE subcommand to request analysis of residuals, and use the ID subcommand to specify a variable whose values or value labels identify cases in output. You can request plots of the actual values and predicted values for each case with the CLASSPLOT subcommand.

Basic Specification

- The minimum specification is the VARIABLES subcommand with one dichotomous dependent variable. You must specify a list of independent variables either following the keyword WITH on the VARIABLES subcommand or on a METHOD subcommand.
- The default output includes goodness-of-fit tests for the model (–2 log-likelihood, goodness-of-fit statistic, Cox and Snell R^2, and Nagelkerke R^2) and a classification table for the predicted and observed group memberships. The regression coefficient, standard error of the regression coefficient, Wald statistic and its significance level, and a multiple correlation coefficient adjusted for the number of parameters (Atkinson, 1980) are displayed for each variable in the equation.

Subcommand Order

- Subcommands can be named in any order. If the VARIABLES subcommand is not specified first, a slash (/) must precede it.
- The ordering of METHOD subcommands determines the order in which models are estimated. Different sequences may result in different models.

Syntax Rules

- Only one dependent variable can be specified for each LOGISTIC REGRESSION.
- Any number of independent variables may be listed. The dependent variable may not appear on this list.
- The independent variable list is required if any of the METHOD subcommands are used without a variable list or if the METHOD subcommand is not used. The keyword TO cannot be used on any variable list.
- If you specify the keyword WITH on the VARIABLES subcommand, all independent variables must be listed.
- If the keyword WITH is used on the VARIABLES subcommand, interaction terms do not have to be specified on the variable list, but the individual variables that make up the interactions must be listed.
- Multiple METHOD subcommands are allowed.

- The minimum truncation for this command is LOGI REG.

Operations

- Independent variables that are specified on the CATEGORICAL subcommand are replaced by sets of contrast variables. In stepwise analyses, the set of contrast variables associated with a categorical variable is entered or removed from the model as a single step.
- Independent variables are screened to detect and eliminate redundancies.
- If the linearly dependent variable is one of a set of contrast variables, the set will be reduced by the redundant variable or variables. A warning will be issued, and the reduced set will be used.
- For the forward stepwise method, redundancy checking is done when a variable is to be entered into the model.
- When backward stepwise or direct-entry methods are requested, all variables for each METHOD subcommand are checked for redundancy before that analysis begins.

Compatibility

Prior to version 14.0, the order of recoded string values was dependent on the order of values in the data file. For example, when recoding the dependent variable, the first string value encountered was recoded to 0, and the second string value encountered was recoded to 1. Beginning with version 14.0, the procedure recodes string variables so that the order of recoded values is the alphanumeric order of the string values. Thus, the procedure may recode string variables differently than in previous versions.

Limitations

- The dependent variable must be dichotomous for each split-file group. Specifying a dependent variable with more or less than two nonmissing values per split-file group will result in an error.

VARIABLES Subcommand

VARIABLES specifies the dependent variable and, optionally, all independent variables in the model. The dependent variable appears first on the list and is separated from the independent variables by the keyword WITH.

- One VARIABLES subcommand is allowed for each Logistic Regression procedure.
- The dependent variable must be dichotomous—that is, it must have exactly two values other than system-missing and user-missing values for each split-file group.
- The dependent variable may be a string variable if its two values can be differentiated by their first eight characters.
- You can indicate an interaction term on the variable list by using the keyword BY to separate the individual variables.
- If all METHOD subcommands are accompanied by independent variable lists, the keyword WITH and the list of independent variables may be omitted.
- If the keyword WITH is used, *all* independent variables must be specified. For interaction terms, only the individual variable names that make up the interaction (for example, X1, X2) need to be specified. Specifying the actual interaction term (for example, X1 BY X2) on the VARIABLES subcommand is optional if you specify it on a METHOD subcommand.

Example

```
LOGISTIC REGRESSION VARIABLES = PROMOTED WITH AGE,JOBTIME,JOBRATE,
    AGE BY JOBTIME.
```

- *PROMOTED* is specified as the dependent variable.
- *AGE*, *JOBTIME*, *JOBRATE*, and the interaction *AGE* by *JOBTIME* are specified as the independent variables.
- Because no METHOD is specified, all three single independent variables and the interaction term are entered into the model.

- LOGISTIC REGRESSION produces the default output.

CATEGORICAL Subcommand

CATEGORICAL identifies independent variables that are nominal or ordinal. Variables that are declared to be categorical are automatically transformed to a set of contrast variables as specified on the CONTRAST subcommand. If a variable that is coded as 0 – 1 is declared as categorical, its coding scheme is given indicator contrasts by default.

- Independent variables that are not specified on CATEGORICAL are assumed to be at least interval level, except for string variables.
- Any variable that is specified on CATEGORICAL is ignored if it does not appear either after WITH on the VARIABLES subcommand or on any METHOD subcommand.
- Variables that are specified on CATEGORICAL are replaced by sets of contrast variables. If the categorical variable has *n* distinct values, there will be *n*–1 contrast variables generated. The set of contrast variables associated with a categorical variable is entered or removed from the model as a step.
- If any one of the variables in an interaction term is specified on CATEGORICAL, the interaction term is replaced by contrast variables.
- All string variables are categorical. Only the first eight characters of each value of a string variable are used in distinguishing between values. Thus, if two values of a string variable are identical for the first eight characters, the values are treated as though they were the same.

Example

```
LOGISTIC REGRESSION VARIABLES = PASS WITH GPA, GRE, MAT, CLASS, TEACHER
 /CATEGORICAL = CLASS,TEACHER.
```

- The dichotomous dependent variable *PASS* is regressed on the interval-level independent variables *GPA*, *GRE*, and *MAT* and the categorical variables *CLASS* and *TEACHER*.

CONTRAST Subcommand

CONTRAST specifies the type of contrast that is used for categorical independent variables. The interpretation of the regression coefficients for categorical variables depends on the contrasts that are used. The default is INDICATOR. The categorical independent variable is specified in parentheses following CONTRAST. The closing parenthesis is followed by one of the contrast-type keywords.

- If the categorical variable has *n* values, there will be *n*–1 rows in the contrast matrix. Each contrast matrix is treated as a set of independent variables in the analysis.
- Only one categorical independent variable can be specified per CONTRAST subcommand, but multiple CONTRAST subcommands can be specified.

The following contrast types are available [35], [36].

INDICATOR(refcat). *Indicator variables.* Contrasts indicate the presence or absence of category membership. By default, refcat is the last category (represented in the contrast matrix as a row of zeros). To omit a category (other than the last category), specify the sequence number of the omitted category (which is not necessarily the same as its value) in parentheses after the keyword INDICATOR.

DEVIATION(refcat). *Deviations from the overall effect.* The effect for each category of the independent variable (except one category) is compared to the overall effect. Refcat is the category for which parameter estimates are not displayed (they must be calculated from the others). By default, refcat is the last category. To omit a category (other than the last category), specify the sequence number of the

[35] Finn, J. D. 1974. *A general model for multivariate analysis*. New York: Holt, Rinehart and Winston.
[36] Kirk, R. E. 1982. *Experimental design*, 2nd ed. Monterey, California: Brooks/Cole.

omitted category (which is not necessarily the same as its value) in parentheses after the keyword DEVIATION.

SIMPLE(refcat). *Each category of the independent variable (except the last category) is compared to the last category.* To use a category other than the last as the omitted reference category, specify its sequence number (which is not necessarily the same as its value) in parentheses following the keyword SIMPLE.

DIFFERENCE. *Difference or reverse Helmert contrasts.* The effects for each category of the independent variable (except the first category) are compared to the mean effects of the previous categories.

HELMERT. *Helmert contrasts.* The effects for each category of the independent variable (except the last category) are compared to the mean effects of subsequent categories.

POLYNOMIAL(metric). *Polynomial contrasts.* The first degree of freedom contains the linear effect across the categories of the independent variable, the second degree of freedom contains the quadratic effect, and so on. By default, the categories are assumed to be equally spaced; unequal spacing can be specified by entering a metric consisting of one integer for each category of the independent variable in parentheses after the keyword POLYNOMIAL. For example, CONTRAST(STIMULUS)=POLYNOMIAL(1,2,4) indicates that the three levels of STIMULUS are actually in the proportion 1:2:4. The default metric is always (1,2, ..., *k*), where *k* categories are involved. Only the relative differences between the terms of the metric matter: (1,2,4) is the same metric as (2,3,5) or (20,30,50) because the difference between the second and third numbers is twice the difference between the first and second numbers in each instance.

REPEATED. *Comparison of adjacent categories.* Each category of the independent variable (except the last category) is compared to the next category.

SPECIAL(matrix). *A user-defined contrast.* After this keyword, a matrix is entered in parentheses with *k* –1 rows and *k* columns (where *k* is the number of categories of the independent variable). The rows of the contrast matrix contain the special contrasts indicating the desired comparisons between categories. If the special contrasts are linear combinations of each other, LOGISTIC REGRESSION reports the linear dependency and stops processing. If *k* rows are entered, the first row is discarded and only the last *k*–1 rows are used as the contrast matrix in the analysis.

Example

```
LOGISTIC REGRESSION VARIABLES = PASS WITH GRE, CLASS
 /CATEGORICAL = CLASS
 /CONTRAST(CLASS)=HELMERT.
```

- A logistic regression analysis of the dependent variable *PASS* is performed on the interval independent variable *GRE* and the categorical independent variable *CLASS*.
- *PASS* is a dichotomous variable representing course pass/fail status and *CLASS* identifies whether a student is in one of three classrooms. A HELMERT contrast is requested.

Example

```
LOGISTIC REGRESSION VARIABLES = PASS WITH GRE, CLASS
 /CATEGORICAL = CLASS
 /CONTRAST(CLASS)=SPECIAL(2 -1 -1
                          0  1 -1).
```

- In this example, the contrasts are specified with the keyword SPECIAL.

METHOD Subcommand

METHOD indicates how the independent variables enter the model. The specification is the METHOD subcommand followed by a single method keyword. The keyword METHOD can be omitted. Optionally, specify the independent variables and interactions for which the method is to be used. Use the keyword BY between variable names of an interaction term.

- If no variable list is specified, or if the keyword ALL is used, all of the independent variables following the keyword WITH on the VARIABLES subcommand are eligible for inclusion in the model.

- If no METHOD subcommand is specified, the default method is ENTER.
- Variables that are specified on CATEGORICAL are replaced by sets of contrast variables. The set of contrast variables associated with a categorical variable is entered or removed from the model as a single step.
- Any number of METHOD subcommands can appear in a Logistic Regression procedure. METHOD subcommands are processed in the order in which they are specified. Each method starts with the results from the previous method. If BSTEP is used, all remaining eligible variables are entered at the first step. All variables are then eligible for entry and removal unless they have been excluded from the METHOD variable list.
- The beginning model for the first METHOD subcommand is either the constant variable (by default or if NOORIGIN is specified) or an empty model (if ORIGIN is specified).

The available METHOD keywords are as follows:

ENTER. *Forced entry.* All variables are entered in a single step. This setting is the default if the METHOD subcommand is omitted.

FSTEP. *Forward stepwise.* The variables (or interaction terms) that are specified on FSTEP are tested for entry into the model one by one, based on the significance level of the score statistic. The variable with the smallest significance less than PIN is entered into the model. After each entry, variables that are already in the model are tested for possible removal, based on the significance of the conditional statistic, the Wald statistic, or the likelihood-ratio criterion. The variable with the largest probability greater than the specified POUT value is removed, and the model is reestimated. Variables in the model are then evaluated again for removal. When no more variables satisfy the removal criterion, covariates that are not in the model are evaluated for entry. Model building stops when no more variables meet entry or removal criteria or when the current model is the same as a previous model.

BSTEP. *Backward stepwise.* As a first step, the variables (or interaction terms) that are specified on BSTEP are entered into the model together and are tested for removal one by one. Stepwise removal and entry then follow the same process as described for FSTEP until no more variables meet entry or removal criteria or when the current model is the same as a previous model.

The statistic that is used in the test for removal can be specified by an additional keyword in parentheses following FSTEP or BSTEP. If FSTEP or BSTEP is specified by itself, the default is COND.

COND. *Conditional statistic.* This setting is the default if FSTEP or BSTEP is specified by itself.

WALD. *Wald statistic.* The removal of a variable from the model is based on the significance of the Wald statistic.

LR. *Likelihood ratio.* The removal of a variable from the model is based on the significance of the change in the log-likelihood. If LR is specified, the model must be reestimated without each of the variables in the model. This process can substantially increase computational time. However, the likelihood-ratio statistic is the best criterion for deciding which variables are to be removed.

Example

```
LOGISTIC REGRESSION
  VARIABLES = PROMOTED WITH AGE JOBTIME JOBRATE RACE SEX AGENCY
 /CATEGORICAL RACE SEX AGENCY
 /METHOD ENTER AGE  JOBTIME
 /METHOD BSTEP (LR) RACE SEX JOBRATE AGENCY.
```

- *AGE*, *JOBTIME*, *JOBRATE*, *RACE*, *SEX*, and *AGENCY* are specified as independent variables. *RACE*, *SEX*, and *AGENCY* are specified as categorical independent variables.
- The first METHOD subcommand enters *AGE* and *JOBTIME* into the model.
- Variables in the model at the termination of the first METHOD subcommand are included in the model at the beginning of the second METHOD subcommand.
- The second METHOD subcommand adds the variables *RACE*, *SEX*, *JOBRATE*, and *AGENCY* to the previous model.
- Backward stepwise logistic regression analysis is then done with only the variables on the BSTEP variable list tested for removal by using the LR statistic.

- The procedure continues until all variables from the BSTEP variable list have been removed or the removal of a variable will not result in a decrease in the log-likelihood with a probability larger than POUT.

SELECT Subcommand

By default, all cases in the active dataset are considered for inclusion in LOGISTIC REGRESSION. Use the optional SELECT subcommand to include a subset of cases in the analysis.

- The specification is either a logical expression or keyword ALL. ALL is the default. Variables that are named on VARIABLES, CATEGORICAL, or METHOD subcommands cannot appear on SELECT.
- In the logical expression on SELECT, the relation can be EQ, NE, LT, LE, GT, or GE. The variable must be numeric, and the value can be any number.
- Only cases for which the logical expression on SELECT is true are included in calculations. All other cases, including those cases with missing values for the variable that is named on SELECT, are unselected.
- Diagnostic statistics and classification statistics are reported for both selected and unselected cases.
- Cases that are deleted from the active dataset with the SELECT IF or SAMPLE command are not included among either the selected or unselected cases.

Example

```
LOGISTIC REGRESSION VARIABLES=GRADE WITH GPA,TUCE,PSI
 /SELECT SEX EQ 1 /CASEWISE=RESID.
```

- Only cases with the value 1 for *SEX* are included in the logistic regression analysis.
- Residual values that are generated by CASEWISE are displayed for both selected and unselected cases.

ORIGIN and NOORIGIN Subcommands

ORIGIN and NOORIGIN control whether the constant is included. NOORIGIN (the default) includes a constant term (intercept) in all equations. ORIGIN suppresses the constant term and requests regression through the origin. (NOCONST can be used as an alias for ORIGIN.)

- The only specification is either ORIGIN or NOORIGIN.
- ORIGIN or NOORIGIN can be specified only once per Logistic Regression procedure, and it affects all METHOD subcommands.

Example

```
LOGISTIC REGRESSION VARIABLES=PASS WITH GPA,GRE,MAT /ORIGIN.
```

- ORIGIN suppresses the automatic generation of a constant term.

ID Subcommand

ID specifies a variable whose values or value labels identify the casewise listing. By default, cases are labeled by their case number.

- The only specification is the name of a single variable that exists in the active dataset.
- Only the first eight characters of the variable's value labels are used to label cases. If the variable has no value labels, the values are used.
- Only the first eight characters of a string variable are used to label cases.

PRINT Subcommand

PRINT controls the display of optional output. If PRINT is omitted, DEFAULT output (defined below) is displayed.

- The minimum specification is PRINT followed by a single keyword.
- If PRINT is used, only the requested output is displayed.

DEFAULT. *Goodness-of-fit tests for the model, classification tables, and statistics for the variables in and not in the equation at each step.* Tables and statistics are displayed for each split file and METHOD subcommand.

SUMMARY. *Summary information.* This output is the same output as DEFAULT, except that the output for each step is not displayed.

CORR. *Correlation matrix of parameter estimates for the variables in the model.*

ITER(value). *Iterations at which parameter estimates are to be displayed.* The value in parentheses controls the spacing of iteration reports. If the value is *n*, the parameter estimates are displayed for every *n*th iteration, starting at 0. If a value is not supplied, intermediate estimates are displayed at each iteration.

GOODFIT. *Hosmer-Lemeshow goodness-of-fit statistic* [37].

CI(level). *Confidence interval for exp(B).* The value in parentheses must be an integer between 1 and 99.

ALL. *All available output.*

Example

```
LOGISTIC REGRESSION VARIABLES=PASS WITH GPA,GRE,MAT
 /METHOD FSTEP
 /PRINT CORR SUMMARY ITER(2).
```

- A forward stepwise logistic regression analysis of *PASS* on *GPA*, *GRE*, and *MAT* is specified.
- The PRINT subcommand requests the display of the correlation matrix of parameter estimates for the variables in the model (CORR), classification tables and statistics for the variables in and not in the equation for the final model (SUMMARY), and parameter estimates at every second iteration (ITER(2)).

CRITERIA Subcommand

CRITERIA controls the statistical criteria that are used in building the logistic regression models. The way in which these criteria are used depends on the method that is specified on the METHOD subcommand. The default criteria are noted in the description of each keyword below. Iterations will stop if the criterion for BCON, LCON, or ITERATE is satisfied.

BCON(value). *Change in parameter estimates to terminate iteration.* Iteration terminates when the parameters change by less than the specified value. The default is 0.001. To eliminate this criterion, specify a value of 0.

ITERATE. *Maximum number of iterations.* The default is 20.

LCON(value). *Percentage change in the log-likelihood ratio for termination of iterations.* If the log-likelihood decreases by less than the specified value, iteration terminates. The default is 0, which is equivalent to not using this criterion.

PIN(value). *Probability of score statistic for variable entry.* The default is 0.05. The larger the specified probability, the easier it is for a variable to enter the model.

[37] Hosmer, D. W., and S. Lemeshow. 2000. *Applied Logistic Regression*, 2nd ed. New York: John Wiley and Sons.

POUT(value). *Probability of conditional, Wald, or LR statistic to remove a variable.* The default is 0.1. The larger the specified probability, the easier it is for a variable to remain in the model.

EPS(value). *Epsilon value used for redundancy checking.* The specified value must be less than or equal to 0.05 and greater than or equal to 10^{-12}. The default is 10^{-8}. Larger values make it harder for variables to pass the redundancy check—that is, they are more likely to be removed from the analysis.

CUT(value). *Cutoff value for classification.* A case is assigned to a group when the predicted event probability is greater than or equal to the cutoff value. The cutoff value affects the value of the dichotomous derived variable in the classification table, the predicted group (PGROUP on CASEWISE), and the classification plot (CLASSPLOT). The default cutoff value is 0.5. You can specify a value between 0 and 1 (0 < `value` < 1).

Example

```
LOGISTIC REGRESSION VARIABLES = PROMOTED WITH AGE JOBTIME RACE
 /CATEGORICAL RACE
 /METHOD BSTEP
 /CRITERIA BCON(0.01) PIN(0.01) POUT(0.05).
```

- A backward stepwise logistic regression analysis is performed for the dependent variable *PROMOTED* and the independent variables *AGE*, *JOBTIME*, and *RACE*.
- CRITERIA alters four of the statistical criteria that control the building of a model.
- BCON specifies that if the change in the absolute value of all of the parameter estimates is less than 0.01, the iterative estimation process should stop. Larger values lower the number of required iterations. Notice that the ITER and LCON criteria remain unchanged and that if either of them is met before BCON, iterations will terminate. (LCON can be set to 0 if only BCON and ITER are to be used.)
- POUT requires that the probability of the statistic that is used to test whether a variable should remain in the model be smaller than 0.05. This requirement is more stringent than the default value of 0.1.
- PIN requires that the probability of the score statistic that is used to test whether a variable should be included be smaller than 0.01. This requirement makes it more difficult for variables to be included in the model than the default value of 0.05.

CLASSPLOT Subcommand

CLASSPLOT generates a classification plot of the actual and predicted values of the dichotomous dependent variable at each step.

- Keyword CLASSPLOT is the only specification.
- If CLASSPLOT is not specified, plots are not generated.

Example

```
LOGISTIC REGRESSION VARIABLES = PROMOTED WITH JOBTIME RACE
 /CATEGORICAL RACE
 /CLASSPLOT.
```

- A logistic regression model is constructed for the dichotomous dependent variable *PROMOTED* and the independent variables *JOBTIME* and *RACE*.
- CLASSPLOT produces a classification plot for the dependent variable *PROMOTED*. The vertical axis of the plot is the frequency of the variable *PROMOTED*. The horizontal axis is the predicted probability of membership in the second of the two levels of *PROMOTED*.

CASEWISE Subcommand

CASEWISE produces a casewise listing of the values of the temporary variables that are created by LOGISTIC REGRESSION.

The following keywords are available for specifying temporary variables (see Fox, 1984). When CASEWISE is specified by itself, the default is to list *PRED*, *PGROUP*, *RESID*, *ZRESID*, and *SRESID*. If a list of variable names is given, only those named temporary variables are displayed.

PRED. *Predicted probability.* For each case, the predicted probability of having the second of the two values of the dichotomous dependent variable.

PGROUP. *Predicted group.* The group to which a case is assigned based on the predicted probability.

RESID. *Difference between observed and predicted probabilities.*

DEV. *Deviance values.* For each case, a log-likelihood-ratio statistic, which measures how well the model fits the case, is computed.

LRESID. *Logit residual.* Residual divided by the product of *PRED* and 1–*PRED*.

SRESID. *Studentized residual.*

ZRESID. *Normalized residual.* Residual divided by the square root of the product of *PRED* and 1–*PRED*.

LEVER. *Leverage value.* A measure of the relative influence of each observation on the model's fit.

COOK. *Analog of Cook's influence statistic.*

DFBETA. *Difference in beta.* The difference in the estimated coefficients for each independent variable if the case is omitted.

The following keyword is available for restricting the cases to be displayed, based on the absolute value of *SRESID*:

OUTLIER (value). *Cases with absolute values of SRESID greater than or equal to the specified value are displayed.* If OUTLIER is specified with no value, the default is 2.

Example

```
LOGISTIC REGRESSION VARIABLES = PROMOTED WITH JOBTIME SEX RACE
 /CATEGORICAL SEX RACE
 /METHOD ENTER
 /CASEWISE SRESID LEVER DFBETA.
```

- CASEWISE produces a casewise listing of the temporary variables *SRESID*, *LEVER*, and *DFBETA*.
- There will be one *DFBETA* value for each parameter in the model. The continuous variable *JOBTIME*, the two-level categorical variable *SEX*, and the constant each require one parameter, while the four-level categorical variable *RACE* requires three parameters. Thus, six values of *DFBETA* will be produced for each case.

MISSING Subcommand

LOGISTIC REGRESSION excludes all cases with missing values on any of the independent variables. For a case with a missing value on the dependent variable, predicted values are calculated if it has nonmissing values on all independent variables. The MISSING subcommand controls the processing of user-missing values. If the subcommand is not specified, the default is EXCLUDE.

EXCLUDE. *Delete cases with user-missing values as well as system-missing values.* This setting is the default.

INCLUDE. *Include user-missing values in the analysis.*

OUTFILE Subcommand

The OUTFILE subcommand allows you to specify files to which output is written.

- Only one OUTFILE subcommand is allowed. If you specify more than one subcommand, only the last subcommand is executed.

- You must specify at least one keyword and a valid filename in parentheses. There is no default.
- MODEL cannot be used if split-file processing is on (SPLIT FILE command) or if more than one dependent variable is specified (DEPENDENT subcommand).

MODEL(filename). *Write parameter estimates and their covariances to an XML file.* Specify the filename in full. LOGISTIC REGRESSION does not supply an extension. You can use this model file to apply the model information to other data files for scoring purposes. See the topic "Scoring expressions" on page 86 for more information.

PARAMETER(filename). *Write parameter estimates only to an XML file.* Specify the filename in full. LOGISTIC REGRESSION does not supply an extension. You can use this model file to apply the model information to other data files for scoring purposes. See the topic "Scoring expressions" on page 86 for more information.

SAVE Subcommand

SAVE saves the temporary variables that are created by LOGISTIC REGRESSION. To specify variable names for the new variables, assign the new names in parentheses following each temporary variable name. If new variable names are not specified, LOGISTIC REGRESSION generates default names.

- Assigned variable names must be unique in the active dataset. Scratch or system variable names (that is, names that begin with # or $) cannot be used.
- A temporary variable can be saved only once on the same SAVE subcommand.

Example

```
LOGISTIC REGRESSION VARIABLES = PROMOTED WITH JOBTIME AGE
 /SAVE PRED (PREDPRO) DFBETA (DF).
```

- A logistic regression analysis of *PROMOTED* on the independent variables *JOBTIME* and *AGE* is performed.
- SAVE adds four variables to the active dataset: one variable named *PREDPRO*, containing the predicted value from the specified model for each case, and three variables named *DF0*, *DF1*, and *DF2*, containing, respectively, the *DFBETA* values for each case of the constant, the independent variable *JOBTIME*, and the independent variable *AGE*.

EXTERNAL Subcommand

EXTERNAL indicates that the data for each split-file group should be held in an external scratch file during processing. This process can help conserve memory resources when running complex analyses or analyses with large data sets.

- The keyword EXTERNAL is the only specification.
- Specifying EXTERNAL may result in slightly longer processing time.
- If EXTERNAL is not specified, all data are held internally, and no scratch file is written.

References

Agresti, A. 2002. *Categorical Data Analysis*, 2nd ed. New York: John Wiley and Sons.

Aldrich, J. H., and F. D. Nelson. 1994. *Linear Probability, Logit and Probit Models*. Thousand Oaks, Calif.: Sage Publications, Inc..

Finn, J. D. 1974. *A general model for multivariate analysis*. New York: Holt, Rinehart and Winston.

Fox, J. 1984. *Linear statistical models and related methods: With applications to social research*. New York: John Wiley and Sons.

Hosmer, D. W., and S. Lemeshow. 2000. *Applied Logistic Regression*, 2nd ed. New York: John Wiley and Sons.

Kirk, R. E. 1982. *Experimental design*, 2nd ed. Monterey, California: Brooks/Cole.

McCullagh, P., and J. A. Nelder. 1989. *Generalized Linear Models*, 2nd ed. London: Chapman & Hall.

LOGLINEAR

LOGLINEAR is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

The syntax for LOGLINEAR is available only in a syntax window, not from the dialog box interface. See GENLOG for information on loglinear models available from the dialog box interface.

```
LOGLINEAR varlist(min,max)...[BY] varlist(min,max)

          [WITH covariate varlist]

 [/CWEIGHT={varname }] [/CWEIGHT=(matrix)...]
           {(matrix)}

 [/GRESID={varlist }]  [/GRESID=(matrix)...]
          {(matrix)}

 [/CONTRAST (varname)={DEVIATION [(refcat)]      } [/CONTRAST...]]
                      {DIFFERENCE                }
                      {HELMERT                   }
                      {SIMPLE [(refcat)]         }
                      {REPEATED                  }
                      {POLYNOMIAL [({1,2,3,...})]}
                      {           {metric   }    }
                      {[BASIS] SPECIAL(matrix)   }

 [/CRITERIA=[CONVERGE({0.001**})] [ITERATE({20**})] [DELTA({0.5**})]
                      {n      }           {n   }          {n    }
            [DEFAULT]]

 [/PRINT={[FREQ**][RESID**][DESIGN][ESTIM][COR]}]
         {DEFAULT                              }
         {ALL                                  }
         {NONE                                 }

 [/PLOT={NONE**  }]
        {DEFAULT }
        {RESID   }
        {NORMPROB}

 [/MISSING=[{EXCLUDE**}]]
           {INCLUDE  }

 [/DESIGN=effect[(n)] effect[(n)]... effect BY effect...] [/DESIGN...]
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
LOGLINEAR JOBSAT (1,2) ZODIAC (1,12) /DESIGN=JOBSAT.
```

Overview

LOGLINEAR is a general procedure for model fitting, hypothesis testing, and parameter estimation for any model that has categorical variables as its major components. As such, LOGLINEAR subsumes a variety of related techniques, including general models of multiway contingency tables, logit models, logistic regression on categorical variables, and quasi-independence models.

LOGLINEAR models cell frequencies using the multinomial response model and produces maximum likelihood estimates of parameters by means of the Newton-Raphson algorithm [38]. HILOGLINEAR, which uses an iterative proportional-fitting algorithm, is more efficient for hierarchical models, but it cannot produce parameter estimates for unsaturated models, does not permit specification of contrasts for parameters, and does not display a correlation matrix of the parameter estimates.

Comparison of the GENLOG and LOGLINEAR Commands

The General Loglinear Analysis and Logit Loglinear Analysis dialog boxes are both associated with the GENLOG command. In previous releases, these dialog boxes were associated with the LOGLINEAR

[38] Haberman, S. J. 1978. *Analysis of qualitative data*. London: Academic Press.

command. The LOGLINEAR command is now available only as a syntax command. The differences are described below.

Distribution Assumptions

- GENLOG can handle both Poisson and multinomial distribution assumptions for observed cell counts.
- LOGLINEAR assumes only multinomial distribution.

Approach

- GENLOG uses a regression approach to parameterize a categorical variable in a design matrix.
- LOGLINEAR uses contrasts to reparameterize a categorical variable. The major disadvantage of the reparameterization approach is in the interpretation of the results when there is a redundancy in the corresponding design matrix. Also, the reparameterization approach may result in incorrect degrees of freedom for an incomplete table, leading to incorrect analysis results.

Contrasts and Generalized Log-Odds Ratios (GLOR)

- GENLOG doesn't provide contrasts to reparameterize the categories of a factor. However, it offers generalized log-odds ratios (GLOR) for cell combinations. Often, comparisons among categories of factors can be derived from GLOR.
- LOGLINEAR offers contrasts to reparameterize the categories of a factor.

Deviance Residual

- GENLOG calculates and displays the deviance residual and its normal probability plot in addition to the other residuals.
- LOGLINEAR does not calculate the deviance residual.

Factor-by-Covariate Design

- When there is a factor-by-covariate term in the design, GENLOG generates one regression coefficient of the covariate for each combination of factor values. The estimates of these regression coefficients are calculated and displayed.
- LOGLINEAR estimates and displays the contrasts of these regression coefficients.

Partition Effect

- In GENLOG, the term *partition effect* refers to the category of a factor.
- In LOGLINEAR, the term *partition effect* refers to a particular contrast.

Options

Model Specification. You can specify the model or models to be fit using the DESIGN subcommand.

Cell Weights. You can specify cell weights, such as structural zeros, for the model with the CWEIGHT subcommand.

Output Display. You can control the output display with the PRINT subcommand.

Optional Plots. You can produce plots of adjusted residuals against observed and expected counts, normal plots, and detrended normal plots with the PLOT subcommand.

Linear Combinations. You can calculate linear combinations of observed cell frequencies, expected cell frequencies, and adjusted residuals using the GRESID subcommand.

Contrasts. You can indicate the type of contrast desired for a factor using the CONTRAST subcommand.

Criteria for Algorithm. You can control the values of algorithm-tuning parameters with the CRITERIA subcommand.

Basic Specification

The basic specification is two or more variables that define the crosstabulation. The minimum and maximum values for each variable must be specified in parentheses after the variable name.

By default, LOGLINEAR estimates the saturated model for a multidimensional table. Output includes the factors or effects, their levels, and any labels; observed and expected frequencies and percentages for each factor and code; residuals, standardized residuals, and adjusted residuals; two goodness-of-fit statistics (the likelihood-ratio chi-square and Pearson's chi-square); and estimates of the parameters with accompanying *z* values and 95% confidence intervals.

Limitations

- A maximum of 10 independent (factor) variables
- A maximum of 200 covariates

Subcommand Order

- The variables specification must come first.
- The subcommands that affect a specific model must be placed before the DESIGN subcommand specifying the model.
- All subcommands can be used more than once and, with the exception of the DESIGN subcommand, are carried from model to model unless explicitly overridden.
- If the last subcommand is not DESIGN, LOGLINEAR generates a saturated model in addition to the explicitly requested model(s).

Variable List

The variable list specifies the variables to be included in the model. LOGLINEAR analyzes two classes of variables: categorical and continuous. Categorical variables are used to define the cells of the table. Continuous variables are used as cell covariates. Continuous variables can be specified only after the keyword WITH following the list of categorical variables.

- The list of categorical variables must be specified first. Categorical variables must be numeric and integer.
- A range must be defined for each categorical variable by specifying, in parentheses after each variable name, the minimum and maximum values for that variable. Separate the two values with at least one space or a comma.
- To specify the same range for a list of variables, specify the list of variables followed by a single range. The range applies to all variables on the list.
- To specify a logit model, use the keyword BY (see "Logit Model " on page 1003). A variable list without the keyword BY generates a general loglinear model.
- Cases with values outside the specified range are excluded from the analysis. Non-integer values within the range are truncated for the purpose of building the table.

Logit Model

- To segregate the independent (factor) variables from the dependent variables in a logit model, use the keyword BY. The categorical variables preceding BY are the dependent variables; the categorical variables following BY are the independent variables.
- A total of 10 categorical variables can be specified. In most cases, one of them is dependent.
- A DESIGN subcommand should be used to request the desired logit model.
- LOGLINEAR displays an analysis of dispersion and two measures of association: entropy and concentration. These measures are discussed elsewhere [39] and can be used to quantify the magnitude of association among the variables. Both are proportional reduction in error measures. The entropy statistic is analogous to Theil's entropy measure, while the concentration statistic is analogous to Goodman and Kruskal's tau-*b.* Both statistics measure the strength of association between the dependent variable and the predictor variable set.

[39] Haberman, S. J. 1982. Analysis of dispersion of multinomial responses. *Journal of the American Statistical Association,* 77, 568-580.

Cell Covariates

- Continuous variables can be used as covariates. When used, the covariates must be specified after the keyword WITH following the list of categorical variables. Ranges are not specified for the continuous variables.
- A variable cannot be named as both a categorical variable and a cell covariate.
- To enter cell covariates into a model, the covariates must be specified on the DESIGN subcommand.
- Cell covariates are not applied on a case-by-case basis. The mean covariate value for a cell in the contingency table is applied to that cell.

Example

```
LOGLINEAR DPREF(2,3) RACE CAMP (1,2) WITH CONSTANT
  /DESIGN=DPREF RACE CAMP CONSTANT.
```

- The variable *CONSTANT* is a continuous variable specified as a cell covariate. Cell covariates must be specified after the keyword WITH following the variable list. No range is defined for cell covariates.
- To include the cell covariate in the model, the variable *CONSTANT* is specified on DESIGN.

CWEIGHT Subcommand

CWEIGHT specifies cell weights, such as structural zeros, for a model. By default, cell weights are equal to 1.

- The specification is either one numeric variable or a matrix of weights enclosed in parentheses.
- If a matrix of weights is specified, the matrix must contain the same number of elements as the product of the levels of the categorical variables. An asterisk can be used to signify repetitions of the same value.
- If weights are specified for a multiple-factor model, the index value of the rightmost factor increments the most rapidly.
- If a numeric variable is specified, only one CWEIGHT subcommand can be used on LOGLINEAR.
- To use multiple cell weights on the same LOGLINEAR command, specify all weights in matrix format. Each matrix must be specified on a separate CWEIGHT subcommand, and each CWEIGHT specification remains in effect until explicitly overridden by another CWEIGHT subcommand.
- CWEIGHT can be used to impose structural, or *a priori*, zeros on the model. This feature is useful in the analysis of symmetric tables.

Example

```
COMPUTE  CWT=1.
IF (HUSED EQ WIFED) CWT=0.
LOGLINEAR HUSED WIFED(1,4) WITH DISTANCE
  /CWEIGHT=CWT
  /DESIGN=HUSED WIFED DISTANCE.
```

- COMPUTE initially assigns *CWT* the value 1 for all cases.
- IF assigns *CWT* the value 0 when *HUSED* equals *WIFED*.
- CWEIGHT imposes structural zeros on the diagonal of the symmetric crosstabulation. Because a variable name is specified, only one CWEIGHT can be used.

Example

```
LOGLINEAR  HUSED WIFED(1,4) WITH DISTANCE
  /CWEIGHT=(0, 4*1, 0, 4*1, 0, 4*1, 0)
  /DESIGN=HUSED WIFED DISTANCE
  /CWEIGHT=(16*1)
  /DESIGN=HUSED WIFED DISTANCE.
```

- The first CWEIGHT matrix specifies the same values as variable *CWT* provided in the first example. The specified matrix is as follows:

```
0 1 1 1
1 0 1 1
1 1 0 1
1 1 1 0
```

- The same matrix can be specified in full as `(0 1 1 1 1 0 1 1 1 1 0 1 1 1 1 0)`.
- By using the matrix format on CWEIGHT rather than a variable name, a different CWEIGHT subcommand can be used for the second model.

GRESID Subcommand

GRESID (generalized residual) calculates linear combinations of observed cell frequencies, expected cell frequencies, and adjusted residuals.

- The specification is either a numeric variable or a matrix whose contents are coefficients of the desired linear combinations.
- If a matrix of coefficients is specified, the matrix must contain the same number of elements as the number of cells implied by the variables specification. An asterisk can be used to signify repetitions of the same value.
- Each GRESID subcommand specifies a single linear combination. Each matrix or variable must be specified on a separate GRESID subcommand. All GRESID subcommands specified are displayed for each design.

Example

```
LOGLINEAR  MONTH(1,18) WITH Z
 /GRESID=(6*1,12*0)
 /GRESID=(6*0,6*1,6*0)
 /GRESID=(12*0,6*1)
 /DESIGN=Z.
```

- The first GRESID subcommand combines the first six months into a single effect. The second GRESID subcommand combines the second six months, and the third GRESID subcommand combines the last six months.
- For each effect, LOGLINEAR displays the observed and expected counts, the residual, and the adjusted residual.

CONTRAST Subcommand

CONTRAST indicates the type of contrast desired for a factor, where a factor is any categorical dependent or independent variable. The default contrast is DEVIATION for each factor.

- The specification is CONTRAST, which is followed by a variable name in parentheses and the contrast-type keyword.
- To specify a contrast for more than one factor, use a separate CONTRAST subcommand for each specified factor. Only one contrast can be in effect for each factor on each DESIGN.
- A contrast specification remains in effect for subsequent designs until explicitly overridden by another CONTRAST subcommand.
- The design matrix used for the contrasts can be displayed by specifying the keyword DESIGN on the PRINT subcommand. However, this matrix is the basis matrix that is used to determine contrasts; it is not the contrast matrix itself.
- CONTRAST can be used for a multinomial logit model, in which the dependent variable has more than two categories.
- CONTRAST can be used for fitting linear logit models. The keyword BASIS is not appropriate for such models.

- In a logit model, CONTRAST is used to transform the independent variable into a metric variable. Again, the keyword BASIS is not appropriate.

The following contrast types are available:

DEVIATION(refcat). *Deviations from the overall effect.* DEVIATION is the default contrast if the CONTRAST subcommand is not used. Refcat is the category for which parameter estimates are not displayed (they are the negative of the sum of the others). By default, refcat is the last category of the variable.

DIFFERENCE. *Levels of a factor with the average effect of previous levels of a factor.* Also known as reverse Helmert contrasts.

HELMERT. *Levels of a factor with the average effect of subsequent levels of a factor.*

SIMPLE(refcat). *Each level of a factor to the reference level.* By default, LOGLINEAR uses the last category of the factor variable as the reference category. Optionally, any level can be specified as the reference category enclosed in parentheses after the keyword SIMPLE. The sequence of the level, not the actual value, must be specified.

REPEATED. *Adjacent comparisons across levels of a factor.*

POLYNOMIAL(metric). *Orthogonal polynomial contrasts.* The default is equal spacing. Optionally, the coefficients of the linear polynomial can be specified in parentheses, indicating the spacing between levels of the treatment measured by the given factor.

[BASIS]SPECIAL(matrix). *User-defined contrast.* As many elements as the number of categories squared must be specified. If BASIS is specified before SPECIAL, a basis matrix is generated for the special contrast, which makes the coefficients of the contrast equal to the special matrix. Otherwise, the matrix specified is transposed and then used as the basis matrix to determine coefficients for the contrast matrix.

Example

```
LOGLINEAR  A(1,4) BY B(1,4)
 /CONTRAST(B)=POLYNOMIAL
 /DESIGN=A A BY B(1)
 /CONTRAST(B)=SIMPLE
 /DESIGN=A A BY B(1).
```

- The first CONTRAST subcommand requests polynomial contrasts of *B* for the first design.
- The second CONTRAST subcommand requests the simple contrast of *B*, with the last category (value 4) used as the reference category for the second DESIGN subcommand.

Example

```
* Multinomial logit model

LOGLINEAR  PREF(1,5) BY RACE ORIGIN CAMP(1,2)
  /CONTRAST(PREF)=SPECIAL(5*1, 1 1 1 1 -4, 3 -1 -1 -1 0,
        0 1 1 -2 0, 0 1 -1 0 0).
```

- LOGLINEAR builds special contrasts among the five categories of the dependent variable *PREF*, which measures preference for training camps among Army recruits. For *PREF*, 1=stay, 2=move to north, 3=move to south, 4=move to unnamed camp, and 5=undecided.
- The four contrasts are: (1) move or stay versus undecided, (2) stay versus move, (3) named camp versus unnamed, and (4) northern camp versus southern. Because these contrasts are orthogonal, SPECIAL and BASIS SPECIAL produce equivalent results.

Example

```
* Contrasts for a linear logit model

LOGLINEAR RESPONSE(1,2) BY YEAR(0,20)
 /PRINT=DEFAULT ESTIM
 /CONTRAST(YEAR)=SPECIAL(21*1, -10, -9, -8, -7, -6, -5, -4,
                         -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7,
                          8, 9, 10, 399*1)
 /DESIGN=RESPONSE RESPONSE BY YEAR(1).
```

- *YEAR* measures years of education and ranges from 0 through 20. Therefore, allowing for the constant effect, *YEAR* has 20 estimable parameters associated with it.
- The SPECIAL contrast specifies the constant—that is, 21*1—and the linear effect of *YEAR*—that is, –10 to 10. The other 399 1's fill out the 21*21 matrix.

Example

```
* Contrasts for a logistic regression model

LOGLINEAR RESPONSE(1,2) BY TIME(1,4)
  /CONTRAST(TIME) = SPECIAL(4*1, 7 14 27 51, 8*1)
  /PRINT=ALL /PLOT=DEFAULT
  /DESIGN=RESPONSE, TIME(1) BY RESPONSE.
```

- CONTRAST is used to transform the independent variable into a metric variable.
- *TIME* represents elapsed time in days. Therefore, the weights in the contrast represent the metric of the passage of time.

CRITERIA Subcommand

CRITERIA specifies the values of some constants in the Newton-Raphson algorithm. Defaults or specifications remain in effect until overridden with another CRITERIA subcommand.

CONVERGE(n). *Convergence criterion.* Specify a value for the convergence criterion. The default is 0.001.

ITERATE(n). *Maximum number of iterations.* Specify the maximum number of iterations for the algorithm. The default number is 20.

DELTA(n). *Cell delta value.* The value of delta is added to each cell frequency for the first iteration. For saturated models, it remains in the cell. The default value is 0.5. LOGLINEAR does not display parameter estimates or correlation matrices of parameter estimates if any sampling zero cells exist in the expected table after delta is added. Parameter estimates and correlation matrices can be displayed in the presence of structural zeros.

DEFAULT. *Default values are used.* DEFAULT can be used to reset the parameters to the default.

Example

```
LOGLINEAR  DPREF(2,3) BY RACE ORIGIN CAMP(1,2)
 /CRITERIA=ITERATION(50) CONVERGE(.0001).
```

- ITERATION increases the maximum number of iterations to 50.
- CONVERGE lowers the convergence criterion to 0.0001.

PRINT Subcommand

PRINT requests statistics that are not produced by default.

- By default, LOGLINEAR displays the frequency table and residuals. The parameter estimates of the model are also displayed if DESIGN is not used.
- Multiple PRINT subcommands are permitted. The specifications are cumulative.

The following keywords can be used on PRINT:

FREQ. *Observed and expected cell frequencies and percentages.* This is displayed by default.

RESID. *Raw, standardized, and adjusted residuals.* This is displayed by default.

DESIGN. *The design matrix of the model, showing the basis matrix corresponding to the contrasts used.*

ESTIM. *The parameter estimates of the model.* If you do not specify a design on the DESIGN subcommand, LOGLINEAR generates a saturated model and displays the parameter estimates for the saturated model. LOGLINEAR does not display parameter estimates or correlation matrices of parameter

estimates if any sampling zero cells exist in the expected table after delta is added. Parameter estimates and a correlation matrix are displayed when structural zeros are present.

COR. *The correlation matrix of the parameter estimates.* Alias COV.

ALL. *All available output.*

DEFAULT. *FREQ and RESID.* ESTIM is also displayed by default if the DESIGN subcommand is not used.

NONE. *The design information and goodness-of-fit statistics only.* This option overrides all other specifications on the PRINT subcommand. The NONE option applies only to the PRINT subcommand.

Example

```
LOGLINEAR A(1,2) B(1,2)
 /PRINT=ESTIM
 /DESIGN=A,B,A BY B
 /PRINT=ALL
 /DESIGN=A,B.
```

- The first design is the saturated model. The parameter estimates are displayed with ESTIM specified on PRINT.
- The second design is the main-effects model, which tests the hypothesis of no interaction. The second PRINT subcommand displays all available display output for this model.

PLOT Subcommand

PLOT produces optional plots. No plots are displayed if PLOT is not specified or is specified without any keyword. Multiple PLOT subcommands can be used. The specifications are cumulative.

RESID. *Plots of adjusted residuals against observed and expected counts.*

NORMPROB. *Normal and detrended normal plots of the adjusted residuals.*

NONE. *No plots.*

DEFAULT. *RESID and NORMPROB.* Alias ALL.

Example

```
LOGLINEAR  RESPONSE(1,2) BY TIME(1,4)
  /CONTRAST(TIME)=SPECIAL(4*1, 7 14 27 51, 8*1)
  /PLOT=DEFAULT
  /DESIGN=RESPONSE TIME(1) BY RESPONSE
  /PLOT=NONE
  /DESIGN.
```

- RESID and NORMPROB plots are displayed for the first design.
- No plots are displayed for the second design.

MISSING Subcommand

MISSING controls missing values. By default, LOGLINEAR excludes all cases with system- or user-missing values on any variable. You can specify INCLUDE to include user-missing values. If INCLUDE is specified, user-missing values must also be included in the value range specification.

EXCLUDE. *Delete cases with user-missing values.* This is the default if the subcommand is omitted. You can also specify the keyword DEFAULT.

INCLUDE. *Include user-missing values.* Only cases with system-missing values are deleted.

Example

```
MISSING VALUES A(0).
LOGLINEAR A(0,2) B(1,2) /MISSING=INCLUDE
 /DESIGN=B.
```

- Even though 0 was specified as missing, it is treated as a nonmissing category of *A* in this analysis.

DESIGN Subcommand

DESIGN specifies the model or models to be fit. If DESIGN is omitted or used with no specifications, the saturated model is produced. The saturated model fits all main effects and all interaction effects.

- To specify more than one model, use more than one DESIGN subcommand. Each DESIGN specifies one model.
- To obtain main-effects models, name all the variables listed on the variables specification.
- To obtain interactions, use the keyword BY to specify each interaction, as in A BY B and C BY D. To obtain the single-degree-of-freedom partition of a specified contrast, specify the partition in parentheses following the factor (see the example below).
- To include cell covariates in the model, first identify them on the variable list by naming them after the keyword WITH, and then specify the variable names on DESIGN.
- To specify an equiprobability model, name a cell covariate that is actually a constant of 1.

Example

```
* Testing the linear effect of the dependent variable

COMPUTE X=MONTH.
LOGLINEAR MONTH (1,12) WITH X
  /DESIGN X.
```

- The variable specification identifies *MONTH* as a categorical variable with values 1 through 12. The keyword WITH identifies *X* as a covariate.
- DESIGN tests the linear effect of *MONTH*.

Example

```
* Specifying main effects models

LOGLINEAR A(1,4) B(1,5)
  /DESIGN=A
  /DESIGN=A,B.
```

- The first design tests the homogeneity of category probabilities for *B*; it fits the marginal frequencies on *A*, but assumes that membership in any of the categories of *B* is equiprobable.
- The second design tests the independence of *A* and *B*. It fits the marginals on both *A* and *B*.

Example

```
* Specifying interactions

LOGLINEAR A(1,4) B(1,5) C(1,3)
  /DESIGN=A,B,C, A BY B.
```

- This design consists of the *A* main effect, the *B* main effect, the *C* main effect, and the interaction of *A* and *B*.

Example

```
* Single-degree-of-freedom partitions
 LOGLINEAR A(1,4) BY B(1,5)
  /CONTRAST(B)=POLYNOMIAL
  /DESIGN=A,A BY B(1).
```

- The value 1 following *B* refers to the first partition of *B*, which is the linear effect of *B*; this follows from the contrast specified on the CONTRAST subcommand.

Example

```
* Specifying cell covariates

LOGLINEAR HUSED WIFED(1,4) WITH DISTANCE
   /DESIGN=HUSED WIFED DISTANCE.
```

- The continuous variable *DISTANCE* is identified as a cell covariate by specifying it after WITH on the variable list. The cell covariate is then included in the model by naming it on DESIGN.

Example

```
* Equiprobability model

COMPUTE  X=1.
LOGLINEAR  MONTH(1,18) WITH X
  /DESIGN=X.
```

- This model tests whether the frequencies in the 18-cell table are equal by using a cell covariate that is a constant of 1.

LOOP-END LOOP

```
LOOP [varname=n TO m [BY {1**}]]  [IF [(]logical expression[)]]
                         {n  }

transformation commands

END LOOP [IF [(]logical expression[)]]
```

**Default if the keyword is omitted.

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic "Command Order" on page 36 for more information.

Examples

```
SET MXLOOPS=10. /*Maximum number of loops allowed
LOOP. /*Loop with no limit other than MXLOOPS
COMPUTE X=X+1.
END LOOP.


LOOP #I=1 TO 5. /*Loop five times
COMPUTE X=X+1.
END LOOP.
```

Overview

The LOOP-END LOOP structure performs repeated transformations specified by the commands within the loop until they reach a specified cutoff. The cutoff can be specified by an indexing clause on the LOOP command, an IF clause on the END LOOP command, or a BREAK command within the loop structure (see BREAK). In addition, the maximum number of iterations within a loop can be specified on the MXLOOPS subcommand on SET. The default MXLOOPS is 40.

The IF clause on the LOOP command can be used to perform repeated transformations on a subset of cases. The effect is similar to nesting the LOOP-END LOOP structure within a DO IF-END IF structure, but using IF on LOOP is simpler and more efficient. You have to use the DO IF-END IF structure, however, if you want to perform different transformations on different subsets of cases. You can also use IF on LOOP to specify the cutoff, especially when the cutoff may be reached before the first iteration.

LOOP and END LOOP are usually used within an input program or with the VECTOR command. Since the loop structure repeats transformations on a single case or on a single input record containing information on multiple cases, it allows you to read complex data files or to generate data for a active dataset. For more information, see INPUT PROGRAM-END INPUT PROGRAM and VECTOR.

The loop structure repeats transformations on single cases across variables. It is different from the DO REPEAT-END REPEAT structure, which replicates transformations on a specified set of variables. When both can be used to accomplish a task, such as selectively transforming data for some cases on some variables, LOOP and END LOOP are generally more efficient and more flexible, but DO REPEAT allows selection of nonadjacent variables and use of replacement values with different intervals.

Options

Missing Values. You can prevent cases with missing values for any of the variables used in the loop structure from entering the loop. See the topic "Missing Values " on page 1017 for more information.

Creating Data. A loop structure within an input program can be used to generate data. See the topic "Creating Data " on page 1017 for more information.

Defining Complex File Structures. A loop structure within an input program can be used to define complex files that cannot be handled by standard file definition facilities.

Basic Specification

The basic specification is LOOP followed by at least one transformation command. The structure must end with the END LOOP command. Commands within the loop are executed until the cutoff is reached.

Syntax Rules

- If LOOP and END LOOP are specified before an active dataset exists, they must be specified within an input program.
- If both an indexing and an IF clause are used on LOOP, the indexing clause must be first.
- Loop structures can be nested within other loop structures or within DO IF structures, and vice versa.

Operations

- The LOOP command defines the beginning of a loop structure and the END LOOP command defines its end. The LOOP command returns control to LOOP unless the cutoff has been reached. When the cutoff has been reached, control passes to the command immediately following END LOOP.
- Commands that take effect immediately without reading the data or executing pending commands (e.g., VALUE LABELS, MISSING VALUES, VARIABLE LABEL), commands that don't require an active dataset (e.g., DISPLAY, HOST, INSERT, OMS, SET) and the GET command are processed unconditionally and immediately each time they are encountered in a loop structure. The specified condition is ignored for these commands.
- Commands that require a data pass, such as statistical or charting procedures, cannot be included in a loop structure and will cause the LOOP command to fail.
- An indexing clause (e.g., LOOP #i=1 to 1000) will override the SET MXLOOPS limit, but a loop with an IF condition will terminate if the MXLOOPS limit is reached before the condition is satisfied.

Examples

Example

```
SET MXLOOPS=10.
LOOP. /*Loop with no limit other than MXLOOPS
COMPUTE X=X+1.
END LOOP.
```

- This and the following examples assume that an active dataset and all of the variables mentioned in the loop exist.
- The SET MXLOOPS command limits the number of times the loop is executed to 10. The function of MXLOOPS is to prevent infinite loops when there is no indexing clause.
- Within the loop structure, each iteration increments *X* by 1. After 10 iterations, the value of *X* for all cases is increased by 10, and, as specified on the SET command, the loop is terminated.

Example

```
*Assume MXLOOPS set to default value of 40.
COMPUTE newvar1=0.
LOOP IF newvar1<100.
COMPUTE newvar1=newvar1+1.
END LOOP.

PRESERVE.
SET MXLOOPS 500.
COMPUTE newvar2=0.
LOOP IF newvar2<100.
COMPUTE newvar2=newvar2+1.
END LOOP.
RESTORE.

COMPUTE newvar3=0.
LOOP #i=1 to 1000.
COMPUTE newvar3=newvar3+1.
END LOOP.
EXECUTE.
```

- In the first loop, the value of *newvar1* will reach 40, at which point the loop will terminate because the MXLOOPS limit has been exceeded.
- In the second loop, the value of MXLOOPS is increased to 500, and the loop will continue to iterate until the value of *newvar2* reaches 100, at which point the IF condition is reached and the loop terminates.

- In the third loop, the indexing clause overrides the MXLOOPS setting, and the loop will iterate 1,000 times.

IF Keyword

The keyword IF and a logical expression can be specified on LOOP or on END LOOP to control iterations through the loop.

- The specification on IF is a logical expression enclosed in parentheses.

Example

```
LOOP.
COMPUTE X=X+1.
END LOOP IF (X EQ 5). /*Loop until X is 5
```

- Iterations continue until the logical expression on END LOOP is true, which for every case is when *X* equals 5. Each case does not go through the same number of iterations.
- This corresponds to the programming notion of DO UNTIL. The loop is always executed at least once.

Example

```
LOOP IF (X LT 5). /*Loop while X is less than 5
COMPUTE X=X+1.
END LOOP.
```

- The IF clause is evaluated each trip through the structure, so looping stops once *X* equals 5.
- This corresponds to the programming notion of DO WHILE. The loop may not be executed at all.

Example

```
LOOP IF (Y GT 10). /*Loop only for cases with Y GT 10
COMPUTE X=X+1.
END LOOP IF (X EQ 5). /*Loop until X IS 5
```

- The IF clause on LOOP allows transformations to be performed on a subset of cases. *X* is increased by 5 only for cases with values greater than 10 for *Y*. *X* is not changed for all other cases.

Indexing Clause

The indexing clause limits the number of iterations for a loop by specifying the number of times the program should execute commands within the loop structure. The indexing clause is specified on the LOOP command and includes an indexing variable followed by initial and terminal values.

- The program sets the *indexing variable* to the *initial value* and increases it by the specified increment each time the loop is executed for a case. When the indexing variable reaches the specified *terminal value*, the loop is terminated for that case.
- By default, the program increases the indexing variable by 1 for each iteration. The keyword BY overrides this increment.
- The indexing variable can have any valid variable name. Unless you specify a scratch variable, the indexing variable is treated as a permanent variable and is saved in the active dataset. If the indexing variable is assigned the same name as an existing variable, the values of the existing variable are altered by the LOOP structure as it is executed, and the original values are lost. See the topic “Creating Data ” on page 1017 for more information.
- The indexing clause overrides the maximum number of loops specified by SET MXLOOPS.
- The initial and terminal values of the indexing clause can be numeric expressions. Noninteger and negative expressions are allowed.
- If the expression for the initial value is greater than the terminal value, the loop is not executed. For example, #J=X TO Y is a zero-trip loop if *X* is 0 and *Y* is –1.

- If the expressions for the initial and terminal values are equal, the loop is executed once. `#J=0 TO Y` is a one-trip loop when *Y* is 0.
- If the loop is exited via BREAK or a conditional clause on the `END LOOP` statement, the iteration variable is not updated. If the LOOP statement contains both an indexing clause and a conditional clause, the indexing clause is executed first, and the iteration variable is updated regardless of which clause causes the loop to terminate.

Example

```
LOOP #I=1 TO 5. /*LOOP FIVE TIMES
COMPUTE X=X+1.
END LOOP.
```

- The scratch variable *#I* (the indexing variable) is set to the initial value of 1 and increased by 1 each time the loop is executed for a case. When *#I* increases beyond the terminal value 5, no further loops are executed. Thus, the value of *X* will be increased by 5 for every case.

Example

```
LOOP #I=1 TO 5 IF (Y GT 10). /*Loop to X=5 only if Y GT 10
COMPUTE X=X+1.
END LOOP.
```

- Both an indexing clause and an `IF` clause are specified on LOOP. *X* is increased by 5 for all cases where *Y* is greater than 10.

Example

```
LOOP #I=1 TO Y. /*Loop to the value of Y
COMPUTE X=X+1.
END LOOP.
```

- The number of iterations for a case depends on the value of the variable *Y* for that case. For a case with value 0 for the variable *Y*, the loop is not executed and *X* is unchanged. For a case with value 1 for the variable *Y*, the loop is executed once and *X* is increased by 1.

Example

```
* Factorial routine.

DATA LIST FREE / X.
BEGIN DATA
1 2 3 4 5 6 7
END DATA.

COMPUTE FACTOR=1.
LOOP #I=1 TO X.
COMPUTE FACTOR=FACTOR * #I.
END LOOP.
LIST.
```

- The loop structure computes *FACTOR* as the factorial value of *X*.

Example

```
* Example of nested loops:
  compute every possible combination of values for each variable.

INPUT PROGRAM.
-LOOP #I=1 TO 4. /* LOOP TO NUMBER OF VALUES FOR I
- LOOP #J=1 TO 3. /* LOOP TO NUMBER OF VALUES FOR J
-  LOOP #K=1 TO 4. /* LOOP TO NUMBER OF VALUES FOR K
-   COMPUTE I=#I.
-   COMPUTE J=#J.
-   COMPUTE K=#K.
-   END CASE.
-  END LOOP.
- END LOOP.
-END LOOP.
END FILE.
END INPUT PROGRAM.
LIST.
```

- The first loop iterates four times. The first iteration sets the indexing variable *#I* equal to 1 and then passes control to the second loop. *#I* remains 1 until the second loop has completed all of its iterations.

- The second loop is executed 12 times, three times for each value of *#I*. The first iteration sets the indexing variable *#J* equal to 1 and then passes control to the third loop. *#J* remains 1 until the third loop has completed all of its iterations.
- The third loop results in 48 iterations (4 × 3 × 4). The first iteration sets #K equal to 1. The COMPUTE statements set the variables *I, J,* and *K* each to 1, and END CASE creates a case. The third loop iterates a second time, setting *#K* equal to 2. Variables *I, J*, and *K* are then computed with values 1, 1, 2, respectively, and a second case is created. The third and fourth iterations of the third loop produce cases with *I, J,* and *K*, equal to 1, 1, 3 and 1, 1, 4, respectively. After the fourth iteration within the third loop, control passes back to the second loop.
- The second loop is executed again. *#I* remains 1, while *#J* increases to 2, and control returns to the third loop. The third loop completes its iterations, resulting in four more cases with *I* equal to 1, *J* to 2, and *K* increasing from 1 to 4. The second loop is executed a third time, resulting in cases with *I*=1, *J*=3, and *K* increasing from 1 to 4. Once the second loop has completed three iterations, control passes back to the first loop, and the entire cycle is repeated for the next increment of *#I*.
- Once the first loop completes four iterations, control passes out of the looping structures to END FILE. END FILE defines the resulting cases as a data file, the input program terminates, and the LIST command is executed.
- This example does not require a LEAVE command because the iteration variables are scratch variables. If the iteration variables were *I, J,* and *K*, LEAVE would be required because the variables would be reinitialized after each END CASE command.

Example

```
* Modifying the loop iteration variable.

INPUT PROGRAM.
PRINT SPACE    2.
LOOP           A = 1 TO 3. /*Simple iteration
+  PRINT          /'A WITHIN LOOP: ' A(F1).
+  COMPUTE        A = 0.
END LOOP.
PRINT          /'A AFTER LOOP:  ' A(F1).

NUMERIC        #B.
LOOP           B = 1 TO 3. /*Iteration + UNTIL
+  PRINT          /'B WITHIN LOOP: ' B(F1).
+  COMPUTE        B = 0.
+  COMPUTE        #B = #B+1.
END LOOP       IF #B = 3.
PRINT          /'B AFTER LOOP:  ' B(F1).

NUMERIC        #C.
LOOP           C = 1 TO 3 IF #C NE 3. /*Iteration + WHILE
+  PRINT          /'C WITHIN LOOP: ' C(F1).
+  COMPUTE        C = 0.
+  COMPUTE        #C = #C+1.
END LOOP.
PRINT          /'C AFTER LOOP:  ' C(F1).

NUMERIC        #D.
LOOP           D = 1 TO 3. /*Iteration + BREAK
+  PRINT          /'D WITHIN LOOP: ' D(F1).
+  COMPUTE        D = 0.
+  COMPUTE        #D = #D+1.
+  DO IF          #D = 3.
+     BREAK.
+  END IF.
END LOOP.
PRINT          /'D AFTER LOOP:  ' D(F1).

LOOP           E = 3 TO 1. /*Zero-trip iteration
+  PRINT          /'E WITHIN LOOP: ' E(F1).
+  COMPUTE        E = 0.
END LOOP.
PRINT          /'E AFTER LOOP:  ' E(F1).
END FILE.
END INPUT PROGRAM.
EXECUTE.
```

- If a loop is exited via BREAK or a conditional clause on the END LOOP statement, the iteration variable is not updated.
- If the LOOP statement contains both an indexing clause and a conditional clause, the indexing clause is executed first, and the actual iteration variable will be updated regardless of which clause causes termination of the loop.

The output from this example is shown below.

```
A WITHIN LOOP: 1
A WITHIN LOOP: 2
A WITHIN LOOP: 3
A AFTER LOOP:  4
B WITHIN LOOP: 1
B WITHIN LOOP: 2
B WITHIN LOOP: 3
B AFTER LOOP:  0
C WITHIN LOOP: 1
C WITHIN LOOP: 2
C WITHIN LOOP: 3
C AFTER LOOP:  4
D WITHIN LOOP: 1
D WITHIN LOOP: 2
D WITHIN LOOP: 3
D AFTER LOOP:  0
E AFTER LOOP:  3
```

Figure 64. Modifying the loop iteration value

BY Keyword

By default, the program increases the indexing variable by 1 for each iteration. The keyword BY overrides this increment.

- The *increment value* can be a numeric expression and can therefore be non-integer or negative. Zero causes a warning and results in a zero-trip loop.
- If the initial value is greater than the terminal value and the increment is positive, the loop is never entered. `#I=1 TO 0 BY 2` results in a zero-trip loop.
- If the initial value is less than the terminal value and the increment is negative, the loop is never entered. `#I=1 TO 2 BY −1` also results in a zero-trip loop.
- Order is unimportant: `2 BY 2 TO 10` is equivalent to `2 TO 10 BY 2`.

Example

```
LOOP #I=2 TO 10 BY 2. /*Loop five times by 2'S
COMPUTE X=X+1.
END LOOP.
```

- The scratch variable *#I* starts at 2 and increases by 2 for each of five iterations until it equals 10 for the last iteration.

Example

```
LOOP #I=1 TO Y BY Z. /*Loop to Y incrementing by Z
COMPUTE X=X+1.
END LOOP.
```

- The loop is executed once for a case with *Y* equal to 2 and *Z* equal to 2 but twice for a case with *Y* equal to 3 and *Z* equal to 2.

Example

```
* Repeating data using LOOP.

INPUT PROGRAM.
DATA LIST      NOTABLE/ ORDER 1-4(N) #BKINFO 6-71(A).
LEAVE ORDER.
LOOP           #I = 1 TO 66 BY 6 IF SUBSTR(#BKINFO,#I,6) <> ' '.
+  REREAD         COLUMN = #I+5.
+  DATA LIST      NOTABLE/ ISBN 1-3(N) QUANTITY 4-5.
+  END CASE.
END LOOP.
END INPUT PROGRAM.
SORT CASES     BY ISBN ORDER.
BEGIN DATA
1045 182 2 155 1 134 1 153 5
1046 155 3 153 5 163 1
1047 161 5 182 2 163 4 186 6
1048 186 2
1049 155 2 163 2 153 2 074 1 161 1
END DATA.

DO IF          $CASENUM = 1.
+  PRINT EJECT    /'Order' 1  'ISBN' 7  'Quantity' 13.
END IF.
PRINT          /ORDER 2-5(N) ISBN 8-10(N) QUANTITY 13-17.
EXECUTE.
```

- This example uses LOOP to simulate a REPEATING DATA command.
- DATA LIST specifies the scratch variable *#BKINFO* as a string variable (format A) to allow blanks in the data.
- LOOP is executed if the SUBSTR function returns anything other than a blank or null value. SUBSTR returns a six-character substring of *#BKINFO*, beginning with the character in the position specified by the value of the indexing variable *#I*. As specified on the indexing clause, *#I* begins with a value of 1 and is increased by 6 for each iteration of LOOP, up to a maximum *#I* value of 61 (1 + 10 × 6 = 61). The next iteration would exceed the maximum *#I* value (1 + 11 × 6 = 67).

Missing Values

- If the program encounters a case with a missing value for the initial, terminal, or increment value or expression, or if the conditional expression on the LOOP command returns missing, a zero-trip loop results and control is passed to the first command after the END LOOP command.
- If a case has a missing value for the conditional expression on an END LOOP command, the loop is terminated after the first iteration.
- To prevent cases with missing values for any variable used in the loop structure from entering the loop, use the IF clause on the LOOP command (see third example below).

Example

```
LOOP #I=1 TO Z  IF (Y GT 10). /*Loop to X=Z for cases with Y GT 10
COMPUTE X=X+1.
END LOOP.
```

- The value of *X* remains unchanged for cases with a missing value for *Y* or a missing value for *Z* (or if *Z* is less than 1).

Example

```
MISSING VALUES X(5).
LOOP.
COMPUTE X=X+1.
END LOOP IF (X GE 10). /*Loop until X is at least 10 or missing
```

- Looping is terminated when the value of *X* is 5 because 5 is defined as missing for *X*.

Example

```
LOOP IF NOT MISSING(Y). /*Loop only when Y isn't missing
COMPUTE X=X+Y.
END LOOP IF (X GE 10). /*Loop until X is at least 10
```

- The variable *X* is unchanged for cases with a missing value for *Y*, since the loop is never entered.

Creating Data

A loop structure and an END CASE command within an input program can be used to create data without any data input. The END FILE command must be used outside the loop (but within the input program) to terminate processing.

Example

```
INPUT PROGRAM.
LOOP #I=1 TO 20.
COMPUTE AMOUNT=RND(UNIFORM(5000))/100.
END CASE.
END LOOP.
END FILE.
END INPUT PROGRAM.

PRINT FORMATS AMOUNT (DOLLAR6.2).
PRINT /AMOUNT.
EXECUTE.
```

- This example creates 20 cases with a single variable, *AMOUNT*. *AMOUNT* is a uniformly distributed number between 0 and 5,000, rounded to an integer and divided by 100 to provide a variable in dollars and cents.
- The END FILE command is required to terminate processing once the loop structure is complete.

Scratch vs. Permanent Index Variables

Permanent variables are reinitialized to system-missing for each case, but scratch variables are initialized to 0 and are not reinitialized for each case; instead they retain their previous values. For loops that don't span cases, this is not an important factor, but in an input program a nested loop can persist across cases. In such instances, the index counter is not affected because it is cached and restored after execution of the loop, but the results of commands within the loop that use the value of the index variables will be different for scratch and permanent index variables.

```
*using scratch index variables.
INPUT PROGRAM.
LOOP #i=1 to 3.
- LOOP #j=#i to 4.
-  COMPUTE var1=#i.
-  COMPUTE var2=#j.
-  END CASE.
- END LOOP.
END LOOP.
END FILE.
END INPUT PROGRAM.
LIST.

*using non-scratch index variables.
INPUT PROGRAM.
LOOP i=1 to 3.
- LOOP j=i to 4.
-  COMPUTE var1=i.
-  COMPUTE var2=j.
-  END CASE.
- END LOOP.
END LOOP.
END FILE.
END INPUT PROGRAM.
LIST.
```

```
    var1     var2

    1.00     1.00
    1.00     2.00
    1.00     3.00
    1.00     4.00
    2.00     2.00
    2.00     3.00
    2.00     4.00
    3.00     3.00
    3.00     4.00
```

Figure 65. Case listing from loop using scratch variables

```
       i        j     var1     var2

    1.00     1.00     1.00     1.00
     .       2.00      .       2.00
     .       3.00      .       3.00
     .       4.00      .       4.00
    2.00     2.00     2.00     2.00
     .       3.00      .       3.00
     .       4.00      .       4.00
    3.00     3.00     3.00     3.00
     .       4.00      .       4.00
```

Figure 66. Case listing from loop using permanent variables

- Aside from the fact that the use of permanent variables as index variables results in the creation (or replacement) of those variables in the active dataset, note that many of the value for *var1* (and *i*) are missing in the results from the input program that uses permanent variables as index variables, while none are missing from the one that uses scratch variables.
- In both cases, the inner loop ends with the END CASE command, which causes the permanent variable *i* to be reinitialized to system-missing for subsequent iterations of the inner loop, but the scratch variable *#i* retains its previous value.
- When control passes to the outer loop again, the cached index value is used to set and increment *i*; so it has a non-missing value for the first iteration of the inner loop, but once END CASE is encountered, it becomes missing again until control passes to the outer loop again.

See the topic "Scratch Variables" on page 44 for more information.

MANOVA

MANOVA is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
MANOVA dependent varlist [BY factor list (min,max)[factor list...]
                         [WITH covariate list]]

 [/WSFACTORS=varname (levels) [varname...] ]

 [/WSDESIGN]*

 [/TRANSFORM [(dependent varlist [/dependent varlist])]=
                     [ORTHONORM] [{CONTRAST}]  {DEVIATION (refcat)   } ]
                                 {BASIS   }   {DIFFERENCE           }
                                              {HELMERT              }
                                              {SIMPLE (refcat)      }
                                              {REPEATED             }
                                              {POLYNOMIAL [({1,2,3...})]}
                                              {            {metric  }  }
                                              {SPECIAL (matrix)     }

 [/MEASURE=newname newname...]

 [/RENAME={newname} {newname}...]
          {*      } {*      }

 [/ERROR={WITHIN           } ]
         {RESIDUAL         }
         {WITHIN + RESIDUAL}
         {n                }

 [/CONTRAST (factorname)={DEVIATION** [(refcat)]    }] †
                         {POLYNOMIAL**[({1,2,3...})]}
                         {             {metric  }  }
                         {SIMPLE [(refcat)]        }
                         {DIFFERENCE               }
                         {HELMERT                  }
                         {REPEATED                 }
                         {SPECIAL (matrix)         }

 [/PARTITION (factorname)[=({1,1...  })]]
                            {n1,n2...}

 [/METHOD=[{UNIQUE**  }]  [{CONSTANT**}] [{QR**    }]]
          {SEQUENTIAL}    {NOCONSTANT}   {CHOLESKY}

 [/{PRINT  }= [CELLINFO [([MEANS] [SSCP] [COV] [COR] [ALL])]]
   {NOPRINT}  [HOMOGENEITY [([ALL] [BARTLETT] [COCHRAN] [BOXM])]]
              [DESIGN [([OVERALL] [ONEWAY] [DECOMP] [BIAS] [SOLUTION]
                       [REDUNDANCY] [COLLINEARITY] [ALL])]]
              [PARAMETERS [([ESTIM] [ORTHO][COR][NEGSUM][EFSIZE][OPTIMAL][ALL])]]
              [SIGNIF [[(SINGLEDF)]
                       [(MULTIV**)] [(EIGEN)] [(DIMENR)]
                       [(UNIV**)] [(HYPOTH)][(STEPDOWN)] [(BRIEF)]
                       [{(AVERF**)}] [(HF)] [(GG)] [(EFSIZE)]]
                        {(AVONLY) }
             [ERROR[(STDDEV)][(COR)][(COV)][(SSCP)]]

 [/OMEANS =[VARIABLES(varlist)] [TABLES ({factor name     }] ]
                                        {factor BY factor}
                                        {CONSTANT        }

 [/PMEANS =[VARIABLES(varlist)] [TABLES ({factor name     })] [PLOT]] ]
                                        {factor BY factor}
                                        {CONSTANT        }

 [/RESIDUALS=[CASEWISE] [PLOT] ]

 [/POWER=[T({.05**})] [F({.05**})] [{APPROXIMATE}]]
           {a    }      {a    }    {EXACT      }

 [/CINTERVAL=[{INDIVIDUAL}][({.95}) ]
              {JOINT     }   {a  }
             [UNIVARIATE ({SCHEFFE})]
                         {BONFER }
             [MULTIVARIATE  ({ROY      })]  ]
                            {PILLAI   }
                            {BONFER   }
                            {HOTELLING}
                            {WILKS    }
 [/PCOMPS [COR] [COV] [ROTATE(rottype)]
          [NCOMP(n)] [MINEIGEN(eigencut)] [ALL] ]

 [/PLOT=[BOXPLOTS] [CELLPLOTS] [NORMAL]  [ALL] ]

 [/DISCRIM [RAW] [STAN] [ESTIM] [COR] [ALL]
           [ROTATE(rottype)] [ALPHA({.25**})]]
                                    {a    }

 [/MISSING=[LISTWISE**] [{EXCLUDE**}] ]
                         {INCLUDE  }

 [/MATRIX=[IN({file})]  [OUT({file})]]
              {[*] }         {[*] }

 [/ANALYSIS [({UNCONDITIONAL**})]=[()dependent varlist
             {CONDITIONAL    }      [WITH covariate varlist]
                                     [/dependent varlist...][])][WITH varlist] ]

 [/DESIGN={factor [(n)]  }[BY factor[(n)]] [WITHIN factor[(n)]][WITHIN...]
```

```
          {[POOL(varlist)}

          [+ {factor [(n)] }...]
             {POOL(varlist)}

          [[= n] {AGAINST} {WITHIN  }
                 {VS     } {RESIDUAL}
                           {WR       }
                           {n        }

          [{factor [(n)] } ... ]
           {POOL(varlist)}

          [MWITHIN factor(n)]
          [MUPLUS]
          [CONSTANT [=n] ]
```

* WSDESIGN uses the same specification as DESIGN, with only within-subjects factors.

† DEVIATION is the default for between-subjects factors, while POLYNOMIAL is the default for within-subjects factors.

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example 1

```
* Analysis of Variance

MANOVA RESULT BY TREATMNT(1,4) GROUP(1,2).
```

Example 2

```
* Analysis of Covariance

MANOVA RESULT BY TREATMNT(1,4) GROUP(1,2) WITH RAINFALL.
```

Example 3

```
* Repeated Measures Analysis

MANOVA SCORE1 TO SCORE4 BY CLASS(1,2)
  /WSFACTORS=MONTH(4).
```

Example 4

```
* Parallelism Test with Crossed Factors

MANOVA YIELD BY PLOT(1,4) TYPEFERT(1,3) WITH FERT
  /ANALYSIS YIELD
  /DESIGN FERT, PLOT, TYPEFERT, PLOT BY TYPEFERT,
   FERT BY PLOT + FERT BY TYPEFERT
   + FERT BY PLOT BY TYPEFERT.
```

Overview

MANOVA (multivariate analysis of variance) is a generalized procedure for analysis of variance and covariance. MANOVA is a powerful procedure and can be used for both univariate and multivariate designs. MANOVA allows you to perform the following tasks:

- Specify nesting of effects.
- Specify individual error terms for effects in mixed-model analyses.
- Estimate covariate-by-factor interactions to test the assumption of homogeneity of regressions.
- Obtain parameter estimates for a variety of contrast types, including irregularly spaced polynomial contrasts with multiple factors.
- Test user-specified special contrasts with multiple factors.
- Partition effects in models.
- Pool effects in models.

MANOVA and General Linear Model (GLM)

MANOVA is available only in syntax. GLM (general linear model), the other generalized procedure for analysis of variance and covariance, is available both in syntax and via the dialog boxes. The major distinction between GLM and MANOVA in terms of statistical design and functionality is that GLM uses a non-full-rank, or overparameterized, indicator variable approach to parameterization of linear models (instead of the full-rank reparameterization approach that is used in MANOVA). GLM uses a generalized inverse approach and uses the aliasing of redundant parameters to zero to allow greater flexibility in handling a variety of data situations, particularly situations involving empty cells. For features that are provided by GLM but unavailable in MANOVA, refer to "General Linear Model (GLM) and MANOVA " on page 839.

To simplify the presentation, MANOVA reference material is divided into three sections: *univariate* designs with one dependent variable; *multivariate* designs with several interrelated dependent variables; and *repeated measures* designs in which the dependent variables represent the same types of measurements, taken at more than one time.

The full syntax diagram for MANOVA is presented here. The sections that follow include partial syntax diagrams that show the subcommands and specifications that are discussed in that section. Individually, those diagrams are incomplete. Subcommands that are listed for univariate designs are available for any analysis, and subcommands that are listed for multivariate designs can be used in any multivariate analysis, including repeated measures.

MANOVA was designed and programmed by Philip Burns of Northwestern University.

MANOVA: Univariate

MANOVA is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
MANOVA dependent var [BY factor list (min,max)][factor list...]
                     [WITH covariate list]
 [/ERROR={WITHIN           } ]
         {RESIDUAL         }
         {WITHIN + RESIDUAL}
         {n                }
 [/CONTRAST (factor name)={DEVIATION** [(refcat)]    }]
                          {POLYNOMIAL  [({1,2,3...})]}
                          {            {metric  }   }
                          {SIMPLE [(refcat)]        }
                          {DIFFERENCE               }
                          {HELMERT                  }
                          {REPEATED                 }
                          {SPECIAL (matrix)         }
 [/PARTITION (factor name)[=({1,1...  })]]
                            {n1,n2...}
 [/METHOD=[{UNIQUE**  }]  [{CONSTANT**}] [{QR**    }]]
          {SEQUENTIAL}    {NOCONSTANT}   {CHOLESKY}
 [/{PRINT   }= [CELLINFO [([MEANS] [SSCP] [COV] [COR] [ALL])]]
   {NOPRINT}   [HOMOGENEITY [([ALL] [BARTLETT] [COCHRAN])]]
               [DESIGN [([OVERALL] [ONEWAY] [DECOMP] [BIAS] [SOLUTION]
                         [REDUNDANCY] [COLLINEARITY])]]
               [PARAMETERS [([ESTIM][ORTHO][COR][NEGSUM][EFSIZE][OPTIMAL][ALL])]]
               [SIGNIF[(SINGLEDF)]]
               [ERROR[(STDDEV)]]                                              ]
 [/OMEANS =[VARIABLES(varlist)] [TABLES ({factor name     }] ]
                                        {factor BY factor}
                                        {CONSTANT        }
 [/PMEANS =[TABLES ({factor name     })] [PLOT]] ]
                   {factor BY factor}
                   {CONSTANT        }
 [/RESIDUALS=[CASEWISE] [PLOT] ]
 [/POWER=[T({.05**})] [F({.05**})] [{APPROXIMATE}]]
          {a    }     {a    }    {EXACT      }
 [/CINTERVAL=[{INDIVIDUAL}][({.95}) ]] [UNIVARIATE ({SCHEFFE})]
             {JOINT     }   {  a}                  {BONFER }
 [/PLOT=[BOXPLOTS] [CELLPLOTS] [NORMAL]  [ALL] ]
 [/MISSING=[LISTWISE**] [{EXCLUDE**}] ]
                        {INCLUDE  }
 [/MATRIX=[IN({file})]  [OUT({file})]]
          {[*] }        {[*] }
 [/ANALYSIS=dependent var [WITH covariate list]]
 [/DESIGN={factor [(n)]  }[BY factor[(n)]] [WITHIN factor[(n)]][WITHIN...]]
          {[POOL(varlist)}
          [+ {factor [(n)] }...]
             {POOL(varlist)}
          [[= n] {AGAINST} {WITHIN  }
                 {VS     } {RESIDUAL}
                           {WR       }
                           {n        }
          [{factor [(n)] } ... ]
           {POOL(varlist)}
          [MUPLUS]
          [MWITHIN factor(n)]
          [CONSTANT [=n] ]
```

** Default if the subcommand or keyword is omitted.

Example

```
MANOVA YIELD BY SEED(1,4) FERT(1,3)
  /DESIGN.
```

Overview

This section describes the use of MANOVA for univariate analyses. However, the subcommands that are described here can be used in any type of analysis with MANOVA. For additional subcommands that are used in multivariate analysis, see “MANOVA: Multivariate” on page 1043. For additional subcommands that are used in repeated measures analysis, see “MANOVA: Repeated Measures” on page 1053. For basic specification, syntax rules, and limitations of the MANOVA procedures, see “MANOVA” on page 1019.

Options

Design Specification. You can use the DESIGN subcommand to specify which terms to include in the design. This ability allows you to estimate a model other than the default full factorial model, incorporate factor-by-covariate interactions, indicate nesting of effects, and indicate specific error terms for each

effect in mixed models. You can specify a different continuous variable as a dependent variable or work with a subset of the continuous variables with the ANALYSIS subcommand.

Contrast Types. You can specify contrasts other than the default deviation contrasts on the CONTRAST subcommand. You can also subdivide the degrees of freedom associated with a factor (using the PARTITION subcommand) and test the significance of a specific contrast or group of contrasts.

Optional Output. You can choose from a variety of optional output on the PRINT subcommand or suppress output using the NOPRINT subcommand. Output that is appropriate to univariate designs includes cell means, design or other matrices, parameter estimates, and tests for homogeneity of variance across cells. Using the OMEANS, PMEANS, RESIDUAL, and PLOT subcommands, you can also request tables of observed and/or predicted means, casewise values and residuals for your model, and various plots that are useful in checking assumptions. In addition, you can use the POWER subcommand to request observed power values (based on fixed-effect assumptions), and you can use the CINTERVAL subcommand to request simultaneous confidence intervals for each parameter estimate and regression coefficient.

Matrix Materials. You can write matrices of intermediate results to a matrix data file, and you can read such matrices in performing further analyses by using the MATRIX subcommand.

Basic Specification

- The basic specification is a variable list that identifies the dependent variable, the factors (if any), and the covariates (if any).
- By default, MANOVA uses a full factorial model, which includes all main effects and all possible interactions among factors. Estimation is performed by using the cell-means model and UNIQUE (regression-type) sums of squares, adjusting each effect for all other effects in the model. Parameters are estimated by using DEVIATION contrasts to determine whether their categories differ significantly from the mean.

Subcommand Order

- The variable list must be specified first.
- Subcommands that are applicable to a specific design must be specified before that DESIGN subcommand. Otherwise, subcommands can be used in any order.

Syntax Rules

- For many analyses, the MANOVA variable list and the DESIGN subcommand are the only specifications that are needed. If a full factorial design is desired, DESIGN can be omitted.
- All other subcommands apply only to designs that follow. If you do not enter a DESIGN subcommand, or if the last subcommand is not DESIGN, MANOVA uses a full factorial model.
- Unless replaced, MANOVA subcommands (other than DESIGN) remain in effect for all subsequent models.
- MISSING can be specified only once.
- The following words are reserved as keywords or internal commands in the MANOVA procedure: AGAINST, CONSPLUS, CONSTANT, CONTIN, MUPLUS, MWITHIN, POOL, R, RESIDUAL, RW, VERSUS, VS, W, WITHIN, and WR. Variable names that duplicate these words should be changed before you invoke MANOVA.
- If you enter one of the multivariate specifications in a univariate analysis, MANOVA will ignore it.

Limitations

- A maximum of 20 factors is in place.
- A maximum of 200 dependent variables is in place.
- Memory requirements depend primarily on the number of cells in the design. For the default full factorial model, the number of cells equals the product of the number of levels or categories in each factor.

Example

```
MANOVA YIELD BY SEED(1,4) FERT(1,3) WITH RAINFALL
  /PRINT=CELLINFO(MEANS) PARAMETERS(ESTIM)
  /DESIGN.
```

- *YIELD* is the dependent variable; *SEED* (with values 1, 2, 3, and 4) and *FERT* (with values 1, 2, and 3) are factors; *RAINFALL* is a covariate.
- The PRINT subcommand requests the means of the dependent variable for each cell and the default deviation parameter estimates.
- The DESIGN subcommand requests the default design, a full factorial model. This subcommand could have been omitted or could have been specified in full as:

  ```
  /DESIGN = SEED, FERT, SEED BY FERT.
  ```

MANOVA Variable List

The variable list specifies all variables that will be used in any subsequent analyses.

- The dependent variable must be the first specification on MANOVA.
- By default, MANOVA treats a list of dependent variables as jointly dependent, implying a multivariate design. However, you can use the ANALYSIS subcommand to change the role of a variable or its inclusion status in the analysis.
- The names of the factors follow the dependent variable. Use the keyword BY to separate the factors from the dependent variable.
- Factors must have adjacent integer values, and you must supply the minimum and maximum values in parentheses after the factor name(s).
- If several factors have the same value range, you can specify a list of factors followed by a single value range in parentheses.
- Certain one-cell designs, such as univariate and multivariate regression analysis, canonical correlation, and one-sample Hotelling's T^2, do not require a factor specification. To perform these analyses, omit the keyword BY and the factor list.
- Enter the covariates, if any, following the factors and their ranges. Use the keyword WITH to separate covariates from factors (if any) and the dependent variable.

Example

```
MANOVA DEPENDNT BY FACTOR1 (1,3) FACTOR2, FACTOR3 (1,2).
```

- In this example, three factors are specified.
- *FACTOR1* has values 1, 2, and 3, while *FACTOR2* and *FACTOR3* have values 1 and 2.
- A default full factorial model is used for the analysis.

Example

```
MANOVA Y BY A(1,3) WITH X
  /DESIGN.
```

- In this example, the *A* effect is tested after adjusting for the effect of the covariate *X*. It is a test of equality of adjusted *A* means.
- The test of the covariate *X* is adjusted for *A*. The test is a test of the pooled within-groups regression of *Y* on *X*.

ERROR Subcommand

ERROR allows you to specify or change the error term that is used to test all effects for which you do not explicitly specify an error term on the DESIGN subcommand. ERROR affects all terms in all subsequent designs, except terms for which you explicitly provide an error term.

WITHIN. *Terms in the model are tested against the within-cell sum of squares.* This specification can be abbreviated to W. This setting is the default unless there is no variance within cells or a continuous variable is named on the DESIGN subcommand.

RESIDUAL. *Terms in the model are tested against the residual sum of squares.* This specification can be abbreviated to R. This specification includes all terms not named on the DESIGN subcommand.

WITHIN+RESIDUAL. *Terms are tested against the pooled within-cells and residual sum of squares.* This specification can be abbreviated to WR or RW. This setting is the default for designs in which a continuous variable appears on the DESIGN subcommand.

error number. *Terms are tested against a numbered error term.* The error term must be defined on each DESIGN subcommand. For a discussion of error terms, see "DESIGN Keyword " on page 1031 .

- If you specify ERROR=WITHIN+RESIDUAL and one of the components does not exist, MANOVA uses the other component alone.
- If you specify your own error term by number and a design does not have an error term with the specified number, MANOVA does not carry out significance tests. MANOVA will, however, display hypothesis sums of squares and, if requested, parameter estimates.

Example

```
MANOVA DEP BY A(1,2) B(1,4)
  /ERROR = 1
  /DESIGN = A, B, A BY B = 1 VS WITHIN
  /DESIGN = A, B.
```

- ERROR defines error term 1 as the default error term.
- In the first design, *A* by *B* is defined as error term 1 and is therefore used to test the *A* and *B* effects. The *A* by *B* effect itself is explicitly tested against the within-cells error.
- In the second design, no term is defined as error term 1, so no significance tests are carried out. Hypothesis sums of squares are displayed for *A* and *B*.

CONTRAST Subcommand

CONTRAST specifies the type of contrast that is desired among the levels of a factor. For a factor with *k* levels or values, the contrast type determines the meaning of its *k*–1 degrees of freedom. If the subcommand is omitted or is specified with no keyword, the default is DEVIATION for between-subjects factors.

- Specify the factor name in parentheses following the subcommand CONTRAST.
- You can specify only one factor per CONTRAST subcommand, but you can enter multiple CONTRAST subcommands.
- After closing the parentheses, enter an equals sign followed by one of the contrast keywords.
- To obtain *F* tests for individual degrees of freedom for the specified contrast, enter the factor name followed by a number in parentheses on the DESIGN subcommand. The number refers to a partition of the factor's degrees of freedom. If you do not use the PARTITION subcommand, each degree of freedom is a distinct partition.

The following contrast types are available:

DEVIATION. *Deviations from the grand mean.* This setting is the default for between-subjects factors. Each level of the factor (except one level) is compared to the grand mean. One category (by default, the last category) must be omitted so that the effects will be independent of one another. To omit a category

other than the last category, specify the number of the omitted category (which is not necessarily the same as its value) in parentheses after the keyword DEVIATION. For example:

```
MANOVA A BY B(2,4)
  /CONTRAST(B)=DEVIATION(1).
```

The specified contrast omits the first category, in which *B* has the value 2. Deviation contrasts are not orthogonal.

POLYNOMIAL. *Polynomial contrasts.* This setting is the default for within-subjects factors. The first degree of freedom contains the linear effect across the levels of the factor, the second degree of freedom contains the quadratic effect, and so on. In a balanced design, polynomial contrasts are orthogonal. By default, the levels are assumed to be equally spaced; you can specify unequal spacing by entering a metric—consisting of one integer for each level of the factor—in parentheses after the keyword POLYNOMIAL. For example:

```
MANOVA RESPONSE BY STIMULUS (4,6)
  /CONTRAST(STIMULUS) = POLYNOMIAL(1,2,4)
```

The specified contrast indicates that the three levels of *STIMULUS* are actually in the proportion 1:2:4. The default metric is always (1,2,...,*k*), where *k* levels are involved. Only the relative differences between the terms of the metric matter. (1,2,4) is the same metric as (2,3,5) or (20,30,50) because, in each instance, the difference between the second and third numbers is twice the difference between the first and second numbers.

DIFFERENCE. *Difference or reverse Helmert contrasts.* Each level of the factor (except the first level) is compared to the mean of the previous levels. In a balanced design, difference contrasts are orthogonal.

HELMERT. *Helmert contrasts.* Each level of the factor (except the last level) is compared to the mean of subsequent levels. In a balanced design, Helmert contrasts are orthogonal.

SIMPLE. *Contrast where each level of the factor (except the last level) is compared to the last level.* To use a category (other than the last category) as the omitted reference category, specify its number (which is not necessarily the same as its value) in parentheses following the keyword SIMPLE. For example:

```
MANOVA A BY B(2,4)
  /CONTRAST(B)=SIMPLE(1).
```

The specified contrast compares the other levels to the first level of *B*, in which *B* has the value 2. Simple contrasts are not orthogonal.

REPEATED. *Comparison of adjacent levels.* Each level of the factor (except the last level) is compared to the next level. Repeated contrasts are not orthogonal.

SPECIAL. *A user-defined contrast.* After this keyword, enter a square matrix in parentheses with as many rows and columns as there are levels in the factor. The first row represents the mean effect of the factor and is generally a vector of 1's. The row represents a set of weights indicating how to collapse over the categories of this factor in estimating parameters for other factors. The other rows of the contrast matrix contain the special contrasts indicating the desired comparisons between levels of the factor. If the special contrasts are linear combinations of each other, MANOVA reports the linear dependency and stops processing.

Orthogonal contrasts are particularly useful. In a balanced design, contrasts are orthogonal if the sum of the coefficients in each contrast row is 0 and if, for any pair of contrast rows, the products of corresponding coefficients sum to 0. DIFFERENCE, HELMERT, and POLYNOMIAL contrasts always meet these criteria in balanced designs.

Example

```
MANOVA DEP BY FAC(1,5)
  /CONTRAST(FAC)=DIFFERENCE
  /DESIGN=FAC(1) FAC(2) FAC(3) FAC(4).
```

- The factor *FAC* has five categories and therefore four degrees of freedom.

- CONTRAST requests DIFFERENCE contrasts, which compare each level (except the first level) with the mean of the previous levels.
- Each of the four degrees of freedom is tested individually on the DESIGN subcommand.

PARTITION Subcommand

PARTITION subdivides the degrees of freedom that are associated with a factor. This process permits you to test the significance of the effect of a specific contrast or group of contrasts of the factor instead of the overall effect of all contrasts of the factor. The default is a single degree of freedom for each partition.

- Specify the factor name in parentheses following the PARTITION subcommand.
- Specify an integer list in parentheses after the optional equals sign to indicate the degrees of freedom for each partition.
- Each value in the partition list must be a positive integer, and the sum of the values cannot exceed the degrees of freedom for the factor.
- The degrees of freedom that are available for a factor are one less than the number of levels of the factor.
- The meaning of each degree of freedom depends on the contrast type for the factor. For example, with deviation contrasts (the default for between-subjects factors), each degree of freedom represents the deviation of the dependent variable in one level of the factor from its grand mean over all levels. With polynomial contrasts, the degrees of freedom represent the linear effect, the quadratic effect, and so on.
- If your list does not account for all the degrees of freedom, MANOVA adds one final partition containing the remaining degrees of freedom.
- You can use a repetition factor of the form *n** to specify a series of partitions with the same number of degrees of freedom.
- To specify a model that tests only the effect of a specific partition of a factor in your design, include the number of the partition in parentheses on the DESIGN subcommand (see the example below).
- If you want the default single degree-of-freedom partition, you can omit the PARTITION subcommand and simply enter the appropriate term on the DESIGN subcommand.

Example

```
MANOVA OUTCOME BY TREATMNT(1,12)
  /PARTITION(TREATMNT) = (3*2,4)
  /DESIGN TREATMNT(2).
```

- The factor *TREATMNT* has 12 categories (hence, 11 degrees of freedom).
- PARTITION divides the effect of *TREATMNT* into four partitions, containing, respectively, 2, 2, 2, and 4 degrees of freedom. A fifth partition is formed to contain the remaining 1 degree of freedom.
- DESIGN specifies a model in which only the second partition of *TREATMNT* is tested. This partition contains the third and fourth degrees of freedom.
- Because the default contrast type for between-subjects factors is DEVIATION, this second partition represents the deviation of the third and fourth levels of *TREATMNT* from the grand mean.

METHOD Subcommand

METHOD controls the computational aspects of the MANOVA analysis. You can specify one of two different methods for partitioning the sums of squares. The default is UNIQUE.

UNIQUE. *Regression approach.* Each term is corrected for every other term in the model. With this approach, sums of squares for various components of the model do not add up to the total sum of squares unless the design is balanced. This is the default if the METHOD subcommand is omitted or if neither of the two keywords is specified.

SEQUENTIAL. *Hierarchical decomposition of the sums of squares.* Each term is adjusted only for the terms that precede it on the DESIGN subcommand. This decomposition is an orthogonal decomposition, and the sums of squares in the model add up to the total sum of squares.

You can control how parameters are to be estimated by specifying one of the following two keywords that are available on MANOVA. The default is QR.

QR. *Use modified Givens rotations.* QR bypasses the normal equations and the inaccuracies that can result from creating the cross-products matrix, and it generally results in extremely accurate parameter estimates. This setting is the default if the METHOD subcommand is omitted or if neither of the two keywords is specified.

CHOLESKY. *Use Cholesky decomposition of the cross-products matrix.* This method is useful for large data sets with covariates entered on the DESIGN subcommand.

You can also control whether a constant term is included in all models. Two keywords are available on METHOD. The default is CONSTANT.

CONSTANT. *All models include a constant (grand mean) term, even if none is explicitly specified on the DESIGN subcommand.* This setting is the default if neither of the two keywords is specified.

NOCONSTANT. *Exclude constant terms from models that do not include the keyword CONSTANT on the DESIGN subcommand.*

Example

```
MANOVA DEP BY A B C (1,4)
  /METHOD=NOCONSTANT
  /DESIGN=A, B, C
  /METHOD=CONSTANT SEQUENTIAL
  /DESIGN.
```

- For the first design, a main-effects model, the METHOD subcommand requests the model to be fitted with no constant.
- The second design requests a full factorial model to be fitted with a constant and with a sequential decomposition of sums of squares.

PRINT and NOPRINT Subcommands

PRINT and NOPRINT control the display of optional output.

- Specifications on PRINT remain in effect for all subsequent designs.
- Some PRINT output, such as CELLINFO, applies to the entire MANOVA procedure and is displayed only once.
- You can turn off optional output that you have requested on PRINT by entering a NOPRINT subcommand with the specifications that were originally used on the PRINT subcommand.
- Additional output can be obtained on the PCOMPS, DISCRIM, OMEANS, PMEANS, PLOT, and RESIDUALS subcommands.
- Some optional output greatly increases the processing time. Request only the output that you want to see.

The following specifications are appropriate for univariate MANOVA analyses. For information about PRINT specifications that are appropriate for multivariate models, see “PRINT and NOPRINT Subcommands ” on page 1047. For information about PRINT specifications that are appropriate for repeated measures models, see “PRINT Subcommand ” on page 1059.

CELLINFO. *Basic information about each cell in the design.*

PARAMETERS. *Parameter estimates.*

HOMOGENEITY. *Tests of homogeneity of variance.*

DESIGN. *Design information.*

ERROR. *Error standard deviations.*

CELLINFO Keyword

You can request any of the following cell information by specifying the appropriate keyword(s) in parentheses after CELLINFO. The default is MEANS.

MEANS. *Cell means, standard deviations, and counts for the dependent variable and covariates.* Confidence intervals for the cell means are displayed if you have set a wide width. This setting is the default when CELLINFO is requested with no further specification.

SSCP. *Within-cell sum-of-squares and cross-products matrices for the dependent variable and covariates.*

COV. *Within-cell variance-covariance matrices for the dependent variable and covariates.*

COR. *Within-cell correlation matrices, with standard deviations on the diagonal, for the dependent variable and covariates.*

ALL. *MEANS, SSCP, COV, and COR.*

- Output from CELLINFO is displayed once before the analysis of any particular design. Specify CELLINFO only once.
- When you specify SSCP, COV, or COR, the cells are numbered for identification, beginning with cell 1.
- The levels vary most rapidly for the factor named last on the MANOVA variables specification.
- Empty cells are neither displayed nor numbered.
- At the beginning of MANOVA output, a table is displayed, showing the levels of each factor corresponding to each cell number.

Example

```
MANOVA DEP BY A(1,4) B(1,2) WITH COV
  /PRINT=CELLINFO(MEANS COV)
  /DESIGN.
```

- For each combination of levels of *A* and *B*, MANOVA displays separately the means and standard deviations of *DEP* and *COV*. Beginning with cell 1, MANOVA will then display the variance-covariance matrix of *DEP* and *COV* within each non-empty cell.
- A table of cell numbers will be displayed to show the factor levels corresponding to each cell.
- The keyword COV, as a parameter of CELLINFO, is not confused with the variable *COV*.

PARAMETERS Keyword

The keyword PARAMETERS displays information about the estimated size of the effects in the model. You can specify any of the following keywords in parentheses on PARAMETERS. The default is ESTIM.

ESTIM. *The estimated parameters themselves, along with their standard errors,* t *tests, and confidence intervals.* Only nonredundant parameters are displayed. This setting is the default if PARAMETERS is requested without further specification.

NEGSUM. *The negative of the sum of parameters for each effect.* For DEVIATION main effects, this value equals the parameter for the omitted (redundant) contrast. NEGSUM is displayed, along with the parameter estimates.

ORTHO. *The orthogonal estimates of parameters that are used to produce the sums of squares.*

COR. *Covariance factors and correlations among the parameter estimates.*

EFSIZE. *The effect size values.*

OPTIMAL. *Optimal Scheffé contrast coefficients.*

ALL. *ESTIM, NEGSUM, ORTHO, COR, EFSIZE, and OPTIMAL.*

SIGNIF Keyword

SIGNIF requests special significance tests, most of which apply to multivariate designs (see "SIGNIF Keyword " on page 1048). The following specification is useful in univariate applications of MANOVA:

SINGLEDF. *Significance tests for each single degree of freedom making up each effect for analysis-of-variance tables.*

- When non-orthogonal contrasts are requested or when the design is unbalanced, the SINGLEDF effects will differ from single degree-of-freedom partitions. SINGLEDEF effects are orthogonal within an effect; single degree-of-freedom partitions are not orthogonal within an effect.

Example

```
MANOVA DEP BY FAC(1,5)
  /CONTRAST(FAC)=POLY
  /PRINT=SIGNIF(SINGLEDF)
  /DESIGN.
```

- POLYNOMIAL contrasts are applied to *FAC*, testing the linear, quadratic, cubic, and quartic components of its five levels. POLYNOMIAL contrasts are orthogonal in balanced designs.
- The SINGLEDF specification on SIGNIF requests significance tests for each of these four components.

HOMOGENEITY Keyword

HOMOGENEITY requests tests for the homogeneity of variance of the dependent variable across the cells of the design. You can specify one or more of the following specifications in parentheses. If HOMOGENEITY is requested without further specification, the default is ALL.

BARTLETT. *Bartlett-Box* F *test.*

COCHRAN. *Cochran's* C.

ALL. *Both BARTLETT and COCHRAN.* This setting is the default.

DESIGN Keyword

You can enter one or more of the following specifications in parentheses following the keyword DESIGN. If DESIGN is requested without further specification, the default is OVERALL.

The DECOMP and BIAS matrices can provide valuable information about the confounding of the effects and the estimability of the chosen contrasts. If two effects are confounded, the entry corresponding to them in the BIAS matrix will be nonzero; if the effects are orthogonal, the entry will be zero. This feature is particularly useful in designs with unpatterned empty cells. For further discussion of the matrices, see Bock (1985).

OVERALL. *The overall reduced-model design matrix (not the contrast matrix).* This setting is the default.

ONEWAY. *The one-way basis matrix (not the contrast matrix) for each factor.*

DECOMP. *The upper triangular QR/CHOLESKY decomposition of the design.*

BIAS. *Contamination coefficients displaying the bias that is present in the design.*

SOLUTION. *Coefficients of the linear combinations of the cell means that are used in significance testing.*

REDUNDANCY. *Exact linear combinations of parameters that form a redundancy.* This keyword displays a table only if QR (the default) is the estimation method.

COLLINEARITY. *Collinearity diagnostics for design matrices.* These diagnostics include the singular values of the normalized design matrix (which are the same as those values of the normalized decomposition matrix), condition indexes corresponding to each singular value, and the proportion of variance of the corresponding parameter accounted for by each principal component. For greatest accuracy, use the QR method of estimation whenever you request collinearity diagnostics.

ALL. *All available options.*

ERROR Keyword

Generally, the keyword ERROR on PRINT produces error matrices. In univariate analyses, the only valid specification for ERROR is STDDEV, which is the default if ERROR is specified by itself.

STDDEV. *The error standard deviation.* Normally, this deviation is the within-cells standard deviation of the dependent variable. If you specify multiple error terms on DESIGN, this specification will display the standard deviation for each term.

OMEANS Subcommand

OMEANS (observed means) displays tables of the means of continuous variables for levels or combinations of levels of the factors.

- Use the keywords VARIABLES and TABLES to indicate which observed means you want to display.
- With no specifications, the OMEANS subcommand is equivalent to requesting CELLINFO (MEANS) on PRINT.
- OMEANS displays confidence intervals for the cell means if you have set the width to 132.
- Output from OMEANS is displayed once before the analysis of any particular design. This subcommand should be specified only once.

VARIABLES. *Continuous variables for which you want means.* Specify the variables in parentheses after the keyword VARIABLES. You can request means for the dependent variable or any covariates. If you omit the VARIABLES keyword, observed means are displayed for the dependent variable and all covariates. If you enter the keyword VARIABLES, you must also enter the keyword TABLES (discussed below).

TABLES. *Factors for which you want the observed means displayed.* In parentheses, list the factors, or combinations of factors, separated with BY. Observed means are displayed for each level, or combination of levels, of the factors that are named (see the example below). Both weighted means and unweighted means (where all cells are weighted equally, regardless of the number of cases that they contain) are displayed. If you enter the keyword CONSTANT, the grand mean is displayed.

Example

```
MANOVA DEP BY A(1,3) B(1,2)
  /OMEANS=TABLES(A,B)
  /DESIGN.
```

- Because there is no VARIABLES specification on the OMEANS subcommand, observed means are displayed for all continuous variables. *DEP* is the only dependent variable here, and there are no covariates.
- The TABLES specification on the OMEANS subcommand requests tables of observed means for each of the three categories of *A* (collapsing over *B*) and for both categories of *B* (collapsing over *A*).
- MANOVA displays weighted means, in which all cases count equally, and displays unweighted means, in which all cells count equally.

PMEANS Subcommand

PMEANS (predicted means) displays a table of the predicted cell means of the dependent variable, adjusted for the effect of covariates in the cell and unadjusted for covariates. For comparison, PMEANS also displays the observed cell means.

- Output from PMEANS can be computationally expensive.
- PMEANS without any additional specifications displays a table showing, for each cell, the observed mean of the dependent variable, the predicted mean adjusted for the effect of covariates in that cell (*ADJ. MEAN*), the predicted mean unadjusted for covariates (*EST. MEAN*), and the raw and standardized residuals from the estimated means.

- Cells are numbered in output from PMEANS so that the levels vary most rapidly on the factor that is named last in the MANOVA variables specification. A table showing the levels of each factor corresponding to each cell number is displayed at the beginning of the MANOVA output.
- Predicted means are suppressed for any design in which the MUPLUS keyword appears.
- Covariates are not predicted.
- In designs with covariates and multiple error terms, use the ERROR subcommand to designate which error term's regression coefficients are to be used in calculating the standardized residuals.

For univariate analysis, the following keywords are available on the PMEANS subcommand:

TABLES. *Additional tables showing adjusted predicted means for specified factors or combinations of factors.* Enter the names of factors or combinations of factors in parentheses after this keyword. For each factor or combination, MANOVA displays the predicted means (adjusted for covariates) collapsed over all other factors.

PLOT. *A plot of the predicted means for each cell.*

Example

```
MANOVA DEP BY A(1,4) B(1,3)
  /PMEANS TABLES(A, B, A BY B)
  /DESIGN = A, B.
```

- PMEANS displays the default table of observed and predicted means for *DEP* and raw and standardized residuals in each of the 12 cells in the model.
- The TABLES specification on PMEANS displays tables of predicted means for *A* (collapsing over *B*), for *B* (collapsing over *A*), and for all combinations of *A* and *B*.
- Because *A* and *B* are the only factors in the model, the means for *A* by *B* in the TABLES specification come from every cell in the model. The means are identical to the adjusted predicted means in the default PMEANS table, which always includes all non-empty cells.
- Predicted means for *A* by *B* can be requested in the TABLES specification, even though the *A* by *B* effect is not in the design.

RESIDUALS Subcommand

Use RESIDUALS to display and plot casewise values and residuals for your models.

- Use the ERROR subcommand to specify an error term other than the default to be used to standardize the residuals.
- If a designated error term does not exist for a given design, no predicted values or residuals are calculated.
- If you specify RESIDUALS without any keyword, CASEWISE output is displayed.

The following keywords are available:

CASEWISE. *A case-by-case listing of the observed, predicted, residual, and standardized residual values for each dependent variable.*

PLOT. *A plot of observed values, predicted values, and case numbers versus the standardized residuals, plus normal and detrended normal probability plots for the standardized residuals (five plots in all).*

POWER Subcommand

POWER requests observed power values based on fixed-effect assumptions for all univariate and multivariate *F* tests and *t* tests. Both approximate and exact power values can be computed, although exact multivariate power is displayed only when there is one hypothesis degree of freedom. If POWER is specified by itself, with no keywords, MANOVA calculates the approximate observed power values of all *F* tests at 0.05 significance level.

The following keywords are available on the POWER subcommand:

APPROXIMATE. *Approximate power values.* This setting is the default if POWER is specified without any keyword. Approximate power values for univariate tests are derived from an Edgeworth-type normal approximation to the noncentral beta distribution. Approximate values are normally accurate to three decimal places and are much cheaper to compute than exact values.

EXACT. *Exact power values.* Exact power values for univariate tests are computed from the noncentral incomplete beta distribution.

F(a). *Alpha level at which the power is to be calculated for* F *tests*. The default is 0.05. To change the default, specify a decimal number between 0 and 1 in parentheses after *F*. The numbers 0 and 1 themselves are not allowed. *F* test at 0.05 significance level is the default when POWER is omitted or specified without any keyword.

T(a). *Alpha level at which the power is to be calculated for* t *tests.* The default is 0.05. To change the default, specify a decimal number between 0 and 1 in parentheses after *t*. The numbers 0 and 1 themselves are not allowed.

- For univariate *F* tests and *t* tests, MANOVA computes a measure of the effect size based on partial η^2:

  partial $\eta^2 = (ssh)/(ssh + sse)$

  where *ssh* is the hypothesis sum of squares and *sse* is the error sum of squares. The measure is an overestimate of the actual effect size. However, the measure is consistent and is applicable to all *F* tests and *t* tests. For a discussion of effect size measures, see [40] or [41].

CINTERVAL Subcommand

CINTERVAL requests simultaneous confidence intervals for each parameter estimate and regression coefficient. MANOVA provides either individual or joint confidence intervals at any desired confidence level. You can compute joint confidence intervals that are using either Scheffé or Bonferroni intervals. Scheffé intervals are based on all possible contrasts, while Bonferroni intervals are based on the number of contrasts that are actually made. For a large number of contrasts, Bonferroni intervals will be larger than Scheffé intervals. Timm [42] provides a good discussion of which intervals are best for certain situations. Both Scheffé and Bonferroni intervals are computed separately for each term in the design. You can request only one type of confidence interval per design.

The following keywords are available on the CINTERVAL subcommand. If the subcommand is specified without any keyword, CINTERVAL automatically displays individual univariate confidence intervals at the 0.95 level.

INDIVIDUAL(a). *Individual confidence intervals.* Specify the desired confidence level in parentheses following the keyword. The desired confidence level can be any decimal number between 0 and 1. When individual intervals are requested, BONFER and SCHEFFE have no effect.

JOINT(a). *Joint confidence intervals.* Specify the desired confidence level in parentheses after the keyword. The default is 0.95. The desired confidence level can be any decimal number between 0 and 1.

UNIVARIATE(type). *Univariate confidence interval.* Specify either SCHEFFE (for Scheffé intervals) or BONFER (for Bonferroni intervals) in parentheses after the keyword. The default specification is SCHEFFE.

[40] Cohen, J. 1977. *Statistical power analysis for the behavioral sciences*. San Diego, California: Academic Press.
[41] Hays, W. L. 1981. *Statistics*, 3rd ed. New York: Holt, Rinehart, and Winston.
[42] Timm, N. H. 1975. *Multivariate statistics: With applications in education and psychology*. Monterey, California: Brooks/Cole.

PLOT Subcommand

MANOVA can display a variety of plots that are useful in checking the assumptions that are needed in the analysis. Plots are produced only once in the MANOVA procedure, regardless of how many DESIGN subcommands you enter. Use the following keywords on the PLOT subcommand to request plots. If the PLOT subcommand is specified by itself, the default is BOXPLOTS.

BOXPLOTS. *Boxplots.* Plots are displayed for each continuous variable (dependent or covariate) that is named on the MANOVA variable list. Boxplots provide a simple graphical means of comparing the cells in terms of mean location and spread. The data must be stored in memory for these plots; if there is not enough memory, boxplots are not produced, and a warning message is issued. This setting is the default if the PLOT subcommand is specified without a keyword.

CELLPLOTS. *Cell statistics, including a plot of cell means versus cell variances, a plot of cell means versus cell standard deviations, and a histogram of cell means.* Plots are produced for each continuous variable (dependent or covariate) that is named on the MANOVA variable list. The first two plots aid in detecting heteroscedasticity (nonhomogeneous variances) and aid in determining an appropriate data transformation (if a transformation is needed). The third plot gives distributional information for the cell means.

NORMAL. *Normal and detrended normal plots.* Plots are produced for each continuous variable (dependent or covariate) that is named on the MANOVA variable list. MANOVA ranks the scores and then plots the ranks against the expected normal deviate, or detrended expected normal deviate, for that rank. These plots aid in detecting non-normality and outlying observations. All data must be held in memory to compute ranks. If not enough memory is available, MANOVA displays a warning and skips the plots.

- ZCORR, an additional plot that is available on the PLOT subcommand, is described in “PLOT Subcommand ” on page 1049.
- You can request other plots on PMEANS and RESIDUALS (see these respective subcommands).

MISSING Subcommand

By default, cases with missing values for any of the variables on the MANOVA variable list are excluded from the analysis. The MISSING subcommand allows you to include cases with user-missing values. If MISSING is not specified, the defaults are LISTWISE and EXCLUDE.

- The same missing-value treatment is used to process all designs in a single execution of MANOVA.
- If you enter more than one MISSING subcommand, the last subcommand that was entered will be in effect for the entire procedure, including designs that were specified before the last MISSING subcommand.
- Pairwise deletion of missing data is not available in MANOVA.
- Keywords INCLUDE and EXCLUDE are mutually exclusive; either keyword can be specified with LISTWISE.

LISTWISE. *Cases with missing values for any variable that is named on the MANOVA variable list are excluded from the analysis.* This process is always true in the MANOVA procedure.

EXCLUDE. *Both user-missing and system-missing values are excluded.* This setting is the default when MISSING is not specified.

INCLUDE. *User-missing values are treated as valid.* For factors, you must include the missing-value codes within the range that is specified on the MANOVA variable list. It may be necessary to recode these values so that they will be adjacent to the other factor values. System-missing values cannot be included in the analysis.

MATRIX Subcommand

MATRIX reads and writes matrix data files. MATRIX writes correlation matrices that can be read by subsequent MANOVA procedures.

- Either IN or OUT is required to specify the matrix file in parentheses. When both IN and OUT are used on the same MANOVA procedure, they can be specified on separate MATRIX subcommands or on the same subcommand.
- The matrix materials include the *N*, mean, and standard deviation. Documents from the file that form the matrix are not included in the matrix data file.
- MATRIX=IN cannot be used in place of GET or DATA LIST to begin a new command syntax file. MATRIX is a subcommand on MANOVA, and MANOVA cannot run before an active dataset is defined. To begin a new command file and immediately read a matrix, first use GET to retrieve the matrix file, and then specify IN(*) on MATRIX.
- Records in the matrix data file that is read by MANOVA can be in any order, with the following exceptions: The order of split-file groups cannot be violated, and all *CORR* vectors must appear contiguously within each split-file group.
- When MANOVA reads matrix materials, it ignores the record containing the total number of cases. In addition, MANOVA skips unrecognized records. MANOVA does not issue a warning when it skips records.

The following two keywords are available on the MATRIX subcommand:

OUT. *Write a matrix data file.* Specify either a file or an asterisk, and enclose the specification in parentheses. If you specify a file, the file is stored on disk and can be retrieved at any time. If you specify an asterisk (*) or leave the parentheses empty, the matrix file replaces the active dataset but is not stored on disk unless you use SAVE or XSAVE.

IN. *Read a matrix data file.* If the matrix file *is not* the current active dataset, specify a file in parentheses. If the matrix file *is* the current active dataset, specify an asterisk (*) or leave the parentheses empty.

Format of the Matrix Data File

The matrix data file includes two special variables: *ROWTYPE_* and *VARNAME_*.

- Variable *ROWTYPE_* is a short string variable having values *N*, *MEAN*, *CORR* (for Pearson correlation coefficients), and *STDDEV*.
- Variable *VARNAME_* is a short string variable whose values are the names of the variables and covariates that are used to form the correlation matrix. When *ROWTYPE_* is *CORR*, *VARNAME_* gives the variable that is associated with that row of the correlation matrix.
- Between *ROWTYPE_* and *VARNAME_* are the factor variables (if any) that are defined in the BY portion of the MANOVA variable list. (Factor variables receive the system-missing value on vectors that represent pooled values.)
- Remaining variables are the variables that are used to form the correlation matrix.

Split Files and Variable Order

- When split-file processing is in effect, the first variables in the matrix system file will be the split variables, followed by *ROWTYPE_*, the factor variable(s), *VARNAME_*, and then the variables that are used to form the correlation matrix.
- A full set of matrix materials is written for each subgroup that is defined by the split variable(s).
- A split variable cannot have the same variable name as any other variable that is written to the matrix data file.
- If a split file is in effect when a matrix is written, the same split file must be in effect when that matrix is read into another procedure.

Additional Statistics

In addition to the *CORR* values, MANOVA always includes the following with the matrix materials:

- The total weighted number of cases used to compute each correlation coefficient.
- A vector of *N*'s for each cell in the data.
- A vector of *MEAN*'s for each cell in the data.
- A vector of pooled standard deviations, *STDDEV*, which is the square root of the within-cells mean square error for each variable.

Example

```
GET FILE IRIS.
MANOVA SEPALLEN SEPALWID PETALLEN PETALWID BY TYPE(1,3)
  /MATRIX=OUT(MANMTX).
```

- MANOVA reads data from the data file *IRIS* and writes one set of matrix materials to the file *MANMTX.*
- The active dataset is still *IRIS.* Subsequent commands are executed on the file *IRIS.*

Example

```
GET FILE IRIS.
MANOVA SEPALLEN SEPALWID PETALLEN PETALWID BY TYPE(1,3)
  /MATRIX=OUT(*).
LIST.
```

- MANOVA writes the same matrix as in the example above. However, the matrix file replaces the active dataset. The LIST command is executed on the matrix file (not on the file *IRIS*).

Example

```
GET FILE=PRSNNL.
FREQUENCIES VARIABLE=AGE.

MANOVA SEPALLEN SEPALWID PETALLEN PETALWID BY TYPE(1,3)
  /MATRIX=IN(MANMTX).
```

- This example assumes that you want to perform a frequencies analysis on the file *PRSNNL* and then use MANOVA to read a different file. The file that you want to read is an existing matrix data file. The external matrix file *MANMTX* is specified in parentheses after IN on the MATRIX subcommand.
- *MANMTX* does not replace *PRSNNL* as the active dataset.

Example

```
GET FILE=MANMTX.
MANOVA SEPALLEN SEPALWID PETALLEN PETALWID BY TYPE(1,3)
  /MATRIX=IN(*).
```

- This example assumes that you are starting a new session and want to read an existing matrix data file. GET retrieves the matrix file *MANMTX*.
- An asterisk is specified in parentheses after IN on the MATRIX subcommand to read the active dataset. You can also leave the parentheses empty to indicate the default.
- If the GET command is omitted, an error message is issued.
- If you specify *MANMTX* in parentheses after IN, an error message is issued.

ANALYSIS Subcommand

ANALYSIS allows you to work with a subset of the continuous variables (dependent variable and covariates) that you named on the MANOVA variable list. In univariate analysis of variance, you can use ANALYSIS to allow factor-by-covariate interaction terms in your model (see the DESIGN subcommand below). You can also use ANALYSIS to switch the roles of the dependent variable and a covariate.

- In general, ANALYSIS gives you complete control over which continuous variables are to be dependent variables, which continuous variables are to be covariates, and which continuous variables are to be neither.
- ANALYSIS specifications are like the MANOVA variables specification, except that factors are not named. Enter the dependent variable and, if there are covariates, the keyword WITH and the covariates.
- Only variables that are listed as dependent variables or covariates on the MANOVA variable list can be entered on the ANALYSIS subcommand.
- In a univariate analysis of variance, the most important use of ANALYSIS is to omit covariates from the analysis list, thereby making them available for inclusion on DESIGN (see the example below and the DESIGN subcommand examples).
- For more information about ANALYSIS, refer to "ANALYSIS Subcommand " on page 1051.

Example

```
MANOVA DEP BY FACTOR(1,3) WITH COV
  /ANALYSIS DEP
  /DESIGN FACTOR, COV, FACTOR BY COV.
```

- *COV*, a continuous variable, is included on the MANOVA variable list as a covariate.
- *COV* is not mentioned on ANALYSIS, so it will not be included in the model as a dependent variable or covariate. *COV* can, therefore, be explicitly included on the DESIGN subcommand.
- DESIGN includes the main effects of *FACTOR* and *COV* and the *FACTOR* by *COV* interaction.

DESIGN Subcommand

DESIGN specifies the effects that are included in a specific model. DESIGN must be the last subcommand entered for any model.

The cells in a design are defined by all of the possible combinations of levels of the factors in that design. The number of cells equals the product of the number of levels of all the factors. A design is *balanced* if each cell contains the same number of cases. MANOVA can analyze both balanced and unbalanced designs.

- Specify a list of terms to be included in the model, separated by spaces or commas.
- The default design, if the DESIGN subcommand is omitted or is specified by itself, is a full factorial model containing all main effects and all orders of factor-by-factor interaction.
- If the last subcommand that is specified is not DESIGN, a default full factorial design is estimated.
- To include a term for the main effect of a factor, enter the name of the factor on the DESIGN subcommand.
- To include a term for an interaction between factors, use the keyword BY to join the factors that are involved in the interaction.
- Terms are entered into the model in the order in which you list them on DESIGN. If you have specified SEQUENTIAL on the METHOD subcommand to partition the sums of squares in a hierarchical fashion, this order may affect the significance tests.
- You can specify other types of terms in the model, as described in the following sections.
- Multiple DESIGN subcommands are accepted. An analysis of one model is produced for each DESIGN subcommand.

Example

```
MANOVA Y BY A(1,2) B(1,2) C(1,3)
  /DESIGN
  /DESIGN A, B, C
  /DESIGN A, B, C, A BY B, A BY C.
```

- The first DESIGN produces the default full factorial design, with all main effects and interactions for factors *A*, *B*, and *C*.

- The second DESIGN produces an analysis with main effects only for *A*, *B*, and *C*.
- The third DESIGN produces an analysis with main effects and the interactions between *A* and the other two factors. The interaction between *B* and *C* is not in the design, nor is the interaction between all three factors.

Partitioned Effects: Number in Parentheses

You can specify a number in parentheses following a factor name on the DESIGN subcommand to identify individual degrees of freedom or partitions of the degrees of freedom that are associated with an effect.

- If you specify PARTITION, the number refers to a partition. Partitions can include more than one degree of freedom (see "PARTITION Subcommand " on page 1028). For example, if the first partition of *SEED* includes two degrees of freedom, the term SEED(1) on a DESIGN subcommand tests the two degrees of freedom.
- If you do not use PARTITION, the number refers to a single degree of freedom that is associated with the effect.
- The number refers to an individual level for a factor if that factor follows the keyword WITHIN or MWITHIN (see the sections about nested effects and pooled effects below).
- A factor has one less degree of freedom than it has levels or values.

Example

```
MANOVA YIELD BY SEED(1,4) WITH RAINFALL
  /PARTITION(SEED)=(2,1)
  /DESIGN=SEED(1) SEED(2).
```

- Factor *SEED* is subdivided into two partitions, one partition containing the first two degrees of freedom and the other partition containing the last degree of freedom.
- The two partitions of *SEED* are treated as independent effects.

Nested Effects: WITHIN Keyword

Use the WITHIN keyword (alias W) to nest the effects of one factor within the effects of another factor or an interaction term.

Example

```
MANOVA YIELD BY SEED(1,4) FERT(1,3) PLOT (1,4)
  /DESIGN = FERT WITHIN SEED BY PLOT.
```

- The three factors in this example are type of seed (*SEED*), type of fertilizer (*FERT*), and location of plots (*PLOT*).
- The DESIGN subcommand nests the effects of *FERT* within the interaction term of *SEED* by *PLOT*. The levels of *FERT* are considered distinct for each combination of levels of *SEED* and *PLOT*.

Simple Effects: WITHIN and MWITHIN Keywords

A factor can be nested within one specific level of another factor by indicating the level in parentheses. This process allows you to estimate simple effects or the effect of one factor within only one level of another factor. Simple effects can be obtained for higher-order interactions as well.

Use WITHIN to request simple effects of between-subjects factors.

Example

```
MANOVA YIELD BY SEED(2,4) FERT(1,3) PLOT (1,4)
  /DESIGN = FERT WITHIN SEED (1).
```

- This example requests the simple effect of *FERT* within the first level of *SEED*.
- The number (*n*) specified after a WITHIN factor refers to the level of that factor. The value is the ordinal position, which is not necessarily the value of that level. In this example, the first level is associated with value 2.

- The number does *not* refer to the number of partitioned effects (see "PARTITION Subcommand " on page 1028).

Example

```
MANOVA YIELD BY SEED(2,4) FERT(1,3) PLOT (3,5)
 /DESIGN = FERT WITHIN PLOT(1) WITHIN SEED(2)
```

- This example requests the effect of *FERT* within the second *SEED* level of the first *PLOT* level.
- The second *SEED* level is associated with value 3, and the first *PLOT* level is associated with value 3.

Use MWITHIN to request simple effects of within-subjects factors in repeated measures analysis (see "MWITHIN Keyword for Simple Effects " on page 1057).

Pooled Effects: Plus Sign

To pool different effects for the purpose of significance testing, join the effects with a plus sign (+). A single test is made for the combined effect of the pooled terms.

- The keyword BY is evaluated before effects are pooled together.
- Parentheses are not allowed for changing the order of evaluation. For example, it is illegal to specify (A + B) BY C. You must specify /DESIGN=A BY C + B BY C.

Example

```
MANOVA Y BY A(1,3) B(1,4) WITH X
 /ANALYSIS=Y
 /DESIGN=A, B, A BY B, A BY X + B BY X + A BY B BY X.
```

- This example shows how to test homogeneity of regressions in a two-way analysis of variance.
- The + signs are used to produce a pooled test of all interactions involving the covariate *X*. If this test is significant, the assumption of homogeneity of variance is questionable.

MUPLUS Keyword

MUPLUS combines the constant term (μ) in the model with the term that is specified after it. The normal use of this specification is to obtain parameter estimates that represent weighted means for the levels of some factor. For example, MUPLUS SEED represents the constant, or overall, mean plus the effect for each level of *SEED*. The significance of such effects is usually uninteresting, but the parameter estimates represent the weighted means for each level of *SEED*, adjusted for any covariates in the model.

- MUPLUS cannot appear more than once on a given DESIGN subcommand.
- MUPLUS is the only way to get standard errors for the predicted mean for each level of the specified factor.
- Parameter estimates are not displayed by default; you must explicitly request them on the PRINT subcommand or via a CONTRAST subcommand.
- You can obtain the unweighted mean by specifying the full factorial model, excluding those terms that are contained by an effect, and prefixing the effect whose mean is to be found by MUPLUS.

Effects of Continuous Variables

Usually you name factors but not covariates on the DESIGN subcommand. The linear effects of covariates are removed from the dependent variable before the design is tested. However, the design can include variables that are measured at the interval level and originally named as covariates or as additional dependent variables.

- Continuous variables on a DESIGN subcommand must be named as dependents or covariates on the MANOVA variable list.
- Before you can name a continuous variable on a DESIGN subcommand, you must supply an ANALYSIS subcommand that does *not* name the variable. This action excludes it from the analysis as a dependent variable or covariate and makes it eligible for inclusion on DESIGN.

- You can use the keyword POOL(varlist) to pool more than one continuous variable into a single effect (provided that the continuous variables are all excluded on an ANALYSIS subcommand). For a single continuous variable, POOL(VAR) is equivalent to VAR.
- The TO convention in the variable list for POOL refers to the order of continuous variables (dependent variables and covariates) on the original MANOVA variable list, which is not necessarily their order on the active dataset. This use is the only allowable use of the keyword TO on a DESIGN subcommand.
- You can specify interaction terms between factors and continuous variables. If *FAC* is a factor and *COV* is a covariate that has been omitted from an ANALYSIS subcommand, FAC BY COV is a valid specification on a DESIGN statement.
- You cannot specify an interaction between two continuous variables. Use the COMPUTE command to create a variable representing the interaction prior to MANOVA.

Example

```
*  This example tests whether the regression of the dependent
   variable Y on the two variables X1 and X2 is the same across
   all the categories of the factors AGE and TREATMNT.

MANOVA Y BY AGE(1,5) TREATMNT(1,3) WITH X1, X2
  /ANALYSIS = Y
  /DESIGN = POOL(X1,X2),
            AGE, TREATMNT, AGE BY TREATMNT,
            POOL(X1,X2) BY AGE + POOL(X1,X2) BY TREATMNT
              + POOL(X1,X2) BY AGE BY TREATMNT.
```

- ANALYSIS excludes *X1* and *X2* from the standard treatment of covariates so that they can be used in the design.
- DESIGN includes five terms. POOL(X1,X2), the overall regression of the dependent variable on *X1* and *X2*, is entered first, followed by the two factors and their interaction.
- The last term is the test for equal regressions. It consists of three factor-by-continuous-variable interactions pooled together. POOL(X1,X2) BY AGE is the interaction between *AGE* and the combined effect of the continuous variables *X1* and *X2*. It is combined with similar interactions between *TREATMNT* and the continuous variables and between the *AGE* by *TREATMNT* interaction and the continuous variables.
- If the last term is not statistically significant, there is no evidence that the regression of *Y* on *X1* and *X2* is different across any combination of the categories of *AGE* and *TREATMNT*.

Error Terms for Individual Effects

The “error” sum of squares against which terms in the design are tested is specified on the ERROR subcommand. For any particular term on a DESIGN subcommand, you can specify a different error term to be used in the analysis of variance. To do so, name the term followed by the keyword VS (or AGAINST) and the error term keyword.

- To test a term against only the within-cells sum of squares, specify the term followed by VS WITHIN on the DESIGN subcommand. For example, GROUP VS WITHIN tests the effect of the factor *GROUP* against only the within-cells sum of squares. For most analyses, this term is the default error term.
- To test a term against only the residual sum of squares (the sum of squares for all terms that are not included in your DESIGN), specify the term followed by VS RESIDUAL.
- To test against the combined within-cells and residual sums of squares, specify the term followed by VS WITHIN+RESIDUAL.
- To test against any other sum of squares in the analysis of variance, include a term corresponding to the desired sum of squares in the design and assign it to an integer between 1 and 10. You can then test against the number of the error term. It is often convenient to test against the term before you define it. This process is perfectly acceptable as long as you define the error term on the same DESIGN subcommand.

Example

```
MANOVA DEP BY A, B, C (1,3)
  /DESIGN=A VS 1,
```

```
          B WITHIN A = 1 VS 2,
          C WITHIN B WITHIN A = 2 VS WITHIN.
```

- In this example, the factors *A*, *B*, and *C* are completely nested; levels of *C* occur within levels of *B*, which occur within levels of *A*. Each factor is tested against everything within it.
- *A*, the outermost factor, is tested against the *B* within *A* sum of squares, to see if it contributes anything beyond the effects of *B* within each of its levels. The *B* within *A* sum of squares is defined as error term number 1.
- *B* nested within *A*, in turn, is tested against error term number 2, which is defined as the *C* within *B* within *A* sum of squares.
- Finally, *C* nested within *B* nested within *A* is tested against the within-cells sum of squares.

User-defined error terms are specified by simply inserting = n after a term, where *n* is an integer from 1 to 10. The equals sign is required. Keywords that are used in building a design term, such as BY or WITHIN, are evaluated first. For example, error term number 2 in the above example consists of the entire term `C WITHIN B WITHIN A`. An error-term *number,* but not an error-term *definition,* can follow the keyword VS.

CONSTANT Keyword

By default, the constant (grand mean) term is included as the first term in the model.

- If you have specified NOCONSTANT on the METHOD subcommand, a constant term will not be included in any design unless you request it with the CONSTANT keyword on DESIGN.
- You can specify an error term for the constant.
- A factor named CONSTANT will not be recognized on the DESIGN subcommand.

References

Bock, R. D. 1985. *Multivariate statistical methods in behavioral research*. Chicago: Scientific Software, Inc..

Cohen, J. 1977. *Statistical power analysis for the behavioral sciences*. San Diego, California: Academic Press.

Hays, W. L. 1981. *Statistics*, 3rd ed. New York: Holt, Rinehart, and Winston.

Timm, N. H. 1975. *Multivariate statistics: With applications in education and psychology*. Monterey, California: Brooks/Cole.

MANOVA: Multivariate

MANOVA is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
MANOVA dependent varlist [BY factor list (min,max) [factor list...]]
                         [WITH covariate list]

 [/TRANSFORM [(dependent varlist [/dependent varlist])]=
                     [ORTHONORM] [{CONTRAST}]  {DEVIATIONS (refcat)     }]
                                 {BASIS   }   {DIFFERENCE               }
                                              {HELMERT                  }
                                              {SIMPLE (refcat)          }
                                              {REPEATED                 }
                                              {POLYNOMIAL[({1,2,3...})]}
                                                          {metric  }
                                              {SPECIAL (matrix)         }

 [/RENAME={newname} {newname}...]
          {*      } {*      }

 [/{PRINT  }=[HOMOGENEITY [(BOXM)]]
   {NOPRINT} [ERROR [([COV] [COR] [SSCP] [STDDEV])]]
             [SIGNIF [([MULTIV**] [EIGEN] [DIMENR]
                       [UNIV**] [HYPOTH][STEPDOWN] [BRIEF])]]
             [TRANSFORM]                                    ]

 [/PCOMPS=[COR] [COV] [ROTATE(rottype)]
          [NCOMP(n)] [MINEIGEN(eigencut)] [ALL]]

 [/PLOT=[ZCORR]]

 [/DISCRIM [RAW] [STAN] [ESTIM] [COR] [ALL]
           [ROTATE(rottype)] [ALPHA({.25**})]]
                                    {a    }

 [/POWER=[T({.05**})] [F({.05**})] [{APPROXIMATE}]]
           {a    }      {a    }    {EXACT      }

 [/CINTERVAL=[MULTIVARIATE  ({ROY      })]]
                             {PILLAI   }
                             {BONFER   }
                             {HOTELLING}
                             {WILKS    }

 [/ANALYSIS [({UNCONDITIONAL**})]=[()dependent varlist
              {CONDITIONAL    }      [WITH covariate varlist]
                                       [/dependent varlist...][])][WITH varlist]]

 [/DESIGN...]*
```

* The DESIGN subcommand has the same syntax as is described in “DESIGN Subcommand ” on page 1038.

**Default if the subcommand or keyword is omitted.

Example

```
MANOVA SCORE1 TO SCORE4 BY METHOD(1,3).
```

Overview

This section discusses the subcommands that are used in multivariate analysis of variance and covariance designs with several interrelated dependent variables. The discussion focuses on subcommands and keywords that do not apply, or apply in different manners, to univariate analyses. It does not contain information on all of the subcommands you will need to specify the design. For subcommands not covered here, see “MANOVA: Univariate” on page 1023.

Options

Dependent Variables and Covariates. You can specify subsets and reorder the dependent variables and covariates using the ANALYSIS subcommand. You can specify linear transformations of the dependent variables and covariates using the TRANSFORM subcommand. When transformations are performed, you can rename the variables using the RENAME subcommand and request the display of a transposed transformation matrix currently in effect using the PRINT subcommand.

Optional Output. You can request or suppress output on the PRINT and NOPRINT subcommands. Additional output appropriate to multivariate analysis includes error term matrices, Box’s *M* statistic,

multivariate and univariate *F* tests, and other significance analyses. You can also request predicted cell means for specific dependent variables on the PMEANS subcommand, produce a canonical discriminant analysis for each effect in your model with the DISCRIM subcommand, specify a principal components analysis of each error sum-of-squares and cross-product matrix in a multivariate analysis on the PCOMPS subcommand, display multivariate confidence intervals using the CINTERVAL subcommand, and generate a half-normal plot of the within-cells correlations among the dependent variables with the PLOT subcommand.

Basic Specification

- The basic specification is a variable list identifying the dependent variables, with the factors (if any) named after BY and the covariates (if any) named after WITH.
- By default, MANOVA produces multivariate and univariate *F* tests.

Subcommand Order

- The variable list must be specified first.
- Subcommands applicable to a specific design must be specified before that DESIGN subcommand. Otherwise, subcommands can be used in any order.

Syntax Rules

- All syntax rules applicable to univariate analysis also apply to multivariate analysis.
- If you enter one of the multivariate specifications in a univariate analysis, MANOVA ignores it.

Limitations

- Maximum of 20 factors.
- Memory requirements depend primarily on the number of cells in the design. For the default full factorial model, the number of cells equals the product of the number of levels or categories in each factor.

MANOVA Variable List

- Multivariate MANOVA calculates statistical tests that are valid for analyses of dependent variables that are correlated with one another. The dependent variables must be specified first.
- The factor and covariate lists follow the same rules as in univariate analyses.
- If the dependent variables are uncorrelated, the univariate significance tests have greater statistical power.

TRANSFORM Subcommand

TRANSFORM performs linear transformations of some or all of the continuous variables (dependent variables and covariates). Specifications on TRANSFORM include an optional list of variables to be transformed, optional keywords to describe how to generate a transformation matrix from the specified contrasts, and a required keyword specifying the transformation contrasts.

- Transformations apply to all subsequent designs unless replaced by another TRANSFORM subcommand.
- TRANSFORM subcommands are not cumulative. Only the transformation specified most recently is in effect at any time. You can restore the original variables in later designs by specifying SPECIAL with an identity matrix.
- You should not use TRANSFORM when you use the WSFACTORS subcommand to request repeated measures analysis; a transformation is automatically performed in repeated measures analysis (see “MANOVA: Repeated Measures” on page 1053).
- Transformations are in effect for the duration of the MANOVA procedure only. After the procedure is complete, the original variables remain in the active dataset.

- By default, the transformation matrix is not displayed. Specify the keyword TRANSFORM on the PRINT subcommand to see the matrix generated by the TRANSFORM subcommand.
- If you do not use the RENAME subcommand with TRANSFORM, the variables specified on TRANSFORM are renamed temporarily (for the duration of the procedure) as *T1*, *T2*, and so on. Explicit use of RENAME is recommended.
- Subsequent references to transformed variables should use the new names. The only exception is when you supply a VARIABLES specification on the OMEANS subcommand after using TRANSFORM. In this case, specify the original names. OMEANS displays observed means of original variables. See "OMEANS Subcommand " on page 1032.

Variable Lists

- By default, MANOVA applies the transformation you request to all continuous variables (dependent variables and covariates).
- You can enter a variable list in parentheses following the TRANSFORM subcommand. If you do, only the listed variables are transformed.
- You can enter multiple variable lists, separated by slashes, within a single set of parentheses. Each list must have the same number of variables, and the lists must not overlap. The transformation is applied separately to the variables on each list.
- In designs with covariates, transform only the dependent variables, or, in some designs, apply the same transformation separately to the dependent variables and the covariates.

CONTRAST, BASIS, and ORTHONORM Keywords

You can control how the transformation matrix is to be generated from the specified contrasts. If none of these three keywords is specified on TRANSFORM, the default is CONTRAST.

CONTRAST. *Generate the transformation matrix directly from the contrast matrix specified* (see "CONTRAST Subcommand " on page 1026). This is the default.

BASIS. *Generate the transformation matrix from the one-way basis matrix corresponding to the specified contrast matrix.* BASIS makes a difference only if the transformation contrasts are not orthogonal.

ORTHONORM. *Orthonormalize the transformation matrix by rows before use.* MANOVA eliminates redundant rows. By default, orthonormalization is not done.

- CONTRAST and BASIS are alternatives and are mutually exclusive.
- ORTHONORM is independent of the CONTRAST/BASIS choice; you can enter it before or after either of those keywords.

Transformation Methods

To specify a transformation method, use one of the following keywords available on the TRANSFORM subcommand. Note that these are identical to the keywords available for the CONTRAST subcommand (see "CONTRAST Subcommand " on page 1026). However, in univariate designs, they are applied to the different levels of a factor. Here they are applied to the continuous variables in the analysis. This reflects the fact that the different dependent variables in a multivariate MANOVA setup can often be thought of as corresponding to different levels of some factor.

- The transformation keyword (and its specifications, if any) must follow all other specifications on the TRANSFORM subcommand.

DEVIATION. *Deviations from the mean of the variables being transformed.* The first transformed variable is the mean of all variables in the transformation. Other transformed variables represent deviations of individual variables from the mean. One of the original variables (by default, the last) is omitted as redundant. To omit a variable other than the last, specify the number of the variable to be omitted in parentheses after the DEVIATION keyword. For example, `/TRANSFORM (A B C) = DEVIATION(1)` omits *A* and creates variables representing the mean, the deviation of *B* from the mean, and the deviation of *C* from the mean. A DEVIATION transformation is not orthogonal.

DIFFERENCE. *Difference or reverse Helmert transformation.* The first transformed variable is the mean of the original variables. Each of the original variables except the first is then transformed by subtracting the mean of those (original) variables that precede it. A DIFFERENCE transformation is orthogonal.

HELMERT. *Helmert transformation.* The first transformed variable is the mean of the original variables. Each of the original variables except the last is then transformed by subtracting the mean of those (original) variables that follow it. A HELMERT transformation is orthogonal.

SIMPLE. *Each original variable, except the last, is compared to the last of the original variables.* To use a variable other than the last as the omitted reference variable, specify its number in parentheses following the keyword SIMPLE. For example, /TRANSFORM(A B C) = SIMPLE(2) specifies the second variable, *B*, as the reference variable. The three transformed variables represent the mean of *A*, *B*, and *C*, the difference between *A* and *B*, and the difference between *C* and *B*. A SIMPLE transformation is not orthogonal.

POLYNOMIAL. *Orthogonal polynomial transformation.* The first transformed variable represents the mean of the original variables. Other transformed variables represent the linear, quadratic, and higher-degree components. By default, values of the original variables are assumed to represent equally spaced points. You can specify unequal spacing by entering a metric consisting of one integer for each variable in parentheses after the keyword POLYNOMIAL. For example, /TRANSFORM(RESP1 RESP2 RESP3) = POLYNOMIAL(1,2,4) might indicate that three response variables correspond to levels of some stimulus that are in the proportion 1:2:4. The default metric is always (1,2,..., *k*), where *k* variables are involved. Only the relative differences between the terms of the metric matter: (1,2,4) is the same metric as (2,3,5) or (20,30,50) because in each instance the difference between the second and third numbers is twice the difference between the first and second.

REPEATED. *Comparison of adjacent variables.* The first transformed variable is the mean of the original variables. Each additional transformed variable is the difference between one of the original variables and the original variable that followed it. Such transformed variables are often called *difference scores.* A REPEATED transformation is not orthogonal.

SPECIAL. *A user-defined transformation.* After the keyword SPECIAL, enter a square matrix in parentheses with as many rows and columns as there are variables to transform. MANOVA multiplies this matrix by the vector of original variables to obtain the transformed variables (see the examples below).

Example

```
MANOVA X1 TO X3 BY A(1,4)
  /TRANSFORM(X1 X2 X3) = SPECIAL( 1  1  1,
                                  1  0 -1,
                                  2 -1 -1)
  /DESIGN.
```

- The given matrix will be post-multiplied by the three continuous variables (considered as a column vector) to yield the transformed variables. The first transformed variable will therefore equal *X1* + *X2* + *X3*, the second will equal *X1* – *X3*, and the third will equal *2X1* – *X2* – *X3*.
- The variable list is optional in this example since all three interval-level variables are transformed.
- You do not need to enter the matrix one row at a time, as shown above. For example,

  ```
  /TRANSFORM = SPECIAL(1 1 1 1 0 -1 2 -1 -1)
  ```

  is equivalent to the TRANSFORM specification in the above example.
- You can specify a repetition factor followed by an asterisk to indicate multiple consecutive elements of a SPECIAL transformation matrix. For example,

  ```
  /TRANSFORM = SPECIAL (4*1 0 -1 2 2*-1)
  ```

  is again equivalent to the TRANSFORM specification above.

Example

```
MANOVA X1 TO X3, Y1 TO Y3 BY A(1,4)
 /TRANSFORM (X1 X2 X3/Y1 Y2 Y3) = SPECIAL( 1  1  1,
                                           1  0 -1,
                                           2 -1 -1)
 /DESIGN.
```

- Here the same transformation shown in the previous example is applied to *X1*, *X2*, *X3* and to *Y1*, *Y2*, *Y3*.

RENAME Subcommand

Use RENAME to assign new names to transformed variables. Renaming variables after a transformation is strongly recommended. If you transform but do not rename the variables, the names *T1*, *T2*, ...,*Tn* are used as names for the transformed variables.

- Follow RENAME with a list of new variable names.
- You must enter a new name for each dependent variable and covariate on the MANOVA variable list.
- Enter the new names in the order in which the original variables appeared on the MANOVA variable list.
- To retain the original name for one or more of the interval variables, you can either enter an asterisk or reenter the old name as the new name.
- References to dependent variables and covariates on subcommands following RENAME must use the new names. The original names will not be recognized within the MANOVA procedure. The only exception is the OMEANS subcommand, which displays observed means of the original (untransformed) variables. Use the original names on OMEANS.
- The new names exist only during the MANOVA procedure that created them. They do not remain in the active dataset after the procedure is complete.

Example

```
MANOVA A, B, C, V4, V5 BY TREATMNT(1,3)
  /TRANSFORM(A, B, C) = REPEATED
  /RENAME = MEANABC, AMINUSB, BMINUSC, *, *
  /DESIGN.
```

- The REPEATED transformation produces three transformed variables, which are then assigned mnemonic names *MEANABC*, *AMINUSB*, and *BMINUSC*.
- *V4* and *V5* retain their original names.

Example

```
MANOVA WT1, WT2, WT3, WT4 BY TREATMNT(1,3) WITH COV
  /TRANSFORM (WT1 TO WT4) = POLYNOMIAL
  /RENAME = MEAN, LINEAR, QUAD, CUBIC, *
  /ANALYSIS = MEAN, LINEAR, QUAD WITH COV
  /DESIGN.
```

- After the polynomial transformation of the four *WT* variables, RENAME assigns appropriate names to the various trends.
- Even though only four variables were transformed, RENAME applies to all five continuous variables. An asterisk is required to retain the original name for *COV*.
- The ANALYSIS subcommand following RENAME refers to the interval variables by their new names.

PRINT and NOPRINT Subcommands

All of the PRINT specifications described in “PRINT and NOPRINT Subcommands ” on page 1029 are available in multivariate analyses. The following additional output can be requested. To suppress any optional output, specify the appropriate keyword on NOPRINT.

ERROR. *Error matrices.* Three types of matrices are available.

SIGNIF. *Significance tests.*

TRANSFORM. *Transformation matrix.* It is available if you have transformed the dependent variables with the TRANSFORM subcommand.

HOMOGENEITY. *Test for homogeneity of variance.* BOXM is available for multivariate analyses.

ERROR Keyword

In multivariate analysis, error terms consist of entire matrices, not single values. You can display any of the following error matrices on a PRINT subcommand by requesting them in parentheses following the keyword ERROR. If you specify ERROR by itself, without further specifications, the default is to display COV and COR.

SSCP. *Error sums-of-squares and cross-products matrix.*

COV. *Error variance-covariance matrix.*

COR. *Error correlation matrix with standard deviations on the diagonal.* This also displays the determinant of the matrix and Bartlett's test of sphericity, a test of whether the error correlation matrix is significantly different from an identity matrix.

SIGNIF Keyword

You can request any of the optional output listed below by entering the appropriate specification in parentheses after the keyword SIGNIF on the PRINT subcommand. Further specifications for SIGNIF are described in "PRINT Subcommand " on page 1059.

MULTIV. *Multivariate* F *tests for group differences.* MULTIV is always printed unless explicitly suppressed with the NOPRINT subcommand.

EIGEN. *Eigenvalues of the* $S_k S_e^{-1}$ *matrix*. This matrix is the product of the hypothesis sums-of-squares and cross-products (SSCP) matrix and the inverse of the error SSCP matrix. To print EIGEN, request it on the PRINT subcommand.

DIMENR. *A dimension-reduction analysis.* To print DIMENR, request it on the PRINT subcommand.

UNIV. *Univariate* F *tests.* UNIV is always printed except in repeated measures analysis. If the dependent variables are uncorrelated, univariate tests have greater statistical power. To suppress UNIV, use the NOPRINT subcommand.

HYPOTH. *The hypothesis SSCP matrix.* To print HYPOTH, request it on the PRINT subcommand.

STEPDOWN. *Roy-Bargmann stepdown* F *tests.* To print STEPDOWN, request it on the PRINT subcommand.

BRIEF. *Abbreviated multivariate output.* This is similar to a univariate analysis of variance table but with Wilks' multivariate *F* approximation (lambda) replacing the univariate *F*. BRIEF overrides any of the SIGNIF specifications listed above.

SINGLEDF. *Significance tests for the single degree of freedom making up each effect for ANOVA tables.* Results are displayed separately corresponding to each hypothesis degree of freedom. See the topic "SIGNIF Keyword " on page 1031 for more information.

- If neither PRINT nor NOPRINT is specified, MANOVA displays the results corresponding to MULTIV and UNIV for a multivariate analysis not involving repeated measures.
- If you enter any specification except BRIEF or SINGLEDF for SIGNIF on the PRINT subcommand, the requested output is displayed in addition to the default.
- To suppress the default, specify the keyword(s) on the NOPRINT subcommand.

TRANSFORM Keyword

The keyword TRANSFORM specified on PRINT displays the transposed transformation matrix in use for each subsequent design. This matrix is helpful in interpreting a multivariate analysis in which the interval-level variables have been transformed with either TRANSFORM or WSFACTORS.

- The matrix displayed by this option is the transpose of the transformation matrix.
- Original variables correspond to the rows of the matrix, and transformed variables correspond to the columns.
- A **transformed variable** is a linear combination of the original variables using the coefficients displayed in the column corresponding to that transformed variable.

HOMOGENEITY Keyword

In addition to the BARTLETT and COCHRAN specifications described in "HOMOGENEITY Keyword " on page 1031, the following test for homogeneity is available for multivariate analyses:

BOXM. *Box's* M *statistic.* BOXM requires at least two dependent variables. If there is only one dependent variable when BOXM is requested, MANOVA prints Bartlett-Box *F* test statistic and issues a note.

PLOT Subcommand

In addition to the plots described in "PLOT Subcommand " on page 1035, the following is available for multivariate analyses:

ZCORR. *A half-normal plot of the within-cells correlations among the dependent variables.* MANOVA first transforms the correlations using Fisher's *Z* transformation. If errors for the dependent variables are uncorrelated, the plotted points should lie close to a straight line.

PCOMPS Subcommand

PCOMPS requests a principal components analysis of each error matrix in a multivariate analysis. You can display the principal components of the error correlation matrix, the error variance-covariance matrix, or both. These principal components are corrected for differences due to the factors and covariates in the MANOVA analysis. They tend to be more useful than principal components extracted from the raw correlation or covariance matrix when there are significant group differences between the levels of the factors or when a significant amount of error variance is accounted for by the covariates. You can specify any of the keywords listed below on PCOMPS.

COR. *Principal components analysis of the error correlation matrix.*

COV. *Principal components analysis of the error variance-covariance matrix.*

ROTATE. *Rotate the principal components solution.* By default, no rotation is performed. Specify a rotation type (either VARIMAX, EQUAMAX, or QUARTIMAX) in parentheses after the keyword ROTATE. To cancel a rotation specified for a previous design, enter NOROTATE in the parentheses after ROTATE.

NCOMP(n). *The number of principal components to rotate.* Specify a number in parentheses. The default is the number of dependent variables.

MINEIGEN(n). *The minimum eigenvalue for principal component extraction.* Specify a cutoff value in parentheses. Components with eigenvalues below the cutoff will not be retained in the solution. The default is 0; all components (or the number specified on NCOMP) are extracted.

ALL. *COR, COV, and ROTATE.*

- You must specify either COR or COV (or both). Otherwise, MANOVA will not produce any principal components.
- Both NCOMP and MINEIGEN limit the number of components that are rotated.
- If the number specified on NCOMP is less than two, two components are rotated provided that at least two components have eigenvalues greater than any value specified on MINEIGEN.
- Principal components analysis is computationally expensive if the number of dependent variables is large.

DISCRIM Subcommand

DISCRIM produces a canonical discriminant analysis for each effect in a design. (For covariates, DISCRIM produces a canonical correlation analysis.) These analyses aid in the interpretation of multivariate effects. You can request the following statistics by entering the appropriate keywords after the subcommand DISCRIM:

RAW. *Raw discriminant function coefficients.*

STAN. *Standardized discriminant function coefficients.*

ESTIM. *Effect estimates in discriminant function space.*

COR. *Correlations between the dependent variables and the canonical variables defined by the discriminant functions.*

ROTATE. *Rotation of the matrix of correlations between dependent and canonical variables.* Specify rotation type VARIMAX, EQUAMAX, or QUARTIMAX in parentheses after this keyword.

ALL. *RAW, STAN, ESTIM, COR, and ROTATE.*

By default, the significance level required for the extraction of a canonical variable is 0.25. You can change this value by specifying the keyword ALPHA and a value between 0 and 1 in parentheses:

ALPHA. *The significance level required before a canonical variable is extracted.* The default is 0.25. To change the default, specify a decimal number between 0 and 1 in parentheses after ALPHA.

- The correlations between dependent variables and canonical functions are not rotated unless at least two functions are significant at the level defined by ALPHA.
- If you set ALPHA to 1.0, all discriminant functions are reported (and rotated, if you so request).
- If you set ALPHA to 0, no discriminant functions are reported.

POWER Subcommand

The following specifications are available for POWER in multivariate analysis. For applications of POWER in univariate analysis, see "POWER Subcommand " on page 1033.

APPROXIMATE. *Approximate power values.* This is the default. Approximate power values for multivariate tests are derived from procedures presented by Muller and Peterson [43]. Approximate values are normally accurate to three decimal places and are much cheaper to compute than exact values.

EXACT. *Exact power values.* Exact power values for multivariate tests are computed from the noncentral *F* distribution. Exact multivariate power values will be displayed only if there is one hypothesis degree of freedom, where all the multivariate criteria have identical power.

- For information on the multivariate generalizations of power and effect size, see [44], [45], and [46].

CINTERVAL Subcommand

In addition to the specifications described in MANOVA: Univariate, the keyword MULTIVARIATE is available for multivariate analysis. You can specify a type in parentheses after the MULTIVARIATE keyword. The following type keywords are available on MULTIVARIATE:

ROY. *Roy's largest root.* An approximation given by Pillai [47] is used. This approximation is accurate for upper percentage points (0.95 to 1), but it is not as good for lower percentage points. Thus, for Roy intervals, the user is restricted to the range 0.95 to 1.

---

[43] Muller, K. E., and B. L. Peterson. 1984. Practical methods for computing power in testing the multivariate general linear hypothesis. *Computational Statistics and Data Analysis,* 2, 143-158.

[44] Muller, K. E., and B. L. Peterson. 1984. Practical methods for computing power in testing the multivariate general linear hypothesis. *Computational Statistics and Data Analysis,* 2, 143-158.

[45] Green, P. E. 1978. *Analyzing multivariate data*. Hinsdale, Ill.: The Dryden Press.

[46] Huberty, C. J. 1972. Multivariate indices of strength of association. *Multivariate Behavioral Research,* 7, 516-523.

[47] Pillai, K. C. S. 1967. Upper percentage points of the largest root of a matrix in multivariate analysis. *Biometrika,* 54, 189-194.

PILLAI. *Pillai's trace.* The intervals are computed by approximating the percentage points with percentage points of the *F* distribution.

WILKS. *Wilks' lambda.* The intervals are computed by approximating the percentage points with percentage points of the *F* distribution.

HOTELLING. *Hotelling's trace.* The intervals are computed by approximating the percentage points with percentage points of the *F* distribution.

BONFER. *Bonferroni intervals.* This approximation is based on Student's *t* distribution.

- The Wilks', Pillai's, and Hotelling's approximate confidence intervals are thought to match exact intervals across a wide range of alpha levels, especially for large sample sizes [48]. Use of these intervals, however, has not been widely investigated.
- To obtain multivariate intervals separately for each parameter, choose individual multivariate intervals. For individual multivariate confidence intervals, the hypothesis degree of freedom is set to 1, in which case Hotelling's, Pillai's, Wilks', and Roy's intervals will be identical and equivalent to those computed from percentage points of Hotelling's T^2 distribution. Individual Bonferroni intervals will differ and, for a small number of dependent variables, will generally be shorter.
- If you specify MULTIVARIATE on CINTERVAL, you must specify a type keyword. If you specify CINTERVAL without any keyword, the default is the same as with univariate analysis—CINTERVAL displays individual-univariate confidence intervals at the 0.95 level.

ANALYSIS Subcommand

ANALYSIS is discussed in "ANALYSIS Subcommand " on page 1037 as a means of obtaining factor-by-covariate interaction terms. In multivariate analyses, it is considerably more useful.

- ANALYSIS specifies a subset of the continuous variables (dependent variables and covariates) listed on the MANOVA variable list and completely redefines which variables are dependent and which are covariates.
- All variables named on an ANALYSIS subcommand must have been named on the MANOVA variable list. It does not matter whether they were named as dependent variables or as covariates.
- Factors cannot be named on an ANALYSIS subcommand.
- After the keyword ANALYSIS, specify the names of one or more dependent variables and, optionally, the keyword WITH followed by one or more covariates.
- An ANALYSIS specification remains in effect for all designs until you enter another ANALYSIS subcommand.
- Continuous variables named on the MANOVA variable list but omitted from the ANALYSIS subcommand currently in effect can be specified on the DESIGN subcommand. See the topic "DESIGN Subcommand " on page 1038 for more information.
- You can use an ANALYSIS subcommand to request analyses of several groups of variables provided that the groups do not overlap. Separate the groups of variables with slashes and enclose the entire ANALYSIS specification in parentheses.

CONDITIONAL and UNCONDITIONAL Keywords

When several analysis groups are specified on a single ANALYSIS subcommand, you can control how each list is to be processed by specifying CONDITIONAL or UNCONDITIONAL in the parentheses immediately following the ANALYSIS subcommand. The default is UNCONDITIONAL.

UNCONDITIONAL. *Process each analysis group separately, without regard to other lists.* This is the default.

CONDITIONAL. *Use variables specified in one analysis group as covariates in subsequent analysis groups.*

[48] Burns, P. R. 1984. Multiple comparison methods in MANOVA. In: *Proceedings of the 7th SPSS Users and Coordinators Conference*. : .

- CONDITIONAL analysis is not carried over from one ANALYSIS subcommand to another.
- You can specify a final covariate list outside the parentheses. These covariates apply to every list within the parentheses, regardless of whether you specify CONDITIONAL or UNCONDITIONAL. The variables on this global covariate list must not be specified in any individual lists.

Example

```
MANOVA A B C BY FAC(1,4) WITH D, E
  /ANALYSIS = (A, B / C / D WITH E)
  /DESIGN.
```

- The first analysis uses *A* and *B* as dependent variables and uses no covariates.
- The second analysis uses *C* as a dependent variable and uses no covariates.
- The third analysis uses *D* as the dependent variable and uses *E* as a covariate.

Example

```
MANOVA A, B, C, D, E BY FAC(1,4) WITH F G
  /ANALYSIS = (A, B / C / D WITH E) WITH F G
  /DESIGN.
```

- A final covariate list WITH F G is specified outside the parentheses. The covariates apply to every list within the parentheses.
- The first analysis uses *A* and *B*, with *F* and *G* as covariates.
- The second analysis uses *C*, with *F* and *G* as covariates.
- The third analysis uses *D*, with *E*, *F*, and *G* as covariates.
- Factoring out *F* and *G* is the only way to use them as covariates in all three analyses, since no variable can be named more than once on an ANALYSIS subcommand.

Example

```
MANOVA A B C BY FAC(1,3)
  /ANALYSIS(CONDITIONAL) = (A WITH B / C)
  /DESIGN.
```

- In the first analysis, *A* is the dependent variable, *B* is a covariate, and *C* is not used.
- In the second analysis, *C* is the dependent variable, and both *A* and *B* are covariates.

MANOVA: Repeated Measures

MANOVA is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
MANOVA dependent varlist [BY factor list (min,max)[factor list...]
         [WITH [varying covariate list] [(constant covariate list)]]

  /WSFACTORS = varname (levels) [varname...]

 [/WSDESIGN = [effect effect...]

 [/MEASURE = newname newname...]

 [/RENAME = newname newname...]

 [/{PRINT   }=[SIGNIF({AVERF**}) (HF) (GG) (EFSIZE)]]
   {NOPRINT}          {AVONLY }

 [/DESIGN]*
```

* The DESIGN subcommand has the same syntax as is described in “DESIGN Subcommand ” on page 1038.

** Default if the subcommand or keyword is omitted.

Example

```
MANOVA Y1 TO Y4 BY GROUP(1,2)
  /WSFACTORS=YEAR(4).
```

Overview

This section discusses the subcommands that are used in repeated measures designs, in which the dependent variables represent measurements of the same variable (or variables) at different times. This section does not contain information on all subcommands you will need to specify the design. For some subcommands or keywords not covered here, such as DESIGN, see “MANOVA: Univariate” on page 1023. For information on optional output and the multivariate significance tests available, see “MANOVA: Multivariate” on page 1043.

- In a simple repeated measures analysis, all dependent variables represent different measurements of the same variable for different values (or levels) of a within-subjects factor. Between-subjects factors and covariates can also be included in the model, just as in analyses not involving repeated measures.
- A **within-subjects factor** is simply a factor that distinguishes measurements made on the same subject or case, rather than distinguishing different subjects or cases.
- MANOVA permits more complex analyses, in which the dependent variables represent levels of two or more within-subjects factors.
- MANOVA also permits analyses in which the dependent variables represent measurements of several variables for the different levels of the within-subjects factors. These are known as **doubly multivariate designs**.
- A repeated measures analysis includes a within-subjects design describing the model to be tested with the within-subjects factors, as well as the usual between-subjects design describing the effects to be tested with between-subjects factors. The default for both types of design is a full factorial model.
- MANOVA always performs an orthonormal transformation of the dependent variables in a repeated measures analysis. By default, MANOVA renames them as *T1*, *T2*, and so forth.

Basic Specification

- The basic specification is a variable list followed by the WSFACTORS subcommand.
- By default, MANOVA performs special repeated measures processing. Default output includes SIGNIF(AVERF) but not SIGNIF(UNIV). In addition, for any within-subjects effect involving more than one transformed variable, the Mauchly test of sphericity is displayed to test the assumption that the covariance matrix of the transformed variables is constant on the diagonal and zero off the diagonal.

The Greenhouse-Geiser epsilon and the Huynh-Feldt epsilon are also displayed for use in correcting the significance tests in the event that the assumption of sphericity is violated.

Subcommand Order

- The list of dependent variables, factors, and covariates must be first.
- WSFACTORS must be the first subcommand used after the variable list.

Syntax Rules

- The WSFACTORS (within-subjects factors), WSDESIGN (within-subjects design), and MEASURE subcommands are used only in repeated measures analysis.
- WSFACTORS is required for any repeated measures analysis.
- If WSDESIGN is not specified, a full factorial within-subjects design consisting of all main effects and interactions among within-subjects factors is used by default.
- The MEASURE subcommand is used for doubly multivariate designs, in which the dependent variables represent repeated measurements of more than one variable.
- Do not use the TRANSFORM subcommand with the WSFACTORS subcommand because WSFACTORS automatically causes an orthonormal transformation of the dependent variables.

Limitations

- Maximum of 20 between-subjects factors. There is no limit on the number of measures for doubly multivariate designs.
- Memory requirements depend primarily on the number of cells in the design. For the default full factorial model, this equals the product of the number of levels or categories in each factor.

Example

```
MANOVA Y1 TO Y4 BY GROUP(1,2)
  /WSFACTORS=YEAR(4)
  /CONTRAST(YEAR)=POLYNOMIAL
  /RENAME=CONST, LINEAR, QUAD, CUBIC
  /PRINT=TRANSFORM PARAM(ESTIM)
  /WSDESIGN=YEAR
  /DESIGN=GROUP.
```

- WSFACTORS immediately follows the MANOVA variable list and specifies a repeated measures analysis in which the four dependent variables represent a single variable measured at four levels of the within-subjects factor. The within-subjects factor is called *YEAR* for the duration of the MANOVA procedure.
- CONTRAST requests polynomial contrasts for the levels of *YEAR*. Because the four variables, *Y1*, *Y2*, *Y3*, and *Y4*, in the active dataset represent the four levels of *YEAR*, the effect is to perform an orthonormal polynomial transformation of these variables.
- RENAME assigns names to the dependent variables to reflect the transformation.
- PRINT requests that the transformation matrix and the parameter estimates be displayed.
- WSDESIGN specifies a within-subjects design that includes only the effect of the *YEAR* within-subjects factor. Because *YEAR* is the only within-subjects factor specified, this is the default design, and WSDESIGN could have been omitted.
- DESIGN specifies a between-subjects design that includes only the effect of the *GROUP* between-subjects factor. This subcommand could have been omitted.

MANOVA Variable List

The list of dependent variables, factors, and covariates must be specified first.

- WSFACTORS determines how the dependent variables on the MANOVA variable list will be interpreted.
- The number of dependent variables on the MANOVA variable list must be a multiple of the number of cells in the within-subjects design. If there are six cells in the within-subjects design, each group of six

dependent variables represents a single within-subjects variable that has been measured in each of the six cells.

- Normally, the number of dependent variables should equal the number of cells in the within-subjects design multiplied by the number of variables named on the MEASURE subcommand (if one is used). If you have more groups of dependent variables than are accounted for by the MEASURE subcommand, MANOVA will choose variable names to label the output, which may be difficult to interpret.
- Covariates are specified after the keyword WITH. You can specify either varying covariates or constant covariates, or both. **Varying covariates**, similar to dependent variables in a repeated measures analysis, represent measurements of the same variable (or variables) at different times while **constant covariates** represent variables whose values remain the same at each within-subjects measurement.
- If you use varying covariates, the number of covariates specified must be an integer multiple of the number of dependent variables.
- If you use constant covariates, you must specify them in parentheses. If you use both constant and varying covariates, constant variates must be specified after all varying covariates.

Example

```
MANOVA MATH1 TO MATH4 BY METHOD(1,2) WITH PHYS1 TO PHYS4 (SES)
  /WSFACTORS=SEMESTER(4).
```

- The four dependent variables represent a score measured four times (corresponding to the four levels of *SEMESTER*).
- The four varying covariates *PHYS1* to *PHYS4* represents four measurements of another score.
- *SES* is a constant covariate. Its value does not change over the time covered by the four levels of *SEMESTER*.
- The default contrast (POLYNOMIAL) is used.

WSFACTORS Subcommand

WSFACTORS names the within-subjects factors and specifies the number of levels for each.

- For repeated measures designs, WSFACTORS must be the first subcommand after the MANOVA variable list.
- Only one WSFACTORS subcommand is permitted per execution of MANOVA.
- Names for the within-subjects factors are specified on the WSFACTORS subcommand. Factor names must not duplicate any of the dependent variables, factors, or covariates named on the MANOVA variable list.
- If there are more than one within-subjects factors, they must be named in the order corresponding to the order of the dependent variables on the MANOVA variable list. MANOVA varies the levels of the last-named within-subjects factor most rapidly when assigning dependent variables to within-subjects cells (see the example below).
- Levels of the factors must be represented in the data by the dependent variables named on the MANOVA variable list.
- Enter a number in parentheses after each factor to indicate how many levels the factor has. If two or more adjacent factors have the same number of levels, you can enter the number of levels in parentheses after all of them.
- Enter only the number of levels for within-subjects factors, not a range of values.
- The number of cells in the within-subjects design is the product of the number of levels for all within-subjects factors.

Example

```
MANOVA X1Y1 X1Y2 X2Y1 X2Y2 X3Y1 X3Y2 BY TREATMNT(1,5) GROUP(1,2)
  /WSFACTORS=X(3) Y(2)
  /DESIGN.
```

- The MANOVA variable list names six dependent variables and two between-subjects factors, *TREATMNT* and *GROUP*.
- WSFACTORS identifies two within-subjects factors whose levels distinguish the six dependent variables. *X* has three levels and *Y* has two. Thus, there are 3 × 2 = 6 cells in the within-subjects design, corresponding to the six dependent variables.
- Variable *X1Y1* corresponds to levels 1,1 of the two within-subjects factors; variable *X1Y2* corresponds to levels 1,2; *X2Y1* to levels 2,1; and so on up to *X3Y2*, which corresponds to levels 3,2. The first within-subjects factor named, *X*, varies most slowly, and the last within-subjects factor named, *Y*, varies most rapidly on the list of dependent variables.
- Because there is no WSDESIGN subcommand, the within-subjects design will include all main effects and interactions: *X*, *Y*, and *X* by *Y*.
- Likewise, the between-subjects design includes all main effects and interactions: *TREATMNT*, *GROUP*, and *TREATMNT* by *GROUP*.
- In addition, a repeated measures analysis always includes interactions between the within-subjects factors and the between-subjects factors. There are three such interactions for each of the three within-subjects effects.

CONTRAST for WSFACTORS

The levels of a within-subjects factor are represented by different dependent variables. Therefore, contrasts between levels of such a factor compare these dependent variables. Specifying the type of contrast amounts to specifying a transformation to be performed on the dependent variables.

- An orthonormal transformation is automatically performed on the dependent variables in a repeated measures analysis.
- To specify the type of orthonormal transformation, use the CONTRAST subcommand for the within-subjects factors.
- Regardless of the contrast type you specify, the transformation matrix is orthonormalized before use.
- If you do not specify a contrast type for within-subjects factors, the default contrast type is orthogonal POLYNOMIAL. Intrinsically orthogonal contrast types are recommended for within-subjects factors if you wish to examine each degree-of-freedom test. Other orthogonal contrast types are DIFFERENCE and HELMERT. MULTIV and AVERF tests are identical, no matter what contrast was specified.
- To perform non-orthogonal contrasts, you must use the TRANSFORM subcommand instead of CONTRAST. The TRANSFORM subcommand is discussed in “TRANSFORM Subcommand ” on page 1044.
- When you implicitly request a transformation of the dependent variables with CONTRAST for within-subjects factors, the same transformation is applied to any covariates in the analysis. The number of covariates must be an integer multiple of the number of dependent variables.
- You can display the transpose of the transformation matrix generated by your within-subjects contrast using the keyword TRANSFORM on the PRINT subcommand.

Example

```
MANOVA SCORE1 SCORE2 SCORE3 BY GROUP(1,4)
  /WSFACTORS=ROUND(3)
  /CONTRAST(ROUND)=DIFFERENCE
  /CONTRAST(GROUP)=DEVIATION
  /PRINT=TRANSFORM PARAM(ESTIM).
```

- This analysis has one between-subjects factor, *GROUP*, with levels 1, 2, 3, and 4, and one within-subjects factor, *ROUND*, with three levels that are represented by the three dependent variables.
- The first CONTRAST subcommand specifies difference contrasts for *ROUND*, the within-subjects factor.
- There is no WSDESIGN subcommand, so a default full factorial within-subjects design is assumed. This could also have been specified as WSDESIGN=ROUND, or simply WSDESIGN.
- The second CONTRAST subcommand specifies deviation contrasts for *GROUP*, the between-subjects factor. This subcommand could have been omitted because deviation contrasts are the default.

- PRINT requests the display of the transformation matrix generated by the within-subjects contrast and the parameter estimates for the model.
- There is no DESIGN subcommand, so a default full factorial between-subjects design is assumed. This could also have been specified as DESIGN=GROUP, or simply DESIGN.

PARTITION for WSFACTORS

The PARTITION subcommand also applies to factors named on WSFACTORS. See the topic "PARTITION Subcommand " on page 1028 for more information.

WSDESIGN Subcommand

WSDESIGN specifies the design for within-subjects factors. Its specifications are like those of the DESIGN subcommand, but it uses the within-subjects factors rather than the between-subjects factors.

- The default WSDESIGN is a full factorial design, which includes all main effects and all interactions for within-subjects factors. The default is in effect whenever a design is processed without a preceding WSDESIGN or when the preceding WSDESIGN subcommand has no specifications.
- A WSDESIGN specification can include main effects, factor-by-factor interactions, nested terms (term within term), terms using the keyword MWITHIN, and pooled effects using the plus sign. The specification is the same as on the DESIGN subcommand but involves only within-subjects factors.
- A WSDESIGN specification cannot include between-subjects factors or terms based on them, nor does it accept interval-level variables, the keywords MUPLUS or CONSTANT, or error-term definitions or references.
- The WSDESIGN specification applies to all subsequent within-subjects designs until another WSDESIGN subcommand is encountered.

Example

```
MANOVA JANLO,JANHI,FEBLO,FEBHI,MARLO,MARHI BY SEX(1,2)
  /WSFACTORS MONTH(3) STIMULUS(2)
  /WSDESIGN MONTH, STIMULUS
  /WSDESIGN
  /DESIGN SEX.
```

- There are six dependent variables, corresponding to three months and two different levels of stimulus.
- The dependent variables are named on the MANOVA variable list in such an order that the level of stimulus varies more rapidly than the month. Thus, *STIMULUS* is named last on the WSFACTORS subcommand.
- The first WSDESIGN subcommand specifies only the main effects for within-subjects factors. There is no *MONTH* by *STIMULUS* interaction term.
- The second WSDESIGN subcommand has no specifications and, therefore, invokes the default within-subjects design, which includes the main effects and their interaction.

MWITHIN Keyword for Simple Effects

You can use MWITHIN on either the WSDESIGN or the DESIGN subcommand in a model with both between- and within-subjects factors to estimate simple effects for factors nested within factors of the opposite type.

Example

```
MANOVA WEIGHT1 WEIGHT2 BY TREAT(1,2)
 /WSFACTORS=WEIGHT(2)
 /DESIGN=MWITHIN TREAT(1) MWITHIN TREAT(2)
MANOVA WEIGHT1 WEIGHT2 BY TREAT(1,2)
 /WSFACTORS=WEIGHT(2)
 /WSDESIGN=MWITHIN WEIGHT(1) MWITHIN WEIGHT(2)
 /DESIGN.
```

- The first DESIGN tests the simple effects of *WEIGHT* within each level of *TREAT*.
- The second DESIGN tests the simple effects of *TREAT* within each level of *WEIGHT*.

MEASURE Subcommand

In a doubly multivariate analysis, the dependent variables represent multiple variables measured under the different levels of the within-subjects factors. Use MEASURE to assign names to the variables that you have measured for the different levels of within-subjects factors.

- Specify a list of one or more variable names to be used in labeling the averaged results. If no within-subjects factor has more than two levels, MEASURE has no effect.
- The number of dependent variables on the DESIGN subcommand should equal the product of the number of cells in the within-subjects design and the number of names on MEASURE.
- If you do not enter a MEASURE subcommand and there are more dependent variables than cells in the within-subjects design, MANOVA assigns names (normally *MEAS.1*, *MEAS.2*, and so on) to the different measures.
- All of the dependent variables corresponding to each measure should be listed together and ordered so that the within-subjects factor named last on the WSFACTORS subcommand varies most rapidly.

Example

```
MANOVA TEMP1 TO TEMP6, WEIGHT1 TO WEIGHT6 BY GROUP(1,2)
  /WSFACTORS=DAY(3) AMPM(2)
  /MEASURE=TEMP WEIGHT
  /WSDESIGN=DAY, AMPM, DAY BY AMPM
  /PRINT=SIGNIF(HYPOTH AVERF)
  /DESIGN.
```

- There are 12 dependent variables: six temperatures and six weights, corresponding to morning and afternoon measurements on three days.
- WSFACTORS identifies the two factors (*DAY* and *AMPM*) that distinguish the temperature and weight measurements for each subject. These factors define six within-subjects cells.
- MEASURE indicates that the first group of six dependent variables correspond to *TEMP* and the second group of six dependent variables correspond to *WEIGHT*.
- These labels, *TEMP* and *WEIGHT*, are used on the output requested by PRINT.
- WSDESIGN requests a full factorial within-subjects model. Because this is the default, WSDESIGN could have been omitted.

RENAME Subcommand

Because any repeated measures analysis involves a transformation of the dependent variables, it is always a good idea to rename the dependent variables. Choose appropriate names depending on the type of contrast specified for within-subjects factors. This is easier to do if you are using one of the orthogonal contrasts. The most reliable way to assign new names is to inspect the transformation matrix.

Example

```
MANOVA LOW1 LOW2 LOW3 HI1 HI2 HI3
  /WSFACTORS=LEVEL(2) TRIAL(3)
  /CONTRAST(TRIAL)=DIFFERENCE
  /RENAME=CONST LEVELDIF TRIAL21 TRIAL312 INTER1 INTER2
  /PRINT=TRANSFORM
  /DESIGN.
```

- This analysis has two within-subjects factors and no between-subjects factors.
- Difference contrasts are requested for *TRIAL*, which has three levels.
- Because all orthonormal contrasts produce the same *F* test for a factor with two levels, there is no point in specifying a contrast type for *LEVEL*.
- New names are assigned to the transformed variables based on the transformation matrix. These names correspond to the meaning of the transformed variables: the mean or constant, the average difference between levels, the average effect of trial 2 compared to 1, the average effect of trial 3 compared to 1 and 2; and the two interactions between *LEVEL* and *TRIAL*.

- The transformation matrix requested by the PRINT subcommand looks like the following table.

Table 66. Transformation matrix

	CONST	LEVELDIF	TRIAL1	TRIAL2	INTER1	INTER2
LOW1	0.408	0.408	-0.500	-0.289	-0.500	-0.289
LOW2	0.408	0.408	0.500	-0.289	0.500	-0.289
LOW3	0.408	0.408	0.000	0.577	0.000	0.577
HI1	0.408	-0.408	-0.500	-0.289	0.500	0.289
HI2	0.408	-0.408	0.500	-0.289	-0.500	0.289
HI3	0.408	-0.408	0.000	0.577	0.000	-0.577

PRINT Subcommand

The following additional specifications on PRINT are useful in repeated measures analysis:

SIGNIF(AVERF). *Averaged* F *tests for use with repeated measures.* This is the default display in repeated measures analysis. The averaged *F* tests in the multivariate setup for repeated measures are equivalent to the univariate (or split-plot or mixed-model) approach to repeated measures.

SIGNIF(AVONLY). *Only the averaged* F *test for repeated measures.* AVONLY produces the same output as AVERF and suppresses all other SIGNIF output.

SIGNIF(HF). *The Huynh-Feldt corrected significance values for averaged univariate* F *tests*.

SIGNIF(GG). *The Greenhouse-Geisser corrected significance values for averaged univariate* F *tests.*

SIGNIF(EFSIZE). *The effect size for the univariate* F *and* t *tests.*

- The keywords AVERF and AVONLY are mutually exclusive.
- When you request repeated measures analysis with the WSFACTORS subcommand, the default display includes SIGNIF(AVERF) but does not include the usual SIGNIF(UNIV).
- The averaged *F* tests are appropriate in repeated measures because the dependent variables that are averaged actually represent contrasts of the WSFACTOR variables. When the analysis is not doubly multivariate, as discussed above, you can specify PRINT=SIGNIF(UNIV) to obtain significance tests for each degree of freedom, just as in univariate MANOVA.

References

Burns, P. R. 1984. Multiple comparison methods in MANOVA. In: *Proceedings of the 7th SPSS Users and Coordinators Conference*. : .

Green, P. E. 1978. *Analyzing multivariate data*. Hinsdale, Ill.: The Dryden Press.

Huberty, C. J. 1972. Multivariate indices of strength of association. *Multivariate Behavioral Research,* 7, 516-523.

Muller, K. E., and B. L. Peterson. 1984. Practical methods for computing power in testing the multivariate general linear hypothesis. *Computational Statistics and Data Analysis,* 2, 143-158.

Pillai, K. C. S. 1967. Upper percentage points of the largest root of a matrix in multivariate analysis. *Biometrika,* 54, 189-194.

MATCH FILES

```
MATCH FILES FILE={'savfile'} [PASSWORD='password'] [TABLE={'savfile'} [PASSWORD='password'] ]
                 {'dataset'}                           {'dataset'}
                 {*        }                           {*        }

 [/RENAME=(old varnames=new varnames)...]

 [/IN=varname]

 /FILE=... [PASSWORD='password']... [TABLE= ... [PASSWORD='password']...]

 [/BY varlist]

 [/MAP]

 [/KEEP={ALL**  }] [/DROP=varlist]
        {varlist}

 [/FIRST=varname]  [/LAST=varname]
```

**Default if the subcommand is omitted.

Release History

Release 22.0

- PASSWORD keyword introduced on the FILE and TABLE subcommands.

Example

```
MATCH FILES FILE='/data/part1.sav'
  /FILE='/data/part2.sav'
  /FILE=*.
```

Overview

MATCH FILES combines variables from 2 up to 50 IBM SPSS Statistics data files. MATCH FILES can make parallel or nonparallel matches between different files or perform table lookups. **Parallel matches** combine files sequentially by case (they are sometimes referred to as **sequential matches**). **Nonparallel matches** combine files according to the values of one or more key variables. In a table lookup, MATCH FILES looks up variables in one file and transfers those variables to a case file.

The files specified on MATCH FILES can be IBM SPSS Statistics data files or open datasets in the current session. The combined file becomes the new active dataset.

In general, MATCH FILES is used to combine files containing the same cases but different variables. To combine files containing the same variables but different cases, use ADD FILES. To update existing IBM SPSS Statistics data files, use UPDATE.

Options

Variable Selection. You can specify which variables from each input file are included in the new active dataset using the DROP and KEEP subcommands.

Variable Names. You can rename variables in each input file before combining the files using the RENAME subcommand. This permits you to combine variables that are the same but whose names differ in different input files or to separate variables that are different but have the same name.

Variable Flag. You can create a variable that indicates whether a case came from a particular input file using IN. You can use the FIRST or LAST subcommands to create a variable that flags the first or last case of a group of cases with the same value for the key variable.

Variable Map. You can request a map showing all variables in the new active dataset, their order, and the input files from which they came using the MAP subcommand.

Basic specification

The basic specification is two or more FILE subcommands, each of which specifies a file to be matched. In addition, BY is required to specify the key variables for nonparallel matches. Both BY and TABLE are required to match table-lookup files.

- All variables from all input files are included in the new active dataset unless DROP or KEEP is specified.

Subcommand order

- RENAME and IN must immediately follow the FILE or TABLE subcommand to which they apply.
- Any BY, FIRST, LAST, KEEP, DROP, and MAP subcommands must follow all of the TABLE, FILE, RENAME, and IN subcommands.

Syntax rules

- RENAME can be repeated after each FILE or TABLE subcommand and applies only to variables in the file named on the immediately preceding FILE or TABLE.
- IN can be used only for a nonparallel match or for a table lookup. (Thus, IN can be used only if BY is specified.)
- BY can be specified only once. However, multiple variables can be specified on BY. When BY is used, all files must be sorted in ascending order of the key variables named on BY.
- MAP can be repeated as often as desired.

Operations

- MATCH FILES reads all files named on FILE or TABLE and builds a new active dataset.
  - If the current active dataset is included and is specified with an asterisk (FILE=*), the new merged dataset replaces the active dataset. If that dataset is a named dataset, the merged dataset retains that name. If the current active dataset is not included or is specified by name (for example, FILE=Dataset1), a new unnamed, merged dataset is created, and it becomes the active dataset. For information on naming datasets, see "DATASET NAME" on page 543.
- The new active dataset contains complete dictionary information from the input files, including variable names, labels, print and write formats, and missing-value indicators. The new file also contains the documents from each of the input files. See DROP DOCUMENTS for information on deleting documents.
- Variables are copied in order from the first file specified, then from the second file specified, and so on.
- If the same variable name is used in more than one input file, data are taken from the file specified first. Dictionary information is taken from the first file containing value labels, missing values, or a variable label for the common variable. If the first file has no such information, MATCH FILES checks the second file, and so on, seeking dictionary information.
- All cases from all input files are included in the combined file. Cases that are absent from one of the input files will be assigned system-missing values for variables unique to that file.
- BY specifies that cases should be combined according to a common value on one or more key variables. All input files must be sorted in ascending order of the key variables.
- If BY is not used, the program performs a parallel (sequential) match, combining the first case from each file, then the second case from each file, and so on, without regard to any identifying values that may be present.
- If the active dataset is named as an input file, any N and SAMPLE commands that have been specified are applied to that file before files are matched.
- Filter conditions defined by the FILTER command are preserved. Filtered cases are included in the merged file. If you want to exclude cases, use the SELECT IF command to specify selection criteria before running MATCH FILES.

Limitations

- Maximum 50 files can be combined on one MATCH FILES command.
- Maximum one BY subcommand. However, BY can specify multiple variables.
- The TEMPORARY command cannot be in effect if the active dataset is used as an input file.

Examples

Example

```
MATCH FILES FILE='/data/part1.sav'
  /FILE='/data/part2.sav'
  /FILE=*.
```

- MATCH FILES combines three files (the active dataset and two external IBM SPSS Statistics data files) in a parallel match. Cases are combined according to their order in each file.
- The new active dataset contains as many cases as are contained in the largest of the three input files.

Example

```
GET FILE='/examples/data/mydata.sav'.
SORT CASES BY ID.
DATASET NAME mydata.
GET DATA /TYPE=XLS
  /FILE='/examples/data/excelfile.xls'.
SORT CASES BY ID.
DATASET NAME excelfile.
GET DATA /TYPE=ODBC /CONNECT=
 'DSN=MS Access Database;DBQ=/examples/data/dm_demo.mdb;'+
 'DriverId=25;FIL=MS Access;MaxBufferSize=2048;PageTimeout=5;'
 /SQL='SELECT * FROM main'.
SORT CASES BY ID.
MATCH FILES
 /FILE='mydata'
 /FILE='excelfile'
 /FILE=*
 /BY ID.
```

- A data file in IBM SPSS Statistics format and assigned the dataset name *mydata*. Since it has been assigned a dataset name, it remains available for subsequent use even after other data sources have been opened.
- An Excel file is then read and assigned the dataset name *excelfile*. Like the IBM SPSS Statistics data file, since it has been assigned a dataset name, it remains available after other data sources have been opened.
- Then a table from a database is read. Since it is the most recently opened or activated dataset, it is the active dataset.
- The three datasets are then merged together with MATCH FILES command, using the dataset names on the FILE subcommands instead of file names.
- An asterisk (*) is used to specify the active dataset, which is the database table in this example.
- The files are merged together based on the value of the key variable *ID*, specified on the BY subcommand.
- Since all the files being merged need to be sorted in the same order of the key variable(s), SORT CASES is performed on each dataset.

FILE Subcommand

FILE identifies the files to be combined (except table files). At least one FILE subcommand is required on MATCH FILES. A separate FILE subcommand must be used for each input file.

- An asterisk can be specified on FILE to refer to the active dataset.
- Dataset names instead of filenames can be used to refer to currently open datasets.
- The order in which files are specified determines the order of variables in the new active dataset. In addition, if the same variable name occurs in more than one input file, the variable is taken from the file specified first.

- If the files have unequal numbers of cases, cases are generated from the longest file. Cases that do not exist in the shorter files have system-missing values for variables that are unique to those files.

PASSWORD Keyword

The PASSWORD keyword specifies the password required to open an encrypted IBM SPSS Statistics data file. The specified value must be enclosed in quotation marks and can be provided as encrypted or as plain text. Encrypted passwords are created when pasting command syntax from the Save Data As dialog. The PASSWORD keyword is ignored if the file is not encrypted.

Text Data Files

You can add variables from one or more text data files by reading the files into IBM SPSS Statistics (with DATA LIST or GET DATA), defining dataset names for each file (DATASET NAME command), and then using MATCH FILES to add the cases from each file.

Example

```
DATA LIST FILE="/data/textdata1.txt"
  /id 1-3 var1 5-7 var2 9-12.
SORT CASES by ID.
DATASET NAME file1.
DATA LIST FILE="/data/textdata2.txt"
  /id 1-3 var3 5-9 var4 11-15.
SORT CASES BY ID.
DATASET NAME file2.
DATA LIST FILE="/data/textdata3.txt"
  /id 1-3 var5 5-6 var6 8-10.
DATASET NAME file3.
MATCH FILES FILE='file1'
  /FILE='file2'
  /FILE='file3'
  /BY id.
SAVE OUTFILE='/data/combined_data.sav'.
```

BY Subcommand

BY specifies one or more identification, or key, variables that determine which cases are to be combined. When BY is specified, cases from one file are matched only with cases from other files that have the same values for the key variables. BY is required unless all input files are to be matched sequentially according to the order of cases.

- BY must follow the FILE and TABLE subcommands and any associated RENAME and IN subcommands.
- BY specifies the names of one or more key variables. The key variables must exist in all input files. The key variables can be numeric or long or short strings.
- All input files must be sorted in ascending order of the key variables. If necessary, use SORT CASES before MATCH FILES.
- Missing values for key variables are handled like any other values.
- Unmatched cases are assigned system-missing values (for numeric variables) or blanks (for string variables) for variables from files that do not contain a match.

Duplicate Cases

Duplicate cases are those with the same values for the key variables named on the BY subcommand.

- Duplicate cases are permitted in any input files except table files.
- When there is no table file, the first duplicate case in each file is matched with the first matching case (if any) from the other files; the second duplicate case is matched with a second matching duplicate, if any; and so on. (In effect, a parallel match is performed within groups of duplicate cases.) Unmatched cases are assigned system-missing values (for numeric variables) or blanks (for string variables) for variables from files that do not contain a match.
- The program displays a warning if it encounters duplicate keys in one or more of the files being matched.

TABLE Subcommand

TABLE specifies a table lookup (or keyed table) file. A lookup file contributes variables but not cases to the new active dataset. Variables from the table file are added to all cases from other files that have matching values for the key variables. FILE specifies the files that supply the cases.

- A separate TABLE subcommand must be used to specify each lookup file, and a separate FILE subcommand must be used to specify each case file.
- The BY subcommand is required when TABLE is used.
- All specified files must be sorted in ascending order of the key variables. If necessary, use SORT CASES before MATCH FILES.
- A lookup file cannot contain duplicate cases (cases for which the key variable[s] named on BY have identical values).
- An asterisk on TABLE refers to the active dataset.
- Dataset names instead of file names can be used to refer to currently open datasets.
- Cases in a case file that do not have matches in a table file are assigned system-missing values (for numeric variables) or blanks (for string variables) for variables from that table file.
- Cases in a table file that do not match any cases in a case file are ignored.

PASSWORD Keyword

The PASSWORD keyword specifies the password required to open an encrypted IBM SPSS Statistics data file. The specified value must be enclosed in quotation marks and can be provided as encrypted or as plain text. Encrypted passwords are created when pasting command syntax from the Save Data As dialog. The PASSWORD keyword is ignored if the file is not encrypted.

For an alternative to MATCH CASES with table lookup files that does not require sorting on key variables, see STAR JOIN.

Example

```
MATCH FILES FILE=*
  /TABLE='/data/primary.sav'
  /BY EMP_ID.
```

- MATCH FILES combines variables from the IBM SPSS Statistics data file *primary.sav* with the active dataset, matching cases by the variable *EMP_ID*.
- No new cases are added to the active dataset as a result of the table lookup.
- Cases whose value for *EMP_ID* is not included in the *primary.sav* file are assigned system-missing values for variables taken from the table.

RENAME Subcommand

RENAME renames variables on the input files *before* they are processed by MATCH FILES. RENAME must follow the FILE or TABLE subcommand that contains the variables to be renamed.

- RENAME applies only to the immediately preceding FILE or TABLE subcommand. To rename variables from more than one input file, specify a RENAME subcommand after each FILE or TABLE subcommand.
- Specifications for RENAME consist of a left parenthesis, a list of old variable names, an equals sign, a list of new variable names, and a right parenthesis. The two variable lists must name or imply the same number of variables. If only one variable is renamed, the parentheses are optional.
- More than one rename specification can be specified on a single RENAME subcommand, each enclosed in parentheses.
- The TO keyword can be used to refer to consecutive variables in the file and to generate new variable names.

- RENAME takes effect immediately. Any KEEP and DROP subcommands entered prior to a RENAME must use the old names, while KEEP and DROP subcommands entered after a RENAME must use the new names.
- All specifications within a single set of parentheses take effect simultaneously. For example, the specification RENAME (A,B = B,A) swaps the names of the two variables.
- Variables cannot be renamed to scratch variables.

Example

```
MATCH FILES FILE='/data/update.sav'
 /RENAME=(NEWID = ID)
 /FILE='/data/primary.sav'
 /BY ID.
```

- MATCH FILES matches a primary IBM SPSS Statistics data file (*primary.sav*) with an update data file (*update.sav*).
- The variable *NEWID* in the *update.sav* file is renamed *ID* so that it will have the same name as the identification variable in the primary file and can be used on the BY subcommand.

DROP and KEEP Subcommands

DROP and KEEP are used to include a subset of variables in the new active dataset. DROP specifies a set of variables to exclude and KEEP specifies a set of variables to retain.

- DROP and KEEP do not affect the input files on disk.
- DROP and KEEP must follow all FILE, TABLE, and RENAME subcommands.
- DROP and KEEP must specify one or more variables. If RENAME is used to rename variables, specify the new names on DROP and KEEP.
- The keyword ALL can be specified on KEEP. ALL must be the last specification on KEEP, and it refers to all variables not previously named on KEEP.
- DROP cannot be used with variables created by the IN, FIRST, or LAST subcommands.
- KEEP can be used to change the order of variables in the resulting file. By default, MATCH FILES first copies the variables in order from the first file, then copies the variables in order from the second file, and so on. With KEEP, variables are kept in the order in which they are listed on the subcommand. If a variable is named more than once on KEEP, only the first mention of the variable is in effect; all subsequent references to that variable name are ignored.

Example

```
MATCH FILES FILE='/data/particle.sav'
  /RENAME=(PARTIC=POLLUTE1)
  /FILE='/data/gas.sav'
  /RENAME=(OZONE TO SULFUR=POLLUTE2 TO POLLUTE4)
  /DROP=POLLUTE4.
```

- The renamed variable *POLLUTE4* is dropped from the resulting file. DROP is specified after all of the FILE and RENAME subcommands, and it refers to the dropped variable by its new name.

IN Subcommand

IN creates a new variable in the resulting file that indicates whether a case came from the input file named on the preceding FILE subcommand. IN applies only to the file specified on the immediately preceding FILE subcommand.

- IN can be used only for a nonparallel match or table lookup.
- IN has only one specification—the name of the flag variable.
- The variable created by IN has the value 1 for every case that came from the associated input file and the value 0 if the case came from a different input file.

- Variables created by IN are automatically attached to the end of the resulting file and cannot be dropped. If FIRST or LAST is used, the variable created by IN precedes the variables created by FIRST or LAST.

Example

```
MATCH FILES  FILE='/data/week10.sav'
  /FILE='/data/week11.sav'
  /IN=INWEEK11
  /BY=EMPID.
```

- IN creates the variable *INWEEK11*, which has the value 1 for all cases in the resulting file that had values in the input file *week11.sav* and the value 0 for those cases that were not in file *week11.sav*.

FIRST and LAST Subcommands

FIRST and LAST create logical variables that flag the first or last case of a group of cases with the same value for the BY variables.

- FIRST and LAST must follow all TABLE and FILE subcommands and any associated RENAME and IN subcommands.
- FIRST and LAST have only one specification—the name of the flag variable.
- FIRST creates a variable with the value 1 for the first case of each group and the value 0 for all other cases.
- LAST creates a variable with the value 1 for the last case of each group and the value 0 for all other cases.
- Variables created by FIRST and LAST are automatically attached to the end of the resulting file and cannot be dropped.
- If one file has several cases with the same values for the key variables, FIRST or LAST can be used to create a variable that flags the first or last case of the group.

Example

```
MATCH FILES  TABLE='/data/house.sav'
 /FILE='/data/persons.sav'
 /BY=HOUSEID /FIRST=HEAD.
```

- The variable *HEAD* contains the value 1 for the first person in each household and the value 0 for all other persons. Assuming that the *persons.sav* file is sorted with the head of household as the first case for each household, the variable *HEAD* identifies the case for the head of household.

Example

```
* Using match files with only one file.

* This example flags the first of several cases with
  the same value for a key variable.

MATCH FILES  FILE='/data/persons.sav'
 /BY HOUSEID /FIRST=HEAD.
SELECT IF  (HEAD EQ 1).
CROSSTABS  JOBCAT BY SEX.
```

- MATCH FILES is used instead of GET to read the data file *persons.sav*. The BY subcommand identifies the key variable (*HOUSEID*), and FIRST creates the variable *HEAD* with the value 1 for the first case in each household and the value 0 for all other cases.
- SELECT IF selects only the cases with the value 1 for *HEAD*, and the CROSSTABS procedure is run on these cases.

MAP Subcommand

MAP produces a list of the variables that are in the new active dataset and the file or files from which they came. Variables are listed in the order in which they appear in the resulting file. MAP has no specifications and must be placed after all FILE, TABLE, and RENAME subcommands.

- Multiple MAP subcommands can be used. Each MAP shows the current status of the active dataset and reflects only the subcommands that precede the MAP subcommand.
- To obtain a map of the resulting file in its final state, specify MAP last.
- If a variable is renamed, its original and new names are listed. Variables created by IN, FIRST, and LAST are not included in the map, since they are automatically attached to the end of the file and cannot be dropped.

MATRIX-END MATRIX

```
MATRIX

matrix statements

END MATRIX
```

The following matrix language statements can be used in a matrix program: BREAK, CALL, COMPUTE, DISPLAY, DO IF, ELSE, ELSE IF, END IF, END LOOP, GET, LOOP, MGET, MSAVE, PRINT, READ, RELEASE, SAVE, WRITE.

For information on which functions can be used in matrix language statements, see “Matrix Functions ” on page 1077.

Example

```
MATRIX.
READ A /FILE=MATRDATA /SIZE={6,6} /FIELD=1 TO 60.
CALL EIGEN(A,EIGENVEC,EIGENVAL).
LOOP J=1 TO NROW(EIGENVAL).
+ DO IF (EIGENVAL(J) > 1.0).
+   PRINT EIGENVAL(J) / TITLE="Eigenvalue:" /SPACE=3.
+   PRINT T(EIGENVEC(:,J)) / TITLE="Eigenvector:" /SPACE=1.
+ END IF.
END LOOP.
END MATRIX.
```

Release History

Statistics 27.0

- The **NCDF.BETA** cumulative distribution function is now supported.
- Probability density functions are now supported (they were previously only supported by the **COMPUTE** command).
- Tail probability functions are now supported (they were previously only supported by the **COMPUTE** command).
- Random variable functions are now supported (they were previously only supported by the **COMPUTE** command).

Subscription November 2017 update / Statistics 26.0

- Long variable names (up to 64 bytes) can be used to name a matrix or vector name (such as COMPUTE, CALL, PRINT, READ, WRITE, GET, SAVE, MGET, MSAVE, DISPLAY, RELEASE, and so on).
- Variable names that are included in a vector or matrix object are truncated to 8 bytes. This is because the matrix/vector structure is an array of numbers, and each number can match a string only up to 8 bytes. Long names (up to 64 bytes) are supported only when explicitly specified.
- Long variable names are supported in GET and SAVE commands when explicitly specified on the /VARIABLES subcommand (and when specified on the /STRINGS subcommand for the SAVE command). Variable names for GET and SAVE commands are truncated to 8 bytes when they are referenced through a vector in the /NAMES subcommand.
- The GET, SAVE, MGET, or MSAVE statements support both dataset references and physical file specifications.
- **MATRIX-END MATRIX** supports statistical functions that were previously only supported by the **COMPUTE** command (for example **IDF.CHISQ**, **CDF.NORMAL**, **NCDF.F**, and so on). For more information, see Matrix Functions (MATRIX-END MATRIX command).

Overview

The MATRIX and END MATRIX commands enclose statements that are executed by the matrix processor. Using matrix programs, you can write your own statistical routines in the compact language of matrix

algebra. Matrix programs can include mathematical calculations, control structures, display of results, and reading and writing matrices as character files or data files.

As discussed below, a matrix program is for the most part independent of the rest of the session, although it can read and write IBM SPSS Statistics data files, including the active dataset.

This section does not attempt to explain the rules of matrix algebra. Many textbooks teach the application of matrix methods to statistics.

The MATRIX procedure was originally developed at the Madison Academic Computing Center, University of Wisconsin.

Using open datasets

Use of datasets that are currently open in a session is not supported, except as specifically noted for the active dataset.

Terminology

A variable within a matrix program represents a **matrix**, which is simply a set of values arranged in a rectangular array of rows and columns.

- An $n \times m$ (read "n by m") matrix is one that has n rows and m columns. The integers n and m are the dimensions of the matrix. An $n \times m$ matrix contains $n \times m$ elements, or data values.
- An $n \times 1$ matrix is sometimes called a **column vector**, and a $1 \times n$ matrix is sometimes called a **row vector**. A vector is a special case of a matrix.
- A 1×1 matrix, containing a single data value, is often called a **scalar**. A scalar is also a special case of a matrix.
- An **index** to a matrix or vector is an integer that identifies a specific row or column. Indexes normally appear in printed works as subscripts, as in A_{31}, but are specified in the matrix language within parentheses, as in $A(3,1)$. The row index for a matrix precedes the column index.
- The **main diagonal** of a matrix consists of the elements whose row index equals their column index. It begins at the top left corner of the matrix; in a square matrix, it runs to the bottom right corner.
- The **transpose** of a matrix is the matrix with rows and columns interchanged. The transpose of an $n \times m$ matrix is an $m \times n$ matrix.
- A **symmetric matrix** is a square matrix that is unchanged if you flip it about the main diagonal. That is, the element in row i, column j equals the element in row j, column i. A symmetric matrix equals its transpose.
- Matrices are always rectangular, although it is possible to read or write symmetric matrices in triangular form. Vectors and scalars are considered degenerate rectangles.
- It is an error to try to create a matrix whose rows have different numbers of elements.

A matrix program does not process individual cases unless you so specify, using the control structures of the matrix language. Unlike ordinary IBM SPSS Statistics variables, matrix variables do not have distinct values for different cases. A matrix is a single entity.

Vectors in matrix processing should not be confused with the vectors temporarily created by the VECTOR command. The latter are shorthand for a list of IBM SPSS Statistics variables and, like all ordinary IBM SPSS Statistics variables, are unavailable during matrix processing.

Matrix Variables

A matrix variable is created by a matrix statement that assigns a value to a variable name.

- A matrix variable name follows the same rules as those applicable to an ordinary IBM SPSS Statistics variable name.

- The names of matrix functions and procedures cannot be used as variable names within a matrix program. (In particular, the letter *T* cannot be used as a variable name because T is an alias for the TRANSPOS function.)
- The COMPUTE, READ, GET, MGET, and CALL statements create matrices. An index variable named on a LOOP statement creates a scalar with a value assigned to it.
- A variable name can be redefined within a matrix program without regard to the dimensions of the matrix it represents. The same name can represent scalars, vectors, and full matrices at different points in the matrix program.
- MATRIX-END MATRIX does not include any special processing for missing data. When reading a data matrix from a data file, you must therefore specify whether missing data are to be accepted as valid or excluded from the matrix.

String Variables in Matrix Programs

Matrix variables can contain short string data. Support for string variables is limited, however.

- MATRIX will attempt to carry out calculations with string variables if you so request. The results will not be meaningful.
- You must specify a format (such as A8) when you display a matrix that contains string data.

Support for Long Variable Names

- Long variable names (up to 64 bytes) can be used to name a matrix, vector, or scalar (such as COMPUTE, CALL, PRINT, READ, WRITE, GET, SAVE, MGET, MSAVE, DISPLAY, RELEASE, and so on).
- Variable names that are included in a vector or matrix object are truncated to 8 bytes. This is because the matrix/vector structure is an array of numbers, and each number can match a string only up to 8 bytes. Long names (up to 64 bytes) are supported only when explicitly specified.
- Long variable names are supported in GET and SAVE commands when explicitly specified on the /VARIABLES subcommand (and when specified on the /STRINGS subcommand for the SAVE command). Variable names for **GET** and **SAVE** commands are truncated to 8 bytes when they are referenced through a vector in the /NAMES subcommand.

Syntax of Matrix Language

A matrix program consists of statements. Matrix statements must appear in a matrix program, between the MATRIX and END MATRIX commands. They are analogous to IBM SPSS Statistics commands and follow the rules of the command language regarding the abbreviation of keywords; the equivalence of upper and lower case; the use of spaces, commas, and equals signs; and the splitting of statements across multiple lines. However, commas are required to separate arguments to matrix functions and procedures and to separate variable names on the RELEASE statement.

Matrix statements are composed of the following elements:

- Keywords, such as the names of matrix statements
- Variable names
- Explicitly written matrices, which are enclosed within braces ({})
- Arithmetic and logical operators
- Matrix functions
- The command terminator, which serves as a statement terminator within a matrix program

Comments in Matrix Programs

Within a matrix program, you can enter comments in any of the following forms: on lines beginning with the COMMENT command, on lines beginning with an asterisk, or between the characters /* and */ on a command line.

Matrix Notation

To write a matrix explicitly:

- Enclose the matrix within braces ({}).
- Separate the elements of each row by commas.
- Separate the rows by semicolons.
- String elements must be enclosed in either quotes, as is generally true in the command language.

Matrix Notation Shorthand

You can simplify the construction of matrices using notation shorthand.

Consecutive Integers. Use a colon to indicate a range of consecutive integers. For example, the vector `{1,2,3,4,5,6}` can be written as `{1:6}`.

Incremented Ranges of Integers. Use a second colon followed by an integer to indicate the increment. The matrix `{1,3,5,7;2,5,8,11}` can be written as `{1:7:2;2:11:3}`, where `1:7:2` indicates the integers from 1 to 7 incrementing by 2, and `2:11:3` indicates the integers from 2 to 11 incrementing by 3.

- You must use integers when specifying a range in either of these ways. Numbers with fractional parts are truncated to integers.
- If an arithmetic expression is used, it should be enclosed in parentheses.

Construction of a Matrix from Other Matrices

You can use vector or rectangular matrices to construct a new matrix, separating row expressions by semicolons and components of row expressions by commas. If a column vector V_c has n elements and matrix M has the dimensions $n \times m$, then $\{M; V_c\}$ is an $n \times (m + 1)$ matrix. Similarly, if the row vector V_r has m elements and M is the same, then $\{M; V_r\}$ is an $(n + 1) \times m$ matrix. In fact, you can paste together any number of matrices and vectors this way.

- All of the components of each column expression must have the same number of actual rows, and all of the row expressions must have the same number of actual columns.
- The distinction between row vectors and column vectors must be observed carefully when constructing matrices in this way, so that the components will fit together properly.
- Several of the matrix functions are also useful in constructing matrices; see in particular the `MAKE`, `UNIFORM`, and `IDENT` functions in “Matrix Functions ” on page 1077 .

Matrix Operations

You can perform matrix calculations according to the rules of matrix algebra and compare matrices using relational or logical operators.

Conformable Matrices

Many operations with matrices make sense only if the matrices involved have “suitable” dimensions. Most often, this means that they should be the same size, with the same number of rows and the same number of columns. Matrices that are the right size for an operation are said to be **conformable matrices**. If you attempt to do something in a matrix program with a matrix that is not conformable for that operation--a matrix that has the wrong dimensions--you will receive an error message, and the operation will not be performed. An important exception, where one of the matrices is a scalar, is discussed below.

Requirements for carrying out matrix operations include:

- Matrix addition and subtraction require that the two matrices be the same size.
- The relational and logical operations described below require that the two matrices be the same size.

- Matrix multiplication requires that the number of columns of the first matrix equal the number of rows of the second matrix.
- Raising a matrix to a power can be done only if the matrix is square. This includes the important operation of *inverting* a matrix, where the power is –1.
- Conformability requirements for matrix functions are noted in “Matrix Functions ” on page 1077 and in “COMPUTE Statement ” on page 1076 .

Scalar Expansion

When one of the matrices involved in an operation is a scalar, the scalar is treated as a matrix of the correct size in order to carry out the operation. This internal scalar expansion is performed for the following operations:

- Addition and subtraction.
- Elementwise multiplication, division, and exponentiation. Note that multiplying a matrix elementwise by an expanded scalar is equivalent to ordinary scalar multiplication--each element of the matrix is multiplied by the scalar.
- All relational and logical operators.

Arithmetic Operators

You can add, subtract, multiply, or exponentiate matrices according to the rules of matrix algebra, or you can perform elementwise arithmetic, in which you multiply, divide, or exponentiate each element of a matrix separately. The arithmetic operators are listed below.

Unary –. *Sign reversal.* A minus sign placed in front of a matrix reverses the sign of each element. (The unary + is also accepted but has no effect.)

+. *Matrix addition.* Corresponding elements of the two matrices are added. The matrices must have the same dimensions, or one must be a scalar.

–. *Matrix subtraction.* Corresponding elements of the two matrices are subtracted. The matrices must have the same dimensions, or one must be a scalar.

*. *Multiplication.* There are two cases. First, *scalar multiplication*: if either of the matrices is a scalar, each element of the other matrix is multiplied by that scalar. Second, *matrix multiplication*.

/. *Division.* The division operator performs elementwise division (described below). True matrix division, the inverse operation of matrix multiplication, is accomplished by taking the INV function (square matrices) or the GINV function (rectangular matrices) of the denominator and multiplying.

**. *Matrix exponentiation.* A matrix can be raised only to an integer power. The matrix, which must be square, is multiplied by itself as many times as the absolute value of the exponent. If the exponent is negative, the result is then inverted.

&*. *Elementwise multiplication.* Each element of the matrix is multiplied by the corresponding element of the second matrix. The matrices must have the same dimensions, or one must be a scalar.

&/. *Elementwise division.* Each element of the matrix is divided by the corresponding element of the second matrix. The matrices must have the same dimensions, or one must be a scalar.

&**. *Elementwise exponentiation.* Each element of the first matrix is raised to the power of the corresponding element of the second matrix. The matrices must have the same dimensions, or one must be a scalar.

:. *Sequential integers.* This operator creates a vector of consecutive integers from the value preceding the operator to the value following it. You can specify an optional increment following a second colon. See “Matrix Notation Shorthand ” on page 1072 for the principal use of this operator.

- Use these operators only with numeric matrices. The results are undefined when they are used with string matrices.

Relational Operators

The relational operators are used to compare two matrices, element by element. The result is a matrix of the same size as the (expanded) operands and containing either 1 or 0. The value of each element, 1 or 0, is determined by whether the comparison between the corresponding element of the first matrix with the corresponding element of the second matrix is true or false--1 for true and 0 for false. The matrices being compared must be of the same dimensions unless one of them is a scalar. The relational operators are listed in the following table.

Table 67. Relational operators in matrix programs

Symbol	Keyword	Definition
>	GT	Greater than
<	LT	Less than
<> or ~= (¬=)	NE	Not equal to
<=	LE	Less than or equal to
>=	GE	Greater than or equal to
=	EQ	Equal to

- The symbolic and alphabetic forms of these operators are equivalent.
- The symbols representing NE (~= or ¬=) are system dependent. In general, the tilde (~) is valid for ASCII systems, while the logical-not sign (¬), or whatever symbol is over the number 6 on the keyboard, is valid for IBM EBCDIC systems.
- Use these operators only with numeric matrices. The results are undefined when they are used with string matrices.

Logical Operators

Logical operators combine two matrices, normally containing values of 1 (true) or 0 (false). When used with other numerical matrices, they treat all positive values as true and all negative and 0 values as false. The logical operators are:

NOT. *Reverses the truth of the matrix that follows it.* Positive elements yield 0, and negative or 0 elements yield 1.

AND. *Both must be true.* The matrix *A* AND *B* is 1 where the corresponding elements of *A* and *B* are both positive and 0 elsewhere.

OR. *Either must be true.* The matrix *A* 0R *B* is 1 where the corresponding element of either *A* or *B* is positive and 0 where both elements are negative or 0.

XOR. *Either must be true but not both.* The matrix *A* X0R *B* is 1 where one but not both of the corresponding elements of *A* and *B* is positive and 0 where both are positive or neither is positive.

Precedence of Operators

Parentheses can be used to control the order in which complex expressions are evaluated. When the order of evaluation is not specified by parentheses, operations are carried out in the order listed below. The operations higher on the list take precedence over the operations lower on the list.

+ – (unary)

:

** &**

* &* &/

+ – (addition and subtraction)

> >= < <= <>=

NOT

AND

OR XOR

Operations of equal precedence are performed left to right of the expressions.

Examples

```
COMPUTE A = {1,2,3;4,5,6}.
COMPUTE B = A + 4.
COMPUTE C = A &** 2.
COMPUTE D = 2 &** A.
COMPUTE E = A < 5.
COMPUTE F = (C &/ 2) < B.
```

MATRIX and Other Commands

A matrix program is a single procedure within a session.

- No active dataset is needed to run a matrix program. If one exists, it is ignored during matrix processing unless you specifically reference it (with an asterisk) on the GET, SAVE, MGET, or MSAVE statements.
- The GET, SAVE, MGET, or MSAVE statements support both dataset references and physical file specifications. Take the following example:

```
DATASET CLOSE ALL.
NEW FILE.

GET FILE 'Samples\English\Employee data.sav'.

DATASET DECLARE corr.
CORRELATIONS bdate to minority /MATRIX=out(corr).
MATRIX.
MGET  /file=corr/type=corr.
PRINT CR.
END MATRIX.
```

  The example uses `corr` as a reference to a virtual dataset. The correlation output can also be saved to a physical file.
- Variables defined in the active dataset are unavailable during matrix processing, except with the GET or MGET statements.
- Matrix variables are unavailable after the END MATRIX command unless you use SAVE or MSAVE to write them to the active dataset.
- You cannot run a matrix program from a syntax window if split-file processing is in effect. If you save the matrix program into a syntax file, however, you can use the INCLUDE command to run the program even if split-file processing is in effect.

Matrix Statements

Most valid matrix statements have the same name as an analogous IBM SPSS Statistics command and perform an exactly analogous function. Use only these statements between the MATRIX and END MATRIX commands. Any command not recognized as a valid matrix statement will be rejected by the matrix processor.

Valid matrix statements are: BREAK, CALL, COMPUTE, DISPLAY, DO IF, ELSE, ELSE IF, END IF, END LOOP, GET, LOOP, MGET, MSAVE, PRINT, READ, RELEASE, SAVE, WRITE.

There is a maximum of 100 SAVE commands in a matrix program.

Exchanging Data with IBM SPSS Statistics Data Files

Matrix programs can read and write IBM SPSS Statistics data files.

- The GET and SAVE statements read and write ordinary (case-oriented) IBM SPSS Statistics data files, treating each case as a row of a matrix and each ordinary variable as a column.

A matrix program cannot contain more than 100 SAVE commands.

- The MGET and MSAVE statements read and write matrix-format data files, respecting the structure defined by IBM SPSS Statistics when it creates the file.
- Case weighting in a data file is ignored when the file is read into a matrix program.

MATRIX and END MATRIX Commands

The MATRIX command, when encountered in a session, invokes the matrix processor, which reads matrix statements until the END MATRIX or FINISH command is encountered.

- MATRIX is a procedure and cannot be entered inside a transformation structure such as DO IF or LOOP.
- The MATRIX procedure does not require an active dataset.
- Comments are removed before subsequent lines are passed to the matrix processor.
- Macros are expanded before subsequent lines are passed to the matrix processor.

The END MATRIX command terminates matrix processing and returns control to the command processor.

- The contents of matrix variables are lost after an END MATRIX command.
- The active dataset, if present, becomes available again after an END MATRIX command.

COMPUTE Statement

The COMPUTE statement carries out most of the calculations in the matrix program. It closely resembles the COMPUTE command in the IBM SPSS Statistics transformation language.

- The basic specification is the target variable, an equals sign, and the assignment expression. Values of the target variable are calculated according to the specification on the assignment expression.
- The target variable must be named first, and the equals sign is required. Only one target variable is allowed per COMPUTE statement.
- Expressions that extract portions of a matrix, such as *M*(1,:) or *M*(1:3,4), are allowed to assign values. (See the topic “Matrix Notation Shorthand ” on page 1072 for more information.) The target variable must be specified as a variable.
- Matrix functions must specify at least one argument enclosed in parentheses. If an expression has two or more arguments, each argument must be separated by a comma. For a complete discussion of the functions and their arguments, see “Matrix Functions ” on page 1077 .

String Values on COMPUTE Statements

Matrix variables, unlike those in the transformation language, are not checked for data type (numeric or string) when you use them in a COMPUTE statement.

- Numerical calculations with matrices containing string values will produce meaningless results.
- One or more elements of a matrix can be set equal to string constants by enclosing the string constants in quotes on a COMPUTE statement.
- String values can be copied from one matrix to another with the COMPUTE statement.
- There is no way to display a matrix that contains both numeric and string values, if you compute one for some reason.

Example

```
COMPUTE LABELS={"Observe", "Predict", "Error"}.
PRINT LABELS /FORMAT=A7.
```

- *LABELS* is a row vector containing three string values.

Arithmetic Operations and Comparisons

The expression on a COMPUTE statement can be formed from matrix constants and variables, combined with the arithmetic, relational, and logical operators. Matrix constructions and matrix functions are also allowed.

Examples

```
COMPUTE PI = 3.14159265.
COMPUTE RSQ = R * R.
COMPUTE FLAGS = EIGENVAL >= 1.
COMPUTE ESTIM = {OBS, PRED, ERR}.
```

- The first statement computes a scalar. Note that the braces are optional on a scalar constant.
- The second statement computes the square of the matrix *R*. *R* can be any square matrix, including a scalar.
- The third statement computes a vector named *FLAGS*, which has the same dimension as the existing vector *EIGENVAL*. Each element of *FLAGS* equals 1 if the corresponding element of *EIGENVAL* is greater than or equal to 1, and 0 if the corresponding element is less than 1.
- The fourth statement constructs a matrix *ESTIM* by concatenating the three vectors or matrices *OBS*, *PRED*, and *ERR*. The component matrices must have the same number of rows.

Matrix Functions

The following functions are available in the matrix program. Except where noted, each takes one or more numeric matrices as arguments and returns a matrix value as its result. The arguments must be enclosed in parentheses, and multiple arguments must be separated by commas.

On the following list, matrix arguments are represented by names beginning with *M*. Unless otherwise noted, these arguments can be vectors or scalars. Arguments that must be vectors are represented by names beginning with *V*, and arguments that must be scalars are represented by names beginning with *S*.

ABS(M). *Absolute value.* Takes a single argument. Returns a matrix having the same dimensions as the argument, containing the absolute values of its elements.

ALL(M). *Test for all elements nonzero.* Takes a single argument. Returns a scalar: 1 if all elements of the argument are nonzero and 0 if any element is zero.

ANY(M). *Test for any element nonzero.* Takes a single argument. Returns a scalar: 1 if any element of the argument is nonzero and 0 if all elements are zero.

ARSIN(M). *Inverse sine.* Takes a single argument, whose elements must be between −1 and 1. Returns a matrix having the same dimensions as the argument, containing the inverse sines (arcsines) of its elements. The results are in radians and are in the range from −π/2 to π/2.

ARTAN(M). *Inverse tangent.* Takes a single argument. Returns a matrix having the same dimensions as the argument, containing the inverse tangents (arctangents) of its elements, in radians. To convert radians to degrees, multiply by 180/π, which you can compute as 45/ARTAN(1). For example, the statement `COMPUTE DEGREES=ARTAN(M)*45/ARTAN(1)` returns a matrix containing inverse tangents in degrees.

BLOCK(M1,M2,...). *Create a block diagonal matrix.* Takes any number of arguments. Returns a matrix with as many rows as the sum of the rows in all the arguments, and as many columns as the sum of the columns in all the arguments, with the argument matrices down the diagonal and zeros elsewhere.

CHOL(M). *Cholesky decomposition.* Takes a single argument, which must be a symmetric positive-definite matrix (a square matrix, symmetric about the main diagonal, with positive eigenvalues). Returns a matrix having the same dimensions as the argument. If *M* is a symmetric positive-definite matrix and *B*=CHOL(*M*), then T(*B*)* *B*=*M*, where T is the transpose function defined below.

CMAX(M). *Column maxima.* Takes a single argument. Returns a row vector with the same number of columns as the argument. Each column of the result contains the maximum value of the corresponding column of the argument.

CMIN(M). *Column minima.* Takes a single argument. Returns a row vector with the same number of columns as the argument. Each column of the result contains the minimum value of the corresponding column of the argument.

COS(M). *Cosines.* Takes a single argument. Returns a matrix having the same dimensions as the argument, containing the cosines of the elements of the argument. Elements of the argument matrix are assumed to be measured in radians. To convert degrees to radians, multiply by π/180, which you can compute as ARTAN(1)/45. For example, the statement `COMPUTE COSINES=COS(DEGREES*ARTAN(1)/45)` returns cosines from a matrix containing elements measured in degrees.

CSSQ(M). *Column sums of squares.* Takes a single argument. Returns a row vector with the same number of columns as the argument. Each column of the result contains the sum of the squared values of the elements in the corresponding column of the argument.

CSUM(M). *Column sums.* Takes a single argument. Returns a row vector with the same number of columns as the argument. Each column of the result contains the sum of the elements in the corresponding column of the argument.

DESIGN(M). *Main-effects design matrix from the columns of a matrix.* Takes a single argument. Returns a matrix having the same number of rows as the argument, and as many columns as the sum of the numbers of unique values in each column of the argument. Constant columns in the argument are skipped with a warning message. The result contains 1 in the row(s) where the value in question occurs in the argument and 0 otherwise.

DET(M). *Determinant.* Takes a single argument, which must be a square matrix. Returns a scalar, which is the determinant of the argument.

DIAG(M). *Diagonal of a matrix.* Takes a single argument. Returns a column vector with as many rows as the minimum of the number of rows and the number of columns in the argument. The *i*th element of the result is the value in row *i*, column *i* of the argument.

EOF(file). *End of file indicator.* Normally used after a READ statement. Takes a single argument, which must be either a filename in quotes, or a file handle defined on a `FILE HANDLE` command that precedes the matrix program. Returns a scalar equal to 1 if the last attempt to read that file encountered the last record in the file, and equal to 0 if the last attempt did not encounter the last record in the file. Calling the EOF function causes a REREAD specification on the READ statement to be ignored on the next attempt to read the file.

EVAL(M). *Eigenvalues of a symmetric matrix.* Takes a single argument, which must be a symmetric matrix. Returns a column vector with the same number of rows as the argument, containing the eigenvalues of the argument in decreasing numerical order.

EXP(M). *Exponentials of matrix elements.* Takes a single argument. Returns a matrix having the same dimensions as the argument, in which each element equals *e* raised to the power of the corresponding element in the argument matrix.

GINV(M). *Moore-Penrose generalized inverse of a matrix.* Takes a single argument. Returns a matrix with the same dimensions as the transpose of the argument. If *A* is the generalized inverse of a matrix *M*, then *M*A*M=M* and *A*M*A=A*. Both *A*M* and *M*A* are symmetric.

GRADE(M). *Ranks elements in a matrix*. Takes a single argument. Uses sequential integers for ties.

GSCH(M). *Gram-Schmidt orthonormal basis for the space spanned by the column vectors of a matrix.* Takes a single argument, in which there must be as many linearly independent columns as there are rows. (That is, the rank of the argument must equal the number of rows.) Returns a square matrix with as many rows as the argument. The columns of the result form a basis for the space spanned by the columns of the argument.

IDENT(S1 [,S2]). *Create an identity matrix.* Takes either one or two arguments, which must be scalars. Returns a matrix with as many rows as the first argument and as many columns as the second argument, if any. If the second argument is omitted, the result is a square matrix. Elements on the main diagonal of the result equal 1, and all other elements equal 0.

INV(M). *Inverse of a matrix.* Takes a single argument, which must be square and nonsingular (that is, its determinant must not be 0). Returns a square matrix having the same dimensions as the argument. If *A* is the inverse of *M*, then *M*A=A*M=I*, where *I* is the identity matrix.

KRONEKER(M1,M2).

A(1,1)**B A*(1,2)* *B* ... *A*(1,*N*)**B*

A(2,1)**B A*(2,2)* *B* ... *A*(2,*N*)* *B*

...

A(*M*,1)**B A*(*M*,2)**B* ... *A*(*M*, *N*)**B*

LG10(M). *Base 10 logarithms of the elements.* Takes a single argument, all of whose elements must be positive. Returns a matrix having the same dimensions as the argument, in which each element is the logarithm to base 10 of the corresponding element of the argument.

LN(M). *Natural logarithms of the elements.* Takes a single argument, all of whose elements must be positive. Returns a matrix having the same dimensions as the argument, in which each element is the logarithm to base *e* of the corresponding element of the argument.

MAGIC(S). *Magic square.* Takes a single scalar, which must be 3 or larger, as an argument. Returns a square matrix with *S* rows and *S* columns containing the integers from 1 through S^2. All of the row sums and all of the column sums are equal in the result matrix. (The result matrix is only one of several possible magic squares.)

MAKE(S1,S2,S3). *Create a matrix, all of whose elements equal a specified value.* Takes three scalars as arguments. Returns an *S*1 × *S*2 matrix, all of whose elements equal *S*3.

MDIAG(V). *Create a square matrix with a specified main diagonal.* Takes a single vector as an argument. Returns a square matrix with as many rows and columns as the dimension of the vector. The elements of the vector appear on the main diagonal of the matrix, and the other matrix elements are all 0.

MMAX(M). *Maximum element in a matrix.* Takes a single argument. Returns a scalar equal to the numerically largest element in the argument *M*.

MMIN(M). *Minimum element in a matrix.* Takes a single argument. Returns a scalar equal to the numerically smallest element in the argument *M*.

MOD(M,S). *Remainders after division by a scalar.* Takes two arguments, a matrix and a scalar (which must not be 0). Returns a matrix having the same dimensions as *M*, each of whose elements is the remainder after the corresponding element of *M* is divided by *S*. The sign of each element of the result is the same as the sign of the corresponding element of the matrix argument *M*.

MSSQ(M). *Matrix sum of squares.* Takes a single argument. Returns a scalar that equals the sum of the squared values of all of the elements in the argument.

MSUM(M). *Matrix sum.* Takes a single argument. Returns a scalar that equals the sum of all of the elements in the argument.

NCOL(M). *Number of columns in a matrix.* Takes a single argument. Returns a scalar that equals the number of columns in the argument.

NROW(M). *Number of rows in a matrix.* Takes a single argument. Returns a scalar that equals the number of rows in the argument.

RANK(M). *Rank of a matrix.* Takes a single argument. Returns a scalar that equals the number of linearly independent rows or columns in the argument.

RESHAPE(M,S1,S2). *Matrix of different dimensions.* Takes three arguments, a matrix and two scalars, whose product must equal the number of elements in the matrix. Returns a matrix whose dimensions are given by the scalar arguments. For example, if *M* is any matrix with exactly 50 elements, then `RESHAPE(M, 5, 10)` is a matrix with 5 rows and 10 columns. Elements are assigned to the reshaped matrix in order by row.

RMAX(M). *Row maxima.* Takes a single argument. Returns a column vector with the same number of rows as the argument. Each row of the result contains the maximum value of the corresponding row of the argument.

RMIN(M). *Row minima.* Takes a single argument. Returns a column vector with the same number of rows as the argument. Each row of the result contains the minimum value of the corresponding row of the argument.

RND(M). *Elements rounded to the nearest integers.* Takes a single argument. Returns a matrix having the same dimensions as the argument. Each element of the result equals the corresponding element of the argument rounded to an integer.

RNKORDER(M). *Ranking of matrix elements in ascending order.* Takes a single argument. Returns a matrix having the same dimensions as the argument *M*. The smallest element of the argument corresponds to a result element of 1, and the largest element of the argument to a result element equal to the number of elements, except that ties (equal elements in *M*) are resolved by assigning a rank equal to the arithmetic mean of the applicable ranks.

RSSQ(M). *Row sums of squares.* Takes a single argument. Returns a column vector having the same number of rows as the argument. Each row of the result contains the sum of the squared values of the elements in the corresponding row of the argument.

RSUM(M). *Row sums.* Takes a single argument. Returns a column vector having the same number of rows as the argument. Each row of the result contains the sum of the elements in the corresponding row of the argument.

SIN(M). *Sines.* Takes a single argument. Returns a matrix having the same dimensions as the argument, containing the sines of the elements of the argument. Elements of the argument matrix are assumed to be measured in radians. To convert degrees to radians, multiply by π/180, which you can compute as ARTAN(1)/45. For example, the statement `COMPUTE SINES=SIN(DEGREES*ARTAN(1)/45)` computes sines from a matrix containing elements measured in degrees.

SOLVE(M1,M2). *Solution of systems of linear equations.* Takes two arguments, the first of which must be square and nonsingular (its determinant must be nonzero), and the second of which must have the same number of rows as the first. Returns a matrix with the same dimensions as the second argument. If *M1*X=M2*, then *X*= SOLVE(*M1*, *M2*). In effect, this function sets its result *X* equal to INV(*M1*)**M2*.

SQRT(M). *Square roots of elements.* Takes a single argument whose elements must not be negative. Returns a matrix having the same dimensions as the arguments, whose elements are the positive square roots of the corresponding elements of the argument.

SSCP(M). *Sums of squares and cross-products.* Takes a single argument. Returns a square matrix having as many rows (and columns) as the argument has columns. SSCP *(M)* equals T *(M)*M*, where T is the transpose function defined below.

SVAL(M). *Singular values of a matrix.* Takes a single argument. Returns a column vector containing as many rows as the minimum of the numbers of rows and columns in the argument, containing the singular values of the argument in decreasing numerical order. The singular values of a matrix *M* are the square roots of the eigenvalues of T *(M)*M*, where T is the transpose function discussed below.

SWEEP(M,S).

Sweep transformation of a matrix. Takes two arguments, a matrix and a scalar, which must be less than or equal to both the number of rows and the number of columns of the matrix. In other words, the pivot element of the matrix, which is *M*(*S*,*S*), must exist. Returns a matrix of the same dimensions as *M*. Suppose that *S*={ *k*} and *A* = SWEEP(*M*,*S*). If *M*(*k,k*) is not 0, then

A(*k*,*k*) = 1/*M*(*k*,*k*)

A(*i*,*k*) = –*M*(*i*,*k*)/*M*(*k*,*k*), for *i* not equal to *k*

A(*k*,*j*) = *M*(*k*,*j*)/*M*(*k*,*k*), for *j* not equal to *k*

A(*i*,*j*) = (*M*(*i*,*j*)**M*(*k*,*k*), – *M*(*i*,*k*)**M*(*k*,*j*))/*M*(*k*,*k*), for *i,j* not equal to *k*

and if *M*(*k,k*) equals 0, then

$A(i,k) = A(k,i) = 0$, for all i

$A(i,j) = M(i,j)$, for i,j not equal to k

TRACE(M). *Sum of the main diagonal elements.* Takes a single argument. Returns a scalar, which equals the sum of the elements on the main diagonal of the argument.

TRANSPOS(M). *Transpose of the matrix.* Takes a single argument. Returns the transpose of the argument. TRANSP0S can be shortened to T.

TRUNC(M). *Truncation of elements to integers.* Takes a single argument. Returns a matrix having the same dimensions as the argument, whose elements equal the corresponding elements of the argument truncated to integers.

UNIFORM(S1,S2). *Uniformly distributed pseudo-random numbers between 0 and 1.* Takes two scalars as arguments. Returns a matrix with the number of rows specified by the first argument and the number of columns specified by the second argument, containing pseudo-random numbers uniformly distributed between 0 and 1. The function respects the traditional versus twister and seed settings for the corresponding generator.

MATRIX probability density functions

The following functions give the value of the density function with the specified distribution at the value *quant*, the first argument. All functions accept matrix, vector, or scalar as the first argument. The **PDF . BVN0R** function accepts only scalar as the third argument. The result has the same dimension as the first argument. Note the period in each function name.

PDF.BERNOULLI. PDF.BERNOULLI(quant, prob). Numeric. Returns the probability that a value from the Bernoulli distribution, with the given probability parameter, will be equal to quant.

PDF.BETA. PDF.BETA(quant, shape1, shape2). Numeric. Returns the probability density of the beta distribution, with the given shape parameters, at quant.

PDF.BINOM. PDF.BINOM(quant, n, prob). Numeric. Returns the probability that the number of successes in n trials, with probability prob of success in each, will be equal to quant. When n is 1, this is the same as PDF.BERNOULLI.

PDF.BVNOR. PDF.BVNOR(quant1, quant2, corr). Numeric. Returns the probability density of the standard bivariate normal distribution, with the given correlation parameter, at quant1, quant2.

PDF.CAUCHY. PDF.CAUCHY(quant, loc, scale). Numeric. Returns the probability density of the Cauchy distribution, with the given location and scale parameters, at quant.

PDF.CHISQ. PDF.CHISQ(quant, df). Numeric. Returns the probability density of the chi-square distribution, with df degrees of freedom, at quant.

PDF.EXP. PDF.EXP(quant, shape). Numeric. Returns the probability density of the exponential distribution, with the given shape parameter, at quant.

PDF.F. PDF.F(quant, df1, df2). Numeric. Returns the probability density of the F distribution, with degrees of freedom df1 and df2, at quant.

PDF.GAMMA. PDF.GAMMA(quant, shape, scale). Numeric. Returns the probability density of the gamma distribution, with the given shape and scale parameters, at quant.

PDF.GEOM. PDF.GEOM(quant, prob). Numeric. Returns the probability that the number of trials to obtain a success, when the probability of success is given by prob, will be equal to quant.

PDF.HALFNRM. PDF.HALFNRM(quant, mean, stddev). Numeric. Returns the probability density of the half normal distribution, with specified mean and standard deviation, at quant.

PDF.HYPER. PDF.HYPER(quant, total, sample, hits). Numeric. Returns the probability that the number of objects with a specified characteristic, when sample objects are randomly selected from a universe of size total in which hits have the specified characteristic, will be equal to quant.

PDF.IGAUSS. PDF.IGAUSS(quant, loc, scale). Numeric. Returns the probability density of the inverse Gaussian distribution, with the given location and scale parameters, at quant.

PDF.LAPLACE. PDF.LAPLACE(quant, mean, scale). Numeric. Returns the probability density of the Laplace distribution, with the specified mean and scale parameters, at quant.

PDF.LOGISTIC. PDF.LOGISTIC(quant, mean, scale). Numeric. Returns the probability density of the logistic distribution, with the specified mean and scale parameters, at quant.

PDF.LNORMAL. PDF.LNORMAL(quant, a, b). Numeric. Returns the probability density of the log-normal distribution, with the specified parameters, at quant.

PDF.NEGBIN. PDF.NEGBIN(quant, thresh, prob). Numeric. Returns the probability that the number of trials to obtain a success, when the threshold parameter is thresh and the probability of success is given by prob, will be equal to quant.

PDF.NORMAL. PDF.NORMAL(quant, mean, stddev). Numeric. Returns the probability density of the normal distribution, with specified mean and standard deviation, at quant.

PDF.PARETO. PDF.PARETO(quant, threshold, shape). Numeric. Returns the probability density of the Pareto distribution, with the specified threshold and shape parameters, at quant.

PDF.POISSON. PDF.POISSON(quant, mean). Numeric. Returns the probability that a value from the Poisson distribution, with the specified mean or rate parameter, will be equal to quant.

PDF.T. PDF.T(quant, df). Numeric. Returns the probability density of Student's t distribution, with the specified degrees of freedom df, at quant.

PDF.UNIFORM. PDF.UNIFORM(quant, min, max). Numeric. Returns the probability density of the uniform distribution, with the specified minimum and maximum, at quant.

PDF.WEIBULL. PDF.WEIBULL(quant, a, b). Numeric. Returns the probability density of the Weibull distribution, with the specified parameters, at quant.

NPDF.BETA. NPDF.BETA(quant, shape1, shape2, nc). Numeric. Returns the probability density of the noncentral beta distribution, with the given shape and noncentrality parameters, at quant.

NPDF.CHISQ. NPDF.CHISQ(quant, df, nc). Numeric. Returns the probability density of the noncentral chi-square distribution, with df degrees of freedom and the specified noncentrality parameter, at quant.

NPDF.F. NPDF.F(quant, df1, df2, nc). Numeric. Returns the probability density of the noncentral F distribution, with degrees of freedom df1 and df2 and noncentrality nc, at quant.

NPDF.T. NPDF.T(quant, df, nc). Numeric. Returns the probability density of the noncentral Student's t distribution, with the specified degrees of freedom df and noncentrality nc, at quant.

MATRIX tail probability functions

The following functions give the probability that a random variable with the specified distribution will be greater than *quant*, the first argument. Subsequent arguments are the parameters of the distribution. Note the period in each function name.

SIG.CHISQ. SIG.CHISQ(quant, df). Numeric. Returns the cumulative probability that a value from the chi-square distribution, with df degrees of freedom, will be greater than quant

SIG.F. SIG.F(quant, df1, df2). Numeric. Returns the cumulative probability that a value from the F distribution, with degrees of freedom df1 and df2, will be greater than quant.

MATRIX cumulative distribution functions

The following functions give the probability that a random variable with the specified distribution will be less than *quant*, the first argument. All functions accept matrix, vector, or scalar as the first argument. All functions except **CDF.BVNOR** accept only scalar as the second argument. The result has the same dimension as the first argument. Note the period in each function name.

CDF.BERNOULLI. CDF.BERNOULLI(quant, prob). Numeric. Returns the cumulative probability that a value from the Bernoulli distribution, with the given probability parameter, will be less than or equal to quant.

CDF.BETA. CDF.BETA(quant, shape1, shape2). Numeric. Returns the cumulative probability that a value from the Beta distribution, with the given shape parameters, will be less than quant.

CDF.BINOM. CDF.BINOM(quant, n, prob). Numeric. Returns the cumulative probability that the number of successes in n trials, with probability prob of success in each, will be less than or equal to quant. When n is 1, this is the same as CDF.BERNOULLI.

CDF.BVNOR. CDF.BVNOR(quant1, quant2, corr). Numeric. Returns the cumulative probability that a value from the standard bivariate normal distribution, with the given correlation parameter, will be less than quant1 and quant2.

Both the first and second arguments can be matrices or vectors, in which case they should have the same dimensions and the result will have the same dimension as either of these arguments. It is possible that either first or second argument is a matrix or vector, and the other argument is scalar. In this case, the result has the same dimension as the matrix/vector. It is also possible that both the first and second arguments are scalars (in which case, the result will be scalar). The function's third argument should always be scalar.

CDF.CAUCHY. CDF.CAUCHY(quant, loc, scale). Numeric. Returns the cumulative probability that a value from the Cauchy distribution, with the given location and scale parameters, will be less than quant.

CDF.CHISQ. CDF.CHISQ(quant, df). Numeric and same as CHICDF. Chi-square cumulative distribution function of elements. Takes two arguments, a matrix of chi-square values and a scalar giving the degrees of freedom (which must be positive). Returns a matrix having the same dimensions as the first argument, containing the values of the cumulative chi-square distribution function for each of its elements. If an element of the first argument is *x* and the second argument is *S*, the corresponding element of the result is a number between 0 and 1, giving the proportion of a chi-square distribution with *S* degrees of freedom that is less than *x*. If *x* is not positive, the result is 0.

CDF.EXP. CDF.EXP(quant, scale). Numeric. Returns the cumulative probability that a value from the exponential distribution, with the given scale parameter, will be less than quant.

CDF.F. CDF.F(M,S1,S2). Numeric and same as FCDF. *Cumulative* F *distribution function of elements.* Takes three arguments, a matrix of *F* values and two scalars giving the degrees of freedom (which must be positive). Returns a matrix having the same dimensions as the first argument *M*, containing the values of the cumulative *F* distribution function for each of its elements. If an element of the first argument is *x* and the second and third arguments are *S1* and *S2*, the corresponding element of the result is a number between 0 and 1, giving the proportion of an *F* distribution with *S1* and *S2* degrees of freedom that is less than *x*. If *x* is not positive, the result is 0.

CDF.GAMMA. CDF.GAMMA(quant, shape, scale). Numeric. Returns the cumulative probability that a value from the Gamma distribution, with the given shape and scale parameters, will be less than quant.

CDF.GEOM. CDF.GEOM(quant, prob). Numeric. Returns the cumulative probability that the number of trials to obtain a success, when the probability of success is given by prob, will be less than or equal to quant.

CDF.HALFNRM. CDF.HALFNRM(quant, mean, stddev). Numeric. Returns the cumulative probability that a value from the half normal distribution, with specified mean and standard deviation, will be less than quant.

CDF.HYPER. CDF.HYPER(quant, total, sample, hits). Numeric. Returns the cumulative probability that the number of objects with a specified characteristic, when sample objects are randomly selected from a universe of size total in which hits have the specified characteristic, will be less than or equal to quant.

CDF.IGAUSS. CDF.IGAUSS(quant, loc, scale). Numeric. Returns the cumulative probability that a value from the inverse Gaussian distribution, with the given location and scale parameters, will be less than quant.

CDF.LAPLACE. CDF.LAPLACE(quant, mean, scale). Numeric. Returns the cumulative probability that a value from the Laplace distribution, with the specified mean and scale parameters, will be less than quant.

CDF.LNORMAL. CDF.LNORMAL(quant, a, b). Numeric. Returns the cumulative probability that a value from the log-normal distribution, with the specified parameters, will be less than quant.

CDF.LOGISTIC. CDF.LOGISTIC(quant, mean, scale). Numeric. Returns the cumulative probability that a value from the logistic distribution, with the specified mean and scale parameters, will be less than quant.

CDF.NEGBIN. CDF.NEGBIN(quant, thresh, prob). Numeric. Returns the cumulative probability that the number of trials to obtain a success, when the threshold parameter is thresh and the probability of success is given by prob, will be less than or equal to quant.

CDF.NORMAL. CDF.NORMAL(quant, mean, stddev). Numeric. Returns the cumulative probability that a value from the normal distribution, with specified mean and standard deviation, will be less than quant.

CDF.PARETO. CDF.PARETO(quant, threshold, shape). Numeric. Returns the cumulative probability that a value from the Pareto distribution, with the specified threshold and shape parameters, will be less than quant.

CDF.POISSON. CDF.POISSON(quant, mean). Numeric. Returns the cumulative probability that a value from the Poisson distribution, with the specified mean or rate parameter, will be less than or equal to quant.

CDF.SMOD. CDF.SMOD(quant, a, b). Numeric. Returns the cumulative probability that a value from the Studentized maximum modulus, with the specified parameters, will be less than quant.

CDF.SRANGE. CDF.SRANGE(quant, a, b). Numeric. Returns the cumulative probability that a value from the Studentized range statistic, with the specified parameters, will be less than quant.

CDF.T. CDF.T(M,S). Numeric and same as TCDF. *Cumulative* t *distribution function of elements.* Takes two arguments, a matrix of *t* values and a scalar giving the degrees of freedom (which must be positive). Returns a matrix having the same dimensions as *M*, containing the values of the cumulative *t* distribution function for each of its elements. If an element of the first argument is *x* and the second argument is *S*, then the corresponding element of the result is a number between 0 and 1, giving the proportion of a *t* distribution with *S* degrees of freedom that is less than *x*.

CDF.UNIFORM. CDF.UNIFORM(quant, min, max). Numeric. Returns the cumulative probability that a value from the uniform distribution, with the specified minimum and maximum, will be less than quant.

CDF.WEIBULL. CDF.WEIBULL(quant, a, b). Numeric. Returns the cumulative probability that a value from the Weibull distribution, with the specified parameters, will be less than quant.

CDFNORM(M). *Standard normal cumulative distribution function of elements.* Takes a single argument. Returns a matrix having the same dimensions as the argument, containing the values of the cumulative normal distribution function for each of its elements. If an element of the argument is *x*, the corresponding element of the result is a number between 0 and 1, giving the proportion of a normal distribution that is less than *x*. For example, `CDFNORM({-1.96,0,1.96})` results in, approximately, {.025,.5,.975}.

Note: CDFNORM(M) is a special case of CDF.NORMAL(x, mean, stddev), where mean = 0, stddev = 1.

CHICDF. CHICDF(M,S). Numeric and same as CDF.CHISQ. Chi-square cumulative distribution function of elements. Takes two arguments, a matrix of chi-square values and a scalar giving the degrees of freedom (which must be positive). Returns a matrix having the same dimensions as the first argument, containing the values of the cumulative chi-square distribution function for each of its elements. If an element of the first argument is *x* and the second argument is *S*, the corresponding element of the result is a number between 0 and 1, giving the proportion of a chi-square distribution with *S* degrees of freedom that is less than *x*. If *x* is not positive, the result is 0.

FCDF. FCDF(M,S1,S2). Numeric and same as CDF.F. *Cumulative* F *distribution function of elements.* Takes three arguments, a matrix of *F* values and two scalars giving the degrees of freedom (which must be positive). Returns a matrix having the same dimensions as the first argument *M*, containing the values of the cumulative *F* distribution function for each of its elements. If an element of the first argument is *x* and the second and third arguments are *S1* and *S2*, the corresponding element of the result is a number between 0 and 1, giving the proportion of an *F* distribution with *S1* and *S2* degrees of freedom that is less than *x*. If *x* is not positive, the result is 0.

NCDF.BETA. NCDF.BETA(quant, shape1, shape2, nc). Numeric. Returns the cumulative probability that a value from the noncentral Beta distribution, with the given shape and noncentrality parameters, will be less than quant.

NCDF.CHISQ. NCDF.CHISQ(quant, df, nc). Numeric. Returns the cumulative probability that a value from the noncentral chi-square distribution, with df degrees of freedom and the specified noncentrality parameter, will be less than quant.

NCDF.F. NCDF.F(quant, df1, df2, nc). Numeric. Returns the cumulative probability that a value from the noncentral F distribution, with degrees of freedom df1 and df2, and noncentrality nc, will be less than quant.

NCDF.T. NCDF.T(quant, df, nc). Numeric. Returns the cumulative probability that a value from the noncentral Student's t distribution, with the specified degrees of freedom df and noncentrality nc, will be less than quant.

TCDF. TCDF(M,S). Numeric and same as CDF.T. *Cumulative* t *distribution function of elements.* Takes two arguments, a matrix of *t* values and a scalar giving the degrees of freedom (which must be positive). Returns a matrix having the same dimensions as *M*, containing the values of the cumulative *t* distribution function for each of its elements. If an element of the first argument is *x* and the second argument is *S*, then the corresponding element of the result is a number between 0 and 1, giving the proportion of a *t* distribution with *S* degrees of freedom that is less than *x*.

MATRIX inverse distribution functions

The following functions give the value in a specified distribution having a cumulative probability equal to *prob*, the first argument. Subsequent arguments are the parameters of the distribution. Note the period in each function name.

IDF.BETA. IDF.BETA(prob, shape1, shape2). Numeric. Returns the value from the Beta distribution, with the given shape parameters, for which the cumulative probability is prob.

IDF.CAUCHY. IDF.CAUCHY(prob, loc, scale). Numeric. Returns the value from the Cauchy distribution, with the given location and scale parameters, for which the cumulative probability is prob.

IDF.CHISQ. IDF.CHISQ(prob, df). Numeric. Returns the value from the chi-square distribution, with the specified degrees of freedom df, for which the cumulative probability is prob. For example, the chi-square value that is significant at the 0.05 level with 3 degrees of freedom is IDF.CHISQ(0.95,3).

IDF.EXP. IDF.EXP(p, scale). Numeric. Returns the value of an exponentially decaying variable, with rate of decay scale, for which the cumulative probability is p.

IDF.F. IDF.F(prob, df1, df2). Numeric. Returns the value from the F distribution, with the specified degrees of freedom, for which the cumulative probability is prob. For example, the F value that is significant at the 0.05 level with 3 and 100 degrees of freedom is IDF.F(0.95,3,100).

IDF.GAMMA. IDF.GAMMA(prob, shape, scale). Numeric. Returns the value from the Gamma distribution, with the specified shape and scale parameters, for which the cumulative probability is prob.

IDF.HALFNRM. IDF.HALFNRM(prob, mean, stddev). Numeric. Returns the value from the half normal distribution, with the specified mean and standard deviation, for which the cumulative probability is prob.

IDF.IGAUSS. IDF.IGAUSS(prob, loc, scale). Numeric. Returns the value from the inverse Gaussian distribution, with the given location and scale parameters, for which the cumulative probability is prob.

IDF.LAPLACE. IDF.LAPLACE(prob, mean, scale). Numeric. Returns the value from the Laplace distribution, with the specified mean and scale parameters, for which the cumulative probability is prob.

IDF.LNORMAL. IDF.LNORMAL(prob, a, b). Numeric. Returns the value from the log-normal distribution, with specified parameters, for which the cumulative probability is prob.

IDF.LOGISTIC. IDF.LOGISTIC(prob, mean, scale). Numeric. Returns the value from the logistic distribution, with specified mean and scale parameters, for which the cumulative probability is prob.

IDF.NORMAL. IDF.NORMAL(prob, mean, stddev). Numeric. Returns the value from the normal distribution, with specified mean and standard deviation, for which the cumulative probability is prob.

IDF.PARETO. IDF.PARETO(prob, threshold, shape). Numeric. Returns the value from the Pareto distribution, with specified threshold and scale parameters, for which the cumulative probability is prob.

IDF.SMOD. IDF.SMOD(prob, a, b). Numeric. Returns the value from the Studentized maximum modulus, with the specified parameters, for which the cumulative probability is prob.

IDF.SRANGE. IDF.SRANGE(prob, a, b). Numeric. Returns the value from the Studentized range statistic, with the specified parameters, for which the cumulative probability is prob.

IDF.T. IDF.T(prob, df). Numeric. Returns the value from Student's t distribution, with specified degrees of freedom df, for which the cumulative probability is prob.

IDF.UNIFORM. IDF.UNIFORM(prob, min, max). Numeric. Returns the value from the uniform distribution between min and max for which the cumulative probability is prob.

IDF.WEIBULL. IDF.WEIBULL(prob, a, b). Numeric. Returns the value from the Weibull distribution, with specified parameters, for which the cumulative probability is prob.

PROBIT. PROBIT(prob). Numeric. Returns the value in a standard normal distribution having a cumulative probability equal to prob. The argument prob is a probability greater than 0 and less than 1.

Note: PROBIT(prob) is the inverse of CDFNORM(M) and is a special case of IDF.NORMAL(prob, mean, stddev), where mean = 0, stddev = 1.

MATRIX random variable functions

The following functions give a random variate from a specified distribution. The arguments are the parameters of the distribution. The first and second variables You can repeat the sequence of pseudorandom numbers by setting a seed in the Preferences dialog box before each sequence. Note the period in each function name.

NORMAL. NORMAL(stddev). Numeric. Returns a normally distributed pseudorandom number from a distribution with mean 0 and standard deviation stddev, which must be a positive number. You can repeat the sequence of pseudorandom numbers by setting a seed in the Random Number Seed dialog box before each sequence.

RV.BERNOULLI. RV.BERNOULLI(prob). Numeric. Returns a random value from a Bernoulli distribution with the specified probability parameter prob.

RV.BETA. RV.BETA(shape1, shape2). Numeric. Returns a random value from a Beta distribution with specified shape parameters.

RV.BINOM. RV.BINOM(n, prob). Numeric. Returns a random value from a binomial distribution with specified number of trials and probability parameter.

RV.CAUCHY. RV.CAUCHY(loc, scale). Numeric. Returns a random value from a Cauchy distribution with specified location and scale parameters.

RV.CHISQ. RV.CHISQ(df). Numeric. Returns a random value from a chi-square distribution with specified degrees of freedom df.

RV.EXP. RV.EXP(scale). Numeric. Returns a random value from an exponential distribution with specified scale parameter.

RV.F. RV.F(df1, df2). Numeric. Returns a random value from an F distribution with specified degrees of freedom, df1 and df2.

RV.GAMMA. RV.GAMMA(shape, scale). Numeric. Returns a random value from a Gamma distribution with specified shape and scale parameters.

RV.GEOM. RV.GEOM(prob). Numeric. Returns a random value from a geometric distribution with specified probability parameter.

RV.HALFNRM. RV.HALFNRM(mean, stddev). Numeric. Returns a random value from a half normal distribution with the specified mean and standard deviation.

RV.HYPER. RV.HYPER(total, sample, hits). Numeric. Returns a random value from a hypergeometric distribution with specified parameters.

RV.IGAUSS. RV.IGAUSS(loc, scale). Numeric. Returns a random value from an inverse Gaussian distribution with the specified location and scale parameters.

RV.LAPLACE. RV.LAPLACE(mean, scale). Numeric. Returns a random value from a Laplace distribution with specified mean and scale parameters.

RV.LOGISTIC. RV.LOGISTIC(mean, scale). Numeric. Returns a random value from a logistic distribution with specified mean and scale parameters.

RV.LNORMAL. RV.LNORMAL(a, b). Numeric. Returns a random value from a log-normal distribution with specified parameters.

RV.NEGBIN. RV.NEGBIN(threshold, prob). Numeric. Returns a random value from a negative binomial distribution with specified threshold and probability parameters.

RV.NORMAL. RV.NORMAL(mean, stddev). Numeric. Returns a random value from a normal distribution with specified mean and standard deviation.

RV.PARETO. RV.PARETO(threshold, shape). Numeric. Returns a random value from a Pareto distribution with specified threshold and shape parameters.

RV.POISSON. RV.POISSON(mean). Numeric. Returns a random value from a Poisson distribution with specified mean/rate parameter.

RV.T. RV.T(df). Numeric. Returns a random value from a Student's t distribution with specified degrees of freedom df.

RV.UNIFORM. RV.UNIFORM(min, max). Numeric. Returns a random value from a uniform distribution with specified minimum and maximum. See also the UNIFORM function.

WEIBULL. RV.WEIBULL(a, b). Numeric. Returns a random value from a Weibull distribution with specified parameters.

UNIFORM. UNIFORM(max). Numeric. Returns a uniformly distributed pseudorandom number between 0 and the argument max, which must be numeric (but can be negative).

CALL Statement

Closely related to the matrix functions are the matrix procedures, which are invoked with the `CALL` statement. Procedures, similarly to functions, accept arguments enclosed in parentheses and separated by commas. They return their result in one or more of the arguments as noted in the individual descriptions below. They are implemented as procedures rather than as functions so that they can return more than one value or (in the case of `SETDIAG`) modify a matrix without making a copy of it.

EIGEN(M,var1,var2). *Eigenvectors and eigenvalues of a symmetric matrix.* Takes three arguments: a symmetric matrix and two valid variable names to which the results are assigned. If *M* is a symmetric matrix, the statement `CALL EIGEN(M, A, B)` will assign to *A* a matrix having the same dimensions as *M*, containing the eigenvectors of *M* as its columns, and will assign to *B* a column vector having as many rows as *M*, containing the eigenvalues of *M* in descending numerical order. The eigenvectors in *A* are ordered to correspond with the eigenvalues in *B*; thus, the first column corresponds to the largest eigenvalue, the second to the second largest, and so on.

SETDIAG(M,V). *Set the main diagonal of a matrix.* Takes two arguments, a matrix and a vector. Elements on the main diagonal of *M* are set equal to the corresponding elements of *V*. If *V* is a scalar, all the diagonal elements are set equal to that scalar. Otherwise, if *V* has fewer elements than the main diagonal of *M*, remaining elements on the main diagonal are unchanged. If *V* has more elements than are needed, the extra elements are not used. See also the `MDIAG` matrix function.

SVD(M,var1,var2,var3). *Singular value decomposition of a matrix.* Takes four arguments: a matrix and three valid variable names to which the results are assigned. If *M* is a matrix, the statement `CALL SVD(M,U,Q,V)` will assign to *Q* a diagonal matrix of the same dimensions as *M*, and to *U* and *V* unitary matrices (matrices whose inverses equal their transposes) of appropriate dimensions, such that *M=U*Q** T(*V*), where T is the transpose function defined above. The singular values of *M* are in the main diagonal of *Q*.

PRINT Statement

The PRINT statement displays matrices or matrix expressions. Its syntax is as follows:

```
PRINT [matrix expression]
      [/FORMAT="format descriptor"]
      [/TITLE="title"]
      [/SPACE={NEWPAGE}]
              {n       }
      [{/RLABELS=list of quoted names}]
       {/RNAMES=vector of names      }

      [{/CLABELS=list of quoted names}]
       {/CNAMES=vector of names      }
```

Notes

- The MDISPLAY global setting specifies how matrices are displayed. By default, IBM SPSS Statistics displays matrices to TEXT. The TABLES setting uses pivot tables to display matrices. When the **MDISPLAY** global setting is used, the PRINT statement uses either TEXT or TABLES mode exclusively (whichever is specified for **MDISPLAY**).
- When the PRINT statement is used in TABLES mode, it supports long row and column names when they are explicitly specified in either the /RLABELS or /CLABELS subcommands. When a PRINT statement is used in TEXT mode, row and column names are truncated to 8 bytes on any PRINT subcommand in which they are specified. When row and column names are not specified, default names `row1`, `row2`, ... and `col1`, `col2`, ... are used.
- When the PRINT statement is used in TABLES mode, the scaling factor is shown in the output annotation.

Matrix Expression

Matrix expression is a single matrix variable name or an expression that evaluates to a matrix. PRINT displays the specified matrix.

- The matrix specification must precede any other specifications on the PRINT statement. If no matrix is specified, no data will be displayed, but the TITLE and SPACE specifications will be honored.
- You can specify a matrix name, a matrix raised to a power, or a matrix function (with its arguments in parentheses) by itself, but you must enclose other matrix expressions in parentheses. For example, `PRINT A`, `PRINT INV(A)`, and `PRINT B**DET(T(C)*D)` are all legal, but `PRINT A+B` is not. You must specify `PRINT (A+B)`.
- Constant expressions are allowed.
- A matrix program can consist entirely of PRINT statements, without defining any matrix variables.

FORMAT Keyword

FORMAT specifies a single format descriptor for display of the matrix data.

- All matrix elements are displayed with the same format.
- You can use any printable numeric format (for numeric matrices) or string format (for string matrices) as defined in FORMATS.
- The matrix processor will choose a suitable numeric format if you omit the FORMAT specification, but a string format such as A8 is essential when displaying a matrix containing string data.
- String values exceeding the width of a string format are truncated.
- See “Scaling Factor in Displays ” on page 1090 for default formatting of matrices containing large or small values.

TITLE Keyword

TITLE specifies a title for the matrix displayed. The title must be enclosed in quotes. If it exceeds the maximum display width, it is truncated. The slash preceding TITLE is required, even if it is the only

specification on the PRINT statement. If you omit the TITLE specification, the matrix name or expression from the PRINT statement is used as a default title.

SPACE Keyword

SPACE controls output spacing before printing the title and the matrix. You can specify either a positive number or the keyword NEWPAGE. The slash preceding SPACE is required, even if it is the only specification on the PRINT statement.

NEWPAGE. *Start a new page before printing the title.*

n. *Skip* n *lines before displaying the title.*

Note: The /SPACE subcommand is ignored when MDISPLAY is in TABLES mode. However, /SPACE is not ignored when only TITLE and no matrix is specified. In this case, the title is printed as in TEXT mode.

RLABELS Keyword

RLABELS allows you to supply row labels for the matrix.

- The labels must be separated by commas.
- Enclose individual labels in quotes if they contain embedded commas or if you want to preserve lowercase letters. Otherwise, quotes are optional.
- If too many names are supplied, the extras are ignored. If not enough names are supplied, the last rows remain unlabeled.
- When the PRINT statement is used in TABLES mode, the /RLABELS subcommand supports long row names (up to 64 bytes). When the PRINT statement is used in TEXT mode, long row names are truncated to 8 bytes.

RNAMES Keyword

RNAMES allows you to supply the name of a vector or a vector expression containing row labels for the matrix.

- Either a row vector or a column vector can be used, but the vector must contain string data.
- If too many names are supplied, the extras are ignored. If not enough names are supplied, the last rows remain unlabeled.
- Row names that are referenced through a vector in /RNAMES are truncated to 8 bytes. When row names are not specified, default names row1, row2,... are used.

CLABELS Keyword

CLABELS allows you to supply column labels for the matrix.

- The labels must be separated by commas.
- Enclose individual labels in quotes if they contain embedded commas or if you want to preserve lowercase letters. Otherwise, quotes are optional.
- If too many names are supplied, the extras are ignored. If not enough names are supplied, the last columns remain unlabeled.
- When the PRINT statement is used in TABLES mode, the /CLABELS subcommand supports long column names (up to 64 bytes). When the PRINT statement is used in TEXT mode, long column names are truncated to 8 bytes.

CNAMES Keyword

CNAMES allows you to supply the name of a vector or a vector expression containing column labels for the matrix.

- Either a row vector or a column vector can be used, but the vector must contain string data.

- If too many names are supplied, the extras are ignored. If not enough names are supplied, the last columns remain unlabeled.
- Column names that are referenced through a vector in /CNAMES are truncated to 8 bytes. When column names are not specified, default names col1, col2,... are used.

Scaling Factor in Displays

When a matrix contains very large or very small numbers, it may be necessary to use scientific notation to display the data. If you do not specify a display format, the matrix processor chooses a power-of-10 multiplier that will allow the largest value to be displayed, and it displays this multiplier on a heading line before the data. The multiplier is not displayed for each element in the matrix. The displayed values, multiplied by the power of 10 that is indicated in the heading, equal the actual values (possibly rounded).

- Values that are very small, relative to the multiplier, are displayed as 0.
- If you explicitly specify a scientific-notation format (Ew.d), each matrix element is displayed using that format. This permits you to display very large and very small numbers in the same matrix without losing precision.

Example

```
COMPUTE M = {.0000000001357, 2.468, 3690000000}.
PRINT M /TITLE "Default format".
PRINT M /FORMAT "E13" /TITLE "Explicit exponential format".
```

- The first PRINT subcommand uses the default format with 10^9 as the multiplier for each element of the matrix. This results in the following output:

```
Default format
  10 ** 9   X
    .000000000    .000000002   3.690000000
```

Note that the first element is displayed as 0 and the second is rounded to one significant digit.

- An explicitly specified exponential format on the second PRINT subcommand allows each element to be displayed with full precision, as the following output shows:

```
Explicit exponential format
       1.E-010       2.E+000       4.E+009
```

Matrix Control Structures

The matrix language includes two structures that allow you to alter the flow of control within a matrix program.

- The DO IF statement tests a logical expression to determine whether one or more subsequent matrix statements should be executed.
- The LOOP statement defines the beginning of a block of matrix statements that should be executed repeatedly until a termination criterion is satisfied or a BREAK statement is executed.

These statements closely resemble the DO IF and LOOP commands in the IBM SPSS Statistics transformation language. In particular, these structures can be nested within one another as deeply as the available memory allows.

DO IF Structures

A DO IF structure in a matrix program affects the flow of control exactly as the analogous commands affect a transformation program, except that missing-value considerations do not arise in a matrix program. The syntax of the DO IF structure is as follows:

```
DO IF [(]logical expression[)]
  matrix statements
```

```
[ELSE IF [(]logical expression[)]]
  matrix statements
[ELSE IF...]
  .
  .
  .
[ELSE]
 matrix statements
END IF.
```

- The DO IF statement marks the beginning of the structure, and the END IF statement marks its end.
- The ELSE IF statement is optional and can be repeated as many times as desired within the structure.
- The ELSE statement is optional. It can be used only once and must follow any ELSE IF statements.
- The END IF statement must follow any ELSE IF and ELSE statements.
- The DO IF and ELSE IF statements must contain a logical expression, normally one involving the relational operators EQ, GT, and so on. However, the matrix language allows any expression that evaluates to a scalar to be used as the logical expression. Scalars greater than 0 are considered true, and scalars less than or equal to 0 are considered false.

A DO IF structure affects the flow of control within a matrix program as follows:

- If the logical expression on the DO IF statement is true, the statements immediately following the DO IF are executed up to the next ELSE IF or ELSE in the structure. Control then passes to the first statement following the END IF for that structure.
- If the expression on the DO IF statement is false, control passes to the first ELSE IF, where the logical expression is evaluated. If this expression is true, statements following the ELSE IF are executed up to the next ELSE IF or ELSE statement, and control passes to the first statement following the END IF for that structure.
- If the expressions on the DO IF and the first ELSE IF statements are both false, control passes to the next ELSE IF, where that logical expression is evaluated. If none of the expressions is true on any of the ELSE IF statements, statements following the ELSE statement are executed up to the END IF statement, and control falls out of the structure.
- If none of the expressions on the DO IF statement or the ELSE IF statements is true and there is no ELSE statement, control passes to the first statement following the END IF for that structure.

LOOP Structures

A LOOP structure in a matrix program affects the flow of control exactly as the analogous commands affect transformation program, except that missing-value considerations do not arise in a matrix program. Its syntax is as follows:

```
LOOP [varname=n TO m [BY k]] [IF [(]logical expression[)]
matrix statements
[BREAK]
matrix statements
END LOOP [IF [(]logical expression[)]]
```

The matrix statements specified between LOOP and END LOOP are executed repeatedly until one of the following conditions is met:

- A logical expression on the IF clause of the LOOP statement is evaluated as false.
- An index variable used on the LOOP statement passes beyond its terminal value.
- A logical expression on the IF clause of the END LOOP statement is evaluated as true.
- A BREAK statement is executed within the loop structure (but outside of any nested loop structures).

Note: Unlike the LOOP command (outside the matrix language), the index value of a matrix LOOP structure does not override the maximum number of loops controlled by SET MXLOOPS. You must explicitly set the

MXLOOPS value to a value high enough to accommodate the index value. See the topic "MXLOOPS Subcommand " on page 1799 for more information.

Index Clause on the LOOP Statement

An index clause on a LOOP statement creates an index variable whose name is specified immediately after the keyword LOOP. The variable is assigned an initial value of *n*. Each time through the loop, the variable is tested against the terminal value *m* and incremented by the increment value *k* if *k* is specified or by 1 if *k* is not specified. When the index variable is greater than *m* for positive increments or less than *m* for negative increments, control passes to the statement after the END LOOP statement.

- Both the index clause and the IF clause are optional. If both are present, the index clause must appear first.
- The index variable must be scalar with a valid matrix variable name.
- The initial value, *n*, the terminal value, *m*, and the increment, *k* (if present), must be scalars or matrix expressions evaluating to scalars. Non-integer values are truncated to integers before use.
- If the keyword BY and the increment *k* are absent, an increment of 1 is used.

IF Clause on the LOOP Statement

The logical expression is evaluated before each iteration of the loop structure. If it is false, the loop terminates and control passes to the statement after END LOOP.

- The IF clause is optional. If both the index clause and the IF clause are present, the index clause must appear first.
- As in the DO IF structure, the logical expression of the IF clause is evaluated as scalar, with positive values being treated as true and 0 or negative values, as false.

IF Clause on the END LOOP Statement

When an IF clause is present on an END LOOP statement, the logical expression is evaluated after each iteration of the loop structure. If it is true, the loop terminates and control passes to the statement following the END LOOP statement.

- The IF clause is optional.
- As in the LOOP statement, the logical expression of the IF clause is evaluated as scalar, with positive values being treated as true and 0 or negative values, as false.

BREAK Statement

The BREAK statement within a loop structure transfers control immediately to the statement following the (next) END LOOP statement. It is normally placed within a DO IF structure inside the LOOP structure to exit the loop when the specified conditions are met.

Example

```
LOOP LOCATION = 1, NROW(VEC).
+  DO IF (VEC(LOCATION) = TARGET).
+     BREAK.


+  END IF.
END LOOP.
```

- This loop searches for the (first) location of a specific value, *TARGET*, in a vector, *VEC*.
- The DO IF statement checks whether the vector element indexed by *LOCATION* equals the target.
- If so, the BREAK statement transfers control out of the loop, leaving *LOCATION* as the index of *TARGET* in *VEC*.

READ Statement: Reading Character Data

The READ statement reads data into a matrix or submatrix from a character-format file--that is, a file containing ordinary numbers or words in readable form. The syntax for the READ statement is:

```
READ  variable reference
  [/FILE = file reference]
   /FIELD = c1 TO c2 [BY w]
  [/SIZE = size expression]
  [/MODE = {RECTANGULAR}]
           {SYMMETRIC  }
  [/REREAD]
  [/FORMAT = format descriptor]
```

- The file can contain values in freefield or fixed-column format. The data can appear in any of the field formats supported by DATA LIST.
- More than one matrix can be read from a single input record by rereading the record.
- If the end of the file is encountered during a READ operation (that is, fewer values are available than the number of elements required by the specified matrix size), a warning message is displayed and the contents of the unread elements of the matrix are unpredictable.

Variable Specification

The variable reference on the READ statement is a matrix variable name, with or without indexes.

For a name without indexes:

- READ creates the specified matrix variable.
- The matrix need not exist when READ is executed.
- If the matrix already exists, it is replaced by the matrix read from the file.
- You must specify the size of the matrix using the SIZE specification.

For an indexed name:

- READ creates a submatrix from an existing matrix.
- The matrix variable named must already exist.
- You can define any submatrix with indexes; for example, M(:,I). To define an entire existing matrix, specify M(:,:).
- The SIZE specification can be omitted. If specified, its value must match the size of the specified submatrix.

FILE Specification

FILE designates the character file containing the data. It can be an actual filename in quotes, or a file handle defined on a FILE HANDLE command that precedes the matrix program.

- The filename or handle must specify an existing file containing character data, not a data file in IBM SPSS Statistics format or a specially formatted file of another kind, such as a spreadsheet file.
- The FILE specification is required on the first READ statement in a matrix program (first in order of appearance, not necessarily in order of execution). If you omit the FILE specification from a later READ statement, the statement uses the most recently named file (in order of appearance) on a READ statement in the same matrix program.

FIELD Specification

FIELD specifies the column positions of a fixed-format record where the data for matrix elements are located.

- The FIELD specification is required.
- Startcol is the number of the leftmost column of the input area.
- Endcol is the number of the rightmost column of the input area.

- Both `startcol` and `endcol` are required and both must be constants. For example, `FIELD = 9 TO 72` specifies that values to be read appear between columns 9 and 72 (inclusive) of each input record.
- The BY clause, if present, indicates that each value appears within a fixed set of columns on the input record; that is, one value is separated from the next by its column position rather than by a space or comma. Width is the width of the area designated for each value. For example, `FIELD = 1 TO 80 BY 10` indicates that there are eight possible values per record and that one will appear between columns 1 and 10 (inclusive), another between columns 11 and 20, and so on, up to columns 71 and 80. The BY value must evenly divide the length of the field. That is, `endcol-startcol+1` must be a multiple of the width.
- You can use the FORMAT specification to supply the same information as the BY clause of the FIELD specification. If you omit the BY clause and do not specify a format on the FORMAT specification, READ assumes that values are separated by blanks or commas within the designated field.

SIZE Specification

The SIZE specification is a matrix expression that, when evaluated, specifies the size of the matrix to be read.

- The expression should evaluate to a two-element row or column vector. The first element designates the number of rows in the matrix to be read; the second element gives the number of columns.
- Values of the SIZE specification are truncated to integers if necessary.
- The size expression may be a constant, such as `{5;5}`, or a matrix variable name, such as MSIZE, or any valid expression, such as `INFO(1,:)`.
- If you use a scalar as the size expression, a column vector containing that number of rows is read. Thus, `SIZE=1` reads a scalar, and `SIZE=3` reads a 3 × 1 column vector.

You must include a SIZE specification whenever you name an entire matrix (rather than a submatrix) on the READ statement. If you specify a submatrix, the SIZE specification is optional but, if included, must agree with the size of the specified submatrix.

MODE Specification

MODE specifies the format of the matrix to be read in. It can be either rectangular or symmetric. If the MODE specification is omitted, the default is RECTANGULAR.

RECTANGULAR. *Matrix is completely represented in file.* Each row begins on a new record, and all entries in that row are present on that and (possibly) succeeding records. This is the default if the MODE specification is omitted.

SYMMETRIC. *Elements of the matrix below the main diagonal are the same as those above it.* Only matrix elements on and below the main diagonal are read; elements above the diagonal are set equal to the corresponding symmetric elements below the diagonal. Each row is read beginning on a new record, although it may span more than one record. Only a single value is read from the first record, two values are read from the second, and so on.

- If SYMMETRIC is specified, the matrix processor first checks that the number of rows and the number of columns are the same. If the numbers, specified either on SIZE or on the variable reference, are not the same, an error message is displayed and the command is not executed.

REREAD Specification

The REREAD specification indicates that the current READ statement should begin with the last record read by a previous READ statement.

- REREAD has no further specifications.
- REREAD cannot be used on the first READ statement to read from a file.
- If you omit REREAD, the READ statement begins with the first record following the last one read by the previous READ statement.

- The REREAD specification is ignored on the first READ statement following a call to the EOF function for the same file.

FORMAT Specification

FORMAT specifies how the matrix processor should interpret the input data. The format descriptor can be any valid IBM SPSS Statistics data format, such as F6, E12.2, or A6, or it can be a type code; for example, F, E, or A.

- If you omit the FORMAT specification, the default is F.
- You can specify the width of fixed-size data fields with either a FORMAT specification or a BY clause on a FIELD specification. You can include it in both places only if you specify the same value.
- If you do not include either a FORMAT or a BY clause on FIELD, READ expects values separated by blanks or commas.
- An additional way of specifying the width is to supply a repetition factor without a width (for example, 10F, 5COMMA, or 3E). The field width is then calculated by dividing the width of the whole input area on the FIELD specification by the repetition factor. A format with a digit for the repetition factor must be enclosed in quotes.
- Only one format can be specified. A specification such as FORMAT='5F2.0 3F3.0 F2.0' is invalid.

WRITE Statement: Writing Character Data

WRITE writes the value of a matrix expression to an external file. The syntax of the WRITE statement is:

```
WRITE  matrix expression
   [/OUTFILE = file reference]
    /FIELD = startcol TO endcol [BY width]
   [/MODE = {RECTANGULAR}]
            {TRIANGULAR }
   [/HOLD]
   [/FORMAT = format descriptor]
```

Matrix Expression Specification

Specify any matrix expression that evaluates to the value(s) to be written.

- The matrix specification must precede any other specifications on the WRITE statement.
- You can specify a matrix name, a matrix raised to a power, or a matrix function (with its arguments in parentheses) by itself, but you must enclose other matrix expressions in parentheses. For example, WRITE A, WRITE INV(A), or WRITE B**DET(T(C)*D) is legal, but WRITE A+B is not. You must specify WRITE (A+B).
- Constant expressions are allowed.

OUTFILE Specification

OUTFILE designates the character file to which the matrix expression is to be written. The file reference can be an actual filename in quotes or a file handle defined on a FILE HANDLE command that precedes the matrix program. The filename or file handle must be a valid file specification.

- The OUTFILE specification is required on the first WRITE statement in a matrix program (first in order of appearance, not necessarily in order of execution).
- If you omit the OUTFILE specification from a later WRITE statement, the statement uses the most recently named file (in order of appearance) on a WRITE statement in the same matrix program.

FIELD Specification

FIELD specifies the column positions of a fixed-format record to which the data should be written.

- The FIELD specification is required.
- The start column, *c1*, is the number of the leftmost column of the output area.

- The end column, *c2*, is the number of the rightmost column of the output area.
- Both *c1* and *c2* are required, and both must be constants. For example, `FIELD = 9 TO 72` specifies that values should be written between columns 9 and 72 (inclusive) of each output record.
- The BY clause, if present, indicates how many characters should be allocated to the output value of a single matrix element. The value *w* is the width of the area designated for each value. For example, `FIELD = 1 TO 80 BY 10` indicates that up to eight values should be written per record, and that one should go between columns 1 and 10 (inclusive), another between columns 11 and 20, and so on up to columns 71 and 80. The value on the BY clause must evenly divide the length of the field. That is, *c2* – *c1* + 1 must be a multiple of *w*.
- You can use the FORMAT specification (see below) to supply the same information as the BY clause. If you omit the BY clause from the FIELD specification and do not specify a format on the FORMAT specification, WRITE uses freefield format, separating matrix elements by single blank spaces.

MODE Specification

MODE specifies the format of the matrix to be written. If MODE is not specified, the default is RECTANGULAR.

RECTANGULAR. *Write the entire matrix*. Each row starts a new record, and all of the values in that row are present in that and (possibly) subsequent records. This is the default if the MODE specification is omitted.

TRIANGULAR. *Write only the lower triangular entries and the main diagonal*. Each row begins a new record and may span more than one record. This mode may save file space.

- A matrix written with `MODE = TRIANGULAR` must be square, but it need not be symmetric. If it is not, values in the upper triangle are not written.
- A matrix written with `MODE = TRIANGULAR` may be read with `MODE = SYMMETRIC`.

HOLD Specification

HOLD causes the last line written by the current WRITE statement to be held so that the next WRITE to that file will write on the same line. Use HOLD to write more than one matrix on a line.

FORMAT Specification

FORMAT indicates how the internal (binary) values of matrix elements should be converted to character format for output.

- The format descriptor is any valid IBM SPSS Statistics data format, such as F6, E12.2, or A6, or it can be a format type code, such as F, E, or A. It specifies how the written data are encoded and, if a width is specified, how wide the fields containing the data are. (See FORMATS for valid formats.)
- If you omit the FORMAT specification, the default is F.
- The data field widths may be specified either here or after BY on the FIELD specification. You may specify the width in both places only if you give the same value.
- An additional way of specifying the width is to supply a repetition factor without a width (for example, 10F or 5COMMA). The field width is then calculated by dividing the width of the whole output area on the FIELD specification by the repetition factor. A format with a digit for the repetition factor must be enclosed in quotes.
- If the field width is not specified in any of these ways, then the freefield format is used--matrix values are written separated by one blank, and each value occupies as many positions as necessary to avoid the loss of precision. Each row of the matrix is written starting with a new output record.
- Only one format descriptor can be specified. Do *not* try to specify more than one format; for example, `'5F2.0 3F3.0 F2.0'` is invalid as a FORMAT specification on WRITE.

GET Statement: Reading IBM SPSS Statistics Data Files

GET reads matrices from an external IBM SPSS Statistics data file or from the active dataset. The syntax of GET is as follows:

```
GET variable reference
  [/FILE={file reference}]
         {*             }
  [/VARIABLES = variable list]
  [/NAMES = names vector]
  [/MISSING = {ACCEPT}]
              {OMIT  }
              {value }
  [/SYSMIS = {OMIT }]
             {value}
```

Variable Specification

The variable reference on the GET statement is a matrix variable name with or without indexes.

For a name without indexes:

- GET creates the specified matrix variable.
- The size of the matrix is determined by the amount of data read from the IBM SPSS Statistics data file or the active dataset.
- If the matrix already exists, it is replaced by the matrix read from the file.

For an indexed name:

- GET creates a submatrix from an existing matrix.
- The matrix variable named must already exist.
- You can define any submatrix with indexes; for example, `M(:,I)`. To define an entire existing matrix, specify `M(:,:)`.
- The indexes, along with the size of the existing matrix, specify completely the size of the submatrix, which must agree with the dimensions of the data read from the IBM SPSS Statistics data file.
- The specified submatrix is replaced by the matrix elements read from the IBM SPSS Statistics data file.

FILE Specification

FILE designates the IBM SPSS Statistics data file to be read. Use an asterisk, or simply omit the FILE specification, to designate the current active dataset.

- The file reference can be either a filename enclosed in quotes or a file handle defined on a FILE HANDLE command that precedes the matrix program.
- Both dataset references and physical file specifications are supported.
- If you omit the FILE specification, the active dataset is used.
- In a matrix program executed with the INCLUDE command, if a SPLIT FILE command is in effect, a GET statement that references the active dataset will read a single split-file group of cases. (A matrix program cannot be executed from a syntax window if a SPLIT FILE command is in effect.)

VARIABLES Specification

VARIABLES specifies a list of variables to be read from the IBM SPSS Statistics data file.

- The keyword TO can be used to reference consecutive variables on the data file.
- The variable list can consist of the keyword ALL to get all the variables in the data file. ALL is the default if the VARIABLES specification is omitted.
- Long variable names (up to 64 bytes) are supported when specified on the /VARIABLES subcommand (either explicitly or when using the TO or ALL keywords).
- All variables read from the data file should be numeric. If a string variable is specified, a warning message is issued and the string variable is skipped.

Example

```
GET M /VARIABLES = AGE, RESIDE, INCOME TO HEALTH.
```

- The variables *AGE*, *RESIDE*, and *INCOME TO HEALTH* from the active dataset will form the columns of the matrix *M*.

NAMES Specification

NAMES specifies a vector to store the variable names from the data file.

- If you omit the NAMES specification, the variable names are not available to the MATRIX procedure.
- Variable names are truncated to 8 bytes when they are referenced through a vector in the /NAMES subcommand.

MISSING Specification

MISSING specifies how missing values declared for the IBM SPSS Statistics data file should be handled.

- The MISSING specification is required if the data file contains missing values for any variable being read.
- If you omit the MISSING specification and a missing value is encountered for a variable being read, an error message is displayed and the GET statement is not executed.

The following keywords are available on the MISSING specification. There is no default.

ACCEPT. *Accept user-missing values for entry.* If the system-missing value exists for a variable to be read, you must specify SYSMIS to indicate how the system-missing value should be handled.

OMIT. *Skip an entire observation when a variable with a missing value is encountered.*

value. *Recode all missing values encountered (including the system-missing value) to the specified value for entry.* The replacement value can be any numeric constant.

SYSMIS Specification

SYSMIS specifies how system-missing values should be handled when you have specified ACCEPT on MISSING.

- The SYSMIS specification is ignored unless ACCEPT is specified on MISSING.
- If you specify ACCEPT on MISSING but omit the SYSMIS specification, and a system-missing value is encountered for a variable being read, an error message is displayed and the GET statement is not executed.

The following keywords are available on the SYSMIS specification. There is no default.

OMIT. *Skip an entire observation when a variable with a system-missing value is encountered.*

value. *Recode all system-missing values encountered to the specified value for entry.* The replacement value can be any numeric constant.

Example

```
GET SCORES
  /VARIABLES = TEST1,TEST2,TEST3
  /NAMES = VARNAMES
  /MISSING = ACCEPT
  /SYSMIS = -1.0.
```

- A matrix named *SCORES* is read from the active dataset.
- The variables *TEST1*, *TEST2*, and *TEST3* form the columns of the matrix, while the cases in the active dataset form the rows.
- A vector named *VARNAMES*, whose three elements contain the variable names *TEST1*, *TEST2*, and *TEST3*, is created.
- User-missing values defined in the active dataset are accepted into the matrix *SCORES*.

- System-missing values in the active dataset are converted to the value –1 in the matrix *SCORES*.

SAVE Statement: Writing IBM SPSS Statistics Data Files

SAVE writes matrices to an external IBM SPSS Statistics data file or to the current active dataset. The rows of the matrix expression become cases, and the columns become variables. The syntax of the SAVE statement is as follows:

```
SAVE matrix expression
  [/OUTFILE = {file reference}]
              {*             }
  [/VARIABLES = variable list]
  [/NAMES = names vector]
  [/STRINGS = variable list]
```

Matrix Expression Specification

The matrix expression following the keyword SAVE is any matrix language expression that evaluates to the value(s) to be written to a data file.

- The matrix specification must precede any other specifications on the SAVE statement.
- You can specify a matrix name, a matrix raised to a power, or a matrix function (with its arguments in parentheses) by itself, but you must enclose other matrix expressions in parentheses. For example, `SAVE A`, `SAVE INV(A)`, or `SAVE B**DET(T(C)*D)` is legal, but `SAVE A+B` is not. You must specify `SAVE (A+B)`.
- Constant expressions are allowed.

OUTFILE Specification

`OUTFILE` designates the file to which the matrix expression is to be written. It can be an actual filename in quotes or a file handle defined on a `FILE HANDLE` command that precedes the matrix program.

- To save a matrix expression as the active dataset, specify an asterisk (`*`). If there is no active dataset, one will be created; if there is one, it is replaced by the saved matrices.
- Both dataset references and physical file specifications are supported.
- The `OUTFILE` specification is required on the first SAVE statement in a matrix program (first in order of appearance, not necessarily in order of execution). If you omit the `OUTFILE` specification from a later SAVE statement, the statement uses the most recently named file (in order of appearance) on a SAVE statement in the same matrix program.
- If more than one SAVE statement writes to the active dataset in a single matrix program, the dictionary of the new active dataset is written on the basis of the information given by the first such SAVE. All of the subsequently saved matrices are appended to the new active dataset as additional cases. If the number of columns differs, an error occurs.
- When you execute a matrix program with the `INCLUDE` command, the SAVE statement creates a new IBM SPSS Statistics data file at the end of the matrix program's execution, so any attempt to `GET` the data file obtains the original data file, if any.
- When you execute a matrix program from a syntax window, SAVE creates a new IBM SPSS Statistics data file immediately, but the file remains open, so you cannot `GET` it until after the `END MATRIX` statement.

VARIABLES Specification

You can provide variable names for the IBM SPSS Statistics data file with the `VARIABLES` specification. The variable list is a list of valid variable names separated by commas.

- You can use the `TO` convention, as shown in the example below.
- You can also use the `NAMES` specification, discussed below, to provide variable names.

- Long variable names (up to 64 bytes) are supported when explicitly specified on the /VARIABLES subcommand.

Example

```
SAVE {A,B,X,Y} /OUTFILE=*
  /VARIABLES = A,B,X1 TO X50,Y1,Y2.
```

- The matrix expression on the SAVE statement constructs a matrix from two column vectors *A* and *B* and two matrices *X* and *Y*. All four matrix variables must have the same number of rows so that this matrix construction will be valid.
- The VARIABLES specification provides descriptive names so that the variable names in the new active dataset will resemble the names used in the matrix program.

NAMES Specification

As an alternative to the explicit list on the VARIABLES specification, you can specify a name list with a vector containing string values. The elements of this vector are used as names for the variables.

- The NAMES specification on SAVE is designed to complement the NAMES specification on the GET statement. Names extracted from IBM SPSS Statistics data files can be used in a new data file by specifying the same vector name on both NAMES specifications.
- If you specify both VARIABLES and NAMES, a warning message is displayed and the VARIABLES specification is used.
- If you omit both the VARIABLES and NAMES specifications, or if you do not specify names for all columns of the matrix, the MATRIX procedure creates default names. The names have the form *COLn*, where *n* is the column number.
- Variable names are truncated to 8 bytes when they are referenced through a vector in the /NAMES subcommand.

STRINGS Specification

The STRINGS specification provides the names of variables that contain short string data rather than numeric data.

- By default, all variables are assumed to be numeric.
- The variable list specification following STRINGS consists of a list of variable names separated by commas. The names must be among those used by SAVE.
- Long variable names (up to 64 bytes) are supported when explicitly specified on the /STRINGS subcommand

MGET Statement: Reading Matrix Data Files

MGET reads a matrix-format data file. MGET puts the data it reads into separate matrix variables. It also names these new variables automatically. The syntax of MGET is as follows:

```
MGET [ [/] FILE = file reference]
  [/TYPE = {COV   }]
           {CORR  }
           {MEAN  }
           {STDDEV}
           {N     }
           {COUNT }
```

- Since MGET assigns names to the matrices it reads, do not specify matrix names on the MGET statement.

FILE Specification

FILE designates a matrix-format data file. See “MATRIX DATA” on page 1107 for a discussion of matrix-format data files. To designate the active dataset (if it is a matrix-format data file), use an asterisk, or simply omit the FILE specification.

- The file reference can be either a filename enclosed in quotes or a file handle defined on a FILE HANDLE command that precedes the matrix program. The file reference cannot be the name of a dataset in the current session.
- Both dataset references and physical file specifications are supported.
- The same matrix-format data file can be read more than once.
- If you omit the FILE specification, the current active dataset is used.
- MGET ignores the SPLIT FILE command when reading the active dataset. It does honor the split-file groups that were in effect when the matrix-format data file was created.
- The maximum number of split-file groups that can be read is 99.
- The maximum number of cells that can be read is 99.

TYPE Specification

TYPE specifies the rowtype(s) to read from the matrix-format data file.

- By default, records of all rowtypes are read.
- If the matrix-format data file does not contain rows of the requested type, an error occurs.

Valid keywords on the TYPE specification are:

COV. *A matrix of covariances.*

CORR. *A matrix of correlation coefficients.*

MEAN. *A vector of means.*

STDDEV. *A vector of standard deviations.*

N. *A vector of numbers of cases.*

COUNT. *A vector of counts.*

Names of Matrix Variables from MGET

- The MGET statement automatically creates matrix variable names for the matrices it reads.
- All new variables created by MGET are reported to the user.
- If a matrix variable already exists with the same name that MGET chose for a new variable, the new variable is not created and a warning is issued. The RELEASE statement can be used to get rid of a variable. A COMPUTE statement followed by RELEASE can be used to change the name of an existing matrix variable.

MGET constructs variable names in the following manner:

- The first two characters of the name identify the row type. If there are no cells and no split file groups, these two characters constitute the name:

  CV . A covariance matrix (rowtype COV)

  CR . A correlation matrix (rowtype CORR)

  MN . A vector of means (rowtype MEAN)

  SD . A vector of standard deviations (rowtype STDDEV)

  NC . A vector of numbers of cases (rowtype N)

  CN . A vector of counts (rowtype COUNT)
- Characters 3–5 of the variable name identify the cell number or the split-group number. Cell identifiers consist of the letter *F* and a two-digit cell number. Split-group identifiers consist of the letter *S* and a two-digit split-group number; for example, *MNF12* or *SDS22*.
- If there are both cells and split groups, characters 3–5 identify the cell and characters 6–8 identify the split group. The same convention for cell or split-file numbers is used; for example, *CRF12S21*.

- After the name is constructed as described above, any leading zeros are removed from the cell number and the split-group number; for example, *CNF2S99* or *CVF2S1*.

MSAVE Statement: Writing Matrix Data Files

The MSAVE statement writes matrix expressions to a matrix-format data file that can be used as matrix input to other procedures. (See "MATRIX DATA" on page 1107 for a discussion of matrix-format data files.) The syntax of MSAVE is as follows:

```
MSAVE matrix expression
   /TYPE = {COV   }
           {CORR  }
           {MEAN  }
           {STDDEV}
           {N     }
           {COUNT }
  [/OUTFILE = 'filespec']
  [/VARIABLES = variable list]
  [/SNAMES = variable list]
  [/SPLIT = split vector]
  [/FNAMES = variable list]
  [/FACTOR = factor vector]
```

- Only one matrix-format data file can be saved in a single matrix program.
- Each MSAVE statement writes records of a single rowtype. Therefore, several MSAVE statements will normally be required to write a complete matrix-format data file.
- Most specifications are retained from one MSAVE statement to the next so that it is not necessary to repeat the same specifications on a series of MSAVE statements. The exception is the FACTOR specification, as noted below.

Example

```
MSAVE M /TYPE=MEAN /OUTFILE=CORRMAT /VARIABLES=V1 TO V8.
MSAVE S /TYPE STDDEV.
MSAVE MAKE(1,8,24) /TYPE N.
MSAVE C /TYPE CORR.
```

- The series of MSAVE statements save the matrix variables *M*, *S*, and *C*, which contain, respectively, vectors of means and standard deviations and a matrix of correlation coefficients. The matrix-format data file thus created is suitable for use in a procedure such as FACTOR.
- The first MSAVE statement saves *M* as a vector of means. This statement specifies OUTFILE, a previously defined file handle, and VARIABLES, a list of variable names to be used in the IBM SPSS Statistics data file.
- The second MSAVE statement saves *S* as a vector of standard deviations. Note that the OUTFILE and VARIABLES specifications do not have to be repeated.
- The third MSAVE statement saves a vector of case counts. The matrix function MAKE constructs an eight-element vector with values equal to the case count (24 in this example).
- The last MSAVE statement saves *C*, an 8 × 8 matrix, as the correlation matrix.

Matrix Expression Specification

- The matrix expression must be specified first on the MSAVE statement.
- The matrix expression specification can be any matrix language expression that evaluates to the value(s) to be written to the matrix-format file.
- You can specify a matrix name, a matrix raised to a power, or a matrix function (with its arguments in parentheses) by itself, but you must enclose other matrix expressions in parentheses. For example, `MSAVE A`, `SAVE INV(A)`, or `MSAVE B**DET(T(C)*D)` is legal, but `MSAVE N * WT` is not. You must specify `MSAVE (N * WT)`.
- Constant expressions are allowed.

TYPE Specification

TYPE specifies the rowtype to write to the matrix-format data file. Only a single rowtype can be written by any one MSAVE statement.Valid keywords on the TYPE specification are:

COV. *A matrix of covariances.*

CORR. *A matrix of correlation coefficients.*

MEAN. *A vector of means.*

STDDEV. *A vector of standard deviations.*

N. *A vector of numbers of cases.*

COUNT. *A vector of counts.*

OUTFILE Specification

OUTFILE designates the matrix-format data file to which the matrices are to be written. It can be a filename in quotes or a file handle defined on a FILE HANDLE command that precedes the matrix program. The filename or handle must be a valid file specification. The file reference cannot be the name of a dataset in the current session.

- The OUTFILE specification is required on the first MSAVE statement in a matrix program.
- Since only one matrix-format data file can be written in a single matrix program, any OUTFILE specification on the second and later MSAVE statements in one matrix program must be the same as that on the first MSAVE statement.
- Both dataset references and physical file specifications are supported.

VARIABLES Specification

You can provide variable names for the matrix-format data file with the VARIABLES specification. The variable list is a list of valid variable names separated by commas. You can use the TO convention.

- The VARIABLES specification names only the data variables in the matrix. Split-file variables and grouping or factor variables are named on the SNAMES and FNAMES specifications.
- The names in the VARIABLES specification become the values of the special variable *VARNAME_* in the matrix-format data file for rowtypes of CORR and COV.
- You cannot specify the reserved names *ROWTYPE_* and *VARNAME_* on the VARIABLES specification.
- Long variable names (up to 64 bytes) are supported when explicitly specified on the /VARIABLES subcommand.
- If you omit the VARIABLES specification, the default names *COL1*, *COL2*, ..., etc., are used.

FACTOR Specification

To write a matrix-format data file with factor or group codes, you must use the FACTOR specification to provide a row matrix containing the values of each of the factors or group variables for the matrix expression being written by the current MSAVE statement.

- The factor vector must have the same number of columns as there are factors in the matrix data file being written. You can use a scalar when the groups are defined by a single variable. For example, FACTOR=1 indicates that the matrix data being written are for the value 1 of the factor variable.
- The values of the factor vector are written to the matrix-format data file as values of the factors in the file.
- To create a complete matrix-format data file with factors, you must execute an MSAVE statement for every combination of values of the factors or grouping variables (in other words, for every group). If split-file variables are also present, you must execute an MSAVE statement for every combination of factor codes within every combination of values of the split-file variables.

Example

```
MSAVE M11 /TYPE=MEAN /OUTFILE=CORRMAT /VARIABLES=V1 TO V8
    /FNAMES=SEX, GROUP /FACTOR={1,1}.
MSAVE S11 /TYPE STDDEV.
MSAVE MAKE(1,8,N(1,1)) /TYPE N.
MSAVE C11 /TYPE CORR.


MSAVE M12 /TYPE=MEAN /FACTOR={1,2}.
MSAVE S12 /TYPE STDDEV.
MSAVE MAKE(1,8,N(1,2)) /TYPE N.
MSAVE C12 /TYPE CORR.


MSAVE M21 /TYPE=MEAN /FACTOR={2,1}.
MSAVE S21 /TYPE STDDEV.
MSAVE MAKE(1,8,N(2,1)) /TYPE N.
MSAVE C21 /TYPE CORR.


MSAVE M22 /TYPE=MEAN /FACTOR={2,2}.
MSAVE S22 /TYPE STDDEV.
MSAVE MAKE(1,8,N(2,2)) /TYPE N.
MSAVE C22 /TYPE CORR.
```

- The first four MSAVE statements provide data for a group defined by the variables *SEX* and *GROUP*, with both factors having the value 1.
- The second, third, and fourth groups of four MSAVE statements provide the corresponding data for the other groups, in which *SEX* and *GROUP*, respectively, equal 1 and 2, 2 and 1, and 2 and 2.
- Within each group of MSAVE statements, a suitable number-of-cases vector is created with the matrix function MAKE.

FNAMES Specification

To write a matrix-format data file with factor or group codes, you can use the FNAMES specification to provide variable names for the grouping or factor variables.

- The variable list following the keyword FNAMES is a list of valid variable names, separated by commas.
- If you omit the FNAMES specification, the default names *FAC1*, *FAC2*, ..., etc., are used.

SPLIT Specification

To write a matrix-format data file with split-file groups, you must use the SPLIT specification to provide a row matrix containing the values of each of the split-file variables for the matrix expression being written by the current MSAVE statement.

- The split vector must have the same number of columns as there are split-file variables in the matrix data file being written. You can use a scalar when there is only one split-file variable. For example, SPLIT=3 indicates that the matrix data being written are for the value 3 of the split-file variable.
- The values of the split vector are written to the matrix-format data file as values of the split-file variable(s).
- To create a complete matrix-format data file with split-file variables, you must execute MSAVE statements for every combination of values of the split-file variables. (If factor variables are present, you must execute MSAVE statements for every combination of factor codes within every combination of values of the split-file variables.)

SNAMES Specification

To write a matrix-format data file with split-file groups, you can use the SNAMES specification to provide variable names for the split-file variables.

- The variable list following the keyword SNAMES is a list of valid variable names separated by commas.
- If you omit the SNAMES specification, the default names *SPL1*, *SPL2*, ..., etc., are used.

DISPLAY Statement

DISPLAY provides information on the matrix variables currently defined in a matrix program and on usage of internal memory by the matrix processor. Two keywords are available on DISPLAY:

DICTIONARY. *Display variable name and row and column dimensions for each matrix variable currently defined.*

STATUS. *Display the status and size of internal tables.* This display is intended as a debugging aid when writing large matrix programs that approach the memory limitations of your system.

If you enter the DISPLAY statement with no specifications, both DICTIONARY and STATUS information is displayed.

RELEASE Statement

Use the RELEASE statement to release the work areas in memory assigned to matrix variables that are no longer needed.

- Specify a list of currently defined matrix variables. Variable names on the list must be separated by commas.
- RELEASE discards the contents of the named matrix variables. Releasing a large matrix when it is no longer needed makes memory available for additional matrix variables.
- All matrix variables are released when the END MATRIX statement is encountered.

Macros Using the Matrix Language

Macro expansion (see "DEFINE-!ENDDEFINE" on page 555) occurs before command lines are passed to the matrix processor. Therefore, previously defined macro names can be used within a matrix program. If the macro name expands to one or more valid matrix statements, the matrix processor will execute those statements. Similarly, you can define an entire matrix program, including the MATRIX and END MATRIX commands, as a macro, but you cannot define a macro within a matrix program, since DEFINE and END DEFINE are not valid matrix statements.

MATRIX DATA

```
MATRIX DATA VARIABLES=varlist   [/FILE={INLINE**}]
                                       {file    }

 [/FORMAT=[{LIST**}]  [{LOWER**}]  [{DIAGONAL**}]]
          {FREE  }    {UPPER  }    {NODIAGONAL}
                      {FULL   }

 [/SPLIT=varlist]    [/FACTORS=varlist]

 [/CELLS=number of cells]    [/N=sample size]

 [/CONTENTS= [CORR**]  [COV]  [MAT]  [MSE]  [DFE]  [MEAN]  [PROX]
             [{STDDEV}]  [N_SCALAR]  [{N_VECTOR}]  [N_MATRIX]  [COUNT]]
              {SD    }                {N       }
```

**Default if the subcommand is omitted.

Example

```
MATRIX DATA VARIABLES=ROWTYPE_ SAVINGS POP15 POP75 INCOME GROWTH.
BEGIN DATA
MEAN 9.6710 35.0896 2.2930 1106.7784 3.7576
STDDEV 4.4804 9.1517 1.2907 990.8511 2.8699
N 50 50 50 50 50
CORR 1
CORR -.4555 1
CORR .3165 -.9085 1
CORR .2203 -.7562 .7870 1
CORR .3048 -.0478 .0253 -.1295  1
END DATA.
```

Overview

MATRIX DATA reads raw matrix materials and converts them to a matrix data file that can be read by procedures that handle matrix materials. The data can include vector statistics, such as means and standard deviations, as well as matrices.

MATRIX DATA is similar to a DATA LIST command: it defines variable names and their order in a raw data file. However, MATRIX DATA can read only data that conform to the general format of matrix data files.

Matrix Files

Like the matrix data files created by procedures, the file that MATRIX DATA creates contains the following variables in the indicated order. If the variables are in a different order in the raw data file, MATRIX DATA rearranges them in the active dataset.

- *Split-file variables.* These optional variables define split files. There can be up to eight split variables, and they must have numeric values. Split-file variables will appear in the order in which they are specified on the SPLIT subcommand.
- *ROWTYPE_.* This is a string variable with A8 format. Its values define the data type for each record. For example, it might identify a row of values as means, standard deviations, or correlation coefficients. Every matrix data file has a *ROWTYPE_* variable.
- *Factor variables.* There can be any number of factors. They occur only if the data include within-cells information, such as the within-cells means. Factors have the system-missing value on records that define pooled information. Factor variables appear in the order in which they are specified on the FACTORS subcommand.
- *VARNAME_.* This is a string variable with A8 format. MATRIX DATA automatically generates *VARNAME_* and its values based on the variables named on VARIABLES. You never enter values for *VARNAME_*. Values for *VARNAME_* are blank for records that define vector information. Every matrix in the program has a *VARNAME_* variable.

- *Continuous variables.* These are the variables that were used to generate the correlation coefficients or other aggregated data. There can be any number of them. Continuous variables appear in the order in which they are specified on VARIABLES.

Options

Data Files. You can define both inline data and data in an external file.

Important: Inline data are limited to 1024 bytes per line; any data past that limit will be truncated. Consider moving the inline data to an external file and using MATRIX DATA FILE=<path/file> to avoid any possible data loss.

Data Format. By default, data are assumed to be entered in freefield format with each vector or row beginning on a new record (the keyword LIST on the FORMAT subcommand). If each vector or row does not begin on a new record, use the keyword FREE. You can also use FORMAT to indicate whether matrices are entered in upper or lower triangular or full square or rectangular format and whether or not they include diagonal values.

Variable Types. You can specify split-file and factor variables using the SPLIT and FACTORS subcommands. You can identify record types by specifying ROWTYPE_ on the VARIABLES subcommand if *ROWTYPE_* values are included in the data or by implying *ROWTYPE_* values on CONTENTS.

Basic Specification

The basic specification is VARIABLES and a list of variables. Additional specifications are required as follows:

- FILE is required to specify the data file if the data are not inline.
- If data are in any format other than lower triangular with diagonal values included, FORMAT is required.
- If the data contain values in addition to matrix coefficients, such as the mean and standard deviation, either the variable *ROWTYPE_* must be specified on VARIABLES and *ROWTYPE_* values must be included in the data or CONTENTS must be used to describe the data.
- If the data include split-file variables, SPLIT is required. If there are factors, FACTORS is required.

Specifications on most MATRIX DATA subcommands depend on whether *ROWTYPE_* is included in the data and specified on VARIABLES or whether it is implied using CONTENTS.

*Table 68. Subcommand requirements in relation to ROWTYPE_*

Subcommand	Implicit ROWTYPE_ using CONTENTS	Explicit ROWTYPE_ on VARIABLES
FILE	Defaults to INLINE	Defaults to INLINE
VARIABLES	Required	Required
FORMAT	Defaults to LOWER DIAG	Defaults to LOWER DIAG
SPLIT	Required if split files*	Required if split files
FACTORS	Required if factors	Required if factors
CELLS	Required if factors	Inapplicable
CONTENTS	Defaults to CORR	Optional
N	Optional	Optional

* If the data do not contain values for the split-file variables, this subcommand can specify a single variable, which is not specified on the VARIABLES subcommand.

Subcommand Order

- SPLIT and FACTORS, when used, must follow VARIABLES.

- The remaining subcommands can be specified in any order.

Syntax Rules

- No commands can be specified between MATRIX DATA and BEGIN DATA, not even a VARIABLE LABELS or FORMAT command. Data transformations cannot be used until after MATRIX DATA is executed.

Examples

Reading a Correlation Matrix

```
MATRIX DATA
    VARIABLES=ROWTYPE_ SAVINGS POP15 POP75 INCOME GROWTH.
BEGIN DATA
MEAN 9.6710 35.0896 2.2930 1106.7784 3.7576
STDDEV 4.4804 9.1517 1.2907 990.8511 2.8699
N 50 50 50 50 50
CORR 1
CORR -.4555 1
CORR .3165 -.9085 1
CORR .2203 -.7562 .7870 1
CORR .3048 -.0478 .0253 -.1295  1
END DATA.
```

- The variable *ROWTYPE_* is specified on VARIABLES. *ROWTYPE_* values are included in the data.
- No other specifications are required.

MATRIX DATA with DISCRIMINANT

```
MATRIX DATA VARIABLES=WORLD ROWTYPE_ FOOD APPL SERVICE RENT
  /FACTORS=WORLD.
BEGIN DATA
1 N      25 25 25 25
1 MEAN   76.64 77.32 81.52 101.40
2 N      7 7 7 7
2 MEAN   76.1428571 85.2857143 60.8571429 249.571429
3 N      13 13 13 13
3 MEAN   55.5384615 76 63.4615385 86.3076923
. SD     16.4634139 22.5509310 16.8086768 77.1085326
. CORR   1
. CORR   .1425366 1
. CORR   .5644693 .2762615 1
. CORR   .2133413 -.0499003 .0417468 1
END DATA.

DISCRIMINANT GROUPS=WORLD(1,3)
  /VARIABLES=FOOD APPL SERVICE RENT /METHOD=WILKS /MATRIX=IN(*).
```

- MATRIX DATA is used to generate a active dataset that DISCRIMINANT can read. DISCRIMINANT reads the mean, count (unweighted *N*), and *N* (weighted *N*) for each cell in the data, as well as the pooled values for the standard deviation and correlation coefficients. If count equals *N*, only *N* needs to be supplied.
- *ROWTYPE_* is specified on VARIABLES to identify record types in the data. Though CONTENTS and CELLS can be used to identify record types and distinguish between within-cells data and pooled values, it is usually easier to specify *ROWTYPE_* on VARIABLES and enter the *ROWTYPE_* values in the data.
- Because factors are present in the data, the continuous variables (*FOOD, APPL, SERVICE,* and *RENT*) must be specified last on VARIABLES and must be last in the data.
- The FACTORS subcommand identifies *WORLD* as the factor variable.
- BEGIN DATA immediately follows MATRIX DATA.
- *N* and *MEAN* values for each cell are entered in the data.
- *ROWTYPE_* values for the pooled records are *SD* and *COR*. MATRIX DATA assigns the values *STDDEV* and *CORR* to the corresponding vectors in the matrix. Records with pooled information have the system-missing value (.) for the factors.
- The DISCRIMINANT procedure reads the data matrix. An asterisk (*) is specified as the input file on the MATRIX subcommand because the data are in the active dataset.

MATRIX DATA with REGRESSION

```
MATRIX DATA VARIABLES=SAVINGS POP15 POP75 INCOME GROWTH
  /CONTENTS=MEAN SD N CORR /FORMAT=UPPER NODIAGONAL.

BEGIN DATA
9.6710 35.0896 2.2930 1106.7784 3.7576
4.4804 9.1517 1.2908 990.8511 2.8699
50 50 50 50 50
-.4555 .3165 .2203 .3048
-.9085 -.7562 -.0478
 .7870 .0253
-.1295
END DATA.

REGRESSION MATRIX=IN(*) /VARIABLES=SAVINGS TO GROWTH
  /DEP=SAVINGS /ENTER.
```

- MATRIX DATA is used to generate a matrix that REGRESSION can read. REGRESSION reads and writes matrices that always contain the mean, standard deviation, *N*, and Pearson correlation coefficients. Data in this example do not have *ROWTYPE_* values, and the correlation values are from the upper triangle of the matrix without the diagonal values.
- *ROWTYPE_* is not specified on VARIABLES because its values are not included in the data.
- Because there are no *ROWTYPE_* values, CONTENTS is required to define the record types and the order of the records in the file.
- By default, MATRIX DATA reads values from the lower triangle of the matrix, including the diagonal values. FORMAT is required in this example to indicate that the data are in the upper triangle and do not include diagonal values.
- BEGIN DATA immediately follows the MATRIX DATA command.
- The REGRESSION procedure reads the data matrix. An asterisk (*) is specified as the input file on the MATRIX subcommand because the data are in the active dataset. Since there is a single vector of *N*'s in the data, missing values are handled listwise (the default for REGRESSION).

MATRIX DATA with ONEWAY

```
MATRIX DATA VARIABLES=EDUC ROWTYPE_ WELL /FACTORS=EDUC.
BEGIN DATA
1 N 65
2 N 95
3 N 181
4 N 82
5 N 40
6 N 37
1 MEAN 2.6462
2 MEAN 2.7737
3 MEAN 4.1796
4 MEAN 4.5610
5 MEAN 4.6625
6 MEAN 5.2297
. MSE 6.2699
. DFE 494
END DATA.

ONEWAY WELL BY EDUC /MATRIX=IN(*)
```

- One of the two types of matrices that the ONEWAY procedure reads includes a vector of frequencies for each factor level, a vector of means for each factor level, a record containing the pooled variance (within-group mean square error), and the degrees of freedom for the mean square error. MATRIX DATA is used to generate an active dataset containing this type of matrix data for the ONEWAY procedure.
- *ROWTYPE_* is explicit on VARIABLES and identifies record types.
- Because factors are present in the data, the continuous variables (*WELL*) must be specified last on VARIABLES and must be last in the data.
- The FACTORS subcommand identifies *EDUC* as the factor variable.
- *MSE* is entered in the data as the *ROWTYPE_* value for the vector of square pooled standard deviations.
- *DFE* is entered in the data as the *ROWTYPE_* value for the vector of degrees of freedom.
- Records with pooled information have the system-missing value (.) for the factors.

Operations

- MATRIX DATA defines and writes data in one step.
- MATRIX DATA clears the active dataset and defines a new active dataset.
- If CONTENTS is not specified and *ROWTYPE_* is not specified on VARIABLES, MATRIX DATA assumes that the data contain only CORR values and issues warning messages to alert you to its assumptions.
- With the default format, data values, including diagonal values, must be in the lower triangle of the matrix. If MATRIX DATA encounters values in the upper triangle, it ignores those values and issues a series of warnings.
- With the default format, if any matrix rows span records in the data file, MATRIX DATA cannot form the matrix properly.
- MATRIX DATA does not allow format specifications for matrix materials. The procedure assigns the formats shown in the following table. To change data formats, execute MATRIX DATA and then assign new formats with the FORMATS, PRINT FORMATS, or WRITE FORMATS command.

Table 69. Print and write formats for matrix variables

Variable type	Format
*ROWTYPE_, VARNAME_*	A8
Split-file variables	F4.0
Factors	F4.0
Continuous variables	F10.4

Format of the Raw Matrix Data File

- If LIST is in effect on the FORMAT subcommand, the data are entered in freefield format, with blanks and commas used as separators and each scalar, vector, or row of the matrix beginning on a new record. Unlike LIST format with DATA LIST, a vector or row of the matrix can be contained on multiple records. The continuation records do not have a value for *ROWTYPE_*.
- *ROWTYPE_* values can be enclosed in quotes.
- The order of variables in the raw data file must match the order in which they are specified on VARIABLES. However, this order does not have to correspond to the order of variables in the resulting matrix data file.
- The way records are entered for pooled vectors or matrices when factors are present depends upon whether *ROWTYPE_* is specified on the VARIABLES subcommand. See the topic “FACTORS Subcommand ” on page 1115 for more information.
- MATRIX DATA recognizes plus and minus signs as field separators when they are not preceded by the letter *D* or *E*. This allows MATRIX DATA to read scientific notation as well as correlation matrices written by FORTRAN in F10.8 format. A plus sign preceded by a *D* or *E* is read as part of the number in scientific notation.

VARIABLES Subcommand

VARIABLES specifies the names of the variables in the raw data and the order in which they occur.

- VARIABLES is required.
- There is no limit to the number of variables that can be specified.
- If *ROWTYPE_* is specified on VARIABLES, the continuous variables must be the last variables specified on the subcommand and must be last in the data.
- If split-file variables are present, they must also be specified on SPLIT.

- If factor variables are present, they must also be specified on FACTORS.

When either of the following is true, the only variables that must be specified on VARIABLES are the continuous variables:

1. The data contain only correlation coefficients. There can be no additional information, such as the mean and standard deviation, and no factor information or split-file variables. MATRIX DATA assigns the record type *CORR* to all records.
2. CONTENTS is used to define all record types. The data can then contain information such as the mean and standard deviation, but no factor, split-file, or *ROWTYPE_* variables. MATRIX DATA assigns the record types defined on the CONTENTS subcommand.

Variable VARNAME_

*VARNAME_* cannot be specified on the VARIABLES subcommand or anywhere on MATRIX DATA, and its values cannot be included in the data. The MATRIX DATA command generates the variable *VARNAME_* automatically.

Variable ROWTYPE_

- *ROWTYPE_* is a string variable with A8 format. Its values define the data types. All matrix data files contain a *ROWTYPE_* variable.
- If *ROWTYPE_* is specified on VARIABLES and its values are entered in the data, MATRIX DATA is primarily used to define the names and order of the variables in the raw data file.
- *ROWTYPE_* must precede the continuous variables.
- Valid values for *ROWTYPE_* are *CORR*, *COV*, *MAT*, *MSE*, *DFE*, *MEAN*, *STDDEV* (or *SD*), *N_VECTOR* (or *N*), *N_SCALAR*, *N_MATRIX*, *COUNT*, or *PROX*. For definitions of these values. See the topic "CONTENTS Subcommand " on page 1117 for more information. Three-character abbreviations for these values are permitted. These values can also be enclosed in quotation marks or apostrophes.
- If *ROWTYPE_* is not specified on VARIABLES, CONTENTS must be used to define the order in which the records occur within the file. MATRIX DATA follows these specifications strictly and generates a *ROWTYPE_* variable according to the CONTENTS specifications. A data-entry error, especially skipping a record, can cause the procedure to assign the wrong values to the wrong records.

Example

```
* ROWTYPE_ is specified on VARIABLES.

MATRIX DATA
     VARIABLES=ROWTYPE_ SAVINGS POP15 POP75 INCOME GROWTH.
BEGIN DATA
MEAN 9.6710 35.0896 2.2930 1106.7784 3.7576
STDDEV 4.4804 9.1517 1.2907 990.8511 2.8699
N 50 50 50 50 50
CORR 1
CORR -.4555 1
CORR .3165 -.9085 1
CORR .2203 -.7562 .7870 1
CORR .3048 -.0478 .0253 -.1295  1
END DATA.
```

- *ROWTYPE_* is specified on VARIABLES. *ROWTYPE_* values in the data identify each record type.
- Note that *VARNAME_* is not specified on VARIABLES, and its values are not entered in the data.

Example

```
* ROWTYPE_ is specified on VARIABLES.

MATRIX DATA
     VARIABLES=ROWTYPE_ SAVINGS POP15 POP75 INCOME GROWTH.
BEGIN DATA
'MEAN    ' 9.6710 35.0896 2.2930 1106.7784 3.7576
'SD      ' 4.4804 9.1517 1.2907 990.8511 2.8699
'N       ' 50 50 50 50 50
"CORR    " 1
"CORR    " -.4555 1
"CORR    " .3165 -.9085 1
"CORR    " .2203 -.7562 .7870 1
"CORR    " .3048 -.0478 .0253 -.1295  1
END DATA.
```

- *ROWTYPE_* values for the mean, standard deviation, *N*, and Pearson correlation coefficients are abbreviated and enclosed in quotes.

Example

```
* ROWTYPE_ is not specified on VARIABLES.

MATRIX DATA VARIABLES=SAVINGS POP15 POP75 INCOME GROWTH
  /CONTENTS=MEAN SD N CORR.
BEGIN DATA
9.6710 35.0896 2.2930 1106.7784 3.7576
4.4804 9.1517 1.2907 990.8511 2.8699
50 50 50 50 50
 1
-.4555 1
 .3165 -.9085 1
 .2203 -.7562 .7870 1
 .3048 -.0478 .0253 -.1295 1
END DATA.
```

- *ROWTYPE_* is not specified on VARIABLES, and its values are not included in the data.
- CONTENTS is required to define the record types and the order of the records in the file.

FILE Subcommand

FILE specifies the matrix file containing the data. The default specification is INLINE, which indicates that the data are included within the command sequence between the BEGIN DATA and END DATA commands.

- If the data are in an external file, FILE must specify the file.
- If the FILE subcommand is omitted, the data must be inline.

Example

```
MATRIX DATA FILE=RAWMTX /VARIABLES=varlist.
```

- FILE indicates that the data are in the file *RAWMTX*.

FORMAT Subcommand

FORMAT indicates how the matrix data are formatted. It applies only to matrix values in the data, not to vector values, such as the mean and standard deviation.

- FORMAT can specify up to three keywords: one to specify the data-entry format, one to specify matrix shape, and one to specify whether the data include diagonal values.
- The minimum specification is a single keyword.
- Default settings remain in effect unless explicitly overridden.

Data-Entry Format

FORMAT has two keywords that specify the data-entry format:

LIST. *Each scalar, vector, and matrix row must begin on a new record.* A vector or row of the matrix may be continued on multiple records. This is the default.

FREE. *Matrix rows do not need to begin on a new record.* Any item can begin in the middle of a record.

Matrix Shape

FORMAT has three keywords that specify the matrix shape. With either triangular shape, no values—not even missing indicators—are entered for the implied values in the matrix.

LOWER. *Read data values from the lower triangle.* This is the default.

UPPER. *Read data values from the upper triangle.*

FULL. *Read the full square matrix of data values.* FULL cannot be specified with NODIAGONAL.

Diagonal Values

FORMAT has two keywords that refer to the diagonal values:

DIAGONAL. *Data include the diagonal values.* This is the default.

NODIAGONAL. *Data do not include diagonal values.* The diagonal value is set to the system-missing value for all matrices except the correlation matrices. For correlation matrices, the diagonal value is set to 1. NODIAGONAL cannot be specified with FULL.

The following table shows how data might be entered for each combination of FORMAT settings that govern matrix shape and diagonal values. With UPPER NODIAGONAL and LOWER NODIAGONAL, you do not enter the matrix row that has blank values for the continuous variables. If you enter that row, MATRIX DATA cannot properly form the matrix.

Table 70. Various FORMAT settings

FULL	UPPER DIAGONAL	UPPER NODIAGONAL	LOWER DIAGONAL	LOWER NODIAGONAL
MEAN 5 4 3	MEAN 5 4 3	MEAN 5 4 3	MEAN 5 4 3	MEAN 5 4 3
SD 3 2 1	SD 3 2 1	SD 3 2 1	SD 3 2 1	SD 3 2 1
N 9 9 9	N 9 9 9	N 9 9 9	N 9 9 9	N 9 9 9
CORR 1 .6 .7	CORR 1 .6 .7	CORR .6 .7	CORR 1	CORR .6
CORR .6 1 .8	CORR 1 .8	CORR .8	CORR .6 1	CORR .7 .8
CORR .7 .8 1	CORR 1		CORR .7 .8 1	

Example

```
MATRIX DATA VARIABLES=ROWTYPE_ V1 TO V3
   /FORMAT=UPPER NODIAGONAL.
BEGIN DATA
MEAN   5  4  3
SD     3  2  1
N      9  9  9
CORR     .6 .7
CORR        .8
END DATA.
LIST.
```

- FORMAT specifies the upper-triangle format with no diagonal values. The default LIST is in effect for the data-entry format.

Example

```
MATRIX DATA VARIABLES=ROWTYPE_ V1 TO V3
   /FORMAT=UPPER NODIAGONAL.
BEGIN DATA
MEAN 5 4 3
SD 3 2 1
N 9 9 9
CORR .6 .7
CORR .8
END DATA.
LIST.
```

- This example is identical to the previous example. It shows that data do not have to be aligned in columns. Data throughout this section are aligned in columns to emphasize the matrix format.

SPLIT Subcommand

SPLIT specifies the variables whose values define the split files. SPLIT must follow the VARIABLES subcommand.

- SPLIT can specify a subset of up to eight of the variables named on VARIABLES. All split variables must be numeric. The keyword TO can be used to imply variables in the order in which they are named on VARIABLES.
- A separate matrix must be included in the data for each value of each split variable. MATRIX DATA generates a complete set of matrix materials for each.
- If the data contain neither *ROWTYPE_* nor split-file variables, a single split-file variable can be specified on SPLIT. This variable is *not* specified on the VARIABLES subcommand. MATRIX DATA generates a complete set of matrix materials for each set of matrix materials in the data and assigns values 1, 2, 3, etc., to the split variable until the end of the data is encountered.

Example

```
MATRIX DATA  VARIABLES=S1 ROWTYPE_ V1 TO V3 /SPLIT=S1.
BEGIN DATA
0 MEAN   5  4  3
0 SD     1  2  3
0 N      9  9  9
0 CORR   1
0 CORR  .6  1
0 CORR  .7 .8  1
1 MEAN   9  8  7
1 SD     5  6  7
1 N      9  9  9
1 CORR   1
1 CORR  .4  1
1 CORR  .3 .2  1
END DATA.
LIST.
```

- The split variable *S1* has two values: 0 and 1. Two separate matrices are entered in the data, one for each value *S1*.
- *S1* must be specified on both VARIABLES and SPLIT.

Example

```
MATRIX DATA VARIABLES=V1 TO V3 /CONTENTS=MEAN SD N CORR
   /SPLIT=SPL.
BEGIN DATA
  5  4  3
  1  2  3
  9  9  9
  1
 .6  1
 .7 .8  1
  9  8  7
  5  6  7
  9  9  9
  1
 .4  1
 .3 .2  1
END DATA.
LIST.
```

- The split variable *SPL* is not specified on VARIABLES, and values for *SPL* are not included in the data.
- Two sets of matrix materials are included in the data. MATRIX DATA therefore assigns values 1 and 2 to variable *SPL* and generates two matrices in the matrix data file.

FACTORS Subcommand

FACTORS specifies the variables whose values define the cells represented by the within-cells data. FACTORS must follow the VARIABLES subcommand.

- FACTORS specifies a subset of the variables named on the VARIABLES subcommand. The keyword TO can be used to imply variables in the order in which they are named on VARIABLES.
- If *ROWTYPE_* is explicit on VARIABLES and its values are included in the data, records that represent pooled information have the system-missing value (indicated by a period) for the factors, since the values of *ROWTYPE_* are ambiguous.
- If *ROWTYPE_* is not specified on VARIABLES and its values are not in the data, enter data values for the factors only for records that represent within-cells information. Enter nothing for the factors for records that represent pooled information. CELLS must be specified to indicate the number of within-cells records, and CONTENTS must be specified to indicate which record types have within-cells data.

Example

```
* Rowtype is explicit.

MATRIX DATA VARIABLES=ROWTYPE_ F1 F2  VAR1 TO VAR3
  /FACTORS=F1 F2.
BEGIN DATA
MEAN 1 1  1  2  3
SD   1 1  5  4  3
N    1 1  9  9  9
MEAN 1 2  4  5  6
SD   1 2  6  5  4
N    1 2  9  9  9
MEAN 2 1  7  8  9
SD   2 1  7  6  5
N    2 1  9  9  9
MEAN 2 2  9  8  7
SD   2 2  8  7  6
N    2 2  9  9  9
CORR . . 1
CORR . . .6  1
CORR . . .7 .8  1
END DATA.
```

- *ROWTYPE_* is specified on VARIABLES.
- Factor variables must be specified on both VARIABLES and FACTORS.
- Periods in the data represent missing values for the *CORR* factor values.

Example

```
* Rowtype is implicit.

MATRIX DATA VARIABLES=F1 F2  VAR1 TO VAR3
  /FACTORS=F1 F2 /CONTENTS=(MEAN SD N) CORR /CELLS=4.
BEGIN DATA
1 1  1  2  3
1 1  5  4  3
1 1  9  9  9
1 2  4  5  6
1 2  6  5  4
1 2  9  9  9
2 1  7  8  9
2 1  7  6  5
2 1  9  9  9
2 2  9  8  7
2 2  8  7  6
2 2  9  9  9
     1
    .6  1
    .7 .8 1
END DATA.
```

- *ROWTYPE_* is not specified on VARIABLES. The three values after the factor values in each row contain the *ROWTYPE_* values for the three continuous variables: *VAR1*, *VAR2*, and *VAR3*.
- Nothing is entered for the *CORR* factor values because the records contain pooled information.
- CELLS is required because there are factors in the data and *ROWTYPE_* is implicit.
- CONTENTS is required to define the record types and to differentiate between the within-cells and pooled types.

Note that the data rows in this example are the same as in the previous example, except for the presence/absence of *ROWTYPE_*.

CELLS Subcommand

CELLS specifies the number of within-cells records in the data. The only valid specification for CELLS is a single integer, which indicates the number of sets of within-cells information that MATRIX DATA must read.

- CELLS is required when there are factors in the data and *ROWTYPE_* is implicit.
- If CELLS is used when *ROWTYPE_* is specified on VARIABLES, MATRIX DATA issues a warning and ignores the CELLS subcommand.

Example

```
MATRIX DATA VARIABLES=F1 VAR1 TO VAR3 /FACTORS=F1 /CELLS=2
  /CONTENTS=(MEAN SD N) CORR.
BEGIN DATA
1  5  4  3
1  3  2  1
```

```
1  9  9  9
2  8  7  6
2  6  7  8
2  9  9  9
   1
  .6  1
  .7 .8  1
END DATA.
```

- The specification for CELLS is 2 because the factor variable *F1* has two values (1 and 2) and there are therefore two sets of within-cells information.
- If there were two factor variables, *F1* and *F2*, and each had two values, 1 and 2, CELLS would equal 4 to account for all four possible factor combinations (assuming all that 4 combinations are present in the data).

CONTENTS Subcommand

CONTENTS defines the record types when *ROWTYPE_* is not included in the data. The minimum specification is a single keyword indicating a type of record. The default is CORR.

- CONTENTS is required to define record types and record order whenever *ROWTYPE_* is not specified on VARIABLES and its values are not in the data. The only exception to this rule is the rare situation in which all data values represent pooled correlation records and there are no factors. In that case, MATRIX DATA reads the data values and assigns the default *ROWTYPE_* of *CORR* to all records.
- The order in which keywords are specified on CONTENTS must correspond to the order in which records appear in the data. If the keywords on CONTENTS are in the wrong order, MATRIX DATA will incorrectly assign values.

CORR. *Matrix of correlation coefficients.* This is the default. If *ROWTYPE_* is not specified on the VARIABLES subcommand and you omit the CONTENTS subcommand, MATRIX DATA assigns the *ROWTYPE_* value *CORR* to all matrix rows.

COV. *Matrix of covariance coefficients.*

MAT. *Generic square matrix.*

MSE. *Vector of mean squared errors.*

DFE. *Vector of degrees of freedom.*

MEAN. *Vector of means.*

STDDEV. *Vector of standard deviations.* SD is a synonym for STDDEV. MATRIX DATA assigns the *ROWTYPE_* value STDDEV to the record if either STDDEV or SD is specified.

N_VECTOR. *Vector of counts. N* is a synonym for N_VECTOR. MATRIX DATA assigns the *ROWTYPE_* value *N* to the record.

N_SCALAR. *Count.* Scalars are a shorthand mechanism for representing vectors in which all elements have the same value, such as when a vector of *N*'s is calculated using listwise deletion of missing values. Enter N_SCALAR as the *ROWTYPE_* value in the data and then the N_SCALAR value for the first continuous variable only. MATRIX DATA assigns the *ROWTYPE_* value *N* to the record and copies the specified N_SCALAR value across all of the continuous variables.

N_MATRIx. *Square matrix of counts.* Enter N_MATRIX as the *ROWTYPE_* value for each row of counts in the data. MATRIX DATA assigns the *ROWTYPE_* value *N* to each of those rows.

COUNT. *Count vector accepted by procedure DISCRIMINANT.* This contains unweighted *N*'s.

PROx. *Matrix produced by PROXIMITIES.* Any proximity matrix can be used with PROXIMITIES or CLUSTER. A value label of *SIMILARITY* or *DISSIMILARITY* should be specified for PROX by using the VALUE LABELS command after END DATA.

Example

```
MATRIX DATA VARIABLES=V1 TO V3 /CONTENTS=MEAN SD N_SCALAR CORR.
BEGIN DATA
  5  4  3
```

```
  3  2  1
  9
  1
 .6  1
 .7 .8  1
END DATA.
LIST.
```

- *ROWTYPE_* is not specified on VARIABLES, and *ROWTYPE_* values are not in the data. CONTENTS is therefore required to identify record types.
- CONTENTS indicates that the matrix records are in the following order: mean, standard deviation, *N*, and correlation coefficients.
- The *N_SCALAR* value is entered for the first continuous variable only.

Example

```
MATRIX DATA VARIABLES=V1 TO V3 /CONTENTS=PROX.
BEGIN DATA

data records

END DATA.
VALUE LABELS ROWTYPE_ 'PROX' 'DISSIMILARITY'.
```

- CONTENTS specifies PROX to read a raw matrix and create a matrix data file in the same format as one produced by procedure PROXIMITIES. PROX is assigned the value label *DISSIMILARITY*.

Within-Cells Record Definition

When the data include factors and *ROWTYPE_* is not specified, CONTENTS distinguishes between within-cells and pooled records by enclosing the keywords for within-cells records in parentheses.

- If the records associated with the within-cells keywords appear together for each set of factor values, enclose the keywords together within a single set of parentheses.
- If the records associated with each within-cells keyword are grouped together across factor values, enclose the keyword within its own parentheses.

Example

```
MATRIX DATA VARIABLES=F1 VAR1 TO VAR3 /FACTORS=F1 /CELLS=2
  /CONTENTS=(MEAN SD N) CORR.
```

- MEAN, SD, and N contain within-cells information and are therefore specified within parentheses. CORR is outside the parentheses because it identifies pooled records.
- CELLS is required because there is a factor specified and *ROWTYPE_* is implicit.

Example

```
MATRIX DATA VARIABLES=F1 VAR1 TO VAR3 /FACTORS=F1 /CELLS=2
  /CONTENTS=(MEAN SD N) CORR.
BEGIN DATA
1  5  4  3
1  3  2  1
1  9  9  9
2  4  5  6
2  6  5  4
2  9  9  9
   1
  .6  1
  .7 .8  1
END DATA.
```

- The parentheses around the CONTENTS keywords indicate that the mean, standard deviation, and *N* for value 1 of factor *F1* are together, followed by the mean, standard deviation, and *N* for value 2 of factor *F1*.

Example

```
MATRIX DATA VARIABLES=F1 VAR1 TO VAR3 /FACTORS=F1 /CELLS=2
  /CONTENTS=(MEAN) (SD) (N) CORR.
BEGIN DATA
1  5  4  3
2  4  5  6
1  3  2  1
2  6  5  4
1  9  9  9
2  9  9  9
```

```
   1
  .6  1
  .7 .8  1
END DATA.
```

- The parentheses around each CONTENTS keyword indicate that the data include the means for all cells, followed by the standard deviations for all cells, followed by the *N* values for all cells.

Example

```
MATRIX DATA VARIABLES=F1 VAR1 TO VAR3 /FACTORS=F1 /CELLS=2
  /CONTENTS=(MEAN SD) (N) CORR.
BEGIN DATA
1  5  4  3
1  3  2  1
2  4  5  6
2  6  5  4
1  9  9  9
2  9  9  9
   1
  .6  1
  .7 .8  1
END DATA.
```

- The parentheses around the CONTENTS keywords indicate that the data include the mean and standard deviation for value 1 of *F1*, followed by the mean and standard deviation for value 2 of *F1*, followed by the *N* values for all cells.

Optional Specification When ROWTYPE_ Is Explicit

When *ROWTYPE_* is explicitly named on VARIABLES, MATRIX DATA uses *ROWTYPE_* values to determine record types.

- When *ROWTYPE_* is explicitly named on VARIABLES, CONTENTS can be used for informational purposes. However, *ROWTYPE_* values in the data determine record types.
- If MATRIX DATA reads values for *ROWTYPE_* that are not specified on CONTENTS, it issues a warning.
- Missing values for factors are entered as periods, even though CONTENTS is specified. See the topic “FACTORS Subcommand ” on page 1115 for more information.

Example

```
MATRIX DATA VARIABLES=ROWTYPE_ F1 F2 VAR1 TO VAR3
  /FACTORS=F1 F2 /CONTENTS=(MEAN SD N) CORR.
BEGIN DATA
MEAN 1 1  1  2  3
SD   1 1  5  4  3
N    1 1  9  9  9
MEAN 1 2  4  5  6
SD   1 2  6  5  4
N    1 2  9  9  9
CORR . .  1
CORR . . .6  1
CORR . . .7 .8  1
END DATA.
```

- *ROWTYPE_* is specified on VARIABLES. MATRIX DATA therefore uses *ROWTYPE_* values in the data to identify record types.
- Because *ROWTYPE_* is specified on VARIABLES, CONTENTS is optional. However, CONTENTS is specified for informational purposes. This is most useful when data are in an external file and the *ROWTYPE_* values cannot be seen in the data.
- Missing values for factors are entered as periods, even though CONTENTS is specified.

N Subcommand

N specifies the population *N* when the data do not include it. The only valid specification is an integer, which indicates the population *N*.

- MATRIX DATA generates one record with a *ROWTYPE_* of *N* for each split file, and it uses the specified *N* value for each continuous variable.

Example

```
MATRIX DATA VARIABLES=V1 TO V3 /CONTENTS=MEAN SD CORR
   /N=99.
BEGIN DATA
  5  4  3
  3  4  5
  1
 .6  1
 .7 .8  1
END DATA.
```

- MATRIX DATA uses 99 as the *N* value for all continuous variables.

MCONVERT

```
MCONVERT [[/MATRIX=] [IN({*   })] [OUT({*   })]]
                        {file}        {file}
         [{/REPLACE}]
          {/APPEND }
```

Example

```
MCONVERT MATRIX=OUT(CORMTX) /APPEND.
```

Overview

MCONVERT converts covariance matrix materials to correlation matrix materials, or vice versa. For MCONVERT to convert a correlation matrix, the matrix data must contain *CORR* values (Pearson correlation coefficients) and a vector of standard deviations (*STDDEV*). For MCONVERT to convert a covariance matrix, only *COV* values are required in the data.

Options

Matrix Files. MCONVERT can read matrix materials from an external matrix data file, and it can write converted matrix materials to an external file.

Matrix Materials. MCONVERT can write the converted matrix only or both the converted matrix and the original matrix to the resulting matrix data file.

Basic Specification

The minimum specification is the command itself. By default, MCONVERT reads the original matrix from the active dataset and then replaces it with the converted matrix.

Syntax Rules

- The keywords IN and OUT cannot specify the same external file.
- The APPEND and REPLACE subcommands cannot be specified on the same MCONVERT command.

Operations

- If the data are covariance matrix materials, MCONVERT converts them to a correlation matrix plus a vector of standard deviations.
- If the data are a correlation matrix and vector of standard deviations, MCONVERT converts them to a covariance matrix.
- If there are multiple *CORR* or *COV* matrices (for example, one for each grouping (factor) or one for each split variable), each will be converted to a separate matrix, preserving the values of any factor or split variables.
- All cases with *ROWTYPE_* values other than *CORR* or *COV*, such as *MEAN*, *N*, and *STDDEV*, are always copied into the new matrix data file.
- MCONVERT cannot read raw matrix values. If your data are raw values, use the MATRIX DATA command.
- Split variables (if any) must occur first in the file that MCONVERT reads, followed by the variable *ROWTYPE_*, the grouping variables (if any), and the variable *VARNAME_*. All variables following *VARNAME_* are the variables for which a matrix will be read and created.

Limitations

- The total number of split variables plus grouping variables cannot exceed eight.

Examples

```
MATRIX DATA VARIABLES=ROWTYPE_ SAVINGS POP15 POP75 INCOME GROWTH
  /FORMAT=FULL.
BEGIN DATA
COV    20.0740459  -18.678638    1.8304990  978.181242 3.9190106
COV   -18.678638    83.7541100 -10.731666 -6856.9888  -1.2561071
COV     1.8304990  -10.731666    1.6660908 1006.52742   .0937992
COV   978.181242 -6856.9888   1006.52742 981785.907 -368.18652
COV     3.9190106   -1.2561071    .0937992 -368.18652  8.2361574
END DATA.
MCONVERT.
```

- MATRIX DATA defines the variables in the file and creates a active dataset of matrix materials. The values for the variable *ROWTYPE_* are *COV*, indicating that the matrix contains covariance coefficients. The FORMAT subcommand indicates that data are in full square format.
- MCONVERT converts the covariance matrix to a correlation matrix plus a vector of standard deviations. By default, the converted matrix is written to the active dataset.

MATRIX Subcommand

The MATRIX subcommand specifies the file for the matrix materials. By default, MATRIX reads the original matrix from the active dataset and replaces the active dataset with the converted matrix.

- MATRIX has two keywords, IN and OUT. The specification on both IN and OUT is the name of an external file in parentheses or an asterisk (*) to refer to the active dataset (the default).
- The actual keyword MATRIX is optional.
- IN and OUT cannot specify the same external file.
- MATRIX=IN cannot be specified unless an active dataset has already been defined. To convert an existing matrix at the beginning of a session, use GET to retrieve the matrix file and then specify IN(*) on MATRIX.

IN. *The matrix file to read.*

OUT. *The matrix file to write.*

Example

```
GET FILE=COVMTX.
MCONVERT MATRIX=OUT(CORMTX).
```

- GET retrieves the matrix data file *COVMTX*. *COVMTX* becomes the active dataset.
- By default, MCONVERT reads the original matrix from the active dataset. IN(*) can be specified to make the default explicit.
- The keyword OUT on MATRIX writes the converted matrix to file *CORMTX*.

REPLACE and APPEND Subcommands

By default, MCONVERT writes only the converted matrix to the resulting matrix file. Use APPEND to copy both the original matrix and the converted matrix.

- The only specification is the keyword REPLACE or APPEND.
- REPLACE and APPEND are alternatives.
- REPLACE and APPEND affect the resulting matrix file only. The original matrix materials, whether in the active dataset or in an external file, remain intact.

APPEND. *Write the original matrix followed by the converted matrix to the matrix file.* If there are multiple sets of matrix materials, APPEND appends each converted matrix to the end of a copy of its original matrix.

REPLACE. *Write the original matrix followed by the covariance matrix to the matrix file.*

Example

```
MCONVERT MATRIX=OUT(COVMTX) /APPEND.
```

- MCONVERT reads matrix materials from the active dataset.
- The APPEND subcommand copies original matrix materials, appends each converted matrix to the end of the copy of its original matrix, and writes both sets to the file *COVMTX*.

MEANS

MEANS is available in the Statistics Base Edition.

```
MEANS [TABLES=]{varlist} [BY varlist] [BY...] [/varlist...]
               {ALL    }

 [/MISSING={TABLE     }]
           {INCLUDE   }
           {DEPENDENT}

 [/CELLS= [MEAN**   ] [COUNT**   ]  [STDDEV**]
          [MEDIAN] [GMEDIAN] [SEMEAN]  [SUM ]
          [MIN] [MAX] [RANGE] [VARIANCE]
          [KURT] [SEKURT] [SKEW] [SESKEW]
          [FIRST] [LAST]
          [NPCT] [SPCT] [NPCT(var)] [SPCT(var)]
          [HARMONIC] [GEOMETRIC]
          [DEFAULT]
          [ALL] ]

 [/STATISTICS=[ANOVA] [{LINEARITY}]  [NONE**]]
                       {ALL      }
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
MEANS TABLES=V1 TO V5 BY GROUP.
```

Overview

By default, MEANS (alias BREAKDOWN) displays means, standard deviations, and group counts for a numeric dependent variable and group counts for a string variable within groups defined by one or more control (independent) variables. Other procedures that display univariate statistics are SUMMARIZE, FREQUENCIES, and DESCRIPTIVES.

Options

Cell Contents. By default, MEANS displays means, standard deviations, and cell counts for a dependent variable across groups defined by one or more control variables. You can also display sums and variances using the CELLS subcommand.

Statistics. In addition to the statistics displayed for each cell of the table, you can obtain a one-way analysis of variance and test of linearity using the STATISTICS subcommand.

Basic Specification

The basic specification is TABLES with a table list. The actual keyword TABLES can be omitted.

- The minimum table list specifies a dependent variable.
- By default, MEANS displays means, standard deviations, and number of cases.

Subcommand Order

The table list must be first if the keyword TABLES is omitted. If the keyword TABLES is explicitly used, subcommands can be specified in any order.

Operations

- MEANS displays the number and percentage of the processed and missing cases in the Case Process Summary table.
- MEANS displays univariate statistics for the population as a whole and for each value of each successive control variable defined by the BY keyword on the TABLE subcommand in the Group Statistics table.

- ANOVA and linearity statistics, if requested, are displayed in the ANOVA and Measures of Association tables.
- If a string variable is specified as a dependent variable on any table lists, the MEANS procedure produces limited statistics (COUNT, FIRST, and LAST).

Limitations

- Each TABLES subcommand can contain a maximum of 10 BY variable lists.
- There is a maximum of 30 TABLES subcommands for each MEANS command.

Examples

Specifying a Range of Dependent Variables

```
MEANS TABLES=V1 TO V5 BY GROUP
   /STATISTICS=ANOVA.
```

- TABLES specifies that *V1* through *V5* are the dependent variables. *GROUP* is the control variable.
- Assuming that variables *V2, V3,* and *V4* lie between *V1* and *V5* in the active dataset, five tables are produced: *V1* by *GROUP*, *V2* by *GROUP*, *V3* by *GROUP*, and so on.
- STATISTICS requests one-way analysis-of-variance tables of *V1* through *V5* by *GROUP*.

Creating Analyses for Two Separate Sets of Dependent Variables

```
MEANS VARA BY VARB BY VARC/V1 V2 BY V3 V4 BY V5.
```

- This command contains two TABLES subcommands that omit the optional TABLES keyword.
- The first table list produces a Group Statistics table for *VARA* within groups defined by each combination of values as well as the totals of *VARB* and *VARC*.
- The second table list produces a Group Statistics table displaying statistics for *V1* by *V3* by *V5*, *V1* by *V4* by *V5*, *V2* by *V3* by *V5*, and *V2* by *V4* by *V5*.

TABLES Subcommand

TABLES specifies the table list.

- You can specify multiple TABLES subcommands on a single MEANS command (up to a maximum of 30). The slash between the subcommands is required. You can also name multiple table lists separated by slashes on one TABLES subcommand.
- The dependent variable is specified first. If the dependent variable is a string variable, MEANS produces only limited statistics (COUNT, FIRST, and LAST). The control (independent) variables follow the BY keyword and can be numeric (integer or noninteger) or string.
- Each use of the keyword BY in a table list adds a dimension to the table requested. Statistics are displayed for each dependent variable by each combination of values and the totals of the control variables across dimensions. There is a maximum of 10 BY variable lists for each TABLES subcommand.
- The order in which control variables are displayed is the same as the order in which they are specified on TABLES. The values of the first control variable defined for the table appear in the leftmost column of the table and change the most slowly in the definition of groups.
- More than one dependent variable can be specified in a table list, and more than one control variable can be specified in each dimension of a table list.

CELLS Subcommand

By default, MEANS displays the means, standard deviations, and cell counts in each cell. Use CELLS to modify cell information.

- If CELLS is specified without keywords, MEANS displays the default statistics.
- If any keywords are specified on CELLS, only the requested information is displayed.
- MEDIAN and GMEDIAN are expensive in terms of computer resources and time. Requesting these statistics (via these keywords or ALL) may slow down performance.

DEFAULT. *Means, standard deviations, and cell counts.* This is the default if CELLS is omitted.

MEAN. *Cell means.*

STDDEV. *Cell standard deviations.*

COUNT. *Cell counts.*

MEDIAN. *Cell median.*

GMEDIAN. *Grouped median.*

SEMEAN. *Standard error of cell mean.*

SUM. *Cell sums.*

MIN. *Cell minimum.*

MAx. *Cell maximum.*

RANGE. *Cell range.*

VARIANCE. *Variances.*

KURT. *Cell kurtosis.*

SEKURT. *Standard error of cell kurtosis.*

SKEW. *Cell skewness.*

SESKEW. *Standard error of cell skewness.*

FIRST. *First value.*

LAST. *Last value.*

NPCT. *Percentage of the total number of cases.*

SPCT. *Percentage of the total sum.*

NPCT(var). *Percentage of the total number of cases within the specified variable.* The specified variable must be one of the control variables.

SPCT(var). *Percentage of the total sum within the specified variable.* The specified variable must be one of the control variables.

HARMONIC. *Harmonic mean.*

GEOMETRIC. *Geometric mean.*

ALL. *All cell information.*

STATISTICS Subcommand

Use STATISTICS to request a one-way analysis of variance and a test of linearity for each TABLE list.

- Statistics requested on STATISTICS are computed in addition to the statistics displayed in the Group Statistics table.
- If STATISTICS is specified without keywords, MEANS computes ANOVA.
- If two or more dimensions are specified, the second and subsequent dimensions are ignored in the analysis-of-variance table. To obtain a two-way and higher analysis of variance, use the ANOVA or MANOVA procedure. The ONEWAY procedure calculates a one-way analysis of variance with multiple comparison tests.

ANOVA. *Analysis of variance.* ANOVA displays a standard analysis-of-variance table and calculates eta and eta squared (displayed in the Measures of Association table). This is the default if STATISTICS is specified without keywords.

LINEARITY . *Test of linearity.* LINEARITY (alias ALL) displays additional statistics to the tables created by the ANOVA keyword: the sums of squares, degrees of freedom, and mean square associated with linear and nonlinear components, the *F* ratio, and significance level for the ANOVA table and Pearson's *r* and r^2 for the Measures of Association table. LINEARITY is ignored if the control variable is a string.

NONE. *No additional statistics.* This is the default if STATISTICS is omitted.

Example

```
MEANS TABLES=INCOME BY SEX BY RACE
  /STATISTICS=ANOVA.
```

- MEANS produces a Group Statistics table of *INCOME* by *RACE* within *SEX* and computes an analysis of variance only for *INCOME* by *SEX*.

MISSING Subcommand

MISSING controls the treatment of missing values. If no MISSING subcommand is specified, each combination of a dependent variable and control variables is handled separately.

TABLE. *Delete cases with missing values on a tablewise basis.* A case with a missing value for any variable specified for a table is not used. Thus, every case contained in a table has a complete set of nonmissing values for all variables in that table. When you separate table requests with a slash, missing values are handled separately for each list. Any MISSING specification will result in tablewise treatment of missing values.

INCLUDE. *Include user-missing values.* This option treats user-missing values as valid values.

DEPENDENT. *Exclude user-missing values for dependent variables only.* DEPENDENT treats user-missing values for all control variables as valid.

References

Hays, W. L. 1981. *Statistics for the social sciences*, 3rd ed. New York: Holt, Rinehart, and Winston.

MISSING VALUES

```
MISSING VALUES {varlist}(value list) [[/]{varlist} ...]
               {ALL    }               {ALL    }
```

Keywords for numeric value lists:

`LO, LOWEST, HI, HIGHEST, THRU`

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
MISSING VALUES V1 (8,9) V2 V3 (0) V4 ('X') V5 TO V9 ('    ').
```

Overview

`MISSING VALUES` declares values user-missing. These values can then receive special treatment in data transformations, statistical calculations, and case selection. By default, user-missing values are treated the same as the system-missing values. System-missing values are automatically assigned by the program when no legal value can be produced, such as when an alphabetical character is encountered in the data for a numeric variable, or when an illegal calculation, such as division by 0, is requested in a data transformation.

Basic Specification

The basic specification is a single variable followed by the user-missing value or values in parentheses. Each specified value for the variable is treated as user-missing for any analysis.

Syntax Rules

- Each variable can have a maximum of three individual user-missing values. A space or comma must separate each value. For numeric variables, you can also specify a range of missing values. See the topic “Specifying Ranges of Missing Values ” on page 1130 for more information.
- The missing-value specification must correspond to the variable type (numeric or string).
- The same values can be declared missing for more than one variable by specifying a variable list followed by the values in parentheses. Variable lists must have either all numeric or all string variables.
- Different values can be declared missing for different variables by specifying separate values for each variable. An optional slash can be used to separate specifications.
- Missing values for string variables must be enclosed in single or double quotes. The value specifications must include any leading blanks. See the topic “String Values in Command Specifications” on page 35 for more information.
- For date format variables (for example, `DATE`, `ADATE`), missing values expressed in date formats must be enclosed in single or double quotes, and values must be expressed in the same date format as the defined date format for the variable.
- A variable list followed by an empty set of parentheses ( ) deletes any user-missing specifications for those variables.
- The keyword `ALL` can be used to refer to all user-defined variables in the active dataset, provided the variables are either all numeric or all string. `ALL` can refer to both numeric and string variables if it is followed by an empty set of parentheses. This will delete all user-missing specifications in the active dataset.
- More than one `MISSING VALUES` command can be specified per session.

Operations

- Unlike most transformations, MISSING VALUES takes effect as soon as it is encountered. Special attention should be paid to its position among commands. See the topic "Command Order" on page 36 for more information.
- Missing-value specifications can be changed between procedures. New specifications replace previous ones. If a variable is mentioned more than once on one or more MISSING VALUES commands before a procedure, only the last specification is used.
- Missing-value specifications are saved in IBM SPSS Statistics data files (see SAVE) and portable files (see EXPORT).

Limitations

Missing values for string variables cannot exceed 8 bytes. (There is no limit on the defined width of the string variable, but defined missing values cannot exceed 8 bytes.)

Examples

Declaring Missing Values for Multiple Variables

```
MISSING VALUES V1 (8,9) V2 V3 (0) V4 ('X') V5 TO V9 ('    ').
```

- The values 8 and 9 are declared missing for the numeric variable *V1*.
- The value 0 is declared missing for the numeric variables *V2* and *V3*.
- The value X is declared missing for the string variable *V4*.
- Blanks are declared missing for the string variables between and including *V5* and *V9*. All of these variables must have a width of four columns.

Clearing Missing Values for Selected Variables

```
MISSING VALUES V1 ().
```

- Any previously declared missing values for *V1* are deleted.

Declaring Missing Values for All Variables

```
MISSING VALUES ALL (9).
```

- The value 9 is declared missing for all variables in the active dataset; the variables must all be numeric. All previous user-missing specifications are overridden.

Clearing Missing Values for All Variables

```
MISSING VALUES ALL ().
```

- All previously declared user-missing values for all variables in the active dataset are deleted. The variables in the active dataset can be both numeric and string.

Specifying Ranges of Missing Values

A range of values can be specified as missing for numeric variables but *not* for string variables.

- The keyword THRU indicates an inclusive list of values. Values must be separated from THRU by at least one blank space.
- The keywords HIGHEST and LOWEST with THRU indicate the highest and lowest values of a variable. HIGHEST and LOWEST can be abbreviated to HI and LO.
- Only one THRU specification can be used for each variable or variable list. Each THRU specification can be combined with one additional missing value.

Example

```
MISSING VALUES  V1 (LOWEST THRU 0).
```

- All negative values and 0 are declared missing for the variable *V1*.

Example

```
MISSING VALUES  V1 (0 THRU 1.5).
```

- Values from 0 through and including 1.5 are declared missing.

Example

```
MISSING VALUES V1 (LO THRU 0, 999).
```

- All negative values, 0, and 999 are declared missing for the variable *V1*.

MIXED

MIXED is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
MIXED dependent varname [BY factor list] [WITH covariate list]

[/CRITERIA = [CIN({95** })] [HCONVERGE({0**  } {ABSOLUTE**})
             {value}              {value} {RELATIVE  }

             [DFMETHOD({SATTERTHWAITE**} {RESIDUAL} {KENWARDROGER})]

             [LCONVERGE({0**  } {ABSOLUTE**})] [MXITER({100**})]
                        {value} {RELATIVE  }           {n    }

             [MXSTEP({10**})] [PCONVERGE({1E-6**},{ABSOLUTE**})] [SCORING({1**})]
                     {n  }              {value } {RELATIVE  }            {n  }

             [SINGULAR({1E-12**})] ]
                       {value  }

[/EMMEANS = TABLES ({OVERALL          })]
                   {factor           }
                   {factor*factor ...}
            [WITH (covariate=value [covariate = value ...])
            [COMPARE [({factor})] [REFCAT({value})] [ADJ({LSD**     })] ]
                                          {FIRST}        {BONFERRONI}
                                          {LAST }        {SIDAK     }

[/FIXED = [effect [effect ...]] [| [NOINT] [SSTYPE({1  })] ] ]
                                                 {3**}

[/METHOD = {ML    }]
           {REML**}

[/MISSING = {EXCLUDE**}]
            {INCLUDE  }

[/PRINT = [CORB] [COVB] [CPS] [DESCRIPTIVES] [G] [HISTORY(1**)] [LMATRIX] [R]
                                                        (n  )
          [SOLUTION] [TESTCOV]]

[/RANDOM = effect [effect ...]
           [| [SUBJECT(varname[*varname[*...]])] [COVTYPE({VC**      })] [SOLUTION]]
                                                   {covstruct+}

[/REGWGT = varname]

[/REPEATED = varname[*varname[*...]] | SUBJECT(varname[*varname[*...]]) KRONECKER(varname[*varname[*...]])
             [COVTYPE({DIAG**    })]]
                      {covstruct†}
             [SPCOORDS(varname...)]

[/SAVE = [tempvar [(name)] [tempvar [(name)]] ...]

[/TEST[(valuelist)] =
    ['label']  effect valuelist ... [| effect valuelist ...] [divisor=value]]
    [; effect valuelist ... [| effect valuelist ...] [divisor=value]]

[/TEST[(valuelist)] = ['label'] ALL list [| list] [divisor=value]]
                      [; ALL list [| list] [divisor=value]]
```

** Default if the subcommand is omitted.

† covstruct can take the following values: AD1, AR1, ARH1, ARMA11, CS, CSH, CSR, DIAG, FA1, FAH1, HF, ID, TP, TPH, UN, UNR, VC.

For the REPEATED subcommand, covstruct can take the following additional values: SP_POWER, SP_EXPONENTIAL, SP_GAUSSIAN, SP_LINEAR, SP_LINEARLOG, SP_SPHERICAL, UN_AR1, UN_CS, UN_UN.

The KRONECKER keyword should be used only when COVTYPE is defined as UN_AR1, UN_CS, or UN_UN.

When both KRONECKER and REPEATED are in effect, they may or may not have common fields, but their values cannot be exactly the same (even when the values are not in the same order).

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 25

- SP_POWER, SP_EXPONENTIAL, SP_GAUSSIAN, SP_LINEAR, SP_LINEARLOG, and SP_SPHERICAL options added to the COVTYPE keyword on the REPEATED subcommand.
- SPCOORDS keyword introduced on the REPEATED subcommand.

- SOLUTION keyword introduced on the RANDOM subcommand.

Subscription (April 2018 update)

- DFMETHOD keyword introduced on the CRITERIA subcommand.
- KRONECKER keyword added to the REPEATED subcommand. The keyword should be used only when COVTYPE is one of three following Kronecker types.
- UN_AR1, UN_CS, and UN_UN options added to the COVTYPE keyword on the REPEATED subcommand.

Example

```
MIXED Y.
```

Overview

The MIXED procedure fits a variety of mixed linear models. The mixed linear model expands the general linear model used in the GLM procedure in that the data are permitted to exhibit correlation and non-constant variability. The mixed linear model, therefore, provides the flexibility of modeling not only the means of the data but also their variances and covariances.

The MIXED procedure is also a flexible tool for fitting other models that can be formulated as mixed linear models. Such models include multilevel models, hierarchical linear models, and random coefficient models.

Important changes to MIXED compared to previous versions

Specifying how degrees of freedom are computed for significance tests. Prior to version 26, SATTERTHWAITE method was the only method available for specifying how degrees of freedom are computed for significance tests. Starting with version 26, the RESIDUAL and KENWARDROGER methods are also available for specifying how degrees of freedom are computed for significance tests.

Independence of random effects. Prior to version 11.5, random effects were assumed to be independent. If you are using MIXED syntax jobs from a version prior to 11.5, be aware that the interpretation of the covariance structure may have changed.

Default covariance structures. Prior to version 11.5, the default covariance structure for random effects was ID, and the default covariance structure for repeated effects was VC.

Interpretation of VC covariance structure. Prior to version 11.5, the variance components (VC) structure was a diagonal matrix with heterogenous variances. Now, when the variance components structure is specified on a RANDOM subcommand, a scaled identity (ID) structure is assigned to each of the effects specified on the subcommand. If the variance components structure is specified on the REPEATED subcommand, it will be replaced by the diagonal (DIAG) structure. Note that the diagonal structure has the same interpretation as the variance components structure in versions prior to 11.5.

Basic features

Covariance structures. Various structures are available. Use multiple RANDOM subcommands to model a different covariance structure for each random effect.

Standard errors. Appropriate standard errors will be automatically calculated for all hypothesis tests on the fixed effects, and specified estimable linear combinations of fixed and random effects.

Subject blocking. Complete independence can be assumed across subject blocks.

Choice of estimation method. Two estimation methods for the covariance parameters are available.

Tuning the algorithm. You can control the values of algorithm-tuning parameters with the CRITERIA subcommand.

Optional output. You can request additional output through the PRINT subcommand. The SAVE subcommand allows you to save various casewise statistics back to the active dataset.

Basic specification

- The basic specification is a variable list identifying the dependent variable, the factors (if any) and the covariates (if any).
- By default, MIXED adopts the model that consists of the intercept term as the only fixed effect and the residual term as the only random effect.

Subcommand order

- The variable list must be specified first.
- Subcommands can be specified in any order.

Syntax rules

- For many analyses, the MIXED variable list, the FIXED subcommand, and the RANDOM subcommand are the only specifications needed.
- A dependent variable must be specified.
- Empty subcommands are silently ignored.
- Multiple RANDOM subcommands are allowed. However, if an effect with the same subject specification appears in multiple RANDOM subcommands, only the last specification will be used.
- Multiple TEST subcommands are allowed.
- All subcommands, except the RANDOM and the TEST subcommands, should be specified only once. If a subcommand is repeated, only the last specification will be used.
- The following words are reserved as keywords in the MIXED procedure: BY, WITH, and WITHIN.

Examples

The following are examples of models that can be specified using MIXED:

Model 1: Fixed-Effects ANOVA Model

Suppose that *TREAT* is the treatment factor and *BLOCK* is the blocking factor.

```
MIXED Y BY TREAT BLOCK
  /FIXED = TREAT BLOCK.
```

Model 2: Randomized Complete Blocks Design

Suppose that *TREAT* is the treatment factor and *BLOCK* is the blocking factor.

```
MIXED Y BY TREAT BLOCK
  /FIXED = TREAT
  /RANDOM = BLOCK.
```

Model 3: Split-Plot Design

An experiment consists of two factors, *A* and *B*. The experiment unit with respect to *A* is *C*. The experiment unit with respect to *B* is the individual subject, a subdivision of the factor *C*. Thus, *C* is the whole-plot unit, and the individual subject is the split-plot unit.

```
MIXED Y BY A B C
  /FIXED = A B A*B
  /RANDOM = C(A).
```

Model 4: Purely Random-Effects Model

Suppose that *A*, *B*, and *C* are random factors.

```
MIXED Y BY A B C
  /FIXED = | NOINT
  /RANDOM  = INTERCEPT A B C A*B A*C B*C | COVTYPE(CS).
```

The MIXED procedure allows effects specified on the same RANDOM subcommand to be correlated. Thus, in the model above, the parameters of a compound symmetry covariance matrix are computed across all levels of the random effects. In order to specify independent random effects, you need to specify separate RANDOM subcommands. For example:

```
MIXED Y BY A B C
  /FIXED = | NOINT
  /RANDOM  = INTERCEPT | COVTYPE(ID)
  /RANDOM  = A | COVTYPE(CS)
  /RANDOM  = B | COVTYPE(CS)
  /RANDOM  = C | COVTYPE(CS)
  /RANDOM  = A*B | COVTYPE(CS)
  /RANDOM  = A*C | COVTYPE(CS)
  /RANDOM  = B*C | COVTYPE(CS).
```

Here, the parameters of compound symmetry matrices are computed separately for each random effect.

Model 6: Multilevel Analysis

Suppose that *SCORE* is the score of a particular achievement test given over *TIME*. *STUDENT* is nested within *CLASS*, and *CLASS* is nested within *SCHOOL*.

```
MIXED SCORE WITH TIME
  /FIXED = TIME
  /RANDOM = INTERCEPT TIME | SUBJECT(SCHOOL) COVTYPE(ID)
  /RANDOM = INTERCEPT TIME | SUBJECT(SCHOOL*CLASS) COVTYPE(ID)
  /RANDOM = INTERCEPT TIME | SUBJECT(SCHOOL*CLASS*STUDENT) COVTYPE(ID).
```

Model 7: Unconditional Linear Growth Model

Suppose that *SUBJ* is the individual's identification and *Y* is the response of an individual observed over *TIME*. The covariance structure is unspecified.

```
MIXED Y WITH TIME
  /FIXED = TIME
  /RANDOM = INTERCEPT TIME | SUBJECT(SUBJ) COVTYPE(ID).
```

Model 8: Linear Growth Model with a Person-Level Covariate

Suppose that *PCOVAR* is the person-level covariate.

```
MIXED Y WITH TIME PCOVAR
  /FIXED = TIME PCOVAR TIME*PCOVAR
  /RANDOM = INTERCEPT TIME | SUBJECT(SUBJ) COVTYPE(ID).
```

Model 9: Repeated Measures Analysis

Suppose that *SUBJ* is the individual's identification and *Y* is the response of an individual observed over several *STAGE*s. The covariance structure is compound symmetry.

```
MIXED Y BY STAGE
  /FIXED = STAGE
  /REPEATED = STAGE | SUBJECT(SUBJ) COVTYPE(CS).
```

Model 10: Repeated Measures Analysis with Time-Dependent Covariate

Suppose that *SUBJ* is the individual's identification and *Y* is the response of an individual observed over several *STAGE*s. *X* is an individual-level covariate that also measures over several *STAGE*s. The residual covariance matrix structure is *AR(1)*.

```
MIXED Y BY STAGE WITH X
  /FIXED = X STAGE
  /REPEATED = STAGE | SUBJECT(SUBJ) COVTYPE(AR1).
```

Case Frequency

- If a WEIGHT variable is specified, its values are used as frequency weights by the MIXED procedure.
- Cases with missing weights or weights less than 0.5 are not used in the analyses.
- The weight values are rounded to the nearest whole numbers before use. For example, 0.5 is rounded to 1, and 2.4 is rounded to 2.

Covariance Structure List

The following is the list of covariance structures being offered by the MIXED procedure. Unless otherwise implied or stated, the structures are not constrained to be non-negative definite in order to avoid nonlinear constraints and to reduce the optimization complexity. However, the variances are restricted to be non-negative.

- Separate covariance matrices are computed for each random effect; that is, while levels of a given random effect are allowed to co-vary, they are considered independent of the levels of other random effects.

AD1. *First-order ante-dependence.*

AR1. *First-order autoregressive.*

ARH1. *Heterogenous first-order autoregressive.*

ARMA11. *Autoregressive moving average (1,1).*

CS. *Compound symmetry.* This structure has constant variance and constant covariance.

CSH. *Heterogenous compound symmetry.* This structure has non-constant variance and constant correlation.

CSR. *Compound symmetry with correlation parameterization.* This structure has constant variance and constant covariance.

DIAG. *Diagonal.* This is a diagonal structure with heterogenous variance. This is the default covariance structure for repeated effects.

FA1. *First-order factor analytic with constant diagonal offset* ($d \geq 0$).

FAH1. *First-order factor analytic with heterogenous diagonal offsets* ($d_k \geq 0$).

HF. *Huynh-Feldt.*

ID. *Identity.* This is a scaled identity matrix.

TP. *Toeplitz*

TPH. *Heterogenous Toeplitz*

SP_POWER. *Spatial power structure.* Adapts the first-order autoregressive structure to a time-decaying correlation of unequal-spaced and repeated measurements. This structure is only supported on the REPEATED subcommand.

SP_EXPONENTIAL. *Spatial exponential structure.* Assumes that time-decaying correlations decrease exponentially with increasing spatial distances between repeated measurements. This structure is only supported on the REPEATED subcommand.

SP_GAUSSIAN. *Spatial Gaussian structure.* Assumes that time-decaying correlations decrease more rapidly, with increasing spatial distances between repeated measurements, than for the exponential structure. This structure is only supported on the REPEATED subcommand.

SP_LINEAR. *Spatial linear structure.* Assumes that time-decaying correlations decrease linearly with increasing spatial distances between repeated measurements. This structure is only supported on the REPEATED subcommand.

SP_LINEARLOG. *Spatial linear-log structure.* Assumes that time-decaying correlations decrease linearly with increasing logarithmic spatial distances between repeated measurements. This structure is only supported on the REPEATED subcommand.

SP_SPHERICAL. *Spatial spherical structure.* Allows cubic terms of both the correlation function and the spatial distances between repeated measurements. This structure is only supported on the REPEATED subcommand.

UN_AR1. *UN@AR1.* Specifies the Kronecker product of one unstructured matrix and the other first-order auto-regression covariance matrix. The first unstructured matrix models the multivariate observation, and

the second first-order auto-regression covariance structure models the data covariance across time or another factor.

UN_CS. *UN@CS.* Specifies the Kronecker product of one unstructured matrix and the other compound-symmetry covariance matrix with constant variance and covariance. The first unstructured matrix models the multivariate observation, and the second compound symmetry covariance structure models the data covariance across time or another factor.

UN_UN. *UN@UN.* Specifies the Kronecker product of two unstructured matrices, with the first one modeling the multivariate observation, and second one modeling the data covariance across time or another factor.

UN. *Unstructured.* This is a completely general covariance matrix.

UNR. *Unstructured correlations*

VC. *Variance components.* This is the default covariance structure for random effects. When the variance components structure is specified on a RANDOM subcommand, a scaled identity (ID) structure is assigned to each of the effects specified on the subcommand. If the variance components structure is specified on the REPEATED subcommand, it is replaced by the diagonal (DIAG) structure.

Variable List

The variable list specifies the dependent variable, the factors, and the covariates in the model.

- The dependent variable must be the first specification on MIXED.
- The names of the factors, if any, must be preceded by the keyword BY.
- The names of the covariates, if any, must be preceded by the keyword WITH.
- The dependent variable and the covariates must be numeric.
- The factor variables can be of any type (numeric and string).
- Only cases with no missing values in all of the variables specified will be used.

CRITERIA Subcommand

The CRITERIA subcommand controls the iterative algorithm used in the estimation and specifies numerical tolerance for checking singularity.

CIN(value). *Confidence interval level.* This value is used whenever a confidence interval is constructed. Specify a value greater than or equal to 0 and less than 100. The default value is 95.

HCONVERGE(value, type). *Hessian convergence criterion.* Convergence is assumed if $g'_k H_k^{-1} g_k$ is less than a multiplier of *value*. The multiplier is 1 for ABSOLUTE type and is the absolute value of the current log-likelihood function for RELATIVE type. The criterion is not used if *value* equals 0. This criterion is not used by default. Specify a non-negative value and a measure type of convergence.

DFMETHOD(type). *Specify how degrees of freedom are computed for significance tests.* This keyword can specify the options RESIDUAL, SATTERTHWAITE (default), and KENWARDROGER.

SATTERTHWAITE
The Satterthwaite method has a field degrees of freedom across tests. It is useful if your sample size is small, or the data are unbalanced, or the model uses a complicated covariance type; for example, unstructured. This is the default.

RESIDUAL
The residual method has a fixed degrees of freedom for all tests. It is useful if your sample size is sufficiently large, or the data are balanced, or the model uses a simpler covariance type; for example, scaled identity or diagonal.

KENWARDROGER

The Kenward-Roger method offers a more precise small-sample estimator for the variance-covariance of the fixed effects parameters and the approximate denominator degrees of freedom in t-tests and F-tests. The method introduces a scale factor for the F-statistic and estimates it and the denominator degrees of freedom by using a Taylor series expansion for the estimated random structure within the data. The value of the estimated denominator degrees of freedom is the Kenward-Roger degrees of freedom in GLMM. The corresponding derived F- and t-statistic are used in EMMEANS and Customs Tests for statistical inference.

Note: The Kenward-Roger method is used in the model based covariance (instead of robust covariance). When both the Kenward-Roger method and robust covariance is selected, the Kenward-Roger method is applied to model based covariance, and the following warning is presented: “Since Kenward-Roger method is selected, the robust covariance method is changed to model-based covariance method”.

LCONVERGE(value, type). *Log-likelihood function convergence criterion.* Convergence is assumed if the ABSOLUTE or RELATIVE change in the log-likelihood function is less than *value*. The criterion is not used if *a* equals 0. This criterion is not used by default. Specify a non-negative value and a measure type of convergence.

MXITER(n). *Maximum number of iterations.* Specify a non-negative integer. The default value is 100.

PCONVERGE(value, type). *Parameter estimates convergence criterion.* Convergence is assumed if the maximum ABSOLUTE or maximum RELATIVE change in the parameter estimates is less than *value*. The criterion is not used if *a* equals 0. Specify a non-negative value and a measure type of convergence. The default value for *a* is 10^{-6}.

MXSTEP(n). *Maximum step-halving allowed.* At each iteration, the step size is reduced by a factor of 0.5 until the log-likelihood increases or maximum step-halving is reached. Specify a positive integer. The default value is 10.

Note: When a spatial or Kronecker product covariance structure is specified on the **REPEATED** subcommand, the **SOLUTION** keyword is specified on a **RANDOM** subcommand, or **KENWARDROGER** or **RESIDUAL** is specified for the **DFMETHOD** on the **CRITERIA** subcommand, an alternative method using cubic interpolation is used to determine step sizes during iterations, and the maximum number of cubic interpolation steps is fixed at 20. The **MXSTEP** specification may, or may not, be used in these situations, but the number of cubic interpolations displays in the Iterations History table.

SCORING(n). *Apply scoring algorithm.* Requests to use the Fisher scoring algorithm up to iteration number *n*. Specify a non-negative integer. The default is 1.

SINGULAR(value). *Value used as tolerance in checking singularity.* Specify a positive value. The default value is 10^{-12}.

Example

```
MIXED SCORE BY SCHOOL CLASS WITH AGE
/CRITERIA = CIN(90) LCONVERGE(0) MXITER(50) PCONVERGE(1E-5 RELATIVE)
/FIXED = AGE
/RANDOM = SCHOOL CLASS.
```

- The CRITERIA subcommand requests that a 90% confidence interval be calculated whenever appropriate.
- The log-likelihood convergence criterion is not used. Convergence is attained when the maximum relative change in parameter estimates is less than 0.00001 and number of iterations is less than 50.

Example

```
MIXED SCORE BY SCHOOL CLASS WITH AGE
  /CRITERIA = MXITER(100) SCORING(100)
  /FIXED = AGE
  /RANDOM = SCHOOL CLASS.
```

- The Fisher scoring algorithm is used for all iterations.

EMMEANS Subcommand

EMMEANS displays estimated marginal means of the dependent variable in the cells and their standard errors for the specified factors. Note that these are predicted, not observed, means.

- The TABLES keyword, followed by an option in parentheses, is required. COMPARE is optional; if specified, it must follow TABLES.
- Multiple EMMEANS subcommands are allowed. Each is treated independently.
- If identical EMMEANS subcommands are specified, only the last identical subcommand is in effect. EMMEANS subcommands that are redundant but not identical (for example, crossed factor combinations such as *A*B* and *B*A*) are all processed.

TABLES(option). *Table specification.* Valid options are the keyword OVERALL, factors appearing on the factor list, and crossed factors constructed of factors on the factor list. Crossed factors can be specified by using an asterisk (*) or the keyword BY. All factors in a crossed factor specification must be unique.

- If OVERALL is specified, the estimated marginal means of the dependent variable are displayed, collapsing over all factors.
- If a factor, or a crossing factor, is specified on the TABLES keyword, MIXED will compute the estimated marginal mean for each level combination of the specified factor(s), collapsing over all other factors not specified with TABLES.

WITH (option).

Covariate values. Valid options are covariates appearing on the covariate list on the VARIABLES subcommand. Each covariate must be followed by a numeric value or the keyword MEAN.

- If a numeric value is used, the estimated marginal mean will be computed by holding the specified covariate at the supplied value.
- When the keyword MEAN is used, the estimated marginal mean will be computed by holding the covariate at its overall mean. If a covariate is not specified in the WITH option, its overall mean will be used in estimated marginal mean calculations.

COMPARE(factor) REFCAT(value) ADJ(method). *Main- or simple-main-effects omnibus tests and pairwise comparisons of the dependent variable.* This option gives the mean difference, standard error, degrees of freedom, significance, and confidence intervals for each pair of levels for the effect specified in the COMPARE keyword, and an omnibus test for that effect. If only one factor is specified on TABLES, COMPARE can be specified by itself; otherwise, the factor specification is required. In this case, levels of the specified factor are compared with each other for each level of the other factors in the interaction.

- The optional ADJ keyword allows you to apply an adjustment to the confidence intervals and significance values to account for multiple comparisons. Methods available are LSD (no adjustment), BONFERRONI, or SIDAK.
- By default, all pairwise comparisons of the specified factor will be constructed. Optionally, comparisons can be made to a reference category by specifying the value of that category after the REFCAT keyword. If the compare factor is a string variable, the category value must be a quoted string. If the compare factor is a numeric variable, the category value should be specified as an unquoted numeric value. Alternatively, the keywords FIRST or LAST can be used to specify whether the first or the last category will be used as a reference category.

Example

```
MIXED Y BY A B WITH X
  /FIXED A B X
  /EMMEANS TABLES(A*B) WITH(X=0.23) COMPARE(A) ADJ(SIDAK)
  /EMMEANS TABLES(A*B) WITH(X=MEAN) COMPARE(A) REFCAT(LAST) ADJ(LSD).
```

- In the example, the first EMMEANS subcommand will compute estimated marginal means for all level combinations of *A*B* by fixing the covariate *X* at 0.23. Then for each level of *B*, all pairwise comparisons on *A* will be performed using SIDAK adjustment.

- In the second EMMEANS subcommand, the estimated marginal means will be computed by fixing the covariate *X* at its mean. Since REFCAT(LAST) is specified, comparison will be made to the last category of factor *A* using LSD adjustment.

FIXED Subcommand

The FIXED subcommand specifies the fixed effects in the mixed model.

- Specify a list of terms to be included in the model, separated by commas or spaces.
- The intercept term is included by default.
- The default model is generated if the FIXED subcommand is omitted or empty. The default model consists of only the intercept term (if included).
- To explicitly include the intercept term, specify the keyword INTERCEPT on the FIXED subcommand. The INTERCEPT term must be specified first on the FIXED subcommand.
- To include a main-effect term, enter the name of the factor on the FIXED subcommand.
- To include an interaction-effect term among factors, use the keyword BY or the asterisk (*) to connect factors involved in the interaction. For example, A*B*C means a three-way interaction effect of the factors *A, B,* and *C*. The expression A BY B BY C is equivalent to A*B*C. Factors inside an interaction effect must be distinct. Expressions such as A*C*A and A*A are invalid.
- To include a nested-effect term, use the keyword WITHIN or a pair of parentheses on the FIXED subcommand. For example, A(B) means that *A* is nested within *B*, where *A* and *B* are factors. The expression A WITHIN B is equivalent to A(B). Factors inside a nested effect must be distinct. Expressions such as A(A) and A(B*A) are invalid.
- Multiple-level nesting is supported. For example, A(B(C)) means that *B* is nested within *C*, and *A* is nested within *B(C)*. When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, A(B)(C) is invalid.
- Nesting within an interaction effect is valid. For example, A(B*C) means that *A* is nested within *B*C*.
- Interactions among nested effects are allowed. The correct syntax is the interaction followed by the common nested effect inside the parentheses. For example, the interaction between *A* and *B* within levels of *C* should be specified as A*B(C) instead of A(C)*B(C).
- To include a covariate term in the model, enter the name of the covariate on the FIXED subcommand.
- Covariates can be connected using the keyword BY or the asterisk (*). For example, X*X is the product of *X* and itself. This is equivalent to entering a covariate whose values are the squared values of *X*.
- Factor and covariate effects can be connected in many ways. Suppose that *A* and *B* are factors and *X* and *Y* are covariates. Examples of valid combinations of factor and covariate effects are A*X, A*B*X, X(A), X(A*B), X*A(B), X*Y(A*B), and A*B*X*Y.
- No effects can be nested within a covariate effect. Suppose that *A* and *B* are factors and *X* and *Y* are covariates. The effects A(X), A(B*Y), X(Y), and X(B*Y) are invalid.
- The following options, which are specific for the fixed effects, can be entered after the effects. Use the vertical bar (|) to precede the options.

NOINT. *No intercept.* The intercept terms are excluded from the fixed effects.

SSTYPE(n). *Type of sum of squares.* Specify the methods for partitioning the sums of squares. Specify *n* = 1 for Type I sum of squares or *n* = 3 for Type III sum of squares. The default is Type III sum of squares.

Example

```
MIXED  SCORE BY SCHOOL CLASS WITH AGE PRETEST
  /FIXED = AGE(SCHOOL) AGE*PRETEST(SCHOOL)
  /RANDOM = CLASS.
```

- In this example, the fixed-effects design consists of the default INTERCEPT, a nested effect *AGE* within *SCHOOL*, and another nested effect of the product of *AGE* and *PRETEST* within *SCHOOL*.

Example

```
MIXED SCORE BY SCHOOL CLASS
  /FIXED = | NOINT
  /RANDOM = SCHOOL CLASS.
```

- In this example, a purely random-effects model is fitted. The random effects are *SCHOOL* and *CLASS*. The fixed-effects design is empty because the implicit intercept term is removed by the NOINT keyword.
- You can explicitly insert the INTERCEPT effect as /FIXED = INTERCEPT | NOINT. But the specification will be identical to /FIXED = | NOINT.

METHOD Subcommand

The METHOD subcommand specifies the estimation method.

- If this subcommand is not specified, the default is REML.
- The keywords ML and REML are mutually exclusive. Only one of them can be specified once.

ML. *Maximum likelihood.*

REML. *Restricted maximum likelihood.* This is the default.

MISSING Subcommand

The MISSING subcommand specifies the way to handle cases with user-missing values.

- If this subcommand is not specified, the default is EXCLUDE.
- Cases, which contain system-missing values in one of the variables, are always deleted.
- The keywords EXCLUDE and INCLUDE are mutually exclusive. Only one of them can be specified at once.

EXCLUDE. *Exclude both user-missing and system-missing values.* This is the default.

INCLUDE. *User-missing values are treated as valid.* System-missing values cannot be included in the analysis.

PRINT Subcommand

The PRINT subcommand specifies additional output. If no PRINT subcommand is specified, the default output includes:

- A model dimension summary table
- A covariance parameter estimates table
- A model fit summary table
- A test of fixed effects table

CORB. *Asymptotic correlation matrix of the fixed-effects parameter estimates.*

COVB. *Asymptotic covariance matrix of the fixed-effects parameter estimates.*

CPS. *Case processing summary.* Displays the sorted values of the factors, the repeated measure variables, the repeated measure subjects, the random-effects subjects, and their frequencies.

DESCRIPTIVES. *Descriptive statistics.* Displays the sample sizes, the means, and the standard deviations of the dependent variable, and covariates (if specified). These statistics are displayed for each distinct combination of the factors.

G. *Estimated covariance matrix of random effects.* This keyword is accepted only when at least one RANDOM subcommand is specified. Otherwise, it will be ignored. If a SUBJECT variable is specified for a random effect, then the common block is displayed.

HISTORY(n). *Iteration history.* The table contains the log-likelihood function value and parameter estimates for every *n* iterations beginning with the 0*th* iteration (the initial estimates). The default is to print every iteration (*n* = 1). If HISTORY is specified, the last iteration is always printed regardless of the value of *n*.

LMATRIX. *Estimable functions.* Displays the estimable functions used for testing the fixed effects and for testing the custom hypothesis.

R. *Estimated covariance matrix of residual.* This keyword is accepted only when a REPEATED subcommand is specified. Otherwise, it will be ignored. If a SUBJECT variable is specified, the common block is displayed.

SOLUTION. *A solution for the fixed-effects parameters.* The fixed-effects parameter estimates are displayed. Their approximate standard errors are also displayed.

TESTCOV. *Tests for the covariance parameters.* Displays the asymptotic standard errors and Wald tests for the covariance parameters.

RANDOM Subcommand

The RANDOM subcommand specifies the random effects in the mixed model.

- Depending on the covariance type specified, random effects specified in one RANDOM subcommand may be correlated.
- One covariance G matrix will be constructed for each RANDOM subcommand. The dimension of the random effect covariance G matrix is equal to the sum of the levels of all random effects in the subcommand.
- When the variance components (VC) structure is specified, a scaled identity (ID) structure will be assigned to each of the effects specified. This is the default covariance type for the RANDOM subcommand.
- Note that the RANDOM subcommand in the MIXED procedure is different in syntax from the RANDOM subcommand in the GLM and VARCOMP procedures.
- Use a separate RANDOM subcommand when a different covariance structure is assumed for a list of random effects. If the same effect is listed on more than one RANDOM subcommand, it must be associated with a different SUBJECT combination.
- Specify a list of terms to be included in the model, separated by commas or spaces.
- No random effects are included in the mixed model unless a RANDOM subcommand is specified correctly.
- Specify the keyword INTERCEPT to include the intercept as a random effect. The MIXED procedure does not include the intercept in the RANDOM subcommand by default. The INTERCEPT term must be specified first on the RANDOM subcommand.
- To include a main-effect term, enter the name of the factor on the RANDOM subcommand.
- To include an interaction-effect term among factors, use the keyword BY or the asterisk (*) to join factors involved in the interaction. For example, A*B*C means a three-way interaction effect of *A*, *B*, and *C*, where *A*, *B*, and *C* are factors. The expression A BY B BY C is equivalent to A*B*C. Factors inside an interaction effect must be distinct. Expressions such as A*C*A and A*A are invalid.
- To include a nested-effect term, use the keyword WITHIN or a pair of parentheses on the RANDOM subcommand. For example, A(B) means that *A* is nested within *B*, where *A* and *B* are factors. The expression A WITHIN B is equivalent to A(B). Factors inside a nested effect must be distinct. Expressions such as A(A) and A(B*A) are invalid.

- Multiple-level nesting is supported. For example, A(B(C)) means that *B* is nested within *C*, and *A* is nested within *B(C)*. When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, A(B)(C) is invalid.
- Nesting within an interaction effect is valid. For example, A(B*C) means that *A* is nested within *B*C*.
- Interactions among nested effects are allowed. The correct syntax is the interaction followed by the common nested effect inside the parentheses. For example, the interaction between *A* and *B* within levels of *C* should be specified as A*B(C) instead of A(C)*B(C).
- To include a covariate term in the model, enter the name of the covariate on the FIXED subcommand.
- Covariates can be connected using the keyword BY or the asterisk (*). For example, *X*X* is the product of *X* and itself. This is equivalent to entering a covariate whose values are the squared values of *X*.
- Factor and covariate effects can be connected in many ways. Suppose that *A* and *B* are factors and *X* and *Y* are covariates. Examples of valid combinations of factor and covariate effects are A*X, A*B*X, X(A), X(A*B), X*A(B), X*Y(A*B), and A*B*X*Y.
- No effects can be nested within a covariate effect. Suppose that *A* and *B* are factors and *X* and *Y* are covariates. The effects A(X), A(B*Y), X(Y), and X(B*Y) are invalid.
- The following options, which are specific for the random effects, can be entered after the effects. Use the vertical bar (|) to precede the options.

  SUBJECT(varname*varname*...). *Identify the subjects.* Complete independence is assumed across subjects, thus producing a block-diagonal structure in the covariance matrix of the random effect with identical blocks. Specify a list of variable names (of any type) connected by asterisks. The number of subjects is equal to the number of distinct combinations of values of the variables. A case will not be used if it contains a missing value on any of the subject variables.

  COVTYPE(type). *Covariance structure.* Specify the covariance structure of the identical blocks for the random effects (see “Covariance Structure List ” on page 1137). The default covariance structure for random effects is VC.

  SOLUTION. Specifies to display the random-effects parameter estimates.
- If the REPEATED subcommand is specified, the variables in the RANDOM subject list must be a subset of the variables in the REPEATED subject list.
- Random effects are considered independent of each other, and a separate covariance matrix is computed for each effect.

Example

```
MIXED  SCORE BY SCHOOL CLASS
  /RANDOM = INTERCEPT SCHOOL CLASS.
```

REGWGT Subcommand

The REGWGT subcommand specifies the name of a variable containing the regression weights.

- Specify a numeric variable name following the REGWGT subcommand.
- Cases with missing or non-positive weights are not used in the analyses.
- The regression weights will be applied only to the covariance matrix of the residual term.

REPEATED Subcommand

The REPEATED subcommand specifies the residual covariance matrix in the mixed-effects model. If no REPEATED subcommand is specified, the residual covariance matrix assumes the form of a scaled identity matrix with the scale being the usual residual variance.

- Specify a list of variable names (of any type) connected by asterisks (repeated measure) following the REPEATED subcommand.

- Distinct combinations of values of the variables are used simply to identify the repeated observations. Order of the values will determine the order of occurrence of the repeated observations. Therefore, the lowest values of the variables associate with the first repeated observation, and the highest values associate with the last repeated observation.
- The VC covariance structure is obsolete in the REPEATED subcommand. If it is specified, it will be replaced with the DIAG covariance structure. An annotation will be made in the output to indicate this change.
- The default covariance type for repeated effects is DIAG.
- The following keywords, which are specific for the REPEATED subcommand, can be entered after the effects. Use the vertical bar (|) to precede the options.

  SUBJECT(varname*varname*...)
  Identify the subjects. Complete independence is assumed across subjects, thus producing a block-diagonal structure in the residual covariance matrix with identical blocks. The number of subjects is equal to the number of distinct combinations of values of the variables. A case will not be used if it contains a missing value on any of the subject variables.

  KRONECKER(varname*varname*...)
  The Kronecker product of two matrices can handle doubly repeated-measure data in which there are two repeated effects; one of them indicating the multivariate observation of target, and the other the time or the order of the data measured. The KRONECKER keyword should be used only when COVTYPE is defined as UN_AR1, UN_CS, or UN_UN.

  When both KRONECKER and REPEATED are in effect, they may or may not have common fields, but their values cannot be exactly the same (even when the values are not in the same order).

  COVTYPE(type)
  Covariance structure. Specify the covariance structure of the identical blocks for the residual covariance matrix (see “Covariance Structure List ” on page 1137). The default structure for repeated effects is DIAG.

  SPCOORDS(varname...)
  Specifies the variables that define the spatial covariance coordinates. One or more numeric variables must be specified. SPCOORDS is required when SP_POWER, SP_EXPONENTIAL, SP_GAUSSIAN, SP_LINEAR, SP_LINEARLOG, or SP_SPHERICAL is specified for the COVTYPE keyword. It is ignored otherwise.
- The SUBJECT keyword must be specified to identify the subjects in a repeated measurement analysis. The analysis will not be performed if this keyword is omitted.
- The list of subject variables must contain all of the subject variables specified in all RANDOM subcommands.
- Any variable used in the repeated measure list must not be used in the repeated subject specification.

Example

```
MIXED  SCORE BY CLASS
  /RANDOM = CLASS | SUBJECT(SCHOOL)
  /REPEATED = FLOOR | SUBJECT(SCHOOL*STUDENT).
```

However, the syntax in each of the following examples is invalid:

```
MIXED  SCORE BY CLASS
  /RANDOM = CLASS | SUBJECT(SCHOOL)
  /REPEATED = FLOOR | SUBJECT(STUDENT).


MIXED  SCORE BY CLASS
  /RANDOM = CLASS | SUBJECT(SCHOOL*STUDENT)
  /REPEATED = FLOOR | SUBJECT(STUDENT).


MIXED  SCORE BY CLASS
  /RANDOM = CLASS | SUBJECT(SCHOOL)
  /REPEATED = STUDENT | SUBJECT(STUDENT*SCHOOL).
```

- In the first two examples, the RANDOM subject list contains a variable not on the REPEATED subject list.
- In the third example, the REPEATED subject list contains a variable on the REPEATED variable list.

SAVE Subcommand

Use the SAVE subcommand to save one or more casewise statistics to the active dataset.

- Specify one or more temporary variables, each followed by an optional new name in parentheses.
- If new names are not specified, default names are generated.

FIXPRED. *Fixed predicted values.* The regression means without the random effects.

PRED. *Predicted values.* The model fitted value.

RESID. *Residuals.* The data value minus the predicted value.

SEFIXP. *Standard error of fixed predicted values.* These are the standard error estimates for the fixed effects predicted values obtained by the keyword FIXPRED.

SEPRED. *Standard error of predicted values.* These are the standard error estimates for the overall predicted values obtained by the keyword PRED.

DFFIXP. *Degrees of freedom of fixed predicted values.* These are the Satterthwaite degrees of freedom for the fixed effects predicted values obtained by the keyword FIXPRED.

DFPRED. *Degrees of freedom of predicted values.* These are the Satterthwaite degrees of freedom for the fixed effects predicted values obtained by the keyword PRED.

Example

```
MIXED  SCORE BY SCHOOL CLASS WITH AGE
  /FIXED = AGE
  /RANDOM = SCHOOL CLASS(SCHOOL)
  /SAVE = FIXPRED(BLUE) PRED(BLUP) SEFIXP(SEBLUE) SEPRED(SEBLUP).
```

- The SAVE subcommand appends four variables to the active dataset: *BLUE*, containing the fixed predicted values, *BLUP*, containing the predicted values, *SEBLUE*, containing the standard error of *BLUE*, and *SEBLUP*, containing the standard error of *BLUP*.

TEST Subcommand

The TEST subcommand allows you to customize your hypotheses tests by directly specifying null hypotheses as linear combinations of parameters.

- Multiple TEST subcommands are allowed. Each is handled independently.
- The basic format for the TEST subcommand is an optional list of values enclosed in a pair of parentheses, an optional label in quotes, an effect name or the keyword ALL, and a list of values.
- When multiple linear combinations are specified within the same TEST subcommand, a semicolon (;) terminates each linear combination except the last one.
- At the end of a contrast coefficients row, you can use the option DIVISOR=value to specify a denominator for coefficients in that row. When specified, the contrast coefficients in that row will be divided by the given value. Note that the equals sign is required.
- The value list preceding the first effect or the keyword ALL contains the constants, to which the linear combinations are equated under the null hypotheses. If this value list is omitted, the constants are assumed to be zeros.
- The optional label is a string with a maximum length of 255 bytes. Only one label per TEST subcommand can be specified.
- The effect list is divided into two parts. The first part is for the fixed effects, and the second part is for the random effects. Both parts have the same syntax structure.
- Effects specified in the fixed-effect list should have already been specified or implied on the FIXED subcommand.

- Effects specified in the random-effect list should have already been specified on the RANDOM subcommand.
- To specify the coefficient for the intercept, use the keyword INTERCEPT. Only one value is expected to follow INTERCEPT.
- The number of values following an effect name must be equal to the number of parameters (including the redundant ones) corresponding to that effect. For example, if the effect *A*B* takes up to six parameters, then exactly six values must follow *A*B*.
- A number can be specified as a fraction with a positive denominator. For example, 1/3 or –1/3 are valid, but 1/–3 is invalid.
- When ALL is specified, only a list of values can follow. The number of values must be equal to the number of parameters (including the redundant ones) in the model.
- Effects appearing or implied on the FIXED and RANDOM subcommands but not specified on TEST are assumed to take the value 0 for all of their parameters.
- If ALL is specified for the first row in a TEST matrix, then all subsequent rows should begin with the ALL keyword.
- If effects are specified for the first row in a TEST matrix, then all subsequent rows should use the effect name (thus ALL is not allowed).
- When SUBJECT( ) is specified on a RANDOM subcommand, the coefficients given in the TEST subcommand will be divided by the number of subjects of that random effect automatically.

Example

```
MIXED Y BY A B C
  /FIX = A
  /RANDOM = B C
  /TEST = 'Contrasts of A' A 1/3 1/3 1/3; A 1 -1 0; A 1 -1/2 -1/2
  /TEST(1) = 'Contrast of B' | B 1 -1
  /TEST = 'BLUP at First Level of A'
          ALL 0 1 0 0 | 1 0 1 0;
          ALL         | 1 0 0 1;
          ALL 0 1 0 0;
          ALL 0 1 0 0 | 0 1 0 1.
```

Suppose that factor *A* has three levels and factors *B* and *C* each have two levels.

- The first TEST is labeled *Contrasts of A*. It performs three contrasts among levels of *A*. The first is technically not a contrast but the mean of level 1, level 2, and level 3 of *A*, the second is between level 1 and level 2 of *A*, and the third is between level 1 and the mean of level 2 and level 3 of *A*.
- The second TEST is labeled *Contrast of B*. Coefficients for *B* are preceded by the vertical bar (|) because *B* is a random effect. This contrast computes the difference between level 1 and level 2 of *B*, and tests if the difference equals 1.
- The third TEST is labeled *BLUP at First Level of A*. There are four parameters for the fixed effects (intercept and *A*), and there are four parameters for the random effects (*B* and *C*). Coefficients for the fixed-effect parameters are separated from those for the random-effect parameters by the vertical bar (|). The coefficients correspond to the parameter estimates in the order in which the parameter estimates are listed in the output.

Example

Suppose that factor *A* has three levels and factor *B* has four levels.

```
MIXED Y BY A B
  /FIXED = A B
  /TEST = 'test example' A 1 -1 0 DIVISOR=3;
                  B 0 0 1 -1 DIVISOR=4.
```

- For effect *A*, all contrast coefficients will be divided by 3; therefore, the actual coefficients are (1/3,–1/3,0).
- For effect *B*, all contrast coefficients will be divided by 4; therefore, the actual coefficients are (0,0,1/4,–1/4).

MLP

MLP is available in SPSS Statistics Premium Edition or the Neural Network option.

```
MLP dependent variable [(MLEVEL = {S})] [dependent variable...]
                               {O}
                               {N}

    [BY factor list] [WITH covariate list]

[/EXCEPT VARIABLES = varlist]

[/RESCALE [COVARIATE = {STANDARDIZED**}]
                       {NORMALIZED    }
                       {ADJNORMALIZED }
                       {NONE          }

          [DEPENDENT = {STANDARDIZED                              }]]
                       {NORMALIZED    [(CORRECTION = {0.02**})]}
                                                     {number}
                       {ADJNORMALIZED [(CORRECTION = {0.02**})]}
                                                     {number}
                       {NONE                                   }

[/PARTITION {TRAINING = {70**   } TESTING = {30**   } HOLDOUT = {0**    }}]
             {integer}           {integer}            {integer}
            {VARIABLE = varname                                  }

[/ARCHITECTURE [AUTOMATIC = {YES**} [(MINUNITS = {1**    }, MAXUNITS = {50**   })]]
                                                 {integer}             {integer}
                           {NO   }

               [HIDDENLAYERS = {1** [(NUMUNITS = {AUTO** })]       }]
                                                 {integer}
                               {2 [(NUMUNITS = {AUTO**          })]}
                                                {integer, integer}
               [HIDDENFUNCTION = {TANH** }]   [OUTPUTFUNCTION = {IDENTITY}]]
                                 {SIGMOID}                      {SIGMOID }
                                                                {SOFTMAX }
                                                                {TANH    }

[/CRITERIA [TRAINING = {BATCH**  }]   [MINIBATCHSIZE = {AUTO** }]
                       {ONLINE   }                      {integer}
                       {MINIBATCH}

           [MEMSIZE = {1000** }]      [OPTIMIZATION = {GRADIENTDESCENT}]
                      {integer}                       {SCALEDCONJUGATE}

           [LEARNINGINITIAL = {0.4** }]   [LEARNINGLOWER = {0.001**}]
                              {number}                     {number }

           [MOMENTUM = {0.9** }]   [LEARNINGEPOCHS = {10**   }]
                       {number}                      {integer}

           [LAMBDAINITIAL = {0.0000005**}]   [SIGMAINITIAL = {0.00005**}]
                            {number     }                    {number   }

           [INTERVALCENTER = {0**   }]    [INTERVALOFFSET = {0.5** }]]
                             {number}                       {number}

[/STOPPINGRULES [ERRORSTEPS = {1**    } [(DATA = {AUTO** })]]
                              {integer}          {BOTH   }

                [TRAININGTIMER = {ON**} [(MAXTIME = {15**  })]]
                                 {OFF }             {number}

                [MAXEPOCHS = {AUTO** }
                             {integer}]

                [ERRORCHANGE = {0.0001**}]    [ERRORRATIO = {0.001**}]]
                               {number  }                   {number }

[/MISSING USERMISSING = {EXCLUDE**}]
                        {INCLUDE  }

[/PRINT [CPS**]  [NETWORKINFO**]  [SUMMARY**]  [CLASSIFICATION**]
        [SOLUTION]  [IMPORTANCE]  [NONE]]

[/PLOT [NETWORK**]  [PREDICTED]  [RESIDUAL]  [ROC]
       [GAIN]  [LIFT]  [NONE]]

[/SAVE [PREDVAL[(varname [varname...])]]

       [PSEUDOPROB[(rootname[:{25     }] [rootname...])]]]
                              {integer}

[/OUTFILE [MODEL = 'file' ['file'...]]]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 16.0

- Command introduced.

Example

```
MLP dep_var BY A B C WITH X Y Z.
```

Overview

Neural networks are a data mining tool for finding unknown patterns in databases. Neural networks can be used to make business decisions by forecasting demand for a product as a function of price and other variables or by categorizing customers based on buying habits and demographic characteristics. The MLP procedure fits a particular kind of neural network called a multilayer perceptron. The multilayer perceptron uses a feedforward architecture and can have multiple hidden layers. It is one of the most commonly used neural network architectures.

Options

Prediction or classification. One or more dependent variables may be specified, and they may be scale, categorical, or a combination. If a dependent variable has a scale measurement level, then the neural network predicts continuous values that approximate the “true” value of some continuous function of the input data. If a dependent variable is categorical, then the neural network is used to classify cases into the “best” category based on the input predictors.

Rescaling. MLP optionally rescales covariates or scale dependent variables before training the neural network. There are three rescaling options: standardization, normalization, and adjusted normalization.

Training, testing, and holdout data. MLP optionally divides the dataset into training, testing, and holdout data. The neural network is trained using the training data. The training data or testing data, or both, can be used to track errors across steps and determine when to stop training. The holdout data is completely excluded from the training process and is used for independent assessment of the final network.

Architecture selection. MLP can perform automatic architecture selection, or it can build a neural network based on user specifications. Automatic architecture selection creates a neural network with one hidden layer and finds the “best” number of hidden units. Alternatively, you can specify one or two hidden layers and define the number of hidden units in each layer.

Activation functions. Units in the hidden layers can use the hyperbolic or sigmoid activation functions. Units in the output layer can use the hyperbolic, sigmoid, identity, or softmax activation functions.

Training methods. The neural network can be built using batch, online, or mini-batch training. Gradient descent and scaled conjugate gradient optimization algorithms are available.

Missing Values. The MLP procedure has an option for treating user-missing values of categorical variables as valid. User-missing values of scale variables are always treated as invalid.

Output. MLP displays pivot table output but offers an option for suppressing most such output. Graphical output includes a network diagram (default) and a number of optional charts: predicted-by-observed values, residual-by-predicted values, ROC (Receiver Operating Characteristic) curves, cumulative gains, lift, and independent variable importance. The procedure also optionally saves predicted values in the active dataset. Synaptic weight estimates can be saved in IBM SPSS Statistics or XML files also.

Basic Specification

The basic specification is the MLP command followed by one or more dependent variables, the BY keyword and one or more factors, and the WITH keyword and one or more covariates. By default, the MLP procedure standardizes covariates and selects a training sample before training the neural network. Automatic architecture selection is used to find the “best” neural network architecture. User-missing values are excluded, and default pivot table output is displayed.

Note: Since measurement level can affect the results, if any variables (fields) have an unknown measurement level, an initial data pass will be performed to determine default measurement level for any variables with an unknown measurement level. For information on the criteria used to determine default measurement level, see SET SCALEMIN.

Syntax Rules

- All subcommands are optional.
- Subcommands may be specified in any order.
- Only a single instance of each subcommand is allowed.
- An error occurs if a keyword is specified more than once within a subcommand.
- Parentheses, equals signs, and slashes shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not allowed.
- Any split variable defined on the SPLIT FILE command may not be used as a dependent variable, factor, covariate, or partition variable.

Limitations

The WEIGHT setting is ignored with a warning by the MLP procedure.

Categorical Variables

Although the MLP procedure accepts categorical variables as predictors or as the dependent variable, the user should be cautious when using a categorical variable with a very large number of categories.

The MLP procedure temporarily recodes categorical predictors and dependent variables using one-of-*c* coding for the duration of the procedure. If there are *c* categories of a variable, then the variable is stored as *c* vectors, with the first category denoted (1,0,...,0), the next category (0,1,0,...,0), ..., and the final category (0,0,...,0,1).

This coding scheme increases the number of synaptic weights. In particular, the total number of input units is the number of scale predictors plus the number of categories across all categorical predictors. As a result, this coding scheme can lead to slower training, but more "compact" coding methods usually lead to poorly fit neural networks. If your network training is proceeding very slowly, you might try reducing the number of categories in your categorical predictors by combining similar categories or dropping cases that have extremely rare categories before running the MLP procedure.

All one-of-*c* coding is based on the training data, even if a testing or holdout sample is defined (see "PARTITION Subcommand " on page 1155). Thus, if the testing or holdout samples contain cases with predictor categories that are not present in the training data, then those cases are not used by the procedure or in scoring. If the testing or holdout samples contain cases with dependent variable categories that are not present in the training data, then those cases are not used by the procedure but they may be scored.

Replicating results

The MLP procedure uses random number generation during random assignment of partitions, random subsampling for initialization of synaptic weights, random subsampling for automatic architecture selection, and the simulated annealing algorithm used in weight initialization and automatic architecture selection. To reproduce the same randomized results in the future, use the SET command to set the initialization value for the random number generator before each run of the MLP procedure.

MLP results are also dependent on data order. The online and mini-batch training methods are explicitly dependent upon data order; however, even batch training is data-order dependent due to the weight initialization algorithm. For all three training methods, initialization of synaptic weights uses simulated annealing, which takes a random sample from the entire data set and randomly splits it into training (70%) and testing (30%) samples. The size of the random sample is $N = \min(1000, memsize)$, where *memsize* is the user-controlled maximum number of cases stored in memory. (See the /CRITERIA MEMSIZE keyword.) If the entire dataset has less than *N* cases, then all *N* cases are used.

To minimize data order effects, randomly order the cases before running the MLP procedure. To verify the stability of a given solution, you may want to obtain several different solutions with cases sorted in different random orders. In situations with extremely large file sizes, multiple runs can be performed with a sample of cases sorted in different random orders.

Finally, MLP results may be influenced by the variable order on the command line due to the different pattern of initial values assigned when the command line variable order is changed. As with data order effects, you might try different command line variable orders to assess the stability of a given solution.

In summary, if you want to exactly replicate MLP results in the future, use the same initialization value for the random number generator, the same data order, and the same command line variable order, in addition to using the same MLP procedure settings.

Examples

Basic specification with default neural network settings

```
MLP DepVar BY A B C WITH X Y Z.
```

- The MLP procedure treats *DepVar* as the dependent variable.
- Predictors *A*, *B*, and *C* are factors, and *X*, *Y*, and *Z* are covariates.
- By default, covariates are standardized before training. Also, the active dataset is partitioned into training and testing data samples, with 70% going to the training data and 30% to the testing data sample.
- Automatic architecture selection is used to find the “best” neural network architecture.
- User-missing values are excluded and default output is displayed.

User-specified neural network with two hidden layers

```
MLP DepVar BY A B C WITH X Y Z
  /PARTITION
     TRAINING=100 TESTING=0
  /ARCHITECTURE
     AUTOMATIC=NO
     HIDDENLAYERS=2 (NUMUNITS=25,10)
     OUTPUTFUNCTION=SIGMOID.
```

- The MLP procedure treats *DepVar* as the dependent variable. Predictors *A*, *B*, and *C* are factors, and *X*, *Y*, and *Z* are covariates.
- By default, covariates are standardized before training. The PARTITION subcommand overrides the default partitioning of the active dataset into training and testing data and treats all cases as training data.
- The ARCHITECTURE subcommand turns off automatic architecture selection (AUTOMATIC = NO) and specifies a neural network with two hidden layers. There are 25 hidden units in the first hidden layer and 10 hidden units in the second hidden layer. The sigmoid activation function is used for units in the output layer.
- User-missing values are excluded and default output is displayed.

Variable Lists

The command line variable lists specify the dependent variables, any categorical predictors (also known as factors), and any scale predictors (also known as covariates).

Dependent Variables

- A list of one or more dependent variables must be the first specification on the MLP command.
- Each dependent variable may be followed by the measurement level specification, which contains, in parentheses, the MLEVEL keyword followed by an equals sign and then S for scale, O for ordinal, or N for nominal. MLP treats ordinal and nominal dependent variables equivalently as categorical.

- If a measurement level is specified, then it temporarily overrides a dependent variable's setting in the data dictionary.
- If no measurement level is specified, then MLP defaults to the dictionary setting.
- If a measurement level is not specified and no setting is recorded in the data dictionary, then a numeric variable is treated as scale and a string variable is treated as categorical.
- Dependent variables can be numeric or string.
- A string variable may be defined as ordinal or nominal only.

Predictor Variables

- The names of the factors, if any, must be preceded by the keyword BY.
- If keyword BY is specified with no factors, then a warning is issued and BY is ignored.
- The names of the covariates, if any, must be preceded by the keyword WITH.
- If keyword WITH is specified with no covariates, then a warning is issued and WITH is ignored.
- A dependent variable may not be specified within a factor or covariate list. If a dependent variable is specified within one of these lists, then an error is issued.
- All variables specified within a factor or covariate list must be unique. If duplicate variables are specified within a list, then the duplicates are ignored.
- If duplicate variables are specified across the factor and covariate lists, then an error is issued.
- The universal keywords TO and ALL may be specified in the factor and covariate lists.
- Factor variables can be numeric or string.
- Covariates must be numeric.
- If no predictors at all are specified, then the procedure fits an input layer containing only the bias unit—that is, the constant-only input layer.
- At least one predictor must be specified.

EXCEPT Subcommand

The EXCEPT subcommand lists any variables that the MLP procedure should exclude from the factor or covariate lists on the command line. This subcommand is useful if the factor or covariate lists contain a large number of variables—specified using the TO or ALL keyword, for example—but there are a few variables (for example, Case ID) that should be excluded.

The EXCEPT subcommand is introduced strictly for the purpose of simplifying syntax. Missing values on factors or covariates specified on EXCEPT do not affect whether a case is included in the analysis. For example, the following two MLP commands are equivalent. In both commands, listwise deletion is based on the dependent variable and factors *A*, *B*, and *C*.

```
MLP DepVar BY A B C.
```

```
MLP DepVar BY A B C D   /EXCEPT VARIABLES=D.
```

- The EXCEPT subcommand ignores duplicate variables and variables that are not specified on the command line's factor or covariate lists.
- There is no default variable list on the EXCEPT subcommand.

RESCALE Subcommand

The RESCALE subcommand is used to rescale covariates or scale dependent variables.

All rescaling is performed based on the training data, even if a testing or holdout sample is defined (see "PARTITION Subcommand " on page 1155). That is, depending on the type of rescaling, the mean, standard deviation, minimum value, or maximum value of a covariate or dependent variable are computed

using only the training data. It is important that these covariates or dependent variables have similar distributions across the training, testing, and holdout samples. If the data are partitioned by specifying percentages on the PARTITION subcommand, then the MLP procedure attempts to ensure this similarity by random assignment. However, if you use the PARTITION subcommand VARIABLE keyword to assign cases to the training, testing, and holdout samples, then we recommend that you confirm the distributions are similar across samples before running the MLP procedure.

COVARIATE Keyword

The COVARIATE keyword specifies the rescaling method to use for covariates specified following WITH on the command line. If no covariates are specified on the command line, then the COVARIATE keyword is ignored.

STANDARDIZED. *Subtract the mean and divide by the standard deviation, (x–mean)/s.* This is the default rescaling method for covariates.

NORMALIZED. *Subtract the minimum and divide by the range, (x–min)/(max–min).*

ADJNORMALIZED. *Adjusted version of subtract the minimum and divide by the range, [2*(x–min)/(max –min)]–1 .*

NONE. *No rescaling of covariates.*

DEPENDENT Keyword

The DEPENDENT keyword specifies the rescaling method to use for scale dependent variables.

- This keyword applies only to scale dependent variables—that is, either MLEVEL=S is specified on the command line or the variable has a scale measurement level based on its data dictionary setting. If a dependent variable is not scale, then the DEPENDENT keyword is ignored for that variable.
- The availability of these rescaling methods for scale dependent variables depends on the output layer activation function in effect. See the OUTPUTFUNCTION keyword in “ARCHITECTURE Subcommand ” on page 1156 for details about the activation functions.
- If the identity activation function is in effect, then any of the rescaling methods, including NONE, may be requested. If the sigmoid activation function is in effect, then NORMALIZED is required. If the hyperbolic tangent activation function is in effect, then ADJNORMALIZED is required.
- If automatic architecture selection is in effect (/ARCHITECTURE AUTOMATIC=YES), then the default output layer activation function (identity if there are any scale dependent variables) is always used. In this case, the default rescaling method (STANDARDIZED) is also used and the DEPENDENT keyword is ignored.

STANDARDIZED. *Subtract the mean and divide by the standard deviation, (x–mean)/s.* This is the default rescaling method for scale dependent variables if the output layer uses the identity activation function. This rescaling method may not be specified if the output layer uses the sigmoid or hyperbolic tangent activation function.

NORMALIZED. *Subtract the minimum and divide by the range, (x–min)/(max–min).* This is the required rescaling method for scale dependent variables if the output layer uses the sigmoid activation function. This rescaling method may not be specified if the output layer uses the hyperbolic tangent activation function. The NORMALIZED keyword may be followed by the CORRECTION option, which specifies a number ε that is applied as a correction to the rescaling formula. In particular, the corrected formula is [x –(min–ε)]/[(max+ε)–(min–ε)]. This correction ensures that all rescaled dependent variable values will be within the range of the activation function. A real number greater than or equal to 0 must be specified. The default is 0.02.

ADJNORMALIZED. *Adjusted version of subtract the minimum and divide by the range, [2*(x–min)/(max –min)]–1 .* This is the required rescaling method for scale dependent variables if the output layer uses the hyperbolic tangent activation function. This rescaling method may not be specified if the output layer uses the sigmoid activation function. The ADJNORMALIZED keyword may be followed by the CORRECTION option, which specifies a number ε that is applied as a correction to the rescaling formula. In particular, the corrected formula is {2*[(x–(min–ε))/((max+ε)–(min–ε))]}–1. This correction ensures that all rescaled dependent variable values will be within the range of the activation function. A real number greater than or equal to 0 must be specified. The default is 0.02.

NONE. *No rescaling of scale dependent variables.*

PARTITION Subcommand

The PARTITION subcommand specifies the method of partitioning the active dataset into training, testing, and holdout samples. The training sample comprises the data records used to train the neural network. The testing sample is an independent set of data records used to track prediction error during training in order to prevent overtraining. The holdout sample is another independent set of data records used to assess the final neural network.

- The partition can be defined by specifying the ratio of cases randomly assigned to each sample (training, testing, and holdout) or by a variable that assigns each case to the training, testing, or holdout sample.
- If the PARTITION subcommand is not specified, then the default partition randomly assigns 70% of the cases to the training sample, 30% to the testing sample, and 0% to the holdout sample. If you want to specify a different random assignment, then you must specify new values for the TRAINING, TESTING, and HOLDOUT keywords. The value specified on each keyword gives the relative number of cases in the active dataset to assign to each sample. For example, `/PARTITION TRAINING = 50 TESTING = 30 HOLDOUT = 20` is equivalent to `/PARTITION TRAINING = 5 TESTING = 3 HOLDOUT = 2`; both subcommands randomly assign 50% of the cases to the training sample, 30% to the testing sample, and 20% to the holdout sample.
- If you want to be able to reproduce results based on the TRAINING, TESTING, and HOLDOUT keywords later, use the SET command to set the initialization value for the random number generator before running the MLP procedure.
- Be aware of the relationship between rescaling and partitioning. See the topic "RESCALE Subcommand " on page 1153 for more information.
- All partitioning is performed after listwise deletion of any cases with invalid data for any variable used by the procedure. See "MISSING Subcommand " on page 1161 for details about valid and invalid data.

TRAINING Keyword

The TRAINING keyword specifies the relative number of cases in the active dataset to randomly assign to the training sample. The value must be an integer greater than 0. The default (if the PARTITION subcommand is not specified) is 70.

TESTING Keyword

The TESTING keyword specifies the relative number of cases in the active dataset to randomly assign to the testing sample. The value must be an integer greater than 0. The default (if the PARTITION subcommand is not specified) is 30.

HOLDOUT Keyword

The HOLDOUT keyword specifies the relative number of cases in the active dataset to randomly assign to the holdout sample. The value must be an integer greater than 0. The default (if the PARTITION subcommand is not specified) is 0.

VARIABLE Keyword

The VARIABLE keyword specifies a variable that assigns each case in the active dataset to the training, testing, or holdout sample. Cases with a positive value on the variable are assigned to the training sample, cases with a value of 0 to the testing sample, and cases with a negative value to the holdout sample. Cases with a system-missing value are excluded from the analysis. (Any user-missing values for the partition variable are always treated as valid.)

The variable may not be the dependent variable or any variable specified on the command line factor or covariate lists. The variable must be numeric.

ARCHITECTURE Subcommand

The ARCHITECTURE subcommand is used to specify the neural network architecture. By default, automatic architecture selection is used to build the network. However, you have the option of overriding automatic architecture selection and building a more specific structure.

AUTOMATIC Keyword

The AUTOMATIC keyword indicates whether to use automatic architecture selection to build the neural network. Automatic architecture selection builds a network with one hidden layer. Using a prespecified range defining the minimum and maximum number of hidden units, automatic architecture selection computes the "best" number of units in the hidden layer. Automatic architecture selection uses the default activation functions for the hidden and output layers.

If automatic architecture selection is used, then a random sample from the total dataset (excluding any data records included in the holdout sample as defined on the PARTITION subcommand) is taken and split into training (70%) and testing (30%) samples. This random sample is used to find the architecture and fit the network. Then, the network is retrained using the entire dataset (taking into account the training, testing, and holdout samples defined on the PARTITION subcommand), with the synaptic weights obtained from the random sample used as the initial weights.

The size of the random sample N = min(1000, memsize), where memsize is the user-specified maximum number of cases to store in memory (see the MEMSIZE keyword in "CRITERIA Subcommand " on page 1157). If the total dataset (excluding holdout cases) has less than N cases, then all cases (excluding holdout cases) are used. If you want to be able to reproduce results based on the AUTOMATIC keyword later, use the SET command to set the initialization value for the random number generator before running the MLP procedure.

YES. *Use automatic architecture selection to build the network.* This is the default.

- The YES keyword may be followed by parentheses containing the MINUNITS and MAXUNITS options, which specify the minimum and maximum number of units, respectively, that automatic architecture selection will consider in determining the "best" number of units. It is invalid to specify only one option; you must specify both or neither. The options may be specified in any order and must be separated by a comma or space character. Both numbers must be integers greater than 0, with MINUNITS less than MAXUNITS. The defaults are MINUNITS=1, MAXUNITS=50.
- If AUTOMATIC=YES is specified, then all other ARCHITECTURE subcommand keywords are invalid.

NO. *Do not use automatic architecture selection to build the network.* All other ARCHITECTURE subcommand keywords are valid only if AUTOMATIC=NO is specified.

HIDDENLAYERS Keyword

The HIDDENLAYERS keyword specifies the number of hidden layers in the neural network.

This keyword is honored only if automatic architecture selection is not used—that is, if AUTOMATIC=NO. If automatic architecture selection is in effect, then the HIDDENLAYERS keyword is ignored.

1. *One hidden layer.* This is the default. The HIDDENLAYERS=1 specification may be followed by the NUMUNITS option, which gives the number of units in the first hidden layer, excluding the bias unit. Specify AUTO to automatically compute the number of units based on the number of input and output units. Alternatively, specify an integer greater than or equal to 1 to request a particular number of hidden units. The default is AUTO.

2. *Two hidden layers.* The HIDDENLAYERS=2 specification may be followed by the NUMUNITS option, which gives the number of units in the first and second hidden layers, excluding the bias unit in each layer. Specify AUTO to automatically compute the numbers of units based on the number of input and output units. Alternatively, specify two integers greater than or equal to 1 to request particular numbers of hidden units in the first and second hidden layers, respectively. The default is AUTO.

HIDDENFUNCTION Keyword

The HIDDENFUNCTION keyword specifies the activation function to use for all units in the hidden layers. This keyword is honored only if automatic architecture selection is not used—that is, if AUTOMATIC=NO. If automatic architecture selection is in effect, then the HIDDENFUNCTION keyword is ignored.

TANH. *Hyperbolic tangent.* This function has form: $\gamma(c) = \tanh(c) = (e^{c} - e^{-c})/(e^{c} + e^{-c})$. It takes real-valued arguments and transforms them to the range (–1, 1). This is the default activation function for all units in the hidden layers.

SIGMOID. *Sigmoid.* This function has form: $\gamma(c) = 1/(1+e^{-c})$. It takes real-valued arguments and transforms them to the range (0, 1).

OUTPUTFUNCTION Keyword

The OUTPUTFUNCTION keyword specifies the activation function to use for all units in the output layer.

The activation function used in the output layer has a special relationship with the error function, which is the measure that the neural network is trying to minimize. In particular, the error function is automatically assigned based on the activation function for the output layer. Sum-of-squares error, the sum of the squared deviations between the observed dependent variable values and the model-predicted values, is used when the identity, sigmoid, or hyperbolic tangent activation function is applied to the output layer. Cross-entropy error is used when the softmax activation function is applied to the output layer.

The OUTPUTFUNCTION keyword is honored only if automatic architecture selection is not used—that is, if AUTOMATIC=NO. If automatic architecture selection is in effect, then OUTPUTFUNCTION is ignored.

IDENTITY. *Identity.* This function has form: $\gamma(c) = c$. It takes real-valued arguments and returns them unchanged. This is the default activation function for units in the output layer if there are any scale dependent variables.

SIGMOID. *Sigmoid.* This function has form: $\gamma(c) = 1/(1+e^{-c})$. It takes real-valued arguments and transforms them to the range (0, 1).

TANH. *Hyperbolic tangent.* This function has form: $\gamma(c) = \tanh(c) = (e^{c} - e^{-c})/(e^{c} + e^{-c})$. It takes real-valued arguments and transforms them to the range (–1, 1).

SOFTMAX. *Softmax.* This function has form: $\gamma(c_k) = \exp(c_k)/\Sigma_j \exp(c_j)$. It takes a vector of real-valued arguments and transforms it to a vector whose elements fall in the range (0, 1) and sum to 1. Softmax is available only if all dependent variables are categorical; if SOFTMAX is specified and there are any scale dependent variables, then an error is issued. This is the default activation function for units in the output layer if all dependent variables are categorical.

CRITERIA Subcommand

The CRITERIA subcommand specifies computational and resource settings for the MLP procedure.

TRAINING Keyword

The TRAINING keyword specifies the training type, which determines how the neural network processes training data records.

The online and mini-batch training methods are explicitly dependent upon data order; however, even batch training is dependent upon data order because initialization of synaptic weights involves subsampling from the dataset. To minimize data order effects, randomly order the cases before running the MLP procedure.

BATCH. *Batch training.* Updates the synaptic weights only after passing all training data records—that is, batch training uses information from all records in the training dataset. Batch training is often preferred because it directly minimizes the total prediction error. However, batch training may need to update the weights many times until one of the stopping rules is met and, hence, may need many data passes. It is most useful for smaller datasets. This is the default training type.

ONLINE. *Online training.* Updates the synaptic weights after every single training data record—that is, online training uses information from one record at a time. Online training continuously gets a record and updates the weights until one of the stopping rules is met. If all the records are used once and none of the

stopping rules is met, then the process continues by recycling the data records. Online training is superior to batch only for larger datasets with associated predictors. If there are many records and many inputs, and their values are not independent of each other, then online training can more quickly obtain a reasonable answer than batch training.

MINIBATCH. *Mini-batch training.* Divides the training data records into *K* groups of approximately equal size, then updates the synaptic weights after passing one group—that is, mini-batch training uses information from a group of records. The process then recycles the data group if necessary. The number of training records per mini-batch is determined by the MINIBATCHSIZE keyword. Mini-batch training offers a compromise between batch and online training, and it may be best for "medium-size" datasets.

MINIBATCHSIZE Keyword

The MINIBATCHSIZE keyword specifies the number of training records per mini-batch.

- Specify AUTO to automatically compute the number of records per mini-batch as $R = \min(\max(M/10,2), memsize)$, where M is the number of training records and *memsize* is the maximum number of cases to store in memory (see the MEMSIZE keyword below). If the remainder of M/R is r, then when the end of the data is reached, the process places the final r records in the same mini-batch with the first $R - r$ records of the next data pass. This “wrapping” of mini-batches will place different cases in the mini-batches with each data pass unless R divides M with no remainder.
- Alternatively, specify an integer greater than or equal to 2 and less than or equal to *memsize* to request a particular number of records. If the number of training records turns out to be less than the specified MINIBATCHSIZE, the number of training records is used instead.
- The default is AUTO.
- This keyword is ignored if TRAINING = MINIBATCH is not in effect.

MEMSIZE Keyword

The MEMSIZE keyword specifies the maximum number of cases to store in memory when synaptic weight initialization, automatic architecture selection, and/or mini-batch training is in effect.

- Specify an integer greater than or equal to 2. The default is 1000.

OPTIMIZATION Keyword

The OPTIMIZATION keyword specifies the optimization algorithm used to determine the synaptic weights.

GRADIENTDESCENT. *Gradient descent.* Gradient descent is the required optimization algorithm for online and mini-batch training. It is optional for batch training. When gradient descent is used with online and mini-batch training, the algorithm’s user-specified parameters are the initial learning rate, lower bound for the learning rate, momentum, and number of data passes (see the LEARNINGINITIAL, LEARNINGLOWER, MOMENTUM, and LEARNINGEPOCHS keywords, respectively). With batch training, the user-specified parameters are the initial learning rate and the momentum.

SCALEDCONJUGATE. *Scaled conjugate gradient.* Scaled conjugate gradient is the default for batch training. The assumptions that justify the use of conjugate gradient methods do not apply to the online and mini-batch training, so this method may not be used if TRAINING = ONLINE or MINIBATCH. The user-specified parameters are the initial lambda and sigma (see the LAMBDAINITIAL and SIGMAINITIAL keywords).

LEARNINGINITIAL Keyword

The LEARNINGINITIAL keyword specifies the initial learning rate η_0 for the gradient descent optimization algorithm.

- Specify a number greater than 0. The default is 0.4.
- This keyword is ignored if OPTIMIZATION = GRADIENTDESCENT is not in effect.

LEARNINGLOWER Keyword

The LEARNINGLOWER keyword specifies the lower boundary for the learning rate η_{low} when gradient descent is used with online or mini-batch training.

- Specify a number greater than 0 and less than the initial learning rate (see the LEARNINGINITIAL keyword). The default is 0.001.
- This keyword is ignored if TRAINING = ONLINE or MINIBATCH and OPTIMIZATION = GRADIENTDESCENT are not in effect.

MOMENTUM Keyword

The MOMENTUM keyword specifies the initial momentum rate α for the gradient descent optimization algorithm.

- Specify a number greater than 0. The default is 0.9.
- This keyword is ignored if OPTIMIZATION = GRADIENTDESCENT is not in effect.

LEARNINGEPOCHS Keyword

The LEARNINGEPOCHS keyword specifies the number of epochs (data passes of the training set) *p* to reduce the learning rate when gradient descent is used with online or mini-batch training. You can control the learning rate decay factor β by specifying the number of epochs it takes for the learning rate to decrease from η_0 to η_{low}. This corresponds to $\beta = (1/p\,K)*\ln(\eta_0/\eta_{low})$, where *K* is the total number of mini-batches in the training dataset. For online training, *K* = *M*, where *M* is the number of training records.

- Specify an integer greater than 0. The default is 10.
- This keyword is ignored if TRAINING = ONLINE or MINIBATCH and OPTIMIZATION = GRADIENTDESCENT are not in effect.

LAMBDAINITIAL Keyword

The LAMBDAINITIAL keyword specifies the initial lambda, λ_0, for the scaled conjugate gradient optimization algorithm.

- Specify a number greater than 0 and less than 10^{-6}. The default is 0.0000005.
- This keyword is ignored if OPTIMIZATION = SCALEDCONJUGATE is not in effect.

SIGMAINITIAL Keyword

The SIGMAINITIAL keyword specifies the initial sigma, σ_0, for the scaled conjugate gradient optimization algorithm.

- Specify a number greater than 0 and less than 10^{-4}. The default is 0.00005.
- This keyword is ignored if OPTIMIZATION = SCALEDCONJUGATE is not in effect.

INTERVALCENTER and INTERVALOFFSET Keywords

The INTERVALCENTER and INTERVALOFFSET keywords specify the interval $[\alpha_0-\alpha, \alpha_0+\alpha]$ in which weight vectors are randomly generated when simulated annealing is used. INTERVALCENTER corresponds to α_0 and INTERVALOFFSET corresponds to α.

- Simulated annealing is used to break out of a local minimum, with the goal of finding the global minimum, during the optimization algorithm. This approach is used in weight initialization and automatic architecture selection.
- Specify a number for INTERVALCENTER. The INTERVALCENTER default is 0. Specify a number greater than 0 for INTERVALOFFSET. The INTERVALOFFSET default is 0.5. The default interval is [–0.5, 0.5].

STOPPINGRULES Subcommand

The STOPPINGRULES subcommand specifies the rules that determine when to stop training the neural network. Training proceeds through at least one data pass. Training can then be stopped according to the following criteria, which are listed as STOPPINGRULES keywords. Stopping rules are checked in the listed order. (In the keyword descriptions, a step is a data pass for the online and mini-batch methods, an iteration for the batch method.)

Note: After each complete data pass, online and mini-batch training require an extra data pass in order to compute the training error. This extra data pass can slow training considerably, so if you use online or

mini-batch training, we recommend specifying a testing dataset. Then, if you use only the testing set in the ERRORSTEPS criterion, the ERRORCHANGE and ERRORRATIO criteria will not be checked.

ERRORSTEPS Keyword

The ERRORSTEPS keyword specifies the number of steps, *n*, to allow before checking for a decrease in error. If there is no decrease in error after *n* steps, then training stops.

- Any integer greater than or equal to 1 may be specified. The default is 1.
- The DATA option following ERRORSTEPS specifies how to compute error.

(DATA=AUTO). *Compute error using the testing sample if it exists or using the training sample otherwise.* If the error at any step does not decrease below the current minimum error (based on preceding steps) over the next *n* steps, then training stops.

- For online and mini-batch training, if there is no testing sample, then the procedure computes error using the training sample. Batch training, on the other hand, guarantees a decrease in the training sample error after each data pass, thus this option is ignored if batch training is in effect and there is no testing sample.
- DATA = AUTO is the default option.

(DATA=BOTH). *Compute errors using the testing sample and the training sample.* If neither the testing sample error nor the training sample error decreases below its current minimum error over the next *n* steps, then training stops.

- For batch training, which guarantees a decrease in the training sample error after each data pass, this option is the same as DATA= AUTO.
- DATA = BOTH may be specified only if testing data are defined—that is, /PARTITION TESTING is specified with a number greater than zero or /PARTITION VARIABLE is used. If DATA = BOTH is specified when /PARTITION TESTING = 0, or when /PARTITION VARIABLE is used but no testing data exist in the active dataset, then an error is issued.

TRAININGTIMER Keyword

The TRAININGTIMER keyword specifies whether the training timer is turned on or off.

- If TRAININGTIMER = ON, then the MAXTIME option gives the maximum number of minutes allowed for training. Training stops if the algorithm exceeds the maximum allotted time.
- If TRAININGTIMER = OFF, then the MAXTIME option is ignored.
- TRAININGTIMER may be specified with keyword ON or OFF. The default is ON.
- The MAXTIME option may be specified with any number greater than 0. The default is 15.

MAXEPOCHS Keyword

The MAXEPOCHS keyword specifies the maximum number of epochs (data passes) allowed for the training data. If the maximum number of epochs is exceeded, then training stops.

- Specify AUTO to automatically compute the maximum number of epochs as max(2*N*+1, 100), where *N* is the number of synaptic weights in the neural network.
- Alternatively, specify an integer greater than 0 to request a particular maximum number of epochs.
- The default is AUTO.

ERRORCHANGE Keyword

The ERRORCHANGE keyword specifies the relative change in training error criterion. Training stops if the relative change in the training error compared to the previous step is less than the criterion value.

- Any number greater than 0 may be specified. The default is 0.0001.
- For online and mini-batch training, this criterion is ignored if the ERRORSTEPS criterion uses only testing data.

ERRORRATIO Keyword

The ERRORRATIO keyword specifies the training error ratio criterion. Training stops if the ratio of the training error to the error of the null model is less than the criterion value. The null model predicts the average value for all dependent variables.

- Any number greater than 0 may be specified. The default is 0.001.
- For online and mini-batch training, this criterion is ignored if the ERRORSTEPS criterion uses only testing data.

MISSING Subcommand

The MISSING subcommand is used to control whether user-missing values for categorical variables—that is, factors and categorical dependent variables—are treated as valid values. By default, user-missing values for categorical variables are treated as invalid.

- User-missing values for scale variables are always treated as invalid.
- System-missing values for any variables are always treated as invalid.

USERMISSING = EXCLUDE. *User-missing values for categorical variables are treated as invalid.* This is the default.

USERMISSING = INCLUDE. *User-missing values for categorical variables are treated as valid values.*

PRINT Subcommand

The PRINT subcommand indicates the tabular output to display and can be used to request a sensitivity analysis. If PRINT is not specified, then the default tables are displayed. If PRINT is specified, then only the requested PRINT output is displayed.

CPS Keyword

The CPS keyword displays the case processing summary table, which summarizes the number of cases included and excluded in the analysis, in total and by training, testing, and holdout samples. This table is shown by default.

NETWORKINFO Keyword

The NETWORKINFO keyword displays information about the neural network, including the dependent variables, number of input and output units, number of hidden layers and units, and activation functions. This table is shown by default.

SUMMARY Keyword

The SUMMARY keyword displays a summary of the neural network results, including the error, the relative error or percent of incorrect predictions, the stopping rule used to stop training, and the training time.

- The error is the sum-of-squares error when the identity, sigmoid, or hyperbolic tangent activation function is applied to the output layer. It is the cross-entropy error when the softmax activation function is applied to the output layer.
- In addition, relative errors or percents of incorrect predictions are displayed, depending on the dependent variable measurement levels. If any dependent variable has a scale measurement level, then the average overall relative error (relative to the mean model) is displayed. If all dependent variables are categorical, then the average percent of incorrect predictions is displayed. Relative errors or percents of incorrect predictions are also displayed for individual dependent variables.
- Summary results are given for the training data and for testing and hold-out data if they exist.
- This table is shown by default.

CLASSIFICATION Keyword

The CLASSIFICATION keyword displays a classification table for each categorical dependent variable. The table gives the number of cases classified correctly and incorrectly for each dependent variable category.

- In addition to classification tables, the CLASSIFICATION keyword reports the percent of the total cases that were correctly classified. A case is correctly classified if its highest predicted probabilities correspond to the observed categories for that case.
- Classification results are given for the training data and for testing and holdout data if they exist.
- Classification results are shown by default.
- The CLASSIFICATION keyword is ignored for scale dependent variables.

SOLUTION Keyword

The SOLUTION keyword displays the synaptic weights—that is, the coefficient estimates, from layer *i*–1 unit *j* to layer *i* unit *k*. The synaptic weights are based on the training sample even if the active dataset is partitioned into training, testing, and holdout data.

This table is not shown by default because the number of synaptic weights may be extremely large, and these weights are generally not used for interpreting network results.

IMPORTANCE Keyword

The IMPORTANCE keyword performs a sensitivity analysis, which computes the importance of each predictor in determining the neural network. The analysis is based on the combined training and testing samples or only the training sample if there is no testing sample. This keyword creates a table and a chart displaying importance and normalized importance for each predictor.

Sensitivity analysis is not performed by default because it is computationally expensive and time-consuming if there are a large number of predictors or cases.

NONE Keyword

The NONE keyword suppresses all PRINT output except the Notes table and any warnings. This keyword may not be specified with any other PRINT keywords.

PLOT Subcommand

The PLOT subcommand indicates the chart output to display. If PLOT is not specified, then the default chart (the network diagram) is displayed. If PLOT is specified, then only the requested PLOT output is displayed.

NETWORK Keyword

The NETWORK keyword displays the network diagram. This chart is shown by default.

PREDICTED Keyword

The PREDICTED keyword displays a predicted-by-observed value chart for each dependent variable. For categorical dependent variables, a boxplot of predicted pseudo-probabilities is displayed. For scale dependent variables, a scatterplot is displayed.

- Predicted-by-observed value charts are based on the combined training and testing samples or only the training sample if there is no testing sample.

RESIDUAL Keyword

The RESIDUAL keyword displays a residual-by-predicted value chart for each scale dependent variable. This chart is available only for scale dependent variables. The RESIDUAL keyword is ignored for categorical dependent variables.

- Residual-by-predicted value charts are based on the combined training and testing samples or only the training sample if there is no testing sample.

ROC Keyword

The ROC keyword displays an ROC (Receiver Operating Characteristic) chart for each categorical dependent variable. It also displays a table giving the area under each curve in the chart.

- For a given dependent variable, the ROC chart displays one curve for each category. If the dependent variable has two categories, then each curve treats the category at issue as the positive state versus the other category. If the dependent variable has more than two categories, then each curve treats the category at issue as the positive state versus the aggregate of all other categories.
- This chart is available only for categorical dependent variables. The ROC keyword is ignored for scale dependent variables.
- ROC charts and area computations are based on the combined training and testing samples or only the training sample if there is no testing sample.

GAIN Keyword

The GAIN keyword displays a cumulative gains chart for each categorical dependent variable.

- The display of one curve for each dependent variable category is the same as for the ROC keyword.
- This chart is available only for categorical dependent variables. The GAIN keyword is ignored for scale dependent variables.
- Cumulative gains charts are based on the combined training and testing samples or only the training sample if there is no testing sample.

LIFT Keyword

The LIFT keyword displays a lift chart for each categorical dependent variable.

- The display of one curve for each dependent variable category is the same as for the ROC keyword.
- This chart is available only for categorical dependent variables. The LIFT keyword is ignored for scale dependent variables.
- Lift charts are based on the combined training and testing samples or only the training sample if there is no testing sample.

NONE Keyword

The NONE keyword suppresses all PLOT output. This keyword may not be specified with any other PLOT keywords.

SAVE Subcommand

The SAVE subcommand writes optional temporary variables to the active dataset.

PREDVAL (varname varname...). *Predicted value or category.* This saves the predicted value for scale dependent variables and the predicted category for categorical dependent variables.

- Specify one or more unique, valid variable names. There should be as many variable names specified as there are dependent variables, and the names should be listed in the order of the dependent variables on the command line. If you do not specify enough variable names, then default names are used for any remaining variables. If you specify too many variable names, then any remaining names are ignored.
- If there is only one dependent variable, then the default variable name is *MLP_PredictedValue*. If there are multiple dependent variables, then the default variable names are *MLP_PredictedValue_1*, *MLP_PredictedValue_2*, etc., corresponding to the order of the dependent variables on the command line.

PSEUDOPROB (rootname:n rootname...). *Predicted pseudo-probability.* If a dependent variable is categorical, then this keyword saves the predicted pseudo-probabilities of the first *n* categories of that dependent variable.

- Specify one or more unique, valid variable names. There should be as many variable names specified as there are categorical dependent variables, and the names should be listed in the order of the categorical dependent variables on the command line. The specified names are treated as rootnames. Suffixes are added to each rootname to get a group of variable names corresponding to the categories

for a given dependent variable. If you do not specify enough variable names, then default names are used for any remaining categorical dependent variables. If you specify too many variable names, then any remaining names are ignored.

- A colon and a positive integer giving the number of probabilities to save for a dependent variable can follow the rootname.
- If there is only one dependent variable, then the default rootname is *MLP_PseudoProbability*. If there are multiple dependent variables, then the default rootnames are *MLP_PseudoProbability_1*, *MLP_PseudoProbability_2*, etc., corresponding to the order of the categorical dependent variables on the command line and taking into account the position of any scale dependent variables.
- The default *n* is 25.
- This keyword is ignored for scale dependent variables.

Probabilities and Pseudo-probabilities

Categorical dependent variables with softmax activation and cross-entropy error will have a predicted value for each category, where each predicted value is the probability that the case belongs to the category.

Categorical dependent variables with sum-of-squares error will have a predicted value for each category, but the predicted values cannot be interpreted as probabilities. The SAVE subcommand saves these predicted pseudo-probabilities even if any are less than zero or greater than one or the sum for a given dependent variable is not 1.

The ROC, cumulative gains, and lift charts (see `/PLOT ROC`, `GAIN`, and `LIFT`, respectively) are created based on pseudo-probabilities. In the event that any of the pseudo-probabilities are less than zero or greater than one or the sum for a given variable is not 1, they are first rescaled to be between zero and one and to sum to 1. The SAVE subcommand saves the original pseudo-probabilities, but the charts are based on rescaled pseudo-probabilities.

Pseudo-probabilities are rescaled by dividing by their sum. For example, if a case has predicted pseudo-probabilities of 0.50, 0.60, and 0.40 for a three-category dependent variable, then each pseudo-probability is divided by the sum 1.50 to get 0.33, 0.40, and 0.27.

If any of the pseudo-probabilities are negative, then the absolute value of the lowest is added to all pseudo-probabilities before the above rescaling. For example, if the pseudo-probabilities are -0.30, 0.50, and 1.30, then first add 0.30 to each value to get 0.00, 0.80, and 1.60. Next, divide each new value by the sum 2.40 to get 0.00, 0.33, and 0.67.

OUTFILE Subcommand

The `OUTFILE` subcommand saves XML-format (PMML) files containing the synaptic weights. You can use this model file to apply the model information to other data files for scoring purposes. See the topic "Scoring expressions" on page 86 for more information.

- Filenames must be specified in full. `MLP` does not supply extensions.
- The `MODEL` keyword is not honored if split-file processing is in effect (see "SPLIT FILE" on page 1897). If this keyword is specified when split-file processing is on, a warning is displayed.

MODEL = 'file' 'file'.... *Writes the synaptic weights to XML (PMML) files.* Specify one or more unique, valid filenames. There should be as many filenames as there are dependent variables, and the names should be listed in the order of the dependent variables on the command line. If you do not specify enough filenames, then an error is issued. If you specify too many filenames, then any remaining names are ignored.

If any 'file' specification refers to an existing file, then the file is overwritten. If any 'file' specifications refer to the same file, then only the last instance of this 'file' specification is honored.

MODEL CLOSE

```
MODEL CLOSE NAME={handlelist}
                 {ALL       }
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release history

Release 13.0

- Command introduced.

Example

```
MODEL CLOSE NAME=discrimmod1 twostep1.
MODEL CLOSE NAME=ALL.
```

Overview

MODEL CLOSE is used to discard cached models and their associated model handle names (see) .

Basic specification

The basic specification is NAME followed by a list of model handles. Each model handle name should match the name specified on the MODEL HANDLE command. The keyword ALL specifies that all model handles are to be closed.

MODEL HANDLE

```
MODEL HANDLE NAME=handle FILE='file specification'
 [/OPTIONS  [MISSING=[{SUBSTITUTE**}]] ]
                      {SYSMIS      }

 [/MAP  VARIABLES=varlist MODELVARIABLES=varlist  ]
```

**Default if the keyword is omitted.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic "Command Order" on page 36 for more information.

Release history

Release 13.0

- Command introduced.

Example

```
MODEL HANDLE NAME=discrimmod1 FILE='/modelfiles/discrim1.mml'.
```

Overview

MODEL HANDLE reads an external XML file or ZIP archive containing specifications for a predictive model. It caches the model specifications and associates a unique name (handle) with the cached model. The model can then be used by the APPLYMODEL and STRAPPLYMODEL transformation functions to calculate scores and other results (see) . The MODEL CLOSE command is used to discard a cached model from memory.

Different models can be applied to the same data by using separate MODEL HANDLE commands for each of the models.

Options

Variable Mapping. You can map any or all of the variables in the original model to different variables in the current active dataset. By default, the model is applied to variables in the current active dataset with the same names as the variables in the original model.

Handling Missing Values. You can choose how to handle cases with missing values. By default, an attempt is made to substitute a sensible value for a missing value, but you can choose to treat missing values as system-missing.

Basic specification

The basic specification is NAME and FILE. NAME specifies the model handle name to be used when referring to this model. FILE specifies the external file containing the model specifications.

Subcommand order

- Subcommands can be specified in any order.

Syntax rules

- When using the MAP subcommand, you must specify both the VARIABLES and MODELVARIABLES keywords.
- Multiple MAP subcommands are allowed. Each MAP subcommand should provide the mappings for a distinct subset of the variables. Subsequent mappings of a given variable override any previous mappings of that same variable.

Operations

- A model handle is used only during the current working session. The handle is not saved with the data file.
- Issuing a SET LOCALE command that changes the computer's code page requires closing any existing model handles (using MODEL CLOSE) and reopening the models (using MODEL HANDLE) before proceeding with scoring.

Models Supported for Scoring

IBM SPSS Statistics can score models created by IBM SPSS Statistics, IBM SPSS Modeler, and IBM SPSS AnswerTree.

The following table lists the set of scoring functions available for each type of model that supports scoring. The function type denoted as PROBABILITY (category) refers to specification of a particular category (the optional third parameter) for the PROBABILITY function.

Table 71. Supported functions by model type

Model type	**Supported functions**
Tree (categorical target)	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE, NODEID
Tree (scale target)	PREDICT, NODEID, STDDEV
Boosted Tree (C5.0)	PREDICT, CONFIDENCE
Linear Regression	PREDICT, STDDEV
Automatic Linear Models	PREDICT
Binary Logistic Regression	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE
Conditional Logistic Regression	PREDICT
Multinomial Logistic Regression	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE
General Linear Model	PREDICT, STDDEV
Discriminant	PREDICT, PROBABILITY, PROBABILITY (category)
TwoStep Cluster	PREDICT
K-Means Cluster	PREDICT
Kohonen	PREDICT
Neural Net (categorical target)	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE
Neural Net (scale target)	PREDICT
Naive Bayes	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE
Anomaly Detection	PREDICT

Table 71. Supported functions by model type (continued)	
Model type	**Supported functions**
Ruleset	PREDICT, CONFIDENCE
Generalized Linear Model (categorical target)	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE
Generalized Linear Model (scale target)	PREDICT, STDDEV
Generalized Linear Mixed Model (categorical target)	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE
Generalized Linear Mixed Model (scale target)	PREDICT
Ordinal Multinomial Regression	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE
Cox Regression	PREDICT, CUMHAZARD
Nearest Neighbor (scale target)	PREDICT, NEIGHBOR, NEIGHBOR(K), DISTANCE, DISTANCE(K)
Nearest Neighbor (categorical target)	PREDICT, PROBABILITY, PROBABILITY (category), CONFIDENCE,NEIGHBOR, NEIGHBOR(K),DISTANCE, DISTANCE(K)

- For the Binary Logistic Regression, Multinomial Logistic Regression, and Naive Bayes models, the value returned by the CONFIDENCE function is identical to that returned by the PROBABILITY function.
- For the K-Means model, the value returned by the CONFIDENCE function is the least distance.
- For tree and ruleset models, the confidence can be interpreted as an adjusted probability of the predicted category and is always less than the value given by PROBABILITY. For these models, the confidence value is more reliable than the value given by PROBABILITY.
- For neural network models, the confidence provides a measure of whether the predicted category is much more likely than the second-best predicted category.
- For Ordinal Multinomial Regression and Generalized Linear Model, the PROBABILITY function is supported when the target variable is binary.
- For nearest neighbor models without a target variable, the available functions are NEIGHBOR and DISTANCE.

For information on applying scoring functions from a model, see “Scoring expressions” on page 86.

NAME Subcommand

NAME specifies the model handle name. The rules for valid model handle names are the same as for IBM SPSS Statistics variable names with the addition of the $ character as an allowed first character. The model handle name should be unique for each model.

FILE keyword

The FILE keyword is used to specify the external model file that you want to refer to by the model handle.

- File specifications should be enclosed in quotation marks.
- Fully qualified paths are recommended to avoid ambiguity.

OPTIONS subcommand

Use OPTIONS to control the treatment of missing data.

MISSING keyword

The MISSING keyword controls the treatment of missing values, encountered during the scoring process, for the predictor variables defined in the model. A missing value in the context of scoring refers to one of the following:

- A predictor contains no value. For numeric fields (variables), this means the system-missing value. For string fields, this means a null string.
- The value has been defined as user-missing, in the model, for the given predictor. Values defined as user-missing in the active dataset, but not in the model, are not treated as missing values in the scoring process.
- The predictor is categorical and the value is not one of the categories defined in the model.

SYSMIS. *Return the system-missing value when scoring a case with a missing value.*

SUBSTITUTE. *Use value substitution when scoring cases with missing values.* This is the default.

The method for determining a value to substitute for a missing value depends on the type of predictive model:

- **IBM SPSS Statistics models.** For independent variables in linear regression (REGRESSION command) and discriminant (DISCRIMINANT command) models, if mean value substitution for missing values was specified when building and saving the model, then this mean value is used in place of the missing value in the scoring computation, and scoring proceeds. If the mean value is not available, then APPLYMODEL and STRAPPLYMODEL return the system-missing value.
- **IBM SPSS AnswerTree models & TREE command models.** For the CHAID and Exhaustive CHAID algorithms, the biggest child node is selected for a missing split variable. The biggest child node is determined by the algorithm to be the one with the largest population among the child nodes using learning sample cases. For C&RT and QUEST algorithms, surrogate split variables (if any) are used first. (Surrogate splits are splits that attempt to match the original split as closely as possible using alternate predictors.) If no surrogate splits are specified or all surrogate split variables are missing, the biggest child node is used.
- **IBM SPSS Modeler models.** Linear regression models are handled as described under IBM SPSS Statistics models. Logistic regression models are handled as described under Logistic Regression models. C&R Tree models are handled as described for C&RT models under AnswerTree models.
- **Logistic Regression models.** For covariates in logistic regression models, if a mean value of the predictor was included as part of the saved model, then this mean value is used in place of the missing value in the scoring computation, and scoring proceeds. If the predictor is categorical (for example, a factor in a logistic regression model), or if the mean value is not available, then APPLYMODEL and STRAPPLYMODEL return the system-missing value.

Example

```
MODEL HANDLE NAME=twostep1 FILE='twostep1.mml'
   /OPTIONS MISSING=SYSMIS.
```

- In this example, missing values encountered during scoring give rise to system-missing results.

MAP subcommand

Use MAP to map a set of variable names from the input model to a different set of variable names in the active dataset. Both the VARIABLES and MODELVARIABLES keywords must be included. MODELVARIABLES is used to specify the list of variable names from the model that are to be mapped. VARIABLES is used to specify the list of target variable names in the active dataset.

- Both variable lists must contain the same number of names.
- No validation is performed against the current active file dictionary when the MODEL HANDLE command is processed. Errors associated with incorrect target variable names or variable data type mismatch are signaled when an APPLYMODEL or STRAPPLYMODEL transformation is processed.

Example

```
MODEL HANDLE NAME=creditmod1 FILE='credit1.mml'
   /MAP VARIABLES=agecat curdebt
        MODELVARIABLES=age debt.
```

- In this example, the variable *age* from the model file is mapped to the variable *agecat* in the active dataset. Likewise, the variable *debt* from the model file is mapped to the variable *curdebt* in the active dataset.

MODEL LIST

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 13.0

- Command introduced.

Example

```
MODEL LIST.
```

Overview

MODEL LIST produces a list, in pivot table format, of the existing model handles (see) . The listing includes the handle name, the type of predictive model (for example, NOMREG) associated with the model handle, the external XML model file associated with the model handle, and the method (specified on the MODEL HANDLE command) for handling cases with missing values.

Basic specification

The basic specification is simply MODEL LIST. There are no additional specifications.

Operations

- The MODEL LIST command lists only the handles created in the current working session.

MODEL NAME

```
MODEL NAME [model name] ['model label']
```

Example

```
MODEL NAME PLOTA1 'PLOT OF THE OBSERVED SERIES'.
```

Overview

MODEL NAME specifies a model name and label for the next procedure in the session.

Basic Specification

The specification on MODEL NAME is a name, a label, or both.

- The default model name is *MOD_n*, where *n* increments by 1 each time an unnamed model is created. This default is in effect if it is not changed on the MODEL NAME command, or if the command is not specified. There is no default label.

Syntax Rules

- If both a name and label are specified, the name must be specified first.
- Only one model name and label can be specified on the command.
- The model name must be unique. The name can contain up to eight characters and must begin with a letter (*A*–*Z*).
- The model label can contain up to 60 characters and must be specified in quotes.

Operations

- MODEL NAME is executed at the next model-generating procedure.
- If the MODEL NAME command is used more than once before a procedure, the last command is in effect.
- If a duplicate model name is specified, the default *MOD_n* name will be used instead.
- *MOD_n* reinitializes at the start of every session and when the READ MODEL command is specified (see READ MODEL). If any models in the active dataset are already named *MOD_n*, those numbers are skipped when new *MOD_n* names are assigned.
- The following procedures can generate models that can be named with the MODEL NAME command: AREG, ARIMA, EXSMOOTH, SEASON, and SPECTRA in the Forecasting add-on module; ACF, CASEPLOT, CCF, CURVEFIT, PACF, PPLOT, and TSPLOT in the Core system; and WLS and 2SLS in the Regression add-on module.

Example

```
MODEL NAME CURVE1 'First CURVEFIT model'.
CURVEFIT Y1.
CURVEFIT Y2.
CURVEFIT Y3
   /APPLY 'CURVE1'.
```

- In this example, the model name *CURVE1* and the label *First CURVEFIT model* are assigned to the first CURVEFIT command.
- The second CURVEFIT command has no MODEL NAME command before it, so it is assigned the name *MOD_n*, where *n* is the next unused integer in the sequence.

- The third CURVEFIT command applies the model named *CURVE1* to the series *Y3*. This model is named *MOD_m*, where *m* = *n* + 1.

MRSETS

```
MRSETS

 /MDGROUP NAME= setname  {LABEL= 'label'      }
                         {LABELSOURCE=VARLABEL}
  CATEGORYLABELS={VARLABELS    }
                 {COUNTEDVALUES}
  VARIABLES= varlist
  VALUE= {value  }
         {'chars'}

 /MCGROUP NAME= setname  LABEL= 'label'
    VARIABLES= varlist

 /DELETE NAME= {[setlist]}
               {ALL      }

 /DISPLAY NAME= {[setlist]}
                {ALL      }
```

The set name must begin with a $ and follow IBM SPSS Statistics variable naming conventions.

Square brackets shown in the DELETE and DISPLAY subcommands are required if one or more set names is specified, but not with the keyword ALL.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 14.0

- LABELSOURCE keyword introduced on MDGROUP subcommand.
- CATEGORYLABELS keyword introduced on MDGROUP subcommand.

Example

```
MRSETS
  /MDGROUP NAME=$mltnews LABEL='News sources'
   VARIABLES=news5 news4 news3 news2 news1
   VALUE=1
  /DISPLAY NAME=[$mltnews].


MRSETS
  /MCGROUP NAME=$mltcars
   LABEL='Car maker, most recent car'
   VARIABLES=car1 car2 car3
  /DISPLAY NAME=[$mltcars].
```

Overview

The MRSETS command defines and manages multiple response sets. The set definitions are saved in the IBM SPSS Statistics data file, so they are available whenever the file is in use. Multiple response sets can be used in the GGRAPH and CTABLES (Tables option) commands. Two types of multiple response sets can be defined:

- Multiple dichotomy (MD) groups combine variables so that each variable becomes a category in the group. For example, take five variables that ask for *yes*/*no* responses to the questions: Do you get news from the Internet? Do you get news from the radio? Do you get news from television? Do you get news from news magazines? Do you get news from newspapers? These variables are coded 1 for *yes* and 0 for *no*. A multiple dichotomy group combines the five variables into a single variable with five categories in which a respondent could be counted zero to five times, depending on how many of the five elementary variables contain a 1 for that respondent. It is not required that the elementary variables be dichotomous. If the five elementary variables had the values 1 for *regularly*, 2 for *occasionally*, and 3 for *never*, it would still be possible to create a multiple dichotomy group that counts the variables with 1’s and ignores the other responses.
- Multiple category (MC) groups combine variables that have identical categories. For example, suppose that instead of having five *yes*/*no* questions for the five news sources, there are three variables, each

coded 1 = *Internet*, 2 = *radio*, 3 = *television*, 4 = *magazines*, and 5 = *newspapers*. For each variable, a respondent could select one of these values. In a multiple category group based on these variables, a respondent could be counted zero to three times, once for each variable for which he or she selected a news source. For this sort of multiple response group, it is important that all of the source variables have the same set of values and value labels and the same missing values.

The MRSETS command also allows you to delete sets and to display information about the sets in the data file.

Syntax Conventions

The following conventions apply to the MRSETS command:

- All subcommands are optional, but at least one must be specified.
- Subcommands can be issued more than once in any order.
- Within a subcommand, attributes can be specified in any order. If an attribute is specified more than once, the last instance is honored.
- Equals signs are required where shown in the syntax diagram.
- Square brackets are required where shown in the syntax diagram.
- The TO convention and the ALL keyword are honored in variable lists.

MDGROUP Subcommand

```
/MDGROUP NAME= setname  {LABEL= 'label'      }
                        {LABELSOURCE=VARLABEL}
  CATEGORYLABELS={VARLABELS    }
                 {COUNTEDVALUES}
  VARIABLES= varlist
  VALUE= {value  }
         {'chars'}
```

The MDGROUP subcommand defines or modifies a multiple dichotomy set. A name, variable list, and value must be specified. Optionally, you can control assignment of set and category labels.

NAME. *The name of the multiple dichotomy set.* The name must follow IBM SPSS Statistics variable naming conventions and begin with a $. If the name refers to an existing set, the set definition is overwritten.

LABEL. *The label for the set.* The label must be quoted and cannot be wider than the limit for variable labels. By default, the set is unlabeled. LABEL and LABELSOURCE are mutually exclusive.

LABELSOURCE. *Use the variable label for the first variable in the set with a defined variable label as the set label.* If none of the variables in the set have defined variable labels, the name of the first variable in the set is used as the set label. LABELSOURCE is an alternative to LABEL an is only available with CATEGORYLABELS=COUNTEDVALUES.

CATEGORYLABELS = [VARLABELS | COUNTEDVALUES]. *Use variable labels or value labels of the counted values as category labels for the set.* VARLABELS uses the defined variable labels (or variable names for variables without defined variable labels) as the set category labels. This is the default. COUNTEDVALUES uses the defined value labels of the counted values as the set category labels. The counted value for each variable must have a defined value label and the labels must be unique (the value label for the counted value must be different for each variable).

VARIABLES. *The list of elementary variables that define the set*. Variables must be of the same type (numeric or string). At least two variables must be specified.

VALUE. *The value that indicates presence of a response.* This is also referred to as the “counted” value. If the set type is numeric, the counted value must be an integer. If the set type is string, the counted value, after trimming trailing blanks, cannot be wider than the narrowest elementary variable.

Elementary variables need not have variable labels, but because variable labels are used as value labels for categories of the MD variable, a warning is issued if two or more variables of an MD set have the same variable label. A warning is also issued if two or more elementary variables use different labels for the counted value—for example, if it is labeled *Yes* for Q1 and *No* for Q2. When checking for label conflicts, case is ignored.

MCGROUP Subcommand

```
/MCGROUP NAME= setname  LABEL= 'label'
    VARIABLES= varlist
```

The MCGROUP subcommand defines or modifies a multiple category group. A name and variable list must be specified. Optionally, a label can be specified for the set.

NAME. *The name of the multiple category set.* The name must follow IBM SPSS Statistics variable naming conventions and begin with a $. If the name refers to an existing set, the set definition is overwritten.

LABEL. *The label for the set.* The label must be quoted and cannot be wider than the limit for variable labels. By default, the set is unlabeled.

VARIABLES. *The list of elementary variables that define the set*. Variables must be of the same type (numeric or string). At least two variables must be specified.

The elementary variables need not have value labels, but a warning is issued if two or more elementary variables have different labels for the same value. When checking for label conflicts, case is ignored.

DELETE Subcommand

```
/DELETE NAME= {[setlist]}
              {ALL      }
```

The DELETE subcommand deletes one or more set definitions. If one or more set names is given, the list must be enclosed in square brackets. ALL can be used to delete all sets; it is not enclosed in brackets.

DISPLAY Subcommand

```
/DISPLAY NAME= {[setlist]}
               {ALL      }
```

The DISPLAY subcommand creates a table of information about one or more sets. If one or more set names is given, the list must be enclosed in square brackets. ALL can be used to refer to all sets; it is not enclosed in brackets.

MULT RESPONSE

MULT RESPONSE is available in the Statistics Base Edition.

```
MULT RESPONSE†

 {/GROUPS=groupname['label'](varlist ({value1,value2}))}
                                      {value        }
      ...[groupname...]

 {/VARIABLES=varlist(min,max)  [varlist...]            }


 {/FREQUENCIES=varlist                                 }

 {/TABLES=varlist BY varlist... [BY varlist] [(PAIRED)]}
        [/varlist BY...]


 [/MISSING=[{TABLE**}] [INCLUDE]]
           {MDGROUP}
           {MRGROUP}

 [/FORMAT={LABELS**}  {TABLE** }  [DOUBLE]]
          {NOLABELS}  {CONDENSE}
                      {ONEPAGE }

 [/BASE={CASES**  }]
        {RESPONSES}

 [/CELLS=[COUNT**] [ROW] [COLUMN] [TOTAL] [ALL]]
```

†A minimum of two subcommands must be used: at least one from the pair GROUPS or VARIABLES and one from the pair FREQUENCIES or TABLES.

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
MULT RESPONSE  GROUPS=MAGS (TIME TO STONE (2))
   /FREQUENCIES=MAGS.
```

Overview

MULT RESPONSE displays frequencies and optional percentages for multiple-response items in univariate tables and multivariate crosstabulations. Another procedure that analyzes multiple-response items is TABLES, which has most, but not all, of the capabilities of MULT RESPONSE. TABLES has special formatting capabilities that make it useful for presentations.

Multiple-response items are questions that can have more than one value for each case. For example, the respondent may have been asked to circle all magazines read within the last month in a list of magazines. You can organize multiple-response data in one of two ways for use in the program. For each possible response, you can create a variable that can have one of two values, such as 1 for *no* and 2 for *yes*; this is the multiple-dichotomy method. Alternatively, you can estimate the maximum number of possible answers from a respondent and create that number of variables, each of which can have a value representing one of the possible answers, such as 1 for *Time*, 2 for *Newsweek*, and 3 for *PC Week*. If an individual did not give the maximum number of answers, the extra variables receive a missing-value code. This is the multiple-response or multiple-category method of coding answers.

To analyze the data entered by either method, you combine variables into groups. The technique depends on whether you have defined multiple-dichotomy or multiple-response variables. When you create a multiple-dichotomy group, each component variable with at least one *yes* value across cases becomes a category of the group variable. When you create a multiple-response group, each value becomes a category and the program calculates the frequency for a particular value by adding the frequencies of all component variables with that value. Both multiple-dichotomy and multiple-response groups can be crosstabulated with other variables in MULT RESPONSE.

Options

Cell Counts and Percentages. By default, crosstabulations include only counts and no percentages. You can request row, column, and total table percentages using the CELLS subcommand. You can also base percentages on responses instead of respondents using BASE.

Format. You can suppress the display of value labels and request condensed format for frequency tables using the FORMAT subcommand.

Basic Specification

The subcommands required for the basic specification fall into two groups: GROUPS and VARIABLES name the elements to be included in the analysis; FREQUENCIES and TABLES specify the type of table display to be used for tabulation. The basic specification requires at least one subcommand from each group:

- GROUPS defines groups of multiple-response items to be analyzed and specifies how the component variables will be combined.
- VARIABLES identifies all individual variables to be analyzed.
- FREQUENCIES requests frequency tables for the groups and/or individual variables specified on GROUPS and VARIABLES.
- TABLES requests crosstabulations of groups and/or individual variables specified on GROUPS and VARIABLES.

Subcommand Order

- The basic subcommands must be used in the following order: GROUPS, VARIABLES, FREQUENCIES, and TABLES. Only one set of basic subcommands can be specified.
- All basic subcommands must precede all optional subcommands. Optional subcommands can be used in any order.

Operations

- Empty categories are not displayed in either frequency tables or crosstabulations.
- If you define a multiple-response group with a very wide range, the tables require substantial amounts of workspace. If the component variables are sparsely distributed, you should recode them to minimize the workspace required.
- MULT RESPONSE stores category labels in the workspace. If there is insufficient space to store the labels after the tables are built, the labels are not displayed.

Limitations

- The component variables must have integer values. Non-integer values are truncated.
- A maximum of 100 existing variables named or implied by GROUPS and VARIABLES together.
- A maximum of 20 groups defined on GROUPS.
- A maximum of 32,767 categories for a multiple-response group or an individual variable.
- A maximum of 10 table lists on TABLES.
- A maximum of 5 dimensions per table.
- A maximum of 100 groups and variables named or implied on FREQUENCIES and TABLES together.
- A maximum of 200 non-empty rows and 200 non-empty columns in a single table.

GROUPS Subcommand

GROUPS defines both multiple-dichotomy and multiple-response groups.

- Specify a name for the group and an optional label, followed by a list of the component variables and the value or values to be used in the tabulation.

- Enclose the variable list in parentheses and enclose the values in an inner set of parentheses following the last variable in the list.
- The label for the group is optional and can be up to 40 characters in length, including imbedded blanks. Quotes around the label are not required.
- To define a multiple-dichotomy group, specify only one tabulating value (the value that represents *yes*) following the variable list. Each component variable becomes a value of the group variable, and the number of cases that have the tabulating value becomes the frequency. If there are no cases with the tabulating value for a given component variable, that variable does not appear in the tabulation.
- To define a multiple-response group, specify two values following the variable list. These are the minimum and maximum values of the component variables. The group variable will have the same range of values. The frequency for each value is tabulated across all component variables in the list.
- You can use any valid variable name for the group except the name of an existing variable specified on the same MULT RESPONSE command. However, you can reuse a group name on another MULT RESPONSE command.
- The group names and labels exist only during MULT RESPONSE and disappear once MULT RESPONSE has been executed. If group names are referred to in other procedures, an error results.
- For a multiple-dichotomy group, the category labels come from the variable labels defined for the component variables.
- For a multiple-response group, the category labels come from the value labels for the first component variable in the group. If categories are missing for the first variable but are present for other variables in the group, you must define value labels for the missing categories. (You can use the ADD VALUE LABELS command to define extra value labels.)

Example

```
MULT RESPONSE  GROUPS=MAGS 'MAGAZINES READ' (TIME TO STONE (2))
   /FREQUENCIES=MAGS.
```

- The GROUPS subcommand creates a multiple-dichotomy group named *MAGS*. The variables between and including *TIME* and *STONE* become categories of *MAGS*, and the frequencies are cases with the value 2 (indicating *yes, read the magazine*) for the component variables.
- The group label is *MAGAZINES READ*.

Example

```
MULT RESPONSE  GROUPS=PROBS 'PERCEIVED NATIONAL PROBLEMS'
   (PROB1 TO PROB3 (1,9))
   /FREQUENCIES=PROBS.
```

- The GROUPS subcommand creates the multiple-response group *PROBS*. The component variables are the existing variables between and including *PROB1* and *PROB3*, and the frequencies are tabulated for the values 1 through 9.
- The frequency for a given value is the number of cases that have that value in any of the variables *PROB1* to *PROB3*.

VARIABLES Subcommand

VARIABLES specifies existing variables to be used in frequency tables and crosstabulations. Each variable is followed by parentheses enclosing a minimum and a maximum value, which are used to allocate cells for the tables for that variable.

- You can specify any numeric variable on VARIABLES, but non-integer values are truncated.
- If GROUPS is also specified, VARIABLES follows GROUPS.
- To provide the same minimum and maximum for each of a set of variables, specify a variable list followed by a range specification.

- The component variables specified on GROUPS can be used in frequency tables and crosstabulations, but you must specify them again on VARIABLES, along with a range for the values. You do not have to respecify the component variables if they will not be used as individual variables in any tables.

Example

```
MULT RESPONSE  GROUPS=MAGS 'MAGAZINES READ' (TIME TO STONE (2))
   /VARIABLES SEX(1,2) EDUC(1,3)
   /FREQUENCIES=MAGS SEX EDUC.
```

- The VARIABLES subcommand names the variables *SEX* and *EDUC* so that they can be used in a frequencies table.

Example

```
MULT RESPONSE  GROUPS=MAGS 'MAGAZINES READ' (TIME TO STONE (2))
   /VARIABLES=EDUC (1,3) TIME (1,2).
   /TABLES=MAGS BY EDUC TIME.
```

- The variable *TIME* is used in a group and also in a table.

FREQUENCIES Subcommand

FREQUENCIES requests frequency tables for groups and individual variables. By default, a frequency table contains the count for each value, the percentage of responses, and the percentage of cases. For another method of producing frequency tables for individual variables, see the FREQUENCIES procedure.

- All groups must be created by GROUPS, and all individual variables to be tabulated must be named on VARIABLES.
- You can use the keyword TO to imply a set of group or individual variables. TO refers to the order in which variables are specified on the GROUPS or VARIABLES subcommand.

Example

```
MULT RESPONSE  GROUPS=MAGS 'MAGAZINES READ' (TIME TO STONE (2))
   /FREQUENCIES=MAGS.
```

- The FREQUENCIES subcommand requests a frequency table for the multiple-dichotomy group *MAGS*, tabulating the frequency of the value 2 for each of the component variables *TIME* to *STONE*.

Example

```
MULT RESPONSE
 GROUPS=MAGS 'MAGAZINES READ' (TIME TO STONE (2))
      PROBS 'PERCEIVED NATIONAL PROBLEMS' (PROB1 TO PROB3 (1,9))
      MEMS 'SOCIAL ORGANIZATION MEMBERSHIPS' (VFW AMLEG ELKS (1))
 /VARIABLES SEX(1,2) EDUC(1,3)
 /FREQUENCIES=MAGS TO MEMS SEX EDUC.
```

- The FREQUENCIES subcommand requests frequency tables for *MAGS, PROBS, MEMS, SEX*, and *EDUC*.
- You cannot specify MAGS TO EDUC because *SEX* and *EDUC* are individual variables, and *MAGS*, *PROBS*, and *MEMS* are group variables.

TABLES Subcommand

TABLES specifies the crosstabulations to be produced by MULT RESPONSE. Both individual variables and group variables can be tabulated together.

- The first list defines the rows of the tables; the next list (following BY) defines the columns. Subsequent lists following BY keywords define control variables, which produce subtables. Use the keyword BY to separate the dimensions. You can specify up to five dimensions (four BY keywords) for a table.
- To produce more than one table, name one or more variables for each dimension of the tables. You can also specify multiple table lists separated by a slash. If you use the keyword TO to imply a set of group

or individual variables, TO refers to the order in which groups or variables are specified on the GROUPS or VARIABLES subcommand.

- If FREQUENCIES is also specified, TABLES follows FREQUENCIES.
- The value labels for columns are displayed on three lines with eight characters per line. To avoid splitting words, reverse the row and column variables, or redefine the variable or value labels (depending on whether the variables are multiple-dichotomy or multiple-response variables).

Example

```
MULT RESPONSE  GROUPS=MAGS 'MAGAZINES READ' (TIME TO STONE (2))
   /VARIABLES=EDUC (1,3)/TABLES=EDUC BY MAGS.
```

- The TABLES subcommand requests a crosstabulation of variable *EDUC* by the multiple-dichotomy group *MAGS*.

Example

```
MULT RESPONSE  GROUPS=MAGS 'MAGAZINES READ' (TIME TO STONE (2))
   MEMS 'SOCIAL ORGANIZATION MEMBERSHIPS' (VFW AMLEG ELKS (1))
   /VARIABLES EDUC (1,3)/TABLES=MEMS MAGS BY EDUC.
```

- The TABLES subcommand specifies two crosstabulations—*MEMS* by *EDUC* and *MAGS* by *EDUC*.

Example

```
MULT RESPONSE  GROUPS=MAGS 'MAGAZINES READ' (TIME TO STONE (2))
   /VARIABLES SEX (1,2) EDUC (1,3)
   /TABLES=MAGS BY EDUC SEX/EDUC BY SEX/MAGS BY EDUC BY SEX.
```

- The TABLES subcommand uses slashes to separate three table lists. It produces two tables from the first table list (*MAGS* by *EDUC* and *MAGS* by *SEX*) and one table from the second table list (*EDUC* by *SEX*). The third table list produces separate tables for each sex (*MAGS* by *EDUC* for male and for female).

Example

```
MULT RESPONSE  GROUPS=MAGS 'MAGAZINES READ' (TIME TO STONE (2))
   PROBS 'NATIONAL PROBLEMS MENTIONED' (PROB1 TO PROB3 (1,9))
   /TABLES=MAGS BY PROBS.
```

- The TABLES subcommand requests a crosstabulation of the multiple-dichotomy group *MAGS* with the multiple-response group *PROBS*.

PAIRED Keyword

When MULT RESPONSE crosstabulates two multiple-response groups, by default it tabulates each variable in the first group with each variable in the second group and sums the counts for each cell. Thus, some responses can appear more than once in the table. Use PAIRED to pair the first variable in the first group with the first variable in the second group, the second variable in the first group with the second variable in the second group, and so on.

- The keyword PAIRED is specified in parentheses on the TABLES subcommand following the last variable named for a specific table list.
- When you request paired crosstabulations, the order of the component variables on the GROUPS subcommand determines the construction of the table.
- Although the tables can contain individual variables and multiple-dichotomy groups in a paired table request, only variables within multiple-response groups are paired.
- PAIRED also applies to a multiple-response group used as a control variable in a three-way or higher-order table.
- Paired tables are identified in the output by the label *PAIRED GROUP*.
- Percentages in paired tables are always based on responses rather than cases.

Example

```
MULT RESPONSE GROUPS=PSEX 'SEX OF CHILD'(P1SEX P2SEX P3SEX (1,2))
   /PAGE 'AGE OF ONSET OF PREGNANCY' (P1AGE P2AGE P3AGE (1,4))
   /TABLES=PSEX BY PAGE (PAIRED).
```

- The PAIRED keyword produces a paired crosstabulation of *PSEX* by *PAGE*, which is a combination of the tables *P1SEX* by *P1AGE*, *P2SEX* by *P2AGE*, and *P3SEX* by *P3AGE*.

Example

```
MULT RESPONSE GROUPS=PSEX 'SEX OF CHILD'(P1SEX P2SEX P3SEX (1,2))
   PAGE 'AGE OF ONSET OF PREGNANCY' (P1AGE P2AGE P3AGE (1,4))
   /VARIABLES=EDUC (1,3)
   /TABLES=PSEX BY PAGE BY EDUC (PAIRED).
```

- The TABLES subcommand pairs only *PSEX* with *PAGE*. *EDUC* is not paired because it is an individual variable, not a multiple-response group.

CELLS Subcommand

By default, MULT RESPONSE displays cell counts but not percentages in crosstabulations. CELLS requests percentages for crosstabulations.

- If you specify one or more keywords on CELLS, MULT RESPONSE displays cell counts plus the percentages you request. The count cannot be eliminated from the table cells.

COUNT. *Cell counts.* This is the default if you omit the CELLS subcommand.

ROW. *Row percentages.*

COLUMN. *Column percentages.*

TOTAL. *Two-way table total percentages.*

ALL. *Cell counts, row percentages, column percentages, and two-way table total percentages.* This is the default if you specify the CELLS subcommand without keywords.

Example

```
MULT RESPONSE  GROUPS=MAGS 'MAGAZINES READ' (TIME TO STONE (2))
  /VARIABLES=SEX (1,2) (EDUC (1,3)
  /TABLES=MAGS BY EDUC SEX
  /CELLS=ROW COLUMN.
```

- The CELLS subcommand requests row and column percentages in addition to counts.

BASE Subcommand

BASE lets you obtain cell percentages and marginal frequencies based on responses rather than respondents. Specify one of two keywords:

CASES. *Base cell percentages on cases.* This is the default if you omit the BASE subcommand and do not request paired tables. You cannot use this specification if you specify PAIRED on TABLE.

RESPONSES. *Base cell percentages on responses.* This is the default if you request paired tables.

Example

```
MULT RESPONSE  GROUPS=PROBS 'NATIONAL PROBLEMS MENTIONED'
   (PROB1 TO PROB3 (1,9))/VARIABLES=EDUC (1,3)
   /TABLES=EDUC BY PROBS
   /CELLS=ROW COLUMN
   /BASE=RESPONSES.
```

- The BASE subcommand requests marginal frequencies and cell percentages based on responses.

MISSING Subcommand

MISSING controls missing values. Its minimum specification is a single keyword.

- By default, MULT RESPONSE deletes cases with missing values on a table-by-table basis for both individual variables and groups. In addition, values falling outside the specified range are not tabulated and are included in the missing category. Thus, specifying a range that excludes missing values is equivalent to the default missing-value treatment.
- For a multiple-dichotomy group, a case is considered missing by default if none of the component variables contains the tabulating value for that case. The keyword MDGROUP overrides the default and specifies listwise deletion for multiple-dichotomy groups.
- For a multiple-response group, a case is considered missing by default if none of the components has valid values falling within the tabulating range for that case. Thus, cases with missing or excluded values on some (but not all) of the components of a group are included in tabulations of the group variable. The keyword MRGROUP overrides the default and specifies listwise deletion for multiple-response groups.
- You can use INCLUDE with MDGROUP, MRGROUP, or TABLE. The user-missing value is tabulated if it is included in the range specification.

TABLE. *Exclude missing values on a table-by-table basis.* Missing values are excluded on a table-by-table basis for both component variables and groups. This is the default if you omit the MISSING subcommand.

MDGROUP. *Exclude missing values listwise for multiple-dichotomy groups.* Cases with missing values for any component dichotomy variable are excluded from the tabulation of the multiple-dichotomy group.

MRGROUP. *Exclude missing values listwise for multiple-response groups.* Cases with missing values for any component variable are excluded from the tabulation of the multiple-response group.

INCLUDE. *Include user-missing values*. User-missing values are treated as valid values if they are included in the range specification on the GROUPS or VARIABLES subcommands.

Example

```
MULT RESPONSE  GROUPS=FINANCL 'FINANCIAL PROBLEMS MENTIONED'
   (FINPROB1 TO FINPROB3 (1,3))
   SOCIAL 'SOCIAL PROBLEMS MENTIONED'(SOCPROB1 TO SOCPROB4 (4,9))
   /VARIABLES=EDUC (1,3)
   /TABLES=EDUC BY FINANCL SOCIAL
   /MISSING=MRGROUP.
```

- The MISSING subcommand indicates that a case will be excluded from counts in the first table if any of the variables in the group *FINPROB1* to *FINPROB3* has a missing value or a value outside the range 1 to 3. A case is excluded from the second table if any of the variables in the group *SOCPROB1* to *SOCPROB4* has a missing value or value outside the range 4 to 9.

FORMAT Subcommand

FORMAT controls table formats. The minimum specification on FORMAT is a single keyword.

Labels are controlled by two keywords:

LABELS. *Display value labels in frequency tables and crosstabulations.* This is the default.

NOLABELS. *Suppress value labels in frequency tables and crosstabulations for multiple-response variables and individual variables.* You cannot suppress the display of variable labels used as value labels for multiple-dichotomy groups.

The following keywords apply to the format of frequency tables:

DOUBLE. *Double spacing for frequency tables.* By default, MULT RESPONSE uses single spacing.

TABLE. *One-column format for frequency tables.* This is the default if you omit the FORMAT subcommand.

CONDENSE. *Condensed format for frequency tables.* This option uses a three-column condensed format for frequency tables for all multiple-response groups and individual variables. Labels are suppressed. This option does not apply to multiple-dichotomy groups.

ONEPAGE. *Conditional condensed format for frequency tables.* Three-column condensed format is used if the resulting table would not fit on a page. This option does not apply to multiple-dichotomy groups.

Example

```
MULT RESPONSE  GROUPS=PROBS 'NATIONAL PROBLEMS MENTIONED'
   (PROB1 TO PROB3 (1,9))/VARIABLES=EDUC (1,3)
   /FREQUENCIES=EDUC PROBS
   /FORMAT=CONDENSE.
```

- The FORMAT subcommand specifies condensed format, which eliminates category labels and displays the categories in three parallel sets of columns, each set containing one or more rows of categories (rather than displaying one set of columns aligned vertically down the page).

MULTIPLE CORRESPONDENCE

MULTIPLE CORRESPONDENCE is available in SPSS Statistics Professional Edition or the Categories option.

```
MULTIPLE CORRESPONDENCE [/VARIABLES =] varlist

  /ANALYSIS = varlist [([WEIGHT={1**}]
                        {n  }

 [/DISCRETIZATION = [varlist [([{GROUPING   }] [{NCAT={7}  }] [DISTR={NORMAL }])]]]
                                              {n}             {UNIFORM}
                                          {EQINTV={n}}
                        {RANKING    }
                        {MULTIPLYING}

 [/MISSING = [{varlist} ([{PASSIVE**}] [{MODEIMPU**}])]]
              {ALL**  }                {EXTRACAT  }
                       {ACTIVE   }     {MODEIMPU**}
                                       {EXTRACAT  }
                                       {LISTWISE  }

 [/SUPPLEMENTARY = [OBJECT(objlist)] [VARIABLE(varlist)]]

 [/CONFIGURATION = [{INITIAL}]('filename')]
                   {FIXED  }

 [/DIMENSION = {2**}]
               {n  }

 [/NORMALIZATION = {VPRINCIPAL**}]
                   {OPRINCIPAL  }
                   {SYMMETRICAL }
                   {INDEPENDENT }
                   {n           }

 [/MAXITER = {100**}]
             {n    }

 [/CRITITER = {.00001**}]
              {value   }

 [/PRINT = [CORR**] [DESCRIP**[(varlist)]] [HISTORY][DISCRIM**] [NONE]
           [OBJECT[([(varname)]varlist)]] [OCORR] [QUANT[(varlist)]]

 [/PLOT = [BIPLOT[((varlist))][(varlist)][(n)]]
          [CATEGORY (varlist)[(n)]]
          [JOINTCAT[(varlist)][(n)]]
          [DISCRIM**[(varlist)][(n)]]
          [NONE]
          [OBJECT**[(varlist)][(n)]]
          [RESID(varlist[({1**})])[(n)]]
                        {n  }
          [TRANS(varlist[({1**})])[(n)]]
                        {n  }
          [NDIM(value,value)]

 [/SAVE = [TRDATA[({TRA     }[(n)])]] [OBJECT[({OBSCO   }[(n)])]] ]
                  {rootname}                {rootname}

[/OUTFILE = {DISCRDATA('filename')} {OBJECT('filename')} {TRDATA('filename')}]
```

** Default if subcommand is omitted

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 13.0

- Command introduced.

Overview

MULTIPLE CORRESPONDENCE (Multiple Correspondence Analysis; also known as homogeneity analysis) quantifies nominal (categorical) data by assigning numerical values to the cases (objects) and categories, such that in the low-dimensional representation of the data, objects within the same category are close together and objects in different categories are far apart. Each object is as close as possible to the category points of categories that apply to the object. In this way, the categories divide the objects into homogeneous subgroups. Variables are considered homogeneous when they classify objects in the same categories into the same subgroups.

Basic Specification

The basic specification is the command MULTIPLE CORRESPONDENCE with the VARIABLES and ANALYSIS subcommands.

Syntax Rules

- The VARIABLES and ANALYSIS subcommands always must appear.
- All subcommands can appear in any order.
- For the first subcommand after the procedure name, a slash is accepted, but not required.
- Variables specified in the ANALYSIS subcommand must be found in the VARIABLES subcommand.
- Variables specified in the SUPPLEMENTARY subcommand must be found in the ANALYSIS subcommand.

Operations

- If the same subcommand is repeated, it causes a syntax error and the procedure terminates.

Limitations

- MULTIPLE CORRESPONDENCE operates on category indicator variables. The category indicators should be positive integers. You can use the DISCRETIZATION subcommand to convert fractional value variables and string variables into positive integers. If DISCRETIZATION is not specified, fractional value variables are automatically converted into positive integers by grouping them into seven categories (or into the number of distinct values of the variable if this number is less than seven) with a close-to-normal distribution, and string variables are automatically converted into positive integers by ranking.
- In addition to system-missing values and user-defined missing values, MULTIPLE CORRESPONDENCE treats category indicator values less than 1 as missing. If one of the values of a categorical variable has been coded 0 or some negative value and you want to treat it as a valid category, use the COMPUTE command to add a constant to the values of that variable such that the lowest value will be 1. You can also use the RANKING option of the DISCRETIZATION subcommand for this purpose, except for variables you want to treat as numerical, since the spacing of the categories will not be maintained.
- There must be at least three valid cases.
- Split-File has no implications for MULTIPLE CORRESPONDENCE.

Example

```
MULTIPLE CORRESPONDENCE /VARIABLES = TEST1 TEST2 TEST3 TO TEST6 TEST7 TEST8
  /ANALYSIS = TEST1 TO TEST2(WEIGHT=2)
              TEST3 TO TEST5
              TEST6 TEST7
              TEST8
  /DISCRETIZATION = TEST1(GROUPING NCAT=5 DISTR=UNIFORM)
                    TEST6(GROUPING) TEST8(MULTIPLYING)
  /MISSING = TEST5(ACTIVE) TEST6(ACTIVE EXTRACAT) TEST8(LISTWISE)
  /SUPPLEMENTARY = OBJECT(1 3) VARIABLE(TEST1)
  /CONFIGURATION = ('iniconf.sav')
  /DIMENSION = 2
  /NORMALIZATION = VPRINCIPAL
  /MAXITER = 150
  /CRITITER = .000001
  /PRINT =  DESCRIP DISCRIM CORR QUANT(TEST1 TO TEST3) OBJECT
  /PLOT = TRANS(TEST2 TO TEST5) OBJECT(TEST2 TEST3)
  /SAVE = TRDATA OBJECT
  /OUTFILE = TRDATA('/data/trans.sav')  OBJECT('/data/obs.sav').
```

- VARIABLES defines variables. The keyword TO refers to the order of the variables in the working data file.
- The ANALYSIS subcommand defines variables used in the analysis. It is specified that *TEST1* and *TEST2* have a weight of 2 (for the other variables, WEIGHT is not specified and thus they have the default weight value of 1).
- DISCRETIZATION specifies that *TEST6* and *TEST8*, which are fractional value variables, are discretized: *TEST6* by recoding into seven categories with a normal distribution (default because unspecified) and

TEST8 by “multiplying”. *TEST1*, which is a categorical variable, is recoded into five categories with a close to uniform distribution.

- MISSING specifies that objects with missing values on *TEST5* and *TEST6* are included in the analysis: missing values on *TEST5* are replaced with the mode (default if not specified) and missing values on *TEST6* are treated as an extra category. Objects with a missing value on *TEST8* are excluded from the analysis. For all other variables, the default is in effect; that is, missing values (not objects) are excluded from the analysis.
- CONFIGURATION specifies *iniconf.sav* as the file containing the coordinates of a configuration that is to be used as the initial configuration (default because unspecified).
- DIMENSION specifies the number of dimensions to be 2. This is the default, so this subcommand could be omitted here.
- The NORMALIZATION subcommand specifies optimization of the association between variables. This is the default, so this subcommand could be omitted here.
- MAXITER specifies the maximum number of iterations to be 150 instead of the default value of 100.
- CRITITER sets the convergence criterion to a value smaller than the default value.
- PRINT specifies descriptives, discrimination measures, and correlations (all default), and quantifications for *TEST1* to *TEST3*, and the object scores.
- PLOT is used to request transformation plots for the variables *TEST2* to *TEST5*, an object points plot labeled with the categories of *TEST2*, and an object points plot labeled with the categories of *TEST3*.
- The SAVE subcommand adds the transformed variables and the object scores to the working data file.
- The OUTFILE subcommand writes the transformed data to a data file called *trans.sav* and the object scores to a data file called *obs.sav*, both in the directory */data*.

Options

Discretization. You can use the DISCRETIZATION subcommand to discretize fractional value variables or to recode categorical variables.

Missing data. You can specify the treatment of missing data per variable with the MISSING subcommand.

Supplementary objects and variables. You can specify objects and variables that you want to treat as supplementary.

Read configuration. MULTIPLE CORRESPONDENCE can read a configuration from a file through the CONFIGURATION subcommand. This configuration can be used as the initial configuration or as a fixed configuration in which to fit variables.

Number of dimensions. You can specify how many dimensions MULTIPLE CORRESPONDENCE should compute.

Normalization. You can specify one of five different options for normalizing the objects and variables.

Tuning the algorithm. You can control the values of algorithm-tuning parameters with the MAXITER and CRITITER subcommands.

Optional output. You can request optional output through the PRINT subcommand.

Optional plots. You can request a plot of object points, transformation plots per variable, plots of category points per variable, or a joint plot of category points for specified variables. Other plot options include residuals plots, a biplot, and a plot of discrimination measures.

Writing discretized data, transformed data, and object scores. You can write the discretized data, the transformed data, and the object scores to outfiles for use in further analyses.

Saving transformed data and object scores. You can save the transformed variables and the object scores in the working data file.

VARIABLES Subcommand

VARIABLES specifies the variables that may be analyzed in the current MULTIPLE CORRESPONDENCE procedure.

- The VARIABLES subcommand is required. The actual keyword VARIABLES can be omitted.
- At least two variables must be specified, except if the CONFIGURATION subcommand with the FIXED keyword is used.
- The keyword TO on the VARIABLES subcommand refers to the order of variables in the working data file. (Note that this behavior of TO is different from that in the varlist in the ANALYSIS subcommand.)

ANALYSIS Subcommand

ANALYSIS specifies the variables to be used in the computations, and the variable weight for each variable or variable list. ANALYSIS also specifies supplementary variables; no weight can be specified for supplementary variables.

- At least two variables must be specified, except if the CONFIGURATION subcommand with the FIXED keyword is used.
- All the variables on ANALYSIS must be specified on the VARIABLES subcommand.
- The ANALYSIS subcommand is required.
- The keyword TO in the variable list honors the order of variables in the VARIABLES subcommand.
- Variable weights are indicated by the keyword WEIGHT in parentheses following the variable or variable list.

WEIGHT. *Specifies the variable weight.* The default value is 1. If WEIGHT is specified for supplementary variables, this is ignored (but with a syntax warning).

DISCRETIZATION Subcommand

DISCRETIZATION specifies fractional value variables you want to discretize. Also, you can use DISCRETIZATION for ranking or for two ways of recoding categorical variables.

- A string variable's values are always converted into positive integers, by assigning category indicators according to the ascending alphanumeric order. DISCRETIZATION for string variables applies to these integers.
- When the DISCRETIZATION subcommand is omitted, or when the DISCRETIZATION subcommand is used without a varlist, fractional value variables are converted into positive integers by grouping them into seven categories (or into the number of distinct values of the variable if this number is less than seven) with a close-to-normal distribution.
- When no specification is given for variables in a varlist following DISCRETIZATION, these variables are grouped into seven categories (or into the number of distinct values of the variable if this number is less than seven) with a close-to-normal distribution.
- In MULTIPLE CORRESPONDENCE a system-missing value, user-defined missing values, and values less than 1 are considered to be missing values (see next section). However, in discretizing a variable, values less than 1 are considered to be valid values and are thus included in the discretization process. System-missing values and user-defined missing values are excluded.

GROUPING. *Recode into the specified number of categories or recode intervals of equal size into categories.*

RANKING. *Rank cases.* Rank 1 is assigned to the case with the smallest value on the variable.

MULTIPLYING. *Multiplying the standardized values (z-scores) of a fractional value variable by 10, rounding, and adding a value such that the lowest value is 1.*

GROUPING Keyword

NCAT. *Recode into ncat categories.* When NCAT is not specified, the number of categories is set to seven (or the number of distinct values of the variable if this number is less than seven).

EQINTV. *Recode intervals of equal size into categories.* The interval size must be specified (there is no default value). The resulting number of categories depends on the interval size.

NCAT Keyword

NCAT has the keyword DISTR, which has the following keywords:

NORMAL. *Normal distribution.* This is the default when DISTR is not specified.

UNIFORM. *Uniform distribution.*

MISSING Subcommand

In MULTIPLE CORRESPONDENCE, system-missing values, user-defined missing values, and values less than 1 are treated as missing values. However, in discretizing a variable, values less than 1 are considered as valid values. The MISSING subcommand allows you to indicate how to handle missing values for each variable.

PASSIVE. *Exclude missing values on a variable from analysis.* This is the default applicable to all variables, when the MISSING subcommand is omitted or specified without variable names or keywords. Also, any variable which is not included in the subcommand gets this specification. Passive treatment of missing values means that, in optimizing the quantification of a variable, only objects with non-missing values on the variable are involved and that only the non-missing values of variables contribute to the solution. Thus, when PASSIVE is specified, missing values do not affect the analysis. If an object has only missing values, and for all variables the MISSING option is passive, the object will be handled as a supplementary object. If on the PRINT subcommand, correlations are requested and passive treatment of missing values is specified for a variable, the missing values have to be imputed. For the correlations of the original variables, missing values on a variable are imputed with the most frequent category (mode) of the variable.

ACTIVE. *Impute missing values.* You can choose to use mode imputation, or to consider objects with missing values on a variable as belonging to the same category and impute missing values with an extra category indicator.

LISTWISE. *Exclude cases with missing values on the specified variable(s).* The cases used in the analysis are cases without missing values on the variable(s) specified. Also, any variable that is not included in the subcommand gets this specification.

- The ALL keyword may be used to indicate all variables. If it is used, it must be the only variable specification.
- A mode or extracat imputation is done before listwise deletion.

PASSIVE Keyword

MODEIMPU. *Impute missing values on a variable with the mode of the quantified variable.* This is the default.

EXTRACAT. *Impute missing values on a variable with the quantification of an extra category.* This implies that objects with a missing value are considered to belong to the same (extra) category.

Note: With passive treatment of missing values, imputation only applies to correlations and is done afterwards. Thus the imputation has no effect on the quantification or the solution.

ACTIVE Keyword

MODEIMPU. *Impute missing values on a variable with the most frequent category (mode)*. When there are multiple modes, the smallest category indicator is used. This is the default.

EXTRACAT. *Impute missing values on a variable with an extra category indicator.* This implies that objects with a missing value are considered to belong to the same (extra) category.

Note: With active treatment of missing values, imputation is done before the analysis starts, and thus will affect the quantification and the solution.

SUPPLEMENTARY Subcommand

The SUPPLEMENTARY subcommand specifies the objects or/and variables that you want to treat as supplementary. Supplementary variables must be found in the ANALYSIS subcommand. You can not weight supplementary objects and variables (specified weights are ignored). For supplementary variables, all options on the MISSING subcommand can be specified except LISTWISE.

OBJECT. *Objects you want to treat as supplementary are indicated with an object number list in parentheses following OBJECT.* The keyword TO is allowed. The OBJECT specification is not allowed when CONFIGURATION = FIXED.

VARIABLE. *Variables you want to treat as supplementary are indicated with a variable list in parentheses following VARIABLE.* The keyword TO is allowed and honors the order of variables in the VARIABLES subcommand. The VARIABLE specification is ignored when CONFIGURATION = FIXED, for in that case all the variables in the ANALYSIS subcommand are automatically treated as supplementary variables.

CONFIGURATION Subcommand

The CONFIGURATION subcommand allows you to read data from a file containing the coordinates of a configuration. The first variable in this file should contain the coordinates for the first dimension, the second variable should contain the coordinates for the second dimension, and so forth.

INITIAL('filename'). Use the configuration in the specified file as the starting point of the analysis.

FIXED('filename'). *Fit variables in the fixed configuration found in the specified file.* The variables to fit in should be specified on the ANALYSIS subcommand but will be treated as supplementary variables. The SUPPLEMENTARY subcommand will be ignored. Also, variable weights will be ignored.

DIMENSION Subcommand

DIMENSION specifies the number of dimensions you want MULTIPLE CORRESPONDENCE to compute.

- If you do not specify the DIMENSION subcommand, MULTIPLE CORRESPONDENCE computes a two dimensional solution.
- DIMENSION is followed by an integer indicating the number of dimensions.
- The maximum number of dimensions is the smaller of a) the number of observations minus 1 and b) the total number of valid variable levels (categories) minus the number of variables if there are no variables with missing values to be treated as passive. If there are variables with missing values to be treated as passive, the maximum number of dimensions is the smaller of a) the number of observations minus 1 and b) the total number of valid variable levels (categories) minus the larger of c) 1 and d) the number of variables without missing values to be treated as passive.
- The maximum number of dimensions is the smaller of the number of observations minus 1 and the total number of valid variable levels (categories) minus the number of variables without missing values.
- MULTIPLE CORRESPONDENCE adjusts the number of dimensions to the maximum if the specified value is too large.
- The minimum number of dimensions is 1.

NORMALIZATION Subcommand

The NORMALIZATION subcommand specifies one of five options for normalizing the object scores and the variables.

- Only one normalization method can be used in a given analysis.

VPRINCIPAL. *Optimize the association between variables.* With VPRINCIPAL, the categories are in the centroid of the objects in the particular categories. VPRINCIPAL is the default if the NORMALIZATION subcommand is not specified. This is useful when you are primarily interested in the association between the variables.

OPRINCIPAL. *Optimize distances between objects.* This is useful when you are primarily interested in differences or similarities between the objects.

SYMMETRICAL. *Use this normalization option if you are primarily interested in the relation between objects and variables.*

INDEPENDENT. *Use this normalization option if you want to examine distances between objects and associations between variables separately.*

The fifth method allows the user to specify any real value in the closed interval [–1, 1]. A value of 1 is equal to the OPRINCIPAL method, a value of 0 is equal to the SYMMETRICAL method, and a value of –1 is equal to the VPRINCIPAL method. By specifying a value greater than –1 and less than 1, the user can spread the eigenvalue over both objects and variables. This method is useful for making a tailor-made biplot. If the user specifies a value outside of this interval, the procedure issues a syntax error message and terminates.

MAXITER Subcommand

MAXITER specifies the maximum number of iterations MULTIPLE CORRESPONDENCE can go through in its computations.

- If MAXITER is not specified, the maximum number of iterations is 100.
- The specification on MAXITER is a positive integer indicating the maximum number of iterations. There is no uniquely predetermined (that is, hard-coded) maximum for the value that can be used.

CRITITER Subcommand

CRITITER specifies a convergence criterion value. MULTIPLE CORRESPONDENCE stops iterating if the difference in fit between the last two iterations is less than the CRITITER value.

- If CRITITER is not specified, the convergence value is 0.00001.
- The specification on CRITITER is any positive value.

PRINT Subcommand

The Model Summary statistics (Cronbach's alpha and the variance accounted for) and the HISTORY statistics (the variance accounted for, the loss, and the increase in variance accounted for) for the last iteration are always displayed. That is, they cannot be controlled by the PRINT subcommand. The PRINT subcommand controls the display of optional additional output. The output of the MULTIPLE CORRESPONDENCE procedure is always based on the transformed variables. However, the correlations of the original variables can be requested as well by the keyword OCORR.

The default keywords are DESCRIP, DISCRIM, and CORR. That is, the three keywords are in effect when the PRINT subcommand is omitted or when the PRINT subcommand is given without any keywords. Note that when some keywords are specified, the default is nullified and only the keywords specified become

in effect. If a keyword that cannot be followed by a varlist is duplicated or if a contradicting keyword is encountered, then the later one silently becomes effective (in case of a contradicting use of NONE, only the keywords following NONE are effective). For example,

```
/PRINT <=> /PRINT = DESCRIP DISCRIM CORR

/PRINT = DISCRIM DISCRIM <=> /PRINT = DISCRIM

/PRINT = DISCRIM NONE CORR <=> /PRINT = CORR
```

If a keyword that can be followed by a varlist is duplicated, it will cause a syntax error and the procedure will terminate. For example, /PRINT = QUANT QUANT is a syntax error.

The following keywords can be specified:

DESCRIP(varlist). *Descriptive statistics (frequencies, missing values, and mode).* The variables in the varlist must be specified on the VARIABLES subcommand, but need not appear on the ANALYSIS subcommand. If DESCRIP is not followed by a varlist, Descriptives tables are displayed for all the variables in the varlist on the ANALYSIS subcommand.

DISCRIM. *Discrimination measures per variable and per dimension.*

QUANT(varlist). *Category quantifications (centroid coordinates), mass, inertia of the categories, contribution of the categories to the inertia of the dimensions, and contribution of the dimensions to the inertia of the categories.* Any variable in the ANALYSIS subcommand may be specified in parentheses after QUANT. If QUANT is not followed by a varlist, Quantification tables are displayed for all variables in the varlist on the ANALYSIS subcommand. *Note:* the coordinates and contributions (including the mass and inertia) are displayed in separate layers of the pivot table output, with the coordinates shown by default. To display the contributions, activate (double-click) on the table and select Contributions from the Layer dropdown list.

HISTORY. *History of iterations.* For each iteration, the variance accounted for, the loss, and the increase in variance accounted for are shown.

CORR. *Correlations of the transformed variables, and the eigenvalues of this correlation matrix.* Correlation tables are displayed for each set of quantifications, thus there are *ndim* (the number of dimensions in the analysis) correlation tables; the *i*th table contains the correlations of the quantifications of dimension *i*, *i* = 1, ..., *ndim*. For variables with missing values specified to be treated as PASSIVE on the MISSING subcommand, the missing values are imputed according to the specification on the PASSIVE keyword (if nothing is specified, mode imputation is used).

OCORR. *Correlations of the original variables, and the eigenvalues of this correlation matrix.* For variables with missing values specified to be treated as PASSIVE on the MISSING subcommand, the missing values are imputed with the variable mode.

OBJECT((varname)varlist). *Object scores (component scores) and, in separate table, mass, inertia of the objects, contribution of the objects to the inertia of the dimensions, and contribution of the dimensions to the inertia of the objects.* Following the keyword, a varlist can be given in parentheses to display variables (category indicators) along with the object scores. If you want to use a variable to label the objects, this variable must occur in parenthesis as the first variable in the varlist. If no labeling variable is specified, the objects are labeled with case numbers. The variables to display along with the object scores and the variable to label the objects must be specified on the VARIABLES subcommand but need not appear on the ANALYSIS subcommand. If no varlist is given, only the object scores are displayed.

NONE. *No optional output is displayed.* The only output shown is the Model Summary and the HISTORY statistics for the last iteration.

The keyword TO in a variable list can only be used with variables that are in the ANALYSIS subcommand, and TO applies only to the order of the variables in the ANALYSIS subcommand. For variables that are in the VARIABLES subcommand but not in the ANALYSIS subcommand, the keyword TO cannot be used. For example, if /VARIABLES = v1 TO v5 and the ANALYSIS subcommand has /ANALYSIS v2 v1 v4, then /PLOT OBJECT(v1 TO v4) will give two object plots, one labeled with *v1* and one labeled with *v4*. (/PLOT OBJECT(v1 TO v4 v2 v3 v5) will give objects plots labeled with *v1*, *v2*, *v3*, *v4*, and *v5*).

PLOT Subcommand

The PLOT subcommand controls the display of plots. The default keywords are OBJECT and DISCRIM. That is, the two keywords are in effect when the PLOT subcommand is omitted, or when the PLOT subcommand is given without any keyword. If a keyword is duplicated (for example, /PLOT = RESID RESID), then it will cause a syntax error and the procedure will terminate. If the keyword NONE is used together with other keywords (for example, /PLOT = RESID NONE DISCRIM), then only the keywords following NONE are effective. That is, when keywords contradict, the later one overwrites the earlier ones.

- All the variables to be plotted must be specified in the ANALYSIS subcommand.
- If the variable list following the keywords CATEGORIES, TRANS, and RESID is empty, then it will cause a syntax error and the procedure will terminate.
- The variables in the varlist for labeling the object points following OBJECT and BIPLOT must be specified on the VARIABLES subcommand but need not appear on the ANALYSIS subcommand. This means that variables not included in the analysis can still be used to label plots.
- The keyword TO in a variable list can only be used with variables that are in the ANALYSIS subcommand, and TO applies only to the order of the variables in the ANALYSIS subcommand For variables that are in the VARIABLES subcommand but not in the ANALYSIS subcommand, the keyword TO cannot be used. For example, if /VARIABLES = v1 TO v5 and /ANALYSIS is v2 v1 v4, then /PLOT OBJECT(v1 TO v4) will give two object plots, one labeled with *v1* and one labeled with *v4*. (/PLOT OBJECT(v1 TO v4 v2 v3 v5) will give objects plots labeled with *v1*, *v2*, *v3*, *v4*, and *v5*).
- For multidimensional plots, all of the dimensions in the solution are produced in a matrix scatterplot if the number of dimensions in the solution is greater than two and the NDIM keyword is not specified; if the specified number of dimensions is 2, a scatterplot is produced.

The following keywords can be specified:

OBJECT (varlist)(n). *Plots of the object points.* Following the keyword, a list of variables in parentheses can be given to indicate that plots of object points labeled with the categories of the variables should be produced (one plot for each variable). If the variable list is omitted, a plot labeled with case numbers is produced.

CATEGORY(varlist)(n). *Plots of the category points (centroid coordinates).* A list of variables must be given in parentheses following the keyword. Categories are in the centroids of the objects in the particular categories.

DISCRIM(varlist)(n). *Plot of the discrimination measures.* DISCRIM can be followed by a varlist to select the variables to include in the plot. If the variable list is omitted, a plot including all variables is produced.

TRANS(varlist(n)). *Transformation plots per variable (optimal category quantifications against category indicators).* Following the keyword, a list of variables in parentheses must be given. Each variable can be followed by a number of dimensions in parentheses to indicate you want to display *p* residual plots, one for each of the first *p* dimensions. If the number of dimensions is not specified, a plot for the first dimension is produced.

RESID(varlist(n))(n). *Plot of residuals per variable (approximation against optimal category quantifications).* Following the keyword, a list of variables in parentheses must be given. Each variable can be followed by a number of dimensions in parentheses to indicate you want to display *p* residual plots, one for each of the first *p* dimensions. If the number of dimensions is not specified, a plot for the first dimension is produced.

BIPLOT((varlist))(varlist)(n). *Plot of objects and variables (centroids).* When NORMALIZATION = INDEPENDENT, this plot is incorrect and therefore not available. BIPLOT can be followed by a varlist in double parentheses to select the variables to include in the plot. If this variable list is omitted, a plot including all variables is produced. Following BIPLOT or BIPLOT((varlist)), a list of variables in single parentheses can be given to indicate that plots with objects labeled with the categories of the variables should be produced (one plot for each variable). If this variable list is omitted, a plot with objects labeled with case numbers is produced.

JOINTCAT(varlist)(n). *Joint plot of the category points for the variables in the varlist.* If no `varlist` is given, the category points for all variables are displayed.

NONE. *No plots.*

- For all of the keywords except TRANS and NONE, the user can specify an optional parameter *l* in parentheses after the variable list in order to control the global upper boundary of variable name/label and value label lengths in the plot. Note that this boundary is applied uniformly to all variables in the list. The label length parameter *l* can take any non-negative integer less than or equal to the applicable maximum length (64 for variable names, 255 for variable labels, and 60 for value labels). If *l* = 0, names/values instead of variable/value labels are displayed to indicate variables/categories. If *l* is not specified, `MULTIPLE CORRESPONDENCE` assumes that each variable name/label and value label at its full length is displayed. If *l* is an integer larger than the applicable maximum, then we reset it to the applicable maximum but do not issue a warning. If a positive value of *l* is given but if some or all of the variables/category values do not have labels, then for those variables/values the names/values themselves are used as the labels.

In addition to the plot keywords, the following can be specified:

NDIM(value,value). *Dimension pairs to be plotted.* `NDIM` is followed by a pair of values in parentheses. If `NDIM` is not specified or if `NDIM` is specified without parameter values, a matrix scatterplot including all dimensions is produced.

- The first value (an integer that can range from 1 to the number of dimensions in the solution minus 1) indicates the dimension that is plotted against higher dimensions.
- The second value (an integer that can range from 2 to the number of dimensions in the solution) indicates the highest dimension to be used in plotting the dimension pairs.
- The `NDIM` specification applies to all requested multidimensional plots.

SAVE Subcommand

The SAVE subcommand is used to add the transformed variables (category indicators replaced with optimal quantifications) and the object scores to the working data file.

- Excluded cases are represented by a dot (the sysmis symbol) on every saved variable.

TRDATA. *Transformed variables.* Missing values specified to be treated as passive are represented by a dot. Following TRDATA, a rootname, and the number of dimensions to be saved can be specified in parentheses (if the number of dimensions is not specified, all dimensions are saved).

OBJECT. *Object scores.*

`MULTIPLE CORRESPONDENCE` adds three numbers. The first number uniquely identifies the source variable names, the middle number corresponds to the dimension number, and the last number uniquely identifies the `MULTIPLE CORRESPONDENCE` procedures with the successfully executed SAVE subcommands. Only one rootname can be specified and it can contain up to three characters. If more than one rootname is specified, the first rootname is used; if a rootname contains more than three characters, the first three characters are used at most.

- If a rootname is not specified for TRDATA, rootname *TRA* is used to automatically generate unique variable names. The formula is *ROOTNAMEk_m_n*, where *k* increments from 1 to identify the source variable names by using the source variables' position numbers in the `ANALYSIS` subcommand, *m* increments from 1 to identify the dimension number, and *n* increments from 1 to identify the `MULTIPLE CORRESPONDENCE` procedures with the successfully executed SAVE subcommands for a given data file in a continuous session. For example, with two variables specified on `ANALYSIS` and 2 dimensions to save, the first set of default names, if they do not exist in the data file, would be *TRA1_1_1*, *TRA1_2_1*, *TRA2_1_1*, *TRA2_2_1*. The next set of default names, if they do not exist in the data file, would be *TRA1_1_2*, *TRA1_2_2*, *TRA2_1_2*, *TRA2_2_2*. However, if, for example, *TRA1_1_2* already exists in the data file, then the default names should be attempted as *TRA1_1_3*, *TRA1_2_3*, *TRA2_1_3*, *TRA2_2_3*. That is, the last number increments to the next available integer.

- Following OBJECT, a rootname and the number of dimensions can be specified in parentheses (if the number of dimensions is not specified, all dimensions are saved), to which MULTIPLE CORRESPONDENCE adds two numbers separated by the underscore symbol (_). The first number corresponds to the dimension number. The second number uniquely identifies the MULTIPLE CORRESPONDENCE procedures with the successfully executed SAVE subcommands. Only one rootname can be specified, and it can contain up to five characters. If more than one rootname is specified, the first rootname is used; if a rootname contains more than five characters, the first five characters are used at most.
- If a rootname is not specified for OBJECT, the rootname *OBSCO* is used to automatically generate unique variable names. The formula is *ROOTNAMEm_n*, where *m* increments from 1 to identify the dimension number and *n* increments from 1 to identify the MULTIPLE CORRESPONDENCE procedures with the successfully executed SAVE subcommands for a given data file in a continuous session. For example, if 2 dimensions are specified following OBJECT, the first set of default names, if they do not exist in the data file, would be *OBSCO1_1*, *OBSCO2_1*. The next set of default names, if they do not exist in the data file, would be *OBSCO1_2*, *OBSCO2_2*. However, if, for example, *OBSCO2_2* already exists in the data file, then the default names should be attempted as *OBSCO1_3*, *OBSCO2_3*. That is, the second number increments to the next available integer.
- Variable labels are created automatically. They are shown in the Notes table and can also be displayed in the Data Editor window.
- If the number of dimensions is not specified, the SAVE subcommand saves all dimensions.

OUTFILE Subcommand

The OUTFILE subcommand is used to write the discretized data, transformed data (category indicators replaced with optimal quantifications), and the object scores to an external IBM SPSS Statistics data file or previously declared dataset. Excluded cases are represented by a dot (the sysmis symbol) on every saved variable.

DISCRDATA('savfile'|'dataset'). *Discretized data.*

TRDATA('savfile'|'dataset'). *Transformed variables.* Missing values specified to be treated as passive are represented by a dot.

OBJECT('savfile'|'dataset'). *Object scores.*

Filenames should be enclosed in quotes and are stored in the working directory unless a path is included as part of the file specification. datasets are available during the current session but are not available in subsequent sessions unless you explicitly save them as data files. The names should be different for each of the keywords.

- The active dataset, in principle, should not be replaced by this subcommand, and the asterisk (*) file specification is not supported. This strategy also helps prevent OUTFILE interference with the SAVE subcommand.

MULTIPLE IMPUTATION

MULTIPLE IMPUTATION is available in SPSS Statistics Professional Edition or the Missing Values option.

```
MULTIPLE IMPUTATION varlist
[/IMPUTE [METHOD={AUTO**  }]
                 {FCS     }
                 {MONOTONE}
                 {NONE    }
         [NIMPUTATIONS={5**}]
                       {int}
         [SCALEMODEL = {LINEAR**}]
                       {PMM     }
         [INTERACTIONS = {NONE**}]
                         {TWOWAY}
         [MAXPCTMISSING={NONE**}]
                        {num   }
         [MAXCASEDRAWS={50**}]
                       {int }
         [MAXPARAMDRAWS={2**}]
                        {int}
         [MAXITER={10**}]
                  {int }
         [SINGULAR={1E-12**}]]
                   {num    }
         [MAXMODELPARAM={100**}]
                        {int  }
 [/CONSTRAINTS varlist ([MIN={NONE**}]
                           {num   }
                       [MAX={NONE**}]
                           {num   }
                       [RND={NO**  }]
                           {num   }
                       [ROLE={BOTH**}])]
                             {DEP   }
                             {IND   }
 [/MISSINGSUMMARIES [OVERALL**]
                    [VARIABLES[([MAXVARS={25**}][MINPCTMISSING={10**}])]
                                         {int }                {num }
                    [PATTERNS]
                    [NONE]]
 [/IMPUTATIONSUMMARIES [MODELS**] [DESCRIPTIVES[(varlist)]] [NONE]]
 [/ANALYSISWEIGHT var]
 [/OUTFILE [IMPUTATIONS = 'savfile' | 'dataset']]
           [FCSITERATIONS = 'savfile' | 'dataset']
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 17.0

- Command introduced.

Example

```
DATASET DECLARE imputedData.
MULTIPLE IMPUTATION x y z
 /OUTFILE IMPUTATIONS = imputedData.
```

Overview

The MULTIPLE IMPUTATION procedure performs multiple imputation of missing data values. Given a dataset containing missing values, it outputs one or more datasets in which missing values are replaced with plausible estimates. The procedure also summarizes missing values in the working dataset.

Datasets produced by the MULTIPLE IMPUTATION procedure can be analyzed using supported analysis procedures to obtain final (combined) parameter estimates that take into account the inherent uncertainty among the various sets of imputed values.

Options

Variables. You can specify which variables to impute and specify constraints on the imputed values such as minimum and maximum values. You can also specify which variables are used as predictors when imputing missing values of other variables.

Methods. Three imputation methods are offered. The monotone method is an efficient method for data that have a monotone pattern of missingness. Fully conditional specification (FCS) is an iterative Markov Chain Monte Carlo (MCMC) method that is appropriate when the data have an arbitrary (monotone or nonmonotone) missing pattern. The default method (AUTO) scans the data to determine the best imputation method (monotone or FCS). For each method you can control the number of imputations.

Output. By default the procedure displays an overall summary of missingness in your data as well as an imputation summary and the imputation model for each variable whose values are imputed. You can obtain analysis of missing values by variable as well as tabulated patterns of missing values. If you request imputation you can obtain descriptive statistics for imputed values.

Basic Specification

The basic specification is two or more variables and a file specification where the imputed data will be written.

- By default, the procedure imputes missing values using the AUTO method. Five imputations are performed.
- When imputing the default model type depends on the measurement level of the variable: for categorical variables, logistic regression is used; for scale variables linear regression is used.
- The procedure generates output that summarizes missingness in the data and summaries how values were imputed.

Operations

- The output dataset contains the original (nonmissing) data and data for one or more imputations. Each imputation includes all of the observed data and imputed data values. The original and imputed data are stacked in the output dataset. A special variable, *Imputation_*, identifies whether a case represents original data (*Imputation_* = 0) or imputed data (*Imputation_* =1...m).
- Multiple imputation datasets can be analyzed using supported analysis procedures to obtain final (combined) parameter estimates that take into account the inherent uncertainty in the various sets of imputed values. Variable *Imputation_* must be defined as a split variable in order to obtain pooled parameter estimates.
- The procedure honors the random number generator and seed specified via the global SET command. Specify the same seed across invocations of the procedure if you want to be able to reproduce your imputed values.
- The procedure honors the WEIGHT variable. It is treated as a replication weight when summarizing missing values and estimating imputation models. Cases with negative or zero value of replication weight are ignored. Noninteger weights are rounded to the nearest integer. The procedure also accepts analysis weights (see ANALYSISWEIGHT subcommand).
- The procedure honors SPLIT FILE. A separate missing value analysis and set of imputations is produced for each combination of values of the split variables. An error occurs if imputation is requested and the input dataset has eight split variables.
- The procedure honors the FILTER command. Cases filtered out are ignored by the procedure.
- The procedure accepts string variables and treats them as categorical. Completely blank string values are treated as valid values, i.e., they are not replaced.
- The procedure treats both user- and system-missing values as invalid values. Both types of missing values are replaced when values are imputed and both are treated as invalid values of variables used as predictors in imputation models. User- and system-missing values are also treated as missing in analyses of missingness (counts of missing values, etc.).
- Cases that have a missing value for each analysis variable are included in analyses of missingness but are excluded from imputation. Specifically, values of such cases are not imputed and are excluded when when building imputation models. The determination of which cases are completely missing is made after any variables are filtered out of the imputation model by the MAXPCTMISSING keyword.
- An error occurs if imputation is requested and the input dataset contains a variable named *Imputation_*.
- An error occurs if imputation and iteration history are requested and the input dataset contains a variable named *Iteration_*, or *SummaryStatistic_*.

Note: Since measurement level can affect the results, if any variables (fields) have an unknown measurement level, an initial data pass will be performed to determine default measurement level for any variables with an unknown measurement level. For information on the criteria used to determine default measurement level, see SET SCALEMIN.

Syntax Rules

- Two or more analysis variables are required.
- The OUTFILE subcommand is required unless imputation is turned off (/IMPUTE METHOD=NONE). All other subcommands are optional.
- Only a single instance of each subcommand is allowed except for the CONSTRAINTS command, which can be repeated.
- An error occurs if an attribute or keyword is specified more than once within a subcommand.
- Equals signs shown in the syntax chart are required.
- Subcommand names and keywords must be spelled in full.
- Empty subcommands are not allowed.

Examples

Automatic Imputation

```
DATASET DECLARE imputedData.
MULTIPLE IMPUTATION x y z
  /OUTFILE IMPUTATIONS = imputedData.
```

- The command requests multiple imputation of scale variables x, y, and z.
- By default, the procedure chooses an imputation method based on a scan of the data. Five imputations are produced by default.
- As all variables are scale, a linear regression model is used for each. When a particular variable is imputed, all other variables are treated as covariates in the imputation model if FCS is automatically chosen as the imputation method. If the monotone method is chosen, only preceding variables are used as predictors.
- The imputed data are written to a new dataset *imputedData*.
- By default, an overall summary of missingness is displayed, as well as information about how values were imputed.

Analysis of Missingness

```
MULTIPLE IMPUTATION x y z
  /IMPUTE METHOD=NONE
  /MISSINGSUMMARIES VARIABLES.
```

- The command requests analysis of missing values for variables x, y, and z.
- No imputation is performed.

Custom Imputation Model

```
DATASET DECLARE imputedData.
MULTIPLE IMPUTATION x y z
  /IMPUTE METHOD=FCS
  /CONSTRAINTS x y (RND=1 MIN=1)
  /CONSTRAINTS x   (MAX=5)
  /CONSTRAINTS y   (MAX=10)
  /OUTFILE IMPUTATIONS = imputedData.
```

- Scale variables x, y, and z are defined as analysis variables.
- Missing values are imputed using the FCS method. Variables are imputed in order of their appearance in the variable list. Variable x is imputed first, using x and z as predictors. Then y is imputed, treating x and z as predictors. Finally, z is imputed using x and y as predictors.

- CONSTRAINTS subcommands specify constraints on imputed values. Imputed values of x and y are rounded to the nearest integer and are constrained to be no less than 1. Maximum values of 5 and 10 are specified for x and y, respectively. No constraints are defined for variable z.
- Imputed data are written to a new dataset *imputedData*.
- By default, an overall summary of missingness is displayed, as well as information about how values were imputed.

Variable Lists

Analysis variables are specified after the command name. Two or more variables must be specified. The TO and ALL keywords can be used to refer to multiple variables. If any variable is specified more than once, the last instance of that variable is honored.

The variable list specifies variables to impute and to include in analyses of missingness. By default, analyis variables are also used as predictors to in imputation models of other analysis variables. The lists of variables to impute and predictors can be restricted via the CONSTRAINTS subcommand.

Variable Order

When imputing using the FCS and MONOTONE methods, variables are imputed sequentially in the order in which they are listed in the variables list. The order of the variables is ignored by the AUTO method except to break ties.

Predictors

The set of predictors that is used when a particular variable is imputed depends on the imputation method:

- For the FCS method, when a variable is imputed all other analysis variables are used as predictors in the imputation model.
- When the MONOTONE method is used, only variables that precede the variable to be imputed in the variable list are used as predictors.
- When the AUTO method is used the set of predictors depends on the pattern of missingness in the data. If the data have a nonmonotone pattern of missingness, FCS is used and all other analysis variables are used as predictors. If the data have a monotone pattern, MONOTONE is used and all variables that precede the variable to be imputed are used predictors. Note that AUTO sorts the analysis variables to detect monotone pattern, so the actual order of variables may not correspond to their order in which they are specified in the variables list.

Measurement Level

Measurement level recorded in the data dictionary is honored for each analysis variable. Measurement level determines the following:

- The default type of imputation model for variables whose values are imputed (linear regression or logistic regression).
- Whether a variable is treated as a factor (categorical) or covariate (scale) when used as a predictor in imputation models.
- Whether the variable is treated as scale or categorical in summaries of missing values.

The procedure treats ordinal and nominal variables equivalently as categorical variables.

IMPUTE Subcommand

The IMPUTE subcommand controls the imputation method and model. By default, the AUTO method is used to impute missing data values.

METHOD Keyword

The METHOD keyword specifies the imputation method. Specify one of the following options:

AUTO. *Automatic method.* This is the default. Chooses the best method based on a scan of the data. The monotone method is used when the data have a monotone pattern of missingness; otherwise, if the data have a nonmonotone pattern, FCS is used. The AUTO method internally sorts the analysis variables in order from the least missing to most missing to detect a monotone pattern of missingness if it exists; thus, order of variables in the variable list is ignored when AUTO is used.

MONOTONE. *Monotone method.* This is a noniterative method that can be used only when the data have a monotone pattern of missingness. A monotone pattern exists when you can order the variables such that, if a variable has a nonmissing value, all preceding variables also have nonmissing values. The monotone method imputes missing values sequentially in the order specified in the variable list. An error occurs if the input data do not have a monotone pattern for the order in which variables are specified.

FCS. *Fully conditional specification method.* This is an iterative Markov chain Monte Carlo (MCMC) method that can be used when the pattern of missing data is arbitrary (monotone or nonmonotone). In each iteration, the FCS method sequentially imputes missing values in the order specified in the variable list.

NONE. *None.* No imputation is performed. Choose this option if you want analyses of missingness only. A warning is issued if you turn off imputation and suppress all analyses of missing values.

NIMPUTATIONS Keyword

By default, five imputations are performed. To request a different number of imputations, specify a positive integer. As a rule of thumb, the higher the degree of missing information in the data, the more imputations that are needed to obtain reasonable efficiency relative to an infinite number of imputations.

- NIMPUTATIONS is ignored if imputation is turned off.

SCALEMODEL Keyword

The AUTO, FCS, and MONOTONE methods are multivariate methods that can be used when several variables have missing values. Each uses univariate models when imputing values of individual variables.

By default, the type of univariate model that is used depends on the measurement level of the variable whose missing values are to be imputed. Multinomial logistic regression is always used for categorical variables. For scale variables, linear regression (LINEAR) is used by default. Optionally you can use the predictive mean matching (PMM) method for scale variables. PMM is a variant of linear regression that ensures that the imputed values are plausible. For PMM the imputed value always matches an observed value (specifically, the observed value that is closest to the value drawn by the imputation model).

- SCALEMODEL is honored only when the FCS or MONOTONE method is chosen explicitly. Linear regression is always used for scale variables when the AUTO method is requested. A warning is issued if SCALEMODEL is specified for the AUTO method.
- SCALEMODEL is ignored if impution is turned off.

INTERACTIONS Keyword

By default, the imputation model for each variable includes a constant term and main effects for predictor variables. You can optionally include all possible two-way interactions among categorical predictor variables. Specify INTERACTIONS=TWOWAY. Interaction terms do not include scale predictors.

- INTERACTIONS is honored only when the FCS or MONOTONE method is chosen explicitly. Two-way interactions are not included when AUTO method is requested. A warning is issued if INTERACTIONS is specified for the AUTO method.
- INTERACTIONS is ignored if imputation is turned off or if there are fewer than two categorical predictors.

MAXPCTMISSING Keyword

By default analysis variables are imputed and used as predictors without regard to how many missing values they have provided they at least have sufficient data to estimate an imputation model. The optional MAXPCTMISSING keyword is used to exclude variables that have a high percentage of missing values. Specify the maximum allowable percentage of missing values as a positive number less than 100. For example, if you specify MAXMISSINGPCT=50, analysis variables that have more than 50% missing values are not imputed, nor are they used as predictors in imputation models.

- MAXPCTMISSING is ignored if imputation is turned off.

Maximum Number of Draws

If minimum or maximum values are specified for imputed values of scale variables (see CONSTRAINTS subcommand), the procedure attempts to draw values for a case until it finds a set of values that are within the specified ranges. By default, a maximum of 50 sets of values are drawn per case. To override the maximum, specify a positive integer for MAXCASEDRAWS. If a set of values is not obtained within the specified number of draws per case, the procedure draws another set of model parameters and the repeats the case-drawing process. By default, a maximum of 2 sets of model parameters are drawn. To override the maximum, specify a positive integer for MAXPARAMDRAWS. An error occurs if a set of values within the ranges is not obtained within the specified number of case and parameter draws.

Note that increasing these values can increase the processing time. If the procedure is taking a long time (or is unable) to find suitable draws, check the minimum and maximum values specified on the CONSTRAINTS subcommand to ensure they are appropriate.

- MAXCASEDRAWS and MAXPARAMDRAWS are ignored if imputation is turned off, when predictive mean matching is used, and when imputing categorical values.

MAXITER Keyword

Iteration stops when the maximum number of iterations is reached. To override the default number of iterations (10), specify a positive integer value for MAXITER.

- MAXITER is honored only if the FCS method is chosen explicitly. The keyword is ignored when the AUTO method is used.
- The MAXITER keyword is ignored if the monotone method is used or imputation is turned off.

SINGULAR Keyword

SINGULAR specifies the tolerance value used to test for singularity in univariate imputation models (linear regression, logistic regression, and predictive mean matching). The default value is 10e–12. Specify a positive value.

- SINGULAR is honored only if the FCS or MONOTONE method is chosen explicitly. The keyword is ignored and the default singularity tolerance is used when AUTO method is requested.
- SINGULAR is ignored if imputation is turned off.

MAXMODELPARAM Keyword

MAXMODELPARAM specifies the maximum number of model parameters allowed when imputing any variable. If a model has more parameters than the specified limit, processing terminates with an error (no missing values are imputed). The default value is 100. Specify a positive integer value.

- MAXMODELPARAM is ignored if imputation is turned off.

CONSTRAINTS Subcommand

The CONSTRAINTS subcommand specifies constraints on imputed values. CONSTRAINTS specifications are ignored when imputation is turned off.

To define constraints, specify one or more analysis variables and one or more of following in parentheses:

MIN = NONE | num. *Minimum allowable imputed value for scale variables.* Specify a number. If an imputed value is less than the minimum, the procedure draws another value until it finds one that is greater than or equal to MIN or the MAXCASEDRAWS or MAXPARAMDRAWS threshold is reached (See METHOD subcommand). There is no default minimum. MIN is ignored when predictive mean matching is used or when applied to a categorical variable. An error occurs if MIN is greater than or equal to MAX. For date format variables, values must be enclosed in single or double quotes and expressed in the same date format as the defined date format for the variable.

MAX = NONE | num. *Maximum allowable imputed value for scale variables.* Specify a number. If an imputed value is greater than the maximum, the procedure draws another value until it finds one that is

less than or equal to MAX or the MAXCASEDRAWS or MAXPARAMDRAWS threshold is reached (See METHOD subcommand). There is no default maximum. MAX is ignored when predictive mean matching is used or when applied to a categorical variable. For date format variables, values must be enclosed in single or double quotes and expressed in the same date format as the defined date format for the variable.

RND = NO | num. *Controls rounding of imputed values.* By default imputed values are not rounded. To round imputed values, specify a numeric multiplier value. Imputed values are rounded to the nearest integer multiple of the specified multiplier. For example, if you specify RND=1, imputed values are rounded to the nearest integer; if you specify RND=0.1, imputed values are rounded to the nearest tenth. Note that no fuzz factor is added before rounding (that is, rounding is 'fuzzless'). RND is ignored when applied to a categorical or date format variable.

ROLE = BOTH | DEP | IND. *Note:* Missing values of variables that are defined as predictor-only (ROLE=IND) variables are imputed, but only for internal use (otherwise those values of the variable could not be used to impute other variables). Imputed values of such variables are not saved to the output dataset. When imputing predictor-only variables, a main-effects only model is always used; the INTERACTIONS keyword is ignored.

The variable list honors the TO and ALL keywords. Constraints are ignored when applied to a variable that is not an analysis variable.

The CONSTRAINTS subcommand can be specified more than once, and its settings are cumulative. That is, if more than one set of specifications is provided for a variable across CONSTRAINTS subcommands, the specifications are merged. When merging, if a particular keyword has different values for the same variable, the value specified in the last CONSTRAINTS subcommand for that variable is honored.

Cumulative Effects of Multiple Constraints

```
MULTIPLE IMPUTATION x y z
  /CONSTRAINTS x y z (RND=1 MIN=1)
  /CONSTRAINTS x     (MAX=5)
  /CONSTRAINTS y     (MIN=2 MAX=10)
  /CONSTRAINTS z     (ROLE=DEP)
  /OUTFILE IMPUTATIONS = imputedData.
```

- The first CONSTRAINTS subcommand specifies that imputed values of x, y, and z are rounded to the nearest integer and are constrained to be no less than 1.
- The second CONSTRAINTS subcommand specifies a maximum value of 5 for x.
- The third CONSTRAINTS subcommand specifies a maximum value of 10 for y, and overrules the previous minimum of 1 with a minimum of 2 for y.
- The fourth CONSTRAINTS subcommand specifies that z should be used as a dependent variable only in the imputation models.

The constraints can be summarized as follows:

Variable	Role	Minimum	Maximum	Rounding
x	Dependent and Predictor	1	5	1
y	Dependent and Predictor	2	10	1
z	Dependent only	1		1

MISSINGSUMMARIES Subcommand

The MISSINGSUMMARIES subcommand controls the display of missing values of analysis variables.

OVERALL Keyword

The OVERALL keyword displays a graph that summarizes missingness for cases, variables, and individual data (cell) values:

- Number and percent of analysis variables that have one or more missing values.
- Number and percent of cases that have one or more missing values on analysis variables.
- Number and percent of individual data values that are missing among all analysis variables and cases.

Overall summaries are displayed by default.

VARIABLES Keyword

The VARIABLES keyword displays a table of analysis variables sorted by percent of missing values. The table includes descriptive statistics (mean and standard deviation) for scale variables. The table is not shown by default.

You can optionally control the maximum number of variables to display (default=25) and minimum percentage missing (default=10) for a variable to be included in the display. Specify one or both in parentheses. For example, VARIABLES(MAXVARS=50 MINPCTMISSING=25) requests that the table display up to 50 variables that have at least 25% missing values. The set of variables that meet both criteria are displayed. For example, by default, if there are 25 analysis variables but only 5 have 10% or more missing values, the output includes only 5 variables. If there are more analysis variables than MAXVARS and all meet the MINPCTMISSING criteria, then variables with the most missing values are displayed.

- MAXVARS must be a positive integer.
- MINPCTMISSING must be a nonnegative number that is less than 100.

PATTERNS Keyword

The PATTERNS keyword displays tabulated patterns of missing values. Each pattern corresponds to a group of cases with the same pattern of incomplete and complete data on analysis variables. You can use PATTERNS output to determine whether monotone imputation method can be used for your data, or if not, how closely your data approximate a monotone pattern.

To display patterns, specify the PATTERNS keyword. The chart orders analysis variables and patterns to reveal monotonicity where it exists. Specifically, variables are ordered from left to right in increasing order of missingness. Rows (patterns) are the sorted first by the last variable (nonmissing values first, then missing values), then by the second to last variable, and so on, working from right to left.

You can use PATTERNS output to determine whether monotone imputation method can be used for your data, or, if not, how closely your data approximate a monotone pattern. If no nonmonotone pattern exists after reordering you can conclude that the data have a monotonic pattern when analysis variables are ordered as such. If data are not monotone, the chart shows how to achieve reach monotonicity.

NONE Keyword

NONE suppresses all MISSINGSUMMARIES output.

NONE generates an error when used with any other keyword.

IMPUTATIONSUMMARIES Subcommand

If imputation is requested an overall imputation summary is displayed. Contents include analysis specifications and results:

- Specifications include imputation method, number of imputations, model type for scale variables, whether two-way interactions included in models, MAXPCTMISSING setting, and analysis weight.
- Results include imputation method, iterations (for FCS method), dependent variables imputed, dependent variables excluded from imputation, and imputation sequence.

If specified, constaints for analysis variables are also shown when imputation is requested. You can also obtain optional summaries of imputed data by specifying one or more of the following keywords. Imputation summary keywords are ignored if imputation is turned off.

MODELS Keyword

The MODELS keyword displays the imputation model for dependent variables and predictors. Imputation models are shown by default. Includes univariate model type, model effects, and number of values imputed.

DESCRIPTIVES Keyword

The DESCRIPTIVES keyword displays descriptive statistics for dependent variables for which values are imputed. Descriptive statistics are not shown by default.

- For scale variables the descriptive statistics include mean, count, standard deviation, min, and max for the original input data (prior to imputation), imputed values (by imputation), and complete data (original and imputed values together—by imputation).
- For categorical variables the descriptive statistics include count and percent by category for the original input data (prior to imputation), imputed values (by imputation), and complete data (original and imputed values together—by imputation).

If you specify DESCRIPTIVES, statistics are produced by default for each dependent variables for which values are imputed. You can optionally specify a list of dependent variables in parentheses. Descriptives are shown only for the requested variables. Non-analysis variables in the list are ignored.

NONE Keyword

The NONE keyword suppresses IMPUTATIONSUMMARIES output.

NONE generates an error if any other keyword is specified.

ANALYSISWEIGHT Subcommand

The ANALYSISWEIGHT command specifies a variable containing analysis (regression) weights. The procedure incorporates analysis weights in regression and classification models used to impute missing values. Analysis weights are also used in summaries of imputed values (e.g,. mean, standard deviation, and standard error).

The analysis weight variable must be numeric. Cases with a negative or zero analysis weight are ignored.

OUTFILE Subcommand

The OUTFILE subcommand allows you to export IBM SPSS Statistics-format data containing imputed values or IBM SPSS Statistics-format data containing an FCS iteration history. The OUTFILE subcommand is ignored if imputation is turned off.

IMPUTATIONS Keyword

The IMPUTATIONS keyword specifies an optional dataset containing imputed data. The output dataset consists of the original data and one complete dataset per imputation. All variables in the input dataset are included in the output dataset. Dictionary properties (names, labels, etc.) of existing variables are copied to the new dataset. The file also contains a new variable, *Imputation_*, a numeric variable that indicates the imputation (0 for original data, 1..*n* for cases having imputed values).

The procedure automatically defines the *Imputation_* variable as a split variable (see "SPLIT FILE" on page 1897) when the output dataset is created. If SPLIT FILE is in effect when the procedure executes, the output dataset includes one set of imputations for each combination of values of split variables.

FCSITERATIONS Keyword

The FCSITERATIONS keyword specifies an optional dataset that contains iteration history data for FCS imputation.

The dataset contains means and standard deviations by iteration and imputation for each scale dependent varable for which values are imputed. You can plot the data to help assess model convergence.

- The FCSITERATIONS keyword is ignored if the monotone method is used. A warning is shown and the dataset is not generated if all imputed variables are categorical.

MVA

MVA is available in the Missing Values Analysis option.

```
MVA VARIABLES= {varlist}
               {ALL    }

  [/CATEGORICAL=varlist]

  [/MAXCAT={25**}]
           {n   }

  [/ID=varname]
```

Description:

```
  [/NOUNIVARIATE]

  [/TTEST [PERCENT={5}] [{T  }] [{DF  } [{PROB  }] [{COUNTS  }] [{MEANS  }]]
                   {n}   {NOT}   {NODF}  {NOPROB}]  {NOCOUNTS}   {NOMEANS}

  [/CROSSTAB [PERCENT={5}]]
                      {n}

  [/MISMATCH [PERCENT={5}] [NOSORT]]
                      {n}

  [/DPATTERN [SORT=varname[({ASCENDING })] [varname ... ]]
                           {DESCENDING}
             [DESCRIBE=varlist]]

  [/MPATTERN [NOSORT] [DESCRIBE=varlist]]

  [/TPATTERN [NOSORT] [DESCRIBE=varlist] [PERCENT={1}]]
                                                 {n}
```

Estimation:

```
  [/LISTWISE]

  [/PAIRWISE]

  [/EM  [predicted_varlist] [WITH predictor_varlist]
      [([TOLERANCE={0.001}   ]
                   {value}
        [CONVERGENCE={0.0001}]
                     {value }
        [ITERATIONS={25}     ]
                    {n }
        [TDF=n               ]
        [LAMBDA=a            ]
        [PROPORTION=b        ]
        [OUTFILE='file'      ])]


  [/REGRESSION  [predicted_varlist] [WITH predictor_varlist]
              [([TOLERANCE={0.001}                        ]
                           {n    }
                [FLIMIT={4.0}                             ]
                        {N  }
                [NPREDICTORS=number_of_predictor_variables]
                [ADDTYPE={RESIDUAL*}                      ]
                         {NORMAL   }
                         {T[({5})  }
                            {n}
                         {NONE     }
                [OUTFILE='file'                           ])]]
```

*If the number of complete cases is less than half the number of cases, the default ADDTYPE specification is NORMAL.

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Examples

```
MVA VARIABLES=populatn density urban religion lifeexpf region
  /CATEGORICAL=region
  /ID=country
  /MPATTERN DESCRIBE=region religion.


MVA VARIABLES=all
 /EM males msport WITH males msport gradrate facratio.
```

Overview

MVA (Missing Value Analysis) describes the missing value patterns in a data file (data matrix). It can estimate the means, the covariance matrix, and the correlation matrix by using listwise, pairwise, regression, and EM estimation methods. Missing values themselves can be estimated (imputed), and you can then save the new data file.

Options

Categorical variables. String variables are automatically defined as categorical. For a long string variable, only the first eight characters are used to define categories. Quantitative variables can be designated as categorical by using the CATEGORICAL subcommand.

MAXCAT specifies the maximum number of categories for any categorical variable. If any categorical variable has more than the specified number of distinct values, MVA is not executed.

Analyzing Patterns. For each quantitative variable, the TTEST subcommand produces a series of *t* tests. Values of the quantitative variable are divided into two groups, based on the presence or absence of other variables. These pairs of groups are compared using the *t* test.

Crosstabulating Categorical Variables. The CROSSTAB subcommand produces a table for each categorical variable, showing, for each category, how many nonmissing values are in the other variables and the percentages of each type of missing value.

Displaying Patterns. DPATTERN displays a case-by-case data pattern with codes for system-missing, user-missing, and extreme values. MPATTERN displays only the cases that have missing values and sorts by the pattern that is formed by missing values. TPATTERN tabulates the cases that have a common pattern of missing values. The pattern tables have sorting options. Also, descriptive variables can be specified.

Labeling Cases. For pattern tables, an ID variable can be specified to label cases.

Suppression of Rows. To shorten tables, the PERCENT keyword suppresses missing-value patterns that occur relatively infrequently.

Statistics. Displays of univariate, listwise, and pairwise statistics are available.

Estimation. EM and REGRESSION use different algorithms to supply estimates of missing values, which are used in calculating estimates of the mean vector, the covariance matrix, and the correlation matrix of dependent variables. The estimates can be saved as replacements for missing values in a new data file.

Basic Specification

The basic specification depends on whether you want to describe the missing data pattern or estimate statistics. Often, description is done first, and then, considering the results, an estimation is done. Alternatively, both description and estimation can be done by using the same MVA command.

Descriptive Analysis. A basic descriptive specification includes a list of variables and a statistics or pattern subcommand. For example, a list of variables and the subcommand DPATTERN would show missing value patterns for all cases with respect to the list of variables.

Estimation. A basic estimation specification includes a variable list and an estimation method. For example, if the EM method is specified, the following are estimated: the mean vector, the covariance matrix, and the correlation matrix of quantitative variables with missing values.

Syntax Rules

- A variables specification is required directly after the command name. The specification can be either a variable list or the keyword ALL.
- The CATEGORICAL, MAXCAT, and ID subcommands, if used, must be placed after the variables list and before any other subcommand. These three subcommands can be in any order.

- Any combination of description and estimation subcommands can be specified. For example, both the EM and REGRESSION subcommands can be specified in one MVA command.
- Univariate statistics are displayed unless the NOUNIVARIATE subcommand is specified. Thus, if only a list of variables is specified, with no description or estimation subcommands, univariate statistics are displayed.
- If a subcommand is specified more than once, only the last subcommand is honored.
- The following words are reserved as keywords or internal commands in the MVA procedure: VARIABLES, SORT, NOSORT, DESCRIBE, and WITH. They cannot be used as variable names in MVA.
- The tables *Summary of Estimated Means* and *Summary of Estimated Standard Deviations* are produced if you specify more than one way to estimate means and standard deviations. The methods include univariate (default), listwise, pairwise, EM, and regression. For example, these tables are produced when you specify both LISTWISE and EM.

Symbols

The symbols that are displayed in the DPATTERN and MPATTERN table cells are:

+. Extremely high value

–. Extremely low value

S. System-missing value

A. First type of user-missing value

B. Second type of user-missing value

C. Third type of user-missing value

- An extremely high value is more than 1.5 times the interquartile range above the 75th percentile, if (number of variables) × n log$n$ ≤ 150000, where n is the number of cases.
- An extremely low value is more than 1.5 times the interquartile range below the 25th percentile, if (number of variables) × n log$n$ ≤ 150000, where n is the number of cases.
- For larger files—that is, (number of variables) × n log$n$ > 150000—extreme values are two standard deviations from the mean.

Missing Indicator Variables

For each variable in the variables list, a binary indicator variable is formed (internal to MVA), indicating whether a value is present or missing.

VARIABLES Subcommand

A list of variables or the keyword ALL is required.

- The order in which the variables are listed determines the default order in the output.
- If the keyword ALL is used, the default order is the order of variables in the active dataset.
- String variables that are specified in the variable list, whether short or long, are automatically defined as categorical. For a long string variable, only the first eight characters of the values are used to distinguish categories.
- The list of variables must precede all other subcommands.
- Multiple lists of variables are not allowed.

CATEGORICAL Subcommand

The MVA procedure automatically treats all string variables in the variables list as categorical. You can designate numeric variables as categorical by listing them on the CATEGORICAL subcommand. If a variable is designated categorical, it will be ignored if listed as a dependent or independent variable on the REGRESSION or EM subcommand.

MAXCAT Subcommand

The MAXCAT subcommand sets the upper limit of the number of distinct values that each categorical variable in the analysis can have. The default is 25. This limit affects string variables in the variables list and also the categorical variables that are defined by the CATEGORICAL subcommand. A large number of categories can slow the analysis considerably. If any categorical variable violates this limit, MVA does not run.

Example

```
MVA VARIABLES=populatn density urban religion lifeexpf region
  /CATEGORICAL=region
  /MAXCAT=30
  /MPATTERN.
```

- The CATEGORICAL subcommand specifies that *region*, a numeric variable, is categorical. The variable *religion*, a string variable, is automatically categorical.
- The maximum number of categories in *region* or *religion* is 30. If either variable has more than 30 distinct values, MVA produces only a warning.
- Missing data patterns are shown for those cases that have at least one missing value in the specified variables.
- The summary table lists the number of missing and extreme values for each variable, including those with no missing values.

ID Subcommand

The ID subcommand specifies a variable to label cases. These labels appear in the pattern tables. Without this subcommand, the case numbers are used.

Example

```
MVA VARIABLES=populatn density urban religion lifeexpf region
  /CATEGORICAL=region
  /MAXCAT=20
  /ID=country
  /MPATTERN.
```

- The values of the variable *country* are used as case labels.
- Missing data patterns are shown for those cases that have at least one missing value in the specified variables.

NOUNIVARIATE Subcommand

By default, MVA computes univariate statistics for each variable—the number of cases with nonmissing values, the mean, the standard deviation, the number and percentage of missing values, and the counts of extreme low and high values. (Means, standard deviations, and extreme value counts are not reported for categorical variables.)

- To suppress the univariate statistics, specify NOUNIVARIATE.

Examples

```
MVA VARIABLES=populatn density urban religion lifeexpf region
  /CATEGORICAL=region
  /CROSSTAB PERCENT=0.
```

- Univariate statistics (number of cases, means, and standard deviations) are displayed for *populatn*, *density*, *urban*, and *lifeexpf*. Also, the number of cases, counts and percentages of missing values, and counts of extreme high and low values are displayed.
- The total number of cases and counts and percentages of missing values are displayed for *region* and *religion* (a string variable).
- Separate crosstabulations are displayed for *region* and *religion*.

```
MVA VARIABLES=populatn density urban religion lifeexpf region
  /CATEGORICAL=region.
  /NOUNIVARIATE
  /CROSSTAB PERCENT=0.
```

- Only crosstabulations are displayed (no univariate statistics).

TTEST Subcommand

For each quantitative variable, a series of *t* tests are computed to test the difference of means between two groups defined by a missing indicator variable for each of the other variables. (See the topic “Missing Indicator Variables ” on page 1213 for more information.) For example, a *t* test is performed on *populatn* between two groups defined by whether their values are present or missing for *calories*. Another *t* test is performed on *populatn* for the two groups defined by whether their values for *density* are present or missing, and the tests continue for the remainder of the variable list.

PERCENT=n. *Omit indicator variables with less than the specified percentage of missing values*. You can specify a percentage from 0 to 100. The default is 5, indicating the omission of any variable with less than 5% missing values. If you specify 0, all rows are displayed.

Display of Statistics

The following statistics can be displayed for a *t* test:

- The *t* **statistic**, for comparing the means of two groups defined by whether the indicator variable is coded as missing or nonmissing. (See the topic “Missing Indicator Variables ” on page 1213 for more information.)

  T. *Display the* t *statistics.* This setting is the default.

  NOT. *Suppress the* t *statistics.*
- The **degrees of freedom** associated with the *t* statistic.

  DF. *Display the degrees of freedom.* This setting is the default.

  NODF. *Suppress the degrees of freedom.*
- The **probability** (two-tailed) associated with the *t* test, calculated for the variable that is tested without reference to other variables. Care should be taken when interpreting this probability.

  PROB. *Display probabilities.*

  NOPROB. *Suppress probabilities.* This setting is the default.
- The **number of values in each group**, where groups are defined by values that are coded as missing and present in the indicator variable.

  COUNTS. *Display counts.* This setting is the default.

  NOCOUNTS. *Suppress counts.*
- The **means** of the groups, where groups are defined by values that are coded as missing and present in the indicator variable.

MEANS. *Display means.* This setting is the default.

NOMEANS. *Suppress means.*

Example

```
MVA VARIABLES=populatn density urban religion lifeexpf region
  /CATEGORICAL=region
  /ID=country
  /TTEST.
```

- The TTEST subcommand produces a table of *t* tests. For each quantitative variable named in the variables list, a *t* test is performed, comparing the mean of the values for which the other variable is present against the mean of the values for which the other variable is missing.
- The table displays default statistics, including values of *t*, degrees of freedom, counts, and means.

CROSSTAB Subcommand

CROSSTAB produces a table for each categorical variable, showing the frequency and percentage of values that are present (nonmissing) and the percentage of missing values for each category as related to the other variables.

- No tables are produced if there are no categorical variables.
- Each categorical variable yields a table, whether it is a string variable that is assumed to be categorical or a numeric variable that is declared on the CATEGORICAL subcommand.
- The categories of the categorical variable define the columns of the table.
- Each of the remaining variables defines several rows—one each for the number of values present, the percentage of values present, and the percentage of system-missing values; and one each for the percentage of values defined as each discrete type of user-missing (if they are defined).

PERCENT=n. *Omit rows for variables with less than the specified percentage of missing values*. You can specify a percentage from 0 to 100. The default is 5, indicating the omission of any variable with less than 5% missing values. If you specify 0, all rows are displayed.

Example

```
MVA VARIABLES=age income91 childs jazz folk
  /CATEGORICAL=jazz folk
  /CROSSTAB PERCENT=0.
```

- A table of univariate statistics is displayed by default.
- In the output are two crosstabulations (one crosstabulation for *jazz* and one crosstabulation for *folk*). The table for *jazz* displays, for each category of *jazz*, the number and percentage of present values for *age*, *income91*, *childs*, and *folk*. It also displays, for each category of *jazz*, the percentage of each type of missing value (system-missing and user-missing) in the other variables. The second crosstabulation shows similar counts and percentages for each category of *folk*.
- No rows are omitted, because PERCENT=0.

MISMATCH Subcommand

MISMATCH produces a matrix showing percentages of cases for a pair of variables in which one variable has a missing value and the other variable has a nonmissing value (a mismatch). The diagonal elements are percentages of missing values for a single variable, while the off-diagonal elements are the percentage of mismatch of the indicator variables. See the topic “Missing Indicator Variables ” on page 1213 for more information. Rows and columns are sorted on missing patterns.

PERCENT=n. *Omit patterns involving less than the specified percentage of cases*. You can specify a percentage from 0 to 100. The default is 5, indicating the omission of any pattern that is found in less than 5% of the cases.

NOSORT. *Suppress sorting of the rows and columns*. The order of the variables in the variables list is used. If ALL was used in the variables list, the order is that of the data file.

DPATTERN Subcommand

DPATTERN lists the missing values and extreme values for each case symbolically. For a list of the symbols that are used, see .

By default, the cases are listed in the order in which they appear in the file. The following keywords are available:

SORT=varname [(order)]. *Sort the cases according to the values of the named variables*. You can specify more than one variable for sorting. Each sort variable can be in ASCENDING or DESCENDING order. The default order is ASCENDING.

DESCRIBE=varlist. *List values of each specified variable for each case*.

Example

```
MVA VARIABLES=populatn density urban religion lifeexpf region
  /CATEGORICAL=region
  /ID=country
  /DPATTERN DESCRIBE=region religion SORT=region.
```

- In the data pattern table, the variables form the columns, and each case, identified by its country, defines a row.
- Missing and extreme values are indicated in the table, and, for each row, the number missing and percentage of variables that have missing values are listed.
- The values of *region* and *religion* are listed at the end of the row for each case.
- The cases are sorted by *region* in ascending order.
- Univariate statistics are displayed.

MPATTERN Subcommand

The MPATTERN subcommand symbolically displays patterns of missing values for cases that have missing values. The variables form the columns. Each case that has any missing values in the specified variables forms a row. The rows are sorted by missing-value patterns. For use of symbols, see .

- The rows are sorted to minimize the differences between missing patterns of consecutive cases.
- The columns are also sorted according to missing patterns of the variables.

The following keywords are available:

NOSORT. *Suppress the sorting of variables*. The order of the variables in the variables list is used. If ALL was used in the variables list, the order is that of the data file.

DESCRIBE=varlist. *List values of each specified variable for each case*.

Example

```
MVA VARIABLES=populatn density urban religion lifeexpf region
  /CATEGORICAL=region
  /ID=country
  /MPATTERN DESCRIBE=region religion.
```

- A table of missing data patterns is produced.
- The *region* and the *religion* are named for each listed case.

TPATTERN Subcommand

The TPATTERN subcommand displays a tabulated patterns table, which lists the frequency of each missing value pattern. The variables in the variables list form the columns. Each pattern of missing values forms a row, and the frequency of the pattern is displayed.

- An *X* is used to indicate a missing value.
- The rows are sorted to minimize the differences between missing patterns of consecutive cases.
- The columns are sorted according to missing patterns of the variables.

The following keywords are available:

NOSORT. *Suppress the sorting of the columns.* The order of the variables in the variables list is used. If ALL was used in the variables list, the order is that of the data file.

DESCRIBE=varlist. *Display values of variables for each pattern.* Categories for each named categorical variable form columns in which the number of each pattern of missing values is tabulated. For quantitative variables, the mean value is listed for the cases having the pattern.

PERCENT=n . *Omit patterns that describe less than 1% of the cases.* You can specify a percentage from 0 to 100. The default is 1, indicating the omission of any pattern representing less than 1% of the total cases. If you specify 0, all patterns are displayed.

Example

```
MVA VARIABLES=populatn density urban religion lifeexpf region
  /CATEGORICAL=region
  /TPATTERN NOSORT DESCRIBE=populatn region.
```

- Missing value patterns are tabulated. Each row displays a missing value pattern and the number of cases having that pattern.
- DESCRIBE causes the mean value of *populatn* to be listed for each pattern. For the categories in *region*, the frequency distribution is given for the cases having the pattern in each row.

LISTWISE Subcommand

For each quantitative variable in the variables list, the LISTWISE subcommand computes the mean, the covariance between the variables, and the correlation between the variables. The cases that are used in the computations are listwise nonmissing; that is, they have no missing value in any variable that is listed in the VARIABLES subcommand.

Example

```
MVA VARIABLES=populatn density urban religion lifeexpf region
  /CATEGORICAL=region
  /LISTWISE.
```

- Means, covariances, and correlations are displayed for *populatn*, *density*, *urban*, and *lifeexpf*. Only cases that have values for all of these variables are used.

PAIRWISE Subcommand

For each pair of quantitative variables, the PAIRWISE subcommand computes the number of pairwise nonmissing values, the pairwise means, the pairwise standard deviations, the pairwise covariances, and the pairwise correlation matrices. These results are organized as matrices. The cases that are used are all cases having nonmissing values for the pair of variables for which each computation is done.

Example

```
MVA VARIABLES=populatn density urban religion lifeexpf region
  /CATEGORICAL=region
  /PAIRWISE.
```

- Frequencies, means, standard deviations, covariances, and the correlations are displayed for *populatn*, *density*, *urban*, and *lifeexpf*. Each calculation uses all cases that have values for both variables under consideration.

EM Subcommand

The EM subcommand uses an EM (expectation-maximization) algorithm to estimate the means, the covariances, and the Pearson correlations of quantitative variables. This process is an iterative process, which uses two steps for each iteration. The E step computes expected values conditional on the observed data and the current estimates of the parameters. The M step calculates maximum-likelihood estimates of the parameters based on values that are computed in the E step.

- If no variables are listed in the EM subcommand, estimates are performed for all quantitative variables in the variables list.
- If you want to limit the estimation to a subset of the variables in the list, specify a subset of quantitative variables to be estimated after the subcommand name EM. You can also list, after the keyword WITH, the quantitative variables to be used in estimating.
- The output includes tables of means, correlations, and covariances.
- The estimation, by default, assumes that the data are normally distributed. However, you can specify a multivariate *t* distribution with a specified number of degrees of freedom or a mixed normal distribution with any mixture proportion (PROPORTION) and any standard deviation ratio (LAMBDA).
- You can save a data file with the missing values filled in. You must specify a filename and its complete path in single or double quotation marks.
- Criteria keywords and OUTFILE specifications must be enclosed in a single pair of parentheses.

The criteria for the EM subcommand are as follows:

TOLERANCE=value. *Numerical accuracy control.* Helps eliminate predictor variables that are highly correlated with other predictor variables and would reduce the accuracy of the matrix inversions that are involved in the calculations. The smaller the tolerance, the more inaccuracy is tolerated. The default value is 0.001.

CONVERGENCE=value. *Convergence criterion.* Determines when iteration ceases. If the relative change in the likelihood function is less than this value, convergence is assumed. The value of this ratio must be between 0 and 1. The default value is 0.0001.

ITERATIONS=n. *Maximum number of iterations.* Limits the number of iterations in the EM algorithm. Iteration stops after this many iterations even if the convergence criterion is not satisfied. The default value is 25.

Possible distribution assumptions are as follows:

TDF=n. *Student's* t *distribution with* n *degrees of freedom.* The degrees of freedom must be specified if you use this keyword. The degrees of freedom must be an integer that is greater than or equal to 2.

LAMBDA=a. *Ratio of standard deviations of a mixed normal distribution.* Any positive real number can be specified.

PROPORTION=b. *Mixture proportion of two normal distributions.* Any real number between 0 and 1 can specify the mixture proportion of two normal distributions.

The following keyword produces a new data file:

OUTFILE='file'. *Specify a filename or previously declared dataset name.* Filenames should be enclosed in quotation marks and are stored in the working directory unless a path is included as part of the file specification. Datasets are available during the current session but are not available in subsequent

sessions unless you explicitly save them as data files. Missing values for predicted variables in the file are filled in by using the EM algorithm. (Note that the data that are completed with EM-based imputations will not in general reproduce the EM estimates from MVA.)

Examples

```
MVA VARIABLES=males to tuition
 /EM (OUTFILE='/colleges/emdata.sav').
```

- All variables on the variables list are included in the estimations.
- The output includes the means of the listed variables, a correlation matrix, and a covariance matrix.
- A new data file named *emdata.sav* with imputed values is saved in the */colleges* directory.

```
MVA VARIABLES=all
 /EM males msport WITH males msport gradrate facratio.
```

- For *males* and *msport*, the output includes a vector of means, a correlation matrix, and a covariance matrix.
- The values in the tables are calculated by using imputed values for *males* and *msport*. Existing observations for *males*, *msport*, *gradrate*, and *facratio* are used to impute the values that are used to estimate the means, correlations, and covariances.

```
MVA VARIABLES=males to tuition
 /EM verbal math WITH males msport gradrate facratio
  (TDF=3 OUTFILE '/colleges/emdata.sav').
```

- The analysis uses a *t* distribution with three degrees of freedom.
- A new data file named *emdata.sav* with imputed values is saved in the */colleges* directory.

REGRESSION Subcommand

The REGRESSION subcommand estimates missing values by using multiple linear regression. It can add a random component to the regression estimate. Output includes estimates of means, a covariance matrix, and a correlation matrix of the variables that are specified as predicted.

- By default, all of the variables that are specified as predictors (after WITH) are used in the estimation, but you can limit the number of predictors (independent variables) by using NPREDICTORS.
- Predicted and predictor variables, if specified, must be quantitative.
- By default, REGRESSION adds the observed residuals of a randomly selected complete case to the regression estimates. However, you can specify that the program add random normal, *t*, or no variates instead. The normal and *t* distributions are properly scaled, and the degrees of freedom can be specified for the *t* distribution.
- If the number of complete cases is less than half the total number of cases, the default ADDTYPE is NORMAL instead of RESIDUAL.
- You can save a data file with the missing values filled in. You must specify a filename and its complete path in single or double quotation marks.
- The criteria and OUTFILE specifications for the REGRESSION subcommand must be enclosed in a single pair of parentheses.

The criteria for the REGRESSION subcommand are as follows:

TOLERANCE=value. *Numerical accuracy control.* Helps eliminate predictor variables that are highly correlated with other predictor variables and would reduce the accuracy of the matrix inversions that are involved in the calculations. If a variable passes the tolerance criterion, it is eligible for inclusion. The smaller the tolerance, the more inaccuracy is tolerated. The default value is 0.001.

FLIMIT=n. F-*to-enter limit.* The minimum value of the *F* statistic that a variable must achieve in order to enter the regression estimation. You may want to change this limit, depending on the number of variables and the correlation structure of the data. The default value is 4.

NPREDICTORS=n. *Maximum number of predictor variables.* Limits the total number of predictors in the analysis. The analysis uses the stepwise selected *n* best predictors, entered in accordance with the tolerance. If *n*=0, it is equivalent to replacing each variable with its mean.

ADDTYPE . *Type of distribution from which the error term is randomly drawn.* Random errors can be added to the regression estimates before the means, correlations, and covariances are calculated. You can specify one of the following types:

- **RESIDUAL.** Error terms are chosen randomly from the observed residuals of complete cases to be added to the regression estimates.
- **NORMAL.** Error terms are randomly drawn from a distribution with the expected value 0 and the standard deviation equal to the square root of the mean squared error term (sometimes called the **root mean squared error**, or RMSE) of the regression.
- **T(n).** Error terms are randomly drawn from the t(n) distribution and scaled by the RMSE. The degrees of freedom can be specified in parentheses. If T is specified without a value, the default degrees of freedom is 5.
- **NONE.** Estimates are made from the regression model with no error term added.

The following keyword produces a new data file:

OUTFILE. *Specify a filename or previously declared dataset name.* Filenames should be enclosed in quotation marks and are stored in the working directory unless a path is included as part of the file specification. Datasets are available during the current session but are not available in subsequent sessions unless you explicitly save them as data files. Missing values for the dependent variables in the file are imputed (filled in) by using the regression algorithm.

Examples

```
MVA VARIABLES=males to tuition
 /REGRESSION (OUTFILE='/colleges/regdata.sav').
```

- All variables in the variables list are included in the estimations.
- The output includes the means of the listed variables, a correlation matrix, and a covariance matrix.
- A new data file named *regdata.sav* with imputed values is saved in the */colleges* directory.

```
MVA VARIABLES=males to tuition
 /REGRESSION males verbal math WITH males verbal math faculty
  (ADDTYPE = T(7)).
```

- The output includes the means of the listed variables, a correlation matrix, and a covariance matrix.
- A *t* distribution with 7 degrees of freedom is used to produce the randomly assigned additions to the estimates.

N OF CASES

```
N OF CASES n
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

Example

```
N OF CASES 100.
```

Overview

N OF CASES (alias N) limits the number of cases in the active dataset to the first *n* cases.

Basic Specification

The basic specification is N OF CASES followed by at least one space and a positive integer. Cases in the active dataset are limited to the specified number.

Syntax Rules

- To limit the number of cases for the next procedure only, use the TEMPORARY command before N OF CASES (see TEMPORARY).
- In some versions of the program, N OF CASES can be specified only after a active dataset is defined.

Operations

- N OF CASES takes effect as soon as it is encountered in the command sequence. Thus, special attention should be paid to the position of N OF CASES among commands. See the topic “Command Order” on page 36 for more information.
- N OF CASES limits the number of cases that are analyzed by all subsequent procedures in the session. The active dataset will have no more than *n* cases after the first data pass following the N OF CASES command. Any subsequent N OF CASES command that specifies a greater number of cases will be ignored.
- If N OF CASES specifies more cases than can actually be built, the program builds as many cases as possible.
- If N OF CASES is used with SAMPLE or SELECT IF, the program reads as many records as required to build the specified *n* cases. It makes no difference whetherN OF CASES precedes or follows SAMPLE or SELECT IF.

Example

```
GET FILE='/data/city.sav'.
N 100.
```

- N OF CASES limits the number of cases on the active dataset to the first 100 cases. Cases are limited for all subsequent analyses.

Example

```
DATA LIST FILE='/data/prsnnl.txt'
  / NAME 1-20 (A) AGE 22-23 SALARY 25-30.
N 25.
SELECT IF (SALARY GT 20000).
LIST.
```

- DATA LIST defines variables from file *prsnnl.txt*.
- N OF CASES limits the active dataset to 25 cases after cases have been selected by SELECT IF.
- SELECT IF selects only cases in which *SALARY* is greater than $20,000.

- LIST produces a listing of the cases in the active dataset. If the original active dataset has fewer than 25 cases in which *SALARY* is greater than 20,000, fewer than 25 cases will be listed.

Example

```
DATA LIST FILE='/data/prsnnl.txt'
  / NAME 1-20 (A) AGE 22-23 SALARY 25-30 DEPT 32.
LIST.
TEMPORARY.
N 25.
FREQUENCIES VAR=SALARY.
N 50.
FREQUENCIES VAR=AGE.
REPORT FORMAT=AUTO /VARS=NAME AGE SALARY /BREAK=DEPT
  /SUMMARY=MEAN.
```

- The first N OF CASES command is temporary. Only 25 cases are used in the first FREQUENCIES procedure.
- The second N OF CASES command is permanent. The second frequency table and the report are based on 50 cases from file *prsnnl.txt*. The active dataset now contains 50 cases (assuming that the original active dataset had at least that many cases).

NAIVEBAYES

NAIVEBAYES is available in Statistics Base Edition.

```
NAIVEBAYES dependent variable BY factor list WITH covariate list

[/EXCEPT VARIABLES=varlist]

[/FORCE [FACTORS=varlist] [COVARIATES=varlist]]

[/TRAININGSAMPLE {PERCENT=number  }]
                 {VARIABLE=varname}

[/SUBSET {MAXSIZE={AUTO** } [(BESTSUBSET={PSEUDOBIC })]}]
                  {integer}              {TESTDATA  }
         {EXACTSIZE=integer                          }
         {NOSELECTION                                }

[/CRITERIA [BINS={10**   }]
                 {integer}
           [MEMALLOCATE={1024** }]
                        {number }
           [TIMER={5**   }]]
                  {number}

[/MISSING USERMISSING={EXCLUDE**}]
                      {INCLUDE  }

[/PRINT [CPS**]  [EXCLUDED**]  [SUMMARY**]
        [SELECTED**]  [CLASSIFICATION**]
        [NONE]]

[/SAVE [PREDVAL[(varname)]]
       [PREDPROB[(rootname[:{25     }])]]]
                            {integer}

[/OUTFILE MODEL=file]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 14.0

- Command introduced.

Example

```
NAIVEBAYES default.
```

Overview

The NAIVEBAYES procedure can be used in three ways:

1. **Predictor selection followed by model building.** The procedure submits a set of predictor variables and selects a smaller subset. Based on the Naïve Bayes model for the selected predictors, the procedure then classifies cases.
2. **Predictor selection only.** The procedure selects a subset of predictors for use in subsequent predictive modeling procedures but does not report classification results.
3. **Model building only.** The procedure fits the Naïve Bayes classification model by using all input predictors.

NAIVEBAYES is available for categorical dependent variables only and is not intended for use with a very large number of predictors.

Options

Methods. The NAIVEBAYES procedure performs predictor selection followed by model building, or the procedure performs predictor selection only, or the procedure performs model building only.

Training and test data. NAIVEBAYES optionally divides the dataset into training and test samples. Predictor selection uses the training data to compute the underlying model, and either the training or the test data can be used to determine the "best" subset of predictors. If the dataset is partitioned, classification results are given for both the training and test samples. Otherwise, results are given for the training sample only.

Binning. The procedure automatically distributes scale predictors into 10 bins, but the number of bins can be changed.

Memory allocation. The NAIVEBAYES procedure automatically allocates 128MB of memory for storing training records when computing average log-likelihoods. The amount of memory that is allocated for this task can be modified.

Timer. The procedure automatically limits processing time to 5 minutes, but a different time limit can be specified.

Maximum or exact subset size. Either a maximum or an exact size can be specified for the subset of selected predictors. If a maximum size is used, the procedure creates a sequence of subsets, from an initial (smaller) subset to the maximum-size subset. The procedure then selects the "best" subset from this sequence.

Missing values. Cases with missing values for the dependent variable or for all predictors are excluded. The NAIVEBAYES procedure has an option for treating user-missing values of categorical variables as valid. User-missing values of scale variables are always treated as invalid.

Output. NAIVEBAYES displays pivot table output by default but offers an option for suppressing most such output. The procedure displays the lists of selected categorical and scale predictors in a text block. These lists can be copied for use in subsequent modeling procedures. The NAIVEBAYES procedure also optionally saves predicted values and probabilities based on the Naïve Bayes model.

Basic Specification

The basic specification is the NAIVEBAYES command followed by a dependent variable.

By default, NAIVEBAYES treats all variables — except the dependent variable and the weight variable if it is defined — as predictors, with the dictionary setting of each predictor determining its measurement level. NAIVEBAYES selects the "best" subset of predictors (based on the Naïve Bayes model) and then classifies cases by using the selected predictors. User-missing values are excluded and pivot table output is displayed by default.

Syntax Rules

- All subcommands are optional.
- Subcommands may be specified in any order.
- Only a single instance of each subcommand is allowed.
- An error occurs if a keyword is specified more than once within a subcommand.
- Parentheses, equal signs, and slashes that are shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not honored.

Operations

The NAIVEBAYES procedure automatically excludes cases and predictors with any of the following properties:

- Cases with a missing value for the dependent variable.
- Cases with missing values for all predictors.
- Predictors with missing values for all cases.
- Predictors with the same value for all cases.

The NAIVEBAYES procedure requires predictors to be categorical. Any scale predictors that are input to the procedure are temporarily binned into categorical variables for the procedure.

If predictor selection is used, the NAIVEBAYES procedure selects a subset of predictors that “best” predict the dependent variable, based on the training data. The procedure first creates a sequence of subsets, with an increasing number of predictors in each subset. The predictor that is added to each subsequent subset is the predictor that increases the average log-likelihood the most. The procedure uses simulated data to compute the average log-likelihood when the training dataset cannot fit into memory.

The final subset is obtained by using one of two approaches:

- By default, a maximum subset size is used. This approach creates a sequence of subsets from the initial subset to the maximum-size subset. The “best” subset is chosen by using a BIC-like criterion or a test data criterion.
- A particular subset size may be used to select the subset with the specified size.

If model building is requested, the NAIVEBAYES procedure classifies cases based on the Naïve Bayes model for the input or selected predictors, depending on whether predictor selection is requested. For a given case, the classification—or predicted category—is the dependent variable category with the highest posterior probability.

The NAIVEBAYES procedure uses the IBM SPSS Statistics random number generator in the following two scenarios: (1) if a percentage of cases in the active dataset is randomly assigned to the test dataset, and (2) if the procedure creates simulated data to compute the average log-likelihood when the training records cannot fit into memory. To ensure that the same results are obtained regardless of which scenario is in effect when NAIVEBAYES is invoked repeatedly, specify a seed on the SET command. If a seed is not specified, a default random seed is used, and results may differ across runs of the NAIVEBAYES procedure.

Frequency Weight

If a WEIGHT variable is in effect, its values are used as frequency weights by the NAIVEBAYES procedure.

- Cases with missing weights or weights that are less than 0.5 are not used in the analyses.
- The weight values are rounded to the nearest whole numbers before use. For example, 0.5 is rounded to 1, and 2.4 is rounded to 2.

Limitations

SPLIT FILE settings are ignored by the NAIVEBAYES procedure.

Examples

Predictor selection followed by model building

```
NAIVEBAYES default
  /EXCEPT VARIABLES=preddef1 preddef2 preddef3 training
  /TRAININGSAMPLE VARIABLE=training
  /SAVE PREDVAL PREDPROB.
```

- This analysis specifies *default* as the response variable.
- All other variables are to be considered as possible predictors, with the exception of *preddef1*, *preddef2*, *preddef3*, and *training*.
- Cases with a value of 1 on the variable *training* are assigned to the training sample and used to create the series of predictor subsets, while all other cases are assigned to the test sample and used to select the "best" subset.
- Model-predicted values of *default* are saved to the variable *PredictedValue*.
- Model-estimated probabilities for the values of *default* are saved to the variables *PredictedProbability_1* and *PredictedProbability_2*.

For discussion of output from this syntax and to place the example in context, see

Predictor selection only

```
NAIVEBAYES default
  /EXCEPT VARIABLES=preddef1 preddef2 preddef3 validate
  /TRAININGSAMPLE VARIABLE=validate
  /SUBSET EXACTSIZE=5
  /PRINT CLASSIFICATION=NO.
```

- The NAIVEBAYES procedure treats *default* as the dependent variable and selects a subset of five predictors from all other variables, with the exception of *preddef1*, *preddef2*, *preddef3*, and *validate*.

Variable Lists

The variable lists specify the dependent variable, any categorical predictors (also known as factors), and any scale predictors (also known as covariates).

- The dependent variable must be the first specification on the NAIVEBAYES command.
- The dependent variable may not be the weight variable.
- The dependent variable is the only required specification on the NAIVEBAYES command.
- The dependent variable must have a dictionary setting of ordinal or nominal. In either case, NAIVEBAYES treats the dependent variable as categorical.
- The names of the factors, if any, must be preceded by the keyword BY.
- If keyword BY is specified with no factors, a warning is issued and the keyword is ignored.
- The names of covariates must be preceded by the keyword WITH.
- If keyword WITH is specified with no covariates, a warning is issued and the keyword is ignored.
- If the dependent variable or the weight variable is specified within a factor list or a covariate list, the variable is ignored in the list.
- All variables that are specified within a factor or covariate list must be unique. If duplicate variables are specified within a list, the duplicates are ignored.
- If duplicate variables are specified across the factor and covariate lists, an error is issued.
- The universal keywords TO and ALL may be specified in the factor and covariate lists.
- If the BY and WITH keywords are not specified, all variables in the active dataset—except the dependent variable, the weight variable, and any variables that are specified on the EXCEPT subcommand—are treated as predictors. If the dictionary setting of a predictor is nominal or ordinal, the predictor is treated as a factor. If the dictionary setting is scale, the predictor is treated as a covariate. (Note that any variables on the FORCE subcommand are still forced into each subset of selected predictors.)
- The dependent variable and factor variables can be numeric or string.
- The covariates must be numeric.

EXCEPT Subcommand

The EXCEPT subcommand lists any variables that the NAIVEBAYES procedure should exclude from the factor or covariate lists on the command line. This subcommand is useful if the factor or covariate lists contain a large number of variables--specified by using the TO or ALL keyword, for example--but a few variables (e.g., Case ID or a weight variable) should be excluded.

- The EXCEPT subcommand ignores the following types of variables if they are specified: Duplicate variables; the dependent variable; variables that are not specified on the command line's factor or covariate lists; and variables that are specified on the FORCE subcommand.
- There is no default variable list on the EXCEPT subcommand.

FORCE Subcommand

The FORCE subcommand specifies any predictors that will be in the initial predictor subset and all subsequent predictor subsets. The specified predictors are considered important and will be in the final subset irrespective of any other chosen predictors.

- Variables that are specified on the FORCE subcommand do not need to be specified in the variable lists on the command line.
- The FORCE subcommand overrides variable lists on the command line and overrides the EXCEPT subcommand. If a variable specified on the FORCE subcommand is also specified on the command line or the EXCEPT subcommand, the variable is forced into all subsets.
- There is no default list of forced variables; the default initial subset is the empty set.

FACTORS Keyword

The FACTORS keyword specifies any factors that should be forced into each subset.

- If duplicate variables are specified, the duplicates are ignored.
- The specified variables may not include the dependent variable, the weight variable, or any variable that is specified on the COVARIATES keyword.
- Specified variables may be numeric or string.

COVARIATES Keyword

The COVARIATES keyword specifies any covariates that should be forced into each subset.

- If duplicate variables are specified, the duplicates are ignored.
- The specified variables may not include the dependent variable, the weight variable, or any variable that is specified on the FACTORS keyword.
- Specified variables must be numeric.

TRAININGSAMPLE Subcommand

The TRAININGSAMPLE subcommand indicates the method of partitioning the active dataset into training and test samples. You can specify either a percentage of cases to assign to the training sample, or you can specify a variable that indicates whether a case is assigned to the training sample.

- If TRAININGSAMPLE is not specified, all cases in the active dataset are treated as training data records.
- TRAININGSAMPLE is automatically excluded from factor and covariate variable lists on the NAIVEBAYES command line.

PERCENT Keyword

The PERCENT keyword specifies the percentage of cases in the active dataset to randomly assign to the training sample. All other cases are assigned to the test sample. The percentage must be a number that is greater than 0 and less than 100. There is no default percentage.

If a weight variable is defined, the PERCENT keyword may not be used.

VARIABLE Keyword

The VARIABLE keyword specifies a variable that indicates which cases in the active dataset are assigned to the training sample. Cases with a value of 1 on the variable are assigned to the training sample. All other cases are assigned to the test sample.

- The specified variable may not be the dependent variable, the weight variable, any variable that is specified in the factor or covariate lists of the command line, or any variable that is specified in the factor or covariate lists of the FORCE subcommand.
- The variable must be numeric.

SUBSET Subcommand

The SUBSET subcommand gives settings for the subset of selected predictors.

- There are three mutually exclusive settings: (1) specify a maximum subset size and a method of selecting the best subset, (2) specify an exact subset size, or (3) do not specify a selection.
- Only one of the keywords MAXSIZE, EXACTSIZE, or NOSELECTION may be specified. The BESTSUBSET option is available only if MAXSIZE is specified.

MAXSIZE Keyword

The MAXSIZE keyword specifies the maximum subset size to use when creating the sequence of predictor subsets. The MAXSIZE value is the size of the largest subset beyond any predictors that were forced via the FORCE subcommand. If no predictors are forced, the MAXSIZE value is simply the size of the largest subset.

- Value AUTO indicates that the number should be computed automatically. Alternatively, a positive integer may be specified. The integer must be less than or equal to the number of unique predictors on the NAIVEBAYES command.
- By default, MAXSIZE is used and AUTO is the default value.

BESTSUBSET Keyword

The BESTSUBSET keyword indicates the criterion for finding the best subset when a maximum subset size is used.

- This keyword is honored only if the MAXSIZE keyword is in effect and must be given in parentheses immediately following the MAXSIZE specification.

PSEUDOBIC. *Use the pseudo-BIC criterion.* The pseudo-BIC criterion is based on the training sample. If the active dataset is not partitioned into training and test samples, PSEUDOBIC is the default. If the active dataset is partitioned, PSEUDOBIC is available but is not the default.

TESTDATA. *Use the test data criterion.* The test data criterion is based on the test sample. If the active dataset is partitioned into training and test samples, TESTDATA is the default. If the active dataset is not partitioned, TESTDATA may not be specified.

EXACTSIZE Keyword

The EXACTSIZE keyword specifies a particular subset size to use. The EXACTSIZE value is the size of the subset beyond any predictors forced via the FORCE subcommand. If no predictors are forced, then the EXACTSIZE value is simply the size of the subset.

- A positive integer may be specified. The integer must be less than the number of unique predictors on the NAIVEBAYES command.
- There is no default value.

NOSELECTION Keyword

The NOSELECTION keyword indicates that all predictors that are specified on the NAIVEBAYES command—excluding any predictors that are also specified on the EXCEPT subcommand—are included in the final subset. This specification is useful if the NAIVEBAYES procedure is used for model building but not predictor selection.

CRITERIA Subcommand

The CRITERIA subcommand specifies computational and resource settings for the NAIVEBAYES procedure.

BINS Keyword

The BINS keyword specifies the number of bins to use when dividing the domain of a scale predictor into equal-width bins. A positive integer greater than 1 may be specified. The default is 10.

MEMALLOCATE Keyword

The MEMALLOCATE keyword specifies the maximum amount of memory in megabytes (MB) that the NAIVEBAYES procedure uses to store training data records when computing the average log-likelihood. If the amount of memory that is required to store records is larger, simulated data are used instead.

- Any number that is greater than or equal to 4 may be specified. Consult your system administrator for the largest value that can be specified on your system. The default is 1024.

TIMER Keyword

The TIMER keyword specifies the maximum number of minutes during which the NAIVEBAYES procedure can run. If the time limit is exceeded, the procedure is terminated and no results are given. Any number that is greater than or equal to 0 may be specified. Specifying 0 turns the timer off completely. The default is 5.

MISSING Subcommand

The MISSING subcommand controls whether user-missing values for categorical variables are treated as valid values. By default, user-missing values for categorical variables are treated as invalid.

- User-missing values for scale variables are always treated as invalid.
- System-missing values for any variables are always treated as invalid.

USERMISSING=EXCLUDE. *User-missing values for categorical variables are treated as invalid values.* This setting is the default.

USERMISSING=INCLUDE. *User-missing values for categorical variables are treated as valid values.*

PRINT Subcommand

The PRINT subcommand indicates the statistical output to display.

CPS. *Case processing summary.* The table summarizes the number of cases that are included and excluded in the analysis. This table is shown by default.

EXCLUDED. *Predictors excluded due to missing or constant values for all cases.* The table lists excluded predictors by type (categorical or scale) and the reasons for being excluded.

SUMMARY. *Statistical summary of the sequence of predictor subsets.* This table is shown by default. The SUMMARY keyword is ignored if NOSELECTION is specified on the SUBSET subcommand.

SELECTED. *Selected predictors by type (categorical or scale).* This table is shown by default. The SELECTED keyword is ignored if NOSELECTION is specified on the SUBSET subcommand.

CLASSIFICATION. *Classification table.* The table gives the number of cases that are classified correctly and incorrectly for each dependent variable category. If test data are defined, classification results are given for the training and the test samples. If test data are not defined, classification results are given only for the training sample. This table is shown by default.

NONE. *Suppress all displayed output except the Notes table and any warnings.* This keyword may not be specified with any other keywords.

SAVE Subcommand

The SAVE subcommand writes optional temporary variables to the active dataset.

PREDVAL(varname). *Predicted value.* The predicted value is the dependent variable category with the highest posterior probability as estimated by the Naïve Bayes model. A valid variable name must be specified. The default variable name is *PredictedValue*.

PREDPROB(rootname:n). *Predicted probability.* The predicted probabilities of the first *n* categories of the dependent variable are saved. Suffixes are added to the root name to get a group of variable names that correspond to the dependent variable categories. If a root name is specified, it must be a valid variable name. The root name can be followed by a colon and a positive integer that indicates the number of probabilities to save. The default root name is *PredictedProbability*. The default *n* is 25. To specify *n* without a root name, enter a colon before the number.

OUTFILE Subcommand

The OUTFILE subcommand writes the Naïve Bayes model to an XML file. The Naïve Bayes model is based on the training sample even if the active dataset is partitioned into training and test samples.

A valid file name must be specified on the MODEL keyword.

NEW FILE

```
NEW FILE
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Overview

Basic Specification

NEW FILE is always specified by itself. No other keyword is allowed.

Operations

- NEW FILE creates a new, blank active dataset. The command takes effect as soon as it is encountered.
- When you build an active dataset with GET, DATA LIST, or other file-definition commands (such as ADD FILES or MATCH FILES), the active dataset is automatically replaced. It is not necessary to specify NEW FILE.

NLR

NLR and CNLR are available in Custom Tables and Advanced Statistics.

```
MODEL PROGRAM parameter=value [parameter=value ...]
transformation commands

[DERIVATIVES
transformation commands]

[CLEAR MODEL PROGRAMS]
```

Procedure CNLR (Constrained Nonlinear Regression):

```
[CONSTRAINED FUNCTIONS
transformation commands]

CNLR dependent var

 [/FILE=file]   [/OUTFILE=file]

 [/PRED=varname]

 [/SAVE [PRED] [RESID[(varname)]] [DERIVATIVES] [LOSS]]

 [/CRITERIA=[ITER n] [MITER n] [CKDER {0.5**}]
                                      {n    }
            [ISTEP {1E+20**}] [FPR n] [LFTOL n]
                   {n      }
            [LSTOL n] [STEPLIMIT {2**}] [NFTOL n]
                                 {n  }
            [FTOL n] [OPTOL n] [CRSHTOL {.01**}]]
                                        {n    }

 [/BOUNDS=expression, expression, ...]

 [/LOSS=varname]

 [/BOOTSTRAP [=n]]
```

Procedure NLR (Nonlinear Regression):

```
NLR dependent var

 [/FILE=file]   [/OUTFILE=file]

 [/PRED=varname]

 [/SAVE [PRED] [RESID [(varname)] [DERIVATIVES]]

 [/CRITERIA=[ITER {100**}] [CKDER {0.5**}]
                  {n    }        {n    }
            [SSCON {1E-8**}]  [PCON {1E-8**}]  [RCON {1E-8**}]]
                   {n     }         {n     }         {n     }
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
MODEL PROGRAM A=.6.
COMPUTE PRED=EXP(A*X).

NLR Y.
```

Overview

Nonlinear regression is used to estimate parameter values and regression statistics for models that are not linear in their parameters. There are two procedures for estimating nonlinear equations. CNLR (constrained nonlinear regression), which uses a sequential quadratic programming algorithm, is applicable for both constrained and unconstrained problems. NLR (nonlinear regression), which uses a Levenberg-Marquardt algorithm, is applicable only for unconstrained problems.

CNLR is more general. It allows linear and nonlinear constraints on any combination of parameters. It will estimate parameters by minimizing any smooth loss function (objective function) and can optionally

compute bootstrap estimates of parameter standard errors and correlations. The individual bootstrap parameter estimates can optionally be saved in a separate IBM SPSS Statistics data file.

Both programs estimate the values of the parameters for the model and, optionally, compute and save predicted values, residuals, and derivatives. Final parameter estimates can be saved in IBM SPSS Statistics data files and used in subsequent analyses.

CNLR and NLR use much of the same syntax. Some of the following sections discuss features that are common to both procedures. In these sections, the notation [C]NLR means that either the CNLR or NLR procedure can be specified. Sections that apply only to CNLR or only to NLR are clearly identified.

Options

The Model. You can use any number of transformation commands under MODEL PROGRAM to define complex models.

Derivatives. You can use any number of transformation commands under DERIVATIVES to supply derivatives.

Adding Variables to Active Dataset. You can add predicted values, residuals, and derivatives to the active dataset with the SAVE subcommand.

Writing Parameter Estimates to a New Data File. You can save final parameter estimates as an external IBM SPSS Statistics data file by using the OUTFILE subcommand; you can retrieve them in subsequent analyses by using the FILE subcommand.

Controlling Model-Building Criteria. You can control the iteration process that is used in the regression with the CRITERIA subcommand.

Additional CNLR Controls. For CNLR, you can impose linear and nonlinear constraints on the parameters with the BOUNDS subcommand. Using the LOSS subcommand, you can specify a loss function for CNLR to minimize and, using the BOOTSTRAP subcommand, you can provide bootstrap estimates of the parameter standard errors, confidence intervals, and correlations.

Basic Specification

The basic specification requires three commands: MODEL PROGRAM, COMPUTE (or any other computational transformation command), and [C]NLR.

- The MODEL PROGRAM command assigns initial values to the parameters and signifies the beginning of the model program.
- The computational transformation command generates a new variable to define the model. The variable can take any legitimate name, but if the name is not *PRED*, the PRED subcommand will be required.
- The [C]NLR command provides the regression specifications. The minimum specification is the dependent variable.
- By default, the residual sum of squares and estimated values of the model parameters are displayed for each iteration. Statistics that are generated include regression and residual sums of squares and mean squares, corrected and uncorrected total sums of squares, R^2, parameter estimates with their asymptotic standard errors and 95% confidence intervals, and an asymptotic correlation matrix of the parameter estimates.

Command Order

- The model program, beginning with the MODEL PROGRAM command, must precede the [C]NLR command.
- The derivatives program (when used), beginning with the DERIVATIVES command, must follow the model program but precede the [C]NLR command.
- The constrained functions program (when used), beginning with the CONSTRAINED FUNCTIONS command, must immediately precede the CNLR command. The constrained functions program cannot be used with the NLR command.
- The CNLR command must follow the block of transformations for the model program and the derivatives program when specified; the CNLR command must also follow the constrained functions program when specified.

- Subcommands on [C]NLR can be named in any order.

Syntax Rules

- The FILE, OUTFILE, PRED, and SAVE subcommands work the same way for both CNLR and NLR.
- The CRITERIA subcommand is used by both CNLR and NLR, but iteration criteria are different. Therefore, the CRITERIA subcommand is documented separately for CNLR and NLR.
- The BOUNDS, LOSS, and BOOTSTRAP subcommands can be used only with CNLR. They cannot be used with NLR.

Operations

- By default, the predicted values, residuals, and derivatives are created as temporary variables. To save these variables, use the SAVE subcommand.

Weighting Cases

- If case weighting is in effect, [C]NLR uses case weights when calculating the residual sum of squares and derivatives. However, the degrees of freedom in the ANOVA table are always based on unweighted cases.
- When the model program is first invoked for each case, the weight variable's value is set equal to its value in the active dataset. The model program may recalculate that value. For example, to effect a robust estimation, the model program may recalculate the weight variable's value as an inverse function of the residual magnitude. [C]NLR uses the weight variable's value after the model program is executed.

Missing Values

Cases with missing values for any of the dependent or independent variables that are named on the [C]NLR command are excluded.

- Predicted values, but not residuals, can be calculated for cases with missing values on the dependent variable.
- [C]NLR ignores cases that have missing, negative, or zero weights. The procedure displays a warning message if it encounters any negative or zero weights at any time during its execution.
- If a variable that is used in the model program or the derivatives program is omitted from the independent variable list on the [C]NLR command, the predicted value and some or all of the derivatives may be missing for every case. If this situation happens, an error message is generated.

Examples

```
MODEL PROGRAM A=.5 B=1.6.
COMPUTE PRED=A*SPEED**B.

DERIVATIVES.
COMPUTE D.A=SPEED**B.
COMPUTE D.B=A*LN(SPEED)*SPEED**B.

NLR STOP.
```

- MODEL PROGRAM assigns values to the model parameters *A* and *B*.
- COMPUTE generates the variable *PRED* to define the nonlinear model using parameters *A* and *B* and the variable *SPEED* from the active dataset. Because this variable is named *PRED*, the PRED subcommand is not required on NLR.
- DERIVATIVES indicates that calculations for derivatives are being supplied.
- The two COMPUTE statements on the DERIVATIVES transformations list calculate the derivatives for the parameters *A* and *B*. If either parameter had been omitted, NLR would have calculated it numerically.

- NLR specifies *STOP* as the dependent variable. It is not necessary to specify *SPEED* as the independent variable because it has been used in the model and derivatives programs.

MODEL PROGRAM Command

The MODEL PROGRAM command assigns initial values to the parameters and signifies the beginning of the model program. The model program specifies the nonlinear equation that is chosen to model the data. There is no default model.

- The model program is required and must precede the [C]NLR command.
- The MODEL PROGRAM command must specify all parameters in the model program. Each parameter must be individually named. Keyword TO is not allowed.
- Parameters can be assigned any acceptable variable name. However, if you intend to write the final parameter estimates to a file with the OUTFILE subcommand, do not use the name *SSE* or *NCASES* (see "OUTFILE Subcommand " on page 1240).
- Each parameter in the model program must have an assigned value. The value can be specified on MODEL PROGRAM or read from an existing parameter data file named on the FILE subcommand.
- Zero should be avoided as an initial value because it provides no information about the scale of the parameters. This situation is especially true for CNLR.
- The model program must include at least one command that uses the parameters and the independent variables (or preceding transformations of these) to calculate the predicted value of the dependent variable. This predicted value defines the nonlinear model. There is no default model.
- By default, the program assumes that *PRED* is the name assigned to the variable for the predicted values. If you use a different variable name in the model program, you must supply the name on the PRED subcommand (see "PRED Subcommand " on page 1241).
- In the model program, you can assign a label to the variable holding predicted values and also change its print and write formats, but you should not specify missing values for this variable.
- You can use any computational commands (such as COMPUTE, IF, DO IF, LOOP, END LOOP, END IF, RECODE, or COUNT) or output commands (WRITE, PRINT, or XSAVE) in the model program, but you cannot use input commands (such as DATA LIST, GET, MATCH FILES, or ADD FILES).
- Transformations in the model program are used only by [C]NLR, and they do not affect the active dataset. The parameters that are created by the model program do not become a part of the active dataset. Permanent transformations should be specified before the model program.

Caution: Initial Values

The selection of good initial values for the parameters in the model program is very important to the operation of [C]NLR. The selection of poor initial values can result in no solution, a local solution rather than a general solution, or a physically impossible solution.

Example

```
MODEL PROGRAM A=10 B=1 C=5 D=1.
COMPUTE PRED= A*exp(B*X) + C*exp(D*X).
```

- The MODEL PROGRAM command assigns starting values to the four parameters *A, B, C,* and *D*.
- COMPUTE defines the model to be fit as the sum of two exponentials.

DERIVATIVES Command

The optional DERIVATIVES command signifies the beginning of the derivatives program. The derivatives program contains transformation statements for computing some or all of the derivatives of the model. The derivatives program must follow the model program but precede the [C]NLR command.

If the derivatives program is not used, [C]NLR numerically estimates derivatives for all the parameters. Providing derivatives reduces computation time and, in some situations, may result in a better solution.

- The DERIVATIVES command has no further specifications but must be followed by the set of transformation statements that calculate the derivatives.
- You can use any computational commands (such as COMPUTE, IF, DO IF, LOOP, END LOOP, END IF, RECODE, or COUNT) or output commands (WRITE, PRINT, or XSAVE) in the derivatives program, but you cannot use input commands (such as DATA LIST, GET, MATCH FILES, or ADD FILES).
- To name the derivatives, specify the prefix *D.* before each parameter name. For example, the derivative name for the parameter *PARM1* must be *D.PARM1*.
- When a derivative has been calculated by a transformation, the variable for that derivative can be used in subsequent transformations.
- You do not need to supply all of the derivatives. Those derivatives that are not supplied will be estimated by the program. During the first iteration of the nonlinear estimation procedure, derivatives that are calculated in the derivatives program are compared with numerically calculated derivatives. This process serves as a check on the supplied values (see "CRITERIA Subcommand " on page 1242).
- Transformations in the derivatives program are used by [C]NLR only and do not affect the active dataset.
- For NLR, the derivative of each parameter must be computed with respect to the predicted function (see "LOSS Subcommand " on page 1245).

Example

```
MODEL PROGRAM A=1, B=0, C=1, D=0.
COMPUTE PRED = A * exp(B * X) + C * exp(D * X).
DERIVATIVES.
COMPUTE D.A = exp (B * X).
COMPUTE D.B = A * exp (B * X) * X.
COMPUTE D.C = exp (D * X).
COMPUTE D.D = C * exp (D * X) * X.
```

- The derivatives program specifies derivatives of the PRED function for the sum of the two exponentials in the model described by the following equation:

$Y=Ae^{Bx}+Ce^{Dx}$

Example

```
DERIVATIVES.
COMPUTE D.A = exp (B * X).
COMPUTE D.B = A * X * D.A.
COMPUTE D.C = exp (D * X).
COMPUTE D.D = C * X * D.C.
```

- This example is an alternative way to express the same derivatives program that was specified in the previous example.

CONSTRAINED FUNCTIONS Command

The optional CONSTRAINED FUNCTIONS command signifies the beginning of the constrained functions program, which specifies nonlinear constraints. The constrained functions program is specified after the model program and the derivatives program (when used). It can only be used with, and must precede, the CNLR command. See the topic "BOUNDS Subcommand " on page 1245 for more information.

Example

```
MODEL PROGRAM A=.5 B=1.6.
COMPUTE PRED=A*SPEED**B.

CONSTRAINED FUNCTIONS.
COMPUTE CF=A-EXP(B).

CNLR STOP
  /BOUNDS CF LE 0.
```

CLEAR MODEL PROGRAMS Command

CLEAR MODEL PROGRAMS deletes all transformations that are associated with the previously submitted model program, derivative program, and/or constrained functions program. It is primarily used in interactive mode to remove temporary variables that were created by these programs without affecting the active dataset or variables that were created by other transformation programs or temporary programs. It allows you to specify new models, derivatives, or constrained functions without having to run [C]NLR.

It is not necessary to use this command if you have already executed the [C]NLR procedure. Temporary variables that are associated with the procedure are automatically deleted.

CNLR and NLR Commands

Either the CNLR or the NLR command is required to specify the dependent and independent variables for the nonlinear regression.

- For either CNLR or NLR, the minimum specification is a dependent variable.
- Only one dependent variable can be specified. It must be a numeric variable in the active dataset and cannot be a variable that is generated by the model or the derivatives program.

OUTFILE Subcommand

OUTFILE stores final parameter estimates for use on a subsequent [C]NLR command. The only specification on OUTFILE is the target file. Some or all of the values from this file can be read into a subsequent [C]NLR procedure with the FILE subcommand. The parameter data file that is created by OUTFILE stores the following variables:

- All of the split-file variables. OUTFILE writes one case of values for each split-file group in the active dataset.
- All of the parameters named on the MODEL PROGRAM command.
- The labels, formats, and missing values of the split-file variables and parameters defined for them previous to their use in the [C]NLR procedure.
- The sum of squared residuals (named *SSE*). *SSE* has no labels or missing values. The print and write format for *SSE* is F10.8.
- The number of cases on which the analysis was based (named *NCASES*). *NCASES* has no labels or missing values. The print and write format for *NCASES* is F8.0.

When OUTFILE is used, the model program cannot create variables named *SSE* or *NCASES*.

Example

```
MODEL PROGRAM A=.5 B=1.6.
COMPUTE PRED=A*SPEED**B.
NLR STOP /OUTFILE=PARAM.
```

- OUTFILE generates a parameter data file containing one case for four variables: *A*, *B*, *SSE*, and *NCASES*.

FILE Subcommand

FILE reads starting values for the parameters from a parameter data file that was created by an OUTFILE subcommand from a previous [C]NLR procedure. When starting values are read from a file, they do not have to be specified on the MODEL PROGRAM command. Rather, the MODEL PROGRAM command simply names the parameters that correspond to the parameters in the data file.

- The only specification on FILE is the file that contains the starting values.

- Some new parameters may be specified for the model on the MODEL PROGRAM command while other parameters are read from the file that is specified on the FILE subcommand.
- You do not have to name the parameters on MODEL PROGRAM in the order in which they occur in the parameter data file. In addition, you can name a partial list of the variables that are contained in the file.
- If the starting value for a parameter is specified on MODEL PROGRAM, the specification overrides the value that is read from the parameter data file.
- If split-file processing is in effect, the starting values for the first subfile are taken from the first case of the parameter data file. Subfiles are matched with cases in order until the starting-value file runs out of cases. All subsequent subfiles use the starting values for the last case.
- To read starting values from a parameter data file and then replace those values with the final results from [C]NLR, specify the same file on the FILE and OUTFILE subcommands. The input file is read completely before anything is written in the output file.

Example

```
MODEL PROGRAM A B C=1 D=3.
COMPUTE PRED=A*SPEED**B + C*SPEED**D.
NLR STOP /FILE=PARAM /OUTFILE=PARAM.
```

- MODEL PROGRAM names four of the parameters that are used to calculate *PRED* but assigns values to only *C* and *D*. The values of *A* and *B* are read from the existing data file *PARAM*.
- After NLR computes the final estimates of the four parameters, OUTFILE writes over the old input file. If, in addition to these new final estimates, the former starting values of *A* and *B* are still desired, specify a different file on the OUTFILE subcommand.

PRED Subcommand

PRED identifies the variable holding the predicted values.

- The only specification is a variable name, which must be identical to the variable name that is used to calculate predicted values in the model program.
- If the model program names the variable *PRED*, the PRED subcommand can be omitted. Otherwise, the PRED subcommand is required.
- The variable for predicted values is not saved in the active dataset unless the SAVE subcommand is used.

Example

```
MODEL PROGRAM A=.5 B=1.6.
COMPUTE PSTOP=A*SPEED**B.
NLR STOP /PRED=PSTOP.
```

- COMPUTE in the model program creates a variable named *PSTOP* to temporarily store the predicted values for the dependent variable *STOP*.
- PRED identifies *PSTOP* as the variable that is used to define the model for the NLR procedure.

SAVE Subcommand

SAVE is used to save the temporary variables for the predicted values, residuals, and derivatives that are created by the model and the derivatives programs.

- The minimum specification is a single keyword.
- The variables to be saved must have unique names on the active dataset. If a naming conflict exists, the variables are not saved.
- Temporary variables—for example, variables that are created after a TEMPORARY command and parameters that are specified by the model program—are not saved in the active dataset. They will not cause naming conflicts.

The following keywords are available and can be used in any combination and in any order. The new variables are always appended to the active dataset in the order in which these keywords are presented here:

PRED. *Save the predicted values.* The variable's name, label, and formats are those specified for it (or assigned by default) in the model program.

RESID [(varname)]. *Save the residuals variable.* You can specify a variable name in parentheses following the keyword. If no variable name is specified, the name of this variable is the same as the specification that you use for this keyword. For example, if you use the three-character abbreviation RES, the default variable name will be *RES*. The variable has the same print and write format as the predicted values variable that is created by the model program. It has no variable label and no user-defined missing values. It is system-missing for any case in which either the dependent variable is missing or the predicted value cannot be computed.

DERIVATIVES. *Save the derivative variables.* The derivative variables are named with the prefix *D.* to the first six characters of the parameter names. Derivative variables use the print and write formats of the predicted values variable and have no value labels or user-missing values. Derivative variables are saved in the same order as the parameters named on MODEL PROGRAM. Derivatives are saved for all parameters, whether or not the derivative was supplied in the derivatives program.

LOSS. *Save the user-specified loss function variable.* This specification is available only with CNLR and only if the LOSS subcommand has been specified.

Asymptotic standard errors of predicted values and residuals, and special residuals used for outlier detection and influential case analysis are not provided by the [C]NLR procedure. However, for a squared loss function, the asymptotically correct values for all these statistics can be calculated by using the SAVE subcommand with [C]NLR and then using the REGRESSION procedure. In REGRESSION, the dependent variable is still the same, and derivatives of the model parameters are used as independent variables. Casewise plots, standard errors of prediction, partial regression plots, and other diagnostics of the regression are valid for the nonlinear model.

Example

```
MODEL PROGRAM A=.5 B=1.6.
COMPUTE PSTOP=A*SPEED**B.
NLR STOP /PRED=PSTOP
  /SAVE=RESID(RSTOP) DERIVATIVES PRED.
REGRESSION VARIABLES=STOP D.A D.B /ORIGIN
  /DEPENDENT=STOP /ENTER D.A D.B /RESIDUALS.
```

- The SAVE subcommand creates the residuals variable *RSTOP* and the derivative variables *D.A* and *D.B*.
- Because the PRED subcommand identifies *PSTOP* as the variable for predicted values in the nonlinear model, keyword PRED on SAVE adds the variable *PSTOP* to the active dataset.
- The new variables are added to the active dataset in the following order: *PSTOP, RSTOP, D.A,* and *D.B*.
- The subcommand RESIDUALS for REGRESSION produces the default analysis of residuals.

CRITERIA Subcommand

CRITERIA controls the values of the cutoff points that are used to stop the iterative calculations in [C]NLR.

- The minimum specification is any of the criteria keywords and an appropriate value. The value can be specified in parentheses after an equals sign, a space, or a comma. Multiple keywords can be specified in any order. Defaults are in effect for keywords that are not specified.
- Keywords available for CRITERIA differ between CNLR and NLR and are discussed separately. However, with both CNLR and NLR, you can specify the critical value for derivative checking.

Checking Derivatives for CNLR and NLR

Upon entering the first iteration, [C]NLR always checks any derivatives that are calculated on the derivatives program by comparing them with numerically calculated derivatives. For each comparison, it

computes an agreement score. A score of 1 indicates agreement to machine precision; a score of 0 indicates definite disagreement. If a score is less than 1, either an incorrect derivative was supplied or there were numerical problems in estimating the derivative. The lower the score, the more likely it is that the supplied derivatives are incorrect. Highly correlated parameters may cause disagreement even when a correct derivative is supplied. Be sure to check the derivatives if the agreement score is not 1.

During the first iteration, [C]NLR checks each derivative score. If any score is below 1, it begins displaying a table to show the worst (lowest) score for each derivative. If any score is below the critical value, the program stops.

To specify the critical value, use the following keyword on CRITERIA:

CKDER n. *Critical value for derivative checking.* Specify a number between 0 and 1 for *n*. The default is 0.5. Specify 0 to disable this criterion.

Iteration Criteria for CNLR

The CNLR procedure uses NPSOL (Version 4.0) Fortran Package for Nonlinear Programming [49]. The CRITERIA subcommand of CNLR gives the control features of NPSOL. The following section summarizes the NPSOL documentation.

CNLR uses a sequential quadratic programming algorithm, with a quadratic programming subproblem to determine the search direction. If constraints or bounds are specified, the first step is to find a point that is feasible with respect to those constraints. Each major iteration sets up a quadratic program to find the search direction, *p.* Minor iterations are used to solve this subproblem. Then, the major iteration determines a steplength α by a line search, and the function is evaluated at the new point. An optimal solution is found when the optimality tolerance criterion is met.

The CRITERIA subcommand has the following keywords when used with CNLR:

ITER n. *Maximum number of major iterations.* Specify any positive integer for *n*. The default is max(50, $3(p+m_L)+10m_N$), where p is the number of parameters, m_L is the number of linear constraints, and m_N is the number of nonlinear constraints. If the search for a solution stops because this limit is exceeded, CNLR issues a warning message.

MINORITERATION n. *Maximum number of minor iterations.* Specify any positive integer. This value is the number of minor iterations allowed within each major iteration. The default is max(50, $3(n+m_L+m_N)$).

CRSHTOL n. *Crash tolerance.* CRSHTOL is used to determine whether initial values are within their specified bounds. Specify any value between 0 and 1. The default value is 0.01.

STEPLIMIT n. *Step limit.* The CNLR algorithm does not allow changes in the length of the parameter vector to exceed a factor of *n*. The limit prevents very early steps from going too far from good initial estimates. Specify any positive value. The default value is 2.

FTOLERANCE n. *Feasibility tolerance.* This value is the maximum absolute difference allowed for both linear and nonlinear constraints for a solution to be considered feasible. Specify any value greater than 0. The default value is the square root of your machine's epsilon.

LFTOLERANCE n. *Linear feasibility tolerance.* If specified, this value overrides FTOLERANCE for linear constraints and bounds. Specify any value greater than 0. The default value is the square root of your machine's epsilon.

NFTOLERANCE n. *Nonlinear feasibility tolerance.* If specified, this value overrides FTOLERANCE for nonlinear constraints. Specify any value greater than 0. The default value is the square root of your machine's epsilon.

LSTOLERANCE n. *Line search tolerance.* This value must be between 0 and 1 (but not including 1). It controls the accuracy required of the line search that forms the innermost search loop. The default value, 0.9, specifies an inaccurate search. This setting is appropriate for many problems, particularly if nonlinear

[49] Gill, P. E., W. M. Murray, M. A. Saunders, and M. H. Wright. 1986. *User's guide for NPSOL (version 4.0): A FORTRAN package for nonlinear programming. Technical Report SOL 86-2*. Stanford University: Department of Operations Research.

constraints are involved. A smaller positive value, corresponding to a more accurate line search, may give better performance if there are no nonlinear constraints, all (or most) derivatives are supplied in the derivatives program, and the data fit in memory.

OPTOLERANCE n. *Optimality tolerance.* If an iteration point is a feasible point, and the next step will not produce a relative change in either the parameter vector or the objective function of more than the square root of OPTOLERANCE, an optimal solution has been found. OPTOLERANCE can also be thought of as the number of significant digits in the objective function at the solution. For example, if OPTOLERANCE=10^{-6}, the objective function should have approximately six significant digits of accuracy. Specify any number between the FPRECISION value and 1. The default value for OPTOLERANCE is epsilon**0.8.

FPRECISION n. *Function precision.* This measure is a measure of the accuracy with which the objective function can be checked. It acts as a relative precision when the function is large and an absolute precision when the function is small. For example, if the objective function is larger than 1, and six significant digits are desired, FPRECISION should be 1E-6. If, however, the objective function is of the order 0.001, FPRECISION should be 1E-9 to get six digits of accuracy. Specify any number between 0 and 1. The choice of FPRECISION can be very complicated for a badly scaled problem. Chapter 8 of Gill et al. (1981) gives some scaling suggestions. The default value is epsilon**0.9.

ISTEP n. *Infinite step size.* This value is the magnitude of the change in parameters that is defined as infinite. That is, if the change in the parameters at a step is greater than ISTEP, the problem is considered unbounded, and estimation stops. Specify any positive number. The default value is 1E+20.

Iteration Criteria for NLR

The NLR procedure uses an adaptation of subroutine LMSTR from the MINPACK package by Garbow et al. Because the NLR algorithm differs substantially from CNLR, the CRITERIA subcommand for NLR has a different set of keywords.

NLR computes parameter estimates by using the Levenberg-Marquardt method. At each iteration, NLR evaluates the estimates against a set of control criteria. The iterative calculations continue until one of five cutoff points is met, at which point the iterations stop and the reason for stopping is displayed.

The CRITERIA subcommand has the following keywords when used with NLR:

ITER n. *Maximum number of major and minor iterations allowed.* Specify any positive integer for *n*. The default is 100 iterations per parameter. If the search for a solution stops because this limit is exceeded, NLR issues a warning message.

SSCON n. *Convergence criterion for the sum of squares.* Specify any non-negative number for *n*. The default is 1E-8. If successive iterations fail to reduce the sum of squares by this proportion, the procedure stops. Specify 0 to disable this criterion.

PCON n. *Convergence criterion for the parameter values.* Specify any non-negative number for *n*. The default is 1E-8. If successive iterations fail to change any of the parameter values by this proportion, the procedure stops. Specify 0 to disable this criterion.

RCON n. *Convergence criterion for the correlation between the residuals and the derivatives.* Specify any non-negative number for *n*. The default is 1E-8. If the largest value for the correlation between the residuals and the derivatives equals this value, the procedure stops because it lacks the information that it needs to estimate a direction for its next move. This criterion is often referred to as a gradient convergence criterion. Specify 0 to disable this criterion.

Example

```
MODEL PROGRAM A=.5 B=1.6.
COMPUTE PRED=A*SPEED**B.
NLR STOP /CRITERIA=ITER(80) SSCON=.000001.
```

- CRITERIA changes two of the five cutoff values affecting iteration, ITER and SSCON, and leaves the remaining three, PCON, RCON, and CKDER, at their default values.

BOUNDS Subcommand

The BOUNDS subcommand can be used to specify both linear and nonlinear constraints. It can be used only with CNLR; it cannot be used with NLR.

Simple Bounds and Linear Constraints

BOUNDS can be used to impose bounds on parameter values. These bounds can involve either single parameters or a linear combination of parameters and can be either equalities or inequalities.

- All bounds are specified on the same BOUNDS subcommand and are separated by semicolons.
- The only variables that are allowed on BOUNDS are parameter variables (those variables that are named on MODEL PROGRAM).
- Only * (multiplication), + (addition), - (subtraction), = or EQ, >= or GE, and <= or LE can be used. When two relational operators are used (as in the third bound in the example below), they must both be in the same direction.

Example

```
/BOUNDS 5 >= A;
        B >= 9;
        .01 <= 2*A + C <= 1;
        D + 2*E = 10
```

- BOUNDS imposes bounds on the parameters *A, B, C,* and *D*. Specifications for each parameter are separated by a semicolon.

Nonlinear Constraints

Nonlinear constraints on the parameters can also be specified with the BOUNDS subcommand. The constrained function must be calculated and stored in a variable by a constrained functions program directly preceding the CNLR command. The constraint is then specified on the BOUNDS subcommand.

In general, nonlinear bounds will not be obeyed until an optimal solution has been found. This process is different from simple and linear bounds, which are satisfied at each iteration. The constrained functions must be smooth near the solution.

Example

```
MODEL PROGRAM A=.5 B=1.6.
COMPUTE PRED=A*SPEED**B.

CONSTRAINED FUNCTIONS.
COMPUTE DIFF=A-10**B.

CNLR STOP /BOUNDS DIFF LE 0.
```

- The constrained function is calculated by a constrained functions program and stored in variable *DIFF*. The constrained functions program immediately precedes CNLR.
- BOUNDS imposes bounds on the function (less than or equal to 0).
- CONSTRAINED FUNCTIONS variables and parameters that are named on MODEL PROGRAM cannot be combined in the same BOUNDS expression. For example, you *cannot* specify (DIFF + A) >= 0 on the BOUNDS subcommand.

LOSS Subcommand

LOSS specifies a loss function for CNLR to minimize. By default, CNLR minimizes the sum of squared residuals. LOSS can be used only with CNLR; it cannot be used with NLR.

- The loss function must first be computed in the model program. LOSS is then used to specify the name of the computed variable.

- The minimizing algorithm may fail if it is given a loss function that is not smooth, such as the absolute value of residuals.
- If derivatives are supplied, the derivative of each parameter must be computed with respect to the loss function, rather than the predicted value. The easiest way to do this is in two steps: First compute derivatives of the model, and then compute derivatives of the loss function with respect to the model and multiply by the model derivatives.
- When LOSS is used, the usual summary statistics are not computed. Standard errors, confidence intervals, and correlations of the parameters are available only if the BOOTSTRAP subcommand is specified.

Example

```
MODEL PROGRAM  A=1 B=1.
COMPUTE PRED=EXP(A+B*T)/(1+EXP(A+B*T)).
COMPUTE LOSS=-W*(Y*LN(PRED)+(1-Y)*LN(1-PRED)).

DERIVATIVES.
COMPUTE D.A=PRED/(1+EXP(A+B*T)).
COMPUTE D.B=T*PRED/(1+EXP(A+B*T)).
COMPUTE D.A=(-W*(Y/PRED - (1-Y)/(1-PRED)) * D.A).
COMPUTE D.B=(-W*(Y/PRED - (1-Y)/(1-PRED)) * D.B).

CNLR Y /LOSS=LOSS.
```

- The second COMPUTE command in the model program computes the loss function and stores its values in the variable *LOSS*, which is then specified on the LOSS subcommand.
- Because derivatives are supplied in the derivatives program, the derivatives of all parameters are computed with respect to the loss function, rather than the predicted value.

BOOTSTRAP Subcommand

BOOTSTRAP provides bootstrap estimates of the parameter standard errors, confidence intervals, and correlations. BOOTSTRAP can be used only with CNLR; it cannot be used with NLR.

Bootstrapping is a way of estimating the standard error of a statistic, using repeated samples from the original data set. This process is done by sampling with replacement to get samples of the same size as the original data set.

- The minimum specification is the subcommand keyword. Optionally, specify the number of samples to use for generating bootstrap results.
- By default, BOOTSTRAP generates bootstrap results based on $10*p*(p+1)/2$ samples, where p is the number of parameters. That is, 10 samples are drawn for each statistic (standard error or correlation) to be calculated.
- When BOOTSTRAP is used, the nonlinear equation is estimated for each sample. The standard error of each parameter estimate is then calculated as the standard deviation of the bootstrapped estimates. Parameter values from the original data are used as starting values for each bootstrap sample. Even so, bootstrapping is computationally expensive.
- If the OUTFILE subcommand is specified, a case is written to the output file for each bootstrap sample. The first case in the file will be the actual parameter estimates, followed by the bootstrap samples. After the first case is eliminated (using SELECT IF), other procedures (such as FREQUENCIES) can be used to examine the bootstrap distribution.

Example

```
MODEL PROGRAM A=.5 B=1.6.
COMPUTE PSTOP=A*SPEED**B.
CNLR STOP /BOOTSTRAP /OUTFILE=PARAM.
GET FILE=PARAM.
LIST.
COMPUTE ID=$CASENUM.
SELECT IF (ID > 1).
FREQUENCIES A B /FORMAT=NOTABLE /HISTOGRAM.
```

- CNLR generates the bootstrap standard errors, confidence intervals, and parameter correlation matrix. OUTFILE saves the bootstrap estimates in the file *PARAM*.

- GET retrieves the system file *PARAM*.
- LIST lists the different sample estimates, along with the original estimate. *NCASES* in the listing (see “OUTFILE Subcommand ” on page 1240) refers to the number of distinct cases in the sample because cases are duplicated in each bootstrap sample.
- FREQUENCIES generates histograms of the bootstrapped parameter estimates.

References

Gill, P. E., W. M. Murray, M. A. Saunders, and M. H. Wright. 1986. *User’s guide for NPSOL (version 4.0): A FORTRAN package for nonlinear programming. Technical Report SOL 86-2*. Stanford University: Department of Operations Research.

NOMREG

NOMREG is available in SPSS Statistics Standard Edition or the Regression Option.

```
NOMREG dependent varname [(BASE = {FIRST } ORDER = {ASCENDING**})] [BY factor list]
                                 {LAST**}          {DATA       }
                                 {value }          {DESCENDING }
       [WITH covariate list]

 [/CRITERIA = [CIN({95**})] [DELTA({0**})] [MXITER({100**})] [MXSTEP({5**})]
                  {n   }           {n  }           {n    }           {n  }
              [LCONVERGE({0**})] [PCONVERGE({1.0E-6**})] [SINGULAR({1E-8**})]
                         {n  }             {n       }              {n     }
              [BIAS({0**})] [CHKSEP({20**})]  ]
                    {n  }           {n   }


 [/FULLFACTORIAL]


 [/INTERCEPT = {EXCLUDE   }]
               {INCLUDE** }


 [/MISSING = {EXCLUDE**}]
             {INCLUDE  }

 [/MODEL = {[effect effect ...]} [| {BACKWARD} = { effect effect ...}]]
                                    {FORWARD }
                                    {BSTEP   }
                                    {FSTEP   }

 [/STEPWISE =[RULE({SINGLE**   })][MINEFFECT({0**  })][MAXEFFECT(n)]]
                   {SFACTOR    }             {value}
                   {CONTAINMENT}
                   {NONE       }

              [PIN({0.05**})]  [POUT({0.10**})]
                   {value }          {value }

              [ENTRYMETHOD({LR** })] [REMOVALMETHOD({LR**})]
                           {SCORE}                 {WALD}

 [/OUTFILE = [{MODEL    }(filename)]]
              {PARAMETER}

 [/PRINT = [CELLPROB] [CLASSTABLE] [CORB] [HISTORY({1**})] [IC] ]
                                                   {n  }
           [SUMMARY ] [PARAMETER ] [COVB] [FIT] [LRT] [KERNEL]
     [ASSOCIATION] [CPS**] [STEP**] [MFI**] [NONE]

 [/SAVE = [ACPROB[(newname)]] [ESTPROB[(rootname[:{25**}])] ]
                                              {n   }
          [PCPROB[(newname)]] [PREDCAT[(newname)]]

 [/SCALE = {1**     }]
           {n       }
           {DEVIANCE}
           {PEARSON }

 [/SUBPOP = varlist]

 [/TEST[(valuelist)] = {['label'] effect valuelist effect valuelist...;}]
                       {['label'] ALL list;                            }
                       {['label'] ALL list                           }
```

** Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 13.0

- ENTRYMETHOD keyword introduced on STEPWISE subcommand.
- REMOVALMETHOD keyword introduced on STEPWISE subcommand.
- IC keyword introduced on PRINT subcommand.

Release 15.0

- ASSOCIATION keyword introduced on PRINT subcommand.

Release 17.0

- Added support for SET THREADS and SET MCACHE.

Example

```
NOMREG response.
```

Overview

NOMREG is a procedure for fitting a multinomial logit model to a polytomous nominal dependent variable.

Options

Tuning the algorithm. You can control the values of algorithm-tuning parameters with the CRITERIA subcommand.

Optional output. You can request additional output through the PRINT subcommand.

Exporting the model. You can export the model to an external file. The model information will be written using the Extensible Markup Language (XML).

Basic Specification

The basic specification is one dependent variable.

Operations

- This procedure uses the multithreaded options specified by SET THREADS and SET MCACHE.

Syntax Rules

- Minimum syntax—at least one dependent variable must be specified.
- The variable specification must come first.
- Subcommands can be specified in any order.
- Empty subcommands except the MODEL subcommand are ignored.
- The MODEL and the FULLFACTORIAL subcommands are mutually exclusive. Only one of them can be specified at any time.
- The MODEL subcommand stepwise options and the TEST subcommand are mutually exclusive. Only one of them can be specified at any time.
- When repeated subcommands except the TEST subcommand are specified, all specifications except the last valid one are discarded.
- The following words are reserved as keywords or internal commands in the NOMREG procedure: BY, WITH, and WITHIN.
- The set of factors and covariates used in the MODEL subcommand (or implied on the FULLFACTORIAL subcommand) must be a subset of the variable list specified or implied on the SUBPOP subcommand.

Variable List

The variable list specifies the dependent variable and the factors in the model.

- The dependent variable must be the first specification on NOMREG. It can be of any type (numeric or string). Values of the dependent variable are sorted according to the ORDER specification.

  ORDER = ASCENDING. *Response categories are sorted in ascending order.* The lowest value defines the first category, and the highest value defines the last category.

  ORDER = DATA. *Response categories are not sorted.* The first value encountered in the data defines the first category. The last distinct value defines the last category.

  ORDER = DESCENDING. *Response categories are sorted in descending order.* The highest value defines the first category, and the lowest value defines the last category.
- By default, the last response category is used as the base (or reference) category. No model parameters are assigned to the base category. Use the BASE attribute to specify a custom base category.

BASE = FIRST. *The first category is the base category.*

BASE = LAST. *The last category is the base category.*

BASE = value. *The category with the specified value is the base category.* Put the value inside a pair of quotes if either the value is formatted (such as date or currency) or if the dependent variable is the string type.

- Factor variables can be of any type (numeric or string). The factors follow the dependent variable separated by the keyword BY.
- Covariate variables must be numeric. The covariates follow the factors, separated by the keyword WITH.
- Listwise deletion is used. If any variables in a case contain missing values, that case will be excluded.
- If the WEIGHT command was specified, the actual weight values are used for the respective category combination. No rounding or truncation will be done. However, cases with negative and zero weight values are excluded from the analyses.

Example

```
NOMREG response (ORDER = DESCENDING BASE='No') BY factor1.
```

- Values of the variable *response* are sorted in descending order, and the category whose value is *No* is the base category.

Example

```
NOMREG
  movie  BY gender date
  /CRITERIA = CIN(95) DELTA(0) MXITER(100) MXSTEP(5) LCONVERGE(0)
              PCONVERGE(0)
  /INTERCEPT = EXCLUDE
  /PRINT = CLASSTABLE FIT PARAMETER SUMMARY LRT .
```

- The dependent variable is *movie*, and *gender* and *date* are factors.
- CRITERIA specifies that the confidence level to use is 95, no delta value should be added to cells with observed zero frequency, and neither the log-likelihood nor parameter estimates convergence criteria should be used. This means that the procedure will stop when either 100 iterations or five step-halving operations have been performed.
- INTERCEPT specifies that the intercept should be excluded from the model.
- PRINT specifies that the classification table, goodness-of-fit statistics, parameter statistics, model summary, and likelihood-ratio tests should be displayed.

CRITERIA Subcommand

The CRITERIA subcommand offers controls on the iterative algorithm used for estimation and specifies numerical tolerance for checking singularity.

BIAS(n). *Bias value added to observed cell frequency.* Specify a non-negative value less than 1. The default value is 0.

CHKSEP(n). *Starting iteration for checking for complete separation.* Specify a non-negative integer. The default value is 20.

CIN(n). *Confidence interval level.* Specify a value greater than or equal to 0 and less than 100. The default value is 95.

DELTA(n). *Delta value added to zero cell frequency.* Specify a non-negative value less than 1. The default value is 0.

LCONVERGE(n). *Log-likelihood function convergence criterion.* Convergence is assumed if the absolute change in the log-likelihood function is less than this value. The criterion is not used if the value is 0. Specify a non-negative value. The default value is 0.

MXITER(n). *Maximum number of iterations.* Specify a positive integer. The default value is 100.

MXSTEP(n). *Maximum step-halving allowed.* Specify a positive integer. The default value is 5.

PCONVERGE(a). *Parameter estimates convergence criterion.* Convergence is assumed if the absolute change in the parameter estimates is less than this value. The criterion is not used if the value is 0. Specify a non-negative value. The default value is 10^{-6}.

SINGULAR(a). *Value used as tolerance in checking singularity.* Specify a positive value. The default value is 10^{-8}.

FULLFACTORIAL Subcommand

The FULLFACTORIAL subcommand generates a specific model: first, the intercept (if included); second, all of the covariates (if specified), in the order in which they are specified; next, all of the main factorial effects; next, all of the two-way factorial interaction effects, all of the three-way factorial interaction effects, and so on, up to the highest possible interaction effect.

- The FULLFACTORIAL and the MODEL subcommands are mutually exclusive. Only one of them can be specified at any time.
- The FULLFACTORIAL subcommand does not take any keywords.

INTERCEPT Subcommand

The INTERCEPT subcommand controls whether intercept terms are included in the model. The number of intercept terms is the number of response categories less one.

INCLUDE. *Includes the intercept terms.* This is the default.

EXCLUDE. *Excludes the intercept terms.*

MISSING Subcommand

By default, cases with missing values for any of the variables on the NOMREG variable list are excluded from the analysis. The MISSING subcommand allows you to include cases with user-missing values.

- Note that missing values are deleted at the subpopulation level.

EXCLUDE. *Excludes both user-missing and system-missing values.* This is the default.

INCLUDE. *User-missing values are treated as valid.* System-missing values cannot be included in the analysis.

MODEL Subcommand

The MODEL subcommand specifies the effects in the model.

- The MODEL and the FULLFACTORIAL subcommands are mutually exclusive. Only one of them can be specified at any time.
- If more than one MODEL subcommand is specified, only the last one is in effect.
- Specify a list of terms to be included in the model, separated by commas or spaces. If the MODEL subcommand is omitted or empty, the default model is generated. The default model contains: first, the intercept (if included); second, all of the covariates (if specified), in the order in which they are specified; and next, all of the main factorial effects, in the order in which they are specified.
- If a SUBPOP subcommand is specified, then effects specified in the MODEL subcommand can only be composed using the variables listed on the SUBPOP subcommand.
- To include a main-effect term, enter the name of the factor on the MODEL subcommand.

- To include an interaction-effect term among factors, use the keyword BY or the asterisk (*) to join factors involved in the interaction. For example, A*B*C means a three-way interaction effect of A, B, and C, where A, B, and C are factors. The expression `A BY B BY C` is equivalent to A*B*C. Factors inside an interaction effect must be distinct. Expressions such as A*C*A and A*A are invalid.
- To include a nested-effect term, use the keyword `WITHIN` or a pair of parentheses on the `MODEL` subcommand. For example, A(B) means that A is nested within B, where A and B are factors. The expression `A WITHIN B` is equivalent to A(B). Factors inside a nested effect must be distinct. Expressions such as A(A) and A(B*A) are invalid.
- Multiple-level nesting is supported. For example, A(B(C)) means that B is nested within C, and A is nested within B(C). When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, A(B)(C) is not valid.
- Nesting within an interaction effect is valid. For example, A(B*C) means that A is nested within B*C.
- Interactions among nested effects are allowed. The correct syntax is the interaction followed by the common nested effect inside the parentheses. For example, interaction between A and B within levels of C should be specified as A*B(C) instead of A(C)*B(C).
- To include a covariate term in the model, enter the name of the covariate on the `MODEL` subcommand.
- Covariates can be connected, but not nested, using the keyword BY or the asterisk (*) operator. For example, X*X is the product of X and itself. This is equivalent to a covariate whose values are the square of those of X. However, X(Y) is invalid.
- Factor and covariate effects can be connected in many ways. No effects can be nested within a covariate effect. Suppose A and B are factors, and X and Y are covariates. Examples of valid combination of factor and covariate effects are A*X, A*B*X, X(A), X(A*B), X*A(B), X*Y(A*B), and A*B*X*Y.
- A stepwise method can be specified by following the model effects with a vertical bar (|), a stepwise method keyword, an equals sign (=), and a list of variables (or interactions or nested effects) for which the method is to be used.
- If a stepwise method is specified, then the TEST subcommand is ignored.
- If a stepwise method is specified, then it begins with the results of the model defined on the left side of the `MODEL` subcommand.
- If a stepwise method is specified but no effects are specified on the left side of the `MODEL` subcommand, then the initial model contains the intercept only (if `INTERCEPT = INCLUDE`) or the initial model is the null model (if `INTERCEPT = EXCLUDE`).
- The intercept cannot be specified as an effect in the stepwise method option.
- For all stepwise methods, if two effects have tied significance levels, then the removal or entry is performed on the effect specified first. For example, if the right side of the `MODEL` subcommand specifies `FORWARD A*B A(B)`, where A*B and A(B) have the same significance level less than `PIN`, then A*B is entered because it is specified first.

The available stepwise method keywords are:

BACKWARD. *Backward elimination.* As a first step, the variables (or interaction effects or nested effects) specified on `BACKWARD` are entered into the model together and are tested for removal one by one. The variable with the largest significance level of the likelihood-ratio statistic, provided that the value is larger than `POUT`, is removed, and the model is reestimated. This process continues until no more variables meet the removal criterion or when the current model is the same as a previous model.

FORWARD. *Forward entry.* The variables (or interaction effects or nested effects) specified on `FORWARD` are tested for entry into the model one by one, based on the significance level of the likelihood-ratio statistic. The variable with the smallest significance level less than `PIN` is entered into the model, and the model is reestimated. Model building stops when no more variables meet the entry criteria.

BSTEP. *Backward stepwise.* As a first step, the variables (or interaction effects or nested effects) specified on BSTEP are entered into the model together and are tested for removal one by one. The variable with the largest significance level of the likelihood-ratio statistic, provided that the value is larger than `POUT`, is removed, and the model is reestimated. This process continues until no more variables

meet the removal criterion. Next, variables not in the model are tested for possible entry, based on the significance level of the likelihood-ratio statistic. The variable with the smallest significance level less than PIN is entered, and the model is reestimated. This process repeats, with variables in the model again evaluated for removal. Model building stops when no more variables meet the removal or entry criteria or when the current model is the same as a previous model.

FSTEP. *Forward stepwise.* The variables (or interaction effects or nested effects) specified on FSTEP are tested for entry into the model one by one, based on the significance level of the likelihood-ratio statistic. The variable with the smallest significance level less than PIN is entered into the model, and the model is reestimated. Next, variables that are already in the model are tested for removal, based on the significance level of the likelihood-ratio statistic. The variable with the largest probability greater than the specified POUT value is removed, and the model is reestimated. Variables in the model are then evaluated again for removal. Once no more variables satisfy the removal criterion, variables not in the model are evaluated again for entry. Model building stops when no more variables meet the entry or removal criteria or when the current model is the same as a previous one.

Examples

```
NOMREG y BY a b c
/INTERCEPT = INCLUDE
/MODEL = a b c | BACKWARD = a*b a*c b*c a*b*c.
```

- The initial model contains the intercept and main effects a, b, and c. Backward elimination is used to select among the two- and three-way interaction effects.

```
NOMREG y BY a b c
/MODEL = INTERCEPT | FORWARD = a b c.
```

- The initial model contains the intercept. Forward entry is used to select among the main effects a, b, and c.

```
NOMREG y BY a b c
/INTERCEPT = INCLUDE
/MODEL = | FORWARD = a b c.
```

- The initial model contains the intercept. Forward entry is used to select among the main effects a, b, and c.

```
NOMREG y BY a b c
/INTERCEPT = EXCLUDE
/MODEL = | BSTEP = a b c.
```

- The initial model is the null model. Backward stepwise is used to select among the main effects a, b, and c.

```
NOMREG y BY a b c
/MODEL = | FSTEP =.
```

- This MODEL specification yields a syntax error.

STEPWISE Subcommand

The STEPWISE subcommand gives you control of the statistical criteria when stepwise methods are used to build a model. This subcommand is ignored if a stepwise method is not specified on the MODEL subcommand.

RULE(keyword). *Rule for entering or removing terms in stepwise methods.* The default SINGLE indicates that only one effect can be entered or removed at a time, provided that the hierarchy requirement is satisfied for all effects in the model. SFACTOR indicates that only one effect can be entered or removed at a time, provided that the hierarchy requirement is satisfied for all factor-only effects in the model. CONTAINMENT indicates that only one effect can be entered or removed at a time, provided that the containment requirement is satisfied for all effects in the model. NONE indicates that only one effect can be entered or removed at a time, where neither the hierarchy nor the containment requirement need be satisfied for any effects in the model.

MINEFFECT(n). *Minimum number of effects in final model.* The default is 0. The intercept, if any, is not counted among the effects. This criterion is ignored unless one of the stepwise methods BACKWARD or BSTEP is specified.

MAXEFFECT(n). *Maximum number of effects in final model.* The default value is the total number of effects specified or implied on the NOMREG command. The intercept, if any, is not counted among the effects. This criterion is ignored unless one of the stepwise methods FORWARD or FSTEP is specified.

ENTRYMETHOD (keyword). *Method for entering terms in stepwise methods.* The default LR indicates that the likelihood ratio test is used to determine whether a term is entered into the model. SCORE indicates that the score test is used. This criterion is ignored unless one of the stepwise methods FORWARD, BSTEP, or FSTEP is specified.

REMOVALMETHOD (keyword). *Method for removing terms in stepwise methods.* The default LR indicates that the likelihood ratio test is used to determine whether a term is entered into the model. WALD indicates that the Wald test is used. This criterion is ignored unless one of the stepwise methods BACKWARD, BSTEP, or FSTEP is specified.

PIN(a). *Probability of the likelihood-ratio statistic for variable entry.* The default is 0.05. The larger the specified probability, the easier it is for a variable to enter the model. This criterion is ignored unless one of the stepwise methods FORWARD, BSTEP, or FSTEP is specified.

POUT(a). *Probability of the likelihood-ratio statistic for variable removal.* The default is 0.1. The larger the specified probability, the easier it is for a variable to remain in the model. This criterion is ignored unless one of the stepwise methods BACKWARD, BSTEP, or FSTEP is specified.

The hierarchy requirement stipulates that among the effects specified or implied on the MODEL subcommand, for any effect to be in a model, all lower-order effects that are part of the former effect must also be in the model. For example, if A, X, and A*X are specified, then for A*X to be in a model, the effects A and X must also be in the model.

The containment requirement stipulates that among the effects specified or implied on the MODEL subcommand, for any effect to be in the model, all effects contained in the former effect must also be in the model. For any two effects F and F', F is contained in F' if:

- Both effects F and F' involve the same covariate effect, if any. (Note that effects A*X and A*X*X are not considered to involve the same covariate effect because the first involves covariate effect X and the second involves covariate effect X**2.)
- F' consists of more factors than F.
- All factors in F also appear in F'.

The following table illustrates how the hierarchy and containment requirements relate to the RULE options. Each row of the table gives a different set of effects specified on the MODEL subcommand. The columns correspond to the RULE options SINGLE, SFACTOR, and CONTAINMENT. The cells contain the order in which effects must occur in the model. For example, unless otherwise noted, all effects numbered 1 must be in the model for any effects numbered 2 to be in the model.

OUTFILE Subcommand

The OUTFILE subcommand allows you to specify files to which output is written.

- Only one OUTFILE subcommand is allowed. If you specify more than one, only the last one is executed.
- You must specify at least one keyword and a valid filename in parentheses. There is no default.
- Neither MODEL nor PARAMETER is honored if split file processing is on (SPLIT FILE command) or if more than one dependent (DEPENDENT subcommand) variable is specified.

MODEL(filename). *Write parameter estimates and their covariances to an XML (PMML) file.* Specify the filename in full. NOMREG does not supply an extension. You can use this model file to apply the model information to other data files for scoring purposes. See the topic "Scoring expressions" on page 86 for more information.

PARAMETER(filename). *Write parameter estimates only to an XML (PMML) file.* Specify the filename in full. NOMREG does not supply an extension. You can use this model file to apply the model information to other data files for scoring purposes. See the topic "Scoring expressions" on page 86 for more information.

PRINT Subcommand

The PRINT subcommand displays optional output. If no PRINT subcommand is specified, the default output includes a factor information table.

ASSOCIATION. *Measures of Monotone Association.* Displays a table with information on the number of concordant pairs, discordant pairs, and tied pairs. The Somers' D, Goodman and Kruskal's Gamma, Kendall's tau-a, and Concordance Index C are also displayed in this table.

CELLPROB. *Observed proportion, expected probability, and the residual for each covariate pattern and each response category.*

CLASSTABLE. *Classification table.* The square table of frequencies of observed response categories versus the predicted response categories. Each case is classified into the category with the highest predicted probability.

CORB. *Asymptotic correlation matrix of the parameter estimates.*

COVB. *Asymptotic covariance matrix of the parameter estimates.*

FIT. *Goodness-of-fit statistics.* The change in chi-square statistics with respect to a model with intercept terms only (or to a null model when INTERCEPT= EXCLUDE). The table contains the Pearson chi-square and the likelihood-ratio chi-square statistics. The statistics are computed based on the subpopulation classification specified on the SUBPOP subcommand or the default classification.

HISTORY(n). *Iteration history.* The table contains log-likelihood function value and parameter estimates at every *n*th iteration beginning with the 0th iteration (the initial estimates). The default is to print every iteration (*n* = 1). The last iteration is always printed if HISTORY is specified, regardless of the value of *n*.

IC. *Information criteria.* The Akaike Information Criterion (AIC) and the Schwarz Bayesian Information Criterion (BIC) are displayed.

KERNEL. *Kernel of the log-likelihood.* Displays the value of the kernel of the –2 log-likelihood. The default is to display the full –2 log-likelihood. Note that this keyword has no effect unless the MFI or LRT keywords are specified.

LRT. *Likelihood-ratio tests.* The table contains the likelihood-ratio test statistics for the model and model partial effects. If LRT is not specified, just the model test statistic is printed.

PARAMETER. *Parameter estimates.*

SUMMARY. *Model summary.* Cox and Snell's, Nagelkerke's, and McFadden's R^2 statistics.

CPS. *Case processing summary.* This table contains information about the specified categorical variables. Displayed by default.

STEP. *Step summary.* This table summarizes the effects entered or removed at each step in a stepwise method. Displayed by default if a stepwise method is specified. This keyword is ignored if no stepwise method is specified.

MFI. *Model fitting information.* This table compares the fitted and intercept-only or null models. Displayed by default.

NON. *No statistics are displayed.* This option overrides all other specifications on the PRINT subcommand.

SAVE Subcommand

The SAVE subcommand puts casewise post-estimation statistics back into the active file.

- The new names must be valid variable names and not currently used in the active dataset.
- The rootname must be a valid variable name.
- The rootname can be followed by the number of predicted probabilities saved. The number is a positive integer. For example, if the integer is 5, then the first five predicted probabilities across all split files (if applicable) are saved. The default is 25.
- The new variables are saved into the active file in the order in which the keywords are specified on the subcommand.

ACPROB(newname). *Estimated probability of classifying a factor/covariate pattern into the actual category.*

ESTPROB(rootname:n). *Estimated probabilities of classifying a factor/covariate pattern into the response categories.* There are as many number of probabilities as the number of response categories. The predicted probabilities of the first *n* response categories will be saved. The default value for *n* is 25. To specify n without a rootname, enter a colon before the number.

PCPROB(newname). *Estimated probability of classifying a factor/covariate pattern into the predicted category.* This probability is also the maximum of the estimated probabilities of the factor/covariate pattern.

PREDCAT(newname). *The response category that has the maximum expected probability for a factor/covariate pattern.*

SCALE Subcommand

The SCALE subcommand specifies the dispersion scaling value. Model estimation is not affected by this scaling value. Only the asymptotic covariance matrix of the parameter estimates is affected.

N. *A positive number corresponding to the amount of overdispersion or underdispersion.* The default scaling value is 1, which corresponds to no overdispersion or underdispersion.

DEVIANCE. *Estimates the scaling value by using the deviance function statistic.*

PEARSON. *Estimates the scaling value by using the Pearson chi-square statistic.*

SUBPOP Subcommand

The SUBPOP subcommand allows you to define the subpopulation classification used in computing the goodness-of-fit statistics.

- A variable list is expected if the SUBPOP subcommand is specified. The variables in the list must be a subset of the combined list of factors and covariates specified on the command line.
- Variables specified or implied on the MODEL subcommand must be a subset of the variables specified or implied on the SUBPOP subcommand.
- If the SUBPOP subcommand is omitted, the default classification is based on all of the factors and the covariates specified.
- Missing values are deleted listwise on the subpopulation level.

Example

```
NOMREG
  movie  BY gender date WITH age
  /CRITERIA = CIN(95) DELTA(0) MXITER(100) MXSTEP(5) LCONVERGE(0)
              PCONVERGE(1.0E-6) SINGULAR(1.0E-8)
  /MODEL = gender
  /SUBPOP = gender date
  /INTERCEPT = EXCLUDE .
```

- Although the model consists only of *gender*, the SUBPOP subcommand specifies that goodness-of-fit statistics should be computed based on both *gender* and *date*.

TEST Subcommand

The TEST subcommand allows you to customize your hypothesis tests by directly specifying null hypotheses as linear combinations of parameters.

- TEST is offered only through syntax.
- Multiple TEST subcommands are allowed. Each is handled independently.
- The basic format for the TEST subcommand is an optional list of values enclosed in parentheses, an optional label in quotes, an effect name or the keyword ALL, and a list of values.
- The value list preceding the first effect or the keyword ALL are the constants to which the linear combinations are equated under the null hypotheses. If this value list is omitted, the constants are assumed to be all zeros.
- The label is a string with a maximum length of 255 characters (or 127 double-byte characters). Only one label per linear combination can be specified.
- When ALL is specified, only a list of values can follow. The number of values must equal the number of parameters (including the redundant ones) in the model.
- When effects are specified, only valid effects appearing or implied on the MODEL subcommand can be named. The number of values following an effect name must equal the number of parameters (including the redundant ones) corresponding to that effect. For example, if the effect A*B takes up six parameters, then exactly six values must follow A*B. To specify the coefficient for the intercept, use the keyword INTERCEPT. Only one value is expected to follow INTERCEPT.
- When multiple linear combinations are specified within the same TEST subcommand, use semicolons to separate each hypothesis.
- The linear combinations are first tested separately for each logit and then simultaneously tested for all of the logits.
- A number can be specified as a fraction with a positive denominator. For example, 1/3 or –1/3 are valid, but 1/–3 is invalid.
- Effects appearing or implied on the MODEL subcommand but not specified on the TEST are assumed to take the value 0 for all of their parameters.

Example

```
NOMREG
  movie  BY gender date
  /CRITERIA = CIN(95) DELTA(0) MXITER(100) MXSTEP(5) LCONVERGE(0)
              PCONVERGE(1.0E-6) SINGULAR(1.0E-8)
  /INTERCEPT = EXCLUDE
  /PRINT = CELLPROB CLASSTABLE FIT CORB COVB HISTORY(1) PARAMETER
           SUMMARY LRT
  /TEST (0 0) = ALL 1 0 0 0;
                ALL 0 1 0 0 .
```

- TEST specifies two separate tests: one in which the coefficient corresponding to the first category for gender is tested for equality with zero, and one in which the coefficient corresponding to the second category for gender is tested for equality with zero.

NONPAR CORR

NONPAR CORR is available in Statistics Base Edition.

```
NONPAR CORR VARIABLES= varlist [WITH varlist] [/varlist...]

 [/PRINT={TWOTAIL**} {SIG**} {SPEARMAN**} {FULL**  }]
         {ONETAIL  } {NOSIG} {KENDALL    } {LOWER   }
                             {BOTH       } {LNODIAG }
                                           {NOMATRIX}

 [/SAMPLE]

 [/MISSING=[{PAIRWISE**} [INCLUDE]]
           {LISTWISE  }

 [/MATRIX=OUT({*                  })]
             {'savfile'|'dataset'}

 [/CI
     [METHOD ({FHP**} {CC} {BW})]
     [CILEVEL ({95**} {value})]
 ]
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
NONPAR CORR
  VARIABLES=PRESTIGE SPPRES PAPRES16 DEGREE PADEG MADEG.
```

Release History

Release 27.0.1

- Added support for the **CI** subcommand.
- Added support for the **NOMATRIX** keyword in the **PRINT** subcommand.
- The **CILEVEL** subcommand has been deprecated.

Release 27.0

- Added support for the **FULL**, **LOWER**, and **LNODIAG** keywords in the **PRINT** subcommand.

Overview

NONPAR CORR computes two rank-order correlation coefficients, Spearman's rho and Kendall's tau-*b*, with their significance levels. You can obtain one or both coefficients. NONPAR CORR automatically computes the ranks and stores the cases in memory. Therefore, memory requirements are directly proportional to the number of cases that are being analyzed.

Options

Coefficients and Significance Levels. By default, NONPAR CORR computes Spearman coefficients and displays the two-tailed significance level. You can request a one-tailed test, and you can display the significance level for each coefficient as an annotation by using the PRINT subcommand.

Random Sampling. You can request a random sample of cases by using the SAMPLE subcommand when there is not enough space to store all cases.

Matrix Output. You can write matrix materials to a matrix data file by using the MATRIX subcommand. The matrix materials include the number of cases that are used to compute each coefficient and the Spearman or Kendall coefficients for each variable. These materials can be read by other procedures.

Basic Specification

The basic specification is VARIABLES and a list of numeric variables. By default, Spearman correlation coefficients are calculated.

Subcommand Order

- VARIABLES must be specified first.
- The remaining subcommands can be used in any order.

Operations

- NONPAR CORR produces one or more matrices of correlation coefficients. For each coefficient, NONPAR CORR displays the number of used cases and the significance level.
- The number of valid cases is always displayed. Depending on the specification on the MISSING subcommand, the number of valid cases can be displayed for each pair or in a single annotation.
- If all cases have a missing value for a given pair of variables, or if all cases have the same value for a variable, the coefficient cannot be computed. If a correlation cannot be computed, NONPAR CORR displays a decimal point.
- If both Spearman and Kendall coefficients are requested, and MATRIX is used to write matrix materials to a matrix data file, only Spearman's coefficient will be written with the matrix materials.

Limitations

- A maximum of 25 variable lists is allowed.
- A maximum of 100 variables total per NONPAR CORR command is allowed.

Examples

```
NONPAR CORR VARIABLES=PRESTIGE SPPRES PAPRES16 DEGREE PADEG MADEG.
```

- By default, Spearman correlation coefficients are calculated. The number of cases upon which the correlations are based and the two-tailed significance level are displayed for each correlation.

VARIABLES Subcommand

VARIABLES specifies the variable list.

- All variables must be numeric.
- If keyword WITH is not used, NONPAR CORR displays the correlations of each variable with every other variable in the list.
- To obtain a rectangular matrix, specify two variable lists that are separated by keyword WITH. NONPAR CORR writes a rectangular matrix of variables in the first list correlated with variables in the second list.
- Keyword WITH cannot be used when the MATRIX subcommand is used.
- You can request more than one analysis. Use a slash to separate the specifications for each analysis.
- When only **varlist1** is specified, the confidence limits are estimated for each different pair of the variables contained within **varlist1**. When varlist1 WITH varlist2 is specified, the confidence limits are estimated for each different pair of the variables with one contained within **varlist1** and the other **varlist2**. The confidence limits of the pair containing a variable and itself are not estimated.

Examples

```
NONPAR CORR
  VARIABLES = PRESTIGE SPPRES PAPRES16 WITH DEGREE PADEG MADEG.
```

- The three variables that are listed before WITH define the rows; the three variables that are listed after WITH define the columns of the correlation matrix.

- Spearman’s rho is displayed by default.

```
NONPAR CORR VARIABLES=SPPRES PAPRES16 PRESTIGE
                    /SATCITY WITH SATHOBBY SATFAM.
```

- NONPAR CORR produces two Correlations tables.
- By default, Spearman’s rho is displayed.

PRINT Subcommand

By default, NONPAR CORR displays Spearman correlation coefficients. The significance levels are displayed below the coefficients. The significance level is based on a two-tailed test. Use PRINT to change these defaults.

Note: The Spearman and Kendall coefficients are both based on ranks.

SPEARMAN
Spearman’s rho. Only Spearman coefficients are displayed. This specification is the default.

KENDALL
Kendall’s tau-b. Only Kendall coefficients are displayed.

BOTH
Kendall and Spearman coefficients. Both coefficients are displayed. If MATRIX is used to write the correlation matrix to a matrix data file, only Spearman coefficients are written with the matrix materials.

SIG
Do not flag significant values. SIG is the default.

NOSIG
Flag significant values. Values significant at the 0.05 level are flagged with a single asterisk; those that are significant at the 0.01 level are flagged with two asterisks.

TWOTAIL
Two-tailed test of significance. This test is appropriate when the direction of the relationship cannot be determined in advance, as is often the case in exploratory data analysis. The keyword also controls whether or not two-tailed confidence limits are estimated. This specification is the default.

ONETAIL
One-tailed test of significance. This test is appropriate when the direction of the relationship between a pair of variables can be specified in advance of the analysis. The keyword also controls whether one-tailed confidence limits are calculated.

FULL
Show the full table. The full table is presented in the output.

LOWER
Show only the table's lower triangle. Only the table's lower triangle is presented in the output. The keyword allows table output to adhere to APA style guidelines.

LNODIAG
Show only the table's lower triangle and hide diagonal values. Only the correlation matrix table's lower triangle is presented in the output. Diagonal values are also hidden. The keyword allows table output to adhere to APA style guidelines.

NOMATRIX
Suppresses the Correlations table. Honored only when **/CI** is specified; otherwise, **FULL** is assumed.

SAMPLE Subcommand

NONPAR CORR must store cases in memory to build matrices. SAMPLE selects a random sample of cases when computer resources are insufficient to store all cases. SAMPLE has no additional specifications.

MISSING Subcommand

MISSING controls the treatment of missing values.

- PAIRWISE and LISTWISE are alternatives. You can specify INCLUDE with either PAIRWISE or LISTWISE.

PAIRWISE. *Exclude missing values pairwise.* Cases with a missing value for one or both variables for a specific correlation coefficient are excluded from the computation of that coefficient. This process allows the maximum available information to be used in every calculation. This process also results in a set of coefficients based on a varying number of cases. The number is displayed for each pair. This specification is the default.

LISTWISE. *Exclude missing values listwise.* Cases with a missing value for any variable that is named in a list are excluded from the computation of all coefficients in the Correlations table. The number of used cases is displayed in a single annotation. Each variable list on a command is evaluated separately. Thus, a case that is missing for one matrix might be used in another matrix. This option decreases the amount of required memory and significantly decreases computational time.

INCLUDE. *Include user-missing values*. User-missing values are treated as valid values.

MATRIX Subcommand

MATRIX writes matrix materials to a matrix data file. The matrix materials always include the number of cases that are used to compute each coefficient, and the materials include either the Spearman or the Kendall correlation coefficient for each variable, whichever is requested. See the topic "Format of the Matrix Data File " on page 1263 for more information.

- You cannot write both Spearman's and Kendall's coefficients to the same matrix data file. To obtain both Spearman's and Kendall's coefficients in matrix format, specify separate NONPAR CORR commands for each coefficient and define different matrix data files for each command.
- If PRINT=BOTH is in effect, NONPAR CORR displays a matrix in the listing file for both coefficients but writes only the Spearman coefficients to the matrix data file.
- NONPAR CORR cannot write matrix materials for rectangular matrices (variable lists containing keyword WITH). If more than one variable list is specified, only the last variable list that does not use keyword WITH is written to the matrix data file.
- The specification on MATRIX is keyword OUT and a quoted file specification or previously declared dataset name (DATASET DECLARE command), enclosed in parentheses.
- If you want to use a correlation matrix that is written by NONPAR CORR in another procedure, change the *ROWTYPE_* value RHO or TAUB to CORR by using the RECODE command.
- Any documents that are contained in the active dataset are not transferred to the matrix file.

OUT ('savfile'|'dataset'). *Write a matrix data file or dataset.* Specify either a filename, a previously declared dataset name, or an asterisk, enclosed in parentheses. Filenames should be enclosed in quotes and are stored in the working directory unless a path is included as part of the file specification. If you specify an asterisk (*), the matrix data file replaces the active dataset.

Multiple nonparametric correlation tables

```
NONPAR CORR
  VARIABLES=PRESTIGE SPPRES PAPRES16 DEGREE PADEG MADEG
  /PRESTIGE TO DEGREE /PRESTIGE WITH DEGREE
  /MATRIX OUT('/data/npmat.sav').
```

- Only the matrix for *PRESTIGE* to *DEGREE* is written to the matrix data file because it is the last variable list that does not use keyword WITH.

Format of the Matrix Data File

- The matrix data file has two special variables that are created by the program: *ROWTYPE_* and *VARNAME_*.
- *ROWTYPE_* is a short string variable with values N and RHO for Spearman's correlation coefficient. If you specify Kendall's coefficient, the values are N and TAUB.
- *VARNAME_* is a short string variable whose values are the names of the variables that are used to form the correlation matrix. When *ROWTYPE_* is RHO (or TAUB), *VARNAME_* gives the variable that is associated with that row of the correlation matrix.
- The remaining variables in the file are the variables that are used to form the correlation matrix.

Split Files

- When split-file processing is in effect, the first variables in the matrix data file are the split variables, followed by *ROWTYPE_*, *VARNAME_*, and the variables that are used to form the correlation matrix.
- A full set of matrix materials is written for each split-file group that is defined by the split variables.
- A split variable cannot have the same name as any other variable that is written to the matrix data file.
- If split-file processing is in effect when a matrix is written, the same split file must be in effect when that matrix is read by a procedure.

Missing Values

- With PAIRWISE treatment of missing values (the default), the matrix of Ns that is used to compute each coefficient is included with the matrix materials.
- With LISTWISE or INCLUDE treatments, a single N that is used to calculate all coefficients is included with the matrix materials.

Examples

Writing results to a matrix data file

```
GET FILE='/data/GSS80.sav'
  /KEEP PRESTIGE SPPRES PAPRES16 DEGREE PADEG MADEG.
NONPAR CORR VARIABLES=PRESTIGE TO MADEG
  /MATRIX OUT('/data/npmat.sav').
```

- NONPAR CORR reads data from file *GSS80.sav* and writes one set of correlation matrix materials to the file *npmat.sav*.
- The active dataset is still *GSS80.sav*. Subsequent commands are executed on file *GSS80.sav*.

Replacing the active dataset with matrix results

```
GET FILE='/data/GSS80.sav'
  /KEEP PRESTIGE SPPRES PAPRES16 DEGREE PADEG MADEG.
NONPAR CORR VARIABLES=PRESTIGE TO MADEG
  /MATRIX OUT(*).
LIST.
DISPLAY DICTIONARY.
```

- NONPAR CORR writes the same matrix as in the example above. However, the matrix data file replaces the active dataset. The LIST and DISPLAY commands are executed on the matrix file (not on the original active dataset *GSS80.sav*).

CI Subcommand

The optional CI subcommand controls the estimation of the confidence intervals. It is not specified by default, which suppresses the estimation. When specified, the confidence limits are estimated and displayed in the output.

METHOD

The optional keyword determines which estimation method is used to compute the confidence intervals for the Spearman's rank-order correlation. The keyword takes effect when either **SPEARMAN** or **BOTH** is specified for the **/PRINT** subcommand. By default, the keyword is omitted, which suppresses the estimation. The keyword specified in parentheses determines how the standard error is calculated. **FHP**, which is the default setting, estimates the standard error based on the method discussed by [Fieller et al., 1957]. **CC** and **BW** estimate the standard error based on the method discussed by [Caruso and Cliff, 1997][Caruso and Cliff, 1997] and [Bonett and Wright, 2000], respectively.

CILEVEL

The optional keyword determines the level of the confidence intervals or limits. The value must be a single value ∈(0,100) with 95 as the default setting.

References

Bonett, D. G. and Wright, T. A. 2000. Sample size requirements for estimating pearson, kendall and spearman correlations. *Psychometrika*, 65(1), 23–28.

Caruso, J. C. and Cliff, N. 1997. Empirical size, coverage, and power of confidence intervalsfor spearman's rho. *Educational and Psychological Measurement*, 57(4), 637–654.

Fieller, E. C., Hartley, H. O., and Pearson, E. S. 1957. Tests for rank correlation coefficients. *Biometrika*, 44(3/4), 470–481.

Fisher, R. A. 1921. On the probable error of a coefficient of correlation deduced from a small sample. *Metron*, 1:3–32.

Shieh, G. 2006. Exact interval estimation, power calculation, and sample size determination in normal correlation analysis. *Psychometrika*, 71(3), 529.

NPTESTS

NPTESTS is available in the Statistics Base Edition.

```
NPTESTS
  [/MISSING [SCOPE={ANALYSIS**}] [USERMISSING={EXCLUDE**}]]
                {LISTWISE  }               {INCLUDE  }
  [/CRITERIA ALPHA={0.05**}] CILEVEL={95** }
                {value }               {value}
        [SEED={2000000**}{integer}{RANDOM}]

  [/ONESAMPLE TEST(fieldlist)                                    ]
     [CHISQUARE(
         [EXPECTED={EQUAL**                      }]
                   {CUSTOM(FREQUENCIES=valuelist
                           CATEGORIES=valuelist) }
      )]

     [BINOMIAL([TESTVALUE={0.5**}]
                          {value}
         [SUCCESSCATEGORICAL={FIRST**               }]
                             {LIST(valuelist)       }
         [SUCCESSCONTINUOUS=CUTPOINT({MIDPOINT**})]
                                    {value     }
         [CLOPPERPEARSON] [JEFFREYS] [LIKELIHOOD]
      )]

     [KOLMOGOROV_SMIRNOV(
         [NSAMPLES={1000**}{integer}]
         [MC_CILEVEL-{99**}{value}]
         [NORMAL={SAMPLE**(SIMULATION={TRUE**}{FALSE})}
                 {CUSTOM(MEAN=value
                         SD=value   )}]
         [UNIFORM={SAMPLE**           }]
                  {CUSTOM(MIN=value
                          MAX=value   )}
         [EXPONENTIAL={SAMPLE**          }]
                      {CUSTOM(MEAN=value )}
         [POISSON=CUSTOM(MEAN=value )]
      )]

     [RUNS([GROUPCATEGORICAL={SAMPLE**                 }]
                            {LIST(valuelist)           }
           [GROUPCONTINUOUS=CUTPOINT({SAMPLEMEDIAN**})]
                                    {SAMPLEMEAN     }
                                    {value          }
      )]

     [WILCOXON(TESTVALUE=value)]

  [/INDEPENDENT TEST(fieldlist)                          ]
                GROUP(fieldname)

     [MANN_WHITNEY] [WALD_WOLFOWITZ] [KOLMOGOROV_SMIRNOV]
     [HODGES_LEHMAN]
     [MOSES([TRIMOUTLIERS={SAMPLE**}])]
                          {integer }

     [KRUSKAL_WALLIS([COMPARE={NONE      }])]
                             {PAIRWISE**}
                             {STEPWISE  }

     [MEDIAN([TESTVALUE={SAMPLE**}]  [COMPARE={NONE      }])]
                       {value   }             {PAIRWISE**}
                                              {STEPWISE  }

     [JONCKHEERE_TERPSTRA(
         [ORDER={ASCENDING**     }]  [COMPARE={NONE      }]
                {DESCENDING      }             {PAIRWISE**}
                                               {STEPWISE  }
      )]

  [/RELATED TEST(fieldlist)                          ]
     [MCNEMAR( [SUCCESS={FIRST**        }])]
                        {LIST(valuelist)}

     [WILCOXON] [SIGN] [MARGINAL_HOMOGENEITY]
     [HODGES_LEHMAN]

     [COCHRAN(
         [SUCCESS={FIRST          }]
                  {LIST(valuelist)}
         [COMPARE={NONE      }]
                  {PAIRWISE**}
                  {STEPWISE  }
      )]

     [FRIEDMAN([COMPARE={NONE      }])]
                       {PAIRWISE**}
                       {STEPWISE  }

     [KENDALL([COMPARE={NONE      }])]
                      {PAIRWISE**}
                      {STEPWISE  }
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 18

- Command introduced.

Release 22

- Added support for ordinal fields to tests that were previously restricted to continuous fields.

Release 27

- Added support for the **SEED** keyword in the **CRITERIA** subcommand.
- **ONESAMPLE** subcommand:
  - Added support for the **NSAMPLES** keyword in the **KOLMOGOROV_SMIRNOV** keyword.
  - Added support for the **MC_CILEVEL** keyword in the **KOLMOGOROV_SMIRNOV** keyword.
  - Added support for the **SIMULATION** keyword in the **KOLMOGOROV_SMIRNOV** keyword.
  - Removed the **SAMPLE** setting from the **POISSON** keyword in the **KOLMOGOROV_SMIRNOV** keyword.

Example

```
NPTESTS
  /ONESAMPLE TEST(field1 TO field10).
```

Overview

NPTESTS is a collection of nonparametric tests. These tests make minimal assumptions about the underlying distribution of the data.

The tests that are available in NPTESTS can be grouped into three broad categories based on how the data are organized.

- A one-sample test analyzes one field.
- A test for related samples compares two or more fields for the same set of cases.
- An independent-samples test analyzes one field that is grouped by categories of another field.

Options

Automatic test selection
If a test subcommand (ONESAMPLE, INDEPENDENT, or RELATED) is specified without any test specifications, then NPTESTS automatically chooses the "best" tests to perform for each field specified on the subcommand. See the individual subcommands for details.

Multiple comparisons
In addition to the "omnibus" tests for *k* independent or related samples, the samples can also be tested pairwise, either by all pairwise comparisions or through a stepwise stepdown procedure.

Missing value handling
Records with missing values can be handled listwise or analysis by analysis. User-missing values on categorical fields can be treated as valid or excluded from analysis.

Basic Specification

The basic specification is a single test subcommand (ONESAMPLE, INDEPENDENT, or RELATED) and a TEST keyword with a list of fields to be tested.

Syntax Rules

- At least one test subcommand must be specified; all other subcommands are optional.

- Subcommands may be specified in any order.
- Only a single instance of each subcommand is allowed.
- An error occurs if a keyword is specified more than once within a subcommand.
- Parentheses, equals signs, and slashes shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not allowed.
- Any split field defined on the SPLIT FILE command cannot be used on this command.
- Any field specified on the WEIGHT command cannot be used on this command.

Operations

Note: Since measurement level can affect the results, if any variables (fields) have an unknown measurement level, an initial data pass will be performed to determine default measurement level for any variables with an unknown measurement level. For information on the criteria used to determine default measurement level, see SET SCALEMIN.

MISSING Subcommand

The MISSING subcommand controls the treatment of records with missing values.

```
[/MISSING [SCOPE={ANALYSIS**}] [USERMISSING={EXCLUDE**}]]
                 {LISTWISE  }               {INCLUDE  }
```

SCOPE Keyword

The SCOPE keyword specifies how to determine the case basis for tests.

ANALYSIS. *Exclude records with missing values on a test-by-test basis.* Records with missing values for a field that is used for a specific test are omitted from that test. On subcommands that specify several tests, each test is evaluated separately. This setting is the default.

LISTWISE. *Exclude records with missing values listwise.* Records with missing values for any field that is named on any subcommand are excluded from all analyses.

USERMISSING Keyword

The USERMISSING keyword is used to control whether user-missing values for categorical fields are treated as valid values. By default, user-missing values for categorical fields are treated as invalid.

- User-missing values for continuous fields are always treated as invalid.
- System-missing values for any fields are always treated as invalid.

EXCLUDE. *User-missing values for categorical fields are treated as invalid.* This is the default.

INCLUDE. *User-missing values for categorical fields are treated as valid values.*

CRITERIA Subcommand

The CRITERIA subcommand controls the significance level used for tests and the confidence level used for intervals.

```
[/CRITERIA ALPHA={0.05**}] CILEVEL={95** }
                {value }          {value}
                [SEED={2000000**}{integer}{RANDOM}]
```

ALPHA keyword

The ALPHA keyword specifies the significance level for all tests specified on the NPTESTS command.

- Specify a numeric value between 0 and 1. 0.05 is the default.

CILEVEL keyword

The CILEVEL keyword specifies the confidence level for all confidence intervals produced by the NPTESTS command.

- Specify a numeric value between 0 and 100. 95 is the default.

SEED keyword

The SEED keyword resets the random seed used for the Monte Carlo sampling.

The integer in the parentheses must be a single integer between 1 and 2147483647, with 2000000 as the default setting. When **RANDOM** is specified, a random seed will be assigned by the procedure.

- Specify a single integer between 1 and 2147483647. 2000000 is the default.

ONESAMPLE Subcommand

The ONESAMPLE subcommand produces one-sample nonparametric tests. The TEST keyword is required, all other keywords are optional. If ONESAMPLE is specified with none of the optional keywords, the following tests are performed automatically:

- Categorical fields with two values are tested using a binomial test.
- Categorical fields with more than two values are tested using a chi-square test with equal frequencies on category values found in the sample.
- Continuous fields are tested using a Kolmogorov-Smirnov test against a normal distribution with the sample mean and standard deviation.

TEST keyword

The TEST keyword lists the fields that you want to test.

- Specify one or more fields. Note that certain tests are not applicable to fields of a particular measurement level; for example, the chi-square test is only performed for categorical fields. NPTESTS automatically determines which tests are applicable to which fields. See the individual keyword descriptions for details.

CHISQUARE keyword

```
[CHISQUARE([EXPECTED={EQUAL**                       }])]
                     {CUSTOM(FREQUENCIES=valuelist
                             CATEGORIES=valuelist) }
```

The CHISQUARE keyword produces a one-sample test that computes a chi-square statistic based on the differences between the observed and expected frequencies of categories of a field.

- A separate chi-square test is performed for each and every categorical field specified on the TEST keyword.
- The test specifications given on the CHISQUARE keyword apply to all chi-square tests performed.
- If CHISQUARE is specified without any keywords, equal frequencies are expected in each category.

EXPECTED = EQUAL|CUSTOM(FREQUENCIES=valuelist CATEGORIES=valuelist). *Expected frequencies.*

- The EXPECTED keyword defines how expected frequencies are derived. The default is EQUAL.
- EQUAL produces equal frequencies among all categories in the sample. This is the default when CHISQUARE is specified without any other keywords..
- CUSTOM allows you to specify unequal frequencies for a specified list of categories.
- On the CATEGORIES keyword, specify a list of string or numeric values. The values in the list do not need to be present in the sample.

- On the FREQUENCIES keyword, specify a value greater than 0 for each category, and in the same order as the categories, on the CATEGORIES keyword. Custom frequencies are treated as ratios so that, for example, FREQUENCIES=1 2 3 is equivalent to FREQUENCIES=10 20 30, and both specify that 1/6 of the records are expected to fall into the first category on the CATEGORIES keyword, 1/3 into the second, and 1/2 into the third.
- When CUSTOM is specified, the number of expected frequencies must match the number of category values; otherwise the test is not performed for that field.

BINOMIAL keyword

```
[BINOMIAL([TESTVALUE={0.5**}]
                     {value}
    [SUCCESSCATEGORICAL={FIRST**              }]
                        {LIST(valuelist)      }
    [SUCCESSCONTINUOUS=CUTPOINT({MIDPOINT**})]
                               {value     }
    [CLOPPERPEARSON] [JEFFREYS] [LIKELIHOOD]
 )]
```

The BINOMIAL keyword produces a one-sample test of whether the observed distribution of a dichotomous field is the same as what is expected from a specified binomial distribution. In addition, you can request confidence intervals.

- A separate binomial test is performed for each and every field specified on the TEST keyword.
- The test specifications given on the BINOMIAL keyword apply to all binomial tests performed.
- If BINOMIAL is specified without any keywords, each categorical field is assumed to have only two values and each continuous field is dichotomized using the average of the minimum and maximum as a cut point. The distribution of each named field is compared to a binomial distribution with *p* (the proportion of cases expected in the first category) equal to 0.5.

TESTVALUE. *Hypothesized proportion.* The TESTVALUE keyword specifies the expected proportion of cases in the first category. Specify a value greater than 0 and less than 1. The default is 0.5.

SUCCESSCATEGORICAL=FIRST|LIST(valuelist).

- The SUCCESSCATEGORICAL keyword specifies how "success", the data value(s) tested against the test value, is defined for categorical fields.
- FIRST performs the binomial test using the first value found in the sample to define "success". This option is only applicable to nominal or ordinal fields with only two values; all other categorical fields specified on a ONESAMPLE subcommand where FIRST is used will not be tested. This is the default.
- LIST performs the binomial test using the specified list of values to define "success". Specify a list of string or numeric values. The values in the list do not need to be present in the sample.

SUCCESSCONTINUOUS=CUTPOINT (MIDPOINT|value). *Define success for continuous fields.* The SUCCESSCONTINUOUS keyword specifies how "success", the data value(s) tested against the test value, is defined for continuous fields.

- CUTPOINT defines values that are equal to or less than the cut point as "success". MIDPOINT sets the cut point at the average of the minimum and maximum values. Alternatively, specify a value for the cut point. The default is MIDPOINT.

CLOPPERPEARSON. *Exact interval based on the cumulative binomial distribution.*

JEFFREYS. *Bayesian interval based on the posterior distribution of p using the Jeffreys prior.*

LIKELIHOOD. *Interval based on the likelihood function for p.*

KOLMOGOROV_SMIRNOV keyword

```
[KOLMOGOROV_SMIRNOV(
    [NSAMPLES={1000**}{integer}]
    [MC_CILEVEL-{99**}{value}]
    [NORMAL={SAMPLE**(SIMULATION={TRUE**}{FALSE})}
            {CUSTOM(MEAN=value
                    SD=value   )}]
    [UNIFORM={SAMPLE**           }]
             {CUSTOM(MIN=value
                     MAX=value   )}
    [EXPONENTIAL={SAMPLE**           }]
```

```
                    {CUSTOM(MEAN=value )}
    [POISSON=CUSTOM(MEAN=value )]
)]
```

The KOLMOGOROV_SMIRNOV keyword produces a one-sample test of whether the sample cumulative distribution function for a field is homogenous with a uniform, normal, Poisson, or exponential distribution.

- A separate Kolmogorov-Smirnov test is performed for each and every continuous and ordinal field specified on the TEST keyword.
- The test specifications given on the KOLMOGOROV_SMIRNOV keyword apply to all Kolmogorov-Smirnov tests performed.
- If KOLMOGOROV_SMIRNOV is specified without any keywords, each field is tested against a normal distribution using its sample mean and sample standard deviation.

NSAMPLES=integer. NSAMPLES resets the number of replicates used by the Lilliefors test for Monte Carlo sampling.

MC_CILEVEL=value.*Monte Carlo confidence interval.* MC_CILEVEL resets the confidence interval level that is estimated by the Kolmogorov-Smirnov test.

NORMAL (SAMPLE (SIMULATION=boolean)|CUSTOM (MEAN=value SD=value)). *Normal distribution.* SAMPLE uses the observed mean and standard deviation, SIMULATION controls whether the Monte Carlo simulation will be used to conduct the Lilliefors test for Normal distribution when the parameters are not specified, and CUSTOM allows you to specify parameters.

UNIFORM (SAMPLE|CUSTOM (MIN=value MAX=value)). *Uniform distribution.* SAMPLE uses the observed minimum and maximum, CUSTOM allows you to specify values.

POISSON=CUSTOM (MEAN=value)). *Poisson distribution.* CUSTOM allows you to specify a mean value.

EXPONENTIAL(SAMPLE|CUSTOM (MEAN=value)). *Exponential distribution.* SAMPLE uses the observed mean, CUSTOM allows you to specify a value.

RUNS keyword

```
    [RUNS([GROUPCATEGORICAL={SAMPLE**                 }]
                            {LIST(valuelist)          }
          [GROUPCONTINUOUS=CUTPOINT({SAMPLEMEDIAN**})]
                                    {SAMPLEMEAN    }
                                    {value         }
 )]
```

The RUNS keyword produces a one-sample test of whether the sequence of values of a dichotomized field is random.

- A separate runs test is performed for each and every field specified on the TEST keyword.
- The test specifications given on the RUNS keyword apply to all runs tests performed.
- If RUNS is specified without any keywords, each categorical field is assumed to have only two values and each continuous field is dichotomized using the sample median as a cut point.

GROUPCATEGORICAL= SAMPLE|LIST(valuelist). *Determine groups for categorical fields.* SAMPLE is the default.

- SAMPLE performs the runs test using the values found in the sample to define the groups. This option is only applicable to nominal or ordinal fields with only two values; all other categorical fields specified on a ONESAMPLE subcommand where SAMPLE is used will not be tested.
- LIST performs the runs test using the specified list of values to define one of the groups. All other values in the sample define the other group. The values in the list do not all need to be present in the sample, but at least one record must be in each group.

GROUPCONTINUOUS= CUTPOINT (SAMPLEMEDIAN | SAMPLEMEAN | value). *Determine groups for continuous fields.* CUTPOINT defines values that are equal to or less than the cut point as the first group; all other values define the other group. SAMPLEMEDIAN sets the cut point at the sample median. SAMPLEMEAN sets the cut point at the sample men. Alternatively, specify a value for the cut point. The default is SAMPLEMEDIAN.

WILCOXON keyword

```
[WILCOXON(TESTVALUE=value)]
```

The WILCOXON keyword produces a one sample test of median value of a field.

- A separate Wilcoxon test is performed for each and every continuous and ordinal field specified on the TEST keyword.
- The test specifications given on the WILCOXON keyword apply to all Wilcoxon tests performed.
- The TESTVALUE keyword is required.

TESTVALUE=value. *Hypothesized median.* The Wilcoxon test is performed using the specified value. The TESTVALUE keyword is required. There is no default.

INDEPENDENT Subcommand

The INDEPENDENT subcommand produces independent samples nonparametric tests. The TEST and GROUP keywords are required, all other keywords are optional. If INDEPENDENT is specified with none of the optional keywords, the following tests are performed automatically:

- If the grouping field has only two values, continuous fields are tested using a Mann-Whitney test.
- If the grouping field has more than two values, continuous fields are tested using a Kruskal-Wallis test.
- If there are no continuous fields on the TEST keyword, no tests are performed.

TEST Keyword

The TEST keyword lists the fields that you want to test.

- Specify one or more continuous fields. Independent samples tests are only applicable to continuous fields.

GROUP Keyword

The GROUP keyword specifies the field used to determine groups. The test is performed using the values found in the sample to define the groups. Note that certain tests are not applicable when there are more than two groups. NPTESTS automatically determines which tests are applicable. See the individual keyword descriptions for details.

- Specify a categorical field. A grouping field that does not have nominal or ordinal measurement level will cause an error.
- GROUP applies to all fields that are specified on the TEST keyword.

MANN_WHITNEY Keyword

The MANN_WHITNEY keyword produces an independent samples test of whether two samples are from the same population. The test statistic uses the rank of each case to test whether the groups are drawn from the same population.

- A separate test is performed for each and every continuous and ordinal field specified on the TEST keyword.
- The first value in ascending order of the grouping field defines the first group and the second value defines the second. If the grouping field has more than two values, the test is not performed.

WALD_WOLFOWITZ Keyword

The WALD_WOLFOWITZ keyword produces an independent samples test whether the distribution of a field is the same in two independent samples. A runs test is performed with group membership as the criterion.

- A separate test is performed for each and every continuous and ordinal field specified on the TEST keyword.

- The first value in ascending order of the grouping field defines the first group and the second value defines the second. If the grouping field has more than two values, the test is not performed.

KOLMOGOROV_SMIRNOV Keyword

The KOLMOGOROV_SMIRNOV keyword produces an independent samples test of whether the distribution of a field is the same in two independent samples that are defined by a grouping field. The test is sensitive to any difference in median, dispersion, skewness, and so forth, between the two distributions.

- A separate test is performed for each and every continuous and ordinal field specified on the TEST keyword.
- The first value in ascending order of the grouping field defines the first group and the second value defines the second. If the grouping field has more than two values, the test is not performed.

MOSES Keyword

```
[MOSES([TRIMOUTLIERS={SAMPLE**}])]
                     {integer }
```

The MOSES keyword produces an independent samples test of whether the range of a continuous or ordinal field is the same in a control group and a comparison group.

- A separate test is performed for each and every continuous and ordinal field specified on the TEST keyword.
- The test specifications given on the MOSES keyword apply to all Moses tests performed.
- The first value in ascending order of the grouping field defines the control group and the second defines the comparison group. If the grouping field has more than two values, the test is not performed.

TRIMOUTLIERS= SAMPLE|integer. *Records to trim from each end of the range of the control group to remove outliers.* SAMPLE trims 5% of the records from each end of the range of the control group to remove outliers, with a minimum of 1 record removed from each end. Specifying an integer causes that exact number of records to be trimmed. SAMPLE is the default.

HODGES_LEHMAN Keyword

The HODGES_LEHMAN keyword produces an independent samples estimate and confidence interval for the difference in the medians of two groups.

- A separate confidence interval is computed for each and every continuous and ordinal field specified on the TEST keyword.
- The first value in ascending order of the grouping field defines the first group and the second value defines the second. If the grouping field has more than two values, the confidence interval is not produced.

KRUSKAL_WALLIS Keyword

```
[KRUSKAL_WALLIS([COMPARE={PAIRWISE**}])]
                         {STEPWISE  }
                         {NONE      }
```

The KRUSKAL_WALLIS keyword produces an independent samples test of whether *k* independent samples that are defined by a grouping field are from the same population.

- A separate test is performed for each and every continuous and ordinal field specified on the TEST keyword.
- The test specifications given on the KRUSKAL_WALLIS keyword apply to all tests performed.

COMPARE= PAIRWISE | STEPWISE | NONE. *Multiple comparisons.* The COMPARE keyword controls how and whether multiple comparisons should be performed. PAIRWISE produces all pairwise multiple comparisons. STEPWISE produces stepwise stepdown comparisons. NONE turns off multiple comparisons. By default, all pairwise comparisons are produced.

MEDIAN Keyword

```
[MEDIAN([TESTVALUE={SAMPLE**}]  [COMPARE={PAIRWISE**}])]
                   {value   }            {STEPWISE  }
                                         {NONE      }
```

The MEDIAN keyword produces an independent samples test of whether *k* independent samples are drawn from populations with the same median.

- A separate test is performed for each and every continuous and ordinal field specified on the TEST keyword.
- The test specifications given on the MEDIAN keyword apply to all tests performed.

TESTVALUE= SAMPLE|value. *Median value to use in the test.* SAMPLE causes the test to be performed using the sample median, calculated from all records that are included in the test. Alternatively, you can specify an exact value. SAMPLE is the default.

COMPARE= PAIRWISE| STEPWISE| NONE. *Multiple comparisons.* The COMPARE keyword controls how and whether multiple comparisons should be performed. PAIRWISE produces all pairwise multiple comparisons. STEPWISE produces stepwise stepdown comparisons. NONE turns off multiple comparisons. By default, all pairwise comparisons are produced.

JONCKHEERE_TERPSTRA Keyword

```
[JONCKHEERE_TERPSTRA(
    [ORDER={ASCENDING**      }]  [COMPARE={PAIRWISE**}]
           {DESCENDING       }            {STEPWISE  }
                                          {NONE      }
 )]
```

The JONCKHEERE-TERPSTRA keyword produces an independent samples test of whether *k* independent samples that are defined by a grouping field are from the same population. This test is particularly powerful when the *k* populations have a natural ordering.

- A separate test is performed for each and every continuous and ordinal field specified on the TEST keyword.
- The test specifications given on the JONCKHEERE_TERPSTRA keyword apply to all tests performed.

ORDER=ASCENDING| DESCENDING. *Order of the k populations.* ASCENDING specifies the alternative hypothesis that the location parameter of the first group is less than or equal to the second, which is less than or equal to the third, and so on. This is the default. DESCENDING specifies the alternative hypothesis that the location parameter of the first group is greater than or equal to the second, which is greater than or equal to the third, and so on. For both options, the alternative hypothesis also assumes that the locations are not all equal.

COMPARE=PAIRWISE|STEPWISE|NONE. *Multiple comparisons.* The COMPARE keyword controls how and whether multiple comparisons should be performed. PAIRWISE produces all pairwise multiple comparisons. STEPWISE produces stepwise stepdown comparisons. NONE turns off multiple comparisons. By default, all pairwise comparisons are produced.

RELATED Subcommand

The RELATED subcommand produces related-samples nonparametric tests. The TEST keyword is required, all other keywords are optional. If RELATED is specified with none of the optional keywords, the following tests are performed automatically:

- If two categorical fields are specified and they have only two values, McNemar's test is performed.
- If two continuous fields are specified, a Wilcoxon Matched-Pairs Signed-Rank test is performed.
- If more than two categorical fields are specified and they each have only two values, Cochran's test is performed.
- If more than two continuous fields are specified, Friedman's Two-Way Analysis of Variance by Ranks is performed.

- Otherwise, no tests are performed.

When fields of differing measurement level are specified, they are first separated by measurement level and then the appropriate test is applied to each group. For example, if you specify:

```
NPTESTS
  /RELATED TEST(c1 n1 o1 c2 o2 n2 o3 c3 o4).
```

Where *x1*, *x2*, and *x3* are continuous fields, *a1* and *a2* are nominal fields, and *o1*, *o2*, *o3*, and *o4* are ordinal fields, then Friedman's test is applied to the continuous fields, McNemar's test is applied to the nominal fields, and no test is applied to the ordinal fields.

TEST Keyword

The TEST keyword lists the fields that you want to test.

- Specify one or more fields. Note that certain tests are not applicable to fields of a particular measurement level; for example, the McNemar test is only performed for categorical fields. NPTESTS automatically determines which tests are applicable to which fields. See the individual keyword descriptions for details.

```
     [MCNEMAR( [SUCCESS={FIRST**        }])]
                        {LIST(valuelist)}
```

The MCNEMAR keyword produces a related samples test of whether combinations of values between two dichotomous fields are equally likely.

- A separate test is performed for all of the nominal fields and for all of the ordinal fields specified on the TEST keyword.
- The test specifications given on the MCNEMAR keyword apply to all tests performed.
- If any test has more than two fields, or any field has more than two values, the test is not performed.

Example

```
NPTESTS /RELATED TEST(n1 n2 n3 o1 o2) MCNEMAR.
```

If *n1*, *n2*, *n3* are nominal fields and *o1* and *o2* are ordinal fields, then no test is performed for the nominal fields and a test is performed for the ordinal fields so long as *o1* and *o2* each have only two values.

SUCCESS= FIRST|LIST(valuelist). The SUCCESS keyword specifies how "success" is defined for categorical fields.

- FIRST performs the test using the first value found in the sample to define "success". This option is only applicable to nominal or ordinal fields with only two values; all other categorical fields specified on a RELATED subcommand where FIRST is used will not be tested. This is the default.
- LIST performs the test using the specified list of values to define "success". Specify a list of string or numeric values. The values in the list do not need to be present in the sample.

WILCOXON Keyword

The WILCOXON keyword produces a related samples test of whether the distribution of two paired fields in two related samples is the same. The test takes into account the magnitude of the differences between two paired fields.

- A single test is performed for all of the continuous and ordinal fields specified on the TEST keyword.
- If there are more than two continuous or ordinal fields, the test is not performed.

SIGN Keyword

The SIGN keyword produces a related samples test of whether the distribution of two paired fields in a two-related-samples test is the same.

- A single test is performed for all of the continuous and ordinal fields specified on the TEST keyword.
- If there are more than two continuous or ordinal fields, the test is not performed.

MARGINAL_HOMOGENEITY Keyword

The MARGINAL_HOMOGENEITY keyword produces a related samples test of whether combinations of values between two paired ordinal fields are equally likely. The marginal homogeneity test is typically used in repeated measures situations. This test is an extension of the McNemar test from binary response to multinomial response.

- A single test is performed for all of the ordinal fields specified on the TEST keyword.
- If there are more than two ordinal fields, the test is not performed.

HODGES_LEHMAN Keyword

The HODGES_LEHMAN keyword produces a related samples estimate and confidence interval for the median difference between two paired fields.

- A single confidence interval is computed for all of the continuous and ordinal fields specified on the TEST keyword.
- If there are more than two continuous or ordinal fields, no confidence intervals are created.

```
    [COCHRAN(
        [SUCCESS={FIRST**        }]
                 {LIST(valuelist)}
        [COMPARE={PAIRWISE**}]
                 {STEPWISE  }
                 {NONE      }
     )]
```

The COCHRAN keyword produces a related samples test of whether the distribution of values is the same for *k* related dichotomous fields.

- A separate test is performed for all of the nominal fields and for all of the ordinal fields specified on the TEST keyword.
- The test specifications given on the COCHRAN keyword apply to all tests performed.
- If any field has more than two values, the test is not performed.

Example

```
NPTESTS /RELATED TEST(n1 n2 n3 o1 o2) COCHRAN.
```

If *n1*, *n2*, *n3* are nominal fields and *o1* and *o2* are ordinal fields, then a test is performed for the nominal fields and a test is performed for the ordinal fields so long as each field has only two values.

SUCCESS= FIRST|LIST(valuelist). The SUCCESS keyword specifies how "success" is defined for categorical fields.

- FIRST performs the test using the first value found in the sample to define "success". This option is only applicable to nominal or ordinal fields with only two values; all other categorical fields specified on a RELATED subcommand where FIRST is used will not be tested. This is the default.
- LIST performs the test using the specifed list of values to define "success". Specify a list of string or numeric values. The values in the list do not need to be present in the sample.

COMPARE= PAIRWISE| STEPWISE|NONE. *Multiple comparisons.* The COMPARE keyword controls how and whether multiple comparisons should be performed. PAIRWISE produces all pairwise multiple comparisons. STEPWISE produces stepwise stepdown comparisons. NONE turns off multiple comparisons. By default, all pairwise comparisons are produced.

FRIEDMAN Keyword

```
    [FRIEDMAN([COMPARE={PAIRWISE**}])]
                       {STEPWISE  }
                       {NONE      }
```

The FRIEDMAN subcommand produces a related samples test of whether *k* related samples have been drawn from the same population.

A single test is performed for all of the continuous and ordinal fields specified on the TEST keyword.

COMPARE= PAIRWISE| STEPWISE| NONE. *Multiple comparisons.* The COMPARE keyword controls how and whether multiple comparisons should be performed. PAIRWISE produces all pairwise multiple

comparisons. STEPWISE produces stepwise stepdown comparisons. NONE turns off multiple comparisons. By default, all pairwise comparisons are produced.

KENDALL Keyword

```
[KENDALL([COMPARE={PAIRWISE**}])]
                  {STEPWISE  }
                  {NONE      }
```

The KENDALL keyword produces a related samples test of whether *k* related samples are from the same population. Kendall's *W* is a measure of agreement among judges or raters, where each case is one judge's rating of several items (fields).

A single test is performed for all of the continuous and ordinal fields specified on the TEST keyword.

COMPARE= PAIRWISE| STEPWISE| NONE. *Multiple comparisons.* The COMPARE keyword controls how and whether multiple comparisons should be performed. PAIRWISE produces all pairwise multiple comparisons. STEPWISE produces stepwise stepdown comparisons. NONE turns off multiple comparisons. By default, all pairwise comparisons are produced.

NPAR TESTS

NPAR TESTS is available in Statistics Base Edition.

```
NPAR TESTS [CHISQUARE=varlist[(lo,hi)]/] [/EXPECTED={EQUAL      }]
                                                    {f1,f2,...fn}

 [/K-S({UNIFORM [min,max]   })=varlist]
       {NORMAL [mean,stddev]}
       {POISSON [mean]      } @@
       {EXPONENTIAL [mean]  }

 [/KS_SIM
       [CIN({99**}{value})] [SAMPLES({10000**}{integer})] [NONORMAL] ]

 [/RUNS({MEAN  })=varlist]
        {MEDIAN}
        {MODE  }
        {value }

 [/BINOMIAL[({.5})]=varlist[({value1,value2})]]
             { p}            {value        }

 [/MCNEMAR=varlist [WITH varlist [(PAIRED)]]]

 [/SIGN=varlist [WITH varlist [(PAIRED)]]]

 [/WILCOXON=varlist [WITH varlist [(PAIRED)]]]

    |/MH=varlist [WITH varlist [(PAIRED)]]]††

 [/COCHRAN=varlist]

 [/FRIEDMAN=varlist]

 [/KENDALL=varlist]

 [/M-W=varlist BY var (value1,value2)]

 [/K-S=varlist BY var (value1,value2)]

 [/W-W=varlist BY var (value1,value2)]

 [/MOSES[(n)]=varlist BY var (value1,value2)]

 [/K-W=varlist BY var (value1,value2)]

 [/J-T=varlist BY var (value1, value2)]††

 [/MEDIAN[(value)]=varlist BY var (value1,value2)]

 [/MISSING=[{ANALYSIS**}]  [INCLUDE]]
           {LISTWISE  }

 [/SAMPLE]

 [/STATISTICS=[DESCRIPTIVES]  [QUARTILES] [ALL]]


    [/METHOD={MC [CIN({99.0 })] [SAMPLES({10000})] }]††
                    {value}             {value}
          {EXACT [TIMER({5    })]             }
                        {value}
```

**Default if the subcommand is omitted.

@@ An error message is produced when **varlist** is specified for the **POISSON** keyword.

††Available only if SPSS Statistics Premium Edition or the Exact Tests option is installed (available only on Windows operating systems).

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 17.0

- Increased limits on number of variables allowed in the analysis.

Release 27

- Added support for the **KS_SIM** subcommand.
  - Added support for the **CIN** keyword in the **KS_SIM** subcommand.
  - Added support for the **SAMPLES** keyword in the **KS_SIM** subcommand.

  - Added support for the **NONORMAL** keyword in the **KS_SIM** subcommand.
- Removed support for **POISSON=varlist** in the **K-S** subcommand.

Example

```
NPAR TESTS K-S(UNIFORM)=V1 /K-S(NORMAL,0,1)=V2.
```

Overview

NPAR TESTS is a collection of nonparametric tests. These tests make minimal assumptions about the underlying distribution of the data. [50] In addition to the nonparametric tests that are available in NPAR TESTS, the *k*-sample chi-square and Fisher's exact test are available in procedure CROSSTABS.

The tests that are available in NPAR TESTS can be grouped into three broad categories based on how the data are organized: one-sample tests, related-samples tests, and independent-samples tests. A one-sample test analyzes one variable. A test for related samples compares two or more variables for the same set of cases. An independent-samples test analyzes one variable that is grouped by categories of another variable.

The one-sample tests that are available in procedure NPAR TESTS are:

- BINOMIAL
- CHISQUARE
- K-S (Kolmogorov-Smirnov)
- RUNS

Tests for two related samples are:

- MCNEMAR
- SIGN
- WILCOXON

Tests for *k* related samples are:

- COCHRAN
- FRIEDMAN
- KENDALL

Tests for two independent samples are:

- M-W (Mann-Whitney)
- K-S (Kolmogorov-Smirnov)
- W-W (Wald-Wolfowitz)
- MOSES

Tests for *k* independent samples are:

- K-W (Kruskal-Wallis)
- MEDIAN

[50] Siegel, S., and N. J. Castellan. 1988. *Nonparametric statistics for the behavioral sciences*. New York: McGraw-Hill, Inc..

Options

Statistical Display

In addition to the tests, you can request univariate statistics, quartiles, and counts for all variables that are specified on the command. You can also control the pairing of variables in tests for two related samples.

Random Sampling

NPAR TESTS must store cases in memory when computing tests that use ranks. You can use random sampling when there is not enough space to store all cases.

Basic specification

The basic specification is a single test subcommand and a list of variables to be tested. Some tests require additional specifications. CHISQUARE has an optional subcommand.

Subcommand order

Subcommands can be used in any order.

Syntax rules

- The KS_SIM, STATISTICS, SAMPLE, and MISSING subcommands are optional. Each subcommand can be specified only once per NPAR TESTS command.
- You can request any or all tests, and you can specify a test subcommand more than once on a single NPAR TESTS command.
- If you specify a variable more than once on a test subcommand, only the first variable is used.
- Keyword ALL in any variable list refers to all user-defined variables in the active dataset.
- Keyword WITH controls pairing of variables in two-related-samples tests.
- Keyword BY introduces the grouping variable in two- and *k*-independent-samples tests.
- Keyword PAIRED can be used with keyword WITH on the MCNEMAR, SIGN, and WILCOXON subcommands to obtain sequential pairing of variables for two related samples.
- POISSON = varlist is not supported in the **K-S** subcommand.

Operations

- If a string variable is specified on any subcommand, NPAR TESTS will stop executing.
- When ALL is used, requests for tests of variables with themselves are ignored and a warning is displayed.

Limitations

- A maximum of 100 subcommands is allowed.
- A maximum of 48,000 variables total per NPAR TESTS command is allowed.
- A maximum of 200 values for subcommand CHISQUARE is allowed.

BINOMIAL Subcommand

```
NPAR TESTS BINOMIAL [({.5})]=varlist[({value,value})]
                    {p  }            {value      }
```

BINOMIAL tests whether the observed distribution of a dichotomous variable is the same as what is expected from a specified binomial distribution. By default, each named variable is assumed to have only two values, and the distribution of each named variable is compared to a binomial distribution with *p* (the proportion of cases expected in the first category) equal to 0.5. The default output includes the number of valid cases in each group, the test proportion, and the two-tailed probability of the observed proportion.

Syntax

- The minimum specification is a list of variables to be tested.
- To change the default 0.5 test proportion, specify a value in parentheses immediately after keyword BINOMIAL.
- A single value in parentheses following the variable list is used as a cutting point. Cases with values that are equal to or less than the cutting point form the first category; the remaining cases form the second category.
- If two values appear in parentheses after the variable list, cases with values that are equal to the first value form the first category, and cases with values that are equal to the second value form the second category.
- If no values are specified, the variables must be dichotomous. The first value encountered in the dataset defines the first category, and the other value defines the second category.

Operations

- The proportion observed in the first category is compared to the test proportion. The probability of the observed proportion occurring given the test proportion and a binomial distribution is then computed. A test statistic is calculated for each variable specified.
- If the test proportion is the default (0.5), a two-tailed probability is displayed. For any other test proportion, a one-tailed probability is displayed. The direction of the one-tailed test depends on the observed proportion in the first category. If the observed proportion is more than the test proportion, the significance of observing that many or more in the first category is reported. If the observed proportion is less than or equal to the test proportion, the significance of observing that many or fewer in the first category is reported. In other words, the test is always done in the observed direction.

Example

```
NPAR TESTS BINOMIAL(.667)=V1(0,1).
```

- NPAR TESTS displays the Binomial Test table, showing the number of cases, observed proportion, test proportion (0.667), and the one-tailed significance for each category.
- If more than 0.667 of the cases have value 0 for *V1*, BINOMIAL gives the probability of observing that many or more values of 0 in a binomial distribution with probability 0.667. If fewer than 0.667 of the cases are 0, the test will be of observing that many or fewer values.

CHISQUARE Subcommand

```
NPAR TESTS CHISQUARE=varlist [(lo,hi)] [/EXPECTED={EQUAL**     }]
                                                  {f1,f2,..., fn}
```

The CHISQUARE (alias CHI-SQUARE) one-sample test computes a chi-square statistic based on the differences between the observed and expected frequencies of categories of a variable. By default, equal frequencies are expected in each category. The output includes the frequency distribution, expected frequencies, residuals, chi-square, degrees of freedom, and probability.

Syntax

- The minimum specification is a list of variables to be tested. Optionally, you can specify a value range in parentheses following the variable list. You can also specify expected proportions with the EXPECTED subcommand.
- If you use the EXPECTED subcommand to specify unequal expected frequencies, you must specify a value greater than 0 for each observed category of the variable. The expected frequencies are specified in ascending order of category value. You can use the notation *n*f* to indicate that frequency *f* is expected for *n* consecutive categories.
- Specifying keyword EQUAL on the EXPECTED subcommand has the same effect as omitting the EXPECTED subcommand.

- EXPECTED applies to all variables that are specified on the CHISQUARE subcommand. Use multiple CHISQUARE and EXPECTED subcommands to specify different expected proportions for variables.

Operations

- If no range is specified for the variables that are to be tested, a separate Chi-Square Frequency table is produced for each variable. Each distinct value defines a category.
- If a range is specified, integer-valued categories are established for each value within the range. Non-integer values are truncated before classification. Cases with values that are outside the specified range are excluded. One combined Chi-Square Frequency table is produced for all specified variables.
- Expected values are interpreted as proportions, not absolute values. Values are summed, and each value is divided by the total to calculate the proportion of cases expected in the corresponding category.
- A test statistic is calculated for each specified variable.

Example

```
NPAR TESTS CHISQUARE=V1 (1,5) /EXPECTED= 12, 3*16, 18.
```

- This example requests the chi-square test for values 1 through 5 of variable *V1*.
- The observed frequencies for variable *V1* are compared with the hypothetical distribution of 12/78 occurrences of value 1; 16/78 occurrences each of values 2, 3, and 4; and 18/78 occurrences of value 5.

COCHRAN Subcommand

```
NPAR TESTS COCHRAN=varlist
```

COCHRAN calculates Cochran's *Q,* which tests whether the distribution of values is the same for *k* related dichotomous variables. The output shows the frequency distribution for each variable in the Cochran Frequencies table and the number of cases, Cochran's *Q*, degrees of freedom, and probability in the Test Statistics table.

Syntax

- The minimum specification is a list of two variables.
- The variables must be dichotomous and must be coded with the same two values.

Operations

- A *k* × 2 contingency table (variables by categories) is constructed for dichotomous variables, and the proportions for each variable are computed. A single test is calculated, comparing all variables.
- Cochran's *Q* statistic has approximately a chi-square distribution.

Example

```
NPAR TESTS COCHRAN=RV1 TO RV3.
```

- This example tests whether the distribution of values 0 and 1 for *RV1*, *RV2*, and *RV3* is the same.

FRIEDMAN Subcommand

```
NPAR TESTS FRIEDMAN=varlist
```

FRIEDMAN tests whether *k* related samples have been drawn from the same population. The output shows the mean rank for each variable in the Friedman Ranks table and the number of valid cases, chi-square, degrees of freedom, and probability in the Test Statistics table.

Syntax

- The minimum specification is a list of two variables.
- Variables should be at least at the ordinal level of measurement.

Operations

- The values of *k* variables are ranked from 1 to *k* for each case, and the mean rank is calculated for each variable over all cases.
- The test statistic has approximately a chi-square distribution. A single test statistic is calculated, comparing all variables.

Example

```
NPAR TESTS FRIEDMAN=V1 V2 V3
  /STATISTICS=DESCRIPTIVES.
```

- This example tests variables *V1, V2*, and *V3*, and the example requests univariate statistics for all three variables.

J-T Subcommand

```
NPAR TESTS /J-T=varlist BY variable(value1,value2)
```

J-T (alias JONCKHEERE-TERPSTRA) performs the Jonckheere-Terpstra test, which tests whether *k* independent samples that are defined by a grouping variable are from the same population. This test is particularly powerful when the *k* populations have a natural ordering. The output shows the number of levels in the grouping variable; the total number of cases; observed, standardized, mean, and standard deviation of the test statistic; the two-tailed asymptotic significance; and, if a /METHOD subcommand is specified, one-tailed and two-tailed exact or Monte Carlo probabilities. This subcommand is available only if SPSS Statistics Premium Edition or the Exact Tests option is installed.

Syntax

- The minimum specification is a test variable, the keyword BY, a grouping variable, and a pair of values in parentheses.
- Every value in the range defined by the pair of values for the grouping variable forms a group.
- If the / METHOD subcommand is specified, and the number of populations, *k*, is greater than 5, the *p* value is estimated by using the Monte Carlo sampling method. The exact *p* value is not available when *k* exceeds 5.

Operations

- Cases from the *k* groups are ranked in a single series, and the rank sum for each group is computed. A test statistic is calculated for each variable that is specified before BY.
- The Jonckheere-Terpstra statistic has approximately a normal distribution.
- Cases with values other than values in the range that is specified for the grouping variable are excluded.
- The direction of a one-tailed inference is indicated by the sign of the standardized test statistic.

Example

```
NPAR TESTS /J-T=V1 BY V2(0,4)
  /METHOD=EXACT.
```

- This example performs the Jonckheere-Terpstra test for groups that are defined by values 0 through 4 of *V2*. The exact *p* values are calculated.

K-S Subcommand (One-Sample)

```
NPAR TESTS K-S({NORMAL [mean,stddev]})=varlist
              {POISSON [mean]      }
```

```
                {UNIFORM [min,max]   }
                {EXPONENTIAL [mean]  }
```

The K-S (alias KOLMOGOROV-SMIRNOV) one-sample test compares the cumulative distribution function for a variable with a uniform, normal, Poisson, or exponential distribution, and the test tests whether the distributions are homogeneous. The parameters of the test distribution can be specified; the defaults are the observed parameters. The output shows the number of valid cases, parameters of the test distribution, most-extreme absolute, positive, and negative differences, Kolmogorov-Smirnov *Z*, and two-tailed probability for each variable.

Syntax

The minimum specification is a distribution keyword and a list of variables. The distribution keywords are NORMAL, POISSON, EXPONENTIAL, and UNIFORM.

- The distribution keyword and its optional parameters must be enclosed within parentheses.
- The distribution keyword must be separated from its parameters by blanks or commas.

NORMAL [mean, stdev]. *Normal distribution.* The default parameters are the observed mean and standard deviation.

POISSON [mean]. *Poisson distribution.* The default parameter is the observed mean.

Note: POISSON = varlist is not supported in the **K-S** subcommand.

UNIFORM [min,max]. *Uniform distribution.* The default parameters are the observed minimum and maximum values.

EXPONENTIAL [mean]. *Exponential distribution.* The default parameter is the observed mean.

Operations

- The Kolmogorov-Smirnov *Z* is computed from the largest difference in absolute value between the observed and test distribution functions.
- The K-S probability levels assume that the test distribution is specified entirely in advance. The distribution of the test statistic and resulting probabilities are different when the parameters of the test distribution are estimated from the sample. The K-S test for normality uses the Lilliefors adjustment if you do not enter a population mean and standard deviation. The power of the test to detect departures from the hypothesized distribution may be seriously diminished. For testing against a normal distribution with estimated parameters, consider the adjusted K-S Lilliefors test that is available in the EXAMINE procedure.
- For a mean of 100,000 or larger, a normal approximation to the Poisson distribution is used.
- A test statistic is calculated for each specified variable.

Example

```
NPAR TESTS K-S(UNIFORM)=V1 /K-S(NORMAL,0,1)=V2.
```

- The first K-S subcommand compares the distribution of *V1* with a uniform distribution that has the same range as *V1*.
- The second K-S subcommand compares the distribution of *V2* with a normal distribution that has a mean of 0 and a standard deviation of 1.

K-S Subcommand (Two-Sample)

```
NPAR TESTS K-S=varlist BY variable(value1,value2)
```

K-S (alias KOLMOGOROV-SMIRNOV) tests whether the distribution of a variable is the same in two independent samples that are defined by a grouping variable. The test is sensitive to any difference in median, dispersion, skewness, and so forth, between the two distributions. The output shows the valid number of cases in each group in the Frequency table. The output also shows the largest absolute,

positive, and negative differences between the two groups, the Kolmogorov-Smirnov *Z*, and the two-tailed probability for each variable in the Test Statistics table.

Syntax

- The minimum specification is a test variable, the keyword BY, a grouping variable, and a pair of values in parentheses.
- The test variable should be at least at the ordinal level of measurement.
- Cases with the first value form one group, and cases with the second value form the other group. The order in which values are specified determines which difference is the largest positive and which difference is the largest negative.

Operations

- The observed cumulative distributions are computed for both groups, as are the maximum positive, negative, and absolute differences. A test statistic is calculated for each variable that is named before BY.
- Cases with values other than values that are specified for the grouping variable are excluded.

Example

```
NPAR TESTS K-S=V1 V2 BY V3(0,1).
```

- This example specifies two tests. The first test compares the distribution of *V1* for cases with value 0 for *V3* with the distribution of *V1* for cases with value 1 for *V3*.
- A parallel test is calculated for *V2*.

KS_SIM Subcommand (One-Sample)

```
NPAR TESTS
   [K-S({NORMAL=varlist
         {POISSON}
         {UNIFORM}
         {EXPONENTIAL}
   [/KS_SIM ({CIN [value]} {SAMPLES [integer]} {NONORMAL}) ]
```

The optional KS_SIM (alias KOLMOGOROV-SMIRNOV simulation) one-sample subcommand takes effect when a continuous distribution, **Normal**, **Uniform**, or **Exponential**, is specified on the **K-S** one-sample subcommand (without parameters being specified). The KS_SIM subcommand controls the parameters for the Monte Carlo simulation for Normal, Uniform, and Exponential distributions.

Syntax

CIN [value]. *Confidence interval.* Resets the estimated confidence interval level used by the Kolmogorov-Smirnov test (using the Monte Carlo simulations).

SAMPLES [integer]. *Monte Carlo sampling.* resets the number of replicates used by the Lilliefors test for Monte Carlo sampling.

NONORMAL. *Do not include Monte Carlo sampling.* When specified, results will not include the Monte Carlo sampling for Normal distribution.

Example

```
NPAR TESTS
  /K-S(NORMAL)=agecat gender
  /K-S(UNIFORM)=agecat gender
  /K-S(POISSON 1)=agecat gender
  /K-S(EXPONENTIAL)=agecat gender
  /KS_SIM CIN(99) SAMPLES(10000).
```

K-W Subcommand

```
NPAR TESTS K-W=varlist BY variable(value1,value2)
```

K-W (alias KRUSKAL-WALLIS) tests whether *k* independent samples that are defined by a grouping variable are from the same population. The output shows the number of valid cases and the mean rank of the variable in each group in the Ranks table. the output also shows the chi-square, degrees of freedom, and probability in the Test Statistics table.

Syntax

- The minimum specification is a test variable, the keyword BY, a grouping variable, and a pair of values in parentheses.
- Every value in the range defined by the pair of values for the grouping variable forms a group.

Operations

- Cases from the *k* groups are ranked in a single series, and the rank sum for each group is computed. A test statistic is calculated for each variable that is specified before BY.
- Kruskal-Wallis *H* has approximately a chi-square distribution.
- Cases with values other than values in the range that is specified for the grouping variable are excluded.

Example

```
NPAR TESTS K-W=V1 BY V2(0,4).
```

- This example tests *V1* for groups that are defined by values 0 through 4 of *V2*.

KENDALL Subcommand

```
NPAR TESTS KENDALL=varlist
```

KENDALL tests whether *k* related samples are from the same population. *W* is a measure of agreement among judges or raters, where each case is one judge's rating of several items (variables). The output includes the mean rank for each variable in the Ranks table and the valid number of cases, Kendall's *W*, chi-square, degrees of freedom, and probability in the Test Statistics table.

Syntax

The minimum specification is a list of two variables.

Operations

- The values of the *k* variables are ranked from 1 to *k* for each case, and the mean rank is calculated for each variable over all cases. Kendall's *W* and a corresponding chi-square statistic are calculated, correcting for ties. In addition, a single test statistic is calculated for all variables.
- *W* ranges between 0 (no agreement) and 1 (complete agreement).

Example

```
DATA LIST /V1 TO V5 1-10.
BEGIN DATA
2 5 4 5 1
3 3 4 5 3
3 4 4 6 2
2 4 3 6 2
END DATA.
NPAR TESTS KENDALL=ALL.
```

- This example tests four judges (cases) on five items (variables *V1* through *V5*).

M-W Subcommand

```
NPAR TESTS M-W=varlist BY variable(value1,value2)
```

M-W (alias MANN-WHITNEY) tests whether two independent samples that are defined by a grouping variable are from the same population. The test statistic uses the rank of each case to test whether the groups are drawn from the same population. The output shows the number of valid cases of each group; the mean rank of the variable within each group and the sum of ranks in the Ranks table and the Mann-Whitney *U*; Wilcoxon *W* (the rank sum of the smaller group); *Z* statistic; and probability in the Test Statistics table.

Syntax

- The minimum specification is a test variable, the keyword BY, a grouping variable, and a pair of values in parentheses.
- Cases with the first value form one group and cases with the second value form the other group. The order in which the values are specified is unimportant.

Operations

- Cases are ranked in order of increasing size, and test statistic *U* (the number of times that a score from group 1 precedes a score from group 2) is computed.
- An exact significance level is computed if there are 40 or fewer cases. For more than 40 cases, *U* is transformed into a normally distributed *Z* statistic, and a normal approximation *p* value is computed.
- A test statistic is calculated for each variable that is named before BY.
- Cases with values other than values that are specified for the grouping variable are excluded.

Example

```
NPAR TESTS M-W=V1 BY V2(1,2).
```

- This example tests *V1* based on the two groups that are defined by values 1 and 2 of *V2*.

MCNEMAR Subcommand

```
NPAR TESTS MCNEMAR=varlist [WITH varlist [(PAIRED)]]
```

MCNEMAR tests whether combinations of values between two dichotomous variables are equally likely. The output includes a Crosstabulation table for each pair and a Test Statistics table for all pairs, showing the number of valid cases, chi-square, and probability for each pair.

Syntax

- The minimum specification is a list of two variables. Variables must be dichotomous and must have the same two values.
- If keyword WITH is not specified, each variable is paired with every other variable in the list.
- If WITH is specified, each variable before WITH is paired with each variable after WITH. If PAIRED is also specified, the first variable before WITH is paired with the first variable after WITH, the second variable before WITH is paired with the second variable after WITH, and so on. PAIRED cannot be specified without WITH.
- With PAIRED, the number of variables that are specified before and after WITH must be the same. PAIRED must be specified in parentheses after the second variable list.

Operations

- For the purposes of computing the test statistics, only combinations for which the values for the two variables are different are considered.

- If fewer than 25 cases change values from the first variable to the second variable, the binomial distribution is used to compute the probability.

Example

```
NPAR TESTS MCNEMAR=V1 V2 V3.
```

- This example performs the MCNEMAR test on variable pairs *V1* and *V2*, *V1* and *V3*, and *V2* and *V3*.

MEDIAN Subcommand

```
NPAR TESTS MEDIAN [(value)]=varlist BY variable(value1,value2)
```

MEDIAN determines whether *k* independent samples are drawn from populations with the same median. The independent samples are defined by a grouping variable. For each variable, the output shows a table of the number of cases that are greater than and less than or equal to the median in each category in the Frequency table. The output also shows the number of valid cases, the median, chi-square, degrees of freedom, and probability in the Test Statistics table.

Syntax

- The minimum specification is a single test variable, the keyword BY, a grouping variable, and two values in parentheses.
- If the first grouping value is less than the second value, every value in the range that is defined by the pair of values forms a group, and a *k*-sample test is performed.
- If the first value is greater than the second value, two groups are formed by using the two values, and a two-sample test is performed.
- By default, the median is calculated from all cases that are included in the test. To override the default, specify a median value in parentheses following the MEDIAN subcommand keyword.

Operations

- A 2 × *k* contingency table is constructed with counts of the number of cases that are greater than the median and less than or equal to the median for the *k* groups.
- Test statistics are calculated for each variable that is specified before BY.
- For more than 30 cases, a chi-square statistic is computed. For 30 or fewer cases, Fisher's exact procedure (two-tailed) is used instead of chi-square.
- For a two-sample test, cases with values other than the two specified values are excluded.

Example

```
NPAR TESTS MEDIAN(8.4)=V1 BY V2(1,2) /MEDIAN=V1 BY V2(1,2)
  /MEDIAN=V1 BY V3(1,4) /MEDIAN=V1 BY V3(4,1).
```

- The first two MEDIAN subcommands test variable *V1* grouped by values 1 and 2 of variable *V2*. The first test specifies a median of 8.4, and the second test uses the observed median.
- The third MEDIAN subcommand requests a four-samples test, dividing the sample into four groups based on values 1, 2, 3, and 4 of variable *V3*.
- The last MEDIAN subcommand requests a two-samples test, grouping cases based on values 1 and 4 of *V3* and ignoring all other cases.

MH Subcommand

```
NPAR TESTS /MH=varlist [WITH varlist [(PAIRED)]]
```

MH performs the marginal homogeneity test, which tests whether combinations of values between two paired ordinal variables are equally likely. The marginal homogeneity test is typically used in repeated

measures situations. This test is an extension of the McNemar test from binary response to multinomial response. The output shows the number of distinct values for all test variables; the number of valid off-diagonal cell counts; mean; standard deviation; observed and standardized values of the test statistics; the asymptotic two-tailed probability for each pair of variables; and, if a /METHOD subcommand is specified, one-tailed and two-tailed exact or Monte Carlo probabilities.

This subcommand is available only if SPSS Statistics Premium Edition or the Exact Tests option is installed (available only on Windows operating systems).

Syntax

- The minimum specification is a list of two variables. Variables must be polychotomous and must have more than two values. If the variables contain only two values, the McNemar test is performed.
- If keyword WITH is not specified, each variable is paired with every other variable in the list.
- If WITH is specified, each variable before WITH is paired with each variable after WITH. If PAIRED is also specified, the first variable before WITH is paired with the first variable after WITH, the second variable before WITH is paired with the second variable after WITH, and so on. PAIRED cannot be specified without WITH.
- With PAIRED, the number of variables that are specified before and after WITH must be the same. PAIRED must be specified in parentheses after the second variable list.

Operations

- The data consist of paired, dependent responses from two populations. The marginal homogeneity test tests the equality of two multinomial *c* × 1 tables, and the data can be arranged in the form of a square *c* × *c* contingency table. A 2 × *c* table is constructed for each off-diagonal cell count. The marginal homogeneity test statistic is computed for cases with different values for the two variables. Only combinations for which the values for the two variables are different are considered. The first row of each 2 × *c* table specifies the category that was chosen by population 1, and the second row specifies the category that was chosen by population 2. The test statistic is calculated by summing the first row scores across all 2 x *c* tables.

Example

```
NPAR TESTS /MH=V1 V2 V3
  /METHOD=MC.
```

- This example performs the marginal homogeneity test on variable pairs *V1* and *V2*, *V1* and *V3*, and *V2* and *V3*. The exact *p* values are estimated by using the Monte Carlo sampling method.

MOSES Subcommand

```
NPAR TESTS MOSES[(n)]=varlist BY variable(value1,value2)
```

The MOSES test of extreme reactions tests whether the range of an ordinal variable is the same in a control group and a comparison group. The control and comparison groups are defined by a grouping variable. The output includes a Frequency table, showing, for each variable before *BY*, the total number of cases and the number of cases in each group. The output also includes a Test Statistics table, showing the number of removed outliers, span of the control group before and after outliers are removed, and one-tailed probability of the span with and without outliers.

Syntax

- The minimum specification is a test variable, the keyword BY, a grouping variable, and two values in parentheses.
- The test variable must be at least at the ordinal level of measurement.
- The first value of the grouping variable defines the control group, and the second value defines the comparison group.

- By default, 5% of the cases are trimmed from each end of the range of the control group to remove outliers. You can override the default by specifying a value in parentheses following the MOSES subcommand keyword. This value represents an actual number of cases, not a percentage.

Operations

- Values from the groups are arranged in a single ascending sequence. The span of the control group is computed as the number of cases in the sequence containing the lowest and highest control values.
- No adjustments are made for tied cases.
- Cases with values other than values that are specified for the grouping variable are excluded.
- Test statistics are calculated for each variable that is named before BY.

Example

```
NPAR TESTS MOSES=V1 BY V3(0,1) /MOSES=V1 BY V3(1,0).
```

- The first MOSES subcommand tests *V1* by using value 0 of *V3* to define the control group and value 1 for the comparison group. The second MOSES subcommand reverses the comparison and control groups.

RUNS Subcommand

```
NPAR TESTS RUNS({MEAN  })=varlist
               {MEDIAN}
               {MODE  }
               {value }
```

RUNS tests whether the sequence of values of a dichotomized variable is random. The output includes a Run Test table, showing the test value (cut point that is used to dichotomize the variable tested), number of runs, number of cases that are below the cut point, number of cases that are greater than or equal to the cut point, and test statistic *Z* with its two-tailed probability for each variable.

Syntax

- The minimum specification is a cut point in parentheses followed by a test variable.
- The cut point can be specified by an exact value or one of the keywords MEAN, MEDIAN, or MODE.

Operations

- All tested variables are treated as dichotomous: cases with values that are less than the cut point form one category, and cases with values that are greater than or equal to the cut point form the other category.
- Test statistics are calculated for each specified variable.

Example

```
NPAR TESTS RUNS(MEDIAN)=V2 /RUNS(24.5)=V2 /RUNS(1)=V3.
```

- This example performs three runs tests. The first test tests variable *V2* by using the median as the cut point. The second test also tests *V2* by using 24.5 as the cut point. The third test tests variable *V3*, with value 1 specified as the cut point.

SIGN Subcommand

```
NPAR TESTS SIGN=varlist [WITH varlist [(PAIRED)] ]
```

SIGN tests whether the distribution of two paired variables in a two-related-samples test is the same. The output includes a Frequency table, showing, for each pair, the number of positive differences, number of negative differences, number of ties, and the total number. The output also includes a Test Statistics table, showing the *Z* statistic and two-tailed probability.

Syntax

- The minimum specification is a list of two variables.
- Variables should be at least at the ordinal level of measurement.
- If keyword WITH is not specified, each variable in the list is paired with every other variable in the list.
- If keyword WITH is specified, each variable before WITH is paired with each variable after WITH. If PAIRED is also specified, the first variable before WITH is paired with the first variable after WITH, the second variable before WITH is paired with the second variable after WITH, and so on. PAIRED cannot be specified without WITH.
- With PAIRED, the number of variables that are specified before and after WITH must be the same. PAIRED must be specified in parentheses after the second variable list.

Operations

- The positive and negative differences between the pair of variables are counted. Ties are ignored.
- The probability is taken from the binomial distribution if 25 or fewer differences are observed. Otherwise, the probability comes from the *Z* distribution.
- Under the null hypothesis for large sample sizes, *Z* is approximately normally distributed with a mean of 0 and a variance of 1.

Example

```
NPAR TESTS SIGN=N1,M1 WITH N2,M2 (PAIRED).
```

- *N1* is tested with *N2*, and *M1* is tested with *M2*.

W-W Subcommand

```
NPAR TESTS W-W=varlist BY variable(value1,value2)
```

W-W (alias WALD-WOLFOWITZ) tests whether the distribution of a variable is the same in two independent samples. A runs test is performed with group membership as the criterion. The output includes a Frequency table, showing the total number of valid cases for each variable that is specified before BY and the number of valid cases in each group. The output also includes a Test Statistics table, showing the number of runs, *Z*, and one-tailed probability of *Z*. If ties are present, the minimum and maximum number of possible runs, their *Z* statistics, and one-tailed probabilities are displayed.

Syntax

- The minimum specification is a single test variable, the keyword BY, a grouping variable, and two values in parentheses.
- Cases with the first value form one group, and cases with the second value form the other group. The order in which values are specified is unimportant.

Operations

- Cases are combined from both groups and ranked from lowest to highest, and a runs test is performed, using group membership as the criterion. For ties involving cases from both groups, both the minimum and maximum number of possible runs are calculated. Test statistics are calculated for each variable that is specified before BY.
- For a sample size of 30 or less, the exact one-tailed probability is calculated. For a sample size that is greater than 30, the normal approximation is used.
- Cases with values other than values that are specified for the grouping variable are excluded.

Example

```
NPAR TESTS W-W=V1 BY V3(0,1).
```

- This example ranks cases from lowest to highest based on their values for *V1*, and a runs test is performed. Cases with value 0 for *V3* form one group, and cases with value 1 form the other group.

WILCOXON Subcommand

```
NPAR TESTS WILCOXON=varlist [WITH varlist [(PAIRED)] ]
```

WILCOXON tests whether the distribution of two paired variables in two related samples is the same. This test takes into account the magnitude of the differences between two paired variables. The output includes a Ranks table, showing, for each pair, the number of valid cases, positive and negative differences, their respective mean and sum of ranks, and the number of ties. The output also includes a Test Statistics table, showing *Z* and probability of *Z*.

Syntax

- The minimum specification is a list of two variables.
- If keyword WITH is not specified, each variable is paired with every other variable in the list.
- If keyword WITH is specified, each variable before WITH is paired with each variable after WITH. If PAIRED is also specified, the first variable before WITH is paired with the first variable after WITH, the second variable before WITH is paired with the second variable after WITH, and so on. PAIRED cannot be specified without WITH.
- With PAIRED, the number of variables that are specified before and after WITH must be the same. PAIRED must be specified in parentheses after the second variable list.

Operations

- The differences between the pair of variables are counted, the absolute differences are ranked, the positive and negative ranks are summed, and the test statistic *Z* is computed from the positive and negative rank sums.
- Under the null hypothesis for large sample sizes, *Z* is approximately normally distributed with a mean of 0 and a variance of 1.

Example

```
NPAR TESTS WILCOXON=A B WITH C D (PAIRED).
```

- This example pairs *A* with *C* and *B* with *D*. If PAIRED were not specified, the example would also pair *A* with *D* and *B* with *C*.

STATISTICS Subcommand

STATISTICS requests summary statistics for variables that are named on the NPAR TESTS command. Summary statistics are displayed in the Descriptive Statistics table before all test output.

- If STATISTICS is specified without keywords, univariate statistics (keyword DESCRIPTIVES) are displayed.

DESCRIPTIVES. *Univariate statistics.* The displayed statistics include the mean, maximum, minimum, standard deviation, and number of valid cases for each variable named on the command.

QUARTILES. *Quartiles and number of cases.* The 25th, 50th, and 75th percentiles are displayed for each variable that is named on the command.

ALL. *All statistics available on* NPAR TESTS.

MISSING Subcommand

MISSING controls the treatment of cases with missing values.

- ANALYSIS and LISTWISE are alternatives. However, each of those commands can be specified with INCLUDE.

ANALYSIS. *Exclude cases with missing values on a test-by-test basis.* Cases with missing values for a variable that is used for a specific test are omitted from that test. On subcommands that specify several tests, each test is evaluated separately. This setting is the default.

LISTWISE. *Exclude cases with missing values listwise.* Cases with missing values for any variable that is named on any subcommand are excluded from all analyses.

INCLUDE. *Include user-missing values.* User-missing values are treated as valid values.

SAMPLE Subcommand

NPAR TESTS must store cases in memory. SAMPLE allows you to select a random sample of cases when there is not enough space on your computer to store all cases. SAMPLE has no additional specifications.

- Because sampling would invalidate a runs test, this option is ignored when the RUNS subcommand is used.

METHOD Subcommand

METHOD displays additional results for each requested statistic. If no METHOD subcommand is specified, the standard asymptotic results are displayed. If fractional weights have been specified, results for all methods will be calculated on the weight rounded to the nearest integer. This subcommand is available only if you have the Exact Tests add-on option installed, which is only available on Windows operating systems.

MC. Displays an unbiased point estimate and confidence interval, based on the Monte Carlo sampling method, for all statistics. Asymptotic results are also displayed. When exact results can be calculated, they will be provided instead of the Monte Carlo results. See Exact Tests for situations under which exact results are provided instead of Monte Carlo results.

CIN(n). Controls the confidence level for the Monte Carlo estimate. CIN is available only when /METHOD=MC is specified. CIN has a default value of 99.0. You can specify a confidence interval between 0.01 and 99.9, inclusive.

SAMPLES. Specifies the number of tables that were sampled from the reference set when calculating the Monte Carlo estimate of the exact *p* value. Larger sample sizes lead to narrower confidence limits but also take longer to calculate. You can specify any integer between 1 and 1,000,000,000 as the sample size. SAMPLES has a default value of 10,000.

EXACT. Computes the exact significance level for all statistics, in addition to the asymptotic results. If both the EXACT and MC keywords are specified, only exact results are provided. Calculating the exact *p* value can be memory-intensive. If you have specified /METHOD=EXACT and find that you have insufficient memory to calculate results, close any other applications that are currently running. You can also enlarge the size of your swap file (see your Windows manual for more information). If you still cannot obtain exact results, specify /METHOD=MC to obtain the Monte Carlo estimate of the exact *p* value. An optional TIMER keyword is available if you choose /METHOD=EXACT.

TIMER(n). Specifies the maximum number of minutes during which the exact analysis for each statistic can run. If the time limit is reached, the test is terminated, no exact results are provided, and the program begins to calculate the next test in the analysis. TIMER is available only when /METHOD=EXACT is specified. You can specify any integer value for TIMER. Specifying a value of 0 for TIMER turns the timer off completely. TIMER has a default value of 5 minutes. If a test exceeds a time limit of 30 minutes, it is recommended that you use the Monte Carlo method, rather than the exact method.

References

Siegel, S., and N. J. Castellan. 1988. *Nonparametric statistics for the behavioral sciences*. New York: McGraw-Hill, Inc..

NUMERIC

```
NUMERIC varlist[(format)] [/varlist...]
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
NUMERIC V1.
```

Overview

NUMERIC declares new numeric variables that can be referred to in the transformation language before they are assigned values. Commands such as COMPUTE, IF, RECODE, and COUNT can be used to assign values to the new numeric variables.

Basic Specification

The basic specification is the name of the new variables. By default, variables are assigned a format of F8.2 (or the format that is specified on the SET command).

Syntax Rules

- A FORTRAN-like format can be specified in parentheses following a variable or variable list. Each specified format applies to all variables in the list. To specify different formats for different groups of variables, separate each format group with a slash.
- Keyword TO can be used to declare multiple numeric variables. The specified format applies to each variable that is named and implied by the TO construction.
- NUMERIC can be used within an input program to predetermine the order of numeric variables in the dictionary of the active dataset. When used for this purpose, NUMERIC must precede DATA LIST in the input program.

Operations

- NUMERIC takes effect as soon as it is encountered in the command sequence. Special attention should be paid to the position of NUMERIC among commands. See the topic “Command Order” on page 36 for more information.
- The specified formats (or the defaults) are used as both print and write formats.
- Permanent or temporary variables are initialized to the system-missing value. Scratch variables are initialized to 0.
- Variables that are named on NUMERIC are added to the working file in the order in which they are specified. The order in which they are used in transformations does not affect their order in the active dataset.

Examples

Declaring Multiple Numeric Variables

```
NUMERIC V1 V2 (F4.0) / V3 (F1.0).
```

- NUMERIC declares variables *V1* and *V2* with format F4.0 and declares variable *V3* with format F1.0.

```
NUMERIC V1 TO V6 (F3.1) / V7 V10 (F6.2).
```

- NUMERIC declares variables *V1*, *V2*, *V3*, *V4*, *V5*, and *V6* with format F3.1 and declares variables *V7* and *V10* with format F6.2.

Specifying Variable Order in the Active Dataset

```
NUMERIC SCALE85 IMPACT85 SCALE86 IMPACT86 SCALE87 IMPACT87
        SCALE88 IMPACT88.
```

- Variables *SCALE85* to *IMPACT88* are added to the active dataset in the order that is specified on NUMERIC. The order in which they are used in transformations does not affect their order in the active dataset.

```
INPUT PROGRAM.
STRING CITY (A24).
NUMERIC POP81 TO POP83 (F9)/ REV81 TO REV83(F10).
DATA LIST FILE=POPDATA RECORDS=3
  /1 POP81 22-30 REV81 31-40
  /2 POP82 22-30 REV82 31-40
  /3 POP83 22-30 REV83 31-40
  /4 CITY 1-24(A).
END INPUT PROGRAM.
```

- STRING and NUMERIC are specified within an input program to predetermine variable order in the active dataset. Though data in the file are in a different order, the working file dictionary uses the order that is specified on STRING and NUMERIC. Thus, *CITY* is the first variable in the dictionary, followed by *POP81, POP82, POP83, REV81, REV82,* and *REV83*.
- Formats are specified for the variables on NUMERIC. Otherwise, the program uses the default numeric format (F8.2) from the NUMERIC command for the dictionary format, even though it uses the format on DATA LIST to read the data. In other words, the dictionary uses the first formats specified, even though DATA LIST may use different formats to read cases.

OLAP CUBES

OLAP CUBES is available in Statistics Base Edition.

```
OLAP CUBES {varlist} BY varlist [BY...]

 [/CELLS= [MEAN**] [COUNT**] [STDDEV**]
          [NPCT**] [SPCT**] [SUM** ]
          [MEDIAN] [GMEDIAN] [SEMEAN] [MIN] [MAX] [RANGE]
          [VARIANCE] [KURT] [SEKURT] [SKEW] [SESKEW] [FIRST]
          [LAST] [NPCT(var)][SPCT(var)] [HARMONIC] [GEOMETRIC]
          [DEFAULT] [ALL] [NONE] ]


 [/CREATE [{'catname'}...] =  {GAC    } (gvarname {(gvarvalue gvarvalue)   }
           {DEFAULT  }        {GPC    }          [{(gvarvalue gvarvalue)...}])]
                              {GAC GPC}

                              --or--

                              {VAC    } {(svarname svarname)}
                              {VPC    } {(svarname svarname)...}
                              {VAC VPC}


 [/TITLE ='string'][FOOTNOTE= 'string']

 [/HIDESMALLCOUNTS [COUNT = {5      }] ]
                            {integer}
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 19.0

- HIDESMALLCOUNTS subcommand introduced.

Example

```
OLAP CUBES sales BY quarter by region.
```

Overview

OLAP CUBES produces summary statistics for continuous, quantitative variables within categories defined by one or more categorical grouping variables.

Basic Specification

The basic specification is the command name, OLAP CUBES, with a summary variable, the keyword BY, and one or more grouping variables.

- The minimum specification is a summary variable, the keyword BY, and a grouping variable.
- By default, OLAP CUBES displays a Case Processing Summary table showing the number and percentage of cases included, excluded, and their total, and a Layered Report showing means, standard deviations, sums, number of cases for each category, percentage of total N, and percentage of total sum.

Syntax Rules

- Both numeric and string variables can be specified. String variables can be short or long. Summary variables must be numeric.
- String specifications for TITLE and FOOTNOTE cannot exceed 255 characters. Values must be enclosed in quotes. When the specification breaks on multiple lines, enclose each line in quotes and separate the specifications for each line by at least one blank. To specify line breaks in titles and footnotes, use the \n specification.

- Each subcommand can be specified only once. Multiple use results in a warning, and the last specification is used.
- When a variable is specified more than once, only the first occurrence is honored. The same variables specified after different BY keywords will result in an error.

Limitations

- Up to 10 BY keywords can be specified.
- See SET TABLERENDER for information on how multiple layer dimensions are rendered in lightweight tables.

Operations

- The data are processed sequentially. It is not necessary to sort the cases before processing. If a BY keyword is used, the output is always sorted.
- A Case Processing Summary table is always generated, showing the number and percentage of the cases included, excluded, and the total.
- For each combination of grouping variables specified after different BY keywords, OLAP CUBES produces a group in the report.

Options

Cell Contents. By default, OLAP CUBES displays means, standard deviations, cell counts, sums, percentage of total N, and percentage of total sum. Optionally, you can request any combination of available statistics.

Group Differences. You can display arithmetic and/or percentage differences between categories of a grouping variable or between different variables with the CREATE subcommand.

Format. You can specify a title and a caption for the report using the TITLE and FOOTNOTE subcommands.

TITLE and FOOTNOTE Subcommands

TITLE and FOOTNOTE provide a title and a caption for the Layered Report.

- TITLE and FOOTNOTE are optional and can be placed anywhere.
- The specification on TITLE or FOOTNOTE is a string within quotes. To specify a multiple-line title or footnote, enclose each line in quotes and separate the specifications for each line by at least one blank.
- To insert line breaks in the displayed title or footnote, use the \n specification.
- The string you specify cannot exceed 255 characters.

CELLS Subcommand

By default, OLAP CUBES displays the means, standard deviations, number of cases, sum, percentage of total cases, and percentage of total sum.

- If CELLS is specified without keywords, OLAP CUBES displays the default statistics.
- If any keywords are specified on CELLS, only the requested information is displayed.

DEFAULT. *Means, standard deviations, cell counts, sum, percentage of total N, and percentage of total sum.* This is the default if CELLS is omitted.

MEAN. *Cell means.*

STDDEV. *Cell standard deviations.*

COUNT. *Cell counts.*

MEDIAN. *Cell median.*

GMEDIAN. *Grouped median.*

SEMEAN. *Standard error of cell mean.*

SUM. *Cell sums.*

MIN. *Cell minimum.*

MAx. *Cell maximum.*

RANGE. *Cell range.*

VARIANCE. *Variances.*

KURT. *Cell kurtosis.*

SEKURT. *Standard error of cell kurtosis.*

SKEW. *Cell skewness.*

SESKEW. *Standard error of cell skewness.*

FIRST. *First value.*

LAST. *Last value.*

SPCT. *Percentage of total sum.*

NPCT. *Percentage of total number of cases.*

SPCT(var). *Percentage of total sum within specified variable.* The specified variable must be one of the grouping variables.

NPCT(var). *Percentage of total number of cases within specified variable.* The specified variable must be one of the grouping variables.

HARMONIC. *Harmonic mean.*

GEOMETRIC. *Geometric mean.*

ALL. *All cell information.*

CREATE Subcommand

CREATE allows you to calculate and display arithmetic and percentage differences between groups or between variables. You can also define labels for these difference categories.

GAC (gvar(cat1 cat2)). *Arithmetic difference (change) in the summary variable(s) statistics between each specified pair of grouping variable categories.* The keyword must be followed by a grouping variable name specified in parentheses, and the variable name must be followed by one or more pairs of grouping category values. Each pair of values must be enclosed in parentheses inside the parentheses that contain the grouping variable name. String values must be enclosed in single or double quotation marks. You can specify multiple pairs of category values, but you can only specify one grouping variable, and the grouping variable must be one of the grouping variables specified at the beginning of the OLAP CUBES command, after the BY keyword. The difference calculated is the summary statistic value for the second category specified minus the summary statistic value for the first category specified: `cat2 – cat1`.

GPC (gvar(cat1 cat2)). *Percentage difference (change) in the summary variable(s) statistics between each specified pair of grouping variable categories.* The keyword must be followed by a grouping variable name enclosed in parentheses, and the variable name must be followed by one or more pairs of grouping category values. Each pair of values must be enclosed in parentheses inside the parentheses that contain the grouping variable name. String values must be enclosed in single or double quotation marks. You can specify multiple pairs of category values, but you can only specify one grouping variable, and the grouping variable must be one of the grouping variables specified at the beginning of the OLAP CUBES command, after the BY keyword. The percentage difference calculated is the summary statistic value for the second

category specified minus the summary statistic value for the first category specified, divided by the summary statistic value for the first category specified: `(cat2 – cat1)/cat1`.

VAC(svar1 svar2). *Arithmetic difference (change) in summary statistics between each pair of specified summary variables.* Each pair of variables must be enclosed in parentheses, and all specified variables must be specified as summary variables at the beginning of the OLAP CUBES command. The difference calculated is the summary statistic value for the second variable specified minus the summary statistic value for the first variable specified: `svar2 – svar1`.

VPC(svar1 svar2). *Percentage difference (change) in summary statistics between each pair of specified summary variables*. Each pair of variables must be enclosed in parentheses, and all specified variables must be specified as summary variables at the beginning of the OLAP CUBES command. The percentage difference calculated is the summary statistic value for the second variable specified minus the summary statistic value for the first variable specified: `(svar2 – svar1)/svar1`.

'category label'. *Optional label for each difference category created*. These labels must be the first specification in the CREATE subcommand. Each label must be enclosed in single or double quotation marks. If no labels are specified, defined value or variable labels are used. If no labels are defined, data values or variable names are displayed. If multiple differences are created, the order of the labels corresponds to the order the differences are specified. To mix custom labels with default labels, use the keyword DEFAULT for the difference categories without custom labels.

Both arithmetic and percentage differences can be specified in the same command, but you cannot specify both grouping variable differences (GAC/GPC) and summary variable differences (VAC/VPC) in the same command.

Example

```
OLAP CUBES
 sales96   BY region
 /CELLS=SUM NPCT
 /CREATE GAC GPC (region (1 3) (2 3)).
```

- Both the arithmetic (GAC) and percentage (GPC) differences will be calculated.
- Differences will be calculated for two different pairs of categories of the grouping variable *region*.
- The grouping variable specified in the CREATE subcommand, *region*, is also specified as a grouping variable at the beginning of the OLAP CUBES command.

Example

```
OLAP CUBES
 sales95 sales96   BY region
 /CELLS=SUM NPCT
 /CREATE VAC VPC (sales95 sales96).
```

- Both the arithmetic (VAC) and percentage (VPC) differences will be calculated.
- The difference calculated will be *sales96 - sales95*.
- The percentage difference calculated will be (*sales96 - sales95*)/*sales95*.
- The two variables, *sales95* and *sales96* are also specified as summary variables at the beginning of the OLAP CUBES command.

Example

```
OLAP CUBES
 sales96   BY region
 /CELLS=SUM NPCT
 /CREATE DEFAULT 'West-East GPC'  DEFAULT 'West-Central % Difference'
 GAC GPC (region (1 3) (2 3)).
```

- Four labels are specified, corresponding to the four difference categories that will be created: arithmetic and percentage differences between regions 3 and 1 and between regions 3 and 2.
- The two DEFAULT labels will display the defined value labels or values if there aren't any value labels for the two arithmetic (GAC) difference categories.

HIDESMALLCOUNTS Subcommand

HIDESMALLCOUNTS allows you to hide counts displayed in tables for count values that are less than a specified integer. Hidden values are displayed as **<N**, where **N** is the specified integer.

COUNT Keyword

The COUNT keyword specifies the cutoff, below which count values are hidden. Values equal to the cutoff are not hidden. The value of COUNT must be an integer and greater than or equal to 2. The default is 5. If weighting is on, the weighted value determines the cutoff.

OMS

Note: Square brackets used in the OMS syntax chart are required parts of the syntax and are not used to indicate optional elements. Any equals signs (=) displayed in the syntax chart are required. All subcommands except DESTINATION are optional.

```
OMS

 /SELECT CHARTS HEADINGS LOGS MODELS
        TABLES TEXTS TREES WARNINGS
 or
 /SELECT ALL EXCEPT = [list]

 /IF  COMMANDS = ["expression" "expression"...]
      SUBTYPES = ["expression" "expression"...]
      LABELS = ["expression" "expression"...]
      INSTANCES = [n n... LAST]

 /EXCEPTIF (same keywords as IF, except for INSTANCES)

 /DESTINATION
  FORMAT=
         {DOC       }
         {HTML      }
         {OXML      }
         {PDF       }
         {TABTEXT   }
         {TEXT      }
         {SAV       }
         {SPV       }
         {XLS       }
         {XLSX      }

 {OUTFILE = "outfile expression"           }
 {XMLWORKSPACE="name"                      }
 {OUTPUTSET = {SUBTYPES} FOLDER = "dirspec"}
              {LABELS  }

  VIEWER={YES|NO}
  NUMBERED = 'varname'
  IMAGES={YES|NO}
  IMAGEFORMAT={PNG|JPG|BMP}
  IMAGEROOT='rootname'
  CHARTSIZE=percent
  IMAGEMAP={NO|YES}
  TREEFORMAT={XML|IMAGE}
  CHARTFORMAT={XML|IMAGE}
  MODELFORMAT={XML|IMAGE}
  TABLES={PIVOTABLE|STATIC}

 /COLUMNS DIMNAMES = ["dimension1" "dimension2" ...]
or
 /COLUMNS SEQUENCE = [R1 R2 ... RALL C1 C2... CALL L1 L2... LALL]

 /TAG = "string"

 /NOWARN
```

- NUMBERED only applies to FORMAT=SAV.
- IMAGES applies only to FORMAT=OXML, HTML, and SPV.
- IMAGEFORMAT applies only to FORMAT=OXML, HTML, and SPV.
- IMAGEROOT and CHARTSIZE only apply to FORMAT=OXML and HTML.
- IMAGEMAP only applies to FORMAT=HTML.
- TREEFORMAT only applies to FORMAT=OXML and SPV.
- CHARTFORMAT only applies to FORMAT=OXML.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 13.0

- TREES keyword introduced on SELECT subcommand.
- IMAGES, IMAGEROOT, CHARTSIZE, and IMAGEFORMAT keywords introduced on DESTINATION subcommand.

Release 14.0

- XMLWORKSPACE keyword introduced on DESTINATION subcommand.

Release 16.0

- IMAGEFORMAT=VML introduced for FORMAT=HTML on DESTINATION subcommand.
- IMAGEMAP keyword introduced for FORMAT=HTML on DESTINATION subcommand.
- FORMAT=SPV introduced for saving output in Viewer format.
- CHARTFORMAT keyword introduced.
- TREEFORMAT keyword introduced.
- TABLES keyword introduced.
- FORMAT=SVWSOXML is no longer supported.

Release 17.0

- MODELS keyword introduced on SELECT subcommand.
- FORMAT=DOC, XLS, PDF, and SPW introduced.
- MODELFORMAT keyword introduced.

Release 19.0

- IMAGEFORMAT=VML introduced for FORMAT=OXML on DESTINATION subcommand.
- For version 19.0.0.1 and higher, the IMAGEMAP keyword will no longer generate image map tooltips for major tick labels.

Release 21.0

- FORMAT=XLSX added to DESTINATION subcommand.

Release 22.0

- FORMAT=REPORTHTML and FORMAT=REPORTMHT added to DESTINATION subcommand.
- REPORTTITLE keyword added to DESTINATION subcommand.

Release 27.0

- FORMAT=REPORTHTML and FORMAT=REPORTMHT deprecated from the DESTINATION subcommand.
- REPORTTITLE keyword deprecated from the DESTINATION subcommand.

Example

```
OMS
 /DESTINATION FORMAT = OXML OUTFILE = '/mydir/myfile.xml'
  VIEWER = NO.
OMS
 /SELECT TABLES
 /IF COMMANDS = ['Regression']
     SUBTYPES = ['Coefficients']
 /DESTINATION FORMAT = SAV
  OUTFILE = '/mydir/regression_coefficients.sav'.
REGRESSION
  /STATISTICS COEFF OUTS R ANOVA
  /DEPENDENT income
  /METHOD=ENTER age address ed employ.
OMSEND.
```

Overview

The OMS command controls the routing and format of output from IBM SPSS Statistics to files and can suppress Viewer output. Output formats include: Word, Excel, PDF, IBM SPSS Statistics data file format (SAV), Viewer file format (SPV), XML, HTML, and text.

Basic Specification

The basic specification is the command name followed by a DESTINATION subcommand that contains a FORMAT and/or a VIEWER specification. For FORMAT, an OUTFILE or OUTPUTSET specification is also required.

Syntax Rules

- All subcommands except DESTINATION are optional. No subcommand may occur more than once in each OMS command.
- Multiple OMS commands are allowed. For more information, see below.
- Subcommands can appear in any order.
- If duplicates are found in a list, they are ignored except in /COLUMNS SEQUENCE where they cause an error.
- When a keyword takes a square-bracketed list, the brackets are required even if the list contains only a single item.

Limitations

- For OXML format, the HEADINGS keyword has no effect. If any output from a procedure is included, the procedure title output is included.
- If the OMS specification results in nothing other than Headings objects or a Notes tables being included for a procedure, then nothing is included for that procedure.

Basic Operation

- Once an OMS command is executed, it remains in effect until the end of the session or until ended by an OMSEND command.
- A destination file specified on an OMS command is unavailable to other commands and other applications until the OMS command is ended by an OMSEND command or the end of the session.
- While an OMS command is in effect, the specified destination files are stored in memory (RAM), so active OMS commands that write a large amount of output to external files may consume a large amount of memory.
- Multiple OMS commands are independent of each other (except as noted below). The same output can be routed to different locations in different formats based on the specifications in different OMS commands.
- Display of output objects in the Viewer is determined by the most recent OMS command that includes the particular output type. For example, if an OMS command includes all tables from the FREQUENCIES command and also contains a VIEWER = YES specification, and a subsequent OMS command includes all tables of the subtype 'Statistics' with VIEWER = NO, Statistics tables for subsequent FREQUENCIES commands will *not* be displayed in the Viewer.
- The COLUMNS subcommand has no effect on pivot tables displayed in the Viewer.
- The order of the output objects in any particular destination is the order in which they were created, which is determined by the order and operation of the commands that generate the output.

SELECT Subcommand

SELECT specifies the types of output objects to be routed to the specified destination(s). You can select multiple types. You can also specify ALL with EXCEPT to exclude specified types. If there is no SELECT subcommand, all supported output types are selected.

ALL. *All output objects.* This is the default.

CHARTS. *Chart objects.* This includes charts created by the commands such as GRAPH and GGRAPH and charts created by statistical procedures (for example, the BARCHART subcommand of the FREQUENCIES command). It does not include tree diagrams produced by the TREE procedure or charts in models displayed in the Model Viewer.

HEADINGS. *Text objects labeled Title in the outline pane of the Viewer*. For destination format OXML, the HEADINGS keyword has no effect. If any output for the procedure is included in OXML output, the procedure title is included as the text attribute value of the command element.

LOGS. *Log text objects.* Log objects contain certain types of error and warning messages. With SET PRINTBACK=ON, log objects also contain the command syntax executed during the session. Log objects are labeled *Log* in the outline pane of the Viewer.

MODELS. *Output objects displayed in the Model Viewer.* A single model object can contain multiple views of the model, including both tables and charts.

TABLES. *Output objects that are pivot tables in the Viewer*. This includes Notes tables. It does not include tables displayed in the Model Viewer. Pivot tables are the only output objects that can be routed to the destination format SAV.

TEXTS. *Text objects that aren't logs or headings.* This includes objects labeled *Text Output* in the outline pane of the Viewer.

TREES. *Tree model diagrams produced by the TREE procedure (SPSS Statistics Professional Edition or the Decision Trees option).*

WARNINGS. *Warnings objects.* Warnings objects contain certain types of error and warning messages.

Example

```
OMS /SELECT TABLES LOGS TEXTS WARNINGS HEADINGS
    /DESTINATION FORMAT = HTML OUTFILE = '/mypath/myfile1.htm'.
OMS /SELECT ALL EXCEPT = [CHARTS]
    /DESTINATION FORMAT = HTML OUTFILE = '/mypath/myfile2.htm'.
```

The two SELECT subcommands are functionally equivalent. The first one explicitly lists all types but CHARTS, and the second one explicitly excludes only CHARTS.

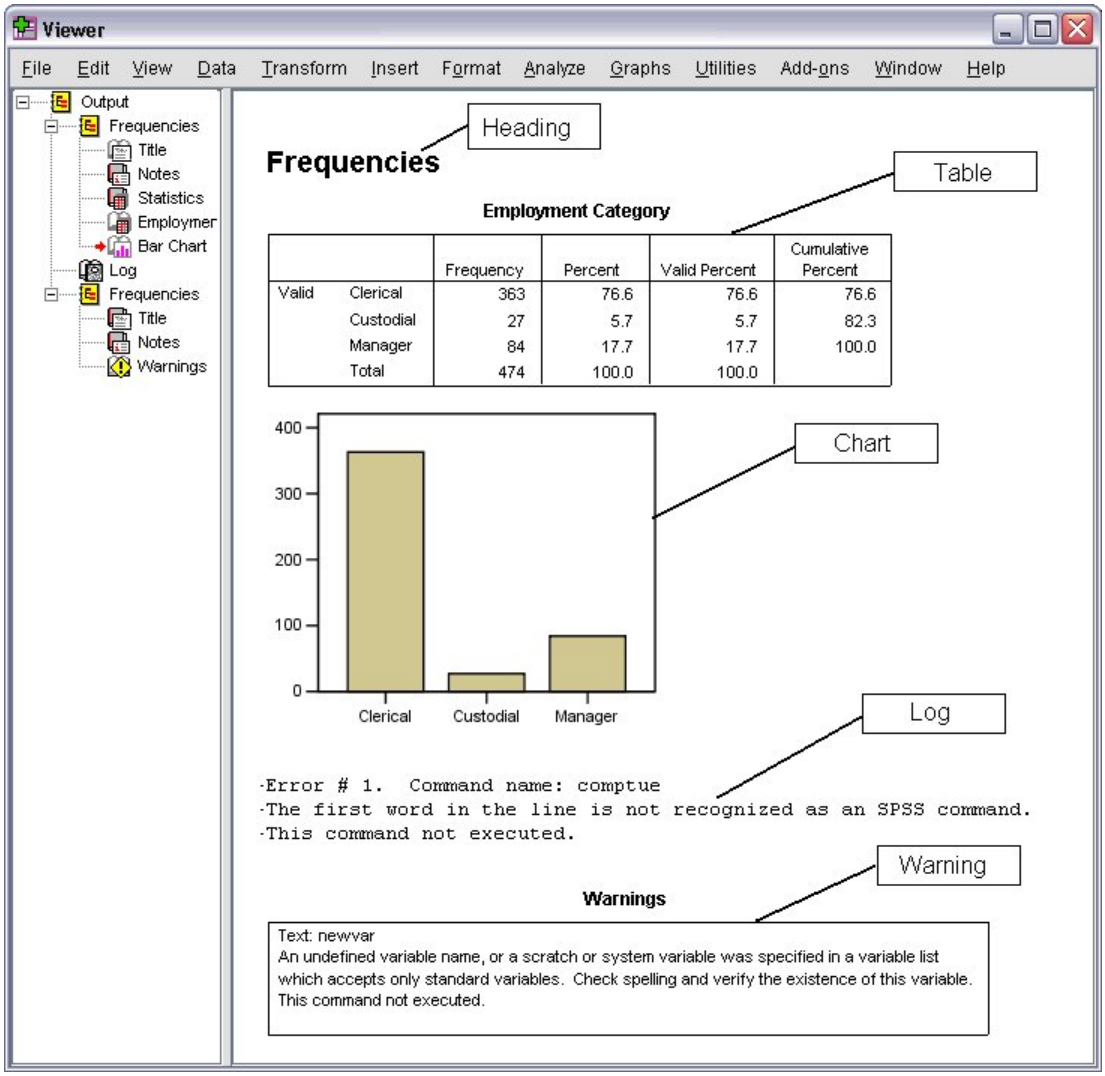


Figure 67. Output object types in the Viewer

Notes Table Limitation

An OMS command that selects only tables will not select a Notes table if the Notes tables is the only table produced by a procedure. This can occur if the command contains syntax errors that result in a Notes table and a warning object, but no other tables. For example:

```
DATA LIST FREE /var1 var2.
BEGIN DATA
1 2
END DATA.
OMS SELECT TABLES
  /DESTINATION FORMAT=HTML
   OUTFILE='/temp/htmltest.htm'.
FREQUENCIES VARIABLES=var1.
DESCRIPTIVES VARIABLES=var02.
OMSEND.
```

The DESCRIPTIVES command refers to a variable that doesn't exist, causing an error that results in the creation of a Notes table and a warning object, but the HTML file will not include this Notes table. To make sure Notes tables are selected when no other tables are created by a procedure, include WARNINGS in the SELECT subcommand, as in:

```
OMS SELECT TABLES WARNINGS
  /DESTINATION FORMAT=HTML
   OUTFILE='/temp/htmltest.htm'.
```

IF Subcommand

The IF subcommand specifies particular output objects of the types determined by SELECT. Without an IF subcommand, all objects of the specified types are selected. If you specify multiple conditions, only those objects that meet *all* conditions will be selected.

Example

```
OMS
 /SELECT TABLES
 /IF COMMANDS = ['Regression']
     SUBTYPES = ['Coefficients']
 /DESTINATION FORMAT = SAV
  OUTFILE = '/mydir/regression_coefficients.sav'.
```

This OMS command specifies that only coefficient tables from the REGRESSION command will be selected.

COMMANDS Keyword

The COMMANDS keyword restricts the selection to the specified command(s). The keyword COMMANDS must be followed by an equals sign (=) and a list of quoted command identifiers enclosed in square bracket, as in:

```
OMS
 /SELECT TABLES
 /IF COMMANDS = ['Frequencies' 'Factor Analysis']
 /DESTINATION...
```

Command identifiers are:

- Unique. No two commands have the same identifier.
- Not case-sensitive.
- Not subject to translation, which means they are the same for all language versions and output languages.
- Often not exactly the same or even similar to the command name. You can obtain the identifier for a particular command from the OMS Control Panel (Utilities menu) or by generating output from the command in the Viewer and then right-clicking the command heading in the outline pane and selecting **Copy OMS Command Identifier** from the context menu.

Command identifiers are available for all statistical and charting procedures and any other commands that produce blocks of output with their own identifiable heading in the outline pane of the Viewer. For example, CASESTOVARS and VARSTOCASES have corresponding identifiers ('Cases to Variables' and 'Variables to Cases') because they produce their own output blocks (with command headings in the outline pane that happen to match the identifiers), but FLIP does not because any output produced by FLIP is included in a generic Log text object.

SUBTYPES Keyword

The SUBTYPES keyword restricts the selection to the specified table types The keyword SUBTYPES must be followed by an equals sign (=) and a list of quoted subtype identifiers enclosed in square bracket, as in:

```
OMS
 /SELECT TABLES
 /IF SUBTYPES = ['Descriptive Statistics' 'Coefficients']
 /DESTINATION...
```

- Subtypes apply only to tables that would be displayed as pivot tables in the Viewer.
- Like command identifiers, subtype identifiers are not case-sensitive and are not subject to translation.
- Unlike command identifiers, subtype identifiers are not necessarily unique. For example, multiple commands produce a table with the subtype identifier "Descriptive Statistics," but not all of those tables share the same structure. If you want only a particular table type for a particular command, use both the COMMANDS and SUBTYPES keywords.

- The OMS Control Panel (Utilities menu) provides a complete list of subtypes. You can also obtain the identifier for a particular table by generating output from the command in the Viewer and then right-clicking outline item for the Table in the outline pane of the Viewer and selecting **Copy OMS Table Subtype** from the context menu. The identifiers are generally fairly descriptive of the particular table type.

LABELS Keyword

The LABELS keyword selects particular output objects according to the text displayed in the outline pane of the Viewer. The keyword LABELS must be followed by an equals sign (=) and a list of quoted label text enclosed in square brackets, as in:

```
OMS
 /SELECT TABLES
 /IF LABELS = ['Job category * Gender Crosstabulation']
 /DESTINATION...
```

The LABELS keyword is useful for differentiating between multiple graphs or multiple tables of the same type in which the outline text reflects some attribute of the particular output object such as the variable names or labels. There are, however, a number of factors that can affect the label text:

- If split file processing is on, split file group identification may be appended to the label.
- Labels that include information about variables or values are affected by the OVARS and ONUMBERS settings on the SET command.
- Labels are affected by the current output language setting (SET OLANG).

INSTANCES Keyword

The INSTANCES subcommand selects the *n*th instance of an object matching the other criteria on the IF subcommand within a single command execution. The keyword INSTANCES must be followed by an equals sign (=) and a list of positive integers and/or the keyword LAST enclosed in square brackets.

Example

```
OMS
 /SELECT TABLES
 /IF COMMANDS = ['Frequencies']
     SUBTYPES = ['Frequencies']
     INSTANCES = [1 LAST]
 /DESTINATION...
OMS
 /SELECT TABLES
 /IF COMMANDS = ['Frequencies']
     INSTANCES = [1 LAST]
 /DESTINATION...
```

- The first OMS command will select the first and last frequency tables from each FREQUENCIES command.
- The second OMS command, in the absence of a SUBTYPES or LABELS specification, will select the first and last tables of *any* kind from the selected command. For the FREQUENCIES command (and most other statistical and charting procedures), the first table would be the Notes table.

Wildcards

For COMMANDS, SUBTYPES, and LABELS, you can use an asterisk (*) as a wildcard indicator at the end of a quoted string to include all commands, tables, and/or charts that start with that quoted string, as in:

```
OMS
 /SELECT TABLES
 /IF SUBTYPES = ['Correlation*']
 /DESTINATION...
```

In this example, all table subtypes that begin with "Correlation" will be selected.

The values of LABELS can contain asterisks as part of the value as in "First variable * Second variable Crosstabulation," but only an asterisk as the last character in the quoted string is interpreted as a wildcard, so:

```
OMS
 /SELECT TABLES
 /IF LABELS = ['First Variable **']
 /DESTINATION...
```

will select all tables with labels that start with "First Variable *".

EXCEPTIF Subcommand

The EXCEPTIF subcommand excludes specified output object types. It has the same keywords and syntax as IF, with the exception of INSTANCES, which will cause an error if used with EXCEPTIF.

Example

```
OMS
 /SELECT TABLES
 /IF COMMANDS = ['Regression']
 /EXCEPTIF SUBTYPES = ['Notes' 'Case Summar*']
 /DESTINATION...
```

DESTINATION Subcommand

The DESTINATION subcommand is the only required subcommand. It specifies the format and location for the routed output. You can also use this subcommand to control what output is displayed in the Viewer.

- Output continues to flow to a specified destination until its OMS specification is ended, at which point the file is closed. See the topic “Basic Operation ” on page 1303 for more information.
- Different OMS commands may refer to the same destination file as long as the FORMAT is the same. When a request becomes active, it starts contributing to the appropriate output stream. If the FORMAT differs, an error results. When multiple requests target the same destination, the output is written in the order in which it is created, not the order of OMS commands.

Example

```
OMS
 /DESTINATION FORMAT = OXML OUTFILE = '/mydir/myfile.xml'.
```

FORMAT Keyword

The DESTINATION subcommand must include either a FORMAT or VIEWER specification (or both). The FORMAT keyword specifies the format for the routed output. The keyword must be followed by an equals sign (=) and one of the following alternatives:

DOC. *Word/RTF format.* Pivot tables are exported as Word tables with all formatting attributes intact (for example, cell borders, font styles, and background colors). Text output is exported as formatted RTF. Charts, tree diagrams, and model views are included in PNG format.

HTML. *HTML 5.0.* Output objects that would be pivot tables in the Viewer are converted to HTML tables. Text output objects are tagged <PRE> in the HTML. Charts, tree diagrams, and model views are embedded in the document in the selected format.

OXML. *Output XML.* XML that conforms to the spss-output schema (*xml.spss.com/spss/oms*). See the topic “OXML Table Structure ” on page 1323 for more information. You can specify the version of the spss-output schema that generates the XML output and you can specify whether language invariant versions of `text` attributes are included in the generated XML. These options are specified with the XVERSION and OATTRS subcommands of the SET command. See the topic “OATTRS and XVERSION Subcommands ” on page 1800 for more information.

PDF. *Portable Document Format.* Output is exported as it would appear in Print Preview, with all formatting attributes intact. The PDF file includes bookmarks that correspond to the entries in the Viewer outline pane.

SAV. *IBM SPSS Statistics format data file.* This format is a binary file format. All output object types other than tables are excluded. Each column of a table becomes a variable in the data file. To use a data file that is created with Export Rule Settings in the same session, you must end the active Export Rule Settings request before you can open the data file.See the topic "Routing Output to SAV Files " on page 1316 for more information. In Unicode mode, the file encoding is UTF-8; in code page mode, the file encoding is the code page determined by the current locale. See SET command, UNICODE subcommand for more information.

SPV. *Viewer file format.* This is the same format used when you save the contents of a Viewer window.

TEXT. *Space-separated text.* Output is written as text, with tabular output aligned with spaces for fixed-pitch fonts. Charts, tree diagrams, and model views are excluded. In Unicode mode, the file encoding is UTF-8; in code page mode, the file encoding is the code page determined by the current locale. See SET command, UNICODE subcommand for more information.

TABTEXT. *Tab-delimited text*. For output that is displayed as pivot tables in the Viewer, tabs delimit table column elements. Text block lines are written as is; no attempt is made to divide them with tabs at useful places. Charts, tree diagrams, and model views are excluded. In Unicode mode, the file encoding is UTF-8; in code page mode, the file encoding is the code page determined by the current locale. See SET command, UNICODE subcommand for more information.

XLS and XLSX. *Excel 97-2004 and Excel 2007 and higher formats.* Pivot table rows, columns, and cells are exported as Excel rows, columns, and cells, with all formatting attributes intact -- for example, cell borders, font styles, and background colors. Text output is exported with all font attributes intact. Each line in the text output is a row in the Excel file, with the entire contents of the line contained in a single cell. Charts, tree diagrams, and model views are included in PNG format.

NUMBERED Keyword

For FORMAT=SAV, you can also specify the NUMBERED keyword to identify the source tables, which can be useful if the data file is constructed from multiple tables. This creates an additional variable in the data file. The value of the variable is a positive integer that indicates the sequential table number. The default variable name is *TableNumber_*. You can override the default with an equals sign (=) followed by a valid variable name in quotes after the NUMBERED keyword.

Example

```
OMS
 /SELECT TABLES
 /IF COMMANDS = ['Regression'] SUBTYPES = ['Coefficients']
 /DESTINATION = SAV NUMBERED = 'Table_number'
  OUTFILE = 'data.sav'.
```

IMAGES and IMAGEFORMAT Keywords

For HTML and OXML document format, you can include charts, tree diagrams, and model views in a number of different graphic formats with the IMAGES and IMAGEFORMAT keywords. For the SPV format, these settings only apply to tree diagrams, not charts or model views.

IMAGES Keyword

- IMAGES=YES produces a separate image file for each chart and/or tree diagram. Image files are saved in a separate subdirectory (folder). The subdirectory name is the name of the destination file without any extension and with "_files" appended to the end. This is the default setting.

  Note: When the destination is the XML workspace (XMLWORKSPACE keyword), images are stored in the workspace rather than in a physical file. See the topic "XMLWORKSPACE Keyword" on page 1312 for more information.
- For HTML document format, standard <IMG SRC='filename'> tags are included in the HTML document for each image file.

- For charts in OXML document format, the XML file contains `chart` elements with an `ImageFile` attribute of the general form `<chart imageFile="filepath/filename"/>` for each image file. `IMAGES=YES` has no effect on `FORMAT=OXML` unless you also specify `CHARTFORMAT=IMAGE`. See the topic "CHARTFORMAT Keyword " on page 1311 for more information.
- For model views in OXML document format, the XML file contains `modelview` elements with an `ImageFile` attribute of the general form `<modelview imageFile="filepath/filename"/>` for each image file. `IMAGES=YES` has no effect on `FORMAT=OXML` unless you also specify `MODELFORMAT=IMAGE`. See the topic "MODELFORMAT Keyword " on page 1311 for more information.
- For HTML format, `IMAGES=NO` excludes charts, tree diagrams, and modelviews.
- For OXML format, `IMAGES=NO` causes charts, tree diagrams, and model views to be included in the document in XML format instead of as separate image files.

IMAGEFORMAT Keyword

- PNG is the default image format.
- For HTML document format, the available image formats are PNG, JPG, and BMP.
- For OXML document format, the available image formats are PNG, JPG, and BMP.
- For the SPV document format, the available image formats are PNG, JPG, and BMP.

Example

```
OMS SELECT TABLES CHARTS
  /DESTINATION FORMAT=HTML
               IMAGES=YES
               IMAGEFORMAT=JPG
               OUTFILE='/htmloutput/julydata.htm'.
```

CHARTSIZE and IMAGEROOT Keywords

For HTML and OXML document formats, you can control the relative size and root name of chart and tree diagram images, if charts and/or tree diagrams are saved as separate image files.

- **CHARTSIZE=n.** Specifies the scaling, expressed as a percentage value between 10 and 200. The default is `CHARTSIZE=100`.
- **IMAGEROOT='rootname'.** User-specified rootname for image files. Image files are constructed from the rootname, an underscore, and a sequential three-digit number. The rootname should be specified in quotes, as in: `IMAGEROOT='julydata'`. When the destination is the XML workspace (`XMLWORKSPACE` keyword), the rootname applies to the name associated with the image in the workspace.

Example

```
OMS SELECT TABLES CHARTS
  /DESTINATION FORMAT=HTML
   IMAGES=YES
   CHARTSIZE=50
   IMAGEROOT='rutabaga'
   OUTFILE='/htmloutput/julydata.htm'.
```

IMAGEMAP Keyword

For HTML document format, you can use the `IMAGEMAP` keyword to create image map tooltips that display information for some chart elements, such as the value of the selected point on a line chart or bar on a bar chart. The default is `IMAGEMAP=NO`. To include image map tooltips, use `IMAGEMAP=YES`.

`IMAGEMAP` has no effect on tree diagram images or document formats other than HTML.

Example

```
OMS SELECT TABLES CHARTS
  /DESTINATION FORMAT=HTML
               IMAGES=YES
               IMAGEFORMAT=PNG
               IMAGEMAP=YES
               OUTFILE='/htmloutput/julydata.htm'.
```

TREEFORMAT Keyword

For OXML and SPV document formats, TREEFORMAT controls the format of tree diagrams (produced by the TREE command). The keyword is followed by and equals sign and one of the following alternatives:

- **XML.** Tree diagrams are included as XML that conforms to the pmml schema (*www.dmg.org*). For SPV format, this is the format required to activate and edit trees in the Viewer window. This is the default.
- **IMAGE.** For SPV format, tree diagrams are included in the Viewer document as static images in the selected format. For OXML format, image files are saved in a separate folder. See the topic "IMAGES and IMAGEFORMAT Keywords " on page 1309 for more information.

Note: When the destination is the XML workspace (XMLWORKSPACE keyword), images are stored in the workspace rather than in a physical file. See the topic "XMLWORKSPACE Keyword" on page 1312 for more information.

IMAGES=NO overrides the TREEFORMAT setting and includes tree diagrams in XML format. See the topic "IMAGES and IMAGEFORMAT Keywords " on page 1309 for more information.

Example

```
OMS SELECT TABLES TREES
  /DESTINATION FORMAT=SPV
               IMAGES=YES
               IMAGEFORMAT=PNG
               TREEFORMAT=IMAGE
               OUTFILE='/viewerdocs/results.spv'.
```

CHARTFORMAT Keyword

For OXML document format, CHARTFORMAT controls the format of charts. The keyword is followed by and equals sign and one of the following alternatives:

- **XML.** Charts are included as XML that conforms to the vizml schema (*xml.spss.com/spss/visualization*). This is the default.
- **IMAGE.** For SPV format, charts are included in the Viewer document as static images in the selected format. Image files are saved in the selected format in a separate folder. See the topic "IMAGES and IMAGEFORMAT Keywords " on page 1309 for more information.

Note: When the destination is the XML workspace (XMLWORKSPACE keyword), images are stored in the workspace rather than in a physical file. See the topic "XMLWORKSPACE Keyword" on page 1312 for more information.

IMAGES=NO overrides the CHARTFORMAT setting and includes charts in XML format. See the topic "IMAGES and IMAGEFORMAT Keywords " on page 1309 for more information.

MODELFORMAT Keyword

For OXML document format, MODELFORMAT controls the format of models. The keyword is followed by and equals sign and one of the following alternatives:

- **XML.** Models are included as XML that conforms to the PMML schema (*www.dmg.org*), plus additional XML that specifies data and styles. This is the default.
- **IMAGE.** The OXML contains modelview elements that specify the image file names. The image files are saved in the selected format in a separate folder. One model may produce multiple image files.See the topic "IMAGES and IMAGEFORMAT Keywords " on page 1309 for more information.

Note: When the destination is the XML workspace (XMLWORKSPACE keyword), images are stored in the workspace rather than in a physical file. See the topic "XMLWORKSPACE Keyword" on page 1312 for more information.

IMAGES=NO overrides the MODELFORMAT setting and includes models in XML format. See the topic "IMAGES and IMAGEFORMAT Keywords " on page 1309 for more information.

XML output is not available for models created by the LINEAR and GENLINMIXED commands.

TABLES Keyword

The TABLES keyword controls the format of pivot tables. The keyword is followed by an equals sign and one of the following alternatives:

- **PIVOTABLE.** Tables are included as dynamic pivot tables. This is the default.
- **STATIC.** Tables cannot be pivoted.

OUTFILE Keyword

If a FORMAT is specified, the DESTINATION subcommand must also include either an OUTFILE, XMLWORKSPACE, or OUTPUTSET specification. OUTFILE specifies an output file. The keyword must be followed by an equals sign (=) and a file specification in quotes or a previously defined file handle defined with the FILE HANDLE command. With FORMAT=SAV, you can specify a previously defined dataset name instead of a file.

Example

```
OMS
 /DESTINATION FORMAT = OXML OUTFILE = '/mydir/myfile.xml'.
```

XMLWORKSPACE Keyword

For FORMAT=OXML, you can route the output to an area in memory referred to as the **XML workspace** where it can be used in flow control and other programming features available with BEGIN PROGRAM-END PROGRAM. The keyword must be followed by an equals sign (=) and a quoted string specifying a name that identifies this output in the workspace.

Example

```
OMS SELECT TABLES
  /IF COMMANDs=['Frequencies'] SUBTYPES=['Frequencies']
 /DESTINATION FORMAT=OXML XMLWORKSPACE='freq_table'.
```

For more information, see the documentation for the EvaluateXPath function in the Help system.

OUTPUTSET Keyword

OUTPUTSET is an alternative to OUTFILE that allows you to route each output object to a separate file. The keyword must be followed by an equals sign (=) and one of the following alternatives:

LABELS. *Output file names based on output object label text.* Label text is the text that appears in the outline pane of the Viewer. See the topic “LABELS Keyword ” on page 1307 for more information.

SUBTYPES. *Output file names based on subtype identifiers.* Subtypes apply only to tables. See the topic “SUBTYPES Keyword ” on page 1306 for more information.

Example

```
OMS
 /SELECT TABLES
 /DESTINATION FORMAT = OXML OUTPUTSET = SUBTYPES.
```

- OUTPUTSET will not overwrite existing files. If a specified file name already exists, an underscore and a sequential integer will be appended to the file name.

FOLDER Keyword

With *OUTPUTSET*, you can also use the FOLDER keyword to specify the location for the routed output. Since you may not know what is considered to be the "current" directory, it's probably a good idea to explicitly specify the location. The keyword must be followed by an equals sign (=) and a valid location specification in quotes.

Example

```
OMS
 /SELECT TABLES
 /IF SUBTYPES = ['Frequencies' 'Descriptive Statistics']
 /DESTINATION FORMAT = OXML OUTPUTSET = SUBTYPES
  FOLDER = '/maindir/nextdir/newdir'.
```

- If the last folder (directory) specified on the path does not exist, it will be created.
- If any folders prior to the last folder on the path do not already exist, the specification is invalid, resulting in an error.

VIEWER Keyword

By default, output is displayed in the Viewer as well as being routed to other formats and destinations specified with the FORMAT keyword. You can use VIEWER = NO to suppress the Viewer display of output for the specified output types. The VIEWER keyword can be used without the FORMAT keyword (and associated OUTFILE or OUPUTSET keywords) to simply control what output is displayed in the Viewer.

Example

```
OMS
 /SELECT TABLES
 /IF SUBTYPES = ['Correlations*']
 /DESTINATION FORMAT SAV OUTFILE = '/mydir/myfile.sav'
  VIEWER = NO.
OMS
 /SELECT TABLES
 /IF SUBTYPES = ['NOTES']
 /DESTINATION VIEWER = NO.
```

- The first OMS command will route tables with subtype names that start with "Correlation" to data file in IBM SPSS Statistics format and will not display those tables in the Viewer. All other output will be displayed in the Viewer
- The second OMS command simply suppresses the Viewer display of all Notes tables, without routing the Notes table output anywhere else.

Note: The VIEWER keyword has no effect on OMS output saved to external formats or files, including the Viewer SPV format. It also has no effect on output saved to SPV format in a batch job executed with the Batch Facility (available with IBM SPSS Statistics Server).

COLUMNS Subcommand

You can use the COLUMNS subcommand to specify the dimension elements that should appear in the columns. All other dimension elements appear in the rows.

- This subcommand applies only to tables that would be displayed as pivot tables in the Viewer and is ignored without warning if the OMS command does not include any tables.
- With DESTINATION FORMAT = SAV, columns become variables in the data file. If you specify multiple dimension elements on the COLUMNS subcommand, then variable names will be constructed by combining nested element and column labels. See the topic “Routing Output to SAV Files ” on page 1316 for more information.
- The COLUMNS subcommand has no effect on pivot tables displayed in the Viewer.
- If you specify multiple dimension elements, they are nested in the columns in the order in which they are listed on the COLUMNS subcommand. For example: COLUMNS DIMNAMES=['Variables' 'Statistics'] will nest statistics within variables in the columns.
- If a table doesn't contain any of the dimension elements listed, then all dimension elements for that table will appear in the rows.

DIMNAMES Keyword

The COLUMNS subcommand must be followed by either the DIMNAMES or SEQUENCE keyword. Each dimension of a table may contain zero or more elements. For example, a simple two-dimensional

crosstabulation contains a single row dimension element and a single column dimension element, each with labels based on the variables in those dimensions, plus a single layer dimension element labeled *Statistics* (if English is the output language). These element labels may vary based on the output language (SET OLANG) and/or settings that affect the display of variable names and/or labels in tables (SET TVARS). The keyword DIMNAMES must be followed by an equals sign (=) and a list of quoted dimension element labels enclosed in square brackets.

Example

```
OMS
 /SELECT TABLES
 /IF COMMANDS = ['Correlations' 'Frequencies']
 /DESTINATION FORMAT = SAV OUTPUTSET = SUBTYPES
 /COLUMNS DIMNAMES = ['Statistics'].
```

The labels associated with the dimension elements may not always be obvious. To see all the dimension elements and their labels for a particular pivot table:

1. Activate (double-click) the table in the Viewer.
2. From the menus choose **View > Show All**.

   and/or
3. If the pivoting trays aren't displayed, from the menus choose **Pivot > Pivoting Trays**.
4. Hover over each icon in the pivoting trays for a ToolTip pop-up that displays the label.

Table without Dimension Element Labels Displayed

		Current Salary	Beginning Salary
Female	Mean	$26,031.92	$13,091.97
	Median	$24,300.00	$12,375.00
Male	Mean	$41,441.78	$20,301.40
	Median	$32,850.00	$15,750.00
Total	Mean	$34,419.57	$17,016.09
	Median	$28,875.00	$15,000.00

Table with Dimension Element Labels Displayed

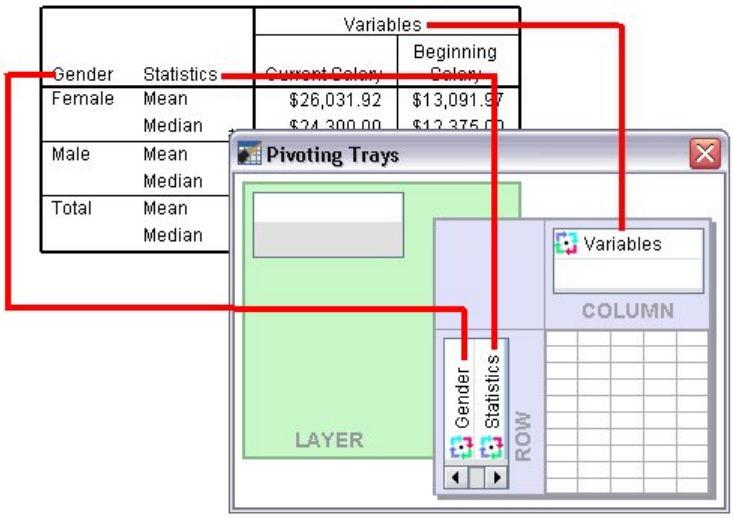


Figure 68. Displaying table dimension element labels

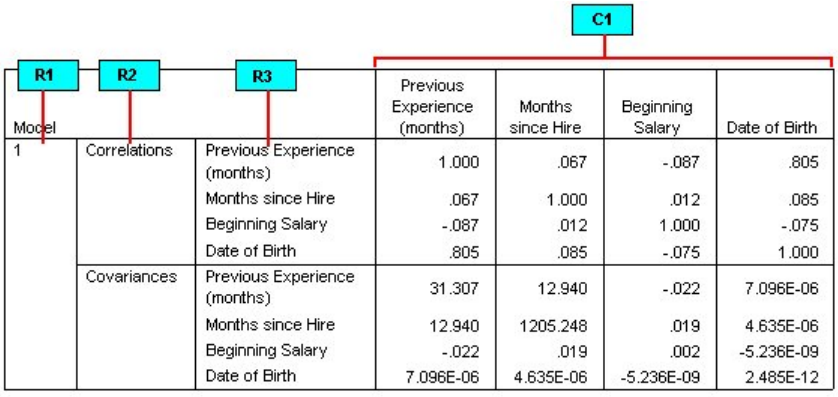
SEQUENCE Keyword

SEQUENCE is an alternative to DIMNAMES that uses positional arguments. These positional arguments do not vary based on output language or output display settings. The SEQUENCE keyword must be followed by an equals sign (=) and a list of positional arguments enclosed in square brackets.

- The general form of a positional argument is a letter indicating the default position of the element--C for column, R for row, or L for layer--followed by a positive integer indicating the default position within that dimension. For example, R1 would indicate the outermost row dimension element.
- A letter indicating the default dimension followed by ALL indicates all elements in that dimension in their default order. For example, RALL would indicate all row dimension elements, and CALL by itself would be unnecessary since it would not alter the default arrangement of the table. ALL cannot be combined with positional sequence numbers in the same dimension.
- SEQUENCE=[CALL RALL LALL] will put all dimension elements in the columns. With FORMAT=SAV, this will result in one case per table in the data file.

Example

```
OMS
 /SELECT TABLES
 /IF COMMANDS = ['Regression'] SUBTYPES = ['Coefficient Correlations']
 /DESTINATION FORMAT = SAV OUTFILE = '/mydir/myfile.sav'
 /COLUMNS SEQUENCE = [R1 R2].
```



Model			Previous Experience (months)	Months since Hire	Beginning Salary	Date of Birth
1	Correlations	Previous Experience (months)	1.000	.067	-.087	.805
		Months since Hire	.067	1.000	.012	.085
		Beginning Salary	-.087	.012	1.000	-.075
		Date of Birth	.805	.085	-.075	1.000
	Covariances	Previous Experience (months)	31.307	12.940	-.022	7.096E-06
		Months since Hire	12.940	1205.248	.019	4.635E-06
		Beginning Salary	-.022	.019	.002	-5.236E-09
		Date of Birth	7.096E-06	4.635E-06	-5.236E-09	2.485E-12

Figure 69. Positional arguments for dimension elements

TAG Subcommand

OMS commands remain in effect until the end of the session or until you explicitly end them with the OMSEND command, and you can have multiple OMS commands in effect at the same time. You can use the TAG subcommand to assign an ID value to each OMS command, which allows you to selectively end particular OMS commands with a corresponding TAG keyword on the OMSEND command. The ID values assigned on the TAG subcommand are also used to identify OMS commands in the log created by the OMSLOG command.

Example

```
OMS
 /DESTINATION FORMAT = OXML OUTFILE = '/mydir/myfile.xml'
 /TAG = 'oxmlout'.
```

- The TAG subcommand must be followed by an equals sign (=) and a quoted ID value.
- The ID value cannot start with a dollar sign.
- Multiple active OMS commands cannot use the same TAG value.

See OMSEND and OMSLOG for more information.

NOWARN Subcommand

The NOWARN subcommand suppresses all warnings from OMS. The NOWARN subcommand applies only to the current OMS command. It has no additional specifications.

Routing Output to SAV Files

IBM SPSS Statistics data files consist of variables in the columns and cases in the rows, and that's essentially how pivot tables are converted to data files:

- Columns in the table are variables in the data file. Valid variable names are constructed from the column labels.
- Row labels in the table become variables with generic variable names (*Var1*, *Var2*, *Var3*...) in the data file. The values of these variables are the row labels in the table.
- Three table-identifier variables are automatically included in the data file: *Command_*, *Subtype_*, and *Label_*. All three are string variables. The first two are the command and subtype identifiers. *Label_* contains the table title text.
- Rows in the table become cases in the data file.

Data File Created from One Table

Data files can be created from one or more tables. There are two basic variations for data files created from a single table:

- Data file created from a two-dimensional table with no layers.
- Data file created from a three-dimension table with one or more layers.

Example

In the simplest case--a single, two-dimensional table--the table columns become variables and the rows become cases in data file.

Report

Gender		Current Salary	Beginning Salary
Female	Mean	$26,031.92	$13,091.97
	Median	$24,300.00	$12,375.00
	Minimum	$15,750	$9,000
	Maximum	$58,125	$30,000
Male	Mean	$41,441.78	$20,301.40
Total			

*temp.sav [DataSet1] - Data Editor

File Edit View Data Transform Analyze Graphs Utilities Add-ons Window Help

Visible: 7 of 7 Variables

	Command_	Subtype_	Label_	Var1	Var2	CurrentSalary	BeginningSal
1	Means	Report	Report	Female	Mean	$26,031.92	$13,091.97
2	Means	Report	Report	Female	Median	$24,300.00	$12,375.00
3	Means	Report	Report	Female	Minimum	$15,750	$9,000
4	Means	Report	Report	Female	Maximum	$58,125	$30,000
5	Means	Report	Report	Male	Mean	$41,441.78	$20,301.40
6	Means	Report	Report	Male	Median	$32,850.00	$15,750.00
7	Means	Report	Report	Male	Minimum	$19,650	$9,000
8	Means	Report	Report	Male	Maximum	$135,000	$79,980
9	Means	Report	Report	Total	Mean	$34,419.57	$17,016.09

Data View Variable View

Figure 70. Single two-dimensional table

- The first three variables identify the source table by command, subtype, and label.
- The two elements that defined the rows in the table--values of the variable *Gender* and statistical measures--are assigned the generic variable names *Var1* and *Var2*. These are both string variables.

- The column labels from the table are used to create valid variable names. In this case, those variable names are based on the variable labels of the three scale variables summarized in the table. If the variables didn't have defined variable labels or you chose to display variable names instead of variable labels as the column labels in the table, then the variable names in the new data file would be the same as in the source data file.

Example

If the default table display places one or more elements in layers, additional variables are created to identify the layer values.

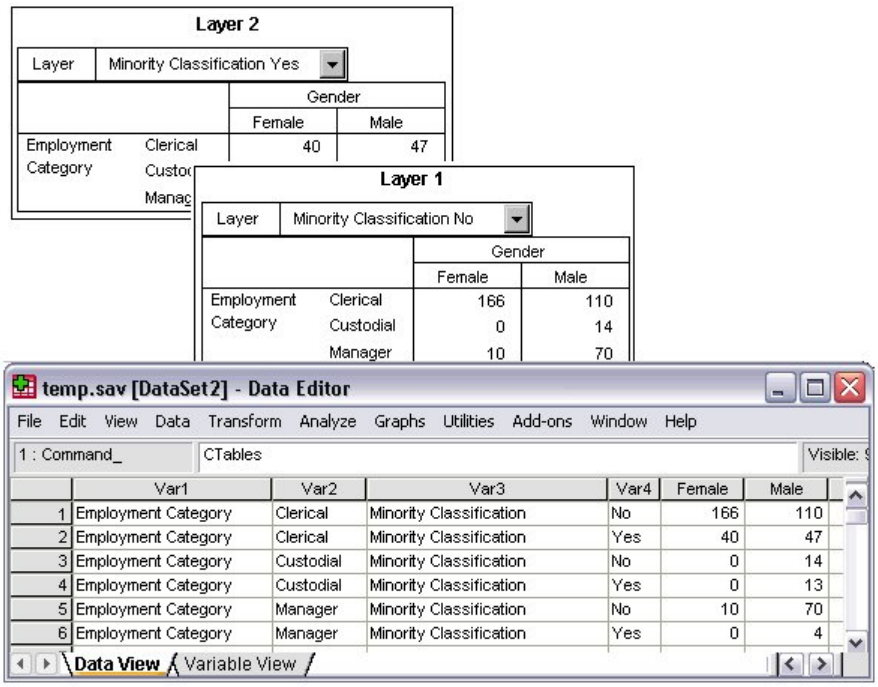


Figure 71. Table with layers

- In the table, the variable labeled *Minority Classification* defines the layers. In the data file, this creates two additional variables: one that identifies the layer element, and one that identifies the categories of the layer element.
- As with the variables created from the row elements, the variables created from the layer elements are string variables with generic variable names (the prefix *Var* followed by a sequential number).

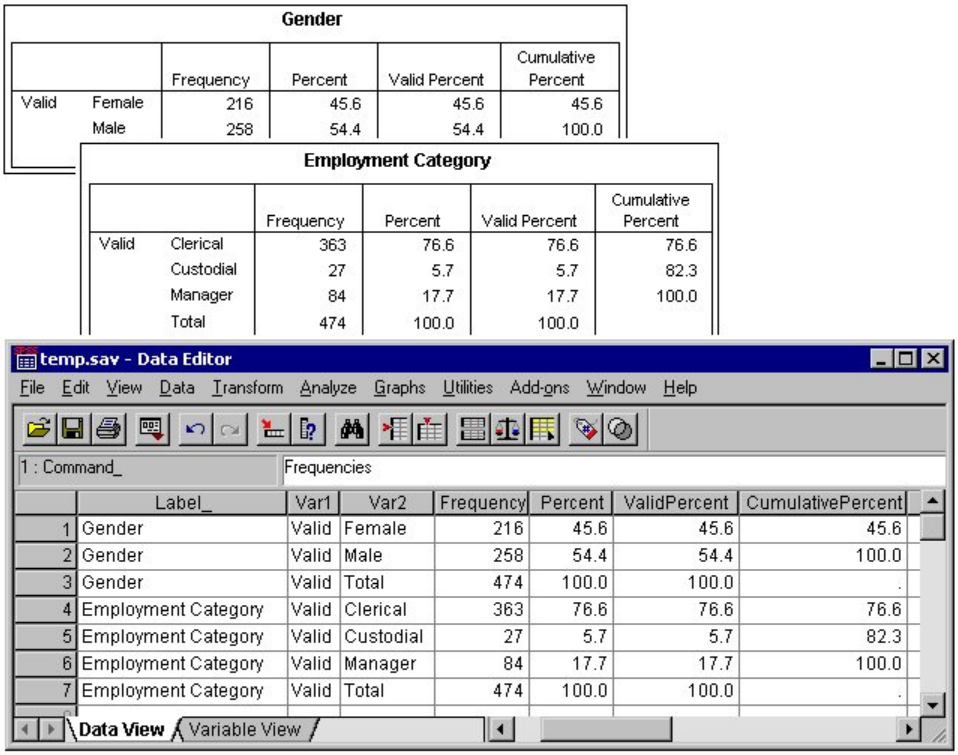
Data Files Created from Multiple Tables

When multiple tables are routed to the same data file, each table is added to the data file in a fashion similar to the `ADD FILES` command.

- Each subsequent table will always add cases to the data file.
- If column labels in the tables differ, each table may also add variables to the data file, with missing values for cases from other tables that don't have an identically labeled column.

Example

Multiple tables that contain the same column labels will typically produce the most immediately useful data files (data files that don't require additional manipulation).



Gender

		Frequency	Percent	Valid Percent	Cumulative Percent
Valid	Female	216	45.6	45.6	45.6
	Male	258	54.4	54.4	100.0

Employment Category

		Frequency	Percent	Valid Percent	Cumulative Percent
Valid	Clerical	363	76.6	76.6	76.6
	Custodial	27	5.7	5.7	82.3
	Manager	84	17.7	17.7	100.0
	Total	474	100.0	100.0	

temp.sav - Data Editor

File Edit View Data Transform Analyze Graphs Utilities Add-ons Window Help

1 : Command_ | Frequencies

	Label_	Var1	Var2	Frequency	Percent	ValidPercent	CumulativePercent
1	Gender	Valid	Female	216	45.6	45.6	45.6
2	Gender	Valid	Male	258	54.4	54.4	100.0
3	Gender	Valid	Total	474	100.0	100.0	.
4	Employment Category	Valid	Clerical	363	76.6	76.6	76.6
5	Employment Category	Valid	Custodial	27	5.7	5.7	82.3
6	Employment Category	Valid	Manager	84	17.7	17.7	100.0
7	Employment Category	Valid	Total	474	100.0	100.0	.

Data View | Variable View

Figure 72. Multiple tables with the same column labels

- The second table contributes additional cases (rows) to the data file but no new variables because the column labels are exactly the same; so there are no large patches of missing data.
- Although the values for *Command_* and *Subtype_* are the same, the *Label_* value identifies the source table for each group of cases because the two frequency tables have different titles.

Example

A new variable is created in the data file for each unique column label in the tables routed to the data file, which will result in blocks of missing values if the tables contain different column labels.

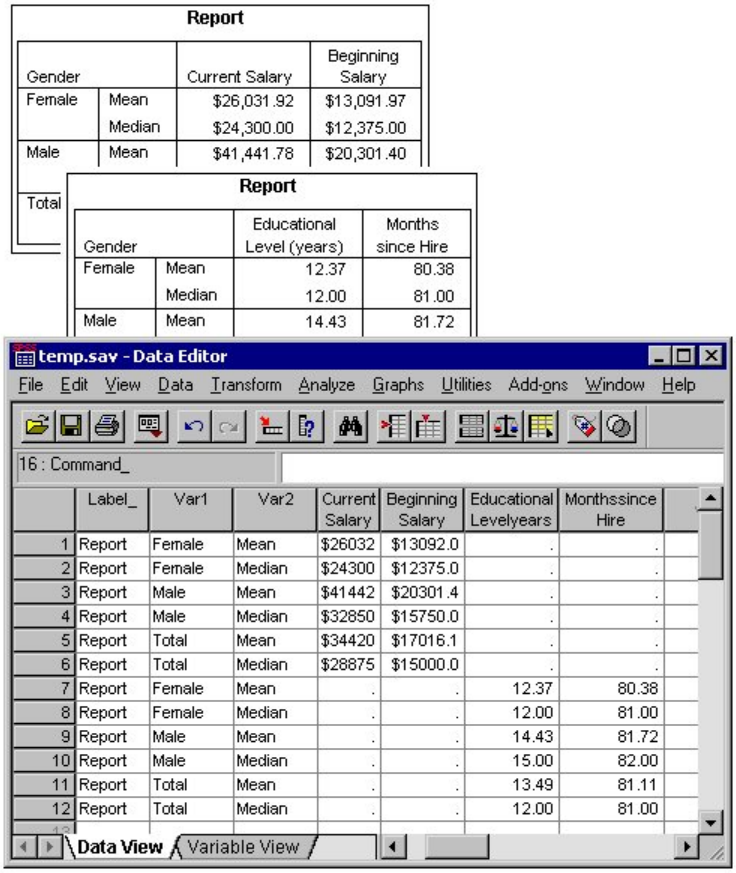


Figure 73. Multiple tables with different column labels

- The first table has columns labeled *Beginning Salary* and *Current Salary*, which are not present in the second table, resulting in missing values for those variables for cases from the second table.
- Conversely, the second table has columns labeled *Education level* and *Months since hire*, which are not present in the first table, resulting in missing values for those variables for cases from the first table.
- Mismatched variables, such as those in this example, can occur even with tables of the same subtype. In fact, in this example, both tables are of the same subtype.

Data Files Not Created from Multiple Tables

If any tables do not have the same number of row elements as the other tables, no data file will be created. The number of rows doesn't have to be the same; the number of row *elements* that become variables in the data file must be the same.

For example, a two-variable crosstabulation and a three-variable crosstabulation from CROSSTABS contain different numbers of row elements, since the "layer" variable is actually nested within the row variable in the default three-variable crosstabulation display.

Employment Category * Gender Crosstabulation

		Gender		
		Female	Male	Total
Employment Category	Clerical	206	157	363
	Custodial	0	27	27
	Manager	10	74	84
Total		216	258	474

Employment Category * Gender * Minority Classification Crosstabulation

Minority Classification			Gender		
			Female	Male	Total
No	Employment Category	Clerical	166	110	276
		Custodial	0	14	14
		Manager	10	70	80
	Total		176	194	370
Yes	Employment Category	Clerical	40	47	87
		Custodial	0	13	13
		Manager	0	4	4
	Total		40	64	104

Figure 74. Tables with different numbers of row elements

In general, the less specific the subtype selection in the OMS command, the less likely you are to get sensible data files, or any data files at all. For example:

```
OMS /SELECT TABLES /DESTINATION FORMAT=SAV OUTFILE='mydata.sav'.
```

will probably fail to create a data file more often than not, since it will select all tables, including Notes tables, which have a table structure that is incompatible with most other table types.

Controlling Column Elements to Control Variables in the Data File

You can use the COLUMNS subcommand to specify which dimension elements should be in the columns and therefore are used to create variables in the generated data file. This is equivalent to pivoting the table in the Viewer.

Example

The DESCRIPTIVES command produces a table of descriptive statistics with variables in the rows and statistics in the columns. A data file created from that table would therefore use the statistics as variables and the original variables as cases. If you want the original variables to be variables in the generated data file and the statistics to be cases:

```
OMS
  /SELECT TABLES
  /IF COMMANDS=['Descriptives'] SUBTYPES=['Descriptive Statistics']
  /DESTINATION FORMAT=SAV OUTFILE='/temp/temp.sav'
  /COLUMNS DIMNAMES=['Variables'].
DESCRIPTIVES VARIABLES=salary salbegin.
OMSEND.
```

- When you use the COLUMNS subcommand, any dimension elements not listed on the subcommand will become rows (cases) in the generated data file.
- Since the descriptive statistics table has only two dimension elements, the syntax COLUMNS DIMNAMES=['Variables'] will put the variables in the columns *and* will put the statistics in the row. So this is equivalent to swapping the positions of the original row and column elements.

Default Descriptive Statistics Table

	N	Minimum	Maximum	Mean	Std. Deviation
Current Salary	474	$15,750	$135,000	$34,419.57	$17,075.661
Beginning Salary	474	$9,000	$79,980	$17,016.09	$7,870.638
Valid N (listwise)	474				

Descriptive Statistics Table with Variables in Columns and Statistics in Rows

	Current Salary	Beginning Salary	Valid N (listwise)
N	474	474	474
Minimum	$15,750	$9,000	
Maximum	$135,000	$79,980	
Mean	$34,419.57	$17,016.09	
Std. Deviation	$17,075.661	$7,870.638	

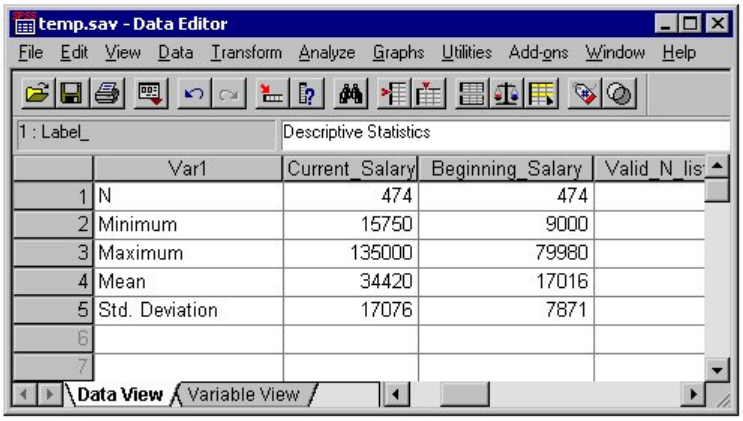


Figure 75. Default and pivoted table and generated data file

Example

The FREQUENCIES command produces a descriptive statistics table with statistics in the rows, while the DESCRIPTIVES command produces a descriptive statistics table with statistics in the columns. To include both table types in the same data file in a meaningful fashion, you need to change the column dimension for one of them.

```
OMS
  /SELECT TABLES
  /IF COMMANDS=['Frequencies' 'Descriptives']
   SUBTYPES=['Statistics' 'Descriptive Statistics']
  /DESTINATION FORMAT=SAV OUTFILE='/temp/temp.sav'
  /COLUMNS DIMNAMES=['Statistics'].
FREQUENCIES
  VARIABLES=salbegin salary
  /FORMAT=NOTABLE
  /STATISTICS=MINIMUM MAXIMUM MEAN.
DESCRIPTIVES
  VARIABLES=jobtime prevexp
  /STATISTICS=MEAN MIN MAX.
OMSEND.
```

- The COLUMNS subcommand will be applied to all selected table types that have a *Statistics* dimension element.
- Both table types have a *Statistics* dimension element, but since it's already in the column dimension for the table produced by the DESCRIPTIVES command, the COLUMNS subcommand has no effect on the structure of the data from that table type.
- For the *FREQUENCIES* statistics table, `COLUMNS DIMNAMES=['Statistics']` is equivalent to pivoting the *Statistics* dimension element into the columns and pivoting the *Variables* dimension element into the rows.
- Some of the variables will have missing values, since the table structures still aren't exactly the same with statistics in the columns.

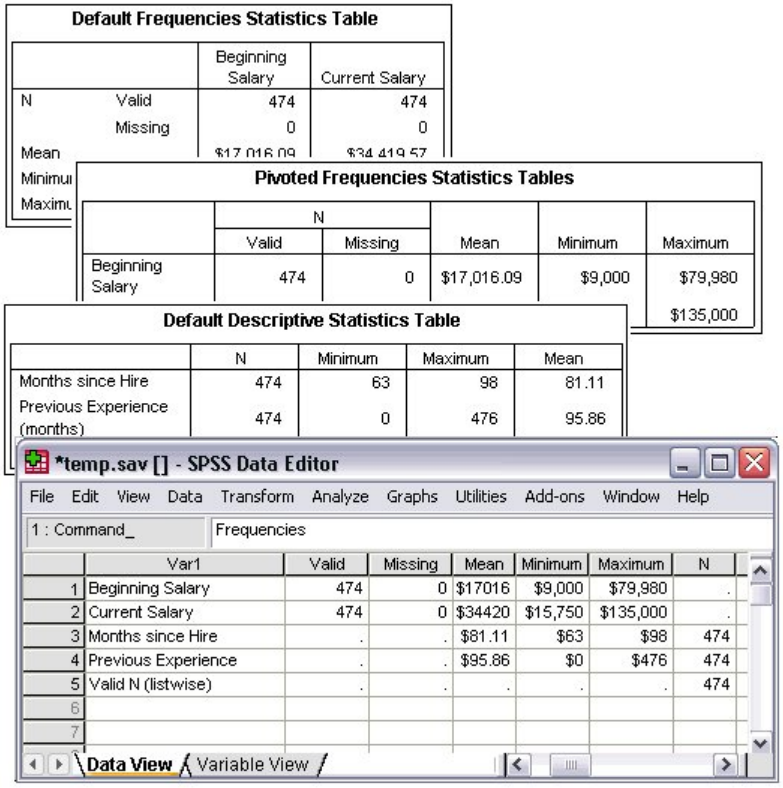


Figure 76. Combining different table types in same data file

Variable Names

OMS constructs valid, unique variable names from column labels.

- Row and layer elements are assigned generic variable names: the prefix `Var` followed by a sequential number.
- Characters that aren't allowed in variable names, such as spaces and parentheses, are removed. For example, "This (Column) Label" would become a variable named *ThisColumnLabel*.
- If the label begins with a character that is allowed in variable names but not allowed as the first character (for example, a number), "@" is inserted as a prefix. For example "2nd" would become a variable named *@2nd*.
- Column labels that would result in duplicate variable names are resolved by appending an underscore and a sequential letter. For example, the second instance of "Count" would become a variable named *Count_A*.
- Underscores or periods at the end of labels are removed from the resulting variable names. (The underscores at the end of the automatically generated variables *Command_*, `Subtype_`, and *Label_* are not removed.)
- If more than one element is in the column dimension, variable names are constructed by combining category labels with underscores between category labels. Group labels are not included. For example, if *VarB* is nested under *VarA* in the columns, you would get variables like *CatA1_CatB1*, not *VarA_CatA1_VarB_CatB1*.

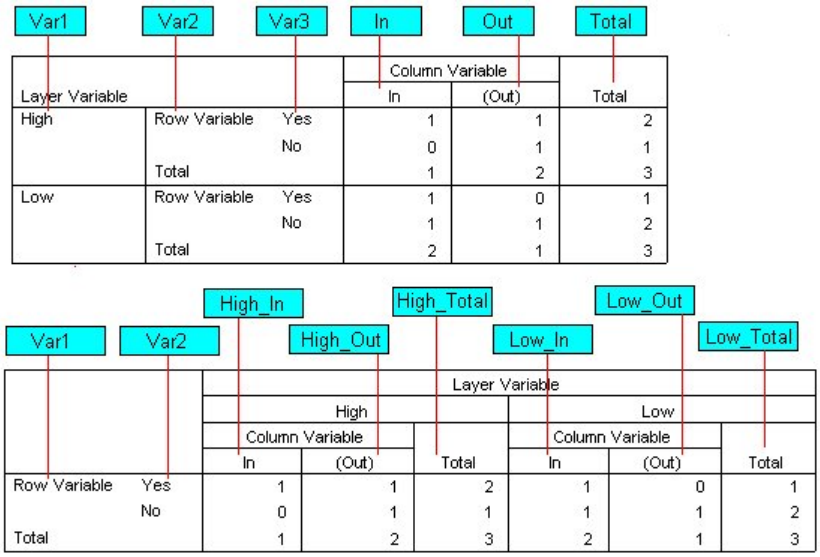


Figure 77. Variable names in SAV files

OXML Table Structure

OXML is XML that conforms to the spss-output schema.

- OMS command and subtype identifiers are used as values of the `command` and `subType` attributes in OXML. For example:

```
<command text="Frequencies" command="Frequencies"...>
  <pivotTable text="Gender" label="Gender" subType="Frequencies"...>
```

  These attribute values are not affected by output language (`SET OLANG`) or display settings for variable names/labels or values/value labels (`SET TVARS` and `SET TNUMBERS`).
- XML is case-sensitive. The element name `pivotTable` is considered a different element from one named "pivottable" or "Pivottable" (the latter two don't exist in OXML).
- Command and subtype identifiers generated by the OMS Control Panel or the OMS Identifiers dialog box (both on the Utilities menu) use the same case as that used for values of the `command` and `subType` OXML attributes.
- All of the information displayed in a table is contained in attribute values in OXML. At the individual cell level, OXML consists of "empty" elements that contain attributes but no "content" other than that contained in attribute values.
- Table structure in OXML is represented row by row; elements that represent columns are nested within the rows, and individual cells are nested within the column elements:

```
<pivotTable...>
 <dimension axis="row"...>
   <dimension axis="column"...>
    <category...>
     <cell text="..." number="..." decimals="..."/>
    </category>
   <category...>
    <cell text="..." number="..." decimals="..."/>
   </category>
  </dimension>
 </dimension>
  ...
</pivotTable>
```

The preceding example is a simplified representation of the structure that shows the descendant/ancestor relationships of these elements, but not necessarily the parent/child relationships, because there are typically intervening nested element levels. The following figures show a simple table as displayed in the Viewer and the OXML that represents that table.

Gender

		Frequency	Percent	Valid Percent	Cumulative Percent
Valid	Female	216	45.6	45.6	45.6
	Male	258	54.4	54.4	100.0
	Total	474	100.0	100.0	

Figure 78. Simple frequency table

```
<?xml version="1.0" encoding="UTF-8" ?>
<outputTree>
 <command text="Frequencies" command="Frequencies"
  displayTableValues="label" displayOutlineValues="label"
  displayTableVariables="label" displayOutlineVariables="label">
  <pivotTable text="Gender" label="Gender" subType="Frequencies"
   varName="gender" variable="true">
   <dimension axis="row" text="Gender" label="Gender"
    varName="gender" variable="true">
    <group text="Valid">
     <group hide="true" text="Dummy">
      <category text="Female" label="Female" string="f"
       varName="gender">
       <dimension axis="column" text="Statistics">
        <category text="Frequency">
         <cell text="216" number="216"/>
        </category>
        <category text="Percent">
         <cell text="45.6" number="45.569620253165" decimals="1"/>
        </category>
        <category text="Valid Percent">
         <cell text="45.6" number="45.569620253165" decimals="1"/>
        </category>
        <category text="Cumulative Percent">
         <cell text="45.6" number="45.569620253165" decimals="1"/>
        </category>
       </dimension>
      </category>
      <category text="Male" label="Male" string="m" varName="gender">
       <dimension axis="column" text="Statistics">
        <category text="Frequency">
         <cell text="258" number="258"/>
        </category>
        <category text="Percent">
         <cell text="54.4" number="54.430379746835" decimals="1"/>
        </category>
        <category text="Valid Percent">
         <cell text="54.4" number="54.430379746835" decimals="1"/>
        </category>
        <category text="Cumulative Percent">
         <cell text="100.0" number="100" decimals="1"/>
        </category>
       </dimension>
      </category>
     </group>
     <category text="Total">
      <dimension axis="column" text="Statistics">
       <category text="Frequency">
        <cell text="474" number="474"/>
       </category>
       <category text="Percent">
        <cell text="100.0" number="100" decimals="1"/>
       </category>
       <category text="Valid Percent">
        <cell text="100.0" number="100" decimals="1"/>
       </category>
      </dimension>
     </category>
    </group>
   </dimension>
  </pivotTable>
 </command>
</outputTree>
```

Figure 79. OXML for simple frequency table

As you may notice, a simple, small table produces a substantial amount of XML. That's partly because the XML contains some information not readily apparent in the original table, some information that might not even be available in the original table, and a certain amount of redundancy.

- The table contents as they are (or would be) displayed in a pivot table in the Viewer are contained in `text` attributes. For example:

  ```
  <command text="Frequencies" command="Frequencies"...>
  ```

  These text attributes can be affected by both output language (SET OLANG) and settings that affect the display of variable names/labels and values/value labels (SET TVARS and SET TNUMBERS). In this example, the `text` attribute value will differ depending on the output language, whereas the `command` attribute value remains the same regardless of output language.

- Wherever variables or values of variables are used in row or column labels, the XML will contain a text attribute and one or more additional attribute values. For example:

```
<dimension axis="row" text="Gender" label="Gender" varName="gender">
 ...
  <category text="Female"label="Female" string="f" varName="gender">
```

  For a numeric variable, there would be a `number` attribute instead of a `string` attribute. The `label` attribute is present only if the variable or values have defined labels.

- The <cell> elements that contain cell values for numbers will contain the `text` attribute and one or more additional attribute values. For example:

```
<cell text="45.6" number="45.569620253165" decimals="1"/>
```

  The `number` attribute is the actual, unrounded numeric value, and the `decimals` attribute indicates the number of decimal positions displayed in the table.

- Since columns are nested within rows, the category element that identifies each column is repeated for each row. For example, since the statistics are displayed in the columns, the element `<category text="Frequency">` appears three times in the XML—once for the male row, once for the female row, and once for the total row.

Examples of using XSLT to transform OXML are provided in the Help system.

Command and Subtype Identifiers

The OMS Control Panel (Utilities menu) provides a complete list of command and subtype identifiers. For any command or table displayed in the Viewer, you can find out the command or subtype identifier by right-clicking the item in the Viewer outline pane.

OMSEND

Note: Square brackets used in the OMSEND syntax chart are required parts of the syntax and are not used to indicate optional elements. Any equals signs (=) displayed in the syntax chart are required. All specifications other than the command name OMSEND are optional.

```
OMSEND
 TAG = {['idvalue' 'idvalue'...]}
       {ALL                     }
 FILE = ['filespec' 'filespec'...]
 LOG
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic "Command Order" on page 36 for more information.

Example

```
OMS
 /DESTINATION FORMAT = OXML OUTFILE = '/mydir/myfile.xml'.
[some commands that produce output]
OMSEND.
[some more commands that produce output]
```

Overview

OMSEND ends active OMS commands. The minimum specification is the command name OMSEND. In the absence of any other specifications, this ends all active OMS commands and logging.

TAG Keyword

The optional TAG keyword identifies specific OMS commands to end, based on the ID value assigned on the OMS TAG subcommand or automatically generated if there is no TAG subcommand. To display the automatically generated ID values for active OMS commands, use the OMSINFO command

The TAG keyword must be followed by an equals sign (=) and a list of quoted ID values or the keyword ALL enclosed in square brackets.

Example

```
OMSEND TAG = ['reg_tables_to_sav' 'freq_tables_to_html'].
```

A warning is issued if any of the specified values don't match any active OMS commands.

FILE Keyword

The optional FILE keyword ends specific OMS commands based on the filename specified with the OUTFILE keyword of the DESTINATION subcommand of the OMS command. The FILE keyword must be followed by an equals sign (=), and a list of quoted file specifications must be enclosed in square brackets.

Example

```
OMSEND
 FILE = ['/mydir/mysavfile.sav' '/otherdir/myhtmlfile.htm'].
```

- If the specified file doesn't exist or isn't associated with a currently running OMS command, a warning is issued.
- The FILE keyword specification has no effect on OMS commands that use OUTPUTSET instead of OUTFILE.

LOG Keyword

IF OMS logging is in effect (OMSLOG command), the LOG keyword ends logging.

Examples

```
OMSEND LOG.
```

In this example, the OMSEND command ends logging without ending any active OMS commands.

OMSINFO

```
OMSINFO.
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
OMSINFO.
```

Overview

The OMSINFO command displays a table of all active OMS commands It has no additional specifications.

OMSLOG

```
OMSLOG FILE = 'filespec'
 [/APPEND = [NO ]]
            [YES]
 [/FORMAT = [XML ]]
            [TEXT]
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
OMSLOG FILE = '/mydir/mylog.xml'.
```

Overview

OMSLOG creates a log file in either XML or text form for subsequent OMS commands during a session.

- The log contains one line or main XML element for each destination file and contains the event name, filename, and location, the ID tag value, and a timestamp. The log also contains an entry when an OMS command is started and stopped.
- The log file remains open, and OMS activity is appended to the log, unless logging is turned off by an OMSEND command or the end of the session.
- A subsequent OMSLOG command that specifies a different log file ends logging to the file specified on the previous OMSLOG command.
- A subsequent OMSLOG file that specifies the same log file will overwrite the current log file for the default FORMAT = XML or in the absence of APPEND = YES for FORMAT = TEXT.
- OMS activity for any OMS commands executed *before* the first OMSLOG command in the session is not recorded in any log file.

Basic Specification

The basic specification is the command name OMSLOG followed by a FILE subcommand that specifies the log filename and location.

Syntax Rules

- The FILE subcommand is required. All other specifications are optional.
- Equals signs (=) shown in the command syntax chart and examples are required, not optional.

FILE Subcommand

The FILE subcommand specifies the log filename and location. The subcommand name must be followed by an equals sign (=) and a file specification in quotes. If the file specification includes location information (drive, directory/folder), the location must be a valid, existing location; otherwise an error will result.

Example

```
OMSLOG FILE = '/mydir/mylog.xml'.
```

APPEND Subcommand

If the FILE subcommand specifies an existing file, by default the file is overwritten. For text format log files, you can use the APPEND subcommand to append new logging information to the file instead of overwriting.

Example

```
OMSLOG FILE = '/mydir/mylog.txt'
  /APPEND = YES
  /FORMAT = TEXT.
```

- APPEND = YES is only valid with FORMAT = TEXT. For XML log files, the APPEND subcommand is ignored.
- APPEND = YES with FORMAT = TEXT will append to an existing file, even if the existing file contains XML-format log information. (An XML file *is* a text file, and OMSLOG does not differentiate based on file extension or content.)
- If the specified file does not exist, APPEND has no effect.

FORMAT Subcommand

The FORMAT subcommand specifies the format of the log file. The default format is XML. You can use FORMAT = TEXT to write the log in simple text format.

ONEWAY

ONEWAY is available in the Statistics Base Edition.

```
ONEWAY  varlist BY varname

 [/POLYNOMIAL=n]  [/CONTRAST=coefficient list] [/CONTRAST=... ]

 [/POSTHOC=([SNK] [TUKEY] [BTUKEY] [DUNCAN] [SCHEFFE] [DUNNETT[refcat)]
           [DUNNETTL(refcat)] [DUNNETTR(refcat)] [BONFERRONI] [LSD]
           [SIDAK] [GT2] [GABRIEL] [FREGW] [QREGW] [T2] [T3] [GH] [C]
           [WALLER({100** })]) [ALPHA({0.05**})]
                   {Kratio}           {α     }

 [/RANGES={LSD         }([{0.05**}])] [/RANGES=...]
          {DUNCAN      }  {α     }
          {SNK         }
          {TUKEYB      }
          {TUKEY       }
          {MODLSD      }
          {SCHEFFE     }

 [/STATISTICS=[NONE**] [DESCRIPTIVES]  [EFFECTS]  [HOMOGENEITY]  [ALL]  ]
              [WELCH] [BROWNFORSYTHE]

 [/PLOT MEANS ]

 [/MISSING=[{ANALYSIS**}]  [{EXCLUDE**}] ]
           {LISTWISE  }    {INCLUDE  }

 [/MATRIX =[IN({*                  })] [OUT({*                  })]  [NONE] ]
              {'savfile'|'dataset'}        {'savfile'|'dataset'}

 [/TEMPLATE='filename']

 [/CRITERIA=CILEVEL({0.95**}{value})  ]

 [/ES=[OVERALL] [CONTRAST({POOLED**}{GROUP})]  ],
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 19.0

- TEMPLATE subcommand introduced.

Release 27.0

- CRITERIA subcommand introduced.
- ES subcommand introduced.

Example

```
ONEWAY V1 BY V2.
```

Overview

ONEWAY produces a one-way analysis of variance for an interval-level dependent variable by one numeric independent variable that defines the groups for the analysis. Other procedures that perform an analysis of variance are SUMMARIZE, UNIANOVA, and GLM (GLM is available in SPSS Statistics Standard Edition or the Advanced Statistics Option). Some tests not included in the other procedures are available as options in ONEWAY.

Options

Trend and contrasts

You can partition the between-groups sums of squares into linear, quadratic, cubic, and higher-order trend components using the POLYNOMIAL subcommand. You can specify up to 10 contrasts to be tested with the *t* statistic on the CONTRAST subcommand.

Post hoc tests
You can specify 20 different post hoc tests for comparisons of all possible pairs of group means or multiple comparisons using the POSTHOC subcommand.

Statistical display
In addition to the default display, you can obtain means, standard deviations, and other descriptive statistics for each group using the STATISTICS subcommand. Fixed- and random-effects statistics as well as Levene tests for homogeneity of variance are also available.

Matrix input and output
You can write means, standard deviations, and category frequencies to a matrix data file that can be used in subsequent ONEWAY procedures using the MATRIX subcommand. You can also read matrix materials consisting of means, category frequencies, pooled variance, and degrees of freedom for the pooled variance.

Templates
You can specify a template, using the **TEMPLATE** subcommand, to override the default chart attribute settings on your system.

Criteria
Specifies the options for the procedure. The **CILEVEL** keyword controls the significance level to estimate the confidence intervals.

Effect size estimation
Controls the effect size estimation. The **OVERALL** keyword controls the calculation of the effect size for the overall test; the **CONTRAST** keyword controls the calculation of the effect size for the contrast test.

Basic Specification

The basic specification is a dependent variable, keyword BY, and an independent variable. ONEWAY produces an ANOVA table displaying the between- and within-groups sums of squares, mean squares, degrees of freedom, the *F* ratio, and the probability of *F* for each dependent variable by the independent variable.

Subcommand Order

- The variable list must be specified first.
- The remaining subcommands can be specified in any order.

Operations

- All values of the independent variable are used. Each different value creates one category.
- If a string variable is specified as an independent or dependent variable, ONEWAY is not executed.

Limitations

- Maximum 100 dependent variables and 1 independent variable.
- An unlimited number of categories for the independent variable. However, post hoc tests are not performed if the number of nonempty categories exceeds 50. Contrast tests are not performed if the total of empty and nonempty categories exceeds 50.
- Maximum 1 POLYNOMIAL subcommand.
- Maximum 1 POSTHOC subcommand.
- Maximum 10 CONTRAST subcommands.

Analysis List

The analysis list consists of a list of dependent variables, keyword BY, and an independent (grouping) variable.

- Only one analysis list is allowed, and it must be specified before any of the optional subcommands.
- All variables named must be numeric.

POLYNOMIAL Subcommand

POLYNOMIAL partitions the between-groups sums of squares into linear, quadratic, cubic, or higher-order trend components. The display is an expanded analysis-of-variance table that provides the degrees of freedom, sums of squares, mean square, *F*, and probability of *F* for each partition.

- The value specified on POLYNOMIAL indicates the highest-degree polynomial to be used.
- The polynomial value must be a positive integer less than or equal to 5 and less than the number of groups. If the polynomial specified is greater than the number of groups, the highest-degree polynomial possible is assumed.
- Only one POLYNOMIAL subcommand can be specified per ONEWAY command. If more than one is used, only the last one specified is in effect.
- ONEWAY computes the sums of squares for each order polynomial from weighted polynomial contrasts, using the category of the independent variable as the metric. These contrasts are orthogonal.
- With unbalanced designs and equal spacing between groups, ONEWAY also computes sums of squares using the unweighted polynomial contrasts. These contrasts are not orthogonal.
- The deviation sums of squares are always calculated from the weighted sums of squares[51].

Example

```
ONEWAY WELL BY EDUC6
  /POLYNOMIAL=2.
```

- ONEWAY requests an analysis of variance of *WELL* by *EDUC6* with second-order (quadratic) polynomial contrasts.
- The ANOVA table is expanded to include both linear and quadratic terms.

CONTRAST Subcommand

CONTRAST specifies a priori contrasts to be tested by the *t* statistic. The specification on CONTRAST is a vector of coefficients, where each coefficient corresponds to a category of the independent variable. The Contrast Coefficients table displays the specified contrasts for each group and the Contrast Tests table displays the value of the contrast and its standard error, the *t* statistic, and the degrees of freedom and two-tailed probability of *t* for each variable. Both pooled- and separate-variance estimates are displayed.

- A contrast coefficient must be specified or implied for every group defined for the independent variable. If the number of contrast values is not equal to the number of groups, the contrast test is not performed.
- The contrast coefficients for a set should sum to 0. If they do not, a warning is issued. ONEWAY will still give an estimate of this contrast.
- Coefficients are assigned to groups defined by ascending values of the independent variable.
- The notation *n***c* can be used to indicate that coefficient *c* is repeated *n* times.

Example

```
ONEWAY V1 BY V2
  /CONTRAST = -1 -1 1 1
  /CONTRAST = -1 0 0 1
  /CONTRAST = -1 0 .5 .5.
```

- *V2* has four levels.

[51] Speed, M. F. 1976. Response curves in the one way classification with unequal numbers of observations per cell. In: *Proceedings of the Statistical Computing Section*. Alexandria, VA: American Statistical Association.

- The first CONTRAST subcommand contrasts the combination of the first two groups with the combination of the last two groups.
- The second CONTRAST subcommand contrasts the first group with the last group.
- The third CONTRAST subcommand contrasts the first group with the combination of the third and fourth groups.

Example

```
ONEWAY V1 BY V2
  /CONTRAST = -1 1 2*0
  /CONTRAST = -1 1 0 0
  /CONTRAST = -1 1.
```

- The first two CONTRAST subcommands specify the same contrast coefficients for a four-group analysis. The first group is contrasted with the second group in both cases.
- The first CONTRAST uses the *n*c* notation.
- The last CONTRAST does not work because only two coefficients are specified for four groups.

POSTHOC Subcommand

POSTHOC produces post hoc tests for comparisons of all possible pairs of group means or multiple comparisons. In contrast to a priori analyses specified on the CONTRAST subcommand, post hoc analyses are usually not planned at the beginning of the study but are suggested by the data in the course of the study.

- Twenty post hoc tests are available. Some detect homogeneity subsets among the groups of means, some produce pairwise comparisons, and others perform both. POSTHOC produces a Multiple Comparison table showing up to 10 test categories. Nonempty group means are sorted in ascending order, with asterisks indicating significantly different groups. In addition, homogeneous subsets are calculated and displayed in the Homogeneous Subsets table if the test is designed to detect homogeneity subsets.
- When the number of valid cases in the groups varies, the harmonic mean of the group sizes is used as the sample size in the calculation for homogeneity subsets except for QREGW and FREGW. For QREGW and FREGW and tests for pairwise comparison, the sample sizes of individual groups are always used.
- You can specify only one POSTHOC subcommand per ONEWAY command. If more than one is specified, the last specification takes effect.
- You can specify one alpha value used in all POSTHOC tests using keyword ALPHA. The default is 0.05.

SNK. *Student-Newman-Keuls procedure based on the Studentized range test.* Used for detecting homogeneity subsets.

TUKEY. *Tukey's honestly significant difference.* This test uses the Studentized range statistic to make all pairwise comparisons between groups. Used for pairwise comparison and for detecting homogeneity subsets.

BTUKEY. *Tukey's b.* Multiple comparison procedure based on the average of Studentized range tests. Used for detecting homogeneity subsets.

DUNCAN. *Duncan's multiple comparison procedure based on the Studentized range test*. Used for detecting homogeneity subsets.

SCHEFFE. *Scheffé's multiple comparison* t *test.* Used for pairwise comparison and for detecting homogeneity subsets.

DUNNETT(refcat). *Dunnett's two-tailed t test.* Used for pairwise comparison. Each group is compared to a reference category. You can specify a reference category in parentheses. The default is the last category. This keyword must be spelled out in full.

DUNNETTL(refcat). *Dunnett's one-tailed t test.* Used for pairwise comparison. This test indicates whether the mean of each group (except the reference category) is *smaller* than that of the reference category.

You can specify a reference category in parentheses. The default is the last category. This keyword must be spelled out in full.

DUNNETTR(refcat). *Dunnett's one-tailed t test.* Used for pairwise comparison. This test indicates whether the mean of each group (except the reference category) is *larger* than that of the reference category. You can specify a reference category in parentheses. The default is the last category. This keyword must be spelled out in full.

BONFERRONI. *Bonferroni t test.* This test is based on Student's *t* statistic and adjusts the observed significance level for the fact that multiple comparisons are made. Used for pairwise comparison.

LSD. *Least significant difference t test.* Equivalent to multiple *t* tests between all pairs of groups. Used for pairwise comparison. This test does not control the overall probability of rejecting the hypotheses that some pairs of means are different, while in fact they are equal.

SIDAK. *Sidak t test.* Used for pairwise comparison. This test provides tighter bounds than the Bonferroni test.

GT2. *Hochberg's GT2.* Used for pairwise comparison and for detecting homogeneity subsets. This test is based on the Studentized maximum modulus test. Unless the cell sizes are extremely unbalanced, this test is fairly robust even for unequal variances.

GABRIEL. *Gabriel's pairwise comparisons test based on the Studentized maximum modulus test.* Used for pairwise comparison and for detecting homogeneity subsets.

FREGW. *Ryan-Einot-Gabriel-Welsch's multiple stepdown procedure based on an* F *test.* Used for detecting homogeneity subsets.

QREGW. *Ryan-Einot-Gabriel-Welsch's multiple stepdown procedure based on the Studentized range test.* Used for detecting homogeneity subsets.

T2. *Tamhane's T2.* Used for pairwise comparison. This test is based on a *t* test and can be applied in situations where the variances are unequal.

T3. *Tamhane's T3.* Used for pairwise comparison. This test is based on the Studentized maximum modulus test and can be applied in situations where the variances are unequal.

GH. *Games and Howell's pairwise comparisons test based on the Studentized range test.* Used for pairwise comparison. This test can be applied in situations where the variances are unequal.

C. *Dunnett's* C. Used for pairwise comparison. This test is based on the weighted average of Studentized ranges and can be applied in situations where the variances are unequal.

WALLER(kratio). *Waller-Duncan* t *test.* Used for detecting homogeneity subsets. This test uses a Bayesian approach. The *k*-ratio is the Type 1/Type 2 error seriousness ratio. The default value is 100. You can specify an integer greater than 1 within parentheses.

Example

```
ONEWAY WELL BY EDUC6
  /POSTHOC=SNK SCHEFFE ALPHA=.01.
```

- ONEWAY requests two different post hoc tests. The first uses the Student-Newman-Keuls test and the second uses Scheffé's test. Both tests use an alpha of 0.01.

RANGES Subcommand

RANGES produces results for some post hoc tests. It is available only through syntax. You can always produce the same results using the POSTHOC subcommand.

- Up to 10 RANGE subcommands are allowed. The effect is cumulative. If you specify more than one alpha value for different range tests, the last specified value takes effect for all tests. The default is 0.05.
- Keyword MODLSD on the RANGE subcommand is equivalent to keyword BONFERRONI on the POSTHOC subcommand. Keyword LSDMOD is an alias for MODLSD.

PLOT MEANS Subcommand

PLOT MEANS produces a chart that plots the subgroup means (the means for each group defined by values of the factor variable).

STATISTICS Subcommand

By default, ONEWAY displays the ANOVA table showing between- and within-groups sums of squares, mean squares, degrees of freedom, *F* ratio, and probability of *F*. Use STATISTICS to obtain additional statistics.

BROWNFORSYTHE. *Brown-Forsythe statistic.* The Brown-Forsythe statistic, degrees of freedom, and the significance level are computed for each dependent variable.

WELCH. *Welch statistic.* The Welch statistic, degrees of freedom, and the significance level are computed for each dependent variable.

DESCRIPTIVES. *Group descriptive statistics.* The statistics include the number of cases, mean, standard deviation, standard error, minimum, maximum, and 95% confidence interval for each dependent variable for each group.

EFFECTS. *Fixed- and random-effects statistics.* The statistics include the standard deviation, standard error, and 95% confidence interval for the fixed-effects model, and the standard error, 95% confidence interval, and estimate of between-components variance for the random-effects model.

HOMOGENEITY. *Homogeneity-of-variance tests.* The statistics include Levene test statistics based on the mean, median, and trimmed mean, degrees of freedom (including adjusted degrees of freedom for the test based on the median), and the significance level displayed in the Tests of Homogeneity of Variances table.

NONE. *No optional statistics.* This is the default.

ALL. *All statistics available for* ONEWAY.

MISSING Subcommand

MISSING controls the treatment of missing values.

- Keywords ANALYSIS and LISTWISE are alternatives. Each can be used with INCLUDE or EXCLUDE. The default is ANALYSIS and EXCLUDE.
- A case outside of the range specified for the grouping variable is not used.

ANALYSIS. *Exclude cases with missing values on a pair-by-pair basis.* A case with a missing value for the dependent or grouping variable for a given analysis is not used for that analysis. This is the default.

LISTWISE. *Exclude cases with missing values listwise.* Cases with missing values for any variable named are excluded from all analyses.

EXCLUDE. *Exclude cases with user-missing values.* User-missing values are treated as missing. This is the default.

INCLUDE. *Include user-missing values.* User-missing values are treated as valid values.

MATRIX Subcommand

MATRIX reads and writes matrix data files.

- Either IN or OUT and a matrix file in parentheses are required.
- You cannot specify both IN and OUT on the same ONEWAY procedure.

- Use MATRIX=NONE to explicitly indicate that a matrix data file is not being written or read.

OUT ('savfile'|'dataset'). *Write a matrix data file or dataset.* Specify either a filename, a previously declared dataset name, or an asterisk, enclosed in parentheses. Filenames should be enclosed in quotes and are stored in the working directory unless a path is included as part of the file specification. If you specify an asterisk (*), the matrix data file replaces the active dataset. If you specify an asterisk or a dataset name, the file is not stored on disk unless you use SAVE or XSAVE.

IN ('savfile'|'dataset'). *Read a matrix data file or dataset.* Specify either a filename, dataset name created during the current session, or an asterisk enclosed in parentheses. An asterisk reads the matrix data from the active dataset. Filenames should be enclosed in quotes and are read from the working directory unless a path is included as part of the file specification.

Matrix Output

- ONEWAY writes means, standard deviations, and frequencies to a matrix data file that can be used by subsequent ONEWAY procedures. For a description of the file, see below.

Matrix Input

- ONEWAY can read the matrices it writes, and it can also read matrix materials that include the means, category frequencies, pooled variance, and degrees of freedom for the pooled variance. The pooled variance has a *ROWTYPE_* value MSE, and the vector of degrees of freedom for the pooled variance has the *ROWTYPE_* value DFE.
- The dependent variables named on ONEWAY can be a subset of the dependent variables in the matrix data file.
- MATRIX=IN cannot be specified unless an active dataset has already been defined. To read an existing matrix data file at the beginning of a session, use GET to retrieve the matrix file and then specify IN(*) on MATRIX.

Format of the Matrix Data File

- The matrix data file includes two special variables created by the program: *ROWTYPE_* and *VARNAME_*.
- *ROWTYPE_* is a short string variable with values MEAN, STDDEV, and N.
- *VARNAME_* is a short string variable that never has values for procedure ONEWAY. *VARNAME_* is included with the matrix materials so that matrices written by ONEWAY can be read by procedures that expect to read a *VARNAME_* variable.
- The independent variable is between variables *ROWTYPE_* and *VARNAME_*.
- The remaining variables in the matrix file are the dependent variables.

Split Files

- When split-file processing is in effect, the first variables in the matrix data file are the split variables, followed by *ROWTYPE_*, the independent variable, *VARNAME_*, and the dependent variables.
- A full set of matrix materials is written for each split-file group defined by the split variable(s).
- A split variable cannot have the same variable name as any other variable written to the matrix data file.
- If split-file processing is in effect when a matrix is written, the same split file must be in effect when that matrix is read by any procedure.
- Generally, matrix rows, independent variables, and dependent variables can be in any order in the matrix data file read by keyword IN. However, all split-file variables must precede variable *ROWTYPE_*, and all split-group rows must be consecutive. ONEWAY ignores unrecognized *ROWTYPE_* values.

Missing Values

Missing-value treatment affects the values written to an matrix data file. When reading a matrix data file, be sure to specify a missing-value treatment on ONEWAY that is compatible with the treatment that was in effect when the matrix materials were generated.

Example

```
GET FILE=GSS80.
ONEWAY  WELL BY EDUC6
  /MATRIX=OUT(ONEMTX).
```

- ONEWAY reads data from file *GSS80* and writes one set of matrix materials to the file *ONEMTX*.
- The active dataset is still *GSS80*. Subsequent commands are executed on *GSS80*.

Example

```
GET FILE=GSS80.
ONEWAY  WELL BY EDUC6
  /MATRIX=OUT(*).
LIST.
```

- ONEWAY writes the same matrix as in the example above. However, the matrix data file replaces the active dataset. The LIST command is executed on the matrix file, not on the *GSS80* file.

Example

```
GET FILE=PRSNNL.
FREQUENCIES VARIABLE=AGE.
ONEWAY  WELL BY EDUC6
 /MATRIX=IN(ONEMTX).
```

- This example performs a frequencies analysis on *PRSNNL* and then uses a different file for ONEWAY. The file is an existing matrix data file.
- MATRIX=IN specifies the matrix data file.
- *ONEMTX* does not replace *PRSNNL* as the active dataset.

Example

```
GET FILE=ONEMTX.
ONEWAY  WELL BY EDUC6
  /MATRIX=IN(*).
```

- The GET command retrieves the matrix data file *ONEMTX*.
- MATRIX=IN specifies an asterisk because the active dataset is the matrix data file *ONEMTX*. If MATRIX=IN(ONEMTX) is specified, the program issues an error message, since *ONEMTX* is already open.
- If the GET command is omitted, the program issues an error message.

TEMPLATE Subcommand

TEMPLATE uses an existing file as a template and applies it to the means plot (/PLOT MEANS), if requested, for the current ONEWAY command. The template overrides the default settings that are used to create the plot. Templates are created in the Chart Editor by saving an existing chart as a template.

Example

```
ONEWAY perform BY group
  /MISSING ANALYSIS
  /STATISTICS HOMOGENEITY
  /PLOT MEANS
  /TEMPLATE='/templates/mytemplate.sgt'.
```

CRITERIA Subcommand

The optional CRIERIA subcommand controls the significance level to estimate the confidence intervals. The **CILEVEL** keyword controls the significance level to estimate the confidence intervals. The specified number must be a single numerical value between 0 and 1, with 0.95 as the default setting.

Note: **CILEVEL** resets the confidence interval value for both the descriptive statistics and the effect sizes.

Example

```
ONEWAY perform BY group
  /MISSING ANALYSIS
  /STATISTICS HOMOGENEITY
  /PLOT MEANS
  /TEMPLATE='/templates/mytemplate.sgt'.
  /CRITERIA = CILEVEL(0.95)
```

ES Subcommand

The optional ES subcommand controls the effect size estimation. It is omitted by default. When specified, at least one of the following keywords must be specified to calculate the effect sizes:

OVERALL
This optional keyword controls the calculation of the effect size for the overall test. When specified, the “ANOVA Effect Sizes” table displays in the output.

CONTRAST
This optional keyword controls the calculation of the effect size for the contrast test. When specified, the “Contrast Effect Sizes” table displays in the output. **CONTRAST(POOLED)** uses the pooled standard deviation for all the groups as the standardizer when estimating the effect size, (the default setting). **CONTRAST({GROUP})** uses the pooled standard deviation for groups involved in the contrast as the standardizer. This keyword takes effect in the presence of at least one user-supplied contrast, and is ignored when no contrasts are specified.

Example

```
ONEWAY V1 BY V2
  /CONTRAST = -1 -1 1 1
  /CONTRAST = -1 0 0 1
  /CONTRAST = -1 0 .5 .5.
  /ES = CONTRAST(POOLED)
```

References

Speed, M. F. 1976. Response curves in the one way classification with unequal numbers of observations per cell. In: *Proceedings of the Statistical Computing Section*. Alexandria, VA: American Statistical Association.

OPTIMAL BINNING

OPTIMAL BINNING is available in Statistics Base Edition.

```
OPTIMAL BINNING

/VARIABLES [GUIDE = variable] BIN = varlist [SAVE = {NO**                        }]
                                                   {YES [(INTO = new varlist)]}

[/CRITERIA

    [PREPROCESS = {EQUALFREQ**[(BINS = {1000**})]}]
                                       {n     }
                  {NONE                        }

    [METHOD = {MDLP**                      }]
              {EQUALFREQ [(BINS = {10**})]}
                                  {n   }

    [LOWEREND = {UNBOUNDED**}]     [UPPEREND = {UNBOUNDED**}]
                {OBSERVED   }                  {OBSERVED   }


    [LOWERLIMIT = {INCLUSIVE**}]
                  {EXCLUSIVE  }

    [FORCEMERGE = {0**  }]]
                  {value}

[/MISSING  [SCOPE = {PAIRWISE**}]]
                    {LISTWISE  }

[/OUTFILE  RULES = filespec]

[/PRINT  [ENDPOINTS**] [DESCRIPTIVES] [ENTROPY] [NONE]]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 15.0

- Command introduced.

Example

```
OPTIMAL BINNING
  /VARIABLES GUIDE = guide-variable BIN = binning-input-variable
```

Overview

The OPTIMAL BINNING procedure discretizes one or more scale variables (referred to henceforth as **binning input** variables) by distributing the values of each variable into bins. Bins can then be used instead of the original data values of the binning input variables for further analysis. OPTIMAL BINNING is useful for reducing the number of distinct values in the given binning input variables.

Options

Methods. The OPTIMAL BINNING procedure offers the following methods of discretizing binning input variables.

- Unsupervised binning via the equal frequency algorithm discretizes the binning input variables. A guide variable is not required.
- Supervised binning via the MDLP (Minimal Description Length Principle) algorithm discretizes the binning input variables without any preprocessing. It is suitable for datasets with a small number of cases. A guide variable is required.

Output. The OPTIMAL BINNING procedure displays every binning input variable's end point set in pivot table output and offers an option for suppressing this output. In addition, the procedure can save new

binned variables corresponding to the binning input variables and can save a command syntax file with commands corresponding to the binning rules.

Basic Specification

The basic specification is the OPTIMAL BINNING command and a VARIABLES subcommand. VARIABLES provides the binning input variables and, if applicable, the guide variable.

- For unsupervised binning via the equal frequency algorithm, a guide variable is not required.
- For supervised binning via the MDLP algorithm and hybrid binning, a guide variable must be specified.

Syntax Rules

- When a supervised binning method is used, a guide variable must be specified on the VARIABLES subcommand.
- Subcommands may be specified only once.
- An error occurs if a variable or keyword is specified more than once within a subcommand.
- Parentheses, slashes, and equals signs shown in the syntax chart are required.
- Empty subcommands are not honored.
- The command name, subcommand names, and keywords must be spelled in full.

Case Frequency

- If a WEIGHT variable is specified, then its values are used as frequency weights by the OPTIMAL BINNING procedure.
- Weight values are rounded to the nearest whole numbers before use. For example, 0.5 is rounded to 1, and 2.4 is rounded to 2.
- The WEIGHT variable may not be specified on any subcommand in the OPTIMAL BINNING procedure.
- Cases with missing weights or weights less than 0.5 are not used in the analyses.

Limitations

The number of distinct values in a guide variable should be less than or equal to 256, irrespective of the platform on which IBM SPSS Statistics is running. If the number is greater than 256, this results in an error.

VARIABLES Subcommand

The VARIABLES subcommand specifies the guide variable (if applicable) and one or more binning input variables. It can also be used to save new variables containing the binned values.

GUIDE=variable. *Guide variable.* The bins formed by supervised binning methods are "optimal" with respect to the specified guide variable. You must specify a guide variable to perform MDLP (CRITERIA METHOD = MDLP) or the hybrid method (CRITERIA PREPROCESS = EQUALFREQ METHOD = MDLP). This option is silently ignored if it is specified when the equal frequency method (CRITERIA METHOD = EQUALFREQ) is in effect. The guide variable may be numeric or string.

BIN=varlist. *Binning input variable list.* These are the variables to be binned. The variable list must include at least one variable. Binning input variables must be numeric.

SAVE = NO | YES (INTO = new varlist). *Create new variables containing binned values.* By default, the procedure does not create any new variables (NO). If YES is specified, variables containing the binned values are saved to the active dataset. Optionally, specify the names of the new variables using the INTO keyword.

- The number of variables specified on the INTO list must equal the number of variables on the BIN list. All specified names must be valid variable names. Violation of either of these rules results in an error.
- If INTO is omitted, new variable names are created by concatenating the guide variable name (if applicable) and an underscore '_', followed by the binning input variable name and an underscore,

followed by 'bin'. For example, /VARIABLES GUIDE=E BIN=F G SAVE=YES will generate two new variables: *E_F_bin* and *E_G_bin*.

CRITERIA Subcommand

The CRITERIA subcommand specifies bin creation options.

PREPROCESS = EQUALFREQ (BINS=n) | NONE. *Preprocessing method when MDLP binning is used.* PREPROCESS = EQUALFREQ creates preliminary bins using the equal frequency method before performing MDLP binning. These preliminary bins—rather than the original data values of the binning input variables—are input to the MDLP binning method.

- EQUALFREQ may be followed by parentheses containing the BINS keyword, an equals sign, and an integer greater than 1. The BINS value serves as a preprocessing threshold and specifies the number of bins to create. The default value is EQUALFREQ (BINS = 1000).
- If the number of distinct values in a binning input variable is greater than the BINS value, then the number of bins created is no more than the BINS value. Otherwise, no preprocessing is done for the input variable.
- NONE requests no preprocessing.

METHOD = MDLP | EQUALFREQ (BINS=n). *Binning method.* The MDLP option performs supervised binning via the MDLP algorithm. If METHOD = MDLP is specified, then a guide variable must be specified on the VARIABLES subcommand.

- Alternatively, METHOD = EQUALFREQ performs unsupervised binning via the equal frequency algorithm. EQUALFREQ may be followed by parentheses containing the BINS keyword, an equals sign, and an integer greater than 1. The BINS value specifies the number of bins to create. The default value of the BINS argument is 10.
- If the number of distinct values in a binning input variable is greater than the BINS value, then the number of bins created is no more than the BINS value. Otherwise, BINS gives an upper bound on the number of bins created. Thus, for example, if BINS = 10 is specified but a binning input variable has at most 10 distinct values, then the number of bins created will equal the number of distinct values in the input variable.
- If EQUALFREQ is specified, then the VARIABLES subcommand GUIDE keyword and the CRITERIA subcommand PREPROCESS keyword are silently ignored.
- The default METHOD option depends on the presence of a GUIDE specification on the VARIABLES subcommand. If GUIDE is specified, then METHOD = MDLP is the default. If GUIDE is not specified, then METHOD = EQUALFREQ is the default.

LOWEREND = UNBOUNDED | OBSERVED. *Specifies how the minimum end point for each binning input variable is defined.* Valid option values are UNBOUNDED or OBSERVED. If UNBOUNDED, then the minimum end point extends to negative infinity. If OBSERVED, then the minimum observed data value is used.

UPPEREND = UNBOUNDED | OBSERVED. *Specifies how the maximum end point for each binning input variable is defined.* Valid option values are UNBOUNDED or OBSERVED. If UNBOUNDED, then the maximum end point extends to positive infinity. If OBSERVED, then the maximum of the observed data is used.

LOWERLIMIT =INCLUSIVE | EXCLUSIVE. *Specifies how the lower limit of an interval is defined.* Valid option values are INCLUSIVE or EXCLUSIVE. Suppose the start and end points of an interval are *p* and *q*, respectively. If LOWERLIMIT = INCLUSIVE, then the interval contains values greater than or equal to *p* but less than *q*. If LOWERLIMIT = EXCLUSIVE, then the interval contains values greater than *p* and less than or equal to *q*.

FORCEMERGE = value. *Small bins threshold.* Occasionally, the procedure may produce bins with very few cases. The default value of FORCEMERGE is 0; by default, forced merging of very small bins is not performed.

MISSING Subcommand

The MISSING subcommand specifies whether missing values are handled using listwise or pairwise deletion.

- User-missing values are always treated as invalid. When recoding the original binning input variable values into a new variable, user-missing values are converted into system-missing values.

SCOPE = PAIRWISE | LISTWISE. *Missing value handling method.* LISTWISE provides a consistent case base. It operates across all variables specified on the VARIABLES subcommand. If any variable is missing for a case, then the entire case is excluded.

PAIRWISE makes use of as many valid values as possible. When METHOD = MDLP, it operates on each guide and binning input variable pair. The procedure will make use of all cases with nonmissing values on the guide and binning input variable. When METHOD = EQUALFREQ, it uses all cases with nonmissing values for each binning input variable. PAIRWISE is the default.

OUTFILE Subcommand

The OUTFILE subcommand writes syntax to an external command syntax file.

RULES=filespec. *Rules file specification.* The procedure can generate command syntax that can be used to bin other datasets. The recoding rules are based on the end points determined by the binning algorithm. Specify an external file to contain the saved syntax.

Note that saved variables (see the SAVE keyword in theVARIABLES subcommand) are generated using end points exactly as computed by the algorithm, while the bins created via saved syntax rules use end points converted to and from a decimal representation. Conversion errors in this process can, in certain cases, cause the end points read from syntax to differ from the original ones. The syntax precision of end points is 17 digits.

PRINT Subcommand

The PRINT subcommand controls the display of the output results. If the PRINT subcommand is not specified, then the default output is the end point set for each binning input variable.

ENDPOINTS. *Display the binning interval end points for each input variable.* This is the default output.

DESCRIPTIVES. *Display descriptive information for all binning input variables.* For each binning input variable, this option displays the number of cases with valid values, the number of cases with missing values, the number of distinct valid values, and the minimum and maximum values. For the guide variable, this option displays the class distribution for each related binning input variable.

ENTROPY. *Display the model entropy for each binning input variable interval when MDLP binning is used.* The ENTROPY keyword is ignored with a warning if METHOD = EQUALFREQ is specified or implied on the CRITERIA subcommand.

NONE. *Suppress all displayed output except the notes table and any warnings.* Specifying NONE with any other keywords results in an error.

ORTHOPLAN

ORTHOPLAN is available in SPSS Statistics Premium Edition or the Conjoint option.

```
ORTHOPLAN
 [/FACTORS=varlist ['labels'] (values ['labels'])...]

 [{/REPLACE     }]
  {/OUTFILE='savfile'|'dataset'}

 [/MINIMUM=value]

 [/HOLDOUT=value]  [/MIXHOLD={YES}]
                             {NO }
```

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example:

```
ORTHOPLAN FACTORS=SPEED 'Highest possible speed'
   (70 '70 mph' 100 '100 mph' 130 '130mph')
 WARRANTY 'Length of warranty'
   ('1 year' '3 year' '5 year')
 SEATS (2, 4)
 /MINIMUM=9 /HOLDOUT=6.
```

Overview

ORTHOPLAN generates an orthogonal main-effects plan for a full-concept conjoint analysis. ORTHOPLAN can append or replace an existing active dataset, or it can build an active dataset (if one does not already exist). The generated plan can be listed in full-concept profile, or card, format using PLANCARDS. The file that is created by ORTHOPLAN can be used as the plan file for CONJOINT.

Basic specification

- The basic specification is ORTHOPLAN followed by FACTORS, a variable list, and a value list in parentheses. ORTHOPLAN will generate cases in the active dataset, with each case representing a profile in the conjoint experimental plan and consisting of a new combination of the factor values. By default, the smallest possible orthogonal plan is generated.
- If you are appending to an existing active dataset that has previously defined values, the FACTORS subcommand is optional.

Subcommand order

- Subcommands can be named in any order.

Operations

- ORTHOPLAN builds an active dataset (if one does not already exist) by using the variable and value information on the FACTORS subcommand.
- When ORTHOPLAN appends to an active dataset and FACTORS is not used, the factor levels (values) must be defined on a previous ORTHOPLAN or VALUE LABELS command.
- New variables *STATUS_* and *CARD_* are created and added to the active dataset by ORTHOPLAN if they do not already exist. *STATUS_*=0 for experimental cases, 1 for holdout cases, and 2 for simulation cases. Holdout cases are judged by the subjects but are not used when CONJOINT estimates utilities. Instead, the cases are used as a check on the validity of the estimated utilities. Simulation cases are entered by the user. They are factor-level combinations that are not rated by the subjects but are estimated by CONJOINT based on the ratings of the experimental cases. *CARD_* contains the case identification numbers in the generated plan.
- Duplication between experimental cases and simulation cases is reported.

- If a user-entered experimental case (*STATUS_*=0) is duplicated by ORTHOPLAN, only one copy of the case is kept.
- Occasionally, ORTHOPLAN may generate duplicate experimental cases. One way to handle these duplicates is to edit or delete them, in which case the plan is no longer orthogonal. Alternatively, you can try running ORTHOPLAN again. With a different *seed*, ORTHOPLAN might produce a plan without duplicates. See the SEED subcommand on SET for more information about the random seed generator.
- If you do not specify OUTFILE or REPLACE and there are no open datasets, the procedure creates a new, unnamed dataset. If the active dataset and the ORTHOPLAN command contain the same factors, the factor values from the ORTHOPLAN command are appended to the factor values in the active dataset. If the active dataset is a named dataset, a new dataset is created. If the active dataset is an unnamed dataset, the active dataset is replaced. If the variables in the active dataset do not match factors in the ORTHOPLAN command, an error occurs.
- The SPLIT FILE and WEIGHT commands are ignored by ORTHOPLAN.

Limitations

- Missing data are not allowed.
- A maximum of 9 levels can be specified for each factor.
- The total number of levels for all factors cannot exceed 90.
- A maximum of 81 cases can be generated by ORTHOPLAN.

Examples

```
ORTHOPLAN FACTORS=SPEED 'Highest possible speed'
   (70 '70 mph' 100 '100 mph' 130 '130mph')
  WARRANTY 'Length of warranty'
   ('1 year' '3 year' '5 year')
  SEATS (2, 4) /MINIMUM=9 /HOLDOUT=6 /OUTFILE='CARPLAN.SAV'.
```

- The FACTORS subcommand defines the factors and levels to be used in building the file. Labels for some of the factors and some of the levels of each factor are also supplied.
- The MINIMUM subcommand specifies that the orthogonal plan should contain at least nine full-concept cases.
- HOLDOUT specifies that six holdout cases should be generated. A new variable, *STATUS_*, is created by ORTHOPLAN to distinguish these holdout cases from the regular experimental cases. Another variable, *CARD_*, is created to assign identification numbers to the plan cases.
- The OUTFILE subcommand saves the plan that is generated by ORTHOPLAN as a data file so that it can be used at a later date with CONJOINT.

Example: Appending Plan to the Working File

```
DATA LIST FREE /SPEED WARRANTY SEATS.
VALUE LABELS speed 70 '70 mph' 100 '100 mph' 130 '130 mph'
 /WARRANTY 1 '1 year' 3 '3 year' 5 '5 year'
 /SEATS 2 '2 seats' 4 '4 seats'.
BEGIN DATA
130 5 2
130 1 4
END DATA.
ORTHOPLAN
 /OUTFILE='CARPLAN.SAV'.
```

- In this example, ORTHOPLAN appends the plan to the active dataset and uses the variables and values that were previously defined in the active dataset as the factors and levels of the plan.
- The data between BEGIN DATA and END DATA are assumed to be simulation cases and are assigned a value of 2 on the newly created *STATUS_* variable.
- The OUTFILE subcommand saves the plan that is generated by ORTHOPLAN as a data file so that it can be used at a later date with CONJOINT.

FACTORS Subcommand

FACTORS specifies the variables to be used as factors and the values to be used as levels in the plan.

- FACTORS is required for building a new active dataset or replacing an existing one. FACTORS is optional for appending to an existing file.
- The keyword FACTORS is followed by a variable list, an optional label for each variable, a list of values for each variable, and optional value labels.
- The list of values and the value labels are enclosed in parentheses. Values can be numeric or they can be strings enclosed in apostrophes.
- The optional variable and value labels are enclosed in apostrophes.
- If the FACTORS subcommand is not used, every variable in the active dataset (other than *STATUS_* and *CARD_*) is used as a factor, and level information is obtained from the value labels that are defined in the active dataset. ORTHOPLAN *must* be able to find value information either from a FACTORS subcommand or from a VALUE LABELS command. (See the VALUE LABELS command for more information.)

Example

```
ORTHOPLAN FACTORS=SPEED 'Highest possible speed'
   (70 '70 mph' 100 '100 mph' 130 '130mph')
  WARRANTY 'Length of warranty'
   (1 '1 year' 3 '3 year' 5 '5 year')
  SEATS 'Number of seats' (2 '2 seats' 4 '4 seats')
  EXCOLOR 'Exterior color'
  INCOLOR 'Interior color' ('RED' 'BLUE' 'SILVER').
```

- *SPEED*, *WARRANTY*, *SEATS*, *EXCOLOR*, and *INCOLOR* are specified as the factors. They are given the labels *Highest possible speed*, *Length of warranty*, *Number of seats*, *Exterior color*, and *Interior color*.
- Following each factor and its label are the list of values and the value labels in parentheses. Note that the values for two of the factors, *EXCOLOR* and *INCOLOR*, are the same and thus need to be specified only once after both factors are listed.

REPLACE Subcommand

- If the active dataset is an unnamed dataset, REPLACE replaces the contents of the active dataset with the generated plan.
- If the active dataset is a named dataset, REPLACE creates a new, unnamed dataset.

OUTFILE Subcommand

OUTFILE saves the orthogonal design to an external IBM SPSS Statistics data file or previously declared dataset. The only specification is a name for the output file. This specification can be a filename or a previously declared dataset name. Filenames should be enclosed in quotation marks and are stored in the working directory unless a path is included as part of the file specification. Datasets are available during the current session but are not available in subsequent sessions unless you explicitly save them as data files.

- By default, a new data file is not created. Any new variables that are specified on a FACTORS subcommand plus the variables *STATUS_* and *CARD_* are appended to the active dataset.
- The output data file contains variables *STATUS_*, *CARD_*, and any other variables that are specified on the FACTORS subcommand.
- The file that is created by OUTFILE can be used by other syntax commands, such as PLANCARDS and CONJOINT.
- If both OUTFILE and REPLACE are specified, REPLACE is ignored.

MINIMUM Subcommand

MINIMUM specifies a minimum number of cases for the plan.

- By default, the minimum number of cases necessary for the orthogonal plan is generated.
- MINIMUM is followed by a positive integer that is less than or equal to the total number of cases that can be formed from all possible combinations of the factor levels.
- If ORTHOPLAN cannot generate at least the number of cases requested on MINIMUM, it will generate the largest number it can that fits the specified factors and levels.

HOLDOUT Subcommand

HOLDOUT creates holdout cases in addition to the regular plan cases. Holdout cases are judged by the subjects but are not used when CONJOINT estimates utilities.

- If HOLDOUT is not specified, no holdout cases are produced.
- HOLDOUT is followed by a positive integer that is less than or equal to the total number of cases that can be formed from all possible combinations of factor levels.
- Holdout cases are generated from another random plan, not the main-effects experimental plan. The holdout cases will not duplicate the experimental cases or each other.
- The experimental and holdout cases will be randomly mixed in the generated plan or the holdout cases will be listed after the experimental cases, depending on subcommand MIXHOLD. The value of *STATUS_* for holdout cases is 1. Any simulation cases will follow the experimental and holdout cases.

MIXHOLD Subcommand

MIXHOLD indicates whether holdout cases should be randomly mixed with the experimental cases or should appear separately after the experimental plan in the file.

- If MIXHOLD is not specified, the default is NO, meaning holdout cases will appear after the experimental cases in the file.
- MIXHOLD followed by keyword YES requests that the holdout cases be randomly mixed with the experimental cases.
- MIXHOLD specified without a HOLDOUT subcommand has no effect.

OUTPUT ACTIVATE

```
OUTPUT ACTIVATE [NAME=]name
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 15.0

- Command introduced.

Example

```
GET FILE='/examples/data/SalaryData2005.sav'.
DESCRIPTIVES salbegin salary.
OUTPUT NAME alleduclevels.

TEMPORARY.
SELECT IF (educ>12).
OUTPUT NEW NAME=over12.
DESCRIPTIVES salbegin salary.

GET FILE='/examples/data/SalaryData2000.sav'.
TEMPORARY.
SELECT IF (educ>12).
DESCRIPTIVES salbegin salary.

OUTPUT ACTIVATE alleduclevels.
DESCRIPTIVES salbegin salary.
```

Overview

The OUTPUT commands (OUTPUT NEW, OUTPUT NAME, OUTPUT ACTIVATE, OUTPUT OPEN, OUTPUT SAVE, OUTPUT CLOSE) provide the ability to programmatically manage one or many output documents. These functions allow you to:

- Save an output document through syntax.
- Programmatically partition output into separate output documents (for example, results for males in one output document and results for females in a separate one).
- Work with multiple open output documents in a given session, selectively appending new results to the appropriate document.

The OUTPUT ACTIVATE command activates an open output document, making it the designated output document. Subsequent procedure output is directed to this output document until the document is closed or another output document is created, opened, or activated.

Basic Specification

The basic specification for OUTPUT ACTIVATE is the command name followed by the name of an open output document. This is the name assigned by a previous OUTPUT NAME, OUTPUT OPEN, OUTPUT NEW command, or automatically assigned when the document is created or opened in the current session; it is not the file name associated with a saved output document. In the Viewer window title bar, the name is enclosed in square brackets. The NAME keyword is optional, but if it is used it must be followed by an equals sign.

Operations

- The window containing the activated document becomes the designated output window in the user interface.
- An error occurs, but processing continues, if the named output document does not exist. Output continues to be directed to the last designated output document.

Example

```
GET FILE='/examples/data/SurveyData.sav'.

TEMPORARY.
SELECT IF (Sex='Male').
FREQUENCIES VARIABLES=ALL.
OUTPUT NAME males.

TEMPORARY.
SELECT IF (Sex='Female').
OUTPUT NEW NAME=females.
FREQUENCIES VARIABLES=ALL.

GET FILE='/examples/data/Preference.sav'.

TEMPORARY.
SELECT IF (Sex='Female').
DESCRIPTIVES VARIABLES=product1 product2 product3.

TEMPORARY.
SELECT IF (Sex='Male').
OUTPUT ACTIVATE males.
DESCRIPTIVES VARIABLES=product1 product2 product3.

OUTPUT SAVE NAME=males OUTFILE='/examples/output/Males.spv'.
OUTPUT SAVE NAME=females OUTFILE='/examples/output/Females.spv'.
```

- The first GET command loads survey data for males and females.
- FREQUENCIES output for male respondents is written to the designated output document. The OUTPUT NAME command is used to assign the name *males* to the designated output document.
- FREQUENCIES output for females is written to a new output document named *females*.
- The second GET command loads preferences data for males and females.
- After the second GET command, the output document named *females* is still the designated output document. Descriptive statistics for females are appended to this output document.
- OUTPUT ACTIVATE males activates the output document named *males*. Descriptive statistics for males are appended to this output document.
- The two open output documents are saved to separate files. Because the operation of saving an output document does not close it, both documents remain open. The output document named *males* remains the designated output document.

OUTPUT CLOSE

```
OUTPUT CLOSE [NAME=]{name}
                    {*   }
                    {ALL }
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 15.0

- Command introduced.

Example

```
GET FILE='/examples/data/Males.sav'.
FREQUENCIES VARIABLES=ALL.
OUTPUT SAVE OUTFILE='/examples/output/Males.spv'.
OUTPUT CLOSE *.

GET FILE='/examples/data/Females.sav'.
FREQUENCIES VARIABLES=ALL.
```

Overview

The OUTPUT commands (OUTPUT NEW, OUTPUT NAME, OUTPUT ACTIVATE, OUTPUT OPEN, OUTPUT SAVE, OUTPUT CLOSE) provide the ability to programmatically manage one or many output documents. These functions allow you to:

- Save an output document through syntax.
- Programmatically partition output into separate output documents (for example, results for males in one output document and results for females in a separate one).
- Work with multiple open output documents in a given session, selectively appending new results to the appropriate document.

The OUTPUT CLOSE command closes one or all open output documents.

Basic Specification

The only specification for OUTPUT CLOSE is the command name followed by the name of an open output document, an asterisk (*), or the keyword ALL. The NAME keyword is optional, but if it is used it must be followed by an equals sign.

Operations

- If a name is provided, the specified output document is closed and the association with that name is broken.
- If an asterisk (*) is specified, the designated output document is closed. If the designated output document has a name, the association with that name is broken.
- If ALL is specified, all open output documents are closed and all associations of names with output documents are broken.
- Output documents are not saved automatically when they are closed. Use OUTPUT SAVE to save the contents of an output document.
- OUTPUT CLOSE is ignored if you specify a nonexistent document.

Example

```
GET FILE='/examples/data/Males.sav'.
FREQUENCIES VARIABLES=ALL.
OUTPUT SAVE OUTFILE='/examples/output/Males.spv'.
OUTPUT CLOSE *.
```

```
GET FILE='/examples/data/Females.sav'.
FREQUENCIES VARIABLES=ALL.
```

- FREQUENCIES produces summary statistics for each variable. Procedure output is added to the designated output document (one is created automatically if no output document is currently open).
- OUTPUT SAVE writes contents of the designated output document to the file */examples/output/Males.spv*.
- OUTPUT CLOSE closes the designated output document.
- Output from the second FREQUENCIES command is written to a new output document, which was created automatically when the previous designated output document was closed. If OUTPUT CLOSE had not been issued, output for females would have been directed to the output document that contained summaries for males.

OUTPUT DISPLAY

```
OUTPUT DISPLAY
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 15.0

- Command introduced.

Example

```
OUTPUT DISPLAY.
```

Overview

The OUTPUT commands (OUTPUT NEW, OUTPUT NAME, OUTPUT ACTIVATE, OUTPUT OPEN, OUTPUT SAVE, OUTPUT CLOSE) provide the ability to programmatically manage one or many output documents. These functions allow you to:

- Save an output document through syntax.
- Programmatically partition output into separate output documents (for example, results for males in one output document and results for females in a separate one).
- Work with multiple open output documents in a given session, selectively appending new results to the appropriate document.

The OUTPUT DISPLAY command displays a list of open output documents and identifies the one that is currently active, and referred to as the designated output document. The only specification is the command name OUTPUT DISPLAY.

OUTPUT EXPORT

```
OUTPUT EXPORT  [NAME = {*   }]
                      {name}

 [/CONTENTS [EXPORT = {ALL**  }]]
                     {VISIBLE}
  [LAYERS = {PRINTSETTING**}]
            {ALL           }
            {VISIBLE       }
  [MODELVIEWS = {PRINTSETTING**}]
                {ALL           }
                {VISIBLE       }

 [/DOC DOCUMENTFILE = 'filespec']
  [NOTESCAPTIONS = {YES**}]
                   {NO   }
  [WIDETABLES = {WRAP**}]
                {SHRINK}
                {ASIS  }
  [PAGESIZE = {INCHES(width,height)}]
              {MM(width,height)    }
  [TOPMARGIN = {INCHES(n)}]
               {MM(n)    }
  [BOTTOMMARGIN = {INCHES(n)}]
                  {MM(n)    }
  [LEFTMARGIN = {INCHES(n)}]
                {MM(n)    }
  [RIGHTMARGIN = {INCHES(n)}]
                 {MM(n)    }
  [PAGEBREAKS = {YES**}]
                {NO   }

 [/HTML DOCUMENTFILE = 'filespec']
  [NOTESCAPTIONS = {YES**}]
                   {NO   }
  [IMAGEFORMAT = {JPG**}]
                 {BMP  }
                 {PNG  }
  [STYLING = {YES** }
             {NO    }
  [INTERACTIVELAYERS = {NO**}]
                       {YES }

 [/PDF DOCUMENTFILE = 'filespec']
  [EMBEDBOOKMARKS = {YES**}]
                    {NO   }
  [EMBEDFONTS = {YES**}]
                {NO   }

 [/PPT DOCUMENTFILE = 'filespec']
  [NOTESCAPTIONS = {YES**}]
                   {NO   }
  [SLIDETITLES = {YES**}]
                 {NO   }
  [WIDETABLES = {WRAP**}]
                {SHRINK}
                {ASIS  }
 [SLIDESIZE = {INCHES(width,height)}]
              {MM(width,height)    }
 [TOPMARGIN = {INCHES(n)}]
              {MM(n)    }
 [BOTTOMMARGIN = {INCHES(n)}]
                 {MM(n)    }
 [LEFTMARGIN = {INCHES(n)}]
               {MM(n)    }
 [RIGHTMARGIN = {INCHES(n)}]
                {MM(n)    }

[/TEXT DOCUMENTFILE = 'filespec']
 [ENCODING = {LOCALE**}]
             {UTF8    }
             {UTF16   }
 [NOTESCAPTIONS = {YES**}]
                  {NO   }
 [COLUMNWIDTH = {AUTOFIT**}]
                {width    }
 [SEPARATOR = {SPACE**}]
              {TAB    }
 [ROWBORDER = {'-'**      }]
              {'character'}
 [COLUMNBORDER = {'|'**      }]
                 {'character'}
 [IMAGEFORMAT = {JPG**}]
                {BMP  }
                {EMF  }
                {EPS  }
                {PNG  }
                {TIF  }

[/{XLS|XLSX|XLSM} DOCUMENTFILE = 'filespec']
 [OPERATION = {CREATEFILE**}]
              {CREATESHEET }
              {MODIFYSHEET }
 [SHEET = {'sheetname'}]
 [LOCATION = {LASTCOLUMN**        }]
             {LASTROW             }
             {STARTCELL@@('cellref')}
 [NOTESCAPTIONS = {YES**}]
                  {NO   }
```

```
[/BMP IMAGEROOT = 'rootname']
 [PERCENTSIZE = {100**}]
                {n    }
 [COMPRESS = {NO**}]
             {YES }

[/EMF IMAGEROOT = 'rootname']
 [PERCENTSIZE = {100**}]
                {n    }

[/EPS IMAGEROOT = 'rootname']
 [WIDTH = {PERCENT**({100**})}]
                    {n    }
           {CENTIMETERS(n)     }
           {INCHES(n)          }
           {POINTS(n)          }
 [PREVIEW = {YES**}]
            {NO   }
 [FONTHANDLING = {REFERENCES**}]
                 {CURVES     }
[/JPG IMAGEROOT = 'rootname']
 [PERCENTSIZE = {100**}]
                {n    }
 [GRAYSCALE = {NO**}]
              {YES }

[/PNG IMAGEROOT = 'rootname']
 [PERCENTSIZE = {100**}]
                {n    }
 [DEPTH = {SCREEN**  }]
           {BLACKWHITE}
           {GRAYS256  }
           {COLORS16  }
           {COLORS256 }
           {TRUE24BIT }
           {TRUE32BIT }

[/TIF IMAGEROOT = 'rootname']
 [PERCENTSIZE = {100**}]
                {n    }
```

** Default if the subcommand or keyword is omitted.

@@ **STARTCELL** is not designed to overwrite existing worksheet data. The export can result in corrupted tables and an unusable worksheet in cases where **STARTCELL** overwrites existing worksheet data.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 17.0

- Command introduced.

Release 21.0

- Subcommands XLSX and XLSM added.
- STYLING keyword added to HTML subcommand.
- BREAKPOINTS keyword added to DOC subcommand.

Release 22.0.

- Subcommand REPORT added.
- INTERACTIVELAYERS keyword added to HTML subcommand

Release 27.0

- The REPORT subcommand was deprecated. The REPORT subcommand syntax has been mapped to the HTML subcommand.

Example

```
OUTPUT EXPORT
  /DOC DOCUMENTFILE='/mydocs/myresults.doc'.
```

Overview

The OUTPUT EXPORT command exports output from an open Viewer document to an external file format. By default, the contents of the designated Viewer document are exported, but a different Viewer document can be specified by name.

Options

Options include:

- Export output in Word, Excel, HTML, PDF, PowerPoint, and text format.
- Export all items or only visible items.
- Export graphics only.

Basic Specification

The basic specification is the OUTPUT EXPORT command, followed by one of the document type subcommands with the DOCUMENTFILE keyword or by one of the image type subcommands with the IMAGEROOT keyword.

Syntax Rules

- The command name, subcommand names, and all keywords must be spelled in full. (Abbreviation is not allowed.)
- Each subcommand must be preceded by a forward slash.
- Equals signs shown in the syntax diagram are required elements.
- Subcommands can be specified in any order.
- Each subcommand can be specified only once.
- Each subcommand must be followed by at least one keyword specification.
- Within a subcommand, each keyword can be specified only once.

Operations

By default, the contents of the **designated** Viewer document are exported. The designated Viewer document is the one most recently opened (by OUTPUT NEW or OUTPUT OPEN, or from the menus and dialogs), or explicitly designated (by OUTPUT ACTIVATE or by selecting Designate Window from the Utilities menu).

To export an open Viewer document other than the designated one, use the NAME keyword. See the topic "NAME Keyword " on page 1360 for more information.

Limitations

- Export of selected Viewer items is not supported in OUTPUT EXPORT. You can only export all items or all visible items. To export selected items, use the Export Output dialog.
- Exporting to PowerPoint (*.ppt) format is supported only on Windows operating systems that have PowerPoint installed.
- EMF graphic format is available only on Windows operating systems.
- OUTPUT EXPORT is not supported in the IBM SPSS Statistics Batch Facility (available with IBM SPSS Statistics Server). To export output created in the Batch Facility, use "OMS" on page 1301.

Examples

Export to Word with Minimum Specifications

```
OUTPUT EXPORT
  /DOC DOCUMENTFILE='/mydocs/myresults.doc'.
```

All output in the designated Viewer document is exported to the specified file in Word (RTF) format. If the file already exists, the contents are overwritten.

Export Named Viewer Document to New Sheet in Existing Excel Document

```
OUTPUT EXPORT NAME=Region1Results
  /XLS DOCUMENTFILE='/mydocs/ResultsByRegion.xls'
   OPERATION=CREATESHEET SHEET='Region 1'.
```

- Contents of the Viewer document named *Region1Results* are exported to the specified Excel file.
- If the Excel file already exists, a new worksheet named *Region 1* is added to the file. If a worksheet with that name already exists in the file, the contents of that worksheet are overwritten.
- If the Excel file does not exist, a file with the specified name is created, and a worksheet named *Region 1* is added to the new file.

NAME Keyword

The optional NAME keyword specifies the Viewer document from which to export output. This is useful when more than one Viewer document is open and you want to ensure that the command operates on the correct document.

NAME=*. *Export the contents of the designated output document.* This is the default.

NAME=name. *Export the contents of the named output document.* This is the name assigned by a previous OUTPUT NAME, OUTPUT OPEN, OUTPUT NEW command, or automatically assigned when the document is created or opened in the current session; it is not the file name associated with a saved output document. In the Viewer window title bar, the name is enclosed in square brackets.

Example

```
OUTPUT EXPORT NAME=Region1Results
  /XLS DOCUMENTFILE='/mydocs/ResultsByRegion.xls'.
```

CONTENTS Subcommand

The optional CONTENTS subcommand Viewer controls which contents are exported.

EXPORT Keyword

The EXPORT keyword specifies whether to export all Viewer items or only those that are visible.

ALL. *Export all Viewer items.* All Viewer items are exported regardless of whether they are visible or hidden. This is the default.

VISIBLE. *Export visible items only.* Hidden items are not included.

LAYERS Keyword

By default, inclusion or exclusion of pivot table layers is controlled by the table properties for each pivot table. You can override this setting and include all layers or exclude all but the currently visible layer. See the topic for more information.

PRINTSETTING. *Export layers as specified for printing in Table Properties.* This is the default.

ALL. *Export all layers.* Each layer is included as a separate table in the exported file.

VISIBLE. *Export visible layer only.* Only the layer currently displayed in the Viewer window is included in the exported file.

MODELVIEWS Keyword

By default, inclusion or exclusion of model views is controlled by the model properties for each model. You can override this setting and include all views or exclude all but the currently visible view in the Model Viewer. See the topic for more information.

PRINTSETTING. *Export model views as specified for printing in Model Properties.* This is the default.

ALL. *Export all model views.* Each item is included as a separate image.

VISIBLE. *Export visible view only.* Only the model view currently displayed in the Viewer window is included.

Note: All model views are exported as images. This includes any tables.

Example

```
OUTPUT EXPORT
  /CONTENTS EXPORT=VISIBLE LAYERS=ALL MODELVIEWS=VISIBLE
  /DOC DOCUMENTFILE='/mydocs/myresults.doc'.
```

DOC Subcommand

The DOC subcommand exports Viewer documents in Word (RTF) format. Pivot tables are exported as Word tables with all formatting attributes intact (for example, cell borders, font styles, and background colors). Text output is exported as formatted RTF. Charts, tree diagrams, and model views are included in PNG format.

DOCUMENTFILE Keyword

The required DOCUMENTFILE keyword specifies the name of the Word document. The keyword is followed by an equals sign and a quoted file specification, as in:

```
OUTPUT EXPORT
  /DOC DOCUMENTFILE='/mydocs/myresults.doc'.
```

PAGESIZE Keyword

The PAGESIZE keyword controls the page dimensions of the exported document. The keyword is followed by an equals sign and one of the following alternatives:

INCHES(width, height). *Height and width specified in inches.* Both values are required and both must be positive numbers.

MM(width, height). *Height and width specified in millimeters.* Both values are required and must be positive numbers.

- The default page dimensions are the default settings for the default printer.
- The orientation of the page is implied by the dimensions. Specifying a width that is less than the height is equivalent to choosing portrait orientation for the page. Specifying a width that is greater than the height is equivalent to choosing landscape orientation.

Note: the page size width setting is ignored if WIDETABLES=ASIS and one or more pivot tables is wider than the specified width. The page width automatically expands to accommodate the widest table.

TOPMARGIN, BOTTOMMARGIN, LEFTMARGIN, and RIGHTMARGIN Keywords

These keywords control the margins for the exported document. Each keyword is followed by an equals sign and one of the following alternatives:

INCHES(n). *Margin specified in inches.* The value must be a positive number.

MM(n). *Margin specified in millimeters.* The value must be a positive number.

NOTESCAPTIONS Keyword

The NOTESCAPTIONS keyword controls the inclusion of pivot table footnotes and captions in the exported document. The keyword is followed by an equals sign and one of the following alternatives:

YES. *Include footnotes and captions.* This is the default.

NO. *Do not include footnotes and captions.*

WIDETABLES Keyword

The WIDETABLES keyword controls the treatment of pivot tables that are too wide for the document width (the specified page width minus the left and right margins). The keyword is followed by an equals sign and one of the following alternatives:

WRAP. *Wrap wide tables.* The table is divided into sections that will fit within the defined document width. Row labels are repeated for each section of the table. This is the default. If the row labels are too wide for the defined document width, the table is exported without wrapping and will appear truncated in the document.

SHRINK. *Shrink wide tables to fit within the document width.* Font size and column width are reduced so that the table fits within the document width.

ASIS. *Do not shrink or wrap wide tables.* The document width expands to accommodate the widest table.

PAGEBREAKS keyword

If you have inserted break points in pivot tables, the PAGEBREAKS keyword indicates if these should be preserved in Word tables.

Yes. Break points and keep together groups are preserved in Word tables. This is the default.

No. Break points and keep together groups are ignored.

Example

```
OUTPUT EXPORT
  /DOC DOCUMENTFILE='/mydocs/myresults.doc'
   PAGESIZE=INCHES(11, 8.5)
   TOPMARGIN=INCHES(1)
   BOTTOMMARGIN=INCHES(1)
   LEFTMARGIN=INCHES(1)
   RIGHTMARGIN=INCHES(1)
   NOTESCAPTIONS=NO
   WIDETABLES=SHRINK.
```

HTML Subcommand

The HTML subcommand exports Viewer documents in HTML format. Output objects that would be pivot tables in the Viewer are converted to HTML tables. Text output objects are tagged <PRE> in the HTML. Charts, tree diagrams, and model views are embedded in the document in the selected format.

DOCUMENTFILE Keyword

The required DOCUMENTFILE keyword specifies the name of the HTML document. The keyword is followed by an equals sign and a quoted file specification, as in:

```
OUTPUT EXPORT
  /HTML DOCUMENTFILE='/mydocs/myresults.htm'.
```

NOTESCAPTIONS Keyword

The NOTESCAPTIONS keyword controls the inclusion of pivot table footnotes and captions in the exported document. The keyword is followed by an equals sign and one of the following alternatives:

YES. *Include footnotes and captions.* This is the default.

NO. *Do not include footnotes and captions.*

IMAGEFORMAT Keyword

The IMAGEFORMAT keyword controls the format of the exported graphics. The keyword is followed by an equals sign and one of the following alternatives:

BMP. *Bitmap format.* A raster graphics format.

JPG. *Joint Photographic Experts Group format.* This is the default.

PNG. *Portable Network Graphics format.* A raster graphics format.

Note: Graphics includes charts, tree diagrams, and any Model Viewer items, including Model Viewer tables.

STYLING Keyword

The STYLING keyword controls style information included for exported pivot tables.

YES. All pivot table style information (font styles, background colors, etc.) and column widths are preserved. This is the default.

NO. Pivot tables are converted to default HTML tables. No style attributes are preserved. Column width is determined automatically.

INTERACTIVELAYERS Keyword

The optional INTERACTIVELAYERS keyword controls interactivity of layered tables.

NO. Layered tables are not interactive. Each table layer is displayed as a separate table. This option is the default.

YES. Layered tables are displayed as they appear in the Viewer, and you can interactively change the displayed layer in the browser.

Note: INTERACTIVELAYERS is ignored if LAYERS=VISIBLE or LAYERS=PRINTSETTING and the print setting only includes the visible layer.

Example

```
OUTPUT EXPORT
  /HTML DOCUMENTFILE='/mydocs/myresults.htm'
   NOTESCAPTIONS=NO IMAGEFORMAT=PNG INTERACTIVELAYERS=YES.
```

PDF Subcommand

The PDF subcommand exports output in PDF format. Output is exported as it appears in Print Preview, with all formatting attributes intact.

DOCUMENTFILE Keyword

The required DOCUMENTFILE keyword specifies the name of the PDF document. The keyword is followed by an equals sign and a quoted file specification, as in:

```
OUTPUT EXPORT
  /PDF DOCUMENTFILE='/mydocs/myresults.pdf'.
```

EMBEDBOOKMARKS Keyword

The EMBEDBOOKMARKS keyword specifies whether to include bookmarks in the PDF document that correspond to the Viewer outline entries. Like the Viewer outline pane, bookmarks can make it much easier to navigate documents with a large number of output objects. The keyword is followed by an equals sign and one of the following alternatives:

YES. *Include bookmarks.* This is the default.

NO. *Do not include bookmarks.*

EMBEDFONTS Keyword

The EMBEDFONTS keyword specifies whether to embed fonts in the exported document. Embedding fonts ensures that the PDF document will look the same on all computers. Otherwise, if some fonts used in the document are not available on the computer being used to view (or print) the PDF document, font substitution may yield suboptimal results. The keyword is followed by an equals sign and one of the following alternatives:

YES. *Embed fonts.* This is the default.

NO. *Do not embed fonts.*

Other Settings That Affect PDF Output

Page Setup/Page Attributes. Page size, orientation, margins, content and display of page headers and footers, and printed chart size in PDF documents are controlled by page setup and page attribute options.

Table Properties/TableLooks. Scaling of wide and/or long tables and printing of table layers are controlled by table properties for each table. These properties can also be saved in TableLooks.

Default/Current Printer. The resolution (DPI) of the PDF document is the current resolution setting for the default or currently selected printer (which can be changed using Page Setup). The maximum resolution is 1200 DPI. If the printer setting is higher, the PDF document resolution will be 1200 DPI.

Note: High-resolution documents may yield poor results when printed on lower-resolution printers.

Example

```
OUTPUT EXPORT
  /PDF DOCUMENTFILE='/mydocs/myresults.pdf'
   EMBEDBOOKMARKS=NO EMBEDFONTS=NO.
```

PPT Subcommand

The PPT subcommand exports output in PowerPoint format. This is available only on Windows operating systems and requires an installed version of PowerPoint 97 or later.

- Pivot tables are exported as Word tables, and each table is a separate slide in the PowerPoint file. All formatting attributes of the pivot table are retained--for example, cell borders, font styles, and background colors.
- Graphics are exported in TIFF format, and each graphic is separate slide in the PowerPoint file.
- Text output is not included.

DOCUMENTFILE Keyword

The required DOCUMENTFILE keyword specifies the name of the PowerPoint document. The keyword is followed by an equals sign and a quoted file specification, as in:

```
OUTPUT EXPORT
  /PPT DOCUMENTFILE='/mydocs/myresults.ppt'.
```

SLIDESIZE Keyword

The SLIDESIZE keyword controls the slide dimensions of the exported file. The keyword is followed by an equals sign and one of the following alternatives:

INCHES(width, height). *Height and width specified in inches.* Both values are required and both must be positive numbers.

MM(width, height). *Height and width specified in millimeters.* Both values are required and must be positive numbers.

TOPMARGIN, BOTTOMMARGIN, LEFTMARGIN, and RIGHTMARGIN Keywords

These keywords control the margins for the exported document. Each keyword is followed by an equals sign and one of the following alternatives:

INCHES(n). *Margin specified in inches.* The value must be a positive number.

MM(n). *Margin specified in millimeters.* The value must be a positive number.

NOTESCAPTIONS Keyword

The NOTESCAPTIONS keyword controls the inclusion of pivot table footnotes and captions in the exported document. The keyword is followed by an equals sign and one of the following alternatives:

YES. *Include footnotes and captions.* This is the default.

NO. *Do not include footnotes and captions.*

SLIDETITLES Keyword

The SLIDETITLES keyword controls the inclusion slide titles of the exported file. The title is formed from the outline entry for the item in the outline pane of the Viewer. The keyword is followed by an equals sign and one of the following alternatives:

YES. *Include slide titles.* This is the default.

NO. *Do not include slide titles*.

WIDETABLES Keyword

The WIDETABLES keyword controls the treatment of pivot tables that are too wide for the document width (the specified page width minus the left and right margins). The keyword is followed by an equals sign and one of the following alternatives:

WRAP. *Wrap wide tables.* The table is divided into sections that will fit within the defined document width. Row labels are repeated for each section of the table. This is the default. If the row labels are too wide for the defined document width, the table is exported without wrapping and will appear truncated in the document.

SHRINK. *Shrink wide tables to fit within the document width.* Font size and column width are reduced so that the table fits within the document width.

ASIS. *Do not shrink or wrap wide tables.* The document width expands to accommodate the widest table.

Example

```
OUTPUT EXPORT
  /PPT DOCUMENTFILE='/mydocs/myresults.ppt'
   SLIDESIZE=INCHES(10, 7.5)
   SLIDETITLES=NO
   WIDETABLES=ASIS.
```

TEXT Subcommand

The TEXT subcommand exports output in text format.

- Text output formats include local encoding, UTF-8, and UTF-16.
- Pivot tables can be exported in tab-separated or space-separated format.
- All text output is exported in space-separated format.
- Graphics are exported as separate files in the selected graphics format. A line is inserted in the text file indicating the file name of each exported image.
- Graphics are exported to the same folder as the text document. Image file names use the text file name as the root name, followed by a sequential integer, starting with 0.

DOCUMENTFILE Keyword

The required DOCUMENTFILE keyword specifies the name of the text document. The keyword is followed by an equals sign and a quoted file specification, as in:

```
OUTPUT EXPORT
  /TEXT DOCUMENTFILE='/mydocs/myresults.txt'.
```

ENCODING Keyword

The ENCODING keyword specifies the type of text file to create. The keyword is followed by an equals sign and one of the following alternatives:

LOCALE. *Export in the character set (code page) of the current locale.* This is the default. For more information, see SET Command, LOCALE subcommand.

UTF8. *Export as 8-bit Unicode text.*

UTF16. *Export as 16-bit Unicode text.*

NOTESCAPTIONS Keyword

The NOTESCAPTIONS keyword controls the inclusion of pivot table footnotes and captions in the exported document. The keyword is followed by an equals sign and one of the following alternatives:

YES. *Include footnotes and captions.* This is the default.

NO. *Do not include footnotes and captions.*

SEPARATOR Keyword

Pivot table output can be exported as space-separated or tab-separated text. The SEPARATOR keyword is followed by an equals sign and one of the following alternatives:

SPACE. *Use spaces to separate columns.* This is the default.

TAB. *Use tabs to separate columns.*

COLUMNWIDTH Keyword

For space-separated pivot table output, the COLUMNWIDTH keyword controls column width. The available alternatives are:

COLUMNWIDTH=AUTOFIT. *Make each column as wide as the widest label or value in that column.* Column contents will not be wrapped. This is the default.

COLUMNWIDTH=n. *Wrap labels or values longer than n characters wide.* The value of *n* must be a positive integer.

Note: This setting affects only pivot tables exported in space-separated format (SEPARATOR=SPACE).

ROWBORDER Keyword

For space-separated pivot table output, the ROWBORDER keyword specifies the character used to create row borders. The keyword is followed by an equals sign and a single quoted character. The default is a hyphen (-).

Note: This setting affects only pivot tables exported in space-separated format (SEPARATOR=SPACE).

COLUMNBORDER Keyword

For space-separated pivot table output, the COLUMNBORDER keyword specifies the character used to create column borders. The keyword is followed by an equals sign and a single quoted character. The default is the vertical bar/pipe symbol (|).

Note: This setting affects only pivot tables exported in space-separated format (SEPARATOR=SPACE).

IMAGEFORMAT Keyword

The IMAGEFORMAT keyword controls the format of the exported graphics. The keyword is followed by an equals sign and one of the following alternatives:

BMP. *Bitmap format.* A raster graphics format.

EMF. *Enhanced metafile format.* A vector graphic format that is available only on Windows operating systems.

EPS. *Encapsulated PostScript format.* This format is often used when the quality of printed images is important.

JPG. *Joint Photographic Experts Group format.* This is the default.

PNG. *Portable Network Graphics format.* A raster graphics format.

TIF. *Tagged image file format.* A raster graphics format.

Note: Graphics includes charts, tree diagrams, and any Model Viewer items, including Model Viewer tables.

Example

```
OUTPUT EXPORT
  /TEXT DOCUMENTFILE='/mydocs/myresults.txt'
   ENCODING=UTF8
```

```
  SEPARATOR=TAB
  IMAGEFORMAT=PNG.
```

XLS, XLSX, and XLSM subcommands

The XLS, XLSX, and XLSM subcommands export output in Excel 97-2004, Excel 2007 and higher, and Excel 2007 and higher, macro-enabled formats respectively. Pivot table rows, columns, and cells are exported as Excel rows, columns, and cells, with all formatting attributes intact -- for example, cell borders, font styles, and background colors. Text output is exported with all font attributes intact. Each line in the text output is a row in the Excel file, with the entire contents of the line contained in a single cell. Charts, tree diagrams, and model views are included in PNG format.There are no macros in the output; the XLSM option allows you to append output to existing macro-enabled (.xlsm) Excel files.

DOCUMENTFILE keyword

The required DOCUMENTFILE keyword specifies the name of the Excel document. The keyword is followed by an equals sign and a quoted file specification, as in:

```
OUTPUT EXPORT
  /XLS DOCUMENTFILE='/mydocs/myresults.xls'.
```

OPERATION keyword

The OPERATION keyword specifies the type of operation to be performed by the export. The keyword is followed by an equals sign and one of the following alternatives:

CREATEFILE. *Create an Excel file.* If a file with the specified name already exists, it is overwritten. This is the default.

CREATESHEET. *Create a new worksheet within an Excel file.* If a worksheet with the specified name (specified on the SHEET keyword) already exists in the file , that worksheet is overwritten. If the file doesn't already exist, a new file is created with a worksheet with the specified name.

MODIFYSHEET. *Modify the contents of the specified worksheet.* Content is added to the worksheet based on the specification on the LOCATION keyword. You must specify a sheet name with the SHEET keyword. If you modify an existing Excel 97-2004 worksheet, charts, model views, and tree diagrams are not included in the exported output.

SHEET keyword

The SHEET keyword specifies the name of the worksheet to which items will be exported. The keyword is followed by an equals sign and a sheet name enclosed in quotes.

- Sheet names cannot exceed 31 characters and cannot contain forward or back slashes, square brackets, question marks, or asterisks.
- If the specified worksheet doesn't exist in the Excel file, a new worksheet with that name will be created, regardless of operation method.
- The SHEET keyword is required with OPERATION=MODIFYSHEET. It is optional with the other operation methods.

LOCATION keyword

The LOCATION keyword specifies the location within the worksheet for the exported items. The keyword is followed by an equals sign and one of the following alternatives:

LASTCOLUMN. *Add exported items after the last column.* Exported items will be added after the last column, starting in the first row, without modifying any existing contents. This is a good choice for adding new columns to an existing worksheet. This is the default.

LASTROW. *Add exported items after the last row.* Exported items will be added after the last row, starting in the first column, without modifying any existing contents. This is a good choice for adding new rows to an existing worksheet.

STARTCELL('cell'). *Add exported items beginning at a specific cell location.* Exported items will be added to the worksheet starting at the specified cell location. The cell reference is required and must be enclosed in quotes. Any existing content in the area where the exported items are added will be overwritten.



Warning: STARTCELL is not designed to overwrite existing worksheet data. The export can result in corrupted tables and an unusable worksheet in cases where **STARTCELL** overwrites existing worksheet data.

NOTECAPTIONS keyword

The NOTESCAPTIONS keyword controls the inclusion of pivot table footnotes and captions in the exported document. The keyword is followed by an equals sign and one of the following alternatives:

YES. *Include footnotes and captions.* This is the default.

NO. *Do not include footnotes and captions.*

Example

```
OUTPUT EXPORT
  /XLS DOCUMENTFILE='/mydocs/myresults.xls'
   OPERATION=MODIFYSHEET
   SHEET='Region 1'
   LOCATION=STARTCELL('B2').
```

BMP Subcommand

For graphics exported as separate files in BMP format, the BMP subcommand controls settings for those graphic files.

IMAGEROOT Keyword

- The IMAGEROOT keyword is required if you are exporting graphics only, without any document type.
- The keyword is followed by an equals sign and a root name enclosed in quotes. The quoted root name can also include a directory path.
- File names are generated by adding a sequential integer to the root name, starting with 0.
- If any subcommand with a DOCUMENTFILE keyword is specified, IMAGEROOT is ignored.

PERCENTSIZE Keyword

The PERCENTSIZE keyword specifies the size of graphics as a percentage of the original size. The keyword is followed by an equals sign and a positive number that does not exceed 200. The default value is 100.

COMPRESS Keyword

The COMPRESSION keyword controls image compression to reduce file size. The keyword is followed by an equals sign and one of the following alternatives:

YES. *Compress images.* A lossless technique is used to reduce file size without affecting image quality.

NO. *Do not compress images.* This is the default.

Example

```
OUTPUT EXPORT
  /BMP IMAGEROOT='/mydocs/mygraphic'
   PERCENTSIZE=50 COMPRESS=YES
```

EMF Subcommand

For graphics exported as separate files in enhanced metafile format, the EMF subcommand controls settings for those graphic files. EMF format is available only on Windows operating systems.

IMAGEROOT Keyword

- The IMAGEROOT keyword is required if you are exporting graphics only, without any document type.
- The keyword is followed by an equals sign and a root name enclosed in quotes. The quoted root name can also include a directory path.
- File names are generated by adding a sequential integer to the root name, starting with 0.
- If any subcommand with a DOCUMENTFILE keyword is specified, IMAGEROOT is ignored.

PERCENTSIZE Keyword

The PERCENTSIZE keyword specifies the size of graphics as a percentage of the original size. The keyword is followed by an equals sign and a positive number that does not exceed 200. The default value is 100.

Example

```
OUTPUT EXPORT
  /EMF IMAGEROOT='/mydocs/mygraphic'
   PERCENT=50.
```

EPS Subcommand

For graphics exported as separate files in encapuslated PostScript format, the EPS subcommand controls settings for those graphic files.

IMAGEROOT Keyword

- The IMAGEROOT keyword is required if you are exporting graphics only, without any document type.
- The keyword is followed by an equals sign and a root name enclosed in quotes. The quoted root name can also include a directory path.
- File names are generated by adding a sequential integer to the root name, starting with 0.
- If any subcommand with a DOCUMENTFILE keyword is specified, IMAGEROOT is ignored.

WIDTH Keyword

The WIDTH keyword controls the size of the exported image by specifying a value for the width of the image. The original aspect ratio is always maintained. The keyword is followed by an equals sign and one of the following alternatives:

PERCENT(n). Width specified as percentage of the original width. The value enclosed in parentheses must be a positive number that does not exceed 200. The default value is 100.

INCHES(n). Width specified in inches. The value enclosed in parentheses must be a positive number.

CM(n). Width specified in centimeters. The value enclosed in parentheses must be a positive number.

POINTS(n). Width specified in points. The value enclosed in parentheses must be a positive number.

PREVIEW Keyword

The PREVIEW keyword controls the inclusion of a TIFF preview image along with the EPS image. The preview image will be displayed if the application you are using is unable to display EPS images on the screen. The keyword is followed by an equals sign and one of the following alternatives:

YES. Include a preview imaged. This is the default.

NO. Do not include a preview image.

FONTHANDLING Keyword

The FONTHANDLING keyword controls the treatment of fonts in EPS images. The keyword is followed by an equals sign and one of the following alternatives:

REFERENCE. Use font references. If the fonts that are used in the graphic are available on the output device, those fonts are used. Otherwise, the output device uses alternate fonts. This is the default.

CURVES. Convert fonts to curves. This option turns fonts into PostScript curve data. The text itself is no longer editable as text in applications that can edit EPS graphics. This option is useful if the fonts that are used in the chart are not available on the output device.

Example

```
OUTPUT EXPORT
  /EPS IMAGEROOT='/mydocs/mygraphic'
   WIDTH=PERCENT(50) PREVIEW=NO FONTHANDLING=CURVES.
```

JPG Subcommand

For graphics exported as separate files in JPEG format, the JPG subcommand controls settings for those graphic files.

IMAGEROOT Keyword

- The IMAGEROOT keyword is required if you are exporting graphics only, without any document type.
- The keyword is followed by an equals sign and a root name enclosed in quotes. The quoted root name can also include a directory path.
- File names are generated by adding a sequential integer to the root name, starting with 0.
- If any subcommand with a DOCUMENTFILE keyword is specified, IMAGEROOT is ignored.

PERCENTSIZE Keyword

The PERCENTSIZE keyword specifies the size of graphics as a percentage of the original size. The keyword is followed by an equals sign and a positive number that does not exceed 200. The default value is 100.

GRAYSCALE Keyword

The GRAYSCALE keyword controls conversion of color images to grayscale images. The keyword is followed by an equals sign and one of the following alternatives:

YES. *Convert images to grayscale.* Color are converted to shades of gray.

NO. *Do not convert images to grayscale.* This is the default.

Example

```
OUTPUT EXPORT
  /JPG IMAGEROOT='/mydocs/mygraphic'
   PERCENT=50 GRAYSCALE=YES.
```

PNG Subcommand

For graphics exported as separate files in PNG format, the PNG subcommand controls settings for those graphic files.

IMAGEROOT Keyword

- The IMAGEROOT keyword is required if you are exporting graphics only, without any document type.
- The keyword is followed by an equals sign and a root name enclosed in quotes. The quoted root name can also include a directory path.
- File names are generated by adding a sequential integer to the root name, starting with 0.

- If any subcommand with a DOCUMENTFILE keyword is specified, IMAGEROOT is ignored.

PERCENTSIZE Keyword

The PERCENTSIZE keyword specifies the size of graphics as a percentage of the original size. The keyword is followed by an equals sign and a positive number that does not exceed 200. The default value is 100.

DEPTH Keyword

The DEPTH keyword controls the number of colors in the exported graphics. If the number of colors in the graphic exceeds the number of colors for that depth, the colors will be dithered to replicate the colors in the graphic. The keyword is followed by an equals sign and one of the following alternatives:

SCREEN. *Current screen depth.* The number of colors currently displayed on your computer monitor. This is the default.

BLACKWHITE. *Black and white.* Each color is converted to either black or white.

GRAYS256. *Grayscale.* Colors are converted to shades of gray.

COLORS16. *16 colors.*

COLORS256. *256 colors.*

TRUE24BIT. *24-bit true color.*

TRUE32BIT. *32-bit true color.*

Example

```
OUTPUT EXPORT
  /PNG IMAGEROOT='/mydocs/mygraphic'
   PERCENT=50
   DEPTH=COLORS256.
```

TIF Subcommand

For graphics exported as separate files in TIFF, the TIF subcommand controls settings for those graphic files.

IMAGEROOT Keyword

- The IMAGEROOT keyword is required if you are exporting graphics only, without any document type.
- The keyword is followed by an equals sign and a root name enclosed in quotes. The quoted root name can also include a directory path.
- File names are generated by adding a sequential integer to the root name, starting with 0.
- If any subcommand with a DOCUMENTFILE keyword is specified, IMAGEROOT is ignored.

PERCENTSIZE Keyword

The PERCENTSIZE keyword specifies the size of graphics as a percentage of the original size. The keyword is followed by an equals sign and a positive number that does not exceed 200. The default value is 100.

Example

```
OUTPUT EXPORT
  /TIF IMAGEROOT='/mydocs/mygraphic'
   PERCENT=50.
```

OUTPUT MODIFY

Note: Square brackets used in the OUTPUT MODIFY syntax chart are required parts of the syntax and are not used to indicate optional elements. Any equals signs (=) displayed in the syntax chart are required. All subcommands except SELECT are optional.

```
OUTPUT MODIFY NAME={*   }
                   {ALL }
                   {name}

 /REPORT PRINTREPORT = {NO**  }
                       {YES   }

 /SELECT CHARTS HEADINGS LOGS MODELS
         TABLES TEXTS TREES WARNINGS
         OUTLINEHEADERS UNKNOWN OTHER
         PAGETITLE NOTES
 or
 /SELECT ALL EXCEPT = [list]

 /IF  COMMANDS = ["expression", "expression(integer)"
                  "expression(LAST)", LAST]
      SUBTYPES = ["expression" "expression"...]
      LABELS = [EXACT("expression") BEGINS("expression")
                ENDS("expression") CONTAINS("expression")]
      INSTANCES = [n n... LAST]
      VISIBLEONLY={NO**}
                  {YES }

 /DELETEOBJECT DELETE={NO**}
                      {YES }

 /INDEXING
  INDEXTYPE = {NUMBER** }
              {LETTERLOW}
              {LETTERCAP}
              {ROMANLOW }
              {ROMANCAP }
  INDEXSTART = {1**     }
               {integer }
               {"letter"}
               {"roman" }

 /OBJECTPROPERTIES
  VISIBLE = {ASIS**}
            {YES   }
            {NO    }
  OUTLINELABEL = "expression"
  SIZE = {PERCENT    }  ({width, height       })
         {CENTIMETERS}   {PROPORTIONAL, height}
         {INCHES     }   {width, PROPORTIONAL }
         {POINTS     }

 /TABLE
  TABLETITLE = "expression"
  TRANSPOSE = {NO**}
              {YES }
  TLOOK = {"filespec"}
          {"name"    }
          {NONE      }
  SELECTEDLAYER = [{"variable", "category"}]
                  {"variable", number    }
                  {number, "category"    }
                  {number, number        }
  SORT = {COLLABEL("label")  }
       {COLPOSITION(number)}
  SORTDIRECTION = {ASCENDING**}
                  {DESCENDING }
  TABLESUMMARY = "expression"
  PIVOT = [expression]

 /TABLECELLS
  SELECT = ["expression", COUNT, MEAN, MEDIAN, RESIDUAL,
            PERCENT, SIGNIFICANCE, VALID, TOTAL, MISSING,
            CUMULATIVEPERCENT, VALIDPERCENT, POSITION(integer),
            BODY, HEADERS, FOOTNOTES, TITLE, CAPTION, …]
  SELECTDIMENSION = {BOTH** }
                    {COLUMNS}
                    {ROWS   }
  SELECTCONDITION = {ALL**,                                                  }
                    {["expression", TOP(integer), BOTTOM(integer),
                      PARENT(expression), CHILD(expression),…]}
  COLUMNWIDTH = {CENTIMETERS(value)}
                {INCHES(value)     }
                {POINTS(value)     }
  APPLYTO = {CELL**      }
            {COLUMN      }
            {ROW         }
            (COLUMNDATA  }
            {ROWDATA     }
            {COLUMNHEADER}
            {ROWHEADER   }
  REVERTTODEFAULT = {NO**}
                    {YES }
  FORMAT=format
  FONT = "fontfamily"
  FONTSIZE = value
  STYLE = REGULAR BOLD ITALIC UNDERLINE
```

```
  TEXTCOLOR = {color       }
              {RGB(r, g, b)}
  BACKGROUNDCOLOR = {color       }
                    {RGB(r, g, b)}
  REPLACE = {"expression"}
            {number      }
  HIDE = {NO**}
         {YES }
  UNGROUP = {NO**}
            {YES }

 /GRAPHS
  CTEMPLATE = {["filespec", "filespec", ...]}
              {[name, name, ...]           }
  VIZSTYLESHEET = "stylesheet name"

 /TEXTS
  TEXT = "expression"
  FONT = "fontfamily"
  FONTSIZE = value
  STYLE = REGULAR BOLD ITALIC UNDERLINE
  TEXTCOLOR = {color       }
              {RGB(r, g, b)}
```

- If included, the REPORT subcommand must be the first subcommand.
- Multiple blocks of output modifications are allowed. Each block of modification specifications must begin with a SELECT subcommand.
- Valid color values for TEXTCOLOR and BACKGROUNDCOLOR are `BLACK, BLUE, CYAN, DARKGRAY, GRAY, GREEN, LIGHTGRAY, MAGENTA, ORANGE, PINK, RED, WHITE, YELLOW`.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 27.0

- Added support for the **PIVOT** keyword in the **TABLES** subcommand.
- Added support for the **VALID**, **TOTAL**, **MISSING**, **CUMULATIVEPERCENT**, and **VALIDPERCENT** options in the SELECT keyword for the TABLECELLS subcommand.
- Added support for the **HIDE** and **UNGROUP** keywords in the **TABLECELLS** subcommand.
- Added support for the **PARENT** and **CHILD** options for the **SELECTCONDITION** keyword in the **TABLECELLS** subcommand.

Release 22.0

- Command introduced.

Example

```
OUTPUT MODIFY
 /SELECT TABLES
 /IF SUBTYPES=["Frequencies"]
 /TABLECELLS SELECT=["Total"]
  SELECTDIMENSION=ROWS
  STYLE=BOLD APPLYTO=ROW.
```

Overview

The `OUTPUT MODIFY` command applies formatting and other changes to the contents of the active Viewer window. Changes that can be applied include:

- All or selected viewer objects
- Selected types of output objects (for example, charts, logs, pivot tables)
- Pivot table content based on conditional expressions
- Outline (navigation) pane content

The types of changes you can make include:

- Delete objects
- Index objects (add a sequential numbering scheme)

- Change the visible property of objects
- Change the outline label text
- Transpose rows and columns in pivot tables
- Change the selected layer of pivot tables
- Change the formatting of selected areas or specific cells in a pivot table based on conditional expressions (for example, make all significance values less than 0.05 bold)

Basic Specification

The basic specification is the command name followed by a SELECT subcommand that contains and one or more subcommands that specify how you want to change the output. For example:

```
OUTPUT MODIFY
  /SELECT TABLES
  /IF SUBTYPES=["Frequencies"]
  /TABLECELLS SELECT=["Total"]
    SELECTDIMENSION=ROWS
    STYLE=BOLD APPLYTO=ROW.
```

- The SELECT subcommand specifies that the command applies only to pivot tables.
- The IF subcommand specifies that the command applies only to the table subtype "Frequencies".
- The TABLECELLS subcommand specifies that the font style of the row that contains the text "Total" should be bold.

Syntax Rules

- All subcommands except SELECT are optional.
- If included, the REPORT subcommand must be the first subcommand.
- Multiple blocks of specifications for different conditions can be specified. Each block must begin with a SELECT subcommand. For more information, see the "Basic Operations" topic.
- Subcommands other than SELECT can appear in any order.

Basic Operation

- OUTPUT MODIFY operates on all applicable output objects that exist in the active Viewer before the OUTPUT MODIFY command runs.
- OUTPUT MODIFYmust come after any commands that produce output to which you want OUTPUT MODIFY to apply.
- The effects of multiple SELECT blocks within a single OUTPUT MODIFY command and the effects of multiple OUTPUT MODIFY commands are both cumulative. The main difference between these approaches is that subsequent OUTPUT MODIFY commands can apply to output objects created by previous OUTPUT MODIFY commands. For example, if the first OUTPUT MODIFY command includes a REPORT subcommand, and a second OUTPUT MODIFY command that immediately follows it includes IF COMMANDS=[LAST], the changes that are specified on the second OUTPUT MODIFY command are applied to the report output created by the first OUTPUT MODIFY command, not the last command before both OUTPUT MODIFY commands.

NAME Keyword

The optional NAME keyword specifies the Viewer document you want to modify. This is useful when more than one Viewer document is open and you want to ensure that the command operates on the correct document.

NAME=*
The currently active output document.

NAME=ALL
All open output documents.

NAME=name
This is the name assigned by a previous OUTPUT NAME, OUTPUT OPEN, OUTPUT NEW command, or automatically assigned when the document is created or opened in the current session; it is not the file name associated with a saved output document. In the Viewer window title bar, the name is enclosed in square brackets.

SELECT Subcommand

The SELECT subcommand specifies the types of output objects that you want to modify.

CHARTS
Chart objects. This object type includes charts that are created by commands such as GRAPH and GGRAPH and charts that are created by statistical procedures (for example, the BARCHART subcommand of the FREQUENCIES command). It does not include tree diagrams that are produced by the TREE procedure or charts in models that are displayed in the Model Viewer.

HEADINGS
Text objects that are labeled "Title" in the outline pane of the Viewer.

LOGS
Log text objects. Log objects contain certain types of error and warning messages. With SET PRINTBACK=ON, log objects also contain the command syntax run during the session. Log objects are labeled "Log" in the outline pane of the Viewer.

MODELS
Output objects that are displayed in the Model Viewer. A single model object can contain multiple views of the model, including both tables and charts.

TABLES
Output objects that are pivot tables in the Viewer. This category does not include Notes tables or tables displayed in model viewers.

TEXTS
Text objects that are not logs or headings. This category includes objects that are labeled "Text Output" in the outline pane of the Viewer.

TREES
Tree model diagrams that are produced by the TREE procedure (Forecasting and Decision Trees).

WARNINGS
Warnings objects. Warnings objects contain certain types of error and warning messages.

OUTLINEHEADERS
Headers in the outline pane that have no corresponding object in the contents pane. For example, output objects that are created by a statistical procedure are nested under the procedure name in the outline pane. The procedure name is the header that has no corresponding object in the contents pane.

PAGETITLE
Page titles that are created by the TITLE or SUBTITLE command.

NOTES
Notes tables that are created by statistical and charting procedures. The Notes table contains information about the dataset that was used, missing values, the command syntax that was used to run the procedure, and the computer resources that were used to run the procedure.

UNKNOWN
IBM SPSS Statistics objects that do not have a known type. If new object types are available in future releases, these objects will be treated as "unknown" in earlier releases.

OTHER
Objects that are created in other applications and pasted into the Viewer, such as images.

ALL
All output objects. If you specify ALL, any other output type keywords are ignored. You can use EXCEPT with ALL to exclude specific output types. The exception list must be enclosed in parentheses.

Example

```
OUTPUT MODIFY
  /SELECT ALL EXCEPT (TABLES)
  /DELETEOBJECT DELETE = YES.
```

In this example, all output objects except pivot tables are deleted from the Viewer.

IF Subcommand

The IF subcommand specifies particular output objects of the types determined by SELECT. Without an IF subcommand, all objects of the specified types are selected. If you specify multiple conditions, only those objects that meet all conditions are selected.

COMMANDS Keyword

The COMMANDS keyword restricts the selection to the specified commands. The keyword COMMANDS must be followed by an equals sign (=) and a list of quoted command identifiers enclosed in square bracket, as in:

```
OUTPUT MODIFY
 /SELECT TABLES
 /IF COMMANDS = ['Frequencies' 'Factor Analysis']
  ...
```

The command identifier can be followed by a value in parentheses. The value can be an integer that indicates the instance of the command or the keyword LAST. The keyword LAST can be used by itself to specify the output of the last command before OUTPUT MODIFY. For example:

```
OUTPUT MODIFY
  /SELECT TABLES
  /IF COMMANDS=['Frequencies(2)', 'Descriptives(LAST)', LAST]
  ...
```

In this example, the specified changes are applied to:

- The second instance of the FREQUENCIES command.
- The last instance of the DESCRIPTIVES command.
- The last command before the OUTPUT MODIFY command, if that command contains pivot table output.

Command identifiers are:

- Unique. No two commands have the same identifier.
- Not case-sensitive.
- Not subject to translation, which means they are the same for all language versions and output languages.
- Often not the same or even similar to the command name. You can obtain the identifier for a particular command by generating output from the command in the Viewer and then right-clicking the command heading in the outline pane and selecting **Copy OMS Command Identifier** from the menu.

Command identifiers are available for all statistical and charting procedures and any other commands that produce blocks of output with their own identifiable heading in the outline pane of the Viewer. For example, CASESTOVARS and VARSTOCASES have corresponding identifiers ('Cases to Variables' and 'Variables to Cases') because they produce their own output blocks (with command headings in the outline pane that happen to match the identifiers), but FLIP does not because any output produced by FLIP is included in a generic Log text object.

SUBTYPES Keyword

The SUBTYPES keyword restricts the selection to the specified table types The keyword SUBTYPES must be followed by an equals sign (=) and a list of quoted subtype identifiers enclosed in square bracket, as in:

```
OUTPUT MODIFY
 /SELECT TABLES
 /IF SUBTYPES = ['Descriptive Statistics' 'Coefficients']
 ...
```

- Subtypes apply only to tables that would be displayed as pivot tables in the Viewer.
- Like command identifiers, subtype identifiers are not case-sensitive and are not subject to translation.
- Unlike command identifiers, subtype identifiers are not necessarily unique. For example, multiple commands produce a table with the subtype identifier "Descriptive Statistics," but not all of those tables share the same structure. If you want only a particular table type for a particular command, use both the COMMANDS and SUBTYPES keywords.
- You can obtain the identifier for a particular table by generating output from the command in the Viewer and then right-clicking outline item for the Table in the outline pane of the Viewer and selecting **Copy OMS Table Subtype** from the menu. The identifiers are generally fairly descriptive of the particular table type.

LABELS Keyword

The LABELS keyword selects particular output objects according to the text displayed in the outline pane of the Viewer. The keyword LABELS must be followed by an equals sign (=) and a list of expressions enclosed in square brackets, as in:

```
OUTPUT MODiFY
 /SELECT TABLES
 /IF LABELS = [EXACT('Age'), CONTAINS('Crosstabulation')]
 ...
```

- Each expression must start with a keyword, followed by a quoted value enclosed in parentheses.
- Valid keywords are EXACT, CONTAINS, BEGINS, and ENDS.
- Label values are case-sensitive.

The LABELS keyword is useful for differentiating between multiple graphs or multiple tables of the same type in which the outline text reflects some attribute of the particular output object such as the variable names or labels. There are, however, a number of factors that can affect the label text:

- If split file processing is on, split file group identification is often appended to the label.
- Labels that include information about variables or values are affected by the OVARS and ONUMBERS settings on the SET command.
- Labels are affected by the current output language setting (SET OLANG).

INSTANCES Keyword

The INSTANCES subcommand selects the *n*th instance of an object that matches the other criteria on the IF subcommand within a single instance of a command. The keyword INSTANCES must be followed by an equals sign (=) and a list of positive integers or the keyword LAST enclosed in square brackets.

Example

```
OUTPUT MODIFY
 /SELECT TABLES
 /IF COMMANDS = ['Frequencies']
     SUBTYPES = ['Frequencies']
     INSTANCES = [1 LAST]
 ...
OUTPUT MODIFY
 /SELECT TABLES
 /IF COMMANDS = ['Frequencies']
     INSTANCES = [1 LAST]
 ...
```

- The first command selects the first and last frequency tables from each FREQUENCIES command.

- The second command, in the absence of a SUBTYPES or LABELS specification, selects the first and last tables of any kind from the selected command.

VISIBLEONLY Keyword

The VISIBLEONLY keyword indicates if the changes should be applied to all output objects that meet the selection criteria or only those objects that meet the criteria and are currently visible in the Viewer document (objects can be present but hidden).

VISIBLEONLY=NO
Changes are applied to both visible and hidden objects.

VISIBLE=YES
Changes are applied only to visible objects.

DELETEOBJECT Subcommand

The DELETEOBJECT subcommand deletes all objects that meet the selection criteria.

DELETE=NO
The objects that meet the selection criteria are not deleted. This option is the default.

DELETE=YES
The objects that meet the selection criteria are deleted.

Example

```
OUTPUT MODIFY
  /SELECT ALL EXCEPT (TABLES)
  /DELETEOBJECT DELETE = YES.
```

In this example, all output objects except pivot tables are deleted from the Viewer.

INDEXING Subcommand

The INDEXNG subcommand adds a sequential number, letter, or roman numeral to the objects that meet the selection criteria. Index values are used by the OBJECTPROPERTIES command (OUTLINELABEL keyword) and TABLE subcommand (TABLETITLE keyword).

Example

```
OUTPUT MODIFY
  /SELECT ALL
  /INDEXING
   INDEXTYPE=LETTERLOW
   INDEXSTART='a'
  /OBJECTPROPERTIES
   OUTLINELABEL = ")INDEX - )LABEL".
```

INDEXTYPE Keyword

The INDEXTYPE keyword indicates whether numbers, letters, or roman numerals are used for the index values.

NUMBER
Integers are used to index the objects. This option is the default.

LETTERLOW
Lowercase letters are used to index the objects.

LETTERCAP
Uppercase letters are used to index the objects.

ROMANLOW
Lowercase roman numerals are used to index the objects.

ROMANCAP
Uppercase roman numerals are used to index the objects.

INDEXSTART Keyword

The INDEXSTART keyword specifies the starting index value.

- For INDEXTYPE=NUMBER, the value of INDEXSTART must be an integer. The default is 1.
- For INDEXTYPE=LETTERLOW, the value of INDEXSTART must be a quoted string of one or more lowercase letters. The default is 'a'.
- For INDEXTYPE=LETTERCAP, the value of INDEXSTART must be a quoted string of one or more uppercase letters. The default is 'A'.
- For INDEXTYPE=ROMANLOW, the value of INDEXSTART must be a quoted string of lowercase letters that represents a valid number that is expressed in roman numerals. The default is 'i'.
- For INDEXTYPE=ROMANCAP, the value of INDEXSTART must be a quoted string of uppercase letters that represents a valid number that is expressed in roman numerals. The default is 'I'.

OBJECTPROPERTIES Subcommand

The OBJECTPROPERTIES subcommand modifies the visible property and outline text of all objects that meet the selection criteria. It also modifies the size of charts and tree diagrams.

VISIBLE Keyword

The VISIBLE keyword determines whether the objects that meet the selection criteria are visible or hidden in the Viewer. (Hidden object can be shown by double-clicking the icon next to the object in the outline pane.)

ASIS
Do not change the visible property of the objects. This option is the default.

YES
Make all objects that meet the selection criteria visible.

NO
Hide all objects that meet the selection criteria.

Example

```
OUTPUT MODIFY
  /SELECT ALL EXCEPT (TABLES)
  /OBJECTPROPERTIES VISIBLE=NO.
```

In this example, all output objects except pivot tables are hidden in the Viewer.

OUTLINELABEL Keyword

The OUTLINELABEL keyword modifies the text of outline labels in the outline pane. The keyword is followed by an equals sign and a quoted value. You can also include special variables to insert date, time, and index values.

)DATE
Current date in the form dd-mmm-yyyy.

)ADATE
Current date in the form mm/dd/yyyy.

)SDATE
Current date in the form yyyy/mm/dd.

)EDATE
Current date in the form dd.mm.yyyy.

)TIME
Current 12-hour clock time in the form hh:mm:ss.

)ETIME
Current 24-hour clock time in the form hh:mm:ss.

)INDEX
The index as specified on the INDEXING subcommand.

)LABEL
The current label text. This setting allows you to add other text or special variable values to the existing label.

Example

```
OUTPUT MODIFY
  /SELECT TABLES
  /INDEXING
   INDEXTYPE=NUMBER INDEXSTART=1
  /OBJECTPROPERTIES
   OUTLINELABEL = ")INDEX - )EDATE - )LABEL".
```

In this example, the outline labels for all pivot tables are modified to insert a sequential index number and the current date, separated by dashes, before the current label.

SIZE Keyword

The SIZE keyword modifies the size of charts, tree diagrams, and objects that are created by other applications and pasted into the Viewer document. The keyword is followed by an equals sign, a keyword that specifies measurement unit, and width and height values that are enclosed in parentheses.

- Valid keywords for measurement unit are PERCENT, CENTIMETERS, INCHES, and POINTS..
- The keyword PROPORTIONAL can be used instead of a width or height value. This keyword maintains the aspect ratio of the original image.

Example

```
OUTPUT MODIFY
  /SELECT CHARTS
  /OBJECTPROPERTIES
   SIZE=PERCENT(50,PROPORTIONAL).
```

In this example, the width of all charts is set to 50% of the original width. The keyword PROPORTIONAL sets the height to 50%, which preserves the original aspect ratio.

TABLE Subcommand

The TABLE subcommand modifies properties of pivot tables, notes tables, and warning objects in Viewer (TABLES, NOTES, and WARNINGS keywords on the SELECT subcommand).

TABLETITLE keyword

The TABLETITLE keyword modifies the text of title text in the contents pane of the Viewer. The keyword is followed by an equals sign and a quoted value. You can also include special variables to insert date, time, and index values.

)DATE
Current date in the form dd-mmm-yyyy.

)ADATE
Current date in the form mm/dd/yyyy.

)SDATE
Current date in the form yyyy/mm/dd.

)EDATE
Current date in the form dd.mm.yyyy.

)TIME
Current 12-hour clock time in the form hh:mm:ss.

)ETIME
Current 24-hour clock time in the form hh:mm:ss.

)INDEX
The index as specified on the INDEXING subcommand.

)LABEL
The current label text. This setting allows you to add other text or special variable values to the existing label.

Example

```
OUTPUT MODIFY
  /SELECT TABLES
  /INDEXING
   INDEXTYPE=NUMBER INDEXSTART=1
  /TABLE
   TABLETITLE = ")INDEX - )EDATE - )LABEL".
```

In this example, the titles for all pivot tables are modified to insert a sequential index number and the current date, separated by dashes, before the current label.

TRANSPOSE keyword

The TRANSPOSE keyword transposes rows and columns in pivot tables.

NO
Do not transpose rows and columns. This option is the default.

YES
Transpose rows and columns.

TLOOK keyword

The TLOOK keyword applies the specified TableLook to pivot tables, notes tables, and warning objects.

TLOOK='filespec'
Applies the TableLook defined in the specified STT file. The file specification takes the general form '/path/filename.stt'.

TLOOK='name'
Applies a TableLook from the Looks directory of the application installation directory. The value of 'name' is the file name without the .stt extension.

TLOOK=NONE
Applies the default TableLook.

SORT and SORTDIRECTION keywords

The SORT and SORTDIRECTION keywords sort the contents of pivot tables based on the values in the specified column.

Note: These keywords work only for tables with a simple row dimension structure. These keywords are ignored if there are nested row elements.

SORT=COLLABEL('label text')
Sort based on the values in the column with the specified label.

SORT=COLPOSITION(integer)
Sort based on the values in the *n*th column. Column numbers start with the first row label column. For example, if the table has two row label columns, then the first data column is column 3.

SORTDIRECTION=ASCENDING
Sort in ascending order of values in the specified column. This option is the default.

SORTDIRECTION=DESCENDING
Sort in descending order of values in the specified column.

Example

```
OUTPUT MODIFY
   /SELECT TABLES
   /TABLE  SORT=COLLABEL("Percent")
    SORTDIRECTION=DESCENDING.
```

In this example, pivot tables that have a column labeled "Percent" will be sorted in descending order of the values in that column.

SELECTEDLAYER keyword

The SELECTEDLAYER keyword changes the displayed layer of pivot table that contains more than one layer. The keyword is followed by an equals sign and a pair of values enclosed in square brackets.

- The first value identifies the layer dimension. The value can be a quoted string that contains the label of the dimension as displayed in the table or an integer that represents the numeric position of the layer. The top layer dimension is 1.
- The second value identifies the category within the layer dimension. The value can be a quoted string that contains the label of the category as displayed in the table or an integer that represents the numeric position of the category within the layer dimension.
- For tables with multiple layer dimensions, you can specify multiple, dimension and category values.

Example

```
OUTPUT MODIFY
  /SELECT TABLES
  /TABLE SELECTLAYER=["Marital status", "Married", 2, 3].
```

- For any table with a layer dimension with the layer "Marital status", the category with the label "Married" will be displayed, if that category is present.
- For any table with at least two layer dimensions, the third category of the second layer dimension will be displayed, if that layer dimension has at least three categories.

TABLESUMMARY keyword

The TABLESUMMARY keyword adds comment text to the table.

- Comment text is displayed in a tooltip when you hover over a table in the Viewer.
- Screen readers read the comment text when the table has focus.
- The tooltip in the Viewer displays only the first 200 characters of the comment, but screen readers read the entire text.
- When you export output to HTML, the comment text is used as alt text.

The keyword is followed by an equals sign and a quoted value. You can also include special variables to insert date, time, and other values.

)DATE
Current date in the form dd-mmm-yyyy.

)ADATE
Current date in the form mm/dd/yyyy.

)SDATE
Current date in the form yyyy/mm/dd.

)EDATE
Current date in the form dd.mm.yyyy.

)TIME
Current 12-hour clock time in the form hh:mm:ss.

)ETIME
Current 24-hour clock time in the form hh:mm:ss.

)INDEX
The index as specified on the INDEXING subcommand.

)TITLE
The text of the outline label for the table.

)PROCEDURE
The name of the procedure that created the table.

)DATASET
The name of the dataset used to create the table.

\n
Inserts a line break.

PIVOT keyword

The PIVOT keyword pivots the specified row dimension to the specified column dimension. Any existing column dimensions are incremented outwards. For example, PIVOT=[R1C1] will pivot the first row dimension (the outer-most row dimension) to be the first column dimension. If there are any existing column dimensions, they will be moved outwards so that the old C1 becomes C2. The keyword allows table output to adhere to APA style guidelines.

TABLECELLS Subcommand

The TABLECELLS subcommand specifies areas of tables to change and conditions under which to make those changes.

SELECT keyword

The SELECT keyword specifies the area of the table. The keyword is followed by an equals sign and a list of keywords or quoted strings, which are enclosed in square brackets.

The following keywords apply to general areas of the table:

BODY
The data cells of the tables. The body of the table does not include row and column labels.

HEADERS
All row and column labels. This option includes all labels for nested rows and columns.

FOOTNOTES
All footnote text.

TITLE
The title that is displayed above the table.

CAPTION
All caption text.

The following keywords apply to specific conditions:

COUNT
Rows or columns with any of these labels or the equivalent in the current output language: "Frequency", "Count", "N".

MEAN
Rows or columns with the label "Mean" or the equivalent in the current output language.

MEDIAN
Rows or columns with the label "Median" or the equivalent in the current output language.

RESIDUAL
Rows or columns with any of these labels or the equivalent in the current output language: "Resid", "Residual", "Std. Residual".

PERCENT
Rows or columns with the label "Percent" or the equivalent in the current output language.

SIGNIFICANCE
Rows or columns with any of these labels or the equivalents in the current output language: "Approx Sig.", "Asymp. Sig (2-sided)", "Exact Sig.", "Exact Sig. (1-sided)", "Exact Sig. (2-sided)", "Sig.", "Sig. (1-tailed)", "Sig. (2-tailed)"

VALID
Rows or columns with the label "Valid" or the equivalent in the current output language.

TOTAL
Rows or columns with the label "Total" or the equivalent in the current output language.

MISSING
Rows or columns with the label "Missing" or the equivalent in the current output language.

CUMULATIVEPERCENT
Rows or columns with the label "Cumulative Percent" or the equivalent in the current output language.

VALIDPERCENT
Rows or columns with the label "Valid Percent" or the equivalent in the current output language.

CORRELATION
Rows or columns with any of these labels or the equivalent in the current output language: "Correlation Coefficient", "Correlations", "Pearson Correlation".

POSITION(integer)
Data row or column position. (The SELECTDIMENSION keyword specifies either rows of columns.) This does not include row label columns or column label rows.

"expression"
A quoted string that represents the value of a row or column label.

SELECTDIMENSION keyword

The SELECTDIMENSION keyword specifies whether SELECT applies to columns or rows or both.

- The keyword is followed by an equals sign and the keyword COLUMNS, ROWS, or BOTH. The default option is BOTH.
- The keyword is ignored for SELECT conditions BODY, HEADERS, FOOTERNOTES, TITLE, and CAPTION.

SELECTCONDITION keyword

The SELECTCONDITION keyword specifies the conditions under which to apply the changes within the area of the table specified by SELECT and SELECTDIMENSION. The keyword is followed by an equals sign and a list of keywords or quoted strings, which are enclosed in square brackets.

ALL
All conditions. This option is the default. If ALL is specified with any other conditions, the other conditions are ignored.

TOP(integer)
The *n* highest values in the specified area.

BOTTOM(integer)
The *n* lowest values in the specified area.

PARENT(expression)
The primary expression value in the specified area. The value can be a quoted string, **MISSING**, or **VALID**.

CHILD(expression)
The secondary expression value in the specified area. The value can be a quoted string, **MISSING**, or **VALID**.

"expression"
A quoted string that contains a comparison expression.

- The expression can include standard comparison operators (<, <=, >, >=, <>) and the keyword ABS for absolute value, followed by an expression in parentheses.
- The keyword SYSMIS indicates the system-missing value (for numeric values only).
- A lower case "x" is used as a stand-in variable to represent all values in the area of the table specified by the SELECT and SELECTDIMENSION keywords. For example: "x<=.05".
- If the expression contains string values, those values must be quoted.

COLUMNWIDTH keyword

The COLUMNWIDTH keyword specifies the width of the column or columns in the specified area of the table. The keyword is followed by an equals sign and the keyword CENTIMETERS, INCHES, or POINTS, followed by a positive number that is enclosed in parentheses.

APPLYTO keyword

The APPLYTO Keyword specifies what areas of the table to change based on the selection criteria.

- APPLYTO can apply changes to elements of the table that are not included in the selection criteria. For example, if the selection criteria specifies specific values in a specific column, APPLYTO=ROW applies the changes to entire rows that contain the column values that meet the selection criteria.
- APPLYTO is ignored for BODY, HEADERS, FOOTNOTE, TITLE, and CAPTION.

CELL
Applies changes only to data cells that meet the selection criteria.

COLUMN
Applies changes to entire columns that contain data values or an innermost column label that meet the selection criteria.

ROW
Applies changes to entire rows that contain data values or an innermost row label that meet the selection criteria.

COLUMNDATA
Applies changes to all data cells in the columns that contain data values that meet the selection criteria.

ROWDATA
Applies changes to all data cells in the rows that contain data values that meet the selection criteria.

COLUMNHEADER
Applies changes to the innermost column labels that meet the selection criteria.

ROWHEADER
Applies changes to the innermost row labels that meet the selection criteria.

REVERTTODEFAULT keyword

The REVERTTODEFAULT keyword removes any special formats or styles that have been applied to a table and reverts to the original Tablelook style.

- The keyword is followed by an equals sign and the keyword YES or NO. The default option is NO.
- When this keyword is used with other TABLECELLS format or style changes, REVERTTODEFAULT will undo any changes that are specified after it. Specify REVERTODEFAULT first if you want to make other changes after reverting to the default TableLook.

FORMAT keyword

The FORMAT keyword specifies the format for data values. The keyword is followed by an equals sign and a valid format specification, such as F10.4, PCT5.1, or A20. For detailed information about available formats and specifications, see "Variable Types and Formats" on page 46

FONT keyword

The FONT keyword specifies the font to use.

- The keyword is followed by an equals sign and the name of a font that is enclosed in quotation marks.
- If the specified font is not available, this setting is ignored.

FONTSIZE keyword

The FONTSIZE keyword specifies the font size (in points). The keyword is followed by an equals sign and a positive number.

STYLE keyword

The STYLE keyword specifies the font style.

- The keyword is followed by an equals sign and a list of styles.
- Valid style values are REGULAR, BOLD, ITALIC, and UNDERLINE.
- Style specifications are cumulative. If the text is already italic, the BOLD keyword makes the text italic and bold.
- The REGULAR keyword removes any special style characteristics. When used with other style keywords, it must be specified first; otherwise any previous style specifications are removed. For example, STYLE=REGULAR BOLD results in bold text without any other special style characteristics, but STYLE=BOLD REGULAR results in text that is not bold.

TEXTCOLOR and BACKGROUNDCOLOR keywords

The TEXTCOLOR keyword specifies the text color. The BACKGROUNDCOLOR keyword specifies the background color.

- The keyword is followed by an equals sign and a color keyword or RGB(r, g, b), where r, g, and b are numeric red, green, and blue color values. The numeric values are integers between 1 and 255.
- Valid color keywords are BLACK, BLUE, CYAN, DARKGRAY, GRAY, GREEN, LIGHTGRAY, MAGENTA, ORANGE, PINK, RED, WHITE, YELLOW.

REPLACE keyword

The REPLACE keyword replaces any text that meets the selection criteria with the specified number or quoted string value.

HIDE keyword

When set to YES, the HIDE keyword suppresses the selected row or column. The keyword allows table output to adhere to APA style guidelines.

UNGROUP keyword

When set to YES, the UNGROUP keyword deletes the selected row or column group header. The keyword allows table output to adhere to APA style guidelines.

Example

```
OUTPUT MODIFY
  /SELECT TABLES /IF SUBTYPES =['Frequencies']
  /TABLECELLS  SELECT=["Percent"]
   SELECTDIMENSION=COLUMNS SELECTCONDITION=["x>=30"]
   APPLYTO=CELL STYLE=REGULAR BOLD
  /SELECT TABLES /IF SUBTYPES=['Frequencies']
  /TABLECELLS SELECT=["Cumulative Percent"]
   SELECTDIMENSION=COLUMNS SELECTCONDITION=["x>=50"]
   APPLYTO=ROW BACKGROUNDCOLOR=GREEN
  /SELECT TABLES /IF SUBTYPES=["Frequencies"]
  /TABLECELLS SELECT=["Total"]
   SELECTDIMENSION=ROWS APPLYTO=ROW
   REVERTTODEFAULT=YES.
```

Do you agree?

		Frequency	Percent	Valid Percent	Cumulative Percent
Valid	Strongly Disagree	15	15.0	15.0	15.0
	Disagree	10	10.0	10.0	25.0
	Let me think about it	30	**30.0**	30.0	55.0
	Agree	10	10.0	10.0	65.0
	Strongly Agree	35	**35.0**	35.0	100.0
	Total	100	100.0	100.0	

Figure 80. Table modified by TABLECELLS subcommands

- The TABLESCELLS subcommand in the first SELECT block finds values greater than or equal to 30 in the column labeled "Percent" and makes them bold.
- The TABLECELLS subcommand in the second SELECT block finds values greater than or equal to 50 in the column labeled "Cumulative Percent" and makes the background color of those rows green.
- Since the first two TABLECELLS subcommands also change the total row at the bottom of the table, the last TABLECELLS subcommand uses REVERTTODEFAULT to remove any special formatting that was applied to the total row.

GRAPHS Subcommand

The GRAPHS subcommand applies a template or style sheet to charts that meet the selection criteria.

CTEMPLATE=["filespec", "filespec"...] | [name, name, ...]
Applies the specified chart template.

- Chart templates can be applied to most charts, except charts that are created with the Graphboard Template Chooser (GGRAPH command,VIZSTYLESHEET keyword).
- CTEMPLATE=["filespec"] applies the chart template that is defined in the specified SGT file. The file specification takes the general form '/path/filename.sgt'

- CTEMPLATE=[name] applies a chart template from the Looks directory of the application installation directory. The value of 'name' is the file name without the .sgt extension.
- You can specify more than one chart template. Template attributes are applied in the order the templates are specified.

VIZSTYLESHEET=name
Applies the specified style sheet.

- Style sheets apply only to charts created with the Graphboard Template Chooser (GGRAPH command,VIZSTYLESHEET keyword).
- For a list of valid names, go to the Options tab of the Graphboard Template Chooser and click **Select**.

TEXTS Subcommand

The TEXTS subcommand applies changes to text objects that meet the selection criteria. This includes HEADINGS, LOGS, TEXTS, WARNINGS, and PAGETITLES specified on the SELECT subcommand.

TEXT Keyword

The TEXT keyword modifies the text of text objects. The keyword is followed by an equals sign and a quoted value. You can also include special variables to insert date, time, and index values.

)DATE
Current date in the form dd-mmm-yyyy.

)ADATE
Current date in the form mm/dd/yyyy.

)SDATE
Current date in the form yyyy/mm/dd.

)EDATE
Current date in the form dd.mm.yyyy.

)TIME
Current 12-hour clock time in the form hh:mm:ss.

)ETIME
Current 24-hour clock time in the form hh:mm:ss.

)INDEX
The index as specified on the INDEXING subcommand.

)LABEL
The current text of the text object. This setting allows you to add other text or special variable values to the existing text. Without)LABEL, the original text is overwritten.

Example

```
OUTPUT MODIFY
  /SELECT LOGS
  /TEXTS
   TEXT=")EDATE  )LABEL".
```

In this example, the current date is added to all log objects.

FONT Keyword

The FONT keyword specifies the font to use.

- The keyword is followed by an equals sign and the name of a font that is enclosed in quotation marks.
- If the specified font is not available, this setting is ignored.

FONTSIZE Keyword

The FONTSIZE keyword specifies the font size (in points). The keyword is followed by an equals sign and a positive number.

STYLE Keyword

The STYLE keyword specifies the font style.

- The keyword is followed by an equals sign and a list of styles.
- Valid style values are REGULAR, BOLD, ITALIC, and UNDERLINE.
- Style specifications are cumulative. If the text is already italic, the BOLD keyword makes the text italic and bold.
- The REGULAR keyword removes any special style characteristics. When used with other style keywords, it must be specified first; otherwise any previous style specifications are removed. For example, STYLE=REGULAR BOLD results in bold text without any other special style characteristics, but STYLE=BOLD REGULAR results in text that is not bold.

TEXTCOLOR Keyword

The TEXTCOLOR keyword specifies the text color.

- The keyword is followed by an equals sign and a color keyword or RGB(r, g, b), where r, g, and b are numeric red, green, and blue color values. The numeric values are integers between 1 and 255.
- Valid color keywords are BLACK, BLUE, CYAN, DARKGRAY, GRAY, GREEN, LIGHTGRAY, MAGENTA, ORANGE, PINK, RED, WHITE, YELLOW.

REPORT Subcommand

The REPORT subcommand produces a table that summarizes the results of the OUTPUT MODIFY command. If included, this subcommand must be the first subcommand.

PRINTREPORT=NO
Do not include a summary report table. This option is the default.

PRINTREPORT=YES
Include a summary report table.

OUTPUT NAME

```
OUTPUT NAME [NAME]=name
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 15.0

- Command introduced.

Example

```
GET FILE='/examples/data/SalaryData2005.sav'.
DESCRIPTIVES salbegin salary.
OUTPUT NAME alleduclevels.

TEMPORARY.
SELECT IF (educ>12).
OUTPUT NEW NAME=over12.
DESCRIPTIVES salbegin salary.

GET FILE='/examples/data/SalaryData2000.sav'.
TEMPORARY.
SELECT IF (educ>12).
DESCRIPTIVES salbegin salary.

OUTPUT ACTIVATE alleduclevels.
DESCRIPTIVES salbegin salary.
```

Overview

The OUTPUT commands (OUTPUT NEW, OUTPUT NAME, OUTPUT ACTIVATE, OUTPUT OPEN, OUTPUT SAVE, OUTPUT CLOSE) provide the ability to programmatically manage one or many output documents. These functions allow you to:

- Save an output document through syntax.
- Programmatically partition output into separate output documents (for example, results for males in one output document and results for females in a separate one).
- Work with multiple open output documents in a given session, selectively appending new results to the appropriate document.

The OUTPUT NAME command assigns a name to the designated output document. The designated output document is the one most recently opened (by OUTPUT NEW or OUTPUT OPEN) or activated (by OUTPUT ACTIVATE). The document name is used to reference the document in any subsequent OUTPUT ACTIVATE, OUTPUT SAVE, and OUTPUT CLOSE commands.

Basic Specification

The basic specification for OUTPUT NAME is the command name followed by a name that conforms to variable naming rules. See the topic “Variable Names” on page 42 for more information. The NAME keyword is optional, but if it is used it must be followed by an equals sign.

Operations

- The association with the existing name is broken, and the new name is assigned to the document.
- If the specified name is associated with another document, that association is broken and the name is associated with the designated output document. The document previously associated with the specified name is assigned a new unique name.

Example

```
GET FILE='/examples/data/SurveyData.sav'.

TEMPORARY.
```

```
SELECT IF (Sex='Male').
FREQUENCIES VARIABLES=ALL.
OUTPUT NAME males.

TEMPORARY.
SELECT IF (Sex='Female').
OUTPUT NEW NAME=females.
FREQUENCIES VARIABLES=ALL.

GET FILE='/examples/data/Preference.sav'.

TEMPORARY.
SELECT IF (Sex='Female').
DESCRIPTIVES VARIABLES=product1 product2 product3.

TEMPORARY.
SELECT IF (Sex='Male').
OUTPUT ACTIVATE males.
DESCRIPTIVES VARIABLES=product1 product2 product3.

OUTPUT SAVE NAME=males OUTFILE='/examples/output/Males.spv'.
OUTPUT SAVE NAME=females OUTFILE='/examples/output/Females.spv'.
```

- The first GET command loads survey data for males and females.
- FREQUENCIES output for male respondents is written to the designated output document. The OUTPUT NAME command is used to assign the name *males* to the designated output document.
- FREQUENCIES output for female respondents is written to a new output document named *females*.
- The second GET command loads preferences data for males and females.
- Descriptive statistics for females are appended to the output document named *females* and those for males are appended to the output document named *males*. Each output document now contains both survey and preferences results.
- The two open output documents are saved to separate files. Because the operation of saving an output document does not close it, both documents remain open. The output document named *males* remains the designated output document.

OUTPUT NEW

```
OUTPUT NEW [NAME=name]
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 15.0

- Command introduced.

Release 16.0

- TYPE keyword is obsolete and is ignored.

Example

```
GET FILE='/examples/data/Males.sav'.
FREQUENCIES VARIABLES=ALL.
OUTPUT SAVE OUTFILE='/examples/output/Males.spv'.
OUTPUT NEW.

GET FILE='/examples/data/Females.sav'.
FREQUENCIES VARIABLES=ALL.
OUTPUT SAVE OUTFILE='/examples/output/Females.spv'.
```

Overview

The OUTPUT commands (OUTPUT NEW, OUTPUT NAME, OUTPUT ACTIVATE, OUTPUT OPEN, OUTPUT SAVE, OUTPUT CLOSE) provide the ability to programmatically manage one or many output documents. These functions allow you to:

- Save an output document through syntax.
- Programmatically partition output into separate output documents (for example, results for males in one output document and results for females in a separate one).
- Work with multiple open output documents in a given session, selectively appending new results to the appropriate document.

The OUTPUT NEW command creates a new output document, which becomes the designated output document. Subsequent procedure output is directed to the new output document until the document is closed or another output document is created, opened, or activated.

Basic Specification

The basic specification for OUTPUT NEW is simply the command name.

TYPE Keyword

This keyword is obsolete and is ignored. The only valid output type is Viewer. Draft Viewer format is no longer supported. To produce text output equivalent to Draft Viewer output use OMS. See the topic “OMS” on page 1301 for more information.

NAME Keyword

By default, the newly created output document is provided with a unique name. You can optionally specify a custom name for the output document, overriding the default name. The document name is used to reference the document in any subsequent OUTPUT ACTIVATE, OUTPUT SAVE, and OUTPUT CLOSE commands.

- The specified name must conform to variable naming rules. See the topic “Variable Names” on page 42 for more information.

- If the specified name is associated with another document, that association is broken and the name is associated with the new document. The document previously associated with the specified name is assigned a new unique name.

Syntax Rules

- An error occurs if a keyword is specified more than once.
- Keywords must be spelled in full.
- Equals signs (=) used in the syntax chart are required elements.

Operations

The new output document is opened in a window in the user interface and becomes the designated output window.

Limitations

Because each window requires a minimum amount of memory, there is a limit to the number of windows, IBM SPSS Statistics or otherwise, that can be concurrently open on a given system. The particular number depends on the specifications of your system and may be independent of total memory due to OS constraints.

Example

```
GET FILE='/examples/data/Males.sav'.
FREQUENCIES VARIABLES=ALL.
OUTPUT SAVE OUTFILE='/examples/output/Males.spv'.
OUTPUT NEW.

GET FILE='/examples/data/Females.sav'.
FREQUENCIES VARIABLES=ALL.
OUTPUT SAVE OUTFILE='/examples/output/Females.spv'.
```

- FREQUENCIES produces summary statistics for each variable in */examples/data/Males.sav*. The output from FREQUENCIES is added to the designated output document (one is created automatically if no output document is currently open).
- OUTPUT SAVE writes the contents of the designated output document to */examples/output/Males.spv*.
- OUTPUT NEW creates a new Viewer document, which becomes the designated output document.
- The subsequent FREQUENCIES command produces output for females using the data in */examples/data/Females.sav*. OUTPUT SAVE writes this output to */examples/output/Females.spv*.

As shown in this example, OUTPUT NEW allows you to direct results to an new output document. If OUTPUT NEW were not specified, */examples/output/Females.spv* would contain frequencies for both males and females.

OUTPUT OPEN

```
OUTPUT OPEN FILE='file specification' [NAME=name]
   [LOCK={YES**}] [PASSWORD='password']
         {NO   }
```

** Default if the subcommand or keyword is omitted.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic "Command Order" on page 36 for more information.

Release History

Release 15.0

- Command introduced.

Release 17.0

- LOCK keyword introduced.

Release 21.0

- PASSWORD keyword introduced.

Example

```
OUTPUT OPEN FILE='/examples/output/Q1Output.spv'.
GET FILE='/examples/data/March.sav'.
FREQUENCIES VARIABLES=ALL.
OUTPUT SAVE OUTFILE='/examples/output/Q1Output.spv'.
```

Overview

The OUTPUT commands (OUTPUT NEW, OUTPUT NAME, OUTPUT ACTIVATE, OUTPUT OPEN, OUTPUT SAVE, OUTPUT CLOSE) provide the ability to programmatically manage one or many output documents. These functions allow you to:

- Save an output document through syntax.
- Programmatically partition output into separate output documents (for example, results for males in one output document and results for females in a separate one).
- Work with multiple open output documents in a given session, selectively appending new results to the appropriate document.

The OUTPUT OPEN command opens a Viewer document, which becomes the designated output document. You can use OUTPUT OPEN to append output to an existing output document. Once opened, subsequent procedure output is directed to the document until it is closed or until another output document is created, opened, or activated.

Basic Specification

The basic specification for OUTPUT OPEN is the command name followed by a file specification for the file to open.

NAME Keyword

By default, the newly opened output document is provided with a unique name. You can optionally specify a custom name for the output document, overriding the default name. The document name is used to reference the document in any subsequent OUTPUT ACTIVATE, OUTPUT SAVE, and OUTPUT CLOSE commands.

- The specified name must conform to variable naming rules. See the topic "Variable Names" on page 42 for more information.

- If the specified name is associated with another document, that association is broken and the name is associated with the newly opened document. The document previously associated with the specified name is assigned a new unique name.

LOCK Keyword

The LOCK keyword only applies when opening a file from a IBM SPSS Collaboration and Deployment Services Repository. It specifies whether the file is locked (in the repository) when opened. Locking a file prevents other users from modifying it while you are working with it. Other users can open locked files but they cannot save them back to the repository. The lock is released when the file is closed or when the current session ends.

- By default, files are opened as locked.
- The LOCK keyword is ignored if the specified file does not reside in a IBM SPSS Collaboration and Deployment Services Repository.

See the topic "File Specifications for IBM SPSS Collaboration and Deployment Services Repository Objects" on page 2217 for more information.

PASSWORD Keyword

The PASSWORD keyword specifies the password required to open an encrypted IBM SPSS Statistics output file. The specified value must be enclosed in quotation marks and can be provided as encrypted or as plain text. Encrypted passwords are created when pasting command syntax from the Save Output As dialog. The PASSWORD keyword is ignored if the file is not encrypted.

Syntax Rules

- An error occurs if a keyword is specified more than once.
- Keywords must be spelled in full.
- Equals signs (=) used in the syntax chart are required elements.

Operations

- The output document is opened in a window in the user interface and becomes the designated output window.
- An error occurs, but processing continues, if the specified file is not found. Output continues to be directed to the last designated output document.
- An error occurs, but processing continues, if the specified file is not a Viewer document. Output continues to be directed to the last designated output document.
- OUTPUT OPEN honors file handles and changes to the working directory made with the CD command.

Limitations

- Because each window requires a minimum amount of memory, there is a limit to the number of windows, IBM SPSS Statistics or otherwise, that can be concurrently open on a given system. The particular number depends on the specifications of your system and may be independent of total memory due to OS constraints.
- OUTPUT OPEN is not supported in the IBM SPSS Statistics Batch Facility (available with IBM SPSS Statistics Server).

Example

```
OUTPUT OPEN FILE='/examples/output/Q1Output.spv'.
GET FILE='/examples/data/March.sav'.
FREQUENCIES VARIABLES=ALL.
OUTPUT SAVE OUTFILE='/examples/output/Q1Output.spv'.
```

- OUTPUT OPEN opens the Viewer document */examples/output/Q1Output.spv*. The document contains summaries for the months of January and February.
- The GET command opens a file containing data for the month of March.
- The FREQUENCIES command produces summaries for March data, which are appended to the designated output document.

- OUTPUT SAVE saves the designated output document to */examples/output/Q1Output.spv*. The saved document contains results for each of the three months in the first quarter.

OUTPUT SAVE

```
OUTPUT SAVE [NAME={*   }]
                 {name}
  OUTFILE='file specification'
  [TYPE={SPV**}]
  [LOCK={NO**}]
        {YES }
  [STOREMODEL={NO   }]
              {YES**}

  [/PASSPROTECT PASSWORD={'password'}  [ENCRYPTEDPW={NO**}] ]
                        {NONE**    }               {YES }
```

** Default if the keyword is omitted.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 15.0

- Command introduced.

Release 16.0

- TYPE keyword introduced.

Release 17.0

- LOCK keyword introduced.

Release 21.0

- PASSPROTECT subcommand introduced.

Release 23.0

- STOREMODEL keyword introduced.

Release 27.0

- TYPE keyword's **SPW** option was deprecated.

Example

```
OUTPUT OPEN FILE='/examples/output/Q1Output.spv'.
GET FILE='/examples/data/March.sav'.
FREQUENCIES VARIABLES=ALL.
OUTPUT SAVE OUTFILE='/examples/output/Q1Output.spv'.
```

Overview

The OUTPUT commands (OUTPUT NEW, OUTPUT NAME, OUTPUT ACTIVATE, OUTPUT OPEN, OUTPUT SAVE, OUTPUT CLOSE) provide the ability to programmatically manage one or many output documents. These functions allow you to:

- Save an output document through syntax.
- Programmatically partition output into separate output documents (for example, results for males in one output document and results for females in a separate one).
- Work with multiple open output documents in a given session, selectively appending new results to the appropriate document.

The OUTPUT SAVE command saves the contents of an open output document to a file.

Basic specification

The basic specification for OUTPUT SAVE is the command name followed by a file specification for the destination file.

NAME keyword

Use the NAME keyword to save an output document other than the designated one. Provide the name associated with the document.

TYPE keyword

Use the TYPE keyword to specify the format of the output file--SPV for standard output files.

LOCK keyword

Use LOCK=YES to prevent editing of Viewer documents in IBM SPSS Smartreader (a separate product for working with Viewer documents). If a Viewer document is locked, you can manipulate pivot tables (swap rows and columns, change the displayed layer, etc.) but you cannot edit any output or save any changes to the Viewer document in IBM SPSS Smartreader. This setting has no effect on Viewer documents opened in IBM SPSS Statistics. The default is LOCK=NO.

STOREMODEL keyword

The STOREMODEL keyword applies only when there are model viewer items in the output document that require auxiliary information to enable some of the interactive features. The default is STOREMODEL=YES, which specifies to store the auxiliary information with the output document. Storing this information with the output document might substantially increase the document size. If you choose not to store this information, you can still open these output items but the specified interactive features will not be available. You can view the affected model viewer items and interactive features by clicking **More Info** on the Save Output As dialog.

Syntax rules

- An error occurs if a keyword is specified more than once.
- Keywords must be spelled in full.
- Equals signs (=) used in the syntax chart are required elements.

Operations

- By default, the designated output document is saved. The designated output document is the one most recently opened (by OUTPUT NEW or OUTPUT OPEN) or activated (by OUTPUT ACTIVATE).
- If the specified file already exists, OUTPUT SAVE overwrites it without warning.
- An error occurs if you specify a nonexistent output document.
- An error occurs if the file specification is invalid.
- OUTPUT SAVE saves the document but does not close it. Use OUTPUT CLOSE to close the document.
- OUTPUT SAVE honors file handles and changes to the working directory made with the CD command.

Limitations

- OUTPUT SAVE is not supported in the IBM SPSS Statistics Batch Facility (available with IBM SPSS Statistics Server).

Example

```
OUTPUT OPEN FILE='/examples/output/Q1Output.spv'.
GET FILE='/examples/data/March.sav'.
FREQUENCIES VARIABLES=ALL.
OUTPUT SAVE OUTFILE='/examples/output/Q1Output.spv'.
```

- OUTPUT OPEN opens the Viewer document */examples/output/Q1Output.spv*. The document contains summaries for the months of January and February.
- GET opens a file containing new data for March.
- FREQUENCIES produces frequencies for March data, which are appended to the designated output document.
- OUTPUT SAVE saves the contents of the designated output document to */examples/output/Q1Output.spv*, which now contains results for the entire first quarter.

PASSPROTECT Subcommand

The PASSPROTECT subcommand specifies the settings for saving an output document as an encrypted file. Once encrypted, the file can only be opened by providing the correct password. IBM SPSS Smartreader users will also be required to provide the password in order to open the file.

PASSWORD. Specifies the password required to open this file. Passwords are limited to 10 characters and are case-sensitive. The specified value must be enclosed in quotation marks. The keyword NONE specifies that the file will not be encrypted and is the default.

Warning: Passwords cannot be recovered if they are lost. If the password is lost the file cannot be opened.

ENCRYPTEDPW Keyword

The ENCRYPTEDPW keyword specifies whether the password is encrypted.

NO. The password is not encrypted. It is treated as plain text. This is the default.

YES. The password is encrypted. Passwords are always encrypted when pasting syntax from the Save Output As dialog. *Note*: ENCRYPTEDPW=YES should only be used when the password is known to be encrypted.

Creating strong passwords

- Use eight or more characters.
- Include numbers, symbols and even punctuation in your password.
- Avoid sequences of numbers or characters, such as "123" and "abc", and avoid repetition, such as "111aaa".
- Do not create passwords that use personal information such as birthdays or nicknames.
- Periodically change the password.

Note: Encrypted data files and output documents cannot be opened in versions of IBM SPSS Statistics prior to version 21. Encrypted syntax files cannot be opened in versions prior to version 22.

OVERALS

OVERALS is available in SPSS Statistics Professional Edition or the Categories option.

```
OVERALS VARIABLES=varlist (max)

/ANALYSIS=varlist[({ORDI**})]
                  {SNOM  }
                  {MNOM  }
                  {NUME  }

/SETS= n (# of vars in set 1, ..., # of vars in set n)

[/NOBSERVATIONS=value]

[/DIMENSION={2**  }]
            {value}

[/INITIAL={NUMERICAL**}]
          {RANDOM     }

[/MAXITER={100**}]
          {value}

[/CONVERGENCE={.00001**}]
              {value   }

[/PRINT=[DEFAULT] [FREQ**] [QUANT] [CENTROID**]
        [HISTORY] [WEIGHTS**]
        [OBJECT] [FIT] [NONE]]

[/PLOT=[NDIM=({1    ,2    }**)]
              {value,value}
              {ALL  ,MAX  }
       [DEFAULT[(n)]] [OBJECT**[(varlist)][(n)]]
       [QUANT[(varlist)][(n)]] [LOADINGS**[(n)]]
       [TRANS[(varlist)]] [CENTROID[(varlist)][(n)]]
       [NONE]]

[/SAVE=[rootname][(value)]]

[/MATRIX=OUT({*                 })]
            {'savfile'|'dataset'}
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
OVERALS VARIABLES=PRETEST1 PRETEST2 POSTEST1 POSTEST2(20)
SES(5) SCHOOL(3)
/ANALYSIS=PRETEST1 TO POSTEST2 (NUME) SES (ORDI) SCHOOL (SNOM)
/SETS=3(2,2,2).
```

Overview

OVERALS performs nonlinear canonical correlation analysis on two or more sets of variables. Variables can have different optimal scaling levels, and no assumptions are made about the distribution of the variables or the linearity of the relationships.

Options

Optimal Scaling Levels. You can specify the level of optimal scaling at which you want to analyze each variable.

Number of Dimensions. You can specify how many dimensions OVERALS should compute.

Iterations and Convergence. You can specify the maximum number of iterations and the value of a convergence criterion.

Display Output. The output can include all available statistics, only the default statistics, or only the specific statistics that you request. You can also control whether some of these statistics are plotted.

Saving Scores. You can save object scores in the active dataset.

Writing Matrices. You can write a matrix data file containing quantification scores, centroids, weights, and loadings for use in further analyses.

Basic Specification

- The basic specification is command OVERALS, the VARIABLES subcommand, the ANALYSIS subcommand, and the SETS subcommand. By default, OVERALS estimates a two-dimensional solution and displays a table listing optimal scaling levels of each variable by set, eigenvalues and loss values by set, marginal frequencies, centroids and weights for all variables, and plots of the object scores and component loadings.

Subcommand Order

- The VARIABLES subcommand, ANALYSIS subcommand, and SETS subcommand must appear in that order before all other subcommands.
- Other subcommands can appear in any order.

Operations

- If the ANALYSIS subcommand is specified more than once, OVERALS is not executed. For all other subcommands, if a subcommand is specified more than once, only the last occurrence is executed.
- OVERALS treats every value in the range 1 to the maximum value that is specified on VARIABLES as a valid category. To avoid unnecessary output, use the AUTORECODE or RECODE command to recode a categorical variable that has nonsequential values or that has a large number of categories. For variables that are treated as numeric, recoding is not recommended because the characteristic of equal intervals in the data will not be maintained (see AUTORECODE and RECODE for more information).

Limitations

- String variables are not allowed; use AUTORECODE to recode nominal string variables.
- The data must be positive integers. Zeros and negative values are treated as system-missing, which means that they are excluded from the analysis. Fractional values are truncated after the decimal and are included in the analysis. If one of the levels of a categorical variable has been coded 0 or some negative value, and you want to treat it as a valid category, use the AUTORECODE or RECODE command to recode the values of that variable.
- OVERALS ignores user-missing value specifications. Positive user-missing values that are less than the maximum value that is specified on the VARIABLES subcommand are treated as valid category values and are included in the analysis. If you do not want the category to be included, use COMPUTE or RECODE to change the value to a value outside of the valid range. Values outside of the range (less than 1 or greater than the maximum value) are treated as system-missing and are excluded from the analysis.
- If one variable in a set has missing data, all variables in that set are missing for that object (case).
- Each set must have at least three valid (non-missing, non-empty) cases.

Examples

```
OVERALS VARIABLES=PRETEST1 PRETEST2 POSTEST1 POSTEST2(20)
SES(5) SCHOOL(3)
/ANALYSIS=PRETEST1 TO POSTEST2 (NUME) SES (ORDI) SCHOOL (SNOM)
/SETS=3(2,2,2)
/PRINT=OBJECT FIT
/PLOT=QUANT(PRETEST1 TO SCHOOL).
```

- VARIABLES defines the variables and their maximum values.
- ANALYSIS specifies that all variables from *PRETEST1* to *POSTEST2* are to be analyzed at the numeric level of optimal scaling, *SES* is to be analyzed at the ordinal level, and *SCHOOL* is to be analyzed as a single nominal. These variables are all of the variables that will be used in the analysis.
- SETS specifies that there are three sets of variables to be analyzed and two variables in each set.
- PRINT lists the object and fit scores.
- PLOT plots the single-category and multiple-category coordinates of all variables in the analysis.

VARIABLES Subcommand

VARIABLES specifies all variables in the current OVERALS procedure.

- The VARIABLES subcommand is required and precedes all other subcommands. The actual word VARIABLES can be omitted.
- Each variable or variable list is followed by the maximum value in parentheses.

ANALYSIS Subcommand

ANALYSIS specifies the variables to be used in the analysis and the optimal scaling level at which each variable is to be analyzed.

- The ANALYSIS subcommand is required and follows the VARIABLES subcommand.
- The specification on ANALYSIS is a variable list and an optional keyword in parentheses, indicating the level of optimal scaling.
- The variables on ANALYSIS must also be specified on the VARIABLES subcommand.
- Only active variables are listed on the ANALYSIS subcommand. **Active variables** are those variables that are used in the computation of the solution. **Passive variables**, those variables that are listed on the VARIABLES subcommand but not on the ANALYSIS subcommand, are ignored in the OVERALS solution. Object score plots can still be labeled by passive variables.

The following keywords can be specified to indicate the optimal scaling level:

MNOM. *Multiple nominal.* The quantifications can be different for each dimension. When all variables are multiple nominal, and there is only one variable in each set, OVERALS gives the same results as HOMALS.

SNOM. *Single nominal*. OVERALS gives only one quantification for each category. Objects in the same category (cases with the same value on a variable) obtain the same quantification. When all variables are SNOM, ORDI, or NUME, and there is only one variable per set, OVERALS gives the same results as PRINCALS.

ORDI. *Ordinal*. This setting is the default for variables that are listed without optimal scaling levels. The order of the categories of the observed variable is preserved in the quantified variable.

NUME. *Numerical*. Interval or ratio scaling level. OVERALS assumes that the observed variable already has numerical values for its categories. When all variables are quantified at the numerical level, and there is only one variable per set, the OVERALS analysis is analogous to classical principal components analysis.

These keywords can apply to a variable list as well as to a single variable. Thus, the default ORDI is not applied to a variable without a keyword if a subsequent variable on the list has a keyword.

SETS Subcommand

SETS specifies how many sets of variables exist and how many variables are in each set.

- SETS is required and must follow the ANALYSIS subcommand.
- SETS is followed by an integer to indicate the number of variable sets. Following this integer is a list of values in parentheses, indicating the number of variables in each set.
- There must be at least two sets.
- The sum of the values in parentheses must equal the number of variables specified on the ANALYSIS subcommand. The variables in each set are read consecutively from the ANALYSIS subcommand.

An example is as follows:

```
/SETS=2(2,3)
```

This specification indicates that there are two sets. The first two variables that are named on ANALYSIS are the first set, and the last three variables that are named on ANALYSIS are the second set.

NOBSERVATIONS Subcommand

NOBSERVATIONS specifies how many cases are used in the analysis.

- If NOBSERVATIONS is not specified, all available observations in the active dataset are used.
- NOBSERVATIONS is followed by an integer, indicating that the first *n* cases are to be used.

DIMENSION Subcommand

DIMENSION specifies the number of dimensions that you want OVERALS to compute.

- If you do not specify the DIMENSION subcommand, OVERALS computes two dimensions.
- DIMENSION is followed by an integer indicating the number of dimensions.
- If all variables are SNOM (single nominal), ORDI (ordinal), or NUME (numerical), the maximum number of dimensions that you can specify is the total number of variables on the ANALYSIS subcommand.
- If some or all variables are MNOM (multiple nominal), the maximum number of dimensions that you can specify is the number of MNOM variable levels (categories) plus the number of non-MNOM variables, minus the number of MNOM variables.
- The maximum number of dimensions must be less than the number of observations minus 1.
- If the number of sets is 2, and all variables are SNOM, ORDI, or NUME, the number of dimensions should not be more than the number of variables in the smaller set.
- If the specified value is too large, OVERALS tries to adjust the number of dimensions to the allowable maximum. OVERALS might not be able to adjust if there are MNOM variables with missing data.

INITIAL Subcommand

The INITIAL subcommand specifies the method that is used to compute the initial configuration.

- The specification on INITIAL is keyword NUMERICAL or RANDOM. If the INITIAL subcommand is not specified, NUMERICAL is the default.

NUMERICAL. *Treat all variables except multiple nominal as numerical*. This specification is best to use when there are no SNOM variables.

RANDOM. *Compute a random initial configuration*. This specification should be used only when some or all variables are SNOM.

MAXITER Subcommand

MAXITER specifies the maximum number of iterations that OVERALS can go through in its computations.

- If MAXITER is not specified, OVERALS will iterate up to 100 times.
- The specification on MAXITER is an integer indicating the maximum number of iterations.

CONVERGENCE Subcommand

CONVERGENCE specifies a convergence criterion value. OVERALS stops iterating if the difference in fit between the last two iterations is less than the CONVERGENCE value.

- The default CONVERGENCE value is 0.00001.

- The specification on CONVERGENCE is any value that is greater than 0.000001. (Values that are less than this value might seriously affect performance.)

PRINT Subcommand

PRINT controls which statistics are included in your display output. The default output includes a table that lists optimal scaling levels of each variable by set; eigenvalues and loss values by set by dimension; and the output that is produced by keywords FREQ, CENTROID, and WEIGHTS.

The following keywords are available:

FREQ. *Marginal frequencies for the variables in the analysis*.

HISTORY. *History of the iterations*.

FIT. *Multiple fit, single fit, and single loss per variable*.

CENTROID. *Category quantification scores, the projected centroids, and the centroids*.

OBJECT. *Object scores*.

QUANT. *Category quantifications and the single and multiple coordinates*.

WEIGHTS. *Weights and component loadings*.

DEFAULT. *FREQ, CENTROID, and WEIGHTS*.

NONE. *Summary loss statistics*.

PLOT Subcommand

PLOT can be used to produce plots of transformations, object scores, coordinates, centroids, and component loadings.

- If PLOT is not specified, plots of the object scores and component loadings are produced.

The following keywords can be specified on PLOT:

LOADINGS. *Plot of the component loadings*.

OBJECT. *Plot of the object scores*.

TRANS. *Plot of category quantifications*.

QUANT. *Plot of all category coordinates*.

CENTROID. *Plot of all category centroids*.

DEFAULT. *OBJECT and LOADINGS*.

NONE. *No plots*.

- Keywords OBJECT, QUANT, and CENTROID can each be followed by a variable list in parentheses to indicate that plots should be labeled with these variables. For QUANT and CENTROID, the variables must be specified on both the VARIABLES and ANALYSIS subcommands. For OBJECT, the variables must be specified on VARIABLES but need not appear on ANALYSIS, meaning that variables that are not used in the computations can still be used to label OBJECT plots. If the variable list is omitted, the default plots are produced.
- Object score plots use category labels corresponding to all categories within the defined range. Objects in a category that is outside the defined range are labeled with the label corresponding to the category immediately following the defined maximum category.
- If TRANS is followed by a variable list, only plots for those variables are produced. If a variable list is not specified, plots are produced for each variable.
- All keywords except NONE can be followed by an integer in parentheses to indicate how many characters of the variable or value label are to be used on the plot. (If you specified a variable list after

OBJECT, CENTROID, TRANS, or QUANT, you can specify the value in parentheses after the list.) The value can range from 1 to 20. If the value is omitted, 12 characters are used. Spaces between words count as characters.

- If a variable label is missing, the variable name is used for that variable. If a value label is missing, the actual value is used.
- Make sure that your variable and value labels are unique by at least one letter in order to distinguish them on the plots.
- When points overlap, the points are described in a summary following the plot.

In addition to the plot keywords, the following keyword can be specified:

NDIM. *Dimension pairs to be plotted*. NDIM is followed by a pair of values in parentheses. If NDIM is not specified, plots are produced for dimension 1 versus dimension 2.

- The first value indicates the dimension that is plotted against all higher dimensions. This value can be any integer from 1 to the number of dimensions minus 1.
- The second value indicates the highest dimension to be used in plotting the dimension pairs. This value can be any integer from 2 to the number of dimensions.
- Keyword ALL can be used instead of the first value to indicate that all dimensions are paired with higher dimensions.
- Keyword MAX can be used instead of the second value to indicate that plots should be produced up to and including the highest dimension fit by the procedure.

Example

```
OVERALS COLA1 COLA2 JUICE1 JUICE2 (4)
/ANALYSIS=COLA1 COLA2 JUICE1 JUICE2 (SNOM)
/SETS=2(2,2)
/PLOT NDIM(1,3) QUANT(5).
```

- The NDIM(1,3) specification indicates that plots should be produced for two dimension pairs—dimension 1 versus dimension 2 and dimension 1 versus dimension 3.
- QUANT requests plots of the category quantifications. The (5) specification indicates that the first five characters of the value labels are to be used on the plots.

Example

```
OVERALS COLA1 COLA2 JUICE1 JUICE2 (4)
/ANALYSIS=COLA1 COLA2 JUICE1 JUICE2 (SNOM)
/SETS=2(2,2)
/PLOT NDIM(ALL,3) QUANT(5).
```

- This plot is the same as above except for the ALL specification following NDIM, which indicates that all possible pairs up to the second value should be plotted. QUANT plots will be produced for dimension 1 versus dimension 2, dimension 2 versus dimension 3, and dimension 1 versus dimension 3.

SAVE Subcommand

SAVE lets you add variables containing the object scores that are computed by OVERALS to the active dataset.

- If SAVE is not specified, object scores are not added to the active dataset.
- A variable rootname can be specified on the SAVE subcommand, to which OVERALS adds the number of the dimension. Only one rootname can be specified, and it can contain up to six characters.
- If a rootname is not specified, unique variable names are automatically generated. The variable names are *OVE n_m*, where *n* is a dimension number and *m* is a set number. If three dimensions are saved, the first set of names are *OVE1_1*, *OVE2_1*, and *OVE3_1*. If another OVERALS is then run, the variable names for the second set are *OVE1_2*, *OVE2_2*, *OVE3_2*, and so on.
- Following the name, the number of dimensions for which you want object scores saved can be listed in parentheses. The number cannot exceed the value of the DIMENSION subcommand.

- The prefix should be unique for each OVERALS command in the same session. Otherwise,, OVERALS replaces the prefix with *DIM*, *OBJ*, or *OBSAVE*. If all of these prefixes already exist, SAVE is not executed.
- If the number of dimensions is not specified, the SAVE subcommand saves object scores for all dimensions.
- If you replace the active dataset by specifying an asterisk (*) on a MATRIX subcommand, the SAVE subcommand is not executed.

Example

```
OVERALS CAR1 CAR2 CAR3(5) PRICE(10)
/SET=2(3,1)
/ANALYSIS=CAR1 TO CAR3(SNOM) PRICE(NUME)
/DIMENSIONS=3
/SAVE=DIM(2).
```

- Analyzed items include three single nominal variables, *CAR1*, *CAR2*, and *CAR3* (each with 5 categories) and one numeric level variable (with 10 categories).
- The DIMENSIONS subcommand requests results for three dimensions.
- SAVE adds the object scores from the first two dimensions to the active dataset. The names of these new variables will be *DIM00001* and *DIM00002*, respectively.

MATRIX Subcommand

The MATRIX subcommand is used to write category quantifications, coordinates, centroids, weights, and component loadings to a matrix data file.

- The specification on MATRIX is keyword OUT and a quoted file specification or previously declared dataset name (DATASET DECLARE command), enclosed in parentheses.
- You can specify an asterisk (*) instead of a file to replace the active dataset.
- All values are written to the same file.
- The matrix data file has one case for each value of each original variable.

The variables of the matrix data file and their values are as follows:

ROWTYPE_. *String variable containing value QUANT for the category quantifications, SCOOR_ for the single-category coordinates, MCOOR_ for multiple-category coordinates, CENTRO_ for centroids, PCENTRO_ for projected centroids, WEIGHT_ for weights, and LOADING_ for the component scores*.

LEVEL. *String variable containing the values (or value labels, if present) of each original variable for category quantifications*. For cases with *ROWTYPE_=LOADING_* or *WEIGHT_*, the value of *LEVEL* is blank.

VARNAME_. *String variable containing the original variable names*.

VARTYPE_. *String variable containing values MULTIPLE, SINGLE N, ORDINAL, or NUMERICAL, depending on the level of optimal scaling that is specified for the variable*.

SET_. *The set number of the original variable*.

DIM1...DIMn. *Numeric variables containing the category quantifications, the single-category coordinates, multiple-category coordinates, weights, centroids, projected centroids, and component loadings for each dimension*. Each variable is labeled *DIM n*, where *n* represents the dimension number. Any values that cannot be computed are assigned 0 in the file.

PACF

```
PACF VARIABLES= series names
 [/DIFF={1}]
        {n}
 [/SDIFF={1}]
         {n}
 [/PERIOD=n]
 [/{NOLOG**}]
   {LN     }
 [/SEASONAL]
 [/MXAUTO={16**}]
          {n   }
 [/APPLY ['model name']]
```

**Default if the subcommand is omitted and there is no corresponding specification on the TSET command.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
PACF VARIABLES = TICKETS.
```

Overview

PACF displays and plots the sample partial autocorrelation function of one or more time series. You can also display and plot the partial autocorrelations of transformed series by requesting natural log and differencing transformations from within the procedure.

Options

Modification of the Series. You can use the LN subcommand to request a natural log transformation of the series, and you can use the SDIFF and DIFF subcommand to request seasonal and nonseasonal differencing to any degree. With seasonal differencing, you can specify the periodicity on the PERIOD subcommand.

Statistical Output. With the MXAUTO subcommand, you can specify the number of lags for which you want values to be displayed and plotted, overriding the maximum value that is specified on TSET. You can also use the SEASONAL subcommand to display and plot values only at periodic lags.

Basic Specification

The basic specification is one or more series names. For each specified series, PACF automatically displays the partial autocorrelation value and standard error value for each lag. PACF also plots the partial autocorrelations and marks the bounds of two standard errors on the plot. By default, PACF displays and plots partial autocorrelations for up to 16 lags (or the number of lags that are specified on TSET).

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- VARIABLES can be specified only once.
- Other subcommands can be specified more than once, but only the last specification of each subcommand is executed.

Operations

- Subcommand specifications apply to all series that are named on the PACF command.
- If the LN subcommand is specified, any differencing that is requested on that PACF command is done on log-transformed series.
- Confidence limits are displayed in the plot, marking the bounds of two standard errors at each lag.

Limitations

- A maximum of one VARIABLES subcommand is allowed. There is no limit on the number of series that are named on the list.

Example

```
PACF VARIABLES = TICKETS
  /LN
  /DIFF=1
  /SDIFF=1
  /PERIOD=12
  /MXAUTO=25.
```

- This example produces a plot of the partial autocorrelation function for the series *TICKETS* after a natural log transformation, differencing, and seasonal differencing have been applied to the series. Along with the plot, the partial autocorrelation value and standard error are displayed for each lag.
- LN transforms the data by using the natural logarithm (base *e*) of the series.
- DIFF differences the series once.
- SDIFF and PERIOD apply one degree of seasonal differencing with a period of 12.
- MXAUTO specifies 25 for the maximum number of lags for which output is to be produced.

VARIABLES Subcommand

VARIABLES specifies the series names and is the only required subcommand.

DIFF Subcommand

DIFF specifies the degree of differencing that is used to convert a nonstationary series to a stationary series with a constant mean and variance before the partial autocorrelations are computed.

- You can specify 0 or any positive integer on DIFF.
- If DIFF is specified without a value, the default is 1.
- The number of values that are used in the calculations decreases by 1 for each degree of differencing.

Example

```
PACF VARIABLES = SALES
  /DIFF=1.
```

- In this example, the series *SALES* will be differenced once before the partial autocorrelations are computed and plotted.

SDIFF Subcommand

If the series exhibits a seasonal or periodic pattern, you can use the SDIFF subcommand to seasonally difference the series before obtaining partial autocorrelations. SDIFF indicates the degree of seasonal differencing.

- The specification on SDIFF can be 0 or any positive integer.
- If SDIFF is specified without a value, the default is 1.

- The number of seasons that are used in the calculations decreases by 1 for each degree of seasonal differencing.
- The length of the period that is used by SDIFF is specified on the PERIOD subcommand. If the PERIOD subcommand is not specified, the periodicity that was established on the TSET or DATE command is used (see the PERIOD subcommand).

PERIOD Subcommand

PERIOD indicates the length of the period to be used by the SDIFF or SEASONAL subcommand. PERIOD indicates how many observations are in one period or season.

- The specification on PERIOD can be any positive integer that is greater than 1.
- PERIOD is ignored if it is used without the SDIFF or SEASONAL subcommand.
- If PERIOD is not specified, the periodicity that was established on TSET PERIOD is in effect. If TSET PERIOD is not specified, the periodicity that was established on the DATE command is used. If periodicity was not established anywhere, the SDIFF and SEASONAL subcommands are not executed.

Example

```
PACF VARIABLES = SALES
  /SDIFF=1
  /PERIOD=12.
```

- This PACF command applies one degree of seasonal differencing with a periodicity of 12 to the series *SALES* before partial autocorrelations are computed and plotted.

LN and NOLOG Subcommands

LN transforms the data by using the natural logarithm (base *e*) of the series and is used to remove varying amplitude over time. NOLOG indicates that the data should not be log transformed. NOLOG is the default.

- If you specify LN on a PACF command, any differencing that is requested on that command is performed on the log-transformed series.
- There are no additional specifications on LN or NOLOG.
- Only the last LN or NOLOG subcommand on a PACF command is executed.
- If a natural log transformation is requested when there are values in the series that are less than or equal to 0, PACF will not be produced for that series because nonpositive values cannot be log-transformed.
- NOLOG is generally used with an APPLY subcommand to turn off a previous LN specification.

Example

```
PACF VARIABLES = SALES
  /LN.
```

- This command transforms the series *SALES* by using the natural log transformation and then computes and plots partial autocorrelations.

SEASONAL Subcommand

Use SEASONAL to focus attention on the seasonal component by displaying and plotting autocorrelations only at periodic lags.

- There are no additional specifications on SEASONAL.
- If SEASONAL is specified, values are displayed and plotted at the periodic lags that are indicated on the PERIOD subcommand. If PERIOD is not specified, the periodicity that was established on the TSET or DATE command is used (see the PERIOD subcommand).

- If SEASONAL is not specified, partial autocorrelations for all lags (up to the maximum) are displayed and plotted.

Example

```
PACF VARIABLES = SALES
  /SEASONAL
  /PERIOD=12.
```

- In this example, partial autocorrelations are displayed and plotted at every 12th lag.

MXAUTO Subcommand

MXAUTO specifies the maximum number of lags for a series.

- The specification on MXAUTO must be a positive integer.
- If MXAUTO is not specified, the default number of lags is the value that was set on TSET MXAUTO. If TSET MXAUTO is not specified, the default is 16.
- The value on MXAUTO overrides the value that was set on TSET MXAUTO.

Example

```
PACF VARIABLES = SALES
  /MXAUTO=14.
```

- This command specifies 14 for the maximum number of partial autocorrelations that can be displayed and plotted for series *SALES*.

APPLY Subcommand

APPLY allows you to use a previously defined PACF model without having to repeat the specifications.

- The only specification on APPLY is the name of a previous model enclosed in quotes. If a model name is not specified, the model that was specified on the previous PACF command is used.
- To change one or more model specifications, specify the subcommands of only those portions that you want to change, placing the specifications after the APPLY subcommand.
- If no series are specified on the PACF command, the series that were originally specified with the model that is being reapplied are used.
- To change the series that are used with the model, enter new series names before or after the APPLY subcommand.

Example

```
PACF VARIABLES = TICKETS
  /LN
  /DIFF=1
  /SDIFF=1
  /PER=12
  /MXAUTO=25.
PACF VARIABLES = ROUNDTRP
  /APPLY.
```

- The first command specifies a maximum of 25 partial autocorrelations for the series *TICKETS* after it has been log-transformed, differenced once, and had one degree of seasonal differencing with a periodicity of 12 applied to it. This model is assigned the default name *MOD_1*.
- The second command displays and plots partial autocorrelations for series *ROUNDTRP* by using the same model that was specified for series *TICKETS*.

References

Box, G. E. P., and G. M. Jenkins. 1976. *Time series analysis: Forecasting and control*, Rev. ed. San Francisco: Holden-Day.

PARTIAL CORR

PARTIAL CORR is available in Statistics Base Edition.

```
PARTIAL CORR VARIABLES= varlist [WITH varlist]
    BY varlist [(levels)]

 [/SIGNIFICANCE={TWOTAIL**}]
                {ONETAIL  }

 [/STATISTICS=[NONE**] [CORR] [DESCRIPTIVES] [BADCORR] [ALL]]

 [/FORMAT={MATRIX** }]
          {SERIAL   }
          {CONDENSED}

 [/MISSING=[{LISTWISE**}]  [{EXCLUDE**}]]
            {ANALYSIS  }    {INCLUDE  }

 [/MATRIX= [IN({*                  })] [OUT({*                  })]]
               {'savfile'|'dataset'}        {'savfile'|'dataset'}
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 16.0

- Added support for SET THREADS and SET MCACHE.

Example

```
PARTIAL CORR VARIABLES=PUBTRANS MECHANIC BY NETPURSE(1).
```

Overview

PARTIAL CORR produces partial correlation coefficients that describe the relationship between two variables while adjusting for the effects of one or more additional variables. PARTIAL CORR calculates a matrix of Pearson product-moment correlations. PARTIAL CORR can also read the zero-order correlation matrix as input. Other procedures that produce zero-order correlation matrices that can be read by PARTIAL CORR include CORRELATIONS, REGRESSION, DISCRIMINANT, and FACTOR.

Options

Significance Levels. By default, the significance level for each partial correlation coefficient is based on a two-tailed test. Optionally, you can request a one-tailed test using the SIGNIFICANCE subcommand.

Statistics. In addition to the partial correlation coefficient, degrees of freedom, and significance level, you can use the STATISTICS subcommand to obtain the mean, standard deviation, and number of nonmissing cases for each variable, as well as zero-order correlation coefficients for each pair of variables.

Format. You can specify condensed format, which suppresses the degrees of freedom and significance level for each coefficient, and you can print only nonredundant coefficients in serial string format by using the FORMAT subcommand.

Matrix Input and Output. You can read and write zero-order correlation matrices by using the MATRIX subcommand.

Basic Specification

The basic specification is the VARIABLES subcommand, which specifies a list of variables to be correlated, and one or more control variables following keyword BY. PARTIAL CORR calculates the partial correlation of each variable with every other variable that was specified on the correlation variable list.

Subcommand Order

Subcommands can be specified in any order.

Operations

- PARTIAL CORR produces one matrix of partial correlation coefficients for each of up to five order values. For each coefficient, PARTIAL CORR prints the degrees of freedom and the significance level.
- This procedure uses the multithreaded options specified by SET THREADS and SET MCACHE.

Limitations

- A maximum of 25 variable lists on a single PARTIAL CORR command is allowed. Each variable list contains a correlation list, a control list, and order values.
- A maximum of 400 variables total can be named or implied per PARTIAL CORR command.
- A maximum of 100 control variables is allowed.
- A maximum of 5 different order values per single list is allowed. The largest order value that can be specified is 100.

VARIABLES Subcommand

VARIABLES requires a *correlation list* of one or more pairs of variables for which partial correlations are desired and requires a *control list* of one or more variables that will be used as controls for the variables in the correlation list, followed by optional order values in parentheses.

- The correlation list specifies pairs of variables to be correlated while controlling for the variables in the control list.
- To request a square or lower-triangular matrix, do not use keyword WITH in the correlation list. This specification obtains the partial correlation of every variable with every other variable in the list.
- To request a rectangular matrix, specify a list of correlation variables followed by keyword WITH and a second list of variables. This specification obtains the partial correlation of specific variable pairs. The first variable list defines the rows of the matrix, and the second list defines the columns.
- The control list is specified after keyword BY.
- The correlation between a pair of variables is referred to as a *zero-order correlation*. Controlling for one variable produces a first-order partial correlation, controlling for two variables produces a second-order partial correlation, and so on.
- To indicate the exact partials that are to be computed, you can specify order values in parentheses following the control list. These values also determine the partial correlation matrix or matrices to be printed. Up to five order values can be specified. Separate each value with at least one space or comma. The default order value is the number of control variables.
- One partial is produced for every unique combination of control variables for each order value.

Obtaining the Partial Correlation for Specific Variable Pairs

```
PARTIAL CORR
  VARIABLES = RENT FOOD PUBTRANS WITH TEACHER MANAGER BY NETSALRY(1).
```

- PARTIAL CORR produces a rectangular matrix. Variables *RENT*, *FOOD*, and *PUBTRANS* form the matrix rows, and variables *TEACHER* and *MANAGER* form the columns.

Specifying Order Values

```
PARTIAL CORR
  VARIABLES = RENT WITH TEACHER BY NETSALRY, NETPRICE (1).
PARTIAL CORR
  VARIABLES = RENT WITH TEACHER BY NETSALRY, NETPRICE (2).
PARTIAL CORR
  VARIABLES = RENT WITH TEACHER BY NETSALRY, NETPRICE (1,2).
PARTIAL CORR
  VARIABLES = RENT FOOD PUBTRANS BY NETSALRY NETPURSE NETPRICE (1,3).
```

- The first PARTIAL CORR produces two first-order partials: *RENT* with *TEACHER* controlling for *NETSALRY*, and *RENT* with *TEACHER* controlling for *NETPRICE*.
- The second PARTIAL CORR produces one second-order partial of *RENT* with *TEACHER* controlling simultaneously for *NETSALRY* and *NETPRICE*.
- The third PARTIAL CORR specifies both sets of partials that were specified by the previous two commands.
- The fourth PARTIAL CORR produces three first-order partials (controlling for *NETSALRY*, *NETPURSE*, and *NETPRICE* individually) and one third-order partial (controlling for all three control variables simultaneously).

SIGNIFICANCE Subcommand

SIGNIFICANCE determines whether the significance level is based on a one-tailed or two-tailed test.

- By default, the significance level is based on a two-tailed test. This setting is appropriate when the direction of the relationship between a pair of variables cannot be specified in advance of the analysis.
- When the direction of the relationship can be determined in advance, a one-tailed test is appropriate.

TWOTAIL. *Two-tailed test of significance.* This setting is the default.

ONETAIL. *One-tailed test of significance.*

STATISTICS Subcommand

By default, the partial correlation coefficient, degrees of freedom, and significance level are displayed. Use STATISTICS to obtain additional statistics.

- If both CORR and BADCORR are requested, CORR takes precedence over BADCORR, and the zero-order correlations are displayed.

CORR. *Zero-order correlations with degrees of freedom and significance level.*

DESCRIPTIVES . *Mean, standard deviation, and number of nonmissing cases.* Descriptive statistics are not available with matrix input.

BADCORR. *Zero-order correlation coefficients only if any zero-order correlations cannot be computed.* Noncomputable coefficients are displayed as a period.

NONE. *No additional statistics.* This setting is the default.

ALL. *All additional statistics that are available with* PARTIAL CORR.

FORMAT Subcommand

FORMAT determines page format.

- If both CONDENSED and SERIAL are specified, only SERIAL is in effect.

MATRIx. *Display degrees of freedom and significance level in matrix format.* This format requires four lines per matrix row and displays the degrees of freedom and the significance level. The output includes redundant coefficients. This setting is the default.

CONDENSED . *Suppress the degrees of freedom and significance level*. This format requires only one line per matrix row and suppresses the degrees of freedom and significance. A single asterisk (*) following a coefficient indicates a significance level of 0.05 or less. Two asterisks (**) following a coefficient indicate a significance level of 0.01 or less.

SERIAL. *Display only the nonredundant coefficients in serial string format*. The coefficients, degrees of freedom, and significance levels from the first row of the matrix are displayed first, followed by all unique coefficients from the second row and so on for all rows of the matrix.

MISSING Subcommand

MISSING controls the treatment of cases with missing values.

- When pairwise deletion is in effect (keyword ANALYSIS), the degrees of freedom for a particular partial coefficient are based on the smallest number of cases that are used in the calculation of any of the simple correlations.
- LISTWISE and ANALYSIS are alternatives. However, each command can be used with either INCLUDE or EXCLUDE. The default is LISTWISE and EXCLUDE.

LISTWISE. *Exclude cases with missing values listwise.* Cases with missing values for any of the variables that are listed for an analysis—including control variables—are not used in the calculation of the zero-order correlation coefficient. This setting is the default.

ANALYSIS. *Exclude cases with missing values on a pair-by-pair basis.* Cases with missing values for one or both of a pair of variables are not used in the calculation of zero-order correlation coefficients.

EXCLUDE. *Exclude user-missing values*. User-missing values are treated as missing. This setting is the default.

INCLUDE. *Include user-missing values*. User-missing values are treated as valid values.

MATRIX Subcommand

MATRIX reads and writes matrix data files.

- Either IN or OUT and a matrix file in parentheses is required. When both IN and OUT are used on the same PARTIAL CORR procedure, they can be specified on separate MATRIX subcommands or they can both be specified on the same subcommand.

OUT ('savfile'|'dataset'). *Write a matrix data file or dataset.* Specify either a filename, a previously declared dataset name, or an asterisk, enclosed in parentheses. Filenames should be enclosed in quotes and are stored in the working directory unless a path is included as part of the file specification. If you specify an asterisk (*), the matrix data file replaces the active dataset. If you specify an asterisk or a dataset name, the file is not stored on disk unless you use SAVE or XSAVE.

IN ('savfile'|'dataset'). *Read a matrix data file or dataset.* Specify either a filename, dataset name created during the current session, or an asterisk enclosed in parentheses. An asterisk reads the matrix data from the active dataset. Filenames should be enclosed in quotes and are read from the working directory unless a path is included as part of the file specification.

Matrix Output

- The matrix materials that PARTIAL CORR writes can be used by subsequent PARTIAL CORR procedures or by other procedures that read correlation-type matrices.
- In addition to the partial correlation coefficients, the matrix materials that PARTIAL CORR writes include the mean, standard deviation, and number of cases that are used to compute each coefficient (see for a description of the file) . If PARTIAL CORR reads matrix data and then writes matrix materials based on those data, the matrix data file that it writes will not include means and standard deviations.
- PARTIAL CORR writes a full square matrix for the analysis that is specified on the VARIABLES subcommand (or the analysis list if keyword VARIABLES is omitted). No matrix is written for subsequent variable lists.
- Any documents that are contained in the active dataset are not transferred to the matrix file.

Matrix Input

- When matrix materials are read from a file other than the active dataset, both the active dataset and the matrix data file that is specified on IN must contain all variables that are specified on the VARIABLES subcommand.

- MATRIX=IN cannot be specified unless a active dataset has already been defined. To read an existing matrix data file at the beginning of a session, use GET to retrieve the matrix file and then specify IN(*) on MATRIX.
- PARTIAL CORR can read correlation-type matrices written by other procedures.
- The program reads variable names, variable and value labels, and print and write formats from the dictionary of the matrix data file.

Format of the Matrix Data File

- The matrix data file includes two special variables that are created by the program: *ROWTYPE_* and *VARNAME_*.
- *ROWTYPE_* is a short string variable with values N, MEAN, STDDEV, and PCORR (for the partial correlation coefficient).
- *VARNAME_* is a short string variable whose values are the names of the variables that are used to form the correlation matrix. When *ROWTYPE_* is PCORR, *VARNAME_* gives the variable that is associated with that row of the correlation matrix.
- The remaining variables in the file are the variables that are used to form the correlation matrix.

Split Files

- When split-file processing is in effect, the first variables in the matrix data file are the split variables, followed by *ROWTYPE_*, *VARNAME_*, and the variables that are used to form the correlation matrix.
- A full set of matrix materials is written for each split-file group that is defined by the split variables.
- A split variable cannot have the same variable name as any other variable that is written to the matrix data file.
- If split-file processing is in effect when a matrix is written, the same split file must be in effect when that matrix is read by any procedure.

Missing Values

- With pairwise treatment of missing values (MISSING=ANALYSIS is specified), the matrix of Ns that is used to compute each coefficient is included with the matrix materials.
- With LISTWISE treatment, a single N that is used to calculate all coefficients is included with the matrix materials.
- When reading a matrix data file, be sure to specify a missing-value treatment on PARTIAL CORR that is compatible with the missing-value treatment that was in effect when the matrix materials were produced.

Examples

Writing Results to a Matrix Data File

```
GET FILE='/data/city.sav'.
PARTIAL CORR VARIABLES=BUSDRVER MECHANIC ENGINEER TEACHER COOK
                       BY  NETSALRY(1)
  /MATRIX=OUT('/data/partial_matrix.sav').
```

- PARTIAL CORR reads data from file *city.sav* and writes one set of matrix materials to file *partial_matrix.sav*.
- The active dataset is still *city.sav*. Subsequent commands are executed on *city.sav*.

Writing Matrix Results That Replace the Active Dataset

```
GET FILE='/data/city.sav'.
PARTIAL CORR VARIABLES=BUSDRVER MECHANIC ENGINEER TEACHER COOK
  BY  NETSALRY(1)   /MATRIX=OUT(*).
LIST.
```

- PARTIAL CORR writes the same matrix as in the example above. However, the matrix data file replaces the active dataset. The LIST command is executed on the matrix file, not on the *CITY* file.

Using a Matrix Data File as Input

```
GET FILE='/data/personnel.sav'.
FREQUENCIES VARIABLES=AGE.
PARTIAL CORR VARIABLES=BUSDRVER MECHANIC ENGINEER TEACHER COOK
   BY  NETSALRY(1)   /MATRIX=IN('/data/corr_matrix.sav').
```

- This example performs a frequencies analysis on file *personnel.sav* and then uses a different file for PARTIAL CORR. The file is an existing matrix data file.
- MATRIX=IN specifies the matrix data file. Both the active dataset and the *corr_matrix.sav* file must contain all variables that are specified on the VARIABLES subcommand on PARTIAL CORR.
- The *corr_matrix.sav* file does not replace *personnel.sav* as the active dataset.

Using an Active Dataset That Contains Matrix Data

```
GET FILE='/data/corr_matrix.sav'.
PARTIAL CORR VARIABLES=BUSDRVER MECHANIC ENGINEER TEACHER COOK
                       BY NETSALRY(1)
  /MATRIX=IN(*).
```

- The GET command retrieves the matrix data file *corr_matrix.sav*.
- MATRIX=IN specifies an asterisk because the active dataset is the matrix file *CORMTX*. If MATRIX=IN('/data/corr_matrix.sav') is specified, the program issues an error message.
- If the GET command is omitted, the program issues an error message.

```
GET FILE='/data/city.sav'.
REGRESSION MATRIX=OUT(*)
  /VARIABLES=NETPURSE PUBTRANS MECHANIC BUSDRVER
  /DEPENDENT=NETPURSE /ENTER.
PARTIAL CORR
  VARIABLES = PUBTRANS MECHANIC BUSDRVER BY NETPURSE(1) /MATRIX=IN(*).
```

- GET retrieves the IBM SPSS Statistics data file *city.sav*.
- REGRESSION computes correlations among the specified variables. MATRIX=OUT(*) writes a matrix data file that replaces the active dataset.

PERMISSIONS

```
PERMISSIONS FILE='filespec'

  /PERMISSIONS {READONLY }
               {WRITEABLE}
```

Example

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

```
PERMISSIONS FILE='/mydir/mydata.sav'
  /PERMISSIONS READONLY.
```

Overview

PERMISSIONS changes the read/write permissions for the specified file, using the operating system facilities for changing permissions.

Syntax Rules

- A FILE specification and a PERMISSIONS subcommand are both required.
- The file specification should be enclosed in single quotation marks or double quotation marks.

PERMISSIONS Subcommand

READONLY. *File permissions are set to read-only for all users.* The file cannot be saved by using the same file name with subsequent changes unless the read/write permissions are changed in the operating system or a subsequent PERMISSIONS command specifies PERMISSIONS=WRITEABLE.

WRITEABLE. *File permissions are set to allow writing for the file owner.* If file permissions were set to read-only for other users, the file remains read-only for them.

Your ability to change the read/write permissions may be restricted by the operating system.

PLANCARDS

PLANCARDS is available in SPSS Statistics Premium Edition or the Conjoint option.

```
PLANCARDS
 [/FACTORS=varlist]

 [/FORMAT={LIST}]
          {CARD}
          {BOTH}

 [/TITLE='string']

 [/FOOTER='string']

 [/OUTFILE=file]
```

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 14.0

- PAGINATE subcommand is obsolete and no longer supported.

Example

```
PLANCARDS /TITLE='Car for Sale'.
```

Overview

PLANCARDS produces profiles, or cards, from a plan file for a conjoint analysis study. The plan file can be generated by ORTHOPLAN or entered by the user. The printed profiles can be used as the experimental stimuli that subjects judge in terms of preference.

Options

Format. You can produce profiles in the format of a single list or formatted so that each profile is displayed separately.

Titles and Footers. You can specify title and footer labels that appear at the top and bottom of the output (for single list format) or at the top and bottom of each profile (when the profiles are displayed separately).

Basic Specification

- The basic specification is PLANCARDS, which produces a listing of profiles, using all variables in the active dataset except *STATUS_* and *CARD_* as factors.

Subcommand Order

- Subcommands can be named in any order.

Operations

- PLANCARDS assumes that the active dataset represents a plan for a full-profile (full-concept) conjoint study. Each "case" in such a file is one profile in the conjoint experimental plan.
- Factor and factor-level labels in the active dataset—generated by ORTHOPLAN or by the VARIABLE and VALUE LABELS commands—are used in the output.
- The command SPLIT FILE is ignored for single-profile format. In listing format, each subfile represents a different plan, and a new listing begins for each subfile.
- The WEIGHT command is ignored by PLANCARDS.

Limitations

- Missing values are not recognized as missing and are treated like other values.

Examples

```
ORTHOPLAN FACTORS=SPEED 'Highest possible speed'
   (70 '70 mph' 100 '100 mph' 130 '130 mph')
  WARRANTY 'Length of warranty' ('1 year' '3 year' '5 year')
  SEATS 'Number of seats' (2, 4) /MINIMUM=9 /HOLDOUT=6.
PLANCARDS FORMAT=BOTH
 /TITLE='Car for Sale'.
```

- ORTHOPLAN generates a set of profiles (cases) for a full-profile conjoint analysis in the active dataset.
- PLANCARDS displays the profiles, along with the title *Car for Sale*.

Example: User-entered Plan

```
DATA LIST FREE/ COST NEWNESS EXPER NAME REP
                GUARAN TRIAL TRUST.
VARIABLE LABELS
  COST 'Product cost'
  NEWNESS 'Product newness'
  EXPER 'Brand experience'
  NAME "Manufacturer's Name"
  REP "Distributor's reputation"
  GUARAN 'Money-back Guarantee'
  TRIAL 'Free sample/trial'
  TRUST 'Endorsed by a trusted person'.
VALUE LABELS
  COST 1 'LOW' 2 'HIGH'/
  NEWNESS 1 'NEW' 2 'OLD'/
  EXPER 1 'SOME' 2 'NONE'/
  NAME 1 'ESTABLISHED' 2 'UNKNOWN'/
  REP 1 'GOOD' 2 'UNKNOWN'/
  GUARAN 1 'YES' 2 'NO'/
  TRIAL 1 'YES' 2 'NO'/
  TRUST 1 'YES' 2 'NO'.
BEGIN DATA
 1 2 2 1 2 2 2 1
 2 2 2 1 1 1 2 1
 2 2 1 2 2 1 1 1
 2 1 2 1 2 2 1 2
 2 1 1 2 2 2 2 1
 2 1 2 2 1 1 2 2
 1 1 2 2 1 2 1 1
 1 1 1 1 2 1 2 2
 1 2 1 2 1 2 2 2
 1 1 1 1 1 1 1 1
 2 2 1 1 1 2 1 2
 1 2 2 2 2 1 1 2
END DATA.
PLANCARDS.
```

- In this example, the plan is entered and defined by the user rather than by ORTHOPLAN.
- PLANCARDS uses the information in the active dataset to produce a set of profiles. Because no format is specified, the default format (single list) is used. The variables and values in this example were taken from Akaah & Korgaonkar [52].

FACTORS Subcommand

FACTORS identifies the variables to be used as factors and the order in which their labels are to appear in the output. String variables are permitted.

- Keyword FACTORS is followed by a variable list.
- By default, if FACTORS is not specified, all variables in the active dataset (except those variables that are named *STATUS_* or *CARD_*) are used as factors in the order in which they appear in the file. (See the ORTHOPLAN command for information about variables *STATUS_* and *CARD_*.)

[52] Akaah, I. P., and P. K. Korgaonkar. 1988. A conjoint investigation of the relative importance of risk relievers in direct marketing. *Journal of Advertising Research,* 28:4, 38-44.

FORMAT Subcommand

FORMAT specifies how the profiles should be displayed. The choices are **listing** format (LIST keyword) and **single-profile** format (CARD keyword). Listing format displays the profiles in the form of a single list. For single-profile format, output is displayed so that each profile is presented separately.

- The keyword FORMAT is followed by LIST, CARD, or BOTH. (ALL is an alias for BOTH.)
- The default format is LIST.
- With LIST format, holdout profiles are differentiated from experimental profiles, and simulation profiles are listed separately following the experimental and holdout profiles. With CARD format, holdout profiles are not differentiated, and simulation profiles are not produced.
- If FORMAT=LIST is specified along with the OUTFILE subcommand, the OUTFILE subcommand is ignored (OUTFILE only applies to CARD format). Specifying OUTFILE with FORMAT=BOTH is equivalent to OUTFILE with FORMAT=CARD.

Example

```
PLANCARDS FORMAT=CARD /OUTFILE='DESIGN.FRM'
  /TITLE=' ' 'Profile #)CARD' /FOOTER='RANK:'.
```

- FORMAT=CARD specifies that the output will be in single-profile format.
- The profiles are written to the file *DESIGN.FRM*.
- Each profile in *DESIGN.FRM* will have the title `Profile #`*n* at the top and the label RANK: at the bottom, where *n* is a profile identification number.

The output for the first two profiles is shown below.

```
Profile #1

Product cost  LOW
Product newness  OLD
Brand experience  NONE
Manufacturer's Name  ESTABLISHED
Distributor's reputation  UNKNOWN
Money-back Guarantee  NO
Free sample/trial  NO
Endorsed by a trusted person  YES

RANK:

Profile #2

Product cost  HIGH
Product newness  OLD
Brand experience  NONE
Manufacturer's Name  ESTABLISHED
Distributor's reputation  GOOD
Money-back Guarantee  YES
Free sample/trial  NO
Endorsed by a trusted person  YES

RANK:

  . . .
```

Figure 81. Single-profile format

OUTFILE Subcommand

OUTFILE names an external file where profiles in single-profile format are to be written. Profiles in listing format are not written to an external file.

- By default, no external file is written.

- The OUTFILE keyword is followed by the name of an external file. The file is specified in the usual manner for your system.
- If the OUTFILE subcommand is specified along with FORMAT=LIST, the OUTFILE subcommand is ignored (OUTFILE only applies to FORMAT=CARD).

TITLE Subcommand

TITLE specifies a string to be used at the top of the output (in listing format) or at the top of each new profile (in single-profile format).

- Default titles are provided, except for output that is directed to an external file with the OUTFILE subcommand.
- The keyword TITLE is followed by a string enclosed in apostrophes.
- Quotation marks can be used to enclose the string instead of apostrophes when you want to use an apostrophe in the title.
- Multiple strings per TITLE subcommand can be specified; each string will appear on a separate line.
- Use an empty string (' ') to cause a blank line.
- Multiple TITLE subcommands can be specified; each subcommand will appear on a separate line.
- If the special character sequence)CARD is specified anywhere in the title, PLANCARDS will replace it with the sequential profile number in single-profile-formatted output. This character sequence is not translated in listing format.

FOOTER Subcommand

FOOTER specifies a string to be used at the bottom of the output (in listing format) or at the bottom of each profile (in single-profile format).

- If FOOTER is not used, nothing appears after the last attribute.
- FOOTER is followed by a string enclosed in apostrophes.
- Quotation marks can be used to enclose the string instead of apostrophes when you want to use an apostrophe in the footer.
- Multiple strings per FOOTER subcommand can be specified; each string will appear on a separate line.
- Use an empty string (' ') to cause a blank line.
- Multiple FOOTER subcommands can be specified; each subcommand will appear on a separate line.
- If the special character sequence)CARD is specified anywhere in the footer, PLANCARDS will replace it with the sequential profile number in single-profile-formatted output. This character sequence is not translated in listing format.

Example

```
PLANCARDS
 TITLE='Profile # )CARD' ' '
   'Circle the number in the scale at the bottom that'
   'indicates how likely you are to purchase this item.' ' '
 /FOOTER= '0   1   2   3   4   5   6   7   8   9  10'
          'Not at all      May or may       Certainly'
          'likely to           not             would'
          'purchase         purchase         purchase'
          '-----------------------------------------'
 /FORMAT=CARD
 /OUTFILE='DESIGN.FRM'.
```

The above example would produce the following output, in *DESIGN.FRM*, for the first profile:

```
Profile # 1

Circle the number in the scale at the bottom that
indicates how likely you are to purchase this item.


Product cost  LOW
Product newness  OLD
Brand experience  NONE
Manufacturer's Name  ESTABLISHED
Distributor's reputation  UNKNOWN
Money-back Guarantee  NO
Free sample/trial  NO
Endorsed by a trusted person  YES

0   1   2   3   4   5   6   7   8   9  10
Not at all      May or may       Certainly
likely to           not             would
purchase         purchase         purchase
-----------------------------------------
```

Figure 82. Footer with multiple strings

PLS

PLS is available in the Statistics Base Edition.

PLS is an extension command that requires the Python Extension Module to be installed on the system where you plan to run PLS. The PLS Extension Module must be installed separately and the installer can be downloaded from https://www.ibm.com/community/spss-statistics .

Note: The PLS Extension Module is dependent upon Python software. IBM Corp. is not the owner or licensor of the Python software. Any user of Python must agree to the terms of the Python license agreement located on the Python Web site. IBM Corp. is not making any statement about the quality of the Python program. IBM Corp. fully disclaims all liability associated with your use of the Python program.

```
PLS dependent variable [MLEVEL={N}] [REFERENCE={FIRST }]
                               {O}             {LAST**}
                               {S}             {value }
    [dependent variable...]
    [BY factor list] [WITH covariate list]

[/ID VARIABLE = variable]

[/MODEL effect [...effect]]

[/OUTDATASET [CASES=dataset]
             [LATENTFACTORS=dataset]
             [PREDICTORS=dataset]]

[/CRITERIA  LATENTFACTORS={5**    }]
                          {integer}
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 16.0

- Command introduced.

Example

```
PLS Response WITH Price.
```

Overview

The PLS procedure estimates partial least squares regression models. Partial least squares is a predictive technique that is an alternative to ordinary least squares (OLS) regression, canonical correlation, or structural equation modeling for analysis of systems of independent and response variables. It is particularly useful when predictor variables are highly correlated or when the number of predictors exceeds the number of cases.

PLS combines features of principal components analysis and multiple regression. It first extracts a set of latent factors that explains as much of the covariance as possible between the independent and dependent variables. Then a regression step predicts values of the dependent variables using the decomposition of the independent variables.

Partial least squares regression is also known as "Projection to Latent Structure".

Options

Response Variables. PLS estimates univariate and multivariate models. If you specify one or more categorical dependent variables, a classification model is estimated. If you specify one or more scale dependent variables, a regression model is estimated. Mixed regression and classification models are supported.

Predictors. Predictors can be categorical or continuous variables. Both main effects and interaction terms can be estimated.

Method. You can specify the maximum number of latent factors to extract. By default, five latent factors are extracted.

Export. You can save casewise, factorwise, and predictorwise model results to IBM SPSS Statistics datasets.

Basic Specification

- PLS is an extension command and will not be recognized by the system until you use the EXTENSION command to add PLS to the command table. The syntax diagram for PLS is defined in *plscommand.xml*, which is installed in the *extensions* subdirectory of the main installation directory. See the topic "EXTENSION" on page 663 for more information.
- The minimum specification is one or more dependent variables and one or more predictors.
- The procedure displays the following tables: proportion of variance explained (by latent factor), latent factor weights, latent factor loadings, independent variable importance in projection (VIP), and regression parameter estimates (by dependent variable).

Operations

- All model variables are centered and standardized, including indicator variables representing categorical variables.
- If a WEIGHT variable is specified, its values are used as frequency weights. Weight values are rounded to the nearest whole number before use. Cases with missing weights or weights less than 0.5 are not used in the analyses.
- User- and system-missing values are treated as invalid.
- Memory allocated via SET WORKSPACE is unavailable to extension commands; when running PLS on large datasets, you may actually *lower* the size of your workspace.

Syntax Rules

- The PLS command is required. All subcommands are optional.
- Only a single instance of each subcommand is allowed.
- An error occurs if an attribute or keyword is specified more than once within a subcommand.
- Equals signs and parentheses shown in the syntax chart are required.
- Subcommand names and keywords must be spelled in full.
- Empty subcommands are not allowed.

Examples

```
PLS Response WITH Price.
```

- PLS estimates a univariate prediction model that regresses *Response* on *Price*.
- Measurement level is not explicitly specified for dependent variable *Response*. Thus, if *Response* is categorical (nominal or ordinal), a classification model is estimated; otherwise, if *Response* is scale, a regression model is estimated.
- Variable *Price* is treated as a continuous predictor (covariate).

Classification Model (Explicitly Specified)

```
PLS Response MLEVEL=N REFERENCE=5 BY Region WITH Price.
```

- PLS estimates a univariate prediction model.
- Since dependent variable *Response* is specified as nominal (N), a classification model is estimated. Value 5 is used as the reference category.

- Variable *Region* is treated as a categorical predictor and *Price* is treated as a continuous predictor (covariate). By default, a main effects model is estimated.

Multivariate Model

```
PLS Q1 MLEVEL=S Q2 MLEVEL=S BY Region Edlevel
  /MODEL Region Edlevel Region*Edlevel.
```

- PLS estimates a multivariate regression model.
- Both dependent variables, *Q1* and *Q2*, are treated as scale (continuous) variables.
- Variables *Region* and *Edlevel* are treated as categorical predictors.
- MODEL specifies a fully factorial ANOVA model that includes main effects for *Region* and *Edlevel* as well as the interaction between the two predictors.

Variable Lists

The command line variable lists specify the dependent variables, any categorical predictors (factors), and any scale predictors (covariates). ALL and TO keywords are supported in all variable lists.

Dependent Variables

A list of one or more dependent variables must be the first specification on the PLS command.

- Dependent variables can be numeric or string.
- If a dependent variable is specified more than once, only the last specification is honored for the variable.

MLEVEL Keyword

By default, the measurement level recorded in the data dictionary is honored for dependent variables.

- The MLEVEL keyword can be specified after the name of any dependent variable to override its dictionary level of measurement. Specify N (nominal), O (ordinal), or S (scale). Note that the PLS procedure does not distinguish between ordinal and nominal dependent variables; it treats both as categorical.
- If MLEVEL is not specified and no setting is recorded in the data dictionary, then a numeric variable is treated as scale and a string variable is treated as categorical.
- A string dependent variable may be defined as ordinal or nominal only.
- MLEVEL applies only to the variable that immediately precedes it in the dependent variable list. For example, `Age Income [S]` and `Age TO Income [S]` assign the scale level of measurement to *Income* only. The keyword ALL followed by a measurement level specification applies that measurement level to all dependent variables.

REFERENCE Keyword

The REFERENCE keyword specifies the value of the dependent variable to use as the reference category for parameter estimation. REFERENCE can be specified after the name of any dependent variables but is honored only for categorical dependent variables. It is silently ignored for scale variables.

- Specify FIRST, LAST, or a value. If you specify FIRST, the lowest value is treated as the reference category. LAST (the default) treats highest value of the dependent variable as the reference category.
- If you specify a value it must exist in the data and be consistent with the type and format of the dependent variable. String and formatted (for example, date or time) values must be quoted.
- REFERENCE applies only to the variable that immediately precedes it in the dependent variable list. The keyword ALL followed by a REFERENCE specification applies that reference category to all categorical dependent variables.

Predictor Variables

- At least one predictor must be specified.

- The names of the categorical predictors, if any, must be preceded by the keyword BY. If keyword BY is specified with no factors, then a warning is issued and BY is ignored.
- The names of the covariates, if any, must be preceded by the keyword WITH. If keyword WITH is specified with no covariates, then a warning is issued and WITH is ignored.
- Categorical predictor variables can be numeric or string.
- Covariates must be numeric.
- If the dependent variable is specified within a categorical predictor list or a covariate list, then it is ignored in the list.
- All variables specified within a categorical predictor or covariate list must be unique. If duplicate variables are specified within a list, then the duplicates are ignored.
- If duplicate variables are specified across the predictor lists, then an error is issued.

ID Subcommand

The ID subcommand specifies a variable that is used as a unique identifier in casewise output and saved datasets (see the OUTDATASET Subcommand). If this option is not specified, case number is used as the identifier.

Specify a string or numeric variable. An error is issued if the identifier variable is specified as a dependent or predictor variable.

MODEL Subcommand

The MODEL subcommand is used to specify model effects.

- Specify a list of terms to be included in the model.
- If the MODEL subcommand is not specified, the default model includes main effects for any categorical predictors and covariates.
- To include a term for the main effect of a categorical predictor or covariate, enter the name of the variable.
- To include a term for an interaction among predictors, use the keyword BY or the asterisk (*) to join the variables involved in the interaction. For example, A*B means a two-way interaction effect of A and B. A*A is not allowed because variables inside an interaction effect must be distinct.
- Interactions among covariates are valid, as are interactions between categorical predictors and covariates.
- Nested terms are not supported in the PLS procedure.

OUTDATASET Subcommand

The OUTDATASET subcommand requests output datasets and plots for model estimates. You can obtain casewise, factorwise, and predictorwise results. None are produced by default.

CASES Keyword

CASES saves the following caseswise model estimates: predicted values, residuals, distance to latent factor model, and latent factor scores. It also plots latent factor scores.

Specify the name of the IBM SPSS Statistics dataset. The dataset name must be unique within an invocation of the PLS procedure. If you specify the name of an existing dataset, its contents are replaced; otherwise, a new dataset is created. An error is generated if an external file is specified.

LATENTFACTORS Keyword

LATENTFACTORS saves latent factor loadings and latent factor weights. It also plots latent factor weights.

Specify the name of the IBM SPSS Statistics dataset. The dataset name must be unique within an invocation of the PLS procedure. If you specify the name of an existing dataset, its contents are replaced; otherwise, a new dataset is created. An error is generated if an external file is specified.

PREDICTORS Keyword

PREDICTORS saves regression parameter estimates and variable importance to projection (VIP). It also plots VIP by latent factor.

Specify the name of the IBM SPSS Statistics dataset. The dataset name must be unique within an invocation of the PLS procedure. If you specify the name of an existing dataset, its contents are replaced; otherwise, a new dataset is created. An error is generated if an external file is specified.

CRITERIA Subcommand

The CRITERIA subcommand specifies model estimation criteria.

LATENTFACTORS Keyword

The LATENTFACTORS keyword specifies an upper limit on the number of latent factors that are extracted. By default, a maximum of five factors are extracted. The value must be a positive integer.

The number of latent factors that is actually extracted may be fewer than the number requested. For example, it is limited by the number of cases and predictors (whichever is smaller).

PLUM

PLUM is available in Statistics Base Edition.

```
PLUM dependent variable [BY factor varlist] [WITH covariate varlist]

[/CRITERIA = [CIN({95** })] [DELTA({0**   })] [MXITER({100**})] [MXSTEP({5**})]
                {value}          {value }          {n    }          {n   }
             [LCONVERGE({0**  })] [PCONVERGE({1.0E-6**})] [SINGULAR({1.0E-8**})]
                        {value}              {value   }              {value   }
             [BIAS] ]

[/LINK = {CAUCHIT}]
         {CLOGLOG}
         {LOGIT**}
         {NLOGLOG}
         {PROBIT }

[/LOCATION = [effect effect ...] ]

[/MISSING = {EXCLUDE**}]
            {INCLUDE  }

[/PRINT = [CELLINFO] [CORB] [COVB] [FIT] [HISTORY({1})] [KERNEL]
                                                 {n}
          [TPARALLEL] [PARAMETER] [SUMMARY]]

[/SAVE = [ESTPROB [(rootname [:{25**}])] [PREDCAT [(newname)]] [PCPROB [(newname)]]
                                 {n   }    [ACPROB [(newname)] ]

[/SCALE = [effect effect ...] ]

[/TEST [(valuelist)] = ['label'] effect valuelist [effect valuelist] ...;
       [effect valuelist [effect valuelist] ...;] ... ]

[/TEST [(valuelist)] = ['label'] ALL list; [ALL list;] ... ].
```

** Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
PLUM chist.
```

Overview

This procedure makes use of a general class of models to allow you to analyze the relationship between a polytomous ordinal dependent variable and a set of predictors. These models utilize the ordinal nature of the dependent variable and eliminate the need for rescaling.

Options

Link Functions. Five link functions are available for specifying the model with the LINK subcommand.

Tuning the Algorithm. You can control the values of algorithm-tuning parameters with the CRITERIA subcommand.

Optional Output. You can request additional output through the PRINT subcommand.

Basic Specification

The basic specification is one dependent variable.

Syntax Rules

- A minimum of one dependent variable must be specified.
- The variable specification must come first and can be specified only once.
- Subcommands can be specified in any order.
- When subcommands (except the TEST subcommand) are repeated, previous specifications are discarded and the last subcommand is in effect.

- Empty subcommands (except the LOCATION and the SCALE subcommands) are ignored. An empty LOCATION or SCALE subcommand indicates a simple additive model.
- The words BY, WITH, and WITHIN are reserved keywords in this procedure.

Variable List

The variable list specifies the dependent variable, factors, and covariates in the model.

- The dependent variable must be the first specification on the command line.
- The dependent variable is assumed to be an ordinal variable and can be of any type (numeric versus string). The order is determined by sorting the level of the dependent variable in ascending order. The lowest value defines the first category.
- Factor variables can be of any type (numeric versus string). Factor levels are sorted in ascending order. The lowest value defines the first category.
- Covariate variables must be numeric.
- Names of the factors follow the dependent variable separated by the keyword BY.
- Enter the covariates, if any, following the factors. Use the keyword WITH to separate covariates from factors (if any) and the dependent variable.

Weight Variable

- If a WEIGHT variable is specified, this procedure will take the non-missing weight values, rounded to the nearest integer, as frequencies.
- Cases with negative frequencies are always excluded.

CRITERIA Subcommand

The CRITERIA subcommand offers controls on the iterative algorithm used for estimation, specifies numerical tolerance for checking singularity, and offers options to customize your output.

BIAS. *Bias value added to all observed cell frequencies*. Specify a non-negative value less than 1. The default value is 0.0.

CIN. *Confidence interval level.* Specify a value greater than or equal to 0 and less than 100. The default value is 95.

DELTA. *Delta value added to observed zero frequency.* Specify a non-negative value less than 1. The default value is 0.0.

LCONVERGE. *Log-likelihood function convergence criterion.* Convergence is assumed if the absolute change or relative change in the log-likelihood function is less than this value. The criterion is not used if the value is 0. Specify a non-negative value. The default value is 0.

MXITER. *Maximum number of iterations.* Specify a non-negative integer. The default value is 100. Specifying 0 gives the initial estimates.

MXSTEP. *Maximum step-halving allowed.* Specify a positive integer. The default value is 5.

PCONVERGE. *Parameter estimates convergence criterion.* Convergence is assumed if the maximum absolute change in each of the parameter estimates is less than this value. The criterion is not used if the value is 0. Specify a non-negative value. The default value is 10^{-6}.

SINGULAR. *Value used as tolerance in checking singularity.* Specify a positive value. The default value is 10^{-8}.

LINK Subcommand

The LINK subcommand offers five link functions to specify the model.

- If LINK is not specified, LOGIT is the default.
- The five keywords are mutually exclusive. Only one of them can be specified and only once.

CAUCHIT. *Cauchit function.* $f(x) = \tan(\pi(x - 0.5))$.

CLOGLOG. *Complementary log-log function.* $f(x) = \log(-\log(1 - x))$.

LOGIT. *Logit function.* $f(x) = \log(x / (1 - x))$. This is the default link function.

NLOGLOG. *Negative log-log function.* $f(x) = -\log(-\log(x))$.

PROBIT. *Probit function.* $f(x) = \Phi^{-1}(x)$, where Φ^{-1} is the inverse standard normal cumulative distribution function.

LOCATION Subcommand

The LOCATION subcommand specifies the location model.

- Specify a list of terms to be included in the location model, separated by commas or spaces.
- The default location model is generated if the subcommand is not specified or empty. The default model contains the intercept, all of the covariates (if specified) in the order in which they are specified, and all of the main factorial effects in the order in which they are specified on the variable list.
- To include the intercept term explicitly, enter the keyword INTERCEPT on the subcommand.
- To include a main effect term, enter the name of the factor on the subcommand.
- To include an interaction effect term among factors, use the keyword BY or the asterisk (*) to join factors involved in the interaction. For example, A*B*C means a three-way interaction effect of A, B, and C, where A, B, and C are factors. The expression A BY B BY C is equivalent to A*B*C. Factors inside an interaction effect must be distinct. Expressions such as A*C*A and A*A are invalid. The keyword INTERCEPT cannot be used to construct an interaction term.
- To include a nested effect term, use the keyword WITHIN or a pair of parentheses on the subcommand. For example, A(B) means that A is nested within B, where A and B are factors. The expression A WITHIN B is equivalent to A(B). Factors inside a nested effect must be distinct. Expressions such as A(A) and A(B*A) are invalid.
- Multiple level nesting is supported. For example, A(B(C)) means that B is nested within C, and A is nested within B(C). When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, A(B)(C) is not valid.
- Nesting within an interaction effect is valid. For example, A(B*C) means that A is nested within B*C.
- Interactions among nested effects are allowed. The correct syntax is the interaction followed by the common nested effect inside the parentheses. For example, interaction between A and B within levels of C should be specified as A*B(C) instead of A(C)*B(C).
- To include a covariate term in the model, enter the name of the covariate on the subcommand.
- Covariates can be connected, but not nested, using the keyword BY or the asterisk (*) operator. For example, X*X is the product of X and itself. This is equivalent to a covariate whose values are the square of those of X. On the contrary, X(Y) is invalid.
- Factor and covariate effects can be connected in many ways. No effects can be nested within a covariate effect. Suppose A and B are factors and X and Y are covariates. Examples of valid combination of factor and covariate effects are A*X, A*B*X, X(A), X(A*B), X*A(B), X*Y(A*B), and A*B*X*Y.

Example

```
PLUM
  chist  BY numcred othnstal
  /CRITERIA = CIN(95) DELTA(0) LCONVERGE(0) MXITER(100) MXSTEP(5)
```

```
              PCONVERGE(0)
  /LOCATION = numcred othnstal numcred*othnstal.
```

- LOCATION specifies that the location model consists of *numcred*, *othnstal*, and their interaction effect.

MISSING Subcommand

By default, cases with missing values for any of the variables on the variable list are excluded from the analysis. The MISSING subcommand allows you to include cases with user-missing values.

- If MISSING is not specified, the default is EXCLUDE.
- Listwise deletion is always used in this procedure.
- Keywords EXCLUDE and INCLUDE are mutually exclusive. Only one of them can be specified and only once.

EXCLUDE. *Exclude both user-missing and system-missing values.* This is the default.

INCLUDE. *User-missing values are treated as valid.* System-missing values cannot be included in the analysis.

PRINT Subcommand

The PRINT subcommand controls the display of optional output. If no PRINT subcommand is specified, default output includes a case-processing summary table.

CELLINFO. *Cell information.* Observed and expected frequencies by category and cumulative, Pearson residual for cumulative and category frequencies, and observed and expected probabilities of each response category separately and cumulatively by covariate pattern combination.

CORB. *Asymptotic correlation matrix of the parameter estimates.*

COVB. *Asymptotic covariance matrix of the parameter estimates.*

FIT. *Goodness-of-fit statistics.* The Pearson chi-square and the likelihood-ratio chi-square statistics. The statistics are computed based on the classification specified on the variable list.

HISTORY. *Iteration history.* The table contains log-likelihood function value and parameter estimates every *n* iterations. The default value is *n* = 1. The first and the last iterations are always printed if HISTORY is specified and regardless of the value of *n*.

KERNEL. *Use the kernel of the log-likelihood function for display instead of the complete log-likelihood function.*

TPARALLEL. *Test of parallel lines assumption.* Produce a chi-squared score test of the parallel lines assumption.

PARAMETER. *Parameter statistics.* The parameter estimates, the standard errors, the significances, and the confidence interval.

SUMMARY. *Model summary.* The Cox & Snell's R^2, the Nagelkerke's R^2, and the McFadden's R^2 statistics.

SAVE Subcommand

The SAVE subcommand puts casewise post-estimation statistics back into the active file.

- The new variables must have valid variable names that are not in use in the working file.
- The rootname must be a valid variable name.
- The new variables are saved to the working file in the order the keywords are specified on the subcommand.

ESTPROB. *Estimated probabilities of classifying a factor/covariate pattern into the response categories.* The predicted probabilities of the first *n* categories are saved. The default number of categories is 25. To specify a number of categories without a rootname, put a colon before the number.

PREDCAT. *The response category that has the maximum expected probability for a factor/covariate pattern.*

PCPROB. *Estimated probability of classifying a factor/covariate pattern into the predicted category.* This probability is the maximum of the estimated probabilities of the factor/covariate pattern.

ACPROB. *Estimated probability of classifying a factor/covariate pattern into the actual category.*

Example

```
PLUM
  chist  BY numcred othnstal
  /CRITERIA = CIN(95) DELTA(0) LCONVERGE(0) MXITER(100) MXSTEP(5)
              PCONVERGE(0)
  /SAVE = ACPROB(correct) PRPROB.
```

- SAVE specifies that the estimated probabilities of correctly classifying each case should be saved to the variable *correct*. The estimated probabilities of classifying each case into the predicted category are saved to the default variable *pcp_k*, where *k* is the smallest integer for which *pcp_k* does not already exist.

SCALE Subcommand

The SCALE subcommand specifies the scale component in the model.

- Specify a list of terms to be included in the model, separated by commas or spaces.
- The model will have no scale component if the subcommand is omitted.
- No scale component is generated if the subcommand is not specified or empty.
- To include a main effect term, enter the name of the factor on the subcommand.
- The keyword `INTERCEPT` is not allowed on the subcommand.
- To include an interaction effect term among factors, use the keyword BY or the asterisk (*) to join factors involved in the interaction. For example, A*B*C means a three-way interaction effect of A, B, and C, where A, B, and C are factors. The expression A BY B BY C is equivalent to A*B*C. Factors inside an interaction effect must be distinct. Expressions such as A*C*A and A*A are invalid.
- To include a nested effect term, use the keyword `WITHIN` or a pair of parentheses on the subcommand. For example, A(B) means that A is nested within B, where A and B are factors. The expression A `WITHIN` B is equivalent to A(B). Factors inside a nested effect must be distinct. Expressions such as A(A) and A(B*A) are invalid.
- Multiple level nesting is supported. For example, A(B(C)) means that B is nested within C, and A is nested within B(C). When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, A(B)(C) is not valid.
- Nesting within an interaction effect is valid. For example, A(B*C) means that A is nested within B*C.
- Interactions among nested effects are allowed. The correct syntax is the interaction followed by the common nested effect inside the parentheses. For example, interaction between A and B within levels of C should be specified as A*B(C) instead of A(C)*B(C).
- To include a covariate term in the model, enter the name of the covariate on the subcommand.
- Covariates can be connected, but not nested, using the keyword BY or the asterisk (*) operator. For example, X*X is the product of X and itself. This is equivalent to a covariate whose values are the square of those of X. On the contrary, X(Y) is invalid.
- Factor and covariate effects can be connected in many ways. No effects can be nested within a covariate effect. Suppose A and B are factors, and X and Y are covariates. Examples of valid combination of factor and covariate effects are A*X, A*B*X, X(A), X(A*B), X*A(B), X*Y(A*B), and A*B*X*Y.

TEST Subcommand

The TEST subcommand allows you to customize your hypothesis tests by directly specifying null hypotheses as linear combinations of parameters.

- TEST is offered only through syntax.
- Multiple TEST subcommands are allowed. Each is handled independently.
- The basic format of the TEST subcommand is an optional list of values enclosed in a pair of parentheses, an optional label in quotes, an effect name or the keyword ALL, and a list of values.
- To specify the coefficient for the intercept, use the keyword INTERCEPT. The number of values after INTERCEPT must be equal to the number of response categories minus 1.
- When multiple linear combinations are specified within the same TEST subcommand, a semicolon terminates each linear combination, except the last one.
- The linear combinations are separately tested for each category of the dependent variable and then simultaneously tested for all the categories.
- If specified, the value list that immediately follows the subcommand name is the constant that the linear combinations are equated to under the null hypotheses. If this value list is omitted, the constants are assumed to be all zeros.
- The optional label is a string with a maximum length of 255 bytes. Only one label per TEST subcommand can be specified.
- Only valid effects appearing or implied on the LOCATION or the SCALE subcommands can be specified in a linear combination. If an effect appears in both subcommands, then enter the effect only once on the TEST subcommand.
- To specify coefficient for the intercept, use the keyword INTERCEPT. Only one value is expected to follow INTERCEPT.
- The number of values following an effect name must equal the number of parameters (including the redundant ones) corresponding to that effect. For example, if the effect A*B takes up six parameters, then exactly six values must follow A*B.
- A number can be specified as a fraction with a positive denominator. For example, 1/3 or –1/3 are valid, but 1/–3 is invalid.
- When ALL is specified, only a list of values can follow. The number of values must equal the combined number of LOCATION and SCALE parameters (including the redundant ones).
- Effects appearing or implied on the LOCATION or the SCALE subcommands but not specified on the TEST are assumed to take the value 0 for all their parameters.
- Effect names and the ALL keywords are mutually exclusive within a single TEST subcommand.
- If ALL is specified for the first row in a TEST matrix, then all subsequent rows should begin with the ALL keyword.
- If effects are specified for the first row in a TEST matrix, then all subsequent rows should use effect name (thus ALL is not allowed).

Example

```
PLUM
  chist  BY housng
  /CRITERIA = CIN(95) DELTA(0) LCONVERGE(0) MXITER(100) MXSTEP(5)
              PCONVERGE(1.0E-6) SINGULAR(1.0E-8)
  /LINK = CLOGLOG
  /PRINT = CELLINFO CORB COVB FIT HISTORY(1) PARAMETER
           SUMMARY TPARALLEL
  /TEST(0 0) =  ALL 1 -1 0 0 0 0 0;
                ALL 0 0 1 -1 0 0 0.
```

- There are a total of seven parameter coefficients in the model; four for the thresholds, and three for the factor *housng*. TEST specifies two separate tests: one in which the first and second thresholds are tested for equality, and one in which the third and fourth thresholds are tested for equality.

POINT

```
POINT KEY=varname
  [FILE='file'] [ENCODING='encoding specification']
```

Release History

Release 16.0

- ENCODING subcommand added for Unicode support.

Example

```
POINT FILE=DRIVERS /KEY=#FRSTAGE.
```

Overview

POINT establishes the location at which sequential access begins (or resumes) in a keyed file. A keyed file is a file that provides access to information by a record key. An example of a keyed file is a file containing a social security number and other information about a firm's employees. The social security number can be used to identify the records in the file. For additional information about keyed files, see KEYED DATA LIST.

POINT prepares for reading the key-sequenced dataset sequentially from a point that the key value controls. Data selection commands can then be used to limit the file to the portion that you want to analyze. A DATA LIST command is used to read the data. To read keyed files (and also direct-access files), see KEYED DATA LIST.

Basic Specification

The basic specification is the KEY subcommand and a string variable. The value of the string variable is used as the file key for determining where sequential retrieval (via DATA LIST) begins or resumes.

Subcommand Order

- Subcommands can be named in any order.
- Each POINT command must precede its corresponding DATA LIST command.

Syntax Rules

- POINT can be used more than once to change the order of retrieval during processing.
- POINT must be specified in an input program and therefore cannot be used to add cases to an existing file.

Operations

- The next DATA LIST command that is executed after the POINT command (for the same file) will read a record whose key value is at least as large as the value of the specified key. To prevent an infinite loop in which the same record is read again and again, the value of the variable that is specified on KEY must change from case to case, or the POINT command must be set up to execute only once.
- If the file contains a record whose key exactly matches the value of the KEY variable, the next execution of DATA LIST will read that record, the second execution of DATA LIST will read the next record, and so on.
- If an exact match between key and variable is not found, the results depend on the operating system. On IBM implementations, reading begins or resumes at the record that has the next higher key. If the value of the key is shorter than the file key, the value of the key variable is logically extended with the lowest character in the collating sequence. For example, if the value of the key variable is the single letter M, retrieval begins or resumes at the first record that has a key (regardless of length) beginning with the letter M or a character that is higher in the collating sequence.

- POINT does not report on whether the file contains a record that exactly matches the specified key. To check for missing records, use LIST to display the data that were read by the subsequent DATA LIST command.

Examples

Basic Example

```
FILE HANDLE DRIVERS/ file specifications.
POINT FILE=DRIVERS /KEY=#FRSTAGE.
```

- FILE HANDLE defines the handle for the data file to be read by POINT. The handle is specified on the FILE subcommand on POINT.
- KEY on POINT specifies the key variable. The key variable must be a string, and it must already exist as the result of a prior DATA LIST, KEYED DATA LIST, or transformation command.

Selecting a Subset of Records from a Keyed File

```
FILE HANDLE    DRIVERS/ file specifications.
INPUT PROGRAM.
STRING         #FRSTAGE(A2).
DO IF          #FRSTAGE = ' '.     /* First case check
+  COMPUTE     #FRSTAGE = '26'.    /* Initial key
+  POINT       FILE=DRIVERS /KEY=#FRSTAGE.
END IF.
DATA LIST      FILE=DRIVERS NOTABLE/
               AGE 19-20(A) SEX 21(A) TICKETS 12-13.
DO IF          AGE > '30'.
+  END FILE.
END IF.
END INPUT PROGRAM.
LIST.
```

- This example illustrates how to execute POINT for only the first case. The file contains information about traffic violations, and it uses the individual's age as the key. Ages between 26 and 30 are selected.
- FILE HANDLE specifies the file handle DRIVERS.
- The INPUT PROGRAM and END INPUT PROGRAM commands begin and end the block of commands that build cases. POINT must appear in an input program.
- STRING declares the string variable *#FRSTAGE*, whose value will be used as the key on the POINT command. Because *#FRSTAGE* is a string variable, it is initialized as blanks.
- The first DO IF-END IF structure is executed only if no records have been read (that is, when *#FRSTAGE* is blank). When *#FRSTAGE* is blank, COMPUTE resets *#FRSTAGE* to 26, which is the initial value. POINT is executed, and it causes the first execution of DATA LIST to read a record whose key is at least 26. Because the value of *#FRSTAGE* is now 26, the DO IF-END IF structure is not executed again.
- DATA LIST reads the variables *AGE*, *SEX*, and *TICKETS* from the file *DRIVERS*.
- The second DO IF–END IF structure executes an END FILE command as soon as a record is read that contains a driver's age that is greater than 30. The program does not add this last case to the working file when it ends the file (see END FILE).

FILE Subcommand

FILE specifies a file handle for the keyed data file. The file handle must have been previously defined on a FILE HANDLE command.

- FILE is optional.
- If FILE is omitted, POINT reads from the last file that is specified on an input command, such as DATA LIST.

Example

```
FILE HANDLE DRIVERS/ file specifications.
POINT FILE=DRIVERS /KEY=#NXTCASE.
```

- FILE HANDLE specifies *DRIVERS* as the file handle for the data. The FILE subcommand on POINT specifies file handle *DRIVERS*.

ENCODING Subcommand

ENCODING specifies the encoding format of the file. The keyword is followed by an equals sign and a quoted encoding specification.

- In Unicode mode, the default is UTF8. For more information, see SET command, UNICODE subcommand.
- In code page mode, the default is the current locale setting. For more information, see SET command, LOCALE subcommand.
- The quoted encoding value can be: Locale (the current locale setting), UTF8, UTF16, UTF16BE (big endian), UTF16LE (little endian), a numeric Windows code page value (for example, '1252'), or an IANA code page value (for example, 'iso8859-1' or cp1252).
- In Unicode mode, the defined width of string variables is tripled for code page and UTF-16 text data files. Use ALTER TYPE to automatically adjust the defined width of string variables.

KEY Subcommand

KEY specifies the variable whose value will be used as the file key for determining where sequential retrieval by DATA LIST will begin or resume. This variable must be a string variable, and it must already exist as the result of a prior DATA LIST, KEYED DATA LIST, or transformation command.

- KEY is required. Its only specification is a single variable. The variable can be a permanent variable or a scratch variable.
- Although the keys on a file are inherently numbers, such as social security numbers, the STRING function can be used to convert the numeric variable to a string. See the topic “String/numeric conversion functions” on page 81 for more information.

Example

```
FILE HANDLE DRIVERS/ file specifications.
POINT FILE=DRIVERS /KEY=#NXTCASE.
```

- KEY indicates that the value of the existing scratch variable *#FRSTAGE* will be used as the key to reading each record.
- Variable *#FRSTAGE* must be an existing string variable.

POWER ONEWAY ANOVA

POWER ONEWAY ANOVA is available in IBM SPSS Statistics Base Edition. The procedure estimates the power for two types of hypothesis to compare the multiple group means, the overall test, and the test with specified contrasts. The over test focuses on the null hypothesis that all group means are equal. The test with specified contrasts breaks down the overall ANOVA hypotheses into smaller but more describable and useful pieces of the means.

```
POWER ONEWAY ANOVA
     /PARAMETERS
          [ SIGNIFICANCE={0.05**} {value}] ]
          { GROUP_SIZES={integers} } {POWER={value} }
          POOLED_SD={value}
          GROUP_MEANS={values}
          [ GROUP_WEIGHTS={EQUAL**} {values} ]
     [ /CONTRAST
          [ COEFFICIENTS ={values}]
          [ TEST ={NONDIRECTIONAL**} {DIRECTIONAL}]
          [ PAIRWISE={FALSE**} {TRUE}]
          [ ADJUST={BONFERRONI**} {SIDAK} {LSD}] ]
     [ /PLOT
          [ TOTAL_N([integer1 integer2]) ]
          [ POOLED_SD([value1 value2]) ]
          [ TOTAL_N_XAXIS([integer1 integer2]) POOLED_SD_YAXIS([value1 value2]) ]
          [ TOTAL_N_YAXIS([integer1 integer2]) POOLED_SD_XAXIS([value1 value2]) ]
          [ VROTATE({10**} {integer}) ]
          [ HROTATE({325**} {integer}) ] ]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 27.0

- Command introduced

Overview

POWER ONEWAY ANOVA is an extension of the two sample *t*-test procedure, and focuses on the one-way ANOVA and statistical inference about population means for multiple groups. The procedure estimates the power for two types of hypothesis to compare the multiple group means, the overall test, and the test with specified contrasts. The over test focuses on the null hypothesis that all group means are equal. The test with specified contrasts breaks down the overall ANOVA hypotheses into smaller but more describable and useful pieces of the means.

Options

Test Assumptions
Specifies the keywords that control the test assumptions. The keywords specify the group sizes, the pooled population standard deviation, and the group means.

Significance Level
Specifies the significance level of the test's Type I error rate.

Contrast Test
Specifies the contrast coefficients and requests the contrast test and determines whether the contrast test is one or two sided.

Pairwise Differences
Controls whether or not to estimate the power of testing for the pairwise differences, and determines the adjustment of multiple comparisons.

Two-Dimensional Plot
Controls the two-dimensional power by total sample size chart, and the two-dimensional power by pooled standard deviation chart.

Three-Dimensional Plot

Controls the three-dimensional power by total sample size (x-axis) and pooled standard deviation (y-axis) chart, and the three-dimensional power by total sample size (y-axis) and pooled standard deviation (x-axis) chart.

Basic specification

- POWER ONEWAY ANOVA: Invokes the Power Analysis of One-way ANOVA with Contrasts procedure.

Subcommand order

- The subcommands can be named in any order.

Example

```
POWER ONEWAY ANOVA
  /PARAMETERS GROUP_SIZES=2 2 POOLED_SD=1 GROUP_MEANS=2 2 SIGNIFICANCE=0.05
  /CONTRAST PAIRWISE=TRUE ADJUST=BONFERRONI
  /PLOT TOTAL_N TOTAL_N_YAXIS POOLED_SD_XAXIS VROTATE(10) HROTATE(325).
```

- **PARAMETERS** specifies that both group sizes are set to 2, the pooled population standard deviation is set to 1, both group means are set to 2, and the significance level uses the default 0.05 value.
- **CONTRAST** specifies that estimating the power of testing for the pairwise differences is enabled and the adjustment of multiple comparisons uses the Bonferroni correction in estimating the power of pairwise differences.
- **PLOT** specifies that the two-dimensional power by total sample size chart displays in the output. The three-dimensional power by total sample size (x -axis) and pooled standard deviation (y-axis) chart also displays in the output. The vertical and horizontal rotation settings use their default values (10 degrees and 325 degrees respectively).

PARAMETERS Subcommand

PARAMETERS controls the test assumptions. The keywords specify the group sizes, the pooled population standard deviation, and the group means.

SIGNIFICANCE = value

An optional keyword that specifies the significance level of the Type I error rate for the test. The value must be a single double value $\in(0,1)$. The default level is 0.05.

GROUP_SIZES = integers

A required keyword that specifies the group sizes. At least two integers are required. Each integer must be ≥ 2. Multiple integers are separated by a blank space (or blank spaces). The number of specified integers must be equal to that for **GROUP_MEANS**.

POWER=value

A required keyword that specifies the power for sample size estimation. The value must be a single value $\in(0,1)$. **POWER** and **GROUP_SIZES** cannot be specified simultaneously.

POOLED_SD = value

A required keyword that specifies the pooled population standard deviation. The value must be a single, numeric value > 0.

GROUP_MEANS = values

A required keyword that specifies the group means. At least two values are required. Multiple values are separated by a blank space (or blank spaces). The number of specified values must be equal to that for **GROUP_SIZES**.

GROUP_WEIGHTS = values

A required keyword that assigns the group size weights when **POWER** is specified. The keyword takes effect when **POWER** is specified, and is ignored when **GROUP_SIZES** is specified. The default setting is **EQUAL**, which assigns the group weights all equal 1. The keyword can also be reset by specifying

multiple values, separated by a blank space (or blank spaces). Each value must be ∈[1,100]. The number of specified values must be equal to that for **GROUP_MEANS**.

CONTRAST Subcommand

The optional CONTRAST subcommand specifies the contrasts and pairwise differences.

COEFFICIENTS = values
The optional keyword specifies the contrast coefficients and requests the contrast test. The keyword is omitted by default. When the keyword is specified, the number of specified values must be equal to that for **/PARAMETERS GROUP_SIZES** or **/PARAMETERS GROUP_MEANS**. All of the specified values must be summed up to 0, otherwise the last value is automatically adjusted.

TEST = {NONDIRECTIONAL}{DIRECTIONAL}
The optional keyword determines whether the contrast test is one or two-sided. **NONDIRECTIONAL**, or a two-sided test, is the default setting. When **DIRECTIONAL** is specified, power is computed for a one-sided test. The keyword takes effect when **/CONTRAST COEFFICIENTS** is specified.

PAIRWISE = boolean
The optional keyword controls whether or not to estimate the power of testing for the pairwise differences. **FALSE** is the default setting, which suppresses output for the pairwise differences. When **TRUE** is specified Table 3 is required and displays. In cases where **COEFFICIENTS** is omitted and **PAIRWISE = FALSE**, a warning message is created that indicates:

```
COEFFICIENTS is required
```

ADJUST = {BONFERRONI} {SIDAK} {LSD}
The optional keyword determines the adjustment of multiple comparisons. **BONFERRONI** is the default setting, which uses the Bonferroni correction in estimating the power of pairwise differences. **SIDAK** and **LSD** use the Sidak correction and the least significant difference, respectively. The keyword takes effect when **/CONTRAST PAIRWISE = TRUE** is specified.

PLOT Subcommand

The optional PLOT subcommand controls the output charts. By default, no charts are created. When the subcommand is specified, only the outputs for the following keywords are created.

TOTAL_N(integer1 integer2)
The optional keyword controls the two-dimensional power by total sample size chart. By default, the chart is suppressed. If specified, the chart will be displayed. When no integers are specified, the default plot range of the sample size ratio is used. When the keyword is specified, the plot boundaries are determined by **integer1** and **integer2**, respectively. Both **integer1** and **integer2** are integers, separated by a blank space (or blank spaces), and must satisfy integer1 ≥ 2* the number of integers specified for **/PARAMETERS GROUP_SIZES**, integer1 < integer2, and integer2 ≤ 5000. You can either omit or specify both numeric values.

POOLED_SD(value1 value2)
The optional keyword controls the two-dimensional power by pooled standard deviation chart. The chart is suppressed by default. When the keyword is specified, the chart displays. When no values are specified, the default plot range of the pooled standard deviation is used. When values are specified, the plot boundaries are determined by **value1** and **value2**, respectively. Both **value1** and **value2** are numeric values, separated by a blank space (or blank spaces) and must satisfy 0 < value1 < value2. You can either omit or specify both numeric values.

TOTAL_N_XAXIS(integer1 integer2) POOLED_SD_YAXIS(value1 value2)
The optional keyword controls the three-dimensional power by total sample size (x-axis) and pooled standard deviation (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: integer1, **integer2**, **value1** and **value2** follow the same rules as specified for **TOTAL_N** and **POOLED_SD**.

TOTAL_N_YAXIS(integer1 integer2) POOLED_SD_XAXIS(value1 value2)

The optional keyword controls the three-dimensional power by total sample size (y-axis) and pooled standard deviation (x-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: integer1, **integer2**, **value1** and **value2** follow the same rules as specified for **TOTAL_N** and **POOLED_SD**.

VROTATE(integer)

The optional keyword sets the vertical rotation degrees (clockwise from the left) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in$ `[0,359]` with `10` as the default setting.

HROTATE(integer)

The optional keyword sets the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in$ `[0,359]` with 325 as the default setting.

POWER MEANS INDEPENDENT

POWER MEANS INDEPENDENT is available in IBM SPSS Statistics Base Edition. The command invokes power analysis for the independent sample *t*-test to draw statistical inference about the difference of the two means. It is assumed that the data in each sample independently and identically follow a normal distribution with a fixed mean and variance.

```
POWER MEANS INDEPENDENT
     /PARAMETERS
          [ TEST={NONDIRECTIONAL**} {DIRECTIONAL}]
          [ SIGNIFICANCE={0.05**} {value}] ]
          { N={integer1 integer2} {POWER={value} }
          [ NRATIO={1**} {value} ]
          SD={value} {value1 value2}
          MEAN={value} {value1 value2}
     [ /PLOT
          [ NRATIO([value1 value2]) ]
          [ ES([value1 value2]) ]
          [ NRATIO_XAXIS([integer1 integer2]) ES_YAXIS([value1 value2]) ]
          [ NRATIO_YAXIS([integer1 integer2]) ES_XAXIS([value1 value2]) ]
          [ VROTATE({10**} {integer}) ]
          [ HROTATE({325**} {integer}) ] ]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 27.0

- Command introduced

Overview

POWER MEANS INDEPENDENT invokes power means analysis for the independent sample *t*-test.

Options

Test Assumptions
Specifies the keywords that control the test assumptions. The keywords specify the sample size, population standard deviation, and the population mean under testing. For more information, see "PARAMETERS Subcommand " on page 1452.

Significance Level
Specifies the significance level of the test's Type I error rate.

Test Direction
Determines whether the test is one or two-sided.

Two-Dimensional Plot
Controls the two-dimensional power by sample size ratio chart, and the two-dimensional power by effect size (or mean difference) chart.

Three-Dimensional Plot
Controls the three-dimensional power by sample size (x-axis) and effect size (y-axis) chart, and the three-dimensional power by sample size (y-axis) and effect size (x-axis) chart. Also controls the vertical rotation degrees (clockwise from the left) and the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart.

Basic specification

- POWER MEANS INDEPENDENT: Invokes the power analysis for the independent sample *t*-test.

Subcommand order

- The subcommands can be named in any order.

Example

```
POWER MEANS INDEPENDENT
  /PARAMETERS TEST=NONDIRECTIONAL SIGNIFICANCE=0.05 N=2 4 MEAN=1 SD=.5
  /PLOT ES NRATIO_YAXIS ES_XAXIS VROTATE(10) HROTATE(325).
```

- **PARAMETERS** specifies that the test is non-directional (two-sided), the significance level uses the default 0.05 value, the population mean(s) under testing is set to 1, and the population standard deviation is set to 0.5.
- **PLOT** specifies that the two-dimensional power by effect size (or mean difference) chart displays in the output. The three-dimensional power by sample size ratio (y-axis) and effect size (x -axis) chart also displays in the output. The vertical and horizontal rotation settings use their default values (10 degrees and 325 degrees respectively).

PARAMETERS Subcommand

PARAMETERS controls the test assumptions. The keywords specify the sample sizes, population standard deviations, and the population means under testing.

TEST = {NONDIRECTIONAL}{DIRECTIONAL}
An optional keyword that determines whether the test is one or two-sided. **NONDIRECTIONAL**, or a two-sided test, is the default setting. When **DIRECTIONAL** is specified, power is computed for a one-sided test.

SIGNIFICANCE = value
An optional keyword that specifies the significance level of the Type I error rate for the test. The value must be a single double value $\in$ `(0,1)`. The default level is `0.05`.

N = {integer1 integer2}
A required keyword that specifies the sample size for the two groups for comparison. The values for both integers must be > 1.

POWER = value
A required keyword that specifies the power for sample size estimation. The value must be a single value $\in$ `(0,1)`. **POWER** and **N** cannot be specified simultaneously.

NRATIO = {value}
A required keyword that specifies the ratio of the sample sizes. Must be a single value $\in$ `[0.01,100]` with 1 as the default setting. The keyword takes effect when **POWER** is specified, and is ignored when **N** is specified.

SD = {value}{value1 value2}
A required keyword that specifies the population standard deviation(s). Note that **value**, **value1**, and **value2** are all numeric values > 0. Multiple parameters **value1** and **value2**, if specified, are separated by a blank space (or blank spaces). When a single value is specified, it denotes the population standard deviation of the group difference σ, and assumes that the two group variances are equal, or $\sigma_1 = \sigma_2 = \sigma$. When two different values (**value1** and **value2**) are specified, they denote σ_1 and σ_2, respectively.

Note: When two identical **value1** and **value2** values are specified, the result is the same as a when a single value is specified for the keyword.

MEAN = {value}{value1 value2}
A required keyword that specifies the population mean(s) under testing. Note that **value**, **value1**, and **value2** are all numeric values. Multiple parameters **value1** and **value2**, if specified, are separated by a blank space (or blank spaces). When a single value is specified, it denotes the population mean difference μ_d. When two values (**value1** and **value2**) are specified, they denote μ_1 and μ_2, respectively.

PLOT Subcommand

The optional PLOT subcommand controls the output charts. By default, no charts are created. When the subcommand is specified, only the outputs for the following keywords are created.

NRATIO(value1 value2)
The optional keyword controls the two-dimensional power by sample size ratio chart. By default, the chart is suppressed. If specified, the chart will be displayed. When no integers are specified, the default plot range of the sample size ratio is used (depending on whether the group variances are assumed to be equal or not). When the keyword is specified, the plot boundaries are determined by **value1** and **value2**, respectively. Both **value1** and **value2** are numeric values, separated by a blank space (or blank spaces), and must satisfy `0.2 ≤ value1 < value2 ≤ 5.0`. You can either omit or specify both numeric values.

ES(value1 value2)
The optional keyword controls the two-dimensional power by effect size (or mean difference) chart. The chart is suppressed by default. When the keyword is specified, the chart displays. When no values are specified, the default plot range of the effect size (or mean difference) is used, depending wither the group variances are assumed to be equal or not. When values are specified, the plot boundaries are determined by **value1** and **value2**, respectively. Both **value1** and **value2** are numeric values, separated by a blank space (or blank spaces). For the equal group variances, it must satisfy `−5.0 ≤ value1 < value2 ≤ 5.0` when **/CRITERIA TEST = DIRECTIONAL**, or `0 ≤ value1 < value2 ≤ 5.0` when **/CRITERIA TEST = NONDIRECTIONAL**. For the unequal group variances, it must satisfy `0 ≤ value1 < value2 ≤ 5000`. You can either omit or specify both numeric values.

NRATIO_XAXIS(integer1 integer2) ES_YAXIS(value1 value2)
The optional keyword controls the three-dimensional power by sample size (x-axis) and effect size (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: value1 and **value2** follow the same rules as specified for **NRATIO** and **ES**.

NRATIO_YAXIS(integer1 integer2) ES_XAXIS(value1 value2)
The optional keyword controls the three-dimensional power by sample size (y-axis) and effect size (x-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: value1 and **value2** follow the same rules as specified for **NRATIO** and **ES**.

VROTATE(integer)
The optional keyword sets the vertical rotation degrees (clockwise from the left) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in$`[0,359]` with `10` as the default setting.

HROTATE(integer)
The optional keyword sets the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in$`[0,359]` with `325` as the default setting.

POWER MEANS ONESAMPLE

POWER MEANS ONESAMPLE is available in IBM SPSS Statistics Base Edition. The command invokes power analysis for the one sample *t*-test to draw statistical inference about the mean parameter. It is assumed that the sample data independently and identically follow a normal distribution with a fixed mean and variance.

```
POWER MEANS ONESAMPLE
     /PARAMETERS
          [ TEST={NONDIRECTIONAL**} {DIRECTIONAL} ]
          [ SIGNIFICANCE={0.05**} {value}] ]
          { N={integer} {POWER={value} }
          SD={value}
          MEAN={value}
          [ NULL={0**} {value} ]
     [ /PLOT
          [ N([integer1 integer2]) ]
          [ ES([value1 value2]) ]
          [ N_XAXIS([integer1 integer2]) ES_YAXIS([value1 value2]) ]
          [ N_YAXIS([integer1 integer2]) ES_XAXIS([value1 value2]) ]
          [ VROTATE({10**} {integer}) ]
          [ HROTATE({325**} {integer}) ] ]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 27.0

- Command introduced

Overview

POWER MEANS ONESAMPLE invokes the power means analysis for the one sample *t*-test.

Options

Test Assumptions
Specifies the keywords that control the test assumptions. The keywords specify the sample size, population standard deviation, population mean under testing, and hypothesized value to be tested. For more information, see "PARAMETERS Subcommand " on page 1456.

Significance Level
Specifies the significance level of the test's Type I error rate.

Test Direction
Determines whether the test is one or two-sided.

Two-Dimensional Plot
Controls the two-dimensional power by sample size chart, and the two-dimensional power by effect size chart.

Three-Dimensional Plot
Controls the three-dimensional power by sample size (x-axis) and effect size (y-axis) chart, and the three-dimensional power by sample size (y-axis) and effect size (x-axis) chart. Also controls the vertical rotation degrees (clockwise from the left) and the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart.

Basic specification

- POWER MEANS ONESAMPLE: Invokes the power analysis for the one sample *t*-test.

Subcommand order

- The subcommands can be named in any order.

Example

```
POWER MEANS ONESAMPLE
  /PARAMETERS TEST=NONDIRECTIONAL SIGNIFICANCE=0.05 N=2 SD=1 MEAN=1 NULL=.5
  /PLOT N(2 5) N_YAXIS(2 5) ES_XAXIS VROTATE(10) HROTATE(325).
```

- **PARAMETERS** specifies that the test is non-directional (two-sided), the significance level uses the default 0.05 value, the sample size for power estimation is set to 1, the population standard deviation is set to 0.5, the population mean(s) under testing is set to 1, and the null hypothesis value to be tested is set to 0.5.
- **PLOT** specifies that the two-dimensional power by sample size chart displays in the output. The three-dimensional power by sample size (x -axis) and effect size (y-axis) chart also displays in the output. The vertical and horizontal rotation settings use their default values (10 degrees and 325 degrees respectively).

PARAMETERS Subcommand

PARAMETERS controls the test assumptions. The keywords specify the sample size, population standard deviation, population mean under testing, and hypothesized value to be tested.

TEST = {NONDIRECTIONAL}{DIRECTIONAL}
An optional keyword that determines whether the test is one or two-sided. **NONDIRECTIONAL**, or a two-sided test, is the default setting. When **DIRECTIONAL** is specified, power is computed for a one-sided test.

SIGNIFICANCE = value
An optional keyword that specifies the significance level of the Type I error rate for the test. The value must be a single double value $\in(0,1)$. The default level is `0.05`.

N = integer
A required keyword that specifies the sample size. The value must be a single integer > 1.

POWER = value
A required keyword that specifies the power for sample size estimation. The value must be a single value $\in(0,1)$. **POWER** and **N** cannot be specified simultaneously.

SD = value
A required keyword that specifies the population standard deviation. The value must be a single, numeric value > 0.

MEAN = value
A required keyword that specifies the population mean under testing. The value must be a single, numeric value.

NULL = value
An optional keyword that specifies the null hypothesis value to be tested. Must be a single numeric value. The default setting is 0.

PLOT Subcommand

The optional PLOT subcommand controls the output charts. By default, no charts are created. When the subcommand is specified, only the outputs for the following keywords are created.

N(integer1 integer2)
The optional keyword controls the two-dimensional power by sample size chart. The chart is suppressed by default. When specified, the chart displays and the plot boundaries are determined by ***integer1*** and ***integer2***, respectively. When no integers are specified, the default plot range is

used. Both **integer1** and **integer2** are integers, separated by a blank space (or blank spaces), and must satisfy: `1 < integer1 < integer2 ≤ 5000`. You can either omit or specify both integer values.

ES(value1 value2)
The optional keyword controls the two-dimensional power by effect size chart. The chart is suppressed by default. When the keyword is specified, the chart displays. When no values are specified, the default plot range of the effect size is used. When values are specified, the plot boundaries are determined by **value1** and **value2**, respectively. Both **value1** and **value2** are numeric values, separated by a blank space (or blank spaces), and must satisfy `–5.0 ≤ value1 < value2 ≤ 5.0` when **/CRITERIA TEST = DIRECTIONAL**, or `0 ≤ value1 < value2 ≤ 5.0` when **/CRITERIA TEST = NONDIRECTIONAL**. You can either omit or specify both numeric values.

N_XAXIS(integer1 integer2) ES_YAXIS(value1 value2)
The optional keyword controls the three-dimensional power by sample size (x-axis) and effect size (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: integer1, **integer2**, **value1**, and **value2** follow the same rules as specified for **N** and **ES**.

N_YAXIS(integer1 integer2) ES_XAXIS(value1 value2)
The optional keyword controls the three-dimensional power by sample size (y-axis) and effect size (x-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: integer1, **integer2**, **value1**, and **value2** follow the same rules as specified for **N** and **ES**.

VROTATE(integer)
The optional keyword sets the vertical rotation degrees (clockwise from the left) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in$`[0,359]` with 10 as the default setting.

HROTATE(integer)
The optional keyword sets the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in$`[0,359]` with 325 as the default setting.

POWER MEANS RELATED

POWER MEANS RELATED is available in IBM SPSS Statistics Base Edition. The command invokes power analysis for the related sample *t*-test to draw statistical inference about the difference of the two means. It is assumed that the data in each sample independently and identically follow a normal distribution with a fixed mean and variance.

```
POWER MEANS RELATED
     /PARAMETERS
          [ TEST={NONDIRECTIONAL**} {DIRECTIONAL} ]
          [ SIGNIFICANCE={0.05**} {value}] ]
          { NPAIRS={integer} {POWER={value} }
          SD={value} {value1 value2}
          [ RHO ={0.5**} {value}]
          MEAN={value} {value1 value2}
     [ /PLOT
          [ NPAIRS([integer1 integer2]) ]
          [ ES([value1 value2]) ]
          [ NPAIRS_XAXIS([integer1 integer2]) ES_YAXIS([value1 value2]) ]
          [ NPAIRS_YAXIS([integer1 integer2]) ES_XAXIS([value1 value2]) ]
          [ VROTATE({10**} {integer}) ]
          [ HROTATE({325**} {integer}) ] ]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 27.0

- Command introduced

Overview

POWER MEANS RELATED invokes the power means analysis for the related samples *t*-test.

Options

Test Assumptions
Specifies the keywords that control the test assumptions. The keywords specify the number of group pairs, the population standard deviation(s), the Pearson product-moment correlation co-efficient ρ, and the population mean(s) under testing. For more information, see "PARAMETERS Subcommand " on page 1460.

Significance Level
Specifies the significance level of the test's Type I error rate.

Test Direction
Determines whether the test is one or two-sided.

Two-Dimensional Plot
Controls the two-dimensional power by sample size chart, and the two-dimensional power by effect size chart.

Three-Dimensional Plot
Controls the three-dimensional power by sample size (x-axis) and effect size (y-axis) chart, and the three-dimensional power by sample size (y-axis) and effect size (x-axis) chart. Also controls the vertical rotation degrees (clockwise from the left) and the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart.

Basic specification

- POWER MEANS RELATED: Invokes the power analysis for the related samples *t*-test.

Subcommand order

- The subcommands can be named in any order.

Example

```
POWER MEANS RELATED
  /PARAMETERS TEST=NONDIRECTIONAL SIGNIFICANCE=0.05 NPAIRS=4 SD=.75 MEAN=1
  /PLOT NPAIRS(2 4) NPAIRS_YAXIS(2 4) ES_XAXIS VROTATE(10) HROTATE(325).
```

- **PARAMETERS** specifies that the test is non-directional (two-sided), the significance level uses the default 0.05 value, the number of group pairs for power estimation is set to 4, the population standard deviation is set to 0.75, and the population mean under testing is set to 1.
- **PLOT** specifies that the two-dimensional power by sample size chart displays in the output. The three-dimensional power by sample size (x -axis) and effect size (y-axis) chart also displays in the output. The vertical and horizontal rotation settings use their default values (10 degrees and 325 degrees respectively).

PARAMETERS Subcommand

PARAMETERS controls the test assumptions. The keywords specify the number of group pairs, the population standard deviation(s), the Pearson product-moment correlation co-efficient ρ, and the population mean(s) under testing.

TEST = {NONDIRECTIONAL}{DIRECTIONAL}
An optional keyword that determines whether the test is one or two-sided. **NONDIRECTIONAL**, or a two-sided test, is the default setting. When **DIRECTIONAL** is specified, power is computed for a one-sided test.

SIGNIFICANCE = value
An optional keyword that specifies the significance level of the Type I error rate for the test. The value must be a single double value $\in(0,1)$. The default level is `0.05`.

NPAIRS = integer
A required keyword that specifies the number of group pairs. The value must be a single integer > 1. **NPAIRS** and **POWER** cannot be specified simultaneously.

POWER=value
A required keyword that specifies the power for sample size estimation. The value must be a single value $\in(0,1)$. POWER and NPAIRS cannot be specified simultaneously.

SD = {value} {value1 value2}
A required keyword that specifies the population standard deviation(s). Note that **value**, **value1**, and **value2** are all numeric values > 0. Multiple parameters **value1** and **value2**, if specified, are separated by a blank space (or blank spaces). When a single value is specified, it denotes the population standard deviation of the group difference σ_d. When two values (**value1** and **value2**) are specified, they denote σ_1 and σ_2, respectively.

RHO = value
An optional keyword that specifies the Pearson product-moment correlation coefficient ρ. The value must be a single numeric $\in[-1,1]$ and cannot be `0`. The default value is `0.5`. Note that this keyword is ignored when a single value is specified for **SD**. Otherwise, when two values (**value1** and **value2**) are specified for **SD**, this keyword takes effect and is used to compute σ_d. In cases where **value1=value2** is specified for **SD**, ρ=1 is not allowed.

MEAN = {value} {value1 value2}
A required keyword that specifies the population mean(s) under testing. Note that **value**, **value1**, and **value2** are all numeric values. Multiple parameters **value1** and **value2**, if specified, are separated by a blank space (or blank spaces). When a single value is specified, it denotes the population mean difference μ_d. When two values(**value1** and **value2**) are specified, they denote μ_1 and μ_2, respectively.

PLOT Subcommand

The optional PL0T subcommand controls the output charts. By default, no charts are created. When the subcommand is specified, only the outputs for the following keywords are created.

NPAIRS(integer1 integer2)
The optional keyword controls the two-dimensional power by sample size chart. The chart is suppressed by default. When specified, the chart displays and the plot boundaries are determined by **integer1** and **integer2**, respectively. When no integers are specified, the default plot range is used. Both **integer1** and **integer2** are integers, separated by a blank space (or blank spaces), and must satisfy: `1 < integer1 < integer2 ≤ 5000`. You can either omit or specify both integer values.

ES(value1 value2)
The optional keyword controls the two-dimensional power by effect size chart. The chart is suppressed by default. When the keyword is specified, the chart displays. When no values are specified, the default plot range of the effect size is used. When values are specified, the plot boundaries are determined by **value1** and **value2**, respectively. Both **value1** and **value2** are numeric values, separated by a blank space (or blank spaces), and must satisfy `−5.0 ≤ value1 < value2 ≤ 5.0` when **/CRITERIA TEST = DIRECTIONAL**, or `0 ≤ value1 < value2 ≤ 5.0` when **/CRITERIA TEST = NONDIRECTIONAL**. You can either omit or specify both numeric values.

NPAIRS_XAXIS(integer1 integer2) ES_YAXIS(value1 value2)
The optional keyword controls the three-dimensional power by sample size (x-axis) and effect size (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: integer1, **integer2**, **value1**, and **value2** follow the same rules as specified for **NPAIRS** and **ES**.

NPAIRS_YAXIS(integer1 integer2) ES_XAXIS(value1 value2)
The optional keyword controls the three-dimensional power by sample size (y-axis) and effect size (x-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: integer1, **integer2**, **value1**, and **value2** follow the same rules as specified for **NPAIRS** and **ES**.

VROTATE(integer)
The optional keyword sets the vertical rotation degrees (clockwise from the left) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in$`[0,359]` with `10` as the default setting.

HROTATE(integer)
The optional keyword sets the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in$`[0,359]` with `325` as the default setting.

POWER PARTIALCORR

POWER PARTIALCORR is available in IBM SPSS Statistics Base Edition. The command invokes the power analysis for the one-sample partial correlation test. Partial correlation can be explained as the association between two random variables after eliminating the effect of another or several other variables. It is a useful measurement in the presence of confounding

```
POWER PARTIALCORR
    /PARAMETERS
        [ TEST = {NONDIRECTIONAL**} {DIRECTIONAL} ]
        [ SIGNIFICANCE = {0.05**} {value} ]
        PARTIALOUT = {integer}
        { N = {integer} } { POWER = {value} }
        [ NULL = {0**} {value} ]
        ALTERNATIVE = {value}
    [ /PLOT
        [ N([integer1 integer2 ]) ] [ PARTIALOUT ] [ NULL ] [ ALTERNATIVE ] [ DIFF ]
        [ N_XAXIS([integer1 integer2]) DIFF_YAXIS ]
        [ N_YAXIS([integer1 integer2]) DIFF_XAXIS ]
        [ NULL_XAXIS ALTERNATIVE_YAXIS ]
        [ NULL_YAXIS ALTERNATIVE_XAXIS ]
        [ VROTATE({10**} {integer}) ]
        [ HROTATE({325**} {integer}) ]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 27.0

- Command introduced

Overview

POWER PARTIALCORR invokes the power analysis for the one-sample partial correlation test.

Options

Test Assumptions
Specifies the keywords that control the test assumptions. The keywords specify the sample size, power for sample size estimation, null hypothesis value of the partial correlation parameter, alternative hypothesis value of the partial correlation parameter, test direction, variables assumed to be partialled-out, and the significance level of the type I error rate. For more information, see “PARAMETERS Subcommand ” on page 1464.

Two-Dimensional Plot
Controls the two-dimensional power by number of partialled-out variables chart, the two-dimensional power by null value chart, the two-dimensional power by alternative value chart, and the two-dimensional power by difference between hypothesized values chart.

Three-Dimensional Plot
Controls the three-dimensional power by sample size (x-axis) and difference between hypothesized values (y-axis) chart, the three-dimensional power by sample size (y-axis) and difference between hypothesized values (x-axis) chart, the three-dimensional power by null (x-axis) and alternative value (y-axis) chart, and the three-dimensional power by null (y-axis) and alternative value (x-axis) chart. Also controls the vertical rotation degrees (clockwise from the left) and the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart.

Basic specification

- POWER PARTIALCORR: Invokes the power analysis for the one-sample partial correlation test.

Subcommand order

- The subcommands can be named in any order.

Example

```
POWER PARTIALCORR
  /PARAMETERS TEST=NONDIRECTIONAL SIGNIFICANCE=0.05
      PARTIALOUT=1 N=5 NULL=0 ALTERNATIVE=1
  /PLOT NULL NULL_XAXIS ALTERNATIVE_YAXIS VROTATE(10)
      HROTATE(325).
```

- **PARAMETERS** specifies that the test direction is non-directional, the significance level uses the default 0.05 value, the variables assumed to be partialled-out is set to 1, the sample size is set to 5, the null hypothesis value of the partial correlation parameter is set to 0, and the alternative hypothesis value of the partial correlation parameter is set to 1.
- **PLOT** specifies that the two-dimensional power by null value chart displays in the output. The three-dimensional power by null (x-axis) and alternative value (y-axis) chart also displays in the output. The vertical and horizontal rotation settings use their default values (10 degrees and 325 degrees respectively).

PARAMETERS Subcommand

PARAMETERS controls the test assumptions. The keywords specify the sample size, power for sample size estimation, null hypothesis value of the partial correlation parameter, alternative hypothesis value of the partial correlation parameter, test direction, variables assumed to be partialled-out, and the significance level of the type I error rate.

TEST = {NONDIRECTIONAL}{DIRECTIONAL}
An optional keyword that determines whether the test is one or two-sided. **NONDIRECTIONAL**, or a two-sided test, is the default setting. When **DIRECTIONAL** is specified, power is computed for a one-sided test.

SIGNIFICANCE = value
An optional keyword that specifies the significance level of the Type I error rate for the test. The value must be a single double value $\in(0,1)$. The default level is 0.05.

PARTIALOUT = integer
A required keyword to specify the number of the variables assumed to be partialled-out. Must be a single integer ≥ 0.

N = integer
A required keyword that specifies the sample size. The value must be a single integer > **PARTIALOUT** + 3. **N** and **POWER** cannot be specified simultaneously.

POWER = value
A required keyword that specifies the power for sample size estimation. The value must be a single value $\in(0,1)$. **POWER** and **N** cannot be specified simultaneously.

NULL = value
An optional keyword that specifies the null hypothesis value of the partial correlation parameter. Must be a single numeric value $\in(-1,1)$. The default setting is 0. When **POWER** is specified, the **NULL** value cannot be -1 or 1.

ALTERNATIVE = value
A required keyword that specifies the alternative hypothesis value of the partial correlation parameter. Must be a single numeric value $\in(-1,1)$. When **POWER** is specified, the **ALTERNATIVE** value cannot be -1 or 1, or equal to the **NULL** value.

PLOT Subcommand

The optional PLOT subcommand controls the output charts. By default, no charts are created. When the subcommand is specified, only the outputs for the following keywords are created.

N(integer1 integer2)
The optional keyword controls the two-dimensional power by sample size chart. The chart is suppressed by default. When specified, the chart displays and the plot boundaries are determined by **integer1** and **integer2**, respectively. When no integers are specified, the default plot range is used. Both **integer1** and **integer2** are integers, separated by a blank space (or blank spaces), and must satisfy: **PARTIALOUT** + 4 ≤ integer1 < integer2 ≤ **PARTIALOUT** + 5000. You can either omit or specify both integer values.

PARTIALOUT
The optional keyword controls the two-dimensional power by number of partialled-out variables chart. The chart is suppressed by default. When specified, the chart displays.

NULL
The optional keyword controls the two-dimensional power by null value chart. The chart is suppressed by default. When specified, the chart displays.

ALTERNATIVE
The optional keyword controls the two-dimensional power by alternative value chart. The chart is suppressed by default. When specified, the chart displays.

DIFF
The optional keyword controls the two-dimensional power by risk difference chart. The chart is suppressed by default. When specified, the chart displays.

N_XAXIS(integer1 integer2) DIFF_YAXIS
The optional keyword controls the three-dimensional power by total number of trials (x -axis) and risk difference (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: integer1and **integer2** follow the same rules as specified for **N**.

N_YAXIS(integer1 integer2) DIFF_XAXIS
The optional keyword controls the three-dimensional power by total number of trials (y-axis) and risk difference (x -axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: integer1and **integer2** follow the same rules as specified for **N**.

NULL_XAXIS ALTERNATIVE_YAXIS
The optional keyword controls the three-dimensional power by null (x -axis) and alternative value (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

NULL_YAXIS ALTERNATIVE_XAXIS
The optional keyword controls the three-dimensional power by null (y-axis) and alternative value (x -axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

VROTATE(integer)
The optional keyword sets the vertical rotation degrees (clockwise from the left) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer ∈[0,359] with 10 as the default setting.

HROTATE(integer)
The optional keyword sets the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer ∈[0,359] with 325 as the default setting.

POWER PEARSON ONESAMPLE

POWER PEARSON ONESAMPLE is available in IBM SPSS Statistics Base Edition. The command invokes the power analysis for the one-sample Pearson correlation test. Pearson product-moment correlation coefficient measures the strength of linear association between two scale random variables, which are assumed to follow a bivariate normal distribution.

```
POWER PEARSON ONESAMPLE
    /PARAMETERS
        [ TEST = {NONDIRECTIONAL**} {DIRECTIONAL} ]
        [ ADJUST_BIAS = {TRUE**} {FALSE} ]
        [ SIGNIFICANCE = {0.05**} {value} ]
        { N = {integer} } { POWER = {value} }
        [ NULL = {0**} {value} ]
        ALTERNATIVE = {value}
    [ /PLOT
        [ N([integer1 integer2 ]) ] [ NULL ] [ ALTERNATIVE ] [ DIFF ]
        [ N_XAXIS([integer1 integer2]) DIFF_YAXIS ]
        [ N_YAXIS([integer1 integer2]) DIFF_XAXIS ]
        [ NULL_XAXIS ALTERNATIVE_YAXIS ]
        [ NULL_YAXIS ALTERNATIVE_XAXIS ]
        [ VROTATE({10**} {integer}) ]
        [ HROTATE({325**} {integer}) ] ]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 27.0

- Command introduced

Overview

POWER PEARSON ONESAMPLE invokes the power analysis for the one-sample Pearson correlation test.

Options

Test Assumptions
Specifies the keywords that control the test assumptions. The keywords specify the sample size, power for sample size estimation, null hypothesis value of the correlation parameter, alternative hypothesis value of the correlation parameter, test direction, bias adjustment, and the significance level of the type I error rate. For more information, see "PARAMETERS Subcommand " on page 1468.

Two-Dimensional Plot
Controls the two-dimensional power estimation by charts.

Three-Dimensional Plot
Controls the three-dimensional power estimation by sample size chart, and the three-dimensional power by null hypothesis value chart. Also controls the vertical rotation degrees (clockwise from the left) and the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart.

Basic specification

- POWER PEARSON ONESAMPLE: Invokes the power analysis for the one-sample Pearson correlation test.

Subcommand order

- The subcommands can be named in any order.

Example

```
POWER PEARSON ONESAMPLE
  /PARAMETERS TEST=NONDIRECTIONAL ADJUST_BIAS=TRUE SIGNIFICANCE=0.05
       N=4 NULL=1 ALTERNATIVE=.5
  /PLOT DIFF N_YAXIS DIFF_XAXIS VROTATE(10) HROTATE(325).
```

- **PARAMETERS** specifies that the test direction is non-directional, bias adjustment is enabled, the significance level uses the default 0.05 value, the sample size is set to 4, the null hypothesis value of the correlation parameter is set to 1, and the alternative hypothesis value of the correlation parameter is set to 0.5.
- **PLOT** specifies that the two-dimensional power by difference between hypothesized values chart displays in the output. The three-dimensional power by sample size (y-axis) and difference between hypothesized values (x-axis) chart also displays in the output. The vertical and horizontal rotation settings use their default values (10 degrees and 325 degrees respectively).

PARAMETERS Subcommand

PARAMETERS controls the test assumptions. The keywords specify the sample size, total number of trials for groups, proportion parameters for groups, significance level, statistical tests, estimation method, and test direction.

TEST = {NONDIRECTIONAL}{DIRECTIONAL}
An optional keyword that determines whether the test is one or two-sided. **NONDIRECTIONAL**, or a two-sided test, is the default setting. When **DIRECTIONAL** is specified, power is computed for a one-sided test.

ADJUST_BIAS = boolean
An optional keyword that determines whether the bias adjustment is involved or ignored. **TRUE** is the default setting, which includes the bias adjustment term in the power estimation. When **FALSE** is specified, the bias adjustment term is ignored.

SIGNIFICANCE = value
An optional keyword that specifies the significance level of the Type I error rate for the test. The value must be a single double value $\in(0,1)$. The default level is 0.05.

N = integer
A required keyword that specifies the sample size. The value must be a single integer > 3. **N** and **POWER** cannot be specified simultaneously.

POWER = value
A required keyword that specifies the power for sample size estimation. The value must be a single value $\in(0,1)$. **POWER** and **N** cannot be specified simultaneously.

NULL = value
An optional keyword that specifies the null hypothesis value of the correlation parameter. Must be a single numeric value $\in(-1,1)$. The default setting is 0. When **POWER** is specified, the **NULL** value cannot be -1 or 1.

ALTERNATIVE = value
A required keyword that specifies the alternative hypothesis value of the correlation parameter. Must be a single numeric value $\in(-1,1)$. When **POWER** is specified, the **ALTERNATIVE** value cannot be -1 or 1, or equal to the **NULL** value.

PLOT Subcommand

The optional PLOT subcommand controls the output charts. By default, no charts are created. When the subcommand is specified, only the outputs for the following keywords are created.

N(integer1 integer2)
The optional keyword controls the two-dimensional power by sample size chart. The chart is suppressed by default. When specified, the chart displays and the plot boundaries are determined by **integer1** and **integer2**, respectively. When no integers are specified, the default plot range is used. Both **integer1** and **integer2** are integers, separated by a blank space (or blank spaces), and must satisfy: `4 ≤ integer1 < integer2 ≤ 5000`. You can either omit or specify both integer values.

NULL
The optional keyword controls the two-dimensional power by null value chart. The chart is suppressed by default. When specified, the chart displays.

ALTERNATIVE
The optional keyword controls the two-dimensional power by alternative value chart. The chart is suppressed by default. When specified, the chart displays.

DIFF
The optional keyword controls the two-dimensional power by difference between hypothesized values chart. The chart is suppressed by default. When specified, the chart displays.

N_XAXIS(integer1 integer2) DIFF_YAXIS
The optional keyword controls the three-dimensional power by sample size (x-axis) and difference between hypothesized values (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: integer1and **integer2** follow the same rules as specified for **N**.

N_YAXIS(integer1 integer2) DIFF_XAXIS
The optional keyword controls the three-dimensional power by sample size (y-axis) and difference between hypothesized values (x-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: integer1and **integer2** follow the same rules as specified for **N**.

NULL_XAXIS ALTERNATIVE_YAXIS
The optional keyword controls the three-dimensional power by null (x -axis) and alternative value (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

NULL_YAXIS ALTERNATIVE_XAXIS
The optional keyword controls the three-dimensional power by null (y-axis) and alternative value (x -axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

VROTATE(integer)
The optional keyword sets the vertical rotation degrees (clockwise from the left) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in$`[0,359]` with `10` as the default setting.

HROTATE(integer)
The optional keyword sets the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in$`[0,359]` with 325 as the default setting.

POWER PROPORTIONS INDEPENDENT

POWER PROPORTIONS INDEPENDENT is available in IBM SPSS Statistics Base Edition. The command invokes the power analysis for the independent-sample binomial test to compare two independent proportion parameters.

```
POWER PROPORTIONS INDEPENDENT
    /PARAMETERS
        [ TEST = {NONDIRECTIONAL**} {DIRECTIONAL} ]
        [ SIGNIFICANCE = {0.05**} {value} ]
        { N = {integer1 integer2} {POWER = {value}} }
        [ NRATIO = {1**} {value} ]
        PROPORTIONS = {value1 value2}
        [ METHOD = {CHISQ**} {T} {LRT} {FISHER} ]
        [ ESTIMATE = {NORMAL**} {BINOMIAL} ]
        [ TIMER = {value} ]
        [ CONTINUITY = {FALSE**} {TRUE} ]
        [ POOLED = {TRUE**} {FALSE} ]
    [ /PLOT
        [ N1_XAXIS([integer1 integer2]) N2_YAXIS([integer1 integer2]) ]
        [ N1_YAXIS([integer1 integer2]) N2_XAXIS([integer1 integer2]) ]
        [ P1_XAXIS P2_YAXIS ]
        [ P1_YAXIS P2_XAXIS ]
        [ NRATIO([value1 value2]) ]
        [ RISK_DIFF ]
        [ RISK_RATIO([value1 value2]) ]
        [ ODDS_RATIO([value1 value2]) ]
        [ VROTATE({10**} {integer}) ]
        [ HROTATE({325**} {integer}) ] ]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 27.0

- Command introduced

Overview

POWER PROPORTIONS INDEPENDENT invokes power analysis for the independent-sample binomial test.

Options

Test Assumptions
Specifies the keywords that control the test assumptions. The keywords specify the sample size, total number of trials for groups, proportion parameters for groups, significance level, tests, estimation method, and test direction. For more information, see "PARAMETERS Subcommand " on page 1472.

Two-Dimensional Plot
Controls the two-dimensional power estimation by charts.

Three-Dimensional Plot
Controls the three-dimensional power estimation by sample chart, and the three-dimensional power by null hypothesis value chart. Also controls the vertical rotation degrees (clockwise from the left) and the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart.

Basic specification

- POWER PROPORTIONS INDEPENDENT: Invokes power analysis for the independent-sample binomial test.

Subcommand order

- The subcommands can be named in any order.

Example

```
POWER PROPORTIONS INDEPENDENT
  /PARAMETERS TEST=NONDIRECTIONAL SIGNIFICANCE=0.05 N=1 2
      PROPORTIONS=.5 .75 METHOD=CHISQ ESTIMATE=BINOMIAL POOLED=TRUE
  /PLOT  N1_YAXIS  N2_XAXIS RISK_DIFF VROTATE(10) HROTATE(325).
```

- **PARAMETERS** specifies that the test direction is non-directional, the significance level uses the default 0.05 value, the total number of trials for the two groups is set to 1 and 2, the statistical test is chi-square, the estimation method is binomial approximation, and the estimation of the standard deviation is pooled.
- **PLOT** specifies that the three-dimensional power by number of trials in Group 2 (x -axis) and number of trials in Group 1 (y-axis) chart will display in the output. The two-dimensional power by risk ratio chart will also display in the output. The vertical and horizontal rotation settings use their default values (10 degrees and 325 degrees respectively).

PARAMETERS Subcommand

PARAMETERS controls the test assumptions. The keywords specify the sample size, total number of trials for groups, proportion parameters for groups, significance level, statistical tests, estimation method, and test direction.

TEST = {NONDIRECTIONAL}{DIRECTIONAL}
An optional keyword that determines whether the test is one or two-sided. **NONDIRECTIONAL**, or a two-sided test, is the default setting. When **DIRECTIONAL** is specified, power is computed for a one-sided test.

SIGNIFICANCE = value
An optional keyword that specifies the significance level of the Type I error rate for the test. The value must be a single double value $\in(0,1)$. The default level is 0.05.

N = (integer1 integer2)
A required keyword that specifies the total number of trials for two groups. Both integer values must be ≥ 1. When METHOD = T, integer1 + integer2 > 2.

POWER = value
A required keyword that specifies the power for sample size estimation. The value must be a single value $\in(0,1)$. **POWER** and **N** cannot be specified simultaneously.

NRATIO = value
An optional keyword that specifies the sample size ratios. Must be a single value $\in(0.01,100)$ with 1 as the default setting. The keyword takes effect when **POWER** is specified and is ignored when **N** is specified.

PROPORTIONS = (value1 value2)
An optional keyword that specifies the proportion parameters for two groups. Both values must be $\in(0,1)$. **value1** and **value2** cannot be the same when **POWER** is specified.

METHOD = {CHISQ} {T} {LRT} {FISHER}
An optional keyword that specifies the statistical test. **CHISQ** estimates the power based on the Pearson's chi-squared test (the default setting). **T**, **LRT**, and **FISHER** estimate the power based on the Student's *t*-test, likelihood ratio test, and Fisher's exact test, respectively. Note that the combination of METHOD = FISHER and ESTIMATE = BINOMIAL takes effect when the sum of **integer1** and **integer2** specified for /**PARAMETERS N** does not exceed 500 (otherwise **CHISQ** is used).

Note: In some cases, METHOD=FISHER may take an extended amount of time to complete.

ESTIMATE = {NORMAL} {BINOMIAL}
An optional keyword that determines the power estimation method, if the total sample size specified for /PARAMETERS N does not exceed 500. **NORMAL** calls for the normal approximation (the default setting). **BINOMIAL** calls for the binomial enumeration method. The keyword has no effect when the specified /PARAMETERS N value exceeds 500 (normal approximation is used).

TIMER = {value}
An optional keyword to specify the maximum number of minutes allowed to estimate the sample size. If specified, the value must be a single positive integer to denote the number of minutes. The keyword takes effect when `ESTIMATE = BINOMIAL` and **POWER** is specified. The keyword is not pasted by default, which turns the timer off completely. When the time limit is reached, the analysis is terminated and a warning message is displayed (no results are provided).

CONTINUITY = boolean
An optional keyword that controls whether or not continuity correction is used. **FALSE** is the default setting. Continuity correction is applied when **TRUE** is specified. The keyword takes effect when `METHOD = CHISQ`.

POOLED = boolean
An optional keyword that controls whether the estimation of the standard deviation is pooled or unpooled. **TRUE** is the default setting. When **FALSE** is specified, the unpooled standard deviation is calculated. The keyword takes effect when `METHOD = CHISQ` or `METHOD = T`.

PLOT Subcommand

The optional PLOT subcommand controls the output charts. By default, no charts are created. When the subcommand is specified, only the outputs for the following keywords are created.

N1_XAXIS(integer1 integer2) N2_YAXIS(integer1 integer2)
The optional keyword controls the three-dimensional power by number of trials in Group 1 (x -axis) and number of trials in Group 2 (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays. When no integers are specified, the default plot range is used. Both **integer1** and **integer2** are integers, separated by a blank space (or blank spaces). You can either omit or specify both integer values.

N1_YAXIS(integer1 integer2) N2_XAXIS(integer1 integer2)
The optional keyword controls the three-dimensional power by number of trials in Group 2 (x -axis) and number of trials in Group 1 (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays. When no integers are specified, the default plot range is used. Both **integer1** and **integer2** are integers, separated by a blank space (or blank spaces). You can either omit or specify both integer values.

P1_XAXIS P2_YAXIS
The optional keyword controls the three-dimensional power by proportion of Group 1 (x -axis) and proportion of Group 2 (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

P1_YAXIS P2_XAXIS
The optional keyword controls the three-dimensional power by proportion of Group 2 (x -axis) and proportion of Group 1 (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

NRATIO (value1 value2)
The optional keyword controls the two-dimensional power by group size ratio chart. The chart is suppressed by default. When the keyword is specified, the chart displays. When no values are specified, the default plot range is used. Both **value1** and **value2** are separated by a blank space (or blank spaces). You can either omit or specify both values.

RISK_DIFF
The optional keyword controls the two-dimensional power by risk difference chart. The chart is suppressed by default. When specified, the chart displays.

RISK_RATIO (value1 value2)
The optional keyword controls the two-dimensional power by risk ratio chart. The chart is suppressed by default. When specified, the chart displays. When no values are specified within the parentheses, the default plot range of the risk ratio is used. When specified, the plot boundaries are determined by **value1** and **value2**, respectively. Both **value1** and **value2** are separated by a blank space (or blank spaces). You can either omit or specify both values.

ODDS_RATIO (value1 value2)

The optional keyword controls the two-dimensional power by odds ratio chart. The chart is suppressed by default. When specified, the chart displays. When no values are specified within the parentheses, the default plot range of the odds ratio is used. When specified, the plot boundaries are determined by **value1** and **value2**, respectively. Both **value1** and **value2** are separated by a blank space (or blank spaces). You can either omit or specify both values.

VROTATE(integer)

The optional keyword sets the vertical rotation degrees (clockwise from the left) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in$ `[0,359]` with `10` as the default setting.

HROTATE(integer)

The optional keyword sets the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in$ `[0,359]` with 325 as the default setting.

POWER PROPORTIONS ONESAMPLE

POWER PROPORTIONS ONESAMPLE is available in IBM SPSS Statistics Base Edition. The command invokes power analysis for the one-sample binomial test to make statistical inference about the proportion parameter by comparing it with a hypothesized value.

```
POWER PROPORTIONS ONESAMPLE
   /PARAMETERS
      [ TEST = {NONDIRECTIONAL**} {DIRECTIONAL} ]
      [ ESTIMATE = {NORMAL**} {BINOMIAL} ]
      [ TIMER = {value} ]
      [ CONTINUITY = {FALSE**} {TRUE} ]
      [ SIGNIFICANCE = {0.05**} {value} ]
      { N = {integer} } { POWER = {value} }
      [ NULL = {0.50**} {value} ]
      ALTERNATIVE = {value}
   [ /PLOT
      [ N([integer1 integer2 ]) ] [ NULL ] [ ALTERNATIVE ] [ DIFF ]
      [ N_XAXIS([integer1 integer2]) DIFF_YAXIS ]
      [ N_YAXIS([integer1 integer2]) DIFF_XAXIS ]
      [ NULL_XAXIS ALTERNATIVE_YAXIS ]
      [ NULL_YAXIS ALTERNATIVE_XAXIS ]
      [ VROTATE({10**} {integer}) ]
      [ HROTATE({325**} {integer}) ] ]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 27.0

- Command introduced

Overview

POWER PROPORTIONS ONESAMPLE invokes power analysis for the one-sample binomial test.

Options

Test Assumptions
Specifies the keywords that control the test assumptions. The keywords specify the sample size, total number of trials, sample size estimation, population proportion, null hypothesis value, estimation method, test direction, and significance level. For more information, see "PARAMETERS Subcommand " on page 1476.

Two-Dimensional Plot
Controls the two-dimensional power by charts.

Three-Dimensional Plot
Controls the three-dimensional power estimation by total number of trials chart, and the three-dimensional power by null hypothesis value chart. Also controls the vertical rotation degrees (clockwise from the left) and the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart.

Basic specification

- POWER PROPORTIONS ONESAMPLE: Invokes power analysis for the one-sample binomial test.

Subcommand order

- The subcommands can be named in any order.

Example

```
POWER PROPORTIONS ONESAMPLE
  /PARAMETERS TEST=NONDIRECTIONAL ESTIMATE=NORMAL CONTINUITY=FALSE
       SIGNIFICANCE=0.05 N=2 NULL=.5 ALTERNATIVE=1
  /PLOT NULL ALTERNATIVE N_XAXIS DIFF_YAXIS VROTATE(10)
       HROTATE(325).
```

- **PARAMETERS** specifies that the test direction is non-directional, the estimation method is normal approximation, continuity correction is disabled, the significance level uses the default 0.05 value, the total number of trials is set to 2, the null value uses the default value of 0.5, and the population proportion is set to 1.
- **PLOT** specifies that the two dimensional Power estimation versus null hypothesis value and the Power estimation versus alternative hypothesis value charts will display in the output. The Power estimation versus total number of trials on x-axis and the difference between hypothesized values on y-axis three dimensional chart will also display in the output. The vertical and horizontal rotation settings use their default values (10 degrees and 325 degrees respectively).

PARAMETERS Subcommand

PARAMETERS controls the test assumptions. The keywords specify the sample size, total number of trials, sample size estimation, population proportion, null hypothesis value, estimation method, test direction, and significance level.

TEST = {NONDIRECTIONAL}{DIRECTIONAL}
An optional keyword that determines whether the test is one or two-sided. **NONDIRECTIONAL**, or a two-sided test, is the default setting. When **DIRECTIONAL** is specified, power is computed for a one-sided test.

ESTIMATE = {NORMAL} {BINOMIAL}
An optional keyword that determines the power estimation method, if the total sample size specified for /PARAMETERS N does not exceed 500. **NORMAL** calls for the normal approximation (the default setting). **BINOMIAL** calls for the binomial enumeration method. The keyword has no effect when the specified /PARAMETERS N value exceeds 500 (normal approximation is used).

TIMER = {value}
An optional keyword to specify the maximum number of minutes allowed to estimate the sample size. If specified, the value must be a single positive integer to denote the number of minutes. The keyword takes effect when ESTIMATE = BINOMIAL and **POWER** is specified. The keyword is not pasted by default, which turns the timer off completely. When the time limit is reached, the analysis is terminated and a warning message is displayed (no results are provided).

CONTINUITY = boolean
An optional keyword that controls whether or not the continuity correction is used for the normal approximation method. **FALSE** is the default setting. Continuity correction is applied when **TRUE** is specified. The keyword takes effect when ESTIMATE = NORMAL.

SIGNIFICANCE = value
An optional keyword that specifies the significance level of the Type I error rate for the test. The value must be a single double value $\in(0,1)$. The default level is 0.05.

N = integer
A required keyword that specifies the sample size. The value must be a single integer ≥ 1.

POWER = value
A required keyword that specifies the power for sample size estimation. The value must be a single value $\in(0,1)$. **POWER** and **N** cannot be specified simultaneously.

NULL = value
An optional keyword that specifies the null hypothesis value of the proportion parameter. Must be a single numeric value $\in(0,1)$. The default setting is 0.50.

ALTERNATIVE = value
A required keyword that specifies the alternative hypothesis value of the proportion parameter. Must be a single numeric value ∈(0,1). When **POWER** is specified, the **ALTERNATIVE** value cannot be equal to the **NULL** value, and must be a single numeric value ∈(0,1).

PLOT Subcommand

The optional PLOT subcommand controls the output charts. By default, no charts are created. When the subcommand is specified, only the outputs for the following keywords are created.

N(integer1 integer2)
The optional keyword controls the two-dimensional power by total number of trials chart. The chart is suppressed by default. When specified, the chart displays and the plot boundaries are determined by **integer1** and **integer2**, respectively. When no integers are specified, the default plot range is used. Both **integer1** and **integer2** are integers, separated by a blank space (or blank spaces), and must satisfy: `0 < integer1 < integer2 ≤ 5000`. You can either omit or specify both integer values.

NULL
The optional keyword controls the two-dimensional power by null value chart. The chart is suppressed by default. When specified, the chart displays.

ALTERNATIVE
The optional keyword controls the two-dimensional power by alternative value chart. The chart is suppressed by default. When specified, the chart displays.

DIFF
The optional keyword controls the two-dimensional power by risk difference chart. The chart is suppressed by default. When specified, the chart displays.

N_XAXIS(integer1 integer2) DIFF_YAXIS
The optional keyword controls the three-dimensional power by total number of trials (x -axis) and risk difference (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: integer1and **integer2** follow the same rules as specified for **N**.

N_YAXIS(integer1 integer2) DIFF_XAXIS
The optional keyword controls the three-dimensional power by total number of trials (y-axis) and risk difference (x -axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: integer1and **integer2** follow the same rules as specified for **N**.

NULL_XAXIS ALTERNATIVE_YAXIS
The optional keyword controls the three-dimensional power by null (x -axis) and alternative value (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

NULL_YAXIS ALTERNATIVE_XAXIS
The optional keyword controls the three-dimensional power by null (y-axis) and alternative value (x -axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

VROTATE(integer)
The optional keyword sets the vertical rotation degrees (clockwise from the left) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer ∈[0,359] with 10 as the default setting.

HROTATE(integer)
The optional keyword sets the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer ∈[0,359] with 325 as the default setting.

POWER PROPORTIONS RELATED

POWER PROPORTIONS RELATED is available in IBM SPSS Statistics Base Edition. The command invokes power analysis for the related-sample binomial test (or McNemar's test) to compare two proportion parameters based on the matched pair subjects sampled from two related binomial populations.

```
POWER PROPORTIONS RELATED
    /PARAMETERS
        [ TEST = {NONDIRECTIONAL**} {DIRECTIONAL} ]
        [ ESTIMATE = {NORMAL**} {BINOMIAL} ]
        [ TIMER = {value} ]
        [ SIGNIFICANCE = {0.05**} {value} ]
        { NPAIRS = {integer} } { POWER = {value} }
        VALUES = {PROPORTIONS({value1 value2})} {COUNTS({integer1 integer2})}
        [ MARGINAL = {FALSE**} {TRUE} ]
        CORRELATION = {value}
    [ /PLOT
        [ NPAIRS([integer1 integer2]) ]
        [ RISK_DIFF ]
        [ RISK_RATIO([value1 value2]) ]
        [ ODDS_RATIO([value1 value2]) ]
        [ CORRELATION ]
        [ DISCORDANT ]
        [ MARGINAL ]
        [ VROTATE({10**} {integer}) ]
        [ HROTATE({325**} {integer}) ] ]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 27.0

- Command introduced

Overview

POWER PROPORTIONS RELATED invokes power analysis for the related-sample binomial test (or McNemar's test).

Options

Test Assumptions
Specifies the keywords that control the test assumptions. The keywords specify the sample size,test values for proportions or counts, marginal test values, estimation method, test direction, and significance level. For more information, see "PARAMETERS Subcommand " on page 1480.

Two-Dimensional Plot
Controls the two-dimensional power by charts.

Three-Dimensional Plot
Controls the three-dimensional power estimation by discordant proportions chart, and the three-dimensional power by marginal proportions chart. Also controls the vertical rotation degrees (clockwise from the left) and the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart.

Basic specification

- POWER PROPORTIONS RELATED: Invokes power analysis for the related-sample binomial test (or McNemar's test).

Subcommand order

- The subcommands can be named in any order.

Example

```
POWER PROPORTIONS RELATED
  /PARAMETERS TEST=NONDIRECTIONAL SIGNIFICANCE=0.05 NPAIRS=2
      VALUES=PROPORTIONS(.5 .75) MARGINAL=TRUE CORRELATION=0 ESTIMATE=NORMAL
  /PLOT RISK_DIFF DISCORDANT VROTATE(10) HROTATE(325).
```

- **PARAMETERS** specifies that the test direction is non-directional, the significance level uses the default 0.05 value, the total number of pairs is set to 2, the testing values for proportions are set the 0.5 and 0.75, marginal test values are enabled, the correlation between matched pairs is set to 0, and the estimation method is normal approximation.
- **PLOT** specifies that the two-dimensional Power estimation versus risk difference chart will display in the output. The three-dimensional Power estimation versus discordant proportions chart will also display in the output. The vertical and horizontal rotation settings use their default values (10 degrees and 325 degrees respectively).

PARAMETERS Subcommand

PARAMETERS controls the test assumptions. The keywords specify the sample size,test values for proportions or counts, marginal test values, estimation method, test direction, and significance level.

TEST = {NONDIRECTIONAL}{DIRECTIONAL}
An optional keyword that determines whether the test is one or two-sided. **NONDIRECTIONAL**, or a two-sided test, is the default setting. When **DIRECTIONAL** is specified, power is computed for a one-sided test.

ESTIMATE = {NORMAL} {BINOMIAL}
An optional keyword that determines the power estimation method, if the total sample size specified for /PARAMETERS N does not exceed 500. **NORMAL** calls for the normal approximation (the default setting). **BINOMIAL** calls for the binomial enumeration method. The keyword has no effect when the specified /PARAMETERS N value exceeds 500 (normal approximation is used).

TIMER = {value}
An optional keyword to specify the maximum number of minutes allowed to estimate the sample size. If specified, the value must be a single positive integer to denote the number of minutes. The keyword takes effect when ESTIMATE = BINOMIAL and **POWER** is specified. The keyword is not pasted by default, which turns the timer off completely. When the time limit is reached, the analysis is terminated and a warning message is displayed (no results are provided).

SIGNIFICANCE = value
An optional keyword that specifies the significance level of the Type I error rate for the test. The value must be a single double value $\in(0,1)$. The default level is 0.05.

NPAIRS = integer
A required keyword that specifies the total number of the matched pairs. Must be a positive integer ≥ 2. **NPAIRS** and **POWER** cannot be specified simultaneously.

POWER = value
A required keyword that specifies the power for sample size estimation. The value must be a single value $\in(0,1)$. **POWER** and **NPAIRS** cannot be specified simultaneously.

VALUES = {PROPORTIONS({value1 value2 })} {COUNTS({integer1 integer2 })}
A required keyword that specifies the proportions or counts. Only **PROPORTIONS** is supported when **POWER** is specified.

MARGINAL = boolean
An optional keyword that controls whether or not the proportions or the counts specified for **VALUES** are marginal. **FALSE** is the default setting, which denotes the discordant proportions or counts. When **TRUE** is specified, it denotes the marginal proportions or counts.

CORRELATION = value
A required keyword that specifies the correlation between matched pairs, which takes effect when MARGINAL = TRUE. The keyword is ignored when MARGINAL = FALSE. Must be a single value ∈ (-1,1).

PLOT Subcommand

The optional PLOT subcommand controls the output charts. By default, no charts are created. When the subcommand is specified, only the outputs for the following keywords are created.

NPAIRS(integer1 integer2)
The optional keyword controls the two-dimensional power by total matched pairs chart. The chart is suppressed by default. When specified, the chart displays and the plot boundaries are determined by **integer1** and **integer2**, respectively. When no integers are specified, the default plot range is used. Both **integer1** and **integer2** are integers, separated by a blank space (or blank spaces), and must satisfy: $1 < integer1 < integer2 \leq 2500$. You can either omit or specify both integer values.

RISK_DIFF
The optional keyword controls the two-dimensional power by risk difference chart. The chart is suppressed by default. When specified, the chart displays.

RISK_RATIO (value1 value2)
The optional keyword controls the two-dimensional power by risk ratio chart. The chart is suppressed by default. When specified, the chart displays. When no values are specified within the parentheses, the default plot range of the risk ratio is used. When specified, the plot boundaries are determined by **value1** and **value2**, respectively. Both **value1** and **value2** are separated by a blank space (or blank spaces). You can either omit or specify both values.

ODDS_RATIO (value1 value2)
The optional keyword controls the two-dimensional power by odds ratio chart. The chart is suppressed by default. When specified, the chart displays. When no values are specified within the parentheses, the default plot range of the odds ratio is used. When specified, the plot boundaries are determined by **value1** and **value2**, respectively. Both **value1** and **value2** are separated by a blank space (or blank spaces). You can either omit or specify both values.

CORRELATION
The optional keyword controls the two-dimensional power by correlation chart. The chart is suppressed by default. When specified, the chart displays. The keyword takes effect when MARGINAL = TRUE. The keyword is ignored when MARGINAL = FALSE.

DISCORDANT
The optional keyword controls the three-dimensional power by discordant proportions chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

MARGINAL
The optional keyword controls the three-dimensional power by marginal proportions chart. The chart is suppressed by default. When the keyword is specified, the chart displays. The keyword takes effect when MARGINAL = TRUE. The keyword is ignored when MARGINAL = FALSE.

VROTATE(integer)
The optional keyword sets the vertical rotation degrees (clockwise from the left) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer ∈[0,359] with 10 as the default setting.

HROTATE(integer)
The optional keyword sets the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer ∈[0,359] with 325 as the default setting.

POWER SPEARMAN ONESAMPLE

POWER SPEARMAN ONESAMPLE is available in IBM SPSS Statistics Base Edition. The command invokes the power analysis for the one-sample Spearman rank-order correlation test. Spearman rank-order correlation coefficient is a rank-based nonparametric statistic to measure the monotonic relationship between two variables that are usually censored and not normally distributed.

```
POWER SPEARMAN ONESAMPLE
    /PARAMETERS
        [ TEST = {NONDIRECTIONAL**} {DIRECTIONAL} ]
        [ VARIANCE = {BW**} {FHP} {CC} ]
        [ SIGNIFICANCE = {0.05**} {value} ]
        { N = {integer} } { POWER = {value} }
        [ NULL = {0**} {value} ]
        ALTERNATIVE = {value}
    [ /PLOT
        [ N([integer1 integer2 ]) ] [ NULL ] [ ALTERNATIVE ] [ DIFF ]
        [ N_XAXIS([integer1 integer2]) DIFF_YAXIS ]
        [ N_YAXIS([integer1 integer2]) DIFF_XAXIS ]
        [ NULL_XAXIS ALTERNATIVE_YAXIS ]
        [ NULL_YAXIS ALTERNATIVE_XAXIS ]
        [ VROTATE({10**} {integer}) ]
        [ HROTATE({325**} {integer}) ] ]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 27.0

- Command introduced

Overview

POWER SPEARMAN ONESAMPLE invokes the power analysis for the one-sample Spearman rank-order correlation test.

Options

Test Assumptions
Specifies the keywords that control the test assumptions. The keywords specify the sample size, power for sample size estimation, asymptotic variance, null hypothesis value of the correlation parameter, alternative hypothesis value of the correlation parameter, test direction, and the significance level of the type I error rate. For more information, see “PARAMETERS Subcommand ” on page 1484.

Two-Dimensional Plot
Controls the two-dimensional power estimation by charts.

Three-Dimensional Plot
Controls the three-dimensional power estimation by charts. Also controls the vertical rotation degrees (clockwise from the left) and the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart.

Basic specification

- POWER SPEARMAN ONESAMPLE: Invokes the power analysis for the one-sample Spearman rank-order correlation test.

Subcommand order

- The subcommands can be named in any order.

Example

```
POWER SPEARMAN ONESAMPLE
  /PARAMETERS TEST=NONDIRECTIONAL SIGNIFICANCE=0.05
      VARIANCE=FHP N=4 NULL=1 ALTERNATIVE=1
  /PLOT DIFF NULL_XAXIS ALTERNATIVE_YAXIS VROTATE(10)
      HROTATE(325).
```

- **PARAMETERS** specifies that the test direction is non-directional, the significance level uses the default 0.05 value, Fieller, Hartley and Pearson (FHP) is specified as the asymptotic variance, the sample size is set to 4, the null hypothesis value of the correlation parameter is set to 1, and the alternative hypothesis value of the correlation parameter is set to 1.
- **PLOT** specifies that the two-dimensional power by difference between hypothesized values chart displays in the output. The three-dimensional power by sample size (x-axis) and difference between hypothesized values (y-axis) chart also displays in the output. The vertical and horizontal rotation settings use their default values (10 degrees and 325 degrees respectively).

PARAMETERS Subcommand

PARAMETERS controls the test assumptions. The keywords specify the sample size, power for sample size estimation, asymptotic variance, null hypothesis value of the correlation parameter, alternative hypothesis value of the correlation parameter, test direction, and the significance level of the type I error rate.

TEST = {NONDIRECTIONAL}{DIRECTIONAL}
An optional keyword that determines whether the test is one or two-sided. **NONDIRECTIONAL**, or a two-sided test, is the default setting. When **DIRECTIONAL** is specified, power is computed for a one-sided test.

Variance = {BW} {FHP} {CC}
An optional keyword that determines how the asymptotic variance is estimated for the power analysis. **BW** is the setting by default, which estimates the variance suggested

by Bonett and Wright. **FHP** and **CC** estimate the variance suggested by Fieller et al.,and Caruso and Cliff, respectively.

SIGNIFICANCE = value
An optional keyword that specifies the significance level of the Type I error rate for the test. The value must be a single double value $\in(0,1)$. The default level is 0.05.

N = integer
A required keyword that specifies the sample size. The value must be a single integer > 3 when VARIANCE = BW or FHP, and > 2 when VARIANCE = CC. **N** and **POWER** cannot be specified simultaneously.

POWER = value
A required keyword that specifies the power for sample size estimation. The value must be a single value $\in(0,1)$. **POWER** and **N** cannot be specified simultaneously.

NULL = value
An optional keyword that specifies the null hypothesis value of the correlation parameter. Must be a single numeric value $\in(-1,1)$. The default setting is 0.

ALTERNATIVE = value
A required keyword that specifies the alternative hypothesis value of the correlation parameter. Must be a single numeric value $\in(-1,1)$. When **POWER** is specified, the **ALTERNATIVE** value cannot be equal to the **NULL** value.

PLOT Subcommand

The optional PL0T subcommand controls the output charts. By default, no charts are created. When the subcommand is specified, only the outputs for the following keywords are created.

N(integer1 integer2)
The optional keyword controls the two-dimensional power by total number of trials chart. The chart is suppressed by default. When specified, the chart displays and the plot boundaries are determined by **integer1** and **integer2**, respectively. When no integers are specified, the default plot range is used. Both **integer1** and **integer2** are integers, separated by a blank space (or blank spaces), and must satisfy: `4 ≤ integer1 < integer2 ≤ 5000`. You can either omit or specify both integer values.

NULL
The optional keyword controls the two-dimensional power by null value chart. The chart is suppressed by default. When specified, the chart displays.

ALTERNATIVE
The optional keyword controls the two-dimensional power by alternative value chart. The chart is suppressed by default. When specified, the chart displays.

DIFF
The optional keyword controls the two-dimensional power by risk difference chart. The chart is suppressed by default. When specified, the chart displays.

N_XAXIS(integer1 integer2) DIFF_YAXIS
The optional keyword controls the three-dimensional power by total number of trials (x -axis) and risk difference (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: integer1and **integer2** follow the same rules as specified for **N**.

N_YAXIS(integer1 integer2) DIFF_XAXIS
The optional keyword controls the three-dimensional power by total number of trials (y-axis) and risk difference (x -axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

Note: integer1and **integer2** follow the same rules as specified for **N**.

NULL_XAXIS ALTERNATIVE_YAXIS
The optional keyword controls the three-dimensional power by null (x -axis) and alternative value (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

NULL_YAXIS ALTERNATIVE_XAXIS
The optional keyword controls the three-dimensional power by null (y-axis) and alternative value (x -axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

VROTATE(integer)
The optional keyword sets the vertical rotation degrees (clockwise from the left) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in$`[0,359]` with `10` as the default setting.

HROTATE(integer)
The optional keyword sets the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in$`[0,359]` with 325 as the default setting.

POWER UNIVARIATE LINEAR

POWER UNIVARIATE LINEAR is available in IBM SPSS Statistics Base Edition. The command invokes power analysis for the type III *F*-test in univariate linear regression. Univariate linear regression is a basic and standard statistical approach in which researchers use the values of several variables to explain or predict values of a scale outcome.

```
POWER UNIVARIATE LINEAR
    /PARAMETERS
        [ MODEL ={FIXED**} {RANDOM}]
        [ SIGNIFICANCE = {0.05**} {value}]
        {N ={integer} } {POWER ={value} }
        TOTAL_PREDICTORS = {integer}
        TEST_PREDICTORS = {integer}
        {PARTIAL_CORR = {value} } {FULL_MODEL = {value} NESTED_MODEL ={value} }
        [ INTERCEPT = {TRUE**} {FALSE}]
    [ /PLOT
        [ N([integer1 integer2]) ]
        [ PARTIAL_CORR ]
        [ N_XAXIS([integer1 integer2]) PARTIAL_CORR_YAXIS ]
        [ N_YAXIS([integer1 integer2]) PARTIAL_CORR_XAXIS ]
        [ VROTATE({10**} {integer}) ]
        [ HROTATE({325**} {integer}) ] ]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 27.0

- Command introduced

Overview

POWER UNIVARIATE LINEAR invokes power analysis for the type III *F*-test in univariate linear regression. Univariate linear regression is a basic and standard statistical approach in which researchers use the values of several variables to explain or predict values of a scale outcome.

Options

Test Assumptions

Specifies the keywords that control the test assumptions. The keywords specify the sample size, the power for sample size estimation, the population multiple partial correlation, number of either the total predictors or the predictors in the full model, number of either the test predictors or the predictors in the nested mode, the multiple partial correlation coefficient, the squared multiple correlation coefficients for the full and nested model, the intercept term, model predictors, and the significance level of the type I error rate for the test. For more information, see "PARAMETERS Subcommand " on page 1488.

Two-Dimensional Plot

Controls the two-dimensional power by sample size chart, and the two-dimensional power by effect size chart.

Three-Dimensional Plot

Controls the three-dimensional power by sample size (x-axis) and effect size (y-axis) chart, and the three-dimensional power by sample size (y-axis) and effect size (x-axis) chart. Also controls the vertical rotation degrees (clockwise from the left) and the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart.

Basic specification

- `POWER UNIVARIATE LINEAR`: Invokes power analysis for the type III *F*-test in univariate linear regression.

Subcommand order

- The subcommands can be named in any order.

Example

```
POWER UNIVARIATE LINEAR
  /PARAMETERS MODEL=FIXED SIGNIFICANCE=0.05 N=5 TOTAL_PREDICTORS=2 TEST_PREDICTORS=2 PARTIAL_CORR
    =0.5 INTERCEPT =TRUE
  /PLOT PARTIAL_CORR N_XAXIS  PARTIAL_CORR_YAXIS VROTATE(10) HROTATE(325).
```

- **PARAMETERS** specifies that the model predictors are fixed, the significance level uses the default 0.05 value, the total number of predictors is 2, the total number of test predictors is 2, the multiple partial correlation coefficient is 0.5, and the intercept term is included.
- **PLOT** specifies that the two-dimensional power by multiple partial correlation coefficient chart displays in the output. The three-dimensional power by sample size (x-axis) and multiple partial correlation coefficient (y-axis) chart also displays in the output. The vertical and horizontal rotation settings use their default values (10 degrees and 325 degrees respectively).

PARAMETERS Subcommand

PARAMETERS controls the test assumptions. The keywords specify the sample size, the power for sample size estimation, the population multiple partial correlation, number of either the total predictors or the predictors in the full model, number of either the test predictors or the predictors in the nested mode, the multiple partial correlation coefficient, the squared multiple cor-relation coefficients for the full and nested model, the intercept term, model predictors, and the significance level of the type I error rate for the test.

MODEL = {FIXED}{RANDOM}
An optional keyword that determines whether the predictors are assumed to be fixed or random. **FIXED** is the default setting, which assumes that the predictors have fixed values. When **RANDOM** is specified, the predictors are assumed to be random.

SIGNIFICANCE = value
An optional keyword that specifies the significance level of the Type I error rate for the test. The value must be a single double value $\in(0,1)$. The default level is `0.05`.

N = integer
A required keyword that specifies the sample size. Must be a single integer $\geq$ `(p + 2)` when `INTERCEPT = TRUE`, and $\geq$ `(p + 1)` in all other conditions (*p* is the value specified for **TOTAL_PREDICTORS**. **N** and **POWER** cannot be specified simultaneously.

POWER = value
A required keyword that specifies the power for sample size estimation. The value must be a single value $\in(0,1)$. POWER and N cannot be specified simultaneously.

TOTAL_PREDICTORS = integer
A required keyword that specifies the number of either the total predictors or the predictors in the full model (not including the intercept if applicable). Must be a single integer ≥ 1.

TEST_PREDICTORS = integer
A required keyword that specifies the number of either the test predictors or the predictors in the nested model (not including the intercept if applicable). When **PARTIAL_CORR** is specified, the value must be a single integer $\in[1,p]$, where p is the value specified for **TOTAL_PREDICTORS**. When **FULL_MODEL** and **NESTED_MODEL** are specified, the value must be a single integer $\in[1,p)$, where *p* is the value specified for **TOTAL_PREDICTORS**.

PARTIAL_CORR = value
A required keyword that specifies the value of the multiple partial correlation coefficient. Must be a single value $\in(-1,1)$. When **POWER** is specified, the value cannot be 0. **PARTIAL_CORR** and **FULL_MODEL**/**NESTED_MODEL** are mutually exclusive keywords, and only one is allowed.

FULL_MODEL = value NESTED_MODEL = value
A required keyword that specifies the squared multiple correlation coefficients for the full and nested models, respectively. **FULL_MODEL**/**NESTED_MODEL** and **PARTIAL_CORR** are mutually exclusive keywords, and only one is allowed. When specified, both **FULL_MODEL**/**NESTED_MODEL** are required, and a single value $\in(0,1)$ for each keyword is required. In addition, the values specified for **FULL_MODEL** must be greater than or equal to the value specified for **NESTED_MODEL**. When **POWER** is specified, the value specified for **FULL_MODEL** must be greater than the value specified for **NESTED_MODEL**.

INTERCEPT = boolean
An optional keyword that determines whether or not the intercept term is included. **TRUE** is the default setting, which includes the intercept term. When **FALSE** is specified, the intercept term is excluded from the power analysis.

PLOT Subcommand

The optional PLOT subcommand controls the output charts. By default, no charts are created. When the subcommand is specified, only the outputs for the following keywords are created.

N(integer1 integer2)
The optional keyword controls the two-dimensional power by sample size chart. By default, the chart is suppressed. When specified, the chart is displayed. When no integers are specified within the parentheses, the default plot range of the sample size is used. When the keyword is specified, the plot boundaries are determined by **integer1** and **integer2**, respectively. It is required that $p + 2 \leq integer1 < integer2 \leq p + 5000$ (where *p* is the value specified for TOTAL_PREDICTORS) when /PARAMETERS INTERCEPT = TRUE. In all other scenarios, $p + 1 \leq integer1 < integer2 \leq p + 5000$. Note that both **integer1** and **integer2** are integers, separated by a blank space (or blank spaces). You can either omit or specify both values.

PARTIAL_CORR
The optional keyword controls the two-dimensional power by multiple partial correlation coefficient chart. The chart is suppressed by default. When the keyword is specified, the chart displays.

N_XAXIS(integer1 integer2) PARTIAL_CORR_YAXIS
The optional keyword controls the three-dimensional power by sample size (x-axis) and multiple partial correlation coefficient (y-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays. Note that **integer1** and **integer2** follow the same rules as specified for **N**.

N_YAXIS(integer1 integer2) PARTIAL_CORR_XAXIS
The optional keyword controls the three-dimensional power by sample size (y-axis) and multiple partial correlation coefficient (x-axis) chart. The chart is suppressed by default. When the keyword is specified, the chart displays. Note that **integer1** and **integer2** follow the same rules as specified for **N**.

VROTATE(integer)
The optional keyword sets the vertical rotation degrees (clockwise from the left) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in[0,359]$ with 10 as the default setting.

HROTATE(integer)
The optional keyword sets the horizontal rotation degrees (clockwise from the front) for the three-dimensional chart. The keyword takes effect when the three-dimensional plot is requested. The value must be a single integer $\in[0,359]$ with 325 as the default setting.

PPLOT

PPLOT is available in the Statistics Base Edition.

```
PPLOT VARIABLES= varlist

 [/DISTRIBUTION={NORMAL(a,b)** }  ]
                {EXPONENTIAL(a)}
                {WEIBUL(a,b)   }
                {PARETO(a,b)   }
                {LNORMAL(a,b)  }
                {BETA(a,b)     }
                {GAMMA(a,b)    }
                {LOGISTIC(a,b) }
                {LAPLACE(a,b)  }
                {UNIFORM(a,b)  }
                {HNORMAL(a)    }
                {CHI(df)       }
                {STUDENT(df)   }

 [/FRACTION={BLOM**}]
            {RANKIT}
            {TUKEY }
            {VW    }

 [/TIES={MEAN**  }]
        {LOW     }
        {HIGH    }
        {BREAK}

 [/{NOSTANDARDIZE**}]
   {STANDARDIZE    }

 [/TYPE={Q-Q**}]
        {P-P  }

 [/PLOT={BOTH**   }]
        {NORMAL   }
        {DETRENDED}

 [/DIFF={1}]
        {n}

 [/SDIFF={1}]
         {n}

 [/PERIOD=n]

 [/{NOLOG**}]
   {LN     }

 [/APPLY ['model name']]

 [/TEMPLATE='filename']
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 19.0

- TEMPLATE subcommand introduced.

Example

```
PPLOT VARIABLES = VARX
  /FRACTION=TUKEY
  /DIFF=2.
```

Overview

PPLOT (alias NPPLOT) produces probability plots of one or more sequence or time series variables. The variables can be standardized, differenced, and/or transformed before plotting. Expected normal values or deviations from expected normal values can be plotted.

Options

Variable Modification. You can use the LN subcommand to request a natural log transformation of the sequence or time series variables, and you can use the SDIFF and DIFF subcommands to request

seasonal and nonseasonal differencing to any degree. With seasonal differencing, you can specify the periodicity on the PERIOD subcommand. You can also plot standardized series by using the STANDARDIZE subcommand.

Plot Type. You can request p-p (proportion-proportion) or q-q (quantile-quantile) plots on the TYPE subcommand. With the PLOT subcommand, you can display normal plots, detrended plots, or both.

Distribution Type. You can specify the distribution type on the DISTRIBUTION subcommand. The cumulative distribution function (CDF) and the inverse distribution function (IDF) for the specified distribution type are used to compute the expected values in the p-p and q-q plots, respectively.

Score Calculations. On the FRACTION subcommand, you can specify one of several fractional rank formulas to use for estimating the empirical distribution in p-p plots and computing expected quantiles in q-q plots. You can specify the treatment of tied values on the TIE subcommand.

Templates. You can specify a template, using the TEMPLATE subcommand, to override the default chart attribute settings on your system.

Basic Specification

The basic specification is one or more variable names.

- For each specified variable, PPLOT produces two q-q plots of the observed values (one plot versus expected normal values and the other plot versus deviations from normal values. By default, expected normal values are calculated by using Blom's transformation.
- Observed values define the horizontal axis, and expected normal values or deviations define the vertical axis.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- VARIABLES can be specified only once.
- Other subcommands can be specified more than once, but only the last specification of each subcommand is executed.

Operations

- Subcommand specifications apply to all plots that are produced by PPLOT.
- If the LN subcommand is specified, any differencing or standardization that is requested on that PPLOT is done on the log-transformed series.
- If differencing (DIFF or SDIFF) is specified, any standardization is done on the differenced series.

Limitations

- A maximum of 1 VARIABLES subcommand is allowed. There is no limit on the number of variables that are named on the list.

Example

```
PPLOT VARIABLES = VARX
  /FRACTION=TUKEY
  /DIFF=2.
```

- This command produces two normal q-q plots of *VARX*(one plot not detrended and the other plot detrended).
- The expected quantile values are calculated by using Tukey's transformation.
- The variable is differenced twice before plotting.

VARIABLES Subcommand

VARIABLES specifies the sequence or time series variables to be plotted and is the only required subcommand.

DISTRIBUTION Subcommand

DISTRIBUTION specifies the distribution type of your data. The default is NORMAL if the subcommand is not specified or is specified without a keyword. If the parameters of the distribution type are not specified, DISTRIBUTION estimates them from the sample data and displays them with the plots.

NORMAL(a,b). *Normal distribution.* The location parameter *a* can be any numeric value, while the scale parameter *b* must be positive. If they are not specified, DISTRIBUTION estimates them from the sample mean and sample standard deviation.

EXPONENTIAL(a). *Exponential distribution.* The scale parameter *a* must be positive. If the parameter is not specified, DISTRIBUTION estimates it from the sample mean. Negative observations are not allowed.

WEIBULL(a,b). *Weibull distribution.* The scale and shape parameters *a* and *b* must be positive. If they are not specified, DISTRIBUTION estimates them using the least square method. Negative observations are not allowed.

PARETO(a,b). *Pareto distribution.* The threshold and shape parameters *a* and *b* must be positive. If they are not specified, DISTRIBUTION assumes *a* equals the minimum observation and estimates *b* by the maximum likelihood method. Negative observations are not allowed.

LNORMAL(a,b). *Lognormal distribution.* The scale and shape parameters *a* and *b* must be positive. If they are not specified, DISTRIBUTION estimates them from the mean and standard deviation of the natural logarithm of the sample data. Negative observations are not allowed.

BETA(a,b). *Beta distribution.* The shape1 and shape2 parameters *a* and *b* must be positive. If they are not specified, DISTRIBUTION estimates them from the sample mean and sample standard deviation. All observations must be between 0 and 1, inclusive.

GAMMA(a,b). *Gamma distribution.* The shape and scale parameters *a* and *b* must be positive. If they are not specified, DISTRIBUTION estimates them from the sample mean and sample standard deviation. Negative observations are not allowed.

LOGISTIC(a,b). *Logistic distribution.* LOGISTIC takes a location and a scale parameter (*a* and *b).* The scale parameter *(b*) must be positive. If the parameters are not specified, DISTRIBUTION estimates them from the sample mean and sample standard deviation.

LAPLACE(a,b). *Laplace or double exponential distribution.* LAPLACE takes a location and a scale parameter (*a* and *b).* The scale parameter *(b*) must be positive. If the parameters are not specified, DISTRIBUTION estimates them from the sample mean and sample standard deviation.

UNIFORM(a,b). *Uniform distribution.* UNIFORM takes a minimum and a maximum parameter *(a* and *b).* Parameter *a* must be equal to or greater than *b.* If the parameters are not specified, DISTRIBUTION assumes them from the sample data.

HNORMAL(a). *Half-normal distribution.* Data are assumed to be location-free or centralized. (Location parameter=0.) You can specify the scale parameter *a* or let DISTRIBUTION estimate it by using the maximum likelihood method.

CHI(df). *Chi-square distribution.* You must specify the degrees of freedom (*df)*. Negative observations are not allowed.

STUDENT(df). *Student's* t *distribution.* You must specify the degrees of freedom (*df).*

FRACTION Subcommand

FRACTION specifies the formula to be used in estimating the empirical distribution in p-p plots and calculating the expected quantile values in q-q plots.

- Only one formula can be specified. If more than one formula is specified, only the first formula is used.
- If the FRACTION subcommand is not specified, BLOM is used by default.
- These formulas produce noticeable differences for short series only.

Four formulas are available:

BLOM. Blom's transformation, defined by the formula (*r* – (3/8)) / (*n* + (1/4)), where *n* is the number of observations and *r* is the rank, ranging from 1 to *n* [53].

RANKIT. Formula (*r* – (1/2)) / *n*, where *n* is the number of observations and *r* is the rank, ranging from 1 to *n* [54].

TUKEY. Tukey's transformation, defined by the formula (*r* – (1/3)) / (*n* + (1/3)), where *n* is the number of observations and *r* is the rank, ranging from 1 to *n* [55].

VW. Van der Waerden's transformation, defined by the formula *r* / (*n* +1), where *n* is the number of observations and *r* is the rank, ranging from 1 to *n* [56].

Example

```
PPLOT VARIABLES = VARX
  /FRACTION=VW.
```

- This PPLOT command uses van der Waerden's transformation to approximate the proportion estimate *p*, which is used in the inverse distribution function.
- By default, two q-q plots are produced.

TIES Subcommand

TIES determines the way that tied values are handled. The default method is MEAN.

MEAN. *Mean rank of tied values is used for ties.* This setting is the default.

LOW. *Lowest rank of tied values is used for ties.*

HIGH. *Highest rank of tied values is used for ties.*

BREAK. *Consecutive ranks with ties sharing the same value.* Each distinct value of the ranked variable is assigned a consecutive rank. Ties share the same rank.

TYPE Subcommand

TYPE specifies the type of plot to produce. The default is Q-Q. The plots show a quantile-quantile plot and a proportion-proportion plot using the same data (with a normal distribution).

Q-Q. *Quantile-quantile plots.* The quantiles of the observed values are plotted against the quantiles of the specified distribution.

P-P. *Proportion-proportion plots*. The observed cumulative proportion is plotted against the expected cumulative proportion if the data were a sample from a specified distribution.

---

[53] Blom, G. 1958. *Statistical estimates and transformed beta variables*. New York: John Wiley and Sons.
[54] Chambers, J. M., W. S. Cleveland, B. Kleiner, and P. A. Tukey. 1983. *Graphical methods for data analysis*. Boston: Duxbury Press.
[55] Tukey, J. W. 1962. The future of data analysis. *Annals of Mathematical Statistics,* 33:22, 1-67.
[56] Lehmann, E. L. 1975. *Nonparametrics: Statistical methods based on ranks*. San Francisco: Holden-Day.

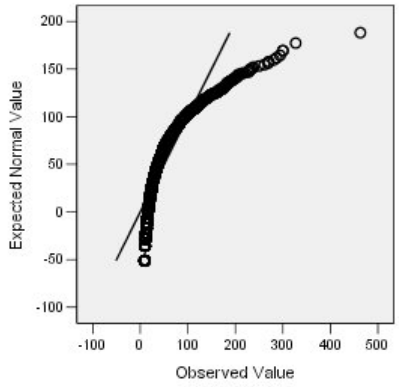


Figure 83. Normal q-q plot of household income

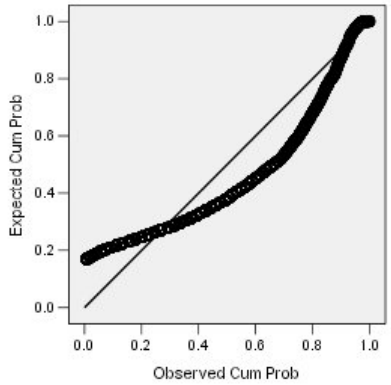


Figure 84. Normal p-p plot of household income

PLOT Subcommand

PL0T specifies whether to produce a plot of observed values versus expected values, a plot of observed values versus deviations from expected values, or both. The plots shown in "TYPE Subcommand " on page 1494 are nondetrended plots. The figure below shows a detrended q-q plot.

BOTH. *Display both detrended and nondetrended normal plots.* This is the default.

NORMAL. *Display nondetrended normal plots.* The observed values are plotted against the expected values.

DETRENDED. *Display detrended plots.* The observed values are plotted against the deviations from the expected values.

- If you specify PL0T more than once, only the last specification is executed.
- Deviations are calculated by subtracting the expected value from the observed value.
- In low resolution, a dash is used in a detrended plot to indicate where the deviation from the expected is 0.

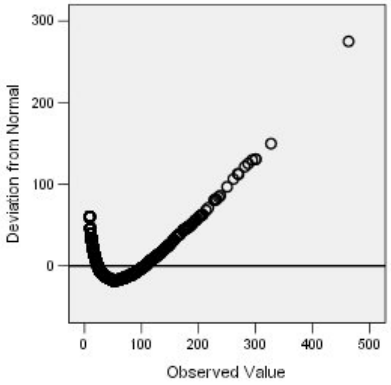


Figure 85. Detrended normal q-q plot of household income

STANDARDIZE and NOSTANDARDIZE Subcommands

STANDARDIZE transforms the sequence or time series variables into a sample with a mean of 0 and a standard deviation of 1. NOSTANDARDIZE is the default and indicates that the series should not be standardized.

- There are no additional specifications on the STANDARDIZE or NOSTANDARDIZE subcommands.
- Only the last STANDARDIZE or NOSTANDARDIZE subcommand on the PPLOT command is executed.
- The STANDARDIZE and NOSTANDARDIZE subcommands have no effect on expected values, which are always standardized.
- NOSTANDARDIZE is generally used with an APPLY subcommand to turn off a previous STANDARDIZE specification.

Example

```
PPLOT VARIABLES = VARX
  /STANDARDIZE.
```

- This example produces two q-q normal-probability plots of *VARX* with standardized observed values.

DIFF Subcommand

DIFF specifies the degree of differencing that is used before plotting to convert a nonstationary variable into a stationary variable with a constant mean and variance.

- You can specify any positive integer on DIFF.
- If DIFF is specified without a value, the default is 1.
- The number of plotted values decreases by 1 for each degree of differencing.

Example

```
PPLOT VARIABLES = TICKETS
  /DIFF=2.
```

- In this example, *TICKETS* is differenced twice before the expected and observed values are plotted.

SDIFF Subcommand

If the variable exhibits a seasonal or periodic pattern, you can use the SDIFF subcommand to seasonally difference the variable before plotting.

- The specification on SDIFF indicates the degree of seasonal differencing and can be any positive integer.
- If SDIFF is specified without a value, the degree of seasonal differencing defaults to 1.
- The number of plotted seasons decreases by 1 for each degree of seasonal differencing.
- The length of the period that is used by SDIFF is specified on the PERIOD subcommand. If the PERIOD subcommand is not specified, the periodicity that is established on the TSET or DATE command is used (see “PERIOD Subcommand ” on page 1497).

PERIOD Subcommand

PERIOD indicates the length of the period to be used by the SDIFF subcommand.

- The specification on PERIOD indicates how many observations are in one period or season. You can specify any positive integer on PERIOD.
- The PERIOD subcommand is ignored if it is used without the SDIFF subcommand.
- If PERIOD is not specified, the periodicity that is established on TSET PERIOD is in effect. If TSET PERIOD is not specified either, the periodicity that is established on the DATE command is used. If periodicity was not established anywhere, the SDIFF subcommand will not be executed.

Example

```
PPLOT VARIABLES = TICKETS
  /SDIFF=1
  /PERIOD=12.
```

- This command applies 1 degree of seasonal differencing with 12 observations per season to the variable *TICKETS*.

LN and NOLOG Subcommands

LN transforms the data by using the natural logarithm (base *e*) to remove varying amplitude. NOLOG indicates that the data should not be log transformed. NOLOG is the default.

- There are no additional specifications on LN or NOLOG.
- Only the last LN or NOLOG subcommand on a PPLOT command is executed.
- If a natural log transformation is requested, cases with values that are less than or equal to 0 will be set to system-missing, because nonpositive values cannot be log-transformed.
- NOLOG is generally used with an APPLY subcommand to turn off a previous LN specification.

Example

```
PPLOT VARIABLES = TICKETS
  /FRACTION=TUKEY
  /DIFF=1
  /LN.
PPLOT VARIABLES = EARNINGS
  /APPLY
  /NOLOG.
```

- The first command requests a natural log transformation of variable *TICKETS* before plotting.
- The second command applies the previous PPLOT specifications to variable *EARNINGS*. However, *EARNINGS* is not log-transformed before plotting.

APPLY Subcommand

APPLY allows you to produce a plot by using previously defined specifications without having to repeat the PPLOT subcommands.

- The only specification on APPLY is the name of a previous model in quotation marks. If a model name is not specified, the model that is specified on the previous PPLOT command is used.
- To change any plot specifications, specify the subcommands of only those portions that you want to change. Make these entries after the APPLY subcommand.
- If no variables are specified, the variables that were specified for the original plot are used.
- To change the variables that are used with the model, enter new variable names before or after the APPLY subcommand.
- The distribution type is applied, but the parameters are not applied.

Example

```
PPLOT VARIABLES = X1
  /FRACTION=TUKEY.
PPLOT VARIABLES = Z1
  /APPLY.
```

- The first command produces two q-q normal-probability plots of *X1*, using Tukey's transformation to compute the expected values.
- The second command requests the same plots for variable *Z1*.

Example

```
PPLOT VARIABLES = X1 Y1 Z1
  /FRACTION=VW.
PPLOT APPLY
  /FRACTION=BLOM.
```

- The first command uses van der Waerden's transformation to calculate expected normal values of *X1*, *Y1*, and *Z1*.
- The second command uses Blom's transformation for the same three series.

Example

```
PPLOT VARIABLES = VARX
  /FRACTION=RANKIT
  /DIFF
  /STANDARDIZE.
PPLOT VARIABLES = VARY
  /APPLY
  /NOSTANDARDIZE.
```

- The first command differences and standardizes series *VARX* and then produces a normal probability plot by using the RANKIT transformation.
- The second command applies the previous plot specifications to *VARY* but does not standardize the series.

TEMPLATE Subcommand

TEMPLATE uses an existing file as a template and applies it to any charts generated by the current PPLOT command. The template overrides the default settings that are used to create any charts, and the specifications on the current PPLOT command override the template. Templates are created in the Chart Editor by saving an existing chart as a template.

Example

```
PPLOT VARIABLES = varx
  /FRACTION=TUKEY
  /DIFF=2
  /TEMPLATE='/templates/mytemplate.sgt'.
```

References

Blom, G. 1958. *Statistical estimates and transformed beta variables*. New York: John Wiley and Sons.

Chambers, J. M., W. S. Cleveland, B. Kleiner, and P. A. Tukey. 1983. *Graphical methods for data analysis*. Boston: Duxbury Press.

Lehmann, E. L. 1975. *Nonparametrics: Statistical methods based on ranks*. San Francisco: Holden-Day.

Tukey, J. W. 1962. The future of data analysis. *Annals of Mathematical Statistics,* 33:22, 1-67.

PREDICT

```
PREDICT  [{start date       }] [THRU [{end date       }]]
          {start case number}         {end case number}
                                      {END             }
```

Example

```
PREDICT Y 61 THRU Y 65.
```

Overview

PREDICT specifies the observations that mark the beginning and end of the forecast period. If the forecast period extends beyond the length of the series, PREDICT extends the series in the active dataset to allow room for the forecast observations.

Basic Specification

The minimum specification on PREDICT is either the start or the end of the range, or it is keyword THRU. PREDICT sets up a forecast period beginning and ending with the specified dates or case numbers. The default starting point is the observation immediately after the end of the series or, if USE is specified, the observation immediately after the end of the use range (the historical period). The default end is the last observation in the series.

Operations

- PREDICT is executed when the data are read for the next forecasting procedure (for example, CURVEFIT in the Core system, and 2SLS in the Regression add-on module).
- PREDICT is ignored by non-forecasting procedures.
- Case number specifications refer to the sequential numbers that are assigned to cases as they are read.
- If the forecast period extends beyond the length of the series, PREDICT extends the series in the active dataset to allow room for the forecast observations.
- New observations that are added to the end of existing series will contain non-missing date variables, forecast values (variable *FIT#n*), confidence interval limits (variables *LCL#n* and *UCL#n*), and, for ARIMA models, standard error of the predicted value (*SEP#n*). For all other variables, including the original series, the new cases will be system-missing.
- PREDICT cannot forecast beyond the end of the series for ARIMA with regressors and 2SLS. However, PREDICT *can* forecast values for the dependent variable if the independent variables have valid values in the predict period.
- If the use and predict periods overlap, the model is still estimated by using all observations in the use period.
- USE and PREDICT can be used together to perform forecasting validation. To do this, specify a use period that ends before the existing end of the series, and specify a predict period starting with the next observation.
- If there is a gap between the end of the use period and the start of the specified predict period, the program uses the first observation after the end of the use period as the start of the predict period. (This setting is the default.)
- The DATE command turns off all existing USE and PREDICT specifications.
- PREDICT remains in effect in a session until it is changed by another PREDICT command or until a new DATE command is issued.
- If more than one forecasting procedure is specified after PREDICT, the USE command should be specified between procedures so that the original series—without any new, system-missing cases—will be used each time. Alternatively, you can specify

```
TSET NEWVAR = NONE
```

before the first forecasting procedure so that you can evaluate model statistics without creating new variables or adding new cases with missing values to the original series.

Limitations

A maximum of one range (one start and/or one end) can be specified per PREDICT command.

Syntax Rules

- You can specify a start, an end, or both.
- The start and end are specified as either date specifications or case (observation) numbers.
- Date specifications and case numbers cannot be mixed on one PREDICT command.
- Keyword THRU is required if the end of the range is specified.
- Keyword THRU by itself defines a PREDICT range starting with the first observation after the use range and ending with the end of the series. If USE has not been specified, PREDICT THRU is meaningless.

Date Specifications

- A date specification consists of DATE keywords and values (see the DATE command). These specifications must correspond to existing date variables.
- If more than one date variable exists, the highest-order variable must be included in the date specification.
- Values on keyword YEAR must have the same format (two or four digits) as the YEAR specifications on the DATE command.

Case Specifications

The case number specification is the sequence number of the case (observation) as it is read by the program.

Valid Range

- The start date must precede the end date.
- The start case number must be less than the end case number.
- The start can be any observation ranging from the second observation in the historical period that is specified on USE to the observation immediately following the end of the historical period. If USE is not specified, the start can be any observation ranging from the second observation in the series to the observation immediately following the end of the series.
- For most models, the start of the predict period should not be too close to the start of the use period.
- The predict and use periods should not be exactly the same.
- The start of the predict period should not precede the start of the use period.

Examples

Specifying the Forecast Period as a Date Range

```
PREDICT Y 61 THRU Y 65.
```

- This command specifies a forecast period from 1961 to 1965.
- The active dataset must include variable *YEAR_*, which, in this example, contains only the last two digits of each year.
- If variable *MONTH_* also exists, the above command is equivalent to

```
PREDICT Y 61 M 1 THRU Y 65 M 12.
```

```
PREDICT W 28 THRU W 56.
```

- This command specifies a forecast period from the 28th week to the 56th week.
- The active dataset must include variable *WEEK_*.
- If variable *DAY_* also exists, the above command is equivalent to

```
PREDICT W 28 D 1 THRU W 56 D 7.
```

Specifying the Forecast Period as a Case Range

```
PREDICT 61 THRU 65.
```

- This command specifies a forecast period from the 61st case (observation) to the 65th case.

Using the Default Start Date

```
PREDICT THRU Y 65.
```

- This command uses the default start date, which is the observation immediately following the end of the use period. If USE is not specified, the default start is the observation immediately following the end of the series.
- The forecast period extends from the start date through year 1965.
- The active dataset must include variable *YEAR_*.
- Keyword THRU is required.

Specifying the Forecast Period by Using Date Variables

```
PREDICT THRU CYCLE 4 OBS 17.
```

- This example uses the date variables *OBS_* and *CYCLE_*, which must exist in the active dataset.
- *CYCLE*, the highest order, must be included on PREDICT.
- Keyword THRU is required.
- The forecast period extends from the default start to the 17th observation of cycle 4.

PREFSCAL

PREFSCAL is available in SPSS Statistics Professional Edition or the Categories option.

```
PREFSCAL VARIABLES = varlist

   [/INPUT = [ROWS({n    })]
             {rowid}
          [SOURCES({1**     })]]
                   {n       }
                   {sourceid}

   [/PROXIMITIES = {DISSIMILARITIES**}]
                  {SIMILARITIES     }

   [/WEIGHTS = varlist]

   [/INITIAL = {CLASSICAL[({TRIANGLE**})]}]
                          {SPEARMAN  }
               {CORRESPONDENCE           }
               {ROSSCLIFF                }
               {CENTROIDS[({1})]         }
                          {n}
               {RANDOM[({1})]            }
                       {n}
               {(filespec) [varlist]     }

   [/CONDITION = {ROW**  }]
                 {MATRIX}
                 {UNCONDITIONAL    }

   [/TRANSFORMATION = {NONE[(INTERCEPT)]                              }]
                      {LINEAR[(INTERCEPT)]                            }
                      {ORDINAL[({UNTIE   })]                          }
                               {KEEPTIES}**
                      {SMOOTH[({UNTIE   })]                           }
                               {KEEPTIES}
                      {SPLINE[([INTERCEPT] [ORDER={2}] [INKNOT={1}])]}
                                                  {n}          {n}

   [/MODEL = {IDENTITY**  }]
             {WEIGHTED    }
             {GENERALIZED}

   [/RESTRICTIONS = {ROW   ({NONE**     } (filespec) [varlist])}]
                           {COORDINATES}
                    {COLUMN({NONE**     } (filespec) [varlist])}
                           {COORDINATES}

   [/PENALTY = [LAMBDA({0.5**})]
                       {value}
               [OMEGA({1.0**})]
                      {value}

   [/CRITERIA = [DIMENSIONS({2**      })]
                           {min[,max]}
                [MAXITER({5000**})]
                         {value }
                [DIFFSTRESS({0.000001**})]
                            {value    }
                [MINSTRESS({0.0001**})]]
                           {value   }

   [/PRINT = [NONE]  [INPUT]  [MULTIPLE]  [INITIAL]
             [HISTORY]  [MEASURES**]  [DECOMPOSITION]
             [COMMON**]  [DISTANCES]  [WEIGHTS**]
             [INDIVIDUAL]  [TRANSFORMATION]]

   [/PLOT = [NONE]  [MULTIPLE]  [INITIAL]  [STRESS]
            [COMMON**]  [WEIGHTS**]
            [INDIVIDUAL[(valuelist) [...]]]
            [TRANSFORMATIONS[(valuelist) [...]]]
            [SHEPARD[(valuelist) [...]]]
            [FIT[(valuelist) [...]]]
            [RESIDUALS[(valuelist) [...]]]]

   [/OPTIONS = [MARKERS(rowid)]
               [COLORS(rowid)]]

   [/OUTFILE = [COMMON('savfile'|'dataset')]
               [WEIGHTS('savfile'|'dataset')]
               [DISTANCES('savfile'|'dataset')]
               [TRANSFORMATIONS('savfile'|'dataset')]]
```

* Default if the keyword is omitted.

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 14.0

- Command introduced.

Example

```
PREFSCAL var01 TO var02.
```

Overview

PREFSCAL performs multidimensional unfolding of proximity data to find a least-squares representation of the row and column objects in a low-dimensional space. Individual differences models are allowed for multiple sources. A majorization algorithm minimizes penalized Stress and guarantees monotone convergence for optionally transformed, metric and nonmetric data under a variety of models and constraints.

Options

Data Input. You can read one or more rectangular matrices of proximities. Additionally, you can read weights, an initial configuration, and fixed coordinates.

Methodological Assumptions. On the CONDITION subcommand, you can specify transformations for all sources (unconditional), separate transformations for each source (matrix-conditional), or separate transformations for each row (row-conditional). Using the TRANSFORMATION subcommand, you can treat proximities as nonmetric (ordinal or smooth ordinal), as quasi-metric (splines), or as metric (linear with or without intercept). Ordinal and smooth ordinal transformations can keep tied observations tied (discrete) or untie them (continuous). You can use the PROXIMITIES subcommand to specify whether your proximities are similarities or dissimilarities.

Model Selection. You can specify multidimensional unfolding models by selecting a combination of PREFSCAL subcommands, keywords, and criteria. The subcommand MODEL offers the Identity model and two individual differences models. You can specify other selections on the CRITERIA subcommand.

Penalties. You can specify two penalty parameters in order to avoid degenerate solutions. On the PENALTY subcommand, LAMBDA is specified for the strength, and OMEGA is specified for the range. Penalized Stress penalizes solutions with insufficient variation in the transformed proximities.

Constraints. You can specify fixed coordinates on the RESTRICTIONS subcommand to restrict some or all common space coordinates of either row objects or column objects.

Output. You can produce output that includes the original and transformed proximities, history of iterations, common and individual configurations, individual space weights, distances, and decomposition of the Stress. Plots can be produced of common and individual configurations (biplots), individual space weights, transformations, fit, and residuals.

Basic Specification

The basic specification is PREFSCAL followed by a variable list. By default, PREFSCAL produces a two-dimensional metric Euclidean multidimensional unfolding solution (Identity model). Input is expected to contain one or more rectangular matrices with proximities that are dissimilarities. The transformation of the proximities is row-conditional. The analysis uses a classical scaling start as initial configuration. By default, output includes fit and Stress values, the coordinates of the common space, and a joint plot of the common space configuration.

Syntax Rules

- If there is only one source, the model is always assumed to be Identity.
- In the case of duplicate or contradicting subcommand specification, only the later subcommand applies.
- There is no constraint with respect to the syntax order.

Limitations

- PREFSCAL needs at least two objects in each set. At least two variables need to be specified in the variable list (two column objects), and the active data file needs to contain at least two cases (two row objects).
- PREFSCAL does not honor SPLIT FILE.

Examples

```
PREFSCAL
  VARIABLES=TP BT EMM JD CT BMM HRB TMd BTJ TMn CB DP GD CC CMB
  /INPUT=SOURCES(srcid )
  /INITIAL=CLASSICAL (SPEARMAN)
  /CONDITION=ROW
  /TRANSFORMATION=NONE
  /PROXIMITIES=DISSIMILARITIES
  /MODEL=WEIGHTED
  /CRITERIA=DIMENSIONS(2,2) DIFFSTRESS(.000001) MINSTRESS(.0001)
  MAXITER(5000)
  /PENALTY=LAMBDA(0.5) OMEGA(1.0)
  /PRINT=MEASURES COMMON
  /PLOT=COMMON WEIGHTS INDIVIDUAL ( ALL ) .
```

- This syntax specifies an analysis on variables *tp (Toast pop-up)* through *cmb (Corn muffin and butter)*. The variable *srcid* is used to identify the sources.
- The INITIAL subcommand specifies that the starting values be imputed using Spearman distances.
- The MODEL subcommand specifies a weighted Euclidean model, which allows each individual space to weight the dimensions of the common space differently.
- The PLOT subcommand requests plots of the common space, individual spaces, and individual space weights.
- All other parameters fall back to their default values.

For discussion of output from this syntax and to place the example in context, see .

VARIABLES Subcommand

The variable list identifies the columns in the proximity matrix or matrices that PREFSCAL reads. Each variable identifies one column of the proximity matrix, with each case in the active dataset representing one row.

- Only numeric variables may be specified.
- PREFSCAL reads data row by row, and the columns are represented by the variables on the variable list.

Example

```
DATA LIST
  /var01 var02.
BEGIN DATA
  1 6
  5 4
  4 2
END DATA.
PREFSCAL var01 TO var02.
```

- This example specifies an analysis on a 3 × 2 proximity matrix (3 rows and 2 columns).

INPUT Subcommand

The INPUT subcommand specifies the number of rows in one source, the number of sources, or both. Specifying a row identifier, *rowid*, or a source identifier, *sourceid*, specifically identifies either the row objects or sources and provides a variable that may contain row object or source labels. Specifying only one keyword computes the number of row objects or sources according to the following formula: C=R × S, where C is the number of cases, R is the number of row objects, and S is the number of sources. By specifying both keywords, PREFSCAL takes the first R × S cases from the active file.

ROWS. *Number of rows.* This specifies the number of row objects in one source. A variable in parentheses specifies a row identifier. The values must be positive nonzero integers. The values of this variable specify the identifier of the rows that contain the proximities. Within one source, the values of this identifier only need to be discriminating. Over sources, the values of the row identifier must be in the same order.

SOURCES. *Number of sources.* This keyword specifies the number of sources. By default, the number of sources is 1. Otherwise, the number of cases in the active file must be dividable by the number of sources. A variable in parentheses specifically specifies a source identifier. The values of this identifier specify the sources and must be positive nonzero integers. The rows within one source must be consecutive cases in the active data file.

- *rowid* and *sourceid* may not be specified on the PREFSCAL variable list.
- Duplicate cell specification is not taken care of. In this case, the final specification applies.

Example

```
DATA LIST
  /var01 TO var15 rowid sourceid.
BEGIN DATA
  13 12 07 03 05 04 08 11 10 15 02 01 06 09 14   1  1
  15 11 06 03 10 05 14 08 09 12 07 01 04 02 13   2  1
  15 10 12 14 03 02 09 08 07 11 01 06 04 05 13   3  1
  (...)
  10 03 02 14 09 01 08 12 13 04 11 05 15 06 07  41  1
  13 03 01 14 04 10 05 15 06 02 11 07 12 08 09  42  1
  15 03 05 12 02 08 07 13 01 04 06 10 11 09 14   1  2
  15 04 03 11 07 05 14 01 02 06 08 13 09 12 10   2  2
  15 02 07 12 05 06 04 08 01 03 09 10 11 14 13   3  2
  (...)

  (...)
  09 04 07 10 11 02 08 12 13 05 14 06 15 01 03  41  6
  15 10 01 12 02 06 08 14 13 11 09 03 04 05 07  42  6
END DATA.
```

- The active data file has 252 cases, containing 6 sources with 42 row objects per source and containing 15 column objects. Additionally, 2 identifying variables, *rowid* and *sourceid*, are specified to identify the row objects and sources, respectively.

```
PREFSCAL var01 TO var15
  /INPUT = ROWS(42).
```

- PREFSCAL reads 15 columns and 42 rows per source—thus, 6 sources in total (252/42).

```
PREFSCAL var01 TO var15
  /INPUT = SOURCES(6).
```

- PREFSCAL reads 15 columns and 6 sources, with 42 rows each (252/6).

```
PREFSCAL var01 TO var15
  /INPUT = ROWS(rowid).
```

- PREFSCAL reads 15 columns and 42 rows per source. The row objects are specified by *rowid*, which ranges from 1 to 42, the number of row objects in this case (per source, thus 6 sources). When a lower value is found in the row object identifier variable, a new source is started.

```
PREFSCAL var01 TO var15
  /INPUT = SOURCES(sourceid).
```

- PREFSCAL reads 15 columns and 6 sources. The sources are specified by *sourceid*, which ranges from 1 to the number of sources (in this case, from 1 to 6). When a higher value is found in the source identifier variable, a new source is started.

```
COMPUTE rowid = 1+MOD($casenum-1,42).
COMPUTE sourceid = 1+TRUNC(($casenum-1)/42).
SORT CASES BY sourceid (A) rowid (A).
VALUE LABELS sourceid 1 'overall'  2 'bacon'  3 'cereal'
                      4 'pancakes' 5 'normal' 6 'snack'.
PREFSCAL var01 TO var15
  /INPUT = ROWS(rowid) SOURCES(sourceid).
```

- First, a row object identifier and a source identifier are computed. The active data file is sorted by *rowid* and *sourceid*. The variable *sourceid* is given value labels. PREFSCAL reads 15 columns and 6 sources per row object, with 42 row objects in total. The first nine case of the active data file look like this:

```
13 12  7  3  5  4  8 11 10 15  2  1  6  9 14 1 1
15  3  5 12  2  8  7 13  1  4  6 10 11  9 14 1 2
```

```
14 10  9  3  7  6 12 15  8 11  5  1  4  2 13 1 3
14  9  7  1 11  6 12 15 13 10  3  4  2  5  8 1 4
10 12  8  4  9  6  7 15 14 13  3  1  5  2 11 1 5
14 11  8  4  9  6  7 15 10 12  3  2  5  1 13 1 6
15 11  6  3 10  5 14  8  9 12  7  1  4  2 13 2 1
15  4  3 11  7  5 14  1  2  6  8 13  9 12 10 2 2
15  8  2  3 10  1 14  5  6  9 11 12  4 13  7 2 3
(...)
```

PROXIMITIES Subcommand

The PROXIMITIES subcommand specifies the type of proximities that are used in the analysis. The term **proximity** is used for either similarity or dissimilarity data. Internally, PREFSCAL works with dissimilarities. Therefore, PREFSCAL converts similarities into dissimilarities by reflecting the data about its midpoint (depending on the conditionality chosen on the CONDITION subcommand), thus preserving the endpoints and the range.

DISSIMILARITIES. *Dissimilarity data.* This specification is the default when PROXIMITIES is not specified. Small dissimilarities correspond to small distances, and large dissimilarities correspond to large distances.

SIMILARITIES. *Similarity data.* Small similarities correspond to large distances, and large similarities correspond to small distances.

Example

```
PREFSCAL var01 TO var09
  /PROXIMITIES = SIMILARITIES.
```

- In this example, PREFSCAL expects the proximities to be similarities.

WEIGHTS Subcommand

The WEIGHTS subcommand specifies the variables that contain the nonnegative weights for the proximities that are included in the active dataset.

- The number and order of the variables in the variable list is important. The first variable in the WEIGHTS variable list corresponds to the first variable on the PREFSCAL variable list. This correspondence is repeated for all variables on the variable lists. Every proximity has its own weight. Therefore, the number of variables on the WEIGHTS subcommand must be equal to the number of variables on the PREFSCAL variable list.
- Negative weights are not allowed. If negative weights are specified, an error message is issued, and the procedure aborts.
- The weight variable (set with WEIGHT BY) allows for the weighting of entire rows. The weight variable must contain positive, nonzero values.

Example

```
DATA LIST FILE = 'breakfast.dat' FREE
  /var01 TO var15 wgt01 TO wgt15.
PREFSCAL var01 TO var15
  /WEIGHTS = wgt01 TO wgt15.
```

- In this example, the PREFSCAL variable list indicate that there are 15 column objects, of which the weights can be found in *wgt01* to *wgt15*.
- *wgt01* contains the weights for *var01*, *wgt02* contains the weights for *var02*, and so on.

INITIAL Subcommand

INITIAL defines the initial or starting configuration of the common space for the analysis. When a reduction in dimensionality is specified on the CRITERIA subcommand, a derivation of coordinates in the higher dimensionality is used as starting configuration in the lower dimensionality.

- You can specify one of the five keywords that are listed below.
- You can specify a variable list containing the initial configuration.

CLASSICAL. *Classical scaling start.* This specification is the default. The rectangular proximity matrix is used to supplement the intra-blocks (values between rows and between columns) of the complete symmetrical MDS matrix by means of the triangular inequality or Spearman distances. When the complete matrix is formed, a classical scaling solution is used as initial configuration.

ROSSCLIFF. *Ross-Cliff start.* The Ross-Cliff start uses the results of a singular value decomposition on the double centered and squared proximity matrix as the initial values for the row and column objects.

CORRESPONDENCE. *Correspondence start.* The correspondence start uses the results of a correspondence analysis on the reversed data (similarities instead of dissimilarities) with symmetric normalization of row and column scores. See the topic "CORRESPONDENCE" on page 355 for more information.

CENTROIDS(n). *Centroids start.* PREFSCAL starts by positioning the row objects in the configuration by using an eigenvalue decomposition. Then, the column objects are positioned at the centroid of first choices (or second if *n*=2 or third if *n*=3, etc.). The number of choices (*n*) must be a positive integer between 1 and the number of columns. The default is 1.

RANDOM(n). *(Multiple) random start.* You can specify the number of random starts (*n*), where *n* is any positive integer. The random sequence can be controlled by the SET SEED procedure (thus, not by a subcommand within the PREFSCAL procedure). All *n* analyses start with a different random configuration. In the output, all *n* final Stress values are reported, as well as the initial seeds of each analysis (for reproduction purposes), followed by the full output of the analysis with the lowest penalized Stress value. The default number of random starts is 1.

CLASSICAL Keyword

TRIANGLE. *Imputation using the triangle inequality.* If TRIANGLE is specified, the intra-blocks are filled by using the triangular inequality.

SPEARMAN. *Imputation with Spearman distances.* If SPEARMAN is specified, the Spearman distances between all objects are used to create a symmetrical MDS matrix.

Instead of these keywords, a filespec in parentheses can be given to specify the IBM SPSS Statistics data file containing the coordinates of the initial configuration. The row and column coordinates are stacked, with the column coordinates following the row coordinates. The closing parenthesis of the filespec can be followed by a variable list. If the variable list is omitted, the procedure automatically selects the first *MAXDIM* variables in the external file, where *MAXDIM* is the maximum number of dimensions that are requested for the analysis on /CRITERIA = DIMENSIONS(min, max). Missing values are not allowed as initial coordinates. An error is issued whenever this situation occurs.

Example

```
PREFSCAL var01 TO var15
  /INITIAL = RANDOM(100).
```

- This example performs 100 analyses (each analysis starting with a different random configuration). The results of the analysis with the lowest final Stress are displayed in the output.

CONDITION Subcommand

CONDITION specifies the range of proximities that are compared within one transformation list. The TRANSFORMATION subcommand specifies the type of transformation.

ROW. *Row conditional.* Only the proximities within each row are compared with each other. The comparison is carried out for each row separately. This setting is the default.

MATRIX. *Matrix conditional.* Only the proximities within each source are compared with each other. The comparison is carried out for each source separately.

UNCONDITIONAL. *Unconditional.* This specification is appropriate when the proximities in all sources can be compared with each other, and it results in a single transformation of all sources simultaneously.

- Note that if there is only one source, MATRIX and UNCONDITIONAL yield the same result.

Example

```
PREFSCAL var01 TO var09
  /CONDITION = UNCONDITIONAL
  /TRANSFORMATION = LINEAR(INTERCEPT).
```

- In this example, the proximities are linearly transformed, including an intercept. The transformation is carried out over all proximities simultaneously.

TRANSFORMATION Subcommand

The TRANSFORMATION subcommand offers four different options for optimal transformation of the original proximities. The resulting values are called transformed proximities. The distances between the objects in the configuration should match these transformed proximities as closely as possible. The CONDITION subcommand specifies over which proximities the transformation is computed. The default transformation is ORDINAL with ties kept tied.

NONE. *No scale transformation.* The INTERCEPT keyword can be specified in parentheses following the NONE keyword. If INTERCEPT is specified, an intercept is estimated in the transformation.

LINEAR. *Linear transformation.* With this transformation, the transformed proximities are proportional to the original proximities (that is, the transformation function estimates a slope, and the intercept is fixed at 0). The INTERCEPT keyword can be specified in parentheses following the LINEAR keyword. If INTERCEPT is specified, an intercept is estimated in the transformation, resulting in an interval transformation. Without the keyword INTERCEPT, LINEAR only estimates a slope, which coincides with a ratio transformation.

ORDINAL. *Ordinal transformation.* The transformed proximities have the same order as the original proximities. ORDINAL can be followed by a keyword in parentheses to indicate how to handle tied proximities.

SMOOTH. *Smooth ordinal transformation.* The transformed proximities have the same order as the original proximities, including a smoothness restriction. This restriction takes the differences between subsequent values into account. Restricting subsequent differences allows for a smooth ordinal transformation. SMOOTH can be followed by a keyword in parentheses to indicate how to handle tied proximities.

SPLINE. *Monotone spline transformation.* The transformed proximities are a smooth nondecreasing piecewise polynomial transformation of the original proximities of the chosen degree. The pieces are specified by the number and placement of the interior knots, of which the number can be specified with INKNOT.

ORDINAL and SMOOTH Keywords

UNTIE. *Untie ties.* Allowing tied proximities to be untied during transformations (also known as the primary approach to ties).

KEEPTIES. *Keep ties tied.* Keeping tied proximities tied during transformations (also known as secondary approach to ties). This setting is the default.

SPLINE Keyword

INTERCEPT. *Include intercept.* If INTERCEPT is specified, an intercept is estimated in the transformation. Omitting this keyword sets the lower exterior knot equal to 0.

DEGREE. *The degree of the polynomial.* If DEGREE is not specified, the degree is assumed to be 2. The integer range of DEGREE is 1, 2, or 3.

INKNOT. *The number of interior knots.* If INKNOT is not specified, the number of interior knots is assumed to be 1. The integer range of INKNOT is between 0 and the number of different proximities minus 2.

Example

```
PREFSCAL var01 TO var15
  /TRANSFORMATION = ORDINAL(UNTIE).
```

- In this example, the proximities are ordinally transformed, where tied proximities are allowed to be untied.
- The CONDITION subcommand is omitted, and thus, the default conditionality ROW is in effect, which implies that the transformation is performed for each row separately.

MODEL Subcommand

MODEL defines the scaling model for the analysis if more than one source is present. IDENTITY is the default model. The other models are individual differences models.

IDENTITY. *Identity model.* All sources have the same individual configuration. This model is the default model, and it is not an individual differences model.

WEIGHTED. *Weighted Euclidean model.* This model is an individual differences model (and equivalent to the INDSCAL model). Each source has an individual space, in which every dimension of the common space is weighted differentially.

GENERALIZED. *Generalized Euclidean model.* This model is an individual differences model (and equivalent to the IDIOSCAL model). Each source has an individual space that is equal to a differential rotation of the common space, followed by a differential weighting of the dimensions.

- If IDENTITY is specified for only one source, this subcommand is silently ignored.
- If an individual differences model is specified for only one source, a warning is issued, and the model is set to IDENTITY.

Example

```
PREFSCAL var01 TO var15
  /INPUT = SOURCES(6)
  /MODEL = WEIGHTED.
```

- A weighted Euclidean model is fitted for the six specified sources. As indicated on the INPUT subcommand, the number of cases must be dividable by 6 in this case.

RESTRICTIONS Subcommand

PREFSCAL allows (some) coordinates to be fixed in the common space configuration. Fixing an entire set (all row objects or all column objects) corresponds to performing external unfolding.

ROW. *Row restriction.* PREFSCAL allows one row object, multiple row objects, or all row objects to be free (NONE) or fixed to given coordinates (COORDINATES).

COLUMN. *Column restriction.* PREFSCAL allows one column object, multiple column objects, or all column objects to be free (NONE) or fixed to given coordinates (COORDINATES).

ROW or COLUMN Keywords

NONE. *No restriction.* The specified set of objects (ROW or COLUMN) has no restriction.

COORDINATES. Coordinates must be followed by a filespec in parentheses to specify the external IBM SPSS Statistics data file that contains the fixed coordinates for the specified set of objects. Following the parenthesized filespec, a variable list can be given. If the variable list is omitted, the procedure automatically selects the first *MAXDIM* variables in the external IBM SPSS Statistics data file, where *MAXDIM* is the maximum number of dimensions that are requested for the analysis on /CRITERIA = DIMENSIONS(min, max).

- The number of cases for each variable in the external IBM SPSS Statistics data file must be equal to the number of objects of the specified set (ROW or COLUMN).

- A missing value can be used to indicate that the coordinate on that dimension is free. The coordinates of objects with nonmissing values are kept fixed during the analysis.

Example

```
PREFSCAL var01 TO var15
  /RESTRICTIONS = ROW(NONE)
  /RESTRICTIONS = COLUMN(COORDINATES ("indcol.sav")).
```

- In this example, there are 15 column objects.
- The coordinates of the row objects are not restricted. Although this specification is the default, it is explicitly stated here in the syntax.
- The column objects have restrictions on the coordinates. The fixed coordinates are specified in the IBM SPSS Statistics data file *indcol.sav*. If *indcol.sav* contains more than two variables, only the first two variables are taken as fixed coordinates, because the maximum dimensionality is 2, and specific variables on the RESTRICTIONS subcommand are not given.

PENALTY Subcommand

The PENALTY subcommand specifies the values for the penalty parameters. The two keywords can be used to set the strength and the range of the penalty. The penalty itself is based on the coefficient of variation of the transformed proximities.

LAMBDA. *Strength parameter.* This parameter sets the strength of the penalty. The default value is 0.75. The range of this parameter is between 0 (exclusive) and 1 (inclusive). The smaller the values of lambda, the stronger the penalty (and vice versa).

OMEGA. *Range parameter.* This parameter sets the range of the penalty (that is, the moment the penalty becomes active). The parameter must have a non-negative value. If OMEGA is 0, the penalty is inactive. Increasing OMEGA provides a more active penalty. By default (OMEGA = 1.0), the range is equal to the variation coefficient of the original proximities. If OMEGA is increased, the function will search for a solution with a higher variation of the transformed proximities

Example

```
PREFSCAL var01 TO var09
  /PENALTY = LAMBDA(0.5)
             OMEGA(2.0).
```

- In this example, the variation range is increased by setting OMEGA equal to 2.0.

CRITERIA Subcommand

You can use CRITERIA to set the dimensionality and criteria for terminating the algorithm. You can specify one or more of the following keywords:

DIMENSIONS(min,max). *Minimum and maximum number of dimensions.* By default, PREFSCAL computes a solution in two dimensions. The minimum and maximum number of dimensions can be any integer between 1 and the number of objects minus 1 inclusive, as long as the minimum is less than or equal to the maximum. PREFSCAL starts computing a solution in the largest dimensionality and reduces the dimensionality in steps of one, until the lowest dimensionality is reached. Specifying a single value represents both minimum and maximum number of dimensions; thus, DIMENSIONS(4) is equivalent to DIMENSIONS(4,4).

MAXITER(n). *Maximum number of iterations.* By default, *n*=5000, specifying the maximum number of iterations that are performed while one of the convergence criteria below (DIFFSTRESS and MINSTRESS) is not reached. Decreasing this number might give less accurate results, but will take less time. The value *n* must have a non-negative integer value.

DIFFSTRESS. *Convergence criterion.* PREFSCAL minimizes the goodness-of-fit index "penalized Stress." By default, PREFSCAL stops iterating when the relative difference in consecutive penalized Stress values

is less than or equal to 0.000001. To obtain a more accurate solution, you can specify a smaller value. The specified value must be nonnegative.

MINSTRESS. *Minimum Stress value.* By default, PREFSCAL stops iterating when the penalized Stress value itself is small (that is, less than or equal to 0.001). To obtain a more accurate solution, you can specify a smaller value. The specified value must be nonnegative.

Example

```
PREFSCAL var01 TO var15
  /CRITERIA = DIMENSIONS(2,4) MAXITER(10000) DIFFSTRESS(1.0E-8).
```

- The maximum number of dimensions equals 4, and the minimum number of dimensions equals 2. PREFSCAL computes a four-dimensional, three-dimensional, and two-dimensional solution, respectively.
- The maximum number of iterations is set to 10000.
- The relative difference in penalized Stress convergence criterion is sharpened to 1.0E-8.

PRINT Subcommand

The PRINT subcommand controls the display of tables. By default, PREFSCAL displays the Stress and fit values for each analysis, the coordinates of the common space, and, if applicable, the individual space weights.

- Omitting the PRINT subcommand or specifying PRINT without keywords is equivalent to specifying COMMON and WEIGHTS.
- If a keyword(s) is specified, only the output for that particular keyword(s) is displayed.
- Inapplicable keywords are silently ignored. That is, a specified keyword for which no output is available —for example, the keyword INDIVIDUAL with only one source specified—will be silently ignored.

NONE. *No optional output.* Displays only the penalized Stress and corresponding fit values.

INPUT. *Input data.* Displays tables of the original proximities and, if present, the data weights, the initial configuration, and the fixed coordinates.

MULTIPLE. *Multiple random starts.* Displays the random number seed and penalized Stress value of each random start.

INITIAL. *Initial common space.* Displays the coordinates of the initial common space.

HISTORY. *History of iterations.* Displays the history of iterations of the main algorithm.

MEASURES. *Fit measures.* Displays different measures. The table contains several goodness-of-fit, badness-of-fit, Stress, and fit values. This setting is specified by default.

DECOMPOSITION. *Decomposition of Stress.* Displays a objects, rows, and sources decomposition of penalized Stress, including row, column, and source totals.

COMMON. *Common space.* Displays the coordinates of the common space. This is specified by default.

DISTANCES. *Distances.* Displays the distances between the objects in the configuration. This keyword must be used in combination with COMMON or INDIVIDUAL to actually produce a table with distances.

WEIGHTS. *Individual space weights.* Displays the individual space weights, if applicable (that is, if one of the individual differences models is specified on the MODEL subcommand). Depending on the model, the space weights are decomposed in rotation weights and dimension weights, which are also displayed. This setting is specified by default.

INDIVIDUAL. *Individual spaces.* The coordinates of the individual spaces are displayed only if one of the individual differences models is specified on the MODEL subcommand.

TRANSFORMATION. *Transformed proximities.* Displays the transformed proximities.

Example

```
PREFSCAL var01 TO var15
  /INPUT = ROWS(42)
  /MODEL = WEIGHTED
  /PRINT = HISTORY COMMON MEASURES.
```

- Here, a weighted Euclidean model is specified with multiple sources.
- The optional output consists of a table with the history of iterations, the coordinates of the common space, and Stress and fit measures.

PLOT Subcommand

The PLOT subcommand controls the display of plots. By default, PREFSCAL displays the object points of the common space and, if applicable, the individual space weights.

- Omitting the PLOT subcommand or specifying PLOT without keywords produces the default plots.
- If a keyword(s) is specified, only the plot for that particular keyword(s) is displayed.
- Inapplicable keywords (for example, STRESS with equal minimum and maximum number of dimensions on the CRITERIA subcommand) are silently ignored.
- Multiple value lists are allowed for INDIVIDUAL, TRANSFORMATIONS, SHEPARD, FIT, and RESIDUALS. For each value list, a separate plot will be displayed.

NONE. *No plots.* PREFSCAL does not produce any plot.

MULTIPLE. *Multiple random starts.* Displays a stacked histogram of penalized Stress, displaying both Stress and penalty.

INITIAL. *Initial common space.* Displays a scatterplot matrix of the coordinates of the initial common space.

STRESS. *Scree plot.* Produces a lineplot of penalized Stress versus dimensions. This plot is only produced if the maximum number of dimensions is larger than the minimum number of dimensions.

COMMON. *Common space.* A scatterplot matrix of coordinates of the common space is displayed. This setting is the default.

WEIGHTS. *Individual space weights.* A scatterplot is produced for the individual space weights. This setting is only applicable if one of the individual differences models is specified on the MODEL subcommand. For the weighted Euclidean model, the weights for all sources are displayed in a plot, with one dimension on each axis. For the generalized Euclidean model, one plot is produced per dimension, indicating both rotation and weighting of that dimension for each source.

INDIVIDUAL(valuelist). *Individual spaces.* The coordinates of the individual spaces are displayed in scatterplot matrices. This setting is only applicable if one of the individual differences models is specified on the MODEL subcommand. For each source that is specified on the value list, a scatterplot matrix of coordinates of the individual space is displayed. The sources are specified by a number between 1 and the total number of sources or is specified by a value from the sourceid, which is specified on the INPUT subcommand.

TRANSFORMATIONS(valuelist). *Transformation plots.* A line plot is produced of the original proximities versus the transformed proximities. On the value list, the names (identifiers) for which the plot is to be produced must be specified. Because the CONDITION subcommand allows for the specification of multiple transformation lists, the value lists depend on the conditionality. In case of row-conditional transformations, the names are row identifiers (either a number between 1 and the total number of rows, or a value from the rowid, which is specified on the INPUT subcommand). In the case of matrix-conditional transformations, the values indicate sources identifiers (either a number between 1 and the total number of sources, or a value from the sourceid, which is specified on the INPUT subcommand). An unconditional transformation only consists of one list and does not allow further specification.

SHEPARD(valuelist). *Shepard plots.* The original proximities versus both transformed proximities and distances. The distances are indicated by points, and the transformed proximities are indicated by a line.

On the value list, the names (identifiers) for which the plot is to be produced must be specified. Because the CONDITION subcommand allows for the specification of multiple transformation lists, the value lists depend on the conditionality. In case of row-conditional transformations, the names are row identifiers (either a number between 1 and the total number of rows, or a value from the rowid, which is specified on the INPUT subcommand). In the case of matrix-conditional transformations, the values indicate sources identifiers (either a number between 1 and the total number of sources, or a value from the sourceid, which is specified on the INPUT subcommand). An unconditional transformation only consists of one list and does not allow further specification.

FIT(valuelist). *Scatterplots of Fit.* The transformed proximities versus the distances are plotted in a scatterplot. On the value list, the names (identifiers) of the sources for which the plot is to be produced must be specified. The sources are specified by a number between 1 and the total number of sources or are specified by a value from the sourceid, which is specified on the INPUT subcommand.

RESIDUALS(valuelist). *Residuals plots.* The transformed proximities versus the residuals (transformed proximities minus distances) are plotted in a scatterplot. On the value list, the names (identifiers) of the sources for which the plot is to be produced must be specified. The sources are specified by a number between 1 and the total number of sources or are specified by a value from the sourceid, which is specified on the INPUT subcommand.

Example

```
PREFSCAL var01 TO var15
  /INPUT = SOURCE(6)
  /MODEL = WEIGHTED
  /CRITERIA = DIMENSIONS(3)
  /PLOT = COMMON
          INDIVIDUAL(2)
          TRANSFORMATIONS(1 TO 42)(1 2)
          FIT(2).
```

- Here, the syntax specifies a weighted Euclidean model with six sources in three dimensions.
- COMMON produces a scatterplot matrix defined by dimensions 1, 2, and 3.
- A scatterplot matrix with threedimensions is produced only for the source 2.
- Two transformation plots are produced, one plot with all 42 rows and one plot with only row 1 and 2. Rows are specified with the TRANSFORMATIONS keyword because the default value on CONDITION is ROW.
- A scatterplot of fit is produced for the source 2.

OPTIONS Subcommand

The OPTIONS subcommand specifies additional markings for the row objects in plots. For this purpose, the values of variables are used to specify markers and colors for the row objects.

MARKERS(variable). *Row object markers.* The values of the variable are used to cycle through all possible markers.

COLORS(variable). *Row object colors.* The values of the variable are used to cycle through all colors.

Example

```
DATA LIST
  /var01 TO var15 rowid gender age.
PREFSCAL var01 TO var15
  /INPUT   = ROW(rowid)
  /OPTIONS = MARKERS(gender) COLORS(age).
```

- In the joint plot of the common space configuration, the row objects are labeled with the values or value labels of the variable *rowid*. Additionally, the points are marked according to the values on the variable *gender* and are colored depending on the values of the variable *age*.

OUTFILE Subcommand

OUTFILE saves coordinates of the common space, individual space weights, distances, and transformed proximities to an external IBM SPSS Statistics data file or previously declared dataset (DATASET DECLARE command). The data file/dataset name must be different for each keyword.

COMMON('savfile'|'dataset'). *Common space coordinates.* The coordinates of the common space. The columns (variables) represent the dimensions *DIM_1*, *DIM_2*, ..., *DIM_n* of the common space. The number of cases in the external file equals the total number of objects (row plus column objects).

WEIGHTS('savfile'|'dataset'). *Individual space weights.* The individual space weights. The columns represent the dimensions *DIM_1*, *DIM_2*, ..., *DIM_n* of the space weights. The number of cases depends on the individual differences model specified on the MODEL subcommand. The weighted Euclidean model uses diagonal weight matrices. Only the diagonals are written to file, and the number of cases is equal to the number of sources. The generalized Euclidean model has full-rank nonsingular weight matrices, one matrix for each source. The matrices are stacked beneath each other in the external IBM SPSS Statistics data file. The number of cases equals the number of sources times the number of dimensions.

DISTANCES('savfile'|'dataset'). *Distances.* The matrices containing the distances between the objects for each source are stacked beneath each other in the external IBM SPSS Statistics data file. The number of variables in the data file is equal to the total number of objects (*ROW_1*, *ROW_2*, ..., *ROW_n*, *COL_1*, *COL_2*, ..., *COL_m*). The number of cases in the data file is equal to the total number of objects times the number of sources.

TRANSFORMATION('file'|'dataset'). *Transformed proximities.* The matrices containing the transformed proximities for each source are stacked beneath each other in the external IBM SPSS Statistics data file. The number of variables in the external file is equal to the total number of objects (*ROW_1*, *ROW_2*, ..., *ROW_n*, *COL_1*, *COL_2*, ..., *COL_m*). The number of cases in the external file is equal to the total number of objects times the number of sources.

Example

```
PREFSCAL var01 TO var15
  /OUTFILE = COMMON('/data/start.sav').
```

- Here, the coordinates of the common space are written to the external IBM SPSS Statistics data file *start.sav*.
- Filenames should be enclosed in quotes and are stored in the working directory unless a path is included as part of the file specification. datasets are available during the current session but are not available in subsequent sessions unless you explicitly save them as data files.

PRESERVE

```
PRESERVE
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Overview

PRESERVE stores current SET and CD specifications that can later be restored by the RESTORE command. PRESERVE and RESTORE are especially useful with the macro facility. PRESERVE-RESTORE sequences can be nested up to five levels.

Basic Specification

The only specification is the command keyword. PRESERVE has no additional specifications.

Limitations

PRESERVE does not store SET THREADS and SET MCACHE settings.

Example

```
PRESERVE.
SET CD '/mydata'.
GET FILE 'mydatafile.sav'.
SET TNUMBERS=BOTH.
FREQUENCIES VARIABLES=DIVISION
RESTORE.
FREQUENCIES VARIABLES=DIVISION.
```

- PRESERVE stores all current SET specifications and the current working directory setting.
- SET CD changes the working directory.
- SET TNUMBERS=BOTH changes the TNUMBERS setting to display both values and value labels in pivot tables.
- The pivot table results from the first FREQUENCIES command will display both category values and labels.
- RESTORE reestablishes the SET specifications and the working directory that were in effect prior to when PRESERVE was specified.
- The pivot table results from the second FREQUENCIES command will use the TNUMBERS setting in effect prior to PRESERVE (the default is to display only labels).

PRINCALS

PRINCALS (*prin*cipal *c*omponents analysis by means of *a*lternating *l*east *s*quares) analyzes a set of variables for major dimensions of variation. The variables can be of mixed optimal scaling levels, and the relationships among observed variables are not assumed to be linear.

```
PRINCALS VARIABLES=varlist(max)

[/ANALYSIS=varlist[({ORDI**})]]
                    {SNOM  }
                    {MNOM  }
                    {NUME  }

[/NOBSERVATIONS=value]

[/DIMENSION={2**  }]
            {value}

[/MAXITER={100**}]
          {value}

[/CONVERGENCE={.00001**}]
              {value   }

[/PRINT=[DEFAULT] [FREQ**] [EIGEN**] [LOADINGS**] [QUANT]
        [HISTORY] [CORRELATION] [OBJECT] [ALL] [NONE]]

[/PLOT=[NDIM=({1    ,2    }**)]
              {value,value}
              {ALL  ,MAX  }
       [DEFAULT[(n)]] [OBJECT**[(varlist)][(n)]]
       [QUANT**[(varlist)][(n)]] [LOADINGS[(n)]]
       [ALL[(n)]] [NONE]]

[/SAVE=[rootname] [(value}]

[/MATRIX=OUT({*   })]
            {'file'|'dataset'}
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Overview

PRINCALS (*prin*cipal *c*omponents analysis by means of *a*lternating *l*east *s*quares) analyzes a set of variables for major dimensions of variation. The variables can be of mixed optimal scaling levels, and the relationships among observed variables are not assumed to be linear.

Options

Optimal Scaling Level. You can specify the optimal scaling level for each variable to be used in the analysis.

Number of Cases. You can restrict the analysis to the first *n* observations.

Number of Dimensions. You can specify how many dimensions PRINCALS should compute.

Iterations and Convergence. You can specify the maximum number of iterations and the value of a convergence criterion.

Display Output. The output can include all available statistics, only the default statistics, or only the specific statistics you request. You can also control whether some of these statistics are plotted.

Saving Scores. You can save object scores in the active dataset.

Writing Matrices. You can write a matrix data file containing category quantifications and loadings for use in further analyses.

Basic Specification

- The basic specification is the PRINCALS command and the VARIABLES subcommand. PRINCALS performs the analysis assuming an ordinal level of optimal scaling for all variables and uses all cases to

compute a two-dimensional solution. By default, marginal frequencies, eigenvalues, and summary measures of fit and loss are displayed, and quantifications and object scores are plotted.

Subcommand Order

- The VARIABLES subcommand must precede all others.
- Other subcommands can appear in any order.

Operations

- If the ANALYSIS subcommand is specified more than once, PRINCALS is not executed. For all other subcommands, only the last occurrence of each subcommand is executed.
- PRINCALS treats every value in the range of 1 to the maximum value specified on VARIABLES as a valid category. Use the AUTORECODE or RECODE command if you want to recode a categorical variable with nonsequential values or with a large number of categories to avoid unnecessary output. For variables treated as numeric, recoding is *not* recommended because the intervals between consecutive categories will not be maintained.

Limitations

- String variables are not allowed; use AUTORECODE to recode nominal string variables into numeric ones before using PRINCALS.
- The data must be positive integers. Zeros and negative values are treated as system-missing and are excluded from the analysis. Fractional values are truncated after the decimal and are included in the analysis. If one of the levels of a categorical variable has been coded 0 or a negative value and you want to treat it as a valid category, use the AUTORECODE or RECODE command to recode the values of that variable (see AUTORECODE and RECODE for more information).
- PRINCALS ignores user-missing value specifications. Positive user-missing values less than the maximum value on the VARIABLES subcommand are treated as valid category values and are included in the analysis. If you do not want the category included, you can use COMPUTE or RECODE to change the value to something outside of the valid range. Values outside of the range (less than 1 or greater than the maximum value) are treated as system-missing.

Example

```
PRINCALS VARIABLES=ACOLA BCOLA(2) PRICEA PRICEB(5)
/ANALYSIS=ACOLA BCOLA(SNOM) PRICEA PRICEB(NUME)
/PRINT=QUANT OBJECT.
```

- VARIABLES defines the variables and their maximum number of levels.
- The ANALYSIS subcommand specifies that variables *ACOLA* and *BCOLA* are single nominal (SNOM) and that variables *PRICEA* and *PRICEB* are numeric (NUME).
- The PRINT subcommand lists the category quantifications and object scores.
- By default, plots of the category quantifications and the object scores are produced.

VARIABLES Subcommand

VARIABLES specifies all of the variables that will be used in the current PRINCALS procedure.

- The VARIABLES subcommand is required and precedes all other subcommands. The actual word VARIABLES can be omitted.
- Each variable or variable list is followed by the maximum number of categories (levels) in parentheses.
- The number specified in parentheses indicates the number of categories *and* the maximum category value. For example, *VAR1(3)* indicates that *VAR1* has three categories coded 1, 2, and 3. However, if a variable is not coded with consecutive integers, the number of categories used in the analysis will differ from the number of observed categories. For example, if a three category variable is coded {2, 4, 6}, the

maximum category value is 6. The analysis treats the variable as having six categories, three of which are not observed and receive quantifications of 0.

- To avoid unnecessary output, use the AUTORECODE or RECODE command before PRINCALS to recode a categorical variable that was coded with nonsequential values. As noted in "Limitations," recoding is *not* recommended with variables treated as numeric (see AUTORECODE and RECODE for more information).

Example

```
DATA LIST FREE/V1 V2 V3.
BEGIN DATA
3 1 1
6 1 1
3 1 3
3 2 2
3 2 2
6 2 2
6 1 3
6 2 2
3 2 2
6 2 1
END DATA.
AUTORECODE V1 /INTO NEWVAR1.
PRINCALS VARIABLES=NEWVAR1 V2(2) V3(3).
```

- DATA LIST defines three variables, *V1*, *V2*, and *V3*.
- *V1* has two levels, coded 3 and 6, *V2* has two levels, coded 1 and 2, and *V3* has three levels, coded 1, 2, and 3.
- The AUTORECODE command creates *NEWVAR1* containing recoded values of *V1*. Values of 3 are recoded to 1, and values of 6 are recoded to 2.
- A maximum value of 2 can then be specified on the VARIABLES subcommand as the maximum category value for both *NEWVAR1* and *V2*. A maximum value of 3 is specified for *V3*.

ANALYSIS Subcommand

ANALYSIS specifies the variables to be used in the computations and the optimal scaling level used by PRINCALS to quantify each variable or variable list.

- If ANALYSIS is not specified, an ordinal level of optimal scaling is assumed for all variables.
- The specification on ANALYSIS is a variable list and an optional keyword in parentheses to indicate the optimal scaling level.
- The variables on the variable list must also be specified on the VARIABLES subcommand.
- Variables listed on the VARIABLES subcommand but not on the ANALYSIS subcommand can still be used to label object scores on the PLOT subcommand.

The following keywords can be specified to indicate the optimal scaling level:

MNOM. *Multiple nominal*. The quantifications can be different for each dimension. When all variables are multiple nominal, PRINCALS gives the same results as HOMALS.

SNOM. *Single nominal*. PRINCALS gives only one quantification for each category. Objects in the same category (cases with the same value on a variable) obtain the same quantification. When DIMENSION=1 and all variables are SNOM, this solution is the same as that of the first HOMALS dimension.

ORDI. *Ordinal*. This is the default for variables listed without optimal scaling levels and for all variables if the ANALYSIS subcommand is not used. The order of the categories of the observed variable is preserved in the quantified variable.

NUME. *Numerical*. This is the interval or ratio level of optimal scaling. PRINCALS assumes that the observed variable already has numerical values for its categories. When all variables are at the numerical level, the PRINCALS analysis is analogous to classical principal components analysis.

These keywords can apply to a variable list as well as to a single variable. Thus, the default ORDI is not applied to a variable without a keyword if a subsequent variable on the list has a keyword.

NOBSERVATIONS Subcommand

NOBSERVATIONS specifies how many cases are used in the analysis.

- If NOBSERVATIONS is not specified, all available observations in the active dataset are used.
- NOBSERVATIONS is followed by an integer indicating that the first *n* cases are to be used.

DIMENSION Subcommand

DIMENSION specifies the number of dimensions that you want PRINCALS to compute.

- If you do not specify the DIMENSION subcommand, PRINCALS computes two dimensions.
- DIMENSION is followed by an integer indicating the number of dimensions.
- If all of the variables are SNOM (single nominal), ORDI (ordinal), or NUME (numerical), the maximum number of dimensions you can specify is the smaller of the number of observations minus 1 *or* the total number of variables.
- If some or all of the variables are MNOM (multiple nominal), the maximum number of dimensions is the smaller of the number of observations minus 1 *or* the total number of valid MNOM variable levels (categories) plus the number of SNOM, ORDI, and NUME variables, minus the number of MNOM variables without missing values.
- PRINCALS adjusts the number of dimensions to the maximum if the specified value is too large.
- The minimum number of dimensions is 1.

MAXITER Subcommand

MAXITER specifies the maximum number of iterations PRINCALS can go through in its computations.

- If MAXITER is not specified, PRINCALS will iterate up to 100 times.
- MAXITER is followed by an integer indicating the maximum number of iterations allowed.

CONVERGENCE Subcommand

CONVERGENCE specifies a convergence criterion value. PRINCALS stops iterating if the difference in total fit between the last two iterations is less than the CONVERGENCE value.

- If CONVERGENCE is not specified, the default value is 0.00001.
- The specification on CONVERGENCE is a convergence criterion value.

PRINT Subcommand

PRINT controls which statistics are included in your output. The default output includes frequencies, eigenvalues, loadings, and summary measures of fit and loss.

PRINT is followed by one or more of the following keywords:

FREQ. *Marginal frequencies for the variables in the analysis*.

HISTORY. *History of the iterations*.

EIGEN. *Eigenvalues*.

CORRELATION. *Correlation matrix for the transformed variables in the analysis*. No correlation matrix is produced if there are any missing data.

OBJECT. *Object scores*.

QUANT. *Category quantifications and category coordinates for SNOM, ORDI, and NUME variables and category quantifications in each dimension for MNOM variables*.

LOADINGS. *Component loadings for SNOM, ORDI, and NUME variables*.

DEFAULT. *FREQ, EIGEN, LOADINGS, and QUANT*.

ALL. *All of the available statistics*.

NONE. *Summary measures of fit*.

PLOT Subcommand

PLOT can be used to produce plots of category quantifications, object scores, and component loadings.

- If PLOT is not specified, plots of the object scores and the quantifications are produced.
- No plots are produced for a one-dimensional solution.

PLOT is followed by one or more of the following keywords:

LOADINGS. *Plots of the component loadings of SNOM, ORDI, and NUME variables*.

OBJECT. *Plots of the object scores*.

QUANT. *Plots of the category quantifications for MNOM variables and plots of the single-category coordinates for SNOM, ORDI, and NUME variables*.

DEFAULT. *QUANT and OBJECT*.

ALL. *All available plots*.

NONE. *No plots*.

- The keywords OBJECT and QUANT can each be followed by a variable list in parentheses to indicate that plots should be labeled with these variables. For QUANT, the variables must be specified on both the VARIABLES and ANALYSIS subcommands. For OBJECT, the variables must be specified on VARIABLES but need not appear on the ANALYSIS subcommand. This means that variables not included in the computations can still be used to label OBJECT plots. If the variable list is omitted, only the default plots are produced.
- Object scores plots labeled with variables that appear on the ANALYSIS subcommand use category labels corresponding to all categories within the defined range. Objects in a category that is outside the defined range are labeled with the label corresponding to the next category greater than the defined maximum category.
- Object scores plots labeled with variables not included on the ANALYSIS subcommand use all category labels, regardless of whether or not the category value is inside the defined range.
- All of the keywords except NONE can be followed by an integer in parentheses to indicate how many characters of the variable or value label are to be used on the plot. (If you specify a variable list after OBJECT or QUANT, you can specify the value in parentheses after the list.) The value can range from 1 to 20. If the value is omitted, 12 characters are used. Spaces between words count as characters.
- The LOADINGS plots and one of the QUANT plots use variable labels; all other plots that use labels use value labels.
- If a variable label is missing, the variable name is used for that variable. If a value label is missing, the actual value is used.
- You should make sure that your variable and value labels are unique by at least one letter in order to distinguish them on the plots.
- When points overlap, the points involved are described in a summary following the plot.

Example

```
PRINCALS VARIABLES COLA1 (4) COLA2 (4) COLA3 (4) COLA4 (2)
/ANALYSIS COLA1 COLA2 (SNOM) COLA3 (ORDI) COLA4 (ORDI)
/PLOT OBJECT(COLA4).
```

- Four variables are included in the analysis.
- OBJECT requests a plot of the object scores labeled with the values of *COLA4*. Any object whose *COLA4* value is not 1 or 2 is labeled 3 (or the value label for category 3, if defined).

Example

```
PRINCALS VARIABLES COLA1 (4) COLA2 (4) COLA3 (4) COLA4 (2)
/ANALYSIS COLA1 COLA2 (SNOM) COLA3 (ORDI)
/PLOT OBJECT(COLA4).
```

- Three variables are included in the analysis.
- OBJECT requests a plot of the object scores labeled with the values of *COLA4,* a variable not included in the analysis. Objects are labeled using all values of *COLA4.*

In addition to the plot keywords, the following can be specified:

NDIM. *Dimension pairs to be plotted.* NDIM is followed by a pair of values in parentheses. If NDIM is not specified, plots are produced for dimension 1 versus dimension 2.

- The first value indicates the dimension that is plotted against all higher dimensions. This value can be any integer from 1 to the number of dimensions minus 1.
- The second value indicates the highest dimension to be used in plotting the dimension pairs. This value can be any integer from 2 to the number of dimensions.
- The keyword ALL can be used instead of the first value to indicate that all dimensions are paired with higher dimensions.
- The keyword MAX can be used instead of the second value to indicate that plots should be produced up to, and including, the highest dimension fit by the procedure.

Example

```
PRINCALS COLA1 COLA2 COLA3 COLA4 (4)
/PLOT NDIM(1,3) QUANT(5).
```

- The NDIM(1,3) specification indicates that plots should be produced for two dimension pairs—dimension 1 versus dimension 2 and dimension 1 versus dimension 3.
- QUANT requests plots of the category quantifications. The (5) specification indicates that the first five characters of the value labels are to be used on the plots.

Example

```
PRINCALS COLA1 COLA2 COLA3 COLA4 (4)
/PLOT NDIM(ALL,3) QUANT(5).
```

- This plot is the same as above except for the ALL specification following NDIM. This indicates that all possible pairs up to the second value should be plotted, so QUANT plots will be produced for dimension 1 versus dimension 2, dimension 2 versus dimension 3, and dimension 1 versus dimension 3.

SAVE Subcommand

SAVE lets you add variables containing the object scores computed by PRINCALS to the active dataset.

- If SAVE is not specified, object scores are not added to the active dataset.
- A variable rootname can be specified on the SAVE subcommand to which PRINCALS adds the number of the dimension. Only one rootname can be specified, and it can contain up to six characters.
- If a rootname is not specified, unique variable names are automatically generated. The variable names are *PRI n_m*, where *n* is a dimension number and *m* is a set number. If three dimensions are saved, the first set of names is *PRI1_1*, *PRI2_1*, and *PRI3_1*. If another PRINCALS is then run, the variable names for the second set are *PRI1_2*, *PRI2_2*, *PRI3_2*, and so on.
- Following the name, the number of dimensions for which you want to save object scores can be listed in parentheses. The number cannot exceed the value of the DIMENSION subcommand.

- If the number of dimensions is not specified, the SAVE subcommand saves object scores for all dimensions.
- If you replace the active dataset by specifying an asterisk (*) on a MATRIX subcommand, the SAVE subcommand is not executed.
- The prefix should be unique for each PRINCALS command in the same session. If it is not, PRINCALS replaces the prefix with *DIM*, *OBJ*, or *OBSAVE*. If all of these already exist, SAVE is not executed.

Example

```
PRINCALS CAR1 CAR2 CAR3(5) PRICE (10)
/ANALYSIS=CAR1 TO CAR3(SNOM) PRICE(NUM)
/DIMENSIONS=3
/SAVE=DIM(2).
```

- Three nominal variables, *CAR1*, *CAR2*, and *CAR3*, each with five categories, and one numerical (interval level) variable, with ten categories, are analyzed in this PRINCALS example.
- The DIMENSIONS subcommand requests results for three dimensions.
- SAVE adds the object scores from the first two dimensions to the active dataset. The names of these new variables will be *DIM00001* and *DIM00002*, respectively.

MATRIX Subcommand

The MATRIX subcommand is used to write category quantifications, single-category coordinates, and component loadings to a matrix data file.

- The specification on MATRIX is the keyword OUT and a quoted file specification of previously declared dataset name (DATASET DECLARE command), enclosed in parentheses.
- You can specify an asterisk (*) instead of a file to replace the active dataset .
- The category quantifications, coordinates, and component loadings are written to the same file.
- The matrix data file has one case for each value of each original variable.

The variables of the matrix data file and their values are:

ROWTYPE_. *String variable rowtype_ containing value QUANT for the category quantifications, SCOOR_ for single-category coordinates, MCOOR_ for multiple-category coordinates, and LOADING_ for the component scores*.

LEVEL. *String variable containing the values (or value labels if present) of each original variable for category quantifications.* For cases with *ROWTYPE_=LOADING_*, the value of *LEVEL* is blank.

VARNAME_. *String variable containing the original variable names*.

VARTYPE_. *String variable containing values MULTIPLE, SINGLE N, ORDINAL, or NUMERICAL, depending on the optimal scaling level specified for the variable*.

DIM1...DIMn. *Numeric variables containing category quantifications, the single-category coordinates, and component loadings for each dimension*. Each variable is labeled *DIM n*, where *n* represents the dimension number. The single-category coordinates and component loadings are written only for SNOM, ORDI, and NUME variables.

PRINT

```
PRINT [OUTFILE=file] [ENCODING='encoding specification']
  [RECORDS={1}] [{NOTABLE}]
           {n}   {TABLE  }

 /{1    } varlist [{col location [(format)]}] [varlist...]
  {rec #}          {(format list)          }
                   {*                      }

 [/{2    }...]
   {rec #}
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic "Command Order" on page 36 for more information.

Release History

Release 16.0

- ENCODING subcommand added for Unicode support.

Example

```
PRINT / MOHIRED YRHIRED DEPT SALARY NAME.
```

Overview

PRINT displays the values of variables for each case in the data. PRINT is simple enough for a quick check on data definitions and transformations and flexible enough for formatting simple reports.

Options

Formats. You can specify formats for the variables (see) .

Strings. You can specify string values within the variable specifications. The strings can be used to label values or to create extra space between values. Strings can also be used as column headings. (See .)

Output File. You can use the OUTFILE subcommand to direct the output to a specified file.

Summary Table. You can use the TABLE subcommand to display a table that summarizes variable formats.

Basic Specification

The basic specification is a slash followed by a variable list. The output displays values for all variables that are named on the list.

Subcommand Order

Subcommands can be specified in any order. However, all subcommands must be specified before the slash that precedes the start of the variable specifications.

Syntax Rules

- A slash must precede the variable specifications. The first slash begins the definition of the first (and possibly only) line per case of the PRINT display.
- Specified variables must already exist, but they can be numeric, string, scratch, temporary, or system variables. Subscripted variable names, such as *X(1)* for the first element in vector *X,* cannot be used.
- Keyword ALL can be used to display the values of all user-defined variables in the active dataset.

Operations

- PRINT is executed once for each case that is constructed from the data file.
- PRINT is a transformation and is not executed unless it is followed by a procedure or the EXECUTE command.

- Because PRINT is a transformation command, the output might be mixed with casewise procedure output. Procedures that produce individual case listings (such as LIST) should not be used immediately after PRINT. An intervening EXECUTE or procedure command should be specified.
- Values are displayed with a blank space between them. However, if a format is specified for a variable, the blank space for that variable's values is suppressed.
- Values are displayed in the output as the data are read. The PRINT output appears before the output from the first procedure.
- If more variables are specified than can be displayed in 132 columns or within the width that is specified on SET WIDTH, the program displays an error message. You must reduce the number of variables or split the output into several records.
- User-missing values are displayed exactly like valid values. System-missing values are represented by a period.

Examples

Displaying Values for a Selected List of Variables

```
PRINT  / MOHIRED YRHIRED DEPT SALARY NAME.
FREQUENCIES VARIABLES=DEPT.
```

- PRINT displays values for each variable on the variable list. The FREQUENCIES procedure reads the data and causes PRINT to be executed.
- All variables are displayed by using their dictionary formats. One blank space separates the values of each variable.

Displaying Values for All User-Defined Variables

```
PRINT /ALL.
EXECUTE.
```

- PRINT displays values for all user-defined variables in the active dataset. The EXECUTE command executes PRINT.

Formats

By default, PRINT uses the dictionary print formats. You can specify formats for some or all variables that are specified on PRINT. For a string variable, the specified format must have a width at least as large as the width of the dictionary format. String values are truncated if the specified width is smaller than the width of the dictionary format.

- Format specifications can be either column-style or FORTRAN-like (see DATA LIST). The column location that is specified with column-style formats or that is implied with FORTRAN-like formats refers to the column in which the variable will be displayed.
- A format specification following a list of variables applies to all variables in the list. Use an asterisk to prevent the specified format from applying to variables that precede the asterisk. The specification of column locations implies a default print format, and that format applies to all previous variables if no asterisk is used.
- Printable numeric formats are F, COMMA, DOLLAR, CC, DOT, N, E, PCT, PIBHEX, RBHEX, Z, and the date and time formats. Printable string formats are A and AHEX. Note that hex and binary formats use different widths. For example, the AHEX format must have a width that is twice the width of the corresponding A format. For more information about specifying formats and more information about the available formats, see DATA LIST and .
- Format specifications are in effect only for the PRINT command. The specifications do not change the dictionary print formats.

- When a format is specified for a variable, the automatic blank following the variable in the output is suppressed. To preserve the blank between variables, use a string (see) , specify blank columns in the format, or use an X or T format element (see DATA LIST for information about X and T).

Example

```
PRINT / TENURE (F2.0) ' ' MOHIRED YRHIRED DEPT *
        SALARY85 TO SALARY88 (4(DOLLAR8,1X)) NAME.
EXECUTE.
```

- Format F2.0 is specified for *TENURE*. A blank string is specified after *TENURE* because the automatic blank following the variable is suppressed by the format specification.
- *MOHIRED, YRHIRED,* and *DEPT* are displayed with default formats because the asterisk prevents them from receiving the DOLLAR8 format that is specified for *SALARY85* to *SALARY88*. The automatic blank is preserved for *MOHIRED, YRHIRED,* and *DEPT*, but the blank is suppressed for *SALARY85* to *SALARY88* by the format specification. The 1X format element is therefore specified with DOLLAR8 to add one blank after each value of *SALARY85* to *SALARY88*.
- NAME uses the default dictionary format.

Strings

You can specify string values within the variable list. Strings must be enclosed in quotes.

- If a format is specified for a variable list, the application of the format is interrupted by a specified string. Thus, the string has the same effect within a variable list as an asterisk.
- Strings can be used to create column headings for the displayed variables. The PRINT command that specifies the column headings must be used within a DO IF-END IF structure. If you want the column headings to begin a new page in the output, use a PRINT EJECT command (rather than PRINT) to specify the headings (see PRINT EJECT).

Including Strings in the Output

```
PRINT / NAME 'HIRED=' MOHIRED(F2) '/' YRHIRED
            '  SALARY=' SALARY (DOLLAR8).
EXECUTE.
```

- Three strings are specified. The strings HIRED= and SALARY= label the values being displayed. The slash that is specified between *MOHIRED* and *YRHIRED* creates a composite hiring date. The F2 format is supplied for variable *MOHIRED* in order to suppress the blank that would follow it if the dictionary format were used.
- *NAME* and *YRHIRED* are displayed with default formats. The 'HIRED=' specification prevents the F2 format from applying to *NAME*, and the 'SALARY=' specification prevents the DOLLAR8 format from applying to *YRHIRED*.

Setting Up Column Headers

```
DO IF $CASENUM EQ 1.
PRINT /'   NAME ' 1 'DEPT' 25 'HIRED' 30 '  SALARY' 35.
END IF.
PRINT / NAME DEPT *
        MOHIRED 30-31 '/' YRHIRED *
        SALARY 35-42(DOLLAR).
EXECUTE.
```

- The first PRINT command specifies strings only. The integer after each string specifies the beginning column number of the string. The strings will be used as column headings for the variables. DO IF $CASENUM EQ 1 causes the first PRINT command to be executed only once, as the first case is processed. END IF closes the structure.
- The second PRINT command specifies the variables to be displayed. This command is executed once for each case in the data. Column locations are specified to align the values with the column headings. In this example, the T format element could also have been used to align the variables and the column headings. For example, MOHIRED (T30,F2) begins the display of values for variable *MOHIRED* in column 30.

- The asterisk after *DEPT* prevents the format that is specified for *MOHIRED* from applying to *NAME* and *DEPT*. The asterisk after *YRHIRED* prevents the format that is specified for *SALARY* from applying to *YRHIRED*.

RECORDS Subcommand

RECORDS indicates the total number of lines that are displayed per case. The number that is specified on RECORDS is informational only. The actual specification that causes variables to be displayed on a new line is a slash within the variable specifications. Each new line is requested by another slash.

- RECORDS must be specified before the slash that precedes the start of the variable specifications.
- The only specification on RECORDS is an integer to indicate the number of records for the output. If the number does not agree with the actual number of records that are indicated by slashes, the program issues a warning and ignores the specification on RECORDS.
- Specifications for each line of output must begin with a slash. An integer can follow the slash, indicating the line on which values are to be displayed. The integer is informational only and cannot be used to rearrange the order of records in the output. If the integer does not agree with the actual record number that is indicated by the number of slashes in the variable specifications, the integer is ignored.
- A slash that is not followed by a variable list generates a blank line in the output.

Examples

```
PRINT RECORDS=3 /EMPLOYID NAME DEPT
                /EMPLOYID TENURE SALARY
                /.
EXECUTE.
```

- PRINT displays the values of an individual's name and department on one line, displays tenure and salary on the next line, and displays the employee identification number on both lines, followed by a blank third line. Two lines are displayed for each case, and cases in the output are separated by a blank line.

```
PRINT RECORDS=3 /1 EMPLOYID NAME DEPT
                /2 EMPLOYID TENURE SALARY
                /3.
```

- This PRINT command is equivalent to the command in the preceding example.

```
PRINT / EMPLOYID NAME DEPT / EMPLOYID TENURE SALARY /.
```

- This PRINT command is equivalent to the commands in the two preceding examples.

OUTFILE Subcommand

OUTFILE specifies a file for the output from the PRINT command. By default, PRINT output is included with the rest of the output from the session.

- OUTFILE must be specified before the slash that precedes the start of the variable specifications.
- The output from PRINT cannot exceed 132 characters, even if the external file is defined with a longer record length.

Example

```
PRINT OUTFILE=PRINTOUT
   /1 EMPLOYID DEPT SALARY /2 NAME.
EXECUTE.
```

- OUTFILE specifies *PRINTOUT* as the file that receives the PRINT output.

ENCODING Subcommand

ENCODING specifies the encoding format of the file. The keyword is followed by an equals sign and a quoted encoding specification.

- In Unicode mode, the default is UTF8. For more information, see SET command, UNICODE subcommand.
- In code page mode, the default is the current locale setting. For more information, see SET command, LOCALE subcommand.
- The quoted encoding value can be: Locale (the current locale setting), UTF8, UTF16, UTF16BE (big endian), UTF16LE (little endian), a numeric Windows code page value (for example, '1252'), or an IANA code page value (for example, 'iso8859-1' or cp1252).
- If there is no OUTFILE subcommand, the ENCODING subcommand is ignored.

TABLE Subcommand

TABLE requests a table that shows how the variable information is formatted. NOTABLE, which suppresses the format table, is the default.

- TABLE must be specified before the slash that precedes the start of the variable specifications.

Example

```
PRINT TABLE /1 EMPLOYID DEPT SALARY /2  NAME.
EXECUTE.
```

- TABLE requests a summary table that describes the PRINT specifications. The table is included with the PRINT output.

PRINT EJECT

```
PRINT EJECT [OUTFILE='file'] [ENCODING='encoding specification']
  [RECORDS={1}] [{NOTABLE}]
           {n}   {TABLE  }

  /{1    } varlist [{col location [(format)]}] [varlist...]
   {rec #}          {(format list)          }
                    {*                      }

 [/{2    }...]
   {rec #}
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic "Command Order" on page 36 for more information.

Release History

Release 16.0

- ENCODING subcommand added for Unicode support.

Example

```
DO IF $CASENUM EQ 1.
PRINT EJECT /'   NAME ' 1 'DEPT' 25 'HIRED' 30 '  SALARY' 35.
END IF.
PRINT / NAME DEPT *
        MOHIRED(T30,F2) '/' YRHIRED *
        SALARY (T35,DOLLAR8).
EXECUTE.
```

Overview

PRINT EJECT displays specified information at the top of a new page of the output. Each time that it is executed, PRINT EJECT causes a page ejection. If not used in a DO IF-END IF structure, PRINT EJECT is executed for each case in the data, and each case is displayed on a separate page.

PRINT EJECT is designed to be used with the PRINT command to insert titles and column headings above the values that are displayed by PRINT. PRINT can also generate titles and headings, but PRINT cannot be used to control page ejections.

PRINT EJECT and PRINT can be used for writing simple reports.

Options

The options that are available for PRINT EJECT are identical to the options that are available for PRINT:

- You can specify formats for the variables.
- You can specify string values within the variable specifications. With PRINT EJECT, the strings are usually used as titles or column headings and often include a specification for column location.
- You can use the RECORDS subcommand to display each case on more than one line.
- You can use the OUTFILE subcommand to direct the output to a specified file.
- You can use the TABLE subcommand to display a table that summarizes variable formats.
- You can use ENCODING to specify the file encoding. If there is no OUTFILE subcommand, the ENCODING subcommand is ignored.

For additional information, refer to PRINT.

Basic Specification

The basic specification is a slash followed by a variable list and/or a list of string values that will be used as column headings or titles. The values for each variable or string are displayed on the top line of a new page in the output. PRINT EJECT is usually used within a DO IF-END IF structure to control the page ejections.

Operations

- PRINT EJECT is a transformation and is not executed unless it is followed by a procedure or the EXECUTE command.
- If not used within a DO IF-END IF structure, PRINT EJECT is executed for each case in the data and displays the values for each case on a separate page.
- Values are displayed with a blank space between them. However, if a format is specified for a variable, the blank space for that variable's values is suppressed.
- Values are displayed in the output as the data are read. The PRINT output appears before the output from the first procedure.
- If more variables are specified than can be displayed in 132 columns or within the width that is specified on SET WIDTH, the program displays an error message. You must reduce the number of variables or split the output into several records.
- User-missing values are displayed exactly like valid values. System-missing values are represented by a period.

Examples

Displaying Column Headings on the First Output Page Only

```
DO IF $CASENUM EQ 1.
PRINT EJECT /'   NAME ' 1 'DEPT' 25 'HIRED' 30 '  SALARY' 35.
END IF.
PRINT / NAME DEPT *
        MOHIRED(T30,F2) '/' YRHIRED *
        SALARY (T35,DOLLAR8).
EXECUTE.
```

- PRINT EJECT specifies strings to be used as column headings and causes a page ejection. DO IF-END IF causes the PRINT EJECT command to be executed only once, when the system variable *$CASENUM* equals 1 (the value that is assigned to the first case in the file). Thus, column headings are displayed on the first page of the output only. The next example shows how to display column headings at the top of every page of the output.
- If a PRINT command were used in place of PRINT EJECT, the column headings would begin immediately after the command printback.

Displaying Column Headings on Each Output Page

```
DO IF MOD($CASENUM,50) = 1.
PRINT  EJECT  OUTFILE='/mydir/out.txt' /'   NAME ' 1 'DEPT' 25 'HIRED' 30 '  SALARY' 35.
END IF.
PRINT OUTFILE='/mydir/out.txt' / NAME DEPT *
        MOHIRED 30-31 '/' YRHIRED *
        SALARY 35-42(DOLLAR).
EXECUTE.
```

- In this example, DO IF specifies that PRINT EJECT is executed if MOD (the remainder) of *$CASENUM* divided by 50 equals 1 (see for information on the MOD function) . Thus, column headings are displayed on a new page after every 50th case.
- If PRINT were used instead of PRINT EJECT, column headings would be displayed after every 50th case but would not appear at the top of a new page.
- Both PRINT EJECT and PRINT specify the same file for the output. If the OUTFILE subcommands on PRINT EJECT and PRINT do not specify the same file, the column headings and the displayed values end up in different files.

PRINT FORMATS

```
PRINT FORMATS varlist(format) [varlist...]
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
PRINT FORMATS SALARY (DOLLAR8) / HOURLY (DOLLAR7.2)
           / RAISE BONUS (PCT2).
```

Overview

PRINT FORMATS changes variable print formats. Print formats are output formats and control the form in which values are displayed by a procedure or by the PRINT command.

PRINT FORMATS changes only print formats. To change write formats, use the WRITE FORMATS command. To change both the print and write formats with a single specification, use the FORMATS command. For information about assigning input formats during data definition, see DATA LIST. See the topic “Variable Types and Formats” on page 46 for more information.

Basic Specification

The basic specification is a variable list followed by the new format specification in parentheses. All specified variables receive the new format.

Syntax Rules

- You can specify more than one variable or variable list, followed by a format in parentheses. Only one format can be specified after each variable list. For clarity, each set of specifications can be separated by a slash.
- You can use keyword TO to refer to consecutive variables in the active dataset.
- The specified width of a format must include enough positions to accommodate any punctuation characters, such as decimal points, commas, dollar signs, or date and time delimiters. (This situation differs from assigning an *input* format on DATA LIST, where the program automatically expands the input format to accommodate punctuation characters in output.)
- Custom currency formats (CCw, CCw.d) must first be defined on the SET command before they can be used on PRINT FORMATS.
- For string variables, you can only use PRINT FORMATS to switch between A and AHEX formats. PRINT FORMATS cannot be used to change the length of string variables. To change the defined length of a string variable, use the ALTER TYPE command.

Operations

- Unlike most transformations, PRINT FORMATS takes effect as soon as it is encountered in the command sequence. Special attention should be paid to the position of PRINT FORMATS among commands.
- Variables that are not specified on PRINT FORMATS retain their current print formats in the active dataset. To see the current formats, use the DISPLAY command.
- The new print formats are changed only in the active dataset and are in effect for the duration of the session or until changed again with a PRINT FORMATS or FORMATS command. Print formats in the original data file (if this file exists) are not changed, unless the file is resaved with the SAVE or XSAVE command.

- New numeric variables that are created with transformation commands are assigned default print formats of F8.2 (or the format that is specified on the FORMAT subcommand of SET). The FORMATS command can be used to change the new variable's print formats.
- New string variables that are created with transformation commands are assigned the format that is specified on the STRING command that declares the variable. PRINT FORMATS cannot be used to change the format of a new string variable.
- If a numeric data value exceeds its width specification, the program still attempts to display some value. First, the program rounds decimal values, then removes punctuation characters, and then tries scientific notation. Finally, if there is still not enough space, the program produces asterisks indicating that a value is present but cannot be displayed in the assigned width.

Examples

Basic Example

```
PRINT FORMATS SALARY (DOLLAR8) / HOURLY (DOLLAR7.2)
          / RAISE BONUS (PCT2).
```

- The print format for *SALARY* is changed to DOLLAR with eight positions, including the dollar sign and comma when appropriate. The value 11550 is displayed as $11,550. An eight-digit number requires a DOLLAR11 format specification: eight characters for digits, two characters for commas, and one character for the dollar sign.
- The print format for *HOURLY* is changed to DOLLAR with seven positions, including the dollar sign, decimal point, and two decimal places. The number 115 is displayed as $115.00. If DOLLAR6.2 had been specified, the value 115 would be displayed as $115.0. The program would truncate the last 0 because a width of 6 is not enough to display the full value.
- The print format for both *RAISE* and *BONUS* is changed to PCT with two positions: one position for the percentage and one position for the percent sign. The value 9 is displayed as 9%. Because the width allows for only two positions, the value 10 is displayed as 10.

Changing Default Formats

```
COMPUTE V3=V1 + V2.
PRINT FORMATS V3 (F3.1).
```

- COMPUTE creates the new numeric variable *V3*. By default, *V3* is assigned an F8.2 format (or the default format that is specified on SET).
- PRINT FORMATS changes the print format for *V3* to F3.1.

Working With Custom Currency Formats

```
SET CCA='-/-.Dfl ..-'.
PRINT FORMATS COST (CCA14.2).
```

- SET defines a European currency format for the custom currency format type CCA.
- PRINT FORMATS assigns the print format CCA to variable *COST*. With the format defined for CCA on SET, the value 37419 is displayed as Dfl'37.419,00. See the SET command for more information about custom currency formats.

PRINT SPACE

```
PRINT SPACE [OUTFILE='file'] [ENCODING='encoding specification']
  [numeric expression]
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

Release History

Release 16.0

- ENCODING subcommand added for Unicode support.

Example

```
PRINT SPACE.
```

Overview

PRINT SPACE displays blank lines in the output and is generally used with a PRINT or WRITE command. Because PRINT SPACE displays a blank line each time that the command is executed, it is often used in a DO IF-END IF structure.

Basic Specification

The basic specification is the command PRINT SPACE.

Syntax Rules

- To display more than one blank line, specify a numeric expression after PRINT SPACE. The expression can be an integer or a complex expression.
- OUTFILE directs the output to a specified file. OUTFILE should be specified if an OUTFILE subcommand is specified on the PRINT or WRITE command that is used with PRINT SPACE. The OUTFILE subcommand on PRINT SPACE and PRINT or WRITE should specify the same file.
- ENCODING specifies the encoding for the output file. If there is no OUTFILE subcommand, the ENCODING subcommand is ignored.

Operations

- If not used in a DO IF-END IF structure, PRINT SPACE is executed for each case in the data and displays a blank line for every case.

Examples

Inserting a Blank Line after the Output for Each Case

```
PRINT / NAME DEPT82 *
        MOHIRED(T30,F2) '/' YRHIRED *
        SALARY82 (T35,DOLLAR8).
PRINT SPACE.
EXECUTE.
```

- Each time that it is executed, PRINT SPACE displays one blank line. Because PRINT SPACE is not used in a DO IF-END IF structure, PRINT SPACE is executed once for each case. In effect, the output is double-spaced.

Using PRINT SPACE Inside a DO IF-END IF Structure

```
NUMERIC #LINE.
DO IF MOD(#LINE,5) = 0.
PRINT SPACE 2.
```

```
END IF.
COMPUTE #LINE=#LINE + 1.
PRINT / NAME DEPT *
        MOHIRED 30-31 '/' YRHIRED *
        SALARY 35-42(DOLLAR).
EXECUTE.
```

- DO IF specifies that PRINT SPACE will be executed if MOD (the remainder) of *#LINE* divided by 5 equals 1. Because *#LINE* is incremented by 1 for each case, PRINT SPACE is executed once for every five cases. (See for information about the MOD function.)
- PRINT SPACE specifies two blank lines. Cases are displayed in groups of five with two blank lines between each group.

Using an Expression to Specify the Number of Blank Lines

```
* Printing addresses on labels.

COMPUTE #LINES=0.               /*Initiate #LINES to 0
DATA LIST FILE=ADDRESS/RECORD 1-40 (A).  /*Read a record
COMPUTE #LINES=#LINES+1.        /*Bump counter and print
WRITE OUTFILE=LABELS /RECORD.

DO IF RECORD EQ ' '.            /*Blank between addresses
+  PRINT SPACE OUTFILE=LABELS 8 - #LINES.  /*Add extra blank #LINES
+  COMPUTE #LINES=0.
END IF.
EXECUTE.
```

- PRINT SPACE uses a complex expression for specifying the number of blank lines to display. The data contain a variable number of input records for each name and address, which must be printed in a fixed number of lines for mailing labels. The goal is to know when the last line for each address has been printed, how many lines have printed, and therefore how many blank records must be printed in order for the next address to fit on the next label. The example assumes that there is already one blank line between each address on input and that you want to print eight lines per label.
- The DATA LIST command defines the data. Each line of the address is contained in columns 1–40 of the data file and is assigned the variable name *RECORD*. For the blank line between each address, *RECORD* is blank.
- Variable *#LINES* is initialized to 0 as a scratch variable and is incremented for each record that is written. When the program encounters a blank line (RECORD EQ ' '), PRINT SPACE prints a number of blank lines that is equal to 8 minus the number already printed, and *#LINES* is then reset to 0.
- OUTFILE on PRINT SPACE specifies the same file that is specified by OUTFILE on WRITE.

PROBIT

PROBIT is available in SPSS Statistics Standard Edition or the Regression Option.

```
PROBIT response-count varname OF observation-count varname
        WITH varlist [BY varname(min,max)]

 [/MODEL={PROBIT**}]
         {LOGIT   }
         {BOTH    }

 [/LOG=[{10**  }]
        {2.718}
        {value}
        {NONE }

 [/CRITERIA=[{OPTOL   }({epsilon**0.8})][P({0.15**})][STEPLIMIT({0.1**})]
             {CONVERGE} {n            }     {p     }              {n    }

            [ITERATE({max(50,3(p+1)**})]]
                     {n              }

 [/NATRES[=value]]

 [/PRINT={[CI**] [FREQ**] [RMP**]} [PARALL] [NONE] [ALL]]
         {DEFAULT**              }

 [/MISSING=[{EXCLUDE**}]  ]
            {INCLUDE  }
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
PROBIT  R OF N BY ROOT(1,2) WITH X
  /MODEL = BOTH.
```

Overview

PROBIT can be used to estimate the effects of one or more independent variables on a dichotomous dependent variable (such as dead or alive, employed or unemployed, product purchased or not). The program is designed for dose-response analyses and related models, but PROBIT can also estimate logistic regression models.

Options

The Model. You can request a probit or logit response model, or both, for the observed response proportions with the MODEL subcommand.

Transform Predictors. You can control the base of the log transformation applied to the predictors or request no log transformation with the LOG subcommand.

Natural Response Rates. You can instruct PROBIT to estimate the natural response rate (threshold) of the model or supply a known natural response rate to be used in the solution with the NATRES subcommand.

Algorithm Control Parameters. You can specify values of algorithm control parameters, such as the limit on iterations, using the CRITERIA subcommand.

Statistics. By default, PROBIT calculates frequencies, fiducial confidence intervals, and the relative median potency. It also produces a plot of the observed probits or logits against the values of a single independent variable. Optionally, you can use the PRINT subcommand to request a test of the parallelism of regression lines for different levels of the grouping variable or to suppress any or all of these statistics.

Basic Specification

- The basic specification is the response-count variable, keyword OF, the observation-count variable, keyword WITH, and at least one independent variable.

- PROBIT calculates maximum-likelihood estimates for the parameters of the default probit response model and automatically displays estimates of the regression coefficient and intercept terms, their standard errors, a covariance matrix of parameter estimates, and a Pearson chi-square goodness-of-fit test of the model.

Subcommand Order

- The variable specification must be first.
- Subcommands can be named in any order.

Syntax Rules

- The variables must include a response count, an observation count, and at least one predictor. A categorical grouping variable is optional.
- All subcommands are optional and each can appear only once.
- Generally, data should not be entered for individual observations. PROBIT expects predictor values, response counts, and the total number of observations as the input case.
- If the data are available only in a case-by-case form, use AGGREGATE first to compute the required response and observation counts.

Operations

- The transformed response variable is predicted as a linear function of other variables using the nonlinear-optimization method. Note that the previous releases used the iteratively weighted least-squares method, which has a different way of transforming the response variables. See the topic “MODEL Subcommand ” on page 1543 for more information.
- If individual cases are entered in the data, PROBIT skips the plot of transformed response proportions and predictor values.
- If individual cases are entered, the chi-square goodness-of-fit statistic and associated degrees of freedom are based on the individual cases. The case-based chi-square goodness-of-fit statistic generally differs from that calculated for the same data in aggregated form.

Limitations

- Only one prediction model can be tested on a single PROBIT command, although both probit and logit response models can be requested for that prediction.
- Confidence limits, the plot of transformed response proportions and predictor values, and computation of relative median potency are necessarily limited to single-predictor models.

Variable Specification

The variable specification on PROBIT identifies the variables for response count, observation count, groups, and predictors. The variable specification is required.

- The variables must be specified first. The specification must include the response-count variable, followed by the keyword OF and then the observation-count variable.
- If the value of the response-count variable exceeds that of the observation-count variable, a procedure error occurs and PROBIT is not executed.
- At least one predictor (covariate) must be specified following the keyword WITH. The number of predictors is limited only by available workspace. All predictors must be continuous variables.
- You can specify a grouping variable (factor) after the keyword BY. Only one variable can be specified. It must be numeric and can contain only integer values. You must specify, in parentheses, a range indicating the minimum and maximum values for the grouping variable. Each integer value in the specified range defines a group.
- Cases with values for the grouping variable that are outside the specified range are excluded from the analysis.

- Keywords BY and WITH can appear in either order. However, both must follow the response-and-observation-count variables.

Example

```
PROBIT R OF N WITH X.
```

- The number of observations having the measured response appears in variable *R*, and the total number of observations is in *N*. The predictor is *X*.

Example

```
PROBIT  R OF N BY ROOT(1,2) WITH X.
PROBIT  R OF N WITH X BY ROOT(1,2).
```

- Because keywords BY and WITH can be used in either order, these two commands are equivalent. Each command specifies *X* as a continuous predictor and *ROOT* as a categorical grouping variable.
- Groups are identified by the levels of variable *ROOT*, which may be 1 or 2.
- For each combination of predictor and grouping variables, the variable *R* contains the number of observations with the response of interest, and *N* contains the total number of observations.

MODEL Subcommand

MODEL specifies the form of the dichotomous-response model. Response models can be thought of as transformations (*T*) of response rates, which are proportions or probabilities (*p*). Note the difference in the transformations between the current version and the previous versions.

- A **probit** is the inverse of the cumulative standard normal distribution function. Thus, for any proportion, the probit transformation returns the value below which that proportion of standard normal deviates is found. For the probit response model, the program uses *T* (*p*) = PROBIT (*p*). Hence:

  T (0.025) = PROBIT (0.025) = –1.96

  T (0.400) = PROBIT (0.400) = –0.25

  T (0.500) = PROBIT (0.500) = 0.00

  T (0.950) = PROBIT (0.950) = 1.64
- A **logit** is simply the natural log of the odds ratio, *p*/(1-*p*). In the Probit procedure, the response function is given as T (p) = $\log_e(p/(1-p))$. Hence:

  T (0.025) = LOGIT (0.025) = –3.66

  T (0.400) = LOGIT (0.400) = –0.40

  T (0.500) = LOGIT (0.500) = 0.00

  T (0.950) = LOGIT (0.950) = 2.94

You can request one or both of the models on the MODEL subcommand. The default is PROBIT if the subcommand is not specified or is specified with no keyword.

PROBIT. *Probit response model.* This is the default.

LOGIT. *Logit response model.*

BOTH. *Both probit and logit response models.* PROBIT displays all the output for the logit model followed by the output for the probit model.

- If subgroups and multiple-predictor variables are defined, PROBIT estimates a separate intercept, a_j, for each subgroup and a regression coefficient, b_i, for each predictor.

LOG Subcommand

LOG specifies the base of the logarithmic transformation of the predictor variables or suppresses the default log transformation.

- LOG applies to all predictors.
- To transform only selected predictors, use COMPUTE commands before the Probit procedure. Then specify NONE on the LOG subcommand.
- If LOG is omitted, a logarithm base of 10 is used.
- If LOG is used without a specification, the natural logarithm base *e* (2.718) is used.
- If you have a control group in your data and specify NONE on the LOG subcommand, the control group is included in the analysis. See the topic "NATRES Subcommand " on page 1544 for more information.

You can specify one of the following on LOG:

value. *Logarithm base to be applied to all predictors.*

NONE. *No transformation of the predictors.*

Example

```
PROBIT R OF N BY ROOT (1,2) WITH X
  /LOG = 2.
```

- LOG specifies a base-2 logarithmic transformation.

CRITERIA Subcommand

Use CRITERIA to specify the values of control parameters for the PROBIT algorithm. You can specify any or all of the keywords below. Defaults remain in effect for parameters that are not changed.

OPTOL(n). *Optimality tolerance.* Alias CONVERGE. If an iteration point is a feasible point and the next step will not produce a relative change in either the parameter vector or the log-likelihood function of more than the square root of *n*, an optimal solution has been found. OPTOL can also be thought of as the number of significant digits in the log-likelihood function at the solution. For example, if OPTOL=10^{-6}, the log-likelihood function should have approximately six significant digits of accuracy. The default value is machine epsilon**0.8.

ITERATE(n). *Iteration limit.* Specify the maximum number of iterations. The default is max (50, 3(*p* + 1)), where *p* is the number of parameters in the model.

P(p). *Heterogeneity criterion probability.* Specify a cutoff value between 0 and 1 for the significance of the goodness-of-fit test. The cutoff value determines whether a heterogeneity factor is included in calculations of confidence levels for effective levels of a predictor. If the significance of chi-square is greater than the cutoff, the heterogeneity factor is not included. If you specify 0, this criterion is disabled; if you specify 1, a heterogeneity factor is automatically included. The default is 0.15.

STEPLIMIT(n). *Step limit.* The PROBIT algorithm does not allow changes in the length of the parameter vector to exceed a factor of *n*. This limit prevents very early steps from going too far from good initial estimates. Specify any positive value. The default value is 0.1.

CONVERGE(n). *Alias of OPTOL*.

NATRES Subcommand

You can use NATRES either to supply a known natural response rate to be used in the solution or to instruct PROBIT to estimate the natural (or threshold) response rate of the model.

- To supply a known natural response rate as a constraint on the model solution, specify a value less than 1 on NATRES.

- To instruct PROBIT to estimate the natural response rate of the model, you can indicate a control group by giving a 0 value to any of the predictor variables. PROBIT displays the estimate of the natural response rate and the standard error and includes the estimate in the covariance/correlation matrix as NAT RESP.
- If no control group is indicated and NATRES is specified without a given value, PROBIT estimates the natural response rate from the entire data and informs you that no control group has been provided. The estimate of the natural response rate and the standard error are displayed and NAT RESP is included in the covariance/correlation matrix.
- If you have a control group in your data and specify NONE on the LOG subcommand, the control group is included in the analysis.

Example

```
DATA LIST FREE / SOLUTION DOSE NOBSN NRESP.
BEGIN DATA
1  5 100 20
1 10  80 30
1  0 100 10
...
END DATA.

PROBIT NRESP OF NOBSN BY SOLUTION(1,4) WITH DOSE
  /NATRES.
```

- This example reads four variables and requests a default analysis with an estimate of the natural response rate.
- The predictor variable, *DOSE*, has a value of 0 for the third case.
- The response count (10) and the observation count (100) for this case establish the initial estimate of the natural response rate.
- Because the default log transformation is performed, the control group is not included in the analysis.

Example

```
DATA LIST FREE / SOLUTION DOSE NOBSN NRESP.
BEGIN DATA
1  5 100 20
1 10  80 30
1  0 100 10
  ...
END DATA.

PROBIT NRESP OF NOBSN BY SOLUTION(1,4) WITH DOSE
  /NATRES = 0.10.
```

- This example reads four variables and requests an analysis in which the natural response rate is set to 0.10. The values of the control group are ignored.
- The control group is excluded from the analysis because the default log transformation is performed.

PRINT Subcommand

Use PRINT to control the statistics calculated by PROBIT.

- PROBIT always displays the plot (for a single-predictor model) and the parameter estimates and covariances for the probit model.
- If PRINT is used, the requested statistics are calculated and displayed in addition to the parameter estimates and plot.
- If PRINT is not specified or is specified without any keyword, FREQ, CI, and RMP are calculated and displayed in addition to the parameter estimates and plot.

DEFAULT. *FREQ, CI, and RMP.* This is the default if PRINT is not specified or is specified by itself.

FREQ. *Frequencies.* Display a table of observed and predicted frequencies with their residual values. If observations are entered on a case-by-case basis, this listing can be quite lengthy.

CI. *Fiducial confidence intervals.* Print fiducial confidence intervals [57] for the levels of the predictor needed to produce each proportion of responses. PROBIT displays this default output for single-predictor

models only. If a categorical grouping variable is specified, PROBIT produces a table of confidence intervals for each group. If the Pearson chi-square goodness-of-fit test is significant ($p < 0.15$ by default), PROBIT uses a heterogeneity factor to calculate the limits.

RMP. *Relative median potency.* Display the relative median potency (RMP) of each pair of groups defined by the grouping variable. PROBIT displays this default output for single-predictor models only. For any pair of groups, the RMP is the ratio of the stimulus tolerances in those groups. **Stimulus tolerance** is the value of the predictor necessary to produce a 50% response rate. If the derived model for one predictor and two groups estimates that a predictor value of 21 produces a 50% response rate in the first group, and that a predictor value of 15 produces a 50% response rate in the second group, the relative median potency would be 21/15 = 1.40. In biological assay analyses, RMP measures the comparative strength of preparations.

PARALL. *Parallelism test.* Produce a test of the parallelism of regression lines for different levels of the grouping variable. This test displays a chi-square value and its associated probability. It requires an additional pass through the data and, thus, additional processing time.

NONE. *Display only the unconditional output.* This option can be used to override any other specification on the PRINT subcommand for PROBIT.

ALL. *All available output.* This is the same as requesting FREQ, CI, RMP, and PARALL.

MISSING Subcommand

PROBIT always deletes cases having a missing value for any variable. In the output, PROBIT indicates how many cases it rejected because of missing data. This information is displayed with the DATA Information that prints at the beginning of the output. You can use the MISSING subcommand to control the treatment of user-missing values.

EXCLUDE. *Delete cases with user-missing values.* This is the default. You can also make it explicit by using the keyword DEFAULT.

INCLUDE. *Include user-missing values.* PROBIT treats user-missing values as valid. Only cases with system-missing values are rejected.

References

Finney, D. J. 1971. *Probit analysis*. Cambridge: Cambridge University Press.

[57] Finney, D. J. 1971. *Probit analysis*. Cambridge: Cambridge University Press.

PROCEDURE OUTPUT

```
PROCEDURE OUTPUT OUTFILE=file.
```

Example

```
PROCEDURE OUTPUT OUTFILE=CELLDATA.
```

Overview

PROCEDURE OUTPUT specifies the files to which CROSSTABS and SURVIVAL (included in SPSS Statistics Standard Edition or the Advanced Statistics Option) can write procedure output. PROCEDURE OUTPUT has no other applications.

Basic Specification

The only specification is OUTFILE and the file specification. PROCEDURE OUTPUT must precede the command to which it applies.

Operations

Commands with the WRITE subcommand or keyword write to the output file that is specified on the most recent PROCEDURE OUTPUT command. If only one output file has been specified, the output from the last such procedure overwrites all previous ones.

Examples

Using PROCEDURE OUTPUT with CROSSTABS

```
PROCEDURE OUTPUT OUTFILE=CELLDATA.
CROSSTABS VARIABLES=FEAR SEX (1,2)
  /TABLES=FEAR BY SEX
  /WRITE=ALL.
```

- PROCEDURE OUTPUT precedes CROSSTABS and specifies *CELLDATA* as the file to receive the cell frequencies.
- The WRITE subcommand on CROSSTABS is required for writing cell frequencies to a procedure output file.

Using PROCEDURE OUTPUT with SURVIVAL

```
PROCEDURE OUTPUT OUTFILE=SURVTBL.
SURVIVAL  TABLES=ONSSURV,RECSURV BY TREATMNT(1,3)
  /STATUS = RECURSIT(1,9) FOR RECSURV
  /STATUS = STATUS(3,4) FOR ONSSURV
  /INTERVAL=THRU 50 BY 5 THRU 100 BY 10/PLOTS/COMPARE
  /CALCULATE=CONDITIONAL PAIRWISE
  /WRITE=TABLES.
```

- PROCEDURE OUTPUT precedes SURVIVAL and specifies *SURVTBL* as the file to receive the survival tables.
- The WRITE subcommand on SURVIVAL is required for writing survival tables to a procedure output file.

PROPORTIONS

PROPORTIONS is available in Statistics Base Edition.

One-sample tests and confidence intervals

```
PROPORTIONS
   [/ONESAMPLE varlist
      [TESTVAL={0.5**}
               {value}]
      [TESTTYPES={EXACT} {MIDP**} {SCORE**} {SCORECC}{WALD} {WALDCC} {NONE}]
      [CITYPES={AGRESTI_COULL**} {ANSCOMBE} {CLOPPER_PEARSON} {JEFFREYS**}
               {LOGIT}{WALD} {WALDCC} {WILSON_SCORE**} {WILSON_SCORECC}{NONE}]]
   [/SUCCESSVALUE={LAST**          }
                  {FIRST           }
                  {LEVEL(values)   }
                  {MIDPOINT        }
                  {CUTPOINT(value)}]
   [/MISSING
      [SCOPE={ANALYSIS**} {LISTWISE}]
      [USERMISSING={EXCLUDE**} {INCLUDE}]
   [/CRITERIA
      [CILEVEL={95** }
               {value}]]
```

**Default if subcommand or keyword is omitted.

Paired-samples tests and confidence intervals for differences in proportions

```
PROPORTIONS
   [/PAIREDSAMPLES varlist [WITH varlist[(PAIRED)]
      [TESTTYPES={EXACT} {MIDP**} {MCNEMAR**} {MCNEMARCC} {WALD} {WALDCC}{NONE}]
      [CITYPES={AGRESTI_MIN} {BONETT_PRICE**} {NEWCOMBE**} {WALD} {WALDCC}{NONE]]
   [/SUCCESSVALUE={LAST**          }
                  {FIRST           }
                  {LEVEL(values)   }
                  {MIDPOINT        }
                  {CUTPOINT(value)}]
   [/MISSING
      [SCOPE={ANALYSIS**} {LISTWISE}]
      [USERMISSING={EXCLUDE**} {INCLUDE}]
   [/CRITERIA
      [CILEVEL={95** }
               {value}]]
```

**Default if subcommand or keyword is omitted.

Independent-samples tests and confidence intervals for differences in proportions

```
PROPORTIONS
   [/INDEPENDENTSAMPLES varlist BY varname
      [SELECT={LEVEL(1,2**                  })
              {value1/string1 value2/string2}
              {MIDPOINT}
              {CUTPOINT(value)}]
      [TESTTYPES={HAUCK_ANDERSON} {WALD} {WALDCC} {WALDH0**} {WALDH0CC} {NONE}]
      [CITYPES={AGRESTI_CAFFO**} {BROWN_LI_JEFFREYS} {HAUCK_ANDERSON}{NEWCOMBE**}
               {NEWCOMBECC} {WALD}{WALDCC}{NONE}]]
   [/SUCCESSVALUE={LAST**          }
                  {FIRST           }
                  {LEVEL(values)   }
                  {MIDPOINT        }
                  {CUTPOINT(value)}]
   [/MISSING
      [SCOPE={ANALYSIS**} {LISTWISE}]
      [USERMISSING={EXCLUDE**} {INCLUDE}]
   [/CRITERIA
      [CILEVEL={95** }
               {value}]]
```

**Default if subcommand or keyword is omitted.

Example

```
PROPORTIONS
   /ONESAMPLE measure1 measure2 measure3
   /TESTSTYPES=EXACT
   /CITYPES=CLOPPER_PEARSON.
```

Release history

Release 27.0.1

- Command introduced.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Overview

PROPORTIONS computes tests and confidence intervals for binomial proportions or differences of proportions. Statistics are available for one-sample proportions (tested against a specified value), paired samples (different variables), or independent samples (different groups of cases). Various options for types of test statistics and confidence intervals are offered. Other procedures that provide some of the same functionality include **CROSSTABS**, **NPAR TESTS**, and **NPTESTS**.

Options

Statistics
For each type of analysis (one sample, paired samples, or independent samples) various versions of test statistics and confidence intervals are available.

Basic specification

The basic specification depends on whether you want one-sample tests and/or confidence intervals, paired-samples tests and/or confidence intervals, or independent-samples tests and/or confidence intervals. For all types of analyses, **PROPORTIONS** displays requested test statistics and two-sided probabilities, confidence intervals for differences of proportions, as well as proportions, standard errors, and counts for each group or variable.

- Use the **ONESAMPLE** subcommand to request one-sample tests and confidence intervals. Output includes the observed proportion, the estimate of the difference between the population proportion and the hypothesized population proportion, asymptotic standard errors under null and alternative hypotheses, specified test statistics with two-sided probabilities, and specified confidence intervals for proportions.
- Use the **PAIREDSAMPLES** subcommand to request paired-samples tests and confidence intervals for differences among proportions. Output includes observed proportions, estimates of differences in population proportions, asymptotic standard errors of population differences under null and alternative hypotheses, specified test statistics with two-sided probabilities, and specified confidence intervals for differences in proportions.
- Use the **INDEPENDENTSAMPLES** subcommand to request independent-samples tests and confidence intervals. Output includes observed proportions, estimates of differences in population proportions, asymptotic standard errors of population differences under null and alternative hypotheses, specified test statistics with two-sided probabilities, and specified confidence intervals for differences in proportions.

Subcommand order

Subcommands can be named in any order.

Limitations

- Only one of the three types of analyses is allowed within a single command. Therefore, only one instance of one of the **ONESAMPLE**, **PAIREDSAMPLES**, or **INDEPENDENTSAMPLES** subcommands is allowed.
- Maximum of one **TESTVAL** value on the **ONESAMPLE** subcommand.
- Maximum of one **WITH** and one **PAIRED** specification on the **PAIREDSAMPLES** subcommand.

- Maximum of one **GROUPS** specification on the **INDEPENDENTSAMPLES** subcommand.

Examples

One-sample tests and confidence intervals

```
PROPORTIONS
   /ONESAMPLE measure1 measure2 measure3
   /TESTSTYPES=EXACT
   /CITYPES=CLOPPER_PEARSON.
```

- The procedure tests the variables **measure1**, **measure2**, and **measure3** against the default 0.5 test value.
- Instead of providing the default test statistics, the exact probability for the binomial test is computed.
- Confidence intervals for the population proportions for each measure are computed using the Clopper-Pearson pseudo-exact method instead of providing the default types of intervals.

Paired-samples tests and confidence intervals

```
PROPORTIONS
   /PAIREDSAMPLES pre1 pre2 WITH post1 post2 (PAIRED)
   /SUCCESS=FIRST
   /CRITERIA CILEVEL=99.
```

- The procedure compares **pre1** with **post1** and **pre2** with **post2**, computing default test types and confidence intervals for differences in proportions.
- The first or lowest value in the sorted values for each variable is deemed the “success” value used in the numerator of each proportion.
- Confidence interval coverage levels are 99% instead of the default 95%.

Independent-samples tests and confidence intervals

```
PROPORTIONS
   /INDEPENDENTSAMPLES test1 TO test4 BY group
   /SELECT=LEVEL('a','b')
   /TESTTYPES=HAUCK_ANDERSON WALDH0
   /CITYPES=AGRESTI_CAFFO HAUCK_ANDERSON NEWCOMBE
   /MISSING SCOPE=LISTWISE.
```

- The procedure compares each of the variables test1 through **test4** across groups **a** and **b**.
- Hauck-Anderson test statistics are provided in addition to the default Wald statistics using the null difference in calculating standard errors of differences.
- Hauck-Anderson confidence intervals for differences in proportions are computed in addition to the default Agresti-Caffo and Newcombe intervals.
- Cases missing on any of the variables named on the command are omitted from all of the analyses.

ONESAMPLE subcommand

ONESAMPLE specifies analyses for one-sample proportions.

- A variable list containing at least one variable is required.

TESTVAL
An optional keyword that specifies a test value between 0 and 1. The default value is 0.5.

TESTTYPES
An optional keyword that specifies which types of test statistics are displayed. The keyword can also suppress all tests. Available test types include: **EXACT**, **MIDP**, **SCORE**, **SCORECC**, **WALD**, and **WALDCC**. By default, **MIDP** “pseudo-exact” probabilities and **SCORE** test statistics and associated probabilities are displayed.

CITYPES
An optional keyword that specifies which types of confidence intervals are displayed. The keyword can also suppress all confidence intervals. Available confidence intervals include: **AGRESTI_COULL**, **ANSCOMBE**, **CLOPPER_PEARSON**, **JEFFREYS**, **LOGIT**, **WALD**, **WALDCC**, **WILSON_SCORE**, and **WILSON_SCORECC**. By default, **AGRESTI_COULL**, **JEFFREYS**, and **WILSON_SCORE** intervals are displayed.

PAIREDSAMPLES subcommand

PAIREDSAMPLES compares sample pairs by computing default test types and confidence intervals for differences in proportions.

- A variable list containing at least two variables is required.
- If a single list of variables is specified, each member of the list is paired with every other member of the list.
- If two lists of variables are separated by **WITH** without the (**PAIRED**) keyword, each member of the first list is paired with each member of the second list.
- If two lists of variables are separated by **WITH** and the second list is followed by (**PAIRED**), members of the two lists in order are paired: the first member of the first list is paired with the first member of the second list, the second members of each list are paired, etc. Unmatched variables are ignored and a warning message is issued.

TESTTYPES
An optional keyword that specifies which types of test statistics are displayed. The keyword can also suppress all tests. Available test types include: **EXACT**, **MIDP**, **MCNEMAR**, **MCNEMARCC**, **WALD**, and **WALDCC**. By default, **MIDP** "pseudo-exact" probabilities and **MCNEMAR** test statistics and associated probabilities are displayed.

CITYPES
An optional keyword that specifies which types of confidence intervals are displayed. The keyword can also suppress all confidence intervals. Available confidence intervals include: **AGRESTI_MIN**, **BONETT_PRICE**, **NEWCOMBE**, **WALD**, and **WALDCC**. By default, **BONETT_PRICE** and **NEWCOMBE** intervals are displayed.

INDEPENDENTSAMPLES subcommand

INDEPENDENTSAMPLES compares each of the variables across groups and computes hypothesis tests and confidence intervals for the difference in two independent binomial proportions. Each hypothesis test or confidence interval is a separate test or individual interval.

- A variable list containing at least one dependent variable and the keyword **BY**, followed by a single variable to identify the two groups to be compared, are required.
- The grouping variable can be either numeric or string.
- If the **SELECT** keyword is not specified, the grouping variable must be numeric, and contain values 1 and 2, which define the groups to be compared. Cases with other values are ignored.
- If two lists of variables are separated by **WITH** and the second list is followed by (**PAIRED**), members of the two lists in order are paired: the first member of the first list is paired with the first member of the second list, the second members of each list are paired, etc. Unmatched variables are ignored and a warning message is issued.

SELECT=LEVEL
Allows specification within parentheses of the two numeric or string values to be compared. String values should be enclosed in single quotes (cases with other values are ignored).

SELECT=MIDPOINT applies only to numeric variables. Cases at or above the midpoint of the distribution of the grouping variable are assigned to the second group; cases below the midpoint are assigned to the first group.

SELECT=CUTPOINT applies only the numeric variables and allows specification with parentheses of a single numeric value. Cases at or above the cut point on the grouping variable are assigned to the second group; cases below the cut point are assigned to the first group.

TESTTYPES
An optional keyword that specifies which types of test statistics are displayed. The keyword can also suppress all tests. Available test types include: **HAUCK_ANDERSON**, **WALD**, **WALDCC**, **WALDH0**, and **WALDH0CC**. By default, the **WALDH0** test statistic and associated probabilities are displayed.

CITYPES
An optional keyword that specifies which types of confidence intervals are displayed. The keyword can also suppress all confidence intervals. Available confidence intervals include: **AGRESTI_CAFFO**, **BROWN_LI_JEFFREYS**, **HAUCK_ANDERSON**, **NEWCOMBE**, **NEWCOMBECC**, **WALD**, and **WALDCC**. By default, **AGRESTI_CAFFO** and **NEWCOMBE** intervals are displayed.

SUCCESS subcommand

SUCCESS optionally specifies how to identify the value or values indicating successes contributing to the numerators of proportions.

LAST

The last or highest value among the sorted distinct values in the data is used. This applies to numeric or string variables. This is the default setting.

FIRST
The first or lowest value among the sorted distinct values in the data is used. This applies to numeric or string variables.

LEVEL
Specifies one or more parenthesized values. Multiple values must be separated by spaces. The keyword applies to numeric or string variables. String values must be enclosed in single quotes. Cases with other values are ignored.

MIDPOINT
Values at or above the middle of the range of observed values in the data. The keyword applies only to numeric data.

CUTPOINT
Defines values that are equal to or above the cut point as "success". You can specify a value for the cut point. The keyword applies only to numeric data.

MISSING subcommand

MISSING optionally specifies the handling of missing values.

SCOPE
The specified keyword should be either **ANALYSIS** or **LISTWISE**. **ANALYSIS** indicates inclusion of all cases with sufficient data on the variables used in each particular analysis, while **LISTWISE** indicates inclusion of all cases with sufficient data on all variables used across all analyses specified by the command. **ANALYSIS** is the default setting.

USERMISSING
The specified keyword should be either **EXCLUDE** or **INCLUDE**. **EXCLUDE** treats user missing values as missing. **INCLUDE** ignores user missing value designations and treats user missing values as valid values. **EXCLUDE** is the default setting.

Note: This keyword setting has no effect on system missing values.

CRITERIA subcommand

CRITERIA optionally specifies coverage levels for confidence intervals for proportions and differences of proportions.

CILEVEL
Specifies a numeric value in the range (0,100). `0.95` is the default, which indicates 95% confidence intervals.

References

Agresti, A. A., & Coull, B. A. 1998. *Approximate is better than "exact" for interval estimation of binomial proportions.* The American Statistician, 52(2): 199-126.

Agresti, A. A., & Caffo, B. 2000. Simple and effective confidence intervals for proportions and differences of proportions result from adding two successes and two failures. *The American Statistician*, 54(4): 280-288.

Agresti, A. A., & Min, Y. 2005. Simple improved confidence intervals for comparing matched proportions. *Statistics in Medicine*, 24(5), 729-740.

Bonett, D. G., & Price, R. M. 2012. Adjusted Wald confidence interval for a difference of binomial proportions based on paired data. *Journal of Educational and Behavioral Statistics*, 37(4), 479-488.

Brown, L. D., T. Cai, and A. DasGupta. 2001. Interval estimation for a binomial proportion. *Statistical Science,* 16(2), 101-133.

Clopper, C., & Pearson, E. S. 1934. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika,* 26, 404-413.

Fagerland, M. W., Lydersen, S., & Laake, P. 2013. The McNemar test for binary matched-pairs data: mid-pand asymptotic are better than exact conditional. *BMC Medical Research Methodology*, 13, 91-98.

Fagerland, M. W., Lydersen, S., & Laake, P. 2014. Recommended tests and confidence intervals for paired binomial proportions. *Statistics in Medicine*, 33(16), 2850-2875.

Fagerland, M. W., Lydersen, S., & Laake, P. 2015. Recommended confidence intervals for two independent binomial proportions. *Statistical Methods in Medical Research*, 24(2), 224-254.

Newcombe, R. G. 1998a. Two-sided confidence intervals for the single proportion:Comparison of seven methods. *Statistics in Medicine*, 17: 857-872.

Newcombe, R. G. 1998b. Interval estimation for the difference between independent proportions: Comparison of eleven methods. *Statistics in Medicine*, 17: 873-890.

Newcombe, R. G. 1998c. Improved confidence intervals for the difference between binomial proportions based on paired data. *Statistics in Medicine*, 17: 2635-2650.

Wilson, E. B. 1927. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22, 209-212.

PROXIMITIES

PROXIMITIES is available in Statistics Base Edition.

```
PROXIMITIES  varlist  [/VIEW={CASE**  }]
                            {VARIABLE}

 [/STANDARDIZE=[{VARIABLE}] [{NONE** }]]
                {CASE    }   {Z      }
                             {SD     }
                             {RANGE  }
                             {MAX    }
                             {MEAN   }
                             {RESCALE}

 [/MEASURE=[{EUCLID**          }] [ABSOLUTE] [REVERSE] [RESCALE]
           {SEUCLID            }
           {COSINE             }
           {CORRELATION        }
           {BLOCK              }
           {CHEBYCHEV          }
           {POWER(p,r)         }
           {MINKOWSKI(p)       }
           {CHISQ              }
           {PH2                }
           {RR[(p[,np])]       }
           {SM[(p[,np])]       }
           {JACCARD[(p[,np])]  }
           {DICE[(p[,np])]     }
           {SS1[(p[,np])]      }
           {RT[(p[,np])]       }
           {SS2[(p[,np])]      }
           {K1[(p[,np])]       }
           {SS3[(p[,np])]      }
           {K2[(p[,np])]       }
           {SS4[(p[,np])]      }
           {HAMANN[(p[,np])]   }
           {OCHIAI[(p[,np])]   }
           {SS5[(p[,np])]      }
           {PHI[(p[,np])]      }
           {LAMBDA[(p[,np])]   }
           {D[(p[,np])]        }
           {Y[(p[,np])]        }
           {Q[(p[,np])]        }
           {BEUCLID[(p[,np])]  }
           {SIZE[(p[,np])]     }
           {PATTERN[(p[,np])]  }
           {BSEUCLID[(p[,np])]}
           {BSHAPE[(p[,np])]   }
           {DISPER[(p[,np])]   }
           {VARIANCE[(p[,np])]}
           {BLWMN[(p[,np])]    }
           {NONE               }

 [/PRINT=[{PROXIMITIES**}]]  [/ID=varname]
         {NONE           }

 [/MISSING=[EXCLUDE**]  [INCLUDE]]

 [/MATRIX=[IN({'savfile'|'dataset'})] [OUT({'savfile'|'dataset'})]]
            {*                    }        {*                    }
```

**Default if subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
PROXIMITIES A B C.
```

Overview

PROXIMITIES computes a variety of measures of similarity, dissimilarity, or distance between pairs of cases or pairs of variables for moderate-sized datasets (see “Limitations” below). PROXIMITIES matrix output can be used as input to procedures ALSCAL, CLUSTER, and FACTOR.

Options

Standardizing Data. With the STANDARDIZE subcommand, you can use several different methods to standardize the values for each variable or for each case.

Proximity Measures. You can use the MEASURE subcommand to compute a variety of similarity, dissimilarity, and distance measures. (Similarity measures increase with greater similarity; dissimilarity and distance measures decrease.) MEASURE can compute measures for interval data, frequency-count data, and binary data. Only one measure can be requested in any one PROXIMITIES procedure. With the VIEW subcommand, you can control whether proximities are computed between variables or between cases.

Output. You can use the PRINT subcommand to display a computed matrix.

Matrix Input and Output. You can use the MATRIX subcommand to write a computed proximities matrix to IBM SPSS Statistics data files. This matrix can be used as input to procedures CLUSTER, ALSCAL, and FACTOR. You can also use MATRIX to read a similarity, dissimilarity, or distance matrix. This option lets you rescale or transform existing proximity matrices.

Basic Specification

The basic specification is a variable list, which obtains Euclidean distances between cases based on the values of each specified variable.

Subcommand Order

- The variable list must be first.
- Subcommands can be named in any order.

Operations

- PROXIMITIES ignores case weights when computing coefficients.

Limitations

- PROXIMITIES keeps the raw data for the current split-file group in memory. Storage requirements increase rapidly with the number of cases and the number of items (cases or variables) for which PROXIMITIES computes coefficients.

Example

```
PROXIMITIES A B C.
```

- PROXIMITIES computes Euclidean distances between cases based on the values of variables *A*, *B*, and *C*.

Variable Specification

- The variable list must be specified first.
- The variable list can be omitted when an input matrix data file is specified. A slash must then be specified before the first subcommand to indicate that the variable list is omitted.

STANDARDIZE Subcommand

Use STANDARDIZE to standardize data values for either cases or variables before computing proximities. One of two options can be specified to control the direction of standardization:

VARIABLE. *Standardize the values for each variable.* This setting is the default.

CASE. *Standardize the values within each case.*

Several standardization methods are available. These methods allow you to equalize selected properties of the values. All methods can be used with either VARIABLE or CASE. Only one standardization method can be specified.

- If STANDARDIZE is omitted, proximities are computed by using the original values (keyword NONE).

- If STANDARDIZE is used without specifications, proximities are computed by using *Z* scores (keyword Z).
- STANDARDIZE cannot be used with binary measures.

NONE. *Do not standardize.* Proximities are computed by using the original values. This setting is the default if STANDARDIZE is omitted.

Z. *Standardize values to Z scores, with a mean of 0 and a standard deviation of 1.* PROXIMITIES subtracts the mean value for the variable or case from each value that is being standardized, and then PROXIMITIES divides by the standard deviation. If the standard deviation is 0, PROXIMITIES sets all values for the case or variable to 0. This setting is the default if STANDARDIZE is used without specifications.

RANGE. *Standardize values to have a range of 1.* PROXIMITIES divides each value that is being standardized by the range of values for the variable or case. If the range is 0, PROXIMITIES leaves all values unchanged.

RESCALE. *Standardize values to have a range from 0 to 1.* From each value that is being standardized, PROXIMITIES subtracts the minimum value and then divides by the range for the variable or case. If a range is 0, PROXIMITIES sets all values for the case or variable to 0.50.

MAX. *Standardize values to a maximum magnitude of 1.* PROXIMITIES divides each value that is being standardized by the maximum value for the variable or case. If the maximum of the values is 0, PROXIMITIES divides each value by the absolute magnitude of the smallest value and adds 1.

MEAN. *Standardize values to a mean of 1.* PROXIMITIES divides each value that is being standardized by the mean of the values for the variable or case. If the mean is 0, PROXIMITIES adds 1 to all values for the case or variable to produce a mean of 1.

SD. *Standardize values to unit standard deviation.* PROXIMITIES divides each value that is being standardized by the standard deviation of the values for the variable or case. PROXIMITIES does not change the values if their standard deviation is 0.

Example

```
PROXIMITIES A B C
  /STANDARDIZE=CASE RANGE.
```

- Within each case, values are standardized to have ranges of 1.

VIEW Subcommand

VIEW indicates whether proximities are computed between cases or between variables.

CASE. *Compute proximity values between cases.* This is the default.

VARIABLE. *Compute proximity values between variables.*

MEASURE Subcommand

MEASURE specifies the similarity, dissimilarity, or distance measure that PROXIMITIES computes. Three transformations are available:

ABSOLUTE. *Take the absolute values of the proximities.* Use ABSOLUTE when the sign of the values indicates the direction of the relationship (as with correlation coefficients) but only the magnitude of the relationship is of interest.

REVERSE. *Transform similarity values into dissimilarities, or vice versa.* Use this specification to reverse the ordering of the proximities by negating the values.

RESCALE. *Rescale the proximity values to a range of 0 to 1.* RESCALE standardizes the proximities by first subtracting the value of the smallest proximity and then dividing by the range. You would not usually use

RESCALE with measures that are already standardized on meaningful scales, as are correlations, cosines, and many binary coefficients.

PROXIMITIES can compute any one of a number of measures between items. You can choose among measures for interval data, frequency-count data, or binary data. Available keywords for each type of measures are defined in the following sections.

- Only one measure can be specified. However, each measure can be specified with any of the transformations ABSOLUTE, REVERSE, or RESCALE. To apply a transformation to an existing matrix of proximity values without computing any measures, use keyword NONE (see).
- If more than one transformation is specified, PROXIMITIES handles them in the order listed above: ABSOLUTE, REVERSE, and then RESCALE (regardless of the order in which they are specified).
- Each entry in the resulting proximity matrix represents a pair of items. The items can be either cases or variables, whichever is specified on the VIEW subcommand.
- When the items are cases, the computation for each pair of cases involves pairs of values for the specified variables.
- When the items are variables, the computation for each pair of variables involves pairs of values for the variables across all cases.

Example

```
PROXIMITIES A B C
  /MEASURE=EUCLID REVERSE.
```

- MEASURE specifies a EUCLID measure and a REVERSE transformation.

Measures for Interval Data

To obtain proximities for interval data, use one of the following keywords on MEASURE:

EUCLID. *Euclidean distance.* The distance between two items, *x* and *y*, is the square root of the sum of the squared differences between the values for the items. This setting is the default.

SEUCLID. *Squared Euclidean distance.* The distance between two items is the sum of the squared differences between the values for the items.

CORRELATION. *Correlation between vectors of values.* This measure is a pattern-similarity measure.

COSINE. *Cosine of vectors of values.* This measure is a pattern-similarity measure.

CHEBYCHEV. *Chebychev distance metric.* The distance between two items is the maximum absolute difference between the values for the items.

BLOCK. *City-block or Manhattan distance.* The distance between two items is the sum of the absolute differences between the values for the items.

MINKOWSKI(p). *Distance in an absolute Minkowski power metric.* The distance between two items is the *p*th root of the sum of the absolute differences to the *p*th power between the values for the items. Appropriate selection of the integer parameter *p* yields Euclidean and many other distance metrics.

POWER(p,r). *Distance in an absolute power metric.* The distance between two items is the *r*th root of the sum of the absolute differences to the *p*th power between the values for the items. Appropriate selection of the integer parameters *p* and *r* yields Euclidean, squared Euclidean, Minkowski, city-block, and many other distance metrics.

Measures for Frequency-Count Data

To obtain proximities for frequency-count data, use either of the following keywords on MEASURE:

CHISQ. *Based on the chi-square test of equality for two sets of frequencies.* The magnitude of this dissimilarity measure depends on the total frequencies of the two cases or variables whose dissimilarity is computed. Expected values are from the model of independence of cases or variables *x* and *y*.

PH2. *Phi-square between sets of frequencies.* This measure is the CHISQ measure normalized by the square root of the combined frequency. Therefore, its value does not depend on the total frequencies of the two cases or variables whose dissimilarity is computed.

Measures for Binary Data

Different binary measures emphasize different aspects of the relationship between sets of binary values. However, all measures are specified in the same way. Each measure has two optional integer-valued parameters, *p* (present) and *np* (not present).

- If both parameters are specified, PROXIMITIES uses the value of the first parameter as an indicator that a characteristic is present, and PROXIMITIES uses the value of the second parameter as an indicator that a characteristic is absent. PROXIMITIES skips all other values.
- If only the first parameter is specified, PROXIMITIES uses that value to indicate presence and uses all other values to indicate absence.
- If no parameters are specified, PROXIMITIES assumes that 1 indicates presence and 0 indicates absence.

Using the indicators for presence and absence within each item (case or variable), PROXIMITIES constructs a 2×2 contingency table for each pair of items and uses this table to compute a proximity measure for the pair.

Table 72. 2 x 2 contingency table

	Item 2 characteristics Present	**Item 2 characteristics Absent**
Item 1 characteristics Present	*a*	*b*
Item 1 characteristics Absent	*c*	*d*

PROXIMITIES computes all binary measures from the values of *a*, *b*, *c*, and *d*. These values are tallied across variables (when the items are cases) or cases (when the items are variables). For example, if variables *V*, *W*, *X*, *Y*, *Z* have values 0, 1, 1, 0, 1 for case 1 and have values 0, 1, 1, 0, 0 for case 2 (where 1 indicates presence and 0 indicates absence), the contingency table is as follows:

Case 1 characteristics Present. 2

Case 2 characteristics Absent. 0

The contingency table indicates that both cases are present for two variables (*W* and *X*), both cases are absent for two variables (*V* and *Y*), and case 1 is present and case 2 is absent for one variable (*Z*). There are no variables for which case 1 is absent and case 2 is present.

The available binary measures include matching coefficients, conditional probabilities, predictability measures, and other measures.

Matching Coefficients. The following table shows a classification scheme for PROXIMITIES matching coefficients. In this scheme, *matches* are joint presences (value *a* in the contingency table) or joint absences (value *d*). *Nonmatches* are equal in number to value *b* plus value *c*. Matches and non-matches may be weighted equally or not. The three coefficients JACCARD, DICE, and SS2 are related monotonically, as are SM, SS1, and RT. All coefficients in the table are similarity measures, and all coefficients exceptK1 and SS3 range from 0 to 1. K1 and SS3 have a minimum value of 0 and have no upper limit.

Table 73. Binary matching coefficients in PROXIMITIES

	Joint absences excluded from numerator	**Joint absences included in numerator**
All matches included in denominator, equal weight for matches and non-matches	RR	SM

Table 73. Binary matching coefficients in PROXIMITIES (continued)

	Joint absences excluded from numerator	**Joint absences included in numerator**
All matches included in denominator, double weight for matches		SS1
All matches included in denominator, double weight for non-matches		RT
Joint absences excluded from denominator, equal weight for matches and non-matches	JACCARD	
Joint absences excluded from denominator, double weight for matches	DICE	
Joint absences excluded from denominator, double weight for non-matches	SS2	
All matches excluded from denominator, equal weight for matches and non-matches	K1	SS3

RR[(p[,np])]. *Russell and Rao similarity measure.* This measure is the binary dot product.

SM[(p[,np])]. *Simple matching similarity measure.* This measure is the ratio of the number of matches to the total number of characteristics.

JACCARD[(p[,np])]. *Jaccard similarity measure.* This measure is also known as the *similarity ratio.*

DICE[(p[,np])]. *Dice (or Czekanowski or Sorenson) similarity measure.*

SS1[(p[,np])]. *Sokal and Sneath similarity measure 1.*

RT[(p[,np])]. *Rogers and Tanimoto similarity measure.*

SS2[(p[,np])]. *Sokal and Sneath similarity measure 2.*

K1[(p[,np])]. *Kulczynski similarity measure 1.* This measure has a minimum value of 0 and no upper limit. The measure is undefined when there are no non-matches (*b*=0 and *c*=0).

SS3[(p[,np])]. *Sokal and Sneath similarity measure 3.* This measure has a minimum value of 0 and no upper limit. The measure is undefined when there are no non-matches (*b*=0 and *c*=0).

Conditional Probabilities. The following binary measures yield values that can be interpreted in terms of conditional probability. All three measures are similarity measures.

K2[(p[,np])]. *Kulczynski similarity measure 2.* This measure yields the average conditional probability that a characteristic is present in one item given that the characteristic is present in the other item. The measure is an average over both items that are acting as predictors. The measure has a range of 0 to 1.

SS4[(p[,np])]. *Sokal and Sneath similarity measure 4.* This measure yields the conditional probability that a characteristic of one item is in the same state (presence or absence) as the characteristic of the other item. The measure is an average over both items that are acting as predictors. The measure has a range of 0 to 1.

HAMANN[(p[,np])]. *Hamann similarity measure.* This measure gives the probability that a characteristic has the same state in both items (present in both or absent from both) minus the probability that a characteristic has different states in the two items (present in one and absent from the other). HAMANN has a range of −1 to +1 and is monotonically related to SM, SS1, and RT.

Predictability Measures. The following four binary measures assess the association between items as the predictability of one item given the other item. All four measures yield similarities.

LAMBDA[(p[,np])]. *Goodman and Kruskal's lambda (similarity).* This coefficient assesses the predictability of the state of a characteristic on one item (present or absent) given the state on the other item. Specifically, LAMBDA measures the proportional reduction in error, using one item to predict the other item when the directions of prediction are of equal importance. LAMBDA has a range of 0 to 1.

D[(p[,np])]. *Anderberg's* D *(similarity).* This coefficient assesses the predictability of the state of a characteristic on one item (present or absent) given the state on the other item. D measures the actual reduction in the error probability when one item is used to predict the other item. The range of D is 0 to 1.

Y[(p[,np])]. *Yule's* Y *coefficient of colligation (similarity).* This measure is a function of the cross ratio for a 2×2 table and has a range of −1 to +1.

Q[(p[,np])]. *Yule's* Q *(similarity).* This measure is the 2×2 version of Goodman and Kruskal's ordinal measure *gamma.* Like Yule's *Y*, Q is a function of the cross ratio for a 2×2 table and has a range of −1 to +1.

Other Binary Measures. The remaining binary measures that are available in PROXIMITIES are either binary equivalents of association measures for continuous variables or measures of special properties of the relationship between items.

OCHIAI[(p[,np])]. *Ochiai similarity measure.* This measure is the binary form of the cosine and has a range of 0 to 1.

SS5[(p[,np])]. *Sokal and Sneath similarity measure 5.* The range is 0 to 1.

PHI[(p[,np])]. *Fourfold point correlation (similarity).* This measure is the binary form of the Pearson product-moment correlation coefficient.

BEUCLID[(p[,np])]. *Binary Euclidean distance.* This measure is a distance measure. Its minimum value is 0, and it has no upper limit.

BSEUCLID[(p[,np])]. *Binary squared Euclidean distance.* This measure is a distance measure. Its minimum value is 0, and it has no upper limit.

SIZE[(p[,np])]. *Size difference.* This measure is a dissimilarity measure with a minimum value of 0 and no upper limit.

PATTERN[(p[,np])]. *Pattern difference.* This measure is a dissimilarity measure. The range is 0 to 1.

BSHAPE[(p[,np])]. *Binary shape difference.* This dissimilarity measure has no upper limit or lower limit.

DISPER[(p[,np])]. *Dispersion similarity measure.* The range is −1 to +1.

VARIANCE[(p[,np])]. *Variance dissimilarity measure.* This measure has a minimum value of 0 and no upper limit.

BLWMN[(p[,np])]. *Binary Lance-and-Williams nonmetric dissimilarity measure.* This measure is also known as the Bray-Curtis nonmetric coefficient. The range is 0 to 1.

Example

```
PROXIMITIES A B C
  /MEASURE=RR(1,2).
```

- MEASURE computes Russell and Rao coefficients from data in which 1 indicates the presence of a characteristic and 2 indicates the absence. Other values are ignored.

Example

```
PROXIMITIES A B C
  /MEASURE=SM(2).
```

- MEASURE computes simple matching coefficients from data in which 2 indicates presence and all other values indicate absence.

Transforming Measures in Proximity Matrix

Use keyword NONE to apply the ABSOLUTE, REVERSE, and/or RESCALE transformations to an existing matrix of proximity values without computing any proximity measures.

NONE. *Do not compute proximity measures.* Use NONE only if you have specified an existing proximity matrix on keyword IN on the MATRIX subcommand.

PRINT Subcommand

PROXIMITIES always prints the name of the measure that it computes and the number of cases. Use PRINT to control printing of the proximity matrix.

PROXIMITIES. *Print the matrix of the proximities between items.* This setting is the default. The matrix may have been either read or computed. When the number of cases or variables is large, this specification produces a large volume of output and uses significant CPU time.

NONE. *Do not print the matrix of proximities.*

ID Subcommand

By default, PROXIMITIES identifies cases by case number alone. Use ID to specify an identifying string variable for cases.

- Any string variable in the active dataset can be named as the identifier. PROXIMITIES uses the first eight characters of this variable to identify cases in the output.
- When used with the MATRIX IN subcommand, the variable that is specified on the ID subcommand identifies the labeling variable in the matrix file.

MISSING Subcommand

MISSING controls the treatment of cases with missing values.

- PROXIMITIES deletes cases with missing values listwise. By default, PROXIMITIES excludes user-missing values from the analysis.

EXCLUDE. *Exclude cases with user-missing values.* This setting is the default.

INCLUDE. *Include cases with user-missing values.* Only cases with system-missing values are deleted.

MATRIX Subcommand

MATRIX reads and writes matrix data files.

- Either IN or OUT and the matrix file in parentheses are required. When both IN and OUT are used on the same PROXIMITIES command, they can be specified on separate MATRIX subcommands or on the same subcommand.

OUT ('savfile'|'dataset'). *Write a matrix data file or dataset.* Specify either a filename, a previously declared dataset name, or an asterisk, enclosed in parentheses. Filenames should be enclosed in quotes and are stored in the working directory unless a path is included as part of the file specification. If you specify an asterisk (*), the matrix data file replaces the active dataset. If you specify an asterisk or a dataset name, the file is not stored on disk unless you use SAVE or XSAVE.

IN ('savfile'|'dataset'). *Read a matrix data file or dataset.* Specify either a filename, dataset name created during the current session, or an asterisk enclosed in parentheses. An asterisk reads the matrix data from the active dataset. Filenames should be enclosed in quotes and are read from the working directory unless a path is included as part of the file specification.

When a matrix is produced by using the MATRIX OUT subcommand, the matrix corresponds to a unique dataset. All subsequent analyses that are performed on this matrix would match the corresponding analysis on the original data. However, if the data file is altered in any way, this matching process would no longer occur.

For example, if the original file is edited or rearranged it would no longer correspond to the initially produced matrix. You need to make sure that the data match the matrix whenever inferring the results from the matrix analysis. Specifically, when the cluster membership is saved into an active dataset in the CLUSTER procedure, the proximity matrix in the MATRIX IN statement must match the current active dataset.

Matrix Output

- PROXIMITIES writes a variety of proximity matrices, each matrix with *ROWTYPE_* values of PROX. PROXIMITIES neither reads nor writes additional statistics with its matrix materials. See "Format of the Matrix Data File " on page 1564 for a description of the file.
- The matrices that PROXIMITIES writes can be used by PROXIMITIES or other procedures. Procedures CLUSTER and ALSCAL can read a proximity matrix directly. Procedure FACTOR can read a correlation matrix that is written by PROXIMITIES, but RECODE must first be used to change the *ROWTYPE_* value PROX to *ROWTYPE_* value CORR. Also, the ID subcommand cannot be used on PROXIMITIES if the matrix will be used in FACTOR.
- If VIEW=VARIABLE, the variables in the matrix file will have the names and labels of the original variables.
- If VIEW=CASE (the default), the variables in the matrix file are named *VAR1, VAR2, ...VAR n*, where *n* is the sequential number of the variable in the new file. The numeric suffix *n* is consecutive and does not necessarily match the number of the actual case. If there are no split files, the case number appears in the variable label in the form *CASE m*. The numeric suffix *m* is the actual case number and may not be consecutive (for example, if cases were selected before PROXIMITIES was executed).
- If VIEW=CASE, a numeric variable *CASENO_* is added to the matrix file. Values of *CASENO_* are the case numbers in the original file.
- The new file preserves the names and values of any split-file variables that are in effect. When split-file processing is in effect, no labels are generated for variables in the new file. The actual case number is retained by the variable *ID*.
- Any documents that are contained in the active dataset are not transferred to the matrix file.

Matrix Input

- PROXIMITIES can read a matrix file that is written by a previous PROXIMITIES procedure.
- Values for split-file variables should precede values for *ROWTYPE_*. *CASENO_* and the labeling variable (if present) should come after *ROWTYPE_* and before *VARNAME_*.
- If *CASENO_* is of type string rather than numeric, it is considered unavailable and a warning is issued.
- If *CASENO_* appears on a variable list, a syntax error results.
- PROXIMITIES ignores unrecognized *ROWTYPE_* values. In addition, PROXIMITIES ignores variables that are present in the matrix file that are not specified (or used by default) on the PROXIMITIES variable list.
- The program reads variable names, variable and value labels, and print and write formats from the dictionary of the matrix data file.
- MATRIX=IN cannot be used unless an active dataset has already been defined. To read an existing matrix data file at the beginning of a session, use GET to retrieve the matrix file, and then specify IN(*) on MATRIX.
- When you read a matrix that is created with MATRIX DATA, you should supply a value label for PROX of either *SIMILARITY* or *DISSIMILARITY* so that the matrix is correctly identified. If you do not supply a label, PROXIMITIES assumes *DISSIMILARITY*. See "Format of the Matrix Data File " on page 1564.

- The variable list on PROXIMITIES can be omitted when a matrix file is used as input. When the variable list is omitted, all variables in the matrix data file are used in the analysis. If a variable list is specified, the specified variables can be a subset of the variables in the matrix file.
- With a large number of variables, the matrix data file will wrap when displayed (as with LIST) and will be difficult to read. Nonetheless, the matrix values are accurate and can be used as matrix input.

Format of the Matrix Data File

- The matrix data file includes three special variables created by the program: *ROWTYPE_*, *VARNAME_*, and *CASENO_*. Variable *ROWTYPE_* is a short string variable with value PROX (for proximity measure). PROX is assigned value labels containing the distance measure that is used to create the matrix and either *SIMILARITY* or *DISSIMILARITY* as an identifier. Variable *VARNAME_* is a short string variable whose values are the names of the new variables. Variable *CASENO_* is a numeric variable with values equal to the original case numbers.
- The matrix file includes the string variable that is named on the ID subcommand. This variable is used to identify cases. Up to 20 characters can be displayed for the identifier variable; longer values are truncated. The identifier variable is present only when VIEW=CASE (the default) and when the ID subcommand is used.
- The remaining variables in the matrix file are the variables that are used to form the matrix.

Split Files

- When split-file processing is in effect, the first variables in the matrix system file are the split variables, followed by *ROWTYPE_*, the case-identifier variable (if VIEW=CASE and ID are used), *VARNAME_*, and the variables that form the matrix.
- A full set of matrix materials is written for each split-file group that is defined by the split variables.
- A split variable cannot have the same name as any other variable that is written to the matrix data file.
- If split-file processing is in effect when a matrix is written, the same split file must be in effect when that matrix is read by any procedure.

Example: Matrix Output to IBM SPSS Statistics External File

```
PROXIMITIES  V1 TO V20
  /MATRIX=OUT(DISTOUT).
```

- PROXIMITIES produces a default Euclidean distance matrix for cases by using variables *V1* through *V20* and saves the matrix in the IBM SPSS Statistics data file *DISTOUT*.
- The names of the variables on the matrix file will be *VAR1*, *VAR2, ...VARn*.

Example: Matrix Output to External File

```
GET FILE='/data/crime.sav'.
PROXIMITIES MURDER TO MOTOR
  /ID=CITY
  /MEASURE=EUCLID
  /MATRIX=OUT(PROXMTX).
```

- PROXIMITIES reads data from the IBM SPSS Statistics data file *crime.sav* and writes one set of matrix materials to file *PROXMTX*.
- The active dataset is still *crime.sav*. Subsequent commands are executed on this file.

Example: Matrix Output to Working File

```
GET FILE='/data/crime.sav'.
PROXIMITIES MURDER TO MOTOR
  /ID=CITY
  /MEASURE=EUCLID
  /MATRIX=OUT(*).
LIST.
```

- PROXIMITIES writes the same matrix as in the example above. However, the matrix data file replaces the active dataset. The LIST command is executed on the matrix file, not on the *crime.sav* file.

Example: Matrix Input from External File

```
GET FILE PRSNNL.
FREQUENCIES VARIABLE=AGE.

PROXIMITIES CASE1 TO CASE8
  /ID=CITY
  /MATRIX=IN(PROXMTX).
```

- This example performs a frequencies analysis on file *PRSNNL* and then uses a different file that contains matrix data for PROXIMITIES.
- MATRIX=IN specifies the matrix data file *PROXMTX*. *PROXMTX* does not replace *PRSNNL* as the active dataset.

Example: Matrix Input from Working File

```
GET FILE PROXMTX.
PROXIMITIES CASE1 TO CASE8
  /ID=CITY
  /MATRIX=IN(*).
```

- This example assumes that you are starting a new session and want to read an existing matrix data file. GET retrieves the matrix file *PROXMTX*.
- MATRIX=IN specifies an asterisk because the matrix data file is the active dataset. If MATRIX=IN(PROXMTX) is specified, the program issues an error message.
- If the GET command is omitted, the program issues an error message.

Example: Matrix Output to and Then Input from Working File

```
GET FILE='/data/crime.sav'.
PROXIMITIES MURDER TO MOTOR
  /ID=CITY
  /MATRIX=OUT(*).
PROXIMITIES
  /MATRIX=IN(*)
  /STANDARDIZE.
```

- GET retrieves the IBM SPSS Statistics data file *crime.sav*.
- The first PROXIMITIES command specifies variables for the analysis and reads data from file *crime.sav*. ID specifies *CITY* as the case identifier. MATRIX writes the resulting matrix to the active dataset.
- The second PROXIMITIES command uses the matrix file that is written by the first PROXIMITIES command as input. The asterisk indicates that the matrix file is the active dataset. The variable list is omitted, indicating that all variables in the matrix are to be used.
- The slash preceding the MATRIX subcommand on the second PROXIMITIES command is required. Without the slash, PROXIMITIES attempts to interpret MATRIX as a variable name rather than as a subcommand.

Example: Q-factor Analysis

In this example, PROXIMITIES and FACTOR are used for a *Q*-factor analysis, in which factors account for variance shared among observations rather than among variables. Procedure FACTOR does not perform *Q*-factor analysis without some preliminary transformation such as what is provided by PROXIMITIES. Because the number of cases exceeds the number of variables, the model is not of full rank, and FACTOR will print a warning. This result is a common occurrence when case-by-case matrices from PROXIMITIES are used as input to FACTOR.

```
* Recoding a PROXIMITIES matrix for procedure FACTOR.

GET FILE='/data/crime.sav'.
PROXIMITIES   MURDER TO MOTOR
  /MEASURE=CORR
  /MATRIX=OUT('/data/tempfile.sav').
GET FILE='/data/tempfile.sav' /DROP=ID.
RECODE ROWTYPE_ ('PROX' = 'CORR').
FACTOR MATRIX IN(COR=*).
```

- The MATRIX subcommand on PROXIMITIES writes the correlation matrix to the active dataset. Because the matrix materials will be used in procedure FACTOR, the ID subcommand is not specified.

- RECODE recodes *ROWTYPE_* value PROX to CORR so that procedure FACTOR can read the matrix.
- When FACTOR reads matrix materials, it reads all variables in the file. The MATRIX subcommand on FACTOR indicates that the matrix is a correlation matrix and that data are in the active dataset.

References

Anderberg, M. R. 1973. *Cluster analysis for applications*. New York: Academic Press.

Romesburg, H. C. 1984. *Cluster analysis for researchers*. Belmont, Calif.: Lifetime Learning Publications.

PROXSCAL

PROXSCAL is available in SPSS Statistics Professional Edition or the Categories option.

```
PROXSCAL varlist

[/TABLE = {rowid BY columnid [BY sourceid]}]
          {sourceid                       }

[/SHAPE = [{LOWER**}]]
           {UPPER  }
           {BOTH   }

[/INITIAL = [{SIMPLEX**        }]]
             {TORGERSON        }
             {RANDOM[({1})]    }
                     {n}
             {[('file'|'dataset')] [varlist] }

[/WEIGHTS = varlist]

[/CONDITION = [{MATRIX**      }]]
               {UNCONDITIONAL }

[/TRANSFORMATION = [{RATIO**                              }]]
                    {INTERVAL                             }
                    {ORDINAL[({UNTIE   })]                }
                             {KEEPTIES}
                    {SPLINE [DEGREE = {2}] [INKNOT = {1}]}
                                      {n}            {n}

[/PROXIMITIES = [{DISSIMILARITIES**}]]
                 {SIMILARITIES     }

[/MODEL = [{IDENTITY**    }]]
           {WEIGHTED      }
           {GENERALIZED   }
           {REDUCED[({2})]}
                     {n}

[/RESTRICTIONS = {COORDINATES('file'|'dataset') [{ALL    }]                                  }]
                                                {varlist}
                 {VARIABLES('file'|'dataset') [{ALL    }][({INTERVAL                      })]}
                                              {varlist}   {NOMINAL                        }
                                                          {ORDINAL[({UNTIE   })]          }
                                                                   {KEEPTIES}
                                                          {SPLINE[DEGREE={2}][INKNOT={1}]}
                                                                        {n}         {n}

[/ACCELERATION = NONE]

[/CRITERIA = [DIMENSIONS({2**      })]
                         {min[,max]}
             [MAXITER({100**})]
                      {n    }
             [DIFFSTRESS({0.0001**})]
                         {value   }
             [MINSTRESS({0.0001**}) ]]
                        {value   }

[/PRINT = [NONE][INPUT][RANDOM][HISTORY][STRESS**][DECOMPOSITION]
          [COMMON**][DISTANCES][WEIGHTS**][INDIVIDUAL]
          [TRANSFORMATIONS][VARIABLES**][CORRELATIONS**]]

[/PLOT = [NONE][STRESS][COMMON**][WEIGHTS**][CORRELATIONS**]
         [INDIVIDUAL({varlist})]
                     {ALL    }
         [TRANSFORMATIONS({varlist}) [({varlist})[...]] ]
                         {ALL    }   {ALL    }
         [RESIDUALS({varlist}) [({varlist})[...]] ]
                    {ALL    }   {ALL    }
         [VARIABLES({varlist})]]
                    {ALL    }

[/OUTFILE = [COMMON('file'|'dataset')] [WEIGHTS('file'|'dataset')] [DISTANCES('file'|'dataset')]
            [TRANSFORMATIONS('file'|'dataset')] [VARIABLES('file'|'dataset')] ]

[/MATRIX = IN('file'|'dataset')]].
```

** Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Overview

PROXSCAL performs multidimensional scaling of proximity data to find a least-squares representation of the objects in a low-dimensional space. Individual differences models are allowed for multiple sources. A

majorization algorithm guarantees monotone convergence for optionally transformed metric and nonmetric data under a variety of models and constraints.

Options

Data Input. You can read one or more square matrices of proximities that can either be symmetrical or asymmetrical. Alternatively, you can provide specifications with the TABLE subcommand for matrices with proximities in a stacked format. You can read proximity matrices created by PROXIMITIES and CLUSTER with the MATRIX subcommand. Additionally, you can read weights, initial configurations, fixed coordinates, and independent variables.

Methodological Assumptions. You can specify transformations considering all sources (unconditional) or separate transformations for each source (matrix-conditional) on the CONDITION subcommand. You can treat proximities as nonmetric (ordinal) or as metric (numerical or splines) using the TRANSFORMATION subcommand. Ordinal transformations can treat tied observations as tied (discrete) and untied (continuous). You can specify whether your proximities are similarities or dissimilarities on the PROXIMITIES subcommand.

Model Selection. You can specify multidimensional scaling models by selecting a combination of PROXSCAL subcommands, keywords, and criteria. The subcommand MODEL offers, besides the identity model, three individual differences models. You can specify other selections on the CRITERIA subcommand.

Constraints. You can specify fixed coordinates or independent variables to restrict the configuration(s) on the RESTRICTIONS subcommand. You can specify transformations (numerical, nominal, ordinal, and splines) for the independent variables on the same subcommand.

Output. You can produce output that includes the original and transformed proximities, history of iterations, common and individual configurations, individual space weights, distances, and decomposition of the stress. Plots can be produced of common and individual configurations, individual space weights, transformations, and residuals.

Basic Specification

The basic specification is PROXSCAL followed by a variable list. By default, PROXSCAL produces a two-dimensional metric Euclidean multidimensional scaling solution (identity model). Input is expected to contain one or more square matrices with proximities that are dissimilarities. The ratio transformation of the proximities is matrix-conditional. The analysis uses a simplex start as an initial configuration. By default, output includes fit and stress values, the coordinates of the common space, and a chart of the common space configuration.

Syntax Rules

- The number of dimensions (both minimum and maximum) may not exceed the number of proximities minus one.
- Dimensionality reduction is omitted if combined with multiple random starts.
- If there is only one source, then the model is always assumed to be identity.

Limitations

- PROXSCAL needs at least three objects, which means that at least three variables must be specified in the variable list. In the case of the TABLE subcommand, the minimum value for `rowid` and `columnid` must be at least three.
- PROXSCAL recognizes data weights created by the WEIGHT command but only in combination with the TABLE subcommand.
- Split-file has no implications for PROXSCAL.

Variable List Subcommand

The variable list identifies the columns in the proximity matrix or matrices that PROXSCAL reads. Each variable identifies one column of the proximity matrix, with each case in the active dataset representing

one row, unless specified otherwise with the TABLE subcommand. In this case, the variable list identifies whole matrices or sources.

- Only numeric variables can be specified.
- The total number of cases must be divisible by the number of variables. This is not applicable when the TABLE subcommand is used.
- PROXSCAL reads data row by row; the columns are represented by the variables on the variable list. The order of the variables on the list is crucial.

Example

```
DATA LIST
  /object01 object02 object03 object04.

BEGIN DATA
  0 2 6 3
  2 0 5 4
  6 5 0 1
  3 4 1 0
END DATA.

PROXSCAL VARIABLES=object01 TO object04.
```

- This example specifies an analysis on a 4×4 proximity matrix.
- The total number of cases must be divisible by 4.

TABLE Subcommand

The TABLE subcommand specifies the row identifier `rowid` and the column identifier `columnid`. Using TABLE, the proximities of separate sources are given in separate variables on the PROXSCAL variable list.

In the same manner, sources are identified by `sourceid`. In combination with `rowid` and `columnid`, the proximities are stacked in one single variable, containing the proximities of all sources, where sources are distinguished by the values of `sourceid`.

Using `sourceid` as the only variable on the TABLE subcommand indicates the use of stacked matrices, where individual stacked matrices are recognized by different values of `sourceid`.

- Rowid, `columnid`, and `sourceid` should not be specified on the variable list.
- When specifying both upper- and lower-triangular parts of the matrix, the SHAPE subcommand will determine the handling of the data.
- If a cell's value is specified multiple times, the final specification is used.
- Rowid, `columnid`, and `sourceid` must appear in that order.
- Omitting `sourceid` causes PROXSCAL to use the sources specified on the PROXSCAL variable list. Each variable is assumed to contain the proximities of one source.
- Specifying multiple sources on the PROXSCAL variable list in conjunction with specifying `rowid`, `columnid`, and `sourceid` is not possible and causes PROXSCAL to ignore `sourceid`.

rowid. *Row identifying variable.* The values of this variable specify the row object of a proximity. The values must be integers between 1 and the number of objects, inclusive.

columnid. *Column identifying variable.* The values specify the column object of a proximity. The values must be integers between 1 and the number of objects, inclusive.

sourceid. *Source identifying variable.* The values specify the source number and must be integers between 1 and the number of sources, inclusive. The value labels of this variable are used to identify sources on other subcommands. These value labels must comply with variable name conventions. Omitting a value label causes PROXSCAL to use the default label *SRC_n*, where *n* is the number of the source.

Example

```
DATA LIST
  /r_id c_id men women.


BEGIN DATA
2 1 1.08 1.14
3 1 0.68 1.12
3 2 0.95 0.75
4 1 0.96 0.32
4 2 0.76 0.98
4 3 0.47 0.69
. . .... ....
.. .. .... ....
13 10 0.55 0.86
13 11 0.61 0.97
13 12 0.46 0.83
END DATA.


PROXSCAL men women
  /TABLE=r_id BY c_id
  /PLOT = INDIVIDUAL (women).
```

- PROXSCAL reads two proximity matrices (*men* and *women*), where the row objects are specified by *r_id* and the column objects by *c_id*.
- A chart of the individual space for *women* is plotted.

This is one way to proceed. Another way is to add the proximities of the additional source below the proximities of the first source and specify `sourceid` on the TABLE subcommand, containing values distinguishing the first and the additional source (see the next example).

Example

```
DATA LIST
  /r_id c_id s_id prox.


BEGIN DATA
2 1 1 1.08
3 1 1 0.68
3 2 1 0.95
4 1 1 0.96
4 2 1 0.76
4 3 1 0.47
. . . ....
.. .. . ....
13 10 1 0.55
13 11 1 0.61
13 12 1 0.46
2 1 2 1.14
3 1 2 1.12
3 2 2 0.75
4 1 2 0.32
4 2 2 0.98
4 3 2 0.69
. . . ....
.. .. . ....
13 10 2 0.86
13 11 2 0.97
13 12 2 0.83
END DATA.


VALUE LABELS s_id 1 'men' 2 'women'.


PROXSCAL prox
  /TABLE=r_id BY c_id BY s_id
  /PLOT = INDIVIDUAL (women).
```

- PROXSCAL reads two proximity matrices. The row objects are identified by *r_id* and the column objects, by *c_id*. The proximity matrices are gathered in one variable, *source01*, where each source is distinguished by a value of the source identifying variable *s_id*.
- A chart of the individual space for *women* is plotted.

Example

```
DATA LIST LIST
  /obj_1 obj_2 obj_3 obj_4 s_id


BEGIN DATA
0  0  0  0 1
1  0  0  0 1
2  3  0  0 1
4  5  6  0 1
7  0  0  0 2
0  0  0  0 2
8  9  0  0 2
12 11 12 0 2
```

```
END DATA.

VALUE LABELS s_id 1 'women' 2 'men'.

PROXSCAL obj_1 obj_2 obj_3 obj_4
  /TABLE = s_id
  /MODEL = WEIGHTED
  /PLOT = INDIVIDUAL (women).
```

- PROXSCAL reads two proximity matrices. The objects are given on the PROXSCAL variable list. Each source is distinguished by a value of the source identifying variable *s_id*, which is also used for labeling.
- A chart of the individual space for *women* is plotted.

SHAPE Subcommand

The SHAPE subcommand specifies the structure of the proximity matrix.

LOWER. *Lower-triangular data matrix.* For a lower-triangular matrix, PROXSCAL expects a square matrix of proximities of which the lower-triangular elements are used under the assumption that the full matrix is symmetric. The diagonal is ignored but must be included.

UPPER. *Upper-triangular data matrix.* For an upper-triangular matrix, PROXSCAL expects a square matrix of proximities of which the upper-triangular elements are used under the assumption that the full matrix is symmetric. The diagonal is ignored but must be included.

BOTH. *Full data matrix.* The values in the corresponding cells in the upper and lower triangles may be different. PROXSCAL reads the complete square matrix and, after obtaining symmetry, continues with the lower-triangular elements. The diagonal is ignored but must be included.

- System or other missing values on the (virtual) diagonal are ignored.

Example

```
PROXSCAL object01 TO object07
  /SHAPE=UPPER.
```

- PROXSCAL reads square matrices of seven columns per matrix of which the upper-triangular parts are used in computations.
- Although specified, the diagonal and lower-triangular part of the matrix are not used.

INITIAL Subcommand

INITIAL defines the initial or starting configuration of the common space for the analysis. When a reduction in dimensionality is specified on the CRITERIA subcommand, a derivation of coordinates in the higher dimensionality is used as a starting configuration in the lower dimensionality.

- You can specify one of the three keywords listed below.
- You can specify a variable list containing the initial configuration.

SIMPLEX. *Simplex start.* This specification is the default. PROXSCAL starts by placing the objects in the configuration all at the same distance of each other and taking one iteration to improve this high-dimensional configuration, followed by a dimension-reduction operation to obtain the user-provided maximum dimensionality specified in the CRITERIA subcommand with the keyword DIMENSIONS.

TORGERSON. *Torgerson start.* A classical scaling solution is used as initial configuration.

RANDOM. *(Multiple) random start.* You can specify the number of random starts (*n*). *n* is any positive integer. The random sequence can be controlled by the RANDOM SEED command and not by a subcommand within the PROXSCAL command. Each analysis starts with a different random configuration. In the output, all *n* final stress values are reported, as well as the initial seeds of each analysis (for reproduction purposes), followed by the full output of the analysis with the lowest stress value. The default number of random starts is 1. Reduction of dimensionality—that is, using a maximum

dimensionality that is larger than the minimum dimensionality—is not allowed within this option and the minimum dimensionality is used, if reduction is specified anyway.

Instead of these keywords, a parenthesized IBM SPSS Statistics data file can be specified containing the coordinates of the initial configuration. If the variable list is omitted, the first MAXDIM variables are automatically selected, where MAXDIM is the maximum number of dimensions requested for the analysis on the CRITERIA subcommand. Only nonmissing values are allowed as initial coordinates.

Example

```
PROXSCAL object01 TO object17
  /INITIAL=RANDOM(100).
```

- This example performs 100 analyses each, starting with different random configurations. The results of the analysis with the lowest final stress are displayed in the output.

WEIGHTS Subcommand

The WEIGHTS subcommand specifies non-negative weights on the proximities included in the active dataset.

- The number and order of the variables in the variable list is important. The first variable on the WEIGHTS variable list corresponds to the first variable on the PROXSCAL variable list. This is repeated for all variables on the variable lists. Every proximity has its own weight. The number of variables on the WEIGHTS subcommand must therefore be equal to the number of variables on the PROXSCAL variable list.
- Negative weights are not allowed. If specified, a warning will be issued and the procedure will abort.

Example

```
DATA LIST FILE='cola.dat' FREE
  /object01 TO object14 weight01 TO weight14.


PROXSCAL object01 TO object14
  /WEIGHTS=weight01 TO weight14.
```

- In this example, the VARIABLES subcommand indicates that there are 14 columns per matrix of which the weights can be found in *weight01* to *weight14*.
- *weight01* contains the weights for *object01*, etc.

CONDITION Subcommand

CONDITION specifies how transformations among sources are compared. The TRANSFORMATION subcommand specifies the type of transformation.

MATRIX. *Matrix conditional.* Only the proximities within each source are compared with each other. This is the default.

UNCONDITIONAL. *Unconditional.* This specification is appropriate when the proximities in all sources can be compared with each other and result in a single transformation of all sources simultaneously.

- Note that if there is only one source, then MATRIX and UNCONDITIONAL give the same results.

Example

```
PROXSCAL object01 TO object15
  /CONDITION=UNCONDITIONAL
  /TRANSFORMATION=ORDINAL(UNTIE).
```

- In this example, the proximities are ordinally transformed, where tied proximities are allowed to be untied. The transformations are performed simultaneously over all possible sources.

TRANSFORMATION Subcommand

TRANSFORMATION offers four different options for optimal transformation of the original proximities. The resulting values are called transformed proximities. The distances between the objects in the configuration should match these transformed proximities as closely as possible.

RATIO. *No transformation.* Omitting the entire subcommand is equivalent to using this keyword. In both cases, the transformed proximities are proportional to the original proximities. This “transformation” is only allowed for positive dissimilarities. In all other cases, a warning is issued and the transformation is set to INTERVAL.

INTERVAL. *Numerical transformation.* In this case, the transformed proximities are proportional to the original proximities, including free estimation of the intercept. The inclusion of the intercept assures that all transformed proximities are positive.

ORDINAL. *Ordinal transformation.* The transformed proximities have the same order as the original proximities. In parentheses, the approach to tied proximities can be specified. Keeping tied proximities tied, also known as secondary approach to ties, is default. Specification may be implicit, ORDINAL, or explicit, ORDINAL(KEEPTIES). Allowing tied proximities to be untied, also known as the primary approach to ties, is specified as ORDINAL (UNTIE).

SPLINE. *Monotone spline transformation.* The transformed proximities are a smooth nondecreasing piecewise polynomial transformation of the original proximities of the chosen degree. The pieces are specified by the number and placement of the interior knots.

SPLINE Keyword

SPLINE has the following keywords:

DEGREE. *The degree of the polynomial.* If DEGREE is not specified, the degree is assumed to be 2. The range of DEGREE is between 1 and 3 (inclusive).

INKNOT. *The number of interior knots.* If INKNOT is not specified, the number of interior knots is assumed to be 1. The range of INKNOT is between 1 and the number of different proximities.

Example

```
PROXSCAL object01 TO object05
  /TRANSFORMATION=ORDINAL(UNTIE).
```

- In this example, the proximities are ordinally transformed, where tied proximities are allowed to be untied.
- The default conditionality (MATRIX) implies that the transformation is performed for each source separately.

PROXIMITIES Subcommand

The PROXIMITIES subcommand specifies the type of proximities used in the analysis. The term proximity is used for either similarity or dissimilarity data.

DISSIMILARITIES. *Dissimilarity data.* This specification is the default when PROXIMITIES is not specified. Small dissimilarities correspond to small distances, and large dissimilarities correspond to large distances.

SIMILARITIES. *Similarity data.* Small similarities correspond to large distances and large similarities correspond to small distances.

Example

```
PROXSCAL object01 TO object12
  /PROXIMITIES=SIMILARITIES.
```

- In this example, PROXSCAL expects the proximities to be similarities.

MODEL Subcommand

MODEL defines the scaling model for the analysis if more than one source is present. IDENTITY is the default model. The three other models are individual differences models.

IDENTITY. *Identity model.* All sources have the same configuration. This is the default model, and it is not an individual differences model.

WEIGHTED. *Weighted Euclidean model.* This model is an individual differences model and equivalent to the INDSCAL model in the ALSCAL procedure. Each source has an individual space, in which every dimension of the common space is weighted differentially.

GENERALIZED. *Generalized Euclidean model.* This model is equivalent to the GEMSCAL model in the ALSCAL procedure. Each source has an individual space that is equal to a rotation of the common space, followed by a differential weighting of the dimensions.

REDUCED. *Reduced rank model.* This model is similar to GENERALIZED, but the rank of the individual space is equal to n. This number is always smaller than the maximum number of dimensions and equal to or greater than 1. The default is 2.

- If IDENTITY is specified for only one source, this subcommand is silently ignored.
- If an individual differences model is specified for only one source, a warning is issued, and the model is set to IDENTITY.

Example

```
PROXSCAL object01 TO object07
  /MODEL=WEIGHTED.
```

- A weighted Euclidean model is fitted, but only when the number of cases in the active dataset is a multiple of 7, starting from 14 (14, 21, 28, and so on). Otherwise, there is only one source, and the model is set to IDENTITY.

RESTRICTIONS Subcommand

PROXSCAL provides two types of restrictions for the user to choose from. The first type fixes (some) coordinates in the configuration. The second type specifies that the common space is a weighted sum of independent variables.

COORDINATES. *Fixed coordinates.* A parenthesized IBM SPSS Statistics data filename must be specified containing the fixed coordinates for the common space. A variable list may be given, if some specific variables need to be selected from the external file. If the variable list is omitted, the procedure automatically selects the first MAXDIM variables in the external file, where MAXDIM is the maximum number of dimensions requested for the analysis on the CRITERIA subcommand. A missing value indicates that a coordinate on a dimension is free. The coordinates of objects with nonmissing values are kept fixed during the analysis. The number of cases for each variable must be equal to the number of objects.

VARIABLES. *Independent variables.* The common space is restricted to be a linear combination of the independent variables in the variable list. A parenthesized IBM SPSS Statistics data file must be specified containing the independent variables. If the variable list is omitted, the procedure automatically selects all variables in the external file. Instead of the variable list, the user may specify the keyword FIRST(n), where n is a positive integer, to select the first *n* variables in the external file. The number of cases for each variable must be equal to the number of objects. After the variable selection specification, we may provide a list of keywords (in number equal to the number of the independent variables) indicating the transformations for the independent variables.

VARIABLES Keyword

The following keywords may be specified:

INTERVAL. *Numerical transformation.* In this case, the transformed values of a variable are proportional to the original values of the variable, including free estimation of the intercept.

NOMINAL. *Nominal transformation.* The values are treated as unordered. The same values will obtain the same transformed values.

ORDINAL. *Ordinal transformation.* The values of the transformed variable have the same order as the values of the original variable. In parenthesis, the approach to tied values can be specified. Keeping tied values tied, also known as secondary approach to ties, is default. Specification may be implicit, `ORDINAL`, or explicit, `ORDINAL(KEEPTIES)`. Allowing tied values to be untied, also known as the primary approach to ties, is specified as `ORDINAL (UNTIE)`.

SPLINE. *Monotone spline transformation.* The transformed values of the variable are a smooth nondecreasing piecewise polynomial transformation of the original values of the chosen degree. The pieces are specified by the number and placement of the interior knots.

SPLINE Keyword

`SPLINE` has the following keywords:

DEGREE. *The degree of the polynomial.* If `DEGREE` is not specified, the degree is assumed to be 2. The range of `DEGREE` is between 1 and 3 (inclusive).

INKNOT. *The number of interior knots.* If `INKNOT` is not specified, the number of interior knots is assumed to be 1. The range of `INKNOT` is between 0 and the number of different values of the variable.

Example

```
PROXSCAL aunt TO uncle
  /RESTRICTIONS=VARIABLES(ivars.sav) degree generation gender
                         (ORDINAL ORDINAL NOMINAL).
```

- In this example, there are three independent variables specified: *degree*, *generation*, and *gender*.
- The variables are specified in the data file *ivars.sav*.
- On both *degree* and *generation*, ordinal transformations are allowed. By default, tied values in ordinal variables are kept tied. *Gender* is allowed to be nominally transformed.

ACCELERATION Subcommand

By default, a fast majorization method is used to minimize stress.

NONE. *The standard majorization update.* This turns off the fast method.

- If the subcommand `RESTRICTION` is used with fixed coordinates or independent variables, `ACCELERATION=NONE` is in effect.
- If an individual differences model is specified on the `MODEL` subcommand, `ACCELERATION=NONE` is in effect.

Example

```
PROXSCAL VARIABLES=object01 TO object12
  /ACCELERATION=NONE.
```

- Here, relaxed updates are switched off through the specification of the keyword `NONE` after `ACCELERATION`.

CRITERIA Subcommand

Use CRITERIA to set the dimensionality and criteria for terminating the algorithm, or minimization process. You can specify one or more of the following keywords:

DIMENSIONS. *Minimum and maximum number of dimensions.* By default, PROXSCAL computes a solution in two dimensions (min=2 and max=2). The minimum and maximum number of dimensions can be any integers inclusively between 1 and the number of objects minus 1, as long as the minimum is less than or equal to the maximum. PROXSCAL starts computing a solution in the largest dimensionality and reduces the dimensionality in steps, until the lowest dimensionality is reached. Specifying a single value represents both minimum and maximum number of dimensions, thus DIMENSIONS(4) is equivalent to DIMENSIONS(4,4).

MAXITER. *Maximum number of iterations.* By default, *n*=100, specifying the maximum number of iterations that is performed while one of the convergence criterion below (CONVERGENCE and STRESSMIN) is not yet reached. Decreasing this number might give less accurate results but will take less time. *N* must have a positive integer value.

DIFFSTRESS. *Convergence criterion.* PROXSCAL minimizes the goodness-of-fit index normalized raw stress. By default, PROXSCAL stops iterating when the difference in consecutive stress values is less than 0.0001 (*n*=0.0001). To obtain a more precise solution, you can specify a smaller value. The value specified must lie between 0.0 and 1.0, inclusively.

MINSTRESS. *Minimum stress value.* By default, PROXSCAL stops iterating when the stress value itself is small, that is, less than 0.0001 (*n*=0.0001). To obtain an even more precise solution, you can specify a smaller value. The value specified must lie between 0.0 and 1.0, inclusively.

Example

```
PROXSCAL VARIABLES=object01 TO object24
  /CRITERIA=DIMENSIONS(2,4) MAXITER(200) DIFFSTRESS(0.00001).
```

- The maximum number of dimensions equals 4 and the minimum number of dimensions equals 2. PROXSCAL computes a four-, three-, and two-dimensional solution, respectively.
- The maximum number of iteration is raised to 200.
- The convergence criterion is sharpened to 0.00001.

PRINT Subcommand

PRINT specifies the optional output. By default, PROXSCAL displays the stress and fit values for each analysis, the coordinates of the common space, and, with appropriate specification on corresponding subcommands, the individual space weights and transformed independent variables, corresponding regression weights, and correlations.

- Omitting the PRINT subcommand or specifying PRINT without keywords is equivalent to specifying COMMON, WEIGHTS, and VARIABLES.
- If a keyword(s) is specified, only the output for that particular keyword(s) is displayed.
- In the case of duplicate or contradicting keyword specification, the last keyword applies.
- Inapplicable keywords are silently ignored. That is, specifying a keyword for which no output is available (for example, specifying INDIVIDUAL with only one source) will silently ignore this keyword.

NONE. *No output.* Display only the normalized raw stress and corresponding fit values.

INPUT. *Input data.* The display includes the original proximities, and, if present, the data weights, the initial configuration, and the fixed coordinates or the independent variables.

RANDOM. *Multiple random starts.* Displays the random number seed and stress value of each random start.

HISTORY. *History of iterations.* Displays the history of iterations of the main algorithm.

STRESS. *Stress measures.* Displays different stress values. The table contains values for normalized raw stress, Stress-I, Stress-II, S-Stress, dispersion accounted for (D.A.F.), and Tucker's coefficient of congruence. This is specified by default.

DECOMPOSITION. *Decomposition of stress.* Displays an object and source decomposition of stress, including row and column totals.

COMMON. *Common space.* Displays the coordinates of the common space. This is specified by default.

DISTANCES. *Distances.* Displays the distances between the objects in the configuration.

WEIGHTS. *Individual space weights.* Displays the individual space weights, only if one of the individual differences models is specified on the MODEL subcommand. Depending on the model, the space weights are decomposed in rotation weights and dimension weights, which are also displayed. This is specified by default.

INDIVIDUAL. *Individual spaces.* The coordinates of the individual spaces are displayed, only if one of the individual differences models is specified on the MODEL subcommand.

TRANSFORMATION. *Transformed proximities.* Displays the transformed proximities between the objects in the configuration.

VARIABLES. *Independent variables.* If VARIABLES was specified on the RESTRICTIONS subcommand, this keyword triggers the display of the transformed independent variables and the corresponding regression weights. This is specified by default.

CORRELATIONS. *Correlations.* The correlations between the independent variables and the dimensions of the common space are displayed. This is specified by default.

Example

```
PROXSCAL VARIABLES=source01 TO source02
  /TABLE=row_id BY col_id
  /MODEL=WEIGHTED
  /PRINT=HISTORY COMMON STRESS.
```

- Here, a weighted Euclidean model is specified with two sources.
- The output consists of the history of iterations of the main algorithm, the coordinates of the common space, the individual space weights, and several measures of fit.

PLOT Subcommand

PLOT controls the display of plots. By default, PROXSCAL produces a scatterplot of object coordinates of the common space, the individual space weights, and the correlations between the independent variables (that is, equivalent to specifying COMMON, WEIGHTS, and CORRELATIONS).

- Specifying a keyword overrides the default output and only output is generated for that keyword.
- Duplicate keywords are silently ignored.
- In case of contradicting keywords, only the last keyword is considered.
- Inapplicable keywords (for example, stress with equal minimum and maximum number of dimensions on the CRITERIA subcommand) are silently ignored.
- Multiple variable lists are allowed for TRANSFORMATIONS and RESIDUALS. For each variable list, a separate plot will be displayed.

NONE. *No plots.* PROXSCAL does not produce any plots.

STRESS. *Stress plot.* A plot is produced of stress versus dimensions. This plot is only produced if the maximum number of dimensions is larger than the minimum number of dimensions.

COMMON. *Common space.* A scatterplot matrix of coordinates of the common space is displayed.

WEIGHTS. *Individual space weights.* A scatterplot is produced of the individual space weights. This is only possible if one of the individual differences models is specified on the MODEL subcommand. For the weighted Euclidean model, the weights are printed in plots with one dimension on each axis. For the

generalized Euclidean model, one plot is produced per dimension, indicating both rotation and weighting of that dimension. The reduced rank model produces the same plot as the generalized Euclidean model does but reduces the number of dimensions for the individual spaces.

INDIVIDUAL. *Individual spaces.* For each source specified on the variable list, the coordinates of the individual spaces are displayed in scatterplot matrices. This is only possible if one of the individual differences models is specified on the MODEL subcommand.

TRANSFORMATIONS. *Transformation plots.* Plots are produced of the original proximities versus the transformed proximities. On the variable list, the sources can be specified of which the plot is to be produced.

RESIDUALS. *Residuals plots.* The transformed proximities versus the distances are plotted. On the variable list, the sources can be specified of which the plot is to be produced.

VARIABLES. *Independent variables.* Transformation plots are produced for the independent variables specified on the variable list.

CORRELATIONS. *Correlations.* A plot of correlations between the independent variables and the dimensions of the common space is displayed.

Example

```
PROXSCAL VARIABLES=source01 TO source02
  /TABLE=row_id BY col_id
  /MODEL=WEIGHTED
  /CRITERIA=DIMENSIONS(3)
  /PLOT=COMMON INDIVIDUAL(source02).
```

- Here, the syntax specifies a weighted Euclidean model with two sources in three dimensions.
- COMMON produces a scatterplot matrix defined by dimensions 1, 2, and 3.
- For the individual spaces, a scatterplot matrix with 3 dimensions is only produced for the individual space of *source02*.

OUTFILE Subcommand

OUTFILE saves coordinates of the common space, individual space weights, distances, transformed proximities, and transformed independent variables to an external IBM SPSS Statistics data file or previously declared dataset. The only specification required is a name for the output file.

COMMON. *Common space coordinates.* The coordinates of the common space are written to an external IBM SPSS Statistics data file or previously declared dataset. The columns (variables) represent the dimensions *DIM_1*, *DIM_2*, ..., *DIM_n* of the common space. The number of cases (rows) in the IBM SPSS Statistics data file equals the number of objects.

WEIGHTS. *Individual space weights.* The individual space weights are written to an external IBM SPSS Statistics data file or previously declared dataset. The columns represent the dimensions *DIM_1*, *DIM_2*, ..., *DIM_n* of the space weights. The number of cases depends on the individual differences model specified on the MODEL subcommand. The weighted Euclidean model uses diagonal weight matrices. Only the diagonals are written to file and the number of cases is equal to the number of dimensions. The generalized Euclidean model uses full-rank nonsingular weight matrices. The matrices are written to the data file row by row. The reduced rank model writes matrices to the data file in the same way as the generalized Euclidean model does but does not write the reduced part.

DISTANCES. *Distances.* The matrices containing the distances for each source are stacked beneath each other and written to an external IBM SPSS Statistics data file or previously declared dataset. The number of variables in the data file are equal to the number of objects (*OBJ_1*, *OBJ_2*, ..., *OBJ_n*) and the number of cases in the data file are equal to the number of objects times the number of sources.

TRANSFORMATION. *Transformed proximities.* The matrices containing the transformed proximities for each source are stacked beneath each other and written to an external IBM SPSS Statistics data file or previously declared dataset. The number of variables in the file are equal to the number of objects

(*OBJ_1*, *OBJ_2*, ..., *OBJ_n*) and the number of cases in the data file are equal to the number of objects times the number of sources.

VARIABLES. *Independent variables.* The transformed independent variables are written to an external IBM SPSS Statistics data file or previously declared dataset. The variables are written to the columns (*VAR_1*, *VAR_2*, ..., *VAR_n*). The number of variables in the data file are equal to the number of independent variables and the number of cases are equal to the number of objects.

Example

```
PROXSCAL VARIABLES=source01 TO source04
  /TABLE=row_id BY col_id
  /OUTFILE=COMMON('/data/start.sav').
```

- Here, the coordinates of the common space are written to the IBM SPSS Statistics data file *start.sav*.
- Filenames should be enclosed in quotes and are stored in the working directory unless a path is included as part of the file specification. datasets are available during the current session but are not available in subsequent sessions unless you explicitly save them as data files.

MATRIX Subcommand

MATRIX reads matrix data files. It can read a matrix written by either PROXIMITIES or CLUSTER.

- The specification on MATRIX is the keyword IN and the matrix file in parentheses.
- Generally, data read by PROXSCAL are already in matrix form, whether in square format, or in stacked format using the TABLE subcommand.
- The proximity matrices PROXSCAL reads have ROWTYPE_ values of PROX.
- Using MATRIX=IN, PROXSCAL will ignore variables specified on the main variable list. All numerical variables from the matrix data file are processed.
- PROXSCAL ignores variables specified in the WEIGHTS subcommand in combination with the use of MATRIX=IN.
- With MATRIX=IN, only a source identifying variable can be specified on the TABLE subcommand. The sources are created as a result of a split file action.

IN('file'|'dataset'). *Read a matrix data file.* Specify a quoted file specification or previously declared dataset name, enclosed in parentheses. Data read through the MATRIX subcommand does not replace the active dataset.

Example

```
GET FILE = '/data/proxmtx.SAV'.

PROXSCAL
  /MATRIX=IN('/data/matrix.sav').
```

- MATRIX=IN specifies an external matrix data file called *matrix.sav*, of which all numerical variables are used for the current analysis.

QUANTILE REGRESSION

QUANTILE REGRESSION is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
QUANTILE REGRESSION dependent_varname [BY factor_list] [WITH covariate_list]
[ /MISSING
     [ CLASSMISSING = {EXCLUDE**} {INCLUDE}]]
[ /CRITERIA
     [ QUANTILE = {0.5**} {[values] [value1 TO value2 BY value3]} ]   @@
     [ IID = {TRUE**} {FALSE} ]
     [ BANDWIDTH = {BOFINGER**} {HALL_SHEATHER} ]
     [ CILEVEL = {95**} {value} ]
     [ METHOD = {AUTO**} {SIMPLEX} {INTERIOR_POINT} ]
     [ TOL = {1E-12**} {value} ]   ##
     [ CONV = {1E-6**} {value}]   $$
     [ MAXITER = {2000**} {integer} ] ]
[ /MODEL
     [ effect_1 effect_2 ... ]
     [ REGWGT = weight_varname ]   %%
     [ INTERCEPT = {TRUE**} {FALSE} ] ]  ^^
[ /SAVE
     [ PRED([ {varname} {rootname}) ] ]
     [ RESID( {varname} {rootname}) ]
     [ CIPREDL({varname} {rootname}) ]
     [ CIPREDU({varname} {rootname}) ] ]
[ /PREDICT_EFFECTS
     [ NUM_TOP_EFFECTS = {3**} {integer}] ]
[ /PREDICTION_TABLES ]
     [ effect_1 effect_2 ... ]   &&
[ /PREDICTION_LINES ]
     [ effect_1 effect_2 ... ]   ⊕
[ /PRINT
     [ PARAMETER** ]
     [ COVB ]
     [ CORB ] ]
[ /PLOT
     [ MAX_CATEGORIES = {10**} {integer} ]   ⊖
     [ PREDICTED_BY_OBSERVED = {FALSE**} {TRUE} ]
     [ PARAM_EST = {50**} {integer} {ALL} ] ]
[ /OUTFILE
     [ COVB = 'savfile' | 'dataset' ]
     [ CORB = 'savfile' | 'dataset' ]
     [ FILE_SEPARATE = {FALSE**} {TRUE} ]
     [ {MODEL = 'filename'} {PARAMETER = 'filename'} ] ]
[ /EXTERNAL ]
```

** Default if the subcommand or keyword is omitted.

@@ Use unique values ∈ [0,1]

For the matrix manipulations

$$ For the numerical method

%% Must be a numeric variable

^^ Either intercept or at least one predictor

&& A subset of the effects contained in the model

⊕ A subset of the effects contained in the model

⊖ Maximum categories plotted for a mixed effect

Release History

Release 26.0

- Command introduced

Release 26.0 Fix Pack 1

- QUANTILE keyword support for grid of quantiles (TO and BY)

Overview

`QUANTILE REGRESSION dependent_varname [BY factor_list] [WITH covariate_list]` is a required command to invoke the quantile regression procedure. A single numeric dependent variable is

required. BY and WITH are followed by an optional list of factors and covariates, respectively. Note that either the intercept term or at least one predictor is required to run the analysis.

Options

Analysis options
You can specify the regression percentile, the estimation method, numerical method settings, post-estimation attributes, and the significance level for computing credible intervals.

Model design
You can specify the model design for the analysis, which includes target variables, factor variables, covariates, regression weights, and model effects.

Display settings
You can specify which estimates to output, as well as plot and tabulation output options for the effects, prediction lines and tables.

Save settings
You can specify to save the predicted value of responses, residuals, and the lower/upper bound of prediction intervals.

Export options
You can specify to export the covariance and correlation matrices for parameter estimates, and specify a target file to generate the PMML file.

Basic specification

- `QUANTILE REGRESSION dependent_varname [BY factor_list] [WITH covariate_list]` is a required command to invoke the quantile regression procedure.

Subcommand order

- The subcommands can be named in any order.

Syntax rules

- In cases where duplicated values are specified, only unique values are used, and a warning message is generated to indicate that all duplicated values are excluded from the analysis.
- Only one regression weight variable is allowed.
- The specified **FACTORS** must be a subset of **factor_list**.
- The specified **COVARIATES** must be a subset of **covariate_list**.
- **BAYESPRED** can be **TRUE** if values are specified for **REGRESSORS**.
- **VARIABLES** must be a subset of **COVARIATES** and **FACTORS**.
- **INTERCEPT** must be **TRUE** when **VARIABLES** is empty.

CRITERIA Subcommand

CRITERIA controls the statistical criteria used in building the Bayesian Linear Regression models. The default criteria are noted in the description of each keyword below.

QUANTILE = value(s). *Optional keyword that specifies the quantile(s)* The single value `0.5` is the default setting. Users are allowed to specify multiple values with each value separated by a blank space (or blank spaces). Users are also allowed to specify a grid of quantiles from `value1` to `value2` with the increment of `value3`, connected by the keywords **TO** and **BY**. When a grid of quantiles is specified, only one valid set of `[value1 TO value2 BY value3]` is allowed. It must satisfy that $0 \leq value1 \leq value2 \leq 1$. In cases where $value1 = value2$, it is equivalent to specifying a single `value1`, regardless of `value3`. In order to restrict the number of quantiles, $0.01 \leq value3 \leq 1$. The grid can be mixed with other quantiles, and can be placed anywhere.

Notes:

- All specified values must be in [0,1].
- In cases where duplicated values are specified, only unique values are used, and a warning message is generated to indicate that all of the duplicated values are excluded from the analysis.

IID = boolean. *Determines whether or not the error terms are independently and identically distributed.* This is an option for the post-estimation of the variance-covariance of the parameter estimates and the confidence intervals for the predicted target values. The default setting is **TRUE**. When **FALSE** is specified, the computation time may be significantly increased for large models.

BANDWIDTH = value. *Determines which bandwidth method is used to estimate the variance-covariance matrix of the parameter estimates.* **BOFINGER** and **HALL_SHEATHER** are possible values (**BOFINGER** is the default value).

CILEVEL = value. *Specify the significance level for computing credible intervals.* Must be a single double value $\in$(0,100). The default level is 95%

METHOD = value. *Specifies the model estimation method.* The **SIMPLEX** option calls for the simplex algorithm developed by Barrodale and Roberts. The **INTERIOR-POINT** option calls for the Frisch-Newton interior-point non-linear optimization algorithm. The **AUTO** option is the default setting, which allows the procedure to automatically choose the appropriate estimation method.

TOL = value. *Specifies the tolerance value for the matrix manipulations in the interior-point method.* 0.000001 is the default setting. Only in effect when METHOD = AGL. Must be one value $\in (0, 10^{-3}]$.

CONV = value. *Specifies the convergence criterion for the numerical method.* Must be a single double value in $(0, 10^{-3}]$ with 10^{-6} as the default value.

MAXITER = integer. *Specify the maximum number of iterations the AGL method can go through in its computations.* The integer 2000 is the default setting. You must specify a positive integer. Valid when METHOD = AGL.

MODEL Subcommand

DESIGN specifies the model design for the analysis.

REGWGT = variable. *Specify the regression weights.* The parameter is optional. Only one variable is allowed.

FACTORS = factors. *Specify categorical variables for the model.* Must be a subset of the factors in **BAYES REGRESSION**.

COVARIATES = covariates. *Specify scale variables for the model.* Must be a subset of the factors in **BAYES REGRESSION**.

SAVE Subcommand

SAVE specifies the statistics to be scored for the Bayesian prediction distribution. Score the statistics derived from the Bayesian predictions, which include to following:

- **{MEANPRED(varname)}**: Mean of the posterior predictive distribution.
- **{VARIANCE(varname)}**: Variance of the posterior predictive distribution.
- **{MODE(varname)}**: Mode of the posterior predictive distribution.
- **{CIMEANPREDL(varname)}**: Lower bound of the credible interval of the posterior predictive distribution.
- **{CIMEANPREDU(varname)}**: Upper bound of the credible interval of the posterior predictive distribution.

Corresponding variable names can also be assigned.

PREDICT_EFFECTS Subcommand

PREDICT specifies the regressors to generate predictive distributions.

REGRESSORS = values. *Specify an observed vector with the values for the regressors.* Each regressor is assigned one value or string (allow one case can be predicted). For factors, both values and strings are allowed.

PREDICTION_TABLES Subcommand

PREDICT specifies the regressors to generate predictive distributions.

REGRESSORS = values. *Specify an observed vector with the values for the regressors.* Each regressor is assigned one value or string (allow one case can be predicted). For factors, both values and strings are allowed.

PREDICTION_LINES Subcommand

PREDICT specifies the regressors to generate predictive distributions.

REGRESSORS = values. *Specify an observed vector with the values for the regressors.* Each regressor is assigned one value or string (allow one case can be predicted). For factors, both values and strings are allowed.

PRINT Subcommand

PLOT Subcommand

PLOT controls the plots to be output.

COVARIATES = {covariate_sublist}. *Specify the covariate(s) to be plotted.* Must be a sublist of the covariates specified in **/DESIGN**.

FACTORS = {factor_sublist}. *Specify the factor(s) to be plotted.* Must be a sublist of the factors specified in **/DESIGN**.

MAX_CATEGORY = integer. *Specify the maximum factor levels to be plotted.* Applies to all factors. By default, the first 2 levels are plotted for each factor. Only a single, positive integer is valid.

INTERCEPT. *Specify whether the intercept term is plotted.* **FALSE** is the default setting.

ERRORVAR. *Specify whether the variance of errors is plotted.* **FALSE** is the default setting.

BAYESPRED. *Specify whether the predictive distribution is plotted.* **FALSE** is the default setting. The value can be **TRUE** only when valid values are specified for **REGRESSORS**.

OUTFILE Subcommand

OUTFILE specifies the target file to generate the PMML file.

PARAMETER = 'filename'. *Specify whether or not the variance-covariance matrix of the parameters will be scored in the output PMML file.*

EXTERNAL Subcommand

```
BAYES REGRESSION salary WITH educ prevexp
     /CRITERIA CILEVEL = 95
     /DESIGN COVARIATES = educ prevexp
     /INFERENCE ANALYSIS = BAYESFACTOR
     /PRIOR TYPE = REFERENCE .
```

- **CRITERIA** specifies that the significance level for computing credible intervals (**CILEVEL**) is set to the default value of 95.
- **DESIGN** specifies the model design for the analysis. The example uses the **educ** and **prevexp** as the scale variables.
- **INFERENCE** specifies the distribution for the analysis. The example uses Estimate Bayes Factor (**BAYESFACTOR**) as the **ANALYSIS** method.
- **PRIOR** specifies the prior distribution.

QUICK CLUSTER

QUICK CLUSTER is available in the Statistics Base Edition.

```
QUICK CLUSTER {varlist}
              {ALL    }

 [/MISSING=[{LISTWISE**}] [INCLUDE]]
           {PAIRWISE  }
           {DEFAULT   }

 [/FILE='savfile'|'dataset']

 [/INITIAL=(value list)]

 [/CRITERIA=[CLUSTER({2**})][NOINITIAL][MXITER({10**})] [CONVERGE({0**})]]
                    {n  }                      {n   }             {n  }

 [/METHOD=[{KMEANS[(NOUPDATE)]**}]
          {KMEANS(UPDATE)}     }
          {CLASSIFY            }

 [/PRINT=[INITIAL**] [CLUSTER] [ID(varname)] [DISTANCE] [ANOVA] [NONE]]

 [/OUTFILE='savfile'|'dataset']

 [/SAVE=[CLUSTER[(varname)]] [DISTANCE[(varname)]]]
```

**Default if subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
QUICK CLUSTER V1 TO V4
  /CRITERIA=CLUSTER(4)
  /SAVE=CLUSTER(GROUP).
```

Overview

When the desired number of clusters is known, QUICK CLUSTER groups cases efficiently into clusters. It is not as flexible as CLUSTER, but it uses considerably less processing time and memory, especially when the number of cases is large.

Options

Algorithm Specifications. You can specify the number of clusters to form with the CRITERIA subcommand. You can also use CRITERIA to control initial cluster selection and the criteria for iterating the clustering algorithm. With the METHOD subcommand, you can specify how to update cluster centers, and you can request classification only when working with very large data files.

Initial Cluster Centers. By default, QUICK CLUSTER chooses the initial cluster centers. Alternatively, you can provide initial centers on the INITIAL subcommand. You can also read initial cluster centers from IBM SPSS Statistics data files using the FILE subcommand.

Optional Output. With the PRINT subcommand, you can display the cluster membership of each case and the distance of each case from its cluster center. You can also display the distances between the final cluster centers and a univariate analysis of variance between clusters for each clustering variable.

Saving Results. You can write the final cluster centers to a data file using the OUTFILE subcommand. In addition, you can save the cluster membership of each case and the distance from each case to its classification cluster center as new variables in the active dataset using the SAVE subcommand.

Basic Specification

The basic specification is a list of variables. By default, QUICK CLUSTER produces two clusters. The two cases that are farthest apart based on the values of the clustering variables are selected as initial cluster centers and the rest of the cases are assigned to the nearer center. The new cluster centers are calculated as the means of all cases in each cluster, and if neither the minimum change nor the maximum

iteration criterion is met, all cases are assigned to the new cluster centers again. When one of the criteria is met, iteration stops, the final cluster centers are updated, and the distance of each case is computed.

Subcommand Order

- The variable list must be specified first.
- Subcommands can be named in any order.

Operations

The procedure generally involves four steps:

- First, initial cluster centers are selected, either by choosing one case for each cluster requested or by using the specified values.
- Second, each case is assigned to the nearest cluster center, and the mean of each cluster is calculated to obtain the new cluster centers.
- Third, the maximum change between the new cluster centers and the initial cluster centers is computed. If the maximum change is not less than the minimum change value and the maximum iteration number is not reached, the second step is repeated and the cluster centers are updated. The process stops when either the minimum change or maximum iteration criterion is met. The resulting clustering centers are used as classification centers in the last step.
- In the last step, all cases are assigned to the nearest classification center. The final cluster centers are updated and the distance for each case is computed.

When the number of cases is large, directly clustering all cases may be impractical. As an alternative, you can cluster a sample of cases and then use the cluster solution for the sample to classify the entire group. This can be done in two phases:

- The first phase obtains a cluster solution for the sample. This involves all four steps of the QUICK CLUSTER algorithm. OUTFILE then saves the final cluster centers to a data file.
- The second phase requires only one pass through the data. First, the FILE subcommand specifies the file containing the final cluster centers from the first analysis. These final cluster centers are used as the initial cluster centers for the second analysis. CLASSIFY is specified on the METHOD subcommand to skip the second and third steps of the clustering algorithm, and cases are classified using the initial cluster centers. When all cases are assigned, the cluster centers are updated and the distance of each case is computed. This phase can be repeated until final cluster centers are stable.

Variable List

The variable list identifies the clustering variables.

- The variable list is required and must be the first specification on QUICK CLUSTER.
- You can use keyword ALL to refer to all user-defined variables in the active dataset.
- QUICK CLUSTER uses squared Euclidean distances, which equally weight all clustering variables. If the variables are measured in units that are not comparable, the procedure will give more weight to variables with large variances. Therefore, you should standardize variables measured on different scales using procedure DESCRIPTIVES before performing QUICK CLUSTER.

CRITERIA Subcommand

CRITERIA specifies the number of clusters to form and controls options for the clustering algorithm. You can use any or all of the keywords below.

- The NOINITIAL option followed by the remaining steps of the default QUICK CLUSTER algorithm makes QUICK CLUSTER equivalent to MacQueen's *n*-means clustering method.

CLUSTER(n). *Number of clusters.* QUICK CLUSTER assigns cases to *n* clusters. The default is 2.

NOINITIAL. *No initial cluster center selection.* By default, initial cluster centers are formed by choosing one case (with valid data for the clustering variables) for each cluster requested. The initial selection requires a pass through the data to ensure that the centers are well separated from one another. If NOINITIAL is specified, QUICK CLUSTER selects the first *n* cases without missing values as initial cluster centers.

MXITER(n). *Maximum number of iterations for updating cluster centers.* The default is 10. Iteration stops when the maximum number of iterations has been reached. MXITER is ignored when METHOD=CLASSIFY.

CONVERGE(n). *Convergence criterion controlling minimum change in cluster centers.* The default value for *n* is 0. The minimum change value equals the convergence value (*n*) times the minimum distance between initial centers. Iteration stops when the largest change of any cluster center is less than or equal to the minimum change value. CONVERGE is ignored when METHOD=CLASSIFY.

METHOD Subcommand

By default, QUICK CLUSTER recalculates cluster centers after assigning all the cases and repeats the process until one of the criteria is met. You can use the METHOD subcommand to recalculate cluster centers after each case is assigned or to suppress recalculation until after classification is complete. When METHOD=KMEANS is specified, QUICK CLUSTER displays the iteration history table.

KMEANS(NOUPDATE). *Recalculate cluster centers after all cases are assigned for each iteration.* This is the default.

KMEANS(UPDATE). *Recalculate a cluster center each time a case is assigned.* QUICK CLUSTER calculates the mean of cases currently in the cluster and uses this new cluster center in subsequent case assignment.

CLASSIFY. *Do not recalculate cluster centers.* QUICK CLUSTER uses the initial cluster centers for classification and computes the final cluster centers as the means of all the cases assigned to the same cluster. When CLASSIFY is specified, the CONVERGE or MXITER specifications on CRITERIA are ignored.

INITIAL Subcommand

INITIAL specifies the initial cluster centers. Initial cluster centers can also be read from a data file (see “FILE Subcommand ” on page 1589).

- One value for each clustering variable must be included for each cluster requested. Values are specified in parentheses cluster by cluster.

Example

```
QUICK CLUSTER  A B C D
  /CRITERIA = CLUSTER(3)
  /INITIAL = (13 24  1  8
               7 12  5  9
              10 18 17 16).
```

- This example specifies four clustering variables and requests three clusters. Thus, twelve values are supplied on INITIAL.
- The initial center of the first cluster has a value of 13 for variable *A*, 24 for variable *B*, 1 for *C*, and 8 for *D*.

FILE Subcommand

Use FILE to obtain initial cluster centers from an external IBM SPSS Statistics data file or currently open dataset. (DATASET DECLARE command).

- The only specifications are the quoted file specification or dataset name and the numeric cluster ID variable.

Example

```
QUICK CLUSTER  A B C D
  /FILE='/data/init.sav'
  /CRITERIA = CLUSTER(3).
```

- In this example, the initial cluster centers are read from file *init.sav*. The file must contain cluster centers for the same four clustering variables specified (*A*, *B*, *C*, and *D*).

PRINT Subcommand

QUICK CLUSTER always displays in a Final Cluster Centers table listing the centers used to classify cases and the mean values of the cases in each cluster and a Number of Cases in Each Cluster table listing the number of weighted (if weighting is on) and unweighted cases in each cluster. Use PRINT to request other types of output.

- If PRINT is not specified or is specified without keywords, the default is INITIAL.

INITIAL. *Initial cluster centers.* When SPLIT FILES is in effect, the initial cluster center for each split file is displayed. This is the default.

CLUSTER. *Cluster membership.* Each case displays an identifying number or value, the number of the cluster to which it was assigned, and its distance from the center of that cluster. This output is extensive when the number of cases is large.

ID(varname). *Case identification.* The value of the specified variable is used in addition to the case numbers to identify cases in output. Case numbers may not be sequential if cases have been selected.

DISTANCE. *Pairwise distances between all final cluster centers.* This output can consume a great deal of processing time when the number of clusters requested is large.

ANOVA. *Descriptive univariate* F *tests for the clustering variables.* Since cases are systematically assigned to clusters to maximize differences on the clustering variables, these tests are descriptive only and should not be used to test the null hypothesis that there are no differences between clusters. Statistics after clustering are also available through procedure DISCRIMINANT or GLM (GLM is available in SPSS Statistics Standard Edition or the Advanced Statistics Option).

NONE. *No additional output.* Only the default output is displayed. NONE overrides any other specifications on PRINT.

Example

```
QUICK CLUSTER A B C D E
  /CRITERIA=CLUSTERS(6)
  /PRINT=CLUSTER ID(CASEID) DISTANCE.
```

- Six clusters are formed on the basis of the five variables *A*, *B*, *C*, *D*, and *E*.
- For each case in the file, cluster membership and distance from cluster center are displayed. Cases are identified by the values of the variable *CASEID*.
- Distances between all cluster centers are printed.

OUTFILE Subcommand

OUTFILE saves the final cluster centers in an external IBM SPSS Statistics data file or a previously declared dataset in the current session. You can later use these final cluster centers as initial cluster centers for a different sample of cases that use the same variables. You can also cluster the final cluster centers themselves to obtain clusters of clusters.

- The only specification is a filename or previously declared dataset name for the file. Filenames should be enclosed in quotes and are stored in the working directory unless a path is included as part of the file specification. Datasets are available during the current session but are not available in subsequent sessions unless you explicitly save them as data files.

- The program displays the name of the saved file in the procedure information notes.

Example

```
QUICK CLUSTER A B C D
  /CRITERIA = CLUSTER(3)
  /OUTFILE = '/data/QC1.sav'.
```

- QUICK CLUSTER writes the final cluster centers to the file *QC1.sav*.

SAVE Subcommand

Use SAVE to save results of cluster analysis as new variables in the active dataset.

- You can specify a variable name in parentheses following either keyword. If no variable name is specified, QUICK CLUSTER forms unique variable names by appending an underscore and a sequential number to the rootname *QCL*. The number increments with each new variable saved.
- The program displays the new variables and a short description of each in the procedure information notes.

CLUSTER[(varname)]. *The cluster number of each case.* The value of the new variable is set to an integer from 1 to the number of clusters.

DISTANCE[(varname)]. *The distance of each case from its classification cluster center.*

Example

```
QUICK CLUSTER A B C D
  /CRITERIA=CLUSTERS(6)
  /SAVE=CLUSTER DISTANCE.
```

- Six clusters of cases are formed on the basis of the variables *A*, *B*, *C*, and *D*.
- A new variable *QCL_1* is created and set to an integer between 1 and 6 to indicate cluster membership for each case.
- Another new variable *QCL_2* is created and set to the Euclidean distance between a case and the center of the cluster to which it is assigned.

MISSING Subcommand

MISSING controls the treatment of cases with missing values.

- LISTWISE, PAIRWISE, and DEFAULT are alternatives. However, each can be used with INCLUDE.

LISTWISE. *Delete cases with missing values listwise.* A case with a missing value for any of the clustering variables is deleted from the analysis and will not be assigned to a cluster. This is the default.

PAIRWISE. *Assign each case to the nearest cluster on the basis of the clustering variables for which the case has nonmissing values.* Only cases with missing values for *all* clustering variables are deleted.

INCLUDE. *Treat user-missing values as valid.*

DEFAULT. *Same as LISTWISE.*

RANK

```
RANK VARIABLES= varlist [({A**})] [BY varlist]
                         {D  }

  [/TIES={MEAN**  }]
         {LOW     }
         {HIGH    }
         {CONDENSE}

  [/FRACTION={BLOM**}]
             {TUKEY }
             {VW    }
             {RANKIT}

  [/PRINT={YES**}]
          {NO   }

  [/MISSING={EXCLUDE**}]
            {INCLUDE  }
```

The following function subcommands can each be specified once:

```
  [/RANK**] [/NTILES(k)] [/NORMAL] [/PERCENT]
  [/RFRACTION] [/PROPORTION] [/N] [/SAVAGE]
```

The following keyword can be used with any function subcommand:

```
  [INTO varname]
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
RANK VARIABLES=SALARY JOBTIME.
```

Overview

RANK produces new variables containing ranks, normal scores, and Savage and related scores for numeric variables.

Options

Methods. You can rank variables in ascending or descending order by specifying A or D on the VARIABLES subcommand. You can compute different rank functions and also name the new variables using the function subcommands. You can specify the method for handling ties on the TIES subcommand, and you can specify how the proportion estimate is computed for the NORMAL and PROPORTIONAL functions on the FRACTION subcommand.

Format. You can suppress the display of the summary table that lists the ranked variables and their associated new variables in the active dataset using the PRINT subcommand.

Basic Specification

The basic specification is VARIABLES and at least one variable from the active dataset. By default, the ranking function is RANK. Direction is ascending, and ties are handled by assigning the mean rank to tied values. A summary table that lists the ranked variables and the new variables into which computed ranks have been stored is displayed.

Subcommand Order

- VARIABLES must be specified first.
- The remaining subcommands can be specified in any order.

Operations

- RANK does not change the way the active dataset is sorted.
- If new variable names are not specified with the INTO keyword on the function subcommand, RANK creates default names. (Note: The automatically generated new variable names are limited to a maximum length of 8 bytes.)
- RANK automatically assigns variable labels to the new variables. The labels identify the source variables. For example, the label for a new variable with the default name *RSALARY* is *RANK of SALARY*.

Example

```
RANK VARIABLES=SALARY JOBTIME.
```

- RANK ranks *SALARY* and *JOBTIME* and creates two new variables in the active dataset, *RSALARY* and *RJOBTIME*, which contain the ranks.

VARIABLES Subcommand

VARIABLES specifies the variables to be ranked.

- VARIABLES is required and must be the first specification on RANK. The minimum specification is a single numeric variable. To rank more than one variable, specify a variable list.
- After the variable list, you can specify the direction for ranking in parentheses. Specify A for ascending (smallest value gets smallest rank) or D for descending (largest value gets smallest rank). A is the default.
- To rank some variables in ascending order and others in descending order, use both A and D in the same variable list. A or D applies to all preceding variables in the list up to the previous A or D specification.
- To organize ranks into subgroups, specify keyword BY followed by the variable whose values determine the subgroups. The active dataset does not have to be sorted by this variable.
- String variables cannot be specified. Use AUTORECODE to recode string variables for ranking.

Examples

```
RANK VARIABLES=MURDERS ROBBERY (D).
```

- RANK ranks *MURDERS* and *ROBBERY* and creates two new variables in the active dataset: *RMURDERS* and *RROBBERY*.
- D specifies descending order of rank. D applies to both *MURDERS* and *ROBBERY*.

```
RANK VARIABLES=MURDERS (D) ROBBERY (A) BY ETHNIC.
```

- Ranks are computed within each group defined by *ETHNIC*. *MURDERS* is ranked in descending order and *ROBBERY* in ascending order within each group of *ETHNIC*. The active dataset does not have to be sorted by *ETHNIC*.

Function Subcommands

The optional function subcommands specify different rank functions. RANK is the default function.

- Any combination of function subcommands can be specified for a RANK procedure, but each function can be specified only once.
- Each function subcommand must be preceded by a slash.
- The functions assign default names to the new variables unless keyword INTO is specified.

RANK. *Simple ranks.* The values for the new variable are the ranks. Rank can either be ascending or descending, as indicated on the VARIABLES subcommand. Rank values can be affected by the specification on the TIES subcommand.

RFRACTION . *Fractional ranks.* The values for the new variable equal the ranks divided by the sum of the weights of the nonmissing cases. If HIGH is specified on TIES, fractional rank values are an empirical cumulative distribution.

NORMAL. *Normal scores* [58]. The new variable contains the inverse of the standard normal cumulative distribution of the proportion estimate defined by the FRACTION subcommand. The default for FRACTION is BLOM.

PERCENT. *Fractional ranks as a percentage.* The new variable contains fractional ranks multiplied by 100.

PROPORTION . *Proportion estimates.* The estimation method is specified by the FRACTION subcommand. The default for FRACTION is BLOM.

N. *Sum of case weights.* The new variable is a constant.

SAVAGE. *Savage scores* [59]. The new variable contains Savage (exponential) scores.

NTILES(k) . *Percentile groups.* The new variable contains values from 1 to *k*, where *k* is the number of groups to be generated. Each case is assigned a group value, which is the integer part of $1+rk/(w+1)$, where *r* is the rank of the case, *k* is the number of groups specified on NTILES, and *w* is the sum of the case weights. Group values can be affected by the specification on TIES. There is no default for *k*.

INTO Keyword

INTO specifies variable names for the new variable(s) added to the active dataset. INTO can be used with any of the function subcommands.

- INTO must follow a function subcommand. You must specify the INTO subcommand to assign names to the new variables created by the function.
- You can specify multiple variable names on INTO. The names are assigned to the new variables in the order they are created (the order the variables are specified on the VARIABLES subcommand).
- If you specify fewer names than the new variables, default names are used for the remaining new variables. If you specify more names, the program issues a message and the command is not executed.

If INTO is not specified on a function, RANK creates default names for the new variables according to the following rules:

- The first letter of the ranking function is added to the first seven characters of the original variable name.
- New variable names cannot duplicate variable names in the active dataset or names specified after INTO or generated by default.
- If a new default name is a duplicate, the scheme *XXXnnn* is used, where *XXX* represents the first three characters of the function and *nnn* is a three-digit number starting with 001 and increased by 1 for each variable. (If the ranking function is N, *XXX* is simply *N*.) If this naming scheme generates duplicate names, the duplicates are named *RNKXXnn*, where *XX* is the first two characters of the function and *nn* is a two-digit number starting with 01 and increased by 1 for each variable.
- If it is not possible to generate unique names, an error results.

Example

```
RANK VARIABLES=SALARY
 /NORMAL INTO SALNORM
 /SAVAGE INTO SALSAV
 /NTILES(4) INTO SALQUART.
```

- RANK generates three new variables from variable *SALARY*.

[58] Lehmann, E. L. 1975. *Nonparametrics: Statistical methods based on ranks*. San Francisco: Holden-Day.
[59] Lehmann, E. L. 1975. *Nonparametrics: Statistical methods based on ranks*. San Francisco: Holden-Day.

- NORMAL produces the new variable *SALNORM*. *SALNORM* contains normal scores for *SALARY* computed with the default formula BLOM.
- SAVAGE produces the new variable *SALSAV*. *SALSAV* contains Savage scores for *SALARY*.
- NTILES(4) produces the new variable *SALQUART*. *SALQUART* contains the value 1, 2, 3, or 4 to represent one of the four percentile groups of *SALARY*.

TIES Subcommand

TIES determines the way tied values are handled. The default method is MEAN.

MEAN. *Mean rank of tied values is used for ties.* This is the default.

LOW. *Lowest rank of tied values is used for ties.*

HIGH. *Highest rank of tied values is used for ties.*

CONDENSE. *Consecutive ranks with ties sharing the same value.* Each distinct value of the ranked variable is assigned a consecutive rank. Ties share the same rank.

Example

```
RANK VARIABLES=BURGLARY /RANK INTO RMEAN /TIES=MEAN.
RANK VARIABLES=BURGLARY /RANK INTO RCONDS /TIES=CONDENSE.
RANK VARIABLES=BURGLARY /RANK INTO RHIGH /TIES=HIGH.
RANK VARIABLES=BURGLARY /RANK INTO RLOW /TIES=LOW.
```

- The values of *BURGLARY* and the four new ranking variables are shown below:

```
BURGLARY   RMEAN   RCONDS   RHIGH   RLOW
   0         3       1       5       1
   0         3       1       5       1
   0         3       1       5       1
   0         3       1       5       1
   0         3       1       5       1
   1         6.5     2       7       6
   1         6.5     2       7       6
   3         8       3       8       8
```

FRACTION Subcommand

FRACTION specifies the way to compute a proportion estimate *P* for the NORMAL and PROPORTION rank functions.

- FRACTION can be used only with function subcommands NORMAL or PROPORTION. If it is used with other function subcommands, FRACTION is ignored and a warning message is displayed.
- Only one formula can be specified for each RANK procedure. If more than one is specified, an error results.

In the following formulas, *r* is the rank and *w* is the sum of case weights.

BLOM. *Blom's transformation, defined by the formula (r – 3/8) / (w + 1/4).* [60] This is the default.

RANKIT. *The formula is (r – 1/2) / w.* [61]

TUKEY. *Tukey's transformation, defined by the formula (r – 1/3) / (w + 1/3).* [62]

VW. *Van der Waerden's transformation, defined by the formula r / (w +1).* [63]

[60] Blom, G. 1958. *Statistical estimates and transformed beta variables*. New York: John Wiley and Sons.
[61] Chambers, J. M., W. S. Cleveland, B. Kleiner, and P. A. Tukey. 1983. *Graphical methods for data analysis*. Boston: Duxbury Press.
[62] Tukey, J. W. 1962. The future of data analysis. *Annals of Mathematical Statistics,* 33:22, 1-67.
[63] Lehmann, E. L. 1975. *Nonparametrics: Statistical methods based on ranks*. San Francisco: Holden-Day.

Example

```
RANK VARIABLES=MORTGAGE VALUE /FRACTION=BLOM
  /NORMAL INTO MORTNORM VALNORM.
```

- RANK generates new variables *MORTNORM* and *VALNORM*. *MORTNORM* contains normal scores for *MORTGAGE*, and *VALNORM* contains normal scores for *VALUE*.

PRINT Subcommand

PRINT determines whether the summary tables are displayed. The summary table lists the ranked variables and their associated new variables in the active dataset.

YES. *Display the summary tables.* This is the default.

NO. *Suppress the summary tables.*

MISSING Subcommand

MISSING controls the treatment of user-missing values.

INCLUDE. *Include user-missing values.* User-missing values are treated as valid values.

EXCLUDE. *Exclude all missing values.* User-missing values are treated as missing. This is the default.

Example

```
MISSING VALUE SALARY (0).
RANK VARIABLES=SALARY /RANK INTO SALRANK /MISSING=INCLUDE.
```

- RANK generates the new variable *SALRANK*.
- INCLUDE causes the user-missing value 0 to be included in the ranking process.

References

Blom, G. 1958. *Statistical estimates and transformed beta variables*. New York: John Wiley and Sons.

Chambers, J. M., W. S. Cleveland, B. Kleiner, and P. A. Tukey. 1983. *Graphical methods for data analysis*. Boston: Duxbury Press.

Fisher, R. A. 1973. *Statistical methods for research workers*, 14th ed. New York: Hafner Publishing Company.

Frigge, M., D. C. Hoaglin, and B. Iglewicz. 1987. Some implementations for the boxplot. In: *Computer Science and Statistics Proceedings of the 19th Symposium on the Interface*, R. M. Heiberger, and M. Martin, eds. Alexandria, Virginia: American Statistical Association.

Lehmann, E. L. 1975. *Nonparametrics: Statistical methods based on ranks*. San Francisco: Holden-Day.

Tukey, J. W. 1962. The future of data analysis. *Annals of Mathematical Statistics,* 33:22, 1-67.

RATIO STATISTICS

RATIO STATISTICS is available in the Statistics Base Edition.

```
RATIO STATISTICS numerator varname WITH denominator varname
  [BY group varname[({ASCENDING**})]]
                     {DESCENDING }
                     {NOSORT     }

 [/MISSING = {EXCLUDE**}]
             {INCLUDE  }

  [/OUTFILE('file'|'dataset') = [AAD] [BCOC((low,high) [(low,high)] ...)] [CIN[({95   })]]
                                                                           {value}
                         [COD] [MAX] [MDCOV] [MEAN] [MEDIAN] [MIN] [MNCOV] [PRD]
                         [RANGE] [STDDEV] [WCOC(value list)] [WGTMEAN]]

  [/PRINT = [AAD] [BCOC(low,high)...] [CIN[({95   })]]
                                       {value}
            [COD] [MAX] [MDCOV] [MEAN] [MEDIAN] [MIN] [MNCOV] [PRD]
            [RANGE] [STDDEV] [WCOC(value list)] [WGTMEAN]]
```

** Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
RATIO STATISTICS appraise WITH price
  /PRINT = AAD BCOC((1,2) (3,4)) MEAN.
```

Overview

RATIO STATISTICS provides a variety of descriptive statistics for the ratio between two variables.

Basic Specification

The minimum specification is a numerator variable and a denominator variable, and either an OUTFILE subcommand or a PRINT subcommand.

Subcommand Order

- The variable list must be specified first.
- Subcommands can be specified in any order.

Syntax Rules

- Empty subcommands are silently ignored.
- All subcommands should be specified only once. If a subcommand is repeated, only the last specification will be used.
- The following words are reserved as keywords in this procedure: BY and WITH.

Examples

```
RATIO STATISTICS lastval  WITH saleval  BY town  (ASCENDING)
  /MISSING = EXCLUDE
  /PRINT =  BCOC(0.8,1.2)  COD MEDIAN WCOC( 20 ) .
```

- The procedure computes ratios by dividing *lastval* by *saleval*. Separate ratio statistics are computed for each category of *town*, with values displayed in ascending order.
- By default, user-missing values are excluded from the analysis.

- The PRINT subcommand requests the coefficient of concentration for the range 0.8-1.2, the coefficient of dispersion, the median, and the coefficient of concentration for ratios falling within 20 percent of the median.

Case Frequency

- If a WEIGHT variable is specified, its values are used as frequency weights by this procedure.
- Cases with missing or nonpositive weights are not used for computing the ratio statistics.
- The weight values are rounded to the nearest whole numbers before use. For example, 0.5 is rounded to 1, and 2.4 is rounded to 2.

Variable List

The variable list specifies the numerator variable, denominator variable, and optional group variable.

- The numerator variable must be the first specification after the procedure name.
- The denominator variable must be preceded by the keyword WITH.
- The group variable, if specified, must be preceded by the keyword BY.
- Both the numerator and the denominator variables must be numeric.
- The group variable can be of any type (numeric or string).
- By default or when the keyword ASCENDING is specified, values of the group variable are displayed in ascending order. Specify the keyword DESCENDING to display in descending order. Specify NOSORT to preserve the appearance order in the data.
- Only cases with no (system- or user-) missing values in both the numerator and the denominator variables will be used. Please note that this rule does not apply to the group variable.

Example

```
RATIO STATISTICS appraise WITH price
  /PRINT = AAD BCOC((1,2) (3,4)) MEAN.
```

- This is a typical analysis where *appraise* is the appraised value and *price* is the transaction price. The ratio is computed by dividing *appraise* by *price*.

Example

```
RATIO STATISTICS appraise WITH price BY county
  /PRINT = CIN(90) MEDIAN.
```

- The ratio is still computed by dividing *appraise* by *price*. However, separate ratio statistics are requested for each category of *county*.

MISSING Subcommand

MISSING specifies the way to handle cases with user-missing values.

- A case is never used if it contains system-missing values in the numerator and/or the denominator variables.
- If this subcommand is not specified, the default is EXCLUDE.
- Keywords EXCLUDE and INCLUDE are mutually exclusive. Only one of them can be specified once.

EXCLUDE. *Exclude both user-missing and system-missing values.* This is the default.

INCLUDE. *User-missing values are treated as valid.* System-missing values cannot be included in the analysis.

OUTFILE Subcommand

OUTFILE saves the requested statistics to an external IBM SPSS Statistics data file or a previously declared dataset (DATASET DECLARE command).

- The requested statistics are saved in a single record in the external file.
- If a group variable has been specified, the requested statistics at each category of the group variable will also be saved as additional records in the external file.
- The file specification or dataset name should be quoted and enclosed in quotes.

The following statistics are available.

AAD. *Average absolute deviation.* The result of summing the absolute deviations of the ratios about the median and dividing the result by the total number of ratios.

BCOC (low,high) ...). *Coefficient of concentration.* The percentage of ratios that fall into an interval. Pairs of low and high values enclosed in parentheses specify the intervals.

CIN(a). *Confidence interval.* Specifying this keyword displays confidence intervals for the mean, median, and weighted mean (if those statistics are requested). Specify a value greater than or equal to 0 and less than 100 as the confidence level.

COD. *Coefficient of dispersion.* The result of expressing the average absolute deviation as a percentage of the median.

MAX. *Maximum.* The largest ratio.

MDCOV. *Median-centered coefficient of variation.* The result of expressing the root mean squares of deviation from the median as a percentage of the median.

MEAN. *Mean.* The result of summing the ratios and dividing the result by the total number ratios.

MEDIAN. *Median.* The value such that number of ratios less than this value and the number of ratios greater than this value are the same.

MIN. *Minimum.* The smallest ratio.

MNCOV. *Mean-centered coefficient of variation.* The result of expressing the standard deviation as a percentage of the mean.

PRD. *Price-related differential.* Also known as the index of regressivity, the result of dividing the mean by the weighted mean.

RANGE. *Range.* The result of subtracting the minimum ratio from the maximum ratio.

STDDEV. *Standard deviation.* The result of summing the squared deviations of the ratios about the mean, dividing the result by the total number of ratios minus one, and taking the positive square root.

WCOC(value list). *Coefficient of concentration.* The percentage of ratios that fall within the specified percent of the median. Specify a list of values that are greater than 0 and less than 100.

WGTMEAN. *Weighted mean.* The result of dividing the mean of the numerator by the mean of the denominator. It is also the mean of the ratios weighted by the denominator.

Example

```
RATIO STATISTICS appraise WITH price BY county
  /OUTFILE('/PropertyTax/Ratio.sav') = CIN(90) MEDIAN.
```

- The median ratios and their 90% confidence intervals at each category of *county* are saved to */PropertyTax/Ratio.sav*.
- The overall median ratio and its 90% confidence intervals are also saved.

PRINT Subcommand

PRINT displays optional output. If no PRINT subcommand is specified, only a case processing summary table is displayed by default.

AAD. *Average absolute deviation.* The result of summing the absolute deviations of the ratios about the median and dividing the result by the total number of ratios.

BCOC(low,high) ...). *Coefficient of concentration.* The percentage of ratios that fall into an interval. Pairs of low and high values enclosed in parentheses specify the intervals.

CIN(a). *Confidence interval.* Specifying this keyword displays confidence intervals for the mean, median, and weighted mean (if those statistics are requested). Specify a value greater than or equal to 0 and less than 100 as the confidence level.

COD. *Coefficient of dispersion.* The result of expressing the average absolute deviation as a percentage of the median.

MAX. *Maximum.* The largest ratio.

MDCOV. *Median-centered coefficient of variation.* The result of expressing the root mean squares of deviation from the median as a percentage of the median.

MEAN. *Mean.* The result of summing the ratios and dividing the result by the total number ratios.

MEDIAN. *Median.* The value such that number of ratios less than this value and the number of ratios greater than this value are the same.

MIN. *Minimum.* The smallest ratio.

MNCOV. *Mean-centered coefficient of variation.* The result of expressing the standard deviation as a percentage of the mean.

PRD. *Price-related differential.* Also known as the index of regressivity, the result of dividing the mean by the weighted mean.

RANGE. *Range.* The result of subtracting the minimum ratio from the maximum ratio.

STDDEV. *Standard deviation.* The result of summing the squared deviations of the ratios about the mean, dividing the result by the total number of ratios minus one, and taking the positive square root.

WCOC(value list). *Coefficient of concentration.* The percentage of ratios that fall within the specified percentage of the median. Specify a list of values that are greater than 0 and less than 100.

WGTMEAN. *Weighted mean.* The result of dividing the mean of the numerator by the mean of the denominator. It is also the mean of the ratios weighted by the denominator.

Example

```
RATIO STATISTICS appraise WITH price BY county
  /PRINT = BCOC((0.5,0.9) (1.3,1.5)) WCOC(15 30 45) MEDIAN PRD.
```

- The median ratios and priced related differentials at each category of *county* are displayed. The overall median ratio and the overall price-related differential are also displayed.
- Five coefficients of concentration are also displayed. The first two COC are percentages of ratios that fall into the intervals: (0.5, 0.9) and (1.3, 1.5). The next three COC are percentages of ratios that fall within 15% of the median, 30% of the median, and 45% of the median.

RBF

RBF is available in SPSS Statistics Premium Edition or the Neural Network option.

```
RBF dependent variable [(MLEVEL = {S})] [dependent variable...]
                                  {O}
                                  {N}

    [BY factor list] [WITH covariate list]

[/EXCEPT VARIABLES = varlist]

[/RESCALE [COVARIATE = {STANDARDIZED**}]   [DEPENDENT = {STANDARDIZED**}]]
                       {NORMALIZED    }                 {NORMALIZED    }
                       {ADJNORMALIZED }                 {ADJNORMALIZED }
                       {NONE          }                 {NONE          }

[/PARTITION {TRAINING = {70**   } TESTING = {30**   } HOLDOUT = {0**    }}]
                        {number}            {number}            {number}
            {VARIABLE = varname                                          }

[/ARCHITECTURE [{[MINUNITS = {AUTO** } MAXUNITS = {AUTO** }]}]
                             {integer}            {integer}
                {NUMUNITS = integer                          }

               [HIDDENFUNCTION = {NRBF**}]]
                                 {ORBF  }

[/CRITERIA OVERLAP = {AUTO**}]
                     {number}

[/MISSING USERMISSING = {EXCLUDE**}]
                        {INCLUDE  }

[/PRINT [CPS**]  [NETWORKINFO**]  [SUMMARY**]  [CLASSIFICATION**]
        [SOLUTION]  [IMPORTANCE] [NONE]]

[/PLOT [NETWORK**]  [PREDICTED]  [RESIDUAL]  [ROC]
       [GAIN]  [LIFT]  [NONE]]

[/SAVE [PREDVAL[(varname [varname...])]]

       [PSEUDOPROB[(rootname[:{25     }] [rootname...])]]]
                              {integer}

[/OUTFILE MODEL = 'file' ['file'...]]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 16.0

- Command introduced.

Example

```
RBF dep_var BY A B C WITH X Y Z.
```

Overview

Neural networks are a data mining tool for finding unknown patterns in databases. Neural networks can be used to make business decisions by forecasting demand for a product as a function of price and other variables or by categorizing customers based on buying habits and demographic characteristics. The RBF procedure fits a radial basis function neural network, which is a feedforward, supervised learning network with an input layer, a hidden layer called the radial basis function layer, and an output layer. The hidden layer transforms the input vectors into radial basis functions. Like the MLP (multilayer perceptron) procedure, the RBF procedure performs prediction and classification.

The RBF procedure trains the network in two stages:

1. The procedure determines the radial basis functions using clustering methods. The center and width of each radial basis function are determined.

2. The procedure estimates the synaptic weights given the radial basis functions. The sum-of-squares error function with identity activation function for the output layer is used for both prediction and classification. Ordinary least squares regression is used to minimize the sum-of-squares error.

Because of this two-stage training approach, the RBF network is in general trained much faster than MLP.

Options

Prediction or classification. One or more dependent variables may be specified, and they may be scale, categorical, or a combination. If a dependent variable has scale measurement level, then the neural network predicts continuous values that approximate the "true" value of some continuous function of the input data. If a dependent variable is categorical, then the neural network is used to classify cases into the "best" category based on the input predictors.

Rescaling. RBF optionally rescales covariates (predictors with scale measurement level) or scale dependent variables before training the neural network. There are three rescaling options: standardization, normalization, and adjusted normalization.

Training, testing, and holdout data. RBF optionally divides the dataset into training, testing, and holdout data. The neural network is trained using the training data. The testing data can be used to determine the "best" number of hidden units for the network. The holdout data is completely excluded from the training process and is used for independent assessment of the final network.

Architecture selection. The RBF procedure creates a neural network with one hidden layer and can perform automatic architecture selection to find the "best" number of hidden units. By default, the procedure automatically computes a reasonable range and finds the "best" number within the range. However, you can override these computations by providing your own range or a specific number of hidden units.

Activation functions. Units in the hidden layer can use the normalized radial basis function or the ordinary radial basis function.

Missing values. The RBF procedure has an option for treating user-missing values of categorical variables as valid. User-missing values of scale variables are always treated as invalid.

Output. RBF displays pivot table output but offers an option for suppressing most such output. Graphical output includes a network diagram (default) and a number of optional charts: predicted by observed values, residual by predicted values, ROC (Receiver Operating Characteristic) curves, cumulative gains, lift, and independent variable importance. The procedure also optionally saves predicted values in the active dataset. Hidden unit center and width vectors and synaptic weight estimates can be saved in XML files.

Basic Specification

The basic specification is the RBF command followed by one or more dependent variables, the BY keyword and one or more factors, and the WITH keyword and one or more covariates.

By default, the RBF procedure standardizes covariates and scale dependent variables and selects a training sample before training the neural network. Automatic architecture selection is used to find the "best" neural network architecture. User-missing values are excluded and default pivot table output is displayed.

Note: Since measurement level can affect the results, if any variables (fields) have an unknown measurement level, an initial data pass will be performed to determine default measurement level for any variables with an unknown measurement level. For information on the criteria used to determine default measurement level, see SET SCALEMIN.

Syntax Rules

- All subcommands are optional.
- Subcommands may be specified in any order.
- Only a single instance of each subcommand is allowed.
- An error occurs if a keyword is specified more than once within a subcommand.
- Parentheses, equals signs, and slashes shown in the syntax chart are required.

- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not allowed.
- Any split variable defined on the SPLIT FILE command may not be used as a dependent variable, factor, covariate, or partition variable.

Limitations

Frequency weights specified on the WEIGHT command are ignored with a warning by the RBF procedure.

Categorical Variables

The RBF procedure temporarily recodes categorical predictors and dependent variables using one-of-*c* coding for the duration of the procedure. If there are *c* categories of a variable, then the variable is stored as c vectors, with the first category denoted (1,0,...,0), the next category, (0,1,0,...,0), ..., and the final category, (0,0,...,0,1).

Because of the one-of-*c* coding, the total number of input units is the number of scale predictors plus the number of categories across all categorical predictors. However, unlike the multilayer perceptron (MLP), this coding scheme does not increase the number of syntaptic weights for categorical predictors and hence should not significantly increase the training time.

All one-of-*c* coding is based on the training data, even if a testing or holdout sample is defined (see “PARTITION Subcommand ” on page 1608). Thus, if the testing or holdout samples contain cases with predictor categories that are not present in the training data, then those cases are not used by the procedure or in scoring. If the testing or holdout samples contain cases with dependent variable categories that are not present in the training data, then those cases are not used by the procedure but they may be scored.

Replicating Results

The RBF procedure uses random number generation during random assignment of partitions. To reproduce the same randomized results in the future, use the SET command to set the initialization value for the random number generator before each run of the RBF procedure.

RBF results are also dependent on data order because the two-step cluster algorithm is used to determine the radial basis functions. To minimize data order effects, randomly order the cases before running the RBF procedure. To verify the stability of a given solution, you may want to obtain several different solutions with cases sorted in different random orders. In situations with extremely large file sizes, multiple runs can be performed with a sample of cases sorted in different random orders.

In summary, if you want to exactly replicate RBF results in the future, use the same initialization value for the random number generator and the same data order, in addition to using the same RBF procedure settings.

Examples

Basic specification with default neural network settings

```
RBF DepVar BY A B C WITH X Y Z.
```

- The RBF procedure treats *DepVar* as the dependent variable. Predictors *A*, *B*, and *C* are factors, and *X*, *Y*, and *Z* are covariates.
- By default, covariates and the dependent variable (if it has scale measurement level) are standardized before training. Also, the active dataset is partitioned into training and testing data samples, with 70% going to the training data and 30% to the testing data sample.
- Automatic architecture selection is used to find the “best” neural network architecture.
- User-missing values are excluded and default output is displayed.

User-specified neural network with three hidden units

```
RBF DepVar BY A B C WITH X Y Z
  /ARCHITECTURE
```

```
    NUMUNITS=3
    HIDDENFUNCTION=ORBF.
```

- The RBF procedure treats *DepVar* as the dependent variable. Predictors *A*, *B*, and *C* are factors, and *X*, *Y*, and *Z* are covariates.
- By default, covariates and the dependent variable (if it has scale measurement level) are standardized before training. Also, the active dataset is partitioned into training and testing data samples, with 70% going to the training data and 30% to the testing data sample.
- The ARCHITECTURE subcommand specifies a neural network with three hidden units, thereby overriding automatic architecture selection. In addition, the hidden layer uses the ordinary radial basis function.

Variable Lists

The command line variable lists specify the dependent variables, any categorical predictors (also known as factors), and any scale predictors (also known as covariates).

Dependent Variables

- A list of one or more dependent variables must be the first specification on the RBF command.
- Each dependent variable may be followed by the measurement level specification, which contains, in parentheses, the MLEVEL keyword followed by an equals sign and then S for scale, O for ordinal, or N for nominal. RBF treats ordinal and nominal dependent variables equivalently as categorical.
- If a measurement level is specified, then it temporarily overrides a dependent variable's setting in the data dictionary.
- If no measurement level is specified, then RBF defaults to the dictionary setting.
- If a measurement level is not specified and no setting is recorded in the data dictionary, then a numeric variable is treated as scale and a string variable is treated as categorical.
- Dependent variables can be numeric or string.
- A string variable may be defined as ordinal or nominal only.

Predictor Variables

- The names of the factors, if any, must be preceded by the keyword BY.
- If the keyword BY is specified with no factors, then a warning is issued and BY is ignored.
- The names of the covariates, if any, must be preceded by the keyword WITH.
- If the keyword WITH is specified with no covariates, then a warning is issued and WITH is ignored.
- If the dependent variable is specified within a factor list or a covariate list, then it is ignored in the list.
- All variables specified within a factor or covariate list must be unique. If duplicate variables are specified within a list, then the duplicates are ignored.
- If duplicate variables are specified across the factor and covariate lists, then an error is issued.
- Factor variables can be numeric or string.
- Covariates must be numeric.
- At least one predictor must be specified.

EXCEPT Subcommand

The EXCEPT subcommand lists any variables that the RBF procedure should exclude from the factor or covariate lists on the command line. This subcommand is useful if the factor or covariate lists contain a large number of variables—specified using the TO or ALL keyword, for example—but there are a few variables (for example, Case ID) that should be excluded.

The EXCEPT subcommand is introduced strictly for the purpose of simplifying syntax. Missing values on factors or covariates specified on EXCEPT do not affect whether a case is included in the analysis. For example, the following two RBF commands are equivalent. In both commands, listwise deletion is based on the dependent variable and factors *A*, *B*, and *C*.

```
RBF DepVar BY A B C.
```

```
RBF DepVar BY A B C D   /EXCEPT VARIABLES=D.
```

- The EXCEPT subcommand ignores duplicate variables and variables that are not specified on the command line's factor or covariate lists.
- There is no default variable list on the EXCEPT subcommand.

RESCALE Subcommand

The RESCALE subcommand is used to rescale covariates or scale dependent variables.

All rescaling is performed based on the training data, even if a testing or holdout sample is defined (see "PARTITION Subcommand " on page 1608). That is, depending on the type of rescaling, the mean, standard deviation, minimum value, or maximum value of a covariate or dependent variable is computed using only the training data. It is important that these covariates or dependent variables have similar distributions across the training, testing, and holdout samples. If the data are partitioned by specifying percentages on the PARTITION subcommand, then the RBF procedure attempts to ensure this similarity by random assignment. However, if you use the PARTITION subcommand VARIABLE keyword to assign cases to the training, testing, and holdout samples, then we recommend that you confirm that the distributions are similar across samples before running the RBF procedure.

COVARIATE Keyword

The COVARIATE keyword specifies the rescaling method to use for covariates specified following WITH on the command line.

If no covariates are specified on the command line, then the COVARIATE keyword is ignored.

STANDARDIZED. *Subtract the mean and divide by the standard deviation, (x-mean)/s.* This is the default rescaling method for covariates.

NORMALIZED. *Subtract the minimum and divide by the range, (x-min)/(max-min).*

ADJNORMALIZED. *Adjusted version of subtracting the minimum and dividing by the range, [2*(x-min)/(max-min)]-1 .*

NONE. *No rescaling of covariates.*

DEPENDENT Keyword

The DEPENDENT keyword specifies the rescaling method to use for the dependent variables.

This keyword is applied only for scale dependent variables; that is, either MLEVEL=S is specified on the command line or the variable has a scale measurement level based on its data dictionary setting. If a dependent variable is not scale, then the DEPENDENT keyword is ignored for that variable.

STANDARDIZED. *Subtract the mean and divide by the standard deviation, (x-mean)/s.* This is the default rescaling method for scale dependent variables.

NORMALIZED. *Subtract the minimum and divide by the range, (x-min)/(max-min).*

ADJNORMALIZED. *Adjusted version of subtracting the minimum and dividing by the range, [2*(x-min)/(max-min)]-1.*

NONE. *No rescaling of scale dependent variables.*

PARTITION Subcommand

The PARTITION subcommand specifies the method of partitioning the active dataset into training, testing, and holdout samples. The training sample comprises the data records used to train the neural network. The testing sample is an independent set of data records used to track prediction error during training in order to prevent overtraining. The holdout sample is another independent set of data records used to assess the final neural network.

- The partition can be defined by specifying the ratio of cases randomly assigned to each sample (training, testing, and holdout) or by a variable that assigns each case to the training, testing, or holdout sample.
- If the PARTITION subcommand is not specified, then the default partition randomly assigns 70% of the cases to the training sample, 30% to the testing sample, and 0% to the holdout sample. If you want to specify a different random assignment, then you must specify new values for the TRAINING, TESTING, and HOLDOUT keywords. The value specified on each keyword gives the relative number of cases in the active dataset to assign to each sample. For example, `/PARTITION TRAINING = 50 TESTING = 30 HOLDOUT = 20` is equivalent to `/PARTITION TRAINING = 5 TESTING = 3 HOLDOUT = 2`; both subcommands randomly assign 50% of the cases to the training sample, 30% to the testing sample, and 20% to the holdout sample.
- If you want to be able to reproduce results based on the TRAINING, TESTING, and HOLDOUT keywords later, use the SET command to set the initialization value for the random number generator before running the RBF procedure.
- Be aware of the relationship between rescaling and partitioning. See the topic "RESCALE Subcommand " on page 1607 for more information.
- All partitioning is performed after listwise deletion of any cases with invalid data for any variable used by the procedure. See "MISSING Subcommand " on page 1161 for details about valid and invalid data.

TRAINING Keyword

The TRAINING keyword specifies the relative number of cases in the active dataset to randomly assign to the training sample. The value must be an integer greater than 0. The default (if the PARTITION subcommand is not specified) is 70.

TESTING Keyword

The TESTING keyword specifies the relative number of cases in the active dataset to randomly assign to the testing sample. The value must be an integer greater than 0. The default (if the PARTITION subcommand is not specified) is 30.

HOLDOUT Keyword

The HOLDOUT keyword specifies the relative number of cases in the active dataset to randomly assign to the holdout sample. The value must be an integer greater than 0. The default (if the PARTITION subcommand is not specified) is 0.

VARIABLE Keyword

The VARIABLE keyword specifies a variable that assigns each case in the active dataset to the training, testing, or holdout sample. Cases with a positive value on the variable are assigned to the training sample, cases with a value of 0, to the testing sample, and cases with a negative value, to the holdout sample. Cases with a system-missing value are excluded from the analysis. (Any user-missing values for the partition variable are always treated as valid.)

The variable may not be the dependent variable or any variable specified on the command line factor or covariate lists. The variable must be numeric.

ARCHITECTURE Subcommand

The ARCHITECTURE subcommand is used to specify the neural network architecture. RBF creates a neural network with one hidden layer. There are three ways of choosing the number of hidden units:

1. By default, RBF uses automatic architecture selection to find the "best" number of hidden units within a range. The procedure automatically computes the minimum and maximum and finds the best number of hidden units within the range.

   If a testing sample is defined; that is, if /PARTITION TESTING is specified with a number greater than zero or via /PARTITION VARIABLE, then the procedure uses the testing data criterion: The best number of hidden units is the one that yields the smallest error in the testing data. If a testing sample is not defined, then the procedure uses the Bayesian information criterion (BIC): The best number of hidden units is the one that yields the smallest BIC based on the training data.
2. You can provide your own range by specifying the MINUNITS and MAXUNITS keywords with integers, and the procedure will find the "best" number of hidden units within that range. It is invalid to specify only MINUNITS or only MAXUNITS; both must be given. The best number of hidden units from the range is determined using the testing data criterion or the BIC.
3. You can override the use of a range and specify a particular number of units directly using the NUMUNITS keyword. It is invalid to specify NUMUNITS with either the MINUNITS or MAXUNITS keyword.

MINUNITS Keyword

The MINUNITS keyword specifies the minimum number of units to use in the range. Value AUTO indicates that the number should be computed automatically. Alternatively, a positive integer may be specified. The integer must be less than the number specified on the MAXUNITS keyword.

The default value is AUTO.

MAXUNITS Keyword

The MAXUNITS keyword specifies the maximum number of units to use in the range. Value AUTO indicates that the number should be computed automatically. Alternatively, a positive integer may be specified. The integer must be greater than the number specified on the MINUNITS keyword.

The default value is AUTO.

NUMUNITS Keyword

The NUMUNITS keyword specifies a particular number of hidden units instead of a range. A positive integer may be specified.

- If NUMUNITS is specified when testing data are defined (see "PARTITION Subcommand " on page 1608), then the testing data are not used in determining the network architecture and a warning is issued.
- There is no default value.

HIDDENFUNCTION Keyword

The HIDDENFUNCTION keyword specifies the Gaussian radial basis function used in the hidden layer.

NRBF. *Normalized radial basis function.* Uses the softmax activation function so the activations of all hidden units are normalized to sum to 1. This is the default activation function for all units in the hidden layer.

ORBF. *Ordinary radial basis function.* Uses the exponential activation function so the activation of the hidden unit is a Gaussian "bump" as a function of the inputs.

CRITERIA Subcommand

The CRITERIA subcommand specifies computational settings for the RBF procedure.

OVERLAP Keyword

The OVERLAP keyword specifies the hidden-unit overlapping factor that controls how much overlap occurs among the hidden units.

- Specify AUTO to automatically compute the overlapping factor 1+0.1*d*, where *d* is the number of input units. (The number of input units equals the sum of the number of categories across all factors and the number of covariates.) This is the default.
- Alternatively, specify a number greater than 0 to request a particular overlapping factor.

MISSING Subcommand

The MISSING subcommand is used to control whether user-missing values for categorical variables; that is, factors and categorical dependent variables, are treated as valid values.

- By default, user-missing values for categorical variables are treated as invalid.
- User-missing values for scale variables are always treated as invalid. System-missing values for any variables are always treated as invalid.

USERMISSING = EXCLUDE. *User-missing values for categorical variables are treated as invalid.* This is the default.

USERMISSING = INCLUDE. *User-missing values for categorical variables are treated as valid values.*

PRINT Subcommand

The PRINT subcommand indicates the tabular output to display and can be used to request a sensitivity analysis. If PRINT is not specified, then the default tables are displayed. If PRINT is specified, then only the requested PRINT output is displayed.

CPS Keyword

The CPS keyword displays the case processing summary table, which summarizes the number of cases included and excluded in the analysis, in total and by training, testing, and holdout samples. This table is shown by default.

NETWORKINFO Keyword

The NETWORKINFO keyword displays information about the neural network, including the dependent variables, number of input and output units, number of hidden units, and activation functions. This table is shown by default.

SUMMARY Keyword

The SUMMARY keyword displays a summary of the neural network results, including the error, the relative error or percentage of incorrect predictions, and the training time.

- The error is the sum-of-squares error. In addition, relative errors or percentages of incorrect predictions are displayed, depending on the dependent variable measurement levels. If any dependent variable has scale measurement level, then the average overall relative error (relative to the mean model) is displayed. If all dependent variables are categorical, then the average percentage of incorrect predictions is displayed. Relative errors or percentages of incorrect predictions are also displayed for individual dependent variables.
- Summary results are given for the training data, and for testing and holdout data if they exist.
- This table is shown by default.

CLASSIFICATION Keyword

The CLASSIFICATION keyword displays a classification table for each categorical dependent variable. The table gives the number of cases classified correctly and incorrectly for each dependent variable category.

- In addition to classification tables, the CLASSIFICATION keyword reports the percentage of the total cases that were correctly classified. A case is correctly classified if its highest predicted probabilities correspond to the observed categories for that case.
- Classification results are given for the training data, and for testing and holdout data if they exist.
- Classification results are shown by default.
- The CLASSIFICATION keyword is ignored for scale dependent variables.

SOLUTION Keyword

The SOLUTION keyword displays the center and width vectors for each hidden unit, and the synaptic weight estimates (that is, the coefficient estimates) from the hidden units to the output units. These results are based on the training sample even if the active dataset is partitioned into training, testing, and holdout data.

This table is not shown by default because the number of synaptic weight estimates may be large, and these weights are generally not used for interpretation.

IMPORTANCE Keyword

The IMPORTANCE keyword performs a sensitivity analysis, which computes the importance of each predictor in determining the neural network. The analysis is based on the combined training and testing samples, or only the training sample if there is no testing sample. This keyword creates a table and a chart displaying importance and normalized importance for each predictor.

Sensitivity analysis is not performed by default because it is computationally expensive and time-consuming if there are a large number of predictors or cases.

NONE Keyword

The NONE keyword suppresses all PRINT output. This keyword may not be specified with any other PRINT keywords.

PLOT Subcommand

The PLOT subcommand indicates the chart output to display. If PLOT is not specified, then the default chart (the network diagram) is displayed. If PLOT is specified, then only the requested PLOT output is displayed.

NETWORK Keyword

The NETWORK keyword displays the network diagram. This chart is shown by default.

PREDICTED Keyword

The PREDICTED keyword displays a predicted-by-observed-value chart for each dependent variable. For categorical dependent variables, a boxplot of predicted pseudo-probabilities is displayed. For scale dependent variables, a scatterplot is displayed.

- Charts are based on the combined training and testing samples, or only the training sample if there is no testing sample.

RESIDUAL Keyword

The RESIDUAL keyword displays a residual-by-predicted-value chart for each scale dependent variable.

- This chart is available only for scale dependent variables. The RESIDUAL keyword is ignored for categorical dependent variables.
- Charts are based on the combined training and testing samples, or only the training sample if there is no testing sample.

ROC Keyword

The ROC keyword displays an ROC (Receiver Operating Characteristic) chart for each categorical dependent variable. It also displays a table giving the area under each curve in the chart.

- For a given dependent variable, the ROC chart displays one curve for each category. If the dependent variable has two categories, then each curve treats the category at issue as the positive state versus the other category. If the dependent variable has more than two categories, then each curve treats the category at issue as the positive state versus the aggregate of all other categories.
- This chart is available only for categorical dependent variables. The ROC keyword is ignored for scale dependent variables.
- Charts are based on the combined training and testing samples, or only the training sample if there is no testing sample.

GAIN Keyword

The GAIN keyword displays a cumulative gains chart for each categorical dependent variable.

- The display of one curve for each dependent variable category is the same as for the ROC keyword.
- This chart is available only for categorical dependent variables. The GAIN keyword is ignored for scale dependent variables.
- Charts are based on the combined training and testing samples, or only the training sample if there is no testing sample.

LIFT Keyword

The LIFT keyword displays a lift chart for each categorical dependent variable.

- The display of one curve for each dependent variable category is the same as for the ROC keyword.
- This chart is available only for categorical dependent variables. The LIFT keyword is ignored for scale dependent variables.
- Charts are based on the combined training and testing samples, or only the training sample if there is no testing sample.

NONE Keyword

The NONE keyword suppresses all PLOT output. This keyword may not be specified with any other PLOT keywords.

SAVE Subcommand

The SAVE subcommand writes optional temporary variables to the active dataset.

PREDVAL (varname varname...). *Predicted value or category.* If a dependent variable has scale measurement level, then this is the predicted value. If a dependent variable is categorical, then this is the predicted category.

- Specify one or more unique, valid variable names. There should be as many variable names specified as there are dependent variables, and the names should be listed in the order of the dependent variables on the command line. If you do not specify enough variable names, then default names are used for any remaining variables. If you specify too many variable names, then any remaining names are ignored.
- If there is only one dependent variable, then the default variable name is *RBF_PredictedValue*. If there are multiple dependent variables, then the default variable names are *RBF_PredictedValue_1*, *RBF_PredictedValue_2*, etc., corresponding to the order of the dependent variables on the command line.

PSEUDOPROB (rootname:n rootname...). *Predicted pseudo-probability.* If a dependent variable is categorical, then this keyword saves the predicted pseudo-probabilities of the first *n* categories of that dependent variable.

- Specify one or more unique, valid variable names. There should be as many variable names specified as there are categorical dependent variables, and the names should be listed in the order of the categorical dependent variables on the command line. The specified names are treated as rootnames. Suffixes are added to each rootname to get a group of variable names corresponding to the categories for a given dependent variable. If you do not specify enough variable names, then default names are

used for any remaining categorical dependent variables. If you specify too many variable names, then any remaining names are ignored.

- A colon and a positive integer giving the number of probabilities to save for a dependent variable can follow the rootname.
- If there is only one dependent variable, then the default rootname is *RBF_PseudoProbability*. If there are multiple dependent variables, then the default rootnames are *RBF_PseudoProbability_1*, *RBF_PseudoProbability_2*, etc., corresponding to the order of the categorical dependent variables on the command line and taking into account the position of any scale dependent variables.
- The default *n* is 25.
- This keyword is ignored for scale dependent variables.

Probabilities and Pseudo-Probabilities

Predicted pseudo-probabilities cannot be interpreted as probabilities because the RBF procedure uses the sum-of-squares error and identity activation function for the output layer. The SAVE subcommand saves these predicted pseudo-probabilities even if any are less than 0 or greater than 1, or the sum for a given dependent variable is not 1.

The ROC, cumulative gains, and lift charts (see `/PLOT ROC`, `GAIN`, and `LIFT`, respectively) are created based on pseudo-probabilities. In the event that any of the pseudo-probabilities are less than 0 or greater than 1, or the sum for a given variable is not 1, they are first rescaled to be between 0 and 1 and to sum to 1. The SAVE subcommand saves the original pseudo-probabilities, but the charts are based on rescaled pseudo-probabilities.

Pseudo-probabilities are rescaled by dividing by their sum. For example, if a case has predicted pseudo-probabilities of 0.50, 0.60, and 0.40 for a three-category dependent variable, then each pseudo-probability is divided by the sum 1.50 to get 0.33, 0.40, and 0.27.

If any of the pseudo-probabilities are negative, then the absolute value of the lowest is added to all pseudo-probabilities before the above rescaling. For example, if the pseudo-probabilities are –0.30, 0.50, and 1.30, then first add 0.30 to each value to get 0.00, 0.80, and 1.60. Next, divide each new value by the sum 2.40 to get 0.00, 0.33, and 0.67.

OUTFILE Subcommand

The `OUTFILE` subcommand saves XML-format (PMML) files containing the hidden unit center and width vectors, and the synaptic weight estimates. You can use this model file to apply the model information to other data files for scoring purposes. See the topic "Scoring expressions" on page 86 for more information.

- Filenames must be specified in full. The RBF procedure does not supply extensions.
- The `MODEL` keyword is not honored if split-file processing is in effect (see "SPLIT FILE" on page 1897). If this keyword is specified when split-file processing is on, then a warning is displayed.

MODEL = 'file' 'file'.... *Writes the hidden unit center and width vectors and synaptic weight estimates to XML (PMML) files.* Specify one or more unique, valid filenames. There should be as many filenames as there are dependent variables, and the names should be listed in the order of the dependent variables on the command line. If you do not specify enough filenames, then an error is issued. If you specify too many filenames, then any remaining names are ignored.

If any `'file'` specification refers to an existing file, then the file is overwritten. If any `'file'` specifications refer to the same file, then only the last instance of this `'file'` specification is honored.

READ MODEL

```
READ MODEL FILE='filename'

 [/KEEP={ALL**      }]
        {model names}
        {procedures }

 [/DROP={model names}]
        {procedures }

 [/TYPE={MODEL**}]
        {COMMAND}

 [/TSET={CURRENT**}]
        {RESTORE  }
```

**Default if the subcommand is omitted.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
READ MODEL FILE='ACFMOD.DAT'.
```

Overview

READ MODEL reads a model file that has been previously saved on the SAVE MODEL command (see SAVE MODEL).

Options

You can restore a subset of models from the model file by using the DROP and KEEP subcommands. You can use the TYPE subcommand to control whether models are specified by model name or by the name of the procedure that generated them. With the TSET subcommand, you can restore the TSET settings that were in effect when the model file was created.

Basic Specification

The basic specification is the FILE subcommand, specifying the name of a previously saved model file.

- By default, all models that are contained in the specified file are restored, replacing all models that are currently active. The restored models have their original *MOD_n* default names or names that are assigned by the MODEL NAME command.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- If a subcommand is specified more than once, only the last subcommand is executed.

Operations

- READ MODEL is executed immediately.
- Models that are currently active are erased when READ MODEL is executed. To save these models for later use, specify the SAVE MODEL command before READ MODEL.
- Model files are designed to be read by specific procedures and should not be edited.
- DATE specifications are not saved in model files. Therefore, the DATE specifications from the current session are applied to the restored models.
- The following procedures can generate models that can be read by the READ MODEL command: AREG, ARIMA, EXSMOOTH, SEASON, and SPECTRA in the Forecasting module; ACF, CASEPLOT, CCF, CURVEFIT, PACF, PPLOT, and TSPLOT in the Core system; and WLS and 2SLS in the Regression module.

Limitations

- A maximum of one filename can be specified.
- The READ MODEL command can only read models created by a SAVE MODEL command on the same operating system/platform.

Example

```
READ MODEL FILE='ACFMOD.DAT'
  /DROP=MOD_1.
```

- In this example, all models are restored except *MOD_1* in the model file *ACFMOD.DAT*.

FILE Subcommand

FILE names the model file to be read and is the only required subcommand.

- The only specification on FILE is the name of the model file.
- The filename should be enclosed in quotes.
- Only one filename can be specified.
- Only files that are saved with the SAVE MODEL command can be read.
- You can specify files residing in other directories by supplying a fully qualified filename.

KEEP and DROP Subcommands

KEEP and DROP allow you to restore a subset of models. By default, all models in the model file are restored.

- KEEP specifies the models to be restored.
- DROP specifies the models to be excluded.
- Models can be specified by using individual model names or the names of the procedures that created them. To use procedure names, you must specify COMMAND on the TYPE subcommand.
- Model names are either the default *MOD_n* names or the names that are assigned with MODEL NAME.
- If a procedure name is specified on KEEP, all models that are created by that procedure are restored; on DROP, all models that are created by the procedure are dropped.
- Model names and procedure names cannot be mixed on a single READ MODEL command.
- If more than one KEEP or DROP subcommand is specified, only the last subcommand is executed.
- You can specify the keyword ALL on KEEP to restore all models in the model file. This setting is the default.
- The stored model file is not affected by the KEEP or DROP specification on READ MODEL.

Example

```
READ MODEL FILE='ACFCCF.DAT'
  /KEEP=ACF1 ACF2.
```

- In this example, only models *ACF1* and *ACF2* are restored from model file *ACFCCF.DAT*.

TYPE Subcommand

TYPE indicates whether models are specified by model name or procedure name on DROP and KEEP.

- One keyword, MODEL or COMMAND, can be specified after TYPE.

- MODEL is the default and indicates that models are specified as model names.
- COMMAND indicates that models are specified by procedure name.
- TYPE has no effect if KEEP or DROP is not specified.
- The TYPE specification applies only to the current READ MODEL command.

Example

```
READ MODEL FILE='CURVE1.DAT'
  /KEEP=CURVEFIT
  /TYPE=COMMAND.
```

- In this example, all models that are created by CURVEFIT are restored from model file *CURVE1.DAT*.

TSET Subcommand

TSET allows you to restore the TSET settings that were in effect when the model was created.

- The specification on TSET is either CURRENT or RESTORE.
- CURRENT (the default) indicates that you want to continue to use the current TSET settings.
- RESTORE indicates that you want to restore the TSET settings that were in effect when the model file was saved. The current TSET settings are replaced with the model file settings when the file is restored.

RECODE

For numeric variables:

```
RECODE varlist (value list=newvalue)...(value list=newvalue) [INTO varlist]
       [/varlist...]
```

Input keywords:

`LO, LOWEST, HI, HIGHEST, THRU, MISSING, SYSMIS, ELSE`

Output keywords:

`COPY, SYSMIS`

For string variables:

```
RECODE varlist [('string',['string'...]='newstring')][INTO varlist]
       [/varlist...]
```

Input keywords:

`CONVERT, ELSE`

Output keyword:`

`COPY`

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

Examples

```
RECODE V1 TO V3 (0=1) (1=0) (2,3=-1) (9=9) (ELSE=SYSMIS).
RECODE STRNGVAR ('A','B','C'='A')('D','E','F'='B')(ELSE=' ').
```

- The numeric variables between and including *V1* and *V3* are recoded: original values 0 and 1 are switched respectively to 1 and 0; 2 and 3 are changed to -1; 9 remains 9; and any other value is changed to the system-missing value.
- Values A, B, and C are changed to value A. Values D, E, and F are changed to value B. All other values are changed to a blank.

Overview

RECODE changes, rearranges, or consolidates the values of an existing variable. RECODE can be executed on a value-by-value basis or for a range of values. Where it can be used, RECODE is much more efficient than the series of IF commands that produce the same transformation.

With RECODE, you must specify the new values. Use AUTORECODE to automatically recode the values of string or numeric variables to consecutive integers.

Options

You can generate a new variable as the recoded version of an existing variable using the keyword INTO. You can also use INTO to recode a string variable into a new numeric variable for more efficient processing, or to recode a numeric variable into a new string variable to provide more descriptive values.

Basic Specification

The basic specification is a variable name and, within parentheses, the original values followed by a required equals sign and a new value. RECODE changes the values to the left of the equals sign into the single value to the right of the equals sign.

Syntax Rules

- The variables to be recoded must already exist and must be specified before the value specifications.
- Value specifications are enclosed in parentheses. The original value or values must be specified to the left of an equals sign. A single new value is specified to the right of the equals sign.
- Multiple values can be consolidated into a single recoded value by specifying, to the left of the equals sign, a list of values separated by blanks or commas. Only one recoded value per set is allowed to the right of the equals sign.
- Multiple sets of value specifications are permitted. Each set must be enclosed in parentheses and can result in only one new value.
- To recode multiple variables using the same set of value specifications, specify a variable list before the value specifications. Each variable in the list is recoded identically.
- To recode variables using different value specifications, separate each variable (or variable list) and its specifications from the others by a slash.
- Original values that are not specified remain unchanged unless the keyword ELSE or INTO is used to recode into a new variable. ELSE refers to all original values not previously mentioned, including the system-missing value. ELSE should be the last specification for the variable. When recoding INTO another variable, unspecified values are set to system-missing or blank for strings.
- COPY replicates original values without recoding them.
- INTO is required to recode a string variable into a numeric variable or a numeric variable into a string variable.

Numeric Variables

- Keywords that can be used in the list of original values are LO (or LOWEST), HI (or HIGHEST), THRU, MISSING, SYSMIS, and ELSE. Keywords that can be used in place of a new value are COPY and SYSMIS.
- THRU specifies a value range and includes the specified end values.
- LOWEST and HIGHEST (LO and HI) specify the lowest and highest values encountered in the data. LOWEST and HIGHEST include user-missing values but not the system-missing value.
- MISSING specifies user-missing and system-missing values for recoding. MISSING can be used in the list of original values only.
- SYSMIS specifies the system-missing value and can be used as both an original value and a new value.

String Variables

- Keywords that can be used in the list of original values are CONVERT and ELSE. The only keyword that can be used in place of a new value is COPY.
- Values must be enclosed in quotes.
- Blanks are significant characters.

Operations

- Value specifications are scanned left to right.
- A value is recoded only once per RECODE command.
- Invalid specifications on a RECODE command that result in errors stop all processing of that RECODE command. No variables are recoded.

Numeric Variables

- Blank fields for numeric variables are handled according to the SET BLANKS specification prior to recoding.

- When you recode a value that was previously defined as user-missing on the MISSING VALUE command, the new value is not missing.

String Variables

- If the original or new value specified is shorter than the format width defined for the variable, the string is right-padded with blanks.
- If the original or recoded value specified is longer than the format width defined for that variable, the program issues an error message and RECODE is not executed.

Examples

Recoding Numeric Variables

```
RECODE V1 TO V3 (0=1) (1=0) (2,3=-1) (9=9) (ELSE=SYSMIS)
  /QVAR(1 THRU 5=1)(6 THRU 10=2)(11 THRU HI=3)(ELSE=0).
```

- The numeric variables between and including *V1* and *V3* are recoded: original values 0 and 1 are switched respectively to 1 and 0; 2 and 3 are changed to –1; 9 remains 9; and any other value is changed to the system-missing value.
- Variable *QVAR* is also recoded: original values 1 through 5 are changed to 1; 6 through 10 are changed to 2; 11 through the highest value in the data are changed to 3; and any other value, including system-missing, is changed to 0.

Recoding String Variables

```
RECODE STRNGVAR ('A','B','C'='A')('D','E','F'='B')(ELSE=' ').
RECODE PET ('IGUANA', 'SNAKE ' = 'WILD  ').
```

- Values A, B, and C are changed to value A. Values D, E, and F are changed to value B. All other values are changed to a blank.
- Values IGUANA and SNAKE are changed to value WILD. The defined width of the variable *PET* is 6. Thus, values SNAKE and WILD include trailing blanks for a total of six characters. If blanks are not specified, the values are right-padded. In this example, the results will be the same.
- Each string value is enclosed within quotes.

INTO Keyword

INTO specifies a **target** variable to receive recoded values from the original, or **source**, variable. Source variables remain unchanged after being recoded.

- INTO must follow the value specifications for the source variables that are being recoded into the target variables.
- The number of target variables must equal the number of source variables.

Numeric Variables

- Target variables can be existing or new variables. For existing variables, cases with values not mentioned in the value specifications are not changed. For new variables, cases with values not mentioned are assigned the system-missing value.
- New numeric variables have default print and write formats of F8.2 (or the format specified on SET FORMAT).

Recoding a Single Variable Into a Target Variable

```
RECODE AGE (MISSING=9) (18 THRU HI=1) (0 THRU 18=0) INTO VOTER.
```

- The recoded *AGE* values are stored in target variable *VOTER*, leaving *AGE* unchanged.

- Value 18 and higher values are changed to value 1. Values between 0 and 18, but not including 18, are recoded to 0. If the specification `0 THRU 18` preceded the specification `18 THRU HI`, value 18 would be recoded to 0.

Recording Multiple Variables Into Target Variables

```
RECODE V1 TO V3 (0=1) (1=0) (2=-1) INTO DEFENSE WELFARE HEALTH.
```

- Values for *V1* through *V3* are recoded and stored in *DEFENSE*, *WELFARE*, and *HEALTH*. *V1*, *V2,* and *V3* are not changed.

String Variables

- Target variables must already exist. To create a new string variable, declare the variable with the STRING command before specifying it on RECODE.
- The new string values cannot be longer than the defined width of the target variable.
- If the new values are shorter than the defined width of the target variable, the values are right-padded with blanks.
- Multiple target variables are allowed. The target variables must all be the same defined width; the source variables can have different widths.
- If the source and target variables have different widths, the criterion for the width of the original values is the width defined for the source variable; the criterion for the width of the recoded values is the width defined for the target variable.

Using Keyword COPY With Target Variables

```
STRING STATE1 (A2).
RECODE STATE ('IO'='IA') (ELSE=COPY) INTO STATE1.
```

- STRING declares the variable *STATE1* so that it can be used as a target variable on RECODE.
- RECODE specifies *STATE* as the source variable and *STATE1* as the target variable. The original value IO is recoded to IA. The keywords ELSE and COPY copy all other state codes over unchanged. Thus, *STATE* and *STATE1* are identical except for cases with the original value IO.

Recoding a String Variable Into a Numeric Target

```
RECODE SEX ('M'=1) ('F'=2) INTO NSEX.
```

- RECODE recodes the string variable *SEX* into the numeric variable *NSEX*. Any value other than M or F becomes system-missing.
- The program can process a large number of cases more efficiently with the numeric variable *NSEX* than it can with the string variable *SEX*.

CONVERT Keyword

CONVERT recodes the string representation of numbers to their numeric representation.

- If the keyword CONVERT precedes the value specifications, cases with numbers are recoded immediately and blanks are recoded to the system-missing value, even if you specifically recode blanks into a value.
- To recode blanks to a value other than system-missing or to recode a string value to a noncorresponding numeric value (for example, '0' to 10), you must specify a recode specification *before* the keyword CONVERT.
- RECODE converts numbers as if the variable were being reread using the F format.
- If RECODE encounters a value that cannot be converted, it scans the remaining value specifications. If there is no specific recode specification for that value, the target variable will be system-missing for that case.

Examples

```
RECODE #JOB (CONVERT) ('-'=11) ('&'=12) INTO JOB.
```

- RECODE first recodes all numbers in the string variable *#JOB* to numbers. The target variable is *JOB*.
- RECODE then specifically recodes the minus sign (the “eleven” punch) to 11 and the ampersand (or “twelve” punch in EBCDIC) to 12. The keyword CONVERT is specified first as an efficiency measure to recode cases with numbers immediately. Blanks are recoded to the system-missing value.

  ```
  RECODE #JOB (' '=-99) (CONVERT) ('-'=11) ('&'=12) INTO JOB.
  ```

- The result is the same as in the above example except that blanks are changed to –99.

RECORD TYPE

For mixed file types:

```
RECORD TYPE {value list} [SKIP]
            {OTHER     }
```

For grouped file types:

```
RECORD TYPE {value list} [SKIP] [CASE=col loc]
            {OTHER     }

 [DUPLICATE={WARN   }] [MISSING={WARN   }]
            {NOWARN }            {NOWARN }
```

For nested file types:

```
RECORD TYPE {value list} [SKIP] [CASE=col loc]
            {OTHER     }

 [SPREAD={YES}] [MISSING={WARN   }]
         {NO }           {NOWARN }
```

Example

```
FILE TYPE  MIXED RECORD=RECID 1-2.
RECORD TYPE 23.
DATA LIST   /SEX 5 AGE 6-7 DOSAGE 8-10 RESULT 12.
END FILE TYPE.
```

Overview

RECORD TYPE is used with DATA LIST within a FILE TYPE–END FILE TYPE structure to define any one of the three types of complex raw data files: **mixed files**, which contain several types of records that define different types of cases; **hierarchical** or **nested files**, which contain several types of records with a defined relationship among the record types; or **grouped files**, which contain several records for each case with some records missing or duplicated (see FILE TYPE for more complete information). A fourth type of complex file, files with **repeating groups** of information, can be read with the REPEATING DATA command. REPEATING DATA can also be used to read mixed files and the lowest level of nested files.

Each type of complex file has varying types of records. One set of RECORD TYPE and DATA LIST commands is used to define each type of record in the data. The specifications available for RECORD TYPE vary according to whether MIXED, GROUPED, or NESTED is specified on FILE TYPE.

Basic Specification

For each record type being defined, the basic specification is the value of the record type variable defined on the RECORD subcommand on FILE TYPE.

- RECORD TYPE must be followed by a DATA LIST command defining the variables for the specified records, unless SKIP is used.
- One pair of RECORD TYPE and DATA LIST commands must be used for each defined record type.

Syntax Rules

- A list of values can be specified if a set of different record types has the same variable definitions. Each value must be separated by a space or comma.
- String values must be enclosed in quotes.
- For mixed files, each DATA LIST can specify variables with the same variable name, since each record type defines a separate case. For grouped and nested files, the variable names on each DATA LIST must be unique, since a case is built by combining all record types together onto a single record.

- For mixed files, if the same variable is defined for more than one record type, the format type and width of the variable should be the same on all DATA LIST commands. The program refers to the first DATA LIST command that defines a variable for the print and write formats to include in the dictionary of the active dataset.
- For nested files, the order of the RECORD TYPE commands defines the hierarchical structure of the file. The first RECORD TYPE defines the highest-level record type, the next RECORD TYPE defines the next highest-level record, and so forth. The last RECORD TYPE command defines a case in the active dataset.

Operations

- If a record type is specified on more than one RECORD TYPE command, the program uses the DATA LIST command associated with the first specification and ignores all others.
- For NESTED files, the first record in the file should be the type specified on the first RECORD TYPE command—the highest-level record of the hierarchy. If the first record in the file is not the highest-level type, the program skips all records until it encounters a record of the highest-level type. If the MISSING or DUPLICATE subcommands have been specified on the FILE TYPE command, these records may produce warning messages but will not be used to build a case in the active dataset.

Examples

Reading a Single Record Type From a Mixed File

```
FILE TYPE  MIXED RECORD=RECID 1-2.
RECORD TYPE 23.
DATA LIST   /SEX 5 AGE 6-7 DOSAGE 8-10 RESULT 12.
END FILE TYPE.

BEGIN DATA
21  145010 1
22  257200 2
25  235  250  2
35  167          300    3
24  125150 1
23  272075 1
21  149050 2
25  134  035  3
30  138          300    3
32  229          500    3
END DATA.
```

- FILE TYPE begins the file definition, and END FILE TYPE indicates the end of file definition. FILE TYPE specifies a mixed file type. Since the data are included between BEGIN DATA–END DATA, the FILE subcommand is omitted. The record identification variable *RECID* is located in columns 1 and 2.
- RECORD TYPE indicates that records with value 23 for variable *RECID* will be copied into the active dataset. All other records are skipped. The program does not issue a warning when it skips records in mixed files.
- DATA LIST defines variables on records with the value 23 for variable *RECID*.

Reading Multiple Record Types From a Mixed File

```
FILE TYPE  MIXED FILE=TREATMNT RECORD=RECID 1-2.
+ RECORD TYPE 21,22,23,24.
+ DATA LIST   /SEX 5 AGE 6-7 DOSAGE 8-10 RESULT 12.
+ RECORD TYPE 25.
+ DATA LIST   /SEX 5 AGE 6-7 DOSAGE 10-12 RESULT 15.
END FILE TYPE.
```

- Variable *DOSAGE* is read from columns 8–10 for record types 21, 22, 23, and 24 and from columns 10–12 for record type 25. *RESULT* is read from column 12 for record types 21, 22, 23, and 24 and from column 15 for record type 25.
- The active dataset contains values for all variables defined on the DATA LIST commands for record types 21 through 25. All other record types are skipped.

Working With Nested Files

```
* A nested file of accident records.

FILE TYPE NESTED RECORD=6 CASE=ACCID 1-4.
RECORD TYPE 1.
```

```
DATA LIST /ACC_ID 9-11 WEATHER 12-13 STATE 15-16 (A) DATE 18-24 (A).
RECORD TYPE 2.
DATA LIST /STYLE 11 MAKE 13 OLD 14 LICENSE 15-16(A) INSURNCE 18-21 (A).
RECORD TYPE 3.
DATA LIST /PSNGR_NO 11 AGE 13-14 SEX 16 (A) INJURY 18 SEAT 20-21 (A)
          COST 23-24.
END FILE TYPE.

BEGIN DATA
0001 1  322 1 IL 3/13/88   /* Type 1:  accident record
0001 2    1 44MI 134M      /* Type 2:    vehicle record
0001 3    1 34 M 1 FR  3   /* Type 3:      person record
0001 2    2 16IL 322F      /*            vehicle record
0001 3    1 22 F 1 FR 11   /*              person record
0001 3    2 35 M 1 FR  5   /*              person record
0001 3    3 59 M 1 BK  7   /*              person record
0001 2    3 21IN 146M      /*            vehicle record
0001 3    1 46 M 0 FR  0   /*              person record
END DATA.
```

- FILE TYPE specifies a nested file type. The record identifier, located in column 6, is not assigned a variable name, so the default scratch variable name *####RECD* is used. The case identification variable *ACCID* is located in columns 1–4.
- Because there are three record types, there are three RECORD TYPE commands. For each RECORD TYPE, there is a DATA LIST command to define variables on that record type. The order of the RECORD TYPE commands defines the hierarchical structure of the file.
- END FILE TYPE signals the end of file definition.
- The program builds a case for each lowest-level (type 3) record, representing each person in the file. There can be only one type 1 record for each type 2 record, and one type 2 record for each type 3 record. Each vehicle can be in only one accident, and each person can be in only one vehicle. The variables from the type 1 and type 2 records are spread to their corresponding type 3 records.

OTHER Keyword

OTHER specifies all record types that have not been mentioned on previous RECORD TYPE commands.

- OTHER can be specified only on the last RECORD TYPE command in the file definition.
- OTHER can be used with SKIP to skip all undefined record types.
- For nested files, OTHER can be used only with SKIP. Neither can be used separately.
- If WILD=WARN is in effect for the FILE TYPE command, OTHER cannot be specified on the RECORD TYPE command.

Using Keyword OTHER With a Mixed File

```
FILE TYPE  MIXED FILE=TREATMNT RECORD=RECID 1-2.
RECORD TYPE 21,22,23,24.
DATA LIST   /SEX 5 AGE 6-7 DOSAGE 8-10 RESULT 12.
RECORD TYPE 25.
DATA LIST   /SEX 5 AGE 6-7 DOSAGE 10-12 RESULT 15.
RECORD TYPE OTHER.
DATA LIST   /SEX 5 AGE 6-7 DOSAGE 18-20 RESULT 25.
END FILE TYPE.
```

- The first two RECORD TYPE commands specify record types 21–25. All other record types are specified by the third RECORD TYPE.

Using Keyword OTHER With a Nested File

```
FILE TYPE NESTED FILE=ACCIDENT RECORD=#RECID 6 CASE=ACCID 1-4.
RECORD TYPE 1.        /* Accident record
DATA LIST   /WEATHER 12-13.
RECORD TYPE 2.        /* Vehicle record
DATA LIST /STYLE 16.
RECORD TYPE OTHER SKIP.
END FILE TYPE.
```

- The third RECORD TYPE specifies OTHER SKIP. Type 2 records are therefore the lowest-level records included in the active dataset. These commands build one case for each vehicle record. The person records are skipped.
- Because the data are in a nested file, OTHER can be specified only with SKIP.

SKIP Subcommand

SKIP specifies record types to skip.

- To skip selected record types, specify the values for the types you want to skip and then specify SKIP. To skip all record types other than those specified on previous RECORD TYPE commands, specify OTHER and then SKIP.
- For nested files, SKIP can be used only with OTHER. Neither can be used separately.
- For grouped files, OTHER cannot be specified on SKIP if WILD=WARN (the default) is in effect for FILE TYPE.
- For mixed files, all record types that are not specified on a RECORD TYPE command are skipped by default. No warning is issued (WILD=NOWARN on FILE TYPE is the default for mixed files).
- For grouped files, a warning message is issued by default for all record types not specified on a RECORD TYPE command (WILD=WARN on FILE TYPE is the default for grouped files). If the record types are explicitly specified on SKIP, no warning is issued.

Examples

```
FILE TYPE GROUPED FILE=HUBDATA RECORD=#RECID 80 CASE=ID 1-5
                               WILD=NOWARN.
RECORD TYPE 1.
DATA LIST   /MOHIRED YRHIRED 12-15 DEPT79 TO DEPT82 SEX 16-20.
RECORD TYPE OTHER SKIP.
END FILE TYPE.
```

- The program reads variables from type 1 records and skips all other types.
- WILD=NOWARN on the FILE TYPE command suppresses the warning messages that is issued by default for undefined record types for grouped files. Keyword OTHER cannot be used when the default WILD=WARN specification is in effect.

```
FILE TYPE GROUPED FILE=HUBDATA RECORD=#RECID 80 CASE=ID 1-5.
RECORD TYPE 1.
DATA LIST   /MOHIRED YRHIRED 12-15 DEPT79 TO DEPT82 SEX 16-20.
RECORD TYPE 2,3 SKIP.
END FILE TYPE.
```

- Record type 1 is defined for each case, and record types 2 and 3 are skipped.
- WILD=WARN (the default) on FILE TYPE GROUPED is in effect. The program therefore issues a warning message for any record types it encounters other than types 1, 2, and 3. No warning is issued for record types 2 and 3 because they are explicitly specified on a RECORD TYPE command.

CASE Subcommand

CASE specifies the column locations of the case identification variable when that variable is not in the location defined by the CASE subcommand on FILE TYPE.

- CASE on RECORD TYPE applies only to those records specified by that RECORD TYPE command. The identifier for record types without CASE on RECORD TYPE must be in the location specified by CASE on FILE TYPE.
- CASE can be used for nested and grouped files only. CASE cannot be used for mixed files.
- CASE can be used on RECORD TYPE only if a CASE subcommand is specified on FILE TYPE.
- The format type of the case identification variable must be the same on all records, and the same format must be assigned on the RECORD TYPE and FILE TYPE commands. For example, if the case identification variable is defined as a string on FILE TYPE, it cannot be defined as a numeric variable on RECORD TYPE.

Example

```
* Specifying case on the record type command for a grouped file.

FILE TYPE GROUPED FILE=HUBDATA RECORD=#RECID 80 CASE=ID 1-5.
RECORD TYPE 1.
```

```
DATA LIST   /MOHIRED YRHIRED 12-15 DEPT79 TO DEPT82 SEX 16-20.
RECORD TYPE 2.
DATA LIST   /SALARY79 TO SALARY82 6-25
             HOURLY81 HOURLY82 40-53 (2)
             PROMO81 72  AGE 54-55 RAISE82 66-70.
RECORD TYPE 3  CASE=75-79.
DATA LIST   /JOBCAT 6 NAME 25-48 (A).
END FILE TYPE.
```

- CASE on FILE TYPE indicates that the case identification variable is located in columns 1–5. On the third RECORD TYPE command, the CASE subcommand overrides the identifier location for type 3 records. For type 3 records, the case identification variable is located in columns 75–79.

MISSING Subcommand

MISSING controls whether the program issues a warning when it encounters a missing record type for a case. Regardless of whether the program issues the warning, it builds the case in the active dataset with system-missing values for the variables defined on the missing record.

- The only specification is a single keyword. NOWARN is the default for nested files. WARN is the default for grouped files. MISSING cannot be used with MIXED files.
- MISSING on RECORD TYPE applies only to those records specified by that RECORD TYPE command. The treatment of missing records for record types without the MISSING specification on RECORD TYPE is determined by the MISSING subcommand on FILE TYPE.
- For grouped files, the program checks whether there is a record for each case identification number. For nested files, the program verifies that each defined case includes one record of each type.

WARN. *Issue a warning message when a record type is missing for a case.* This is the default for grouped files.

NOWARN. *Suppress the warning message when a record type is missing for a case.* This is the default for nested files.

Example

```
FILE TYPE GROUPED FILE=HUBDATA RECORD=#RECID 80 CASE=ID 1-5.
RECORD TYPE 1.
DATA LIST   /MOHIRED YRHIRED 12-15 DEPT79 TO DEPT82 SEX 16-20.
RECORD TYPE 2  MISSING=NOWARN.
DATA LIST   /SALARY79 TO SALARY82 6-25
  HOURLY81 HOURLY82 40-53 (2) PROMO81 72 AGE 54-55 RAISE82 66-70.
RECORD TYPE 3.
DATA LIST   /JOBCAT 6 NAME 25-48 (A).
END FILE TYPE.
```

- MISSING is not specified on FILE TYPE. Therefore the default MISSING=WARN is in effect for all record types.
- MISSING=NOWARN is specified on the second RECORD TYPE, overriding the default setting for type 2 records. WARN is still in effect for type 1 and type 3 records.

DUPLICATE Subcommand

DUPLICATE controls whether the program issues a warning when it encounters more than one record of each type for a single case.

- DUPLICATE on RECORD TYPE can be used for grouped files only. DUPLICATE cannot be used for mixed or nested files.
- The only specification is a single keyword. WARN is the default.
- DUPLICATE on RECORD TYPE applies only to those records specified by that RECORD TYPE command. The treatment of duplicate records for record types without DUPLICATE specification is determined by the DUPLICATE subcommand on FILE TYPE.
- Regardless of the specification on DUPLICATE, only the last record from a set of duplicates is included in the active dataset.

WARN. *Issue a warning message.* The program issues a message and the first 80 characters of the last record of the duplicate set of record types. This is the default.

NOWARN. *Suppress the warning message.*

Example

```
* Specifying DUPLICATE on RECORD TYPE for a grouped file.

FILE TYPE GROUPED FILE=HUBDATA RECORD=#RECID 80 CASE=ID 1-5.
RECORD TYPE 1.
DATA LIST   /MOHIRED YRHIRED 12-15 DEPT79 TO DEPT82 SEX 16-20.
RECORD TYPE 2  DUPLICATE=NOWARN.
DATA LIST   /SALARY79 TO SALARY82 6-25
  HOURLY81 HOURLY82 40-53 (2) PROMO81 72  AGE 54-55 RAISE82 66-70.
RECORD TYPE 3.
DATA LIST   /JOBCAT 6 NAME 25-48 (A).
END FILE TYPE.
```

- DUPLICATE is not specified on FILE TYPE. Therefore the default DUPLICATE=WARN is in effect for all record types.
- DUPLICATE=NOWARN is specified on the second RECORD TYPE, overriding the FILE TYPE setting for type 2 records. WARN is still in effect for type 1 and type 3 records.

SPREAD Subcommand

SPREAD controls whether the values for variables defined for a record type are spread to all related cases.

- SPREAD can be used for nested files only. SPREAD cannot be used for mixed or grouped files.
- The only specification is a single keyword. YES is the default.
- SPREAD=NO applies only to the record type specified on that RECORD TYPE command. The default YES is in effect for all other defined record types.

YES. *Spread the values from the specified record type to all related cases.* This is the default.

NO. *Spread the values from the specified type only to the first related case.* All other cases built from the same record are assigned the system-missing value for the variables defined on the record type.

Example

```
* A nested file.

FILE TYPE NESTED RECORD=#RECID 6 CASE=ACCID 1-4.
RECORD TYPE 1.
DATA LIST   /ACC_NO 9-11 WEATHER 12-13
             STATE 15-16 (A) DATE 18-24 (A).
RECORD TYPE 2  SPREAD=NO.
DATA LIST /STYLE 11 MAKE 13 OLD 14
           LICENSE 15-16 (A) INSURNCE 18-21 (A).
RECORD TYPE 3.
DATA LIST /PSNGR_NO 11 AGE 13-14 SEX 16 (A)
       INJURY 18 SEAT 20-21 (A) COST 23-24.
END FILE TYPE.

BEGIN DATA
0001 1  322 1 IL 3/13/88   /* Type 1:  accident record
0001 2    1 44MI 134M      /* Type 2:    vehicle record
0001 3    1 34 M 1 FR  3   /* Type 3:       person record
0001 2    2 16IL 322F      /*            vehicle record
0001 3    1 22 F 1 FR 11   /*               person record
0001 3    2 35 M 1 FR  5   /*               person record
0001 3    3 59 M 1 BK  7   /*               person record
0001 2    3 21IN 146M      /*            vehicle record
0001 3    1 46 M 0 FR  0   /*               person record
END DATA.
```

- The accident record (type 1) is spread to all related cases (in this example, all cases).
- The first vehicle record has one related person record. The values for *STYLE, MAKE, OLD, LICENSE*, and *INSURNCE* are spread to the case built for the person record.
- The second vehicle record has three related person records. The values for *STYLE, MAKE, OLD, LICENSE,* and *INSURNCE* are spread only to the case built from the first person record. The other two cases have the system-missing values for *STYLE, MAKE, OLD, LICENSE,* and *INSURNCE*.
- The third vehicle record has one related person record, and the values for type 2 records are spread to that case.

REFORMAT

This command is no longer supported.

REGRESSION

REGRESSION is available in Statistics Base Edition.

```
REGRESSION [MATRIX=[IN({file})]  [OUT({file})]]
                    {*   }         {*   }

  [/VARIABLES={varlist    }]
              {(COLLECT)**}
              {ALL        }

  [/DESCRIPTIVES=[DEFAULTS] [MEAN] [STDDEV] [CORR] [COV]
                 [VARIANCE] [XPROD] [SIG] [N] [BADCORR]
                 [ALL] [NONE**]]

  [/SELECT={varname relation value}

  [/MISSING=[{LISTWISE**      }] [INCLUDE]]
            {PAIRWISE         }
            {MEANSUBSTITUTION}

  [/REGWGT=varname]

  [/STATISTICS=[DEFAULTS**] [R**] [COEFF**] [ANOVA**] [OUTS**]
               [ZPP] [LABEL] [CHANGE] [CI({95   })] [F] [BCOV] [SES]
                                         {value}
               [XTX] [COLLIN] [TOL] [SELECTION] [ALL]]

  [/CRITERIA=[DEFAULTS**] [TOLERANCE({0.0001**})] [MAXSTEPS(n)]
                                     {value   }

             [PIN[({0.05**})]] [POUT[({0.10**})]]
                   {value }          {value }

             [FIN[({3.84 })]] [FOUT[({2.71 })]]
                   {value}          {value}

             [CIN[({ 95**})]]]
                   {value}

 [/{NOORIGIN**}]
   {ORIGIN    }

  /DEPENDENT=varlist

  [/METHOD=]{STEPWISE [varlist]   } [...] [/...]
            {FORWARD [varlist]    }
            {BACKWARD [varlist]   }
            {ENTER [varlist]      }
            {REMOVE varlist       }
            {TEST(varlist)(varlist)...}

 [/RESIDUALS=[DEFAULTS] [DURBIN] [OUTLIERS({ZRESID  })] [ID (varname)]
                                           {tempvars}

       [NORMPROB({ZRESID  })] [HISTOGRAM({ZRESID  })]
                 {tempvars}             {tempvars}

       [SIZE({SEPARATE}]
             {POOLED  }

 [/CASEWISE=[DEFAULTS]  [{OUTLIERS({3    })}] [PLOT({ZRESID })]
                          {         {value} }        {tempvar}
                          {ALL              }

       [{DEPENDENT PRED RESID}]]
        {tempvars            }

 [/SCATTERPLOT [varname,varname]...[

 [/PARTIALPLOT=[{ALL    }]
               {varlist}

 [/OUTFILE={COVB ('savfile'|'dataset')}]  [{MODEL('file')    }]
           {CORB ('savfile'|'dataset')}    {PARAMETER('file')}

 [/SAVE=tempvar[(newname)]  [tempvar[(newname)]...]  [FITS]]

 [/TEMPLATE='filename']
```

**Default if the subcommand is omitted.

Temporary residual variables are:

PRED, ADJPRED, SRESID, MAHAL, RESID, ZPRED, SDRESID, COOK, DRESID, ZRESID, SEPRED, LEVER, DFBETA, SDBETA, DFFIT, SDFFIT, COVRATIO, MCIN, ICIN

SAVE FITS saves:

DFFIT, SDFIT, DFBETA, SDBETA, COVRATIO

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 13.0

- PARAMETER keyword introduced on OUTFILE subcommand.

Release 16.0

- Added support for SET THREADS and SET MCACHE.

Release 17.0

- Added option to specify confidence level on CI keyword of STATISTICS subcommand.

Release 19.0

- TEMPLATE subcommand introduced.

Example

```
REGRESSION VARIABLES=POP15,POP75,INCOME,GROWTH,SAVINGS
  /DEPENDENT=SAVINGS
  /METHOD=ENTER POP15,POP75,INCOME
  /METHOD=ENTER GROWTH.
```

Overview

REGRESSION calculates multiple regression equations and associated statistics and plots. REGRESSION also calculates collinearity diagnostics, predicted values, residuals, measures of fit and influence, and several statistics based on these measures.

Options

Input and Output Control Subcommands. DESCRIPTIVES requests descriptive statistics on the variables in the analysis. SELECT estimates the model based on a subset of cases. REGWGT specifies a weight variable for estimating weighted least-squares models. MISSING specifies the treatment of cases with missing values. MATRIX reads and writes matrix data files.

Equation-Control Subcommands. These optional subcommands control the calculation and display of statistics for each equation. STATISTICS controls the statistics displayed for the equation(s) and the independent variable(s), CRITERIA specifies the criteria used by the variable selection method, and ORIGIN specifies whether regression is through the origin.

Analysis of Residuals, Fit, and Influence. REGRESSION creates temporary variables containing predicted values, residuals, measures of fit and influence, and several statistics based on these measures. These temporary variables can be analyzed within REGRESSION in Casewise Diagnostics tables (CASEWISE subcommand), scatterplots (SCATTERPLOT subcommand), histograms and normal probability plots (RESIDUALS subcommand), and partial regression plots (PARTIALPLOT subcommand). Any of the residuals subcommands can be specified to obtain descriptive statistics for the predicted values, residuals, and their standardized versions. Any of the temporary variables can be added to the active dataset with the SAVE subcommand.

Templates. You can specify a template, using the TEMPLATE subcommand, to override the default chart attribute settings on your system.

Basic Specification

The basic specification is DEPENDENT, which initiates the equation(s) and defines at least one dependent variable, followed by METHOD, which specifies the method for selecting independent variables.

- By default, all variables named on DEPENDENT and METHOD are used in the analysis.
- The default display for each equation includes a Model Summary table showing R^2, an ANOVA table, a Coefficients table displaying related statistics for variables in the equation, and an Excluded Variables table displaying related statistics for variables not yet in the equation.

- By default, all cases in the active dataset with valid values for all selected variables are used to compute the correlation matrix on which the regression equations are based. The default equations include a constant (intercept).
- All residuals analysis subcommands are optional. Most have defaults that can be requested by including the subcommand without any further specifications. These defaults are described in the section for each subcommand.

Subcommand Order

The standard subcommand order for REGRESSION is

```
REGRESSION MATRIX=...
   /VARIABLES=...
   /DESCRIPTIVES=...
   /SELECT=...
   /MISSING=...
   /REGWGT=...

            --Equation Block--
   /STATISTICS=...
   /CRITERIA=...
   /ORIGIN
   /DEPENDENT=...

            --Method Block(s)--
   /METHOD=...
   [/METHOD=...]

            --Residuals Block--
   /RESIDUALS=...
   /SAVE=...
   /CASEWISE=...
   /SCATTERPLOT=...
   /PARTIALPLOT=...
   /OUTFILE=...
```

- When used, MATRIX must be specified first.
- Subcommands listed before the equation block must be specified before any subcommands within the block.
- Only one equation block is allowed per REGRESSION command.
- An equation block can contain multiple METHOD subcommands. These methods are applied, one after the other, to the estimation of the equation for that block.
- The STATISTICS, CRITERIA, and ORIGIN/NOORIGIN subcommands must precede the DEPENDENT subcommand.
- The residuals subcommands RESIDUALS, CASEWISE, SCATTERPLOT, and PARTIALPLOT follow the last METHOD subcommand of any equation for which residuals analysis is requested. Statistics are based on this final equation.
- Residuals subcommands can be specified in any order. All residuals subcommands must follow the DEPENDENT and METHOD subcommands.

Syntax Rules

- VARIABLES can be specified only once. If omitted, VARIABLES defaults to COLLECT.
- The DEPENDENT subcommand can be specified only once and must be followed immediately by one or more METHOD subcommands.
- CRITERIA, STATISTICS, and ORIGIN must be specified before DEPENDENT and METHOD. If any of these subcommands are specified more than once, only the last specified is in effect for all subsequent equations.
- More than one variable can be specified on the DEPENDENT subcommand. An equation is estimated for each.
- If no variables are specified on METHOD, all variables named on VARIABLES but not on DEPENDENT are considered for selection.

Operations

- This procedure uses the multithreaded options specified by SET THREADS and SET MCACHE.

Operations

- REGRESSION calculates a correlation matrix that includes all variables named on VARIABLES. All equations requested on the REGRESSION command are calculated from the same correlation matrix.
- The MISSING, DESCRIPTIVES, and SELECT subcommands control the calculation of the correlation matrix and associated displays.
- If multiple METHOD subcommands are specified, they operate in sequence on the equations defined by the preceding DEPENDENT subcommand.
- Only independent variables that pass the tolerance criterion are candidates for entry into the equation. See the topic "CRITERIA Subcommand " on page 1641 for more information.
- The temporary variables *PRED* (unstandardized predicted value), *ZPRED* (standardized predicted value), *RESID* (unstandardized residual), and *ZRESID* (standardized residual) are calculated and descriptive statistics are displayed whenever any residuals subcommand is specified. If any of the other temporary variables are referred to on the command, they are also calculated.
- Predicted values and statistics based on predicted values are calculated for every observation that has valid values for all variables in the equation. Residuals and statistics based on residuals are calculated for all observations that have a valid predicted value and a valid value for the dependent variable. The missing-values option therefore affects the calculation of residuals and predicted values.
- No residuals or predictors are generated for cases deleted from the active dataset with SELECT IF, a temporary SELECT IF, or SAMPLE.
- All variables are standardized before plotting. If the unstandardized version of a variable is requested, the standardized version is plotted.
- Residuals processing is not available when the active dataset is a matrix file or is replaced by a matrix file with MATRIX OUT(*) on REGRESSION. If RESIDUALS, CASEWISE, SCATTERPLOT, PARTIALPLOT, or SAVE are used when MATRIX IN(*) or MATRIX OUT(*) is specified, the REGRESSION command is not executed.

For each analysis, REGRESSION can calculate the following types of temporary variables:

PRED. *Unstandardized predicted values*.

RESID. *Unstandardized residuals*.

DRESID. *Deleted residuals*.

ADJPRED. *Adjusted predicted values*.

ZPRED. *Standardized predicted values*.

ZRESID. *Standardized residuals*.

SRESID. *Studentized residuals*.

SDRESID. *Studentized deleted residuals*. [64]

SEPRED. *Standard errors of the predicted values*.

MAHAL. *Mahalanobis distances*.

COOK. *Cook's distances*. [65]

LEVER. *Centered leverage values*. [66]

DFBETA. *Change in the regression coefficient that results from the deletion of the* i*th case.* A DFBETA value is computed for each case for each regression coefficient generated by a model. [67]

[64] Hoaglin, D. C., and R. E. Welsch. 1978. The hat matrix in regression and ANOVA. *American Statistician,* 32, 17-22.

[65] Cook, R. D. 1977. Detection of influential observations in linear regression. *Technometrics,* 19, 15-18.

[66] Velleman, P. F., and R. E. Welsch. 1981. Efficient computing of regression diagnostics. *American Statistician,* 35, 234-242.

[67] Belsley, D. A., E. Kuh, and R. E. Welsch. 1980. *Regression diagnostics: Identifying influential data and sources of collinearity*. New York: John Wiley and Sons.

SDBETA. *Standardized DFBETA.* An SDBETA value is computed for each case for each regression coefficient generated by a model. [68]

DFFIT. *Change in the predicted value when the ith case is deleted.* [69]

SDFIT. *Standardized DFFIT.* [70]

COVRATIO. *Ratio of the determinant of the covariance matrix with the ith case deleted to the determinant of the covariance matrix with all cases included.* [71]

MCIN. *Lower and upper bounds for the prediction interval of the mean predicted response.* A lowerbound *LMCIN* and an upperbound *UMCIN* are generated. The default confidence interval is 95%. The confidence interval can be reset with the CIN subcommand. [72]

ICIN. *Lower and upper bounds for the prediction interval for a single observation.* A lowerbound *LICIN* and an upperbound *UICIN* are generated. The default confidence interval is 95%. The confidence interval can be reset with the CIN subcommand. [73]

Examples

Multiple METHOD subcommands

```
REGRESSION VARIABLES=POP15,POP75,INCOME,GROWTH,SAVINGS
 /DEPENDENT=SAVINGS
 /METHOD=ENTER POP15,POP75,INCOME
 /METHOD=ENTER GROWTH.
```

- VARIABLES calculates a correlation matrix of five variables for use by REGRESSION.
- DEPENDENT defines a single equation, with *SAVINGS* as the dependent variable.
- The first METHOD subcommand enters *POP15, POP75,* and *INCOME* into the equation.
- The second METHOD subcommand adds *GROWTH* to the equation containing *POP15* to *INCOME*.

VARIABLES Subcommand

VARIABLES names all the variables to be used in the analysis.

- The minimum specification is a list of two variables or the keyword ALL or COLLECT. COLLECT, which must be specified in parentheses, is the default.
- Only one VARIABLES subcommand is allowed, and it must precede any DEPENDENT or METHOD subcommands.
- You can use keyword TO to refer to consecutive variables in the active dataset.
- The order of variables in the correlation matrix constructed by REGRESSION is the same as their order on VARIABLES. If (COLLECT) is used, the order of variables in the correlation matrix is the order in which they are first listed on the DEPENDENT and METHOD subcommands.

ALL. *Include all user-defined variables in the active dataset.*

---

[68] Belsley, D. A., E. Kuh, and R. E. Welsch. 1980. *Regression diagnostics: Identifying influential data and sources of collinearity*. New York: John Wiley and Sons.

[69] Belsley, D. A., E. Kuh, and R. E. Welsch. 1980. *Regression diagnostics: Identifying influential data and sources of collinearity*. New York: John Wiley and Sons.

[70] Belsley, D. A., E. Kuh, and R. E. Welsch. 1980. *Regression diagnostics: Identifying influential data and sources of collinearity*. New York: John Wiley and Sons.

[71] Belsley, D. A., E. Kuh, and R. E. Welsch. 1980. *Regression diagnostics: Identifying influential data and sources of collinearity*. New York: John Wiley and Sons.

[72] Dillon, W. R., and M. Goldstein. 1984. *Multivariate analysis: Methods and applications*. New York: John Wiley and Sons.

[73] Dillon, W. R., and M. Goldstein. 1984. *Multivariate analysis: Methods and applications*. New York: John Wiley and Sons.

(COLLECT). *Include all variables named on the DEPENDENT and METHOD subcommands.* COLLECT is the default if the VARIABLES subcommand is omitted. COLLECT must be specified in parentheses. If COLLECT is used, the METHOD subcommands must specify variable lists.

Example

```
REGRESSION VARIABLES=(COLLECT)
 /DEPENDENT=SAVINGS
 /METHOD=STEP POP15 POP75 INCOME
 /METHOD=ENTER GROWTH.
```

- COLLECT requests that the correlation matrix include *SAVINGS, POP15, POP75, INCOME,* and *GROWTH*. Since COLLECT is the default, the VARIABLES subcommand could have been omitted.
- The DEPENDENT subcommand defines a single equation in which *SAVINGS* is the dependent variable.
- The first METHOD subcommand requests that the block of variables *POP15, POP75,* and *INCOME* be considered for inclusion using a stepwise procedure.
- The second METHOD subcommand adds variable *GROWTH* to the equation.

DEPENDENT Subcommand

DEPENDENT specifies a list of variables and requests that an equation be built for each. DEPENDENT is required.

- The minimum specification is a single variable. There is no default variable list.
- Only one DEPENDENT subcommand can be specified. It must be followed by at least one METHOD subcommand.
- Keyword TO on a DEPENDENT subcommand refers to the order in which variables are specified on the VARIABLES subcommand. If VARIABLES=(COLLECT), TO refers to the order of variables in the active dataset.
- If DEPENDENT names more than one variable, an equation is built for each using the same independent variables and methods.

METHOD Subcommand

METHOD specifies a variable selection method and names a block of variables to be evaluated using that method. METHOD is required.

- The minimum specification is a method keyword and, for some methods, a list of variables. The actual keyword METHOD can be omitted.
- When more than one METHOD subcommand is specified, each METHOD subcommand is applied to the equation that resulted from the previous METHOD subcommands.
- The default variable list for methods FORWARD, BACKWARD, STEPWISE, and ENTER consists of all variables named on VARIABLES that are not named on the DEPENDENT subcommand. If VARIABLES=(COLLECT), the variables must be specified for these methods.
- There is no default variable list for the REMOVE and TEST methods.
- Keyword TO in a variable list on METHOD refers to the order in which variables are specified on the VARIABLES subcommand. If VARIABLES=(COLLECT), TO refers to the order of variables in the active dataset.

The available stepwise methods are as follows:

BACKWARD [varlist]. *Backward elimination.* Variables in the block are considered for removal. At each step, the variable with the largest probability-of-*F* value is removed, provided that the value is larger than POUT. See the topic “CRITERIA Subcommand ” on page 1641 for more information. If no variables are in the equation when BACKWARD is specified, all independent variables in the block are first entered.

FORWARD [varlist]. *Forward entry.* Variables in the block are added to the equation one at a time. At each step, the variable not in the equation with the smallest probability of *F* is entered if the value is smaller than PIN. See the topic "CRITERIA Subcommand " on page 1641 for more information.

STEPWISE [varlist]. *Stepwise selection.* If there are independent variables already in the equation, the variable with the largest probability of *F* is removed if the value is larger than POUT. The equation is recomputed without the variable and the process is repeated until no more independent variables can be removed. Then, the independent variable not in the equation with the smallest probability of *F* is entered if the value is smaller than PIN. All variables in the equation are again examined for removal. This process continues until no variables in the equation can be removed and no variables not in the equation are eligible for entry, or until the maximum number of steps has been reached. See the topic "CRITERIA Subcommand " on page 1641 for more information.

The methods that enter or remove the entire variable block in a single step are as follows:

ENTER [varlist]. *Forced entry.* All variables specified are entered in a single step in order of decreasing tolerance. You can control the order in which variables are entered by specifying the variables on multiple METHOD=ENTER subcommands.

REMOVE varlist. *Forced removal.* All variables specified are removed in a single step. REMOVE requires a variable list.

TEST (varlist) (varlist). *R^2 change and its significance for sets of independent variables.* This method first adds all variables specified on TEST to the current equation. It then removes in turn each subset from the equation and displays requested statistics. Specify test subsets in parentheses. A variable can be used in more than one subset, and each subset can include any number of variables. Variables named on TEST remain in the equation when the method is completed.

Example

```
REGRESSION VARIABLES=POP15 TO GROWTH, SAVINGS
 /DEPENDENT=SAVINGS
 /METHOD=STEPWISE
 /METHOD=ENTER.
```

- STEPWISE applies the stepwise procedure to variables *POP15* to *GROWTH*.
- All variables not in the equation when the STEPWISE method is completed will be forced into the equation with ENTER.

Example

```
REGRESSION VARIABLES=(COLLECT)
 /DEPENDENT=SAVINGS
 /METHOD=TEST(MEASURE3 TO MEASURE9)(MEASURE3,INCOME)
 /METHOD=ENTER GROWTH.
```

- The VARIABLES=(COLLECT) specification assembles a correlation matrix that includes all variables named on the DEPENDENT and METHOD subcommands.
- REGRESSION first builds the full equation of all the variables named on the first METHOD subcommand: *SAVINGS* regressed on *MEASURE3* to *MEASURE9* and *INCOME*. For each set of test variables (MEASURE3 to MEASURE9, and MEASURE3 and INCOME), the R^2 change, *F*, probability, sums of squares, and degrees of freedom are displayed.
- *GROWTH* is added to the equation by the second METHOD subcommand. Variables *MEASURE3* to *MEASURE9* and *INCOME* are still in the equation when this subcommand is executed.

STATISTICS Subcommand

STATISTICS controls the display of statistics for the equation and for the independent variables.

- If STATISTICS is omitted or if it is specified without keywords, R, ANOVA, COEFF, and OUTS are displayed.
- If any statistics are specified on STATISTICS, only those statistics specifically requested are displayed.

- STATISTICS must be specified before DEPENDENT and METHOD subcommands. The last specified STATISTICS affects all equations.

Global Statistics

DEFAULTS. *R, ANOVA, COEFF, and OUTS.* These are displayed if STATISTICS is omitted or if it is specified without keywords.

ALL. *All statistics except F.*

Equation Statistics

R. *Multiple R.* R includes R^2, adjusted R^2, and standard error of the estimate displayed in the Model Summary table.

ANOVA. *Analysis of variance table.* This option includes regression and residual sums of squares, mean square, *F*, and probability of *F* displayed in the ANOVA table.

CHANGE. *Change in* R^2. This option includes the change in R^2 between steps, along with the corresponding *F* and its probability, in the Model Summary table. For each equation, *F* and its probability are also displayed.

BCOV. *Variance-covariance matrix for unstandardized regression coefficients.* The statistics are displayed in the Coefficient Correlations table.

XTX. *Swept correlation matrix*.

COLLIN. *Collinearity diagnostics* [74]. COLLIN includes the variance-inflation factors (VIF) displayed in the Coefficients table, and the eigenvalues of the scaled and uncentered cross-products matrix, condition indexes, and variance-decomposition proportions displayed in the Collinearity Diagnostics table.

SELECTION. *Selection statistics.* This option includes Akaike information criterion (AIK), Ameniya's prediction criterion (PC), Mallows conditional mean squared error of prediction criterion (C_p), and Schwarz Bayesian criterion (SBC) [75]. The statistics are displayed in the Model Summary table.

Statistics for the Independent Variables

COEFF. *Regression coefficients.* This option includes regression coefficients (B), standard errors of the coefficients, standardized regression coefficients (beta), *t*, and two-tailed probability of *t*. The statistics are displayed in the Coefficients table.

OUTS. *Statistics for variables not yet in the equation that have been named on METHOD subcommands for the equation.* OUTS displays the Excluded Variables table showing beta, *t,* two-tailed probability of *t,* and minimum tolerance of the variable if it were the only variable entered next.

ZPP. *Zero-order, part, and partial correlation*. The statistics are displayed in the Coefficients table.

CI(value). *value% confidence interval for the unstandardized regression coefficients.* The statistics are displayed in the Coefficients table. If the optional value is unspecified, it defaults to 95. If specified, the value must be greater than 0 and less than or equal to 100.

SES. *Approximate standard error of the standardized regression coefficients.* [76] The statistics are displayed in the Coefficients table.

TOL. *Tolerance.* This option displays tolerance for variables in the equation in the Coefficients table. For variables not yet entered into the equation, TOL displays in the Excluded Variables table the tolerance each variable would have if it were the only variable entered next.

---

[74] Belsley, D. A., E. Kuh, and R. E. Welsch. 1980. *Regression diagnostics: Identifying influential data and sources of collinearity*. New York: John Wiley and Sons.

[75] Judge, G. G., W. E. Griffiths, R. C. Hill, H. Lutkepohl, and T. C. Lee. 1980. *The theory and practice of econometrics*, 2nd ed. New York: John Wiley and Sons.

[76] Meyer, L. S., and M. S. Younger. 1976. Estimation of standardized coefficients. *Journal of the American Statistical Association,* 71, 154-157.

F. *F value for B and its probability.* This is displayed instead of the *t* value in the Coefficients or Excluded Variables table.

CRITERIA Subcommand

CRITERIA controls the statistical criteria used to build the regression equations. The way in which these criteria are used depends on the method specified on METHOD. The default criteria are noted in the description of each CRITERIA keyword below.

- The minimum specification is a criterion keyword and its arguments, if any.
- If CRITERIA is omitted or included without specifications, the default criteria are in effect.
- The CRITERIA subcommand must be specified before DEPENDENT and METHOD subcommands. The last specified CRITERIA affects all equations.

Tolerance and Minimum Tolerance Tests

Variables must pass both tolerance and minimum tolerance tests in order to enter and remain in a regression equation. Tolerance is the proportion of the variance of a variable in the equation that is not accounted for by other independent variables in the equation. The minimum tolerance of a variable not in the equation is the smallest tolerance any variable already in the equation would have if the variable being considered were included in the analysis.

If a variable passes the tolerance criteria, it is eligible for inclusion based on the method in effect.

Criteria for Variable Selection

- The ENTER, REMOVE, and TEST methods use only the TOLERANCE criterion.
- BACKWARD removes variables according to the probability of *F*-to-remove (keyword POUT). Specify FOUT to use *F*-to-remove instead.
- FORWARD enters variables according to the probability of *F*-to-enter (keyword PIN). Specify FIN to use *F*-to-enter instead.
- STEPWISE uses both PIN and POUT (or FIN and FOUT) as criteria. If the criterion for entry (PIN or FIN) is less stringent than the criterion for removal (POUT or FOUT), the same variable can cycle in and out until the maximum number of steps is reached. Therefore, if PIN is larger than POUT or FIN is smaller than FOUT, REGRESSION adjusts POUT or FOUT and issues a warning.
- The values for these criteria are specified in parentheses. If a value is not specified, the default values are used.

DEFAULTS. *PIN(0.05), POUT(0.10), and TOLERANCE(0.0001).* These are the defaults if CRITERIA is omitted. If criteria have been changed, DEFAULTS restores these defaults.

PIN[(value)]. *Probability of* F*-to-enter*. The default value is 0.05. Either PIN or FIN can be specified. If more than one is used, the last one specified is in effect.

FIN[(value)]. F*-to-enter*. The default value is 3.84. Either PIN or FIN can be specified. If more than one is used, the last one specified is in effect.

POUT[(value)]. *Probability of* F*-to-remove*. The default value is 0.10. Either POUT or FOUT can be specified. If more than one is used, the last one specified is in effect.

FOUT[(value)]. F*-to-remove*. The default value is 2.71. Either POUT or FOUT can be specified. If more than one is used, the last one specified is in effect.

TOLERANCE[(value)]. *Tolerance*. The default value is 0.0001. If the specified tolerance is very low, REGRESSION issues a warning.

MAXSTEPS[(n)]. *Maximum number of steps.* The value of MAXSTEPS is the sum of the maximum number of steps for each method for the equation. The default values are, for the BACKWARD or FORWARD methods, the number of variables meeting PIN/POUT or FIN/FOUT criteria, and for the STEPWISE method, twice the number of independent variables.

Confidence Intervals

CIN[(value)]. *Reset the value of the percent for confidence intervals.* The default is 95%. The specified value sets the percentage interval used in the computation of temporary variable types MCIN and ICIN.

Example

```
REGRESSION VARIABLES=POP15 TO GROWTH, SAVINGS
 /CRITERIA=PIN(.1) POUT(.15)
 /DEPENDENT=SAVINGS
 /METHOD=FORWARD.
```

- The CRITERIA subcommand relaxes the default criteria for entry and removal for the FORWARD method. Note that the specified PIN is less than POUT.

ORIGIN and NOORIGIN Subcommands

ORIGIN and NOORIGIN control whether or not the constant is suppressed. By default, the constant is included in the model (NOORIGIN).

- The specification is either the ORIGIN or NOORIGIN subcommand.
- ORIGIN and NOORIGIN must be specified before the DEPENDENT and METHOD subcommands. The last specified remains in effect for all equations.
- ORIGIN requests regression through the origin. The constant term is suppressed.
- If you specify ORIGIN, statistics requested on the DESCRIPTIVES subcommand are computed as if the mean were 0.
- ORIGIN and NOORIGIN affect the way the correlation matrix is built. If matrix materials are used as input to REGRESSION, the keyword that was in effect when the matrix was written should be in effect when that matrix is read.

Example

```
REGRESSION VAR=(COL)
 /ORIGIN
 /DEP=HOMICIDE
 /METHOD=ENTER POVPCT.
```

- The REGRESSION command requests an equation that regresses *HOMICIDE* on *POVPCT* and suppresses the constant (ORIGIN).

REGWGT Subcommand

The only specification on REGWGT is the name of the variable containing the weights to be used in estimating a weighted least-squares model. With REGWGT, the default display is the usual REGRESSION display.

- REGWGT is a global subcommand.
- If more than one REGWGT subcommand is specified on a REGRESSION procedure, only the last one is in effect.
- REGWGT can be used with MATRIX OUT but not with MATRIX IN.
- Residuals saved from equations using the REGWGT command are not weighted. To obtain weighted residuals, multiply the residuals created with SAVE by the square root of the weighting variable in a COMPUTE statement.
- REGWGT is in effect for all equations and affects the way the correlation matrix is built. Thus, if REGWGT is specified on a REGRESSION procedure that writes matrix materials to a matrix data file, subsequent REGRESSION procedures using that file will be automatically weighted.

Example

```
REGRESSION VARIABLES=GRADE GPA STARTLEV TREATMNT
 /DEPENDENT=GRADE
 /METHOD=ENTER
 /SAVE PRED(P).
COMPUTE WEIGHT=1/(P*(1-P)).
REGRESSION VAR=GRADE GPA STARTLEV TREATMNT
 /REGWGT=WEIGHT
 /DEP=GRADE
 /METHOD=ENTER.
```

- VARIABLES builds a correlation matrix that includes *GRADE*, *GPA*, *STARTLEV*, and *TREATMNT*.
- DEPENDENT identifies *GRADE* as the dependent variable.
- METHOD regresses *GRADE* on *GPA*, *STARTLEV*, and *TREATMNT*.
- SAVE saves the predicted values from the regression equation as variable *P* in the active dataset. See the topic "SAVE Subcommand " on page 1649 for more information.
- COMPUTE creates the variable *WEIGHT* as a transformation of *P*.
- The second REGRESSION procedure performs a weighted regression analysis on the same set of variables using *WEIGHT* as the weighting variable.

Example

```
REGRESSION VAR=GRADE GPA STARTLEV TREATMNT
 /REGWGT=WEIGHT
 /DEP=GRADE
 /METHOD=ENTER
 /SAVE RESID(RGRADE).
COMPUTE WRGRADE=RGRADE * SQRT(WEIGHT).
```

- This example illustrates the use of COMPUTE with SAVE to weight residuals.
- REGRESSION performs a weighted regression analysis of *GRADE* on *GPA*, *STARTLEV*, and *TREATMNT*, using *WEIGHT* as the weighting variable.
- SAVE saves the residuals as *RGRADE*. These residuals are not weighted.
- COMPUTE creates variable *WRGRADE*, which contains the weighted residuals.

DESCRIPTIVES Subcommand

DESCRIPTIVES requests the display of correlations and descriptive statistics. By default, descriptive statistics are not displayed.

- The minimum specification is simply the subcommand keyword DESCRIPTIVES, which obtains MEAN, STDDEV, and CORR.
- If DESCRIPTIVES is specified with keywords, only those statistics specifically requested are displayed.
- Descriptive statistics are displayed only once for all variables named or implied on VARIABLES.
- Descriptive statistics are based on all valid cases for each variable if PAIRWISE or MEANSUBSTITUTION has been specified on MISSING. Otherwise, only cases with valid values for all variables named or implied on the VARIABLES subcommand are included in the calculation of descriptive statistics.
- If regression through the origin has been requested (subcommand ORIGIN), statistics are computed as if the mean were 0.

NONE. *No descriptive statistics.* This is the default if the subcommand is omitted.

DEFAULTS. *MEAN, STDDEV, and CORR*. This is the same as specifying DESCRIPTIVES without specifications.

MEAN. *Display variable means in the Descriptive Statistics table*.

STDDEV. *Display variable standard deviations in the Descriptive Statistics table.*

VARIANCE. *Display variable variances in the Descriptive Statistics table*.

CORR. *Display Pearson correlation coefficients in the Correlations table*.

SIG. *Display one-tailed probabilities of the correlation coefficients in the Correlations table.*

BADCORR. *Display the correlation coefficients only if some coefficients cannot be computed.*

COV. *Display covariance in the Correlations table.*

XPROD. *Display sum of squares and cross-product deviations from the mean in the Correlations table.*

N. *Display numbers of cases used to compute correlation coefficients in the Correlations table.*

ALL. *All descriptive statistics.*

Example

```
REGRESSION DESCRIPTIVES=DEFAULTS SIG COV
 /VARIABLES=AGE,FEMALE,YRS_JOB,STARTPAY,SALARY
 /DEPENDENT=SALARY
 /METHOD=ENTER STARTPAY
 /METHOD=ENTER YRS_JOB.
```

- The variable means, standard deviations, and number of cases are displayed in the Descriptive Statistics table and the correlation coefficients, one-tailed probabilities of the correlation coefficients, and covariance are displayed in the Correlations table.
- Statistics are displayed for all variables named on VARIABLES, even though only variables *SALARY, STARTPAY*, and *YRS_JOB* are used to build the equations.
- *STARTPAY* is entered into the equation by the first METHOD subcommand. *YRS_JOB* is entered by the second METHOD subcommand.

SELECT Subcommand

By default, all cases in the active dataset are considered for inclusion on REGRESSION. Use SELECT to include a subset of cases in the correlation matrix and resulting regression statistics.

- The required specification on SELECT is a logical expression.
- The syntax for the SELECT subcommand is as follows:

```
/SELECT=varname relation value
```

- The variable named on SELECT should not be specified on the VARIABLES subcommand.
- The relation can be EQ, NE, LT, LE, GT, or GE.
- Only cases for which the logical expression on SELECT is true are included in the calculation of the correlation matrix and regression statistics.
- All other cases, including those with missing values for the variable named on SELECT, are not included in the computations.
- If SELECT is specified, residuals and predicted values are calculated and reported separately for both selected and unselected cases by default. See the topic “RESIDUALS Subcommand ” on page 1646 for more information.
- Cases deleted from the active dataset with SELECT IF, a temporary SELECT IF, or SAMPLE are not passed to REGRESSION and are not included among either the selected or unselected cases.

Example

```
REGRESSION SELECT SEX EQ 'M'
 /VARIABLES=AGE,STARTPAY,YRS_JOB,SALARY
 /DEPENDENT=SALARY
 /METHOD=STEP
 /RESIDUALS=NORMPROB.
```

- Only cases with the value M for *SEX* are included in the correlation matrix calculated by REGRESSION.
- Separate normal P_P plots are displayed for cases with *SEX* equal to M and for other cases. See the topic “RESIDUALS Subcommand ” on page 1646 for more information.

MATRIX Subcommand

MATRIX reads and writes matrix data files. It can read matrix data files or datasets written by previous REGRESSION procedures or data files or datasets written by other procedures such as CORRELATIONS. The matrix materials REGRESSION writes also include the mean, standard deviation, and number of cases used to compute each coefficient. This information immediately precedes the correlation matrix in the matrix file.

- Either IN or OUT and a matrix file or previously declared dataset name in parentheses are required on MATRIX.
- When used, MATRIX must be the first subcommand specified in a REGRESSION procedure.
- ORIGIN and NOORIGIN affect the way the correlation matrix is built. If matrix materials are used as input to REGRESSION, the keyword that was in effect when the matrix was written should be in effect when that matrix is read.

OUT ('savfile'|'dataset'). *Write a matrix data file or dataset.* Specify either a filename, a previously declared dataset name, or an asterisk, enclosed in parentheses. Filenames should be enclosed in quotes and are stored in the working directory unless a path is included as part of the file specification. If you specify an asterisk (*), the matrix data file replaces the active dataset. If you specify an asterisk or a dataset name, the file is not stored on disk unless you use SAVE or XSAVE.

IN ('savfile'|'dataset'). *Read a matrix data file or dataset.* Specify either a filename, dataset name created during the current session, or an asterisk enclosed in parentheses. An asterisk reads the matrix data from the active dataset. Filenames should be enclosed in quotes and are read from the working directory unless a path is included as part of the file specification.

Format of the Matrix Data File

- The file has two special variables created by the program: *ROWTYPE_* and *VARNAME_*.
- *ROWTYPE_* is a short string variable with values MEAN, STDDEV, N, and CORR (for Pearson correlation coefficient).
- *VARNAME_* is a short string variable whose values are the names of the variables used to form the correlation matrix. When *ROWTYPE_* is CORR, *VARNAME_* gives the variable associated with that row of the correlation matrix.
- The remaining variables in the file are the variables used to form the correlation matrix.
- To suppress the constant term when ORIGIN is used in the analysis, value OCORR (rather than value CORR) is written to the matrix system file. OCORR indicates that the regression passes through the origin.

Split Files

- When split-file processing is in effect, the first variables in the matrix data file are the split variables, followed by *ROWTYPE_*, the independent variable, *VARNAME_*, and the dependent variables.
- A full set of matrix materials is written for each subgroup defined by the split variable(s).
- A split variable cannot have the same variable name as any other variable written to the matrix data file.
- If a split file is in effect when a matrix is written, the same split file must be in effect when that matrix is read.

Missing Values

- With PAIRWISE treatment of missing values, the matrix of N's used to compute each coefficient is included with the matrix materials.
- With LISTWISE treatment (the default) or MEANSUBSTITUTION, a single N used to calculate all coefficients is included.

Example

```
REGRESSION MATRIX IN('/data/pay_data.sav') OUT(*)
 /VARIABLES=AGE,STARTPAY,YRS_JOB,SALARY
 /DEPENDENT=SALARY
 /METHOD=STEP.
```

- MATRIX IN reads the matrix data file *pay_data.sav*.
- A stepwise regression analysis of *SALARY* is performed using *AGE*, *STARTPAY*, and *YRS_JOB.*
- MATRIX OUT replaces the active dataset with the matrix data file that was previously stored in the *pay_data.sav* file.

MISSING Subcommand

MISSING controls the treatment of cases with missing values. By default, a case that has a user-missing or system-missing value for any variable named or implied on VARIABLES is omitted from the computation of the correlation matrix on which all analyses are based.

- The minimum specification is a keyword specifying a missing-value treatment.

LISTWISE. *Delete cases with missing values listwise.* Only cases with valid values for all variables named on the current VARIABLES subcommand are used. If INCLUDE is also specified, only cases with system-missing values are deleted listwise. LISTWISE is the default if the MISSING subcommand is omitted.

PAIRWISE. *Delete cases with missing values pairwise*. Each correlation coefficient is computed using cases with complete data for the pair of variables correlated. If INCLUDE is also specified, only cases with system-missing values are deleted pairwise.

MEANSUBSTITUTION. *Replace missing values with the variable mean*. All cases are included and the substitutions are treated as valid observations. If INCLUDE is also specified, user-missing values are treated as valid and are included in the computation of the means.

INCLUDE. *Includes cases with user-missing values*. All user-missing values are treated as valid values. This keyword can be specified along with the methods LISTWISE, PAIRWISE, or MEANSUBSTITUTION.

Example

```
REGRESSION  VARIABLES=POP15,POP75,INCOME,GROWTH,SAVINGS
 /DEPENDENT=SAVINGS
 /METHOD=STEP
 /MISSING=MEANSUBSTITUTION.
```

- System-missing and user-missing values are replaced with the means of the variables when the correlation matrix is calculated.

RESIDUALS Subcommand

RESIDUALS controls the display and labeling of summary information on outliers as well as the display of the Durbin-Watson statistic and histograms and normal probability plots for the temporary variables.

- If RESIDUALS is specified without keywords, it displays a histogram of residuals, a normal probability plot of residuals, the values of *$CASENUM* and *ZRESID* for the 10 cases with the largest absolute value of *ZRESID*, and the Durbin-Watson test statistic. The histogram and the normal plot are standardized.
- If any keywords are specified on RESIDUALS, only the requested information and plots are displayed.

DEFAULTS. *DURBIN, NORMPROB(ZRESID), HISTOGRAM(ZRESID), OUTLIERS(ZRESID).* These are the defaults if RESIDUALS is used without specifications.

HISTOGRAM(tempvars). *Histogram of the temporary variable or variables*. The default is *ZRESID*. You can request histograms for *PRED*, *RESID*, *ZPRED*, *DRESID*, *ADJPRED*, *SRESID*, *SDRESID*, *SEPRED*, *MAHAL*, *COOK*, and *LEVER*. The specification of any other temporary variable will result in an error.

NORMPROB(tempvars). *Normal probability (P-P) plot*. The default is *ZRESID*. The other temporary variables for which normal probability plots are available are *PRED*, *RESID*, *ZPRED*, *DRESID*, *SRESID*, and *SDRESID*. The specification of any other temporary variable will result in an error. Normal probability plots are always displayed in standardized form; therefore, when *PRED*, *RESID*, or *DRESID* is requested, the standardized equivalent *ZPRED*, *ZRESID* or *SDRESID* is displayed.

OUTLIERS(tempvars). *The 10 cases with the largest absolute values of the specified temporary variables.* The default is *ZRESID*. The output includes the values of *$CASENUM* and of the temporary variables for the 10 cases. The other temporary variables available for OUTLIERS are *RESID*, *SRESID*, *SDRESID*, *DRESID*, *MAHAL*, and *COOK*. The specification of any temporary variable other than these will result in an error.

DURBIN. *Display Durbin-Watson test statistic in the Model Summary table*.

ID(varname). *ID variable providing case labels for use with point selection mode in the Chart Editor*. Applicable to scatterplots produced by SCATTERPLOT, PARTIALPLOT, and RESIDUALS. Any variable in the active dataset can be named.

SEPARATE. *Separate reporting of residuals statistics and plots for selected and unselected cases.* This is the default.

POOLED. *Pooled plots and statistics using all cases in the working file when the SELECT subcommand is in effect.* This is an alternative to SEPARATE.

Example

```
/RESID=DEFAULT ID(SVAR)
```

- DEFAULT produces the default residuals statistics: Durbin-Watson statistic, a normal probability plot and histogram of *ZRESID*, and an outlier listing for *ZRESID*.
- Descriptive statistics for *ZRESID, RESID, PRED,* and *ZPRED* are automatically displayed.
- *SVAR* is specified as the case identifier on the outlier output.

CASEWISE Subcommand

CASEWISE requests a Casewise Diagnostics table of residuals. You can specify a temporary residual variable for casewise listing (via the PLOT keyword). You can also specify variables to be listed in the table for each case.

- If CASEWISE is used without any additional specifications, it displays a Casewise Diagnostics table of *ZRESID* for cases whose absolute value of *ZRESID* is at least 3. By default, the values of the case sequence number, *DEPENDENT*, *PRED*, and *RESID* are listed for each case.
- Defaults remain in effect unless specifically altered.

DEFAULTS. *OUTLIERS(3), PLOT(ZRESID), DEPENDENT, PRED, and RESID*. These are the defaults if the subcommand is used without specifications.

OUTLIERS(value). *List only cases for which the absolute standardized value of the listed variable is at least as large as the specified value.* The default value is 3. Keyword OUTLIERS is ignored if keyword ALL is also present.

ALL. *Include all cases in the Casewise Diagnostic table.* ALL is the alternative to keyword OUTLIERS.

PLOT(tempvar). *List the values of the temporary variable in the Casewise Diagnostics table*. The default temporary variable is *ZRESID*. Other variables that can be listed are *RESID*, *DRESID*, *SRESID*, and *SDRESID*. The specification of any temporary variable other than these will result in an error. When requested, *RESID* is standardized and *DRESID* is Studentized in the output.

tempvars. *Display the values of these variables next to the casewise list entry for each case.* The default variables are *DEPENDENT* (the dependent variable), *PRED*, and *RESID*. Any of the other temporary variables can be specified. If an ID variable is specified on RESIDUALS, the ID variable is also listed.

Example

```
/CASEWISE=DEFAULT ALL SRE MAH COOK SDR
```

- This example requests a Casewise Diagnostics table of the standardized residuals for all cases.
- *ZRESID*, the dependent variable, and the temporary variables *PRED*, *RESID*, *SRESID*, *MAHAL*, *COOK*, and *SDRESID* are for all cases.

SCATTERPLOT Subcommand

SCATTERPLOT names pairs of variables for scatterplots.

- The minimum specification for SCATTERPLOT is a pair of variables in parentheses. There are no default specifications.
- You can specify as many pairs of variables in parentheses as you want.
- The first variable named in each set of parentheses is plotted along the vertical axis, and the second variable is plotted along the horizontal axis.
- Plotting symbols are used to represent multiple points occurring at the same position.
- You can specify any variable named on the VARIABLES subcommand.
- You can specify *PRED*, *RESID*, *ZPRED*, *ZRESID*, *DRESID*, *ADJPRED*, *SRESID*, *SDRESID*, *SEPRED*, *MAHAL*, *COOK*, and *LEVER*. The specification of any other temporary variables will result in an error.
- Specify an asterisk before temporary variable names to distinguish them from user-defined variables. For example, use **PRED* to specify *PRED*.

Example

```
/SCATTERPLOT (*RES,*PRE)(*RES,SAVINGS)
```

- This example specifies two scatterplots: residuals against predicted values and residuals against the values of the variable *SAVINGS*.

PARTIALPLOT Subcommand

PARTIALPLOT requests partial regression plots. Partial regression plots are scatterplots of the residuals of the dependent variable and an independent variable when both of these variables are regressed on the rest of the independent variables.

- If PARTIALPLOT is included without any additional specifications, it produces a partial regression plot for every independent variable in the equation. The plots appear in the order the variables are specified or implied on the VARIABLES subcommand.
- If variables are specified on PARTIALPLOT, only the requested plots are displayed. The plots appear in the order the variables are listed on the PARTIALPLOT subcommand.
- At least two independent variables must be in the equation for partial regression plots to be produced.

ALL. *Plot all independent variables in the equation.* This is the default.

varlist. *Plot the specified variables.* Any variable entered into the equation can be specified.

Example

```
REGRESSION VARS=PLOT15 TO SAVINGS
  /DEP=SAVINGS
  /METH=ENTER
  /RESID=DEFAULTS
  /PARTIAL.
```

- A partial regression plot is produced for every independent variable in the equation.

OUTFILE Subcommand

OUTFILE saves the parameter covariance or correlation matrix with parameter estimates, standard errors, significance values, and residual degrees of freedom for each term in the final equation in IBM SPSS Statistics format data files. It also saves model information in XML format.

- The OUTFILE subcommand must follow the last METHOD subcommand.
- Only one OUTFILE subcommand is allowed. If you specify more than one, only the last one is executed.
- You must specify at least one keyword and a quoted file specification , enclosed in parentheses. For COVB and CORB, you can specify a previously declared dataset (DATASET DECLARE command) instead of a file.
- You cannot save the parameter statistics as the active dataset.
- COVB and CORB are mutually exclusive.
- MODEL cannot be used if split file processing is on (SPLIT FILE command) or if more than one dependent (DEPENDENT subcommand) variable is specified.
- If you specify an external file name, you should include the .sav extension in the specification. There is no default extension.

COVB ('savfile'|'dataset'). *Write the parameter covariance matrix with other statistics.*

CORB ('savfile'|'dataset'). *Write the parameter correlation matrix with other statistics.*

MODEL ('file'). *Write model information to an XML file.* You can use this model file to apply the model information to other data files for scoring purposes. See the topic "Scoring expressions" on page 86 for more information.

PARAMETER('file'). *Write parameter estimates only to an XML file.* You can use this model file to apply the model information to other data files for scoring purposes. See the topic "Scoring expressions" on page 86 for more information.

Example

```
REGRESSION DEPENDENT=Y
 /METHOD=ENTER X1 X2
 /OUTFILE CORB ('/data/covx1x2y.sav').
```

- The OUTFILE subcommand saves the parameter correlation matrix, and the parameter estimates, standard errors, significance values, and residual degrees of freedom for the constant term, *X1* and *X2*.

SAVE Subcommand

Use SAVE to add one or more residual or fit variables to the active dataset.

- The specification on SAVE is one or more of the temporary variable types, each followed by an optional name in parentheses for the new variable.
- New variable names must be unique.
- If new names are not specified, REGRESSION generates a rootname using a shortened form of the temporary variable name with a suffix to identify its creation sequence.
- If you specify DFBETA or SDBETA on the SAVE subcommand, the number of new variables saved is the total number of variables in the equation.

FITS. *Save all influence statistics.* FITS saves *DFFIT, SDFIT, DFBETA, SDBETA,* and *COVRATIO*. You cannot specify new variable names when using this keyword. Default names are generated.

Example

```
/SAVE=PRED(PREDVAL) RESID(RESIDUAL) COOK(CDISTANC)
```

- This subcommand adds three variables to the end of the active dataset: *PREDVAL*, containing the unstandardized predicted value for each case; *RESIDUAL*, containing the unstandardized residual; and *CDISTANC*, containing Cook's distance.

Example

```
/SAVE=PRED RESID
```

- This subcommand adds two variables named *PRE_1* and *RES_1* to the end of the active dataset.

Example

```
REGRESSION DEPENDENT=Y
 /METHOD=ENTER X1 X2
 /SAVE DFBETA(DFBVAR).
```

- The SAVE subcommand creates and saves three new variables with the names *DFBVAR0*, *DFBVAR1*, and *DFBVAR2*.

Example

```
REGRESSION VARIABLES=SAVINGS INCOME POP15 POP75 GROWTH
 /DEPENDENT=SAVINGS
 /METHOD=ENTER INCOME POP15 POP75
 /SAVE=PRED(PREDV) SDBETA(BETA) ICIN.
```

- The SAVE subcommand adds seven variables to the end of the file: *PREDV*, containing the unstandardized predicted value for the case; *BETA0*, the standardized *DFBETA* for the intercept; *BETA1*, *BETA2*, and *BETA3*, the standardized *DFBETA*'s for the three independent variables in the model; *LICI_1*, the lower bound for the prediction interval for an individual case; and *UICI_1*, the upper bound for the prediction interval for an individual case.

TEMPLATE Subcommand

TEMPLATE uses an existing file as a template and applies it to any charts generated by the current REGRESSION command. The template overrides the default settings that are used to create any charts. Templates are created in the Chart Editor by saving an existing chart as a template.

Example

```
REGRESSION VARIABLES=pop15,pop75,income,growth,savings
 /DEPENDENT=savings
 /METHOD=ENTER pop15,pop75,income
 /METHOD=ENTER growth
 /RESIDUALS HISTOGRAM(ZRESID)
 /TEMPLATE='/templates/mytemplate.sgt'.
```

References

Belsley, D. A., E. Kuh, and R. E. Welsch. 1980. *Regression diagnostics: Identifying influential data and sources of collinearity*. New York: John Wiley and Sons.

Berk, K. N. 1977. Tolerance and condition in regression computation. *Journal of the American Statistical Association,* 72, 863-866.

Cook, R. D. 1977. Detection of influential observations in linear regression. *Technometrics,* 19, 15-18.

Dillon, W. R., and M. Goldstein. 1984. *Multivariate analysis: Methods and applications*. New York: John Wiley and Sons.

Hoaglin, D. C., and R. E. Welsch. 1978. The hat matrix in regression and ANOVA. *American Statistician,* 32, 17-22.

Judge, G. G., W. E. Griffiths, R. C. Hill, H. Lutkepohl, and T. C. Lee. 1980. *The theory and practice of econometrics*, 2nd ed. New York: John Wiley and Sons.

Meyer, L. S., and M. S. Younger. 1976. Estimation of standardized coefficients. *Journal of the American Statistical Association,* 71, 154-157.

Velleman, P. F., and R. E. Welsch. 1981. Efficient computing of regression diagnostics. *American Statistician,* 35, 234-242.

RELIABILITY

RELIABILITY is available in Statistics Base Edition.

```
RELIABILITY VARIABLES={varlist}
                      {ALL    }

 [/SCALE(scalename)=varlist ]

 [/MODEL={ALPHA         }]
         {SPLIT[(n)]    }
         {GUTTMAN       }
         {PARALLEL      }
         {STRICTPARALLEL}
         {OMEGA         }

 [/STATISTICS=[DESCRIPTIVE]   [SCALE]      [{ANOVA         }] [ALL] ]
              [COVARIANCES]   [TUKEY]       {ANOVA FRIEDMAN}
              [CORRELATIONS]  [HOTELLING]   {ANOVA COCHRAN }

 [/SUMMARY=[MEANS] [VARIANCE] [COV] [CORR] [TOTAL] [ALL] ]

 [/ICC=[{MODEL(ONEWAY)                              }]
        {[MODEL({MIXED**})] [TYPE({CONSISTENCY**})]}
                {RANDOM }         {ABSOLUTE     }
         [CIN={95**}] [TESTVAL={0**}]]
              {n   }           {p  }

 [/METHOD=COVARIANCE]

 [/MISSING={EXCLUDE**}]
           {INCLUDE  }

 [/MATRIX =[IN({*                  })] [OUT({*                  })] [NOPRINT]]
               {'savfile'|'dataset'}        {'savfile'|'dataset'}
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 27.1

- Support for **OMEGA** keyword added to **/MODEL** subcommand.

Release 17.0

- Increased limits on numbers of variables allowed on the VARIABLES and SCALE lists.

Example

```
RELIABILITY  VARIABLES=SCORE1 TO SCORE10
  /SCALE (OVERALL) = ALL
  /MODEL = ALPHA
  /SUMMARY = MEANS TOTAL.
```

Overview

RELIABILITY estimates reliability statistics for the components of multiple-item additive scales. It uses any one of five models for reliability analysis and offers a variety of statistical displays. RELIABILITY can also be used to perform a repeated measures analysis of variance, a two-way factorial analysis of variance with one observation per cell, Tukey's test for additivity, Hotelling's *T*-square test for equality of means in repeated measures designs, and Friedman's two-way analysis of variance on ranks. For more complex repeated measures designs, use the GLM procedure (available in the Advanced Models option).

Options

Model Type. You can specify any one of five models on the MODEL subcommand.

Statistical Display. Statistics available on the STATISTICS subcommand include descriptive statistics, correlation and covariance matrices, a repeated measures analysis of variance table, Hotelling's *T*-square, Tukey's test for additivity, Friedman's chi-square for the analysis of ranked data, and Cochran's *Q*.

Computational Method. You can force RELIABILITY to use the covariance method, even when you are not requesting any output that requires it, by using the METHOD subcommand.

Matrix Input and Output. You can read data in the form of correlation matrices and you can write correlation-type matrix materials to a data file using the MATRIX subcommand.

Basic Specification

The basic specification is VARIABLES and a variable list. By default, RELIABILITY displays the number of cases, number of items, and Cronbach's alpha. Whenever possible, it uses an algorithm that does not require the calculation of the covariance matrix.

Subcommand Order

- VARIABLES must be specified first.
- The remaining subcommands can be named in any order.

Operations

- STATISTICS and SUMMARY are cumulative. If you enter them more than once, all requested statistics are produced for each scale.
- If you request output that is not available for your model or for your data, RELIABILITY ignores the request.
- RELIABILITY uses an economical algorithm whenever possible but calculates a covariance matrix when necessary (see "METHOD Subcommand " on page 1656).

Limitations

- Maximum 1 VARIABLES subcommand.
- Maximum 1 SCALE subcommand.
- Maximum 10,000 variables on the VARIABLES subcommand.
- Maximum 10,000 variables on the SCALE subcommand.

VARIABLES Subcommand

VARIABLES specifies the variables to be used in the analysis. Only numeric variables can be used.

- VARIABLES is required and must be specified first.
- You can use keyword ALL to refer to all user-defined variables in the active dataset.

SCALE Subcommand

SCALE defines a scale for analysis, providing a label for the scale and specifying its component variables. If SCALE is omitted, all variables named on VARIABLES are used, and the label for the scale is *ALL*.

- The label is specified in parentheses after SCALE. It can have a maximum of 255 bytes before being truncated.
- RELIABILITY does not add any new variables to the active dataset. The label is used only to identify the output. If the analysis is satisfactory, use COMPUTE to create a new variable containing the sum of the component items.
- Variables named on SCALE must have been named on the VARIABLES subcommand. Use the keyword ALL to refer to all variables named on the VARIABLES subcommand.

Example

```
RELIABILITY VARIABLES = ITEM1 TO ITEM20
  /SCALE (A) = ITEM1 TO ITEM10.

RELIABILITY VARIABLES = ITEM1 TO ITEM20
  /SCALE (B) = ITEM1 TO ITEM20.
```

- Analyses for scales A and B both use only cases that have complete data for items 1 through 20.

MODEL Subcommand

MODEL specifies the type of reliability analysis for the scale named on the SCALE subcommand.

ALPHA
Cronbach's α. Standardized item α is displayed. This is the default.

OMEGA
Specifies the Omega Ω model.

SPLIT [(n)]
Split-half coefficients. You can specify a number in parentheses to indicate how many items should be in the second half. For example, MODEL SPLIT (6) uses the last six variables for the second half and all others for the first. By default, each half has an equal number of items, with the odd item, if any, going to the first half.

GUTTMAN
Guttman's lower bounds for true reliability.

PARALLEL
Maximum-likelihood reliability estimate under parallel assumptions. This model assumes that items have the same variance but not necessarily the same mean.

STRICTPARALLEL
Maximum-likelihood reliability estimate under strictly parallel assumptions. This model assumes that items have the same means, the same true score variances over a set of objects being measured, and the same error variance over replications.

STATISTICS Subcommand

STATISTICS displays optional statistics. There are no default statistics.

- STATISTICS is cumulative. If you enter it more than once, all requested statistics are produced for each scale.

DESCRIPTIVES. *Item means and standard deviations.*

COVARIANCES. *Inter-item variance-covariance matrix.*

CORRELATIONS. *Inter-item correlation matrix.*

SCALE. *Scale means and scale variances.*

TUKEY. *Tukey's test for additivity.* This helps determine whether a transformation of the items is needed to reduce nonadditivity. The test displays an estimate of the power to which the items should be raised in order to be additive.

HOTELLING. *Hotelling's* T-*square.* This is a test for equality of means among the items.

ANOVA. *Repeated measures analysis of variance table.*

FRIEDMAN. *Friedman's chi-square and Kendall's coefficient of concordance.* These apply to ranked data. You must request ANOVA in addition to FRIEDMAN; Friedman's chi-square appears in place of the usual *F* test. If the ANOVA keyword is not specified, the FRIEDMAN keyword is silently ignored.

COCHRAN. *Cochran's* Q. This applies when all items are dichotomies. You must request ANOVA in addition to COCHRAN; the *Q* statistic appears in place of the usual *F* test. If the ANOVA keyword is not specified, the COCHRAN keyword is silently ignored.

ALL. *All applicable statistics.*

ICC Subcommand

ICC displays intraclass correlation coefficients for single measure and average measure. Single measure applies to single measurements—for example, the rating of judges, individual item scores, or the body weights of individuals. Average measure, however, applies to average measurements, for example, the average rating of *k* judges, or the average score for a *k*-item test.

MODEL. *Model.* You can specify the model for the computation of ICC. There are three keywords for this option. ONEWAY is the one-way random effects model (people effects are random). RANDOM is the two-way random effect model (people effects and the item effects are random). MIXED is the two-way mixed (people effects are random and the item effects are fixed). MIXED is the default. Only one model can be specified.

TYPE. *Type of definition.* When the model is RANDOM or MIXED, one of the two TYPE keywords may be given. CONSISTENCY is the consistency definition and ABSOLUTE is the absolute agreement definition. For the consistency coefficient, the between measures variance is excluded from the denominator variance, and for absolute agreement, it is not.

CIN. *The value of the percent for confidence interval and significance level of the hypothesis testing.*

TESTVAL. *The value with which an estimate of ICC is compared.* The value should be between 0 and 1.

SUMMARY Subcommand

SUMMARY displays summary statistics for each individual item in the scale.

- SUMMARY is cumulative. If you enter it more than once, all requested statistics are produced for the scale.
- You can specify one or more of the following:

MEANS. *Statistics on item means.* The average, minimum, maximum, range, ratio of maximum to minimum, and variance of the item means.

VARIANCE. *Statistics on item variances.* This displays the same statistics as for MEANS.

COVARIANCES. *Statistics on item covariances.* This displays the same statistics as for MEANS.

CORRELATIONS. *Statistics on item correlations.* This displays the same statistics as for MEANS.

TOTAL. *Statistics comparing each individual item to the scale composed of the other items.* The output includes the scale mean, variance, and Cronbach's α without the item, and the correlation between the item and the scale without it.

ALL. *All applicable summary statistics.*

METHOD Subcommand

By default, RELIABILITY uses a computational method that does not require the calculation of a covariance matrix wherever possible. METHOD forces RELIABILITY to calculate the covariance matrix. Only a single specification applies to METHOD:

COVARIANCE. *Calculate and use the covariance matrix, even if it is not needed.*

If METHOD is not specified, RELIABILITY computes the covariance matrix for all variables on each VARIABLES subcommand only if any of the following is true:

- You specify a model other than ALPHA or SPLIT.
- You request COV, CORR, FRIEDMAN, or HOTELLING on the STATISTICS subcommand.
- You request anything other than TOTAL on the SUMMARY subcommand.
- You write the matrix to a matrix data file, using the MATRIX subcommand.

MISSING Subcommand

MISSING controls the deletion of cases with user-missing data.

- RELIABILITY deletes cases from analysis if they have a missing value for any variable named on the VARIABLES subcommand. By default, both system-missing and user-missing values are excluded.

EXCLUDE. *Exclude user-missing and system-missing values.* This is the default.

INCLUDE. *Treat user-missing values as valid.* Only system-missing values are excluded.

MATRIX Subcommand

MATRIX reads and writes IBM SPSS Statistics matrix data files.

- Either IN or OUT and the matrix file in parentheses are required. When both IN and OUT are used on the same RELIABILITY procedure, they can be specified on separate MATRIX subcommands or on the same subcommand.
- If both IN and OUT are used on the same RELIABILITY command and there are grouping variables in the matrix input file, these variables are treated as if they were split variables. Values of the grouping variables in the input matrix are passed on to the output matrix (see “Split Files ” on page 1658).

OUT ('savfile'|'dataset'). *Write a matrix data file or dataset.* Specify either a filename, a previously declared dataset name, or an asterisk, enclosed in parentheses. Filenames should be enclosed in quotes and are stored in the working directory unless a path is included as part of the file specification. If you specify an asterisk (*), the matrix data file replaces the active dataset. If you specify an asterisk or a dataset name, the file is not stored on disk unless you use SAVE or XSAVE.

IN ('savfile'|'dataset'). *Read a matrix data file or dataset.* Specify either a filename, dataset name created during the current session, or an asterisk enclosed in parentheses. An asterisk reads the matrix data from the active dataset. Filenames should be enclosed in quotes and are read from the working directory unless a path is included as part of the file specification.

Matrix Output

- RELIABILITY writes correlation-type matrices that include the number of cases, means, and standard deviations with the matrix materials. These matrix materials can be used as input to RELIABILITY or other procedures.
- Any documents contained in the active dataset are not transferred to the matrix file.
- RELIABILITY displays the scale analysis when it writes matrix materials. To suppress the display of scale analysis, specify keyword NOPRINT on MATRIX.

Matrix Input

- RELIABILITY can read a matrix data file created by a previous RELIABILITY command or by another procedure. The matrix input file must have records of type N, MEAN, STDDEV, and CORR for each split-file group. For more information, see the Universals section.
- Variable names, variable and value labels, and print and write formats are read from the dictionary of the matrix data file.
- MATRIX=IN cannot be used unless an active dataset has already been defined. To read an existing matrix data file at the beginning of a session, use GET to retrieve the matrix file and then specify IN(*) on MATRIX.

Format of the Matrix Data File

- The matrix data file includes two special variables: *ROWTYPE_* and *VARNAME_*. Variable *ROWTYPE_* is a short string variable having values N, MEAN, STDDEV, and CORR. Variable *VARNAME_* is a short string variable whose values are the names of the variables used to form the correlation matrix.

- When *ROWTYPE_* is CORR, *VARNAME_* gives the variable associated with that row of the correlation matrix.
- The remaining variables in the matrix file are the variables used to form the correlation matrix.

Split Files

- When split-file processing is in effect, the first variables in the matrix data file will be the split variables, followed by *ROWTYPE_*, *VARNAME_*, and the dependent variable(s).
- If grouping variables are in the matrix input file, their values are between *ROWTYPE_* and *VARNAME_*. The grouping variables are treated like split-file variables.
- A full set of matrix materials is written for each split-file group defined by the split variables.
- A split variable cannot have the same variable name as any other variable written to the matrix data file.
- If split-file processing is in effect when a matrix is written, the same split file must be in effect when that matrix is read by any procedure.

Missing Values

Missing-value treatment affects the values written to a matrix data file. When reading a matrix data file, be sure to specify a missing-value treatment on RELIABILITY that is compatible with the treatment that was in effect when the matrix materials were generated.

Example: Matrix Output to External File

```
DATA LIST / TIME1 TO TIME5 1-10.
BEGIN DATA
 0 0 0 0 0
 0 0 1 1 0
 0 0 1 1 1
 0 1 1 1 1
 0 0 0 0 1
 0 1 0 1 1
 0 0 1 1 1
 1 0 0 1 1
 1 1 1 1 1
 1 1 1 1 1
END DATA.
RELIABILITY  VARIABLES=TIME1 TO TIME5
  /MATRIX=OUT('/data/relmtx.sav').
LIST.
```

- RELIABILITY reads data from the active dataset and writes one set of matrix materials to file *relmtx.sav*.
- The active dataset is still the file defined by DATA LIST. Subsequent commands are executed in this file.

Example: Matrix Output to Active Dataset

```
DATA LIST  / TIME1 TO TIME5 1-10.
BEGIN DATA
 0 0 0 0 0
 0 0 1 1 0
 0 0 1 1 1
 0 1 1 1 1
 0 0 0 0 1
 0 1 0 1 1
 0 0 1 1 1
 1 0 0 1 1
 1 1 1 1 1
 1 1 1 1 1
END DATA.
RELIABILITY  VARIABLES=TIME1 TO TIME5
  /MATRIX=OUT(*) NOPRINT.
LIST.
```

- RELIABILITY writes the same matrix as in the previous example. However, the matrix data file replaces the active dataset. The LIST command is executed in the matrix file, not in the file defined by DATA LIST.
- Because NOPRINT is specified on MATRIX, scale analyses are not displayed.

Example: Matrix Output to Active Dataset

```
GET FILE='/data/relmtx.sav'.
RELIABILITY VARIABLES=ALL
  /MATRIX=IN(*).
```

- This example assumes that you are starting a new session and want to read an existing matrix data file. GET retrieves the matrix data file *relmtx.sav*.
- MATRIX=IN specifies an asterisk because the matrix data file is the active dataset. If MATRIX=IN('/data/relmtx.sav') is specified, an error message is issued.
- If the GET command is omitted, an error message is issued.

Example: Matrix Input from External File

```
GET FILE='/data/personnel.sav'.
FREQUENCIES VARIABLE=AGE.

RELIABILITY VARIABLES=ALL
  /MATRIX=IN('/data/relmtx.sav').
```

- This example performs a frequencies analysis on file *personnel.sav* and then uses a different file containing matrix data for RELIABILITY. The file is an existing matrix data file. In order for this to work, the analysis variables named in *relmtx.sav* must also exist in *personnel.sav*.
- *relmtx.sav* must have records of type N, MEAN, STDDEV, and CORR for each split-file group.
- *relmtx.sav* does not replace *personnel.sav* as the active dataset.

Example: Matrix Input from Working File

```
GET FILE='/data/personnel.sav'.
CORRELATIONS VARIABLES=V1 TO V5
  /MATRIX=OUT(*).
RELIABILITY VARIABLES=V1 TO V5
  /MATRIX=IN(*).
```

- RELIABILITY uses matrix input from procedure CORRELATIONS. An asterisk is used to specify the active dataset for both the matrix output from CORRELATIONS and the matrix input for RELIABILITY.

RENAME VARIABLES

```
RENAME VARIABLES {(varname=newname)  [(varname ...)]}
                 {(varnames=newnames)              }
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
RENAME VARIABLES (JOBCAT=TITLE).
```

Overview

RENAME VARIABLES changes the names of variables in the active dataset while preserving their original order, values, variable labels, value labels, missing values, and print and write formats.

Basic Specification

- The basic specification is an old variable name, an equals sign, and the new variable name. The equals sign is required.

Syntax Rules

- Multiple sets of variable specifications are allowed. Each set can be enclosed in parentheses.
- You can specify a list of old variable names followed by an equals sign and a list of new variable names. The same number of variables must be specified on both lists. A single set of parentheses enclosing the entire specification is required for this method.
- Keyword TO can be used on the left side of the equals sign to refer to variables in the active dataset, and on the right side of the equals sign to generate new variable names.
- Old variable names do not need to be specified according to their order in the active dataset.
- Name changes take place in one operation. Therefore, variable names can be exchanged between two variables.
- Multiple RENAME VARIABLES commands are allowed.
- RENAME VARIABLES cannot follow either a TEMPORARY or a MODEL PROGRAM command.

Examples

Renaming Multiple Variables

```
RENAME VARIABLES (MOHIRED=MOSTART) (YRHIRED=YRSTART).
```

- MOHIRED is renamed to MOSTART and YRHIRED to YRSTART. The parentheses are optional.

  ```
  RENAME VARIABLES (MOHIRED YRHIRED=MOSTART YRSTART).
  ```

- The same name changes are specified as in the previous example. The parentheses are required, since variable lists are used.

Exchanging Variable Names

```
RENAME VARIABLES (A=B) (B=A).
```

- Variable names are exchanged between two variables: A is renamed to B, and B is renamed to A.

Mixed Case Variable Names

You can use the RENAME VARIABLES command to change the case of any characters in a variable name.

Example

```
RENAME VARIABLES (newvariable = NewVariable).
```

- For the existing variable name specification, case is ignored. Any combination of upper and lower case will work.
- For the new variable name, case will be preserved as entered for display purposes.

REPEATING DATA

```
REPEATING DATA [FILE=file]

 [/ENCODING='encoding specification']

 /STARTS=beg col[-end col] /OCCURS={value  }
                                   {varname}

 [/LENGTH={value  }] [/CONTINUED[=beg col[-end col]]]
          {varname}

 [/ID={col loc}=varname] [/{TABLE  }]
      {format }           {NOTABLE}

 /DATA=variable specifications
```

Release History

Release 16.0

- ENCODING subcommand added for Unicode support.

Example

```
INPUT PROGRAM.
DATA LIST / SEQNUM 2-4 NUMPERS 6-7 NUMVEH 9-10.
REPEATING DATA STARTS=12 /OCCURS=NUMVEH
 /DATA=MAKE 1-8 (A) MODEL 9 (A) NUMCYL 10.
END INPUT PROGRAM.
```

Overview

REPEATING DATA reads input cases whose records contain repeating groups of data. For each repeating group, REPEATING DATA builds one output case in the active dataset. All of the repeating groups in the data must contain the same type of information, although the number of groups for each input case may vary. Information common to the repeating groups for each input case can be recorded once for that case and then spread to each resulting output case. In this respect, a file with a repeating data structure is like a hierarchical file with both levels of information recorded on a single record rather than on separate record types. For information on reading hierarchical files, see FILE TYPE–END FILE TYPE.

REPEATING DATA must be used within an INPUT PROGRAM structure or within a FILE TYPE structure with mixed or nested data. In an INPUT PROGRAM structure, REPEATING DATA must be preceded by a DATA LIST command. In a FILE TYPE structure, DATA LIST is needed only if there are variables to be spread to each resulting output case.

Options

Length of Repeating Groups. If the length of the repeating groups varies across input cases, you can specify a variable that indicates the length on the LENGTH subcommand. You can also use LENGTH if you do not want to read all the data in each repeating group.

Continuation Records. You can use the CONTINUED subcommand to indicate that the repeating groups for each input case are contained on more than one record. You can check the value of an identification variable across records for the same input case using the ID subcommand.

Summary Tables. You can suppress the display of the table that summarizes the names, locations, and formats of the variables specified on the DATA subcommand using the NOTABLE subcommand.

Basic Specification

The basic specification requires three subcommands: STARTS, OCCURS, and DATA.

- STARTS specifies the beginning column of the repeating data segments. When there are continuation records, STARTS can specify the ending column of the last repeating group on the first record of each input case.

- OCCURS specifies the number of repeating groups on each input case. OCCURS can specify a number if the number of repeating groups is the same for all input cases. Otherwise, OCCURS should specify the name of a variable whose value for each input case indicates the number of repeating groups for that case.
- DATA specifies names, location within the repeating segment, and format for each variable to be read from the repeated groups.

Subcommand Order

- DATA must be the last subcommand specified on REPEATING DATA.
- The remaining subcommands can be named in any order.

Syntax Rules

- REPEATING DATA can be specified only within an INPUT PROGRAM structure, or within a FILE TYPE structure with mixed or nested data. DATA LIST, REPEATING DATA, and any transformation commands used to build the output cases must be placed within the INPUT PROGRAM or FILE TYPE structure. Transformations that apply to the output cases should be specified after the END INPUT PROGRAM or END FILE TYPE command.
- LENGTH must be used if the last variable specified on the DATA subcommand is not read from the last position of each repeating group or if the length of the repeating groups varies across input cases.
- CONTINUED must be used if repeating groups for each input case are continued on successive records.
- The DATA LIST command used with REPEATING DATA must define all fixed-format data for the records.
- Repeating groups are usually recorded at the end of the fixed-format records, but fixed-format data may follow the repeating data in data structures such as IBM SMF and RMF records. Use the following sequence in such cases.

```
DATA LIST .../* Read the fixed-format data before repeating data
REREAD COLUMNS= .../* Skip repeating data
DATA LIST .../* Read the fixed-format data after repeating data
REPEATING DATA ... /*Read repeating data
```

Operations

- Fixed-location data specified on the DATA LIST are spread to each output case.
- If LENGTH is not specified, the program uses the default length for repeating data groups, which is determined from specifications on the DATA subcommand. For more information on the default length, see the LENGTH subcommand.

Cases Generated

- The number of output cases generated is the number specified on the OCCURS subcommand. Physical record length or whether fields are non-blank does not affect the number of cases generated.
- If the number specified for OCCURS is nonpositive or missing, no cases are generated.

Records Read

- If CONTINUED is not specified, all repeating groups are read from the first record of each input case.
- If CONTINUED is specified, the first continuation record is read when the first record for the input case is exhausted, that is, when the next repeating group would extend past the end of the record. The ending column for the first record is defined on STARTS. If the ending column is not specified on STARTS, the logical record length is used.
- Subsequent continuation records are read when the current continuation record is exhausted. Exhaustion of the current continuation record is detected when the next repeating group would extend past the end of the record. The ending column for continuation records is defined on CONTINUED. If the ending column is not specified on CONTINUED, the logical record length is used.

- For inline data, the record length is always 80. For data stored in a file, the record length is generally whatever was specified on the FILE HANDLE command or the default of 1024. Shorter records are extended with blanks when they are read. For IBM implementations, the physical record length is available and is used.

Reading Past End of Record

If one or more fields extend past the end of the actual record, or if CONTINUED is specified and the ending column specified on either STARTS or CONTINUED is beyond the end of the actual record, the program takes the following action:

- For string data with format A, the data record is considered to be extended logically with blanks. If the entire field lies past the end of the record, the resulting value will be all blanks.
- For numeric data, a warning is issued and the resulting value is system-missing.

Examples

Basic Example

```
* Build a file with each case representing one vehicle and
  spread information about the household to each case.

INPUT PROGRAM.
DATA LIST / SEQNUM 2-4 NUMPERS 6-7 NUMVEH 9-10.
REPEATING DATA STARTS=12 /OCCURS=NUMVEH
 /DATA=MAKE 1-8 (A) MODEL 9 (A) NUMCYL 10.
END INPUT PROGRAM.

BEGIN DATA
1001 02 02 FORD    T8PONTIAC C6
1002 04 01 CHEVY   C4
1003 02 03 CADILAC C8FORD    T6VW      C4
END DATA.
LIST.
```

- Data are extracted from a file representing household records. Each input case is recorded on a single record; there are no continuation records.
- The total number of persons living in the house and number of vehicles owned by the household is recorded on each record. The first field of numbers (columns 1–4) for each record is an identification number unique to each record. The next two fields of numbers are number of persons in household and number of vehicles. The remainder of the record contains repeating groups of information about each vehicle: the make of vehicle, model, and number of cylinders.
- INPUT PROGRAM indicates the beginning of the input program and END INPUT PROGRAM indicates the end of the input program.
- DATA LIST reads the variables from the household portion of the record. All fixed-format variables are defined on DATA LIST.
- REPEATING DATA reads the information from the repeating groups and builds the output cases. Repeating groups start in column 12. The number of repeating groups for each input case is given by the value of variable *NUMVEH*. Three variables are defined for each repeating group: *MAKE*, *MODEL*, and *NUMCYL*.
- The first input record contains two repeating groups, producing two output cases in the active dataset. One output case is built from the second input record which contains information on one vehicle, and three output cases are built from the third record. The values of the fixed-format variables defined on DATA LIST are spread to every new case built in the active dataset. Six cases result, as shown below.

```
SEQNUM NUMPERS NUMVEH MAKE     MODEL NUMCYL

    1      2      2   FORD     T        8
    1      2      2   PONTIAC  C        6
    2      4      1   CHEVY    C        4
    3      2      3   CADILAC  C        8
    3      2      3   FORD     T        6
    3      2      3   VW       C        4

 NUMBER OF CASES READ =       6    NUMBER OF CASES LISTED =       6
```

Using REPEATING DATA With Mixed File Types

```
* Use REPEATING DATA with FILE TYPE MIXED: read only type 3 records.

FILE TYPE  MIXED RECORD=#SEQNUM 2-4.
RECORD TYPE 003.
REPEATING DATA STARTS=12 /OCCURS=3
 /DATA=MAKE 1-8(A) MODEL 9(A) NUMCYL 10.
END FILE.
END FILE TYPE.

BEGIN DATA
1001 02 02 FORD    T8PONTIAC C6
1002 04 01 CHEVY   C4
1003 02 03 CADILAC C8FORD    T6VW      C4
END DATA.
LIST.
```

- The task in this example is to read only the repeating data for records with value 003 for variable *#SEQNUM*.
- REPEATING DATA is used within a FILE TYPE structure, which specifies a mixed file type. The record identification variable *#SEQNUM* is located in columns 2–4.
- RECORD TYPE specifies that only records with value 003 for *#SEQNUM* are copied into the active dataset. All other records are skipped.
- REPEATING DATA indicates that the repeating groups start in column 12. The OCCURS subcommand indicates there are three repeating groups on each input case, and the DATA subcommand specifies names, locations, and formats for the variables in the repeating groups.
- The DATA LIST command is not required in this example, since none of the information on the input case is being spread to the output cases. However, if there were multiple input cases with value 003 for *#SEQNUM* and they did not all have three repeating groups, DATA LIST would be required to define a variable whose value for each input case indicated the number of repeating groups for that case. This variable would then be specified on the OCCURS subcommand.

Using Transformations With REPEATING DATA

```
INPUT PROGRAM.
DATA LIST / PARENTID 1 DATE 3-6 NCHILD 8.
REPEATING DATA STARTS=9 /OCCURS=NCHILD
 /DATA=BIRTHDAY 2-5 VACDATE 7-10.
END INPUT PROGRAM.

COMPUTE AGE=DATE - BIRTHDAY.
COMPUTE VACAGE=VACDATE - BIRTHDAY.

DO IF PARENTID NE LAG(PARENTID,1) OR $CASENUM EQ 1.
COMPUTE CHILD=1.
ELSE.
COMPUTE CHILD=LAG(CHILD,1)+1.
END IF.
FORMAT AGE VACAGE CHILD (F2).

BEGIN DATA
1 1987 2 1981 1983 1982 1984
2 1988 1 1979 1984
3 1988 3 1978 1981 1981 1986 1983 1986
4 1988 1 1984 1987
END DATA.
LIST.
```

- Data are from a file that contains information on parents within a school district. Each input case is recorded on a single record; there are no continuation records.
- Each record identifies the parents by a number and indicates how many children they have. The repeating groups give the year of birth and year of vaccination for each child.
- REPEATING DATA indicates that the repeating groups begin in column 9. The value of *NCHILD* indicates how many repeating groups there are for each record.
- The first two COMPUTE commands compute the age for each child and age at vaccination. These transformation commands are specified outside the input program.
- Because the repeating groups do not have descriptive values, the DO IF structure computes variable *CHILD* to distinguish between the first-born child, second-born child, and so forth. The value for *CHILD* will be 1 for the first-born, 2 for the second-born, and so forth. The LIST output is shown below.

```
PARENTID DATE NCHILD BIRTHDAY VACDATE AGE VACAGE CHILD

    1    1987    2     1981     1983    6    2      1
    1    1987    2     1982     1984    5    2      2
    2    1988    1     1979     1984    9    5      1
    3    1988    3     1978     1981   10    3      1
    3    1988    3     1981     1986    7    5      2
    3    1988    3     1983     1986    5    3      3
    4    1988    1     1984     1987    4    3      1

NUMBER OF CASES READ =       7    NUMBER OF CASES LISTED =       7
```

STARTS Subcommand

STARTS indicates the beginning location of the repeating data segment on the first record of each input case. STARTS is required and can specify either a number or a variable name.

- If the repeating groups on the first record of each input case begin in the same column, STARTS specifies a column number.
- If the repeating groups on the first record of each input case do not begin in the same column, STARTS specifies the name of a variable whose value for each input case indicates the beginning location of the repeating groups on the first record. The variable can be defined on DATA LIST or created by transformation commands that precede REPEATING DATA.
- When repeating groups are continued on multiple records for each input case, STARTS must also specify an ending location if there is room on the logical record length for more repeating groups than are contained on the first record of each input case. The ending column applies only to the first record of each input case. See the CONTINUED subcommand for an example.
- The ending column can be specified as a number or a variable name. Specifications for the beginning column and the ending column are separated by a hyphen. The values of the variable used to define the ending column must be valid values and must be larger than the starting value.
- If the variable specified for the ending column is undefined or missing for an input case, the program displays a warning message and builds no output cases from that input case. If the variable specified for the ending column on STARTS has a value that is less than the value specified for the starting column, the program issues a warning and builds output cases only from the continuation records of that input case; it does not build cases from the first record of the case.
- If the ending location is required but not supplied, the program generates output cases with system-missing values for the variables specified on the DATA subcommand and may misread all data after the first or second record in the data file (see the CONTINUED subcommand).

Repeating Groups in the Same Location

```
INPUT PROGRAM.
DATA LIST FILE=VEHICLE / SEQNUM 2-4 NUMPERS 6-7 NUMVEH 9-10.
REPEATING DATA STARTS=12 /OCCURS=NUMVEH
 /DATA=MAKE 1-8 (A) MODEL 9 (A) NUMCYL 10.
END INPUT PROGRAM.
```

- STARTS specifies column number 12. The repeating groups must therefore start in column 12 of the first record of each input case.

Repeating Groups in Varying Locations

```
INPUT PROGRAM.
DATA LIST FILE=VEHICLE / SEQNUM 2-4 NUMPERS 6-7 NUMVEH 9-10.
+    DO IF   (SEQNUM LE 100).
+    COMPUTE FIRST=12.
+    ELSE.
+    COMPUTE FIRST=15.
+    END IF.
REPEATING DATA STARTS=FIRST /OCCURS=NUMVEH
 /DATA=MAKE 1-8 (A) MODEL 9 (A) NUMCYL 10.
END INPUT PROGRAM.
```

- This example assumes that each input case is recorded on a single record and that there are no continuation records. Repeating groups begin in column 12 for all records with sequence numbers 1 through 100 and in column 15 for all records with sequence numbers greater than 100.

- The sequence number for each record is defined as variable *SEQNUM* on the DATA LIST command. The DO IF–END IF structure creates the variable *FIRST* with value 12 for records with sequence numbers through 100 and value 15 for records with sequence numbers greater than 100.
- Variable *FIRST* is specified on the STARTS subcommand.

OCCURS Subcommand

OCCURS specifies the number of repeating groups for each input case. OCCURS is required and specifies a number if the number of groups is the same for all input cases or a variable if the number of groups varies across input cases. The variable must be defined on a DATA LIST command or created with transformation commands.

Specifying the Number of Repeating Groups Using a Data Field

```
INPUT PROGRAM.
DATA LIST / SEQNUM 2-4 NUMPERS 6-7 NUMVEH 9-10.
REPEATING DATA STARTS=12 /OCCURS=NUMVEH
 /DATA=MAKE 1-8 (A) MODEL 9 (A) NUMCYL 10.
END INPUT PROGRAM.

BEGIN DATA
1001 02 02 FORD    T8PONTIAC C6
1002 04 01 CHEVY   C4
1003 02 03 CADILAC C8FORD    T6VW      C4
END DATA.
LIST.
```

- Data for each input case are recorded on a single record; there are no continuation records.
- The value for variable *NUMVEH* in columns 9 and 10 indicates the number of repeating groups on each record. One output case is built in the active dataset for each occurrence of a repeating group.
- In the data, *NUMVEH* has the value 2 for the first case, 1 for the second, and 3 for the third. Thus, six cases are built from these records. If the value of *NUMVEH* is 0, no cases are built from that record.

Specifying a Fixed Number of Repeating Groups

```
* Read only the first repeating group from each record.

INPUT PROGRAM.
DATA LIST FILE=VEHICLE / SEQNUM 2-4 NUMPERS 6-7 NUMVEH 9-10.
REPEATING DATA STARTS=12 /OCCURS=1
 /DATA=MAKE 1-8 (A) MODEL 9 (A) NUMCYL 10.
END INPUT PROGRAM.
LIST.
```

- Since OCCURS specifies that there is only one repeating group for each input case, only one output case is built from each input case regardless of the actual number of repeating groups.

DATA Subcommand

DATA specifies a name, location within each repeating segment, and format for each variable to be read from the repeating groups. DATA is required and must be the last subcommand on REPEATING DATA.

- The specifications for DATA are the same as for the DATA LIST command.
- The specified location of the variables on DATA is their location within each repeating group—*not* their location within the record.
- Any input format available on the DATA LIST command can be specified on the DATA subcommand. Both FORTRAN-like and the column-style specifications can be used.

Example

```
INPUT PROGRAM.
DATA LIST FILE=VEHICLE / SEQNUM 2-4 NUMPERS 6-7 NUMVEH 9-10.
REPEATING DATA STARTS=12 /OCCURS=NUMVEH
 /DATA=MAKE 1-8 (A) MODEL 9 (A) NUMCYL 10.
END INPUT PROGRAM.
LIST.
```

- Variable *MAKE* is a string variable read from positions 1 through 8 of each repeating group; *MODEL* is a single-character string variable read from position 9; and *NUMCYL* is a one-digit numeric variable read from position 10.
- The DATA LIST command defines variables *SEQNUM, NUMPERS*, and *NUMVEH*. These variables are spread to each output case built from the repeating groups.

FILE Subcommand

REPEATING DATA always reads the file specified on its associated DATA LIST or FILE TYPE command. The FILE subcommand on REPEATING DATA explicitly specifies the name of the file.

- FILE must specify the same file as its associated DATA LIST or FILE TYPE command.

Example

```
INPUT PROGRAM.
DATA LIST FILE=VEHICLE / SEQNUM 2-4 NUMPERS 6-7 NUMVEH 9-10.
REPEATING DATA FILE=VEHICLE /STARTS=12 /OCCURS=NUMVEH
 /DATA=MAKE 1-8 (A) MODEL 9 (A) NUMCYL 10.
END INPUT PROGRAM.
LIST.
```

- FILE on REPEATING DATA specifically identifies the *VEHICLE* file, which is also specified on the DATA LIST command.

ENCODING Subcommand

ENCODING specifies the encoding format of the file. The keyword is followed by an equals sign and a quoted encoding specification.

- In Unicode mode, the default is UTF8. For more information, see SET command, UNICODE subcommand.
- In code page mode, the default is the current locale setting. For more information, see SET command, LOCALE subcommand.
- The quoted encoding value can be: Locale (the current locale setting), UTF8, UTF16, UTF16BE (big endian), UTF16LE (little endian), a numeric Windows code page value (for example, '1252'), or an IANA code page value (for example, 'iso8859-1' or cp1252).
- In Unicode mode, the defined width of string variables is tripled for code page and UTF-16 text data files. Use ALTER TYPE to automatically adjust the defined width of string variables.

LENGTH Subcommand

LENGTH specifies the length of each repeating data group. The default length is the number of columns between the beginning column of the repeating data groups and the ending position of the last variable specified on DATA. (For the first record of each input case, STARTS specifies the beginning column of the repeating groups. For continuation records, repeating groups are read from column 1 by default or from the column specified on CONTINUED.)

- The specification on LENGTH can be a number or the name of a variable.
- LENGTH must be used if the last variable specified on the DATA subcommand is not read from the last position of each repeating group, or if the length of the repeating groups varies across input cases.
- If the length of the repeating groups varies across input cases, the specification must be a variable whose value for each input case is the length of the repeating groups for that case. The variable can be defined on DATA LIST or created with transformation commands.
- If the value of the variable specified on LENGTH is undefined or missing for an input case, the program displays a warning message and builds only one output case for that input case.

Example

```
* Read only the variable MAKE for each vehicle.

* The data contain two values that are not specified on the
  DATA subcommand.  The first is in position 9 of the repeating
  groups, and the second is in position 10.

INPUT PROGRAM.
DATA LIST FILE=VEHICLE / SEQNUM 2-4 NUMPERS 6-7 NUMVEH 9-10.
REPEATING DATA STARTS=12 /OCCURS=NUMVEH /LENGTH=10
 /DATA=MAKE 1-8 (A).
END INPUT PROGRAM.
```

- LENGTH indicates that each repeating group is 10 columns long. LENGTH is required because *MAKE* is not read from the last position of each repeating group. As illustrated in previous examples, each repeating group also includes variable *MODEL* (position 9) and *NUMCYL* (position 10).
- DATA specifies that *MAKE* is in positions 1 through 8 of each repeating group. Positions 9 and 10 of each repeating group are skipped.

CONTINUED Subcommand

CONTINUED indicates that the repeating groups are contained on more than one record for each input case.

- Each repeating group must be fully recorded on a single record: a repeating group cannot be split across records.
- The repeating groups must begin in the same column on all continuation records.
- If CONTINUED is specified without beginning and ending columns, the program assumes that the repeating groups begin in column 1 of continuation records and searches for repeating groups by scanning to the end of the record or to the value specified by OCCURS. See the topic “Operations ” on page 1664 for more information.
- If the repeating groups on continuation records do not begin in column 1, CONTINUED must specify the column in which the repeating groups begin.
- If there is room on the logical record length for more repeating groups than are contained on the first record of each input case, the STARTS subcommand must indicate an ending column for the records. The ending column on STARTS applies only to the first record of each input case.
- If there is room on the logical record length for more repeating groups than are contained on the continuation records of each input case, the CONTINUED subcommand must indicate an ending column. The ending column on CONTINUED applies to all continuation records.

Basic Example

```
* This example assumes the logical record length is 80.

INPUT PROGRAM.
DATA LIST / ORDERID 1-5 NITEMS 7-8.
REPEATING DATA STARTS=10 /OCCURS=NITEMS /CONTINUED=7
 /DATA=ITEM 1-9 (A) QUANTITY 11-13 PRICE (DOLLAR7.2,1X).
END INPUT PROGRAM.

BEGIN DATA
10020 07 01-923-89 001  25.99 02-899-56 100 101.99 03-574-54 064  61.29
10020 04-780-32 025  13.95 05-756-90 005  56.75 06-323-47 003  23.74
10020 07-350-95 014  11.46
20030 04 01-781-43 010  10.97 02-236-54 075 105.95 03-655-83 054  22.99
20030 04-569-38 015  75.00
END DATA.
LIST.
```

- Data are extracted from a mail-order file. Each input case represents one complete order. The data show two complete orders recorded on a total of five records.
- The order number is recorded in columns 1 through 5 of each record. The first three records contain information for order 10020; the next two records contain information for order 20030. The second field of numbers on the first record of each order indicates the total number of items ordered. The repeating groups begin in column 10 on the first record and in column 7 on continuation records. Each repeating data group represents one item ordered and contains three variables—the item inventory number, the quantity ordered, and the price.

- DATA LIST defines variables *ORDERID* and *NITEMS* on the first record of each input case.
- STARTS on REPEATING DATA indicates that the repeating groups on the first record of each input case begin in column 10.
- OCCURS indicates that the total number of repeating groups for each input case is the value of *NITEMS*.
- CONTINUED must be used because the repeating groups are continued on more than one record for each input case. CONTINUED specifies a beginning column because the repeating groups begin in column 7 rather than in column 1 on the continuation records.
- DATA defines variables *ITEM*, *QUANTITY*, and *PRICE* for each repeating data group. *ITEM* is in positions 1–9, *QUANTITY* is in positions 11–13, and *PRICE* is in positions 14–20 and is followed by one blank column. The length of the repeating groups is therefore 21 columns. The LIST output is shown below.

```
ORDERID NITEMS ITEM      QUANTITY      PRICE

 10020     7   01-923-89      1       $25.99
 10020     7   02-899-56    100      $101.99
 10020     7   03-574-54     64       $61.29
 10020     7   04-780-32     25       $13.95
 10020     7   05-756-90      5       $56.75
 10020     7   06-323-47      3       $23.74
 10020     7   07-350-95     14       $11.46
 20030     4   01-781-43     10       $10.97
 20030     4   02-236-54     75      $105.95
 20030     4   03-655-83     54       $22.99
 20030     4   04-569-38     15       $75.00

NUMBER OF CASES READ =      11    NUMBER OF CASES LISTED =      11
```

Specifying an Ending Column on the STARTS Subcommand

```
* This example assumes the logical record length is 80.

INPUT PROGRAM.
DATA LIST / ORDERID 1-5 NITEMS 7-8.
REPEATING DATA STARTS=10-55 /OCCURS=NITEMS /CONTINUED=7
 /DATA=ITEM 1-9 (A) QUANTITY 11-13 PRICE (DOLLAR7.2,1X).
END INPUT PROGRAM.

BEGIN DATA
10020 07 01-923-89 001  25.99 02-899-56 100 101.99
10020 03-574-54 064  61.29 04-780-32 025  13.95 05-756-90 005  56.75
10020 06-323-47 003  23.74 07-350-95 014  11.46
20030 04 01-781-43 010  10.97 02-236-54 075 105.95
20030 03-655-83 054  22.99 04-569-38 015  75.00
END DATA.
LIST.
```

- Data are the same as in the previous example; however, records are entered differently. The first record for each input case contains only two repeating groups.
- DATA LIST defines variables *ORDERID* and *NITEMS* in columns 1–8 on the first record of each input case. Column 9 is blank. DATA defines variables *ITEM*, *QUANTITY*, and *PRICE* in positions 1–20 of each repeating group, followed by a blank. Thus, each repeating group is 21 columns wide. The length of the first record of each input case is therefore 51 columns: 21 columns for each of two repeating groups, plus the eight columns defined on DATA LIST, plus column 9, which is blank. The operating system's logical record length is 80, which allows room for one more repeating group on the first record of each input case. STARTS must therefore specify an ending column that does not provide enough columns for another repeating group; otherwise, the program creates an output case with missing values for the variables specified on DATA.
- STARTS specifies that the program is to scan only columns 10–55 of the first record of each input case looking for repeating data groups. It will scan continuation records beginning in column 7 until the value specified on the OCCURS subcommand is reached.

Specifying an Ending Column on the CONTINUED Subcommand

```
* This example assumes the logical record length is 80.

INPUT PROGRAM.
DATA LIST / ORDERID 1-5 NITEMS 7-8.
REPEATING DATA STARTS=10-55 /OCCURS=NITEMS /CONTINUED=7-55
 /DATA=ITEM 1-9 (A) QUANTITY 11-13 PRICE (DOLLAR7.2,1X).
END INPUT PROGRAM.

BEGIN DATA
10020 07 01-923-89 001  25.99 02-899-56 100 101.99
10020 03-574-54 064  61.29 04-780-32 025  13.95
10020 05-756-90 005  56.75 06-323-47 003  23.74
10020 07-350-95 014  11.46
20030 04 01-781-43 010  10.97 89-236-54 075 105.95
20030 03-655-83 054  22.99 04-569-38 015  75.00
```

```
END DATA.
LIST.
```

- The data are the same as in the previous two examples, but records are entered differently. The first record and the continuation records for each input case store only two repeating groups each.
- The operating system's logical record length is 80, which allows room for more repeating groups on all records.
- STARTS specifies that the program is to scan only columns 10-55 of the first record of each input case looking for repeating data groups.
- CONTINUED specifies that the program is to scan only columns 7–55 of all continuation records.

ID Subcommand

ID compares the value of an identification variable across records of the same input case. ID can be used only when CONTINUED is specified. The identification variable must be defined on a DATA LIST command and must be recorded on all records in the file.

- The ID subcommand has two specifications: the location of the variable on the continuation records and the name of the variable (as specified on the DATA LIST command). The specifications must be separated from each other by an equals sign.
- The format specified on the ID subcommand must be the same as the format specified for the variable on DATA LIST. However, the location can be different on the continuation records.
- If the values of the identification variable are not the same on all records for a single input case, the program displays an error message and stops reading data.

Example

```
INPUT PROGRAM.
DATA LIST / ORDERID 1-5 NITEMS 7-8.
REPEATING DATA STARTS=10-50 /OCCURS=NITEMS
 /CONTINUED=7 /ID=1-5=ORDERID
 /DATA=ITEM 1-9 (A) QUANTITY 11-13 PRICE 15-20 (2).
END INPUT PROGRAM.

BEGIN DATA
10020 04 45-923-89 001  25.9923-899-56 100 101.99
10020 63-780-32 025  13.9554-756-90 005  56.75
20030 03 45-781-43 010  10.9789-236-54 075 105.95
20030 32-569-38 015  75.00
END DATA.
LIST.
```

- The order number in the data is recorded in columns 1–5 of each record.
- *ORDERID* is defined on the DATA LIST command as a five-column integer variable. The first specification on the ID subcommand must therefore specify a five-column integer variable. The location of the variable can be different on continuation records.

TABLE and NOTABLE Subcommands

TABLE displays a table summarizing all variables defined on the DATA subcommand. The summary table lists the names, locations, and formats of the variables and is identical in format to the summary table displayed by the DATA LIST command. NOTABLE suppresses the table. TABLE is the default.

Example

```
INPUT PROGRAM.
DATA LIST FILE=VEHICLE / SEQNUM 2-4 NUMPERS 6-7 NUMVEH 9-10.
REPEATING DATA STARTS=12 /OCCURS=NUMVEH /NOTABLE
 /DATA=MAKE 1-8 (A) MODEL 9 (A) NUMCYL 10.
END INPUT PROGRAM.
```

- NOTABLE suppresses the display of the summary table.

REPORT

REPORT is available in the Statistics Base Edition.

```
REPORT [/FORMAT=[{MANUAL   }] [{NOLIST   }] [ALIGN({LEFT  })] [TSPACE({1})]
               {AUTOMATIC}   {LIST[(n)]}         {CENTER}           {n}
                                                 {RIGHT }
       [CHDSPACE({1})] [FTSPACE({1})] [SUMSPACE({1})] [COLSPACE({4})]
                  {n}            {n}            {n}             {n}
       [BRKSPACE({ 1 })][LENGTH({1,length})] [MARGINS({1,width})]
                  { n }          {t,b     }           {l,r    }
                  {-1†}          {*,*     }           {*,*    }
       [CHALIGN({TOP    })] [UNDERSCORE({OFF})] [PAGE1({1})] [MISSING {'.'}]]
                {BOTTOM†}                {ON†}          {n}          {'s'}
       [ONEBREAKCOLUMN {OFF**}] [INDENT {2**}] [CHWRAP {OFF**}] [PREVIEW {OFF**}]
                       {ON   }          {n  }          {ON   }           {ON   }
 [/OUTFILE=file]
 [/STRING=stringname (varname[(width)] [(BLANK)] ['literal'])
 /VARIABLES=varname ({VALUE}) [+ varname({VALUE})] ['col head'] [option list]
                    {LABEL}             {LABEL}
                    {DUMMY}             {DUMMY}

where option list can contain any of the following:


      (width)   (OFFSET({0      }))   ({LEFT   })
                        {n      }      {CENTER†}
                        {CENTER†}      {RIGHT  }

 [/MISSING={VAR                   }]
           {NONE                  }
           {LIST[([varlist][{1}])]}
                          {n}

 [/TITLE=[{LEFT  }] 'line1' 'line2'...]   [/FOOTNOTE=[{LEFT  }] 'line1' 'line2'...]
          {CENTER}                                  {CENTER}
          {RIGHT }                                  {RIGHT }
         [)PAGE]   [)DATE]   [)var]

 [/BREAK=varlist ['col head'] [option list]]

where option list can contain any of the following:


      (width)   ({VALUE })   ({NOTOTAL})   ({SKIP({1}     }))
                 {LABEL†}     {TOTAL  }          {n}
                                            {PAGE[(RESET)]}
      (OFFSET({0      }))   (UNDERSCORE[(varlist)])   ({LEFT   })   ({NONAME})
             {n      }                                 {CENTER†}     {NAME  }
             {CENTER†}                                 {RIGHT  }

 [/SUMMARY=function...['summary title']][(break col #)] [SKIP({0})]
                                                        {n}
       or
 [/SUMMARY=PREVIOUS[({1})]]
                    {n}

where function is


 aggregate [(varname[({PLAIN   })][(d)][varname...])]
                    {format††}
      or
 composite(argument)[(report col[({PLAIN   })][(d)])]
                                 {format††}
```

**Default if the keyword is omitted.

†Default if FORMAT=AUTOMATIC.

††Any printable output format is valid. See FORMATS.

Aggregate functions: VALIDN, VARIANCE, PLT(n), SUM, KURTOSIS, PIN(min, max), MIN, SKEWNESS, FREQUENCY(min, max), MAX, MEDIAN(min, max), PERCENT(min, max), MEAN, MODE(min, max), STDDEV, PGT(n).

Composite functions: DIVIDE(arg_1 arg_2 [factor]), MULTIPLY(arg_1...arg_n), PCT(arg_1 arg_2), SUBTRACT(arg_1 arg_2), ADD(arg_1...arg_n), GREAT(arg_1...arg_n), LEAST(arg_1...arg_n), AVERAGE(arg_1...arg_n) .

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
REPORT FORMAT=LIST
  /VARIABLES=PRODUCT (LABEL) ' ' 'Retail' 'Products'
           SALES 'Annual' 'Sales' '1981'
```

```
/BREAK=DEPT 'Department' (LABEL)
/SUMMARY=VALIDN (PRODUCT) MEAN (SALES).
```

Overview

REPORT produces case listings and summary statistics and gives you considerable control over the appearance of the output. REPORT calculates all the univariate statistics available in DESCRIPTIVES and the statistics and subpopulation means available in MEANS. In addition, REPORT calculates statistics not directly available in any other procedure, such as computations involving aggregated statistics.

REPORT provides complete report format defaults but also lets you customize a variety of table elements, including column widths, titles, footnotes, and spacing. Because REPORT is so flexible and the output has so many components, it is often efficient to preview report output using a small number of cases until you find the format that best suits your needs.

Basic specification

The basic specification depends on whether you want a listing report or a summary report. A listing report without subgroup classification requires FORMAT and VARIABLES. A listing report with subgroup classification requires FORMAT, VARIABLES, and BREAK. A summary report requires VARIABLES, BREAK, and SUMMARY.

Listing reports. FORMAT=LIST and VARIABLES with a variable list are required. Case listings are displayed for each variable named on VARIABLES. There are no break groups or summary statistics unless BREAK or SUMMARY is specified.

Summary reports. VARIABLES, BREAK, and SUMMARY are required. The report is organized according to the values of the variable named on BREAK. The variable named on BREAK must be named on a preceding SORT CASES command. Specified statistics are computed for the variables specified on VARIABLES for each subgroup defined by the break variables.

Subcommand order

The following order must be observed among subcommands when they are used:

- FORMAT must precede all other subcommands.
- VARIABLES must precede BREAK.
- OUTFILE must precede BREAK.
- Each SUMMARY subcommand must immediately follow its associated BREAK. Multiple SUMMARY subcommands associated with the same BREAK must be specified consecutively.
- TITLE and FOOTNOTE can appear anywhere after FORMAT except between BREAK and SUMMARY.
- MISSING must follow VARIABLES and precede the first BREAK.
- STRING must precede VARIABLES.

Syntax rules

- Only one each of the FORMAT, STRING, VARIABLES, and MISSING subcommands is allowed.
- To obtain multiple break groups, use multiple BREAK subcommands.
- To obtain multiple summaries for a break level, specify multiple SUMMARY subcommands for the associated BREAK.
- Keywords on REPORT subcommands have default specifications that are in effect if the keyword is not specified. Specify keywords only when you wish to change a default.
- Keywords are enclosed in parentheses if the subcommand takes variable names as arguments.

Operations

- REPORT processes cases sequentially. When the value of a break variable changes, REPORT displays a statistical summary for cases processed since the last set of summary statistics was displayed. Thus, the file must be sorted in order on the break variable or variables.

- The maximum width and page length of the report are determined by the SET command.
- If a column is not wide enough to display numeric values, REPORT first rounds decimal digits, then converts to scientific notation if possible, and then displays asterisks. String variables that are wider than the column are truncated.
- The format used to display values in case listings is controlled by the dictionary format of the variable. Each statistical function in REPORT has a default format.

Limitations

- Maximum 500 variables per VARIABLES subcommand. You can specify more than 500 variables if you stack them. See the topic "VARIABLES subcommand " on page 1679 for more information.
- Maximum 10 dummy variables per VARIABLES subcommand.
- Maximum 20 MODE and MEDIAN requests per SUMMARY subcommand.
- Maximum 20 PGT, PLT, and PIN requests per SUMMARY subcommand.
- Maximum 50 strings per STRING subcommand.
- The length of titles and footnotes cannot exceed the report width.
- The length of string variables created on STRING cannot exceed the page width.
- There is no fixed limit on the number of BREAK and SUMMARY subcommands. However, the page width limits the number of variables that can be displayed and thereby limits the number of break variables.
- The maximum width of a report is 255 characters.
- The number of report variables that can be specified depends upon the width of the report, the width of the variable columns, and the number of BREAK subcommands.
- Maximum 50 variables for the FREQUENCY or PERCENT functions.
- Memory requirements significantly increase if FREQUENCY, PERCENT, MEDIAN, or MODE is requested for variables with a wide range of values. The amount of workspace required is 20 + 8*(max – min + 1) bytes per variable per function per break. If the same range is used for different statistics for the same variable, only one set of cells is collected. For example, FREQUENCY(1,100)(VARA) PERCENT(1,100)(VARA) requires only 820 bytes.
- If TOTAL is in effect, workspace requirements are almost doubled.
- Memory requirements also increase if value labels are displayed for variables with many value labels. The amount of workspace required is 4 + 24*n bytes per variable, where n is the number of value labels specified for the variable.

Examples

```
SORT CASES BY DEPT.
REPORT FORMAT=LIST
  /VARIABLES=PRODUCT (LABEL) ' ' 'Retail' 'Products'
           SALES 'Annual' 'Sales' '1981'
  /BREAK=DEPT 'Department' (LABEL)
  /SUMMARY=VALIDN (PRODUCT) MEAN (SALES) 'No.Sold,Mean Sales'.
```

- This report is a listing of products and sales by department. A summary of the total number of products sold and the average sales by department is also produced.
- Cases are first sorted by *DEPT* so that cases are grouped by department for the case listing and for the calculation of statistics.
- FORMAT requests a report that lists individual cases within each break group.
- VARIABLES specifies *PRODUCT* and *SALES* as the report variables. Keyword LABEL requests that the case listings for *PRODUCT* display value labels instead of values. Three-line column headings are provided for each report column. The first line of the column heading is blank for the variable *PRODUCT*.
- BREAK identifies *DEPT* as the break variable and provides a one-line column title for the break column. LABEL displays the value label instead of the value itself.

- SUMMARY calculates the valid number of cases for *PRODUCT* and the mean of *SALES* for each value of *DEPT*. A title is provided for the summary line to override the default title, *VALIDN*.

Defaults

Column heads. REPORT uses variable labels as default column heads; if no variable labels have been specified, variable names are used. If ONEBREAKCOLUMN is ON, the default head for the first BREAK subcommand is used.

Column widths. Default column widths are determined by REPORT, using the maximum of the following for each column:

- The widest print format in the column, whether it is a variable print format or a summary print format.
- The width of any temporary variable defined with the STRING subcommand on REPORT.
- If a column heading is assigned, the length of the longest title line in the heading when CHWRAP is off, and the longest word in the title when CHWRAP is on. Underscores, which are removed on printing, can be used to create longer words in the title.
- When no column heading is specified, the length of the longest word in the variable label, or the length of the variable name.
- If you specify LABEL on VARIABLES or BREAK, the length of the variable's longest value label. If FORMAT=MANUAL is in effect, 20 is the maximum value used for this criterion.
- The minimum column width is 8 when FORMAT=MANUAL; it can be less when FORMAT=AUTOMATIC.

Automatic fit. When the above criteria for column width result in a report that is too wide for the report margins, FORMAT=AUTOMATIC shrinks the report. AUTOMATIC performs the following two steps sequentially, stopping as soon as the report fits within the margins:

1. AUTOMATIC reduces intercolumn spacing incrementally until it reaches a minimum intercolumn space of 1. It will never reduce it to 0.
2. AUTOMATIC shortens widths for strings specified on the STRING subcommand or for value label strings when the LABEL option is specified. It begins with the longest string if that string is at least 15 characters wide and shortens the column width as much as needed (up to 40% of its length), wrapping the string within the new width. If necessary, it repeats the step, using different defined strings. It will not shorten the column width of the same string twice.

REPORT does *not* implement the automatic fit unless AUTOMATIC is specified on the FORMAT subcommand.

AUTOMATIC versus MANUAL defaults. Many default settings depend on whether you specify AUTOMATIC or MANUAL on FORMAT.

[1] No effect when ONEBREAKCOLUMN is on.

[2] LEFT when ONEBREAKCOLUMN is on.

[3] 0 when ONEBREAKCOLUMN is on.

Options

Format. REPORT provides full format defaults and offers you optional control over page length, vertical spacing, margin and column widths, page titles, footnotes, and labels for statistics. The maximum width and length of the report are controlled by specifications on the SET command. The FORMAT subcommand on REPORT controls how the report is laid out on a page and whether case listings are displayed. The VARIABLES subcommand specifies the variables that are listed or summarized in the report (**report variables**) and controls the titles, width, and contents of report columns. The BREAK subcommand specifies the variables that define groups (**break variables**) and controls the titles, width, and contents of break columns. SUMMARY specifies statistics and controls the titles and spacing of summary lines. The TITLE and FOOTNOTE subcommands control the specification and placement of multiple-line titles and

footnotes. STRING concatenates variables to create temporary variables that can be specified on VARIABLES or BREAK.

Output file. You can direct reports to a file separate from the file used for the rest of the output from your session using the OUTFILE subcommand.

Statistical display. The statistical display is controlled by the SUMMARY subcommand. Statistics can be calculated for each category of a break variable and for the group as a whole. Available statistics include mean, variance, standard deviation, skewness, kurtosis, sum, minimum, maximum, mode, median, and percentages. Composite functions perform arithmetic operations using two or more summary statistics calculated on single variables.

Missing values. You can override the default to include user-missing values in report statistics and listings with the MISSING subcommand. You can also use FORMAT to define a missing-value symbol to represent missing data.

FORMAT subcommand

FORMAT controls the overall width and length of the report and vertical spacing. Keywords and their arguments can be specified in any order.

- MANUAL and AUTOMATIC are alternatives. The default is MANUAL.
- LIST and NOLIST are alternatives. The default is NOLIST.

MANUAL. *Default settings for manual format.* MANUAL displays values for break variables, right-justifies numeric values and their column headings, left-justifies value labels and string values and their column headings, top-aligns and does not underscore column headings, extends column widths to accommodate the variable's longest value label (but not the longest word in the variable label) up to a width of 20, and generates an error message when a report is too wide for its margins. MANUAL is the default.

AUTOMATIC. *Default settings for automatic format.* AUTOMATIC displays labels for break variables, centers all data, centers column headings but left-justifies column headings if value labels or string values exceed the width of the longest word in the heading, bottom-aligns and underscores column headings, extends column widths to accommodate the longest word in a variable label or the variable's longest value label, and shrinks a report that is too wide for its margins.

LIST(n). *Individual case listing.* The values of all variables named on VARIABLES are displayed for each case. The optional *n* inserts a blank line after each *n* cases. By default, no blank lines are inserted. Values for cases are listed using the default formats for the variables.

NOLIST. *No case listing.* This is the default.

PAGE(n). *Page number for the first page of the report.* The default is 1.

LENGTH(t,b) . *Top and bottom line numbers of the report.* You can specify any numbers to define the report page length. By default, the top of the report begins at line 1, and the bottom of the report is the last line of the system page. You can use an asterisk for *t* or *b* to indicate a default value. If the specified length does not allow even one complete line of information to be displayed, REPORT extends the length specification and displays a warning.

MARGINS(l,r) . *Columns for the left and right margins.* The right column cannot exceed 255. By default, the left margin is display column 1 and the right margin is the rightmost display column of the system page. You can use an asterisk for *l* or *r* to indicate a default value.

ALIGN. *Placement of the report relative to its margins.* LEFT, CENTER, or RIGHT can be specified in the parentheses following the keyword. LEFT left-justifies the report. CENTER centers the report between its margins. RIGHT right-justifies the report. The default is LEFT.

COLSPACE(n) . *Number of spaces between each column.* The default is 4 or the average number of spaces that will fit within report margins, whichever is less. When AUTOMATIC is in effect, REPORT overrides the specified column spacing if necessary to fit the report between its margins.

CHALIGN. *Alignment of column headings.* Either TOP or BOTTOM can be specified in the parentheses following the keyword. TOP aligns all column headings with the first, or top, line of multiple-line headings. BOTTOM aligns headings with the last, or bottom, line of multiple-line headings. When AUTOMATIC is in effect, the default is BOTTOM; when MANUAL is in effect, the default is TOP.

UNDERSCORE . *Underscores for column headings.* Either ON or OFF can be specified in the parentheses following the keyword. ON underscores the bottom line of each column heading for the full width of the column. OFF does not underscore column headings. The default is ON when AUTOMATIC is in effect and OFF when MANUAL is in effect.

TSPACE(n). *Number of blank lines between the report title and the column heads.* The default is 1.

CHDSPACE(n) . *Number of blank lines beneath the longest column head.* The default is 1.

BRKSPACE(n) . *Number of blank lines between the break head and the next line.* The next line is a case if LIST is in effect or the first summary line if NOLIST is in effect. BRKSPACE(−1) places the first summary statistic or the first case listing on the same line as the break value. When a summary line is placed on the same line as the break value, the summary title is suppressed. When AUTOMATIC is in effect, the default is –1; when MANUAL is in effect, it is 1.

SUMSPACE(n) . *Number of blank lines between the last summary line at the lower break level and the first summary line at the higher break level when they break simultaneously.* SUMSPACE also controls spacing between the last listed case and the first summary line if LIST is in effect. The default is 1.

FTSPACE(n) . *Minimum number of blank lines between the last listing on the page and the footnote.* The default is 1.

MISSING 's' . *Missing-value symbol.* The symbol can be only one character and represents both system- and user-missing values. The default is a period.

ONEBREAKCOLUMN. *Display subgroups defined on multiple BREAK subcommands in a single column.* You can specify OFF or ON in parentheses after the keyword. The default is OFF. When ONEBREAKCOLUMN is ON, it applies to all BREAK subcommands. See the topic "BREAK subcommand " on page 1682 for more information.

INDENT(n). *Indention of break values and summary titles of each successive subgroup defined by one BREAK subcommand in a single break column.* INDENT is effective only when ONEBREAKCOLUMN is on. Multiple variables specified on one BREAK subcommand are indented as a block. The default specification is 2. When ONEBREAKCOLUMN is OFF, specification on INDENT is ignored.

CHWRAP. *Automatically wrap user-specified column heads.* You can specify OFF or ON in parentheses after the keyword. The default is OFF. When CHWRAP is ON, user-specified heads for either break or variable columns are wrapped. If multiple lines are specified for a head, each line is wrapped, if necessary, independent of other lines. To prevent wrapping at blanks, use the underscore character (_) to signify a hard blank in your head specification. The underscore serves as a hard blank only in user-specified heads and only when CHWRAP is ON. The underscore does not appear in the printed heading.

PREVIEW. *Display the first page of output only.* You can specify OFF or ON either in parentheses or with one blank space separating the specification from the keyword. The default is OFF. When PREVIEW is ON, the program stops processing after the first page for you to quickly check the format of your report.

OUTFILE subcommand

OUTFILE directs the report to a file separate from the file used for the rest of the output from your session. This allows you to print the report without having to delete the extraneous material that would be present in the output.

- OUTFILE must follow FORMAT and must precede BREAK.
- You can append multiple reports to the same file by naming the same file on the OUTFILE subcommand for each REPORT command.

Note: The first time a particular file is named on the OUTFILE subcommand within a job or session, the file is created or, if it already exists, it is erased. It remains open until the job or session finishes. Later uses of the same file in REPORT append their output to it. Attempts to erase the file may fail.

Example

```
REPORT FORMAT=AUTOMATIC LIST
  /OUTFILE=PRSNLRPT
  /VARIABLES=LNAME AGE TENURE JTENURE SALARY
  /BREAK=DIVISION
  /SUMMARY=MEAN.

REPORT FORMAT=AUTOMATIC
  /OUTFILE=PRSNLRPT
  /VARIABLES=LNAME AGE TENURE JTENURE SALARY
  /BREAK=DIVISION
  /SUMMARY=MEAN
  /SUMMARY=MIN
  /SUMMARY=MAX.
```

- Both a listing report and a summary report are written to file *PRSNLRPT*.

VARIABLES subcommand

The required VARIABLES subcommand names the variables to be listed and summarized in the report. You can also use VARIABLES to control column titles, column widths, and the contents of report columns.

- The minimum specification on VARIABLES is a list of report variables. The number of variables that can be specified is limited by the system page width.
- Each report variable defines a report column. The value of the variable or an aggregate statistic calculated for the variable is displayed in that variable's report column.
- Variables are assigned to columns in the order in which they are named on VARIABLES.
- Variables named on BREAK can also be named on VARIABLES.
- When FORMAT=LIST, variables can be stacked in a single column by linking them with plus signs (+) on the VARIABLES subcommand. If no column heading is specified, REPORT uses the default heading from the first variable on the list. Only values from the first variable in the column are used to calculate summaries.
- Optional specifications apply only to the immediately preceding variable or list of variables implied by the TO keyword. Options can be specified in any order.
- All optional specifications except column headings must be enclosed in parentheses; column headings must be enclosed in quotes.

Column contents

The following options can be used to specify the contents of the report column for each variable:

(VALUE). *Display the values of the variable.* This is the default.

(LABEL). *Display value labels*. If value labels are not defined, values are displayed.

(DUMMY). *Display blank spaces.* DUMMY defines a report column for a variable that does not exist in the active dataset. Dummy variables are used to control spacing or to reserve space for statistics computed for other variables. Do not name an existing variable as a dummy variable.

- VALUE and LABEL have no effect unless LIST has been specified on the FORMAT subcommand.
- When AUTOMATIC is in effect, value labels or string values are centered in the column based on the length of the longest string or label; numeric values are centered based on the width of the widest value or summary format. When MANUAL is in effect, value labels or string values are left-justified in the column and numeric values are right-justified. (See the OFFSET keyword.)

Column heading

The following option can be used to specify a heading for the report column:

'column heading'. *Column heading for the preceding variable.* The heading must be enclosed in quotes. If no column heading is specified, the default is the variable label or, if no variable label has been specified, the variable name.

- To specify multiple-line headings, enclose each line in quotes, using the conventions for strings. The specifications for title lines should be separated by at least one blank.
- Default column headings wrap for as many lines as are required to display the entire label. If AUTOMATIC is in effect, user-specified column headings appear exactly as specified, even if the column width must be extended. If MANUAL is in effect, user-specified titles wrap to fit within the column width.

Column heading alignment

The following options can be used to specify how column headings are aligned:

(LEFT). *Left-aligned column heading.*

(CENTER). *Centered column heading.*

(RIGHT). *Right-aligned column heading.*

- If AUTOMATIC is in effect, column headings are centered within their columns by default. If value labels or string values exceed the width of the longest word in the heading, the heading is left-justified.
- If MANUAL is in effect, column headings are left-justified for value labels or string values and right-justified for numeric values by default.

Column format

The following options can be used to specify column width and adjust the position of the column contents:

(width). *Width for the report column.* If no width is specified for a variable, REPORT determines a default width using the criteria described under Defaults. If you specify a width that is not wide enough to display numeric values, REPORT first rounds decimal digits, then converts to scientific notation if possible, and then displays asterisks. Value labels or string values that exceed the width are wrapped.

(OFFSET). *Position of the report column contents.* The specification is either *n* or CENTER specified in parentheses. OFFSET(*n*) indicates the number of spaces to offset the contents from the left for value labels or string values, and from the right for numeric values. OFFSET(CENTER) centers contents within the center of the column. If AUTOMATIC is in effect, the default is CENTER. If MANUAL is in effect, the default is 0. Value labels and string values are left-justified and numeric values are right-justified.

Example

```
/VARIABLES=V1 TO V3(LABEL) (15)
  V4 V5 (LABEL)(OFFSET (2))(10)
  SEP1 (DUMMY) (2) ''
  V6 'Results using' "Lieben's Method" 'of Calculation'
```

- The width of the columns for variables *V1* through *V3* is 15 each. Value labels are displayed for these variables in the case listing.
- The column for variable *V4* uses the default width. Values are listed in the case listing.
- Value labels are displayed for variable *V5*. The column width is 10. Column contents are offset two spaces from the left.
- *SEP1* is a dummy variable. The column width is 2, and there is at least one space on each side of *SEP1*. Thus, there are at least four blanks between the columns for *V5* and *V6*. *SEP1* is given a null title to override the default column title *SEP1*.
- *V6* has a three-line title. Its column uses the default width, and values are listed in the case listing.

STRING subcommand

STRING creates a temporary string variable by concatenating variables and user-specified strings. These variables exist only within the REPORT procedure.

- The minimum specification is a name for the string variable followed by a variable name or a user-specified string enclosed in parentheses.
- The name assigned to the string variable must be unique.
- Any combination of string variables, numeric variables, and user-specified strings can be used in the parentheses to define the string.
- Keyword TO cannot be used within the parentheses to imply a variable list.
- More than one string variable can be defined on STRING.
- If a case has a missing value for a variable within the parentheses, the variable passes the missing value to the temporary variable without affecting other elements specified.
- A string variable defined in REPORT cannot exceed the system page width.
- String variables defined on STRING can be used on VARIABLES or BREAK.

The following options can be used to specify how components are to be concatenated:

(width). *Width of the preceding variable within the string.* The default is the dictionary width of the variable. The maximum width for numeric variables within the string definition is 16. The maximum width for a string variable is the system page width. If the width specified is less than that required by the value, numeric values are displayed as asterisks and string values are truncated. If the width exceeds the width of a value, numeric values are padded with zeros on the left and string values are padded with blanks on the right.

(BLANK). *Left-pad values of the preceding numeric variable with blanks.* The default is to left-pad values of numeric variables with zeros. If a numeric variable has a dollar or comma format, it is automatically left-padded with blanks.

'literal' . *User-specified string.* Any combination of characters can be specified within quotes.

Example

```
/STRING=JOB1(AVAR NVAR)
        JOB2(AVAR(2) NVAR(3))
        JOB3(AVAR(2) NVAR(BLANK) (4))
```

- STRING defines three string variables to be used within the report.
- Assume that *AVAR* is a string variable read from a four-column field using keyword FIXED on DATA LIST and that *NVAR* is a computed numeric variable with the default format of eight columns with two implied decimal places.
- If a case has value KJ for *AVAR* and value 241 for *NVAR*, *JOB1* displays the value 'KJ 00241.00', *JOB2* the value 'KJ241', and *JOB3* the value 'KJ 241'. If *NVAR* has the system-missing value for a case, *JOB1* displays the value 'KJ'.

Example

```
 /STRING=SOCSEC(S1 '-' S2 '-' S3)
```

- STRING concatenates the three variables *S1*, *S2*, and *S3*, each of which contains a segment of the social security number.
- Hyphens are inserted between the segments when the values of *SOCSEC* are displayed.
- This example assumes that the variables *S1*, *S2*, and *S3* were read from three-column, two-column, and four-column fields respectively, using the keyword FIXED on DATA LIST. These variables would then have default format widths of 3, 2, and 4 columns and would not be left-padded with zeros.

BREAK subcommand

BREAK specifies the variables that define the subgroups for the report, or it specifies summary totals for reports with no subgroups. BREAK also allows you to control the titles, width, and contents of break columns and to begin a new page for each level of the break variable.

- A break occurs when any one of the variables named on BREAK changes value. Cases must be sorted by the values of all BREAK variables on all BREAK subcommands.
- The BREAK subcommand must precede the SUMMARY subcommand that defines the summary line for the break.
- A break column is reserved for each BREAK subcommand if ONEBREAKCOLUMN is OFF (the default).
- To obtain multiple break levels, specify multiple break variables on a BREAK subcommand.
- If more than one variable is specified on a BREAK subcommand, a single break column is used. The value or value label for each variable is displayed on a separate line in the order in which the variables are specified on BREAK. The first variable specified changes most slowly. The default column width is the longest of the default widths for any of the break variables.
- To obtain summary totals without any break levels, use keyword TOTAL in parentheses on BREAK without listing any variables. TOTAL must be specified on the first BREAK subcommand.
- When MISSING=VAR is specified, user-missing values are displayed in case listings but are not included in summary statistics. When NONE is specified, user-missing values are ignored. System-missing values are displayed as missing in case and break listings.
- Optional specifications apply to all variables in the break column and to the break column as a whole. Options can be specified in any order following the last variable named.
- All optional specifications except column headings must be enclosed in parentheses; column headings must be enclosed in quotes.

Column contents

The following can be used to specify the contents of the break column:

(VALUE). *Display values of the break variables.*

(LABEL). *Display value labels*. If no value labels have been defined, values are displayed.

- The value or label is displayed only once for each break change but it is repeated at the top of the page in a multiple-page break group.
- When AUTOMATIC is in effect, the default is LABEL; when MANUAL is in effect, the default is VALUE.
- When AUTOMATIC is in effect, the value or label is centered in the column. When MANUAL is in effect, value labels and string values are left-justified and numeric values are right-justified. Keywords OFFSET, ONEBREAKCOLUMN, and INDENT can also affect positioning.

Column heading

The following option specifies headings used for the break column.

'column heading'. *Column heading for the break column.* The heading must be included in quotes. The default heading is the variable label of the break variable or, if no label has been defined, the variable name. If the break column is defined by more than one variable, the label or name of the first variable is used. If ONEBREAKCOLUMN is ON, the specified or implied column heading for the first BREAK subcommand is used.

- To specify multiple-line headings, enclose each line in a set of quotes, following the conventions for strings. Separate the specifications for heading lines with at least one blank.
- Default column headings wrap for as many lines as are required to display the entire label.
- User-specified column headings appear exactly as specified if CHWRAP is OFF (the default). If CHWRAP is ON, any user-defined line longer than the specified or default column width is automatically wrapped.

Column heading alignment

The following options can be used to specify how column headings are aligned:

(LEFT). *Left-aligned column heading.*

(CENTER). *Centered column heading.*

(RIGHT). *Right-aligned column heading.*

- When AUTOMATIC is in effect, column headings are centered within their columns by default. If value labels or string values exceed the width of the longest word in the heading, the heading is left-justified.
- When MANUAL is in effect, column headings are left-justified for value labels or string values and right-justified for numeric values.
- When ONEBREAKCOLUMN is ON, all column contents are left aligned. Specifications of CENTER and RIGHT on BREAK are ignored.

Column format

The following options can be used to format break columns:

(width). *Column width for the break column.* If no width is specified for a variable, REPORT determines a default width using the criteria described under Defaults. If ONEBREAKCOLUMN is ON, the column width specified or implied by the first BREAK subcommand is used. If you specify a width that is not wide enough to display numeric values, REPORT first rounds decimal digits, then converts them to scientific notation if possible, and then displays asterisks. Value labels or string values that exceed the width are wrapped.

(OFFSET). *Position of the break column contents.* The specification is either *n* or CENTER specified in parentheses. OFFSET(*n*) indicates the number of spaces to offset the contents from the left for value labels or string values, and from the right for numeric values. OFFSET(CENTER) centers contents within the column. If AUTOMATIC is in effect, the default is CENTER. If MANUAL is in effect, the default is 0: value labels and string values are left-justified and numeric values are right-justified. If ONEBREAKCOLUMN is ON, the offset is applied along with the indentation specified on INDENT, always from the left. The specification of CENTER on OFFSET is ignored.

(UNDERSCORE). *Use underscores below case listings.* Case listing columns produced by FORMAT LIST are underscored before summary statistics are displayed. You can optionally specify the names of one or more report variables after UNDERSCORE; only the specified columns are underscored.

(TOTAL). *Display the summary statistics requested on the next SUMMARY subcommand for all the cases in the report.* TOTAL must be specified on the first BREAK subcommand and applies only to the next SUMMARY subcommand specified.

(NOTOTAL). *Display summary statistics only for each break.* This is the default.

(SKIP(n)). *Skip* n *lines after the last summary line for a break before beginning the next break*. The default for *n* is 1.

(PAGE). *Begin each break on a new page.* If RESET is specified on PAGE, the page counter resets to the PAGE1 setting on the FORMAT subcommand every time the break value changes for the specified variable. PAGE cannot be specified for listing reports with no break levels.

(NAME). *Display the name of the break variable next to each value or value label of the break variable.* NAME requires enough space for the length of the variable name plus two additional characters (for a colon and a blank space) in addition to the space needed to display break values or value labels. NAME is ignored if the break column width is insufficient.

(NONAME). *Suppress the display of break variable names.* This is the default.

Example

```
SORT DIVISION BRANCH DEPT.
REPORT FORMAT=AUTOMATIC MARGINS (1,70) BRKSPACE(-1)

  /VARIABLES=SPACE(DUMMY) ' ' (4)
             SALES 'Annual' 'Sales' '1981' (15) (OFFSET(2))
```

```
            EXPENSES 'Annual' 'Expenses' '1981' (15) (OFFSET(2))

 /BREAK=DIVISION
       BRANCH (10) (TOTAL) (OFFSET(1))
 /SUMMARY=MEAN

 /BREAK=DEPT 'Department' (10)
 /SUMMARY=MEAN.
```

- This example creates a report with three break variables. *BRANCH* breaks within values of *DIVISION*, and *DEPT* breaks within values of *BRANCH*.
- FORMAT sets margins to a maximum of 70 columns and requests that summary lines be displayed on the same line as break values. Because LIST is not specified on FORMAT, only summary statistics are displayed.
- VARIABLES defines three report columns, each occupied by a report variable: *SPACE*, *SALES*, and *EXPENSES*.
- The variable *SPACE* is a dummy variable that exists only within REPORT. It has a null heading and a width of 4. It is used as a space holder to separate the break columns from the report columns.
- *SALES* has a three-line heading and a width of 15. The values of *SALES* are offset two spaces from the right.
- *EXPENSES* is the third report variable and has the same width and offset specifications as *SALES*.
- The leftmost column in the report is reserved for the first two break variables, *DIVISION* and *BRANCH*. Value labels are displayed, since this is the default for AUTOMATIC. The break column has a width of 10 and the value labels are offset one space from the left. Value labels more than nine characters long are wrapped. The default column heading is used. TOTAL requests a summary line at the end of the report showing the mean of all cases in the report.
- The first SUMMARY subcommand displays the mean of each report variable in its report column. This line is displayed each time the value of *DIVISION* or *BRANCH* changes.
- The third break variable, *DEPT*, occupies the second column from the left in the report. The break column has a width of 10 and has a one-line heading. Value labels are displayed in the break column, and those exceeding 10 characters are wrapped.
- The second SUMMARY subcommand displays the mean for each report variable when the value of *DEPT* changes.

Using Dates as break variables

Under some circumstances, date values used as break variables may display as asterisks in the output because there is not enough space to display the date value. There are several options for dealing this situation:

- Decrease the offset value of the date variable on the BREAK subcommand. The default offset for dates is 4. Try using an offset of 2 or 0. For example: `DateVar (OFFSET(0))`.
- Include a width specification for the date variable on the BREAK subcommand. The value should be at greater than or equal to the defined width of the variable plus the offset value. For example, with a defined variable width of 10 and the default offset of 4, specify a width of 14, as in: `DateVar (14)`.
- Use the FORMATS command to increase the defined width of the date variable. For example, if the defined width is 10, increase it to 14, as in: `FORMATS DateVar (Date14)`. See the topic "FORMATS" on page 713 for more information.

SUMMARY subcommand

SUMMARY calculates a wide range of aggregate and composite statistics.

- SUMMARY must be specified if LIST is not specified on FORMAT.
- The minimum specification is an aggregate or a composite function and its arguments. This must be the first specification on SUMMARY.
- Each SUMMARY subcommand following a BREAK subcommand specifies a new summary line.

- The default location of the summary title is the column of the break variable to which the summary applies. When more than one function is named on SUMMARY, the default summary title is that of the function named first. Both the title and its default column location can be altered. See the topic “Summary titles ” on page 1687 for more information.
- The default format can be altered for any function. (See the topic “Summary print formats ” on page 1688 for more information.)
- SUMMARY subcommands apply only to the preceding BREAK subcommand. If there is no SUMMARY subcommand after a BREAK subcommand, no statistics are displayed for that break level.
- To use the summary specifications from a previous BREAK subcommand for the current BREAK subcommand, specify keyword PREVIOUS on SUMMARY. See the topic “Other summary keywords ” on page 1690 for more information.
- Summary statistics are displayed in report columns. With aggregate functions, you can compute summary statistics for all report variables or for a subset. See the topic “Aggregate functions ” on page 1685 for more information. With composite functions, you can compute summaries for all or a subset of report variables and you have additional control over the placement of summary statistics in particular report columns. See the topic “Composite functions ” on page 1686 for more information.
- Multiple summary statistics requested on one SUMMARY subcommand are all displayed on the same line. More than one function can be specified on SUMMARY as long as you do not attempt to place two results in the same report column (REPORT will not be executed if you do). To place results of more than one function in the same report column, use multiple SUMMARY subcommands.
- Any composite and aggregate functions except FREQUENCY and PERCENT can be specified on the same summary line.
- To insert blank lines between summaries when more than one summary line is requested for a break, use keyword SKIP followed by the number of lines to skip in parentheses. The default is 0. See the topic “Other summary keywords ” on page 1690 for more information.

Aggregate functions

Use the aggregate functions to request descriptive statistics for report variables.

- If no variable names are specified as arguments to an aggregate function, the statistic is calculated for all variables named on VARIABLES (all report variables).
- To request an aggregate function for a subset of report variables, specify the variables in parentheses after the function keyword.
- All variables specified for an aggregate function must have been named on VARIABLES.
- Keyword TO cannot be used to specify a list of variables for an aggregate function.
- The result of an aggregate function is always displayed in the report column reserved for the variable for which the function was calculated.
- To use several aggregate functions for the same report variable, specify multiple SUMMARY subcommands. The results are displayed on different summary lines.
- The aggregate functions FREQUENCY and PERCENT have special display formats and cannot be placed on the same summary line with other aggregate or composite functions. They can be specified only once per SUMMARY subcommand.
- Aggregate functions use only cases with valid values.

VALIDN. *Valid number of cases.* This is the only function available for string variables.

SUM. *Sum of values.*

MIN. *Minimum value.*

MAX. *Maximum value.*

MEAN. *Mean.*

STDDEV. *Standard deviation.* Aliases are SD and STDEV.

VARIANCE. *Variance.*

KURTOSIS. *Kurtosis.*

SKEWNESS. *Skewness.*

MEDIAN(min,max) . *Median value for values within the range.* MEDIAN sets up integer-valued bins for counting all values in the specified range. Noninteger values are truncated when the median is calculated.

MODE(min,max) . *Modal value for values within the range.* MODE sets up integer-valued bins for counting all values in the specified range. Noninteger values are truncated when the mode is calculated.

PGT(n). *Percentage of cases with values greater than* n. Alias PCGT.

PLT(n). *Percentage of cases with values less than* n. Alias PCLT.

PIN(min,max) . *Percentage of cases within the inclusive value range specified.* Alias PCIN.

FREQUENCY(min,max) . *Frequency counts for values within the inclusive range.* FREQUENCY sets up integer-valued bins for counting all values in the specified range. Noninteger values are truncated when the frequency is computed. FREQUENCY cannot be mixed with other aggregate statistics on a summary line.

PERCENT(min,max) . *Percentages for values within the inclusive range.* PERCENT sets up integer-valued bins for counting all values in the specified range. Noninteger values are truncated when the percentages are computed. PERCENT cannot be mixed with other aggregate statistics on a summary line.

Example

```
SORT CASES BY BVAR AVAR.
REPORT FORMAT=AUTOMATIC LIST /VARIABLES=XVAR YVAR ZVAR

  /BREAK=BVAR
    /SUMMARY=SUM
    /SUMMARY=MEAN (XVAR YVAR ZVAR)
    /SUMMARY=VALIDN(XVAR)

  /BREAK=AVAR
    /SUMMARY=PREVIOUS.
```

- FORMAT requests a case listing, and VARIABLES establishes a report column for variables *XVAR*, *YVAR*, and *ZVAR*. The report columns have default widths and titles.
- Both break variables, *BVAR* and *AVAR*, have default widths and headings.
- Every time the value of *BVAR* changes, three summary lines are displayed. The first line contains the sums for variables *XVAR*, *YVAR*, and *ZVAR*. The second line contains the means of all three variables. The third line displays the number of valid cases for *XVAR* in the report column for *XVAR*.
- Every time the value of *AVAR* changes within each value of *BVAR*, the three summary lines requested for *BVAR* are displayed. These summary lines are based on cases with the current values of *BVAR* and *AVAR*.

Example

```
SORT CASES BY DEPT.
REPORT FORMAT=AUTOMATIC
  /VARIABLES=WAGE BONUS TENURE
  /BREAK=DEPT (23)
  /SUMMARY=SUM(WAGE BONUS) MEAN(TENURE) 'Sum Income: Mean Tenure'.
```

- SUMMARY defines a summary line consisting of the sums of *WAGE* and *BONUS* and the mean of TENURE. The result of each aggregate function is displayed in the report column of the variable for which the function is calculated.
- A title is assigned to the summary line. A width of 23 is defined for the break column to accommodate the title for the summary line.

Composite functions

Use composite functions to obtain statistics based on aggregated statistics, to place a summary statistic in a column other than that of the report variable for which it was calculated, or to manipulate variables not named on VARIABLES.

- Composite functions can be computed for the following aggregate functions: VALIDN, SUM, MIN, MAX, MEAN, STDEV, VARIANCE, KURTOSIS, SKEWNESS, PGT, PLT, and PIN. Constants can also be arguments to composite functions.
- When used within composite functions, aggregate functions can have only one variable as an argument.
- A composite function and its arguments cannot be separated by other SUMMARY specifications.
- The result of a composite function can be placed in any report column, including columns of dummy or string variables, by specifying a target column. To specify a target column, enclose the variable name of the column in parentheses after the composite function and its arguments. By default, the results of a composite function are placed in the report column of the first variable specified on the composite function that is also specified on VARIABLES.
- The format for the result of a composite function can be specified in parentheses after the name of the column location, within the parentheses that enclose the column-location specification.

DIVIDE(arg arg. *Divide the first argument by the second and then multiply the result by the factor if it is specified.*

MULTIPLY(arg. *Multiply the arguments.*

PCT(arg. *The percentage of the first argument over the second.*

SUBTRACT(arg. *Subtract the second argument from the first.*

ADD(arg. *Add the arguments.*

GREAT(arg. *The maximum of the arguments.*

LEAST(arg. *The minimum of the arguments.*

AVERAGE(arg. *The average of the arguments.*

Example

```
SORT CASES BY DEPT.
REPORT FORMAT=AUTOMATIC BRKSPACE(-1)
  /VARIABLES=WAGE BONUS SPACE1 (DUMMY) '' BNFT1 BNFT2 SPACE2 (DUMMY)''
  /BREAK=DEPT

  /SUMMARY=MEAN(WAGE BONUS BNFT1 BNFT2)
       ADD(VALIDN(WAGE)) (SPACE2)

  /SUMMARY=ADD(SUM(WAGE) SUM(BONUS))
        ADD(SUM(BNFT1) SUM(BNFT2)) 'Totals' SKIP(1)

  /SUMMARY=DIVIDE(MEAN(WAGE) MEAN(BONUS)) (SPACE1 (COMMA)(2))
        DIVIDE(MEAN(BNFT1) MEAN(BNFT2)) (SPACE2 (COMMA)(2)) 'Ratios'
        SKIP(1).
```

- VARIABLES defines six report columns. The columns for *WAGE, BONUS, BNFT1,* and *BNFT2* contain aggregate statistics based on those variables. The variables *SPACE1* and *SPACE2* are dummy variables that are created for use as space holders; each is given a blank heading to suppress the default column heading.
- The first SUMMARY computes the means of the variables *WAGE, BONUS, BNFT1,* and *BNFT2*. Because BRKSPACE=−1, this summary line will be placed on the same line as the break value and will have no summary title. The means are displayed in the report column for each variable. SUMMARY also computes the valid number of cases for *WAGE* and places the result in the *SPACE2* column.
- The second SUMMARY adds the sum of *WAGE* to the sum of *BONUS*. Since no location is specified, the result is displayed in the *WAGE* column. In addition, the sum of *BNFT1* is added to the sum of *BNFT2* and the result is placed in the *BNFT1* column. The title for the summary line is *Totals*. One line is skipped before the summary line requested by this SUMMARY subcommand is displayed.
- The third summary line divides the mean of *WAGE* by the mean of *BONUS* and places the result in *SPACE1*. The ratio of the mean of *BNFT1* to the mean of *BNFT2* is displayed in the *SPACE2* column. The results are displayed with commas and two decimal places. The title for the summary line is *Ratios*. One line is skipped before the summary line requested by this SUMMARY subcommand is displayed.

Summary titles

- You can specify a summary title enclosed in quotes, following the conventions for strings.

- The summary title must be specified after the first function and its arguments. It cannot separate any function from its arguments.
- A summary title can be only one line long.
- A summary title wider than the break column extends into the next break column to the right. If the title is wider than all of the available break columns, it is truncated.
- Only one summary title can be specified per summary line. If more than one is specified, the last is used.
- The summary title is left- or right-justified depending upon whether the break title is left- or right-justified.
- The default location for the summary title is the column of the BREAK variable to which the summary applies. With multiple breaks, you can override the default placement of the title by specifying, in parentheses following the title, the number of the break column in which you want the summary title to be displayed.
- In a report with no break levels, REPORT displays the summary title above the summary line at the left margin.

Table 74. Default title for summary lines

Function	Title
VALIDN	N
VARIANCE	Variance
SUM	Sum
MEAN	Mean
STDDEV	StdDev
MIN	Minimum
MAX	Maximum
SKEWNESS	Skewness
KURTOSIS	Kurtosis
PGT(n)	>n
PLT(n)	<n
PIN(min,max)	In n_1 to n_2
FREQUENCY(min,max)	Total
PERCENT(min,max)	Total
MEDIAN(min,max)	Median
MODE(min,max)	Mode

Summary print formats

All functions have default formats that are used to display results. You can override these defaults by specifying a format keyword and/or the number of decimal places.

- Any printable formats or the PLAIN keyword can be specified. Format specifications must be enclosed in parentheses.
- For aggregate functions, the format and/or number of decimal places is specified after the variable name, within the parentheses that enclose the variable name. The variable must be explicitly named as an argument.

- For composite functions, the format and/or number of decimal places is specified after the variable name of the column location, within the parentheses that enclose the variable name. The column location must be explicitly specified.
- If the report column is wide enough, SUM, MEAN, STDDEV, MIN, MAX, MEDIAN, MODE, and VARIANCE use DOLLAR or COMMA format if a DOLLAR or COMMA format has been declared for the variable on either the FORMATS or PRINT FORMATS command.
- If the column is not wide enough to display the decimal digits for a given function, REPORT displays fewer decimal places. If the column is not wide enough to display the integer portion of the number, REPORT uses scientific notation if possible, or, if not, displays asterisks.
- An exact value of 0 is displayed with one 0 to the left of the decimal point and as many 0 digits to the right as specified by the format. A number less than 1 in absolute value is displayed without a 0 to the left of the decimal point, except with DOLLAR and COMMA formats.

(PLAIN). *Uses the setting on SET DECIMAL for the thousands separator and decimal delimiter.* PLAIN overrides dictionary formats. This is the default for all functions except SUM, MEAN, STDDEV, MIN, MAX, MEDIAN, MODE, and VARIANCE. For these functions, the default is the dictionary format of the variable for which the function is computed.

(d). *Number of decimal places.*

Example

```
/SUMMARY=MEAN(INCOME (DOLLAR)(2))
         ADD(SUM(INCOME)SUM(WEALTH)) (WEALTH(DOLLAR(2))
```

- SUMMARY displays the mean of *INCOME* with dollar format and two decimal places. The result is displayed in the *INCOME* column.
- The sums of *INCOME* and *WEALTH* are added, and the result is displayed in the *WEALTH* column with dollar format and two decimal places.

Table 75. Default print formats for functions

Function	Format type	Width	Decimal places
VALIDN	F	5	0
SUM	Dictionary	Dictionary + 2	Dictionary
MEAN	Dictionary	Dictionary	Dictionary
STDDEV	Dictionary	Dictionary	Dictionary
VARIANCE	Dictionary	Dictionary	Dictionary
MIN	Dictionary	Dictionary	Dictionary
MAX	Dictionary	Dictionary	Dictionary
SKEWNESS	F	5	2
KURTOSIS	F	5	2
PGT	PCT	6	1
PLT	PCT	6	1
PIN	PCT	6	1
MEDIAN	Dictionary	Dictionary	Dictionary
MODE	Dictionary	Dictionary	Dictionary
PERCENT	F	6	1
FREQUENCY	F	5	0
DIVIDE	F	Dictionary	0

Table 75. Default print formats for functions (continued)

Function	Format type	Width	Decimal places
PCT	PCT	6	2
SUBTRACT	F	Dictionary	0
ADD	F	Dictionary	0
GREAT	F	Dictionary	0
LEAST	F	Dictionary	0
AVERAGE	F	Dictionary	0
MULTIPLY	F	Dictionary	0

Where DATE formats are specified, functions with the dictionary format type display the DATE formats, using the column width as the display width.

Other summary keywords

The following additional keywords can be specified on SUMMARY. These keywords are not enclosed in parentheses.

SKIP(n). *Blank lines before the summary line.* The default is 0. If SKIP is specified for the first SUMMARY subcommand for a BREAK, it skips the specified lines after skipping the number of lines specified for BRKSPACE on FORMAT. Similarly, with case listings SKIP skips *n* lines after the blank line at the end of the listing.

PREVIOUS(n) . *Use the SUMMARY subcommands specified for the* n*th BREAK.* If *n* is not specified, PREVIOUS refers to the set of SUMMARY subcommands for the previous BREAK. If an integer is specified, the SUMMARY subcommands from the *n*th BREAK are used. If PREVIOUS is specified, no other specification can be used on that SUMMARY subcommand.

TITLE and FOOTNOTE subcommands

TITLE and FOOTNOTE provide titles and footnotes for the report.

- TITLE and FOOTNOTE are optional and can be placed anywhere after FORMAT except between the BREAK and SUMMARY subcommands.
- The specification on TITLE or FOOTNOTE is the title or footnote in quotes. To specify a multiple-line title or footnote, enclose each line in quotes and separate the specifications for each line by at least one blank.
- The default REPORT title is the title specified on the TITLE command. If there is no TITLE command specified in your session, the default REPORT title is the first line of the header.
- Titles begin on the first line of the report page. Footnotes end on the last line of the report page.
- Titles and footnotes are repeated on each page of a multiple-page report.
- The positional keywords LEFT, CENTER, and RIGHT can each be specified once. The default is CENTER.
- If the total width needed for the combined titles or footnotes for a line exceeds the page width, REPORT generates an error message.

LEFT. *Left-justify titles or footnotes within the report page margins.*

RIGHT. *Right-justify titles or footnotes within the report page margins.*

CENTER. *Center titles and footnotes within the report page width.*

The following can be specified as part of the title or footnote.

)PAGE. *Display the page number right-justified in a five-character field.*

)DATE. *Display the current date in the form* dd/mmm/yy, *right-justified in a nine-character field.*

)var. *Display this variable's value label at this position.* If you specify a variable that has no value label, the value is displayed, formatted according to its print format. You cannot specify a scratch or system variable or a variable created with the STRING subcommand. If you want to use a variable named *DATE* or *PAGE* in the file, change the variable's name with the RENAME VARIABLES command before you use it on the TITLE or FOOTNOTE subcommands, to avoid confusion with the)PAGE and)DATE keywords.

-)PAGE,)DATE, and)var are specified within quotes and can be mixed with string segments within the quotes.
- A variable specified on TITLE or FOOTNOTE must be defined in the active dataset, but does not need to be included as a column on the report.
- One label or value from each variable specified on TITLE or FOOTNOTE is displayed on every page of the report. If a new page starts with a case listing, REPORT takes the value label from the first case listed. If a new page starts with a BREAK line, REPORT takes the value label from the first case of the new break group. If a new page starts with a summary line, REPORT takes the value label from the last case of the break group being summarized.

Example

```
/TITLE=LEFT 'Personnel Report' 'Prepared on )DATE'
    RIGHT 'Page: )PAGE'
```

- TITLE specifies two lines for a left-justified title and one line for a right-justified title. These titles are displayed at the top of each page of the report.
- The second line of the left-justified title contains the date on which the report was processed.
- The right-justified title displays the page number following the string *Page:* on the same line as the first line of the left-justified title.

MISSING subcommand

MISSING controls the treatment of cases with missing values.

- MISSING specifications apply to variables named on VARIABLES and SUMMARY and to strings created with the STRING subcommand.
- Missing-value specifications are ignored for variables named on BREAK when MISSING=VAR or NONE. There is one break category for system-missing values and one for each user-missing value.
- The character used to indicate missing values is controlled by the FORMAT subcommand.

VAR. *Missing values are treated separately for each variable.* Missing values are displayed in case listings but are not included in the calculation of summary statistics on a function-by-function basis. This is the default.

NONE. *User-missing values are treated as valid values.* This applies to all variables named on VARIABLES.

LIST[([varlist][n])] . *Cases with the specified number of missing values across the specified list of variables are not used.* The variable list and *n* are specified in parentheses. If *n* is not specified, the default is 1. If no variables are specified, all variables named on VARIABLES are assumed.

Example

```
/MISSING= LIST (XVAR,YVAR,ZVAR 2)
```

- Any case with two or more missing values across the variables *XVAR, YVAR,* and *ZVAR* is omitted from the report.

REPOSITORY ATTRIBUTES

```
REPOSITORY ATTRIBUTES FILE='file specification'
    [DESCRIPTION='description']
    [KEYWORDS='keywords']
    [AUTHOR='author']
    [VERSIONLABEL='label']
    [EXPIRATION=days]
    [TOPICS='topics']
    [/SECURITY ID='id' [PERMISSION= [READ**] [WRITE] [DELETE] [MODIFY] [OWNER] ] ]
```

** Default if the keyword is omitted.

Release History

Release 16.0

- Command introduced as PER ATTRIBUTES.

Release 17.0

- VERSIONLABEL keyword extended to support multiple labels.

Release 18.0

- PER ATTRIBUTES deprecated. Command name changed to REPOSITORY ATTRIBUTES.

Example

```
REPOSITORY COPY
  FILE='/myscripts/cust_value.py'
  OUTFILE='SPSSCR://scripts/cust_value.py'.
REPOSITORY ATTRIBUTES FILE='SPSSCR://scripts/cust_value.py'
  DESCRIPTION='Customer Value Calculation'
  KEYWORDS='customer;value'.
```

Overview

The REPOSITORY ATTRIBUTES command allows you to set attributes--such as a version label and security settings--for an object in a IBM SPSS Collaboration and Deployment Services Repository.

- When copying to a IBM SPSS Collaboration and Deployment Services Repository with the REPOSITORY COPY command, use the REPOSITORY ATTRIBUTES command (following REPOSITORY COPY) to specify attributes of the object.

Basic Specification

The basic specification is the FILE keyword, which specifies the IBM SPSS Collaboration and Deployment Services Repository object whose attributes are to be set. All other keywords and subcommands are optional.

Syntax Rules

- The SECURITY subcommand can be specified multiple times.
- Each keyword can only be specified once.
- Keywords and subcommands can be used in any order.
- Keywords and subcommand names must be spelled in full.
- Equals signs (=) shown in the syntax chart are required.

Operations

- Use of the REPOSITORY ATTRIBUTES command requires a connection to a IBM SPSS Collaboration and Deployment Services Repository. Connections are established with the REPOSITORY CONNECT command.
- REPOSITORY ATTRIBUTES overwrites any existing values of specified attributes.

FILE Keyword

The FILE keyword is required and specifies the IBM SPSS Collaboration and Deployment Services Repository object whose attributes are to be set.

- The form of the file specification for an object in a IBM SPSS Collaboration and Deployment Services Repository is the scheme name SPSSCR , followed by a colon, either one or two slashes (forward or backward), and a file path, all enclosed in quotes. For example:

  ```
  'SPSSCR://scripts/myscript.py'
  ```

- Paths can be specified with forward slashes (/) or backslashes (\).
- You can define a file handle to a file or a directory in a IBM SPSS Collaboration and Deployment Services Repository and use that handle in file specifications for IBM SPSS Collaboration and Deployment Services Repository objects.
- You can use the CD command to set the working directory to a directory in the currently connected IBM SPSS Collaboration and Deployment Services Repository, allowing you to use relative paths in file specifications for IBM SPSS Collaboration and Deployment Services Repository objects.
- File specifications for IBM SPSS Collaboration and Deployment Services Repository objects must specify the filename exactly as provided when the file was stored. If the file was stored with an extension, then you must supply the extension. If the file was stored without an extension then do not include one.

For examples of file specifications, see the examples for the REPOSITORY COPY command .

DESCRIPTION Keyword

The DESCRIPTION keyword specifies a description for an object in a IBM SPSS Collaboration and Deployment Services Repository and replaces any existing description for the object. Specify the value as a quoted string.

Example

```
REPOSITORY ATTRIBUTES FILE='SPSSCR://scripts/cust_value.py'
  DESCRIPTION='Customer Value Calculation'.
```

KEYWORDS Keyword

KEYWORDS specifies one or more keywords to associate with an object in a IBM SPSS Collaboration and Deployment Services Repository to aid in searching. Specify the value as a quoted string.

- Multiple keywords should be separated by semicolons.
- Blank spaces at the beginning and end of each keyword are ignored, but blank spaces within keywords are honored.
- The specified keywords replace any existing ones for the object.

Example

```
REPOSITORY ATTRIBUTES FILE='SPSSCR://scripts/cust_value.py'
  KEYWORDS='customer;value'.
```

AUTHOR Keyword

The AUTHOR keyword specifies the author of an object in a IBM SPSS Collaboration and Deployment Services Repository. By default, the author is set to the login name of the user who created the object. Specify the value as a quoted string.

Example

```
REPOSITORY ATTRIBUTES FILE='SPSSCR://scripts/cust_value.py'
  AUTHOR='GSWEET'.
```

VERSIONLABEL Keyword

The VERSIONLABEL keyword specifies one or more version labels for an object in a IBM SPSS Collaboration and Deployment Services Repository--for example, "production" or "development". Two versions of an object cannot have any of the same labels. If you specify a label associated with another version, it will be removed from that version and associated with the one you're modifying. For a single label, specify the value as a quoted string. For multiple labels, specify the value as a blank-separated list of quoted strings--for example, `'label1' 'label2'`.

- By default, the specified labels will be applied to the latest version of the object that is accessible to the current user. You can apply labels to a version other than the latest one by specifying the version in the file specification on the FILE keyword. See the topic "File Specifications for IBM SPSS Collaboration and Deployment Services Repository Objects" on page 2217 for more information.

Example

```
REPOSITORY ATTRIBUTES FILE='SPSSCR://scripts/cust_value.py'
  VERSIONLABEL='development'.
```

EXPIRATION Keyword

The EXPIRATION keyword specifies an expiration date for an object in a IBM SPSS Collaboration and Deployment Services Repository. This provides a mechanism to make sure dated information is not displayed after a certain date. Expired documents are not deleted but are automatically removed to a special category where they can be accessed only by the site administrator, who can delete, archive, or republish them.

- Specify the value as an integer representing the number of days from the current day (inclusive) to the last day (inclusive) that the document will be active.

Example

```
REPOSITORY ATTRIBUTES FILE='SPSSCR://scripts/cust_value.py'
  EXPIRATION = 366.
```

TOPICS Keyword

The TOPICS keyword allows you to associate an object in a IBM SPSS Collaboration and Deployment Services Repository with one or more topics. Topics allow you to organize documents by subject matter and have a hierarchical structure.

- Topics are specified as a quoted path that includes each level of the hierarchy for that topic, with successive levels separated by a forward slash. A forward slash at the beginning of the path is optional.
- Use a semicolon as the delimiter when specifying multiple topics.
- Objects can only be associated with existing topics. REPOSITORY ATTRIBUTES cannot be used to create new topics.
- The specified topics replace any existing ones for the object.

Example

```
REPOSITORY ATTRIBUTES FILE='SPSSCR://scripts/cust_value.py'
  TOPICS = '/engineering/scripts;/marketing/analyses'.
```

SECURITY Subcommand

The SECURITY subcommand allows you to specify security settings for an object in a IBM SPSS Collaboration and Deployment Services Repository. Specify the identifier of the user or group with the ID keyword, and specify the access level with the PERMISSION keyword. You can specify settings for multiple users or groups by including multiple instances of the SECURITY subcommand.

ID Keyword

The ID keyword specifies the ID of the user or group for which access is being granted. Specify the value in quotes.

PERMISSION Keyword

The PERMISSION keyword is optional and specifies the access level granted to the specified user or group. One or more of the following access levels can be specified: READ, WRITE, MODIFY (grants ability to modify permissions), DELETE, and OWNER.

- If PERMISSION is omitted, the access level is READ.
- READ access is always granted, whether or not it is specified on the PERMISSION keyword.
- If OWNER is specified, all other values are ignored (an owner has all permissions).

Example

```
REPOSITORY ATTRIBUTES FILE='SPSSCR://scripts/cust_value.py'
  /SECURITY ID='admin' PERMISSION=OWNER
  /SECURITY ID='--everyone--' PERMISSION=READ WRITE.
```

REPOSITORY CONNECT

```
REPOSITORY CONNECT
    /SERVER HOST='host[:{8080**}]' [SSL={NO**}]
                       {port  }        {YES }
    /LOGIN  USER='userid'
            PASSWORD='password'
            [PROVIDER='security provider']
            [ENCRYPTEDPWD={YES**}]
                          {NO   }
```

** Default if the keyword or value is omitted.

Release History

Release 15.0

- Command introduced as PER CONNECT.

Release 17.0

- DOMAIN keyword deprecated on the LOGIN subcommand.
- PROVIDER keyword introduced on the LOGIN subcommand.

Release 18.0

- PER CONNECT deprecated. Command name changed to REPOSITORY CONNECT.

Example

```
REPOSITORY CONNECT
    /SERVER HOST='PER1'
    /LOGIN  USER='MyUserID'
            PASSWORD='abc12345'
            ENCRYPTEDPWD=NO.
```

Overview

The REPOSITORY CONNECT command establishes a connection to a IBM SPSS Collaboration and Deployment Services Repository and logs in the user. A connection enables you to store objects to, and retrieve objects from, a IBM SPSS Collaboration and Deployment Services Repository.

Options

Server. You can specify a connection port and whether to connect to the specified server using Secure Socket Layer (SSL) technology, if it is enabled on the server.

Login. You can specify whether the password is provided as encrypted or unencrypted (plain text).

Single Sign-On. Single sign-on allows users to connect to a IBM SPSS Collaboration and Deployment Services Repository without explicitly providing a user ID and password. The necessary authentication is performed with the user's existing credentials on the current machine--obtained, for example, from Windows Active Directory. If your IBM SPSS Collaboration and Deployment Services Repository supports single sign-on (contact your local system administrator for details), you can omit the LOGIN subcommand.

Basic Specification

The basic specification for REPOSITORY CONNECT is the host server, user name, and password. By default, server port 8080 is used, the connection is established without SSL, and the specified password is assumed to be encrypted. To create an encrypted password, generate (paste) the REPOSITORY CONNECT command syntax from the IBM SPSS Collaboration and Deployment Services Repository Connect dialog box.

Syntax Rules

- Each subcommand can be specified only once.

- Subcommands can be used in any order.
- An error occurs if a keyword or attribute is specified more than once within a subcommand.
- Equals signs (=) and forward slashes (/) shown in the syntax chart are required.
- Subcommand names and keywords must be spelled in full.

Operations

- REPOSITORY CONNECT establishes a connection to an IBM SPSS Collaboration and Deployment Services Repository and logs in the specified user. Any existing connection terminates when the new one is established.
- You can specify whether a connection is established when the REPOSITORY CONNECT command is encountered or whether the connection is deferred until it is required. The behavior is determined by the REPDEFER subcommand of the SET command.
- The connection terminates if the IBM SPSS Statistics session ends.
- An error occurs if a connection cannot be established to the specified host server.
- An error occurs if the connection cannot be authenticated--for example, if the password is invalid for the specified user.

Example

```
REPOSITORY CONNECT
    /SERVER HOST='PER1:80'
    /LOGIN  USER='MyUserID'
            PASSWORD='abc12345'
            ENCRYPTEDPWD=NO.
```

- The SERVER subcommand specifies a connection to host 'PER1' on port 80.
- ENCRYPTEDPWD=NO indicates that the password is not encrypted.

SERVER Subcommand

The SERVER subcommand specifies the host server and whether to establish a secure connection.

HOST. *Server that hosts the IBM SPSS Collaboration and Deployment Services Repository.* Specify the name of the server in quotes. The default port is 8080. To connect to another port, specify the port number after the host name; for example, 'PER1:80'. A colon must separate the host name and port.

SSL. *Use Secure Socket Layer technology.* Specifies whether to establish a secure connection to the host server. The default is NO. SSL is available only if supported on the host server.

LOGIN Subcommand

The LOGIN subcommand specifies login information, including user name and password.

USER. *User name.* Specify the user name in quotes.

PASSWORD. *Password.* Specify the password in quotes.

PROVIDER. *Security provider.* You can optionally specify the security provider against which to validate the user/password combination. Contact your local administrator for details.

ENCRYPTEDPWD. *Password encryption.* By default, the specified password is treated as encrypted. To indicate that the password is entered as plain text, specify ENCRYPTEDPWD=NO.

REPOSITORY COPY

```
REPOSITORY COPY
    FILE='file specification'
    OUTFILE='file specification'.
```

Release History

Release 16.0

- Command introduced as PER COPY.

Release 18.0

- PER COPY deprecated. Command name changed to REPOSITORY COPY.

Example

```
REPOSITORY COPY
  FILE='/myscripts/demo.py'
  OUTFILE='SPSSCR://scripts/demo.py'.
```

Overview

The REPOSITORY COPY command allows you to copy an arbitrary file from the local file system to a IBM SPSS Collaboration and Deployment Services Repository or to copy a file from a IBM SPSS Collaboration and Deployment Services Repository to the local file system.

- When copying to a IBM SPSS Collaboration and Deployment Services Repository, use the REPOSITORY ATTRIBUTES command--following REPOSITORY COPY--to specify properties such as a description, keywords, and security settings for the object.

Basic Specification

The only specification is the FILE keyword, which specifies the source file, and the OUTFILE keyword, which specifies the target location. Each keyword can specify a location in the local file system or a location in the current IBM SPSS Collaboration and Deployment Services Repository.

File Specifications for IBM SPSS Collaboration and Deployment Services Repository Objects

- The form of the file specification for an object in a IBM SPSS Collaboration and Deployment Services Repository is the scheme name SPSSCR , followed by a colon, either one or two slashes (forward or backward), and a file path, all enclosed in quotes. For example:

  ```
  'SPSSCR://scripts/myscript.py'
  ```

- Paths can be specified with forward slashes (/) or backslashes (\).
- You can define a file handle to a file or a directory in a IBM SPSS Collaboration and Deployment Services Repository and use that handle in file specifications for IBM SPSS Collaboration and Deployment Services Repository objects.
- You can use the CD command to set the working directory to a directory in the currently connected IBM SPSS Collaboration and Deployment Services Repository, allowing you to use relative paths in file specifications for IBM SPSS Collaboration and Deployment Services Repository objects.
- File extensions are not added to files stored to a IBM SPSS Collaboration and Deployment Services Repository. Files are stored to a IBM SPSS Collaboration and Deployment Services Repository with an automatically determined MIME type that describes the type of file. Including a file extension is not necessary but is recommended.
- When copying a file from a IBM SPSS Collaboration and Deployment Services Repository you must specify the filename exactly as provided when the file was stored. If the file was stored with an

extension, then you must supply the extension. If the file was stored without an extension then do not include one.

- When copying from a IBM SPSS Collaboration and Deployment Services Repository, the latest version of the specified object that is accessible to the current user is retrieved. To specify a version by label, use SPSSCR://<path>#L.<label>. To specify a version by the version marker (consists of an integer version number followed by a time stamp that uniquely specifies a version), use SPSSCR://<path>#M.<version marker>. See the topic "File Specifications for IBM SPSS Collaboration and Deployment Services Repository Objects" on page 2217 for more information.

Operations

- REPOSITORY COPY will fail if a IBM SPSS Collaboration and Deployment Services Repository location is specified (for either FILE or OUTFILE) and there is no connection to a IBM SPSS Collaboration and Deployment Services Repository. Connections are established with the REPOSITORY CONNECT command.
- Specifying an existing IBM SPSS Collaboration and Deployment Services Repository file on the OUTFILE keyword results in a new version of the file, as opposed to overwriting the latest version.
- The PERMISSIONS, SAVE, and XSAVE commands will generate errors if you attempt to set permissions for IBM SPSS Collaboration and Deployment Services Repository objects. Permissions for IBM SPSS Collaboration and Deployment Services Repository objects are set with the REPOSITORY ATTRIBUTES command or using administration tools included with IBM SPSS Collaboration and Deployment Services.
- The ERASE FILE command will generate an error if you attempt to delete a IBM SPSS Collaboration and Deployment Services Repository object.

Examples

Copying from the Local File System to a IBM SPSS Collaboration and Deployment Services Repository

```
REPOSITORY COPY
  FILE='/myscripts/cust_value.py'
  OUTFILE='SPSSCR://scripts/cust_value.py'.
REPOSITORY ATTRIBUTES FILE='SPSSCR://scripts/cust_value.py'
  KEYWORDS='customer;value'
  /SECURITY ID='rprime' PERMISSION=OWNER.
```

- The local file */myscripts/cust_value.py* is copied to the location */scripts/cust_value.py* in the current IBM SPSS Collaboration and Deployment Services Repository.
- The REPOSITORY ATTRIBUTES command specifies keywords and security settings for this IBM SPSS Collaboration and Deployment Services Repository object.

Copying from a IBM SPSS Collaboration and Deployment Services Repository to the Local File System

```
REPOSITORY COPY
  FILE='SPSSCR://scripts/demo.py'
  OUTFILE='/myscripts/demo.py'.
```

- The IBM SPSS Collaboration and Deployment Services Repository object */scripts/demo.py* is copied to the local file */myscripts/demo.py*.

Copying from a Labelled Version in a IBM SPSS Collaboration and Deployment Services Repository to the Local File System

```
REPOSITORY COPY
  FILE='SPSSCR://scripts/demo.py#L.production'
  OUTFILE='/myscripts/demo.py'.
```

- The version of the IBM SPSS Collaboration and Deployment Services Repository object */scripts/demo.py* with the label *production* is copied to the local file */myscripts/demo.py*.

Using a File Handle to a IBM SPSS Collaboration and Deployment Services Repository Location

```
FILE HANDLE custval /NAME='SPSSCR://CustomerValue'.
REPOSITORY COPY
```

```
  FILE='custval/cust_value.py'
  OUTFILE='/myscripts/cust_value.py'.
```

- The handle *custval* is associated with the IBM SPSS Collaboration and Deployment Services Repository directory */CustomerValue*, and the REPOSITORY COPY command retrieves the latest version of the file *cust_value.py* from this directory in the current IBM SPSS Collaboration and Deployment Services Repository.

Setting the Working Directory to a IBM SPSS Collaboration and Deployment Services Repository Location

```
CD 'SPSSCR://CustomerValue'.
REPOSITORY COPY
  FILE='cust_value.py'
  OUTFILE='/myscripts/cust_value.py'.
```

- The working directory is set to the IBM SPSS Collaboration and Deployment Services Repository directory */CustomerValue*, and the REPOSITORY COPY command retrieves the latest version of the file *cust_value.py* from this directory in the current IBM SPSS Collaboration and Deployment Services Repository.

REREAD

```
REREAD [FILE=file]
       [COLUMN=expression]
```

Example

```
INPUT PROGRAM.
DATA LIST /KIND 10-14 (A).

DO IF (KIND EQ 'FORD').
REREAD.
DATA LIST /PARTNO 1-2 PRICE 3-6 (DOLLAR,2) QUANTITY 7-9.
END CASE.

ELSE IF (KIND EQ 'CHEVY').
REREAD.
DATA LIST /PARTNO 1-2 PRICE 15-18 (DOLLAR,2) QUANTITY 19-21.
END CASE.
END IF.

END INPUT PROGRAM.

BEGIN DATA
111295100FORD
11       CHEVY 295015
END DATA.
```

Overview

REREAD instructs the program to reread a record in the data. It is available only within an INPUT PROGRAM structure and is generally used to define data using information obtained from a previous reading of the record. REREAD is usually specified within a conditional structure, such as DO IF–END IF, and is followed by a DATA LIST command. When it receives control for a case, REREAD places the pointer back to the column specified for the current case and begins reading data as defined by the DATA LIST command that follows.

Options

Data Source. You can use inline data or data from an external file specified on the FILE subcommand. Using external files allows you to open multiple files and merge data.

Beginning Column. You can specify a beginning column other than column 1 using the COLUMN subcommand.

Basic Specification

The basic specification is the command keyword REREAD. The program rereads the current case according to the data definitions specified on the following DATA LIST.

Subcommand Order

Subcommands can be specified in any order.

Syntax Rules

- REREAD is available only within an INPUT PROGRAM structure.
- Multiple REREAD commands can be used within the input program. Each must be followed by an associated DATA LIST command.

Operations

- REREAD causes the next DATA LIST command to reread the most recently processed record in the specified file.
- When it receives control for a case, REREAD places the pointer back to column 1 for the current case and begins reading data as defined by the DATA LIST that follows. If the COLUMN subcommand is specified, the pointer begins reading in the specified column and uses it as column 1 for data definition.

- REREAD can be used to read part of a record in FIXED format and the remainder in LIST format. Mixing FIXED and FREE formats yields unpredictable results.
- Multiple REREAD commands specified without an intervening DATA LIST do not have a cumulative effect. All but the last are ignored.

Examples

Using REREAD to Process Different Record Formats

```
INPUT PROGRAM.
DATA LIST /PARTNO 1-2 KIND 10-14 (A).

DO IF (KIND EQ 'FORD').
REREAD.
DATA LIST /PRICE 3-6 (DOLLAR,2) QUANTITY 7-9.
END CASE.

ELSE IF (KIND EQ 'CHEVY').
REREAD.
DATA LIST /PRICE 15-18 (DOLLAR,2) QUANTITY 19-21.
END CASE.
END IF.
END INPUT PROGRAM.

BEGIN DATA
111295100FORD      CHAPMAN AUTO SALES
121199005VW    MIDWEST VOLKSWAGEN SALES
11 395025FORD      BETTER USED CARS
11       CHEVY 195005        HUFFMAN SALES & SERVICE
11       VW    595020        MIDWEST VOLKSWAGEN SALES
11       CHEVY 295015        SAM'S AUTO REPAIR
11       CHEVY 210 20        LONGFELLOW CHEVROLET
 9555032 VW                  HYDE PARK IMPORTS
END DATA.
LIST.
```

- Data are extracted from an inventory of automobile parts. The automobile part number always appears in columns 1 and 2, and the automobile type always appears in columns 10 through 14. The location of other information such as price and quantity depends on both the part number and the type of automobile.
- The first DATA LIST extracts the part number and type of automobile.
- Depending on the information from the first DATA LIST, the records are reread using one of two DATA LIST commands, pulling the price and quantity from different places.
- The two END CASE commands limit the active dataset to only those cases with part 11 and automobile type Ford or Chevrolet. Without the END CASE commands, cases would be created for other part numbers and automobile types, with missing values for price, quantity, and buyer.

The output from the LIST command is as follows:

```
PARTNO KIND   PRICE QUANTITY

  11   FORD  $12.95    100
  11   FORD   $3.95     25
  11   CHEVY  $1.95      5
  11   CHEVY  $2.95     15
  11   CHEVY  $2.10     20
```

Multiple REREAD Commands for the Same Record

```
INPUT PROGRAM.
DATA LIST      NOTABLE/ CDIMAGE 1-20(A).
REREAD         COLUMN = 6.  /* A, C, and E are in column 6
REREAD         COLUMN = 11. /* B, D, and F are in column 11
DATA LIST      NOTABLE/ INFO 1(A).
END INPUT PROGRAM.
BEGIN DATA
1    A    B
2    C    D
3    E    F
END DATA.
LIST.
```

- Multiple REREAD commands are used without an intervening DATA LIST. Only the last one is used. Thus, the starting column comes from the last REREAD specified and the pointer is reset to column 11.

The output from the LIST command is as follows:

```
CDIMAGE             INFO

1    A    B         B
2    C    D         D
3    E    F         F
```

FILE Subcommand

FILE specifies an external raw data file from which the next DATA LIST command reads data.

- The default file is the file specified on the immediately preceding DATA LIST command.
- If the file specified on FILE is not the default file, the same file must be specified on the next DATA LIST. Otherwise, the FILE subcommand is ignored and the DATA LIST command reads the next record from the file specified on it or, if no file is specified, from the file specified on the previous DATA LIST command.

Example

```
INPUT PROGRAM.
DATA LIST FILE=UPDATE END=#EOF NOTABLE
     /#ID 1-3.                    /*Get rep ID in new sales file.
DATA LIST FILE = SALESREP NOTABLE
     /ID 1-3 SALES 4-11(F,2)
      NEWSALE 12-19(F,2).       /*Get rep record from primary file.

LOOP IF #EOF OR (#ID GT ID).  /*If UPDATE ends or no new sales made.
+  COMPUTE NEWSALE = 0.      /*Set NEWSALE to 0
+  END CASE.                 /*Build a case.
+  DATA LIST FILE = SALESREP NOTABLE
     /ID 1-3 SALES 4-11(F,2)
      NEWSALE 12-19(F,2).           /*Continue reading primary file.
END LOOP

DO IF NOT #EOF.                      /*If new sales made.
+  REREAD FILE=UPDATE COLUMN = 4.     /*Read new sales from UPDATE.
+  DATA LIST FILE=UPDATE
     /NEWSALE 1-8(F,2).
+  COMPUTE SALES=SALES+NEWSALE.      /*Update primary file.
END IF.
END CASE.                            /*Build a case.
END INPUT PROGRAM.

LIST.
```

- This example uses REREAD to merge two raw data files (*SALESREP* and *UPDATE*).
- Both files are sorted by sales representative ID number. The *UPDATE* file contains only records for sales representatives who have made new sales, with variables *ID* and *NEWSALE*. The primary file *SALESREP* contains records for all sales representatives, with variables *SALES* (which contains year-to-date sales) and *NEWSALE* (which contains the update values each time the file is updated).
- If a sales representative has made no new sales, there is no matching ID in the *UPDATE* file. When *UPDATE* is exhausted or when the ID's in the two files do not match, the loop structure causes the program to build a case with *NEWSALE* equal to 0 and then continue reading the primary file.
- When the ID's match (and the *UPDATE* file is not yet exhausted), the REREAD command is executed. The following DATA LIST rereads the record in *UPDATE* that matches the *ID* variable. *NEWSALE* is read from the *UPDATE* file starting from column 4 and *SALES* is updated. Note that the following DATA LIST specifies the same file.
- When the updated base is built, the program returns to the first DATA LIST command in the input program and reads the next ID from the *UPDATE* file. If the *UPDATE* file is exhausted (*#EOF*=1), the loop keeps reading records from the primary file until it reaches the end of the file.
- The same task can be accomplished using MATCH FILES. With MATCH FILES, the raw data must be read and saved as IBM SPSS Statistics data files first.

COLUMN Subcommand

COLUMN specifies a beginning column for the REREAD command to read data. The default is column 1. You can specify a numeric expression for the column.

Specifying a Numeric Expression for the Column

```
INPUT PROGRAM.
DATA LIST /KIND 10-14 (A).
COMPUTE #COL=1.
IF (KIND EQ 'CHEVY') #COL=13.

DO IF (KIND EQ 'CHEVY' OR KIND EQ 'FORD').
REREAD COLUMN #COL.
DATA LIST /PRICE 3-6 (DOLLAR,2) QUANTITY 7-9.
END CASE.
END IF.
END INPUT PROGRAM.
BEGIN DATA
111295100FORD      CHAPMAN AUTO SALES
121199005VW    MIDWEST VOLKSWAGEN SALES
11 395025FORD      BETTER USED CARS
11       CHEVY 195005        HUFFMAN SALES & SERVICE
11       VW    595020        MIDWEST VOLKSWAGEN SALES
11       CHEVY 295015        SAM'S AUTO REPAIR
12       CHEVY 210 20        LONGFELLOW CHEVROLET
 9555032 VW                  HYDE PARK IMPORTS
END DATA.
LIST.
```

- The task in this example is to read *PRICE* and *QUANTITY* for Chevrolets and Fords only. A scratch variable is created to indicate the starting column positions for *PRICE* and *QUANTITY*, and a single DATA LIST command is used to read data for both types of automobiles.
- Scratch variable *#COL* is set to 13 for Chevrolets and 1 for all other automobiles. For Fords, the data begin in column 1. Variable *PRICE* is read from columns 3–6 and *QUANTITY* is read from columns 7–9. When the record is a Chevrolet, the data begins in column 13. Variable *PRICE* is read from columns 15–18 (15 is 3, 16 is 4, and so forth), and *QUANTITY* is read from columns 19–21.

Reading Both FIXED and LIST Input With REREAD

```
INPUT PROGRAM.
DATA LIST    NOTABLE FIXED/ A 1-14(A).  /*Read the FIXED portion
REREAD       COLUMN = 15.
DATA LIST    LIST/ X Y Z.               /*Read the LIST portion
END INPUT PROGRAM.

*  The value 1 on the first record is in column 15.

BEGIN DATA
FIRST RECORD  1 2 3 -1 -2 -3
NUMBER 2      4 5
THE THIRD     6 7 8
#4
FIFTH AND LAST9 10 11
END DATA.
LIST.
```

- Columns 1–14 are read in FIXED format. REREAD then resets the pointer to column 15. Thus, beginning in column 15, values are read in LIST format.
- The second DATA LIST specifies only three variables. Thus, the values –1, –2, and –3 on the first record are not read.
- The program generates a warning for the missing value on record 2 and a second warning for the three missing values on record 4.
- On the fifth and last record there is no delimiter between value LAST and value 9. REREAD can still read the 9 in LIST format.

RESPONSE RATE

RESPONSE RATE is available in SPSS Statistics Premium Edition or the Direct Marketing option.

```
RESPONSE RATE
  /VARIABLES
   NUMERATOR=varname
   DENOMINATOR=varname
   GROUP=varname
  [/MINRATE {RATE=NONE**|value     }]
            {FORMULA=value1, value1}
  [/MAXCOUNT {COUNT=ALL**|value}]
             {PERCENT=value    }

**Default if subcommand omitted.
```

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release history

Release 18.0

- Command introduced.

Example

```
RESPONSE RATE
  /VARIABLES
   NUMERATOR=Responses
   DENOMINATOR=Customers
   GROUP=Rank
  /MINRATE RATE=5
  /MAXCOUNT COUNT=6000.
```

Overview

RESPONSE RATE calculates group response rates and cumulative group response rates, expressed as a percentage, where response rate is calculated as NUMERATOR divided by DENOMINATOR for each category of GROUP, and cumulative response rate is calculated as the cumulative sum of NUMERATOR divided by the cumulative sum of DENOMINATOR for the current and all preceding categories of GROUP.

- Output includes a table of response rates, sum of the DENOMINATOR variable, cumulative response rates, and cumulative sum of the DENOMINATOR variable in ascending order of the GROUP variable, and a line chart of response rates and cumulative sum of the DENOMINATOR variable.
- If MINRATE or MAXCOUNT are specified, the table is color-coded based on the specified cumulative value(s), and the line charts include reference lines at the specified value(s).

The RESPONSE RATE command is intended primarily for use as part of the Postal Code Response Rate dialog interface, and some of the results refer to a new, aggregated dataset that is created by the dialog interface. See the topic for more information. When you use the RESPONSE RATE command outside of the Postal Code Response Rate dialog interface, this new, aggregated dataset is not created. See the topic for more information.

Basic specification

- The basic specification is the command name RESPONSE RATE and a VARIABLES subcommand.
- The MINRATE and MAXCOUNT subcommands are optional.

Operations

- Cases with missing values for any variables specified on the VARIABLES subcommand are excluded from the analysis (listwise deletion).
- If the sum of DENOMINATOR is 0 or negative for a value of GROUP, that group is excluded from the analysis.

Subcommand order

Subcommands can be specified in any order.

Syntax rules

- The VARIABLES subcommand is required.
- The VARIABLES subcommmand must include NUMERATOR, DENOMINATOR and GROUP variables.
- Equals signs (=) shown in the syntax chart and in examples are required.
- Command, subcommand, and keyword names must be spelled out fully. Abbreviation is not allowed.

Examples

Minimum Specification

```
RESPONSE RATE
  /VARIABLES
   NUMERATOR=Responses
   DENOMINATOR=Customers
   GROUP=PostalRank.
```

- Response rates are displayed for each group defined by *PostalRank*.
- Results are displayed in ascending order of the value of *PostalRank*.

MINRATE and MAXCOUNT

```
RESPONSE RATE
  /VARIABLES
   NUMERATOR=Responses
   DENOMINATOR=Customers
   GROUP=PostalRank
  /MINRATE=5
  /MAXCOUNT=6000.
```

- Rows with a cumulative response rate above 5% and a cumulative customer count below 6,000 are displayed with a green background in the table; those with cumulative response rates below 5% or a cumulative count above 6,000 are displayed with a red background.
- The upper panel of the chart includes a reference line at the 5% response rate level, and the lower panel includes a reference line at a cumulative count of 6,000 customers.

VARIABLES subcommand

The VARIABLES subcommand specifies the variables to use. This subcommand is required and includes three required keywords:

NUMERATOR. *The variable used as the numerator to calculate response rates.* The variable must be numeric.

DENOMINATOR. *The variable used as the denominator to calculate response rates.* The variable must be numeric.

GROUP. *The grouping variable.* Response rates are calculated for each category of the grouping variable. The variable can be string or numeric.

Each keyword must be followed by an equals sign (=) and a variable name.

Response rate is calculated as the NUMERATOR divided by DENOMINATOR for each category of GROUP. Cumulative response rate is calculated as the cumulative sum of NUMERATOR divided by the cumulative sum of DENOMINATOR, where cumulative sum is the sum for the current and all preceding (lower) values of GROUP. Response rates are expressed as percentages.

Example

```
RESPONSE RATE
  /VARIABLES
   NUMERATOR=Responses
```

```
   DENOMINATOR=Customers
   GROUP=PostalRank.
```

MINRATE subcommand

The optional MINRATE subcommand specifies a minimum cumulative response rate to use to color-code the output table and to use as a reference line on the response rate line chart. The following alternatives are available for specifying minimum response rate:

RATE=NONE. *No minimum response rate.* This is the default if the subcommand is omitted.

RATE=value. *Use the specified percentage value as the minimum cumulative response rate.* Specify a value greater than 0 and less than 100.

FORMULA=value1,value2. *Calculate minimum cumulative response rate based on the formula (value1/value2)*100.* Both values must be positive numbers. The result should be a value greater than 0 and less than 100. In direct marketing, for example, *value1* might be the cost of mailing a package, and *value2* might be net revenue per response. If the cost of mailing a package is $0.75 and the net revenue per response is $56, then the minimum response rate is: (0.75/56)*100=1.34%

Each keyword must be followed by an equals sign (=) and a variable name.

- If MINRATE is specified without MAXCOUNT, all rows with a cumulative response rate equal to or greater than the specified minimum are displayed with a green background, and all rows with a cumulative response rate less than the specified minimum are displayed with a red background.
- If MAXCOUNT is also specified, only rows with a cumulative response rate above the specified minimum *and* rows with a cumulative count below the specified maximum are displayed with a green background. For example, if the response rate is higher than the specified minimum, but the cumulative count is greater than the specified maximum, then the row will have a red background.

Example

```
RESPONSE RATE
  /VARIABLES
   NUMERATOR=Responses
   DENOMINATOR=Customers
   GROUP=PostalRank
  /MINRATE=5.
```

MAXCOUNT subcommand

The optional MAXCOUNT subcommand specifies the maximum cumulative total of the DENOMINATOR variable to use to color-code the output table and to use as a reference line on the cumulative count line chart. For each category of the GROUP total, the cumulative total is the sum of the DENOMINATOR values for all preceding categories plus the current category. For example, if DENOMINATOR represents number of customers, then the cumulative total for any GROUP category is the total number of customers for all groups up to and including that group.

The following alternatives are available for specifying maximum count:

COUNT=ALL. *No maximum.* This is the default if the subcommand is omitted.

COUNT=value. *Use the specified value as the maximum cumulative total.* The value must be a positive integer.

PERCENT=value. *Determine the maximum cumulative total based on the specified percentage of the overall total.* The value must be greater than 0 and less than 100.

- If MAXCOUNT is specified without MINRATE, all rows with a cumulative total for the DENOMINATOR variable less than or equal to the specified maximum are displayed with a green background, and all subsequent rows are displayed with a red background.
- If MINRATE is also specified, rows are only displayed with a green background if the maximum cumulative total is not exceeded *and* the cumulative response rate is higher than the specified

minimum. For example, if the response rate is less than the specified minimum, the row will be displayed with a red background regardless of the cumulative total.

Example

```
RESPONSE RATE
  /VARIABLES
   NUMERATOR=Responses
   DENOMINATOR=Customers
   GROUP=PostalRank
 /MAXCOUNT=6000.
```

RESTORE

```
RESTORE
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Overview

RESTORE restores SET and CD specifications that were stored by a previous PRESERVE command. RESTORE and PRESERVE are especially useful when using the macro facility. PRESERVE–RESTORE sequences can be nested up to five levels.

Basic Specification

The only specification is the command keyword. RESTORE has no additional specifications.

Limitations

RESTORE does not restore SET THREADS and SET MCACHE settings.

Example

```
PRESERVE.
SET CD '/mydata'.
GET FILE 'mydatafile.sav'.
SET TNUMBERS=BOTH.
FREQUENCIES VARIABLES=DIVISION
RESTORE.
FREQUENCIES VARIABLES=DIVISION.
```

- PRESERVE stores all current SET specifications and the current working directory setting.
- SET CD changes the working directory.
- SET TNUMBERS=BOTH changes the TNUMBERS setting to display both values and value labels in pivot tables.
- The pivot table results from the first FREQUENCIES command will display both category values and labels.
- RESTORE reestablishes the SET specifications and the working directory that were in effect prior to when PRESERVE was specified.
- The pivot table results from the second FREQUENCIES command will use the TNUMBERS setting in effect prior to PRESERVE (the default is to display only labels).

RMV

```
RMV new variables={LINT (varlist)          }
                  {MEAN (varlist [{,2 }])  }
                  {               {,n }    }
                  {               {ALL}    }
                  {MEDIAN (varlist [{,2 }])}
                  {                 {,n }  }
                  {                 {ALL}  }
                  {SMEAN (varlist)         }
                  {TREND (varlist)         }

   [/new variables=function (varlist [,span])]
```

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
RMV NEWVAR1=LINT(OLDVAR1).
```

Overview

RMV produces new variables by copying existing variables and replacing any system- or user-missing values with estimates computed by one of several methods. You can also use RMV to replace the values of existing variables. The estimated values are computed from valid data in the existing variables.

Basic Specification

The basic specification is one or more new variable names, an equals sign, a function, and an equal number of existing variables. RMV displays a list of the new variables, the number of missing values replaced, the case numbers of the first and last nonmissing cases, the number of valid cases, and the function used to produce the variables.

Syntax Rules

- The existing variables (and span, if specified) must be enclosed in parentheses.
- The equals sign is required.
- You can specify more than one equation on RMV.
- Equations are separated by slashes.
- You can specify only one function per equation.
- You can create more than one new variable per equation by specifying more than one new variable name on the left and an equal number of existing variables on the right.

Operations

- Each new variable is added to the active dataset.
- If the new variable named already exists, its values are replaced.
- If the new variable named does not already exist, it is created.
- If the same variable is named on both sides of the equation, the new variable will replace the existing variable. Valid values from the existing variable are copied into the new variable, and missing values are replaced with estimates.
- Variables are created in the order in which they are specified on the RMV command.
- If multiple variables are created on a single equation, the first new variable is based on the first existing variable, the second new variable is based on the second existing variable, and so forth.
- RMV automatically generates a variable label for each new variable describing the function and variable used to create it and the date and time of creation.

- The format of a new variable depends on the function specified and on the format of the existing variable.
- RMV honors the TSET MISSING setting that is currently in effect.
- RMV does not honor the USE command.

Limitations

- Maximum 1 function per equation.
- There is no limit on the number of variables created by an equation.
- There is no limit on the number of equations per RMV command.

LINT Function

LINT replaces missing values using linear interpolation. The last valid value before the missing value and the first valid value after the missing value are used for the interpolation.

- The only specification on LINT is a variable or variable list in parentheses.
- LINT will not replace missing values at the endpoints of variables.

Example

```
RMV NEWVAR1=LINT(OLDVAR1).
```

- This example produces a new variable called *NEWVAR1*.
- *NEWVAR1* will have the same values as *OLDVAR1* but with missing values replaced by linear interpolation.

MEAN Function

MEAN replaces missing values with the mean of valid surrounding values. The number of surrounding values used to compute the mean depends on the span.

- The specification on MEAN is a variable or variable list and a span, in parentheses.
- The span specification is optional and can be any positive integer or keyword ALL.
- A span of *n* uses *n* valid cases before and after the missing value.
- If span is not specified, it defaults to 2.
- Keyword ALL computes the mean of all valid values.
- MEAN will not replace missing values if there are not enough valid surrounding cases to satisfy the span specification.

Example

```
RMV B=MEAN(A,3).
```

- This example produces a new variable called *B*.
- *B* will have the same values as variable *A* but with missing values replaced by means of valid surrounding values.
- Each mean is based on six values—that is, the three nearest valid values on each side of the missing value.

MEDIAN Function

MEDIAN replaces missing values with the median of valid surrounding values. The number of surrounding values used to compute the median depends on the span.

- The specification on MEDIAN is a variable or variable list and a span, in parentheses.
- The span specification is optional and can be any positive integer or keyword ALL.
- A span of *n* uses *n* valid cases before and after the missing value.
- If span is not specified, it defaults to 2.
- Keyword ALL computes the median of all valid values.
- MEDIAN will not replace missing values if there are not enough valid surrounding cases to satisfy the span specification.

Example

```
RMV B=MEDIAN(A,3).
```

- This example produces a new variable called *B*.
- *B* will have the same values as *A* but with missing values replaced by medians of valid surrounding values.
- Each median is based on six values—that is, the three nearest valid values on each side of the missing value.

SMEAN Function

SMEAN replaces missing values in the new variable with the variable mean.

- The only specification on SMEAN is a variable or variable list in parentheses.
- The SMEAN function is equivalent to the MEAN function with a span specification of ALL.

Example

```
RMV VAR1 TO VAR4=SMEAN(VARA VARB VARC VARD).
```

- Four new variables (*VAR1*, *VAR2*, *VAR3*, and *VAR4*) are created.
- *VAR1* copies the values of *VARA*, *VAR2* copies *VARB*, *VAR3* copies *VARC*, and *VAR4* copies *VARD*. Any missing values in an existing variable are replaced with the mean of that variable.

TREND Function

TREND replaces missing values in the new variable with the linear trend for that point. The existing variable is regressed on an index variable scaled 1 to *n*. Missing values are replaced with their predicted values.

- The only specification on TREND is a variable or variable list in parentheses.

Example

```
RMV YHAT=TREND(VARY).
```

- This example creates a new variable called *YHAT*.
- *YHAT* has the same values as *VARY* but with missing values replaced by predicted values.

ROC ANALYSIS

ROC ANALYSIS is available in Statistics Base Edition.

```
ROC ANALYSIS varList BY varName({varvalue} {'varvalue'})
     [ /MISSING
          [ USERMISSING = {EXCLUDE**} {INCLUDE} ] ]
     [ /CRITERIA
          [ CUTOFF = {INCLUDE**} {EXCLUDE} ]
          [ TESTPOS = {LARGE**} {SMALL} ]
          [ CI = {95**} {value} ]
          [ DISTRIBUTION = {FREE**} {NEGEXPO} ] ]
     [ /DESIGN
          [ PAIR = {FALSE**} {TRUE} ]
          [ GROUP = groupVarName ]
          [ SELECT = {LEVEL(value1/string1 value2/string2)} {MIDPOINT} {CUTPOINT(value)} ] ]
     [ /PLOT
          [ CURVE = {ROC**[(REFERENCE)] [PR]} {NONE} ]
          [ PR_INTERPOLATE = {TP**} {FP} ]
          [ MODELQUALITY = {FALSE**} {TRUE} ] ]
     [ /PRINT
          [ SE = {FALSE**} {TRUE} ]
          [ COORDINATES = [ROC] [PR] ] ]
          [ CLASSIFIER ={FALSE**}{TRUE}]
```

** Default if subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 26.0 Fix pack 1

- **CLASSIFIER** keyword introduced

Release 26.0

- Command introduced

Overview

ROC ANALYSIS assess the accuracy of model predictions by plotting sensitivity versus (1-specificity) of a classification test (as the threshold varies over an entire range of diagnostic test results). The full area under a given ROC curve, or AUC, formulates an important statistic that represents the probability that the prediction will be in the correct order when a test variable is observed (for one subject randomly selected from the case group, and the other randomly selected from the control group). ROC Analysis supports the inference regarding a single AUC, precision-recall (PR) curves, and provides options for comparing two ROC curves that are generated from either independent groups or paired subjects.

Options

Paired-sample design
It is not uncommon to compare two ROC curves in a paired-sample scenario where multiple test values are measured on the same subjects that are associated with a state variable. The paired-sample design uses two, or more than two, test measurements.

Grouping variable
When a grouping variable is specified, you can request the independent group design for the test variable(s). Numeric or string values can be specified as user-specified values, or the midpoint and cut point settings can be used.

Classification
The cutoff value for positive classification can be included or excluded from analysis.

Test direction
The test direction can be set to have either larger or smaller test results indicate a more positive test.

Parameters for standard of error area
Provides options for defining the distribution assumption and confidence level percentage.

Missing values
Provides options for excluding both user-missing ans system-missing values, or treating user-missing values as valid.

Plot options
Provides options for plotting the ROC and Precision-Recall curves, and controls whether or not a bar chart is created to display the value of the lower bound of the confidence interval of the estimated "Area Under the Curve".

Print options
Provides options for defining the output for the corresponding statistics, including which statistics display in the "Area Under the Curve" table, the coordinate points of ROC and Precision-Recall curves, and classifier evaluation metrics

Basic specification

The basic specification is one or more numeric variables as the test result variable(s) and one variable as the actual state variable with one of its values. When **/DESIGN PAIR = TRUE**, at least two numeric variables must defined as test result variables. ROC ANALYSIS uses the nonparametric (distribution-free) method to calculate the area under two ROC curve. The default and minimum output are charts of the ROC curves and tables of the areas under the curves.

The **/PLOT** subcommand provides options for defining ROC curves or Precision-Recall curves.

Syntax rules

- Minimum syntax: You always need a test result variable and one actual state variable with one of its values in the ROC ANALYSIS command line.
- The test result variable must be numeric, but the state variable can be any type with any format.
- Subcommands can be specified in any order.
- When a subcommand is duplicated, only the last one is honored given that all duplicates have no syntax errors. A syntax warning is issued.
- Within a subcommand, if two or more exclusive or contradictory keywords are given, the latter keywords override the earlier ones. A syntax warning is issued.
- If a keyword is duplicated within a subcommand, it is silently ignored.

Limitations

Distributional assumptions
In the CRITERIA subcommand, the user can choose the nonparametric or parametric method to estimate the standard error of the area under the curve. Currently, the bi-negative exponential distribution is the only parametric option.

Optional output
In addition to an estimate of the area under the ROC curves, the user may request its standard error, a confidence interval, and a *p* value under the null hypothesis that the area under the curve equals 0.5. Tables of cutoff values and coordinates used to plot the ROC curves may also be displayed.

varlist BY varname(varvalue)

The varlist specifies one or more test result variables. They must be of numeric type.

The varname separated from the varlist by the word BY specifies the actual state variable. It can be of any type and any format. The user must also specify a varvalue in brackets after the second varname to define the “positive” group of the actual state variable. All other valid state values are assumed to

indicate the negative group. When **/DESIGN PAIR = TRUE**, at least two numeric variables must defined in the varlist.

MISSING Subcommand

Cases with system-missing values in the test result variable and the actual state variable are always excluded from the analysis. However, the MISSING subcommand allows the user to redefine the user-missing values to be valid.

EXCLUDE
Exclude both user-missing and system-missing values. Cases with a system-missing value or a user-missing value in either the test result variable or the actual state variable are excluded from the analysis. This is the default setting.

INCLUDE
User-missing values are treated as valid. Cases with a system-missing value in either the test result variable or the actual state variable are excluded from the analysis.

CRITERIA Subcommand

The CRITERIA subcommand allows the user to decide whether or not the cutoff value is included as the positive test result value, whether or not the larger or smaller value direction of the test result variable indicates the positive test result, the confidence level of the asymptotic confidence interval produced by /PRINT = SE, and the estimation method for the standard error of the area under the curve.

CUTOFF(INCLUDE)
Positive classification of test result values includes the cutoff values. Note that the positive test result direction is controlled by the TESTPOS keyword. This is the default setting.

CUTOFF(EXCLUDE)
Positive classification of test result values excludes the cutoff values. This distinction leads to the different sets of cutoff values, but none of the output (chart or table) is affected at this moment because the user cannot choose the cutoff values for the classification.

TESTPOS(LARGE)
The user can specify which direction of the test result variable indicates increasing strength of conviction that the subject is test positive. The larger the test result value is, the more positive the test result is. This is the default setting.

TESTPOS(SMALL)
The smaller the test result value is, the more positive the test result is.

CI(n)
Confidence level in the (two-sided) asymptotic confidence interval of the area. The user can specify any confidence level in (0, 100) for the asymptotic confidence interval created by /PRINT = SE. The default setting is **95**.

DISTRIBUTION(FREE)
The method of calculating the standard error of the area under the curve. When FREE, the standard error of the area under the curve is estimated non-parametrically—that is, without any distributional assumption. This is the default setting.

DISTRIBUTION(NEGEXPO)
The standard error is estimated assuming the bi-negative exponential distribution. This latter option is valid only when the number of positive actual state observations equals the number of negative actual state observations.

DESIGN Subcommand

The DESIGN subcommand specifies the design for the comparison of two ROC curves.

PAIR(TRUE)
Requests the paired-sample design for the test variable(s). When **TRUE** is specified with **GROUP** but not **SELECT**, **GROUP** is ignored, and the paired-sample design is implemented. When TRUE is specified with both valid **GROUP** and **SELECT**, **PAIR** is ignored and the independent-group design is implemented. **/CRITERIA = DISTRIBUTION** is disabled in the paired-sample design.

PAIR(FALSE)
The paired-sample design for the test variable(s) is not used. This is the default setting.

GROUP(*groupVarName*)
Specifies the grouping variable, and request the independent-group design for the test variable(s). A single variable name is required when this keyword is specified.

SELECT(LEVEL)
Defines the groups by specifying either two numerical values or two strings values. The values or strings must exist in the selected **GROUP** variable. The keyword is required when the **GROUP** keyword is specified.

SELECT(MIDPOINT)
When **groupVarName** is continuous, divides the test variable(s) into ≥ midpoint versus < midpoint. The keyword is required when the **GROUP** keyword is specified.

SELECT(CUTPOINT)
When **groupVarName** is continuous, divides the test variable(s) into ≥ cut point versus < cut point. The keyword is required when the **GROUP** keyword is specified.

PLOT Subcommand

The PLOT subcommand plots the ROC and precision-recall (PR) curves.

CURVE(ROC)
Creates the ROC curve chart. Use **(REFERENCE)** to draw a diagonal reference line on the resulting chart. This is the default setting.

CURVE(PR)
Creates the PR curve chart. Use **(REFERENCE)** to draw a diagonal reference line on the resulting chart.

Note: Specify **CURVE = ROC PR** to create both the ROC and PR curve charts.

CURVE(NONE)
Plotting is suppressed, and no chart is created.

PRINTERPOLATE(TP)
Specifies that stepwise interpolation, along with the true positives, is used as the interpolation method for the PR curve. This is the default setting.

PRINTERPOLATE(FP)
Specifies that PR curve interpolation is made along the false positives.

MODELQUALITY(FALSE)
A bar chart is created to display the value of the lower bound of the confidence interval of the estimated area under the curve.

MODELQUALITY(TRUE)
A bar chart is not created. This is the default setting.

PRINT Subcommand

The PRINT subcommand specifies the output settings for the corresponding statistics.

SE(TRUE)

Displays additional statistics in the area under the curve (AUC) table. The statistics include the standard error of the AUC, the asymptotic significance (2-tail), and the Asymptotic CI bounds under the null hypothesis.

SE(FALSE)

Displays only the estimated AUC. This is the default setting.

COORDINATES(ROC)

Controls the coordinate points of the ROC curves along with the cutoff values. The pairs of Sensitivity and (1-Specificity) values are given with the cutoff values for each ROC curve. By default the keyword is not specified,which suppresses the output of any coordinate points.

COORDINATES(PR)

Controls the coordinate points of the precision-recall (PR) curves along with the cutoff values. The pairs of precision and recall values are given with the cutoff values for each PR curve. By default the keyword is not specified,which suppresses the output of any coordinate points.

CLASSIFIER(TRUE)

Controls the display of the Classifier Evaluation Metrics table in the output. The table shows how well a classification model fits the data compared to a random assignment.

CLASSIFIER(FALSE)

When set to **FALSE**, the Classifier Evaluation Metrics table does not display in the output. This is the default setting.

ROC

ROC is available in Statistics Base Edition.

```
ROC varlist BY varname({varvalue  })
                      {'varvalue'}

[/MISSING = {EXCLUDE**}]
            {INCLUDE  }

[/CRITERIA = [CUTOFF({INCLUDE**})] [TESTPOS({LARGE**}) [CI({95** })]
                    {EXCLUDE  }            {SMALL  }      {value}

             [DISTRIBUTION({FREE** })]]
                           {NEGEXPO}

[/PLOT = [CURVE**[(REFERENCE)]] [NONE]]

[/PRINT = [SE][(MODELQUALITY)] [COORDINATES]].
```

** Default if subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 18.0

- MODELQUALITY keyword introduced.

Example

```
ROC
  pred  BY default (1).
```

Overview

ROC produces a receiver operating characteristic (ROC) curve and an estimate of the area under the curve.

Options

Distributional Assumptions. In the CRITERIA subcommand, the user can choose the nonparametric or parametric method to estimate the standard error of the area under the curve. Currently, the bi-negative exponential distribution is the only parametric option.

Optional Output. In addition to an estimate of the area under the ROC curve, the user may request its standard error, a confidence interval, and a *p* value under the null hypothesis that the area under the curve equals 0.5. A table of cutoff values and coordinates used to plot the ROC curve may also be displayed.

Basic Specification

The basic specification is one variable as the test result variable and one variable as the actual state variable with one of its values. ROC uses the nonparametric (distribution-free) method to calculate the area under the ROC curve. The default and minimum output is a chart of the ROC curve and a table of the area under the curve.

Syntax Rules

- Minimum syntax: You always need a test result variable and one actual state variable with one of its values in the ROC command line.
- The test result variable must be numeric, but the state variable can be any type with any format.
- Subcommands can be specified in any order.
- When a subcommand is duplicated, only the last one is honored given that all duplicates have no syntax errors. A syntax warning is issued.

- Within a subcommand, if two or more exclusive or contradictory keywords are given, the latter keywords override the earlier ones. A syntax warning is issued.
- If a keyword is duplicated within a subcommand, it is silently ignored.

Limitations

- Only the nonparametric method and one parametric method are available as the computational options for the standard error of the area under the curve at this moment. In the future, binormal and bilogistic distributions may be added to the parametric option.

varlist BY varname(varvalue)

The `varlist` specifies one or more test result variables. They must be of numeric type.

The `varname` separated from the `varlist` by the word BY specifies the actual state variable. It can be of any type and any format. The user must also specify a `varvalue` in brackets after the second `varname` to define the "positive" group of the actual state variable. All other valid state values are assumed to indicate the negative group.

MISSING Subcommand

Cases with system-missing values in the test result variable and the actual state variable are always excluded from the analysis. However, the `MISSING` subcommand allows the user to redefine the user-missing values to be valid.

EXCLUDE. *Exclude both user-missing and system-missing values.* Cases with a system-missing value or a user-missing value in either the test result variable or the actual state variable are excluded from the analysis. This is the active default.

INCLUDE. *User-missing values are treated as valid.* Cases with a system-missing value in either the test result variable or the actual state variable are excluded from the analysis.

CRITERIA Subcommand

The `CRITERIA` subcommand allows the user to decide whether or not the cutoff value is included as the positive test result value, whether or not the larger or smaller value direction of the test result variable indicates the positive test result, the confidence level of the asymptotic confidence interval produced by `/PRINT = SE`, and the estimation method for the standard error of the area under the curve.

CUTOFF(INCLUDE). *Positive classification of test result values includes the cutoff values.* Note that the positive test result direction is controlled by the TESTPOS keyword. This is the active default.

CUTOFF(EXCLUDE). *Positive classification of test result values excludes the cutoff values.* This distinction leads to the different sets of cutoff values, but none of the output (chart or table) is affected at this moment because the user cannot choose the cutoff values for the classification.

TESTPOS(LARGE). *The user can specify which direction of the test result variable indicates increasing strength of conviction that the subject is test positive.* The larger the test result value is, the more positive the test result is. LARGE is the active default.

TESTPOS(SMALL). *The smaller the test result value is, the more positive the test result is.*

CI(n). *Confidence level in the (two-sided) asymptotic confidence interval of the area.* The user can specify any confidence level in (0, 100) for the asymptotic confidence interval created by `/PRINT = SE`. The active default parameter value is 95.

DISTRIBUTION(FREE). *The method of calculating the standard error of the area under the curve.* When FREE, the standard error of the area under the curve is estimated nonparametrically—that is, without any distributional assumption.

DISTRIBUTION(NEGEXPO). *The standard error is estimated assuming the bi-negative exponential distribution.* This latter option is valid only when the number of positive actual state observations equals the number of negative actual state observations.

PRINT Subcommand

The PRINT subcommand controls the display of optional table and chart output. Note that the area under the curve is always displayed.

SE (MODELQUALITY). *Standard error of the area estimate.* In addition to the standard error of the estimated area under the curve, the asymptotic 95% (or other user-specified confidence level) confidence interval as well as the asymptotic *p* value under the null hypothesis that the true area = 0.5 are calculated. The optional (MODELQUALITY) specification produces a simple bar chart that displays the value of the lower bound of the confidence interval of the area under the curve, expressed as a proportion, on a scale of 0 to 1. Note that the area under the curve statistic is asymptotically normally distributed.

COORDINATES. *Coordinate points of the ROC curve along with the cutoff values.* Pairs of sensitivity and 1 – specificity values are given with the cutoff values for each curve.

PLOT Subcommand

The PLOT subcommand controls the display of chart output.

CURVE(REFERENCE). *The ROC curve chart is displayed.* The keyword CURVE is the active default. In addition, the user has an option to draw the diagonal reference line (sensitivity = 1 – specificity) by the bracketed parameter REFERENCE.

NONE. *No charts are displayed.*

SAMPLE

```
SAMPLE {decimal value}
       {n FROM m     }
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

Example

```
SAMPLE .25.
```

Overview

SAMPLE permanently draws a random sample of cases for processing in all subsequent procedures. For a temporary sample, use a TEMPORARY command before SAMPLE.

Basic Specification

The basic specification is either a decimal value between 0 and 1 or the sample size followed by keyword FROM and the size of the active dataset.

- To select an approximate percentage of cases, specify a decimal value between 0 and 1.
- To select an exact-size random sample, specify a positive integer that is less than the file size, and follow it with keyword FROM and the file size.

Operations

- SAMPLE is a permanent transformation.
- Sampling is based on a pseudo-random-number generator that depends on a seed value that is established by the program. On some implementations of the program, this number defaults to a fixed integer, and a SAMPLE command that specifies *n* FROM *m* will generate the identical sample whenever a session is rerun. To generate a different sample each time, use the SET command to reset SEED to a different value for each session. See the SET command for more information.
- If sampling is done by using the *n* FROM *m* method, and the TEMPORARY command is specified, successive samples will not be the same because the seed value changes each time that a random-number series is needed within a session.
- A proportional sample (a sample that is based on a decimal value) usually does not produce the exact proportion that is specified.
- If the number that is specified for *m* following FROM is less than the actual file size, the sample is drawn only from the first *m* cases.
- If the number following FROM is greater than the actual file size, the program samples an equivalent proportion of cases from the active dataset.
- If SAMPLE follows SELECT IF, SAMPLE samples only cases that are selected by SELECT IF.
- If SAMPLE precedes SELECT IF, cases are selected from the sample.
- If more than one SAMPLE command is specified in a session, each command acts on the sample that was selected by the preceding SAMPLE command.
- If N OF CASES is used with SAMPLE, the program reads as many records as required to build the specified *n* cases. It makes no difference whether N OF CASES precedes or follows SAMPLE.

Limitations

SAMPLE cannot be placed in a FILE TYPE-END FILE TYPE or INPUT PROGRAM-END INPUT PROGRAM structure. SAMPLE can be placed nearly anywhere following these commands in a transformation program. See the topic “Commands and Program States” on page 2187 for more information. .

Examples

Sampling an Approximate Percentage of Cases

```
SAMPLE .25.
```

- This command samples approximately 25% of the cases in the active dataset.

Sampling a Specific Number of Cases

```
SAMPLE 500 FROM 3420.
```

- The active dataset must have 3420 or more cases to obtain a random sample of exactly 500 cases.
- If the file contains fewer than 3420 cases, proportionally fewer cases are sampled.
- If the file contains more than 3420 cases, a random sample of 500 cases is drawn from the first 3420 cases.

Sampling Subgroups

```
DO IF  SEX EQ 'M'.
SAMPLE 1846 FROM 8000.
END IF.
```

- SAMPLE is placed inside a DO IF-END IF structure to sample subgroups differently. Assume that this survey is for 10,000 people in which 80% of the sample is male, while the known universe is 48% male. To obtain a sample that corresponds to the known universe and that maximizes the size of the sample, 1846 (48/52*2000) males and all females must be sampled. The DO IF structure restricts the sampling process to the males.

SAVE

```
SAVE OUTFILE='filespec'

 [/VERSION={3**}]
           {2  }

 [/UNSELECTED=[{RETAIN}]
              {DELETE }

 [/KEEP={ALL**  }] [/DROP=varlist]
        {varlist}

 [/RENAME=(old varlist=new varlist)...]

 [/MAP]  [/{COMPRESSED**}]
           {UNCOMPRESSED}
           {ZCOMPRESSED }

 [/NAMES]

 [/PERMISSIONS={READONLY }]
               {WRITEABLE}

 [/PASSPROTECT PASSWORD={NONE**   }  [ENCRYPTEDPW={NO**} ]
                       {'password}               {YES }
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release history

Release 21.0

- ZCOMPRESSED subcommand added.
- PASSPROTECT subcommand added.

Example

```
SAVE OUTFILE='/data/empl.sav'.
```

Overview

SAVE produces a data file in IBM SPSS Statistics format, which contains data plus a dictionary. The dictionary contains a name for each variable in the data file plus any assigned variable and value labels, missing-value flags, and variable print and write formats. The dictionary also contains document text that was created with the DOCUMENTS command.

XSAVE also creates IBM SPSS Statistics data files. The difference is that SAVE causes data to be read, while XSAVE is not executed until data are read for the next procedure.

See SAVE TRANSLATE for information about saving data files that can be used by other programs.

Options

Compatibility with Earlier Releases. You can save a data file that can be read by releases prior to 7.5.

Variable Subsets and Order. You can use the DROP and KEEP subcommands to save a subset of variables and reorder the variables that are saved.

Filtered Cases. If filtering is in effect, you use the UNSELECTED subcommand to specify inclusion or exclusion of filtered cases. By default, all cases are included.

Variable Names. You can use the RENAME subcommand to rename variables as they are copied into the IBM SPSS Statistics data file. Long variable names are supported when explicitly specified on the /VARIABLES and /STRINGS subcommands. Variable names are truncated to 8 bytes when they are referenced through a vector in the /NAMES subcommand.

Variable Map. To confirm the names and order of the variables that are saved in the data file, use the MAP subcommand. MAP displays the variables that are saved in the data file next to their corresponding names in the active dataset.

Data Compression. You can use the COMPRESSED or ZCOMPRESSED subcommand to write the data file in compressed form. You can use the UNCOMPRESSED subcommand to write the data file in an uncompressed form. See the topic "COMPRESSED, UNCOMPRESSED, and ZCOMPRESSED Subcommands " on page 1733 for more information.

Encryption. You can use the PASSPROTECT subcommand to save the file as encrypted, allowing you to protect confidential information stored in the file. See the topic "PASSPROTECT Subcommand " on page 1734 for more information.

Basic Specification

The basic specification is the OUTFILE subcommand, which specifies a name for the data file to be saved.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- OUTFILE is required and can be specified only once. If OUTFILE is specified more than once, only the last specified OUTFILE is in effect.
- KEEP, DROP, RENAME, and MAP can each be used as many times as needed.
- Only one of the subcommands COMPRESSED, UNCOMPRESSED, or ZCOMPRESSED can be specified per SAVE command.

Operations

- SAVE is executed immediately and causes the data to be read.
- The new data file dictionary is arranged in the same order as the active dataset dictionary, unless variables are reordered with the KEEP subcommand. Documentary text from the active dataset dictionary is always saved unless it is dropped with the DROP DOCUMENTS command before SAVE.
- New variables that were created by transformations and procedures that occurred prior to the SAVE command are included in the new data file, and variables that were altered by transformations are saved in their modified form. Results of any temporary transformations that immediately precede the SAVE command are included in the file; scratch variables are not included.
- The active dataset is still available for transformations and procedures after SAVE is executed.
- SAVE processes the dictionary first and displays a message that indicates how many variables will be saved. After the data are written, SAVE indicates how many cases were saved. If the second message does not appear, the file was probably not completely written.

Examples

Renaming Variables for a Saved Version

```
GET FILE='/hubempl.sav'.
SAVE OUTFILE='/data/empl88.sav' /RENAME=(AGE=AGE88) (JOBCAT=JOBCAT88).
```

- The GET command retrieves the IBM SPSS Statistics data file *hubempl.sav*.
- The RENAME subcommand renames variable *AGE* to *AGE88* and variable *JOBCAT* to *JOBCAT88*.
- SAVE causes the data to be read and saves a new data file with the filename *empl88.sav*. The original data file *hubempl.sav* is not changed.

Performing Temporary Transformations Prior to Saving

```
GET FILE='/data/hubempl.sav'.
TEMPORARY.
RECODE DEPT85 TO DEPT88 (1,2=1) (3,4=2) (ELSE=9).
VALUE LABELS DEPT85 TO DEPT88 1 'MANAGEMENT' 2 'OPERATIONS' 9 'UNKNOWN'.
```

```
SAVE OUTFILE='/data/hubtemp.sav'.
CROSSTABS DEPT85 TO DEPT88 BY JOBCAT.
```

- The GET command retrieves the IBM SPSS Statistics data file *hubempl.sav.*
- The TEMPORARY command indicates that RECODE and VALUE LABELS are in effect only for the next command that reads the data (SAVE).
- The RECODE command recodes values for all variables between and including *DEPT85* and *DEPT88* on the active dataset.
- The VALUE LABELS command specifies new labels for the recoded values.
- The OUTFILE subcommand on SAVE specifies *hubtemp.sav* as the new data file. The *hubtemp.sav* file will include the recoded values for *DEPT85* to *DEPT88* and the new value labels.
- The CROSSTABS command crosstabulates *DEPT85* to *DEPT88* with *JOBCAT*. Because the RECODE and VALUE LABELS commands were temporary, the CROSSTABS output does not reflect the recoding and new labels.
- If XSAVE were specified instead of SAVE, the data would be read only once. Both the saved data file and the CROSSTABS output would reflect the temporary recoding and labeling of the department variables.

OUTFILE Subcommand

OUTFILE specifies the data file to be saved. OUTFILE is required and can be specified only once. If OUTFILE is specified more than once, only the last OUTFILE is in effect.

VERSION Subcommand

VERSION allows you to save a data file that can be opened in releases prior to 7.5. If VERSION is not specified or is specified with no value, the default of 3 is used, which saves the file in a format that cannot be read by releases prior to 7.5. Specify 2 to save a file that is compatible with earlier releases.

Variable Names

For data files with variable names that are longer than eight bytes in version 10.*x* or 11.*x*, unique, eight-byte versions of variable names are used (but the original variable names are preserved for use in release 12.0 or later). In releases prior to version 10.0, the original long variable names are lost if you save the data file.

UNSELECTED Subcommand

UNSELECTED determines whether cases that were excluded on a previous FILTER or USE command are to be retained or deleted in the saved data file. The default is RETAIN. The UNSELECTED subcommand has no effect when the active dataset does not contain unselected cases.

RETAIN. *Retain the unselected cases*. All cases in the active dataset are saved. This setting is the default when UNSELECTED is specified by itself.

DELETE. *Delete the unselected cases*. Only cases that meet the FILTER or USE criteria are saved in the data file.

DROP and KEEP Subcommands

DROP and KEEP are used to save a subset of variables. DROP specifies the variables that are not to be saved in the new data file; KEEP specifies the variables that are to be saved in the new data file; variables that are not named on KEEP are dropped.

- Variables can be specified in any order. The order of variables on KEEP determines the order of variables in the data file. The order on DROP does not affect the order of variables in the data file.
- Keyword ALL on KEEP refers to all remaining variables that were not previously specified on KEEP or DROP. ALL must be the last specification on KEEP.
- If a variable is specified twice on the same subcommand, only the first mention is recognized.
- Multiple DROP and KEEP subcommands are allowed. If a variable is specified that is not in the active dataset, or that has been dropped because of a previous DROP or KEEP subcommand, an error results, and the SAVE command is not executed.
- Keyword TO can be used to specify a group of consecutive variables in the active file.

Example

```
GET FILE='/data/personnel.sav'.
COMPUTE TENURE=(12-CMONTH +(12*(88-CYEAR)))/12.
COMPUTE JTENURE=(12-JMONTH +(12*(88-JYEAR)))/12.
VARIABLE LABELS       TENURE 'Tenure in Company'
                     JTENURE 'Tenure in Grade'.
SAVE OUTFILE='/data/personnel88.sav' /DROP=GRADE STORE
  /KEEP=LNAME NAME TENURE JTENURE ALL.
```

- The variables *TENURE* and *JTENURE* are created by COMPUTE commands and are assigned variable labels by the VARIABLE LABELS command. *TENURE* and *JTENURE* are added to the end of the active dataset.
- DROP excludes variables *GRADE* and *STORE* from file *personnel88.sav*. KEEP specifies that *LNAME*, *NAME*, *TENURE*, and *JTENURE* are the first four variables in file *personnel88.sav*, followed by all remaining variables not specified on DROP. These remaining variables are saved in the same sequence as they appear in the original file.

RENAME Subcommand

RENAME changes the names of variables as they are copied into the new data file.

- The specification on RENAME is a list of old variable names followed by an equals sign and a list of new variable names. The same number of variables must be specified on both lists. Keyword TO can be used in the first list to refer to consecutive variables in the active dataset and can be used in the second list to generate new variable names. The entire specification must be enclosed in parentheses.
- Alternatively, you can specify each old variable name individually, followed by an equals sign and the new variable name. Multiple sets of variable specifications are allowed. The parentheses around each set of specifications are optional.
- RENAME does not affect the active dataset. However, if RENAME precedes DROP or KEEP, variables must be referred to by their new names on DROP or KEEP.
- Old variable names do not need to be specified according to their order in the active dataset.
- Name changes take place in one operation. Therefore, variable names can be exchanged between two variables.
- Multiple RENAME subcommands are allowed.

Examples

```
SAVE OUTFILE='/data/empl88.sav'
  /RENAME AGE=AGE88 JOBCAT=JOBCAT88.
```

- RENAME specifies two name changes for the file *empl88.sav*: The variable *AGE* is renamed to *AGE88*, and the variable *JOBCAT* is renamed to *JOBCAT88.*

```
SAVE OUTFILE='/data/empl88.sav'
  /RENAME (AGE JOBCAT=AGE88 JOBCAT88).
```

- The name changes are identical to the changes in the previous example: *AGE* is renamed to *AGE88*, and *JOBCAT* is renamed to *JOBCAT88*. The parentheses are required with this method.

MAP Subcommand

MAP displays a list of the variables in the saved data file and their corresponding names in the active dataset.

- The only specification is keyword MAP.
- Multiple MAP subcommands are allowed. Each MAP subcommand maps the results of subcommands that precede it (but not results of subcommands that follow it).

Example

```
GET FILE='/data/hubempl.sav'.
SAVE OUTFILE='/data/empl88.sav' /RENAME=(AGE=AGE88)(JOBCAT=JOBCAT88)
  /KEEP=LNAME NAME JOBCAT88 ALL /MAP.
```

- MAP confirms the new names for *AGE* and *JOBCAT* and the order of variables in the *empl88.sav* file (*LNAME*, *NAME*, and *JOBCAT88*, followed by all remaining variables from the active dataset).

COMPRESSED, UNCOMPRESSED, and ZCOMPRESSED Subcommands

COMPRESSED and ZCOMPRESSED save the file in compressed form. UNCOMPRESSED saves the file in uncompressed form.

- The only specification is the keyword COMPRESSED, UNCOMPRESSED, or ZCOMPRESSED.
- In a file compressed with COMPRESSED, small integers (from –99 to 155) are stored in one byte instead of the eight bytes that are used in an uncompressed file.
- In a file compressed with ZCOMPRESSED, case data (regardless of type) are compressed block by block with a high compression ratio. Only IBM SPSS Statistics version 21 or later can open a file compressed with ZCOMPRESSED. Therefore, it is recommended that you use the *.zsav* file extension when saving files with ZCOMPRESSED. Older versions of IBM SPSS Statistics will ignore this extension.
- Compressed data files occupy less disk space than uncompressed data files. Files compressed with ZCOMPRESSED occupy the least disk space.
- Compressed files may take more or less time to open and save, depending on the file size and system configuration. Extra time is needed to de-compress and compress the files. However, because compressed files are smaller on disk, they reduce the time needed to read and write from disk. As the file size gets larger, this time savings surpasses the extra time needed to de-compress and compress the files. This is especially true for files compressed with ZCOMPRESSED, because these files are smallest on disk.
- The GET command, which reads IBM SPSS Statistics data files, does not need to specify whether the files that it reads are compressed or uncompressed.
- Only one of the subcommands COMPRESSED, UNCOMPRESSED, or ZCOMPRESSED can be specified per SAVE command. COMPRESSED is the default.

NAMES Subcommand

For all variable names that are longer than eight bytes, NAMES displays the eight-byte equivalents that will be used if you read the data file into a version prior to release 12.0.

PERMISSIONS Subcommand

The PERMISSIONS subcommand sets the operating system read/write permissions for the file.

READONLY. *File permissions are set to read-only for all users.* The file cannot be saved by using the same file name with subsequent changes unless the read/write permissions are changed in the operating system or the subsequent SAVE command specifies PERMISSIONS=WRITEABLE.

WRITEABLE. *File permissions are set to allow writing for the file owner.* If file permissions were set to read-only for other users, the file remains read-only for them.

Your ability to change the read/write permissions may be restricted by the operating system.

PASSPROTECT Subcommand

The PASSPROTECT subcommand specifies the settings for saving a data file as an encrypted file. Once encrypted, the file can only be opened by providing the correct password.

PASSWORD. Specifies the password required to open this file. Passwords are limited to 10 characters and are case-sensitive. The specified value must be enclosed in quotation marks. The keyword NONE specifies that the file will not be encrypted and is the default.

Warning: Passwords cannot be recovered if they are lost. If the password is lost the file cannot be opened.

ENCRYPTEDPW Keyword

The ENCRYPTEDPW keyword specifies whether the password is encrypted.

NO. The password is not encrypted. It is treated as plain text. This is the default.

YES. The password is encrypted. Passwords are always encrypted when pasting syntax from the Save Data As dialog. *Note*: ENCRYPTEDPW=YES should only be used when the password is known to be encrypted.

Creating strong passwords

- Use eight or more characters.
- Include numbers, symbols and even punctuation in your password.
- Avoid sequences of numbers or characters, such as "123" and "abc", and avoid repetition, such as "111aaa".
- Do not create passwords that use personal information such as birthdays or nicknames.
- Periodically change the password.

Note: Encrypted data files and output documents cannot be opened in versions of IBM SPSS Statistics prior to version 21. Encrypted syntax files cannot be opened in versions prior to version 22.

SAVE CODEPAGE

```
SAVE CODEPAGE
 /OUTFILE='filespec'

 [/ENCODING={LOCALE**  }]
            {'encoding'}

 [/UNSELECTED=[{RETAIN}]
               {DELETE}

 [/KEEP={ALL**  }] [/DROP=varlist]
        {varlist}

 [/PASSPROTECT PASSWORD={'password'}  [ENCRYPTEDPW={NO**}] ]
                        {NONE**    }               {YES }
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release history

Release 23.0

- Command introduced.

Example

```
SAVE CODEPAGE
  /OUTFILE='/data/empl.sav'
  /ENCODING='de_DE.windows-1252'.
```

Overview

In Unicode mode, SAVE CODEPAGE produces a data file in the specified locale character encoding.

Options

Encoding. You can use the ENCODING subcommand to specify the character encoding.

Variable Subsets and Order. You can use the DROP and KEEP subcommands to save a subset of variables and reorder the variables that are saved.

Filtered Cases. If filtering is in effect, you can use the UNSELECTED subcommand to specify inclusion or exclusion of filtered cases. By default, all cases are included.

Encryption. You can use the PASSPROTECT subcommand to save the file as encrypted, allowing you to protect confidential information stored in the file.

Basic specification

The basic specification is the OUTFILE subcommand, which specifies a name for the data file to be saved.

Syntax rules

- OUTFILE is required. All other subcommands are optional.
- KEEP and DROP can each be used as many times as needed.
- ENCODING, UNSELECTED, and PASSPROTECT can each be used only once.

Operations

- SAVE CODEPAGE is executed immediately and causes the data to be read.

- The new data file dictionary is arranged in the same order as the active dataset dictionary, unless variables are reordered with the KEEP subcommand.
- New variables that were created by transformations and procedures that occurred prior to the SAVE CODEPAGE command are included in the new data file, and variables that were altered by transformations are saved in their modified form. Results of any temporary transformations that immediately precede the SAVE CODEPAGE command are included in the file; scratch variables are not included.
- The active dataset is not changed by SAVE CODEPAGE. The file encoding specified on SAVE CODEPAGE does not affect the active dataset. SAVE CODEPAGE operates in a manner similar to commands that create data files in external formats, such as WRITE and SAVE TRANSLATE.

Limitations

- SAVE CODEPAGE is only available in Unicode mode. An error occurs if you use this command in code page mode. For more information on Unicode and code page mode, see the SET command, UNICODE subcommand.

OUTFILE Subcommand

OUTFILE specifies the data file to be saved. OUTFILE is required and can be specified only once. The file specification should be enclosed in quotation marks.

ENCODING Subcommand

ENCODING specifies the code page character encoding format of the file. The keyword is followed by an equals sign and the keyword LOCALE or a quoted encoding specification.

LOCALE
Saves the file in the current locale encoding. This is the default setting. For more information, see the SET command, LOCALE subcommand.

'encoding'
The quoted encoding value can be a numeric Windows code page value (for example, '1252'), or an IANA code page value (for example, 'iso8859-1' or cp1252).

Example

```
SAVE CODEPAGE
  /OUTFILE='/data/empl.sav'
  /ENCODING='cp1252'.
```

UNSELECTED Subcommand

UNSELECTED determines whether cases that were excluded on a previous FILTER or USE command are to be retained or deleted in the saved data file. The default is RETAIN. The UNSELECTED subcommand has no effect when the active dataset does not contain unselected cases.

RETAIN
Retain the unselected cases. All cases in the active dataset are saved. This setting is the default when UNSELECTED is specified by itself.

DELETE
Delete the unselected cases. Only cases that meet the FILTER or USE criteria are saved in the data file.

DROP and KEEP Subcommands

DROP and KEEP are used to save a subset of variables. DROP specifies the variables that are not to be saved in the new data file; KEEP specifies the variables that are to be saved in the new data file; variables that are not named on KEEP are dropped.

- Variables can be specified in any order. The order of variables on KEEP determines the order of variables in the data file. The order on DROP does not affect the order of variables in the data file.
- Keyword ALL on KEEP refers to all remaining variables that were not previously specified on KEEP or DROP. ALL must be the last specification on KEEP.
- If a variable is specified twice on the same subcommand, only the first mention is recognized.
- Multiple DROP and KEEP subcommands are allowed. If a variable is specified that is not in the active dataset, or that has been dropped because of a previous DROP or KEEP subcommand, an error results, and the SAVE CODEPAGE command is not executed.
- Keyword TO can be used to specify a group of consecutive variables in the active file.

PASSPROTECT Subcommand

The PASSPROTECT subcommand specifies the settings for saving a data file as an encrypted file. Once encrypted, the file can only be opened by providing the correct password.

PASSWORD. Specifies the password required to open this file. Passwords are limited to 10 characters and are case-sensitive. The specified value must be enclosed in quotation marks. The keyword NONE specifies that the file will not be encrypted and is the default.

Warning: Passwords cannot be recovered if they are lost. If the password is lost the file cannot be opened.

ENCRYPTEDPW Keyword

The ENCRYPTEDPW keyword specifies whether the password is encrypted.

NO. The password is not encrypted. It is treated as plain text. This is the default.

YES. The password is encrypted. Passwords are always encrypted when pasting syntax from the Save Data As dialog. *Note*: ENCRYPTEDPW=YES should only be used when the password is known to be encrypted.

Creating strong passwords

- Use eight or more characters.
- Include numbers, symbols and even punctuation in your password.
- Avoid sequences of numbers or characters, such as "123" and "abc", and avoid repetition, such as "111aaa".
- Do not create passwords that use personal information such as birthdays or nicknames.
- Periodically change the password.

Note: Encrypted data files and output documents cannot be opened in versions of IBM SPSS Statistics prior to version 21. Encrypted syntax files cannot be opened in versions prior to version 22.

SAVE DATA COLLECTION

This command is available only on Windows operating systems.

```
SAVE DATA COLLECTION

  /OUTFILE='filespec'

  /METADATA='filespec'

 [/UNSELECTED=[{RETAIN**}]
               {DELETE}

 [/KEEP={ALL**  }] [/DROP=varlist]
        {varlist}

 [/MAP]
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release history

Release 15.0

- Command introduced as SAVE DIMENSIONS.

Release 18.0

- SAVE DIMENSIONS deprecated. Command name changed to SAVE DATA COLLECTION.

Example

```
SAVE DATA COLLECTION
  /OUTFILE='/data/survey.sav'
  /METADATA='/data/survey.mdd'.
```

Overview

SAVE DATA COLLECTION produces IBM SPSS Statistics data files and Data Collection metadata files that you can use to read the IBM SPSS Statistics data file into Data Collection applications.

Note: There may be instances where the data cannot be re-loaded to the Data Collection database in the correct format due to the complexity in the structure.

Options

Variable subsets and order. You can use the DROP and KEEP subcommands to save a subset of variables and reorder the variables that are saved.

Filtered cases. If filtering is in effect, you use the UNSELECTED subcommand to specify inclusion or exclusion of filtered cases. By default, all cases are included.

Variable map. To confirm the names and order of the variables that are saved in the IBM SPSS Statistics data file, use the MAP subcommand. MAP displays the variables that are saved in the IBM SPSS Statistics data file next to their corresponding names in the active dataset.

Basic specification

The basic specification is the OUTFILE subcommand that specifies a name for the IBM SPSS Statistics data file to be saved and the METADATA subcommand that specifies the name for the Data Collection metadata file.

Subcommand order

- Subcommands can be specified in any order.

Syntax rules

- OUTFILE and METADATA are required and can be specified only once.
- KEEP, DROP, MAP, and UNSELECTED are optional.

Operations

For new variables and datasets not created from Data Collection data sources, IBM SPSS Statistics variable attributes are mapped to Data Collection metadata attributes.

If the active dataset was created from a Data Collection data source:

- The new metadata file is created by merging the original metadata attributes with metadata attributes for any new variables, plus any changes to original variables that might affect their metadata attributes (for example, addition of, or changes to, value labels).
- For original variables read from the Data Collection data source, any metadata attributes not recognized by IBM SPSS Statistics are preserved in their original state. For example, IBM SPSS Statistics converts grid variables to regular IBM SPSS Statistics variables, but the metadata that defines these grid variables is preserved when you save the new metadata file.
- If any Data Collection variables were automatically renamed to conform to IBM SPSS Statistics variable naming rules, the metadata file maps the converted names back to the original Data Collection variable names.

The presence or absence of value labels can affect the metadata attributes of variables and consequently the way those variables are read by Data Collection applications. If value labels have been defined for any nonmissing values of a variable, they should be defined for all nonmissing values of that variable; otherwise, the unlabeled values will be dropped when the data file is read by Data Collection.

Limitations

This feature is only available with IBM SPSS Statistics installed on Microsoft Windows operating systems, and is only available in local analysis mode. This feature is not available in distributed analysis mode using IBM SPSS Statistics Server.

To write Data Collection metadata files, you must have the following items installed:

- .NET framework. To obtain the most recent version of the .NET framework, go to http://www.microsoft.com/net.
- Data Collection Survey Reporter Developer Kit.

OUTFILE subcommand

OUTFILE specifies the IBM SPSS Statistics data file to be saved.

- OUTFILE is required and can be specified only once.
- File specifications should be enclosed in quotes.
- The standard extension for IBM SPSS Statistics data files is *.sav*. You should explicitly specify the extension; it will not be automatically appended to the file name.

METADATA subcommand

METADATA specifies the Data Collection metadata file to be saved. See the Operations section of the Overview for more information.

- METADATA is required and can be specified only once.
- File specifications should be enclosed in quotes.
- The standard extension for a Data Collection metadata file is *.mdd*. You should explicitly specify the extension; it will not be automatically appended to the file name.

UNSELECTED subcommand

UNSELECTED determines whether cases that were excluded on a previous FILTER or USE command are to be retained or deleted in the IBM SPSS Statistics data file. The default is RETAIN. The UNSELECTED subcommand has no effect when the active dataset does not contain unselected cases. Since the metadata file does not contain case data, this subcommand has no effect on the contents of the metadata file.

RETAIN. *Retain the unselected cases*. All cases in the active dataset are saved. This is the default.

DELETE. *Delete the unselected cases*. Only cases that meet the FILTER or USE criteria are saved in the IBM SPSS Statistics data file.

DROP and KEEP subcommands

DROP and KEEP are used to save a subset of variables. DROP specifies the variables that are not to be saved in the new data file; KEEP specifies the variables that are to be saved in the new data file; variables that are not named on KEEP are dropped.

- Variables can be specified in any order. The order of variables on KEEP determines the order of variables in the data file. The order on DROP does not affect the order of variables in the data file.
- Keyword ALL on KEEP refers to all remaining variables that were not previously specified on KEEP or DROP. ALL must be the last specification on KEEP.
- If a variable is specified twice on the same subcommand, only the first mention is recognized.
- Multiple DROP and KEEP subcommands are allowed. If a variable is specified that is not in the active dataset or that has been dropped because of a previous DROP or KEEP subcommand, an error results, and the SAVE DATA COLLECTION command is not executed.
- Keyword TO can be used to specify a group of consecutive variables in the active file.
- If the active dataset was created from a Data Collection data source, any original variables defined as grid or array elements in the Data Collection data source are retained in the metadata file, even if those variables are not included in the IBM SPSS Statistics data file. Thus, the original grid or array structure is preserved, but there will be no case data for any variables not included in the IBM SPSS Statistics data file.

Example

```
SAVE DATA COLLECTION
  /OUTFILE='/data/survey.sav'
  /METADATA='/data/survey.mdd'
  /DROP gridVar7
  /KEEP gridVar1 to gridVar8.
```

Assuming that the order of the variables in the active dataset is *gridVar1*, *gridVar2*, *gridVar3*,...*gridVar8* and that all eight variables are grid variables in the original Data Collection data source, *gridVar7* will be dropped from *survey.sav*, but the original metadata for *gridVar7* will be preserved in *survey.mdd*.

MAP subcommand

MAP displays a list of the variables in the IBM SPSS Statistics data file and their corresponding names in the active dataset.

- The only specification is keyword MAP.
- Multiple MAP subcommands are allowed. Each MAP subcommand maps the results of the DROP and KEEP subcommands that precede it (but not results of subcommands that follow it).

Example

```
SAVE DATA COLLECTION
  /OUTFILE='/data/survey.sav'
```

```
  /METADATA='/data/survey.mdd'
  /DROP var7 /MAP
  /KEEP var1 to var8 /MAP.
```

SAVE MODEL

```
SAVE MODEL OUTFILE='filename'

 [/KEEP={ALL**      }]
        {model names}
        {procedures }

 [/DROP={model names}]
        {procedures }

 [/TYPE={MODEL**}]
        {COMMAND}
```

**Default if the subcommand is omitted.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic "Command Order" on page 36 for more information.

Example

```
SAVE MODEL OUTFILE='ACFMOD.DAT'.
```

Overview

SAVE MODEL saves the models that are created by certain procedures in a model file. The saved model file can be read later in the session or in another session with the READ MODEL command.

Options

You can save a subset of models into the file using the DROP and KEEP subcommands. You can use the TYPE subcommand to control whether models are specified by model name or by the name of the procedure that generated them.

Basic Specification

The basic specification is the OUTFILE subcommand followed by a filename.

- By default, SAVE MODEL saves all currently active models in the specified file. Each model that is saved in the file includes information such as the procedure that created it, the model name, the specified variable names, subcommands and specifications used, and parameter estimates. The names of the models are either the default *MOD_n* names or the names that are assigned on the MODEL NAME command. In addition to the model specifications, the TSET settings that are currently in effect are saved.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- If a subcommand is specified more than once, only the last subcommand is executed.

Operations

- SAVE MODEL is executed immediately.
- Model files are designed to be written and read by specific procedures and should not be edited.
- The active models are not affected by the SAVE MODEL command.
- DATE specifications are not saved in the model file.
- Models are not saved in IBM SPSS Statistics data files.
- The following procedures can generate models that can be saved with the SAVE MODEL command: AREG, ARIMA, EXSMOOTH, SEASON, and SPECTRA in the Forecasting module; ACF, CASEPLOT, CCF, CURVEFIT, PACF, PPLOT, and TSPLOT in the Core system; and WLS and 2SLS in the Regression module.

Limitations

- A maximum of one filename can be specified.
- The READ MODEL command can only read models created by a SAVE MODEL command on the same operating system/platform.

OUTFILE Subcommand

OUTFILE names the file where models will be stored and is the only required subcommand.

- The only specification on OUTFILE is the name of the model file.
- The filename should be enclosed in quotes.
- Only one filename can be specified.
- You can store models in other directories by specifying a fully qualified filename.

KEEP and DROP Subcommands

KEEP and DROP allow you to save a subset of models. By default, all currently active models are saved.

- KEEP specifies models to be saved in the model file.
- DROP specifies models that are not saved in the model file.
- Models can be specified by using individual model names or the names of the procedures that created them. To use procedure names, you must specify COMMAND on the TYPE subcommand.
- Model names are either the default *MOD_n* names or the names that are assigned with MODEL NAME.
- If you specify a procedure name on KEEP, all models that are created by that procedure are saved; on DROP, any models created by that procedure are not included in the model file.
- Model names and procedure names cannot be mixed on a single SAVE MODEL command.
- If more than one KEEP or DROP subcommand is specified, only the last subcommand is executed.
- You can specify the keyword ALL on KEEP to save all models that are currently active. This setting is the default.

Example

```
SAVE MODEL OUTFILE='ACFCCF.DAT'
  /KEEP=ACF1 ACF2.
```

- In this example, only models *ACF1* and *ACF2* are saved in model file *ACFCCF.DAT*.

TYPE Subcommand

TYPE indicates whether models are specified by model name or procedure name on DROP and KEEP.

- One keyword, MODEL or COMMAND, can be specified after TYPE.
- MODEL is the default and indicates that models are specified as model names.
- COMMAND indicates that the models are specified by procedure name.
- TYPE has no effect if KEEP or DROP is not specified.
- The TYPE specification applies only to the current SAVE MODEL command.

Example

```
SAVE MODEL OUTFILE='CURVE1.DAT'
  /KEEP=CURVEFIT
  /TYPE=COMMAND.
```

- This command saves all models that were created by the CURVEFIT procedure into the model file *CURVE1.DAT*.

SAVE TRANSLATE

```
SAVE TRANSLATE

[{/OUTFILE=file               }]
 {/CONNECT=ODBC connect string}

 [{/UNENCRYPTED}]
  {/ENCRYPTED  }

 [/TYPE={CSV  }]
        {DB2  }
        {DB3  }
        {DB4  }
        {ODBC }
        {PC   }
        {SAS  }
        {STATA}
        {SYM  }
        {SLK  }
        {TAB  }
        {WKS  }
        {WK1  }
        {WK3  }
        {XLS  }

 [/VERSION={1}]
           {2}
           {3}
           {4}
           {5}
           {6}
           {7}
           {8}
           {9}
           {10}
           {11}
           {12}
           {13}
           {X}

 [/ENCODING={'Locale'}] [BOM={YES}]
            {'System'}       {NO }
            {'value' }

 [/FIELDNAMES [VALUE={NAMES }]
                     {LABELS}

 [/CELLS={VALUES}]
         {LABELS}

 [/TEXTOPTIONS [DELIMITER='char'] [QUALIFIER='char']
               [DECIMAL={DOT  }]
                        {COMMA}
               [FORMAT={PLAIN   }]]
                       {VARIABLE}

 [/EXCELOPTIONS SHEET='name']

 [/EDITION={INTERCOOLED}
           {SE         }

 [/PLATFORM={ALPHA}]
            {WINDOWS}
            {UNIX}

 [{/VALFILE=filename}]

 [/TABLE = 'database table name']

 [/SQL = 'SQL statement' [/SQL = 'SQL statement']]

 [/BULKLOADING]
  [METHOD={OFF  }]
          {BATCH}
          {ODBC }
  [BATCHSIZE=integer]
  [BINDING={ROW   }]
           {COLUMN}

[/ODBCOPTIONS VALLABELS={NONE   }]
                        {varlist}

 [{/APPEND }]
  {/REPLACE}

 [/RENAME=(old varlist=new varlist)[(...)]

 [/KEEP={ALL    }]
        {varlist}

 [/DROP=varlist]

 [/UNSELECTED=[{RETAIN}]
              {DELETE}

 [/MISSING={IGNORE}]
           {RECODE}

 [{/COMPRESSED  }]
  {/UNCOMPRESSED}
```

```
[/MAP]
```

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 14.0

- Value STATA added to list for TYPE subcommand.
- EDITION subcommand introduced for TYPE=STATA.
- SQL subcommand introduced.
- MISSING subcommand introduced.
- Field/column names specified on the RENAME subcommand can contain characters (for example, spaces, commas, slashes, plus signs) that are not allowed in IBM SPSS Statistics variable names.
- Continuation lines for connection strings on the CONNECT subcommand do not need to begin with a plus sign.

Release 15.0

- ENCRYPTED subcommand introduced.
- Value CSV added to list for TYPE subcommand.
- TEXTOPTIONS subcommand introduced for TYPE=CSV and TYPE=TAB.

Release 16.0

- VERSION=12 introduced for writing data in Excel 2007 XLSX format with TYPE=XLS.

Release 17.0

- UNENCRYPTED subcommand introduced.

Release 18.0

- VERSION=9 introduced for writing SAS 9+ files with TYPE=SAS.

Release 19

- ENCODING subcommand introduced.

Release 22.0

- BOM keyword added to ENCODING subcommand.

Release 23.0

- Support for versions 9-13 of Stata added to VERSION subcommand.
- BULKLOADING subcommand added.

Release 24.0

- VALUE keyword added to FIELDNAMES subcommand.
- ODBCOPTIONS subcommand added.
- EXCELOPTIONS subcommand added.

Release 25.0

- Support for version 14 of Stata added to VERSION subcommand. Stata 14 supports Unicode data.

Example

```
SAVE TRANSLATE OUTFILE='/data/sales.xlsx'
  /TYPE=XLS /VERSION=12.
```

Overview

SAVE TRANSLATE translates the active dataset into a file that can be used by other software applications. Supported formats are 1-2-3, Symphony, Multiplan, Excel, dBASE II, dBASE III, dBASE IV, tab-delimited text files, comma-delimited text files, SAS, Stata, SPSS/PC+ data files, and ODBC database sources. For ODBC database sources, it can also manipulate database tables.

Options

Variable Subsets. You can use the DROP and KEEP subcommands to specify variables to omit or retain in the resulting file.

Variable Names. You can use the RENAME subcommand to rename variables as they are copied to the external file format. For spreadsheets and text data files, use FIELDNAMES to include the variable names in the file.

Variable Map. To confirm the names and order of the variables that are saved in the resulting file, use the MAP subcommand. MAP displays the variables that are saved in the file next to their corresponding names in the active dataset.

Value Labels. For Excel spreadsheets and tab-delimited and comma-delimited text files, you can use the CELLS subcommand to export value labels rather than values in spreadsheet files. For SAS files, you can use the VALFILE subcommand to create a SAS syntax file containing your data's value labels. For Stata files, value labels for numeric variables are automatically saved as Stata value labels.

Limitations

- A maximum of 2,048 cases can be saved to 1-2-3 Release 1A; a maximum of 8,192 cases can be saved to 1-2-3 Release 2.0 or Symphony files.
- A maximum of 16,384 cases can be saved to Excel 2.1. For Excel 95, Excel 97 and Excel 2007, if the number of cases exceeds the single sheet maximum, multiple sheets are created in the workbook.
- A maximum of 256 variables can be saved to Excel 2.1 Excel 95, and Excel 97; a maximum of 16,000 variables can be saved to Excel 2007.
- A maximum of 65,535 cases and 32 variables can be saved to dBASE II, a maximum of one billion cases (subject to disk space availability) and 128 variables can be saved to dBASE III, and a maximum of one billion cases (subject to disk space availability) and 255 variables can be saved to dBASE IV.
- A maximum of 2,047 variables can be saved to Stata 5–6 and Intercooled Stata 7–14. A maximum of 32,767 variables can be saved to Stata SE 7–14.
- A maximum of 32,767 variables can be saved to SAS 6-8.

Operations

- The active dataset remains available after SAVE TRANSLATE is executed.
- If the active dataset contains more variables than the file can receive, SAVE TRANSLATE writes the maximum number of variables that the file can receive.

Spreadsheets

Variables in the active dataset become columns, and cases become rows in the spreadsheet file.

- If you specify FIELDNAMES, variable names become the first row.
- To save value labels instead of data values, use the CELLS subcommand. See the topic "CELLS Subcommand " on page 1757 for more information.
- System-missing values are translated to #NULL! in Excel files.

IBM SPSS Statistics formats are translated as follows:

Table 76. Format conversion for spreadsheets

IBM SPSS Statistics	1-2-3/Symphony	Excel
Number	Fixed	0.00;#,##0.00;...
COMMA	Comma	0.00;#,##0.00;...
DOLLAR	Currency	$#,##0_);...
DATE	Date	d-mmm-yy
TIME	Time	hh:mm:ss
String	Label	General

dBASE

Variables in the active dataset become fields, and cases become records in the database file.

- Characters that are allowed in variable names, but not in dBASE field names, are translated to colons in dBASE II and underscores in dBASE III and dBASE IV.
- Numeric variables containing the system-missing value are translated to **** in dBASE III and dBASE IV and 0 in dBASE II.
- The width and precision of translated numeric variables are taken from the print format—the total number of characters for the number is taken from the width of the print format, and the number of digits to the right of the decimal point is taken from the decimals in the print format. To adjust the width and precision, use the PRINT FORMATS command before using SAVE TRANSLATE. Values that cannot be converted to the given width and precision are converted to missing values.
- A maximum of 65,535 cases and 32 variables can be saved to dBASE II, a maximum of one billion cases (subject to disk space availability) and 128 variables can be saved to dBASE III, and a maximum of one billion cases (subject to disk space availability) and 255 variables can be saved to dBASE IV.

Variable formats are translated to dBASE formats as follows:

Table 77. Format conversion for dBASE files

IBM SPSS Statistics	dBASE
Number	Numeric
String	Character
Dollar	Numeric
Comma	Numeric

Comma-Delimited (CSV) Text Files

Variables in the active dataset become columns, and cases become rows in the text file.

- If you specify FIELDNAMES, variable names are written in the first row.
- To save value labels instead of data values, use the CELLS subcommand. See the topic “CELLS Subcommand ” on page 1757 for more information.
- Use the TEXTOPTIONS subcommand to override the default delimiter, qualifier, decimal indicator, or data value formatting. See the topic “TEXTOPTIONS Subcommand ” on page 1757 for more information.
- System-missing values are translated to a blank space in text files.

Tab-Delimited Text Files

Variables in the active dataset become columns, and cases become rows in the text file.

- If you specify FIELDNAMES, variable names are written to the first row.

- To save value labels instead of data values, use the CELLS subcommand. See the topic "CELLS Subcommand " on page 1757 for more information.
- Use the TEXTOPTIONS subcommand to override the default decimal indicator or data value formatting. See the topic "TEXTOPTIONS Subcommand " on page 1757 for more information.
- System-missing values are translated to a blank space in text files.

Note: Tab characters embedded in string values are preserved as tab characters in the tab-delimited file. No distinction is made between tab characters embedded in values and tab characters that separate values.

SAS Files

Data can be saved in numerous SAS data file formats. A SAS transport file is a sequential file that is written in SAS transport format and can be read by SAS with the XPORT engine and PROC COPY or the DATA step.

- Certain characters that are allowed in IBM SPSS Statistics variable names are not valid in SAS, such as @, #, and $. These illegal characters are replaced with an underscore when the data are exported.
- IBM SPSS Statistics variable names that contain multibyte characters (e.g., Japanese or Chinese characters) are converted to variable names of the general form *Vnnn*, where *nnn* is an integer value.
- IBM SPSS Statistics variable labels containing more than 40 characters are truncated when exported to a SAS v6 file.
- Where they exist, IBM SPSS Statistics variable labels are mapped to the SAS variable labels. If no variable label exists in the IBM SPSS Statistics data, the variable name is mapped to the SAS variable label.
- SAS allows only one value for missing, whereas IBM SPSS Statistics allows the definition of numerous missing values. As a result, all missing values in IBM SPSS Statistics are mapped to a single missing value in the SAS file.
- SAS data file names can be up to 32 characters in length. Blank spaces and non-alphanumeric characters other than the underscore ("_") are not allowed and names have to start with a letter or an underscore, numbers can follow.

The following table shows the variable type matching between the original data in IBM SPSS Statistics and the exported data in SAS.

Table 78. Format conversion for SAS files

IBM SPSS Statistics Variable Type	SAS Variable Type	SAS Data Format
Numeric	Numeric	12
Comma	Numeric	12
Dot	Numeric	12
Scientific Notation	Numeric	12
Date	Numeric	Date
Date (Time)	Numeric	Time
Date (Date-Time)	Numeric	DateTime
Dollar	Numeric	12
Custom Currency	Numeric	12
String	Character	$8

Stata Files

- Data can be written in Stata 5–14 format and in both Intercooled and SE format (version 7 or later).

- Data files that are saved in Stata 5 format can be read by Stata 4.
- The first 80 bytes of variable labels are saved as Stata variable labels.
- For Stata releases 4-8, the first 80 bytes of value labels for numeric variables are saved as Stata value labels. For Stata release 9 or later, the complete value labels for numeric variables are saved. Value labels are dropped for string variables, non-integer numeric values, and numeric values greater than an absolute value of 2,147,483,647.
- For versions 7 and later, the first 32 bytes of variable names in case-sensitive form are saved as Stata variable names. For earlier versions, the first eight bytes of variable names are saved as Stata variable names. Any characters other than letters, numbers, and underscores are converted to underscores.
- IBM SPSS Statistics variable names that contain multi-byte characters (for example, Japanese or Chinese characters) are converted to generic single-byte variable names. This only applies to Stata 4-13 data. IBM SPSS Statistics can save multi-byte variable names to Stata 14 data (because Stata 14 data is UTF-8 encoded).
- For versions 5–6 and Intercooled versions 7 and later, the first 80 bytes of string values are saved. For Stata SE 7–12, the first 244 bytes of string values are saved. For Stata SE 13 or later, complete string values are saved, regardless of length.
- For versions 5–6 and Intercooled versions 7 and later, only the first 2,047 variables are saved. For Stata SE 7 or later, only the first 32,767 variables are saved.
- Stata 14 supports Unicode data.

Table 79. How Stata variable type and format map to IBM SPSS Statistics type and format

IBM SPSS Statistics Variable Type	Stata Variable Type	Stata Data Format
Numeric	Numeric	g
Comma	Numeric	g
Dot	Numeric	g
Scientific Notation	Numeric	g
Date*, Datetime	Numeric	D_m_Y
Time, DTime	Numeric	g (number of seconds)
Wkday	Numeric	g (1–7)
Month	Numeric	g (1–12)
Dollar	Numeric	g
Custom Currency	Numeric	g
String	String	s

*Date, Adate, Edate, SDate, Jdate, Qyr, Moyr, Wkyr

SPSS/PC+ System Files

Variables are saved as they are defined. The resulting file is given the extension *.sys* if no extension is explicitly specified. The dictionary is saved so that labels, formats, missing value specifications, and other dictionary information are preserved. This format is available only on Windows operating systems.

ODBC Database Sources

The following rules apply when writing to a database with TYPE=ODBC:

- If you specify a table name that does not exist in the database, a new table with that name is created.
- If any case cannot be stored in the database for any reason, an error is returned. Therefore, either all cases are stored or no cases are stored.

- At insert time, a check is performed to see whether the value that is being stored in a column is likely to cause an overflow. If so, the user is warned about the overflow and is informed that a *SYSMIS* is stored instead.
- If any variable names in the active dataset contain characters that are not allowed by the database, they are replaced by an underscore. If this process causes a duplicate variable name, a new variable name is generated.

  By default, IBM SPSS Statistics variable formats are mapped to database field types based on the following general scheme. Actual database field types may vary, depending on the database.

  Table 80. Format conversion for databases

IBM SPSS Statistics Variable Format	Database Field Type
Numeric	Float or Double
Comma	Float or Double
Dot	Float or Double
Scientific Notation	Float or Double
Date	Date or Datetime or Timestamp
Datetime	Datetime or Timestamp
Time, DTime	Float or Double (number of seconds)
Wkday	Integer (1–7)
Month	Integer (1–12)
Dollar	Float or Double
Custom Currency	Float or Double
String	Char or Varchar

- For database Datetime/Timestamp formats that don't allow fractional seconds, fraction seconds in IBM SPSS Statistics Datetime values are truncated.
- The default width of Char database data types for IBM SPSS Statistics string variables is the defined width of the string variable. If the defined width of the string exceeds the maximum width allowed by the database, the values are truncated to the maximum width, and a warning is issued.
- You can override the default settings by using the SQL subcommand to create or update tables. See the topic "SQL Subcommand " on page 1760 for more information.

TYPE Subcommand

TYPE indicates the format of the resulting file.

- TYPE can be omitted for spreadsheet files if the file extension that is named on OUTFILE is the default for the type of file that you are saving.
- TYPE with the keyword DB2, DB3, or DB4 is required for translating to dBASE files.
- TYPE takes precedence over the file extension.

ODBC. *Database accessed via ODBC.*

XLS. *Excel.*

CSV. *Comma-delimited text data files.*

TAB. *Tab-delimited text data files.*

SAS. *SAS data files and SAS transport files.*

STATA. *Stata data files.*

DB2. *dBASE II.*

DB3. *dBASE III or dBASE III PLUS.*

DB4. *dBASE IV.*

WK1. *1-2-3 Release 2.0.*

WKS. *1-2-3 Release 1.4.*

SYM. *Symphony releases.*

SLK. *Multiplan (symbolic format).*

PC. *SPSS/PC+ system files.* This format is available only on Windows operating systems.

Example

```
SAVE TRANSLATE OUTFILE='PROJECT.XLS' /TYPE=XLS (/VERSION=8).
```

- SAVE TRANSLATE translates the active dataset into the Excel workbook file named *PROJECT.XLS*.

VERSION Subcommand

VERSION specifies the file version for multiversion applications. For example, this subcommand is necessary to differentiate between different versions of Excel.

- VERSION is required for all applications listed below, except Excel.
- If VERSION is not specified with TYPE=XLS, the Excel file is saved in version 2.1 format.

Table 81. Version numbers and applications

Version	Application
1	Symphony Release 1.0
2	Excel 2.1, Symphony Release 2.0, dBASE II
3	dBASE III or dBASE III PLUS
4	dBASE IV
5	Excel 95 Workbook, Stata 4–5
6	SAS v6, Stata 6
7	SAS v7–8, Stata 7
8	Excel 97–2003 Workbook, Stata 8
9	SAS 9, Stata 9
10	Stata 10
11	Stata 11
12	Excel 2007 Workbook, Stata 12
13	Stata 13
14	Stata 14
X	SAS transport file

Example

```
SAVE TRANSLATE OUTFILE='STAFF.XLS' /TYPE=XLS /VERSION=8.
```

- SAVE TRANSLATE creates an Excel workbook file in the version 97 format.

ENCODING Subcommand

The ENCODING subcommand specifies the character encoding for SAS, Stata, tab-delimited text, and CSV data files.

- The ENCODING subcommand is only valid with TYPE=SAS, TYPE=STATA, TYPE=TAB, and TYPE=CSV.
- The subcommand name is preceded by a slash and followed by an optional equals sign and a quoted value.
- For SAS and Stata files, the quoted value can be LOCALE or SYSTEM or one of the values in the Encodings column in Character Encoding table. For SAS 9 files, the value can also be UTF8.
- For tab-delimited text files and CSV files, the quoted value can be LOCALE, UTF8, UTF16, UTF16BE, UTF16LE, a numeric Windows code page value (for example, '1252'), or an IANA code page value (for example, 'iso8859-1' or cp1252)
- For SAS 9, tab-delimited text, and CSV files, the default is UTF8 in Unicode mode and LOCALE in code page mode. For Stata and earlier releases of SAS, the default is always LOCALE.
- The ENCODING setting also applies to the value labels file specified on the optional VALFILE subcommand for TYPE=SAS.

Example

```
SAVE TRANSLATE
  /OUTFILE='/data/sasdata.sas7bdat'
  /VALFILE='/data/saslabels.sas'
  /TYPE=SAS /VERSION=7 /PLATFORM=WINDOWS
  /ENCODING='Windows-1252'.
```

BOM Keyword

By default, files encoded in any of the UTF formats include a byte order mark (BOM). Some applications cannot interpret the byte order mark. You can use the BOM keyword to suppress the byte order mark.

BOM=YES
Include the byte order mark in UTF files. This option is the default.

BOM=NO
No not include the byte order mark in UTF files.

Character encoding values for SAS and Stata

Table 82. Character Encoding

Character Set	Encoding
IBM SPSS Statistics Locale	Locale
Operating System Locale	System
Western	ISO-8859-1
Western	ISO-8859-15
Western	IBM850
Western	Windows-1252
Celtic	ISO-8859-14
Greek	ISO-8859-7
Greek	Windows-1253
Nordic	ISO-8859-10

Table 82. Character Encoding (continued)	
Character Set	**Encoding**
Baltic	Windows-1257
Central European	IBM852
Central European	ISO-8859-2
Cyrillic	IBM855
Cyrillic	ISO-8859-5
Cyrillic	Windows-1251
Cyrillic/Russian	CP-866
Chinese Simplified	GBK
Chinese Simplified	ISO-2022-CN
Chinese Traditional	Big5
Chinese Traditional	EUC-TW
Japanese	EUC-JP
Japanese	ISO-2022-JP
Japanese	Shift-JIS
Korean	EUC-KR
Thai	Windows-874
Turkish	IBM857
Turkish	ISO-8859-9
Arabic	Windows-1256
Arabic	IBM864
Hebrew	ISO-8859-8
Hebrew	Windows-1255
Hebrew	IBM862

OUTFILE Subcommand

OUTFILE assigns a name to the file to be saved. The only specification is the name of the file. On some operating systems, file specifications should be enclosed in quotes.

Example

```
SAVE TRANSLATE OUTFILE='STAFF.DBF'/TYPE=DB3.
```

- SAVE TRANSLATE creates a dBASE III file called *STAFF.DBF*. The TYPE subcommand is required to specify the type of dBASE file to save.

FIELDNAMES Subcommand

FIELDNAMES writes variable names to the first row in spreadsheets and tab-delimited and comma-delimited text files.

- By default, variable names are not included in the file.
- For Excel files, version 8 or later, you can also select to write either variable names or variable labels.

VALUE=NAMES
Writes variable names. This setting is the default setting if FIELDNAMES is used without a VALUE keyword.

VALUE=LABELS
Writes variable labels. If a variable does not have a defined variable label, the variable name is used.

Example

```
SAVE TRANSLATE OUTFILE='/data/csvfile.csv' /TYPE=CSV /FIELDNAMES.
```

CELLS Subcommand

For TYPE=XLS (/VERSION=8), TYPE=TAB, and TYPE=CSV, the CELLS subcommand specifies whether data values or value labels are saved.

CELLS=VALUES. *Saves data values.* This is the default.

CELLS=LABELS. *Saves value labels instead of values.* For values with no corresponding value label, the value is saved.

Example

```
SAVE TRANSLATE OUTFILE='/data/sales.xls'
  /TYPE=XLS /VERSION=8
  /CELLS=LABELS.
```

TEXTOPTIONS Subcommand

For TYPE=CSV, the TEXTOPTIONS subcommand controls the delimiter and qualifier. For TYPE=CSV and TYPE=TAB, it controls the decimal indicator and the data value formatting. For all other types, this subcommand is ignored.

DELIMITER="char". *Specifies the delimiter (separator) between values.* The value must be a single character (single byte), enclosed in quotes (single or double quotes). The default is either a comma or semicolon, based on the decimal indicator. If the decimal indicator is a period, then the default separator is a comma. If the decimal indicator is a comma, then the default separator is a semicolon. The value cannot be the same as the QUALIFIER or DECIMAL value. This setting only applies for TYPE=CSV. For other types, it is ignored.

QUALIFIER="char". *Specifies the qualifier to use to enclose values that contain the delimiter character.* The default is a double quote. The value must be a single character (single byte), enclosed in quotes (single or double quotes). Values that contain the qualifier character will also be enclosed by the qualifier and qualifiers within the value will be doubled. The value cannot be the same as the DELIMITER or DECIMAL value. This setting only applies for TYPE=CSV. For other types, it is ignored.

DECIMAL=DOT|COMMA. *Specifies the value to use as the decimal indicator for numeric values.* The default is the current IBM SPSS Statistics decimal indicator. This setting has no effect on the decimal indicator used with FORMAT=VARIABLE for comma, dollar, custom currency, and dot formats. (Comma and dollar always use the period as the decimal indicator; dot always uses the comma as the decimal indicator, custom currency formats honor the decimal indicator defined in the format) The value cannot be the same as the DELIMITER or QUALIFER value. (If DECIMAL=DOT, the default delimiter is a comma; if DECIMAL=COMMA, the default delimiter is a semicolon.)

- **DOT.** *Use a period (.) as the decimal indicator.*
- **COMMA.** *Use a comma (,) as the decimal indicator.*

The default decimal indicator is the current IBM SPSS Statistics decimal indicator. The current IBM SPSS Statistics decimal indicator can be set with SET LOCALE (which uses the OS locale decimal indicator) or SET DECIMAL and can be shown with SHOW DECIMAL. The TEXTOPTIONS DECIMAL setting overrides the default decimal indicator in the current SAVE TRANSLATE command but does not change the default/current IBM SPSS Statistics decimal indicator, and it has no effect on the determination of the default delimiter for TYPE=CSV.

FORMAT=PLAIN|VARIABLE. *Specifies the data formats to use when writing out data values.*

- **PLAIN.** *Remove all "extraneous" formatting from numeric values.* This is the default. For example, a dollar format value of $12,345.67 is written simply as 12345.67 (or 12345,67, depending on the decimal indicator). The number of decimal positions for each value is the number of decimal positions necessary to preserve the entire value (and different values of the same variable may have a different number of decimal positions). Dates are written in the general format mm/dd/yyyy. Times are written in the general format hh:mm:ss.
- **VARIABLE.** *Use the print format for each variable to write data values.* For example, a value of 12345.67 with a print format of DOLLAR10.2 is written as $12,345.67 (and is qualified with double quotes or the user-specified qualifier if a comma is the value delimiter).

Print formats can be set with the PRINT FORMATS or FORMATS commands and displayed with DISPLAY DICTIONARY.

EXCELOPTIONS subcommand

For XLSX Excel files (TYPE=XLS, VERSION=12), you can use EXCELOPTIONS to specify a sheet name.

```
/EXCELOPTIONS SHEET='name'
```

- The name must be enclosed in quotation marks.
- The name cannot exceed 31 characters.
- The name cannot include the following characters: / \ ? [ ].
- If the /APPEND subcommand is also specified, the sheet is appended to the file. If a sheet with that name exists in the file, it is replaced.

EDITION Subcommand

The EDITION subcommand specifies the edition for Stata files. EDITION only applies to Stata version 7 or later.

INTERCOOLED. Saves the data file in Stata Intercooled format. This setting is the default. Only the first 2,047 variables are saved, and only the first 80 bytes of string values are saved.

SE. Saves the data file in Stata SE format. The first 32,767 variables are saved, and the first 244 bytes of string values are saved.

Example

```
SAVE TRANSLATE OUTFILE='/data/newdata.dta' /TYPE=STATA
  /VERSION=8 /EDITION=SE.
```

This subcommand is ignored for any file types other than Stata version 7 or later.

PLATFORM Subcommand

The PLATFORM subcommand is required for all SAS file types, with the exception of SAS transport files. Enter the keyword corresponding to the platform for which the target SAS file is intended. There is no default value for this command. Choose from the following keywords: WINDOWS, ALPHA, UNIX.

Example

```
SAVE TRANSLATE OUTFILE='STAFF.SD7' /TYPE=SAS /VERSION=7
/PLATFORM=WINDOWS /VALFILE='LABELS.SAS'.
```

VALFILE Subcommand

The VALFILE subcommand, which is available only for SAS file formats, excluding the SAS transport file format, saves value labels to a SAS syntax file. This syntax file is used in conjunction with the saved SAS data file to re-create value labels in SAS.

- The syntax file has an *.sas* file extension.

Example

```
SAVE TRANSLATE OUTFILE='STAFF.SD7' /TYPE=SAS /VERSION=7
/PLATFORM=WINDOWS /VALFILE='LABELS.SAS'.
```

- SAVE TRANSLATE saves the current data as a SAS v7 file and creates a SAS syntax file named *LABELS.SAS*, which contains the value labels for the data.

ODBC Database Subcommands

The CONNECT, ENCRYPTED, TABLE, SQL, BULKLOADING, ODBCOPTIONS, and APPEND subcommands are used with TYPE=ODBC.

CONNECT Subcommand

CONNECT identifies the database name and other parameters for TYPE=ODBC.

- The database must already exist. You cannot create a new database with SAVE TRANSLATE (although you can create new tables in an existing database).
- If you are unsure what the connect string should be, you can use the Export to Database Wizard to generate SAVE TRANSLATE command syntax. (See Export to Database Wizard for more information.) If the connection string is specified on multiple lines, each line must be enclosed in quotes.

Example

```
SAVE TRANSLATE
   /TYPE=ODBC
   /CONNECT
   'DSN=MS Access Database;DBQ=C:\examples\data\dm_demo.mdb;'
   'DriverId=25;FIL=MS Access;MaxBufferSize=2048;PageTimeout=5;'
   /TABLE='mytable'.
```

ENCRYPTED and UNENCRYPTED Subcommands

The ENCRYPTED subcommand indicates that the password in the CONNECT string is encrypted. The default is UNENCRYPTED. (Both the Database Wizard and Export to Database Wizard generate encrypted passwords.)

The ENCRYPTED and UNENCRYPTED subcommands have no additional specifications.

TABLE Subcommand

TABLE identifies the table name for TYPE=ODBC.

- Table names should be enclosed in quotes.
- You can replace an existing table or append new records to an existing table.
- You can create a new table in the database by specifying a table name that does not currently exist in the database.

Example

```
SAVE TRANSLATE
   /TYPE=ODBC
   /CONNECT
   'DSN=MS Access Database;DBQ=C:\examples\data\dm_demo.mdb;'
   'DriverId=25;FIL=MS Access;MaxBufferSize=2048;PageTimeout=5;'
   /TABLE='mytable'.
```

SQL Subcommand

The SQL subcommand provides the ability to issue any SQL directives that are needed in the target database. It can be used, for example, to define joins or alter table properties in the database to include new columns or modify column properties.

- Each SQL statement must be enclosed in quotes.
- You can use multiple lines for a single SQL statement by using multiple quoted strings connected with plus signs (the standard string concatenation symbol).
- Each quoted line cannot exceed 256 characters.
- Multiple SQL statements can be included by using multiple SQL subcommands.
- Table and field specifications in SQL statements refer to tables and fields available in the database, not datasets and variables available in the IBM SPSS Statistics session (although in many cases the names may be the same).
- Regardless of the position of the SQL subcommand, the SQL statements are executed last, after all other actions executed by the SAVE TRANSLATE command.

Example: Adding New Columns to an Existing Table

```
SAVE TRANSLATE /TYPE=ODBC
 /CONNECT=
 'DSN=MS Access Database;DBQ=C:\examples\data\dm_demo.mdb;'
 'DriverId=25;FIL=MS Access;MaxBufferSize=2048;PageTimeout=5;'
  /TABLE = 'NewColumn'
  /KEEP ID income_score
  /REPLACE
  /SQL='ALTER TABLE CombinedTable ADD COLUMN income_score REAL'
  /SQL='UPDATE CombinedTable INNER JOIN NewColumn ON ' +
  'CombinedTable.ID=NewColumn.ID SET ' +
  'CombinedTable.income_score=NewColumn.income_score'.
```

- The TABLE, KEEP, and REPLACE subcommands create or replace a table named *NewColumn* that contains two variables: a key variable (*ID*) and a computed score (*income_score*).
- The first SQL subcommand, specified on a single line, adds a column to an existing table that will contain values of the computed variable *income_score*. At this point, all we have done is create an empty column in the existing database table, and the fact that both database tables and the active dataset use the same name for that column is merely a convenience for simplicity and clarity.
- The second SQL subcommand, specified on multiple lines with the quoted strings concatenated with plus signs, adds the *income_score* values from the new table to the existing table, matching rows (cases) based on the value of the key variable *ID*.

The end result is that an existing table is modified to include a new column containing the values of the computed variable.

Example: Specifying Data Types for a New Table

```
SAVE TRANSLATE /TYPE=ODBC
  /CONNECT='DSN=MS Access Database;DBQ=c:\temp\temp.mdb;DriverId=25;FIL=MS'+
  ' Access;MaxBufferSize=2048;PageTimeout=5;'
  /TABLE=tempTable    /REPLACE
  /SQL='CREATE TABLE NewTable(numVar double, intVar integer, dollarVar currency)'
  /SQL='INSERT INTO NewTable(numVar, intVar, dollarVar) SELECT * FROM tempTable'
  /SQL='DROP TABLE tempTable'.
```

- The TABLE subcommand creates a new table that contains variables in the active dataset with the default database data types. In this example, the original variables have IBM SPSS Statistics variable formats of F8.2, F4.0, and Dollar10.2 respectively, but the default database type for all three is double.
- The first SQL subcommand creates another new table with data types explicitly defined for each field. At this point, this new table contains no data.

- The second SQL subcommand inserts the data from the *tempTable* into *NewTable*. This does not affect the data types previously defined for *NewTable*, so *intVar* will have a data type of integer and *dollarVar* will have a data type of currency.
- The last SQL subcommand deletes *tempTable*, since it is no longer needed.

BULKLOADING Subcommand

The BULKLOADING subcommand controls settings for bulk loading to a database. Bulk loading can make the export much faster, particularly for large data files.

METHOD=OFF | BATCH | ODBC
Specifies the bulk loading method.

- OFF. Bulk loading is turned off. Records are committed to the database one at a time. This option is the default setting.
- BATCH. Commits records to the database in the batch size value that is specified for the BATCHSIZE keyword.
- ODBC. Uses the ODBC binding method to commit records in the specified batch size. This option is available only if the database supports ODBC binding. This option is not available on Mac OS.

BATCHSIZE=integer
Specifies the batch size. The value must be a positive integer. The default value is 10000.

BINDING=ROW | COLUMN
For ODBC binding, specifies row or column binding. This setting is ignored unless METHOD=ODBC.

- ROW. Row-wise binding typically improves speed compared to the use of parameterized inserts that insert data on a record-by-record basis. This option is the default setting.
- COLUMN. Column-wise binding improves performance by binding each database column to an array of *n* values.

ODBCOPTIONS subcommand

You can use ODBCOPTIONS to export defined value labels instead of data values.

```
ODBCOPTIONS VALLABELS=NONE | varlist
```

- The default setting is VALLABELS=NONE.
- If a variable list is specified, the value label text is exported instead of the data values.
- For values that do not have a defined value label, the data value is exported as a text string.
- This option is ignored for date format variables and variables that do not have any defined value labels.
- This option is only available for creating a new table, replacing an existing table, and adding new fields to an existing table. You cannot use this option for appending rows to an existing table or modifying the content of existing fields in a table.

APPEND Subcommand

TYPE=ODBC

For TYPE=ODBC, APPEND appends rows (cases) to an existing database table after type and variable name validations. There must be a matching column in the table for each IBM SPSS Statistics variable. If a column that can correctly store a variable is not found, a failure is returned. If the table contains more columns than the number of IBM SPSS Statistics variables, the command still stores the data in the table. The variable names and column names must match exactly. A variable can be stored in a column as long as the column type is a type that can store values of the variable type. A column of any numeric type (short integer, integer, float, double, etc.) is valid for a numeric variable, and a column of any character type is valid for a string variable.

Note: APPEND can only add rows to a table, not columns. If you want to add columns (fields) to a database table, see the SQL subcommand.

TYPE=XLS

For TYPE=XLS, APPEND can append a new sheet to an existing file.

- APPEND is valid only for VERSION=12.
- A sheet name must be specified on the EXCELOPTIONS subcommand.

REPLACE Subcommand

REPLACE gives permission to overwrite an existing file (or database table) of the same name. REPLACE takes no further specifications.

- SAVE TRANSLATE will not overwrite an existing file without an explicit REPLACE subcommand. The default behavior is to not overwrite.
- For database tables (TYPE=ODBC), APPEND and REPLACE are mutually exclusive.

Note: For database tables, REPLACE destroys an existing table and replaces it with the data written by the SAVE TRANSLATE command, which may lead to loss of database-specific information, such as the designation of key variables (fields) and data formats. If you want to update an existing table with new values in some existing rows or by adding additional fields to the table, see the SQL subcommand.

UNSELECTED Subcommand

UNSELECTED determines whether cases that were excluded on a previous FILTER or USE command are to be retained or deleted. The default is RETAIN. The UNSELECTED subcommand has no effect when the active dataset does not contain unselected cases.

RETAIN. *Retain the unselected cases*. All cases in the active dataset are saved. This setting is the default when UNSELECTED is specified by itself.

DELETE. *Delete the unselected cases*. Only cases that meet the FILTER or USE criteria are saved.

DROP and KEEP Subcommands

Use DROP or KEEP to include only a subset of variables in the resulting file. DROP specifies a set of variables to exclude. KEEP specifies a set of variables to retain. Variables that are not specified on KEEP are dropped.

- Specify a list of variable, column, or field names separated by commas or spaces.
- KEEP does not affect the order of variables in the resulting file. Variables are kept in their original order.
- Specifying a variable that is not in the active dataset or that has been dropped because of a previous DROP or KEEP subcommand results in an error, and the SAVE command is not executed.

Example

```
SAVE TRANSLATE OUTFILE='ADDRESS.DBF' /TYPE=DB4 /DROP=PHONENO, ENTRY.
```

- SAVE TRANSLATE creates a dBASE IV file named *ADDRESS.DBF*, dropping the variables *PHONENO* and *ENTRY*.

RENAME Subcommand

RENAME changes the names of variables as they are copied into the resulting file.

- The specification on RENAME is a list of old variable names followed by an equals sign and a list of new variable names. The same number of variables must be specified on both lists. The keyword TO can be used in the first list to refer to consecutive variables in the active dataset and can be in the second list to generate new variable names. The entire specification must be enclosed in parentheses.
- Alternatively, you can specify each old variable name individually, followed by an equals sign and the new variable name. Multiple sets of variable specifications are allowed. The parentheses around each set of specifications are optional.
- New names cannot exceed 64 bytes. Characters not allowed in IBM SPSS Statistics variable names can be used in new names for the target file, but if the name contains special characters (e.g., spaces, commas, slashes, plus signs) the name must be enclosed in quotes.
- RENAME does not affect the active dataset. However, if RENAME precedes DROP or KEEP, variables must be referred to by their new names on DROP or KEEP.
- Old variable names do not need to be specified according to their order in the active dataset.
- Name changes take place in one operation. Therefore, variable names can be exchanged between two variables.
- Multiple RENAME subcommands are allowed.

Examples

```
SAVE TRANSLATE OUTFILE='STAFF.SYM' VERSION=2 /FIELDNAMES
  /RENAME AGE=AGE88 JOBCAT=JOBCAT88.
```

- RENAME renames the variable *AGE* to *AGE88* and renames *JOBCAT* to *JOBCAT88* before they are copied to the first row of the spreadsheet.

```
SAVE TRANSLATE OUTFILE='STAFF.SYM' VERSION=2 /FIELDNAMES
  /RENAME (AGE JOBCAT=AGE88 JOBCAT88).
```

- The name changes are identical to the changes in the previous example: *AGE* is renamed to *AGE88*, and *JOBCAT* is renamed to *JOBCAT88*. The parentheses are required with this method.

MISSING Subcommand

The MISSING subcommand controls the treatment of user-missing values. By default, user-missing values are treated as regular valid values in the target file.

IGNORE. *Treat user-missing values as regular valid values*. This setting is the default.

RECODE. *Recode numeric user-missing values to system-missing, and recode string user-missing values to blank spaces*.

COMPRESSED and UNCOMPRESSED Subcommands

For TYPE=PC, COMPRESSED saves the file in compressed form. UNCOMPRESSED saves the file in uncompressed form.

- The only specification is the keyword COMPRESSED or UNCOMPRESSED.
- The COMPRESSED and UNCOMPRESSED keywords are valid only for TYPE=PC.

MAP Subcommand

MAP displays a list of the variables in the resulting file and their corresponding names in the active dataset.

- The only specification is the keyword MAP.
- Multiple MAP subcommands are allowed. Each MAP subcommand maps the results of subcommands that precede it but not the results of subcommands that follow it.

Example

```
GET FILE=HUBEMPL.
SAVE TRANSLATE OUTFILE='STAFF.SYM' /VERSION=2 /FIELDNAMES
  /RENAME=(AGE=AGE88)(JOBCAT=JOBCAT88).
```

- MAP is specified to confirm that the variable *AGE* is renamed to *AGE88* and *JOBCAT* is renamed to *JOBCAT88*.

SAVETM1

```
SAVETM1

  /CONNECTION PMSYSURL='string'
   SERVER='string'
  [NAMESPACE='string']
   USERNAME='string'
   PASSWORD='string' ENCRYPTED={YES**}
                               {NO   }

  /CUBE NAME='string'

  /MAPPINGS  MEASURE_FIELDS = fieldname (MAPTO='element name' NEW={YES |
NO}) ...
             [DIMENSION_FIELDS = fieldname (MAPTO='dimension name') ... ]
             [SLICES = 'dimension name' (LEAF='element name') ... ]
```

**Default if keyword omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 22.0.0.1

- Command introduced.

Example

```
SAVETM1
  /CONNECTION PMSYSURL='http://mypmsystem:9510/pmhub/pm'
   SERVER='myserver'
   USERNAME='tm1user'
   PASSWORD='#o/B!a.],-./.-#('  ENCRYPTED=YES

  /CUBE NAME='mycube'

  /MAPPINGS MEASURE_FIELDS=PredictedSales (MAPTO='Predicted Sales' NEW=YES)

            DIMENSION_FIELDS=Region (MAPTO='Region')
                             Product (MAPTO='Product')
                             Month (MAPTO='Month')

            SLICES='Store Type' (LEAF='Department Store').
```

Overview

SAVETM1 saves data from the active dataset to an IBM Cognos TM1 cube. This feature is particularly useful when you import data from TM1, transform or score the data in SPSS Statistics, and then want to export the results back to TM1. You must have access to a TM1 Performance Management system and an associated TM1 server to use this feature.

Important: To enable the exchange of data between SPSS Statistics and TM1, you must copy the following three processes from SPSS Statistics to the TM1 server: `ExportToSPSS.pro`, `ImportFromSPSS.pro`, and `SPSSCreateNewMeasures.pro`. To add these processes to the TM1

server, you must copy them to the data directory of the TM1 server and restart the TM1 server. These files are available from the common/scripts/TM1 directory under the SPSS Statistics installation directory.

- All subcommands are required.
- The TM1 cube to which you export must include a measure dimension.
- System-missing and user-missing values of fields that are mapped to elements in the measure dimension of the TM1 cube are ignored on export. The associated cells in the TM1 cube are unchanged.
- Fields with a value of zero, that are mapped to elements in the measure dimension, are exported as a valid value.

Related information

"Overview " on page 815

CONNECTION subcommand

The required CONNECTION subcommand specifies the IBM Cognos TM1 Performance Management system and associated TM1 server and any credentials that are required to access data on the TM1 server.

PMSYSURL
Specifies the URL for the Performance Management system where the TM1 server you want to connect to is installed. The Performance Management system is defined as a single URL for all TM1 servers. From this URL, all TM1 servers that are installed and running on your environment can be discovered and accessed. The keyword is followed by an equals sign (=) and a URL enclosed in quotation marks.

SERVER
Specifies the TM1 server. The keyword is followed by an equals sign (=) and the name of the server enclosed in quotation marks.

MODE=SETCREDENTIALS
Specifies that credentials are explicitly specified. This setting is the default. For SETCREDENTIALS, you must specify values for USERNAME and PASSWORD.

USERNAME
The TM1 user name with which to log on to the server. The keyword is followed by an equals sign (=) and a value that is enclosed in quotation marks.

PASSWORD
The password that is associated with the specified user name. The keyword is followed by an equals sign (=) and a value that is enclosed in quotation marks. By default, the password is assumed to be encrypted. If you do not want the password to be treated as encrypted, then specify ENCRYPTED=NO.

NAMESPACE
The IBM Cognos Business Intelligence security authentication provider that is used to log on to the server. This setting is only required when the server uses authentication mode 5 (IBM Cognos security). The keyword is followed by an equals sign (=) and a value that is enclosed in quotation marks. The keyword is ignored if the authentication mode is anything other than mode 5.

MODE=STOREDCREDENTIAL
Specifies to use the login information from a stored credential. To use a stored credential, you must be connected to the IBM SPSS Collaboration and Deployment Services Repository that contains the credential. For STOREDCREDENTIAL, you must specify the CREDENTIAL keyword, followed by an equals sign (=) and the name of the credential in quotation marks.

CUBE subcommand

The required CUBE subcommand specifies the IBM Cognos TM1 cube where the data is saved. The subcommand name is followed by the keyword NAME, an equals sign (=), and the name of the cube enclosed in quotation marks. The specified cube must include a measure dimension.

MAPPINGS subcommand

The required MAPPINGS subcommand defines the mappings from the fields in the active dataset to the dimensions and measures in the specified IBM Cognos TM1 cube.

- The MEASURE_FIELDS keyword is required.
- For each regular dimension in the cube, you must either map a field in the active dataset to the dimension or specify a slice for the dimension.

MEASURE_FIELDS
Required keyword that specifies the mappings from fields in the active dataset to elements in the measure dimension of the cube.

- The keyword is followed by an equals sign (=) and a list of fields.
- The form of a field specification is `fieldname (MAPTO='element name' NEW={YES | NO})`, where *fieldname* is the name of a field in the active dataset and *'element name'* is the name of the element in the measure dimension that it is mapped to, enclosed in quotation marks. The NEW keyword specifies whether the element is a new element in the measure dimension: YES specifies a new element, NO specifies an existing element.

  Note:

  – Only numeric fields can be specified on the MEASURE_FIELDS keyword.
  – Values that are exported to an existing element in the measure dimension overwrite the associated cells in the TM1 cube.

DIMENSION_FIELDS
Optional keyword that specifies the mappings from fields in the active dataset to regular dimensions in the cube.

- The keyword is followed by an equals sign (=) and a list of fields.
- The form of a field specification is `fieldname (MAPTO='dimension name')`, where *fieldname* is the name of a field in the active dataset and *'dimension name'* is the name of the dimension it is mapped to, enclosed in quotation marks.

  Note:

  – Only string fields can be specified on the DIMENSION_FIELDS keyword.
  – Cases with field values that do not match a leaf element in the specified dimension are not exported. In that regard, you can export only to leaf elements.

SLICES
Optional keyword that specifies slices for one or more regular dimensions. A slice specifies a single leaf element of a dimension so that all exported cases are associated with the specified leaf element.

- The keyword is followed by an equals sign (=) and a list of fields.
- The form of a field specification is `'dimension name' (LEAF='element name')`, where *'dimension name'* is the name of a dimension and *'element name'* is the name of the leaf element that defines the slice. Both values must be enclosed in quotation marks.

Example

```
/MAPPINGS MEASURE_FIELDS=PredictedSales (MAPTO='Predicted Sales' NEW=YES)

          DIMENSION_FIELDS=Region (MAPTO='Region')
                           Product (MAPTO='Product')
                           Month (MAPTO='Month')

          SLICES='Store Type' (LEAF='Department Store')
```

SCRIPT

```
SCRIPT 'filename' [(quoted string)]  [PYTHONVERSION = {3**}]
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 16.0

- Scripts run from the SCRIPT command now run synchronously with the command syntax stream.

Release 17.0

- Ability to run Python scripts introduced.

Release 24.0

- PYTHONVERSION keyword introduced.

Release 27.0

- PYTHONVERSION = 2 was deprecated.

Overview

SCRIPT runs a script to customize the program or automate regularly performed tasks. You can run a Basic script or a Python script.

Basic specification

The basic specification is keyword SCRIPT with a filename. The filename is required. The optional quoted string, enclosed in parentheses, can be passed to the script.

PYTHONVERSION keyword

The PYTHONVERSION keyword specifies the Python 3 processor to run the script.

Operations

- SCRIPT runs the specified script, and works in both local and distributed mode. The effect is the same as opening the script file in the script editor (Basic or Python) and running it from there.

Limitations

- Python scripts run from the SCRIPT command cannot execute command syntax.
- The SCRIPT command is not supported in the IBM SPSS Statistics Batch Facility (available with IBM SPSS Statistics Server).

Running Basic Scripts That Contain Syntax Commands

If a Basic script that is run via the SCRIPT command contains syntax commands, those commands must be run asynchronously. To run commands asynchronously, set the bSync parameter of the ExecuteCommands method to False, as in:

```
Dim objSpssApp As spsswinLib.Application16
Dim strCommands As String
Set objSpssApp=CreateObject("SPSS.Application16")
```

```
' Construct and execute syntax commands:

strCommands = "GET FILE = '/data/bank.sav'. " & vbCr
strCommands = strCommands & "Display Dictionary."
objSpssApp.ExecuteCommands strCommands, False
```

SEASON

SEASON is available in SPSS Statistics Professional Edition or the Forecasting option.

```
SEASON VARIABLES= series names

 [/MODEL={MULTIPLICATIVE**}]
         {ADDITIVE        }

 [/MA={EQUAL   }]
      {CENTERED}

 [/PERIOD=n]

 [/APPLY [='model name']]
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
SEASON VARIABLES = VARX
  /MODEL=ADDITIVE.
```

Overview

SEASON estimates multiplicative or additive seasonal factors for time series by using any specified periodicity. SEASON is an implementation of the Census Method I, otherwise known as the ratio-to-moving-average method. See [77] and [78].

Options

Model Specification. You can specify either a multiplicative or additive model on the MODEL subcommand. You can specify the periodicity of the series on the PERIOD subcommand.

Computation Method. Two methods of computing moving averages are available on the MA subcommand for handling series with even periodicities.

Statistical Output. Specify TSET PRINT=BRIEF to display only the initial seasonal factor estimates. TSET PRINT=DETAILED produces the same output as the default.

New Variables. To evaluate the displayed averages, ratios, factors, adjusted series, trend-cycle, and error components without creating new variables, specify TSET NEWVAR=NONE prior to SEASON. This can result in faster processing time. To add new variables without erasing the values of previous Forecasting-generated variables, specify TSET NEWVAR=ALL. This saves all new variables that are generated during the current session to the active dataset and may require extra processing time. When the default (TSET NEWVAR=CURRENT) is in effect, only variables from the current procedure are saved to the active dataset, and the suffix #n is used to distinguish variables that are generated by different series. TSET MXNEWVAR specifies the maximum number of new variables that can be generated by a procedure. The default is 60. The order in which new variables are added to the active dataset's dictionary is *ERR*, *SAS*, *SAF*, and *STC*.

Basic Specification

The basic specification is one or more series names.

- By default, SEASON uses a multiplicative model to compute and display moving averages, ratios, seasonal factors, the seasonally adjusted series, the smoothed trend-cycle components, and the

[77] Makridakis, S., S. C. Wheelwright, and V. E. McGee. 1983. *Forecasting: Methods and applications*. New York: John Wiley and Sons.

[78] McLaughlin, R. L. 1984. *Forecasting techniques for decision making*. Rockville, Md.: Control Data Management Institute.

irregular (error) component for each specified series (variable). The default periodicity is the periodicity that is established with TSET PERIOD or DATE.

- Unless the default on TSET NEWVAR is changed prior to the procedure, SEASON creates four new variables, with the following prefixes, for each specified series (these variables are automatically named, labeled, and added to the active dataset):

  SAF. *Seasonal adjustment factors.* These values indicate the effect of each period on the level of the series.

  SAS. *Seasonally adjusted series.* These values are the values that are obtained after removing the seasonal variation of a series.

  STC. *Smoothed trend-cycle components.* These values show the trend and cyclical behavior that are present in the series.

  ERR. *Residual or "error" values.* These values are the values that remain after the seasonal, trend, and cycle components have been removed from the series.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- VARIABLES can be specified only once.
- Other subcommands can be specified more than once, but only the last specification of each one is executed.

Operations

- The endpoints of the moving averages and ratios are displayed as system-missing in the output.
- Missing values are not allowed anywhere in the series. (You can use the RMV command to replace missing values, use TSET MISSING=INCLUDE to include user-missing values, and use USE to ignore missing observations at the beginning or end of a series. See RMV and USE for more information.)

If SPLIT FILE processing is on, SEASON ignores USE settings.

Limitations

- A maximum of one VARIABLES subcommand is allowed. There is no limit on the number of series that are named on the list.

VARIABLES Subcommand

VARIABLES specifies the series names and is the only required subcommand.

- Each specified series must contain at least four full seasons of data.

MODEL Subcommand

MODEL specifies whether the seasonal decomposition model is multiplicative or additive.

- The specification on MODEL is the keyword MULTIPLICATIVE or ADDITIVE.
- If more than one keyword is specified, only the first keyword is used.
- MULTIPLICATIVE is the default if the MODEL subcommand is not specified or if MODEL is specified without any keywords.

Example

```
SEASON VARIABLES = VARX
  /MODEL=ADDITIVE.
```

- This example uses an additive model for the seasonal decomposition of *VARX*.

MA Subcommand

MA specifies how to treat an even-periodicity series when computing moving averages.

- MA should be specified only when the periodicity is even. When periodicity is odd, the EQUAL method is always used.
- For even-periodicity series, the keyword EQUAL or CENTERED can be specified. CENTERED is the default.
- EQUAL calculates moving averages with a span (number of terms) equal to the periodicity and all points weighted equally.
- CENTERED calculates moving averages with a span (number of terms) equal to the periodicity plus 1 and endpoints weighted by 0.5.
- The periodicity is specified on the PERIOD subcommand.

Example

```
SEASON VARIABLES = VARY
  /MA=CENTERED
  /PERIOD=12.
```

- In this example, moving averages are computed with spans of 13 terms and endpoints weighted by 0.5.

PERIOD Subcommand

PERIOD indicates the size of the period.

- The specification on PERIOD indicates how many observations are in one period or season and can be any positive integer.
- If PERIOD is not specified, the periodicity that is established on TSET PERIOD is in effect. If TSET PERIOD is not specified, the periodicity that is established on the DATE command is used. If periodicity was not established anywhere, the SEASON command will not be executed.

Example

```
SEASON VARIABLES = SALES
  /PERIOD=12.
```

- In this example, a periodicity of 12 is specified for *SALES*.

APPLY Subcommand

APPLY allows you to use a previously defined SEASON model without having to repeat the specifications.

- The only specification on APPLY is the name of a previous model in quotation marks. If a model name is not specified, the model that was specified on the previous SEASON command is used. Model names are either the default *MOD_n* names or the names that are assigned on the MODEL NAME command.
- To change one or more model specifications, specify the subcommands of only those portions that you want to change after the APPLY subcommand.
- If no series are specified on the command, the series that were originally specified with the model that is being reapplied are used.
- To change the series used with the model, enter new series names before or after the APPLY subcommand. If a series name is specified before APPLY, the slash before the subcommand is required.

Example

```
SEASON VARIABLES = X1
  /MODEL=ADDITIVE.
SEASON VARIABLES = Z1
  /APPLY.
```

- The first command specifies an additive model for the seasonal decomposition of *X1*.
- The second command applies the same type of model to series *Z1*.

Example

```
SEASON X1 Y1 Z1
  /MODEL=MULTIPLICATIVE.
SEASON APPLY
  /MODEL=ADDITIVE.
```

- The first command specifies a multiplicative model for the seasonal decomposition of *X1*, *Y1*, and *Z1*.
- The second command applies an additive model to the same three variables.

References

Makridakis, S., S. C. Wheelwright, and V. E. McGee. 1983. *Forecasting: Methods and applications*. New York: John Wiley and Sons.

McLaughlin, R. L. 1984. *Forecasting techniques for decision making*. Rockville, Md.: Control Data Management Institute.

SELECT IF

```
SELECT IF [(]logical expression[)]
```

The following relational operators can be used in logical expressions:

Table 83. Relational operators

Symbol	Definition
EQ or =	Equal to
NE or ~= or ¬ = or <>	Not equal to
LT or <	Less than
LE or <=	Less than or equal to
GT or >	Greater than
GE or >=	Greater than or equal to

The following logical operators can be used in logical expressions:

Table 84. Logical operators

Symbol	Definition
AND or &	Both relations must be true
OR or |	Either relation can be true
NOT	Reverses the outcome of an expression

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

Example

```
SELECT IF (SEX EQ 'MALE').
```

Overview

SELECT IF permanently selects cases for analysis based on logical conditions that are found in the data. These conditions are specified in a *logical expression.* The logical expression can contain relational operators, logical operators, arithmetic operations, and any functions that are allowed in COMPUTE transformations. For temporary case selection, specify a TEMPORARY command before SELECT IF.

Basic Specification

The basic specification is simply a logical expression.

Syntax Rules

- Logical expressions can be simple logical variables or relations, or these expressions can be complex logical tests involving variables, constants, functions, relational operators, and logical operators. The logical expression can use any of the numeric or string functions that are allowed in COMPUTE transformations (see COMPUTE and see).
- Parentheses can be used to enclose the logical expression. Parentheses can also be used within the logical expression to specify the order of operations. Extra blanks or parentheses can be used to make the expression easier to read.

- A relation can compare variables, constants, or more complicated arithmetic expressions. Relations cannot be abbreviated. For example, `(A EQ 2 OR A EQ 5)` is valid while `(A EQ 2 OR 5)` is not valid. Blanks (not commas) must be used to separate relational operators from the expressions that are being compared.
- A relation cannot compare a string variable to a numeric value or variable, or vice versa. A relation cannot compare the result of the logical functions SYSMIS, MISSING, ANY, or RANGE to a number.
- String values that are used in expressions must be specified in quotation marks and must include any leading or trailing blanks. Lowercase letters are considered distinct from uppercase letters.

Operations

- SELECT IF permanently selects cases. Cases that are not selected are dropped from the active dataset.
- The logical expression is evaluated as true, false, or missing. If a logical expression is true, the case is selected; if it is false or missing, the case is not selected.
- Multiple SELECT IF commands that are issued prior to a procedure command must all be true for a case to be selected.
- SELECT IF should be placed before other transformations for efficiency.
- Logical expressions are evaluated in the following order: numeric functions, exponentiation, arithmetic operators, relational operators, and logical operators. Use parentheses to change the order of evaluation.
- If N OF CASES is used with SELECT IF, the program reads as many records as required to build the specified *n* cases. It makes no difference whether N OF CASES precedes or follows SELECT IF.
- System variable $CASENUM is the sequence number of a case in the active dataset. Although it is syntactically correct to use $CASENUM on SELECT IF, it does not produce the expected results. To select a set of cases based on their sequence in a file, create your own sequence variable with the transformation language prior to making the selection (see the “Examples ” on page 1776).

Missing Values

- If the logical expression is indeterminate because of missing values, the case is not selected. In a simple relational expression, a logical expression is indeterminate if the expression on either side of the relational operator is missing.
- If a compound expression is used in which relations are joined by the logical operator OR, the case is selected if either relation is true, even if the other relation is missing.
- To select cases with missing values for the variables within the expression, use the missing-value functions. To include cases with values that have been declared user-missing, along with other cases, use the VALUE function.

Limitations

SELECT IF cannot be placed within a FILE TYPE-END FILE TYPE or INPUT PROGRAM-END INPUT PROGRAM structure. SELECT IF can be placed nearly anywhere following these commands in a transformation program. See the topic “Commands and Program States” on page 2187 for more information.

Examples

Working With Simple Logical Expressions

```
SELECT IF (SEX EQ 'MALE').
```

- All subsequent procedures will use only cases in which the value of *SEX* is MALE.

- Because uppercase and lowercase are treated differently in comparisons of string variables, cases for which the value of *SEX* is male are not selected.

```
SELECT IF (INCOME GT 75000 OR INCOME LE 10000).
```

- The logical expression tests whether a case has a value that is either greater than 75,000 or less than or equal to 10,000. If either relation is true, the case is used in subsequent analyses.

```
SELECT IF (V1 GE V2).
```

- This example selects cases where variable *V1* is greater than or equal to *V2*. If either *V1* or *V2* is missing, the logical expression is indeterminate, and the case is not selected.

```
SELECT IF (SEX = 'F' & INCOME <= 10000).
```

- The logical expression tests whether string variable *SEX* is equal to F and whether numeric variable *INCOME* is less than or equal to 10,000. Cases that meet both conditions are included in subsequent analyses. If either *SEX* or *INCOME* is missing for a case, the case is not selected.

```
SELECT IF (SYSMIS(V1)).
```

- The logical expression tests whether *V1* is system-missing. If it is system-missing, the case is selected for subsequent analyses.

```
SELECT IF (VALUE(V1) GT 0).
```

- Cases are selected if *V1* is greater than 0, even if the value of *V1* has been declared user-missing.

```
SELECT IF (V1 GT 0).
```

- Cases are not selected if *V1* is user-missing, even if the user-missing value is greater than 0.

```
SELECT IF ((V1-15) LE (V2*(-0.001))).
```

- The logical expression compares whether *V1* minus 15 is less than or equal to *V2* multiplied by –0.001. If it is true, the case is selected.

```
SELECT IF ((YRMODA(88,13,0) - YRMODA(YVAR,MVAR,DVAR)) LE 30).
```

- The logical expression subtracts the number of days representing the date *(YVAR*, *MVAR*, and *DVAR*) from the number of days representing the last day in 1988. If the difference is less than or equal to 30, the case is selected.

Understanding and Changing the Order of Evaluation

```
SELECT IF (RECEIV GT DUE AND (REVNUS GE EXPNS OR BALNCE GT 0)).
```

- By default, AND is executed before 0R. This expression uses parentheses to change the order of evaluation.
- The program first tests whether variable *REVNUS* is greater than or equal to variable *EXPNS*, or variable *BALNCE* is greater than 0. Second, the program tests whether *RECEIV* is greater than *DUE*. If one of the expressions in parentheses is true and *RECEIV* is greater than *DUE*, the case is selected.
- Without the parentheses, the program would first test whether *RECEIV* is greater than *DUE* and *REVNUS* is greater than or equal to *EXPNS*. Second, the program would test whether *BALNCE* is greater than 0. If the first two expressions are true *or* if the third expression is true, the case is selected.

Selecting Cases Based on Their Sequence in a File

```
COMPUTE  #CASESEQ=#CASESEQ+1.
SELECT IF (MOD(#CASESEQ,2)=0).
```

- This example computes a scratch variable, *#CASESEQ*, containing the sequence numbers for each case. Every other case is selected, beginning with the second case.
- *#CASESEQ* must be a scratch variable so that it is not reinitialized for every case. An alternative is to use the LEAVE command.

Using SELECT IF within a DO IF Structure

```
DO IF  SEX EQ 'M'.
+   SELECT IF PRESTIGE GT 50.
ELSE IF  SEX EQ 'F'.
+   SELECT IF PRESTIGE GT 45.
END IF.
```

- The SELECT IF commands within the DO IF structure select males with prestige scores above 50 and females with prestige scores above 45.

SELECTPRED

SELECTPRED is available in the Statistics Base Edition.

```
SELECTPRED dependent variable [(MLEVEL={S})]
                                 {O}
                                 {N}
          [BY factor list] [WITH covariate list]

[/EXCEPT VARIABLES=varlist]

[/SCREENING [STATUS={ON**}]  [PCTMISSING={70**  }]
                {OFF }                   {number}
            [PCTEQUAL={95**  }]  [PCTUNEQUAL={90**  }]
                      {number}               {number}
            [STDDEV={0**   }
                    {number}
            [CV={0.001** }]]
                {number}

[/CRITERIA [SIZE={AUTO** }]  [PCUTOFF={0.05**}]
              {integer}              {number}
           [RANKING={PEARSONCHISQ**}]  [TIMER={5**   }]
                    {LRCHISQ       }          {number}
                    {CRAMERSV      }
                    {LAMBDA        }
           [SHOWUNSELECTED={0**    }]
                           {integer}

[/MISSING USERMISSING={EXCLUDE**}]
                      {INCLUDE  }

[/PRINT [CPS**]  [EXCLUDED**] [SELECTED**]  [NONE]]
        [SUMMARY**[([PEARSONCHISQ] [LRCHISQ] [CRAMERSV] [LAMBDA])]]

[/PLOT SUMMARY]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release history

Release 14.0

- Command introduced.

Example

```
SELECTPRED response_01.
```

Overview

The SELECTPRED procedure submits a large number of predictor variables and selects a smaller subset for use in predictive modeling procedures. This procedure uses a univariable method that considers each predictor in isolation, as opposed to the multivariable method of selecting procedures that is used by the NAIVEBAYES procedure. The SELECTPRED procedure supports both categorical and scale dependent variables and accepts very large sets of predictors. SELECTPRED is useful when rapid computing is required.

Options

Methods. The SELECTPRED procedure includes a screening step as well as the univariable predictor selection method.

Missing values. Cases with missing values for the dependent variable or for all predictors are excluded. The SELECTPRED procedure has an option for treating user-missing values of categorical variables as valid. User-missing values of scale variables are always treated as invalid.

Output. SELECTPRED displays pivot table output by default but offers an option for suppressing most such output. The procedure optionally displays lists of categorical and scale predictors—not a model—by

way of global macro variables. These global macro variables can be used to represent the subset of selected predictors in subsequent procedures.

Basic specification

The basic specification is the SELECTPRED command followed by a dependent variable.

By default, SELECTPRED determines the measurement level of the dependent variable based on its dictionary setting. All other variables in the dataset — except the weight variable if it is defined — are treated as predictors, with the dictionary setting of each predictor determining its measurement level. SELECTPRED performs the screening step and then selects a subset of predictors by using a univariable method. User-missing values are excluded, and default pivot table output is displayed.

Syntax rules

- All subcommands are optional.
- Only a single instance of each subcommand is allowed.
- An error occurs if a keyword is specified more than once within a subcommand.
- Parentheses, equals signs, and slashes that are shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not honored.

Operations

The SELECTPRED procedure begins by excluding the following types of cases and predictors.

- Cases with a missing value for the dependent variable.
- Cases with missing values for all predictors.
- Predictors with missing values for all cases.
- Predictors with the same value for all cases.

A further screening step can be used to exclude the following types of predictors:

- Predictors with a large percentage of missing values.
- Categorical predictors with a large percentage of cases representing a single category.
- Categorical predictors with a large percentage of categories containing one case.
- Scale predictors with a small coefficient of variation (standard deviation divided by mean).

The univariable predictor selection method also ranks each predictor based on its association with the dependent variable and selects the top subset of predictors to use in subsequent modeling procedures. The ranking criterion that is used for any given predictor depends on the measurement level of the dependent variable as well as the measurement level of the predictor.

Note: Since measurement level can affect the results, if any variables (fields) have an unknown measurement level, an initial data pass will be performed to determine default measurement level for any variables with an unknown measurement level. For information on the criteria used to determine default measurement level, see SET SCALEMIN.

Categorical dependent variable.

- For categorical predictors, the ranking criterion can be the Pearson chi-square p-value, likelihood ratio chi-square p-value, Cramér's *V*, or Lambda.
- For scale predictors, the F p-value from a one-way ANOVA is always used as the criterion.
- For mixed predictors, the categorical predictors use the chi-square p-value or the likelihood ratio chi-square p-value, and the scale predictors use the F p-value from a one-way ANOVA.

Scale dependent variable.

- For categorical predictors, the F p-value from a one-way ANOVA is always used as the criterion.
- For scale predictors, the p-value for a Pearson correlation coefficient is always used.
- For mixed predictors, both types of p-values are used, depending on the predictor type.

Frequency weight

If a WEIGHT variable is specified, its values are used as frequency weights by the SELECTPRED procedure.

- Cases with missing weights or weights that are less than 0.5 are not used in the analyses.
- The weight values are rounded to the nearest whole numbers before use. For example, 0.5 is rounded to 1, and 2.4 is rounded to 2.

Limitations

SPLIT FILE settings are ignored by the SELECTPRED procedure.

Examples

```
SELECTPRED response_01
  /EXCEPT VARIABLES=custid response_02 response_03
  /MISSING USERMISSING=INCLUDE.
```

- This analysis specifies *response_01* as the dependent variable.
- All other variables are to be considered as possible predictors, with the exception of *custid*, *response_02*, and *response_03*.
- User-missing values of categorical variables are treated as valid for the purpose of selecting predictors.
- All other settings fall back to their default values.

For discussion of output from this syntax and to place the example in context, see

Variable lists

The variable lists specify the dependent variable, any categorical predictors (also known as factors), and any scale predictors (also known as covariates).

- The dependent variable must be the first specification on SELECTPRED.
- The dependent variable may not be the weight variable.
- The dependent variable may be followed by the measurement-level specification, which contains, in parentheses, the MLEVEL keyword followed by an equals sign and then S for scale, O for ordinal, or N for nominal. SELECTPRED treats ordinal and nominal dependent variables equivalently as categorical.
- If a measurement level is specified, it temporarily overrides the dependent variable's setting in the data dictionary.
- If no measurement level is specified, SELECTPRED defaults to the dictionary setting.
- If a measurement level is not specified, and no setting is recorded in the data dictionary, a numeric variable is treated as scale and a string variable is treated as categorical.
- A string variable may be defined as ordinal or nominal only.
- The names of the factors, if any, must be preceded by the keyword BY. If BY is specified with no factors, a warning is issued, and the keyword is ignored.
- The names of the covariates, if any, must be preceded by the keyword WITH. If WITH is specified without covariates, a warning is issued, and the keyword is ignored.
- If the dependent variable or the weight variable is specified within a factor list or a covariate list, the variable is ignored in the list.
- All variables that are specified within a factor or covariate list must be unique. If duplicate variables are specified within a list, the duplicates are ignored.
- If duplicate variables are specified across the factor and covariate lists, an error is issued.
- The universal keywords TO and ALL may be specified in the factor and covariate lists.
- If neither BY nor WITH is specified, all variables in the active dataset except the dependent variable, the weight variable, and any variables that are specified on the EXCEPT subcommand, are treated as

predictors. If the dictionary setting of a predictor is nominal or ordinal, the predictor is treated as a factor. If the dictionary setting is scale, the predictor is treated as a covariate.

- The dependent variable and factor variables can be numeric or string.
- The covariates must be numeric.

EXCEPT subcommand

The EXCEPT subcommand lists any variables that the SELECTPRED procedure should exclude from the factor or covariate lists on the command line. This subcommand is useful if the factor or covariate lists contain a large number of variables—specified by using the TO or ALL keyword, for example—but there are a few variables (for example, Case ID) that should be excluded.

- The EXCEPT subcommand ignores duplicate variables, the dependent variable, and variables that are not specified on the command line's factor or covariate lists.
- There is no default variable list on the EXCEPT subcommand.

SCREENING subcommand

The SCREENING subcommand specifies settings for the screening step, which excludes unsuitable predictors.

STATUS=option. *Perform the screening step.* Valid options are ON or OFF. If OFF is specified, any other SCREENING specifications are ignored. The default value is ON.

PCTMISSING=number. *Percentage of missing values in a predictor.* If the percentage of cases representing a single category in a predictor is greater than the specified value, the predictor is excluded. The specified value must be a positive number that is less than or equal to 100. The default value is 70.

PCTEQUAL=number. *Percentage of cases representing a single category in a categorical predictor.* If the percentage of cases representing a single category in a predictor is greater than the specified value, the predictor is excluded. This setting applies only to categorical predictors. The specified value must be a positive number that is less than or equal to 100. The default value is 95.

PCTUNEQUAL=number. *Percentage of categories containing one case in a categorical predictor.* If the percentage of a predictor's categories containing only one case is greater than the specified value, the predictor is excluded. This setting applies only to categorical predictors. The specified value must be a positive number that is less than or equal to 100. The default value is 90.

STDDEV=number. *Minimum standard deviation for a scale predictor.* If the standard deviation is less than the specified value, the predictor is excluded. This setting applies only to scale predictors. The specified value must be a non-negative number. The default value is 0, which turns off the standard deviation check.

CV=number. *Coefficient of variation for a scale predictor.* A predictor's coefficient of variation is defined as its standard deviation divided by its mean. If the absolute value of the coefficient of variation is less than the specified value, the predictor is excluded. This setting applies only to scale predictors and applies only if the mean is not equal to 0. The specified value must be a positive number. The default value is 0.001.

CRITERIA subcommand

The CRITERIA subcommand specifies computational settings for the SELECTPRED procedure.

SIZE keyword

The SIZE keyword specifies the number of predictors to select. Value AUTO indicates that the number should be computed automatically. Alternatively, a positive integer less than the number of unique predictors on the SELECTPRED command may be specified. AUTO is the default.

RANKING keyword

The RANKING keyword specifies the ranking criterion that is used for categorical predictors if the target is categorical.

PEARSONCHISQ. *Rank predictors based on Pearson chi-square p-value.* Predictors are sorted in ascending order of Pearson chi-square p-values. This criterion is the default.

LRCHISQ. *Rank predictors based on likelihood ratio chi-square p-value.* Predictors are sorted in ascending order of likelihood ratio chi-square p-values.

CRAMERSV. *Rank predictors based on Cramer's* V. Predictors are sorted in descending order of Cramer's *V*. If this criterion is used when the target is categorical but predictors are mixed (that is, some categorical, some scale), the PEARSONCHISQ default is used instead.

LAMBDA. *Rank predictors based on Lambda (asymmetric).* Predictors are sorted in descending order of Lambda. If this criterion is used when the target is categorical but predictors are mixed; that is, some categorical, some scale, then the PEARSONCHISQ default is used instead.

PCUTOFF keyword

The PCUTOFF keyword specifies the cutoff whenever a p-value is used as a ranking criterion. More particularly, if p-values are used to rank predictors, only those predictors for which the p-value is less than the PCUTOFF value may be selected. A positive number that is less than or equal to 1 may be specified. Specifying 1 turns off the limit on p-values. The default is 0.05.

SHOWUNSELECTED keyword

The SHOWUNSELECTED keyword specifies a limit on the number of unselected predictors to display in tables and charts. If the specified number *n* is greater than 0, the top *n* unselected predictors are displayed. A non-negative integer may be specified. The default is 0.

TIMER keyword

The TIMER keyword specifies the maximum number of minutes during which the SELECTPRED procedure can be run. If the time limit is exceeded, the procedure is terminated, and no results are given. Any number that is greater than or equal to 0 may be specified. Specifying 0 turns the timer off completely. The default is 5.

MISSING Subcommand

The MISSING subcommand controls whether user-missing values for categorical variables are treated as valid values. By default, user-missing values for categorical variables are treated as invalid.

- User-missing values for scale variables are always treated as invalid.
- System-missing values for any variables are always treated as invalid.

USERMISSING=EXCLUDE. *User-missing values for categorical variables are treated as invalid.* This setting is the default.

USERMISSING=INCLUDE. *User-missing values for categorical variables are treated as valid values.*

PRINT subcommand

The PRINT subcommand indicates the tabular output to display. If PRINT is not specified, the default tables are displayed. If PRINT is specified, only the requested tables are displayed.

CPS Keyword

The CPS keyword displays the case processing summary table, which summarizes the number of cases that are included and excluded in the analysis.

EXCLUDED keyword

The EXCLUDED keyword displays a table of screened predictors. The table lists screened predictors by type (categorical or scale) and reason for being screened. The table includes predictors that are screened due to missing or constant values for all cases, as well as any additional screening that is requested on the SCREENING subcommand.

SUMMARY keyword

The SUMMARY keyword displays a statistical summary of selected and unselected predictors. Selected predictors are always displayed. The maximum number of unselected predictors that can be displayed is determined by the SHOWUNSELECTED option on the CRITERIA subcommand.

The SUMMARY keyword may be followed, optionally, by one or more ranking criterion keywords in parentheses. When the target is categorical, and predictors are categorical or of mixed type, the RANKING option on the CRITERIA subcommand specifies the ranking criterion actually used by the algorithm to select predictors. The parenthesized list that follows the SUMMARY keyword displays additional ranking criteria for comparative purposes. The criterion that is actually used is always displayed, irrespective of whether it is specified following SUMMARY.

If a parenthesized list follows SUMMARY, then keywords in the list may be given in any order. If a specified keyword is inapplicable, it is ignored, and a warning is issued.

PEARSONCHISQ. *Display the Pearson chi-square statistics.*

LRCHISQ. *Display the likelihood ratio chi-square statistics.*

CRAMERSV. *Display the Cramer's* V *statistics*.

LAMBDA. *Display the Lambda statistics.*

SELECTED keyword

The SELECTED keyword displays a table of selected predictors by type (categorical or scale).

NONE keyword

The NONE keyword suppresses all tabular output except the Notes table and any warnings. This keyword may not be specified with any other keywords.

PLOT subcommand

The PLOT subcommand and SUMMARY keyword display a chart that summarizes the selected and unselected predictors. Selected predictors are always displayed. The maximum number of unselected predictors that can be displayed is determined by the SHOWUNSELECTED option on the CRITERIA subcommand.

SET

Note: Square brackets used in the SET syntax chart are required parts of the syntax and are not used to indicate optional elements. All subcommands are optional.

```
SET

  BASETEXTDIRECTION={AUTOMATIC**}
                    {RIGHTTOLEFT}
                    {LEFTTORIGHT}

  BLANKS={SYSMIS**}
         {value   }

  BLOCK={X'2A'**    }
        {X'hexchar' }
        {'character'}

  BOX={X'2D7C2B2B2B2B2B2B2B2B'**}
      {X'hexstring'              }
      {'character'               }

  CACHE={20**}
        {n   }

  CCA={'-,,,'  } CCB={'-,,,'  } CCC={'-,,,'  }
      {'format'}     {'format'}     {'format'}

  CCD={'-,,,'  } CCE={'-,,,'  }
      {'format'}     {'format'}

  CELLSBREAK={10000**}
             {n      }

  CMPTRANS={YES}
           {NO }

  CTEMPLATE {NONE**              }
            {'full path and name'}

  DECIMAL={COMMA}
          {DOT  }

  DEFOLANG=default output language

  DIGITGROUPING={NO**}
                {YES }

  EPOCH={AUTOMATIC }
        {begin year

  ERRORS={LISTING**}
         {NONE     }

  FOOTNOTE={NO   }
           {YES**}

  FORMAT={F8.2**}
         {Fw.d  }

  FUZZBITS={6**}
           {n  }

  HEADER={NO** }
         {YES  }
         {BLANK}

  LEADZERO={NO**}
           {YES }

  LENGTH={59**} WIDTH={80**}
         {n   }       {255 }
         {NONE}       {n   }

  LOCALE={OSLOCALE**}
         {'localeid'}

  MCACHE=n

  MDISPLAY = [TEXT** | TABLES]

  MESSAGES={NONE** }
           {LISTING}

  MEXPAND={ON**}
          {OFF }

  MIOUTPUT = [OBSERVED** IMPUTED** POOLED** DIAGNOSTICS**]

  MITERATE={1000**}
           {n     }

  MNEST={50**}
        {n   }

  MPRINT={OFF**}
         {ON   }
```

```
MTINDEX={2000000**}
        {RANDOM   }
        {n        }

MXCELLS={AUTOMATIC**}
        {n         }

MXERRS={100**}
       {n    }

MXLOOPS={40**}
        {n   }

MXWARNS={10**}
        {n   }

OATTRS={OLANG**}
       {ENG    }
       {BOTH   }

ODISPLAY={MODELVIEWER**}
         {TABLES**   }

OLANG=output language

ONUMBERS={LABELS**}
         {VALUES  }
         {BOTH    }

OVARS={LABELS**}
      {NAMES   }  SEED={2000000**}
      {BOTH    }       {RANDOM   }
     {n        }

PRINTBACK={NONE** }
          {LISTING}

REPDEFER={NO }
         {YES}

RESULTS={LISTING**}
        {NONE     }

ROWSBREAK={100**}
          {n    }
          {NONE }

SCALEMIN=n

SIGLESS={ON**}
        {OFF }

SMALL=n

SORT={EXTERNAL**}
     {INTERNAL  }
     {SS        }

SUMMARY={NONE**      }
        {'expression'}

TABLERENDER={FAST**}
            {FULL  }
            {LIGHT }

TFIT={BOTH**   }  TVARS={LABELS**}
     {LABELS   }        {NAMES   }
     {BOTHLARGE}        {BOTH    }

THREADS={AUTO**}
        {n     }

TLOOK={NONE**              }
      {'full path and name'}

TNUMBERS={LABELS**}
         {VALUES  }  RNG={MC**}
         {BOTH    }      {MT  }

TOLERANCE={1**}
          {n }

UNDEFINED={WARN**}
          {NOWARN}

UNICODE={ON**}
        {OFF }

WORKSPACE={24576**}
          {n      }

XVERSION={DEFAULT**}
         {'version'}

ZCOMPRESSION={YES}
             {NO }
```

** Default setting at installation.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 13.0

- RNG and MTINDEX subcommands introduced.
- Default for MXERRS subcommand increased to 100.
- SORT subcommand introduced.
- LOCALE subcommand introduced.

Release 14.0

- Default for WORKSPACE subcommand increased to 6144.

Release 15.0

- LABELS replaces VALUES as the default for the TNUMBERS subcommand.
- JOURNAL subcommand is obsolete and no longer supported.
- Value EXTERNAL added to list for SORT subcommand, replacing the value INTERNAL as the default. Value SS is deprecated.

Release 16.0

- MCACHE subcommand introduced.
- THREADS subcommand introduced.
- UNICODE subcommand introduced.

Release 16.0.1

- BOTHLARGE keyword introduced for the TFIT subcommand.

Release 17.0

- FUZZBITS subcommand introduced.
- MIOUTPUT subcommand introduced.

Release 18.0

- ROWSBREAK, CELLSBREAK, and TOLERANCE subcommands introduced for controlling display of large pivot tables.
- ZCOMPRESSION subcommand introduced.
- COMPRESSION subcommand is obsolete and ignored.
- REPDEFER subcommand introduced.

Release 19.0

- XVERSION subcommand introduced.
- OATTRS subcommand introduced.
- DIGITGROUPING subcommand introduced.
- TABLERENDER subcommand introduced.
- CMPTRANS subcommand introduced.

Release 20.0

- FAST keyword introduced for the TABLERENDER subcommand, replacing the LIGHT keyword, which is deprecated.
- Value BPortugu (Brazilian Portuguese) added to list for OLANG subcommand.

Release 21.0

- ODISPLAY subcommand introduced.

Release 22.0

- OSLOCALE keyword added to LOCALE subcommand.

- BASETEXTDIRECTION subcommand added.
- SUMMARY subcommand added.

Release 24.0

- LEADZERO subcommand added.

Release Subscription (November 2017 update)

- MDISPLAY subcommand added.

Release 27.0.1

- FOOTNOTE and SIGLESS subcommands added.

Overview

Many of the running options in the program can be tailored to your own preferences with the SET command. The default settings for these options vary from system to system. To display the current settings, use the SHOW command. A setting that is changed by SET remains in effect for the entire working session unless changed again by another SET command. The PRESERVE command saves the current settings so that you can return to them later in the session by using the RESTORE command. PRESERVE and RESTORE are especially useful with the macro facility.

Options

Memory Management. Use the WORKSPACE subcommand to dynamically allocate memory when some procedures indicate memory shortage. Use the MXCELLS subcommand to increase maximum cell numbers for a pivot table.

Output Format. Use the FORMAT subcommand to change the default (F8.2) print and write formats used for numeric variables. Use the TLOOK and CTEMPLATE subcommands to specify a TableLook file and/or a chart template file. (*Note*: TableLooks created in earlier versions of IBM SPSS Statistics cannot be used in version 16.0 or later.) Use the ONUMBERS, OVARS, TNUMBERS, and TVARS subcommands to define default display of variables in the outline or pivot tables. Use the TFIT subcommand to specify default column widths. Use the DIGITGROUPING subcommand to specify whether the current IBM SPSS Statistics locale's digit grouping format is applied to numeric values in pivot tables, charts, and Model Viewer items, as well as in the Data Editor.

Samples and Random Numbers. You can use the RNG, SEED, and MTINDEX subcommands to change the random number generator and initialization value.

Output Destination. You can use the ERRORS, MESSAGES, PRINTBACK, and RESULTS subcommands to send error messages, resource-utilization messages, command printback, and the output from your commands to your screen and/or to a file. You can also suppress each of these items by using the keyword NONE.

Journal Files. The JOURNAL subcommand is obsolete. You can only set the journal location via the Options dialog (Edit menu>Options>File Locations tab).

Macro Displays. You can use the MEXPAND, MITERATE, and MNEST subcommands to control macro expansion, the maximum number of loop iterations, and nesting levels within a macro. You can also use the MPRINT subcommands to control the display of the variables, commands, and parameters that a macro uses.

Blanks and Undefined Input Data. You can use the BLANKS subcommand to specify the value that the program should use when it encounters a completely blank field for a numeric variable. You can also use UNDEFINED to turn off the warning message that the program issues when it encounters an invalid value for a numeric variable.

Maximum Warnings. You can use MXWARNS to limit the warning messages for each set of commands that read the data, after which further warnings are suppressed.

Maximum Loops. You can use MXLOOPS to raise or lower the maximum number of iterations that are allowed for the LOOP-END LOOP structure.

Scratch File Compression. You can specify whether scratch files are kept in compressed or uncompressed form using the ZCOMPRESSION subcommand.

Custom Currency Formats. You can customize currency formats for your own applications using the CCA, CCB, CCC, CCD, and CCE subcommands. For example, you can display currency as French francs rather than American dollars.

Cache File. The CACHE subcommand creates a complete copy of the active data file in temporary disk space after a specified number of changes in the active data file. Caching the active data file can improve performance.

Large Pivot Table Display. The ROWSBREAK, CELLSBREAK, and TOLERANCE subcommands control the display of large pivot tables in the Viewer. By default, tables with many rows are displayed in sections of 100 rows. To turn off this feature and always display the complete table regardless of size, specify ROWSBREAK=NONE. See the topic "CELLSBREAK, ROWSBREAK, and TOLERANCE Subcommands " on page 1792 for more information.

Basic Specification

The basic specification is at least one subcommand.

Subcommand Order

Subcommands can be specified in any order.

Syntax Rules

- You can specify as many subcommands as needed. Subcommands must be separated by at least one space or slash.
- Only one keyword or argument can be specified for each subcommand.
- SET can be used more than once in the command sequence.
- YES and ON are aliases for each other. NO and OFF are aliases for each other.

Operations

- Settings that are specified on SET remain in effect until they are changed by another SET command or until the current session is ended.
- Each time that SET is used, only the specified settings are changed. All other settings remain at their previous settings or the default.
- Where filenames are specified, they must include the full path. Relative file specifications, file handles, and filenames without a path are not allowed.

Example

```
SET BLANKS=0/UNDEFINED=NOWARN.
```

- BLANKS specifies 0 as the value that the program should use when it encounters a completely blank field for a numeric variable.
- UNDEFINED=NOWARN suppresses the message that is displayed whenever anything other than a number or a blank is encountered as the value for a numeric variable.

BASETEXTDIRECTION Subcommand

The BASETEXTDIRECTION subcommand controls the direction for text flow. Individual words will still flow in the correct direction, based on the language. This option controls only the text flow for complete blocks of text (for example, all the text that is entered in an edit field).

AUTOMATIC. Text flow is determined by characters that are used in each word. This option is the default.

RIGHTTOLEFT. Text flows right to left.

LEFTTORIGHT. Text flows left to right.

BLANKS Subcommand

BLANKS specifies the value that the program should use when it encounters a completely blank field for a numeric variable. By default, the program uses the system-missing value.

- BLANKS controls only the translation of numeric fields. If a blank field is read with a string format, the resulting value is a blank.
- The value that is specified on BLANKS is not automatically defined as a missing value.
- The BLANKS specification applies to all numeric variables. You cannot use different specifications for different variables.
- BLANKS must be specified before data are read. Otherwise, blanks in numeric fields are converted to the system-missing value (the default) as they are read.

BLOCK Subcommand

BLOCK specifies the character that is used for drawing icicle plots.

- You can specify any single character either as a quoted string or as a quoted hexadecimal pair preceded by the character *X*.
- The default is X'2A'.

Example

```
SET BLOCK='#'.
```

- This command specifies a pound sign (#) as the character to be used for drawing bar charts. The character is specified as a quoted string.

BOX Subcommand

BOX specifies the characters that are used to draw table borders in text output. The specification is either a 3-character or 11-character quoted string, in which the characters have the following representations: 1=horizontal line, 2=vertical line, 3=middle (cross), 4=lower left corner, 5=upper left corner, 6=lower right corner, 7=upper right corner, 8=left T, 9=right T, 10=Top T, 11=bottom T.

- The characters can be specified either as a quoted string or hexadecimal pairs. Specify an *X* before the quoted hexadecimal pairs.
- The defaults vary from system to system. To display the current settings, use the SHOW command.

CACHE Subcommand

The CACHE subcommand creates a complete copy of the active data file in temporary disk space after a specified number of changes in the active data file. If you have the available disk space, this feature can improve performance. The default number of changes that can occur before the active file is cached is 5.

Example

```
SET CACHE 10.
```

CCA, CCB, CCC, CCD, and CCE Subcommands

You can use the subcommands CCA, CCB, CCC, CCD, and CCE to specify up to five custom currency formats.

- Each custom currency subcommand defines one custom format and can include four specifications in the following order: a negative prefix, a prefix, a suffix, and a negative suffix.
- The specifications are delimited by either periods or commas, whichever you do not want to use as a decimal point in the format.
- If your custom currency format includes periods or commas that you need to distinguish from delimiters, use a single quotation mark as an escape character before the period or comma that is part of the custom currency format. For example, if the format includes a period but the decimal indicator is a comma, the period must also be used as the delimiter.
- Each currency specification must always contain three commas or three periods. All other specifications are optional.
- Use blanks in the specification only where you want blanks in the format.
- The entire specification must be enclosed in single or double quotation marks. If the format includes a single quotation mark as an escape character, the entire specification must be enclosed in double quotation marks.
- A specification cannot exceed 16 characters (excluding the apostrophes).
- Custom currency formats cannot be specified as input formats on DATA LIST. Use them only as output formats in the FORMATS, WRITE FORMATS, PRINT FORMATS, WRITE, and PRINT commands.

Specifying a Custom Currency Format

```
SET CCA='-,$,,'.
```

- A minus sign (-) preceding the first command is used as the negative prefix.
- A dollar sign is specified for the prefix.
- No suffixes are specified (there are two consecutive commas before the closing apostrophe).
- Because commas are used as separators in the specification, the decimal point is represented by a period.

Specifying Multiple Custom Currency Formats

```
SET CCA='(,,,-)'  CCB=',,%,' CCC='(,$,,)' CCD='-/-.Dfl ..-'.
FORMATS VARA(CCA9.0)/ VARB(CCB6.1)/ VARC(CCC8.0)/ VARD(CCD14.2).
```

- SET defines four custom currency formats.
- FORMATS assigns these formats to specific variables.

Table 85. Custom currency examples

	CCA	**CCB**	**CCC**	**CCD**
negative prefix	(	none	(	–/–
prefix	none	none	$	Dfl
suffix	none	%	none	none
negative suffix	–)	none	)	–
separator	,	,	,	.
sample positive number	23,456	13.7%	$352	Dfl 37.419,00
sample negative number	(19,423–)	13.7%	($189)	–/–Dfl 135,19–

CELLSBREAK, ROWSBREAK, and TOLERANCE Subcommands

These subcommands control the display of large pivot tables in the Viewer. These settings have no effect on printing large pivot tables or exporting output to external formats with command such as OUTPUT EXPORT or OMS. They also have no effect on tables rendered with the fast tables engine.

CELLSBREAK=n. *Sets the maximum number of cells to display.* The default is 10000. The value must an integer between 1000 and 100000. If ROWSBREAK=NONE, this setting is ignored.

ROWSBREAK=n|NONE. *Sets the number of rows to display.* The default is 100. The value must be an integer between 10 and 1000 or the keyword NONE, which turns off this feature and displays the entire table, regardless of size. When the feature is on and large tables are displayed in sections, navigation controls allow you to view different sections of the table.

TOLERANCE=n. *Controls the maximum number of rows of the inner most row dimension of the table to split across displayed views of the table.* For example, if there are six categories in each group of the inner most row dimension, specifying a value of six would prevent any group from splitting across displayed views. The value must be an integer and cannot be greater than the ROWSBREAK value. If ROWSBREAK=NONE, this setting is ignored.

Several factors can affect the actual number of rows displayed, including:

- If the CELLSBREAK value is reached before the ROWSBREAK value, then the table is split at that point. For example, if ROWSBREAK=1000 and CELLSBREAK=10000, a table with 20 columns will be displayed in blocks of 500 rows.
- The TOLERANCE setting can cause fewer or more rows than the ROWSBREAK value to be displayed.

CMPTRANS Subcommand

Note: This command works only if you are connected to the IBM SPSS Statistics Server running on Windows Server.

CMPTRANS interacts with a setting on the IBM SPSS Statistics Server to determine whether transformation commands (such as COMPUTE and RECODE) are compiled to machine code when a syntax job is run.

- If the administrator of the IBM SPSS Statistics Server turns on compiled transformations with the IBM SPSS Statistics Server setting *and* CMPTRANS is set to YES, transformation commands are compiled to machine code when the syntax job is run. The compiled code is used for the subsequent execution of the transformations.
- If the administrator does not turn on compiled transformations, a warning message is displayed and the command is ignored.
- Compiling transformations may improve performance when there are a large number of cases and multiple transformation commands.
- Because there is an overhead involved in compiling the transformations, you should set CMPTRANS to YES only when there are a large number of cases and multiple transformation commands.
- CMPTRANS has no effect on transformation commands enclosed in INPUT PROGRAM and END INPUT PROGRAM commands.

YES. *Compile transformations to machine code.*

NO. *Do not compile transformations to machine code.*

CTEMPLATE and TLOOK Subcommands

TLOOK and CTEMPLATE specify a file that is used to define the table and chart appearance in the output. The default for either command is NONE, which produces tables and charts that use the system defaults.

- TLOOK determines the properties of output tables that are produced. The properties include the borders, placement of titles, column and row labels, text font, and column and cell formats. (*Note*: TableLooks created in earlier versions of IBM SPSS Statistics cannot be used in version 16.0 or later.)
- CTEMPLATE determines the properties of output charts and plots. The properties include line style, color, fill pattern, and text font of relevant chart elements (such as frames, titles, labels, and legends).
- The specification on TLOOK or CTEMPLATE remains in effect until a new TLOOK or CTEMPLATE is specified.

NONE. *Use the system defaults.* The tables and charts in the output do not use customized properties.

filename. *Use the specified file as a template for tables/charts in the output.* For TLOOK you should specify a full path, enclosed in quotes (Directory settings on the CD and INSERT commands do not apply to the template file location for TLOOK).

DECIMAL Subcommand

DECIMAL can be used to override the default decimal indicator for reading and writing text data. The default decimal indicator is the OS locale decimal indicator or the decimal indicator for the locale specified on the LOCALE subcommand.

DOT. The decimal indicator is a period.

COMMA. The decimal indicator is a comma.

- A subsequent LOCALE subcommand—either on the same or separate SET command—will override the DECIMAL setting. See the topic "LOCALE Subcommand " on page 1796 for more information.
- This setting only affects reading and writing text data. It has no effect on how data values are displayed in the Data Editor or in results displayed in the Viewer.

DEFOLANG Subcommand

The DEFOLANG subcommand specifies the default output language (if the language that is specified on the OLANG subcommand is not available). The initial default setting is the language of the installed software version. For example, if you install the English version of the software on a Japanese operating system, the default output language is English.

Example

```
SET DEFOLANG = JAPANESE.
```

- The language keyword is not case-sensitive.
- Do *not* enclose the language keyword in quotation marks or other string delimiters.

DIGITGROUPING Subcommand

DIGITGROUPING specifies whether the current IBM SPSS Statistics locale's digit grouping format will be applied to numeric values in pivot tables and charts as well as in the Data Editor. For example, in a French locale, with DIGITGROUPING set to YES the value 34419,57 is displayed as 34 419,57.

The grouping format does not apply to trees, Model Viewer items, restricted numeric values (N format), numeric values with the DOT or COMMA format, or numeric values with a DOLLAR or custom currency format. It does, however, apply to the display of the days value for numeric values with a DTIME format (for example, to the value of ddd in the format ddd hh:mm).

YES. *Apply the current locale's digit grouping format.*

NO. *Do not apply the current locale's digit grouping format.* This is the default.

EPOCH Subcommand

EPOCH defines the 100-year-span dates that are entered with two-digit years and date functions with a two-digit year specification.

AUTOMATIC. *100-year span beginning 69 years prior to the current date and ending 30 years after the current date.*

begin year. *First year of the 100-year span.*

Examples

```
SET EPOCH=1900.
```

- All dates that are entered with two-digit year values are read as years between 1900 and 1999. For example, a date that is entered as 10/29/87 is read as 10/29/1987.

  ```
  SET EPOCH=1980.
  ```

- Dates that are entered with two-digit year values between 80 and 99 are read as years between 1980 and 1999.
- Dates that are entered with two-digit year values between 00 and 79 are read as years between 2000 and 2079.

ERRORS, MESSAGES, RESULTS, and PRINTBACK Subcommands

ERRORS, MESSAGES, RESULTS, and PRINTBACK are used with keywords LISTING and NONE to route program output. ERRORS, MESSAGES, and RESULTS apply only to text output. PRINTBACK applies to all commands that are entered in a syntax window or generated from a dialog box during a session.

- ERRORS refers to both error messages and warning messages for text output.
- MESSAGES refers to resource-utilization messages that are displayed with text output, including the heading and the summaries (such as the amount of memory that is used by a command).
- RESULTS refers to the text output that is generated by program commands.
- PRINTBACK refers to command printback in the log in the Viewer window and in the journal file. Syntax is always displayed as part of the Notes in the syntax window.

LISTING. *Display output in the designated output window.* This alias is ON or YES. For PRINTBACK, the alias is BOTH. The executed commands are printed back in the journal and displayed in the log in the output window. You can either display an icon only or list all commands.

NONE. *Suppress the output.* The alias is NO or OFF.

EXTENSIONS Subcommand

This subcommand is no longer supported.

FOOTNOTE Subcommand

FOOTNOTE specifies whether footnotes and captions display in tables.

YES
Footnotes and captions display in tables. This is the default setting.

NO
Footnotes and captions do not display in tables.

FORMAT Subcommand

FORMAT specifies the default print and write formats for numeric variables. This default format applies to numeric variables that are defined on DATA LIST in freefield format and to all numeric variables that are created by transformation commands (unless a format is explicitly specified).

- The specification must be a simple F format. The default is F8.2.
- You can use the PRINT FORMATS, WRITE FORMATS, and FORMATS commands to change print and write formats.
- Format specifications on FORMAT are output formats. When specifying the width, enough positions must be allowed so that any punctuation characters, such as decimal points, commas, and dollar signs, can be included.
- If a numeric data value exceeds its width specification, the program still attempts to display some value. The program rounds decimal values, removes punctuation characters, tries scientific notation, and finally, if there is still not enough space, produces asterisks indicating that a value is present but cannot be displayed in the assigned width.

FUZZBITS Subcommand

FUZZBITS specifies the default setting of the *fuzzbits* argument for the RND and TRUNC functions. The value controls the number of least-significant bits by which the internal representation of the value to be rounded or truncated (expressed as a 64-bit floating point binary) may fall short of the nearest rounding boundary and be rounded up. See the topic "Arithmetic functions" on page 61 for more information.

- The value must be an integer between 0 and 20. The default is 6, which should be sufficient for most applications.
- A value of 0 produces the same results as in release 10. A value of 10 produces the same results as in releases 11 and 12.
- The FUZZBITS setting persists across sessions.

HEADER Subcommand

HEADER controls whether the output includes headings. The HEADER subcommand applies to both default headings and headings that are specified on the TITLE and SUBTITLE commands. This command applies only to text output from this program. The default is NO.

NO. *Suppress headings in text output.* All general headings, including pagination, are replaced by a single blank line.

YES. *Display headings in text output.*

BLANK. *Suppress headings but start a new page.*

JOURNAL Subcommand

This subcommand is obsolete and no longer supported. To set the location of the journal file, which contains a log of submitted commands and error and warning messages that are generated during a session, and turn the journal on and off:

1. From the menus in any IBM SPSS Statistics window, choose:

   Edit > **Options**
2. On the File Locations tab, specify the journal location and select journal options.

LEADZERO Subcommand

LEADZERO specifies whether leading zeros are displayed for numeric values that consist only of a decimal part. For example, when leading zeros are displayed, the value .123 is displayed as 0.123.

- The LEADZERO setting does not apply to numeric values that have a currency or percent format.
- With the exception of fixed ASCII files (*.dat), leading zeros are not included when the data are saved to an external file.

NO
Do not display leading zeros. This setting is the default.

YES
Display leading zeros.

LENGTH and WIDTH Subcommands

LENGTH and WIDTH specify the maximum page length and width for the text output, respectively. These two subcommands apply only to text output.

- Length represents number of lines, and width represents number of characters.
- You can specify any length from 40 through 999,999 lines. If a long page length is specified, the program continues to provide page ejects and titles at the start of each procedure and at logical points in the display, such as between crosstabulations.
- To suppress page ejects, use keyword NONE on LENGTH. The program will insert titles at logical points in the display but will not supply page ejects.
- You can specify any number of characters from 80 through 255 for WIDTH.

LOCALE Subcommand

The LOCALE subcommand allows you to change the locale used for data analysis. By default, the locale is set to the computer's system locale if possible. With the LOCALE subcommand, you can override the default behavior and process data files in other locales without changing your computer's system or user locale.

- The default locale is the current operating system locale.
- The LOCALE subcommand persists. The next time that IBM SPSS Statistics is started on your computer, IBM SPSS Statistics will run in that locale.
- The OSLOCALE keyword sets the locale to the operating system locale. The application gets this information from the operating system each time you start the application. This is the default.
- If the locale ID does not match the system locale, not all output will be rendered correctly.
- In code page mode, locale changes that also change the encoding can only be made when there are no open datasets. For information on code page mode and Unicode mode, see "UNICODE Subcommand" on page 1805.
- You can use SHOW LOCALE to view the current IBM SPSS Statistics locale.

Example

```
SET LOCALE='Japanese'.
```

- When you are connecting to a server, the relevant locale ID is defined in the *loclmap.xml* file, which is located on the server computer. Check with your server administrator for available locale IDs.
- Locale ID values of the general form *Language_Country.codepage#*, which were valid in previous releases, are not valid in release 16.0 or later. In most instances, the "Language" portion of the old locale specification should be a valid locale ID in release 16.0 or later. For example, instead of

'Japanese_Japan.932', you could specify simply 'Japanese'. For a complete list of names and aliases that can be used to specify locale, go to http://www.iana.org/assignments/character-sets

Two-letter language abbreviations adhere to the ISO 639-1 standard: http://www.loc.gov/standards/iso639-2/php/code_list.php

Two-letter country codes adhere to the ISO 3166 standard: http://en.wikipedia.org/wiki/ISO_3166-1

Example

```
SET LOCALE='ja_JP.Shift_JIS'
```

- "ja" is the two-letter language code for Japanese.
- "JP" is the two-letter country code for Japan.
- "Shift_JIS" is the name of the character set.

MCACHE Subcommand

The MCACHE subcommand specifies the memory cache (in kilobytes) that multithreaded procedures use when calculating results. By default, IBM SPSS Statistics uses an internal algorithm to determine the memory cache for a particular computer. You can change this setting, but the default will often provide the best performance. The default also appears as the recommended setting when you run the SHOW command.

Using the memory cache is relevant only if your computer has multiple processors or each processor has multiple cores. Also, this setting affects only selected procedures that take advantage of it. These are:

- CORRELATIONS
- DISCRIMINANT
- FACTOR
- NOMREG
- PARTIAL CORR
- REGRESSION

Note: PRESERVE and RESTORE do not operate on this subcommand.

MDISPLAY Subcommand

The MDISPLAY subcommand specifies how matrices are displayed. By default, IBM SPSS Statistics displays matrices to TEXT. The TABLES setting uses pivot tables to display matrices.

The default also appears as the recommended setting when you run the SHOW command.

MEXPAND and MPRINT Subcommands

MEXPAND and MPRINT control whether macros are expanded and whether the expanded macros are displayed. For more information about macros, see the DEFINE command and .

The specifications for MEXPAND are:

ON. *Expand macros.* This setting is the default.

OFF. *Do not expand macros.* The command line that calls the macro is treated like any other command line. If the macro call is a command, it will be executed; otherwise, it will trigger an error message.

The specifications for MPRINT are:

ON. *Include expanded macro commands in the output.*

OFF. *Exclude expanded macro commands from the output.* This is the default.

- MPRINT is effective only when MEXPAND is ON and is independent of the PRINTBACK subcommand.

MIOUTPUT Subcommand

MIOUTPUT specifies the display of pooled output when a multiply imputed dataset is analyzed.

OBSERVED. *Show results for observed data.*

IMPUTED. *Show results for each imputed dataset.*

POOLED. *Show final pooled statistics.*

DIAGNOSTICS. *Display pooling diagnostics for parameters pooled by the univariate method.*

- At least one keyword must be specified in square brackets. If a keyword is not specified the corresponding output is suppressed.
- DIAGNOSTICS is ignored with a warning if POOLED is not specified.
- All options are on by default.
- The MIOUTPUT setting persists across sessions. It is ignored by tables that do not support splits. It is also ignored when analysis is performed without variable *Imputation_* defined as a split variable.
- MIOUTPUT has no effect on charts or PMML.

See the topic for more information.

MITERATE and MNEST Subcommands

MITERATE and MNEST control the maximum loop traversals and the maximum nesting levels permitted in macro expansions, respectively.

- The specification on MITERATE or MNEST is a positive integer. The default for MITERATE is 1000. The default for MNEST is 50.

MTINDEX, RNG, and SEED Subcommands

Two random number generators are available. The generator that is currently in effect is set by the RNG subcommand:

RNG=MC. *The random number generator that is used in version 12 and previous releases.* If you need to reproduce randomized results that were generated in previous releases based on a specified seed value, use this random number generator. This setting is the default.

RNG=MT. *Mersenne Twister random number generator.* This generator is a newer random number generator that is more reliable for simulation purposes. If reproducing randomized results from version 12 or earlier is not an issue, use this random number generator.

If you need to reproduce the same randomized results in the future, you can set the initialization value for the random number generator. The initialization value applies to the current session only. It is automatically reset to the default at the start of each new session.

SEED={integer | RANDOM}. *Initialization value for MC random number generator*. The value must be a positive integer that is less than 2,000,000,000 or the keyword RANDOM, which randomly sets the initialization value. The default is 2,000,000.

MTINDEX={value | RANDOM} . *Initialization value for the MT random number generator.* The value can be any positive or negative value, including fractional values (expressed as decimals), or the keyword RANDOM. The default is 2,000,000.

Example

```
SET RNG=MT MTINDEX=-12345.678.
```

MXCELLS and WORKSPACE Subcommands

WORKSPACE allocates more memory for some procedures when you receive a message indicating that the available memory has been used up or indicating that only a given number of variables can be processed. MXCELLS increases the maximum number of cells you can create for a new pivot table when you receive a warning that a pivot table cannot be created because it exceeds the maximum number of cells that are allowed.

- WORKSPACE allocates workspace memory in kilobytes for some procedures that allocate only one block of memory. The default and minimum value is 24576.
- Do not increase the workspace memory allocation unless the program issues a message that there is not enough memory to complete a procedure.
- Use MXCELLS with caution. Set MXCELLS at a number higher than the limit indicated in the warning message that you receive. After the table is created, restore the number to the default.
- The memory-allocation or cell maximum number increase takes effect as soon as you run the SET command.

Note: The MXMEMORY subcommand is no longer supported.

MXERRS Subcommand

MXERRS controls the maximum number of errors that are allowed in a session. The default is 100.

- MXERRS applies only to command files that are submitted for execution through IBM SPSS Statistics Batch Facility (available with IBM SPSS Statistics Server).
- Eligible errors are errors that stop execution of a command but continue the session.
- When the MXERRS limit is exceeded, IBM SPSS Statistics stops processing commands but continues to scan for additional errors.
- In interactive mode or in IBM SPSS Statistics and other windowed environments, MXERRS does not apply.

MXLOOPS Subcommand

MXLOOPS specifies the maximum number of times that a loop that is defined by the LOOP-END LOOP structure is executed for a single case or input record. The default is 40.

- MXLOOPS prevents infinite loops, which may occur if no cutoff is specified for the loop structure (see LOOP-END LOOP).
- MXLOOPS will limit the number of loops for any loop structure that doesn't have an indexing clause, including loops with conditional IF clauses. If a loop has an indexing clause (e.g., LOOP #i=1 to 1000), the indexing clause overrides the MXLOOPS setting.
- When a loop is terminated, control passes to the command immediately following the END LOOP command, even if the END LOOP condition is not yet met.

MXWARNS Subcommand

MXWARNS controls the number of warnings that are issued. The default is 10. The behavior of this setting depends on mode of operation: interactive or IBM SPSS Statistics Batch Facility. In this context, "interactive" includes any mode of operation other than IBM SPSS Statistics Batch Facility.

Interactive. In interactive mode, MXWARNS limits the number of warnings that are issued for each set of commands that read the data (for example, a group of transformation commands followed by a statistical procedure).

- Exceeding the limit does not halt execution of commands; it simply suppresses further warnings.
- MXWARNS=0 suppresses all warnings except a warning that further warnings have been suppressed.
- MXWARNS does not affect the display of error messages, and errors do not count toward the warning limit.

Batch Facility. In the Batch Facility (available with IBM SPSS Statistics Server), MXWARNS limits the total number of warnings that are allowed in a job.

- When the MXWARNS limit is exceeded, IBM SPSS Statistics stops processing commands but continues to scan for errors.
- When the combined total number of warnings and errors exceeds the MXWARNS limit, IBM SPSS Statistics stops processing commands but continues to scan for errors.

OATTRS and XVERSION Subcommands

XVERSION and OATTRS control aspects of OXML and SAV output from the Output Management System (OMS).

- XVERSION specifies the version of the Output XML schema used to generate OXML output from OMS.
- OATTRS specifies whether language invariant versions of `text` attributes are included in OXML and whether variable names in datasets generated from SAV output are in English. When specified, the English value of a `text` attribute is stored in the `text_eng` attribute which either replaces the `text` attribute or is in addition to it. The `text_eng` attribute allows you to write XPath expressions in a language invariant manner. See the topic for more information.

Note: The settings specified by XVERSION and OATTRS apply to subsequent OMS requests.

The specifications for XVERSION are:

DEFAULT. *Use the schema associated with the version of IBM SPSS Statistics from which the SET command is executed.* This is the default.

'version' . *A quoted string specifying the output schema version--for example, '1.6'.* The version string is the value of the `version` attribute of the `xs:schema` element from the associated output schema file. You can specify the version for any of the installed output schema files. Output schema files are located in the IBM SPSS Statistics installation folder.

The specifications for OATTRS are:

OLANG. *The text_eng attribute is not included in OXML output and variable names in datasets generated from SAV output are translated into the current output language (when available).* This is the default.

ENG. *The text attribute is replaced (other than in cell elements) with a text_eng attribute whose value is the English value of the text attribute. Variable names in datasets generated from SAV output will be in English.* The value ENG requires that the output schema version in use is 1.6 or higher, which is satisfied by the default setting of the XVERSION subcommand.

BOTH. *A text_eng attribute is added to each element (other than a cell element) that has a text attribute. The value of the text_eng attribute is the English value of the text attribute. Variable names in datasets generated from SAV output will be in English.* The value BOTH requires that the output schema version in use is 1.6 or higher, which is satisfied by the default setting of the XVERSION subcommand.

ODISPLAY Subcommand

For procedures that can generate either Model Viewer output or pivot table and chart output, ODISPLAY specifies which type of output is generated. For version 27, ODISPLAY only applies to output from the

Generalized Linear Mixed Models procedure (GENLINMIXED command) and the Nonparametric Tests procedure (NPTESTS command).

MODELVIEWER. *Specifies that Model Viewer output will be generated.*

TABLES. *Specifies that pivot table and chart output will be generated.* This is the default.

OLANG Subcommand

The OLANG subcommand controls the language that is used in output. OLANG does not apply to simple text output. Available languages may vary. (The General tab in the Options dialog box displays a list of available output languages.) Valid language keywords include English, French, German, Spanish, Italian, Japanese, Korean, TChinese (traditional Chinese), SChinese (simplified Chinese), Polish, Russian, and BPortugu (Brazilian Portuguese).

Output that is produced after the command is executed will be in the specified language (if that language is available). Additional language materials may be available for downloading from the IBM Corp. web site.

Depending on the language, you may also need to switch to UNICODE mode or change the LOCALE setting to match the output language for characters to render properly.

Example

```
SET OLANG = German.
```

- The language keyword is not case-sensitive.
- Do *not* enclose the language keyword in quotation marks or other string delimiters.

ONUMBERS, OVARS, TNUMBERS, and TVARS Subcommands

ONUMBERS, OVARS, TNUMBERS, and TVARS control how variables are displayed in the outline for pivot table output and in the pivot tables.

- ONUMBERS controls the display of variable values in the outline for pivot tables. The default is LABELS.
- OVARS controls the display of variables in the outline for pivot tables. The default is LABELS.
- TNUMBERS controls the display of variable values and/or value labels in the pivot tables. The default is LABELS.
- TVARS controls the display of variable names and/or variable labels in the pivot tables. The default is LABELS.

NAMES. *Display variable names.*

VALUES. *Display variable values.*

LABELS. *Display variable labels.*

BOTH. *Display both labels and values for variables or both names and labels for variables.*

REPDEFER Subcommand

REPDEFER specifies whether REPOSITORY CONNECT commands (connection requests to an IBM SPSS Collaboration and Deployment Services Repository) are deferred until a connection is required.

YES. *Defer connection requests until a connection is required.* This is the default when executing command syntax with the IBM SPSS Statistics Batch Facility (available with IBM SPSS Statistics Server).

NO. *Attempt to connect when a REPOSITORY CONNECT command is encountered.* This is the default unless executing command syntax with the IBM SPSS Statistics Batch Facility.

SCALEMIN Subcommand

For IBM SPSS Statistics data files that were created prior to release 8.0 and data read from external sources, you can specify the minimum number of data values for a numeric variable that is used to classify the variable as continuous (scale) or nominal. Variables with fewer than the specified number of unique values are classified as nominal. All string variables are classified as nominal, regardless of the number of unique values.

There are numerous other conditions that are evaluated prior to applying the minimum number of data values rule when determining to apply the continuous (scale) or nominal measurement level. Conditions are evaluated in the order listed in the table below. The measurement level for the first condition that matches the data is applied.

Table 86. Rules for determining default measurement level

Condition	Measurement Level
Format is dollar or custom-currency	Continuous
Format is date or time (excluding Month and Wkday)	Continuous
All values of a variable are missing	Nominal
Variable contains at least one non-integer value	Continuous
Variable contains at least one negative value	Continuous
Variable contains no valid values less than 10,000	Continuous
Variable has *N* or more valid, unique values*	Continuous
Variable has no valid values less than 10	Continuous
Variable has less than *N* valid, unique values*	Nominal

* *N* is the user-specified cut-off value. The default is 24.

SIGLESS Subcommand

SIGLESS specifies whether small significance levels (such as < 0.001) are displayed in tables.

YES
Small significance levels (such as < 0.001) are displayed in tables. This is the default setting.

NO
Small significance levels (such as < 0.001) are not displayed in tables.

SMALL Subcommand

The SMALL subcommand controls the display of numbers in scientific notation in pivot tables for small decimal values.

This setting only affects output with the "general" format, which is determined by the application. This setting does not affect the display of significance levels or other statistics with a standard range of values. The format of many numeric values in pivot tables is based on the format of the variable associated with the numeric value.

Example

```
SET SMALL = 0.
SET SMALL = .001.
```

- The first SET SMALL command suppresses the display of scientific notation in all output that uses the general format.
- The second SET SMALL command will only display scientific notation for values that are less than 0.001.

SORT Subcommand

By default, IBM SPSS Statistics tries to use an external, third-party sorting mechanism, which may reduce processing time with large data sources. The third-party sorting option is available only if you have IBM SPSS Statistics Server. The specific sorting engine is defined by your server administrator. If you are not connected to the IBM SPSS Statistics Server or the IBM SPSS Statistics Server cannot find the third-party sort engine, the built-in sorting mechanism is used.

EXTERNAL. *Use the external, third-party sort engine if available.* This setting is the default. If the third-party sort engine is not available, this setting is ignored, and the built-in sorting mechanism is used. COXREG and CURVEFIT use built-in sorting regardless of the SORT setting.

INTERNAL. *Use the built-in sorting mechanism.*

SS. This setting is deprecated. It has the same effect as EXTERNAL.

SUMMARY Subcommand

The SUMMARY subcommand adds comments to pivot tables.

- Comment text is displayed in a tooltip when you hover over a table in the Viewer.
- Screen readers read the comment text when the table has focus.
- The tooltip in the Viewer displays only the first 200 characters of the comment, but screen readers read the entire text.
- When you export output to HTML, the comment text is used as alt text.

The keyword is followed by an equals sign and the keyword NONE or a quoted value. The default option is NONE. You can also include special variables to insert date, time, and other values.

)DATE
Current date in the form dd-mmm-yyyy.

)ADATE
Current date in the form mm/dd/yyyy.

)SDATE
Current date in the form yyyy/mm/dd.

)EDATE
Current date in the form dd.mm.yyyy.

)TIME
Current 12-hour clock time in the form hh:mm:ss.

)ETIME
Current 24-hour clock time in the form hh:mm:ss.

)TITLE
The text of the outline label for the table.

)PROCEDURE
The name of the procedure that created the table.

)DATASET
The name of the dataset used to create the table.

\n
Inserts a line break.

TABLERENDER Subcommand

You can specify whether tables are rendered as legacy tables. Legacy tables may render slowly and are only recommended if you require compatibility with IBM SPSS Statistics releases prior to 20. For release 20 and higher, all tables have full support for pivoting and editing.

- Tables, other than legacy tables, created in IBM SPSS Statistics release 20 or later and lightweight tables in output documents that are modified in release 20 or later (but created in release 19) cannot be viewed in releases prior to 19.0.0.2. Such tables are viewable in release 19.0.0.2, where they are rendered as lightweight tables; however, they may not render the same as in release 20 or later.
- Lightweight tables created in IBM SPSS Statistics release 19 automatically have full support for pivoting and editing in release 20 or later.

FAST. *Tables render much faster than legacy tables.* This is the default.

FULL. *Tables are rendered as legacy tables.*

LIGHT. This setting is deprecated. It has the same effect as FAST.

TFIT Subcommand

TFIT controls the default column widths of the pivot tables. The default at installation is BOTH.

BOTH. *Adjust column widths to accommodate both labels and data.*

LABELS. *Adjust column widths to accommodate labels only.* This setting produces compact tables, but data values that are wider than labels may be truncated.

BOTHLARGE. *Adjust column widths to accommodate both labels and data, except for very large tables.* For very large tables (more than 10,000 cells), column widths are adjusted to accomodate labels only, and data values that are wider than the column labels may be truncated. *Note*: For tables that are not rendered as legacy tables, this setting is equivalent to BOTH.

THREADS Subcommand

The THREADS subcommand specifies the number of threads that multithreaded procedures use when calculating results. By default, IBM SPSS Statistics uses an internal algorithm to determine the number of threads for a particular computer. You can change this setting, but the default (THREADS=AUTO) will often provide the best performance. The default also appears as the recommended setting when you run the SHOW command.

In local analysis mode, any changes to the setting persist across sessions until you explicitly change the setting to a different value. In distributed analysis mode (available with IBM SPSS Statistics Server), the setting does not persist across sessions, and the default value is AUTO. Any changes to the setting in distributed analysis mode remain in effect only for the duration of the current session. In distributed analysis mode, the maximum number of threads can be limited by the administrator.

Using more than one thread is relevant only if your computer has multiple processors or each processor has multiple cores. Also, this setting affects only selected procedures that take advantage of it. These are:

- CORRELATIONS
- CSCOXREG
- CSLOGISTIC
- CSORDINAL
- CSSELECT

- DISCRIMINANT
- FACTOR
- NOMREG
- PARTIAL CORR
- REGRESSION
- SIMRUN
- SORT CASES

Note: PRESERVE and RESTORE do not operate on this subcommand.

UNDEFINED Subcommand

UNDEFINED controls whether the program displays a warning message when it encounters anything other than a number or a blank as the value for a numeric variable. The default is WARN.

WARN. *Display a warning message when an invalid value is encountered for a numeric variable.* This setting is the default.

NOWARN. *Suppress warning messages for invalid values.*

UNICODE Subcommand

SET UNICODE NO|YES controls the default behavior for determining the encoding for reading and writing data files and syntax files.

NO. *Use the current locale setting to determine the encoding for reading and writing data and command syntax files.* This is referred to as **code page mode**. The alias is OFF. For information on the current locale setting, see "LOCALE Subcommand " on page 1796.

YES. *Use Unicode encoding (UTF-8) for reading and writing data and command syntax files.* This is referred to as **Unicode mode**. The alias is ON. This is the default.

- You can change the UNICODE setting only when there are no open data sources.
- The UNICODE setting persists across sessions and remains in effect until it is explicitly changed.

There are a number of important implications regarding Unicode mode and Unicode files:

- Data and syntax files saved in Unicode encoding should not be used in releases prior to 16.0.
- When code page data files are read in Unicode mode, the defined width of all string variables is tripled. You can use ALTER TYPE to automatically adjust the width of all string variables.
- The GET command determines the file encoding for IBM SPSS Statistics data files from the file itself, regardless of the current mode setting (and defined string variable widths in code page files are tripled in Unicode mode).See the topic "GET" on page 779 for more information.
- For syntax files, the encoding is changed *after* execution of the block of commands that includes SET UNICODE. For example, if your are currently in code page mode, you must run SET UNICODE=YES separately from subsequent commands that contain Unicode characters not recognized by the local encoding in effect prior to switching to Unicode.

ZCOMPRESSION Subcommand

Note: This command works only if you are connected to the IBM SPSS Statistics Server.

ZCOMPRESSION is an override mechanism that works with a setting on the IBM SPSS Statistics Server to determine whether scratch files that are created during a session are in compressed or uncompressed form. Scratch files are created by CACHE, SORT, AGGREGATE, transformations, and any other procedure that adds columns to the data file.

- The administrator of the IBM SPSS Statistics Server must give you permission to override the IBM SPSS Statistics Server setting. If you don't have permission, a warning message is displayed and the command is ignored.
- A compressed scratch file reduces the amount of disk I/O compared to an uncompressed scratch file but requires more processing on the IBM SPSS Statistics Server.
- The extra overhead associated with compression is considerably greater when writing than reading a file. Consider using a higher value for SET CACHE to reduce the frequency of writing new scratch files containing the entire data set. A higher SET CACHE value will also increase the number of times a scratch file is read.
- If you are working with a very large data file (larger than half of the Server's RAM) and compression is on, you may consider issuing an explicit CACHE command before running other procedures. Doing this will ensure the procedures take full advantage of the disk I/O benefits.
- The override specification takes effect the next time that a scratch file is written and stays in effect until SET ZCOMPRESSION is specified again or until the end of the session.
- The default setting for compressing scratch files is set on the IBM SPSS Statistics Server. There is no syntax default.

YES. *Compress scratch files.*

NO. *Do not compress scratch files.*

SHIFT VALUES

```
SHIFT VALUES VARIABLE=varname RESULT=varname {LEAD=non-negative integer}
                                             {LAG=non-negative integer }
                                             {SHIFT=integer            }
  [/VARIABLE=varname...]
```

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 17.0

- Command introduced.

Example

```
SHIFT VALUES VARIABLE=OldVar RESULT=NewVar LEAD=1.
```

Overview

The SHIFT VALUES command creates new variables that contain the values of existing variables from preceding or subsequent cases.

Basic Specification

The basic specification is the command name followed by:

VARIABLE=varname. This must be a variable that already exists in the active dataset.

RESULT=newname. This must be a variable that does not currently exist in the active dataset.

One of the following alternative shift methods:

- **LEAD=n.** Get the value from a subsequent case in the dataset. The value must be a non-negative integer that indicates the number of cases. For example, LEAD=1 returns the value of the existing variable from the case immediately following the current case.
- **LAG=n.** Get the value from a preceding case in the dataset. The value must be a non-negative integer that indicates the number of cases. For example, LAG=1 returns the value of the existing variable from the case immediately before the current case.
- **SHIFT=n.** Get the value from a preceding or subsequent case. The value must be an integer that indicates the number of cases. For example, SHIFT=1 returns the value of the existing variable from the case immediately following the current case, and SHIFT=-1 returns the value of the existing variable from the case immediately before the current case.

To specify more than one variable on the same command, use a forward slash (/) to separate the full specification for each subsequent variable.

Syntax Rules

- The RESULT keyword must specify a new variable name. The name cannot be a variable that already exists in the active dataset.For information on variable naming rules, see “Variable Names” on page 42.
- Each keyword must be followed by an equals sign (=) before the value. For example, RESULT=NewVar.

Operations

- If split file processing is on, the scope of the shift is limited to each split group. A shift value cannot be obtained from a case in a preceding or subsequent split group.
- Filter status is ignored.

- The value of the result variable is set to system-missing for the first or last *n* cases in the dataset or split group, where *n* is the number of cases specified on LEAD, LAG, or SHIFT. For example, LAG=1 would set the result variable to system-missing for the first case in the dataset (or first case in each split group).
- A value of 0 for number of cases will copy the value of the original value to the same case for the new variable.
- User-missing values are preserved.
- Dictionary information from the original variable, including defined value labels and user-missing value assignments, is applied to the new variable. (Note: Custom variable attributes are not included.)
- A variable label is automatically generated for the new variable that describes the shift operation that created the variable.

Example

```
DATA LIST FREE /Var1 (F1).
BEGIN DATA
0 1 9 0 1
END DATA.
VARIABLE LABEL Var1 'Original Variable'.
VALUE LABELS Var1 0 'No' 1 'Yes' 9 'Missing'.
MISSING VALUES Var1(9).
SHIFT VALUES VARIABLE=Var1 RESULT=Var1_Lead LEAD=1
    /VARIABLE=Var1 RESULT=Var1_Lag LAG=1
    /VARIABLE=Var1 RESULT=Var1_Shift SHIFT=-1.
```

Var1	Var1_Lead	Var1_Lag	Var1_Shift
0	1	.	.
1	9	0	0
9	0	1	1
0	1	9	9
1	.	0	0

Figure 86. Results displayed in Data View

Name	Type	Width	Decimals	Label	Values	Missing
Var1	Numeric	1	0	Original Variable	{0, No}...	9
Var1_Lead	Numeric	1	0	Lead(Var1,1)	{0, No}...	9
Var1_Lag	Numeric	1	0	Lag(Var1,1)	{0, No}...	9
Var1_Shift	Numeric	1	0	Shift(Var1,-1)	{0, No}...	9

Figure 87. Dictionary information displayed in Variable View

SHOW

```
SHOW [ALL] [BASETEXTDIRECTION] [BLANKS] [BOX] [BLOCK] [CC] [CCA] [CCB]
 [CCC] [CCD] [CCE] [CACHE] [CTEMPLATE] [DECIMAL] [DEFOLANG] [DIGITGROUPING]
 [DIRECTORY] [ENVIRONMENT] [EPOCH] [ERRORS] [EXTPATHS] [FILTER] [FORMAT] [FUZZBITS]
 [HANDLES] [HEADER] [LEADZERO] [LENGTH] [LICENSE] [LOCALE] [MDISPLAY] [MESSAGES] [MEXPAND] [MIOUTPUT]
 [MITERATE] [MNEST] [MPRINT] [MXCELLS] [MXERRS] [MXLOOPS] [MXWARNS]
 [N] [OATTRS] [ODISPLAY] [OLANG] [ONUMBERS] [OVARS] [PLUGINS] [PRINTBACK] [REPDEFER] [RESULTS]
 [RNG] [SCALEMIN] [SCOMPRESSION] [SEED] [SMALL] [SORT] [SPLIT] [SYSMIS] [TABLERENDER]
 [TFIT] [TLOOK] [TMSRECORDING] [TNUMBERS] [TVARS] [UNDEFINED] [UNICODE] [VERSION]
 [WEIGHT] [WIDTH] [WORKSPACE] [XVERSION] [ZCOMPRESSION] [$VARS]
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 13.0

- BLKSIZE and BUFNO subcommands are obsolete and no longer supported.
- HANDLES subcommand introduced.
- SORT subcommand introduced.

Release 15.0

- TMSRECORDING subcommand introduced.

Release 16.0

- UNICODE subcommand introduced.
- MCACHE subcommand introduced.
- THREADS subcommand introduced.

Release 17.0

- FUZZBITS subcommand introduced.

Release 18.0

- EXTPATHS subcommand introduced.
- ZCOMPRESSION subcommand introduced.
- COMPRESSION subcommand removed because it is obsolete.
- REPDEFER subcommand introduced.

Release 19.0

- TABLERENDER subcommand introduced.
- XVERSION subcommand introduced.
- OATTRS subcommand introduced.
- DIGITGROUPING subcommand introduced.
- CMPTRANS subcommand introduced.

Release 21.0

- ODISPLAY subcommand introduced.

Release 22.0

- PLUGINS subcommand introduced.

Release 24.0

- LEADZERO subcommand introduced.

Release Subscription (November 2017 update)

- MDISPLAY subcommand introduced.

Example

```
SHOW.
```

Overview

SHOW displays current settings for running options. Most of these settings can be changed by using the SET command.

Basic Specification

The basic specification is simply the command keyword, which displays important current settings (keyword ALL). Some displayed option settings are applicable only when you have options such as Tables and Categories.

Subcommand Order

Subcommands can be named in any order.

Syntax

- If any subcommands are specified, only the requested settings are displayed.
- SHOW can be specified more than once.

Example

```
SHOW BLANKS /UNDEFINED /MXWARNS.
```

- BLANKS shows the value to which a completely blank field for a numeric variable is translated.
- UNDEFINED indicates whether a message is displayed whenever the program encounters anything other than a number or a blank as the value for a numeric variable.
- MXWARNS displays the maximum number of warnings that are allowed before a session is terminated.

Subcommands

The following alphabetical list shows the available subcommands.

ALL. Display important settings that are applicable to your system. This setting is the default.

BASETEXTDIRECTION. The direction for text flow. The setting is AUTOMATIC, RIGHTTOLEFT, or LEFTTORIGHT. The default setting is AUTOMATIC

BLANKS. Value to which a blank field for a numeric variable is translated. The default is the system-missing value.

BOX. Characters that are used to draw boxes. Both character and hexadecimal representations are displayed. The default is X'2D7C2B2B2B2B2B2B2B2B'. This setting applies only to text output from the program.

BLOCK. Character that is used to draw bar charts. Both character and hexadecimal representations are displayed. The default is X'2A'. This setting applies only to the text output from the program.

CC. Custom currency formats. CC shows the current custom currency formats that are defined for CCA, CCB, CCC, CCD, and CCE on SET. In Windows environments, the formats reflect the Regional Settings Properties. You can also request any of these keywords individually.

CACHE. Cache active data file. This setting shows the number of changes in the active data file before a cache file is created. The default is 5.

CMPTRANS. Override IBM SPSS Statistics Server setting for compiled transformations. The setting is either ON or OFF (alias YES or NO). There is no default because the default behavior is determined by the IBM SPSS Statistics Server.

CTEMPLATE. Chart template file. The setting is either NONE or a file name.

DECIMAL. Decimal indicator. This setting indicates the current character that is used as the decimal indicator. DOT indicates that a period is the decimal indicator; COMMA indicates that a comma is the decimal indicator.

DEFOLANG. Default output language. This setting indicates the default output language to use if the language that is specified on the OLANG subcommand is not available. The initial default setting is the language version of the installed software.

DIGITGROUPING. Whether the current IBM SPSS Statistics locale's digit grouping format is applied to numeric output. The setting is YES (alias ON) or NO (alias OFF). The default is NO.

DIRECTORY. Default directory. This setting indicates the root directory that is used to determine the locations of files that are specified with no paths or relative paths. A wide variety of actions can change the current default directory during a session.

ENVIRONMENT. Operating system and computer information. This setting includes information about environment variables, defined paths, and domain.

EPOCH. Range of years for date-format variables and date functions that are entered with a two-digit year value. AUTOMATIC indicates a 100-year range that begins 69 years before the current year and 30 years after the current year.

ERRORS. Error messages for text output. The setting can be LISTING (alias YES or ON) or NONE (alias NO or OFF).

EXTENSIONS. No longer supported.

EXTPATHS. Displays the locations for installing and searching for custom dialogs, extension commands, and R packages. When more than one location exists, location 1 takes precedence over location 2, and so on. The location R_HOME (in R_HOME\library) specifies the directory where R is installed.

FILTER. Filter status. This setting indicates whether filtering is in effect (FILTER command) and indicates the filter variable that is in use (if any).

FORMAT. Default print and write formats for numeric variables that are defined on DATA LIST in freefield format and all numeric variables that are created by transformation commands. The default is F8.2.

FUZZBITS. Default setting of the fuzzbits argument for the RND and TRUNC functions. The default is 6.

HANDLES. Displays a list of all file handles currently in effect. File handles are created with the FILE HANDLE command.

HEADER. Headings for text output. The setting is YES, NO, or BLANK. The default is NO.

JOURNAL. No longer supported.

LEADZERO. Whether leading zeros are displayed for numeric values that consist only of a decimal part. The setting is either YES or NO.

LENGTH. Maximum page length for output. The default is 59. This setting applies only to the text output from the program.

LICENSE. Licensed components, expiration date, release number, and maximum number of users who are allowed by the license.

LOCALE. Operating system locale setting and code page. In Windows operating systems, locale is set in the Regional Options of the Control Panel.

MCACHE. Memory cache size in kilobytes. The default and recommended setting are based on your specific hardware.

MDISPLAY. When defined, the subcommand uses pivot tables to display matrices. When the subcommand is not defined, the default behavior is to display matrices to TEXT.

MESSAGES. Resource-utilization messages for text output. The setting can be LISTING (alias YES or ON) or NONE (alias NO or OFF).

MEXPAND. Macro expansion. The setting is either ON (alias YES) or OFF (alias NO). The default is ON.

MIOUTPUT. Displays the settings for pooled output when a multiply imputed dataset is analyzed.

MITERATE. Maximum loop iterations that are allowed in macro expansions. The default is 1000.

MNEST. Maximum nesting level for macros. The default is 50.

MPRINT. Inclusion of expanded macros in the output. The setting is either ON (alias YES) or OFF (alias NO). The default is OFF.

MXCELLS. Maximum number of cells that are allowed for a new pivot table. The default is AUTOMATIC, which allows the number to be determined by the available memory.

MXERRS. Maximum number of errors that are allowed and number of errors so far in current session. In most implementations of IBM SPSS Statistics, the maximum number of errors that are defined on SET MXERRS is ignored. However, the information that SHOW MXERRS provides about number of errors in the current session can be useful.

MXLOOPS. Maximum executions of a loop on a single case. The default is 40.

MXMEMORY. No longer supported.

MXWARNS. Maximum number of warnings and errors that are shown for text output. The default is 10.

N. Unweighted number of cases in the active dataset. N displays UNKNOWN if there is no active dataset. N cannot be changed with SET.

OATTRS. Whether OXML and SAV output from OMS contains English-only attribute values and English-only variable names respectively. The setting is OLANG, ENG, or BOTH. The default is OLANG.

ODISPLAY. Displays what type of output is generated for procedures that can generate either Model Viewer output or pivot table and chart output. The setting is MODELVIEWER or TABLES. The default is MODELVIEWER. For version 27, ODISPLAY applies only to output from the Generalized Linear Mixed Models procedure (GENLINMIXED command).

OLANG. Output language for pivot tables.

ONUMBERS. Display of variable values in the outline for pivot tables. The settings can be LABELS, VALUES, and BOTH.

OVARS. Display of variables as headings. The settings can be LABELS, NAMES, and BOTH.

PLUGINS. Displays which programmability plug-ins are installed. For Python and R, the path to the Python or R home directory is also shown.

PRINTBACK. Command print back. The setting can be BOTH (alias LISTING, YES, or ON) or NONE (alias NO or OFF). The default is BOTH at system installation.

REPDEFER. Whether connection requests to an IBM SPSS Collaboration and Deployment Services Repository are deferred until needed. The setting is YES (alias ON) or NO (alias OFF). The default is NO unless you are running command syntax with the IBM SPSS Statistics Batch Facility (available with IBM SPSS Statistics Server), in which case the default is YES.

RESULTS. Output from commands. This setting is not applicable to output that is displayed in pivot tables. The setting can be LISTING (alias YES or ON) or NONE (alias NO or OFF).

RNG. Random number generator. Indicates which random number generator is being used.

SCALEMIN. For data files that were created in versions before version 8.0, the minimum number of unique values for a numeric variable that is used to classify the variable as scale. This setting affects only data files created in versions before version 8.0 that are opened in later versions.

SCOMPRESSION . Compression of IBM SPSS Statistics data files. This setting can be overridden by the COMPRESSED or UNCOMPRESSED subcommands on the SAVE or XSAVE commands. The default setting varies by system. SCOMPRESSION cannot be changed with SET.

SEED. Seed for the random-number generator. The default is generally 2,000,000 but might vary by system.

SMALL. Decimal value to control display of scientific notation in output.

SORT. Sorting mechanism that is in effect: IBM SPSS Statistics or external, third-party (if available).

SPLIT. Split file status.

SYSMIS. The system-missing value. SYSMIS cannot be changed with SET.

TABLERENDER. Method for rendering tables. The settings can be FULL or LIGHT. Lightweight tables render faster but cannot be pivoted or edited.

TFIT. Adjust column widths in pivot tables. The settings can be BOTH (label and data) and LABELS.

THREADS. Number of threads. The default and recommended setting are based on your specific hardware.

TLOOK. Pivot table template file. The setting can be either NONE or a file name.

TMSRECORDING. TMS recording status. Yes indicates that a TMS block for writing transformations to PMML is in effect. No indicates that TMS is not currently in effect. For more information, see "TMS BEGIN" on page 1977.

TNUMBERS. Display of variable values in pivot tables. The settings can be VALUES, LABELS, and BOTH.

TVARS. Display of variables as headings. The settings can be NAMES, LABELS, and BOTH.

UNDEFINED. Warning message for undefined data. YES is the value that is associated with the default setting of SET UNDEFINED=WARN.NO is the value that is associated with SET UNDEFINED=NOWARN, which suppresses messages but does not alter the count of warnings toward the MXWARNS total.

UNICODE. Unicode mode status. The setting is either YES (alias ON) or NO (alias OFF).

VERSION. Version number and creation date.

WEIGHT. Variable that is used to weight cases. WEIGHT can be specified for SHOW only and cannot be changed with SET.

WIDTH. Maximum page width for the output. The default is 132 columns for batch mode and 80 for interactive mode. This setting applies only to text output from the program.

WORKSPACE. Special workspace memory limit in kilobytes. The default is 6148.

XVERSION. Version of Output XML schema that is used to generate OXML output from OMS. The setting is DEFAULT or the version number (if the version differs from the default one). The default is the value DEFAULT.

ZCOMPRESSION . Override Server setting for compression of scratch files. The setting is either ON or OFF (alias YES or NO). There is no default because the default behavior is determined by the IBM SPSS Statistics Server.

$VARS. Values of system variables. $VARS cannot be changed with SET.

SIMPLAN

SIMPLAN is available in the Statistics Base Edition.

```
SIMPLAN CREATE [SOURCE=AUTOFIT** | 'filespec']

[/MODEL
   FILE='filespec'
   [CUSTOM={YES }]
           {NO**}

[/TARGETOPTS
   TARGET=fieldSpec [fieldSpec [...] ]
   [COMMENT='string']
   [OUTPUT=YES** | NO]
   [TARGETFILTER = {NO**}
                  {YES }([MIN={value[({INCLUSIVE**})]}] [MAX={value[({INCLUSIVE**})]}] )
                                 {EXCLUSIVE  }                  {EXCLUSIVE  }
                              {LO**                 }       {HI**                 }


/SIMINPUT
   INPUT=fieldSpec [fieldSpec [...] ]
   [OUTPUT=YES** | NO]
   [TYPE= {MANUAL   }([LOCK={YES**}] [SAVEASFITTED={YES }])
                          {NO   }                 {NO**}

          {AUTOFIT**}({[DEFAULT**]                                          }
                      {[CATEGORICAL]                                        }
                      {[BINOM] [NEGBIN] [POISSON]                           }
                      {[BETA] [EXP] [GAMMA] [LNORMAL] [NORMAL] [TRIANGULAR]}
                       [UNIFORM] [WEIBULL]
                     )
   [DISTRIBUTION= {BERNOULLI(PROB=value...)                                                    }
                  {BETA(SHAPE1=value... SHAPE2=value...)                                       }
                  {BINOM(N=value... PROB=value...)                                             }
                  {CATEGORICAL(CATS=valuelist PROBS=valuelist [;valuelist [;...]] [CONTINGENCY={YES }])}
                                                                                              {NO**}
                  {EMPIRICAL[(SOURCE=AUTOFIT** | 'filespec')]                                  }
                  {EXP(SCALE=value...)                                                         }
                  {GAMMA(SHAPE=value... SCALE=value...)                                        }
                  {LNORMAL(A=value... B=value...)                                              }
                  {NEGBIN([TYPE={FAILURES**}] THRESHOLD=value... PROB=value...)                }
                                {TRIALS    }
                  {NORMAL(MEAN=value... STDDEV=value...)                                       }
                  {POISSON(MEAN=value...)                                                      }
                  {TRIANGULAR(MIN=value... MAX=value... MODE=value...)                         }
                  {UNIFORM(MIN=value... MAX=value...)                                          }
                  {USER_RANGES(MIN=valuelist MAX=valuelist PROBS=valuelist [;valuelist [;...]])}
                  {WEIBULL(A=value... B=value... [C=value...])                                 }
   [MINVAL=number]
   [MAXVAL=number]

[/FIXEDINPUT
   INPUT=fieldSpec [fieldSpec [...] ]
   [VALUE=value]
   [ITERATIONS={VALUES(valuelist)                  } ]
               {RANGE(MIN=value MAX=value N=integer)}
   [LOCK=YES | NO**]

[/CORRELATIONS
  VARORDER= varlist
  CORRMATRIX= 1; value, 1; ... [value, value, ... , 1]
  [LOCK=YES | NO**]

[/CONTINGENCY
  MULTIWAY={YES }
           {NO**}

[/AUTOFIT
  [NCASES={integer}]
          {ALL**  }
  [FIT= AD** | KS]
  [MAXITERATIONS={100**  }]
                 {integer}
  [BINS={100**  }]
        {integer}

[/STOPCRITERIA
  [MAXCASES={100000**}]
            {integer }
  [TARGET=fieldName]
  [TYPE={MEAN}(THRESH=value UNIT={PERCENT} [CILEVEL={95** }])
                                 {VALUE   }          {value}
        {TAIL}(UNIT={PERCENTILE} THRESH=value [SIDE={RIGHT**}]  FREQUENCY=integer)
                    {VALUE     }                  {LEFT   }

[/MISSING CLASSMISSING = {EXCLUDE**}
                         {INCLUDE  }

[/VALUELABELS
  VARIABLES = varlist
  VALUES = value 'label' [value 'label' [...]]

/PLAN
 FILE='filespec'
 [DISPLAY=YES** | NO]
```

SIMPLAN CREATE reads the active dataset and causes execution of any pending transformations.

Display an existing plan

```
SIMPLAN VIEW
/PLAN FILE='filespec'
```

SIMPLAN VIEW causes execution of any pending transformations.

** Default if the subcommand or keyword is omitted.

Release History

Release 21.0

- Command introduced.

Release 22.0

- LOCK keyword introduced on FIXEDINPUT subcommand.
- CONTINGENCY subcommand added.
- CONTINGENCY keyword added to specifications for CATEGORICAL distribution on SIMINPUT subcommand.
- Added global SOURCE keyword and deprecated SOURCE keyword for DISTRIBUTION=EMPIRICAL.
- MISSING subcommand added.
- VALUELABELS subcommand added.

Example

```
SIMPLAN CREATE
/MODEL FILE='/models/mymodel.xml'
/SIMINPUT INPUT=labor materials
/PLAN FILE='/simulations/myplan.splan'.

SIMPLAN VIEW
/PLAN FILE='/simulations/myplan.splan'.
```

Overview

The SIMPLAN command creates a simulation plan that contains all of the specifications required to run a simulation. The simulation plan is used as input to the SIMRUN command, which runs the simulation.

Simulation in IBM SPSS Statistics refers to simulating input data to predictive models, evaluating the model based on the simulated data and using the distribution of predicted target values to evaluate the likelihood of various outcomes. Inputs to the model are specified as fixed or simulated. Values for simulated inputs are drawn from a specified probability distribution. Fixed inputs are those whose values are known and remain constant when evaluating the model. The simulation plan specifies the model to be simulated, probability distributions for simulated inputs and values for fixed inputs, as well as various other settings.

You can also simulate data in the absence of a predictive model by specifying probability distributions for variables that are to be simulated. The simulation plan then specifies those probability distributions, as well as various other settings.

Options

Probability distributions for simulated inputs. You can automatically determine the probability distribution that most closely fits the data, in the active dataset, associated with each simulated input. You can also manually specify distributions if historical data are not available or you require specific distributions.

Correlations. You can automatically determine correlations between simulated inputs from historical data for each input in the active dataset. You can also manually specify correlations between simulated inputs if historical data are not available or you require specific correlations.

Associations for categorical inputs. You can automatically compute a multiway contingency table, from the historical data, that describes the associations between inputs with a categorical distribution. The contingency table is then used when generating data for those inputs.

Input and target filtering. You can specify the valid range of values for both simulated inputs and targets. For simulated inputs, only values in the specified range will be used in simulated data. For targets, values outside of the specified range are excluded from all output and analyses.

Specify sensitivity analysis. Sensitivity analysis allows you to investigate the effect of systematic changes in a fixed input or in a distribution parameter for a simulated input by generating an independent set of simulated data—effectively, a separate simulation—for each specified value.

Stopping criteria. You can specify criteria for stopping the simulation, potentially before the maximum number of allowable cases of simulated data have been generated. In particular, you can specify to continue generating cases until the confidence interval of the mean of a target is within a specified threshold, or you can specify to continue generating cases until one of the tails of a target has been sampled to a specified degree.

Creating a simulation plan that includes a predictive model

The basic specification is the SIMPLAN command with the CREATE keyword, followed by the MODEL subcommand, one or more SIMINPUT subcommands for simulated inputs and the PLAN subcommand specifying where the simulation plan should be saved.

Creating a simulation plan without a predictive model

The basic specification is the SIMPLAN command with the CREATE keyword, followed by one or more SIMINPUT subcommands for simulated inputs and the PLAN subcommand specifying where the simulation plan should be saved.

Displaying an existing plan

The basic specification is the SIMPLAN command with the VIEW keyword, followed by the PLAN subcommand which specifies the simulation plan to be displayed.

Syntax rules

- The PLAN subcommand, and at least one SIMINPUT subcommand are required. All other subcommands are optional.
- Subcommands can be specified in any order.
- There can be multiple instances of SIMINPUT, FIXEDINPUT, TARGETOPTS, and VALUELABELS subcommands. Only a single instance of any other subcommand is allowed.
- A keyword cannot be specified more than once within a subcommand.
- Only one SIMINPUT subcommand can specify iterations of distribution parameters.
- If multiple input fields are specified for a SIMINPUT subcommand, then you cannot specify iterations of distribution parameters for that subcommand.
- Only one FIXEDINPUT subcommand can contain the ITERATIONS keyword.
- If multiple input fields are specified for a FIXEDINPUT subcommand, then the subcommand cannot contain the ITERATIONS keyword.
- If a FIXEDINPUT subcommand contains the ITERATIONS keyword, then iterations of distribution parameters cannot be specified for a SIMINPUT subcommand.
- Parentheses, equals signs, and slashes shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not allowed.

Limitations

- SPLIT FILE is ignored by this command.

Examples

Automatically fitting distributions and correlations

```
SIMPLAN CREATE
 /MODEL FILE='/models/sales_model.xml'
 /SIMINPUT INPUT=advert agents cci(MAPTO=confidence_index)
 /PLAN FILE='/simulations/sales.splan'.
```

- The MODEL subcommand specifies the model file. Since the CUSTOM keyword is omitted, the model file is assumed to be an IBM SPSS model file (an XML file that contains model PMML created from IBM SPSS Statistics or IBM SPSS Modeler).
- The SIMINPUT subcommand specifies that the model fields *advert*, *agents* and *cci* are simulated inputs. Since the TYPE keyword on SIMINPUT is omitted, the default of TYPE=AUTOFIT is used. This results in automatically finding the distribution that most closely fits the data in the active dataset associated with each of the simulated inputs.
- The MAPTO keyword for the specification of *cci* indicates that *cci* maps to the field named *confidence_index* in the active dataset. Data for *confidence_index* will then be used to find the closest fitting distribution for *cci*.
- Since the CORRELATIONS subcommand is omitted, correlations between the simulated inputs will be determined from the data for those inputs in the active dataset. The mapping of the model field *cci* to the dataset field *confidence_index* applies to calculations of correlations, as well as to distribution fitting.
- The PLAN subcommand specifies that the simulation plan will be saved to the file */simulations/sales.splan*.

Manually specifying distributions

```
SIMPLAN CREATE
/MODEL FILE='/models/profitmodel.xml' CUSTOM=YES
/SIMINPUT INPUT=volume
   TYPE=MANUAL DISTRIBUTION=TRIANGULAR(MIN=5000 MAX=7500 MODE=6250)
/SIMINPUT INPUT=unit_cost_materials
   TYPE=MANUAL DISTRIBUTION=TRIANGULAR(MIN=10 MAX=15 MODE=13)
/FIXEDINPUT INPUT=price VALUE=89
/FIXEDINPUT INPUT=fixed VALUE=7500
/FIXEDINPUT INPUT=unit_cost_labor VALUE=14
/PLAN FILE='/simulations/profitmodel.splan'.
```

- The setting CUSTOM=YES on the MODEL subcommand specifies that this is a custom model created from a SIMPREP BEGIN-SIMPREP END block.
- The setting TYPE=MANUAL on the two SIMINPUT subcommands specifies that the probability distributions for *volume* and *unit_cost_materials* will be explicitly specified. The DISTRIBUTION keyword specifies that both inputs have Triangular distributions.
- The three FIXEDINPUT subcommands specify that the model inputs *price*, *fixed* and *unit_cost_labor* are fixed inputs. The VALUE keyword specifies the fixed value.

Specifying sensitivity analysis for a fixed input

```
SIMPLAN CREATE
/MODEL FILE='/models/profitmodel.xml' CUSTOM=YES
/SIMINPUT INPUT=volume
   TYPE=MANUAL DISTRIBUTION=TRIANGULAR(MIN=5000 MAX=7500 MODE=6250)
/SIMINPUT INPUT=unit_cost_materials
   TYPE=MANUAL DISTRIBUTION=TRIANGULAR(MIN=10 MAX=15 MODE=13)
/FIXEDINPUT INPUT=price VALUE=89
/FIXEDINPUT INPUT=fixed VALUE=7500
/FIXEDINPUT INPUT=unit_cost_labor ITERATIONS=VALUES(14 17 19)
/PLAN FILE='/simulations/profitmodel.splan'.
```

- The ITERATIONS keyword on the FIXEDINPUT subcommand for *unit_cost_labor* specifies that sensitivity analysis will be carried out and will be based on varying values of this fixed input. The VALUES keyword specifies the three values 14, 17 and 19. An independent set of simulated data will be generated for each of these three values of *unit_cost_labor*.

Generating data without a predictive model

```
SIMPLAN CREATE
 /SIMINPUT INPUT=age glucose income
 /PLAN FILE='/simulations/diabetes_data.splan'.
```

- The SIMINPUT subcommand specifies that data are to be generated for the variables *age*, *glucose* and *income*. Since the TYPE keyword on SIMINPUT is omitted, the default of TYPE=AUTOFIT is used. This results in automatically finding the distribution that most closely fits the data in the active dataset associated with each of the variables.
- Since the CORRELATIONS subcommand is omitted, correlations between the specified variables will be determined from the data for those variables in the active dataset.
- The PLAN subcommand specifies that the simulation plan will be saved to the file */simulations/diabetes_data.splan*.

MODEL subcommand

The MODEL subcommand specifies the file containing the model that is to be simulated. You can specify an IBM SPSS model file (an XML file or a compressed file archive (.zip file) that contains model PMML created from IBM SPSS Statistics or IBM SPSS Modeler) or a file created from a SIMPREP BEGIN-SIMPREP END block. SIMPREP BEGIN-SIMPREP END blocks create a model, for use with SIMPLAN, based on a set of custom equations that you define.

FILE = 'filespec'. *Specifies the model file.* File specifications should be enclosed in quotation marks and fully qualified paths are recommended to avoid ambiguity. The file is assumed to be a model file unless it has an extension of .splan, in which case it is treated as a simulation plan file from which the model file will be extracted.

Note: The optional combination of an asterisk (*) and a backslash (\) preceding the model file name specifies that the file is a temporary file. This only applies in the case that SIMPLAN CREATE follows a SIMPREP BEGIN-SIMPREP END block. See the topic “OUTFILE subcommand ” on page 1831 for more information.

CUSTOM = YES | NO. *Specifies whether the model consists of a set of custom equations.* Models for sets of custom equations are created using a SIMPREP BEGIN-SIMPREP END block. The default is NO, which specifies that the model was not created by a SIMPREP BEGIN-SIMPREP END block.

PMML models supported by Simulation

- Linear Regression
- Automatic Linear Model
- Generalized Linear Model
- Generalized Linear Mixed Model
- General Linear Model
- Binary Logistic Regression
- Multinomial Logistic Regression
- Ordinal Multnomial Regression
- Cox Regression
- Tree
- Boosted Tree (C5)
- Discriminant
- Two-step Cluster
- K-Means Cluster
- Neural Net
- Ruleset (Decision List)

Note:

- PMML models that have multiple target fields (variables) or splits are not supported for use in Simulation.
- Values of string inputs to binary logistic regression models are limited to 8 bytes in the model. If you are fitting such string inputs to the active dataset, make sure that the values in the data do not exceed 8 bytes in length. Data values that exceed 8 bytes are excluded from the associated categorical distribution for the input, and are displayed as unmatched in the Unmatched Categories output table.

TARGETOPTS subcommand

The TARGETOPTS subcommand specifies optional settings for targets.

TARGET keyword

The TARGET keyword specifies names and optional settings for one or more targets. The TARGET keyword is required when TARGETOPTS is specified. The specification for each target is the field name with an optional qualifier of the form `(FORMAT=format)`, where the parentheses are required. Use a blank space to separate specifications for multiple targets. For example:

```
TARGET=target1(FORMAT=DOLLAR) target2(FORMAT=F,4)
```

- **FORMAT.** *Specifies the output format of the field.* The format consists of a format type, such as F, optionally followed by a comma and the number of decimal places. If the number of decimal places is omitted then it is assumed that there are 0 decimal places. The following formats are supported:

Table 87. Format specifications

Format Specification	Definition
F,d	Numeric
E,d	Scientific notation
N,d	Restricted numeric
DOT,d	Numeric with dots
COMMA,d	Numeric with commas
DOLLAR,d	Numeric with commas and dollar sign
PCT,d	Numeric with percent sign
CCA,d	Custom currency
CCB,d	Custom currency
CCC,d	Custom currency
CCD,d	Custom currency
CCE,d	Custom currency

If no format is specified, then the Numeric format F is used for numeric inputs.

COMMENT = 'string'. *Specifies a description that will be associated with the targets listed on the TARGET keyword.* Use separate TARGETOPTS subcommands for targets requiring different descriptions. The COMMENT keyword is only supported for models with CUSTOM=YES on the MODEL subcommand. Comments are displayed on the Model tab of the Simulation Builder, when the simulation plan has been loaded into the Simulation Builder.

OUTPUT. *Specifies whether the targets listed on the TARGET keyword are included in table and chart output.* The default is YES.

TARGETFILTER keyword

The TARGETFILTER keyword specifies a filter to apply to the specified targets. The filter allows you to specify the range of valid values for one or more targets. Values outside of the specified range are excluded from all output and analyses associated with the targets.

- **MIN.** *Specifies the minimum value for the range.* You can use the keywords LO or LOWEST. The default is LO. By default, the value specified for MIN is included in the range. You can exclude the value from the range by using the EXCLUSIVE keyword.
- **MAX.** *Specifies the maximum value for the range.* You can use the keywords HI or HIGHEST. The default is HI. By default, the value specified for MAX is included in the range. You can exclude the value from the range by using the EXCLUSIVE keyword.

SIMINPUT subcommand

The SIMINPUT subcommand specifies input fields whose values will be simulated. It is a required subcommand. When the MODEL subcommand is used, each input in the associated model file must be specified as either simulated or fixed. Use the FIXEDINPUT subcommand to specify fixed inputs.

INPUT keyword

The INPUT keyword specifies names and optional settings for one or more simulated inputs. The INPUT keyword is required. The specification for each input is the field name with an optional qualifier of the form (MAPTO=name FORMAT=format), where the parentheses are required. Use a blank space to separate specifications for multiple inputs. The keywords TO and ALL are not supported for variable lists specified on the INPUT keyword. For example:

```
INPUT=input1(MAPTO=field1 FORMAT=F,4) input2(FORMAT=DOLLAR)
```

- **MAPTO.** *Maps a simulated field to a field in the active dataset.* The MAPTO keyword is only needed when you are automatically fitting to data in the active dataset and the name of a simulated field differs from the name of the associated field in the active dataset. The value of MAPTO should be the name of the field in the active dataset.
- **FORMAT.** *Specifies the output format of the field.* The format consists of a format type, such as F, optionally followed by a comma and the number of decimal places. If the number of decimal places is omitted then it is assumed that there are 0 decimal places. The following formats are supported:

Table 88. Format specifications

Format Specification	Definition
F,d	Numeric
E,d	Scientific notation
N,d	Restricted numeric
DOT,d	Numeric with dots
COMMA,d	Numeric with commas
DOLLAR,d	Numeric with commas and dollar sign
PCT,d	Numeric with percent sign
CCA,d	Custom currency
CCB,d	Custom currency
CCC,d	Custom currency
CCD,d	Custom currency
CCE,d	Custom currency

If no format is specified, then the Numeric format F is used for numeric inputs.

OUTPUT. *Specifies whether the inputs listed on the INPUT keyword are included in table and chart output.* The default is YES.

TYPE keyword

The TYPE keyword specifies whether the probability distribution for this simulated input field is determined by automatically fitting to the data for this field in the active dataset or by manually specifying the distribution.

- **MANUAL(LOCK = YES | NO SAVEASFITTED = YES | NO).** *Indicates that the probability distribution is manually specified.* When MANUAL is specified, the DISTRIBUTION keyword must be used unless the global SOURCE keyword specifies a simulation plan file (for more information, see the topic "SOURCE Keyword " on page 1830).

  The LOCK keyword specifies whether the distribution for this simulated input field will be locked. Locked distributions will not be modified when automatically fitting distributions interactively in the user interface, using the Simulation Builder or the Run Simulation dialog. If you are creating a simulation plan that you or someone else will work with in the user interface and you want to prevent refitting a simulated input to historical data, then be sure to specify LOCK=YES for that input. Users who open the plan in the Run Simulation dialog will not be able to make any modifications to the distribution of the input. Users who open the plan in the Simulation Builder will be able to make changes to the distribution once they unlock the input.

  The SAVEASFITTED keyword occurs when pasting syntax for a simulated input whose distribution was automatically fitted to historical data and then locked in the user interface. The keyword allows the state of the input to be stored in the plan file so that it can be restored in the user interface when re-opening the plan file. The default is SAVEASFITTED=NO.
- **AUTOFIT.** *Indicates that the probability distribution associated with this input field will be automatically determined from the data for the associated field in the active dataset.* By default, the measurement level of the field is used to determine the set of distributions that are considered.

  For nominal input fields, the default set of distributions only includes the categorical distribution.

  For ordinal input fields, the default set of distributions includes the following: binomial, negative binomial and Poisson. The chi-square test is used to determine the distribution that most closely fits the data.

  For continuous input fields, the default set of distributions includes the following: beta, exponential, gamma, lognormal, normal, triangular, uniform and Weibull. By default, the Anderson-Darling test for goodness of fit is used to determine the distribution that most closely fits the data. Optionally, you can specify the Kolmogorov-Smirnoff test for goodness of fit. See the topic "AUTOFIT Subcommand " on page 1827 for more information.

  You can override the default set of distributions by explicitly specifying one or more distributions on the AUTOFIT keyword, but you cannot mix distributions belonging to different measurement levels. For example, you can specify one or more of the distributions for ordinal data (BINOM, NEGBIN and POISSON) but you cannot specify those in conjunction with distributions for continuous data such as NORMAL.

  Note: For the negative binominal distribution (NEGBIN), AUTOFIT uses the form of the distribution that describes the probability of a given number of failures before a given number of successes occur. If you require the alternate parameterization that describes the probability of a given number of trials before a given number of successes occur, then manually specify the distribution with the DISTRIBUTION keyword. Also note that for the Weibull distribution, AUTOFIT only considers the case where the location parameter C equals 0.

The DISTRIBUTION keyword specifies the probability distribution for a simulated input field and is used when you want to explicitly specify the probability distribution rather than have it automatically determined from the data for the associated field in the active dataset. The DISTRIBUTION keyword can only be used with TYPE = MANUAL.

BERNOULLI(PROB=value). *Bernoulli distribution.*

BETA(SHAPE1=value SHAPE2=value). *Beta distribution.*

BINOM(N=value PROB=value). *Binomial distribution.*

CATEGORICAL(CATS=valuelist PROBS=valuelist CONTINGENCY=YES | NO)**. *Categorical distribution.* The Categorical distribution describes an input field that has a fixed number of values, referred to as categories. Each category has an associated probability such that the sum of the probabilities over all categories equals one.

- The CATS keyword specifies a list of the categories and the PROBS keyword specifies a list of the probabilities associated with each category. The n-th item in each list specifies the associated value for the n-th category. The number of values in each of the lists must be the same.
- For string inputs with a Categorical distribution, the category values can be specified with or without quotation marks; however, numeric categories for such inputs must be enclosed in quotation marks.
- The CONTINGENCY keyword specifies whether the input is included in a multiway contingency table that is computed from the active dataset and that describes associations between inputs with categorical distributions. By default, the input is not included in a contingency table. When CONTINGENCY=YES, MULTIWAY=YES must be specified on the CONTINGENCY subcommand.

  Because the contingency table is computed from the active dataset, inputs with CONTINGENCY=YES must either exist in the active dataset or be mapped to a field in the active dataset with the MAPTO keyword. In addition, when CONTINGENCY=YES values specified for the CATS and PROBS keywords are ignored because the categories and category probabilities are determined from the contingency table.

  Inputs specified as TYPE=AUTOFIT that are fit to a categorical distribution are automatically included in the contingency table when MULTIWAY=YES is specified on the CONTINGENCY subcommand.

Note: If the source of your predictive model is a PMML model file and you are specifying a categorical distribution for an input that is categorical in the model (such as gender), then you should specify the same category values as used in the model (or a subset of those values).

EMPIRICAL([SOURCE=AUTOFIT | 'filespec'])**. *Empirical distribution.* The empirical distribution is calculated from the data in the active dataset corresponding to the input field. EMPIRICAL is only supported for inputs with a continuous or ordinal measurement level.

- For continuous inputs, the empirical distribution is the cumulative distribution function of the data.
- For ordinal inputs, the empirical distribution is the categorical distribution of the data.
- For nominal input fields, use TYPE=AUTOFIT(CATEGORICAL).
- The SOURCE keyword is deprecated. Use the global SOURCE keyword instead, which has the same specifications but applies to both contingency tables and parameters for empirical distributions. See the topic "SOURCE Keyword " on page 1830 for more information.

EXP(SCALE=value). *Exponential distribution.*

GAMMA(SHAPE=value SCALE=value). *Gamma distribution.*

LNORMAL(A=value B=value). *Lognormal distribution.*

NEGBIN(TYPE=FAILURES | TRIALS THRESHOLD=value PROB=value). *Negative Binomial distribution.* Two parameterizations of the negative binomial distribution are supported.

- TYPE=FAILURES specifies a distribution that describes the probability of a given number of failures before a given number of successes occur.
- TYPE=TRIALS specifies a distribution that describes the probability of a given number of trials before a given number of successes occur, and is the parameterization used in the command syntax function PDF.NEGBIN.

NORMAL(MEAN=value STDDEV=value). *Normal distribution.*

POISSON(MEAN=value). *Poisson distribution.*

TRIANGULAR(MIN=value MAX=value MODE=value). *Triangular distribution.*

UNIFORM(MIN=value MAX=value). *Uniform distribution.*

USER_RANGES(MIN=valuelist MAX=valuelist PROBS=valuelist). *User-defined ranges.* This distribution consists of a set of intervals with a probability assigned to each interval such that the sum of the probabilities over all intervals equals 1. Values within a given interval are drawn from a uniform distribution defined on that interval.

- The MIN keyword specifies a list of the left endpoints of each interval, the MAX keyword specifies a list of the right endpoints of each interval, and the PROBS keyword specifies a list of the probabilities associated with each interval. The n-th item in each list specifies the associated value for the n-th interval. The number of values in each of the lists must be the same. The specified endpoints are included in the intervals.
- Intervals can be overlapping. For example, you can specify MIN = 10 12 and MAX = 15 20, which defines the two intervals [10,15] and [12,20].

WEIBULL(A=value B=value [C=value]). *Weibull distribution.* The parameter *C* is an optional location parameter, which specifies where the origin of the distribution is located. Omitting the value of *C* is equivalent to setting its value to 0. When *C* equals 0, the distribution reduces to the Weibull distribution function in command syntax (PDF.WEIBULL).

Iterating distribution parameters

For any of the above distributions, you can specify multiple values for one of the distribution parameters. An independent set of simulated cases--effectively, a separate simulation--is generated for each specified value, allowing you to investigate the effect of varying the input. This is referred to as **Sensitivity Analysis**, and each set of simulated cases is referred to as an **iteration**.

- The set of specified values for a given distribution parameter should be separated by spaces.
- For the CATEGORICAL distribution, the CATS parameter can only specify a single set of category values, but you can specify multiple sets of values for the PROBS parameter, allowing you to vary the set of probabilities associated with the categories. Each set of probabilities should be separated by a semicolon.
- For the USER_RANGES distribution, the MIN and MAX parameters can only specify a single set of intervals, but you can specify multiple sets of values for the PROBS parameter, allowing you to vary the set of probabilities associated with the specified intervals. Each set of probabilities should be separated by a semicolon.
- You can only iterate distribution parameters for a single simulated input. An error results if you specify iterations of distribution parameters and there are multiple fields on the INPUT keyword.

Example: Normal distribution with iterated parameters

The following specification results in two iterations, one with NORMAL(MEAN=15 STDDEV=2) and one with NORMAL(MEAN=15 STDDEV=3).

```
DISTRIBUTION= NORMAL(MEAN=15 STDDEV=2 3)
```

Example: Categorical distribution with iterated sets of probabilities

The example shows three iterations for the set of probabilities associated with the specified set of categories.

```
DISTRIBUTION= CATEGORICAL(CATS=1 2 3 PROBS=0.5 0.25 0.25; 0.4 0.3 0.3; 0.2 0.6 0.2)
```

The CATS keyword specifies the set of categories. The probabilities for the first iteration are specified by the set of values up to the first semi-colon following the PROBS keyword. Thus the category with value 1 has probability 0.5, the category with value 2 has probability 0.25 and the category with value 3 has probability 0.25. The probabilities for the second and third iterations are (0.4, 0.3, 0.3) and (0.2, 0.6, 0.2) respectively.

Example: User Ranges distribution with iterated sets of probabilities

The example shows two iterations for the set of probabilities associated with the specified intervals.

```
DISTRIBUTION= USER_RANGES(MIN=10 13 17 MAX=12 16 20 PROBS=0.3 0.3 0.4; 0.2 0.3 0.5)
```

The MIN and MAX keywords specify the three intervals [10-12], [13-16] and [17-20]. For the first iteration, the interval from 10 to 12 has probability 0.3, the interval from 13 to 16 has probability 0.3 and the interval from 17 to 20 has probability 0.4. The probabilities for the second iteration are (0.2, 0.3, 0.5).

MINVAL Keyword

The MINVAL keyword specifies the minimum allowed value for the simulated input field. If MINVAL is omitted, the minimum value is determined by the range of the associated probability distribution. If the specified value is less than the minimum allowed for the associated probability distribution, then the minimum for the probability distribution is used. MINVAL is not supported for the following distributions: Bernoulli, categorical, empirical, triangular, uniform and user ranges.

MAXVAL Keyword

The MAXVAL keyword specifies the maximum allowed value for the simulated input field. If MAXVAL is omitted, the maximum value is determined by the range of the associated probability distribution. If the specified value is greater than the maximum allowed for the associated probability distribution, then the maximum for the probability distribution is used. MAXVAL is not supported for the following distributions: Bernoulli, categorical, empirical, triangular, uniform and user ranges.

FIXEDINPUT Subcommand

The FIXEDINPUT subcommand specifies input fields whose values are fixed. When the MODEL subcommand is used, each input in the model file must be specified as either simulated or fixed. Use the SIMINPUT subcommand to specify simulated inputs.

INPUT Keyword

The INPUT keyword specifies names and optional settings for one or more fixed inputs. The INPUT keyword is required. The specification for each input is the field name with an optional qualifier of the form (FORMAT=format), where the parentheses are required. Use a blank space to separate specifications for multiple inputs. The keywords TO and ALL are not supported for variable lists specified on the INPUT keyword. For example:

```
INPUT=fixed1(FORMAT=PCT)
```

- **FORMAT.** *Specifies the output format of the field.* The format consists of a format type, such as F, optionally followed by a comma and the number of decimal places. If the number of decimal places is omitted then it is assumed that there are 0 decimal places. The following formats are supported:

Table 89. Format specifications

Format Specification	Definition
F,d	Numeric
E,d	Scientific notation
N,d	Restricted numeric
DOT,d	Numeric with dots
COMMA,d	Numeric with commas
DOLLAR,d	Numeric with commas and dollar sign
PCT,d	Numeric with percent sign
CCA,d	Custom currency
CCB,d	Custom currency
CCC,d	Custom currency
CCD,d	Custom currency

Table 89. Format specifications (continued)	
Format Specification	**Definition**
CCE,d	Custom currency

If no format is specified, then the Numeric format F is used for numeric inputs.

VALUE. *Specifies the fixed value for the fields listed on the INPUT keyword.* Use separate FIXEDINPUT subcommands for fixed inputs that have different values. VALUE cannot be used with the ITERATIONS keyword. String values should be enclosed in quotation marks.

ITERATIONS keyword

The ITERATIONS keyword specifies a set of values for a fixed input. An independent set of simulated cases--effectively, a separate simulation--is generated for each specified value, allowing you to investigate the effect of varying the input. This is referred to as sensitivity analysis, and each set of simulated cases is referred to as an iteration.

- ITERATIONS cannot be used with the VALUE keyword.
- You can only iterate values for a single fixed input field. An error results if the ITERATIONS keyword is specified and there are multiple fields on the INPUT keyword.

VALUES(valuelist). *Specifies an explicit set of values for the fixed input.* Values should be separated by spaces and string values should be enclosed in quotation marks. The list cannot contain a mixture of string and numeric values.

RANGE(MIN=value MAX=value N=integer). *Specifies a range of equally spaced discrete numeric values for the fixed input.* The parameter N specifies the number of values, including the values specified for MIN and MAX. For example, to specify the list [20, 30, 40, 50], use RANGE(MIN=20 MAX=50 N=4).

LOCK = YES | NO. *Specifies whether the fixed input will be locked.* If you are creating a simulation plan for use in the Run Simulation dialog and you want to prevent users from changing the value of the fixed input then specify LOCK=YES. The default is LOCK=NO. Locked inputs can be modified in the Simulation Builder but not in the Run Simulation dialog.

CORRELATIONS Subcommand

The CORRELATIONS subcommand specifies the correlation matrix that describes any correlations between the simulated input fields. Correlations between inputs that will be simulated must be accounted for in order to ensure that the simulated values preserve those correlations. If the CORRELATIONS subcommand is omitted, correlations will be automatically determined from the fields in the active dataset associated with the simulated input fields. In that case, correlations for fields that do not exist in the active dataset are set to 0.

VARORDER. *A list of variable names specifying the variable associated with each row in the correlation matrix.* The n-th row in the correlation matrix specifies the correlations associated with the n-th variable in the list. The list must contain all input fields, except any fields that meet one of the following conditions:

- The field has a nominal measurement level and its distribution is to be automatically determined (TYPE=AUTOFIT).
- The distribution for the field has been explicitly set to the categorical distribution.
- The field has been specified as a fixed input.

CORRMATRIX. *A set of lists of values specifying the lower triangular portion of the correlation matrix.* The n-th list specifies the n-th row of the lower triangular portion of the correlation matrix, including the diagonal element which always has the value 1. The number of lists must equal the number of fields specified on the VARORDER keyword. In the case that two input fields are uncorrelated, the associated element in the correlation matrix should be set to 0. Specified values must be between -1 and 1.

LOCK = YES | NO. *Specifies whether the correlation matrix will be locked.* If you are creating a simulation plan that you or someone else will work with in the user interface (in the Simulation Builder or the Run Simulation dialog) and you want to prevent recalculation of correlations from historical data (carried out as part of distribution fitting), then specify LOCK=YES. The default is LOCK=NO.

Example

Consider a model with continuous inputs *input1*, *input2* and *input3*, all of which will be simulated. Assume the correlation matrix for these inputs is:

```
          input1  input2  input3
input1         1   0.091   0.011
input2     0.091       1  -0.105
input3     0.011  -0.105       1
```

The following specification represents this correlation matrix:

```
/CORRELATIONS
  VARORDER= input1 input2 input3
  CORRMATRIX= 1; 0.091, 1; 0.011, -0.105 1
```

CONTINGENCY Subcommand

The CONTINGENCY subcommand specifies whether a multiway contingency table that describes associations between inputs with a categorical distribution is to be used when data are generated for those inputs. The multiway contingency table is computed from the active dataset and includes all inputs that are fit to a categorical distribution. It ensures that associations between those inputs in the historical data are preserved when simulating data for those inputs.

MULTIWAY = YES | NO
Specifies whether a multiway contingency table is used. The default is NO, which specifies that inputs with a categorical distribution are independent of each other.

Note: You can use the global SOURCE keyword to specify a simulation plan file that contains a multiway contingency table that is to be used instead of computing the table from the active dataset. For more information, see the topic “SOURCE Keyword ” on page 1830.

AUTOFIT Subcommand

The AUTOFIT subcommand specifies options for automatic fitting of probability distributions and correlations to fields in the active dataset.

NCASES. *Specifies the number of cases in the active dataset to be used for automatic fitting of probability distributions and correlations.* For a specification NCASES = n, the first n cases will be used. The default is all cases.

FIT = AD | KS. *Specifies whether to use the Anderson-Darling or Kolmogorov-Smirnoff test for goodness of fit when automatically fitting probability distributions for simulated input fields.* Only applies to simulated input fields with a continuous measurement level. The default is Anderson-Darling (FIT = AD). For input fields with ordinal or nominal measurement levels, a chi-square test is used.

MAXITERATIONS. *Specifies the maximum number of iterations SIMPLAN can go through in its computations when automatically fitting probability distributions.* The default is 100.

BINS. *Specifies the number of bins used when fitting the empirical distribution for a continuous input field.* The default is 100 and the maximum is 1000. The empirical distribution is the cumulative distribution function of the historical data.

STOPCRITERIA subcommand

The STOPCRITERIA subcommand specifies criteria for stopping the simulation, potentially before the maximum number of allowable cases has been generated.

MAXCASES. *Specifies the maximum number of cases of simulated data to generate.* The default is 100000. A larger value for MAXCASES will allow for more accurate predictions but will also require a longer execution time to complete the simulation.

- When the TARGET and TYPE keywords are omitted, this is the number of cases that will be generated.
- When sensitivity analysis is specified, this is the maximum number of cases for each iteration.
- When the MODEL subcommand is omitted, MAXCASES is the only option.

TARGET. *The name of the target to which the criteria specified on the TYPE keyword are applied.*

TYPE Keyword

The TYPE keyword specifies the criteria for stopping the simulation. The TYPE keyword is ignored if the TARGET keyword is omitted.

MEAN. *Specifies to continue generating cases until the confidence interval of the mean of the target distribution is within a specified threshold.* To use this option, you specify a confidence level and a threshold. For example, you can use this option to specify that cases are generated until the confidence interval of the mean at the 95% confidence level is within 5% of the mean value.

- The keyword THRESH specifies the threshold value for the confidence interval at the confidence level specified by the CILEVEL keyword. The default for CILEVEL is 95, representing a confidence level of 95%.
- The keyword UNIT indicates whether the value specified for THRESH represents a percentage of the mean value of the target (UNIT=PERCENT) or a numeric value (UNIT=VALUE). For example, if your target is revenue and you want to ensure that the width of the 95% confidence interval of the mean is less than $1000 then you would specify THRESH=1000 UNIT=VALUE. When UNIT=PERCENT, THRESH is specified as an integer, as in the value 5 for 5%.
- Generation of cases will continue until the criteria specified in MEAN is reached or the maximum number of allowed cases (MAXCASES keyword) has been reached

TAIL. *Specifies to continue generating cases until a given number of cases have been generated in one of the tails of the distribution for the specified target.*

- For UNIT=VALUE, the THRESH keyword specifies the value of the target that defines the boundary of the tail region. By default, the value is assumed to specify the right tail region. To specify the left tail region, use SIDE=LEFT.
- For UNIT=PERCENTILE, the THRESH keyword specifies the boundary of the tail region as a percentile of the target distribution. The SIDE keyword is ignored when UNIT=PERCENTILE, since the side is implicit in the specified value of THRESH--for example, a value of 99 implies the right tail and a value of 1 implies the left tail.
- The FREQUENCY keyword specifies the number of cases that must be generated in the specified tail region.
- Generation of cases will continue until the criteria specified in TAILS is reached or the maximum number of allowed cases (MAXCASES keyword) has been reached

Examples

This example specifies that exactly 1,000,000 cases of simulated data will be generated.

```
/STOPCRITERIA MAXCASES=1000000
```

This example specifies that cases will be generated until either the width of the 99% confidence interval of the mean of the target *profit* is within 1% of the mean value, or 1,000,000 cases have been generated.

```
/STOPCRITERIA MAXCASES=1000000
 TARGET=profit
 TYPE=MEAN(THRESH=1 UNIT=PERCENT CILEVEL=99)
```

This example specifies that the boundary of the right tail region for the target *loss* is at the value 50,000,000. Cases will be generated until either 500 cases have a target value greater than 50,000,000 or 10,000,000 cases have been generated.

```
/STOPCRITERIA MAXCASES=10000000
 TARGET=loss
 TAIL(UNIT=VALUE THRESH=50000000 FREQUENCY=500)
```

MISSING Subcommand

The MISSING subcommand specifies how missing values are handled. Cases with invalid values for any input are excluded from the following analyses: distribution fitting; computation of correlations; and computation of the contingency table, if requested. System missing values are always invalid. User-missing values are invalid except for inputs that are fit to a categorical distribution when CLASSMISSING=INCLUDE.

The CLASSMISSING keyword specifies whether user-missing values of inputs that are fit to a categorical distribution are treated as valid.

EXCLUDE
Exclude user-missing values. Treat user-missing values for these inputs as invalid data. This setting is the default.

INCLUDE
Include user-missing values. Treat user-missing values for these inputs as valid data

VALUELABELS Subcommand

The VALUELABELS subcommand specifies value labels for simulated inputs and fixed inputs. It applies only in the case of simulating data without a predictive model.

VARIABLES
A list of variables to which the value labels are applied.

VALUES
A list of pairs that consist of a value followed by the associated label.

- Each value label must be enclosed in quotation marks. For string variables, the values themselves must also be quoted.
- Value labels can contain any characters, including blanks.
- To enter an apostrophe as part of a label, enclose the label in double quotation marks or enter two apostrophes (two single quotation marks).
- Each value label can be up to 120 bytes long.
- To continue a label from one command line to the next, specify a plus (+) sign before the continuation of the label. Each string segment of the label must be enclosed in quotation marks. To insert a blank between the strings, the blank must be included in the label specification.
- To control line-wrapping of labels in pivot tables and charts, insert \n as part of the label wherever you want a line break. The \n is not displayed in output; it is interpreted as a line-break character.

  Note: Labels always wrap wherever \n occurs in the defined label even if there is enough space to display the label without wrapping.

```
/VALUELABELS VARIABLES=vehicletype VALUES= 0 'economy' 1 'compact' 2 'midsize' 3 'standard' 4 'fullsize'
```

PLAN Subcommand

The PLAN subcommand specifies the name of a simulation plan file to be written or displayed by SIMPLAN. The file contains all of the specifications required to run a simulation. The PLAN subcommand is required.

FILE = 'filespec'. *Specifies the simulation plan file.* File specifications should be enclosed in quotation marks and fully qualified paths are recommended to avoid ambiguity.

Note: The optional combination of an asterisk (*) and a backslash (\) preceding the plan file name specifies that the file is a temporary file--for example, `FILE='*\SimulationPlan.splan'`. You would then use this file specification on the PLAN subcommand of a subsequent SIMRUN command.

DISPLAY. *Specifies whether the details of the plan file are displayed in the output.* The default is YES.

SOURCE Keyword

The SOURCE keyword specifies whether associations between inputs with categorical distributions (specified by a contingency table) and parameters for inputs with empirical distributions are automatically calculated from the active dataset or whether they can be taken from a simulation plan file, which is given by `'filespec'`. The default is SOURCE=AUTOFIT, which specifies that empirical distributions and contingency tables are calculated from the active dataset.

Specifying TYPE=MANUAL, without the DISTRIBUTION keyword, for an input on the SIMINPUT subcommand indicates that the specifications for the input are to be taken from the simulation plan file that is designated by SOURCE=`'filespec'`. This specification is only valid for the following types of inputs: inputs that are described by an empirical distribution in the plan file; inputs that are described by a categorical distribution and included in a contingency table in the plan file.

The following rules apply to inputs with a categorical distribution and that are included in a contingency table in the plan file:

- When TYPE=MANUAL is specified without the DISTRIBUTION keyword, MULTIWAY=YES must be specified on the CONTINGENCY subcommand.
- You cannot specify TYPE=MANUAL (without the DISTRIBUTION keyword) for a subset of inputs that are included in the contingency table. You must either specify TYPE=MANUAL for all such inputs or for none of them.

Note: The contingency table in the plan file determines both the associations between the included inputs and the categories and category probabilities for those inputs.

SIMPREP BEGIN-SIMPREP END

SIMPREP BEGIN and SIMPREP END are deprecated for release 23 and higher. The command names are changed to BEGIN EXPR and END EXPR.

Related information

“BEGIN EXPR-END EXPR” on page 237

OUTFILE subcommand

The OUTFILE subcommand of SIMPREP BEGIN saves an XML-format file that specifies the custom model. This file can be used as the model file for creating a simulation plan with the SIMPLAN command.

PREPXML = 'filespec'. *Specifies the XML-format file.* File specifications should be enclosed in quotation marks and file names must be specified in full. SIMPREP BEGIN does not supply file extensions. If the file specification refers to an existing file, then the file is overwritten.

Note: The optional combination of an asterisk (*) and a backslash (\) preceding the XML file name specifies that the file is a temporary file--for example, PREPXML='*\SIMPREP_transformation.xml'. You would then use this file specification on the MODEL subcommand of the subsequent SIMPLAN command.

SIMRUN

SIMRUN is available in the Statistics Base Edition.

```
SIMRUN
/PLAN FILE=file

[/CRITERIA
    [REPRESULTS = {TRUE**}]
                  {FALSE }
    [SEED = {629111597**}]
            {number     }

[/DISTRIBUTION
   [DISPLAY = [CDF] [PDF**] [NONE] ]

   [CDFORDER = [ASCENDING**] [DESCENDING] ]

   [SCALE= [MEAN] [MEDIAN] [PCT[({5** 95**     })]] [SIGMAS(integer)]
                                 {value [value]}

           [CUSTOM(value [value [...] ] )] [PDFVIEW([CURVE**] [HISTOGRAM])]

           [OVERLAYTARGETS({NO**})]
                          {YES }

           [REFLINES([LOWER={DEFAULT**}] [UPPER={DEFAULT**}])]
                           {value    }          {value    }
                           {LO       }          {HI       }

   [CATEGORICAL= [PREDVAL**] [PREDPROB] [GROUP([CATS**] [ITERS])] ]


[/SCATTERPLOT
   [DISPLAY = {YES }]
              {NO**}
   [ROWSIZE={DEFAULT**}]
            {integer  }

[/BOXPLOT
   [DISPLAY = {YES }]
              {NO**}
   [OVERLAYTARGETS= {NO**}
                    {YES }[(GROUP=[TARGETS**] [ITERS])]

[/TORNADO
   [DISPLAY = {YES**}]
              {NO   }

   [TYPE= [CORR**] [DELTASTDDEV(number)] [VARIANCE] ]

[/PRINT
   [DESCRIPTIVES = {YES**}[(CILEVEL={95** })] ]
                                    {value}
                  {NO   }

   [PERCENTILES = {YES }([NTILES = integer] [PTILES = valueList]) ]
                  {NO**}

   [ASSOCIATIONS={YES }]
                 {NO**}

[/OPTIONS
   [VIZSTYLESHEET="style sheet name"[LOCATION={LOCAL**               }] ]
                                             {"repository folderspec"}

[/OUTFILE
   FILE={'filespec' | 'dataset'}
```

** Default if the subcommand or keyword is omitted.

This command does not read the active dataset or execute pending transformations.

Release History

Release 21.0

- Command introduced.

Release 22.0

- Added support for saving the simulated data to the active dataset by specifying an asterisk (*) on the FILE keyword of the OUTFILE subcommand.
- REFLINES keyword added to DISTRIBUTION subcommand.
- ASSOCIATIONS keyword added to PRINT subcommand.
- OPTIONS subcommand added.

Example

```
SIMRUN
/PLAN FILE='/simulations/myplan.splan'.
```

Overview

The SIMRUN command runs a simulation based on a simulation plan created from the SIMPLAN command. All settings required to run the simulation are specified in the simulation plan file. The SIMRUN command only provides options for customizing the output.

Simulation in IBM SPSS Statistics refers to simulating input data to predictive models using the Monte Carlo method and evaluating the model based on the simulated data. The distribution of predicted target values can then be used to evaluate the likelihood of various outcomes.

Uncertain inputs are modeled with probability distributions and simulated values for those inputs are generated by drawing from those distributions. Inputs whose values are known are held fixed at the known values. The predictive model is evaluated using a simulated value for each uncertain input and fixed values for the known inputs to calculate the target (or targets) of the model. The process is repeated many times (typically tens of thousands or hundreds of thousands of times), resulting in a distribution of target values. Each repetition of the process generates a separate data record that consists of the set of simulated values for the uncertain inputs, the values of the fixed inputs, and the predicted target (or targets) of the model.

You can also simulate data in the absence of a predictive model by specifying probability distributions for variables that are to be simulated. Each generated data record consists of the set of simulated values for the specified variables.

Options

Distributions of target values. For continuous targets you can display the probability density function and the cumulative distribution function of the predicted values. For categorical targets, you can display the percentage of simulated cases that fall in each category of the target. For categorical targets of PMML models, you can also display histograms of the probability distribution (over the simulated cases) for each of the categories of the target. For Two-Step cluster models and K-Means cluster models, you can display a bar chart of cluster membership.

Scatterplots and boxplots. You can display scatterplots of each target with each of its simulated inputs, and you can display boxplots of the distribution of predicted values for each continuous target.

Tornado charts. You can create tornado charts (horizontal bar charts) that display relationships between targets and simulated inputs using a variety of metrics.

Associations. You can display a table of correlation coefficients between simulated inputs. When simulated inputs with categorical distributions are generated from a contingency table, you can display the contingency table of the simulated data for those inputs.

Customizing results from sensitivity analysis. When the simulation plan contains specifications for sensitivity analysis, independent sets of simulated data are generated, allowing you to investigate the effect of systematic changes in a fixed input or in a distribution parameter for a simulated input. The SIMRUN command provides options for displaying the results from the different sets of simulated data.

Basic Specification

The basic specification is the SIMRUN command with a PLAN subcommand that specifies the simulation plan file.

Syntax Rules

- The PLAN subcommand is required. All other subcommands are optional.
- Subcommands can be specified in any order.
- An error occurs if a keyword is specified more than once within a subcommand.

- Parentheses, equals signs, and slashes shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not allowed.

Example

```
SIMRUN
 /PLAN FILE='/simulations/myplan.splan'
 /CRITERIA REPRESULTS=TRUE SEED=498654860
 /DISTRIBUTION DISPLAY=PDF CDF
 /BOXPLOT DISPLAY=YES
 /SCATTERPLOT DISPLAY=YES
 /TORNADO DISPLAY=YES TYPE=CORR VARIANCE
 /PRINT DESCRIPTIVES=YES PERCENTILES=YES(NTILES=4).
```

- The CRITERIA subcommand specifies an explicit seed value so that results of this simulation can be reproduced by using the same seed.
- The DISTRIBUTION subcommand specifies that both probability density functions and cumulative distribution functions of the predicted target values will be displayed.
- The BOXPLOT subcommand specifies that a boxplot of the distribution of predicted target values will be displayed.
- The SCATTERPLOT subcommand specifies that scatterplots of each target with each of its simulated inputs will be displayed.
- The TORNADO subcommand specifies that two tornado charts will be displayed. One chart will display the correlation coefficients between a target and its simulated inputs, and the other will display the contribution to the variance of the target from each of its simulated inputs.
- The PRINT subcommand specifies that tables of descriptive statistics of targets and inputs will be generated as well as a table of the quartiles for continuous targets.

PLAN Subcommand

The PLAN subcommand identifies the plan file whose specifications are to be used for running the simulation. It is a required subcommand.

FILE = 'filespec'. *Specifies the simulation plan file.* File specifications should be enclosed in quotation marks and fully qualified paths are recommended to avoid ambiguity. Simulation plan files have the file extension *.splan*.

Note: The optional combination of an asterisk (*) and a backslash (\) preceding the plan file name specifies that the file is a temporary file. This only applies in the case that SIMRUN follows the SIMPLAN command that creates the simulation plan. See the topic “PLAN Subcommand ” on page 1830 for more information.

CRITERIA Subcommand

REPRESULTS. *Specifies whether the results for this simulation are reproducible.* The default is TRUE. When TRUE is specified, use the SEED keyword to specify the seed value for generating random numbers, or simply accept the default value. When FALSE is specified, a seed value will be randomly generated and the results of the simulation will not be reproducible.

SEED. *Specifies the value of the random seed.* Specify an integer. The default is 629111597. The SEED keyword is ignored if REPRESULTS=FALSE.

Note: For a particular random seed, results are replicated unless the number of threads is changed. On a particular computer, the number of threads is constant unless you change it by running SET THREADS command syntax. The number of threads might change if you run the simulation on a different computer because an internal algorithm is used to determine the number of threads for each computer.

DISTRIBUTION Subcommand

The DISTRIBUTION subcommand provides settings for displaying the distributions of predicted target values. For continuous targets, you can display cumulative distribution functions (CDF) and probability density functions (PDF). For categorical targets (targets with a measurement level of nominal or ordinal), a bar chart is generated that displays the percentage of cases that fall in each category of the target. Additional options for categorical targets of PMML models are available with the CATEGORICAL keyword. For Two-Step cluster models and K-Means cluster models, a bar chart of cluster membership is produced.

DISPLAY. Specifying DISPLAY=PDF displays the probability density function for continuous targets, the bar chart of the predicted values for categorical targets, and the bar chart of cluster membership for cluster models. You can display the cumulative distribution function for continuous targets by specifying CDF. Specifying DISPLAY=NONE suppresses all charts generated from the DISTRIBUTION subcommand.

CDFORDER. *Specifies the default view for cumulative distribution functions.* By default, cumulative distribution functions are displayed as ascending functions. Use CDFORDER=DESCENDING to display them as descending functions. When displayed as a descending function, the value of the function at a given point on the horizontal axis is the probability that the target lies to the right of that point. You can change the view in the Output Viewer by activating the chart. Specifying both ASCENDING and DESCENDING will generate two charts, one in ascending order and the other in descending order.

SCALE Keyword

The SCALE keyword specifies settings for continuous targets. With the exception of the PDFVIEW keyword, all keywords below apply to charts of both cumulative distribution functions and probability density functions.

MEAN. Displays a vertical reference line at the mean of the target.

MEDIAN. Displays a vertical reference line at the median of the target.

PCT. Displays fixed vertical reference lines at one or two specified percentiles of the target distribution. You can specify one or two values for PCT, as in PCT(10) or PCT(25 75). If PCT is specified without values, reference lines will be displayed at the 5-th and 95-th percentiles.

SIGMAS. Displays vertical reference lines at the specified number of standard deviations above and below the mean of the target distribution. Specify a numeric value between 1 and 10.

CUSTOM. Displays vertical reference lines at the specified positions along the horizontal axis. Use spaces to separate values, for example: CUSTOM(50000 100000).

PDFVIEW. By default, probability density functions are displayed as a continuous curve. Use PDFVIEW=HISTOGRAM to display them as histograms. You can change the view in the Output Viewer by activating the chart. Specifying both HISTOGRAM and CURVE will generate two charts, one for each of the two views.

OVERLAYTARGETS. In the case of multiple continuous targets, this specifies whether distribution functions for all continuous targets are displayed on a single chart (separate charts for cumulative distribution functions and probability density functions). The default is NO and results in a separate chart for each target.

- If the plan file contains specifications for sensitivity analysis, OVERLAYTARGETS=YES will overlay all targets for a given iteration of the analysis on a single chart, with a separate chart for each iteration; and OVERLAYTARGETS=NO (the default) will overlay all iterations for a given target on a single chart, with a separate chart for each target.
- OVERLAYTARGETS is ignored for probability density functions when PDFVIEW=HISTOGRAM.

REFLINES. Controls the initial positions of the moveable reference lines on PDF and CDF charts. Values specified for the lower and upper lines refer to positions along the horizontal axis, not percentiles. You can remove the lower line by specifying LOWER=LO, effectively setting the position to negative infinity; and you can remove the upper line by specifying UPPER=HI, effectively setting the position to infinity. You cannot specify both LOWER=LO and UPPER=HI. By default, the lines are positioned at the 5-th and 95-th

percentiles. When multiple distributions are displayed on a single chart, the default refers to the distribution for the first iteration or first target.

Note: If the plan file contains specifications for sensitivity analysis and there is only one continuous target, then results for all iterations of the analysis will be displayed on the same chart. This applies to cumulative distribution functions and to probability density functions when PDFVIEW=CURVE. In addition, when multiple distributions are displayed on a single chart, vertical reference lines will only be applied to the distribution for the first iteration or first target. You can add reference lines to the other distributions from the Chart Options dialog, accessed from the PDF or CDF chart.

CATEGORICAL Keyword

The CATEGORICAL keyword specifies settings for categorical targets (targets with a measurement level of nominal or ordinal) and cluster models (Two-Step and K-Means models), and is in effect when PDF is specified on the DISPLAY keyword, or the DISPLAY keyword is omitted. It is ignored otherwise.

PREDVAL. For categorical targets, this option generates a bar chart that displays the percentage of simulated cases that fall in each category of the target. If the plan file contains specifications for sensitivity analysis, a clustered bar chart is displayed. By default, results are clustered by each iteration of the sensitivity analysis, with results for all categories (for a given iteration) grouped together (GROUP=CATS). You can cluster by category by specifying GROUP=ITERS. You can change the grouping in the Output Viewer by activating the chart. Specifying GROUP=CATS ITERS will generate two charts, one for each of the two groupings.

For Two-Step cluster models and K-Means cluster models, a bar chart of cluster membership is produced. When the plan file contains specifications for sensitivity analysis, the result is a clustered bar chart. The same choices for groupings, as for categorical targets, applies except categories are now replaced by the clusters from the predictive model.

PREDPROB. For categorical targets of PMML models, this option displays histograms of the probability distribution (over the simulated cases) for each of the categories of the target (one histogram for each category). If the plan file contains specifications for sensitivity analysis, the histograms for each iteration of the sensitivity analysis are grouped together by default (GROUP=CATS). You can group all histograms (one for each iteration) for each category together by specifying GROUP=ITERS. Specifying GROUP=CATS ITERS will generate the charts for each of the two groupings.

Note: The DISTRIBUTION subcommand is ignored when data are generated in the absence of a predictive model.

SCATTERPLOT Subcommand

The SCATTERPLOT subcommand generates scatterplots of each target with each of its simulated inputs.

DISPLAY. Controls whether scatterplots are displayed. The default is NO, which specifies that scatterplots are not displayed.

ROWSIZE. Scatterplots for a given target are grouped together. The ROWSIZE keyword specifies the number of scatterplots to display in each row.

Note: The SCATTERPLOT subcommand is ignored when data are generated in the absence of a predictive model.

BOXPLOT Subcommand

The BOXPLOT subcommand generates boxplots of the distribution of each continuous target. If the plan file contains specifications for sensitivity analysis and there is only one continuous target, then results for all iterations of the sensitivity analysis (one boxplot for each iteration) will be displayed on the same chart. Boxplots are not available for categorical targets.

DISPLAY. Controls whether boxplots are displayed. The default is NO, which specifies that boxplots are not displayed.

OVERLAYTARGETS. Specifies the behavior when there are multiple continuous targets. By default, results for each target are displayed on a separate chart. To overlay results for all targets on a single chart, specify OVERLAYTARGETS=YES.

- If the plan file contains specifications for sensitivity analysis and there are multiple continuous targets, then OVERLAYTARGETS=YES will produce a clustered boxplot. In that case, the GROUP keyword controls the clustering. By default, the chart is clustered by sensitivity analysis iteration with results for a given iteration (the boxplots for the targets) grouped together (GROUP=TARGETS). You can cluster by targets by specifying GROUP=ITERS. You can change the grouping in the Output Viewer by activating the chart. Specifying GROUP=TARGETS ITERS will generate two charts, one for each of the two groupings.

Note: The BOXPLOT subcommand is ignored when data are generated in the absence of a predictive model.

TORNADO Subcommand

The TORNADO subcommand generates tornado charts. Tornado charts are horizontal bar charts that display relationships between targets and simulated inputs using a variety of metrics.

DISPLAY. Controls whether tornado charts are displayed. The default is YES, which specifies that tornado charts are displayed.

TYPE Keyword

The TYPE keyword specifies the metrics for the tornado charts. Each specified metric generates a separate chart.

- **CORR.** Displays a tornado chart of the correlation coefficient between a target and its simulated inputs, and is the default tornado chart. This type of tornado chart does not support targets with a nominal or ordinal measurement level or simulated inputs with a categorical distribution.
- **DELTASTDDEV(number).** Displays a tornado chart of the effect on the target of modulating each simulated input by plus or minus a specified number of standard deviations of the distribution associated with the input. For example, DELTASTDDEV(0.5) displays the change in the target when the input changes by +/- 0.5 standard deviations. The multiplier must be a numeric value greater than 0 and less than or equal to 10. This type of tornado chart does not support targets with ordinal or nominal measurement levels, or simulated inputs with any of the following distributions: categorical, Bernoulli, binomial, Poisson, or negative binomial.
- **VARIANCE.** Displays a tornado chart of the contribution to the variance of a target from each of its simulated inputs, allowing you to assess the degree to which each input contributes to the overall uncertainty in the target. This type of tornado chart does not support targets with ordinal or nominal measurement levels, or simulated inputs with any of the following distributions: categorical, Bernoulli, binomial, Poisson, or negative binomial.

Note: The TORNADO subcommand is ignored when data are generated in the absence of a predictive model.

PRINT Subcommand

The PRINT subcommand controls tabular output.

DESCRIPTIVES. By default, descriptive statistics are displayed for continuous and categorical targets as well as for continuous inputs.

- For continuous targets the table includes the mean, standard deviation, median, minimum and maximum, confidence interval of the mean at the specified level, and the 5-th and 95-th percentiles of the target distribution. By default, the confidence interval of the mean is calculated for a confidence level of 95%. You can specify the confidence level using the CILEVEL keyword. *Note*: In the absence of a predictive model, the CILEVEL keyword is ignored.

- For categorical targets the table includes the percentage of cases that fall in each category of the target. For Two-Step cluster models and K-Means cluster models the table includes the percentage of cases that fall in each cluster of the model. For categorical targets of PMML models, the table also includes the mean probability of each category of the target.
- For continuous inputs, the table includes the mean, standard deviation, minimum and maximum.

ASSOCIATIONS. Displays a table of correlation coefficients between simulated inputs. When inputs with categorical distributions are generated from a contingency table, the contingency table of the data generated for those inputs is also displayed.

PERCENTILES Keyword

The PERCENTILES keyword displays percentiles of the distribution of each continuous target. This is in addition to the 5-th and 95-th percentiles, which are always displayed.

- **NTILES.** Calculates the percentages that divide the distribution into the specified number of categories and displays the values below which the requested percentages of cases fall. The value must be an integer between 2 and 100 inclusive.
- **PTILES.** Displays the values below which the specified percentages of cases fall. Specify one or more values separated by spaces.

Note: In the absence of a predictive model, the PERCENTILES and CILEVEL keywords are ignored.

VIZSTYLESHEET Keyword

The VIZSTYLESHEET keyword specifies an existing visualization style sheet and applies it to all charts that are generated from the current SIMRUN command.

The keyword is followed by an equals sign (=) and the name of the style sheet in quotation marks. By default, SIMRUN looks for a built-in or imported template that is stored on the local workstation. This default behavior is equivalent to appending the qualifier [LOCATION=LOCAL] to the quoted style sheet name. To use a style sheet in a IBM SPSS Collaboration and Deployment Services Repository, append the qualifier [LOCATION="repository folderspec"] to the quoted style sheet name. For more information about IBM SPSS Collaboration and Deployment Services Repository file specifications, see "File Specifications for IBM SPSS Collaboration and Deployment Services Repository Objects" on page 2217. The IBM SPSS Collaboration and Deployment Services Repository file specification is enclosed in quotation marks.

Note: The square brackets that are shown in the specification for the LOCATION keyword are required and are not used to indicate an optional element.

New Style sheets can be created with the separate IBM SPSS Visualization Designer product. The SIMRUN command can reference these style sheets if they are stored in the IBM SPSS Collaboration and Deployment Services Repository or if they are imported by using the Graphboard Template Chooser. You can access the Template Chooser by selecting **Graphboard Template Chooser** from the **Graphs** menu.

"style sheet name"
Apply the specified style sheet.

Note: The VIZSTYLESHEET keyword is ignored when data are generated in the absence of a predictive model.

OUTFILE Subcommand

The OUTFILE subcommand specifies the name of a file or dataset where the simulated data and predicted target values will be written. The results include all targets and all inputs (both simulated and fixed) regardless of whether they were excluded from chart and table output. Each case of the data file consists of the predicted values of the targets along with the simulated inputs and fixed inputs that generate the target values. When sensitivity analysis is specified, each iteration of the analysis gives rise to a contiguous set of cases that are labeled with the iteration number.

FILE = 'filespec' | dataset. *Specifies a file or the name of a previously declared dataset.* File specifications should be enclosed in quotation marks and fully qualified paths are recommended to avoid ambiguity. You can use an asterisk (*) to specify that the simulated data will be written to the active dataset, replacing the current active dataset.

SORT CASES

```
SORT CASES [BY] varlist[({A**})] [varlist...]
                         {D  }
  [/OUTFILE='filespec' [INDEX={NO**} [SIZE={DEFAULT**}]]]
                              {YES }       {value    }

 [/PASSPROTECT PASSWORD={'password'}  [ENCRYPTEDPW={NO**}] ]
                       {NONE**    }               {YES }
```

**Default if keyword omitted

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 17.0

- Added support for SET THREADS.

Release 20.0

- Added OUTFILE subcommand.

Release 21.0

- Added INDEX keyword to OUTFILE subcommand.
- Added SIZE keyword to OUTFILE subcommand..
- PASSPROTECT subcommand added.

Example

```
SORT CASES BY DIVISION (A) STORE (D).
```

Overview

SORT CASES reorders the sequence of cases in the active dataset based on the values of one or more variables. You can optionally sort cases in ascending or descending order, or you can use combinations of ascending and descending order for different variables.

Basic Specification

The basic specification is a variable or list of variables that are used as sort keys. By default, cases are sorted in ascending order of each variable, starting with the first variable that is named. For each subsequent variable, cases are sorted in ascending order within categories of the previously named variables.

Syntax Rules

- Keyword BY is optional.
- BY variables can be numeric or string but not scratch, system, or temporary variables.
- You can explicitly request the default sort order (ascending) by specifying A or UP in parentheses after the variable name. To sort cases in descending order, specify D or DOWN.
- An order specification (A or D) applies to all variables in the list, up to the previous order specification. If you combine ascending and descending order on the same SORT CASES command, you may need to specify the default A explicitly.

Operations

- SORT CASES first sorts the file according to the first variable that is named. For subsequent variables, cases are sorted within categories of the previously named variables.

- The sort sequence is based on the locale-defined order (and is not necessarily the same as the numerical order of the character codes). The default locale is the operating system locale. You can change the locale with SET LOCALE. Use SHOW LOCALE to display the current locale.
- This procedure uses the multithreaded options specified by SET THREADS.

Limitations

- You cannot specify more than 64 sort variables.

OUTFILE Subcommand

OUTFILE specifies the data file to be saved when the active dataset is sorted. This subcommand is optional and is provided as a convenience. It will avoid the extra data pass that occurs if you follow SORT CASES with the SAVE command to save the sorted data. Reducing the number of data passes can improve performance.

- Filenames should be enclosed in quotes and are stored in the working directory unless a path is included as part of the file specification.
- If OUTFILE is specified more than once, only the last OUTFILE is in effect.

INDEX=NO|YES. The optional INDEX keyword indicates if the file should be indexed. Indexing table lookup files can improve performance when merging data files with STAR JOIN. The default is NO.

Note: Sorting and saving with an index may take longer and result in a different sort order than sorting and saving without an index. The resulting file will usually take more disk space.

SIZE=DEFAULT|value. For indexed files, the optional SIZE keyword specifies the memory (in megabytes) to allocate for the index. The default is DEFAULT, which is SQRT(N)*4, where *N* is the number of cases in the file. For example, if there are 100 cases, the memory allocation is 40 MB.

PASSPROTECT Subcommand

The PASSPROTECT subcommand specifies the settings for saving a data file as an encrypted file. Once encrypted, the file can only be opened by providing the correct password.

PASSWORD. Specifies the password required to open this file. Passwords are limited to 10 characters and are case-sensitive. The specified value must be enclosed in quotation marks. The keyword NONE specifies that the file will not be encrypted and is the default.

Warning: Passwords cannot be recovered if they are lost. If the password is lost the file cannot be opened.

ENCRYPTEDPW Keyword

The ENCRYPTEDPW keyword specifies whether the password is encrypted.

NO. The password is not encrypted. It is treated as plain text. This is the default.

YES. The password is encrypted. Passwords are always encrypted when pasting syntax from the Sort Cases dialog. *Note*: ENCRYPTEDPW=YES should only be used when the password is known to be encrypted.

Creating strong passwords

- Use eight or more characters.
- Include numbers, symbols and even punctuation in your password.
- Avoid sequences of numbers or characters, such as "123" and "abc", and avoid repetition, such as "111aaa".
- Do not create passwords that use personal information such as birthdays or nicknames.
- Periodically change the password.

Note: Encrypted data files and output documents cannot be opened in versions of IBM SPSS Statistics prior to version 21. Encrypted syntax files cannot be opened in versions prior to version 22.

Examples

```
SORT CASES BY DIVISION (A) STORE (D).
```

- Cases are sorted in ascending order of variable *DIVISION*. Cases are further sorted in descending order of *STORE* within categories of *DIVISION*. A must be specified so that D applies to *STORE* only.

  ```
  SORT CASES DIVISION STORE (A) AGE (D).
  ```

- Cases are sorted in ascending order of *DIVISION*. Keyword BY is not used in this example.
- Cases are further sorted in ascending order of *STORE* within values of *DIVISION*. Specification A applies to both *DIVISION* and *STORE*.
- Cases are further sorted in descending order of *AGE* within values of *STORE* and *DIVISION*.

SORT CASES with Other Procedures

- In AGGREGATE, cases are sorted in order of the break variable or variables. You do not have to use SORT CASES prior to running AGGREGATE, because the procedure does its own sorting.
- You can use SORT CASES in conjunction with the BY keyword in ADD FILES to interleave cases with the same variables but from different files.
- With MATCH FILES, cases must be sorted in the same order for all files that you combine.
- With UPDATE, cases must be sorted in ascending order of the key variable or variables in both the primary file and all transaction files.
- You can use the PRINT command to check the results of a SORT CASES command. To be executed, PRINT must be followed by a procedure or EXECUTE.

SORT VARIABLES

```
SORT VARIABLES [BY] {NAME          } [({A})]
                    {TYPE          }   {D}
                    {FORMAT        }
                    {LABEL         }
                    {VALUES        }
                    {MISSING       }
                    {MEASURE       }
                    {ROLE          }
                    {COLUMNS       }
                    {ALIGNMENT     }
                    {ATTRIBUTE name}
```

If there are any pending transformations, this command reads the active dataset and causes execution of any pending commands. Otherwise, it takes effect immediately without reading the active dataset.

Release History

Release 16.0.

- Command introduced.

Release 18.0.

- ROLE keyword introduced.

Example

```
SORT VARIABLES BY NAME (A).
```

Overview

SORT VARIABLES sorts the variables in the active dataset based on the values of the selected dictionary attribute.

Basic Specification

The basic specification is the command name SORT VARIABLES followed by the name of a single dictionary attribute.

Syntax Rules

- Only one dictionary attribute can be specified.
- The BY keyword is optional.
- You can explicitly request the default sort order (ascending) by specifying A or UP in parentheses after the attribute name. To sort variables in descending order, specify D or DOWN.

Sorting Options

Variables can be sorted based on the values of any one of the following dictionary attributes:

NAME. *Sort variables by variable names.* The primary sort is alphabetical, but trailing digits are sorted numerically within ties on the preceding part. For example, *V2* comes before *V10* in ascending order.

TYPE. *Group variables by type (numeric or string) and sort string variables by defined width.* In default ascending order, numeric variables come before string variables, and shorter string variables come before longer string variables.

FORMAT. *Group variables by format (for example, Date, Dollar, String).*

LABEL. *Sort variables in alphabetical order by variable labels.* In default ascending order, all variables without defined variable labels come before variables with defined variable labels. See the topic “VARIABLE LABELS” on page 2125 for more information.

VALUES. *Sort variables by value labels.* In default ascending order, all variables without defined value labels come before variables with defined value labels. See the topic “VALUE LABELS ” on page 2109 for more information.

MISSING. *Group variables by defined missing values.* In default ascending order, all variables without defined missing values come before variables with defined missing values. See the topic “MISSING VALUES ” on page 1129 for more information.

MEASURE. *Sort variables by measurement level.* The default ascending order is: nominal, ordinal, scale. See the topic “VARIABLE LEVEL ” on page 2127 for more information.

ROLE. *Sort variables by role.* Some dialogs support predefined roles that can be used to pre-select variables for analysis. See the topic “Overview ” on page 2129 for more information.

COLUMNS. *Sort variables by column width.* Column width is the width of the column that displays the variable in Data View of the Data Editor. See the topic “VARIABLE WIDTH ” on page 2131 for more information.

ALIGNMENT. *Group variables by alignment.* The default ascending order is: left, right, center. See the topic “VARIABLE ALIGNMENT ” on page 2119 for more information.

ATTRIBUTE name. *Sort variables by values of the specified custom variable attribute name.* In default ascending order, all variables without values for the named attribute come before variables with values for the named attribute. See the topic “VARIABLE ATTRIBUTE” on page 2121 for more information.

SPATIAL ASSOCIATION RULES

```
SPATIAL ASSOCIATION RULES
  /MAPSPEC FILE='filespec'

  [/AUTOBINNING BINS={5**    }]
                     {integer}

  [/AGGREGATION AGGMAP={YES**}
                       {NO   }
  [CONTINUOUS={MEAN**}]
              {MEDIAN}
              {SUM   }
  [ORDINAL={MODE**  }
           {MEDIAN  }
           {HIGHEST }
           {LOWEST  }

  /DATASET
   DATAID=id [id]...
   [KEEP=field (id id [id]...[{NUMERIC}] [{NOMINAL}]) [field (id id...)]]
                              {STRING }   {ORDINAL}
                                          {SCALE  }
   [PREDICTIONS=fieldlist]
   [CONDITIONS=fieldlist]
   [BINEXCEPTIONS=field(value list);
                  [field(value list);]…
   [AGGEXCEPTIONS=fieldname({MEAN  }); fieldname…]
                            {MEDIAN}
                            {MODE  }
                            {SUM   }
                            {HIGH  }
                            {LOW   }

  [/RULEGENERATION]
    [MAXCONDITION={5**    }]
                  {integer}
    [MAXPREDICTION={1**    }]
                   {integer}
    [MINSUPPORT={.05**}
                {value}
    [MINCONDITIONSUPPORT={.05** }]
                         {value }
    [MINCONFIDENCE={.1** }]
                   {value}
    [MINLIFT={1.2**  }]
             {value  }
    [EXCLUDE=fieldname(id) fieldname (id);]
            [fieldname(id) fieldname (id);]…
    [ENCODEASSAME=fieldname(id) fieldname(id);]
                 [fieldname(id) fieldname(id);]…

  [/MODELTABLES]
    [RECORDSUMMARY={YES**}]
                   {NO   }
    [EVALUATION={YES**}]
                {NO   }
    [FIELDFREQ={YES**}]
               {NO   }
    [ITEMFREQ={NO**}]
              {YES }
    [FIELDTRANSFORMATION={NO**}]
                         {YES }
    [EXCLUDEDINPUTS={NO**}
                    {YES }

  [/MAPOUTPUT]
    [DISPLAY={YES**}]
             {NO   }
    [CRITERION={CONFIDENCE** }]
               {RULESUPPORT  }
               {LIFT         }
               {CONDSUPPORT  }
               {DEPLOYABILITY}
    [NUMRULES={5**    }]
              {integer}

  [/WORDCLOUD]
    [DISPLAY={NO**}]
             {YES  }
    [CRITERION={CONFIDENCE** }]
               {RULESUPORT   }
               {LIFT         }
               {CONDSUPPORT  }
               {DEPLOYABILITY}
    [NUMRULES={10**   }]
              {integer}

  [/RULESTABLE]
    [DISPLAY={YES**}]
             {NO   }
    [CRITERION={CONFIDENCE** }]
               {RULESUPORT   }
               {LIFT         }
               {CONDSUPPORT  }
               {DEPLOYABILITY}
    [NUMRULES={30**   }]
              {integer}

  [/SAVE]
    SOURCESTOSCORE=id ('filespec'|dataset)
```

```
                [id ('filespec'|dataset)...]
    [BESTRULE={3**    }]
              {integer}
    [BESTNAME='name']
    [TARGETS=fieldname(id) [fieldname(id)]…]
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 23.0

- Command introduced.

Example

```
SPATIAL ASSOCIATION RULES

  /MAPSPEC FILE="/MyFolder/MyFile.xml"
  /DATASET DATAID=EventData
    EVENTDATA=YES
    PREDICTIONS=EVar1 EVar2 EVar3
    CONDITIONS=EVar2 EVar3 EVar4 EVar5
  /DATASET DATAID=ContextDataA
    EVENTDATA=NO
    PREDICTIONS=AVar1 AVar2
    CONDITIONS=AVar2 AVar3
  /DATASET DATAID=ContextDataB
    EVENTDATA=NO
    PREDICTIONS=BVar1 BVar2
    CONDITIONS=BVar2 BVar3
  /SAVE SOURCESTOSCORE=EventData(EventData)
    BESTNAME="BestRules".
```

Overview

The SPATIAL ASSOCIATION RULES command finds associations based on event and geospatial data (geospatial association rules).

Basic specifications

- MAPSPEC subcommand that specifies the map specification file.
- DATASET subcommand that specifies the data source that contains event data.
- One or more DATASET subcommands that specify the data sources that contain map information.
- All other subcommands are optional.

Operations

Unlike most statistical procedure commands, this command does not require an active dataset that is open in the current session. All data sources used by the procedure must be specified in a map specification file defined with the SPATIAL MAPSPEC command.

Syntax rules

- Subcommands can be specified in any order.
- Parentheses, equals signs, and slashes shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not allowed.

Limitations

SPLIT FILE is ignored by SPATIAL ASSOCIATION RULES.

MAPSPEC subcommand

The MAPSPEC subcommand specifies the map specification file that identifies the required maps and data.

- A map specification file is a .mplan file created with the SPATIAL MAPSPEC command.
- The file specification must be enclosed in quotation marks.

AUTOBINNING subcommand

The AUTOBINNING subcommand specifies the number of bins for continuous fields.

- The BINS keyword specifies the number of bins.
- The default value is BINS=5.
- The value must be an integer between 1 and 10.

AGGREGATION subcommand

The AGGREGATION subcommand specifies the aggregation methods for data and maps.

- Aggregation is necessary when there are more records in the data than there are features in the map. For example, you have data records for individual counties but you have a map of states.
- You can specify the aggregation method for continuous and ordinal fields. Nominal fields are aggregated based on the modal value.

AGGMAP=NO | YES
The AGGMAP keyword specifies if aggregation is applied to both data and maps. The default value is NO, and aggregation is applied only to data.

CONTINUOUS=MEAN | MEDIAN | SUM
The aggregation method for continuous fields. The default value is MEAN.

ORDINAL=MODE | MEDIAN | HIGHEST | LOWEST
The aggregation method for ordinal fields. The default value is MODE.

Measurement level

For data sources that are not IBM SPSS Statistics format data files, measurement level is automatically assigned based on the rules in the following table.

Table 90. Rules for determining default measurement level

Condition	Measurement Level
Format is dollar or custom-currency	Continuous
Format is date or time (excluding Month and Wkday)	Continuous
All values of a variable are missing	Nominal
Variable contains at least one non-integer value	Continuous
Variable contains at least one negative value	Continuous
Variable contains no valid values less than 10,000	Continuous
Variable has *N* or more valid, unique values*	Continuous
Variable has no valid values less than 10	Continuous
Variable has less than *N* valid, unique values*	Nominal

* *N* is the user-specified cut-off value. The default is 24.

DATASET subcommand

The DATASET subcommand identifies the data sources used in the analysis. The datasets are defined in a map specification file created by the SPATIAL MAPSPEC command.

- You can specify multiple DATASET subcommands for multiple data sources.
- Only one DATASET subcommand can specify event data.

DATAID=id
The IDENTIFIER value of the dataset, specified on a DATA subcommand of the SPATIAL MAPSPEC command.

- If EVENTDATA=YES and more than one data source provides event data, you can specify multiple ID values .

KEEP=fieldname (id id [id...] [type] [measurement level]) fieldname...
Required for multiple event data sources, KEEP creates a single data source from the different data sources. For each field, specify the datasets that contain that field.

ALL
ALL can be used in place of a list of dataset ID values. This keyword includes all fields from all datasets specified on the DATAID keyword.

STRING | NUMERIC
You can optionally specify the data type as STRING or NUMERIC. Data type for each field must be the same in all datasets. The type specification on KEEP treats all instances of the field as the same type. If a type keyword is specified, you must also specify the measurement level.

NOMINAL | ORDINAL | SCALE
You can optionally specify the measurement level as NOMINAL, ORDINAL, or SCALE. This keyword must be preceded by a type keyword. SCALE cannot be specified with STRING.

PREDICTIONS=fieldlist
Specifies the fields that are predictions in the rules.

- For multiple event data sources, all fields specified for PREDICTIONS must also be specified on KEEP.
- Fields specified as predictions can also be specified as conditions.

CONDITIONS=fieldlist
Specifies the fields that are conditions. Fields specified as conditions can also be specified as predictions.

BINEXCEPTIONS=fieldname(value list); fieldname(value list);...
For the specified continuous fields, overrides the AUTOBINNING setting and uses the specified list of cut points for binning.

- The value list is enclosed in parentheses.
- Specifications for multiple fields are separated by semi-colons.

AGGEXCEPTIONS=fieldname(MEAN|MEDIAN|MODE|SUM|HIGH|LOW); fieldname...
For the specified fields, overrides the aggregation method specified on the AGGREGATION subcommand and uses the specified method.

- The method name is enclosed in parentheses.
- Specifications for multiple fields are separated by semi-colons.

RULEGENERATION subcommand

The RULEGENERATION subcommand specifies parameters for how rules are generated. A rule consists of conditions and predictions. A rule can have multiple conditions and multiple predictions.

MAXCONDITION=integer
Maximum number of field values that can occur in the conditions for a rule. The value must be a positive integer between 1 and 9. The default value is 5. The sum of MAXCONDITION and MAXPREDICTION cannot exceed 10.

MAXPREDICTION=integer
Maximum number of field values that can occur in the predictions for a rule. The value must be a positive integer between 1 and 5. The default value is 1. The sum of MAXCONDITION and MAXPREDICTION cannot exceed 10.

MINSUPPORT=value
Minimum rule support a rule must have to be included in the output. The value represents the proportion of cases for which the rule is true in the observed data. For example, if the rule is "If x=1 and y=2, then z=3," rule support is the actual proportion of cases in the data for which x=1, y=2, and z=3. The value must be greater than 0 and less than 1. The default value is 0.05. Using the default value, a rule must be true at least 5% of the time in the observed data.

MINCONDITIONSUPPORT=value
Minimum condition support a rule must have to be included in the output. The value represents the proportion of cases for which the condition exists. For example, if the rule is "If x=1 and y=2, then z=3," condition support is the proportion of cases in the data for which x=1 and y=2. The value must be greater than 0 and less than 1. The default value is 0.05. Using the default value, a condition must occur in 5% of the cases.

MINCONFIDENCE=value
Minimum confidence a rule must have to be included in the output. Confidence is the proportion of correct predictions. The value must be greater than 0 and less than 1. The default value is 0.1. Using the default value, the prediction of the rule must be correct at least 10% of the time.

MINLIFT=value
Minimum lift a rule must have to be included in the output. Lift is a measure of how much the rule improves prediction compared to random chance. It is the ratio of correct predictions to the overall occurrence of the predicted value. The value must be greater than 1. The default value is 1.2. For example, if the predicted value occurs 20% of the time and the confidence in the prediction is 80%, then the lift value is 4. The default value of 1.2 means that prediction must be correct 20% more often than the overall occurrence of the predicted value.

EXCLUDE=fieldname(id) fieldname(id); fieldname(id) fieldname(id);...
Excludes the specified pairs of fields from being included in the same rule.

- Each field name is followed by the ID value of the data source it comes from.
- The ID value is the IDENTIFIER value that is specified on the DATA subcommand of the SPATIAL MAPSPEC command.
- If there are multiple event data sources, use the keyword EVENT for the ID value.
- Each pair of field names is separated by a semi-colon.

ENCODEASSAME=fieldname(id) fieldname(id); fieldname(id) fieldname(id);...
Identifies pairs of fields that should be treated as the same field.

- Each field name is followed by the ID value of the data source it comes from.
- The ID value is the IDENTIFIER value that is specified on the DATA subcommand of the SPATIAL MAPSPEC command.
- If there are multiple event data sources, use the keyword EVENT for the ID value.
- Each pair of field names is separated by a semi-colon.

MODELTABLES subcommand

The MODELTABLES subcommand specifies the model evaluation tables to display in the output.

RECORDSUMMARY=YES | NO
Number and percentage of included and excluded records. The default display setting is YES.

EVALUATION=YES | NO
Summary statistics for condition support, confidence, rule support, lift and deployability. The statistics include mean, minimum, maximum, and standard deviation. The default display setting is YES.

FIELDFREQ=YES | NO
Fields that occur most frequently in the rules. The default display setting is YES.

FIELDTRANSFORMATIONS=NO | YES
Describes the transformations that are applied to fields used in the analysis. The default display setting is NO.

ITEMFREQ=NO | YES
Items that occur most frequently. An item is a condition or a prediction in a rule. For example, age < 18 or gender=female. The default display setting is NO.

EXCLUDEDINPUTS=NO | YES
Fields excluded from the analysis and the reason each field was excluded. The default display setting is NO.

MAPOUTPUT subcommand

The MAPOUTPUT subcommand controls the display of interactive map output, based on the specified criterion. To display results for multiple criteria, use multiple MAPOUTPUT subcommands.

DISPLAY=YES | NO
Controls the display of the interactive map output. The default display setting is YES.

NUMRULES=integer
The maximum number of rules to include in the output.

- The value must be an integer between 2 and 10.
- The specified value is the maximum number of rules. The final number of rules included could be lower than that.
- The default value is 5.

CRITERION=value
The criterion to use for rules.

- Only one criterion can be specified on each subcommand.
- To display rules for more than one criterion, use multiple subcommands.

CONFIDENCE
The percentage of correct rule predictions. This is the default setting.

RULESUPPORT
The percentage of cases for which the rule is true. For example, if the rule is "If x=1 and y=2, then z=3," rule support is the actual percentage of cases in the data for which x=1, y=2, and z=3.

LIFT
Lift is a measure of how much the rule improves prediction compared to random chance. It is the ratio of correct predictions to the overall occurrence of the predicted value. The value must be greater than 1. For example, if the predicted value occurs 20% of the time and the confidence in the prediction is 80%, then the lift value is 4.

CONDSUPPORT
The percentage of cases for which the rule condition exists. For example, if the rule is "If x=1 and y=2, then z=3," condition support is the proportion of cases in the data for which x=1 and y=2.

DEPLOYABILITY
The percentage of incorrect predictions when the conditions are true. Deployability is equal to (1-confidence) multiplied by condition support or condition support minus rule support.

WORDCLOUD subcommand

The WORDCLOUD subcommand controls the display of a word cloud of rules for the specified criterion. The size and position of the text indicates its relative importance. To display results for multiple criteria, use multiple WORDCLOUD subcommands.

DISPLAY=NO | YES
Controls the display of the word cloud. The default display setting is NO.

NUMRULES=integer
The number of rules to include in the output. The value must be an integer between 2 and 30. The default value is 10.

CRITERION=value
The criterion to use for rules.

- Only one criterion can be specified on each subcommand.
- To display rules for more than one criterion, use multiple subcommands.

CONFIDENCE
The percentage of correct rule predictions. This is the default setting.

RULESUPPORT
The percentage of cases for which the rule is true. For example, if the rule is "If x=1 and y=2, then z=3," rule support is the actual percentage of cases in the data for which x=1, y=2, and z=3.

LIFT
Lift is a measure of how much the rule improves prediction compared to random chance. It is the ratio of correct predictions to the overall occurrence of the predicted value. The value must be greater than 1. For example, if the predicted value occurs 20% of the time and the confidence in the prediction is 80%, then the lift value is 4.

CONDSUPPORT
The percentage of cases for which the rule condition exists. For example, if the rule is "If x=1 and y=2, then z=3," condition support is the proportion of cases in the data for which x=1 and y=2.

DEPLOYABILITY
The percentage of incorrect predictions when the conditions are true. Deployability is equal to (1-confidence) multiplied by condition support or condition support minus rule support.

RULESTABLE subcommand

The RULESTABLE subcommand controls the display of a table of rules, sorted by the value of the specified criterion.

- The table displays the top *n* rules (based on NUMRULES), based on the selected criterion.
- The table displays values for all criteria.
- To display results sorted by different criteria, use multiple RULESTABLE subcommands.

DISPLAY=NO | YES
Controls the display of the rules. The default display setting is NO.

NUMRULES=integer
The maximum number of rules to include in the output.

- The value must be an integer between 2 and 50.
- The specified value is the maximum number of rules. The final number of rules included could be lower than that.
- The default value is 30.

CRITERION=value
The criterion to use for rules.

- Only one criterion can be specified on each subcommand.
- To display rules for more than one criterion, use multiple subcommands.

CONFIDENCE
The percentage of correct rule predictions. This is the default setting.

RULESUPPORT
The percentage of cases for which the rule is true. For example, if the rule is "If x=1 and y=2, then z=3," rule support is the actual percentage of cases in the data for which x=1, y=2, and z=3.

LIFT
Lift is a measure of how much the rule improves prediction compared to random chance. It is the ratio of correct predictions to the overall occurrence of the predicted value. The value must be greater than 1. For example, if the predicted value occurs 20% of the time and the confidence in the prediction is 80%, then the lift value is 4.

CONDSUPPORT
The percentage of cases for which the rule condition exists. For example, if the rule is "If x=1 and y=2, then z=3," condition support is the proportion of cases in the data for which x=1 and y=2.

DEPLOYABILITY
The percentage of incorrect predictions when the conditions are true. Deployability is equal to (1-confidence) multiplied by condition support or condition support minus rule support.

SAVE subcommand

The SAVE subcommand saves score values as new fields in a data file or new dataset.

SOURCESTOSCORE=id ('filespec'|dataset)
New fields containing score values are written to the specified data source. The ID value is the value of DATAID on the DATASET subcommand. If the data source is not open in the current session, it is opened in the current session. You must explicitly save the modified file to save the new fields.

BESTRULE=integer
Saves the best *n* rules. Three new fields are saved for each rule: rule ID, predicted value, and confidence. The default number of best rules is 3.

BESTNAME='name'
Root name for the new fields. Names must conform to variable naming rules. The default root name is "BestRule".

TARGETS=fieldname(id) fieldname(id)...
Create new fields for the selected target (prediction) fields.

- Two new fields are created for each target field: predicted value and confidence value.
- For continuous (scale) target fields, the predicted value is string that describes a value range. A value of the form "(value1, value2]" means "greater than value1 and less than or equal to value2."

SPATIAL MAPSPEC

```
SPATIAL MAPSPEC
  [/MAPPROJECTION]
   [WKID=value]
   [PROJECTION={name     }]
               {CARTESIAN}
   [DISPLAY={SIMPLE**     }
            {MERCATOR     }
            {STEREOGRAPHIC}
            {WINKELTRIPEL }

  [/MAP]
   TYPE={FILE    }
        {SERVICE }
   SOURCE={'filespec'}
          {'url'     }
   [LAYER=number]
   [LAYERNAME='name']
   IDENTIFIER=name
   [COORDSYSTEM={GEOGRAPHIC**     }]
                {WKID (value)     }
                {PROJECTION (name)}
   [SPATIALRELATION={CLOSE**   }]
                    {WITHIN    }
                    {CONTAINS  }
                    {INTERSECTS}
                    {CROSS     }
                    {OVERLAP   }
                    {TOUCH     }
                    {EAST      }
                    {WEST      }
                    {NORTH     }
                    {SOUTH     }
   [CLOSERADIUS=value]
   [CLOSEMEASURE={MILE**    }]
                 {FOOT      }
                 {KILOMETER }
                 {METER     }
                 {CENTIMETER}
   [MAXRELATIONS={5**    }]
                 {integer}

  /DATA
   TYPE={FILE}
        {MAP }
        {DATASET}
   SOURCE={'filespec'}
          {map id    }
          {dataset   }
          {*         }
   IDENTIFIER=name
   ROLE={EVENTDENSITY}
        {CONTEXT     }
        {SCORE       }
   [KEY={SHAPEID**          }]
        {DEFINED (fieldlist)}
   [MAPID=map]
   [DATAKEY=fieldlist]
   [MAPKEY=fieldlist]
   [XCOORDINATE=field]
   [YCOORDINATE=field]
   [ZCOORDINATE=field]
   [COORDSYSTEM={GEOGRAPHIC**     }]
                {CARTESIAN        }
                {WKID (value)     }
                {PROJECTION (name)}
   [DATATOMAP='data value' TO 'map value';]
             ['data value' TO 'map value';]
             ...

  /SPECFILE FILE={'*\mapspec.zip'**}
                 {'filespec'       }
   [DISPLAY={YES**}]
            {NO   }
   [SAVEDATA={NO**}]
             {YES }
```

**Default if the subcommand or keyword is omitted.

Release History

Release 23.0

- Command introduced.

Overview

The SPATIAL MAPSPEC command defines map specification files for use with SPATIAL TEMPORAL PREDICTION and SPATIAL ASSOCIATION RULES.

Syntax rules

- Subcommands can be specified in any order.
- You can specify multiple MAP and DATA subcommands. The order of MAP subcommands determines the layer order of the maps.
- Parentheses, equals signs, and slashes shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not allowed.

MAPPROJECTION subcommand

The optional MAPPROJECTION subcommand specifies the type of projection that is used to read and display maps.

- All maps specified in the SPATIAL MAPSPEC command must use the same map projection.
- If both WKID and PROJECTION are specified, the WKID value is used.

WKID=code
"Well known ID" for common projections. For more information, go to http://resources.arcgis.com

PROJECTION
The PROJECTION keyword identifies the type of projection.

name
A valid projection name. For more information, go to http://help.arcgis.com/en/webapi/javascript/gmaps/help/topics/pcs.html

CARTESIAN
The coordinate system is simple X and Y coordinates.

DISPLAY
The DISPLAY keyword sets the display projection.

SIMPLE
Use the projection from the WKID or the PROJECTION value. This setting is the default value.

MERCATOR
A cylindrical map projection that preserves the angles and the shapes of small objects. The Mercator projection distorts the size and shape of large objects, as the scale increases from the Equator to the poles, where it becomes infinite.

STEREOGRAPHIC
A projection recommended for maps close to the poles.

WINKELTRIPEL
Winkel tripel projection. This projection is a modified azimuthal map projection.

MAP subcommand

Each optional MAP subcommand represents a map and layer in the model. The z-order of the maps is determined by the order of the MAP subcommands. The first MAP subcommand represents the map furthest back in map order.

- If specified, the MAP subcommand must include TYPE, SOURCE, and IDENTIFIER keywords.
- This subcommand is not required if you are using a dataset in the current session or an IBM SPSS Statistics data file that includes fields that contain X and Y coordinates.

TYPE=FILE | SERVICE
Specifies the type of map. FILE indicates that the source is a shape file (.shp). SERVICE indicates that the source is an online map service.

SOURCE='filespec' | 'url'

- If TYPE=FILE, the SOURCE value must be a shape file (.shp) with a corresponding dBase (.dbf) file in the same directory location. The name of the .dbf file must be the same as the name of the .shp file. The source file can also be a .zip archive. The name of the .shp file, .dbf file, and any other supporting files (.shx, .sbx, .sbn, .prj) must be the same as the name of the .zip archive.
- If TYPE=SERVICE, the SOURCE value must be an internet address (URL) for the service. The value must be enclosed in quotation marks.

LAYER=number
If the source is a map service, the index number of the layer. The default value is the first layer of the map. You can specify LAYER or LAYERNAME.

LAYERNAME='name'
If the source is a map service, the name of the layer. The name must be enclosed in quotation marks. The default value is the first layer of the map. You can specify LAYER or LAYERNAME.

IDENTIFIER=name
A unique name that identifies the map. The name must conform to variable name rules.

COORDSYSTEM
If there is no projection (.prj) file associated with the map, COORDSYSTEM specifies the coordinate system.

GEOGRAPHIC
The coordinate system is latitude and longitude. This option is the default setting.

WKID (value)
The coordinate system is based on the WKID value. The value is enclosed in parentheses.

PROJECTION (name)
The coordinate system is based on the named projection. The name is enclosed in parentheses. For more information, go to http://help.arcgis.com/en/webapi/javascript/gmaps/help/topics/pcs.html

SPATIALRELATION
The geospatial relationship is used by the model to determine how the events relate to the features in the map.

- This setting is used only for maps that are used in the SPATIAL ASSOCIATION RULES procedure.
- This setting only applies to data sources associated with maps where ROLE=CONTEXT is specified on the DATA subcommand.

CLOSE
The event occurs close to a specified point on the map. The CLOSERADIUS keyword determines the area that is "close".

WITHIN
The event occurs within a specified region on the map.

CONTAINS
The event area contains geographic context object.

INTERSECTS
Locations where lines or regions from the different maps intersect each other.

CROSS
Locations where lines (for roads, rivers, railroads) from different lines cross each other.

NORTH, SOUTH, EAST, WEST
The event occurs within an area north, south, east, or west of a specified point on the map. The CLOSERADIUS keyword determines the area.

CLOSERADIUS=value
The radius for what is considered "close" to the features in the map. This setting is used only when the spatial relation value is CLOSE, NORTH, SOUTH, EAST, or WEST.

CLOSEMEASURE=MILE|KILOMETER|METER|CENTIMETER
The measurement unit that is used for the CLOSERADIUS value. The default measurement unit is MILE.

MAXRELATIONS=integer
The maximum number of relationships between the features on the map and the events. The default value is 5.

DATA subcommand

The DATA subcommand specifies data sources to include.

- You can specify multiple DATA subcommands.

TYPE=FILE|MAP|DATASET
Specifies the type for the data source.

- FILE. An IBM SPSS Statistics format data file (.sav).
- MAP. A DBF (.dbf) file associated with a shape file (.shp) specified on the MAP subcommand.
- DATASET. A dataset that is open in the current session.

SOURCE='file spec' | map id | dataset | *
Specifies the name of the data source.

- For TYPE=FILE, the name must be a file specification enclosed in quotation marks.
- For TYPE=MAP, the name must be the value of IDENTIFIER specified on the MAP subcommand.
- For TYPE=DATASET, the name must be the name of a dataset that is open in the current session. You can use an asterisk (*) to refer to the active dataset.

IDENTIFIER=name
A unique name that identifies the dataset. The name must conform to variable name rules.

ROLE=EVENTDENSITY | CONTEXT | SCORE
Specifies the role of the data source.

EVENTDENSITY

- For SPATIAL ASSOCIATION RULES, this value indicates that the data source contains event data. Event data contain information on events that occur, such as crimes, accidents, or individual addresses.
- For SPATIAL TEMPORAL PREDICTION, this value indicates that the data source contains data for computing kernel density estimates. This data source must use coordinates, specified with the XCOORDINATE, YCOORDINATE and optional ZCOORDINATE keywords.

CONTEXT

A context data identifies features on the map. A context data can also contain fields that can be used as inputs for the model. To use a context dBase (.dbf) file that is associated with a map shape (.shp) file, the context dBase file must be in the same location as the shape file and must have the same root name. For example, if the shape file is geodata.shp, the dBase file must be named geodata.dbf.

SCORE
For SPATIAL TEMPORAL PREDICTION, fields that contain values for predicted values, variance, and upper and lower confidence bounds are written to the specified data source. You must include a SCORE specification if if SCORE=YES on the SAVE subcommand of the SPATIAL TEMPORAL PREDICTION command.

KEY
For TYPE=MAP, an optional list of hierarchical key fields. For example, a map could have a region for every county and every state. Since county names are not unique, you need to specify state and county as hierarchical keys.

SHAPEID
A unique internal ID value. This is the default.

DEFINED (field list)
A list of field names that define the hierarchical keys. The list of field names is enclosed in parentheses.

MAPID, DATAKEY, and MAPKEY
For TYPE=FILE and TYPE=DATASET, the DATA subcommand must specify either key values associated with a map or coordinate values.

MAPID=id
This setting specifies the map associated with the data. The ID value for the map is the IDENTIFIER value specified on the MAP subcommmand.

DATAKEY=fieldlist
The key fields in the data source that match the data with the map. Fields specified on DATAKEY are matched with fields specified on MAPKEY in the order specified. Both keywords must contain the same number of fields.

MAPKEY=fieldlist
The key fields in the map that match the map with the data. Fields specified on MAPKEY are matched with fields specified on DATAKEY in the order specified. Both keywords must contain the same number of fields.

XCOORDINATE, YCOORDINATE, and ZCOORDINATE
For TYPE=FILE and TYPE=DATASET, the DATA subcommand must specify either key values associated with a map or coordinate values. If the data source contains coordinate values, these settings specify the X, Y, and Z coordinates. The Z coordinate is optional and is ignored by the SPATIAL ASSOCIATION RULES command.

COORDSYSTEM
Specifies the coordinate system.

GEOGRAPHIC
The coordinate system is latitude and longitude. This option is the default setting.

CARTESIAN
The coordinate system is simple X and Y coordinates.

WKID (value)
The coordinate system is based on the WKID value. The value is enclosed in parentheses.

PROJECTION (name)
The coordinate system is based on the named projection. The name is enclosed in parentheses. For more information, go to http://help.arcgis.com/en/webapi/javascript/gmaps/help/topics/pcs.html

DATATOMAP='data value' TO 'map value'
Maps specified unmatched data key values to specified map key values.

- Multiple pairs of values are separated by semi-colons.
- Quotation marks are optional for numeric values.

SPECFILE subcommand

The SPECFILE subcommand specifies the name and location for the map specification file defined by the SPATIAL MAPSPEC command.

FILE='filespec'
Specifies the name and location of the file.

- The file specification must be enclosed in quotation marks.
- The output file is .zip archive with the default file extension .mplan.

- You can use "*\" as a directory path to specify the SPSS Statistics temporary directory. The temporary file is deleted at the end of the session.
- If you do not specify a file, a temporary file named mapspec.zip is created in the temporary directory. This file is deleted at the end of the session.

DISPLAY=YES | NO
Controls the display of an output table that contains the details of the map specification. The default setting is YES.

SAVEDATA=NO | YES
Controls whether data from the files are saved with the map specification.

- NO. The map specification contains only references to the files. This option is the default setting.
- YES. Data from map shape files, external data files, and datasets used in the map specification are saved in the .zip archive.
- Data from unsaved datasets in the map specification are always saved.
- Only references are saved for map services and databases.

SPATIAL TEMPORAL PREDICTION

```
SPATIAL TEMPORAL PREDICTION
  /MAPSPEC FILE='filespec'

 [/AGGREGATION [CONTINUOUS={MEAN**}]]
                           {MEDIAN}
                           {SUM   }
   [ORDINAL={MEDIAN**}]
            {MODE    }
            {HIGHEST }
            {LOWEST  }

  /DATASET
   DATAID=id
   [KDENAME=rootname]
   [TARGET=fieldname]
   [INPUTS=fieldlist]
   [TIMEINDEX=fieldlist]
   [AGGEXCEPTIONS=fieldname({MEAN  }); fieldname…]
                           {MEDIAN}
                           {MODE  }
                           {SUM   }
                           {HIGH  }
                           {LOW   }
  /TIMEFIELDS
   INPUTINTERVAL={YEAR    }
                 {QUARTER }
                 {MONTH   }
                 {WEEK    }
                 {DAY     }
                 {HOUR    }
                 {MINUTE  }
                 {SECOND  }
                 {PERIOD  }
                 {DURATION INTERVAL={HOUR  }}
                                    {MINUTE}
                                    {SECOND}
  additional, optional keywords YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, DURATION, PERIOD:
    [INCREMENT=integer]
  additional, optional keywords for DAY, HOUR, MINUTE, SECOND:
    [DAYSWK=7** | integer]
    [WKSTART=MON**|TUE|WED|THU|FRI|SAT|SUN]
  additional, optional keywords for HOUR, MINUTE, SECOND:
    [HRSDAY=24** | integer]
    [DAYSTART=00:00** | hh:mm]
additional, optional keywords for PERIOD:
    [START=1** | value list]
    [LENGTH=value list]

  [/MODELBUILDING]
    [SIGNIFICANCE={.05**}]
                  {value}
    [INTERCEPT={YES**}]
               {NO   }
    [COVARIANCE={VARIOGRAM(2)**}]
                {EMPIRICAL     }
    [AUTOREGRESSIONLAG={1**    }]
                       {integer}
    [MISSINGRECORDS={.05**}]
                    {value}
    [UNCERTAINTY={.05**}]
                 {value}

  [/MODELTABLES]
    [SPECIFICATIONS={YES**}
                    {NO   }
    [TEMPORALINFO={YES**}]
                  {NO   }
    [LOCATIONS={YES**}]
               {NO   }
    [QUALITY={NO**}]
             {YES }
    [EFFECTS={NO**}]
             {YES }
    [MEANSTRUCTURE={NO**}]
                   {YES }
    [COEFFICIENTS={NO**}]
                  {YES }
    [COVARIANCE={YES**}]
                {NO   }
    [PARAMETRIC={NO**}
                {YES }

  [/MAPOUTPUT]
    [TARGETVALUES={YES**}]
                  {NO   }
    [CORRELATIONS={YES**}]
                  {NO   }
    [CLUSTERS={YES**}]
              {NO   }
    [SIMILARITY={.2** }]
                {value}
    [MAXCLUSTERS={10**  }]
                 {integer}
  [/SAVE]
    SCORE={NO**}
          {YES }
    SOURCETOSCORE=id ('filespec'|dataset)
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 23.0

- Command introduced.

Overview

SPATIAL TEMPORAL PREDICTION is a time series modeling procedure that include spatial aspects in the analysis.

Syntax rules

- Parentheses, equals signs, and slashes shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.
- Empty subcommands are not allowed.
- A maximum of two DATASET subcommands are allowed.

Limitations

SPLIT FILE is ignored by SPATIAL TEMPORAL PREDICTION.

Data requirements

Spatial temporal modeling requires data values that define both time and location, including:

- One or more fields that define time intervals. These fields can be dates or times or numbers that represent cyclic time periods.
- One or more fields that define location. These fields can be key values that match key values in a map file, or coordinates such as longitude and latitude.
- For each time interval, there is an instance of each location.
  - For regular time data, it is assumed that there are instances of each location for each time interval. If some locations are missing for some time intervals, those locations are treated as missing data.
  - For point occurrence data used for calculating kernel density estimates, it is assumed that the number of locations changes over time.

MAPSPEC subcommand

The MAPSPEC subcommand specifies the map specification file that identifies the required maps and data.

FILE='filespec'
A map specification file is a .mplan file created with the SPATIAL MAPSPEC command. The file specification must be enclosed in quotation marks.

AGGREGATION subcommand

The AGGREGATION subcommand specifies the aggregation methods for any fields specified as INPUTS on the DATASET subcommand.

- Aggregation is necessary when there is more than one record in a defined time interval. For example, if the time interval is month, then multiple dates in the same month are aggregated together.
- You can specify the aggregation method for continuous and ordinal fields. Nominal fields are aggregated based on the modal value.

CONTINUOUS=MEAN | MEDIAN | SUM
The aggregation method for continuous fields. The default value is MEAN.

ORDINAL=MEDIAN | MODE | HIGHEST | LOWEST
The aggregation method for ordinal fields. The default value is MEDIAN.

Measurement level

For data sources that are not IBM SPSS Statistics format data files, measurement level is automatically assigned based on the rules in the following table.

Table 91. Rules for determining default measurement level

Condition	Measurement Level
Format is dollar or custom-currency	Continuous
Format is date or time (excluding Month and Wkday)	Continuous
All values of a variable are missing	Nominal
Variable contains at least one non-integer value	Continuous
Variable contains at least one negative value	Continuous
Variable contains no valid values less than 10,000	Continuous
Variable has *N* or more valid, unique values*	Continuous
Variable has no valid values less than 10	Continuous
Variable has less than *N* valid, unique values*	Nominal

* *N* is the user-specified cut-off value. The default is 24.

DATASET subcommand

The DATASET subcommand identifies the data sources that are used in the analysis. The datasets are defined in a map specification file that is created by the SPATIAL MAPSPEC command. You can specify a maximum of two DATASET subcommands.

The following requirements apply when there are two data sources:

- Kernel density estimates (KDE) are calculated and used as the target.
- The map specification file must include a map (SPATIAL MAPSPEC command, MAP subcommand).
- Both data sources must contain a time field.
- One of the data sources must specify ROLE=EVENTDENSITY on the DATA subcommand of the SPATIAL MAPSPEC command. This data source must contain coordinates.
- The other data source must specify ROLE=CONTEXT on the DATA subcommand of the SPATIAL MAPSPEC command. This data source must be matched to the map with key values.

DATAID=id
The IDENTIFIER value of the dataset, which is specified on a DATA subcommand of the SPATIAL MAPSPEC command.

KDENAME=rootname
Root field name for calculated kernel density estimates. If a field name with that name exists, a unique new name is generated by appending an integer value to the name.

TARGET=fieldname
The name of the target field.

- If there is only one DATASET subcommand, a single target field is required.
- If there are two DATASET subcommands, the TARGET keyword is not valid.

INPUTS=fieldlist
The inputs (predictors) in the dataset. Inputs for each dataset are specified on separate DATASET subcommands.

TIMEINDEX=fieldlist
The fields that define time periods. Multiple fields can be specified to define periodicity. Each DATASET subcommand must contain a TIMEINDEX specification, and they must define the same time interval.

AGGEXCEPTIONS=fieldname(MEAN|MEDIAN|MODE|SUM|HIGH|LOW); fieldname...
For the specified fields, overrides the aggregation method specified on the AGGREGATION subcommand and uses the specified method.

- The method name is enclosed in parentheses.
- Specifications for multiple fields are separated by semi-colons.

TIMEFIELDS subcommand

The TIMEFIELDS subcommand defines the time intervals for TIMEINDEX fields specified on the DATASET subcommand.

- The procedure assumes that all cases (records) represent equally spaced intervals.
- Based on the selected time interval, the procedure can detect missing observations or multiple observations in the same time interval that need to be aggregated together. For example, if the time interval is days and the date 2014-10-27 is followed by 2014-10-29, then there is a missing observation for 2014-10-28. If the time interval is month, then multiple dates in the same month are aggregated together.
- For some time intervals, the additional setting can define breaks in the normal equally spaced intervals. For example, if the time interval is days, but only week days are valid, you can specify that there are five days in a week, and the week begins on Monday.

INPUTINTERVAL=keyword
The basic time interval. The value can be: YEAR, QUARTER, MONTH, WEEK, DAY, HOUR, MINUTE, SECOND, DURATION, or PERIOD.

- DURATION applies to time values (hours, minutes, seconds) that represent length of time from a starting time, rather than clock time.
- Time fields must be date format or time format fields unless the input interval is PERIOD.

INTERVAL=HOUR | MINUTE | SECOND
For DURATION, this setting specifies what the duration interval represents. This setting is valid only for DURATION.

INCREMENT=integer
The increment for the input interval. For example, if the input interval is years and the increment is 2, the time interval between observations is two years. The value must be a positive integer. The default value is 1. This setting is valid for YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, DURATION, and PERIOD.

DAYSWK=integer
Number of days in a week. The value must be a positive integer. The default setting is 7. This setting is valid for DAY, HOUR, MINUTE, and SECOND.

WKSTART=MON | TUE | WED | THU | FRI | SAT | SUN
The starting day of the week value. The default value is MON. This setting is valid for DAY, HOUR, MINUTE, and SECOND.

HRSDAY=integer
Number of hours in a day. The value must be a positive integer. The default value is 24. This setting is valid for HOUR, MINUTE, and SECOND.

DAYSTART=hh:mm
Starting time for the day. The value must be a time value in format hh:mm. The default value is 00:00. This setting is valid for HOUR, MINUTE, and SECOND.

START=value list
List of starting values for cyclic periods. The value must be a positive integer. This setting is valid only with PERIOD.

LENGTH=value list
List of cycle lengths of cyclic periods. The value must be a positive integer. This setting is valid only with PERIOD.

INPUTINTERVAL=PERIOD

- If there is only one time field, START and LENGTH are not valid. For a single time field, the input interval PERIOD defines the time interval for a non-cyclic time field that is not a date format or time format field.
- If there are multiple time fields, the fields represent cyclic periods, such as years, quarters within years, and months within quarters.
- If there are multiple time fields, START and LENGTH are required. If there are n time fields, the number of START and LENGTH values must be n-1, where the values correspond to the last n-1 fields specified for TIMEINDEX.
- INCREMENT represents the input interval at the lowest cycle level, which is the last field specified for TIMEINDEX.

For example, if TIMEFIELDS=year quarter month, the INPUTINTERVAL specification could be:

```
INPUTINTERVAL=PERIOD START=1 1 LENGTH=4 3
```

- The START and LENGTH values apply to quarter and month.
- LENGTH specifies that there are four quarters in each year and 3 months in each quarter.

MODELBUILDING subcommand

The MODELBUILDING subcommand specifies parameters for building the model.

SIGNIFICANCE=value
The significance level for determining if a parametric variogram-based model is appropriate. The value must be greater than 0 and less than 1. The default value is 0.05. If the significance level is higher than the specified value, an empirical model is used, regardless of the setting on the COVARIANCE keyword. The results of one of the tests used to make this determination are displayed in the test of spatial covariance table (MODELTABLES subcommand, COVARIANCE keyword).

INTERCEPT=NO | YES
Include the intercept in the model. The default setting value is NO.

COVARIANCE=VARIOGRAM (value) | EMPIRICAL
The covariance method for the model. The default method is VARIOGRAM. The keyword VARIOGRAM is followed by a value enclosed in parentheses. The value must be between 1 and 2.

AUTOREGRESSIONLAG=value
The maximum autoregression lag. The value must be an integer between 1 and 5. The default value is 1. The number of time intervals in the data must be at least double the autoregression lag value.

MISSINGRECORDS=value
Maximum allowed proportion of records (cases) with missing values. If a record has a missing value for the target field or a predictor field, it is not included in the analysis. If the total number of records

with missing target or predictor values exceeds the specified proportion, the procedure does not run. The value must be greater than 0 and less than 1. The default value is 0.05.

UNCERTAINTY=value
The uncertainty factor is a proportion value that represents the growth in uncertainty when forecasting into the future. The upper and lower limits of forecast uncertainty increase by the specified proportion for each step into the future. The value must be greater than 0 and less than 1. The default value is 0.05.

MODELTABLES subcommand

The MODELTABLES subcommand specifies the tables to include in the output.

SPECIFICATIONS=YES | NO
Summary of specifications used to run the analysis, including target, input, and location fields. The default setting is YES.

TEMPORALINFO=YES | NO
Identifies the time field and time intervals used in the model. The default setting is YES.

LOCATIONS=YES | NO
Number of locations included and excluded. The default setting is YES.

COVARIANCE=YES | NO
For variogram-based parametric models, displays the results of the test of decay over space. If the test result is not significant, a nonparametric, empirical model is used instead of a parametric model. The default setting is YES.

QUALITY=NO | YES
Model quality statistics including mean squared error, pseudo R-squared and pseudo adjusted R-squared. The default setting is NO.

EFFECTS=NO | YES
Tests of effects in the mean structure model. The output includes test statistics value, degrees of freedom, and significance level for the corrected model and each effect. The default setting is NO.

MEANSTRUCTURE=NO | YES
Mean structure model coefficients. The output includes the coefficient value, standard error, test statistics value, significance level, and confidence intervals for each model term. The default setting is NO.

COEFFICIENTS=NO | YES
Autoregressive coefficients. The output includes the coefficient value, standard error, test statistics value, significance level, and confidence intervals for each lag. The default setting is NO.

PARAMETRIC=NO | YES

For variogram-based parametric models, displays parameter estimates for parametric spatial covariance. The default setting is NO.

MAPOUTPUT subcommand

The MAPOUTPUT subcommand controls the display of interactive map output

TARGETVALUES=YES | NO
Map of target values. The default setting is YES.

CORRELATIONS=YES | NO
Map of correlations. The default setting is YES.

CLUSTER=YES | NO
Map that highlights clusters of locations that are similar to each other. The default setting is YES. Maps of clusters are displayed only for non-parametric empirical models.

sub

SIMILARITY=value
Specifies the similarity required to create clusters. The value must be a number greater than zero and less than 1. The default value is 0.2.

MAXCLUSTERS=integer
The maximum number of clusters to include. The value must be an integer greater than 1. The default value is 10.

SAVE subcommand

The SAVE subcommand saves predicted values to an open dataset in the current session or an IBM SPSS Statistics- format data file.

- The data source cannot be a data source used in the model.
- The data source must contain all the time fields and predictors used in the model.
- The time values must be greater than the time values used in the model.
- The data source must also be specified on the SPATIAL MAPSPEC command with ROLE=SCORE.

SCORE=NO | YES
Specifies if predicted values are saved. The default setting is NO.

SOURCESTOSCORE=id ('filespec'|dataset)
New fields containing predicted values, variance, and upper and lower confidence bounds are written to the specified data source.

- The ID value is the value of IDENTIFIER on the DATA subcommand of the SPATIAL MAPSPEC command. The DATA subcommand must also include ROLE=SCORE for this data source.
- If you are writing scores to an external file (instead of an open dataset), you must use the ID value specified on the SPATIAL MAPSPEC command with a different file specification. You cannot write scores to the file specified on SPATIAL MAPSPEC.

SPCHART

SPCHART is available in the Statistics Base Edition.

```
SPCHART
 [/TEMPLATE='filename']

 [/TITLE='line 1' ['line 2']]
 [/SUBTITLE='line 1']
 [/FOOTNOTE='line 1' ['line 2']]

 {[/XR=]{var BY var                  } }
        {var var [var var...][BY var]} [(XBARONLY)]
 { /XS= {var BY var                  } }
        {var var [var var...][BY var]} [(XBARONLY)]
 { /IR= var [BY var]                   }
 { /I= var [BY var]                    }
 { /NP= {var BY var                   }}
        {COUNT(var) N({var })} [BY var]}
        {value                 }
 { /P= {var BY var                   } }
       {COUNT(var) N({var })} [BY var]}
       {value                 }
 { /C= {var BY var                   } }
       {COUNT(var) N({var })} [BY var]}
       {value                 }
 { /U= {var BY var                   }}
       {COUNT(var) N({var  }) [BY var]}
                    {value}
 { /CPCHART= {var BY var      } }
             {var var [var...]}
             {var             }

 [/STATISTICS = [CP] [CPL] [CPU] [K] [CPK] [CR] [CPM]
                [CZL] [CZU] [CZMIN] [CZMAX] [CZOUT]
                [PP] [PPL] [PPU] [PPK] [PR] [PPM]
                [PZL] [PZU] [PZMIN] [PZMAX] [PZOUT]
                [AZOUT] [N] [MEAN] [STDDEV] [CAPSIGMA]
                [LSL] [USL] [TARGET] [AZLOUT] [AZUOUT]
                [CZLOUT] [CZUOUT] [PZLOUT] [PZUOUT] ]

 [/RULES = [ALL] [UCL] [R2UPPER] [R4UPPER] [R8UPPER]
           [R8LOWER] [R4LOWER] [R2LOWER] [LCL] [TRUP] [TRDOWN] [ALTERNATING] ]

 [/ID=var]

 [/CAPSIGMA = [{RBAR  }]]
              {SBAR  }
              {MRBAR }
              {WITHIN}

 [/SPAN={2**}]
        {n  }

 [{/CONFORM  }=value]
  {/NONCONFORM}

 [/SIGMAS={3**}]
          {n  }

 [/MINSAMPLE={2**}]
             {n  }

 [/LSL=value]    [/USL=value]

 [TARGET = value]

 [/MISSING=[{NOREPORT**}] [{EXCLUDE**}] ]
           {REPORT    }   {INCLUDE  }

 [/NORMAL = {CAPABILITY**} ]
            {PERFORMANCE }
            {BOTH        }
            {NONE        }

 [/REFERENCE = {[SL**] [SIGMAS]} ]
               {NONE           }
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 15.0

- (XBARONLY) keyword introduced on XR and XS subcommands.
- RULES subcommand introduced.
- ID subcommand introduced.

Release 19.0

- CPCHART subcommand introduced.
- NORMAL subcommand introduced.
- REFERENCE subcommand introduced.
- Following keywords introduced on STATISTICS subcommand: N, MEAN, STDDEV, CAPSIGMA, LSL, USL, TARGET, AZLOUT, AZUOUT, CZLOUT, CZUOUT, PZLOUT, PZUOUT.

Example

```
SPCHART  /TEMPLATE='cntr.sgt'
  /IR=subsize.
```

Overview

SPCHART generates several types of high-resolution control charts. A control chart plots a quality characteristic that is measured or computed from a sample versus the sample number or time. This technique is a widely used process-control technique for testing the hypothesis that the process is in a state of statistical control. All control charts display four series:

- The process line representing the quality characteristic for each sample.
- The center line indicating the average value of the quality characteristic that corresponds to the in-control state.
- Two horizontal lines showing the upper control limit and lower control limit.

Control charts are used for improving productivity, preventing defects and unnecessary process adjustments, and gathering information about process capability.

SPCHART produces X-bar, R, s, individuals, and moving range charts as well as np, p, c, and u charts. It also produces a process capability chart. You may need to transform your data to conform to the required data organization that is described under each chart type subcommand.

Control charts are available only on systems where high-resolution display is available.

Options

Titles and Footnotes. You can use the TITLE, SUBTITLE, and FOOTNOTE subcommands to specify a title, subtitle, and footnote for the chart.

Chart Type. You can request a specific type of chart by using the XR, XS, IR, I, NP, P, C, U, or CPCHART subcommand.

Templates. You can specify a template, using the TEMPLATE subcommand, to override the default chart attribute settings on your system.

Control Limits. You can specify a sigma value on the SIGMAS subcommand to modify the calculated upper and lower control limits. You can also use the USL and LSL subcommands to specify fixed limits. The upper and lower limits that you specify will be displayed simultaneously with the calculated control limits. *Note*: Calculated control limits are not displayed on process capability charts.

Control Rules. You can specify control rules that help you quickly identify out-of-control points.

Basic Specification

The basic specification is a chart type subcommand with a variable specification that describes the process data.

Subcommand Order

Subcommands can be specified in any order.

Syntax Rules

- Only one chart type subcommand can be specified.

- Keyword SPAN is used only with IR and I subcommands.
- Keyword CONFORM or NONCONFORM is used only with NP and P subcommands.
- Keywords NORMAL and REFERENCE are used only with the CPCHART subcommand.
- Keywords LSL and USL are required when the CPCHART subcommand is specified.

Operations

- All control charts generated by SPCHART contain four basic series: the process, the center line, the upper control line, and the lower control line.
- The chart title, subtitle, and footnote are assigned as they are specified on TITLE, SUBTITLE, and FOOTNOTE subcommands. For control charts, the default chart title is **Control Chart**, followed by the label of the process variable, and a one-line default footnote displays the sigma level. For a process capability chart, the default chart title is **Process Capability** and there is no default footnote. For all chart types, if split-file processing is in effect then a default subtitle is added that provides split-file information.
- For control charts, the category variable label is used as the title for the *category axis*. If no variable label is defined, the variable name is used. If no category variable is defined, the title is null.
- For control charts, the category variable value labels are used as the *category axis* labels. If no value labels are defined, values are used. If no category variable is defined, integer values from 1 to *n* are used, where *n* is the number of subgroups or units plotted.
- For control charts, all series are plotted as lines. When a series has a constant value across all samples, the value is reported in the legend entry for the series.
- For all chart types, case weights are not honored when each case is a subgroup. Case weights are honored when each case is a unit and when the weights are integers. When weighted data are used in an individuals chart, replicated cases are plotted on the control chart.
- For control charts, the calculated control limits are always displayed and can be suppressed only by editing the chart in a chart window.
- You can specify preset control limits for an X-bar or I chart, as some industries often do. The specified control limits are displayed simultaneously with the calculated limits.

Limitations

- Control charts cannot have fewer than 2 points or more than 3000 points.
- The subgroup size in X-bar and range charts cannot exceed 100.
- The span for individuals charts is limited to 100.
- For process capability charts, the subgroup size cannot exceed 100 when /CAPSIGMA=RBAR, and the span cannot exceed 100 when /CAPSIGMA=MRBAR. In addition, the subgroup size or span cannot be less than 2.

Example

```
SPCHART  /TEMPLATE='cntr.sgt'
  /IR=subsize.
```

- This command generates an individuals chart and a moving range chart. The process variable *subsize* is a numeric variable that measures the size variation of the product.
- Both charts use the attributes that are defined for the template that is saved in *cntr.sgt*.
- The default span (2) and sigma value (3) are used.
- Because no BY variable is specified, the *x* axis is labeled by sequence numbers.

TEMPLATE Subcommand

TEMPLATE uses an existing file as a template and applies it to all charts generated by the current SPCHART command. The template overrides the default settings that are used to create any charts, and the specifications on the current SPCHART command override the template. Templates are created in the Chart Editor by saving an existing chart as a template.

Example

```
SPCHART  /TEMPLATE='/templates/cntr.sgt'
  /IR=subsize.
```

TITLE, SUBTITLE, and FOOTNOTE Subcommands

TITLE, SUBTITLE, and FOOTNOTE specify lines of text that are placed at the top or bottom of the control chart.

- One or two lines of text can be specified for TITLE or FOOTNOTE, and one line of text can be specified for SUBTITLE.
- Each line of text must be enclosed in quotes. The maximum length of any line is 72 characters.
- The default font sizes and types are used for the title, subtitle, and footnote.
- By default, the title, subtitle, and footnote are left-aligned with the *y* axis.
- For control charts, if you do not specify TITLE the default is **Control Chart** followed by the label of the process variable. For a process capability chart the default title is **Process Capability**.
- If you do not specify SUBTITLE, the subtitle provides the split-file information if split-file processing is in effect; otherwise, it is null, which leaves more space for the chart.
- For control charts, if you do not specify FOOTNOTE the sigma level is identified as the first line of the footnote. There is no default footnote for a process capability chart.

Example

```
SPCHART TITLE = 'Wheel Production'
  /SUBTITLE = 'Process Control'
  /IR=SUBSIZE.
```

XR and XS Subcommands

XR produces an X-bar chart and an R chart. XS produces an X-bar chart and an s chart. X-bar, R, and s charts are control charts for continuous variables, such as size, weight, length, and temperature.

An X-bar chart plots the mean of each subgroup. The center line indicates the mean of subgroup means. The control limits are calculated from subgroup means, numbers, standard deviations, and the user-specified SIGMAS value. The following figure shows an X-bar chart.

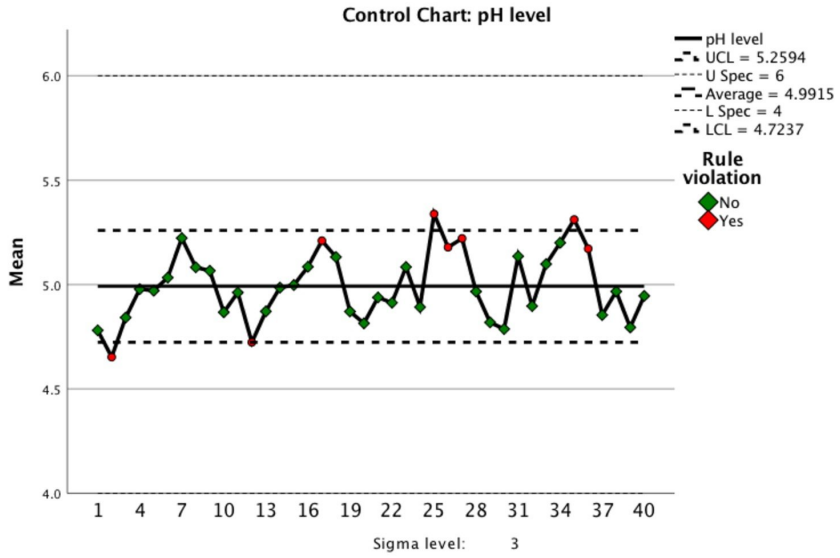


Figure 88. X-bar chart

An R chart plots range values (maximum-minimum) of successive subgroups. The center line indicates the mean of subgroup ranges. The control limits are calculated from subgroup ranges, numbers, and the user-specified SIGMAS value. The R chart tests whether the process variability is in control. When the subgroup size is relatively small (4, 5, or 6), the range method yields almost as good an estimator of the variance as does the subgroup variance. The following figure shows an R chart.

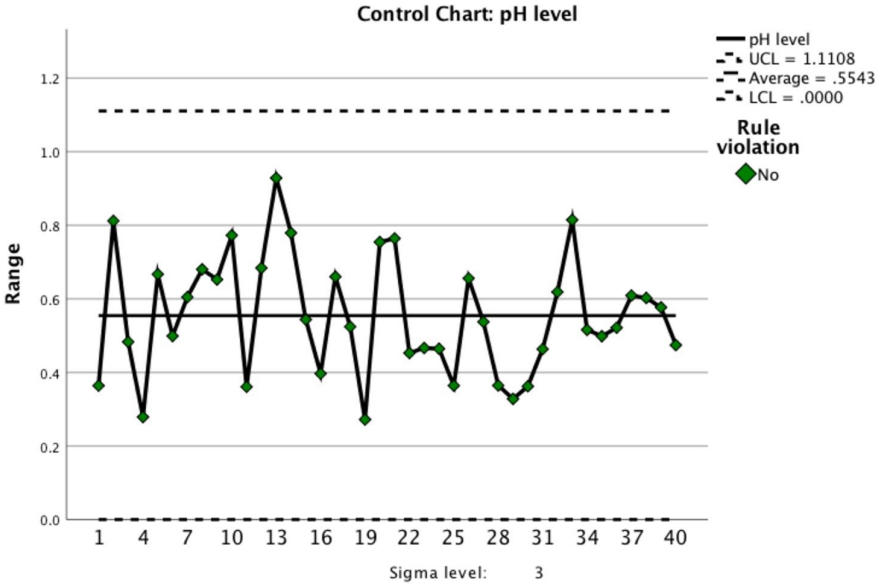


Figure 89. R chart

An s chart plots subgroup standard deviations. The center line indicates the mean of subgroup standard deviations. The control limits are calculated from subgroup standard deviations, numbers, and the user-specified SIGMAS value. The s chart tests whether the process variability is in control, especially when the subgroup size is moderate to large. The following figure shows an s chart.

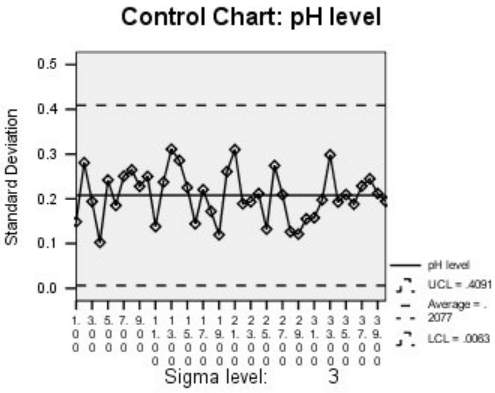


Figure 90. An s chart

Data Organization

For X-bar, R, or s charts, data can be organized where each case is a unit or where each case is a subgroup.

- If each case is a unit with a subgroup identifier, cases are assigned to a category according to the value of the identifier. Table 92 on page 1874 is an example of this type of data organization. The data do not have to be sorted by subgroup. A BY variable (the subgroup identifier) is required to sort and aggregate data and label the process variable.
- If each case is a subgroup, there are as many variables as individuals within one sample. A sample identifier is not required. When there is a sample identifier, it is used for labeling. Table 93 on page 1874 shows this type of organization.

Table 92. Each case is a unit for X-bar, R, and s charts

Subgroup	Length
8:50	6.35
11:30	6.39
8:50	6.40
11:30	6.46
8:50	6.32
11:30	6.37
8:50	6.39
11:30	6.36
...	...

Table 93. Each case is a subgroup for X-bar, R, and s charts

Subgroup	N1	N2	N3	N4
8:50	6.35	6.40	6.32	6.39
11:30	6.39	6.46	6.37	6.36
...	...	...	...	...

Variable Specification

If data are organized as shown in Table 92 on page 1874, the variable specifications on XR and XS subcommands are

```
VAR BY VAR
```

The variable that is specified before BY is the process variable, which is the variable that contains values for all instances to be plotted (for example, *LENGTH* in Table 92 on page 1874). The variable that is specified after BY is the category variable or the BY variable, which is the subgroup identifier (for example, *SUBGROUP* in Table 92 on page 1874). The process variable must be numeric, while the category variable can be of any type. The chart is sorted by the category variable.

If data are organized as shown in Table 93 on page 1874, the variable specifications on XR and XS subcommands are

```
VAR VAR [VAR...] [BY VAR]
```

Each of the variables that is specified before BY represents an instance to be plotted (for example, *N1* to *N3* in Table 93 on page 1874). At least two variables are required, and each variable must be numeric. Keyword BY and the category variable (for example, *SUBGROUP* in Table 93 on page 1874) are optional; if specified, the category variable provides labels for the *category axis* and can be any type of variable. If omitted, the *category axis* is labeled from 1 to the number of variables that are specified before keyword BY.

Example

```
SPCHART /TEMPLATE='ctrl.sgt'
 /XR SUBSIZE BY SHIFT.
```

- The data are organized as shown in Table 92 on page 1874. *SUBSIZE* is a numeric variable that measures the part size. *SHIFT* contains the subgroup identifier (work shift number).
- The chart template is stored in the chart file *ctrl.sgt*.

(XBARONLY) Keyword

(XBARONLY) suppresses the R or s secondary charts. If this keyword is omitted, the R or s chart will be generated with the X-bar chart.

Example

```
SPCHART
  /XR=ph BY time (XBARONLY)
  /CAPSIGMA=RBAR
  /SIGMAS=3
  /MINSAMPLE=2.
```

I and IR Subcommands

I produces an individuals chart, and IR produces an individuals chart and a moving range chart. Both types are control charts for continuous variables, such as size, weight, length, and temperature.

An individuals chart plots each individual observation on a control chart. The center line indicates the mean of all individual values, and the control limits are calculated from the mean of the moving ranges, the span, and the user-specified SIGMAS value. Individuals charts are often used with moving range charts to test process variability when the subgroup size is 1. This situation occurs frequently when automated inspection and measurement technology is used and every manufactured unit is analyzed. The situation also occurs when the process is so slow that a larger subgroup size becomes impractical. The following figure shows an individuals chart.

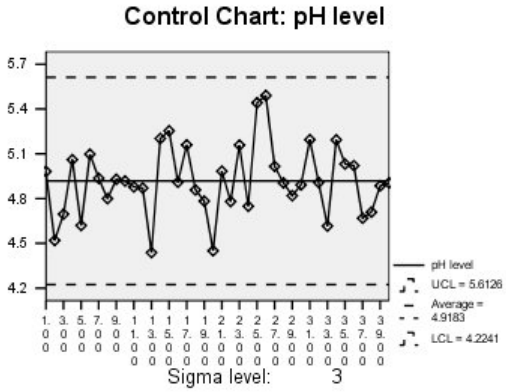


Figure 91. Individuals chart

A moving range chart plots moving ranges of *n* successive observations on a control chart, where *n* is the specified span (see). The center line is the mean of moving ranges, and the control limits are calculated from the ranges, the span, and the user-specified SIGMAS value (see). The following figure shows a moving range chart.

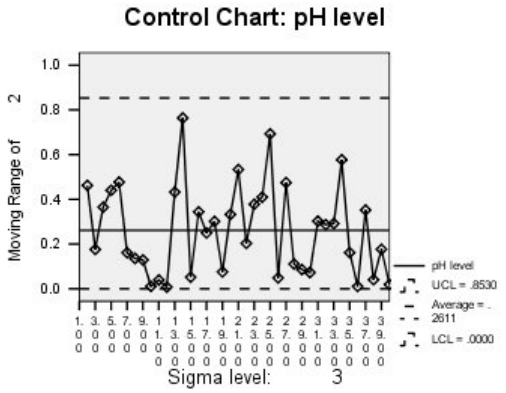


Figure 92. Moving range chart

Data Organization

For individuals charts and moving range charts, data must be organized so that each case is a unit. Cases are not sorted or aggregated before plotting.

Variable Specification

The variable specification for I or IR subcommand is

```
VAR [BY VAR]
```

You must specify the process variable that contains the value for each individual observation. Each observation is plotted for the individuals chart. The range of *n* consecutive observations (where *n* is the value that is specified on the SPAN subcommand) is calculated and plotted for the moving range chart. The range data for the first *n*-1 cases are missing, but the mean and the limit series are not missing.

Keyword BY and the category variable are optional. When specified, the category variable is used for labeling the *category axis* and can be any type of variable. If omitted, the *category axis* is labeled 1 to the number of individual observations in the process variable.

Example

```
SPCHART /TEMPLATE='ctrl.sgt'
  /IR=SUBSIZE.
```

- This command requests an individuals chart and a moving range chart.
- The default span (2) and sigma value (3) are used.

P and NP Subcommands

P produces a p chart and NP produces an np chart. Both charts are control charts for attributes. That is, these charts use data that can be counted, such as the number of nonconformities and the percentage of defects.

A p chart plots the **fraction nonconforming** on a control chart. Fraction nonconforming is the proportion of nonconforming or defective items in a subgroup to the total number of items in that subgroup. This measurement is expressed as a decimal or, occasionally, as a percentage. The center line of the control chart is the mean of the subgroup fractions, and the control limits are based on a binomial distribution and can be controlled by the user-specified SIGMAS value.

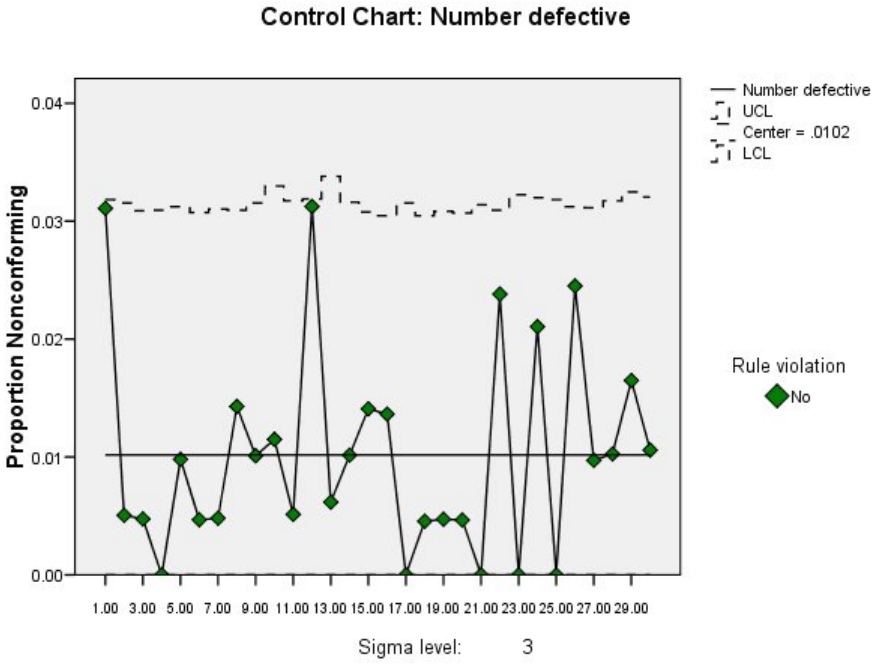


Figure 93. A p chart

An np chart plots the number nonconforming rather than the fraction nonconforming. The center line is the mean of the numbers of nonconforming or defective items. The control limits are based on the binomial distribution and can be controlled by the user-specified SIGMAS value. When the subgroup sizes are unequal, np charts are not recommended.

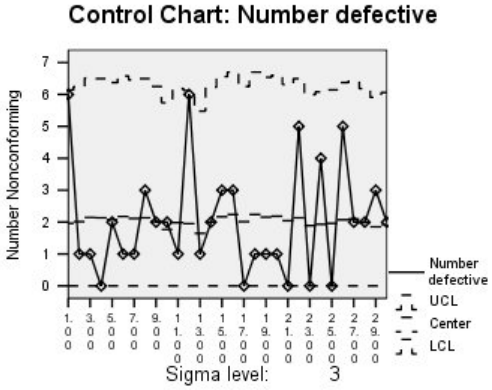


Figure 94. A np chart

Data Organization

Data for p and np charts can be organized where each case is a unit or where each case is a subgroup.

- If each case is a unit with a conformity status variable and a subgroup identifier, cases are assigned to a category by the value of the subgroup identifier. Table 94 on page 1878 is an example of this type of data organization. The data do not have to be sorted. A BY variable (the subgroup identifier) is required to sort and aggregate data and label the *category axis*.
- If each case is a subgroup, one variable contains the total number of items within a subgroup, and one variable contains the total number of nonconforming or defective items in the subgroup. The subgroup identifier is optional. If specified, the subgroup identifier is used for labeling purposes. Table 95 on page 1878 is an example of this type of data organization. The data are the same as the data that are used in Table 94 on page 1878.

Table 94. Each case is a unit for p and np charts

Subgroup	Outcome
January	Cured
January	Cured
January	Cured
January	Relapse
February	Relapse
February	Cured
February	Relapse
February	Relapse
...	...

Table 95. Each case is a subgroup for p and np charts

Subgroup	Relapse	N
January	1	4
February	3	4
...	...	...

Variable Specification

If data are organized as illustrated in Table 94 on page 1878, the variable specification on P or NP subcommands is

```
VAR BY VAR
```

The variable that is specified before BY is the status variable (for example, *OUTCOME* in Table 94 on page 1878). The value of this variable determines whether an item is considered conforming or nonconforming. The status variable can be any type, but if it is a string, the value that is specified on CONFORM (or NONCONFORM) must be enclosed in quotes (see). The variable that is specified after BY is the category variable and can be any type of variable. The chart is sorted by values of the category variable.

If data are organized as shown in Table 95 on page 1878, the variable specification on P or NP is

```
COUNT(VAR) N({VAR})  [BY VAR]
             {VAL}
```

The variable that is specified on keyword COUNT is the variable that contains the number of nonconforming or defective items (for example, *RELAPSE* in Table 95 on page 1878). The specification on keyword N is either the variable that contains the sample size or a positive integer for a constant size

across samples (for example, *N* in Table 95 on page 1878). The COUNT variable cannot be larger than the N variable for any given subgroup; if it is larger, the subgroup is dropped from calculation and plotting. Keyword BY and the category variable are optional. When specified, the category variable is used for *category axis* labels; otherwise, the *category axis* is labeled 1 to the number of subgroups. Cases are unsorted for the control chart.

C and U Subcommands

C produces a c chart and U produces a u chart. Both charts are control charts for attributes. That is, the charts use data that can be counted.

A c chart plots the total number of defects or nonconformities in each subgroup. A defect or nonconformity is one specification that an item fails to satisfy. Each nonconforming item has at least one defect, but any nonconforming item may have more than one defect. The center line of the c chart indicates the mean of the defect numbers of all subgroups. The control limits are based on the Poisson distribution and can be controlled by the user-specified SIGMAS value. When the sample sizes are not equal, c charts are not recommended.

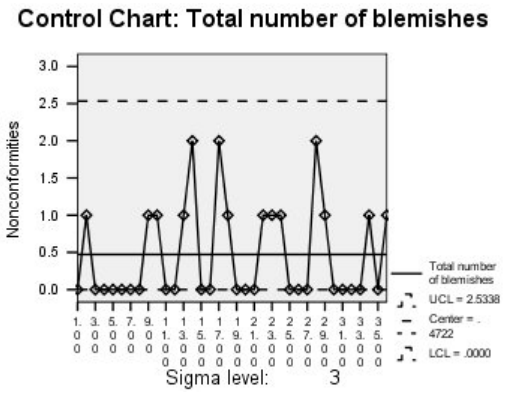


Figure 95. A c chart

A u chart plots the average number of defects or nonconformities per inspection unit within a subgroup. Each subgroup contains more than one inspection unit. The center line of the u chart indicates the average number of defects per unit of all subgroups. The control limits are based on Poisson distribution and can be controlled by the user-specified SIGMAS value.

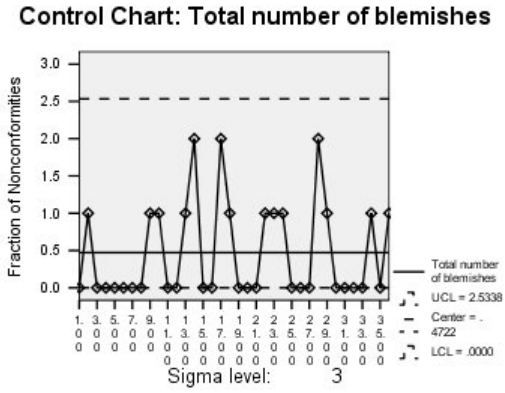


Figure 96. A u chart

Data Organization

Data for c and u charts can be organized where each case is a unit or where each case is a subgroup.

- If each case is a unit with a variable containing the number of defects for that unit and a subgroup identifier, cases are assigned to each subgroup by the value of the identifier. Table 96 on page 1880 is

an example of this type of data organization. Data do not have to be sorted by subgroup. A BY variable (the subgroup identifier) is required to sort and aggregate data and to label the *category axis*.

- If each case is a subgroup, one variable contains the total number of units within the subgroup, and one variable contains the total number of defects for all units within the subgroup. The subgroup identifier is optional. When specified, the subgroup identifier is used as *category axis* labels; otherwise, the number 1 to the number of subgroups are used to label the *category axis*. Table 97 on page 1880 is an example of this method of data organization. The data are the same as the data in Table 96 on page 1880.

Table 96. Each case is a unit for c and u charts

ID	Subgroup	Count
1	January	0
2	January	2
3	January	0
4	January	0
5	February	5
6	February	1
7	February	0
8	February	0
...	...	...

Table 97. Each case is a subgroup for c and u charts

Subgroup	Relapses	N
JANUARY	1	4
FEBRUARY	3	4
...	...	...

Variable Specification

If data are organized as shown in Table 96 on page 1880, the variable specification on C and U subcommands is

```
VAR BY VAR
```

The variable that is specified before keyword BY contains the number of defects in each unit (for example, *COUNT* in Table 96 on page 1880). The variable must be numeric. The variable that is specified after keyword BY is the subgroup identifier (for example, *SUBGROUP* in Table 96 on page 1880). This variable can be any type of variable. The chart is sorted by values of the subgroup identifier.

If data are organized as shown in Table 97 on page 1880, the variable specification on C and U subcommands is

```
COUNT(VAR) N({VAR}) [BY VAR]
             {VAL}
```

The specification is the same as the specification for p and np charts.

CPCHART Subcommand

CPCHART produces a process capability chart. The default process capability chart consists of a histogram of the process values, a superimposed normal curve based on the capability sigma and the process mean, and vertical reference lines at the fixed upper and lower control limits and target value (if

specified). It also displays the process capability indices, the process performance indices, and process data such as the process mean and process standard deviation.

- When CPCHART is specified the LSL and USL subcommands are required. The TARGET subcommand is optional with CPCHART.
- You can control which normal curves are superimposed on the histogram with the NORMAL subcommand.
- You can display vertical reference lines at the plus or minus three sigma level, or a user-specified sigma level, with the REFERENCE subcommand.
- All of the statistics displayed on the process capability chart are available in table form from the STATISTICS subcommand.
- Process capability charts have a default title of **Process Capability**. You can specify your own title using the TITLE and SUBTITLE subcommands.

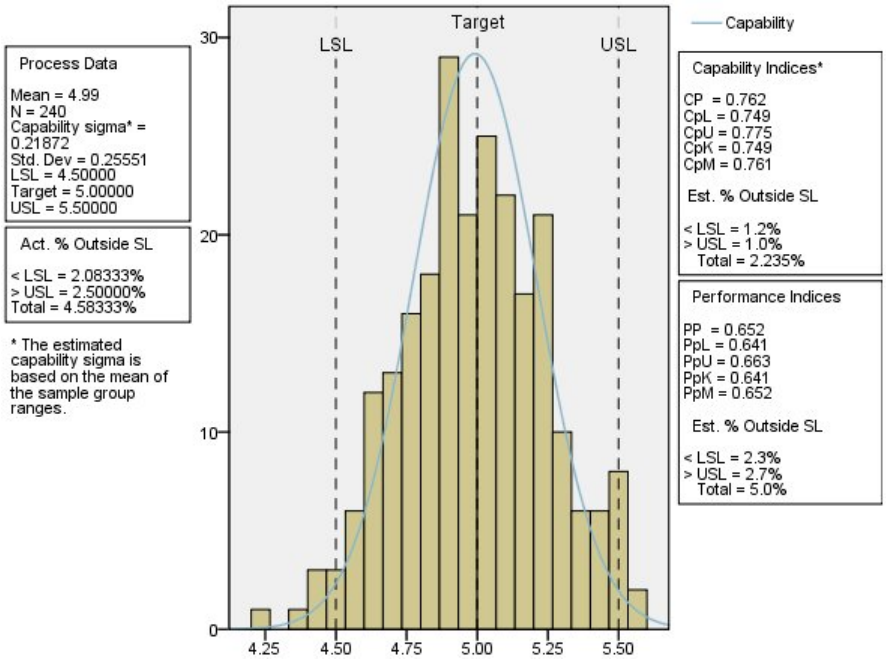


Figure 97. Process capability chart

Data Organization

The data can be organized in the following ways:

- Each case is a unit with a subgroup identifier that is specified with a BY variable. Table 98 on page 1881 is an example of this type of data organization. The data do not have to be sorted by subgroup.
- Each case is a subgroup and there are as many variables as individuals within one sample. Table 99 on page 1882 shows this type of organization.
- Each case is a unit and there are no subgroups. Table 100 on page 1882 shows this type of organization. Cases are not sorted or aggregated before plotting.

Table 98. Each case is a unit with a subgroup identifier

Subgroup	Length
8:50	6.35
11:30	6.39
8:50	6.40
11:30	6.46

Table 98. Each case is a unit with a subgroup identifier (continued)

Subgroup	Length
8:50	6.32
11:30	6.37
8:50	6.39
11:30	6.36
...	...

Table 99. Each case is a subgroup

N1	N2	N3	N4
6.35	6.40	6.32	6.39
6.39	6.46	6.37	6.36
...	...	...	...

Table 100. Each case is a unit and there are no subgroups

pH
4.9
4.5
4.7
5.1
4.6
5.1
4.9
4.8
...

Variable Specification

If data are organized as shown in Table 98 on page 1881, the variable specification on the CPCHART subcommand is

```
var BY var
```

The variable that is specified before BY is the process variable, which is used to generate the histogram (for example, *length* in Table 98 on page 1881). The variable that is specified after BY is the subgroup identifier (for example, *subgroup* in Table 98 on page 1881). The process variable must be numeric, while the subgroup identifier variable can be of any type.

If data are organized as shown in Table 99 on page 1882, the variable specification on the CPCHART subcommand is

```
var var [var...]
```

Each variable contains process values for a particular subgroup. Values from all subgroups are used to generate the histogram. At least two variables are required, and each variable must be numeric.

If data are organized as shown in Table 100 on page 1882, the variable specification on the CPCHART subcommand is

```
var
```

The specified variable contains the process values used to generate the histogram, and must be numeric.

Example

```
SPCHART /TEMPLATE='cpchart.sgt'
 /CPCHART length BY shift.
```

- The data are organized as shown in Table 98 on page 1881. *length* is a numeric variable that measures the length of a part. *shift* contains the subgroup identifier (work shift number).
- The chart uses the chart template from *cpchart.sgt*.

STATISTICS Subcommand

Any keyword may be specified in any place in the subcommand, but for conceptual clarity, the keywords are organized as follows: the Process Capability Indices, the Process Performance Indices, the Process Data, and the Measure(s) for Assessing Normality.

- This subcommand is silently ignored if the chart is not an XR, XS, IR, I, or CPCHART chart.
- A duplicated subcommand name causes a syntax error.
- A duplicated keyword is silently ignored.
- There is no default keyword or parameter value.

The Process Capability Indices

CP. *Capability of the process.*

CPU. *The distance between the process mean and the upper specification limit scaled by capability sigma.*

CPL. *The distance between the process mean and the lower specification limit scaled by capability sigma.*

K. *The deviation of the process mean from the midpoint of the specification limits.* This measurement is computed independently of the estimated capability sigma.

CPK. *Capability of the process related to both dispersion and centeredness.* It is the minimum of CpU and CpL. If only one specification limit is provided, we compute and report a unilateral CpK instead of taking the minimum.

CR. *The reciprocal of CP.*

CPM. *An index relating capability sigma and the difference between the process mean and the target value.* A target value must be specified on the TARGET subcommand by the user.

CZU. *The number of capability sigmas between the process mean and the upper specification limit.*

CZL . *The number of capability sigmas between the process mean and the lower specification limit.*

CZMIN . *The minimum number of capability sigmas between the process mean and the specification limits.*

CZMAX. *The maximum number of capability sigmas between the process mean and the specification limits.*

CZOUT. *The estimated percentage outside the specification limits.* The standard normal approximation is based on CZ_U and CZ_L.

CZLOUT. *The estimated percentage outside the lower specification limit.* The standard normal approximation is based on CZ_L.

CZUOUT. *The estimated percentage outside the upper specification limit.* The standard normal approximation is based on CZ_U.

- For each of the keywords (other than CPK), both the LSL subcommand and the USL subcommand must be specified. Otherwise, the keyword(s) are ignored, and a syntax warning is issued. For CPK, at least one of the LSL and USL subcommands must be specified.
- If the TARGET subcommand is not specified, the keyword CPM is ignored, and a syntax warning is issued.

The Process Performance Indices

PP. *Performance of the process.*

PPU. *The distance between the process mean and the upper specification limit scaled by process standard deviation.*

PPL. *The distance between the process mean and the lower specification limit scaled by process standard deviation.*

PPK. *Performance of the process related to both dispersion and centeredness.* It is the minimum of PpU and PpL. If only one specification limit is provided, we compute and report a unilateral PpK instead of taking the minimum.

PR. *The reciprocal of PP.*

PPM. *An index relating process variance and the difference between the process mean and the target value.* A target value must be specified on the TARGET subcommand by the user.

PZU. *The number of standard deviations between the process mean and the upper specification limit.*

PZL. *The number of standard deviations between the process mean and the lower specification limit.*

PZMIN. *The minimum number of standard deviations between the process mean and the specification limits.*

PZMAX. *The maximum number of standard deviations between the process mean and the specification limits.*

PZOUT. *The estimated percentage outside the specification limits.* The standard normal approximation is based on PZ_U and PZ_L.

PZLOUT. *The estimated percentage outside the lower specification limit.* The standard normal approximation is based on PZ_L.

PZUOUT. *The estimated percentage outside the upper specification limit.* The standard normal approximation is based on PZ_U.

- For each of the keywords (other than PPK), both the LSL subcommand and the USL subcommand must be specified. Otherwise, we ignore the keyword(s) and issue a syntax warning. For PPK, at least one of the LSL and USL subcommands must be specified.
- If the TARGET subcommand is not specified, the keyword PPM is ignored, and a syntax warning is issued.

Process Data

N. *Total sample size.*

MEAN. *The process mean.*

STDDEV. *The process standard deviation.*

CAPSIGMA. *The capability sigma.*

LSL. *The fixed lower control limit.*

USL. *The fixed upper control limit.*

TARGET. *The target value.*

Measure(s) for Assessing Normality

AZOUT. *The observed percentage outside the specification limits.* A point is defined outside the specification limits when its value is greater than or equal to the upper specification limit or is less than or equal to the lower specification limit.

AZLOUT. *The observed percentage outside the lower specification limit.* A point is defined outside the lower specification limit when its value is less than or equal to the lower specification limit.

AZUOUT. *The observed percentage outside the upper specification limit.* A point is defined outside the upper specification limit when its value is greater than or equal to the upper specification limit.

- For each of the keywords, both the LSL subcommand and the USL subcommand must be specified. Otherwise, we ignore the keyword and issue a syntax warning.

RULES Subcommand

RULES specifies the rules for identifying out-of-control points. If a point violates any rule, it appears in the primary chart with a different shape and color compared to in-control points. Note that only the last point in a discovered pattern shall be marked for a rule violation, and any and all overlapping patterns are treated as separate patterns. A table of rule violations is also included in the output that displays the points marked in the control chart. If desired, use the ID keyword to specify the variable that identifies points in this table.

- Any keyword may be specified in any place in the subcommand.
- A duplicated subcommand name causes a syntax error.
- A duplicated keyword is silently ignored.
- The default keyword is ALL.
- If the subcommand is omitted, no control rules are used.
- RULES does not apply when generating a process capability chart (CPCHART subcommand).

ALL. *All rules.*

UCL. *Greater than +3 sigma.*

R2UPPER. *2 points out of the last 3 greater than +2 sigma.*

R4UPPER. *4 points out of the last 5 greater than +1 sigma.*

R8UPPER. *8 consecutive points above the center line.*

R8LOWER. *8 consecutive points below the center line.*

R4LOWER. *4 points out of the last 5 less than -1 sigma.*

R2LOWER. *2 points out of the last 3 less than -2 sigma.*

LCL. *Less than -3 sigma.*

TRUP. *6 consecutive points trending up.*

TRDOWN. *6 consecutive points trending down.*

ALTERNATING. *14 consecutive points alternating.*

ID Subcommand

ID specifies a variable that identifies points in the table of rule violations. If this subcommand is omitted, the BY variable is used. Without the RULES subcommand, ID has no effect.

CAPSIGMA Subcommand

This subcommand defines the capability sigma estimator, which is required in computing all the Process Capability Indices except K. There are four options:

RBAR. *Mean sample range.* The estimated capability sigma is based on the mean of the sample group ranges.

SBAR. *Mean sample standard deviation.* The estimated capability sigma is based on the mean of the sample group standard deviations.

MRBAR. *Mean sample moving range.* The estimated capability sigma is based on the mean of the sample moving ranges. The span that is defined by the SPAN subcommand is used. (Recall that its passive default value is 2.)

WITHIN. *Sample within-group variance.* The estimated capability sigma is the square root of the sample within-group variance.

The validity of specification depends on the chart specification (i.e., /XR, /XS, /I, /IR, or /CPCHART).

Table 101. Valid CAPSIGMA options by chart specification

Chart Specification	Valid CAPSIGMA Options
XR	RBAR (default), SBAR, WITHIN
XS	RBAR, SBAR (default), WITHIN
I	MRBAR (default)
IR	MRBAR (default)
CPCHART	RBAR, SBAR, WITHIN, MRBAR

- When this subcommand is omitted or specified without a keyword by the user, the default conditional on the chart specification is implicitly assumed (see the table above). When the chart specification is /CPCHART, the default is RBAR when the data are organized in subgroups, and MRBAR otherwise.
- An invalid specification (e.g., /I and /CAPSIGMA = RBAR) causes a syntax error when the chart specification is /XR, /XS, /I, or /IR and an applicable STATISTICS keyword is specified. An invalid specification (for those same chart types) without an applicable STATISTICS keyword generates a syntax warning.
- When the chart specification is /CPCHART, an invalid specification (e.g., /CAPSIGMA = RBAR when the data are not organized in subgroups) causes a syntax error.
- When the chart specification is /XR, /XS, /I, or /IR, CAPSIGMA is silently ignored if the subcommand specification is valid (with respect to the chart specification) but an applicable STATISTICS keyword is not specified.
- CAPSIGMA is silently ignored if the chart specification is not /XR, /XS, /I, /IR, or /CPCHART.
- A duplicated subcommand name causes a syntax error.
- A duplicated keyword is silently ignored, but if two or more keywords are specified and they do not have identical meanings, a syntax error message is issued.

SPAN Subcommand

SPAN specifies the span from which the moving range for an individuals chart or process capability chart (when the data are specified in the form var) is calculated. The specification must be an integer value that is greater than 1. The default is 2. SPAN applies only to I, IR, and CPCHART chart specifications.

Example

```
SPCHART /IR=SUBSIZE /SPAN=5.
```

- The SPAN subcommand specifies that the moving ranges are computed from every five individual samples.

CONFORM and NONCONFORM Subcommands

Either CONFORM or NONCONFORM is required when you specify a status variable on the P or NP subcommand. You make that specification when data are organized so that each case is an inspection unit (see).

- Either subcommand requires a value specification. The value can be numeric or string. String values must be enclosed within quotes.
- If CONFORM is specified, all values for the status variable (other than the specified value) are tabulated as nonconformities. If NONCONFORM is specified, only the specified value is tabulated as a nonconformity.
- CONFORM and NONCONFORM apply only to P and NP chart specifications.

SIGMAS Subcommand

SIGMAS allows you to define the sigma level for a control chart. The value specified on SIGMAS is used in calculating the upper and lower control limits on the chart. It also determines the placement of reference lines on a process capability chart (/CPCHART) when the REFERENCE subcommand includes the keyword SIGMAS. You can specify a number larger than 1 but less than or equal to 10. A larger SIGMAS value means a greater range between the upper and the lower control limits. The default is 3.

MINSAMPLE Subcommand

MINSAMPLE specifies the minimum sample size for X-bar, R, or s charts, as well as process capability charts when the data are organized in subgroups. When you specify XR, XS, or CPCHART on SPCHART, any subgroup with a size that is smaller than the size that is specified on MINSAMPLE is excluded from the chart and from all computations. If each case is a subgroup, there must be at least as many variables named as the number that is specified on MINSAMPLE. The default, as well as the minimum, is 2.

LSL and USL Subcommand

LSL and USL allow you to specify fixed lower and upper control limits. Fixed control limits are often used in manufacturing processes as designer-specified limits.

- For chart types other than CPCHART, these limits are displayed on the chart, along with the calculated limits. If you do not specify LSL and USL, no fixed control limits are displayed. However, if you want only the specified control limits, you must edit the chart in a chart window to suppress the calculated series.
- For a chart type of CPCHART (process capability chart), LSL and USL are required. The REFERENCE subcommand controls whether these limits are displayed on the process capability chart. By default, they are displayed.

Example

```
SPCHART /TEMPLATE='ctrl.sgt'
  /XS=SUBSIZE
  /USL=74.50
  /LSL=73.50.
```

- The USL and LSL subcommands specify the control limits according to the designing engineer. The center line is probably at 74.00.
- The specified upper and lower limits are displayed together with the control limits that are calculated from the observed standard deviation and the sigma value.

TARGET Subcommand

This subcommand defines the target value that is used in computing CpM and PpM requested by the STATISTICS subcommand. It is also the target value that is displayed by default on a process capability chart (/CPCHART). The value may be any real number that is less than or equal to the USL value and greater than or equal to the LSL value.

- This subcommand is silently ignored if no applicable STATISTICS keyword is specified or the chart type is not CPCHART.
- If the value is numeric but out of the valid range, we issue a warning and ignore the CPM and/or PPM keyword(s), if any, in the STATISTICS subcommand.

MISSING Subcommand

MISSING controls the treatment of missing values.

- The default is NOREPORT and EXCLUDE.
- REPORT and NOREPORT are alternatives and apply only to category variables. REPORT and NOREPORT control whether categories (subgroups) with missing values are created. They do not apply to process capability charts (/CPCHART).
- INCLUDE and EXCLUDE are alternatives and apply to process variables.

NOREPORT. *Suppress missing-value categories.* This setting is the default.

REPORT. *Report and plot missing-value categories.*

EXCLUDE . *Exclude user-missing values.* Both user-missing and system-missing values for the process variable are excluded from computation and plotting. This setting is the default.

INCLUDE . *Include user-missing values.* Only system-missing values for the process variable are excluded from computation and plotting.

NORMAL Subcommand

NORMAL controls which normal curves are superimposed on a process capability chart. It only applies when the chart specification is /CPCHART. Specify one of the following values:

CAPABILITY. *Superimposes a normal curve with mean equal to the process mean and standard deviation equal to the capability sigma.* This is the default.

PERFORMANCE . *Superimposes a normal curve with mean equal to the process mean and standard deviation equal to the process standard deviation.*

BOTH . *Superimposes both the CAPABILITY and PERFORMANCE normal curves.*

NONE. *Specifies that no normal curves will be displayed.*

REFERENCE Subcommand

REFERENCE controls which vertical reference lines are displayed on a process capability chart. It only applies when the chart specification is /CPCHART.

SL. *Vertical reference lines are displayed at the specified values of LSL and USL as well as at the value of TARGET if specified.* This is the default.

SIGMAS. *Vertical reference lines are displayed at the process mean and at a specified number of capability sigmas above and below the mean.* The number of capability sigmas is given by the value of the SIGMAS subcommand, which is 3 by default.

NONE. *Specifies that no reference lines will be displayed.* This is an alternative to SL and/or SIGMAS and cannot be used in combination with them.

SPECTRA

SPECTRA is available in SPSS Statistics Professional Edition or the Forecasting option.

```
SPECTRA VARIABLES= series names

 [/{CENTER NO**}]
   {CENTER     }

 [/{CROSS NO**}]
   {CROSS     }

 [/WINDOW={HAMMING** [({5   })]    }]
          {          {span}      }
          {BARTLETT [(span)]       }
          {PARZEN [(span)]         }
          {TUKEY [(span)]          }
          {UNIT or DANIELL [(span)]}
          {NONE                    }
          {w-p, ..., w0, ..., wp   }

 [/PLOT= [P] [S] [CS] [QS] [PH] [A]
         [G] [K] [ALL] [NONE]
         [BY {FREQ  }]]
             {PERIOD}

 [/SAVE = [FREQ (name)] [PER (name)] [SIN (name)]
          [COS (name)]  [P (name)]   [S (name)]
          [RC (name)]   [IC (name)]  [CS (name)]
          [QS (name)]   [PH (name)]  [A (name)]
          [G (name)]    [K (name)]]

 [/APPLY [='model name']]
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
SPECTRA VARIABLES = HSTARTS.
```

Overview

SPECTRA plots the periodogram and spectral density function estimates for one or more series. You can also request bivariate spectral analysis. Moving averages, termed *windows,* can be used for smoothing the periodogram values to produce spectral densities.

Options

Output. In addition to the periodogram, you can use the PLOT subcommand to produce a plot of the estimated spectral density. You can use the keyword BY on PLOT to suppress the display of the plot by frequency or the plot by period. To reduce the range of values that are displayed in the plots, you can center the data by using the CENTER subcommand.

Cross-Spectral Analysis. You can specify cross-spectral (bivariate) analysis with the CROSS subcommand and select which bivariate plots are produced by using PLOT.

New Variables. Variables that are computed by SPECTRA can be saved to the active dataset for use in subsequent analyses with the SAVE subcommand. TSET MXNEWVAR specifies the maximum number of new variables that can be generated by a procedure. The default is 60.

Spectral Windows. You can specify a spectral window and its span for calculation of the spectral density estimates.

Basic Specification

The basic specification is one or more series names.

- By default, SPECTRA plots the periodogram for each specified series. The periodogram is shown first by frequency and then by period. No new variables are saved by default.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- VARIABLES can be specified only once.
- Other subcommands can be specified more than once, but only the last specification of each subcommand is executed.

Operations

- SPECTRA cannot process series with missing observations. (You can use the RMV command to replace missing values, use TSET MISSING=INCLUDE to include user-missing values, and use USE to ignore missing observations at the beginning or end of a series. See RMV and USE for more information.)
- If the number of observations in the series is odd, the first case is ignored.
- If the SAVE subcommand is specified, new variables are created for each specified series. For bivariate analyses, new variables are created for each series pair.
- SPECTRA requires memory both to compute variables and to build plots. Requesting fewer plots may enable you to analyze larger series.

Limitations

- A maximum of one VARIABLES subcommand is allowed. There is no limit on the number of series named on the list.

Example

```
SPECTRA VARIABLES = HSTARTS
  /CENTER
  /PLOT P S BY FREQ.
```

- This example produces a plot of the periodogram and spectral density estimate for series *HSTARTS*.
- CENTER adjusts the series to have a mean of 0.
- PLOT specifies that the periodogram (P) and the spectral density estimate (S) should be plotted against frequency (BY FREQ).

VARIABLES Subcommand

VARIABLES specifies the series names and is the only required subcommand.

- VARIABLES must be specified before the other subcommands.
- Each specified series is analyzed separately unless the CROSS subcommand is specified.
- The series must contain at least six cases.

Example

```
SPECTRA VARIABLES = VARX VARY.
```

- This command produces the default display for two series, *VARX* and *VARY*.

CENTER Subcommand

CENTER adjusts the series to have a mean of 0. This process reduces the range of values that are displayed in the plots.

- If CENTER is not specified, the ordinate of the first periodogram value is $2n$ times the square of the mean of the series, where n is the number of cases.

- You can specify CENTER NO to suppress centering when applying a previous model with APPLY.

Example

```
SPECTRA VARIABLES = VARX VARY
  /CENTER.
```

- This example produces the default display for *VARX* and *VARY*. The plots are based on the series after their means have been adjusted to 0.

WINDOW Subcommand

WINDOW specifies a spectral window to use when the periodogram is smoothed to obtain the spectral density estimate. If WINDOW is not specified, the Tukey-Hamming window with a span of 5 is used.

- The specification on WINDOW is a window name and a span in parentheses, or a sequence of user-specified weights.
- The window name can be any one of the keywords listed below.
- Only one window keyword is accepted. If more than one keyword is specified, the first keyword is used.
- The span is the number of periodogram values in the moving average and can be any integer. If an even number is specified, it is decreased by 1.
- Smoothing near the end of series is accomplished via reflection. For example, if the span is 5, the second periodogram value is smoothed by averaging the first, third, and fourth values and twice the second value.

The following data windows can be specified. Each formula defines the upper half of the window. The lower half is symmetric with the upper half. In all formulas, *p* is the integer part of the number of spans divided by 2, D_p is the Dirichlet kernel of order *p*, and F_p is the Fejer kernel of order *p* [79].

HAMMING. *Tukey-Hamming window.*

TUKEY. *Tukey-Hanning window.*

PARZEN. *Parzen window.*

BARTLETT. *Bartlett window.*

UNIT. *Equal-weight window.* The weights are $w_k = 1$ where $k=0, \ldots p$. DANIELL is an alias for UNIT.

NONE. *No smoothing.* If NONE is specified, the spectral density estimate is the same as the periodogram.

w. *User-specified weights.* W_0 is applied to the periodogram value that is being smoothed, and the weights on either side are applied to preceding and following values. If the number of weights is even, it is assumed that w_p is not supplied. The weight after the middle one is applied to the periodogram value being smoothed. W_0 must be positive.

Example

```
SPECTRA VARIABLES = VAR01
  /WINDOW=TUKEY(3)
  /PLOT=P S.
```

- In this example, the Tukey window weights with a span of 3 are used.
- The PLOT subcommand plots both the periodogram and the spectral density estimate, both by frequency and period.

PLOT Subcommand

PLOT specifies which plots are displayed.

[79] Priestley, M. B. 1981. *Spectral analysis and time series, volumes 1 and 2*. London: Academic Press.

- If PLOT is not specified, only the periodogram is plotted for each specified series. Each periodogram is shown both by frequency and by period.
- You can specify more than one plot keyword.
- Keywords can be specified in any order.
- The plot keywords K, CS, QS, PH, A, and G apply only to bivariate analyses. If the subcommand CROSS is not specified, these keywords are ignored.
- The period (horizontal) axis on a plot by period (BY PERIOD) is scaled in natural logarithms from 0.69 to ln(*n*), where *n* is the number of cases.
- The frequency (horizontal) axis on a plot by frequency (BY FREQ) is scaled from 0 to 0.5, expressing the frequency as a fraction of the length of the series.
- The periodogram and estimated spectrum (vertical axis) are scaled in natural logs.

The following plot keywords are available:

P. *Periodogram.* This setting is the default.

S. *Spectral density estimate.*

K. *Squared coherency.* Applies only to bivariate analyses.

CS. *Cospectral density estimate.* Applies only to bivariate analyses.

QS. *Quadrature spectrum estimate.* Applies only to bivariate analyses.

PH. *Phase spectrum.* Applies only to bivariate analyses.

A. *Cross amplitude.* Applies only to bivariate analyses.

G. *Gain.* Applies only to bivariate analyses.

ALL. *All plots.* For bivariate analyses, this setting includes all plots listed above. For univariate analyses, this setting includes the periodogram and the spectral density estimate.

BY Keyword

By default, SPECTRA displays both frequency and period plots. You can use BY to produce only frequency plots or only period plots.

- BY FREQ indicates that all plots are plotted by frequency only. Plots by period are not produced.
- BY PERIOD indicates that all plots are plotted by period only. Plots by frequency are not produced.

Example

```
SPECTRA VARIABLES = SER01
  /PLOT=P S BY FREQ.
```

- This command plots both the periodogram and the spectral density estimate for *SER01*. The plots are shown by frequency only.

CROSS Subcommand

CROSS is used to specify bivariate spectral analysis.

- When CROSS is specified, the first series named on the VARIABLES subcommand is the independent variable. All remaining variables are dependent.
- Each series after the first series is analyzed with the first series independently of other series that is named.
- Univariate analysis of each specified series is still performed.
- You can specify CROSS NO to turn off bivariate analysis when applying a previous model with APPLY.

Example

```
SPECTRA VARIABLES = VARX VARY VARZ
  /CROSS.
```

- In this example, bivariate spectral analyses of series *VARX* with *VARY* and *VARX* with *VARZ* are requested in addition to the usual univariate analyses of *VARX*, *VARY*, and *VARZ*.

SAVE Subcommand

SAVE saves computed SPECTRA variables to the active dataset for later use. SPECTRA displays a list of the new variables and their labels, showing the type and source of those variables.

- You can specify any or all of the output keywords listed below.
- A name to be used for generating variable names must follow each output keyword. The name must be enclosed in parentheses.
- For each output keyword, one variable is created for each series named on SPECTRA and for each bivariate pair.
- The keywords RC, IC, CS, QS, PH, A, G, and K apply only to bivariate analyses. If CROSS is not specified, these keywords are ignored.
- SAVE specifications are not used when models are reapplied by using APPLY. They must be specified each time variables are to be saved.
- The output variables correspond to the Fourier frequencies. They do not correspond to the original series.
- Because each output variable has only (n/2 + 1) cases (where n is the number of cases), the values for the second half of the series are set to system-missing.
- Variable names are generated by adding _n to the specified name, where n ranges from 1 to the number of series specified.
- For bivariate variables, the suffix is _n_n, where the ns indicate the two variables that are used in the analysis.
- The frequency (FREQ) and period (PER) variable names are constant across all series and do not have a numeric suffix.
- If the generated variable name is longer than the maximum variable name length, or if the specified name already exists, the variable is not saved.

The following output keywords are available:

FREQ . *Fourier frequencies.*

PER. *Fourier periods.*

SIN. *Value of a sine function at the Fourier frequencies.*

COS. *Value of a cosine function at the Fourier frequencies.*

P. *Periodogram values.*

S. *Spectral density estimate values.*

RC. *Real part values of the cross-periodogram.* Applies only to bivariate analyses.

IC. *Imaginary part values of the cross-periodogram.* Applies only to bivariate analyses.

CS. *Cospectral density estimate values.* Applies only to bivariate analyses.

QS. *Quadrature spectrum estimate values.* Applies only to bivariate analyses.

PH. *Phase spectrum estimate values.* Applies only to bivariate analyses.

A. *Cross-amplitude values.* Applies only to bivariate analyses.

G. *Gain values.* Applies only to bivariate analyses.

K. *Squared coherency values.* Applies only to bivariate analyses.

Example

```
SPECTRA VARIABLES=STRIKES RUNS
   /SAVE= FREQ (FREQ) P (PGRAM) S (SPEC).
```

- This example creates five variables: *FREQ*, *PGRAM_1*, *PGRAM_2*, *SPEC_1*, and *SPEC_2*.

APPLY Subcommand

APPLY allows you to use a previously defined SPECTRA model without having to repeat the specifications.

- The only specification on APPLY is the name of a previous model in quotation marks. If a model name is not specified, the model that was specified on the previous SPECTRA command is used. Model names are either the default *MOD_n* names that are assigned by Forecasting or the names that are assigned on the MODEL NAME command.
- To change one or more model specifications, specify the subcommands of only those portions you want to change after the APPLY subcommand.
- If no series are specified on the command, the series that were originally specified with the model that is being reapplied are used.
- To change the series that are used with the model, enter new series names before or after the APPLY subcommand. If a variable name is specified before APPLY, the slash before the subcommand is required.
- The SAVE specifications from the previous model are *not* reused by APPLY. They must be specified each time that variables are to be saved.

Examples

```
SPECTRA VARIABLES = VAR01
  /WINDOW=DANIELL (3)
  /CENTER
  /PLOT P S BY FREQ.
SPECTRA APPLY
  /PLOT P S.
```

- The first command plots both the periodogram and the spectral density estimate for *VAR01*. The plots are shown by frequency only.
- Because the PLOT subcommand is respecified, the second command produces plots by both frequency and period. All other specifications remain the same as in the first command.

References

Bloomfield, P. 1976. *Fourier analysis of time series*. New York: John Wiley and Sons.

Fuller, W. A. 1976. *Introduction to statistical time series*. New York: John Wiley and Sons.

Gottman, J. M. 1981. *Time-series analysis: A comprehensive introduction for social scientists*. Cambridge: Cambridge University Press.

Priestley, M. B. 1981. *Spectral analysis and time series, volumes 1 and 2*. London: Academic Press.

SPLIT FILE

```
SPLIT FILE [{LAYERED }] {BY varlist}
           {SEPARATE}
           {OFF     }
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
SPLIT FILE BY SEX.
```

Overview

SPLIT FILE splits the active dataset into subgroups that can be analyzed separately. These subgroups are sets of adjacent cases in the file that have the same values for the specified split variables. Each value of each split variable is considered a break group, and cases within a break group must be grouped together in the active dataset. If they are not grouped together, the SORT CASES command must be used before SPLIT FILE to sort cases in the proper order.

Basic Specification

The basic specification is keyword BY followed by the variable or variables that define the split-file groups.

- By default, the split-file groups are compared within the same table(s).
- You can turn off split-file processing by using keyword OFF.

Syntax Rules

- SPLIT FILE can specify both numeric and string split variables, including variables that are created by temporary transformations. SPLIT FILE cannot specify scratch or system variables.
- SPLIT FILE is in effect for all procedures in a session unless you limit it with a TEMPORARY command, turn it off, or override it with a new SPLIT FILE or SORT CASES command.

Operations

- SPLIT FILE takes effect as soon as it is encountered in the command sequence. Therefore, pay special attention to the position of SPLIT FILE among commands. See the topic “Command Order” on page 36 for more information.
- The file is processed sequentially. A change or break in values on any one of the split variables signals the end of one break group and the beginning of the next break group.
- AGGREGATE ignores the SPLIT FILE command. To split files by using AGGREGATE, name the variables that are used to split the file as break variables ahead of any other break variables on AGGREGATE. AGGREGATE still produces one file, but the aggregated cases are in the same order as the split-file groups.
- If SPLIT FILE is in effect when a procedure writes matrix materials, the program writes one set of matrix materials for every split group. If a procedure reads a file that contains multiple sets of matrix materials, the procedure automatically detects the presence of multiple sets.
- If SPLIT FILE names any variable that was defined by the NUMERIC command, the program prints page headings that indicate the split-file grouping.

Limitations

- SPLIT FILE can specify or imply up to eight variables.

- Each eight bytes of a string variable counts as a variable toward the limit of eight variables. So a string variable with a defined width of greater than 64 bytes cannot be used as a split file variable.

LAYERED and SEPARATE Subcommands

LAYERED and SEPARATE specify how split-file groups are displayed in the output.

- Only one of these subcommands can be specified. If neither subcommand is specified with the BY variable list, LAYERED is the default.
- LAYERED and SEPARATE do not apply to the text output.

LAYERED. *Display split-file groups in the same table in the outermost column*.

SEPARATE. *Display split-file groups as separate tables*.

Examples

Sorting and Splitting a File

```
SORT CASES BY SEX.
SPLIT FILE BY SEX.
FREQUENCIES VARS=INCOME /STATISTICS=MEDIAN.
```

- SORT CASES arranges cases in the file according to the values of variable *SEX*.
- SPLIT FILE splits the file according to the values of variable *SEX*, and FREQUENCIES generates separate median income tables for men and women.
- By default, the two groups (men and women) are compared in the same Frequency and Statistics tables.

Applying a Temporary Split File

```
SORT CASES BY SEX.
TEMPORARY.
SPLIT FILE SEPARATE BY SEX.
FREQUENCIES VARS=INCOME /STATISTICS=MEDIAN.
FREQUENCIES VARS=INCOME /STATISTICS=MEDIAN.
```

- Because of the TEMPORARY command, SPLIT FILE applies to the first procedure only. Thus, the first FREQUENCIES procedure generates separate tables for men and women. The second FREQUENCIES procedure generates tables that include both sexes.

Turning Off a Split File

```
SORT CASES BY SEX.
SPLIT FILE SEPARATE BY SEX.
FREQUENCIES VARS=INCOME /STATISTICS=MEDIAN.
SPLIT FILE OFF.
FREQUENCIES VARS=INCOME /STATISTICS=MEDIAN.
```

- SPLIT FILE does not apply to the second FREQUENCIES procedure because it is turned off after the first FREQUENCIES procedure. This example produces the same results as the example above.

Overriding a Previous Split File

```
SORT CASES BY SEX RACE.
SPLIT FILE BY SEX.
FREQUENCIES VARS=INCOME /STATISTICS=MEDIAN.
SPLIT FILE BY SEX RACE.
FREQUENCIES VARS=INCOME /STATISTICS=MEDIAN.
```

- The first SPLIT FILE command applies to the first FREQUENCIES procedure. The second SPLIT FILE command overrides the first command and splits the file by sex and race. This split is in effect for the second FREQUENCIES procedure.

STAR JOIN

```
STAR JOIN
  /SELECT {* | alias0.field [AS newname], alias1.field [AS newname],...}
  /FROM {'data file' [PASSWORD='password'] | dataset | *} AS alias0
  /JOIN {'data file' [PASSWORD='password'] | dataset | *} AS alias1
    ON alias0.key1=alias1.key1
    [AND alias0.key2=alias1.key2] [AND...]
    [IN=field]
  [/JOIN...] [/JOIN...]
 /OUTFILE FILE={'data file' | dataset | *}
  [/PASSPROTECT]
    PASSWORD={'password'|NONE**}
    [ENCRYPTEDPW={NO**|YES}]

**Default if subcommand or keyword omitted
```

- All subcommands except PASSPROTECT are required.
- All subcommands must be in the order indicated in the syntax chart.
- Multiple JOIN subcommands are allowed, up to a maximum of the operating system limit for file handles or until you run out of memory.
- All command specifications must be spelled out in full; three letter abbreviation is not allowed.
- The syntax for SELECT, FROM, and JOIN is based on standard SQL syntax, but this is not SQL. You cannot include any SQL not explicitly specified in the syntax chart.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 21.0

- Command introduced.

Release 22.0

- PASSWORD keyword introduced on the FROM and JOIN subcommands.

Example

```
STAR JOIN
  /SELECT t1.Name1, t1.Name2, t1.Field1, t1.Field2, t1.Field3,
   t2.Value1, t3.Value2
  /FROM '/myfiles/fact.sav' AS t1
  /JOIN '/myfiles/field1_values.sav' AS t2
   ON t1.Field1 = t2.Field1
  /JOIN 'myfiles/field2_values .sav' AS t3
   ON t1.Field2 = t3.Field2
  /OUTFILE FILE='/myfiles/merged.sav'.
```

Overview

STAR JOIN performs the equivalent of an SQL left outer join. The FROM subcommand specifies the case data file (fact table). The JOIN subcommands specify the table lookup files (dimension tables).

- All cases (records) from the case data file are included in the merged file.
- Cases from the table lookup files are included only if they have key values that match key values in the case data file.
- The merged file contains all fields specified on the SELECT subcommand and all fields specified as keys on the JOIN subcommand(s).

This is similar to MATCH FILES with one or more TABLE lookup subcommands, but STAR JOIN provides several features not available in MATCH FILES:

- The files do not have to be presorted in order of the key values.
- Different keys can be specified for each table lookup file.

- The defined width of string keys does not have to be the same in both the case data file and the table lookup file.

Note: Unlike MATCH FILES, STAR JOIN reads the active dataset and causes execution of any pending transformations.

Syntax

- All subcommands except PASSPROTECT are required.
- All subcommands must be in the order indicated in the syntax chart: SELECT, FROM, JOIN, OUTFILE, PASSPROTECT.
- Key values in files specified on the JOIN subcommand must be unique. Duplicate key values within the same file will result in an error. If there are multiple keys, each combination of keys must be unique.

Operations

The merged dataset contains complete dictionary information from the input files, including variable names, labels, print and write formats, and missing-value indicators.

- For key variables or other variables with the same name in more than one file, variable dictionary information is taken from the first file containing value labels, missing values, or a variable label for the common variable. If the case data file has no such information, the first lookup table specified is checked, and so on. This includes custom variable attributes.
- Custom file attributes (DATAFILE ATTRIBUTES command) are taken from the first file that contains custom file attributes. If the case data file has none, the first table lookup file is checked, and so on.
- The merged file includes all documents from all files (documents are created with the DOCUMENT and ADD DOCUMENTS commands).
- No file labels (FILE LABEL command) from any of the files are preserved.

SELECT subcommand

The SELECT subcommand specifies the fields to be included from all data sources specified on the FROM and JOIN subcommands, with the exception of the key fields specified on the JOIN subcommand.

- The SELECT subcommand is required and must be the first subcommand.
- All field specifications take the general form of *alias.fieldname*, where *alias* is a stand-in name for each data source specified on the FROM and JOIN subcommands.
- Field specifications must be separated by commas.
- An asterisk (*) can be used to specify all fields in all the data sources.

  Note: In SQL the asterisk convention will cause duplicate entries for key fields in the merged result. For STAR JOIN, asterisk means “everything that isn’t a key”.
- Only one SELECT subcommand is allowed.
- Any fields specified as keys on the JOIN subcommand(s) that are also specified on the SELECT subcommand are ignored on the SELECT subcommand.
- Duplicate field names in different data sources are made unique by including the table alias and a period as a prefix (for example, *t1.field1*, *t2.field1*).
- Duplicate field names specified for the same data source are ignored.

FROM subcommand

The FROM subcommand specifies the name of the case data file (fact table). It is followed by the keyword AS, followed by the alias used to identify the file source when specifying fields.

- The FROM subcommand is required and must immediately follow the SELECT subcommand.

- The data file name can be an external data file in IBM SPSS Statistics format or a dataset in the current session.
- External data file names must be quoted.
- An asterisk can be used to specify the currently active dataset.
- Alias names must follow standard variable naming rules. See the topic "Variable Names" on page 42 for more information.
- Only one FROM subcommand is allowed.

PASSWORD Keyword

The PASSWORD keyword specifies the password required to open an encrypted IBM SPSS Statistics data file. The specified value must be enclosed in quotation marks and can be provided as encrypted or as plain text. Encrypted passwords are created when pasting command syntax from the Save Data As dialog. The PASSWORD keyword is ignored if the file is not encrypted.

JOIN subcommand

The JOIN subcommand specifies the table lookup file (dimension table). It is followed by the keyword AS, followed by the alias used to identify the file source when specifying fields, followed by the keyword ON, which specifies the key(s) to use to match the file with the case data file.

- At least one JOIN subcommand is required, and all JOIN subcommands must follow the FROM subcommand.
- The maximum number of JOIN subcommands is determined by the operating system limit for file handles or available memory.
- The data file name can be an external data file in IBM SPSS Statistics format or a dataset in the current session.
- External data file names must be quoted.
- An asterisk can be used to specify the currently active dataset.
- Alias names must follow standard variable naming rules. See the topic "Variable Names" on page 42 for more information.
- To specify the same lookup table on multiple JOIN subcommands, use different alias names for each instance.
- Key values within each file must be unique. Duplicate key values within the same file will result in an error. If there are multiple keys, each combination of keys must be unique.

ON Keyword

The ON keyword specifies the keys(s) used to match cases in the table lookup file with cases in the case data file. The general form is:

```
ON alias0.field=alias1.field
```

- Each key or key combination that identifies each case must be unique in the table lookup file.
- The basic data type (string or numeric) for each key comparison must be the same in both files.
- To specify multiple keys that identify each unique case in the table lookup file, use the keyword AND, as in:

  ```
  ON alias0.field1=alias1.field1 AND alias0.field2=alias1.field2
  ```
- The field names do not have to be the same in the two files. If they are different, both fields are included in the merged data file.
- The only comparison that is allowed is equality between field values. Other comparisons (for example, >=, <=, <>) are not allowed, and arithmetic expressions (for example, t1.field1 = t2.field1 + t2.field2) are not allowed.

- Comparisons can only be evaluated between the current table lookup file and the case data file. For example, if the case data file is *t1*, and you have two lookup files, *t2* and *t3*, ON `t2.field1=t3.field1` is not valid.

IN Keyword

The optional `IN` keyword creates a new variable (field) in the merged file that indicates if the case (based on the key values) is present in that table lookup file. A value 1 indicates that the case is present; a value of 0 indicates that it is not. You can use this to identify cases in the case data file that have no matching cases in the table lookup file.

The keyword is followed by an equals sign (=) and the name of the new variable. The name must conform to variable naming rules.

PASSWORD Keyword

The `PASSWORD` keyword specifies the password required to open an encrypted IBM SPSS Statistics data file. The specified value must be enclosed in quotation marks and can be provided as encrypted or as plain text. Encrypted passwords are created when pasting command syntax from the Save Data As dialog. The `PASSWORD` keyword is ignored if the file is not encrypted.

OUTFILE subcommand

The `OUTFILE` subcommand specifies the name of the new, merged dataset.

- The subcommand has one required keyword, `FILE`, which must be followed by an equals sign (=) and the name of an external data file, a previously declared dataset, or an asterisk.
- The data file name can be an external data file in IBM SPSS Statistics format or a dataset in the current session.
- External data file names must be quoted.
- An asterisk can be used to specify the currently active dataset. The currently active dataset will be replaced by the merged dataset.
- If you specify the name of an existing dataset in the current session, the merged dataset will replace the existing dataset.
- To specify a new dataset name in the current session, you must first declare the new dataset name with the `DATASET DECLARE` command.

PASSPROTECT subcommand

The optional `PASSPROTECT` subcommand saves the file in encrypted format.

- `PASSPROTECT` is ignored unless `OUTFILE` specifies an external file.
- The `PASSWORD` keyword is required. The password can be up to 10 characters long.
- By default, passwords are not encrypted. Use the optional `ENCRYPTEDPW=YES` to encrypt the password.

Example: STAR JOIN with two lookup table files

In this example, the case data file contains numeric codes for *Field1* and *Field2*, and descriptive values for each of those fields are contained in two separate lookup tables.

Data

Table 102. Case data file: fact.sav

Name1	Name2	Field1	Field2	Field3
Barney	Jones	1	1	10/29/1987
Fred	Smith	1	3	10/28/1986
Barney	Jones	2	2	12/1/2001
Joe	Smith	5	4	12/1/2011
Nancy	Jones	99		

Table 103. Lookup table: field1_values.sav

Field1	Value1
1	One
2	Two
3	Three
99	Missing

Table 104. Lookup table: field2_values.sav

Field2	Value2
1	A
2	B
3	C
5	E

Syntax

```
STAR JOIN
  /SELECT t1.Name1, t1.Name2, t1.Field1, t1.Field2, t1.Field3,
   t2.Value1, t3.Value2
  /FROM '/myfiles/fact.sav' AS t1
  /JOIN '/myfiles/field1_values.sav' AS t2
   ON t1.Field1 = t2.Field1
  /JOIN 'myfiles/field2_values.sav' AS t3
   ON t1.Field2 = t3.Field2
  /OUTFILE FILE='/myfiles/merged_data.sav'.
```

Result

Table 105. Merged files

Name1	Name2	Field1	Field2	Field3	Value1	Value2
Barney	Jones	1	1	10/29/1987	One	A
Fred	Smith	1	3	10/28/1986	One	C
Barney	Jones	2	2	12/1/2001	Two	B
Joe	Smith	5	4	12/1/2011	(blank)	(blank)
Nancy	Jones	99	(sysmis)	(sysmis)	Missing	

- All records from the case data file are included; so the fourth record (Joe Smith) is included even though the *Field1* value of 5 and the *Field2* value of 4 are not present in either lookup table.

- Only records from the lookup tables that have matching keys in the case data file are included; so the *Field1* record with a value of 3 in *field1_values.sav* and the *Field2* record with a value of 5 in *field2_values.sav* are not included.
- In this example, since the SELECT statement includes all fields in all three tables other than those specified as key fields with ON, an asterisk (*) instead of the explicit list of fields would yield the same result.

Example: STAR JOIN with two key-value pairs in the same file

This is similar to the previous example, except both key-value pairs are contained in the same lookup table. This syntax is less complex than standard SQL.

Data

Table 106. Case data file: fact.sav

Name1	Name2	Field1	Field2	Field3
Barney	Jones	1	1	10/29/1987
Fred	Smith	1	3	10/28/1986
Barney	Jones	2	2	12/1/2001
Joe	Smith	5	4	12/1/2011
Nancy	Jones	99		

Table 107. Lookup table: mixed_values.sav

Field1	Value1	Field2	Value2
1	One	1	A
2	Two	2	B
3	Three	3	C
99	Missing	5	E

Syntax

```
STAR JOIN
  /SELECT t1.Name1, t1.Name2, t1.Field1, t1.Field2, t1.Field3,
   t2.Value1, t3.Value2
  /FROM '/myfile/fact.sav' AS t1
  /JOIN '/myfiles/mixed_values.sav' AS t2
   ON t1.Field1 = t2.Field1
  /JOIN 'myfiles/mixed_values.sav' AS t3
   ON t1.Field2 = t3.Field2
   /OUTFILE FILE='/myfiles/merged_file.sav'.
```

Result

- The result is exactly the same as the previous example.
- The syntax is very similar to the previous example. The only difference is that both of the JOIN statements refer to the same lookup table, but each one assigns a different alias to the same lookup table.

Example: STAR JOIN with two keys required to match cases

This is similar to the previous example, except both key/value pairs are contained in the same lookup table. This syntax is less complex than standard SQL.

Data

Table 108. Case data file: fact.sav

Name1	Name2	Field1	Field2	Field3
Barney	Jones	1	1	10/29/1987
Fred	Smith	1	3	10/28/1986
Barney	Jones	2	2	12/1/2001
Joe	Smith	5	4	12/1/2011
Nancy	Jones	99		

Table 109. Lookup table: two_keys.sav

Name1	Name2	City
Barney	Jones	Anytown
Fred	Smith	Centerville
Nancy	Jones	Metropolis
Joe	Jones	Anytown

Syntax

```
STAR JOIN
  /SELECT t1.Name1, t1.Name2, t1.Field1, t1.Field2, t1.Field3, t2.City
  /FROM '/myfiles/fact.sav' AS t1
  /JOIN '/myfiles/two_keys.sav' AS t2
   ON t1.Name1 = t2.Name1 AND t1.Name2 = t2.Name2
  /OUTFILE FILE=*.
```

Result

Table 110. Merged file

Name1	Name2	Field1	Field2	Field3	City
Fred	Smith	1	3	10/29/1987	Centerville
Barney	Jones	1	1	10/28/1986	Anytown
Joe	Smith	5	4	12/1/2001	(blank)
Barney	Jones	2	2	12/1/2011	Anytown
Nancy	Jones	99	(sysmis)	(sysmis)	Metropolis

- All records from the case data file are included; so “Joe Smith” is included even though there is no matching record in the lookup table.
- Only records from the lookup table that match both keys in the case data file are included; so “Joe Jones” is not included.

STRING

```
STRING varlist (An) [/varlist...]
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
STRING STATE1 (A2).
```

Overview

STRING declares new string variables that can be used as target variables in data transformations.

Basic Specification

The basic specification is the name of the new variables and, in parentheses, the variable format.

Syntax Rules

- If keyword TO is used to create multiple string variables, the specified format applies to each variable that is named and implied by TO.
- To declare variables with different formats, separate each format group with a slash.
- STRING can be used within an input program to determine the order of string variables in the dictionary of the active dataset. When used for this purpose, STRING must precede DATA LIST in the input program. See the Examples for the NUMERIC command .
- STRING cannot be used to redefine an existing variable.
- String variables cannot have zero length; A0 is an illegal format.
- All implementations of the program allow the A format. Other string formats may be available on some systems. In addition, the definition of a long string depends on your operating system. Use keyword LOCAL on the INFO command to obtain documentation for your operating system.

Operations

- STRING takes effect as soon as it is encountered in the command sequence. Therefore, pay special attention to the position of STRING among commands. See the topic “Command Order” on page 36 for more information.
- New string variables are initialized as blanks.
- Variables that are declared on STRING are added to the active dataset in the order in which they are specified. This order is not changed by the order in which the variables are used in the transformation language.
- To change the defined width of a string variable use the ALTER TYPE command. See the topic “ALTER TYPE” on page 149 for more information.

Examples

```
STRING STATE1 (A2).
RECODE STATE ('IO'='IA') (ELSE=COPY) INTO STATE1.
```

- STRING declares variable *STATE1* with an A2 format.

- RECODE specifies *STATE* as the source variable and specifies *STATE1* as the target variable. The original value IO is recoded to IA. Keywords ELSE and COPY copy all other state codes unchanged. Thus, *STATE* and *STATE1* are identical except for cases with the original value IO.

```
STRING V1 TO V6 (A8) / V7 V10 (A16).
```

- STRING declares variables *V1*, *V2*, *V3*, *V4*, *V5*, and *V6*, each with an A8 format, and variables *V7* and *V10*, each with an A16 format.

SUBTITLE

```
SUBTITLE  ']text[']
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
SUBTITLE "Children's Training Shoes Only".
```

Overview

SUBTITLE inserts a subtitle on the second line from the top of each page of the output. The default subtitle contains the installation name and information about the hardware and operating system.

Basic Specification

The only specification is the subtitle itself.

Syntax Rules

- The subtitle can include any characters. To specify a blank subtitle, enclose a blank within quotes.
- The subtitle can be up to 60 characters. Subtitles that are longer than 60 characters are truncated.
- The quotes enclosing the subtitle are optional; using them allows you to include single or double quotes in the subtitle.
- More than one SUBTITLE command is allowed in a single session.
- A subtitle cannot be placed between a procedure command and BEGIN DATA-END DATA or within data records when the data are inline.

Operations

- Each SUBTITLE command overrides the previous command and takes effect on the next output page.
- SUBTITLE is independent of TITLE and each command can be changed separately.
- The subtitle will not be displayed if HEADER=NO is specified on SET.

Examples

Using Single and Double Quotes

```
TITLE 'Running Shoe Study from Runner''s World Data'.
SUBTITLE "Children's Training Shoes Only".
```

- The title is enclosed in single quotes, so the single quote (apostrophe) in *Runner’s* must be specified as a two single quotes.
- The subtitle is enclosed in double quotes, so the apostrophe in *Children’s* is simply specified as a single quote.

Suppressing the Default Subtitle

```
TITLE 'Running Shoe Study from Runner''s World Data'.
SUBTITLE ' '.
```

- This subtitle is specified as a blank, which suppresses the default subtitle.

SUMMARIZE

SUMMARIZE is available in Statistics Base Edition.

```
SUMMARIZE [TABLES=]{varlist} [BY varlist] [BY...] [/varlist...]
                   {ALL    }

 [/TITLE ='string']     [/FOOTNOTE= 'string']

 [/CELLS= [COUNT**] [MEAN ] [STDDEV]
                  [MEDIAN] [GMEDIAN] [SEMEAN]  [SUM ]
                  [MIN] [MAX] [RANGE] [VARIANCE]
                  [KURT] [SEKURT] [SKEW] [SESKEW]
                  [FIRST] [LAST]
                  [NPCT] [SPCT] [NPCT(var)] [SPCT(var)]
                  [HARMONIC] [GEOMETRIC]
                  [DEFAULT]
                  [ALL] [NONE]  ]

 [/MISSING=[{EXCLUDE**}][{VARIABLE**  }]
            {INCLUDE  }  {TABLE       }
            {DEPENDENT}

 [/FORMAT=[{NOLIST**      }] [{CASENUM   }] [{TOTAL**}]  [MISSING='string']
          {LIST [LIMIT=n]}   {NOCASENUM }   {NOTOTAL}
          {VALIDLIST     }

 [/STATISTICS=[ANOVA] [{LINEARITY}] [NONE**] ]
                       {ALL      }
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
SUMMARIZE SALES.
```

Overview

SUMMARIZE produces univariate statistics for specified variables. You can break the variables into groups defined by one or more control (independent) variables. Another procedure that displays univariate statistics is FREQUENCIES.

Options

Cell Contents. By default, SUMMARIZE displays means, standard deviations, and cell counts. You can also use the CELLS subcommand to display aggregated statistics, including sums, variances, median, range, kurtosis, skewness, and their standard error.

Statistics. In addition to the statistics that are displayed for each cell of the table, you can use the STATISTICS subcommand to obtain a one-way analysis of variance and test of linearity.

Format. By default, SUMMARIZE produces a Summary Report with a total category for each group that is defined by the control variables. You can use the FORMAT subcommand to request a Listing Report with or without case numbers. You can also remove the total category from each group. You can use the TITLE and FOOTNOTE subcommands to specify a title and a caption for the Summary or Listing Report.

Basic Specification

The basic specification is TABLES with a variable list. Each variable creates a category. The actual keyword TABLES can be omitted.

- The minimum specification is a dependent variable.
- By default, SUMMARIZE displays a Case Processing Summary table, showing the number and percentage of cases included, excluded, and their total. SUMMARIZE also displays a Summary Report, showing means, standard deviations, and number of cases for each category.

Syntax Rules

- Both numeric and string variables can be specified. String variables can be short or long.
- If there is more than one TABLES subcommand, FORMAT=LIST or VALIDLIST results in an error.
- String specifications for TITLE and FOOTNOTE must be enclosed in quotes and cannot exceed 255 bytes. When the specification breaks on multiple lines, enclose each line in quotes, and separate the specifications for each line by at least one blank.
- Each subcommand (except TABLES) can be specified only once. If a subcommand is specified multiple times, a warning results, and the last specification is used.
- Multiple TABLES subcommands are allowed, but multiple specifications use a lot of computer resources and time.
- There is no limit on the number of variables that you can specify on each TABLES subcommand.
- When a variable is specified more than once, only the first occurrence is honored. If the same variables are specified after different BY keywords, an error results.

Limitations

- Only 10 BY keywords can be specified.

Operations

- The data are processed sequentially. It is not necessary to sort the cases before processing. If a BY keyword is used, the output is always sorted.
- A Case Processing Summary table is always generated, showing the number and percentage of the cases included, excluded, and the total.
- For each combination of control variables specified after different BY keywords, SUMMARIZE produces a group in the Summary Report (depending on the specification on the FORMAT subcommand). By default, mean, standard deviation, and number of cases are displayed for each group and for the total.
- An ANOVA table and a Measure of Association table are produced if additional statistics are requested.

Example

Creating a Grouped Summary Report

```
SUMMARIZE
  /TABLES=value  BY street
  /FORMAT=NOLIST TOTAL
  /TITLE='Home Sale Statistics' /FOOTNOTE 'Grouped by Street'
  /MISSING=VARIABLE
  /CELLS=COUNT MEAN MEDIAN MIN MAX .
```

- The procedure produces summaries of *value* by values of *street*.
- FORMAT specifies that no case listing should be produced but that summary statistics should be produced.
- TITLE produces the requested title for the summary report and FOOTNOTE produces the specified table footnote.
- CELLS requests the cell counts, means, medians, minimums, and maximums.
- All other options are set to their default values.

For discussion of output from this syntax and to place the example in context, see .

Displaying Individual Cases

```
SUMMARIZE
  /TABLES=address value marktime
  /FORMAT=VALIDLIST NOCASENUM MISSING='Unavailable' TOTAL
  /TITLE='Case Listing' /FOOTNOTE 'Only homes selling for $335,000 or more'
  /MISSING=VARIABLE
  /CELLS=COUNT MEAN MEDIAN .
```

- The procedure produces summaries of *address*, *value*, and *marktime*.

- FORMAT specifies that the case listing report should contain only valid cases, without the case numbers, and with system-missing values displayed with the label *Unavailable*.
- TITLE produces the requested title for the summary report and FOOTNOTE produces the specified table footnote.
- CELLS requests the cell counts, means, and medians.
- All other options are set to their default values.

For discussion of output from this syntax and to place the example in context, see .

TABLES Subcommand

TABLES specifies the dependent and control variables.

- You can specify multiple TABLES subcommands on a single SUMMARIZE command.
- For FORMAT=LIST or VALIDLIST, only one TABLES subcommand is allowed. Multiple dependent and control variables add more breaks to the Listing Report. Total statistics are displayed at the end for each combination that is defined by different values of the control variables.
- For FORMAT=NOLIST, which is the default, each use of keyword BY adds a dimension to the requested table. Total statistics are displayed with each group.
- The order in which control variables are displayed is the same as the order in which they are specified on TABLES. The values of the first control variable that is defined for the table appear in the leftmost column of the table and change the most slowly in the definition of groups.
- Statistics are displayed for each dependent variable in the same report.
- More than one dependent variable can be specified in a table list, and more than one control variable can be specified in each dimension of a table list.

TITLE and FOOTNOTE Subcommands

TITLE and FOOTNOTE provide a title and a caption for the Summary or Listing Report.

- TITLE and FOOTNOTE are optional and can be placed anywhere.
- The specification on TITLE or FOOTNOTE is a string enclosed in quotes. To specify a multiple-line title or footnote, use "\n" to specify line breaks.
- The string that you specify cannot exceed 255 characters.

CELLS Subcommand

By default, SUMMARIZE displays the means, standard deviations, and cell counts in each cell. Use CELLS to modify cell information.

- If CELLS is specified without keywords, SUMMARIZE displays the default statistics.
- If any keywords are specified on CELLS, only the requested information is displayed.
- MEDIAN and GMEDIAN use a lot of computer resources and time. Requesting these statistics (via these keywords or ALL) may slow down performance.

DEFAULT. *Means, standard deviations, and cell counts.* This setting is the default if CELLS is omitted.

MEAN. *Cell means.*

STDDEV. *Cell standard deviations.*

COUNT. *Cell counts.*

MEDIAN. *Cell median.*

GMEDIAN. *Grouped median.*

SEMEAN. *Standard error of cell mean.*

SUM. *Cell sums.*

MIN. *Cell minimum.*

MAX. *Cell maximum.*

RANGE. *Cell range.*

VARIANCE. *Variances.*

KURT. *Cell kurtosis.*

SEKURT. *Standard error of cell kurtosis.*

SKEW. *Cell skewness.*

SESKEW. *Standard error of cell skewness.*

FIRST. *First value.*

LAST. *Last value.*

SPCT. *Percentage of total sum.*

NPCT. *Percentage of total number of cases.*

SPCT(var). *Percentage of total sum within specified variable.* The specified variable must be one of the control variables.

NPCT(var). *Percentage of total number of cases within specified variable.* The specified variable must be one of the control variables.

HARMONIC. *Harmonic mean.*

GEOMETRIC. *Geometric mean.*

ALL. *All cell information.*

MISSING Subcommand

MISSING controls the treatment of cases with missing values. There are two groups of keywords.

- EXCLUDE, INCLUDE, and DEPENDENT specify the treatment of user-missing values. The default is EXCLUDE.

  EXCLUDE. *All user-missing values are excluded.* This setting is the default.

  INCLUDE. *User-missing values are treated as valid values.*

  DEPENDENT. *User-missing values are considered missing in a dependent variable and valid in a grouping variable (variables that are specified after a BY keyword).*
- VARIABLE and TABLE specify how cases with missing values for dependent variables are excluded. The default is VARIABLE.
- Cases with missing values for any control variables are always excluded.

VARIABLE. *A case is excluded when all values for variables in that table are missing.* This setting is the default.

TABLE. *A case is excluded when any value is missing within a table.*

FORMAT Subcommand

FORMAT specifies whether you want a case listing for your report and whether you want case numbers displayed for the listing. FORMAT also determines whether your reports will display a total category for each group and how the reports will indicate missing values.

NOLIST. *Display a Summary Report without a case listing.* This setting is the default.

LIST. *Display a Listing Report.* By default, LIST shows all cases. It can be followed by the optional keyword LIMIT, an equals sign, and a positive integer to limit the number of cases in the report. For example, /FORMAT LIST LIMIT=10 will limit the report to the first 10 cases.

VALIDLIST. *Display a Listing Report showing only valid cases.*

CASENUM. *Display case numbers as a category in the Listing Reports.* This setting is the default when FORMAT=LIST or VALIDLIST.

NOCASENUM. *Do not display case numbers.*

TOTAL. *Display the summary statistics for the total of each group with the label* Total. This is the default.

NOTOTAL. *Display the total category without a label.*

MISSING='string'. *Display system-missing values as a specified string.*

STATISTICS Subcommand

Use STATISTICS to request a one-way analysis of variance and a test of linearity for each table list.

- Statistics that are requested on STATISTICS are computed in addition to the statistics that are displayed in the Group Statistics table.
- If STATISTICS is specified without keywords, SUMMARIZE computes ANOVA.
- If two or more dimensions are specified, the second and subsequent dimensions are ignored in the analysis-of-variance table.

ANOVA. *Analysis of variance.* ANOVA displays a standard analysis-of-variance table and calculates eta and eta squared (displayed in the Measures of Association table). This setting is the default if STATISTICS is specified without keywords.

LINEARITY. *Test of linearity.* LINEARITY (alias ALL) displays additional statistics to the tables that are created by the ANOVA keyword—the sums of squares, degrees of freedom, and mean square associated with linear and nonlinear components, the *F* ratio, and the significance level for the ANOVA table and Pearson's *r* and r^2 for the Measures of Association table. LINEARITY is ignored if the control variable is a string.

NONE. *No additional statistics.* This setting is the default if STATISTICS is omitted.

Example

```
SUMMARIZE TABLES=INCOME BY SEX BY RACE
  /STATISTICS=ANOVA.
```

- SUMMARIZE produces a Group Statistics table of *INCOME* by *RACE* within *SEX* and computes an analysis of variance only for *INCOME* by *SEX*.

SURVIVAL

SURVIVAL is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
SURVIVAL TABLE=survival varlist
               [BY varlist (min, max)...][BY varlist (min, max)...]

  /INTERVAL=THRU n BY a [THRU m BY b ...]

  /STATUS=status variable({min, max}) FOR {ALL             }
                         {value   }      {survival varlist}
 [/STATUS=...]

 [/PLOTS  ({ALL     })={ALL            } BY {ALL    }     BY {ALL    }]
          {LOGSURV }  {survival varlis}    {varlist}        {varlist}
          {SURVIVAL}
          {HAZARD  }
          {DENSITY }
          {OMS     }

 [/PRINT={TABLE**}]
         {NOTABLE}

 [/COMPARE={ALL**           } BY {ALL**  }   BY {ALL**  }]
           {survival varlist}    {varlist}      {varlist}

 [/CALCULATE=[{EXACT**    }] [PAIRWISE] [COMPARE] ]
             {CONDITIONAL}
             {APPROXIMATE}

 [/MISSING={GROUPWISE**}  [INCLUDE] ]
           {LISTWISE   }

 [/WRITE=[{NONE**}] ]
          {TABLES}
          {BOTH  }
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
SURVIVAL TABLE=MOSFREE BY TREATMNT(1,3)
  /STATUS = PRISON (1) FOR MOSFREE
  /INTERVAL=THRU 24 BY 3.
```

Overview

SURVIVAL produces actuarial life tables, plots, and related statistics for examining the length of time to the occurrence of an event, often known as **survival time**. Cases can be classified into groups for separate analyses and comparisons. Time intervals can be calculated with the IBM SPSS Statistics date- and time-conversion functions—for example, CTIME.DAYS or YRMODA. For a closely related alternative nonparametric analysis of survival times using the product-limit Kaplan-Meier estimator, see the KM command. For an analysis of survival times with covariates, including time-dependent covariates, see the COXREG command.

Options

Life Tables. You can list the variables to be used in the analysis, including any control variables on the TABLE subcommand. You can also suppress the life tables in the output with the PRINT subcommand.

Intervals. SURVIVAL reports the percentage alive at various times after the initial event. You can select the time points for reporting with the INTERVAL subcommand.

Plots. You can plot the survival functions for all cases or separately for various subgroups with the PLOTS subcommand.

Comparisons. When control variables are listed on the TABLE subcommand, you can compare groups based on the Wilcoxon (Gehan) statistic using the COMPARE subcommand. You can request pairwise or approximate comparisons with the CALCULATE subcommand.

Writing a File. You can write the life tables, including the labeling information, to a file with the WRITE subcommand.

Basic Specification

- The basic specification requires three subcommands: TABLE, INTERVAL, and STATUS. TABLE identifies at least one survival variable from the active dataset, INTERVAL divides the time period into intervals, and STATUS names a variable that indicates whether the event occurred.
- The basic specification prints one or more life tables, depending on the number of survival and control variables specified.

Subcommand Order

- TABLE must be first.
- Remaining subcommands can be named in any order.

Syntax Rules

- Only one TABLE subcommand can be specified, but multiple survival variables can be named. A survival variable cannot be specified as a control variable on any subcommands.
- Only one INTERVAL subcommand can be in effect on a SURVIVAL command. The interval specifications apply to all of the survival variables listed on TABLE. If multiple INTERVAL subcommands are used, the last specification supersedes all previous ones.
- Only one status variable can be listed on each STATUS subcommand. To specify multiple status variables, use multiple STATUS subcommands.
- You can specify multiple control variables on one BY keyword. Use a second BY keyword to specify second-order control variables to interact with the first-order control variables.
- All variables, including survival variables, control variables, and status variables, must be numeric. SURVIVAL does not process string variables.

Operations

- SURVIVAL computes time intervals according to specified interval widths, calculates the survival functions for each interval, and builds one life table for each group of survival variables. The life table is displayed unless explicitly suppressed.
- When the PLOTS subcommand is specified, SURVIVAL plots the survival functions for all cases or separately for various groups.
- When the COMPARE subcommand is specified, SURVIVAL compares survival-time distributions of different groups based on the Wilcoxon (Gehan) statistic.

Limitations

- Maximum 20 survival variables.
- Maximum 100 control variables total on the first- and second-order control-variable lists combined.
- Maximum 20 THRU and BY specifications on INTERVAL.
- Maximum 35 values can appear on a plot.

Examples

```
SURVIVAL TABLE=MOSFREE BY TREATMNT(1,3)
  /STATUS = PRISON (1) FOR MOSFREE
  /INTERVAL = THRU 24 BY 3.
```

- The survival analysis is used to examine the length of time between release from prison and return to prison for prisoners in three treatment programs. The variable *MOSFREE* is the length of time in months a prisoner stayed out of prison. The variable *TREATMNT* indicates the treatment group for each case.
- A value of 1 on the variable *PRISON* indicates a terminal outcome—that is, cases coded as 1 have returned to prison. Cases with other non-negative values for *PRISON* have not returned. Because we don't know their final outcome, such cases are called censored.

- Life tables are produced for each of the three subgroups. INTERVAL specifies that the survival experience be described every three months for the first two years.

TABLE Subcommand

TABLE identifies the survival and control variables to be included in the analysis.

- The minimum specification is one or more survival variables.
- To specify one or more first-order control (or factor) variables, use the keyword BY followed by the control variable(s). First-order control variables are processed in sequence. For example, `BY A(1,3) B(1,2)` results in five groups (A=1, A=2, A=3, B=1, and B=2).
- You can specify one or more second-order control variables following a second BY keyword. Separate life tables are generated for each combination of values of the first-order and second-order controls. For example, `BY A(1,3) BY B(1,2)` results in six groups (A=1 B=1, A=1 B=2, A=2 B=1, A=2 B=2, A=3 B=1, and A=3, B=2).
- Each control variable must be followed by a value range in parentheses. These values must be integers separated by a comma or a blank. Non-integer values in the data are truncated, and the case is assigned to a subgroup based on the integer portion of its value on the variable. To specify only one value for a control variable, use the same value for the minimum and maximum.
- To generate life tables for all cases combined, as well as for control variables, use COMPUTE to create a variable that has the same value for all cases. With this variable as a control, tables for the entire set of cases, as well as for the control variables, will be produced.

Example

```
SURVIVAL TABLE = MOSFREE BY TREATMNT(1,3) BY RACE(1,2)
  /STATUS = PRISON(1)
  /INTERVAL = THRU 24 BY 3.
```

- *MOSFREE* is the survival variable, and *TREATMNT* is the first-order control variable. The second BY defines *RACE* as a second-order control group having a value of 1 or 2.
- Six life tables with the median survival time are produced, one for each pair of values for the two control variables.

INTERVAL Subcommand

INTERVAL determines the period of time to be examined and how the time will be grouped for the analysis. The interval specifications apply to all of the survival variables listed on TABLE.

- SURVIVAL always uses 0 as the starting point for the first interval. Do not specify the 0. The INTERVAL specification *must* begin with the keyword THRU.
- Specify the terminal value of the time period after the keyword THRU. The final interval includes any observations that exceed the specified terminal value.
- The grouping increment, which follows the keyword BY, must be in the same units as the survival variable.
- The period to be examined can be divided into intervals of varying lengths by repeating the THRU and BY keywords. The period must be divided in ascending order. If the time period is not a multiple of the increment, the endpoint of the period is adjusted upward to the next even multiple of the grouping increment.
- When the period is divided into intervals of varying lengths by repeating the THRU and BY specifications, the adjustment of one period to produce even intervals changes the starting point of subsequent periods. If the upward adjustment of one period completely overlaps the next period, no adjustment is made and the procedure terminates with an error.

Example

```
SURVIVAL TABLE = MOSFREE BY TREATMNT(1,3)
  /STATUS = PRISON(1) FOR MOSFREE
  /INTERVAL = THRU 12 BY 1 THRU 24 BY 3.
```

- INTERVAL produces life tables computed from 0 to 12 months at one-month intervals and from 13 to 24 months at three-month intervals.

Example

```
SURVIVAL  ONSSURV BY TREATMNT (1,3)
  /STATUS = OUTCOME (3,4) FOR ONSSURV
  /INTERVAL = THRU 50 BY 6.
```

- On the INTERVAL subcommand, the value following BY (6) does not divide evenly into the period to which it applies (50). Thus, the endpoint of the period is adjusted upward to the next even multiple of the BY value, resulting in a period of 54 with 9 intervals of 6 units each.

Example

```
SURVIVAL  ONSSURV BY TREATMNT (1,3)
  /STATUS = OUTCOME (3,4) FOR ONSSURV
  /INTERVAL = THRU 50 BY 6 THRU 100 BY 10 THRU 200 BY 20.
```

- Multiple THRU and BY specifications are used on the INTERVAL subcommand to divide the period of time under examination into intervals of different lengths.
- The first THRU and BY specifications are adjusted to produce even intervals as in the previous example. As a result, the following THRU and BY specifications are automatically readjusted to generate 5 intervals of 10 units (through 104), followed by 5 intervals of 20 units (through 204).

STATUS Subcommand

To determine whether the terminal event has occurred for a particular observation, SURVIVAL checks the value of a status variable. STATUS lists the status variable associated with each survival variable and the codes that indicate that a terminal event occurred.

- Specify a status variable followed by a value range enclosed in parentheses. The value range identifies the codes that indicate that the terminal event has taken place. All cases with non-negative times that do not have a code in the value range are classified as **censored cases**, which are cases for which the terminal event has not yet occurred.
- If the status variable does not apply to all the survival variables, specify FOR and the name of the survival variable(s) to which the status variable applies.
- Each survival variable on TABLE must have an associated status variable identified by a STATUS subcommand.
- Only one status variable can be listed on each STATUS subcommand. To specify multiple status variables, use multiple STATUS subcommands.
- If FOR is omitted on the STATUS specification, the status-variable specification applies to all of the survival variables not named on another STATUS subcommand.
- If more than one STATUS subcommand omits the keyword FOR, the final STATUS subcommand without FOR applies to all survival variables not specified by FOR on other STATUS subcommands. No warning is printed.

Example

```
SURVIVAL  ONSSURV BY TREATMNT (1,3)
  /INTERVAL = THRU 50 BY 5, THRU 100 BY 10
  /STATUS = OUTCOME (3,4) FOR ONSSURV.
```

- STATUS specifies that a code of 3 or 4 on *OUTCOME* means that the terminal event for the survival variable *ONSSURV* occurred.

Example

```
SURVIVAL TABLE = NOARREST MOSFREE BY TREATMNT(1,3)
  /STATUS = ARREST (1) FOR NOARREST
  /STATUS = PRISON (1)
  /INTERVAL=THRU 24 BY 3.
```

- STATUS defines the terminal event for *NOARREST* as a value of 1 for *ARREST*. Any other value for *ARREST* is considered censored.
- The second STATUS subcommand defines the value of 1 for *PRISON* as the terminal event. The keyword FOR is omitted. Thus, the status-variable specification applies to *MOSFREE*, which is the only survival variable not named on another STATUS subcommand.

PLOTS Subcommand

PLOTS produces plots of the cumulative survival distribution, the hazard function, and the probability density function. The PLOTS subcommand can plot only the survival functions generated by the TABLE subcommand; PLOTS cannot eliminate control variables.

- When specified by itself, the PLOTS subcommand produces all available plots for each survival variable. Points on each plot are identified by values of the first-order control variables. If second-order controls are used, a separate plot is generated for every value of the second-order control variables.
- To request specific plots, specify, in parentheses following PLOTS, any combination of the keywords defined below.
- Optionally, generate plots for only a subset of the requested life tables. Use the same syntax as used on the TABLE subcommand for specifying survival and control variables, omitting the value ranges. Each survival variable named on PLOTS must have as many control levels as were specified for that variable on TABLE. However, only one control variable needs to be present for each level. If a required control level is missing on the PLOTS specification, the default BY ALL is used for that level. The keyword ALL can be used to refer to an entire set of survival or control variables.
- To determine the number of plots that will be produced, multiply the number of functions plotted by the number of survival variables times the number of first-order controls times the number of distinct values represented in all of the second-order controls.

ALL. *Plot all available functions.* ALL is the default if PLOTS is used without specifications.

LOGSURV. *Plot the cumulative survival distribution on a logarithmic scale.*

SURVIVAL. *Plot the cumulative survival distribution on a linear scale.*

HAZARD. *Plot the hazard function.*

DENSITY. *Plot the density function.*

OMS. *Plot the one-minus-survival function.*

Example

```
SURVIVAL TABLE = NOARREST MOSFREE BY TREATMNT(1,3)
  /STATUS = ARREST (1) FOR NOARREST
  /STATUS = PRISON (1) FOR MOSFREE
  /INTERVAL = THRU 24 BY 3
  /PLOTS (SURVIVAL,HAZARD) = MOSFREE.
```

- Separate life tables are produced for each of the survival variables (*NOARREST* and *MOSFREE*) for each of the three values of the control variable *TREATMNT*.
- PLOTS produces plots of the cumulative survival distribution and the hazard rate for *MOSFREE* for the three values of *TREATMNT* (even though *TREATMNT* is not included on the PLOTS specification).
- Because plots are requested only for the survival variable *MOSFREE*, no plots are generated for the variable *NOARREST*.

PRINT Subcommand

By default, SURVIVAL prints life tables. PRINT can be used to suppress the life tables.

TABLE. *Print the life tables.* This is the default.

NOTABLE. *Suppress the life tables.* Only plots and comparisons are printed. The WRITE subcommand, which is used to write the life tables to a file, can be used when NOTABLE is in effect.

Example

```
SURVIVAL TABLE = MOSFREE BY TREATMNT(1,3)
  /STATUS = PRISON (1) FOR MOSFREE
  /INTERVAL = THRU 24 BY 3
  /PLOTS (ALL)
  /PRINT = NOTABLE.
```

- PRINT NOTABLE suppresses the printing of life tables.

COMPARE Subcommand

COMPARE compares the survival experience of subgroups defined by the control variables. At least one first-order control variable is required for calculating comparisons.

- When specified by itself, the COMPARE subcommand produces comparisons using the TABLE variable list.
- Alternatively, specify the survival and control variables for the comparisons. Use the same syntax as used on the TABLE subcommand for specifying survival and control variables, omitting the value ranges. Only variables that appear on the TABLE subcommand can be listed on COMPARE, and their role as survival, first-order, and second-order control variables cannot be altered. The keyword TO can be used to refer to a group of variables, and the keyword ALL can be used to refer to an entire set of survival or control variables.
- By default, COMPARE calculates exact comparisons between subgroups. Use the CALCULATE subcommand to obtain pairwise comparisons or approximate comparisons.

Example

```
SURVIVAL TABLE = MOSFREE BY TREATMNT(1,3)
  /STATUS = PRISON (1) FOR MOSFREE
  /INTERVAL = THRU 24 BY 3
  /COMPARE.
```

- COMPARE computes the Wilcoxon (Gehan) statistic, degrees of freedom, and observed significance level for the hypothesis that the three survival curves based on the values of *TREATMNT* are identical.

Example

```
SURVIVAL TABLE=ONSSURV,RECSURV BY TREATMNT(1,3)
  /STATUS = RECURSIT(1,9) FOR RECSURV
  /STATUS = STATUS(3,4) FOR ONSSURV
  /INTERVAL = THRU 50 BY 5 THRU 100 BY 10
  /COMPARE = ONSSURV BY TREATMNT.
```

- COMPARE requests a comparison of *ONSSURV* by *TREATMNT*. No comparison is made of *RECSURV* by *TREATMNT*.

CALCULATE Subcommand

CALCULATE controls the comparisons of survival for subgroups specified on the COMPARE subcommand.

- The minimum specification is the subcommand keyword by itself. EXACT is the default.
- Only one of the keywords EXACT, APPROXIMATE, and CONDITIONAL can be specified. If more than one keyword is used, only one is in effect. The order of precedence is APPROXIMATE, CONDITIONAL, and EXACT.

- The keywords PAIRWISE and COMPARE can be used with any of the EXACT, APPROXIMATE, or CONDITIONAL keywords.
- If CALCULATE is used without the COMPARE subcommand, CALCULATE is ignored. However, if the keyword COMPARE is specified on CALCULATE and the COMPARE subcommand is omitted, an error message is generated.
- Data can be entered into SURVIVAL for each individual case or aggregated for all cases in an interval. The way in which data are entered determines whether an exact or an approximate comparison is most appropriate. See the topic "Using Aggregated Data " on page 1923 for more information. .

EXACT. *Calculate exact comparisons.* This is the default. You can obtain exact comparisons based on the survival experience of each observation with individual data. While this method is the most accurate, it requires that all of the data be in memory simultaneously. Thus, exact comparisons may be impractical for large samples. It is also inappropriate when individual data are not available and data aggregated by interval must be used.

APPROXIMATE. *Calculate approximate comparisons only.* Approximate comparisons are appropriate for aggregated data. The approximate-comparison approach assumes that all events occur at the midpoint of the interval. With exact comparisons, some of these midpoint ties can be resolved. However, if interval widths are not too great, the difference between exact and approximate comparisons should be small.

CONDITIONAL. *Calculate approximate comparisons if memory is insufficient.* Approximate comparisons are produced only if there is insufficient memory available for exact comparisons.

PAIRWISE. *Perform pairwise comparisons.* Comparisons of all pairs of values of the first-order control variable are produced along with the overall comparison.

COMPARE. *Produce comparisons only.* Survival tables specified on the TABLE subcommand are not computed, and requests for plots are ignored. This allows all available workspace to be used for comparisons. The WRITE subcommand cannot be used when this specification is in effect.

Example

```
SURVIVAL TABLE = MOSFREE BY TREATMNT(1,3)
  /STATUS = PRISON (1) FOR MOSFREE
  /INTERVAL = THRU 24 BY 3
  /COMPARE /CALCULATE = PAIRWISE.
```

- PAIRWISE on CALCULATE computes the Wilcoxon (Gehan) statistic, degrees of freedom, and observed significance levels for each pair of values of *TREATMNT*, as well as for an overall comparison of survival across all three *TREATMNT* subgroups: group 1 with group 2, group 1 with group 3, and group 2 with group 3.
- All comparisons are exact comparisons.

Example

```
SURVIVAL TABLE = MOSFREE BY TREATMNT(1,3)
  /STATUS = PRISON (1) FOR MOSFREE
  /INTERVAL = THRU 24 BY 3
  /COMPARE /CALCULATE = APPROXIMATE COMPARE.
```

- APPROXIMATE on CALCULATE computes the Wilcoxon (Gehan) statistic, degrees of freedom, and probability for the overall comparison of survival across all three *TREATMNT* subgroups using the approximate method.
- Because the keyword COMPARE is specified on CALCULATE, survival tables are not computed.

Using Aggregated Data

When aggregated survival information is available, the number of censored and uncensored cases at each time point must be entered. Up to two records can be entered for each interval, one for censored cases and one for uncensored cases. The number of cases included on each record is used as the weight factor. If control variables are used, there will be up to two records (one for censored and one for uncensored cases) for each value of the control variable in each interval. These records must contain the value of the

control variable and the number of cases that belong in the particular category as well as values for survival time and status.

Example

```
DATA LIST  / SURVEVAR 1-2 STATVAR 4 SEX 6 COUNT 8.
VALUE LABELS  STATVAR 1 'DECEASED' 2 'ALIVE'
              /SEX 1 'FEMALE' 2 'MALE'.
BEGIN DATA
 1 1 1 6
 1 1 1 1
 1 2 2 2
 1 1 2 1
 2 2 1 1
 2 1 1 2
 2 2 2 1
 2 1 2 3
   ...
END DATA.
WEIGHT COUNT.
SURVIVAL TABLE = SURVEVAR BY SEX (1,2)
  /INTERVAL = THRU 10 BY 1
  /STATUS = STATVAR (1) FOR SURVEVAR.
```

- This example reads aggregated data and performs a SURVIVAL analysis when a control variable with two values is used.
- The first data record has a code of 1 on the status variable *STATVAR*, indicating that it is an uncensored case, and a code of 1 on *SEX*, the control variable. The number of cases for this interval is 6, the value of the variable *COUNT*. Intervals with weights of 0 do not have to be included.
- *COUNT* is not used in SURVIVAL but is the weight variable. In this example, each interval requires four records to provide all of the data for each *SURVEVAR* interval.

MISSING Subcommand

MISSING controls missing-value treatments. The default is GROUPWISE.

- Negative values on the survival variables are automatically treated as missing data. In addition, cases outside of the value range on a control variable are excluded.
- GROUPWISE and LISTWISE are mutually exclusive. However, each can be used with INCLUDE.

GROUPWISE. *Exclude missing values groupwise.* Cases with missing values on a variable are excluded from any calculation involving that variable. This is the default.

LISTWISE. *Exclude missing values listwise.* Cases missing on any variables named on TABLE are excluded from the analysis.

INCLUDE. *Include user-missing values.* User-missing values are included in the analysis.

WRITE Subcommand

WRITE writes data in the survival tables to a file. This file can be used for further analyses or to produce graphics displays.

- When WRITE is omitted, the default is NONE. No output file is created.
- When WRITE is used, a PROCEDURE OUTPUT command must precede the SURVIVAL command. The OUTFILE subcommand on PROCEDURE OUTPUT specifies the output file.
- When WRITE is specified without a keyword, the default is TABLES.

NONE. *Do not write procedure output to a file.* This is the default when WRITE is omitted.

TABLES. *Write survival-table data records.* All survival-table statistics are written to a file.

BOTH. *Write out survival-table data and label records.* Variable names, variable labels, and value labels are written out along with the survival table statistics.

Format

WRITE writes five types of records. The keyword TABLES writes record types 30, 31, and 40. The keyword BOTH writes record types 10, 20, 30, 31, and 40. The format of each record type is described in the following tables.

Table 111. Record type 10, produced only by keyword BOTH

Columns	Content	Format
1–2	Record type (10)	F2.0
3–7	Table number	F5.0
8-47	Variable label of survival variable	A40
48	Number of BY's (0, 1, or 2)	F1.0
49-52	Number of rows in current survival table	F4.0
53-116	Name of survival variable	A64

- One type-10 record is produced for each life table.
- Column 48 specifies the number of orders of control variables (0, 1, or 2) that have been applied to the life table.
- Columns 49-52 specify the number of rows in the life table. This number is the number of intervals in the analysis that show subjects entering; intervals in which no subjects enter are not noted in the life tables.

Table 112. Record type 20, produced by keyword BOTH

Columns	Content	Format
1–2	Record type (20)	F2.0
3–7	Table number	F5.0
8-47	Variable label of control variable	A40
48-52	Value of control variable	F5.0
53-72	Value label for this value	A20
73-136	Name of control variable	A64

- One type-20 record is produced for each control variable in each life table.
- If only first-order controls have been placed in the survival analysis, one type-20 record will be produced for each table. If second-order controls have also been applied, two type-20 records will be produced per table.

Table 113. Record type 30, produced by both keywords TABLES and BOTH

Columns	Content	Format
1–2	Record type (30)	F2.0
3–7	Table number	F5.0
8–13	Beginning of interval	F6.2
14–21	Number entering interval	F8.2
22–29	Number withdrawn in interval	F8.2
30–37	Number exposed to risk	F8.2
38–45	Number of terminal events	F8.2

- Information on record type 30 continues on record type 31. Each pair of type-30 and type-31 records contains the information from one line of the life table.

Table 114. Record type 31, continuation of record type 30

Columns	Content	Format
1–2	Record type (31)	F2.0
3–7	Table number	F5.0
8–15	Proportion terminating	F8.6
16–23	Proportion surviving	F8.6
24–31	Cumulative proportion surviving	F8.6
32–39	Probability density	F8.6
40–47	Hazard rate	F8.6
48–54	S.E. of cumulative proportion surviving	F7.4
55–61	S.E. of probability density	F7.4
62–68	S.E. of hazard rate	F7.4

- Record type 31 is a continuation of record type 30.
- As many type-30 and type-31 record pairs are output for a table as it has lines (this number is noted in columns 57–60 of the type-10 record for the table).

Table 115. Record type 40, produced by both keywords TABLES and BOTH

Columns	Content	Format
1–2	Record type (40)	F2.0

- Type-40 records indicate the completion of the series of records for one life table.

Record Order

The SURVIVAL output file contains records for each of the life tables specified on the TABLE subcommand. All records for a given table are produced together in sequence. The records for the life tables are produced in the same order as the tables themselves. All life tables for the first survival variable are written first. The values of the first- and second-order control variables rotate, with the values of the first-order controls changing more rapidly.

Example

```
PROCEDURE OUTPUT OUTFILE = SURVTBL.
SURVIVAL TABLE = MOSFREE BY TREATMNT(1,3)
  /STATUS = PRISON (1) FOR MOSFREE
  /INTERVAL = THRU 24 BY 3
  /WRITE = BOTH.
```

- WRITE generates a procedure output file called *SURVTBL*, containing life tables, variable names and labels, and value labels stored as record types 10, 20, 30, 31, and 40.

SYSFILE INFO

```
SYSFILE INFO [FILE=] 'file specification' [PASSWORD='password']
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 22.0

- PASSWORD keyword introduced.

Example

```
SYSFILE INFO FILE='PERSNL.SAV'.
```

Overview

SYSFILE INFO displays complete dictionary information for all variables in an external IBM SPSS Statistics data file. You do not have to retrieve the file with GET to use SYSFILE INFO. If the file has already been retrieved, use DISPLAY DICTIONARY to display dictionary information.

Basic Specification

The basic specification is the command keyword and a complete file specification that is enclosed in quotes.

Syntax Rules

- Only one file specification is allowed per command. To display dictionary information for more than one IBM SPSS Statistics data file, use multiple SYSFILE INFO commands.
- The file extension, if there is one, must be specified, even if it is the default.
- The subcommand keyword FILE is optional. When FILE is specified, the equals sign is required.

Operations

- No procedure is needed to execute SYSFILE INFO, because SYSFILE INFO obtains information from the dictionary alone.
- SYSFILE INFO displays file and variable information. File information includes number of variables and cases in the file; file label; file documents; defined multiple response sets; variable sets; and information that is used by other applications (such as IBM SPSS Modeler, Data Entry for Windows, and TextSmart). Variable information includes the variable name; label; sequential position in the file; print and write format; missing values; and value labels for each variable in the specified file.

PASSWORD Keyword

The PASSWORD keyword specifies the password required to open an encrypted IBM SPSS Statistics data file. The specified value must be enclosed in quotation marks and can be provided as encrypted or as plain text. Encrypted passwords are created when pasting command syntax from the Save Data As dialog. The PASSWORD keyword is ignored if the file is not encrypted.

TCM ANALYSIS

TCM ANALYSIS is available in SPSS Statistics Professional Edition or the Forecasting option.

```
TCM ANALYSIS
     /MODELSYSTEM FILE=file

     [/EXPRESSIONS FILE=file

     /SCENARIOPERIOD
        TIMES = {ABSOLUTE}
                {RELATIVE}

        FORMAT = {VARIABLE}
                 {ISO8601 }

        START={datetime   }   END={datetime   }   [PREDICTTHRU = {datetime   } ]
              {integerList}       {integerList}                  {integerList}

     [/SCENARIO
      ID = string

      VALUES = {ACTUAL(valueList)   }
               {EXPRESSION(variable)}

      ROOTFIELD = field

      [DIMVALUES = dimField(VALUE=dimValue) dimField(VALUE=dimValue) ...]

      [EFFECTS = {AUTOMATIC**         }
                 {TARGETLIST[(listID)]}

     [/SCENARIOGROUP
      IDPREFIX = string

      EXPRESSION = variable

      ROOTFIELD = field

      [DIMVALUES = dimField(VALUES=dimValueList) dimField(VALUES=dimValueList) ...]

      [EFFECTS = {AUTOMATIC**         }
                 {TARGETLIST[(listID)]}

    [/TARGETLIST
        [LISTID = string
        FIELDS = fieldList
        [DIMVALUES = dimField(VALUES=dimValueList) dimField(VALUES=dimValueList) ...]

    [/OPTIONS

        [CILEVEL = {95**  } ]
                   {number}

        [PREDICTEDCI = {YES } ]
                       {NO**}

        [PRINT = FORECASTS ]

        [PLOT = [IMPACTDIAGRAM**] [SERIESPLOT] [NONE] ]

        [MAXTARGETS = {25**   } ]
                      {integer}

        [MAXLEVELS = {3**    } ]
                     {integer}
```

**Default if keyword omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Syntax for the TCM ANALYSIS command can be generated from the dialog.

Release History

Release 23.0

- Command introduced.

Example

```
TCM ANALYSIS
   /MODELSYSTEM FILE='/models/mymodel.zip'
   /SCENARIOPERIOD
      TIMES = ACTUAL
      FORMAT = VARIABLE
      START = '10/01/2013'
      END = '12/01/2013'
      PREDICTTHRU = '03/01/2014'
   /SCENARIO
      ID = advert_1
      VALUES = ACTUAL(5000 5500 6000)
      ROOTFIELD = advert.
```

Related information

"TCM MODEL" on page 1949
"TCM APPLY" on page 1937

Overview

The TCM ANALYSIS procedure runs user-defined scenarios for a temporal causal model system, with the data from the active dataset. A *scenario* is defined by a time series, that is referred to as the *root series*, and a set of user-defined values for that series over a specified time range. The specified values are then used to generate predictions for the time series that are affected by the root series. The procedure requires a model system file that was created by the TCM MODEL procedure. It is assumed that the active dataset is the same data as was used to create the model system file.

Basic Specification

For column-based data, the basic specification is the MODELSYSTEM subcommand, the SCENARIOPERIOD subcommand and at least one SCENARIO subcommand. For multidimensional data, the basic specification is the MODELSYSTEM subcommand, the SCENARIOPERIOD subcommand and at least one SCENARIO subcommand or SCENARIOGROUP subcommand.

Syntax Rules

- Multiple SCENARIO subcommands are allowed.
- Multiple SCENARIOGROUP subcommands are allowed. SCENARIOGROUP subcommands apply only to multidimensional data.
- Multiple TARGETLIST subcommands are allowed.
- Each keyword can be specified only once within a subcommand.
- Empty subcommands are not allowed.
- Parentheses, equals signs, and slashes that are shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.

Operations

- The procedure does not honor the PREDICT command. The TCM ANALYSIS procedure provides an option to specify how far to forecast.

Limitations

- The SPLIT FILE setting is ignored by this procedure.
- Case weighting is ignored by this procedure.

MODELSYSTEM Subcommand

The MODELSYSTEM subcommand is required and specifies the external file that contains the model system that you want.

FILE
Specifies the compressed file archive (.zip file) that contains the model system.

EXPRESSIONS Subcommand

The EXPRESSIONS subcommand specifies an external file that was created by BEGIN EXPR-END EXPR and that contains XML representations of expressions for computing scenario values. The EXPRESSIONS subcommand is required when a SCENARIOGROUP subcommand is used or when VALUES=EXPRESSION on a SCENARIO subcommand.

FILE
Specifies the external XML file that contains the expressions.

Note: The combination of an asterisk (*) and a backslash (\) preceding the XML file name specifies that the file is a temporary file.

Related information

“Specifying expressions ” on page 238

SCENARIOPERIOD subcommand

The SCENARIOPERIOD subcommand defines the period over which you specify the values that are used to run your scenarios. This period is referred to as the *scenario period*. The subcommand also specifies how far into the future to predict the effects of the scenario values. All scenarios use the same scenario period and the same specification for how far to predict into the future.

- The scenario period can start before or after the end of the estimation period. The estimation period is the same as was used to build the model system. The keyword START specifies the start of the scenario period and is required. The keyword END specifies the end of the scenario period and is required.
- The PREDICTTHRU keyword specifies how far to predict beyond the end of the scenario period. If PREDICTTHRU is omitted, then predictions are generated through the end of the scenario period.

  Note: Predictions start at the first time period after the beginning of the scenario period. For example, if the scenario period starts on 2014-11-01 and the time interval is months, then the first prediction is for 2014-12-01.

TIMES=ABSOLUTE
Specifies that START, END, and PREDICTTHRU are defined by date/time values or by lists of integers (for observations that are not defined by date/time values).

- For observations that are defined by a date, datetime, time, or string field, the FORMAT keyword is required (and ignored otherwise). It specifies the format of the values for START, END, and PREDICTTHRU. Values for START, END, and PREDICTTHRU must be quoted in this case.

  VARIABLE
  Specifies to use the variable format for the variable that defines the observations; for example, mm/dd/yyyy. This option applies only when observations are defined by a date, datetime, or time field.

  ISO8601
  Specifies that values for START, END, and PREDICTTHRU are in the general form of yyyy-mm-dd hh:mm:ss. FORMAT=ISO8601 must be specified when the observations are defined by a string field.

  - Dates are specified in the form yyyy-mm-dd. Datetimes are specified in the form yyyy-mm-dd hh:mm:ss. Times are specified in the form hh:mm:ss.
  - Two digits are not required for month, day, hours, minutes or seconds specifications. For example, 2014-1-1 is interpreted as 2014-01-01 and 3:2:1 is interpreted as 03:02:01.
  - Values can be specified with reduced precision. For example, 2014-10 is interpreted as 2014-10-01 and 03:02 is interpreted as 03:02:00.

- For observations that are defined by cyclic period fields, START, END, and PREDICTTHRU are lists of integers, consisting of one value for each of the cyclic period fields. The first value is the value for the highest level cycle field, followed by the next highest cycle field, and so on.

  For example, the following syntax specifies the start, end, and predict through time when there are two cycle levels, in addition to a period field:

```
START = 3 6 2  END = 3 6 4 PREDICTTHRU = 3 6 5
```

  The start of the scenario period is defined by a value of 3 for the second-level cycle, a value of 6 for the first-level cycle, and a value of 2 for the period field. The end of the scenario period is defined by a value of 3 for the second- level cycle, a value of 6 for the first-level cycle, and a value of 4 for the period field. The predict through time is specified by a value of 3 for the second-level cycle, a value of 6 for the first-level cycle, and a value of 5 for the period field.

- For observations that are defined by record number, START, END, and PREDICTTHRU consist of a single value that specifies the starting record number, the ending record number, and how far to predict beyond the ending record number. For example, the following syntax specifies a scenario period that starts at record 58 and ends at record 60. Predictions are generated for one more time period beyond the end of the scenario period.

```
START = 58  END = 60 PREDICTTHRU = 61
```

- For observations that are defined by a date specification (DATE command), START, END, and PREDICTTHRU are lists of integers that consist of one value for each of the components of the date specification. For example, if the date specification is YEAR and MONTH, then the lists consist of two items, one for the year and one for the month. The order of the values is from the highest level component to the lowest level component. For example, the following syntax specifies start, end, and predict through times for a date specification that is defined by YEAR and MONTH:

```
START = 2013 10  END = 2013 12 PREDICTTHRU = 2014 3
```

  The start of the scenario period is defined by a value of 2013 for the year and 10 for the month. The end of the scenario period is defined by a value of 2013 for the year and 12 for the month. Predictions are generated through to a value of 2014 for the year and 3 for the month.

TIMES=RELATIVE

Specifies that START and END are defined by time intervals that are relative to the end of the estimation period. The time interval refers to the interval at which the data were modeled. For example, if the observations are daily but the data are aggregated to months, then the time interval is months.

- The end of the estimation period is defined as interval 0. Negative values represent intervals prior to the end of the estimation period and positive values represent intervals after the end of the estimation period.
- PREDICTTHRU must be a non-negative integer and represents the number of intervals past the time that is specified for the END keyword.

SCENARIO subcommand

The SCENARIO subcommand defines a single scenario. Multiple SCENARIO subcommands are allowed.

ID
Specifies an identifier for the scenario. The identifier is displayed in output associated with the scenario. The value must be a quoted string. The ID keyword is required.

VALUES
Specifies the values of the root field that are used for the scenario.

ACTUAL
Specifies that explicit values are provided for the scenario. The specification is a list of numeric values that defines the values for the scenario. The list must have the same number of values as the number of time intervals in the scenario period (as defined by the SCENARIOPERIOD subcommand).

EXPRESSION
Specifies that the scenario values are computed from an expression. The specification is the name of the variable, from the expressions file (specified on the EXPRESSIONS subcommand), that defines the expression.

Note: Expressions are specified with COMPUTE statements in a BEGIN EXPR-END EXPR block. The variable that is specified on the EXPRESSION keyword is a target of one of those COMPUTE statements.

ROOTFIELD
Specifies the root field for the scenario. For multidimensional data, the ROOTFIELD keyword must be accompanied by the DIMVALUES keyword which specifies the set of dimension values that (along with ROOTFIELD) uniquely defines the root time series.

DIMVALUES
Specifies the list of dimension field values that (along with ROOTFIELD) uniquely defines the root time series. The keyword is required for multidimensional data and ignored otherwise. The form of the specification for a particular dimension field is dimField(VALUE=dimValue), where dimField is the name of the field and dimValue is the value for that field. A value must be specified for each dimension field.

- Fields that are specified on the DIMVALUES keyword do not support ALL and TO.
- String values do not need to be quoted.

EFFECTS
Specifies how to determine the set of series that are affected by the root series.

AUTOMATIC
Specifies that the procedure automatically determines the series that are affected by the root series. This is the default. You can specify the breadth of the series that are affected by the scenario with settings on the OPTIONS subcommand.

TARGETLIST
Specifies that the effects of the root series are limited to a specified set of series.

- If TARGETLIST is specified without *listID*, then the set of affected targets is given by the TARGETLIST subcommands that do not have a LISTID keyword.
- If TARGETLIST is specified with a value for *listID*, then the set of affected targets is given by the TARGETLIST subcommands whose LISTID keyword equals the specified value. The *listID* value is not quoted.

Related information
“Specifying expressions ” on page 238
“OPTIONS subcommand ” on page 1935
“TARGETLIST subcommand ” on page 1935

SCENARIOGROUP subcommand

The SCENARIOGROUP subcommand defines a group of scenarios that are based on a single root field and multiple sets of dimension values for that root field. Each set of dimension values (one value per dimension field) defines a time series. An individual scenario is then generated for each such time series, whose values in the scenario period are then replaced by the scenario values. Scenario values are specified by an expression, which is applied to each time series in the group. The SCENARIOGROUP subcommand applies only to the case of multidimensional data. Multiple SCENARIOGROUP subcommands are allowed.

IDPREFIX
Specifies a prefix that is used to generate an identifier for each scenario in the scenario group. The identifier is displayed in output associated with each scenario. The value must be a quoted string. The identifier for a particular scenario contains the prefix and the set of dimension values that define the series for that scenario. Specifically, the identifier is the prefix, followed by an underscore, followed by the value of the first dimension field, followed by an underscore, followed by the value of the second dimension field, and so on. The IDPREFIX keyword is required.

EXPRESSION
Specifies the name of the variable, from the expressions file (specified on the EXPRESSIONS subcommand), that defines the expression that is used to compute the scenario values.

Note: Expressions are specified with COMPUTE statements in a BEGIN EXPR-END EXPR block. The variable that is specified on the EXPRESSION keyword is a target of one of those COMPUTE statements.

ROOTFIELD
Specifies the root field which, along with a set of dimension values, defines a root series.

DIMVALUES
Specifies optional lists of dimension field values that define the sets of dimension values that, together with the root field, define the set of root series for the scenario group. The form of the specification for a particular dimension field is `dimField(VALUES=dimValueList)`, where `dimField` is the name of the field and `dimValueList` is the list of values for that field.

- If a dimension field is not specified on DIMVALUES then all values in that dimension are included.
- Fields that are specified on the DIMVALUES keyword do not support ALL and TO.
- String values do not need to be quoted.

EFFECTS
Specifies how to determine the set of series that are affected by each root series in the scenario group.

AUTOMATIC
Specifies that the procedure automatically determines the series that are affected by the root series. This is the default. You can specify the breadth of the series that are affected by each scenario with settings on the OPTIONS subcommand.

TARGETLIST
Specifies that the effects of the root series are limited to a specified set of series. Each root series in the group has the same set of affected targets

- If TARGETLIST is specified without *listID*, then the set of affected targets is given by the TARGETLIST subcommands that do not have a LISTID keyword.
- If TARGETLIST is specified with a value for *listID*, then the set of affected targets is given by the TARGETLIST subcommands whose LISTID keyword equals the specified value. The *listID* value is not quoted.

Related information

“Specifying expressions ” on page 238
“OPTIONS subcommand ” on page 1935

“TARGETLIST subcommand ” on page 1935

TARGETLIST subcommand

The TARGETLIST subcommand defines a set of target series that is used to limit the set of series that are affected by one or more scenarios. Multiple TARGETLIST subcommands can be specified.

LISTID
Optional setting that identifies a particular group of TARGETLIST subcommands. Multiple TARGETLIST subcommands can have the same LISTID parameter. The LISTID parameter is not quoted.

FIELDS
Specifies the fields for the current TARGETLIST subcommand.

DIMVALUES
Specifies optional lists of dimension field values that define a subset of series for the fields that are specified on the FIELDS keyword. The DIMVALUES keyword applies only to multidimensional data. The form of the specification for a particular dimension field is `dimField(VALUES=dimValueList)`, where `dimField` is the name of the field and `dimValueList` is the list of values for that field.

- If a dimension field is not specified on DIMVALUES then all values in that dimension are included.
- Fields that are specified on the DIMVALUES keyword do not support ALL and TO.
- String values do not need to be quoted.

Related information
“SCENARIO subcommand ” on page 1932
“SCENARIOGROUP subcommand ” on page 1934

OPTIONS subcommand

CILEVEL
Specifies the confidence level for the scenario predictions. The value is a positive number that is greater than 0 and less than 100. The default is 95.

PREDICTEDCI
Specifies whether confidence intervals for scenario predictions are included in both chart and table output. The default is NO, which specifies that confidence intervals are not included.

PRINT=FORECASTS
Tables of predicted values and scenario values for each scenario. These tables contains the same information as the tables in the impact diagram (PLOT=IMPACTDIAGRAM). These tables support all standard features for pivoting and editing tables. By default, these tables are not generated.

PLOT

IMPACTDIAGRAM
Displays a graphical representation of the causal relations between the root series for each scenario and the target series that it affects. Tables of both the predicted values for the affected targets and the scenario values are included as part of the output item. The graph includes plots of the predicted values of the affected targets. Single-clicking any node in the impact diagram opens a detailed sequence diagram for the series that is associated with the node. A separate impact diagram is generated for each scenario. Impact diagrams are generated by default.

SERIESPLOT
Generates series plots of the predicted values for each of the affected targets in each scenario. By default, series plots are not generated.

NONE
Specifies that no plots are generated. NONE cannot be specified with any other keywords.

MAXTARGETS

Specifies the maximum number of targets that are automatically detected when EFFECTS=AUTOMATIC is specified. The value is a positive integer and the default is 25. Increasing the value of this setting increases the complexity of the computation and might affect performance.

MAXLEVELS

Specifies the maximum number of levels of affected targets that are considered. Each successive level includes targets that are more indirectly affected by the root series. Specifically, the second level includes targets that have the root series as a direct input. Targets in the third level have targets in the second level as a direct input, and so on. The allowed values are the integers from 1 to 5. The default is 3. Increasing the value of this setting increases the complexity of the computation and might affect performance.

TCM APPLY

TCM APPLY is available in SPSS Statistics Professional Edition or the Forecasting option.

```
TCM APPLY
     /MODELSYSTEM FILE=file

    [/OPTIONS
        [REESTIMATE = {NO**} ]
                      {YES }

        [ESTIMATIONPERIOD = {MODEL**
                            {ALL
                            {TIMES(START={datetime   } END={datetime   } FORMAT={VARIABLE})
                                         {integerList}     {integerList}        {ISO8601 }
                            {OBSERVATIONS(TYPE={NEWEST} [OFFSET={0**    }] [NUMBER = {1000** }])
                                              {OLDEST}         {integer}            {integer}

        [FORECASTPERIODS = {integer}]
                           {1**    }

        [CILEVEL = {95**  }]
                   {number}

        [OUTLIERTHRESH={95** }
                       {value}


    [/TARGETFILTER
        DISPLAY = {BESTFIT([{N=10** | integer }] [FITSTAT = {RSQUARE**}]) }
                           {PCT=percent      }             {RMSPE    }
                                                           {RMSE     }
                                                           {BIC      }
                                                           {AIC      }
                  {ALLTARGETS                                        }
                  {TARGETSNOINPUTS                                   }

    [/SERIESFILTER
        FIELDS = fieldList
        [DIMVALUES = dimField({INCLUDE}=dimValueList) dimField({INCLUDE}=dimValueList) ...]
                             {EXCLUDE}                        {EXCLUDE}

    [/FILTEREDOUTPUT
        [INCLUDE = {TARGETS | SERIES | BOTH**} ]

        [NETWORKGRAPH = {YES**}([SIGLEVEL = {0.05**}])
                        {NO   }             {value }


        [PRINT = [MODELFIT] [OUTLIERS] [MODELEFFECTS] [PARAMETERS] [FORECASTS] ]

        [PLOT = [SERIESPLOT] [TOPINPUTS] [NRESIDUAL]

        [IMPACTDIAGRAMS = CAUSES[({3**    })] EFFECTS[({3**    })] ]
                                  {integer}            {integer}

        [OUTLIER_ROOTCAUSES = {NO**}
                              {YES }([LEVELS = {3**    }] [DISPLAY = {INTERACTIVE**}])
                                               {integer}             {TABLE        }
                                                                     {BOTH         }

    [/SYSTEMOUTPUT
        [MODELFIT = [RSQUARE**] [RMSPE] [RMSE] [BIC] [AIC] [NONE] ]

        [TRANSFORMATIONS = {NO   } ]
                           {YES**}

        [OUTLIERPLOT = {NO**} ]
                       {YES }

    [/SAVE
       [TARGETS({FILTERED**})
                {All       }
       [PREDICTED[(rootname)]]
       [CL[(LCLROOT=rootname UCLROOT=rootname)]]
       [NRESIDUAL[(rootname)]]
       [FORECASTFLAG[(name)]]

    [/OUTFILE
        [MODELSYSTEM = 'filespec']
        [DATA = {'filespec' | dataset}]

    [/PASSPROTECT
        [PASSWORD={'password'|NONE**}]
        [ENCRYPTEDPW={NO**|YES}]
```

**Default if keyword omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Syntax for the TCM APPLY command can be generated from the dialog.

Release History

Release 23.0

- Command introduced.

Example

```
TCM APPLY
   /MODELSYSTEM FILE='/models/mymodel.zip'
   /OPTIONS REESTIMATE = YES FORECASTPERIODS = 3
   /SAVE TARGETS(ALL)
```

Related information

"TCM MODEL" on page 1949
"TCM ANALYSIS" on page 1929

Overview

The TCM APPLY procedure loads a model system file that was created by the TCM MODEL procedure and applies the models to the active dataset. You can use this procedure to obtain forecasts for series for which more current data is available, without rebuilding your models. You can also generate any of the output that is available with the TCM MODEL procedure.

Assumptions

- The structure of the data in the active dataset, either column-based or multidimensional, must be the same structure that was used when the model system was built. For multidimensional data, the dimension fields must be the same as used to build the model system. Also, the dimension values that were used to build the model system must exist in the active dataset.
- Models are applied to fields in the active dataset with the same names as the fields specified in the model system.
- The field, or fields, that defined the observations when the model system was built must exist in the active dataset. The time interval between observations is assumed to be the same as when the models were built. If the observations were defined by a date specification, then the same date specification must exist in the active dataset. Date specifications are created from the Define Dates dialog or the DATE command.
- The time interval of the analysis and any settings for aggregation, distribution, and missing values are the same as when the models were built.

Basic Specification

The basic specification is the MODELSYSTEM subcommand that specifies the model system file. This specification generates the Overall Model System visualization for the 10 best-fitting models, a histogram of model fit for all models, and a table of any transformations that were applied to the series in the model system. Also, forecasts are produced for the 10 best-fitting models, for one time interval beyond the end of the original estimation period. The forecasts are saved as new variables.

Syntax Rules

- Multiple SERIESFILTER subcommands are allowed.

- Multiple FILTEREDOUTPUT subcommands are allowed. If the same value of the INCLUDE keyword appears on multiple FILTEREDOUTPUT subcommands, then the specifications for only the last such subcommand are honored.
- Each keyword can be specified only once within a subcommand.
- Empty subcommands are not allowed.
- Parentheses, equals signs, and slashes that are shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.

Operations

- The procedure honors the USE command but it also has settings for specifying the estimation period. Settings on the ESTIMATIONPERIOD keyword are applied after any case selection that is specified with the USE command.
- The procedure does not honor the PREDICT command. The TCM APPLY procedure provides an option to specify the forecast period.
- The procedure does not honor the TSET command.
- The procedure does not honor the following commands: MODEL NAME or SAVE MODEL. The TCM APPLY procedure provides the option to save the model system when model parameters are reestimated.
- The TDISPLAY command does not display model systems that are saved from TCM APPLY.

Limitations

- The SPLIT FILE setting is ignored by this procedure.
- Case weighting is ignored by this procedure.

MODELSYSTEM Subcommand

The MODELSYSTEM subcommand is required and specifies the external file that contains the model system that you want.

FILE
Specifies the compressed file archive (.zip file) that contains the model system.

OPTIONS subcommand

REESTIMATE
Specifies whether model parameters are reestimated. The default is NO, which specifies that the parameters from the model system file are used to generate forecasts.

ESTIMATIONPERIOD
Specifies the estimation period that defines the set of observations that are used for the analysis.

MODEL
Specifies to use the estimation period that was used for building the model system. This setting is the default.

ALL
Specifies that all observations are used. When ALL is specified, the estimation period starts at the time of the earliest observation and ends at the time of the latest observation across all series.

TIMES
Specifies a range of observations by start and end time.

- If START is specified without END, then all observations from START to the last observation are used. If END is specified without START, then the first observation through END is used.

- For observations that are defined by a date, datetime, time, or string field, the FORMAT keyword is required (and ignored otherwise). It specifies the format of the values for START and END. Values for START and END must be quoted in this case.

  VARIABLE
  Specifies to use the variable format for the variable that defines the observations; for example, mm/dd/yyyy. This option only applies when observations are defined by a date, datetime, or time field.

  ISO8601
  Specifies that values for START and END are in the general form of yyyy-mm-dd hh:mm:ss. FORMAT=ISO8601 must be specified when the observations are defined by a string field.

  – Dates are specified in the form yyyy-mm-dd. Datetimes are specified in the form yyyy-mm-dd hh:mm:ss. Times are specified in the form hh:mm:ss.

  – Two digits are not required for month, day, hours, minutes or seconds specifications. For example, 2014-1-1 is interpreted as 2014-01-01 and 3:2:1 is interpreted as 03:02:01.

  – Values can be specified with reduced precision. For example, 2014-10 is interpreted as 2014-10-01 and 03:02 is interpreted as 03:02:00.

- For observations that are defined by cyclic period fields, START and END are lists of integers, consisting of one value for each of the cyclic period fields. The first value is the value for the highest level cycle field, followed by the next highest cycle field, and so on.

  For example, the following syntax specifies the start and end time when there are two cycle levels, in addition to a period field:

  ```
  START = 3 6 2  END = 5 1 4
  ```

  The start of the estimation period is defined by a value of 3 for the second-level cycle, a value of 6 for the first-level cycle, and a value of 2 for the period field. The end of the estimation period is defined by a value of 5 for the second- level cycle, a value of 1 for the first-level cycle, and a value of 4 for the period field.

- For observations that are defined by record number, START and END consist of a single value that specifies the starting record number and the ending record number. For example, the following syntax specifies an estimation period that starts at record 1 and ends at record 60:

  ```
  START = 1  END = 60
  ```

- For observations that are defined by a date specification (DATE command), then START and END are lists of integers that consist of one value for each of the components of the date specification. For example, if the date specification is YEAR and MONTH, then the lists consist of two items, one for the year and one for the month. The order of the values is from the highest level component to the lowest level component. For example, the following syntax specifies start and end time for a date specification that is defined by YEAR and MONTH:

  ```
  START = 2008 1  END = 2013 12
  ```

  The start of the estimation period is defined by a value of 2008 for the year and 1 for the month. The end of the estimation period is defined by a value of 2013 for the year and 12 for the month.

OBSERVATIONS
Defines the estimation period as a specified number of time intervals that start at the earliest time interval or end at the latest time interval in the data, with an optional offset. In this context, the time interval refers to the interval at which the data were modeled. For example, if the observations are daily but the data are aggregated to months, then the time interval that is associated with OBSERVATIONS is months.

TYPE
TYPE=NEWEST specifies to use the most recent time intervals. TYPE=OLDEST specifies to use the earliest time intervals.

OFFSET
Specifies an offset from the earliest time interval (when TYPE=OLDEST) or the most recent time interval (when TYPE=NEWEST). The value of OFFSET must be an integer greater than or equal to 0. The default is 0 and specifies to start at the earliest time interval or end at the most recent time interval.

NUMBER
Specifies the number of time intervals to include. The value must be an integer that is greater than or equal to 2. The default is 1000. If the specified value exceeds the number of time intervals in the data, then all time intervals, starting from the optional offset, are included.

FORECASTPERIODS
Specifies the number of time intervals to forecast beyond the end of the estimation period. The default value is 1. In this context, the time interval refers to the interval at which the data were modeled. For example, if the observations are daily but the data are aggregated to months, then specifying 3 for FORECASTPERIODS means 3 months.

CILEVEL
Specifies the confidence level for computing confidence intervals for forecasts and parameter estimates. The value must be positive number that is greater than 0 and less than 100. The default is 95.

OUTLIERTHRESH
An observation is flagged as an outlier if the probability, as calculated from the model, that it is an outlier exceeds this threshold. You can specify a value in the range 50 - 100. The default value is 95.

TARGETFILTER subcommand

The TARGETFILTER subcommand specifies a set of target series for which output is displayed. The content of the output for the specified series is determined by the FILTEREDOUTPUT subcommand. You can specify particular series, for which output is displayed, from the SERIESFILTER subcommand.

BESTFIT
Specifies to generate output for the target series that have the best-fitting models. By default, output is generated for the top 10 models according to the value of the root mean square percentage error.

- The N keyword specifies the number of models to include, as an integer. If the count exceeds the number of models, all models are shown. The default is 10.
- The PCT keyword specifies the number of models to include, as a percentage of the total number of models. The percentage value must be an integer between 1 and 100.
- The FITSTAT keyword specifies the fit statistic that is used to determine the best-fitting models. The default is RSQUARE.

  RSQUARE
  Goodness-of-fit measure of a linear model, sometimes called the coefficient of determination. It is the proportion of variation in the target variable explained by the model. It ranges in value from 0 to 1. Small values indicate that the model does not fit the data well.

  RMSPE
  Root Mean Square Percentage Error. A measure of how much the model-predicted values differ from the observed values of the series. It is independent of the units that are used and can therefore be used to compare series with different units.

  RMSE
  Root Mean Square Error. The square root of mean square error. A measure of how much a dependent series varies from its model-predicted level, expressed in the same units as the dependent series.

  BIC
  Bayesian Information Criterion. A measure for selecting and comparing models based on the -2 reduced log likelihood. Smaller values indicate better models. The BIC also "penalizes"

overparameterized models (complex models with a large number of inputs, for example), but more strictly than the AIC.

AIC
Akaike Information Criterion. A measure for selecting and comparing models based on the -2 reduced log likelihood. Smaller values indicate better models. The AIC "penalizes" overparameterized models (complex models with a large number of inputs, for example).

ALLTARGETS
Specifies that output is generated for all target series.

TARGETSNOINPUTS
Specifies that only target series with no inputs (other than lagged values of the target itself) are included.

SERIESFILTER subcommand

The SERIESFILTER subcommand specifies particular series for which output is displayed. The content of the output for the specified series is determined by the FILTEREDOUTPUT subcommand. You can specify multiple SERIESFILTER subcommands.

FIELDS
The FIELDS keyword is required and specifies the list of fields that contain the series that you want. The keyword TO is supported but the keyword ALL is not supported. For multidimensional data, all series that are contained in each specified field are included by default. You can specify a subset of series by specifying particular sets of values for one or more dimension fields on the DIMVALUES keyword.

DIMVALUES
Specifies optional lists of dimension field values that define a subset of series for the fields that are specified on the FIELDS keyword. The DIMVALUES keyword applies only to multidimensional data. The form of the specification for a particular dimension field is `dimField(INCLUDE|EXCLUDE=dimValueList)`, where `dimField` is the name of the field and `dimValueList` is a list of values for that field.

- The INCLUDE or EXCLUDE keyword is required.
- INCLUDE specifies a list of values, from the associated dimension field, to include.
- EXCLUDE specifies a list of values, from the associated dimension field, to exclude.
- You can specify included or excluded values for one or more dimension fields. For dimension fields that are not specified on DIMVALUES, all values of the associated field are included.
- Fields that are specified on the DIMVALUES keyword do not support ALL and TO.
- String values do not need to be quoted, unless the string contains apostrophes or quotation marks. If the string contains apostrophes or single quotes, then enclose the string in double quotes. If the string contains double quotes, then enclose the string in single quotes.

FILTEREDOUTPUT subcommand

The FILTEREDOUTPUT subcommand controls the output that is displayed for the series that are specified on the TARGETFILTER and SERIESFILTER subcommands.

INCLUDE
Specifies the set of series to which the current FILTEREDOUTPUT subcommand applies. You can specify up to three FILTEREDOUTPUT subcommands, one for each of the values of INCLUDE.

- TARGETS specifies that the current settings apply to the targets that are specified by the TARGETFILTER subcommand.
- SERIES specifies that the current settings apply to the series that are specified by the SERIESFILTER subcommands.

- BOTH specifies that the current settings apply to the targets that are specified by the TARGETFILTER subcommand and the series that are specified by the SERIESFILTER subcommands.

NETWORKGRAPH
Specifies whether the Overall Model System output item is generated. The default is YES. The Overall Model System item displays a graphical representation of the causal relations between series in the model system. Tables of both model fit statistics and outliers for the displayed targets are included as part of the output item. When this option is specified with INCLUDE=SERIES or INCLUDE=BOTH, a separate output item is created for each series that is specified on a SERIESFILTER subcommand.

- The SIGLEVEL keyword specifies the significance level threshold for hiding causal connections between series in the Overall Model System graph. A smaller significance level indicates a more significant connection. Connections with significance levels that are higher than the threshold are hidden. Specify a numeric value in the range 0-1. The default is 0.05.

PRINT
Controls the display of tabular output. One or more of the following keyword values can be specified. By default, none of this tabular output is displayed.

MODELFIT
Tables of model fit statistics for the target series that are selected for display. The tables contain the same information as the model fit statistics table in the Overall Model System item. The tables support all standard features for pivoting and editing tables.

OUTLIERS
Tables of outliers for the target series that are selected for display. The tables contain the same information as the outliers table in the Overall Model System item. The tables support all standard features for pivoting and editing tables.

MODELEFFECTS
Tables of model effects tests for the target series that are selected for display. Model effects tests include the F statistic and associated significance value for each input included in the model.

PARAMETERS
Tables of model parameters for the target series that are selected for display.

FORECASTS
Tables of forecasted values and confidence intervals of those forecasts for the target series that are selected for display. The FORECASTS keyword is ignored if forecasts are not requested. Forecasts are requested with the FORECASTPERIODS keyword on the OPTIONS subcommand.

PLOT
Controls the display of chart output. One or more of the following keyword values can be specified. By default, none of this chart output is displayed.

SERIESPLOT
Plots of observed and predicted values for the target series that are selected for display. When forecasts are requested, the plot also shows the forecasted values and the confidence intervals for the forecasts.

TOPINPUTS
Plots of each displayed target, over time, along with the top 3 inputs for the target. The top inputs are the inputs with the lowest significance value. To accommodate different scales for the inputs and target, the y axis represents the z score for each series.

NRESIDUAL
Plots of the model residuals for the target series that are selected for display.

IMPACTDIAGRAMS
Specifies whether impact diagrams are generated. An impact diagram displays a graphical representation of the causal relations between a series of interest and other series that it affects or that affect it. Series that affect the series of interest are referred to as *causes*. When the IMPACTDIAGRAMS keyword is specified, impact diagrams are generated for all series that are selected for display. By default, impact diagrams are not generated.

EFFECTS
Creates an impact diagram that is initialized to show the series that are affected by the series of interest. You can specify the number of levels of effects to display, as in EFFECTS(4) for four levels of effects. The first level represents the series of interest. Each additional level shows more indirect effects of the series of interest. For example, the third level consists of the series that contain series in the second level as a direct input. Series in the third level are then indirectly affected by the series of interest since the series of interest is a direct input to the series in the second level. By default, 3 levels of effects are displayed. You can specify any number of levels up to a maximum of 99.

CAUSES
Creates an impact diagram that is initialized to show the series that affect the series of interest. You can specify the number of levels of causes to display, as in CAUSES(4) for four levels of causes. The concept of levels for causes is the same as for levels of effects. By default, 3 levels of causes are displayed. You can specify any number of levels up to a maximum of 99.

Note: You can interactively toggle between causes and effects in the output item that displays the impact diagram.

OUTLIER_ROOTCAUSES
Specifies whether outlier root cause analysis is done. Outlier root cause analysis determines which series are most likely to be the cause of each outlier in a series of interest. The analysis is done for each series that is specified on a SERIESFILTER subcommand and that has outliers. By default, outlier root cause analysis is not done.

Note: Outliers are detected based on the active dataset and the temporal causal model system.

LEVELS
Specifies the number of causal levels that are searched for root causes of outliers. By default, 3 causal levels are searched. The maximum number of levels is 5. The concept of levels that is used here is the same as described for impact diagrams.

DISPLAY

- The value INTERACTIVE creates an interactive visualization that contains a table and chart of outliers and root causes of those outliers for each series of interest. The table contains a single row for each outlier. The chart is an impact diagram. Selecting a row in the table highlights the path, in the impact diagram, from the series of interest to the series that most likely causes the associated outlier.
- The value TABLE creates a table of outliers and root causes of those outliers for each series of interest. This table contains the same information as the table in the interactive display. This table supports all standard features for pivoting and editing tables.
- The value BOTH generates the interactive visualization and the separate table.

SYSTEMOUTPUT subcommand

The SYSTEMOUTPUT subcommand controls output associated with the entire temporal causal model system.

MODELFIT
Controls the display of histograms of model fit for all models. One or more fit statistics can be specified. A single histogram that is based on RSQUARE is generated by default. When REESTIMATE=NO, the fit statistics are taken from the model system file.

RSQUARE
Goodness-of-fit measure of a linear model, sometimes called the coefficient of determination. It is the proportion of variation in the target variable explained by the model. It ranges in value from 0 to 1. Small values indicate that the model does not fit the data well.

RMSPE
Root Mean Square Percentage Error. A measure of how much the model-predicted values differ from the observed values of the series. It is independent of the units that are used and can therefore be used to compare series with different units.

RMSE
Root Mean Square Error. The square root of mean square error. A measure of how much a dependent series varies from its model-predicted level, expressed in the same units as the dependent series.

BIC
Bayesian Information Criterion. A measure for selecting and comparing models based on the -2 log likelihood. Smaller values indicate better models. The BIC also "penalizes" overparameterized models (complex models with a large number of inputs, for example), but more strictly than the AIC.

AIC
Akaike Information Criterion. A measure for selecting and comparing models based on the -2 log likelihood. Smaller values indicate better models. The AIC "penalizes" overparameterized models (complex models with a large number of inputs, for example).

NONE
Specifies that histograms are not generated. NONE cannot be specified with any other keywords.

TRANSFORMATIONS
Table of any transformations that were applied to the series in the model system. The possible transformations are missing value imputation, aggregation, and distribution. The default is YES, which specifies that the table is generated.

OUTLIERPLOT
Bar chart of the number of outliers, across all targets, for each time interval in the estimation period. The default is NO, which specifies that the bar chart is not generated.

Note: Outliers are detected based on the active dataset and the temporal causal model system.

SAVE subcommand

The SAVE subcommand saves new variables that represent predicted values, residuals, and confidence intervals. By default, no new variables are saved.

- Specify one or more keywords, along with optional rootnames to be used as the prefix for new variable names.
- The rootname, if specified, must conform to the rules for valid variable names.
- If the DATA keyword of the OUTFILE subcommand is specified, then the new variables are written to that location. If the DATA keyword is not specified and the new variables cannot be written to the active dataset, then they are written to an autogenerated dataset. New variables can be written to the active dataset only if the observations are defined by a date specification, or record order, and the data are not aggregated.

TARGETS

ALL
Specifies that new variables are created for all targets in the model system.

FILTERED
Specifies that new variables are created just for the targets that are specified by the TARGETFILTER and SERIESFILTER subcommands. This option is the default.

PREDICTED(rootname)
Saves model fit and forecast values. The default rootname is *Predicted*.

CL(LCLROOT=rootname UCLROOT=rootname)
Saves upper and lower confidence limits. LCLROOT specifies the root name for the lower confidence limit and UCLROOT specifies the root name for the upper confidence limit. The default root name for the lower confidence limit is *LCL* and the default root name for the upper confidence limit is *UCL*.

NRESIDUAL(rootname)
Saves noise residuals. The default rootname is *NResidual*.

FORECASTFLAG(name)
Creates a variable that indicates whether a record contains forecast data. You can specify the variable name. The default is *ForecastIndicator*.

OUTFILE subcommand

MODELSYSTEM
Saves the model system, which is associated with reestimated model parameters, to a compressed file archive (.zip file). The model system file can be reused by this procedure. It can also be used by the TCM ANALYSISprocedure to run scenario analysis. The keyword is ignored if REESTIMATE=NO on the OPTIONS subcommand.

DATA
Specifies the name of a file or a previously declared dataset where new variables that are specified on the SAVE subcommand are written, along with any transformations of the data (such as aggregation or imputation of missing values).

- Date/time values in the saved data are aligned to the beginning of each time interval, such as the first of the month, and represent the time interval at which the data were modeled. For example, if the observations are daily but the data are aggregated to months then date/time values in the saved data represent months.
- Enclose file specifications in quotation marks and use fully qualified paths to avoid ambiguity. You can use an asterisk (*) to specify that the data is written to the active dataset, replacing the current active dataset.

Related information
"TCM ANALYSIS" on page 1929

PASSPROTECT subcommand

The PASSPROTECT subcommand saves an external file, that was specified on the DATA keyword of the OUTFILE subcommand, as an encrypted file. An encrypted file can be opened only by providing the correct password.

PASSWORD
Specifies the password that is required to open this file. Passwords are limited to 10 characters and are case-sensitive. The specified value must be enclosed in quotation marks. The keyword NONE specifies not to encrypt the file and is the default.

ENCRYPTEDPW
Specifies whether the password is encrypted.

NO
The password is not encrypted. It is treated as plain text. This option is the default.

YES
The password is encrypted. Use ENCRYPTEDPW=YES only when the password is known to be encrypted. Passwords are always encrypted in pasted syntax.

Creating strong passwords

- Use eight or more characters.
- Include numbers, symbols and even punctuation in your password.

- Avoid sequences of numbers or characters, such as "123" and "abc", and avoid repetition, such as "111aaa".
- Do not create passwords that use personal information such as birthdays or nicknames.
- Periodically change the password.

Note: Encrypted data files and output documents cannot be opened in versions of IBM SPSS Statistics prior to version 21. Encrypted syntax files cannot be opened in versions prior to version 22.

TCM MODEL

TCM MODEL is available in SPSS Statistics Professional Edition or the Forecasting option.

```
TCM MODEL

Global subcommands:

     /DATASETTINGS

        [DIMENSIONS = varlist

        [MAXDIMVALUES = {integer}
                        {10000**}

         TIMETYPE = {DATETIMEVAR  }
                    {ISO8601      }
                    {CYCLICPERIODS}
                    {NONE         }

        [TIMEVAR = {var
                   {var(TYPE=PERIOD [INCREMENT={1**    }] [START={1**    }]) [var(TYPE=CYCLE LEVEL=integer LENGTH=integer
[START={1**    }]) ...]
                                             {integer}
{integer}                                                  {integer}
        [INPUTINTERVAL = {AUTODETECT**
                         {IRREGULAR
                         {YEAR[(INCREMENT = {integer})]
                                           {1**    }
                         {QUARTER
                         {MONTH[(INCREMENT = {integer})]
                                           {1**    }
                         {WEEK
                         {DAY[(DAYSWK = {integer} WKSTART={MON | TUE | WED | THU | FRI | SAT | SUN**} INCREMENT = {integer}) ]
                                        {7**    }                                                    {1**    }
                         {HOUR[(DAYSWK = {integer} WKSTART={MON | TUE | WED | THU | FRI | SAT | SUN**}
                                         {7**    }
                                HRSDAY = {integer} DAYSTART = {integer} INCREMENT = {integer}) ]
                                         {24**   }            {0**    }             {1**    }
                         {MINUTE[(DAYSWK = {integer} WKSTART={MON | TUE | WED | THU | FRI | SAT | SUN**}
                                         {7**    }
                                  HRSDAY = {integer} DAYSTART = {integer} INCREMENT = {integer} )]
                                         {24**   }            {0**    }             {1**    }
                         {SECOND[(DAYSWK = {integer} WKSTART={MON | TUE | WED | THU | FRI | SAT | SUN**}
                                         {7**    }
                                  HRSDAY = {integer} DAYSTART = {integer} INCREMENT = {integer} )]
                                         {24**   }            {0**    }             {1**    }
                         {DURATION(INTERVAL={HOUR  } [INCREMENT = {integer}] )
                                            {MINUTE}              {1**    }
                                            {SECOND}

        [ANALYSISINTERVAL = {USEINPUT** | YEAR | QUARTER | MONTH | WEEK | DAY | HOUR | MINUTE | SECOND | CYCLE(LEVEL=integer) }]

        [GROUP = {SUM**}  [var({SUM }) var({SUM }) ...] ]
                 {MEAN }       {MEAN}      {MEAN}
                 {MODE }       {MODE}      {MODE}
                 {MIN  }       {MIN }      {MIN }
                 {MAX  }       {MAX }      {MAX }

        [AGGREGATE = {SUM**}  [var({SUM }) var({SUM }) ...] ]
                     {MEAN }       {MEAN}      {MEAN}
                     {MODE }       {MODE}      {MODE}
                     {MIN  }       {MIN }      {MIN }
                     {MAX  }       {MAX }      {MAX }

        [DISTRIBUTE = {SUM**}  [var({SUM }) var({SUM }) ...] ]
                      {MEAN }       {MEAN}      {MEAN}

        [INTERPOLATE = {LINT** | MEAN | KMEAN[(integer)] | KMEDIAN[(integer)] | LTREND}]
                                          (2**    )            (2**    )

        [USERMISSING = {EXCLUDE**} ]
                       {INCLUDE  }

        [MAXMISSING = {value}]
                      {25** }

        [CROSSDAYHRS = {EXCLUDE**}]
                       {INCLUDE  }

    [/DATAFILTER
        [DIMVALUES = dimField({INCLUDE}=dimValueList) dimField({INCLUDE}=dimValueList) ...]
                             {EXCLUDE}                       {EXCLUDE}

    [/BUILDOPTIONS
        [ESTIMATIONPERIOD = {ALL**
                            {TIMES(START={datetime   } END={datetime   } FORMAT={VARIABLE})
                                         {integerList}     {integerList}        {ISO8601 }
                            {OBSERVATIONS(TYPE={NEWEST} [OFFSET={0**    }] [NUMBER = {1000** }])
                                         {OLDEST}         {integer}            {integer}

        [FORECASTPERIODS = {integer}]
                           {0**    }

        [CILEVEL = {95**  }]
                   {number}

        [MAXINPUTS = {DEFAULT**}]
                     {integer  }
```

```
        [LAGS = {AUTO** }]
               {integer}

        [TOLERANCE = {1E-3**}]
                     {value }

        [OUTLIERTHRESH={95**  }
                       {value }

    [/TARGETFILTER
        DISPLAY = {BESTFIT([{N=10** | integer }] [FITSTAT = {RSQUARE**}]) }
                           {PCT=percent      }             {RMSPE    }
                                                           {RMSE     }
                                                           {BIC      }
                                                           {AIC      }
                  {ALLTARGETS                                        }
                  {TARGETSNOINPUTS                                   }

    [/SERIESFILTER
        FIELDS = fieldList
        [DIMVALUES = dimField({INCLUDE}=dimValueList) dimField({INCLUDE}=dimValueList) ...]
                              {EXCLUDE}                       {EXCLUDE}

    [/FILTEREDOUTPUT
        [INCLUDE = {TARGETS | SERIES | BOTH**} ]

        [NETWORKGRAPH = {YES**}([SIGLEVEL = {0.05**}])
                        {NO   }             {value }


        [PRINT = [MODELFIT] [OUTLIERS] [MODELEFFECTS] [PARAMETERS] [FORECASTS] ]
        [PLOT = [SERIESPLOT] [TOPINPUTS] [NRESIDUAL] ]

        [IMPACTDIAGRAMS = CAUSES[({3**    })] EFFECTS[({3**    })] ]
                                  {integer}            {integer}

        [OUTLIER_ROOTCAUSES = {NO**}
                              {YES }([LEVELS = {3**    }] [DISPLAY = {INTERACTIVE**}])
                                             {integer}              {TABLE        }
                                                                    {BOTH         }

    [/SYSTEMOUTPUT
        [MODELFIT = [RSQUARE**] [RMSPE] [RMSE] [BIC] [AIC] [NONE] ]

        [TRANSFORMATIONS = {NO   } ]
                           {YES**}

        [OUTLIERPLOT = {NO**} ]
                       {YES }

    [/SAVE
       [TARGETS({FILTERED**})
                {All       }
       [PREDICTED[(rootname)]]
       [CL[(LCLROOT=rootname UCLROOT=rootname)]]
       [NRESIDUAL[(rootname)]]
       [FORECASTFLAG[(name)]]

    [/OUTFILE
        [MODELSYSTEM = 'filespec']
        [DATA = {'filespec' | dataset}]

    [/PASSPROTECT
        [PASSWORD={'password'|NONE**}]
        [ENCRYPTEDPW={NO**|YES}]

Fields group block subcommands:

    /FIELDSGROUP

    /FIELDS
        [TARGETS = fieldList]
        [INPUTS = fieldList]
        [FORCE = fieldList]
        [EXCLUDEDTARGETS = fieldList]
        [EXCLUDEDINPUTS = fieldList]
        [DIMVALUES = dimField({INCLUDE}=dimValueList) dimField({INCLUDE}=dimValueList) ...]
                              {EXCLUDE}                       {EXCLUDE}
```

**Default if keyword omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Syntax for the TCM MODEL command can be generated from the dialog.

Release History

Release 23.0

- Command introduced.

Example

```
TCM MODEL
   /DATASETTINGS TIMETYPE=DATETIMEVAR TIMEVAR=date INPUTINTERVAL=MONTH
```

```
    /FIELDSGROUP
    /FIELDS TARGETS=ARI TO UNF_ROOM_NTS_YOY INPUTS=ARI TO UNF_ROOM_NTS_YOY
```

Related information

"TCM APPLY" on page 1937
"TCM ANALYSIS" on page 1929

Overview

The TCM MODEL procedure builds temporal causal models. In temporal causal modeling, you specify a set of target series and a set of candidate inputs to those targets. The procedure then builds an autoregressive time series model for each target and includes only those inputs that have a causal relationship with the target. This approach differs from traditional time series modeling where you must explicitly specify the predictors for a target series. Since temporal causal modeling typically involves building models for multiple related time series, the result is referred to as a *model system*.

In the context of temporal causal modeling, the term *causal* refers to Granger causality. A time series X is said to "Granger cause" another time series Y if regressing for Y in terms of past values of both X and Y results in a better model for Y than regressing only on past values of Y.

Data Structures

Temporal causal modeling supports two types of data structures.

Column-based data
For column-based data, each time series field contains the data for a single time series. This structure is the traditional structure of time series data, as used by the Time Series Modeler.

Multidimensional data
For multidimensional data, each time series field contains the data for multiple time series. Separate time series, within a particular field, are then identified by a set of values of categorical fields referred to as *dimension* fields.

For example, sales data for different regions and brands might be stored in a single *sales* field, so that the dimensions in this case are *region* and *brand*. Each combination of *region* and *brand* identifies a particular time series for *sales*. For example, in the following table, the records that have 'north' for *region* and 'brandX' for *brand* define a single time series.

Table 116. Multidimensional data

date	region	brand	sales
01/01/2014	north	brandX	82350
01/01/2014	north	brandY	86380
01/01/2014	south	brandX	91375
01/01/2014	south	brandY	70320
01/02/2014	north	brandX	83275
01/02/2014	north	brandY	85260
01/02/2014	south	brandX	94760
01/02/2014	south	brandY	69870

Basic Specification

The basic specification is the DATASETTINGS subcommand with the TIMETYPE and TIMEVAR keywords, a FIELDSGROUP subcommand, and a FIELDS subcommand with the TARGETS and INPUTS keywords.

- For multidimensional data, the DIMENSIONS keyword on the DATASETTINGS subcommand must be specified.
- If the active dataset has a date specification, then the DATASETTINGS subcommand is not required.
- If the observations are defined by record order, then the TIMEVAR keyword is not included. Record order means that the first record represents the first observation, the second record represents the second observation, and so on.

Syntax Rules

- The following subcommands are global and must appear before any FIELDSGROUP subcommand: DATASETTINGS, DATAFILTER, BUILDOPTIONS, TARGETFILTER, SERIESFILTER, FILTEREDOUTPUT, SYSTEMOUTPUT, SAVE, OUTFILE, and PASSPROTECT.
- Multiple SERIESFILTER subcommands are allowed.
- Multiple FILTEREDOUTPUT subcommands are allowed. If the same value of the INCLUDE keyword appears on multiple FILTEREDOUTPUT subcommands, then the specifications for only the last such subcommand are honored.
- Multiple FIELDSGROUP subcommands are allowed, but at least one is required. The specified set of FIELDSGROUP subcommands defines a single temporal causal model system.
- Multiple FIELDS subcommands can be specified for each FIELDSGROUP subcommand.
- Global subcommands can be specified in any order.
- Each keyword can be specified only once within a subcommand.
- Empty subcommands are not allowed, with the one exception that the FIELDSGROUP subcommand is always empty.
- Parentheses, equals signs, and slashes that are shown in the syntax chart are required.
- The command name, subcommand names, and keywords must be spelled in full.

Operations

- The procedure honors the USE command but it also has settings for specifying the estimation period. Settings on the ESTIMATIONPERIOD keyword are applied after any case selection that is specified with the USE command.
- The procedure does not honor the PREDICT command. The TCM MODEL procedure provides an option to specify the forecast period.
- The procedure does not honor the TSET command. The TCM MODEL procedure provides options for handling missing values.
- The procedure does not honor the following commands: MODEL NAME or SAVE MODEL. The TCM MODEL procedure provides the option to save the model system.
- The TDISPLAY command does not display model systems that are saved from TCM MODEL.

Limitations

- The SPLIT FILE setting is ignored by this procedure.
- Case weighting is ignored by this procedure.

DATASETTINGS subcommand

The DATASETTINGS subcommand specifies settings that are used to prepare the data for the analysis. It includes the following settings:

- The dimension fields that define each time series for multidimensional data.
- The field, or fields, that define the observations, the time interval between observations, and the time interval for the analysis.

- The method by which data are aggregated or distributed when the time interval for the analysis differs from the time interval between the observations.
- Handling of missing values.

DIMENSIONS
Specifies the dimension fields, if any, that identify each time series. Specify one or more field names. The keyword T0 is supported but the keyword ALL is not supported.

MAXDIMVALUES
Specifies the maximum number of distinct values per dimension field. The default is 10000. The value must be an integer that is greater than 100.

TIMETYPE
Specifies how the time of each observation is defined. The keyword is required unless the active dataset has a date specification, in which case it cannot be specified. Date specifications are created with the DATE command.

DATETIMEVAR
Specifies that a field with a date, datetime, or time (duration) format defines the observations.

ISO8601
Specifies that the observations are defined by a string variable that represents dates or times (durations). Dates are specified in the form yyyy-mm-dd. Datetimes are specified in the form yyyy-mm-dd hh:mm:ss. Times are specified in the form hh:mm:ss. Two digits are not required for month, day, hours, minutes or seconds specifications. For example, 2014-1-1 is interpreted as 2014-01-01 and 3:2:1 is interpreted as 03:02:01.

CYCLICPERIODS
Specifies that the observations are defined by one or more integer fields that define a hierarchy of periodic levels. The field that defines the lowest level is referred to as the *period* field. Fields that define higher levels are referred to as *cycle* fields.

With this structure, you can describe series of observations that don't fit one of the standard time intervals. For example, a fiscal year with only 10 months can be described with a cycle field that represents years and a period field that represents months, where the length of one cycle is 10.

Field values for each level, except the highest, must be periodic with respect to the next highest level. Values for the highest level cannot be periodic. For example, in the case of the 10-month fiscal year, months are periodic within years and years are not periodic.

NONE
Specifies that the observations are defined by record order so that the first record represents the first observation, the second record represents the second observation, and so on. It is then assumed that the records represent observations that are equally spaced in time. The value NONE cannot be used when the DIMENSIONS keyword is specified.

TIMEVAR
Specifies the field, or fields, that define each observation. It is required when TIMETYPE=DATETIMEVAR, TIMETYPE=ISO8601, or TIMETYPE=CYCLICPERIODS.

var
Specifies the date, datetime, or time field that defines the observations when TIMETYPE=DATETIMEVAR. It also specifies the string field that defines the observations when TIMETYPE=ISO8601. Specify a field name for var.

var(TYPE=PERIOD [INCREMENT=integer] [START=integer])
Specifies the period field when TIMETYPE=CYCLICPERIODS. Specify a field name for var.

- The optional INCREMENT keyword specifies the integer increment between successive observations. The default is 1.
- The optional START keyword specifies the starting value when there are one or more cycle fields. The value must be a positive integer. The default is 1. For example, a field with the periodic sequence 2,3,4,5 has a starting value of 2. START is ignored if there are no cycle fields.

var(TYPE=CYCLE LEVEL=integer LENGTH=integer [START=integer])
Specifies a cycle field when TIMETYPE=CYCLICPERIODS. Specify a field name for var. An arbitrary number of cycle fields can be specified.

- The LEVEL keyword is required and must specify a positive integer. The lowest level cycle field must have LEVEL=1. The next cycle field in the hierarchy must have LEVEL=2, and so on.
- The LENGTH keyword is required and specifies the length of the cycle. The length of a cycle at a particular level is the periodicity of the next lowest level. Consider the example of a 10-month fiscal year that is described by a period field for month and a first level cycle field for year. The cycle field for year has a length of 10 since the next lowest level represents months and there are 10 months in the specified fiscal year. The value of LENGTH must be a positive integer.
- The optional START keyword specifies the starting value when the cycle field is periodic. The value must be a positive integer, and the default is 1. This setting is necessary for detecting missing values. For example, if a periodic field starts from 2 but the starting value is specified as 1, then the procedure assumes that there is a missing value for the first period in each cycle of that field. START is ignored for the highest level cycle field.

INPUTINTERVAL
Specifies the interval between successive observations and any additional information needed to specify the sequence of observations. The keyword can be specified only when TIMETYPE=DATETIMEVAR or TIMETYPE=ISO8601.

AUTODETECT
Specifies that the interval between successive observations will be automatically determined from the data. This is the default.

IRREGULAR
Specifies that successive observations are not equally spaced, such as observations that represent the time at which a sales order is processed. When IRREGULAR is specified, you must specify ANALYSISINTERVAL.

YEAR
Specifies that the interval between successive observations is one or more years. Use the INCREMENT keyword to specify the number of years between successive observations. The default is 1.

QUARTER
Specifies that the interval between successive observations is one quarter.

MONTH
Specifies that the interval between successive observations is one or more months. Use the INCREMENT keyword to specify the number of months between successive observations. The default is 1.

WEEK
Specifies that the interval between successive observations is one week.

DAY
Specifies that the interval between successive observations is one or more days.

- Use the WKSTART keyword to specify the start of the week. The default is SUN, which specifies Sunday.
- Use the DAYSWK keyword to specify the number of days per week. Specify an integer between 1 and 7. The default is 7.
- Use the INCREMENT keyword to specify the number of days between successive observations. The default is 1.

HOUR
Specifies that the interval between successive observations is one or more hours and that observations are represented as a datetime.

- Use the WKSTART keyword to specify the start of the week. The default is SUN, which specifies Sunday.

- Use the DAYSWK keyword to specify the number of days per week. Specify an integer between 1 and 7. The default is 7.
- Use the HRSDAY keyword to specify the number of hours per day. Specify an integer between 1 and 24. The default is 24.
- Use the DAYSTART keyword to specify the starting hour of the day. Specify an integer between 0 and 23. The default is 0.
- Use the INCREMENT keyword to specify the number of hours between successive observations. The default is 1.

MINUTE
Specifies that the interval between successive observations is one or more minutes and that observations are represented as a datetime.

- Use the WKSTART keyword to specify the start of the week. The default is SUN, which specifies Sunday.
- Use the DAYSWK keyword to specify the number of days per week. Specify an integer between 1 and 7. The default is 7.
- Use the HRSDAY keyword to specify the number of hours per day. Specify an integer between 1 and 24. The default is 24.
- Use the DAYSTART keyword to specify the starting hour of the day. Specify an integer between 0 and 23. The default is 0.
- Use the INCREMENT keyword to specify the number of minutes between successive observations. The default is 1.

SECOND
Specifies that the interval between successive observations is one or more seconds and that observations are represented as a datetime.

- Use the WKSTART keyword to specify the start of the week. The default is SUN, which specifies Sunday.
- Use the DAYSWK keyword to specify the number of days per week. Specify an integer between 1 and 7. The default is 7.
- Use the HRSDAY keyword to specify the number of hours per day. Specify an integer between 1 and 24. The default is 24.
- Use the DAYSTART keyword to specify the starting hour of the day. Specify an integer between 0 and 23. The default is 0.
- Use the INCREMENT keyword to specify the number of seconds between successive observations. The default is 1.

DURATION
Specifies that observations are represented by a field with a time (duration) format, as in hh:mm:ss.

- The INTERVAL keyword specifies the interval between observations and is required.
- Use the INCREMENT keyword to specify the number of hours, minutes, or seconds between observations. The default is 1.

ANALYSISINTERVAL
Specifies the time interval for the analysis. For example, if the time interval of the observations is Days, you might choose Months for the time interval for analysis. The data are then aggregated from daily to monthly data before the model is built. You can also choose to distribute the data from a longer to a shorter time interval. For example, if the observations are quarterly then you can distribute the data from quarterly to monthly data.

- USEINPUT specifies that the time interval for analysis is the same as the interval between the observations. It is the default unless INPUTINTERVAL=IRREGULAR. If INPUTINTERVAL=IRREGULAR, ANALYSISINTERVAL must be specified and it cannot be specified as USEINPUT.

- CYCLE(LEVEL=integer) applies when TIMETYPE=CYCLICPERIODS and there are one or more cycle fields (in addition to the period field). It specifies to aggregate the data to the cycle level specified by the LEVEL keyword. The allowed values for LEVEL are 1 to the number of cycle fields.

  Note: When TIMETYPE=CYCLICPERIODS, only CYCLE(LEVEL=integer) or USEINPUT can be specified for ANALYSISINTERVAL.

- When TIMETYPE=DATETIMEVAR or TIMETYPE=ISO8601, the following rules apply:

  – ANALYSISINTERVAL can be set to any value that is longer than, or equal to, the value of INPUTINTERVAL. For example, if INPUTINTERVAL=QUARTER then you can set ANALYSISINTERVAL to QUARTER or YEAR. The one exception is that when INPUTINTERVAL=WEEK, ANALYSISINTERVAL cannot be set to MONTH or QUARTER.

  – ANALYSISINTERVAL can be set to certain values that are shorter than the value of INPUTINTERVAL. The values of INPUTINTERVAL, followed by the allowed values of ANALYSISINTERVAL for this case, are as follows:

    - YEAR: QUARTER, MONTH
    - QUARTER: MONTH
    - MONTH: DAY
    - WEEK: DAY
    - DAY: HOUR
    - HOUR or DURATION(INTERVAL=HOUR): MINUTE
    - MINUTE or DURATION(INTERVAL=MINUTE): SECOND

    Note: If INCREMENT > 1 (for INPUTINTERVAL) then ANALYSISINTERVAL cannot be shorter than INPUTINTERVAL.

- When the active dataset has a date specification, aggregation is supported but distribution is not supported. For example, if the date specification is YEAR, QUARTER then ANALYSISINTERVAL can be set to YEAR but it cannot be set to MONTH.

  – You cannot aggregate from MONTH to QUARTER for a date specification.

  – If the date specification has a cycle field, then CYCLE(LEVEL=1) can be specified for ANALYSISINTERVAL.

GROUP

Specifies the method to use when multiple observations occur in the same time interval. For example, if INPUTINTERVAL is MONTH then multiple dates in the same month are grouped together. The specification for GROUP applies to all time series fields. You can, however, override that specification for particular fields. For example, to group values of *temperature* using the mean of the values, specify temperature(MEAN). The following methods are available:

SUM
The sum of the original values. This is the default.

MEAN
The mean of the original values.

MODE
The mode of the original values.

MIN
The minimum of the original values.

MAX
The maximum of the original values.

Note: Although grouping is a form of aggregation, it is done before any handling of missing values whereas formal aggregation is done after any missing values are handled. When the time interval of the observations is specified as Irregular, aggregation is done only with the grouping function.

AGGREGATE
Specifies the method to be used when the data are aggregated. Aggregation is done when the value for ANALYSISINTERVAL is longer than the value for INPUTINTERVAL; for example, when ANALYSISINTERVAL is YEAR and INPUTINTERVAL is MONTH. The specification for AGGREGATE applies to all time series fields. You can, however, override that specification for particular fields. For example, to aggregate values of *temperature* using the mean of the values, specify `temperature(MEAN)`. The available methods are the same as for the GROUP keyword.

DISTRIBUTE
Specifies the method to be used when data are distributed. Distribution is done when the value for ANALYSISINTERVAL is shorter than the value for INPUTINTERVAL; for example, when ANALYSISINTERVAL is QUARTER and INPUTINTERVAL is YEAR. The specification for DISTRIBUTE applies to all time series fields. You can, however, override that specification for particular fields. For example, to distribute values of *temperature* using the mean of the values, specify `temperature(MEAN)`.

INTERPOLATE
Specifies the method to be used for interpolating missing values. The following options are available:

LINT
Replaces missing values by using a linear interpolation. The last valid value before the missing value and the first valid value after the missing value are used for the interpolation. If the first or last observation in the series has a missing value, then the two nearest non-missing values at the beginning or end of the series are used. This option is the default.

MEAN
Replaces missing values with the mean for the entire series.

KMEAN(integer)
Missing values are replaced by the mean of the specified number of nearest preceding and subsequent values. The default is 2.

KMEDIAN(integer)
Missing values are replaced by the median of the specified number of nearest preceding and subsequent observations. The default is 2.

LTREND
This option uses all non-missing observations in the series to fit a simple linear regression model, which is then used to impute the missing values.

USERMISSING
Specifies whether user-missing values are treated as valid values.

EXCLUDE
Specifies that user-missing values are treated as invalid. This is the default.

INCLUDE
Specifies that user-missing values are treated as valid.

MAXMISSING
Specifies the maximum percentage of missing values that are allowed for any series. Series with more missing values than the specified maximum are excluded from the analysis. The value is specified as a percentage and the default is 25.

CROSSDAYHRS
Specifies whether observations with times that cross a day boundary are aggregated to the values for the previous day. For example, for hourly observations with an eight-hour day that starts at 20:00, this setting specifies whether observations between 00:00 and 04:00 are included in the aggregated results for the previous day. CROSSDAYHRS only applies when INPUTINTERVAL is HOUR, MINUTE, or SECOND and ANALYSISINTERVAL is DAY.

EXCLUDE
Specifies that observations that cross a day boundary are not aggregated to the results for the previous day. This is the default.

INCLUDE
Specifies that observations that cross a day boundary are aggregated to the results for the previous day.

DATAFILTER subcommand

The DATAFILTER subcommand defines filter criteria for specifying which dimension values are included in the analysis. It applies only to multidimensional data. The specified filter applies to all time series that are included in the analysis. For example, if there are dimension fields for *region* and *brand*, then you might choose to limit the analysis to particular regions but include all brands for those regions.

DIMVALUES
The DIMVALUES keyword specifies the filter. The form of the specification for a particular dimension field is `dimField(INCLUDE|EXCLUDE=dimValueList)`, where `dimField` is the name of the field and `dimValueList` is a list of values for that field.

- The INCLUDE or EXCLUDE keyword is required.
- INCLUDE specifies a list of values, from the associated dimension field, to include.
- EXCLUDE specifies a list of values, from the associated dimension field, to exclude.
- You can specify included or excluded values for one or more dimension fields. For dimension fields that are not specified on DIMVALUES, all values of the associated field are included.
- Fields that are specified on the DIMVALUES keyword do not support ALL and TO.
- String values do not need to be quoted, unless the string contains apostrophes or quotation marks. If the string contains apostrophes or single quotes, then enclose the string in double quotes. If the string contains double quotes, then enclose the string in single quotes.

BUILDOPTIONS subcommand

ESTIMATIONPERIOD
Specifies the estimation period that defines the set of observations that are used for the analysis.

ALL
Specifies that all observations are used. This setting is the default. When ALL is specified, the estimation period starts at the time of the earliest observation and ends at the time of the latest observation across all series.

TIMES
Specifies a range of observations by start and end time.

- If START is specified without END, then all observations from START to the last observation (across all series) are used. If END is specified without START, then all observations from the first observation (across all series) through END are used.
- The FORMAT keyword specifies the format of the values for START and END. It is required when TIMETYPE=DATETIMEVAR or TIMETYPE=ISO8601, and is ignored otherwise. Values for START and END must be quoted in this case.

  VARIABLE
  Specifies to use the variable format of the field that is specified on the TIMEVAR keyword; for example, mm/dd/yyyy. This option only applies when TIMETYPE=DATETIMEVAR.

  ISO8601
  Specifies that values for START and END are in the general form of yyyy-mm-dd hh:mm:ss. FORMAT=ISO8601 must be specified when TIMETYPE=ISO8601.

  – Dates are specified in the form yyyy-mm-dd. Datetimes are specified in the form yyyy-mm-dd hh:mm:ss. Times are specified in the form hh:mm:ss.

  – Two digits are not required for month, day, hours, minutes or seconds specifications. For example, 2014-1-1 is interpreted as 2014-01-01 and 3:2:1 is interpreted as 03:02:01.

– Values can be specified with reduced precision. For example, 2014-10 is interpreted as 2014-10-01 and 03:02 is interpreted as 03:02:00.

- For TIMETYPE=CYCLICPERIODS, START and END are lists of integers, consisting of one value for each field that is specified by the TIMEVAR keyword. The first value is the value for the highest level cycle field, followed by the next highest cycle field, and so on.

  For example, the following syntax specifies the start and end time when there are two cycle levels:

  ```
  START = 3 6 2  END = 5 1 4
  ```

  The start of the estimation period is defined by a value of 3 for the second-level cycle, a value of 6 for the first-level cycle, and a value of 2 for the period field. The end of the estimation period is defined by a value of 5 for the second- level cycle, a value of 1 for the first-level cycle, and a value of 4 for the period field.
- For TIMETYPE=NONE, START and END consist of a single value that specifies the starting record number and the ending record number. For example, the following syntax specifies an estimation period that starts at record 1 and ends at record 60:

  ```
  START = 1  END = 60
  ```
- If a date specification exists for the active dataset, then START and END are lists of integers that consist of one value for each of the components of the date specification. For example, if the date specification is YEAR and MONTH, then the lists consist of two items, one for the year and one for the month. The order of the values is from the highest level component to the lowest level component. For example, the following syntax specifies start and end time for a date specification that is defined by YEAR and MONTH:

  ```
  START = 2008 1  END = 2013 12
  ```

  The start of the estimation period is defined by a value of 2008 for the year and 1 for the month. The end of the estimation period is defined by a value of 2013 for the year and 12 for the month.

OBSERVATIONS
Defines the estimation period as a specified number of time intervals that start at the earliest time interval or end at the latest time interval in the data, with an optional offset. In this context, the time interval refers to the time interval of the analysis.

TYPE
TYPE=NEWEST specifies to use the most recent time intervals. TYPE=OLDEST specifies to use the earliest time intervals.

OFFSET
Specifies an offset from the earliest time interval (when TYPE=OLDEST) or the most recent time interval (when TYPE=NEWEST). The value of OFFSET must be an integer greater than or equal to 0. The default is 0 and specifies to start at the earliest time interval or end at the most recent time interval.

NUMBER
Specifies the number of time intervals to include. The value must be an integer that is greater than or equal to 2. The default is 1000. If the specified value exceeds the number of time intervals in the data, then all time intervals, starting from the optional offset, are included.

FORECASTPERIODS
Specifies the number of time intervals to forecast beyond the end of the estimation period. The time interval in this case is the time interval of the analysis, which is specified by the ANALYSISINTERVAL keyword. The default value of FORECASTPERIODS is 0, which means that forecasts are not created.

CILEVEL
Specifies the confidence level for computing confidence intervals for forecasts and parameter estimates. The value must be positive number that is greater than 0 and less than 100. The default is 95.

MAXINPUTS
The MAXINPUTS keyword specifies the maximum number of inputs that can be included in the model for any target.

- The minimum value is 1 and the maximum value is 20.
- The default value, which is specified by DEFAULT, is the lesser of 5 or the total number of series.
- The model for each target always includes lagged values of itself, so setting this value to 1 specifies that the only input is the target itself.

LAGS
Specifies the number of lag terms that are included for each time series in the system. The minimum value is 1 and the maximum value is 20. The default is AUTO, which automatically determines the number of lags from the time interval that is used for the analysis. For example, if the time interval is months (with an increment of one month) then the number of lags is 12.

TOLERANCE
This setting controls the iterative process that is used for determining the best set of inputs for each target. You can specify any value that is greater than zero. The default is 0.001. Model tolerance is a stop criterion for predictor selection. It can affect the number of predictors that are included in the final model. But if a target can predict itself very well, other predictors may not be included in the final model. Some trial and error may be required (for example, if you have this value set to high, you can try setting it to a smaller value to see if other predictors can be included or not).

OUTLIERTHRESH
An observation is flagged as an outlier if the probability, as calculated from the model, that it is an outlier exceeds this threshold. You can specify a value in the range 50 - 100. The default value is 95.

TARGETFILTER subcommand

The TARGETFILTER subcommand specifies a set of target series for which output is displayed. The content of the output for the specified series is determined by the FILTEREDOUTPUT subcommand. You can specify particular series, for which output is displayed, from the SERIESFILTER subcommand.

BESTFIT
Specifies to generate output for the target series that have the best-fitting models. By default, output is generated for the top 10 models according to the value of the root mean square percentage error.

- The N keyword specifies the number of models to include, as an integer. If the count exceeds the number of models, all models are shown. The default is 10.
- The PCT keyword specifies the number of models to include, as a percentage of the total number of models. The percentage value must be an integer between 1 and 100.
- The FITSTAT keyword specifies the fit statistic that is used to determine the best-fitting models. The default is RSQUARE.

  RSQUARE
  Goodness-of-fit measure of a linear model, sometimes called the coefficient of determination. It is the proportion of variation in the target variable explained by the model. It ranges in value from 0 to 1. Small values indicate that the model does not fit the data well.

  RMSPE
  Root Mean Square Percentage Error. A measure of how much the model-predicted values differ from the observed values of the series. It is independent of the units that are used and can therefore be used to compare series with different units.

  RMSE
  Root Mean Square Error. The square root of mean square error. A measure of how much a dependent series varies from its model-predicted level, expressed in the same units as the dependent series.

BIC
Bayesian Information Criterion. A measure for selecting and comparing models based on the -2 reduced log likelihood. Smaller values indicate better models. The BIC also "penalizes" overparameterized models (complex models with a large number of inputs, for example), but more strictly than the AIC.

AIC
Akaike Information Criterion. A measure for selecting and comparing models based on the -2 reduced log likelihood. Smaller values indicate better models. The AIC "penalizes" overparameterized models (complex models with a large number of inputs, for example).

ALLTARGETS
Specifies that output is generated for all target series.

TARGETSNOINPUTS
Specifies that only target series with no inputs (other than lagged values of the target itself) are included.

SERIESFILTER subcommand

The SERIESFILTER subcommand specifies particular series for which output is displayed. The content of the output for the specified series is determined by the FILTEREDOUTPUT subcommand. You can specify multiple SERIESFILTER subcommands.

FIELDS
The FIELDS keyword is required and specifies the list of fields that contain the series that you want. The keyword TO is supported but the keyword ALL is not supported. For multidimensional data, all series that are contained in each specified field are included by default. You can specify a subset of series by specifying particular sets of values for one or more dimension fields on the DIMVALUES keyword.

DIMVALUES
Specifies optional lists of dimension field values that define a subset of series for the fields that are specified on the FIELDS keyword. The DIMVALUES keyword applies only to multidimensional data. The form of the specification for a particular dimension field is `dimField(INCLUDE|EXCLUDE=dimValueList)`, where `dimField` is the name of the field and `dimValueList` is a list of values for that field.

- The INCLUDE or EXCLUDE keyword is required.
- INCLUDE specifies a list of values, from the associated dimension field, to include.
- EXCLUDE specifies a list of values, from the associated dimension field, to exclude.
- You can specify included or excluded values for one or more dimension fields. For dimension fields that are not specified on DIMVALUES, all values of the associated field are included.
- Fields that are specified on the DIMVALUES keyword do not support ALL and TO.
- String values do not need to be quoted, unless the string contains apostrophes or quotation marks. If the string contains apostrophes or single quotes, then enclose the string in double quotes. If the string contains double quotes, then enclose the string in single quotes.

FILTEREDOUTPUT subcommand

The FILTEREDOUTPUT subcommand controls the output that is displayed for the series that are specified on the TARGETFILTER and SERIESFILTER subcommands.

INCLUDE
Specifies the set of series to which the current FILTEREDOUTPUT subcommand applies. You can specify up to three FILTEREDOUTPUT subcommands, one for each of the values of INCLUDE.

- TARGETS specifies that the current settings apply to the targets that are specified by the TARGETFILTER subcommand.
- SERIES specifies that the current settings apply to the series that are specified by the SERIESFILTER subcommands.
- BOTH specifies that the current settings apply to the targets that are specified by the TARGETFILTER subcommand and the series that are specified by the SERIESFILTER subcommands.

NETWORKGRAPH

Specifies whether the Overall Model System output item is generated. The default is YES. The Overall Model System item displays a graphical representation of the causal relations between series in the model system. Tables of both model fit statistics and outliers for the displayed targets are included as part of the output item. When this option is specified with INCLUDE=SERIES or INCLUDE=BOTH, a separate output item is created for each series that is specified on a SERIESFILTER subcommand.

- The SIGLEVEL keyword specifies the significance level threshold for hiding causal connections between series in the Overall Model System graph. A smaller significance level indicates a more significant connection. Connections with significance levels that are higher than the threshold are hidden. Specify a numeric value in the range 0-1. The default is 0.05.

PRINT

Controls the display of tabular output. One or more of the following keyword values can be specified. By default, none of this tabular output is displayed.

MODELFIT

Tables of model fit statistics for the target series that are selected for display. The tables contain the same information as the model fit statistics table in the Overall Model System item. The tables support all standard features for pivoting and editing tables.

OUTLIERS

Tables of outliers for the target series that are selected for display. The tables contain the same information as the outliers table in the Overall Model System item. The tables support all standard features for pivoting and editing tables.

MODELEFFECTS

Tables of model effects tests for the target series that are selected for display. Model effects tests include the F statistic and associated significance value for each input included in the model.

PARAMETERS

Tables of model parameters for the target series that are selected for display.

FORECASTS

Tables of forecasted values and confidence intervals of those forecasts for the target series that are selected for display. The FORECASTS keyword is ignored if forecasts are not requested. Forecasts are requested with the FORECASTPERIODS keyword on the BUILDOPTIONS subcommand.

PLOT

Controls the display of chart output. One or more of the following keyword values can be specified. By default, none of this chart output is displayed.

SERIESPLOT

Plots of observed and predicted values for the target series that are selected for display. When forecasts are requested, the plot also shows the forecasted values and the confidence intervals for the forecasts.

TOPINPUTS

Plots of each displayed target, over time, along with the top 3 inputs for the target. The top inputs are the inputs with the lowest significance value. To accommodate different scales for the inputs and target, the y axis represents the z score for each series.

NRESIDUAL

Plots of the model residuals for the target series that are selected for display.

IMPACTDIAGRAMS
Specifies whether impact diagrams are generated. An impact diagram displays a graphical representation of the causal relations between a series of interest and other series that it affects or that affect it. Series that affect the series of interest are referred to as *causes*. When the IMPACTDIAGRAMS keyword is specified, impact diagrams are generated for all series that are selected for display. By default, impact diagrams are not generated.

EFFECTS
Creates an impact diagram that is initialized to show the series that are affected by the series of interest. You can specify the number of levels of effects to display, as in EFFECTS(4) for four levels of effects. The first level represents the series of interest. Each additional level shows more indirect effects of the series of interest. For example, the third level consists of the series that contain series in the second level as a direct input. Series in the third level are then indirectly affected by the series of interest since the series of interest is a direct input to the series in the second level. By default, 3 levels of effects are displayed. You can specify any number of levels up to a maximum of 99.

CAUSES
Creates an impact diagram that is initialized to show the series that affect the series of interest. You can specify the number of levels of causes to display, as in CAUSES(4) for four levels of causes. The concept of levels for causes is the same as for levels of effects. By default, 3 levels of causes are displayed. You can specify any number of levels up to a maximum of 99.

Note: You can interactively toggle between causes and effects in the output item that displays the impact diagram.

OUTLIER_ROOTCAUSES
Specifies whether outlier root cause analysis is done. Outlier root cause analysis determines which series are most likely to be the cause of each outlier in a series of interest. The analysis is done for each series that is specified on a SERIESFILTER subcommand and that has outliers. By default, outlier root cause analysis is not done.

LEVELS
Specifies the number of causal levels that are searched for root causes of outliers. By default, 3 causal levels are searched. The maximum number of levels is 5. The concept of levels that is used here is the same as described for impact diagrams.

DISPLAY

- The value INTERACTIVE creates an interactive visualization that contains a table and chart of outliers and root causes of those outliers for each series of interest. The table contains a single row for each outlier. The chart is an impact diagram. Selecting a row in the table highlights the path, in the impact diagram, from the series of interest to the series that most likely causes the associated outlier.
- The value TABLE creates a table of outliers and root causes of those outliers for each series of interest. This table contains the same information as the table in the interactive display. This table supports all standard features for pivoting and editing tables.
- The value BOTH generates the interactive visualization and the separate table.

SYSTEMOUTPUT subcommand

The SYSTEMOUTPUT subcommand controls output associated with the entire temporal causal model system.

MODELFIT
Controls the display of histograms of model fit for all models. One or more fit statistics can be specified. A single histogram that is based on RSQUARE is generated by default.

RSQUARE
Goodness-of-fit measure of a linear model, sometimes called the coefficient of determination. It is the proportion of variation in the target variable explained by the model. It ranges in value from 0 to 1. Small values indicate that the model does not fit the data well.

RMSPE
Root Mean Square Percentage Error. A measure of how much the model-predicted values differ from the observed values of the series. It is independent of the units that are used and can therefore be used to compare series with different units.

RMSE
Root Mean Square Error. The square root of mean square error. A measure of how much a dependent series varies from its model-predicted level, expressed in the same units as the dependent series.

BIC
Bayesian Information Criterion. A measure for selecting and comparing models based on the -2 log likelihood. Smaller values indicate better models. The BIC also "penalizes" overparameterized models (complex models with a large number of inputs, for example), but more strictly than the AIC.

AIC
Akaike Information Criterion. A measure for selecting and comparing models based on the -2 log likelihood. Smaller values indicate better models. The AIC "penalizes" overparameterized models (complex models with a large number of inputs, for example).

NONE
Specifies that histograms are not generated. NONE cannot be specified with any other keywords.

TRANSFORMATIONS
Table of any transformations that were applied to the series in the model system. The possible transformations are missing value imputation, aggregation, and distribution. The default is YES, which specifies that the table is generated.

OUTLIERPLOT
Bar chart of the number of outliers, across all targets, for each time interval in the estimation period. The default is NO, which specifies that the bar chart is not generated.

SAVE subcommand

The SAVE subcommand saves new variables that represent predicted values, residuals, and confidence intervals. By default, no new variables are saved.

- Specify one or more keywords, along with optional rootnames to be used as the prefix for new variable names.
- The rootname, if specified, must conform to the rules for valid variable names.
- If the DATA keyword of the OUTFILE subcommand is specified, then the new variables are written to that location. If the DATA keyword is not specified and the new variables cannot be written to the active dataset, then they are written to an autogenerated dataset. New variables can be written to the active dataset only if the observations are defined by a date specification, or record order, and the data are not aggregated.

TARGETS

ALL
Specifies that new variables are created for all targets in the model system.

FILTERED
Specifies that new variables are created just for the targets that are specified by the TARGETFILTER and SERIESFILTER subcommands. This option is the default.

PREDICTED(rootname)
Saves model fit and forecast values. The default rootname is *Predicted*.

CL(LCLROOT=rootname UCLROOT=rootname)
Saves upper and lower confidence limits. LCLROOT specifies the root name for the lower confidence limit and UCLROOT specifies the root name for the upper confidence limit. The default root name for the lower confidence limit is *LCL* and the default root name for the upper confidence limit is *UCL*.

NRESIDUAL(rootname)
Saves noise residuals. The default rootname is *NResidual*.

FORECASTFLAG(name)
Creates a variable that indicates whether a record contains forecast data. You can specify the variable name. The default is *ForecastIndicator*.

OUTFILE subcommand

MODELSYSTEM
Saves the model system to a compressed file archive (.zip file). The model system file can be used by the Temporal Causal Model Forecasting procedure to obtain updated forecasts or to generate any of the available output. It can also be used by the Temporal Causal Model Scenarios procedure to run scenario analysis.

DATA
Specifies the name of a file or a previously declared dataset where the data that was used for modeling, and any new variables that are specified on the SAVE subcommand, are written. The data that is used for modeling includes any aggregation or distribution that was specified for the original data, and any imputation of missing values.

- Date/time values in the saved data are aligned to the beginning of each time interval, such as the first of the month, and represent the time interval of the analysis for the model system.
- Enclose file specifications in quotation marks and use fully qualified paths to avoid ambiguity. You can use an asterisk (*) to specify that the data is written to the active dataset, replacing the current active dataset.

PASSPROTECT subcommand

The PASSPROTECT subcommand saves an external file, that was specified on the DATA keyword of the OUTFILE subcommand, as an encrypted file. An encrypted file can be opened only by providing the correct password.

PASSWORD
Specifies the password that is required to open this file. Passwords are limited to 10 characters and are case-sensitive. The specified value must be enclosed in quotation marks. The keyword NONE specifies not to encrypt the file and is the default.

ENCRYPTEDPW
Specifies whether the password is encrypted.

NO
The password is not encrypted. It is treated as plain text. This option is the default.

YES
The password is encrypted. Use ENCRYPTEDPW=YES only when the password is known to be encrypted. Passwords are always encrypted in pasted syntax.

Creating strong passwords

- Use eight or more characters.
- Include numbers, symbols and even punctuation in your password.
- Avoid sequences of numbers or characters, such as "123" and "abc", and avoid repetition, such as "111aaa".
- Do not create passwords that use personal information such as birthdays or nicknames.

- Periodically change the password.

Note: Encrypted data files and output documents cannot be opened in versions of IBM SPSS Statistics prior to version 21. Encrypted syntax files cannot be opened in versions prior to version 22.

FIELDSGROUP subcommand

The FIELDSGROUP subcommand is required and signals the start of a fields group block. A fields group block consists of one or more FIELDS subcommands. Multiple FIELDSGROUP subcommands can be specified. The set of all FIELDSGROUP subcommands defines the complete set of fields, and their roles, for the temporal causal model system.

- At least one FIELDSGROUP subcommand must be specified.
- At least one FIELDS subcommand in each fields group block must contain a TARGETS keyword.

FIELDS subcommand

The FIELDS subcommand specifies targets and inputs for the current fields group block. Fields group blocks are created with the FIELDSGROUP subcommand

- All fields on the FIELDS subcommand, except those specified on the DIMVALUES keyword, must be numeric.
- Field lists that use the keyword ALL include all numeric fields in the active dataset with the following exceptions: fields specified on the TIMEVAR keyword (on the DATASETTINGS subcommand); fields created from the DATE command.
- For multidimensional data, all series that are contained in each specified field are included by default. You can specify a subset of series by specifying particular sets of values for one or more dimension fields on the DIMVALUES keyword.

TARGETS
Specifies target fields for the current fields group block.

INPUTS
Specifies inputs that are considered for inclusion in the model for each target in the current fields group block.

- Fields specified on the INPUTS keyword can also be specified on the TARGETS keyword.
- The model for each target automatically contains lagged values of the target itself as an input.

FORCE
Specifies fields that are always included as inputs in the model for each target in the current fields group block. Specifications on the FORCE keyword override those on the INPUTS and EXCLUDEDINPUTS keywords in the current fields group block.

EXCLUDEDTARGETS
Specifies fields that are excluded from the specified targets in the current fields group block. The EXCLUDEDTARGETS keyword is most useful when ALL or TO is used on the TARGETS keyword.

EXCLUDEDINPUTS
Specifies fields that are excluded from the specified inputs in the current fields group block. The EXCLUDEDINPUTS keyword is most useful when ALL or TO is used on the INPUTS keyword.

DIMVALUES
Specifies optional lists of dimension field values that define a subset of series for fields specified in all keywords on the current FIELDS subcommand. The DIMVALUES keyword applies only to multidimensional data.

- The INCLUDE or EXCLUDE keyword is required.
- INCLUDE specifies a list of values, from the associated dimension field, to include.
- EXCLUDE specifies a list of values, from the associated dimension field, to exclude.

- You can specify included or excluded values for one or more dimension fields. For dimension fields that are not specified on DIMVALUES, all values of the associated field are included.
- Fields that are specified on the DIMVALUES keyword do not support ALL and TO.
- String values do not need to be quoted, unless the string contains apostrophes or quotation marks. If the string contains apostrophes or single quotes, then enclose the string in double quotes. If the string contains double quotes, then enclose the string in single quotes.

TDISPLAY

```
TDISPLAY  [{ALL        }]
          {model names  }
          {command names}

[/TYPE={MODEL**}]
       {COMMAND}
```

**Default if the subcommand or keyword is omitted.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
TDISPLAY.
```

Overview

TDISPLAY displays information about currently active models. These models are automatically generated by a number of procedures for use with the APPLY subcommand.

Options

If models are specified on TDISPLAY, information about only those models is displayed. You can use the TYPE subcommand to control whether models are specified by model name or by the name of the procedure that generated them.

Basic Specification

The basic specification is simply the command keyword TDISPLAY.

- By default, TDISPLAY produces a list of all currently active models. The list includes the model names, the commands that created each model, model labels (if specified), and creation dates and times.

Syntax Rules

- To display information about a subset of active models, specify those models after TDISPLAY.
- Models can be specified by using individual model names or the names of the procedures that created them. To use procedure names, you must specify the TYPE subcommand with the keyword COMMAND.
- Model names are either the default *MOD_n* names or the names that are assigned with MODEL NAME.
- If procedure names are specified, all models that are created by those procedures are displayed.
- Model names and procedure names cannot be mixed on the same TDISPLAY command.
- You can specify the keyword ALL after TDISPLAY to display all models that are currently active. This setting is the default.

Operations

- Only currently active models are displayed.
- The following procedures can generate models that can be displayed with the TDISPLAY command: AREG, ARIMA, EXSMOOTH, SEASON, and SPECTRA in the Forecasting add-on module; ACF, CASEPLOT, CCF, CURVEFIT, PACF, PPLOT, and TSPLOT in the Core system; and WLS and 2SLS in the Regression add-on module.

TYPE Subcommand

TYPE indicates whether models are specified by model name or procedure name.

- One keyword, MODEL or COMMAND, can be specified after TYPE.
- MODEL is the default and indicates that models are specified as model names.
- COMMAND specifies that models are specified by procedure name.
- TYPE has no effect if model names or command names are not listed after TDISPLAY.
- If more than one TYPE subcommand is specified, only the last subcommand is used.
- The TYPE specification applies only to the current TDISPLAY command.

Example

```
TDISPLAY ACF CURVEFIT
  /TYPE=COMMAND.
```

- This command displays all currently active models that were created by procedures ACF and CURVEFIT.

TEMPORARY

```
TEMPORARY
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

Overview

TEMPORARY signals the beginning of temporary transformations that are in effect only for the next procedure. With the exception of variables created or modified by the next procedure (for example, residual values saved as variables by REGRESSION), numeric or string variables created while the TEMPORARY command is in effect are temporary variables, and any modifications made to existing variables while the TEMPORARY command is in effect are also temporary. Any variables created or modified by the next procedure are permanent.

With TEMPORARY, you can perform separate analyses for subgroups in the data and then repeat the analysis for the file as a whole. You can also use TEMPORARY to transform data for one analysis but not for other subsequent analyses.

TEMPORARY can be applied to the following commands:

- Transformation commands COMPUTE, RECODE, IF, and COUNT, and the DO REPEAT utility.
- The LOOP and DO IF structures.
- Format commands PRINT FORMATS, WRITE FORMATS, and FORMATS.
- Data selection commands SELECT IF, SAMPLE, FILTER, and WEIGHT.
- Variable declarations NUMERIC, STRING, and VECTOR.
- Labeling commands VARIABLE LABELS and VALUE LABELS, and the MISSING VALUES command.
- SPLIT FILE.
- PRINT, WRITE, XSAVE.

Basic specification

The only specification is the keyword TEMPORARY. There are no additional specifications.

Operations

- Once TEMPORARY is specified, you cannot refer to previously existing scratch variables.
- Temporary transformations apply to the next command that reads the data. Once the data are read, the temporary transformations are no longer in effect.
- The XSAVE command leaves temporary transformations in effect. SAVE, however, reads the data and turns temporary transformations off after the file is written. (See "Examples.")
- TEMPORARY cannot be used with SORT CASES, MATCH FILES, ADD FILES, or COMPUTE with a LAG function. If any of these commands follows TEMPORARY in the command sequence, there must be an intervening procedure or command that reads the data to first execute the TEMPORARY command.
- TEMPORARY cannot be used within the DO IF–END IF or LOOP–END LOOP structures.

If any variables have an unknown measurement level, the measurement level for those variables remains unknown after a procedure that follows a TEMPORARY command. (In the absence of the TEMPORARY command, the data pass performed by the procedure would assign default measurement levels to all variables with an unknown measurement level.)

Examples

Basic example

```
SORT CASES BY SEX.
TEMPORARY.
SPLIT FILE BY SEX.
FREQUENCIES VARS=INCOME /STATISTICS=MEDIAN.
FREQUENCIES VARS=INCOME /STATISTICS=MEDIAN.
```

- SPLIT FILE applies to the first FREQUENCIES procedure, which generates separate median income tables for men and women.
- SPLIT FILE is not in effect for the second FREQUENCIES procedure, which generates a single median income table that includes both men and women.

Temporary transformations for a new variable

```
DATA LIST FILE=HUBDATA RECORDS=3
  /1 #MOBIRTH #DABIRTH #YRBIRTH 6-11 DEPT88 19.
COMPUTE    AGE=($JDATE - YRMODA(#YRBIRTH,#MOBIRTH,#DABIRTH))/365.25.
VARIABLE LABELS AGE 'EMPLOYEE''S AGE'
          DEPT88 'DEPARTMENT CODE IN 1988'.

TEMPORARY.
RECODE AGE (LO THRU 20=1)(20 THRU 25=2)(25 THRU 30=3)(30 THRU 35=4)
       (35 THRU 40=5)(40 THRU 45=6)(45 THRU 50=7)(50 THRU 55=8)
       (55 THRU 60=9)(60 THRU 65=10)(65 THRU HI=11).
VARIABLE LABELS AGE 'EMPLOYEE AGE CATEGORIES'.
VALUE LABELS AGE 1 'Up to 20' 2 '20 to 25' 3 '25 to 30' 4 '30 to 35'
       5 '35 to 40' 6 '40 to 45' 7 '45 to 50' 8 '50 to 55'
       9 '55 to 60' 10 '60 to 65' 11 '65 and older'.

FREQUENCIES VARIABLES=AGE.
MEANS AGE BY DEPT88.
```

- COMPUTE creates variable *AGE* from the dates in the data.
- FREQUENCIES uses the temporary version of variable *AGE* with temporary variable and value labels.
- MEANS uses the unrecoded values of *AGE* and the permanent variable label.

Using XSAVE with TEMPORARY

```
GET FILE='/data/hub.sav'.
TEMPORARY.
RECODE DEPT85 TO DEPT88 (1,2=1) (3,4=2) (ELSE=9).
VALUE LABELS DEPT85 TO DEPT88 1 'MANAGEMENT'
                              2 'OPERATIONS'
                              3 'UNKNOWN'.
XSAVE OUTFILE='/temp/hubtemp.sav'.
CROSSTABS DEPT85 TO DEPT88 BY JOBCAT.
```

- Both the saved data file and the CROSSTABS output will reflect the temporary recoding and labeling of the department variables.
- If XSAVE is replaced with SAVE, the saved data file will reflect the temporary recoding and labeling but the CROSSTABS output will not.

TIME PROGRAM

```
TIME PROGRAM.
[commands to compute time dependent covariates]
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

Overview

TIME PROGRAM is required to define time-dependent covariates. These are covariates whose values change during the course of the study.

Basic Specification

The basic specification is the TIME PROGRAM command followed by one or more commands that define time-dependent covariates.

Syntax Rules

- TIME PROGRAM and the transformations that define the time-dependent covariate(s) must precede the command that analyzes the time-dependent covariate(s).
- A time-dependent covariate is a function of the current time, which is represented by the special variable *T_*.
- The active dataset must not have a variable named *T_*. If it does, rename the variable. Otherwise, you will trigger an error.
- *T_* cannot be specified as a covariate. Any other variable in the TIME PROGRAM can be specified on the covariate list.
- For every time-dependent covariate, values are generated for each valid case for all uncensored times in the same stratum that occur before the observed time. If no STRATA subcommand is specified, all cases are considered to belong to one stratum.
- If any function defined by the time program results in a missing value for a case that has no missing values for any other variable used in the procedure, this causes an error.

Example

```
TIME PROGRAM.
COMPUTE Z=AGE + T_.

COXREG VARIABLES = SURVIVAL WITH Z
  /STATUS SURVSTA EVENT (1).
```

- TIME PROGRAM defines the time-dependent covariate *Z* as the current age. *Z* is then specified as a covariate in COXREG.

TITLE

```
TITLE [']text[']
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
TITLE "Running Shoe Study from Runner's World Data".
```

Overview

TITLE inserts a title on the top line of each page of output. The default title indicates the version of the system being used.

Basic Specification

The only specification is the title.

Syntax Rules

- The title can include any characters. To specify a blank title, enclose a blank in quotes (" ").
- The title can be up to 60 bytes long. Titles longer than 60 bytes are truncated.
- The quotes enclosing the title are optional; using them allows you to include apostrophes or quotation marks in the title. See the topic “String Values in Command Specifications” on page 35 for more information.
- More than one TITLE command is allowed in a single session.
- A title cannot be placed between a procedure command and BEGIN DATA–END DATA or within data records when the data are inline.

Operations

- The title is displayed as part of the output heading, which also includes the date and page number. If HEADER=NO is specified on SET, the heading, including the title and subtitle, will not be displayed.
- Each TITLE command overrides the previous one and takes effect on the next output page.
- Only the title portion of the heading changes. The date and page number are still displayed.
- TITLE is independent of SUBTITLE, and each can be changed separately.

Examples

```
TITLE My Title.
TITLE "Fred's Title".
TITLE 'Fred''s "Quoted" Title'.
```

- The first title doesn't need to be enclosed in quotes because it doesn't contain any quotes or apostrophes.
- The second title is enclosed in double quotes because it contains an apostrophe (single quote).
- The last title is enclosed in single quotes since it contains double quotes, and two single quotes are used to specify the apostrophe.

TMS BEGIN

```
TMS BEGIN
   /DESTINATION OUTFILE='file specification'
```

Release History

Release 15.0

- Command introduced.

Release 16.0

- Added support for new string functions CHAR.CONCAT, CHAR.LENGTH, and CHAR.SUBSTR within TMS blocks.

Release 21.0

- Added support for comparison operators and logical operators.

Example

```
TMS BEGIN
 /DESTINATION OUTFILE='/mydir/mytransformations.xml'.
COMPUTE modelvar=ln(var).
TMS END.
```

Overview

The TMS BEGIN command indicates the beginning of a block of transformations to be exported to a file in PMML format--specifically, PMML 3.1 with IBM SPSS Statistics extensions. The exported transformations can be merged (using TMS MERGE) with a PMML model file in order to include any transformations of the raw data required to create the variables used in the model. The merged PMML file can be used to score data with scoring functions. (For more information on scoring functions, see "Scoring expressions" on page 86.) When you score data with a model file containing transformations, the transformations will be applied to the data before scoring. The transformations are carried out as part of the internal scoring process and have no effect on the active dataset. The TMS END command is used to end the block.

From the point of view of the active dataset, transformations in a TMS BEGIN-TMS END block are no different than transformations outside a TMS block--they are pending and will be executed with the next command that reads the active dataset. Source variables used in transformation expressions in TMS blocks must exist in the active dataset. In a typical scenario, you would run TMS BEGIN-TMS END command syntax on the raw dataset used to build the associated model, assuming it contains the source variables used in any transformation expressions.

Commands within a TMS block are categorized as belonging to one of four types: exported, tolerated, invalid, and those causing an implicit TMS END.

Exported. These are commands that produce export to PMML. Commands in this category include the supported transformations--for example, RECODE. Many transformations are expressed with functions included on a COMPUTE command. Export to PMML is supported for a limited set of such functions. The set of commands that produce export to PMML, as well as the set of supported functions, are provided at the end of this topic.

Tolerated. These are commands that can be included in a TMS BEGIN-TMS END block, are executed in normal fashion, but do not produce export to PMML. Examples include NUMERIC, STRING, USE, INCLUDE, and INSERT. In the case of INCLUDE or INSERT, supported transformations from the inserted file will be exported to PMML.

Invalid. These are commands that cannot be processed within a TMS block and will give rise to an error. The list of invalid commands is provided at the end of this topic.

Implicit TMS END. These commands are executed in normal fashion but cause an implicit end to the current block, which then causes the output PMML file to be written. Transformations (in the current block) occurring after one of these commands are not exported to the output PMML file. Commands in this category include all of the statistical procedures--for example, ANOVA and REGRESSION--as well as the EXECUTE command. ADD FILES, MATCH FILES and COMPARE DATASETS belong to this category as well as any command that causes a different dataset to become active--for example, DATASET ACTIVATE, GET, and GET DATA.

Basic Specification

The basic specification for TMS BEGIN is the command name followed by a DESTINATION subcommand that contains an OUTFILE specification.

Syntax Rules

- The DESTINATION subcommand is required.
- An error occurs if the DESTINATION subcommand is specified more than once.
- An error occurs if the OUTFILE keyword is specified more than once.
- Equal signs (=) shown in the syntax chart are required.
- Subcommand names and keywords must be spelled in full.

Operations

- Once a TMS BEGIN command is executed, it remains in effect until the end of the session, until explicitly ended by a TMS END command, or until a command that implicitly ends a TMS block--for example, any statistical procedure--is encountered.
- TMS commands (for example, TMS MERGE) are not allowed within a TMS block.
- TMS BEGIN verifies that the path specified for the destination file is valid. If the path is invalid, the commands in the block will be processed in the normal manner but no PMML will be exported.
- The following conditions will cause the destination file to be written: a command that causes an implicit end of the block is executed; the TMS block ends explicitly with TMS END; the current session ends before the block ends (either explicitly or implicitly). The destination file is unavailable to other commands until after it is written. TMS END has no effect if the block is ended implicitly.
- An error occurs if a transformation in the block is not supported. The command containing the invalid transformation is not executed. This type of error does not cause an implicit end to the TMS block and has no effect on when the destination file is written.
- The order of the transformations in the destination file may not be the order in which they were specified in the TMS block.
- Transformations specified in a TMS block are processed in the normal manner--that is, pending and to be executed with the next command that reads the active dataset.

Exported Commands

The following commands produce export to PMML: ADD VALUE LABELS, COMPUTE, DO REPEAT, MISSING VALUES, RECODE, VALUE LABELS, VARIABLE LABELS, VARIABLE LEVEL.

- For COMPUTE, the target variable must be a variable that has not already been assigned a value or otherwise used in the associated TMS block. For example, the following is invalid:

```
COMPUTE var = var + 1.
```

- For RECODE, all input and output keywords are supported except CONVERT. In addition, you must always recode INTO a variable that has not already been assigned a value or otherwise used in the associated TMS block. When a recoding transformation is carried out on a dataset to be scored, the results depend on whether the target variable exists in the dataset. If the target variable already exists, cases with values (of the source variable) not mentioned in the value specifications for the recode are not changed. If the target variable does not already exist, cases with values not mentioned are assigned the system-missing value.

Invalid Commands

The following commands are invalid inside a TMS block: AGGREGATE, BREAK, COUNT, DO IF - END IF, ELSE, ELSE IF, END CASE, END FILE, FILE TYPE - END FILE TYPE, FILTER, IF INPUT PROGRAM - END INPUT PROGRAM, KEYED DATA LIST, LEAVE, LOOP - END LOOP, N OF CASES, POINT, RECORD TYPE, REPEATING DATA, REREAD, SAMPLE, SELECT IF, VECTOR, WEIGHT, WRITE.

The following functions, operations, and system variables are supported for use with the COMPUTE command in TMS blocks. In a few cases--for example, CHAR.SUBSTR--multiple forms (differing in the number of parameters) exist, but only a single form is supported. The parameters available in the supported form are:

- + Addition
- - Subtraction
- * Multiplication
- / Division
- ** Exponentiation
- & Logical AND
- | Logical OR
- ~ Logical NOT
- = Equal to
- ~= Not equal to
- < Less than
- > Greater than <= Less than or equal to
- >= Greater than or equal to
- ABS
- CDF.NORMAL
- CDFNORM
- CHAR.CONCAT
- CHAR.LENGTH
- CHAR.SUBSTR(strexpr,pos)
- CONCAT
- EXP
- IDF.NORMAL
- LG10
- LENGTH
- LN
- LOWER
- LTRIM(strexpr)
- MAX
- MEAN
- MIN
- MOD
- PDF.NORMAL
- RND
- RTRIM(strexpr)
- SD
- SQRT

- SUBSTR(strexpr,pos)
- SUM
- SYSMIS
- TRUNC
- UPCASE
- VALUE
- VARIANCE

Notes

- Within a TMS block it is recommended to use CHAR.CONCAT, CHAR.LENGTH, and CHAR.SUBSTR in place of CONCAT, LENGTH, and SUBSTR.
- String arguments for the MAX and MIN functions are not supported for scoring.

EXAMPLES

```
TMS BEGIN
 /DESTINATION OUTFILE='/mydir/mytransformations.xml'.
COMPUTE modelvar=ln(var).
TMS END.
```

- TMS BEGIN marks the beginning of a block of transformation commands that will be evaluated for export to PMML.
- The variable *var* is transformed with a log function to create the variable *modelvar*, which is to be used in a model. The details of this transformation will be included as PMML in the file */mydir/mytransformations.xml*.
- TMS END marks the end of the block and causes the output file to be written, but has no effect on the state of the transformations contained in the block. In the present example, the transformation to create *modelvar* is still pending after the completion of the block.

Tolerated Commands in a TMS Block

```
TMS BEGIN
 /DESTINATION OUTFILE='/mydir/mytransformations.xml'.
STRING new_strvar (A1).
RECODE strvar ('A','B','C'='A') ('D','E','F'='B') (ELSE=' ')
   INTO new_strvar.
TMS END.
```

- The STRING command is used to create a new string variable that is to be the target of a recode transformation of an existing string variable. STRING is a tolerated command so it is executed in the normal manner, without generating an error or implicitly ending the TMS block.

Commands that Cause an Implicit TMS END

```
TMS BEGIN
 /DESTINATION OUTFILE='/mydir/mytransformations.xml'.
RECODE numvar1 (0=1) (1=0) (ELSE=SYSMIS) INTO new_numvar1.
FREQUENCIES new_numvar1.
RECODE numvar2 (1 THRU 5=1)(6 THRU 10=2)(11 THRU HI=3)(ELSE=0)
   INTO new_numvar2.
TMS END.
```

- The FREQUENCIES command causes an implicit end to the TMS block, causing the output PMML file to be written. The output file only contains the recode transformation for *numvar1*--that is, the transformations prior to the command causing the implicit TMS END.
- Although the FREQUENCIES command implicitly ends the block, the recode transformation for *numvar2* is processed in the normal manner and remains pending until the next command that reads the active dataset.
- Although we have included it in this example, the TMS END command is not needed when there is a command that implicitly ends the TMS block. In this example, execution of the TMS END command generates a warning indicating that there is no current TMS command in effect.

Using EXECUTE in a TMS Block

```
TMS BEGIN
 /DESTINATION OUTFILE='/mydir/mytransformations.xml'.
RECODE numvar1 (0=1) (1=0) (ELSE=SYSMIS) INTO new_numvar1.
RECODE numvar2 (1 THRU 5=1)(6 THRU 10=2)(11 THRU HI=3)(ELSE=0)
   INTO new_numvar2.
EXECUTE.
```

- You can include EXECUTE in a TMS block. In its usual manner, it will cause all pending transformations to be executed. In addition, it causes an implicit end to the block which then causes the output PMML file to be written.
- In the current example, we have omitted the TMS END command since the block is implicitly ended by the EXECUTE command.

DESTINATION Subcommand

The DESTINATION subcommand is required. It must include the OUTFILE keyword followed by an equals sign (=) and a file specification in quotes or a previously defined file handle defined with the FILE HANDLE command. It is recommended to include the extension *.xml* in the file specification.

TMS END

```
TMS END
   [/PRINT RESULTS={SUMMARY**}]
                   {NONE}
```

** Default if the subcommand or keyword is omitted.

Release History

Release 15.0

- Command introduced.

Example

```
TMS BEGIN
 /DESTINATION OUTFILE='/mydir/mytransformation.xml'.
COMPUTE mymodelvar=ln(var).
TMS END
 /PRINT RESULTS=NONE.
```

Overview

The TMS END command is used to indicate the end of a block of transformations to be exported to a file in PMML format--specifically, PMML 3.1 with IBM SPSS Statistics extensions. The exported transformations can be merged (using TMS MERGE) with a PMML model file in order to include any transformations of the raw data required to create the variables used in the model. The merged PMML file can be used to score data with scoring functions. (For more information on scoring functions, see “Scoring expressions” on page 86.) When you score data with a model file containing transformations, the transformations will be applied to the data before scoring. The transformations are carried out as part of the internal scoring process and have no effect on the active dataset. The TMS BEGIN command is used to indicate the beginning of the block.

Basic Specification

The basic specification for TMS END is the command name. For examples of TMS BEGIN-TMS END blocks, see the TMS BEGIN command.

Syntax Rules

- An error occurs if the PRINT subcommand is specified more than once.
- An error occurs if the RESULTS keyword is specified more than once.
- Equal signs (=) shown in the syntax chart are required.
- Subcommand names and keywords must be spelled in full.

Operations

- TMS END will generate a warning if there is no TMS BEGIN command in effect. This condition occurs in the obvious case that there is no prior TMS BEGIN command, but will also occur if there is a TMS BEGIN command followed by a command that implicitly ends a TMS block--for instance, any statistical procedure or the EXECUTE command--followed by TMS END.
- TMS END causes the destination file to be written, unless the associated TMS block ended implicitly (see previous bullet), thus causing the file to be written before execution of TMS END. For more information, see the Examples for the TMS BEGIN command.
- TMS END does not cause transformations in the associated TMS block to be executed. In normal fashion, they are pending and will be executed with the next command that reads the active dataset.

PRINT Subcommand

The PRINT subcommand is optional and specifies whether or not to print a table summarizing the derived variables defined by the transformations in the block.

SUMMARY. *Print a summary of the derived variables.* This is the default.

NONE. *Do not print a summary.*

TMS IMPORT

```
TMS IMPORT
  /INFILE TRANSFORMATIONS='filespec'
     [MODE={FORWARD**(ROLES={UPDATE**})        }]
                           {LEAVE   }
           {BACK(PREDICTED=fieldname
                 SUFFIX={'_backtransformed'}
                        {suffixname        }  )}
  [/SAVE TRANSFORMED={YES**}]
                     {NO   }
  [/OUTFILE SYNTAX='filespec'].
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 18

- Command introduced.

Example

```
TMS IMPORT
  /INFILE TRANSFORMATIONS='file.xml'.
```

Overview

`TMS IMPORT` converts transformations contained in an XML file created by ADP (transformations files created by `TMS BEGIN-END` are not currently "reversible") into command syntax and applies the converted transformations to the active dataset or saves them to a syntax file.

Basic Specification

The basic specification is the `TMS IMPORT` command and an `INFILE` subcommand with a transformations file specification.

Syntax Rules

- The `INFILE` subcommand is required. All other subcommands are optional.
- Each subcommand can be specified only once.
- Subcommands may be used in any order.
- An error occurs if a keyword is specified more than once within a subcommand.
- Equal signs (=) shown in the syntax chart are required.
- Subcommand names and keywords must be spelled in full.

Examples

```
TMS IMPORT
  /INFILE TRANSFORMATIONS='file.xml'.
```

- `TMS IMPORT` reads the file *file.xml*, converts the transformations from XML to command syntax, and applies them to the active dataset.

Using Backtransformations

```
TMS IMPORT
  /INFILE TRANSFORMATIONS='workingDirectory/car_sales_transformations.xml'
    MODE=BACK (PREDICTED=PRE_2 SUFFIX='_backtransformed').
EXECUTE.
```

- The TMS IMPORT command reads the transformations in *car_sales_transformations.xml* and applies the backtransformation to *PRE_2*.
- The new field containing the backtransformed values is named *PRE_2_backtransformed*.
- The EXECUTE command causes the transformations to be processed. When using this as part of a longer stream of syntax, you may be able to remove the EXECUTE command to save some processing time.

INFILE Subcommand

The INFILE subcommand specifies the location of the XML file containing the transformations.

TRANSFORMATIONS Keyword

The TRANSFORMATIONS keyword specifies the location of the XML file containing the transformations. Specify a filename in quotation marks. The filename must be specified in full.

The MODE keyword specifies the direction of the transformations.

FORWARD (ROLES= UPDATE|LEAVE). *Convert the "forward" transformations from XML to command syntax.* These are the transformations from the original fields into fields that are used in modeling. ROLES determines whether to update field roles based upon transformations in the file. UPDATE will set the role of any "obsolete" field to None. LEAVE will leave the roles of existing fields as they currently are. FORWARD is the default mode. See the topic "VARIABLE ROLE" on page 2129 for more information. .

BACK(PREDICTED=fieldname SUFFIX=suffixname). *Convert the backtransformation of target values into command syntax and apply to the field specified on the PREDICTED keyword.* The backtransformed values are saved to a field with the name on the PREDICTED keyword and suffix on the SUFFIX keyword. *_backtransformed* is the default suffix.

Example

```
TMS IMPORT
  /INFILE TRANSFORMATIONS='file.xml'.
REGRESSION VARIABLES=target_transformed input_transformed
 /DEPENDENT=target_transformed
 /METHOD=ENTER input_transformed
 /SAVE=PRED(predicted) SDBETA(BETA) ICIN.
TMS IMPORT
  /INFILE TRANSFORMATIONS='file.xml' MODE=BACK(PREDICTED=predicted).
```

- The first TMS IMPORT reads the file *file.xml*, converts the transformations from XML to command syntax, and applies them to the active dataset. The fields *target_transformed* and *input_transformed* result.
- The REGRESSION command builds a model for *target_transformed* using *input_transformed*, and adds *predicted* to the active dataset, containing the predicted values.
- The second TMS IMPORT reads the file *file.xml*, converts the backtransformations from XML to command syntax, and applies them to the active dataset. The field *predicted_backtransformed* results, and contains predicted values in the original scale of the target field.

SAVE Subcommand

The SAVE subcommand adds the prepared fields to the active dataset.

TRANSFORMED. *Transformed data.* The transformed fields are added to the active dataset. The values are not computed immediately; instead the transformations are stored, pending execution with the next command that reads the dataset.

OUTFILE Subcommand

The OUTFILE subcommand specifies the location of the file to which command syntax should be saved.

SYNTAX=filename. *Syntax file.* Specify a filename in quotation marks. The filename must be specified in full; no extension is supplied. It is recommended to include the extension *.sps* in the file specification.

TMS MERGE

```
TMS MERGE
   /DESTINATION OUTFILE='file specification'
   /TRANSFORMATIONS INFILE='file specification'
   /MODEL INFILE='file specification'
   [/PRINT RESULTS={SUMMARY**}]
                   {NONE}
```

** Default if the subcommand or keyword is omitted.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic "Command Order" on page 36 for more information.

Release History

Release 15.0

- Command introduced.

Example

```
TMS MERGE
   /DESTINATION OUTFILE='/mydir/mymergedmodel.xml'
   /TRANSFORMATIONS INFILE='/mydir/mytransformations.xml'
   /MODEL INFILE='/mydir/mymodel.xml'.
```

Overview

The TMS MERGE command is used to merge a PMML file containing transformations with a PMML model file containing variables whose derivations are expressed by those transformations. The merged PMML file can be used to score data with scoring functions. (For more information on scoring functions, see "Scoring expressions" on page 86.) When you score data with a model file containing transformations, the transformations will be applied to the data before scoring. The transformations are carried out as part of the internal scoring process and have no effect on the active dataset. Transformations are exported as PMML using the TMS BEGIN and TMS END commands. See the topic "TMS BEGIN" on page 1977 for more information.

Basic Specification

The basic specification is the command name followed by DESTINATION, TRANSFORMATIONS, and MODEL subcommands that contain file specifications.

Syntax Rules

- Each subcommand can be specified only once.
- Subcommands may be used in any order.
- An error occurs if a keyword is specified more than once within a subcommand.
- Equal signs (=) shown in the syntax chart are required.
- Subcommand names and keywords must be spelled in full.

Operations

- TMS MERGE has no effect on the active dataset. It relies completely on the information contained in the transformations and model files.
- TMS MERGE verifies that the paths specified for the input files and destination file are valid. The destination file is unavailable to other commands until the TMS MERGE command is ended.
- An error occurs if properties of derived variables in the transformations file do not match the properties for those same variables in the model file--for example, conflicting definitions of missing values.

Limitations

- TMS MERGE cannot be used to merge transformations with an ensemble model.
- TMS MERGE cannot be used to merge transformations with a model that contains splits (separate model information for each split group).

TRANSFORMATIONS, MODEL, and DESTINATION Subcommands

The TRANSFORMATIONS, MODEL, and DESTINATION subcommands specify the file containing the transformations (created from a TMS BEGIN-TMS END block), the file containing the model specifications, and the destination for the merged file. All three of these subcommands are required.

- File specifications should be enclosed in quotation marks.
- The filename must be specified in full. No extension is supplied.
- It is recommended to include the extension *.xml* in the specification for the destination file.

PRINT Subcommand

The PRINT subcommand is optional and specifies whether or not to print tables summarizing the variables in the merged XML file.

SUMMARY. *Print summary tables.* This is the default.

NONE. *Do not print summary tables.*

TREE

TREE is available in SPSS Statistics Professional Edition or the Decision Trees option.

Note: Square brackets used in the TREE syntax chart are required parts of the syntax and are not used to indicate optional elements. Equals signs (=) used in the syntax chart are required elements. All subcommands are optional.

```
TREE dependent_variable [level]
     BY variable [level] variable [level]...
     FORCE = variable [level]

  /TREE DISPLAY ={TOPDOWN**  } NODES = {STATISTICS**}
                 {LEFTTORIGHT}         {CHART      }
                 {RIGHTTOLEFT}         {BOTH       }
                 {NONE       }
        BRANCHSTATISTICS ={YES**} NODEDEFS = {YES**}
                          {NO   }            {NO   }
        SCALE={AUTO**      }
              {percent value}

  /DEPCATEGORIES USEVALUES=[VALID** value, value...MISSING]
                 TARGET=[value value...]

  /PRINT MODELSUMMARY** CLASSIFICATION** RISK** CPS** IMPORTANCE SURROGATES
         TREETABLE CATEGORYSPECS NONE

  /GAIN SUMMARYTABLE = {YES**} CATEGORYTABLE = {YES**}
                       {NO   }                {NO   }
        TYPE = [NODE** PTILE] SORT = {DESCENDING**}
                                     {ASCENDING   }
        INCREMENT = {10** } CUMULATIVE = {YES**}
                    {value}              {NO   }

  /PLOT GAIN RESPONSE INDEX MEAN PROFIT ROI IMPORTANCE INCREMENT = {10   }
                                                                  {value}

  /RULES NODES = {TERMINAL**     } SYNTAX = {INTERNAL** }
                 {ALL            }          {SQL        }
                 {TOPN(value)    }          {GENERIC    }
                 {TOPPCT(value)  }
                 {MININDEX(value)}
         TYPE = {SCORING**} SURROGATES = {EXCLUDE**} LABELS = {YES**}
                {SELECTION}              {INCLUDE  }          {NO   }
         OUTFILE = 'filespec'

  /SAVE NODEID(varname) PREDVAL(varname) PREDPROB(rootname) ASSIGNMENT(varname)

 /METHOD TYPE = {CHAID**        }
                {EXHAUSTIVECHAID}
                {CRT            }
                {QUEST          }
         MAXSURROGATES = {AUTO**} PRUNE = {NONE**     }
                         {value }         {SE({1    })}
                                             {value}

 /GROWTHLIMIT MAXDEPTH = {AUTO**} MINPARENTSIZE = {100**}
                         {value }                 {value}

              MINCHILDSIZE = {50** }
                             {value}

 /VALIDATION  TYPE = {NONE**                  } OUTPUT = {BOTHSAMPLES}
                     {SPLITSAMPLE({50       })}          {TESTSAMPLE }
                                  {percent  }
                                  {varname}
                     {CROSSVALIDATION({10   })}
                                      {value}

 /CHAID  ALPHASPLIT = {.05**}  ALPHAMERGE = {.05**}
                      {value}               {value}
         SPLITMERGED = {NO**} CHISQUARE = {PEARSON**}
                       {YES }             {LR      }
         CONVERGE = {.001**}  MAXITERATIONS = {100**}
                    {value }                  {value}
         ADJUST = {BONFERRONI**}
                  {NONE       }
         INTERVALS = {10**                          }
                     {value                         }
                     {varlist(value) varlist(value) …}

  /CRT IMPURITY = {GINI**      } MINIMPROVEMENT = {.0001**}
                  {TWOING      }                  {value  }
                  {ORDEREDTWOING}

  /QUEST ALPHASPLIT = {.05**}
                      {value}

 /COSTS {EQUAL**                                                 }
        {CUSTOM = actcat, predcat [value] actcat, predcat [value] ...}

 /PRIORS {FROMDATA**                        } ADJUST = {NO**}
         {EQUAL                             }          {YES }
         {CUSTOM = cat [value] cat [value] ...}

 /SCORES {EQUALINCREMENTS**                 }
         {CUSTOM = cat [value] cat [value] ...}
```

```
/PROFITS CUSTOM = cat [revenue, expense] cat [revenue, expense] ...

/INFLUENCE varname

/OUTFILE TRAININGMODEL = 'filespec' TESTMODEL = 'filespec'

/MISSING  NOMINALMISSING = {MISSING**}
                           {VALID    }

/TARGETRESPONSE RATE = {NONE   }
                       {value  }
```

** Default if subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 13.0

- Command introduced.

Release 18.0

- TARGETRESPONSE subcommand introduced.

Example

```
TREE risk BY income age creditscore employment.
```

Overview

The TREE procedure creates a tree-based model. It classifies cases into groups or predicts values of a dependent variable based on values of predictor variables. The procedure provides validation tools for exploratory and confirmatory classification analysis.

Options

Model. You can specify the dependent (target) variable and one or more independent (predictor) variables. Optionally you can force one independent variable into the model as the first variable.

Growing Method. Four growing algorithms are available: CHAID (the default), Exhaustive CHAID, CRT, and QUEST. Each performs a type of recursive splitting. First, all predictors are examined to find the one that gives the best classification or prediction by splitting the sample into subgroups (nodes). The process is applied recursively, dividing the subgroups into smaller and smaller groups. It stops when one or more stopping criteria are met.

The four growing methods have different performance characteristics and features:

- CHAID chooses predictors that have the strongest interaction with the dependent variable. Predictor categories are merged if they are not significantly different with respect to the dependent variable (Kass, 1980).
- Exhaustive CHAID is a modification of CHAID that examines all possible splits for each predictor (Biggs et al., 1991).
- CRT is a family of methods that maximizes within-node homogeneity (Breiman et al., 1984).
- QUEST trees are computed rapidly, but the method is available only if the dependent variable is nominal. (Loh and Shih, 1997).

Stopping Criteria. You can set parameters that limit the size of the tree and control the minimum number of cases in each node.

Validation. You can assess how well your tree structure generalizes to a larger sample. Split-sample partitioning and cross-validation are supported. Partitioning divides your data into a training sample, from which the tree is grown, and a testing sample, on which the tree is tested. Cross-validation involves dividing the sample into a number of smaller samples. Trees are generated excluding the data from each

subsample in turn. For each tree, misclassification risk is estimated using data for the subsample that was excluded in generating it. A cross-validated risk estimate is calculated as the average risk across trees.

Output. Default output includes a tree diagram and risk statistics. Classification accuracy is reported if the dependent variable is categorical. Optionally, you can obtain charts of gain- and profit-related measures as well as classification rules that can be used to select or score new cases. You can also save the model's predictions to the active dataset, including assigned segment (node), predicted class/value, and predicted probability.

Basic Specification

- The basic specification is a dependent variable and one or more independent variables.

Operations

- The tree is grown until one or more stopping criteria are met. The default growing method is CHAID.
- The type of model depends on the measurement level of the dependent variable. If the dependent variable is scale (continuous), a prediction model is computed. If it is categorical (nominal or ordinal), a classification model is generated.
- Measurement level determines allowable combinations of predictor values within a node. For ordinal and scale predictors, only adjacent categories/values may occur in a node. There are no restrictions on grouping of nominal categories.
- TREE honors the SET SEED value if split-sample model validation is requested.
- SPLIT FILE is ignored by the TREE procedure.
- If a WEIGHT variable is defined the weights are treated as replication weights. Fractional weights are rounded.

Note: Since measurement level can affect the results, if any variables (fields) have an unknown measurement level, an initial data pass will be performed to determine default measurement level for any variables with an unknown measurement level. For information on the criteria used to determine default measurement level, see SET SCALEMIN.

Syntax Rules

- The minimum specification is a dependent variable, the keyword BY and one or more independent variables.
- All subcommands are optional.
- Only a single instance of each subcommand is allowed.
- A keyword may be specified only once within a subcommand.
- Equals signs (=) shown in the syntax chart are required.
- Subcommand names and keywords must be spelled in full.
- Subcommands may be used in any order.
- SPLIT FILE is ignored by the TREE procedure.
- CHAID and Exhaustive CHAID: A categorical dependent variable may not have more than 126 categories. If the dependent variable is categorical, then the limit for a categorical predictor is also 126 categories.
- CRT: A nominal predictor may not have more than 32 categories.
- QUEST: If a predictor is nominal, then the limit for the dependent variable (which must be nominal) is 127 categories. A nominal predictor may not have more than 25 categories.

Examples

```
TREE risk BY income age creditscore employment.
```

- A tree model is computed that estimates credit risk using an individual's income, age, credit score, and employment category as predictor variables.
- The default method, CHAID, is used to grow the tree.

- Since measurement level is not specified, it is obtained from the data dictionary for each model variable. If no measurement levels have been defined, numeric variables are treated as scale and string variables are treated as nominal.

```
TREE risk [o] BY income [o] age [s] creditscore [s] employment [n]
  /METHOD TYPE=CRT
  /VALIDATION TYPE=SPLITSAMPLE
  /SAVE NODEID PREDVAL.
```

- A tree model is computed that estimates credit risk using an individual's income, age, credit score, and employment category as predictor variables.
- Age and credit score will be treated as scale variables, risk and income as ordinal, and employment category as nominal.
- The CRT method, which performs binary splits, is used to grow the tree.
- Split-sample validation is requested. By default, 50% of cases are assigned to the training sample. Remaining cases are used to validate the tree.
- Two variables are saved to the active dataset: node (segment) identifier and predicted value.

Model Variables

The command name TREE is followed by the dependent variable, the keyword BY, and one or more independent variables.

Dependent Variable

The dependent (target) variable must be the first specification on the TREE command.

- Only one dependent variable can be specified.
- The dependent variable cannot be the WEIGHT variable.

Independent Variables

You can specify one or more independent (predictor) variables.

- ALL and TO can be used in the independent variable list. ALL represents all variables in the active dataset. TO refers to a range of variables in the active dataset.
- In general, if the independent variable list refers to the WEIGHT variable or a variable that is used elsewhere in the tree specification (e.g., dependent variable or cell weight variable), the variable is not used as an independent variable. The exception is that scale variables used both in the independent variable list and the INTERVAL subcommand are not dropped from the independent variable list.
- Repeated instances of the same independent variable are filtered out of the list. For example, a b a c a is equivalent to a b c. Assuming that the active dataset contains variables *a*, *b*, and *c*, a ALL is also equivalent to a b c.

Measurement Level

Optionally, a measurement level can be specified in square brackets following any model variable. The measurement level can be defined as scale ([S]), ordinal ([O]), or nominal ([N]). Categorical ([C]) is accepted as a synonym for nominal.

- If a measurement level is specified, it temporarily overrides the setting recorded in the data dictionary. If a measurement level is not specified for a variable, the dictionary setting is used.
- If a measurement level is not specified and no setting is recorded in the data dictionary, a numeric variable is treated as scale and a string variable is treated as nominal.
- If a string variable is defined as scale, a warning is issued and the variable is treated as nominal.
- A measurement level applies only to the variable that immediately precedes it in the variable list. For example, age income [S] and age TO income [S] assign the scale level of measurement to *income* only.

- The keyword ALL followed by a measurement level specification will apply that measurement level to all independent variables.
- If a variable appears more than once in the independent variable list and the measurement level varies across instances, the measurement level specified for the last instance of the variable is used. For example, if `ALL [S] z [N] is specified`, *z* is treated as a nominal predictor.

FORCE Keyword

- By default, the procedure evaluates each predictor variable for inclusion in the model. The optional FORCE keyword forces an independent variable into the model as the first split variable (the first variable used to split the root node into child nodes).
- The FORCE keyword must be followed by an equals sign (=) and a single variable.
- A measurement level may be specified in square brackets following the variable name. If the variable is also included in the independent variable list, the measurement level specified for the FORCE variable is used. If the variable has a specified measurement level in the independent list but no explicit measurement level on the FORCE specification, the defined measurement level for the variable is used.
- The variable cannot be the dependent variable or the WEIGHT variable.

Example

```
TREE risk [o] BY income [o] age [s] creditscore [s] FORCE = gender [n].
```

- The independent variables *income*, *age*, and *creditscore* may or may not be included in the model, depending on how strongly they interact with the dependent variable *risk*.
- The independent variable *gender* will be included in the model regardless of the level of interaction with the dependent variable, and the root node will first be split into nodes for male and female before any other independent variables are considered.

DEPCATEGORIES Subcommand

For categorical dependent variables, the DEPCATEGORIES subcommand controls the categories of the dependent variable included in the model and/or specifies target categories.

- By default, all valid categories are used in the analysis and user-missing values are excluded.
- There is no default target category.
- DEPCATEGORIES applies only to categorical dependent variables. If the dependent variable is scale, the subcommand is ignored and a warning is issued.

Example

```
TREE risk [n] BY income [o] age [s] creditscore [s] employment [n]
  /DEPCATEGORIES USEVALUES=[VALID 99] TARGET=[1].
```

USEVALUES keyword

USEVALUES controls the categories of the dependent variable included in the model.

- The keyword is followed by an equals sign (=) and a list of values enclosed in square brackets, as in: `USEVALUES=[1 2 3]`.
- At least two values must be specified.
- Any cases with dependent variable values not included in the list are excluded from the analysis.
- Values can be string or numeric but must be consistent with the data type of the dependent variable. String and date values must be quoted. Date values must be consistent with the variable's print format.
- If the dependent variable is nominal, the list can include user-missing values. If the dependent variable is ordinal, any user-missing values in the list are excluded and a warning is issued.

- The keywords VALID and MISSING can be used to specify all valid values and all user-missing values respectively. For example, USEVALUES=[VALID MISSING] will include all cases with valid or user-missing values for the dependent variable.
- A warning is issued if a specified category does not exist in the data or in the training sample if split-sample validation is in effect. See the topic “VALIDATION Subcommand ” on page 2003 for more information.

TARGET Keyword

The TARGET keyword specifies categories of the dependent variable that are of primary interest in the analysis. For example, if you are trying to predict mortality, "death" would be defined as the target category. If you are trying to identify individuals with high or medium credit risk, you would define both "high credit risk" and "medium credit risk" as target categories.

- The keyword is followed by an equals sign (=) and a list of values enclosed in square brackets, as in: TARGET=[1] .
- There is no default target category. If not specified, some classification rule options and gains-related output are not available.
- Values can be string or numeric but must be consistent with the data type of the dependent variable. String and date values must be quoted. Date values must be consistent with the variable’s print format.
- If USEVALUES is also specified, the target categories must also be included (either implicitly or explicitly) in the USEVALUES list.
- If a target category is user-missing and the dependent variable is not nominal, the target category is ignored and a warning is issued.

TREE Subcommand

The TREE subcommand allows you to specify the appearance of the tree or suppress it altogether. You can control its orientation, node contents, and the labeling of splits and nodes. Each keyword is followed by an equals sign (=) and the value for that keyword.

Example

```
TREE risk BY income age creditscore
  /TREE DISPLAY=LEFTTORIGHT NODES=BOTH BRANCHSTATISTICS=NO.
```

DISPLAY Keyword

The DISPLAY keyword controls the orientation the tree and can also be used to suppress the display of the tree. The following alternatives are available:

TOPDOWN. *Tree grows top down*. The root node appears at the top of the diagram. The tree grows downward. This is the default.

LEFTTORIGHT. *Tree grows from left to right*.

RIGHTTOLEFT. *Tree grows from right to left*.

NONE. *Tree diagram is not generated*.

NODES Keyword

The NODES keyword controls the contents of tree nodes.

STATISTICS. *Nodes display summary statistics*. This is the default. For a categorical dependent variable, percentages and counts are shown for each category as well as the total count and percentage. (Total percentage is the percentage of cases in the sample assigned to the node. If you specify prior probabilities, the total percentage is adjusted to account for those priors.)

For a scale dependent variable, nodes display mean, standard deviation, number of cases, and predicted value.

CHART. *Nodes display a summary chart*. For a categorical dependent variable, nodes display a bar chart of percentages for each category. For a scale dependent variable a histogram is shown that summarizes the distribution of the dependent variable.

BOTH. *Statistics and chart are shown*.

BRANCHSTATISTICS Keyword

The BRANCHSTATISTICS keyword controls whether branch (split) statistics are shown in the tree.

- For CHAID and Exhaustive CHAID, branch statistics include *F* value (for scale dependent variables) or chi-square value (for categorical dependent variables), as well as the significance value and degrees of freedom.
- For CRT, the improvement value is shown.
- For QUEST, *F*, the significance value, and degrees of freedom are shown for scale and ordinal independent variables. For nominal independent variables, chi-square, the significance value, and degrees of freedom are shown.

YES. *Branch statistics are displayed.* This is the default.

NO. *Branch statistics are not displayed.*

NODEDEFS Keyword

The NODEDEFS keyword controls whether descriptive labels are shown for each node. Node labels show the independent variable (predictor) values assigned to each node.

YES. *Node labels are displayed.* This is the default.

NO. *Node labels are not displayed.*

SCALE Keyword

The SCALE keyword controls the initial scale of the tree diagram.

AUTO. *Adjust the scale according to the number of nodes and levels of tree growth.* The scale is reduced automatically for large trees. This is the default.

value. *User-specified percentage value.* The value can be between 5 and 200, inclusive.

PRINT Subcommand

The PRINT subcommand controls the display of optional tabular output. If the PRINT subcommand is specified, only the output explicitly requested will be produced. The subcommand name is followed by an equals sign (=) and one or more of the following options:

MODELSUMMARY. *Model summary.* The summary includes the method used, variables included in the model, and variables specified but not included in the model. Shown by default if subcommand omitted.

IMPORTANCE. *Predictor importance.* Ranks each independent (predictor) variable according to its importance to the model. IMPORTANCE can be specified only for the CRT method; otherwise it is ignored, and a warning is issued.

SURROGATES. *Surrogate predictors.* Lists surrogate predictors for each split in the tree. Each node is split based on an independent (predictor) variable; for cases in which the value for that predictor is missing, other predictors having high associations with the original predictor are used for classification. These alternative predictors are called surrogates. SURROGATES can be specified only for the CRT and QUEST methods. If surrogates are not computed, this keyword is ignored, and a warning is issued.

CLASSIFICATION. *Classification table.* Shows the number of cases classified correctly and incorrectly for each category of the dependent variable. A warning is issued if CLASSIFICATION is specified for a scale dependent variable. Shown by default for categorical dependent variables if subcommand omitted.

RISK. *Risk estimate and its standard error.* A measure of the tree's predictive accuracy. For categorical dependent variables, the risk estimate is the proportion of cases correctly classified after adjustment for

prior probabilities and misclassification costs. For scale dependent variables, the risk estimate is within-node variance. Shown by default if subcommand omitted.

CATEGORYSPECS. *Cost, prior probability, score, and profit values used in the analysis.* This keyword is ignored if the dependent variable is a scale variable.

TREETABLE. *Tree model in tabular format.* Summary information for each node in the tree, including parent node number, branch statistics, independent variable value(s) for the node, mean and standard deviation for scale dependent variables or counts and percents for categorical dependent variables.

CPS. *Case processing summary.* Shown by default if subcommand omitted.

NONE. *None of the output controlled by the PRINT subcommand is generated.* An error occurs if any other PRINT keyword is specified with NONE.

GAIN Subcommand

The GAIN subcommand controls the display of tables that summarize gain and index values and other measures that indicate the relative performance of nodes in the tree. You can obtain tables that summarize results by node, percentile group, or both. Each keyword is followed by an equals sign (=) and the value for that keyword.

Example

```
TREE risk BY income age creditscore
  /GAIN SUMMARYTABLE=YES TYPE=[NODE, PTILE].
```

SUMMARYTABLE Keyword

The SUMMARYTABLE keyword controls whether a summary table of index values is shown.

- For scale dependent variables, the table includes node number, number of cases, and mean value of the dependent variable.
- For categorical dependent variables, the table includes node number, number of cases, average profit, and ROI (return on investment) values.
- For categorical dependent variables, the summary table is not displayed unless profits have been specified on the PROFITS subcommand.

YES. *Display summary table*. This is the default for scale dependent variables if the subcommand is omitted or SUMMARYTABLE=NO is not included on the subcommand.

NO. *The summary table is not displayed.*

CATEGORYTABLE Keyword

The CATEGORYTABLE keyword displays gain and index values for target categories of the dependent variable.

- The dependent variable must be categorical.
- Target categories must be specified on the DEPCATEGORIES subcommand.
- A separate table is generated for each target category.
- If the dependent variable is scale or no target categories are specified, the keyword is ignored and a warning is issued.

YES. *Display a table of gain statistics for each target category of the dependent variable.* The table includes percentage gain, response percent, and index percent (lift) by node or percentile group. This is the default.

NO. No category table is shown.

TYPE Keyword

The TYPE keyword specifies the type of gain table(s) that are produced. The keyword is followed by an equals sign (=) and one or both of the following types enclosed in square brackets:

NODE. *Terminal nodes.* Rows of the table represent terminal nodes. This is the default.

PTILE. *Percentile.* Each row of the table summarizes a percentile group.

SORT Keyword

The SORT keyword specifies how rows are sorted in all gain tables.

DESCENDING. *Rows are sorted in descending order of index value.* This is the default.

ASCENDING. *Rows are sorted in ascending order of index value.*

INCREMENT Keyword

The INCREMENT keyword controls the width of percentile groupings for a percentile gain table (TYPE=[PTILE]).

- The default increment is 10, which gives decile intervals.
- You can specify one of the following percentage increments: 1, 2, 5, 10, 20, or 25. If a different value is specified, a warning is issued and the default increment is used.
- INCREMENT is ignored for node-by-node gain tables.

CUMULATIVE Keyword

The CUMULATIVE keyword specifies whether node-by-node tables (TYPE=[NODE]) include cumulative statistics. It is ignored for percentile gain tables, which are always cumulative.

YES. *Cumulative statistics are displayed.* This is the default.

NO. *Cumulative statistics are not displayed.*

PLOT Subcommand

The PLOT subcommand generates charts that may help you evaluate your tree model. The subcommand name is followed by an equals sign (=) and any or all of the following chart types:

GAIN. *Line chart of percentage gain.* The chart shows percentage gain for each target category of the dependent variable. GAIN is available only for categorical dependent variables with target categories defined on the DEPCATEGORIES subcommand.

RESPONSE. *Line chart of response.* The chart shows percentage of responses for each target category of the dependent variable. RESPONSE is available only for categorical dependent variables with target categories defined on the DEPCATEGORIES subcommand.

INDEX. *Line chart of index percent.* The chart shows index percent (lift). INDEX is available only for categorical dependent variables with target categories defined on the DEPCATEGORIES subcommand.

MEAN. *Line chart of mean value for scale dependent variables.* If the dependent variable is categorical, this keyword is ignored and a warning is issued.

PROFIT. *Line chart of average profit for categorical dependent variables.* If profits are not defined or the dependent variable is scale, this keyword is ignored and a warning is issued. See the topic "PROFITS Subcommand " on page 2009 for more information.

ROI. *Line chart of ROI (return on investment).* ROI is computed as the ratio of profits to expenses. If profits are not defined or the dependent variable is scale, this keyword is ignored and a warning is issued.

IMPORTANCE. *Bar chart of model importance by predictor.* IMPORTANCE is available for CRT models only; otherwise the keyword is ignored and a warning is issued.

Example

```
TREE risk [o] BY income age creditscore
 /METHOD TYPE=CRT
 /DEPCATEGORIES TARGET=[1]
 /PLOT=IMPORTANCE GAIN INDEX INCREMENT=5.
```

INCREMENT Keyword

The INCREMENT keyword controls the width of percentile group charts.

- The keyword is followed by an equals sign (=) and one of the following values: 1, 2, 5, 10, 20, or 25. If a different value is specified a warning is issued and the default increment is used.
- INCREMENT is ignored for the predictor importance chart.

RULES Subcommand

The RULES subcommand generates syntax that can be used to select or classify cases based on values of independent (predictor) variables.

- You can generate rules for all nodes, all terminal nodes, the top *n* terminal nodes, terminal nodes that correspond to the top *n* percent of cases, or nodes with index values that meet or exceed a cutoff value.
- Rules are available in three different forms: internal command syntax format, SQL, and generic (plain English pseudocode).
- You can specify an external destination file for the rules.
- Each keyword is followed by an equals sign (=) and the value for that keyword.

Example

```
TREE risk [o] BY income age creditscore
 /RULES NODES=TERMINAL SYNTAX=INTERNAL TYPE=SCORING
  OUTFILE='/jobfiles/treescores.sps'.
```

NODES Keyword

The NODES keyword specifies the scope of generated rules. Specify one of the following alternatives:

TERMINAL. *Generates rules for each terminal node.* This is the default.

ALL. *Generates rules for all nodes.* Rules are shown for all parent and terminal nodes.

For categorical dependent variables with defined target categories, the following additional alternatives are available:

TOPN(value). *Generates rules for the top* n *terminal nodes based on index values.* The number must be a positive integer, enclosed in parentheses. If the number exceeds the number of nodes in the tree, a warning is issued and rules are generated for all terminal nodes.

TOPPCT(value). *Generates rules for terminal nodes for the top* n *percent of cases based on index values.* The percent value must be a positive number greater than zero and less than 100, enclosed in parentheses.

MININDEX(value). *Generates rules for all terminal nodes with an index value greater than or equal to the specified value.* The value be a positive number, enclosed in parentheses.

SYNTAX Keyword

The SYNTAX keyword specifies the syntax of generated rules. It determines the form of the selection rules in both output displayed in the Viewer and selection rules saved to an external file.

INTERNAL. *Command syntax language.* Rules are expressed as a set of commands that define a filter condition that can be used to select subsets of cases (this is the default) or as COMPUTE statements that can be used to score cases (with TYPE=SCORING).

SQL. *SQL.* Standard SQL rules are generated to select/extract records from a database or assign values to those records. The generated SQL rules do not include any table names or other data source information.

GENERIC. *Plain English pseudocode*. Rules are expressed as a set of logical "if...then" statements that describe the model's classifications or predictions for each node.

TYPE Keyword

The TYPE keyword specifies the type of SQL or internal command syntax rules to generate. It is ignored if generic rules are requested.

SCORING. *Scoring of cases.* The rules can be used to assign the model's predictions to cases that meet node membership criteria. A separate rule is generated for each node within the scope specified on the NODES keyword. This is the default.

SELECTION. *Selection of cases.* The rules can be used to select cases that meet node membership criteria. For internal command syntax and SQL rules, a single rule is generated to select all cases within the scope specified on the NODES keyword.

Note: For internal command syntax or SQL rules with NODES=TERMINAL and NODES=ALL, TYPE=SELECTION will produce a rule that effectively selects every case included in the analysis.

SURROGATES Keyword

For CRT and QUEST, the SURROGATES keyword controls whether rules use surrogate predictors to classify cases that have missing predictor values. The keyword is ignored if the method is not CRT or QUEST or if generic rules are requested.

Rules that include surrogates can be quite complex. In general, if you just want to derive conceptual information about your tree, exclude surrogates. If some cases have incomplete predictor data and you want rules that mimic your tree, include surrogates.

INCLUDE. *Include surrogates.* Surrogate predictors are used in generated rules. This is the default.

EXCLUDE. *Exclude surrogates.* Rules exclude surrogate predictors.

LABELS Keyword

The LABELS keyword specifies whether value and variable labels are used in generic decision rules.

- By default, any defined value and variable labels are used. When labels aren't available, values and variable names are used.
- LABELS is ignored for SQL and internal command syntax rules.

YES. *Any defined value and variable labels are used in generic rules.* This is the default.

NO. *Values and variable names are used instead of labels.*

OUTFILE Keyword

OUTFILE writes the rules to an external text file.

- The keyword is followed by an equals sign (=) and a file specification enclosed in quotes.
- If the file specification includes a path, an error will result if the specified directory/folder location doesn't exist.
- For command syntax, the file can be used as a command syntax file in both interactive and batch modes.
- For SQL syntax, the generated SQL does not include any table names or other data source information.
- OUTFILE is ignored if NODES=NONE, and a warning is issued.

SAVE Subcommand

The SAVE subcommand writes optional model variables to the active dataset.

- Specify one or more model variables.
- Optionally, you can specify new variable names in parentheses after each model variable name. The names must be unique, valid variable names.
- If new names are not specified, or if specified names are not valid variable names, unique variable names are automatically generated using the model variable name with a suffix.

NODEID. *Node identifier.* The terminal node to which a case is assigned. The value is the tree node number.

PREDVAL. *Predicted value.* The class or value predicted by the model.

PREDPROB. *Predicted probability.* The probability associated with the model's prediction. One variable is saved for each category of the dependent variable. If you specify an optional variable name, it is used as a root for generated variable names. PREDPROB is ignored if the dependent variable is a scale variable. Probabilities are saved only for the first 126 categories of the dependent variable when the CRT or QUEST algorithms are used.

ASSIGNMENT. *Sample assignment.* The variable indicates whether a case was used in the training or testing sample. Cases in the training sample have a value of 1, and cases in the testing sample have a value of 0. ASSIGNMENT is ignored if split-sample validation is not specified.

Example

```
TREE risk [o] BY income age creditscore
 /SAVE NODEID(node_number) PREDVAL(predicted_value).
```

METHOD Subcommand

The METHOD subcommand specifies the growing method and optional parameters. Each keyword is followed by an equals sign (=) and the value for that keyword.

Example

```
TREE risk [o] BY income age creditscore
 /METHOD TYPE=CRT MAXSURROGATES=2 PRUNE=SE(0).
```

TYPE Keyword

TYPE specifies the growing method. For CRT and QUEST, splits are always binary. CHAID and Exhaustive CHAID allow multiway splits.

CHAID. *Chi-squared Automatic Interaction Detection.* At each step, CHAID chooses the independent (predictor) variable that has the strongest interaction with the dependent variable. Categories of each predictor are merged if they are not significantly different with respect to the dependent variable. This is the default method.

EXHAUSTIVECHAID. *Exhaustive CHAID.* A modification of CHAID that examines all possible ways of merging predictor categories.

CRT. *Classification and Regression Trees.* CRT splits the data into segments that are as homogeneous as possible with respect to the dependent variable.

QUEST. *Quick, Unbiased, Efficient Statistical Tree.* A method that is fast and avoids other methods' bias in favor of predictors with many categories. QUEST can be specified only if the dependent variable is nominal. An error occurs if the dependent variable is ordinal or scale.

MAXSURROGATES Keyword

CRT and QUEST can use surrogates for independent (predictor) variables. For cases in which the value for that predictor is missing, other predictors having high associations with the original predictor are used for classification. These alternative predictors are called surrogates. The MAXSURROGATES keyword specifies the maximum number of surrogate predictors to compute.

If the growing method is CHAID or EXHAUSTIVECHAID, this keyword is ignored and a warning is issued.

AUTO. *The maximum is the number of independent variables minus one.* This is the default.

value. *User-specified value.* The value must be a non-negative integer that is less than the number of independent variables in the model. If you don't want to use surrogates in the model, specify MAXSURROGATES=0. If the value equals or exceeds the number of independent variables, the setting is ignored and a warning is issued.

PRUNE Keyword

For CRT and QUEST, the tree can be automatically pruned. Pruning can help avoid creating a tree that overfits the data. If you request pruning, the tree is grown until stopping criteria are met. Then it is trimmed automatically according to the specified criterion.

If the growing method is CHAID or EXHAUSTIVECHAID, PRUNE is ignored and a warning is issued.

NONE. The tree is not pruned. This is the default.

SE(value). *Prune tree using standard error criterion.* The procedure prunes down to the smallest subtree with a risk value within a specified number of standard errors of that of the subtree with the minimum risk. You can specify the number of standard errors in parentheses. The default is 1. The value must be nonnegative. To obtain the subtree with the minimum risk, specify 0.

GROWTHLIMIT Subcommand

The GROWTHLIMIT subcommand specifies stopping criteria that limit the size of the tree. Each keyword is followed by an equals sign (=) and the value for that keyword.

Example

```
TREE risk [o] BY income age creditscore
 /METHOD TYPE=CRT
 /GROWTHLIMIT MAXDEPTH=4 MINCHILDSIZE=10.
```

MAXDEPTH Keyword

MAXDEPTH specifies the maximum number of levels of growth beneath the root node. You can change the maximum depth to adjust the size of the tree.

AUTO. *Three levels for CHAID and Exhaustive CHAID, five levels for CRT and QUEST.* This is the default.

value. *User-specified value.* The value must be a positive integer.

MINPARENTSIZE Keyword

MINPARENTSIZE specifies the minimum number of cases required to split a node. Nodes with fewer cases are not split. You can use this setting to avoid splitting nodes that have few cases.

- The size value must be a positive integer. The default is 100.
- MINPARENTSIZE must be greater than MINCHILDSIZE.

MINCHILDSIZE Keyword

MINCHILDSIZE specifies the minimum number of cases in any child node. A node will not be split if any of the resulting child nodes would have fewer cases than the specified value.

- The size value must be a positive integer. The default is 50.
- MINCHILDSIZE must be less than MINPARENTSIZE.

MINIMPROVEMENT Keyword

For CRT, you can use MINIMPROVEMENT to specify the minimum decrease in impurity. The CRT growing method attempts to maximize within-node homogeneity. In other words, a terminal node in which all cases have the same value for the dependent variable is a homogeneous, "pure" node. A node is not split if impurity would decrease less than the specified value.

The improvement value must be positive value. The default is 0.0001. As the value increases, the number of nodes in the tree tends to decrease.

VALIDATION Subcommand

The VALIDATION subcommand allows you to assess how well your tree structure generalizes to a larger population.

- Split-sample validation and cross-validation are available.

- By default, validation is not performed.
- If you want to be able to reproduce validated results later, use SET SEED before the TREE procedure to initial the random number seed.

Each keyword in the subcommand is followed by an equals sign (=) and the value for that keyword.

Example

```
TREE risk [o] BY income age creditscore
 /VALIDATION TYPE=SPLITSAMPLE(25) OUTPUT=TESTSAMPLE.
```

TYPE Keyword

NONE. *The tree model is not validated.* This is the default.

SPLITSAMPLE(percent). *Split-sample validation.* The model is generated using a training sample and tested on a hold-out sample. The value or variable specified in parentheses determines the training sample size. Enter a percent value greater than 0 and less than 100, or a numeric variable, the values of which determine how cases are assigned to the training or testing samples: cases with a value of 1 for the variable are assigned to the training sample, and all other cases are assigned to the testing sample. The variable cannot be the dependent variable, weight variable, influence variable or a forced independent variable. See the topic "INFLUENCE Subcommand " on page 2010 for more information. *Note*: Split-sample validation should be used with caution on small data files (data files with a small number of cases). Small training sample sizes may yield poor models since there may not be enough cases in some categories to adequately grow the tree.

CROSSVALIDATION (value). *Crossvalidate the tree model.* The sample is divided into a number of subsamples, or **folds**. Tree models are then generated excluding the data from each subsample in turn. The first tree is based on all of the cases *except* those in the first sample fold, the second tree is based on all of the cases except those in the second sample fold, and so on. For each tree, misclassification risk is estimated by applying the tree to the subsample excluded in generating it. Specify a positive integer between 2 and 25 in parentheses. The higher the value, the fewer the number of cases excluded for each tree model. Crossvalidation produces a single, final tree model. The cross-validated risk estimate for the final tree is calculated as the average of the risks for all of the trees. CROSSVALIDATION is ignored with a warning if FORCE is also specified.

OUTPUT Keyword

With split-sample validation, the OUTPUT keyword controls the output generated. This setting is ignored if SPLITSAMPLE is not specified.

BOTHSAMPLES. *Output is produced for training and test samples.* This is the default. Choose this option if you want to compare results for each partition.

TESTSAMPLE. *Output is produced for the test sample only.* Choose this option if you want validated results only.

CHAID Subcommand

The CHAID subcommand sets parameters for a CHAID tree. Except where noted, all parameters also apply to Exhaustive CHAID. It is ignored for CRT and QUEST trees, and an warning is issued.

Each keyword in the subcommand is followed by an equals sign (=) and the value for that keyword.

Example

```
TREE risk [o] BY income age creditscore
 /METHOD TYPE=CHAID
 /CHAID ALPHASPLIT=.01 INTERVALS=age income (10) creditscore (5).
```

ALPHASPLIT Keyword

The ALPHASPLIT keyword specifies the significance level for splitting of nodes. An independent variable will not be used in the tree if significance level for the split statistic (chi-square or *F*) is less than or equal to specified value.

- Specify a value greater than zero and less than 1.
- The default value is 0.05.

ALPHAMERGE Keyword

The ALPHAMERGE keyword specifies the significance level for merging of predictor categories. Small values tend to result in a greater degree of merging.

- Specify a value greater than zero and less than or equal to 1.
- The default value is 0.05.
- If you specify a value of 1, predictor categories are not merged.
- ALPHAMERGE is available only for the CHAID method. For Exhaustive CHAID, the keyword is ignored, and a warning is issued.

SPLITMERGED Keyword

The SPLITMERGED keyword specifies whether predictor categories that are merged in a CHAID analysis are allowed to be resplit.

NO. *Merged predictor categories cannot be resplit.* This is the default.

YES. *Merged predictor categories can be resplit.*

CHISQUARE Keyword

For nominal dependent variables, the CHISQUARE keyword specifies the chi-square measure used in CHAID analysis. For ordinal and scale dependent variables, the keyword is ignored and a warning is issued.

PEARSON. *Pearson chi-square.* This is the default.

LR. *Likelihood-ratio chi-square.*

CONVERGE Keyword

For nominal and ordinal dependent variables, the CONVERGE keyword specifies the convergence value for estimation of the CHAID model.

- Specify a value greater than zero and less than 1.
- The default value is 0.05.
- If the dependent variable is nominal or scale, this keyword is ignored, and a warning is issued.

MAXITERATIONS Keyword

For nominal and ordinal dependent variables, the MAXITERATIONS keyword specifies the maximum number of iterations for estimation of the CHAID model.

- Specify a positive integer value.
- The default value is 100.
- If the dependent variable is nominal or scale, this keyword is ignored, and a warning is issued.

ADJUST Keyword

The ADJUST keyword specifies how to adjust significance values for multiple comparisons.

BONFERRONI. *Significance values are adjusted using the Bonferroni method.* This is the default.

NONE. *Significance values are not adjusted.*

INTERVALS Keyword

In CHAID analysis, scale independent (predictor) variables are always banded into discrete groups (for example, 0-10, 11-20, 21-30, and so on) prior to analysis. You can use the INTERVALS keyword to control the number of discrete intervals for scale predictors.

- By default, each scale predictor is divided into 10 intervals that have approximately equal numbers of cases.

- The INTERVALS keyword is ignored if the model contains no scale independent variables.

value. *The specified value applies to all scale predictors.* For example: INTERVALS=5. The value must be a positive integer less than or equal to 64.

varlist (value). *The specified value applies to the preceding variable list only.* Specify a list of variables followed by the number of intervals in parentheses. Multiple lists can be specified. For example: `INTERVALS=age income (10) creditscore (5).` The value must be a positive integer less than or equal to 64.

For the `varlist (value)` form:

- If a variable in the list is not a scale variable and/or the variable is not specified as an independent variable, the interval specification is ignored for that variable, and a warning is issued.
- If a variable appears in more than one list, the last specification is used.
- For any scale variables not include in the list(s), the default number of intervals (10) is used.

CRT Subcommand

The CRT subcommand sets parameters for the CRT method. For other tree growing methods, the subcommand is ignored, and a warning is issued.

Each keyword in the subcommand is followed by an equals sign (=) and the value for that keyword.

Example

```
TREE risk [o] BY income age creditscore
 /METHOD TYPE=CRT
 /CRT IMPURITY=TWOING MINIMPROVEMENT=.001.
```

IMPURITY Keyword

For categorical dependent variables, the IMPURITY keyword specifies the impurity measure used.

GINI. Gini is based on squared probabilities of membership for each category of the dependent variable. It reaches its minimum (zero) when all cases in a node fall into a single category. This is the default.

TWOING. Twoing groups categories of the dependent variable into the two best subclasses and computes the variance of the binary variable indicating the subclass to which each case belongs.

ORDEREDTWOING. A variant of twoing for ordinal dependent variables in which only adjacent classes can be combined. This method is available only for ordinal dependent variables only.

For scale dependent variables the least squared deviation (LSD) measure of impurity is used. It is computed as the within-node variance, adjusted for any frequency weights or influence values.

MINIMPROVEMENT Keyword

MINIMPROVEMENT specifies the minimum decrease in impurity for CRT trees. A node is not split if impurity would decrease less than the specified value.

- The default value is 0.0001.
- Specify a positive value. Large improvement values tend to result in smaller trees.

QUEST Subcommand

The ALPHASPLIT keyword on the QUEST subcommand sets the alpha criterion for selection of independent (predictor) variables. Specify a value greater than 0 and less than 1. The default is 0.05.

Example

```
TREE risk [n] BY income age creditscore
 /METHOD TYPE=QUEST
 /QUEST ALPHASPLIT=.01.
```

COSTS Subcommand

For categorical dependent variables, the COSTS subcommand allows you to specify penalties for misclassifying cases.

- Costs are used in node assignment and risk estimation for all trees.
- They also affect the tree-growing process for all trees except QUEST trees and CRT trees that use a twoing measure. To include cost information in the tree-growing process for such trees, use adjusted priors. See the topic "PRIORS Subcommand " on page 2007 for more information.
- The COST subcommand is ignored if the dependent variable is scale, and a warning is issued.

The subcommand name is followed by an equals sign (=) and one of the following alternatives:

EQUAL. *Misclassification costs are equal across categories of the dependent variable.* Cost is set to 1 for all possible misclassifications. This is the default.

CUSTOM. *User-specified costs.* Use this to specify costs for one or more combinations of actual and predicted categories. Any combination for which a cost is not specified defaults to 1.

Custom Costs

For each combination of dependent variable categories:

- Specify the actual category, predicted category, and misclassification cost, in that order.
- The cost value must be enclosed in square brackets.
- At least one cost specification must be provided, but you don't have to specify all possible combinations. Any combination for which a cost is not specified defaults to 1.
- If multiple cost specifications are provided for a particular combination of predicted and actual categories, the last one is used.
- Costs specified for categories that do not exist in the data are ignored, and a warning is issued.
- Cost values must be non-negative.
- Category values must be consistent with the data type of the dependent variable. String and date values must be quoted. Date values must be consistent with the variable's print format.
- Correct classifications, where the predicted category and the actual category match, are always assigned a cost of zero. A warning is issued if a nonzero cost is specified for a correct classification. For example, `COSTS CUSTOM=1 1 [3]` would be ignored and the cost would be set to 0.

Example

```
TREE risk [o] BY age income employment
 /COSTS CUSTOM=3 2 [2]
                3 1 [8].
```

- Assuming that the dependent variable is coded 1=*low*, 2=*medium*, and 3=*high*, the cost of misclassifying a high-risk individual as medium risk is 2.
- The cost of misclassifying a high-risk individual as low risk is 8.
- All other misclassifications, such as classifying a medium-risk individual as high risk, are assigned the default cost, 1.
- Correct classifications such as classifying a high-risk individual as high risk are always assigned a cost of 0.

PRIORS Subcommand

For CRT and QUEST trees with categorical dependent variables, the PRIORS subcommand allows you to specify prior probabilities of group membership. **Prior probabilities** are estimates of the overall relative frequency for each target category of the dependent variable prior to knowing anything about the values

of the independent (predictor) variables. Using prior probabilities helps correct any tree growth caused by data in the sample that is not representative of the entire population.

- If the growing method is CHAID or Exhaustive CHAID, this subcommand is ignored and a warning is issued.
- If the dependent variable is scale, this subcommand is ignored and a warning is issued.

FROMDATA. *Obtain priors from the training sample.* Use this setting if the distribution of groups in the training sample is representative of the population distribution. This is the default. If you have not specified a training sample using the VALIDATION subcommand, then the distribution of values in the entire data file is used.

EQUAL. *Equal priors across categories.* Use this if categories of the dependent variable are represented equally in the population. For example, if there are four categories, approximately 25% of the cases are in each category.

CUSTOM. *User-specified prior probabilities.* For each category of the dependent variable, specify the category followed by a non-negative prior probability value enclosed in square brackets.

- Prior probabilities must be specified for all values of the dependent variable included in the analysis (either all non-missing values found in the data or all values defined on the DEPCATEGORIES subcommand).
- The values must be non-negative.
- The specified category values must be consistent with the data type of the dependent variable. String and date values must be quoted. Date values must be consistent with the variable's print format.
- If you specify the same category more than once, the last one is used.
- A warning is issued if you specify a prior probability for a category that does not exist in the data or in the training sample if split-sample validation is in effect. See the topic "VALIDATION Subcommand " on page 2003 for more information.
- Prior probability values are "normalized" to relative proportions before the tree is grown.

Example

```
TREE risk [o] BY age income employment
 /METHOD TYPE=CRT
 /PRIORS CUSTOM= 1 [30] 2 [75] 3 [45].
```

The prior probabilities of 30, 75, and 45 are normalized to proportions of 0.2, 0.5, and 0.3, respectively.

ADJUST Keyword

The ADJUST keyword specifies whether prior probabilities are adjusted to take into account misclassification costs.

NO. *Priors are unadjusted.* This is the default.

YES. *Priors are adjusted to take into account misclassification costs.*

SCORES Subcommand

For CHAID or Exhaustive CHAID trees with ordinal dependent variables, the SCORES subcommand allows you to specify scores that are used to scale the dependent variable. By default, scores are assigned in sorted value order: the first category is assigned a score of 1, the second category is assigned a score of 2, and so forth.

- If the growing method is CRT or QUEST, this subcommand is ignored, and a warning is issued.
- If the dependent variable is nominal or scale, this subcommand is ignored, and a warning is issued.

The subcommand name is followed by an equals sign (=) and one of the two following alternatives:

EQUALINCREMENTS. *Scores are ordinal ranks.* The lowest category of the dependent variable is assigned a score of 1, the next highest category is assigned a score of 2, and so on. This is the default.

CUSTOM. *User-specified scores.* For each category of the dependent variable, specify the category value followed by a score in square brackets.

- Scores must be numeric and unique. You cannot assign the same score to more than one category.
- Scores must be specified for all values of the dependent variable included in the analysis (either all non-missing values found in the data or all values defined on the DEPCATEGORIES subcommand).
- Category values must be consistent with the data type of the dependent variable. String and date values must be quoted. Date values must be consistent with the variable's print format.
- If you specify multiple score values for the same category value, the last one is used.
- A warning is issued if you specify a score for a category that does not exist in the data or in the training sample if split-sample validation is in effect. See the topic "VALIDATION Subcommand " on page 2003 for more information.

Example

```
TREE income [o] BY age employment gender region
 /SCORES CUSTOM=1 [1] 2 [2.5] 3 [8].
```

- By default, the CHAID method is used to grow the tree.
- CUSTOM contains one score specification for each category of the ordinal dependent variable.
- Category 1 is assigned a score of 1; category 2 is assigned a score of 2.5; and category 3 is assigned a score of 8. In other words, scores are used to rescale income so there is a smaller gap between categories 1 and 2 than between 2 and 3.

PROFITS Subcommand

For categorical dependent variables, the PROFITS subcommand allows you to assign revenue and expense values to levels of the dependent variable. Revenues and expenses are used in tables and charts that summarize expected profits and return on investment (ROI). Profit is computed as revenue minus expense.

If the dependent variable is scale, this subcommand is ignored and warning is issued.

- The subcommand name is followed by an equals sign (=), the keyword CUSTOM, and a list of dependent variable values, each followed by a pair of values, representing revenues and expenses respectively, enclosed in square brackets.
- Revenue and expense values must be numeric and must be specified for all values of the dependent variable included in the analysis (either all non-missing values found in the data or all values defined on the DEPCATEGORIES subcommand).
- Category values must be consistent with the data type of the dependent variable. String and date values must be quoted. Date values must be consistent with the variable's print format.
- If you specify the same category value more than once, the expense and revenue values for the last one are used.
- A warning is issued if you specify a category of the dependent variable that does not exist in the data or in the training sample if split-sample validation is in effect. See the topic "VALIDATION Subcommand " on page 2003 for more information.

Example

```
TREE income [o] BY age employment gender region
 /PROFITS CUSTOM=1 [.1 .25]
                  2 [.5 .25]
                   3 [15 .25].
```

- Expected revenue and expense values are defined for each income category.
- Revenues vary across groups. For group 1, the lowest income group, expected revenue is 0.10 per individual. For group 2, the middle income group, expected revenue is 0.50 per individual. For group 3, the high income group, expected revenue is 15 per individual.

- The matrix specifies a fixed expense for each group. For each group, expected expense is 0.25 per person.

INFLUENCE Subcommand

The INFLUENCE subcommand defines an optional influence variable that defines how much influence a case has on the tree-growing process. Cases with lower influence values have less influence, cases with higher values have more.

- The variable must be numeric.
- The dependent variable cannot be used as the influence variable.
- The weight variable (see the WEIGHT command) cannot be used as the influence variable.
- For QUEST, this subcommand is ignored and a warning is issued.

OUTFILE Subcommand

The OUTFILE subcommand writes the tree model to an XML file.

TRAININGMODEL='filespec'. *Writes the model to the specified file.* For split-sample validated trees, this is the model for the training sample.

TESTMODEL='filespec'. *Writes the model for the test sample to the specified file.* Available only with /VALIDATION TYPE=SPLITSAMPLE. Ignored for unvalidated and cross-validated models.

Example

```
TREE income [o] BY age employment gender region
 /OUTFILE TRAININGMODEL='/myfolder/model.xml'.
```

MISSING Subcommand

The MISSING subcommand controls the handling of nominal, user-missing, independent (predictor) variable values.

- Handling of ordinal and scale user-missing independent variable values varies between growing methods.
- By default, if the values of all independent variables are system- or user-missing, the case is excluded.
- Handling of nominal dependent variables is controlled by the DEPCATEGORIES subcommand.
- For the dependent variable, cases with system-missing or user-missing ordinal or scale values are always excluded.

NOMINALMISSING Keyword

The NOMINALMISSING keyword controls the treatment of user-missing values of nominal independent (predictor) variables.

MISSING. *Handling of user-missing values of nominal predictor variables depends on the growing method.* This is the default.

VALID. *Treat user-missing values of nominal independent variables as valid values.* User-missing values of nominal independent variables are treated as ordinary values in tree growing and classification.

Method-Dependent Rules

If some, but not all independent variable values are system-or user-missing:

- For CHAID and Exhaustive CHAID, system- and user-missing scale and ordinal independent variable values are included in the tree-growing process as a "floating" category that is allowed to merge with other categories in tree nodes. By default, nominal user-missing values are also handled in this fashion.

- For CRT and QUEST, cases with missing independent variable values are excluded from the tree-growing process but are classified using surrogate predictors, if surrogates are included in the method. By default, nominal user-missing values are also handled in this fashion.

Weight Variables

If the analysis uses frequency weights or an influence variable, all cases with zero, negative, system-missing, or user-missing values of either variable are excluded from the analysis.

TARGETRESPONSE Subcommand

The TARGETRESPONSE subcommand displays a description of the attributes of each terminal node and the percentage of cases in that node with the specified target value. The output includes a table of percentages and cumulative percentages of the target value in each terminal node and a chart of cumulative response rates.

The TARGETRESPONSE subcommand is only executed under the following conditions:

- The subcommand must be explicitly specified. There is no default setting.
- The target variable must be nominal or ordinal.
- Target values must be specified on the DEPCATEGORIES subcommand. See the topic “DEPCATEGORIES Subcommand ” on page 1995 for more information.

The results are sorted in descending order of percentages. The table can be color-coded based on a specified cumulative minimum percentage, with rows at or above the minimum colored green and rows below the minimum colored red. The minimum value is also displayed as a reference line on the chart. If more than one target value is specified, a separate table and chart are displayed for each target value.

You must specify one of the following alternatives:

RATE=NONE. *No minimum percentage.* The table is not color-coded, and there is no reference line on the chart.

RATE=value. *Use the specified percentage value as the minimum.* The value must be greater than 0 and less than 100.

Special Considerations

- The terminal node attributes displayed in the table represent a general description. They are not a substitute for precise rules. See the topic “RULES Subcommand ” on page 2000 for more information.
- For the CHAID and Exhaustive CHAID methods, the attributes displayed in the table do not include any missing data conditions, and terminal nodes that represent the missing categories of a predictor are not included in the output. See the topic “MISSING Subcommand ” on page 2010 for more information.
- For the CRT and QUEST methods, surrogate values are not included in the list of attribute conditions. See the topic “METHOD Subcommand ” on page 2002 for more information.

TSAPPLY

TSAPPLY is available in SPSS Statistics Professional Edition or the Forecasting option.

Note: Square brackets that are shown in the TSAPPLY syntax chart are not used to indicate optional elements. Where indicated, they are required parts of syntax when a list of values is specified, but they may be omitted when a single value is specified. Equals signs (=) that are used in the syntax chart are required elements. The MODEL subcommand is required. All other subcommands are optional.

```
TSAPPLY
```

Global subcommands:

```
/MODELSUMMARY  PRINT=[MODELFIT** RESIDACF RESIDPACF NONE]
               PLOT=[SRSQUARE RSQUARE RMSE MAPE MAE MAXAPE
                     MAXAE NORMBIC RESIDACF RESIDPACF]

/MODELSTATISTICS DISPLAY={YES**}
                         {NO   }
                 MODELFIT=[SRSQUARE** RSQUARE RMSE MAPE MAE
                           MAXAPE MAXAE NORMBIC]

/MODELDETAILS    PRINT=[PARAMETERS RESIDACF RESIDPACF FORECASTS]
                 PLOT=[RESIDACF RESIDPACF]

/SERIESPLOT      OBSERVED FORECAST FIT FORECASTCI FITCI

/OUTPUTFILTER DISPLAY={ALLMODELS**                                    }
                      {[BESTFIT({N=integer  }) WORSTFIT({N=integer  })]}
                                {PCT=percent}           {PCT=percent}
              MODELFIT={SRSQUARE**}
                       {RSQUARE   }
                       {RMSE      }
                       {MAPE      }
                       {MAE       }
                       {MAXAPE    }
                       {MAXAE     }
                       {NORMBIC   }

/SAVE  PREDICTED(rootname)
       LCL(rootname)
       UCL(rootname)
       NRESIDUAL(rootname)

/AUXILIARY REESTIMATE={NO**}  CILEVEL={95**  }  MAXACFLAGS={24**   }
                      {YES }          {number}             {integer}
           SEASONLENGTH=integer

/MISSING USERMISSING={EXCLUDE**}
                     {INCLUDE  }
```

MODEL (local subcommand):

```
/MODEL  FILE=file   {DROP=['ModelID' 'ModelID' ...]}
                    {KEEP=['ModelID' 'ModelID' ...]}
        OUTFILE=file
        OUTPMML=file
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 14.0

- Command introduced.

Example

```
PREDICT THRU YEAR 2006 MONTH 6.
TSAPPLY
  /MODELDETAILS PRINT=FORECASTS
  /MODEL FILE='/models/models.xml'.
```

Overview

The TSAPPLY procedure loads existing time series models from an external file and applies them to data. Models are generated by using the TSMODEL procedure. You can use TSAPPLY to obtain forecasts for series for which new or revised data are available.

Options

Models. By default, parameter estimates saved in the model file are used to produce forecasts. Optionally, model parameters can be reestimated. Reestimation of model parameters has no effect on the model structure. For example, an ARIMA(1,0,1) model will remain so, but the autoregressive and moving-average parameters will be reestimated by using the data in the active dataset. Outliers, if any, are always taken from the model file.

Output. TSAPPLY offers the same output as the TSMODEL procedure. However, autocorrelations and partial autocorrelations are available only if model parameters are reestimated.

Saved Variables. You can save forecasts and their confidence intervals to the active dataset. Fit values and noise residuals are available if model parameters are reestimated.

Missing Values. You can control whether user-missing values are treated as valid or invalid values.

Basic Specification

- The basic specification is a MODEL subcommand that specifies a model file. By default, all models in the file are loaded.
- Default output includes a summary of the distribution of goodness of fit across loaded models and a table of stationary *R*-square, Ljung-Box Q, and number of outliers by model.

Syntax Rules

- The following subcommands are global: MODELSUMMARY, MODELSTATISTICS, MODELDETAILS, SERIESPLOT, OUTPUTFILTER, SAVE, AUXILIARY, and MISSING.
- Each global subcommand is optional, may be used only once, and must appear before any MODEL subcommand.
- The MODEL subcommand must be used at least once.
- An error occurs if a keyword or attribute is specified more than once within a subcommand.
- All subcommands other than MODEL are optional.
- Subcommand names and keywords must be spelled in full.
- Equals signs (=) that are shown in the syntax chart are required.
- An error is issued if any subcommand is empty.

Operations

- Models are applied to variables in the active dataset with the same names as the variables that are specified in the model.
- TSAPPLY honors time intervals and periodicity specified with the DATE command.
- The procedure verifies that date variables that are implied by the current date specification exist and that their values coincide with the date specification within the estimation period. If the data are filtered, values of date variables are verified within the filtered subset of data.
- If you choose to reestimate model parameters, reestimation happens within the currently active USE period. If the model has outliers, the outlier periods must be inside the effective USE period. The USE period is ignored if model parameters are not reestimated.
- The PREDICT command defines the end of the forecast period for tables, charts, and model variables that are produced by TSAPPLY.
- TSAPPLY does not honor the following commands: MODEL NAME, SAVE MODEL, and READ MODEL.

- TSAPPLY does not honor TSET. It does, however, provide options for missing-value handling, setting the width of confidence intervals, and setting the maximum number of lags displayed for autocorrelations.
- The TDISPLAY command does not display models that are saved by using TSAPPLY.

Limitations

- An error occurs if SPLIT FILE is in effect.
- WEIGHT is ignored with a warning.

Examples

This section provides simple examples that are designed to get you started with the TSAPPLY procedure. Further examples that are specific to each subcommand are provided in the subcommand topics.

Forecasting

```
PREDICT THRU YEAR 2006 MONTH 6.
TSAPPLY
  /MODELDETAILS PRINT=FORECASTS
  /MODEL FILE='/models/models.xml'.
```

- The PREDICT command is used to specify the forecast period for the TSAPPLY procedure.
- The MODELDETAILS subcommand specifies that the output includes a table containing the forecasts.
- The MODEL subcommand specifies that all models in the file */models/models.xml* will be applied to the active dataset. Models are applied to variables in the active dataset with the same names as the variables that are specified in the model.
- Forecasts are created by using the model parameters from *models.xml*, without reestimating any of those parameters. This behavior is the default behavior.

Forecasting with Reestimated Model Parameters

```
PREDICT THRU YEAR 2006 MONTH 6.
TSAPPLY
  /MODELDETAILS PRINT=FORECASTS
  /AUXILIARY REESTIMATE=YES
  /MODEL FILE='/models/models.xml'
         OUTFILE='/models/models_updated.xml'.
```

- The AUXILIARY subcommand specifies that model parameters for all models in */models/models.xml* will be reestimated prior to forecasting. Model parameters are reestimated by using the data in the active dataset.
- Reestimation of model parameters has no effect on the model structure. For example, an ARIMA(1,0,1) model will remain so, but the autoregressive and moving-average parameters will be reestimated.
- The OUTFILE keyword specifies that the reestimated models will be saved to the file */models/models_updated.xml*.

Applying a Selected Subset of Models

```
PREDICT THRU YEAR 2006 MONTH 6.
TSAPPLY
  /MODELDETAILS PRINT=FORECASTS
  /MODEL FILE='/models/models.xml' KEEP=['CustomArima_1'].
```

- The KEEP keyword specifies that only the model named *CustomArima_1* will be applied to the active dataset. All other models in */models/models.xml* will be excluded.

```
PREDICT THRU YEAR 2006 MONTH 6.
TSAPPLY
  /MODELDETAILS PRINT=FORECASTS
  /MODEL FILE='/models/models.xml' DROP=['Expert_1' 'Expert_2'].
```

- The DROP keyword specifies that the models named *Expert_1* and *Expert_2* will be excluded. All other models in */models/models.xml* will be applied to the active dataset.

Goodness-of-Fit Measures

The following model goodness-of-fit measures are available in TSAPPLY:

SRSQUARE. *Stationary R-squared*. A measure that compares the stationary part of the model to a simple mean model. This measure is preferable to ordinary R-squared when there is a trend or seasonal pattern. Stationary R-squared can be negative with a range of negative infinity to 1. Negative values mean that the model under consideration is worse than the baseline model. Positive values mean that the model under consideration is better than the baseline model.

RSQUARE. *R-squared*. An estimate of the proportion of the total variation in the series that is explained by the model. This measure is most useful when the series is stationary. R-squared can be negative with a range of negative infinity to 1. Negative values mean that the model under consideration is worse than the baseline model. Positive values mean that the model under consideration is better than the baseline model.

RMSE. *RMSE*. Root Mean Square Error. The square root of mean square error. A measure of how much a dependent series varies from its model-predicted level, expressed in the same units as the dependent series.

MAPE. *MAPE*. Mean Absolute Percentage Error. A measure of how much a dependent series varies from its model-predicted level. It is independent of the units used and can therefore be used to compare series with different units.

MAE. *MAE*. Mean absolute error. Measures how much the series varies from its model-predicted level. MAE is reported in the original series units.

MAXAPE. *MaxAPE*. Maximum Absolute Percentage Error. The largest forecasted error, expressed as a percentage. This measure is useful for imagining a worst-case scenario for your forecasts.

MAXAE. *MaxAE*. Maximum Absolute Error. The largest forecasted error, expressed in the same units as the dependent series. Like MaxAPE, it is useful for imagining the worst-case scenario for your forecasts. Maximum absolute error and maximum absolute percentage error may occur at different series points--for example, when the absolute error for a large series value is slightly larger than the absolute error for a small series value. In that case, the maximum absolute error will occur at the larger series value and the maximum absolute percentage error will occur at the smaller series value.

NORMBIC. *Normalized BIC*. Normalized Bayesian Information Criterion. A general measure of the overall fit of a model that attempts to account for model complexity. It is a score based upon the mean square error and includes a penalty for the number of parameters in the model and the length of the series. The penalty removes the advantage of models with more parameters, making the statistic easy to compare across different models for the same series.

MODELSUMMARY Subcommand

The MODELSUMMARY subcommand controls the display of tables and charts that summarize goodness of fit, residual autocorrelations, and residual partial autocorrelations across estimated models. Each keyword is followed by an equals sign (=) and one or more of the available options enclosed in square brackets.

Example

```
PREDICT THRU YEAR 2006 MONTH 6.
TSAPPLY
  /MODELDETAILS PRINT=FORECASTS
  /AUXILIARY REESTIMATE=YES
  /MODELSUMMARY PLOT=[SRSQUARE MAXAPE]
  /MODEL FILE='/models/models.xml'.
```

- The output includes two histograms: one histogram for stationary *R*-squared and one histogram for the maximum absolute percentage error. Each histogram consists of results across all models.

PRINT Keyword

The PRINT keyword controls the display of model summary tables.

MODELFIT. *Goodness of fit.* Table of summary statistics and percentiles for stationary *R*-square, *R*-square, root mean square error, mean absolute percentage error, mean absolute error, maximum absolute percentage error, maximum absolute error, and normalized Bayesian Information Criterion. Note that if models are not reestimated, model fit statistics are loaded from the model file. This table is displayed by default.

RESIDACF. *Residual autocorrelation function.* Table of summary statistics and percentiles for autocorrelations of the residuals. RESIDACF is ignored with a warning if REESTIMATE=NO.

RESIDPACF. *Residual partial autocorrelation function.* Table of summary statistics and percentiles for partial autocorrelations of the residuals. RESIDPACF is ignored with a warning if REESTIMATE=NO.

NONE. *No tables are shown.* An error occurs if NONE is used in combination with any other PRINT option.

PLOT Keyword

The PLOT keyword controls the display of model summary charts. By default no charts are shown. (For detailed definitions of the following terms, see "Goodness-of-Fit Measures " on page 2016.)

SRSQUARE. *Histogram of Stationary* R-Square.

RSQUARE. *Histogram of* R-Square.

RMSE. *Histogram of Root Mean Square Error.*

MAPE. *Histogram of Mean Absolute Percentage Error.*

MAE. *Histogram of Mean Absolute Error.*

MAXAPE. *Histogram of Maximum Absolute Percentage Error.*

MAXAE. *Histogram of Maximum Absolute Error.*

NORMBIC. *Histogram of Normalized Bayesian Information Criterion (BIC).*

RESIDACF. *Boxplot of Residual Autocorrelation Function by Lag.* RESIDACF is ignored with a warning if REESTIMATE=NO.

RESIDPACF. *Boxplot of Residual Partial Autocorrelation Function by Lag.* RESIDPACF is ignored with a warning if REESTIMATE=NO.

MODELSTATISTICS Subcommand

The MODELSTATISTICS subcommand controls display of a table listing all models along with chosen goodness-of-fit statistics. Note that if models are not reestimated, model fit statistics are loaded from the model file.

Example

```
PREDICT THRU YEAR 2006 MONTH 6.
TSAPPLY
  /MODELDETAILS PRINT=FORECASTS
  /AUXILIARY REESTIMATE=YES
  /MODELSTATISTICS DISPLAY=YES MODELFIT=RSQUARE
  /MODEL FILE='/models/models.xml'.
```

- The output includes a table displaying the value of *R*-squared for each model.

DISPLAY Keyword

The DISPLAY keyword controls whether the model statistics table is shown.

YES. *Model statistics table is shown.* Table of model goodness of fit, Ljung-Box Q statistic, and number of outliers by model. This setting is the default. The Q statistic measures the degree of pattern in the residuals. Large values of Q in relation to its degrees of freedom indicate that model residuals are not randomly distributed.

NO. *Model statistics table is not shown.*

MODELFIT Keyword

The MODELFIT keyword controls which fit statistics are shown in the model statistics table, and this keyword is ignored if DISPLAY=NO. The keyword is followed by an equals sign (=) and one or more of the following options enclosed in square brackets. (For detailed definitions of the following terms, see “Goodness-of-Fit Measures ” on page 2016.)

SRSQUARE. *Stationary* R-*square*. This setting is the default.

RSQUARE. R-*Square.*

RMSE. *Root Mean Square Error.*

MAPE. *Mean Absolute Percentage Error.*

MAE. *Mean Absolute Error.*

MAXAPE. *Maximum Absolute Percentage Error.*

MAXAE. *Maximum Absolute Error.*

NORMBIC. *Normalized Bayesian Information Criterion (BIC).*

MODELDETAILS Subcommand

The MODELDETAILS subcommand controls the display of model parameters, model forecasts, and autocorrelations of noise residuals for individual models. Each keyword is followed by an equals sign (=) and one or more of the available options enclosed in square brackets.

Example

```
PREDICT THRU YEAR 2006 MONTH 6.
TSAPPLY
  /AUXILIARY REESTIMATE=YES
  /MODELDETAILS PRINT=[PARAMETERS FORECASTS]
  /MODEL FILE='/models/models.xml'.
```

- The output includes tables displaying the model parameters and forecasts for each model.

PRINT Keyword

The PRINT keyword controls display of tables for individual models. By default, no tables are shown.

PARAMETERS. *Model parameter estimates.* Shows separate tables for ARIMA and exponential smoothing models. If outliers exist, parameter estimates for outliers are also displayed. Note that if models are not reestimated, parameter estimates are loaded from the model file.

RESIDACF. *Residual autocorrelation function.* Shows residual autocorrelations by lag. RESIDACF is ignored with a warning if REESTIMATE=NO.

RESIDPACF. *Residual partial autocorrelation function.* Shows residual partial autocorrelations by lag. RESIDPACF is ignored with a warning if REESTIMATE=NO.

FORECASTS. *Forecasts and confidence intervals.* Shows model forecasts and confidence intervals.

PLOT Keyword

The PLOT keyword controls display of charts for individual models. By default no charts are shown. PLOT is ignored with a warning if REESTIMATE=NO.

RESIDACF. *Residual autocorrelation function.* Shows residual autocorrelations by lag.

RESIDPACF. *Residual partial autocorrelation function.* Shows residual partial autocorrelations by lag.

SERIESPLOT Subcommand

The SERIESPLOT subcommand allows you to obtain plots of predicted values, observed values, and confidence intervals for each model. By default, no plots are shown. The subcommand is followed by one or more of the following keywords:

OBSERVED. *Displays all observed values of the dependent series.* OBSERVED is ignored with a warning if REESTIMATE=NO.

FORECAST. *Displays model-predicted values within the forecast period.*

FIT. *Displays model-predicted values within the estimation period.* FIT is ignored with a warning if REESTIMATE=NO.

FORECASTCI. *Displays upper and lower confidence limits within the forecast period.*

FITCI. *Displays upper and lower confidence limits within the estimation period.* FITCI is ignored with a warning if REESTIMATE=NO.

Example

```
PREDICT THRU YEAR 2006 MONTH 6.
TSAPPLY
  /MODELDETAILS PRINT=FORECASTS
  /AUXILIARY REESTIMATE=YES
  /SERIESPLOT OBSERVED FORECAST
  /MODEL FILE='/models/models.xml'.
```

- The output includes plots of the model-predicted values in the forecast period, as well as all observed values, for each model.

OUTPUTFILTER Subcommand

The OUTPUTFILTER subcommand controls which models are included in the model statistics table (MODELSTATISTICS subcommand), detailed model output (MODELDETAILS subcommand), and time series plots (SERIESPLOT subcommand).

- By default, all models are included in the output. OUTPUTFILTER can be used to display only the best-fitting or worst-fitting models (or both). If you request both, two sets of output are displayed.
- If model parameters are not reestimated, models are selected based on goodness-of-fit statistics that are loaded from the model file.
- OUTPUTFILTER is ignored if no output is requested on the MODELSTATISTICS, MODELDETAILS, or SERIESPLOT subcommands.
- OUTPUTFILTER has no effect on output from the MODELSUMMARY subcommand.

Example

```
PREDICT THRU YEAR 2006 MONTH 6.
TSAPPLY
  /MODELDETAILS PRINT=FORECASTS
  /OUTPUTFILTER DISPLAY=[BESTFIT(N=5) WORSTFIT(PCT=10)]
  /MODEL FILE='/models/models.xml'.
```

- The output consists of two sets of results: those results for the 5 best-fitting models and those results for models with fit values in the bottom 10%. The stationary *R*-squared value is used as the goodness-of-fit measure.

DISPLAY Keyword

The DISPLAY keyword controls which models are included in MODELSTATISTICS, MODELDETAILS, and SERIESPLOT output.

- By default, all models are displayed (DISPLAY=ALLMODELS).
- To display only those models with extreme goodness-of-fit values, specify BESTFIT, WORSTFIT, or both, in square brackets. One or both keywords must be specified.

ALLMODELS. *All models are displayed.* All models that could be loaded (REESTIMATE=NO) or computed (REESTIMATE=YES) are shown. This setting is the default.

BESTFIT. *Models having the highest fit values are shown.* To display the models with the N highest fit values, specify BESTFIT followed by N=count in parentheses—for example, BESTFIT(N=5). The count must be a positive integer. If the count exceeds the number of estimated models, all models are shown. To display the models with fit values in the top N%, specify PCT=percent in parentheses. For example, BESTFIT(PCT=5) displays models with fit values in the top 5% among estimated models. The percentage value must be greater than zero and less than 100.

WORSTFIT. *Models having the lowest fit values are shown.* To display the models with the N lowest fit values, specify WORSTFIT followed by N=count in parentheses—for example, WORSTFIT(N=5). The count must be a positive integer. If the count exceeds the number of models estimated, all models are shown. To display the models with fit values in the lowest N%, specify PCT=percent in parentheses. For example, WORSTFIT(PCT=5) displays models with fit values in the bottom 5% among estimated models. The percentage value must be greater than zero and less than 100.

MODELFIT Keyword

The MODELFIT keyword specifies the fit measure that is used to filter models. MODELFIT is ignored if DISPLAY=ALLMODELS.

Specify one of the following options. (For detailed definitions of the following terms, see "Goodness-of-Fit Measures " on page 2016.)

SRSQUARE. *Stationary* R-*Square*. This setting is the default.

RSQUARE. R-*Square.*

RMSE. *Root Mean Square Error.*

MAPE. *Mean Absolute Percentage Error.*

MAE. *Mean Absolute Error.*

MAXAPE. *Maximum Absolute Percentage Error.*

MAXAE. *Maximum Absolute Error.*

NORMBIC. *Normalized Bayesian Information Criterion (BIC).*

SAVE Subcommand

The SAVE subcommand is used to save new variables representing predicted values, residuals, and confidence intervals to the active dataset. By default, no new variables are saved to the active dataset.

- Specify one or more keywords, each keyword followed by an optional rootname to be used as the prefix for new variable names. Enclose the root name in parentheses. Each keyword gives rise to one new variable for each dependent variable.
- The root name, if specified, must conform to the rules for valid variable names.
- If no root name is specified, TSAPPLY uses a default name.
- The full variable name is the concatenation of the root name, the name of the associated dependent variable, and a model identifier. The variable name is extended if necessary to avoid variable naming conflicts.
- The PREDICT command controls whether new cases are added to the file when new variables are saved. New cases are added if PREDICT specifies a forecast period that extends beyond the length of the dependent-variable series.
- SAVE is ignored with a warning if temporary transformations are in effect.

Example

```
PREDICT THRU YEAR 2006 MONTH 6.
TSAPPLY
  /AUXILIARY REESTIMATE=YES
```

```
  /SAVE PREDICTED(ApplyPred) NRESIDUAL
  /MODEL FILE='/models/models.xml'.
```

- Two new variables are created for each of the models: one variable contains the model predictions, and the other variable contains the noise residuals.
- The root name (prefix) for the variables containing model predictions is *ApplyPred*.
- The root name for the variables containing noise residuals is *NResidual* (the default).

PREDICTED(rootname). *Model fit and forecast values.* The default root name is *Predicted*. If REESTIMATE=NO, only forecasts are produced.

LCL(rootname). *Lower confidence limits.* The default root name is *LCL*. If REESTIMATE=NO, lower confidence limits are only produced for forecasts.

UCL(rootname). *Upper confidence limits.* The default root name is *UCL*. If REESTIMATE=NO, upper confidence limits are only produced for forecasts.

NRESIDUAL(rootname). *Noise residuals.* The default rootname is *NResidual*. When transformations of the dependent variable are performed (for example, natural log), these residuals are the residuals for the transformed series. NRESIDUAL is ignored with a warning if REESTIMATE=NO.

AUXILIARY Subcommand

The AUXILIARY subcommand is used to specify whether model parameters are reestimated, set the confidence interval level for forecasts, set the maximum number of lags for autocorrelation and partial autocorrelation plots and tables, and specify the season length.

Example

```
PREDICT THRU YEAR 2006 MONTH 6.
TSAPPLY
  /MODELDETAILS PRINT=[PARAMETERS FORECASTS]
                PLOT=[RESIDACF RESIDPACF]
  /AUXILIARY REESTIMATE=YES MAXACFLAGS=36
  /MODEL FILE='/models/models.xml'.
```

REESTIMATE Keyword

REESTIMATE controls whether model parameters are reestimated. If REESTIMATE=NO (the default), parameter estimates that are recorded in the model file are used to produce forecasts. If REESTIMATE=YES, forecasts reflect reestimated model parameters. Outliers, if any, are always taken from the model file.

Note that if REESTIMATE=NO, any parameter estimates or goodness-of-fit statistics that are displayed in output are loaded from the model file and reflect the data that were used when each model was developed (or last updated). And if you use the OUTPUTFILTER subcommand, models will be selected on the basis of goodness-of-fit statistics that are recorded in the model file. In addition, forecasts will not take into account historical data--for either dependent or independent variables--in the active dataset. You must set REESTIMATE=YES if you want historical data to impact the forecasts. Finally, forecasts do not take into account values of the dependent series in the forecast period--but the forecasts do take into account values of independent variables in the forecast period. If you have more current values of the dependent series and want them to be included in the forecasts, you need to reestimate, adjusting the estimation period to include these values.

CILEVEL Keyword

The CILEVEL keyword sets the confidence level.

- Specify a positive number that is less than 100.
- The default is 95.

MAXACFLAGS Keyword

The MAXACFLAGS keyword sets the maximum number of lags displayed in residual autocorrelation and partial autocorrelation tables and plots. MAXACFLAGS is ignored if REESTIMATE=NO.

- Specify a positive integer.
- The default is 24.

SEASONLENGTH Keyword

The SEASONLENGTH keyword specifies the length of the seasonal period (the number of observations in one period or season) for the data.

- To adjust the season length, specify a positive integer.
- If SEASONLENGTH is not specified, the periodicity that is specified by using the DATE command defines the season length.
- Note that SEASONLENGTH is used to override the date specification for the active dataset to make it consistent with the date specification from the model file. An error occurs if these two specifications are not consistent.

MISSING Subcommand

The MISSING subcommand controls the handling of user-missing values.

- By default, user-missing values are treated as missing (invalid) data.
- System-missing values are always treated as invalid.
- Cases with missing values of a dependent variable that occur within the estimation period are included in the model. The specific handling of the missing value depends on the estimation method.
- For ARIMA models, a warning is issued if a predictor has any missing values within the estimation period. Any models involving the predictor are not reestimated.
- If any predictor has missing values within the forecast period, the procedure issues a warning and forecasts as far as it can.

Example

```
PREDICT THRU YEAR 2006 MONTH 6.
TSAPPLY
  /MODELDETAILS PRINT=FORECASTS
  /AUXILIARY REESTIMATE=YES
  /MISSING USERMISSING=INCLUDE
  /MODEL FILE='/models/models.xml'.
```

USERMISSING Keyword

The USERMISSING keyword controls the treatment of user-missing values and is required.

EXCLUDE. *Exclude user-missing values.* User-missing values are treated as missing. This setting is the default.

INCLUDE. *Include user-missing values.* User-missing values are treated as valid data.

MODEL Subcommand

The MODEL subcommand is required and specifies an external file, referred to as a model file, containing models that are developed by using TSMODEL. A model file specifies lag structure, parameters, outliers, and variables for each model.

- TSAPPLY assumes that the variables that are specified in the model file exist in the active dataset.
- You can use multiple MODEL subcommands to apply two or more sets of models to your data. Each MODEL subcommand is used to load models from a different model file. For example, one model file might contain models for series that represent unit sales, and another model file might contain models for series that represent revenue.

FILE Keyword

The FILE keyword is used to specify an external file containing models, and the keyword is required.

- A warning is issued if a model variable that is specified in the file does not exist in the active dataset or is not numeric. Models that refer to the variable are ignored.
- A warning is issued if a model specifies a variable transformation (square root or natural log) and one or more model variables to be transformed contain values that are invalid for the specified transformation--for example, negative values. Any such models will not be applied.
- An error occurs if the date interval or cycle length that is specified in the file is inconsistent with the date specification for the active dataset. The SEASONLENGTH keyword on the AUXILIARY subcommand can be used to override the date specification for the active dataset.
- An error occurs if the file cannot be read or is invalid.
- An error occurs if two or more MODEL subcommands specify the same file.
- When multiple MODEL subcommands are used, a warning occurs if the same model ID occurs in more than one model file. The first model with that ID is loaded. The check for duplicate model IDs takes place after DROP and KEEP are applied to each model file.
- A warning is issued, and forecasts are not available for a model, if REESTIMATE=NO and the model's forecast period is inconsistent with the current USE or PREDICT interval. Specifically, forecasts are not available if model forecasts would start before the beginning of the USE period or after the end of the PREDICT interval.

DROP Keyword

The DROP keyword is used to exclude models.

- Specify one or more quoted model identifiers, enclosed in square brackets—for example, ['Model_1' 'Model_2']. All other models in the file are loaded and applied to the data.
- Redundant items in the DROP list are ignored.
- An error occurs if all models are dropped.
- Model identifiers that do not exist in the file are ignored.

KEEP Keyword

The KEEP keyword specifies models to be applied.

- Specify one or more quoted model identifiers, enclosed in square brackets—for example, ['Model_1' 'Model_2']. All other models in the file are excluded.
- Redundant items in the KEEP list are ignored.
- Model identifiers that do not exist in the file are ignored.
- DROP and KEEP are mutually exclusive. An error occurs if both are specified.

OUTFILE and OUTPMML Keywords

The OUTFILE and OUTPMML keywords are used to save updated models

- OUTFILE saves an XML file that can be used with other IBM SPSS applications.
- OUTPMML saves a PMML-compliant XML file that can be used with PMML-compliant applications, including IBM SPSS applications.
- Models are written to an XML file. Each model is assigned a unique name and a description that includes the name of the dependent variable and the model type.
- The filename must be specified in full. No extension is supplied.
- If two or more MODEL subcommands (within a single invocation of TSMODEL) specify the same external file, all models that are created by those MODEL subcommands are saved to that file.
- OUTFILE and OUTPMML are ignored with a warning if SPLIT FILE is in effect.

TSET

```
TSET
 [PRINT={DEFAULT**}]  [/NEWVAR={CURRENT**}] [/MXAUTO={16**}]
        {BRIEF     }            {NONE     }           {lags}
        {DETAILED  }            {ALL      }

 [/MXCROSS={7**  }]  [/MXNEWVARS={60**}] [/MXPREDICT={60**}]
           {lags}               {n   }              {n   }

 [/MISSING={EXCLUDE**}]  [/CIN={95**  }] [/TOLER={0.0001**}]
           {INCLUDE  }         {value}          {value   }

 [/CNVERGE={0.001**}]  [/ACFSE={IND**}]
           {value  }           {MA   }

 [/PERIOD=n]    [/ID=varname]

 [/{CONSTANT**}]
   {NOCONSTANT}

 [/DEFAULT]
```

**Default if the subcommand is omitted.

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
TSET PERIOD 6 NEWVAR NONE MXAUTO 25.
```

Overview

TSET sets global parameters to be used by procedures that analyze time series and sequence variables. To display the current settings of these parameters, use the TSHOW command.

Basic Specification

The basic specification is at least one subcommand.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- The slash between subcommands is optional.
- You can specify DEFAULT on any subcommand to restore the default setting for that subcommand.
- Subcommand DEFAULT restores all TSET subcommands to their defaults.

Operations

- TSET takes effect immediately.
- Only the settings specified are affected. All others remain at their previous settings or the default.
- Subcommands on other procedures that perform the same function as subcommands on TSET override the TSET specifications for those procedures.
- Procedures that are affected by TSET specifications are CURVEFIT, CASEPLOT, NPPLOT, and TSPLOT.

DEFAULT Subcommand

DEFAULT resets all TSET settings back to their defaults. There are no additional specifications on DEFAULT.

ID Subcommand

ID specifies a variable whose values are used to label observations in plots.

- The only specification on ID is the name of a variable in the active dataset.
- If ID is not specified, the *DATE_* variable is used to label observations.
- If ID is specified within the procedure, it overrides the TSET specification for that procedure.

MISSING Subcommand

MISSING controls the treatment of user-missing values.

- The specification on MISSING is keyword INCLUDE or EXCLUDE. The default is EXCLUDE.
- INCLUDE indicates that observations with user-missing values should be treated as valid values and included in analyses.
- EXCLUDE indicates that observations with user-missing values should be excluded from analyses.

MXNEWVARS Subcommand

MXNEWVARS indicates the maximum number of new variables that can be generated by a procedure.

- The specification on MXNEWVARS indicates the maximum and can be any positive integer.
- The default maximum number is 60 new variables per procedure.

MXPREDICT Subcommand

MXPREDICT indicates the maximum number of new cases that can be added to the active dataset per procedure when the PREDICT command is used.

- The specification on MXPREDICT can be any positive integer.
- The default maximum number of new cases is 60 per procedure.

NEWVAR Subcommand

NEWVAR controls the creation of new variables in the active dataset.

- The specification on NEWVAR can be CURRENT, NONE, or ALL. The default is CURRENT.
- CURRENT specifies to save new variables and replace any existing variables of the same name.
- ALL specifies to save new variables without replacing existing ones.
- NONE specifies that no new variables are saved.

PERIOD Subcommand

PERIOD indicates the size of the period to be used for seasonal differencing.

- The specification on PERIOD indicates how many observations are in one season or period and can be any positive integer.
- There is no default for the PERIOD subcommand.
- The specification on TSET PERIOD overrides the periodicity of DATE variables.

- If a period is specified within an individual procedure, it overrides the TSET PERIOD specification for that procedure.

PRINT Subcommand

PRINT controls how much output is produced.

- The specification on PRINT can be BRIEF, DETAILED, or DEFAULT. The amount of output produced by DEFAULT is generally between the amount produced by BRIEF and DETAILED.
- For procedures with multiple iterations, BRIEF generally means that final statistics are displayed with no iteration history. DEFAULT provides a one-line summary at each iteration in addition to the final statistics. DETAILED provides a complete summary of each iteration (where necessary) plus the final statistics.
- For some procedures, the DEFAULT and DETAILED output is the same. For many of the simpler procedures, BRIEF, DEFAULT, and DETAILED are all the same.

TSHOW

```
TSHOW
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Overview

TSHOW displays a list of all the current specifications on the TSET, USE, PREDICT, and DATE commands.

Basic Specification

The command keyword TSHOW is the only specification.

Operations

- TSHOW is executed immediately.
- TSHOW lists every current specification for the TSET, USE, PREDICT, and DATE commands, as well as the default settings.

Example

```
TSHOW.
```

- The TSHOW command produces a list of the current settings on the TSET, USE, PREDICT, and DATE commands.

TSMODEL

TSMODEL is available in SPSS Statistics Professional Edition or the Forecasting option.

Note: Square brackets that are shown in the TSMODEL syntax chart are not used to indicate optional elements. Where indicated, they are required parts of syntax when a list of values is specified, but they may be omitted when a single value is specified. Equals signs (=) that are used in the syntax chart are required elements. The MODEL subcommand is required. All other subcommands are optional.

```
TSMODEL
```

Global subcommands:

```
/MODELSUMMARY  PRINT=[MODELFIT** RESIDACF RESIDPACF NONE]
               PLOT=[SRSQUARE RSQUARE RMSE MAPE MAE MAXAPE
                     MAXAE NORMBIC RESIDACF RESIDPACF]

/MODELSTATISTICS DISPLAY={YES**}
                         {NO   }
                 MODELFIT=[SRSQUARE** RSQUARE RMSE MAPE MAE
                           MAXAPE MAXAE NORMBIC]

/MODELDETAILS    PRINT=[PARAMETERS RESIDACF RESIDPACF FORECASTS]
                 PLOT=[RESIDACF RESIDPACF]

/SERIESPLOT      OBSERVED FORECAST FIT FORECASTCI FITCI

/OUTPUTFILTER DISPLAY={ALLMODELS**                                    }
                      {[BESTFIT({N=integer  }) WORSTFIT({N=integer  })]}
                                {PCT=percent}           {PCT=percent}
              MODELFIT={SRSQUARE**}
                       {RSQUARE   }
                       {RMSE      }
                       {MAPE      }
                       {MAE       }
                       {MAXAPE    }
                       {MAXAE     }
                       {NORMBIC   }

/SAVE  PREDICTED(rootname)
       LCL(rootname)
       UCL(rootname)
       NRESIDUAL(rootname)

/AUXILIARY CILEVEL={95**  }  MAXACFLAGS={24**   }  SEASONLENGTH=integer
                   {number}             {integer}

/MISSING USERMISSING={EXCLUDE**}
                     {INCLUDE  }
```

Model block subcommands:

```
/MODEL  DEPENDENT=varlist
        INDEPENDENT=varspec list
        OUTFILE=model file
        OUTPMML=model file
        PREFIX='prefix'

 {   /EXPERTMODELER TYPE=[ARIMA** EXSMOOTH**]   }
                    TRYSEASONAL={YES**}
                                {NO   }

 {   /EXSMOOTH TYPE={SIMPLE               }     }
                    {SIMPLESEASONAL       }
                    {HOLT                 }
                    {BROWN                }
                    {DAMPEDTREND          }
                    {WINTERSADDITIVE      }
                    {WINTERSMULTIPLICATIVE}
               TRANSFORM={NONE**}
                         {SQRT  }
                         {LN    }

 {   /ARIMA   AR={[integer integer...]}         }
              ARSEASONAL={[0**]              }
                         {[integer integer...]}
              MA={[integer integer...]}
              MASEASONAL={[0**]              }
                         {[integer integer...]}
              DIFF={0**    }
                   {integer}
              DIFFSEASONAL={0**    }
                           {integer}
              CONSTANT={YES**}
                       {NO   }
              TRANSFORM={NONE**}
                        {SQRT  }
                        {LN    }

 /TRANSFERFUNCTION VARIABLES={ALL**  }
```

```
                              {varlist}
           NUM={[0**]              }
               {[integer integer...]}
           NUMSEASONAL={[0**]                 }
                       {[integer integer...]}
           DENOM={[0**]              }
                 {[integer integer...]}
           DENOMSEASONAL={[0**]                 }
                         {[integer integer...]}
           DIFF={0**    }
                {integer}
           DIFFSEASONAL={0**    }
                        {integer}
           DELAY={0**    }
                 {integer}
           TRANSFORM={NONE**}
                     {SQRT  }
                     {LN    }

/AUTOOUTLIER   DETECT={OFF**}
                      {ON   }
               TYPE=[ADDITIVE** LEVELSHIFT** INNOVATIONAL TRANSIENT
                     SEASONALADDITIVE LOCALTREND ADDITIVEPATCH]

/OUTLIER LOCATION=[date specification] TYPE={ADDITIVE        }
                                            {LEVELSHIFT      }
                                            {INNOVATIONAL    }
                                            {TRANSIENT       }
                                            {SEASONALADDITIVE}
                                            {LOCALTREND      }
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 14.0

- Command introduced.

Example

```
TSMODEL
  /MODEL DEPENDENT=sku1 TO sku10.
```

Overview

The TSMODEL procedure estimates exponential smoothing, univariate Autoregressive Integrated Moving Average (ARIMA), and multivariate ARIMA (or transfer function models) models for time series, and the procedure produces forecasts. The procedure includes an Expert Modeler that attempts to identify and estimate an appropriate ARIMA or exponential smoothing model for each dependent-variable series. Alternatively, you can specify a custom ARIMA or exponential smoothing model.

Options

Automatic Model Identification. The Expert Modeler can identify an appropriate seasonal or nonseasonal ARIMA or exponential smoothing model for each dependent-variable series. If predictor variables are specified, the Expert Modeler selects, for inclusion in ARIMA models, those variables that have a statistically significant relationship with the dependent series. Model variables are transformed where appropriate by using differencing and/or a square root or natural log transformation. You can limit the set of candidate models to ARIMA models only or exponential smoothing models only.

Custom Models. You can specify a custom exponential smoothing or ARIMA model for one or more series. Seven exponential smoothing methods are available: simple, simple seasonal, Holt's linear trend, Brown's linear trend, damped trend, Winters' additive, and Winters' multiplicative. For ARIMA models, you can specify seasonal and nonseasonal autoregressive and moving average orders, differencing, as well as transfer functions for predictor variables. For exponential smoothing and ARIMA models, you can request that model variables be transformed prior to model estimation.

Outliers. If you use the Expert Modeler or specify a custom ARIMA model, TSMODEL can detect and model outlier time points automatically. The following outlier types can be identified: additive, additive patch, innovational, level shift, transient, seasonal additive, and local trend. Alternatively, if you request a custom ARIMA model, you can specify that one or more time points be modeled as outliers.

Output. Available output includes plots and tables that summarize the distribution of model goodness of fit, residual autocorrelations, and residual partial autocorrelations across models. In addition, you can obtain a table of model goodness of fit, Ljung-Box Q, and number of outliers by model. You can also obtain details for each model, including parameter estimates, forecasts, as well as autocorrelation and partial autocorrelation functions. Output can be restricted to the best-fitting or worst-fitting models based on goodness-of fit-values.

Saved Variables. You can save fit and forecast values to the active dataset as well as confidence intervals and noise residuals.

Missing Values. You can control whether user-missing values are treated as valid or invalid values.

Basic Specification

- The basic specification is a MODEL subcommand that specifies one or more dependent-variable series.
- By default, TSMODEL uses the Expert Modeler to identify and estimate the best ARIMA or exponential smoothing model for each series.
- Default output includes a summary of the distribution of goodness of fit across estimated models and a table of stationary *R*-square, Ljung-Box Q, and number of outliers by model.

Syntax Rules

- The following subcommands are global and apply to all models specified in a single instance of the TSMODEL command: MODELSUMMARY, MODELSTATISTICS, MODELDETAILS, SERIESPLOT, OUTPUTFILTER, SAVE, AUXILIARY, and MISSING.
- Each global subcommand is optional, may be used only once, and must appear before any MODEL subcommand.
- Models are specified in blocks. The MODEL subcommand indicates the start of a block. The MODEL subcommand must be used at least once.
- The following subcommands apply to the preceding MODEL subcommand and constitute--along with a MODEL subcommand--a MODEL block: EXPERTMODELER, EXSMOOTH, ARIMA, TRANSFERFUNCTION, AUTOOUTLIER, and OUTLIER. An error occurs if any of these subcommands precedes the first MODEL subcommand.
- The EXPERTMODELER, EXSMOOTH, and ARIMA subcommands are used within a MODEL block to specify the estimation method. If none of these subcommands is specified, the implicit default is EXPERTMODELER. An error occurs if more than one of these subcommands is specified within a MODEL block.
- AUTOOUTLIER may be used only once within a MODEL block.
- TRANSFERFUNCTION and OUTLIER may be used more than once within a MODEL block.
- Each keyword may be specified only once within a subcommand.
- Empty subcommands are not allowed; all subcommands must be specified with options.
- All subcommands other than MODEL are optional.
- Subcommand names and keywords must be spelled in full.
- Subcommands can be specified in any order, with the exception that global subcommands must precede the first MODEL subcommand.
- Equals signs (=) that are shown in the syntax chart are required.

Operations

- TSMODEL honors time intervals and periodicity specified with the DATE command.
- The procedure verifies that date variables that are implied by the current date specification exist and that their values coincide with the date specification within the estimation period. If SPLIT FILE is in effect, the check is done for each split. A split is skipped and a warning is issued if verification fails for that split. If the verification fails when SPLIT FILE is not in effect, an error occurs and processing terminates. If the data are filtered, values of date variables are verified within the filtered subset of data.

- The procedure honors the estimation (historical) period that is defined with the USE command. However, depending on available data, the actual use period may vary by dependent variable. For a given dependent variable, the estimation period is the period left after eliminating contiguous missing values of the dependent variable at the beginning and end of the USE period.
- If SPLIT FILE is in effect, and a split has fewer cases than implied by the USE period, a warning is issued and all available cases are used. If there are no cases in the USE period, the split is skipped altogether with a warning.
- The PREDICT command defines the end of the forecast period for tables, charts, and model variables that are produced by TSMODEL. The forecast period always starts after the end of the estimation (USE) period. Like the estimation period, the forecast period can vary by model, depending on available data. If a PREDICT specification ends on or before the effective end of the USE period, the forecast period is empty.
- The procedure does not honor the following commands: MODEL NAME or SAVE MODEL. Options for naming and saving models are provided with the TSMODEL procedure.
- The procedure does not honor TSET. The TSMODEL procedure provides options for handling missing values, setting the width of confidence intervals, setting the maximum number of lags displayed for autocorrelations, and setting season length.
- The TDISPLAY command does not display models that are saved by using TSMODEL.

Limitations

- WEIGHT is ignored with a warning.

Examples

This section provides simple examples that are designed to get you started with using the Expert Modeler, producing forecasts, saving results to the active dataset, and saving your model for later use. Further examples that are specific to each subcommand are provided in the subcommand topics.

Using the Expert Modeler

```
TSMODEL
  /MODEL DEPENDENT=sku1 TO sku10.
```

- The Expert Modeler is used to find the best-fitting exponential smoothing or ARIMA model for each of the dependent series *sku1* thru *sku10*.
- The procedure invokes the Expert Modeler because the MODEL subcommand is not followed by one of the model type subcommands (i.e., EXSMOOTH or ARIMA). The absence of a model type subcommand is equivalent to specifying /EXPERTMODELER TYPE=[ARIMA EXSMOOTH] TRYSEASONAL=YES.

Obtaining Model Forecasts

```
PREDICT THRU YEAR 2006 MONTH 6.
TSMODEL
  /SERIESPLOT FORECAST
  /MODELDETAILS PRINT=FORECASTS
  /MODEL DEPENDENT=revenue.
```

- The PREDICT command is used to specify the forecast period for the TSMODEL procedure.
- The SERIESPLOT subcommand specifies that the output contains a plot of the predicted values within the forecast period.
- The MODELDETAILS subcommand specifies that the output includes a table containing the predicted values within the forecast period.

Saving Models to an External File

```
TSMODEL
  /MODEL DEPENDENT=sku1 TO sku50 OUTFILE='/models/models_sku1TOsku50.xml'.
```

- The OUTFILE keyword specifies that each of the resulting models is to be saved to the file */models/models_sku1TOsku50.xml*.

- The saved models can be used to produce new forecasts with the TSAPPLY command when new data are available. See the topic "TSAPPLY" on page 2013 for more information.

Saving Model Predictions, Residuals, and Confidence Intervals as New Variables

```
TSMODEL
  /SAVE PREDICTED LCL UCL NRESIDUAL
  /MODEL DEPENDENT=revenue.
```

- The SAVE subcommand specifies that new variables containing the model predictions, noise residuals, and confidence intervals are saved to the active dataset.

Specifying Multiple Model Types

```
TSMODEL
  /MODEL DEPENDENT=sku1 TO sku10 INDEPENDENT=adspending
  /EXPERTMODELER TYPE=[ARIMA]
  /MODEL DEPENDENT=sku11 TO sku15
  /EXSMOOTH TYPE=WINTERSMULTIPLICATIVE.
```

- The first MODEL block specifies that the Expert Modeler is used to find the best-fitting ARIMA model for each of the dependent series *sku1* thru *sku10*, using the predictor variable *adspending*.
- The second MODEL block specifies that the Winters' multiplicative method is used to model each of the dependent series *sku11* thru *sku15*.
- In this example, different model types were used for different dependent variables. You can also specify multiple model types for the same dependent variable, thereby obtaining multiple models for the variable.

Goodness-of-Fit Measures

The following model goodness-of-fit measures are available in TSMODEL:

SRSQUARE. *Stationary R-squared*. A measure that compares the stationary part of the model to a simple mean model. This measure is preferable to ordinary R-squared when there is a trend or seasonal pattern. Stationary R-squared can be negative with a range of negative infinity to 1. Negative values mean that the model under consideration is worse than the baseline model. Positive values mean that the model under consideration is better than the baseline model.

RSQUARE. *R-squared*. An estimate of the proportion of the total variation in the series that is explained by the model. This measure is most useful when the series is stationary. R-squared can be negative with a range of negative infinity to 1. Negative values mean that the model under consideration is worse than the baseline model. Positive values mean that the model under consideration is better than the baseline model.

RMSE. *RMSE*. Root Mean Square Error. The square root of mean square error. A measure of how much a dependent series varies from its model-predicted level, expressed in the same units as the dependent series.

MAPE. *MAPE*. Mean Absolute Percentage Error. A measure of how much a dependent series varies from its model-predicted level. It is independent of the units used and can therefore be used to compare series with different units.

MAE. *MAE*. Mean absolute error. Measures how much the series varies from its model-predicted level. MAE is reported in the original series units.

MAXAPE. *MaxAPE*. Maximum Absolute Percentage Error. The largest forecasted error, expressed as a percentage. This measure is useful for imagining a worst-case scenario for your forecasts.

MAXAE. *MaxAE*. Maximum Absolute Error. The largest forecasted error, expressed in the same units as the dependent series. Like MaxAPE, it is useful for imagining the worst-case scenario for your forecasts. Maximum absolute error and maximum absolute percentage error may occur at different series points--for example, when the absolute error for a large series value is slightly larger than the absolute error for a small series value. In that case, the maximum absolute error will occur at the larger series value and the maximum absolute percentage error will occur at the smaller series value.

NORMBIC. *Normalized BIC*. Normalized Bayesian Information Criterion. A general measure of the overall fit of a model that attempts to account for model complexity. It is a score based upon the mean square error and includes a penalty for the number of parameters in the model and the length of the series. The penalty removes the advantage of models with more parameters, making the statistic easy to compare across different models for the same series.

MODELSUMMARY Subcommand

The MODELSUMMARY subcommand controls the display of tables and charts that summarize goodness of fit, residual autocorrelations, and residual partial autocorrelations across estimated models. Each keyword is followed by an equals sign (=) and one or more of the available options enclosed in square brackets.

Example

```
TSMODEL
  /MODELSUMMARY PLOT=[SRSQUARE MAXAPE]
  /MODEL DEPENDENT=sku1 TO sku100.
```

- The output includes two histograms: one histogram for stationary R-squared and one histogram for the maximum absolute percentage error. Each histogram consists of results across all models (one model for each dependent variable).

PRINT Keyword

The PRINT keyword controls the display of model summary tables.

MODELFIT. *Goodness of fit.* Table of summary statistics and percentiles for stationary *R*-square, *R*-square, root mean square error, mean absolute percentage error, mean absolute error, maximum absolute percentage error, maximum absolute error, and normalized Bayesian Information Criterion.

RESIDACF. *Residual autocorrelation function.* Table of summary statistics and percentiles for autocorrelations of the residuals.

RESIDPACF. *Residual partial autocorrelation function.* Table of summary statistics and percentiles for partial autocorrelations of the residuals.

NONE. *No tables are shown.* An error occurs if NONE is used in combination with any other PRINT option.

PLOT Keyword

The PLOT keyword controls the display of model summary charts. By default, no charts are shown. (For detailed definitions of the following terms, see “Goodness-of-Fit Measures ” on page 2035.)

SRSQUARE. *Histogram of Stationary* R*-Square*.

RSQUARE. *Histogram of* R*-Square*.

RMSE. *Histogram of Root Mean Square Error.*

MAPE. *Histogram of Mean Absolute Percentage Error.*

MAE. *Histogram of Mean Absolute Error.*

MAXAPE. *Histogram of Maximum Absolute Percentage Error.*

MAXAE. *Histogram of Maximum Absolute Error.*

NORMBIC. *Histogram of Normalized Bayesian Information Criterion (BIC).*

RESIDACF. *Boxplot of Residual Autocorrelation Function by Lag.*

RESIDPACF. *Boxplot of Residual Partial Autocorrelation Function by Lag.*

MODELSTATISTICS Subcommand

The MODELSTATISTICS subcommand controls display of a table that lists all models, along with chosen goodness-of-fit statistics.

Example

```
TSMODEL
  /MODELSTATISTICS DISPLAY=YES MODELFIT=[RSQUARE]
  /MODEL DEPENDENT=sku1 TO sku25.
```

- The output includes a table displaying the value of R-squared for each model.

DISPLAY Keyword

The DISPLAY keyword controls whether the model statistics table is shown.

YES. *Model statistics table is shown.* Table of model goodness of fit, Ljung-Box Q statistic, and number of outliers detected by model. This setting is the default. The Q statistic measures the degree of pattern in the residuals. Large values of Q in relation to its degrees of freedom indicate that model residuals are not randomly distributed.

NO. *Model statistics table is not shown.*

MODELFIT Keyword

The MODELFIT keyword controls which fit statistics are shown in the model statistics table, and the keyword is ignored if DISPLAY=NO. The keyword is followed by an equals sign (=) and one or more of the following options enclosed in square brackets. (For detailed definitions of the following terms, see “Goodness-of-Fit Measures ” on page 2035.)

SRSQUARE. *Stationary* R *-Square*. This setting is the default.

RSQUARE. R-*Square.*

RMSE. *Root Mean Square Error.*

MAPE. *Mean Absolute Percentage Error.*

MAE. *Mean Absolute Error.*

MAXAPE. *Maximum Absolute Percentage Error.*

MAXAE. *Maximum Absolute Error.*

NORMBIC. *Normalized Bayesian Information Criterion (BIC).*

MODELDETAILS Subcommand

The MODELDETAILS subcommand controls the display of model parameters, model forecasts, and autocorrelations of noise residuals for individual models. Each keyword is followed by an equals sign (=) and one or more of the available options enclosed in square brackets.

Example

```
TSMODEL
  /MODELDETAILS PRINT=[PARAMETERS FORECASTS]
  /MODEL DEPENDENT=sku1 TO sku50.
```

- The output includes tables displaying the model parameters and forecasts for each model (one model for each dependent variable).

PRINT Keyword

The PRINT keyword controls display of tables for individual models. By default, no tables are shown.

PARAMETERS. *Model parameter estimates.* Shows separate tables for ARIMA and exponential smoothing models. If outliers exist, parameter estimates for outliers are also displayed.

RESIDACF. *Residual autocorrelation function.* Shows residual autocorrelations by lag.

RESIDPACF. *Residual partial autocorrelation function.* Shows residual partial autocorrelations by lag.

FORECASTS. *Forecasts and confidence intervals.* Shows model forecasts and confidence intervals.

PLOT Keyword

The PLOT keyword controls display of charts for individual models. By default, no charts are shown.

RESIDACF. *Residual autocorrelation function.* Shows residual autocorrelations by lag.

RESIDPACF. *Residual partial autocorrelation function.* Shows residual partial autocorrelations by lag.

SERIESPLOT Subcommand

The SERIESPLOT subcommand allows you to obtain plots of predicted values, observed values, and confidence intervals for each estimated model. By default, no plots are shown. The subcommand is followed by one or more of the following keywords:

OBSERVED. *Displays all observed values of the dependent series.*

FORECAST. *Displays model-predicted values within the forecast period.*

FIT. *Displays model-predicted values within the estimation period.*

FORECASTCI. *Displays upper and lower confidence limits within the forecast period.*

FITCI. *Displays upper and lower confidence limits within the estimation period.*

Example

```
TSMODEL
  /SERIESPLOT OBSERVED FORECAST
  /MODEL DEPENDENT=sku1 TO sku5
  /MODEL DEPENDENT=revenue INDEPENDENT=adspending.
```

- The plot includes the model-predicted values in the forecast period as well as all observed values for each of the dependent series.

OUTPUTFILTER Subcommand

The OUTPUTFILTER subcommand controls which models are included in the model statistics table (MODELSTATISTICS subcommand), detailed model output (MODELDETAILS subcommand), and time series plots (SERIESPLOT subcommand).

- By default, all models are included in the output. OUTPUTFILTER can be used to display only the best-fitting or worst-fitting models (or both). If you request both, two sets of output are displayed.
- OUTPUTFILTER is ignored if no output is requested on the MODELSTATISTICS, MODELDETAILS, or SERIESPLOT subcommands.
- OUTPUTFILTER has no effect on output from the MODELSUMMARY subcommand.

Example

```
TSMODEL
  /MODELSTATISTICS DISPLAY=YES MODELFIT=[SRSQUARE MAXAPE NORMBIC]
  /OUTPUTFILTER DISPLAY=[BESTFIT(N=5) WORSTFIT(PCT=10)]
  /MODEL DEPENDENT=sku1 TO sku200.
```

- The output consists of two sets of results: those results for the 5 best-fitting models and those results for models with fit values in the bottom 10%. The stationary R-squared value is used as the goodness-of-fit measure.

DISPLAY Keyword

The DISPLAY keyword controls which models are included in MODELSTATISTICS, MODELDETAILS, and SERIESPLOT output.

- By default, all models are displayed (DISPLAY=ALLMODELS).
- To display only those models with extreme goodness-of-fit values, specify BESTFIT, WORSTFIT, or both, in square brackets. One or both keywords must be specified.

ALLMODELS. *All models are displayed.* All models that could be computed are shown. This setting is the default.

BESTFIT. *Models having the highest fit values are shown.* To display the models with the N highest fit values, specify BESTFIT followed by N=count in parentheses—for example, BESTFIT(N=5). The count must be a positive integer. If the count exceeds the number of estimated models, all models are shown. To display the models with fit values in the top N%, specify PCT=percent in parentheses. For example, BESTFIT(PCT=5) displays models with fit values in the top 5% among estimated models. The percentage value must be greater than zero and less than 100.

WORSTFIT. *Models having the lowest fit values are shown.* To display the models with the N lowest fit values, specify WORSTFIT followed by N=count in parentheses—for example, WORSTFIT(N=5). The count must be a positive integer. If the count exceeds the number of estimated models, all models are shown. To display the models with fit values in the lowest N%, specify PCT=percent in parentheses. For example, WORSTFIT(PCT=5) displays models with fit values in the bottom 5% among estimated models. The percentage value must be greater than zero and less than 100.

MODELFIT Keyword

The MODELFIT keyword specifies the fit measure that is used to filter models. MODELFIT is ignored if DISPLAY=ALLMODELS.

Specify one of the following options. (For detailed definitions of the following terms, see "Goodness-of-Fit Measures " on page 2035.)

SRSQUARE. *Stationary* R-*Square*. This setting is the default.

RSQUARE. R-*Square.*

RMSE. *Root Mean Square Error.*

MAPE. *Mean Absolute Percentage Error.*

MAE. *Mean Absolute Error.*

MAXAPE. *Maximum Absolute Percentage Error.*

MAXAE. *Maximum Absolute Error.*

NORMBIC. *Normalized Bayesian Information Criterion (BIC).*

SAVE Subcommand

The SAVE subcommand is used to save new variables representing predicted values, residuals, and confidence intervals to the active dataset. By default, no new variables are saved to the active dataset.

- Specify one or more keywords, each keyword followed by an optional rootname to be used as the prefix for new variable names. Enclose the rootname in parentheses. Each keyword gives rise to one new variable for each dependent variable.
- The rootname, if specified, must conform to the rules for valid variable names.
- If no rootname is specified, TSMODEL uses a default name.
- The full variable name is the concatenation of the rootname, the name of the associated dependent variable, and a model identifier. The variable name is extended if necessary to avoid variable naming conflicts.
- The PREDICT command controls whether new cases are added to the file when new variables are saved. New cases are added if PREDICT specifies a forecast period that extends beyond the length of the dependent-variable series.
- SAVE is ignored with a warning if temporary transformations are in effect.

Example

```
TSMODEL
  /SAVE PREDICTED(Pred) NRESIDUAL
  /MODEL DEPENDENT=sku1 TO sku10.
```

- Two new variables are created for each of the dependent variables *sku1* thru *sku10*: one variable contains the model predictions, and the other variable contains the noise residuals.
- The rootname (prefix) for the variables containing model predictions is *Pred*.
- The rootname for the variables containing noise residuals is *NResidual* (the default).

PREDICTED(rootname). *Model fit and forecast values.* The default rootname is *Predicted*.

LCL(rootname). *Lower confidence limits.* The default rootname is *LCL*.

UCL(rootname). *Upper confidence limits.* The default rootname is *UCL*.

NRESIDUAL(rootname). *Noise residuals.* The default rootname is *NResidual*. When transformations of the dependent variable are performed (for example, natural log), these residuals are the residuals for the transformed series.

AUXILIARY Subcommand

The AUXILIARY subcommand is used to set the confidence interval level for forecasts, set the maximum number of lags for autocorrelation and partial autocorrelation plots and tables, and set the season length.

Example

```
TSMODEL
  /MODELDETAILS PLOT=[RESIDACF RESIDPACF]
  /AUXILIARY MAXACFLAGS=36 SEASONLENGTH=6
  /MODEL DEPENDENT=sku1 TO sku20.
```

CILEVEL Keyword

The CILEVEL keyword sets the confidence level.

- Specify a positive number that is less than 100.
- The default is 95.

MAXACFLAGS Keyword

The MAXACFLAGS keyword sets the maximum number of lags displayed in residual autocorrelation and partial autocorrelation tables and plots.

- Specify a positive integer.
- The default is 24.

SEASONLENGTH Keyword

The SEASONLENGTH keyword is used to specify the length of the seasonal period (the number of observations in one period or season).

- The season length must be a positive integer.
- If SEASONLENGTH is not specified, the periodicity that is specified by using the DATE command defines the season length.
- If the Expert Modeler is used, a warning occurs and the season length is treated as 1 for any splits for which there are gaps in the data. If a custom model is specified, a warning is issued and the model is not estimated for any such splits.
- For custom ARIMA or exponential smoothing models, SEASONLENGTH is ignored if the specified model has no seasonal component.

MISSING Subcommand

The MISSING subcommand controls the handling of user-missing values.

- By default, user-missing values are treated as missing (invalid) data.
- System-missing values are always treated as invalid.
- Cases with missing values of a dependent variable that occur within the estimation period are included in the model. The specific handling of the missing value depends on the estimation method.
- A warning is issued if a predictor has missing values within the estimation period. For the Expert Modeler, models involving the predictor are estimated without the predictor. For custom ARIMA, models involving the predictor are not estimated.
- If any predictor has missing values within the forecast period, the procedure issues a warning and forecasts as far as it can.

Example

```
TSMODEL
  /MISSING USERMISSING=INCLUDE
  /MODEL DEPENDENT=revenue INDEPENDENT=adspending.
```

USERMISSING Keyword

The USERMISSING keyword controls the treatment of user-missing values and is required.

EXCLUDE. *Exclude user-missing values.* User-missing values are treated as missing. This setting is the default.

INCLUDE. *Include user-missing values.* User-missing values are treated as valid data.

MODEL Subcommand

The MODEL subcommand is required and is used to signal the start of a model specification, as well as to specify model variable(s).

- All variables must be numeric. Any string variables are filtered out, with a warning.
- Variables that define split-file groups are filtered out of all variable lists, with a warning.
- ALL and TO can be used in dependent and independent variable lists. ALL represents all variables in the active dataset. TO refers to a range of variables in the active dataset.

Note: Specification of the model type is optional and is accomplished by including an EXPERTMODELER, EXSMOOTH, or ARIMA subcommand following the MODEL subcommand. If the model type is not specified, the best-fitting ARIMA or exponential smoothing model will automatically be selected by the Expert Modeler, which is equivalent to specifying /EXPERTMODELER TYPE=[ARIMA EXSMOOTH]. If you are unsure of which type of model to choose, or you want Expert Modeler to choose for you, use the MODEL subcommand without a model type subcommand.

Example

```
TSMODEL
  /MODEL DEPENDENT=store1 TO store100
         INDEPENDENT=adspending
         OUTFILE='/models/stores.xml'
         PREFIX='Expert'.
```

- DEPENDENT specifies that variables *store1* thru *store100* are to be modeled. The keyword TO refers to the order of the variables in the active dataset.
- The INDEPENDENT keyword specifies that the variable *adspending* is to be considered as a predictor variable.
- OUTFILE specifies that the resulting models are to be stored in the file */models/stores.xml*.

- PREFIX specifies that the models will be named *Expert_1* (the model for *store1*), *Expert_2* (the model for *store2*), etc.
- The absence of a model type subcommand (EXPERTMODELER, ARIMA, or EXSMOOTH) means that the Expert Modeler will be used to find the best-fitting exponential smoothing or ARIMA model.

DEPENDENT Keyword

The DEPENDENT keyword is used to specify one or more dependent variables and is required.

- At least one dependent variable must be specified.
- Repeated instances of the same variable are filtered out of the list.
- A separate model is generated for each dependent variable in each MODEL block. Identical dependent variables in separate MODEL blocks generate separate models.

INDEPENDENT Keyword

The INDEPENDENT keyword is used to specify one or more optional independent variables.

- Order of variables within the independent variable list matters when the Expert Modeler is used. The Expert Modeler drops nonsignificant independent variables one at a time, starting with the last variable in the list.
- In the case of custom ARIMA models, all independent variables are included in the model.
- Repeated instances of an independent variable are filtered out of the list. For example, a b a c a is equivalent to a b c. Assuming that the active dataset contains variables *a*, *b*, and *c*, ALL is also equivalent to a b c.
- An independent variable that also appears in the dependent variable list is treated solely as a dependent variable and is excluded from consideration as an independent variable for the model involving that particular dependent variable. For example, if you specify DEPENDENT=a b and INDEPENDENT=a c, then *c* is the only independent variable when modeling *a*, but *a* and *c* are both used as independent variables when modeling *b*.

Events

- Event variables are special independent variables that are used to model effects of occurrences such as a flood, strike, or introduction of a new product line. Any abrupt shift in the level of the dependent series can be modeled by using event variables.
- To designate an independent variable as an event variable, specify [E] following the name of the variable. For example, INDEPENDENT=strike [E] indicates that the variable *strike* is an event variable.
- When the Expert Modeler is used, event variables enter the model with linear terms only (as opposed to transfer functions). Thus, event designation can save processing time and guard against overfitting when using the Expert Modeler.
- Custom ARIMA models ignore event designation.
- For event variables, the presence of an effect is indicated by cases with a value of 1. Cases with values other than 1 are treated as indicating no effect.
- The effect of an event can last a single time period or several periods. If the effect lasts several periods, the corresponding event variable should contain a series of consecutive 1s.
- Event designation applies only to the variable that immediately precedes the designation in the independent variable list. For example, x1 x2 x3 [E] and x1 TO x3 [E] designate *x3* (only) as an event variable. If event designation follows ALL, it applies to all independent variables.
- If a variable appears more than once in the independent variable list, the event specification for the last instance of the variable is honored. For example, if ALL x1 [E] is specified, *x1* is treated as an event variable; if ALL [E] X1 is specified, *x1* is treated as an ordinary predictor.

OUTFILE and OUTPMML Keywords

The OUTFILE and OUTPMML keywords are used to save models to an external file. Saved models can be used to obtain updated forecasts, based on more current data, using the TSAPPLY command. See the topic "TSAPPLY" on page 2013 for more information.

- OUTFILE saves an XML file that can be used with other IBM SPSS applications.
- OUTPMML saves a PMML-compliant XML file that can be used with PMML-compliant applications, including IBM SPSS applications.
- Models are written to an XML file. Each model is assigned a unique name and a description that includes the name of the dependent variable and the model type.
- The filename must be specified in full. No extension is supplied.
- If two or more MODEL subcommands (within a single invocation of TSMODEL) specify the same external file, all models that are created by those MODEL subcommands are saved to that file.
- OUTFILE and OUTPMML are ignored with a warning if SPLIT FILE is in effect.

PREFIX Keyword

Models are assigned unique names consisting of a customizable prefix, along with an integer suffix. The PREFIX keyword is used to specify a custom prefix.

- Custom prefixes must be specified in quotes—for example, PREFIX='ExpertArima'. The default prefix is 'Model'.
- Integer suffixes are unique across the set of models that have the same prefix; for example, *Model_1*, *Model_2*. Models with different prefixes, however, can have the same integer suffix—for example, *CustomArima_1*, *ExpertArima_1*.

EXPERTMODELER Subcommand

The EXPERTMODELER subcommand controls options for automatic model identification that is performed by the Expert Modeler.

Example

```
TSMODEL
  /MODEL DEPENDENT=store1 TO store100
         INDEPENDENT=adspending
  /EXPERTMODELER TYPE=[ARIMA].
```

- The keyword ARIMA specifies that the Expert Modeler is to limit the model search to the best-fitting ARIMA model.

TYPE Keyword

The TYPE keyword is required and is used to specify the model types that are evaluated: ARIMA, exponential smoothing, or both.

- By default, both ARIMA and exponential smoothing models are considered.
- Either ARIMA, EXSMOOTH, or both must be specified in square brackets.
- An error occurs if you specify the EXSMOOTH keyword by itself and the model contains independent variables.

The keyword is followed by an equals sign (=) and one or both of the following options enclosed in square brackets:

ARIMA. *ARIMA models are considered.*

EXSMOOTH. *Exponential smoothing models are considered.*

TRYSEASONAL Keyword

The TRYSEASONAL keyword specifies whether seasonal models are considered by the Expert Modeler.

- TRYSEASONAL is ignored if the data are nonperiodic (that is, season length is 1).

YES. *The Expert Modeler considers seasonal models.* This setting is the default.

NO. *The Expert Modeler does not consider seasonal models.* Only nonseasonal models are considered.

EXSMOOTH Subcommand

The EXSMOOTH subcommand controls options for custom exponential smoothing models.

- Using EXSMOOTH generates an error if independent variables are defined.
- For seasonal models (SIMPLESEASONAL, WINTERSADDITIVE, and WINTERSMULTIPLICATIVE), an error is generated if the season length has not been defined via the DATE command or the SEASONLENGTH keyword.

Example

```
TSMODEL
  /AUXILIARY SEASONLENGTH=12
  /MODEL DEPENDENT=store1 TO store100
  /EXSMOOTH TYPE=WINTERSMULTIPLICATIVE.
```

TYPE Keyword

The TYPE keyword is required and is used to specify the type of exponential smoothing model. There is no default method.

SIMPLE. *Simple exponential smoothing model*. This model is appropriate for series in which there is no trend or seasonality. Its only smoothing parameter is level. Simple exponential smoothing is most similar to an ARIMA model with zero orders of autoregression, one order of differencing, one order of moving average, and no constant.

SIMPLESEASONAL. *Simple seasonal exponential smoothing model*. This model is appropriate for series with no trend and a seasonal effect that is constant over time. Its smoothing parameters are level and season. Simple seasonal exponential smoothing is most similar to an ARIMA model with zero orders of autoregression, one order of differencing, one order of seasonal differencing, and orders 1, p, and p + 1 of moving average, where p is the number of periods in a seasonal interval (for monthly data, p = 12).

HOLT. *Holt's method*. This model is appropriate for series in which there is a linear trend and no seasonality. Its smoothing parameters are level and trend, which are not constrained by each other's values. Holt's model is more general than Brown's model but may take longer to compute for large series. Holt's exponential smoothing is most similar to an ARIMA model with zero orders of autoregression, two orders of differencing, and two orders of moving average.

BROWN. *Brown's method*. This model is appropriate for series in which there is a linear trend and no seasonality. Its smoothing parameters are level and trend, which are assumed to be equal. Brown's model is therefore a special case of Holt's model. Brown's exponential smoothing is most similar to an ARIMA model with zero orders of autoregression, two orders of differencing, and two orders of moving average, with the coefficient for the second order of moving average equal to the square of one-half of the coefficient for the first order.

DAMPEDTREND. *Damped trend method*. This model is appropriate for series with a linear trend that is dying out and with no seasonality. Its smoothing parameters are level, trend, and damping trend. Damped exponential smoothing is most similar to an ARIMA model with 1 order of autoregression, 1 order of differencing, and 2 orders of moving average.

WINTERSADDITIVE. *Winters' additive method*. This model is appropriate for series with a linear trend and a seasonal effect that does not depend on the level of the series. Its smoothing parameters are level, trend, and season. Winters' additive exponential smoothing is most similar to an ARIMA model with zero orders of autoregression, one order of differencing, one order of seasonal differencing, and p + 1 orders of moving average, where p is the number of periods in a seasonal interval (for monthly data, p = 12).

WINTERSMULTIPLICATIVE. *Winters' multiplicative method*. This model is appropriate for series with a linear trend and a seasonal effect that depends on the level of the series. Its smoothing parameters are level, trend, and season. Winters' multiplicative exponential smoothing is not similar to any ARIMA model.

TRANSFORM Keyword

The TRANSFORM keyword specifies a transformation that is performed on each dependent variable before it is modeled. By default, dependent variables are not transformed.

NONE. *Dependent variable series are not transformed.* This setting is the default.

SQRT. *A square root transformation is performed.*

LN. *A natural log transformation is performed.*

ARIMA Subcommand

The ARIMA subcommand controls options for custom ARIMA models. The model must contain at least one parameter. An error occurs if the model has no autoregressive component, moving average component, outlier, or constant term, unless independent variables are selected.

Example: Basic Custom Seasonal ARIMA Model

```
TSMODEL
  /AUXILIARY SEASONLENGTH=12
  /MODEL DEPENDENT=passengers
  /ARIMA AR=[0] ARSEASONAL=[0] MA=[1] MASEASONAL=[1]
         DIFF=1 DIFFSEASONAL=1 TRANSFORM=LN.
```

- A custom ARIMA(0,1,1)(0,1,1) model is specified, including a natural log transformation of the dependent variable. The seasonal length is 12.

Example: Including Specific Lags in a Custom ARIMA Model

```
TSMODEL
  /MODEL DEPENDENT=revenue
  /ARIMA AR=0 MA=[1 3].
```

- A moving average model is specified. Moving average lags of orders 1 and 3 are included.

AR Keyword

The AR keyword specifies the nonseasonal autoregressive order of the model and is required.

- You can specify a single value or a list of values in square brackets. All values must be nonnegative integers.
- If you specify a single value, square brackets are optional. The specified lag will be included in the model.
- If you specify a list of positive integer values--for example, `[1 3]`--the specified lags are included in the model. Duplicate lags are ignored.

ARSEASONAL Keyword

The ARSEASONAL keyword specifies the seasonal autoregressive order of the model.

- You can specify a single value or a list of values in square brackets. All values must be nonnegative integers. The default order is zero.
  - If you specify a single value, square brackets are optional. The specified lag will be included in the model.
  - If you specify a list of positive integer values--for example, `[1 3]`--the specified lags are included in the model. Duplicate lags are ignored.
- An error occurs if ARSEASONAL specifies a nonzero value and the season length has not been defined via the DATE command or the SEASONLENGTH keyword.

MA Keyword

The MA keyword specifies the nonseasonal moving average order of the model and is required.

- You can specify a single value or a list of values in square brackets. All values must be nonnegative integers.

- If you specify a single value, square brackets are optional. The specified lag will be included in the model.
- If you specify a list of positive integer values--for example, `[1 3]`--the specified lags are included in the model. Duplicate lags are ignored.

MASEASONAL Keyword

The MASEASONAL keyword specifies the seasonal moving average order of the model.

- You can specify a single value or a list of values in square brackets. All values must be nonnegative integers. The default order is zero.
  - If you specify a single value, square brackets are optional. The specified lag will be included in the model.
  - If you specify a list of positive integer values--for example, `[1 3]`--the specified lags are included in the model. Duplicate lags are ignored.
- An error occurs if MASEASONAL specifies a nonzero value and the season length has not been defined via the DATE command or the SEASONLENGTH keyword.

CONSTANT Keyword

The CONSTANT keyword controls whether the model for each dependent variable includes a constant term.

YES. *The model includes a constant term.* This setting is the default.

NO. *No constant term is included in the model.*

TRANSFORM Keyword

The TRANSFORM keyword specifies a transformation that is performed on each dependent variable before it is modeled. By default, the variables are not transformed.

NONE. *Dependent-variable series are not transformed.* This setting is the default.

SQRT. *A square root transformation is performed.*

LN. *A natural log transformation is performed.*

DIFF Keyword

The DIFF keyword specifies the order of nonseasonal differencing.

- The order of nonseasonal differencing must be a nonnegative integer. The default order is zero.

DIFFSEASONAL Keyword

The DIFFSEASONAL keyword specifies the order of seasonal differencing.

- The order of seasonal differencing must be a nonnegative integer. The default order is zero.
- An error occurs if DIFFSEASONAL specifies a nonzero value and the season length has not been defined via the DATE command or the SEASONLENGTH keyword.

TRANSFERFUNCTION Subcommand

The TRANSFERFUNCTION subcommand specifies a transfer function and transformations for predictor variables in an ARIMA model.

- By default, for each predictor in an ARIMA model, the numerator and denominator orders of the transfer function (both seasonal and nonseasonal) are set to zero. The predictors are neither transformed nor differenced by default.
- The TRANSFERFUNCTION subcommand may be used more than once in a MODEL block to specify different transfer functions and/or transformations for different independent variables.
- If there are multiple TRANSFERFUNCTION subcommands that refer to the same predictor variable within a MODEL block, the last set of specifications is honored for that variable.

- The TRANSFERFUNCTION subcommand is ignored if the EXPERTMODELER or EXSMOOTH subcommand is used.

Example

```
TSMODEL
  /MODEL DEPENDENT=revenue INDEPENDENT=adspending
  /ARIMA AR=[1] MA=[0]
  /TRANSFERFUNCTION VARIABLES=adspending NUM=[1] DENOM=[1].
```

- A custom ARIMA(1,0,0) model is specified and includes the independent variable *adspending*.
- The TRANSFERFUNCTION subcommand specifies that *adspending* is to be modeled with numerator and denominator lags of order 1.

VARIABLES Keyword

The VARIABLES keyword is used to specify a list of predictors.

- Repeated instances of the same variable are filtered out of the list.
- Variables that have not been specified as independent (with the INDEPENDENT keyword) in the preceding MODEL subcommand are ignored.
- Use ALL to specify all independent variables. This setting is the default.

NUM Keyword

The NUM keyword specifies the numerator order of the transfer function for nonseasonal models.

- You can specify a single value or a list of values in square brackets. All values must be nonnegative integers. The default order is zero.
- The model always includes an implicit zero, even if not specified via the NUM keyword.
- If you specify a single value, square brackets are optional. The specified lag will be included in the model.
- If you specify a list of positive integer values--for example, `[1 3]`--the specified lags are included in the model. Duplicate lags are ignored.

NUMSEASONAL Keyword

The NUMSEASONAL keyword specifies the numerator order of the transfer function for seasonal models.

- You can specify a single value or a list of values in square brackets. All values must be nonnegative integers. The default order is zero.
  - If you specify a single value, square brackets are optional. The specified lag will be included in the model.
  - If you specify a list of positive integer values--for example, `[1 3]`--the specified lags are included in the model. Duplicate lags are ignored.
- An error occurs if NUMSEASONAL specifies a nonzero value and the season length has not been defined via the DATE command or the SEASONLENGTH keyword.

DENOM Keyword

The DENOM keyword specifies the denominator order of the transfer function for nonseasonal models.

- You can specify a single value or a list of values in square brackets. All values must be nonnegative integers. The default order is zero.
- If you specify a single value, square brackets are optional. The specified lag will be included in the model.
- If you specify a list of positive integer values--for example, `[1 3]`--the specified lags are included in the model. Duplicate lags are ignored.

DENOMSEASONAL Keyword

The DENOMSEASONAL keyword specifies the denominator order of the transfer function for seasonal models.

- You can specify a single value or a list of values in square brackets. All values must be nonnegative integers. The default order is zero.
  - If you specify a single value, square brackets are optional. The specified lag will be included in the model.
  - If you specify a list of positive integer values--for example, `[1 3]`--the specified lags are included in the model. Duplicate lags are ignored.
- An error occurs if DENOMSEASONAL specifies a nonzero value and the season length has not been defined via the DATE command or the SEASONLENGTH keyword.

TRANSFORM Keyword

The TRANSFORM keyword specifies a transformation that is performed on all predictor variables. By default, predictor variables are not transformed.

NONE. *Predictor variables are not transformed.* This setting is the default.

SQRT. *A square root transformation is performed.*

LN. *A natural log transformation is performed.*

DIFF Keyword

The DIFF keyword specifies the order of nonseasonal differencing for predictor variables.

- The order of nonseasonal differencing must be a nonnegative integer. The default order is zero.

DIFFSEASONAL Keyword

The DIFFSEASONAL keyword specifies the order of seasonal differencing for predictor variables.

- The order of seasonal differencing must be a nonnegative integer. The default order is zero.
- DIFFSEASONAL generates an error if the season length has not been defined via the DATE command or the SEASONLENGTH keyword.

DELAY Keyword

The DELAY keyword specifies the delay order for predictor variables.

- The delay order must be a nonnegative integer. The default order is zero.

AUTOOUTLIER Subcommand

The AUTOOUTLIER subcommand specifies whether outlier time points are identified automatically.

- By default, outliers are not identified.
- If identified, outliers are not discarded but are incorporated in the model.
- The AUTOOUTLIER subcommand is ignored with a warning if it is part of a MODEL block that contains an EXSMOOTH subcommand.

Example

```
TSMODEL
  /MODELDETAILS PRINT=PARAMETERS
  /MODEL DEPENDENT=revenue
  /EXPERTMODELER TYPE=[ARIMA]
  /AUTOOUTLIER DETECT=ON.
```

DETECT Keyword

The DETECT keyword controls whether automatic outlier detection is performed.

OFF. *Outliers are not identified.* This setting is the default.

ON. *Outliers are identified.* ON is ignored with a warning if the model type is EXSMOOTH.

TYPE Keyword

The optional TYPE keyword specifies the types of outliers that are identified when DETECT=ON. TYPE is ignored if DETECT=OFF.

- Additive and level shift outliers are identified by default.
- The specification of outlier types overrides the default. That is, if you specify a list of outlier types, additive and level shift outliers are not included unless explicitly requested.

The keyword is followed by an equals sign (=) and one or more of the following options enclosed in square brackets:

ADDITIVE. *Additive*. An outlier that affects a single observation. For example, a data coding error might be identified as an additive outlier. Default if DETECT=ON.

LEVELSHIFT. *Level shift*. An outlier that shifts all observations by a constant, starting at a particular series point. A level shift could result from a change in policy. Default if DETECT=ON.

INNOVATIONAL. *Innovational*. An outlier that acts as an addition to the noise term at a particular series point. For stationary series, an innovational outlier affects several observations. For nonstationary series, it may affect every observation starting at a particular series point.

TRANSIENT. *Transient*. An outlier whose impact decays exponentially to 0.

SEASONALADDITIVE. *Seasonal additive*. An outlier that affects a particular observation and all subsequent observations separated from it by one or more seasonal periods. All such observations are affected equally. A seasonal additive outlier might occur if, beginning in a certain year, sales are higher every January. The keyword generates an error if the season length has not been defined via the DATE command or the SEASONLENGTH keyword.

LOCALTREND. *Local trend*. An outlier that starts a local trend at a particular series point.

ADDITIVEPATCH. *Additive patch*. A group of two or more consecutive additive outliers. Selecting this outlier type results in the detection of individual additive outliers in addition to groups of them.

OUTLIER Subcommand

The OUTLIER subcommand specifies outlier time points to be included in an ARIMA model.

- By default, no time points are modeled as outliers.
- For each outlier, you must specify its time point (location) and type.
- Multiple OUTLIER subcommands can be used to define two or more outliers for an ARIMA model. Duplicate specifications (same location and type) are ignored and a warning is issued.
- A warning occurs and the OUTLIER subcommand is ignored if the subcommand is part of a MODEL block that contains an EXPERTMODELER or EXSMOOTH subcommand.
- If the AUTOOUTLIER and OUTLIER subcommands are both included in a MODEL block containing an ARIMA subcommand, a warning is issued and the OUTLIER specification takes precedence.

Example

```
TSMODEL
  /MODELDETAILS PRINT=PARAMETERS
  /MODEL DEPENDENT=revenue
  /ARIMA AR=[1] MA=[1]
  /OUTLIER LOCATION=[YEAR 2000 MONTH 8] TYPE=LOCALTREND.
```

- A custom ARIMA(1,0,1) model is specified with an outlier of type Local Trend for August, 2000.

LOCATION Keyword

The LOCATION keyword specifies the time point to be treated as an outlier and is required.

- Specify a date in square brackets. If the data are undated, you must specify a case number. Otherwise, specify the outlier location using date keywords and values. For example, LOCATION=[YEAR 2000 MONTH 8] specifies that August, 2000 be treated as an outlier. The following date keywords may be used: CYCLE, YEAR, QUARTER, MONTH, WEEK, DAY, HOUR, MINUTE, SECOND, and OBS. Every keyword in

the IBM SPSS Statistics date specification must appear in LOCATION. Only keywords that correspond to system date variables may be used, and no date keyword may be used more than once. See the topic "DATE" on page 547 for more information.

- A warning is issued if you specify an outlier location that is outside the estimation period or corresponds to a gap in the data. Any such models are ignored.

TYPE Keyword

The TYPE keyword specifies the type of outlier and is required. Specify one of the following types:

ADDITIVE. *Additive*. An outlier that affects a single observation. For example, a data coding error might be identified as an additive outlier.

LEVELSHIFT. *Level shift*. An outlier that shifts all observations by a constant, starting at a particular series point. A level shift could result from a change in policy.

INNOVATIONAL. *Innovational*. An outlier that acts as an addition to the noise term at a particular series point. For stationary series, an innovational outlier affects several observations. For nonstationary series, it may affect every observation starting at a particular series point.

TRANSIENT. *Transient*. An outlier whose impact decays exponentially to 0.

SEASONALADDITIVE. *Seasonal additive*. An outlier that affects a particular observation and all subsequent observations separated from it by one or more seasonal periods. All such observations are affected equally. A seasonal additive outlier might occur if, beginning in a certain year, sales are higher every January. The keyword generates an error if the season length has not been defined via the DATE command or the SEASONLENGTH keyword.

LOCALTREND. *Local trend*. An outlier that starts a local trend at a particular series point.

TSPLOT

```
TSPLOT VARIABLES= variable names
 [/DIFF={1}]
        {n}
 [/SDIFF={1}]
         {n}
 [/PERIOD=n]
 [/{NOLOG**}]
   {LN     }
 [/ID=varname]
 [/MARK={varname} ]
        {date   }
 [/SPLIT {UNIFORM**}]
         {SCALE    }
 [/APPLY [='model name']]
```

For plots with one variable:

```
[/FORMAT=[{NOFILL**}]  [{NOREFERENCE**     }]]
          {BOTTOM  }    {REFERENCE[(value)]}
```

For plots with multiple variables:

```
[/FORMAT={NOJOIN**}]
         {JOIN    }
         {HILO    }
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 14.0

- For plots with one variable, REFERENCE keyword modified to allow specification of a value.

Example

```
TSPLOT VARIABLES = TICKETS.
```

Overview

TSPLOT produces a plot of one or more time series or sequence variables. You can request natural log and differencing transformations to produce plots of transformed variables. Several plot formats are available.

Options

Modifying the Variables. You can request a natural log transformation of the variables using the LN subcommand and seasonal and nonseasonal differencing to any degree using the SDIFF and DIFF subcommands. With seasonal differencing, you can also specify the periodicity on the PERIOD subcommand.

Plot Format. With the FORMAT subcommand, you can fill in the space on one side of the plotted values on plots with one variable. You can also plot a reference line indicating the variable mean. For plots with two or more variables, you can specify whether you want to join the values for each observation with a vertical line. With the ID subcommand, you can label the horizontal axis with the values of a specified variable. You can mark the onset of an intervention variable on the plot with the MARK subcommand.

Split-File Processing. You can control how data that have been divided into subgroups by a SPLIT FILE command should be plotted using the SPLIT subcommand.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- VARIABLES can be specified only once.
- Other subcommands can be specified more than once, but only the last specification of each one is executed.

Operations

- Subcommand specifications apply to all variables named on the TSPLOT command.
- If the LN subcommand is specified, any differencing requested on that TSPLOT command is done on the log-transformed variables.
- Split-file information is displayed as part of the subtitle, and transformation information is displayed as part of the footnote.

Limitations

- Maximum of one VARIABLES subcommand. There is no limit on the number of variables named on the list.

Basic Specification

The basic specification is one or more variable names.

- If the DATE command has been specified, the horizontal axis is labeled with the *DATE_* variable at periodic intervals. Otherwise, sequence numbers are used. The vertical axis is labeled with the value scale of the plotted variable(s).

The following figure shows a default plot with DATE YEAR 1995 MONTH.

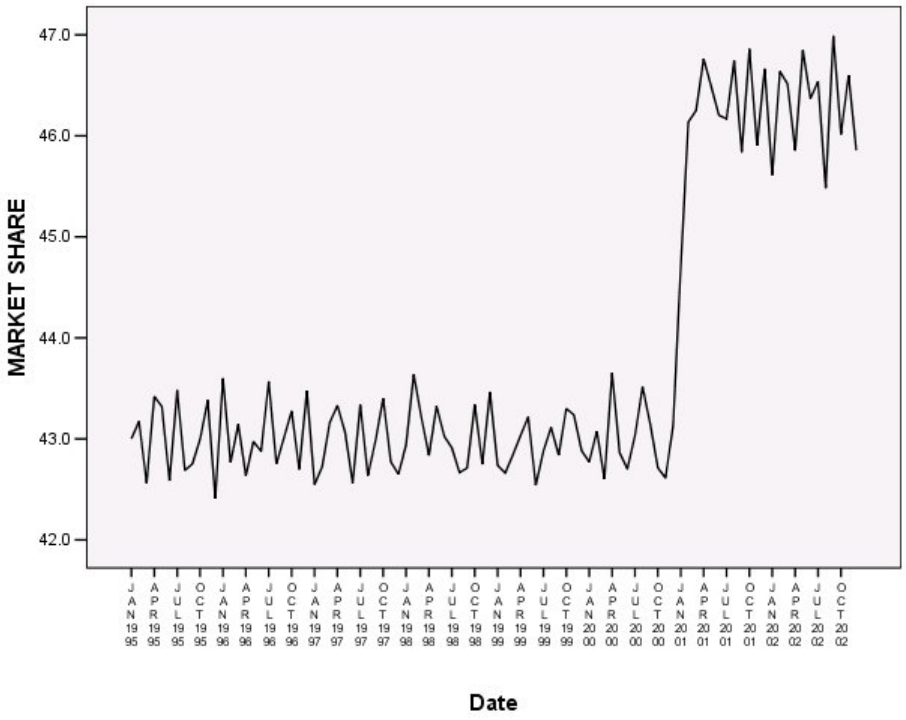


Figure 98. TSPLOT VARIABLES=SHARE

Example

```
TSPLOT VARIABLES = TICKETS
  /LN
  /DIFF
  /SDIFF
  /PERIOD=12
  /FORMAT=REFERENCE
  /MARK=Y 55 M 6.
```

- This command produces a plot of *TICKETS* after a natural log transformation, differencing, and seasonal differencing have been applied.
- LN transforms the data using the natural logarithm (base *e*) of *TICKETS*.
- DIFF differences the logged variable once.
- SDIFF and PERIOD apply one degree of seasonal differencing with a period of 12.
- FORMAT=REFERENCE adds a reference line representing the variable mean.
- MARK provides a marker on the plot at June 1955. The marker is displayed as a vertical reference line.

VARIABLES Subcommand

VARIABLES specifies the names of the variables to be plotted and is the only required subcommand.

DIFF Subcommand

DIFF specifies the degree of differencing used to convert a nonstationary variable to a stationary one with a constant mean and variance before plotting.

- You can specify any positive integer on DIFF.
- If DIFF is specified without a value, the default is 1.
- The number of values plotted decreases by 1 for each degree of differencing.

Example

```
TSPLOT VARIABLES = TICKETS
  /DIFF=2.
```

- In this example, *TICKETS* is differenced twice before plotting.

SDIFF Subcommand

If the variable exhibits a seasonal or periodic pattern, you can use the SDIFF subcommand to seasonally difference the variable before plotting.

- The specification on SDIFF indicates the degree of seasonal differencing and can be any positive integer.
- If SDIFF is specified without a value, the degree of seasonal differencing defaults to 1.
- The number of seasons plotted decreases by 1 for each degree of seasonal differencing.
- The length of the period used by SDIFF is specified on the PERIOD subcommand. If the PERIOD subcommand is not specified, the periodicity established on the TSET or DATE command is used (see the PERIOD subcommand).

PERIOD Subcommand

PERIOD indicates the length of the period to be used by the SDIFF subcommand.

- The specification on PERIOD indicates how many observations are in one period or season and can be any positive integer.
- If PERIOD is not specified, the periodicity established on TSET PERIOD is in effect. If TSET PERIOD is not specified, the periodicity established on the DATE command is used. If periodicity was not established anywhere, the SDIFF subcommand will not be executed.

Example

```
TSPLOT VARIABLES = TICKETS
  /SDIFF=1
  /PERIOD=12.
```

- This command applies one degree of seasonal differencing with 12 observations per season to *TICKETS* before plotting.

LN and NOLOG Subcommands

LN transforms the data using the natural logarithm (base *e*) of the variable and is used to remove varying amplitude over time. NOLOG indicates that the data should not be log transformed. NOLOG is the default.

- If you specify LN on TSPLOT, any differencing requested on that command will be done on the log-transformed variables.
- There are no additional specifications on LN or NOLOG.
- Only the last LN or NOLOG subcommand on a TSPLOT command is executed.
- If a natural log transformation is requested, any value less than or equal to zero is set to system-missing.
- NOLOG is generally used with an APPLY subcommand to turn off a previous LN specification.

Example

```
TSPLOT VARIABLES = TICKETS
  /LN.
```

- In this example, *TICKETS* is transformed using the natural logarithm before plotting.

ID Subcommand

ID names a variable whose values will be used as labels for the horizontal axis.

- The only specification on ID is a variable name. If you have a variable named *ID* in your active dataset, the equals sign (=) after the subcommand is required.
- If the ID subcommand is not used and TSET ID has not been specified, the axis is labeled with the *DATE_* variable created by the DATE command. If the DATE command has not been specified, the observation number is used as the label.

Example

```
TSPLOT VARIABLES = VARA
  /ID=VARB.
```

- In this example, the values of *VARB* will be used to label the horizontal axis of *VARA* at periodic intervals.

FORMAT Subcommand

FORMAT controls the plot format.

- The specification on FORMAT is one of the keywords listed below.

- Keywords NOFILL, BOTTOM, REFERENCE, and NOREFERENCE apply to plots with one variable. NOFILL and BOTTOM are alternatives that indicate how the plot is filled. NOREFERENCE and REFERENCE are alternatives that specify whether a reference line is displayed.
- Keywords JOIN, NOJOIN, and HILO apply to plots with multiple variables and are alternatives. NOJOIN is the default. Only one keyword can be specified on a FORMAT subcommand for plots with multiple variables.

The following formats are available for plots with one variable:

NOFILL. *Plot only the values for the variable with no fill.* NOFILL produces a plot with no fill above or below the plotted values. This is the default format when one variable is specified.

BOTTOM. *Plot the values for the variable and fill in the area below the curve.* If the plotted variable has missing or negative values, BOTTOM is ignored and the default NOFILL is used instead.

NOREFERENCE. *Do not plot a reference line.* This is the default when one variable is specified.

REFERENCE(value) . *Plot a reference line at the specified value or at the variable mean if no value is specified.* A fill chart is displayed as an area chart with a reference line and a non-fill chart is displayed as a line chart with a reference line.

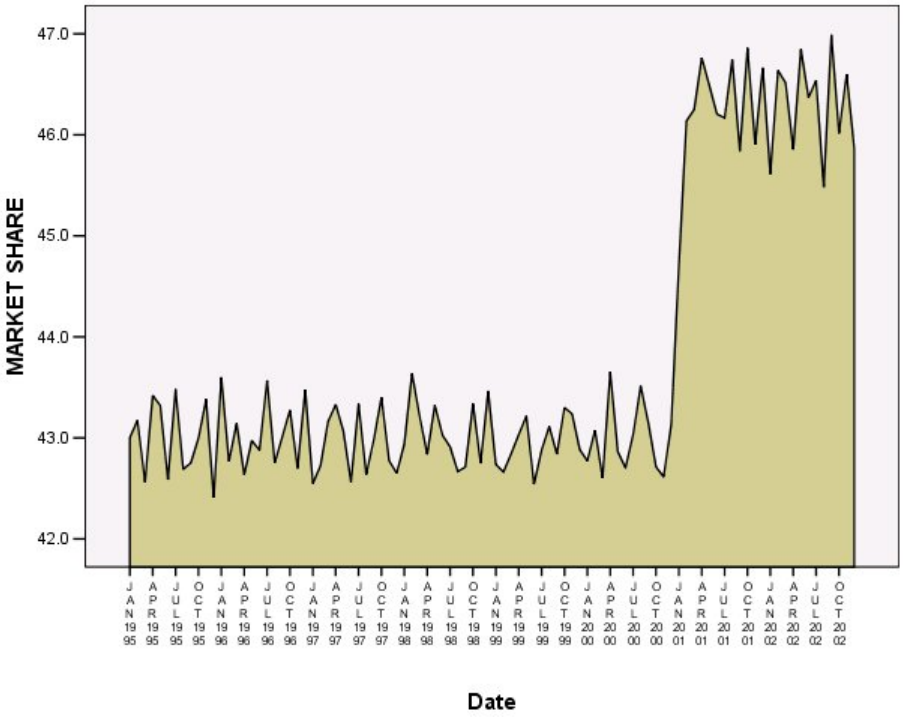


Figure 99. FORMAT=BOTTOM

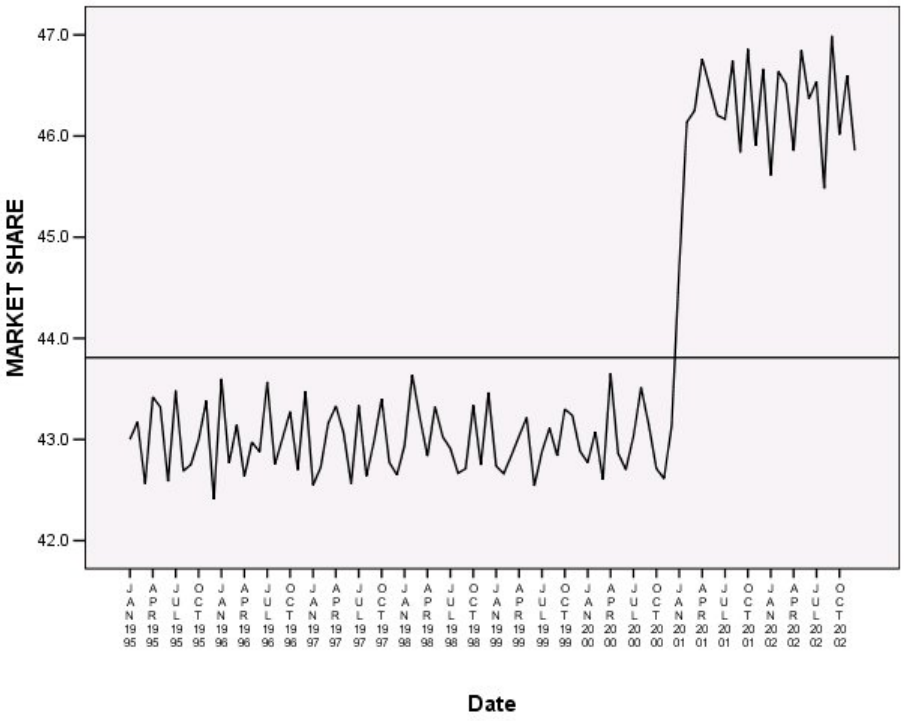


Figure 100. FORMAT=REFERENCE

The following formats are available for plots with multiple variables:

NOJOIN. *Plot the values of each variable named.* Different colors or line patterns are used for multiple variables. Multiple occurrences of the same value for a single observation are plotted using a dollar sign ($). This is the default format for plots with multiple variables.

JOIN. *Plot the values of each variable and join the values for each observation.* Values are plotted as described for `NOJOIN`, and the values for each observation are joined together by a line.

HILO. *Plot the highest and lowest values across variables for each observation and join the two values together.* The high and low values are plotted as a horizontal bar and are joined with a line. If more than three variables are specified, `HILO` is ignored and the default `NOJOIN` is used.

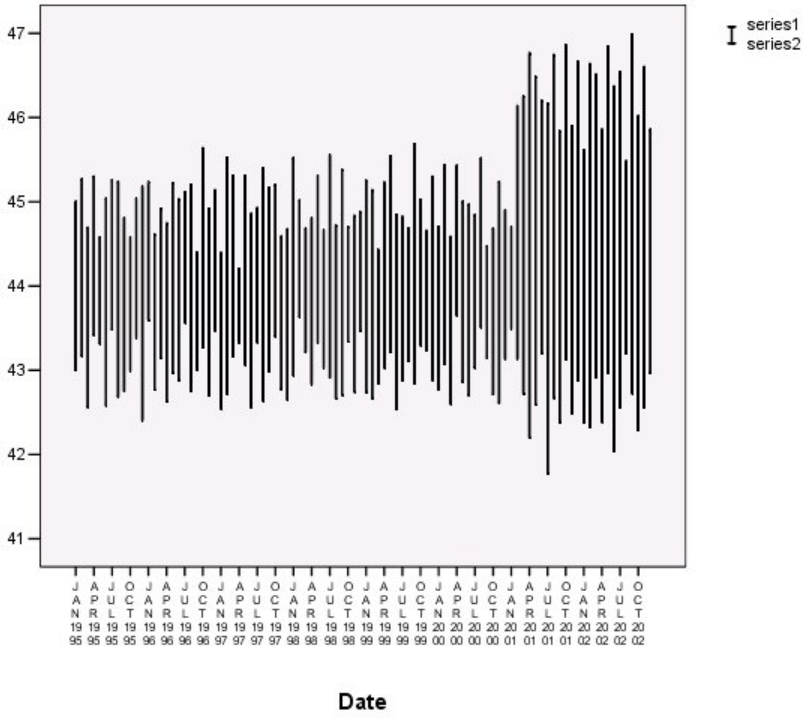


Figure 101. FORMAT=HILO

MARK Subcommand

MARK indicates the onset of an intervention variable.

- The onset date is indicated by a vertical reference line.
- The specification on MARK can be either a variable name or an onset date if the *DATE_* variable exists.
- If a variable is named, the plot indicates where the values of that variable change.
- A date specification follows the same format as the DATE command; that is, a keyword followed by a value. For example, the specification for June 1955 is `Y 1955 M 6` (or `Y 55 M 6` if only the last two digits of the year are used on DATE).

The following figure shows a plot with January 2001 marked as the onset date.

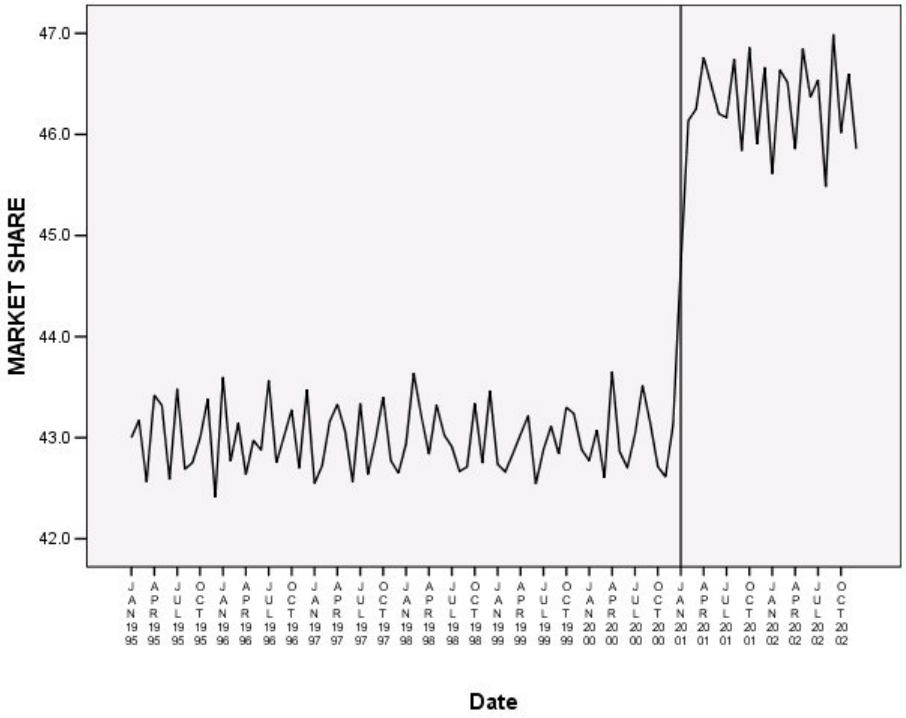


Figure 102. MARK=Y 2001 M 1

SPLIT Subcommand

SPLIT specifies how to plot data that have been divided into subgroups by a SPLIT FILE command. The default is UNIFORM.

UNIFORM. *Scale uniformly.* The vertical axis is scaled according to the values of the entire dataset.

SCALE. *Scale individually.* The vertical axis is scaled according to the values of each individual subgroup.

- If FORMAT=REFERENCE is specified when SPLIT=SCALE, the reference line is placed at the mean of the subgroup. If FORMAT=REFERENCE is specified when SPLIT=UNIFORM, the reference line is placed at the overall mean.

Example

```
SPLIT FILE BY REGION.
TSPLOT VARIABLES = TICKETS / SPLIT=SCALE.
```

- In this example, the data have been split into subgroups by *REGION*. The plots produced with the SCALE subcommand have vertical axes that are individually scaled according to the values of each particular region.

APPLY Subcommand

APPLY allows you to produce a plot using previously defined specifications without having to repeat the TSPLOT subcommands.

- The only specification on APPLY is the name of a previous model enclosed in quotes. If a model name is not specified, the specifications from the previous TSPLOT command are used.
- To change one or more specifications of the plot, specify the subcommands of only those portions you want to change after the subcommand APPLY.
- If no variables are specified, the variables that were specified for the original plot are used.
- To plot different variables, enter new variable names before or after the APPLY subcommand.

Example

```
TSPLOT VARIABLES = TICKETS
  /LN
  /DIFF=1
  /SDIFF=1
  /PERIOD=12.
TSPLOT VARIABLES = ROUNDTRP
  /APPLY.
TSPLOT APPLY
  /NOLOG.
```

- The first command produces a plot of *TICKETS* after a natural log transformation, differencing, and seasonal differencing have been applied.
- The second command plots the values for *ROUNDTRP* using the same subcommands specified for *TICKETS*.
- The third command produces another plot of *ROUNDTRP* but this time without a log transformation. *ROUNDTRP* is still differenced once and seasonally differenced with a periodicity of 12.

T-TEST

T-TEST is available in the Statistics Base Edition.

One-sample tests

```
T-TEST TESTVAL n /VARIABLE=varlist
```

Independent-samples tests

```
T-TEST GROUPS=varname ({1,2**      }) /VARIABLES=varlist
                      {value       }
                      {value,value}
```

**Default if the subcommand is omitted.

Paired-samples tests

```
T-TEST PAIRS=varlist [WITH varlist [(PAIRED)]] [/varlist ...]
[/ES
    [ DISPLAY({TRUE**} {FALSE}) ]
    [ STANDARDIZER({SD**} {CORRECTED_SD} {AVERAGE}) ] ]
```

All test types

```
[/MISSING={ANALYSIS**}  [INCLUDE]]
          {LISTWISE  }

[/CRITERIA=CI({0.95**})
              {value }
```

**Default if the subcommand is omitted.

Examples

```
T-TEST GROUPS=WORLD(1,3) /VARIABLES=NTCPRI NTCSAL NTCPUR.

T-TEST PAIRS=TEACHER CONSTRUC MANAGER.
```

Release History

Release 27.0

- **ES** subcommand introduced

Overview

T-TEST compares sample means by calculating Student's *t* and displays the two-tailed probability of the difference between the means. Statistics are available for one-sample (tested against a specified value), independent samples (different groups of cases), or paired samples (different variables). Other procedures that compare group means are ANOVA, ONEWAY, UNIANOVA, GLM, and MANOVA (GLM and MANOVA are available in SPSS Statistics Standard Edition or the Advanced Statistics Option).

Options

Statistics. There are no optional statistics. All statistics available are displayed by default.

Basic Specification

The basic specification depends on whether you want a one-sample test, an independent-samples test or a paired-samples test. For all types of tests, T-TEST displays Student's *t,* degrees of freedom, and two-tailed probabilities, as well as the mean, standard deviation, standard error, and count for each group or variable.

- To request a one-sample test, use the TESTVAL and VARIABLES subcommands. The output includes a One-Sample Statistics table showing univariate statistics and a One-Sample Test table showing the test value, the difference between the sample mean and the test value, and the two-tailed probability level.
- To request an independent-samples test, use the GROUPS and VARIABLES subcommands. The output includes a Group Statistics table showing summary statistics by group for each dependent variable and an Independent-Samples Test table showing both pooled- and separate-variance estimates, along with the *F* value used to test homogeneity of variance and its probability. The two-tailed probability is displayed for the *t* value.
- To request a paired-samples test, use the PAIRS subcommand. The output includes a Paired Statistics table showing univariate statistics by pairs, a Paired Samples Correlations table showing correlation coefficients and two-tailed probability level for a test of the coefficient for each pair, and a Paired Samples Test table showing the paired differences between the means and two-tailed probability levels for a test of the differences.

Subcommand Order

Subcommands can be named in any order.

Operations

- If a variable specified on GROUPS is a long string, only the short-string portion is used to identify groups in the analysis.
- Probability levels are two-tailed. To obtain the one-tailed probability, divide the two-tailed probability by 2.

Limitations

- Maximum of one TESTVAL and one VARIABLES subcommand per one-sample *t* test.
- Maximum of one GROUPS and one VARIABLES subcommand per independent-samples *t* test.

Examples

One-Sample Test

```
T-TEST
  /TESTVAL=322
  /VARIABLES=brake
  /CRITERIA=CIN (.90).
```

- The procedure compares the mean of *brake* with the test value 322.
- CRITERIA requests a 90% confidence interval for the mean difference.

Independent-Samples Test, using specific values

```
T-TEST GROUPS=insert(0 1)
  /MISSING=ANALYSIS
  /VARIABLES=dollars
  /CRITERIA=CIN(.95).
```

- The procedure compares the means of the two groups defined by values 0 and 1 of *insert* for variable *dollars*.
- All other options are set to their default values.

Paired-Samples Test

```
T-TEST PAIRS=TEACHER CONSTRUC MANAGER.
```

- This paired-samples *t* test compares the means of *TEACHER* with *CONSTRUC*, *TEACHER* with *MANAGER*, and *CONSTRUC* with *MANAGER*.

VARIABLES Subcommand

VARIABLES specifies the dependent variables to be tested in a one-sample or an independent-samples *t* test.

- VARIABLES can specify multiple variables, all of which must be numeric.
- When specified along with TESTVAL, the mean of all cases for each variable is compared with the specified value.
- When specified along with GROUPS, the means of two groups of cases defined by the GROUPS subcommand are compared.
- If both TESTVAL and GROUPS are specified, a one-sample test and an independent-samples test are performed on each variable.

TESTVAL Subcommand

TESTVAL specifies the value with which a sample mean is compared.

- Only one TESTVAL subcommand is allowed.
- Only one value can be specified on the TESTVAL subcommand.

GROUPS Subcommand

GROUPS specifies a variable used to group cases for independent-samples *t* tests.

- GROUPS can specify only one variable, which can be numeric or string.

Any one of three methods can be used to define the two groups for the variable specified on GROUPS:

- Specify a single value in parentheses to group all cases with a value equal to or greater than the specified value into one group and the remaining cases into the other group.
- Specify two values in parentheses to include cases with the first value in one group and cases with the second value in the other group. Cases with other values are excluded.
- If no values are specified on GROUP, T-TEST uses 1 and 2 as default values for numeric variables. There is no default for string variables.

PAIRS Subcommand

PAIRS requests paired-samples *t* tests.

- The minimum specification for a paired-samples test is PAIRS with an analysis list. Only numeric variables can be specified on the analysis list. The minimum analysis list is two variables.
- If keyword WITH is not specified, each variable in the list is compared with every other variable on the list.
- If keyword WITH is specified, every variable to the left of WITH is compared with every variable to the right of WITH. WITH can be used with PAIRED to obtain special pairing.
- To specify multiple analysis lists, use multiple PAIRS subcommands, each separated by a slash. Keyword PAIRS is required only for the first analysis list; a slash can be used to separate each additional analysis list.

(PAIRED). *Special pairing for paired-samples test.* PAIRED must be enclosed in parentheses and must be used with keyword WITH. When PAIRED is specified, the first variable before WITH is compared with the first variable after WITH, the second variable before WITH is compared with the second variable after WITH, and so forth. The same number of variables should be specified before and after WITH; unmatched

variables are ignored and a warning message is issued. PAIRED generates an error message if keyword WITH is not specified on PAIRS.

Example

```
T-TEST  PAIRS=TEACHER CONSTRUC MANAGER.
T-TEST  PAIRS=TEACHER MANAGER WITH CONSTRUC ENGINEER.
T-TEST  PAIRS=TEACHER MANAGER WITH CONSTRUC ENGINEER (PAIRED).
```

- The first T-TEST compares *TEACHER* with *CONSTRUC, TEACHER* with *MANAGER*, and *CONSTRUC* with *MANAGER*.
- The second T-TEST compares *TEACHER* with *CONSTRUC*, *TEACHER* with *ENGINEER*, *MANAGER* with *CONSTRUC*, and *MANAGER* with *ENGINEER*. *TEACHER* is not compared with *MANAGER*, and *CONSTRUC* is not compared with *ENGINEER*.
- The third T-TEST compares *TEACHER* with *CONSTRUC* and *MANAGER* with *ENGINEER*.

CRITERIA Subcommand

CRITERIA resets the value of the confidence interval. Keyword CI is required. You can specify a value between 0 and 1 in the parentheses. The default is 0.95.

ES Subcommand

ES controls the estimation of the *t*-test effect size.

DISPLAY = boolean
An optional keyword that determines whether the effect size table is printed. **TRUE** is the default setting, which invokes the estimation of effect size. When **FALSE** is specified, effect size is not estimated, and the corresponding output table is suppressed.

STANDARDIZER = {SD} {CORRECTED_SD} {AVERAGE}
An optional keyword that controls how the standardizer is computed in estimating the Cohen's *d* and Hedges' correction for each variable pair. **SD** is the default setting (Standard deviation of the difference). The other options are **CORRECTED_SD** (corrected standard deviation of the difference) and **AVERAGE** (average of variances). The keyword takes effect only in the related-sample *t*-test, and it is ignored by the one-sample and independent-sample *t*-tests, or when DISPLAY = FALSE.

MISSING Subcommand

MISSING controls the treatment of missing values. The default is ANALYSIS.

- ANALYSIS and LISTWISE are alternatives; however, each can be specified with INCLUDE.

ANALYSIS. *Delete cases with missing values on an analysis-by-analysis or pair-by-pair basis.* For independent-samples tests, cases with missing values for either the grouping variable or the dependent variable are excluded from the analysis of that dependent variable. For paired-samples tests, a case with a missing value for either of the variables in a given pair is excluded from the analysis of that pair. This is the default.

LISTWISE. *Exclude cases with missing values listwise.* A case with a missing value for any variable specified on either GROUPS or VARIABLES is excluded from any independent-samples test. A case with a missing value for any variable specified on PAIRS is excluded from any paired-samples test.

INCLUDE. *Include user-missing values.* User-missing values are treated as valid values.

TWOSTEP CLUSTER

TWOSTEP CLUSTER is available in the Statistics Base Edition.

```
TWOSTEP CLUSTER
  [/CATEGORICAL VARIABLES = varlist]

  [/CONTINUOUS VARIABLES = varlist]

  [/CRITERIA [INITHRESHOLD({0**  })] [MXBRANCH({8**})]
                          {value}             {n  }
             [MXLEVEL({3**})] ]
                      {n  }

  [/DISTANCE {EUCLIDEAN   }]
             {LIKELIHOOD**}

  [/HANDLENOISE {0**}]
                {n  }

  [/INFILE FILE = filename]

  [/MEMALLOCATE {64**}]
                {n   }

  [/MISSING {EXCLUDE**}]
            {INCLUDE  }

  [/NOSTANDARDIZE [VARIABLES = varlist]]

  [/NUMCLUSTERS {AUTO** {15**} [{AIC  }]}]
                        {n   }  {BIC**}
                {FIXED = n              }

  [/OUTFILE [MODEL = 'file'] [STATE = 'file']]

  [/PRINT [IC] [COUNT] [SUMMARY]]

  [/VIEWMODEL [DISPLAY = {YES**}]
                         {NO   }
              [EVALUATIONFIELDS = varlist]]

  [/SAVE CLUSTER [VARIABLE = varname]]
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
TWOSTEP CLUSTER
  /CONTINUOUS VARIABLES = INCOME
  /CATEGORICAL VARIABLES = GENDER RACE
  /PRINT SUMMARY.
```

Overview

TWOSTEP CLUSTER groups observations into clusters based on a nearness criterion. The procedure uses a hierarchical agglomerative clustering procedure in which individual cases are successively combined to form clusters whose centers are far apart. This algorithm is designed to cluster large numbers of cases. It passes the data once to find the cluster centers and again to assign cluster memberships. In addition to the benefit of few data passes, the procedure allows the user to set the amount of memory used by the clustering algorithm.

Basic Features

Cluster Features (CF) Tree. TWOSTEP CLUSTER clusters observations by building a data structure called the **CF tree**, which contains the cluster centers. The CF tree is grown during the first stage of clustering and values are added to its leaves if they are close to the cluster center of a particular leaf.

Distance Measure. Two types of distance measures are offered—the traditional Euclidean distance and the likelihood distance. The former is available when no categorical variables are specified. The latter is especially useful when categorical variables are used. The likelihood function is computed using the

normal density for continuous variables and the multinomial probability mass function for categorical variables. All variables are treated as independent.

Tuning the Algorithm. You can control the values of algorithm-tuning parameters with the CRITERIA subcommand.

Noise Handling. The clustering algorithm can optionally retain any outliers that do not fit in the CF tree. If possible, these values will be placed in the CF tree after it is completed. Otherwise, TWOSTEP CLUSTER will discard them after preclustering.

Missing Values. TWOSTEP CLUSTER will delete listwise any records with missing fields.

Numclusters. This subcommand specifies the number of clusters into which the data will be partitioned. The user may tell TWOSTEP CLUSTER to automatically select the number of clusters.

Optional Output. You can specify output to an XML file with the OUTFILE subcommand. The cluster membership for each case used can be saved to the active dataset with the SAVE subcommand.

Weights. TWOSTEP CLUSTER ignores specification on the WEIGHT command.

Basic Specification

- The minimum specification is a list of variables, either categorical or continuous, to be clustered and at least one of the following subcommands: OUTFILE, PRINT, or SAVE.
- The number of clusters may be specified with the NUMCLUSTERS subcommand.
- Unless the NOSTANDARDIZE subcommand is given, TWOSTEP CLUSTER will standardize all continuous variables.
- If DISTANCE is Euclidean, TWOSTEP CLUSTER will accept only continuous variables.

Subcommand Order

- The subcommands can be specified in any order.

Syntax Rules

- Minimum syntax: a variable must be specified.
- Empty subcommands are silently ignored.
- Variables listed in the CONTINUOUS subcommand must be numeric.
- If a subcommand is issued more than once, TWOSTEP CLUSTER will ignore all but the last issue.

Variable List

The variable lists specify those variables to be clustered. The first variable list specifies only continuous variables, and the second list specifies only categorical variables (that is, the two lists are disjoint).

CATEGORICAL Subcommand

The CATEGORICAL subcommand specifies a list of categorical variables.

Example

```
TWOSTEP CLUSTER
  /CATEGORICAL VARIABLES = RACE GENDER CITIZEN
  /PRINT SUMMARY COUNT.
```

This tells TWOSTEP CLUSTER to cluster the categorical variables *RACE*, *GENDER,* and *CITIZEN*. Summary statistics by cluster and cluster frequencies are output in tables.

CONTINUOUS Subcommand

The CONTINUOUS subcommand specifies a list of scale variables.

Example

```
TWOSTEP CLUSTER
  /CATEGORICAL VARIABLES = RACE GENDER CITIZEN
  /CONTINUOUS VARIABLES = INCOME
  /PRINT SUMMARY COUNT.
```

This tells TWOSTEP CLUSTER to cluster the categorical variables *RACE*, *GENDER,* and *CITIZEN*, and the numeric variable *INCOME*. Summary statistics by cluster and cluster frequencies are output in tables.

CRITERIA Subcommand

The CRITERIA subcommand specifies the following settings for the clustering algorithm:

INITTHRESHOLD. *The initial threshold used to grow the CF tree.* The default is 0. If inserting a specific case into a leaf of the CF tree would yield tightness less than the threshold, the leaf is not split. If the tightness exceeds the threshold, the leaf is split.

MXBRANCH. *The maximum number of child nodes that a leaf node can have.* The default is 8.

MXLEVEL. *The maximum number of levels that the CF tree can have.* The default is 3.

DISTANCE Subcommand

The distance subcommand determines how distances will be computed between clusters.

EUCLIDEAN. *Use the Euclidean distance to compute distances between clusters.* You may select Euclidean distance if all variables are continuous. TWOSTEP CLUSTER will return a syntax error if you specify Euclidean distance with non-numeric variables.

LIKELIHOOD. *Use the minus log-likelihood to compute distances between clusters.* This is the default. The likelihood function is computed assuming all variables are independent. Continuous variables are assumed to be normally distributed, and categorical variables are assumed to be multinomially distributed.

HANDLENOISE Subcommand

The HANDLENOISE subcommand tells TWOSTEP CLUSTER to treat outliers specially during clustering. During growth of the CF tree, this subcommand is relevant only if the CF tree fills. The CF tree is full if it cannot accept any more cases in a leaf node and no leaf node can be split. The default value of HANDLENOISE is 0, equivalent to no noise handling.

- If the CF tree fills and HANDLENOISE is greater than 0, the CF tree will be regrown after placing any data in sparse leaves into their own noise leaf. A leaf is considered sparse if the ratio of the number of cases in the sparse leaf to the number of cases in the largest leaf is less than HANDLENOISE. After the tree is grown, the outliers will be placed in the CF tree, if possible. If not, the outliers are discarded for the second phase of clustering.
- If the CF tree fills and HANDLENOISE is equal to 0, the threshold will be increased and CF tree regrown with all cases. After final clustering, values that cannot be assigned to a cluster are labeled outliers. The outlier cluster is given an identification number of –1. The outlier cluster is not included in the count of the number of clusters; that is, if you specify *n* clusters and noise handling, TWOSTEP CLUSTER will output *n* clusters and one noise cluster.

Example

```
TWOSTEP CLUSTER
  /CATEGORICAL VARIABLES = RACE GENDER CITIZEN
  /CONTINUOUS VARIABLES = INCOME
  /HANDLENOISE 25
  /PRINT SUMMARY COUNT.
```

This tells TWOSTEP CLUSTER to cluster the categorical variables *RACE*, *GENDER* and *CITIZEN*, and the numeric variable *INCOME*. If the CF tree fills, a noise leaf is constructed from cases whose leaves contain fewer than 25 percent of the cases contained by the largest leaf. The CF tree is then regrown, ignoring the noise leaf. After the tree is regrown, cases from the noise leaf are checked to see if they fit any of the leaves in the new tree. Any cases that still do not fit are discarded as outliers. Summary statistics by cluster and cluster frequencies are output in tables.

INFILE Subcommand

The INFILE subcommand causes TWOSTEP CLUSTER to update a cluster model whose CF Tree has been saved as an XML file with the OUTFILE subcommand and STATE keyword. The model will be updated with the data in the active file. The user must supply variable names in the active file in the order they are stored in the XML file. TWOSTEP CLUSTER will update the cluster model in memory only, leaving unaltered the XML file.

- If the INFILE subcommand is given, TWOSTEP CLUSTER will ignore the CRITERIA, DISTANCE, HANDLENOISE and MEMALLOCATE subcommands, if given.

MEMALLOCATE Subcommand

The MEMALLOCATE subcommand specifies the maximum amount of memory in megabytes (MB) that the cluster algorithm should use. If the procedure exceeds this maximum, it will use the disk to store information that will not fit in memory.

- The minimum value that you can specify is 4. If this subcommand is not specified, the default value is 64 MB.
- Consult your system administrator for the largest value you can specify on your system.

MISSING Subcommand

The MISSING subcommand specifies how to handle cases with user-missing values.

- If this subcommand is not specified, the default is EXCLUDE.
- TWOSTEP CLUSTER deletes any case with a system-missing value.
- Keywords EXCLUDE and INCLUDE are mutually exclusive. Only one of them can be specified.

EXCLUDE. *Exclude both user-missing and system-missing values.* This is the default.

INCLUDE. *User-missing values are treated as valid.* System-missing values cannot be included in the analysis.

NOSTANDARDIZE Subcommand

The NOSTANDARDIZE subcommand will prevent TWOSTEP CLUSTER from standardizing the continuous variables specified with the VARIABLES keyword. If this subcommand is not specified, or given without a variable list, TWOSTEP CLUSTER will standardize all continuous variables by subtracting the mean and dividing by the standard deviation.

NUMCLUSTERS Subcommand

The NUMCLUSTERS subcommand specifies the number of clusters into which the data will be partitioned.

AUTO. *Automatic selection of the number of clusters.* Under AUTO, you may specify a maximum number of possible clusters. TWOSTEP CLUSTER will search for the best number of clusters between 1 and the maximum using the criterion that you specify. The criterion for deciding the number of clusters can be either the Bayesian Information Criterion (BIC) or Akaike Information Criterion (AIC). TWOSTEP CLUSTER will find at least one cluster if the AUTO keyword is given.

FIXED. *User-specified number of clusters.* Specify a positive integer.

Examples

```
TWOSTEP CLUSTER
  /CONTINUOUS VARIABLES = INCOME
  /CATEGORICAL VARIABLES = GENDER RACE
  /NUMCLUSTERS AUTO 10 AIC
  /PRINT SUMMARY COUNT.
```

TWOSTEP CLUSTER uses the variables *RACE*, *GENDER* and *INCOME* for clustering. Specifications on the NUMCLUSTERS subcommand will instruct the procedure to automatically search for the number of clusters using the Akaike Information Criterion and require the answer to lie between 1 and 10.

```
TWOSTEP CLUSTER
  /CONTINUOUS VARIABLES = INCOME
  /CATEGORICAL VARIABLES = RACE GENDER
  /NUMCLUSTERS FIXED 7
  /PRINT SUMMARY COUNT.
```

Here the procedure will find exactly seven clusters.

OUTFILE Subcommand

The OUTFILE subcommand directs TWOSTEP CLUSTER to write its output to the specified filename as XML.

MODEL. *Save the final model output.*

STATE. *Save the CF tree.* Use the STATE keyword if you want to update the model later.

You must supply a valid filename on your operating system. We recommend specifying the full path of the filename.

PRINT Subcommand

The PRINT subcommand causes TWOSTEP CLUSTER to print tables related to each cluster.

IC. *Information criterion.* Prints the chosen information criterion (AIC or BIC) for different numbers of clusters. If this keyword is specified when the AUTO keyword is not used with the NUMCLUSTERS subcommand, TWOSTEP CLUSTER will skip this keyword and issue a warning. TWOSTEP CLUSTER will ignore this keyword if AUTO 1 is specified in the NUMCLUSTERS subcommand.

SUMMARY. *Descriptive statistics by cluster.* This option prints two tables describing the variables in each cluster. In one table, means and standard deviations are reported for continuous variables. The other table reports frequencies of categorical variables. All values are separated by cluster.

COUNT. *Cluster frequencies.* This option prints a table containing a list of clusters and how many observations are in each cluster.

VIEWMODEL Subcommand

The VIEWMODEL subcommand controls options for display of model-related output, including tables and charts. Tables in the model view include a model summary and cluster-by-features grid. Graphical output in the model view includes a cluster quality chart, cluster sizes, variable importance, cluster comparison grid, and cell information.

DISPLAY Keyword

The DISPLAY keyword specifies whether to display the model view in the output.

YES. *Display the model view.* This is the default.

NO. *Do not display the model view.*

EVALUATIONFIELDS Keyword

The EVALUATIONFIELDS keyword calculates cluster data for variables that were not used in cluster creation. Evaluation fields can be displayed along with the input features in the model viewer by selecting them in the Display subdialog. The EVALUATIONFIELDS keyword is optional and its varlist can include fields (variables) in the dataset that are not used to create the cluster. Fields that are used to create clusters will be ignored. Fields with missing values are ignored.

```
TWOSTEP CLUSTER
  /CATEGORICAL VARIABLES=var1 var2 var3
  /CONTINUOUS VARIABLES=var4 var5
  /VIEWMODEL DISPLAY=YES EVALUATIONFIELDS=var1 var2 var8 var9
```

- The procedure creates a cluster model using *var1*, *var2*, *var3* as categorical variables and *var4* and *var5* as continuous variables, and produces a model viewer with *var8* and *var9* available as evaluation fields.

SAVE Subcommand

The SAVE subcommand allows you to save cluster output to the active dataset.

CLUSTER. *Save the cluster identification.* The cluster number for each case is saved. You can specify a variable name using the VARIABLE keyword, otherwise, it is saved to *TSC_n*, where *n* is a positive integer indicating the ordinal of the SAVE operation completed by this procedure in a given session. If you specify a variable name with the VARIABLE keyword, it cannot be a variable that already exists; it must be a new, unique variable name.

UNIANOVA

UNIANOVA is available in Statistics Base Edition.

```
UNIANOVA dependent var [BY factor list [WITH covariate list]]

[/RANDOM=factor factor...]

[/REGWGT=varname]

[/METHOD=SSTYPE({1  })]
               {2  }
               {3**}
               {4  }

[/INTERCEPT=[INCLUDE**] [EXCLUDE]]

[/MISSING=[INCLUDE] [EXCLUDE**]]

[/CRITERIA=[EPS({1E-8**})][ALPHA({0.05**})]
               {a     }         {a     }

[/PRINT = [DESCRIPTIVE] [HOMOGENEITY] [PARAMETER][ETASQ]
          [GEF] [LOF] [OPOWER] [TEST(LMATRIX)]
          [MBP] [BP] [F] [WHITE] ]

[/PLOT=[SPREADLEVEL] [RESIDUALS]
       [PROFILE (factor factor*factor factor*factor*factor ...)
                [WITH(covariate={value} [...])]]
                              {MEAN }
                [TYPE={LINE**}]
                      {BAR   }
                [MEANREFERENCE={NO**}]
                               {YES }
                [ERRORBAR={NO**      }]
                          {CI        }
                          {SE [(2**)]}
                [YAXIS={AUTO**}]
                       {0     }

[/TEST=effect VS {linear combination [DF(df)]}]
                 {value DF (df)              }

[/LMATRIX={["label"] effect list effect list ...;...}]
          {["label"] effect list effect list ...    }
          {["label"] ALL list; ALL...               }
          {["label"] ALL list                       }

[/KMATRIX= {number    }]
           {number;...}

[/CONTRAST (factor name)={DEVIATION[(refcat)]**     }]
                         {SIMPLE [(refcat)]         }
                         {DIFFERENCE                }
                         {HELMERT                   }
                         {REPEATED                  }
                         {POLYNOMIAL  [({1,2,3...})]}
                                      {metric  }
                         {SPECIAL (matrix)          }

[/POSTHOC =effect [effect...]
          ([SNK] [TUKEY] [BTUKEY][DUNCAN]
           [SCHEFFE] [DUNNETT(refcat)] [DUNNETTL(refcat)]
           [DUNNETTR(refcat)] [BONFERRONI] [LSD] [SIDAK]
           [GT2] [GABRIEL] [FREGW] [QREGW]  [T2] [T3] [GH] [C]
           [WALLER ({100** })])]
                   {kratio}
           [VS effect]

[/EMMEANS=TABLES({OVERALL         }) [WITH(covariate={value} [...])]
                 {factor          }                 {MEAN }
                 {factor*factor...}

          [COMPARE ADJ(LSD) (BONFERRONI) (SIDAK)]

[/ROBUST={HC0  })]
         {HC1  }
         {HC2  }
         {HC3**}
         {HC4  }

 [/SAVE=[tempvar [(name)]] [tempvar [(name)]]...]

 [/OUTFILE=[COVB ('savfile'|'dataset')]
           [CORB ('savfile'|'dataset')]
           [HCCOVB ('savfile'|'dataset')]
           [EFFECT('savfile'|'dataset')] [DESIGN('savfile'|'dataset')]


 [/MBPDESIGN=[effect effect...]]

 [/BPDESIGN=[effect effect...]]

 [/FDESIGN=[effect effect...]]

 [/DESIGN={[INTERCEPT...]    }]
          {[effect effect...]}
```

** Default if the subcommand or keyword is omitted.

Temporary variables (`tempvar`) are:

```
PRED, WPRED, RESID, WRESID, DRESID, ZRESID, SRESID, SEPRED, COOK, LEVER
```

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 17.0

- POSTHOC subcommand: T2, T3, GH, and C keywords are not valid when multiple factors in the model.

Release 25.0

- TYPE, ERRORBAR, MEANREFERENCE, and YAXIS keywords added to PLOT subcommand.
- ROBUST subcommand added.
- HCCOVB keyword added to OUTFILE subcommand. Any combination of COVB, CORB, and HCCOVB keywords can now be specified on OUTFILE.
- MBP, BP, F, and WHITE keywords added to PRINT subcommand.
- MBPDESIGN, BPDESIGN, and FDESIGN subcommands added.

Example

```
UNIANOVA YIELD BY SEED FERT
  /DESIGN.
```

Overview

This section describes the use of UNIANOVA for univariate analyses. The UNIANOVA procedure provides regression analysis and analysis of variance for one dependent variable by one or more factors and/or variables.

Options

Design Specification. You can specify which terms to include in the design on the DESIGN subcommand. This allows you to estimate a model other than the default full factorial model, incorporate factor-by-covariate interactions or covariate-by-covariate interactions, and indicate nesting of effects.

Contrast Types. You can specify contrasts other than the default deviation contrasts on the CONTRAST subcommand.

Optional Output. You can choose from a wide variety of optional output on the PRINT subcommand. Output appropriate to univariate designs includes descriptive statistics for each cell, parameter estimates, Levene's test for equality of variance across cells, tests for heteroskedasticity, partial eta-squared for each effect and each parameter estimate, the general estimable function(s) matrix, and a contrast coefficients table (**L'** matrix). The OUTFILE subcommand allows you to write out the covariance or correlation matrix, the design matrix, or the statistics from the between-subjects ANOVA table into a separate data file.

Using the EMMEANS subcommand, you can request tables of estimated marginal means of the dependent variable and their standard deviations. The SAVE subcommand allows you to save predicted values and residuals in weighted or unweighted and standardized or unstandardized forms. You can specify different means comparison tests for comparing all possible pairs of cell means using the POSTHOC subcommand. In addition, you can specify your own hypothesis tests by specifying an **L** matrix and a **K** matrix to test the univariate hypothesis **LB** = **K**.

You can display robust or heteroskedasticity-consistent (HC) standard errors by using the ROBUST subcommand, and you can write the robust covariance matrix estimates to a new file or dataset. You can also display a second set of output for custom hypothesis tests, with results based on the specified robust covariance matrix estimator.

Basic Specification

- The basic specification is a variable list identifying the dependent variable, the factors (if any), and the covariates (if any).
- By default, UNIANOVA uses a model that includes the intercept term, the covariate (if any), and the full factorial model, which includes all main effects and all possible interactions among factors. The intercept term is excluded if it is excluded in the model by specifying the keyword EXCLUDE on the INTERCEPT subcommand. Sums of squares are calculated and hypothesis tests are performed using type-specific estimable functions. Parameters are estimated using the normal equation and a generalized inverse of the SSCP matrix.

Subcommand Order

- The variable list must be specified first.
- Subcommands can be used in any order.

Syntax Rules

- For many analyses, the UNIANOVA variable list and the DESIGN subcommand are the only specifications needed.
- If you do not enter a DESIGN subcommand, UNIANOVA will use a full factorial model, with main effects of covariates, if any.
- At least one dependent variable must be specified, and at least one of the following must be specified: INTERCEPT, a factor, or a covariate. The design contains the intercept by default.
- If more than one DESIGN subcommand is specified, only the last one is in effect.
- Dependent variables and covariates must be numeric, but factors can be numeric or string variables.
- If more than one MISSING subcommand is specified, only the last one is in effect.
- If more than one ROBUST subcommand is specified, only the last one is in effect.
- The following words are reserved as keywords or internal commands in the UNIANOVA procedure:

INTERCEPT, BY, WITH, ALL, OVERALL, WITHIN

Variable names that duplicate these words should be changed before you run UNIANOVA.

Limitations

- Any number of factors can be specified, but if the number of factors plus the number of split variables exceeds 18, the Descriptive Statistics table is not printed even when you request it.
- Memory requirements depend primarily on the number of cells in the design. For the default full factorial model, this equals the product of the number of levels or categories in each factor.

Example

```
UNIANOVA YIELD BY SEED FERT WITH RAINFALL
  /PRINT=DESCRIPTIVE PARAMETER
  /DESIGN.
```

- *YIELD* is the dependent variable; *SEED* and *FERT* are factors; *RAINFALL* is a covariate.
- The PRINT subcommand requests the descriptive statistics for the dependent variable for each cell and the parameter estimates, in addition to the default tables Between-Subjects Factors and Univariate Tests.
- The DESIGN subcommand requests the default design, a full factorial model with a covariate. This subcommand could have been omitted or could have been specified in full as

```
/DESIGN = INTERCEPT RAINFALL, SEED, FERT, SEED BY FERT.
```

UNIANOVA Variable List

The variable list specifies the dependent variable, the factors, and the covariates in the model.

- The dependent variable must be the first specification on UNIANOVA.
- The names of the factors follow the dependent variable. Use the keyword BY to separate the factors from the dependent variable.
- Enter the covariates, if any, following the factors. Use the keyword WITH to separate covariates from factors (if any) and the dependent variable.

Example

```
UNIANOVA DEPENDNT BY FACTOR1 FACTOR2, FACTOR3.
```

- In this example, three factors are specified.
- A default full factorial model is used for the analysis.

Example

```
UNIANOVA Y BY A WITH X
  /DESIGN.
```

- In this example, the DESIGN subcommand requests the default design, which includes the intercept term, the covariate *X*, and the factor *A*.

RANDOM Subcommand

RANDOM allows you to specify which effects in your design are random. When the RANDOM subcommand is used, a table of expected mean squares for all effects in the design is displayed, and an appropriate error term for testing each effect is calculated and used automatically.

- Random always implies a univariate mixed-model analysis.
- If you specify an effect on RANDOM, higher-order effects containing the specified effect (excluding any effects containing covariates) are automatically treated as random effects.
- The keyword INTERCEPT and effects containing covariates are not allowed on this subcommand.
- When the RANDOM subcommand is used, the appropriate error terms for the hypothesis testing of all effects in the model are automatically computed and used.
- More than one RANDOM subcommand is allowed. The specifications are accumulated.

Example

```
UNIANOVA DEP BY A B
  /RANDOM = B
  /DESIGN = A,B, A*B.
```

- In the example, effects *B* and *A*B* are considered as random effects. Notice that if only effect *B* is specified in the RANDOM subcommand, *A*B* is automatically considered as a random effect.
- The hypothesis testing for each effect (*A*, *B*, and *A*B*) in the design will be carried out using the appropriate error term, which is calculated automatically.

REGWGT Subcommand

The only specification on REGWGT is the name of the variable containing the weights to be used in estimating a weighted least-squares model.

- Specify a numeric weight variable name following the REGWGT subcommand. Only observations with positive values in the weight variable will be used in the analysis.

- If more than one REGWGT subcommand is specified, only the last one is in effect.

Example

```
UNIANOVA OUTCOME BY TREATMNT
  /REGWGT WT.
```

- The procedure performs a weighted least-squares analysis. The variable *WT* is used as the weight variable.

METHOD Subcommand

METHOD controls the computational aspects of the UNIANOVA analysis. You can specify one of four different methods for partitioning the sums of squares. If more than one METHOD subcommand is specified, only the last one is in effect.

SSTYPE(1). *Type I sum-of-squares method.* The Type I sum-of-squares method is also known as the hierarchical decomposition of the sum-of-squares method. Each term is adjusted only for the terms that precede it on the DESIGN subcommand. Under a balanced design, it is an orthogonal decomposition, and the sums of squares in the model add up to the total sum of squares.

SSTYPE(2). *Type II sum-of-squares method.* This method calculates the sum of squares of an effect in the model adjusted for all other "appropriate" effects. An appropriate effect is one that corresponds to all effects that do not contain the effect being examined.

For any two effects *F1* and *F2* in the model, *F1* is said to be **contained** in *F2* under the following three conditions: 1) both effects *F1* and *F2* have the same covariate, if any, 2) *F2* consists of more factors than *F1*, and 3) all factors in *F1* also appear in *F2*. The intercept effect is treated as contained in all of the pure factor effects. However, it is not contained in any effect involving a covariate. No effect is contained in the intercept effect. Thus, for any one effect *F* of interest, all other effects in the model can be classified as in one of the following two groups: the effects that do not contain *F* or the effects that contain *F*. If the model is a main-effects design (that is, only main effects are in the model), the Type II sum-of-squares method is equivalent to the regression approach sums of squares. This means that each main effect is adjusted for every other term in the model.

SSTYPE(3). *Type III sum-of-squares method.* This is the default. This method calculates the sum of squares of an effect *F* in the design as the sum of squares adjusted for any other effects that do not contain it, and *orthogonal* to any effects (if any) that contain it. The Type III sums of squares have one major advantage—they are invariant with respect to the cell frequencies as long as the general form of estimability remains constant. Hence, this type of sums of squares is often used for an unbalanced model with no missing cells. In a factorial design with no missing cells, this method is equivalent to the Yates' weighted squares of means technique, and it also coincides with the overparameterized ∑-restricted model.

SSTYPE(4). *Type IV sum-of-squares method.* This method is designed for a situation in which there are missing cells. For any effect *F* in the design, if *F* is not contained in any other effect, then Type IV = Type III = Type II. When *F* is contained in other effects, then Type IV distributes the contrasts being made among the parameters in *F* to all higher-level effects equitably.

Example

```
UNIANOVA DEP BY A B C
  /METHOD=SSTYPE(3)
  /DESIGN=A, B, C.
```

- The design is a main-effects model.
- The METHOD subcommand requests that the model be fitted with Type III sums of squares.

INTERCEPT Subcommand

INTERCEPT controls whether an intercept term is included in the model. If more than one INTERCEPT subcommand is specified, only the last one is in effect.

INCLUDE. *Include the intercept term.* The intercept (constant) term is included in the model. This is the default.

EXCLUDE. *Exclude the intercept term.* The intercept term is excluded from the model. Specification of the keyword INTERCEPT on the DESIGN subcommand overrides INTERCEPT = EXCLUDE.

MISSING Subcommand

By default, cases with missing values for any of the variables on the UNIANOVA variable list are excluded from the analysis. The MISSING subcommand allows you to include cases with user-missing values.

- If MISSING is not specified, the default is EXCLUDE.
- Pairwise deletion of missing data is not available in UNIANOVA.
- Keywords INCLUDE and EXCLUDE are mutually exclusive.
- If more than one MISSING subcommand is specified, only the last one is in effect.

EXCLUDE. *Exclude both user-missing and system-missing values.* This is the default when MISSING is not specified.

INCLUDE. *User-missing values are treated as valid.* System-missing values cannot be included in the analysis.

CRITERIA Subcommand

CRITERIA controls the statistical criteria used to build the models.

- More than one CRITERIA subcommand is allowed. The specifications are accumulated. Conflicts across CRITERIA subcommands are resolved using the conflicting specification given on the last CRITERIA subcommand.
- The keyword must be followed by a positive number in parentheses.

EPS(n). *The tolerance level in redundancy detection.* This value is used for redundancy checking in the design matrix. The default value is 1E–8.

ALPHA(n). *The alpha level.* This keyword has two functions. First, it gives the alpha level at which the power is calculated for the *F* test. Once the noncentrality parameter for the alternative hypothesis is estimated from the data, then the power is the probability that the test statistic is greater than the critical value under the alternative hypothesis. The second function of alpha is to specify the level of the confidence interval. If the alpha level specified is *n*, the value (1–n)×100 indicates the level of confidence for all individual and simultaneous confidence intervals generated for the specified model. The value of *n* must be between 0 and 1, exclusive. The default value of alpha is 0.05. This means that the default power calculation is at the 0.05 level, and the default level of the confidence intervals is 95%, since (1–0.05) ×100=95.

PRINT Subcommand

PRINT controls the display of optional output.

- Some PRINT output applies to the entire UNIANOVA procedure and is displayed only once.
- Additional output can be obtained on the EMMEANS, PLOT, and SAVE subcommands.

- Some optional output may greatly increase the processing time. Request only the output you want to see.
- If no PRINT command is specified, default output for a univariate analysis includes a factor information table and a Univariate Tests table (ANOVA) for all effects in the model.
- If more than one PRINT subcommand is specified, only the last one is in effect.

The following keywords are available for UNIANOVA univariate analyses.

DESCRIPTIVES. *Basic information about each cell in the design.* Observed means, standard deviations, and counts for the dependent variable in all cells. The cells are constructed from the highest-order crossing of the factors. If the number of factors plus the number of split variables exceeds 18, the Descriptive Statistics table is not printed.

HOMOGENEITY. *Test(s) of homogeneity of variance.* Levene's test for equality of variances for the dependent variable across all level combinations of the factors. For full-factorial models without covariates, Levene statistics based on the mean, median, and trimmed mean, and with adjusted degrees of freedom for the test using the median are displayed. For other models only the original Levene statistic based on the mean is displayed.

PARAMETER. *Parameter estimates.* Parameter estimates, standard errors, *t* tests, and confidence intervals for each test.

OPOWER. *Observed power.* The observed power for each test.

LOF. *Lack of fit.* Lack of fit test that allows you to determine if the current model adequately accounts for the relationship between the response variable and the predictors.

ETASQ. Partial eta-squared (η^2). This value is an overestimate of the actual effect size in an *F* test.

GEF. *General estimable function table.* This table shows the general form of the estimable functions.

TEST(LMATRIX). *Set of contrast coefficients (L) matrices.* The transpose of the **L** matrix (**L'**) is displayed. This set always includes one matrix displaying the estimable function for each effect appearing or implied in the DESIGN subcommand. Also, any **L** matrices generated by the LMATRIX or CONTRAST subcommands are displayed. TEST(ESTIMABLE) can be used in place of TEST(LMATRIX).

MBP. *Modified Breusch-Pagan test for heteroskedasticity.* Tests the null hypothesis that the variance of the errors does not depend on the values of the independent variables. You can use the MBPDESIGN subcommand to specify the model on which the test is based. If the MBP keyword is specified without MBPDESIGN, then the model consists of a constant term, a term that is linear in the predicted values, a term that is quadratic in the predicted values, and an error term. For a multivariate model, tests are displayed for each dependent variable.

BP. *Breusch-Pagan test for heteroskedasticity.* Tests the null hypothesis that the variance of the errors does not depend on the values of the independent variables. You can use the BPDESIGN subcommand to specify the model on which the test is based. If the BP keyword is specified without BPDESIGN, then the model consists of a constant term, a term that is linear in the predicted values, a term that is quadratic in the predicted values, and an error term. For a multivariate model, tests are displayed for each dependent variable.

F. *F test for heteroskedasticity.* Tests the null hypothesis that the variance of the errors does not depend on the values of the independent variables. You can use the FDESIGN subcommand to specify the model on which the test is based. If the F keyword is specified without FDESIGN, then the model consists of a constant term, a term that is linear in the predicted values, a term that is quadratic in the predicted values, and an error term. For a multivariate model, tests are displayed for each dependent variable.

WHITE. *White's test for heteroskedasticity.* Tests the null hypothesis that the variance of the errors does not depend on the values of the independent variables. For a multivariate model, tests are displayed for each dependent variable.

Example

```
UNIANOVA DEP BY A B WITH COV
  /PRINT=DESCRIPTIVE, TEST(LMATRIX), PARAMETER
  /DESIGN.
```

- Because the design in the DESIGN subcommand is set as blank, the default design is used. In this case, the design includes the intercept term, the covariate *COV*, and the full factorial terms of *A* and *B*, which are *A*, *B*, and *A*B*.
- For each combination of levels of *A* and *B*, the descriptive statistics of *DEP* are displayed.
- The set of **L** matrices that generates the sums of squares for testing each effect in the design is displayed.
- The parameter estimates, their standard errors, *t* tests, confidence intervals, and the observed power for each test are displayed.

PLOT Subcommand

PLOT provides a variety of plots useful in checking the assumptions needed in the analysis. The PLOT subcommand can be specified more than once. All of the plots requested on each PLOT subcommand are produced.

Use the following keywords on the PLOT subcommand to request plots:

SPREADLEVEL
Spread-versus-level plots. Plots of observed cell means versus standard deviations, and versus variances.

RESIDUALS
Observed by predicted by standardized residuals plot. A plot is produced for each dependent variable. In a univariate analysis, a plot is produced for the single dependent variable.

PROFILE
Plot of dependent variable means for one-way, two-way, or three-way crossed factors. The PROFILE keyword must be followed by parentheses containing a list of one or more factor combinations. All factors specified (either individual or crossed) must be made up of only valid factors on the factor list. Factor combinations on the PROFILE keyword may use an asterisk (*) or the keyword BY to specify crossed factors. A factor cannot occur in a single factor combination more than once.

- The order of factors in a factor combination is important, and there is no restriction on the order of factors. If a single factor is specified after the PROFILE keyword, a line plot of estimated means at each level of the factor is produced. If a two-way crossed factor combination is specified, the output includes a multiple-line plot of estimated means at each level of the first specified factor, with a separate line drawn for each level of the second specified factor. If a three-way crossed factor combination is specified, the output includes multiple-line plots of estimated means at each level of the first specified factor, with separate lines for each level of the second factor, and separate plots for each level of the third factor.
- The PROFILE keyword may be followed by the WITH keyword and a parenthesized list of covariates and values. Plots are produced with the values of the covariates fixed at the specified values. The keyword MEAN causes the covariate to be fixed at its observed mean value.

TYPE=LINE | BAR
Type of plot for profile plots. The default type is LINE.

ERRORBAR=NO | CI | SE (value)
Include error bars in the profile plot. The default setting is NO.

CI
Error bars display the confidence interval. The confidence interval is determined by the ALPHA value on the CRITERIA subcommand. The confidence interval is 1-alpha. For example, if the ALPHA value is 0.05, the confidence interval is 95%.

SE (value)
Error bars display standard errors. The default value is 2. The value must be a positive number.

Example

```
UNIANOVA DEP BY A B
  /PLOT = SPREADLEVEL PROFILE(A A*B A*B*C) WITH(X=1.0)
  /DESIGN.
```

Assume each of the factors *A*, *B*, and *C* has three levels.

- Spread-versus-level plots are produced showing observed cell means versus standard deviations and observed cell means versus variances.
- Five profile plots are produced. For factor *A*, a line plot of estimated means at each level of *A* is produced (one plot). For the two-way crossed factor combination *A*B*, a multiple-line plot of estimated means at each level of *A*, with a separate line for each level of *B*, is produced (one plot). For the three-way crossed factor combination *A*B*C*, a multiple-line plot of estimated means at each level of *A*, with a separate line for each level of *B*, is produced for each of the three levels of *C* (three plots). Plots are produced with the value of the covariate *X* fixed at 1.0.

TEST Subcommand

The TEST subcommand allows you to test a hypothesis term against a specified error term.

- TEST is valid only for univariate analyses. Multiple TEST subcommands are allowed, each executed independently.
- You must specify both the hypothesis term and the error term. There is no default.
- The hypothesis term is specified before the keyword VS. It must be a valid effect specified or implied on the DESIGN subcommand.
- The error term is specified after the keyword VS. You can specify either a linear combination or a value. The linear combination of effects takes the general form: `coefficient*effect +/- coefficient*effect ....`
- All effects in the linear combination must be specified or implied on the DESIGN subcommand. Effects specified or implied on DESIGN but not listed after VS are assumed to have a coefficient of 0.
- Duplicate effects are allowed. UNIANOVA adds coefficients associated with the same effect before performing the text. For example, the linear combination `5*A–0.9*B–A` will be combined to `4*A–0.9B`.
- A coefficient can be specified as a fraction with a positive denominator—for example, 1/3 or –1/3, but 1/–3 is invalid.
- If you specify a value for the error term, you must specify the degrees of freedom after the keyword DF. The degrees of freedom must be a positive real number. DF and the degrees of freedom are optional for a linear combination.

Example

```
UNIANOVA DEP BY A B
  /TEST = A VS B + A*B
  /DESIGN = A, B, A*B.
```

- *A* is tested against the pooled effect of *B + A*B*.

LMATRIX Subcommand

The LMATRIX subcommand allows you to customize your hypotheses tests by specifying the **L** matrix (contrast coefficients matrix) in the general form of the linear hypothesis **LB** = **K**, where **K** = **0** if it is not specified on the KMATRIX subcommand. The vector **B** is the parameter vector in the linear model.

- The basic format for the LMATRIX subcommand is an optional label in quotation marks, an effect name or the keyword ALL, and a list of real numbers. There can be multiple effect names (or the keyword ALL) and number lists.
- The optional label is a string with a maximum length of 255 characters. Only one label can be specified.
- Only valid effects appearing or implied on the DESIGN subcommand can be specified on the LMATRIX subcommand.
- The length of the list of real numbers must be equal to the number of parameters (including the redundant ones) corresponding to that effect. For example, if the effect *A*B* takes up six columns in the design matrix, then the list after *A*B* must contain exactly six numbers.
- A number can be specified as a fraction with a positive denominator—for example, 1/3 or –1/3, but 1/–3 is invalid.
- A semicolon (;) indicates the end of a row in the **L** matrix.
- When ALL is specified, the length of the list that follows ALL is equal to the total number of parameters (including the redundant ones) in the model.
- Effects appearing or implied on the DESIGN subcommand must be explicitly specified here.
- If LMATRIX is specified along with the ROBUST subcommand, then contrast estimates and tests are calculated with the robust covariance matrix, in addition to the tests that use the standard (or model-based) covariance matrix.
- Multiple LMATRIX subcommands are allowed. Each is treated independently.

Example

```
UNIANOVA DEP BY A B
  /LMATRIX = "B1 vs B2 at A1"
             B 1 -1 0 A*B 1 -1 0 0 0 0 0 0 0
  /LMATRIX = "Effect A"
             A 1 0 -1
             A*B 1/3  1/3  1/3
                  0    0    0
                -1/3 -1/3 -1/3;
             A 0 1 -1
             A*B  0    0    0
                 1/3  1/3  1/3
                -1/3 -1/3 -1/3
  /LMATRIX = "B1 vs B2 at A2"
            ALL  0
                 0  0  0
                 1 -1  0
                 0  0  0  1 -1  0 0  0  0
  /DESIGN = A, B, A*B.
```

Assume that factors *A* and *B* each have three levels. There are three LMATRIX subcommands; each is treated independently.

- **B1 versus B2 at A1.** In the first LMATRIX subcommand, the difference is tested between levels 1 and 2 of effect *B* when effect *A* is fixed at level 1. Since there are three levels each in effects *A* and *B*, the interaction effect *A*B* takes up nine columns in the design matrix.
- **Effect A.** In the second LMATRIX subcommand, effect *A* is tested. Since there are three levels in effect *A*, at most two independent contrasts can be formed; thus, there are two rows in the L matrix, which are separated by a semicolon (;). The first row tests the difference between levels 1 and 3 of effect *A*, while the second row tests the difference between levels 2 and 3 of effect *A*.
- **B1 versus B2 at A2.** In the last LMATRIX subcommand, the keyword ALL is used. The first 0 corresponds to the intercept effect; the next three zeros correspond to effect *A*.

KMATRIX Subcommand

The KMATRIX subcommand allows you to customize your hypothesis tests by specifying the **K** matrix (contrast results matrix) in the general form of the linear hypothesis **LB** = **K**. The vector **B** is the parameter vector in the linear model.

- The default **K** matrix is a zero matrix; that is, **LB** = **0** is assumed.

- For the KMATRIX subcommand to be valid, at least one of the following subcommands must be specified: the LMATRIX subcommand or the INTERCEPT = INCLUDE subcommand.
- If KMATRIX is specified but LMATRIX is not specified, the LMATRIX is assumed to take the row vector corresponding to the intercept in the estimable function, provided the subcommand INTERCEPT = INCLUDE is specified. In this case, the **K** matrix can be only a scalar matrix.
- If KMATRIX and LMATRIX are specified, then the number of rows in the requested **K** and **L** matrices must be equal. If there are multiple LMATRIX subcommands, then all requested **L** matrices must have the same number of rows, and **K** must have the same number of rows as these **L** matrices.
- A semicolon (;) can be used to indicate the end of a row in the **K** matrix.
- If more than one KMATRIX subcommand is specified, only the last one is in effect.

Example

```
UNIANOVA DEP BY A B
  /LMATRIX = “Effect A”
             A 1  0 -1; A 1 -1  0
  /LMATRIX = “Effect B”
             B 1  0 -1; B 1 -1  0
  /KMATRIX = 0; 0
  /DESIGN = A B.
```

In this example, assume that factors *A* and *B* each have three levels.

- There are two LMATRIX subcommands; both have two rows.
- The first LMATRIX subcommand tests whether the effect of *A* is 0, while the second LMATRIX subcommand tests whether the effect of *B* is 0.
- The KMATRIX subcommand specifies that the **K** matrix also has two rows, each with value 0.

CONTRAST Subcommand

CONTRAST specifies the type of contrast desired among the levels of a factor. For a factor with *k* levels or values, the contrast type determines the meaning of its *k*–1 degrees of freedom.

- Specify the factor name in parentheses following the subcommand CONTRAST.
- You can specify only one factor per CONTRAST subcommand, but you can enter multiple CONTRAST subcommands.
- After closing the parentheses, enter an equals sign followed by one of the contrast keywords.
- This subcommand creates an **L** matrix such that the columns corresponding to the factor match the contrast given. The other columns are adjusted so that the **L** matrix is estimable.

The following contrast types are available:

DEVIATION. *Deviations from the grand mean.* This is the default for factors. Each level of the factor except one is compared to the grand mean. One category (by default, the last) must be omitted so that the effects will be independent of one another. To omit a category other than the last, specify the number of the omitted category (which is not necessarily the same as its value) in parentheses after the keyword DEVIATION. For example:

```
UNIANOVA Y BY B
  /CONTRAST(B)=DEVIATION(1).
```

Suppose factor *B* has three levels, with values 2, 4, and 6. The specified contrast omits the first category, in which *B* has the value 2. Deviation contrasts are not orthogonal.

POLYNOMIAL. *Polynomial contrasts.* This is the default for within-subjects factors. The first degree of freedom contains the linear effect across the levels of the factor, the second contains the quadratic effect, and so on. In a balanced design, polynomial contrasts are orthogonal. By default, the levels are assumed to be equally spaced; you can specify unequal spacing by entering a metric consisting of one

integer for each level of the factor in parentheses after the keyword POLYNOMIAL. (All metrics specified cannot be equal; thus, (1, 1, . . . 1) is not valid.) For example:

```
UNIANOVA RESPONSE BY STIMULUS
  /CONTRAST(STIMULUS) = POLYNOMIAL(1,2,4)
```

Suppose that factor *STIMULUS* has three levels. The specified contrast indicates that the three levels of *STIMULUS* are actually in the proportion 1:2:4. The default metric is always (1, 2, . . . *k*), where *k* levels are involved. Only the relative differences between the terms of the metric matter; (1, 2, 4) is the same metric as (2, 3, 5) or (20, 30, 50) because, in each instance, the difference between the second and third numbers is twice the difference between the first and second.

DIFFERENCE. *Difference or reverse Helmert contrasts.* Each level of the factor except the first is compared to the mean of the previous levels. In a balanced design, difference contrasts are orthogonal.

HELMERT. *Helmert contrasts.* Each level of the factor except the last is compared to the mean of subsequent levels. In a balanced design, Helmert contrasts are orthogonal.

SIMPLE. *Each level of the factor except the last is compared to the last level.* To use a category other than the last as the omitted reference category, specify its number (which is not necessarily the same as its value) in parentheses following the keyword SIMPLE. For example:

```
UNIANOVA Y BY B
  /CONTRAST(B)=SIMPLE(1).
```

Suppose that factor *B* has three levels with values 2, 4, and 6. The specified contrast compares the other levels to the first level of *B*, in which *B* has the value 2. Simple contrasts are not orthogonal.

REPEATED. *Comparison of adjacent levels.* Each level of the factor except the first is compared to the previous level. Repeated contrasts are not orthogonal.

SPECIAL. *A user-defined contrast.* Values specified after this keyword are stored in a matrix in column major order.

Orthogonal contrasts are particularly useful. In a balanced design, contrasts are orthogonal if the sum of the coefficients in each contrast row is 0 and if, for any pair of contrast rows, the products of corresponding coefficients sum to 0. DIFFERENCE, HELMERT, and POLYNOMIAL contrasts always meet these criteria in balanced designs.

Example

```
UNIANOVA DEP BY FAC
  /CONTRAST(FAC)=DIFFERENCE
  /DESIGN.
```

- Suppose that the factor *FAC* has five categories and therefore four degrees of freedom.
- CONTRAST requests DIFFERENCE contrasts, which compare each level (except the first) with the mean of the previous levels.

POSTHOC Subcommand

POSTHOC allows you to produce multiple comparisons between means of a factor. These comparisons are usually not planned at the beginning of the study but are suggested by the data in the course of study.

- Post hoc tests are computed for the dependent variable. The alpha value used in the tests can be specified by using the keyword ALPHA on the CRITERIA subcommand. The default alpha value is 0.05. The confidence level for any confidence interval constructed is (1−α) × 100. The default confidence level is 95.
- Only factors appearing in the factor list are valid in this subcommand. Individual factors can be specified.
- You can specify one or more effects to be tested. Only fixed main effects appearing or implied on the DESIGN subcommand are valid test effects.

- Optionally, you can specify an effect defining the error term following the keyword VS after the test specification. The error effect can be any single effect in the design that is not the intercept or a main effect named on a POSTHOC subcommand.
- A variety of multiple comparison tests are available. Some tests are designed for detecting homogeneity subsets among the groups of means, some are designed for pairwise comparisons among all means, and some can be used for both purposes.
- For tests that are used for detecting homogeneity subsets of means, non-empty group means are sorted in ascending order. Means that are not significantly different are included together to form a homogeneity subset. The significance for each homogeneity subset of means is displayed. In a case where the numbers of valid cases are not equal in all groups, for most post hoc tests, the harmonic mean of the group sizes is used as the sample size in the calculation. For QREGW or FREGW, individual sample sizes are used.
- For tests that are used for pairwise comparisons, the display includes the difference between each pair of compared means, the confidence interval for the difference, and the significance. The sample sizes of the two groups being compared are used in the calculation.
- Output for tests specified on the POSTHOC subcommand are available according to their statistical purposes. The following table illustrates the statistical purpose of the post hoc tests:

Table 117. Post hoc comparisons

Keyword	Homogeneity Subsets Detection	Pairwise Comparison and Confidence Interval
LSD		Yes
SIDAK		Yes
BONFERRONI		Yes
GH		Yes
T2		Yes
T3		Yes
C		Yes
DUNNETT		Yes*
DUNNETTL		Yes*
DUNNETTR		Yes*
SNK	Yes	
BTUKEY	Yes	
DUNCAN	Yes	
QREGW	Yes	
FREGW	Yes	
WALLER	Yes†	
TUKEY	Yes	Yes
SCHEFFE	Yes	Yes
GT2	Yes	Yes
GABRIEL	Yes	Yes

* Only CIs for differences between test group means and control group means are given.

† No significance for Waller test is given.

- Tests that are designed for homogeneity subset detection display the detected homogeneity subsets and their corresponding significances.
- Tests that are designed for both homogeneity subset detection and pairwise comparison display both kinds of output.
- For the DUNNETT, DUNNETTL, and DUNNETTR keywords, only individual factors can be specified.
- The default reference category for DUNNETT, DUNNETTL, and DUNNETTR is the last category. An integer greater than 0 within parentheses can be used to specify a different reference category. For example, POSTHOC = A (DUNNETT(2)) requests a DUNNETT test for factor *A*, using the second level of *A* as the reference category.
- The keywords DUNCAN, DUNNETT, DUNNETTL, and DUNNETTR must be spelled out in full; using the first three characters alone is not sufficient.
- If the REGWT subcommand is specified, weighted means are used in performing post hoc tests.
- Multiple POSTHOC subcommands are allowed. Each specification is executed independently so that you can test different effects against different error terms.

SNK. *Student-Newman-Keuls procedure based on the Studentized range test.*

TUKEY. *Tukey's honestly significant difference.* This test uses the Studentized range statistic to make all pairwise comparisons between groups.

BTUKEY. *Tukey's* b. Multiple comparison procedure based on the average of Studentized range tests.

DUNCAN. *Duncan's multiple comparison procedure based on the Studentized range test*.

SCHEFFE. *Scheffé's multiple comparison* t *test.*

DUNNETT(refcat). *Dunnett's two-tailed* t *test*. Each level of the factor is compared to a reference category. A reference category can be specified in parentheses. The default reference category is the last category. This keyword must be spelled out in full.

DUNNETTL(refcat). *Dunnett's one-tailed* t *test*. This test indicates whether the mean at any level (except the reference category) of the factor is *smaller* than that of the reference category. A reference category can be specified in parentheses. The default reference category is the last category. This keyword must be spelled out in full.

DUNNETTR(refcat). *Dunnett's one-tailed* t *test*. This test indicates whether the mean at any level (except the reference category) of the factor is *larger* than that of the reference category. A reference category can be specified in parentheses. The default reference category is the last category. This keyword must be spelled out in full.

BONFERRONI. *Bonferroni* t *test*. This test is based on Student's *t* statistic and adjusts the observed significance level for the fact that multiple comparisons are made.

LSD. *Least significant difference* t *test*. Equivalent to multiple *t* tests between all pairs of groups. This test does not control the overall probability of rejecting the hypotheses that some pairs of means are different, while in fact they are equal.

SIDAK. *Sidak* t *test*. This test provides tighter bounds than the Bonferroni test.

GT2. *Hochberg's GT2.* Pairwise comparisons test based on the Studentized maximum modulus test. Unless the cell sizes are extremely unbalanced, this test is fairly robust even for unequal variances.

GABRIEL. *Gabriel's pairwise comparisons test based on the Studentized maximum modulus test.*

FREGW. *Ryan-Einot-Gabriel-Welsch's multiple stepdown procedure based on an* F *test.*

QREGW. *Ryan-Einot-Gabriel-Welsch's multiple stepdown procedure based on the Studentized range test.*

T2. *Tamhane's T2.* Tamhane's pairwise comparisons test based on a *t* test. This test can be applied in situations where the variances are unequal. This test is invalid when there are multiple factors in the model, and the keyword is ignored with a warning in such cases.

T3. *Dunnett's T3.* Pairwise comparisons test based on the Studentized maximum modulus. This test is appropriate when the variances are unequal. This test is invalid when there are multiple factors in the model, and the keyword is ignored with a warning in such cases.

GH. *Games and Howell's pairwise comparisons test based on the Studentized range test.* This test can be applied in situations where the variances are unequal. This test is invalid when there are multiple factors in the model, and the keyword is ignored with a warning in such cases.

C. *Dunnett's C.* Pairwise comparisons based on the weighted average of Studentized ranges. This test can be applied in situations where the variances are unequal. This test is invalid when there are multiple factors in the model, and the keyword is ignored with a warning in such cases.

WALLER(kratio). *Waller-Duncan* t *test.* This test uses a Bayesian approach. It is restricted to cases with equal sample sizes. For cases with unequal sample sizes, the harmonic mean of the sample size is used. The *k*-ratio is the Type 1/Type 2 error seriousness ratio. The default value is 100. You can specify an integer greater than 1 within parentheses.

EMMEANS Subcommand

EMMEANS displays estimated marginal means of the dependent variable in the cells (with covariates held at their overall mean value) and their standard errors for the specified factors. Note that these are predicted, not observed, means. The estimated marginal means are calculated using a modified definition by Searle, Speed, and Milliken (1980).

- TABLES, followed by an option in parentheses, is required. COMPARE is optional; if specified, it must follow TABLES.
- Multiple EMMEANS subcommands are allowed. Each is treated independently.
- If identical EMMEANS subcommands are specified, only the last identical subcommand is in effect. EMMEANS subcommands that are redundant but not identical (for example, crossed factor combinations such as *A*B* and *B*A*) are all processed.

TABLES(option). *Table specification.* Valid options are the keyword OVERALL, factors appearing on the factor list, and crossed factors constructed of factors on the factor list. Crossed factors can be specified using an asterisk (*) or the keyword BY. All factors in a crossed factor specification must be unique.

- If OVERALL is specified, the estimated marginal means of the dependent variable are displayed, collapsing over factors.
- If a factor, or a crossing of factors, is specified on the TABLES keyword, UNIANOVA collapses over any other factors before computing the estimated marginal means for the dependent variable.
- The TABLES keyword may be followed by the WITH keyword and a parenthesized list of covariates and values. Tables are produced with the values of the covariates fixed at the specified values. The keyword MEAN causes the covariate to be fixed at its observed mean value.

COMPARE(factor) ADJ(method). *Main- or simple-main-effects omnibus tests and pairwise comparisons of the dependent variable.* This option gives the mean difference, standard error, significance, and confidence interval for each pair of levels for the effect specified in the TABLES command, as well as an omnibus test for that effect. If only one factor is specified on TABLES, COMPARE can be specified by itself; otherwise, the factor specification is required. In this case, levels of the specified factor are compared with each other for each level of the other factors in the interaction.

- The optional ADJ keyword allows you to apply an adjustment to the confidence intervals and significance values to account for multiple comparisons. Methods available are LSD (no adjustment), BONFERRONI, or SIDAK.
- If OVERALL is specified on TABLES, COMPARE is invalid.

Example

```
UNIANOVA DEP BY A B
  /EMMEANS = TABLES(A*B) WITH(X=1.0) COMPARE(A) ADJ(LSD)
  /DESIGN.
```

- The output of this analysis includes a pairwise comparisons table for the dependent variable *DEP*.
- Assume that *A* has three levels and *B* has two levels. The first level of *A* is compared with the second and third levels, the second level with the first and third levels, and the third level with the first and second levels, with the value of the covariate *X* fixed at 1.0. The pairwise comparison is repeated for the two levels of *B.*

ROBUST Subcommand

ROBUST displays a table of parameter estimates, along with robust or heteroskedasticity-consistent (HC) standard errors; and *t* statistics, significance values, and confidence intervals that use the robust standard errors.

- ROBUST enables specification of the HCCOVB keyword on the OUTFILE subcommand, saving the robust covariance matrix estimates to a new file or dataset.
- ROBUST displays a second set of output for any custom hypothesis tests that are specified on LMATRIX subcommands, with results based on the specified robust covariance matrix estimator.

Five different methods are available for the robust covariance matrix estimation. You can specify only one method. If more than one ROBUST subcommand is specified, then the last subcommand is in effect.

HC0
Based on the original asymptotic or large sample robust, empirical, or "sandwich" estimator of the covariance matrix of the parameter estimates. The middle part of the sandwich contains squared OLS (ordinary least squares) or squared weighted WLS (weighted least squares) residuals.

HC1
A finite-sample modification of HC0, multiplying it by N/(N-p), where N is the sample size and p is the number of non-redundant parameters in the model.

HC2
A modification of HC0 that involves dividing the squared residual by 1-h, where h is the leverage for the case.

HC3
A modification of HC0 that approximates a jackknife estimator. Squared residuals are divided by the square of 1-h. This method is the default if ROBUST is specified without specifying a method.

HC4
A modification of HC0 that divides the squared residuals by 1-h to a power that varies according to h, N, and p, with an upper limit of 4.

SAVE Subcommand

Use SAVE to add one or more residual or fit values to the active dataset.

- Specify one or more temporary variables, each followed by an optional new name in parentheses.
- WPRED and WRESID can be saved only if REGWGT has been specified.
- Specifying a temporary variable on this subcommand results in a variable being added to the active data file for each dependent variable.
- You can specify variable names for the temporary variables. These names must be unique, valid variable names.
- If new names are not specified, UNIANOVA generates a rootname using a shortened form of the temporary variable name with a suffix.
- If more than one SAVE subcommand is specified, only the last one is in effect.

PRED. *Unstandardized predicted values.*

WPRED. *Weighted unstandardized predicted values.* Available only if REGWGT has been specified.

RESID. *Unstandardized residuals.*

WRESID. *Weighted unstandardized residuals.* Available only if REGWGT has been specified.

DRESID. *Deleted residuals.*

ZRESID. *Standardized residuals.*

SRESID. *Studentized residuals.*

SEPRED. *Standard errors of predicted value.*

COOK. *Cook's distances.*

LEVER. *Uncentered leverage values.*

OUTFILE Subcommand

The OUTFILE subcommand writes IBM SPSS Statistics data files that can be used in other procedures.

- You must specify a keyword on OUTFILE. There is no default.
- You must specify a quoted file specification or a previously declared dataset name (DATASET DECLARE command), enclosed in quotes. The asterisk (*) is not allowed.
- If you specify more than one keyword, a different file specification is required for each.
- If more than one OUTFILE subcommand is specified, only the last one is in effect.
- For COVB or CORB, the output will contain, in addition to the covariance or correlation matrix, four rows for each dependent variable: a row of parameter estimates, a row of standard errors of the parameter estimates, a row of residual degrees of freedom, and a row of significance values for the *t* statistics corresponding to the parameter estimates. All statistics are displayed separately by split. For HCCOVB, the covariance matrix is calculated with a robust estimator, the row of standard errors displays the robust standard errors, and the significance values reflect the robust errors.

COVB ('savfile'|'dataset'). *Writes the parameter covariance matrix.*

CORB ('savfile'|'dataset'). *Writes the parameter correlation matrix.*

HCCOVB ('savfile'|'dataset'). *Writes the parameter covariance matrix, calculated with a robust estimator.* HCCOVB is valid only if the ROBUST subcommand is specified.

EFFECT ('savfile'|'dataset'). *Writes the statistics from the between-subjects ANOVA table.*

DESIGN ('savfile'|'dataset'). *Writes the design matrix.* The number of rows equals the number of cases, and the number of columns equals the number of parameters. The variable names are *DES_1*, *DES_2*, *..., DES_p*, where *p* is the number of the parameters.

MBPDESIGN, BPDESIGN, and FDESIGN Subcommands

MBPDESIGN, BPDESIGN, and FDESIGN specify the auxiliary regression model that is used to test for heteroskedasticity. MBPDESIGN specifies the model for the Modified Breusch-Pagan test. BPDESIGN specifies the model for the Breusch-Pagan test. FDESIGN specifies the model for the F test.

Note: MBPDESIGN, BPDESIGN, and FDESIGN are necessary only when you don't want to use the default model for the associated test. The default model consists of a constant term, a term that is linear in the predicted values, a term that is quadratic in the predicted values, and an error term. The default model is requested with the MBP, BP, or F keyword on the PRINT subcommand.

- Specify a list of terms to be included in the model, and separate the terms by spaces or commas. The model that is used to test for heteroskedasticity always includes an intercept term, so specifying INTERCEPT has no effect.

- If MBPDESIGN, BPDESIGN, or FDESIGN is specified by itself (without any explicit terms), then the model that is specified by the DESIGN subcommand is used. An intercept term is included if the specified model does not contain one.
- To include a term for the main effect of a factor, enter the name of the factor.
- To include a term for an interaction between factors, use the keyword BY or the asterisk (*) to join the factors that are involved in the interaction. For example, *A*B* means a two-way interaction effect of *A* and *B*, where *A* and *B* are factors. *A*A* is not allowed because factors inside an interaction effect must be distinct.
- To include a term for nesting one effect within another effect, use the keyword WITHIN or use a pair of parentheses. For example, *A(B)* means that *A* is nested within *B*. The expression *A(B)* is equivalent to the expression *A WITHIN B*. When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, *A(B)(C)* is not valid.
- Multiple nesting is allowed. For example, *A(B(C))* means that *B* is nested within *C* and *A* is nested within *B(C)*.
- Interactions between nested effects are not valid. For example, neither *A(C)*B(C)* nor *A(C)*B(D)* is valid.
- To include a covariate term in the design, enter the name of the covariate.
- Covariates can be connected—but not nested—through the * operator to form another covariate effect. Therefore, interactions among covariates such as *X1*X1* and *X1*X2* are valid but not *X1(X2)*.
- Factor and covariate effects can be connected only by the * operator. Suppose *A* and *B* are factors and *X1* and *X2* are covariates. Examples of valid factor-by-covariate interaction effects are *A*X1*, *A*B*X1*, *X1*A(B)*, *A*X1*X1*, and *B*X1*X2*.
- If more than one MBPDESIGN, BPDESIGN, or FDESIGN subcommand is specified, only the last subcommand is in effect.

DESIGN Subcommand

DESIGN specifies the effects included in a specific model. The cells in a design are defined by all of the possible combinations of levels of the factors in that design. The number of cells equals the product of the number of levels of all the factors. A design is *balanced* if each cell contains the same number of cases. UNIANOVA can analyze both balanced and unbalanced designs.

- Specify a list of terms to be included in the model, separated by spaces or commas.
- The default design, if the DESIGN subcommand is omitted or is specified by itself, is a design consisting of the following terms in order: the intercept term (if INTERCEPT=INCLUDE is specified), next the covariates given in the covariate list, and then the full factorial model defined by all factors on the factor list and excluding the intercept.
- To include a term for the main effect of a factor, enter the name of the factor on the DESIGN subcommand.
- To include the intercept term in the design, use the keyword INTERCEPT on the DESIGN subcommand. If INTERCEPT is specified on the DESIGN subcommand, the subcommand INTERCEPT=EXCLUDE is overridden.
- To include a term for an interaction between factors, use the keyword BY or the asterisk (*) to join the factors involved in the interaction. For example, *A*B* means a two-way interaction effect of *A* and *B*, where *A* and *B* are factors. *A*A* is not allowed because factors inside an interaction effect must be distinct.
- To include a term for nesting one effect within another, use the keyword WITHIN or a pair of parentheses on the DESIGN subcommand. For example, *A(B)* means that *A* is nested within *B*. The expression *A(B)* is equivalent to the expression *A WITHIN B*. When more than one pair of parentheses is present, each pair of parentheses must be enclosed or nested within another pair of parentheses. Thus, *A(B)(C)* is not valid.
- Multiple nesting is allowed. For example, *A(B(C))* means that *B* is nested within *C*, and *A* is nested within *B(C)*.

- Interactions between nested effects are not valid. For example, neither *A(C)*B(C)* nor *A(C)*B(D)* is valid.
- To include a covariate term in the design, enter the name of the covariate on the DESIGN subcommand.
- Covariates can be connected, but not nested, through the * operator to form another covariate effect. Therefore, interactions among covariates such as *X1*X1* and *X1*X2* are valid, but not *X1(X2)*. Using covariate effects such as *X1*X1*, *X1*X1*X1*, *X1*X2*, and *X1*X1*X2*X2* makes fitting a polynomial regression model easy in UNIANOVA.
- Factor and covariate effects can be connected only by the * operator. Suppose *A* and *B* are factors, and *X1* and *X2* are covariates. Examples of valid factor-by-covariate interaction effects are *A*X1*, *A*B*X1*, *X1*A(B)*, *A*X1*X1*, and *B*X1*X2*.
- If more than one DESIGN subcommand is specified, only the last one is in effect.

Example

```
UNIANOVA Y BY A B C WITH X
  /DESIGN A B(A) X*A.
```

- In this example, the design consists of a main effect *A*, a nested effect *B* within *A*, and an interaction effect of a covariate *X* with a factor *A*.

UPDATE

```
UPDATE FILE={primary file} [PASSWORD='password']
            {*           }

 [/RENAME=(old varnames=new varnames)...]

 [/IN=varname]

  /FILE={transaction file1} [PASSWORD='password']
        {*                 }

 [/FILE=transaction file2 [PASSWORD='password'] ]

  /BY key variables

 [/MAP]

 [/KEEP={ALL**  }] [/DROP=varlist]
        {varlist}
```

**Default if the subcommand is omitted.

Release History

Release 22.0

- PASSWORD keyword introduced on the FILE subcommand.

Example

```
UPDATE FILE='/data/maillist.sav'
  /FILE='/data/newlist.sav'
  /BY=ID.
```

Overview

UPDATE replaces values in a primary file with updated values recorded in one or more files called transaction files. Cases in the primary file and transaction file are matched according to a key variable.

The primary file and the transaction files must be IBM SPSS Statistics data files or datasets available in the current session, including the active dataset. UPDATE replaces values and creates a new active dataset, which replaces the original active dataset.

UPDATE is designed to update values of existing variables for existing cases. Use MATCH FILES to add new variables to a data file and ADD FILES to add new cases.

Options

Variable Selection. You can specify which variables from each input file are included in the new active dataset using the DROP and KEEP subcommands.

Variable Names. You can rename variables in each input file before combining the files using the RENAME subcommand. This permits you to combine variables that are the same but whose names differ in different input files, or to separate variables that are different but have the same name.

Variable Flag. You can create a variable that indicates whether a case came from a particular input file using IN. You can use the FIRST or LAST subcommand to create a variable that flags the first or last case of a group of cases with the same value for the key variable.

Variable Map. You can request a map showing all variables in the new active dataset, their order, and the input files from which they came using the MAP subcommand.

Basic Specification

The basic specification is two or more FILE subcommands and a BY subcommand.

- The first FILE subcommand must specify the primary file. All other FILE subcommands identify the transaction files.

- BY specifies the key variables.
- All files must be sorted in ascending order by the key variables.
- By default, all variables from all input files are included in the new active dataset.

Subcommand Order

- The primary file must be specified first.
- RENAME and IN must immediately follow the FILE subcommand to which they apply.
- BY must follow the FILE subcommands and any associated RENAME and IN subcommands.
- MAP, DROP, and KEEP must be specified after all FILE and RENAME subcommands.

Syntax Rules

- BY can be specified only once. However, multiple variables can be specified on BY. All files must be sorted in ascending order by the key variables named on BY.
- The primary file cannot contain duplicate values for the key variables.
- RENAME can be repeated after each FILE subcommand and applies only to variables in the file named on the immediately preceding FILE subcommand.
- MAP can be repeated as often as needed.

Operations

- UPDATE reads all input files named on FILE and builds a new active dataset. The new active dataset is built when the data are read by one of the procedure commands or the EXECUTE, SAVE, or SORT CASES command.
  - If the current active dataset is included and is specified with an asterisk (FILE=*), the new merged dataset replaces the active dataset. If that dataset is a named dataset, the merged dataset retains that name. If the current active dataset is not included or is specified by name (for example, FILE=Dataset1), a new unnamed, merged dataset is created, and it becomes the active dataset. For information on naming datasets, see "DATASET NAME" on page 543.
- The new active dataset contains complete dictionary information from the input files, including variable names, labels, print and write formats, and missing-value indicators. The new active dataset also contains the documents from each input file, unless the DROP DOCUMENTS command is used.
- UPDATE copies all variables in order from the primary file, then all variables in order from the first transaction file, then all variables in order from the second transaction file, and so on.
- Cases are updated when they are matched on the BY variable(s). If the primary and transaction files contain common variables for matched cases, the values for those variables are taken from the transaction file, provided that the values are not missing or blanks. Missing or blank values in the transaction files are not used to update values in the primary file.
- When UPDATE encounters duplicate keys within a transaction file, it applies each transaction sequentially to that case to produce one case per key value in the resulting file. If more than one transaction file is specified, the value for a variable comes from the last transaction file with a nonmissing value for that variable.
- Variables that are in the transaction files but not in the primary file are added to the primary file. Cases that do not contain those variables are assigned the system-missing value (for numerics) or blanks (for strings).
- Cases that are in the transaction files but not in the primary file are added to the primary file and are interleaved according to their values for the key variables.
- If the active dataset is named as an input file, any N and SAMPLE commands that have been specified are applied to the active dataset before files are combined.
- The TEMPORARY command cannot be in effect if the active dataset is used as an input file.

Limitations

- A maximum of one BY subcommand. However, BY can specify multiple variables.

Examples

Basic Example

```
UPDATE FILE='/data/mailist.sav'
  /FILE='/data/newlist.sav'
  /BY=ID.
```

- *mailist.sav* is specified as the primary file. *newlist.sav* is the transaction file. *ID* is the key variable.
- Both *mailist.sav* and *newlist.sav* must be sorted in ascending order of *ID*.
- If *newlist.sav* has cases or nonmissing variables that are not in *mailist.sav*, the new cases or variables are added to the resulting file.

Using Multiple Key Variables

```
SORT CASES BY LOCATN DEPT.
UPDATE  FILE='/data/primary.sav' /FILE=* /BY LOCATN DEPT
  /KEEP AVGHOUR AVGRAISE LOCATN DEPT SEX HOURLY RAISE /MAP.
SAVE OUTFILE='/data/personnel.sav'.
```

- SORT CASES sorts the active dataset in ascending order of the variables to be named as key variables on UPDATE.
- UPDATE specifies *primary.sav* as the primary file and the sorted active dataset as the transaction file. The file *primary.sav* must also be sorted by *LOCATN* and *DEPT*.
- BY specifies the key variables *LOCATN* and *DEPT*.
- KEEP specifies the subset and order of variables to be retained in the resulting file.
- MAP provides a list of the variables in the resulting file and the two input files.
- SAVE saves the resulting file as a data file in IBM SPSS Statistics format.

FILE Subcommand

FILE identifies each input file. At least two FILE subcommands are required on UPDATE: one specifies the primary file and the other a transaction file. A separate FILE subcommand must be used to specify each transaction file.

- The first FILE subcommand must specify the primary file.
- An asterisk on FILE refers to the active dataset.
- The files must be IBM SPSS Statistics data files or datasets available in the current session. File specifications should be enclosed in quotes.
- All files must be sorted in ascending order according to the variables specified on BY.
- The primary file cannot contain duplicate values for the key variables. However, transaction files can and often do contain cases with duplicate keys (see "Operations") .

PASSWORD Keyword

The PASSWORD keyword specifies the password required to open an encrypted IBM SPSS Statistics data file. The specified value must be enclosed in quotation marks and can be provided as encrypted or as plain text. Encrypted passwords are created when pasting command syntax from the Save Data As dialog. The PASSWORD keyword is ignored if the file is not encrypted.

Text Data Files

You can update data from text data files -- or from any data format that IBM SPSS Statistics can read -- using defined datasets.

Example

```
DATA LIST FILE='/data/update.txt'
  ID 1-3 NAME 5-17 (A) ADDRESS 19-28 (A) ZIP 30-34.
```

```
SORT CASES BY ID.
DATASET NAME updatefile.
GET FILE='/maillist.sav'.
SORT CASES BY ID.
DATASET NAME=primaryfile.
UPDATE FILE='primaryfile'
 /RENAME=(STREET=ADDRESS)
 /FILE='updatefile'
 /BY=ID /MAP.
```

BY Subcommand

BY specifies one or more identification, or key, variables that are used to match cases between files.

- BY must follow the FILE subcommands and any associated RENAME and IN subcommands.
- BY specifies the names of one or more key variables. The key variables must exist in all input files and have the same names in all of the files. The key variables can be string variables (long strings are allowed).
- All input files must be sorted in ascending order of the key variables. If necessary, use SORT CASES before UPDATE.
- Missing values for key variables are handled like any other values.
- The key variables in the primary file must identify unique cases. If duplicate cases are found, the program issues an error and UPDATE is not executed. The system-missing value is treated as one single value.

RENAME Subcommand

RENAME renames variables on the input files *before* they are processed by UPDATE. RENAME must follow the FILE subcommand that contains the variables to be renamed.

- RENAME applies only to the immediately preceding FILE subcommand. To rename variables from more than one input file, specify a RENAME subcommand after each FILE subcommand.
- Specifications for RENAME consist of a left parenthesis, a list of old variable names, an equals sign, a list of new variable names, and a right parenthesis. The two variable lists must name or imply the same number of variables. If only one variable is renamed, the parentheses are optional.
- More than one rename specification can be specified on a single RENAME subcommand, each enclosed in parentheses.
- The TO keyword can be used to refer to consecutive variables in the file and to generate new variable names.
- RENAME takes effect immediately. Any KEEP and DROP subcommands entered prior to a RENAME must use the old names, while KEEP and DROP subcommands entered after a RENAME must use the new names.
- All specifications within a single set of parentheses take effect simultaneously. For example, the specification RENAME (A,B = B,A) swaps the names of the two variables.
- Variables cannot be renamed to scratch variables.

Example

```
UPDATE FILE='/data/primary.sav' /FILE=CLIENTS
  /RENAME=(TEL_NO, ID_NO = PHONE, ID)
  /BY ID.
```

- UPDATE updates the primary phone list by using current information from the file *CLIENTS*.
- Two variables on *CLIENTS* are renamed prior to the match. *TEL_NO* is renamed *PHONE* to match the name used for phone numbers in the primary file. *ID_NO* is renamed *ID* so that it will have the same name as the identification variable in the primary file and can be used on the BY subcommand.
- The old variable names are listed before the equals sign, and the new variable names are listed in the same order after the equals sign. The parentheses are required.

- The BY subcommand matches cases according to client ID numbers.

DROP and KEEP Subcommands

DROP and KEEP are used to include a subset of variables in the resulting file. DROP specifies a set of variables to exclude, and KEEP specifies a set of variables to retain.

- DROP and KEEP do not affect the input files on disk.
- DROP and KEEP must follow all FILE and RENAME subcommands.
- DROP and KEEP must specify one or more variables. If RENAME is used to rename variables, specify the new names on DROP and KEEP.
- DROP cannot be used with variables created by the IN subcommand.
- The keyword ALL can be specified on KEEP. ALL must be the last specification on KEEP, and it refers to all variables not previously named on KEEP.
- KEEP can be used to change the order of variables in the resulting file. With KEEP, variables are kept in the order in which they are listed on the subcommand. If a variable is named more than once on KEEP, only the first mention of the variable is in effect; all subsequent references to that variable name are ignored.
- Multiple DROP and KEEP subcommands are allowed. Specifying a variable that is not in the active dataset or that has been dropped because of a previous DROP or KEEP subcommand results in an error and the UPDATE command is not executed.

Example

```
UPDATE FILE='/data/mailist.sav' /FILE='/data/newlist.sav' /RENAME=(STREET=ADDRESS)
  /BY ID /KEEP=NAME ADDRESS CITY STATE ZIP ID.
```

- KEEP specifies the variables to keep in the result file. The variables are stored in the order specified on KEEP.

IN Subcommand

IN creates a new variable in the resulting file that indicates whether a case came from the input file named on the preceding FILE subcommand. IN applies only to the file specified on the immediately preceding FILE subcommand.

- IN has only one specification, the name of the flag variable.
- The variable created by IN has the value 1 for every case that came from the associated input file and the value 0 if the case came from a different input file.
- Variables created by IN are automatically attached to the end of the resulting file and cannot be dropped.

Example

```
UPDATE  FILE=WEEK10 /FILE=WEEK11 /IN=INWEEK11 /BY=EMPID.
```

- IN creates the variable *INWEEK11*, which has the value 1 for all cases in the resulting file that came from the input file *WEEK11* and the value 0 for those cases that were not in file *WEEK11*.

MAP Subcommand

MAP produces a list of the variables in the new active dataset and the file or files from which they came. Variables are listed in the order in which they appear in the resulting file. MAP has no specifications and must be placed after all FILE, RENAME, and IN subcommands.

- Multiple MAP subcommands can be used. Each MAP shows the current status of the active dataset and reflects only the subcommands that precede the MAP subcommand.
- To obtain a map of the resulting file in its final state, specify MAP last.
- If a variable is renamed, its original and new names are listed. Variables created by IN are not included in the map, since they are automatically attached to the end of the file and cannot be dropped.

USE

```
USE [{start date       }] [THRU [{end date       }]]
     {start case number}         {end case number}
     {FIRST            }         {LAST           }

    [ALL]

  /PERMANENT
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
USE Y 1960.
```

Overview

USE designates a range of observations to be used with time series procedures. Cases outside the defined range are filtered but not deleted from the active dataset unless the /PERMANENT subcommand is specified.

Basic Specification

The basic specification is either the start of the range, the end of the range, or both. You can also simply specify the keyword THRU or ALL.

- The default start is the first observation in the file, and the default end is the last observation.
- The keyword THRU is required if the end of the range is specified.
- The keyword ALL defines a USE range starting with the first observation and ending with the last observation in the series. It can be specified to restore processing to the entire series.
- The keyword THRU by itself is the same as specifying the keyword ALL.

Operations

- USE is ignored by the utility procedures CREATE and RMV. These procedures process all of the available data.
- The DATE command turns off all existing USE and PREDICT specifications.
- FILTER and USE are mutually exclusive. USE automatically turns off any previous FILTER command, and FILTER automatically turns off any previous USE command.
- The SORT command automatically turns off any previous USE command specification.
- USE remains in effect in a session until it is changed by another USE command or until a new DATE, FILTER, or SORT command is issued.

Limitations

- A maximum of one range (one start and/or one end) can be specified.

Syntax Rules

- The start and end can be specified as either DATE specifications or case (observation) numbers.
- DATE specifications and case numbers cannot be mixed on a USE command.
- Any observation within the file can be used as the start or end as long as the starting observation comes before the end observation.

DATE Specifications

- A DATE specification consists of DATE keywords and values (see DATE). These specifications must correspond to existing DATE variables.
- If more than one DATE variable exists, the highest-order one must be used in the specification.
- Values on the keyword YEAR must have the same format (two or four digits) as the YEAR specifications on the DATE command.

Case Specifications

- The case number specification is the sequence number of the case (observation) as it is read by the program.

Keywords FIRST and LAST

- The start can also be specified with the keyword FIRST, and the end, with the keyword LAST. These keywords designate the first and last cases in the file, respectively.
- The keywords FIRST and LAST can be used along with either DATE or case specifications.

PERMANENT Subcommand

The PERMANENT subcommand deletes cases outside the defined range from the active dataset.

Examples

```
USE ALL.
```

- This command includes all observations in the file in the USE range.
- This specification is the same as USE THRU or USE FIRST THRU LAST.

```
USE Y 1960.
```

- This command selects observations starting with a *YEAR_* value of 1960 through the last observation in the file. It is equivalent to USE Y 1960 THRU LAST.

```
USE THRU D 5.
```

- This command selects all cases from the first case in the file to the last one with a *DAY_* value of 5. It is equivalent to USE FIRST THRU D 5.

```
USE THRU 5.
```

- This command selects cases starting with the first case and ending with the fifth case.

```
USE Y 1955 M 6 THRU Y 1960 M 6.
```

- This selects cases from June, 1955, through June, 1960.

```
USE W 16 D 3 THRU W 48 D 3.
```

- This example selects cases from day 3 of week 16 through day 3 of week 48.

```
USE CYCLE 2 OBS 4 THRU CYCLE 2 OBS 17.
```

- This example selects observations 4 through 17 of the second cycle.

VALIDATEDATA

VALIDATEDATA is available in Statistics Base Edition.

```
VALIDATEDATA       [VARIABLES=varlist]
                   [ID=varlist]
                   [CROSSVARRULES=rulelist]
 /VARCHECKS        [STATUS={ON**}]
                           {OFF }
                   [PCTMISSING={70**   }]
                               {number }
                   [PCTEQUAL={95**  }]
                             {number}
                   [PCTUNEQUAL={90**  }]
                               {number}
                   [CV={0.001**}]
                       {number }
                   [STDDEV={0**   }]]
                           {number}
 /IDCHECKS         [INCOMPLETE**] [DUPLICATE**] [NONE]]
 /CASECHECKS       [REPORTEMPTY={YES**}] [SCOPE={ANALYSISVARS}]]
                              {NO   }         {ALLVARS     }
 /RULESUMMARIES    [BYVARIABLE**] [BYRULE] [NONE]]
 /CASEREPORT       DISPLAY={YES**}
                           {NO   }
                   [MINVIOLATIONS={1**    }]
                                  {integer}
                   [CASELIMIT={FIRSTN({500**  })}]]
                                     {integer}
                             {NONE            }
 /SAVE             [EMPTYCASE[(varname)]]
                   [INCOMPLETEID[(varname)]]
                   [DUPLICATEID[(varname)]]
                   [RULEVIOLATIONS[(varname)]]]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 14.0

- Command introduced.

Example

```
VALIDATEDATA VARIABLES=ALL.
```

Overview

The VALIDATEDATA procedure identifies suspicious and invalid cases, variables, and data values in the active dataset.

The procedure can summarize two types of validation rules. Single-variable rules consist of a fixed set of checks that are applied to individual data values, such as range checks. Cross-variable rules are user-specified rules that typically examine combinations of data values for two or more variables.

Options

Analysis Variables. The procedure can identify variables that have a high proportion of missing values as well as variables that are constant (or nearly so). You can set the maximum acceptable percentage of missing values as well as thresholds for considering categorical and scale variables as constants.

Case Identifiers. The procedure can identify incomplete and duplicate case identifiers. A single variable or a combination of variables can be treated as the identifier for a case.

Cases. The procedure can identify empty cases. A case can be regarded as empty if all analysis variables are blank or missing or all non-ID variables are blank or missing.

Single-Variable Rules. Single-variable validation rules can be summarized by variable and rule. Single-variable rules are defined using the DATAFILE ATTRIBUTE command and are linked to analysis variables using the VARIABLE ATTRIBUTE command.

Cross-Variable Rules. The procedure can summarize violations of cross-variable validation rules. Cross-variable rules are defined using the DATAFILE ATTRIBUTE command.

Saved Variables. You can save variables that identify suspicious cases and values in the active dataset.

Basic Specification

- The minimum specification is a list of analysis variables, case identifier variables, or cross-variable rules.
- By default, if you specify analysis variables, the procedure reports all analysis variables that have a high proportion of missing values, as well as analysis variables that are constant or nearly so. If single-variable validation rules are defined for analysis variables, the procedure reports rule violations for each variable.
- By default, if case identifier variables are specified, the procedure reports all incomplete and duplicate case identifiers.
- If you specify cross-variable rules, the procedure reports the total number of cases that violated each rule.
- If you specify single-variable or cross-variable rules, a casewise report is shown that lists the first 500 cases that violated at least one validation rule.
- Empty cases are identified by default.

Syntax Rules

- Each subcommand is global and can be used only once.
- Subcommands can be used in any order.
- An error occurs if a keyword or attribute is specified more than once within a subcommand.
- Equals signs and slashes shown in the syntax chart are required.
- Subcommand names and keywords must be spelled out in full.
- Empty subcommands generate a procedure error.

Operations

- The procedure honors SPLIT FILE specifications. Split variables are filtered out of all variable lists.
- The procedure treats user- and system-missing values as missing values.
- An error occurs if no procedure output is requested.

Note: Since measurement level can affect the results, if any variables (fields) have an unknown measurement level, an initial data pass will be performed to determine default measurement level for any variables with an unknown measurement level. For information on the criteria used to determine default measurement level, see SET SCALEMIN.

Limitations

- The weight variable, if specified, is ignored with a warning.

Validation Rules

- VALIDATEDATA ignores invalid rules and links with a warning.
- If an analysis variable is linked to multiple rules, VALIDATEDATA applies the rules independently; it does not check for conflicts or redundancies among the constituent rules.
- A rule outcome variable must be associated with each rule that indicates which cases violated the rule. Outcome variables are created outside the VALIDATEDATA procedure and should be coded such that 1 indicates an invalid value or combination of values and 0 indicates a valid value.
- An error occurs if the same outcome variable is referenced by two or more rules.
- Values of all outcome variables are assumed to be current with respect to the active dataset.

Examples

Default Analysis Variable Validation

```
VALIDATEDATA VARIABLES=ALL.
```

- The only specification is a list of analysis variables (all variables in the active dataset).
- The procedure reports all variables with a high proportion of missing values, as well as variables that are constant or nearly so.
- In addition, empty cases are identified.

Default Case Identifier Variable Validation

```
VALIDATEDATA ID=Firstname Lastname.
```

- Two case identifier variables are specified.
- By default, the procedure reports cases having incomplete or duplicate case identifiers. The combination of *Firstname* and *Lastname* values is treated as a case identifier.
- In addition, empty cases are identified.

Non-Default Analysis and Case Identifier Variable Validation

```
VALIDATEDATA VARIABLES=Satis1 Satis2 Satis3
             ID=Firstname Lastname
 /VARCHECKS STATUS=ON PCTMISSING=30
 /IDCHECKS DUPLICATE
 /SAVE EMPTYCASE.
```

- Three ordinal satisfaction variables (*Satis1*, *Satis2*, *Satis3*) are specified as analysis variables.
- ID specifies two case identifier variables, *Firstname* and *Lastname*.
- VARCHECKS reports analysis variables with more than 30% missing values. By default, the procedure also reports categorical analysis variables if more than 95% of their cases fall into a single category.
- IDCHECKS reports cases that have duplicate case identifiers and turns off the default check for incomplete case identifiers.
- Empty cases are identified by default.
- SAVE saves a variable to the active dataset that indicates which cases are empty.

Using Single-Variable Validation Rules

```
DATAFILE ATTRIBUTE
 ATTRIBUTE=$VD.SRule[1] ("Label='Likert 1 to 5',"+
                         "Description='Likert 5-point scale',"+
                         "Type='Numeric',"+
                         "Domain='Range',"+
                         "FlagUserMissing='No',"+
                         "FlagSystemMissing='No',"+
                         "FlagBlank='No',"+
                         "Minimum='1',"+
                         "Maximum='5',"+
                         "FlagNoninteger='Yes'").

COMPUTE Likert1to5_Satis_=NOT(Satis LE 5 AND Satis GE 1 AND
                              Satis=TRUNC(Satis)).
VARIABLE ATTRIBUTE
  VARIABLES= Likert1to5_Satis_
  ATTRIBUTE=$VD.RuleOutcomeVar("Yes").

VARIABLE ATTRIBUTE
  VARIABLES=Satis
  ATTRIBUTE=$VD.SRuleRef[1]("Rule='$VD.SRule[1]',"+
                            "OutcomeVar='Likert1to5_Satis_'").

VALIDATEDATA VARIABLES=Satis
 /CASEREPORT DISPLAY=YES CASELIMIT=NONE
 /SAVE RULEVIOLATIONS.
```

- DATAFILE ATTRIBUTE defines a single-variable validation rule named *$VD.SRule[1]*. The rule flags any value that is not an integer within the range 1 to 5.
- The first VARIABLE ATTRIBUTE command links the rule to the variable *Satis*. It also references an outcome variable where 1 indicates an invalid value for the variable *Satis*.

- COMPUTE creates the rule outcome variable.
- The second VARIABLE ATTRIBUTE command marks *Likert1to5_Satis_* as an outcome variable in the data dictionary.
- VALIDATEDATA specifies *Satis* as an analysis variable. In addition to performing default checks for constants, variables with a large proportion of missing values, and empty cases, the procedure summarizes violations of single-variable validation rules defined for *Satis*.
- CASEREPORT reports all cases having a rule violation.
- SAVE saves a variable to the active dataset that indicates the total number of validation rule violations per case.

Using Cross-Variable Validation Rules

```
DATAFILE ATTRIBUTE
 ATTRIBUTE=$VD.CRule[1]("Label='Pregnant Male',"+
                        "Expression='Sex ='''Male''' AND Pregnant ''Yes'''',"+
                        "OutcomeVar='PregnantMale_'").

COMPUTE PregnantMale_= Sex ='Male' AND Pregnant = 'Yes'.
VARIABLE ATTRIBUTE
  VARIABLES=PregnantMale_
  ATTRIBUTE=$VD.RuleOutcomeVar("Yes").

VALIDATEDATA CROSSVARRULES=$VD.CRule[1]
 /CASECHECKS REPORTEMPTY=NO.
```

- DATAFILE ATTRIBUTE defines the cross-variable rule *$VD.CRule[1]*.
- COMPUTE creates the outcome variable *PregnantMale_* referenced by the cross-variable rule. For *PregnantMale_* , values of 1 identify cases containing males coded as being pregnant.
- VARIABLE ATTRIBUTE marks *PregnantMale_* as an outcome variable in the data dictionary.
- VALIDATEDATA specifies *$VD.CRule[1]* as a cross-variable rule to be summarized. The procedure reports the total number of pregnant males in the active dataset.
- CASECHECKS turns off the default check for empty cases.
- By default, the first 500 cases that violated at least one validation rule are listed.

Variable Lists

You must specify VARIABLES, ID, or CROSSVARRULES.

VARIABLES Keyword

The VARIABLES list specifies analysis variables to be validated.

- Specify one or more analysis variables. ALL and TO can be used to refer to all variables or a range of variables, respectively.
- Repeated instances of the same variable are filtered out of the list.
- Rule outcome variables are filtered out of the list.

ID Keyword

The ID list specifies case ID variables to be validated. ID variables are also used to label casewise output.

- Specify one or more variables. If two or more ID variables are specified, the combination of their values is treated as a case identifier.
- Repeated instances of the same variable are filtered out of the list.
- Rule outcome variables are filtered out of the list.

CROSSVARRULES Keyword

The CROSSVARRULES keyword specifies cross-variable rules to be summarized.

- Specify one or more cross-variable rules of the form *$VD.CRule[n]*, where *n* is a positive integer. Alternatively, you can provide a list of integers, where 1 means *$VD.CRule[1]*, 2 means *$VD.CRule[2]*, etc.

- The list accepts the keyword ALL, which signifies all cross-variable rules recorded in the data dictionary.
- Repeated instances of the same rule are filtered out of the list.
- A warning occurs if a specified rule does not exist. The rule is ignored.

VARCHECKS Subcommand

The VARCHECKS subcommand specifies checks to be performed on analysis variables. The subcommand is ignored if no analysis variables are defined.

STATUS. *Perform variable checks.* By default, variable checks are performed. To turn off variable checks, specify STATUS=NO; any other VARCHECKS keywords are then ignored.

PCTMISSING. *Maximum percentage of missing values.* Reports analysis variables with a percentage of missing values greater than the specified value. The specified value must be a positive number less than or equal to 100. The default value is 70.

PCTEQUAL. *Maximum percentage of cases representing a single category.* Reports categorical analysis variables with a percentage of cases representing a single nonmissing category greater than the specified value. The specified value must be a positive number less than or equal to 100. The default value is 95. The percentage is based on cases with nonmissing values of the variable. PCTEQUAL is ignored if no categorical analysis variables are specified.

PCTUNEQUAL . *Percentage of categories containing only one case in a categorical variable.* If the percentage of an analysis variable's categories containing only one case is greater than the specified value, the variable is reported. The specified value must be a positive number less than or equal to 100. The default value is 90. PCTUNEQUAL is ignored if no categorical analysis variables are specified.

CV. *Minimum absolute coefficient of variation.* A variable's coefficient of variation is defined as its standard deviation divided by its mean. If the absolute value of the coefficient of variation is less than the specified value, the variable is reported. This setting applies only to continuous predictors and only if the mean is non-zero. The specified value must be a non-negative number. Specifying 0 turns off the coefficient of variation check. The default value is 0.001. CV is ignored if no scale analysis variables are specified.

STDDEV. *Minimum standard deviation.* Reports variables whose standard deviation is less than the specified value. This setting applies only to scale analysis variables. The specified value must be a non-negative number. The default value is 0, which, in effect, turns off the standard deviation check. STDDEV is ignored if no scale analysis variables are specified.

IDCHECKS Subcommand

The IDCHECKS subcommand specifies checks to be performed on case identifiers. The subcommand is ignored if no ID variables are specified.

INCOMPLETE. *Incomplete case identifiers.* Reports cases with incomplete case identifiers. For a particular case, an identifier is considered incomplete if the value of any ID variable is blank or missing. Default.

DUPLICATE. *Duplicate case identifiers.* Reports cases with duplicate case identifiers. Incomplete identifiers are excluded from the set of possible duplicates. Default.

NONE . *Case identifiers are not validated.*

CASECHECKS Subcommand

The CASECHECKS subcommand specifies checks to be performed on cases in the active dataset.

REPORTEMPTY Keyword

Empty cases are identified by default. To turn off the check for empty cases, specify REPORTEMPTY=NO.

SCOPE Keyword

The SCOPE keyword controls which variables are considered when the procedure determines whether a case is empty. SCOPE is ignored if REPORTEMPTY=NO.

ANALYSISVARS. *Procedure considers analysis variables only.* A case is considered empty if values of all analysis variables are blank or missing. This is the default if analysis variables are specified. ANALYSISVARS is ignored if no analysis variables are specified.

ALLVARS. *Procedure considers all variables.* A case is regarded as empty if all variables in the active dataset are blank or missing. This is the default if no analysis variables are specified. ID variables, split file variables, and rule outcome variables are always disregarded when determining whether a case is empty.

RULESUMMARIES Subcommand

The RULESUMMARIES subcommand is used to summarize violations of single-variable validation rules.

- By default, single-variable rule violations are summarized by analysis variable.
- Cross-variable validation rules are always summarized by rule.
- RULESUMMARIES generates a warning if summaries are requested but single-variable validation rules are not defined for at least one analysis variable.

BYVARIABLE. *Summarize by variable.* For each analysis variable, shows all single-variable validation rules that were violated and the number of values that violated each rule. Also reports the total number of single-variable rule violations for each variable. This is the default.

BYRULE . *Summarize by rule.* For each single-variable validation rule, reports variables that violated the rule and the number of invalid values per variable. Also reports the total number of values that violated each rule across variables.

NONE . *No summaries are produced for single-variable rules.* An error occurs if NONE is specified along with any other RULESUMMARIES keyword.

CASEREPORT Subcommand

The CASEREPORT subcommand is used to obtain a report that lists validation rule violations for individual cases.

- The case report includes violations of single-variable and cross-variable rules.
- CASEREPORT is ignored if no single-variable or cross-variable rules are applied.

DISPLAY Keyword

- The case report is shown by default. To turn off the report, specify DISPLAY=NO.
- If you specify a case identifier, it is used to label cases in the report. Cases are also identified by case number.

MINVIOLATIONS Keyword

- MINVIOLATIONS specifies the minimum number of rule violations required for a case to be included in the report. Specify a positive integer.
- The default is 1 violation.
- MINVIOLATIONS is ignored if DISPLAY=NO.

CASELIMIT Keyword

- The CASELIMIT keyword specifies the maximum number of cases included in the case report. By default the maximum is 500.

- To change the maximum number of cases in the report, specify FIRSTN followed by a positive integer in parentheses.
- NONE turns off the case limit. All cases that meet the MINVIOLATIONS threshold are reported.
- CASELIMIT is ignored if DISPLAY=NO.

SAVE Subcommand

The SAVE subcommand specifies optional variables to save. Specify one or more keywords, each followed by an optional variable name.

- The name, if specified, must be enclosed in parentheses—for example, EMPTYCASE(Empty)—and must be a valid variable name.
- If you do not specify a variable name, the procedure uses a default name.
- If the default name is used and it conflicts with that of an existing variable, a suffix is added to make the name unique.
- If you specify a name and it conflicts with that of an existing variable, an error occurs.

EMPTYCASE. *Empty case indicator.* Empty cases are assigned the value 1. All other cases are coded 0. The default rootname is *EmptyCase*. Values of EMPTYCASE reflect the scope specified via the CASECHECKS subcommand.

INCOMPLETEID. *Incomplete ID indicator.* Cases with empty or incomplete case identifiers are assigned a value 1. All other cases are coded 0. The default rootname is *IncompleteID*.

DUPLICATEID. *Duplicate ID group.* Cases that have the same case identifier (other than cases with incomplete identifiers) are assigned the same group number. Cases with unique or incomplete identifiers are coded 0. The default rootname is *DuplicateIDGroup*.

RULEVIOLATIONS . *Total count of single-variable and cross-variable validation rule violations.* The default rootname is *ValidationRuleViolations*. Ignored if no single-variable or cross-variable validation rules are applied.

Defining Validation Rules

Single-Variable Validation Rules

To use a single-variable validation rule with the VALIDATEDATA procedure requires:

1. A rule definition (using the DATAFILE ATTRIBUTE command).

   Defining a Single-Variable Validation Rule

   Single-variable validation rules are attributes of the data file and are generally defined according to the following syntax diagram:

```
DATAFILE ATTRIBUTE
  ATTRIBUTE=$VD.SRule[n]("Label='value',"+
                         "Description='value',"+
                         "Type='{Numeric}',"+
                                {String }
                                {Date   }
                         "Format='dateformat'"+
                         "Domain='{Range}',"+
                                  {List }
                         "FlagUserMissing='{Yes}'"+
                                           {No }
                         "FlagSystemMissing='{Yes}'"+
                                            {No }
                         "FlagBlank='{Yes}'"+
                                    {No }

  -- when Domain='Range' --
                         "Minimum='value',"+
                         "Maximum='value',"+
                         "FlagUnlabeled='{Yes}'"+
                                        {No }
                         "FlagNoninteger='{Yes}'"+
                                         {No }
```

```
  -- when Domain='List' --
                        "List='value1' 'value2' ... 'valuen'"+
                        "CaseSensitive='{Yes}'"+
                                       {No }
                        ").
```

- Single-variable rule attribute names must be of the form *$VD.SRule[n]*, where *n* is a positive integer.
- Each rule specification includes a comma-separated list of properties that define the rule. Except where noted in the following table, each property is required. The order in which properties appear does not matter. Unrecognized rule properties are ignored. If the same property appears more than once in a rule specification, the value for the last one is honored.

Label . The label is text that uniquely identifies the rule among single-variable and cross-variable rules. It cannot be the null string. Leading and trailing blanks are trimmed for the purpose of determining the uniqueness of the label.

Description. Any text (optional).

Type. This is the type of variable to which the rule can be applied. Select from Numeric, String, and Date. It is used by VALIDATEDATA to ensure that the rule is internally consistent and appropriate for the variable to which it is applied.

Format . This allows you to select the date format for rules that can be applied to date variables. Specify any IBM SPSS Statistics date or time format. See the topic “Date and Time Formats” on page 53 for more information. This property is not required if *Type* is not *Date*.

FlagUserMissing. Controls whether user-missing values are flagged as invalid. Specify *Yes* or *No*.

FlagSystemMissing. Controls whether system-missing values are flagged as invalid. This does not apply to string rule types. Specify *Yes* or *No*.

FlagBlank. Controls whether blank—that is, completely empty—string values are flagged as invalid. This does not apply to non-string rule types. Specify *Yes* or *No*.

Domain. You can specify the valid values either as a *Range* or a *List* of values. "Range" properties are ignored if `Domain='List'`, and "List" properties are ignored if `Domain='Range'`.

Minimum. This property is optional**. Its value must be consistent with the rule type and format and be less than or equal to *Maximum*. The value is ignored if null. It cannot have a noninteger value if `FlagNoninteger='Yes'`.

Maximum. This property is optional**. Its value must be consistent with the rule type and format and be greater than or equal to *Minimum*. The value is ignored if null. It cannot have a non-integer value if `FlagNoninteger='Yes'`.

FlagUnlabeled. Controls whether unlabeled values are flagged as invalid. Specify *Yes* or *No*.

FlagNoninteger. Controls whether non-integer values are flagged as invalid. Specify *Yes* or *No*.

List. Specify one or more values consistent with the rule type and format. Null strings are ignored. Each item in the list must be quoted.

CaseSensitive. Controls whether case matters when string data values are checked against the list of acceptable values. Specify *Yes* or *No*.

** Range properties are optional if `Domain='Range'`. However, if `Domain='Range'` and *FlagUserMissing*, *FlagSystemMissing*, and *FlagBlank* are all set to *No*, then *Minimum* or *Maximum* must be specified or *FlagUnlabeled* or *FlagNoninteger* must be *Yes*. Otherwise, no values would be validated when the rule is applied.

2. A rule outcome variable (using COMPUTE and VARIABLE ATTRIBUTE).

   Creating a Rule Outcome Variable

   Rule outcome variables are generated to validate values of variables and are generally defined according to the following syntax diagram:

```
COMPUTE outcomevar1=expression for analysisvar1.
COMPUTE outcomevar2=...

VARIABLE ATTRIBUTE
```

```
  VARIABLES=outcomevarlist
  ATTRIBUTE=$VD.RuleOutcomeVar("Yes").
```

- The COMPUTE expression for each outcome variable should generate the value 1 for invalid cases of a variable, according to the associated single-variable rule definition.
- The rule outcome variable is marked as such using the VARIABLE ATTRIBUTE command. Rule outcome variable attribute names must be *$VD.RuleOutcomeVar* with the property *Yes*.

3. A link between the analysis and outcome variable (using VARIABLE ATTRIBUTE).

Linking Analysis and Outcome Variables

Single-variable rule links are attributes of the analysis variables and are generally defined according to the following syntax diagram:

```
VARIABLE ATTRIBUTE
  VARIABLES=analysisvar1
  ATTRIBUTE=$VD.SRuleRef[n]("Rule='$VD.SRule[n]',"+
                            "OutcomeVar='outcomevar1'")
  /VARIABLES=analysisvar2...
```

- Single-variable rule link attribute names must be of the form *$VD.SRuleRef[n]*, where *n* is a positive integer.
- The *Rule* and *OutcomeVar* properties are required. The order in which they appear does not matter. Unrecognized rule properties are ignored. If the same property appears more than once in a rule specification, the value for the last one is honored.

Rule. An existing single-variable validation rule whose *Type* property is consistent with that of the variable to which it is linked. String rules can be applied to string variables.

Outcomevar. A rule outcome variable associated with the analysis variable.

Cross-Variable Validation Rules

To use a cross-variable validation rule with the VALIDATEDATA procedure requires:

1. A rule definition (using the DATAFILE ATTRIBUTE command).

   Creating a Rule Definition

   Cross-variable validation rules are attributes of the data file and are generally defined according to the following syntax diagram:

   ```
   DATAFILE ATTRIBUTE
    ATTRIBUTE=$VD.CRule[1]("Label='value',"+
                           "Expression='value'."+
                           "OutcomeVar='outcomevar'").
   ```

   Cross-variable rule attribute names must be of the form *$VD.CRule[n]*, where *n* is a positive integer.

   - Each rule specification includes a list of properties that define the rule. Each property is required. The order in which properties appear does not matter. Unrecognized rule properties are ignored. If the same property appears more than once in a rule specification, the value for the last one is honored.

   Label. The label is text that uniquely identifies the rule among single-variable and cross-variable rules. It cannot be the null string. Leading and trailing blanks are trimmed for the purpose of determining the uniqueness of the label.

   Expression. The expression must be valid command syntax for the right side of a COMPUTE statement. Any variables used in the expression must exist in the data file.

   OutcomeVar. A rule outcome variable associated with the cross-variable rule.

2. A rule outcome variable (using COMPUTE and VARIABLE ATTRIBUTE).

Creating a Rule Outcome Variable

```
COMPUTE outcomevar = expression.

VARIABLE ATTRIBUTE
```

```
  VARIABLES=outcomevar
  ATTRIBUTE=$VD.RuleOutcomeVar("Yes").
```

- The COMPUTE expression for each outcome variable should generate the value 1 for invalid cases. It should be equivalent to the *Expression* in the cross-variable rule definition.
- The rule outcome variable is marked as such using the VARIABLE ATTRIBUTE command. Rule outcome variable attribute names must be *$VD.RuleOutcomeVar* with the property *Yes*.

VALUE LABELS

```
VALUE LABELS varlist value 'label' value 'label'... [/varlist...]
            [/datevarlist 'value' 'label'...]
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic "Command Order" on page 36 for more information.

Release History

Release 14.0

- The maximum length of a value label is extended to 120 bytes (previous limit was 60 bytes).

Release 16.0

- Limitation preventing assignment of missing values to strings with a defined width greater than eight bytes removed.

Example

```
VALUE LABELS JOBGRADE 'P' 'Parttime Employee' 'C' 'Customer Support'.
```

Overview

VALUE LABELS deletes all existing value labels for the specified variable(s) and assigns new value labels. ADD VALUE LABELS can be used to add new labels or alter labels for specified values without deleting other existing labels.

Basic Specification

The basic specification is a variable name and the individual values with their assigned labels.

Syntax Rules

- Labels can be assigned to any previously defined variables.
- It is not necessary to enter value labels for all values for a variable.
- Each value label must be enclosed in quotes. For string variables, the values themselves must also be enclosed in quotes.
- For date format variables (for example, DATE, ADATE), values expressed in date formats must be enclosed in quotes, and values must be expressed in the same date format as the defined date format for the variable.
- Value labels can contain any characters, including blanks. To enter an apostrophe as part of a label, enclose the label in double quotes or enter two apostrophes (two single quotes). See the topic "String Values in Command Specifications" on page 35 for more information.
- Each value label can be up to 120 bytes long.
- The same labels can be assigned to the values of different variables by specifying a list of variable names. For string variables, the variables specified must be of equal length.
- Multiple sets of variable names and value labels can be specified on one VALUE LABELS command as long as the sets are separated by slashes.
- To continue a label from one command line to the next, specify a plus (+) sign before the continuation of the label. Each string segment of the label must be enclosed in quotes. To insert a blank between the strings, the blank must be included in the label specification.
- To control line wrapping of labels in pivot tables and charts, insert \n as part of the label wherever you want a line break. The \n is not displayed in output; it is interpreted as a line-break character. (*Note*:

Labels will *always* wrap wherever \n appears in the defined label even if there is enough space to display the label without wrapping.)

Operations

- Unlike most transformations, VALUE LABELS takes effect as soon as it is encountered in the command sequence. Thus, special attention should be paid to its position among commands. See the topic "Command Order" on page 36 for more information.
- VALUE LABELS deletes all previously assigned value labels for the specified variables.
- The value labels assigned are stored in the dictionary of the working file and are automatically displayed on the output from many procedures.
- If a specified value is longer than the format of the variable, the program will be unable to read the full value and may not be able to assign the value label correctly.
- If the value specified for a string variable is shorter than the format of the variable, the value specification is right-padded without warning.

Examples

Assigning Value Labels to Multiple Variables

```
VALUE LABELS V1 TO V3 1 'Officials & Managers'
                      6 'Service Workers'
               /V4 'N' 'New Employee'.
```

- Labels are assigned to the values 1 and 6 for the variables between and including *V1* and *V3* in the active dataset.
- Following the required slash, a label for the value *N* of *V4* is specified. *N* is a string value and must be enclosed in quotes.
- If labels exist for the values 1 and 6 on *V1* to *V3* and the value *N* on *V4*, they are changed in the dictionary of the working file. If labels do not exist for these values, new labels are added to the dictionary.
- Existing labels for values other than 1 and 6 on *V1* to *V3* and the value *N* on *V4* are deleted.

Combining Strings to Construct Value Labels

```
VALUE LABELS  OFFICE88 1 "EMPLOYEE'S OFFICE ASSIGNMENT PRIOR"
   + " TO 1988".
```

- The label for *OFFICE88* is created by combining two strings with the plus sign. The blank between PRIOR and TO must be included in the first or second string to be included in the label.

Value Labels for String Variables

```
VALUE LABELS=STATE REGION 'U' "UNKNOWN".
```

- The label *UNKNOWN* is assigned to the value *U* for both *STATE* and *REGION*.
- *STATE* and *REGION* must be string variables of equal length. If *STATE* and *REGION* have unequal lengths, a separate specification must be made for each, as in

```
VALUE LABELS STATE 'U' "UNKNOWN" / REGION 'U' "UNKNOWN".
```

Using Value Labels with DATA LIST

```
DATA LIST / CITY 1-8(A) STATE 10-12(A).
VALUE LABELS  STATE 'TEX' "TEXAS" 'TEN' "TENNESSEE"
                    'MIN' "MINNESOTA".
BEGIN DATA
AUSTIN   TEX
MEMPHIS  TEN
ST. PAUL MIN
END DATA.
FREQUENCIES VARIABLES=STATE.
```

- The DATA LIST command defines two variables. *CITY* is eight characters wide and *STATE* is three characters. The values are included between the BEGIN DATA and END DATA commands.
- The VALUE LABELS command assigns labels to three values of variable *STATE*. Each value and each label is specified in quotes.
- The format for the variable *STATE* must be at least three characters wide because the specified values, *TEX*, *TEN*, and *MIN*, are three characters. If the format for *STATE* were two characters, the program would issue a warning. This would occur even though the values named on VALUE LABELS and the values after BEGIN DATA agree.

Forcing Value Labels to Wrap

```
VALUE LABELS myvar 1 "A long value label \n that always wraps".
FREQUENCIES myvar.
```

A Fairly Long Label That Always Wraps

		Frequency	Percent	Valid Percent	Cumulative Percent
Valid	A long value label that always wraps	1	100.0	100.0	100.0

Figure 103. Using \n to wrap value labels

VARCOMP

VARCOMP is available in SPSS Statistics Standard Edition or the Advanced Statistics Option.

```
VARCOMP dependent variable BY factor list [WITH covariate list]

  /RANDOM = factor [factor ...]

 [/METHOD = {MINQUE({1})**}
            {0}
           {ML          }
           {REML        }
           {SSTYPE({3})  }
                  {1}

 [/INTERCEPT = {INCLUDE**}]
               {EXCLUDE   }

 [/MISSING = {EXCLUDE**}]
             {INCLUDE  }

 [/REGWGT = varname]

 [/CRITERIA = [CONVERGE({1.0E-8**})] [EPS({1.0E-8**})] [ITERATE({50**})]]
                        {n       }        {n       }             {n   }

 [/PRINT = [EMS] [HISTORY({1**})] [SS]]
                          {n  }

 [/OUTFILE = [VAREST] [{COVB}] ('savfile'|'dataset') ]
                       {CORB}

 [/DESIGN = {[INTERCEPT] [effect effect ...]}]
```

** Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
VARCOMP Y1 BY B C WITH X1 X2
 /RANDOM = C.
```

Overview

The VARCOMP procedure estimates variance components for mixed models. Following the general linear model approach, VARCOMP uses indicator variable coding to construct a design matrix and then uses one of the four available methods to estimate the contribution of each random effect to the variance of the dependent variable.

Options

Regression Weights. You can use the REGWGT subcommand to specify regression weights for the model.

Estimation Methods. You can use the METHOD subcommand to use one of the four methods that are available for estimating variance components.

Tuning the Algorithm. You can control the values of algorithm-tuning parameters with the CRITERIA subcommand.

Optional Output. You can request additional output using the PRINT subcommand.

Saving the Results. You can save the variance component estimates and their asymptotic covariance matrix (if produced) to an external data file.

Basic Specification

The basic specification is one dependent variable and one or more factor variables (that define the crosstabulation) and one or more factor variables on the RANDOM subcommand (to classify factors into either fixed or random factors). By default, VARCOMP uses the minimum norm quadratic unbiased estimator with unit prior weights to estimate variance components. Default output includes a factor-level information table and a variance component estimates table.

Subcommand Order

- The variable specification must come first.
- Other subcommands can be specified in any order.

Syntax Rules

- Only one dependent variable can be specified.
- At least one factor must be specified after BY.
- At least one factor must be specified on the RANDOM subcommand.

Example

```
VARCOMP
  amtspent  BY storeid   shopfor usecoup
  /RANDOM = storeid
  /METHOD = SSTYPE (3)
  /PRINT = SS
  /PRINT = EMS
  /DESIGN = shopfor usecoup shopfor*usecoup storeid
  /INTERCEPT = INCLUDE .
```

- The procedure fits a model to *amtspent* using *storeid*, *shopfor*, and *usecoup* as factors.
- RANDOM identifies *storeid* as a random effect.
- METHOD specifies ANOVA estimation using Type III sums of squares.
- PRINT requests tabular display of the sums of squares and expected mean squares.
- DESIGN indicates that the model fit is a factorial model for the fixed effects, plus a main-effects term for the random effect. The intercept is included.

Variable List

The variable list specifies the dependent variable and the factors in the model.

- The dependent variable must be the first specification on VARCOMP.
- The factors follow the dependent variable and are separated from it by the keyword BY.
- The covariates, if any, follow the factors and are separated from the dependent variable and the factors by the keyword WITH.
- The dependent variable and the covariates must be numeric, but the factor variables can be either numeric or string.

RANDOM Subcommand

The RANDOM subcommand allows you to specify random factors.

- You must specify at least one RANDOM subcommand with one random factor.
- You can specify multiple random factors on a RANDOM subcommand. You can also use multiple RANDOM subcommands. Specifications are accumulative.
- Only factors that are listed after the keyword BY in the variable list are allowed on the RANDOM subcommand.
- If you specify a factor on RANDOM, all effects containing the factor are automatically declared as random effects.

Example

```
VARCOM Y BY DRUG SUBJECT
 /RANDOM = SUBJECT
 /DESIGN = DRUG DRUG*SUBJECT.
```

- This example specifies a mixed model where *DRUG* is the fixed factor and *SUBJECT* is a random factor.
- The default method MINQUE(1) is used to estimate the contribution of the random effect DRUG*SUBJECT to the variance of the dependent variable.

METHOD Subcommand

The METHOD subcommand offers four different methods for estimating the variances of the random effects. If more than one METHOD subcommand is specified, only the last subcommand is in effect. If the subcommand is not specified, the default method MINQUE(1) is used. METHOD cannot be specified without a keyword.

MINQUE(n). *Minimum norm quadratic unbiased estimator.* This method is the default method. When *n* = 0, zero weight is assigned to the random effects and unit weight is assigned to the residual term. When n = 1, unit weight is assigned to both the random effects and the residual term. By default, *n* = 1.

ML. *Maximum likelihood method.* Parameters of the fixed effects and variances of the random effects are estimated simultaneously. However, only the variances are reported.

REML. *Restricted maximum likelihood method.* Variances of the random effects are estimated based on residuals of the model after adjusting for the fixed effects.

SSTYPE(n). *ANOVA method.* The ANOVA method equates the expected mean squares of the random effects to their observed mean squares. Their variances are then estimated by solving a system of linear equations. The expected mean squares are computed based on the chosen type of sum of squares. Two types are available in VARCOMP: Type I (*n* = 1) and Type III (*n* = 3). Type III is the default option for this method.

INTERCEPT Subcommand

The INTERCEPT subcommand controls whether an intercept term is included in the model. If more than one INTERCEPT subcommand is specified, only the last subcommand is in effect.

INCLUDE. *Include the intercept term.* The intercept (constant) term is included in the model. This setting is the default when INTERCEPT is not specified.

EXCLUDE. *Exclude the intercept term.* The intercept (constant) term is excluded from the model. EXCLUDE is ignored if you specify the keyword INTERCEPT on the DESIGN subcommand.

MISSING Subcommand

By default, cases with missing values for any of the variables on the VARCOMP variable list are excluded from the analyses. The MISSING subcommand allows you to include cases with user-missing values.

- Pairwise deletion of missing data is not available in VARCOMP.
- If more than one MISSING subcommand is specified, only the last subcommand is in effect.

EXCLUDE. *Exclude both user-missing and system-missing values.* This setting is the default when MISSING is not specified.

INCLUDE. *Treat user-missing values as valid.* System-missing values cannot be included in the analysis.

REGWGT Subcommand

REGWGT specifies the weight variable. Values of this variable are used as regression weights in a weighted least squares model.

- Specify one numeric variable name on the REGWGT subcommand.
- Cases with nonpositive values in the regression weight variable are excluded from the analyses.

- If more than one variable is specified on the same REGWGT subcommand, only the last variable is in effect.
- If more than one REGWGT subcommand is specified, only the last subcommand is in effect.

CRITERIA Subcommand

The CRITERIA subcommand specifies numerical tolerance for checking singularity and offers control of the iterative algorithm that is used for ML or REML estimation.

- Multiple CRITERIA subcommands are allowed.
- The last specified value for any keyword takes effect. If no value is specified, the default is used.

EPS(n). *Epsilon value that is used as tolerance in checking singularity.* The value for *n* must be a positive value. The default is 1.0E-8.

CONVERGE(n). *Convergence criterion.* Convergence is assumed if the relative change in the objective function is less than the specified value. The value for *n* must be a positive value. The default is 1.0E-8. This process is available only if you specify ML or REML on the METHOD subcommand.

ITERATE(n). *Maximum number of iterations*. The value for *n* must be a positive integer. The default is 50. This process is available only if you specify ML or REML on the METHOD subcommand.

PRINT Subcommand

The PRINT subcommand controls the display of optional output. If PRINT is not specified, the default output includes a factor information table and a variance component estimates table.

- For the maximum likelihood (ML) and restricted maximum likelihood (REML) methods, an asymptotic covariance matrix of the variance estimates table is also displayed.
- If more than one PRINT subcommand is specified, the specifications are accumulated. However, if you specify the keyword HISTORY more than once but with different values for *n*, the last specification is in effect.

EMS. *Expected mean squares*. This output shows expected mean squares of all of the effects. This process is available only if you specify SSTYPE(n) on the METHOD subcommand.

HISTORY(n). *Iteration history*. The table contains the objective function value and variance component estimates at every *n* iteration. The value for *n* must be a positive integer. The default is 1. The last iteration is always printed if HISTORY is specified on PRINT. This process is available only if you specify ML or REML on the METHOD subcommand.

SS. *Sums of squares*. The table contains sums of squares, degrees of freedom, and mean squares for each source of variation. This process is available only if you specify SSTYPE(n) on the METHOD subcommand.

OUTFILE Subcommand

The OUTFILE subcommand writes the variance component estimates to a data file or a previously declared dataset (DATASET DECLARE command) that can be used in other procedures. For the ML and REML methods, OUTFILE can also write the asymptotic covariance or correlation matrix. If more than one OUTFILE subcommand is specified, the last specification is in effect.

- OUTFILE writes to an external data file or previously declared dataset. You must specify a quoted file specification or dataset name in parentheses.
- COVB and CORB are available only if you specify ML or REML on the METHOD subcommand.
- COVB and CORB are mutually exclusive; only one of them can be specified on an OUTFILE subcommand.

VAREST. *Variance component estimates.* A variable will be created to contain the estimates, and another variable will be created to hold the labels of the variance components.

COVB. *Covariance matrix*. This matrix is the asymptotic covariance matrix of the variance component estimates. One variable is created for each variance component.

CORB. *Correlation matrix*. This matrix is the asymptotic correlation matrix of the variance component estimates. One variable is created for each variance component.

('savfile'|'dataset'). *Output file specification or previously declared dataset name*. Specify a quoted file specification or dataset name, enclosed in parentheses. The variance component estimates and the asymptotic covariance or correlation matrix (if requested) are written to the same file.

DESIGN Subcommand

The DESIGN subcommand specifies the effects in a model. DESIGN can be specified anywhere after the variable list. If more than one DESIGN subcommand is specified, only the last subcommand is in effect.

- Specify a list of effect terms to be included in the design. Each term must be separated from the next term by a comma or a space. Valid specifications include the keyword INTERCEPT, factors, covariates, and interaction or nested terms.
- The factors and covariates must have been specified on the variable list.
- If a factor is specified on the RANDOM subcommand, all effects that include that factor are random effects.
- If the DESIGN subcommand is omitted or specified without any term, the default design is generated. The default design includes the intercept term (if INTERCEPT=EXCLUDE is not specified), the covariates (if any) in the order in which they are specified on the variable list, the main factorial effects, and all orders of factor-by-factor interaction.

INTERCEPT. *Include the intercept term.* Specifying INTERCEPT on DESIGN explicitly includes the intercept term, regardless of the specification on the INTERCEPT subcommand.

BY. *Interaction.* You can also use the asterisk (*). Interaction terms can be formed among factors, among covariates, and between factors and covariates.

- Factors inside an interaction effect must be distinct. For factors *A, B,* and *C,* expressions like A*C*A or A*A are invalid.
- Covariates inside an interaction effect do not have to be distinct. For covariate *X*, X*X is the product of *X* and itself. This is equivalent to a covariate whose values are the square of the values of *X*.

WITHIN. *Nesting*. You can also use a pair of parentheses. Factors and covariates can be nested within factors, but no effects can be nested within covariates. Suppose that *A* and *B* are factors and *X* and *Y* are covariates. Both A(B) and X(B) are valid, but X(Y) is not valid.

- Factors inside a nested effect must be distinct. Expressions like A(A) are invalid.
- Multiple-level nesting is supported. For example, A(B(C)) or A WITHIN B WITHIN C means that factor *B* is nested within factor *C*, and factor *A* is nested within *B(C)*. The expression A(B)(C) is invalid.
- Nesting within an interaction effect is valid. For example, A(B*C) means that factor *A* is nested within *B*C* while X(A*B) means that covariate *X* is nested within *A*B.*
- Interactions among nested effects are allowed. For example, A*B(C) means interaction between *A* and *B* within levels of *C*. X*Y(A) means the product of *X* and *Y* nested within levels of *C*. The expression A(C)*B(C) is invalid.

Example

```
VARCOM Y BY DRUG SUBJECT WITH X
 /RANDOM = SUBJECT
 /DESIGN = DRUG SUBJECT DRUG*SUBJECT X*SUBJECT.
```

- The DESIGN subcommand specifies two main effects and two interaction terms.
- All effects that involve the factor *SUBJECT* are assumed to be random.

VARIABLE ALIGNMENT

```
VARIABLE ALIGNMENT varlist ({LEFT  }) ... [/varlist...]
                           {CENTER}
                           {RIGHT }
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
VARIABLE ALIGNMENT sales95 sales96 (LEFT)
  /id gender (RIGHT).
```

Overview

VARIABLE ALIGNMENT specifies the alignment of data values in the Data Editor. It has no effect on the format of the variables or the display of the variables or values in other windows or printed results.

Basic Specification

The basic specification is a variable name and the keyword LEFT, RIGHT, or CENTER in parentheses.

VARIABLE ATTRIBUTE

```
VARIABLE ATTRIBUTE
        VARIABLES=varlist
        ATTRIBUTE=name('value') name('value')...
                  arrayname[1]('value') arrayname[2]('value')...
        DELETE=name name...
               arrayname[n] arrayname...
        /VARIABLES...
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic "Command Order" on page 36 for more information.

Release History

Release 14.0

- Command introduced.

Example

```
VARIABLE ATTRIBUTE VARIABLES=Age Gender Region
  ATTRIBUTE=DemographicVars ('1').
```

Overview

VARIABLE ATTRIBUTE provides you with the ability to define your own variable attributes and assign attribute values to variables in the active dataset. For example, you could create a variable attribute that identifies the type of response for survey questions (e.g., single selection, multiple selection, fill-in-the-blank) or the formulas used for computed variables.

- User-defined variable attributes are saved with the data file in the data dictionary.
- The VARIABLE ATTRIBUTE command takes effect immediately, updating the data dictionary without requiring a data pass.
- You can display a list of data file and variable attributes with DISPLAY ATTRIBUTES. See the topic "DISPLAY" on page 599 for more information.

Basic Specification

The basic specification is:

- The VARIABLES subcommand followed by an equals sign (=) and a list of valid variables.
- The ATTRIBUTE keyword followed by an equals sign (=) and one or more attribute names that follow variable naming rules, with each attribute name followed by a quoted attribute value enclosed in parentheses.

or

- The DELETE keyword followed by an equals sign (=) and a list of defined attribute names or attribute arrays.

Syntax Rules

- The VARIABLES subcommand is required.
- All subcommands and keywords (VARIABLES, ATTRIBUTE, DELETE) must be followed by an equals sign (=).
- Each ATTRIBUTE keyword must be followed by a name that follows variable naming rules and a single, quoted attribute value enclosed in parentheses. See the topic "Variable Names" on page 42 for more information.

- Attribute names that begin with @ cannot be displayed in Variable View of the Data Editor and are not displayed by DISPLAY DICTIONARY or DISPLAY ATTRIBUTES. They can only be displayed with DISPLAY @ATTRIBUTES.
- Attribute names that begin with a dollar sign ($) are reserved for internal IBM SPSS Statistics use.
- All attribute values must be quoted (single or double quotes), even if the values are numbers.
- Attribute values can be up to 32,767 bytes in length.

Attribute Arrays

If you append an integer enclosed in square brackets to the end of an attribute name, the attribute is interpreted as an array of attributes. For example,

```
VARIABLE ATTRIBUTE VARIABLES=Age
  ATTRIBUTE=MyAttribute[99]('not quite 100').
```

will create 99 attributes--*MyAttribute[01]* through *MyAttribute[99]*--and will assign the value "not quite 100" to the last one.

- Array subscripts (the value enclosed in square brackets) must be integers greater than 0. (Array subscript numbering starts with 1, not 0.)
- If the root name of an attribute array is the same as an existing attribute name for any variables specified on the VARIABLES subcommand, the attribute array replaces the existing attribute for those variables (and vice versa). If no value is assigned to the first element in the array (subscript [1]), the original attribute value is used for that element value.

With the DELETE keyword, the following rules apply to attribute arrays:

- If you specify DELETE followed by an array rootname and no value in square brackets, all attributes in the array are deleted.
- If you specify DELETE with an array name followed by an integer value in square brackets, the specified array element is deleted and the integer values for all subsequent attributes in the array (in numeric order) are changed to reflect the new order of array elements.

Example

```
VARIABLE ATTRIBUTE
  VARIABLES=Age Income1 Income2 Income3
  ATTRIBUTE=AnswerFormat('Fill-in')
  /VARIABLES=Gender OwnHome MortgageType
  ATTRIBUTE=AnswerFormat('Single select')
  /VARIABLES=AvgIncome
  ATTRIBUTE=AnswerFormat('Computed value')
  /VARIABLES=AvgIncome
   ATTRIBUTE=Formula('mean(Income1, Income2, Income3)')
   /VARIABLES=AvgIncome
   ATTRIBUTE=DerivedFrom[1]('Income1')
   DerivedFrom[2]('Income2')
   DerivedFrom[3]('Income3').
DISPLAY ATTRIBUTES.
```

- The variables *Age*, *Income1*, *Income2*, and *Income3* are assigned the variable attribute *AnswerFormat* with a value of *Fill-in*.
- The variables *Gender*, *OwnHome*, and *MortgageType* are assigned the variable attribute *AnswerFormat* with a value of *Single select*.
- The variable *AvgIncome* is assigned the variable attribute *AnswerFormat* with a value of *Computed value*.
- The variable *AvgIncome* is assigned the variable attribute *Formula*, with a value the describes the formula used to calculate the value of this variable.
- The variable *AvgIncome* is assigned the variable attribute array *DerivedFrom*, and each array value identifies one of the variables used to calculate the value of *AvgIncome*.
- DISPLAY ATTRIBUTES lists the variable attributes for each variable in the active dataset.

Variable Attributes

Variable	Attribute	Value
Age	AnswerFormat	Fill-in
Gender	AnswerFormat	Single select
Income1	AnswerFormat	Fill-in
Income2	AnswerFormat	Fill-in
Income3	AnswerFormat	Fill-in
OwnHome	AnswerFormat	Single select
MortgageType	AnswerFormat	Single select
AvgIncome	AnswerFormat	Computed value
	DerivedFrom[1]	Income1
	DerivedFrom[2]	Income2
	DerivedFrom[3]	Income3
	Formula	mean(Income1, Income2, Income3)

Figure 104. Variable attributes

VARIABLE LABELS

```
VARIABLE LABELS varname 'label' [/varname...]
```

Example

```
VARIABLE LABELS YRHIRED 'YEAR OF FIRST HIRING'.
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic "Command Order" on page 36 for more information.

Overview

VARIABLE LABELS assigns descriptive labels to variables in the active dataset.

Basic Specification

The basic specification is a variable name and the associated label in quotes.

Syntax Rules

- Labels can be added to any previously defined variable. It is not necessary to enter labels for all variables in the active dataset.
- Each variable label must be enclosed in quotes.
- Variable labels can contain any characters, including blanks. To enter an apostrophe as part of a label, enclose the label in double quotes or enter a two apostrophes (two single quotes). See the topic "String Values in Command Specifications" on page 35 for more information.
- Each variable label can be up to 256 bytes long, although some procedures print fewer than the 256 bytes. All statistical procedures display at least 40 bytes.
- Multiple variables can be assigned labels on a single VARIABLE LABELS command. Only one label can be assigned to each variable, and each label can apply to only one variable.
- To continue a label from one command line to the next, specify a plus (+) sign before the continuation of the label. Each string segment of the label must be enclosed in quotes. To insert a blank between the strings, the blank must be included in the label specification.
- To control line wrapping of labels in pivot tables and charts, insert \n as part of the label wherever you want a line break. The \n is not displayed in output; it is interpreted as a line-break character. (*Note*: Labels will *always* wrap wherever \n appears in the defined label, even if there is enough space to display the label without wrapping.)

Operations

- Unlike most transformations, VARIABLE LABELS takes effect as soon as it is encountered in the command sequence. Thus, special attention should be paid to its position among commands. See the topic "Command Order" on page 36 for more information.
- Variable labels are automatically displayed in the output from many procedures and are stored in the dictionary of the active dataset.
- VARIABLE LABELS can be used for variables that have no previously assigned variable labels. If a variable has a previously assigned variable label, the new label replaces the old label.

Examples

Assigning Variable Labels to Multiple Variables

```
VARIABLE LABELS  YRHIRED 'YEAR OF FIRST HIRING'
  DEPT88 'DEPARTMENT OF EMPLOYMENT IN 1988'
```

```
  SALARY88 'YEARLY SALARY IN 1988'
  JOBCAT 'JOB CATEGORIES'.
```

- Variable labels are assigned to the variables *YRHIRED*, *DEPT88*, *SALARY88*, and *JOBCAT*.

Combining Strings to Construct Variable Labels

```
VARIABLE LABELS  OLDSAL "EMPLOYEE'S GROSS SALARY PRIOR"
   + " TO 1988".
```

- The label for *OLDSAL* is created by combining two strings with the plus sign. The blank between PRIOR and TO must be included in the first or second string to be included in the label.

Forcing Variable Labels to Wrap

```
VARIABLE LABELS myvar "A Fairly Long Label \n That Always Wraps".
FREQUENCIES myvar.
```

A Fairly Long Label
That Always Wraps

		Frequency	Percent	Valid Percent	Cumulative Percent
Valid	1.00	1	100.0	100.0	100.0

Figure 105. Using \n to wrap variable labels

VARIABLE LEVEL

```
VARIABLE LEVEL varlist ({SCALE  }) ... [/varlist...]
                        {ORDINAL}
                        {NOMINAL}
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
VARIABLE LEVEL sales95 sales96 (SCALE)
    /region division (NOMINAL)
    /expense (ORDINAL).
```

Overview

VARIABLE LEVEL specifies the level of measurement for variables.

Basic Specification

The basic specification is a variable name, followed by a measurement level enclosed in parentheses. The measurement level can be:

NOMINAL. A variable can be treated as nominal when its values represent categories with no intrinsic ranking; for example, the department of the company in which an employee works. Examples of nominal variables include region, zip code, or religious affiliation.

ORDINAL. A variable can be treated as ordinal when its values represent categories with some intrinsic ranking; for example, levels of service satisfaction from highly dissatisfied to highly satisfied. Examples of ordinal variables include attitude scores representing degree of satisfaction or confidence and preference rating scores.

SCALE. A variable can be treated as scale when its values represent ordered categories with a meaningful metric, so that distance comparisons between values are appropriate. Examples of scale variables include age in years and income in thousands of dollars.

VARIABLE ROLE

```
VARIABLE ROLE
  [/INPUT varlist]
  [/TARGET varlist]
  [/BOTH varlist]
  [/NONE varlist]
  [/PARTITION varlist]
  [/SPLIT varlist]
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Release History

Release 18.0

- Command introduced.

Example

```
VARIABLE ROLE
  /INPUT Age Income Gender Education Debt
  /TARGET Credit_rating  /NONE Customer_ID.
```

Overview

Some dialogs support predefined roles that can be used to pre-select variables for analysis. When you open one of these dialogs, variables that meet the role requirements will be automatically displayed in the destination list(s). Available role subcommands are:

Input. The variable will be used as an input (e.g., predictor, independent variable).

Target. The variable will be used as an output or target (e.g., dependent variable).

Both. The variable will be used as both input and output.

None. The variable has no role assignment.

Partition. The variable will be used to partition the data into separate samples for training, testing, and validation.

Split. Included for round-trip compatibility with IBM SPSS Modeler. Variables with this role are *not* used as split-file variables in IBM SPSS Statistics.

- By default, all variables are assigned the *Input* role. This includes data from external file formats and data files from versions of IBM SPSS Statistics prior to version 18.
- Role assignment only affects dialogs that support role assignment. It has no effect on command syntax.

Basic Specification

The basis specification is the command name, at least one subcommand, and at least one variable specified on that subcommand.

Syntax Rules

- At least one subcommand must be specified.
- Only a single instance of each subcommand is allowed.
- Each subcommand must be preceded by a forward slash.
- Each subcommand must specify at least one variable.
- The command name and subcommand names must be spelled in full.
- Keyword ALL can be used to specify all variables in the active dataset, and keyword T0 can be used to specify a list of variables (in file order).

Operations

- If a variable is specified more than once, the last role assignment is used.
- This command takes effect immediately. It does not read the active dataset or execute pending transformations.

Example

```
VARIABLE ROLE
  /INPUT Age Income Gender Education Debt
  /TARGET Credit_rating  /NONE Customer_ID.
```

VARIABLE WIDTH

```
VARIABLE WIDTH varlist (n) ... [/varlist...]
```

Example

```
VARIABLE WIDTH sales95 sales96 (10)
  /id gender (2).
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Overview

VARIABLE WIDTH specifies the column width for the display of variables in the Data Editor. It has no effect on the format of the variable or the display of the variable or values in other windows or printed results.

Basic Specification

The basic specification is a variable name and a positive integer in parentheses for the column width. Column width for proportional fonts is based on average character width. Depending on the characters used in the value, more or fewer characters may be displayed in the specified width.

VARSTOCASES

```
VARSTOCASES

/MAKE new variable ["label"] [FROM] varlist [/MAKE ...]

[/INDEX = {new variable ["label"]                                    }]
          {new variable ["label"] (make variable name)               }
          {new variable ["label"] (n) new variable ["label"](n) ...}

[/ID = new variable ["label"]]

[/NULL = {DROP**}]
         {KEEP  }

[/COUNT=new variable ["label"]]

[/KEEP={ALL**  }] [/DROP=varlist]
       {varlist}
```

**Default if the subcommand is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
VARSTOCASES /MAKE newvar FROM var1 TO var4.
```

Overview

A **variable** contains information that you want to analyze, such as a measurement or a test score. A **case** is an observation, such as an individual or an institution. In a simple data structure, each variable is a single **column** in your data. So if you are measuring test scores, for example, all test score values would appear in only one column. In a simple data structure, each case is a single **row** in your data. So if you were measuring scores for all students in a class, there would be a row for each student.

VARSTOCASES restructures **complex** data structures (in which information about a variable is stored in more than one column) into a data file in which those measurements are organized into separate rows of a single column. It replaces the active dataset.

You can use VARSTOCASES to restructure data files in which repeated measurements of a single case were recorded in one row into a new data file in which each measurement for a case appears in a new row.

Options

Creating New Variables. You can create an **identification** variable that identifies the row in the original data file that was used to create a group of new rows, a **count** variable that contains the number of new rows generated by a row in the original data, and one or more **index** variables that identify the original variable from which the new row was created.

Variable Selection. You can use the DROP and KEEP subcommands to specify which variables from the original data file are included in the new data file.

Basic Specification

The basic specification is one or more MAKE subcommands, each of which specifies a list of variables to be combined into a single variable in which each value is displayed on a separate row.

Subcommand Order

Subcommands can be specified in any order.

Syntax Rules

- The MAKE subcommand is required and can be specified as many times as needed.
- The rest of the subcommands can be specified only once.

Operations

- **Row order.** New rows are created in the order in which the variables are specified on the FROM list.
- **Propagated variables.** Variables that are not named on the MAKE or DROP subcommands are kept in the new data file. Their values are propagated for each new row.
- **Split file processing.** The SPLIT FILE command does not affect the results of VARSTOCASES. If split file processing is in effect, it will remain in effect in the new data file unless a variable that is used to split the file is named on the MAKE or DROP subcommands.
- **Weighted files.** The WEIGHT command does not affect the results of VARSTOCASES. If original data are weighted, the new data will be weighted unless the variable that is used to split the file is named on the MAKE or DROP subcommands.
- **Selected cases.** The FILTER and USE commands do not affect the results of VARSTOCASES. It processes all cases.

Limitations

The TEMPORARY command cannot be in effect when VARSTOCASES is executed.

Example

The following is the LIST output for a data file where repeated measurements for the same case are stored in variables on a single row:

```
caseid          var1         var2         var3    var4

001              .00          .05         5.00    3.00
002             7.00         1.00         5.00    4.00
003             6.00         3.00         6.00    2.00
```

The command:

```
VARSTOCASES
    /MAKE newvar FROM var1 TO var4.
```

creates a new variable, *newvar*, using the values of *var1* through *var4*. The LIST output for the new active dataset is as follows:

```
caseid      newvar

001            .00
001            .05
001           5.00
001           3.00
002           7.00
002           1.00
002           5.00
002           4.00
003           6.00
003           3.00
003           6.00
003           2.00
```

The values for the new variable *newvar* are the values from *var1* through *var4* from the original data. There are now four rows for each case—one row for each variable that was named on the FROM list.

MAKE Subcommand

The MAKE subcommand names, and optionally labels, the new variable to be created from the variables on the FROM list.

- One new variable is required on each MAKE subcommand. It must have a unique name.
- The label for the new variable is optional and, if specified, must be enclosed in quotes.
- The new variable will have the values of the variables listed on the FROM list. For each case in the original data, one new row will be created for each variable on the FROM list.

- All of the variables on the FROM list are required to be of the same type. For example, they must all be numeric or they must all be string.
- The dictionary information for the new variable (for example, value labels and format) is taken from the first variable in the FROM list. If string variables of different lengths are specified, the longest length is used.
- Rows are created in the order in which variables appear on the FROM list.
- Variables that appear on the FROM list will not appear in the new data file.
- Variables that are kept in the new data file and not named on the FROM list will have their values propagated for each new row.
- When multiple MAKE subcommands are used, a variable may not appear on more than one FROM list.
- A variable may be listed more than once on a FROM list. Its values are repeated.
- When multiple MAKE subcommands are used, the FROM lists must all contain the same number of variables (variables that are listed more than once must be included in the count).

ID Subcommand

The ID subcommand creates a new variable that identifies the permanent case sequence number (*$casenum*) of the original row that was used to create the new rows. Use the ID subcommand when the original data file does not contain a variable that identifies cases.

- One new variable is named on the ID subcommand. It must have a unique name.
- The label for the new variable is optional and, if specified, must be enclosed in quotes.
- The format of the new variable is F8.0.

INDEX Subcommand

In the original data file, a case appears on a single row. In the new data file, that case will appear on multiple rows. The INDEX subcommand creates a new variable that sequentially identifies a group of new rows based on the original variables from which it was created.

- You can choose among three types of indices—a simple numeric index, an index that lists the variables on a FROM list, or multiple numeric indices.
- New variable(s) are named on the INDEX subcommand. A new variable must have a unique name.
- The label for the new variable is optional and, if specified, must be enclosed in quotes.

Simple Numeric Index

A simple numeric index numbers the rows sequentially within a new group.

- The basic specification is /INDEX=ivar, where *ivar* is a name for the new index variable.
- The new index variable starts with 1 and increments each time a FROM variable is encountered in a row in the original file. After the last FROM variable is encountered, the index restarts at 1.
- Gaps in the index sequence can occur if null data are dropped.

Example

```
VARSTOCASES
    /MAKE newvar FROM var1 TO var4
    /INDEX=ivar.
```

```
caseid      ivar    newvar

001            1       .00
001            2       .05
001            3      5.00
001            4      3.00
002            1      7.00
002            2      1.00
002            3      5.00
```

```
002           4      4.00
003           1      6.00
003           2      3.00
003           3      6.00
003           4      2.00
```

Variable Name Index

A variable name index works like the simple numeric index, except that it lists the name of the original FROM variable instead of a sequential number.

- The basic specification is `/INDEX=ivar (make variable name)`, where *ivar* is the name for the new index variable and *make variable name* is the name of a variable on the MAKE subcommand from which the index is to be constructed.
- The new index variable is a string that lists the name of the FROM variable from which the new row was created.

Example

```
VARSTOCASES
    /MAKE newvar FROM var1 TO var4
    /INDEX=ivar (newvar).
```

```
caseid      ivar    newvar

001         VAR1       .00
001         VAR2       .05
001         VAR3      5.00
001         VAR4      3.00
002         VAR1      7.00
002         VAR2      1.00
002         VAR3      5.00
002         VAR4      4.00
003         VAR1      6.00
003         VAR2      3.00
003         VAR3      6.00
003         VAR4      2.00
```

Multiple Numeric Indices

Multiple numeric indices are used to identify groups of new rows that share a particular combination of factors. You can create multiple numeric indices if the original variables are ordered so that levels of a given factor are grouped together.

- The basic specification is `/INDEX=ivar(n) ivar(n) ...`, where *ivar* is the name of the new index for a factor and *n* is the number of factor levels represented in the variable group for which the index is being constructed.
- The last index specified varies the fastest.

Example

```
       B1      B2

A1    .00     .05

A2   5.00    3.00
```

- Data were collected for a designed experiment with two levels of factor A and two levels of factor B. The table shows the data for the first case.

```
caseid     v_a1b1    v_a1b2    v_a2b1    v_a2b2
001           .00       .05      5.00      3.00
```

- The original data file is structured so that each case has one variable for each combination of factors. Note that factor B varies fastest.

```
VARSTOCASES
    /MAKE newvar FROM v_a1b1 TO v_a2b2
    /INDEX=a(2) b(2).
```

```
caseid      a     b    newvar

001         1     1       .00
001         1     2       .05
001         2     1      5.00
001         2     2      3.00
```

- The command restructures the data file and creates two indices, *A* and *B*.

NULL Subcommand

The NULL subcommand checks each potential new row for **null** values. A null value is a system-missing or blank value. By default, VARSTOCASES does not add a new row that contains null values for all variables created by MAKE subcommands. You can change the default null-value treatment with the NULL subcommand.

DROP. *Do not include a new row when all MAKE variables are null.* A potential new row with null values for all of the variables created by MAKE subcommands is excluded from the new data file. This is the default. With this option, you may want to create a count variable to keep track of new rows because cases in the original data file are not guaranteed to appear in the new data file.

KEEP. *Include a new row when all MAKE variables are null.* A potential new row with null values for all of the variables created by the MAKE subcommand is included in the new data. With this option, you may not need a count variable to keep track of cases because each row in the original data will result in a consistent number of rows in the new data file.

COUNT Subcommand

When there are no null data, VARSTOCASES generates *n* new rows for each row in the original data file, where *n* is the number of variables on the FROM list(s). When the original data file contains null values and you drop them, it is possible to generate a different number of rows for a given subject in the original data file. The COUNT subcommand creates a new variable that contains the number of new rows generated by the original subject.

- One new variable is named on the COUNT subcommand. It must have a unique name.
- The label for the new variable is optional and, if specified, must be enclosed in quotes.
- The format of the new variable is F4.0.

DROP and KEEP Subcommands

The DROP and KEEP subcommands are used to include only a subset of variables in the new active dataset. The DROP subcommand specifies a set of variables to exclude and the KEEP subcommand specifies a set of variables to retain. Variables not specified on the KEEP subcommand are dropped.

- DROP and KEEP cannot be used with variables that appear on a FROM list.
- DROP and KEEP are mutually exclusive. Only one DROP or one KEEP subcommand can be used on the VARSTOCASES command.
- KEEP affects the order of variables in the new data file. The order of the variables kept in the new data file is the order in which they are named on the KEEP subcommand.

Example

```
VARSTOCASES
    /MAKE newvar FROM var1 to var4
    /DROP caseid.
```

- *Caseid* is dropped from the new data file. The new data file contains one variable, *newvar*.

VECTOR

```
VECTOR  {vector  name=varname TO varname}  [/vector  name...]
        {vector  name(n  [format])      }
```

Example

```
VECTOR V=V1 TO V6.
```

Overview

VECTOR associates a vector name with a set of existing variables or defines a vector of new variables. A vector is a set of variables that can be referred to using an index. The vector can refer to either string or numeric variables, and the variables can be permanent or temporary.

For each variable in the reference list, VECTOR generates an element. Element names are formed by adding a subscript in parentheses to the end of the vector name. For example, if the vector *AGES* has three elements, the element names are *AGES(1)*, *AGES(2)*, and *AGES(3)*. Although the VECTOR command has other uses within the transformation language, it is most often used with LOOP structures because the indexing variable on LOOP can be used to refer to successive vector elements.

Options

File Structures. VECTOR can be used with the END CASE command to restructure data files. You can build a single case from several cases or, conversely, you can build several cases from a single case (see the examples for the END CASE command).

Short-Form Vectors. VECTOR can be used to create a list of new variables and the vector that refers to them simultaneously. VECTOR in the short form can be used to establish the dictionary order of a group of variables before they are defined on a DATA LIST command (see "VECTOR: Short Form").

Basic Specification

- The basic specification is VECTOR, a vector name, a required equals sign, and the list of variables that the vector refers to. The TO keyword must be used to specify the variable list, and it defines a variable list based on file order.
- For the short form of VECTOR, the basic specification is VECTOR, an alphabetical prefix, and, in parentheses, the number of variables to be created.

Syntax Rules

- Multiple vectors can be created on the same command by using a slash to separate each set of specifications.
- Variables specified on VECTOR must already be defined unless the short form of VECTOR is used to create variables (see "VECTOR: Short Form").
- The TO convention must be used to specify the variable list. Thus, variables specified must be consecutive and must be from the same dictionary, permanent or scratch.
- A single vector must comprise all numeric variables or all string variables. The string variables must have the same length.
- A scalar (a variable named on NUMERIC), a function, and a vector can all have the same name, for example *MINI*. The scalar can be identified by the lack of a left parenthesis following the name. Where a vector has the same name as a function (or the abbreviation of a function), the vector name takes precedence (see the example on name conflicts in "VECTOR: Short Form").
- Vector element names must always be specified with a subscript in parentheses.

Operations

- VECTOR takes effect as soon as it is encountered in the command sequence, unlike most transformations, which do not take effect until the data are read. Thus, special attention should be paid to its position among commands. See the topic "Command Order" on page 36 for more information.
- VECTOR is in effect only until the first procedure that follows it. The vector must be redeclared to be reused.

Limitations

- A vector element is an *expression* and can be used wherever an expression can be used in transformation commands. It can also be the *target* of an assignment in a COMPUTE or IF command. It cannot be used in other transformation commands where a variable name is required. Specifically, vector elements cannot be used instead of variable names in RECODE, COUNT, XSAVE, DO REPEAT, or other transformation commands that require variable names but not expressions.
- Vectors cannot be used in procedure commands (for example, statistical or charting procedures).
- Vector names (for the whole vector rather than elements) cannot be used anywhere but in the VECTOR command itself.

Examples

Example

```
DATA LIST FREE /height age weight.
BEGIN DATA
1 2 3
END DATA.
VECTOR x=height TO weight.
VECTOR y=age TO weight.
VECTOR z=age TO height.
```

- The list of variables specified with TO defines a list of variables in file order.
- Vector *x* contains the variables *height, age,* and *weight*.
- Vector *y* contains the variables *age* and *weight*.
- The last VECTOR command generates an error because *height* comes before *age* in file order.

Example

```
* Replace a case's missing values with the mean of all
  nonmissing values for that case.

DATA LIST FREE /V1 V2 V3 V4 V5 V6 V7 V8.
MISSING VALUES V1 TO V8 (99).
COMPUTE MEANSUB=MEAN(V1 TO V8).

VECTOR V=V1 TO V8.
LOOP #I=1 TO 8.
+  DO IF MISSING (V(#I)).
+  COMPUTE V(#I)=MEANSUB.
+  END IF.
END LOOP.

BEGIN DATA
1 99 2 3 5 6 7  8
2  3 4 5 6 7 8  9
2  3 5 5 6 7 8 99
END DATA.
LIST.
```

- The first COMPUTE command calculates the variable *MEANSUB* as the mean of all nonmissing values for each case.
- VECTOR defines the vector *V* with the original variables as its elements.
- For each case, the loop is executed once for each variable. The COMPUTE command within the loop is executed only when the variable has a missing value for that case. COMPUTE replaces the missing value with the value of *MEANSUB*.
- For the first case, the missing value for the variable *V2* is changed to the value of *MEANSUB* for that case. The missing value for the variable *V8* for the third case is changed to the value of *MEANSUB* for that case.

For additional examples of VECTOR, see the examples for the END CASE command and the IF command .

VECTOR: Short Form

VECTOR can be used to create a list of new variables and the vector that refers to them simultaneously. The short form of VECTOR specifies a prefix of alphanumeric characters followed, in parentheses, by the length of the vector (the number of variables to be created).

- The new variable names must not conflict with existing variables. If the prefix starts with the # character, the new variables are created according to the rules for scratch variables.
- More than one vector of the same length can be created by naming two or more prefixes before the length specification.
- By default, variables created with VECTOR receive F8.2 formats. Alternative formats for the variables can be specified by including a format specification with the length specification within the parentheses. The format and length can be specified in either order and must be separated by at least one space or comma. If multiple vectors are created, the assigned format applies to all of them unless you specify otherwise.

Creating a Vector from a Set of New Scratch Variables

```
VECTOR #WORK(10).
```

- The program creates the vector *#WORK*, which refers to 10 scratch variables: *#WORK1, #WORK2,* and so on, through *#WORK10*. Thus, the element *#WORK(5)* of the vector is the variable *#WORK5*.

Creating Multiple Vectors of the Same Length

```
VECTOR X,Y(5).
```

- VECTOR creates the vectors *X* and *Y*, which refer to the new variables *X1* through *X5* and *Y1* through *Y5*, respectively.

Specifying the Format of Vector Variables

```
VECTOR X(6,A5).
```

- VECTOR assigns an A5 format to the variables *X1* through *X6*.

Creating Multiple Vectors of Different Lengths and Formats

```
VECTOR X,Y(A5,6) Z(3,F2).
```

- VECTOR assigns A5 formats to the variables *X1* to *X6* and *Y1* to *Y6*, and F2 formats to the variables *Z1* to *Z3*. It doesn't matter whether the format or the length is specified first within the parentheses.

Predetermining Variable Order Using the Short Form of VECTOR

```
INPUT PROGRAM.
VECTOR X Y (4,F8.2).
DATA LIST / X4 Y4 X3 Y3 X2 Y2 X1 Y1 1-8.
END INPUT PROGRAM.

PRINT /X1 TO X4  Y1 TO Y4.
BEGIN DATA
49382716
49382716
49382716
END DATA.
```

- The short form of VECTOR is used to establish the dictionary order of a group of variables before they are defined on a DATA LIST command. To predetermine variable order, both VECTOR and DATA LIST must be enclosed within the INPUT PROGRAM and END INPUT PROGRAM commands.
- The order of the variables in the active dataset will be *X1*, *X2*, *X3*, and *X4*, and *Y1*, *Y2*, *Y3*, and *Y4*, even though they are defined in a different order on DATA LIST.
- The program reads the variables with the F1 format specified on DATA LIST. It writes the variables with the output format assigned on VECTOR (F8.2).

- Another method for predetermining variable order is to use NUMERIC (or STRING if the variables are string variables) before the DATA LIST command (see the example on variable order for the NUMERIC command). The advantage of using NUMERIC or STRING is that you can assign mnemonic names to the variables.

Name Conflicts in Vector Assignments

```
INPUT PROGRAM.
NUMERIC MIN MINI_A MINI_B MINIM(F2).
COMPUTE MINI_A = MINI(2).  /*MINI is function MINIMUM.
VECTOR MINI(3,F2).
DO REPEAT I = 1 TO 3.
+  COMPUTE MINI(I) = -I.
END REPEAT.
COMPUTE MIN = MIN(1).     /*The second MIN is function MINIMUM.
COMPUTE MINI_B = MINI(2). /*MINI now references vector MINI
COMPUTE MINIM = MINIM(3). /*The second MINIM is function MINIMUM.
END CASE.
END FILE.
END INPUT PROGRAM.
```

- In this example, there are potential name conflicts between the scalars (the variables named on NUMERIC), the vectors (named on VECTOR), and the statistical function MINIMUM.
- A name that is not followed by a left parenthesis is treated as a scalar.
- When a name followed by a left parenthesis may refer to a vector element or a function, precedence is given to the vector.

VECTOR outside a Loop Structure

VECTOR is most commonly associated with the loop structure, since the index variable for LOOP can be used as the subscript. However, the subscript can come from elsewhere, including from the data.

Example

```
* Create a single case for each of students 1, 2, and 3.

DATA LIST /STUDENT 1 SCORE 3-4 TESTNUM 6.
BEGIN DATA
1 10 1
1 20 2
1 30 3
1 40 4
2 15 2
2 25 3
3 40 1
3 55 3
3 60 4
END DATA.

VECTOR RESULT(4).
COMPUTE RESULT(TESTNUM)=SCORE.

AGGREGATE OUTFILE=*/BREAK=STUDENT
        /RESULT1 TO RESULT4=MAX(RESULT1 TO RESULT4).

PRINT FORMATS RESULT1 TO RESULT4 (F2.0).
PRINT /STUDENT RESULT1 TO RESULT4.
EXECUTE.
```

- Data are scores on tests recorded in separate cases along with a student identification number and a test number. In this example, there are four possible tests for three students. Not all students took every test.
- The vector *RESULT* creates the variables *RESULT1* through *RESULT4*.
- For each case, COMPUTE assigns the *SCORE* value to one of the four vector variables, depending on the value of *TESTNUM*. The other three vector variables for each case keep the system-missing value they were initialized to.
- Aggregating by the variable *STUDENT* creates new cases, as shown by the output from the PRINT command. The MAX function in AGGREGATE returns the maximum value across cases with the same value for *STUDENT*. If a student has taken a particular test, the one valid value is returned as the value for the variable *RESULT1*, *RESULT2*, *RESULT3*, or *RESULT4*.

```
1 10 20 30 40
2  . 15 25  .
3 40  . 55 60
```

Figure 106. PRINT output after aggregating

VERIFY

```
VERIFY [VARIABLES=series name]
```

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Example

```
VERIFY.
```

Overview

VERIFY produces a report on the status of the most current DATE, USE, and PREDICT specifications. The report lists the first and last observations in the active dataset, the current USE and PREDICT ranges, and any anomalies in the DATE variables. The number of missing values and the values of the first and last observations in the file and in the USE and PREDICT ranges can also be displayed for a specified series.

VERIFY should be used before a time series procedure whenever there is a possibility that DATE variables or USE and PREDICT ranges have been invalidated. In particular, the active dataset should be verified after you have modified the file structure with commands such as SELECT IF, SORT CASES, and AGGREGATE.

Options

If a series is specified after VERIFY, the values of the first and last observations in the file and in the USE and PREDICT periods are reported for that series. In addition, the number of observations in the active dataset that have missing values for that series is displayed. This can be useful for determining the USE ranges that do not include any missing values.

Basic Specification

The basic specification is the command keyword VERIFY.

- VERIFY displays the first and last observations in the active dataset and in the USE and PREDICT ranges. This information is presented by case number and by the values of the DATE variables.
- For DATE variables, VERIFY reports the number of missing or invalid values. In addition, DATE variables that are not properly nested within the next higher-level DATE variable, that have start or end values other than those expected at the beginning or end of a cycle, or that increment by more than the expected increment are flagged with an asterisk next to the problem. An explanation of the problem is given.

Operations

- VERIFY reports on cases remaining after any preceding SELECT IF commands.
- The USE and PREDICT ranges are defined by the last USE and PREDICT commands specified before the VERIFY command. If USE and PREDICT have not been specified, the USE range is the entire series, and the PREDICT range does not exist.

Limitations

- A maximum of 1 VARIABLES subcommand. Only one series can be specified on VARIABLES.

VARIABLES Subcommand

VARIABLES names a series to include in the report and is optional. The actual keyword VARIABLES can be omitted.

- The series named on VARIABLES must be numeric. The *DATE_* series is non-numeric and cannot be specified.
- Only one VARIABLES subcommand can be specified, and it can name only one series.

Examples

```
VERIFY.
```

- This command produces a report on the status of the most recent DATE, USE, and PREDICT specifications, as well as the first and last valid cases in the file.

```
VERIFY VARIABLE=STOCK.
```

- In addition to the default VERIFY information, this command displays information on the series *STOCK*, including the values of the first and last cases and how many values in that series are missing.

WEIGHT

```
WEIGHT  {BY  varname}
        {OFF        }
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
WEIGHT BY V1.
```

Overview

WEIGHT gives cases different weights (by simulated replication) for statistical analysis. WEIGHT can be used to weight a sample up to population size for reporting purposes or to replicate an example from a table or other aggregated data. With WEIGHT, you can arithmetically alter the sample size or its distribution.

To apply weights resulting from your sampling design, see SPSS Statistics Premium Edition or the Complex Samples option.

Basic Specification

The basic specification is keyword BY followed by the name of the weight variable. Cases are weighted according to the values of the specified variable.

Syntax Rules

- Only one numeric variable can be specified. The variable can be a precoded weighting variable, or it can be computed with the transformation language.
- WEIGHT cannot be placed within a FILE TYPE–END FILE TYPE or INPUT PROGRAM–END INPUT PROGRAM structure. It can be placed nearly anywhere following these commands in a transformation program. See the topic “Commands and Program States” on page 2187 for more information.

Operations

- Unlike most transformations, WEIGHT takes effect as soon as it is encountered in the command sequence. Thus, special attention should be paid to its position among commands. See the topic “Command Order” on page 36 for more information.
- Weighting is permanent during a session unless it is preceded by a TEMPORARY command, changed by another WEIGHT command, or turned off with the WEIGHT OFF specification.
- Each WEIGHT command overrides the previous one.
- WEIGHT uses the value of the specified variable to arithmetically replicate cases for subsequent procedures. Cases are not physically replicated.
- Weight values do not need to be integer, and some procedures, such as FREQUENCIES, CROSSTABS, and CTABLES, will use fractional values on the WEIGHT variable. However, most procedures treat the WEIGHT variable as a replication weight and will simply round fractional weights to the nearest integer. Some procedures ignore the WEIGHT variable completely, and this limitation is noted in the procedure-specific documentation. Procedures in the Complex Samples add-on module can use inverse sampling weights specified on the CSPLAN command.
- Cases with missing or nonpositive values for the weighting variable are treated as having a weight of 0 and are thus invisible to statistical procedures. They are not used in calculations even where unweighted counts are specified. These cases do remain in the file, however, and are included in case listings and saved when the file is saved.
- A file saved when weighting is in effect maintains the weighting.

- If the weighted number of cases exceeds the sample size, tests of significance are inflated; if it is smaller, they are deflated.

Examples

```
WEIGHT BY V1.
FREQ VAR=V2.
```

- The frequency counts for the values of variable *V2* will be weighted by the values of variable *V1*.

```
COMPUTE WVAR=1.
IF (GROUP EQ 1) WVAR=.5.
WEIGHT BY WVAR.
```

- Variable *WVAR* is initialized to 1 with the COMPUTE command. The IF command changes the value of *WVAR* to 0.5 for cases where *GROUP* equals 1.
- Subsequent procedures will use a case base in which cases from group 1 count only half as much as other cases.

WEIGHTED KAPPA

WEIGHTED KAPPA is available in Statistics Base Edition.

WEIGHTED KAPPA is an important generalization of the kappa statistic that measures the agreement of two ordinal subjects with identical categories.

Syntax chart

```
WEIGHTED KAPPA {varlist} {varlist1 BY varlist2}
     [ /MISSING
          [ CLASSMISSING ={TABLE**} {INCLUDE}] ]
     [ /CRITERIA
          [ ASYMPTOTICCILEVEL ={95**} {value}]
          [ WEIGHTING ={LINEAR**} {QUADRATIC}]
          [ IGNORECASE ={FALSE**} {TRUE}] ]
     [ /PRINT
          [ TABULATION({ALL} {varlist1_subset BY varlist2_subset}) ]
          [ CASE({UPPER**} {LOWER}) ]
          [ ORDER({ASCENDING**} {DESCENDING}) ] ]
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Release History

Release 27.0

Example

```
WEIGHTED KAPPA  {varlist} {varlist1 BY varlist2}
```

Overview (WEIGHTED KAPPA command)

Cohen's kappa statistic is broadly used in cross-classification as a measure of agreement between two observed raters. It is an appropriate index of agreement when ratings are nominal scales with no order structure. WEIGHTED KAPPA is an important generalization of the kappa statistic that measures the agreement of two ordinal subjects with identical categories.

Basic specification

```
WEIGHTED KAPPA  {varlist} {varlist1 BY varlist2}
```

A required command to invoke the WEIGHTED KAPPA procedure. Either one variable list or two variable lists connected by **BY** are allowed. The variables are separated by a blank space or multiple blank spaces. The procedure estimates the weighted kappa for all different variable pairs.

Syntax rules

- If one variable list **varlist** is specified, it must contain at least two different variables.
- If two variables lists **varlist1** and **varlist2** are specified, each list must contain at least one variable to run the procedure.
- If both lists contain only one variable, the variables cannot be the same. A certain variable is allowed to be specified in both **varlist1** and **varlist2**, but the pair of the same variable is not computed.

Operations

- Unlike most transformations, WEIGHT takes effect as soon as it is encountered in the command sequence. Thus, special attention should be paid to its position among commands. See the topic "Command Order" on page 36 for more information.
- When multiple table lists are in effect, missing values are handled separately for each table.
- Multiple user-missing values, if applicable, are treated as different categories.
- The variable specified first in **varlist** or the variable(s) specified by **varlist1** are tabulated as the row.

Example

```
WEIGHTED KAPPA  {varlist} {varlist1 BY varlist2}
```

varlist1 BY varlist2

Either one variable list or two variable lists connected by **BY** are allowed.

Variables are separated by a blank space or multiple blank spaces. If one variable list **varlist** is specified, it must contain at least two different variables. The procedure estimates the weighted kappa for all different variable pairs. If two variables lists (**varlist1** and **varlist2**) are specified, each list must contain at least one variable to run the procedure. If both lists contain only one variable, the variables cannot be the same. Certain variables are allowed to be specified in both **varlist1** and **varlist2**, but the variable pair will not be computed.

CRITERIA Subcommand

The CRITERIA subcommand allows the user to specify the options for the estimation of the Cohen's weighted kappa statistics.

ASYMPTOTIC_CILEVEL(value)
An optional parameter that specifies the confidence level for the estimation of the asymptotic confidence intervals. Must be a single double value in (0,100) with 95 as the default setting.

WEIGHTING(LINEAR or QUADRATIC)
An optional parameter that specifies the weighting scale type (either **LINEAR** or **QUADRATIC**). The use of **LINEAR** weights is the default setting.

IGNORE_CASE(boolean)
An optional keyword that controls whether or not the string variables are case sensitive. **FALSE** is the default setting, which indicates that strings are case sensitive. When **TRUE** is specified, strings are not case sensitive.

PRINT Subcommand

The PRINT subcommand controls the crosstabulation tables.

TABULATION(ALL or varlist1_subset BY varlist2_sublist)
An optional keyword that specifies the rating variables that are used in crosstabulation. The keyword is omitted by default, which suppresses the crosstabulation of any rating variables. When the keyword is specified, it must be followed by either the keyword **ALL** or varlist1_subset BY varlist2_subset. When **ALL** is specified, the crosstabulation tables are printed for all different variable pairs specified by WEIGHTED KAPPA varlist or WEIGHTED KAPPA varlist1 BY varlist2. The variable specified first in **varlist** or the variable(s) specified by **varlist1** will be tabulated as the row. When varlist1_subset BY varlist2_subset is specified, only those crosstabulation tables containing the pairs in **varlist1_subset** and **varlist2_subset** are

printed. Note that if specified, both **varlist1_subset** and **varlist2_subset** must contain at least one variable, and must be a subset of **varlist**, **varlist1**, and **varlist2**, respectively. The variable(s) specified by **varlist1_subset** are tabulated as the row.

CASE(UPPER or LOWER)
An optional keyword that controls whether the string that rates categories in the crosstabulation tables are displayed in uppercase or lowercase. The keyword takes effect in the presence of string variables and `/CRITERIA IGNORECASE = TRUE`. **UPPER** is the default setting,which displays the rating categories in uppercase. When **LOWER** is specified, the string categories display in lowercase.

ORDER(ASCENDING or DESCENDING)
An optional keyword that controls whether the rating categories in the crosstabulation tables are displayed in ascending or descending order. **ASCENDING** is default the setting. The parameter has no affect when the **TABULATION** keyword is omitted.

MISSING Subcommand

The `MISSING` subcommand handles the treatment of missing values.

CLASSMISSING(TABLE or INCLUDE
An optional subcommand that controls how missing values are treated. **TABLE** is the default setting, which deletes cases with missing values on a table-by-table basis. When multiple table lists are in effect, missing values are handled separately for each table. When **INCLUDE** is specified, user-missing values are included. Multiple user-missing values, if applicable, are treated as different categories.

WLS

WLS is available in SPSS Statistics Standard Edition or the Regression Option.

```
WLS VARIABLES= dependent varname WITH independent varnames

 [/SOURCE=varname]

 [/DELTA=[{1.0**                 }]]
          {value list            }
          {value TO value BY value}

 [/WEIGHT=varname]

 [/{CONSTANT**}
   {NOCONSTANT}

 [/PRINT={BEST}]
         {ALL }

 [/SAVE = WEIGHT]

 [/APPLY['model name']]
```

**Default if the subcommand or keyword is omitted.

This command reads the active dataset and causes execution of any pending commands. See the topic "Command Order" on page 36 for more information.

Example

```
WLS VARIABLES = VARY WITH VARX VARZ
  /SOURCE=VARZ
  /DELTA=2.
```

Overview

WLS (weighted least squares) estimates regression models with different weights for different cases. Weighted least squares should be used when errors from an ordinary regression are heteroscedastic—that is, when the size of the residual is a function of the magnitude of some variable, termed the **source**.

The WLS model is a simple regression model in which the residual variance is a function of the source variable, up to some power transform indicated by a delta value. For fuller regression results, save the weights produced by WLS and specify that weight variable on the REGWGT subcommand in REGRESSION.

Options

Calculated and Specified Weights. WLS can calculate the weights based on a source variable and delta values (subcommands SOURCE and DELTA), or it can apply existing weights contained in a series (subcommand WEIGHT). If weights are calculated, each weight value is calculated as the source series value raised to the negative delta value.

New Variables. You can change NEWVAR settings on the TSET command prior to WLS to evaluate the regression coefficients and log-likelihood function without saving the weight variable, or save the new values to replace the values saved earlier, or save the new values without erasing values saved earlier (see the TSET command). You can also use the SAVE subcommand on WLS to override the NONE or the default CURRENT settings on NEWVAR for the current procedure.

Statistical Output. You can change the PRINT setting on the TSET command prior to WLS to display regression coefficients or the list of log-likelihood functions at each delta value, or to limit the output to only the regression statistics for the delta value at which the log-likelihood function is maximized (see the TSET command). You can also use the PRINT subcommand to override the PRINT setting on the TSET command for the current procedure and obtain regression coefficients at each value of delta in addition to the default output.

Basic Specification

- The basic specification is the VARIABLES subcommand specifying one dependent variable, the keyword WITH, and one or more independent variables. Weights are calculated using the first independent variable as the source variable and a default delta value of 1.
- The default output for calculated weights displays the log-likelihood function for each value of delta. For the value of delta at which the log-likelihood function is maximized, the displayed summary regression statistics include *R*, R^2, adjusted R^2, standard errors, analysis of variance, and *t* tests of the individual coefficients. A variable named *WGT#1* containing the calculated weights is automatically created, labeled, and added to the active dataset.

Syntax Rules

- VARIABLES can be specified only once.
- DELTA can be specified more than once. Each specification will be executed.
- If other subcommands are specified more than once, only the last specification of each one is executed.
- You can specify either SOURCE and DELTA, or just the WEIGHT subcommand. You cannot specify all three, and you cannot specify WEIGHT with SOURCE or with DELTA.

Subcommand Order

- Subcommands can be specified in any order.

Operations

- If neither the WEIGHT subcommand nor the SOURCE and DELTA subcommands are specified, a warning is issued and weights are calculated using the default source and delta value.
- Only one *WGT#1* variable is created per procedure. If more than one delta value is specified, the weights used when the log-likelihood function is maximized are the ones saved as *WGT#1*.
- *WGT#1* is not created when the WEIGHT subcommand is used.
- The WEIGHT command specifies case replication weights, which are *not* the same as the weights used in weighted least squares. If the WEIGHT command and WLS WEIGHT subcommand are both specified, both types of weights are incorporated in WLS.
- WLS uses listwise deletion of missing values. Whenever one variable is missing a value for a particular observation, that observation will not be included in any computations.

Limitations

- Maximum one VARIABLES subcommand.
- Maximum one dependent variable on the VARIABLES subcommand. There is no limit on the number of independent variables.
- Maximum 150 values specified on the DELTA subcommand.

Example

```
WLS cost  WITH sqft inorout yrexp
  /SOURCE sqft
  /POWER 0 TO 5 BY 0.5
  /CONSTANT
  /SAVE WEIGHT
  /PRINT BEST.
```

- This command specifies a weighted least-squares regression in which *cost* is the dependent variable and *sqft*, *inorout*, and *yrexp* are the independent variables.
- The SOURCE subcommand identifies *sqft* as the source of heteroscedasticity.
- The POWER subcommand specifies that several equations will be fit, one for each value from 0 to 5, incrementing by 0.5. Weights will be calculated using *sqft*.
- The SAVE subcommand requests that the weights that maximize the log-likelihood function be saved to the active dataset.
- All other options are set to their default values.

VARIABLES Subcommand

VARIABLES specifies the variable list and is the only required subcommand.

SOURCE Subcommand

SOURCE is used in conjunction with the DELTA subcommand to compute weights. SOURCE names the variable that is the source of heteroscedasticity.

- The only specification on SOURCE is the name of a variable to be used as the source of heteroscedasticity.
- Only one source variable can be specified.
- If neither SOURCE nor WEIGHT is specified, the first independent variable specified on the VARIABLES subcommand is assumed to be the source variable.

DELTA Subcommand

DELTA, alias POWER, is used in conjunction with the SOURCE subcommand to compute weights. DELTA specifies the values to use in computing weights. The weights are equal to 1/(SOURCE raised to the DELTA power).

- The specification on DELTA is a list of possible delta values and/or value grids.
- Multiple values and grids can be specified on one DELTA subcommand.
- Delta values can be any value in the range of –6.5 to +7.5. Values below this range are assigned the minimum (–6.5), and values above are assigned the maximum (7.5).
- A grid is specified by naming the starting value, the keyword TO, an ending value, the keyword BY, and an increment value. Alternatively, the keyword BY and the increment value can be specified after the starting value.
- More than one DELTA subcommand can be specified; each subcommand will be executed.
- If DELTA is not specified, the delta value defaults to 1.0.

Example

```
WLS VARIABLES = X1 WITH Y1 Z1
  /SOURCE=Z1
  /DELTA=0.5.
```

- In this example, weights are calculated using the source variable *Z1* and a delta value of 0.5. Thus, the weights are 1/(SQRT (Z1)).

Example

```
WLS VARIABLES = SHARES WITH PRICE
  /DELTA=0.5 TO 2.5 BY 0.5.
```

- In this example, several regression equations will be fit, one for each value of delta.
- Weights are calculated using the source variable *PRICE* (the default).
- The delta values start at 0.5 and go up to 2.5, incrementing by 0.5. This specification is equivalent to 0.5 BY 0.5 TO 2.5.
- The weights that maximize the log-likelihood function will be saved as variable *WGT#1*.

WEIGHT Subcommand

WEIGHT specifies the variable containing the weights to be used in weighting the cases. WEIGHT is an alternative to computing the weights using the SOURCE and DELTA subcommands. If a variable containing

weights is specified, the output includes the regression coefficients, log-likelihood function, and summary regression statistics such as *R*, R^2, adjusted R^2, standard errors, analysis of variance, and *t* tests of the coefficients. Since no new weights are computed, no new variable is created.

- The only specification on WEIGHT is the name of the variable containing the weights. Typically, *WGT* variables from previous WLS procedures are used.
- Only one variable can be specified.

Example

```
WLS VARIABLES = SHARES WITH PRICE
  /WEIGHT=WGT_1.
```

- This WLS command uses the weights contained in variable *WGT_1* to weight cases.

CONSTANT and NOCONSTANT Subcommands

Specify CONSTANT or NOCONSTANT to indicate whether a constant term should be estimated in the regression equation. The specification of either subcommand overrides the CONSTANT setting on the TSET command for the current procedure.

- CONSTANT is the default and specifies that the constant term is used as an instrument.
- NOCONSTANT eliminates the constant term.

SAVE Subcommand

SAVE saves the weight variable generated during the current session to the end of the active dataset. The default name *WGT_n* will be generated, where *n* increments to make the variable name unique. The only specification on SAVE is WEIGHT. The specification overrides the NONE or the default CURRENT setting on NEWVAR for the current procedure.

PRINT Subcommand

PRINT can be used to override the PRINT setting on the TSET command for the current procedure. Two keywords are available.

BEST. *Display coefficients for the best weight only.* This is the default.

ALL. *Display coefficients for all weights.*

APPLY Subcommand

- The APPLY subcommand allows you to use a previously defined WLS model without having to repeat the specifications.
- The only specification on APPLY is the name of a previous model in quotes. If a model name is not specified, the model specified on the previous WLS command is used.
- To change one or more model specifications, specify the subcommands of only those portions you want to change after the APPLY subcommand.
- If no variables are specified on the command, the variables that were originally specified with the model being reapplied are used.

Example

```
WLS VARIABLES = X1 WITH Y1
  /SOURCE=Y1
  /DELTA=1.5.
WLS APPLY
  /DELTA=2.
```

- The first command produces a weighted least-squares regression of *X1*, with *Y1* as the source variable and delta equal to 1.5.
- The second command uses the same variable and source but changes the delta value to 2.

Example

```
WLS VARIABLES = X1 WITH Y1 Z1
  /SOURCE=Z1
  /DELTA=1 TO 3 BY 0.5
WLS APPLY
  /WEIGHT=WGT#1.
```

- The first command regresses *X1* on *Y1* and *Z1*, using *Z1* as the source variable. The delta values range from 1 to 3, incrementing by 0.5.
- The second command again regresses *X1* on *Y1* and *Z1*, but this time applies the values of *WGT#1* as the weights.

WRITE

```
WRITE [OUTFILE='file']
  [ENCODING='encoding'] [BOM={YES}]
                           {NO }]
  [RECORDS={1}] [{NOTABLE}]
            {n}   {TABLE  }

 /{1    } varlist [{col location [(format)]}] [varlist...]
  {rec #}          {(format list)           }
                   {*                       }

 [/{2    }...]
   {rec #}
```

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic “Command Order” on page 36 for more information.

Release History

Release 16.0

- ENCODING subcommand added for Unicode support.

Release 22.0

- BOM keyword added.

Example

```
WRITE OUTFILE="/data/personnel.txt' / MOHIRED YRHIRED DEPT SALARY NAME.
EXECUTE.
```

Overview

WRITE writes files in a machine-readable format that can be used by other software applications. When used for this purpose, the OUTFILE subcommand is required. If OUTFILE is not specified, the output from WRITE that can be displayed is included with the output from your session in a format similar to that used by the PRINT command.

Options

Formats. You can specify formats for the variables.

Strings. You can include strings within the variable specifications. The strings can be used to label values or to add extra space between values.

Multiple Lines per Case. You can write variables on more than one line for each case. See the RECORDS subcommand.

Output File. You can direct the output to a specified file using the OUTFILE subcommand.

Summary Table. You can display a table that summarizes the variable formats with the TABLE subcommand.

Subcommand Order

Subcommands can be specified in any order. However, all subcommands must be used before the slash that precedes the first variable list.

Basic Specification

The basic specification is a slash followed by a variable list. The values for all of the variables specified on the list are included with the rest of the output from your session.

Syntax Rules

- A slash must precede the variable specifications. The first slash begins the definition of the first (and possibly only) line per case of the WRITE output.
- Specified variables must already exist, but they can be numeric, string, scratch, temporary, or system variables. Subscripted variable names, such as *X(1)* for the first element in vector *X* cannot be used.
- Keyword ALL can be used to write the values of all user-defined variables in the active dataset.

Operations

- WRITE is executed once for each case constructed from the data file.
- Values are written to the file as the data are read.
- WRITE is a transformation and will not be executed unless it is followed by a procedure or the EXECUTE command.
- When writing to an external file with the OUTFILE subcommand, line/record width can be up to 2.1 billion bytes. When writing to the Viewer, however (when there is no OUTFILE subcommand), if a line width exceeds the line width defined by SET WIDTH, an error results and the WRITE command is not executed. The maximum line width you can specify with SET WIDTH is 255 bytes.
- There are no carriage control characters in the output file generated by WRITE.
- User-missing values are written just like valid values. System-missing values are represented by blanks.
- If you are writing a file to be used on another system, you should take into account that some data types cannot be read by all computers.
- If long records are less convenient than short records with multiple records per case, you can write out a case identifier and insert a string as a record identification number. The receiving system can then check for missing record numbers (see for an example).

Examples

Writing a Specific Set of Variables

```
WRITE OUTFILE="/data/personnel.txt' / MOHIRED YRHIRED DEPT SALARY NAME.
FREQUENCIES VARIABLES=DEPT.
```

- WRITE writes values for each variable on the variable list to file *personnel.txt*. The FREQUENCIES procedure reads the data and causes WRITE to be executed.
- All variables are written with their dictionary formats.

Writing All Variables

```
WRITE OUTFILE='/data/personnel.txt' /ALL.
EXECUTE.
```

- WRITE writes values for all user-defined variables in the active dataset to file *personnel.txt*. The EXECUTE command executes WRITE.

Formats

By default, WRITE uses the dictionary write formats. You can specify formats for some or all variables specified on WRITE. For a string variable, the specified format must have the same width as that of the dictionary format.

- Format specifications can be either column-style or FORTRAN-like (see DATA LIST). The column location specified with column-style formats or implied with FORTRAN-like formats refers to the column in which the variable will be written.
- A format specification following a list of variables applies to all the variables in the list. Use an asterisk to prevent the specified format from applying to variables preceding the asterisk. The specification of column locations implies a default print format, and that format will apply to all previous variables if no asterisk is used.

- All available formats can be specified on WRITE. Note that hex and binary formats use different widths. For example, the AHEX format must have a width twice that of the corresponding A format. For more information on specifying formats and on the formats available, see DATA LIST and .
- Format specifications are in effect only for the WRITE command. They do not change the dictionary write formats.
- To specify a blank between variables in the output, use a string (see), specify blank columns in the format, or use an X or T format element in the WRITE specifications (see DATA LIST for information on X and T).

Example

```
WRITE OUTFILE='/data/personnel.txt' / TENURE (F2.0) ' ' MOHIRED YRHIRED DEPT *
        SALARY85 TO SALARY88 (4(DOLLAR8,1X)) NAME.
EXECUTE.
```

- Format F2.0 is specified for *TENURE*. A blank between quotes is specified as a string after *TENURE* to separate values of *TENURE* from those of *MOHIRED*.
- *MOHIRED, YRHIRED,* and *DEPT* are written with default formats because the asterisk prevents them from receiving the DOLLAR8 format specified for *SALARY85* to *SALARY88*. The 1X format element is specified with DOLLAR8 to add one blank after each value of *SALARY85* to *SALARY88*.
- *NAME* uses the default dictionary format.

Strings

You can specify strings within the variable list. Strings must be enclosed in quotes.

- If a format is specified for a variable list, the application of the format is interrupted by a specified string. Thus, the string has the same effect within a variable list as an asterisk.

Example

```
WRITE OUTFILE='/data/personnel.txt'
  /EMPLOYID '1' MOHIRED YRHIRED SEX AGE JOBCAT NAME
  /EMPLOYID '2' DEPT86 TO DEPT88 SALARY86 TO SALARY88.
EXECUTE.
```

- Strings are used to assign the constant 1 to record 1 of each case, and 2 to record 2 to provide record identifiers in addition to the case identifier *EMPLOYID*.

RECORDS Subcommand

RECORDS indicates the total number of lines written per case. The number specified on RECORDS is informational only. The actual specification that causes variables to be written on a new line is a slash within the variable specifications. Each new line is requested by another slash.

- RECORDS must be specified before the slash that precedes the start of the variable specifications.
- The only specification on RECORDS is an integer to indicate the number of records for the output. If the number does not agree with the actual number of records indicated by slashes, the program issues a warning and ignores the specification on RECORDS.
- Specifications for each line of output must begin with a slash. An integer can follow the slash, indicating the line on which values are to be written. The integer is informational only. It cannot be used to rearrange the order of records in the output. If the integer does not agree with the actual record number indicated by the number of slashes in the variable specifications, the integer is ignored.
- A slash that is not followed by a variable list generates a blank line in the output.

Examples

```
WRITE OUTFILE='/data/personnel.txt' RECORDS=2
  /EMPLOYID NAME DEPT
```

```
  /EMPLOYID TENURE SALARY.
EXECUTE.
```

- WRITE writes the values of an individual's name and department on one line, tenure and salary on the next line, and the employee identification number on both lines.

```
WRITE OUTFILE='/data/personnel.txt' RECORDS=2
  /1 EMPLOYID NAME DEPT
  /2 EMPLOYID TENURE SALARY.
EXECUTE.
```

- This command is equivalent to the command in the preceding example.

```
WRITE OUTFILE='/data/personnel.txt' / EMPLOYID NAME DEPT / EMPLOYID TENURE SALARY.
EXECUTE.
```

- This command is equivalent to the commands in both preceding examples.

OUTFILE Subcommand

OUTFILE specifies the target file for the output from the WRITE command. By default, the output is included with the rest of the output from the session.

- OUTFILE must be specified before the slash that precedes the start of the variable specifications.

Example

```
WRITE OUTFILE='/data/writeout.txt'
   /1 EMPLOYID DEPT SALARY /2 NAME.
EXECUTE.
```

- OUTFILE specifies *writeout.txt* as the file that receives the WRITE output.

ENCODING Subcommand

ENCODING specifies the encoding format of the file. The keyword is followed by an equals sign and a quoted encoding specification.

- In Unicode mode, the default is UTF8. For more information, see SET command, UNICODE subcommand.
- In code page mode, the default is the current locale setting. For more information, see SET command, LOCALE subcommand.
- The quoted encoding value can be: Locale (the current locale setting), UTF8, UTF16, UTF16BE (big endian), UTF16LE (little endian), a numeric Windows code page value (for example, '1252'), or an IANA code page value (for example, 'iso8859-1' or cp1252).
- If there is no OUTFILE subcommand, the ENCODING subcommand is ignored.

BOM Keyword

By default, files encoded in any of the UTF formats include a byte order mark (BOM). Some applications cannot interpret the byte order mark. You can use the BOM keyword to suppress the byte order mark.

BOM=YES
Include the byte order mark in UTF files. This option is the default.

BOM=NO
No not include the byte order mark in UTF files.

TABLE Subcommand

TABLE requests a table showing how the variable information is formatted. NOTABLE is the default.

- TABLE must be specified before the slash that precedes the start of the variable specifications.

Example

```
WRITE OUTFILE='/data/personnel.txt' TABLE
  /1 EMPLOYID DEPT SALARY /2  NAME.
EXECUTE.
```

- TABLE requests a summary table describing the WRITE specifications.

WRITE FORMATS

```
WRITE FORMATS varlist (format) [varlist...]
```

This command takes effect immediately. It does not read the active dataset or execute pending transformations. See the topic “Command Order” on page 36 for more information.

Example

```
WRITE FORMATS SALARY (DOLLAR8)
            / HOURLY (DOLLAR7.2)
            / RAISE BONUS (PCT2).
```

Overview

WRITE FORMATS changes variable write formats. Write formats are output formats and control the form in which values are written by the WRITE command.

WRITE FORMATS changes only write formats. To change print formats, use the PRINT FORMATS command. To change both the print and write formats with a single specification, use the FORMATS command. For information on assigning input formats during data definition, see DATA LIST. For detailed information on available formats and specifications, see “Variable Types and Formats” on page 46.

Basic Specification

The basic specification is a variable list followed by the new format specification in parentheses. All specified variables receive the new format.

Syntax Rules

- You can specify more than one variable or variable list, followed by a format in parentheses. Only one format can be specified after each variable list. For clarity, each set of specifications can be separated by a slash.
- You can use keyword TO to refer to consecutive variables in the active dataset.
- The specified width of a format must include enough positions to accommodate any punctuation characters such as decimal points, commas, dollar signs, or date and time delimiters. (This differs from assigning an *input* format on DATA LIST, where the program automatically expands the input format to accommodate punctuation characters in output.)
- Custom currency formats (CCw, CCw.d) must first be defined on the SET command before they can be used on WRITE FORMATS.
- For string variables, you can only use WRITE FORMATS to switch between A and AHEX formats. WRITE FORMATS cannot be used to change the length of string variables. To change the length of a string variable, use the ALTER TYPE command.

Operations

- Unlike most transformations, WRITE FORMATS takes effect as soon as it is encountered in the command sequence. Special attention should be paid to its position among commands. See the topic “Command Order” on page 36 for more information.
- Variables not specified on WRITE FORMATS retain their current formats in the active dataset. To see the current formats, use the DISPLAY command.
- The new write formats are changed only in the active dataset and are in effect for the duration of the session or until changed again with a WRITE FORMATS or FORMATS command. Write formats in the original data file (if one exists) are not changed, unless the file is resaved with the SAVE or XSAVE command.

- New numeric variables created with transformation commands are assigned default print and write formats of F8.2 (or the format specified on the FORMAT subcommand of SET). The WRITE FORMATS command can be used to change the new variable's write formats.
- New string variables created with transformation commands are assigned the format specified on the STRING command that declares the variable. WRITE FORMATS cannot be used to change the format of a new string variable.
- Date and time formats are effective only with the LIST and TABLES procedures and the PRINT and WRITE transformation commands. All other procedures use F format regardless of the date and time formats specified. See the topic "Date and Time Formats" on page 53 for more information.
- If a numeric data value exceeds its width specification, the program attempts to write some value nevertheless. First the program rounds decimal values, then removes punctuation characters, then tries scientific notation, and finally, if there is still not enough space, produces asterisks indicating that a value is present but cannot be written in the assigned width.

Examples

Specifying Write Formats for Multiple Variables

```
WRITE FORMATS SALARY (DOLLAR8)
            / HOURLY (DOLLAR7.2)
            / RAISE BONUS (PCT2).
```

- The write format for *SALARY* is changed to DOLLAR with eight positions, including the dollar sign and comma when appropriate. An eight-digit number would require a DOLLAR11 format specification: eight characters for the digits, two characters for commas, and one character for the dollar sign.
- The write format for *HOURLY* is changed to DOLLAR with seven positions, including the dollar sign, decimal point, and two decimal places.
- The write format for both *RAISE* and *BONUS* is changed to PCT with two positions: one for the percentage and one for the percent sign.

Changing the Default Format of a New Variable

```
COMPUTE V3=V1 + V2.
WRITE FORMATS V3 (F3.1).
```

- COMPUTE creates the new numeric variable *V3*. By default, *V3* is assigned an F8.2 format.
- WRITE FORMATS changes the write format for *V3* to F3.1.

Working With Custom Currency Formats

```
SET CCA='-/-.Dfl ..-'.
WRITE FORMATS COST (CCA14.2).
```

- SET defines a European currency format for the custom currency format type CCA.
- WRITE FORMATS assigns the write format CCA to variable *COST*. See the SET command for more information on custom currency formats.

XGRAPH

XGRAPH is available in the Statistics Base Edition.

XGRAPH is available only on systems with high-resolution graphics capabilities.

Note: Square brackets used in the XGRAPH syntax chart are required parts of the syntax and are not used to indicate optional elements. Any equals signs (=) displayed in the syntax chart are required. All subcommands except the chart expression in the first line are optional.

```
XGRAPH CHART=yvars BY xvars BY zvars

  /BIN START={AUTO**} SIZE={AUTO**   }
            {x     }      {WIDTH (x)}
                          {COUNT (n)}

  /DISPLAY DOT={ASYMMETRIC**}
              {SYMMETRIC   }
              {FLAT        }

  /DISTRIBUTION TYPE=NORMAL

  /COORDINATE SPLIT={NO**}
                    {YES }

  /ERRORBAR {CI     {(95)}}
            {(n)  }
           {STDDEV {(2) }}
            {(n)  }
           {SE     {(2) }}
            {(n)  }

  /MISSING USE={LISTWISE**  } REPORT={NO**}
              {VARIABLEWISE}        {YES }

  /PANEL COLVAR=varlist COLOP={CROSS**} ROWVAR=varlist ROWOP={CROSS**}
                             {NEST   }                      {NEST   }

  /TEMPLATE FILE='filespec'

  /TITLES TITLE='line' 'line2' SUBTITLE='line1'
          FOOTNOTE='line1' 'line2'
```

** Default if the subcommand is omitted.

`yvars` defines the *y* dimension and has the general form:

```
(varname [function] [data_element_type] + ...) > {varname {[COLOR]  }}
                                                 {[PATTERN]}
                                            {1       {[COLOR]  }}
                                                     {[PATTERN]}
```

`xvars` and `zvars` define the *x* and *z* dimensions and have the general form:

```
{varname {[c]}              } > varname {[COLOR]  }
         {[s]}                          {[PATTERN]}
{$CASENUM [LABEL=varname]   }
{1                          }
```

The + operator blends the *y*-axis to accommodate all the *y*-axis variables. It is used when separate variables are displayed on the same axis. The number 1 is a placeholder for the variables on the axis. The > operator stacks data elements on the *y*-axis and clusters data elements on the *x*-axis and *z*-axis. `[c]` or `[s]` indicates whether the variable should be treated as categorical or scale. `$CASENUM` is used when individual cases are displayed on the same axis.

Summary functions available for `yvars`: `VALUE`, `COUNT`, `PCT`, `CUPCT`, `CUFREQ`, `MINIMUM`, `MAXIMUM`, `VALIDN`, `SUM`, `CSUM`, `MEAN`, `STDDEV`, `VARIANCE`, `MEDIAN`, `GMEDIAN`, `MODE`, `PTILE(x)`, `GPTILE(x)`, `PLT(x)`, `PGT(x)`, `NLT(x)`, `NGT(x)`, `PIN(x1,x2)`, and `NIN(x1,x2)`.

Data element types available for `yvars`: `BAR`, `HISTOBAR`, and `POINT`.

This command reads the active dataset and causes execution of any pending commands. See the topic “Command Order” on page 36 for more information.

Release History

Release 13.0

- Command introduced.

Example

```
XGRAPH CHART=([COUNT] [BAR]) BY jobcat [c] BY gender [c].
```

Overview

XGRAPH generates a high-resolution chart by computing statistics from variables in the active dataset and constructing the chart according to your specification. The chart can be a 3-D bar chart, population pyramid, or dot plot. The chart is displayed where high-resolution display is available and can be edited with the Chart Editor and saved in an Output file.

Note: Although you can create other charts with XGRAPH, these are not currently supported.

Basic Specification

The basic specification is the command name followed by a chart expression. The chart expression must identify one or more variables and a data element for the chart.

Syntax Rules

- The chart expression is required and must appear first. It can optionally be preceded by CHART=. If CHART is used, the = must be present. All other subcommands are optional.
- After the chart expression, subcommands can appear in any order.
- Only a single instance of each subcommand is allowed.
- Subcommand names and keywords must be spelled in full.
- Required square brackets cannot be merged for different types of specifications. For example, when a summary function and a data element are adjacent, they cannot be merged. That is, `[VALUE] [BAR]` cannot be written as `[VALUE BAR]`.

Limitations

- XGRAPH does not produce charts with more than 1,000,000 data values.
- When a weight is in effect and the ERRORBAR subcommand is used on a chart that displays medians, counts, or percentages, the case weights must be integers.

CHART Expression

The **chart expression** defines the data and the role of that data in the chart. It also specifies the data element type and any clustering and stacking. For clustered and stacked charts, the chart expression specifies whether color or patterns are used to distinguish the clustered and stacked groups. For charts of individual cases (that is, when $CASENUM defines a categorical axis), the chart expression can also specify a label variable.

- The CHART keyword is optional. If it is used, it must be followed by an equals sign (=).
- The order within the chart expression matters. The `yvars` (the *y* dimension) must be first, followed by `xvars` and then `zvars`.
- The final BY variable is used as the splitter variable in a population pyramid when the COORDINATE subcommand specifies SPLIT.
- There is a limit to the number of stack and cluster specifications. There can be two clusters, or one stack and one cluster.

Functions

Functions must be enclosed in square brackets.

Value function:

The VALUE function yields the value of the specified *y*-axis variable for each case. It always produces one data element for each case.

Aggregation functions:

Two groups of aggregation functions are available: count functions and summary functions.

Count functions:

COUNT. *Frequency of cases in each category.*

PCT. *Frequency of cases in each category expressed as a percentage of the whole.*

CUPCT. *Cumulative percentage sorted by category value.*

CUFREQ. *Cumulative frequency sorted by category value.*

- Count functions yield the count or percentage of valid cases within categories determined by the `xvars` and the `zvars`, as in:

  ```
  XGRAPH ([PCT] [BAR]) BY jobcat [c] BY gender [c].
  ```
- Count functions do not operate on a variable.
- Count functions do not have any arguments.

Summary functions:

MINIMUM. *Minimum value of the variable.*

MAXIMUM. *Maximum value of the variable.*

VALIDN. *Number of cases for which the variable has a nonmissing value.*

SUM. *Sum of the values of the variable.*

CUSUM. *Sum of the summary variable accumulated across values of the category variable.*

MEAN. *Mean.*

STDDEV. *Standard deviation.*

VARIANCE. *Variance.*

MEDIAN. *Median.*

GMEDIAN. *Group median.*

MODE. *Mode.*

PTILE(x). X*th percentile value of the variable. X* must be greater than 0 and less than 100.

PLT(x). *Percentage of cases for which the value of the variable is less than* x.

PGT(x). *Percentage of cases for which the value of the variable is greater than* x.

NLT(x). *Number of cases for which the value of the variable is less than* x.

NGT(x). *Number of cases for which the value of the variable is greater than* x.

PIN(x1,x2) . *Percentage of cases for which the value of the variable is greater than or equal to* x1 *and less than or equal to* x2. *x1* cannot exceed *x2*.

NIN(x1,x2) . *Number of cases for which the value of the variable is greater than or equal to* x1 *and less than or equal to* x2. *x1* cannot exceed *x2*.

- Summary functions operate on summary variables (variables that record continuous values, like age or expenses). To use a summary function, specify the name of one or more variables before the name of the function and the data element type, both of which are in square brackets, as in:

  ```
  XGRAPH (salary [SUM] [BAR]) BY jobcat[c] BY gender[c].
  ```
- You can specify a list of variables on which the function operates, as in:

  ```
  XGRAPH ((salary + salbegin) [SUM] [BAR]) BY jobcat[c] BY gender[c].
  ```

This syntax is equivalent to:

```
XGRAPH (salary [SUM] [BAR] + salbegin [SUM] [BAR]) BY jobcat[c] BY gender[c].
```

Data Element Types

The data element type is a basic description of how the data are represented on the chart.

BAR. *Draws a bar data element.* Bars show the results of a summary function. Used for 3-D bar charts and population pyramids that show the distribution of a categorical variable.

HISTOBAR. *Draws a histogram data element.* Histogram data elements show the counts in a range of cases. A range of cases is called a bin. Used for population pyramids.

POINT. *Draws a point marker.* A point marker shows a case with a specific *x*-axis value. Used for dot plots.

Measurement Level

The chart expression allows the user to specify a measurement level for BY variables. If measurement level is not specified, the following occurs:

- In a chart of BAR elements, `xvars` and `zvars` are treated as categorical.
- In a chart of HISTOBAR elements, `xvars` is treated as scale. `zvars` is treated as categorical, because the COORDINATE subcommand and SPLIT keyword are always used.
- In charts of POINT elements, the measurement level is obtained from the IBM SPSS Statistics data dictionary for each variable. If no measurement levels have been defined, numeric variables are treated as scale and string variables are treated as categorical.

[c]. *Specifies that the variable is treated as a categorical variable.*

[s]. *Specifies that the variable is treated as a scale variable.*

Variable Placeholder

When summaries of separate variables are used to define a categorical axis, clustering, or stacking, 1 must be used as a variable placeholder.

1. *Uses the separate variables specified in yvars as categories on the axis, cluster categories, or stack categories. Must be defined as an integer (no decimal places).*

Examples

```
XGRAPH CHART=((salary + salbegin) [MEAN] [BAR]) BY 1 BY gender[c].
```

- Separate variables appear on the *y*-axis.

```
XGRAPH CHART=((salary + salbegin) [MEAN] [BAR]) BY gender[c] BY 1.
```

- Separate variables appear on the *z*-axis.

Case Numbers

Whenever the VALUE function is used, one categorical axis consists of cases, which are identified by a sequential number.

$CASENUM. *Uses the sequential numbering of selected cases as the categories on the axis.* This is not the actual case number.

Example

```
XGRAPH CHART=(salary [VALUE]) BY gender[c] BY $CASENUM.
```

Blending, Clustering, and Stacking

Blending on an axis is indicated by +. This is used when separate variables are displayed on the same axis.

Clustering and stacking are indicated by >. On the *x*- or *z*-axis, this indicates clustering. On the *y*-axis, this indicates stacking. Stacking is allowed with one clustering variable; or two clustering variables are allowed, one on the *x*-axis and one on the *z*-axis.

+. *Scales the axis to accommodate all of the variables on that dimension.*

>. *Stacks groups on the y-axis, and clusters groups on the x- and z-axes.*

Stacking Example

```
XGRAPH CHART=(salary [MEAN] [BAR]) > educ BY jobcat[c] by gender[c].
```

Clustering Example

```
XGRAPH CHART=(salary [MEAN] [BAR]) BY jobcat[c] > educ by gender[c].
```

When you use clustering or stacking, you can specify how the clusters and stacks are differentiated in the chart.

[COLOR]. *Uses color to differentiate the clusters or stacks.*

[PATTERN]. *Uses pattern to differentiate the clusters or stacks.*

[COLOR] and [PATTERN] follow these rules:

- The actual colors and patterns are specified in the Options. You can access these by choosing **Options** from the Edit menu. Then click the **Charts** tab.
- [COLOR] and [PATTERN] follow the stacking or clustering variable in the chart expression.
- If the specification is omitted, color is used to differentiate the clusters or stacks. The color cycle starts over for each dimension.
- If neither [COLOR] nor [PATTERN] is specified, XGRAPH uses the Style Cycle Preference specified by the Options.
- If [COLOR] or [PATTERN] is specified for both dimensions, these specifications are honored. The color or pattern cycle starts over for each dimension.
- If [COLOR] or [PATTERN] is specified for only one dimension, the specification is honored, and the other dimension uses the other specification.

Example

```
XGRAPH CHART=(salary [MEAN] [BAR]) BY jobcat[c] > educ[PATTERN]
by gender[c].
```

Labels

On categorical charts of individual cases, you can specify a labeling variable by using [LABEL=varname] in the chart expression. [LABEL=varname] follows $CASENUM in the chart expression.

[LABEL=varname]. *The variable whose values are used as the tick labels instead of the case number.*

Example

```
XGRAPH CHART=(salary [VALUE] [BAR]) BY $CASENUM[LABEL=gender]
> jobcat BY educ.
```

BIN Subcommand

For population pyramids, the BIN subcommand controls the starting point and size of the bins. The bins are the bars in the chart and represent a group of data values. For example, if there are four bins in a chart

and the values range from 0-100, the bin width is 25. Therefore, all points with values of 0-24 would be in the first bin, those with values of 25-49 would be in the second bin, and so on. Each keyword is followed by an equals sign (=) and the value for that keyword.

Example

```
XGRAPH CHART=([HISTOBAR]) BY age[s] BY gender[c]
  /COORDINATE SPLIT=YES
  /BIN START=30 SIZE=WIDTH(5).
```

- The first bin begins at age 30.
- Each bin includes cases with *age* values that span 5 years.
- So the first bin contains the cases with ages 30-34, the second bin contains the cases with ages 35-39, and so on.

START Keyword

The START keyword specifies the starting value of the first bin in the chart.

AUTO. *Adjust the starting value of the first bin according to the data values.* The starting value is set so the first bin includes the lowest data value and the bin boundaries are at good values. This is the default.

value. *User-specified starting value.* The value can be set lower than the lowest data value so the first bin includes values that are not in the dataset. For example, if the lowest value in the dataset is 6, the starting value can be set to 0 so the values of 0–5 are included in the first bin.

Note: If the variable being binned is a date, the starting value must be a data literal in quotes and in the date format specified for the variable in the IBM SPSS Statistics data dictionary (for example, `'January 1, 2001'`).

SIZE Keyword

The SIZE keyword specifies the width of the bins. The width can be determined by an exact value or by the number of bins (fewer bins result in wider bins).

AUTO. *Adjust bin width according to the data values.* The size is set so the bins are neither too few nor too narrow. This is the default.

WIDTH (value). *User-specified value for the bin width. Note*: If the variable being binned is a date, the width must be specified in quotes and in the duration format `ddd hh:mm` (for example, `'30 12:00'` for bin widths of 30 days and 12 hours). The `hh:mm` portion of the duration format is optional.

COUNT (value). *User-specified value for the number of bins.*

DISPLAY Subcommand

DISPLAY controls how the data elements are displayed. Currently this subcommand applies to dot plots.

DOT Keyword

The DOT keyword controls the display of points in a dot plot.

ASYMMETRIC. *Stack the points on the x-axis.* This is the default.

SYMMETRIC. *Arrange the points symmetrically across a central horizontal line in the chart.* In other words, the point stacks are vertically centered.

FLAT. *Do not stack the points.*

DISTRIBUTION Subcommand

The DISTRIBUTION subcommand adds distribution curves to the chart. This subcommand applies to population pyramids.

Example

```
XGRAPH CHART=([HISTOBAR]) BY age[s] BY gender[c]
  /DISTRIBUTION TYPE=NORMAL
  /COORDINATE SPLIT=YES.
```

TYPE Keyword

The TYPE keyword specifies the type of distribution to draw on the chart. The keyword is followed by an equals sign (=) and the value for that keyword.

NORMAL. *Display a normal curve on the chart.* The normal curve uses the data mean and standard deviation as parameters. This is the default.

COORDINATE Subcommand

The COORDINATE subcommand specifies the coordinate system for the chart. Currently this subcommand is used to create population pyramids.

SPLIT Keyword

The SPLIT keyword specifies whether zvars (the variable after the last BY in the chart expression) is used to split the chart and create a population pyramid. The keyword is followed by an equals sign (=) and the value for that keyword.

Note: The SPLIT keyword is not the same as the PANEL subcommand. The keyword splits the chart before it is paneled.

NO. *Do not split the chart.* This is the default.

YES. *Split the chart and create a population pyramid.*

ERRORBAR Subcommand

The ERRORBAR subcommand adds errors bars to the chart. Error bars indicate the variability of the summary statistic being displayed. The length of the error bar on either side of the summary statistic represents a confidence interval or a specified number of standard errors or standard deviations. Currently, XGRAPH supports error bars for COUNT and population pyramids.

The keywords are not followed by an equals sign (=). They are followed by a value in parentheses.

Example

```
XGRAPH CHART=([COUNT] [BAR]) BY agecat[c] BY gender[c]
  /COORDINATE SPLIT=YES
  /ERRORBAR CI(95).
```

CI Keyword

(value). *The percentage of the confidence interval to use as the length of the error bars.*

STDDEV Keyword

(value). *A multiplier indicating the number of standard deviations to use as the length of the error bars.*

SE Keyword

(value). *A multiplier indicating the number of standard errors to use as the length of the error bars.*

MISSING Subcommand

MISSING controls the treatment of missing values in the chart drawn by XGRAPH.

- For an aggregated categorical chart, if every aggregated series is empty in a category, the empty category is excluded.

Each keyword in the subcommand is followed by an equals sign (=) and the value for that keyword.

USE Keyword

The USE keyword controls the exclusion of cases with missing values. Excluded cases are excluded from computations and charts. USE applies to variables used in summary functions for a chart.

LISTWISE. *Exclude a case with a missing value for any dependent variable.* This is the default.

VARIABLEWISE. *Exclude a case with a missing value for the dependent variable being analyzed.*

REPORT Keyword

The REPORT keyword specifies whether to report on missing values by creating missing-values categories in the chart. REPORT applies only to categorical variables, including paneling variables and the splitter variable in a population pyramid.

NO. *Do not create a missing-values categories.* This is the default.

YES. *Create a missing-values categories.*

PANEL Subcommand

The PANEL subcommand specifies the variables and method used for paneling.

Each keyword in the subcommand is followed by an equals sign (=) and the value for that keyword.

COLVAR and ROWVAR Keywords

The COLVAR and ROWVAR keywords identify the column and row variables, respectively. Each category in a column variable appears as a vertical column in the resulting chart. Each category in a row variable appears as a horizontal row in the resulting chart.

- If multiple variables are specified for a keyword, the COLOP and ROWOP keywords can be used to change the way in which variable categories are rendered in the chart.
- The ROWVAR keyword is not available for population pyramids.

varlist. *The list of variables used for paneling.*

Examples

```
XGRAPH CHART=([COUNT] [BAR]) BY educ [c] BY gender [c]
  /PANEL COLVAR=minority.
```

- There are two columns in the resulting paneled chart, one for minorities and one for non-minorities.
- Because there is only one paneling variable, there are only as many panels as there are variable values. Therefore, there are two panels.

```
XGRAPH CHART=([COUNT] [BAR]) BY educ [c] BY gender [c]
  /PANEL COLVAR=minority ROWVAR=jobcat.
```

- There are two columns in the resulting paneled chart (for the *minority* variable values) and three rows (for the *jobcat* variable values).

COLOP and ROWOP Keywords

The COLOP and ROWOP keywords specify the paneling method for the column and row variables, respectively. These keywords have no effect on the chart if there is only one variable in the rows and/or columns. They also have no effect if the data are not nested.

CROSS. *Cross variables in the rows or columns.* When the variables are crossed, a panel is created for every combination of categories in the variables. For example, if the categories in one variable are A and B and the categories in another variable are 1 and 2, the resulting chart will display a panel for the combinations of A and 1; A and 2; B and 1; and B and 2. A panel can be empty if the categories in that panel do not cross (for example, if there are no cases in the B category *and* the 1 category). This is the default.

NEST. *Nest variables in the rows or columns.* When the variables are nested, a panel is created for each category that is nested in the parent category. For example, if the data contain variables for states and cities, a panel is created for each city and the relevant state. However, panels are not created for cities that are not in certain states, as would happen with CROSS. See the following example. When nesting, make sure the variables specified for ROWVAR or COLVAR are in the correct order. Parent variables precede the child variables.

Example

Assume data like the following data:

Table 118. Nested data

state	city	temperature
NJ	Springfield	70
MA	Springfield	60
IL	Springfield	50
NJ	Trenton	70
MA	Boston	60

You can create a paneled chart from this data with the following syntax:

```
XGRAPH ([POINT]) BY temperature
  /PANEL COLVAR=state city COLOP=CROSS.
```

The command crosses every variable value to create the panels. Because not every state contains every city, the resulting paneled chart will contain blank panels. For example, there will be a blank panel for Springfield and New Jersey. In this dataset, the *city* variable is really nested in the *state* variable. To nest the variables in the panels and eliminate any blank panels, use the following syntax:

```
XGRAPH ([POINT]) BY temperature
  /PANEL COLVAR=state city COLOP=NEST.
```

TEMPLATE Subcommand

TEMPLATE uses an existing file as a template and applies it to the chart requested by the current XGRAPH command. The template overrides the default settings that are used to create any chart, and the specifications on the current XGRAPH command override the template. Templates are created in the Chart Editor by saving an existing chart as a template.

Example

```
XGRAPH CHART=([COUNT] [BAR]) BY jobcat [c] BY gender [c].
  /TEMPLATE FILE='mytemplate.sgt'.
```

FILE Keyword

The FILE keyword specifies the template file. The keyword is followed by an equals sign (=) and a file specification enclosed in quotes.

filespec. *Applies the specified template file to the chart being created.*

TITLES Subcommand

The TITLES subcommand specifies the titles, subtitles, and footnotes. These are lines of text at the top or bottom of the chart.

- At least one keyword should be specified if the TITLES subcommand is used; otherwise the subcommand is treated as blank.
- If the subcommand is blank, an error is issued.
- Each line of text must be enclosed in quotes. The maximum length of any line is 255 bytes.
- When a keyword is specified with no text, an error is issued.

The following symbols can be used within the lines of text. Each must be specified using an opening right parenthesis and uppercase letters.

)DATE. *Displays the current date as a locale-appropriate date stamp that includes the year, month, and day.*

)TIME. *Displays the current time as a locale-appropriate time stamp.*

)CHART. *Displays the chart expression used to create the chart, stripped of measurement levels, statistics specifications, and* CHART=. If variable labels are available, they are used instead of the variable names in the chart expression.

Example

```
XGRAPH CHART=([COUNT] [BAR]) BY jobcat [c] BY gender [c]
  /TITLES TITLE='Counts by Job Category and Gender' 'Date: )DATE'.
```

TITLE Keyword

'line1' 'line2'. *The lines of text in the title.* The title appears above the chart. Up to two lines can be specified.

SUBTITLE Keyword

'line1'. *The line of text in the subtitle.* The subtitle appears below the title. Only one line can be specified.

FOOTNOTE Keyword

'line1' 'line2'. *The lines of text in the footnote.* The footnote appears below the chart. Up to two lines can be specified.

3-D Bar Examples

A 3-D bar chart is a bar chart with two categorical axes (*x* and *z*). The height of the bars is determined by a summary function. The categories on the *x*- and *z*-axes can be values of categorical variables, separate variables, or individual cases.

Examples of Summaries for Groups of Cases

```
XGRAPH CHART=([COUNT] [BAR]) BY jobcat[c] by gender[c].
```

- The bars in the chart represent counts for males and females in each job category.

```
XGRAPH CHART=(salary [MEAN] [BAR])> educ[PATTERN] BY jobcat[c] >
minority[COLOR] by gender[c].
```

- The bars in the chart represent the mean salary for males and females in each job category clustered by minority membership.
- The bars are also stacked by education level.
- Clusters are distinguished by color, and stacks are distinguished by pattern.

Example of Summaries of Separate Variables by Group

```
XGRAPH CHART=((salary + salbegin) [MEAN] [BAR]) BY 1 BY gender[c].
```

- The bars in the chart represent the mean current salary and mean beginning salary for males and females.
- The blending operator (+) indicates that the chart is for separate variables.
- The 1 placeholder indicates the axis on which the separate variables appear.
- The variables appear on the *x*-axis, and the gender categories are on the *z*-axis.

Examples of Individual Cases in Groups

```
XGRAPH CHART=(salary [VALUE] [BAR]) BY gender[c] BY $CASENUM.
```

- The bars in the chart represent the current salary for each case with the cases grouped by gender.
- VALUE is the only function available when creating charts of individual cases.

Examples for Values of Individual Cases for Separate Variables

```
XGRAPH CHART=((salary + salbegin) [VALUE] [BAR]) BY $CASENUM BY 1.
```

- The bars in the chart represent the value of the current salary and the value of the beginning salary for every case.
- The cases are on the *x*-axis, and the variables are on the *z*-axis.

```
XGRAPH CHART=((salary + salbegin) [VALUE] [BAR]) BY
$CASENUM[LABEL=gender] BY 1.
```

- This is the same as the previous example except that the bars are labeled by gender.

Population Pyramid Examples

A population pyramid uses bars to show the shape of a distribution split by a categorical variable. The variable used for the distribution can be a scale or categorical. When the variable is scale, the data element type must be HISTOBAR. When the variable is categorical, the data element type is BAR.

Example Using Continuous Data

```
XGRAPH CHART=([HISTOBAR]) BY age[s] BY gender[c]
  /COORDINATE SPLIT=YES.
```

- The bars in the chart represent the number of cases in each *age* bin.
- The population pyramid is split by gender.

Example Using Categorical Data

```
XGRAPH CHART=([COUNT] [BAR]) BY agecat[c] BY gender[c]
  /COORDINATE SPLIT=YES.
```

- The *agecat* variable has categories for different age groups.
- The bars in the chart represent the number of cases in each *agecat* category. This is not necessarily the same as the previous example because the size of the bins and the size of the categories might not match.

Example Using Pre-Aggregated Data

```
XGRAPH CHART=(population [SUM] [BAR]) BY agecat[c] BY gender[c]
  /COORDINATE SPLIT=YES.
```

- The bars in the chart represent the number of cases in each *agecat* category.
- The data include a *population* variable that specifies the count for each *agecat* category for each gender. In other words, the data are pre-aggregated.

Example Using Pre-Aggregated Data Without a Categorical Splitter Variable

In the previous example, there was a categorical variable to use as the splitter variable. In some census data, the data are not organized in this manner. Instead of a splitter variable, there is one variable for each split value. An example follows:

Table 119. Pre-aggregated data without a categorical splitter variable

agecat	malepop	femalepop
1	247	228
2	306	300
3	311	303

Data in this format needs to be restructured by using the VARST0CASES command:

```
VARSTOCASES /ID = id
  /MAKE population FROM malepop femalepop
  /INDEX = gender(2)
  /KEEP = agecat
  /NULL = KEEP.
```

Running this command results in the following. The splitter variable is *gender*.

Table 120. Pre-aggregated data with a categorical splitter variable

id	agecat	gender	population
1	1	1	247
1	1	2	228
2	2	1	306
2	2	2	300
3	3	1	311
3	3	2	303

Now a population pyramid can be created using the categorical splitter variable:

```
XGRAPH (population [SUM] [BAR]) BY agecat[c] BY gender[c]
  /COORDINATE SPLIT=YES.
```

Dot Plot Examples

A dot plot shows the value in one dimension (the *x*-axis) of each selected case. The variable on the *x*-axis can be scale or categorical.

```
XGRAPH CHART=([POINT]) BY jobcat[c].
```

- The chart shows cases represented as points.
- Cases are stacked at each *jobcat* value. So, if there are three categories for `jobcat`, there are only three stacks.

```
XGRAPH CHART=([POINT]) BY salary[s].
```

- Cases are stacked at each *salary* value. The *x*-axis is a continuous range of values.

```
XGRAPH CHART=([POINT]) BY salary[s]
  /DISPLAY DOT=SYMMETRIC.
```

- Same as the previous example except the stacked dots are centered vertically in the chart.

XSAVE

```
XSAVE OUTFILE='filespec'

 [/KEEP={ALL**  }] [/DROP=varlist]
        {varlist}

 [/RENAME=(old varlist=new varlist)...]

 [/MAP] [/{COMPRESSED  }]
          {UNCOMPRESSED}
          {ZCOMPRESSED }

 [/PERMISSIONS={READONLY }
               {WRITEABLE}
```

**Default if the subcommand is omitted.

This command does not read the active dataset. It is stored, pending execution with the next command that reads the dataset. See the topic "Command Order" on page 36 for more information.

Release history

Release 21.0

- ZCOMPRESSED subcommand added.

Example

```
XSAVE OUTFILE='/data/empl.sav'.
```

Overview

XSAVE produces IBM SPSS Statistics data files. IBM SPSS Statistics data files contain data plus a dictionary. The dictionary contains a name for each variable in the data file plus any assigned variable and value labels, missing-value flags, and variable print and write formats. The dictionary also contains document text created with the DOCUMENTS command.

SAVE also creates IBM SPSS Statistics data files. The principal difference is that XSAVE is not executed until data are read for the next procedure, while SAVE is executed by itself. Thus, XSAVE can reduce processing time by consolidating two data passes into one.

See SAVE TRANSLATE for information on saving data files that can be used by other programs.

Options

Variable Subsets and Order. You can save a subset of variables and reorder the variables that are saved using the DROP and KEEP subcommands.

Variable Names. You can rename variables as they are copied into the data file using the RENAME subcommand.

Variable Map. To confirm the names and order of the variables saved in the data file, use the MAP subcommand. MAP displays the variables saved in the data file next to their corresponding names in the active dataset.

Data Compression. You can use the COMPRESSED or ZCOMPRESSED subcommand to write the data file in compressed form. You can use the UNCOMPRESSED subcommand to write the data file in an uncompressed form. See the topic "COMPRESSED, UNCOMPRESSED, and ZCOMPRESSED Subcommands " on page 2184 for more information.

Basic Specification

The basic specification is the OUTFILE subcommand, which specifies a name for the data file to be saved.

Subcommand Order

- Subcommands can be specified in any order.

Syntax Rules

- OUTFILE is required and can be specified only once. If OUTFILE is specified more than once, only the last OUTFILE specification is in effect.
- KEEP, DROP, RENAME, and MAP can be used as many times as needed.
- Only one of the subcommands COMPRESSED, UNCOMPRESSED, or ZCOMPRESSED can be specified per XSAVE command.
- Documentary text can be dropped from the active dataset with the DROP DOCUMENTS command.
- XSAVE cannot appear within a DO REPEAT–END REPEAT structure.
- Multiple XSAVE commands writing to the same file are not permitted.

Operations

- Unlike the SAVE command, XSAVE is a transformation command and is executed when the data are read for the next procedure.
- The new data file dictionary is arranged in the same order as the active dataset dictionary unless variables are reordered with the KEEP subcommand. Documentary text from the active dataset dictionary is always saved unless it is dropped with the DROP DOCUMENTS command before XSAVE.
- New variables created by transformations and procedures previous to the XSAVE command are included in the new data file, and variables altered by transformations are saved in their modified form. Results of any temporary transformations immediately preceding the XSAVE command are included in the file; scratch variables are not.
- IBM SPSS Statistics data files are binary files designed to be read and written by IBM SPSS Statistics only.
- The active dataset is still available for transformations and procedures after XSAVE is executed.
- XSAVE processes the dictionary first and displays a message that indicates how many variables will be saved. Once the data are written, XSAVE indicates how many cases were saved. If the second message does not appear, the file was probably not completely written.

Limitations

- Maximum of 64 XSAVE commands are allowed within a single set of transformations.

Examples

Using XSAVE to Consolidate Two Data Passes Into One

```
GET FILE='/data/hubempl.sav'.
XSAVE OUTFILE='/data/empl88.sav'
  /RENAME=(AGE=AGE88) (JOBCAT=JOBCAT88).
MEANS RAISE88 BY DEPT88.
```

- The GET command retrieves the IBM SPSS Statistics data file *hubempl.sav*.
- The RENAME subcommand renames variable *AGE* to *AGE88* and variable *JOBCAT* to *JOBCAT88*.
- XSAVE is not executed until the program reads the data for procedure MEANS. The program saves file *empl88.sav* and generates a MEANS table in a single data pass.
- After MEANS is executed, the *hubempl.sav* file is still the active dataset. Variables *AGE* and *JOBCAT* retain their original names in the active dataset.

Using Temporary Transformations With XSAVE

```
GET FILE=hubempl.sav.
TEMPORARY.
RECODE DEPT85 TO DEPT88 (1,2=1) (3,4=2) (ELSE=9).
VALUE LABELS DEPT85 TO DEPT88 1 'MANAGEMENT' 2 'OPERATIONS' 3 'UNKNOWN'.
XSAVE OUTFILE='/data/hubtemp.sav'.
CROSSTABS DEPT85 TO DEPT88 BY JOBCAT.
```

- Both the saved data file and the CROSSTABS output will reflect the temporary recoding and labeling of the department variables.
- If SAVE were specified instead of XSAVE, the data would be read twice instead of once and the CROSSTABS output would not reflect the recoding.

OUTFILE Subcommand

- OUTFILE specifies the IBM SPSS Statistics data file to be saved. OUTFILE is required and can be specified only once. If OUTFILE is specified more than once, only the last OUTFILE is in effect. The file specification should be enclosed in quotes.
- The OUTFILE specification cannot refer to a dataset in the current session. It must refer to an external IBM SPSS Statistics data file.

DROP and KEEP Subcommands

DROP and KEEP are used to save a subset of variables. DROP specifies the variables not to save in the new data file, while KEEP specifies the variables to save in the new data file; variables not named on KEEP are dropped.

- Variables can be specified in any order. The order of variables on KEEP determines the order of variables in the data file. The order on DROP does not affect the order of variables in the data file.
- Keyword ALL on KEEP refers to all remaining variables not previously specified on KEEP. ALL must be the last specification on KEEP.
- If a variable is specified twice on the same subcommand, only the first mention is recognized.
- Multiple DROP and KEEP subcommands are allowed. Specifying a variable that is not in the active dataset or that has been dropped because of a previous DROP or KEEP subcommand results in an error, and the XSAVE command is not executed.
- Keyword TO can be used to specify a group of consecutive variables in the data file.

Dropping a Range of Variables

```
XSAVE OUTFILE='/data/hubtemp.sav'
  /DROP=DEPT79 TO DEPT84 SALARY79.
CROSSTABS DEPT85 TO DEPT88 BY JOBCAT.
```

- The data file is saved as *hubtemp.sav*. All variables between and including *DEPT79* and *DEPT84*, as well as *SALARY79*, are excluded from the data file. All other variables are saved.

Specifying Variable Order With XSAVE

```
GET FILE='/data/prsnl.sav'.
COMPUTE  TENURE=(12-CMONTH +(12*(88-CYEAR)))/12.
COMPUTE  JTENURE=(12-JMONTH +(12*(88-JYEAR)))/12.
VARIABLE LABELS   TENURE 'Tenure in Company'
                  JTENURE 'Tenure in Grade'.
XSAVE OUTFILE='/data/prsnl88.sav' /DROP=GRADE STORE
 /KEEP=LNAME NAME TENURE JTENURE ALL.
REPORT FORMAT=AUTO /VARS=AGE TENURE JTENURE SALARY
 /BREAK=DIVISION /SUMMARY=MEAN.
```

- Variables *TENURE* and *JTENURE* are created by COMPUTE commands and assigned variable labels by the VARIABLE LABELS command. *TENURE* and *JTENURE* are added to the end of the active dataset.
- DROP excludes variables *GRADE* and *STORE* from file *PRSNL88*. KEEP specifies that *LNAME, NAME, TENURE*, and *JTENURE* are the first four variables in file *prsnl88.sav*, followed by all remaining variables not specified on DROP. These remaining variables are saved in the same sequence as they appear in the original file.

RENAME Subcommand

RENAME changes the names of variables as they are copied into the new data file.

- The specification on RENAME is a list of old variable names followed by an equals sign and a list of new variable names. The same number of variables must be specified on both lists. Keyword TO can be used on the first list to refer to consecutive variables in the active dataset and on the second list to generate new variable names. The entire specification must be enclosed in parentheses.
- Alternatively, you can specify each old variable name individually, followed by an equals sign and the new variable name. Multiple sets of variable specifications are allowed. The parentheses around each set of specifications are optional.
- RENAME does not affect the active dataset. However, if RENAME precedes DROP or KEEP, variables must be referred to by their new names on DROP or KEEP.
- Old variable names do not need to be specified according to their order in the active dataset.
- Name changes take place in one operation. Therefore, variable names can be exchanged between two variables.
- Multiple RENAME subcommands are allowed.

Examples

```
XSAVE OUTFILE='/data/empl88.sav'
  /RENAME  AGE=AGE88 JOBCAT=JOBCAT88.
CROSSTABS DEPT85 TO DEPT88 BY JOBCAT.
```

- RENAME specifies two name changes for file *empl88.sav*: *AGE* is renamed to *AGE88* and *JOBCAT* is renamed to *JOBCAT88*.

```
XSAVE OUTFILE='/data/empl88.sav'
  /RENAME (AGE JOBCAT=AGE88 JOBCAT88).
CROSSTABS DEPT85 TO DEPT88 BY JOBCAT.
```

- The name changes are identical to those in the previous example: *AGE* is renamed to *AGE88* and *JOBCAT* is renamed to *JOBCAT88*. The parentheses are required with this method.

MAP Subcommand

MAP displays a list of the variables in the data file and their corresponding names in the active dataset.

- The only specification is keyword MAP. There are no additional specifications.
- Multiple MAP subcommands are allowed. Each MAP subcommand maps the results of subcommands that precede it, but not results of subcommands that follow it.

Example

```
GET FILE='/data/hubempl.sav'.
XSAVE OUTFILE='/data/empl88.sav'
 /RENAME=(AGE=AGE88) (JOBCAT=JOBCAT88)
 /KEEP=LNAME NAME JOBCAT88 ALL /MAP.
MEANS RAISE88 BY DEPT88.
```

- MAP is used to confirm the new names for *AGE* and *JOBCAT* and the order of variables in the *empl88.sav* file (*LNAME, NAME,* and *JOBCAT88*, followed by all remaining variables from the active dataset).

COMPRESSED, UNCOMPRESSED, and ZCOMPRESSED Subcommands

COMPRESSED and ZCOMPRESSED save the file in compressed form. UNCOMPRESSED saves the file in uncompressed form.

- The only specification is the keyword COMPRESSED, UNCOMPRESSED, or ZCOMPRESSED. There are no additional specifications.

- In a file compressed with COMPRESSED, small integers (from –99 to 155) are stored in one byte instead of the eight bytes that are used in an uncompressed file.
- In a file compressed with ZCOMPRESSED, case data (regardless of type) are compressed block by block with a high compression ratio. Only IBM SPSS Statistics version 21 or later can open a file compressed with ZCOMPRESSED. Therefore, it is recommended that you use the *.zsav* file extension when saving files with ZCOMPRESSED. Older versions of IBM SPSS Statistics will ignore this extension.
- Compressed data files occupy less disk space than uncompressed data files. Files compressed with ZCOMPRESSED occupy the least disk space.
- Compressed files may take more or less time to open and save, depending on the file size and system configuration. Extra time is needed to de-compress and compress the files. However, because compressed files are smaller on disk, they reduce the time needed to read and write from disk. As the file size gets larger, this time savings surpasses the extra time needed to de-compress and compress the files. This is especially true for files compressed with ZCOMPRESSED, because these files are smallest on disk.
- The GET command, which reads IBM SPSS Statistics data files, does not need to specify whether the files it reads are compressed or uncompressed.

Only one COMPRESSED or UNCOMPRESSED subcommand can be specified per XSAVE command. COMPRESSED is the default.

PERMISSIONS Subcommand

The PERMISSIONS subcommand sets the operating system read/write permissions for the file.

READONLY. *File permissions are set to read-only for all users.* The file cannot be saved using the same file name with subsequent changes unless the read/write permissions are changed in the operating system or the subsequent XSAVE command specifies PERMISSIONS=WRITEABLE.

WRITEABLE. *File permissions are set to allow writing for the file owner.* If file permissions were set to read-only for other users, the file remains read-only for them.

Your ability to change the read/write permissions may be restricted by the operating system.

Commands and Program States

Command order is determined only by the system's need to know and do certain things in logical sequence. You cannot label a variable before the variable exists in the file. Similarly, you cannot transform or analyze data before a active dataset is defined. This appendix briefly describes how the program handles various tasks in a logical sequence. It is not necessary to understand the program states in order to construct a command file, but some knowledge of how the program works will help you considerably when you encounter a problem or try to determine why the program doesn't seem to want to accept your commands or seems to be carrying out your instructions incorrectly.

Program States

To run a program session, you need to define your active dataset, transform the data, and then analyze it. This order conforms very closely to the order the program must follow as it processes your commands. Specifically, the program checks command order according to the program state through which it passes. The **program state** is a characteristic of the program before and after a command is encountered. There are four program states. Each session starts in the **initial state**, followed by the **input program state**, the **transformation state**, and the **procedure state.** The four program states in turn enable the program to set up the environment, read data, modify data, and execute a procedure. The figure shows how the program moves through these states. The program determines the current state from the commands that it has already encountered and then identifies which commands are allowed in that state.

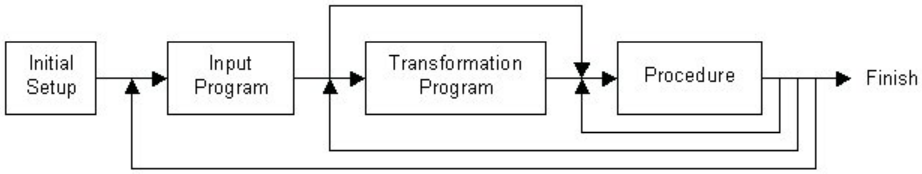


Figure 107. Program States

A session must go through initial, input program, and procedure states to be a complete session. Since all sessions start in the initial state, you need to be concerned primarily with what commands you need to define your active dataset and to analyze the data. The following commands define a very minimal session:

```
GET FILE=DATAIN.
FREQUENCIES VARIABLES=ALL.
```

The GET command defines the active dataset and the FREQUENCIES command reads the data file and analyzes it. Thus, the program goes through the required three states: initial, input, and procedure.

Typically, a session also goes through the transformation state, but it can be skipped as shown in the example above and in the diagram in the preceding figure. Consider the following example:

```
TITLE 'PLOT FOR COLLEGE SURVEY'.

DATA LIST FILE=TESTDATA
 /AGE 1-3 ITEM1 TO ITEM3 5-10.

VARIABLE LABELS ITEM1 'Opinion on level of defense spending'
  ITEM2 'Opinion on level of welfare spending'
  ITEM3 'Opinion on level of health spending'.
VALUE LABELS ITEM1 TO ITEM3 -1 'Disagree' 0 'No opinion' 1 'Agree'.
MISSING VALUES AGE(-99,-98) ITEM1 TO ITEM3 (9).
RECODE ITEM1 TO ITEM3 (0=1) (1=0) (2=-1) (9=9) (ELSE=SYSMIS).
RECODE AGE (MISSING=9) (18 THRU HI=1) (LO THRU 18=0) INTO VOTER.
PRINT /$CASENUM 1-2 AGE 4-6 VOTER 8-10.
VALUE LABELS VOTER 0 'Under 18' 1 '18 or over'.
MISSING VALUES VOTER (9).
PRINT FORMATS VOTER (F1.0).

FREQUENCIES VARIABLES=VOTER, ITEM1 TO ITEM3.
```

The program starts in the initial state, where it processes the TITLE command. It then moves into the input state upon encountering the DATA LIST command. The program can then move into either the transformation or procedure state once the DATA LIST command has been processed.

In this example, the program remains in the transformation state after processing each of the commands from VARIABLE LABELS through PRINT FORMATS. The program then moves into the procedure state to process the FREQUENCIES command. As shown in the preceding figure, the program can repeat the procedure state if it encounters a second procedure. The program can return to the transformation state if it encounters additional transformation commands following the first procedure. Finally, in some sessions the program can return to the input program state when it encounters commands such as FILE TYPE or MATCH FILES.

Determining Command Order

The table Table 121 on page 2188 shows where specific commands can be placed in the command file in terms of program states and what happens when the program encounters a command in each of the four program states. If a column contains a dash, the command is accepted in that program state and it leaves the program in that state. If one of the words *INIT*, *INPUT*, *TRANS*, or *PROC* appears in the column, the command is accepted in the program state indicated by the column heading, but it moves the program into the state indicated by *INIT*, *INPUT*, *TRANS*, or *PROC*. Asterisks in a column indicate errors when the program encounters the command in that program state. Commands marked with the dagger (†) in the column for the procedure state clear the active dataset.

The table shows six groups of commands: utility, file definition, input program, data transformation, restricted transformation, and procedure commands. These groups are discussed in the following sections.

To read the table, first locate the command. If you simply want to know where in the command stream it can go, look for columns without asterisks. For example, the COMPUTE command can be used when the program is in the input program state, the transformation state, or the procedure state, but it will cause an error if you try to use it in the initial state. If you want to know what can follow a command, look at each of the four columns next to the command. If the column is dashed, any commands not showing asterisks in the column for that program state can follow the command. If the column contains one of the words *INIT*, *INPUT*, *TRANS*, or *PROC,* any command not showing asterisks in the column for the program state indicated by that word can follow the command.

For example, if you want to know what commands can follow the INPUT PROGRAM command, note first that it is allowed only in the initial or procedure states. Then note that INPUT PROGRAM puts the program into the input program state wherever it occurs legally. This means that commands with dashes or words in the *INPUT* column can follow the INPUT PROGRAM command. This includes all the utility commands, the DATA LIST command, input program commands, and transformation commands like COMPUTE. Commands that are not allowed after the INPUT PROGRAM command are most of the file definition commands that are their own input program (such as GET), restricted transformations (such as SELECT IF), and procedures.

Table 121. Commands and program states

	INIT	**INPUT**	**TRANS**	**PROC**
Utility commands				
CLEAR TRANSFORMATIONS	**	PROC	PROC	—
COMMENT	—	—	—	—
DISPLAY	**	—	—	—
DOCUMENT	**	—	—	—
DROP DOCUMENTS	**	—	—	—
END DATA	—	—	—	—

Table 121. Commands and program states (continued)

	INIT	INPUT	TRANS	PROC
ERASE	—	—	—	—
FILE HANDLE	—	—	—	—
FILE LABEL	—	—	—	—
FINISH	—	—	—	—
INCLUDE	—	—	—	—
INFO	—	—	—	—
DEFINE—!ENDDEFINE	—	—	—	—
N OF CASES	—	—	—	TRANS
NEW FILE	—	INIT	INIT	INIT†
PROCEDURE OUTPUT	—	—	—	—
SET, SHOW	—	—	—	—
TITLE, SUBTITLE	—	—	—	—
File definition commands				
ADD FILES	TRANS	**	—	TRANS
DATA LIST	TRANS	—	INPUT	TRANS†
FILE TYPE	INPUT	**	INPUT	INPUT†
GET	TRANS	**	—	TRANS†
GET CAPTURE	TRANS	**	—	TRANS†
GET SAS	TRANS	**	—	TRANS†
GET TRANSLATE	TRANS	**	—	TRANS†
IMPORT	TRANS	**	—	TRANS†
INPUT PROGRAM	TRANS	**	—	TRANS†
KEYED DATA LIST	TRANS	—	—	TRANS
MATCH FILES	TRANS	**	—	TRANS
MATRIX DATA	TRANS	**	—	TRANS†
RENAME VARIABLES	**	—	—	TRANS
UPDATE	TRANS	**	—	TRANS
Input program commands				
END CASE	**	—	**	**
END FILE	**	—	**	**
END FILE TYPE	**	TRANS	**	**
END INPUT PROGRAM	**	TRANS	**	**
POINT	**	—	**	**
RECORD TYPE	**	—	**	**
REPEATING DATA	**	—	**	**
REREAD	**	—	**	**

Table 121. Commands and program states (continued)

	INIT	INPUT	TRANS	PROC
Transformation commands				
ADD VALUE LABELS	**	—	—	TRANS
APPLY DICTIONARY	**	—	—	TRANS
COMPUTE	**	—	—	TRANS
COUNT	**	—	—	TRANS
DO IF—END IF	**	—	—	TRANS
DO REPEAT—END REPEAT	**	—	—	TRANS
ELSE	**	—	—	TRANS
ELSE IF	**	—	—	TRANS
FORMATS	**	—	—	TRANS
IF	**	—	—	TRANS
LEAVE	**	—	—	TRANS
LOOP—END LOOP, BREAK	**	—	—	TRANS
MISSING VALUES	**	—	—	TRANS
NUMERIC	**	—	—	TRANS
PRINT	**	—	—	TRANS
PRINT EJECT	**	—	—	TRANS
PRINT FORMATS	**	—	—	TRANS
PRINT SPACE	**	—	—	TRANS
RECODE	**	—	—	TRANS
SPLIT FILE	**	—	—	TRANS
STRING	**	—	—	TRANS
VALUE LABELS	**	—	—	TRANS
VARIABLE LABELS	**	—	—	TRANS
VECTOR	**	—	—	TRANS
WEIGHT	**	—	—	TRANS
WRITE	**	—	—	TRANS
WRITE FORMATS	**	—	—	TRANS
XSAVE	**	—	—	TRANS
Restricted transformations				
FILTER	**	**	—	TRANS
REFORMAT	**	**	—	TRANS
SAMPLE	**	**	—	TRANS
SELECT IF	**	**	—	TRANS
TEMPORARY	**	**	—	TRANS
Procedures				

Table 121. Commands and program states (continued)

	INIT	INPUT	TRANS	PROC
BEGIN DATA	**	**	PROC	—
EXECUTE	**	**	PROC	—
EXPORT	**	**	PROC	—
GRAPH	**	**	PROC	—
LIST	**	**	PROC	—
SAVE	**	**	PROC	—
SAVE TRANSLATE	**	**	PROC	—
SORT CASES	**	**	PROC	—
other procedures	**	**	PROC	—

Unrestricted Utility Commands

Most utility commands can appear in any state. The table Table 121 on page 2188 shows this by the absence of asterisks in the columns.

The dashed lines indicate that after a utility command is processed, the program remains in the same state it was in before the command execution. *INIT*, *TRANS*, or *PROC* indicates that the command moves the program to that state. For example, if the program is in the procedure state, N OF CASES moves the program to the transformation state. The FINISH command terminates command processing wherever it appears. Any commands appearing after FINISH will not be read and therefore will not cause an error.

File Definition Commands

You can use most of the file definition commands in the initial state, the transformation state, and the procedure state. Most of these commands cause errors if you try to use them in the input program state. However, DATA LIST and KEYED DATA LIST can be and often are used in input programs.

After they are used in the initial state, most file definition commands move the program directly to the transformation state, since these commands are the entire input program. FILE TYPE and INPUT PROGRAM move the program into the input program state and require input program commands to complete the input program. Commands in the table Table 121 on page 2188 marked with a dagger (†) clear the active dataset.

Input Program Commands

The commands associated with the complex file facility (FILE TYPE, RECORD TYPE, and REPEATING DATA) and commands associated with the INPUT PROGRAM command are allowed only in the input program state.

The END CASE, END FILE, POINT, RECORD TYPE, REPEATING DATA, and REREAD commands leave the program in the input program state. The two that move the program on to the transformation state are END FILE TYPE for input programs initiated with FILE TYPE and END INPUT PROGRAM for those initiated with INPUT PROGRAM.

Transformation Commands

The entire set of transformation commands from ADD VALUE LABELS to XSAVE can appear in the input program state as part of an input program, in the transformation state, or in the procedure state. When you use transformation commands in the input program state or the transformation state, the program remains in the same state it was in before the command. When the program is in the procedure state, these commands move the program back to the transformation state.

Transformation commands and some file definition and input program commands can be categorized according to whether they are **declarative**, **status-switching**, or **executable**. Declarative commands alter the active dataset dictionary but do not affect the data. Status-switching commands change the program state but do not affect the data. Executable commands alter the data. The following table lists these commands and indicates which of the three categories applies.

Table 122. Taxonomy of transformation commands

Command	Type	Command	Type
ADD FILES	Exec*	LEAVE	Decl
ADD VALUE LABELS	Decl	LOOP	Exec
APPLY DICTIONARY	Decl	MATCH FILES	Exec*
BREAK	Exec	MISSING VALUES	Decl
COMPUTE	Exec	N OF CASES	Decl
COUNT	Exec	NUMERIC	Decl
DATA LIST	Exec*	POINT	Exec
DO IF	Exec	PRINT, PRINT EJECT	Exec
DO REPEAT	Decl†	PRINT FORMATS	Decl
ELSE	Exec	PRINT SPACE	Exec
ELSE IF	Exec	RECODE	Exec
END CASE	Exec	RECORD TYPE	Exec
END FILE	Exec	REFORMAT	Exec
END FILE TYPE	Stat	REPEATING DATA	Exec*
END IF	Exec	REREAD	Exec
END INPUT PROGRAM	Stat	SAMPLE	Exec
END LOOP	Exec	SELECT IF	Exec
END REPEAT	Decl†	SPLIT FILE	Decl
FILE TYPE	Stat**	STRING	Decl
FILTER	Exec	TEMPORARY	Stat
FORMATS	Decl	VALUE LABELS	Decl
GET	Exec*	VARIABLE LABELS	Decl
GET CAPTURE	Exec*	VECTOR	Decl
GET OSIRIS	Exec*	WEIGHT	Decl
IF	Exec	WRITE	Exec
INPUT PROGRAM	Stat	WRITE FORMATS	Decl
KEYED DATA LIST	Exec*	XSAVE	Exec

* This command is also declarative.

**This command is also executable and declarative.

†This command does not fit into these categories; however, it is neither executable nor status-switching, so it is classified as declarative.

Restricted Transformations

Commands REFORMAT, SAMPLE, SELECT IF, and TEMPORARY are restricted transformation commands because they are allowed in either the transformation state or the procedure state but cannot be used in the input program state.

If you use restricted transformation commands in the transformation state, the program remains in the transformation state. If you use them in the procedure state, they move the program back to the transformation state.

Procedures

The procedures and the BEGIN DATA, EXECUTE, EXPORT, LIST, SAVE, SAVE SCSS, SAVE TRANSLATE, and SORT CASES commands cause the data to be read. These commands are allowed in either the transformation state or the procedure state.

When the program is in the transformation state, these commands move the program to the procedure state. When you use these commands in the procedure state, the program remains in that state.

Defining Complex Files

Most data files have a rectangular, case-ordered structure and can be read with the DATA LIST command. This chapter illustrates the use of commands for defining complex, nonrectangular files.

- **Nested** files contain several types of records with a hierarchical relationship among the record types. You can define nested files with the FILE TYPE NESTED command.
- **Grouped** files have several records per case, and a case's records are grouped together in a file. You can use DATA LIST and FILE TYPE GROUPED to define grouped files.
- In a **mixed** file, different types of cases have different kinds of records. You can define mixed files with the FILE TYPE MIXED command.
- A record in a **repeating data** file contains information for several cases. You can use the REPEATING DATA command to define files with repeating data.

It is a good idea to read the descriptions of the FILE TYPE and REPEATING DATA commands before proceeding.

Rectangular File

The following figure shows contents of data file *RECTANG.DAT*, which contains 1988 sales data for salespeople working in different territories. Year, region, and unit sales are recorded for each salesperson. Like most data files, the sales data file has a **rectangular** format, since information on a record applies only to one case.

```
1988 CHICAGO     JONES      900
1988 CHICAGO     GREGORY    400
1988 BATON ROUGE RODRIGUEZ  300
1988 BATON ROUGE SMITH      333
1988 BATON ROUGE GRAU       100
```

Figure 108. File RECTANG.DAT

Since the sales data are rectangular, you can use the DATA LIST command to define these data:

```
DATA LIST FILE='RECTANG.DAT'
 / YEAR     1-4
   REGION   6-16(A)
   SALESPER 18-26(A)
   SALES    29-31.
```

- DATA LIST defines the variable *YEAR* in columns 1 through 4 and string variable *REGION* in columns 6 through 16 in file *RECTANG.DAT*. The program also reads variables *SALESPER* and *SALES* on each record.
- The LIST output below shows the contents of each variable.

```
YEAR REGION      SALESPER  SALES

1988 CHICAGO     JONES      900
1988 CHICAGO     GREGORY    400
1988 BATON ROUGE RODRIGUEZ  300
1988 BATON ROUGE SMITH      333
1988 BATON ROUGE GRAU       100
```

Figure 109. LIST output for RECTANG.DAT

Nested Files

In a nested file, information on some records applies to several cases. The 1988 sales data are arranged in nested format in the figure below. The data contain three kinds of records. A code in the first column indicates whether a record is a year (Y), region (R), or person record (P).

```
Y   1988
R   CHICAGO
P   JONES      900
P   GREGORY    400
R   BATON ROUGE
P   RODRIGUEZ  300
P   SMITH      333
P   GRAU       100
```

Figure 110. File NESTED.DAT

The record types are related to each other hierarchically. Year records represent the highest level in the hierarchy, since the year value 1988 applies to each salesperson in the file (only one year record is used in this example). Region records are intermediate-level records; region names apply to salesperson records that occur before the next region record in the file. For example, Chicago applies to salespersons Jones and Gregory. Baton Rouge applies to Rodriguez, Smith, and Grau. Person records represent the lowest level in the hierarchy. The information they contain—salesperson and unit sales—defines a case. Nested file structures minimize redundant information in a data file. For example, 1988 and Baton Rouge appear several times in the rectangular file, but only once in the nested file.

Since each record in the nested file has a code that indicates record type, you can use the FILE TYPE and RECORD TYPE commands to define the nested sales data:

```
FILE  TYPE  NESTED  FILE='NESTED.DAT' RECORD=#TYPE 1 (A)

RECORD TYPE 'Y'.
DATA LIST /  YEAR 5-8.

RECORD TYPE 'R'.
DATA LIST  / REGION 5-15 (A).


RECORD TYPE 'P'.
DATA LIST / SALESPER 5-15 (A) SALES 20-23

END FILE TYPE.
```

- FILE TYPE indicates that data are in nested form in the file *NESTED.DAT*.
- RECORD defines the record type variable as string variable *#TYPE* in column 1. *#TYPE* is defined as scratch variable so it won't be saved in the active dataset.
- One pair of RECORD TYPE and DATA LIST statements is specified for each record type in the file. The first pair of RECORD TYPE and DATA LIST statements defines the variable *YEAR* in columns 5 through 8 on every year record. The second pair defines the string variable *REGION* on region records. The final pair defines *SALESPER* and *SALES* on person records.
- The order of RECORD TYPE statements defines the hierarchical relationship among the records. The first RECORD TYPE defines the highest-level record type. The next RECORD TYPE defines the next highest level, and so forth. The last RECORD TYPE defines a case in the active dataset.
- END FILE TYPE signals the end of file definition.
- In processing nested data, the program reads each record type you define. Information on the highest and intermediate-level records is spread to cases to which the information applies. The output from the LIST command is identical to that for the rectangular file.

Nested Files with Missing Records

In a nested file, some cases may be missing one or more record types defined in RECORD TYPE commands. For example, in the figure below the region record for salespersons Jones and Gregory is missing.

```
Y   1988
P   JONES      900
P   GREGORY    400
R   BATON ROUGE
P   RODRIGUEZ  300
P   SMITH      333
P   GRAU       100
```

Figure 111. File NESTED.DAT with missing records

The program assigns missing values to variables that are not present for a case. Using the modified *NESTED.DAT* file, the commands in the previous example produce the output shown below. You can see that the program assigned missing values to *REGION* for Jones and Gregory.

```
YEAR REGION      SALESPER  SALES

1988             JONES      900
1988             GREGORY    400
1988 BATON ROUGE RODRIGUEZ  300
1988 BATON ROUGE SMITH      333
1988 BATON ROUGE GRAU       100
```

Figure 112. LIST output for nested data with missing records

You may want to examine cases with missing records, since these cases may indicate data errors. If you add the `MISSING=WARN` subcommand to your `FILE TYPE` command, the program prints a warning message when a case is missing a defined record type. For example, the program would print two warnings when processing data in *NESTED.DAT* with missing records. When `MISSING` is set to `WARN`, cases are built in the same way as when the default setting (`NOWARN`) is in effect.

Grouped Data

In a grouped file, a case has several records that are grouped together in the file. You can use `DATA LIST` to define a grouped file if each case has the same number of records and records appear in the same order for each case. You can use `FILE TYPE GROUPED` whether the number of records per case and record order are fixed or vary. However, `FILE TYPE GROUPED` requires that each record have a case identifier and a record code.

Using DATA LIST

The following table shows the organization of a grouped data file containing school subject scores for three students. Each student has three data records, and each record contains a score. The first record for each student also contains a case identifier. Records for each case are grouped together. Student 1 records appear first, followed by records for student 2 and student 3.

Record order determines whether a score is a reading, math, or science score. The reading score appears on the first record for a case, the math score appears on the second record, and the science score appears on the third record.

Table 123. Data for GROUPED.DAT

Student	Score
1	58
1	59
1	97
2	43
2	88
2	45
3	67
3	75
3	90

Since each case has the same number of records and record order is fixed across cases, you can use `DATA LIST` to define the student data:

```
DATA LIST FILE='GROUPED.DAT' RECORDS=3
 /STUDENT 1 READING 5-6
 /MATH    5-6
 /SCIENCE 5-6.
```

```
LIST.
```

- DATA LIST indicates that data are in file *GROUPED.DAT*.
- RECORDS defines three records per case. The program reads student ID number (*STUDENT*) and reading score (*READING*) in the first record for a case. Math and science scores are read in the second and third records.
- The output from the LIST command is shown below.

```
STUDENT READING MATH SCIENCE

   1       58    59     97
   2       43    88     45
   3       67    75     90
```

Figure 113. LIST output for GROUPED.DAT

Using FILE TYPE GROUPED

To use FILE TYPE GROUPED to define a grouped file, each record must have a case identifier and a record code. In the following commands, each data record contains a student ID number coded 1, 2, or 3 and a code indicating whether the score on that record is a reading (R), math (M), or science (S) score:

```
FILE TYPE GROUPED RECORD=#REC 3(A)  CASE=STUDENT 1.

RECORD TYPE 'R'.
DATA LIST / READING 5-6.

RECORD TYPE 'M'.
DATA LIST / MATH 5-6.

RECORD TYPE 'S'.
DATA LIST / SCIENCE 5-6.

END FILE TYPE.

BEGIN DATA
1 R 58
1 M 59
1 S 97
2 R 43
2 M 88
2 S 45
3 R 67
3 M 75
3 S 90
END DATA.

LIST.
```

- FILE TYPE indicates that data are in grouped format. RECORD defines the variable containing record codes as string variable *#REC* in column 3. CASE defines the case identifier variable *STUDENT* in the first column of each record.
- One pair of RECORD TYPE and DATA LIST statements appears for each record type in the file. The program reads reading score in every R record, math score in M records, and science score in S records.
- END FILE TYPE signals the end of file definition.
- BEGIN DATA and END DATA indicate that data are inline.
- The output from LIST is identical to the output using DATA LIST.

FILE TYPE GROUPED is most useful when record order varies across cases and when cases have missing or duplicate records. In the modified data shown below, only case 1 has all three record types. Also, record order varies across cases. For example, the first record for case 1 is a science record, whereas the first record for cases 2 and 3 is a reading record.

Table 124. Modified grouped data file

Student	Subject	Score
1	S	97
1	R	58
1	M	59

Table 124. Modified grouped data file (continued)

Student	Subject	Score
2	R	43
3	R	67
3	M	75

You can use the same FILE TYPE commands as above to read the modified file. As shown in the output from LIST below, the program assigns missing values to variables that are missing for a case.

```
STUDENT READING MATH SCIENCE

   1       58    59     97
   2       43     .      .
   3       67    75      .
```

Figure 114. LIST output for GROUPED.DAT

By default, the program generates a warning message when a case is missing a defined record type in a grouped file or when a record is not in the same order as in RECORD TYPE commands. Thus, four warnings are generated when the commands for the previous example are used to read the modified *GROUPED.DAT* file. You can suppress these warnings if you add the optional specifications MISSING=NOWARN and ORDERED=NO on your FILE TYPE command.

In the modified *GROUPED.DAT* file, the case identifier *STUDENT* appears in the same column position in each record. When the location of the case identifier varies for different types of records, you can use the CASE option of the RECORD TYPE command to specify different column positions for different records. For example, suppose the case identifier appears in first column position on reading and science records and in column 2 in math records. You could use the following commands to define the data:

```
FILE TYPE GROUPED RECORD=#REC 3(A)  CASE=STUDENT 1.

RECORD TYPE 'R'.
DATA LIST / READING 5-6.

RECORD TYPE 'M' CASE=2.
DATA LIST / MATH 5-6.

RECORD TYPE 'S'.
DATA LIST / SCIENCE 5-6.

END FILE TYPE.

BEGIN DATA
1 S 97
1 R 58
 1M 59
2 R 43
3 R 67
 3M 75
END DATA.

LIST.
```

- FILE TYPE indicates that the data are in grouped format. RECORD defines the variable containing record codes as string variable *#REC*. CASE defines the case identifier variable as *STUDENT* in the first column of each record.
- One pair of RECORD TYPE and DATA LIST statements is coded for each record type in the file.
- The CASE specification on the RECORD TYPE statement for math records overrides the CASE value defined on FILE TYPE. Thus, the program reads *STUDENT* in column 2 in math records and column 1 in other records.
- END FILE TYPE signals the end of file definition.
- BEGIN DATA and END DATA indicate that data are inline.
- The output from LIST is identical to the output above.

Mixed Files

In a mixed file, different types of cases have different kinds of records. You can use FILE TYPE MIXED to read each record or a subset of records in a mixed file.

Reading Each Record in a Mixed File

The following example is for two hypothetical elementary school students referred to a remedial education teacher. Student 1, who was thought to need special reading attention, took reading tests (word identification and comprehension tests). The second student completed writing tests (handwriting, spelling, vocabulary, and grammar tests). Test code (READING or WRITING) indicates whether the record contains reading or writing scores.

The following commands define the test data:

```
FILE TYPE MIXED RECORD=TEST 1-7(A).

RECORD TYPE 'READING'.
DATA LIST / ID  9-10 GRADE  12-13 WORD 15-16 COMPRE 18-19.

RECORD TYPE 'WRITING'.
DATA LIST / ID  9-10 GRADE 12-13 HANDWRIT 15-16 SPELLING 18-19
            VOCAB 21-22 GRAMMAR  24-25.
END FILE TYPE.

BEGIN DATA
READING 1  04 65 35
WRITING 2  03 50 55 30 25
END DATA.

LIST.
```

- FILE TYPE specifies that the data contain mixed record types. RECORD reads the record identifier (variable *TEST*) in columns 1 through 7.
- One pair of RECORD TYPE and DATA LIST statements is coded for each record type in the file. The program reads variables *ID*, *GRADE, WORD,* and *COMPRE* in the record in which the value of *TEST* is READING, and *ID*, *GRADE, HANDWRIT, SPELLING, VOCAB,* and *GRAMMAR* in the WRITING record.
- END FILE TYPE signals the end of file definition.
- BEGIN DATA and END DATA indicate that data are inline. Data are mixed, since some column positions contain different variables for the two cases. For example, word identification score is recorded in columns 15 and 16 for student 1. For student 2, handwriting score is recorded in these columns.
- The following figure shows the output from LIST. Missing values are assigned for variables that are not recorded for a case.

```
TEST    ID GRADE WORD COMPRE HANDWRIT SPELLING VOCAB GRAMMAR

READING  1    4   65    35       .        .      .     .
WRITING  2    3    .     .      50       55     30    25
```

Figure 115. LIST output for mixed file

Reading a Subset of Records in a Mixed File

You may want to process a subset of records in a mixed file. The following commands read only the data for the student who took reading tests:

```
FILE TYPE MIXED RECORD=TEST 1-7(A).

RECORD TYPE 'READING'.
DATA LIST / ID      9-10
            GRADE  12-13
            WORD   15-16
            COMPRE 18-19.

RECORD TYPE 'WRITING'.
DATA LIST / ID        9-10
            GRADE    12-13
            HANDWRIT 15-16
            SPELLING 18-19
            VOCAB    21-22
            GRAMMAR  24-25.

END FILE TYPE.

BEGIN DATA
```

```
READING 1  04 65 35
WRITING 2  03 50 55 30 25
END DATA.

LIST.
```

- FILE TYPE specifies that data contain mixed record types. RECORD defines the record identification variable as *TEST* in columns 1 through 7.
- RECORD TYPE defines variables on reading records. Since the program skips all record types that are not defined by default, the case with writing scores is not read.
- END FILE TYPE signals the end of file definition.
- BEGIN DATA and END DATA indicate that data are inline. Data are identical to those in the previous example.
- The following figure shows the output from LIST.

```
TEST    ID GRADE WORD COMPRE

READING  1    4   65    35
```

Figure 116. LIST output for reading record

Repeating Data

You can use the REPEATING DATA command to read files in which each record contains repeating groups of variables that define several cases. Command syntax depends on whether the number of repeating groups is fixed across records.

Fixed Number of Repeating Groups

The following table shows test score data for students in three classrooms. Each record contains a classroom number and two pairs of student ID and test score variables. For example, in class 101, student 182 has a score of 12 and student 134 has a score of 53. In class 103, student 15 has a score of 87 and student 203 has a score of 69. Each pair of ID and score variables is a repeating group, since these variables appear twice on each record.

Table 125. Data in REPEAT.DAT file

Class	ID	Score	ID	Score
101	182	12	134	53
102	99	112	200	150
103	15	87	203	69

The following commands generate a active dataset in which one case is built for each occurrence of *SCORE* and *ID*, and classroom number is spread to each case on a record.

```
INPUT PROGRAM.
DATA LIST /  CLASS 3-5.
REPEATING DATA STARTS=6 / OCCURS=2
 /DATA STUDENT 1-4 SCORE 5-8.
END INPUT PROGRAM.

BEGIN DATA
  101 182  12 134  53
  102  99 112 200 150
  103  15  87 203  69
END DATA.

LIST.
```

- INPUT PROGRAM signals the beginning of data definition.
- DATA LIST defines variable *CLASS*, which is spread to each student on a classroom record.
- REPEATING DATA specifies that the input file contains repeating data. STARTS indicates that repeating data begin in column 6. OCCURS specifies that the repeating data group occurs twice in each record.

- DATA defines variables that are repeated (*STUDENT* and *SCORE*). The program begins reading the first repeating data group in column 6 (the value of STARTS). Since the value of OCCURS is 2, the program reads the repeating variables a second time, beginning in the next available column (column 14).
- END INPUT PROGRAM signals the end of data definition.
- BEGIN DATA and END DATA specify that data are inline.
- The output from LIST is shown below. Each student is a separate case.

```
CLASS STUDENT SCORE
 101     182     12
 101     134     53
 102      99    112
 102     200    150
 103      15     87
 103     203     69
```

Figure 117. LIST output for repeating data

Varying Number of Repeating Groups

To use REPEATING DATA to define a file in which the number of repeating data groups varies across records, your data must contain a variable indicating the number of repeating data groups on a record. The following commands define such a file:

```
INPUT PROGRAM.
DATA LIST /  #NUM 1 CLASS 3-5.
REPEATING DATA STARTS=6 / OCCURS=#NUM
 /DATA STUDENT 1-4 SCORE 5-8.
END INPUT PROGRAM.

BEGIN DATA
3 101 182  12 134  53 199  30
2 102  99 112 200 150
1 103  15  87
END DATA.

LIST.
```

- INPUT PROGRAM signals the beginning of data definition.
- DATA LIST defines variables *CLASS* in columns 3 through 5 and *#NUM*, a scratch variable in column 1 that contains the number of repeating data groups in a record.
- REPEATING DATA specifies that the input file contains repeating data. STARTS indicates that repeating data begin in column 6. OCCURS sets the number of repeating groups on a record equal to the value of *#NUM*.
- DATA defines variables that are repeated. Since *#NUM* is 3 in the first and third records, the program reads three sets of *STUDENT* and *SCORE* variables in these records. *STUDENT* and *SCORE* are read twice in record 2.
- END INPUT PROGRAM signals the end of data definition.
- Data appear between BEGIN DATA and END DATA.
- The following figure shows the output from LIST.

```
CLASS STUDENT SCORE
 101     182     12
 101     134     53
 101     199     30
 102      99    112
 103      15     87
```

Figure 118. LIST output

If your data file does not have a variable indicating the number of repeating data groups per record, you can use the LOOP and REREAD commands to read the data, as in:

```
INPUT PROGRAM.
DATA LIST /   CLASS 3-5 #ALL 6-29 (A).
LEAVE CLASS.

LOOP #I = 1 TO 17  BY 8 IF SUBSTR(#ALL, #I, 8) NE ''.
-  REREAD COLUMN = #I + 5.
-  DATA LIST / STUDENT 1-4 SCORE 5-8.
-  END CASE.
END LOOP.
END INPUT PROGRAM.
```

```
BEGIN DATA
  101 182  12 134  53 199  30
  102  99 112 200 150
  103  15  87
END DATA.

LIST.
```

- INPUT PROGRAM signals the beginning of data definition.
- DATA LIST reads *CLASS* and *#ALL*, a temporary string variable that contains all of the repeating data groups for a classroom record. The column specifications for *#ALL* (6 through 29) are wide enough to accommodate the classroom record with the most repeating data groups (record 1).
- LOOP and END LOOP define an index loop. As the loop iterates, the program successively reads eight-character segments of *#ALL*, each of which contains a repeating data group or an empty field. The program reads the first eight characters of *#ALL* in the first iteration, the second eight characters in the second iteration, and so forth. The loop terminates when the program encounters an empty segment, which means that there are no more repeating data groups on a record.
- In each iteration of the loop in which an *#ALL* segment is not empty, DATA LIST reads *STUDENT* and *SCORE* in a classroom record. The program begins reading these variables in the first record, in the starting column specified by REREAD COLUMN. For example, in the first iteration, the program reads *STUDENT* and *SCORE* beginning in column 6. In the second iteration, the program reads *STUDENT* and *SCORE* starting in column 14 of the same record. When all repeating groups have been read for a record, loop processing begins on the following record.
- END CASE creates a new case for each repeating group.
- REREAD causes DATA LIST to read repeating data groups in the same record in which it last read *CLASS*. Without REREAD, each execution of DATA LIST would begin on a different record.
- LEAVE preserves the value of CLASS across the repeating data groups on a record. Thus, the same class number is read for each student on a classroom record.
- INPUT PROGRAM signals the beginning of data definition.
- BEGIN DATA and END DATA indicate that the data are inline. The data are identical to those in the previous example except that they do not contain a variable indicating the number of repeating groups per record.
- These commands generate the same output as shown in the figure above.

Using the Macro Facility

A macro is a set of commands that generates customized command syntax. Using macros can reduce the time and effort needed to perform complex and repetitive data analysis tasks.

Macros have two parts: a **macro definition**, which indicates the beginning and end of the macro and gives a name to the macro, and a **macro body**, which contains regular commands or macro commands that build command syntax. When a macro is invoked by the **macro call**, syntax is generated in a process called **macro expansion**. Then the generated syntax is executed as part of the normal command sequence.

This chapter shows how to construct macros that perform three data analysis tasks. In the first example, macros facilitate a file-matching task. In Example 2, macros automate a specialized statistical operation (testing a sample correlation coefficient against a nonzero population correlation coefficient). Macros in Example 3 generate random data. As shown in the following table, each example demonstrates various features of the macro facility. For information on specific macro commands, see the DEFINE command.

Table 126. Macro features

	Example 1	**Example 2**	**Example 3**
Macro argument			
Keyword	x	x	x
Default values	x		x
None	x		x
String manipulation	x		x
Looping			
Index	x		x
List processing		x	
Direct assignment	x		x

Example 1: Automating a File-Matching Task

The following figure shows a listing of 1988 sales data for salespeople working in different regions. The listing shows that salesperson Jones sold 900 units in the Chicago sales territory, while Rodriguez sold 300 units in Baton Rouge.

```
YEAR REGION      SALESPER  SALES

1988 CHICAGO     JONES      900
1988 CHICAGO     GREGORY    400
1988 BATON ROUGE RODRIGUEZ  300
1988 BATON ROUGE SMITH      333
1988 BATON ROUGE GRAU       100
```

Figure 119. Listing of data file SALES88.SAV

You can use command syntax shown below to obtain each salesperson's percentage of total sales for their region.

```
GET FILE = 'SALES88.SAV'.

SORT CASES BY REGION.

AGGREGATE OUTFILE = 'TOTALS.SAV'
  /PRESORTED
  /BREAK = REGION
  /TOTAL@ = SUM(SALES).

MATCH FILES FILE=*
  /TABLE = 'TOTALS.SAV'
  /BY REGION.

COMPUTE PCT = 100 * SALES / TOTAL@.

TITLE 1988 DATA.
LIST.
```

Figure 120. Commands for obtaining sales percentages

- The GET command opens *SALES88.SAV*. This file becomes the active dataset.
- SORT CASES sorts the active dataset in ascending alphabetical order by *REGION*.
- The AGGREGATE command saves total sales (variable *TOTAL@*) for each region in file *TOTALS.SAV*.
- MATCH FILES appends the regional totals to each salesperson's record in the active dataset. (See the MATCH FILES command for more information on matching files.)
- COMPUTE obtains the percentage of regional sales (*PCT*) for each salesperson.
- The LIST command output displayed below shows that Rodriguez sold 41% of the products sold in Baton Rouge. Gregory accounted for 31% of sales in the Chicago area.

```
YEAR REGION      SALESPER  SALES  TOTAL@     PCT

1988 BATON ROUGE RODRIGUEZ  300   733.00   41.00
1988 BATON ROUGE SMITH      333   733.00   45.00
1988 BATON ROUGE GRAU       100   733.00   14.00
1988 CHICAGO     JONES      900  1300.00   69.00
1988 CHICAGO     GREGORY    400  1300.00   69.00
```

Figure 121. Listing of data file SALES88.SAV

The following figure shows a macro that issues the commands for obtaining sales percentages. The macro consists of the commands that produce sales percentages imbedded between macro definition commands DEFINE and !ENDDEFINE.

```
DEFINE !TOTMAC ().

GET FILE = 'SALES88.SAV'.

SORT CASES BY REGION.

AGGREGATE OUTFILE = 'TOTALS.SAV'
  /PRESORTED
  /BREAK = REGION
  /TOTAL@ = SUM(SALES).

MATCH FILES FILE = *
  /TABLE = 'TOTALS.SAV'
  /BY REGION.

COMPUTE PCT = 100 * SALES / TOTAL@.

TITLE 1988 DATA.
LIST.

!ENDDEFINE.

!TOTMAC.
```

Figure 122. !TOTMAC macro

- Macro definition commands DEFINE and !ENDDEFINE signal the beginning and end of macro processing. DEFINE also assigns the name !TOTMAC to the macro (the parentheses following the name of the macro are required). The macro name begins with an exclamation point so that the macro does not conflict with that of an existing variable or command. Otherwise, if the macro name matched a variable name, the variable name would invoke the macro whenever the variable name appeared in the command stream.
- Commands between DEFINE and !ENDDEFINE constitute the macro body. These commands produce sales percentages.

- The final statement, !TOTMAC, is the **macro call**, which invokes the macro. When the program reads the macro call, it issues the commands in the macro body. Then these commands are executed, generating identical output.

While the macro shows you how to construct a simple macro, it doesn't reduce the number of commands needed to calculate regional percentages. However, you can use macro features such as looping to minimize coding in more complicated tasks. For example, let's say that in addition to the 1988 data, you have sales data for 1989 (*SALES89.SAV),* and each file contains the variables *REGION*, *SALESPER*, and *SALES*. The modified !TOTMAC macro below calculates regional sales percentages for each salesperson for 1988 and 1989.

```
DEFINE !TOTMAC ().

!DO !I = 88 !TO 89.

- GET FILE = !CONCAT('SALES', !I, '.SAV').
- SORT CASES BY REGION.
- AGGREGATE OUTFILE = 'TOTALS.SAV'
      /PRESORTED
      /BREAK = REGION
      /TOTAL@ = SUM(SALES).
- MATCH FILES FILE = *
      /TABLE = 'TOTALS.SAV'
      /BY REGION.
- COMPUTE PCT= 100 * SALES / TOTAL@.
- !LET !YEAR = !CONCAT('19',!I).
- TITLE !YEAR DATA.
- LIST.
!DOEND.

!ENDDEFINE.

!TOTMAC.
```

Figure 123. !TOTMAC macro with index loop

- DEFINE and !ENDDEFINE signal the beginning and end of macro processing.
- Commands !DO and !DOEND define an **index loop**. Commands between !DO and !DOEND are issued once in each iteration of the loop. The value of **index variable** !I, which changes in each iteration, is 88 in the first iteration and 89 in the second (final) iteration.
- In each iteration of the loop, the GET command opens a data file. The name of the file is constructed using the **string manipulation function** !CONCAT, which creates a string that is the concatenation of SALES, the value of the index variable, and *.sav*. Thus the file *SALES88.SAV* is opened in the first iteration.
- Commands between AGGREGATE and COMPUTE calculate percentages on the active dataset.
- Next, a customized title is created. In the first iteration, the **direct assignment** command !LET assigns a value of 1988 to the macro variable !YEAR. This variable is used in the TITLE command on the following line to specify a title of *1988 DATA*.
- The LIST command displays the contents of each variable.
- In the second iteration of the loop, commands display percentages for the 1989 data file. The output from the !TOTMAC macro is shown below. Note that the listing for 1988 data is the same as before.

```
1988 DATA

YEAR REGION      SALESPER  SALES  TOTAL@     PCT

1988 BATON ROUGE RODRIGUEZ  300   733.00   41.00
1988 BATON ROUGE SMITH      333   733.00   45.00
1988 BATON ROUGE GRAU       100   733.00   14.00
1988 CHICAGO     JONES      900  1300.00   69.00
1988 CHICAGO     GREGORY    400  1300.00   69.00


1989 DATA
YEAR REGION      SALESPER SALES   TOTAL@     PCT
1989 BATON ROUGE GRAU      320   1459.00   22.00
1989 BATON ROUGE SMITH     800   1459.00   55.00
1989 BATON ROUGE RODRIGUEZ 339   1459.00   23.00
1989 CHICAGO     JONES     300   1439.00   21.00
1989 CHICAGO     STEEL     899   1439.00   62.00
1989 CHICAGO     GREGORY   240   1439.00   17.00
```

Figure 124. Regional sales percentages for 1988 and 1989

Let's look at another application of the !TOTMAC macro, one that uses **keyword arguments** to make the application more flexible. The following figure shows the number of absences for students in two

classrooms. Let's say you want to calculate deviation scores indicating how many more (or fewer) times a student was absent than the average student in his or her classroom. The first step in obtaining deviation scores is to compute the average number of absences per classroom. We can use the !TOTMAC macro to compute classroom means by modifying the macro so that it computes means and uses the absences data file *(SCHOOL.SAV)* as input.

```
CLASS STUDENT ABSENT
 101  BARRY G    3
 101  JENNI W    1
 101  ED F       2
 101  JOHN 0     8
 102  PAUL Y     2
 102  AMY G      3
 102  JOHN D    12
 102  RICH H     4
```

Figure 125. Listing of file SCHOOL.SAV

The !TOTMAC macro below can produce a variety of group summary statistics such as sum, mean, and standard deviation for any IBM SPSS Statistics data file. In the macro call you specify values of keyword arguments indicating the data file (FILE), the break (grouping) variable (BREAKVR), the summary function (FUNC), and the variable to be used as input to the summary function (INVAR). For example, to obtain mean absences for each classroom, we specify *SCHOOL.SAV* as the data file, *CLASS* as the break variable, MEAN as the summary function, and *ABSENT* as the variable whose values are to be averaged.

```
DEFINE !TOTMAC ( BREAKVR = !TOKENS(1)
                 /FUNC = !TOKENS(1)
                 /INVAR = !TOKENS(1)
                 /TEMP = !TOKENS(1) !DEFAULT(TOTALS.SAV)
                 /FILE = !CMDEND).
GET FILE = !FILE.
SORT CASES BY !BREAKVR.
AGGREGATE OUTFILE = '!TEMP'
   /PRESORTED
   /BREAK = !BREAKVR
   /!CONCAT(!FUNC,'@') = !FUNC(!INVAR).

MATCH FILES FILE = *
   /TABLE = '!TEMP'
   /BY !BREAKVR.

!ENDDEFINE.

!TOTMAC BREAKVR=CLASS FUNC=MEAN INVAR=ABSENT FILE=SCHOOL.SAV.

COMPUTE DIFF = ABSENT-MEAN@.

LIST.

!TOTMAC BREAKVR=REGION FUNC=SUM INVAR=SALES FILE=SALES89.SAV.

COMPUTE PCT = 100 * SALES / SUM@.

LIST.
```

Figure 126. !TOTMAC macro with keyword arguments

- The syntax for declaring keyword arguments follows the name of the macro in DEFINE.
- !TOKENS(1) specifies that the value of an argument is a string following the name of the argument in the macro call. Thus the first macro call specifies CLASS as the value of BREAKVR, MEAN as the value of FUNC, and ABSENT as the value of INVAR.
- !CMDEND indicates that the value for FILE is the remaining text in the macro call (*SCHOOL.SAV*).
- TEMP is an optional argument that names an intermediate file to contain the summary statistics. Since TEMP is not assigned a value in the macro call, summary statistics are written to the default intermediate file *(TOTALS.SAV)*.
- In the body of the macro, GET FILE opens *SCHOOL.SAV*.
- SORT CASES sorts the file by *CLASS*.
- AGGREGATE computes the mean number of absences for each class. The name of the variable containing the means (*MEAN*@) is constructed using the !CONCAT function, which concatenates the value of FUNC and the @ symbol.
- MATCH FILES appends the means to student records.
- COMPUTE calculates the deviation from the classroom mean for each student (variable *DIFF*).

- LIST displays the deviation scores, as shown in the output below. For example, John D., who was absent 12 times, had 6.75 more absences than the average student in classroom 102. Rich H., who was absent 4 times, had 1.25 fewer absences than the average student in classroom 102.
- The second macro call and remaining commands generate regional sales percentages for the 1989 sales data.

```
CLASS STUDENT ABSENT MEAN@    DIFF
 101  BARRY G    3    3.50    -.50
 101  JENNI W    1    3.50   -2.50
 101  ED F       2    3.50   -1.50
 101  JOHN 0     8    3.50    4.50
 102  PAUL Y     2    5.25   -3.25
 102  AMY G      3    5.25   -2.25
 102  JOHN D    12    5.25    6.75
 102  RICH H     4    5.25   -1.25


1989 DATA
YEAR REGION      SALESPER SALES   TOTAL@     PCT
1989 BATON ROUGE GRAU      320   1459.00   22.00
1989 BATON ROUGE SMITH     800   1459.00   55.00
1989 BATON ROUGE RODRIGUEZ 339   1459.00   23.00
1989 CHICAGO     JONES     300   1439.00   21.00
1989 CHICAGO     STEEL     899   1439.00   62.00
1989 CHICAGO     GREGORY   240   1439.00   17.00
```

Figure 127. Student absences and 1989 sales percentages

You can modify the macro call to specify a different data file, input variable, break variable, or summary statistic. To get a different summary statistic (such as standard deviation), change the value of FUNC (see the AGGREGATE command for more information on summary functions available in the AGGREGATE procedure).

Example 2: Testing Correlation Coefficients

While the program provides a large variety of statistical procedures, some specialized operations require the use of COMPUTE statements. For example, you may want to test a sample correlation coefficient against a population correlation coefficient. When the population coefficient is nonzero, you can compute a *Z* statistic to test the hypothesis that the sample and population values are equal.

The formula for <inlinetag type="italic">Z</inlinetag> is:

$$Z = \frac{0.5 \ln\left[\frac{1+r}{1-r}\right] - 0.5 \ln\left[\frac{1+p_0}{1-p_0}\right]}{1/\left(\sqrt{n-3}\right)}$$

Figure 128. Formula for Z

Let's say you want to test an *r* of 0.66 obtained from a sample of 30 cases against a population coefficient of 0.85. The following figure shows commands for displaying *Z* and its two-tailed probability.

```
DATA LIST FREE / R N P.

BEGIN DATA
.66 30 .85
END DATA.

COMPUTE #ZR = .5* (LN ((1 + R) / (1 - R))).
COMPUTE #ZP = .5* (LN ((1 + P) / (1 - P))).

COMPUTE Z = (#ZR-#ZP)/(1/(SQRT(N-3))).
COMPUTE PROB = 2*(1-CDFNORM(ABS(Z))).

FORMAT PROB (F8.3).
LIST.
```

Figure 129. Commands for computing Z statistic

- DATA LIST defines variables containing the sample correlation coefficient *(R*), sample size (*N)*, and population correlation coefficient (*P)*.

- BEGIN DATA and END DATA indicate that data are inline.
- COMPUTE statements calculate *Z* and its probability. Variables *#ZR* and *#ZP* are scratch variables used in the intermediate steps of the calculation.
- The LIST command output is shown below. Since the absolute value of *Z* is large and the probability is small, we reject the hypothesis that the sample was drawn from a population having a correlation coefficient of 0.85.

```
  R       N     P       Z   PROB
.66   30.00   .85   -2.41   .016
```

Figure 130. Z statistic and its probability

If you use the *Z* test frequently, you may want to construct a macro like that shown below. The !CORRTST macro computes *Z* and probability values for a sample correlation coefficient, sample size, and population coefficient specified as values of keyword arguments.

```
DEFINE !CORRTST ( R = !TOKENS(1)
                  /N = !TOKENS(1)
                  /P = !TOKENS(1)).
INPUT PROGRAM.
- END CASE.
- END FILE.
END INPUT PROGRAM.

COMPUTE #ZR = .5* (LN ((1 + !R) / (1 - !R))).
COMPUTE #ZP = .5* (LN ((1 + !P) / (1 - !P))).

COMPUTE Z = (#ZR-#ZP) / (1/(SQRT(!N-3))).
COMPUTE PROB = 2*(1-CDFNORM(ABS(Z))).
FORMAT PROB(F8.3).

TITLE SAMPLE R=!R, N=!N, POPULATION COEFFICIENT=!P.

LIST.

!ENDDEFINE.

!CORRTST R=.66 N=30 P=.85.
!CORRTST R=.50 N=50 P=.85.
```

Figure 131. !CORRTST macro

- DEFINE names the macro as !CORRTST and declares arguments for the sample correlation coefficient (R), the sample size (N), and the population correlation coefficient (P).
- !TOKENS(1) specifies that the value of an argument is a string that follows the name of the argument in the macro call. Thus the first macro call specifies values of 0.66, 30, and 0.85 for R, N, and P.
- Commands between INPUT PROGRAM and END INPUT PROGRAM create an active dataset with one case. COMPUTE statements calculate the *Z* statistic and its probability using the values of macro arguments R, N, and P. (INPUT PROGRAM commands would not be needed if COMPUTE statements operated on values in an existing file or inline data, rather than macro arguments.)
- A customized TITLE shows displays the values of macro arguments used in computing *Z*.
- The LIST command displays *Z* and its probability.
- The !CORRTST macro is called twice. The first invocation tests an *r* of 0.66 from a sample of 30 cases against a population coefficient of 0.85 (this generates the same *Z* value and probability as shown earlier). The second macro call tests an *r* of 0.50 from a sample of 50 cases against the same population correlation coefficient. The output from these macro calls is shown below.

```
SAMPLE R= .66 , N= 30 , POPULATION COEFFICIENT= .85

      Z     PROB
   -2.41     .016


SAMPLE R= .50 , N= 50 , POPULATION COEFFICIENT= .85

      Z     PROB
   -4.85      .000
```

Figure 132. Output from !CORRTST

The following figure shows a modified !CORRTST macro that you can use to test a sample *r* against each coefficient in a *list* of population coefficients.

```
DEFINE !CORRTST (R = !TOKENS(1)
                 /N = !TOKENS(1)
                 /P = !CMDEND).
- INPUT PROGRAM.
-   END CASE.
-   END FILE.
- END INPUT PROGRAM.

!DO !I !IN (!P).
- COMPUTE #ZR = .5* (LN ((1 + !R) / (1 - !R))).
- COMPUTE #ZP = .5* (LN ((1 + !I) / (1 - !I))).

- COMPUTE Z = (#ZR-#ZP)/(1/(SQRT(!N-3))).

- COMPUTE PROB=2*(1-CDFNORM(ABS(Z))).
- FORMAT PROB(F8.3).
- TITLE SAMPLE R=!R, N=!N, POPULATION COEFFICIENT=!I.
- LIST.
!DOEND.

!ENDDEFINE.

!CORRTST R=.66 N=30 P=.20 .40 .60 .80 .85 .90.
```

Figure 133. !CORRTST macro with list-processing loop

- DEFINE names the macro as !CORRTST and declares arguments for the sample correlation coefficient (R), the sample size (N), and the population correlation coefficient (P).
- !TOKENS(1) specifies that the value of an argument is a string that follows the name of the argument in the macro call. Thus, the macro call specifies the value of R as 0.66 and N as 0.30.
- !CMDEND indicates that the value for P is the remaining text in the macro call. Thus the value of P is a list containing the elements 0.20, 0.40, 0.60, 0.80, 0.85, and 0.90.
- Commands !DO !IN and !DOEND define a **list-processing loop**. Commands in the loop compute one *Z* statistic for each element in the list of population coefficients. For example, in the first iteration *Z* is computed using 0.20 as the population coefficient. In the second iteration 0.40 is used. The same sample size (30) and *r* value (0.66) are used for each *Z* statistic.
- The output from the macro call is shown below. One *Z* statistic is displayed for each population coefficient.

```
SAMPLE R= .66 , N= 30 , POPULATION COEFFICIENT= .20

       Z     PROB
    3.07     .002


SAMPLE R= .66 , N= 30 , POPULATION COEFFICIENT= .40

       Z     PROB
    1.92     .055


SAMPLE R= .66 , N= 30 , POPULATION COEFFICIENT= .60

       Z     PROB
     .52     .605


SAMPLE R= .66 , N= 30 , POPULATION COEFFICIENT= .80

       Z     PROB
   -1.59     .112


SAMPLE R= .66 , N= 30 , POPULATION COEFFICIENT= .85

       Z     PROB
   -2.41     .016


SAMPLE R= .66 , N= 30 , POPULATION COEFFICIENT= .90

       Z     PROB
   -3.53     .000
```

Figure 134. Output from modified !CORRTST macro

Example 3: Generating Random Data

You can use command syntax to generate variables that have approximately a normal distribution. Commands for generating five standard normal variables (*X1* through *X5*) for 1000 cases are shown in the following figure. As shown in the output below, each variable has a mean of approximately 0 and a standard deviation of approximately 1.

```
INPUT PROGRAM.
-   VECTOR X(5).
-       LOOP #I = 1 TO 1000.
-           LOOP #J = 1 TO 5.
-             COMPUTE X(#J) = NORMAL(1).
-           END LOOP.
-           END CASE.
-       END LOOP.
-       END FILE.
END INPUT PROGRAM.

DESCRIPTIVES VARIABLES X1 TO X5.
```

Figure 135. Data-generating commands

```
                                                Valid
Variable  Mean  Std Dev  Minimum  Maximum      N  Label
X1        -.01     1.02    -3.11     4.15   1000
X2         .08     1.03    -3.19     3.22   1000
X3         .02     1.00    -3.01     3.51   1000
X4         .03     1.00    -3.35     3.19   1000
X5        -.01      .96    -3.34     2.91   1000
```

Figure 136. Descriptive statistics for generated data

The !DATAGEN macro below issues the data-generating commands shown above.

```
DEFINE !DATAGEN ().

INPUT PROGRAM.
-    VECTOR X(5).
-        LOOP #I = 1 TO 1000.
-            LOOP #J = 1 TO 5.
-                COMPUTE X(#J) = NORMAL(1).
-            END LOOP.
-            END CASE.
-        END LOOP.
-        END FILE.
END INPUT PROGRAM.

DESCRIPTIVES VARIABLES X1 TO X5.

!ENDDEFINE.

!DATAGEN.
```

Figure 137. !DATAGEN macro

The data-generating commands are imbedded between macro definition commands. The macro produces the same data and descriptive statistics as shown above.

You can tailor the generation of normally distributed variables if you modify the !DATAGEN macro so it will accept keyword arguments, as shown in the following figure. The macro allows you to specify the number of variables and cases to be generated and the approximate standard deviation.

```
DEFINE !DATAGEN ( OBS =!TOKENS(1) !DEFAULT(1000)
                  /VARS =!TOKENS(1) !DEFAULT(5)
                  /SD =!CMDEND !DEFAULT(1)).
INPUT PROGRAM.
-   VECTOR X(!VARS).
-       LOOP #I = 1 TO !OBS.
-           LOOP #J = 1 TO !VARS.
-               COMPUTE X(#J) = NORMAL(!SD).
-           END LOOP.
-           END CASE.
-       END LOOP.
-       END FILE.
END INPUT PROGRAM.

!LET !LIST = !NULL.
!DO  !I = 1 !TO !VARS.
-   !LET !LIST = !CONCAT(!LIST, ' ', X, !I).
!DOEND.

DESCRIPTIVES VARIABLES !LIST.

!ENDDEFINE.

!DATAGEN OBS=500 VARS=2 SD=1.
!DATAGEN.
```

Figure 138. !DATAGEN macro with keyword arguments

- The DEFINE statement declares arguments that specify the number of cases (OBS), variables (VARS), and standard deviation (SD). By default, the macro creates 1000 cases with 5 variables that have a standard deviation of 1.

- Commands between INPUT PROGRAM and END INPUT PROGRAM generate the new data using values of the macro arguments.
- Commands !LET and !DO/!DOEND construct a variable list (!LIST) that is used in DESCRIPTIVES. The first !LET command initializes the list to a null (blank) string value. For each new variable, the index loop adds to the list a string of the form *X1*, *X2, X3*, and so forth. Thus, DESCRIPTIVES requests means and standard deviations for each new variable.
- The first macro call generates 500 cases with two standard normal variables. The second call requests the default number of variables, cases, and standard deviation. Descriptive statistics (not shown) are also computed for each variable.

As shown in the following figure, you can declare additional keyword arguments that allow you to specify the distribution (normal or uniform) of the generated data and a parameter value that is used as the standard deviation (for normally distributed data) or a range (for uniformly distributed data).

```
DEFINE !DATAGEN (OBS    =!TOKENS(1) !DEFAULT(1000)
                 /VARS  =!TOKENS(1) !DEFAULT(5)
                 /DIST  =!TOKENS(1) !DEFAULT(NORMAL)
                 /PARAM =!TOKENS(1) !DEFAULT(1)).
INPUT PROGRAM.
-   VECTOR X(!VARS).
-       LOOP #I = 1 TO !OBS.
-           LOOP #J = 1 TO !VARS.
-               COMPUTE X(#J) = !DIST(!PARAM).
-           END LOOP.
-           END CASE.
-       END LOOP.
-       END FILE.
END INPUT PROGRAM.

!LET !LIST = !NULL.
!DO  !I = 1 !TO !VARS.
-   !LET !LIST = !CONCAT(!LIST, ' ', X, !I).
!DOEND.

DESCRIPTIVES VARIABLES !LIST.
!ENDDEFINE.

!DATAGEN OBS=500 VARS=2 DIST=UNIFORM PARAM=2.
```

Figure 139. !DATAGEN macro with additional keyword arguments

- The DEFINE statement declares arguments OBS, VARS, DIST, and PARAM. OBS and VARS represent the number of observations and cases to be generated. Arguments DIST and PARAM specify the shape and parameter of the distribution of generated data. By default, the macro generates 1000 observations with 5 standard normal variables.
- Statements between INPUT PROGRAM and END INPUT PROGRAM generate the new data using values of macro arguments.
- Remaining commands in the body of the macro obtain descriptive statistics for generated variables.
- The macro call in creates two approximately uniformly distributed variables with a range of 2. The output from the macro call is shown below.

```
                                          Valid
Variable  Mean  Std Dev  Minimum  Maximum      N  Label
X1         .99      .57      .00     2.00    500
X2        1.00      .57      .00     2.00    500
```

Figure 140. Descriptive statistics for uniform variables

Canonical Correlation and Ridge Regression Macros

Two macro routines are installed with for performing canonical correlation and ridge regression. Macros are inherently less robust than regular commands. Error checking is minimal, and small errors produce large numbers of uninformative messages, giving little clue to the real cause of the problem. Syntax must be entered exactly as documented:

- Enter subcommands in the order shown.
- Do not include a slash before the first subcommand.
- Enter keywords exactly as shown. Do not abbreviate them, and do not extend them (even if they look like abbreviations).
- Do not omit equal signs, but do not add them if none are displayed in the syntax diagram.
- Remember that these "command names" are actually macro calls, which are expanded into (lengthy) sequences of commands and Matrix language statements. The command name should not appear anywhere within the specifications of the command, for example within a variable name.
- These macros create new files in the current directory. If you do not have write permission to the current directory (this can happen if you are running the program on a network), change the current directory. You can do this by opening or saving a file on your own hard disk.
- You cannot run these macros if split-file processing is in effect.

Canonical Correlation Macro

```
INCLUDE '[installdir]/Samples/English/Canonical correlation.sps'.
CANCORR SET1=varlist1 /
        SET2=varlist2 / .
The two variable lists must be separated with a slash.
[installdir] is the installation directory.
```

Ridge Regression Macro

```
INCLUDE '[installdir]/Samples/English/Ridge regression.sps'.
RIDGEREG DEP=varname /ENTER = varlist
 [/START={0**}] [/STOP={1**}] [/INC={0.05**}]
        {value}         {value}        {value   }
 [ /K=value] .
[installdir] is the installation directory.
```

File Specifications for IBM SPSS Collaboration and Deployment Services Repository Objects

Access to a IBM SPSS Collaboration and Deployment Services Repository is available with the Statistics Adapter option. Once a connection to a IBM SPSS Collaboration and Deployment Services Repository has been established, you can use a file specification for a IBM SPSS Collaboration and Deployment Services Repository object on nearly any keyword or subcommand where a file specification can be provided (including, but not limited to `FILE`, `INFILE`, or `OUTFILE`). File specifications for IBM SPSS Collaboration and Deployment Services Repository objects can also be used in procedure dialog boxes that allow manual entry of a file specification—for example, XML model files or plan files for complex samples analyses.

A IBM SPSS Collaboration and Deployment Services Repository file specification has the following form:

```
SPSSCR://{directory path and/or file specification}

         [{#M.versionmarker}]
          {#L.versionlabel }
```

Note: Specification of descriptions (#D) and keywords (#K) is obsolete for release 17.0 and higher.

Basic Specification

The basic specification is the scheme name SPSSCR , followed by a colon, one or two slashes (forward or backward), and a directory path or file: for example:

```
GET FILE='SPSSCR://data/mydata.sav'.
```

- A IBM SPSS Collaboration and Deployment Services Repository file specification is included on a GET command. The latest version of the file */data/mydata.sav* that is accessible by the current user will be retrieved from the currently connected IBM SPSS Collaboration and Deployment Services Repository.

Syntax Rules

- Paths can be specified with forward slashes (/) or backslashes (\).
- The optional components of a file specification--version marker (#M) and version label (#L)--can be specified in any order.
- When including an optional component, the character sequence that specifies the component--for example, #L--must be followed by a period (.).

File Extensions

Files are stored to the IBM SPSS Collaboration and Deployment Services Repository with an associated MIME type that specifies the type of file; for example, IBM SPSS Statistics format data, or viewer output. Including a file extension, such as *.sav* or *.spv*, is not necessary in order for the correct MIME type to be determined; however, it is recommended.

- File extensions are not added to filenames when storing to a IBM SPSS Collaboration and Deployment Services Repository. For example, the following SAVE command results in the creation of a IBM SPSS Collaboration and Deployment Services Repository object with a MIME type of IBM SPSS Statistics-format data, and a fully qualified path of */data/mydata*, but no *sav* extension.

  ```
  SAVE OUTFILE='SPSSCR://data/mydata'.
  ```

- When retrieving IBM SPSS Collaboration and Deployment Services Repository objects, you must specify the filename exactly as provided when the file was stored. If the file was stored with an extension, then you must supply the extension. If the file was stored without an extension then do not include one.

Permissions

- The `PERMISSIONS`, `SAVE`, and `XSAVE` commands will generate errors if you attempt to set permissions for IBM SPSS Collaboration and Deployment Services Repository objects. Permissions for IBM SPSS

Collaboration and Deployment Services Repository objects are set with the REPOSITORY ATTRIBUTES command or using administration tools included with IBM SPSS Collaboration and Deployment Services.

- The ERASE FILE command will generate an error if you attempt to delete a IBM SPSS Collaboration and Deployment Services Repository object.

Note: For users upgrading from an IBM SPSS Collaboration and Deployment Services 3.5 system, be aware that command syntax that retrieved the version labelled *LATEST* (the default version retrieved by GET FILE='filespec') in a 3.5 system will retrieve an earlier version in an IBM SPSS Collaboration and Deployment Services 4.x system if the current user does not have permission to view objects with the *LATEST* label.

Versions

By default, the latest version that is accessible by the current user is returned when a file is retrieved. You can optionally specify a particular version to retrieve by including the version marker or version label, if one exists. Version markers and labels are always ignored if the specification describes a directory path, as opposed to a specific file.

Version label (#L). A label is used to easily identify a version; for example, *production* or *development*.

- The version label, if provided, is displayed in the Label column in the Select Version dialog box. See the topic for more information.

Version marker (#M). The version marker consists of an integer version number followed by a time stamp that uniquely specifies a version—for example, *1:2006-07-26 15:19:49.237*.

- The version marker is displayed in the Version column in the Select Version dialog box. See the topic for more information.

Examples

```
*Retrieve a labelled version.
GET FILE='SPSSCR://data/mydata.sav#L.development'.
```

- The version of */data/mydata.sav* with the label *development* is retrieved from the current IBM SPSS Collaboration and Deployment Services Repository.

```
*Retrieve a version using its version marker.
OUTPUT OPEN FILE=
   'SPSSCR://output/myoutput.spv#M.0:2006-07-26 15:18:15.862'.
```

- The version of the output file */output/myoutput.spv* with version marker *0:2006-07-26 15:18:15.862* is retrieved.

Using File Handles for IBM SPSS Collaboration and Deployment Services Repository Locations

You can define a file handle to a file or a directory in a IBM SPSS Collaboration and Deployment Services Repository and use that file handle when storing or retrieving.

```
*Define and use a file handle for a directory.
FILE HANDLE cust /NAME='SPSSCR://CustomerValue'.
GET FILE='cust/data2002.sav'.
```

- The handle *cust* is associated with the IBM SPSS Collaboration and Deployment Services Repository directory */CustomerValue*, and the GET command retrieves the latest version of the file *data2002.sav* from this directory in the current IBM SPSS Collaboration and Deployment Services Repository.

```
*Define and use a file handle for a specific file.
FILE HANDLE cust /NAME='SPSSCR://CustomerValue/data2002.sav'.
GET FILE='cust'.
```

- The handle *cust* is associated with the IBM SPSS Collaboration and Deployment Services Repository file */CustomerValue/data2002.sav*, and the GET command retrieves the latest version of this file from the current IBM SPSS Collaboration and Deployment Services Repository.

Setting the Working Directory to a IBM SPSS Collaboration and Deployment Services Repository Location

You can use the CD command to set the working directory to a directory in the currently connected IBM SPSS Collaboration and Deployment Services Repository.

```
CD 'SPSSCR://CustomerValue'.
GET FILE='data2002.sav'.
```

- The working directory is set to the IBM SPSS Collaboration and Deployment Services Repository directory */CustomerValue*, and the GET command retrieves the latest version of the file *data2002.sav* from this directory in the current IBM SPSS Collaboration and Deployment Services Repository.

TABLES and IGRAPH Command Syntax Converter

If you have command syntax files that contain TABLES syntax that you want to convert to CTABLES syntax and/or IGRAPH syntax that you want to convert to GGRAPH syntax, a simple utility program is provided to help you get started with the conversion process. There are, however, significant functionality differences between TABLES and CTABLES and between IGRAPH and GGRAPH. It is likely that you will find that the utility program cannot convert some of your TABLES and IGRAPH syntax jobs or may generate CTABLES and GGRAPH syntax that produces tables and graphs that do not closely resemble the originals produced by the TABLES and IGRAPH commands. For most tables, you can edit the converted syntax to produce a table closely resembling the original.

The utility program is designed to:

- Create a new syntax file from an existing syntax file. The original syntax file is not altered.
- Convert only TABLES and IGRAPH commands in the syntax file. Other commands in the file are not altered.
- Retain the original TABLES and IGRAPH syntax in commented form.
- Identify the beginning and end of each conversion block with comments.
- Identify TABLES and IGRAPH syntax commands that could not be converted.
- Convert command syntax files that follow either interactive or production mode syntax rules.

This utility cannot convert commands that contain errors. The following other limitations also apply.

TABLES Limitations

The utility program may convert TABLES commands incorrectly under some circumstances, including TABLES commands that contain:

- Parenthesized variable names with the initial letters "sta" or "lab" in the TABLES subcommand if the variable is parenthesized by itself—for example, `var1 by (statvar) by (labvar)`. These will be interpreted as the (STATISTICS) and (LABELS) keywords.
- SORT subcommands that use the abbreviations A or D to indicate ascending or descending sort order. These will be interpreted as variable names.

The utility program cannot convert TABLES commands that contain:

- Syntax errors.
- OBSERVATION subcommands that refer to a range of variables using the TO keyword (for example, `var01 TO var05`).
- String literals broken into segments separated by plus signs (for example, `TITLE "My" + "Title"`).
- Macro calls that, in the absence of macro expansion, would be invalid TABLES syntax. Since the converter does not expand the macro calls, it treats them as if they were simply part of the standard TABLES syntax.

The utility program will not convert TABLES commands contained in macros. All macros are unaffected by the conversion process.

IGRAPH Limitations

IGRAPH changed significantly in release 16. Because of these changes, some subcommands and keywords in IGRAPH syntax created before that release may not be honored. See the IGRAPH section in the *Command Syntax Reference* for the complete revision history.

The conversion utility program may generate additional syntax that it stores in the INLINETEMPLATE keyword within the GGRAPH syntax. This keyword is created only by the conversion program. Its syntax is not intended to be user-editable.

Using the Conversion Utility Program

The conversion utility program, *SyntaxConverter.exe*, can be found in the installation directory. It is designed to run from a command prompt. The general form of the command is:

```
syntaxconverter.exe [path]/inputfilename.sps [path]/outputfilename.sps
```

You must run this command from the installation directory.

If any directory names contain spaces, enclose the entire path and filename in quotation marks, as in:

```
syntaxconverter.exe /myfiles/oldfile.sps "/new files/newfile.sps"
```

Interactive versus Production Mode Command Syntax Rules

The conversion utility program can convert command files that use interactive or production mode syntax rules.

Interactive. The interactive syntax rules are:

- Each command begins on a new line.
- Each command ends with a period (.).

Production mode. The Production Facility and commands in files accessed via the `INCLUDE` command in a different command file use production mode syntax rules:

- Each command must begin in the first column of a new line.
- Continuation lines must be indented at least one space.
- The period at the end of the command is optional.

If your command files use production mode syntax rules and don't contain periods at the end of each command, you need to include the command line switch `-b` (or `/b`) when you run *SyntaxConverter.exe*, as in:

```
syntaxconverter.exe -b /myfiles/oldfile.sps /myfiles/newfile.sps
```

SyntaxConverter Script (Windows Only)

On Windows, you can also run the syntax converter with the script *SyntaxConverter.wwd*, located in the *Samples* directory of the installation directory.

1. From the menus choose:

   Utilities > **Run Script...**
2. Navigate to the *Samples* directory and select *SyntaxConverter.wwd*.

This will open a simple dialog box where you can specify the names and locations of the old and new command syntax files.

Notices

This information was developed for products and services offered in the US. This material might be available from IBM in other languages. However, you may be required to own a copy of the product or product version in that language in order to access it.

IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing
IBM Corporation
North Castle Drive, MD-NC119
Armonk, NY 10504-1785
US

For license inquiries regarding double-byte (DBCS) information, contact the IBM Intellectual Property Department in your country or send inquiries, in writing, to:

Intellectual Property Licensing
Legal and Intellectual Property Law
IBM Japan Ltd.
19-21, Nihonbashi-Hakozakicho, Chuo-ku
Tokyo 103-8510, Japan

INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some jurisdictions do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk.

IBM may use or distribute any of the information you provide in any way it believes appropriate without incurring any obligation to you.

Licensees of this program who wish to have information about it for the purpose of enabling: (i) the exchange of information between independently created programs and other programs (including this one) and (ii) the mutual use of the information which has been exchanged, should contact:

IBM Director of Licensing
IBM Corporation
North Castle Drive, MD-NC119
Armonk, NY 10504-1785
US

Such information may be available, subject to appropriate terms and conditions, including in some cases, payment of a fee.

The licensed program described in this document and all licensed material available for it are provided by IBM under terms of the IBM Customer Agreement, IBM International Program License Agreement or any equivalent agreement between us.

The performance data and client examples cited are presented for illustrative purposes only. Actual performance results may vary depending on specific configurations and operating conditions.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBMproducts. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

Statements regarding IBM's future direction or intent are subject to change or withdrawal without notice, and represent goals and objectives only.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to actual people or business enterprises is entirely coincidental.

COPYRIGHT LICENSE:

This information contains sample application programs in source language, which illustrate programming techniques on various operating platforms. You may copy, modify, and distribute these sample programs in any form without payment to IBM, for the purposes of developing, using, marketing or distributing application programs conforming to the application programming interface for the operating platform for which the sample programs are written. These examples have not been thoroughly tested under all conditions. IBM, therefore, cannot guarantee or imply reliability, serviceability, or function of these programs. The sample programs are provided "AS IS", without warranty of any kind. IBM shall not be liable for any damages arising out of your use of the sample programs.

Each copy or any portion of these sample programs or any derivative work, must include a copyright notice as follows:

© IBM 2020. Portions of this code are derived from IBM Corp. Sample Programs.

© Copyright IBM Corp. 1989 - 2020. All rights reserved.

Trademarks

IBM, the IBM logo, and ibm.com are trademarks or registered trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the web at "Copyright and trademark information" at www.ibm.com/legal/copytrade.shtml.

Adobe, the Adobe logo, PostScript, and the PostScript logo are either registered trademarks or trademarks of Adobe Systems Incorporated in the United States, and/or other countries.

Intel, Intel logo, Intel Inside, Intel Inside logo, Intel Centrino, Intel Centrino logo, Celeron, Intel Xeon, Intel SpeedStep, Itanium, and Pentium are trademarks or registered trademarks of Intel Corporation or its subsidiaries in the United States and other countries.

Linux is a registered trademark of Linus Torvalds in the United States, other countries, or both.

Microsoft, Windows, Windows NT, and the Windows logo are trademarks of Microsoft Corporation in the United States, other countries, or both.

UNIX is a registered trademark of The Open Group in the United States and other countries.

Java and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and/or its affiliates.

Index

Special Characters

!BLANKS (function)
DEFINE command 564
!BREAK (command)
DEFINE command 567
!BY (keyword)
DEFINE command 567
!CHAREND (keyword)
DEFINE command 561
!CMDEND (keyword)
DEFINE command 561
!CONCAT (function)
DEFINE command 564
!DEFAULT (keyword)
DEFINE command 563
!DO (command)
DEFINE command 567
!DOEND (command)
DEFINE command 567
!ELSE (keyword)
DEFINE command 566
!ENCLOSE (keyword)
DEFINE command 561
!EVAL (function)
DEFINE command 564
!HEAD (function)
DEFINE command 564
!IF (command)
DEFINE command 566
!IFEND (command)
DEFINE command 566
!IN (keyword)
DEFINE command 568
!INDEX (function)
DEFINE command 564
!LENGTH (function)
DEFINE command 564
!LET (command)
DEFINE command 569
!NOEXPAND (keyword)
DEFINE command 563
!NULL (function)
DEFINE command 564
!OFFEXPAND (keyword)
DEFINE command 564
!ONEXPAND (keyword)
DEFINE command 564
!POSITIONAL (keyword)
DEFINE command 559
!QUOTE (function)
DEFINE command 564
!SUBSTRING (function)
DEFINE command 564
!TAIL (function)
DEFINE command 564
!THEN (keyword)
DEFINE command 566
!THEN (keyword) *(continued)*
DEFINE command 566
!TO (keyword)
DEFINE command 567
!TOKENS (keyword)
DEFINE command 561
!UNQUOTE (function)
DEFINE command 564
!UPCASE (function)
DEFINE command 564
(keyword)
IGRAPH command 924
(XBARONLY) (keyword)
SPCHART command 1875
)CARD (keyword)
PLANCARDS command 1428
)DATE (keyword)
CTABLES command 500
)TABLE (keyword)
CTABLES command 500
)TIME (keyword)
CTABLES command 500
@ATTRIBUTES (keyword)
DISPLAY command 599
$CASE (keyword)
IGRAPH command 918
$CASENUM
system variable 45
with SELECT IF command 1775
$CASENUM (system variable)
PRINT EJECT command 1536
$COUNT (keyword)
IGRAPH command 918, 921, 924–926, 928, 930
$DATE
system variable 45
$DATE11
system variable 45
$JDATE
system variable 45
$PCT (keyword)
IGRAPH command 918, 921, 924–926, 928, 930
$SYSMIS
system variable 45
$TIME
system variable 45
$VARS (subcommand)
SHOW command 1810

Numerics

2SLS (command)
APPLY subcommand 95
CONSTANT subcommand 95
covariance matrix 95
ENDOGENOUS subcommand 95
endogenous variables 95
EQUATION subcommand 94

2SLS (command) *(continued)*
including constant 95
instrumental variables 94
INSTRUMENTS subcommand 94
NOCONSTANT subcommand 95
PRINT subcommand 95
SAVE subcommand 95
saving predicted values 95
saving residuals 95
syntax chart 93
using a previous model 95

A

A (keyword)
DESCRIPTIVES command 575
SORT CASES command 1841
SPECTRA command 1893, 1895
AAD (keyword)
RATIO STATISTICS command 1601, 1602
ABS (function)
MATRIX command 1077
ABSOLUTE (keyword)
CSORDINAL command 446
MIXED command 1138
PROXIMITIES command 1557
ACCELERATION (subcommand)
PROXSCAL command 1575
ACF (command)
APPLY subcommand 101
DIFF subcommand 98
LN/NOLOG subcommands 99
MXAUTO subcommand 100
PACF subcommand 101
partial autocorrelation 101
PERIOD subcommand 99
periodic lags 100
SDIFF subcommand 99
SEASONAL subcommand 100
SERROR subcommand 100
specifying periodicity 99
standard error method 100
syntax chart 97
transforming values 98
using a previously defined model 101
VARIABLES subcommand 98
ACPROB (keyword)
NOMREG command 1256
PLUM command 1440
ACTION (keyword)
EXTENSION command 663
ACTIVE (keyword)
CATPCA command 276
MULTIPLE CORRESPONDENCE command 1193
active dataset
appending orthogonal designs 1349
caching 1790
active file
caching 1790
AD1 (keyword)
MIXED command 1137
ADATE format 53, 54
ADD (function)
REPORT command 1686
ADD DOCUMENT (command)
syntax chart 103
ADD FILES (command)
adding cases from different data sources 110
BY subcommand 108
case source variable 109
DROP subcommand 108
FILE subcommand 107
FIRST subcommand 109
IN subcommand 109
KEEP subcommand 108
key variables 108
LAST subcommand 109
MAP subcommand 110
RENAME subcommand 107
syntax chart 105
with SORT CASES command 106, 1843
ADD VALUE LABELS (command)
compared with VALUE LABELS command 2109
string variables 112
syntax chart 111
adding columns to database tables 1760
ADDITIVE (keyword)
SEASON command 1772
additive model
SEASON command 1772
ADDTYPE (keyword)
MVA command 1220
ADJ (keyword)
MIXED command 1140
ADJCHISQUARE (keyword)
CSCOXREG command 404
CSGLM command 424
CSLOGISTIC command 436
CSORDINAL command 447
ADJF (keyword)
CSCOXREG command 404
CSGLM command 424
CSLOGISTIC command 436
CSORDINAL command 447
ADJNORMALIZED (keyword)
KNN command 971
MLP command 1153
RBF command 1607
ADJPRED (keyword)
REGRESSION command 1634
ADJUST (keyword)
AIM command 132
ADJUSTCORR (keyword)
GENLIN command 737
adjusted chi-square
CSGLM command 424
CSLOGISTIC command 436
adjusted F statistic
CSGLM command 424
CSLOGISTIC command 436
adjusted residuals
GENLOG command 773
ADJUSTEDRSQUARED (keyword)
LINEAR command 980
ADJUSTLEVEL (subcommand)
ADP command 117
ADP (command)
ADJUSTLEVEL subcommand 117

ADP (command) *(continued)*
CRITERIA subcommand 120
FIELDS subcommand 115
OUTFILE subcommand 120
OUTLIERHANDLING subcommand 118
PREPDATETIME subcommand 115
REORDERNOMINAL subcommand 118
REPLACEMISSING subcommand 118
RESCALE subcommand 118
SCREENING subcommand 117
syntax chart 113
TRANSFORM subcommand 119
AEMPIRICAL (keyword)
EXAMINE command 642
AFREQ (keyword)
FREQUENCIES command 716
AFTER (keyword)
ANOVA command 161
AGGCOXSNELL (keyword)
CSCOXREG command 407
AGGDEVIANCE (keyword)
CSCOXREG command 407
AGGDFBETA (keyword)
CSCOXREG command 407
agglomeration schedule
CLUSTER command 318
AGGMARTINGALE (keyword)
CSCOXREG command 407
AGGREGATE (command)
BREAK subcommand 124
DOCUMENT subcommand 125
functions 125
MISSING subcommand 127
OUTFILE subcommand 122
PRESORTED subcommand 125
release history 121
syntax chart 121
variable definitions 125
with SORT CASES command 1843
with SPLIT FILE command 121, 1897
aggregate data
ANACOR command 156
aggregated data
SURVIVAL command 1923
aggregating data
aggregate functions 125
aggregate variables 125
break variables 121, 124
saving files 122
variable labels 125
variable names 125
AGGREGATION (subcommand)
SPATIAL ASSOCIATION RULES command 1849
SPATIAL TEMPORAL PREDICTION command 1862
AGGSCORE (keyword)
CSCOXREG command 407
AHEX format 47
AIC (keyword)
FACTOR command 669
TWOSTEP CLUSTER command 2069
AICC (keyword)
LINEAR command 980
AIM (command)
CATEGORICAL subcommand 132
AIM (command) *(continued)*
CONTINUOUS subcommand 132
CRITERIA subcommand 132
grouping variable 131
MISSING subcommand 132
PLOT subcommand 133
syntax chart 131
AINDS (keyword)
ALSCAL command 140
Akaike information criterion
REGRESSION command 1640
ALIGN (keyword)
REPORT command 1677
ALIGNMENT (keyword)
APPLY DICTIONARY command 171
ALL (function)
MATRIX command 1077
ALL (keyword)
ALSCAL command 142
ANACOR command 152, 154
ANOVA command 161, 164
CONJOINT command 347
CORRELATIONS command 351
CORRESPONDENCE command 358
CROSSTABS command 389, 390, 393
DESCRIPTIVES command 575
DISCRIMINANT command 592, 594
EXAMINE command 643, 644
FREQUENCIES command 720
GENLIN command 732, 741
HOMALS command 900
IGRAPH command 921
in variable lists 44
KNN command 972
LOGISTIC REGRESSION command 996
MEANS command 1126, 1127
MULT RESPONSE command 1186
NPAR TESTS command 1278, 1291
OVERALS command 1407
PARTIAL CORR command 1419
PRINCALS command 1524
PRINT command 1529
RELIABILITY command 1655
SUMMARIZE command 1913
TSET command 2026
USE command 2097
WRITE command 2159
ALLVARS (keyword)
VALIDATEDATA command 2103
ALPHA (keyword)
FACTOR command 672
GLM command 850
MANOVA command 1049
NPTESTS command 1267
RELIABILITY command 1655
UNIANOVA command 2076
ALPHA (subcommand)
EXSMOOTH command 658
alpha coefficient
RELIABILITY command 1655
alpha factoring
FACTOR command 672
alpha level
UNIANOVA command 2076

alpha value
for post hoc tests 856
ALSCAL (command)
analysis criteria 141
analysis specification 145
analysis summary 142
CONDITION subcommand 139
conditionality 139
convergence 141
CRITERIA subcommand 141
defining data shape 138
dimensionality of solution 141
displaying input data 142
FILE subcommand 139
input files 139
INPUT subcommand 137
iterations 141
level of measurement 138
LEVEL subcommand 138
limitations 135
matrix input 144
matrix output 143, 144
MATRIX subcommand 144
METHOD subcommand 140
missing values 141
MODEL subcommand 140
models 140, 145
OUTFILE subcommand 143
output files 143
PLOT subcommand 142
plots 142
PRINT subcommand 142
SHAPE subcommand 138
specifying input rows 137
syntax chart 135
VARIABLES subcommand 137
ALTER TYPE (command)
PRINT subcommand 150
syntax chart 149
ALTER TYPE command 150
alternative hypothesis
UNIANOVA command 2076
Ameniya's prediction criterion
REGRESSION command 1640
ANACOR (command)
aggregate data 156
DIMENSION subcommand 153
MATRIX subcommand 155
NORMALIZATION subcommand 153
PLOT subcommand 154
PRINT subcommand 154
syntax chart 151
TABLE subcommand 152
value labels 154
VARIANCES subcommand 154
with WEIGHT command 156
ANALYSIS (keyword)
CONJOINT command 347
CSDESCRIPTIVES command 414
CSPLAN command 457
NPAR TESTS command 1291
NPTESTS command 1267
ONEWAY command 1338
PARTIAL CORR command 1420
ANALYSIS (keyword) *(continued)*
T-TEST command 2064
ANALYSIS (subcommand)
CATPCA command 275
CATREG command 291
DISCRIMINANT command 586
FACTOR command 668
HOMALS command 899
MANOVA command 1037, 1051
MULTIPLE CORRESPONDENCE command 1192
OVERALS command 1405
PRINCALS command 1523
with SETS subcommand 1405
with VARIABLES subcommand 899, 1405
analysis of covariance
GLM command 838
analysis of variance
CURVEFIT command 513
DISCRIMINANT command 592
GLM command 838
MEANS command 1127
QUICK CLUSTER command 1590
REGRESSION command 1640
RELIABILITY command 1655
SUMMARIZE command 1915
ANALYSIS_WEIGHT (keyword)
LINEAR command 980
ANALYSISTYPE (keyword)
GENLIN command 732
ANALYSISVARS (keyword)
VALIDATEDATA command 2103
ANALYSISWEIGHT (keyword)
ADP command 115
CSPLAN command 457
ANALYSISWEIGHT (subcommand)
MULTIPLE IMPUTATION command 1209
analyzing aggregated data
CORRESPONDENCE command 357
analyzing table data
CORRESPONDENCE command 358
ANCOVA model
syntax 840
Anderberg's D
CLUSTER command 314
PROXIMITIES command 1559
Anderson-Rubin factor scores
FACTOR command 674
ANDREW (keyword)
EXAMINE command 644
ANOMALY (keyword)
DETECTANOMALY command 581
Anomaly Detection
command syntax 577
ANOMALYCUTPOINT (keyword)
DETECTANOMALY command 580
ANOMALYLIST (keyword)
DETECTANOMALY command 582
ANOMALYSUMMARY (keyword)
DETECTANOMALY command 582
ANOVA (command)
cell means 165
covariates 160, 165
COVARIATES subcommand 161
defining factor ranges 160

ANOVA (command) *(continued)*
factor variables 160
interaction effects 161
limitations 159
MAXORDERS subcommand 161
METHOD subcommand 161
MISSING subcommand 165
multiple classification analysis 165
STATISTICS subcommand 164
sums of squares 161
VARIABLES subcommand 160
ANOVA (keyword)
CATREG command 296
CURVEFIT command 513
MEANS command 1127
QUICK CLUSTER command 1590
REGRESSION command 1640
RELIABILITY command 1655
SUMMARIZE command 1915
anti-image matrix
FACTOR command 669
ANTIIDEAL (keyword)
CONJOINT command 345
ANY (function)
MATRIX command 1077
APPEND (subcommand)
MCONVERT command 1122
SAVE TRANSLATE command 1761
APPLY (keyword)
DETECTANOMALY command 580
APPLY (subcommand)
2SLS command 95
ACF command 101
AREG command 176
ARIMA command 184
CCF command 302
CURVEFIT command 513
EXSMOOTH command 661
FIT keyword 176, 184
INITIAL keyword 176, 184
PACF command 1414
PPLOT command 1497
SEASON command 1773
SPECIFICATIONS keyword 176, 184
SPECTRA command 1896
TSPLOT command 2058
WLS command 2156
APPLY DICTIONARY (command)
FILEINFO subcommand 170
FROM subcommand 168
NEWVARS subcommand 169
release history 167
SOURCE subcommand 169
syntax chart 167
TARGET subcommand 169
APPLY TEMPLATE (subcommand)
AUTORECODE command 191
Apply Time Series Models
command syntax 2013
ApplyModel (function) 86
APPROX (keyword)
CATPCA command 284, 286
APPROXIMATE (keyword)
MANOVA command 1033, 1050
APPROXIMATE (keyword) *(continued)*
SURVIVAL command 1922
AR (keyword)
FACTOR command 674
GENLIN command 737
AR (subcommand)
ARIMA command 183
AR1 (keyword)
MIXED command 1137
ARCHITECTURE (subcommand)
MLP command 1156
RBF command 1609
arcsine function 61
arctangent function 61
AREA (keyword)
GRAPH command 881
area charts
sequence 258, 2054
AREALABEL (keyword)
IGRAPH command 924
AREG (command)
APPLY subcommand 176
Cochrane-Orcutt method 175
CONSTANT subcommand 175
including constant 175
maximum iterations 175
maximum likelihood estimation 175
METHOD subcommand 175
MXITER subcommand 175
NOCONSTANT subcommand 175
Prais-Winsten method 175
RHO subcommand 175
rho value 175
syntax chart 173
using a previously defined model 176
VARIABLES subcommand 174
arguments
complex 59
defined 61
ARH1 (keyword)
MIXED command 1137
ARIMA
TSMODEL command 2045
ARIMA (command)
APPLY subcommand 184
AR subcommand 183
CINPCT subcommand 184
CON subcommand 183
confidence intervals 184
D subcommand 182
difference transformation 181, 182
FORECAST subcommand 185
initial parameter values 183
iterations 184
log transformation (base 10) 181
MA subcommand 183
model parameters 182
MODEL subcommand 181
MXITER subcommand 184
MXLAMB subcommand 184
natural log transformation 181
P subcommand 182
parameter-order subcommands 182
PAREPS subcommand 184

ARIMA (command) *(continued)*
Q subcommand 182
REG subcommand 183
SAR subcommand 183
SD subcommand 182
seasonal difference transformation 181, 182
single or nonsequential parameters 182
SMA subcommand 183
SP subcommand 182
specifying periodicity 181
SQ subcommand 182
SSQPCT subcommand 184
syntax chart 179
termination criteria 184
using a previously defined model 184
VARIABLES subcommand 181
ARIMA (subcommand)
TSMODEL command 2045
arithmetic functions 61, 337
arithmetic operators
in matrix language 1073
ARMA11 (keyword)
MIXED command 1137
ARRANGEMENT (subcommand)
GET DATA command 790
arrays. See vectors 2139
ARSIN (function)
MATRIX command 1077
ARTAN (function)
MATRIX command 1077
ASCAL (keyword)
ALSCAL command 140
ASCENDING (keyword)
CSORDINAL command 441
GENLIN command 726
NPTESTS command 1271
RATIO STATISTICS command 1600
ASE (keyword)
LINEAR command 980
ASIS (keyword)
CROSSTABS command 393
ASRESID (keyword)
CROSSTABS command 389
CSTABULATE command 477
assignment expression
computing values 335
Associated UI topic if any
command syntax 439
ASSOCIATION (keyword)
HILOGLINEAR command 894
NOMREG command 1256
ASSUMEDSTRWIDTH (subcommand)
GET DATA command 788
ASYMMETRIC (keyword)
ALSCAL command 138
asymmetric Euclidean distance model
ALSCAL command 140
asymmetric individual differences Euclidean distance model
ALSCAL command 140
asymmetric matrix
ALSCAL command 138
ASYMPTOTIC_CILEVEL (keyword)
FLEISS MULTIRATER KAPPA command 706
ASYMPTOTIC_CILEVEL (keyword) *(continued)*
WEIGHTED KAPPA command 2150
attributes
adding and deleting custom variable attributes 2121
user-defined data file attributes 531
ATTRIBUTES (keyword)
APPLY DICTIONARY command 170, 171
CODEBOOK command 325, 326
DISPLAY command 599
ATUO (keyword)
KNN command 973
AUTHOR (keyword)
REPOSITORY ATTRIBUTES command 1694
AUTO (keyword)
KNN command 972
TWOSTEP CLUSTER command 2069
AUTOBINNING (subcommand)
SPATIAL ASSOCIATION RULES command 1849
autocorrelation
command syntax 97
partial autocorrelations 97, 1411
AUTOFIT (subcommand)
SIMPLAN command 1827
AUTOFIX (subcommand)
CASESTOVARS command 267
AUTOINIT (keyword)
ARIMA command 185
Automated Data Preparation
command syntax 113
AUTOMATIC (keyword)
MLP command 1156
REPORT command 1677
AUTOOUTLIER (subcommand)
TSMODEL command 2048
AUTORECODE (command)
APPLY TEMPLATE subcommand 191
BLANK subcommand 189
compared with RECODE command 1619
DESCENDING subcommand 192
GROUP subcommand 189
INTO subcommand 188
missing values 187
PRINT subcommand 192
release history 187
SAVE TEMPLATE subcommand 190
syntax chart 187
VARIABLES subcommand 188
with HOMALS command 897, 898
with OVERALS command 1403
with PRINCALS command 1521
with TABLES command 188
Autoregression
command syntax 173
AUXILIARY (subcommand)
TSAPPLY command 2021
TSMODEL command 2040
auxiliary regression model
in GLM 862
in UNIANOVA 2087
AVALUE (keyword)
CROSSTABS command 393
FREQUENCIES command 716
AVERAGE (function)
REPORT command 1686

average absolute deviation (AAD)
RATIO STATISTICS command 1601, 1602
average linkage between groups
CLUSTER command 317
average linkage within groups
CLUSTER command 317
AVERF (keyword)
MANOVA command 1059
AVONLY (keyword)
MANOVA command 1059
AZLOUT (keyword)
SPCHART command 1885
AZOUT (keyword)
SPCHART command 1885
AZUOUT (keyword)
SPCHART command 1885

B

BACK (keyword)
TMS IMPORT command 1986
BACKWARD (keyword)
HILOGLINEAR command 891
NOMREG command 1252
REGRESSION command 1638
backward elimination
COXREG command 371
HILOGLINEAR command 891
LOGISTIC REGRESSION command 993
REGRESSION command 1638
BADCORR (keyword)
PARTIAL CORR command 1419
REGRESSION command 1643
BAGGING (keyword)
LINEAR command 980
BALANCED (keyword)
CATPCA command 280
balanced designs
in GLM 862
UNIANOVA command 2088
BANDWIDTH (keyword)
IGRAPH command 930
BAR (keyword)
AIM command 133
BAR (subcommand)
GRAPH command 880
IGRAPH command 925
bar charts
3-D 2176
FREQUENCIES command 716, 717
interval width 716, 717
scale 716, 717
BARBASE (keyword)
IGRAPH command 925
BARCHART (subcommand)
CROSSTABS command 393
FREQUENCIES command 716, 717
BART (keyword)
FACTOR command 674
BARTLETT (keyword)
SPECTRA command 1893
Bartlett factor scores
FACTOR command 674
Bartlett window
Bartlett window *(continued)*
SPECTRA command 1893
Bartlett's approximation
ACF command 100
Bartlett's test of sphericity
FACTOR command 669
in MANOVA 1048
BASE (subcommand)
MULT RESPONSE command 1186
BASELINE (keyword)
COXREG command 373
CSCOXREG command 406
IGRAPH command 925, 929
BASELINESTRATA (keyword)
CSCOXREG command 400
BASETEXTDIRECTION (subcommand)
SET command 1789
SHOW command 1810
BASIS (keyword)
MANOVA command 1045
BATCH (keyword)
MLP command 1157
batch syntax rules
inserted command files 946, 950, 951
BAVERAGE (keyword)
CLUSTER command 317
BAYES ANOVA
command syntax 193
BAYES ANOVA (command)
CRITERIA subcommand 194
DESIGN subcommand 194
ESTBF subcommand 194
INFERENCE subcommand 194
iteration criteria 194, 195
limitations 193
PLOT subcommand 195
PRIOR subcommand 195
syntax chart 193
BAYES CORRELATION
command syntax 197
BAYES CORRELATION (command)
CRITERIA subcommand 199
ESTBF subcommand 199
INFERENCE subcommand 199
iteration criteria 198, 199
limitations 197
MISSING subcommand 198
syntax chart 197
BAYES INDEPENDENT
command syntax 201
BAYES INDEPENDENT (command)
CRITERIA subcommand 202
ESTBF subcommand 203
INFERENCE subcommand 203
iteration criteria 202, 203
limitations 201
MISSING subcommand 202
PRIOR subcommand 203
syntax chart 201
BAYES LOGLINEAR
command syntax 205
BAYES LOGLINEAR (command)
CRITERIA subcommand 206
ESTBF subcommand 207

BAYES LOGLINEAR (command) *(continued)*
INFERENCE subcommand 207
iteration criteria 206–208
limitations 205
PRINT subcommand 208
syntax chart 205
TABLE subcommand 207
BAYES ONESAMPLE
command syntax 209
syntax chart 209
BAYES ONESAMPLE: Binomial
command syntax 209
BAYES ONESAMPLE: Binomial (command)
CRITERIA subcommand 211
DATA subcommand 211
INFERENCE subcommand 211
iteration criteria 210–212
limitations 209
MISSING subcommand 210
PRIOR subcommand 212
syntax chart 209
BAYES ONESAMPLE: Normal
command syntax 212
BAYES ONESAMPLE: Normal (command)
CRITERIA subcommand 214
DATA subcommand 215
INFERENCE subcommand 215
iteration criteria 214, 215
limitations 213
MISSING SCOPE subcommand 214
PRIOR subcommand 215
syntax chart 212
BAYES ONESAMPLE: Poisson
command syntax 216
BAYES ONESAMPLE: Poisson (command)
CRITERIA subcommand 218
DATA subcommand 218
INFERENCE subcommand 218
iteration criteria 217, 218
limitations 216
MISSING SCOPE subcommand 217
PRIOR subcommand 218
syntax chart 216
BAYES REGRESSION
command syntax 221
BAYES REGRESSION (command)
CRITERIA subcommand 223, 1582
DESIGN subcommand 223, 1583
ESTBF subcommand 223
FTEST subcommand 225
INFERENCE subcommand 223
iteration criteria 223–225, 1582–1584
limitations 221
OUTFILE subcommand 225, 1584
PLOT subcommand 225, 1584
PREDICT subcommand 224, 1584
PRIOR subcommand 224, 1584
SAVE subcommand 224, 1583
syntax chart 221
BAYES RELATED
command syntax 227
BAYES RELATED (command)
CRITERIA subcommand 229
INFERENCE subcommand 229
BAYES RELATED (command) *(continued)*
iteration criteria 228–230
limitations 227
MISSING SCOPE subcommand 228
PAIR subcommand 229
PRIOR subcommand 230
syntax chart 227
BAYES REPEATED
command syntax 233
BAYES REPEATED (command)
CRITERIA subcommand 234
DESIGN subcommand 234
ESTBF subcommand 234
INFERENCE subcommand 234
iteration criteria 234
limitations 233
PLOT subcommand 234
syntax chart 233
BCOC (keyword)
RATIO STATISTICS command 1601, 1602
BCON (keyword)
COXREG command 373
LOGISTIC REGRESSION command 996
BCOV (keyword)
REGRESSION command 1640
BEGIN DATA (command)
syntax chart 235
with INCLUDE command 235
with SUBTITLE command 1909
with TITLE command 1975
BEGIN EXPR (command)
OUTFILE subcommand 238
BEGIN EXPR-END EXPR (command)
syntax chart 237
BEGIN GPL-END GPL (command)
release history 243
syntax chart 243
BEGIN PROGRAM(command)
release history 245
syntax chart 245
Bernoulli distribution function 64
BESTSUBSET (keyword)
NAIVEBAYES command 1230
BESTSUBSETS (keyword)
LINEAR command 980
beta distribution function 64
BEUCLID (keyword)
CLUSTER command 314
PROXIMITIES command 1559
BIAS (keyword)
CORRELATIONS command 353
NOMREG command 1251
PLUM command 1438
BIC (keyword)
TWOSTEP CLUSTER command 2069
BIN (keyword)
OPTIMAL BINNING command 1344
BIN (subcommand)
XGRAPH command 2171
binary Euclidean distance
CLUSTER command 314
PROXIMITIES command 1559
binary format 50
binary shape difference

binary shape difference *(continued)*
CLUSTER command 314
PROXIMITIES command 1559
binary squared Euclidean distance
CLUSTER command 314
PROXIMITIES command 1559
binary variance measure
CLUSTER command 314
PROXIMITIES command 1559
BINNING (keyword)
ADP command 119
BINOMIAL (keyword)
GENLIN command 728
GENLINMIXED command 757
NPTESTS command 1268
BINOMIAL (subcommand)
NPAR TESTS command 1279
binomial distribution function 64
BINS (keyword)
NAIVEBAYES command 1230
SIMPLAN command 1827
BIPLOT (keyword)
CATPCA command 282
CORRESPONDENCE command 361
MULTIPLE CORRESPONDENCE command 1197
biplots
CATPCA command 282
CORRESPONDENCE command 361
MULTIPLE CORRESPONDENCE command 1197
BIVARIATE (keyword)
GRAPH command 882
Bivariate Correlations 1417
bivariate normal distribution function 64
bivariate spectral analysis
SPECTRA command 1894
blank
delimiter 36
BLANK (keyword)
FACTOR command 669
REPORT command 1681
BLANK (subcommand)
AUTORECODE command 189
blank data fields
treatment of 1790
blank lines
displaying 1539
blank strings
autorecoding to user-missing 189
BLANKS (subcommand)
SET command 1790
SHOW command 1810
BLOCK (function)
MATRIX command 1077
BLOCK (keyword)
CLUSTER command 313
PROXIMITIES command 1558
BLOCK (subcommand)
SET command 1790
SHOW command 1810
BLOM (keyword)
PPLOT command 1494
RANK command 1596
Blom's transformation 1596
BLWMN (keyword)
BLWMN (keyword) *(continued)*
CLUSTER command 314
PROXIMITIES command 1559
BMP (subcommand)
OUTPUT EXPORT command 1368
BONFERRONI (keyword)
AIM command 132
CSCOXREG command 404
CSGLM command 424
CSLOGISTIC command 436
CSORDINAL command 447
GENLIN command 741
GLM command 856
MIXED command 1140
ONEWAY command 1336
UNIANOVA command 2082
Bonferroni correction
CSGLM command 424
CSLOGISTIC command 436
CTABLES command 502
Bonferroni intervals
in MANOVA 1050
Bonferroni test
UNIANOVA command 2082
BOOSTING (keyword)
LINEAR command 980
BOOTSTRAP (command)
CRITERIA subcommand 249
MISSING subcommand 250
SAMPLING subcommand 249
syntax chart 247
VARIABLES subcommand 249
BOOTSTRAP (keyword)
CATPCA command 280
CATREG command 296
BOOTSTRAP (subcommand)
CNLR command 1246
bootstrap estimates
CNLR/NLR command 1246
BOTH (keyword)
IGRAPH command 929
NONPAR CORR command 1261
PLANCARDS command 1427
PPLOT command 1495
PROXSCAL command 1571
SURVIVAL command 1924
BOTH (subcommand
VARIABLE ROLE command 2129
BOTTOM (keyword)
TSPLOT command 2054
BOUNDS (subcommand)
CNLR command 1245
BOX (subcommand)
IGRAPH command 927
SET command 1790
SHOW command 1810
Box-Ljung statistic
ACF command 97
Box's M test
DISCRIMINANT command 592
in MANOVA 1049
BOXBASE (keyword)
IGRAPH command 927
BOXCOX (keyword)

BOXCOX (keyword) *(continued)*
ADP command 118
BOXM (keyword)
DISCRIMINANT command 592
MANOVA command 1049
BOXPLOT (keyword)
EXAMINE command 643
BOXPLOT (subcommand)
SIMRUN command 1837
boxplots
comparing factor levels 641
comparing variables 641
identifying outliers 642
IGRAPH command 927
BP (keyword)
GLM command 850, 2076
BPROP (keyword)
CROSSTABS command 389
BREAK (command)
syntax chart 251
with DO IF command 251
with LOOP command 251
BREAK (keyword)
IGRAPH command 924, 928
BREAK (statement)
MATRIX command 1092
BREAK (subcommand)
AGGREGATE command 124
REPORT command 1682
BREAKDOWN (command). See MEANS 1125
BRESLOW (keyword)
CSCOXREG command 406
KM command 963
Breslow test
KM command 963
Breslow-Day statistic
CROSSTABS command 390
BRIEF (keyword)
MANOVA command 1048
TSET command 2027
BRKSPACE (keyword)
REPORT command 1677
BROWNFORSYTHE (keyword)
ONEWAY command 1338
BSEUCLID (keyword)
CLUSTER command 314
PROXIMITIES command 1559
BSHAPE (keyword)
CLUSTER command 314
PROXIMITIES command 1559
BSTEP (keyword)
COXREG command 371
LOGISTIC REGRESSION command 993
NOMREG command 1252
BTAU (keyword)
CROSSTABS command 390
BTUKEY (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082
BUILD_OPTIONS (subcommand)
GENLINMIXED command 763
LINEAR command 980
BUILDOPTIONS (subcommand)
BUILDOPTIONS (subcommand) *(continued)*
TCM MODEL command 1958
bulk loading 1761
BULKLOADING (subcommand)
SAVE TRANSLATE command 1761
BY (keyword)
ANOVA command 160
CROSSTABS command 388
DATE command 549
GENLOG command 770
LIST command 987
LOGISTIC REGRESSION command 991
LOGLINEAR command 1003
LOOP command 1016
MEANS command 1126
MULT RESPONSE command 1184
NOMREG command 1250
NOMREG subcommand 1252
NPAR TESTS command 1278
PARTIAL CORR command 1417
PROBIT command 1542
RANK command 1594
ROC ANALYSIS command 1718
ROC command 1724
SORT CASES command 1841
SPECTRA command 1894
SPLIT FILE command 1897
SUMMARIZE command 1913
SURVIVAL command 1919
VARCOMP command 2117
WEIGHT command 2147
WEIGHTED KAPPA command 2150
BY (subcommand)
ADD FILES command 108
MATCH FILES command 1064
UPDATE command 2094
BYRULE (keyword)
VALIDATEDATA command 2104
BYVARIABLE (keyword)
VALIDATEDATA command 2104

C

C (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082
C (subcommand)
data organization 1879
SPCHART command 1879
variable specification 1880
c charts
SPCHART command 1879
CACHE (command)
syntax chart 253
CACHE (subcommand)
SET command 1790
SHOW command 1810
caching
active file 1790
CALCULATE (keyword)
EXSMOOTH command 660
CALCULATE (subcommand)
SURVIVAL command 1922

CALL (statement)
MATRIX command 1087
CANONICAL (keyword)
ANACOR command 153
canonical correlation macro 2215
CAPSIGMA (keyword)
SPCHART command 1884
CAPSIGMA (subcommand)
SPCHART command 1886
CAPSTYLE (keyword)
IGRAPH command 929
CAPTION (keyword)
CTABLES command 500
CAPTION (subcommand)
IGRAPH command 922
captions
CTABLES command 500
CAPWIDTH (keyword)
IGRAPH command 927, 929
CARD (keyword)
PLANCARDS command 1427
CASE (keyword)
CROSSTABS command 393
FILE TYPE command 692
FLEISS MULTIRATER KAPPA command 707
PROXIMITIES command 1556, 1557
CASE (subcommand)
FILE TYPE command 690
RECORD TYPE command 1628
case identification variable 1628
case number
system variable $CASENUM 45
case processing summary
MIXED command 1142
case selection 1223
CASECHECKS (subcommand)
VALIDATEDATA command 2103
CASECOUNT (keyword)
CODEBOOK command 326
CASEID (subcommand)
COMPARE DATASETS command 332
CASELABEL (subcommand)
IGRAPH command 921
CASELABELS (subcommand)
KNN command 970
CASELIMIT (keyword)
GGRAPH command 827
VALIDATEDATA command 2104
CASENUM (keyword)
SUMMARIZE command 1914
CASEPLOT (command)
release history 255
syntax chart 255
CASEREPORT (subcommand)
VALIDATEDATA command 2104
cases
excluding from HOMALS command 899
excluding from OVERALS command 1406
limiting 1223
listing 985
sampling 1727
selecting subsets 1223, 1775
sorting 1841
weighting 2147
CASES (keyword)
DISCRIMINANT command 594
MULT RESPONSE command 1186
PLS command 1434
CASES (subcommand)
LIST command 987
Cases to Variables procedure 269
CASESTOVARS (command)
AUTOFIX subcommand 267
COUNT subcommand 267
DROP subcommand 269
FIXED subcommand 267
GROUPBY subcommand 268
ID subcommand 265
INDEX subcommand 266
limitations 263
RENAME subcommand 268
SEPARATOR subcommand 268
syntax chart 263
VIND subcommand 266
with SORT CASES command 265
CASEVALUE (function)
GGRAPH command 823
CASEWISE (subcommand)
LOGISTIC REGRESSION command 997
REGRESSION command 1647
CAT (keyword)
IGRAPH command 924, 928
CATEGORICAL (keyword)
DETECTANOMALY command 579
IGRAPH command 918
SIMRUN command 1836
CATEGORICAL (subcommand)
AIM command 132
COXREG command 370
LOGISTIC REGRESSION command 992
MVA command 1214
TWOSTEP CLUSTER command 2066
Categorical Principal Components Analysis
command syntax 271
Categorical Regression
command syntax 287
categories
showing and hiding empty categories 919
CATEGORIES (keyword)
NPTESTS command 1268
CATEGORIES subcommand
CTABLES command 495
CATEGORY (keyword)
AIM command 133
CATPCA command 282
FLEISS MULTIRATER KAPPA command 707
MULTIPLE CORRESPONDENCE command 1197
category labels
positioning in CTABLES command 494
category order
interactive charts 919
category plots
CATPCA command 282
MULTIPLE CORRESPONDENCE command 1197
category quantifications
CATPCA command 281
MULTIPLE CORRESPONDENCE command 1195
category variables

category variables *(continued)*
CTABLES command 484
CATORDER (subcommand)
IGRAPH command 919
CATPCA (command)
ANALYSIS subcommand 275
CONFIGURATION subcommand 278
CRITITER subcommand 279
DIMENSION subcommand 278
DISCRETIZATION subcommand 276
limitations 272
MAXITER subcommand 279
MISSING subcommand 276
NORMALIZATION subcommand 278
options 272
OUTFILE subcommand 286
PLOT subcommand 282
PRINT subcommand 281
release history 271
RESAMPLE subcommand 280
ROTATION subcommand 279
SAVE subcommand 284
SUPPLEMENTARY subcommand 277
syntax chart 271
syntax rules 272
VARIABLES subcommand 274
CATREG (command)
ANALYSIS subcommand 291
CRITITER subcommand 295
DISCRETIZATION subcommand 292
INITIAL subcommand 294
MAXITER subcommand 295
MISSING subcommand 293
OUTFILE subcommand 298
PLOT subcommand 297
PRINT subcommand 296
REGULARIZATION subcommand 295
release history 287
RESAMPLE subcommand 296
SUPPLEMENTARY subcommand 293
syntax chart 287
VARIABLES subcommand 291, 298
CAUCHIT (keyword)
CSORDINAL command 442
GENLINMIXED command 757
PLUM command 1439
Cauchit link
in Ordinal Regression 1439
Cauchy distribution function 64
CC (keyword)
CROSSTABS command 390
CC (subcommand)
SET command 1791
SHOW command 1810
CCF (command)
APPLY subcommand 302
DIFF subcommand 300
LN/NOLOG subcommands 301
MXCROSS subcommand 302
PERIOD subcommand 301
periodic lags 302
SDIFF subcommand 301
SEASONAL subcommand 302
specifying periodicity 301
CCF (command) *(continued)*
syntax chart 299
transforming values 300
using a previously defined model 302
VARIABLES subcommand 300
CCONF (keyword)
CORRESPONDENCE command 361
CCW (keyword)
IGRAPH command 926
CD (command)
interaction with HOST command 907
release history 305
syntax chart 305
CD (keyword)
INSERT command 947
CDF functions 64
CDF.BERNOULLI (function)
MATRIX command 1082
CDF.BETA (function)
MATRIX command 1082
CDF.BINOM (function)
MATRIX command 1082
CDF.BVNOR (function)
MATRIX command 1082
CDF.CAUCHY (function)
MATRIX command 1082
CDF.CHISQ (function)
MATRIX command 1082
CDF.EXP (function)
MATRIX command 1082
CDF.F (function) 68
CDF.GAMMA (function)
MATRIX command 1082
CDF.GEOM (function)
MATRIX command 1082
CDF.HALFNRM (function)
MATRIX command 1082
CDF.HYPER (function)
MATRIX command 1082
CDF.IGAUSS (function)
MATRIX command 1082
CDF.LAPLACE (function)
MATRIX command 1082
CDF.LNORMAL (function)
MATRIX command 1082
CDF.LOGISTIC (function)
MATRIX command 1082
CDF.NEGBIN (function)
MATRIX command 1082
CDF.NORMAL (function)
MATRIX command 1082
CDF.PARETO (function)
MATRIX command 1082
CDF.POISSON (function)
MATRIX command 1082
CDF.SMOD (function)
MATRIX command 1082
CDF.SRANGE (function)
MATRIX command 1082
CDF.T (function) 68
CDF.UNIFORM (function)
MATRIX command 1082
CDF.WEIBULL (function)
MATRIX command 1082

CDFNORM (function)
MATRIX command 1077
CDFORDER (keyword)
SIMRUN command 1836
CELL (keyword)
CROSSTABS command 393
CELLINFO (keyword)
MANOVA command 1029
PLUM command 1440
CELLPROB (keyword)
NOMREG command 1256
CELLRANGE (subcommand)
GET DATA command 789
CELLS (keyword)
CROSSTABS command 393
CELLS (subcommand)
CROSSTABS command 389
CSTABULATE command 477
MATRIX DATA command 1116
MEANS command 1126
MULT RESPONSE command 1186
OLAP CUBES command 1296
SUMMARIZE command 1913
CELLSBREAK (subcommand)
SET command 1792
censored cases
KM command 961
CENTER (keyword)
REPORT command 1680, 1683, 1690
CENTER (subcommand)
SPECTRA command 1892
CENTERED (keyword)
SEASON command 1773
centered moving average function 381
centered running median function 382
centering transformation
SPECTRA command 1892
CENTR (keyword)
CATPCA command 284
with BIPLOT keyword 284
CENTROID (keyword)
CLUSTER command 317
OVERALS command 1407
centroid method
CLUSTER command 317
centroid plots
OVERALS command 1407
CENTROIDS (keyword)
PREFSCAL command 1509
CFVAR (function) 63
CGT (function)
AGGREGATE command 125
CHAID (subcommand)
TREE command 2004
CHALIGN (keyword)
REPORT command 1677
change
arithmetic and percentage change between groups and variables 1297
CHANGE (keyword)
REGRESSION command 1640
changing the working directory 305, 947
CHAR.INDEX (function) 78
CHAR.LENGTH (function) 78
CHAR.LPAD (function) 78
CHAR.MBLEN (function) 78
CHAR.RINDEX (function) 78
CHAR.RPAD (function) 78
CHAR.SUBSTR (function) 78
character encoding
CSV files 1755
SAS files 796, 1755
Stata files 802
tab-delimited text data files 1755
CHART (subcommand)
XGRAPH command 2168
chart templates 1792
CHARTFORMAT (keyword)
OMS command 1311
CHARTLOOK (subcommand)
IGRAPH command 922
charts
3-D bar 2176
bar 880
clustering 2171
count functions 823, 877, 2168
data element types 2170
difference area 881
dot plots 2179
drop-line 881
error bar 882
error bars 885, 2173
error functions 823
high-low 881
histograms 883
line 881
measurement level 2170
paneled charts 883, 2174
Pareto 883
pie 881
population pyramids 2177
range bar 880
ROC Curve 1725
scatterplots 882
stacking 2171
summary functions 823, 877, 2168
templates 514, 885, 1340, 1498, 1650, 1872, 2175
CHARTSIZE (keyword)
OMS command 1310
CHDSPACE (keyword)
REPORT command 1677
CHEBYCHEV (keyword)
CLUSTER command 313
PROXIMITIES command 1558
Chebychev distance
CLUSTER command 313
PROXIMITIES command 1558
CHECKSEP (keyword)
GENLIN command 732
chi-square
Cochran 1655
CROSSTABS command 390
CSGLM command 424
CSLOGISTIC command 436
distance measure 314, 1558
Friedman 1655
chi-square distance
CORRESPONDENCE command 359

chi-square distribution function 64
CHISQ (keyword)
CLUSTER command 314
CORRESPONDENCE command 359
CROSSTABS command 390
PROXIMITIES command 1558
CHISQUARE (keyword)
CSCOXREG command 404
CSGLM command 424
CSLOGISTIC command 436
CSORDINAL command 447
CTABLES command 501
NPTESTS command 1268
CHISQUARE (subcommand)
NPAR TESTS command 1280
CHKSEP (keyword)
CSLOGISTIC command 435
CSORDINAL command 446
NOMREG command 1251
CHOL (function)
MATRIX command 1077
CHWRAP (keyword)
REPORT command 1677
CI (keyword)
AIM command 132
COXREG command 373
CSCOXREG command 406, 408
GRAPH command 882, 885
IGRAPH command 929
LOGISTIC REGRESSION command 996
PROBIT command 1545
REGRESSION command 1640
XGRAPH command 2173
CI (subcommand)
CORRELATIONS command 353
NONPAR CORR command 1263
CILEVEL (keyword)
BOOTSTRAP command 249
CORRELATIONS command 353
CRITERIA subcommand 1340
CSCOXREG command 403
CSGLM command 424
CSLOGISTIC command 435
CSORDINAL command 446
GENLIN command 732
NPTESTS command 1267, 1271
SIMPLAN command 1828
CIMEANPREDL (keyword)
GENLIN command 746
CIMEANPREDU (keyword)
GENLIN command 746
CIN (keyword)
CROSSTABS command 391
CSDESCRIPTIVES command 414
CSTABULATE command 477
CURVEFIT command 512
GENLOG command 773
MIXED command 1138
NOMREG command 1251
NPAR TESTS command 1292
PLUM command 1438
RATIO STATISTICS command 1601, 1602
REGRESSION command 1642
CIN (subcommand)
CIN (subcommand) *(continued)*
CURVEFIT command 512
CINPCT (subcommand)
ARIMA command 184
CINTERVAL (keyword)
CSCOXREG command 404
CSGLM command 424
CSLOGISTIC command 435
CSORDINAL command 447
CINTERVAL (subcommand)
EXAMINE command 644
MANOVA command 1034, 1050
city-block distance
CLUSTER command 313
PROXIMITIES command 1558
CITYBLOCK (keyword)
KNN command 972
CITYPE (keyword)
BOOTSTRAP command 249
GENLIN command 732
CITYPES (keyword)
INDEPENDENTSAMPLES command 1552
PAIREDSAMPLES command 1552
PROPORTIONS command 1551
CJUMP (keyword)
IGRAPH command 928
CKDER (keyword)
CNLR/NLR command 1242
CLABELS (command)
CATEGORIES subcommand 495
CLABELS (keyword)
MATRIX command 1089
CLABELS (subcommand)
CTABLES command 494
CLASS (keyword)
DISCRIMINANT command 590
CLASSICAL (keyword)
PREFSCAL command 1509
CLASSIFICATION (keyword)
MLP command 1161
NAIVEBAYES command 1231
RBF command 1610
classification plots
LOGISTIC REGRESSION command 997
classification tables
CSLOGISTIC command 437
DISCRIMINANT command 592
LOGISTIC REGRESSION command 996
TREE command 1997
CLASSIFY (keyword)
QUICK CLUSTER command 1589
CLASSIFY (subcommand)
DISCRIMINANT command 593
CLASSMISSING (keyword)
CSCOXREG command 405
CSDESCRIPTIVES command 414
CSGLM command 425
CSLOGISTIC command 436
CSORDINAL command 448
FLEISS MULTIRATER KAPPA command 706
GENLIN command 744
SIMPLAN command 1829
WEIGHTED KAPPA command 2151
CLASSMISSING (subcommand)

CLASSMISSING (subcommand) *(continued)*
CSSELECT command 470
CLASSPLOT (subcommand)
LOGISTIC REGRESSION command 997
CLASSTABLE (keyword)
CSLOGISTIC command 437
CSORDINAL command 449
NOMREG command 1256
CLEAR (subcommand)
INSERT EXEC command 949
CLEAR TIME PROGRAM (command)
syntax chart 307
CLEAR TRANSFORMATIONS (command)
syntax chart 309
CLOGLOG (keyword)
CSORDINAL command 442
GENLIN command 728
GENLINMIXED command 757
PLUM command 1439
CLOPPERPEARSON (keyword)
NPTESTS command 1268
CLS (keyword)
ARIMA command 185
CLT (function)
AGGREGATE command 125
CLUSTER (command)
compared with QUICK CLUSTER command 1587
distance measures 313
ID subcommand 318
labeling cases 318
limitations 311
MATRIX subcommand 319
MEASURE subcommand 313
measures for binary data 314
measures for frequency-count data 314
MISSING subcommand 319
missing values 319, 321
PLOT subcommand 319
PRINT subcommand 318
SAVE subcommand 317
saving cluster memberships 317
statistics 318
syntax chart 311
variable list 313
CLUSTER (keyword)
AIM command 133
CLUSTER command 318
IGRAPH command 920, 926
QUICK CLUSTER command 1588, 1590
TWOSTEP CLUSTER command 2070
cluster membership
CLUSTER command 318
CMAX (function)
MATRIX command 1077
CMEAN (keyword)
CORRESPONDENCE command 360
CMH (keyword)
CROSSTABS command 390
CMIN (function)
MATRIX command 1077
CMPTRANS (subcommand)
SET command 1792
CNAMES (keyword)
MATRIX command 1089
CNLR (command)
bootstrap estimates 1246
BOOTSTRAP subcommand 1246
BOUNDS subcommand 1245
constrained functions 1239
constraints 1245
crash tolerance 1243
CRITERIA subcommand 1242
critical value for derivative checking 1242
dependent variable 1240
derivatives 1238
DERIVATIVES command 1235, 1238
feasibility tolerance 1243
FILE subcommand 1240
function precision 1243
infinite step size 1243
iteration criteria 1243
Levenberg-Marquardt method 1244
line-search tolerance 1243
linear constraint 1245
linear constraints 1245
linear feasibility tolerance 1243
loss function 1245
LOSS subcommand 1245
major iterations 1243
maximum iterations 1243, 1244
minor iterations 1243
model expression 1238
model program 1238
nonlinear constraint 1245
nonlinear constraints 1245
nonlinear feasibility tolerance 1243
optimality tolerance 1243
OUTFILE subcommand 1240
parameter constraints 1245
parameter convergence 1244
parameters 1238
PRED subcommand 1241
residual and derivative correlation convergence 1244
SAVE subcommand 1241
saving new variables 1241
saving parameter estimates 1240
sequential quadratic programming 1243
simple bounds 1245
step limit 1243
sum-of-squares convergence 1244
syntax chart 1235
using parameter estimates from previous analysis 1240
weighting cases 1237
with CONSTRAINED FUNCTIONS command 1235, 1239
with MODEL PROGRAM command 1235, 1238
CNT (function)
AGGREGATE command 125
CO (keyword)
AREG command 175
COCHRAN (keyword)
NPTESTS command 1273
RELIABILITY command 1655
COCHRAN (subcommand)
NPAR TESTS command 1281
Cochran's statistic
CROSSTABS command 390
Cochrane-Orcutt method
AREG command 175

COD (keyword)
RATIO STATISTICS command 1601, 1602
CODEBOOK (command)
dictionary information 325
FILEINFO subcommand 326
measurement level 325, 327
OPTIONS subcommand 327
sorting options 327
STATISTICS subcommand 327
summary statistics 325, 327
syntax chart 323
variables 325
VARINFO subcommand 325
COEFF (keyword)
CATREG command 296
COXREG command 375
DISCRIMINANT command 592, 593
REGRESSION command 1640
coefficient of concentration
RATIO STATISTICS command 1601, 1602
coefficient of dispersion (COD)
RATIO STATISTICS command 1601, 1602
coefficient of variation (COV)
RATIO STATISTICS command 1601, 1602
Cognos
exporting to Cognos TM1 1765
reading Cognos data 811
reading Cognos TM1 data 815
Cohen's kappa
CROSSTABS command 390
Cohen's Weighted Kappa 2149
COINCIDENT (keyword)
IGRAPH command 924
COLCONF (keyword)
ALSCAL command 139, 143
collapsing table categories
CTABLES command 496
COLLECT (keyword)
REGRESSION command 1637
COLLIN (keyword)
REGRESSION command 1640
COLLINEARITY (keyword)
MANOVA command 1031
COLOP (keyword)
GRAPH command 884
XGRAPH command 2175
COLOR (subcommand)
IGRAPH command 920
COLORCYCLE (subcommand)
GGRAPH command 833
COLORS (keyword)
PREFSCAL command 1516
COLPCT (keyword)
CSTABULATE command 477
COLSPACE (keyword)
REPORT command 1677
COLUMN (keyword)
CROSSTABS command 389
MULT RESPONSE command 1186
PREFSCAL command 1512
COLUMN (subcommand)
REREAD command 1705
column headings
See also page ejection 1535
column percentages
CROSSTABS (command) 389
column width
CTABLES command 504
column-style format specifications 526
COLUMNS (keyword)
ANACOR command 154
COLUMNS (subcommand)
OMS command 1313
COLUMNWISE (keyword)
AGGREGATE command 127, 128
COLVAR (keyword)
GRAPH command 883
XGRAPH command 2174
COMBINED (keyword)
DISCRIMINANT command 594
COMBINING_RULE_CONTINUOUS (keyword)
LINEAR command 982
COMM (keyword)
EXPORT command 651
IMPORT command 934
comma
delimiter 36
COMMA format 50
comma-delimited files 1750
COMMAND (keyword)
EXTENSION command 664
READ MODEL command 1616
SAVE MODEL command 1744
TDISPLAY command 1969
command files 39, 937
command order 36, 2187
command syntax
index of all commands 1
Unicode 938, 947, 950, 952
COMMENT (command)
syntax chart 329
COMMENT (keyword)
SIMPLAN command 1820
COMMON (keyword)
PREFSCAL command 1514, 1515, 1517
PROXSCAL command 1576–1578
common space
PROXSCAL command 1576
common space plots
PROXSCAL command 1577
communality
FACTOR command 669
COMPARE (keyword)
GENLIN command 741
GENLINMIXED command 764
MIXED command 1140
NPTESTS command 1271, 1273
SURVIVAL command 1922
COMPARE (subcommand)
EXAMINE command 641
KM command 964
SURVIVAL command 1922
COMPARE DATASETS (command)
CASEID subcommand 332
COMPDATASET subcommand 332
OUTPUT subcommand 334
release history 331
SAVE subcommand 332

COMPARE DATASETS (command) *(continued)*
syntax chart 331
VARIABLES subcommand 332
COMPARETEST (subcommand)
CTABLES command 502
comparing datasets 331
COMPDATASET (subcommand)
COMPARE DATASETS command 332
complementary log-log link
PLUM command 1439
COMPLETE (keyword)
CLUSTER command 317
complex data files
case identification variable 1628
defining 1625
duplicate records 1629
grouped files 1625
missing records 1629
mixed files 1625
nested files 1625
repeating groups 1625
skipping records 1628
spreading values across cases 1630
undefined records 1627
complex files
defining 617, 630
complex raw data files
grouped 688
mixed 688
nested 688
Complex Samples Cox Regression
command syntax 397
Complex Samples Crosstabs
command syntax 475
Complex Samples Descriptives
command syntax 411
Complex Samples Frequencies
command syntax 475
Complex Samples General Linear Model
command syntax 417
Complex Samples Logistic Regression
command syntax 427
missing values 436
saving new variables 437
component loadings
CATPCA command 281
component loadings plots
CATPCA command 282
COMPONENT_MODELS_N (keyword)
LINEAR command 982
COMPOUND (keyword)
CURVEFIT command 511
compound model
CURVEFIT command 509
COMPRESSED (subcommand)
SAVE command 1733
SAVE TRANSLATE command 1763
XSAVE command 2184
COMPUTE (command)
defining cross-variable rules 2107
defining single-variable rules 2105
functions 335
missing values 336
syntax chart 335
COMPUTE (command) *(continued)*
with DO IF command 337
with STRING command 336, 338, 339
COMPUTE (statement)
MATRIX command 1076
computing values
arithmetic functions 337
arithmetic operators 337
assignment expression 335
conditional expressions 612, 909
formats of new variables 336
functions 335
if case satisfies condition 909
logical expressions 612, 909
logical operators 611, 909
missing values 336
missing-value functions 338
relational operators 611, 909
scoring functions 339
statistical functions 337
string functions 338, 339
string variables 335, 336
syntax rules 335
target variable 335
CON (subcommand)
ARIMA command 183
CONCAT (function) 78
concatenation
CTABLES command 485
CONDENSE (keyword)
MULT RESPONSE command 1187
RANK command 1596
CONDENSED (keyword)
PARTIAL CORR command 1419
CONDITION (subcommand)
ALSCAL command 139
PREFSCAL command 1510
PROXSCAL command 1572
condition index
REGRESSION command 1640
CONDITIONAL (keyword)
COXREG command 371
MANOVA command 1051
SURVIVAL command 1922
conditional expressions 612
conditional independence test
CROSSTABS command 390
conditional probability
CLUSTER command 314
PROXIMITIES command 1559
conditional statistic
COXREG command 371
LOGISTIC REGRESSION command 993
conditional transformations
conditional expressions 612, 909
formats of new variables 614, 912
logical expressions 612, 909
logical operators 611, 909
missing values 614, 912
nested 617
relational operators 611, 909
string variables 612, 614, 909, 912
conditionality
matrix 139

conditionality *(continued)*
row 139
unconditional data 139
CONFIDENCE (keyword)
GENLINMIXED command 766
confidence intervals
ARIMA command 184
COXREG command 373
CSGLM command 424
CSLOGISTIC command 435
CURVEFIT command 512
EXAMINE command 644
GENLOG command 773
IGRAPH command 929
in MANOVA 1050
MIXED command 1138
PROBIT command 1545
RATIO STATISTICS command 1601, 1602
REGRESSION command 1634, 1640, 1642
ROC command 1725
TSAPPLY command 2021
TSMODEL command 2040
UNIANOVA command 2076
CONFIDENCE_INTERVALS (keyword)
GENLINMIXED command 766
CONFIDENCE_LEVEL (keyword)
GENLINMIXED command 763
LINEAR command 980
CONFIDENCE_METHOD (keyword)
GENLINMIXED command 766
CONFIG (keyword)
ALSCAL command 139, 143
CONFIGURATION (subcommand)
CATPCA command 278
MULTIPLE CORRESPONDENCE command 1194
CONFORM (subcommand)
SPCHART command 1887
confusion matrix
DISCRIMINANT command 592
CONJOINT (command)
DATA subcommand 344
FACTORS subcommand 345
PLAN subcommand 343
PRINT subcommand 347
RANK subcommand 344
SCORE subcommand 344
SEQUENCE subcommand 344
SUBJECT subcommand 345
syntax chart 341
UTILITY subcommand 347
with ORTHOPLAN command 341, 343
CONNECT (subcommand)
GET CAPTURE command 784
GET DATA command 787
SAVE TRANSLATE command 1759
connecting to a IBM SPSS Collaboration and Deployment Services Repository 1697
CONNECTION (subcommand)
GETCOGNOS command 812
GETTM1 command 816
SAVETM1 command 1766
connection pooling 787
CONSTANT (keyword)
ARIMA command 181
CONSTANT (keyword) *(continued)*
MANOVA command 1042
CONSTANT (subcommand)
2SLS command 95
AREG command 175
CURVEFIT command 512
WLS command 2156
constants 59
CONSTRAIN (keyword)
ALSCAL command 141
CONSTRAINED FUNCTIONS (command)
with CNLR command 1235, 1239
CONSTRAINTS (subcommand)
MULTIPLE IMPUTATION command 1206
CONSTRUCTION (keyword)
ADP command 119
contained effects
in GLM 849
UNIANOVA command 2075
CONTENTS (subcommand)
MATRIX DATA command 1117
OUTPUT EXPORT command 1360
CONTINGENCY (subcommand)
SIMPLAN command 1827
contingency coefficient
CROSSTABS command 390
CONTINUED (subcommand)
REPEATING DATA command 1670
CONTINUOUS (subcommand)
AIM command 132
TWOSTEP CLUSTER command 2067
CONTRAST (keyword)
CSGLM command 422
ES subcommand 1341
GENLIN command 741
GENLINMIXED command 764
MANOVA command 1045
CONTRAST (subcommand)
COXREG command 370
GLM command 842, 855
LOGISTIC REGRESSION command 992
LOGLINEAR command 1005
MANOVA command 1026, 1056
ONEWAY command 1335
UNIANOVA command 2081
contrast coefficients
in GLM 850
UNIANOVA command 2076
contrast coefficients matrix
CSGLM command 425
CSLOGISTIC command 437
UNIANOVA command 2079
contrast results matrix 841, 854, 2080
contrasts
analysis of variance 1335
CSGLM command 423
custom 842
deviation 855
difference 855, 872
for within-subjects factors 1056
Helmert 855, 872
in GLM 855
in MANOVA 1045
orthogonal 855, 2081

contrasts *(continued)*
polynomial 855, 872
repeated 855, 872
reverse Helmert 855
simple 855, 872
special 855, 872
UNIANOVA command 2081
within-subjects factor 871
WSFACTOR 872
CONTRIBUTIONS (keyword)
ANACOR command 154
CONTROL (keyword)
CSLOGISTIC command 433
CSORDINAL command 444
GENLIN command 741
GENLINMIXED command 765
control charts
command syntax 1869
CONVERGE (keyword)
ALSCAL command 141
HILOGLINEAR command 892
MVA command 1219
PROBIT command 1544
QUICK CLUSTER command 1588
VARCOMP command 2116
CONVERGENCE (subcommand)
HOMALS command 900
OVERALS command 1406
PRINCALS command 1524
convergence criterion
ALSCAL command 141
FACTOR command 671
QUICK CLUSTER command 1588
conversion functions 81
CONVERT (keyword)
RECODE command 1622
COOK (keyword)
GENLIN command 746
GLM command 860
LOGISTIC REGRESSION command 997
REGRESSION command 1634
UNIANOVA command 2086
Cook's D
LOGISTIC REGRESSION command 997
Cook's distance
REGRESSION command 1634
UNIANOVA command 2086
COORDINATE (subcommand)
IGRAPH command 921
XGRAPH command 2173
COORDINATES (keyword)
PREFSCAL command 1512
PROXSCAL command 1574
ROC ANALYSIS command 1720
ROC command 1725
WEIGHTED KAPPA command 2150
copying variable definition attributes from other variables in current or external data file 171
COR (keyword)
MANOVA command 1048
CORB (keyword)
CSCOXREG command 406, 409
CSGLM command 425, 426
CSLOGISTIC command 437, 438
CORB (keyword) *(continued)*
CSORDINAL command 449, 450
GENLIN command 744, 748
MIXED command 1142
NOMREG command 1256
PLUM command 1440
VARCOMP command 2116
CORNER (keyword)
CTABLES command 500
corner text
CTABLES command 500
CORR (keyword)
CATPCA command 281
CATREG command 296
COXREG command 373
CROSSTABS command 390
DISCRIMINANT command 592
LOGISTIC REGRESSION command 996
MATRIX DATA command 1117
MULTIPLE CORRESPONDENCE command 1195
PARTIAL CORR command 1419
CORRELATION (keyword)
CLUSTER command 313
FACTOR command 669
PRINCALS command 1524
PROXIMITIES command 1558
REGRESSION command 1643
correlation coefficients 349
correlation matrix
CATPCA command 281
CSGLM command 425
CSLOGISTIC command 437
GENLOG command 773
LOGISTIC REGRESSION command 996
LOGLINEAR command 1007
MIXED command 1142
pooled within-groups 592
correlations
MULTIPLE CORRESPONDENCE command 1195
NONPAR CORR command 1259
PROXSCAL command 1576
REGRESSION command 1640, 1643
CORRELATIONS (command)
CI subcommand 353
limitations 349
matrix output 352
MATRIX subcommand 352
MISSING subcommand 351
PRINT subcommand 351
significance tests 351
STATISTICS subcommand 351
syntax chart 349
with REGRESSION command 1645
CORRELATIONS (keyword)
GENLIN command 741
PROXSCAL command 1576, 1577
RELIABILITY command 1655, 1656
CORRELATIONS (subcommand)
SIMPLAN command 1826
correlations plots
PROXSCAL command 1577
CORRESPONDENCE (command)
DIMENSION subcommand 358
dimensions 358

CORRESPONDENCE (command) *(continued)*
distance measures 359
EQUAL subcommand 359
equality constraints 359
MEASURE subcommand 359
normalization 360
NORMALIZATION subcommand 360
OUTFILE subcommand 362
PLOT subcommand 361
plots 361
PRINT subcommand 361
release history 355
standardization 360
STANDARDIZE subcommand 360
supplementary points 359
SUPPLEMENTARY subcommand 359
syntax chart 355
TABLE subcommand 356
CORRESPONDENCE (keyword)
PREFSCAL command 1509
Correspondence Analysis
command syntax 355
CORRMATRIX (keyword)
SIMPLAN command 1826
CORRTYPE (keyword)
GENLIN command 737
COS (function)
MATRIX command 1077
COS (keyword)
SPECTRA command 1895
cosine
CLUSTER command 313
PROXIMITIES command 1558
COSINE (keyword)
CLUSTER command 313
PROXIMITIES command 1558
cosine function values
saving with SPECTRA command 1895
cospectral density estimate plot
SPECTRA command 1893
cospectral density estimates
saving with SPECTRA command 1895
COSTS (subcommand)
TREE command 2007
COUNT (command)
missing values 365
syntax chart 365
COUNT (function)
GGRAPH command 823
GRAPH command 877
XGRAPH command 2168
COUNT (keyword)
CODEBOOK (command) 327
CROSSTABS command 389
CSDESCRIPTIVES command 414
CSTABULATE command 477
MATRIX DATA command 1117
MEANS command 1126
MULT RESPONSE command 1186
OLAP CUBES command 1296
SUMMARIZE command 1913
TWOSTEP CLUSTER command 2069
COUNT (subcommand)
CASESTOVARS command 267
COUNT (subcommand) *(continued)*
VARSTOCASES command 2137
COUNTCI (function) MEDIANCI (function)
GGRAPH command 823
COUNTDUPLICATES (keyword)
CTABLES command 505
counting occurrences
defining values 365
missing values 365
COUNTS (keyword)
MVA command 1215
COUT (function)
AGGREGATE command 125
COV (keyword)
DISCRIMINANT command 592
MANOVA command 1048
MATRIX DATA command 1117
REGRESSION command 1643
covariance
REGRESSION command 1640, 1643
RELIABILITY command 1655, 1656
COVARIANCE (keyword)
FACTOR command 669
RELIABILITY command 1656
covariance matrix
2SLS command 95
CSGLM command 425
CSLOGISTIC command 437
GENLOG command 773
MIXED command 1142
pooled within-groups 592, 593
separate-groups 592, 593
total 592
covariance method
RELIABILITY command 1656
covariance ratio
REGRESSION command 1634
COVARIANCE_TYPE (keyword)
GENLINMIXED command 755, 761
COVARIANCES (keyword)
GENLIN command 741
RELIABILITY command 1655, 1656
COVARIATE (keyword)
CSLOGISTIC command 433
CSORDINAL command 444
KNN command 971
MLP command 1153
RBF command 1607
COVARIATES (keyword)
NAIVEBAYES command 1229
COVARIATES (subcommand)
ANOVA command 161
COVB (keyword)
CSCOXREG command 405, 406, 409
CSGLM command 425, 426
CSLOGISTIC command 437, 438
CSORDINAL command 447, 449, 450
GENLIN command 732, 737, 744, 748
MIXED command 1142
NOMREG command 1256
PLUM command 1440
VARCOMP command 2116
COVRATIO (keyword)
REGRESSION command 1634

COVTYPE (keyword)
MIXED command 1143, 1144
Cox Regression
command syntax 367
COXREG (command)
categorical covariates 370
CATEGORICAL subcommand 370
CONTRAST subcommand 370
contrasts 370
CRITERIA subcommand 373
display options 373
EXTERNAL subcommand 376
iteration criteria 373
limitations 368
method 371
METHOD subcommand 371
MISSING subcommand 373
missing values 373
OUTFILE subcommand 375
PATTERN subcommand 374
PLOT subcommand 374
plots 374
PRINT subcommand 373
SAVE subcommand 375
saving new variables 375
split-file processing 376
STATUS subcommand 369
STRATA subcommand 370
stratification variable 370
survival status variable 369
syntax chart 367
VARIABLES subcommand 369
COXSNELL (keyword)
CSCOXREG command 407
CP (keyword)
SPCHART command 1883
Cp. See Mallow's Cp 1640
CPCHART (subcommand)
data organization 1881
SPCHART command 1880
variable specification 1882
CPK (keyword)
SPCHART command 1883
CPL (keyword)
SPCHART command 1883
CPM (keyword)
SPCHART command 1883
CPOINTS (keyword)
CORRESPONDENCE command 361
CPRINCIPAL (keyword)
ANACOR command 153
CORRESPONDENCE command 360
CPROFILES (keyword)
CORRESPONDENCE command 361
CPS (keyword)
CSSELECT command 473
DETECTANOMALY command 582
GENLIN command 744
KNN command 975
MIXED command 1142
MLP command 1161
NAIVEBAYES command 1231
NOMREG command 1256
RBF command 1610
CPS (keyword) *(continued)*
SELECTPRED command 1783
CPU (keyword)
SPCHART command 1883
CR (keyword)
SPCHART command 1883
Cramér's V
CROSSTABS command 390
CRAMERSV (keyword)
SELECTPRED command 1782, 1783
CREATE (command)
CSUM function 379
DIFF function 379
FFT function 379
IFFT function 380
LAG function 380
LEAD function 381
MA function 381
PMA function 382
RMED function 382
SDIFF function 383
syntax chart 377
T4253H function 383
CREATE (subcommand)
OLAP CUBES command 1297
CREATEMISPROPVAR (keyword)
DETECTANOMALY command 580
CRITERIA (subcommand)
ADP command 120
AIM command 132
ALSCAL command 141
BAYES ANOVA command 194
BAYES CORRELATION command 199
BAYES INDEPENDENT command 202
BAYES LOGLINEAR command 206
BAYES ONESAMPLE: Binomial command 211
BAYES ONESAMPLE: Normal command 214
BAYES ONESAMPLE: Poisson command 218
BAYES REGRESSION command 223, 1582
BAYES RELATED command 229
BAYES REPEATED command 234
BOOTSTRAP command 249
CNLR command 1242
COXREG command 373
CSCOXREG command 403
CSGLM command 424
CSLOGISTIC command 435
CSORDINAL command 446
CSSELECT command 469
CTABLES command 499
DETECTANOMALY command 580
FLEISS MULTIRATER KAPPA command 706
GENLIN command 732
GENLOG command 773
GLM command 850
HILOGLINEAR command 892
KNN command 973
LOGISTIC REGRESSION command 996
LOGLINEAR command 1007
MIXED command 1138
MLP command 1157
NAIVEBAYES command 1230
NLR command 1242, 1244
NOMREG command 1251

CRITERIA (subcommand) *(continued)*
NPTESTS command 1267
ONEWAY command 1340
OPTIMAL BINNING command 1345
PLS command 1435
PLUM command 1438
PREFSCAL command 1513
PROBIT command 1544
PROPORTIONS command 1554
PROXSCAL command 1576
RBF command 1609
REGRESSION command 1641
ROC ANALYSIS command 1719
ROC command 1724
SELECTPRED command 1782
SIMRUN command 1835
TWOSTEP CLUSTER command 2067
UNIANOVA command 2076
VARCOMP command 2116
WEIGHTED KAPPA command 2150
CRITERIA_BEST_SUBSETS (keyword)
LINEAR command 980
CRITERIA_FORWARD_STEPWISE (keyword)
LINEAR command 980
CRITITER (subcommand)
CATPCA command 279
CATREG command 295
MULTIPLE CORRESPONDENCE command 1195
CROSS (subcommand)
SPECTRA command 1894
cross-amplitude plot
SPECTRA command 1893
cross-amplitude values
saving with SPECTRA command 1895
cross-correlations
command syntax 299
cross-periodogram values
saving with SPECTRA command 1895
cross-product deviation
REGRESSION command 1643
cross-variable rules
defining 2107
CROSSTAB (subcommand)
MVA command 1216
CROSSTABS (command)
BARCHART subcommand 393
cell percentages 389
CELLS subcommand 389
CMH keyword 390
COUNT subcommand 393
exact tests 391
expected count 389
FORMAT subcommand 393
general mode 388
HIDESMALLCOUNTS subcommand 395
integer mode 388
limitations 386
METHOD subcommand 391
MISSING subcommand 392
residuals 389
SHOWDIM subcommand 395
STATISTICS subcommand 390
syntax chart 385
TABLES subcommand 388
CROSSTABS (command) *(continued)*
VARIABLES subcommand 387
with PROCEDURE OUTPUT command 393, 1547
with WEIGHT command 394
WRITE subcommand 393
crosstabulation
multiple response 1184
MVA command 1216
writing to a file 1547
CROSSVAL (keyword)
CATREG command 296
CROSSVALID (keyword)
DISCRIMINANT command 592
CROSSVALIDATION (subcommand)
KNN command 974
CROSSVARRULES (keyword)
VALIDATEDATA command 2102
CRSHTOL (keyword)
CNLR command 1243
CRT (subcommand)
TREE command 2006
CS (keyword)
MIXED command 1137
SPECTRA command 1893, 1895
CSCOXREG (command)
CRITERIA subcommand 403
CUSTOM subcommand 402
DOMAIN subcommand 405
JOINTPROB subcommand 401
MISSING subcommand 405
MODEL subcommand 401
OUTFILE subcommand 409
PATTERN subcommand 408
PLAN subcommand 401
PLOT subcommand 408
PRINT subcommand 406
SAVE subcommand 407
STATISTICS subcommand 404
SURVIVALMETHOD subcommand 406
syntax chart 397
TEST subcommand 404
TESTASSUMPTIONS subcommand 405
VARIABLES subcommand 400
CSDESCRIPTIVES (command)
JOINTPROB subcommand 412
MEAN subcommand 412
MISSING subcommand 414
PLAN subcommand 412
RATIO subcommand 413
STATISTICS subcommand 414
SUBPOP subcommand 414
SUM subcommand 413
SUMMARY subcommand 412
syntax chart 411
CSGLM (command)
CRITERIA subcommand 424
CUSTOM subcommand 420
DOMAIN subcommand 425
EMMEANS subcommand 422
export IBM SPSS Statistics data format 426
export XML format 426
INTERCEPT subcommand 420
JOINTPROB subcommand 419
MISSING subcommand 425

CSGLM (command) *(continued)*
missing values 425
MODEL subcommand 419
OUTFILE subcommand 426
Overview 418
PLAN subcommand 419
PRINT subcommand 425
release history 417
SAVE subcommand 426
saving new variables 426
STATISTICS subcommand 424
syntax chart 417
TEST subcommand 424
CSH (keyword)
MIXED command 1137
CSLOGISTIC (command)
CRITERIA subcommand 435
CUSTOM subcommand 431
DOMAIN subcommand 436
export to IBM SPSS Statistics data file 438
export to XML 438
INTERCEPT subcommand 430
JOINTPROB subcommand 429
MISSING subcommand 436
MODEL subcommand 430
ODDSRATIOS subcommand 433
OUTFILE subcommand 438
PLAN subcommand 429
PRINT subcommand 437
release history 427
SAVE subcommand 437
STATISTICS subcommand 435
syntax chart 427
TEST subcommand 436
CSORDINAL (command)
CRITERIA subcommand 446
CUSTOM subcommand 443
DOMAIN subcommand 448
JOINTPROB subcommand 441
LINK subcommand 442
MISSING subcommand 448
MODEL subcommand 442
NONPARALLEL subcommand 447
ODDSRATIOS subcommand 444
OUTFILE subcommand 450
PLAN subcommand 441
PRINT subcommand 449
release history 439
SAVE subcommand 449
STATISTICS subcommand 447
syntax chart 439
TEST subcommand 447
variable list 441
CSPLAN (command)
DESIGN subcommand 458
ESTIMATOR subcommand 463
INCLPROB subcommand 464
METHOD subcommand 459
MOS subcommand 462
PLAN subcommand 457
PLANVARS subcommand 457
POPSIZE subcommand 464
PRINT subcommand 458
RATE subcommand 461
CSPLAN (command) *(continued)*
SIZE subcommand 460
SRSESTIMATOR subcommand 458
STAGEVARS subcommand 462
syntax chart 451
CSR (keyword)
MIXED command 1137
CSSELECT (command)
CLASSMISSING subcommand 470
CRITERIA subcommand 469
DATA subcommand 470
JOINTPROB subcommand 471
PLAN subcommand 469
PRINT subcommand 473
SAMPLEFILE subcommand 470
SELECTRULE subcommand 472
syntax chart 467
CSSQ (function)
MATRIX command 1077
CSTABULATE (command)
CELLS subcommand 477
JOINTPROB subcommand 476
MISSING subcommand 478
PLAN subcommand 476
STATISTICS subcommand 477
SUBPOP subcommand 478
syntax chart 475
TABLES subcommand 476
TEST subcommand 477
CSTEP (keyword)
IGRAPH command 924, 928
CSTRUCTURE (subcommand)
GENLOG command 771
CSUM (function)
CREATE command 379
MATRIX command 1077
CSUM (keyword)
CORRESPONDENCE command 360
CSV format
reading data 792
saving data 1747, 1750, 1757
CTABLES
converting TABLES command syntax to CTABLES 2221
CTABLES (command)
caption lines 500
category label positioning 494
category variables 484
CLABELS subcommand 494
collapsing table categories 496
column width 504
COMPARETEST subcommand 502
computed categories 506
concatenation 485
corner text 500
CRITERIA sucommand 499
dates in titles 500
empty categories 499
empty cell format 504
empty cells 504
excluding valid values 496
explicit category specification 496
FORMAT subcommand 504
formats for summaries 493
HIDESMALLCOUNTS subcommand 508

CTABLES (command) *(continued)*
missing summaries 504
missing values 493, 505
MRSETS subcommand 505
multiple response functions 486
multiple response sets 484, 505
nesting 485
overview 481
PCOMPUTE subcommand 506
percentage functions 486
position of totals 498
postcomputes 506
PPROPERTIES subcommand 507
release history 479
scale variable functions 486
scale variable totals 498
scale variables 486
SIGTEST subcommand 501
SLABELS subcommand 494
SMISSING subcommand 505
sorting categories 497
split-file processing 481
stacking 485
subtotals 496
summary functions 486
summary functions for multiple response sets 486
summary functions for scale variables 486
summary label positioning 494
summary specifications 486
syntax chart 479
syntax conventions 482
table description in titles 500
table expression 484
TABLE subcommand 484
TITLE keyword 500
TITLES subcommand 500
totals 498
unweighted functions 486
variable labels 504
variable types 484
VLABELS subcommand 504
WEIGHT subcommand 505
CTAU (keyword)
CROSSTABS command 390
CTELLAREA (keyword)
CATPCA command 282
CTEMPLATE (subcommand)
SET command 1792
SHOW command 1810
CTIME.DAYS (function) 73
CTIME.HOURS (function) 73
CTIME.MINUTES (function) 73
CUBE (subcommand)
SAVETM1 command 1767
CUBIC (keyword)
CURVEFIT command 511
cubic model
CURVEFIT command 509
CUFREQ (function)
GRAPH command 877
XGRAPH command 2168
CUM (keyword)
GRAPH command 883
IGRAPH command 930
CUMCAUCHIT (keyword)
GENLIN command 728
CUMCLOGLOG (keyword)
GENLIN command 728
CUMEVENT (keyword)
KM command 965
CUMHAZARD (keyword)
CSCOXREG command 407
CUMLOGIT (keyword)
GENLIN command 728
CUMNLOGLOG (keyword)
GENLIN command 728
CUMPROB (keyword)
CSORDINAL command 449
CUMPROBIT (keyword)
GENLIN command 728
CUMULATIVE (keyword)
CSTABULATE command 477
cumulative distribution functions 64, 68, 1082
cumulative sum function 379
CUMWEIGHT (keyword)
CSPLAN command 463
CUPCT (function)
GRAPH command 877
XGRAPH command 2168
CURRENT (keyword)
TSET command 2026
current date and time
system variable $TIME 45
CURVE (keyword)
IGRAPH command 930
ROC ANALYSIS command 1720
ROC command 1725
Curve Estimation
command syntax 509
CURVEFIT (command)
APPLY subcommand 513
CIN subcommand 512
confidence intervals 512
CONSTANT/NOCONSTANT subcommands 512
ID subcommand 512
including constant 512
MODEL subcommand 511
models 511
PLOT subcommand 512
PRINT subcommand 513
SAVE subcommand 512
syntax chart 509
TEMPLATE subcommand 514
UPPERBOUND subcommand 511
using a previously defined model 513
VARIABLES subcommand 511
CUSTOM (keyword)
GENLIN command 748
NPTESTS command 1268
SIMPLAN command 1819
CUSTOM (subcommand)
CSCOXREG command 402
CSGLM command 420
CSLOGISTIC command 431
CSORDINAL command 443
custom attributes
adding and deleting 2121
custom currency formats

custom currency formats *(continued)*
creating 1791
custom hypothesis tests
CSGLM command 420
CSLOGISTIC command 431
custom models
GENLOG command 775
HILOGLINEAR command 895
LOGLINEAR command 1009
custom tables
converting TABLES command syntax to CTABLES 2221
custom variable attributes 2121
customized distance measures
CLUSTER command 313
PROXIMITIES command 1558
CUSUM (function)
GRAPH command 877
XGRAPH command 2168
CUT (keyword)
LOGISTIC REGRESSION command 996
CUTOFF (keyword)
ADP command 118
ALSCAL command 141
ROC ANALYSIS command 1719
ROC command 1724
CUTPOINT (keyword)
NPTESTS command 1268
CV (keyword)
CSDESCRIPTIVES command 414
CSTABULATE command 477
SELECTPRED command 1782
VALIDATEDATA command 2103
CW (keyword)
IGRAPH command 926
CWEIGHT (subcommand)
HILOGLINEAR command 892
LOGLINEAR command 1004
CYCLE (keyword)
DATE command 547
CZL (keyword)
SPCHART command 1883
CZLOUT (keyword)
SPCHART command 1883
CZMAX (keyword)
SPCHART command 1883
CZMIN (keyword)
SPCHART command 1883
CZOUT (keyword)
SPCHART command 1883
CZU (keyword)
SPCHART command 1883
CZUOUT (keyword)
SPCHART command 1883

D

D (keyword)
CLUSTER command 314
CROSSTABS command 390
DESCRIPTIVES command 575
PROXIMITIES command 1559
SORT CASES command 1841
D (subcommand)
ARIMA command 182
DA (keyword)
EXSMOOTH command 657
damped model
EXSMOOTH command 657
DANIELL (keyword)
SPECTRA command 1893
data
inline 516, 517
invalid 1805
DATA (keyword)
ALSCAL command 142
GENLIN command 726
DATA (subcommand)
BAYES ONESAMPLE: Binomial command 211
BAYES ONESAMPLE: Normal command 215
BAYES ONESAMPLE: Poisson command 218
CONJOINT command 344
CSSELECT command 470
GET SAS command 796
REPEATING DATA command 1668
SPATIAL MAPSPEC command 1858
with PLAN subcommand 344
data definition commands
unconditional execution in do if structures 614
unconditional execution in loops 1011
data dictionary
applying from another file 167
data files
appending orthogonal designs 1349
caching 1790
comma-delimited 1750
comparing 331
complex 630, 1625, 2195
converting 1747
databases 783
dBASE 807, 1747
default file extension 1794
direct access 953
Excel 806, 1747
file information 683, 1927
grouped 1625
keyed 953, 1443
Lotus 1-2-3 806, 1747
mixed 1625
Multiplan 806
multiple data files open at same time 533, 535, 537, 539, 541, 543
nested 1625
primary files 2091
reading 516, 933
repeating data groups 1625
SAS 795
saving 1729, 1735, 2181
saving encrypted data files 1734, 1737
saving IBM SPSS Data Collection data 1739
saving output as data files 1301, 1316
saving profiles in PLANCARDS command 1427
split-file processing 1897
spreadsheet 806, 1749
Stata 801
subsets of cases 1775
SYLK 806, 1747
tab-delimited 807, 1750
text 790

data files *(continued)*
transaction files 2091
updating 2091
DATA LIST (command)
column-style formats 526
decimal indicator 518
ENCODING subcommand 519
END subcommand 522
FILE subcommand 519
FIXED keyword 519
fixed-format data 516, 519, 524
FORTRAN-like formats 526
FREE keyword 519
freefield data 516, 518, 519, 525
inline data 516, 517
LIST keyword 519
NOTABLE subcommand 521
RECORDS subcommand 521
SKIP subcommand 522
syntax chart 515
TABLE subcommand 521
Unicode files 519
variable definition 523
variable formats 515, 525
variable names 524
with INPUT PROGRAM command 522, 617
with MATCH FILES command 1064
with NUMERIC command 1293
with POINT command 1443
with RECORD TYPE command 1625
with REPEATING DATA command 1663, 1665
with REREAD command 1703
with UPDATE command 2093
data records
defining 521, 1625
data transformations
arithmetic functions 337
arithmetic operators 337
clearing 309
conditional expressions 612, 909
converting strings to numeric 1622
counting the same value across variables 365
functions 335
if case satisfies condition 909
logical expressions 612, 909
logical operators 611, 909
missing-value functions 338
recoding values 1619
relational operators 611, 909
scoring functions 339
statistical functions 337
string functions 338, 339
time series 547
data types
custom currency 1791
DATA_STRUCTURE (subcommand)
GENLINMIXED command 755
databases
GET DATA command 785
password encryption 784, 787, 1759
reading 783
saving 1752, 1760
updating 1760
DATAFILE ATTRIBUTE (command)
DATAFILE ATTRIBUTE (command) *(continued)*
defining cross-variable rules 2107
defining single-variable rules 2105
release history 531
syntax chart 531
DATAFILTER (keyword)
GGRAPH command 825
DATAFILTER (subcommand)
TCM MODEL command 1958
DATASET (keyword)
GGRAPH command 821
DATASET (subcommand)
SPATIAL ASSOCIATION RULES command 1850
SPATIAL TEMPORAL PREDICTION command 1863
DATASET ACTIVATE (command)
release history 533
syntax chart 533
DATASET CLOSE (command)
release history 535
syntax chart 535
DATASET COPY (command)
release history 537
syntax chart 537
DATASET DECLARE (command)
release history 539
syntax chart 539
DATASET DISPLAY (command)
release history 541
syntax chart 541
DATASET NAME (command)
release history 543
syntax chart 543
datasets
comparing 331
DATASETTINGS (subcommand)
TCM MODEL command 1952
DATATYPEMIN (subcommand)
GET DATA command 788
date
system variable $TIME 45
DATE (argument)
REPORT command 1690
DATE (command)
BY keyword 549
examples 549
starting value 549
syntax chart 547
date and time functions
aggregation functions 72
conversion functions 73
difference between dates 75
extraction functions 74
incrementing dates 76
DATE format 53, 54
date format variables
input specifications 54
missing values 1129
value labels 2109
date variables
creating 547
current status 2145
DATE.DMY (function) 72
DATE.MDY (function) 72
DATE.MOYR (function) 72

DATE.QYR (function) 72
DATE.WKYR (function) 72
DATE.YRDAY (function) 72
DATEDIFF function 75
DATEDURATION (keyword)
ADP command 115
DATESUM functions 76
DATETIME format 53, 54
DAY (keyword)
DATE command 547
Db2 (keyword)
SAVE TRANSLATE command 1753
DB3 (keyword)
SAVE TRANSLATE command 1753
DB4 (keyword)
SAVE TRANSLATE command 1753
dBASE files
reading 805
saving 1753
DECIMAL (subcommand)
SET command 1793
SHOW command 1810
decimal indicator
DATA LIST command 518
GET DATA command 793
decimal places
implied 527
DECOMPOSITION (keyword)
PREFSCAL command 1514
PROXSCAL command 1576
decomposition of stress
PROXSCAL command 1576
DEFAULT (keyword)
ANACOR command 154
CORRESPONDENCE command 361
COXREG command 373
DESCRIPTIVES command 575
FREQUENCIES command 720
HOMALS command 900
LOGISTIC REGRESSION command 996
MEANS command 1126
OVERALS command 1407
PRINCALS command 1524, 1525
SUMMARIZE command 1913
TSET command 2027
DEFAULT (subcommand)
TSET command 2025
DEFAULTTEMPLATE (keyword)
GGRAPH command 830
DEFF (keyword)
CSCOXREG command 404
CSDESCRIPTIVES command 414
CSGLM command 424
CSLOGISTIC command 435
CSORDINAL command 447
CSTABULATE command 477
DEFFSQRT (keyword)
CSCOXREG command 404
CSDESCRIPTIVES command 414
CSGLM command 424
CSLOGISTIC command 435
CSORDINAL command 447
CSTABULATE command 477
DEFINE (command)
DEFINE (command) *(continued)*
!BREAK command 567
!BY keyword 567
!CHAREND keyword 561
!CMDEND keyword 561
!DEFAULT keyword 563
!DO command 567
!DOEND command 567
!ELSE keyword 566
!ENCLOSE keyword 561
!IF command 566
!IFEND command 566
!IN keyword 568
!LET command 569
!NOEXPAND keyword 563
!OFFEXPAND keyword 564
!ONEXPAND keyword 564
!POSITIONAL keyword 559
!THEN keyword 566
!TO keyword 567
!TOKENS keyword 561
limitations 556
macro arguments 559
release history 555
string functions 564
syntax chart 555
tokens 561
with SET command 565
defining variables
copying variable attributes from another file 167
copying variable definition attributes from other variables in current or external data file 171
creating new variables with variable definition attributes of existing variables 169
DEFOLANG (subcommand)
SET command 1793
SHOW command 1810
DEGREE (keyword)
CATPCA command 276
CATREG command 292
PREFSCAL command 1511
PROXSCAL command 1573, 1575
with SPLINE keyword 1573, 1575
DELCASE (subcommand)
GET DATA command 791
DELETE VARIABLES (command)
syntax chart 571
deleted residuals
in GLM 860
UNIANOVA command 2086
DELETEOBJECT (subcommand)
OUTPUT MODIFY command 1379
deleting custom attributes 2121
DELIMITED (keyword)
GET DATA command 790
delimiter
blank 36
comma 36
special 36
DELIMITERS (subcommand)
GET DATA command 791
delta
GENLOG command 773
HILOGLINEAR command 892

delta *(continued)*
LOGLINEAR command 1007
DELTA (keyword)
HILOGLINEAR command 892
NOMREG command 1251
PLUM command 1438
DELTA (subcommand)
EXSMOOTH command 658
WLS command 2155
DENDROGRAM (keyword)
CLUSTER command 319
dendrograms
CLUSTER command 319
DENSITY (keyword)
SURVIVAL command 1921
density function plots
SURVIVAL command 1921
DEPCATEGORIES (subcommand)
TREE command 1995
DEPENDENT (keyword)
MEANS command 1128
MLP command 1153
RBF command 1607
SUMMARIZE command 1914
DERIVATIVES (command)
CNLR/NLR command 1235, 1238
DERIVATIVES (keyword)
NLR/CNLR command 1241
DESCENDING (keyword)
CSORDINAL command 441
GENLIN command 726
NPTESTS command 1271
RATIO STATISTICS command 1600
DESCENDING (subcommand)
AUTORECODE command 192
DESCRIBE (keyword)
MVA command 1217, 1218
DESCRIP (keyword)
CATPCA command 281
CATREG command 296
MULTIPLE CORRESPONDENCE command 1195
DESCRIPTION (keyword)
REPOSITORY ATTRIBUTES command 1694
descriptive statistics
MIXED command 1142
MULTIPLE CORRESPONDENCE command 1195
DESCRIPTIVES (command)
MISSING subcommand 576
SAVE subcommand 574
SORT subcommand 575
STATISTICS subcommand 575
syntax chart 573
VARIABLES subcommand 574
Z scores 574
DESCRIPTIVES (keyword)
CORRELATIONS command 351
EXAMINE command 644
GENLIN command 744
GLM command 850
MIXED command 1142
MULTIPLE IMPUTATION command 1208
NPAR TESTS command 1291
ONEWAY command 1338
OPTIMAL BINNING command 1346
DESCRIPTIVES (keyword) *(continued)*
PARTIAL CORR command 1419
RELIABILITY command 1655
SIMRUN command 1838
UNIANOVA command 2076
DESCRIPTIVES (subcommand)
REGRESSION command 1643
DESIGN (function)
MATRIX command 1077
DESIGN (keyword)
MANOVA command 1031
DESIGN (subcommand)
BAYES ANOVA command 194
BAYES REGRESSION command 223, 1583
BAYES REPEATED command 234
CSPLAN command 458
GENLOG command 775
HILOGLINEAR command 895
LOGLINEAR command 1009
MANOVA command 1038
ROC ANALYSIS command 1719
VARCOMP command 2117
design effect
CSGLM command 424
CSLOGISTIC command 435
design matrix
GENLOG command 773
designs for heteroskedasticity tests
in GLM 862
in UNIANOVA 2087
DESTINATION (subcommand)
OMS command 1308
TMS BEGIN command 1981
TMS MERGE command 1990
DET (function)
MATRIX command 1077
DET (keyword)
FACTOR command 669
DETAILED (keyword)
TSET command 2027
DETECTANOMALY (command)
CRITERIA subcommand 580
HANDLEMISSING subcommand 580
OUTFILE subcommand 582
PRINT subcommand 582
release history 577
SAVE subcommand 581
syntax chart 577
VARIABLES subcommand 579
determinant
FACTOR command 669
DETRENDED (keyword)
PPLOT command 1495
detrended normal plots
EXAMINE command 643
DEV (keyword)
LOGISTIC REGRESSION command 997
DEVIANCE (keyword)
CSCOXREG command 407
GENLIN command 732
NOMREG command 1257
deviance residuals
GENLOG command 773
DEVIANCERESID (keyword)

DEVIANCERESID (keyword) *(continued)*
GENLIN command 746
DEVIATION (keyword)
COXREG command 370
CSGLM command 423
GENLIN command 741
GENLINMIXED command 764
GLM command 855, 872
LOGISTIC REGRESSION command 992
MANOVA command 1026, 1045
UNIANOVA command 2081
deviation contrasts
COXREG command 370
CSGLM command 423
in MANOVA command 1045
LOGLINEAR command 1005
UNIANOVA command 2081
deviations from the mean
repeated measures 872
DF (keyword)
CSCOXREG command 403
CSGLM command 424
CSLOGISTIC command 435
CSORDINAL command 446
MVA command 1215
DfBeta
LOGISTIC REGRESSION command 997
REGRESSION command 1634
DFBETA (keyword)
COXREG command 375
CSCOXREG command 407
LOGISTIC REGRESSION command 997
REGRESSION command 1634
DFE (keyword)
MATRIX DATA command 1117
DFE (subcommand)
FIT command 702
DfFit
REGRESSION command 1634
DFFIT (keyword)
REGRESSION command 1634
DFFIXP (keyword)
MIXED command 1146
DFH (subcommand)
FIT command 702
DFMETHOD (keyword)
MIXED command 1138
DFPRED (keyword)
MIXED command 1146
DFREQ (keyword)
FREQUENCIES command 716
DIAG (function)
MATRIX command 1077
DIAG (keyword)
MIXED command 1137
DIAGONAL (keyword)
MATRIX DATA command 1114
DIAGONAL (subcommand)
FACTOR command 671
diagonal values
FACTOR command 671
DICE (keyword)
CLUSTER command 314
PROXIMITIES command 1559
Dice measure
CLUSTER command 314
PROXIMITIES command 1559
dictionary
CODEBOOK command 323, 325
DICTIONARY (keyword)
DISPLAY command 599
DIFF (function)
CREATE command 379
DIFF (subcommand)
ACF command 98
CCF command 300
PACF command 1412
PPLOT command 1496
TSPLOT command 2053
difference
arithmetic and percentage differences between groups and variables 1297
DIFFERENCE (keyword)
COXREG command 370
CSGLM command 423
GENLIN command 741
GLM command 855, 872
GRAPH command 881
LOGISTIC REGRESSION command 992
MANOVA command 1026, 1045, 1056
UNIANOVA command 2081
difference area charts 881
difference contrasts
COXREG command 370
CSGLM command 423
in MANOVA command 1045
LOGLINEAR command 1005
repeated measures 872
UNIANOVA command 2081
difference function 379
difference transformation
ACF command 98
ARIMA command 181, 182
CCF command 300
in sequence charts 257, 2053
PACF command 1412
TSMODEL command 2045, 2046
DIFFSTRESS (keyword)
PREFSCAL command 1513
PROXSCAL command 1576
DIGITGROUPING (subcommand)
SET command 1793
SHOW command 1810
DIGITS (subcommand)
EXPORT command 653
DIM variable
ANACOR command 155
HOMALS command 902
OVERALS command 1409
PRINCALS command 1527
DIMENR (keyword)
MANOVA command 1048
DIMENS (keyword)
ALSCAL command 141
DIMENSION (subcommand)
ANACOR command 153
CATPCA command 278
CORRESPONDENCE command 358

DIMENSION (subcommand) *(continued)*
HOMALS command 899
MULTIPLE CORRESPONDENCE command 1194
OVERALS command 1406
PRINCALS command 1524
with SAVE subcommand 902, 1408, 1526
dimension reduction analysis
in MANOVA 1048
dimensions
CORRESPONDENCE command 358
HOMALS command 900
OVERALS command 1407
saving OVERALS command 1408
DIMENSIONS (keyword)
PREFSCAL command 1513
PROXSCAL command 1576
DIMn variable
CORRESPONDENCE command 362
DIMNMBR_ variable
CORRESPONDENCE command 362
DIRECT (keyword)
DISCRIMINANT command 587
direct-access files
reading 953
DIRECTION (keyword)
IGRAPH command 929
DIRECTIONS (keyword)
ALSCAL command 141
directory location 305, 680
DISCRDATA (keyword)
CATREG command 298
MULTIPLE CORRESPONDENCE command 1199
DISCRDATA(keyword)
CATPCA command 286
DISCRETE (keyword)
CONJOINT command 345
discretization
MULTIPLE CORRESPONDENCE command 1192
DISCRETIZATION (subcommand)
CATPCA command 276
CATREG command 292
MULTIPLE CORRESPONDENCE command 1192
DISCRIM (keyword)
HOMALS command 900
MULTIPLE CORRESPONDENCE command 1195, 1197
DISCRIM (subcommand)
MANOVA command 1049
DISCRIMINANT (command)
analysis block 583
ANALYSIS subcommand 586
casewise results 590
classification phase 593
classification summary 592
CLASSIFY subcommand 593
cross-validation 592
exporting model information 588
function coefficients 592, 594
HISTORY subcommand 593
inclusion levels 586
limitations 583
matrices 592
matrix input 594
matrix output 594
MATRIX subcommand 594
DISCRIMINANT (command) *(continued)*
METHOD subcommand 587
MISSING subcommand 594
missing values 594, 596
multiple analyses 586
OUTFILE subcommand 588
PLOT subcommand 594
prior probabilities 590
PRIORS subcommand 590
ROTATE subcommand 593
rotation of matrices 593
SAVE subcommand 590
saving classification variables 590
SELECT subcommand 585
selecting a subset of cases 585
STATISTICS subcommand 592
stepwise methods 586
stepwise output 593
syntax chart 583
variable selection methods 587
with MATRIX DATA command 1109
discriminant analysis
in MANOVA 1049
discriminant function coefficients
standardized 592
unstandardized 592
discriminant scores
DISCRIMINANT command 590, 594
discrimination measures
MULTIPLE CORRESPONDENCE command 1195
DISPER (keyword)
CLUSTER command 314
PROXIMITIES command 1559
dispersion
CLUSTER command 314
PROXIMITIES command 1559
dispersion accounted for
PROXSCAL command 1576
DISPLAY (command)
release history 599
syntax chart 599
VARIABLES subcommand 600
with PRINT FORMATS command 1537
with WRITE FORMATS command 2165
DISPLAY (keyword)
CORRELATIONS command 353
CSDESCRIPTIVES command 414
GENLIN command 741
KNN command 975
SIMPLAN command 1830
SIMRUN command 1836–1838
TWOSTEP CLUSTER command 2070
VALIDATEDATA command 2104
DISPLAY (statement)
MATRIX command 1105
DISPLAY (subcommand)
XGRAPH command 2172
Display Design
command syntax 1425
display formats 515, 1537
DISSIMILARITIES (keyword)
PREFSCAL command 1509
PROXSCAL command 1573
DISTANCE (keyword)

DISTANCE (keyword) *(continued)*
CLUSTER command 318
QUICK CLUSTER command 1590
DISTANCE (subcommand)
TWOSTEP CLUSTER command 2067
distance matrix
ALSCAL command 137
CLUSTER command 318
distance measures
CORRESPONDENCE command 359
Distances
command syntax 1555
DISTANCES (keyword)
PREFSCAL command 1514, 1517
PROXSCAL command 1576, 1578
DISTR (keyword)
CATPCA command 276
MULTIPLE CORRESPONDENCE command 1193
DISTRIBUTION (keyword)
GENLIN command 728
GENLINMIXED command 757
ROC ANALYSIS command 1719
ROC command 1724
SIMPLAN command 1821
DISTRIBUTION (subcommand)
PPLOT command 1493
SIMRUN command 1836
XGRAPH command 2173
distribution functions
Bernoulli 64
beta 64
binomial 64
bivariate normal 64
Cauchy 64
chi-square 64
cumulative 68, 1082
exponential 64
F 64
gamma 64
geometric 64
half-normal 64
hypergeometric 64
inverse 69, 1085
inverse Gaussian 64
Laplace 64
logistic 64
lognormal 64
negative binomial 64
normal 64
Pareto 64
Poisson 64
probability density 66, 1081
random variable 70, 1086
Studentized maximum modulus 64
Studentized range 64
t 64
tail probability 68, 1082
uniform 64
Weibull 64
DIVIDE (function)
REPORT command 1686
DIVISOR (keyword)
MIXED command 1146
DM (keyword)
DM (keyword) *(continued)*
EXSMOOTH command 657
DMCLUSTER (command) 601
DMLOGISTIC (command) 603
DMROC (command) 605
DMTABLES (command) 607
DMTREE (command) 609
DN (keyword)
EXSMOOTH command 657
DO IF (command)
commands that read the data 614
logical expressions 612
missing values 614
nested 617
PRINT SPACE command 1539
string variables 612, 614
syntax chart 611
with ELSE command 615
with ELSE IF command 616
with INPUT PROGRAM command 617
with PRINT command 1531
with PRINT EJECT command 1535
with SAMPLE command 1728
with SELECT IF command 1776
DO IF (statement)
MATRIX command 1090
DO REPEAT (command)
PRINT subcommand 621
release history 619
stand-in variable 619
syntax chart 619
with INPUT PROGRAM command 620
with LOOP command 620
DO REPEAT command
with XSAVE command 2181
DOC (subcommand)
OUTPUT EXPORT command 1361
DOCUMENT (command)
syntax chart 623
DOCUMENT (subcommand)
AGGREGATE command 125
documents
copying documents from another data file 170
retaining in aggregated files 125
DOCUMENTS (keyword)
APPLY DICTIONARY command 170
DISPLAY command 599
DOLLAR format 50
DOMAIN (subcommand)
CSCOXREG command 405
CSGLM command 425
CSLOGISTIC command 436
CSORDINAL command 448
domain errors
defined 60
numeric expressions 60
DOT (keyword)
IGRAPH command 928
dot charts
IGRAPH command 928
DOT format 50
dot plots 2179
DOTLINE (keyword)
IGRAPH command 928

DOUBLE (keyword)
MULT RESPONSE command 1187
doubly multivariate repeated measures
analysis 874
syntax 840
DOWN (keyword)
IGRAPH command 929
SORT CASES command 1841
DPATTERN (subcommand)
MVA command 1217
DRESID (keyword)
GLM command 860
REGRESSION command 1634
UNIANOVA command 2086
DROP (keyword)
GRAPH command 881
VARSTOCASES command 2137
DROP (subcommand)
ADD FILES command 108
CASESTOVARS command 269
EXPORT command 652
GET command 780
GET TRANSLATE command 809
IMPORT command 934
MATCH FILES command 1066
READ MODEL command 1616
SAVE CODEPAGE command 1737
SAVE command 1731
SAVE DATA COLLECTION command 1741
SAVE MODEL command 1744
SAVE TRANSLATE command 1762
UPDATE command 2095
VARSTOCASES command 2137
XSAVE command 2183
DROP DOCUMENTS (command)
syntax chart 625
with MATCH FILES command 1061
with UPDATE command 2091
drop-line charts 881
DROPLINE (keyword)
IGRAPH command 928
DTIME format 53, 54
DUMMY (keyword)
REPORT command 1679
DUNCAN (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082
Duncan's multiple range test
UNIANOVA command 2082
DUNNETT (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082
Dunnett's C
UNIANOVA command 2082
Dunnett's t test
UNIANOVA command 2082
DUNNETTL (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082
DUNNETTR (keyword)
GLM command 856
DUNNETTR (keyword) *(continued)*
ONEWAY command 1336
UNIANOVA command 2082
DUPLICATE (keyword)
VALIDATEDATA command 2103
DUPLICATE (subcommand)
FILE TYPE command 692
RECORD TYPE command 1629
duplicate cases
ORTHOPLAN command 1347
DUPLICATEID (keyword)
VALIDATEDATA command 2105
DURBIN (keyword)
REGRESSION command 1646
Durbin-Watson statistic
REGRESSION command 1646
DVALUE (keyword)
CROSSTABS command 393
FREQUENCIES command 716

E

E (scientific notation) format 48
EA (keyword)
EXSMOOTH command 657
EBCDIC data 680
ECHO (command)
syntax chart 627
ECONVERGE (keyword)
FACTOR command 671
EDITABLE (keyword)
GGRAPH command 830
EDITION (subcommand)
SAVE TRANSLATE command 1758
EFFECT (subcommand)
IGRAPH command 922
effects
random 848, 2074
EFFECTS (keyword)
ONEWAY command 1338
EFRON (keyword)
CSCOXREG command 406
EFSIZE (keyword)
GLM command 850
MANOVA command 1030, 1059
UNIANOVA command 2076
EIGEN (keyword)
FACTOR command 671
HOMALS command 900
MANOVA command 1048
MATRIX command 1087
PRINCALS command 1524
eigenvalues
DISCRIMINANT command 592
FACTOR command 669, 671
in MANOVA 1048
REGRESSION command 1640
ELLCOORD (keyword)
CATPCA command 286
ELSE (command) 611
ELSE (keyword)
RECODE command 1620
ELSE (statement)
MATRIX command 1090

ELSE IF (command) 611
ELSE IF (statement)
MATRIX command 1090
EM
MVA command 1219
EM (keyword)
EXSMOOTH command 657
EM (subcommand)
MVA command 1219
EMF (subcommand)
OUTPUT EXPORT command 1369
EMMEANS (subcommand)
CSGLM command 422
GENLIN command 741
GENLINMIXED command 764
GLM command 859, 875
MIXED command 1140
UNIANOVA command 2085
EMMEANS_OPTIONS (subcommand)
GENLINMIXED command 765
EMPIRICAL (keyword)
EXAMINE command 642
EMPTY (keyword)
CTABLES command 504
empty categories
excluding in CTABLES command 499
including in CTABLES command 499
showing and hiding in interactive charts 919
empty strings
autorecoding to user-missing 189
EMPTYCASE (keyword)
VALIDATEDATA command 2105
EMS (keyword)
VARCOMP command 2116
EN (keyword)
EXSMOOTH command 657
encoding
CSV files 1755
SAS files 796, 1755
Stata files 802
tab-delimited text data files 1755
ENCODING (keyword)
INCLUDE command 938
INSERT command 947
INSERT EXEC command 950
INSERT HIDDEN command 952
ENCODING (subcommand)
DATA LIST command 519
FILE HANDLE command 681
FILE TYPE command 690
GET DATA command 790
GET SAS command 796
GET STATA command 802
KEYED DATA LIST command 957
POINT command 1445
PRINT command 1533
REPEATING DATA command 1669
SAVE CODEPAGE command 1736
SAVE TRANSLATE command 1755
WRITE command 2162
ENCRYPTED (subcommand)
SAVE TRANSLATE command 1759
ENCRYPTEDPW (keyword)
OUTPUT SAVE command 1401
ENCRYPTEDPW (keyword) *(continued)*
SAVE CODEPAGE command 1737
SAVE command 1734
SORT CASES command 1842
ENCRYYPTED (subcommand)
GET DATA command 787
END (keyword)
DISCRIMINANT command 593
END (subcommand)
DATA LIST command 522
END CASE (command)
command syntax 629
with LOOP command 1017
with VECTOR command 630
END FILE (command)
syntax chart 635
with END CASE command 635
with LOOP command 1017
END IF (command) 611
END IF (statement)
MATRIX command 1090
END INPUT PROGRAM (command) 941
END LOOP (command) 1011
END LOOP (statement)
MATRIX command 1091
END MATRIX (command) 1069
END REPEAT (command) 619
end-of-file control
in input programs 522
ENDOGENOUS (subcommand)
2SLS command 95
endogenous variables
2SLS command 95
ENDPOINTS (keyword)
OPTIMAL BINNING command 1346
ENET (keyword)
CATREG command 295
enhanced metafiles 1369
ENSEMBLES (subcommand)
LINEAR command 982
ENTER (keyword)
COXREG command 371
LOGISTIC REGRESSION command 993
REGRESSION command 1638
ENTROPY (keyword)
OPTIMAL BINNING command 1346
ENTRYMETHOD (keyword)
NOMREG command 1254
EOF (function)
MATRIX command 1077
EPANECHNIKOV (keyword)
IGRAPH command 930
EPOCH (subcommand) 1794
EPS (keyword)
GENLOG command 773
GLM command 850
LOGISTIC REGRESSION command 996
UNIANOVA command 2076
VARCOMP command 2116
EPS (subcommand)
OUTPUT EXPORT command 1369
epsilon
GENLOG command 773
EQINTV (keyword)

EQINTV (keyword) *(continued)*
CATPCA command 276
CATREG command 293
MULTIPLE CORRESPONDENCE command 1193
with GROUPING keyword 276
EQUAL (keyword)
DISCRIMINANT command 590
NPTESTS command 1268
SEASON command 1773
EQUAL (subcommand)
CORRESPONDENCE command 359
EQUAL_WOR (keyword)
CSPLAN command 463
equal-weight window
SPECTRA command 1893
equality constraints
CORRESPONDENCE command 355, 359
EQUAMAX (keyword)
CATPCA command 279
FACTOR command 673
MANOVA command 1049
equamax rotation
FACTOR command 673
EQUATION (subcommand)
2SLS command 94
ERASE (command)
syntax chart 637
with REPORT command 1678
ERROR (keyword)
INSERT command 946
INSERT HIDDEN command 951
MANOVA command 1032, 1047
ERROR (subcommand)
MANOVA command 1026
error bar charts 882
ERRORBAR (keyword)
AIM command 133
ERRORBAR (subcommand)
GRAPH command 882
IGRAPH command 929
XGRAPH command 2173
ERRORCHANGE (keyword)
MLP command 1159
ERRORRATIO (keyword)
KNN command 973
MLP command 1159
errors
displaying 1794
inserted command files 946, 951
maximum number 1799
ERRORS (subcommand)
FIT command 702
SET command 1794
SHOW command 1810
ERRORSTEPS (keyword)
MLP command 1159
ES (subcommand)
ONEWAY command 1341
ESSCP matrices
GLM command 866
ESTBF (subcommand)
BAYES ANOVA command 194
BAYES CORRELATION command 199
BAYES INDEPENDENT command 203
ESTBF (subcommand) *(continued)*
BAYES LOGLINEAR command 207
BAYES REGRESSION command 223
BAYES REPEATED command 234
ESTIM (keyword)
HILOGLINEAR command 894
MANOVA command 1049
estimable functions
in GLM 850
intercept 854
UNIANOVA command 2076
estimated marginal means
CSGLM command 422
in GLM 859
MIXED command 1140
repeated measures 875
UNIANOVA command 2085
estimated means plots
UNIANOVA command 2078
ESTIMATOR (subcommand)
CSPLAN command 463
ESTPROB (keyword)
NOMREG command 1256
PLUM command 1440
eta
MEANS command 1127
SUMMARIZE command 1915
ETA (keyword)
CROSSTABS command 390
eta-squared
partial 850
ETASQ (keyword)
GLM command 850
UNIANOVA command 2076
EUCLID (keyword)
ALSCAL command 140
CLUSTER command 313
CORRESPONDENCE command 359
KNN command 972
PROXIMITIES command 1558
EUCLIDEAN (keyword)
TWOSTEP CLUSTER command 2067
Euclidean distance
CLUSTER command 313
CORRESPONDENCE command 359
PROXIMITIES command 1558
TWOSTEP CLUSTER command 2067
Euclidean model
ALSCAL command 140
EVAL (function)
MATRIX command 1077
EVALUATIONFIELDS (keyword)
TWOSTEP CLUSTER command 2070
EVENTINFO (keyword)
CSCOXREG command 406
events
TSMODEL command 2041
EXACT (keyword)
ARIMA command 185
CROSSTABS command 391
MANOVA command 1033, 1050
NPAR TESTS command 1292
SURVIVAL command 1922
exact-size sample 1727

EXACTSIZE (keyword)
NAIVEBAYES command 1230
EXAMINE (command)
CINTERVAL subcommand 644
COMPARE subcommand 641
ID subcommand 642
limitations 639
MESTIMATORS subcommand 644
MISSING subcommand 645
NOTOTAL subcommand 641
PERCENTILES subcommand 642
PLOT subcommand 643
STATISTICS subcommand 644
syntax chart 639
TOTAL subcommand 641
VARIABLES subcommand 641
Excel files
GET DATA command 785
read range 809
read variable names 809
reading 789, 805
saving 1753, 1757
saving value labels instead of values 1757
Excel format
exporting output 1367
EXCELOPTIONS (subcommand)
SAVE TRANSLATE command 1758
EXCEPT (keyword)
DETECTANOMALY command 579
EXCEPT (subcommand)
KNN command 970
MLP command 1153
NAIVEBAYES command 1228
RBF command 1606
SELECTPRED command 1782
EXCEPTIF (subcommand)
OMS command 1308
EXCHANGEABLE (keyword)
GENLIN command 737
EXCLUDE (keyword)
AIM command 132
ANOVA command 165
CLUSTER command 319
CORRELATIONS command 351
COXREG command 373
CSCOXREG command 405
CSDESCRIPTIVES command 414
CSGLM command 425
CSLOGISTIC command 436
CSORDINAL command 448
CSSELECT command 470
CSTABULATE command 478
DISCRIMINANT command 594
EXAMINE command 645
FLEISS MULTIRATER KAPPA command 706
GENLIN command 744
GLM command 849, 850
GRAPH command 886
MANOVA command 1035
MIXED command 1142
NOMREG command 1252
NPTESTS command 1267
ONEWAY command 1338
PARTIAL CORR command 1420
EXCLUDE (keyword) *(continued)*
PLUM command 1440
PROXIMITIES command 1562
RANK command 1597
RATIO STATISTICS command 1600
RELIABILITY command 1657
ROC ANALYSIS command 1719
ROC command 1724
SIMPLAN command 1829
SUMMARIZE command 1914
TSET command 2026
TWOSTEP CLUSTER command 2068
UNIANOVA command 2076
VARCOMP command 2115
EXCLUDED (keyword)
NAIVEBAYES command 1231
SELECTPRED command 1783
EXECUTE (command)
syntax chart 647
EXP (function)
MATRIX command 1077
EXP (keyword)
CSCOXREG command 404
CSLOGISTIC command 435
CSORDINAL command 447
expectation maximization
see EM estimates 1219
EXPECTED (keyword)
CROSSTABS command 389
CSTABULATE command 477
NPTESTS command 1268
expected frequencies
GENLOG command 773
HILOGLINEAR command 894
LOGLINEAR command 1007
PROBIT command 1545
EXPERIMENTAL (keyword)
ANOVA command 161
Expert Modeler
TSMODEL command 2043
EXPERTMODELER (subcommand)
TSMODEL command 2043
EXPIRATION (keyword)
REPOSITORY ATTRIBUTES command 1695
explicit category specification
in CTABLES command 496
Explore
EXAMINE command syntax 639
EXPONENTIAL (keyword)
CURVEFIT command 511
NPTESTS command 1268
exponential distribution function 64
exponential model
CURVEFIT command 509
EXSMOOTH command 657
exponential smoothing
TSMODEL command 2044
Exponential Smoothing
command syntax 655
specifying periodicity 658
exponents 59
EXPORT (command)
DIGITS subcommand 653
DROP subcommand 652

EXPORT (command) *(continued)*
KEEP subcommand 652
MAP subcommand 653
OUTFILE subcommand 651
RENAME subcommand 652
syntax chart 649
TYPE subcommand 651
UNSELECTED subcommand 652
export data 1747
Export Output
command syntax 1357
exporting output
CSGLM command 426
HTML 1308
SAV format 1308, 1316
text format 1308
XML format 1308, 1323
EXPRESSIONS (subcommand)
TCM ANALYSIS command 1931
EXSMOOTH (command)
ALPHA subcommand 658
APPLY subcommand 661
DELTA subcommand 658
GAMMA subcommand 658
INITIAL subcommand 660
MODEL subcommand 657
models 657
PERIOD subcommand 658
PHI subcommand 658
SEASFACT subcommand 658
smoothing parameter subcommands 658
syntax chart 655
using a previously defined model 661
VARIABLES subcommand 656
EXSMOOTH (subcommand)
TSMODEL command 2044
EXTENSION (command)
SPECIFICATION subcommand 664
syntax chart 663
EXTENSIONS (subcommand)
SET command 1794
SHOW command 1810
EXTERNAL (subcommand)
COXREG command 376
LOGISTIC REGRESSION command 999
EXTPATHS (subcommand)
SHOW command 1810
EXTRACAT (keyword)
CATPCA command 277
CATREG command 293
MULTIPLE CORRESPONDENCE command 1193
with ACTIVE keyword 277
with PASSIVE keyword 277
EXTRACTDAY (keyword)
ADP command 115
EXTRACTHOUR (keyword)
ADP command 115
EXTRACTION (keyword)
FACTOR command 669
EXTRACTION (subcommand)
FACTOR command 672
EXTRACTMINUTE (keyword)
ADP command 115
EXTRACTMONTH (keyword)
EXTRACTMONTH (keyword) *(continued)*
ADP command 115
EXTRACTSECOND (keyword)
ADP command 115
EXTRACTYEAR (keyword)
ADP command 115
EXTREME (keyword)
EXAMINE command 644
IGRAPH command 927
extreme values
MVA command 1213

F

F (keyword)
CSCOXREG command 404
CSGLM command 424
CSLOGISTIC command 436
CSORDINAL command 447
GLM command 850, 2076
LINEAR command 980
MANOVA command 1033
REGRESSION command 1640
F (standard numeric) format 48
F distribution function 64
F ratio
MEANS command 1127
REGRESSION command 1640
SUMMARIZE command 1915
F statistic
CSGLM command 424
CSLOGISTIC command 436
F test
in MANOVA 1048, 1059
F-to-enter
REGRESSION command 1641
F-to-remove
REGRESSION command 1641
FA1 (keyword)
MIXED command 1137
FACTOR (command)
ANALYSIS subcommand 668
coefficient display format 669
DIAGONAL subcommand 671
diagonal values 671
extraction methods 672
EXTRACTION subcommand 672
FORMAT subcommand 669
MATRIX subcommand 675
MISSING subcommand 667
rotation methods 673
ROTATION subcommand 673
SELECT subcommand 668
selecting a subset of cases 668
syntax charts 665
VARIABLES subcommand 667
with PROXIMITIES command 1565
FACTOR (keyword)
CSLOGISTIC command 433
CSORDINAL command 444
Factor Analysis
command syntax 665
factor pattern matrix
FACTOR command 669

factor score coefficient matrix
FACTOR command 669
factor structure matrix
FACTOR command 669
factor transformation matrix
FACTOR command 669
factor-loading plots
FACTOR command 671
FACTORS (keyword)
FACTOR command 671
MATRIX command 1103
NAIVEBAYES command 1229
FACTORS (subcommand)
CONJOINT command 345
MATRIX DATA command 1115
ORTHOPLAN command 1349
PLANCARDS command 1426
with REPLACE subcommand 1349
with UTILITY subcommand 347
FAH1 (keyword)
MIXED command 1137
FAILURE (subcommand)
INSERT EXEC command 949
FANCY (keyword)
IGRAPH command 927, 929
fast tables 1804
FCSITERATIONS (keyword)
MULTIPLE IMPUTATION command 1209
FEATURES (keyword)
KNN command 972
FFT (function)
CREATE command 379
FGT (function)
AGGREGATE command 125
FIELD (keyword)
MATRIX command 1093
FIELDNAMES (subcommand)
GET TRANSLATE command 809
SAVE TRANSLATE command 1756
FIELDS (subcommand)
ADP command 115
GENLINMIXED command 757
LINEAR command 980
TCM MODEL command 1966
FIELDSGROUP (subcommand)
TCM MODEL command 1966
FILE (keyword)
CSCOXREG command 401
CSDESCRIPTIVES command 412
CSGLM command 419
CSLOGISTIC command 429
CSORDINAL command 441
CSPLAN command 457
CSTABULATE command 476
GET STATA command 801
INSERT command 946
INSERT EXEC command 949
MATRIX command 1093, 1100
MODEL HANDLE command 1169
REPOSITORY ATTRIBUTES command 1694
SIMPLAN command 1819, 1830
SIMRUN command 1839
SYSFILE INFO command 1927
XGRAPH command 2176
FILE (subcommand)
ADD FILES command 107
ALSCAL command 139
CNLR/NLR command 1240
DATA LIST command 519
FILE TYPE command 689
GET command 780
GET DATA command 787
GET TRANSLATE command 808
IMPORT command 934
INCLUDE command 938
KEYED DATA LIST command 956
MATCH FILES command 1063
MATRIX DATA command 1113
POINT command 1444
READ MODEL command 1616
REPEATING DATA command 1669
REREAD command 1705
UPDATE command 2093
FILE HANDLE (command)
ENCODING subcommand 681
release history 679
syntax chart 679
Unicode files 681
with POINT command 1444
file information
copying file information from another data file 170
data files 683
IBM SPSS Statistics data files 1927
file label
copying file label from another data file 170
FILE LABEL (command)
syntax chart 683
file paths 305, 680
file transformations
subsets of cases 1775
FILE TYPE (command)
CASE subcommand 690
DUPLICATE subcommand 692
ENCODING subcommand 690
FILE subcommand 689
GROUPED keyword 688
MISSING subcommand 693
MIXED keyword 688
NESTED keyword 688
ORDERED subcommand 694
RECORD subcommand 690
subcommand summary 689
syntax chart 685
WILD subcommand 692
with RECORD TYPE command 1625
with REPEATING DATA command 1663, 1665
with SAMPLE command 1727
FILEINFO (subcommand)
APPLY DICTIONARY command 170
CODEBOOK command 326
FILELABEL (keyword)
APPLY DICTIONARY command 170
files
storing to or retrieving from a IBM SPSS Collaboration and Deployment Services Repository 1699
FILTER (command)
syntax chart 697
FILTER (subcommand)

FILTER (subcommand) *(continued)*
GETCOGNOS command 813
FILTEREDOUTPUT (subcommand)
TCM APPLY command 1942
TCM MODEL command 1961
FIN (function)
AGGREGATE command 125
FIN (keyword)
REGRESSION command 1641
find and replace functions 78
FINISH (command)
syntax chart 699
FIRST (function)
AGGREGATE command 125
FIRST (keyword)
ANOVA command 161
GENLIN command 726
IGRAPH command 931
MEANS command 1126
NPTESTS command 1268, 1273
PROXSCAL command 1574
SUMMARIZE command 1913
USE command 2097
with VARIABLES keyword 1574
FIRST (subcommand)
ADD FILES command 109
MATCH FILES command 1067
FIRSTCASE (subcommand)
GET DATA command 791
FISHER (keyword)
CSORDINAL command 446
GENLIN command 732
Fisher's classification function coefficients
DISCRIMINANT command 592
Fisher's exact test
CROSSTABS command 390
FIT (command)
DFE/DFH subcommands 702
ERRORS subcommand 702
OBS subcommand 702
syntax chart 701
FIT (keyword)
APPLY subcommand 176
ARIMA command 184
CURVEFIT command 512, 513
GENLIN command 744
NOMREG command 1256
OVERALS command 1407
PLUM command 1440
SIMPLAN command 1827
FITLINE (subcommand)
IGRAPH command 930
FITLINES (subcommand)
GGRAPH command 833
FITS (keyword)
REGRESSION command 1649
FIXCASE (subcommand)
GET DATA command 791
FIXED (keyword)
CATPCA command 278
DATA LIST command 519
GENLIN command 737
GET DATA command 790
KNN command 972, 973
FIXED (keyword) *(continued)*
MULTIPLE CORRESPONDENCE command 1194
TWOSTEP CLUSTER command 2069
FIXED (subcommand)
CASESTOVARS command 267
GENLINMIXED command 760
MIXED command 1141
fixed effects
MIXED command 1141
syntax 840
fixed format 516, 519, 524
FIXEDINPUT (subcommand)
SIMPLAN command 1825
FIXPRED (keyword)
MIXED command 1146
FIXSIGNS (keyword)
CATREG command 294
flattened weights
ALSCAL command 143
FLATWGHT (keyword)
ALSCAL command 143
FLEISS MULTIRATER KAPPA (command)
MISSING subcommand 706
syntax chart 705
FLIMIT (keyword)
MVA command 1220
FLIP (command)
NEWNAMES subcommand 710
syntax chart 709
VARIABLES subcommand 710
FLT (function)
AGGREGATE command 125
FNAMES (keyword)
MATRIX command 1104
FOCALCASES (keyword)
KNN command 976
FOCALCASES (subcommand)
KNN command 971
FOLD (keyword)
KNN command 975
FOLDS (keyword)
KNN command 974
FOOTER (subcommand)
PLANCARDS command 1428
FOOTNOTE (keyword)
XGRAPH command 2176
FOOTNOTE (subcommand)
GRAPH command 880
OLAP CUBES command 1296
REPORT command 1690
SET command 1794
SPCHART command 1872
SUMMARIZE command 1913
FOR (keyword)
SURVIVAL command 1920
FORCE (subcommand)
NAIVEBAYES command 1229
forced removal
REGRESSION command 1638
forced-entry method
COXREG command 371
DISCRIMINANT command 587
LOGISTIC REGRESSION command 993
REGRESSION command 1638

FORCEMERGE (keyword)
OPTIMAL BINNING command 1345
FORECAST (subcommand)
ARIMA command 185
forecasting
current forecast period 2145
CURVEFIT command 509
TSAPPLY command 2015
TSMODEL command 2032, 2034
FORMAT (keyword)
CODEBOOK command 325
CTABLES command 507
MATRIX command 1095
FORMAT (subcommand)
CROSSTABS command 393
CTABLES command 504
FACTOR command 669
FREQUENCIES command 716
IGRAPH command 923
LIST command 986
MATRIX DATA command 1113
MULT RESPONSE command 1187
PARTIAL CORR command 1419
PLANCARDS command 1427
REPORT command 1677
SET command 1795
SHOW command 1810
SUMMARIZE command 1914
TSPLOT command 2054
formats
numeric variables 48
of new variables 336, 614, 912
string variables 46
FORMATS (command)
syntax chart 713
FORMATS (keyword)
APPLY DICTIONARY command 171
FORMATS (subcommand)
GET SAS command 797
formats for summary functions
CTABLES command 493
FORTRAN-like format specifications 526
FORWARD (keyword)
NOMREG command 1252
REGRESSION command 1638
TMS IMPORT command 1986
forward entry
REGRESSION command 1638
forward selection
COXREG command 371
LOGISTIC REGRESSION command 993
FORWARDSTEPWISE (keyword)
LINEAR command 980
Fourier frequencies
saving with SPECTRA command 1895
Fourier periods
saving with SPECTRA command 1895
Fourier transformation function
inverse 380
FOUT (function)
AGGREGATE command 125
FOUT (keyword)
REGRESSION command 1641
FPAIR (keyword)
FPAIR (keyword) *(continued)*
DISCRIMINANT command 592
FPRECISION (keyword)
CNLR command 1243
FRACTION (subcommand)
PPLOT command 1494
RANK command 1596
FRAME (subcommand)
GGRAPH command 833
FREE (keyword)
DATA LIST command 519
MATRIX DATA command 1113
freefield format 516, 518, 519, 525
FREGW (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082
FREQ (keyword)
FREQUENCIES command 716–718
HILOGLINEAR command 894
HOMALS command 900
OVERALS command 1407
PRINCALS command 1524
PROBIT command 1545
SPECTRA command 1895
FREQUENCIES (command)
BARCHART subcommand 716, 717
charts 716, 717
display order 716
FORMAT subcommand 716
GROUPED subcommand 718
HISTOGRAM subcommand 718
limitations 715
MISSING subcommand 721
NTILES subcommand 719
PERCENTILES subcommand 719
STATISTICS subcommand 720
suppressing tables 716
syntax chart 715
VARIABLES subcommand 716
FREQUENCIES (keyword)
NPTESTS command 1268
FREQUENCIES (subcommand)
MULT RESPONSE command 1184
FREQUENCY (function)
REPORT command 1685
FREQUENCY (keyword)
SIMPLAN command 1828
frequency tables
format 716
writing to a file 1547
FRIEDMAN (keyword)
NPTESTS command 1273
RELIABILITY command 1655
FRIEDMAN (subcommand)
NPAR TESTS command 1281
FROM (keyword)
LIST command 987
SAMPLE command 1727
FROM (subcommand)
APPLY DICTIONARY command 168
START JOIN command 1900
FSCORE (keyword)
FACTOR command 669

FSTEP (keyword)
COXREG command 371
LOGISTIC REGRESSION command 993
NOMREG command 1252
FTEST (subcommand)
BAYES REGRESSION command 225
FTOLERANCE (keyword)
CNLR command 1243
FTSPACE (keyword)
REPORT command 1677
FULL (keyword)
CORRELATIONS command 351
GENLIN command 732
MATRIX DATA command 1113
FULLFACTORIAL (subcommand)
NOMREG command 1252
functions
arithmetic 61
cumulative distribution 68, 1082
date and time 72–76
distribution 64
examples 337
inverse distribution 69
MATRIX command 1077
MATRIX inverse distribution 1085
missing values 92
missing values in 336
numeric variables 61
probability density 66, 1081
random variable 64, 70, 1086
statistical 63
string variables 78
tail probability 68, 1082
time series 379
furthest neighbor method
CLUSTER command 317
FUZZBITS (subcommand)
SET command 1795
SHOW command 1810

G

G (keyword)
MIXED command 1142
SPECTRA command 1893, 1895
GABRIEL (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082
Gabriel's pairwise comparisons test
UNIANOVA command 2082
GAC (keyword)
OLAP CUBES command 1297
GAIN (keyword)
MLP command 1162
RBF command 1611
GAIN (subcommand)
TREE command 1998
gain chart
SPECTRA command 1893
TREE command 1999
gain values
saving with SPECTRA command 1895
Games and Howell's pairwise comparisons test
Games and Howell's pairwise comparisons test *(continued)*
UNIANOVA command 2082
GAMMA (keyword)
CROSSTABS command 390
GENLIN command 728
GENLINMIXED command 757
GAMMA (subcommand)
EXSMOOTH command 658
gamma distribution function 61, 64
GCOV (keyword)
DISCRIMINANT command 592
GEF (keyword)
CSCOXREG command 406
CSGLM command 425
CSLOGISTIC command 437
CSORDINAL command 449
GENLIN command 744
GLM command 850
UNIANOVA command 2076
GEMSCAL (keyword)
ALSCAL command 140
GEMWGHT (keyword)
ALSCAL command 143
general estimable function(s)
CSGLM command 425
CSLOGISTIC command 437
UNIANOVA command 2076
General Loglinear Analysis
command syntax 769
general mode
CROSSTABS command 388
MEANS command 1125
general smoothing parameter
EXSMOOTH command 658
GENERALIZED (keyword)
PREFSCAL command 1512
PROXSCAL command 1574
Generalized Estimating Equations
command syntax 723
generalized linear mixed models
command syntax 749
Generalized Linear Models
command syntax 723
generalized multidimensional scaling
ALSCAL command 140
generalized weights
ALSCAL command 143
Generate Orthogonal Design
command syntax 1347
generating class
HILOGLINEAR command 895
GENLIN (command)
CRITERIA subcommand 732
EMMEANS subcommand 741
MISSING subcommand 744
MODEL subcommand 728
OUTFILE subcommand 748
PRINT subcommand 744
release history 723
REPEATED subcommand 737
SAVE subcommand 746
syntax chart 723
variable list 726
GENLINMIXED (command)

GENLINMIXED (command) *(continued)*
BUILD_OPTIONS subcommand 763
DATA_STRUCTURE subcommand 755
EMMEANS subcommand 764
EMMEANS_OPTIONS subcommand 765
FIELDS subcommand 757
FIXED subcommand 760
OUTFILE subcommand 766
RANDOM subcommand 761
SAVE subcommand 766
syntax chart 749
TARGET_OPTIONS subcommand 757
GENLOG (command)
cell covariates 771, 775
cell structure 771
cell weights 771
CIN keyword 773
compared to LOGLINEAR 1001
criteria 773
CRITERIA subcommand 773
CSTRUCTURE subcommand 771
data distribution 773
delta 773
DESIGN subcommand 775
EPS keyword 773
generalized residuals 772
GLOR subcommand 772
GRESID subcommand 772
limitations 769
log-odds ratio 772
logit model 770
main-effects model 775
maximum iterations 773
MISSING subcommand 775
model specification 775
MODEL subcommand 773
multinomial distribution 773
PLOT subcommand 774
Poisson distribution 773
PRINT subcommand 773
SAVE subcommand 775
simultaneous linear logit model 775
single-degree-of-freedom partitions 775
statistics 773
structural zeros 771
syntax chart 769
variable list 770
WITH keyword 775
GEOMETRIC (keyword)
MEANS command 1126
OLAP CUBES command 1296
SUMMARIZE command 1913
geometric distribution function 64
GET (command)
DROP subcommand 780
FILE subcommand 780
KEEP subcommand 780
MAP subcommand 781
RENAME subcommand 781
syntax chart 779
unconditional execution in do if structures 614
unconditional execution in loops 1011
GET (statement)
MATRIX command 1097
GET CAPTURE (command)
CONNECT subcommand 784
SQL subcommand 784
syntax chart 783
UNENCRYPTED subcommand 784
GET DATA (command)
ARRANGEMENT subcommand 790
ASSUMEDSTRWIDTH subcommand 788
CELLRANGE subcommand 789
CONNECT subcommand 787
DATATYPEMIN subcommand 788
DELCASE subcommand 791
DELIMITED keyword 790
DELIMITERS subcommand 791
ENCODING subcommand 790
ENCRYPTED subcommand 787
FILE subcommand 787
FIRSTCASE subcommand 791
FIXCASE subcommand 791
FIXED keyword 790
HIDDEN subcommand 789
IMPORTCASE subcommand 791
LEADINGSPACES subcommand 788
MAP subcommand 793
MULTIPLESPACES subcommand 791
ODBC keyword 786
OLEDB keyword 786
QUALIFIER subcommand 792
READNAMES subcommand 789
release history 785
SHEET subcommand 789
SQL subcommand 787
syntax chart 785
TRAILINGSPACES subcommand 790
TXT keyword 786
TYPE subcommand 786
UNENCRYPTED subcommand 787
VARIABLES subcommand 792
XLS keyword 786
GET SAS (command)
DATA subcommand 796
ENCODING subcommand 796
FORMATS subcommand 797
syntax chart 795
GET STATA (command)
ENCODING subcommand 802
FILE keyword 801
release history 801
syntax chart 801
GET TRANSLATE (command)
database files 807
DROP subcommand 809
FIELDNAMES subcommand 809
FILE subcommand 808
KEEP subcommand 809
limitation 805
MAP subcommand 810
RANGE subcommand 809
spreadsheet files 806
tab-delimited files 807
TYPE subcommand 808
GETCOGNOS (command)
CONNECTION subcommand 812
FILTER subcommand 813

GETCOGNOS (command) *(continued)*
IMPORT subcommand 813
LOCATION subcommand 812
MODE subcommand 812
PARAMETERS subcommand 813
release history 811
syntax chart 811
GETTM1 (command)
CONNECTION subcommand 816
release history 815
RENAME subcommand 817
syntax chart 815
VIEW subcommand 817
GG (keyword)
MANOVA command 1059
GGRAPH
converting IGRAPH to GGRAPH 2221
GGRAPH (command)
CASELIMIT keyword 827
COLORCYCLE subcommand 833
DATAFILTER keyword 825
DATASET keyword 821
DEFAULTTEMPLATE keyword 830
EDITABLE keyword 830
FITLINES subcommand 833
FRAME subcommand 833
GRAPHDATASET subcommand 821
GRAPHSPEC subcommand 827
GRIDLINES subcommand 833
HIGH qualifier 822
LABEL keyword 830
LEVEL qualifier 823
MISSING keyword 826
NAME keyword 821
NAME qualifier 822
release history 819
REPORTMISSING keyword 826
SOURCE keyword 827
syntax chart 819
TEMPLATE keyword 830
TRANSFORM keyword 826
VIZMAP keyword 831
VIZSTYLESHEET keyword 831
GH (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082
GINV (function)
MATRIX command 1077
GLM
alpha level 850
alternative hypothesis 850
confidence interval 850
contained effects 849
contrast coefficients 850
contrasts 855
deleted residuals 860
estimable functions 850
estimated marginal means 859
estimated means plots 852
homogeneity of variance 850
K matrix 854
L matrix 850, 853
Levene's test 850
GLM *(continued)*
multiple comparisons 856
parameter estimates 850
post hoc tests 856
power 850
profile plots 852
repeated measures syntax 869
residual plots 852
spread-versus-level plots 852
standardized residuals 860
Studentized residuals 860
syntax chart 845
Type I sum-of-squares method 849
Type II sum-of-squares method 849
Type III sum-of-squares method 849
Type IV sum-of-squares method 849
unstandardized predicted residuals 860
unstandardized residuals 860
weighted unstandardized predicted values 860
weighted unstandardized residuals 860
GLM (command)
CONTRAST subcommand 842, 855
CRITERIA subcommand 850
EMMEANS subcommand 859, 875
INTERCEPT subcommand 849
KMATRIX subcommand 841, 854
LMATRIX subcommand 841, 853
MEASURE subcommand 874
METHOD subcommand 849
MISSING subcommand 850
MMATRIX subcommand 841, 867
multivariate syntax 865
OUTFILE subcommand 861
PLOT subcommand 852
POSTHOC subcommand 856
PRINT subcommand 850, 866
RANDOM subcommand 848
REGWGT subcommand 848
ROBUST subcommand 860
sample models 840
SAVE subcommand 860
syntax overview 837, 838
WSDESIGN subcommand 874
WSFACTOR subcommand 871
GLM Multivariate
command syntax 865
HSSCP matrices 866
GLM Repeated Measures 869
GLM Univariate
command syntax 845, 2071
GLOR (subcommand)
GENLOG command 772
GLS (keyword)
FACTOR command 672
GMEDIAN (function)
GGRAPH command 823
XGRAPH command 2168
GMEDIAN (keyword)
MEANS command 1126
OLAP CUBES command 1296
SUMMARIZE command 1913
GOODFIT (keyword)
LOGISTIC REGRESSION command 996
Goodman and Kruskal's gamma

Goodman and Kruskal's gamma *(continued)*
CROSSTABS command 390
Goodman and Kruskal's lambda
CROSSTABS command 390
Goodman and Kruskal's tau
CROSSTABS command 390
goodness of fit
TSAPPLY command 2016
TSMODEL command 2035
GPC (keyword)
OLAP CUBES command 1297
GPTILE (function)
GGRAPH command 823
GRADIENTDESCENT (keyword)
MLP command 1157
GRAPH (command)
BAR subcommand 880
BIVARIATE keyword 882
CI keyword 882
count functions 877
CUM keyword 883
DROP keyword 881
ERRORBAR subcommand 882
FOOTNOTE subcommand 880
GROUPED keyword 881
HILO subcommand 881
HISTOGRAM subcommand 883
INCLUDE keyword 886
INTERVAL subcommand 885
LINE subcommand 881
LISTWISE keyword 886
MATRIX keyword 882
MISSING subcommand 886
NOCUM keyword 883
NOREPORT keyword 886
OVERLAY keyword 882
PANEL subcommand 883
PARETO subcommand 883
PIE subcommand 881
RANGE keyword 880
release history 877
REPORT keyword 886
SCATTERPLOT subcommand 882
SIMPLE keyword 880, 881, 883
STACKED keyword 883
STDDEV keyword 882
STERROR keyword 882
SUBTITLE subcommand 880
summary functions 877
syntax chart 877
TEMPLATE subcommand 885
TITLE subcommand 880
VARIABLE keyword 886
XYZ keyword 882
GRAPHDATASET (subcommand)
GGRAPH command 821
graphs
population pyramid 2173
GRAPHS (subcommand)
OUTPUT MODIFY command 1388
GRAPHSPEC (subcommand)
GGRAPH command 827
GREAT (function)
REPORT command 1686
Greenhouse-Geiser epsilon 1053
GRESID (subcommand)
GENLOG command 772
LOGLINEAR command 1005
GRID (keyword)
EXSMOOTH command 658, 659
grid search
EXSMOOTH command 659
GRIDLINES (subcommand)
GGRAPH command 833
GROUP (keyword)
AIM command 133
NPTESTS command 1268, 1271
ROC ANALYSIS command 1719
GROUP (subcommand)
AUTORECODE command 189
group membership
predicted 590
probabilities 590
GROUPBY (subcommand)
CASESTOVARS command 268
GROUPED (keyword)
FILE TYPE command 688
GRAPH command 880, 881
GROUPED (subcommand)
FREQUENCIES command 718
grouped files 688, 1625
GROUPING (keyword)
CATPCA command 276
CATREG command 292
GENLINMIXED command 755, 761
MULTIPLE CORRESPONDENCE command 1192
GROUPS (keyword)
EXAMINE command 641
GROUPS (subcommand)
MULT RESPONSE command 1182
T-TEST command 2063
GROUPWISE (keyword)
SURVIVAL command 1924
GROWTH (keyword)
CURVEFIT command 511
growth model
CURVEFIT command 509
GROWTHLIMIT (subcommand)
TREE command 2003
GSCH (function)
MATRIX command 1077
GT2 (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082
GUIDE (keyword)
OPTIMAL BINNING command 1344
GUTTMAN (keyword)
RELIABILITY command 1655
Guttman's lower bounds
RELIABILITY command 1655

H

half-normal distribution function 64
HAMANN (keyword)
CLUSTER command 314
PROXIMITIES command 1559

Hamann measure
CLUSTER command 314
PROXIMITIES command 1559
HAMMING (keyword)
SPECTRA command 1893
HAMPEL (keyword)
EXAMINE command 644
HANDLEMISSING (subcommand)
DETECTANOMALY command 580
HANDLENOISE (subcommand)
TWOSTEP CLUSTER command 2067
HANDLES (subcommand)
SHOW command 1810
HARMONIC (keyword)
MEANS command 1126
OLAP CUBES command 1296
SUMMARIZE command 1913
HAVERAGE (keyword)
EXAMINE command 642
HAZARD (keyword)
COXREG command 374, 375
CSCOXREG command 408
KM command 962, 965
SURVIVAL command 1921
hazard plots
COXREG command 374
KM command 962
SURVIVAL command 1921
HCONVERGE (keyword)
GENLIN command 732, 737
HEADER (keyword)
ALSCAL command 142
HEADER (subcommand)
SET command 1795
SHOW command 1810
HELMERT (keyword)
COXREG command 370
CSGLM command 423
GENLIN command 741
GLM command 855, 872
LOGISTIC REGRESSION command 992
MANOVA command 1026, 1045, 1056
UNIANOVA command 2081
Helmert contrasts
COXREG command 370
CSGLM command 423
in MANOVA command 1045
LOGLINEAR command 1005
reverse 872
UNIANOVA command 2081
heterogeneity factor
PROBIT command 1544
heteroscedasticity
WLS command 2155
hexadecimal format 47, 50
HF (keyword)
MANOVA command 1059
MIXED command 1137
HICICLE (keyword)
CLUSTER command 319
HIDDEN (subcommand)
GET DATA command 789
HIDDENFUNCTION (keyword)
MLP command 1156
HIDDENFUNCTION (keyword) *(continued)*
RBF command 1609
HIDDENLAYERS (keyword)
MLP command 1156
HIDENOTSIG (keyword)
AIM command 132
HIDESMALLCOUNTS (subcommand)
CROSSTABS command 395
CTABLES command 508
OLAP CUBES command 1299
HIDESOURCECATS (keyword)
CTABLES command 507
hiding keys
in interactive charts 924
HIERARCHICAL (keyword)
ANOVA command 161
Hierarchical Cluster Analysis
command syntax 311
methods 317
hierarchical files. See nested files 688
HIGH (keyword)
CSLOGISTIC command 429
RANK command 1494, 1596
high-low-close charts
clustered 881
simple 881
HIGHEST (keyword)
COUNT command 365
MISSING VALUES command 1130
RECODE command 1620
HILO (keyword)
TSPLOT command 2054
HILO (subcommand)
GRAPH command 881
HILOGLINEAR (command)
cell weights 892
CRITERIA subcommand 892
custom models 895
CWEIGHT subcommand 892
DESIGN subcommand 895
interaction terms 895
limitations 889
maximum iterations 892
maximum order of terms 891
MAXORDER subcommand 891
METHOD subcommand 891
MISSING subcommand 895
model specification 895
normal probability plots 894
PLOT subcommand 894
PRINT subcommand 894
residual plots 894
syntax chart 889
variable list 891
weighted models 892
HISTOGRAM (keyword)
EXAMINE command 643
REGRESSION command 1646
HISTOGRAM (subcommand)
FREQUENCIES command 718
GRAPH command 883
IGRAPH command 930
histograms
FREQUENCIES command 718

histograms *(continued)*
interval width 718
REGRESSION command 1646
scale 718
with normal curve 718
HISTORY (keyword)
CATPCA command 281
CATREG command 296
CSCOXREG command 406
CSLOGISTIC command 437
CSORDINAL command 449
GENLIN command 744
HOMALS command 900
MIXED command 1142
MULTIPLE CORRESPONDENCE command 1195
NOMREG command 1256
OVERALS command 1407
PLUM command 1440
PREFSCAL command 1514
PRINCALS command 1524
PROXSCAL command 1576
VARCOMP command 2116
HISTORY (subcommand)
DISCRIMINANT command 593
Hochberg's GT2
UNIANOVA command 2082
HODGES_LEHMAN (keyword)
NPTESTS command 1271, 1273
HOLD (keyword)
MATRIX command 1096
HOLDOUT (keyword)
KNN command 971
MLP command 1155
RBF command 1608
HOLDOUT (subcommand)
ORTHOPLAN command 1350
with MIXHOLD subcommand 1350
HOLT (keyword)
EXSMOOTH command 657
HOMALS (command)
ANALYSIS subcommand 899
compared with OVERALS 1405
CONVERGENCE subcommand 900
DIMENSION subcommand 899
dimensions 900
excluding cases 899
labeling plots 900
MATRIX subcommand 902
MAXITER subcommand 900
NOBSERVATIONS subcommand 899
PLOT subcommand 900
PRINT subcommand 900
SAVE subcommand 902
syntax chart 897
value labels 900
variable labels 900
VARIABLES subcommand 898
with AUTORECODE command 897, 898
with RECODE command 897
HOMOGENEITY (keyword)
CSTABULATE command 477
GLM command 850, 866
MANOVA command 1031, 1049
ONEWAY command 1338
HOMOGENEITY (keyword) *(continued)*
UNIANOVA command 2076
Homogeneity Analysis
command syntax 1189
homogeneity of variance
GLM command 866
in GLM 850
UNIANOVA command 2076
homogeneity tests
CROSSTABS command 390
in MANOVA command 1047
HORIZONTAL (keyword)
IGRAPH command 921
Hosmer-Lemeshow goodness-of-fit statistic
LOGISTIC REGRESSION command 996
HOST (command)
interactions with CD and INSERT commands 907
release history 905
syntax chart 905
HOTELLING (keyword)
RELIABILITY command 1655
Hotelling's T 2
RELIABILITY command 1655
Hotelling's trace
in MANOVA 1050
HOUR (keyword)
DATE command 547
HSSCP matrices
GLM command 866
HTML
exporting output as HTML 1308
HTML (subcommand)
OUTPUT EXPORT command 1362
HTML format
exporting output 1362
HUBER (keyword)
EXAMINE command 644
Huynh-Feldt epsilon 1053
hypergeometric distribution function 64
HYPOTH (keyword)
MANOVA command 1048
hypotheses
custom 841, 869

I

I (subcommand)
data organization 1876
SPCHART command 1875
variable specification 1876
IBM SPSS Collaboration and Deployment Services Repository
file specifications 2217
IBM SPSS Data Collection data
saving 1739
IBM SPSS Statistics data file export
CSGLM command 426
CSLOGISTIC command 438
IC (keyword)
NOMREG command 1256
SPECTRA command 1895
TWOSTEP CLUSTER command 2069
ICC (subcommand)
RELIABILITY command 1656

icicle plots
CLUSTER command 319
ICIN (keyword)
REGRESSION command 1634
ID (keyword)
CSCOXREG command 400
DETECTANOMALY command 579
MIXED command 1137
QUICK CLUSTER command 1590
REGRESSION command 1646
VALIDATEDATA command 2102
ID (subcommand)
CASESTOVARS command 265
CLUSTER command 318
CURVEFIT command 512
EXAMINE command 642
KM command 963
LOGISTIC REGRESSION command 995
MVA command 1214
PLS command 1434
PROXIMITIES command 1562
REPEATING DATA command 1672
SPCHART command 1885
TSET command 2026
TSPLOT command 2054
VARSTOCASES command 2135
IDCHECKS (subcommand)
VALIDATEDATA command 2103
IDEAL (keyword)
CONJOINT command 345
IDENT (function)
MATRIX command 1077
IDENTITY (keyword)
GENLIN command 728
GENLINMIXED command 757
MLP command 1156
PREFSCAL command 1512
PROXSCAL command 1574
IDF functions 64
IDF.BETA (function)
MATRIX command 1085
IDF.CAUCHY (function)
MATRIX command 1085
IDF.CHISQ (function)
MATRIX command 1085
IDF.EXP (function)
MATRIX command 1085
IDF.F (function)
MATRIX command 1085
IDF.GAMMA (function)
MATRIX command 1085
IDF.HALFNRM (function)
MATRIX command 1085
IDF.IGAUSS (function)
MATRIX command 1085
IDF.LAPLACE (function)
MATRIX command 1085
IDF.LNORMAL (function)
MATRIX command 1085
IDF.LOGISTIC (function)
MATRIX command 1085
IDF.NORMAL (function)
MATRIX command 1085
IDF.PARETO (function)
IDF.PARETO (function) *(continued)*
MATRIX command 1085
IDF.SMOD (function)
MATRIX command 1085
IDF.SRANGE (function)
MATRIX command 1085
IDF.T (function)
MATRIX command 1085
IDF.UNIFORM (function)
MATRIX command 1085
IDF.WEIBULL (function)
MATRIX command 1085
IF (command)
compared with RECODE command 1619
logical expressions 909
missing values 912
string variables 909, 912
syntax chart 909
with LOOP command 910
IF (keyword)
LOOP command 1013
IF (subcommand)
OMS command 1306
OUTPUT MODIFY command 1377
IFFT (function)
CREATE command 380
IGAUSS (keyword)
GENLIN command 728
GENLINMIXED command 757
IGNORE_CASE (keyword)
FLEISS MULTIRATER KAPPA command 706
WEIGHTED KAPPA command 2150
IGRAPH
converting IGRAPH to GGRAPH 2221
IGRAPH (command)
AREA subcommand 924
BAR subcommand 925
BOX subcommand 927
CAPTION subcommand 922
CASELABEL subcommand 921
CATORDER subcommand 919
CHARTLOOK subcommand 922
COLOR subcommand 920
COORDINATE subcommand 921
EFFECT subcommand 922
ERRORBAR subcommand 929
FITLINE subcommand 930
FORMAT subcommand 923
HISTOGRAM subcommand 930
KEY keyword 924
LINE subcommand 928
NORMALIZE subcommand 920
PANEL subcommand 921
PIE subcommand 926
POINTLABEL subcommand 921
SCATTER subcommand 924
SIZE subcommand 920
smoothers 930
SPIKE subcommand 923
STACK subcommand 921
STYLE subcommand 920
SUBTITLE subcommand 922
summary functions 931
SUMMARYVAR subcommand 921

IGRAPH (command) *(continued)*
syntax chart 915
TITLE subcommand 922
VIEWNAME subcommand 922
X1 subcommand 918
X1LENGTH subcommand 919
X2 subcommand 918
X2LENGTH subcommand 919
Y subcommand 918
YLENGTH subcommand 919
IMAGE (keyword)
FACTOR command 672
image factoring
FACTOR command 672
IMAGEMAP (keyword)
OMS command 1310
IMAGEROOT (keyword)
OMS command 1310
IMAGES (keyword)
OMS command 1309
implicit category specification
CTABLES command 497
implied decimal format 527
IMPORT (command)
DROP subcommand 934
FILE subcommand 934
KEEP subcommand 934
MAP subcommand 935
RENAME subcommand 935
syntax chart 933
TYPE subcommand 934
IMPORT (subcommand)
GETCOGNOS command 813
import data 785
IMPORTANCE (keyword)
AIM command 133
MLP command 1161
RBF command 1610
importance chart
TREE command 1999
IMPORTCASE (subcommand)
GET DATA command 791
IMPUTATIONS (keyword)
MULTIPLE IMPUTATION command 1209
IMPUTATIONSUMMARIES (subcommand)
MULTIPLE IMPUTATION command 1208
IMPUTE (subcommand)
MULTIPLE IMPUTATION command 1204
imputing missing values
MULTIPLE CORRESPONDENCE command 1193
IN (keyword)
ALSCAL command 144
CLUSTER command 319
DISCRIMINANT command 594
FACTOR command 675
MANOVA command 1036
ONEWAY command 1338, 1645
PARTIAL CORR command 1420
PROXIMITIES command 1562
PROXSCAL command 1579
REGRESSION command 1645
RELIABILITY command 1657
IN (subcommand)
ADD FILES command 109
IN (subcommand) *(continued)*
KEYED DATA LIST command 956
MATCH FILES command 1066
UPDATE command 2095
INCLPROB (keyword)
CSPLAN command 463
INCLPROB (subcommand)
CSPLAN command 464
INCLUDE (command)
ENCODING keyword 938
FILE subcommand 938
syntax chart 937
vs. INSERT command 947
INCLUDE (keyword)
AIM command 132
ANOVA command 165
CLUSTER command 319
CORRELATIONS command 351
COXREG command 373
CROSSTABS command 392
CSCOXREG command 405
CSDESCRIPTIVES command 414
CSGLM command 425
CSLOGISTIC command 436
CSORDINAL command 448
CSSELECT command 470
CSTABULATE command 478
DESCRIPTIVES command 576
DISCRIMINANT command 594
EXAMINE command 645
FACTOR command 667
FLEISS MULTIRATER KAPPA command 706
FREQUENCIES command 721
GENLIN command 744
GLM command 849, 850
GRAPH command 886
HILOGLINEAR command 895
MEANS command 1128
MIXED command 1142
MULT RESPONSE command 1187
NOMREG command 1252
NONPAR CORR command 1262
NPAR TESTS command 1291
NPTESTS command 1267
ONEWAY command 1338
PARTIAL CORR command 1420
PLUM command 1440
PROBIT command 1546
PROXIMITIES command 1562
RANK command 1597
RATIO STATISTICS command 1600
REGRESSION command 1646
RELIABILITY command 1657
ROC ANALYSIS command 1719
ROC command 1724
SIMPLAN command 1829
SUMMARIZE command 1914
SURVIVAL command 1924
T-TEST command 2064
TSET command 2026
TWOSTEP CLUSTER command 2068
UNIANOVA command 2076
VARCOMP command 2115
INCOMPLETE (keyword)

INCOMPLETE (keyword) *(continued)*
VALIDATEDATA command 2103
INCOMPLETEID (keyword)
VALIDATEDATA command 2105
increment value
in matrix loop structures 1091
INDENT (keyword)
REPORT command 1677
INDEPENDENCE (keyword)
CSTABULATE command 477
independence model
ACF command 100
INDEPENDENT (keyword)
CATPCA command 278
GENLIN command 737
MULTIPLE CORRESPONDENCE command 1195
INDEPENDENT (subcommand)
NPTESTS command 1271
independent normalization
MULTIPLE CORRESPONDENCE command 1195
independent-samples nonparametric tests
command syntax 1265
INDEPENDENTSAMPLES (subcommand)
PROPORTIONS command 1552
INDEX (function) 78
INDEX (keyword)
CASESTOVARS command 268
CSPLAN command 463
DISPLAY command 599
SORT CASES command 1842
INDEX (subcommand)
CASESTOVARS command 266
VARSTOCASES command 2135
index chart
TREE command 1999
index of regressivity
RATIO STATISTICS command 1601, 1602
INDEXING (subcommand)
OUTPUT MODIFY command 1379
indexing clause
in matrix loop structures 1091
LOOP command 1013
indexing strings 91
indexing variable
in matrix loop structures 1091
INDICATOR (keyword)
COXREG command 370
LOGISTIC REGRESSION command 992
INDIVIDUAL (keyword)
IGRAPH command 930
MANOVA command 1034
PREFSCAL command 1514, 1515
PROXSCAL command 1576, 1577
individual space weights
PROXSCAL command 1576
individual space weights plots
PROXSCAL command 1577
individual spaces
PROXSCAL command 1576
individual spaces plots
PROXSCAL command 1577
individual test names 856, 1337, 1338
individuals charts
SPCHART command 1875
INDSCAL (keyword)
ALSCAL command 140
INFERENCE (subcommand)
BAYES ANOVA command 194
BAYES CORRELATION command 199
BAYES INDEPENDENT command 203
BAYES LOGLINEAR command 207
BAYES ONESAMPLE: Binomial command 211
BAYES ONESAMPLE: Normal command 215
BAYES ONESAMPLE: Poisson command 218
BAYES REGRESSION command 223
BAYES RELATED command 229
BAYES REPEATED command 234
INFILE (subcommand)
TMS IMPORT command 1986
TWOSTEP CLUSTER command 2068
INFLUENCE (subcommand)
TREE command 2010
INFO (command) 939
INITIAL (keyword)
APPLY subcommand 176
ARIMA command 184
CATPCA command 278
FACTOR command 669
GENLIN command 732
MULTIPLE CORRESPONDENCE command 1194
PREFSCAL command 1514, 1515
QUICK CLUSTER command 1590
INITIAL (subcommand)
CATREG command 294
EXSMOOTH command 660
OVERALS command 1406
PREFSCAL command 1509
PROXSCAL command 1571
initial cluster centers
QUICK CLUSTER command 1589
initial parameter values
ARIMA command 183
EXSMOOTH command 660
initial value
in matrix loop structures 1091
initialization
suppressing 977
initializing variables
formats 1293, 1907
numeric variables 1293
scratch variables 1293
string variables 1907
INITTHRESHOLD (keyword)
TWOSTEP CLUSTER command 2067
INKNOT (keyword)
CATPCA command 276
CATREG command 292
PREFSCAL command 1511
PROXSCAL command 1573, 1575
with SPLINE keyword 1573, 1575
INLINE (keyword)
MATRIX DATA command 1113
inline data 516, 517
INPUT (keyword)
ADP command 115, 117, 118, 120
BOOTSTRAP command 249
PREFSCAL command 1514
PROXSCAL command 1576

INPUT (keyword) *(continued)*
SIMPLAN command 1821, 1825
INPUT (subcommand
VARIABLE ROLE command 2129
INPUT (subcommand)
ALSCAL command 137
PREFSCAL command 1507
input data
file 40
input formats
column-style specifications 526
FORTRAN-like specifications 526
numeric 527
string 528
INPUT PROGRAM (command)
examples 522, 617, 620, 630, 635
syntax chart 941
with DATA LIST command 617
with END subcommand on DATA LIST 522
with NUMERIC command 1293
with REPEATING DATA command 1663, 1665
with REREAD command 1703
with SAMPLE command 1727
with STRING command 1907
with VECTOR command 2141
input programs
end-of-file control 522
examples 522, 617, 620, 630, 635, 941, 1293, 1444, 2141
input state 943
INPUTS (keyword)
LINEAR command 980
INSERT (command)
CD keyword 947
ENCODING keyword 947
ERROR keyword 946
FILE keyword 946
interaction with HOST command 907
release history 945
syntax chart 945
SYNTAX keyword 946
vs. INCLUDE command 947
INSERT EXEC (command)
CLEAR subcommand 949
ENCODING keyword 950
FAILURE subcommand 949
FILE keyword 949
SUCCESS subcommand 949
SYNTAX keyword 950
INSERT EXEC(command)
release history 949
syntax chart 949
INSERT HIDDDEN (command)
SYNTAX keyword 951
INSERT HIDDEN (command)
ENCODING keyword 952
ERROR keyword 951
release history 951
SOURCE keyword 951
syntax chart 951
INSIDE (keyword)
IGRAPH command 925, 926
instrumental variables
2SLS command 94
INSTRUMENTS (subcommand)
2SLS command 94
integer mode
CROSSTABS command 388
interaction effects
ANOVA command 161
interaction terms
COXREG command 369
GENLOG command 775
HILOGLINEAR command 895
LOGLINEAR command 1009
interactions
in GLM 862
UNIANOVA command 2088
VARCOMP command 2117
INTERACTIONS (keyword)
MULTIPLE IMPUTATION command 1204
interactive syntax rules
inserted command files 946, 950, 951
intercept
CSGLM command 420
CSLOGISTIC command 430
in estimable function 854
include or exclude 849, 2076, 2115
INTERCEPT (keyword)
GENLIN command 728
PREFSCAL command 1511
VARCOMP command 2117
INTERCEPT (subcommand)
CSGLM command 420
CSLOGISTIC command 430
GLM command 849
NOMREG command 1252
UNIANOVA command 2076
VARCOMP command 2115
INTERCOOLED (keyword)
SAVE TRANSLATE command 1758
INTERPOLATE (keyword)
IGRAPH command 924, 928
INTERVAL (keyword)
ALSCAL command 138
IGRAPH command 930
PROXSCAL command 1573, 1575
with VARIABLES keyword 1575
INTERVAL (subcommand)
GRAPH command 885
SURVIVAL command 1919
interval data
ALSCAL command 138
INTERVALCENTER (keyword)
MLP command 1157
INTERVALOFFSET (keyword)
MLP command 1157
INTO (keyword)
OPTIMAL BINNING command 1344
RANK command 1595
RECODE command 1621
INTO (subcommand)
AUTORECODE command 188
INV (function)
MATRIX command 1077
INV (keyword)
FACTOR command 669
invalid data

invalid data *(continued)*
treatment of 1805
INVERSE (keyword)
CURVEFIT command 511
inverse correlation matrix
FACTOR command 669
inverse distribution functions 64, 69
inverse Fourier transformation function 380
inverse Gaussian distribution function 64
inverse model
CURVEFIT command 511
IR (subcommand)
data organization 1876
SPCHART command 1875
variable specification 1876
ISTEP (keyword)
CNLR command 1243
item statistics
RELIABILITY command 1656
item-total statistics
RELIABILITY command 1656
ITER (keyword)
ALSCAL command 141
CNLR command 1243
COXREG command 373
LOGISTIC REGRESSION command 996
NLR command 1244
ITERATE (keyword)
FACTOR command 671
HILOGLINEAR command 892
LOGISTIC REGRESSION command 996
PROBIT command 1544
VARCOMP command 2116
iteration history
CATPCA command 281
CSLOGISTIC command 437
MIXED command 1142
MULTIPLE CORRESPONDENCE command 1195
PROXSCAL command 1576
iterations
AREG command 175
ARIMA command 184
ITERATIONS (keyword)
MVA command 1219
SIMPLAN command 1825

J

JACCARD (keyword)
CLUSTER command 314
PROXIMITIES command 1559
Jaccard similarity ratio
CLUSTER command 314
PROXIMITIES command 1559
JDATE format 53, 54
JEFFREYS (keyword)
NPTESTS command 1268
JITTER (keyword)
IGRAPH command 924
JOIN (keyword)
TSPLOT command 2054
JOIN (subcommand)
START JOIN command 1901
JOINT (keyword)
JOINT (keyword) *(continued)*
ANACOR command 154
MANOVA command 1034
joint category plots
CATPCA command 282
MULTIPLE CORRESPONDENCE command 1197
joint probabilities
CSGLM command 419
CSLOGISTIC command 429
file structure 471
JOINTCAT (keyword)
MULTIPLE CORRESPONDENCE command 1197
JOINTCAT(keyword)
CATPCA command 282
JOINTPROB (subcommand)
CSCOXREG command 401
CSDESCRIPTIVES command 412
CSGLM command 419
CSLOGISTIC command 429
CSORDINAL command 441
CSSELECT command 471
CSTABULATE command 476
JONCKHEERE_TERPSTRA (keyword)
NPTESTS command 1271
JOURNAL (subcommand)
SET command 1795
SHOW command 1810
journal file 40
JPG (subcommand)
OUTPUT EXPORT command 1370
Julian date 54

K

K (keyword)
SPCHART command 1883
SPECTRA command 1893, 1895
K matrix
in GLM 854
UNIANOVA command 2080
K-Means Cluster Analysis
command syntax 1587
K-S (subcommand)
NPAR TESTS command 1282, 1283
K1 (keyword)
CLUSTER command 314
PROXIMITIES command 1559
K2 (keyword)
CLUSTER command 314
PROXIMITIES command 1559
KAISER (keyword)
FACTOR command 671
Kaiser normalization
FACTOR command 671
Kaiser-Meyer-Olkin measure
FACTOR command 669
Kaplan-Meier
command syntax 959
KAPPA (keyword)
CROSSTABS command 390
KEEP (keyword)
VARSTOCASES command 2137
KEEP (subcommand)
ADD FILES command 108

KEEP (subcommand) *(continued)*
EXPORT command 652
GET command 780
GET TRANSLATE command 809
IMPORT command 934
MATCH FILES command 1066
READ MODEL command 1616
SAVE CODEPAGE command 1737
SAVE command 1731
SAVE DATA COLLECTION command 1741
SAVE MODEL command 1744
SAVE TRANSLATE command 1762
UPDATE command 2095
VARSTOCASES command 2137
XSAVE command 2183
KEEPTIES (keyword)
PREFSCAL command 1511
PROXSCAL command 1575
with ORDINAL keyword 1575
KENDALL (keyword)
NONPAR CORR command 1261
NPTESTS command 1273
KENDALL (subcommand)
NPAR TESTS command 1285
Kendall's coefficient of concordance
RELIABILITY command 1655
Kendall's tau-b
CROSSTABS command 390
Kendall's tau-c
CROSSTABS command 390
KERNEL (keyword)
GENLIN command 732
NOMREG command 1256
PLUM command 1440
KEY (keyword)
IGRAPH command 924
KEY (subcommand)
KEYED DATA LIST command 956
POINT command 1445
key variables
ADD FILES (command) 108
MATCH FILES command 1064
keyed data files
defining 1443
file handle 1444
file key 1443, 1445
reading 953
KEYED DATA LIST (command)
direct-access files 953
ENCODING subcommand 957
FILE subcommand 956
IN subcommand 956
KEY subcommand 956
keyed files 953
NOTABLE subcommand 957
syntax chart 953
TABLE subcommand 957
keyed table 1065
keys
showing and hiding in interactive charts 924
KeywordName (keyword)
KNN command 972
keywords
syntax 35
KEYWORDS (keyword)
REPOSITORY ATTRIBUTES command 1694
KM (command)
censored cases 961
COMPARE subcommand 964
defining event 961
factor variable 961
ID subcommand 963
mean survival time 963
median survival time 963
percentiles 963
PERCENTILES subcommand 963
PLOT subcommand 962
plots 962
PRINT subcommand 963
quartiles 963
SAVE subcommand 965
saving new variables 965
STATUS subcommand 961
status variable 961
STRATA subcommand 962
strata variable 962
survival tables 963
survival time variable 961
syntax chart 959
TEST subcommand 963
TREND subcommand 964
trends for factor levels 964
KM command
case-identification variable 963
comparing factor levels 964
labeling cases 963
KMATRIX (keyword)
CSCOXREG command 402
CSGLM command 420
CSLOGISTIC command 431
KMATRIX (subcommand)
GLM command 841, 854
UNIANOVA command 2080
KMEANS (keyword)
QUICK CLUSTER command 1589
KMO (keyword)
FACTOR command 669
KNN (command)
CASELABELS subcommand 970
CRITERIA subcommand 973
CROSSVALIDATION subcommand 974
EXCEPT subcommand 970
FOCALCASES subcommand 971
MISSING subcommand 974
MODEL subcommand 972
OUTFILE subcommand 976
PARTITION subcommand 971
PRINT subcommand 975
RESCALE subcommand 971
SAVE subcommand 975
syntax chart 967
variable lists 969
VIEWMODEL subcommand 975
KOLMOGOROV_SMIRNOV (keyword)
NPTESTS command 1268, 1271
Kolmogorov-Smirnov Z
NPAR TESTS command 1282–1284
KR20

KR20 *(continued)*
RELIABILITY command 1655
Kronecker product 872
KRONEKER (function)
MATRIX command 1077
KRUSKAL_WALLIS (keyword)
NPTESTS command 1271
KS_SIM (subcommand)
NPAR TESTS command 1284
Kulczynski measures
CLUSTER command 314
PROXIMITIES command 1559
KURT (keyword)
MEANS command 1126
SUMMARIZE command 1913
kurtosis
EXAMINE command 644
FREQUENCIES command 720
KURTOSIS (function)
REPORT command 1685
KURTOSIS (keyword)
DESCRIPTIVES command 575
FREQUENCIES command 720
IGRAPH command 931

L

L matrix
CSGLM command 425
CSLOGISTIC command 437
in GLM 850
UNIANOVA command 2076, 2079
L MATRIX (keyword)
CSORDINAL command 449
LA (keyword)
EXSMOOTH command 657
LABEL (keyword)
CODEBOOK command 325, 326
CTABLES command 507
GGRAPH command 830
IGRAPH command 924–926, 928, 929
REPORT command 1679, 1682
XGRAPH command 2171
labels
positioning category labels in CTABLES command 494
positioning summary labels in CTABLES command 494
LABELS (keyword)
DISPLAY command 599
MULT RESPONSE command 1187
lack of fit
UNIANOVA command 2076
LAG (function)
CREATE command 380
lag function 1807
LAGRANGE (keyword)
GENLIN command 744
LAGRANGE3 (keyword)
IGRAPH command 928
LAGRANGE5 (keyword)
IGRAPH command 928
lambda
Goodman and Kruskal's 314, 1559
Wilks' 587
LAMBDA (keyword)
LAMBDA (keyword) *(continued)*
CLUSTER command 314
CROSSTABS command 390
MVA command 1219
PREFSCAL command 1513
PROXIMITIES command 1559
SELECTPRED command 1782, 1783
LAMBDAINITIAL (keyword)
MLP command 1157
Lance and Williams dissimilarity measure
CLUSTER command 314
PROXIMITIES command 1559
language
changing output language 1801
Laplace distribution function 64
LARGE (keyword)
LINEAR command 980
LASSO (keyword)
CATREG command 295
LAST (function)
AGGREGATE command 125
LAST (keyword)
GENLIN command 726
IGRAPH command 931
MEANS command 1126
SUMMARIZE command 1913
USE command 2097
LAST (subcommand)
ADD FILES command 109
MATCH FILES command 1067
LATENTFACTORS (keyword)
PLS command 1434, 1435
LAYERED (keyword)
CSDESCRIPTIVES command 414
CSTABULATE command 478
LCL_CUMHAZARD (keyword)
CSCOXREG command 407
LCL_SURVIVAL (keyword)
CSCOXREG command 407
LCON (keyword)
COXREG command 373
LOGISTIC REGRESSION command 996
LCONVERGE (keyword)
CSCOXREG command 403
CSLOGISTIC command 435
CSORDINAL command 446
GENLIN command 732
MIXED command 1138
NOMREG command 1251
PLUM command 1438
LDELLAREA (keyword)
CATPCA command 282
LEAD (function)
CREATE command 381
lead function 381, 1807
leading zeros
restricted numeric (N) format 48
LEADINGSPACES (subcommand)
GET DATA command 788
LEADZERO (subcommand)
SET command 1796
SHOW command 1810
LEARNINGEPOCHS (keyword)
MLP command 1157

LEARNINGINITIAL (keyword)
MLP command 1157
LEARNINGLOWER (keyword)
MLP command 1157
LEAST (function)
REPORT command 1686
least significant difference 856, 1336
least-squares method
generalized 672
unweighted 672
LEAVE (command) 977
LEFT (keyword)
REPORT command 1680, 1683, 1690
LEGEND (keyword)
IGRAPH command 920
legends
IGRAPH command 920
LENGTH (function) 78
LENGTH (keyword)
REPORT command 1677
LENGTH (subcommand)
REPEATING DATA command 1669
SHOW command 1810
LESS (keyword)
CONJOINT command 345
LEVEL (keyword)
APPLY DICTIONARY command 171
CATPCA command 275
CATREG command 291
LEVEL (qualifier)
GGRAPH command 823
LEVEL (subcommand)
ALSCAL command 138
level of measurement
copying from other variables in current or external data file 171
specifying 2127
LEVEL variable
ANACOR command 155
HOMALS command 902
OVERALS command 1409
PRINCALS command 1527
LEVEL_ variable
CORRESPONDENCE command 362
levels
within-subjects factors 871
Levenberg-Marquardt method
CNLR/NLR command 1244
Levene test
EXAMINE command 643
GLM command 866
in GLM 850
UNIANOVA command 2076
LEVER (keyword)
GLM command 860
LOGISTIC REGRESSION command 997
REGRESSION command 1634
UNIANOVA command 2086
leverage
LOGISTIC REGRESSION command 997
LEVERAGE (keyword)
GENLIN command 746
leverage values
REGRESSION command 1634
LFTOLERANCE (keyword)
CNLR command 1243
LG10 (function)
MATRIX command 1077
LG10 (keyword)
ARIMA command 181
LGSTIC (keyword)
CURVEFIT command 511
Life Tables
command syntax 1917
LIFT (keyword)
MLP command 1162
RBF command 1611
LIKELIHOOD (keyword)
GENLIN command 732
NPTESTS command 1268
TWOSTEP CLUSTER command 2067
likelihood ratio
COXREG command 371
LOGISTIC REGRESSION command 993
likelihood-ratio chi-square
CROSSTABS command 390
LIKELIHOODRESID (keyword)
GENLIN command 746
Lilliefors test
EXAMINE command 643
LIMIT (keyword)
FREQUENCIES command 716
LINE (keyword)
IGRAPH command 927, 928, 930
LINE (subcommand)
GRAPH command 881
IGRAPH command 928
line breaks
in value labels 2109
in variable labels 2125
line charts
sequence 258, 2054
LINEAR (command)
BUILD_OPTIONS subcommand 980
ENSEMBLES subcommand 982
FIELDS subcommand 980
OUTFILE subcommand 983
SAVE subcommand 983
syntax chart 979
LINEAR (keyword)
CONJOINT command 345
CURVEFIT command 511
IGRAPH command 930
PREFSCAL command 1511
Linear Mixed Models
command syntax 1133
linear model
EXSMOOTH command 657
linear models
command syntax 979
Linear Regression
command syntax 1633
LINEARITY (keyword)
MEANS command 1127
SUMMARIZE command 1915
linearity test
MEANS command 1127
SUMMARIZE command 1915

LINELABEL (keyword)
IGRAPH command 928
LINK (keyword)
GENLIN command 728
GENLINMIXED command 757
LINK (subcommand)
CSORDINAL command 442
PLUM command 1439
LINKPARAMETER (keyword)
GENLINMIXED command 757
LINT (function)
RMV command 1714
LIST (command)
CASES subcommand 987
FORMAT subcommand 986
VARIABLES subcommand 986
with SAMPLE command 987
with SELECT IF command 987
with SPLIT FILE command 987
LIST (keyword)
DATA LIST command 519
MATRIX DATA command 1113
NPTESTS command 1268, 1273
PLANCARDS command 1427
REPORT command 1677, 1691
LIST (subcommand)
SUMMARIZE command 1914
LISTING (keyword)
SET command 1794
LISTWISE (keyword)
CATPCA command 276
CATREG command 293
CORRELATIONS command 351
CSDESCRIPTIVES command 414
CSTABULATE command 478
DESCRIPTIVES command 576
EXAMINE command 645
FACTOR command 667
GRAPH command 886
HILOGLINEAR command 895
MULTIPLE CORRESPONDENCE command 1193
NONPAR CORR command 1262
NPAR TESTS command 1291
NPTESTS command 1267
ONEWAY command 1338
OPTIMAL BINNING command 1346
PARTIAL CORR command 1420
PROBIT command 1546
REGRESSION command 1646
SURVIVAL command 1924
T-TEST command 2064
LISTWISE (subcommand)
MVA command 1218
listwise deletion
CTABLES command 505
MULTIPLE CORRESPONDENCE command 1193
LJUMP (keyword)
IGRAPH command 928
LLEFT (keyword)
IGRAPH command 926
LLR (keyword)
IGRAPH command 930
LM (keyword)
COXREG command 374
LM (keyword) *(continued)*
EXSMOOTH command 657
LMATRIX (keyword)
CSCOXREG command 402, 406
CSGLM command 420, 425
CSLOGISTIC command 431, 437
GENLIN command 744
MIXED command 1142
LMATRIX (subcommand)
GLM command 841, 853
UNIANOVA command 2079
LML (keyword)
COXREG command 375
CSCOXREG command 406, 408
LN (function)
MATRIX command 1077
LN (keyword)
ARIMA command 181
EXSMOOTH command 657
LN (subcommand)
ACF command 99
CCF command 301
PACF command 1413
PPLOT command 1497
TSPLOT command 2054
LNGAMMA (function) 61
LOADING (keyword)
CATPCA command 281, 282, 284
with BIPLOT keyword 284
LOADINGS (keyword)
OVERALS command 1407
PRINCALS command 1524, 1525
LOCALE (subcommand)
SET command 1796
LOCATION (keyword)
CODEBOOK command 326
LOCATION (subcommand)
GETCOGNOS command 812
PLUM command 1439
LOCK (keyword)
SIMPLAN command 1821, 1825, 1826
LOF (keyword)
GLM command 850
UNIANOVA command 2076
LOG (keyword)
CSCOXREG command 406
GENLIN command 728
GENLINMIXED command 757
LOG (subcommand)
PROBIT command 1544
log rank test
KM command 963
log transformation
PROBIT command 1544
log transformation (base 10)
ARIMA command 181
log-likelihood distance measure
TWOSTEP CLUSTER command 2067
log-minus-log plots
COXREG command 374
log-odds ratio
GENLOG command 772
LOGARITHMIC (keyword)
CURVEFIT command 511

LOGC (keyword)
GENLIN command 728
GENLINMIXED command 757
logging in to a IBM SPSS Collaboration and Deployment Services Repository 1697
logical expressions
defined 82
in END LOOP 82
in LOOP 82
in loop structures 1013
in SELECT IF 82
missing values 91
order of evaluation 85
selecting cases 1775
string variables 78
logical functions 85
logical operators
defined 84
in matrix language 1074
missing values 615, 912
logical variables
defined 82
LOGIN (subcommand)
REPOSITORY CONNECT command 1698
logistic distribution function 64
Logistic Regression
command syntax 989
LOGISTIC REGRESSION (command)
casewise listings 997
CASEWISE subcommand 997
categorical covariates 992
CATEGORICAL subcommand 992
classification plots 997
classification tables 996
CLASSPLOT subcommand 997
CONTRAST subcommand 992
contrasts 992
correlation matrix 996
CRITERIA subcommand 996
dependent variable 991
EXTERNAL subcommand 999
Hosmer-Lemeshow goodness-of-fit statistic 996
ID subcommand 995
include constant 995
interaction terms 991
iteration history 996
label casewise listings 995
METHOD subcommand 993
MISSING subcommand 998
missing values 998
NOORIGIN subcommand 995
ORIGIN subcommand 995
OUTFILE subcommand 998
PRINT subcommand 996
release history 989
SAVE subcommand 999
saving new variables 999
SELECT subcommand 995
subsets of cases 995
syntax chart 989
VARIABLES subcommand 991
logit
PROBIT command 1543
LOGIT (keyword)
LOGIT (keyword) *(continued)*
CSORDINAL command 442
GENLIN command 728
GENLINMIXED command 757
PLUM command 1439
PROBIT command 1543
logit link
PLUM command 1439
logit residuals
LOGISTIC REGRESSION command 997
LOGLINEAR (command)
categorical variables 1003
cell covariates 1003
cell weights 1004
compared to GENLOG 1001
CONTRAST subcommand 1005
contrasts 1005
convergence criteria 1007
correlation matrix 1007
covariates 1009
CRITERIA subcommand 1007
custom models 1009
CWEIGHT subcommand 1004
delta 1007
dependent variables 1003
design matrix 1007
DESIGN subcommand 1009
display options 1007
equiprobability model 1009
expected frequencies 1007
factors 1003
general loglinear model 1003
generalized residuals 1005
GRESID subcommand 1005
interaction terms 1009
limitations 1001
logit model 1003, 1005
main-effects model 1009
maximum iterations 1007
measures of association 1003
MISSING subcommand 1008
missing values 1008
model specification 1009
NOPRINT subcommand 1007
normal probability plots 1008
observed frequencies 1007
parameter estimates 1007
PLOT subcommand 1008
plots 1008
PRINT subcommand 1007
residual plots 1008
residuals 1007
simultaneous linear logit model 1009
single-degree-of-freedom partitions 1009
statistics 1007
structural zeros 1004
syntax chart 1001
variable list 1003
lognormal distribution function 64
LOGRANK (keyword)
KM command 963
LOGSURV (keyword)
KM command 962
SURVIVAL command 1921

LOOP (command)
commands that read the data 1011
examples 630
increment value 1016
indexing clause 1013
initial value 1013
logical expressions 1013
missing values 1017
nested 1011, 1013
syntax chart 1011
terminal value 1013
unconditional data definition commands 1011
with END CASE command 1017
with END FILE command 1017
with SET command 1799
with SET MXLOOPS command 1011–1013
with VECTOR command 2139, 2140
LOOP (statement)
MATRIX command 1091
loop structures
macro facility 567
loops
maximum number 1799
LOSS (keyword)
CNLR command 1241
LOSS (subcommand)
CNLR command 1245
loss function
CNLR/NLR command 1245
Lotus 1-2-3 files
read range 809
read variable names 809
reading 805
LOW (keyword)
CSLOGISTIC command 429
RANK command 1494, 1596
LOW (qualifier)
GGRAPH command 822
LOWER (function) 78
LOWER (keyword)
CORRELATIONS command 351
FLEISS MULTIRATER KAPPA command 707
MATRIX DATA command 1113
PROXSCAL command 1571
LOWEREND (keyword)
OPTIMAL BINNING command 1345
LOWERLIMIT (keyword)
OPTIMAL BINNING command 1345
LOWEST (keyword)
COUNT command 365
MISSING VALUES command 1130
RECODE command 1620
LPAD (function) 78
LR (keyword)
COXREG command 371
NOMREG command 1254
LRCHISQ (keyword)
SELECTPRED command 1782, 1783
LRESID (keyword)
LOGISTIC REGRESSION command 997
LRIGHT (keyword)
IGRAPH command 926
LRT (keyword)
NOMREG command 1256
LSD (keyword)
CSCOXREG command 404
CSGLM command 424
CSLOGISTIC command 436
CSORDINAL command 447
GENLIN command 741
GENLINMIXED command 765
GLM command 856
MIXED command 1140
ONEWAY command 1336
UNIANOVA command 2082
LSL (keyword)
SPCHART command 1884
LSL (subcommand)
SPCHART command 1887
LSTEP (keyword)
IGRAPH command 924, 928
LSTOLERANCE (keyword)
CNLR command 1243
LTRIM (function) 78

M

M matrix
displaying 872
GLM command 866
in GLM Multivariate 867
M-W (subcommand)
NPAR TESTS command 1286
MA (function)
CREATE command 381
MA (subcommand)
ARIMA command 183
SEASON command 1773
macro
canonical correlation macro 2215
ridge regression macro 2215
Macro
cautions 2215
macro facility
assigning defaults 563
conditional processing 566
display macro commands 1797
examples 2205
keyword arguments 560
loop structures 567
macro call 556
macro definition 556
macro expansion 1797
positional arguments 560
string functions 564
tokens 561
with MATRIX command 1105
with matrix language 1105
macros 555
MACROS (keyword)
DISPLAY command 599
MAGIC (function)
MATRIX command 1077
MAHAL (keyword)
DISCRIMINANT command 587
REGRESSION command 1634
Mahalanobis distance
DISCRIMINANT command 587

Mahalanobis distance *(continued)*
REGRESSION command 1634
MAKE (function)
MATRIX command 1077
MAKE (subcommand)
VARSTOCASES command 2134
Mallow's Cp
REGRESSION command 1640
MANN_WHITNEY (keyword)
NPTESTS command 1271
Mann-Whitney U
NPAR TESTS command 1286
MANOVA
analysis groups 1051
confidence intervals 1050
contrasts 1045, 1056
cross-products matrix 1048
discriminant analysis 1049
display options 1059
error correlation matrix 1048
error sum of squares 1048
error variance-covariance matrix 1048
linear transformations 1044
naming transformed variables 1047
power estimates 1050
principal components analysis 1049
renaming transformed variables 1058
significance tests 1048
simple effects 1057
within-subjects factors 1057
MANOVA (command)
ANALYSIS subcommand 1037, 1051
between-subjects factors 1053
CINTERVAL subcommand 1034, 1050
compared with GLM command 839, 1021
constant covariate 1054
CONTRAST subcommand 1026, 1056
covariates 1044
dependent variable 1044
DESIGN subcommand 1038
DISCRIM subcommand 1049
display options 1047
doubly multivariate repeated measures 1053
error matrices 1047
ERROR subcommand 1026
factors 1044
homogeneity tests 1047
limitations 1043, 1053
MATRIX subcommand 1036
MEASURE subcommand 1058
METHOD subcommand 1028
MISSING subcommand 1035
multivariate syntax 1043
NOPRINT subcommand 1029, 1047
OMEANS subcommand 1032, 1044
PARTITION subcommand 1028
PCOMPS subcommand 1049
PLOT subcommand 1049
PMEANS subcommand 1032
POWER subcommand 1033, 1050
PRINT subcommand 1029, 1047
RENAME subcommand 1047, 1058
RESIDUALS subcommand 1033
significance tests 1047
MANOVA (command) *(continued)*
syntax chart 1019, 1023, 1043
transformation matrix 1047
variable list 1054
variables specification 1043
within-subjects factors 1053, 1055
WSDESIGN subcommand 1057
WSFACTORS subcommand 1055
Mantel-Haenszel statistic
CROSSTABS command 390
MANUAL (keyword)
REPORT command 1677
MAP (keyword)
DISCRIMINANT command 594
MAP (subcommand)
ADD FILES command 110
EXPORT command 653
GET command 781
GET DATA command 793
GET TRANSLATE command 810
IMPORT command 935
MATCH FILES command 1068
MODEL HANDLE command 1171
SAVE command 1733
SAVE DATA COLLECTION command 1741
SAVE TRANSLATE command 1763
SPATIAL MAPSPEC command 1856
UPDATE command 2095
XSAVE command 2184
MAPOUTPUT (subcommand)
SPATIAL ASSOCIATION RULES command 1852
SPATIAL TEMPORAL PREDICTION command 1866
MAPPINGS (subcommand)
SAVETM1 command 1767
MAPPROJECTION (subcommand)
SPATIAL MAPSPEC command 1856
MAPSPEC (subcommand)
SPATIAL ASSOCIATION RULES command 1849
SPATIAL TEMPORAL PREDICTION command 1862
marginal homogeneity test
NPAR TESTS command 1287
marginal means
CSGLM command 422
MARGINAL_HOMOGENEITY (keyword)
NPTESTS command 1273
MARGINS (keyword)
REPORT command 1677
MARK (subcommand)
TSPLOT command 2057
MARKERS (keyword)
PREFSCAL command 1516
MARTINGALE (keyword)
CSCOXREG command 407
MAT (keyword)
MATRIX DATA command 1117
MATCH FILES (command)
active dataset 1064
BY subcommand 1064
case source variable 1066
DROP subcommand 1066
duplicate cases 1064
FILE subcommand 1063
FIRST subcommand 1067
IN subcommand 1066

MATCH FILES (command) *(continued)*
KEEP subcommand 1066
LAST subcommand 1067
limitations 1061
MAP subcommand 1068
RENAME subcommand 1065
syntax chart 1061
table lookup files 1065
TABLE subcommand 1065
with DATA LIST command 1064
with DROP DOCUMENTS command 1061
with SORT CASES command 1843
matching coefficients
CLUSTER command 314
PROXIMITIES command 1559
matrices
correlation 1417
covariance 351
K 854
L 850, 853
split-file processing 1897
MATRIX (command)
BREAK statement 1092
CALL statement 1087
COMPUTE statement 1076
DISPLAY statement 1105
DO IF statement 1090
ELSE IF statement 1090
ELSE statement 1090
END IF statement 1090
END LOOP statement 1091
GET statement 1097
LOOP statement 1091
MGET statement 1100
MSAVE statement 1102
PRINT statement 1088
READ statement 1093
RELEASE statement 1105
SAVE statement 1099
syntax chart 1069
with macro facility 1105
WRITE statement 1095
MATRIX (keyword)
ALSCAL command 139
CSPLAN command 458, 460, 461, 464
GRAPH command 882
PARTIAL CORR command 1419
PREFSCAL command 1510
PROXSCAL command 1572
MATRIX (subcommand)
ALSCAL command 144
ANACOR command 155
CLUSTER command 319
CORRELATIONS command 352
DISCRIMINANT command 594
FACTOR command 675
HOMALS command 902
MANOVA command 1036
MCONVERT command 1122
NONPAR CORR command 1262
ONEWAY command 1338
OVERALS command 1409
PARTIAL CORR command 1420
PRINCALS command 1527
MATRIX (subcommand) *(continued)*
PROXIMITIES command 1562
PROXSCAL command 1579
REGRESSION command 1645
RELIABILITY command 1657
with SAVE subcommand 902, 1408, 1526
MATRIX DATA (command)
CELLS subcommand 1116
CONTENTS subcommand 1117
data-entry format 1113
entering data 1111
FACTORS subcommand 1115
field separators 1111
FILE subcommand 1113
FORMAT subcommand 1113
matrix shape 1113
N subcommand 1119
ROWTYPE_ variable 1107, 1112
scientific notation 1111
SPLIT subcommand 1114
syntax chart 1107
VARIABLES subcommand 1111
VARNAME_ variable 1112
with DISCRIMINANT command 1109
with ONEWAY command 1109
with REGRESSION command 1109
matrix data files
converting correlation to covariance 1121
converting covariance to correlation 1121
raw 1107
variable names 43
MATRIX functions 1077
matrix input
ALSCAL command 144
CLUSTER command 319
DISCRIMINANT command 594
FACTOR command 675
PROXIMITIES command 1562
RELIABILITY command 1657
MATRIX inverse distribution functions 1085
matrix language
arithmetic operators 1073
column vector 1070
conformable matrices 1072
constructing a matrix from other matrices 1072
control structures 1090
displaying results 1088
functions 1077
logical operators 1074
main diagonal 1070
matrix notation 1072
reading IBM SPSS Statistics data files 1075
reading text files 1093
relational operators 1074
row vector 1070
saving IBM SPSS Statistics data files 1075
scalar 1070
scalar expansion 1073
string variables 1071
symmetric matrix 1070
transpose 1070
variables 1070
with macro facility 1105
matrix output

matrix output *(continued)*
CLUSTER command 319
DISCRIMINANT command 594
FACTOR command 675
HOMALS command 902
OVERALS command 1409
PROXIMITIES command 1562
RELIABILITY command 1657
matrix weights
ALSCAL command 142
Mauchly's test of sphericity
in MANOVA command 1053
MAX (function)
AGGREGATE command 125
REPORT command 1685
MAX (keyword)
ANACOR command 154
CORRESPONDENCE command 361
DESCRIPTIVES command 575
HOMALS command 900
IGRAPH command 920
MEANS command 1126
MULTIPLE IMPUTATION command 1206
OVERALS command 1407
PRINCALS command 1525
PROXIMITIES command 1556
RATIO STATISTICS command 1601, 1602
SIMPLAN command 1820, 1821
SUMMARIZE command 1913
MAX (numeric function) 61
MAX_CATEGORIES (keyword)
GENLINMIXED command 766
MAX_EFFECTS (keyword)
LINEAR command 980
MAX_STEPS (keyword)
LINEAR command 980
MAXCASEDRAWS (keyword)
MULTIPLE IMPUTATION command 1204
MAXCASES (keyword)
SIMPLAN command 1828
MAXCAT (keyword)
KNN command 975
MAXCAT (subcommand)
MVA command 1214
MAXCATS (keyword)
CODEBOOK (command) 327
MAXCOUNT (subcommand)
RESPONSE RATE command 1709
MAXEFFECT (keyword)
NOMREG command 1254
MAXEPOCHS (keyword)
MLP command 1159
maximum
EXAMINE command 644
FREQUENCIES command 720
RATIO STATISTICS command 1601, 1602
MAXIMUM (function)
GGRAPH command 823
GRAPH command 877
XGRAPH command 2168
MAXIMUM (keyword)
FREQUENCIES command 716–718, 720
IGRAPH command 931
maximum likelihood estimation
maximum likelihood estimation *(continued)*
AREG command 175
maximum-likelihood estimation
FACTOR command 672
RELIABILITY command 1655
maximum-likelihood method
VARCOMP command 2115
MAXITER (keyword)
MULTIPLE IMPUTATION command 1204
PREFSCAL command 1513
PROXSCAL command 1576
MAXITER (subcommand)
CATPCA command 279
CATREG command 295
HOMALS command 900
MULTIPLE CORRESPONDENCE command 1195
OVERALS command 1406
PRINCALS command 1524
MAXITERATIONS (keyword)
GENLIN command 732, 737
SIMPLAN command 1827
MAXMINF (keyword)
DISCRIMINANT command 587
MAXMODELPARAM (keyword)
MULTIPLE IMPUTATION command 1204
MAXNUMPEERS (keyword)
DETECTANOMALY command 580
MAXORDER (subcommand)
HILOGLINEAR command 891
MAXORDERS (subcommand)
ANOVA command 161
MAXPARAMDRAWS (keyword)
MULTIPLE IMPUTATION command 1204
MAXPCTMISSING (keyword)
MULTIPLE IMPUTATION command 1204
MAXSIZE (keyword)
NAIVEBAYES command 1230
MAXSTEPHALVING (keyword)
GENLIN command 732
MAXSTEPS (keyword)
HILOGLINEAR command 892
REGRESSION command 1641
MAXUNITS (keyword)
RBF command 1609
MAXVALORDINAL (keyword)
ADP command 117
MBP (keyword)
GLM command 850, 2076
MC (keyword)
CROSSTABS command 391
NPAR TESTS command 1292
MC_CILEVEL (keyword)
NPTESTS command 1268
MCA 165
MCA (keyword)
ANOVA command 164
MCACHE (subcommand)
SET command 1797
SHOW command 1810
MCGROUP (subcommand)
MRSETS command 1179
MCIN (keyword)
REGRESSION command 1634
MCNEMAR (keyword)

MCNEMAR (keyword) *(continued)*
NPTESTS command 1273
MCNEMAR (subcommand)
NPAR TESTS command 1286
McNemar test
CROSSTABS command 390
MCONVERT (command)
APPEND subcommand 1122
MATRIX subcommand 1122
REPLACE subcommand 1122
MDCOV (keyword)
RATIO STATISTICS command 1601, 1602
MDEPENDENT (keyword)
GENLIN command 737
MDGROUP (keyword)
MULT RESPONSE command 1187
MDGROUP (subcommand)
MRSETS command 1178
MDIAG (function)
MATRIX command 1077
MDISPLAY (subcommand)
SET command 1797
mean
EXAMINE command 644
FACTOR command 669
FREQUENCIES command 720
MEANS command 1126
RATIO STATISTICS command 1601, 1602
REGRESSION command 1643
RELIABILITY command 1655, 1656
SUMMARIZE command 1913
MEAN (function)
AGGREGATE command 125
GGRAPH command 823
GRAPH command 877
REPORT command 1685
RMV command 1714
XGRAPH command 2168
MEAN (keyword)
ANOVA command 164
CODEBOOK (command) 327
DESCRIPTIVES command 575
DISCRIMINANT command 592
FREQUENCIES command 720
IGRAPH command 930, 931
KM command 963
KNN command 973
MATRIX DATA command 1117
MEANS command 1126
MIXED command 1140
OLAP CUBES command 1296
PROXIMITIES command 1556
RANK command 1494, 1596
RATIO STATISTICS command 1601, 1602
REGRESSION command 1643
SPCHART command 1884
SUMMARIZE command 1913
MEAN (subcommand)
CSDESCRIPTIVES command 412
mean substitution
DISCRIMINANT command 593
FACTOR command 667
REGRESSION command 1646
mean-centered coefficient of variation
mean-centered coefficient of variation *(continued)*
RATIO STATISTICS command 1601, 1602
MEANCI (function)
GGRAPH command 823
MEANPRED(keyword)
GENLIN command 746
MEANS (command)
CELLS subcommand 1126
layers 1126
limitations 1125
MISSING subcommand 1128
statistics 1126
STATISTICS subcommand 1127
syntax chart 1125
TABLES subcommand 1126
MEANS (keyword)
MVA command 1215
RELIABILITY command 1656
means model
syntax 840
MEANSD (function)
GGRAPH command 823
MEANSE (function)
GGRAPH command 823
MEANSUBSTITUTION (keyword)
DISCRIMINANT command 593
FACTOR command 667
REGRESSION command 1646
MEASURE (keyword)
CODEBOOK command 325
MEASURE (subcommand)
CLUSTER command 313
CORRESPONDENCE command 359
GLM command 874
MANOVA command 1058
PROXIMITIES command 1557
measurement level
copying from other variables in current or external data file 171
default measurement level 1802
specifying 2127
unknown 58
MEASURES (keyword)
PREFSCAL command 1514
median
EXAMINE command 644
FREQUENCIES command 720
RATIO STATISTICS command 1601, 1602
MEDIAN (function)
AGGREGATE command 125
GGRAPH command 823
GRAPH command 877
REPORT command 1685
RMV command 1714
XGRAPH command 2168
MEDIAN (keyword)
CLUSTER command 317
FREQUENCIES command 720
IGRAPH command 927, 931
KNN command 973
MEANS command 1126
NPTESTS command 1271
OLAP CUBES command 1296
RATIO STATISTICS command 1601, 1602

MEDIAN (keyword) *(continued)*
SUMMARIZE command 1913
MEDIAN (subcommand)
NPAR TESTS command 1287
median method
CLUSTER command 317
median-centered coefficient of variation
RATIO STATISTICS command 1601, 1602
MEFFECT (keyword)
IGRAPH command 930
MEMALLOCATE (keyword)
NAIVEBAYES command 1230
MEMALLOCATE (subcommand)
TWOSTEP CLUSTER command 2068
MEMSIZE (keyword)
MLP command 1157
MERGESUPERVISED (keyword)
ADP command 119
MERGEUNSUPERVISED (keyword)
ADP command 119
merging data files
MATCH FILES (command) 1061
raw data files 110, 1064
MESSAGES (subcommand)
SET command 1794
SHOW command 1810
MESTIMATORS (subcommand)
EXAMINE command 644
METADATA (subcommand)
SAVE DATA COLLECTION command 1740
metafiles 1369
METHOD (keyword)
CORRELATIONS command 353
CSORDINAL command 446
GENLIN command 732
MULTIPLE IMPUTATION command 1204
OPTIMAL BINNING command 1345
METHOD (subcommand)
ALSCAL command 140
ANOVA command 161
AREG command 175
CLUSTER command 317
COXREG command 371
CROSSTABS command 391
CSPLAN command 459
DISCRIMINANT command 587
GLM command 849
HILOGLINEAR command 891
LOGISTIC REGRESSION command 993
MANOVA command 1028
MIXED command 1142
NPAR TESTS command 1292
QUICK CLUSTER command 1589
REGRESSION command 1638
RELIABILITY command 1656
TREE command 2002
UNIANOVA command 2075
VARCOMP command 2115
METRIC (keyword)
KNN command 972
MEXPAND (subcommand)
SET command 565, 1797
SHOW command 1810
MFI (keyword)
MFI (keyword) *(continued)*
NOMREG command 1256
MGET (statement)
MATRIX command 1100
MH (subcommand)
NPAR TESTS command 1287
MIDPOINT (keyword)
NPTESTS command 1268
MIN (function)
AGGREGATE command 125
REPORT command 1685
MIN (keyword)
DESCRIPTIVES command 575
IGRAPH command 920
MEANS command 1126
MULTIPLE IMPUTATION command 1206
OLAP CUBES command 1296
RATIO STATISTICS command 1601, 1602
SIMPLAN command 1820, 1821
SUMMARIZE command 1913
MIN (numeric function) 61
MINEFFECT (keyword)
NOMREG command 1254
MINEIGEN (keyword)
FACTOR command 671
MANOVA command 1049
MINIBATCH (keyword)
MLP command 1157
MINIBATCHSIZE (keyword)
MLP command 1157
minimum
EXAMINE command 644
FREQUENCIES command 720
RATIO STATISTICS command 1601, 1602
MINIMUM (function)
GGRAPH command 823
GRAPH command 877
XGRAPH command 2168
MINIMUM (keyword)
FREQUENCIES command 716–718, 720
IGRAPH command 931
MINIMUM (subcommand)
ORTHOPLAN command 1350
minimum norm quadratic unbiased estimator
VARCOMP command 2115
MINKOWSKI (keyword)
CLUSTER command 313
PROXIMITIES command 1558
Minkowski distance
CLUSTER command 313
PROXIMITIES command 1558
MINMAX (keyword)
ADP command 118
MINNUMPEERS (keyword)
DETECTANOMALY command 580
MINORITERATION (keyword)
CNLR command 1243
MINPCT (keyword)
ADP command 119
MINQUE (keyword)
VARCOMP command 2115
MINRATE (subcommand)
RESPONSE RATE command 1709
MINRESID (keyword)

MINRESID (keyword) *(continued)*
DISCRIMINANT command 587
MINSAMPLE (subcommand)
SPCHART command 1887
MINSTRESS (keyword)
PREFSCAL command 1513
PROXSCAL command 1576
MINUNITS (keyword)
RBF command 1609
MINUTE (keyword)
DATE command 547
MINVALCONTINUOUS (keyword)
ADP command 117
MINVIOLATIONS (keyword)
VALIDATEDATA command 2104
MIOUTPUT (subcommand)
SET command 1798
mismatch
MVA command 1216
MISMATCH (subcommand)
MVA command 1216
MISSING (function) 92
MISSING (keyword)
APPLY DICTIONARY command 171
CODEBOOK command 325
COUNT command 365
CTABLES command 497, 504
GGRAPH command 826
IGRAPH command 924, 928
MATRIX command 1098
MODEL HANDLE command 1170
RECODE command 1620
REPORT command 1677
ROC ANALYSIS command 1719
ROC command 1724
SUMMARIZE command 1914
WEIGHTED KAPPA command 2151
MISSING (subcommand)
AGGREGATE command 127
AIM command 132
ANOVA command 165
BAYES CORRELATION command 198
BAYES INDEPENDENT command 202
BAYES ONESAMPLE: Binomial command 210
BOOTSTRAP command 250
CATPCA command 276
CATREG command 293
CLUSTER command 319
CORRELATIONS command 351
COXREG command 373
CROSSTABS command 392
CSCOXREG command 405
CSDESCRIPTIVES command 414
CSGLM command 425
CSLOGISTIC command 436
CSORDINAL command 448
CSTABULATE command 478
DESCRIPTIVES command 576
DISCRIMINANT command 594
EXAMINE command 645
FACTOR command 667
FILE TYPE command 693
FLEISS MULTIRATER KAPPA command 706
FREQUENCIES command 721
MISSING (subcommand) *(continued)*
GENLIN command 744
GENLOG command 775
GLM command 850
GRAPH command 886
HILOGLINEAR command 895
KNN command 974
LOGISTIC REGRESSION command 998
LOGLINEAR command 1008
MANOVA command 1035
MEANS command 1128
MIXED command 1142
MLP command 1161
MULT RESPONSE command 1187
MULTIPLE CORRESPONDENCE command 1193
NAIVEBAYES command 1231
NOMREG command 1252
NONPAR CORR command 1262
NPAR TESTS command 1291
NPTESTS command 1267
ONEWAY command 1338
OPTIMAL BINNING command 1346
PARTIAL CORR command 1420
PLUM command 1440
PROBIT command 1546
PROPORTIONS command 1553
PROXIMITIES command 1562
RANK command 1597
RATIO STATISTICS command 1600
RBF command 1610
RECORD TYPE command 1629
REGRESSION command 1646
RELIABILITY command 1657
REPORT command 1691
SAVE TRANSLATE command 1763
SELECTPRED command 1783
SIMPLAN command 1829
SPCHART command 1888
SUMMARIZE command 1914
SURVIVAL command 1924
T-TEST command 2064
TARGETRESPONSE command 2011
TREE command 2010
TSAPPLY command 2022
TSET command 2026
TSMODEL command 2041
TWOSTEP CLUSTER command 2068
UNIANOVA command 2076
VARCOMP command 2115
XGRAPH command 2174
missing indicator variables
MVA command 1213
MISSING SCOPE (subcommand)
BAYES ONESAMPLE: Normal command 214
BAYES ONESAMPLE: Poisson command 217
BAYES RELATED command 228
missing summary
CTABLES command 504
Missing Value Analysis
command syntax 1211
missing value patterns
MVA command 1218
missing values
and aggregated data 127

missing values *(continued)*
and logical operators 615, 912
autorecoding blank strings to user-missing 189
CATPCA command 276
copying from other variables in current or external data file 171
counting occurrences 365
COXREG command 373
CTABLES command 493, 505
date format variables 1129
defining 1129
functions 92
GENLOG command 775
HILOGLINEAR command 895
in functions 336
in logical expressions 91
in numeric expressions 90
in transformation expressions 89
logical expressions 82
LOGISTIC REGRESSION command 998
LOGLINEAR command 1008
LOOP command 1017
MULT RESPONSE command 1187
MULTIPLE CORRESPONDENCE command 1193
NMISS function 92
NOMREG command 1252
PROBIT command 1546
ROC ANALYSIS command 1719
ROC command 1724
SPCHART (command) 1888
statistical functions 63
string expressions 91
string variables in logical expressions 83
SURVIVAL command 1924
SYSMIS function 92
system variable $SYSMIS 45
system-missing 1129
time series settings 2026
TREE command 1995
TSAPPLY command 2022
TSMODEL command 2041
user-missing 1129
VALUE function 92
WEIGHTED KAPPA command 2151
with OVERALS command 1403
with PRINCALS command 1521
MISSING VALUES (command)
syntax chart 1129
value range 1130
with RECODE command 1620
missing-value functions 338
missing-value patterns
MVA command 1217
MISSINGSUMMARIES (subcommand)
MULTIPLE IMPUTATION command 1207
MITERATE (subcommand)
SET command 565, 1798
SHOW command 1810
MIXED (command)
algorithm criteria 1138
covariance structure 1137
CRITERIA subcommand 1138
EMMEANS subcommand 1140
estimated marginal means 1140
MIXED (command) *(continued)*
fixed effects 1141
FIXED subcommand 1141
METHOD subcommand 1142
MISSING subcommand 1142
missing values 1142
model examples 1134
output 1142
overview 1134
PRINT subcommand 1142
RANDOM subcommand 1143
REGWGT subcommand 1144
REPEATED subcommand 1144
SAVE subcommand 1146
syntax chart 1133
TEST subcommand 1146
MIXED (keyword)
FILE TYPE command 688
mixed files 688, 1625
mixed models
syntax 840
VARCOMP command 2113
MIXHOLD (subcommand)
ORTHOPLAN command 1350
with HOLDOUT subcommand 1350
ML (keyword)
AREG command 175
FACTOR command 672
MIXED command 1142
VARCOMP command 2115
MLE (keyword)
GENLIN command 732
MLEVEL (keyword)
PLS command 1433
MLP (command)
ARCHITECTURE subcommand 1156
CRITERIA subcommand 1157
EXCEPT subcommand 1153
MISSING subcommand 1161
OUTFILE subcommand 1164
PARTITION subcommand 1155
PLOT subcommand 1162
PRINT subcommand 1161
RESCALE subcommand 1153
SAVE subcommand 1163
STOPPINGRULES subcommand 1159
syntax chart 1149
variable lists 1152
MLWEIGHT (keyword)
DETECTANOMALY command 580
MMATRIX (subcommand)
GLM command 841, 867
MMAX (function)
MATRIX command 1077
MMIN (function)
MATRIX command 1077
MNCOV (keyword)
RATIO STATISTICS command 1601, 1602
MNEST (subcommand)
SET command 565, 1798
SHOW command 1810
MNOM (keyword)
CATPCA command 275
OVERALS command 1405

MNOM (keyword) *(continued)*
PRINCALS command 1523
MOD (function)
MATRIX command 1077
MOD_n model names 1175
mode
FREQUENCIES command 720
MODE (function)
GGRAPH command 823
GRAPH command 877
REPORT command 1685
XGRAPH command 2168
MODE (keyword)
FREQUENCIES command 720
IGRAPH command 924, 925, 928, 931
MATRIX command 1094
TMS IMPORT command 1986
MODE (subcommand)
GETCOGNOS command 812
MODEIMPU (keyword)
CATPCA command 277
CATREG command 293
MULTIPLE CORRESPONDENCE command 1193
with ACTIVE keyword 277
with PASSIVE keyword 277
MODEL (keyword)
CSCOXREG command 409
CSGLM command 426
CSLOGISTIC command 438
CSORDINAL command 450
DETECTANOMALY command 582
GENLIN command 737, 748
GENLINMIXED command 766
KNN command 976
LINEAR command 983
LOGISTIC REGRESSION command 998
MLP command 1164
NAIVEBAYES command 1232
NOMREG command 1255
RBF command 1613
READ MODEL command 1616
SAVE MODEL command 1744
TDISPLAY command 1969
TWOSTEP CLUSTER command 2069
MODEL (subcommand)
ALSCAL command 140
ARIMA command 181
CSCOXREG command 401
CSGLM command 419
CSLOGISTIC command 430
CSORDINAL command 442
CURVEFIT command 511
EXSMOOTH command 657
GENLIN command 728
GENLOG command 773
KNN command 972
NOMREG command 1252
PLS command 1434
PREFSCAL command 1512
PROBIT command 1543
PROXSCAL command 1574
RELIABILITY command 1655
SEASON command 1772
SIMPLAN command 1819
MODEL (subcommand) *(continued)*
TMS MERGE command 1990
TSAPPLY command 2022
TSMODEL command 2041
MODEL CLOSE (command)
release history 1165
syntax chart 1165
model file
displaying information 1969
reading 1615
saving 1743, 1744
MODEL HANDLE (command)
MAP subcommand 1171
NAME subcommand 1169
OPTIONS subcommand 1170
release history 1167
syntax chart 1167
model information
exporting from DISCRIMINANT command 588
MODEL LIST (command)
release history 1173
syntax chart 1173
MODEL NAME (command)
syntax chart 1175
model names
TSMODEL command 2041
model PMML file
TREE command 2010
MODEL PROGRAM (command)
with CNLR/NLR command 1235, 1238
Model Selection Loglinear Analysis
command syntax 889
model terms
CSLOGISTIC command 430
model viewer
exporting views 1360
MODEL_SELECTION (keyword)
LINEAR command 980
MODELBUILDING (subcommand)
SPATIAL TEMPORAL PREDICTION command 1865
MODELDETAILS (subcommand)
TSAPPLY command 2018
TSMODEL command 2037
MODELFORMAT (keyword)
OMS command 1311
MODELINFO (keyword)
GENLIN command 744
MODELQUALITY (keyword)
ROC ANALYSIS command 1720
models
exporting transformations to PMML 1977
merging transformation PMML with model XML 1989
MODELS (keyword)
MULTIPLE IMPUTATION command 1208
MODELSTATISTICS (subcommand)
TSAPPLY command 2017
TSMODEL command 2037
MODELSUMMARY (subcommand)
TSAPPLY command 2016
TSMODEL command 2036
MODELSYSTEM (subcommand)
TCM ANALYSIS command 1931
TCM APPLY command 1939
MODELTABLES (subcommand)

MODELTABLES (subcommand) *(continued)*
SPATIAL ASSOCIATION RULES command 1852
SPATIAL TEMPORAL PREDICTION command 1866
MOMENTUM (keyword)
MLP command 1157
monotone spline
PROXSCAL command 1573
MONTH (keyword)
DATE command 547
MONTH format 53, 54
month of year 54
MORE (keyword)
CONJOINT command 345
MOS (subcommand)
CSPLAN command 462
MOSES (keyword)
NPTESTS command 1271
MOSES (subcommand)
NPAR TESTS command 1288
moving averages
SEASON command 1773
moving range charts
SPCHART command 1875
MOYR format 53, 54
MPATTERN (subcommand)
MVA command 1217
MPRINT (subcommand)
SET command 565, 1797
SHOW command 1810
MRBAR (keyword)
SPCHART command 1886
MRGROUP (keyword)
MULT RESPONSE command 1187
MRSETS (command)
DELETE subcommand 1179
DISPLAY subcommand 1179
MCGROUP subcommand 1179
MDGROUP subcommand 1178
release history 1177
syntax chart 1177
syntax conventions 1178
MRSETS (keyword)
APPLY DICTIONARY command 170
MRSETS (subcommand)
CTABLES command 505
MSAVE (statement)
MATRIX command 1102
MSE (keyword)
MATRIX DATA command 1117
MSSQ (function)
MATRIX command 1077
MSUM (function)
MATRIX command 1077
MTIME format 53, 54
MTINDEX (subcommand)
SET command 1798
MULT RESPONSE (command)
BASE subcommand 1186
CELLS subcommand 1186
FORMAT subcommand 1187
FREQUENCIES subcommand 1184
GROUPS subcommand 1182
limitations 1181
MISSING subcommand 1187
MULT RESPONSE (command) *(continued)*
multiple-dichotomy groups 1181
multiple-response groups 1181
PAIRED keyword 1185
TABLES subcommand 1184
VARIABLES subcommand 1183
Multidimensional Scaling
command syntax 135, 1567
Multidimensional Unfolding
command syntax 1505
Multilayer Perceptron
command syntax 1149
MULTINOMIAL (keyword)
GENLIN command 728
GENLINMIXED command 757
multinomial distribution
GENLOG command 773
Multinomial Logistic Regression
command syntax 1249
Multiplan files
read range 809
read variable names 809
saving 1753
MULTIPLE (keyword)
GRAPH command 881
PREFSCAL command 1514, 1515
multiple category group, defined 1177
multiple classification analysis
ANOVA command 165
multiple comparisons
analysis of variance 1336
in GLM 856
UNIANOVA command 2082
MULTIPLE CORRESPONDENCE (command)
ANALYSIS subcommand 1192
CONFIGURATION subcommand 1194
CRITITER subcommand 1195
DIMENSION subcommand 1194
discretization 1192
DISCRETIZATION subcommand 1192
MAXITER subcommand 1195
MISSING subcommand 1193
missing values 1193
normalization 1195
NORMALIZATION subcommand 1195
OUTFILE subcommand 1199
PLOT subcommand 1197
plots 1197
PRINT subcommand 1195
release history 1189
SAVE subcommand 1198
save variables to file 1198
supplementary objects/variables 1194
SUPPLEMENTARY subcommand 1194
syntax chart 1189
variable weight 1192
VARIABLES subcommand 1192
multiple dichotomy group, defined 1177
Multiple Imputation
command syntax 1201
MULTIPLE IMPUTATION (command)
ANALYSISWEIGHT subcommand 1209
CONSTRAINTS subcommand 1206
IMPUTATIONSUMMARIES subcommand 1208

MULTIPLE IMPUTATION (command) *(continued)*
IMPUTE subcommand 1204
MISSINGSUMMARIES subcommand 1207
OUTFILE subcommand 1209
syntax chart 1201
variable lists 1204
multiple R
REGRESSION command 1640
multiple response analysis
defining sets 1181
multiple category 1181
multiple dichotomy 1181
multiple response sets
CODEBOOK command 323
copying sets from another data file 170
CTABLES command 486, 505
functions in CTABLES command 486
MULTIPLESPACES (subcommand)
GET DATA command 791
MULTIPLICATIVE (keyword)
SEASON command 1772
multiplicative model
SEASON command 1772
MULTIPLY (function)
REPORT command 1686
MULTIPLYING (keyword)
CATPCA command 276
CATREG command 292
MULTIPLE CORRESPONDENCE command 1192
MULTIPUNCH (keyword)
FILE HANDLE command 680
multipunch data 680
MULTISTART (keyword)
CATREG command 294
multithreading 1797, 1804
MULTIV (keyword)
MANOVA command 1048
MUPLUS (keyword)
MANOVA command 1040
MVA (command)
CATEGORICAL subcommand 1214
CROSSTAB subcommand 1216
DPATTERN subcommand 1217
EM subcommand 1219
extreme values 1213
ID subcommand 1214
LISTWISE subcommand 1218
MAXCAT subcommand 1214
MISMATCH subcommand 1216
missing indicator variables 1213
MPATTERN subcommand 1217
NOUNIVARIATE subcommand 1214
PAIRWISE subcommand 1218
REGRESSION subcommand 1220
saving imputed data 1219
summary tables 1212
symbols 1213
syntax chart 1211
TPATTERN subcommand 1218
TTEST subcommand 1215
VARIABLES subcommand 1213
MWITHIN (keyword)
MANOVA command 1039, 1057
MXAUTO (subcommand)
MXAUTO (subcommand) *(continued)*
ACF command 100
PACF command 1414
MXBRANCH (keyword)
TWOSTEP CLUSTER command 2067
MXCELLS (subcommand)
SHOW command 1799, 1810
MXCROSS (subcommand)
CCF command 302
MXERRS (subcommand)
SET command 1799
MXITER (keyword)
CSCOXREG command 403
CSLOGISTIC command 435
CSORDINAL command 446
MIXED command 1138
NOMREG command 1251
PLUM command 1438
QUICK CLUSTER command 1588
MXITER (subcommand)
AREG command 175
ARIMA command 184
MXLAMB (subcommand)
ARIMA command 184
MXLEVEL (keyword)
TWOSTEP CLUSTER command 2067
MXLOOPS (subcommand)
SET command 1799
SHOW command 1810
with LOOP command 1011–1013
MXMEMORY (subcommand)
SHOW command 1799, 1810
MXNEWVAR (subcommand)
TSET command 2026
MXPREDICT (subcommand)
TSET command 2026
MXSTEP (keyword)
CSCOXREG command 403
CSLOGISTIC command 435
CSORDINAL command 446
MIXED command 1138
NOMREG command 1251
PLUM command 1438
MXWARNS (subcommand)
SET command 1799
SHOW command 1810

N

N (function)
AGGREGATE command 125
GRAPH command 877
N (keyword)
IGRAPH command 924–926, 928, 929
MATRIX DATA command 1117
REGRESSION command 1643
SPCHART command 1884
N (subcommand)
MATRIX DATA command 1119
RANK command 1594
SHOW command 1810
N OF CASES (command)
with SAMPLE command 1223, 1727
with SELECT IF command 1223, 1775

N OF CASES (command) *(continued)*
with TEMPORARY command 1223
N_MATRIX (keyword)
MATRIX DATA command 1117
N_SCALAR (keyword)
MATRIX DATA command 1117
N_VECTOR (keyword)
MATRIX DATA command 1117
NA (keyword)
EXSMOOTH command 657
Naïve Bayes
command syntax 1225
NAIVEBAYES (command)
CRITERIA subcommand 1230
EXCEPT subcommand 1228
FORCE subcommand 1229
MISSING subcommand 1231
OUTFILE subcommand 1232
PRINT subcommand 1231
release history 1225
SAVE subcommand 1231
SUBSET subcommand 1230
syntax chart 1225
TRAININGSAMPLE subcommand 1229
NAME (keyword)
CODEBOOK command 326
DESCRIPTIVES command 575
GGRAPH command 821
MODEL CLOSE command 1165
REPORT command 1683
NAME (qualifier)
GGRAPH command 822
NAME (subcommand)
MODEL HANDLE command 1169
NAMES (keyword)
DISPLAY command 599
MATRIX command 1098
NAMES (subcommand)
SAVE command 1733
NATRES (subcommand)
PROBIT command 1544
natural log transformation
ACF command 99
ARIMA command 181
CCF command 301
in sequence charts 258, 2054
PACF command 1413
TSMODEL command 2044–2046
natural response rate
PROBIT command 1544
NCASES (keyword)
SIMPLAN command 1827
NCAT (keyword)
CATPCA command 276
CATREG command 293
MULTIPLE CORRESPONDENCE command 1193
with GROUPING keyword 276
NCDF functions 64
NCDF.BETA (function)
MATRIX command 1082
NCDF.CHISQ (function)
MATRIX command 1082
NCDF.F (function)
MATRIX command 1082
NCDF.T (function)
MATRIX command 1082
NCOL (function)
MATRIX command 1077
NCOMP (keyword)
MANOVA command 1049
NDIM (keyword)
ANACOR command 154
CATPCA command 282
CORRESPONDENCE command 361
HOMALS command 900
MULTIPLE CORRESPONDENCE command 1197
OVERALS command 1407
PRINCALS command 1525
nearest neighbor method
CLUSTER command 317
NEGATIVE (keyword)
ALSCAL command 141
negative binomial distribution function 64
negative log-log link
PLUM command 1439
NEGBIN (keyword)
GENLIN command 728
GENLINMIXED command 757
NEIGHBORS (keyword)
KNN command 972
NEQ (function)
GGRAPH command 823
NEQ (keyword)
IGRAPH command 926, 931
NESTED (keyword)
FILE TYPE command 688
nested conditions 617
nested design
in GLM 862
UNIANOVA command 2088
VARCOMP command 2117
nested files 688, 1625
nesting
CTABLES command 485
multiple 862
NETWORK (keyword)
MLP command 1162
RBF command 1611
NETWORKINFO (keyword)
MLP command 1161
RBF command 1610
NEW FILE (command)
syntax chart 1233
NEWNAMES (subcommand)
FLIP command 710
NEWTON (keyword)
CSORDINAL command 446
GENLIN command 732
NEWVAR (subcommand)
TSET command 2026
NEWVARS (subcommand)
APPLY DICTIONARY command 169
NFTOLERANCE (keyword)
CNLR command 1243
NGE (function)
GGRAPH command 823
NGE (keyword)
IGRAPH command 926, 931

NGT (function)
GGRAPH command 823
GRAPH command 877
XGRAPH command 2168
NGT (keyword)
IGRAPH command 926, 931
NIMPUTATIONS (keyword)
MULTIPLE IMPUTATION command 1204
NIN (function)
GGRAPH command 823
GRAPH command 877
XGRAPH command 2168
NIN (keyword)
IGRAPH command 926, 931
NLE (function)
GGRAPH command 823
NLE (keyword)
IGRAPH command 926, 931
NLOGLOG (keyword)
CSORDINAL command 442
GENLIN command 728
GENLINMIXED command 757
PLUM command 1439
NLR (command)
constrained functions 1239
crash tolerance 1243
CRITERIA subcommand 1242, 1244
critical value for derivative checking 1242
dependent variable 1240
derivatives 1238
DERIVATIVES command 1235, 1238
feasibility tolerance 1243
FILE subcommand 1240
function precision 1243
infinite step size 1243
iteration criteria 1244
Levenberg-Marquardt method 1244
line-search tolerance 1243
linear feasibility tolerance 1243
major iterations 1243
maximum iterations 1243, 1244
minor iterations 1243
missing values 1237
model expression 1238
model program 1238
nonlinear feasibility tolerance 1243
optimality tolerance 1243
OUTFILE subcommand 1240
parameter convergence 1244
parameters 1238
PRED subcommand 1241
residual and derivative correlation convergence 1244
SAVE subcommand 1241
saving new variables 1241
saving parameter estimates 1240
sequential quadratic programming 1243
step limit 1243
sum-of-squares convergence 1244
syntax chart 1235
using parameter estimates from previous analysis 1240
weighting cases 1237
with MODEL PROGRAM command 1235, 1238
NLT (function)
GGRAPH command 823
NLT (function) *(continued)*
GRAPH command 877
XGRAPH command 2168
NLT (keyword)
IGRAPH command 926, 931
NM (keyword)
EXSMOOTH command 657
NMISS (function)
AGGREGATE command 125
NN (keyword)
EXSMOOTH command 657
NO (keyword)
AIM command 132
CASESTOVARS command 267
CSGLM command 420
CSLOGISTIC command 430, 431
GENLIN command 728, 737
MLP command 1156
SET command 1788
no trend model
EXSMOOTH command 657
NOBSERVATIONS (subcommand)
HOMALS command 899
OVERALS command 1406
PRINCALS command 1524
NOCASENUM (keyword)
SUMMARIZE command 1914
NOCONFORM (subcommand)
SPCHART command 1887
NOCONSTANT (keyword)
ARIMA command 181
NOCONSTANT (subcommand)
AREG command 175
CURVEFIT command 512
WLS command 2156
NOCONSTANT subcommand
2SLS command 95
NOCOUNTS (keyword)
MVA command 1215
NOCUM (keyword)
GRAPH command 883
nodes
saving terminal node number as variable 2001
NODF (keyword)
MVA command 1215
NODIAGONAL (keyword)
MATRIX DATA command 1114
NOFILL (keyword)
TSPLOT command 2054
NOINITIAL (keyword)
QUICK CLUSTER command 1588
NOINT (keyword)
MIXED command 1141
NOJOIN (keyword)
TSPLOT command 2054
NOKAISER (keyword)
FACTOR command 671
NOLABELS (keyword)
MULT RESPONSE command 1187
NOLIST (keyword)
REPORT command 1677
SUMMARIZE command 1914
NOLOG (keyword)
ARIMA command 181

NOLOG (subcommand)
ACF command 99
CCF command 301
PACF command 1413
PPLOT command 1497
TSPLOT command 2054
NOMEANS (keyword)
MVA command 1215
NOMI (keyword)
CATPCA command 275
CATREG command 292
nominal
ALSCAL command 138
NOMINAL (keyword)
ALSCAL command 138
PROXSCAL command 1575
with VARIABLES keyword 1575
Nominal Regression procedure
variable list 1250
NOMREG (command)
BY keyword 1250
CRITERIA subcommand 1251
FULLFACTORIAL subcommand 1252
INTERCEPT subcommand 1252
MISSING subcommand 1252
missing values 1252
MODEL subcommand 1252
OUTFILE subcommand 1255
PRINT subcommand 1256
release history 1249
SCALE subcommand 1257
SUBPOP subcommand 1257
syntax chart 1249
TEST subcommand 1258
WITH keyword 1250
NONAME (keyword)
REPORT command 1683
noncentral cumulative distribution functions 64
noncentral probability density functions 64
NONE (keyword)
ADP command 118, 119
AIM command 132
ANACOR command 154
ANOVA command 161, 164
CATPCA command 281, 282
CATREG command 295, 296
CLUSTER command 318, 319
CODEBOOK (command) 327
CONJOINT command 347
CORRESPONDENCE command 361
COXREG command 374
CROSSTABS command 389, 390, 393
CSCOXREG command 406
CSGLM command 425
CSLOGISTIC command 437
CSORDINAL command 449
CURVEFIT command 512
DETECTANOMALY command 582
DISCRIMINANT command 593
EXAMINE command 642–644
FREQUENCIES command 720
GENLIN command 741, 744
HILOGLINEAR command 894
HOMALS command 900
NONE (keyword) *(continued)*
IGRAPH command 921, 924, 928–930
KNN command 971, 975
LINEAR command 980
MEANS command 1127
MLP command 1153, 1161, 1162
MULTIPLE CORRESPONDENCE command 1195, 1197
MULTIPLE IMPUTATION command 1207, 1208
NAIVEBAYES command 1231
NOMREG command 1256
NPTESTS command 1271, 1273
OPTIMAL BINNING command 1346
OVERALS command 1407
PARTIAL CORR command 1419
PREFSCAL command 1511, 1512, 1514, 1515
PRINCALS command 1524, 1525
PROXSCAL command 1575–1577
QUICK CLUSTER command 1590
RBF command 1607, 1610, 1611
REPORT command 1691
ROC command 1725
SELECTPRED command 1783
SET command 1794
SPECTRA command 1893
SUMMARIZE command 1915
SURVIVAL command 1924
TSET command 2026
VALIDATEDATA command 2103, 2104
NONE (subcommand
VARIABLE ROLE command 2129
Nonlinear Canonical Correlation Analysis
command syntax 1403
nonlinear constraints 1239
Nonlinear Regression
command syntax 1235
NONMISSING (keyword)
DISCRIMINANT command 593
NONORMAL (keyword)
FREQUENCIES command 718
NONPAR CORR (command)
CI subcommand 1263
limitations 1259
matrix output 1259
MATRIX subcommand 1262
MISSING subcommand 1262
missing values 1263
PRINT subcommand 1261
random sampling 1259, 1261
SAMPLE subcommand 1261
significance tests 1259, 1261
syntax chart 1259
VARIABLES subcommand 1260
with RECODE command 1262
NONPARALLEL (subcommand)
CSORDINAL command 447
nonparametric correlation
NONPAR CORR command 1259
nonparametric tests
command syntax 1265
NOORIGIN (subcommand)
LOGISTIC REGRESSION command 995
REGRESSION command 1642
NOPRINT (subcommand)
LOGLINEAR command 1007

NOPRINT (subcommand) *(continued)*
MANOVA command 1029, 1047
NOPROB (keyword)
MVA command 1215
NOREFERENCE (keyword)
TSPLOT command 2054
NOREPORT (keyword)
EXAMINE command 645
GRAPH command 886
NORMAL (function) 70, 1086
NORMAL (keyword)
CATPCA command 276
CATREG command 293
FREQUENCIES command 718
GENLIN command 728
GENLINMIXED command 757
IGRAPH command 930
MULTIPLE CORRESPONDENCE command 1193
MVA command 1220
NPTESTS command 1268
PPLOT command 1495
with DISTR keyword 276
NORMAL (subcommand)
RANK command 1594
SPCHART command 1888
normal distribution function 64
normal probability plots
EXAMINE command 643
GENLOG command 774
HILOGLINEAR command 894
LOGLINEAR command 1008
REGRESSION command 1646
normalization
CORRESPONDENCE command 360
MULTIPLE CORRESPONDENCE command 1195
NORMALIZATION (subcommand)
ANACOR command 153
CATPCA command 278
CORRESPONDENCE command 360
MULTIPLE CORRESPONDENCE command 1195
with PLOT subcommand 154
NORMALIZE (function) 78
NORMALIZE (subcommand)
IGRAPH command 920
NORMALIZED (keyword)
MLP command 1153
RBF command 1607
normalized raw Stress
PROXSCAL command 1576
NORMPLOT (keyword)
HILOGLINEAR command 894
NORMPROB (keyword)
REGRESSION command 1646
NORMS (keyword)
DETECTANOMALY command 582
NOROTATE (keyword)
CATPCA command 279
FACTOR command 673
MANOVA command 1049
NOSELECTION (keyword)
NAIVEBAYES command 1230
NOSIG (keyword)
CORRELATIONS command 351
NONPAR CORR command 1261
NOSORT (keyword)
MVA command 1216–1218
RATIO STATISTICS command 1600
NOSTANDARDIZE (subcommand)
PPLOT command 1496
TWOSTEP CLUSTER command 2068
NOT (keyword)
MVA command 1215
NOTABLE (keyword)
FREQUENCIES command 716
SURVIVAL command 1922
NOTABLE (subcommand)
DATA LIST command 521
KEYED DATA LIST command 957
PRINT command 1533
REPEATING DATA command 1672
WRITE command 2162
NOTABLES (keyword)
CROSSTABS command 393
NOTOTAL (keyword)
SUMMARIZE command 1914
NOTOTAL (subcommand)
EXAMINE command 641
NOULB (keyword)
ALSCAL command 141
NOUNIVARIATE (subcommand)
MVA command 1214
NOWARN (keyword)
FILE TYPE command 692
RECORD TYPE command 1629
SET command 1805
NOWARN (subcommand)
OMS command 1315
NP (subcommand)
data organization 1878
SPCHART command 1877
variable specification 1878
np charts
SPCHART command 1877
NPAR TESTS (command)
BINOMIAL subcommand 1279
CHISQUARE subcommand 1280
COCHRAN subcommand 1281
FRIEDMAN subcommand 1281
independent-samples test 1278
K-S subcommand 1282, 1283
KENDALL subcommand 1285
KS_SIM subcommand 1284
limitations 1278
M-W subcommand 1286
MCNEMAR subcommand 1286
MEDIAN subcommand 1287
METHOD subcommand 1292
MH subcommand 1287
MISSING subcommand 1291
MOSES subcommand 1288
one-sample test 1278
random sampling 1292
related-samples test 1278
RUNS subcommand 1289
SAMPLE subcommand 1292
SIGN subcommand 1289
STATISTICS subcommand 1291
W-W subcommand 1290

NPAR TESTS (command) *(continued)*
WILCOXON subcommand 1291
NPCT (keyword)
LAYERED REPORTS command 1913
MEANS command 1126
OLAP CUBES command 1296
NPCT(var) (keyword)
MEANS command 1126
NPDF functions 64
NPDF.BETA (function) 66, 1081
NPDF.CHISQ (function) 66, 1081
NPDF.F (function) 66, 1081
NPDF.T (function) 66, 1081
NPPLOT (keyword)
EXAMINE command 643
NPREDICTORS (keyword)
MVA command 1220
NPTESTS (command)
CRITERIA subcommand 1267
INDEPENDENT subcommand 1271
MISSING subcommand 1267
ONESAMPLE subcommand 1268
RELATED subcommand 1273
syntax chart 1265
NRBF (keyword)
RBF command 1609
NROW (function)
MATRIX command 1077
NSAMPLES (keyword)
BOOTSTRAP command 249
NPTESTS command 1268
NTILES (subcommand)
FREQUENCIES command 719
NTILES(k) (subcommand)
RANK command 1594
NTRIM (function) 78
NU (function)
AGGREGATE command 125
NUM (keyword)
IGRAPH command 930
NUMANOMALOUSCASES (keyword)
DETECTANOMALY command 580
NUMBER (function) 81
NUMBERED (keyword)
LIST command 986
numbers
converting to strings 81
NUMCLUSTERS (subcommand)
TWOSTEP CLUSTER command 2069
NUME (keyword)
CATPCA command 275
CATREG command 292
OVERALS command 1405
PRINCALS command 1523
NUMERIC (command)
formats 1293
syntax chart 1293
with DATA LIST command 1293
with INPUT PROGRAM command 1293
with SET command 1293
numeric data
input formats 515, 527
output formats 1537, 2165
numeric expressions
numeric expressions *(continued)*
missing values 90
NUMERICAL (keyword)
CATREG command 294
OVERALS command 1406
numerical scaling level
PROXSCAL command 1573
NUMFEATURES (keyword)
KNN command 973
NUMIN (keyword)
IGRAPH command 926
NUMISS (function)
AGGREGATE command 125
NUMREASONS (keyword)
DETECTANOMALY command 580
NUMUNITS (keyword)
RBF command 1609
NVALID (function) 92

O

OATTRS (subcommand)
SET command 1800
SHOW command 1810
OBELISK (keyword)
IGRAPH command 925
OBELLAREA (keyword)
CATPCA command 282
OBJECT (keyword)
CATPCA command 277, 281, 282, 284, 286
CATREG command 293
HOMALS command 900
MULTIPLE CORRESPONDENCE command 1194, 1195, 1197–1199
OVERALS command 1407
PRINCALS command 1524, 1525
object points plots
CATPCA command 282
MULTIPLE CORRESPONDENCE command 1197
object principal normalization
MULTIPLE CORRESPONDENCE command 1195
object scores
CATPCA command 281
MULTIPLE CORRESPONDENCE command 1195
saving HOMALS command 902
saving OVERALS command 1408
OBJECTIVE (keyword)
LINEAR command 980
OBJECTPROPERTIES (subcommand)
OUTPUT MODIFY command 1380
OBLIMIN (keyword)
CATPCA command 279
FACTOR command 673
oblimin rotation
FACTOR command 673
oblique rotation
FACTOR command 673
OBS (keyword)
DATE command 547
OBS (subcommand)
FIT command 702
observed count
REGRESSION command 1643
observed frequencies

observed frequencies *(continued)*
GENLOG command 773
HILOGLINEAR command 894
LOGLINEAR command 1007
PROBIT command 1545
observed power
UNIANOVA command 2076
OBSVALPROB (keyword)
CSORDINAL command 449
OCCURS (subcommand)
REPEATING DATA command 1668
OCHIAI (keyword)
CLUSTER command 314
PROXIMITIES command 1559
Ochiai measure
CLUSTER command 314
PROXIMITIES command 1559
OCORR (keyword)
CATPCA command 281
CATREG command 296
MULTIPLE CORRESPONDENCE command 1195
ODBC (keyword)
GET DATA command 786
ODBCOPTIONS (subcommand)
SAVE TRANSLATE command 1761
odds ratio
CSLOGISTIC command 433
ODDSPOWER (keyword)
GENLIN command 728
ODDSRATIO (keyword)
CSTABULATE command 477
ODDSRATIOS (subcommand)
CSLOGISTIC command 433
CSORDINAL command 444
ODISPLAY (subcommand)
SET command 1800
SHOW command 1810
OF (keyword)
PROBIT command 1542
OFF (keyword)
SPLIT FILE command 1897
OFFSET (keyword)
GENLIN command 728
GENLINMIXED command 757
REPORT command 1680, 1683
OLANG (subcommand)
SET command 1801
SHOW command 1810
OLAP CUBES (command)
CELLS subcommand 1296
CREATE subcommand 1297
FOOTNOTE subcommand 1296
HIDESMALLCOUNTS subcommand 1299
syntax chart 1295
TITLE subcommand 1296
OLEDB (keyword)
GET DATA command 786
OMEANS (subcommand)
MANOVA command 1032, 1044
OMEGA (keyword)
PREFSCAL command 1513
RELIABILITY command 1655
OMS (command)
CHARTFORMAT keyword 1311
OMS (command) *(continued)*
CHARTSIZE keyword 1310
COLUMNS subcommand 1313
DESTINATION subcommand 1308
EXCEPTIF subcommand 1308
IF subcommand 1306
IMAGEMAP keyword 1310
IMAGEROOT keyword 1310
IMAGES keyword 1309
MODELFORMAT keyword 1311
NOWARN subcommand 1315
release history 1301
SELECT subcommand 1303
syntax chart 1301
TABLES keyword 1312
TAG subcommand 1315
TREEFORMAT keyword 1311
OMS (keyword)
COXREG command 374
CSCOXREG command 408
KM command 962
SURVIVAL command 1921
OMSEND (command)
command syntax 1327
OMSINFO (command)
syntax chart 1329
OMSLOG (command)
syntax chart 1331
one-minus-survival plots
COXREG command 374
KM command 962
SURVIVAL command 1921
one-sample nonparametric tests
command syntax 1265
ONEBREAKCOLUMN (keyword)
REPORT command 1677
ONEPAGE (keyword)
MULT RESPONSE command 1187
ONESAMPLE (subcommand)
NPTESTS command 1268
PROPORTIONS command 1551
ONETAIL (keyword)
CORRELATIONS command 351
NONPAR CORR command 1261
PARTIAL CORR command 1419
ONEWAY (command)
analysis design 1334
CONTRAST subcommand 1335
contrasts 1335
CRITERIA subcommand 1340
defining factor ranges 1334
ES subcommand 1341
factor variables 1334
limitations 1333
matrix input 1338
matrix output 1338
MATRIX subcommand 1338
MISSING subcommand 1338
missing values 1338, 1339
multiple comparisons 1336
orthogonal polynomials 1335
PLOT MEANS subcommand 1338
POLYNOMIAL subcommand 1335
post hoc tests 1336

ONEWAY (command) *(continued)*
RANGES subcommand 1337
statistics 1338
STATISTICS subcommand 1338
syntax chart 1333
TEMPLATE subcommand 1340
with MATRIX DATA command 1109
ONLINE (keyword)
MLP command 1157
ONLY (keyword)
CSGLM command 420
CSLOGISTIC command 430
ONUMBERS (subcommand)
SET command 1801
SHOW command 1810
OPOWER (keyword)
GLM command 850
UNIANOVA command 2076
OPRINCIPAL (keyword)
CATPCA command 278
MULTIPLE CORRESPONDENCE command 1195
OPTIMAL (keyword)
MANOVA command 1030
Optimal Binning
command syntax 1343
OPTIMAL BINNING (command)
CRITERIA subcommand 1345
MISSING subcommand 1346
OUTFILE subcommand 1346
PRINT subcommand 1346
release history 1343
syntax chart 1343
VARIABLES subcommand 1344
optimal scaling level
CATPCA command 275
numerical 1573
ordinal 1573
OVERALS command 1405
optimality tolerance
PROBIT command 1544
OPTIMIZATION (keyword)
MLP command 1157
options
displaying 1809
OPTIONS (subcommand)
CODEBOOK command 327
MODEL HANDLE command 1170
PREFSCAL command 1516
TCM ANALYSIS command 1935
TCM APPLY command 1939
OPTOL (keyword)
PROBIT command 1544
OPTOLERANCE (keyword)
CNLR command 1243
ORBF (keyword)
RBF command 1609
ORDER (keyword)
CTABLES command 497
GENLIN command 726
NPTESTS command 1271
order of commands 36
order of operations
numeric expressions 59
ORDERED (subcommand)
ORDERED (subcommand) *(continued)*
FILE TYPE command 694
ordering categories
interactive charts 919
ORDI (keyword)
CATPCA command 275
CATREG command 292
OVERALS command 1405
PRINCALS command 1523
ordinal
ALSCAL command 138
ORDINAL (keyword)
ALSCAL command 138
PREFSCAL command 1511
PROXSCAL command 1573, 1575
with VARIABLES keyword 1575
Ordinal Regression
command syntax 1437
ordinal scaling level
PROXSCAL command 1573
ORIGIN (keyword)
IGRAPH command 930
ORIGIN (subcommand)
LOGISTIC REGRESSION command 995
REGRESSION command 1642
ORIGINAL (keyword)
CSCOXREG command 406
GENLIN command 741
GENLINMIXED command 765
orthogonal contrasts 855, 2081
orthogonal polynomials
analysis of variance 1335
orthogonal rotation
FACTOR command 673
ORTHONORM (keyword)
MANOVA command 1045
ORTHOPLAN (command)
appending to active datasets 1349
CARD_ variable 1347
duplicate cases 1347
FACTORS subcommand 1349
holdout cases 1347
HOLDOUT subcommand 1350
minimum number of cases 1350
MINIMUM subcommand 1350
MIXHOLD subcommand 1350
REPLACE subcommand 1349
replacing active system file 1349
STATUS_ variable 1347
syntax chart 1347
value labels 1349
with CONJOINT command 341
with PLANCARDS command 1425
with SET SEED command 1347
with VALUE LABELS command 1349
OTHER (keyword)
CSGLM command 422
RECORD TYPE command 1627
OUT (keyword)
ANACOR command 155
CLUSTER command 319
CORRELATIONS command 352
DISCRIMINANT command 594
FACTOR command 675

OUT (keyword) *(continued)*
HOMALS command 902
MANOVA command 1036
NONPAR CORR command 1262
ONEWAY command 1338
PARTIAL CORR command 1420
PROXIMITIES command 1562
REGRESSION command 1645
RELIABILITY command 1657
OUTDATASET (subcommand)
PLS command 1434
OUTFILE (keyword)
MATRIX command 1095, 1099, 1103
MVA command 1219, 1220
OUTFILE (subcommand)
ADP command 120
AGGREGATE command 122
ALSCAL command 143
BAYES REGRESSION command 225, 1584
BEGIN EXPR command 238
CATPCA command 286
CATREG command 298
CNLR/NLR command 1240
CORRESPONDENCE command 362
COXREG command 375
CSCOXREG command 409
CSGLM command 426
CSLOGISTIC command 438
CSORDINAL command 450
DETECTANOMALY command 582
DISCRIMINANT command 588
EXPORT command 651
GENLIN command 748
GENLINMIXED command 766
GLM command 861
KNN command 976
LINEAR command 983
LOGISTIC REGRESSION command 998
MLP command 1164
MULTIPLE CORRESPONDENCE command 1199
MULTIPLE IMPUTATION command 1209
NAIVEBAYES command 1232
NOMREG command 1255
OPTIMAL BINNING command 1346
PLANCARDS command 1427
PREFSCAL command 1517
PRINT command 1532
PRINT SPACE command 1539
PROCEDURE OUTPUT command 1547
PROXSCAL command 1578
RATIO STATISTICS command 1601
RBF command 1613
REGRESSION command 1649
REPORT command 1678
SAVE CODEPAGE command 1736
SAVE command 1731
SAVE DATA COLLECTION command 1740
SAVE MODEL command 1744
SAVE TRANSLATE command 1756
SIMPREP BEGIN command 1831
SIMRUN command 1839
SORT CASES command 1842
START JOIN command 1902
TCM APPLY command 1946
OUTFILE (subcommand) *(continued)*
TCM MODEL command 1965
TMS IMPORT command 1986
TWOSTEP CLUSTER command 2069
UNIANOVA command 2087
VARCOMP command 2116
WRITE command 2162
XSAVE command 2183
OUTLIER (subcommand)
TSMODEL command 2049
OUTLIERHANDLING (subcommand)
ADP command 118
outliers
identifying 642
REGRESSION command 1646, 1647
TSMODEL command 2048, 2049
OUTLIERS (keyword)
IGRAPH command 927
LOGISTIC REGRESSION command 997
REGRESSION command 1646, 1647
output
changing output language 1801
exporting 1301
saving as data files 1301
OUTPUT (keyword)
SIMPLAN command 1820, 1821
OUTPUT (subcommand)
COMPARE DATASETS command 334
OUTPUT ACTIVATE (command)
release history 1351
syntax chart 1351
OUTPUT CLOSE (command)
release history 1353
syntax chart 1353
OUTPUT DISPLAY (command)
release history 1355
syntax chart 1355
OUTPUT EXPORT (command)
BMP subcommand 1368
CONTENTS subcommand 1360
DOC subcommand 1361
EMF subcommand 1369
EPS subcommand 1369
HTML subcommand 1362
JPG subcommand 1370
NAME keyword 1360
PDF subcommand 1363
PNG subcommand 1370
PPT subcommand 1364
syntax chart 1357
TEXT subcommand 1365
TIF subcommand 1371
XLS subcommand 1367
output files
borders for tables 1790
chart characters 1790
destination of 1794
display command syntax 1794
display output page titles 1795
output formats
custom currency 713, 1537, 2165
displaying 1537, 2165
format specification 1537, 2165
string variables 713

output formats *(continued)*
write 2165
OUTPUT MODIFY (command)
DELETEOBJECT subcommand 1379
GRAPHS subcommand 1388
IF subcommand 1377
INDEXING subcommand 1379
NAME keyword 1375
OBJECTPROPERTIES subcommand 1380
REPORT subcommand 1390
SELECT subcommand 1376
TABLE subcommand 1381
TABLECELLS subcommand 1384
TEXTS subcommand 1389
OUTPUT NAME (command)
release history 1391
syntax chart 1391
OUTPUT NEW (command)
release history 1393
syntax chart 1393
OUTPUT OPEN (command)
release history 1395
syntax chart 1395
OUTPUT SAVE (command)
PASSPROTECT subcommand 1401
release history 1399
syntax chart 1399
OUTPUTFILTER (subcommand)
TSAPPLY command 2019
TSMODEL command 2038
OUTPUTFUNCTION (keyword)
MLP command 1156
OUTS (keyword)
REGRESSION command 1640
OUTSIDE (keyword)
IGRAPH command 925, 926
OVARS (subcommand)
SET command 1801
SHOW command 1810
OVERALL (keyword)
ES subcommand 1341
KM command 964
MIXED command 1140
MULTIPLE IMPUTATION command 1207
OVERALS (command)
active variables 1405
ANALYSIS subcommand 1405
compared with HOMALS 1405
compared with PRINCALS 1405
CONVERGENCE subcommand 1406
DIMENSION subcommand 1406
dimensions 1407
excluding cases 1406
INITIAL subcommand 1406
matrix output 1409
MATRIX subcommand 1409
MAXITER subcommand 1406
NOBSERVATIONS subcommand 1406
optimal scaling level 1405
passive variables 1405
PLOT subcommand 1407
PRINT subcommand 1407
SAVE subcommand 1408
SETS subcommand 1405
OVERALS (command) *(continued)*
syntax chart 1403
value labels 1407
variable labels 1407
VARIABLES subcommand 1405
with AUTORECODE command 1403
with RECODE command 1403
OVERLAP (keyword)
RBF command 1609
OVERLAY (keyword)
GRAPH command 882
OVERLAYTARGETS (keyword)
SIMRUN command 1837

P

P (keyword)
HILOGLINEAR command 892
PROBIT command 1544
SPECTRA command 1893, 1895
P (subcommand)
ARIMA command 182
data organization 1878
SPCHART command 1877
variable specification 1878
p charts
SPCHART command 1877
P-P (keyword)
PPLOT command 1494
PA1 (keyword)
FACTOR command 672
PA2 (keyword)
FACTOR command 672
PACF (command)
APPLY subcommand 1414
DIFF subcommand 1412
LN/NOLOG subcommands 1413
MXAUTO subcommand 1414
PERIOD subcommand 1413
periodic lags 1413
SDIFF subcommand 1412
SEASONAL subcommand 1413
specifying periodicity 1413
syntax chart 1411
transforming values 1412
using a previously defined model 1414
VARIABLES subcommand 1412
PACF (subcommand)
ACF command 101
padding strings 91
PADJUST (keyword)
CSCOXREG command 404
CSORDINAL command 447
GENLIN command 741
GENLINMIXED command 765
PAF (keyword)
FACTOR command 672
PAGE (argument)
REPORT command 1690
PAGE (keyword)
REPORT command 1677, 1683
page ejection
missing values 1535
variable list 1535

PAIR (keyword)
ROC ANALYSIS command 1719
PAIR (subcommand)
BAYES RELATED command 229
PAIRED (keyword)
MULT RESPONSE command 1185
NPAR TESTS command 1278
T-TEST command 2063
paired-samples nonparametric tests
command syntax 1265
PAIREDSAMPLES (subcommand)
PROPORTIONS command 1552
PAIRS (subcommand)
T-TEST command 2063
PAIRWISE (keyword)
CORRELATIONS command 351
EXAMINE command 645
FACTOR command 667
GENLIN command 741
GENLINMIXED command 764
KM command 964
NONPAR CORR command 1262
NPTESTS command 1271, 1273
OPTIMAL BINNING command 1346
REGRESSION command 1646
SURVIVAL command 1922
PAIRWISE (subcommand)
MVA command 1218
pairwise comparisons
CROSSTABS command 389
CTABLES command 502
PANEL (subcommand)
GRAPH command 883
IGRAPH command 921
XGRAPH command 2174
paneled charts 883, 2174
PARALL (keyword)
PROBIT command 1545
PARALLEL (keyword)
PLUM command 1440
RELIABILITY command 1655
parallel model
RELIABILITY command 1655
parallelism test
PROBIT command 1545
PARAMETER (keyword)
COXREG command 375
CSCOXREG command 404, 405
CSGLM command 424, 426
CSLOGISTIC command 435, 438
CSORDINAL command 447, 450
GENLIN command 748
GLM command 850
LOGISTIC REGRESSION command 998
NOMREG command 1255, 1256
PLUM command 1440
REGRESSION command 1649
UNIANOVA command 2076
parameter estimates
COXREG command 373
CSGLM command 424
CSLOGISTIC command 435
GENLOG command 773
HILOGLINEAR command 894
parameter estimates *(continued)*
in GLM 850
LOGLINEAR command 1007
MIXED command 1142
UNIANOVA command 2076
parameter-order subcommands
ARIMA command 182
PARAMETERS (keyword)
MANOVA command 1030
PARAMETERS (subcommand)
GETCOGNOS command 813
POWER MEANS INDEPENDENT command 1452
POWER MEANS ONESAMPLE command 1456
POWER MEANS RELATED command 1460
POWER ONEWAY ANOVA command 1448, 1449
POWER PARTIALCORR command 1464
POWER PEARSON ONESAMPLE command 1468
POWER PROPORTIONS INDEPENDENT command 1472
POWER PROPORTIONS ONESAMPLE command 1476
POWER PROPORTIONS RELATED command 1480
POWER SPEARMAN ONESAMPLE command 1484
POWER UNIVARIATE LINEAR command 1488
PAREPS (subcommand)
ARIMA command 184
PARETO (subcommand)
GRAPH command 883
Pareto charts
simple 883
stacked 883
Pareto distribution function 64
part correlation
REGRESSION command 1640
partial associations
HILOGLINEAR command 894
PARTIAL CORR (command)
control variables 1418
correlation list 1418
FORMAT subcommand 1419
limitations 1417
matrix input 1420
matrix output 1420
MATRIX subcommand 1420
MISSING subcommand 1420
missing values 1420, 1421
order values 1418
SIGNIFICANCE subcommand 1419
STATISTICS subcommand 1419
syntax chart 1417
VARIABLES subcommand 1418
Partial Correlations
REGRESSION command 1640
partial eta-squared
UNIANOVA command 2076
Partial Least Squares Regression
command syntax 1431
PARTIALPLOT (subcommand)
REGRESSION command 1648
PARTITION (keyword)
KNN command 975
PARTITION (subcommand
VARIABLE ROLE command 2129
PARTITION (subcommand)
KNN command 971
MANOVA command 1028

PARTITION (subcommand) *(continued)*
MLP command 1155
RBF command 1608
PARZEN (keyword)
SPECTRA command 1893
Parzen window
SPECTRA command 1893
PASSIVE (keyword)
CATPCA command 276
MULTIPLE CORRESPONDENCE command 1193
passive missing value treatment
MULTIPLE CORRESPONDENCE command 1193
PASSPROTECT (subcommand)
OUTPUT SAVE command 1401
SAVE CODEPAGE command 1737
SAVE command 1734
SORT CASES command 1842
START JOIN command 1902
TCM APPLY command 1946
TCM MODEL command 1965
PASSWORD (keyword)
OUTPUT SAVE command 1401
SAVE CODEPAGE command 1737
SAVE command 1734
SORT CASES command 1842
password encryption
databases 784, 787, 1759
paths
in file specifications 305, 680
PATTERN (keyword)
CLUSTER command 314
PROXIMITIES command 1559
PATTERN (subcommand)
COXREG command 374
CSCOXREG command 408
pattern difference measure
CLUSTER command 314
PROXIMITIES command 1559
pattern matrix
DISCRIMINANT command 593
PATTERNS (keyword)
MULTIPLE IMPUTATION command 1207
PC (keyword)
FACTOR command 672
PCOMPS (subcommand)
MANOVA command 1049
PCOMPUTE (subcommand)
CTABLES command 506
PCON (keyword)
NLR command 1244
PCONVERGE (keyword)
CSCOXREG command 403
CSLOGISTIC command 435
CSORDINAL command 446
GENLIN command 732, 737
MIXED command 1138
NOMREG command 1251
PLUM command 1438
PCPROB (keyword)
NOMREG command 1256
PLUM command 1440
PCT (function)
GRAPH command 877
REPORT command 1686
PCT (function) *(continued)*
XGRAPH command 2168
PCT (keyword)
IGRAPH command 924, 926, 928
PCT format 50
PCTANOMALOUSCASES (keyword)
DETECTANOMALY command 580
PCTEQUAL (keyword)
SELECTPRED command 1782
VALIDATEDATA command 2103
PCTMISSING (keyword)
ADP command 117
SELECTPRED command 1782
VALIDATEDATA command 2103
PCTONECASE (keyword)
SELECTPRED command 1782
PCTUNEQUAL (keyword)
VALIDATEDATA command 2103
PDF (subcommand)
OUTPUT EXPORT command 1363
PDF format
exporting output 1363
PDF functions 64
PDF.BERNOULLI (function) 66, 1081
PDF.BETA (function) 66, 1081
PDF.BINOM (function) 66, 1081
PDF.BVNOR (function) 66, 1081
PDF.CAUCHY (function) 66, 1081
PDF.CHISQ (function) 66, 1081
PDF.EXP (function) 66, 1081
PDF.F (function) 66, 1081
PDF.GAMMA (function) 66, 1081
PDF.GEOM (function) 66, 1081
PDF.HALFNRM (function) 66, 1081
PDF.HYPER (function) 66, 1081
PDF.IGAUSS (function) 66, 1081
PDF.LAPLACE (function) 66, 1081
PDF.LNORMAL (function) 66, 1081
PDF.LOGISTIC (function) 66, 1081
PDF.NEGBIN (function) 66, 1081
PDF.NORMAL (function) 66, 1081
PDF.PARETO (function) 66, 1081
PDF.POISSON (function) 66, 1081
PDF.T (function) 66, 1081
PDF.UNIFORM (function) 66, 1081
PDF.WEIBULL (function) 66, 1081
PEARSON (keyword)
GENLIN command 732
NOMREG command 1257
Pearson chi-square
CROSSTABS command 390
Pearson correlation
CLUSTER command 313
Correlations command 349
CROSSTABS command 390
FACTOR command 669
PROXIMITIES command 1558
RELIABILITY command 1655, 1656
PEARSON_RESIDUALS (keyword)
GENLINMIXED command 766
PEARSONCHISQ (keyword)
SELECTPRED command 1782, 1783
PEARSONRESID (keyword)
GENLIN command 746

PEERID (keyword)
DETECTANOMALY command 581
PEERPCTSIZE (keyword)
DETECTANOMALY command 581
PEERSIZE (keyword)
DETECTANOMALY command 581
PENALTY (subcommand)
PREFSCAL command 1513
PEQ (function)
GGRAPH command 823
PEQ (keyword)
IGRAPH command 931
PER (keyword)
SPECTRA command 1895
PERCENT (function)
REPORT command 1685
PERCENT (keyword)
CODEBOOK (command) 327
FREQUENCIES command 717
MVA command 1215, 1216
NAIVEBAYES command 1229
PERCENT (subcommand)
RANK command 1594
percentage change
between groups and variables 1297
percentage functions
CTABLES command 486
percentages
CROSSTABS command 389
percentiles
break points 642
CTABLES command 486
estimating from grouped data 718
FREQUENCIES command 719
KM command 963
methods 642
PERCENTILES (keyword)
SIMRUN command 1838
PERCENTILES (subcommand)
EXAMINE command 642
FREQUENCIES command 719
KM command 963
PERIOD (subcommand)
ACF command 99
CCF command 301
EXSMOOTH command 658
PACF command 1413
PPLOT command 1497
SEASON command 1773
TSET command 2026
TSPLOT command 2053
periodic lags
PACF command 1413
periodicity
ACF command 99
ARIMA command 181
CCF command 301
EXSMOOTH command 658
in sequence charts 257, 2053
PACF command 1413
SEASON command 1773
time series settings 2026
TSAPPLY command 2021
TSMODEL command 2040
periodogram
SPECTRA command 1893
periodogram values
saving with SPECTRA command 1895
PERMANENT (subcommand)
USE command 2098
PERMISSIONS (command)
syntax chart 1423
PERMISSIONS (subcommand)
SAVE command 1733
PERMUTATION (keyword)
ANACOR command 154
CORRESPONDENCE command 361
PERVIOUSWEIGHT (keyword)
CSPLAN command 457
PGE (function)
GGRAPH command 823
PGE (keyword)
IGRAPH command 931
PGROUP (keyword)
LOGISTIC REGRESSION command 997
PGT (function)
AGGREGATE command 125
GGRAPH command 823
GRAPH command 877
REPORT command 1685
XGRAPH command 2168
PGT (keyword)
IGRAPH command 931
PH (keyword)
SPECTRA command 1893, 1895
PH2 (keyword)
CLUSTER command 314
PROXIMITIES command 1558
phase spectrum estimates
saving with SPECTRA command 1895
phase spectrum plot
SPECTRA command 1893
PHI (keyword)
CLUSTER command 314
CROSSTABS command 390
PROXIMITIES command 1559
PHI (subcommand)
EXSMOOTH command 658
phi four-point correlation
CLUSTER command 314
PROXIMITIES command 1559
phi-square distance measure
CLUSTER command 314
PROXIMITIES command 1558
PIE (keyword)
AIM command 133
PIE (subcommand)
GRAPH command 881
IGRAPH command 926
pie charts 881
Pillai's trace
in MANOVA 1050
PIN (function)
AGGREGATE command 125
GGRAPH command 823
GRAPH command 877
REPORT command 1685
XGRAPH command 2168

PIN (keyword)
COXREG command 373
IGRAPH command 931
LOGISTIC REGRESSION command 996
NOMREG command 1254
REGRESSION command 1641
pivot tables
controlling number of rows to display 1792
exporting table layers 1360
fast tables 1804
PLAIN (keyword)
REPORT command 1688
PLAN (keyword)
CSPLAN command 458
PLAN (subcommand)
CONJOINT command 343
CSCOXREG command 401
CSDESCRIPTIVES command 412
CSGLM command 419
CSLOGISTIC command 429
CSORDINAL command 441
CSPLAN command 457
CSSELECT command 469
CSTABULATE command 476
SIMPLAN command 1830
SIMRUN command 1835
with DATA subcommand 344
plan file
CSGLM command 419
CSLOGISTIC command 429
PLANCARDS (command)
FACTORS subcommand 1426
FOOTER subcommand 1428
FORMAT subcommand 1427
OUTFILE subcommand 1427
release history 1425
saving profiles in data files 1427
sequential profile numbers 1428
syntax chart 1425
TITLE subcommand 1428
with ORTHOPLAN command 1425
with VALUE LABELS command 1425
with VARIABLE LABELS command 1425
PLANVARS (subcommand)
CSPLAN command 457
PLE (function)
GGRAPH command 823
PLE (keyword)
IGRAPH command 931
PLOT (keyword)
REGRESSION command 1647
PLOT (subcommand)
AIM command 133
ALSCAL command 142
ANACOR command 154
BAYES ANOVA command 195
BAYES REGRESSION command 225, 1584
BAYES REPEATED command 234
CATPCA command 282
CATREG command 297
CLUSTER command 319
CORRESPONDENCE command 361
COXREG command 374
CSCOXREG command 408
PLOT (subcommand) *(continued)*
CURVEFIT command 512
DISCRIMINANT command 594
EXAMINE command 643
GENLOG command 774
GLM command 852
HILOGLINEAR command 894
HOMALS command 900
LOGLINEAR command 1008
MANOVA command 1049
MLP command 1162
MULTIPLE CORRESPONDENCE command 1197
OVERALS command 1407
POWER MEANS INDEPENDENT command 1453
POWER MEANS ONESAMPLE command 1456
POWER MEANS RELATED command 1461
POWER ONEWAY ANOVA command 1449
POWER PARTIALCORR command 1465
POWER PEARSON ONESAMPLE command 1468
POWER PROPORTIONS INDEPENDENT command 1473
POWER PROPORTIONS ONESAMPLE command 1477
POWER PROPORTIONS RELATED command 1481
POWER SPEARMAN ONESAMPLE command 1485
POWER UNIVARIATE LINEAR command 1489
PPLOT command 1495
PREFSCAL command 1515
PRINCALS command 1525
PROXSCAL command 1577
RBF command 1611
ROC ANALYSIS command 1720
ROC command 1725
SELECTPRED command 1784
SPECTRA command 1893, 1894
SURVIVAL command 962
TREE command 1999
UNIANOVA command 2078
with NORMALIZATION subcommand 154
PLOT MEANS (subcommand)
ONEWAY command 1338
plots
CORRESPONDENCE command 361
TSAPPLY command 2019
TSMODEL command 2038
PLOTS (subcommand)
SURVIVAL command 1921
PLS (command)
CRITERIA subcommand 1435
ID subcommand 1434
MODEL subcommand 1434
OUTDATASET subcommand 1434
syntax chart 1431
variable lists 1433
PLT (function)
AGGREGATE command 125
GGRAPH command 823
GRAPH command 877
REPORT command 1685
XGRAPH command 2168
PLT (keyword)
IGRAPH command 931
PLUGINS (subcommand)
SHOW command 1810
PLUM (command)
CRITERIA subcommand 1438

PLUM (command) *(continued)*
LINK subcommand 1439
LOCATION subcommand 1439
MISSING subcommand 1440
PRINT subcommand 1440
SAVE subcommand 1440
SCALE subcommand 1441
syntax chart 1437
TEST subcommand 1442
PMA (function)
CREATE command 382
PMEANS (subcommand)
MANOVA command 1032
PMML
exporting transformations to PMML 1977
merging transformation PMML with model XML 1989
PNG (subcommand)
OUTPUT EXPORT command 1370
POINT (command)
ENCODING subcommand 1445
FILE subcommand 1444
KEY subcommand 1445
syntax chart 1443
with DATA LIST command 1443
with FILE HANDLE command 1444
POINTLABEL (subcommand)
IGRAPH command 921
POISSON (keyword)
GENLIN command 728
GENLINMIXED command 757
NPTESTS command 1268
Poisson distribution
GENLOG command 773
Poisson distribution function 64
POLYNOMIAL (keyword)
COXREG command 370
CSGLM command 423
GENLIN command 741
GLM command 855, 872
LOGISTIC REGRESSION command 992
MANOVA command 1045, 1056
UNIANOVA command 2081
POLYNOMIAL (subcommand)
ONEWAY command 1335
polynomial contrasts
COXREG command 370
CSGLM command 423
in MANOVA command 1045
LOGLINEAR command 1005
repeated measures 872
UNIANOVA command 2081
POOL (keyword)
MANOVA command 1040
POOLED (keyword)
DISCRIMINANT command 593
KM command 964
REGRESSION command 1646
POPSIZE (keyword)
CSDESCRIPTIVES command 414
CSPLAN command 463
CSTABULATE command 477
POPSIZE (subcommand)
CSPLAN command 464
population pyramids 2177
portable files
reading 933
POSITION (keyword)
CODEBOOK command 325
position of totals
CTABLES command 498
post hoc tests
alpha value 856
Bonferroni test 856, 1336
Duncan's multiple comparison procedure 856, 1336
Dunnett's C 856, 1336
Dunnett's t test 856
Gabriel's pairwise comparisons test 856, 1336
Games and Howell's pairwise comparisons test 856, 1336
GLM 856
Hochberg's GT2 856, 1336
in GLM 856
least significant difference 856, 1336
ONEWAY command 1336
Ryan-Einot-Gabriel-Welsch multiple stepdown procedure 856, 1336
Ryan-Einot-Gabriel-Welsch's multiple stepdown procedure 1336
Scheffé test 856, 1336
Sidak's t test 856, 1336
statistical purpose 856
Student-Newman-Keuls 856, 1336
Tamhane's T2 856, 1336
Tamhane's T3 856, 1336
Tukey's b test 856, 1336
Tukey's honestly significant difference 856, 1336
UNIANOVA command 2082
Waller-Duncan test 856, 1336
posterior probability
DISCRIMINANT command 590
POSTHOC (subcommand)
GLM command 856
UNIANOVA command 2082
POUT (function)
AGGREGATE command 125
POUT (keyword)
COXREG command 373
LOGISTIC REGRESSION command 996
NOMREG command 1254
REGRESSION command 1641
power
observed 850
UNIANOVA command 2076
POWER (keyword)
CLUSTER command 313
CURVEFIT command 511
GENLIN command 728
GENLINMIXED command 757
PROXIMITIES command 1558
POWER (subcommand)
MANOVA command 1033, 1050
WLS command 2155
power estimates
in MANOVA 1050
POWER MEANS INDEPENDENT
command syntax 1451
POWER MEANS INDEPENDENT (command)
PARAMETERS subcommand 1452

POWER MEANS INDEPENDENT (command) *(continued)*
PLOT subcommand 1453
syntax chart 1451
POWER MEANS INDEPENDENT(command)
limitations 1451
POWER MEANS ONESAMPLE
command syntax 1455
POWER MEANS ONESAMPLE (command)
PARAMETERS subcommand 1456
PLOT subcommand 1456
syntax chart 1455
POWER MEANS ONESAMPLE(command)
limitations 1455
POWER MEANS RELATED
command syntax 1459
POWER MEANS RELATED (command)
PARAMETERS subcommand 1460
PLOT subcommand 1461
syntax chart 1459
POWER MEANS RELATED(command)
limitations 1459
power model
CURVEFIT command 509
POWER ONEWAY ANOVA
command syntax 1447
POWER ONEWAY ANOVA (command)
PARAMETERS subcommand 1448, 1449
PLOT subcommand 1449
syntax chart 1447
POWER ONEWAY ANOVA(command)
limitations 1447
POWER PARTIALCORR
command syntax 1463
POWER PARTIALCORR (command)
PARAMETERS subcommand 1464
PLOT subcommand 1465
syntax chart 1463
POWER PARTIALCORR(command)
limitations 1463
POWER PEARSON ONESAMPLE
command syntax 1467
POWER PEARSON ONESAMPLE (command)
PARAMETERS subcommand 1468
PLOT subcommand 1468
syntax chart 1467
POWER PEARSON ONESAMPLE(command)
limitations 1467
POWER PROPORTIONS INDEPENDENT
command syntax 1471
POWER PROPORTIONS INDEPENDENT (command)
PARAMETERS subcommand 1472
PLOT subcommand 1473
syntax chart 1471
POWER PROPORTIONS INDEPENDENT(command)
limitations 1471
POWER PROPORTIONS ONESAMPLE
command syntax 1475
POWER PROPORTIONS ONESAMPLE (command)
PARAMETERS subcommand 1476
PLOT subcommand 1477
syntax chart 1475
POWER PROPORTIONS ONESAMPLE(command)
limitations 1475
POWER PROPORTIONS RELATED
POWER PROPORTIONS RELATED *(continued)*
command syntax 1479
POWER PROPORTIONS RELATED (command)
PARAMETERS subcommand 1480
PLOT subcommand 1481
syntax chart 1479
POWER PROPORTIONS RELATED(command)
limitations 1479
power range
WLS command 2155
POWER SPEARMAN ONESAMPLE
command syntax 1483
POWER SPEARMAN ONESAMPLE (command)
PARAMETERS subcommand 1484
PLOT subcommand 1485
syntax chart 1483
POWER SPEARMAN ONESAMPLE(command)
limitations 1483
POWER UNIVARIATE LINEAR
command syntax 1487
POWER UNIVARIATE LINEAR (command)
PARAMETERS subcommand 1488
PLOT subcommand 1489
syntax chart 1487
POWER UNIVARIATE LINEAR(command)
limitations 1487
PowerPoint format
exporting output 1364
PP (keyword)
SPCHART command 1884
PPK (keyword)
SPCHART command 1884
PPL (keyword)
SPCHART command 1884
PPLOT (command)
APPLY subcommand 1497
DIFF subcommand 1496
DISTRIBUTION subcommand 1493
FRACTION subcommand 1494
LN/NOLOG subcommands 1497
PERIOD subcommand 1497
PLOT subcommand 1495
SDIFF subcommand 1496
STANDARDIZE/NOSTANDARDIZE subcommands 1496
syntax chart 1491
TEMPLATE subcommand 1498
TYPE subcommand 1494
VARIABLES subcommand 1493
PPM (keyword)
SPCHART command 1884
PPROPERTIES (subcommand)
CTABLES command 507
PPS_BREWER (keyword)
CSPLAN command 459
PPS_CHROMY (keyword)
CSPLAN command 459
PPS_MURTHY (keyword)
CSPLAN command 459
PPS_SAMPFORD (keyword)
CSPLAN command 459
PPS_SYSTEMATIC (keyword)
CSPLAN command 459
PPS_WOR (keyword)
CSPLAN command 459

PPS_WR (keyword)
CSPLAN command 459
PPT (subcommand)
OUTPUT EXPORT command 1364
PPU (keyword)
SPCHART command 1884
PR (keyword)
SPCHART command 1884
Prais-Winsten method
AREG command 175
PRD (keyword)
RATIO STATISTICS command 1601, 1602
PRED (keyword)
CATREG command 298
CSGLM command 426
CURVEFIT command 512
GLM command 860
LOGISTIC REGRESSION command 997
MIXED command 1146
NLR/CNLR command 1241
REGRESSION command 1634
UNIANOVA command 2086
PRED (subcommand)
CNLR/NLR command 1241
PREDCAT (keyword)
NOMREG command 1256
PLUM command 1440
PREDICT (command)
syntax chart 1501
PREDICT (subcommand)
BAYES REGRESSION command 224, 1584
predictability measures
CLUSTER command 314
PROXIMITIES command 1559
PREDICTED (keyword)
BOOTSTRAP command 249
KNN command 973
MLP command 1162
RBF command 1611
TMS IMPORT command 1986
predicted group
LOGISTIC REGRESSION command 997
predicted probabilities
LOGISTIC REGRESSION command 997
predicted probability
CSLOGISTIC command 437
predicted values
CSGLM command 426
CSLOGISTIC command 437
saving as variable in Tree command 2001
saving CURVEFIT command 512
saving in 2SLS command 95
PREDICTED_PROBABILITY (keyword)
GENLINMIXED command 766
PREDICTED_VALUES (keyword)
GENLINMIXED command 766
LINEAR command 983
prediction intervals
IGRAPH command 930
saving CURVEFIT command 512
Predictor Selection
command syntax 1779
PREDICTORS (keyword)
PLS command 1434
PREDPROB (keyword)
CSLOGISTIC command 437
CSORDINAL command 449
KNN command 975
NAIVEBAYES command 1231
PREDVAL (keyword)
CSLOGISTIC command 437
CSORDINAL command 449
KNN command 975
MLP command 1163
NAIVEBAYES command 1231
RBF command 1612
PREDVALPROB (keyword)
CSORDINAL command 449
preferences
blank data fields 1790
borders for tables 1790
charts 1790
custom currency formats 1791
default file extension 1794
default variable format 1794, 1795
display errors 1794
display macro commands 1797
display resource messages 1794
display statistical results 1794
display warnings 1794
displaying 1809
errors 1799
invalid data 1805
macro expansion 1797
maximum loops 1799
output 1794, 1795
preserving 1519, 1711
random number seed 1798
restoring 1519, 1711
PREFSCAL (command)
CONDITION subcommand 1510
CRITERIA subcommand 1513
INITIAL subcommand 1509
INPUT subcommand 1507
MODEL subcommand 1512
OPTIONS subcommand 1516
OUTFILE subcommand 1517
PENALTY subcommand 1513
PLOT subcommand 1515
PRINT subcommand 1514
PROXIMITIES subcommand 1509
release history 1505
RESTRICTIONS subcommand 1512
syntax chart 1505
TRANSFORMATION subcommand 1511
VARIABLES subcommand 1507
WEIGHTS subcommand 1509
PREPAREDDATA (keyword)
TMS IMPORT command 1986
PREPDATETIME (subcommand)
ADP command 115
PREPROCESS (keyword)
OPTIMAL BINNING command 1345
PREPXML (keyword)
ADP command 120
PRESERVE (command)
macro facility 566
syntax chart 1519

PRESERVE (command) *(continued)*
with RESTORE command 1711
with SET command 1788
PRESID (keyword)
COXREG command 375
PRESORTED (keyword)
CSSELECT command 470
PRESORTED (subcommand)
AGGREGATE command 125
PREVIEW (keyword)
REPORT command 1677
PREVIOUS (keyword)
REPORT command 1690
price-related differential (PRD)
RATIO STATISTICS command 1601, 1602
primary files 2091
PRINCALS (command)
ANALYSIS subcommand 1523
compared with OVERALS 1405
DIMENSION subcommand 1524
MATRIX subcommand 1527
MAXITER subcommand 1524
NOBSERVATIONS subcommand 1524
PLOT subcommand 1525
PRINT subcommand 1524
SAVE subcommand 1526
syntax chart 1521
value labels 1525
variable labels 1525
VARIABLES subcommand 1522
with AUTORECODE command 1521
with RECODE command 1521
PRINCIPAL (keyword)
ANACOR command 153
CORRESPONDENCE command 360
principal axis factoring
FACTOR command 672
principal components
FACTOR command 672
principal components analysis
in MANOVA 1049
principal directions
ALSCAL command 141
PRINT (command)
ENCODING subcommand 1533
formats 1529, 1530
missing values 1529
NOTABLE subcommand 1533
OUTFILE subcommand 1532
RECORDS subcommand 1532
strings 1529, 1531
syntax chart 1529
TABLE subcommand 1533
variable list 1529
with DO IF command 1531
with PRINT EJECT command 1535
with SET command 1529
with SORT CASES command 1843
PRINT (statement)
MATRIX command 1088
PRINT (subcommand)
2SLS command 95
ALSCAL command 142
ANACOR command 154
PRINT (subcommand) *(continued)*
AUTORECODE command 192
BAYES LOGLINEAR command 208
CATPCA command 281
CATREG command 296
CLUSTER command 318
CONJOINT command 347
CORRELATIONS command 351
CORRESPONDENCE command 361
COXREG command 373
CSCOXREG command 406
CSGLM command 425
CSLOGISTIC command 437
CSORDINAL command 449
CSPLAN command 458
CSSELECT command 473
CURVEFIT command 513
DETECTANOMALY command 582
DO REPEAT command 621
GENLIN command 744
GENLOG command 773
GLM command 850, 866
HILOGLINEAR command 894
HOMALS command 900
KM command 963
KNN command 975
LOGISTIC REGRESSION command 996
LOGLINEAR command 1007
MANOVA command 1029, 1047
MIXED command 1142
MLP command 1161
MULTIPLE CORRESPONDENCE command 1195
NAIVEBAYES command 1231
NOMREG command 1256
NONPAR CORR command 1261
OPTIMAL BINNING command 1346
OVERALS command 1407
PLUM command 1440
PREFSCAL command 1514
PRINCALS command 1524
PROBIT command 1545
PROXIMITIES command 1562
PROXSCAL command 1576
QUICK CLUSTER command 1590
RATIO STATISTICS command 1602
RBF command 1610
ROC ANALYSIS command 1720
ROC command 1725
SELECTPRED command 1783
SIMRUN command 1838
SURVIVAL command 1922
TMS END command 1984
TMS MERGE command 1990
TREE command 1997
TSET command 2027
TWOSTEP CLUSTER command 2069
UNIANOVA command 2076
VARCOMP command 2116
WEIGHTED KAPPA command 2150
WLS command 2156
PRINT EJECT (command)
$CASENUM system variable 1536
missing values 1535
syntax chart 1535

PRINT EJECT (command) *(continued)*
with DO IF command 1535
with PRINT command 1535
with SET command 1535
PRINT FORMATS (command)
format specification 1537
string variables 1537
syntax chart 1537
with DISPLAY command 1537
PRINT SPACE (command)
DO IF command 1539
number of lines 1539
OUTFILE subcommand 1539
syntax chart 1539
PRINTBACK (subcommand)
SET command 1794
SHOW command 1810
PRINTERPOLATE (keyword)
ROC ANALYSIS command 1720
printing cases
column headings 1535
displaying blank lines 1539
formats 1529, 1530, 2160
missing values 1529
number of records 1532
output file 1529, 1532, 1539
page ejection 1535
strings 1529, 1531
summary table 1529, 1533
PRIOR (subcommand)
BAYES ANOVA command 195
BAYES INDEPENDENT command 203
BAYES ONESAMPLE: Binomial command 212
BAYES ONESAMPLE: Normal command 215
BAYES ONESAMPLE: Poisson command 218
BAYES REGRESSION command 224, 1584
BAYES RELATED command 230
prior moving average function 381, 382
prior probability
DISCRIMINANT command 590
TREE command 2007
PRIORS (subcommand)
DISCRIMINANT command 590
TREE command 2007
PROB (keyword)
MVA command 1215
probability density functions 64, 66, 1081
probability of F-to-enter
REGRESSION command 1641
probability of F-to-remove
REGRESSION command 1641
PROBABILITY_ENTRY (keyword)
LINEAR command 980
PROBABILITY_REMOVAL (keyword)
LINEAR command 980
PROBIT (command)
confidence intervals 1545
covariates 1542
CRITERIA subcommand 1544
expected frequencies 1545
factors 1542
grouping variable 1542
limitations 1541
LOG subcommand 1544
PROBIT (command) *(continued)*
log transformation 1544
maximum iterations 1544
MISSING subcommand 1546
missing values 1546
model specification 1543
MODEL subcommand 1543
NATRES subcommand 1544
natural response rate 1544
observation frequency variable 1542
observed frequencies 1545
predictor variables 1542
PRINT subcommand 1545
residuals 1545
response frequency variable 1542
response rate 1544
step limit 1544
syntax chart 1541
variable specification 1542
PROBIT (function)
MATRIX command 1085
PROBIT (keyword)
CSORDINAL command 442
GENLIN command 728
GENLINMIXED command 757
PLUM command 1439
PROBIT command 1543
Probit Analysis
command syntax 1541
probit link
PLUM command 1439
PROBS (keyword)
DISCRIMINANT command 590
procedure output
output file 1547
writing to a file 1547
PROCEDURE OUTPUT (command)
OUTFILE subcommand 1547
syntax chart 1547
with CROSSTABS 393
with CROSSTABS command 1547
with SURVIVAL command 1547
process capability chart
SPCHART command 1880
process capability indices
SPCHART (command) 1883
PROCRU (keyword)
CATPCA command 280
production mode syntax rules 34
PRODUCTLIMIT (keyword)
CSCOXREG command 406
PROFILE (keyword)
GENLIN command 732
GLM command 852
UNIANOVA command 2078
profile plots
UNIANOVA command 2078
profiles
saving in data files 1427
PROFILES (keyword)
ANACOR command 154
profit chart
TREE command 1999
PROFITS (subcommand)

PROFITS (subcommand) *(continued)*
TREE command 2009
program states 2187
PROJCENTR(keyword)
CATPCA command 282
projected centroids plots
CATPCA command 282
PROMAX (keyword)
FACTOR command 673
promax rotation
FACTOR command 673
PROP (keyword)
CROSSTABS command 389
PROPHAZARD (keyword)
CSCOXREG command 405
PROPORTION (keyword)
MVA command 1219
PROPORTION (subcommand)
RANK command 1594
proportional sample 1727
PROPORTIONS
command syntax 1549
PROPORTIONS (command)
CRITERIA subcommand 1554
INDEPENDENTSAMPLES subcommand 1552
limitations 1550
MISSING subcommand 1553
ONESAMPLE subcommand 1551
PAIREDSAMPLES subcommand 1552
SUCCESS subcommand 1553
syntax chart 1549
TESTVAL 1551
PROX (keyword)
MATRIX DATA command 1117
PROXIMITIES (command)
computing distances between cases 1557
computing distances between variables 1557
displaying distance matrix 1562
ID subcommand 1562
labeling cases 1562
limitations 1555
matrix input 1562, 1563
matrix output 1562, 1563
MATRIX subcommand 1562
MEASURE subcommand 1557
measures for binary data 1559
measures for frequency-count data 1558
measures for interval data 1558
MISSING subcommand 1562
missing values 1562
PRINT subcommand 1562
standardization 1556
STANDARDIZE subcommand 1556
syntax chart 1555
transforming measures 1557
transforming values 1556
variable list 1556
VIEW subcommand 1557
with FACTOR command 1565
PROXIMITIES (keyword)
PROXIMITIES command 1562
PROXIMITIES (subcommand)
PREFSCAL command 1509
PROXSCAL command 1573
PROXSCAL (command)
ACCELERATION subcommand 1575
CONDITION subcommand 1572
CRITERIA subcommand 1576
INITIAL subcommand 1571
limitations 1567
MATRIX subcommand 1579
options 1567
OUTFILE subcommand 1578
PLOT subcommand 1577
PRINT subcommand 1576
PROXIMITIES subcommand 1573
RESTRICTIONS subcommand 1574
SHAPE subcommand 1571
syntax chart 1567
TABLE subcommand 1569
TRANSFORMATION subcommand 1573
WEIGHTS subcommand 1572
PSEUDOBIC (keyword)
NAIVEBAYES command 1230
PSEUDOPROB (keyword)
MLP command 1163
RBF command 1612
PTILE (function)
GGRAPH command 823
GRAPH command 877
XGRAPH command 2168
PTILE (keyword)
IGRAPH command 931
PVALUE (keyword)
ADP command 119
AIM command 133
PW (keyword)
AREG command 175
PYRAMID (keyword)
IGRAPH command 925
PZL (keyword)
SPCHART command 1884
PZLOUT (keyword)
SPCHART command 1884
PZMAX (keyword)
SPCHART command 1884
PZMIN (keyword)
SPCHART command 1884
PZOUT (keyword)
SPCHART command 1884
PZU (keyword)
SPCHART command 1884
PZUOUT (keyword)
SPCHART command 1884

Q

Q (keyword)
CLUSTER command 314
PROXIMITIES command 1559
Q (subcommand)
ARIMA command 182
Q-Q (keyword)
PPLOT command 1494
QREGW (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082

QS (keyword)
SPECTRA command 1893, 1895
QUADRATIC (keyword)
CURVEFIT command 511
quadratic model
CURVEFIT command 509
quadratic spectrum estimate plot
SPECTRA command 1893
quadrature spectrum estimates
saving with SPECTRA command 1895
QUALIFIER (subcommand)
GET DATA command 792
QUANT (keyword)
CATPCA command 281
CATREG command 296
HOMALS command 900
MULTIPLE CORRESPONDENCE command 1195
OVERALS command 1407
PRINCALS command 1524, 1525
quantifications
MULTIPLE CORRESPONDENCE command 1195
QUANTILE REGRESSION
command syntax 1581
QUANTILE REGRESSION (command)
limitations 1581
syntax chart 1581
quarter of year 54
quartiles
KM command 963
QUARTILES (keyword)
CODEBOOK (command) 327
NPAR TESTS command 1291
QUARTIMAX (keyword)
CATPCA command 279
FACTOR command 673
MANOVA command 1049
quartimax rotation
FACTOR command 673
QUEST (subcommand)
TREE command 2006
QUICK CLUSTER (command)
cluster distances 1590, 1591
cluster membership 1590, 1591
clustering method 1587, 1589
compared with CLUSTER command 1587
convergence criteria 1588
iterations 1588
labeling cases 1590
METHOD subcommand 1589
missing values 1591
PRINT subcommand 1590
specifying number of clusters 1588
statistics 1590
syntax chart 1587
variable list 1588
with large number of cases 1587
QYR format 54

R

R (keyword)
CATREG command 296
MIXED command 1142
REGRESSION command 1640
R 2
REGRESSION command 1640
R charts
SPCHART command 1872
R statistic
MEANS command 1127
SUMMARIZE command 1915
r-squared
CSGLM command 425
Radial Basis Function
command syntax 1603
RANDIMPU (keyword)
CATPCA command 277
RANDOM (keyword)
CATREG command 294
CSSELECT command 469
OVERALS command 1406
PREFSCAL command 1509
PROXSCAL command 1571, 1576
RANDOM (subcommand)
GENLINMIXED command 761
GLM command 848
MIXED command 1143
UNIANOVA command 2074
VARCOMP command 2114
random effects
VARCOMP command 2113
random number functions 64
random number seed
specifying 1798
random sample
in nonparametric tests 1292
random variable functions 70, 1086
random-effects model
MIXED command 1143
syntax 840
range
EXAMINE command 644
FREQUENCIES command 720
RATIO STATISTICS command 1601, 1602
RANGE (function) 85
RANGE (keyword)
DESCRIPTIVES command 575
FREQUENCIES command 720
GRAPH command 880
MEANS command 1126
PROXIMITIES command 1556
RATIO STATISTICS command 1601, 1602
SUMMARIZE command 1913
RANGE (subcommand)
GET TRANSLATE command 809
range bar charts 880
RANGES (subcommand)
ONEWAY command 1337
RANK (command)
FRACTION subcommand 1596
handling of ties 1494, 1596
MISSING subcommand 1597
missing values 1597
N subcommand 1594
NORMAL subcommand 1594
NTILES(k) subcommand 1594
PERCENT subcommand 1594
PROPORTION subcommand 1594

RANK (command) *(continued)*
RANK subcommand 1594
ranking order 1594
RFRACTION subcommand 1594
SAVAGE subcommand 1594
saving rank variables 1595
syntax chart 1593
VARIABLES subcommand 1594
RANK (function)
MATRIX command 1077
RANK (subcommand)
RANK command 1594
rank-order coefficients
NONPAR CORR command 1259
RANKING (keyword)
CATPCA command 276
CATREG command 292
MULTIPLE CORRESPONDENCE command 1192
SELECTPRED command 1782
ranking cases
method 1594
missing values 1597
new variable names 1595
order 1594
proportion estimates 1596
tied values 1596
within subgroups 1594
RANKIT (keyword)
PPLOT command 1494
RANK command 1596
RAO (keyword)
DISCRIMINANT command 587
Rao's V
DISCRIMINANT command 587
RATE (keyword)
CSPLAN command 463
RATE (subcommand)
CSPLAN command 461
RATIO (keyword)
ALSCAL command 138
PROXSCAL command 1573
RATIO (subcommand)
CSDESCRIPTIVES command 413
ratio data
ALSCAL command 138
Ratio Statistics
command syntax 1599
RATIO STATISTICS (command)
MISSING subcommand 1600
missing values 1600
OUTFILE subcommand 1601
output 1602
overview 1599
PRINT subcommand 1602
saving to external file 1601
syntax chart 1599
RAW (keyword)
DISCRIMINANT command 592
MANOVA command 1049
raw data files
variable definition 1625
raw matrix data files
factors 1115, 1118
format 1107, 1111
raw matrix data files *(continued)*
record types 1117
split files 1114
within-cells records 1116, 1118
RBAR (keyword)
SPCHART command 1886
RBF (command)
ARCHITECTURE subcommand 1609
CRITERIA subcommand 1609
EXCEPT subcommand 1606
MISSING subcommand 1610
OUTFILE subcommand 1613
PARTITION subcommand 1608
PLOT subcommand 1611
PRINT subcommand 1610
RESCALE subcommand 1607
SAVE subcommand 1612
syntax chart 1603
variable lists 1606
RC (keyword)
SPECTRA command 1895
RCMEAN (keyword)
CORRESPONDENCE command 360
RCON (keyword)
NLR command 1244
RCONF (keyword)
CORRESPONDENCE command 361
RCONVERGE (keyword)
FACTOR command 671
READ (statement)
MATRIX command 1093
READ MODEL (command)
DROP subcommand 1616
FILE subcommand 1616
KEEP subcommand 1616
syntax chart 1615
TSET subcommand 1617
TYPE subcommand 1616
READNAMES (subcommand)
GET DATA command 789
REASONMEASURE (keyword)
DETECTANOMALY command 581
REASONNORM (keyword)
DETECTANOMALY command 581
REASONSUMMARY (keyword)
DETECTANOMALY command 582
REASONVALUE (keyword)
DETECTANOMALY command 581
REASONVAR (keyword)
DETECTANOMALY command 581
RECODE (command)
compared with AUTORECODE command 1619
compared with IF command 1619
missing values 1620
numeric variables 1620
string variables 1620
syntax chart 1619
target variable 1621
with HOMALS command 897
with MISSING VALUES command 1620
with NONPAR CORR command 1262
with OVERALS command 1403
with PRINCALS command 1521
recoding values

recoding values *(continued)*
converting strings to numeric 1622
missing values 1620
numeric variables 1620
string variables 1620
target variable 1621
RECORD (subcommand)
FILE TYPE command 690
record length
specifying wide records with FILE HANDLE 680
RECORD TYPE (command)
CASE subcommand 1628
DUPLICATE subcommand 1629
MISSING subcommand 1629
SKIP subcommand 1628
SPREAD subcommand 1630
syntax chart 1625
with DATA LIST command 1625
with FILE TYPE command 1625
records
defining 521, 1625
duplicate 1629
missing 1629
skipping 1628
types 1625
RECORDS (subcommand)
DATA LIST command 521
PRINT command 1532
WRITE command 2161
RECTANGLE (keyword)
IGRAPH command 925
RECTANGULAR (keyword)
ALSCAL command 138
rectangular matrix
ALSCAL command 138
REDUCED (keyword)
PROXSCAL command 1574
REDUNDANCY (keyword)
MANOVA command 1031
reestimate model parameters
TSAPPLY command 2015, 2021
REFCAT (keyword)
MIXED command 1140
REFERENCE (keyword)
ADP command 115
GENLIN command 726
PLS command 1433
TSPLOT command 2054
REFERENCE (subcommand)
SPCHART command 1888
reference lines
in sequence charts 258, 2054, 2057
REFLEC (keyword)
CATPCA command 280
REG (keyword)
ANOVA command 164
FACTOR command 674
REG (subcommand)
ARIMA command 183
regression
syntax 840
REGRESSION (command)
case selection 1644
casewise plots 1647
REGRESSION (command) *(continued)*
CASEWISE subcommand 1647
constant term 1642
CRITERIA subcommand 1641
dependent variable 1638
DESCRIPTIVES subcommand 1643
histograms 1646
matrix data 1645
matrix input 1645
matrix output 1645
MATRIX subcommand 1645
METHOD subcommand 1638
MISSING subcommand 1646
missing values 1645
model criteria 1641
NOORIGIN subcommand 1642
normal probability plots 1646
ORIGIN subcommand 1642
partial residual plots 1648
PARTIALPLOT subcommand 1648
REGWGT subcommand 1642
release history 1633
RESIDUALS subcommand 1646
saving files 1649
saving new variables 1649
saving variables 1649
SCATTERPLOT subcommand 1648
scatterplots 1648
SELECT subcommand 1644
statistics 1639, 1643
STATISTICS subcommand 1639
syntax chart 1633
TEMPLATE subcommand 1650
tolerance 1640, 1641
variable selection 1638, 1641
variable selection methods 1638
VARIABLES subcommand 1637
weighted models 1642
weights 1642
with CORRELATIONS command 1645
with MATRIX DATA command 1109
with SAMPLE command 1644
with SELECT IF command 1644
with SET command 1646
with TEMPORARY command 1644
REGRESSION (keyword)
IGRAPH command 930
REGRESSION (subcommand)
MVA command 1220
regression coefficients
REGRESSION command 1640
regression estimates
MVA command 1220
regression factor scores
FACTOR command 674
regression lines
IGRAPH command 930
REGU (keyword)
CATREG command 296, 297
REGULARIZATION (subcommand)
CATREG command 295
REGWGT (subcommand)
GLM command 848
MIXED command 1144

REGWGT (subcommand) *(continued)*
REGRESSION command 1642
UNIANOVA command 2074
VARCOMP command 2115
RELATED (subcommand)
NPTESTS command 1273
relational operators
defined 84
in matrix language 1074
RELATIVE (keyword)
CSORDINAL command 446
MIXED command 1138
relative median potency
PROBIT command 1545
relative risk
CROSSTABS command 390
RELEASE (statement)
MATRIX command 1105
RELIABILITY (command)
computational method 1656
ICC subcommand 1656
limitations 1653
matrix input 1657
matrix output 1657
MATRIX subcommand 1657
METHOD subcommand 1656
MISSING subcommand 1657
missing values 1657, 1658
MODEL subcommand 1655
models 1655
scale definition 1654
SCALE subcommand 1654
STATISTICS subcommand 1655
SUMMARY subcommand 1656
syntax chart 1653
variable list 1654
VARIABLES subcommand 1654
Reliability Analysis
command syntax 1653
RELRISK (keyword)
CSTABULATE command 477
REML (keyword)
MIXED command 1142
VARCOMP command 2115
REMOVALMETHOD (keyword)
NOMREG command 1254
REMOVE (keyword)
REGRESSION command 1638
RENAME (command)
SAVE TRANSLATE command 1762
RENAME (subcommand)
ADD FILES command 107
CASESTOVARS command 268
EXPORT command 652
GET command 781
GETTM1 command 817
IMPORT command 935
MANOVA command 1047, 1058
MATCH FILES command 1065
SAVE command 1732
UPDATE command 2094
XSAVE command 2184
RENAME VARIABLES (command)
syntax chart 1661
RENAMEVARS (keyword)
CSSELECT command 470
REORDERNOMINAL (subcommand)
ADP command 118
REPDEFER (subcommand)
SET command 1801
SHOW command 1810
REPEATED (keyword)
COXREG command 370
CSGLM command 423
GENLIN command 741
GLM command 855, 872
LOGISTIC REGRESSION command 992
MANOVA command 1026, 1045
UNIANOVA command 2081
REPEATED (subcommand)
GENLIN command 737
MIXED command 1144
repeated contrasts
COXREG command 370
CSGLM command 423
in MANOVA command 1045
LOGLINEAR command 1005
repeated measures 872
UNIANOVA command 2081
repeated measures analysis
RELIABILITY command 1655
repeated measures analysis of variance
limitations 869
repeated measures models
syntax 840
REPEATED_MEASURES (keyword)
GENLINMIXED command 755
repeating data
case identification 1672
defining variables 1668
input file 1669
repeating groups 1668
starting column 1667
summary table 1672
REPEATING DATA (command)
CONTINUED subcommand 1670
DATA subcommand 1668
ENCODING subcommand 1669
FILE subcommand 1669
ID subcommand 1672
LENGTH subcommand 1669
NOTABLE subcommand 1672
OCCURS subcommand 1668
STARTS subcommand 1667
syntax chart 1663
with DATA LIST command 1663, 1665
with FILE TYPE command 1663, 1665
with INPUT PROGRAM command 1663, 1665
repeating data groups 1625
repeating fields. See repeating data 1663
REPLACE (function) 78
REPLACE (subcommand)
MCONVERT command 1122
ORTHOPLAN command 1349
SAVE TRANSLATE command 1762
with FACTORS subcommand 1349
REPLACEMISSING (subcommand)
ADP command 118

REPLACEWITH (keyword)
ADP command 118
replacing missing values
linear interpolation 1714
linear trend 1715
mean of nearby points 1714
median of nearby points 1714
series mean 1715
REPLICATE_RESULTS (keyword)
LINEAR command 980
REPORT (command)
BREAK subcommand 1682
CHWRAP keyword 1677
column headings 1676, 1679
column spacing 1676
column width 1676
defining subgroups 1682
footnotes 1690
FORMAT subcommand 1677
INDENT keyword 1677
limitations 1674
MISSING subcommand 1691
missing values 1676, 1691
ONEBREAKCOLUMN keyword 1677
OUTFILE subcommand 1678
output file 1676, 1678
PREVIEW keyword 1677
print formats 1688
report types 1674
STRING subcommand 1681
string variables 1681
summary statistics 1676, 1684
SUMMARY subcommand 1684
syntax chart 1673
titles 1690
VARIABLES subcommand 1679
with SET command 1676
with SORT CASES command 1843
REPORT (keyword)
CROSSTABS command 392
EXAMINE command 645
GRAPH command 886
XGRAPH command 2174
REPORT (subcommand)
OUTPUT MODIFY command 1390
REPORTEMPTY (keyword)
VALIDATEDATA command 2103
REPORTMISSING (keyword)
GGRAPH command 826
REPOSITORY ATTRIBUTES (command)
AUTHOR keyword 1694
DESCRIPTION keyword 1694
EXPIRATION keyword 1695
FILE keyword 1694
KEYWORDS keyword 1694
release history 1693
SECURITY subcommand 1696
syntax chart 1693
TOPICS keyword 1695
VERSIONLABEL keyword 1695
REPOSITORY CONNECT (command)
LOGIN subcommand 1698
release history 1697
SERVER subcommand 1698
REPOSITORY CONNECT (command) *(continued)*
syntax chart 1697
REPOSITORY COPY (command)
release history 1699
syntax chart 1699
REPR (keyword)
FACTOR command 669
REPRESULTS (keyword)
SIMRUN command 1835
reproduced correlation matrix
FACTOR command 669
reproducing tables 394
REREAD (command)
COLUMN subcommand 1705
FILE subcommand 1705
syntax chart 1703
with DATA LIST command 1703
with INPUT PROGRAM command 1703
REREAD (keyword)
MATRIX command 1094
rereading records
input file 1705
starting column 1705
RES (keyword)
CATREG command 298
RESAMPLE (subcommand)
CATPCA command 280
CATREG command 296
RESCALE (keyword)
PROXIMITIES command 1556, 1557
RESCALE (subcommand)
ADP command 118
KNN command 971
MLP command 1153
RBF command 1607
RESERVEDATTRIBUTES (keyword)
CODEBOOK command 326
RESHAPE (function)
MATRIX command 1077
RESID (keyword)
CATPCA command 282
CATREG command 297
CROSSTABS command 389
CSGLM command 426
CSTABULATE command 477
CURVEFIT command 512
GENLIN command 746
GLM command 860
HILOGLINEAR command 894
LOGISTIC REGRESSION command 997
MIXED command 1146
MULTIPLE CORRESPONDENCE command 1197
NLR/CNLR command 1241
REGRESSION command 1634
UNIANOVA command 2086
RESIDUAL (keyword)
BOOTSTRAP command 249
MANOVA command 1026
MLP command 1162
MVA command 1220
RBF command 1611
residual correlation matrix
GLM command 866
residual covariance matrix

residual covariance matrix *(continued)*
GLM command 866
residual plots
CATPCA command 282
GENLOG command 774
HILOGLINEAR command 894
in GLM 852
LOGLINEAR command 1008
MULTIPLE CORRESPONDENCE command 1197
PROXSCAL command 1577
UNIANOVA command 2078
residual SSCP
GLM command 866
residuals
CROSSTABS command 389
CSGLM command 426
degrees of freedom 702
GENLOG command 773
HILOGLINEAR command 894
LOGISTIC REGRESSION command 997
LOGLINEAR command 1007
PROBIT command 1545
saving CURVEFIT command 512
saving in 2SLS command 95
saving REGRESSION command 1649
RESIDUALS (keyword)
GLM command 852
PREFSCAL command 1515
PROXSCAL command 1577
UNIANOVA command 2078
RESIDUALS (subcommand)
MANOVA command 1033
REGRESSION command 1646
response chart
TREE command 1999
response frequency variable
PROBIT command 1542
RESPONSE RATE (command)
MAXCOUNT subcommand 1709
MINRATE subcommand 1709
syntax chart 1707
VARIABLES subcommand 1708
RESPONSES (keyword)
MULT RESPONSE command 1186
RESTORE (command)
macro facility 566
syntax chart 1711
with PRESERVE command 1711
with SET command 1711, 1788
restricted maximum likelihood estimation
VARCOMP command 2115
restricted numeric (N) format 48
RESTRICTIONS (subcommand)
PREFSCAL command 1512
PROXSCAL command 1574
RESULTS (subcommand)
SET command 1794
SHOW command 1810
REVERSE (keyword)
PROXIMITIES command 1557
reverse Helmert contrasts
UNIANOVA command 2081
RFRACTION (subcommand)
RANK command 1594
RHO (subcommand)
AREG command 175
rho value
AREG command 175
ribbon charts
IGRAPH command 928
RIDGE (keyword)
CATREG command 295
ridge regression macro 2215
RIGHT (keyword)
REPORT command 1680, 1683, 1690
RINDEX (function) 78
RISK (keyword)
CROSSTABS command 390
risk estimates
TREE command 1997
RISKDIFF (keyword)
CSTABULATE command 477
RISKINFO (keyword)
CSCOXREG command 406
RJUMP (keyword)
IGRAPH command 928
RLABELS (keyword)
MATRIX command 1089
RMAX (function)
MATRIX command 1077
RMEAN (keyword)
CORRESPONDENCE command 360
RMED (function)
CREATE command 382
RMIN (function)
MATRIX command 1077
RMP (keyword)
PROBIT command 1545
RMV (command)
LINT function 1714
MEAN function 1714
MEDIAN function 1714
SMEAN function 1715
syntax chart 1713
TREND function 1715
RNAMES (keyword)
MATRIX command 1089
RND (function)
MATRIX command 1077
RND (keyword)
MULTIPLE IMPUTATION command 1206
RNG (subcommand)
SET command 1798
SHOW command 1810
RNKORDER (function)
MATRIX command 1077
ROBUST (keyword)
GENLIN command 737
ROBUST (subcommand)
GLM command 860
UNIANOVA command 2086
ROC (command)
charts 1725
CRITERIA subcommand 1724
limitations 1723
MISSING keyword 1724
missing values 1724
output 1725

ROC (command) *(continued)*
PLOT subcommand 1725
PRINT subcommand 1725
syntax chart 1717, 1723
ROC (keyword)
MLP command 1162
RBF command 1611
ROC ANALYSIS (command)
CRITERIA subcommand 1719
DESIGN subcommand 1719
MISSING keyword 1719
missing values 1719
PLOT subcommand 1720
PRINT subcommand 1720
ROC ANALYSIS(command)
limitations 1717
ROC Curve
command syntax 1717, 1723
Rogers and Tanimoto measure
CLUSTER command 314
PROXIMITIES command 1559
ROI chart
TREE command 1999
ROLE (keyword)
MULTIPLE IMPUTATION command 1206
ROLES (keyword)
TMS IMPORT command 1986
root mean squared error
CSGLM command 425
ROTATE (keyword)
MANOVA command 1049
ROTATE (subcommand)
DISCRIMINANT command 593
ROTATION (keyword)
FACTOR command 669, 671
ROTATION (subcommand)
CATPCA command 279
FACTOR command 673
ROUND (keyword)
CROSSTABS command 393
EXAMINE command 642
IGRAPH command 925, 927
ROW (keyword)
ALSCAL command 139
CROSSTABS command 389
MULT RESPONSE command 1186
PREFSCAL command 1510, 1512
row number 45
row percentages
CROSSTABS (command) 389
ROWCONF (keyword)
ALSCAL command 139, 143
ROWOP (keyword)
GRAPH command 884
XGRAPH command 2175
ROWPCT (keyword)
CSTABULATE command 477
ROWS (keyword)
ALSCAL command 137
ANACOR command 154
PREFSCAL command 1507
ROWSBREAK (subcommand)
SET command 1792
ROWSIZE (keyword)
ROWSIZE (keyword) *(continued)*
SIMRUN command 1837
ROWTYPE_ variable
ANACOR command 155
CORRESPONDENCE command 362
HOMALS command 902
MATRIX DATA command 1107, 1112
OVERALS command 1409
PRINCALS command 1527
ROWVAR (keyword)
GRAPH command 883
XGRAPH command 2174
Roy-Bargmann stepdown F
in MANOVA 1048
Roy's largest root
in MANOVA 1050
RPAD (function) 78
RPOINTS (keyword)
CORRESPONDENCE command 361
RPRINCIPAL (keyword)
ANACOR command 153
CORRESPONDENCE command 360
RPROFILES (keyword)
CORRESPONDENCE command 361
RR (keyword)
CLUSTER command 314
PROXIMITIES command 1559
RSSCP (keyword)
GLM command 866
RSSCP matrices
GLM command 866
RSSQ (function)
MATRIX command 1077
RSTEP (keyword)
IGRAPH command 924, 928
RSUM (function)
MATRIX command 1077
RSUM (keyword)
CORRESPONDENCE command 360
RT (keyword)
CLUSTER command 314
PROXIMITIES command 1559
RTRIM (function) 78
RULE (keyword)
NOMREG command 1254
rule outcome variable
defining cross-variable rules 2107
defining single-variable rules 2105
RULEGENERATION (subcommand)
SPATIAL ASSOCIATION RULES command 1851
RULES (keyword)
OPTIMAL BINNING command 1346
RULES (subcommand)
SPCHART command 1885
TREE command 2000
RULESTABLE (subcommand)
SPATIAL ASSOCIATION RULES command 1853
RULESUMMARIES (subcommand)
VALIDATEDATA command 2104
RULEVIOLATIONS (keyword)
VALIDATEDATA command 2105
running median function 382
RUNS (keyword)
NPTESTS command 1268

RUNS (subcommand)
NPAR TESTS command 1289
Russell and Rao measure
CLUSTER command 314
PROXIMITIES command 1559
RV functions 64
RV.BERNOULLI (function) 70, 1086
RV.BETA (function) 70, 1086
RV.BINOM (function) 70, 1086
RV.CAUCHY (function) 70, 1086
RV.CHISQ (function) 70, 1086
RV.EXP (function) 70, 1086
RV.F (function) 70, 1086
RV.GAMMA (function) 70, 1086
RV.GEOM (function) 70, 1086
RV.HALFNRM (function) 70, 1086
RV.HYPER (function) 70, 1086
RV.IGAUSS (function) 70, 1086
RV.LAPLACE (function) 70, 1086
RV.LNORMAL (function) 70, 1086
RV.LOGISTIC (function) 70, 1086
RV.NEGBIN (function) 70, 1086
RV.NORMAL (function) 70, 1086
RV.PARETO (function) 70, 1086
RV.POISSON (function) 70, 1086
RV.T (function) 70, 1086
RV.UNIFORM (function) 70, 1086
Ryan-Einot-Gabriel-Welsch multiple stepdown procedure
UNIANOVA command 2082

S

S (keyword)
CURVEFIT command 511
SPECTRA command 1893, 1895
s charts
SPCHART command 1872
S-stress
ALSCAL command 141
sample
exact-size 1727
proportional 1727
SAMPLE (command)
limitations 1727
syntax chart 1727
with DO IF command 1728
with FILE TYPE command 1727
with INPUT PROGRAM command 1727
with N OF CASES command 1223, 1727
with REGRESSION command 1644
with SELECT IF command 1727
with SET command 1727
with TEMPORARY command 1727
SAMPLE (keyword)
CSPLAN command 457
NPTESTS command 1268, 1271
SAMPLE (subcommand)
NONPAR CORR command 1261
NPAR TESTS command 1292
SAMPLEFILE (subcommand)
CSSELECT command 470
SAMPLEINFO (keyword)
CSCOXREG command 406
CSGLM command 425
SAMPLEINFO (keyword) *(continued)*
CSLOGISTIC command 437
CSORDINAL command 449
SAMPLEMEAN (keyword)
NPTESTS command 1268
SAMPLEMEDIAN (keyword)
NPTESTS command 1268
SAMPLES (keyword)
CROSSTABS command 391
NPAR TESTS command 1292
SAMPLEWEIGHT (keyword)
CSPLAN command 457
SAMPLING (subcommand)
BOOTSTRAP command 249
sampling cases 1727
SAMPSIZE (keyword)
CSPLAN command 463
SAR (subcommand)
ARIMA command 183
SAS (keyword)
SAVE TRANSLATE command 1753
SAS files
character encoding 796, 1755
conversion 798
reading 795
value labels 797
saturated models
HILOGLINEAR command 895
SAVAGE (subcommand)
RANK command 1594
SAVE (command)
compared with XSAVE command 1729, 2181
COMPRESSED subcommand 1733
DROP command 1731
KEEP subcommand 1731
MAP subcommand 1733
NAMES subcommand 1733
OUTFILE subcommand 1731
PASSPROTECT subcommand 1734
PERMISSIONS subcommand 1733
RENAME subcommand 1732
syntax chart 1729
UNCOMPRESSED subcommand 1733
UNSELECTED subcommand 1731
with TEMPORARY command 1972
ZCOMPRESSED subcommand 1733
SAVE (keyword)
OPTIMAL BINNING command 1344
SAVE (statement)
MATRIX command 1099
SAVE (subcommand)
2SLS command 95
BAYES REGRESSION command 224, 1583
CATPCA command 284
CLUSTER command 317
CNLR/NLR command 1241
COMPARE DATASETS command 332
COXREG command 375
CSCOXREG command 407
CSGLM command 426
CSLOGISTIC command 437
CSORDINAL command 449
CURVEFIT command 512
DESCRIPTIVES command 574

SAVE (subcommand) *(continued)*
DETECTANOMALY command 581
DISCRIMINANT command 590
GENLIN command 746
GENLINMIXED command 766
GENLOG command 775
GLM command 860
HOMALS command 902
KM command 965
KNN command 975
LINEAR command 983
LOGISTIC REGRESSION command 999
MIXED command 1146
MLP command 1163
MULTIPLE CORRESPONDENCE command 1198
NAIVEBAYES command 1231
OVERALS command 1408
PLUM command 1440
PRINCALS command 1526
RBF command 1612
SPATIAL ASSOCIATION RULES command 1854
SPATIAL TEMPORAL PREDICTION command 1867
SPECTRA command 1895
TCM APPLY command 1945
TCM MODEL command 1964
TMS IMPORT command 1986
TREE command 2001
TSAPPLY command 2020
TSMODEL command 2039
TWOSTEP CLUSTER command 2070
UNIANOVA command 2086
VALIDATEDATA command 2105
with DIMENSION subcommand 902, 1408, 1526
with MATRIX subcommand 902, 1408, 1526
WLS command 2156
SAVE CODEPAGE (command)
DROP command 1737
ENCODING subcommand 1736
KEEP subcommand 1737
OUTFILE subcommand 1736
PASSPROTECT subcommand 1737
syntax chart 1735
SAVE CODEPAGE(command)
UNSELECTED subcommand 1736
SAVE DATA COLLECTION (command)
DROP command 1741
KEEP subcommand 1741
MAP subcommand 1741
METADATA subcommand 1740
OUTFILE subcommand 1740
release history 1739
syntax chart 1739
UNSELECTED subcommand 1741
SAVE MODEL (command)
DROP subcommand 1744
KEEP subcommand 1744
OUTFILE subcommand 1744
syntax chart 1743
TYPE subcommand 1744
SAVE TEMPLATE (subcommand)
AUTORECODE command 190
SAVE TRANLATE (command)
UNCOMPRESSED subcommand 1763
SAVE TRANSLATE (command)
SAVE TRANSLATE (command) *(continued)*
APPEND subcommand 1761
BULKLOADING subcommand 1761
CELLS subcommand 1757
COMPRESSED subcommand 1763
CONNECT subcommand 1759
DROP subcommand 1762
EDITION subcommand 1758
ENCODING subcommand 1755
ENCRYPTED subcommand 1759
EXCELOPTIONS subcommand 1758
FIELDNAMES subcommand 1756
KEEP subcommand 1762
MAP subcommand 1763
MISSING subcommand 1763
missing values 1749
ODBCOPTIONS subcommand 1761
OUTFILE subcommand 1756
PLATFORM subcommand 1758
release history 1747
RENAME subcommand 1762
REPLACE subcommand 1762
SQL subcommand 1760
syntax chart 1747
TABLE subcommand 1759
TEXTOPTIONS subcommand 1757
TYPE subcommand 1753
UNENCRYPTED subcommand 1759
UNSELECTED subcommand 1762
VALFILE subcommand 1759
VERSION subcommand 1754
SAVETM1 (command)
CONNECTION subcommand 1766
CUBE subcommand 1767
MAPPINGS subcommand 1767
release history 1765
syntax chart 1765
saving 1750
saving files
aggregated data files 122
CSV format 1747
data compression 1733, 2184
data files 1729, 1735, 2181
dBASE format 1747
dropping variables 1731, 1737, 2183
encoding 1736
encrypted data files 1734, 1737
Excel format 1747
IBM SPSS Data Collection data 1739
keeping variables 1731, 1737, 2183
Lotus 1-2-3 1747
renaming variables 1732, 2184
spreadsheet format 1747
Stata 1747
SYLK format 1747
tab-delimited data files 1747
variable map 2184
saving output
saving encrypted output using command syntax 1401
saving output using command syntax 1399
SBAR (keyword)
SPCHART command 1886
SCALE (keyword)
DETECTANOMALY command 579

SCALE (keyword) *(continued)*
GENLIN command 732, 741
GENLINMIXED command 765
IGRAPH command 918
RELIABILITY command 1655
SIMRUN command 1836
SCALE (subcommand)
NOMREG command 1257
PLUM command 1441
RELIABILITY command 1654
scale model
PLUM command 1441
scale statistics
RELIABILITY command 1655
scale variables
CTABLES command 486
functions in CTABLES command 486
totaling in CTABLES command 498
SCALEDCONJUGATE (keyword)
MLP command 1157
SCALEMIN (subcommand)
SET command 1802
SCALEMODEL (keyword)
MULTIPLE IMPUTATION command 1204
SCALEWEIGHT (keyword)
GENLIN command 728
SCATTER (subcommand)
IGRAPH command 924
SCATTERPLOT (subcommand)
GRAPH command 882
REGRESSION command 1648
SIMRUN command 1837
scatterplots
all-groups 594
separate-groups 594
SCENARIO (subcommand)
TCM ANALYSIS command 1932
SCENARIOGROUP (subcommand)
TCM ANALYSIS command 1934
SCENARIOPERIOD (subcommand)
TCM ANALYSIS command 1931
SCHEDULE (keyword)
CLUSTER command 318
SCHEFFE (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082
Scheffé test
UNIANOVA command 2082
SCHOENEMANN (keyword)
PREFSCAL command 1509
SCHOENFELD (keyword)
CSCOXREG command 407
Schwarz Bayesian criterion
REGRESSION command 1640
scientific notation
controlling display in output 1802
SCOMPRESSION (subcommand)
SHOW command 1810
SCOPE (keyword)
CSDESCRIPTIVES command 414
NPTESTS command 1267
OPTIMAL BINNING command 1346
VALIDATEDATA command 2103
SCORE (keyword)
ANACOR command 155
CORRESPONDENCE command 362
CSCOXREG command 407
NOMREG command 1254
SCORE variable
ANACOR command 155
SCORES (keyword)
ANACOR command 154
DISCRIMINANT command 590
SCORES (subcommand)
TREE command 2008
SCORING (keyword)
MIXED command 1138
scoring functions 339
scoring rules
TREE command 2000
SCRATCH (keyword)
DISPLAY command 599
scratch variables
defined 44
scree plots
FACTOR command 671
SCREENING (subcommand)
ADP command 117
SELECTPRED command 1782
SCRIPT (command)
release history 1769
syntax chart 1769
scripts
storing to or retrieving from a IBM SPSS Collaboration and Deployment Services Repository 1699
SD (function)
AGGREGATE command 125
SD (keyword)
IGRAPH command 929
MATRIX DATA command 1117
PROXIMITIES command 1556
SD (subcommand)
ARIMA command 182
SDATE format 53
SDBETA (keyword)
REGRESSION command 1634
SDFIT (keyword)
REGRESSION command 1634
SDIFF (function)
CREATE command 383
SDIFF (subcommand)
ACF command 99
CCF command 301
PACF command 1412
PPLOT command 1496
TSPLOT command 2053
SDRESID (keyword)
REGRESSION command 1634
SE (keyword)
COXREG command 375
CSCOXREG command 404
CSDESCRIPTIVES command 414
CSGLM command 424
CSLOGISTIC command 435
CSORDINAL command 447
CSTABULATE command 477
GRAPH command 885

SE (keyword) *(continued)*
IGRAPH command 929
KM command 965
ROC ANALYSIS command 1720
ROC command 1725
SAVE TRANSLATE command 1758
WEIGHTED KAPPA command 2150
XGRAPH command 2174
search functions 78
SEASFACT (subcommand)
EXSMOOTH command 658
SEASON (command)
APPLY subcommand 1773
computing moving averages 1773
MA subcommand 1773
MODEL subcommand 1772
PERIOD subcommand 1773
specifying periodicity 1773
syntax chart 1771
using a previously defined model 1773
VARIABLES subcommand 1772
SEASONAL (subcommand)
ACF command 100
CCF command 302
PACF command 1413
Seasonal Decomposition
command syntax 1771
seasonal difference function 383
seasonal difference transformation
ACF command 99
ARIMA command 181, 182
CCF command 301
in sequence charts 257
PACF command 1412
TSMODEL command 2045, 2046
seasonal factor estimates
EXSMOOTH command 658
seasonal smoothing parameter
EXSMOOTH command 658
seasonality
EXSMOOTH command 657
TSAPPLY command 2021
TSMODEL command 2040
SECOND (keyword)
DATE command 547
SECURITY (subcommand)
REPOSITORY ATTRIBUTES command 1696
seed 1798
SEED (keyword)
LINEAR command 980
NPTESTS command 1267
SIMRUN command 1835
SEED (subcommand)
SET command 1798
SHOW command 1810
SEFIXP (keyword)
MIXED command 1146
SEKURT (keyword)
FREQUENCIES command 720
IGRAPH command 931
MEANS command 1126
SELECT (keyword)
ROC ANALYSIS command 1719
SELECT (subcommand)
SELECT (subcommand) *(continued)*
DISCRIMINANT command 585
FACTOR command 668
LOGISTIC REGRESSION command 995
OMS command 1303
OUTPUT MODIFY command 1376
REGRESSION command 1644
START JOIN command 1900
select cases 697, 1775
SELECT IF (command)
limitations 1775
logical expressions 1775
missing values 1775, 1776
syntax chart 1775
with $CASENUM 1775
with DO IF command 1776
with N OF CASES command 1223, 1775
with REGRESSION command 1644
with SAMPLE command 1727
with TEMPORARY command 1775
SELECTED (keyword)
NAIVEBAYES command 1231
PCUTOFF command 1783
SELECTION (keyword)
ADP command 119
CSSELECT command 473
REGRESSION command 1640
selection rules
TREE command 2000
SELECTPRED (command)
CRITERIA subcommand 1782
EXCEPT subcommand 1782
MISSING subcommand 1783
PLOT subcommand 1784
PRINT subcommand 1783
release history 1779
SCREENING subcommand 1782
syntax chart 1779
SELECTRULE (subcommand)
CSSELECT command 472
SELIRT (keyword)
SUMMARIZE command 1913
SEMEAN (keyword)
DESCRIPTIVES command 575
FREQUENCIES command 720
IGRAPH command 931
MEANS command 1126
OLAP CUBES command 1296
SUMMARIZE command 1913
SEPARATE (keyword)
CSDESCRIPTIVES command 414
CSTABULATE command 478
DISCRIMINANT command 593
REGRESSION command 1646
SEPARATOR (subcommand)
CASESTOVARS command 268
SEPRED (keyword)
GLM command 860
MIXED command 1146
REGRESSION command 1634
UNIANOVA command 2086
SEQBONFERRONI (keyword)
CSCOXREG command 404
CSGLM command 424

SEQBONFERRONI (keyword) *(continued)*
CSLOGISTIC command 436
CSORDINAL command 447
GENLIN command 741
GENLINMIXED command 765
SEQSIDAK (keyword)
CSCOXREG command 404
CSGLM command 424
CSLOGISTIC command 436
CSORDINAL command 447
GENLIN command 741
GENLINMIXED command 765
SEQUENCE (subcommand)
CONJOINT command 344
sequence charts
area charts 258, 2054
command syntax 2051
connecting cases between variables 258, 2054
line charts 258, 2054
multiple variables 258, 2054
plotting highest and lowest values 258, 2054
scale axis reference line 258, 2054
specifying periodicity 257, 2053
split-file scaling 261, 2058
time axis reference lines 260, 2057
transforming values 257, 2053
using previously defined specifications 261, 2058
sequential Bonferroni correction
CSGLM command 424
CSLOGISTIC command 436
sequential quadratic programming
CNLR/NLR command 1243
sequential Sidak correction
CSGLM command 424
CSLOGISTIC command 436
SERIAL (keyword)
PARTIAL CORR command 1419
SERIESFILTER (subcommand)
TCM APPLY command 1942
TCM MODEL command 1961
SERIESPLOT (subcommand)
TSAPPLY command 2019
TSMODEL command 2038
SERROR (subcommand)
ACF command 100
SERVER (subcommand)
REPOSITORY CONNECT command 1698
SES (keyword)
REGRESSION command 1640
SESKEW (keyword)
FREQUENCIES command 720
IGRAPH command 931
MEANS command 1126
SUMMARIZE command 1913
SET (command)
BASETEXTDIRECTION subcommand 1789
BLANKS subcommand 1790
BLOCK subcommand 1790
BOX subcommand 1790
CC subcommand 1791
CELLSBREAK subcommand 1792
CMPTRANS subcommand 1792
CTEMPLATE subcommand 1792
DECIMAL subcommand 1793
SET (command) *(continued)*
DEFOLANG subcommand 1793
DIGITGROUPING subcommand 1793
EPOCH subcommand 1794
ERRORS subcommand 1794
EXTENSIONS subcommand 1794
FOOTNOTE subcommand 1794
FORMAT subcommand 1795
FUZZBITS subcommand 1795
HEADER subcommand 1795
JOURNAL subcommand 1795
LEADZERO subcommand 1796
LOCALE subcommand 1796
MCACHE subcommand 1797
MDISPLAY subcommand 1797
MESSAGES subcommand 1794
MEXPAND subcommand 1797
MIOUTPUT subcommand 1798
MITERATE subcommand 1798
MNEST subcommand 1798
MPRINT subcommand 1797
MTINDEX subcommand 1798
MXCELLS subcommand 1799
MXERRS subcommand 1799
MXLOOPS subcommand 1799
MXMEMORY subcommand 1799
MXWARNS subcommand 1799
OATTRS subcommand 1800
ODISPLAY subcommand 1800
OLANG subcommand 1801
ONUMBERS subcommand 1801
OVARS subcommand 1801
PRINTBACK subcommand 1794
release history 1785
REPDEFER subcommand 1801
RESULTS subcommand 1794
RNG subcommand 1798
ROWSBREAK subcommand 1792
SCALEMIN subcommand 1802
SEED subcommand 1798
SIGLESS subcommand 1802
SMALL subcommand 1802
SORT subcommand 1803
SUMMARY subcommand 1803
syntax chart 1785
TABLERENDER subcommand 1804
THREADS subcommand 1804
TLOOK subcommand 1792
TNUMBERS subcommand 1801
TOLERANCE subcommand 1792
TVARS subcommand 1801
UNDEFINED subcommand 1805
UNICODE subcommand 1805
with LOOP command 1799
with NUMERIC command 1293
with PRESERVE command 1519, 1788
with PRINT command 1529
with PRINT EJECT command 1535
with REGRESSION command 1646
with REPORT command 1676
with RESTORE command 1519, 1711, 1788
with SAMPLE command 1727
with SHOW command 1788
with SUBTITLE (command) 1909

SET (command) *(continued)*
with TITLE command 1975
with WRITE command 2159
with WRITE FORMATS command 2166
WORKSPACE subcommand 1799
XVERSION subcommand 1800
ZCOMPRESSION subcommand 1805
SET_ variable
OVERALS command 1409
SETDIAG (keyword)
MATRIX command 1087
SETS (subcommand)
OVERALS command 1405
with ANALYSIS subcommand 1405
settings
displaying 1809
SEUCLID (keyword)
CLUSTER command 313
PROXIMITIES command 1558
SHAPE (keyword)
IGRAPH command 925, 930
SHAPE (subcommand)
ALSCAL command 138
PROXSCAL command 1571
Shapiro-Wilk's test
EXAMINE command 643
SHEET (subcommand)
GET DATA command 789
Shift Values
command syntax 1807
SHIFT VALUES (command)
syntax chart 1807
SHOW (command)
$VARS subcommand 1810
BASETEXTDIRECTION subcommand 1810
BLANKS subcommand 1810
BLOCK subcommand 1810
BOX subcommand 1810
CACHE subcommand 1810
CC subcommand 1810
CTEMPLATE subcommand 1810
DECIMAL subcommand 1810
DEFOLANG subcommand 1810
DIGITGROUPING subcommand 1810
DIRECTORY subcommand 1810
ENVIRONMENT subcommand 1810
EPOCH subcommand 1810
ERRORS subcommand 1810
EXTENSIONS subcommand 1810
EXTPATHS subcommand 1810
FILTER subcommand 1810
FORMAT subcommand 1810
FUZZBITS subcommand 1810
HANDLES subcommand 1810
HEADER subcommand 1810
JOURNAL subcommand 1810
LEADZERO subcommand 1810
LENGTH subcommand 1810
LICENSE subcommand 1810
LOCALE subcommand 1810
MCACHE subcommand 1810
MDISPLAY subcommand 1810
MESSAGES subcommand 1810
MEXPAND subcommand 1810
SHOW (command) *(continued)*
MITERATE subcommand 1810
MNEST subcommand 1810
MPRINT subcommand 1810
MXCELLS subcommand 1810
MXERRS subcommand 1810
MXLOOPS subcommand 1810
MXMEMORY subcommand 1810
MXWARNS subcommand 1810
N subcommand 1810
OATTRS subcommand 1810
ODISPLAY subcommand 1810
OLANG subcommand 1810
ONUMBERS subcommand 1810
OVARS subcommand 1810
PLUGINS subcommand 1810
PRINTBACK subcommand 1810
release history 1809
REPDEFER subcommand 1810
RESULTS subcommand 1810
RNG subcommand 1810
SCALEMIN subcommand 1810
SCOMPRESSION subcommand 1810
SEED subcommand 1810
SMALL subcommand 1810
SPLIT subcommand 1810
syntax chart 1809
SYSMIS subcommand 1810
TABLERENDER subcommand 1810
TFIT subcommand 1810
THREADS subcommand 1810
TLOOK subcommand 1810
TMSRECORDING subcommand 1810
TNUMBERS subcommand 1810
TVARS subcommand 1810
UNDEFINED subcommand 1810
UNICODE subcommand 1810
VERSION (subcommand) 1810
WEIGHT subcommand 1810
WIDTH subcommand 1810
with SET command 1788
WORKSPACE subcommand 1810
XVERSION subcommand 1810
ZCOMPRESSION subcommand 1810
SHOWDIM (subcommand)
CROSSTABS command 395
SHOWREFLINE (keyword)
AIM command 132
SHOWUNSELECTED (keyword)
SELECTPRED command 1782
SIDAK (keyword)
CSCOXREG command 404
CSGLM command 424
CSLOGISTIC command 436
CSORDINAL command 447
GENLIN command 741
GLM command 856
MIXED command 1140
ONEWAY command 1336
UNIANOVA command 2082
Sidak correction
CSGLM command 424
CSLOGISTIC command 436
Sidak's t test

Sidak's t test *(continued)*
UNIANOVA command 2082
SIDE (keyword)
SIMPLAN command 1828
SIG (function) 1081
SIG (keyword)
CORRELATIONS command 351
FACTOR command 669
NONPAR CORR command 1261
REGRESSION command 1643
SIG.CHISQ (function) 68, 1081, 1082
SIG.F (function) 68, 1082
SIGLESS (subcommand)
SET command 1802
SIGMAINITIAL (keyword)
MLP command 1157
SIGMAS (subcommand)
SPCHART command 1887
SIGMOID (keyword)
MLP command 1156
SIGN (keyword)
IGRAPH command 929
NPTESTS command 1273
SIGN (subcommand)
NPAR TESTS command 1289
SIGNIF (keyword)
MANOVA command 1030, 1048, 1059
SIGNIFICANCE (subcommand)
PARTIAL CORR command 1419
significance level
FACTOR command 669
REGRESSION command 1643
SIGTEST (subcommand)
CTABLES command 501
SIMILARITIES (keyword)
PREFSCAL command 1509
PROXSCAL command 1573
SIMINPUT (subcommand)
SIMPLAN command 1821
SIMPLAN (command)
AUTOFIT subcommand 1827
CONTINGENCY subcommand 1827
CORRELATIONS subcommand 1826
FIXEDINPUT subcommand 1825
MISSING subcommand 1829
MODEL subcommand 1819
PLAN subcommand 1830
SIMINPUT subcommand 1821
SOURCE keyword 1830
STOPCRITERIA subcommand 1828
syntax chart 1815
TARGETOPTS subcommand 1820
VALUELABELS subcommand 1829
SIMPLE (keyword)
COXREG command 370
CSGLM command 423
GENLIN command 741
GENLINMIXED command 764
GLM command 855, 872
GRAPH command 880, 881, 883
LOGISTIC REGRESSION command 992
MANOVA command 1026, 1045
UNIANOVA command 2081
simple contrasts
simple contrasts *(continued)*
COXREG command 370
CSGLM command 423
in MANOVA command 1045
LOGLINEAR command 1005
repeated measures 872
UNIANOVA command 2081
simple effects
in MANOVA 1057
simple matching measure
CLUSTER command 314
PROXIMITIES command 1559
SIMPLE_CHROMY (keyword)
CSPLAN command 459
SIMPLE_SYSTEMATIC (keyword)
CSPLAN command 459
SIMPLE_WOR (keyword)
CSPLAN command 459
SIMPLE_WR (keyword)
CSPLAN command 459
SIMPLEX (keyword)
PROXSCAL command 1571
SIMPREP BEGIN (command)
OUTFILE subcommand 1831
SIMRUN (command)
BOXPLOT subcommand 1837
CRITERIA subcommand 1835
DISTRIBUTION subcommand 1836
OUTFILE subcommand 1839
PLAN subcommand 1835
PRINT subcommand 1838
SCATTERPLOT subcommand 1837
syntax chart 1833
TORNADO subcommand 1838
VIZSTYLESHEET keyword 1839
Simulation
command syntax: create simulation plan 1815
command syntax: run simulation 1833
command syntax: simulation preparation 237
custom models 238
SIMULATION (keyword)
NPTESTS command 1268
SIMULATIONS (keyword)
CONJOINT command 347
SIN (function)
MATRIX command 1077
SIN (keyword)
SPECTRA command 1895
sine function values
saving with SPECTRA command 1895
SINGLE (keyword)
CLUSTER command 317
LIST command 986
single-variable rules
defining 2105
SINGLECAT (keyword)
ADP command 117
SINGLEDF (keyword)
MANOVA command 1031, 1048
SINGULAR (keyword)
ANACOR command 154
CSCOXREG command 403
CSGLM command 424
CSLOGISTIC command 435

SINGULAR (keyword) *(continued)*
CSORDINAL command 446
GENLIN command 732
MIXED command 1138
MULTIPLE IMPUTATION command 1204
NOMREG command 1251
PLUM command 1438
SIZE (keyword)
CLUSTER command 314
DISCRIMINANT command 590
MATRIX command 1094
PCUTOFF command 1782
PROXIMITIES command 1559
SELECTPRED command 1782
SORT CASES command 1842
SIZE (subcommand)
CSPLAN command 460
IGRAPH command 920
size difference measure
CLUSTER command 314
PROXIMITIES command 1559
SKEW (keyword)
IGRAPH command 931
MEANS command 1126
SUMMARIZE command 1913
skewness
EXAMINE command 644
FREQUENCIES command 720
SKEWNESS (function)
REPORT command 1685
SKEWNESS (keyword)
DESCRIPTIVES command 575
FREQUENCIES command 720
SKIP (keyword)
REPORT command 1683, 1690
SKIP (subcommand)
DATA LIST command 522
RECORD TYPE command 1628
SLABELS (subcommand)
CTABLES command 494
SLICE (keyword)
IGRAPH command 926
SLK (keyword)
SAVE TRANSLATE command 1753
SM (keyword)
CLUSTER command 314
PROXIMITIES command 1559
SMA (subcommand)
ARIMA command 183
SMALL (subcommand)
SET command 1802
SHOW command 1810
smallest F-ratio criterion
DISCRIMINANT command 587
SMEAN (function)
RMV command 1715
SMISSING (subcommand)
CTABLES command 505
SMOOTH (keyword)
PREFSCAL command 1511
smoothing function 383
smoothing parameter subcommands
EXSMOOTH command 658
SNAMES (keyword)
SNAMES (keyword) *(continued)*
MATRIX command 1104
SNK (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082
SNOM (keyword)
OVERALS command 1405
PRINCALS command 1523
SOFTMAX (keyword)
MLP command 1156
Sokal and Sneath measures
CLUSTER command 314
PROXIMITIES command 1559
SOLUTION (keyword)
GENLIN command 744
GENLINMIXED command 761
MIXED command 1142, 1143
MLP command 1161
RBF command 1610
SOLVE (function)
MATRIX command 1077
Somers' d
CROSSTABS command 390
SORT (command)
with USE command 2097
SORT (keyword)
FACTOR command 669
GENLIN command 737
MVA command 1217
SORT (subcommand)
DESCRIPTIVES command 575
SET command 1803
SORT CASES (command)
INDEX keyword 1842
OUTFILE subcommand 1842
PASSPROTECT subcommand 1842
SIZE keyword 1842
syntax chart 1841
with ADD FILES command 106, 265, 1843, 2134
with AGGREGATE command 1843
with MATCH FILES command 1843
with PRINT command 1843
with REPORT command 1843
with SPLIT FILE command 1897
with UPDATE command 1843, 2093
SORT VARIABLES (command)
syntax chart 1845
SORTED (keyword)
DISPLAY command 600
sorting
using a third-party sort engine 1803
variables 1845
sorting cases
sort keys 1841
sort order 1841
sorting categories
CTABLES command 497
interactive charts 919
SOURCE (keyword)
CSPLAN command 462
GGRAPH command 827
INSERT HIDDEN command 951
SIMPLAN command 1830

SOURCE (subcommand)
APPLY DICTIONARY command 169
WLS command 2155
SOURCES (keyword)
PREFSCAL command 1507
SP (subcommand)
ARIMA command 182
SP_EXPONENTIAL (keyword)
MIXED command 1137
SP_GAUSSIAN (keyword)
MIXED command 1137
SP_LINEAR (keyword)
MIXED command 1137
SP_LINEARLOG (keyword)
MIXED command 1137
SP_POWER (keyword)
MIXED command 1137
SP_SPHERICAL (keyword)
MIXED command 1137
SPACE (keyword)
MATRIX command 1089
SPAN (subcommand)
SPCHART command 1886
SPATIAL ASSOCIATION RULES (command)
AGGREGATION subcommand 1849
AUTOBINNING subcommand 1849
DATASET subcommand 1850
MAPOUTPUT subcommand 1852
MAPSPEC subcommand 1849
MODELTABLES subcommand 1852
RULEGENERATION subcommand 1851
RULESTABLE subcommand 1853
SAVE subcommand 1854
WORDCLOUD subcommand 1853
SPATIAL ASSOCIATION RULES(command)
syntax chart 1847
SPATIAL MAPSPEC (command)
DATA subcommand 1858
MAP subcommand 1856
MAPPROJECTION subcommand 1856
SPECFILE subcommand 1859
SPATIAL MAPSPEC(command)
syntax chart 1855
SPATIAL TEMPORAL PREDICTION (command)
AGGREGATION subcommand 1862
DATASET subcommand 1863
MAPOUTPUT subcommand 1866
MAPSPEC subcommand 1862
MODELBUILDING subcommand 1865
MODELTABLES subcommand 1866
SAVE subcommand 1867
syntax chart 1862
TIMEFIELDS subcommand 1864
SPATIAL TEMPORAL PREDICTION(command)
syntax chart 1861
SPCHART (command)
(XBARONLY) keyword 1875
c charts 1879
C subcommand 1879
CAPSIGMA subcommand 1886
CONFORM subcommand 1887
CPCHART subcommand 1880
FOOTNOTE subcommand 1872
I subcommand 1875
SPCHART (command) *(continued)*
ID subcommand 1885
individuals charts 1875
IR subcommand 1875
LSL subcommand 1887
MINSAMPLE subcommand 1887
MISSING subcommand 1888
missing values 1888
moving range charts 1875
NOCONFORM subcommand 1887
NORMAL subcommand 1888
np charts 1877
NP subcommand 1877
p charts 1877
P subcommand 1877
process capability chart 1880
R charts 1872
REFERENCE subcommand 1888
release history 1869
RULES subcommand 1885
s charts 1872
SIGMAS subcommand 1887
SPAN subcommand 1886
STATISTICS subcommand 1883
SUBTITLE subcommand 1872
syntax chart 1869
TARGET subcommand 1888
TEMPLATE subcommand 1872
TITLE subcommand 1872
u charts 1879
U subcommand 1879
USL subcommand 1887
X-bar charts 1872
XR subcommand 1872
XS subcommand 1872
SPCT (keyword)
MEANS command 1126
OLAP CUBES command 1296
SUMMARIZE command 1913
SPCT(var) (keyword)
MEANS command 1126
SPEARMAN (keyword)
NAIVEBAYES command 1509
NONPAR CORR command 1261
Spearman correlation coefficient
CROSSTABS command 390
SPECFILE (subcommand)
SPATIAL MAPSPEC command 1859
SPECIAL (keyword)
COXREG command 370
GLM command 855, 872
LOGISTIC REGRESSION command 992
MANOVA command 1026, 1045
UNIANOVA command 2081
special contrasts
repeated measures 872
UNIANOVA command 2081
SPECIFICATION (subcommand)
EXTENSION command 664
SPECIFICATIONS (keyword)
APPLY subcommand 176
ARIMA command 184
CURVEFIT command 513
SPECTRA (command)

SPECTRA (command) *(continued)*
APPLY subcommand 1896
bivariate spectral analysis 1894
BY keyword 1894
CENTER subcommand 1892
centering transformation 1892
CROSS subcommand 1894
PLOT subcommand 1893, 1894
plots 1893, 1894
SAVE subcommand 1895
saving spectral variables 1895
syntax chart 1891
using a previously defined model 1896
VARIABLES subcommand 1892
WINDOW subcommand 1893
windows 1893
spectral analysis 1891
spectral density estimate plot
SPECTRA command 1893
spectral density estimates
saving with SPECTRA command 1895
SPIKE (subcommand)
IGRAPH command 923
spikes
in interactive charts 923
SPLINE (keyword)
IGRAPH command 928
PREFSCAL command 1511
PROXSCAL command 1573, 1575
with VARIABLES keyword 1575
spline interpolation
PROXSCAL command 1573
SPLIT (keyword)
MATRIX command 1104
RELIABILITY command 1655
XGRAPH command 2173
SPLIT (subcommand
VARIABLE ROLE command 2129
SPLIT (subcommand)
MATRIX DATA command 1114
SHOW command 1810
TSPLOT command 2058
SPLIT FILE (command)
limitations 1897
syntax chart 1897
with AGGREGATE command 121, 1897
with SORT CASES command 1897
with TEMPORARY command 1897, 1972
split-file processing
break variables 1897
matrices 1897
scratch variables 1897
system variables 1897
temporary 1971
split-half model
RELIABILITY command 1655
SPNOM (keyword)
CATPCA command 275
CATREG command 292
SPORD (keyword)
CATPCA command 275
CATREG command 292
SPREAD (subcommand)
RECORD TYPE command 1630
spread-versus-level plots
in GLM 852
UNIANOVA command 2078
SPREADLEVEL (keyword)
EXAMINE command 643
GLM command 852
UNIANOVA command 2078
spreadsheet files
read ranges 809
read variable names 809
reading 805
saving 1747
SPSS/PC+ files
saving 1753
SQ (subcommand)
ARIMA command 182
SQL (subcommand)
GET CAPTURE command 784
GET DATA command 787
SAVE TRANSLATE command 1760
SQL queries 783
SQRT (function)
MATRIX command 1077
SQUARE (keyword)
IGRAPH command 925, 927
square root function 61
square root of design effect
CSGLM command 424
square root of the design effect
CSLOGISTIC command 435
square root transformation
TSMODEL command 2044–2046
squared coherency plot
SPECTRA command 1893
squared coherency values
saving with SPECTRA command 1895
squared Euclidean distance
CLUSTER command 313
PROXIMITIES command 1558
SRESID (keyword)
CROSSTABS command 389
GLM command 860
LOGISTIC REGRESSION command 997
REGRESSION command 1634
UNIANOVA command 2086
SRSESTIMATOR (subcommand)
CSPLAN command 458
SS (keyword)
VARCOMP command 2116
SS1 through SS5 (keywords)
CLUSTER command 314
PROXIMITIES command 1559
SSCON (keyword)
NLR command 1244
SSCP (function)
MATRIX command 1077
SSCP (keyword)
MANOVA command 1048
SSQPCT (subcommand)
ARIMA command 184
SSTYPE (keyword)
GLM command 849
MIXED command 1141
UNIANOVA command 2075

SSTYPE (keyword) *(continued)*
VARCOMP command 2115
STACK (keyword)
IGRAPH command 920
STACK (subcommand)
IGRAPH command 921
STACKED (keyword)
GRAPH command 880, 883
stacked bar charts
100% stacking 920
stacking
CTABLES command 485
STAGEVARS (subcommand)
CSPLAN command 462
STAN (keyword)
MANOVA command 1049
stand-in variable 619
standard deviation
DESCRIPTIVES command 575
EXAMINE command 644
FACTOR command 669
FREQUENCIES command 720
MEANS command 1126
OLAP CUBES command 1296
RATIO STATISTICS command 1601, 1602
REGRESSION command 1643
RELIABILITY command 1655
REPORT command 1685
SUMMARIZE command 1913
standard deviation function 63
standard error
CSGLM command 424
CSLOGISTIC command 435
EXAMINE command 644
REGRESSION command 1640
ROC command 1725
standard error of the mean
DESCRIPTIVES command 575
FREQUENCIES command 720
standard errors
in GLM 860
UNIANOVA command 2086
standardization
CORRESPONDENCE command 360
PROXIMITIES command 1556
STANDARDIZE (subcommand)
CORRESPONDENCE command 360
PPLOT command 1496
PROXIMITIES command 1556
STANDARDIZED (keyword)
MLP command 1153
RBF command 1607
standardized residuals
GENLOG command 773
HILOGLINEAR command 894
in GLM 860
UNIANOVA command 2086
STAR JOIN (command)
examples 1902, 1904
FROM subcommand 1900
JOIN subcommand 1901
OUTFILE subcommand 1902
PASSPROTECT subcommand 1902
release history 1899
STAR JOIN (command) *(continued)*
SELECT subcommand 1900
syntax chart 1899
START (keyword)
IGRAPH command 926
STARTS (subcommand)
REPEATING DATA command 1667
Stata
saving data 1747
STATA (keyword)
SAVE TRANSLATE command 1753
Stata files
character encoding 802
saving 1751
STATE (keyword)
TWOSTEP CLUSTER command 2069
statistical functions 63, 337
STATISTICS (subcommand)
CODEBOOK command 327
CORRELATIONS command 351
CROSSTABS command 390
CSCOXREG command 404
CSDESCRIPTIVES command 414
CSGLM command 424
CSLOGISTIC command 435
CSORDINAL command 447
CSTABULATE command 477
DESCRIPTIVES command 575
DISCRIMINANT command 592
EXAMINE command 644
FREQUENCIES command 720
MEANS command 1127
NPAR TESTS command 1291
ONEWAY command 1338
PARTIAL CORR command 1419
REGRESSION command 1639
RELIABILITY command 1655
SPCHART command 1883
SUMMARIZE command 1915
STATUS (keyword)
CSCOXREG command 400
SELECTPRED command 1782
VALIDATEDATA command 2103
STATUS (subcommand)
COXREG command 369
KM command 961
SURVIVAL command 1920
STDDEV (function)
GGRAPH command 823
GRAPH command 877
REPORT command 1685
XGRAPH command 2168
STDDEV (keyword)
CODEBOOK (command) 327
DESCRIPTIVES command 575
DISCRIMINANT command 592
FREQUENCIES command 720
GRAPH command 882, 885
IGRAPH command 931
MATRIX DATA command 1117
MEANS command 1126
OLAP CUBES command 1296
RATIO STATISTICS command 1601, 1602
REGRESSION command 1643

STDDEV (keyword) *(continued)*
SPCHART command 1884
SUMMARIZE command 1913
VALIDATEDATA command 2103
XGRAPH command 2173
STDDEVIANCERESID (keyword)
GENLIN command 746
STDPEARSONRESID (keyword)
GENLIN command 746
STEMLEAF (keyword)
EXAMINE command 643
STEP (keyword)
DISCRIMINANT command 593
NOMREG command 1256
STEPDOWN (keyword)
MANOVA command 1048
STEPLIMIT (keyword)
CNLR command 1243
STEPWISE (keyword)
NPTESTS command 1271, 1273
REGRESSION command 1638
stepwise selection
DISCRIMINANT command 586
REGRESSION command 1638
STERROR (keyword)
GRAPH command 882
stimulus configuration coordinates
ALSCAL command 139, 142, 143
stimulus weights
ALSCAL command 139, 143
STIMWGHT (keyword)
ALSCAL command 139, 143
STOPCRITERIA (subcommand)
SIMPLAN command 1828
STOPPINGRULES (subcommand)
MLP command 1159
STRAIGHT (keyword)
IGRAPH command 924, 928
StrApplyModel (function) 86
STRATA (keyword)
BOOTSTRAP command 249
KM command 964
STRATA (subcommand)
COXREG command 370
KM command 962
stratification variable
KM command 962
STRESS (keyword)
PREFSCAL command 1515
PROXSCAL command 1576, 1577
stress measures
PROXSCAL command 1576
stress plots
PROXSCAL command 1577
STRESSMIN (keyword)
ALSCAL command 141
strictly parallel model
RELIABILITY command 1655
STRICTPARALLEL (keyword)
RELIABILITY command 1655
STRING (command)
syntax chart 1907
with INPUT PROGRAM command 1907
STRING (function) 81
STRING (subcommand)
REPORT command 1681
string expressions
defined 78
string functions
macro facility 564
string length 46
string variables
autorecoding blank strings to user-missing 189
computing values 335, 336
conditional transformations 612, 614, 909, 912
format 46
in logical expressions 78
in matrix language 1071
input formats 515, 528
maximum length 46
maximum width 46
missing values 1129
output formats 713, 1537, 2165
value labels 112, 2109
string width
reading databases 788
strings
converting to numbers 81
STRINGS (keyword)
MATRIX command 1100
STRUCTURE (keyword)
DISCRIMINANT command 593
structure matrix
DISCRIMINANT command 592, 593
STRUNC (function) 78
Student-Newman-Keuls
UNIANOVA command 2082
Studentized maximum modulus distribution function 64
Studentized range distribution function 64
Studentized residuals
in GLM 860
LOGISTIC REGRESSION command 997
UNIANOVA command 2086
STYLE (keyword)
IGRAPH command 928
STYLE (subcommand)
IGRAPH command 920
subcommand
syntax 35
subgroups
splitting data files into 1897
SUBJECT (keyword)
GENLIN command 737
MIXED command 1143, 1144
SUBJECT (subcommand)
CONJOINT command 345
subject weights
ALSCAL command 139, 141–143
SUBJECTS (keyword)
GENLINMIXED command 755, 761
SUBJWGHT (keyword)
ALSCAL command 139, 143
SUBPOP (subcommand)
CSDESCRIPTIVES command 414
CSTABULATE command 478
NOMREG command 1257
SUBSET (subcommand)
NAIVEBAYES command 1230

subsets of cases
conditional expressions 1775
exact-size sample 1727
FILTER command 697
if condition is satisfied 1775
proportional sample 1727
selecting 1775
temporary sample 1727
SUBSTR (function) 78
substrings 91
SUBTITLE (command)
syntax chart 1909
with BEGIN DATA command 1909
with SET command 1909
with TITLE command 1909, 1975
SUBTITLE (keyword)
XGRAPH command 2176
SUBTITLE (subcommand)
GRAPH command 880
IGRAPH command 922
SPCHART command 1872
subtotals
CTABLES command 496
SUBTRACT (function)
REPORT command 1686
SUCCESS (keyword)
NPTESTS command 1268, 1273
SUCCESS (subcommand)
INSERT EXEC command 949
PROPORTIONS command 1553
SUFFIX (keyword)
ADP command 115, 120
TMS IMPORT command 1986
sum
FREQUENCIES command 720
SUM (function)
AGGREGATE command 125
GRAPH command 877
REPORT command 1685
SUM (keyword)
DESCRIPTIVES command 575
FREQUENCIES command 720
IGRAPH command 926, 931
MEANS command 1126
OLAP CUBES command 1296
SUMMARIZE command 1913
SUM (subcommand)
CSDESCRIPTIVES command 413
sum of squares
Type I 849, 2075, 2115
Type II 849, 2075
Type III 849, 2075, 2115
Type IV 849, 2075
SUMAV (keyword)
IGRAPH command 926, 931
summaries
CTABLES command 486
SUMMARIZE (command)
CELLS subcommand 1913
FOOTNOTE subcommand 1913
FORMAT subcommand 1914
MISSING subcommand 1914
statistics 1913
STATISTICS subcommand 1915
SUMMARIZE (command) *(continued)*
syntax chart 1911
TABLES subcommand 1913
TITLE subcommand 1913
SUMMARY (keyword)
COXREG command 373
CSGLM command 425
CSLOGISTIC command 437
CSORDINAL command 449
GENLIN command 744
LOGISTIC REGRESSION command 996
MLP command 1161
NAIVEBAYES command 1231
NOMREG command 1256
PLUM command 1440
RBF command 1610
SELECTPRED command 1783, 1784
TWOSTEP CLUSTER command 2069
SUMMARY (subcommand)
CSDESCRIPTIVES command 412
RELIABILITY command 1656
REPORT command 1684
SET command 1803
summary functions
GRAPH command 877
IGRAPH command 924–926, 928, 931
summary labels
CTABLES command 494
SUMMARYVAR (subcommand)
IGRAPH command 921
sums-of-squares and cross-product matrices
GLM command 866
sums-of-squares and cross-products of residuals
GLM command 866
SUMSPACE (keyword)
REPORT command 1677
SUMSQ (keyword)
IGRAPH command 926, 931
SUPERVISED (keyword)
ADP command 119
SUPPLEMENTARY (subcommand)
CATPCA command 277
CATREG command 293
CORRESPONDENCE command 359
MULTIPLE CORRESPONDENCE command 1194
supplementary objects
MULTIPLE CORRESPONDENCE command 1194
supplementary points
CORRESPONDENCE command 359
supplementary variables
MULTIPLE CORRESPONDENCE command 1194
suppressing tables 393
surrogate predictors
TREE command 1997
SURVIVAL (command)
aggregated data 1923
CALCULATE subcommand 1922
COMPARE subcommand 1922
control variables 1919
factor variables 1919
INTERVAL subcommand 1919
limitations 1917
MISSING subcommand 1924
missing values 1924

SURVIVAL (command) *(continued)*
output file 1924
PLOTS subcommand 1921
PRINT subcommand 1922
saving survival table data 1924
STATUS subcommand 1920
survival time variable 1919
syntax chart 1917
TABLE subcommand 1919
time intervals 1919
with PROCEDURE OUTPUT command 1547
WRITE subcommand 1924
SURVIVAL (keyword)
COXREG command 374, 375
CSCOXREG command 407–409
KM command 962, 965
SURVIVAL command 1921
survival plots
COXREG command 374
KM command 962
SURVIVAL command 1921
survival tables
KM command 963
writing to a file 1547
SURVIVALMETHOD (subcommand)
CSCOXREG command 406
SVAL (function)
MATRIX command 1077
SVD (keyword)
MATRIX command 1087
SWEEP (function)
MATRIX command 1077
sweep matrix
REGRESSION command 1640
SYLK files
read ranges 809
read variable names 809
reading 805
saving 1753
SYM (keyword)
SAVE TRANSLATE command 1753
SYMBOL (keyword)
IGRAPH command 929
SYMMETRIC (keyword)
ALSCAL command 138
symmetric matrix
ALSCAL command 138
SYMMETRICAL (keyword)
CATPCA command 278
CORRESPONDENCE command 360
MULTIPLE CORRESPONDENCE command 1195
symmetrical normalization
MULTIPLE CORRESPONDENCE command 1195
syntax
index of all commands 1
Unicode 938, 947, 950, 952
SYNTAX (keyword)
INSERT command 946
INSERT EXEC command 950
INSERT HIDDDEN command 951
TMS IMPORT command 1986
syntax converter 2221
syntax rules
batch vs. interactive 34
syntax rules *(continued)*
inserted command files 946, 950, 951
SYSFILE INFO (command)
syntax chart 1927
SYSMIS (function) 92
SYSMIS (keyword)
COUNT command 365
MATRIX command 1098
RECODE command 1620
SYSMIS (subcommand)
SHOW command 1810
system variable $CASENUM 45
system variables
case number 45
date and time 45
missing values 45
system-missing values 1129
SYSTEMOUTPUT (subcommand)
TCM APPLY command 1944
TCM MODEL command 1963

T

T (function)
MATRIX command 1077
T (keyword)
IGRAPH command 927, 929
MANOVA command 1033
MVA command 1215, 1220
t distribution function 64
t test
CSGLM command 424
CSLOGISTIC command 435
in MANOVA 1059
MVA command 1215
T-TEST (command)
dependent variables 2063
grouping variables 2063
GROUPS subcommand 2063
independent samples 2061, 2063
limitations 2061
MISSING subcommand 2064
missing values 2064
one sample 2061, 2063
paired samples 2061, 2063
PAIRS subcommand 2063
syntax chart 2061
test value 2063
TESTVAL subcommand 2063
variable list 2063
VARIABLES subcommand 2063
T2 (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082
T3 (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082
T4253H (function)
CREATE command 383
T4253H smoothing 383
TAB (keyword)
SAVE TRANSLATE command 1753

tab-delimited files
reading 805
saving 1750, 1753
TABLE (keyword)
ANACOR command 154
CORRESPONDENCE command 361
COXREG command 375
CROSSTABS command 392
CSTABULATE command 478
DISCRIMINANT command 592
KM command 963
MEANS command 1128
MULT RESPONSE command 1187
SUMMARIZE command 1914
SURVIVAL command 1922
TABLE (subcommand)
ANACOR command 152
BAYES LOGLINEAR command 207
casewise data 152
CORRESPONDENCE command 356
CTABLES command 484
DATA LIST command 521
KEYED DATA LIST command 957
MATCH FILES command 1065
OUTPUT MODIFY command 1381
PRINT command 1533
PROXSCAL command 1569
SAVE TRANSLATE command 1759
SURVIVAL command 1919
table data 152
WRITE command 2162
table lookup files 1065
table specifications
in GLM 859
UNIANOVA command 2085, 2086
TABLECELLS (subcommand)
OUTPUT MODIFY command 1384
TableLooks 1792
TABLEPCT (keyword)
CSTABULATE command 477
TABLERENDER (subcommand)
SET command 1804
SHOW command 1810
tables
converting TABLES command syntax to CTABLES 2221
TABLES
converting TABLES command syntax to CTABLES 2221
TABLES (keyword)
CROSSTABS command 393
CSGLM command 422
GENLIN command 741
GENLINMIXED command 764
GLM command 859
MIXED command 1140
OMS command 1312
SURVIVAL command 1924
UNIANOVA command 2085
TABLES (subcommand)
CROSSTABS command 388
CSTABULATE command 476
MEANS command 1126
MULT RESPONSE command 1184
SUMMARIZE command 1913
TAG (subcommand)
TAG (subcommand) *(continued)*
OMS command 1315
tail probability functions 64, 68, 1082
Tamhane's T2
UNIANOVA command 2082
Tamhane's T3
UNIANOVA command 2082
TANH (keyword)
MLP command 1156
TAPE (keyword)
EXPORT command 651
IMPORT command 934
TARGET (keyword)
ADP command 115, 117, 118, 120
BOOTSTRAP command 249
LINEAR command 980
SIMPLAN command 1820, 1828
SPCHART command 1884
TARGET (subcommand
VARIABLE ROLE command 2129
TARGET (subcommand)
APPLY DICTIONARY command 169
SPCHART command 1888
target variables
computing values 335
counting values 365
formats 336
in COMPUTE command 78
TARGET_OPTIONS (subcommand)
GENLINMIXED command 757
TARGETFILTER (keyword)
SIMPLAN command 1820
TARGETFILTER (subcommand)
TCM APPLY command 1941
TCM MODEL command 1960
TARGETLIST (subcommand)
TCM ANALYSIS command 1935
TARGETOPTS (subcommand)
SIMPLAN command 1820
TARONE (keyword)
KM command 963
Tarone-Ware test
KM command 963
Tarone's statistic
CROSSTABS command 390
tau
CROSSTABS command 390
tau-b
CROSSTABS command 390
tau-c
CROSSTABS command 390
TCM ANALYSIS (command)
EXPRESSIONS subcommand 1931
MODELSYSTEM subcommand 1931
OPTIONS subcommand 1935
SCENARIO subcommand 1932
SCENARIOGROUP subcommand 1934
SCENARIOPERIOD subcommand 1931
syntax chart 1929
TARGETLIST subcommand 1935
TCM APPLY (command)
FILTEREDOUTPUT subcommand 1942
MODELSYSTEM subcommand 1939
OPTIONS subcommand 1939

TCM APPLY (command) *(continued)*
OUTFILE subcommand 1946
PASSPROTECT subcommand 1946
SAVE subcommand 1945
SERIESFILTER subcommand 1942
syntax chart 1937
SYSTEMOUTPUT subcommand 1944
TARGETFILTER subcommand 1941
TCM MODEL (command)
BUILDOPTIONS subcommand 1958
DATAFILTER subcommand 1958
DATASETTINGS subcommand 1952
FIELDS subcommand 1966
FIELDSGROUP subcommand 1966
FILTEREDOUTPUT subcommand 1961
OUTFILE subcommand 1965
PASSPROTECT subcommand 1965
SAVE subcommand 1964
SERIESFILTER subcommand 1961
syntax chart 1949
SYSTEMOUTPUT subcommand 1963
TARGETFILTER subcommand 1960
TCOV (keyword)
DISCRIMINANT command 592
TDF (keyword)
MVA command 1219
TDISPLAY (command)
syntax chart 1969
TYPE subcommand 1969
TEMPLATE (keyword)
GGRAPH command 830
TEMPLATE (subcommand)
CURVEFIT command 514
GRAPH command 885
ONEWAY command 1340
PPLOT command 1498
REGRESSION command 1650
SPCHART command 1872
XGRAPH command 2175
templates
in charts 514, 885, 1340, 1498, 1650, 1872, 2175
TEMPORARY (command)
syntax chart 1971
with N OF CASES command 1223
with REGRESSION command 1644
with SAMPLE command 1727
with SAVE command 1972
with SELECT IF command 1775
with SPLIT FILE command 1897, 1972
with WEIGHT command 2147
with XSAVE command 1972
temporary transformations 1971
temporary variables 44, 1971
terminal nodes
saving terminal node number as variable 2001
termination criteria
ARIMA command 184
territorial map
DISCRIMINANT command 594
TEST (keyword)
AIM command 133
CSORDINAL command 447
NPTESTS command 1268, 1271, 1273
REGRESSION command 1638
TEST (subcommand)
CSCOXREG command 404
CSGLM command 424
CSLOGISTIC command 436
CSORDINAL command 447
CSTABULATE command 477
KM command 963
MIXED command 1146
NOMREG command 1258
PLUM command 1442
TEST(ESTIMABLE) (keyword)
GLM command 850
UNIANOVA command 2076
TEST(LMATRIX) (keyword)
GLM command 850
UNIANOVA command 2076
TEST(MMATRIX) (keyword)
GLM command 866
TEST(SSCP) (keyword)
GLM command 866
TEST(TRANSFORM) (keyword)
GLM command 866
TESTASSUMPTIONS (subcommand)
CSCOXREG command 405
TESTCOV (keyword)
MIXED command 1142
TESTDATA (keyword)
NAIVEBAYES command 1230
TESTING (keyword)
MLP command 1155
RBF command 1608
TESTPOS (keyword)
ROC ANALYSIS command 1719
ROC command 1724
TESTTYPES (keyword)
INDEPENDENTSAMPLES command 1552
PAIREDSAMPLES command 1552
PROPORTIONS command 1551
TESTVAL (keyword)
PROPORTIONS command 1551
TESTVAL (subcommand)
T-TEST command 2063
TESTVALUE (keyword)
NPTESTS command 1268, 1271
text
exporting output as text 1308
TEXT (subcommand)
OUTPUT EXPORT command 1365
text data files
blanks 518
data types 515
fixed format 516, 519, 524
freefield format 516, 518, 519, 525
GET DATA command 785
skipping the first n records 522
variable definition 523
text format
exporting output 1365
TEXTIN (keyword)
IGRAPH command 926
TEXTS (subcommand)
OUTPUT MODIFY command 1389
THREADS (subcommand)
SET command 1804

THREADS (subcommand) *(continued)*
SHOW command 1810
THREE (keyword)
IGRAPH command 921
THRESH (keyword)
SIMPLAN command 1828
THRU (keyword)
COUNT command 365
MISSING VALUES command 1130
RECODE command 1620
SURVIVAL command 1919
USE command 2097
TIES (keyword)
CSCOXREG command 403
TIESTORE (keyword)
ALSCAL command 141
TIF (subcommand)
OUTPUT EXPORT command 1371
TIFT (subcommand)
SHOW command 1810
TIME format 53, 54
time formats
input specifications 54
TIME PROGRAM (command)
syntax chart 1973
time series analysis
data transformations 547
date variables 547
time series functions 379
Time Series Modeler
command syntax 2031
TIME.DAYS (function) 72
TIME.HMS (function) 72
TIMEDURATION (keyword)
ADP command 115
TIMEFIELDS (subcommand)
SPATIAL TEMPORAL PREDICTION command 1864
TIMER (keyword)
CROSSTABS command 391
NAIVEBAYES command 1230
NPAR TESTS command 1292
SELECTPRED command 1782
TITLE (command)
syntax chart 1975
with BEGIN DATA command 1975
with SET command 1975
with SUBTITLE command 1909, 1975
TITLE (keyword)
IGRAPH command 918, 920
MATRIX command 1088
XGRAPH command 2176
TITLE (subcommand)
GRAPH command 880
IGRAPH command 922
OLAP CUBES command 1296
PLANCARDS command 1428
REPORT command 1690
SPCHART command 1872
SUMMARIZE command 1913
titles
displaying 1795
page 1975
TITLES (subcommand)
CTABLES command 500
TITLES (subcommand) *(continued)*
XGRAPH command 2176
TLOOK (subcommand)
SET command 1792
SHOW command 1810
TM1
exporting to Cognos TM1 1765
reading Cognos TM1 data 815
TMS BEGIN (command)
DESTINATION subcommand 1981
release history 1977
syntax chart 1977
TMS END (command)
PRINT subcommand 1984
release history 1983
syntax chart 1983
TMS IMPORT (command)
INFILE subcommand 1986
OUTFILE subcommand 1986
SAVE subcommand 1986
syntax chart 1985
TMS MERGE (command)
DESTINATION subcommand 1990
MODEL subcommand 1990
PRINT subcommand 1990
release history 1989
syntax chart 1989
TRANSFORMATIONS subcommand 1990
TMSRECORDING (subcommand)
SHOW command 1810
TNUMBERS (subcommand)
SET command 1801
SHOW command 1810
TO (keyword)
LIST command 987
REGRESSION command 1637, 1638
RENAME VARIABLES command 1661
STRING command 1907
VECTOR command 2139
tolerance
REGRESSION command 1640, 1641
TOLERANCE (keyword)
MVA command 1219, 1220
REGRESSION command 1640, 1641
TOLERANCE (subcommand)
SET command 1792
tolerance level
UNIANOVA command 2076
TOP (keyword)
TSPLOT command 2054
TOPICS (keyword)
REPOSITORY ATTRIBUTES command 1695
TORGERSON (keyword)
PROXSCAL command 1571
TORNADO (subcommand)
SIMRUN command 1838
TOTAL (keyword)
CROSSTABS command 389
CTABLES command 498
IGRAPH command 930
MULT RESPONSE command 1186
RELIABILITY command 1656
REPORT command 1683
SUMMARIZE command 1914

TOTAL (subcommand)
EXAMINE command 641
totals
CTABLES command 498
TP (keyword)
MIXED command 1137
TPATTERN (subcommand)
MVA command 1218
TPH (keyword)
MIXED command 1137
TRACE (function)
MATRIX command 1077
TRAILINGSPACES (subcommand)
GET DATA command 790
TRAINING (keyword)
KNN command 971
MLP command 1155, 1157
RBF command 1608
TRAININGSAMPLE (subcommand)
NAIVEBAYES command 1229
TRAININGTIMER (keyword)
MLP command 1159
TRANS (keyword)
CATPCA command 282
CATREG command 297
MULTIPLE CORRESPONDENCE command 1197
OVERALS command 1407
transaction files 2091
transfer function
TSMODEL command 2046
TRANSFERFUNCTION (subcommand)
TSMODEL command 2046
TRANSFORM (keyword)
GGRAPH command 826
MANOVA command 1048
TRANSFORM (subcommand)
ADP command 119
TRANSFORMATION (keyword)
PREFSCAL command 1514, 1517
PROXSCAL command 1576, 1578
TRANSFORMATION (subcommand)
PREFSCAL command 1511
PROXSCAL command 1573
transformation coefficients matrix 841
transformation expressions
exporting to PMML 1977
merging transformation PMML with model XML 1989
missing values 89
transformation matrix
displaying 869
in MANOVA command 1047
transformation plots
CATPCA command 282
MULTIPLE CORRESPONDENCE command 1197
OVERALS command 1407
PROXSCAL command 1577
transformations
temporary 1971
TRANSFORMATIONS (keyword)
PREFSCAL command 1515
PROXSCAL command 1577
TMS IMPORT command 1986
TRANSFORMATIONS (subcommand)
TMS MERGE command 1990
TRANSFORMED (keyword)
GENLIN command 741
GENLINMIXED command 765
transformed proximities
PROXSCAL command 1576
TRANSPOS (function)
MATRIX command 1077
transposing cases and variables 710
TRCOLUMNS (keyword)
ANACOR command 154
CORRESPONDENCE command 361
TRDATA (keyword)
CATPCA command 284, 286
CATREG command 298
MULTIPLE CORRESPONDENCE command 1198, 1199
TREE (command)
CHAID subcommand 2004
COSTS subcommand 2007
CRT subcommand 2006
DEPCATEGORIES subcommand 1995
forcing a variable into the model 1995
GAIN subcommand 1998
GROWTHLIMIT subcommand 2003
INFLUENCE subcommand 2010
limitations 1992
measurement level 1994
METHOD subcommand 2002
minimum specifications 1992
MISSING (subcommand) 2010
missing values 1995
model variables 1994
PLOT subcommand 1999
PRINT subcommand 1997
prior probability 2007
PRIORS subcommand 2007
PROFITS (subcommand) 2009
QUEST subcommand 2006
release history 1991
RULES subcommand 2000
SAVE subcommand 2001
saving model PMML file 2010
saving predicted probability as variable 2001
saving predicted value as variable 2001
saving terminal node number as variable 2001
SCORES subcommand 2008
selection and scoring rules 2000
significance levels for node splitting and merging 2004
syntax chart 1991
TARGETRESPONSE (subcommand) 2011
tree model in table format 1997
TREE subcommand 1996
VALIDATION subcommand 2003
TREE (subcommand)
TREE command 1996
TREEFORMAT (keyword)
OMS command 1311
TREND (function)
RMV command 1715
TREND (subcommand)
KM command 964
trend modification parameter
EXSMOOTH command 658
trend smoothing parameter
EXSMOOTH command 658

TRIANGLE (keyword)
NAIVEBAYES command 1509
trimmed mean
EXAMINE command 644
TRIMOUTLIERS (keyword)
NPTESTS command 1271
TRIPLOT (keyword)
CATPCA command 282
triplots
CATPCA command 282
TRROWS (keyword)
ANACOR command 154
CORRESPONDENCE command 361
TRUNC (function)
MATRIX command 1077
TRUNCATE (keyword)
CROSSTABS command 393
TSAPPLY (command)
AUXILIARY subcommand 2021
confidence intervals 2021
drop selected models 2015, 2022
forecasting 2015
goodness of fit 2016
keep selected models 2015, 2022
lags displayed 2021
MISSING subcommand 2022
MODEL subcommand 2022
MODELDETAILS subcommand 2018
MODELSTATISTICS subcommand 2017
MODELSUMMARY subcommand 2016
OUTPUTFILTER subcommand 2019
periodicity 2021
reestimate model parameters 2015, 2021
release history 2013
SAVE subcommand 2020
save updated models 2015, 2022
seasonality 2021
SERIESPLOT subcommand 2019
syntax chart 2013
TSET (command)
DEFAULT subcommand 2025
ID subcommand 2026
MISSING subcommand 2026
MXNEWVAR subcommand 2026
MXPREDICT subcommand 2026
NEWVAR subcommand 2026
PERIOD subcommand 2026
PRINT subcommand 2027
syntax chart 2025
TSET (subcommand)
READ MODEL command 1617
TSHOW (command)
syntax chart 2029
TSMODEL (command)
ARIMA subcommand 2045
AUTOOUTLIER subcommand 2048
AUXILIARY subcommand 2040
confidence intervals 2040
difference transformation 2045
events 2041
EXPERTMODELER subcommand 2043
EXSMOOTH subcommand 2044
forecasting 2034
goodness of fit 2035
TSMODEL (command) *(continued)*
lags displayed 2040
MISSING subcommand 2041
model names 2041
MODEL subcommand 2041
MODELDETAILS subcommand 2037
MODELSTATISTICS subcommand 2037
MODELSUMMARY subcommand 2036
natural log transformation 2044–2046
OUTLIER subcommand 2049
OUTPUTFILTER subcommand 2038
periodicity 2040
release history 2031
SAVE subcommand 2039
seasonal difference transformation 2045
seasonality 2040
SERIESPLOT subcommand 2038
square root transformation 2044–2046
syntax chart 2031
TRANSFERFUNCTION subcommand 2046
TSPACE (keyword)
REPORT command 1677
TSPLOT (command)
APPLY subcommand 2058
DIFF subcommand 2053
FORMAT subcommand 2054
ID subcommand 2054
LN/NOLOG subcommands 2054
MARK subcommand 2057
PERIOD subcommand 2053
release history 2051
SDIFF subcommand 2053
SPLIT subcommand 2058
syntax chart 2051
VARIABLES subcommand 2053
TTEST (keyword)
CSCOXREG command 404
CSDESCRIPTIVES command 412, 413
CSGLM command 424
CSLOGISTIC command 435
CSORDINAL command 447
TTEST (subcommand)
MVA command 1215
Tucker's coefficient of congruence
PROXSCAL command 1576
TUKEY (keyword)
EXAMINE command 644
GLM command 856
ONEWAY command 1336
PPLOT command 1494
RANK command 1596
RELIABILITY command 1655
SPECTRA command 1893
UNIANOVA command 2082
Tukey-Hamming window
SPECTRA command 1893
Tukey's b test
UNIANOVA command 2082
Tukey's honestly significant difference
UNIANOVA command 2082
Tukey's test of additivity
RELIABILITY command 1655
Tukey's transformation 1596
TVARS (subcommand)

TVARS (subcommand) *(continued)*
SET command 1801
SHOW command 1810
TWEEDIE (keyword)
GENLIN command 728
Two-Stage Least-Squares Regression
command syntax 93
TWOSTEP CLUSTER (command)
automatic cluster selection 2069
CATEGORICAL subcommand 2066
CONTINUOUS subcommand 2067
CRITERIA subcommand 2067
DISTANCE subcommand 2067
HANDLENOISE subcommand 2067
INFILE subcommand 2068
MEMALLOCATE subcommand 2068
MISSING subcommand 2068
NOSTANDARDIZE subcommand 2068
NUMCLUSTERS subcommand 2069
OUTFILE subcommand 2069
PRINT subcommand 2069
SAVE subcommand 2070
syntax chart 2065
VIEWMODEL subcommand 2070
TWOTAIL (keyword)
CORRELATIONS command 351
NONPAR CORR command 1261
PARTIAL CORR command 1419
TXT (keyword)
GET DATA command 786
text data file encoding 786
TYPE (keyword)
CODEBOOK command 325
CSCOXREG command 404
CSORDINAL command 447
MATRIX command 1101
SIMPLAN command 1821, 1828
SIMRUN command 1838
XGRAPH command 2173
TYPE (subcommand)
EXPORT command 651
GET DATA command 786
GET TRANSLATE command 808
IMPORT command 934
PPLOT command 1494
READ MODEL command 1616
SAVE MODEL command 1744
SAVE TRANSLATE command 1753
TDISPLAY command 1969
Type I sum-of-squares method
VARCOMP command 2115
Type III sum-of-squares method
VARCOMP command 2115

U

U (subcommand)
data organization 1879
SPCHART command 1879
variable specification 1880
u charts
SPCHART command 1879
UC (keyword)
CROSSTABS command 390
UCL_CUMHAZARD (keyword)
CSCOXREG command 407
UCL_SURVIVAL (keyword)
CSCOXREG command 407
ULEFT (keyword)
IGRAPH command 926
ULS (keyword)
FACTOR command 672
UN (keyword)
MIXED command 1137
UN_AR1 (keyword)
MIXED command 1137
UN_CS (keyword)
MIXED command 1137
UN_UN (keyword)
MIXED command 1137
UNBALANCED (keyword)
CATPCA command 280
uncentered leverage values
UNIANOVA command 2086
uncertainty coefficient
CROSSTABS command 390
UNCLASSIFIED (keyword)
DISCRIMINANT command 593
UNCOMPRESSED (subcommand)
SAVE command 1733
SAVE TRANSLATE command 1763
XSAVE command 2184
UNCONDITIONAL (keyword)
ALSCAL command 139
MANOVA command 1051
PREFSCAL command 1510
PROXSCAL command 1572
UNDEFINED (subcommand)
SET command 1805
SHOW command 1810
UNDERSCORE (keyword)
REPORT command 1677, 1683
UNENCRYPTED (subcommand)
GET CAPTURE command 784
GET DATA command 787
SAVE TRANSLATE command 1759
UNEQUAL_WOR (keyword)
CSPLAN command 463
unexplained variance criterion
DISCRIMINANT command 587
UNIANOVA (command)
contained effects 2075
CONTRAST subcommand 2081
CRITERIA subcommand 2076
EMMEANS subcommand 2085
estimated marginal means 2085
INTERCEPT subcommand 2076
KMATRIX subcommand 2080
LMATRIX subcommand 2079
METHOD subcommand 2075
MISSING subcommand 2076
OUTFILE subcommand 2087
PLOT subcommand 2078
POSTHOC subcommand 2082
PRINT subcommand 2076
RANDOM subcommand 2074
REGWGT subcommand 2074
ROBUST subcommand 2086

UNIANOVA (command) *(continued)*
SAVE subcommand 2086
Type I sum-of-squares method 2075
Type II sum-of-squares method 2075
Type III sum-of-squares method 2075
Type IV sum-of-squares method 2075
univariate 2071
Unicode
command syntax 938, 947, 950, 952
text data files 519, 681
UNICODE (subcommand)
SET command 1805
SHOW command 1810
UNIFORM (function)
MATRIX command 1077
UNIFORM (keyword)
CATPCA command 276
CATREG command 293
IGRAPH command 930
MULTIPLE CORRESPONDENCE command 1193
NPTESTS command 1268
with DISTR keyword 276
uniform distribution function 64
UNIQUE (keyword)
ANOVA command 161
UNIQUECAT (keyword)
ADP command 117
UNIT (keyword)
ADP command 115
SIMPLAN command 1828
SPECTRA command 1893
UNIV (keyword)
MANOVA command 1048
UNIVARIATE (keyword)
FACTOR command 669
MANOVA command 1034
UNIVF (keyword)
DISCRIMINANT command 592
unknown measurement level 58
UNNUMBERED (keyword)
LIST command 986
UNR (keyword)
MIXED command 1137
UNSELECTED (keyword)
DISCRIMINANT command 593
UNSELECTED (subcommand)
EXPORT command 652
SAVE CODEPAGE command 1736
SAVE command 1731
SAVE DATA COLLECTION command 1741
SAVE TRANSLATE command 1762
unstandardized predicted values
in GLM 860
UNIANOVA command 2086
unstandardized residuals
in GLM 860
UNIANOVA command 2086
UNSTRUCTURED (keyword)
GENLIN command 737
UNTIE (keyword)
PREFSCAL command 1511
PROXSCAL command 1575
with ORDINAL keyword 1575
unweighted functions
unweighted functions *(continued)*
CTABLES command 486
UP (keyword)
IGRAPH command 929
SORT CASES command 1841
UPCASE (function) 78
UPDATE (command)
BY subcommand 2094
DROP subcommand 2095
FILE subcommand 2093
IN subcommand 2095
KEEP subcommand 2095
limitations 2091
MAP subcommand 2095
RENAME subcommand 2094
syntax chart 2091
with DATA LIST command 2093
with DROP DOCUMENTS command 2091
with SORT CASES command 1843, 2093
UPDATECORR (keyword)
GENLIN command 737
updating data files
dropping variables 2095
flag variables 2095
input files 2093
keeping variables 2095
key variables 2091
limitations 2091
primary files 2091
raw data files 2093
renaming variables 2094
transaction files 2091
variable map 2095
updating database tables 1760
UPPER (keyword)
FLEISS MULTIRATER KAPPA command 707
MATRIX DATA command 1113
PROXSCAL command 1571
UPPERBOUND (subcommand)
CURVEFIT command 511
UPPEREND (keyword)
OPTIMAL BINNING command 1345
URIGHT (keyword)
IGRAPH command 926
USE (command)
case specifications 2097
DATE specifications 2097
examples 2098
FIRST and LAST keywords 2097
syntax chart 2097
with SORT command 2097
USE (commmand)
PERMANENT subcommand 2098
USE (keyword)
XGRAPH command 2174
USE_AUTO_DATA_PREPARATION (keyword)
LINEAR command 980
user-missing values 1129
USERMISSING (keyword)
BOOTSTRAP command 250
KNN command 974
MLP command 1161
NAIVEBAYES command 1231
NPTESTS command 1267

USERMISSING (keyword) *(continued)*
RBF command 1610
SELECTPRED command 1783
USL (keyword)
SPCHART command 1884
USL (subcommand)
SPCHART command 1887
UTILITY (subcommand)
CONJOINT command 347
with FACTORS subcommand 347

V

VAC (keyword)
OLAP CUBES command 1297
VAF (keyword)
CATPCA command 281, 282
VAL (keyword)
IGRAPH command 924–926, 928, 929
valid values
excluding in CTABLES command 496
Validate Data
command syntax 2099
VALIDATEDATA (command)
CASECHECKS subcommand 2103
CASEREPORT subcommand 2104
IDCHECKS subcommand 2103
release history 2099
RULESUMMARIES subcommand 2104
SAVE subcommand 2105
syntax chart 2099
VARCHECKS subcommand 2103
VALIDATION (subcommand)
TREE command 2003
VALIDLIST (subcommand)
SUMMARIZE command 1914
VALIDN (function)
GGRAPH command 823
REPORT command 1685
XGRAPH command 2168
VALLABELS (keyword)
APPLY DICTIONARY command 171
value
syntax 35
VALUE (function)
XGRAPH command 2168
VALUE (keyword)
CSPLAN command 460, 461, 464
REPORT command 1679, 1682
SIMPLAN command 1825
value labels
adding 2109
ANACOR command 154
apostrophes in 2109
as point labels HOMALS command 900
as point labels OVERALS command 1407
concatenating strings 2109, 2110
controlling wrapping 2109
copying from other variables in current or external data file 171
date format variables 2109
HOMALS command 900
length 2109
SAS files 797
value labels *(continued)*
saving in Excel files 1757
string variables 112, 2109
using as values for computed variables 82
VALUELABEL function 82
VALUE LABELS (command)
compared with ADD VALUE LABELS command 2109
release history 2109
syntax chart 2109
with ORTHOPLAN command 1349
with PLANCARDS command 1425
VALUELABEL (function) 82
VALUELABELS (keyword)
CODEBOOK command 325
VALUELABELS (subcommand)
SIMPLAN command 1829
Van der Waerden's transformation 1596
VAR (keyword)
IGRAPH command 918
REPORT command 1691
VARCHECKS (subcommand)
VALIDATEDATA command 2103
VARCOMP (command)
CRITERIA subcommand 2116
DESIGN subcommand 2117
interactions 2117
INTERCEPT subcommand 2115
maximum-likelihood method 2115
METHOD subcommand 2115
minimum norm quadratic unbiased estimator 2115
MINQUE keyword 2115
MISSING subcommand 2115
nested design 2117
OUTFILE subcommand 2116
PRINT subcommand 2116
RANDOM subcommand 2114
REGWGT subcommand 2115
restricted maximum likelihood estimation 2115
sum-of-squares method 2115
syntax chart 2113
VAREST (keyword)
VARCOMP command 2116
VARIABLE (keyword)
AIM command 133
CASESTOVARS command 268
CATPCA command 277
CSGLM command 425
CSLOGISTIC command 436
CSPLAN command 460–462, 464
DESCRIPTIVES command 576
GRAPH command 886
KNN command 971, 974
MLP command 1155
MULTIPLE CORRESPONDENCE command 1194
NAIVEBAYES command 1229
PROXIMITIES command 1556, 1557
RBF command 1608
SUMMARIZE command 1914
VARIABLE ALIGNMENT (command)
syntax chart 2119
VARIABLE ATTRIBUTE (command)
defining cross-variable rules 2107
defining single-variable rules 2105
release history 2121

VARIABLE ATTRIBUTE (command) *(continued)*
syntax chart 2121
variable attributes
adding and deleting custom attributes 2121
variable formats
date and time formats 54
numeric 48
string 46
variable labels
apostrophes in 2125
as plot labels HOMALS command 900
as plot labels OVERALS command 1407
concatenating strings 2125
controlling wrapping 2125
CTABLES command 504
HOMALS command 900
VARIABLE LABELS (command)
syntax chart 2125
with PLANCARDS command 1425
VARIABLE LEVEL (command)
syntax chart 2127
variable list
GENLIN command 726
variable lists
ranges using TO keyword 43
variable names
converting long names in earlier versions 43
in matrix data files 43
OMS command 1322
preserving case 42
rules 42
special considerations for long variable names 43
variable principal normalization
MULTIPLE CORRESPONDENCE command 1195
VARIABLE ROLE (command)
syntax chart 2129
variable sets
copying sets from another data file 170
variable types
CTABLES command 484
variable weight
CATPCA command 275
MULTIPLE CORRESPONDENCE command 1192
VARIABLE WIDTH (command)
syntax chart 2131
VARIABLEINFO (keyword)
CSGLM command 425
CSLOGISTIC command 437
CSORDINAL command 449
variables
controlling default format 1794, 1795
creating new variables with variable definition attributes
of existing variables 169
defining 523, 1293, 1625, 1907
in matrix language 1070
naming rules 524
scratch 44
sorting 1845
temporary 1971
VARIABLES (keyword)
CSTABULATE command 476
DISPLAY command 599
EXAMINE command 641
GGRAPH command 821
VARIABLES (keyword) *(continued)*
MATRIX command 1097
MULTIPLE IMPUTATION command 1207
PROXSCAL command 1574, 1576–1578
VALIDATEDATA command 2102
VARIABLES (subcommand)
ACF command 98
ALSCAL command 137
ANOVA command 160
AREG command 174
ARIMA command 181
AUTORECODE command 188
BOOTSTRAP command 249
CATPCA command 274
CATREG command 291, 298
CCF command 300
COMPARE DATASETS command 332
COXREG command 369
CROSSTABS command 387
CSCOXREG command 400
CURVEFIT command 511
DESCRIPTIVES command 574
DETECTANOMALY command 579
DISPLAY command 600
EXAMINE command 641
EXSMOOTH command 656
FACTOR command 667
FLIP command 710
FREQUENCIES command 716
GET DATA command 792
HOMALS command 898
LIST command 986
LOGISTIC REGRESSION command 991
MATRIX DATA command 1111
MULT RESPONSE command 1183
MULTIPLE CORRESPONDENCE command 1192
MVA command 1213
NONPAR CORR command 1260
OPTIMAL BINNING command 1344
OVERALS command 1405
PACF command 1412
PARTIAL CORR command 1418
PPLOT command 1493
PREFSCAL command 1507
PRINCALS command 1522
RANK command 1594
REGRESSION command 1637
RELIABILITY command 1654
REPORT command 1679
RESPONSE RATE command 1708
SEASON command 1772
SPECTRA command 1892
T-TEST command 2063
TSPLOT command 2053
VERIFY command 2145
with ANALYSIS subcommand 899, 1405
WLS command 2155
variance
EXAMINE command 644
FREQUENCIES command 720
MEANS command 1126
OLAP CUBES command 1296
REGRESSION command 1640, 1643
RELIABILITY command 1655, 1656

variance *(continued)*
REPORT command 1685
SUMMARIZE command 1913
VARIANCE (function)
GGRAPH command 823
GRAPH command 877
REPORT command 1685
XGRAPH command 2168
VARIANCE (keyword)
CLUSTER command 314
CORRESPONDENCE command 362
DESCRIPTIVES command 575
FREQUENCIES command 720
IGRAPH command 931
MEANS command 1126
PROXIMITIES command 1559
REGRESSION command 1643
RELIABILITY command 1656
SUMMARIZE command 1913
variance accounted for
CATPCA command 281
Variance Components
command syntax 2113
variance inflation factor
REGRESSION command 1640
VARIANCES (subcommand)
ANACOR command 154
VARIMAX (keyword)
CATPCA command 279
FACTOR command 673
MANOVA command 1049
varimax rotation
FACTOR command 673
VARINFO (subcommand)
CODEBOOK command 325
VARNAME_ variable
ANACOR command 155
CORRESPONDENCE command 362
HOMALS command 902
OVERALS command 1409
PRINCALS command 1527
VARORDER (keyword)
SIMPLAN command 1826
VARSTOCASES (command)
COUNT subcommand 2137
DROP subcommand 2137
ID subcommand 2135
INDEX subcommand 2135
KEEP subcommand 2137
limitations 2133
MAKE subcommand 2134
overview 2133
syntax chart 2133
with SORT CASES command 2134
VARTYPE_ variable
OVERALS command 1409
PRINCALS command 1527
VC (keyword)
MIXED command 1137
VECTOR (command)
examples 630
index 2139, 2142
short form 2141
syntax chart 2139
VECTOR (command) *(continued)*
TO keyword 2139
variable list 2139
with INPUT PROGRAM command 2141
with LOOP command 2139, 2140
VECTOR (keyword)
DISPLAY command 599
vectors
index 2139, 2142
variable list 2139
VERIFY (command)
syntax chart 2145
VARIABLES subcommand 2145
VERSION (subcommand)
SHOW command 1810
VERSIONLABEL (keyword)
REPOSITORY ATTRIBUTES command 1695
VERTICAL (keyword)
IGRAPH command 921
VICICLE (keyword)
CLUSTER command 319
VIEW (keyword)
CSPLAN command 457
VIEW (subcommand)
GETTM1 command 817
PROXIMITIES command 1557
VIEWMODEL (subcommand)
KNN command 975
TWOSTEP CLUSTER command 2070
VIEWNAME (subcommand)
IGRAPH command 922
VIND (subcommand)
CASESTOVARS command 266
VIZMAP (keyword)
GGRAPH command 831
VIZSTYLESHEET (keyword)
GGRAPH command 831
SIMRUN command 1839
VLABELS (subcommand)
CTABLES command 504
VPC (keyword)
OLAP CUBES command 1297
VPRINCIPAL (keyword)
CATPCA command 278
MULTIPLE CORRESPONDENCE command 1195
VS (keyword)
MANOVA command 1040
VW (keyword)
PPLOT command 1494
RANK command 1596

W

W-W (subcommand)
NPAR TESTS command 1290
WALD (keyword)
COXREG command 371
GENLIN command 732
NOMREG command 1254
Wald statistic
COXREG command 371
LOGISTIC REGRESSION command 993
WALD_WOLFOWITZ (keyword)
NPTESTS command 1271

Wald-Wolfowitz test
NPAR TESTS command 1290
WALLER (keyword)
GLM command 856
ONEWAY command 1336
UNIANOVA command 2082
Waller-Duncan t test
UNIANOVA command 2082
WARD (keyword)
CLUSTER command 317
Ward's method
CLUSTER command 317
WARN (keyword)
FILE TYPE command 692
RECORD TYPE command 1629
SET command 1805
warnings
displaying 1794
maximum number 1799
WAVERAGE (keyword)
CLUSTER command 317
EXAMINE command 642
WCOC (keyword)
RATIO STATISTICS command 1601, 1602
WEEK (keyword)
DATE command 547
weekday 54
WEIBULL (function) 70, 1086
Weibull distribution function 64
WEIGHT (command)
missing values 2147
non-positive values 2147
syntax chart 2147
weight variable 2147
with ANACOR command 156
with CORRESPONDENCE command 357
with CROSSTABS command 394
with TEMPORARY command 2147
WEIGHT (keyword)
APPLY DICTIONARY command 170
CATPCA command 275
CODEBOOK command 326
CSPLAN command 463
MULTIPLE CORRESPONDENCE command 1192
WEIGHT (subcommand)
CTABLES command 505
SHOW command 1810
WLS command 2155
Weight Estimation
command syntax 2153
weight variables
saving WLS command 2156
WEIGHTED (keyword)
PREFSCAL command 1512
PROXSCAL command 1574
WEIGHTED KAPPA (command)
CRITERIA subcommand 2150
MISSING keyword 2151
missing values 2151
PRINT subcommand 2150
syntax chart 2149
weighted least squares
REGRESSION command 1642
weighted mean
weighted mean *(continued)*
RATIO STATISTICS command 1601, 1602
weighted multidimensional scaling
ALSCAL command 140
weighted unstandardized predicted values
in GLM 860
UNIANOVA command 2086
weighted unstandardized residuals
in GLM 860
UNIANOVA command 2086
WEIGHTFEATURES (keyword)
KNN command 973
WEIGHTING (keyword)
WEIGHTED KAPPA command 2150
weighting cases 2147
weights
WLS command 2155
WEIGHTS (keyword)
OVERALS command 1407
PREFSCAL command 1514, 1515, 1517
PROXSCAL command 1576–1578
WEIGHTS (subcommand)
PREFSCAL command 1509
PROXSCAL command 1572
WELCH (keyword)
ONEWAY command 1338
WGTMEAN (keyword)
RATIO STATISTICS command 1601, 1602
WHISKER (keyword)
IGRAPH command 927
WHITE (keyword)
GLM command 850, 2076
wide data files
specifying record length with FILE HANDLE 680
width
controlling column width in Data View 2131
VARIABLE WIDTH command 2131
WIDTH (keyword)
IGRAPH command 930
WIDTH (subcommand)
REGRESSION command 1646
SHOW command 1810
WILCOXON (keyword)
NPTESTS command 1268, 1273
WILCOXON (subcommand)
NPAR TESTS command 1291
WILD (subcommand)
FILE TYPE command 692
WILKS (keyword)
DISCRIMINANT command 587
Wilks' lambda
in MANOVA 1050
WINDOW (subcommand)
SPECTRA command 1893
windows
SPECTRA (command) 1893
WINTERS (keyword)
EXSMOOTH command 657
WITH (keyword)
ANOVA command 161
CORRELATIONS command 352
CURVEFIT command 511
GENLOG command 775
LOGISTIC REGRESSION command 991

WITH (keyword) *(continued)*
LOGLINEAR command 1009
MIXED command 1140
NOMREG command 1250
NONPAR CORR command 1260
NPAR TESTS command 1278
PARTIAL CORR command 1418
PROBIT command 1542
T-TEST command 2063
WITHIN (keyword)
MANOVA command 1026, 1039
NOMREG subcommand 1252
SPCHART command 1886
VARCOMP command 2117
within-subjects factors
in MANOVA 1057
in MANOVA command 1055
within-subjects model 874
WITHINSUBJECT (keyword)
GENLIN command 737
WK1 (keyword)
SAVE TRANSLATE command 1753
WKDAY format 53, 54
WKS (keyword)
SAVE TRANSLATE command 1753
WKYR format 53, 54
WLS (command)
APPLY subcommand 2156
CONSTANT subcommand 2156
DELTA subcommand 2155
including constant 2156
limitations 2153
NOCONSTANT subcommand 2156
power range 2155
POWER subcommand 2155
PRINT subcommand 2156
SAVE subcommand 2156
saving weight variables 2156
SOURCE subcommand 2155
syntax chart 2153
using previous model 2156
VARIABLES subcommand 2155
WEIGHT subcommand 2155
WOR (keyword)
CSPLAN command 458
Word format
exporting output 1361
WORDCLOUD (subcommand)
SPATIAL ASSOCIATION RULES command 1853
working directory
changing 947
WORKINGCORR (keyword)
GENLIN command 744
WORKSPACE (subcommand)
SHOW command 1799, 1810
WPRED (keyword)
GLM command 860
UNIANOVA command 2086
WR (keyword)
CSPLAN command 458, 463
WRAP (keyword)
LIST command 986
wrapping
value labels 2109
wrapping *(continued)*
variable labels 2125
WRESID (keyword)
GLM command 860
UNIANOVA command 2086
WRITE (command)
ENCODING subcommand 2162
formats 2160
missing values 2159
NOTABLE subcommand 2162
OUTFILE subcommand 2162
RECORDS subcommand 2161
strings 2161
syntax chart 2159
TABLE subcommand 2162
variable list 2159
with SET command 2159
WRITE (statement)
MATRIX command 1095
WRITE (subcommand)
CROSSTABS command 393
SURVIVAL command 1924
write formats 2165
WRITE FORMATS (command)
format specification 2165
string variables 2165
syntax chart 2165
with DISPLAY command 2165
with SET command 2166
writing cases 2159
WSDESIGN (subcommand)
GLM command 874
MANOVA command 1057
WSFACTOR (subcommand)
GLM command 871
WSFACTORS (subcommand)
MANOVA command 1055

X

X-bar charts
SPCHART command 1872
X1 (subcommand)
IGRAPH command 918
X1INTERVAL (keyword)
IGRAPH command 930
X1LENGTH (subcommand)
IGRAPH command 919
X1MULTIPLIER (keyword)
IGRAPH command 930
X1START (keyword)
IGRAPH command 930
X2 (subcommand)
IGRAPH command 918
X2INTERVAL (keyword)
IGRAPH command 930
X2LENGTH (subcommand)
IGRAPH command 919
X2MULTIPLIER (keyword)
IGRAPH command 930
X2START (keyword)
IGRAPH command 930
XBETA (keyword)
COXREG command 375

XBETA (keyword) *(continued)*
CSCOXREG command 407
XBPRED (keyword)
GENLIN command 746
XBSTDERROR (keyword)
GENLIN command 746
XDATE.DATE (function) 74
XDATE.HOUR (function) 74
XDATE.JDAY (function) 74
XDATE.MDAY (function) 74
XDATE.MINUTE (function) 74
XDATE.MONTH (function) 74
XDATE.QUARTER (function) 74
XDATE.SECOND (function) 74
XDATE.TDAY (function) 74
XDATE.TIME (function) 74
XDATE.WEEK (function) 74
XDATE.WKDAY (function) 74
XDATE.YEAR (function) 74
XGRAPH (command)
BIN subcommand 2171
CHART subcommand 2168
COORDINATE subcommand 2173
DISPLAY subcommand 2172
DISTRIBUTION subcommand 2173
ERRORBAR subcommand 2173
MISSING subcommand 2174
PANEL subcommand 2174
release history 2167
syntax chart 2167
TEMPLATE subcommand 2175
TITLES subcommand 2176
XLS (keyword)
GET DATA command 786
SAVE TRANSLATE command 1753
XLS (subcommand)
OUTPUT EXPORT command 1367
XLSM (keyword)
GET DATA command 786
XLSX (keyword)
GET DATA command 786
XML
saving output as XML 1308, 1323
XML export
CSGLM command 426
CSLOGISTIC command 438
XPROD (keyword)
CORRELATIONS command 351
REGRESSION command 1643
XR (subcommand)
data organization 1874
SPCHART command 1872
variable specification 1875
XS (subcommand)
data organization 1874
SPCHART command 1872
variable specification 1875
XSAVE (command)
compared with SAVE command 1729, 2181
COMPRESSED subcommand 2184
DROP subcommand 2183
KEEP subcommand 2183
limitations 2181
MAP subcommand 2184
XSAVE (command) *(continued)*
OUTFILE subcommand 2183
PERMISSIONS subcommand 2185
RENAME subcommand 2184
syntax chart 2181
UNCOMPRESSED subcommand 2184
with DO REPEAT command 2181
with TEMPORARY command 1972
ZCOMPRESSED subcommand 2184
XTX (keyword)
REGRESSION command 1640
XVERSION (subcommand)
SET command 1800
SHOW command 1810
XYZ (keyword)
GRAPH command 882

Y

Y (keyword)
CLUSTER command 314
PROXIMITIES command 1559
Y (subcommand)
IGRAPH command 918
Yates' correction for continuity
CROSSTABS command 390
YEAR (keyword)
DATE command 547
YES (keyword)
AIM command 132
CASESTOVARS command 267
CSGLM command 420
CSLOGISTIC command 430, 431
GENLIN command 728, 737
MLP command 1156
SET command 1788
YLENGTH (subcommand)
IGRAPH command 919
YMDHMS format 53, 54
YRMODA (function) 74
Yule's Q
CLUSTER command 314
PROXIMITIES command 1559
Yule's Y
CLUSTER command 314
PROXIMITIES command 1559

Z

Z (keyword)
PROXIMITIES command 1556
z scores
DESCRIPTIVES command 574
PROXIMITIES command 1556
saving as variables 574
ZCOMPRESSED (subcommand)
SAVE command 1733
XSAVE command 2184
ZCOMPRESSION (subcommand)
SET command 1805
SHOW command 1810
ZCORR (keyword)
MANOVA command 1049

ZPP (keyword)
REGRESSION command 1640
ZPRED (keyword)
REGRESSION command 1634
ZRESID (keyword)
GLM command 860
LOGISTIC REGRESSION command 997
REGRESSION command 1634
UNIANOVA command 2086
ZSCORE (keyword)
ADP command 118

